WEBSITE BOOK EDITION

Pixels → Perception

From Pixels to Understanding

Foundations • Vision • Language • Learning • Theory

English Edition

2026-10-17

https://blog.shyandram.dev/

About this edition

This book contains 75 published tutorials and 2 blog posts from Pixels → Perception. The tutorials follow the website's eight learning tracks; the blog essays appear at the end. This is a typeset edition of the website, not a separately peer-reviewed publication.

The prose, code, equations, and references retain the website's content. Exercise hints are expanded for print. Static figures remain in the book, image comparisons show their input and output images, and interactive demonstrations link to the website. Reference numbers are local to each tutorial.

The website is still under review. AI tools assisted its production, and errors or omissions may remain. Read it alongside the original sources and consult the website for subsequent corrections.

English and Traditional Chinese are provided as separate editions. Links to included tutorials navigate within this PDF where possible. Links to specific web sections, interactive demonstrations, notebooks, and external sources remain online. Content snapshot: 2026-10-17

Source revision: `05b94ec`

https://blog.shyandram.dev/

Contents

I Foundations 1

Start Here ·Learning Paths and Recommended Textbooks 2

Images as Arrays 13

II Image Processing 26

IP 01 ·Introduction 27

IP 02 ·Digital Image Fundamentals 46

IP 03 ·Intensity Transformations and Spatial Filtering 69

IP 04 ·Filtering in the Frequency Domain 94

IP 05 ·Image Restoration and Reconstruction 121

IP 06 ·Color Image Processing 148

IP 07 ·Wavelet and Other Image Transforms 172

IP 08 ·Image Compression and Watermarking 195

IP 09 ·Morphological Image Processing 220

IP 10 ·Image Segmentation I: Edge Detection, Thresholding, and Region Detection 246

IP 11 ·Image Segmentation II: Active Contours —Snakes and Level Sets 268

IP 12 ·Feature Extraction 288

IP 13 ·Image Pattern Classification 313

III Natural Language Processing 336

NLP 01 ·Introduction to Natural Language Processing 337

NLP 02 ·Linguistic Essentials 351

NLP 03 ·Neural Networks for NLP 378

NLP 04 ·Lexical Semantics and Word Embeddings 404

NLP 05 ·Recurrent Neural Networks for Language 433

NLP 06 ·Sequence Labeling with RNNs (POS Tagging, Information Extraction) 458

NLP 07 ·Attention Mechanism and Transformers 486

NLP 08 ·Sequence-to-Sequence Models with Transformers (Machine Translation, Chatbots) 504

NLP 09 ·Language Modeling with Transformers (Natural Language Generation) 530

NLP 10 ·Pretrained Models: BERT 569

NLP 11 ·Classification with BERT (Sentiment Analysis, Natural Language Inference) 588

NLP 12 ·Syntactic and Semantic Tasks 611

IV Pattern Recognition 641

PR 01 ·Patterns, Probability and Bayes Decisions 642

PR 02 ·Density Estimation and Bayesian Learning 661

PR 03 ·Linear Models for Regression and Classification 680

PR 04 ·Neighbors, Kernels and Nonlinear Classifiers 699

PR 05 ·Feature Selection, Transforms and Dimensionality 720

PR 06 ·Template Matching and Structured Recognition 741

PR 07 ·Clustering, Mixtures and EM 760

PR 08 ·Approximate Inference and Bayesian Model Complexity 785

PR 09 ·Ensembles, Validation and Cluster Validity 808

PR 10 ·Recognition Capstone: Rejection, Shift and Learned Features 832

V Machine Learning 857

ML 01 ·Introduction to Machine Learning 858

ML 01A ·Linear Algebra for Learning 887

ML 01B ·Probability and Information Theory 909

ML 01C ·Numerical Computation and Gradient Checks 933

ML 02 ·Neural Networks from Perceptron to Deep Networks 958

ML 03 ·Convolution, Image Filtering and CNNs 985

ML 03A ·Recurrent and Recursive Networks 1009

ML 04 ·Attention and Vision Transformers 1032

ML 05 ·Representation Learning and Self-Supervision 1053

ML 05A ·Linear Factor Models and Autoencoders 1076

ML 06 ·Generative Models: Autoencoders, VAEs, GANs and Flows 1100

ML 07 ·Diffusion Models and Flow Matching 1124

ML 08 ·Autoregressive Vision: Pixels, Tokens and Scales 1158

ML 09 ·Foundation Models and Vision–Language Learning 1181

ML 10 ·Evaluation, Efficiency and Deployment 1203

VI Deep Learning for Computer Vision 1231

DLCV 01 · Deep Learning for Image Processing and Computer Vision 1232

DLCV 02 · Deep Image Classification 1257

DLCV 03 · Object Detection: From R-CNN to DETR 1297

DLCV 04 · Deep Image Restoration: Low-Level Vision 1326

DLCV 05 · Low-Light Image Enhancement 1374

DLCV 06 · Object Tracking: From Single-Object to Multi-Object 1410

DLCV 07 · World Models: Video, Latent Dynamics and Planning 1439

VII Optimization 1461

OPT 01 · Modeling Decisions and Convex Sets 1462

OPT 02 · Convex Functions, Composition and Conjugates 1476

OPT 03 · Linear, Quadratic, Conic and Geometric Programs 1492

OPT 04 · Duality, KKT Conditions and Certificates 1507

OPT 05 · Approximation, Statistical Estimation and Experiment Design 1522

OPT 06 · Gradient Descent, Newton Steps and Numerical Reliability 1538

OPT 07 · Equality Constraints and Interior-Point Methods 1554

OPT 08 · Geometric Applications, Splitting and Modern Methods 1570

VIII Information Theory 1586

IT 01 · Entropy, Relative Entropy and Mutual Information 1587

IT 02 · Typical Sequences, AEP and Entropy Rates 1603

IT 03 ·Lossless Coding and Universal Compression 1618

IT 04 ·Channel Capacity and Reliable Communication 1633

IT 05 ·Differential Entropy and Gaussian Channels 1647

IT 06 ·Rate–Distortion and Lossy Representations 1663

IT 07 ·Statistical Evidence, Maximum Entropy and Growth 1679

IT 08 ·Networks, Information Inequalities and Learning 1695

IX Blog Essays 1710

Introducing Pixels → Perception 1711

Welcome to Pixels → Perception 1713

Part I

Foundations

Start Here • Learning Paths and Recommended Textbooks

How the eight tracks fit together, which route to follow for your background and goal, how to run the code and notebooks, and which textbooks to pair with each track.

Online edition: https://blog.shyandram.dev/en/learn/foundations/getting-started/

What you'll learn

- How the site's eight tracks relate to each other and what each one is for.
- How to read a tutorial page: level, prerequisites, the track rail, notebooks and the playground.
- Which learning path fits your background (new to programming, rusty mathematics, already training networks) and your goal (vision, theory, generative models, language, world models).
- How to set up a small Python environment and run the code blocks and notebooks on a CPU.
- Which main textbook to pair with each track, and how to read textbooks, surveys and papers differently.
- How to turn reading into small, checkable milestones.

The big picture

You do not need to finish every mathematical topic before touching an image. You do need to know what each operation assumes. The fastest way to learn is to start from a question you can test: "Why does blur remove an edge?", "Does a classifier still work on images from a new camera?", or "What changes when a model predicts the next token instead of a class?"

> **Explain like I'm five**
>
> Learning this subject is like learning to cook. A textbook explains the ingredients and techniques; a small experiment lets you taste what happens. Pick one dish, change one ingredient at a time, and write down what you notice before you try to cook a whole banquet. This site is the cookbook with the recipes arranged in a sensible order, and every recipe can be cooked in a small kitchen (a laptop CPU).

The site is organized into **eight tracks**. A track is an ordered sequence of tutorials on one theme. The numbering describes the role of each track, not a rule that you must complete one before entering another.

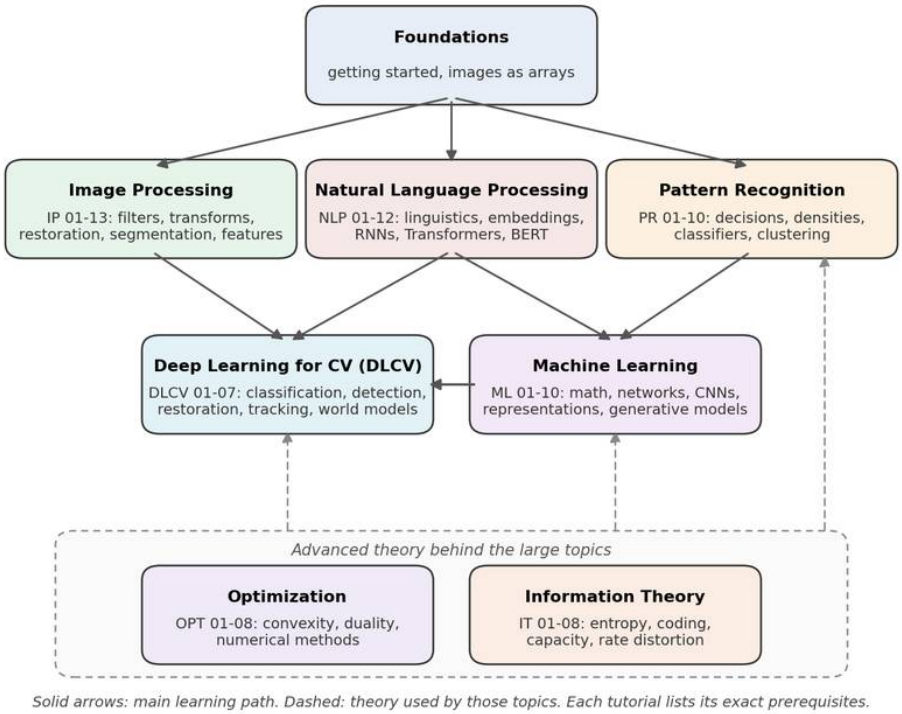


Figure 1 — How the eight tracks fit together. Start with Foundations, then pick an entry track (IP, NLP or PR); Machine Learning and Deep Learning for Computer Vision (DLCV) build on them; Optimization and Information Theory are the advanced theory behind those large topics. Arrows are suggestions, not locks: each tutorial lists its own prerequisites.

Track	Central question	Contents	A useful first outcome
Foundations ·start	What are the data?	this page and Images as Arrays	inspect an image's shape, dtype and coordinate convention
Image Processing (IP) · entry	How can we transform or measure an image?	IP 01–13, from fundamentals to pattern classification	explain a filter and its boundary behavior
NLP ·entry	How are text and language modeled?	NLP 01–12, from the introduction and linguistic essentials through embeddings, RNNs, Transformers and BERT to syntactic and semantic tasks	build and evaluate a small language application
Pattern Recognition (PR) · entry	How can features support decisions under uncertainty?	PR 01–10, from Bayes decisions to a recognition capstone	compare classifiers with a valid train/test split
Machine Learning (ML) · core	How are representations, predictors and generators learned?	ML 01–10 plus the math chapters ML 01A–01C and ML 03A, 05A	train and evaluate a small model

Track	Central question	Contents	A useful first outcome
Deep Learning for Computer Vision (DLCV) · core	How do learned models solve visual tasks?	DLCV 01–07: classification, detection, restoration, low-light enhancement, tracking, world models	understand a classification or detection pipeline end to end
Optimization (OPT) · theory	How can we model and solve constrained decisions?	OPT 01–08, from convex sets to solvers and applications	formulate a convex problem and check its optimality conditions
Information Theory (IT) · theory	What are the limits of representing and communicating uncertainty?	IT 01–08, from entropy to coding, channels, rate distortion and networks	calculate entropy and explain a compression or communication limit

How they connect, in one paragraph: **Image Processing** treats an image as a signal and designs operations by hand. **Pattern Recognition** adds statistics and decision theory: given features, how should we decide, and how sure can we be? **ML/DL** replaces hand-designed features with learned ones and supplies the shared machinery (losses, optimization, architectures, generative models) used everywhere else. **DL for Vision** applies that machinery to concrete visual tasks, and its restoration and enhancement chapters lean on the image-formation ideas from Image Processing. **NLP** reuses the same probability, optimization and Transformer ideas with text as input. **Optimization** develops the modeling rules, constraints, duality and numerical methods behind estimation and training. **Information Theory** explains uncertainty, coding and communication limits; its entropy and divergence concepts connect classification, language modeling and compression.

How a tutorial page works

Every tutorial follows the same layout, so once you know one page you know them all.

- **Level pill.** *Beginner*, *intermediate* or *advanced*. The ML math chapters and the first IP tutorials are beginner; the DLCV tutorials assume you already know CNNs and backpropagation.
- **Needs.** Under the title, a *Needs* line links the tutorials that the page assumes. Prerequisites are language independent, so the Chinese and English pages share them. If a *Needs* link is unfamiliar, read that page first, or at least its *Key takeaways*.
- **Track rail.** The left column lists every tutorial in the current track in order; *Previous* and *Next* links at the bottom follow the same order.
- **Sections.** Each tutorial opens with *What you'll learn* and an *Explain like I'm five* box, then develops each topic in two passes: a plain-language statement first, then the precise version with equations and code. It ends with *Key takeaways*, *Exercises* (hints are folded; try before you open them) and numbered *References*, cited in the text as [1], [2] and so on.
- **Notebook buttons.** A tutorial that has a companion notebook shows an **Open in Colab** pill next to its title and a **View notebook** link at the end. Currently ML 03 has one (convolution and image filtering).
- **Interactive labs.** Many pages embed a live experiment that runs a real algorithm on a real image or dataset in your browser: filters, Fourier transforms, thresholds, nearest neighbors and more.

Each lab asks you to predict the outcome first, then lets you check it. All of them also live on the Playground page.

- **Search and language.** The search page finds tutorials and blog posts in the language you are reading; the language switch in the header jumps to the same page in the other language. If a page has not been translated, the Chinese site shows the English version with a notice.

Choose your entry route

Pick the description that fits you best. The routes overlap on purpose.

If programming is new. Learn Python functions, loops, list and array indexing, and basic plotting first. Then read Images as Arrays and IP 01. Make a synthetic image, inspect its rows and columns, and explain why adding two `uint8` images can overflow. Continue with IP 02 and IP 03, which are written so that every equation has a matching NumPy line.

If you can code but mathematics feels rusty. Read ML 01 for the vocabulary of learning, then the three math chapters: linear algebra, probability and information theory and numerical computation. They follow the "applied math" part of Goodfellow, Bengio and Courville [7] and pair each idea with a short NumPy check. Aim to be able to check matrix shapes, interpret a conditional probability and spot a numerically unstable calculation.

If you know introductory probability and linear algebra. Start PR 01 for statistical decisions and classical methods, or ML 02 for learned representations. Return to the math chapters only when a derivation reveals a specific gap.

If you already train neural networks. Skim ML 03 and ML 04, then pick an application chapter in DL for Vision, and pair it with ML 10 on evaluation and deployment.

Suggested learning paths by goal

Once you have an entry point, choose a goal. Each path is a reading order, typically eight to twelve tutorials. Read each page's *Needs* line to fill gaps as they appear.

Goal	Suggested order
Classical image processing	Images as Arrays → IP 01–05 (fundamentals, filtering, frequency domain, restoration) → IP 06–09 (color, wavelets, compression, morphology) → IP 10–13 (segmentation, active contours, features, classification)
Statistical pattern recognition	ML 01A, ML 01B → PR 01 → PR 02 (density estimation) → PR 03 (linear models) → PR 04 (kernels) → PR 05 (features) → PR 07 (clustering, EM) → PR 08 (approximate inference) → PR 09 (ensembles, validation) → PR 10
Deep learning for vision	ML 01 → ML 01C → ML 02 → ML 03 → ML 04 → DLCV 01 → DLCV 02 → DLCV 03 → DLCV 04–06 (restoration, low-light, tracking) → ML 10

Goal	Suggested order
Generative models	ML 01B → ML 02 → ML 05 → ML 05A → ML 06 → ML 07 → ML 08 → ML 09
Language	NLP 01 → NLP 02 → NLP 03–04 (neural networks, word embeddings) → NLP 05–06 (RNNs, sequence labeling) → NLP 07–09 (attention and Transformers, seq2seq, language modeling) → NLP 10–11 (BERT, classification) → NLP 12; add ML 02, ML 03A and ML 04 for the network math
Convex modeling and solvers	ML 01A, ML 01C → OPT 01 → OPT 02–04 (functions, programs, duality) → OPT 05–07 (estimation, numerical methods, constrained solvers) → OPT 08
Information, compression and communication	ML 01B → IT 01 → IT 02–03 (typicality, source coding) → IT 04–06 (capacity, Gaussian channels, rate distortion) → IT 07–08 (statistical decisions, networks)
World models	ML 03A → ML 05 → ML 07 → ML 08 → DLCV 07

A realistic pace is one tutorial per week if you run the code and do two exercises; a long tutorial can take two weeks. Speed matters less than being able to explain each page's key takeaways in your own words.

Run the code

The website and the Python examples use separate environments. You can read the site without installing anything. To run the code you need Python; to preview the website itself you need Node.

A Python environment for the tutorials

Create one dedicated virtual environment for all tutorials. Python 3.11 or newer is a practical starting point; check each package's requirements if you use a different version.

```
python3 -m venv .venv-study
source .venv-study/bin/activate
python -m pip install --upgrade pip
python -m pip install numpy scipy matplotlib scikit-learn scikit-image opencv-python
 ↪  PyWavelets jupyterlab
```

On Windows, activate with `.venv-study\Scripts\Activate.ps1` in PowerShell. The code blocks across the site import exactly these libraries (NumPy, SciPy, scikit-learn, scikit-image, OpenCV, PyWavelets) plus PyTorch for the neural-network chapters. Install PyTorch with the command produced by its official installation selector [11]; choose the CPU build if you have no GPU. Every example on the site is sized to run on a laptop CPU in seconds to about a minute.

Running a code block

Code blocks are **self-contained**: each one imports what it needs and builds its own data, usually synthetic arrays, the sample images in `skimage.data`, or the toy datasets in scikit-learn. Copy a block into a file and run it, or paste it into a notebook cell. Most blocks end with `assert` statements that check the claim made in the text; if an assertion fails, you have found either a version difference or a misunderstanding worth investigating.

A good habit is to change one thing and re-run: a kernel size, a learning rate, a random seed. The exercises suggest which knob to turn.

Notebooks and Colab

A tutorial with a companion notebook has an **Open in Colab** button. Colab runs the notebook in your browser on a hosted machine, with NumPy, OpenCV, scikit-image and Matplotlib already installed, so nothing needs to be set up locally. To run the same notebook locally, download it from the **View notebook** link, then start `jupyter lab` inside the activated environment and open the file. The notebooks live in the repository's `notebooks/` folder.

Previewing the website (optional)

Only needed if you want to change the site. The repository requires Node 22.12 or newer [12]:

```
npm install
npm run dev
```

The development server prints a local address. Tutorials are MDX files under `src/content/tutorials/<lang>/<track>/`.

Your first CPU experiment

This complete example creates its own image, adds noise and smooths it. No download is needed.

```
import numpy as np
import cv2

rng = np.random.default_rng(0)
clean = np.zeros((32, 32), dtype=np.float32)
clean[8:24, 8:24] = 1.0
noisy = clean + rng.normal(0, 0.1, clean.shape).astype(np.float32)
smooth = cv2.GaussianBlur(noisy, (5, 5), 1.0,
                          borderType=cv2.BORDER_REFLECT_101)
print("Shape and type:", smooth.shape, smooth.dtype)
```

```
print("Noise-region standard deviation:",
      noisy[:5, :5].std(), smooth[:5, :5].std())
print("Across the edge:", np.round(smooth[16, 5:12], 3))
assert smooth.shape == clean.shape
assert np.isfinite(smooth).all()
```

The output shows two effects. The standard deviation in the flat, all-background corner drops from about 0.074 to about 0.026: smoothing averages away noise. Across the edge at column 8, the values now ramp from about 0.04 to 0.93 over three or four pixels instead of jumping from 0 to 1: smoothing also spreads a sharp transition. A lower noise estimate therefore does not mean every important detail improved. IP 03 explains why, and Exercise 2 asks you to measure the trade-off.

Choose one main textbook

Use one main book for continuity and a second one for a different explanation of the same idea. The links point to author or publisher pages, with access conditions as described there. This is a curated starting set, not a ranking.

Need	Recommended book	How to use it
Mathematical preparation	Deisenroth, Faisal and Ong, *Mathematics for Machine Learning* [1]	start with linear algebra, matrix decompositions, vector calculus and probability; apply them through regression and PCA
Image operations	Gonzalez and Woods, *Digital Image Processing*, 4th ed. [2]	pair image fundamentals, spatial and frequency filtering, restoration and segmentation with the DIP track
Pattern Recognition, main text	Theodoridis and Koutroumbas, *Pattern Recognition*, 4th ed. [3]	follow classifier design, features and clustering; work the problems rather than only reading summaries
Pattern Recognition, companion	Bishop, *Pattern Recognition and Machine Learning* [4]	revisit the same topics through probability, latent variables and graphical models
A first modern DL text	Prince, *Understanding Deep Learning* [5]	connect diagrams, mathematics and the accompanying notebooks for networks and generative modeling
Another modern DL explanation	C. M. Bishop and H. Bishop, *Deep Learning: Foundations and Concepts* [6]	use its probabilistic presentation to strengthen the concepts behind architectures and training
DL foundations and reference	Goodfellow, Bengio and Courville, *Deep Learning* [7]	Part I (applied math) matches ML 01A–01C; later chapters cover optimization, CNNs, sequence models and representation learning
Computer vision beyond classification	Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed. [8]	connect image formation, motion, geometry, recognition and reconstruction
Probabilistic machine learning	Murphy, *Probabilistic Machine Learning: An Introduction* [9]	a modern, probability-first reference for the ML and PR tracks

Need	Recommended book	How to use it
NLP applications, main text	Hagiwara, *Real-World Natural Language Processing* [13]	follow the application workflow, text classification, sequence models, attention, pretrained models, training practices and deployment
NLP linguistic foundations, companion	McRoy, *Principles of Natural Language Processing* [14]	study syntax, grammars, semantics, discourse, retrieval, generation and translation alongside the corresponding NLP chapters
Language and text, supplementary	Jurafsky and Martin, *Speech and Language Processing*, 3rd-edition draft [10]	deepen selected topics in tokens, n-grams, classification, embeddings, Transformers and evaluation
Convex optimization	Boyd and Vandenberghe, *Convex Optimization* [15]	OPT 01–05 follow modeling and applications in Chapters 1–8; OPT 06–08 develop numerical methods in Chapters 9–11 and modern extensions
Information theory	Cover and Thomas, *Elements of Information Theory*, 2nd ed. [16]	follow entropy and typicality through source/channel coding, continuous variables, rate distortion, statistical decisions, universal coding and networks

For the Pattern Recognition track, **Theodoridis and Koutroumbas [3] and Bishop's PRML [4] are the two core references**. Read Theodoridis for a systematic recognition pipeline, then compare Bishop's probabilistic treatment of the same problem. Their publication dates do not weaken Bayes decision theory or Gaussian models; turn to newer papers when the architecture, data scale or application has changed.

If you choose just one deep learning introduction, start with Prince [5] and use this site's math chapters as needed. Choose Bishop and Bishop [6] if you prefer a sustained probabilistic viewpoint. Goodfellow et al. [7] remains a valuable foundation, but its 2016 edition predates Transformers in vision, diffusion models and foundation models; the ML track covers those.

The NLP track uses **Hagiwara [13] and McRoy [14] as its two core books**. Hagiwara connects models to applications; McRoy supplies the linguistic and symbolic concepts needed to understand what a language system is trying to represent. Read both around the same problem: a classifier needs a data workflow, but a parser or dialogue system also needs an explicit account of linguistic structure. Newer papers supplement their 2021 editions.

For Optimization, use **Boyd and Vandenberghe [15]** to learn which problem transformations preserve convexity and what an optimality certificate means. For Information Theory, use **Cover and Thomas [16]** to distinguish an achievable construction from a bound that no construction can beat. These tracks extend the introductory mathematics in ML 01B–01C; their book chapter maps appear within each tutorial.

The Jurafsky and Martin draft changes over time; its current release is dated August 19, 2026 [10]. Record the release date when you make chapter notes, because chapter numbering can move between versions. Likewise, record textbook edition numbers so that exercise references stay reproducible.

Read textbooks, surveys and papers differently

A **textbook** gives definitions and a coherent sequence. Re-derive one equation, run a tiny example and solve one exercise before moving on.

A **survey or review** gives a map of alternatives, assumptions and open questions. Each tutorial's *Modern view* section summarizes several. When you read one, make a table with the prediction target, learning objective, required data, compute and evaluation of each method it covers. Check its coverage date: "recent" means recent to its authors.

An **original paper** supplies one method and its evidence. Read the problem, the method, the experimental protocol and the limitations. Trace an important claim to the actual table or ablation that supports it. Knowing what changed relative to a fair baseline is more useful than knowing the method's name.

Keep a reading note with five entries: the question, the mathematical objects, the main assumption, the experiment that tests it, and one remaining doubt. The reference lists in each tutorial suggest focused next readings; you do not need to read every cited paper before running the code.

A sequence of small milestones

1. **Represent data:** inspect an image and explain its axes, range and dtype.
2. **Transform data:** implement one filter, compare it with a library function and account for the boundaries.
3. **Make a decision:** fit a simple classifier with preprocessing learned only from the training data.
4. **Learn features:** train a small neural network and justify its loss and validation choice.
5. **Study a modern method:** reproduce a mechanism such as attention, masking or noising on synthetic data.
6. **Finish a small project:** report a baseline, independent test results, failure cases and limitations.

Move on when you can explain the previous milestone without copying its wording. A modest experiment with a clean comparison teaches more than a large model whose data and evaluation you cannot inspect.

Key takeaways

- Eight tracks: Foundations, Image Processing, NLP, Pattern Recognition, ML/DL, DL for Vision, Optimization and Information Theory. They feed each other; none has to be finished before another is started.
- Each page's *Needs* line, level pill and track rail tell you where it sits; use them instead of reading everything in order.
- Choose an entry route by background and a learning path by goal; one tutorial per week with code and exercises is a sustainable pace.

- All code is self-contained and CPU sized; one Python environment with NumPy, SciPy, scikit-learn, scikit-image, OpenCV, PyWavelets and PyTorch covers the whole site.
- Pair each track with one main textbook and one companion, and read textbooks, surveys and papers for different purposes.

Exercises

1. **Plan.** Pick an entry route, a learning path and a main textbook. Write the question your first experiment will answer and a concrete result that would count as progress.

   Hint

   Keep the question small enough to test on a CPU in one sitting, for example "How much does a 5×5 Gaussian blur reduce noise in flat regions, and how much does it widen an edge?" Progress is a number you can measure, not a feeling.

2. **Kernel sweep.** Change the smoothing example to kernel sizes 3, 5 and 9 (raise σ with the kernel, for example 0.8, 1.0 and 2.0). Report the flat-region standard deviation and the number of pixels the edge takes to rise from 0.1 to 0.9.

   Hint

   Keep the random seed and the noisy image fixed so only the filter changes. Noise should fall and the edge width should grow together: that is the bias–variance trade-off of linear smoothing.

3. **Read a page's metadata.** Open ML 02. What is its level, what does its *Needs* line point to, and which tutorial comes next in the track rail?

   Hint

   The *Needs* line is right under the title. Follow it back until you reach a page whose prerequisites you already know; that chain is your personal reading order.

4. **Classify a source.** Select one reference from any tutorial and classify it as textbook, survey, original paper or software documentation. What question is that source best suited to answer?

   Hint

   A survey answers "what are the options and how do they compare?"; an original paper answers "does this specific change help, and under what protocol?"; documentation answers "what does this function actually compute?"

5. **Split by the right unit.** Sketch a training/validation/test split for a dataset containing many frames from each of 50 videos. Explain why splitting individual frames at random can be misleading.

Hint

Neighboring frames are nearly identical, so a random frame split puts near-duplicates in both training and test sets and inflates the test score. Split by video (or by recording session) instead.

References

1. M. P. Deisenroth, A. A. Faisal and C. S. Ong, *Mathematics for Machine Learning*, Cambridge University Press, 2020. Book site
2. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018. Book site
3. S. Theodoridis and K. Koutroumbas, *Pattern Recognition*, 4th ed., Academic Press, 2009. Companion site
4. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006 (official PDF available). Book page
5. S. J. D. Prince, *Understanding Deep Learning*, MIT Press, 2023. Book and notebooks
6. C. M. Bishop and H. Bishop, *Deep Learning: Foundations and Concepts*, Springer, 2024 (free online version). Book site
7. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016. Book site
8. R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed., Springer, 2022 (PDF for personal use). Book site
9. K. P. Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022. Book site
10. D. Jurafsky and J. H. Martin, *Speech and Language Processing*, 3rd ed. draft, release of August 19, 2026. Draft site
11. PyTorch, "Get Started Locally," installation documentation. Documentation
12. Node.js, "Download Node.js." Download page
13. M. Hagiwara, *Real-World Natural Language Processing*, Manning, 2021. Publisher
14. S. McRoy, *Principles of Natural Language Processing*, open textbook, 2021. Open textbook
15. S. Boyd and L. Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004. Author site and book
16. T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006. Publisher

Links and draft dates were checked on October 4, 2026. A companion website does not imply that the complete textbook is free; access conditions are those stated on each site.

Images as Arrays

What a digital image really is, how NumPy and OpenCV store it, and the indexing, dtype and channel-order habits that prevent the most common bugs.

Online edition: https://blog.shyandram.dev/en/learn/foundations/images-as-arrays/

What you'll learn

- How a grayscale or color image becomes a NumPy array, and what `shape`, `dtype` and `strides` tell you about it.
- Why an image is a *sampled and quantized* version of a continuous scene, and what is lost when you keep fewer pixels or fewer gray levels.
- The coordinate convention (row first, then column), and how slicing, flipping, masking and flattening work, including the difference between a view and a copy.
- The three pitfalls that bite everyone: BGR versus RGB channel order, `uint8` wraparound, and writing through a view by accident.
- How the same array view of an image carries over to deep-learning tensors, image patches and neural image representations.

The big picture

Every method in this site, from a 3×3 blur to a vision Transformer, starts from the same object: a grid of numbers. If you can read, slice and transform that grid with confidence, the rest of the curriculum is a matter of choosing which numbers to combine and how.

> **Explain like I'm five**
>
> Picture a mosaic made of tiny colored tiles. Stand far away and you see a face; stand close and you see only squares, each with one color. A digital photo is the same: a rectangle of tiles, and every tile is described by a number (or three numbers, one each for red, green and blue). To find a tile you say "which row, which column", the way you find a seat in a cinema. Computers store the mosaic as a table of numbers, and everything you do to a picture, such as brightening it, blurring it or finding faces in it, is arithmetic on that table.

Why it matters: almost every beginner bug in image code is really a bug about the array, not about the algorithm. Colors look blue because the channels were swapped. Bright areas turn black because 8-bit values wrapped around. An edit changes the original because a crop was a view, not a copy. This page names those traps once, with code you can run, so you can recognize them later.

An image is a sampled, quantized signal

Plain version. The real world has no pixels. A camera measures how much light reaches each tiny spot of its sensor, and writes the measurement down as a whole number. Two decisions turn a continuous scene into a digital image: *where* to measure (how many spots) and *how finely* to write each measurement (how many possible values).

Precise version. Let a monochrome scene be a continuous brightness function. **Sampling** evaluates it on a regular grid of H rows and W columns. **Quantization** rounds each sample to one of $L = 2^b$ integer levels, where b is the number of bits per pixel. The result is a function on a finite grid,

$$I : \{0, \dots, H-1\} \times \{0, \dots, W-1\} \to \{0, \dots, L-1\}, \qquad (r, c) \mapsto I[r, c],$$

where r is the **row** index (counted from the top), c is the **column** index (counted from the left), $I[r, c]$ is the **pixel value**, and $L = 256$ for the common 8-bit case, so values run from 0 (black) to 255 (white). A color image adds a channel index k and becomes $I[r, c, k]$ with $k \in \{0, 1, 2\}$ for the three color channels. This is the standard treatment of image formation and digitization in textbooks such as Gonzalez and Woods [1, Ch. 2] and Torralba, Isola and Freeman [2].

Written as an array, the grid has a **shape** `(H, W)` for grayscale or `(H, W, 3)` for color. The number of values stored is $H \times W \times C$ for C channels, and the memory needed is that count times the bytes per value.

> **Coordinates: row first**
>
> Arrays are indexed `[row, column]`, which is `[y, x]` in the picture's own axes: the first index runs down the page, the second runs across. Graphics functions often take points as `(x, y)`, which is `(column, row)`. This series' image-processing chapters follow the textbook convention and write $f(x, y)$ with x as the row index, so $f(x, y)$ matches NumPy's `f[x, y]`. Whenever code and math disagree, check which axis is which before debugging anything else.

Shape, dtype and memory

Plain version. An array knows three things about itself: how many numbers it holds along each axis (`shape`), what kind of number each is (`dtype`), and how those numbers are laid out in memory (`strides`).

scikit-image ships a few small test photographs [6], [7], which keeps every example here runnable without downloading anything. The code below loads one grayscale and one color image and prints their basic properties.

```
import numpy as np
import Lab from '../../../../components/Lab.astro';
from skimage import data

gray = data.camera()          # grayscale test photo bundled with scikit-image
```

```
rgb = data.astronaut()        # color test photo, also bundled

for name, a in [("gray", gray), ("rgb", rgb)]:
    print(name, a.shape, a.dtype, a.ndim, a.size, a.nbytes)
print(gray.min(), gray.max())
print(gray.strides, rgb.strides)
```

Output:

```
gray (512, 512) uint8 2 262144 262144
rgb (512, 512, 3) uint8 3 786432 786432
0 255
(512, 1) (1536, 3, 1)
```

Reading the output: the grayscale image has 512 rows and 512 columns, so `size` is 512 × 512 = 262,144 values, and since each `uint8` takes one byte, `nbytes` is the same number. The color image has three times as many values. The pixel range is the full 0 to 255.

`strides` is the number of bytes to jump to move one step along each axis. For `gray`, moving one column costs 1 byte and moving one row costs 512 bytes, which means NumPy stores the image **row by row** (C order). In `rgb`, the three channel values of one pixel sit next to each other in memory. This explains why scanning along a row is fast and why reshaping an image into a long vector keeps rows together, as the next section shows. The NumPy paper [3] and documentation [4] describe the array model in full.

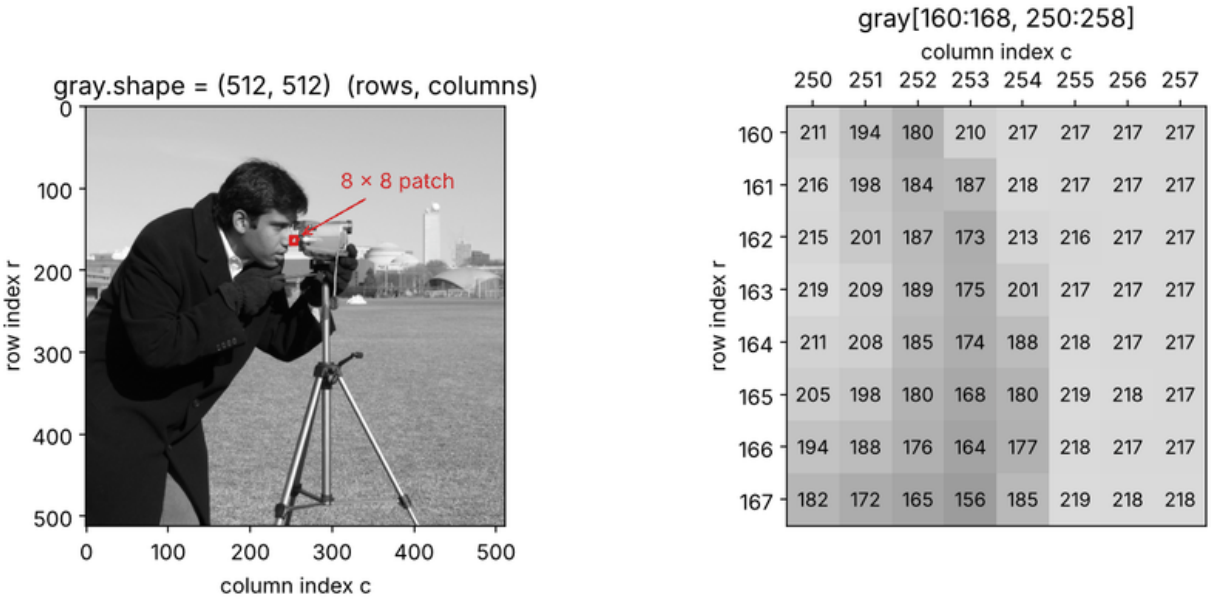


Figure 1 — An image is a table of integers. Left: the 512 × 512 test photo with an 8 × 8 patch outlined. Right: the same patch as the numbers the computer stores, with its row and column indices. Matplotlib places row 0 at the top by default [9].

Indexing, slicing and views

Plain version. Because an image is just an array, you pick out pixels, rectangles and whole rows with the same square-bracket syntax you use for any NumPy array. The surprise is that a rectangle taken this way is a *window onto the same memory*, not a separate copy.

Precise version. For an array `a`, the expression `a[r, c]` returns one pixel, `a[r0:r1, c0:c1]` returns the sub-block of rows $r_0, \dots, r_1 - 1$ and columns $c_0, \dots, c_1 - 1$ (the end index is excluded), and a step such as `a[::2, ::2]` keeps every second row and column. All of these are **views**: new array objects that share the original's memory. Changing a view changes the original. Boolean masks and index lists create copies instead. Calling `.copy()` makes a copy explicitly.

```
r, c = 100, 200
print(gray[r, c])             # row first, then column
print(rgb[r, c])              # three numbers: R, G, B
patch = gray[100:108, 200:208]            # rows 100..107, columns 200..207
print(patch.shape, np.shares_memory(patch, gray))

work = gray.copy()
work[100:108, 200:208] = 0                # writing through a slice edits `work`
print(work[100:108, 200:208].max(), gray[100:108, 200:208].max())

H, W = gray.shape
vec = gray.reshape(-1)                    # flatten: row-major order
print(vec.shape, vec[r * W + c] == gray[r, c], np.shares_memory(vec, gray))
print(np.array_equal(gray[::-1], np.flipud(gray)), gray[:, ::-1].shape)
mask = gray > 200
print(mask.dtype, mask.sum())
```

Output:

```
54
[81 57 17]
(8, 8) True
0 109
(262144,) True True
True (512, 512)
bool 55112
```

What each line shows:

- `gray[100, 200]` is 54 and `rgb[100, 200]` is `[81 57 17]`: a color pixel is a short vector, one number per channel.
- The 8×8 patch shares memory with `gray`, so it is a view. Because `work` is a copy of `gray`, zeroing a patch of `work` leaves `gray` untouched (its patch still has maximum 109). If you had written to the view of `gray` itself, the original photo would have changed.

- Flattening gives a vector of $HW = 262144$ numbers in which pixel (r, c) sits at position $rW + c$. This flattened form is the input vector that classical classifiers and the first layer of a fully connected network see (see ML 01a and ML 01).
- Flipping the image upside down is `gray[::-1]`, a view with a negative row step; no loop is needed. The comparison with `np.flipud` is `True`.
- A comparison such as `gray > 200` yields a boolean array of the same shape, here with 55,112 `True` entries. Boolean masks are how you select, count or edit pixels that satisfy a condition.

Color images and channel order

Plain version. A color image stores three numbers per pixel, one for how red, one for how green and one for how blue. The trouble is that two common conventions order these three numbers differently, and nothing in the array tells you which one you have.

Precise version. Most Python imaging code (NumPy arrays from scikit-image [6], Matplotlib, Pillow) uses the order red, green, blue (**RGB**). OpenCV's `imread` returns **BGR**: blue is channel 0 and red is channel 2. Mixing the two does not raise an error; it silently swaps red and blue, so skin looks blue and the sky looks orange. To make the effect visible without a file on disk, the code below writes the color test image to a temporary PNG, reads it back with OpenCV and compares.

```
import pathlib, tempfile
import cv2
from skimage import io

with tempfile.TemporaryDirectory() as d:
    path = pathlib.Path(d) / "astronaut.png"
    io.imsave(path, rgb)                    # a PNG written in R, G, B order
    bgr = cv2.imread(str(path))             # OpenCV hands the channels back as B, G, R
print(rgb[100, 200], bgr[100, 200])
print(np.array_equal(bgr[..., ::-1], rgb))
print(np.array_equal(cv2.cvtColor(bgr, cv2.COLOR_BGR2RGB), rgb))

weights = np.array([0.299, 0.587, 0.114])
manual = rgb @ weights                      # weighted sum of R, G, B -> float64 (H, W)
cv_gray = cv2.cvtColor(rgb, cv2.COLOR_RGB2GRAY)
print(cv_gray.dtype, np.abs(manual - cv_gray).max())
from skimage.color import rgb2gray
print(np.abs(rgb2gray(rgb) * 255 - cv_gray).max())
```

Output:

```
[81 57 17] [17 57 81]
True
True
uint8 0.5010000000000332
15.529299999999992
```

The same pixel reads `[81 57 17]` as RGB and `[17 57 81]` as BGR. Reversing the last axis, `bgr[..., ::-1]`, is a free way to flip the order, and `cv2.cvtColor` with `COLOR_BGR2RGB` does the same thing explicitly. Both restore the original exactly.

To turn color into gray, the standard approach is a **weighted sum** of the channels, because the eye is more sensitive to green than to blue. The weights $0.299, 0.587, 0.114$ come from the luma definition in the ITU-R BT.601 television standard [8, Sec. 2.5.1], and the check above shows OpenCV's gray conversion agrees with that weighted sum to within 0.501 gray level, which is just rounding to integers. Different libraries use different weights: scikit-image's `rgb2gray` differs from OpenCV's result by up to about 15.5 gray levels on this image. So never assume that "gray" means the same numbers across libraries. Compare against the documentation of the function you call.

Figure 2 — A color image is a stack of three planes. Each channel is itself a grayscale image: the orange suit is bright in the red plane and dark in the blue plane. The last panel is the weighted gray conversion.

Data types and the wraparound trap

Plain version. The most common storage type for images is the 8-bit unsigned integer, which can only hold 0 to 255. If a calculation goes past 255 it does not stop at 255: it silently starts again from 0. A pixel that was almost white can become almost black.

Precise version. For an unsigned integer type `uint8`, arithmetic is performed modulo $2^8 = 256$. So $250 + 10 = 260 \equiv 4$ and $250 - 251 = -1 \equiv 255$. Because values are not saturated, brightening or subtracting images in `uint8` is unsafe. NumPy 2 also refuses to combine an array with a Python integer that does not fit the array's type: it raises an error instead of quietly widening, following NEP 50 [5]. The safe recipe has three steps: convert to a wider type (`int16`, `float32`), compute, then clip back to the valid range and convert back to `uint8`.

```
from skimage.util import img_as_float

a = np.array([250], dtype=np.uint8)
print(a + 10, a - 251, a.astype(np.int16) + 10)
try:
    a + 300
except OverflowError as e:
    print("OverflowError:", e)
```

```
wrapped = gray + np.uint8(60)
clipped = np.clip(gray.astype(np.int16) + 60, 0, 255).astype(np.uint8)
print((gray >= 196).sum(), (wrapped < gray).sum(), clipped.max())

x = gray.astype(np.float32) / 255.0
print(x.dtype, x.min(), x.max(), np.allclose(x, img_as_float(gray)))
y = np.clip(1.2 * x, 0, 1)
out = np.round(y * 255).astype(np.uint8)
print(out.dtype, out.max())
```

Output:

```
[4] [255] [260]
OverflowError: Python integer 300 out of bounds for uint8
72087 72087 255
float32 0.0 1.0 True
uint8 255
```

Reading it: in `uint8`, `250 + 10` gives 4 and `250 - 251` gives 255, while in `int16` the sum is a correct 260. Adding 60 to every pixel wraps exactly the pixels whose value is at least 196 (since $196 + 60 = 256$): there are 72,087 of them, and the same 72,087 pixels come out *darker* than they went in. The clipped version keeps all of them at the maximum 255 instead. The last block shows the usual float workflow: divide by 255 to work in $[0, 1]$, compute, clip, multiply back and round. `skimage.util.img_as_float` performs the same rescaling for `uint8` images.

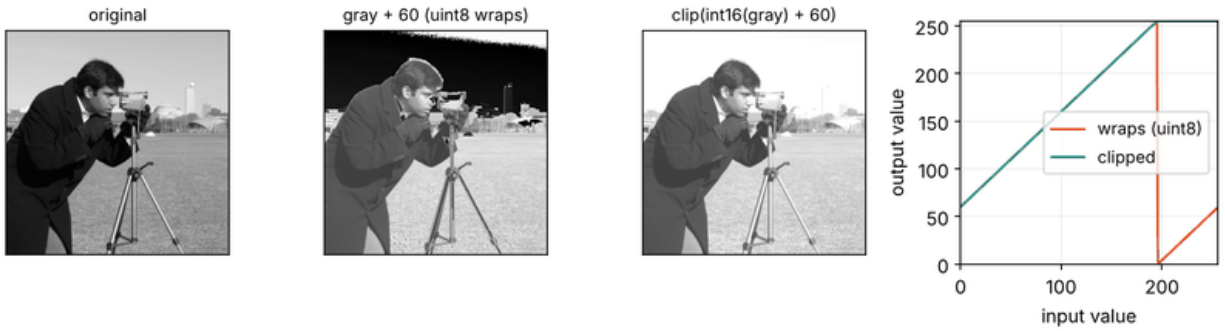


Figure 3 — Brightening by 60 levels. In uint8 the bright sky wraps around to black (orange curve). Converting to a wider type, adding, and clipping saturates it to white instead (green curve).

Interactive demonstration (**online**): open the online edition of this tutorial to use this demonstration.Open demonstration

Choose the dtype before the arithmetic

Casting after an overflow cannot recover the lost value. Decide the working type first: `float32` in $[0, 1]$ for filtering and learning, `int16` for exact integer differences, and convert back to `uint8` only for saving or display. OpenCV's own arithmetic helpers such as `cv2.add` saturate instead of wrapping, but NumPy's `+` operator does not, so know which one a line of code is using. The lab above runs the same arithmetic on the photo, so you can try each choice yourself.

Sampling and quantization on a real image

Plain version. Two independent knobs control how much of the scene survives: the number of pixels (spatial resolution) and the number of gray levels (brightness resolution). Turning down the first makes the picture blocky; turning down the second makes smooth shading break into flat bands.

Precise version. Keeping every s-th row and column, `gray[::s, ::s]`, reduces an $H \times W$ image to about $H/s \times W/s$ samples. Done without smoothing first, this can create **aliasing**: fine detail that the coarser grid cannot represent shows up as false patterns instead of disappearing (the formal treatment is in IP 02 and the frequency-domain chapters). Reducing the bit depth from 8 to b keeps only 2^b distinct values. The helper below rounds each pixel to the center of its bin, so with 4 bits there are 16 allowed gray levels.

```
def quantize(img, bits):
    step = 256 // 2 ** bits
    return (img // step) * step + step // 2    # replace each bin by its center

for bits in (8, 4, 2, 1):
    print(bits, len(np.unique(quantize(gray, bits))))

small = gray[::4, ::4]                          # keep every 4th row and column
print(gray.shape, "->", small.shape, small.nbytes, gray.nbytes)

def invert_loop(img):
    out = np.empty_like(img)
    for r in range(img.shape[0]):
        for c in range(img.shape[1]):
            out[r, c] = 255 - img[r, c]
    return out

print(np.array_equal(invert_loop(gray), 255 - gray))
chw = np.transpose(rgb, (2, 0, 1))
print(rgb.shape, "->", chw.shape, np.shares_memory(chw, rgb))
print(np.mean(rgb, axis=(0, 1)))
```

Output:

```
8 256
4 16
```

```
2 4
1 2
(512, 512) -> (128, 128) 16384 262144
True
(512, 512, 3) -> (3, 512, 512) True
[141.56249237 105.75944519  96.47507477]
```

The number of distinct values falls as 2^b. Keeping every 4th row and column shrinks the image from 262,144 bytes to 16,384, one sixteenth of the memory. The nested loop and the single expression `255 - gray` give identical results (`True`), but the array expression runs as one compiled operation rather than a quarter of a million Python steps: write image code as array expressions whenever you can, and keep explicit loops for checking. Transposing to channels-first, `(3, 512, 512)`, is another view, which is the layout most deep-learning libraries expect. The last line, a mean over both spatial axes, gives one average per color channel.

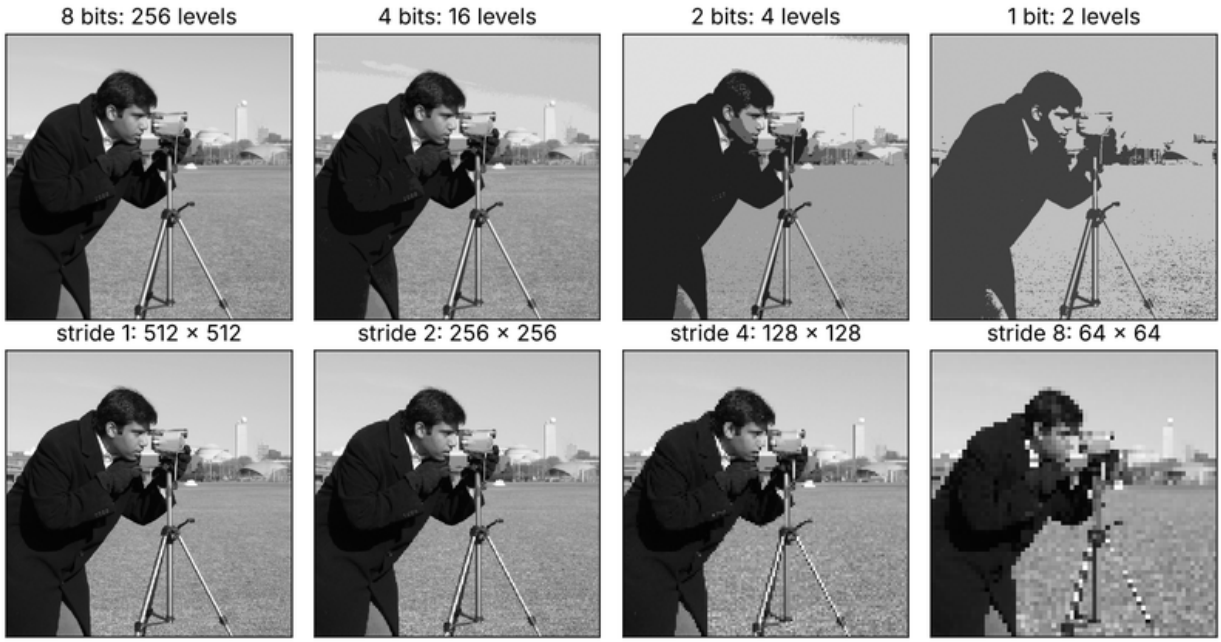


Figure 4 — Two ways to lose information. Top: fewer gray levels (quantization) turns smooth shading into flat bands. Bottom: fewer samples (here plain point sampling, shown at equal size) makes the image blocky and breaks thin structures such as the tripod legs.

> **Interactive demonstration (online)**: open the online edition of this tutorial to use this demonstration.Open demonstration

Eight bits per channel is usually enough to show an image, but not always enough to process one: stretching the contrast of a dark, banded image only magnifies the bands. That is why image formats and sensors with more bits per channel exist, and why processing pipelines convert to floating point for the arithmetic. The lab above moves both knobs on a photo and on a test pattern that exposes aliasing.

A short checklist before you process an image

Check	Why	One line
Print `shape` and `dtype`	Confirms grayscale versus color and the value range	`print(a.shape, a.dtype, a.min(), a.max())`
Know the channel order	RGB and BGR look similar until the colors are wrong	`rgb = bgr[..., ::-1]`
Choose the working dtype	`uint8` wraps; `float32` in $[0, 1]$ is safe for filtering	`x = a.astype(np.float32) / 255`
Copy before editing	Slices are views and edits write through	`b = a.copy()`
Clip before converting back	Out-of-range floats become garbage in `uint8`	`np.clip(y, 0, 1)`
Remember `[row, column]`	Pairs from graphics APIs are `(x, y)`	`a[y, x]`

Modern view

Nothing in this chapter is obsolete, because deep-learning frameworks treat an image as a tensor and tensors behave like NumPy arrays. What changes is the *layout*, the *normalization* and, in recent research, the idea of what an image is.

Tensors and layout. A training batch adds a leading axis, giving `(N, H, W, C)` or the channels-first `(N, C, H, W)` that many libraries expect, as in the `np.transpose` view above. The pixel values are usually converted to `float32` and normalized, for example by dividing by 255 and standardizing each channel. All the pitfalls above carry over unchanged: a model trained on RGB input and fed BGR will silently degrade, and a preprocessing step that wraps in `uint8` can poison a whole dataset without raising an error. The convolution and Transformer chapters build directly on this view: ML 03 and ML 04.

Images as sequences of patches. The Vision Transformer of Dosovitskiy et al. [10] (ICLR 2021) cuts an image into a grid of fixed-size patches, flattens each patch into a vector with the reshaping used in this chapter, and feeds the sequence to a standard Transformer. The paper reports that such a model, with large-scale pre-training, can match or exceed convolutional networks on image classification benchmarks while needing less training compute. The key point for this chapter is that the array operations (slicing into blocks, reshaping, transposing) *are* the model's front end.

Images as functions. A second modern idea replaces the grid with a learned function. Sitzmann et al. [11] (NeurIPS 2020) propose sinusoidal representation networks, small networks with periodic activations that take coordinates (r, c) as input and return the pixel value, and show that they can represent images, audio, video and their derivatives in detail. Essakine et al. [12] (TMLR 2025) survey this *implicit neural representation* field: they propose a taxonomy over four groups of methods (activation functions, position encoding, combined strategies and network-structure optimization), compare them experimentally, and conclude with open problems such as more expressive activation functions, better position encodings and scalability to high-dimensional data. The array is still how such models are trained and evaluated, since the network is fitted to sam-

pled pixels; but resolution becomes a property of how you query the function, not of the stored table.

What is still open and what is not. Questions that remain unsettled include how to store high-dynamic-range and wide-gamut color consistently across tools, and how learned representations should be compared with plain arrays at the same storage cost. Questions that are settled, and worth drilling until they are automatic, are the ones in this chapter: shape, dtype, channel order, views versus copies, and the habit of choosing the arithmetic type before computing.

Key takeaways

- A digital image is a sampled, quantized signal stored as an array of shape `(H, W)` or `(H, W, C)`; $I[r, c]$ means row r, column c.
- `shape`, `dtype` and `strides` describe the array. A 512×512 8-bit grayscale image holds 262,144 bytes; the color version holds three times as many.
- Slicing returns views that share memory with the original. Use `.copy()` before editing, and use array expressions rather than pixel loops.
- Libraries disagree on channel order (OpenCV uses BGR, most others RGB) and on the weights used for gray conversion; check, do not assume.
- `uint8` arithmetic wraps modulo 256. Convert to `int16` or `float32`, compute, clip, and only then convert back.
- Spatial resolution and gray-level resolution are separate choices, and reducing either loses information in a different way: blockiness and aliasing versus banding.
- Deep-learning tensors, image patches and implicit neural representations are all built on the same array view of an image.

Exercises

1. **Channel means.** Load `data.astronaut()` and compute the mean of each color channel with `rgb.mean(axis=(0, 1))`. Which channel is largest? Then write the result for the BGR version of the same image and explain which index now holds the largest value.

   Hint

   For this image the means are about 141.6, 105.8 and 96.5 for R, G and B, so red is largest. In the BGR array the same numbers appear in reversed order, so the largest value is at index 2 instead of index 0.

2. **Flip with slicing.** Using only slicing (no `np.flip` and no loops), produce the upside-down, left-right mirrored and 180°-rotated versions of `gray`. Check each against `np.flipud`, `np.fliplr` and `np.rot90(gray, 2)`.

   Hint

   Use a step of -1 on the axis you want to reverse: `gray[::-1]`, `gray[:, ::-1]`, and `gray[::-1, ::-1]`. All three are views, so copy them if you plan to edit.

3. **Find the wrap.** For the brightening by 60 in this page, which input gray values wrap around, and what output value does an input of 200 produce? Write a one-line check that counts the wrapped pixels without computing the wrapped image.

   Hint

   Inputs of 196 or more wrap, because $196 + 60 = 256 \equiv 0$. An input of 200 gives $260 \bmod 256 = 4$. The count is `(gray >= 196).sum()`.

4. **Safe difference.** Two `uint8` frames A and B should be compared with the absolute difference $|A - B|$. Show with a pair of pixel values that `A - B` fails, and write a safe version in NumPy.

   Hint

   With $A = 10$ and $B = 20$ in `uint8`, `A - B` equals 246, not 10. A safe version is `np.abs(A.astype(np.int16) - B.astype(np.int16)).astype(np.uint8)`.

5. **Design a bit-depth budget.** A grayscale image is stored with 8 bits, then quantized to b bits. Predict the number of distinct values and the memory needed if each pixel is packed to exactly b bits, for $b = 4$ and $b = 2$ at 512×512. Why does the picture in Figure 4 show bands at 4 bits but not at 8?

   Hint

   There are 2^b values: 16 and 4. Packed storage is $512 \times 512 \times b/8$ bytes, which is 131,072 bytes at 4 bits and 65,536 bytes at 2 bits. Bands appear where a smooth gradient crosses a boundary between two bins: at 8 bits the steps are 1 gray level, below what the eye can distinguish on most displays, while at 4 bits they are 16 levels apart.

References

1. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018 (Chapter 2, image sensing, sampling and quantization). publisher page
2. A. Torralba, P. Isola and W. T. Freeman, *Foundations of Computer Vision*, MIT Press, 2024 (open-access edition). book site
3. C. R. Harris et al., "Array programming with NumPy," *Nature*, vol. 585, pp. 357–362, 2020. doi
4. NumPy developers, "Data types," NumPy user guide, current documentation. docs
5. NumPy developers, "NEP 50: Promotion rules for Python scalars," NumPy Enhancement Proposals. NEP 50
6. S. van der Walt et al., "scikit-image: image processing in Python," *PeerJ*, vol. 2, e453, 2014. doi
7. scikit-image developers, "skimage.data: test images and example data," scikit-image documentation. docs
8. ITU-R, "Recommendation BT.601-7: Studio encoding parameters of digital television for standard 4:3 and wide-screen 16:9 aspect ratios," 2011. ITU page

9. Matplotlib developers, “matplotlib.pyplot.imshow,” Matplotlib documentation. docs
10. A. Dosovitskiy et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” *International Conference on Learning Representations* (*ICLR*), 2021. arXiv:2010.11929
11. V. Sitzmann, J. N. P. Martel, A. W. Bergman, D. B. Lindell and G. Wetzstein, “Implicit Neural Representations with Periodic Activation Functions,” *Advances in Neural Information Processing Systems* (*NeurIPS*), vol. 33, 2020. proceedings
12. A. Essakine et al., “Where Do We Stand with Implicit Neural Representations? A Technical and Performance Survey,” *Transactions on Machine Learning Research* (*TMLR*), 2025. arXiv:2411.03688

Part II

Image Processing

IP 01 • Introduction

What digital image processing is, where it came from, which parts of the electromagnetic spectrum it works with, and the roadmap of fundamental steps this series follows.

Online edition: https://blog.shyandram.dev/en/learn/image-processing/dip-01-introduction/

What you'll learn

- What a digital image is, in words and as a function $f(x, y)$, and what "processing" one means
- Where image processing stops and computer vision begins, using the low-, mid- and high-level view
- How the field started: newspaper photos sent by telegraph, lunar probes, and medical scanners
- Which parts of the electromagnetic spectrum (and which non-light signals) produce the images we process
- The fundamental steps of image processing, which double as the roadmap for the 13 chapters of this series
- The hardware and software components of a working image processing system, and how deep learning is reshaping the camera pipeline

The big picture

> **Explain like I'm five**
>
> Imagine a huge mosaic made of tiny square tiles. Each tile has one shade of gray, written as a number: 0 for black, 255 for white. A computer cannot "see" the mosaic. It can only read the list of numbers. **Digital image processing** is the set of tricks we use to change those numbers so the picture becomes clearer, smaller, or easier to understand: wiping away specks of dust, making a dark photo brighter, or drawing a line around every coin on a table so we can count them.

Almost every image you look at today has passed through image processing before it reached your eyes. Your phone ran dozens of operations between the moment light hit its sensor and the moment the photo appeared on screen. A hospital CT scanner never takes a "photo" at all: it measures how X-rays are absorbed along thousands of lines and *computes* the picture. A weather satellite records energy the human eye cannot see and turns it into a map you can read.

This chapter is the map before the journey. It defines the field, tells you where it came from, shows the range of signals it handles, and lays out the steps that the remaining chapters cover one by one [1]. Nothing here is hard, but every later chapter refers back to the vocabulary we build now.

What is digital image processing?

Plain version. A picture becomes *digital* when we chop it into a grid of tiny squares and store one number (or a few numbers, for color) per square. *Processing* means feeding that grid of numbers to a computer and getting something useful back.

Images as functions

Mathematically, a grayscale image is a two-dimensional function

$$f(x, y), \qquad (x, y) \in \mathbb{R}^2.$$

Here x and y are spatial coordinates, and the value $f(x, y)$ is the **intensity** (or *gray level*) of the image at that point. For a camera, intensity is proportional to the light energy that reached the sensor; for an X-ray film, to the radiation that passed through the body. In the real world x, y and f can all vary continuously.

An image is **digital** when all three quantities are finite and discrete. We sample the plane on an $M \times N$ grid and quantize each value to one of L levels:

$$f : \{0, 1, \dots, M-1\} \times \{0, 1, \dots, N-1\} \to \{0, 1, \dots, L-1\}, \qquad L = 2^k.$$

M is the number of rows, N the number of columns, and k the number of bits per value ($k = 8$ gives the familiar range 0 to 255). Each grid element is a **pixel** (short for *picture element*). Chapter 2 covers how sampling and quantization are chosen; for now it is enough to know that an image is a matrix.

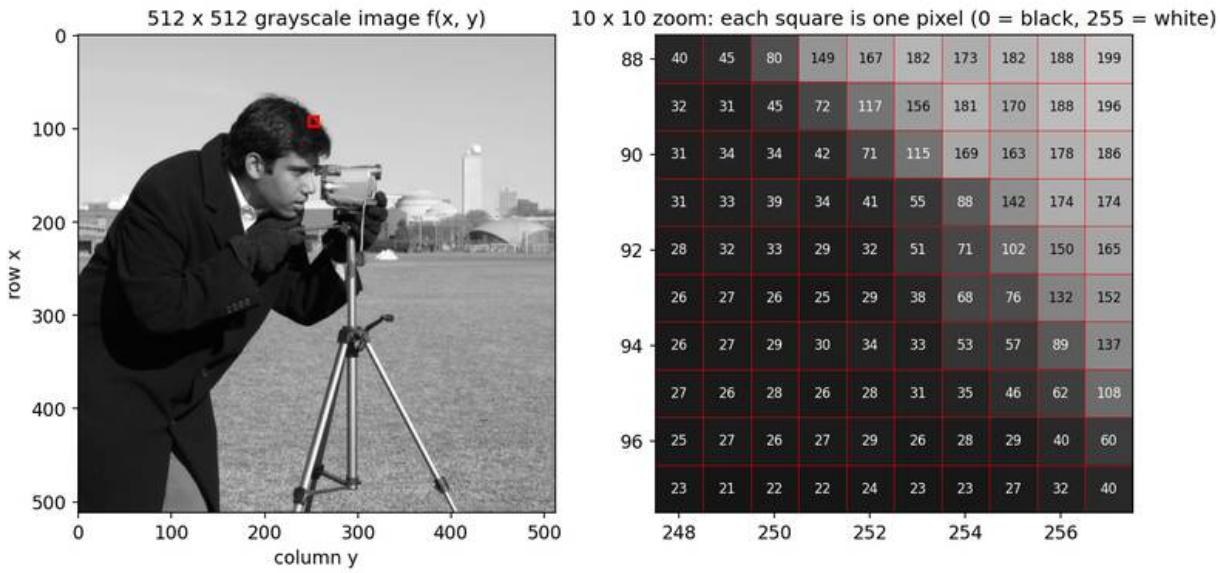


Figure 1.1 — Left: a 512 × 512, 8-bit grayscale image. Right: the 10 × 10 patch inside the red square, magnified. Each square is one pixel; the printed number is its gray level. The edge between dark hair and bright sky is nothing more than a jump from values near 30 to values near 180.

In code, this matrix is a NumPy array. Indexing with `[row, column]` reads a single pixel, and slicing returns a neighborhood:

```
from skimage import data

f = data.camera()            # a 512 x 512 grayscale photograph
print(f.shape, f.dtype)      # (512, 512) uint8
print(f[100, 250])           # intensity at row 100, column 250
print(f.min(), f.max())      # darkest and brightest values
patch = f[88:92, 248:252]    # a 4 x 4 neighborhood is just a sub-array
print(patch)
```

Which axis is x?

This series follows the textbook convention [1]: x indexes rows (top to bottom) and y indexes columns (left to right), so $f(x, y)$ matches NumPy's `f[x, y]`. Many graphics libraries, and the prerequisite tutorial Images as Arrays, use the opposite order (row $= y$, column $= x$). When code and math disagree, check the axes first.

Color images add a third index for the channel, $f(x, y, c)$, and medical volumes add a depth index, $f(x, y, z)$. The ideas in this chapter carry over unchanged.

Where processing ends and vision begins

People disagree about where *image processing* stops and *image analysis* or *computer vision* starts. One extreme says image processing covers only operations whose input and output are both images. That rule is clean but too narrow: it would exclude something as simple as computing the average brightness of a photo. The other extreme is computer vision, whose goal is to emulate human vision: to understand a scene, recognize objects and act on them. Textbooks on vision treat it as a field in its own right, with geometry, 3-D reconstruction and recognition at its core [7].

A more useful picture is a continuum with three kinds of processes [1]:

- **Low-level processes** take an image in and give an image out. Examples: removing noise, increasing contrast, sharpening. No interpretation is involved.
- **Mid-level processes** take an image in and give *attributes* out: regions, edges, contours, or measurements of individual objects. Segmentation (splitting an image into meaningful parts) and describing those parts in a form a computer can sort are typical mid-level tasks.
- **High-level processes** "make sense" of a collection of recognized objects. Reading a whole page of text, or deciding that a medical scan shows a tumor, are high-level tasks. This end of the continuum is where image analysis merges into computer vision.

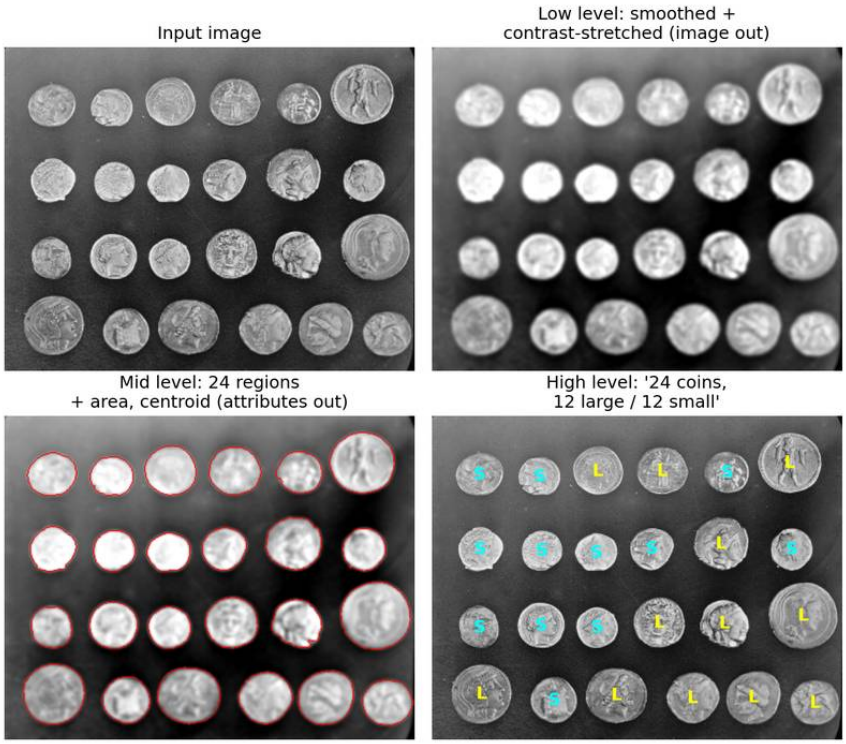


Figure 1.2 — The processing continuum on one image. Low level: denoise and stretch contrast (an image comes out). Mid level: segment the coins and measure each one (a list of regions and numbers comes out). High level: interpret the measurements (a statement comes out: 24 coins, half of them large).

In this series, "digital image processing" means everything from the low level up to and including the recognition of individual regions or objects. That is wide enough to include segmentation, feature extraction and classification (Chapters 10–13), but it stops short of full scene understanding.

The origins of digital image processing

Plain version. The first digital pictures were newspaper photos sent across the Atlantic as punched telegraph tape. Real computer processing of images began in the 1960s, when space agencies needed to clean up pictures from lunar probes, and it exploded in the 1970s with medical scanners that build pictures from measurements.

Pictures over a telegraph cable

In the early 1920s, the **Bartlane cable picture transmission system** sent newspaper photographs between London and New York over the transatlantic submarine telegraph cable [2]. A photograph was converted into codes punched on ordinary five-unit telegraph tape, sent with standard telegraph equipment, and reassembled offline at the receiving end. According to Kobayashi's history of the system, it carried nearly 500 news pictures across the Atlantic until the outbreak of

war in 1939 [2]. The earliest Bartlane pictures used only five distinct gray levels; by the end of the 1920s the number had grown to fifteen [1].

Strictly speaking, no *computer* touched these pictures, so this is the prehistory of digital image processing. But the core idea was already there: a picture is turned into a finite set of codes, transmitted, and rebuilt, and the number of gray levels determines how faithful the result is.

Figure 1.3 — How much do gray levels matter? The same photo quantized to 5 levels (early Bartlane), 15 levels (late 1920s Bartlane) and 256 levels (modern 8-bit). With 5 levels, smooth areas such as the coat and the grass collapse into flat patches; even with 15 levels the sky still shows visible bands. This artifact, called false contouring, is explained in Chapter 2.

You can reproduce this experiment in a few lines. Uniform quantization to L levels maps an intensity $r \in [0, 1]$ to

$$Q_L(r) = \frac{\operatorname{round}\left(r\,(L-1)\right)}{L-1},$$

where round rounds to the nearest integer, so the output takes only the L values $0, \frac{1}{L-1}, \ldots, 1$.

```
import numpy as np
from skimage import data

def quantize(img, levels):
    """Map a uint8 image onto `levels` evenly spaced gray levels."""
    x = img.astype(np.float64) / 255.0
    q = np.round(x * (levels - 1)) / (levels - 1)
    return (q * 255).astype(np.uint8)

f = data.camera()
for L in (5, 15, 256):
    g = quantize(f, L)
    print(L, "levels ->", len(np.unique(g)), "distinct values,",
          f"mean abs error {np.abs(g.astype(int) - f).mean():.2f}")
```

On the test image, the mean absolute error falls from about 18 gray levels with $L = 5$ to about 5 with $L = 15$, and to 0 with $L = 256$.

Computers meet images: the space program

Digital image processing as we know it needed two things that arrived only in the 1960s: computers powerful enough to hold and manipulate an image, and a problem important enough to pay for them. The space program provided both.

On 31 July 1964, the U.S. probe **Ranger 7** took the first image of the Moon by a U.S. spacecraft, about 17 minutes before it crashed into the lunar surface as planned [3]. During those final minutes it returned more than 4,300 images, the last ones resolving details of about half a meter per pixel [3]. Using a computer to correct the distortions in such probe images became one of the founding projects of digital image processing [1]. The same techniques were then refined for later lunar and planetary missions.

Images computed from measurements: CT

The second founding application came from medicine. In 1973, Godfrey Hounsfield described a system for **computerized transverse axial scanning**, now called computed tomography (CT) [4]. Instead of exposing one film, an X-ray source and detectors rotate around the patient and record how strongly the beam is absorbed along many lines at many angles. A computer then calculates the absorption at every point of a cross-section. Hounsfield reported that this revealed soft-tissue differences that conventional X-ray films could not show [4].

CT is a milestone because the image is not *captured*; it is *reconstructed*. The scanner measures line integrals of the unknown slice $f(x, y)$:

$$p(s, \theta) = \iint_{\mathbb{R}^2} f(x, y)\, \delta(x \cos\theta + y \sin\theta - s)\, dx\, dy,$$

where θ is the angle of the beam, s is the detector position along the projection, and δ is the Dirac delta, which keeps only the points on the line $x \cos\theta + y \sin\theta = s$. The collection of all projections $p(s, \theta)$ is the **sinogram**. Recovering f from p is an inverse problem, the subject of the reconstruction half of Chapter 5. The standard test object for such algorithms is a synthetic head slice made of ellipses, introduced by Shepp and Logan in 1974 [5].

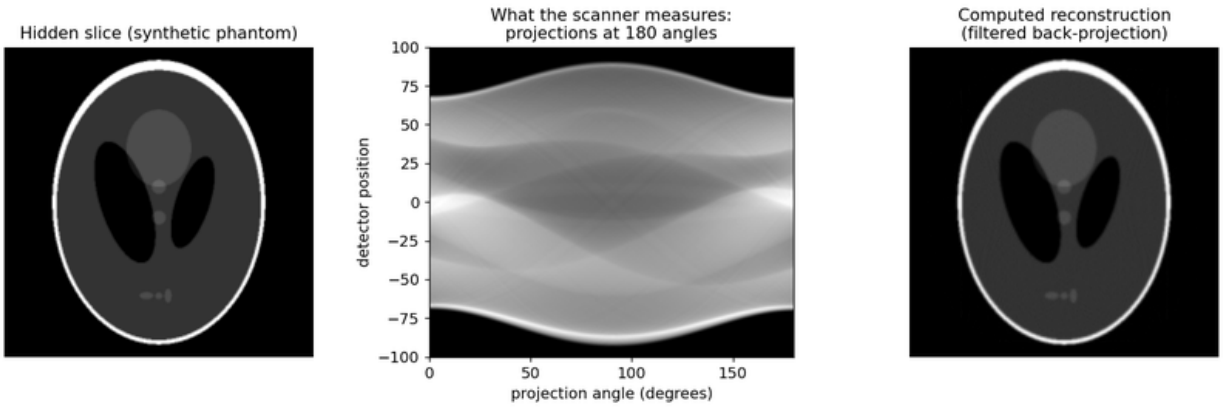


Figure 1.4 — Computed tomography in miniature. Left: an unknown slice (the Shepp–Logan phantom [5]). Middle: what the scanner actually records, one projection per angle. Right: the slice recovered by filtered back-projection, an algorithm derived in Chapter 5.

```
import numpy as np
from skimage.data import shepp_logan_phantom
from skimage.transform import radon, iradon, rescale

slice_ = rescale(shepp_logan_phantom(), 0.5)           # 200 x 200 synthetic head slice
angles = np.linspace(0, 180, 180, endpoint=False)
sinogram = radon(slice_, theta=angles)                  # what the scanner records
recon = iradon(sinogram, theta=angles, filter_name="ramp")
err = np.sqrt(np.mean((recon - slice_) ** 2))
print(sinogram.shape, recon.shape, f"RMS error {err:.3f}")   # (200, 180) (200, 200) ~0.025
```

From the 1960s onward, the same ideas spread into remote sensing, astronomy, biology, industrial inspection, law enforcement and, eventually, every phone in every pocket. Two broad goals drive all of these applications: improving pictures for *human* interpretation, and processing image data for storage, transmission and *machine* perception.

Examples of fields that use digital image processing

Plain version. Our eyes see only a thin slice of the "light" that exists. Other kinds of light, from gamma rays to radio waves, can also make pictures, and so can sound and electron beams. Image processing works on all of them, because they all end up as grids of numbers.

The most useful way to organize applications is by the **source** of the image [1]. The main source is electromagnetic (EM) energy; others are acoustic waves, electrons, and pure computation.

The electromagnetic spectrum

EM waves can be described by wavelength λ or frequency ν, which are linked by

$$\lambda = \frac{c}{\nu},$$

where $c \approx 2.998 \times 10^8$ m/s is the speed of light. Light also comes in packets, **photons**, each carrying energy

$$E = h\nu = \frac{hc}{\lambda},$$

where $h \approx 6.626 \times 10^{-34}$ J·s is Planck's constant. Short wavelength means high frequency and energetic photons; long wavelength means low frequency and gentle photons. That single fact explains much of imaging: energetic photons pass through tissue and metal (X-rays), while low-energy waves pass through clouds and darkness (radar and radio).

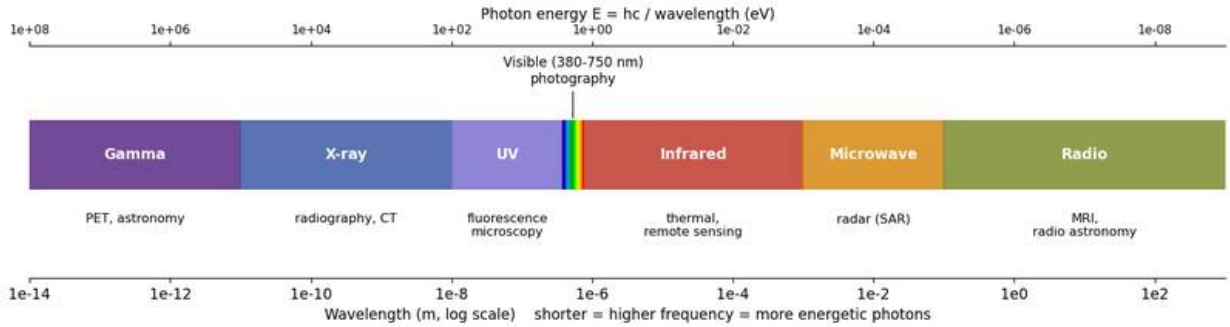


Figure 1.5 — The electromagnetic spectrum on a log scale, with photon energy on the top axis. The visible band is a sliver. Band boundaries are conventions and vary between sources.

```
h, c, eV = 6.626e-34, 2.998e8, 1.602e-19   # Planck, speed of light, joules per eV

def photon_energy_ev(wavelength_m):
    return h * c / wavelength_m / eV

for name, lam in [("X-ray (0.1 nm)", 1e-10), ("green light (550 nm)", 550e-9),
                  ("thermal IR (10 um)", 10e-6), ("radar (3 cm)", 0.03)]:
    print(f"{name:22s} {photon_energy_ev(lam):10.3g} eV")
# X-ray ~1.24e4 eV, green ~2.25 eV, thermal IR ~0.124 eV, radar ~4.1e-5 eV
```

An X-ray photon carries roughly ten thousand times more energy than a photon of green light, and a radar photon carries less than a ten-thousandth of it. The bands below run from the most energetic to the least.

Gamma-ray imaging

Gamma rays are the most energetic photons. In **nuclear medicine**, a patient receives a radioactive tracer, and detectors record the gamma rays it emits; the image shows *where the tracer collects*, so it maps function (for example, metabolism) rather than anatomy. Positron emission tomography

(PET) is a tomographic version of this idea and uses the same reconstruction mathematics as CT. In **astronomy**, gamma-ray telescopes image violent events such as the remains of exploded stars. Gamma-ray images are typically noisy, because every pixel counts a modest number of photon events; denoising and restoration (Chapters 3 and 5) matter here.

X-ray imaging

X-rays are the oldest medical imaging source. In a plain **radiograph**, X-rays pass through the body and expose a detector; dense tissue such as bone absorbs more and appears bright. **Angiography** injects a contrast agent into blood vessels and subtracts a pre-contrast image from a post-contrast one to isolate the vessels, which is an image arithmetic operation you will meet in Chapter 2. **CT**, described above, turns many X-ray projections into slices and stacks of slices [4]. Outside medicine, X-rays inspect welds and circuit boards for hidden defects, and X-ray telescopes image hot gas in galaxy clusters.

Ultraviolet imaging

Ultraviolet (UV) light sits just beyond violet. Its key application is **fluorescence microscopy**: some substances absorb UV photons and re-emit lower-energy visible light. Under a microscope that blocks the UV excitation and passes only the emitted light, the fluorescing structures glow against a dark background. Biologists tag specific proteins with fluorescent markers to see where they are in a cell. UV imaging is also used in astronomy and in industrial inspection, for example to reveal residues invisible in daylight.

Visible and infrared imaging

The visible band is where most everyday imaging happens: photography, **light microscopy**, industrial **machine vision** (checking that a pill packet is full, reading license plates, measuring parts), document scanning, and face and fingerprint analysis. The visible band is often combined with the nearby **infrared** (IR) band.

Remote sensing satellites record the same scene in several narrow bands from blue to infrared, called *multispectral* images. Because vegetation, water and soil reflect these bands differently, combining bands lets analysts map crops, floods or urban growth. **Thermal infrared** cameras sense emitted heat rather than reflected light, which makes them useful at night, in firefighting and in building inspection. Weather satellites use IR channels to track clouds and storms around the clock.

Microwave imaging (radar)

The dominant microwave imaging technology is **imaging radar**. A radar carries its own illumination: it sends microwave pulses and records the echoes. Because microwaves pass through clouds, haze and darkness, radar can image regions that optical satellites cannot see for weeks

at a time, such as rainforests under constant cloud cover. *Synthetic aperture radar* (SAR) uses the motion of an aircraft or satellite to synthesize a very large antenna, giving fine resolution; turning the recorded echoes into an image is itself a computational step [8].

Radio-band imaging

At the long-wavelength end, the major application in medicine is **magnetic resonance imaging** (**MRI**). The patient lies in a strong magnetic field, short radio-frequency pulses excite hydrogen nuclei, and the faint radio signals they emit are recorded. In 1973, Paul Lauterbur showed that adding magnetic field gradients makes these signals depend on position, so an image can be formed from them [6]. Like CT, MRI produces an image only after computation: the raw measurements live in a frequency domain (the subject of Chapter 4) and must be transformed back.

In **radio astronomy**, the long wavelengths mean a single dish has poor resolution, so astronomers combine signals from many telescopes. In 2019, the Event Horizon Telescope Collaboration published the first image of the shadow of a supermassive black hole, at the center of the galaxy M87, made by combining data from radio observatories around the globe [9]. That image is as much the product of reconstruction algorithms as of the telescopes themselves.

Other imaging modalities

Not every image comes from EM radiation.

- **Acoustic imaging.** Sound waves reflect at boundaries between materials. Geologists send low-frequency sound into the ground and record echoes to map underground rock layers for oil, gas and mineral exploration (*seismic imaging*). Medical **ultrasound** uses high-frequency sound (millions of cycles per second) to image the fetus, the heart and abdominal organs in real time and without ionizing radiation. In both cases the image is formed by timing the echoes.
- **Electron microscopy.** Electrons can be focused like light, but their effective wavelength is far shorter, so electron microscopes resolve much finer detail than light microscopes. A *transmission* electron microscope passes a beam through a thin specimen; a *scanning* electron microscope sweeps a focused beam across a surface and records the electrons that bounce off or are knocked out. The images are often noisy and benefit from enhancement and restoration.
- **Synthetic images.** Computers also generate images that no sensor captured: fractals, rendered 3-D scenes, simulations, and, increasingly, images produced by generative models. These images are processed with the same tools, and they are widely used to create training data and test cases.

The lesson of this tour: once the data is a grid of numbers, the same toolbox applies, whether the numbers came from gamma rays, sound echoes or a computer program. What changes is the noise, the resolution and the physics of formation, and those differences decide which tool you reach for.

Fundamental steps in digital image processing

Plain version. Making an image useful is like cooking: you get the ingredients (acquire the image), clean and prepare them (enhance, restore), sometimes change their form (color, transforms, compression), then cut them into pieces (segmentation), describe each piece, and finally decide what each piece is (classification). You do not always need every step.

It helps to split the steps into two groups [1]: methods whose outputs are generally **images**, and methods whose outputs are generally **attributes** extracted from images. A **knowledge base** about the problem domain, such as where defects tend to appear or what a valid label looks like, guides every step. The list is not a mandatory pipeline; a given application uses only the steps it needs, in whatever order makes sense.

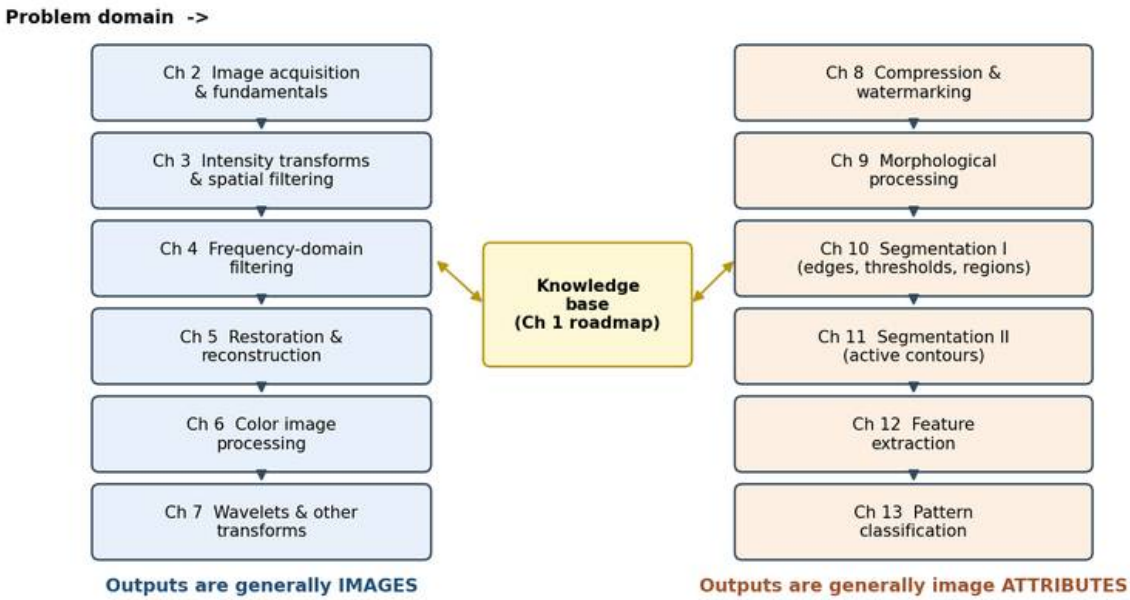


Figure 1.6 — The fundamental steps of digital image processing, labeled with the chapter of this series that covers each. Steps on the left generally produce images; steps on the right generally produce attributes. Domain knowledge (center) informs all of them.

Here is each step, with the chapter that develops it.

1. **Image acquisition** — obtaining the image in digital form, including sensing, sampling and quantization. Chapter 2 also builds the mathematical toolkit (pixel relationships, arithmetic, geometric transforms) used everywhere else. → IP 02 ·Digital Image Fundamentals
2. **Image enhancement** — making an image more suitable for a specific purpose, such as increasing contrast or sharpening detail. Enhancement is subjective: "better" depends on the viewer and the task. → IP 03 ·Intensity Transformations and Spatial Filtering and IP 04 ·Filtering in the Frequency Domain
3. **Image restoration** — undoing a known or estimated degradation such as blur or noise. Unlike enhancement, restoration is objective: it relies on mathematical models of how the image was

degraded. The same chapter covers reconstruction from projections (CT). → IP 05 ·Image Restoration and Reconstruction

4. **Color image processing** — color models, pseudo-color, and processing of full-color images. → IP 06 ·Color Image Processing
5. **Wavelets and other image transforms** — representing images at several resolutions at once, the basis of modern compression and pyramid methods. → IP 07 ·Wavelet and Other Image Transforms
6. **Compression and watermarking** — reducing the storage or bandwidth an image needs, and embedding invisible information to protect ownership. → IP 08 ·Image Compression and Watermarking
7. **Morphological processing** — tools for extracting and cleaning up shape components, such as thinning, filling holes and separating touching objects. This is where outputs start to become attributes. → IP 09 ·Morphological Image Processing
8. **Segmentation** — partitioning an image into its constituent parts or objects. It is one of the hardest steps, and errors here propagate to everything after it. → IP 10 ·Image Segmentation I (edges, thresholds, regions) and IP 11 ·Image Segmentation II (active contours: snakes and level sets)
9. **Feature extraction** — turning segmented regions into numbers: boundary and region descriptors, and point features such as corners. It has two parts: *detecting* features and *describing* them. → IP 12 ·Feature Extraction
10. **Image pattern classification** — assigning a label to an object based on its features, from minimum-distance classifiers to neural networks and deep learning. → IP 13 ·Image Pattern Classification

Chapter 1 (this page) is the overview that ties them together.

The following sketch strings four of these steps together on the coins image of Figure 1.2. Every step is one or two library calls here; the chapters explain what is going on inside each.

```
from scipy import ndimage as ndi
from skimage import data, filters, measure, segmentation

img = data.coins()

# 1) enhancement: low level, image in -> image out
smooth = filters.gaussian(img, sigma=1, preserve_range=True)

# 2) segmentation: mid level, image in -> regions out
markers = (smooth < 30) * 1 + (smooth > 150) * 2       # sure background / sure coin
regions = segmentation.watershed(filters.sobel(smooth), markers) == 2
regions = ndi.binary_fill_holes(regions)
labels = measure.label(regions)

# 3) feature extraction: regions in -> numbers out
props = [p for p in measure.regionprops(labels) if p.area > 300]
areas = [p.area for p in props]

# 4) classification / interpretation: numbers in -> a statement out
median = sorted(areas)[len(areas) // 2]
```

```
n_large = sum(a >= median for a in areas)
print(f"{len(areas)} coins found, {n_large} at or above the median size")
# 24 coins found, 12 at or above the median size
```

Notice the knowledge base hiding in this code: the thresholds 30 and 150, the minimum area 300 and the rule "large means at or above the median" all encode facts about this particular problem. Change the lighting or the coins and those numbers must change too. Much of the history of the field, including the rise of deep learning, is the story of moving such knowledge from hand-set constants into models learned from data.

Components of an image processing system

Plain version. An image processing system is a team: a sensor that catches the image, fast special-purpose chips that do the heavy lifting, a computer that runs the programs, memory that stores the pictures, a screen to look at them, a printer for paper copies, and a network that connects it all.

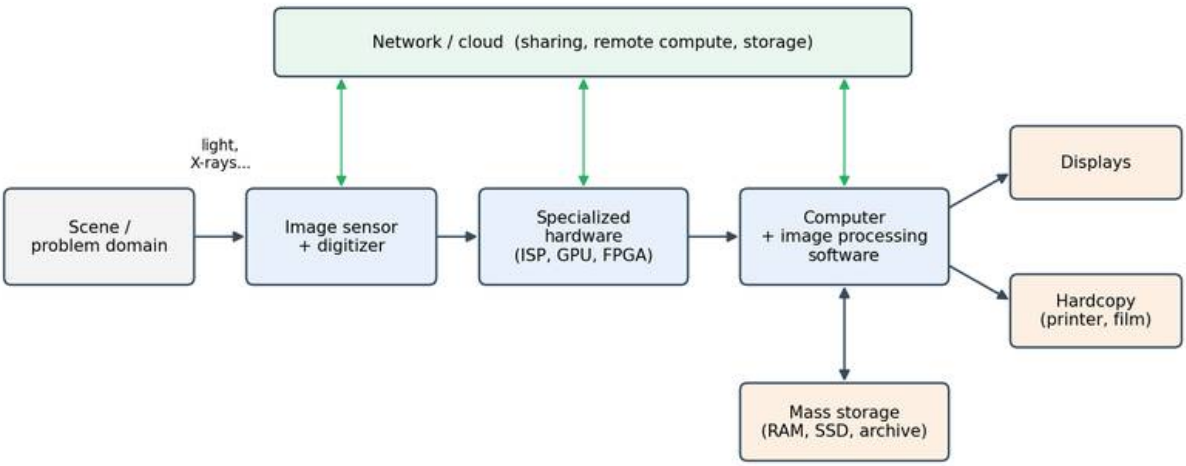


Figure 1.7 — Components of a general-purpose image processing system. Arrows show the main data flow; the network connects every stage to remote storage and computation.

- **Image sensors.** Two elements are needed to acquire an image: a physical device that responds to the energy radiated by the object (a CCD or CMOS chip, an X-ray detector, an ultrasound transducer), and a **digitizer** that converts the device's electrical output into numbers. In a phone, both sit on the same chip.
- **Specialized image processing hardware.** Some operations must run at video rate on every pixel, faster than a general-purpose processor can manage. Classic systems used dedicated arithmetic boards; today the role is filled by the *image signal processor* (ISP) inside every camera

chip, by graphics processing units (GPUs), by field-programmable gate arrays (FPGAs), and by neural processing units in phones.
- **The computer.** Anything from an embedded microcontroller in a smart doorbell to a cluster of servers. For dedicated tasks, the computer may be customized; for research and general work, an ordinary workstation is enough.
- **Software.** Modules that perform specific tasks, plus a way to combine them. In this series we use Python with NumPy, SciPy, scikit-image and OpenCV. The ability to write your own code, rather than only call existing functions, is what this series aims to build.
- **Mass storage.** Images are large, and they arrive in bulk. Storage comes in three tiers: *short-term* storage during processing (RAM, GPU memory, frame buffers), *online* storage for fast recall (SSDs and disk arrays), and *archival* storage for rare access (tape, cold cloud storage).
- **Image displays.** Mostly color flat panels today. For medical diagnosis, specialized calibrated monitors are used, and for stereo or virtual reality, head-mounted displays.
- **Hardcopy devices.** Laser and inkjet printers, film cameras, and heat-sensitive printers. Film still offers very high resolution, and paper remains the medium of choice for written material and many reports.
- **Networking and the cloud.** Practically every system is now connected. Transmitting images is a bandwidth problem, which is why compression (Chapter 8) is a core topic. Cloud services move both storage and heavy computation (for example, training deep models) off the device.

How large is "large"? An image with M rows, N columns, C channels and k bits per value needs

$$b = M \times N \times C \times k \text{ bits}$$

before compression, where $C = 1$ for grayscale and $C = 3$ for RGB color.

```
def raw_size_mb(height, width, channels=1, bits=8, frames=1):
    return height * width * channels * bits * frames / 8 / 1e6

print(f"{raw_size_mb(512, 512):.2f} MB   - this chapter's 8-bit test photo")         # 0.26
print(f"{raw_size_mb(3000, 4000, 3):.0f} MB   - one 12-megapixel RGB photo")         # 36
print(f"{raw_size_mb(512, 512, bits=16, frames=300):.0f} MB  - a 300-slice 16-bit CT
 ↪  volume")  # 157
print(f"{raw_size_mb(2160, 3840, 3, frames=30*60):.0f} MB - one minute of uncompressed 4K
 ↪  video")  # 44790
```

One minute of uncompressed 4K video at 30 frames per second is about 45 gigabytes. Without compression, streaming video and image-heavy web pages would be impractical.

Modern view

The textbook's view of the field, a pipeline of well-understood, hand-designed steps, is still the right way to *learn* image processing. Two developments since about 2010 have changed how the field is *practiced*: computation has moved into the image formation itself, and learned models have replaced or augmented many hand-designed steps. The surveys below are good entry points.

Surveys on computational imaging and imaging beyond the visible

- **Mait, Euliss and Athale, "Computational imaging" (2018)** [8] review two decades of systems in which optics and computation are designed together, so that the sensor records data that is not a recognizable picture until it is processed. Their survey spans topics such as spatial light modulators, synthetic aperture radar and computational microscopy. The main takeaway is that the boundary between "acquisition" and "processing" in Figure 1.6 is now a design choice: CT, MRI and radar were early examples, and phones have made the approach mainstream.
- **Delbracio, Kelly, Brown and Milanfar, "Mobile computational photography: a tour" (2021)** [10] trace how phone cameras overcame their tiny sensors and lenses with algorithms: capturing a burst of frames and merging them, aggressive noise reduction, and multi-frame super-resolution. Their tour shows that a modern phone photo is the output of a long processing pipeline, and it is a useful companion to almost every chapter in this series.
- **Gallego et al., "Event-based vision: a survey" (2020)** [11] cover a sensor that breaks the frame-based model of Figure 1.1 entirely. An *event camera* pixel fires asynchronously whenever its own brightness changes, giving microsecond timing and very high dynamic range. The survey shows how classic tasks such as feature detection, optical flow and image reconstruction must be rethought when the input is a stream of events rather than a grid of numbers.
- **Wang, Ye and De Man, "Deep learning for tomographic image reconstruction" (2020)** [12] review how neural networks entered CT, MRI, PET and related modalities, the reconstruction problems that launched the field in the 1970s. They cover networks that clean up conventional reconstructions, physics-informed approaches that keep the measurement model in the loop, direct learned reconstruction, and generative models, with goals such as faster scans, lower radiation dose and better image quality. Their outlook stresses that large, well-curated datasets will be central to further progress.

How deep learning reshaped the processing pipeline

The broad shift is summarized in the review by **LeCun, Bengio and Hinton (2015)** [13]: deep networks learn representations of data at several levels of abstraction directly from examples, instead of relying on features designed by hand. For image processing, this has played out differently at each level of the continuum in Figure 1.2.

At the **high level**, the change has been nearly complete. Classification and recognition (Chapter 13) are now dominated by learned models; Chapters 12 and 13 explain the hand-designed features and classifiers that came before, which remain useful when data is scarce or interpretability matters.

At the **low level**, the clearest case study is the camera's own processing pipeline. A conventional ISP is a chain of separately tuned stages that turn raw sensor values into a finished photo: black-level correction, demosaicing, denoising, white balance, color correction, tone mapping and compression. **Ramanath et al. (2005)** [14] give a clear overview of this pipeline and its trade-offs, and it is a good reference to keep next to Chapters 3, 5 and 6.

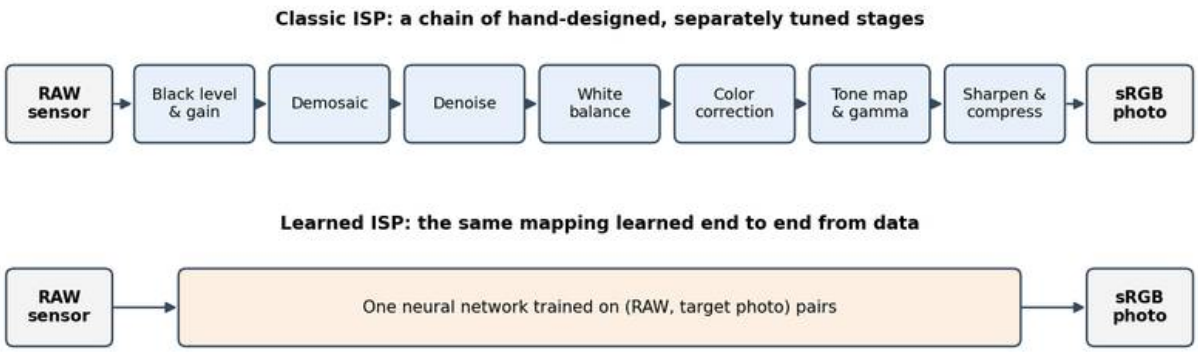


Figure 1.8 — A classic camera ISP (top) is a chain of hand-designed stages, each covered somewhere in this series. A learned ISP (bottom) replaces the chain, or parts of it, with one network trained on pairs of raw data and target photos.

Deep learning has attacked this pipeline from several directions:

- **Learning a hard stage end to end.** Chen et al. (2018) trained a fully convolutional network to map short-exposure raw images taken in very low light directly to clean photos, using long-exposure shots of the same scenes as targets [15]. A traditional pipeline amplifies noise badly in this regime.
- **Making training data realistic.** Brooks et al. (2018) showed how to "unprocess" ordinary photos, inverting the steps of the camera pipeline to synthesize realistic raw data, so that denoising networks trained on it work on real sensor output [16]. This is a good example of classic pipeline knowledge making a learned model better.
- **Replacing the whole ISP.** Ignatov, Van Gool and Timofte (2020) proposed PyNET, a pyramid-shaped convolutional network that maps raw data from a smartphone sensor directly to a high-quality photo, learning demosaicing, denoising, color and tone in a single model [17]. They trained it on paired raw phone images and photos from a high-end DSLR camera of the same scenes.
- **Surveying the space.** da Silva et al. (2023) survey deep learning methods for image signal processing, comparing approaches that replace individual stages with those that replace the entire ISP with one network [18].

What did **not** change? The mathematics of image formation in Chapters 2, 4 and 5 still determines what any method, learned or not, can recover. Learned models are trained and evaluated with the quality measures, transforms and color spaces defined in the classic chapters. And many deployed systems are hybrids: a learned denoiser inside an otherwise conventional pipeline, or a physics-based reconstruction refined by a network [12]. Understanding the classic steps is what lets you see which part of such a system is doing what, and why it fails when it does.

Key takeaways

- A digital image is a function $f(x, y)$ sampled on an $M \times N$ grid and quantized to $L = 2^k$ levels; in code it is simply an array.
- Image processing spans a continuum: low-level (image to image), mid-level (image to attributes) and high-level (attributes to meaning). Computer vision lives at the high end.
- The field grew from telegraph picture transmission (1920s), computer correction of space-probe images (1960s) and computed medical images such as CT (1970s).
- Images come from every part of the EM spectrum, from gamma rays to radio waves, and from non-EM sources such as ultrasound, seismic waves and electron beams. The physics of the source determines noise and resolution, and so which tools you need.
- The fundamental steps split into steps whose outputs are images (acquisition through compression) and steps whose outputs are attributes (morphology through classification), all guided by domain knowledge. Chapters 2–13 follow this roadmap.
- A working system combines sensors, specialized hardware, a computer, software, storage, displays, hardcopy and networking.
- Deep learning has largely taken over high-level recognition and is reshaping low-level steps such as the camera ISP, but the classic models of image formation remain the foundation that learned methods build on.

Exercises

1. **Storage budget.** A microscope records a time-lapse of 16-bit grayscale images of size 2048×2048, one every 30 seconds for 48 hours. How much uncompressed storage does the experiment need? How many frames would fit on a 1 TB disk?

   Hint

   Each frame is $2048 \times 2048 \times 16$ bits $= 8{,}388{,}608$ bytes, about 8.4 MB. There are $48 \times 120 = 5760$ frames, so about 48 GB in total. A 1 TB disk holds roughly $10^{12}/8.39 \times 10^6 \approx 119{,}000$ frames.

2. **Place it on the continuum.** Classify each task as low-, mid- or high-level, and justify your answer: (a) removing salt-and-pepper noise from a scanned page; (b) outlining every cell in a microscope image and reporting each cell's area; (c) deciding whether a chest X-ray is normal; (d) brightening a dark photo.

   Hint

   Ask what comes *out*. An image out means low level ((a), (d)). Regions or measurements out means mid level ((b)). A judgment about the scene means high level ((c)).

3. **Why X-rays and not light?** Using $E = hc/\lambda$, compute the photon energy of a 0.05 nm X-ray and of 600 nm orange light. Explain in two sentences why the first is used to image bones and the second is not.

Hint

About 24.8 keV versus about 2.07 eV, a ratio of roughly 12,000. Visible photons are absorbed or scattered within millimeters of tissue; high-energy X-ray photons pass through soft tissue, and bones absorb a larger share of them, which produces contrast.

4. **How many gray levels?** Using the `quantize` function from this chapter, plot the mean absolute error between `data.camera()` and its quantized version for $L = 2, 3, \ldots, 32$. Then look at the images. Around which L do false contours stop being obvious to you? Is the error curve a good predictor of what you see?

Hint

For uniform quantization the error falls roughly as $1/L$. Visible banding depends on the smooth areas of the image (here, the sky) more than on the average error, which is one reason Chapter 2 treats perceived quality separately from numerical error.

5. **Design a system.** A factory wants to reject cracked ceramic tiles moving on a conveyor belt at two tiles per second. List the components of Figure 1.7 you would need, and for each of the fundamental steps of Figure 1.6, say whether you would use it and why.

Hint

You need a sensor with controlled lighting, enough compute to process two images per second, and a link to the reject mechanism. Likely steps: acquisition, enhancement (contrast), perhaps morphology, segmentation of crack-like structures, features (crack length), and a classification rule. Color, wavelets and compression are probably unnecessary, unless images are archived for audit.

6. **Learned or classic?** Pick one stage of the classic ISP in Figure 1.8. Give one argument for replacing it with a learned model and one argument for keeping the hand-designed version.

Hint

For learning: a network can adapt to the actual sensor noise and scene statistics. For keeping it: a hand-designed stage is predictable, cheap to run, easy to tune and does not need paired training data. Several references in the Modern view discuss exactly this trade-off.

References

1. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018, Ch. 1. publisher page
2. K. Kobayashi, "Birth of a Digital Phototelegraph — the Bartlane System," *Journal of the Institute of Image Electronics Engineers of Japan*, vol. 31, no. 2, pp. 244–249, 2002. J-STAGE
3. NASA, "First Image of the Moon Taken by a U.S. Spacecraft" (Ranger 7, PIA02975), NASA Photojournal. NASA

4. G. N. Hounsfield, "Computerized transverse axial scanning (tomography): Part 1. Description of system," *British Journal of Radiology*, vol. 46, no. 552, pp. 1016–1022, 1973. doi:10.1259/0007-1285-46-552-1016
5. L. A. Shepp and B. F. Logan, "The Fourier reconstruction of a head section," *IEEE Transactions on Nuclear Science*, vol. 21, no. 3, pp. 21–43, 1974. doi:10.1109/TNS.1974.6499235
6. P. C. Lauterbur, "Image formation by induced local interactions: examples employing nuclear magnetic resonance," *Nature*, vol. 242, pp. 190–191, 1973. doi:10.1038/242190a0
7. R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed., Springer, 2022. book site
8. J. N. Mait, G. W. Euliss and R. A. Athale, "Computational imaging," *Advances in Optics and Photonics*, vol. 10, no. 2, pp. 409–483, 2018. doi:10.1364/AOP.10.000409
9. The Event Horizon Telescope Collaboration, "First M87 Event Horizon Telescope Results. I. The Shadow of the Supermassive Black Hole," *The Astrophysical Journal Letters*, vol. 875, no. 1, 2019. doi:10.3847/2041-8213/ab0ec7
10. M. Delbracio, D. Kelly, M. S. Brown and P. Milanfar, "Mobile Computational Photography: A Tour," arXiv:2102.09000, 2021. arXiv
11. G. Gallego et al., "Event-based Vision: A Survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2020. arXiv
12. G. Wang, J. C. Ye and B. De Man, "Deep learning for tomographic image reconstruction," *Nature Machine Intelligence*, vol. 2, no. 12, pp. 737–748, 2020. doi:10.1038/s42256-020-00273-z
13. Y. LeCun, Y. Bengio and G. Hinton, "Deep learning," *Nature*, vol. 521, pp. 436–444, 2015. doi:10.1038/nature14539
14. R. Ramanath, W. E. Snyder, Y. Yoo and M. S. Drew, "Color image processing pipeline," *IEEE Signal Processing Magazine*, vol. 22, no. 1, pp. 34–43, 2005. doi:10.1109/MSP.2005.1407713
15. C. Chen, Q. Chen, J. Xu and V. Koltun, "Learning to See in the Dark," *CVPR*, 2018. arXiv
16. T. Brooks, B. Mildenhall, T. Xue, J. Chen, D. Sharlet and J. T. Barron, "Unprocessing Images for Learned Raw Denoising," arXiv:1811.11127, 2018. arXiv
17. A. Ignatov, L. Van Gool and R. Timofte, "Replacing Mobile Camera ISP with a Single Deep Learning Model," arXiv:2002.05509, 2020. arXiv
18. M. H. M. da Silva et al., "ISP meets Deep Learning: A Survey on Deep Learning Methods for Image Signal Processing," arXiv:2305.11994, 2023. arXiv

IP 02 • Digital Image Fundamentals

How eyes and cameras turn light into numbers, how sampling and quantization shape a digital image, and the basic math every later chapter builds on.

Online edition: https://blog.shyandram.dev/en/learn/image-processing/dip-02-fundamentals/

What you'll learn

- How the human eye senses light, and why what we *see* is not always what is *there* (brightness adaptation, the Weber ratio, Mach bands, simultaneous contrast)
- Where images come from physically: the electromagnetic spectrum, sensors, and the illumination–reflectance model $f = i \cdot r$
- What sampling and quantization do, how they set spatial and intensity resolution, and how nearest, bilinear and bicubic interpolation resize an image
- The vocabulary of pixel relationships: neighbors, adjacency, connectivity, regions, boundaries, and three distance measures
- The mathematical toolbox used in every later chapter: elementwise vs matrix operations, linearity, arithmetic, set and logical operations, geometric transforms, image transforms, and histograms as probabilities

The big picture

Chapter 1 told us *what* digital image processing is. This chapter explains *what a digital image actually is*, starting from the light that leaves a scene and ending with a grid of integers in memory. Along the way it defines the words — pixel, neighbor, resolution, linear operator — that the rest of the series uses without stopping to explain.

> **Explain like I'm five**
>
> Imagine copying a painting onto graph paper with a box of crayons. First you decide how many squares the graph paper has: more squares means you can draw smaller details. That is **sampling**. Then you decide how many crayon colors you are allowed to use: more crayons means smoother shading. That is **quantization**. A digital image is just that graph paper, with a number written in each square saying which crayon you picked. Everything else in image processing is a recipe for changing those numbers.

Two things make this chapter more than bookkeeping. First, the final consumer of most images is a human, so we need to know where the eye is easily fooled and where it is sharp. Second, every choice made when an image is captured — how many samples, how many levels, which interpolation — leaves a fingerprint that later algorithms must live with. Deep networks did

not remove these choices; they mostly moved them into training data and loss functions, as the Modern view section shows.

Elements of visual perception

In plain words: your eye is a camera made of jelly and nerve cells. It is extremely good at seeing *differences* and *edges*, and surprisingly bad at judging absolute brightness.

Structure of the eye and image formation

Light enters through the cornea, passes the pupil (whose size the iris controls), and is focused by the lens onto the **retina**, a thin layer of light receptors at the back of the eye [1]. The lens is flexible: ciliary muscles change its curvature, which changes its focal length so that near and far objects can both be brought into focus. The retinal image is real and inverted, just like the image on a camera sensor.

The retina contains two kinds of receptor [1]:

- **Cones** — a few million per eye, packed most densely in a small central pit called the **fovea**. They work in bright light (*photopic* vision), come in three types with different spectral sensitivities, and give us color and fine detail. Many cones have their own nerve connection, which is why foveal vision is sharp.
- **Rods** — many times more numerous, spread across the retina, absent from the center of the fovea. They work in dim light (*scotopic* vision), have no color discrimination, and many rods share one nerve ending, trading detail for sensitivity. This is why objects seen by moonlight look colorless and soft.

The point where the optic nerve leaves the eye has no receptors at all: the **blind spot**. You do not notice it because the brain fills in the gap — an early hint that perception is an active reconstruction, not a recording.

Brightness adaptation and discrimination

The visual system copes with an enormous range of light levels, from starlight to sunlight. It does this by **brightness adaptation**: at any moment it operates over a much narrower window centered on the current adaptation level, and that window slides as the overall light changes [1]. Within the window, subjective brightness grows roughly with the *logarithm* of the physical light intensity, not linearly.

How small a change can we notice? Put a uniform field of intensity I in front of an observer and flash a small spot of intensity $I + \Delta I$. The increment that is noticed half of the time is ΔI_c. The ratio

$$W = \frac{\Delta I_c}{I}$$

is the **Weber ratio**, where I is the background intensity and ΔI_c is the just-noticeable increment. A *small* W means good discrimination (a small relative change is visible). Over a wide middle range of intensities W is roughly constant, which is Weber's law: what we perceive is the *relative* change, not the absolute one. At low intensities, where rods dominate, W is large and discrimination is poor [1].

This has a direct engineering consequence. Because we perceive relative changes, equally spaced *physical* gray levels are not equally spaced *perceptually*. That is one reason images are stored with a nonlinear (gamma) encoding, and one reason 8 bits are usually enough for display.

Mach bands, simultaneous contrast and illusions

Perceived brightness is not a simple function of intensity. Two classic demonstrations make this obvious [1]:

- **Mach bands.** Show a staircase of flat gray stripes. Each stripe is physically uniform, yet observers report a thin darker band on the dark side of every edge and a thin brighter band on the bright side. The visual system exaggerates edges, a behavior consistent with lateral inhibition between neighboring receptive fields — essentially a built-in sharpening filter.
- **Simultaneous contrast.** A mid-gray square looks lighter on a dark background and darker on a light one, although the square is identical in both cases. Perceived brightness depends on the surround.

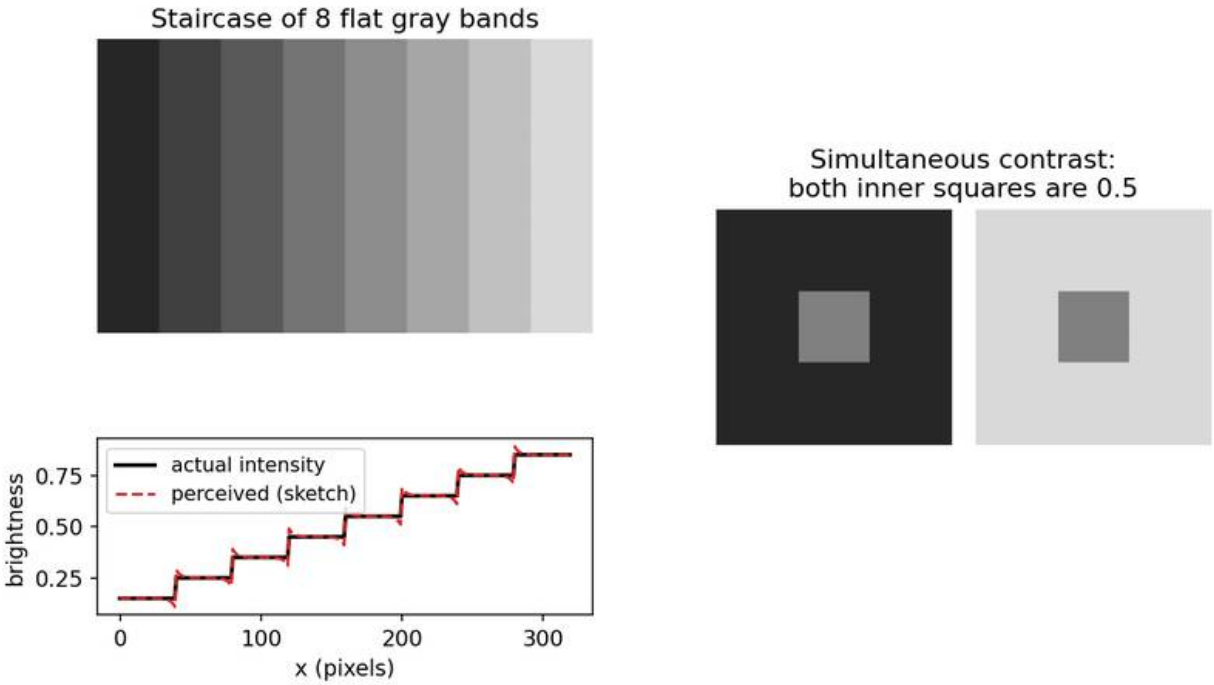


Figure 2.1 — Left: a staircase of uniform bands; the dashed curve sketches the over- and undershoot observers report at each edge (Mach bands). Right: simultaneous contrast — both inner squares have intensity 0.5.

Optical illusions in general — contours that are not drawn, lines of equal length that look unequal — show that the eye fills in and reinterprets. For image processing this matters twice: an algorithm that minimizes squared error may still produce images that *look* wrong, and an artifact that is numerically small may be very visible if it sits on an edge. This is the starting point of perceptual quality metrics, discussed in the Modern view.

Light and the electromagnetic spectrum

In plain words: light is a wave of energy. Visible light is a thin slice of a much wider spectrum that also includes radio, microwaves, infrared, ultraviolet, X-rays and gamma rays — and images can be made from any of them.

Wavelength, frequency and energy

Electromagnetic radiation can be described as a sinusoidal wave or as a stream of massless particles called photons [1]. The two descriptions are linked by

$$\lambda = \frac{c}{\nu}, \qquad E = h\nu,$$

where λ is the wavelength (meters), ν is the frequency (hertz), $c \approx 3 \times 10^8$ m/s is the speed of light, E is the energy of one photon (joules), and h is Planck's constant. Shorter wavelength means higher frequency and higher energy per photon. That is why gamma rays and X-rays can pass through tissue, while radio waves need large antennas to be detected.

The visible band spans roughly 400 nm (violet) to 700 nm (red). An object's perceived color is determined by which wavelengths it reflects: a green leaf reflects mostly in the middle of the band and absorbs the rest. Light with no color, only intensity, is called **achromatic** or **monochromatic**, and its intensity is called the **gray level**.

Radiance, luminance and brightness

Three words are easily confused [1]:

- **Radiance** — the total energy flowing from the light source, measured in watts. It is purely physical.
- **Luminance** — the amount of that energy an observer *perceives*, measured in lumens. It weights radiance by the eye's spectral sensitivity: an infrared source can have high radiance but almost zero luminance.
- **Brightness** — the subjective sensation of intensity. It cannot be measured directly and, as the previous section showed, depends on adaptation and surroundings.

A sensor's sensitivity curve plays the role of the eye's: an image is only as good as the match between the band a sensor detects and the wavelength the object emits or reflects. To image an object, the wavelength must also be comparable to or smaller than the object's size — one reason

electron microscopes, with very short effective wavelengths, resolve far smaller structures than light microscopes.

Image sensing and acquisition

In plain words: a sensor is a tiny light meter that turns light into voltage. Put many of them in a row or a grid and you have a scanner or a camera.

Single, line and array sensors

Every imaging device combines an **illumination source**, the **scene**, and a **sensor** whose material responds to the energy of interest [1]. The sensor's output voltage is digitized to give one number. Sensors are arranged in three ways:

- **Single sensor.** One detector (for example a photodiode) plus mechanical motion in two directions sweeps out an image one sample at a time. Slow, but cheap and very precise; used in some high-precision scanners.
- **Line (strip) sensor.** A row of detectors captures one line at a time, and motion perpendicular to the strip provides the second dimension. Flatbed scanners and airborne push-broom imagers work this way. A *ring* of sensors around a rotating source is the geometry of computed tomography (CT), where the image is reconstructed rather than read out directly.
- **Array sensor.** A 2-D grid of detectors, such as a CCD or CMOS chip, captures the whole image at once. This is the digital camera. A lens focuses the scene onto the array, each detector integrates light over an exposure time, and the readout electronics produce one value per detector.

Color cameras usually place a mosaic of red, green and blue filters over the array, so each detector measures only one color; the missing two values per pixel are interpolated (demosaicked) in the camera's image-processing pipeline [8].

A simple image formation model

We write a monochrome image as a 2-D function $f(x, y)$ whose value is proportional to the energy arriving at (x, y). Because it comes from physical energy, $0 \leq f(x, y) < \infty$. For reflected light, it helps to split f into two factors [1]:

$$f(x, y) = i(x, y)\, r(x, y), \qquad 0 < i(x, y) < \infty, \qquad 0 < r(x, y) < 1,$$

where $i(x, y)$ is the **illumination** (how much light falls on the scene point) and $r(x, y)$ is the **reflectance** (what fraction of it the surface reflects, from total absorption near 0 to total reflection near 1). Illumination depends on the light source; reflectance is a property of the object. For transmission imaging such as X-rays, a **transmissivity** takes the place of reflectance.

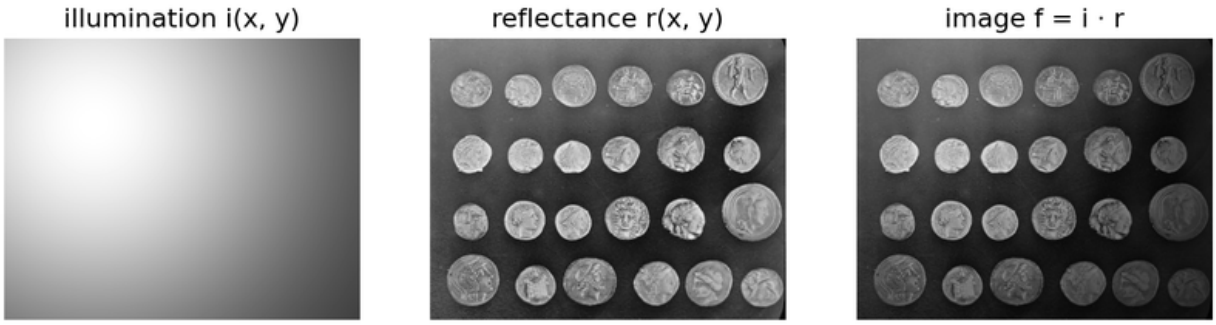


Figure 2.2 — The illumination–reflectance model. A smooth illumination field (left) multiplies the reflectance (center) to give the recorded image (right).

```
import numpy as np
from skimage import data, img_as_float

r = img_as_float(data.coins())          # reflectance in [0, 1]
h, w = r.shape
yy, xx = np.mgrid[0:h, 0:w]
i = 0.2 + 0.8 * xx / (w - 1)            # illumination ramp: dim left, bright right
f = i * r                               # what the sensor records
```

The model is simple but useful. Illumination usually varies slowly across a scene, while reflectance changes abruptly at object edges. Later chapters exploit this: homomorphic filtering (Chapter 4) takes $\log f = \log i + \log r$ to separate the two, and shading correction (later in this chapter) divides out a known i.

Image sampling and quantization

In plain words: a camera measures a continuous scene at a finite number of points (sampling) and rounds each measurement to one of a finite number of levels (quantization).

Basic concepts

A sensor's output is a continuous voltage waveform over a continuous plane. To make it digital, both the coordinates and the amplitude must become discrete [1]:

- **Sampling** digitizes the *coordinates*: we keep values only at a grid of positions.
- **Quantization** digitizes the *amplitude*: each kept value is mapped to the nearest of a finite set of levels.

With an array sensor, the number and spacing of detectors fix the sampling; the analog-to-digital converter fixes the quantization. How finely we must sample depends on the image content:

by the sampling theorem, a band-limited signal can be reconstructed exactly if it is sampled at more than twice its highest frequency. Chapter 4 develops this properly; Unser's review gives the modern, spline-based view of sampling and reconstruction [2].

Representing digital images

After sampling and quantization, an image is a matrix of M rows and N columns:

$$f = \begin{bmatrix} f(0,0) & f(0,1) & \cdots & f(0,N-1) \\ f(1,0) & f(1,1) & \cdots & f(1,N-1) \\ \vdots & \vdots & \ddots & \vdots \\ f(M-1,0) & f(M-1,1) & \cdots & f(M-1,N-1) \end{bmatrix}$$

Each element is a **pixel** (picture element). Following the book's convention [1], x indexes rows and y indexes columns, with the origin at the top-left corner. (Many libraries, including OpenCV's point APIs, use $(x, y) = (\text{column}, \text{row})$. Always check.)

The number of intensity levels is normally a power of two, $L = 2^k$, so values lie in $[0, L-1]$. Storing the image takes

$$b = M \times N \times k \text{ bits},$$

where k is the number of bits per pixel. A 512×512 image with $k = 8$ needs 262,144 bytes. The **dynamic range** of an imaging system is the ratio of the largest measurable intensity (set by saturation) to the smallest (set by noise); the **contrast** of a particular image is the difference between its highest and lowest intensities.

Spatial and intensity resolution

Spatial resolution is the size of the smallest discernible detail. It is meaningful only with units: line pairs per millimeter, or dots per inch (dpi) in printing. Saying an image is "1024 × 1024" says nothing about resolution until we know the physical area it covers. **Intensity resolution** is the smallest discernible change in level, usually stated as the number of bits k.

```
import numpy as np
from skimage import data

img = data.camera()                      # 512 x 512, uint8 (k = 8)
small = img[::4, ::4]                    # keep every 4th sample -> 128 x 128

def quantize(x, bits):
    step = 2 ** (8 - bits)
    return (x // step) * step + step // 2 # map to the centre of each bin

q3 = quantize(img, 3)                    # 8 gray levels
print(small.shape, np.unique(q3).size)   # (128, 128) 8
```

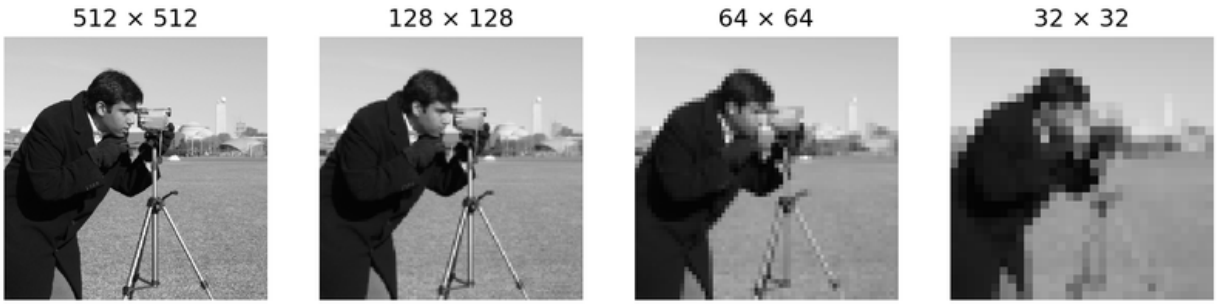


Figure 2.3 — Reducing spatial resolution. Each version is displayed at the same size, so fewer samples show up as larger, blockier pixels.

Figure 2.4 — Reducing intensity resolution. Around 8 levels, smooth regions such as the sky develop false contours; at 2 levels only a silhouette remains.

Two lessons follow. Reducing samples first destroys fine texture (grass, hair), then edges. Reducing levels first damages *smooth* regions: the sky in Figure 2.4 breaks into visible steps called **false contouring**, exactly where the Weber ratio says the eye is most sensitive. Images with a lot of detail tolerate fewer gray levels than smooth ones, because busy texture masks the steps [1].

> **Interactive demonstration (online)**: open the online edition of this tutorial to use this demonstration.Open demonstration

Image interpolation

Interpolation estimates values at positions where we have no samples. It is used whenever an image is resized, rotated, warped or registered [1], [3], [4].

Nearest neighbor copies the value of the closest sample. It is fast and keeps the original values, but it makes edges blocky and, when shrinking, can drop thin structures entirely.

Bilinear interpolation uses the four nearest samples. For a point at fractional offsets $a, b \in [0, 1)$ from the top-left sample f_{00},

$$v = (1-a)(1-b)\,f_{00} + a(1-b)\,f_{01} + (1-a)b\,f_{10} + ab\,f_{11},$$

where f_{01}, f_{10} and f_{11} are the right, lower and lower-right neighbors. Equivalently, $v(x,y) = c_1 x + c_2 y + c_3 xy + c_4$, with the four coefficients fixed by the four samples. The result is continuous but slightly blurred.

```
import numpy as np

def bilinear(img, x, y):
    """Sample img (H, W) at real-valued column x, row y."""
    x0, y0 = int(np.floor(x)), int(np.floor(y))
    x1, y1 = min(x0 + 1, img.shape[1] - 1), min(y0 + 1, img.shape[0] - 1)
    a, b = x - x0, y - y0
    top = (1 - a) * img[y0, x0] + a * img[y0, x1]
    bot = (1 - a) * img[y1, x0] + a * img[y1, x1]
    return (1 - b) * top + b * bot

print(bilinear(np.array([[0., 10.], [20., 30.]]), 0.5, 0.5))   # 15.0
```

Bicubic interpolation uses the 16 nearest samples and fits $v(x,y) = \sum_{i=0}^{3}\sum_{j=0}^{3} a_{ij} x^i y^j$. In practice it is implemented as a separable convolution with a piecewise-cubic kernel. Keys' widely used kernel is [3]

$$u(s) = \begin{cases} (\alpha+2)|s|^3 - (\alpha+3)|s|^2 + 1, & |s| \le 1, \\ \alpha|s|^3 - 5\alpha|s|^2 + 8\alpha|s| - 4\alpha, & 1 < |s| < 2, \\ 0, & \text{otherwise}, \end{cases}$$

where s is the distance (in samples) from the interpolation point to a sample and α is a free parameter. Keys showed that $\alpha = -\tfrac{1}{2}$ gives the best approximation order for this family; libraries differ in the value they actually use. The negative lobes of u sharpen edges but can cause slight overshoot (ringing) next to strong edges.

```
import cv2
from skimage import data

crop = data.camera()[150:214, 230:294]    # 64 x 64
up = {name: cv2.resize(crop, None, fx=4, fy=4, interpolation=flag)
      for name, flag in [("nearest", cv2.INTER_NEAREST),
                         ("bilinear", cv2.INTER_LINEAR),
                         ("bicubic", cv2.INTER_CUBIC)]}
```

Figure 2.5 — Four-times enlargement of the same crop. Nearest neighbor is blocky; bilinear is smooth but soft; bicubic keeps edges a little crisper.

Drag the slider to compare nearest-neighbor and bicubic upsampling of a 96 × 96 crop enlarged to 384 × 384. Look at the camera's edges and the face profile.

Nearest

Bicubic

Cameraman crop upsampled four times with nearest-neighbor and with bicubic interpolation

No interpolation method can create detail that sampling threw away. Thévenaz, Blu and Unser's review [4] makes the point precisely: the quality of an interpolator is governed by its approximation order and by how closely its kernel approaches the ideal (sinc) reconstructor, and higher-order B-spline methods outperform classic cubic convolution at similar cost. Learned super-resolution, discussed in the Modern view, goes further by *hallucinating* plausible detail from training data.

Basic relationships between pixels

In plain words: before we can talk about objects in an image, we need rules for which pixels "touch" each other and how far apart two pixels are.

Neighbors of a pixel

A pixel p at (x, y) has four horizontal and vertical neighbors,

$$N_4(p) = \{(x+1, y),\ (x-1, y),\ (x, y+1),\ (x, y-1)\},$$

four diagonal neighbors,

$$N_D(p) = \{(x+1, y+1),\ (x+1, y-1),\ (x-1, y+1),\ (x-1, y-1)\},$$

and the eight neighbors $N_8(p) = N_4(p) \cup N_D(p)$ [1]. At the image border some neighbors fall outside the image; how we treat them (ignore, pad, reflect) is a choice we meet again in filtering.

Adjacency, connectivity, regions and boundaries

Let V be the set of intensity values that count as "similar" — for a binary image, $V = \{1\}$; for a gray image, perhaps all values between 100 and 120. Two pixels p and q with values in V are [1]:

- **4-adjacent** if $q \in N_4(p)$;
- **8-adjacent** if $q \in N_8(p)$;
- **m-adjacent** (mixed) if $q \in N_4(p)$, *or* $q \in N_D(p)$ and the set $N_4(p) \cap N_4(q)$ contains no pixel whose value is in V.

Why m-adjacency? With 8-adjacency, a diagonal step and a two-step "around the corner" route can both connect the same pair of pixels, so paths become ambiguous. m-adjacency keeps a diagonal link only when no 4-connected route exists, which removes the redundant loops (Figure 2.6, right).

Figure 2.6 — Neighborhoods of a pixel p, and m-adjacency on a binary patch: the dashed diagonal is not an m-adjacency because the two pixels already share a 4-neighbor in V.

A **digital path** from p to q is a sequence of distinct pixels $(x_0, y_0), \ldots, (x_n, y_n)$ with consecutive pixels adjacent; n is the length, and the path is **closed** if the first and last pixels coincide. Depending on the adjacency used we speak of 4-, 8- or m-paths. For a subset S of pixels, p and q are **connected in** S if a path between them lies entirely in S. The set of pixels connected to p in S is a **connected component**; if S has only one, it is a **connected set**.

A connected set is called a **region**. Two regions are adjacent if their union is connected. The **boundary** (or border) of a region R is the set of pixels in R that have at least one neighbor outside R. Note the difference between a boundary and an **edge**: a boundary is a global, closed property of a region, while an edge is a local intensity discontinuity that may not close at all (Chapter 10).

The choice of adjacency changes the answer to simple questions such as "how many objects are there?":

```
import numpy as np
from skimage.measure import label

b = np.array([[1, 0, 0],
              [0, 1, 0],
              [0, 0, 1]])
print(label(b, connectivity=1).max(),     # 4-connectivity: 3 objects
      label(b, connectivity=2).max())     # 8-connectivity: 1 object
```

A classic consequence: if the foreground uses 8-connectivity, the background should use 4-connectivity (and vice versa), otherwise a closed digital curve may fail to separate its inside from its outside. This is a basic result of digital topology [1].

Distance measures

For pixels $p = (x, y)$, $q = (u, v)$ and z, a function D is a **distance metric** if $D(p, q) \geq 0$ with equality only when $p = q$, $D(p, q) = D(q, p)$, and $D(p, z) \leq D(p, q) + D(q, z)$. Three metrics are standard [1], [5]:

$$
\begin{aligned}
D_E(p, q) &= \sqrt{(x-u)^2 + (y-v)^2} && \text{(Euclidean)} \\
D_4(p, q) &= |x-u| + |y-v| && \text{(city-block)} \\
D_8(p, q) &= \max(|x-u|,\ |y-v|) && \text{(chessboard)}
\end{aligned}
$$

The pixels within a fixed D_E of p form a disk; within a fixed D_4, a diamond; within a fixed D_8, a square. The pixels with $D_4 = 1$ are exactly $N_4(p)$, and those with $D_8 = 1$ are exactly $N_8(p)$. D_4 and D_8 depend only on coordinates, not on pixel values; a distance *along* m-paths, by contrast, does depend on the values because the paths do.

```
import numpy as np
p, q = np.array([2, 3]), np.array([5, 7])
d = np.abs(p - q)
print("D_E =", np.hypot(*d), " D_4 =", d.sum(), " D_8 =", d.max())   # 5.0, 7, 4
```

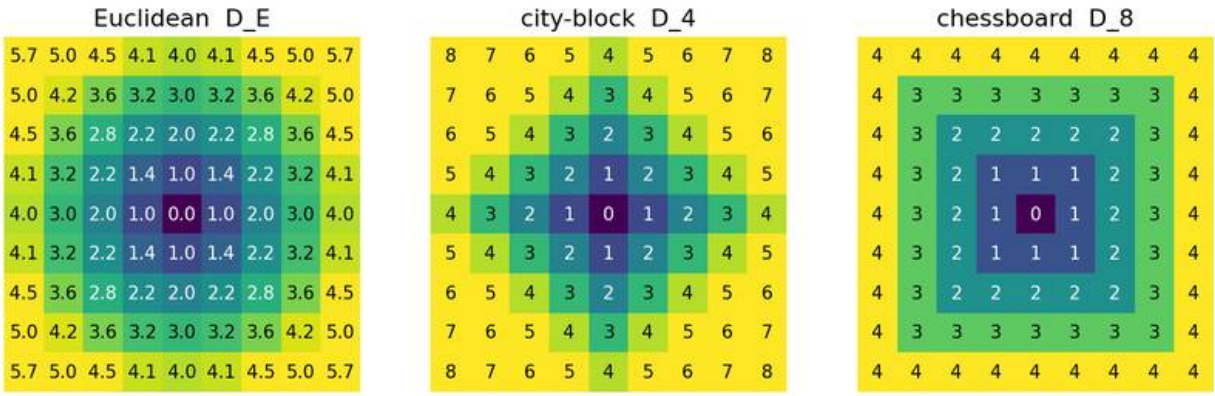


Figure 2.7 — Distance from the center pixel under the three metrics. Equal-distance contours are circles, diamonds and squares.

Rosenfeld and Pfaltz [5] introduced efficient two-pass algorithms to compute such distances to the nearest object pixel for every pixel at once — the **distance transform**, still used for skeletons, morphology and shape matching (Chapter 9).

Introduction to the mathematical tools used in DIP

In plain words: an image is a matrix, so most image processing is arithmetic on matrices. This section lists the operations we will use again and again, and the one property — linearity — that decides which tools apply.

Elementwise versus matrix operations

An **elementwise** (array) operation acts pixel by pixel. For two 2×2 images the elementwise product is

$$\begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \odot \begin{bmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{bmatrix} = \begin{bmatrix} a_{11}b_{11} & a_{12}b_{12} \\ a_{21}b_{21} & a_{22}b_{22} \end{bmatrix},$$

whereas the matrix product follows row-times-column rules. Unless stated otherwise, image operations — including "multiplying two images" — are elementwise [1]. In NumPy, `A * B` is elementwise and `A @ B` is the matrix product.

Linear versus nonlinear operations

Consider an operator $\mathcal{H}$ that maps an input image f to an output $g = \mathcal{H}[f]$. It is **linear** if

$$\mathcal{H}[af_1 + bf_2] = a\,\mathcal{H}[f_1] + b\,\mathcal{H}[f_2]$$

for all images f_1, f_2 and all constants a, b. The two parts of this property are **additivity** (the output of a sum is the sum of outputs) and **homogeneity** (scaling the input scales the output). Summing all pixels is linear; taking the maximum is not:

```
import numpy as np
rng = np.random.default_rng(0)
f1, f2 = rng.random((4, 4)), rng.random((4, 4))
a, b = 2.0, -3.0
print(np.isclose(np.sum(a*f1 + b*f2), a*np.sum(f1) + b*np.sum(f2)))   # True
print(np.isclose(np.max(a*f1 + b*f2), a*np.max(f1) + b*np.max(f2)))   # False
```

Linearity matters because linear operators are fully described by their response to simple inputs, and a large body of theory (convolution, the Fourier transform, Chapters 3–5) applies to them. Nonlinear operators such as the median filter or morphology can do things linear ones cannot, but are harder to analyze.

Arithmetic operations

Sum, difference, product and quotient of two images of the same size are all elementwise [1]. Three uses stand out.

Averaging noisy images. Suppose we capture K images of a static scene, $g_i(x, y) = f(x, y) + \eta_i(x, y)$, where f is the clean image and η_i is noise that is uncorrelated between frames, has zero mean and variance σ_η^2. The average $\bar{g} = \frac{1}{K}\sum_{i=1}^{K} g_i$ satisfies

$$E\{\bar{g}(x,y)\} = f(x,y), \qquad \sigma^2_{\bar{g}}(x,y) = \frac{1}{K}\,\sigma^2_\eta(x,y),$$

so the noise standard deviation falls as $1/\sqrt{K}$. Astronomers stack exposures for this reason, and smartphone "night modes" do the same after aligning a burst of frames [7].

```
import numpy as np
from skimage import data, img_as_float

g = img_as_float(data.camera())
rng = np.random.default_rng(1)
for K in (1, 4, 16, 64):
    avg = np.mean([g + rng.normal(0, 0.2, g.shape) for _ in range(K)], axis=0)
    print(K, round(float(np.std(avg - g)), 3))    # 0.2, 0.1, 0.05, 0.025
```

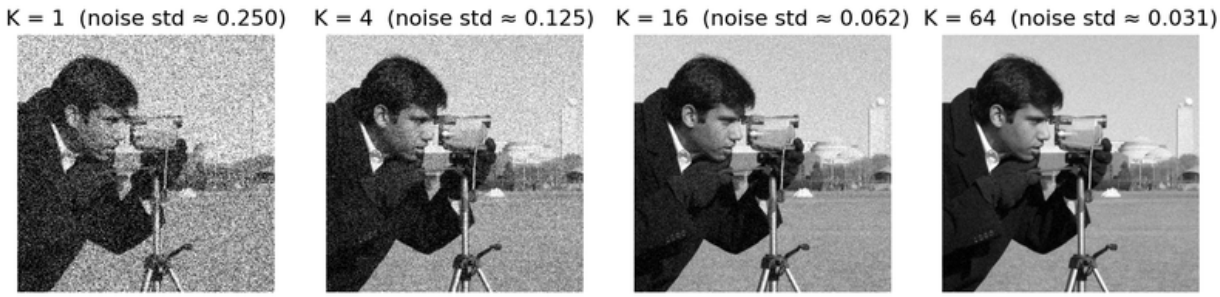


Figure 2.8 — Averaging K independent noisy frames. The noise standard deviation drops by a factor of about two each time K is multiplied by four.

Subtraction reveals differences. Subtracting two frames of a video highlights motion; in medical imaging, subtracting a pre-contrast "mask" image from a post-contrast image leaves mainly the vessels that took up the contrast agent.

Multiplication and division handle shading and masking. If a sensor's shading pattern $h(x, y)$ is known (for example by imaging a uniform target), the observed image $g = f \cdot h$ can be corrected as $f = g/h$. Multiplying by a binary mask keeps a region of interest and zeroes the rest.

One practical rule: arithmetic results often leave the valid range (differences can be negative, sums can exceed 255). Do the arithmetic in floating point, then rescale. A simple rescaling to $[0, K]$ is $f_m = f - \min f$ followed by $f_s = K f_m / \max f_m$.

Set and logical operations

Treat a binary image as a **set** A of pixel coordinates (the foreground). The usual set operations then apply [1]: union $A \cup B$, intersection $A \cap B$, complement $A^c = \{w \mid w \notin A\}$, and difference $A - B = A \cap B^c$. On binary arrays these become the logical operations OR, AND, NOT and AND-NOT. For gray images, union and intersection are commonly defined as elementwise maximum and minimum. Fuzzy sets generalize this further by letting membership vary between 0 and 1.

```
import numpy as np
from skimage import data

coins = data.coins()
A = coins > 100                                          # set A: bright pixels
B = np.zeros_like(A); B[:, : coins.shape[1] // 2] = True   # set B: left half
union, inter, diff = A | B, A & B, A & ~B
```

Set operations are the language of mathematical morphology (Chapter 9) and of combining segmentation masks.

Spatial operations

Spatial operations act directly on pixel values and positions. There are three kinds [1].

Single-pixel operations change each value independently: $s = T(z)$, where z is the input intensity and s the output. The image negative $s = (L-1) - z$ is an example. These are the intensity transformations of Chapter 3.

Neighborhood operations compute each output from a neighborhood S_{xy} around (x, y). Local averaging,

$$g(x,y) = \frac{1}{mn} \sum_{(r,c) \in S_{xy}} f(r,c),$$

where S_{xy} is an $m \times n$ window centered at (x, y), is the simplest; Chapter 3 generalizes it to spatial filtering.

Geometric transformations move pixels. They have two steps: a spatial transformation of coordinates, and intensity interpolation to fill the new grid. The most common family is the **affine** transform. In homogeneous coordinates,

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & t_x \\ a_{21} & a_{22} & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix},$$

where (x, y) is an input location, (x', y') the output location, the a_{ij} encode rotation, scaling and shear, and (t_x, t_y) is a translation. Affine maps preserve straight lines and parallelism, and chaining several is just multiplying their matrices.

Implementations almost always use **inverse mapping**: for each output pixel, apply the inverse transform to find where it came from in the input, and interpolate there. Forward mapping (pushing each input pixel to the output) leaves holes and collisions.

```
import cv2
from skimage import data

img = data.camera()
h, w = img.shape
M = cv2.getRotationMatrix2D(center=(w / 2, h / 2), angle=30, scale=0.8)  # 2x3 affine
rot = cv2.warpAffine(img, M, (w, h), flags=cv2.INTER_LINEAR)  # inverse mapping inside
```

Image registration

Registration aligns two images of the same scene taken at different times, from different viewpoints or with different sensors. Here the transform is unknown and must be estimated. The classic approach picks **tie points** (control points) whose positions are known in both images and fits a model to them; for example, a bilinear model

$$x = c_1 v + c_2 w + c_3 v w + c_4, \qquad y = c_5 v + c_6 w + c_7 v w + c_8,$$

maps input coordinates (v, w) to reference coordinates (x, y), and four tie-point pairs give eight equations for the eight unknowns $c_1, \ldots, c_8$. Zitová and Flusser's survey [6] organizes registra-

tion into four steps that are still the standard description: feature detection, feature matching, transform model estimation, and resampling.

Vector and matrix operations

A color pixel is a vector, $\mathbf{z} = [z_1, z_2, z_3]^T$ for its red, green and blue values. Vector tools then apply directly; for example, the Euclidean distance from $\mathbf{z}$ to a reference color $\mathbf{a}$,

$$D(\mathbf{z}, \mathbf{a}) = \left[(\mathbf{z} - \mathbf{a})^T (\mathbf{z} - \mathbf{a}) \right]^{1/2},$$

is the basis of simple color segmentation (Chapter 6). Stacking an $M \times N$ image into an $MN \times 1$ vector also lets any linear operation be written as $\mathbf{g} = \mathbf{H}\mathbf{f}$, a form used in restoration (Chapter 5).

Image transforms

Some problems are easier in a different domain. A 2-D linear transform takes the general form [1]

$$T(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y)\, r(x, y, u, v),$$

where f is the input image, r is the **forward transformation kernel**, and (u, v) are the transform-domain variables. The inverse uses an inverse kernel $s(x, y, u, v)$. When $r(x, y, u, v) = r_1(x, u)\, r_2(y, v)$ the kernel is **separable**, and the 2-D transform can be done as 1-D transforms along rows and then columns, or in matrix form $\mathbf{T} = \mathbf{AFB}$. The Fourier transform (Chapter 4) and the wavelet transforms (Chapter 7) are the two you will meet most.

Probability methods

Intensities can be treated as random quantities. If n_k pixels in an $M \times N$ image have intensity z_k, the probability of that intensity is estimated by

$$p(z_k) = \frac{n_k}{MN}, \qquad \sum_{k=0}^{L-1} p(z_k) = 1,$$

which is the **normalized histogram**. The mean and variance of intensities are

$$m = \sum_{k=0}^{L-1} z_k\, p(z_k), \qquad \sigma^2 = \sum_{k=0}^{L-1} (z_k - m)^2\, p(z_k),$$

where m measures average brightness and σ (the standard deviation) measures contrast. Higher moments describe skew and peakedness.

```
import numpy as np
from skimage import data
```

```
img = data.camera()
p = np.bincount(img.ravel(), minlength=256) / img.size   # p(z_k)
z = np.arange(256)
m = np.sum(z * p)
sigma = np.sqrt(np.sum((z - m) ** 2 * p))
print(round(m, 2), round(sigma, 2))                      # 129.06 73.64
```

Histogram processing (Chapter 3) and noise modeling (Chapter 5) both start here.

Modern view

The fundamentals in this chapter have not been replaced by deep learning; they have become the *interface* to it. Networks consume sampled, quantized arrays, are trained with losses that encode a model of perception, and are evaluated on data produced by camera pipelines. The reviews below map this territory.

Sampling and interpolation theory. Unser's "Sampling — 50 years after Shannon" [2] reframes sampling as approximation in shift-invariant spaces: instead of demanding a perfectly band-limited signal and an ideal sinc reconstructor, choose a practical basis (such as B-splines) and project onto it. Its main takeaway is that a good prefilter before sampling and a good reconstruction kernel after it matter more than the exact sampling rate. Thévenaz, Blu and Unser [4] apply this to image interpolation and compare many kernels on equal terms. They show that approximation order and kernel support explain most of the differences, and that B-spline-based interpolation (with a proper prefilter) offers a better quality-for-cost trade-off than Keys' cubic convolution [3]. Both papers remain the reference point for anyone who implements resizing.

Learned super-resolution. Wang, Chen and Hoi [9] survey deep super-resolution up to about 2019: network designs, upsampling layers placed early or late, loss functions, and benchmarks. Moser et al. [10] extend the picture to transformers and diffusion models and stress open problems such as flexible scale factors and better evaluation. Two works show how the classical ideas survive inside networks. LIIF [11] represents an image as a *continuous* function queried at any coordinate — a learned interpolator that can upsample by arbitrary factors, including factors not seen in training. Real-ESRGAN [12] tackles the gap between the bicubic downsampling used to create most training pairs and real degradations, by synthesizing training data with a higher-order degradation model that applies blur, noise, resizing and compression more than once. The lesson for this chapter: bicubic interpolation is both the baseline every method is compared against and, through its use in creating training data, a hidden assumption that can make models fail on real images.

Perception and image quality. Wang et al.'s SSIM [13] turned the observations of the perception section — sensitivity to structure and relative change rather than absolute error — into a full-reference quality index that compares local luminance, contrast and structure. Zhai and Min [14] survey the whole field of perceptual quality assessment: full-, reduced- and no-reference methods, and models for specialized content. Deep features changed this area markedly. Zhang et al. [15] collected human similarity judgments and found that distances between deep network

features (LPIPS) agree with people far better than PSNR or SSIM, across architectures and training regimes. Ding et al. [16] then asked a sharper question: if you *optimize* an image-processing network with each quality metric as the loss, which metric gives results people prefer? Their human study shows that rankings on standard quality benchmarks do not reliably predict usefulness as a training objective. In short, Mach bands and the Weber ratio now live inside loss functions.

Camera sensor pipelines. Ramanath et al. [8] describe the classic in-camera pipeline that sits between the sensor array of this chapter and the compressed file you save: demosaicking, white balance, color transforms, gamma, and compression, each a fixed hand-designed stage. Delbracio et al. [7] tour how mobile computational photography reworked this pipeline over two decades, with burst capture, alignment and merging (the averaging-noisy-images idea at scale), and multi-frame super-resolution. Learned replacements now exist for parts or all of it. Chen et al. [17] trained a network to map raw short-exposure sensor data directly to a clean image in very low light, bypassing the traditional pipeline. Ignatov et al. [18] replaced an entire smartphone image signal processor with one network (PyNET) trained to map raw mosaic sensor data to images resembling those from a DSLR.

What did *not* change? The pixel grid, the bit depth, adjacency and connectivity (still the basis of connected-component labeling in every segmentation post-processing step), affine warps (now differentiable layers inside networks), and histograms. Knowing them is what lets you spot when a model's input has been resized with the wrong kernel, quantized too coarsely, or compared with a metric that does not match human judgment.

Key takeaways

- Human vision senses *relative* changes (Weber's law) and exaggerates edges (Mach bands); perceived brightness depends on the surround. Good image processing respects these facts.
- Images can be formed from any part of the electromagnetic spectrum; for reflected light, $f = i \cdot r$ separates illumination from reflectance.
- Sampling sets spatial resolution, quantization sets intensity resolution, and storage is MNk bits. Too few levels cause false contours in smooth regions.
- Interpolation — nearest, bilinear, bicubic — estimates values between samples; it trades speed for smoothness and sharpness but cannot recover lost detail.
- 4-, 8- and m-adjacency define paths, connected components, regions and boundaries; D_4 and D_8 are distance metrics whose unit balls are diamonds and squares.
- Image operations are elementwise unless stated otherwise; linearity decides whether convolution and Fourier tools apply.
- Averaging K independent noisy frames reduces noise standard deviation by $\sqrt{K}$; geometric transforms use inverse mapping plus interpolation.
- Deep learning built on these fundamentals: learned interpolators, perceptual losses and learned camera pipelines all inherit this chapter's concepts.

Exercises

1. A grayscale scanner produces 2400 × 3300 images at 12 bits per pixel. How many megabytes does one uncompressed scan need? How many if it is reduced to 8 bits?

   Hint

   Use $b = MNk$ and divide by 8×10^6 (or 8×2^{20} for MiB). At 12 bits: $2400 \cdot 3300 \cdot 12/8 = 11.88$ MB. At 8 bits: 7.92 MB.

2. Quantize `skimage.data.moon()` to 4, 5 and 6 bits. At which level do false contours disappear for you? Then add a small amount of uniform random noise (amplitude half a quantization step) *before* quantizing at 4 bits. What changes, and why?

   Hint

   The noise (called dither) breaks up the large flat steps, so the eye averages the fine noise instead of seeing contours. The mean error stays similar but the visible structure changes — another reminder that squared error and perceived quality differ.

3. In the binary image below, count the connected components of 1-pixels under 4-, 8- and m-connectivity. Then count the components of 0-pixels under 4- and 8-connectivity. Which pairing gives a consistent "inside/outside" answer?

```
0 0 0 0 0
0 1 1 1 0
0 1 0 1 0
0 1 1 0 0
0 0 0 0 0
```

   Hint

   The 1s are a single component under all three adjacencies, but the ring is *closed* only if diagonal steps count, because of the gap at the lower right. The center 0 is isolated under 4-connectivity (a hole) but joins the outside under 8-connectivity. 4/4 gives an open curve with a hole; 8/8 gives a closed curve without one. Only 8-foreground with 4-background (or the reverse) is consistent. Check your answer with `skimage.measure.label`.

4. Show that $D_8(p,q) \le D_E(p,q) \le D_4(p,q)$ for any two pixels, and find pixel pairs where each inequality becomes an equality.

   Hint

   Let $d_x = |x-u|$, $d_y = |y-v|$. Then $\max(d_x,d_y)^2 \le d_x^2 + d_y^2 \le (d_x+d_y)^2$. Equality holds when one of d_x, d_y is zero.

5. Write a function that rotates an image by θ using *forward* mapping (push each input pixel to its rounded output position). Display the result for $\theta = 17°$ and explain the holes. Then fix it with inverse mapping and bilinear interpolation.

Hint

Rotation slightly stretches the grid spacing in some directions, so some output pixels receive no input pixel. With inverse mapping, every output pixel asks "where did I come from?" and always gets an answer.

6. You have 10 noisy frames of a static scene with noise standard deviation 12 gray levels. How many frames would you need to bring it below 2 gray levels? What assumption could fail in a real handheld burst, and how do mobile pipelines handle it?

Hint

$12/\sqrt{K} < 2$ requires $K > 36$, so 37 frames. The assumption that the scene and camera are static fails; frames must be aligned (registered) before averaging, and moving objects need robust merging [7].

References

1. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018, Ch. 2. publisher page
2. M. Unser, "Sampling—50 years after Shannon," *Proceedings of the IEEE*, vol. 88, no. 4, pp. 569–587, 2000. doi
3. R. Keys, "Cubic convolution interpolation for digital image processing," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 29, no. 6, pp. 1153–1160, 1981. doi
4. P. Thévenaz, T. Blu, and M. Unser, "Interpolation revisited," *IEEE Transactions on Medical Imaging*, vol. 19, no. 7, pp. 739–758, 2000. doi
5. A. Rosenfeld and J. L. Pfaltz, "Distance functions on digital pictures," *Pattern Recognition*, vol. 1, pp. 33–61, 1968. doi
6. B. Zitová and J. Flusser, "Image registration methods: a survey," *Image and Vision Computing*, vol. 21, no. 11, pp. 977–1000, 2003. doi
7. M. Delbracio, D. Kelly, M. S. Brown, and P. Milanfar, "Mobile computational photography: A tour," arXiv:2102.09000, 2021. arXiv
8. R. Ramanath, W. E. Snyder, Y. Yoo, and M. S. Drew, "Color image processing pipeline," *IEEE Signal Processing Magazine*, vol. 22, no. 1, pp. 34–43, 2005. doi
9. Z. Wang, J. Chen, and S. C. H. Hoi, "Deep learning for image super-resolution: A survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2020. arXiv
10. B. Moser, F. Raue, S. Frolov, J. Hees, S. Palacio, and A. Dengel, "Hitchhiker's guide to super-resolution: Introduction and recent advances," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2023. arXiv
11. Y. Chen, S. Liu, and X. Wang, "Learning continuous image representation with local implicit image function," CVPR, 2021. arXiv

12. X. Wang, L. Xie, C. Dong, and Y. Shan, "Real-ESRGAN: Training real-world blind super-resolution with pure synthetic data," arXiv:2107.10833, 2021. arXiv
13. Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004. project page
14. G. Zhai and X. Min, "Perceptual image quality assessment: A survey," *Science China Information Sciences*, vol. 63, no. 11, 2020. doi
15. R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, "The unreasonable effectiveness of deep features as a perceptual metric," CVPR, 2018. arXiv
16. K. Ding, K. Ma, S. Wang, and E. P. Simoncelli, "Comparison of full-reference image quality models for optimization of image processing systems," *International Journal of Computer Vision*, 2021. arXiv
17. C. Chen, Q. Chen, J. Xu, and V. Koltun, "Learning to see in the dark," CVPR, 2018. arXiv
18. A. Ignatov, L. Van Gool, and R. Timofte, "Replacing mobile camera ISP with a single deep learning model," arXiv:2002.05509, 2020. arXiv

IP 03 • Intensity Transformations and Spatial Filtering

Remap pixel values with curves and histograms, then smooth and sharpen images with small kernels: the everyday toolbox of spatial-domain enhancement.

Online edition: https://blog.shyandram.dev/en/learn/image-processing/dip-03-intensity-spatial-filtering/

What you'll learn

- How a single curve $s = T(r)$ can brighten, darken, invert or stretch an image, and when to pick a log, gamma or piecewise-linear curve.
- Why histogram equalization works, derived step by step, and how histogram matching, CLAHE-style local methods and local statistics build on it.
- What linear spatial filtering is, how correlation differs from convolution, and why separable kernels are fast.
- How smoothing (lowpass) filters, including the nonlinear median filter, remove noise, and how derivative-based (highpass) filters sharpen.
- How to derive highpass, bandpass and bandreject kernels from a lowpass one, and how to chain several methods into one pipeline.

The big picture

This chapter is about improving an image without leaving the pixel grid. Every method here works on the pixel values directly, which is what "spatial domain" means. There are two families. **Point operations** change each pixel using only its own value. **Neighborhood operations** (spatial filters) change each pixel using the values around it.

> **Explain like I'm five**
>
> Imagine a box of crayon drawings that are all too pale. A **point operation** is like a magic rule for every crayon color: "light pink becomes red, light blue becomes dark blue." You look at one spot, check the rule, and recolor it. A **filter** is like looking at a spot *and its neighbors* before you decide. If you want the drawing softer, you color each spot as the average of its neighbors. If you want the outlines crisper, you make each spot *more different* from its neighbors.

Why it matters: these are the first tools anyone reaches for, in phone cameras, medical viewers, microscopes and as preprocessing for computer-vision models. They are also the conceptual ancestors of the convolutional layer. A CNN's first layer is a bank of spatial filters, and many learned enhancement networks still output a curve or a filter, as we will see in the Modern view.

Background

Plain version. We write the output image as some operation applied to the input image. If the operation only looks at one pixel at a time, it is a lookup table. If it looks at a small window, it is a filter.

Precise version. Let $I(x, y)$ be the input image and $G(x, y)$ the output, with x, y integer pixel coordinates (the same names as in the convolution tutorial). A spatial-domain process is

$$G(x, y) = T[I](x, y),$$

where T is an operator defined over a **neighborhood** of (x, y). The neighborhood is usually a small square or rectangle centered on (x, y). The operator is applied at every location by moving the neighborhood's center from pixel to pixel.

When the neighborhood is a single pixel (1×1), T depends only on the value at that pixel. We then call it an **intensity transformation** (also gray-level or point transformation) and write it with scalars:

$$s = T(r),$$

where r is the input intensity at a pixel, s is the output intensity at the same pixel, and both lie in $[0, L-1]$ for an image with L intensity levels ($L = 256$ for 8-bit images). Because r takes only L values, any T can be stored as a lookup table of length L, which is why point operations are extremely fast.

Basic intensity transformation functions

Plain version. Draw a curve from input brightness (horizontal axis) to output brightness (vertical axis). Where the curve is steep, nearby gray levels get pulled apart, so contrast increases. Where it is flat, they get squeezed together, so contrast drops.

The slope $T'(r)$ is the local contrast gain: a small difference Δr between two neighboring pixels becomes roughly $T'(r)\,\Delta r$ after the transformation. Every curve below is a choice about where to spend that gain.

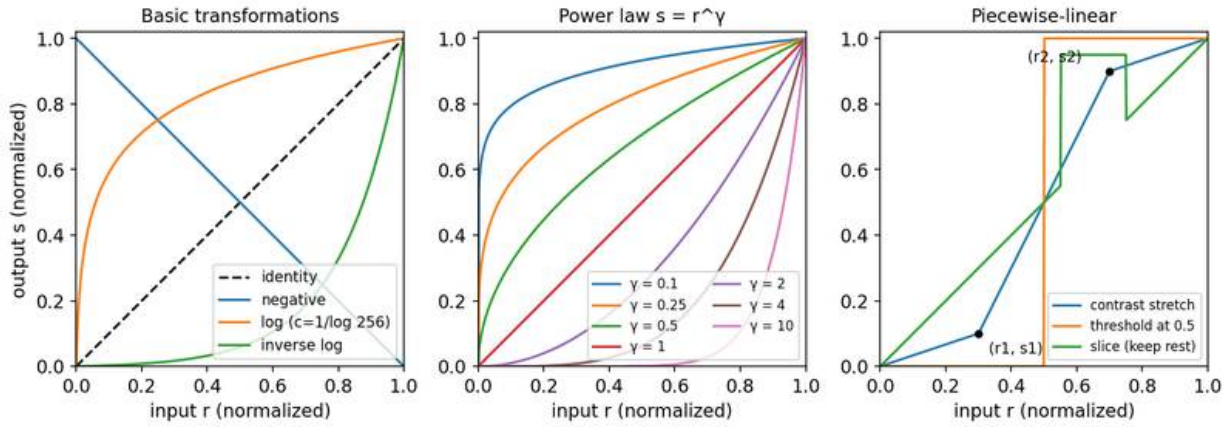


Figure 3.1 — Common intensity transformation curves, with intensities normalized to $[0, 1]$. Left: negative, log and inverse log. Middle: power-law (gamma) curves. Right: piecewise-linear curves.

Image negatives

$$s = (L - 1) - r.$$

Dark becomes light and light becomes dark. The slope is -1 everywhere, so contrast magnitude is unchanged; only the polarity flips. Negatives help when small bright details sit inside large dark regions, because people often judge gray detail more easily on a light background.

Log transformations

$$s = c \log(1 + r),$$

where $c > 0$ is a scale constant, usually chosen so that the largest input maps to $L - 1$, that is $c = (L - 1)/\log L$. The $+1$ keeps $\log 0$ out of the formula. The slope $c/(1 + r)$ is large for small r and small for large r. So the log curve expands dark values and compresses bright ones.

Its classic use is displaying data with a huge dynamic range, such as the magnitude of a Fourier spectrum, whose values can span many orders of magnitude. Scaled linearly, only the brightest point is visible. After the log, the structure appears (Figure 3.2, bottom row). The **inverse log** (exponential) curve does the opposite.

Power-law (gamma) transformations

$$s = c\, r^{\gamma},$$

where $c > 0$ and $\gamma > 0$ are constants (with r normalized to $[0, 1]$, take $c = 1$). For $\gamma < 1$ the curve bows upward and brightens the dark range, like the log. For $\gamma > 1$ it bows downward and darkens it. Unlike the log, one parameter gives a whole family of curves (Figure 3.1, middle).

Gamma matters beyond enhancement. Displays and cameras have power-law responses, and image files usually store gamma-encoded values rather than values proportional to light. Pre-compensating for a device's response is called **gamma correction**. One practical consequence: averaging, blurring or resizing gamma-encoded pixels is not the same as doing it on linear light, which is why careful pipelines linearize first.

```
import numpy as np
from skimage import data, img_as_float

r = img_as_float(data.camera())        # intensities scaled to [0, 1]
negative = 1.0 - r                     # s = (L-1) - r, in normalized form
log_t = np.log1p(r) / np.log(2.0)      # s = c log(1 + r), c chosen so s(1) = 1
bright = r ** 0.4                      # gamma < 1 lifts the shadows
dark = r ** 2.5                        # gamma > 1 deepens them
```

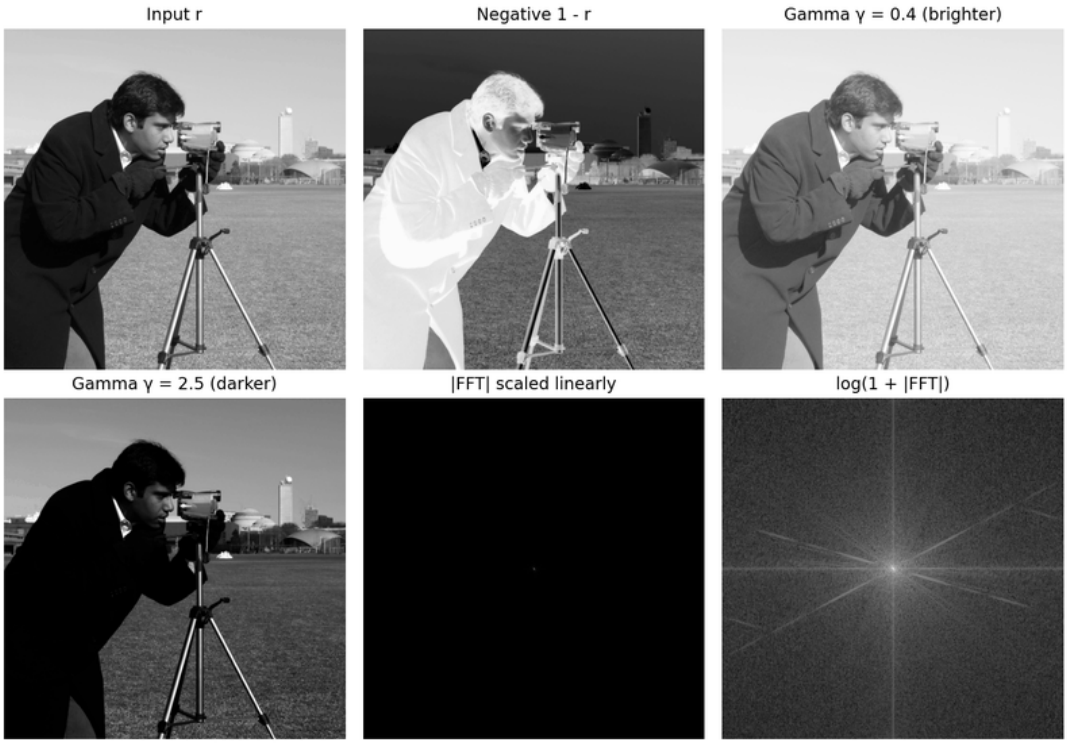


Figure 3.2 — Top: input, negative, and gamma 0.4. Bottom: gamma 2.5; a Fourier magnitude scaled linearly (almost all black); and the same magnitude after $\log(1 + \cdot)$.

Piecewise-linear transformations

Instead of one formula, we can join straight segments. This gives precise control, at the cost of more parameters.

Contrast stretching. Choose two control points (r_1, s_1) and (r_2, s_2) and connect $(0,0) \to (r_1, s_1) \to (r_2, s_2) \to (L-1, L-1)$ with straight lines. The middle segment has slope $(s_2 - s_1)/(r_2 - r_1)$. If $r_1 \le r_2$ and $s_1 \le s_2$, the curve is monotonic and never reverses the order of gray levels. Two special cases are worth remembering:

- $r_1 = r_{\min}$, $s_1 = 0$, $r_2 = r_{\max}$, $s_2 = L - 1$ maps the image's actual range onto the full range (a **min–max stretch**). In practice, use the 1st and 99th percentiles instead of the min and max, so a few outlier pixels do not control the result.
- $r_1 = r_2 = m$, $s_1 = 0$, $s_2 = L - 1$ produces a binary image: a **threshold** at m.

```
def stretch(img, r1, s1, r2, s2, L=256):
    lut = np.interp(np.arange(L), [0, r1, r2, L - 1], [0, s1, s2, L - 1])
    return lut.astype(np.uint8)[img]

img = data.camera()
lo, hi = np.percentile(img, [1, 99])
out = stretch(img, lo, 0, hi, 255)     # robust min-max stretch
```

Intensity-level slicing. Highlight one band of intensities $[A, B]$. Either set the band to a high value and everything else to a low value (a binary result), or brighten the band and leave everything else unchanged. Slicing is useful when the gray range of interest is known, such as one tissue type or one material in an X-ray.

Bit-plane slicing. An 8-bit pixel value is a sum of bits, $r = \sum_{b=0}^{7} a_b \, 2^b$ with $a_b \in \{0, 1\}$. Collecting bit b of every pixel gives a binary image called **bit plane** b. Plane 7, the most significant bit (MSB), is exactly a threshold at 128. The low planes look like noise because they carry fine variations. Keeping only the top four planes reconstructs an image that is very close to the original, with a maximum error below $2^4 = 16$ levels (Figure 3.3). This explains why bit planes matter for compression and why the lowest planes are a classic place to hide watermarks.

```
img = data.camera()                                  # uint8
planes = [(img >> b) & 1 for b in range(8)]          # planes[7] is the MSB
top4 = sum(planes[b].astype(np.uint8) << b for b in range(4, 8))
print(np.abs(img.astype(int) - top4).max())          # 15
```

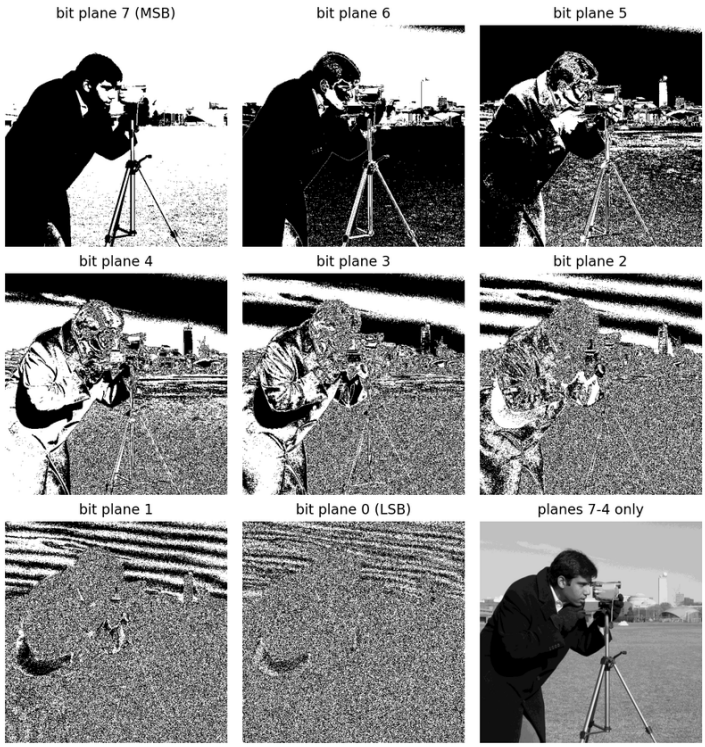


Figure 3.3 — The eight bit planes of an 8-bit image, from the most significant (top left) to the least significant, and the image rebuilt from planes 7–4 only.

Histogram processing

Plain version. A histogram counts how many pixels have each brightness. A dark image has its counts piled up on the left; a washed-out one has them squeezed into a narrow band. Histogram processing chooses a curve T *automatically from those counts*, so that the output histogram has a shape we want.

Precise version. For an $M \times N$ image with levels r_k, $k = 0, \dots, L-1$, let n_k be the number of pixels with value r_k. The **normalized histogram**

$$p_r(r_k) = \frac{n_k}{MN}$$

is an estimate of the probability that a randomly chosen pixel has value r_k. It sums to 1.

Histogram equalization

The goal is a curve T that makes the output histogram as flat as possible, so every gray level is used about equally often.

Derivation (continuous case). Treat intensity as a continuous random variable $r \in [0, L-1]$ with probability density function (PDF) $p_r(r)$. Assume T is strictly increasing on that interval. Then a pixel lands in a small interval $[r, r+dr]$ exactly when its output lands in $[s, s+ds]$ with $s = T(r)$. Equal probabilities give $p_s(s)\,|ds| = p_r(r)\,|dr|$, so

$$p_s(s) = p_r(r)\left|\frac{dr}{ds}\right|.$$

Now choose T as the scaled cumulative distribution function (CDF) of r:

$$s = T(r) = (L-1)\int_0^r p_r(w)\,dw,$$

where w is a dummy integration variable. By the fundamental theorem of calculus, $ds/dr = (L-1)\,p_r(r)$. Substituting,

$$p_s(s) = p_r(r)\cdot\frac{1}{(L-1)\,p_r(r)} = \frac{1}{L-1}, \qquad 0 \le s \le L-1.$$

The output PDF is uniform, whatever the input PDF was. The CDF is the one monotonic curve that spreads probability evenly: it is steep where many pixels live (so they get pulled apart) and flat where few pixels live.

Discrete version. Replace the integral by a sum:

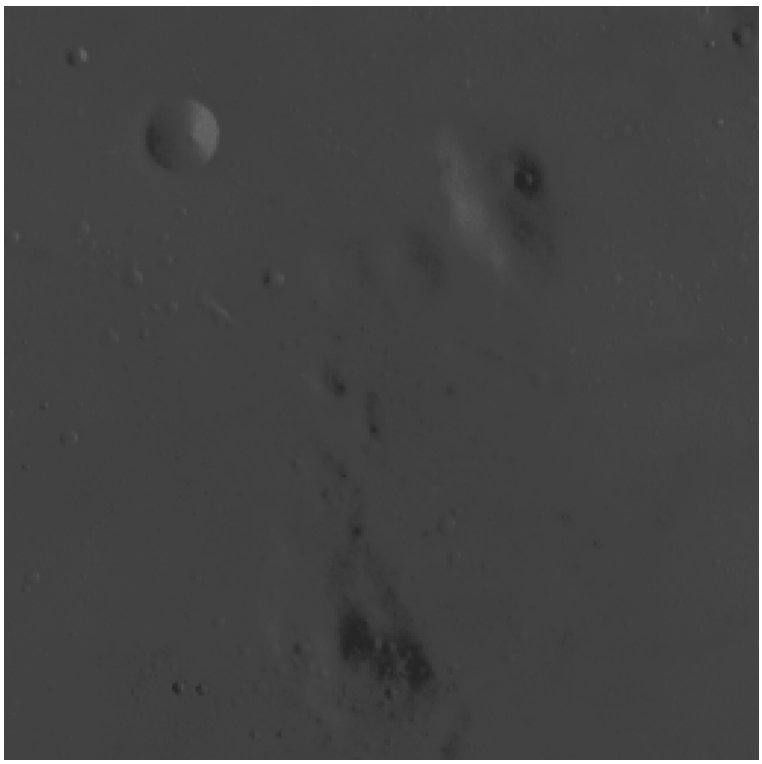
$$s_k = T(r_k) = (L-1)\sum_{j=0}^{k} p_r(r_j), \qquad k = 0, 1, \ldots, L-1,$$

and round s_k to the nearest integer. Because many input levels can round to the same output level and no level can be split, the discrete result is only *approximately* flat. The output histogram usually shows tall spikes separated by gaps (Figure 3.4, middle). What *is* guaranteed is that the output spans the full range and that the CDF of the output is close to a straight line.

```
def equalize(img, L=256):
    hist = np.bincount(img.ravel(), minlength=L)      # n_k
    cdf = np.cumsum(hist) / img.size                  # sum of p_r(r_j), j <= k
    lut = np.round((L - 1) * cdf).astype(np.uint8)    # s_k
    return lut[img]
```

OpenCV's `cv2.equalizeHist` uses a slightly different normalization, $(\mathrm{cdf}_k - \mathrm{cdf}_{\min})/(MN - \mathrm{cdf}_{\min})$, so the darkest occupied level maps to 0. Results differ by at most a level or so.

Drag the slider to compare a deliberately low-contrast image with its globally equalized version. Notice that equalization is not "free": it exaggerates the large smooth areas and grain, because it spends contrast wherever the pixel *counts* are high, not where the *interesting* detail is.



Input

Equalized

A dim, flat-looking lunar surface image and its histogram-equalized version with strong contrast.

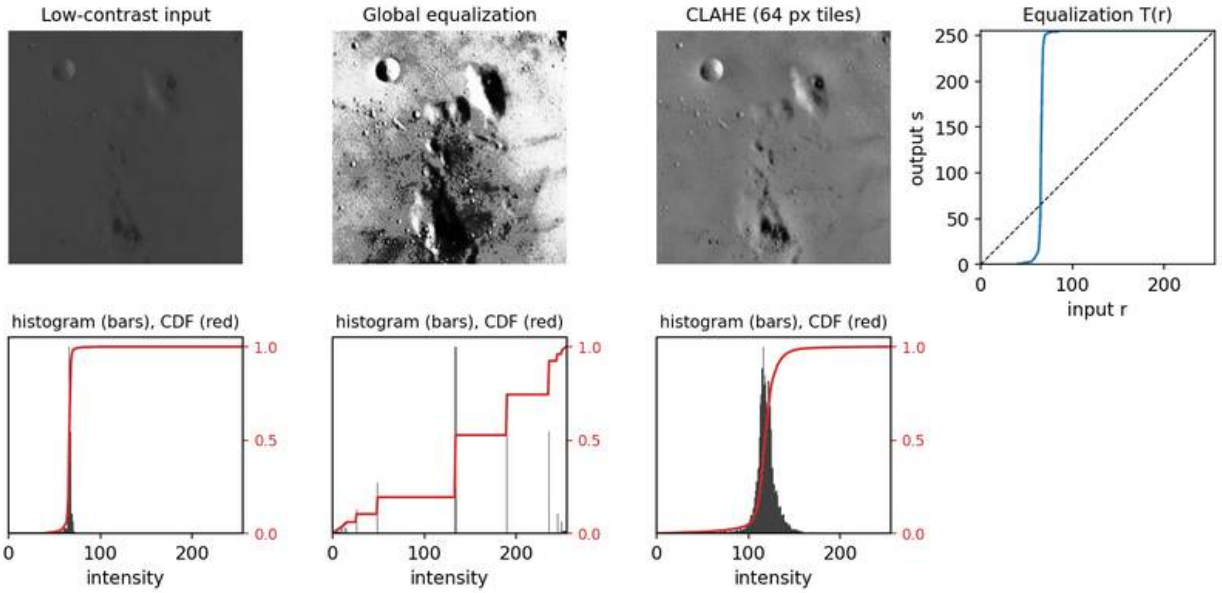


Figure 3.4 — Global histogram equalization versus CLAHE. The input occupies a narrow band of gray levels, so the equalization curve $T(r)$ (right) is almost a step, and the output histogram is spread out with gaps. CLAHE limits the slope and adapts per region.

Histogram matching (specification)

Sometimes a flat histogram is the wrong target. We may want an image to look like a reference image, or to have a histogram shape chosen by hand. **Histogram matching** maps the input so that its histogram approximates a specified PDF $p_z(z)$.

The trick is to use equalization twice. Equalizing the input gives $s = T(r)$ with a uniform s. Equalizing the *target* distribution gives

$$G(z) = (L-1)\int_0^z p_z(t)\,dt,$$

which also yields a uniform variable. Two uniform variables can be matched, so we set $G(z) = T(r)$ and solve:

$$z = G^{-1}(T(r)).$$

Here z is the output intensity, G is the scaled CDF of the desired distribution and t is a dummy variable. In the discrete case G is a staircase and may not be invertible, so for each input level we take the smallest z with $G(z) \geq T(r)$. That is a single `searchsorted` call:

```
def match(img, ref, L=256):
```

```
    cdf_in = np.cumsum(np.bincount(img.ravel(), minlength=L)) / img.size
    cdf_ref = np.cumsum(np.bincount(ref.ravel(), minlength=L)) / ref.size
    # For each input level, the smallest z with G(z) >= T(r).
    lut = np.searchsorted(cdf_ref, cdf_in).clip(0, L - 1).astype(np.uint8)
    return lut[img]
```

`skimage.exposure.match_histograms` implements the same idea, including per-channel matching for color images.

Exact histogram matching

Both methods above are lookup tables, so all pixels with the same input value receive the same output value. If 30% of the pixels share one value, no lookup table can split them, and the output histogram cannot be exactly flat or exactly match a target.

Exact histogram specification removes this limit by making every pixel distinguishable first [5]. Each pixel gets a vector of features: its own value, then the mean over a small neighborhood, then the mean over a larger one, and so on. Sorting pixels lexicographically by this vector produces a *strict* ordering in practice, because ties in one feature are broken by the next. Then assign output values in that order: the first h_0 pixels get level 0, the next h_1 get level 1, and so on, where h_k is the exact count the target histogram requires. The result has precisely the requested histogram. Coltuc, Bolon and Chassery showed that ordering by local means of increasing size gives visually sensible tie-breaking, because pixels in brighter surroundings are pushed slightly upward [5].

Local histogram processing

Global methods use one curve for the whole image. If a dark corner holds the important detail but contains few pixels, the global histogram barely notices it.

Local histogram equalization computes the transformation from the histogram of a neighborhood around each pixel and applies it only to the center pixel, then moves on. **Adaptive histogram equalization** (**AHE**) and its variants were studied systematically by Pizer and colleagues, including the problem that AHE over-amplifies noise in nearly uniform regions, where the local histogram is a single spike and the CDF turns into a near-vertical step [2].

Contrast-limited AHE (**CLAHE**) fixes this by limiting the slope of the mapping. Since the slope of an equalization curve is proportional to the histogram height, clipping each local histogram at a ceiling and redistributing the clipped counts across all bins caps the contrast gain. To make it fast, the image is split into tiles; a mapping is computed per tile, and each pixel's output is interpolated bilinearly from the four nearest tile mappings, which avoids visible tile seams. Zuiderveld's Graphics Gems chapter gave the widely reused implementation [3], and Reza described a real-time hardware realization [4].

```
import cv2
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8, 8))
```

```
out = clahe.apply(data.moon())         # uint8 in, uint8 out
```

In scikit-image, the equivalent is `skimage.exposure.equalize_adapthist(img, clip_limit=0.01)`.

Using histogram statistics for image enhancement

A histogram is a distribution, so its **moments** summarize it. With $p(r_k)$ the normalized histogram, the mean and variance are

$$m = \sum_{k=0}^{L-1} r_k\, p(r_k), \qquad \sigma^2 = \sum_{k=0}^{L-1} (r_k - m)^2\, p(r_k),$$

where m measures average brightness and σ (the standard deviation) measures contrast. Computed over a neighborhood S_{xy} centered on (x, y), they become the **local mean** $m_{S_{xy}}$ and **local standard deviation** $\sigma_{S_{xy}}$.

These give a simple, targeted enhancement rule. Suppose we want to boost only regions that are *dark* and have *some but not too much* contrast (to skip both flat background and strong edges). With global mean m_G and global standard deviation σ_G, we multiply a pixel by a gain $E > 1$ only if

$$m_{S_{xy}} \le k_0\, m_G \quad \text{and} \quad k_1\, \sigma_G \le \sigma_{S_{xy}} \le k_2\, \sigma_G,$$

and leave it alone otherwise. Here k_0, k_1, k_2 are user-chosen fractions (for example $0.5, 0.02, 0.5$). The local statistics need only two box filters, one on I and one on I^2, using $\sigma^2 = \mathbb{E}[I^2] - (\mathbb{E}[I])^2$:

```
from scipy.ndimage import uniform_filter

f = img_as_float(data.moon())
m = uniform_filter(f, size=3)                                  # local mean
sd = np.sqrt(np.maximum(uniform_filter(f**2, size=3) - m**2, 0))  # local std
mG, sG = f.mean(), f.std()
mask = (m <= 0.5 * mG) & (sd >= 0.02 * sG) & (sd <= 0.5 * sG)
g = np.where(mask, np.clip(3.0 * f, 0, 1), f)                  # gain E = 3
```

The same "local mean and local variance from box filters" pattern reappears in the guided filter [8], one of the edge-preserving filters discussed in the Modern view.

Fundamentals of spatial filtering

Plain version. A spatial filter slides a small grid of weights (a **kernel**) across the image. At each position, it multiplies the weights by the pixels underneath and adds the products. The result is the new value of the center pixel.

The convolution tutorial covers the mechanics with a from-scratch NumPy loop, OpenCV calls and border handling. Here we focus on the theory the rest of the chapter needs.

Linear spatial filtering and correlation

Using that tutorial's notation, for a $(2k+1) \times (2k+1)$ kernel K the **correlation** of K with I is

$$(K * I)(x, y) = \sum_{i=-k}^{k} \sum_{j=-k}^{k} K(i, j)\, I(x + i,\ y + j),$$

where (i, j) are offsets from the kernel's center and k is the kernel's half-width. Odd sizes give the kernel a well-defined center. The filter is **linear** because the output is a linear combination of input values: filtering $aI_1 + bI_2$ gives a times the result for I_1 plus b times the result for I_2. It is also **shift-invariant** because the same weights are used everywhere.

Correlation versus convolution

Convolution is the same computation with the kernel rotated by 180°:

$$(K * I)(x, y) = \sum_{i=-k}^{k} \sum_{j=-k}^{k} K(i, j)\, I(x - i,\ y - j).$$

The two coincide for kernels that are symmetric under a 180° rotation (box, Gaussian, the Laplacian), and differ for antisymmetric ones (Sobel, any directional derivative), where they produce the same response with opposite sign. The useful difference shows up with an **impulse** (a single 1 among zeros): correlating a kernel with an impulse produces a rotated copy of the kernel, while convolving produces an exact copy. This is why convolution, not correlation, is the operation with clean algebra:

- **commutative:** $K * I = I * K$,
- **associative:** $K_1 * (K_2 * I) = (K_1 * K_2) * I$, so a chain of filters can be merged into one kernel,
- **distributive:** $K * (I_1 + I_2) = K * I_1 + K * I_2$.

Associativity is what the next two subsections rely on. As noted in the convolution tutorial, `cv2.filter2D` computes correlation, and most deep-learning "convolution" layers also compute correlation; since their weights are learned, the flip does not matter there.

Separable kernels

A kernel is **separable** if it is the outer product of a column vector $\mathbf{c}$ and a row vector $\mathbf{r}$:

$$K = \mathbf{c}\,\mathbf{r}^{\top}.$$

Then $K * I = \mathbf{c} * (\mathbf{r}^{\top} * I)$: filter every row with $\mathbf{r}$, then every column with $\mathbf{c}$. For an $m \times n$ kernel this costs $m + n$ multiplications per pixel instead of mn, so the speedup is $mn/(m + n)$, which is 3.5× for a 7×7 kernel and grows linearly with size. A matrix is an outer product exactly when its **rank** is 1, which also gives a test: compute the singular value decomposition and check that only one

singular value is nonzero. Box and Gaussian kernels are separable; so is the Sobel kernel, which is a smoothing vector times a differencing vector.

```
g1 = cv2.getGaussianKernel(7, 1.5)          # 7x1 column, sums to 1
K = g1 @ g1.T                               # 7x7 Gaussian kernel
print(np.linalg.matrix_rank(K))             # 1  ->  separable
f = img_as_float(data.camera()).astype(np.float32)
a = cv2.filter2D(f, -1, K.astype(np.float32))
b = cv2.sepFilter2D(f, -1, g1, g1)          # two 1-D passes
print(np.abs(a - b).max())                  # ~1e-7
```

Spatial and frequency domains

The **convolution theorem** states that convolution in space is multiplication in frequency: if $\mathcal{F}$ denotes the Fourier transform, then $\mathcal{F}\{K * I\} = \mathcal{F}\{K\} \cdot \mathcal{F}\{I\}$. So every linear shift-invariant filter has two equivalent descriptions: a kernel K in space, and a **transfer function** $H = \mathcal{F}\{K\}$ that says how much of each frequency survives. Smoothing kernels have transfer functions that are large near zero frequency and small at high frequencies: they are **lowpass**. Sharpening kernels do the opposite: they are **highpass**. Chapter 4 develops this view properly. For now, the vocabulary is enough: "lowpass" means "smooth", "highpass" means "edges and detail".

Small kernels are usually cheaper to apply in space. Very large kernels are often cheaper in frequency via the fast Fourier transform.

Constructing spatial filter kernels

There are three common recipes:

1. **From a mathematical property.** Averaging gives the box kernel. Derivatives give difference kernels (the Laplacian, Sobel).
2. **By sampling a continuous function.** Sample a 2-D Gaussian on the integer grid, then normalize the weights to sum to 1.
3. **From a frequency specification.** Design H in the frequency domain and take its inverse transform to get a spatial kernel, usually truncated to a manageable size.

Two sanity checks apply to all of them. A smoothing kernel's weights should sum to 1, so flat regions keep their brightness. A derivative kernel's weights should sum to 0, so flat regions give 0.

Smoothing (lowpass) spatial filters

Plain version. Replace each pixel by an average of its neighborhood. Random noise is pushed toward the average and fades, but sharp edges are also smeared, because the average straddles both sides.

Box filter

The $m \times n$ box kernel has every weight equal to $1/(mn)$. It is the simplest lowpass filter and is separable, and with a running sum it costs a constant amount of work per pixel regardless of size. Its weakness is its frequency response: the transform of a box is a sinc-like function with side lobes, so it lets some high frequencies through with flipped sign, and it smears in a square, direction-dependent way. Small text and thin lines can turn into blocky streaks.

Gaussian filter

The Gaussian kernel samples

$$w(s,t) = C\, e^{-\frac{s^2+t^2}{2\sigma^2}},$$

where (s,t) are offsets from the kernel's center, σ (the standard deviation) sets the amount of blur, and C is chosen so the weights sum to 1. It has three properties that make it the default smoothing kernel:

- **Isotropy.** It depends only on the distance $\sqrt{s^2+t^2}$, so it blurs equally in every direction.
- **Separability.** $e^{-(s^2+t^2)/2\sigma^2} = e^{-s^2/2\sigma^2}\, e^{-t^2/2\sigma^2}$.
- **A smooth, non-negative frequency response.** Its Fourier transform is also a Gaussian, so there are no side lobes.

Since about 99.7% of a 1-D Gaussian's mass is within $\pm 3\sigma$, a kernel of size about 6σ, rounded up to the next odd integer, captures almost all of it. Larger kernels waste work, smaller ones truncate the tails. Convolving two Gaussians with σ_1 and σ_2 gives a Gaussian with $\sigma = \sqrt{\sigma_1^2 + \sigma_2^2}$, so repeated small blurs compose predictably.

Order-statistic (nonlinear) filters

Order-statistic filters **sort** the values in the neighborhood and pick one. They are nonlinear: no kernel can represent them, and the convolution theorem does not apply.

- **Median filter:** output the middle value (the 50th percentile). It is the standard remedy for **impulse noise** (salt-and-pepper), isolated pixels forced to black or white. An impulse is an extreme value, so it sorts to the end of the list and is almost never the median. As long as fewer than half the pixels in the window are corrupted, the median ignores them. It also preserves step edges far better than a linear average of the same size.
- **Max filter** (100th percentile): finds the brightest point; it removes pepper (dark) noise but grows bright specks.
- **Min filter** (0th percentile): the reverse; removes salt (bright) noise but grows dark specks.

Max and min filters reappear in Chapter 9 as grayscale dilation and erosion.

```
from scipy import ndimage as ndi

f = img_as_float(data.camera()).astype(np.float32)
noisy = f.copy()
u = np.random.default_rng(0).random(f.shape)
noisy[u < 0.05], noisy[u > 0.95] = 0.0, 1.0       # 10% salt-and-pepper

box = cv2.blur(noisy, (5, 5))
gauss = cv2.GaussianBlur(noisy, (0, 0), sigmaX=1.5)
med = cv2.medianBlur(noisy, 3)
mn, mx = ndi.minimum_filter(noisy, 3), ndi.maximum_filter(noisy, 3)
```

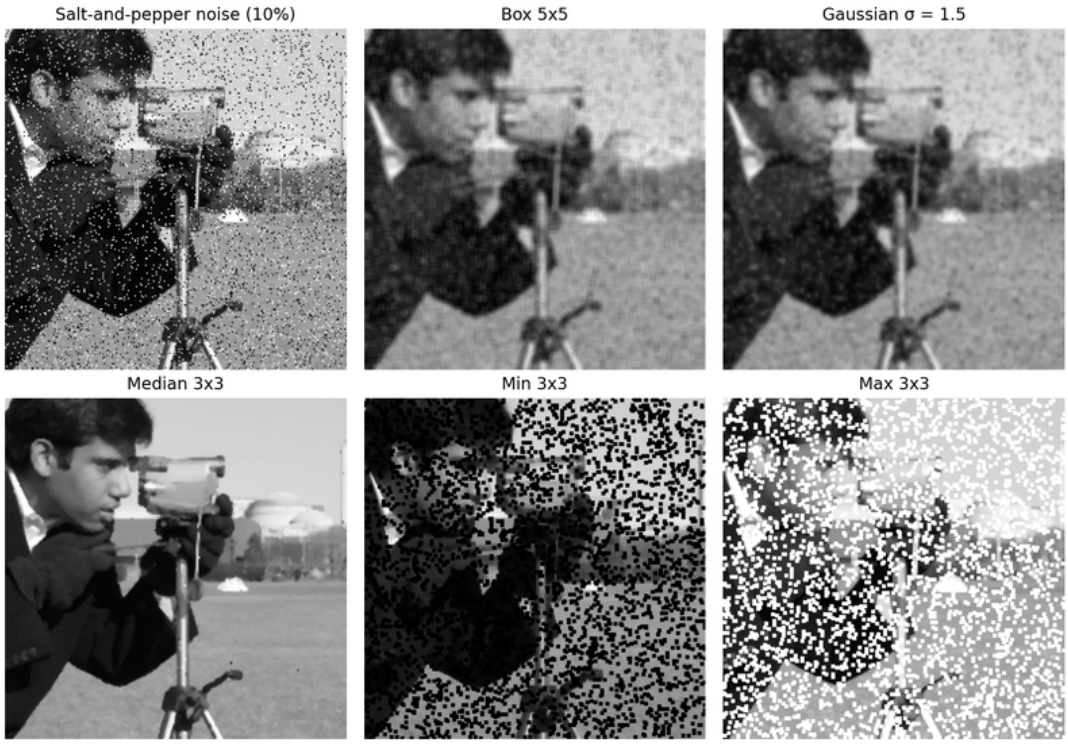


Figure 3.5 — Smoothing 10% salt-and-pepper noise. Linear filters (box, Gaussian) spread each impulse into a gray smudge. The 3×3 median removes nearly all of it. Min and max each remove one polarity and enlarge the other.

Sharpening (highpass) spatial filters

Plain version. Smoothing averages; sharpening does the opposite and highlights *differences*. Differences between neighbors are derivatives, so sharpening is built from derivatives.

First and second derivatives

For a 1-D signal $f(x)$ on a pixel grid, the simplest approximations are

$$\frac{\partial f}{\partial x} = f(x+1) - f(x), \qquad \frac{\partial^2 f}{\partial x^2} = f(x+1) + f(x-1) - 2f(x).$$

Figure 3.6 applies both to a hand-made profile with a ramp, a one-pixel-wide line and a step. Reading it gives the rules that decide which derivative to use:

- In **flat** regions, both derivatives are 0.
- Along a **ramp**, the first derivative is nonzero all the way, which makes thick responses. The second derivative is nonzero only at the ramp's start and end.
- At a **thin line**, the second derivative responds strongly with a double response (positive, strong negative, positive), and its magnitude is larger than the first derivative's. Fine detail is exactly what sharpening should emphasize.
- At a **step**, the second derivative produces a positive and a negative value next to each other. The **zero crossing** between them locates the edge precisely, which Chapter 10 uses for edge detection.

So for enhancing fine detail, the second derivative is the more natural choice.

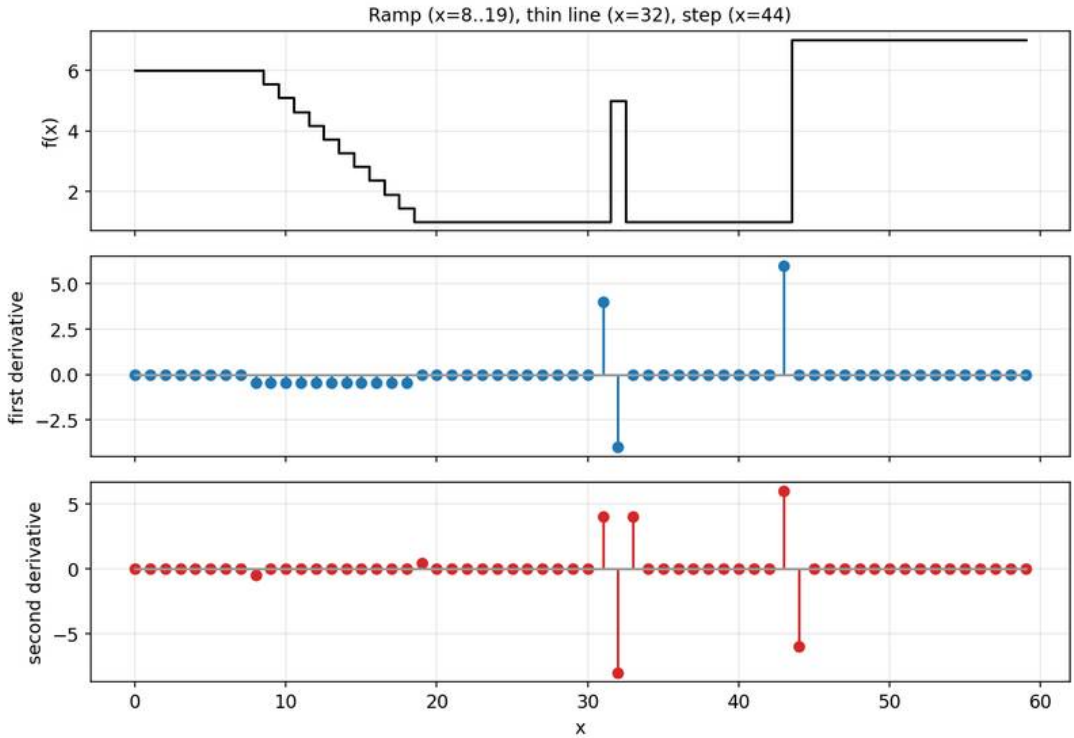


Figure 3.6 — First and second differences of a synthetic profile. The second derivative ignores the inside of the ramp, responds most strongly to the thin line, and changes sign across the step.

The Laplacian

The simplest **isotropic** second-derivative operator (one whose response does not change when the image is rotated) is the **Laplacian**:

$$\nabla^2 I = \frac{\partial^2 I}{\partial x^2} + \frac{\partial^2 I}{\partial y^2}.$$

Adding the 1-D difference above in x and in y gives

$$\nabla^2 I(x,y) = I(x+1,y) + I(x-1,y) + I(x,y+1) + I(x,y-1) - 4I(x,y),$$

which is correlation with the kernel

$$K_{\nabla^2} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}.$$

Adding the two diagonal directions as well gives a version with 1 in all eight outer cells and -8 in the center, which is isotropic in 45° steps instead of 90° steps. Flipping all the signs gives equally valid kernels. The weights sum to 0, so flat regions vanish and only changes survive.

Sharpening with the Laplacian. The Laplacian image is mostly gray (zero) with bright and dark fringes at edges. Adding it back to the original puts those fringes on the edges:

$$G(x,y) = I(x,y) + c\,\nabla^2 I(x,y),$$

where $c = -1$ for kernels with a negative center (like K_{∇^2} above) and $c = +1$ for kernels with a positive center. The sign matters: with the wrong sign, edges get blurred instead of sharpened. Because convolution is distributive, $I - \nabla^2 I$ is itself a single kernel, with center 5 and -1 above, below and to each side, which is exactly the `sharpen` preset in the playground below.

> **Interactive demonstration (online)**: open the online edition of this tutorial to use this demonstration.Open demonstration

Try the `gaussian` and `box-blur` presets for smoothing, then `laplacian` to see the bare second-derivative response. Changing the center of the sharpen kernel from 5 to 6 makes the weights sum to 2 and brightens the whole image, which shows why the sum matters.

Unsharp masking and highboost filtering

A trick from photographic darkrooms: subtract a blurred copy to isolate the detail, then add the detail back.

$$g_{\text{mask}}(x,y) = I(x,y) - \bar{I}(x,y), \qquad G(x,y) = I(x,y) + \alpha\, g_{\text{mask}}(x,y),$$

where $\bar{I}$ is a blurred (lowpass) version of I, g_{mask} is the **unsharp mask** (the detail the blur removed), and $\alpha \geq 0$ is a weight. With $\alpha = 1$ this is **unsharp masking**. With $\alpha > 1$ it is **highboost filtering**. With $0 < \alpha < 1$ the effect is gentle. Because g_{mask} is a highpass signal, this is another

way of adding highpass detail, and if $\bar{I}$ is a Gaussian blur, the blur's σ sets the *size* of the details that get boosted.

Large α causes visible **halos**: overshoot on both sides of strong edges, and the output may leave the valid range, so clip it.

The gradient and Sobel operators

First derivatives are used through the **gradient**, the vector of partial derivatives

$$\nabla I = \begin{bmatrix} g_x \\ g_y \end{bmatrix} = \begin{bmatrix} \partial I/\partial x \\ \partial I/\partial y \end{bmatrix},$$

which points in the direction of the fastest increase of intensity. Its length, the **gradient magnitude**

$$M(x,y) = \sqrt{g_x^2 + g_y^2} \approx |g_x| + |g_y|,$$

is large at edges and near 0 in flat regions. The right-hand approximation avoids the square root but is not isotropic.

The **Sobel operators** estimate g_x and g_y with 3×3 kernels:

$$K_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \qquad K_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}.$$

Each is separable: $K_x = [1,2,1]^\top [-1,0,1]$, a central difference in one direction multiplied by a small smoothing in the other. The 2 in the middle gives more weight to the center row or column, which reduces the operator's sensitivity to noise. (The convolution tutorial uses K_x as its edge-detection example.) The gradient magnitude is rarely used to sharpen directly; it is more often used to *find* edges, or as a mask that says where sharpening should be allowed, as in the combined example below.

```
f = img_as_float(data.chelsea()[..., 1]).astype(np.float32)
lap_k = np.array([[0, 1, 0], [1, -4, 1], [0, 1, 0]], np.float32)
lap = cv2.filter2D(f, -1, lap_k)
sharp = np.clip(f - lap, 0, 1)                # c = -1 because the center is negative

blur = cv2.GaussianBlur(f, (0, 0), 2)
mask = f - blur                               # g_mask
unsharp = np.clip(f + 1.0 * mask, 0, 1)       # alpha = 1
highboost = np.clip(f + 3.0 * mask, 0, 1)     # alpha = 3

gx = cv2.Sobel(f, cv2.CV_32F, 1, 0, ksize=3)
gy = cv2.Sobel(f, cv2.CV_32F, 0, 1, ksize=3)
M = np.hypot(gx, gy)                          # gradient magnitude
```

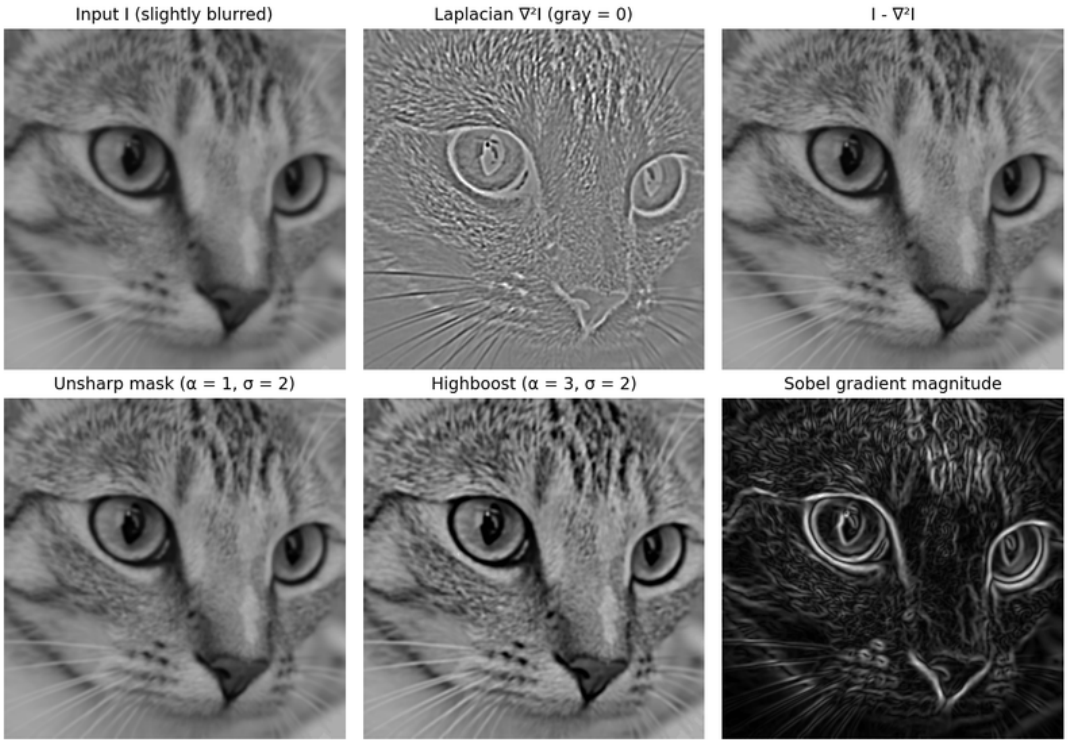


Figure 3.7 — Sharpening a slightly blurred image. Top: input, Laplacian (gray is zero), and $I - \nabla^2 I$. Bottom: unsharp masking, highboost filtering, and the Sobel gradient magnitude.

Always filter in floating point. A `uint8` pipeline clips the negative values of a Laplacian or a Sobel response to 0 and silently loses half the information.

Highpass, bandreject and bandpass filters from lowpass filters

Plain version. If you know how to keep the smooth part, you also know how to keep everything *except* the smooth part: subtract.

Let δ be the **unit impulse kernel**: a 1 at the center and 0 elsewhere, so $\delta * I = I$. Its transfer function equals 1 at every frequency (an all-pass filter). If K_{LP} is a lowpass kernel with transfer function H_{LP}, then

$$K_{\text{HP}} = \delta - K_{\text{LP}} \quad \Longleftrightarrow \quad H_{\text{HP}} = 1 - H_{\text{LP}}.$$

Unsharp masking is exactly this: $g_{\text{mask}} = I - \bar{I} = (\delta - K_{\text{LP}}) * I$.

To isolate a **band** of frequencies, combine two lowpass filters. Let K_1 be a Gaussian with small σ_1 (a high cutoff, so it keeps more detail) and K_2 a Gaussian with larger σ_2 (a low cutoff). Then

$$K_{\text{BP}} = K_1 - K_2, \qquad K_{\text{BR}} = \delta - K_{\text{BP}} = K_2 + (\delta - K_1).$$

The **bandpass** kernel K_{BP} keeps frequencies that K_1 passes but K_2 blocks. The **bandreject** kernel

K_{BR} is the sum of a lowpass and a highpass and removes only that middle band. The bandpass $K_1 - K_2$ is the **difference of Gaussians** (DoG), which is also a good approximation of the Laplacian of a Gaussian and is the basis of the scale-space detectors in Chapter 12. The weights confirm the design: highpass and bandpass kernels sum to 0, and the bandreject kernel sums to 1.

```
def gauss_kernel(sigma):
    n = 2 * int(np.ceil(3 * sigma)) + 1          # about 6 sigma, odd
    g = cv2.getGaussianKernel(n, sigma)
    return g @ g.T

def delta_like(K):
    D = np.zeros_like(K); D[K.shape[0] // 2, K.shape[1] // 2] = 1
    return D

lp2 = gauss_kernel(4.0)                          # low cutoff
lp1 = gauss_kernel(1.0)                          # high cutoff
lp1 = np.pad(lp1, (lp2.shape[0] - lp1.shape[0]) // 2)   # zero-pad to the same size
hp = delta_like(lp1) - lp1                       # highpass, sums to 0
bp = lp1 - lp2                                   # bandpass (DoG), sums to 0
br = delta_like(bp) - bp                         # bandreject, sums to 1
```

Combining spatial enhancement methods

Plain version. Real images rarely have only one problem. A useful enhancement is usually a short pipeline, and the order of the steps matters.

Here is a worked example. The input (Figure 3.8a) is a portrait that is underexposed, has a little Gaussian noise, and has 4% salt-and-pepper impulses. We want a brighter, crisper picture without amplifying the noise.

1. **Remove impulses first, with a 3×3 median filter** (b). Impulses are extreme values; any later contrast boost or sharpening would make them worse, and a linear blur would only spread them. The median removes them while keeping edges.
2. **Lift the shadows with a gamma of 0.45** (c). A point operation is the right tool for a global brightness problem. Doing it after the median means the curve is not distorted by black and white impulses.
3. **Build an edge-weight map** (d). Blur the image with a Gaussian ($\sigma = 2$), compute the Sobel gradient magnitude M of the *blurred* image, and normalize it to $w = \min(M/M_{90}, 1)$, where M_{90} is the 90th percentile of M. Because it is computed after smoothing, w is near 1 on real edges and near 0 on the residual noise in flat areas.
4. **Apply unsharp masking gated by that map** (f): $G = I_c + \alpha\, w \cdot (I_c - \bar{I}_c)$, with I_c the gamma-corrected image, $\bar{I}_c$ its Gaussian blur, $\alpha = 2$, and the product taken pixel by pixel.

Panel (e) shows plain unsharp masking with the same α: the background and the dark collar get grainy, because the noise is high-frequency detail too. The gated version (f) sharpens the face, hair and the suit's outlines nearly as much, while leaving the smooth background clean. This pattern,

using a smoothed first-derivative image to *decide where* a second-order sharpening is applied, is a hand-made version of the edge-aware processing that the Modern view describes.

Figure 3.8 — A four-step enhancement pipeline: (a) input, (b) median 3×3, (c) gamma 0.45, (d) edge weight from a smoothed Sobel magnitude, (e) plain unsharp masking for comparison, (f) unsharp masking gated by (d).

Each step uses one idea from this chapter. The order follows three rules of thumb: remove outliers before any step that amplifies differences, fix global tone with a point operation, and restrict sharpening to where the signal is stronger than the noise.

Modern view

The techniques in this chapter are decades old, and they are still everywhere: CLAHE is a standard preprocessing step in medical imaging and remote sensing, gamma and tone curves are in every camera pipeline, and Gaussian, median and Sobel filters are in every vision library. Research since has pushed in three directions: smarter contrast enhancement, filters that smooth without blurring edges, and learned replacements for both. The surveys below are good entry points.

Contrast enhancement and histogram methods. Pizer et al. [2] remains the reference analysis of adaptive histogram equalization: it describes how local mappings behave, why AHE over-enhances noise in uniform regions, and the variants (including contrast limiting and interpolation between tile mappings) that made the method practical. Zuiderveld's chapter [3] is the com-

pact CLAHE implementation that later libraries follow, and Reza [4] shows the same algorithm mapped to real-time hardware. On the histogram-specification side, Coltuc, Bolon and Chassery [5] show how to obtain an *exact* target histogram by strictly ordering pixels with local means. For a broader map of the field, Qi et al. [11] survey image-enhancement methods from the past two decades, organizing them into supervised and unsupervised algorithms and covering quality evaluation as a third pillar; it is useful for its side-by-side discussion of the limitations, merits and drawbacks of each family.

Edge-preserving filters. The weakness of the box and Gaussian filters is that they average across edges. The **bilateral filter** of Tomasi and Manduchi [6] fixes this by multiplying the spatial Gaussian weight by a second weight that decays with *intensity* difference, so a pixel on the other side of an edge contributes little. The survey by Paris, Kornprobst, Tumblin and Durand [7] explains its behavior, fast implementations and applications such as tone mapping and detail enhancement, where an image is split into a smooth base layer and a detail layer. The **guided filter** of He, Sun and Tang [8] assumes that the output is a local linear function of a guidance image. It is computed only with box filters of local means and variances, so its cost does not depend on the radius, and it behaves better than the bilateral filter near strong edges, where the bilateral filter can produce gradient-reversal artifacts in detail enhancement. Milanfar's tutorial [9] unifies bilateral, non-local means and related filters as *data-adaptive* weighted averages, where the kernel weights depend on the image itself, and analyzes them with linear-algebra tools. Gingichashvili and Lischinski [10] point out that the many edge-preserving filters had been compared mostly by eye, and propose a systematic methodology and common baseline for evaluating them.

How deep learning changed things. The most interesting finding is how much of this chapter survives *inside* learned systems. Many successful enhancement networks do not output pixels directly; they predict the parameters of a classical operator and then apply it.

- **Learned curves.** Zero-DCE [13] (2020) predicts pixel-wise, high-order curves (built by iterating a simple quadratic of the form $s = r + a\,r(1 - r)$, with a predicted per pixel) to brighten low-light images. It is trained without any reference images, using non-reference losses that score the enhanced result directly. It is the gamma and contrast-stretching idea, with the curve chosen per pixel by a small CNN.
- **Learned lookup tables.** Zeng et al. [14] (2020) learn a few basis 3-D color lookup tables and a tiny CNN that predicts how to blend them for each image, which runs in real time on high-resolution photos. This is the lookup-table view of intensity transformations, extended from gray levels to color.
- **Learned edge-aware filters.** HDRNet by Gharbi et al. [15] predicts local affine color transforms in a low-resolution bilateral grid and applies them at full resolution guided by the input, which is the bilateral filter's edge-awareness turned into a learned layer. Wu et al. [16] made the guided filter a differentiable layer that can be trained end-to-end inside a network.
- **Low-light enhancement.** The survey by Li et al. [12] reviews deep low-light image and video enhancement by learning strategy, network structure, loss functions and training data, and introduces a new low-light dataset captured with various phones under diverse illumination, plus an online platform for comparing methods. Later models such as Retinexformer [17] (2023) combine the Retinex model (an image as illumination times reflectance) with transformer attention guided by an illumination estimate.

What did *not* change: the median filter is still the first answer to impulse noise, the Sobel kernels still appear in loss functions and edge maps, CLAHE is still a default preprocessing step, and gamma handling remains a source of bugs in learned pipelines that mix linear and encoded pixel values. Deep models also tend to inherit the classical failure modes: halos, noise amplification in dark regions and over-enhancement are still the artifacts that reviewers look for.

Key takeaways

- An intensity transformation $s = T(r)$ is a lookup table; its slope is the local contrast gain. Log and $\gamma < 1$ curves expand the shadows, $\gamma > 1$ expands the highlights, and piecewise-linear curves give precise control.
- Histogram equalization uses the scaled CDF as T, which makes the output PDF uniform in the continuous case and approximately flat in the discrete case. Matching chains one equalization with the inverse of another; exact matching first breaks ties with local means.
- Local methods (AHE, CLAHE, local mean and variance rules) adapt to regions; CLAHE limits the contrast gain by clipping each local histogram.
- Linear spatial filtering is correlation with a kernel; convolution flips the kernel and has clean algebra. Separable kernels cost $m + n$ instead of mn operations per pixel.
- Lowpass filters (box, Gaussian) average away noise and blur edges; the nonlinear median filter is the tool for impulse noise.
- Sharpening adds highpass detail: the Laplacian (second derivative), unsharp masking and highboost, while the gradient and Sobel operators measure edge strength.
- Highpass, bandpass and bandreject kernels follow from lowpass ones by subtraction from the impulse δ; good pipelines order steps so that nothing amplifies what an earlier step should have removed.

Exercises

1. **A curve for a foggy photo.** An 8-bit image has all its pixels between 90 and 160. Write the piecewise-linear transformation that maps this range onto $[0, 255]$, give its slope, and say what happens to two neighboring pixels with values 120 and 122.

   Hint

   $T(r) = 255\,(r - 90)/70$ for $90 \le r \le 160$, clipped to $[0, 255]$ outside. The slope is $255/70 \approx 3.64$, so a difference of 2 becomes about 7.3 gray levels. After rounding, $T(120) = 109$ and $T(122) = 117$.

2. **Equalize by hand.** A tiny 3-bit image ($L = 8$) with 16 pixels has counts $n = [0, 8, 4, 2, 2, 0, 0, 0]$ for levels 0 to 7. Compute the equalization lookup table and the output histogram. Why is the output not flat?

   Hint

The CDF is $[0, 0.5, 0.75, 0.875, 1, 1, 1, 1]$, so $s_k = \text{round}(7 \cdot \text{CDF})$ gives $[0, 4, 5, 6, 7, 7, 7, 7]$ (with 3.5 rounded to 4). The output histogram has 8 pixels at level 4, 4 at 5, 2 at 6 and 2 at 7. The 8 pixels at level 1 share one input value, and a lookup table cannot split them. Only exact histogram specification could.

3. **Does order matter?** Is applying a gamma curve and then a box (averaging) filter the same as applying the box filter and then the gamma curve? Prove your answer or give a counterexample using a 2-pixel average.

Hint

No. Take a 1-D average of two pixels, 0 and 1, with $\gamma = 2$. Gamma first: $(0 + 1)/2 = 0.5$. Box first: $0.5^2 = 0.25$. A nonlinear point operation does not commute with a linear filter. This is also why blurring gamma-encoded images is not the same as blurring linear light.

4. **Separable or not?** Decide whether each kernel is separable, and if so, give the two vectors: (a) the 3×3 Laplacian with -4 in the center; (b) the 3×3 kernel with all entries 1; (c) $\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$.

Hint

(a) No: its rank is 2 (but it is the *sum* of two separable kernels, one per axis). (b) Yes: $[1, 1, 1]^\top [1, 1, 1]$. (c) Yes: $[1, 2, 1]^\top [1, 2, 1]$, a small binomial approximation of a Gaussian. Check with `np.linalg.matrix_rank`.

5. **Median versus mean on an edge.** A 1-D signal is $[10, 10, 10, 10, 50, 50, 50, 50]$, a perfect step. Apply a 3-sample mean and a 3-sample median (ignore the end samples). Then replace the sample at position 1 (counting from 0) with an impulse of value 255 and repeat. What do you conclude?

Hint

On the clean step, the mean gives a ramp $(\ldots, 10, 23.3, 36.7, 50, \ldots)$ while the median keeps the step exactly. With the impulse, the mean turns it into two raised values of about 92 at positions 1 and 2; the median outputs 10 there and removes it completely.

6. **Design a bandpass kernel.** Using Gaussians with $\sigma_1 = 1$ and $\sigma_2 = 3$, build a bandpass kernel and a bandreject kernel in NumPy. Verify the sums of their weights, filter `skimage.data.brick()` with both, and describe which structures each output keeps.

Hint

Make both kernels the same size (pad the smaller one with zeros), then use $K_1 - K_2$ and $\delta - (K_1 - K_2)$. Sums should be about 0 and 1. The bandpass output keeps the mortar lines and the brick-scale texture, removing both the overall shading and the finest grain; the bandreject output keeps shading and fine grain but weakens the brick pattern.

References

1. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018, Ch. 3. publisher page
2. S. M. Pizer, E. P. Amburn, J. D. Austin, R. Cromartie, A. Geselowitz, T. Greer, B. ter Haar Romeny, J. B. Zimmerman and K. Zuiderveld, "Adaptive histogram equalization and its variations," *Computer Vision, Graphics, and Image Processing*, 1987. doi
3. K. Zuiderveld, "Contrast limited adaptive histogram equalization," in P. S. Heckbert (ed.), *Graphics Gems IV*, Morgan Kaufmann, 1994, pp. 474–485. entry
4. A. M. Reza, "Realization of the contrast limited adaptive histogram equalization (CLAHE) for real-time image enhancement," *Journal of VLSI Signal Processing*, 2004. doi
5. D. Coltuc, P. Bolon and J.-M. Chassery, "Exact histogram specification," *IEEE Transactions on Image Processing*, 2006. doi
6. C. Tomasi and R. Manduchi, "Bilateral filtering for gray and color images," in *Proc. IEEE International Conference on Computer Vision (ICCV)*, 1998. pdf
7. S. Paris, P. Kornprobst, J. Tumblin and F. Durand, "Bilateral filtering: Theory and applications," *Foundations and Trends in Computer Graphics and Vision*, vol. 4, no. 1, pp. 1–73, 2009. doi
8. K. He, J. Sun and X. Tang, "Guided image filtering," in *Proc. European Conference on Computer Vision (ECCV)*, 2010, pp. 1–14; extended version in *IEEE TPAMI*, 2013. doi
9. P. Milanfar, "A tour of modern image filtering," *IEEE Signal Processing Magazine*, vol. 30, no. 1, pp. 106–128, 2013. IEEE Xplore
10. S. Gingichashvili and D. Lischinski, "Evaluation and comparison of edge-preserving filters," arXiv:2012.13778, 2020. arXiv
11. Y. Qi, Z. Yang, W. Sun et al., "A comprehensive overview of image enhancement techniques," *Archives of Computational Methods in Engineering*, 2021. doi
12. C. Li, C. Guo, L. Han, J. Jiang, M.-M. Cheng, J. Gu and C. C. Loy, "Low-light image and video enhancement using deep learning: A survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2021 (arXiv:2104.10729). arXiv
13. C. Guo, C. Li, J. Guo, C. C. Loy, J. Hou, S. Kwong and R. Cong, "Zero-reference deep curve estimation for low-light image enhancement," in *Proc. IEEE/CVF CVPR*, 2020. arXiv
14. H. Zeng, J. Cai, L. Li, Z. Cao and L. Zhang, "Learning image-adaptive 3D lookup tables for high performance photo enhancement in real-time," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2020. arXiv
15. M. Gharbi, J. Chen, J. T. Barron, S. W. Hasinoff and F. Durand, "Deep bilateral learning for real-time image enhancement," *ACM Transactions on Graphics (SIGGRAPH)*, 2017. arXiv
16. H. Wu, S. Zheng, J. Zhang and K. Huang, "Fast end-to-end trainable guided filter," in *Proc. IEEE/CVF CVPR*, 2018. arXiv
17. Y. Cai, H. Bian, J. Lin, H. Wang, R. Timofte and Y. Zhang, "Retinexformer: One-stage Retinex-based transformer for low-light image enhancement," in *Proc. IEEE/CVF ICCV*, 2023. arXiv

IP 04 • Filtering in the Frequency Domain

How the Fourier transform turns an image into a recipe of waves, and how editing that recipe smooths, sharpens and removes periodic noise.

Online edition: https://blog.shyandram.dev/en/learn/image-processing/dip-04-frequency-domain/

What you'll learn

- What the Fourier transform says about a signal, and why "frequency" is a useful way to describe an image
- How sampling works, what the Nyquist rate is, and why too few samples cause aliasing and moiré
- The discrete Fourier transform (DFT) in one and two dimensions, and the properties you use every day: centering, symmetry, spectrum and phase, and the convolution theorem
- The step-by-step recipe for filtering in the frequency domain, including the padding that prevents wraparound error
- Lowpass, highpass, homomorphic, bandreject and notch filters, and why the "ideal" filter rings
- How the fast Fourier transform (FFT) makes all of this cheap, and where Fourier ideas show up in modern deep learning

The big picture

In Chapter 3 we filtered images by sliding a small kernel over the pixels. This chapter looks at the same image from a completely different angle. Instead of asking "what is the brightness at this pixel?", we ask "how much of each kind of wave does this image contain?" That second description is called the image's **spectrum**, and the tool that produces it is the **Fourier transform**.

> **Explain like I'm five**
>
> Think of a song played by a band. Your ear hears one mixed sound, but a musician can tell you it is "a low bass note, a middle piano chord and a high cymbal." The Fourier transform does the same thing for a picture: it says how much slow change (big smooth areas, like the bass), and how much fast change (sharp edges and fine texture, like the cymbal) is inside. Once you know the mix, you can turn the cymbal down to make the picture smoother, or turn it up to make it crisper, and then play the song again.

Why bother? Three reasons. First, some operations are much easier to describe as "keep these frequencies, remove those" than as a kernel. Second, by the convolution theorem, a large convolution becomes a simple multiplication, which the FFT makes fast. Third, the frequency view explains things that are mysterious in the pixel view: why shrinking an image creates strange

patterns, why a sharp cutoff filter produces ripples, and why a repeating stripe of noise can be removed almost perfectly. The chapter follows the arrangement of Gonzalez and Woods [1].

Background

Plain version. Any reasonable signal can be built by adding up sine and cosine waves of different frequencies, each with its own strength and starting point. The Fourier transform finds those strengths and starting points.

In the early 1800s Joseph Fourier claimed that a periodic function can be written as a weighted sum of sines and cosines, and that even non-periodic functions with finite area under the curve can be written as an integral of sines and cosines. The striking part is that nothing is lost: you can go forward to the frequency description and back again exactly.

For images this matters in two ways. A **low frequency** is a wave that changes slowly across the image: it carries overall brightness and large smooth regions. A **high frequency** changes quickly: it carries edges, fine texture and noise. Smoothing an image means weakening the high frequencies. Sharpening means strengthening them. This chapter makes that statement precise.

Fourier analysis only became a practical image-processing tool after the fast Fourier transform appeared in 1965 [4]. Before that, computing a transform of an image was far too slow. Heideman, Johnson and Burrus later showed that Gauss had used the same idea around 1805, long before computers [5].

Preliminary concepts

Plain version. Before we can transform images, we need five tools: complex numbers (to store a wave's strength and starting point together), Fourier series (sums of waves), impulses (perfect "spikes"), the Fourier transform itself, and convolution.

Complex numbers

A complex number is $C = R + jI$, where R is the real part, I is the imaginary part, and $j = \sqrt{-1}$. It can also be written in polar form

$$C = |C|\, e^{j\theta}, \qquad |C| = \sqrt{R^2 + I^2}, \qquad \theta = \arctan(I/R)$$

where $|C|$ is the magnitude (length) and θ is the angle. Euler's formula $e^{j\theta} = \cos\theta + j\sin\theta$ links the two forms. This is why the Fourier transform uses complex exponentials: one complex number holds both the **amplitude** of a wave and its **phase** (where it starts). The complex conjugate is $C^* = R - jI$.

Fourier series

A function $f(t)$ that repeats with period T can be written as

$$f(t) = \sum_{n=-\infty}^{\infty} c_n \, e^{j\frac{2\pi n}{T}t}, \qquad c_n = \frac{1}{T}\int_{-T/2}^{T/2} f(t)\, e^{-j\frac{2\pi n}{T}t}\, dt$$

where n is an integer, c_n is the (complex) coefficient of the wave that completes n cycles per period, and t is the continuous variable (time or position).

Impulses and the sifting property

A continuous **unit impulse** $\delta(t)$ is an idealized spike: zero everywhere except at $t = 0$, with total area 1, so $\int_{-\infty}^{\infty} \delta(t)\, dt = 1$. Its most useful property is **sifting**:

$$\int_{-\infty}^{\infty} f(t)\, \delta(t - t_0)\, dt = f(t_0)$$

where t_0 is any location. Multiplying by a shifted impulse and integrating "picks out" the value of f at t_0. The discrete impulse $\delta(x)$ is 1 at $x = 0$ and 0 elsewhere, and sifting becomes $\sum_x f(x)\, \delta(x - x_0) = f(x_0)$.

An **impulse train** $s_{\Delta T}(t) = \sum_{n=-\infty}^{\infty} \delta(t - n\Delta T)$ is a row of impulses spaced ΔT apart. It is the mathematical model of sampling.

The Fourier transform of a continuous function

The Fourier transform of $f(t)$ and its inverse are

$$F(\mu) = \int_{-\infty}^{\infty} f(t)\, e^{-j2\pi\mu t}\, dt, \qquad f(t) = \int_{-\infty}^{\infty} F(\mu)\, e^{j2\pi\mu t}\, d\mu$$

where μ is the continuous frequency (cycles per unit of t) and $F(\mu)$ is complex in general. Together the two equations are called a **Fourier transform pair**, written $f(t) \Leftrightarrow F(\mu)$.

A worked example builds intuition. Take a box of height A that is nonzero for $|t| \le W/2$. Its transform is

$$F(\mu) = AW\,\frac{\sin(\pi\mu W)}{\pi\mu W} = AW\ \mathrm{sinc}(\mu W)$$

where $\mathrm{sinc}(m) = \sin(\pi m)/(\pi m)$. Two lessons follow. A function with sharp edges has a transform that spreads out with slowly decaying ripples. And there is an inverse relationship: a wider box gives a narrower sinc. Narrow in one domain means wide in the other. We will meet both lessons again when the "ideal" filter rings.

Two more pairs are worth memorizing. The transform of an impulse is a constant: $\delta(t) \Leftrightarrow 1$. And the transform of an impulse train with spacing ΔT is another impulse train with spacing $1/\Delta T$:

$$s_{\Delta T}(t) \Leftrightarrow S(\mu) = \frac{1}{\Delta T} \sum_{n=-\infty}^{\infty} \delta\left(\mu - \frac{n}{\Delta T}\right)$$

Convolution

The convolution of two continuous functions is

$$(f \star h)(t) = \int_{-\infty}^{\infty} f(\tau)\, h(t-\tau)\, d\tau$$

where τ is a dummy integration variable. The **convolution theorem** says

$$(f \star h)(t) \Leftrightarrow H(\mu)F(\mu), \qquad f(t)\, h(t) \Leftrightarrow (H \star F)(\mu)$$

Convolution in one domain is multiplication in the other. This single fact is the reason frequency-domain filtering exists.

Sampling and the Fourier transform of sampled functions

Plain version. A camera does not record a continuous scene; it measures it at a grid of points. If the points are too far apart, fast details get confused with slow ones and you see patterns that are not really there. Sample at more than twice the fastest frequency and you can, in principle, rebuild the original exactly.

Sampling as multiplication

Model sampling as multiplying $f(t)$ by an impulse train:

$$\tilde{f}(t) = f(t)\, s_{\Delta T}(t) = \sum_{n=-\infty}^{\infty} f(n\Delta T)\, \delta(t - n\Delta T)$$

where ΔT is the sampling interval and $f(n\Delta T)$ are the samples. By the convolution theorem, multiplication becomes convolution with $S(\mu)$, and convolving with a train of impulses just copies the spectrum:

$$\tilde{F}(\mu) = \frac{1}{\Delta T} \sum_{n=-\infty}^{\infty} F\left(\mu - \frac{n}{\Delta T}\right)$$

So **the spectrum of a sampled function is an infinite, periodic sequence of copies of the original spectrum**, spaced $1/\Delta T$ apart.

The sampling theorem and the Nyquist rate

Suppose f is **band-limited**: $F(\mu) = 0$ for $|\mu| > \mu_{\max}$. The copies do not overlap as long as their spacing exceeds the width of one copy:

$$\frac{1}{\Delta T} > 2\mu_{\max}$$

This is the **sampling theorem**, made famous in engineering by Shannon [2]. The quantity $2\mu_{\max}$ is the **Nyquist rate**. Sampling faster than it means the samples contain all the information in f.

Reconstruction

If the copies do not overlap, multiplying $\tilde{F}(\mu)$ by a box $H(\mu)$ of height ΔT that covers $[-\mu_{\max}, \mu_{\max}]$ isolates the original spectrum. In the time domain this is convolution with a sinc, giving the interpolation formula

$$f(t) = \sum_{n=-\infty}^{\infty} f(n\Delta T)\ \operatorname{sinc}\left[\frac{t - n\Delta T}{\Delta T}\right]$$

Every sample contributes a sinc centered on itself, and the sum passes through all samples exactly. Unser's review [3] explains how modern practice replaces the impractical infinite sinc with splines and other compact kernels that approximate it well.

Aliasing

If $1/\Delta T \le 2\mu_{\max}$, neighboring copies overlap and their tails add together. No filter can separate them afterwards: high frequencies now masquerade as low ones. This is **aliasing**. The only cure is prevention: remove the high frequencies with a lowpass (**anti-aliasing**) filter *before* sampling.

A one-line experiment shows the disguise. Sample a 9 Hz cosine at 10 samples per second, where the Nyquist limit is 5 Hz:

```
import numpy as np

fs = 10.0                       # samples per second
t = np.arange(0, 2, 1 / fs)     # 2 seconds of samples
x = np.cos(2 * np.pi * 9 * t)  # a 9 Hz tone: above fs/2 = 5 Hz
X = np.abs(np.fft.rfft(x))
freqs = np.fft.rfftfreq(len(x), d=1 / fs)
print("peak at", freqs[X.argmax()], "Hz")   # -> 1.0 Hz, not 9 Hz
```

The 9 Hz tone is indistinguishable from a 1 Hz tone at these sample times. This is the same effect that makes wagon wheels appear to spin backwards in films.

The discrete Fourier transform of one variable

Plain version. A computer only has a finite list of samples, so it uses a finite version of the Fourier transform. The DFT takes M numbers and returns M numbers, each saying how much of one particular wave is present.

Definition

For samples $f(0), f(1), \ldots, f(M-1)$:

$$F(u) = \sum_{x=0}^{M-1} f(x)\, e^{-j2\pi ux/M}, \qquad f(x) = \frac{1}{M} \sum_{u=0}^{M-1} F(u)\, e^{j2\pi ux/M}$$

where $x = 0, \ldots, M-1$ indexes samples, $u = 0, \ldots, M-1$ indexes discrete frequencies (frequency u completes u cycles over the M samples), and the factor $1/M$ sits on the inverse (other conventions split it as $1/\sqrt{M}$ on each side).

Relation to sampling

Where does this come from? Take the transform of the sampled function $\tilde{F}(\mu)$, which is continuous and periodic with period $1/\Delta T$. Sample one period of it at M equally spaced points, $\mu = m/(M\Delta T)$ for $m = 0, \ldots, M-1$. The result is exactly the DFT above. So the DFT is **samples of one period of the transform of the sampled data**. The spacing between DFT frequencies is

$$\Delta u = \frac{1}{M\Delta T}$$

where $M\Delta T$ is the total duration covered by the samples. Longer recordings give finer frequency resolution.

Two consequences matter in practice. Because we sampled in frequency as well, the DFT treats the data as if it repeats with period M. And both $F(u)$ and $f(x)$ are periodic: $F(u) = F(u+M)$ and $f(x) = f(x+M)$.

The definition is just a matrix-vector product, which is easy to check:

```
import numpy as np

def dft(f):
    M = len(f)
    x = np.arange(M)
    u = x.reshape(-1, 1)
    W = np.exp(-2j * np.pi * u * x / M)   # M x M matrix of complex exponentials
    return W @ f

f = np.random.default_rng(0).standard_normal(64)
print(np.allclose(dft(f), np.fft.fft(f)))  # True
```

Extension to functions of two variables

Plain version. Images have two directions, so the waves have two frequencies: one across and one down. A 2-D wave looks like a set of parallel stripes; its two frequencies set how close the stripes are and which way they point.

The 2-D impulse and continuous transform

The 2-D impulse $\delta(t, z)$ has unit volume and sifts like its 1-D cousin: $\iint f(t, z)\, \delta(t - t_0, z - z_0)\, dt\, dz = f(t_0, z_0)$. The 2-D continuous transform is

$$F(\mu, \nu) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(t, z)\, e^{-j2\pi(\mu t + \nu z)}\, dt\, dz$$

where t, z are the spatial coordinates and μ, ν are the corresponding frequencies.

2-D sampling and aliasing

Sampling on a grid with spacings ΔT and ΔZ replicates the spectrum on a 2-D lattice. A band-limited image (zero outside $|\mu| \le \mu_{\max}$, $|\nu| \le \nu_{\max}$) is recoverable when

$$\frac{1}{\Delta T} > 2\mu_{\max}, \qquad \frac{1}{\Delta Z} > 2\nu_{\max}$$

Images break this rule constantly. Real scenes contain sharp edges, which are not band-limited at all, so every digital image has some aliasing. It becomes visible when you **downsample** (shrink) an image by dropping pixels. Figure 4.1 uses a zone plate, a pattern whose frequency increases with distance from the center. Keeping every 4th pixel turns the fine outer rings into false circles that were never in the original. Blurring first removes the frequencies that cannot survive, and the false rings disappear.

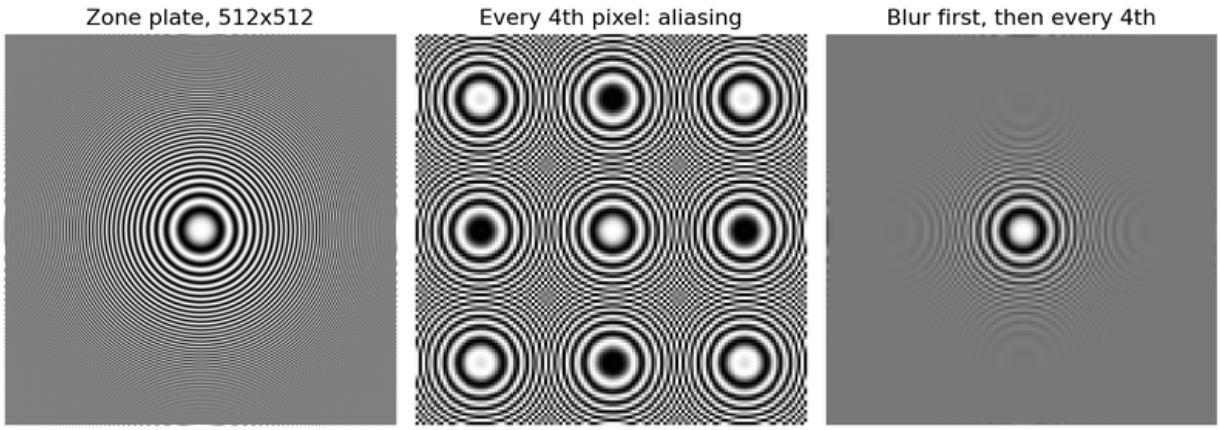


Figure 4.1 — Aliasing in 2-D. Left: a zone plate whose frequency grows outward. Middle: keeping every 4th pixel produces false rings (aliases). Right: a Gaussian blur before subsampling removes the unrepresentable frequencies, leaving a flat gray where the detail was.

Moiré patterns

A **moiré pattern** is aliasing you can see in everyday life: when a fine periodic texture (a shirt's weave, a screen door, the halftone dots of a printed photo) is sampled by a sensor grid of similar

spacing, the two periodicities beat against each other and produce large, slow, wavy bands. The middle panel of Figure 4.1 is a moiré pattern of exactly this kind. Cameras reduce it with an optical anti-aliasing filter in front of the sensor; software reduces it by blurring before resizing.

The 2-D DFT

For an $M \times N$ image $f(x, y)$:

$$F(u, v) = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f(x, y)\, e^{-j2\pi\left(\frac{ux}{M} + \frac{vy}{N}\right)}, \qquad f(x, y) = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F(u, v)\, e^{j2\pi\left(\frac{ux}{M} + \frac{vy}{N}\right)}$$

where x, y are pixel coordinates (rows, columns), u, v are discrete frequency indices, and $F(u, v)$ is an $M \times N$ array of complex numbers. Every pixel of F describes one 2-D wave covering the whole image.

Properties of the 2-D DFT

Plain version. The spectrum behaves in predictable ways. Moving an object does not change the spectrum's magnitude. Rotating the object rotates the spectrum. The center of the (shifted) spectrum holds the average brightness. And the most surprising fact: most of what we recognize in a picture is stored in the *phase*, not the magnitude.

Spatial and frequency intervals

If the image was sampled with spacings ΔT and ΔZ, the frequency spacings of the DFT are

$$\Delta u = \frac{1}{M \Delta T}, \qquad \Delta v = \frac{1}{N \Delta Z}$$

A larger field of view gives finer frequency resolution; finer pixels extend the highest frequency the DFT can represent.

Translation and rotation

Shifting the image multiplies the transform by a linear phase, and vice versa:

$$f(x - x_0, y - y_0) \Leftrightarrow F(u, v)\, e^{-j2\pi\left(\frac{ux_0}{M} + \frac{vy_0}{N}\right)}$$

where (x_0, y_0) is the shift. Because $|e^{-j\cdot}| = 1$, **the magnitude $|F|$ is unchanged by translation**; only the phase changes. With polar coordinates $x = r\cos\theta$, $y = r\sin\theta$ and $u = \omega\cos\varphi$, $v = \omega\sin\varphi$, rotating f by θ_0 rotates F by the same angle: $f(r, \theta + \theta_0) \Leftrightarrow F(\omega, \varphi + \theta_0)$.

Periodicity and centering

The DFT and its inverse are periodic in both directions: $F(u,v) = F(u + k_1 M, v + k_2 N)$ for any integers k_1, k_2. A raw DFT array therefore has its zero frequency in the top-left corner, with four quarter-periods meeting at the center. For viewing and for designing filters it is much easier to move zero frequency to the center. Using the translation property with $x_0 = M/2$, $y_0 = N/2$:

$$f(x,y)\,(-1)^{x+y} \Leftrightarrow F(u - M/2,\ v - N/2)$$

which holds for even M and N. In NumPy, `np.fft.fftshift` does the same reordering after the transform.

Symmetry

For a real image, the transform is **conjugate symmetric**: $F(-u,-v) = F^*(u,v)$. Its magnitude is therefore even, $|F(-u,-v)| = |F(u,v)|$, and its phase is odd. Every spike in a spectrum comes with a mirror twin through the center. You will use this when designing notch filters.

Fourier spectrum and phase angle

Write $F(u,v) = R(u,v) + jI(u,v)$. Then

$$|F(u,v)| = \sqrt{R^2 + I^2}, \qquad \phi(u,v) = \arctan\left[\frac{I(u,v)}{R(u,v)}\right], \qquad P(u,v) = |F(u,v)|^2$$

are the **Fourier spectrum** (magnitude), the **phase angle** and the **power spectrum**. The zero-frequency term is the **DC component**, $F(0,0) = MN\bar{f}$, where $\bar{f}$ is the mean intensity. Because $F(0,0)$ is usually thousands of times larger than everything else, spectra are displayed as $\log(1 + |F|)$.

```
import numpy as np
from skimage import data, img_as_float

f = img_as_float(data.camera())
F = np.fft.fft2(f)
Fc = np.fft.fftshift(F)                       # move (0,0) to the centre
spectrum = np.log1p(np.abs(Fc))               # log(1+|F|) for display
phase = np.angle(Fc)

M, N = f.shape
y, x = np.mgrid[0:M, 0:N]
print(np.allclose(np.fft.fft2(f * (-1.0) ** (x + y)), Fc))  # True for even M, N
print(np.isclose(F[0, 0].real, f.sum()))                    # DC term = sum of pixels
```

Figure 4.2 shows how simple patterns map to spectra. A rectangle gives a 2-D sinc: a bright cross whose arms are longer along the rectangle's *short* side (narrow in space, wide in frequency). Shifting the rectangle leaves the spectrum identical. Rotating it rotates the spectrum. A cosine

grating is a single 2-D frequency, so its spectrum is just three dots: the DC term and a symmetric pair. A Gaussian transforms into another Gaussian.

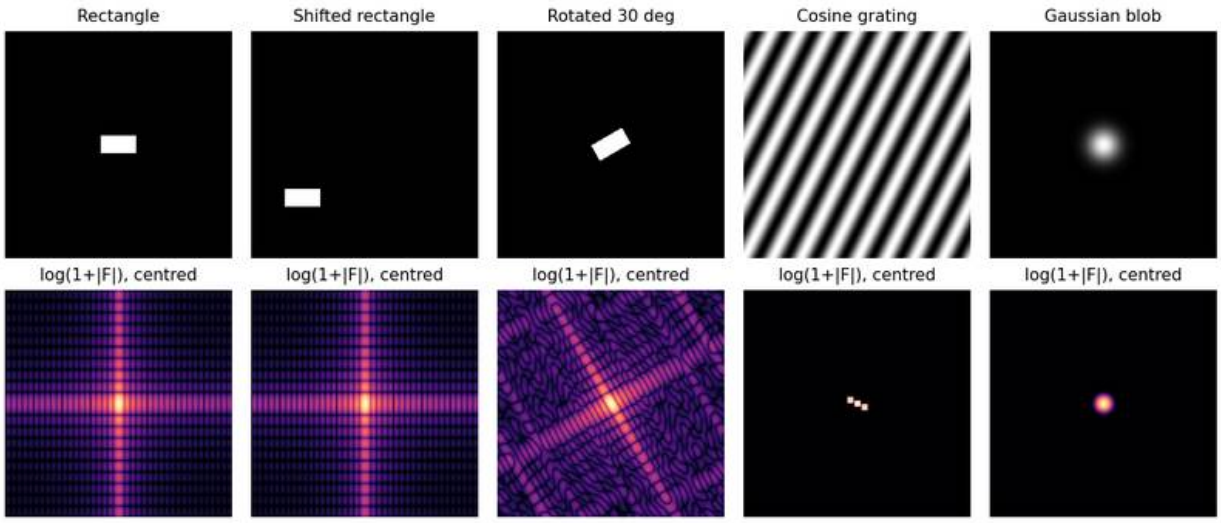


Figure 4.2 — Simple patterns (top) and their centered log spectra (bottom). Translation leaves the magnitude unchanged; rotation rotates it. The grating's three spectral dots are enlarged so they are visible.

The phase-versus-magnitude experiment

Which part of the transform carries the content we recognize? Take two images, keep the magnitude of one and the phase of the other, and invert. Oppenheim and Lim made this experiment famous [7].

```
import numpy as np
from skimage import color, data, img_as_float

a = img_as_float(data.camera())
b = img_as_float(color.rgb2gray(data.astronaut()))
A, B = np.fft.fft2(a), np.fft.fft2(b)
mixed = np.real(np.fft.ifft2(np.abs(A) * np.exp(1j * np.angle(B))))
# 'mixed' looks like the astronaut, not the cameraman
```

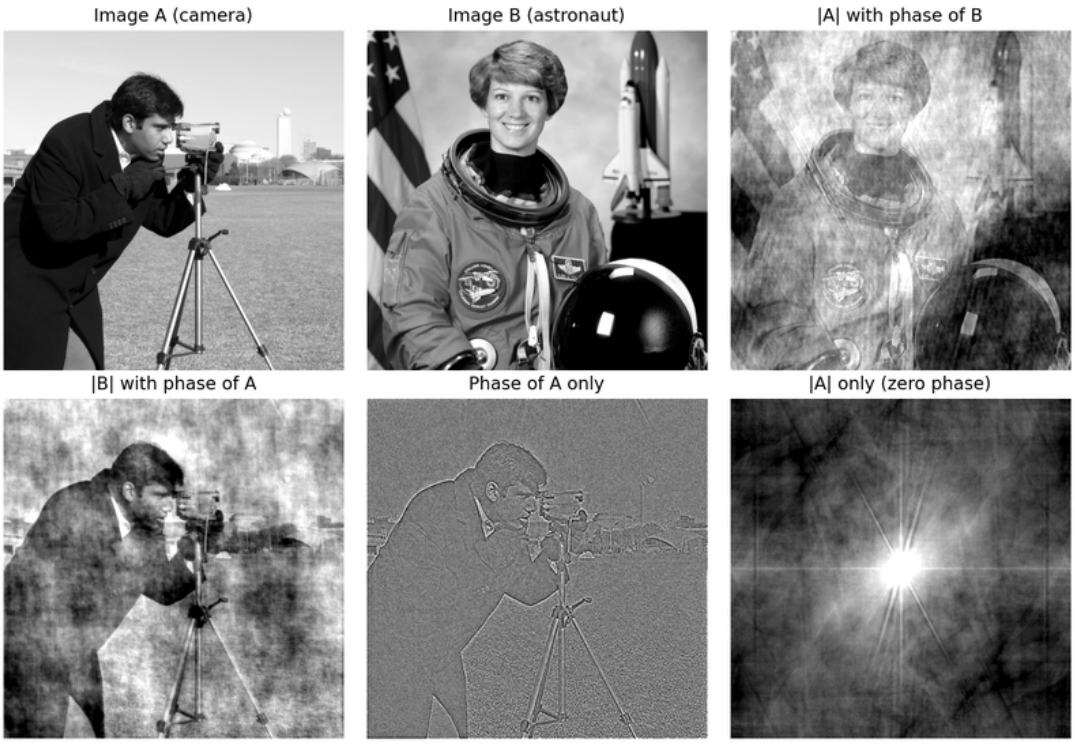


Figure 4.3 — Swapping magnitude and phase. Each hybrid looks like the image whose phase it borrowed. The phase alone (unit magnitude) keeps the edges and layout; the magnitude alone (zero phase) is an unrecognizable blob.

The result is unambiguous: **phase carries the structure**. The magnitude tells you how much energy sits at each frequency, which is roughly similar for most natural photos. The phase tells you where the waves line up, and edges are exactly the places where many waves line up. This is why the filters below are designed to leave phase untouched.

The 2-D convolution theorem

The discrete version of the convolution theorem is

$$(f \star h)(x,y) \;\Leftrightarrow\; F(u,v)\,H(u,v), \qquad (f \star h)(x,y) = \sum_{m=0}^{M-1}\sum_{n=0}^{N-1} f(m,n)\,h(x-m,\;y-n)$$

where the indices $x-m$ and $y-n$ are taken **modulo** M and N. This is **circular convolution**: the DFT assumes the image repeats forever, so whatever is pushed off the right edge reappears on the left.

Wraparound error and padding

Circular convolution is not what we want; it mixes the left and right borders of the image. This is **wraparound error**. The fix is to pad both arrays with zeros so the periods are far enough apart that the copies never touch. If f is $A \times B$ and h is $C \times D$, choose a padded size $P \times Q$ with

$$P \geq A + C - 1, \qquad Q \geq B + D - 1$$

When the filter is specified in the frequency domain and is the same size as the image, the usual choice is simply $P = 2M$, $Q = 2N$.

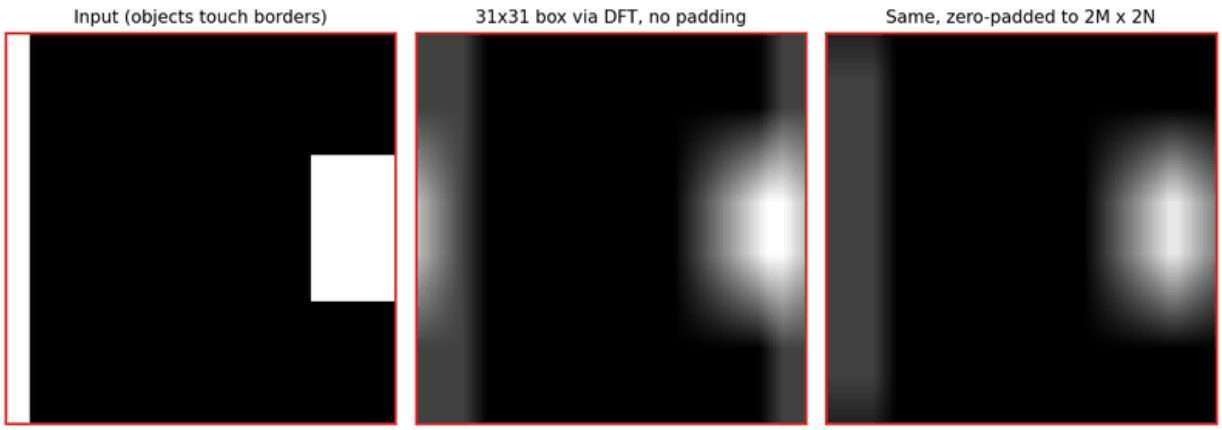


Figure 4.4 — Wraparound error. Without padding (middle), the blur leaks the right-hand block onto the left border and the left bar onto the right border. Zero-padding to twice the size (right) gives the correct linear convolution.

Basics of filtering in the frequency domain

Plain version. Transform the image, multiply each frequency by a number that says "keep" or "remove", and transform back. The list of numbers is the filter.

The filtering procedure

Given an $M \times N$ image $f(x, y)$ and a filter transfer function $H(u, v)$:

1. Choose padding sizes $P = 2M$ and $Q = 2N$.
2. Form the padded image $f_p(x, y)$ of size $P \times Q$ by appending zeros (or mirrored pixels).
3. Multiply f_p by $(-1)^{x+y}$ to center its transform.
4. Compute the DFT, $F(u, v)$.
5. Build a real, symmetric filter $H(u, v)$ of size $P \times Q$ with its center at $(P/2, Q/2)$, and form $G(u, v) = H(u, v)\, F(u, v)$.
6. Compute $g_p(x, y) = \mathrm{Re}\{\mathcal{F}^{-1}[G(u, v)]\}\,(-1)^{x+y}$.

7. Crop the top-left $M \times N$ region to get $g(x, y)$.

Taking the real part in step 6 only discards tiny imaginary values caused by floating-point rounding; with a real image and a symmetric real filter the exact result is real.

```
import numpy as np

def filter_freq(f, H_of_D):
    """Filter image f with a radially symmetric, zero-phase transfer function.
    H_of_D maps a distance array D (from the centre of the P x Q grid) to H."""
    M, N = f.shape
    P, Q = 2 * M, 2 * N                                  # 1. padding size
    fp = np.zeros((P, Q)); fp[:M, :N] = f                # 2. zero-pad
    y, x = np.mgrid[0:P, 0:Q]
    fp = fp * (-1.0) ** (x + y)                          # 3. centre the transform
    F = np.fft.fft2(fp)                                  # 4. DFT
    D = np.hypot(y - P / 2, x - Q / 2)
    G = H_of_D(D) * F                                    # 5. multiply
    gp = np.real(np.fft.ifft2(G)) * (-1.0) ** (x + y)    # 6. inverse, undo centring
    return gp[:M, :N]                                    # 7. crop
```

Zero padding darkens a thin band along the right and bottom borders after blurring, because the filter averages in black pixels. Padding by mirroring the image (`np.pad(f, ..., mode="reflect")`) avoids this, which is what the figure script for this chapter does.

Zero-phase filters

Every filter in this chapter is **real** and **symmetric** about the center. Multiplying a complex number by a real number scales its magnitude and leaves its angle alone (up to a sign), so these filters change *how much* of each frequency remains but never *where* the waves line up. Such filters are called **zero-phase-shift** filters. After Figure 4.3 you can see why this is a requirement: changing the phase would move edges around.

Correspondence between spatial and frequency filters

By the convolution theorem, every frequency filter $H(u, v)$ has a spatial kernel $h(x, y) = \mathcal{F}^{-1}\{H(u, v)\}$, and every spatial kernel has a transfer function. The Gaussian pair is the cleanest example. In one dimension,

$$H(u) = A\, e^{-u^2/2\sigma^2} \;\Leftrightarrow\; h(x) = \sqrt{2\pi}\,\sigma A\, e^{-2\pi^2\sigma^2 x^2}$$

where σ is the width of the Gaussian in frequency. A wide frequency Gaussian corresponds to a narrow spatial kernel and vice versa. A common practical strategy is therefore to *design* in the frequency domain, where "pass below 40 cycles" is easy to state, and then *implement* a small spatial kernel that approximates h.

When is the frequency route faster? Spatial convolution with a $k \times k$ kernel costs about k^2 multiplications per pixel. The FFT route costs a fixed amount per pixel that grows only with $\log(MN)$. For small kernels (3×3, 5×5) the spatial route wins; for kernels tens of pixels wide, the FFT route wins.

Image smoothing using lowpass frequency-domain filters

Plain version. A lowpass filter keeps the slow waves and removes the fast ones, so the image gets blurrier. The way the filter drops from "keep" to "remove" matters a lot: a sudden drop creates ripples.

Let $D(u, v)$ be the distance from the center of the $P \times Q$ frequency rectangle,

$$D(u, v) = \sqrt{(u - P/2)^2 + (v - Q/2)^2}$$

and let $D_0 > 0$ be the **cutoff frequency**.

Ideal lowpass filter (ILPF)

$$H(u, v) = \begin{cases} 1 & D(u, v) \le D_0 \\ 0 & D(u, v) > D_0 \end{cases}$$

It passes everything inside a circle of radius D_0 and nothing outside. A useful way to choose D_0 is by the fraction of total power it keeps: the percentage $\alpha = 100 \sum_{D \le D_0} P(u, v) / \sum_{u,v} P(u, v)$.

Butterworth lowpass filter (BLPF)

$$H(u, v) = \frac{1}{1 + [D(u, v)/D_0]^{2n}}$$

where n is the **order**. At $D = D_0$ the filter is exactly 0.5. Small n gives a gentle roll-off; as $n \to \infty$ the BLPF approaches the ideal filter.

Gaussian lowpass filter (GLPF)

$$H(u, v) = e^{-D^2(u,v)/2D_0^2}$$

where D_0 plays the role of σ. At $D = D_0$ the filter is $e^{-1/2} \approx 0.607$.

Ringing

Figure 4.5 compares the three with the same cutoff. The ideal filter produces visible **ringing**: ghostly copies of edges repeated outward, like ripples around a stone thrown in a pond. The reason is the box-sinc pair from earlier. A sharp cutoff in frequency corresponds to a spatial kernel with a central lobe surrounded by rings of alternating sign. Convolving an edge with that kernel draws a series of faint bright and dark echoes beside it.

The Gaussian is the opposite extreme: its spatial kernel is also Gaussian, always positive, so it can never ring. The Butterworth is in between: order 1 has no ringing, order 2 has a barely visible negative lobe, and higher orders ring more and more. Order 2 is a popular compromise between a sharp cutoff and clean results.

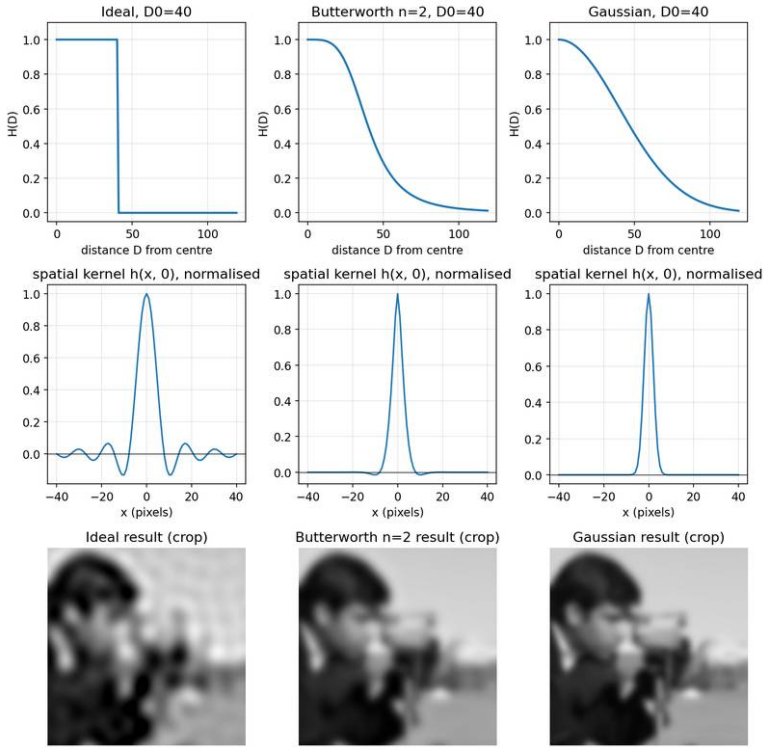


Figure 4.5 — Ideal, Butterworth ($n = 2$) and Gaussian lowpass filters with the same cutoff $D0 = 40$. Top: radial profile $H(D)$. Middle: the corresponding spatial kernel; only the ideal filter has strong negative side lobes. Bottom: the ideal filter's result shows ripples around edges (ringing); the Gaussian's does not.

> **Interactive demonstration (online)**: open the online edition of this tutorial to use this demonstration.Open demonstration

```
D0 = 40
ideal  = lambda D: (D <= D0).astype(float)
butter = lambda D, n=2: 1 / (1 + (D / D0) ** (2 * n))
```

```
gauss  = lambda D: np.exp(-D**2 / (2 * D0**2))
smooth = filter_freq(f, gauss)
```

Lowpass filtering has many quiet uses: joining broken character strokes in a low-resolution scan, softening skin texture in portraits, and suppressing scan-line patterns in remote-sensing images.

Image sharpening using highpass filters

Plain version. A highpass filter is the opposite: it throws away the slow waves and keeps the fast ones, so only edges and texture remain. Add some of that back to the original and the image looks sharper.

Ideal, Butterworth and Gaussian highpass filters

Every highpass filter can be made from a lowpass one:

$$H_{\mathrm{HP}}(u,v) = 1 - H_{\mathrm{LP}}(u,v)$$

So the Gaussian highpass is $1 - e^{-D^2/2D_0^2}$, the Butterworth highpass is $1/(1 + [D_0/D]^{2n})$, and the ideal highpass is 0 inside the circle and 1 outside. The same ringing story applies: ideal rings, Gaussian does not. Because the highpass sets $H(0,0) = 0$, the output has zero mean, so flat regions become gray (zero) after display scaling.

The Laplacian in the frequency domain

The Laplacian $\nabla^2 f = \partial^2 f/\partial x^2 + \partial^2 f/\partial y^2$ from Chapter 3 has a transfer function. Differentiation multiplies the transform by $j2\pi u$, so differentiating twice in each direction gives

$$H(u,v) = -4\pi^2 \left[(u - P/2)^2 + (v - Q/2)^2\right] = -4\pi^2 D^2(u,v)$$

with frequencies measured in cycles per image. The sharpened image is $g(x,y) = f(x,y) - \nabla^2 f(x, y)$, with the minus sign because the center of H is negative. In practice the Laplacian output is far larger than f, so both must be scaled to comparable ranges (for example, f in $[0,1]$ and $\nabla^2 f$ divided by its maximum absolute value) before subtracting.

Unsharp masking, high-boost and high-frequency emphasis

Unsharp masking subtracts a blurred version to obtain a "mask" of details and adds it back:

$$g_{\mathrm{mask}}(x,y) = f(x,y) - f_{\mathrm{LP}}(x,y), \qquad g(x,y) = f(x,y) + k\, g_{\mathrm{mask}}(x,y)$$

where f_{LP} is a lowpass-filtered f and $k \geq 0$ is a weight ($k = 1$: unsharp masking; $k > 1$: high-boost). In the frequency domain the whole operation is one transfer function, $H = 1 + k\, H_{\mathrm{HP}}$. Generalizing gives **high-frequency emphasis**:

$$H_{\text{HFE}}(u,v) = k_1 + k_2\, H_{\text{HP}}(u,v)$$

where $k_1 \geq 0$ controls how much of the low frequencies (and the DC term) survives and $k_2 \geq 0$ controls how strongly high frequencies are boosted. Choosing $k_1 < 1$ also lowers overall contrast, which is why high-frequency emphasis is often followed by histogram equalization.

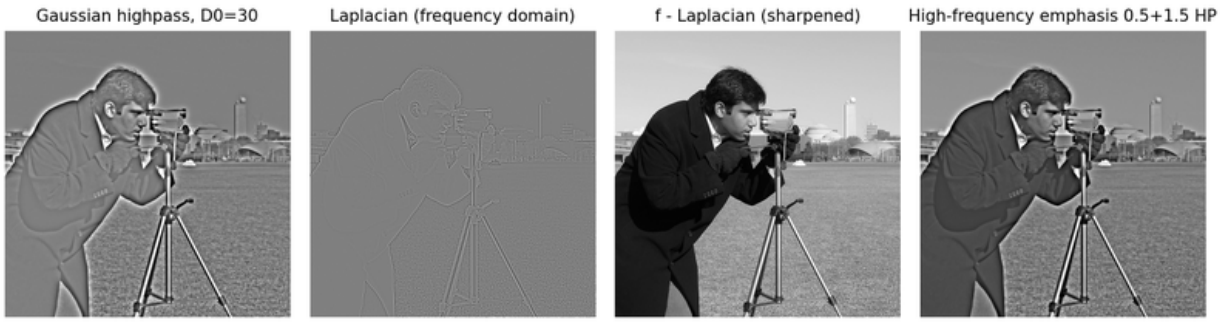


Figure 4.6 — Sharpening in the frequency domain. From left: Gaussian highpass ($D0 = 30$), the Laplacian computed with $H = -4\pi^2 D^2$, the image minus its scaled Laplacian, and high-frequency emphasis with k1 = 0.5, k2 = 1.5.

Homomorphic filtering

A simple image-formation model says each pixel is the product of **illumination** $i(x,y)$ (how much light falls on the scene) and **reflectance** $r(x,y)$ (how much the surface reflects): $f = i \cdot r$. Illumination varies slowly across a scene; reflectance changes abruptly at object boundaries. We would like to squash the illumination and boost the reflectance, but the Fourier transform of a product is not the product of transforms.

The trick, introduced by Oppenheim, Schafer and Stockham [8], is to take the logarithm first, which turns the product into a sum:

$$z(x,y) = \ln f(x,y) = \ln i(x,y) + \ln r(x,y)$$

Now filter z with a transfer function that treats low and high frequencies differently, and exponentiate:

$$s = \mathcal{F}^{-1}\{H\, Z\}, \qquad g(x,y) = e^{s(x,y)}, \qquad H(u,v) = (\gamma_H - \gamma_L)\left[1 - e^{-c\, D^2(u,v)/D_0^2}\right] + \gamma_L$$

where $\gamma_L < 1$ attenuates low frequencies (illumination), $\gamma_H > 1$ amplifies high frequencies (reflectance), c controls the steepness of the transition and D_0 its location. The result compresses the dynamic range and enhances contrast at the same time.

```
def homomorphic(f, gL=0.4, gH=1.6, c=1.0, D0=30, eps=1e-3):
    z = np.log(f + eps)                          # eps avoids log(0)
    H = lambda D: (gH - gL) * (1 - np.exp(-c * D**2 / D0**2)) + gL
    return np.exp(filter_freq(z, H)) - eps
```

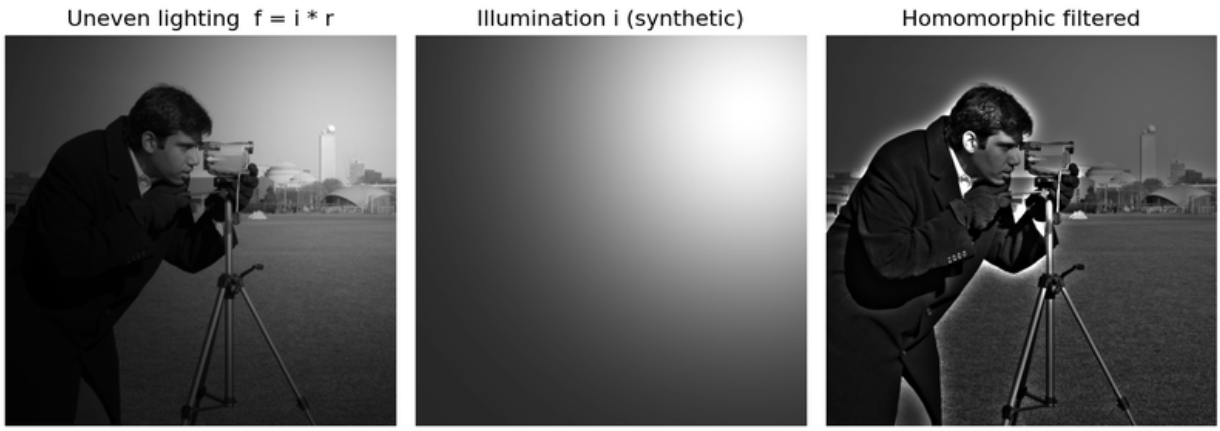


Figure 4.7 — Homomorphic filtering. A synthetic illumination field darkens the left side of the image (left, middle). Filtering in the log domain with $\gamma L = 0.4$ and $\gamma H = 1.6$ evens out the lighting and boosts detail in the dark areas (right), at the cost of mild halos around strong edges.

Selective filtering

Plain version. Sometimes the problem is not "too much detail" or "too little" but one specific, repeating pattern, like electrical interference stripes. In the spectrum such a pattern is a few bright dots. A selective filter removes just those dots and nothing else.

Bandreject and bandpass filters

A **bandreject** filter removes a ring of frequencies centered at radius C_0 with width W. The Gaussian version is

$$H_{\mathrm{BR}}(u,v) = 1 - \exp\left[-\left(\frac{D^2(u,v) - C_0^2}{D(u,v)\,W}\right)^2\right]$$

and the Butterworth version of order n is

$$H_{\mathrm{BR}}(u,v) = \frac{1}{1 + \left[\dfrac{D(u,v)\,W}{D^2(u,v) - C_0^2}\right]^{2n}}$$

where C_0 is the ring's center radius and W its width. A **bandpass** filter keeps only that ring: $H_{\mathrm{BP}} = 1 - H_{\mathrm{BR}}$. Ring-shaped filters suit noise that appears at a fixed distance from the center in all directions, which is uncommon in practice.

Notch filters

A **notch filter** acts on small neighborhoods around chosen frequencies. Because a real image's spectrum is conjugate symmetric, every notch must come with its mirror image through the center. A notch-reject filter with Q notch pairs is built as a product of highpass filters whose centers are moved to the spike locations:

$$H_{\mathrm{NR}}(u,v) = \prod_{k=1}^{Q} H_k(u,v)\, H_{-k}(u,v)$$

where H_k is a highpass filter (ideal, Butterworth or Gaussian) centered at (u_k, v_k) and H_{-k} is the same filter centered at $(-u_k, -v_k)$, both measured relative to the center of the frequency rectangle. The distances are

$$D_k(u,v) = \sqrt{(u - P/2 - u_k)^2 + (v - Q/2 - v_k)^2}, \qquad D_{-k}(u,v) = \sqrt{(u - P/2 + u_k)^2 + (v - Q/2 + v_k)^2}$$

A notch-pass filter is $1 - H_{\mathrm{NR}}$. Applying the notch-pass filter and inverting isolates the interference pattern itself, which is a useful check that you removed noise and not image content.

```
import numpy as np
from skimage import data, img_as_float

f = img_as_float(data.camera())
M, N = f.shape
y, x = np.mgrid[0:M, 0:N]
g = f + 0.2 * np.cos(2 * np.pi * (40 * x / N + 25 * y / M))   # add a ripple

G = np.fft.fftshift(np.fft.fft2(g))
u, v = y - M // 2, x - N // 2              # centred frequency coordinates
H = np.ones((M, N))
for uk, vk in [(25, 40)]:                   # spike location (row, column offset)
    for s in (+1, -1):                      # the conjugate-symmetric partner
        Dk = np.hypot(u - s * uk, v - s * vk)
        H *= 1 - np.exp(-Dk**2 / (2 * 4.0**2))   # Gaussian notch reject
out = np.real(np.fft.ifft2(np.fft.ifftshift(G * H)))
```

Padding is skipped here on purpose. The interference is periodic over the whole image, and padding would smear its spike across many frequency samples, making the notch less effective.

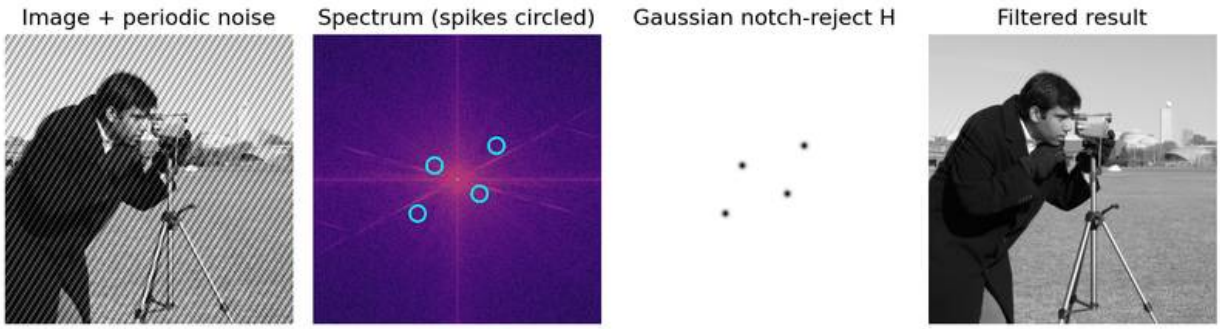


Figure 4.8 — Notch filtering of synthetic periodic noise. Two sinusoidal ripples produce four bright spikes (two conjugate pairs, circled). A Gaussian notch-reject filter with a dark dot at each spike removes them, and the interference disappears.

Drag the slider to compare the corrupted and filtered images directly:

With periodic noise

Notch filtered

Cameraman with diagonal stripe interference compared with the notch-filtered result

Implementation

Plain version. A 2-D DFT can be computed as many 1-D DFTs, an inverse DFT can be computed with the forward routine, and the FFT algorithm makes each 1-D transform dramatically faster.

Separability

The 2-D kernel $e^{-j2\pi(ux/M+vy/N)}$ factors into $e^{-j2\pi ux/M}\,e^{-j2\pi vy/N}$. So

$$F(u,v) = \sum_{x=0}^{M-1} e^{-j2\pi ux/M} \underbrace{\sum_{y=0}^{N-1} f(x,y)\, e^{-j2\pi vy/N}}_{F(x,v)}$$

where $F(x,v)$ is the 1-D DFT of row x. Compute a 1-D DFT along every row, then a 1-D DFT along every column of the result. Any 1-D FFT routine is enough to build a 2-D one.

Computing the inverse DFT with a forward transform

Take the complex conjugate of the inverse formula and multiply by MN:

$$MN\, f^*(x,y) = \sum_{u=0}^{M-1}\sum_{v=0}^{N-1} F^*(u,v)\, e^{-j2\pi\left(\frac{ux}{M}+\frac{vy}{N}\right)}$$

The right side is a *forward* DFT of F^*. So: conjugate F, apply the forward transform, divide by MN and conjugate again. For real images the final conjugation is unnecessary.

```
import numpy as np
F = np.fft.fft2(np.random.default_rng(1).random((8, 8)))
f_back = np.conj(np.fft.fft2(np.conj(F))) / F.size
print(np.allclose(f_back, np.fft.ifft2(F)))   # True
```

The fast Fourier transform

Computing the 1-D DFT from its definition takes about M^2 complex multiplications and additions: M outputs, each a sum of M terms. The **FFT** of Cooley and Tukey [4] reduces this to about $M \log_2 M$ for M a power of 2. The core idea is **divide and conquer**: split the sum into even- and odd-indexed samples,

$$F(u) = F_{\text{even}}(u) + W_M^u\, F_{\text{odd}}(u), \qquad F(u+M/2) = F_{\text{even}}(u) - W_M^u\, F_{\text{odd}}(u)$$

where $W_M = e^{-j2\pi/M}$, and $F_{\text{even}}, F_{\text{odd}}$ are DFTs of length $M/2$. Each half-length DFT is split again, until the pieces have length 1. There are $\log_2 M$ levels, each costing on the order of M operations.

The ratio of the two costs, $M^2/(M\log_2 M) = M/\log_2 M$, grows quickly. For $M = 1024$ it is about 100; for a 1024×1024 image transformed directly in 2-D (cost $(MN)^2$) versus by FFT (cost $MN \log_2 MN$), the ratio is about 50,000. On a laptop the difference is easy to see:

```
import time
x = np.random.default_rng(2).standard_normal(4096)
t0 = time.perf_counter(); dft(x); t1 = time.perf_counter(); np.fft.fft(x); t2 =
 ↪  time.perf_counter()
print(f"direct {t1 - t0:.3f}s  fft {t2 - t1:.5f}s")   # e.g. ~0.7 s vs ~0.2 ms
```

Modern libraries go much further than the textbook radix-2 algorithm. FFTW, described by Frigo and Johnson [6], handles arbitrary sizes, chooses among many algorithms at run time by measuring which is fastest on the actual machine, and generates its small "codelet" kernels automatically. In practice, sizes whose prime factors are small (2, 3, 5, 7) are fast; large primes are slower. `scipy.fft.next_fast_len` returns a good padded size.

Modern view

The Fourier transform is older than every other tool in this series, and it has aged unusually well. Deep learning did not replace it. Instead, Fourier ideas returned in three roles: a way to *compute* convolutions, a way to *build* layers with global reach, and a lens for *understanding* what networks learn and where they fail.

Surveys and reviews worth reading

- **Unser, "Sampling — 50 years after Shannon" [3].** A review of sampling theory from Shannon's band-limited model to modern spline and shift-invariant-space formulations. Main takeaway: the ideal sinc interpolator is a mathematical idealization; practical systems use compact kernels plus a matching prefilter, and the approximation error can be analyzed precisely in the Fourier domain. It is the best bridge from this chapter's sampling theorem to real resampling code.
- **Heideman, Johnson and Burrus, "Gauss and the history of the fast Fourier transform" [5].** A historical review tracing FFT-style algorithms back to Gauss's unpublished work around 1805 and through many rediscoveries before 1965. Main takeaway: the algorithmic idea is simple and was found many times; what changed in 1965 was that digital computers made it matter.
- **Xu, Zhang and Luo, "Overview frequency principle/spectral bias in deep learning" [13].** A survey of the **frequency principle**: neural networks trained by gradient descent tend to fit low-frequency components of the target first and high-frequency components later. It collects the theory and experiments behind the phenomenon and its use in algorithm design. Main takeaway: a neural network behaves, during training, a little like a lowpass filter that slowly opens up.
- **Kovachki et al., "Neural Operator: Learning Maps Between Function Spaces" [15].** A long, comprehensive treatment of neural operators, which learn mappings between functions rather than between fixed-size arrays. Fourier neural operators are one of its four main parameterizations. Main takeaway: parameterizing a layer by its frequency response makes it largely independent of the grid resolution, a direct descendant of this chapter's "filter = multiply in frequency" idea.

Fourier transforms as a fast convolution engine

The convolution theorem promises that large convolutions become cheap products. Mathieu, Henaff and LeCun [9] applied this to convolutional networks: they computed convolutions as pointwise products of FFTs on the GPU, reusing each transformed feature map many times, and reported large speedups for training. Today most CNNs use small 3×3 kernels, where direct spatial convolution usually wins, so FFT convolution is mainly used for large kernels. The reasoning in the "Correspondence between spatial and frequency filters" section above applies unchanged.

Fourier layers with a global receptive field

A pointwise product in frequency affects every pixel, so a single Fourier layer has an image-wide receptive field. Several architectures exploit this. **Fast Fourier Convolution** [10] adds a global branch that operates directly on the image-level spectrum, mixing global context into every layer. **Global Filter Networks** [11] replace the self-attention of a vision transformer with exactly the procedure from this chapter: a 2-D FFT, an element-wise product with a *learned* global filter, and an inverse FFT, with log-linear cost in the number of tokens. The **Fourier Neural Operator** [14] uses learned filters on a truncated set of low frequencies to solve partial differential equations, and because the weights live in frequency space the trained model can be evaluated on finer grids than it was trained on. These are, quite literally, learned frequency-domain filters.

A related line works on transform coefficients directly. Xu et al. [16] feed networks with the block discrete cosine transform (DCT) coefficients that common lossy image codecs already compute, and learn which frequency channels to keep. They found that many high-frequency channels can be dropped with little accuracy loss, which is consistent with the frequency principle.

Spectral bias

Rahaman et al. [12] showed that deep ReLU networks are biased towards learning low-frequency functions: high-frequency components are learned later and are less robust. Together with the overview in [13], this gives a useful mental model. A network is good at the "bass" of a target function and needs more training, data, or special input encodings to get the "cymbals" right. This explains, for example, why coordinate-based networks often struggle to represent fine textures unless their input coordinates are first mapped through sinusoidal Fourier features [20].

Aliasing comes back in deep networks

Strided convolutions and pooling are subsampling, and subsampling without a lowpass prefilter aliases, just as in Figure 4.1. Zhang [17] showed that standard CNNs are surprisingly sensitive to one-pixel shifts for this reason, and that inserting a small blur before each downsampling step (the textbook anti-aliasing filter) improves shift-invariance and, in his experiments, also image-classification accuracy. Karras et al. [18] traced an effect in which fine details of generated images appear glued to pixel coordinates, instead of moving with the depicted surfaces, to aliasing inside the generator. Their alias-free design treats feature maps as sampled continuous signals and applies carefully designed lowpass filters around every nonlinearity and resampling step.

Frequency artifacts as fingerprints

Generators upsample, and upsampling without proper interpolation filters leaves periodic patterns in the spectrum, much like the spikes in Figure 4.8. Frank et al. [19] found that GAN-generated images show strong, consistent artifacts in the frequency domain, traced them to upsampling, and used them to build a simple and accurate detector. The broader lesson is that

looking at the spectrum, the oldest diagnostic in this chapter, remains one of the quickest ways to see what a generative model is doing wrong.

What did not change

Deep learning has not replaced classical frequency filtering for well-understood problems. Removing known periodic interference, anti-aliasing before resizing, fast large-kernel blurs, and homomorphic illumination correction are still done with the explicit filters in this chapter, because they are exact, fast, and need no training data. What changed is that these same ideas are now built into network architectures and used to explain their behavior.

Key takeaways

- The Fourier transform rewrites a signal as a sum of waves. Low frequencies carry smooth structure; high frequencies carry edges, texture and noise.
- Sampling copies the spectrum periodically. Sample faster than twice the highest frequency (the Nyquist rate), or blur first, otherwise aliasing and moiré appear and cannot be undone.
- The DFT treats the image as periodic. That gives circular convolution, so pad to at least $A+C-1$ (typically $2M \times 2N$) to avoid wraparound error.
- Translation changes only the phase; rotation rotates the spectrum; real images have conjugate-symmetric spectra; and the phase carries most of the recognizable structure.
- Filtering is "transform, multiply by a real symmetric H, invert, crop". Sharp cutoffs cause ringing; Gaussian filters do not; Butterworth sits in between.
- Highpass, Laplacian, high-frequency emphasis and homomorphic filters sharpen or rebalance lighting; notch filters remove periodic interference precisely.
- The FFT lowers the cost from $O(M^2)$ to $O(M \log M)$ per 1-D transform, and separability builds 2-D transforms from 1-D ones.
- In deep learning, Fourier ideas power fast convolution, global filter layers and neural operators, and explain spectral bias, aliasing in CNNs and GAN artifacts.

Exercises

1. A 1-D signal contains components at 3, 7 and 12 cycles per meter. You sample it every 0.1 m. Which components survive correctly, and at what frequency does each aliased component appear?

   Hint

   The sampling rate is 10 samples per meter, so the Nyquist limit is 5 cycles per meter. The 3 cycle component is fine. Fold the others back into $[-5, 5]$ by subtracting multiples of 10: 7 appears as $7-10 = -3$, i.e. 3 cycles per meter, and 12 appears as $12 - 10 = 2$ cycles per meter. The 7 cycle component is now indistinguishable from the real 3 cycle one.

2. Show that if $f(x, y)$ is real and symmetric, $f(x, y) = f(-x, -y)$ (indices modulo M, N), then its DFT is real. Verify numerically with a symmetric array in NumPy.

Hint

Conjugate symmetry gives $F(-u, -v) = F^*(u, v)$ for any real f. Symmetry of f gives $F(-u, -v) = F(u, v)$ by substituting $x \to -x, y \to -y$ in the definition. Together, $F = F^*$, so F is real. For the check, build `g = a + np.roll(a[::-1, ::-1], 1, axis=(0, 1))` for a random `a` and confirm `np.abs(np.fft.fft2(g).imag).max()` is near zero.

3. Using the `filter_freq` function, filter the camera image with a Butterworth lowpass filter of orders 1, 2, 5 and 20 at $D_0 = 30$. At what order does ringing first become clearly visible? Plot the central row of each filter's spatial kernel and relate it to what you see.

Hint

Obtain the kernel by inverse-transforming the centered filter and shifting it back to the center. Ringing appears when the kernel develops noticeable negative side lobes. Order 1 has none, order 2 has a tiny one, and by order 5 to 20 the lobes approach those of the ideal filter.

4. You filter a 300×400 image with a 51×51 spatial kernel using FFTs. What is the minimum padded size that avoids wraparound error? Would you choose that size or a slightly larger one, and why?

Hint

The minimum is $(300 + 51 - 1) \times (400 + 51 - 1) = 350 \times 450$. Both are fine, but sizes with only small prime factors are faster for FFT libraries. $350 = 2 \cdot 5^2 \cdot 7$ and $450 = 2 \cdot 3^2 \cdot 5^2$ already are, so they are good choices; `scipy.fft.next_fast_len` confirms this.

5. Photograph (or synthesize) an image with two different periodic interference patterns. Design a notch-reject filter that removes only one of them, then use the corresponding notch-pass filter to display the pattern you removed. How do you choose the notch radius?

Hint

Locate the spikes in the log spectrum, ignoring the DC peak and the axes. Remember each spike's mirror twin. A radius just large enough to cover the spike's spread works best: too small leaves residual stripes, too large removes image content near those frequencies. The notch-pass output should look like clean stripes with no trace of the scene.

6. Explain, using the ideas in this chapter, why inserting a blur before every stride-2 layer can make a CNN's output more stable when the input image is shifted by one pixel.

Hint

Stride-2 subsampling halves the sampling rate, so frequencies above the new Nyquist limit fold back as aliases. A one-pixel shift changes the phase of those components, and after aliasing that changes the low-frequency output, so the next layers see a different signal. A lowpass filter removes those frequencies first, so a shift changes the output much less. See [17].

References

1. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018, Ch. 4. publisher page
2. C. E. Shannon, "Communication in the presence of noise," *Proceedings of the IRE*, vol. 37, no. 1, pp. 10–21, 1949. doi
3. M. Unser, "Sampling—50 years after Shannon," *Proceedings of the IEEE*, vol. 88, no. 4, pp. 569–587, 2000. doi
4. J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series," *Mathematics of Computation*, vol. 19, no. 90, 1965. doi
5. M. T. Heideman, D. H. Johnson and C. S. Burrus, "Gauss and the history of the fast Fourier transform," *IEEE ASSP Magazine*, vol. 1, no. 4, pp. 14–21, 1984. doi
6. M. Frigo and S. G. Johnson, "The design and implementation of FFTW3," *Proceedings of the IEEE*, vol. 93, no. 2, 2005. doi
7. A. V. Oppenheim and J. S. Lim, "The importance of phase in signals," *Proceedings of the IEEE*, vol. 69, no. 5, 1981. doi
8. A. V. Oppenheim, R. W. Schafer and T. G. Stockham, "Nonlinear filtering of multiplied and convolved signals," *Proceedings of the IEEE*, vol. 56, no. 8, pp. 1264–1291, 1968. doi
9. M. Mathieu, M. Henaff and Y. LeCun, "Fast training of convolutional networks through FFTs," arXiv:1312.5851, 2013. arXiv
10. L. Chi, B. Jiang and Y. Mu, "Fast Fourier convolution," *Advances in Neural Information Processing Systems* (*NeurIPS*), vol. 33, 2020. proceedings
11. Y. Rao, W. Zhao, Z. Zhu, J. Lu and J. Zhou, "Global filter networks for image classification," *NeurIPS*, 2021. arXiv
12. N. Rahaman, A. Baratin, D. Arpit, F. Draxler, M. Lin, F. A. Hamprecht, Y. Bengio and A. Courville, "On the spectral bias of neural networks," *ICML*, 2019. arXiv
13. Z.-Q. J. Xu, Y. Zhang and T. Luo, "Overview frequency principle/spectral bias in deep learning," arXiv:2201.07395, 2022. arXiv
14. Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart and A. Anandkumar, "Fourier neural operator for parametric partial differential equations," arXiv:2010.08895, 2020. arXiv
15. N. Kovachki, Z. Li, B. Liu, K. Azizzadenesheli, K. Bhattacharya, A. Stuart and A. Anandkumar, "Neural operator: Learning maps between function spaces," *Journal of Machine Learning Research*, vol. 24, 2023. arXiv
16. K. Xu, M. Qin, F. Sun, Y. Wang, Y.-K. Chen and F. Ren, "Learning in the frequency domain," *CVPR*, 2020. arXiv
17. R. Zhang, "Making convolutional networks shift-invariant again," *ICML*, 2019. arXiv
18. T. Karras, M. Aittala, S. Laine, E. Härkönen, J. Hellsten, J. Lehtinen and T. Aila, "Alias-free generative adversarial networks," arXiv:2106.12423, 2021. arXiv
19. J. Frank, T. Eisenhofer, L. Schönherr, A. Fischer, D. Kolossa and T. Holz, "Leveraging frequency analysis for deep fake image recognition," *ICML*, 2020. arXiv
20. M. Tancik, P. P. Srinivasan, B. Mildenhall, S. Fridovich-Keil, N. Raghavan, U. Singhal, R. Ramamoorthi, J. T. Barron and R. Ng, "Fourier features let networks learn high frequency functions in low dimensional domains," arXiv:2006.10739, 2020. arXiv

IP 05 • Image Restoration and Reconstruction

Model how an image was damaged by blur and noise, then undo it with order-statistic, notch, inverse, Wiener and constrained least squares filters, and rebuild CT slices from projections.

Online edition: https://blog.shyandram.dev/en/learn/image-processing/dip-05-restoration-reconstruction/

What you'll learn

- How to write image damage as a model, $g = h \star f + \eta$, and why that model separates *restoration* from *enhancement*
- The common noise models, how to recognise them from a histogram, and how to measure their parameters from a flat patch
- Which spatial filter to use for which noise: mean filters, order-statistic filters, and adaptive filters
- How to remove periodic interference with notch filters in the frequency domain
- How to estimate a blur and undo it with inverse, Wiener, constrained least squares and geometric mean filters, and why the naive inverse fails
- How a CT scanner turns a set of 1-D projections into a 2-D slice: the Radon transform, the Fourier-slice theorem and filtered backprojection

The big picture

Chapter 3 and Chapter 4 improved images so they *look* better. This chapter asks a stricter question: *what did the image look like before it was damaged?* To answer it we need a model of the damage. Once we have the model, undoing it becomes a well-posed (or at least a well-understood) mathematical problem. The chapter follows the arrangement of Gonzalez and Woods [1].

> **Explain like I'm five**
>
> Imagine someone smudged your drawing with a wet finger and then sprinkled some sand on it. If you just "make it prettier", you might colour over everything. But if you know *how* the finger moved and *what* the sand looks like, you can carefully brush off the sand and un-smudge the lines. Restoration is the careful version: first find out what happened, then reverse exactly that.

Restoration matters wherever the true scene is what counts: astronomy, microscopy, forensic photos, medical imaging and every phone camera pipeline. The last part of the chapter, reconstruction from projections, is the same idea pushed to the extreme. A CT scanner never sees the slice at all. It sees only shadows of it from many angles, and we must *reconstruct* the slice from them.

A model of the image degradation/restoration process

In plain words: the camera sees a slightly blurred copy of the scene, and then random noise is added on top. Restoration tries to get back the scene.

We write the degradation as

$$g(x,y) = h(x,y) \star f(x,y) + \eta(x,y),$$

where $f(x,y)$ is the ideal (unknown) image, $h(x,y)$ is the **degradation function**, also called the point spread function (PSF), $\star$ is 2-D convolution, $\eta(x,y)$ is additive noise, and $g(x,y)$ is the image we actually observe. By the convolution theorem from Chapter 4, the same model in the frequency domain is

$$G(u,v) = H(u,v)\,F(u,v) + N(u,v),$$

where capital letters denote the 2-D discrete Fourier transforms (DFTs) of the matching lowercase functions, and (u,v) are frequency coordinates. $H(u,v)$ is called the **optical transfer function** (OTF).

The goal of restoration is an estimate $\hat{f}(x,y)$ that is as close as possible to $f(x,y)$. The more we know about h and η, the closer we can get.

Restoration versus enhancement

Enhancement (Chapter 3 and 4) is largely *subjective*: we change contrast or sharpen until the result looks good to a viewer. Restoration is *objective*: it uses a model of the degradation and judges the result by how close it gets to the true image, usually with a measurable criterion such as mean squared error. Some tools appear in both chapters (a median filter, for instance), but the reason for using them differs. In restoration we pick the filter *because* the noise model tells us it is the right one.

This chapter first handles the case $h = \delta$ (no blur, only noise), then adds blur.

Noise models

In plain words: noise is the random "grain" on top of the picture. Different sensors and situations produce different kinds of grain, and each kind has its own fingerprint in the histogram.

Spatial and frequency properties of noise

Two questions describe noise. *Spatially*, is the noise at one pixel related to the noise at its neighbours, and is it related to the image itself? In this chapter we assume, unless stated otherwise, that noise is independent of position and uncorrelated with the image. *In frequency*, how is the noise power spread across (u,v)? Noise whose Fourier spectrum is roughly constant is called

white noise, by analogy with white light containing all visible frequencies. Periodic noise is the opposite: its power sits at a few isolated frequencies.

Under these assumptions the noise is fully described by the probability density function (PDF) of its values, written $p(z)$, where z is an intensity.

The common PDFs

Gaussian. The most widely used model, because it is mathematically convenient and fits electronic sensor noise well:

$$p(z) = \frac{1}{\sqrt{2\pi}\,\sigma}\, e^{-(z-\bar{z})^2/2\sigma^2},$$

where $\bar{z}$ is the mean and σ the standard deviation (so σ^2 is the variance). About 68% of values fall within $\bar{z} \pm \sigma$ and about 95% within $\bar{z} \pm 2\sigma$.

Rayleigh. A one-sided, skewed density that describes noise in range imaging:

$$p(z) = \begin{cases} \dfrac{2}{b}(z-a)\, e^{-(z-a)^2/b}, & z \geq a \\ 0, & z < a \end{cases} \qquad \bar{z} = a + \sqrt{\pi b/4}, \quad \sigma^2 = \frac{b(4-\pi)}{4},$$

where a shifts the starting point and $b > 0$ controls the spread.

Erlang (gamma). Also skewed, with a longer tail:

$$p(z) = \begin{cases} \dfrac{a^b z^{b-1}}{(b-1)!}\, e^{-az}, & z \geq 0 \\ 0, & z < 0 \end{cases} \qquad \bar{z} = \frac{b}{a}, \quad \sigma^2 = \frac{b}{a^2},$$

where $a > 0$ is a rate and b is a positive integer (the shape). Allowing non-integer b gives the general gamma density.

Exponential. The Erlang density with $b = 1$:

$$p(z) = a\, e^{-az} \ (z \geq 0), \qquad \bar{z} = \frac{1}{a}, \quad \sigma^2 = \frac{1}{a^2}.$$

It appears in laser imaging (speckle intensity).

Uniform. Every value in an interval $[a, b]$ is equally likely:

$$p(z) = \frac{1}{b-a} \ (a \leq z \leq b), \qquad \bar{z} = \frac{a+b}{2}, \quad \sigma^2 = \frac{(b-a)^2}{12}.$$

It is mainly useful for simulations (quantization error is approximately uniform).

Salt-and-pepper (impulse) noise. A pixel is either untouched, or replaced by a very bright value (salt) with probability P_s, or by a very dark value (pepper) with probability P_p. The total fraction of corrupted pixels is $P = P_s + P_p$, called the **noise density**. Faulty sensor elements and transmission bit errors produce this kind of noise. Unlike the others, it is not additive: it *replaces* pixel values.

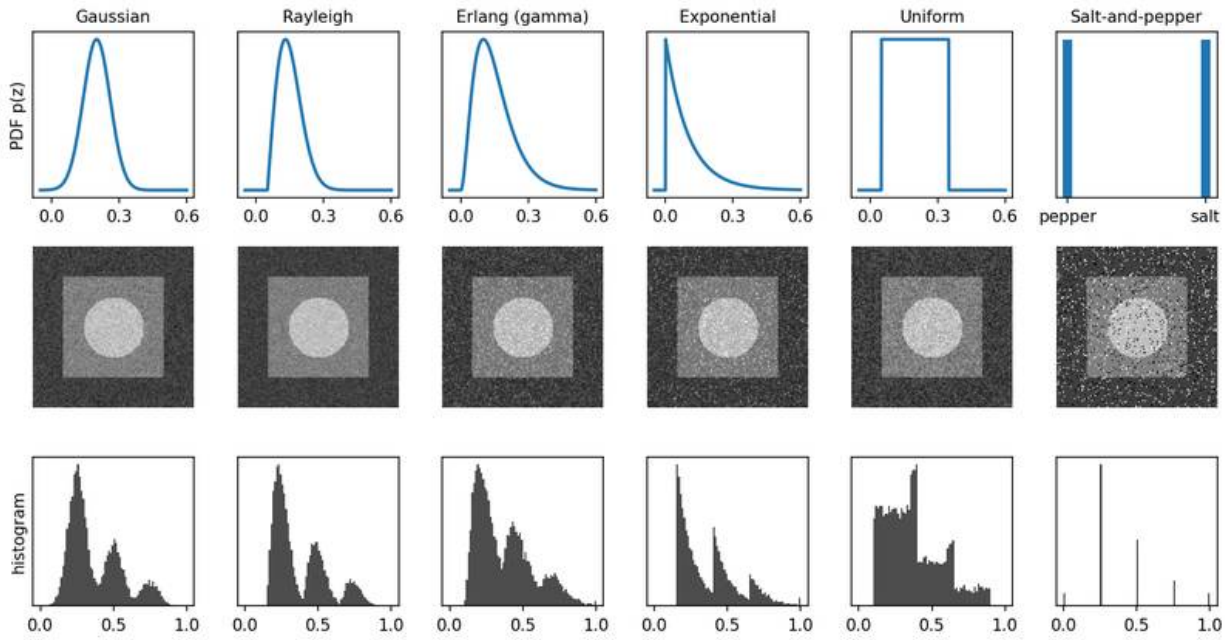


Figure 5.1 — Top: the six noise PDFs. Middle: a test pattern with three flat gray levels corrupted by each noise. Bottom: the histograms. The noisy images look similar, but each histogram has three copies of the noise PDF, one per gray level, so the histogram identifies the noise type. Impulse noise is the exception: it adds spikes at 0 and 1.

Periodic noise

Electrical or electromechanical interference during image capture can add a sinusoid such as $A\sin(2\pi(u_0 x/M + v_0 y/N))$ to the image, where A is its amplitude, (u_0, v_0) its frequency, and $M \times N$ the image size. It is *spatially dependent*, so a PDF does not describe it. In the Fourier spectrum, however, each sinusoid is a pair of bright spikes at $\pm(u_0, v_0)$, which makes it easy to remove there (see the section on periodic noise reduction below).

Estimating noise parameters

If you can control the imaging system, photograph a flat, uniformly lit target: everything you see is noise. If you only have the images, find a small region S that should be constant (sky, a blank wall, the background) and compute its histogram $p_S(z_i)$, the fraction of pixels in S with intensity z_i. Then

$$\bar{z} = \sum_i z_i \, p_S(z_i), \qquad \sigma^2 = \sum_i (z_i - \bar{z})^2 \, p_S(z_i).$$

The *shape* of the patch histogram tells you which PDF to use. The mean and variance then give the parameters (a and b for Rayleigh, Erlang, and so on). For impulse noise, count the pixels at the extreme values to estimate P_s and P_p.

```
import numpy as np
from skimage import data, img_as_float

rng = np.random.default_rng(0)
f = img_as_float(data.camera())                  # values in [0, 1]

g_gauss = f + rng.normal(0.0, 0.05, f.shape)     # additive Gaussian, sigma = 0.05
g_unif = f + rng.uniform(-0.1, 0.1, f.shape)     # additive uniform on [-0.1, 0.1]
g_sp = f.copy()
r = rng.random(f.shape)
g_sp[r < 0.05] = 0.0                             # 5 % pepper
g_sp[r > 0.95] = 1.0                             # 5 % salt

# Estimate the noise from a patch that should be flat (here: sky, top-left)
patch = g_gauss[10:60, 10:60]
print(f"mean = {patch.mean():.3f}, std = {patch.std():.3f}")   # std close to 0.05
```

Restoration in the presence of noise only — spatial filtering

In plain words: when the only problem is noise, replace every pixel by a smart summary of its neighbourhood. The best summary depends on the kind of noise.

With no blur, the model reduces to $g(x,y) = f(x,y) + \eta(x,y)$ (or, for impulse noise, a replacement). Below, S_{xy} is an $m \times n$ window centred at (x,y), and (r,c) runs over the pixels in it.

Mean filters

Arithmetic mean — the plain average:

$$\hat{f}(x,y) = \frac{1}{mn} \sum_{(r,c)\in S_{xy}} g(r,c).$$

It reduces Gaussian or uniform noise but blurs edges.

Geometric mean — the mn-th root of the product:

$$\hat{f}(x,y) = \Big[\prod_{(r,c)\in S_{xy}} g(r,c) \Big]^{1/mn}.$$

Smoothing is similar to the arithmetic mean, but less detail is lost.

Harmonic mean:

$$\hat{f}(x,y) = \frac{mn}{\sum_{(r,c)\in S_{xy}} 1/g(r,c)}.$$

It handles salt noise and Gaussian-like noise well but fails badly on pepper noise, because a single near-zero pixel dominates the sum of reciprocals.

Contraharmonic mean of order Q:

$$\hat{f}(x,y) = \frac{\sum_{(r,c)\in S_{xy}} g(r,c)^{Q+1}}{\sum_{(r,c)\in S_{xy}} g(r,c)^{Q}}.$$

Here Q is the order. Positive Q removes pepper noise, negative Q removes salt noise, and it cannot remove both at once. $Q = 0$ gives the arithmetic mean and $Q = -1$ the harmonic mean. Using the wrong sign is disastrous, as Figure 5.2 shows.

Order-statistic filters

These filters sort the pixels in S_{xy} and pick from the sorted list.

- **Median:** $\hat{f}(x,y) = \text{median}_{(r,c)\in S_{xy}}\, g(r,c)$. The best general tool for impulse noise. It removes isolated outliers while blurring much less than a mean filter of the same size.
- **Max** and **min:** the largest and smallest value. The max filter removes pepper (dark) noise, the min filter removes salt (bright) noise.
- **Midpoint:** $\frac{1}{2}(\max + \min)$. It combines ordering with averaging and suits Gaussian or uniform noise.
- **Alpha-trimmed mean:** drop the $d/2$ lowest and $d/2$ highest values, average the remaining $mn - d$ values $g_R(r,c)$:

$$\hat{f}(x,y) = \frac{1}{mn-d} \sum_{(r,c)\in S_{xy}} g_R(r,c).$$

With $d = 0$ it is the arithmetic mean; with $d = mn - 1$ it is the median. In between, it handles *mixtures*, such as Gaussian noise plus some impulses.

```
import numpy as np
from scipy import ndimage as ndi

def arithmetic_mean(g, m=3):
    return ndi.uniform_filter(g, m)

def geometric_mean(g, m=3):
    return np.exp(ndi.uniform_filter(np.log(g + 1e-6), m))

def harmonic_mean(g, m=3):
    return 1.0 / ndi.uniform_filter(1.0 / (g + 1e-6), m)

def contraharmonic_mean(g, m=3, Q=1.5):
    g = g + 1e-6                                  # avoid 0 ** negative
    return ndi.uniform_filter(g ** (Q + 1), m) / ndi.uniform_filter(g ** Q, m)

def alpha_trimmed_mean(g, m=5, d=6):
    def trim(v):
        v = np.sort(v)
```

```
        return v[d // 2: v.size - d // 2].mean()
    return ndi.generic_filter(g, trim, size=m)

median = lambda g, m=3: ndi.median_filter(g, m)
maxf = lambda g, m=3: ndi.maximum_filter(g, m)    # removes pepper
minf = lambda g, m=3: ndi.minimum_filter(g, m)    # removes salt
midpoint = lambda g, m=3: 0.5 * (maxf(g, m) + minf(g, m))
```

Adaptive filters

The filters above treat every pixel the same way. Adaptive filters change their behaviour according to local statistics in S_{xy}, and usually perform better at a modest extra cost.

Adaptive, local noise reduction filter. Let σ_η^2 be the noise variance (estimated once, for the whole image), and let $\bar{z}_S$ and σ_S^2 be the local mean and local variance in S_{xy}. The filter is

$$\hat{f}(x,y) = g(x,y) - \frac{\sigma_\eta^2}{\sigma_S^2}\,[g(x,y) - \bar{z}_S].$$

Read it as three cases. If $\sigma_\eta^2 = 0$ there is no noise, and the filter returns g unchanged. Where the local variance is much larger than the noise variance, the window probably contains an edge, so the filter stays close to g and keeps the edge. Where the two variances are equal, the window is a flat area, so the filter returns the local mean. In practice the ratio is clipped to at most 1, because a local variance estimate can come out smaller than σ_η^2 by chance.

Adaptive median filter. A fixed median filter works well while impulses are sparse; at high noise density a window may hold more impulses than real pixels, and a bigger fixed window blurs everything. The adaptive median filter grows the window when needed, and it leaves uncorrupted pixels alone. Let $z_{\min}$, $z_{\max}$, z_{med} be the minimum, maximum and median in S_{xy}, z_{xy} the centre pixel, and $S_{\max}$ the largest allowed window size [2].

- **Level A:** if $z_{\min} < z_{\text{med}} < z_{\max}$, the median is not an impulse, so go to Level B. Otherwise enlarge the window; if the size exceeds $S_{\max}$, output z_{med}; else repeat Level A.
- **Level B:** if $z_{\min} < z_{xy} < z_{\max}$, the centre pixel is not an impulse, so output z_{xy} unchanged. Otherwise output z_{med}.

Three goals are met at once: remove impulses, smooth other noise, and reduce the thinning and blurring of edges caused by a plain median.

```
import numpy as np
from scipy import ndimage as ndi

def adaptive_local(g, noise_var, m=7):
    """Adaptive local noise reduction: f = g - (s_n^2 / s_L^2)(g - m_L)."""
    mean = ndi.uniform_filter(g, m)
    var = ndi.uniform_filter(g * g, m) - mean ** 2
    ratio = np.minimum(noise_var / np.maximum(var, 1e-12), 1.0)  # never above 1
```

```
    return g - ratio * (g - mean)

def adaptive_median(g, s_max=7):
    out, done = g.copy(), np.zeros(g.shape, bool)
    for s in range(3, s_max + 1, 2):
        zmin, zmax = ndi.minimum_filter(g, s), ndi.maximum_filter(g, s)
        zmed = ndi.median_filter(g, s)
        stage_a = (zmin < zmed) & (zmed < zmax) & ~done          # median is not an impulse
        keep = (zmin < g) & (g < zmax)                            # centre pixel is not an
         ↪  impulse
        out[stage_a] = np.where(keep, g, zmed)[stage_a]
        done |= stage_a
    out[~done] = zmed[~done]                                      # window grew to s_max
    return out
```

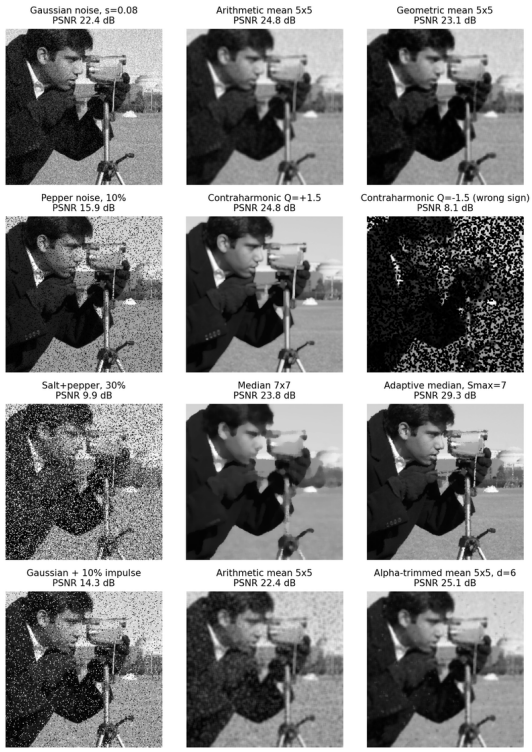


Figure 5.2 — Matching the filter to the noise. Row 1: Gaussian noise, arithmetic and geometric mean. Row 2: pepper noise; the contraharmonic filter with $Q = +1.5$ removes it, while $Q = -1.5$ makes it far worse. Row 3: 30% salt-and-pepper noise; the 7×7 median blurs and smears detail, the adaptive median keeps much more of it. Row 4: Gaussian plus impulse noise; the alpha-trimmed mean beats the plain mean. PSNR is peak signal-to-noise ratio against the clean image (higher is better).

Periodic noise reduction using frequency-domain filtering

In plain words: periodic noise is a few pure "tones" hidden in the image. In the spectrum they show up as bright dots, so we cut out exactly those dots and leave everything else.

Notch reject and notch pass filters

A **notch filter** rejects (or passes) frequencies in small neighbourhoods around chosen centres. Because the DFT of a real image is conjugate-symmetric, notches always come in pairs, at (u_k, v_k) and $(-u_k, -v_k)$ relative to the centre of the spectrum. A notch reject filter with Q notch pairs is a product of highpass filters whose centres have been moved to the notches:

$$H_{NR}(u,v) = \prod_{k=1}^{Q} H_k(u,v)\, H_{-k}(u,v),$$

where H_k and H_{-k} are highpass filters (ideal, Butterworth or Gaussian, as in Chapter 4) centred at (u_k, v_k) and $(-u_k, -v_k)$. For example, a Gaussian notch uses $H_k = 1 - e^{-D_k^2/2D_0^2}$, with $D_k(u,v)$ the distance from (u,v) to (u_k, v_k) and D_0 the notch radius. The matching notch pass filter is

$$H_{NP}(u,v) = 1 - H_{NR}(u,v).$$

The notch pass filter is useful on its own: applying it to G and transforming back gives an image of the interference pattern alone. Notch filters of other shapes (for instance a thin line along an axis) remove scan-line patterns.

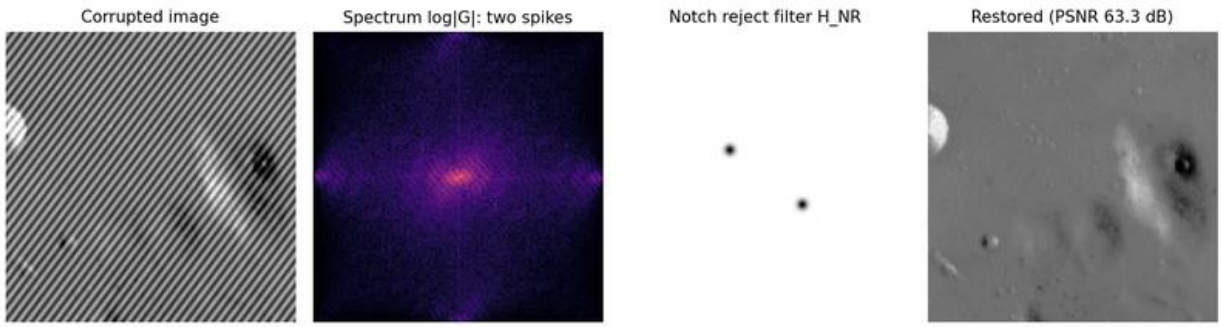


Figure 5.3 — A sinusoidal interference pattern (left) is two isolated spikes in the spectrum. Two Gaussian notches remove them, and the inverse DFT returns the clean image. The restoration here is almost perfect because the interference is a single exact sinusoid; real interference usually spreads over several frequencies.

```
import numpy as np
from skimage import data, img_as_float

f = img_as_float(data.moon())
M, N = f.shape
y, x = np.mgrid[:M, :N]
```

```
g = f + 0.2 * np.sin(2 * np.pi * (40 * y / M + 25 * x / N))    # periodic interference

G = np.fft.fftshift(np.fft.fft2(g))
U, V = np.mgrid[-M // 2:M // 2, -N // 2:N // 2]
H = np.ones((M, N))
for u0, v0 in [(40, 25), (-40, -25)]:                          # a spike and its mirror
    D2 = (U - u0) ** 2 + (V - v0) ** 2
    H *= 1 - np.exp(-D2 / (2 * 3.0 ** 2))                      # Gaussian notch, radius ~3
f_hat = np.real(np.fft.ifft2(np.fft.ifftshift(H * G)))
```

Optimum notch filtering

When there are several interference components, or they are not sharp spikes, a plain notch filter either removes too little or cuts away real image content. **Optimum notch filtering** works in two steps. First, isolate an estimate of the interference with a notch pass filter:

$$\hat{\eta}(x,y) = \mathfrak{F}^{-1}\{H_{NP}(u,v)\,G(u,v)\},$$

where $\mathfrak{F}^{-1}$ is the inverse DFT. Second, subtract a *weighted* version of it, with a weight that varies across the image:

$$\hat{f}(x,y) = g(x,y) - w(x,y)\,\hat{\eta}(x,y).$$

The weight $w(x,y)$ is chosen to make $\hat{f}$ as smooth as possible: it minimizes the variance of $\hat{f}$ inside a small window around each pixel. Setting the derivative of that local variance with respect to w to zero gives

$$w(x,y) = \frac{\overline{g\hat{\eta}} - \bar{g}\,\bar{\hat{\eta}}}{\overline{\hat{\eta}^2} - \bar{\hat{\eta}}^2},$$

where each bar is a local average over the window. The numerator is the local covariance of g and $\hat{\eta}$; the denominator is the local variance of $\hat{\eta}$. So the method subtracts more interference where the image and the estimated pattern move together, and less where they do not.

Linear, position-invariant degradations

In plain words: we assume the blur is "fair". Twice as much light gives twice as much blurred light, and the blur is the same in the top-left corner as in the bottom-right.

Write the degradation (without noise) as an operator $\mathcal{H}$, so $g = \mathcal{H}[f] + \eta$. $\mathcal{H}$ is **linear** if

$$\mathcal{H}[af_1 + bf_2] = a\,\mathcal{H}[f_1] + b\,\mathcal{H}[f_2]$$

for any images f_1, f_2 and constants a, b. It is **position invariant** if shifting the input only shifts the output:

$$\mathcal{H}[f(x-\alpha, y-\beta)] = g(x-\alpha, y-\beta)$$

for any shift (α, β). For such an operator, write f as a sum of shifted impulses. Linearity lets us apply $\mathcal{H}$ to each impulse separately, and position invariance says every impulse produces the same response h, just shifted. Summing these responses is exactly convolution:

$$g(x,y) = \int_{-\infty}^{\infty}\int_{-\infty}^{\infty} f(\alpha,\beta)\, h(x-\alpha, y-\beta)\, d\alpha\, d\beta + \eta(x,y) = (h \star f)(x,y) + \eta(x,y).$$

Here $h(x,y) = \mathcal{H}[\delta(x,y)]$ is the **impulse response**, which in optics is the point spread function: the image of a single point of light. This is why restoring a blurred image is often called **deconvolution**, and why the filters below are built in the frequency domain.

Many real degradations are only approximately LPI. A lens may blur more at the image corners, and an object moving in front of a still background blurs differently from the background. Space-variant restoration exists, but it is far more expensive; the common workaround is to split the image into tiles that are roughly LPI.

Estimating the degradation function

In plain words: before we can undo the blur, we must know the blur. We can look for clues in the picture, test the camera ourselves, or derive the blur from physics.

By observation

Find a small region of the blurred image $g_s(x,y)$ with simple, strong structure, such as an edge or a small bright object, where signal is much stronger than noise. Build by hand (or by sharpening and judgment) an estimate $\hat{f}_s(x,y)$ of what that region should look like. Then

$$H_s(u,v) = \frac{G_s(u,v)}{\hat{F}_s(u,v)}$$

estimates the OTF from that small region. If the blur is position invariant, H_s tells us the shape of H for the whole image, scaled up to full size. This is laborious and used mainly when the image is all there is, for example a historical photograph.

By experimentation

If you have the same (or an equivalent) imaging system, image a tiny bright dot under the same settings. A small enough dot approximates an impulse of strength A, whose Fourier transform is the constant A. The observed image of the dot then gives the OTF directly:

$$H(u,v) = \frac{G(u,v)}{A}.$$

Microscopists do exactly this with sub-resolution fluorescent beads.

By mathematical modeling

Sometimes physics gives the answer. Two classic models are used throughout this chapter.

Atmospheric turbulence. Long-exposure imaging through turbulent air is often modelled by

$$H(u,v) = e^{-k\,(u^2+v^2)^{5/6}},$$

where k is a constant that grows with the strength of the turbulence. Apart from the $5/6$ power it looks like a Gaussian lowpass filter, and it never reaches exactly zero.

Uniform linear motion blur. Suppose the image f moves during an exposure of length T, with displacement $x_0(t)$ and $y_0(t)$. The sensor integrates every position:

$$g(x,y) = \int_0^T f\big(x - x_0(t),\, y - y_0(t)\big)\, dt.$$

Taking the Fourier transform and using the shift property gives

$$H(u,v) = \int_0^T e^{-j2\pi[u\,x_0(t) + v\,y_0(t)]}\, dt.$$

For uniform motion, $x_0(t) = at/T$ and $y_0(t) = bt/T$, where a and b are the total displacements in x and y. The integral evaluates to

$$H(u,v) = \frac{T}{\pi(ua+vb)}\,\sin\!\big[\pi(ua+vb)\big]\, e^{-j\pi(ua+vb)}.$$

This is a sinc function along the direction of motion, with a linear phase term. Crucially, it is exactly **zero** wherever $ua + vb$ is a non-zero integer, so whole lines of frequencies are destroyed (Figure 5.4).

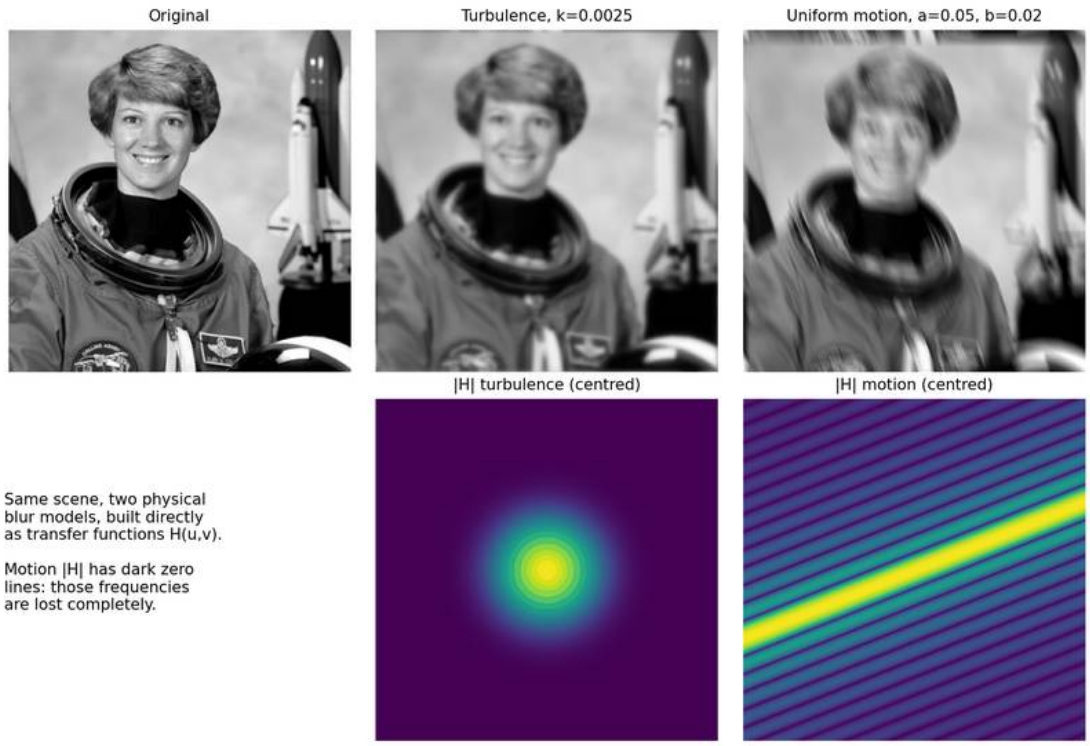


Figure 5.4 — Two degradation functions built from physics. The turbulence OTF is a smooth bump that decays but never reaches zero. The motion OTF is a sinc ridge along the motion direction with dark parallel lines of exact zeros: at those frequencies the blurred image contains no information about the scene.

```
import numpy as np

def freq_grid(shape):
    u = np.fft.fftfreq(shape[0]) * shape[0]            # 0, 1, ..., -1 (un-centred)
    v = np.fft.fftfreq(shape[1]) * shape[1]
    return np.meshgrid(u, v, indexing="ij")

def motion_blur_otf(shape, a, b, T=1.0):
    U, V = freq_grid(shape)
    s = np.pi * (U * a + V * b)
    return T * np.sinc(s / np.pi) * np.exp(-1j * s)    # np.sinc(x) = sin(pi x) / (pi x)

def turbulence_otf(shape, k):
    U, V = freq_grid(shape)
    return np.exp(-k * (U ** 2 + V ** 2) ** (5 / 6))
```

A practical detail

Building H directly in the frequency domain, as above, gives *circular* convolution: blur wraps around the image borders. That is perfect for experiments because the model matches the filter exactly. For real photos, pad the image or taper its borders before deconvolving, or you get strong ringing at the edges.

Inverse filtering

In plain words: if the blur multiplied each frequency by H, just divide by H. It is the obvious idea, and it fails spectacularly as soon as there is noise.

The **direct inverse filter** estimates

$$\hat{F}(u,v) = \frac{G(u,v)}{H(u,v)}.$$

Substituting $G = HF + N$ shows the problem:

$$\hat{F}(u,v) = F(u,v) + \frac{N(u,v)}{H(u,v)}.$$

Even if we knew H perfectly, we cannot recover F exactly, because N is unknown. Worse, blur makes $|H|$ tiny at high frequencies (or exactly zero, as for motion), while noise is roughly white and keeps its power there. So N/H becomes enormous and swamps the image. In Figure 5.5, the full inverse filter turns both images into pure noise, even though the noise standard deviation is only 0.3% and 1% of the intensity range.

A partial fix is to apply the inverse only near the origin of the spectrum, where $|H|$ is large, for instance by multiplying $\hat{F}$ by a sharp Butterworth lowpass of radius D_0. This works for the turbulence blur, whose H shrinks smoothly. It does not work well for motion blur, because the zero lines of H cross the low frequencies too. The radius must be tuned by hand. We need a method that decides *automatically* how much to trust each frequency.

Minimum mean square error (Wiener) filtering

In plain words: at every frequency, ask "is there more picture or more noise here?" Where the picture dominates, undo the blur. Where noise dominates, hold back.

The **Wiener filter** treats both the image and the noise as random fields and seeks the estimate that minimizes the mean squared error

$$e^2 = E\{(f - \hat{f})^2\},$$

where $E\{\cdot\}$ is the expected value. Assume the noise and the image are uncorrelated, that one of them has zero mean, and that $\hat{f}$ is a linear function of g.

Derivation sketch. For stationary signals the DFT decouples the frequencies, so we can choose a gain $W(u,v)$ for each frequency separately and set $\hat{F} = WG$. The expected error at one frequency is

$$E\{|F - WG|^2\} = S_f - WHS_f - W^*H^*S_f + |W|^2(|H|^2 S_f + S_\eta),$$

where $S_f(u,v) = E\{|F(u,v)|^2\}$ is the **power spectrum** of the image, $S_\eta(u,v) = E\{|N(u,v)|^2\}$ is the power spectrum of the noise, * denotes complex conjugation, and the cross terms with N vanish because image and noise are uncorrelated. This is a quadratic in W; setting its derivative with respect to W^* to zero gives

$$W(u,v) = \frac{H^*(u,v)\, S_f(u,v)}{|H(u,v)|^2\, S_f(u,v) + S_\eta(u,v)}.$$

Dividing top and bottom by S_f gives the form that is easiest to read:

$$\hat{F}(u,v) = \left[\frac{1}{H(u,v)}\,\frac{|H(u,v)|^2}{|H(u,v)|^2 + S_\eta(u,v)/S_f(u,v)}\right] G(u,v).$$

The first factor is the inverse filter. The second factor is between 0 and 1. Where the signal is strong compared to the noise, $S_\eta/S_f \approx 0$, the factor is close to 1 and the filter acts as an inverse filter. Where noise dominates, the factor shrinks toward 0 and switches that frequency off. If $H = 0$ at some frequency, the bracket is 0 there (write it as $H^*/(|H|^2 + S_\eta/S_f)$ to avoid dividing by zero). With no noise, $S_\eta = 0$, the Wiener filter is the inverse filter.

The constant-K approximation

We rarely know S_f, the power spectrum of the image we are trying to find. The common shortcut replaces the ratio S_η/S_f by a single constant K:

$$\hat{F}(u,v) = \frac{H^*(u,v)}{|H(u,v)|^2 + K}\, G(u,v).$$

K is tuned by eye or chosen to optimize a quality measure. A useful starting point is the ratio of the noise variance to the image variance, which is the inverse of the signal-to-noise ratio. A small K gives a sharper but noisier result; a large K a smoother one.

```
import numpy as np
from skimage import data, img_as_float

rng = np.random.default_rng(0)
f = img_as_float(data.camera())[::2, ::2]                # 256 x 256
H = motion_blur_otf(f.shape, a=0.1, b=0.1)               # from the snippet above
g = np.real(np.fft.ifft2(H * np.fft.fft2(f))) + rng.normal(0, 0.01, f.shape)
G = np.fft.fft2(g)

def wiener(G, H, K):
    return np.real(np.fft.ifft2(np.conj(H) / (np.abs(H) ** 2 + K) * G))

f_hat = wiener(G, H, K=0.01)
```

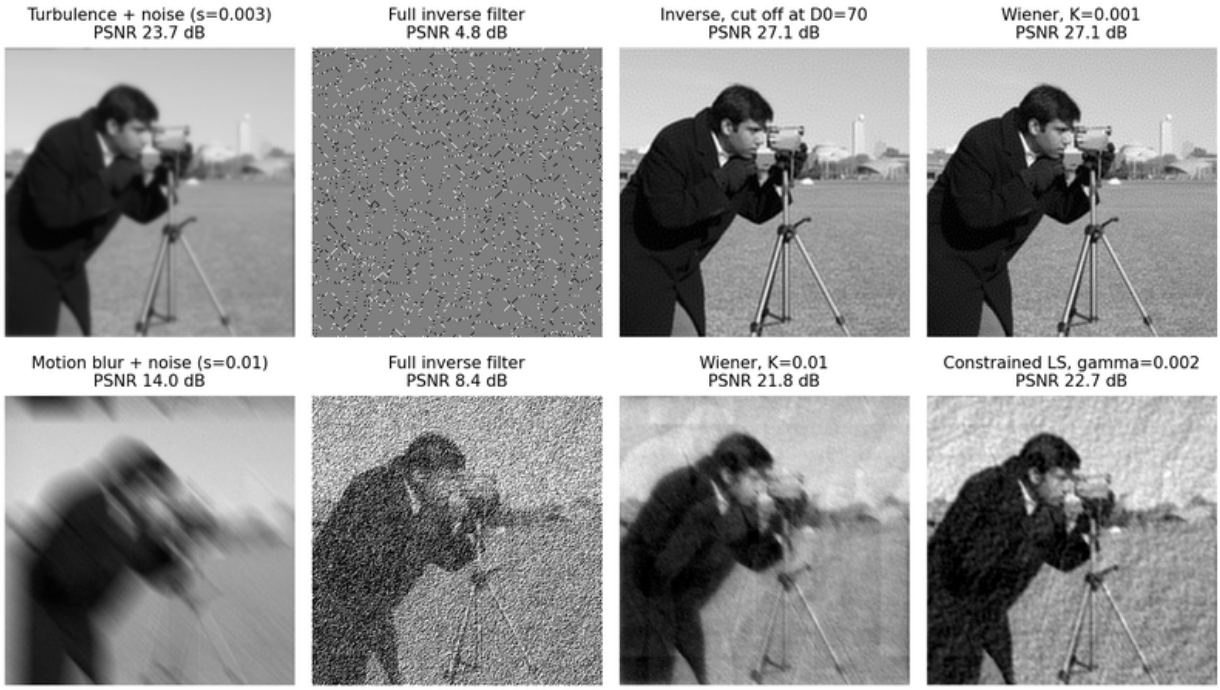


Figure 5.5 — Top row: turbulence blur ($k = 0.0025$) with weak noise. The full inverse filter is destroyed by amplified noise, the inverse restricted to a radius of 70 works, and the Wiener filter reaches the same quality without choosing a radius. Bottom row: diagonal motion blur with noise. The inverse filter fails; the Wiener and constrained least squares filters recover the scene, with some ringing and grain left over.

> **Interactive demonstration (online)**: open the online edition of this tutorial to use this demonstration.Open demonstration

Drag the slider to compare the motion-blurred, noisy image with its Wiener restoration (full 512 × 512 version, $K = 0.01$).

Motion blur + noise

Wiener, K=0.01

Cameraman image with diagonal motion blur and noise, compared with the Wiener-filtered result

Constrained least squares filtering

In plain words: among all images that would explain what we observed, up to the expected amount of noise, pick the smoothest one.

The Wiener filter needs power spectra, or a constant K with no clear meaning. **Constrained least squares (CLS)** filtering needs only the noise mean and variance [3]. In matrix-vector form, stack the pixels into vectors so that the model is

$$\mathbf{g} = \mathbf{H}\mathbf{f} + \boldsymbol{\eta},$$

where $\mathbf{g}, \mathbf{f}, \boldsymbol{\eta}$ are $MN \times 1$ vectors and $\mathbf{H}$ is the $MN \times MN$ blur matrix. These matrices are far too large to invert directly, but because $\mathbf{H}$ represents a circular convolution it is diagonalized by the DFT, which is what makes a closed-form frequency-domain answer possible.

Inverse problems like this are **ill-conditioned**: tiny changes in $\mathbf{g}$ cause huge changes in the solution. CLS fixes this by preferring smooth solutions. It measures roughness with the Laplacian ∇^2 and solves

$$\min_{\hat{\mathbf{f}}} C = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} \left[\nabla^2 \hat{f}(x,y)\right]^2 \quad \text{subject to} \quad \|\mathbf{g} - \mathbf{H}\hat{\mathbf{f}}\|^2 = \|\boldsymbol{\eta}\|^2,$$

where $\|\cdot\|$ is the Euclidean norm. The constraint says: the residual should be exactly as large as the noise, no more (we would be ignoring data) and no less (we would be fitting noise). The solution, found with a Lagrange multiplier, is

$$\hat{F}(u,v) = \left[\frac{H^*(u,v)}{|H(u,v)|^2 + \gamma\,|P(u,v)|^2}\right] G(u,v),$$

where $\gamma \geq 0$ is the multiplier and $P(u,v)$ is the DFT of the Laplacian kernel

$$p(x,y) = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix},$$

zero-padded to $M \times N$ before transforming. With $\gamma = 0$ we are back at the inverse filter. Compare with the constant-K Wiener filter: CLS replaces the flat K with $\gamma|P|^2$, which is small at low frequencies and large at high frequencies, so it holds back exactly where noise tends to dominate.

Choosing γ

γ is not a free knob: it must make the constraint hold. Define the residual $\mathbf{r} = \mathbf{g} - \mathbf{H}\hat{\mathbf{f}}$ and $\phi(\gamma) = \|\mathbf{r}\|^2$. One can show that ϕ is monotonically increasing in γ. So:

1. Start from some γ.
2. Compute $\hat{F}$ and the residual $\|\mathbf{r}\|^2$ (both cheap with the FFT).
3. If $\|\mathbf{r}\|^2 < \|\boldsymbol{\eta}\|^2 - a$, increase γ; if $\|\mathbf{r}\|^2 > \|\boldsymbol{\eta}\|^2 + a$, decrease it; otherwise stop. Here a is a small tolerance.

The target needs only the noise statistics: $\|\boldsymbol{\eta}\|^2 = MN\,(\sigma_\eta^2 + \bar{\eta}^2)$, where σ_η^2 and $\bar{\eta}$ are the noise variance and mean, which we already know how to estimate from a flat patch.

```
def cls(G, H, gamma):
    p = np.zeros(H.shape)
    p[[0, 0, 0, 1, -1], [0, 1, -1, 0, 0]] = [4, -1, -1, -1, -1]   # Laplacian centred at (0,
    ↪  0)
    P = np.fft.fft2(p)
    return np.real(np.fft.ifft2(np.conj(H) / (np.abs(H) ** 2 + gamma * np.abs(P) ** 2) * G))

f_hat = cls(G, H, gamma=0.002)
```

Which is better, Wiener or CLS? Neither in general. Wiener is optimal *on average* over an ensemble of images that share the assumed power spectra. CLS is tailored to the one image at hand and can give a better result for that image when the noise statistics are known. When K or γ is hand-tuned, as is common, the two often look similar (Figure 5.5).

Geometric mean filter

In plain words: a single dial that slides between "undo the blur fully" and "play it safe against noise".

The **geometric mean filter** combines the inverse and the (parametric) Wiener filter:

$$\hat{F}(u,v) = \left[\frac{H^*(u,v)}{|H(u,v)|^2}\right]^{\alpha} \left[\frac{H^*(u,v)}{|H(u,v)|^2 + \beta\,\dfrac{S_\eta(u,v)}{S_f(u,v)}}\right]^{1-\alpha} G(u,v),$$

where α and β are non-negative real constants. Special cases:

- $\alpha = 1$: the inverse filter.
- $\alpha = 0$: the **parametric Wiener filter**; with $\beta = 1$ it is the standard Wiener filter.
- $\alpha = 1/2$, $\beta = 1$: the geometric mean of the inverse and Wiener filters, which gives this family its name. It is also called the **spectrum equalization filter**: you can check that the expected power spectrum of its output, $|W|^2(|H|^2 S_f + S_\eta)$, is then exactly S_f, the power spectrum of the original image.

With $\beta > 1$ the filter is more cautious than Wiener; with $\beta < 1$ it is bolder.

```
def geometric_mean_filter(G, H, alpha, beta, K):
    """K stands in for S_eta / S_f, as in the constant-K Wiener filter."""
    H2 = np.abs(H) ** 2
    inv = np.conj(H) / np.maximum(H2, 1e-12)
    wie = np.conj(H) / (H2 + beta * K)
    # Both factors have the phase of conj(H): combine magnitudes, keep that phase.
    return np.real(np.fft.ifft2(np.abs(inv) ** alpha * np.abs(wie) ** (1 - alpha)
                                * np.exp(1j * np.angle(wie)) * G))
```

Image reconstruction from projections

In plain words: shine X-rays through a body from many angles. Each angle gives a "shadow" that only tells how much stuff the ray passed through in total. Put enough shadows together, cleverly, and you can rebuild the inside slice.

Principles of computed tomography (CT)

In X-ray CT, a source and a row of detectors rotate around the patient. For each angle, every detector measures how much the X-ray beam was attenuated along its straight path. Attenuation along a ray adds up (the log of the intensity ratio is the line integral of the attenuation coefficient), so each measurement is a **line integral** of the unknown slice $f(x, y)$. The set of measurements for one angle is a **projection**. CT scanners evolved from a single source-detector pair that translates and rotates (parallel beams), to fan beams with many detectors, to helical and cone-beam geometries that acquire whole volumes; the reconstruction mathematics below is the core of all of them [4].

Projections and the Radon transform

Describe a straight line in normal form, $x\cos\theta + y\sin\theta = \rho$, where ρ is its distance from the origin and θ the angle of its normal. The line integral of f along that line is

$$g(\rho, \theta) = \int_{-\infty}^{\infty}\int_{-\infty}^{\infty} f(x, y)\,\delta(x\cos\theta + y\sin\theta - \rho)\,dx\,dy,$$

where δ is the Dirac impulse, which keeps only the points on the line. As a function of (ρ, θ), g is the **Radon transform** of f. Displayed as an image with θ on one axis and ρ on the other it is called a **sinogram**: a single point in the slice traces a sinusoid, $\rho = x_0\cos\theta + y_0\sin\theta$.

The simplest way to go back is **backprojection**: smear each projection back across the image along the direction it came from, and add up the smears over all angles:

$$f_\theta(x, y) = g(x\cos\theta + y\sin\theta,\ \theta), \qquad f_{BP}(x, y) = \int_0^{\pi} f_\theta(x, y)\,d\theta.$$

This puts mass in the right places, but also everywhere along each ray, so the result is a heavily blurred version of f (Figure 5.6, third panel). The blur is not random: it is a convolution with $1/r$, where r is the distance from the centre. The Fourier-slice theorem shows how to remove it.

The Fourier-slice theorem

Let $G(\omega, \theta)$ be the 1-D Fourier transform of the projection $g(\rho, \theta)$ with respect to ρ, at a fixed angle θ:

$$G(\omega, \theta) = \int_{-\infty}^{\infty} g(\rho, \theta)\, e^{-j2\pi\omega\rho}\, d\rho.$$

The **Fourier-slice theorem** (also called the projection-slice theorem) states that

$$G(\omega, \theta) = F(\omega\cos\theta,\ \omega\sin\theta),$$

where $F(u, v)$ is the 2-D Fourier transform of f. In words: the 1-D transform of a projection equals a radial *slice*, through the origin at angle θ, of the 2-D transform of the object. The proof is short: substitute the Radon integral into G, integrate out the delta, and the exponent becomes $-j2\pi\omega(x\cos\theta + y\sin\theta)$, which is exactly the 2-D transform evaluated at $(u, v) = (\omega\cos\theta, \omega\sin\theta)$.

So in principle, projections from all angles fill the 2-D spectrum, slice by slice, and a 2-D inverse transform returns f. In practice the slices sample the spectrum on a polar grid, densely near the origin and sparsely far from it, and interpolating onto a Cartesian grid is error-prone. That uneven sampling is also the reason plain backprojection blurs: it over-weights low frequencies.

Filtered backprojection with parallel beams

Write the 2-D inverse Fourier transform in polar coordinates $u = \omega\cos\theta$, $v = \omega\sin\theta$. The area element $du\, dv$ becomes $|\omega|\, d\omega\, d\theta$, and with the Fourier-slice theorem:

$$f(x, y) = \int_0^{\pi} \left[\int_{-\infty}^{\infty} |\omega|\, G(\omega, \theta)\, e^{j2\pi\omega\rho}\, d\omega \right]_{\rho = x\cos\theta + y\sin\theta} d\theta.$$

Read it from the inside out. For each angle, multiply the 1-D spectrum of the projection by $|\omega|$, the **ramp filter**, and transform back. Then backproject the filtered projection and sum over angles. This is **filtered backprojection (FBP)**. The ramp exactly cancels the $1/r$ blur of plain backprojection.

The ramp grows without bound, so it amplifies high-frequency noise and is not integrable. In practice it is band-limited and multiplied by a smooth window, for example a Hamming window $h(\omega) = c + (1 - c)\cos(2\pi\omega/W)$ inside the band $|\omega| \le W/2$ and 0 outside, where W is the bandwidth and $c = 0.54$. The window equals 1 at $\omega = 0$ and falls to $2c - 1 = 0.08$ at the band edge. The window trades a little sharpness for much less ringing and noise. A filter introduced alongside the famous head phantom of Shepp and Logan [5] is still one of the standard options. The number of views matters as well: with too few angles, FBP produces streaks (Figure 5.6, last panel).

```
import numpy as np
from skimage.data import shepp_logan_phantom
from skimage.transform import radon, iradon, rescale

f = rescale(shepp_logan_phantom(), 0.64)                # 256 x 256
```

```
theta = np.linspace(0.0, 180.0, 180, endpoint=False)     # projection angles in degrees
sinogram = radon(f, theta=theta)                          # shape: (detector bins, angles)
backproj = iradon(sinogram, theta=theta, filter_name=None)    # blurry, no filter
fbp = iradon(sinogram, theta=theta, filter_name="ramp")       # filtered backprojection
rmse = np.sqrt(np.mean((fbp - f) ** 2))
```

`iradon` also accepts `"shepp-logan"`, `"cosine"`, `"hamming"` and `"hann"` windows, and `skimage.transform.iradon_sart` gives an iterative alternative [6].

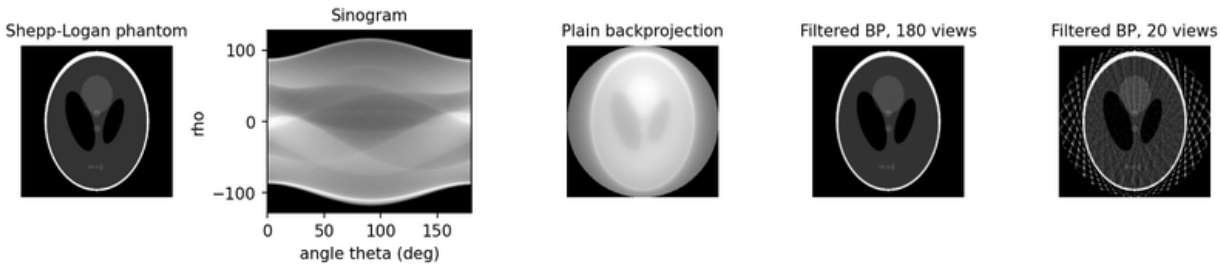


Figure 5.6 — From slice to sinogram and back. Each point of the phantom traces a sinusoid in the sinogram. Plain backprojection gives a blurred halo; adding the ramp filter (FBP) recovers the slice. With only 20 views, FBP shows streak artifacts, which is the problem learned reconstruction methods address (see Modern view).

Fan-beam filtered backprojection

Modern scanners use a **fan beam**: one point source and a curved or flat detector array, so rays in one view are not parallel. Each fan-beam ray is still some line (ρ, θ), so there are two routes. **Rebinning** sorts fan-beam rays from many views into sets of parallel rays and then applies parallel FBP. **Direct fan-beam FBP** changes variables in the FBP integral: projections are first multiplied by a cosine weight that depends on the ray's angle within the fan, then filtered with a modified ramp, and finally backprojected with a weight that depends on the distance from the source. The structure, weight-filter-backproject, is the same [4].

Modern view

The tools of this chapter are not museum pieces. The model $g = h \star f + \eta$ is still how the field states its problems, and Wiener-like steps still sit inside modern networks. What deep learning changed is the *prior*: instead of assuming "the image is smooth" (CLS) or "the image has power spectrum S_f" (Wiener), we now learn what clean images look like from data. For a broad tour of deep image restoration beyond this chapter (super-resolution, dehazing, deraining, low-light, all-in-one restoration and diffusion priors), see DLCV 04 ·Deep Image Restoration. The surveys below focus on the three topics of this chapter: denoising, deblurring and CT reconstruction.

Denoising: from patches to networks

Two classical methods marked the peak of hand-designed denoising. **Non-local means (NLM)** [7] replaces each pixel by a weighted average of pixels whose *surrounding patches* look similar, wherever they are in the image. It is the adaptive filter idea taken much further: the "neighbourhood" is defined by similarity, not distance. **BM3D** [8] groups similar patches into a 3-D stack, filters the stack jointly in a transform domain, and then applies a Wiener-type filter in a second stage, using the first-stage result as its estimate of S_f. It was the standard baseline for about a decade.

DnCNN [9] showed that a plain deep convolutional network trained with residual learning (predicting the noise rather than the clean image) and batch normalization beats BM3D on Gaussian noise, and that one model can handle unknown noise levels.

- **Tian et al., "Deep learning on image denoising: An overview" (2020)** [10]. Organises CNN denoisers into four groups: additive white Gaussian noise, real (camera) noise, blind denoising (unknown noise), and hybrid cases with blur or low resolution. Takeaway (our reading): the taxonomy itself is organised by noise type, with real camera noise and unknown noise levels as separate categories from synthetic Gaussian noise. The noise *model* from this chapter, and how realistic it is, stays central in deep denoising.
- **Elad, Kawar and Vaksman, "Image Denoising: The Deep Learning Revolution and Beyond" (2023)** [11]. Traces the history from classical priors to deep denoisers, then argues that denoisers matter beyond denoising. A good denoiser can serve as the prior inside a solver for *other* inverse problems, and it is the engine of diffusion-based image generation. Takeaway: the classic "noise only" problem of this chapter has become a building block for nearly everything else.

Deblurring and general inverse problems

- **Zhang et al., "Deep Image Deblurring: A Survey" (IJCV 2022)** [12]. Covers causes of blur, datasets and metrics, then a taxonomy of CNN-based non-blind and blind deblurring by architecture and loss, plus specialised domains (faces, text, stereo). Takeaway: non-blind deblurring (known h, as in this chapter) is largely about suppressing the noise amplification and ringing we saw with the inverse filter, while blind deblurring must estimate h too, and that remains the harder problem.
- **Ongie et al., "Deep Learning Techniques for Inverse Problems in Imaging" (2020)** [13]. Gives a taxonomy for any $\mathbf{g} = \mathbf{H}\mathbf{f} + \boldsymbol{\eta}$ problem: is the forward model $\mathbf{H}$ known at training time, only at test time, or not at all, and is training supervised or unsupervised? Takeaway (our reading): how much you know about $\mathbf{H}$ is the first design decision, and the survey also discusses failure modes of the resulting methods. Knowing $\mathbf{H}$, as in this chapter, opens options that a purely end-to-end network ignores.
- **Kamilov, Bouman, Buzzard and Wohlberg, "Plug-and-Play Methods for Integrating Physical and Learned Models in Computational Imaging" (2023)** [14]. Reviews plug-and-play (PnP) priors: iterative algorithms that alternate between a step enforcing the physics ($\mathbf{g} \approx \mathbf{H}\mathbf{f}$) and a step that calls a denoiser, which stands in for the prior. Takeaway: the forward model

and the learned prior can be developed separately and combined at run time. DPIR [15] is a widely used example that plugs a deep CNN denoiser into half-quadratic splitting for deblurring, super-resolution and demosaicing.

The connection to this chapter is direct. For a blur H, the physics step of such a splitting method asks for $\hat{F}$ minimizing $|G - HF|^2 + \mu|F - Z|^2$ at each frequency, where Z is the current denoised estimate and μ a penalty weight. Its solution is $\hat{F} = (H^*G + \mu Z)/(|H|^2 + \mu)$, which is a constant-$K$ Wiener filter pulled toward Z instead of toward zero.

CT reconstruction: learned and unrolled

FBP is fast and exact for complete, low-noise data. It struggles when data are reduced to lower the radiation dose, either fewer views (streaks, Figure 5.6) or fewer photons per view (noise). Classical answers are iterative methods with hand-made priors such as total variation. Deep learning offers three main patterns:

- **Post-processing:** run FBP, then a CNN removes the artifacts. **FBPConvNet** [16] does exactly this with a CNN and reports better quality than total-variation reconstruction on sparse-view CT at a fraction of the run time.
- **Unrolled (learned iterative) reconstruction:** take an iterative algorithm, fix its number of iterations, and replace parts of each iteration with small networks, trained end to end. **Learned Primal-Dual** [17] unrolls a primal-dual algorithm, keeping the forward projector and backprojector inside each iteration. **Monga, Li and Eldar, "Algorithm Unrolling" (2019)** [18] surveys this design pattern across signal and image processing. Takeaway: because each layer mirrors an algorithm step, unrolled networks are more interpretable and more efficient than generic networks.
- **Wang, Ye and De Man, "Deep learning for tomographic image reconstruction" (2020)** [19] reviews the whole landscape for CT and other tomographic modalities: image-domain, sensor-domain and hybrid networks, physics-informed designs, and the open issues of data availability, robustness and clinical validation.

What did *not* change: the Radon transform and the Fourier-slice theorem define the forward model that every one of these methods uses, and FBP remains the default in clinical scanners and the input or starting point of many learned methods. Learned methods also bring a new risk that this chapter's linear filters do not have: a network can produce plausible structure that is not in the data. That is why validation on real, out-of-distribution scans matters in medical use.

Key takeaways

- Restoration is objective: model the degradation as $g = h \star f + \eta$ (or $G = HF + N$), then invert *that* model.
- Identify noise from the histogram of a flat patch; estimate its mean and variance from the same patch.

- Match the spatial filter to the noise: means for Gaussian-like noise, contraharmonic with the right sign of Q for one-sided impulses, median and adaptive median for salt-and-pepper, alpha-trimmed for mixtures.
- Periodic noise is a few spikes in the spectrum; notch filters remove it, and optimum notch filtering adapts the subtraction locally.
- The inverse filter G/H amplifies noise wherever $|H|$ is small. Wiener, constrained least squares and geometric mean filters all add a term to the denominator that holds back those frequencies.
- CT measures line integrals (the Radon transform). By the Fourier-slice theorem, each projection gives one slice of the 2-D spectrum, and filtered backprojection (ramp filter, then backproject) reconstructs the image.
- Deep learning replaced hand-made priors with learned ones, but the best methods still keep the forward model inside (plug-and-play, unrolled networks).

Exercises

1. A dark-field microscope image has a background region whose histogram is one-sided, starting sharply at 12 and with a long right tail; its mean is 20 and its variance is 64 (gray levels). Which PDF from this chapter would you try first, and what are its parameters?

   Hint

   A sharp start and a long tail point to the exponential, Rayleigh or Erlang family. Try an exponential shifted to start at 12: it needs $1/a = 20 - 12 = 8$ and $1/a^2 = 64$, and both hold with $a = 1/8$. A Rayleigh fit is inconsistent: $b(4 - \pi)/4 = 64$ gives $b \approx 298$, so its start would be $20 - \sqrt{\pi b/4} \approx 4.7$, not 12.

2. An image is corrupted only by salt noise. Without testing, predict what happens with (a) a max filter, (b) a min filter, (c) a contraharmonic filter with $Q = 2$, (d) with $Q = -2$. Then verify with the code in this chapter.

   Hint

   Salt pixels are the largest values in their windows. The max filter spreads them (worse), the min filter removes them (but darkens and erodes bright detail). Positive Q weights large values more, so $Q = 2$ makes salt worse; $Q = -2$ removes it.

3. Show that the Wiener filter with constant K reduces to the inverse filter when $K = 0$, and that for a frequency where $|H|^2 \ll K$ the output is approximately $(H^*/K)\,G$. What does this say about how Wiener filtering treats frequencies the blur destroyed?

   Hint

   When $|H|^2 \ll K$ the denominator is about K. The output is then small, proportional to $|H|$, so the filter does *not* try to recover those frequencies. It gives up on them gracefully instead of amplifying noise.

4. A 256×256 image is blurred by uniform motion of 8 pixels along x and none along y. Frequencies are integers (cycles per image), as in the code above. Using the motion blur OTF, find all frequencies (u, v) with $H(u, v) = 0$. Explain why no linear filter can restore detail at those frequencies, and what that looks like in the restored image.

Hint

With u in cycles per image, a is the blur length as a fraction of the image size, so $a = 8/256 = 1/32$ and $b = 0$. Then $H = 0$ when $u/32$ is a non-zero integer: $u = \pm 32, \pm 64, \pm 96, \pm 128$, for every v. Since $G = HF + N$ has no F content there, any linear filter outputs only noise (or zero) at those frequencies. Expect faint ringing at that spatial period along x.

5. In the CT code above, reconstruct with 180, 60, 20 and 10 views, and with the `ramp` and `hann` filters. Plot RMSE versus number of views. Then add Gaussian noise to the sinogram and repeat. When does the smoother `hann` filter win?

Hint

Without noise, `ramp` should win at many views. As noise grows, the ramp amplifies it at high frequencies and `hann` (which rolls off there) wins, which is the same trade-off as inverse filter versus Wiener filter.

6. Derive the per-frequency solution $\hat{F} = (H^* G + \mu Z)/(|H|^2 + \mu)$ of $\min_F |G - HF|^2 + \mu |F - Z|^2$. What filter from this chapter do you get when $Z = 0$?

Hint

Differentiate with respect to F^*: $-H^*(G - HF) + \mu(F - Z) = 0$. With $Z = 0$ it is the Wiener filter with constant $K = \mu$.

References

1. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018, Ch. 5. publisher page
2. H. Hwang and R. A. Haddad, "Adaptive median filters: new algorithms and results," *IEEE Transactions on Image Processing*, vol. 4, no. 4, pp. 499–502, 1995. DOI
3. B. R. Hunt, "The application of constrained least squares estimation to image restoration by digital computer," *IEEE Transactions on Computers*, vol. C-22, no. 9, pp. 805–812, 1973. DOI
4. A. C. Kak and M. Slaney, *Principles of Computerized Tomographic Imaging*, IEEE Press, 1988 (SIAM reprint 2001). free online edition
5. L. A. Shepp and B. F. Logan, "The Fourier reconstruction of a head section," *IEEE Transactions on Nuclear Science*, vol. 21, no. 3, pp. 21–43, 1974. DOI
6. scikit-image developers, "Radon transform," scikit-image documentation (gallery example). docs

7. A. Buades, B. Coll and J.-M. Morel, “A non-local algorithm for image denoising,” *IEEE CVPR*, vol. 2, pp. 60–65, 2005. DOI
8. K. Dabov, A. Foi, V. Katkovnik and K. Egiazarian, “Image denoising by sparse 3-D transform-domain collaborative filtering,” *IEEE Transactions on Image Processing*, vol. 16, no. 8, pp. 2080–2095, 2007. DOI
9. K. Zhang, W. Zuo, Y. Chen, D. Meng and L. Zhang, “Beyond a Gaussian denoiser: Residual learning of deep CNN for image denoising,” *IEEE Transactions on Image Processing*, vol. 26, no. 7, pp. 3142–3155, 2017. doi ·arXiv
10. C. Tian, L. Fei, W. Zheng, Y. Xu, W. Zuo and C.-W. Lin, “Deep learning on image denoising: An overview,” *Neural Networks*, vol. 131, pp. 251–275, 2020. arXiv
11. M. Elad, B. Kawar and G. Vaksman, “Image denoising: The deep learning revolution and beyond — A survey paper,” *SIAM Journal on Imaging Sciences*, vol. 16, no. 3, pp. 1594–1654, 2023. arXiv
12. K. Zhang, W. Ren, W. Luo, W.-S. Lai, B. Stenger, M.-H. Yang and H. Li, “Deep image deblurring: A survey,” *International Journal of Computer Vision*, 2022. arXiv
13. G. Ongie, A. Jalal, C. A. Metzler, R. G. Baraniuk, A. G. Dimakis and R. Willett, “Deep learning techniques for inverse problems in imaging,” arXiv:2005.06001, 2020. arXiv
14. U. S. Kamilov, C. A. Bouman, G. T. Buzzard and B. Wohlberg, “Plug-and-play methods for integrating physical and learned models in computational imaging: Theory, algorithms, and applications,” *IEEE Signal Processing Magazine*, vol. 40, no. 1, pp. 85–97, 2023. arXiv
15. K. Zhang, Y. Li, W. Zuo, L. Zhang, L. Van Gool and R. Timofte, “Plug-and-play image restoration with deep denoiser prior,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 10, pp. 6360–6376, 2022. arXiv
16. K. H. Jin, M. T. McCann, E. Froustey and M. Unser, “Deep convolutional neural network for inverse problems in imaging,” *IEEE Transactions on Image Processing*, vol. 26, no. 9, pp. 4509–4522, 2017. arXiv
17. J. Adler and O. Öktem, “Learned primal-dual reconstruction,” *IEEE Transactions on Medical Imaging*, 2018. arXiv
18. V. Monga, Y. Li and Y. C. Eldar, “Algorithm unrolling: Interpretable, efficient deep learning for signal and image processing,” arXiv:1912.10557, 2019. arXiv
19. G. Wang, J. C. Ye and B. De Man, “Deep learning for tomographic image reconstruction,” *Nature Machine Intelligence*, vol. 2, no. 12, pp. 737–748, 2020. DOI

IP 06 • Color Image Processing

How color is measured and encoded, how to convert between RGB, HSI and L*a*b*, and how to transform, filter and segment color images.

Online edition: https://blog.shyandram.dev/en/learn/image-processing/dip-06-color/

What you'll learn

- Why three numbers are enough to describe a color for a human observer, and what the CIE chromaticity diagram and a device gamut are
- How the RGB, CMY(K), HSI and CIE L*a*b* models relate, with exact conversion formulas, plus where sRGB gamma and YCbCr fit in
- How pseudocolor turns a grayscale image into a readable color picture, and why perceptually uniform colormaps such as viridis beat the rainbow
- How to transform, white-balance, equalize, smooth, sharpen and segment full-color images, and when to process channels separately versus as vectors
- How noise behaves in color images and how vector filters remove it
- What modern research (learning-based color constancy, colormap design, deep colorization) adds to the classical toolkit

The big picture

So far every pixel has been one number: its brightness. Color images store three numbers per pixel. Color helps us find objects ("the red car") and separate materials, and it is the first thing people notice when a photo looks wrong.

> **Explain like I'm five**
>
> Imagine you have only three flashlights: a red one, a green one and a blue one. Shine them on a white wall. If you turn them up and down, you can make almost any color you like: red plus green makes yellow, all three make white. Your eyes work the same way in reverse: they have three kinds of tiny color sensors, and your brain mixes the three signals into every color you see. A color image is just a note, for every dot, of how bright each of the three flashlights should be.

Following the textbook [1], the chapter answers three questions: how to describe color precisely (fundamentals and models), how to use color to show data that is not colorful (pseudocolor), and how to process real color photographs (transformations, filtering, segmentation, noise, compression). Most grayscale tools from Chapters 3 to 5 carry over, with one recurring twist: a color pixel is a **vector**, and treating it as three unrelated numbers is sometimes fine and sometimes badly wrong.

Color fundamentals

In plain words. Light is a mix of wavelengths. Our eyes do not measure that whole mix. They squash it into three numbers, one per type of color sensor. Two very different light mixes that produce the same three numbers look identical to us.

Light, cones and trichromacy

Visible light spans wavelengths of roughly 400 to 700 nanometers. A light source or a surface is described by its **spectral power distribution** $S(\lambda)$: how much energy it emits or reflects at each wavelength λ.

The retina has three kinds of **cones** (color-sensitive photoreceptors), most sensitive to short, medium and long wavelengths. Each cone type $c \in \{S, M, L\}$ has a sensitivity curve $s_c(\lambda)$, and its response is

$$r_c = \int S(\lambda)\, s_c(\lambda)\, d\lambda.$$

Here $S(\lambda)$ is the incoming spectrum and $s_c(\lambda)$ the sensitivity of cone type c. Because a whole spectrum is reduced to three numbers, human color vision is **trichromatic**. A direct consequence is **metamerism**: two different spectra $S_1 \neq S_2$ that give the same three responses look like the same color. Metamerism is why displays work: a monitor never reproduces a lemon's spectrum, only its three cone responses.

Brightness, hue and saturation

People describe a color with three perceptual attributes:

- **Brightness**: how much light seems to be there (for a colorless gray, this is all there is).
- **Hue**: the dominant wavelength, the thing we name with words such as red, orange or blue.
- **Saturation**: how pure the color is, i.e. how little white is mixed in. Pink is a low-saturation red.

Hue and saturation together are called **chromaticity**, so a color is "chromaticity plus brightness", the idea behind the HSI model.

CIE tristimulus values and the chromaticity diagram

To make color measurable, the Commission Internationale de l'Éclairage (CIE) standardized, in 1931, three **color-matching functions** $\bar{x}(\lambda), \bar{y}(\lambda), \bar{z}(\lambda)$ derived from human matching experiments. The **tristimulus values** of a spectrum are

$$X = \int S(\lambda)\, \bar{x}(\lambda)\, d\lambda, \qquad Y = \int S(\lambda)\, \bar{y}(\lambda)\, d\lambda, \qquad Z = \int S(\lambda)\, \bar{z}(\lambda)\, d\lambda.$$

X, Y, Z are non-negative by construction, and $\bar{y}$ was chosen to match the eye's brightness sensitivity, so Y is the **luminance**. To separate chromaticity from brightness, normalize:

$$x = \frac{X}{X+Y+Z}, \qquad y = \frac{Y}{X+Y+Z}, \qquad z = \frac{Z}{X+Y+Z} = 1 - x - y.$$

x, y are the **chromaticity coordinates**; z is redundant. Plotting every pure spectral color (a single wavelength) in the (x, y) plane traces a horseshoe-shaped curve called the **spectral locus**. The straight line joining its two ends is the **line of purples** (purples are not single wavelengths). Every color a human can see lies inside this region.

Two geometric facts make the diagram useful:

1. **Mixing is linear.** Any mixture of two lights lies on the straight segment between their chromaticities. Any mixture of three lights lies inside their triangle.
2. **White sits in the middle.** A reference white, for example the daylight illuminant D65 at $(x, y) \approx (0.3127, 0.3290)$, lies near the center. Moving from white toward the locus increases saturation; the direction gives the hue.

Gamut

A display with three primaries can only produce the colors inside the triangle formed by those primaries. That triangle is the device's **gamut**. Because the spectral locus is curved, no triangle with physically realizable corners can cover it. Every three-primary display therefore misses some visible colors, especially saturated cyans and greens.

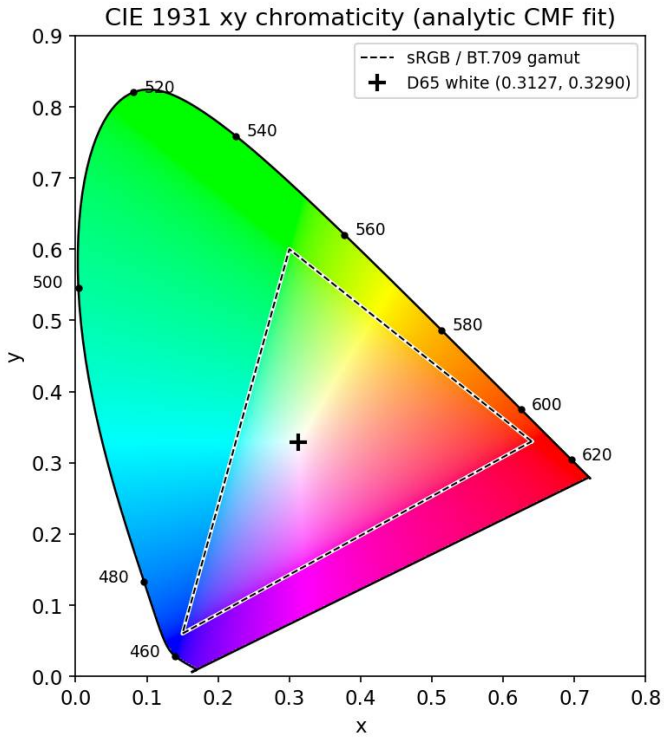


Figure 6.1 — CIE 1931 xy chromaticity diagram computed from the analytic color-matching-function fit

of Wyman et al. [2] (430–650 nm; the far tails of the fit are omitted). The dashed triangle is the sRGB / BT.709 gamut [3], [4]; colors outside it are only approximated on your screen.

The tabulated CIE functions are long tables; Wyman, Sloan and Shirley [2] give a fit made of a few piecewise Gaussians that is accurate enough for figures like this one. Each lobe is $a\,\exp(-\frac{1}{2}t^2)$ with $t = (\lambda - \mu)/\sigma$, where the width σ differs on the left and right of the peak μ.

Color models

In plain words. A color model is a coordinate system for colors. Different jobs need different coordinates: screens think in red-green-blue light, printers in inks, and people in "which color, how vivid, how bright".

RGB

In the **RGB model** each color is a point (R, G, B) in a unit cube. Black is $(0, 0, 0)$, white is $(1, 1, 1)$, and the main diagonal holds the grays. An 8-bit-per-channel image has $2^{24} \approx 16.7$ million possible colors. RGB is **additive**: it describes light being added together, which is how displays and camera sensors work.

Figure 6.2 — An RGB image is three grayscale images stacked together. The orange suit is bright in R and dark in B; the white parts of the suit are bright in all three.

"RGB" alone is not a precise color: $(1, 0, 0)$ means a different red on every display. The **sRGB** standard [3] fixes this by specifying primaries (the same chromaticities as ITU-R BT.709 [4]), the D65 white point, and a nonlinear encoding. The encoding, often loosely called "gamma", converts a stored value $C' \in [0, 1]$ to linear light C:

$$C = \begin{cases} C'/12.92, & C' \le 0.04045, \\ \left(\dfrac{C' + 0.055}{1.055}\right)^{2.4}, & C' > 0.04045. \end{cases}$$

C' is the stored (encoded) value of one channel and C the linear-light value. The curve spends more code values on dark tones, where the eye is more sensitive. **Practical rule:** operations that

model physics (blurring, resizing, mixing light) are more correct on linear values, so decode first when accuracy matters.

CMY and CMYK

Printing works the other way round. Ink on white paper *removes* light, so printing is **subtractive**. The secondary colors cyan, magenta and yellow absorb red, green and blue respectively. With values in $[0,1]$:

$$\begin{bmatrix} C \\ M \\ Y \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} - \begin{bmatrix} R \\ G \\ B \end{bmatrix}.$$

Here C, M, Y are ink amounts (not to be confused with the CIE Y). Equal amounts of real C, M and Y inks give a muddy brown, not black, so printers add a fourth black ink K. A simple conversion is

$$K = \min(C, M, Y), \qquad C_K = \frac{C - K}{1 - K}, \quad M_K = \frac{M - K}{1 - K}, \quad Y_K = \frac{Y - K}{1 - K} \quad (K < 1),$$

with $C_K = M_K = Y_K = 0$ when $K = 1$ (pure black). Real printers use measured profiles instead, but the idea is the same: move the shared gray part into black ink.

HSI

RGB is convenient for hardware but awkward for people: nobody describes a color as "70% red, 40% green, 20% blue". The **HSI model** (hue, saturation, intensity) decouples brightness from chromaticity, which matches how we talk about color and makes many algorithms simpler.

Geometrically, stand the RGB cube on its black corner so the gray diagonal points up. **Intensity** is the height along that axis. Cut the cube with a plane perpendicular to the axis: **hue** is the angle around the axis (red at 0°, green at 120°, blue at 240°), and **saturation** is the distance from the axis.

RGB to HSI (with $R, G, B \in [0,1]$):

$$\theta = \cos^{-1}\left(\frac{\frac{1}{2}[(R-G) + (R-B)]}{\sqrt{(R-G)^2 + (R-B)(G-B)}} \right), \qquad H = \begin{cases} \theta, & B \le G \\ 360^\circ - \theta, & B > G \end{cases}$$

$$S = 1 - \frac{3\,\min(R,G,B)}{R+G+B}, \qquad I = \frac{R+G+B}{3}.$$

θ is the angle between the pixel's chromatic component and the red axis. H is hue in degrees, $S \in [0,1]$ saturation, $I \in [0,1]$ intensity. Two edge cases need care in code. For gray pixels ($R = G = B$) the denominator of θ is zero and hue is **undefined**; set it to 0 by convention. For black ($I = 0$) saturation is undefined; set it to 0 as well.

HSI to RGB depends on which 120° sector H falls in. In the RG sector ($0^\circ \le H < 120^\circ$):

$$B = I(1-S), \qquad R = I\left[1 + \frac{S\cos H}{\cos(60^\circ - H)}\right], \qquad G = 3I - (R+B).$$

In the GB sector ($120^\circ \le H < 240^\circ$) subtract 120° from H and apply the same three formulas with (B,R,G) replaced by (R,G,B). In the BR sector ($240^\circ \le H < 360^\circ$) subtract 240° and replace (B,R,G) by (G,B,R). In each sector the channel "opposite" the hue gets the smallest value $I(1-S)$.

```
import numpy as np

def rgb_to_hsi(rgb):
    """rgb: float array in [0, 1] with shape (..., 3). Returns H (degrees), S, I."""
    R, G, B = rgb[..., 0], rgb[..., 1], rgb[..., 2]
    num = 0.5 * ((R - G) + (R - B))
    den = np.sqrt((R - G) ** 2 + (R - B) * (G - B))
    theta = np.degrees(np.arccos(np.clip(num / (den + 1e-12), -1, 1)))
    H = np.where(B > G, 360.0 - theta, theta)
    I = rgb.mean(axis=-1)
    S = np.where(I > 0, 1 - rgb.min(axis=-1) / (I + 1e-12), 0.0)
    H = np.where(den > 1e-12, H, 0.0)      # gray pixels: hue undefined -> 0
    return H, S, I

def hsi_to_rgb(H, S, I):
    """H in degrees, S and I in [0, 1]. Inverse of rgb_to_hsi."""
    H = np.mod(H, 360.0)
    out = np.zeros(H.shape + (3,))
    # (sector start, index of the 'big' channel, of the smallest, of the rest)
    for lo, (a, b, c) in ((0, (0, 2, 1)), (120, (1, 0, 2)), (240, (2, 1, 0))):
        m = (H >= lo) & (H < lo + 120)
        h = np.radians(H[m] - lo)
        x = I[m] * (1 - S[m])
        y = I[m] * (1 + S[m] * np.cos(h) / np.cos(np.radians(60) - h))
        out[m, a], out[m, b], out[m, c] = y, x, 3 * I[m] - (x + y)
    return out

prim = np.array([[1, 0, 0], [0, 1, 0], [0, 0, 1], [0.5, 0.5, 0.5]], float)
print(np.round(np.stack(rgb_to_hsi(prim), -1), 3))
# [[  0.   1.   0.333] [120.  1.  0.333] [240.  1.  0.333] [  0.  0.  0.5 ]]
```

The round trip `hsi_to_rgb(*rgb_to_hsi(x))` reproduces random RGB inputs to within about 10^{-9}.

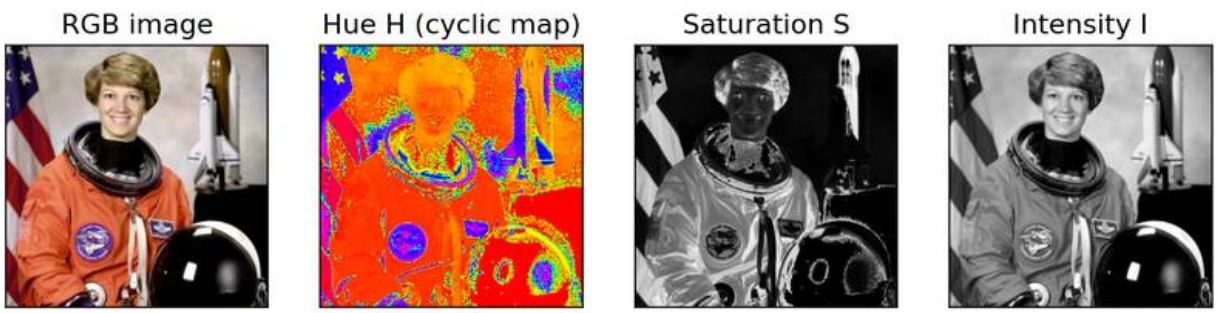


Figure 6.3 — HSI components. Hue is shown with a cyclic colormap because 0° and 360° are the same hue. Hue is noisy in gray regions (background, white suit), where it is nearly undefined; saturation is high only on the orange suit, the flag and the patches.

> **HSI, HSV and HSL are cousins, not twins**
>
> Libraries usually implement HSV (value $= \max(R,G,B)$) or HSL, not the HSI formulas above. All three separate hue from the rest, but their saturation and brightness axes differ, so numbers are not interchangeable. OpenCV also stores 8-bit hue as $H/2$ so that it fits in $[0,179]$.

CIE L*a*b*

HSI separates brightness from chromaticity, but equal steps in HSI do not look equally large. The **CIE 1976 L*a*b*** space [5] was designed so that Euclidean distance roughly tracks perceived difference. It is computed from tristimulus values relative to a reference white (X_n, Y_n, Z_n):

$$L^* = 116 f\left(\frac{Y}{Y_n}\right) - 16, \qquad a^* = 500\left[f\left(\frac{X}{X_n}\right) - f\left(\frac{Y}{Y_n}\right)\right], \qquad b^* = 200\left[f\left(\frac{Y}{Y_n}\right) - f\left(\frac{Z}{Z_n}\right)\right],$$

$$f(t) = \begin{cases} t^{1/3}, & t > \delta^3 \\ \dfrac{t}{3\delta^2} + \dfrac{4}{29}, & t \le \delta^3 \end{cases} \qquad \delta = \frac{6}{29}.$$

$L^* \in [0,100]$ is **lightness**, a^* runs from green (negative) to red (positive), and b^* from blue (negative) to yellow (positive). The cube root mimics the eye's compressive response, and the linear piece avoids an infinite slope near black. The color difference ΔE^*_{ab} is the Euclidean distance between two (L^*, a^*, b^*) points.

To get there from an sRGB file: decode the gamma (formula above), multiply the linear RGB by the sRGB-to-XYZ matrix, then apply the formulas with the D65 white.

```
import numpy as np
from skimage import color

M = np.array([[0.4124, 0.3576, 0.1805],      # linear sRGB -> XYZ (D65)
              [0.2126, 0.7152, 0.0722],
              [0.0193, 0.1192, 0.9505]])

def srgb_to_lab(rgb):
    lin = np.where(rgb <= 0.04045, rgb / 12.92, ((rgb + 0.055) / 1.055) ** 2.4)
    xyz = lin @ M.T / np.array([0.95047, 1.0, 1.08883])   # divide by D65 white
    d = 6 / 29
    f = np.where(xyz > d**3, np.cbrt(xyz), xyz / (3 * d**2) + 4 / 29)
    return np.stack([116 * f[..., 1] - 16,
                     500 * (f[..., 0] - f[..., 1]),
                     200 * (f[..., 1] - f[..., 2])], -1)

x = np.random.default_rng(0).random((1000, 3))
print(np.abs(srgb_to_lab(x) - color.rgb2lab(x[None])[0]).max())   # about 0.02
```

The small residual comes from rounding in the matrix; `skimage.color.rgb2lab` uses more digits.

YCbCr in one paragraph

Video and image codecs separate a **luma** signal Y' from two **chroma** differences, because the eye resolves fine detail in brightness much better than in color. ITU-R BT.601 [6] defines, for gamma-encoded $R', G', B' \in [0, 1]$,

$$Y' = 0.299\, R' + 0.587\, G' + 0.114\, B',$$

with $C_b \propto B' - Y'$ and $C_r \propto R' - Y'$ scaled and offset into 8-bit "studio" ranges (Y' in $[16, 235]$, C_b, C_r in $[16, 240]$). HDTV uses BT.709 [4], whose luma weights are different $(0.2126, 0.7152, 0.0722)$. Mixing them up shifts colors slightly but visibly. `skimage.color.rgb2ycbcr` implements the BT.601 studio-range version.

Pseudocolor image processing

In plain words. Pseudocolor (false color) paints a grayscale image with colors so that small differences in brightness become easy to see. The colors are not "real"; they are a code, like the colors on a weather map.

The motivation is perceptual: people can tell apart far more color combinations than shades of gray. A thermal camera, an X-ray, a depth map or an elevation model only has one value per pixel, but rendering that value with color can reveal structure the eye would miss in gray.

Intensity slicing

The simplest scheme cuts the gray range into bands and gives each band a color. With thresholds $0 = l_0 < l_1 < \cdots < l_P = L - 1$ and colors $c_1, \ldots, c_P$,

$$g(x, y) = c_k \quad \text{if} \quad l_{k-1} \le f(x, y) < l_k.$$

f is the input gray level, L the number of gray levels, P the number of bands, and g the output color. Viewed as a 3-D plot of f, each threshold is a horizontal plane slicing the "intensity terrain", hence the name. With $P = 2$ this is plain thresholding drawn in two colors. Intensity slicing is ideal when the bands mean something: "temperature above 40 °C", "bone versus soft tissue".

Intensity-to-color transformations

A more general approach applies three independent functions to the gray value:

$$g_R(x, y) = T_R(f(x, y)), \qquad g_G(x, y) = T_G(f(x, y)), \qquad g_B(x, y) = T_B(f(x, y)).$$

T_R, T_G, T_B map gray level to the red, green and blue output. Intensity slicing is the special case where all three are staircases. Smooth functions give a **continuous colormap**. The same idea extends to several input images: for multispectral data, three bands (for example, near-infrared, red and green) are each assigned to a display channel to form a false-color composite.

Perceptually uniform colormaps

The choice of T_R, T_G, T_B matters more than it looks. The popular rainbow map (`jet`) passes through bright yellow and cyan in the middle and gets darker at both ends. Its **lightness is not monotonic**: equal steps in data produce unequal visible steps, flat bands look like edges, and real edges can vanish. Kovesi [7] shows that some widely used maps have perceptual "flat spots" that can hide a feature as large as a tenth of the data range, and argues that a colormap should be designed so that lightness changes uniformly.

viridis, designed by van der Walt and Smith and now Matplotlib's default [8], was built to be perceptually uniform in the CAM02-UCS color-appearance space, to stay uniform when printed in grayscale, and to remain readable for common forms of color-vision deficiency. Crameri, Shephard and Heron [9] make the same case across the sciences: rainbow and red–green maps distort data and exclude the roughly 8% of men with color-vision deficiency.

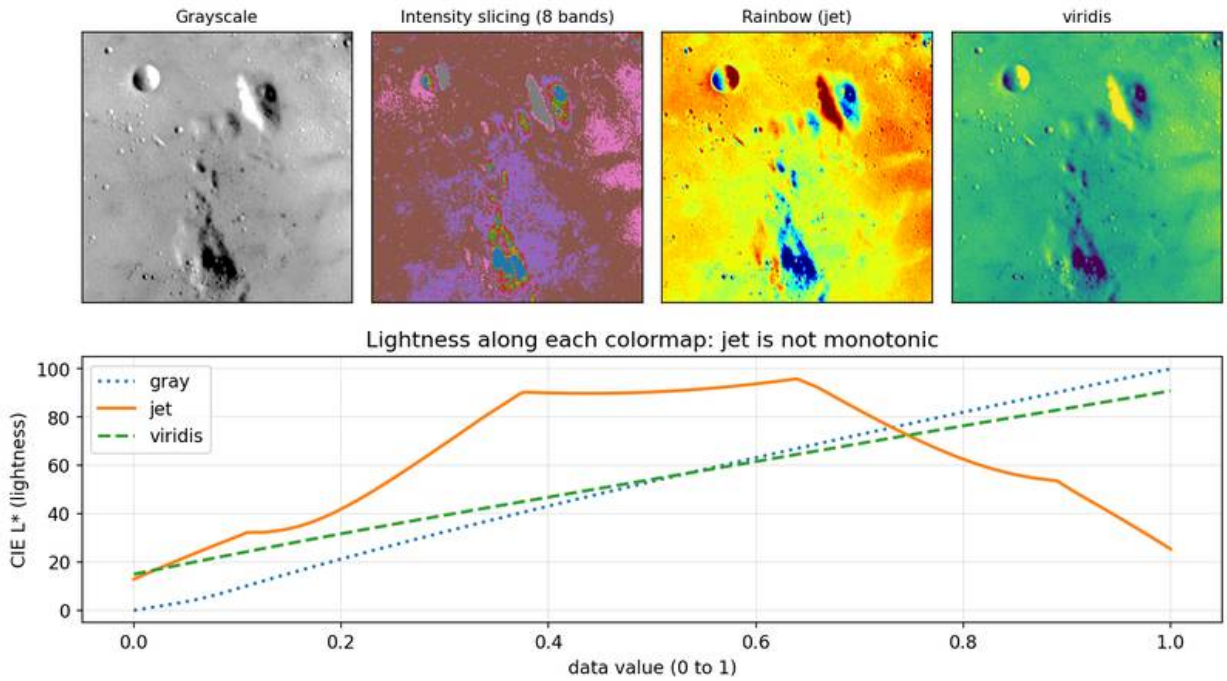


Figure 6.4 — Pseudocolor of a contrast-stretched lunar image. Top: grayscale, intensity slicing into 8 bands, the rainbow map jet, and viridis. Bottom: L lightness along each map. viridis rises steadily like gray; jet rises, plateaus and falls, which creates false edges.*

```
import numpy as np
```

```
import matplotlib.pyplot as plt
from skimage import data, util, color

g = util.img_as_float(data.moon())
# intensity slicing into 8 equal bands
bands = np.digitize(g, np.linspace(0, 1, 9)[1:-1])          # 0..7
palette = plt.get_cmap("tab10")(np.arange(8))[:, :3]
sliced = palette[bands]                                      # (H, W, 3)
# continuous intensity-to-color transformation
rgb = plt.get_cmap("viridis")(g)[..., :3]
# check a colormap's lightness profile
L = color.rgb2lab(plt.get_cmap("jet")(np.linspace(0, 1, 256))[None, :, :3])[0, :, 0]
print(np.all(np.diff(L) >= 0))                               # False: jet is not monotonic
```

Rule of thumb: use a **sequential** uniform map (such as viridis) for ordered data, a **diverging** map when a meaningful midpoint exists (zero, a threshold), a **cyclic** map for angles and hue, and a **qualitative** palette only for unordered categories, as in intensity slicing.

Basics of full-color image processing

In plain words. You can process a color image as three separate gray images, or treat each pixel as a little arrow in color space and process the arrows. Sometimes both give the same answer; often they do not.

A full-color pixel is the vector

$$\mathbf{c}(x,y) = \begin{bmatrix} c_R(x,y) \\ c_G(x,y) \\ c_B(x,y) \end{bmatrix} = \begin{bmatrix} R(x,y) \\ G(x,y) \\ B(x,y) \end{bmatrix}.$$

There are two ways to process it.

1. **Per-channel (componentwise).** Apply a grayscale method to each channel independently and restack.
2. **Vector processing.** Work directly on the vectors $\mathbf{c}(x,y)$ using operations that treat the three components together, such as vector distances, vector medians or the color gradient.

Per-channel processing gives the same result as vector processing when two conditions hold: the operation is applicable to both scalars and vectors, and it acts on each component in the same way without mixing them. **Linear** neighborhood operations satisfy both. Averaging each channel over a window equals averaging the vectors over that window:

$$\frac{1}{|S_{xy}|} \sum_{(s,t)\in S_{xy}} \mathbf{c}(s,t) = \begin{bmatrix} \frac{1}{|S_{xy}|} \sum R(s,t) \\ \frac{1}{|S_{xy}|} \sum G(s,t) \\ \frac{1}{|S_{xy}|} \sum B(s,t) \end{bmatrix},$$

where S_{xy} is the neighborhood centered at (x, y) and $|S_{xy}|$ its number of pixels. **Nonlinear** operations break the equivalence. A per-channel median can pick the red value from one pixel, the green from a second and the blue from a third, creating a color that appeared nowhere in the neighborhood. Thresholds, edge detectors and histogram equalization are other cases where the vector view matters, as later sections show.

Color transformations

In plain words. A color transformation is a recipe that changes each pixel's color using only that pixel: make it darker, flip it to its opposite, warm it up, or keep it only if it is "orange enough".

Formulation

As in Chapter 3, a point transformation is $g(x, y) = T[f(x, y)]$, but now f and g are color vectors. Written per component,

$$s_i = T_i(r_1, r_2, \dots, r_n), \qquad i = 1, \dots, n,$$

where r_i and s_i are the i-th input and output components and n is the number of components ($n = 3$ for RGB and HSI, $n = 4$ for CMYK). Each output may depend on all inputs.

Any effect can be written in any model, but the cost differs. Scaling intensity by k is one multiplication in HSI ($s_3 = k\, r_3$), three in RGB ($s_i = k\, r_i$), and an affine map in CMY ($s_i = k\, r_i + (1-k)$). Pick the model where the operation is natural, but remember that conversions cost time and amplify noise in dark or gray pixels, where hue and saturation are unstable.

> **Interactive demonstration (online)**: open the online edition of this tutorial to use this demonstration.Open demonstration

Color complements

Hues opposite each other on the color circle are **complements**: red and cyan, green and magenta, blue and yellow. In RGB the complement is the photographic negative,

$$s_i = 1 - r_i, \qquad i = R, G, B,$$

which is the color version of the gray-level negative in Chapter 3. In HSI the hue complement is $H \mapsto (H + 180°) \bmod 360°$, but the RGB negative also inverts intensity, so it cannot be computed from hue alone. Complements help reveal detail in dark regions.

Color slicing

Color slicing is intensity slicing for color: keep pixels whose color is close to a **prototype a** $= (a_R, a_G, a_B)$ and push the rest to a neutral color. Two common definitions of "close":

$$\text{cube:}\quad s_i = \begin{cases} 0.5, & \text{if } |r_j - a_j| > W/2 \text{ for any } j \\ r_i, & \text{otherwise} \end{cases} \qquad \text{sphere:}\quad s_i = \begin{cases} 0.5, & \text{if } \|\mathbf{r} - \mathbf{a}\| > R_0 \\ r_i, & \text{otherwise} \end{cases}$$

W is the width of a cube centered at $\mathbf{a}$, R_0 the radius of a sphere, $\mathbf{r}$ the input color vector and 0.5 a mid-gray. The sphere treats all directions equally; the cube is cheaper and easy to reason about per channel. A friendlier variant keeps unmatched pixels as their own gray value instead of a flat gray, so context remains visible.

```
import numpy as np
from skimage import data, util, color

img = util.img_as_float(data.astronaut())
a = np.array([0.85, 0.40, 0.20])            # prototype: the orange of the suit
d = np.linalg.norm(img - a, axis=-1)        # distance of every pixel to a
gray = color.gray2rgb(color.rgb2gray(img))
sliced = np.where((d <= 0.25)[..., None], img, gray)
print((d <= 0.25).mean())                   # fraction kept, about 0.165
```

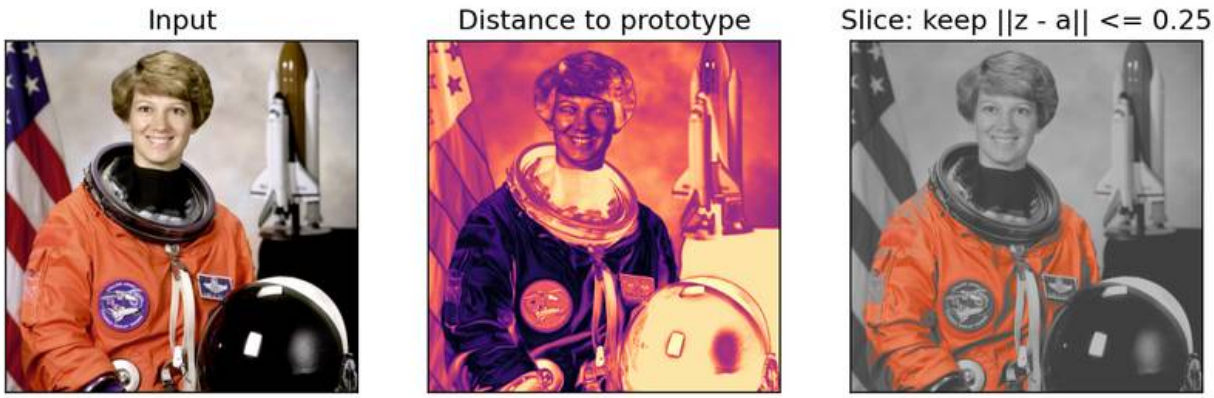


Figure 6.5 — Color slicing with a sphere of radius 0.25 around the suit's orange. Middle: distance to the prototype (dark = close). Right: matched pixels keep their color; everything else is shown in gray.

Tone and color corrections

Tone corrections adjust brightness and contrast and are best applied to an intensity-like channel (I, V or L^*) so hues are preserved. A mostly bright (high-key) image needs a curve that expands the highlights, a dark (low-key) image one that lifts the shadows, a flat image an S-curve.

Color balance corrections remove casts: to reduce a color, add its complement or reduce its neighbors on the color circle. Too much cyan? Increase red, or decrease blue and green. A gray card in the scene gives a precise target: after correction its R, G and B should be equal.

Real workflows convert each device's colors through a calibrated profile into a device-independent space such as L*a*b*, so a color looks the same on screen and on paper; colors outside a device's gamut (Figure 6.1) must be mapped inward.

White balance

White balance is the most common automatic color correction. A camera records surface color multiplied by the color of the light, so the same white wall looks orange under tungsten bulbs and bluish in shade. Our visual system largely discounts the light (**color constancy**); a camera has to do it computationally.

The standard model is diagonal: for illuminant color $\mathbf{e} = (e_R, e_G, e_B)$, the observed pixel is $c_i = e_i \rho_i$, where ρ_i is the surface's response under white light. Correction divides by the estimated illuminant, $\hat{\rho}_i = c_i / \hat{e}_i$ (usually rescaled to keep overall brightness). The hard part is estimating $\hat{\mathbf{e}}$ from one image. Classical estimators make an assumption about the scene:

- **Gray world** [10]: the average reflectance in a scene is achromatic, so $\hat{\mathbf{e}} \propto$ the mean RGB of the image.
- **White patch (max-RGB)** [11], [14]: the brightest surface is white, so $\hat{\mathbf{e}} \propto$ the per-channel maximum (in practice a high percentile, to ignore a few clipped pixels).
- **Gray edge** [11]: the average *edge* (image derivative) is achromatic. Van de Weijer, Gevers and Gijsenij show that all three are members of one family,

$$\left(\int \left| \frac{\partial^n \mathbf{c}_\sigma(\mathbf{x})}{\partial \mathbf{x}^n} \right|^p d\mathbf{x} \right)^{1/p} = k\, \hat{\mathbf{e}},$$

applied per channel, where $\mathbf{c}_\sigma$ is the image smoothed with a Gaussian of scale σ, n the derivative order, p a Minkowski norm and k a constant. $n = 0, p = 1$ gives gray world; $n = 0, p = \infty$ gives max-RGB; $n = 1$ gives gray-edge.

```
import numpy as np
from skimage import data, util

def white_balance(img, method="gray-world", p=99.5):
    """Diagonal white balance. img: float RGB in [0, 1]."""
    px = img.reshape(-1, 3)
    e = px.mean(0) if method == "gray-world" else np.percentile(px, p, axis=0)
    gain = e.mean() / e                 # one gain per channel
    return np.clip(img * gain, 0, 1), e

img = util.img_as_float(data.astronaut()) * 0.8
biased = np.clip(img * [0.75, 0.95, 1.25], 0, 1)      # add a bluish cast
fixed, e = white_balance(biased, "white-patch")
print(e.round(3))                                      # [0.598 0.757 0.996]
```

Drag the slider to compare the bluish input with the white-patch correction.

Bluish cast

White-patch corrected

Astronaut image with a synthetic blue cast and after white-patch white balance

Every one of these assumptions fails on some scene. On the cat photo below, whose true average color is orange-brown, gray world "corrects" the warm fur toward gray and leaves a blue tint, while white patch, helped by the white whiskers and highlights, does much better. The angular error between the estimated and true illuminant (the standard metric in this field) is printed above each result.

Figure 6.6 — Gray world versus white patch on an image whose average color is not gray. Angular errors come from our own synthetic experiment (see the figure script), not from a benchmark.

Histogram processing of color images

Equalizing the R, G and B histograms independently usually produces wrong colors, because it changes the *ratios* between channels and therefore the hue. The safer approach is to equalize only an intensity channel and leave chromaticity alone:

1. Convert to a model that separates brightness (HSI, HSV, or L*a*b*).
2. Equalize I (or V, or L^*) using the Chapter 3 method.
3. Convert back.

```
import numpy as np
from skimage import data, util, exposure, color

img = util.img_as_float(data.astronaut())
per_channel = np.stack([exposure.equalize_hist(img[..., k]) for k in range(3)], -1)  #
 ↪  shifts hues
lab = color.rgb2lab(img)
lab[..., 0] = 100 * exposure.equalize_hist(lab[..., 0] / 100)                        #
 ↪  lightness only
better = np.clip(color.lab2rgb(lab), 0, 1)
```

Hue is preserved, but some colors may look washed out or leave the gamut on the way back, so a clip or mild saturation boost is sometimes needed. Local histogram methods (Chapter 3) apply to L^* the same way.

Color image smoothing and sharpening

In plain words. To blur or sharpen a color photo you can blur or sharpen each of the three channels, because these are linear operations. But doing it on brightness only is often cleaner.

Smoothing

Neighborhood averaging of a color image is

$$\bar{\mathbf{c}}(x,y) = \frac{1}{K} \sum_{(s,t)\in S_{xy}} \mathbf{c}(s,t),$$

where S_{xy} is the neighborhood and K its size. As shown earlier, this equals averaging each channel separately, so `cv2.GaussianBlur` on a 3-channel array does the right thing.

An alternative is to smooth only the intensity of an HSI (or L^*) image and keep hue and saturation. The results differ near edges: per-channel smoothing creates intermediate colors along a red–green boundary (an olive line), while intensity-only smoothing keeps the original hues and only softens brightness. Which you want depends on the task: denoising usually prefers smoothing chroma *more* than luma, since the eye is less sensitive to chroma blur (the same fact that YCbCr subsampling exploits).

Sharpening

The Laplacian of a vector is the vector of the Laplacians of its components:

$$\nabla^2\mathbf{c}(x,y) = \begin{bmatrix} \nabla^2 R(x,y) \\ \nabla^2 G(x,y) \\ \nabla^2 B(x,y) \end{bmatrix},$$

so Laplacian sharpening, $\mathbf{g} = \mathbf{c} - k\,\nabla^2\mathbf{c}$ with strength $k > 0$, can be done per channel. Unsharp masking (Chapter 3) works the same way. In practice sharpening only the intensity channel avoids colored halos ("fringes") around high-contrast edges, which appear when the three channels overshoot by different amounts.

Using color in image segmentation

In plain words. Color is one of the easiest clues for splitting an image into parts: "everything brown is stained tissue, everything blue is background." The question is how to measure "brown enough".

Segmentation in HSI space

If the target is described by its hue, HSI is a natural choice. A typical recipe: threshold saturation to build a mask of pixels that actually have a color (hue is meaningless at low saturation), multiply the hue image by this mask, then threshold or histogram the masked hue to pick the target range. The weakness is the hue singularity: noise in near-gray pixels produces random hues, which is why the saturation mask comes first.

Segmentation in RGB vector space

RGB vectors often give better results because they avoid the unstable hue computation. Choose a set of sample pixels from the target, compute their mean color $\mathbf{a}$, and classify every pixel $\mathbf{z}$ by its distance to $\mathbf{a}$. The **Euclidean distance** is

$$D(\mathbf{z}, \mathbf{a}) = \|\mathbf{z} - \mathbf{a}\| = \left[(\mathbf{z} - \mathbf{a})^{\mathsf{T}}(\mathbf{z} - \mathbf{a})\right]^{1/2},$$

and pixels with $D \le D_0$ are labeled as target. The set $D \le D_0$ is a solid sphere of radius D_0. Sample colors, however, usually spread more in some directions than others (for stained tissue, mostly along the light-to-dark direction). The **Mahalanobis distance** accounts for that:

$$D(\mathbf{z}, \mathbf{a}) = \left[(\mathbf{z} - \mathbf{a})^{\mathsf{T}}\mathbf{C}^{-1}(\mathbf{z} - \mathbf{a})\right]^{1/2},$$

where $\mathbf{C}$ is the 3×3 covariance matrix of the samples. Now $D \le D_0$ is an ellipsoid aligned with the data. A cheaper approximation is a **bounding box** centered at $\mathbf{a}$ whose half-width along each axis is a multiple of that channel's standard deviation; testing a box needs only comparisons, no squares or square roots.

```
import numpy as np
from skimage import data, util

img = util.img_as_float(data.immunohistochemistry())
samples = img[60:100, 60:100].reshape(-1, 3)        # a brown (stained) region
a, C = samples.mean(0), np.cov(samples.T)
z = img.reshape(-1, 3) - a
d_euc = np.sqrt((z ** 2).sum(1)).reshape(img.shape[:2])
d_mah = np.sqrt(np.einsum("ij,jk,ik->i", z, np.linalg.inv(C), z)).reshape(img.shape[:2])
mask_euc, mask_mah = d_euc <= 0.25, d_mah <= 3.0
```

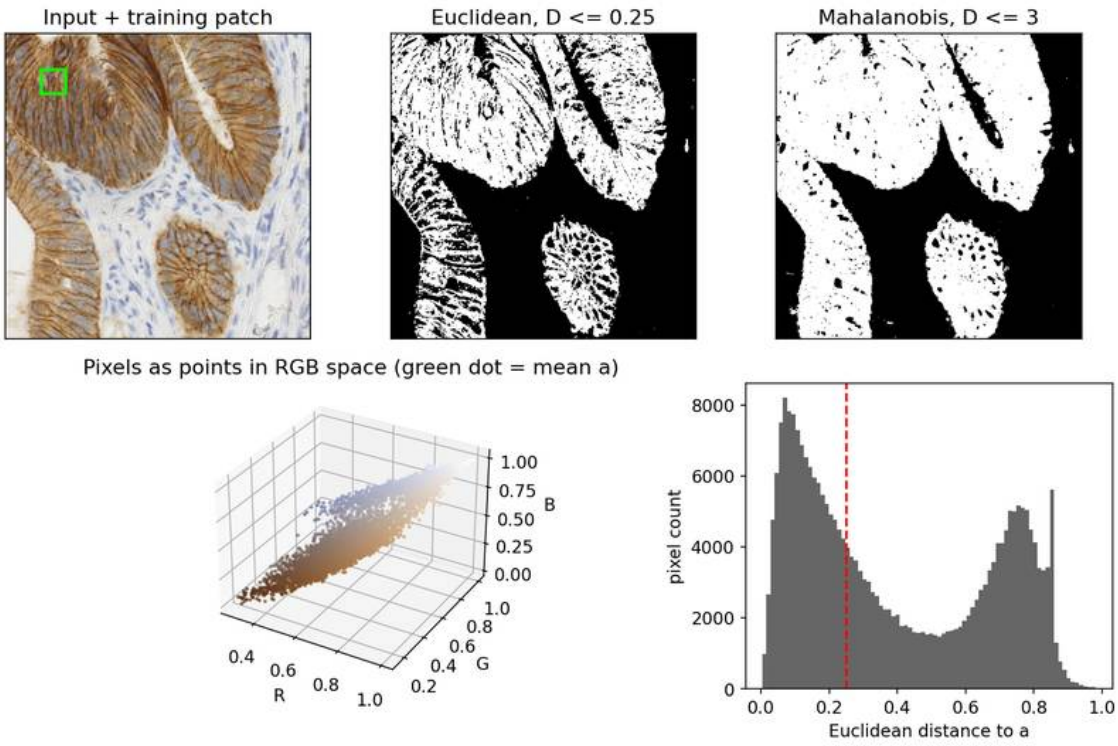


Figure 6.7 — Segmentation in RGB vector space. Top: stained tissue sample with the training patch (green box); Euclidean sphere with radius 0.25; Mahalanobis ellipsoid with radius 3. The ellipsoid follows the elongated brown cluster and fills the stained regions more completely. Bottom: pixels form an elongated cloud from dark brown to pale blue; the distance histogram is bimodal, which guides the choice of threshold.

Color edge detection

Computing a gradient on each channel and adding the results is a common shortcut, but it is not the gradient of the vector image. Worse, edge detection on a grayscale version misses **isoluminant** edges, boundaries between two colors of the same brightness. Di Zenzo [13] defined a proper gradient for multi-channel images. With $\mathbf{u} = (\partial R/\partial x, \partial G/\partial x, \partial B/\partial x)$ and $\mathbf{v} = (\partial R/\partial y, \partial G/\partial y, \partial B/\partial y)$, define

$$g_{xx} = \mathbf{u}\cdot\mathbf{u}, \qquad g_{yy} = \mathbf{v}\cdot\mathbf{v}, \qquad g_{xy} = \mathbf{u}\cdot\mathbf{v}.$$

These form the 2×2 structure tensor $\begin{bmatrix} g_{xx} & g_{xy} \\ g_{xy} & g_{yy} \end{bmatrix}$. The direction of maximum change and the rate of change in that direction are

$$\theta(x,y) = \frac{1}{2}\tan^{-1}\!\left(\frac{2g_{xy}}{g_{xx}-g_{yy}}\right), \qquad F_\theta(x,y) = \left\{\frac{1}{2}\Big[(g_{xx}+g_{yy}) + (g_{xx}-g_{yy})\cos 2\theta + 2g_{xy}\sin 2\theta\Big]\right\}^{1/2}.$$

θ is the gradient direction (use `arctan2` and also test $\theta + 90°$, since the formula gives both extrema), and F_θ the edge strength. Equivalently, F_θ^2 is the largest eigenvalue of the structure tensor,

$\lambda_{\max} = \frac{1}{2}[g_{xx} + g_{yy} + \sqrt{(g_{xx} - g_{yy})^2 + 4g_{xy}^2}]$, which is the numerically safer way to compute it.

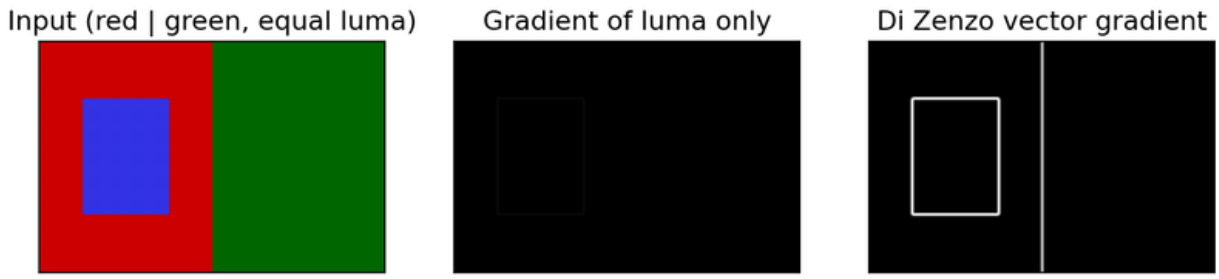


Figure 6.8 — An isoluminant edge. The red and green halves have the same BT.601 luma, so a gradient of the luma image sees no boundary between them (and only a faint square). The Di Zenzo vector gradient finds both.

```
import numpy as np
from skimage import filters

def di_zenzo(img):
    """Edge strength of a float RGB image via the largest structure-tensor eigenvalue."""
    gx = np.stack([filters.sobel_v(img[..., k]) for k in range(3)], -1)   # d/dx
    gy = np.stack([filters.sobel_h(img[..., k]) for k in range(3)], -1)   # d/dy
    gxx, gyy, gxy = (gx * gx).sum(-1), (gy * gy).sum(-1), (gx * gy).sum(-1)
    return np.sqrt(0.5 * (gxx + gyy + np.sqrt((gxx - gyy) ** 2 + 4 * gxy ** 2)))
```

Noise in color images

In plain words. Each color channel gets its own noise. When you turn the noisy channels back into hue and saturation, the noise mixes and can look much worse, especially in dark or gray areas.

The noise models of Chapter 5 apply to each channel. In a camera, noise arises mostly in the sensor, before color is reconstructed. Most sensors sit behind a color filter array, so each photosite measures one color and the other two are interpolated (**demosaicking**); the interpolation spreads and correlates noise across channels. Lukac et al. [12] review this whole chain.

Two observations shape practice:

- **Color conversions redistribute noise.** Intensity $I = (R + G + B)/3$ averages three channels, so independent zero-mean noise with variance σ^2 per channel becomes $\sigma^2/3$ in I: intensity is *cleaner* than any channel. Hue and saturation, in contrast, are nonlinear functions with large derivatives near the gray axis, so they become much *noisier*. This is why denoisers often work in a luma–chroma space and smooth chroma more strongly.
- **Impulse noise in one channel is a colored speck.** A "salt" spike in the green channel alone makes a bright green dot. A per-channel median removes it, but in mixed cases can create colors that were not in the neighborhood.

The **vector median filter** avoids invented colors. In each window it outputs the input vector that minimizes the sum of distances to all other vectors in the window [12]:

$$\mathbf{c}_{\mathrm{VM}} = \arg\min_{\mathbf{c}_i \in W} \sum_{\mathbf{c}_j \in W} \|\mathbf{c}_i - \mathbf{c}_j\|,$$

where W is the set of color vectors in the window. The output is always one of the inputs, and for one-dimensional data it reduces to the ordinary median.

```
import numpy as np
from skimage import data, util

def vector_median(img, k=3):
    r = k // 2
    H, W, _ = img.shape
    p = np.pad(img, ((r, r), (r, r), (0, 0)), mode="edge")
    nb = np.stack([p[i:i + H, j:j + W] for i in range(k) for j in range(k)], 2)  # (H, W,
     ↪  k*k, 3)
    D = np.linalg.norm(nb[:, :, :, None] - nb[:, :, None, :], axis=-1).sum(-1)    # (H, W,
     ↪  k*k)
    return np.take_along_axis(nb, D.argmin(-1)[..., None, None], 2)[:, :, 0]

img = util.img_as_float(data.astronaut())
rng = np.random.default_rng(0)
noisy = img.copy()
hit = rng.random(img.shape[:2]) < 0.05
noisy[hit] = rng.random((hit.sum(), 3))          # 5% random color impulses
out = vector_median(noisy)
psnr = lambda x: 10 * np.log10(1 / ((x - img) ** 2).mean())
print(round(psnr(noisy), 1), round(psnr(out), 1))  # about 20.4 -> 31.3 dB
```

This vectorized form uses memory proportional to k^4 per pixel; use a loop for large windows.

Color image compression

In plain words. A color image has three times the data of a gray one, but much is redundant: the channels look alike, and our eyes see little color detail. Codecs exploit both facts:

1. **Decorrelate the channels.** R, G and B are strongly correlated (Figure 6.2 shows three very similar images). Converting to luma plus two chroma differences, such as YCbCr [6], concentrates most of the energy in one channel.
2. **Subsample chroma.** Because the eye resolves less color detail than brightness detail, the chroma channels can be stored at half resolution horizontally (4:2:2) or in both directions (4:2:0) with little visible loss. 4:2:0 alone halves the raw data: $1 + 2 \cdot \frac{1}{4} = 1.5$ samples per pixel instead of 3.

Each channel is then compressed with the transform, quantization and entropy coding methods

of Chapter 8. Typical color artifacts are bleeding across sharp edges (from chroma subsampling) and blocky color patches at high compression.

Modern view

The classical tools above still run inside every camera and image library. What changed after about 2010 is mostly *how the parameters are chosen*: learned from data instead of fixed by assumption. Five reviews and survey-style papers map the territory.

Color constancy: from assumptions to learning. Gijsenij, Gevers and van de Weijer [14] survey computational color constancy and sort methods into *static* methods with fixed assumptions (gray world, white patch, gray edge and the Minkowski family above), *gamut-based* methods that compare the observed colors with those seen under a canonical light, and *learning-based* methods trained on images with known illuminants. They also propose comparison criteria and benchmark public algorithms on public datasets; a recurring finding is that no single fixed assumption suits every scene. Learning took over afterwards. Fast Fourier Color Constancy (FFCC) [15] turns illuminant estimation into finding a peak in a log-chroma histogram, solved in the frequency domain; it returns a full distribution over illuminants and runs in real time on phones, which allows temporally smooth auto white balance in video. FC4 [16] is a fully convolutional network in which every patch votes for the illuminant with a learned confidence, so a plain wall counts less than a face or a white object. Afifi and Brown [17] correct white balance *after* the camera has already rendered a nonlinear sRGB image, which the diagonal model cannot do: their network re-renders the image under other white-balance settings. The diagonal model is still the backbone; the estimate and the corrections in rendered space are now learned.

Vector filtering for color. Lukac et al. [12] review filters that treat color pixels as vectors: vector median and vector directional filters, their variants, vector edge detectors, and demosaicking. Their takeaway matches this chapter: componentwise processing ignores inter-channel correlation and can create color artifacts, while vector order statistics keep the input's colors. Learned denoisers are now stronger when training data and compute are available, but these filters remain standard baselines where speed and predictability matter.

Colormap design. Kovesi [7] gives a design method and test images for linear, diverging, rainbow and cyclic colormaps, built on one principle: perceived lightness must change uniformly along the map. The viridis family [8] became Matplotlib's default in version 2.0. Crameri et al. [9] review how color is used across scientific publishing and argue that non-uniform and red–green maps should disappear from software defaults. Deep learning changed nothing here: the problem is human perception, and the answer is careful design measured in a perceptual color space.

Colorization with deep learning. Pseudocolor maps gray to color by a fixed rule; *colorization* tries to predict the *true* colors. Zhang, Isola and Efros [18] treat it as classification over quantized colors and rebalance rare classes so that vivid colors are not averaged into brown; they evaluate with a "colorization Turing test" in which people judge whether an image is real. Anwar et al. [19] survey deep colorization, group methods into seven categories, discuss losses and metrics, and introduce a new benchmark dataset. DDColor [20] (ICCV 2023) uses two decoders, one restoring spatial detail and one refining color with cross-attention to multi-scale semantic features, plus a

colorfulness loss; this reduces color bleeding across object boundaries. Two classical ideas persist: work in L*a*b* (predict chroma, keep the given lightness) and treat color as a distribution, because a gray shirt could have been any color.

What did not change. Colorimetry (CIE XYZ, chromaticity, gamuts), encodings (sRGB, BT.601/709 YCbCr) and color-difference spaces are standards, not things to learn. Networks are trained on sRGB data and judged in these spaces. Knowing exactly which space your tensors are in (encoded or linear, BT.601 or BT.709, studio or full range) remains one of the most practical skills in color work.

Key takeaways

- Human vision is trichromatic, so three numbers describe a color for a human observer; different spectra with equal responses (metamers) look the same.
- The CIE xy chromaticity diagram shows all visible chromaticities; a three-primary device covers only a triangle (its gamut).
- RGB suits hardware, CMY(K) suits ink, HSI separates intensity from hue and saturation, and L*a*b* makes Euclidean distance roughly perceptual. Watch for undefined hue on grays, sRGB gamma, and BT.601 versus BT.709.
- Pseudocolor makes gray data readable; use perceptually uniform colormaps such as viridis, not rainbow maps.
- Linear operations (smoothing, Laplacian sharpening) can be done per channel; nonlinear ones (median, histogram equalization, edge detection, segmentation) should treat pixels as vectors or work on an intensity channel.
- White balance divides by an estimated illuminant; gray world, white patch and gray edge are special cases of one framework, and modern systems learn the estimate.
- Segment color with vector distances (Euclidean, Mahalanobis, box), and detect color edges with the Di Zenzo gradient, which sees isoluminant boundaries.
- Intensity is less noisy than any single channel, hue and saturation are more noisy; vector median filters remove color impulses without inventing new colors.

Exercises

1. A pixel has $(R, G, B) = (0.6, 0.6, 0.2)$. Compute its H, S and I by hand. Then predict, without computing, how H changes if B increases to 0.3.

   Hint

   $I = 0.4667$. min $= 0.2$, so $S = 1 - 0.2/0.4667 \approx 0.571$. For θ: numerator $= 0.5(0 + 0.4) = 0.2$, denominator $= \sqrt{0 + 0.4 \cdot 0.4} = 0.4$, so $\theta = \cos^{-1}(0.5) = 60°$, and since $B \leq G$, $H = 60°$ (yellow). Raising B to 0.3 keeps $R = G$, so hue stays 60°; only S drops.

2. Show that the RGB negative $s_i = 1 - r_i$ maps a pixel with HSI coordinates (H, S, I) to one whose intensity is $1 - I$ and whose hue is $H + 180°$ (for non-gray pixels). Is the saturation preserved?

Hint

Intensity follows from linearity. For hue, the chromatic part $\mathbf{c} - I\,\mathbf{1}$ simply changes sign, which rotates it by 180° around the gray axis. Saturation is $1 - \min / I$; after negation the minimum becomes $1 - \max(R, G, B)$ and the intensity $1 - I$, so in general saturation changes. Try $(0.9, 0.1, 0.1)$.

3. You equalize the R, G and B histograms of a photo of a blue sky separately. Explain why the sky may turn gray-cyan, and propose two fixes.

Hint

Equalization stretches each channel to use the full range. A channel that was almost uniformly high (blue) can be pushed down relative to the others, while weak channels are boosted, so the R:G:B ratio changes. Fix by equalizing I, V or L^* only, or by applying one common mapping computed from the average histogram to all three channels.

4. In the gray-world/white-patch framework, what estimator do you get with $n = 0$ and $p = 2$? Run the `white_balance` idea on the cat image with $p = 1, 2, 6, \infty$ (use a high percentile for ∞) and plot the angular error versus p.

Hint

$p = 2$ estimates the illuminant from the root-mean-square value of each channel, a compromise between the mean and the maximum. With the chapter's synthetic cast the error falls from about 12° ($p = 1$) to about 11° ($p = 2$), 7° ($p = 6$) and 4° ($p = 20$). It falls as p grows on this image, because its bright white details are a better clue than its orange average. On a scene without white objects the trend can reverse.

5. Design a color segmenter for green vegetation in outdoor photos that is robust to shadows. Which space and which distance would you use, and why might plain RGB Euclidean distance fail?

Hint

Shadows scale all three channels down, which moves a pixel along a line through the origin and changes its RGB Euclidean distance a lot. Use a chromaticity-based feature that is unaffected by scaling (normalized $r = R/(R + G + B)$, $g = G/(R + G + B)$, or hue with a saturation mask, or a^*, b^*), or a Mahalanobis distance whose covariance is trained on sunlit and shaded samples.

6. Explain why the vector median filter never creates a new color, and give a 3-pixel, 1-D example where the per-channel median does.

Hint

By definition the vector median is one of the window's input vectors. For three pixels $(1, 1, 0)$, $(1, 0, 1)$, $(0, 1, 1)$ (yellow, magenta, cyan) the per-channel median is $(1, 1, 1)$, pure white, which appears nowhere in the window.

The figures in this chapter are generated by `scripts/figures/dip_ch06.py` from `skimage.data` images and synthetic arrays.

References

1. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018, Ch. 6. publisher page
2. C. Wyman, P.-P. Sloan, and P. Shirley, "Simple Analytic Approximations to the CIE XYZ Color Matching Functions," *Journal of Computer Graphics Techniques*, vol. 2, no. 2, 2013. jcgt.org
3. IEC 61966-2-1:1999, "Multimedia systems and equipment – Colour measurement and management – Part 2-1: Colour management – Default RGB colour space – sRGB," IEC, 1999. IEC webstore
4. Recommendation ITU-R BT.709-6, "Parameter values for the HDTV standards for production and international programme exchange," ITU, 2015. ITU
5. ISO/CIE 11664-4:2019, "Colorimetry – Part 4: CIE 1976 L*a*b* colour space," ISO/CIE, 2019. standard page
6. Recommendation ITU-R BT.601-7, "Studio encoding parameters of digital television for standard 4:3 and wide screen 16:9 aspect ratios," ITU, 2011. ITU
7. P. Kovesi, "Good Colour Maps: How to Design Them," arXiv:1509.03700, 2015. arXiv
8. S. van der Walt and N. Smith, viridis colormap design notes (BIDS project page; presented at SciPy 2015), 2015. project page
9. F. Crameri, G. E. Shephard, and P. J. Heron, "The misuse of colour in science communication," *Nature Communications*, vol. 11, art. 5444, 2020. PMC
10. G. Buchsbaum, "A spatial processor model for object colour perception," *Journal of the Franklin Institute*, vol. 310, no. 1, pp. 1–26, 1980. DOI
11. J. van de Weijer, T. Gevers, and A. Gijsenij, "Edge-Based Color Constancy," *IEEE Transactions on Image Processing*, vol. 16, no. 9, pp. 2207–2214, 2007. DOI
12. R. Lukac, B. Smolka, K. Martin, K. N. Plataniotis, and A. N. Venetsanopoulos, "Vector Filtering for Color Imaging," *IEEE Signal Processing Magazine*, vol. 22, no. 1, pp. 74–86, Jan. 2005. PDF
13. S. Di Zenzo, "A note on the gradient of a multi-image," *Computer Vision, Graphics, and Image Processing*, vol. 33, pp. 116–125, 1986. DOI
14. A. Gijsenij, T. Gevers, and J. van de Weijer, "Computational Color Constancy: Survey and Experiments," *IEEE Transactions on Image Processing*, vol. 20, no. 9, pp. 2475–2489, 2011. DOI
15. J. T. Barron and Y.-T. Tsai, "Fast Fourier Color Constancy," *CVPR*, 2017. arXiv
16. Y. Hu, B. Wang, and S. Lin, "FC4: Fully Convolutional Color Constancy with Confidence-Weighted Pooling," *CVPR*, 2017. paper
17. M. Afifi and M. S. Brown, "Deep White-Balance Editing," *CVPR*, 2020. arXiv
18. R. Zhang, P. Isola, and A. A. Efros, "Colorful Image Colorization," arXiv:1603.08511, 2016. arXiv
19. S. Anwar, M. Tahir, C. Li, A. Mian, F. S. Khan, and A. W. Muzaffar, "Image Colorization: A Survey and Dataset," arXiv:2008.10774, 2020. arXiv
20. X. Kang, T. Yang, W. Ouyang, P. Ren, L. Li, and X. Xie, "DDColor: Towards Photo-Realistic Image Colorization via Dual Decoders," *ICCV*, 2023. arXiv

IP 07 • Wavelet and Other Image Transforms

Treat every image transform as a change of basis, then build up from the DCT and Walsh–Hadamard transforms to Haar, multiresolution analysis and the fast wavelet transform.

Online edition: https://blog.shyandram.dev/en/learn/image-processing/dip-07-wavelets-transforms/

What you'll learn

- Why every linear image transform (Fourier, cosine, Walsh–Hadamard, Haar, wavelet) is the same idea: describe the image in a different set of building blocks, called a **basis**.
- How to write a transform as a pair of matrix products, and why separable 2-D transforms are cheap.
- What basis images look like for the DCT, Walsh–Hadamard, slant and Haar transforms, and why the DCT is the workhorse of JPEG.
- How the time–frequency plane explains the difference between Fourier analysis and wavelet analysis.
- How multiresolution analysis, scaling functions and wavelet functions lead to the fast wavelet transform, a simple two-filter bank.
- How to use the 2-D discrete wavelet transform for denoising, edge detection and compression, with PyWavelets.

The big picture

In Chapter 4 you described an image as a sum of sinusoids. That is one choice of building blocks, but not the only one. This chapter steps back and asks a more general question: *given any set of building blocks, how do I find out how much of each one is in my image, and which set is best for the job?*

> **Explain like I'm five**
>
> Imagine describing a LEGO model to a friend over the phone. You could list every single stud ("red at row 1, column 1; red at row 1, column 2; ..."). That always works, but it is slow. Or you could say "a 2×4 red brick here, a 1×2 blue brick there". Now the description is short, because the bricks match the shapes in the model.
>
> An image transform is a choice of bricks. The pixel grid uses tiny 1×1 bricks. The Fourier and cosine transforms use stripy bricks. Wavelets use bricks of every size: big blurry ones for the sky, tiny sharp ones for edges. When the bricks match the picture, only a few of them are needed, and that is what makes compression, denoising and fast analysis possible.

The rigorous version of "choose your bricks" is linear algebra: inner products, bases and change of basis. Once that machinery is in place, every transform in this chapter is just a different matrix.

Wavelets get the most attention, because they are the first basis that is localized in *both* space and frequency. That property made them the core of the JPEG 2000 standard [15] and an important idea behind modern sparse modeling and, more recently, some neural-network layers.

Preliminaries

Plain version: a signal with N samples is an arrow (a vector) in an N-dimensional space. A transform measures the shadow that arrow casts on each of N reference arrows. If the reference arrows are perpendicular and of unit length, the shadows tell you everything and you can rebuild the arrow by adding the shadows back.

Inner product spaces

An **inner product** $\langle \mathbf{f}, \mathbf{g} \rangle$ takes two vectors and returns a number that measures how much they point the same way. For real or complex vectors of length N,

$$\langle \mathbf{f}, \mathbf{g} \rangle = \sum_{x=0}^{N-1} f(x)\, g^{*}(x), \qquad \|\mathbf{f}\| = \sqrt{\langle \mathbf{f}, \mathbf{f} \rangle}.$$

Here $f(x)$ and $g(x)$ are the x-th entries of $\mathbf{f}$ and $\mathbf{g}$, g^{*} is the complex conjugate (it does nothing for real data), and $\|\mathbf{f}\|$ is the length, or **norm**, of $\mathbf{f}$. For continuous functions, the sum becomes an integral, $\langle f, g \rangle = \int f(x) g^{*}(x)\, dx$. Two vectors are **orthogonal** when their inner product is zero: they share nothing.

Orthonormal bases

A set of vectors $\{\mathbf{s}_0, \dots, \mathbf{s}_{N-1}\}$ is an **orthonormal basis** if every vector has unit length and every pair is orthogonal: $\langle \mathbf{s}_u, \mathbf{s}_v \rangle = \delta(u - v)$, where δ is 1 when $u = v$ and 0 otherwise. Then any vector splits cleanly into

$$t(u) = \langle \mathbf{f}, \mathbf{s}_u \rangle, \qquad \mathbf{f} = \sum_{u=0}^{N-1} t(u)\, \mathbf{s}_u.$$

The numbers $t(u)$ are the **transform coefficients**. The first equation is the **analysis** (forward) step and the second is the **synthesis** (inverse) step. Orthonormal bases also preserve energy, $\sum_x |f(x)|^2 = \sum_u |t(u)|^2$ (Parseval's theorem), which is why we can talk about "how much energy" sits in each coefficient.

Biorthogonal bases

Sometimes the most useful building blocks are not perpendicular. Then you need two families: an analysis set $\{\tilde{\mathbf{s}}_u\}$ for measuring and a synthesis set $\{\mathbf{s}_u\}$ for rebuilding. They form a **biorthogonal** pair when

$$\langle \mathbf{s}_u, \tilde{\mathbf{s}}_v \rangle = \delta(u - v), \qquad t(u) = \langle \mathbf{f}, \tilde{\mathbf{s}}_u \rangle, \qquad \mathbf{f} = \sum_u t(u)\, \mathbf{s}_u.$$

The "measuring sticks" $\tilde{\mathbf{s}}_u$ are the **dual** basis. Biorthogonality buys freedom. For example, JPEG 2000 uses biorthogonal wavelet filters, which can be short *and* symmetric, a combination that is hard to get with orthonormal wavelets [15].

Frames

A **frame** is a spanning set with *more* vectors than dimensions, so it is redundant. Its defining property is that the measured energy stays within fixed bounds:

$$A\,\|\mathbf{f}\|^2 \;\le\; \sum_k |\langle \mathbf{f}, \mathbf{s}_k \rangle|^2 \;\le\; B\,\|\mathbf{f}\|^2, \qquad 0 < A \le B < \infty.$$

A and B are the **frame bounds**. If $A = B$ the frame is **tight**, and reconstruction is just $\mathbf{f} = \frac{1}{A}\sum_k \langle \mathbf{f}, \mathbf{s}_k \rangle \mathbf{s}_k$. A classic picture: three unit arrows in the plane, 120° apart. They span the plane, no two are orthogonal, and they form a tight frame with $A = B = 3/2$. Redundant frames are worth the extra coefficients when you want robustness or shift invariance, as in the undecimated wavelet transform and the scattering transform discussed in the Modern view.

```
import numpy as np

v = np.array([3.0, 1.0, 2.0])
e = np.eye(3)                          # standard orthonormal basis
c = e @ v                              # coefficients = inner products <v, e_k>
print(c, np.allclose(c @ e, v))        # synthesis: sum_k c_k e_k

# A biorthogonal pair in 2-D: analysis rows A, synthesis columns B = A^{-1}
A = np.array([[1.0, 0.0], [1.0, 1.0]])
B = np.linalg.inv(A)
x = np.array([2.0, 5.0])
print(A @ x, B @ (A @ x))              # [2. 7.] -> back to [2. 5.]
```

Matrix-based transforms

Plain version: stack the measuring vectors as the rows of a matrix. Multiplying the signal by that matrix measures all the coefficients at once; multiplying by the inverse rebuilds the signal.

For a 1-D signal $\mathbf{f}$ of length N, put the basis vectors $\mathbf{s}_u^{*T}$ in the rows of an $N \times N$ **transformation matrix** $\mathbf{A}$. Then

$$\mathbf{t} = \mathbf{A}\,\mathbf{f}, \qquad \mathbf{f} = \mathbf{A}^{-1}\mathbf{t}.$$

When the basis is orthonormal, $\mathbf{A}$ is **unitary** ($\mathbf{A}^{-1} = \mathbf{A}^{*T}$; for real matrices, **orthogonal**, $\mathbf{A}^{-1} = \mathbf{A}^T$), so the inverse is free.

For an $M \times N$ image $\mathbf{F}$, a general 2-D linear transform would need an $MN \times MN$ matrix. Nearly all practical image transforms are **separable**: they apply a 1-D transform down every column and then along every row. In matrix form,

$$\mathbf{T} = \mathbf{A}\,\mathbf{F}\,\mathbf{B}^T, \qquad \mathbf{F} = \mathbf{A}^{-1}\,\mathbf{T}\,(\mathbf{B}^T)^{-1},$$

where $\mathbf{A}$ $(M \times M)$ acts on columns and $\mathbf{B}$ $(N \times N)$ on rows; for a square image and the same transform in both directions, $\mathbf{B} = \mathbf{A}$. Separability cuts the cost of a direct $N \times N$ transform from $O(N^4)$ to $O(N^3)$ operations, and fast algorithms (FFT, fast DCT, fast wavelet transform) reduce it further.

```
import numpy as np
from scipy.fft import dct, dctn
from skimage import data, img_as_float

N = 8
A = dct(np.eye(N), norm="ortho", axis=0)  # row u = u-th cosine basis vector

print(np.allclose(A @ A.T, np.eye(N)))    # orthonormal -> True

F = img_as_float(data.camera())[200:208, 200:208]
T = A @ F @ A.T                           # forward 2-D transform
print(np.allclose(T, dctn(F, norm="ortho")))   # same as scipy -> True
print(np.allclose(A.T @ T @ A, F))             # inverse -> True
```

Correlation

Plain version: each coefficient answers the question "how much does my image look like this particular pattern?" That question is a correlation.

Compare the analysis equation with the cross-correlation from Chapter 3. The coefficient

$$t(u) = \langle \mathbf{f}, \mathbf{s}_u \rangle = \sum_x f(x)\, s_u^*(x)$$

is exactly the correlation of f with the basis function s_u, evaluated at zero shift. A large $|t(u)|$ means f and s_u are similar; $t(u) = 0$ means f contains none of s_u. Two consequences follow:

- **Transforms are template matching.** Each basis function is a template, and the transform scores every template at once. The Fourier transform scores sinusoids; the Haar transform scores step-like patterns.
- **Wavelets are correlation at many shifts and scales.** The wavelet basis contains shifted and stretched copies of a single pattern. Correlating with all the shifts of one copy is a filtering operation, which is why the wavelet transform will turn into a bank of filters later in this chapter.

Basis functions in the time-frequency plane

Plain version: some building blocks tell you *when* something happened, some tell you *which pitch* it had, and none can tell you both perfectly. The time–frequency plane is a map of that trade-off.

For a basis function $s(x)$ with unit energy, define its spread in time (or space) and in frequency as standard deviations of $|s(x)|^2$ and $|S(\omega)|^2$:

$$\sigma_x^2 = \int (x - \bar{x})^2 |s(x)|^2 \, dx, \qquad \sigma_\omega^2 = \frac{1}{2\pi} \int (\omega - \bar{\omega})^2 |S(\omega)|^2 \, d\omega,$$

where $S(\omega)$ is the Fourier transform of s, and $\bar{x}$, $\bar{\omega}$ are the centers of mass of the two energy distributions. The **Heisenberg uncertainty principle** for signals states

$$\sigma_x \, \sigma_\omega \geq \tfrac{1}{2},$$

with equality only for Gaussian-shaped functions. Draw each basis function as a rectangle (a **Heisenberg box** or tile) of width σ_x and height σ_ω centered at $(\bar{x}, \bar{\omega})$. The area can never shrink below a fixed minimum, but the *shape* is up to you.

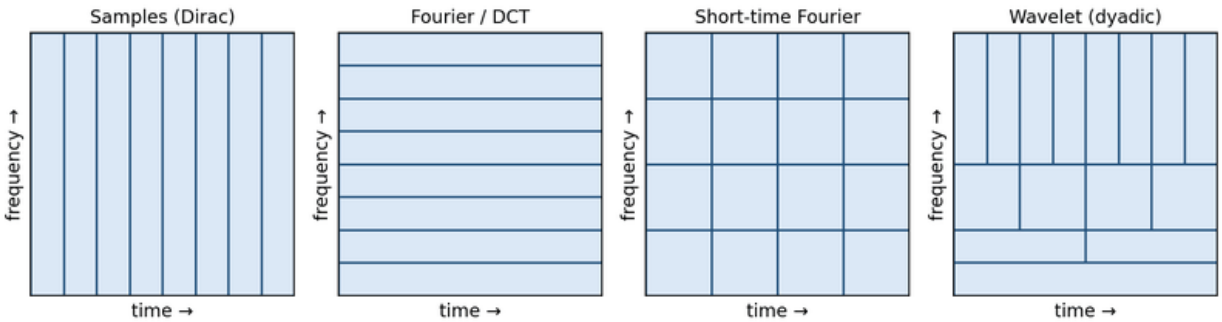


Figure 7.1 — How four bases tile the time–frequency plane. Every tile has the same area. Samples are sharp in time; Fourier/DCT bases are sharp in frequency; the short-time Fourier transform fixes one compromise for all frequencies; wavelets adapt the shape to the frequency.

Figure 7.1 shows the four standard cases:

- **Samples (the identity basis):** perfect position, no frequency information.
- **Fourier or DCT:** perfect frequency, no position. A single sharp edge spreads over all coefficients.
- **Short-time Fourier transform (STFT):** cut the signal into windows and take a Fourier transform of each. Every tile has the same shape, chosen once for all frequencies [2].
- **Wavelets:** high-frequency tiles are narrow in time and tall in frequency; low-frequency tiles are wide and short. This matches natural images well: edges are brief, high-frequency events, while smooth shading is a slow, low-frequency one.

Basis images

Plain version: for a 2-D transform, each coefficient has its own little picture. The image is a weighted sum of those pictures, and the coefficient is the weight.

For a separable transform with matrix $\mathbf{A}$ whose u-th row is $\mathbf{a}_u^T$, rewrite the inverse as a sum of outer products:

$$\mathbf{F} = \sum_{u=0}^{N-1} \sum_{v=0}^{N-1} T(u,v)\, \mathbf{S}_{u,v}, \qquad \mathbf{S}_{u,v} = \mathbf{a}_u^* \mathbf{a}_v^{*T}.$$

Each $N \times N$ matrix $\mathbf{S}_{u,v}$ is a **basis image**. $T(u,v)$ tells you how much of basis image (u,v) to add, u indexes the vertical pattern and v the horizontal one. Plotting all N^2 basis images side by side is the fastest way to understand what a transform "sees".

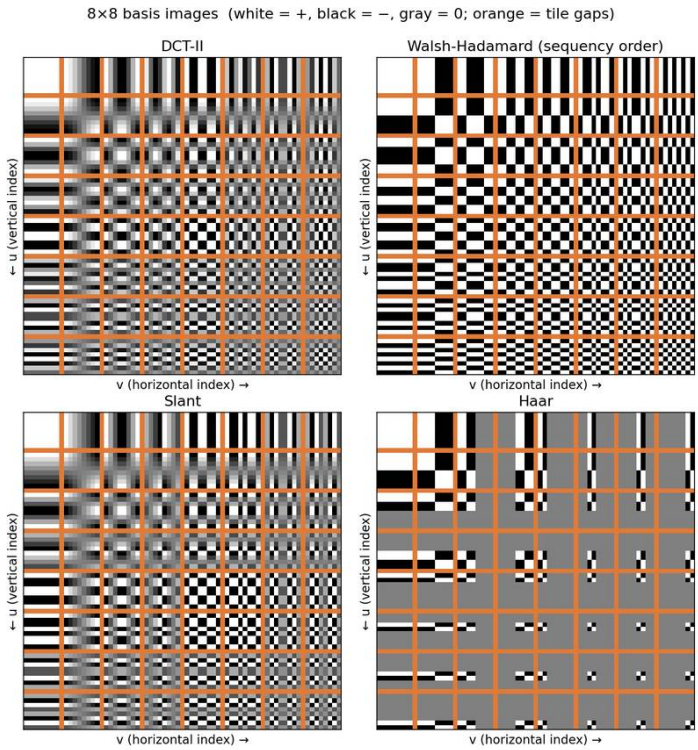


Figure 7.2 — 8×8 basis images of four transforms, arranged so that vertical frequency (sequency) grows downward and horizontal frequency grows to the right. Each tile is scaled to $[-1, 1]$. Note how Haar basis images, unlike the other three, are zero (gray) over most of the tile: they are localized.

The top-left basis image is constant in all four transforms: it measures the average brightness, called the **DC** coefficient. Moving right or down, the patterns oscillate faster. The DCT oscillates smoothly, Walsh–Hadamard switches abruptly between ± 1, slant adds linear ramps, and Haar confines each pattern to a small region of the tile.

Fourier-related transforms

Plain version: the Fourier family uses waves as building blocks. The cosine transform is the member that behaves best at the borders of a block, which is why your phone's JPEG files use it.

The discrete Fourier transform, as a matrix

The 1-D DFT from Chapter 4 is a matrix transform with

$$A_{u,x} = \frac{1}{\sqrt{N}} e^{-j2\pi ux/N}, \qquad u, x = 0, \dots, N-1.$$

$A_{u,x}$ is the entry in row u, column x; $j = \sqrt{-1}$. With the $1/\sqrt{N}$ factor the matrix is unitary. Two drawbacks for compression: the coefficients are complex, and the DFT silently treats the signal as periodic. If the left and right borders of a block differ, the periodic extension has a jump, and a jump needs many high-frequency coefficients.

The discrete cosine transform

The (type-II) **discrete cosine transform** [3] uses real cosines:

$$t(u) = \alpha(u) \sum_{x=0}^{N-1} f(x) \cos\left[\frac{(2x+1)\, u\, \pi}{2N}\right], \qquad \alpha(0) = \sqrt{\tfrac{1}{N}},\ \ \alpha(u) = \sqrt{\tfrac{2}{N}} \text{ for } u \geq 1.$$

Here u is the frequency index and $\alpha(u)$ makes the basis orthonormal. The DCT equals (up to scaling) the DFT of a mirrored, even extension of the signal of length $2N$. Mirroring removes the border jump, so energy concentrates in fewer low-frequency coefficients. This **energy compaction** is the reason the JPEG standard applies an 8×8 DCT to image blocks [4]. Ahmed, Natarajan and Rao introduced the DCT in 1974 and showed that its performance compares closely with that of the Karhunen–Loève transform, the statistically optimal transform, in Wiener filtering and rate–distortion terms [3].

The discrete sine transform

The **discrete sine transform** uses sines instead. The type-I DST matrix is

$$A_{u,x} = \sqrt{\frac{2}{N+1}} \sin\left[\frac{(x+1)(u+1)\pi}{N+1}\right],$$

which corresponds to an *odd* extension of the signal. It is real, orthonormal and symmetric ($\mathbf{A} = \mathbf{A}^T = \mathbf{A}^{-1}$). It suits signals that are close to zero at both ends, such as prediction residuals. In SciPy it is `scipy.fft.dst(x, type=1, norm="ortho")`.

Walsh–Hadamard transform

Plain version: build every basis vector from only $+1$ and -1. Then the transform needs no multiplications at all, just additions and subtractions.

The Hadamard matrix of order $N = 2^n$ is built recursively:

$$\mathbf{H}_1 = [\,1\,], \qquad \mathbf{H}_{2N} = \begin{bmatrix} \mathbf{H}_N & \mathbf{H}_N \\ \mathbf{H}_N & -\mathbf{H}_N \end{bmatrix}, \qquad \mathbf{A}_{\mathrm{WHT}} = \frac{1}{\sqrt{N}}\,\mathbf{H}_N .$$

Every entry of $\mathbf{H}_N$ is ± 1, and its rows are mutually orthogonal, so $\mathbf{A}_{\mathrm{WHT}}$ is orthonormal (and symmetric). The rows of $\mathbf{H}_N$ come out in **natural** (Hadamard) order. For analysis it is nicer to reorder them by **sequency**, the number of sign changes along a row, which plays the role of frequency. Sequency-ordered rows are called Walsh functions, and Figure 7.2 shows their 2-D basis images.

The Walsh–Hadamard transform (WHT) was used for early image coding because it is cheap [5]. Its energy compaction is weaker than the DCT's for natural images, because its square-wave basis approximates smooth shading poorly (see Figure 7.7 later). It remains useful where speed or hardware simplicity matters most, and as a building block for fast randomized algorithms.

```
import numpy as np
from scipy.linalg import hadamard

H = hadamard(8)                                  # natural (Hadamard) order
changes = (np.diff(np.sign(H), axis=1) != 0).sum(1)
W = H[np.argsort(changes)] / np.sqrt(8)          # sequency order
print((np.diff(np.sign(W), axis=1) != 0).sum(1))   # [0 1 2 3 4 5 6 7]

x = np.array([10, 12, 11, 13, 40, 42, 41, 43], float)
print(np.round(W @ x, 2))
```

The test signal is a step with small ripples. Its WHT is almost entirely in coefficient 0 (the mean) and coefficient 1 (a single sign change, matching the step); only a few small values remain.

Slant transform

Plain version: many image regions are not flat but get gradually brighter or darker. The slant transform includes a "ramp" building block so that such regions need only one or two coefficients.

Pratt, Chen and Welch designed the **slant transform** for image coding [6]. Like the WHT, it is orthonormal and built recursively, starting from $\mathbf{S}_2 = \frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix}$:

$$\mathbf{S}_N = \frac{1}{\sqrt{2}}\,\mathbf{Q}_N \begin{bmatrix} \mathbf{S}_{N/2} & \mathbf{0} \\ \mathbf{0} & \mathbf{S}_{N/2} \end{bmatrix},$$

where $\mathbf{Q}_N$ is a sparse $N \times N$ mixing matrix. $\mathbf{Q}_N$ contains two constants,

$$a_N = \sqrt{\frac{3N^2}{4(N^2-1)}}, \qquad b_N = \sqrt{\frac{N^2-4}{4(N^2-1)}},$$

chosen so that the second row of $\mathbf{S}_N$ is a uniformly decreasing staircase (the "slant" vector) and the matrix stays orthonormal. The script for this chapter builds $\mathbf{S}_8$ this way and checks both properties numerically (`scripts/figures/dip_ch07.py`). In Figure 7.2 the slant basis images in the first row and first column are visible ramps. The transform has a fast algorithm of $O(N \log_2 N)$ operations, but in practice the DCT, which also handles ramps well, displaced it.

Haar transform

Plain version: take two neighbors, write down their average and their difference. Repeat on the averages. That is the Haar transform, and it is the simplest wavelet.

The Haar transform [1] uses basis functions that are either constant or a single up-down step, at different widths and positions. For $N = 2^n$ samples, index the functions by a scale p and a position q, with $k = 2^p + q$ for $k \geq 1$. Up to normalization, the k-th basis vector equals $+1$ on the first half of the interval $\left[\frac{q}{2^p}, \frac{q+1}{2^p}\right)$, -1 on its second half, and 0 elsewhere (with positions measured as a fraction of the signal length). The $k = 0$ vector is constant. For $N = 4$,

$$\mathbf{A}_{\text{Haar}} = \begin{bmatrix} \frac{1}{2} & \frac{1}{2} & \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} & -\frac{1}{2} & -\frac{1}{2} \\ \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} & 0 & 0 \\ 0 & 0 & \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{bmatrix}.$$

Rows 0 and 1 look at the whole signal; rows 2 and 3 each look at only one half. That is the key difference from every transform above: **Haar basis functions are localized**. A local event, such as an edge, changes only the few Haar coefficients whose support covers it. In the time–frequency picture, Haar has exactly the dyadic tiling of Figure 7.1. It is the bridge to the wavelet transforms that follow: the Haar transform *is* the discrete wavelet transform with the Haar wavelet.

Wavelet transforms

Plain version: look at the image at many zoom levels. At each level, keep a blurrier copy and write down only the details that the blurring removed. The blurry copies form a pyramid; the details are the wavelet coefficients.

Multiresolution analysis

Multiresolution analysis (MRA), formalized by Mallat [7], is the framework that turns this idea into a basis. Start with a **scaling function** $\varphi(x)$ and form shifted and scaled copies,

$$\varphi_{j,k}(x) = 2^{j/2}\,\varphi(2^j x - k),$$

where j is the **scale** (larger j means narrower functions and finer detail), k is the integer shift, and $2^{j/2}$ keeps the energy at 1. Let V_j be the space spanned by $\{\varphi_{j,k}\}_k$: all signals representable at resolution j. MRA requires four things:

1. The $\varphi_{0,k}$ are orthonormal (or at least a stable basis of V_0).
2. The spaces are nested, $\cdots \subset V_{-1} \subset V_0 \subset V_1 \subset \cdots$: anything visible at a coarse scale is also visible at a finer one.
3. The only function common to all V_j is $f(x) = 0$.
4. Every square-integrable function can be approximated arbitrarily well as $j \to \infty$.

Because $V_0 \subset V_1$, the scaling function itself must be a combination of the finer ones. This gives the **refinement equation** (or dilation equation)

$$\varphi(x) = \sum_n h_\varphi(n)\,\sqrt{2}\,\varphi(2x - n),$$

where the coefficients $h_\varphi(n)$ are the **scaling function coefficients**. They will become a low-pass filter.

Wavelet functions

The details lost when going from V_{j+1} down to V_j live in a complementary space W_j, so that $V_{j+1} = V_j \oplus W_j$ (the symbol $\oplus$ means every element of V_{j+1} splits uniquely into a part in V_j and a part in W_j; for orthonormal wavelets the two parts are orthogonal). W_j is spanned by the **wavelet functions**

$$\psi_{j,k}(x) = 2^{j/2}\,\psi(2^j x - k), \qquad \psi(x) = \sum_n h_\psi(n)\,\sqrt{2}\,\varphi(2x - n).$$

For orthonormal wavelets, the **wavelet function coefficients** follow from the scaling coefficients by a modulation and time reversal, $h_\psi(n) = (-1)^n\,h_\varphi(1 - n)$. They form a high-pass filter. Repeating the split gives

$$L^2(\mathbb{R}) = V_{j_0} \oplus W_{j_0} \oplus W_{j_0+1} \oplus \cdots,$$

where $L^2(\mathbb{R})$ is the space of finite-energy functions: one coarse approximation plus details at every finer scale.

For Haar, $h_\varphi = [\frac{1}{\sqrt{2}}, \frac{1}{\sqrt{2}}]$ and $h_\psi = [\frac{1}{\sqrt{2}}, -\frac{1}{\sqrt{2}}]$: average and difference. Daubechies showed how to construct orthonormal wavelets with **compact support** (finite-length filters) and increasing smoothness; the regularity grows linearly with the filter length [8]. These are the `dbN` families in PyWavelets [10], where `N` is the number of **vanishing moments**: $\int x^m \psi(x)\,dx = 0$ for $m = 0, \ldots,$

$N-1$. A wavelet with N vanishing moments ignores polynomials of degree below N, so smooth regions produce near-zero detail coefficients.

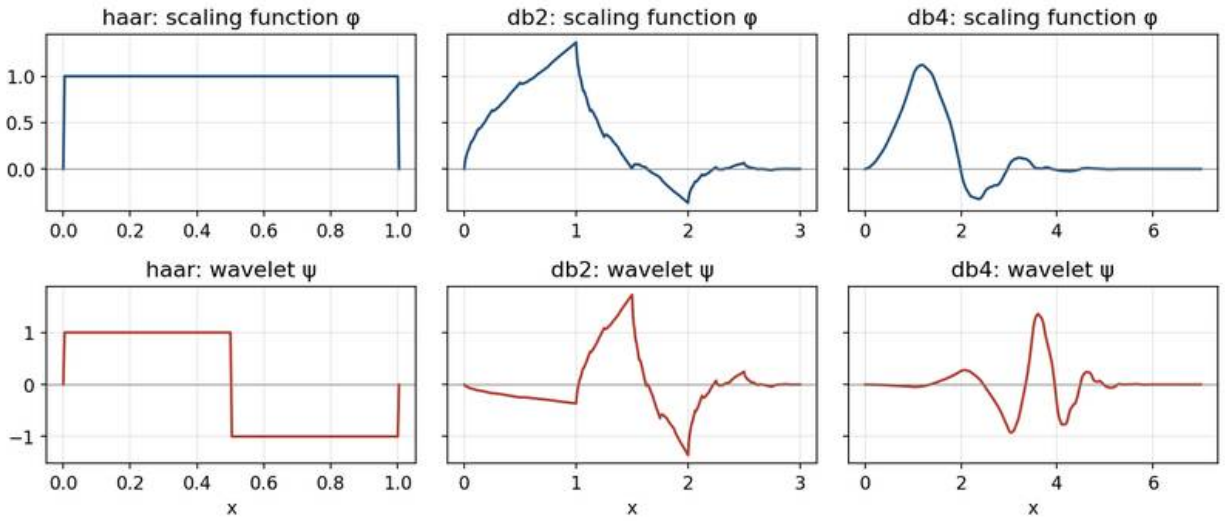


Figure 7.3 — Scaling functions (top) and wavelets (bottom) for Haar, db2 and db4, computed with PyWavelets' cascade algorithm. Longer filters give smoother, longer functions.

The wavelet series expansion

With these two families, a continuous signal $f(x)$ expands as

$$f(x) = \sum_{k} c_{j_0}(k)\,\varphi_{j_0,k}(x) + \sum_{j=j_0}^{\infty}\sum_{k} d_j(k)\,\psi_{j,k}(x),$$

$$c_{j_0}(k) = \langle f, \varphi_{j_0,k}\rangle, \qquad d_j(k) = \langle f, \psi_{j,k}\rangle.$$

j_0 is an arbitrary starting (coarsest) scale. The $c_{j_0}(k)$ are **approximation** (or scaling) coefficients and the $d_j(k)$ are **detail** (or wavelet) coefficients. Notice that every coefficient is again an inner product, exactly as in the Preliminaries. Unser and Blu explain where properties such as vanishing moments, approximation order and smoothness come from in this expansion [9].

The discrete wavelet transform in one dimension

For a sampled signal $f(x)$, $x = 0, \dots, M-1$ with $M = 2^J$, the sums become finite:

$$W_\varphi(j_0,k) = \frac{1}{\sqrt{M}}\sum_{x} f(x)\,\varphi_{j_0,k}(x), \qquad W_\psi(j,k) = \frac{1}{\sqrt{M}}\sum_{x} f(x)\,\psi_{j,k}(x), \quad j \ge j_0,$$

$$f(x) = \frac{1}{\sqrt{M}}\left[\sum_{k} W_\varphi(j_0,k)\,\varphi_{j_0,k}(x) + \sum_{j=j_0}^{J-1}\sum_{k} W_\psi(j,k)\,\psi_{j,k}(x)\right].$$

W_φ and W_ψ are the approximation and detail coefficients of the **discrete wavelet transform** (DWT), and the $\varphi_{j,k}(x)$, $\psi_{j,k}(x)$ here are the continuous functions sampled at x. For an orthonormal wavelet the total number of coefficients equals M: the DWT is a change of basis, not an expansion.

The fast wavelet transform

Plain version: you never need to evaluate φ or ψ. Two short filters and "keep every second sample" do all the work, level after level.

Substituting the refinement equations into the DWT gives Mallat's pyramid algorithm, now called the **fast wavelet transform** (FWT) [7]:

$$W_\psi(j,k) = \sum_n h_\psi(n-2k)\, W_\varphi(j+1,n), \qquad W_\varphi(j,k) = \sum_n h_\varphi(n-2k)\, W_\varphi(j+1,n).$$

Read it as: correlate the finer approximation $W_\varphi(j+1,\cdot)$ with h_ψ (or h_φ), then keep every second output (**downsampling by 2**). In signal-processing terms this is a two-channel **analysis filter bank**: a low-pass branch produces the next approximation and a high-pass branch produces the details. Feed the low-pass output into the same bank again, and so on. Each level halves the data, so the total cost is $O(M)$, faster than the FFT's $O(M \log M)$.

The inverse FWT is a **synthesis filter bank**: upsample each branch by 2 (insert zeros), filter with the synthesis filters, and add. For orthonormal wavelets the synthesis filters are the time-reversed analysis filters, and the bank achieves **perfect reconstruction**: the output equals the input exactly, despite the aliasing introduced by downsampling in each branch, because the aliasing terms of the two branches cancel. Vetterli and Kovačević's textbook develops this subband-coding view in depth [2].

```
import numpy as np
import pywt

x = np.array([4.0, 6.0, 10.0, 12.0, 8.0, 6.0, 5.0, 5.0])
w = pywt.Wavelet("haar")
h0, h1 = np.array(w.dec_lo), np.array(w.dec_hi)   # [.707 .707], [-.707 .707]

def analysis(x):
    lo = np.convolve(x, h0)[1::2]   # filter, then keep every 2nd sample
    hi = np.convolve(x, h1)[1::2]
    return lo, hi

a1, d1 = analysis(x)
a2, d2 = analysis(a1)
print("a2", a2, "d2", d2, "d1", d1.round(3))

ref = pywt.wavedec(x, "haar", level=2)            # [a2, d2, d1]
print(all(np.allclose(p, q) for p, q in zip(ref, [a2, d2, d1])))
print(np.allclose(pywt.waverec(ref, "haar"), x))
```

The two-line filter bank reproduces `pywt.wavedec` exactly. Note how the detail coefficient is 0 for the pair $(5, 5)$: no change, no detail.

Wavelet transforms in two dimensions

Plain version: run the 1-D filter bank along the rows, then along the columns. You get one small blurry image and three "edge maps", one per direction.

The 2-D DWT is separable. One level uses a 2-D scaling function and three 2-D wavelets, each a product of 1-D functions:

$$\varphi(x,y) = \varphi(x)\varphi(y), \quad \psi^H(x,y) = \psi(x)\varphi(y), \quad \psi^V(x,y) = \varphi(x)\psi(y), \quad \psi^D(x,y) = \psi(x)\psi(y).$$

Here x is the row index (vertical position) and y the column index, following the book's convention. ψ^H changes along the vertical direction and is smooth horizontally, so it responds to **horizontal** edges. ψ^V responds to **vertical** edges, and ψ^D to diagonal detail and corners. One level turns an $M \times N$ image into four $\frac{M}{2} \times \frac{N}{2}$ **subbands**:

Subband	Rows filter	Columns filter	Often called	PyWavelets name
Approximation	low	low	LL	`cA`
Horizontal detail	high	low	LH or HL (conventions differ)	`cH`
Vertical detail	low	high	HL or LH	`cV`
Diagonal detail	high	high	HH	`cD`

"Rows filter" here means the filter applied across rows, that is, along the vertical direction. Because the LH/HL labels are swapped between textbooks and libraries, this chapter uses the unambiguous names H, V and D. Repeating the decomposition on the approximation band gives the familiar nested layout in Figure 7.4.

```
import numpy as np
import pywt
from skimage import data, img_as_float

img = img_as_float(data.camera())
LL1, (H1, V1, D1) = pywt.dwt2(img, "haar")   # one level
LL2, (H2, V2, D2) = pywt.dwt2(LL1, "haar")   # second level on LL1
print(img.shape, LL1.shape, LL2.shape)       # (512, 512) (256, 256) (128, 128)

e = lambda a: float((a ** 2).sum())
total = e(img)
print(f"energy in LL2: {e(LL2) / total:.4f}")

coeffs = pywt.wavedec2(img, "haar", level=2)  # same thing in one call
print(np.allclose(pywt.waverec2(coeffs, "haar"), img))
```

After two levels, the 128 × 128 approximation, one sixteenth of the coefficients, holds about 99% of the energy of the camera image. The other 15/16 of the coefficients are mostly near zero, which is exactly the sparsity that compression and denoising exploit.

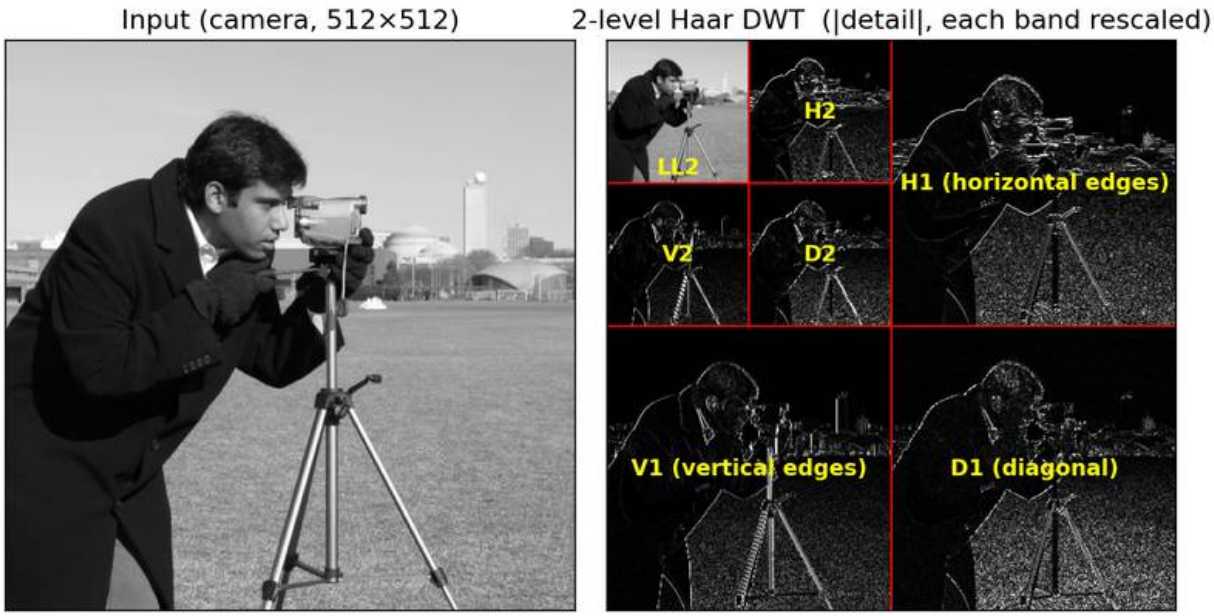


Figure 7.4 — Two-level Haar DWT of the camera image. The detail bands show absolute values, each rescaled to its own 99.5th percentile so that the small coefficients are visible; on a common scale they would look almost black. The tripod's vertical legs light up in V1; horizontal outlines of the buildings show in H1.

Wavelet packets

The standard DWT only splits the low-pass band again. A **wavelet packet** decomposition splits *every* band, detail bands included, producing a full binary tree (a quadtree in 2-D). After L levels a 2-D packet tree has 4^L leaves. Any choice of nodes that covers the frequency axis without overlap is a valid orthonormal basis, so the tree is a *library* of bases. Coifman and Wickerhauser's **best-basis** algorithm searches this library efficiently, choosing the basis that minimizes an additive cost such as the entropy of the normalized coefficients [11]. Packets help for textures, which carry a lot of energy at mid and high frequencies that the plain DWT leaves in a single wide band.

```
import pywt
from skimage import data, img_as_float

img = img_as_float(data.camera())
wp = pywt.WaveletPacket2D(img, "haar", maxlevel=2)
print([n.path for n in wp.get_level(2)][:6], len(wp.get_level(2)))  # 16 subbands
```

Node paths spell out the branch taken at each level (`a` approximation, `h`, `v`, `d` details), so `"hd"` is

the diagonal detail of the horizontal-detail band.

Applications

Denoising by thresholding

Plain version: in the wavelet domain, a clean image is a few large numbers, while white noise is many small numbers spread everywhere. Set the small ones to zero and transform back.

An orthonormal transform maps white Gaussian noise of standard deviation σ to white Gaussian noise of the same σ in every subband, while the image's energy concentrates in few coefficients. Thresholding rules for a detail coefficient w and threshold T:

$$\eta_{\text{hard}}(w) = \begin{cases} w, & |w| > T \\ 0, & |w| \le T \end{cases} \qquad \eta_{\text{soft}}(w) = \operatorname{sign}(w)\ \max(|w| - T,\, 0).$$

Hard thresholding keeps or kills. Soft thresholding also shrinks the survivors toward zero by T, which gives a continuous rule with fewer artifacts but some loss of contrast. Two classic choices of T:

- **Universal threshold** (VisuShrink), $T = \sigma\sqrt{2\ln n}$ for n samples, from Donoho and Johnstone [12]. Donoho showed that soft thresholding with this T gives, with high probability, an estimate that is at least as smooth as the true signal [13]. The noise level is usually estimated robustly from the finest diagonal band as $\hat{\sigma} = \text{median}(|w|)/0.6745$ [12].
- **BayesShrink**, from Chang, Yu and Vetterli, sets a separate threshold for each subband, $T = \hat{\sigma}^2/\hat{\sigma}_X$, where $\hat{\sigma}_X$ is the estimated standard deviation of the clean coefficients in that band [14]. It adapts to how much signal each band contains and usually beats the universal threshold.

```
import numpy as np
import pywt
from skimage import data, img_as_float
from skimage.metrics import peak_signal_noise_ratio as psnr

rng = np.random.default_rng(7)
clean = img_as_float(data.camera())
noisy = clean + rng.normal(0, 0.1, clean.shape)

def wavelet_denoise(y, wavelet="db4", level=3, mode="soft"):
    c = pywt.wavedec2(y, wavelet, mode="periodization", level=level)
    sigma = np.median(np.abs(c[-1][2])) / 0.6745       # noise level from finest HH
    T = sigma * np.sqrt(2 * np.log(y.size))            # universal threshold
    c = [c[0]] + [tuple(pywt.threshold(d, T, mode) for d in lvl) for lvl in c[1:]]
    return pywt.waverec2(c, wavelet, mode="periodization")

for mode in ["hard", "soft"]:
    out = np.clip(wavelet_denoise(noisy, mode=mode), 0, 1)
    print(mode, round(psnr(clean, out, data_range=1), 1), "dB")
print("noisy", round(psnr(clean, np.clip(noisy, 0, 1), data_range=1), 1), "dB")
```

On this example the noisy image scores 20.4 dB PSNR, hard thresholding 25.7 dB and soft thresholding 24.7 dB. The universal threshold is conservative (it is designed to remove essentially all the noise), so with soft shrinkage on top it over-smooths; the cartoon-like look in Figure 7.5 is the price. Subband-adaptive thresholds such as BayesShrink, or an undecimated (shift-invariant) transform, reduce both the blur and the blocky artifacts. scikit-image wraps these variants in `skimage.restoration.denoise_wavelet`.

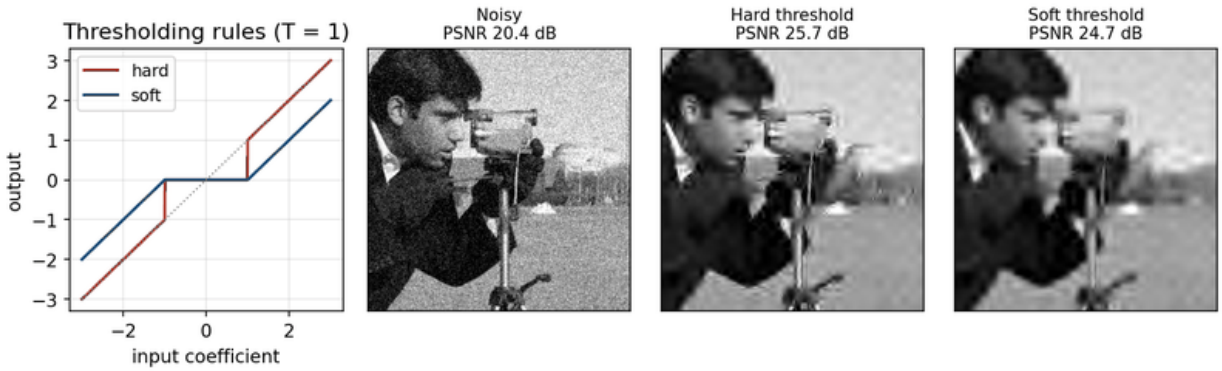
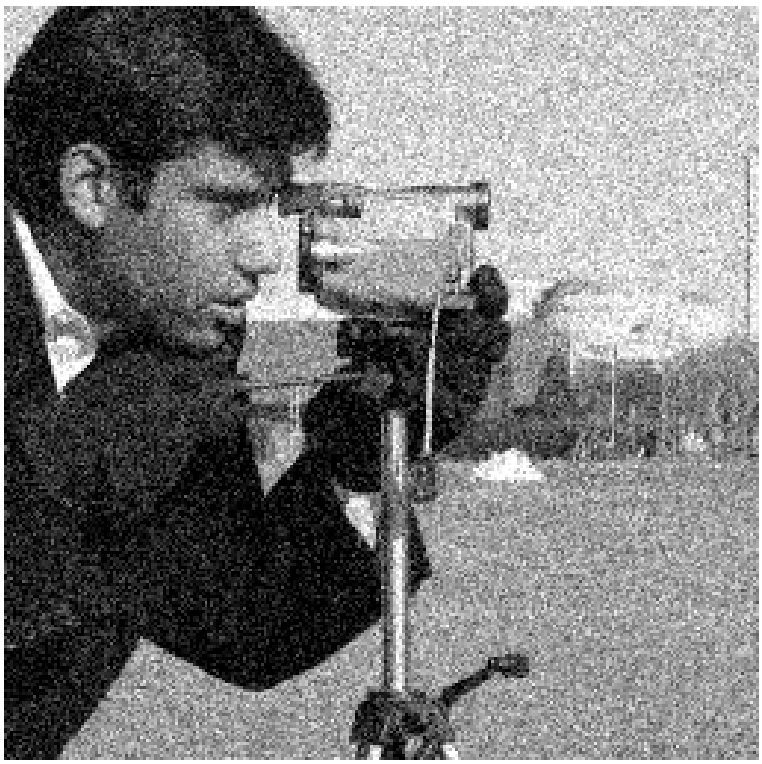


Figure 7.5 — Wavelet denoising of the camera image (Gaussian noise, $\sigma = 0.1$ on a $[0, 1]$ scale) with db4, three levels and the universal threshold. Hard thresholding keeps sharper edges but leaves some isolated blips; soft thresholding is smoother but blurrier.

Drag the slider to compare the noisy input with the soft-thresholded result:



Noisy

Soft threshold

Noisy and wavelet-denoised crops of the camera image

Edge detection

Because the detail bands are high-pass responses in three directions, wavelets give a quick edge detector: set the approximation band to zero and invert the transform. What remains is the image minus its coarse version, which is concentrated at edges. Keeping only some detail bands selects edge orientation, as in Figure 7.6. Coarser levels give thicker but more noise-robust edges, a multiscale behavior similar to choosing the σ of a Gaussian-derivative edge detector.

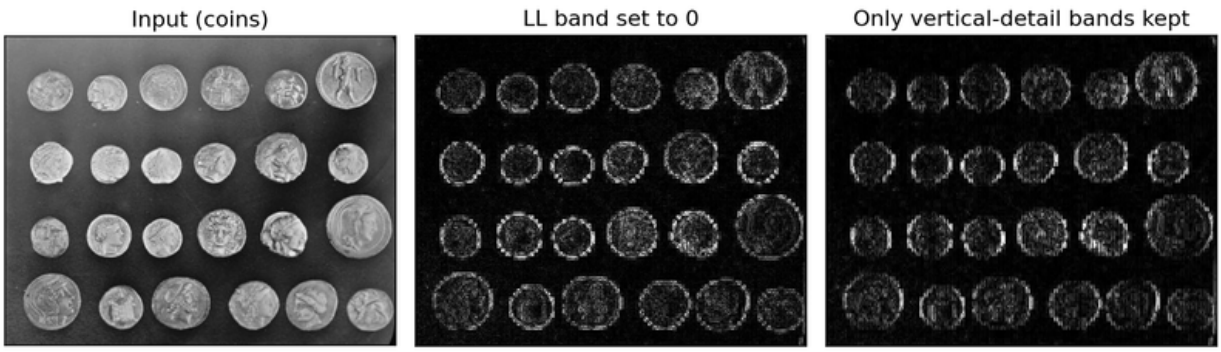


Figure 7.6 — Edges from a two-level Haar DWT of the coins image. Middle: approximation band set to zero. Right: only the vertical-detail bands kept, which emphasizes the left and right rims of each coin.

Compression preview

Compression rests on energy compaction: transform, keep the few big coefficients, and code them efficiently. Figure 7.7 keeps only the largest 5% of coefficients of the whole 512 × 512 image for four transforms. The two wavelets win by more than 2 dB over the DCT and more than 4 dB over the WHT, and their errors look different: the Fourier-type bases ring around edges and lose texture everywhere, while wavelets keep edges crisp and lose detail mainly in smooth or textured regions.

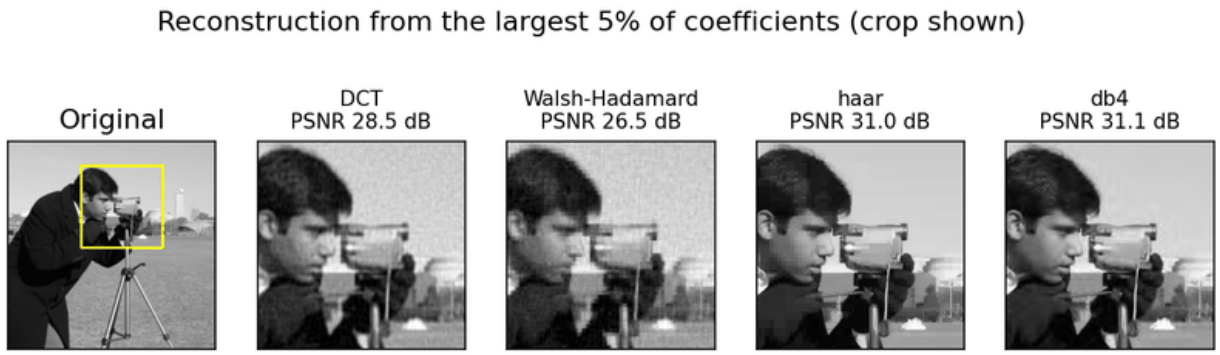


Figure 7.7 — Reconstructions from the largest 5% of coefficients (whole-image transforms, crop shown). PSNR: DCT 28.5 dB, Walsh–Hadamard 26.5 dB, Haar 31.0 dB, db4 31.1 dB.

Real codecs are more careful than "keep the top 5%". JPEG uses a block DCT with quantization and entropy coding [4], and JPEG 2000 uses a multilevel 2-D DWT with biorthogonal filters, followed by bit-plane coding of each subband [15]. IP 08 covers these pipelines in detail, and its transform-coding lab lets you quantize the DCT and Haar coefficients of a photo and see where the error goes.

Modern view

Wavelets arrived in image processing in the late 1980s as a unifying theory, peaked in the 1990s and 2000s as the backbone of denoising and JPEG 2000, and are now mostly an *ingredient*: of sparse models, of signal-processing-inspired network layers, and of efficient generative models. Here are the surveys and landmark papers worth reading, and what each contributes.

Foundations: Mallat (1989) and Daubechies (1988). Mallat's paper [7] introduced multiresolution analysis for images, the link between orthonormal wavelets and two-channel filter banks, and the pyramid algorithm that makes the transform $O(N)$. It also introduced the 2-D separable decomposition into horizontal, vertical and diagonal bands that every library still uses. Daubechies [8] provided the missing piece: orthonormal wavelets with compact support and a chosen amount of smoothness, which made the filters short and practical. Together they explain why `pywt.wavedec2(img, "db4")` exists at all.

Textbook-length survey: Vetterli and Kovačević (1995). *Wavelets and Subband Coding* [2] builds the theory from the signal-processing side: filter banks, perfect reconstruction, time–frequency tilings, and coding. The authors bought back the copyright and distribute the book freely, which makes it the best open reference for the filter-bank view in this chapter.

What makes a wavelet good: Unser and Blu (2003). "Wavelet theory demystified" [9] rebuilds the classical theory in a self-contained way by writing any scaling function as a B-spline convolved with a residual "irregular" part. Its main takeaway is that the B-spline factor alone is responsible for five properties practitioners care about: order of approximation, reproduction of polynomials, vanishing moments, the multiscale differentiation property, and smoothness. If you want to know why `db4` behaves better than Haar on smooth images, this is the paper that explains it.

From bases to dictionaries: Rubinstein, Bruckstein and Elad (2010). "Dictionaries for sparse representation modeling" [16] surveys the move from analytic transforms (Fourier, DCT, wavelets and their directional successors) to dictionaries *learned* from data. Its takeaway is the bridge to modern methods: the wavelet idea of "a few coefficients explain the image" survived, but the basis became something you can train. This line of work leads directly into learned priors for restoration.

The standard in practice: Skodras, Christopoulos and Ebrahimi (2001). Their overview of JPEG 2000 [15] shows how the DWT, biorthogonal filters, subband quantization and embedded coding fit together in a real codec, and what features (resolution scalability, region-of-interest coding, lossless mode) multiresolution makes possible.

Classical denoising theory: Donoho and Johnstone; Chang, Yu and Vetterli. The shrinkage papers [12], [13], [14] showed that a simple nonlinearity in the wavelet domain is close to optimal over broad classes of signals, and that adapting the threshold per subband helps in practice. Every "denoise by thresholding a transform" method, including many hand-crafted priors used before deep learning, descends from these results.

What deep learning changed, and what it did not

Scattering networks. Bruna and Mallat's **wavelet scattering transform** [17] cascades wavelet convolutions with a modulus nonlinearity and averaging. It is a convolutional network whose filters are fixed wavelets rather than learned, and the authors prove that it is stable to small deformations and locally translation invariant. The first layer behaves like SIFT-type descriptors, and higher layers add information that can tell apart textures with the same Fourier power spectrum. It gave state-of-the-art results on handwritten digit and texture classification at the time and became a reference point for explaining *why* CNNs work: a wavelet frame (redundant, as in the Preliminaries) plus a simple nonlinearity already yields invariance.

Wavelets as pooling and downsampling layers. Strided convolution and max-pooling downsample without proper low-pass filtering, so they alias. Li et al. replaced these operations with DWT layers in standard classifiers (WaveCNet) and kept only the low-frequency band, which improved robustness to noise [19]. In restoration, Liu et al.'s **MWCNN** replaces the pooling and unpooling of a U-Net with the 2-D DWT and its inverse [18]. Because the DWT is invertible, no information is lost when the network shrinks the feature maps, and the network gets a large receptive field cheaply. The same idea, an invertible and alias-aware down-sampler, appears in many later restoration and generation models.

Large receptive fields. Finder et al.'s **WTConv** layer (ECCV 2024) applies small convolutions to the subbands of a multilevel DWT, so the effective receptive field grows exponentially with the number of levels while parameters grow only logarithmically with kernel size [20]. It works as a drop-in layer in architectures such as ConvNeXt and MobileNetV2, and the authors report better robustness to image corruptions and a stronger response to shapes over textures.

What did not change. The DCT still sits inside JPEG, and the DCT and its integer approximations remain core tools in conventional image and video codecs. Hand-tuned wavelet thresholding has been overtaken in denoising quality by trained networks, but it is still the fastest strong baseline, needs no training data, and has guarantees. Most importantly, the vocabulary of this chapter (bases, frames, filter banks, multiresolution, aliasing, sparsity) is the vocabulary used to analyze what convolutional networks do.

Key takeaways

- A linear transform is a change of basis. Each coefficient is an inner product, that is, a correlation of the image with one basis function.
- Orthonormal transforms are matrix products, $\mathbf{T} = \mathbf{A}\mathbf{F}\mathbf{A}^T$ for separable 2-D cases, with the transpose as the inverse and energy preserved.
- The DCT compacts the energy of smooth images better than the DFT or WHT because its implied even extension has no border jump; this is why JPEG uses it.
- WHT and slant transforms trade some compaction for simplicity (only ± 1) or for an explicit ramp vector; Haar adds localization.
- The time–frequency plane shows the trade-off: Fourier bases are sharp in frequency, samples sharp in time, wavelets adapt the tile shape to the frequency.

- Multiresolution analysis turns two short filters into a fast $O(N)$ wavelet transform. In 2-D it yields an approximation band and horizontal, vertical and diagonal detail bands at each level.
- Sparsity in the wavelet domain powers denoising (hard/soft thresholding), quick edge maps and compression (JPEG 2000).
- Modern networks reuse wavelets as fixed feature extractors (scattering), as invertible downsamplers (MWCNN, WaveCNet) and as cheap large-receptive-field layers (WTConv).

Exercises

1. **A frame by hand.** Take the three unit vectors in the plane at angles 90°, 210° and 330°. Compute $\sum_k \langle \mathbf{f}, \mathbf{s}_k \rangle^2$ for $\mathbf{f} = (1, 0)$ and for $\mathbf{f} = (0, 1)$. What are the frame bounds, and how do you reconstruct $\mathbf{f}$ from its three coefficients?

   Hint

   Both sums equal 3/2; in fact, for every unit $\mathbf{f}$ the sum is 3/2, so the frame is tight with $A = B = 3/2$. Reconstruct with $\mathbf{f} = \frac{2}{3} \sum_k \langle \mathbf{f}, \mathbf{s}_k \rangle \mathbf{s}_k$.

2. **Separable cost.** For a 256 × 256 image, count the multiplications for (a) a general 2-D linear transform written as one 65536 × 65536 matrix, (b) the separable form $\mathbf{AFA}^T$ with dense $\mathbf{A}$, and (c) a three-level Haar FWT.

   Hint

   (a) $N^4 \approx 4.3 \times 10^9$. (b) two products of N^3 each, $2N^3 \approx 3.4 \times 10^7$. (c) Each level filters every row and column of the current approximation with 2-tap filters at half rate; the total is a small constant times N^2, a few hundred thousand.

3. **Why mirror?** Take $f = [1, 2, 3, 4, 5, 6, 7, 8]$ (a ramp). Compute its DFT and its DCT with SciPy and count how many coefficients you need to capture 99% of the energy in each. Explain the difference using the implied periodic and even extensions.

   Hint

   The periodic extension jumps from 8 back to 1, which needs many DFT harmonics. The even extension $\ldots 2, 1, 1, 2, \ldots, 8, 8, 7, \ldots$ is continuous, so the DCT needs only two or three coefficients.

4. **Haar by hand.** Compute a two-level Haar DWT of $[2, 2, 6, 6, 3, 5, 9, 9]$ using averages and differences scaled by $1/\sqrt{2}$. Which detail coefficients are zero, and why? Check your answer with `pywt.wavedec`.

   Hint

   Level 1 pairs: (2,2), (6,6), (3,5), (9,9). Only the pair (3,5) has a non-zero difference. At level 2, compare the scaled sums of neighboring pairs. Watch PyWavelets' sign convention: with its Haar filters, each detail coefficient comes out as $(\text{first} - \text{second})/\sqrt{2}$, so the pair $(3, 5)$ gives $-\sqrt{2}$.

5. **Thresholds in practice.** Modify the denoising code to use (a) a threshold equal to $3\hat{\sigma}$, and (b) a per-subband BayesShrink threshold $\hat{\sigma}^2/\hat{\sigma}_X$ with $\hat{\sigma}_X = \sqrt{\max(\overline{w^2} - \hat{\sigma}^2, 0)}$ computed in each band. Compare PSNR with the universal threshold for $\sigma = 0.05$ and $\sigma = 0.2$.

   Hint

   The universal threshold for a 512×512 image is about $5\hat{\sigma}$, so (a) keeps more coefficients. Expect (b) to be the best of the three for soft thresholding. If $\hat{\sigma}_X = 0$, set the band to zero.

6. **Shift sensitivity.** Shift the camera image by one pixel horizontally, take a one-level Haar DWT of both versions, and compare the energy in the V band. Why does a one-pixel shift change the coefficients so much, and how would an undecimated transform (`pywt.swt2`) behave?

   Hint

   Downsampling by 2 makes the DWT shift-variant: an edge that falls inside a Haar pair produces a detail coefficient, while the same edge falling between pairs produces none. The stationary (undecimated) transform skips downsampling, so it is a redundant frame and shifts its coefficients along with the image.

References

1. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018, Ch. 7. publisher page
2. M. Vetterli and J. Kovačević, *Wavelets and Subband Coding*, Prentice Hall, 1995 (free edition from the authors). book site
3. N. Ahmed, T. Natarajan and K. R. Rao, "Discrete Cosine Transform," *IEEE Transactions on Computers*, 1974. doi
4. G. K. Wallace, "The JPEG still picture compression standard," *IEEE Transactions on Consumer Electronics*, 1992. doi
5. W. K. Pratt, J. Kane and H. C. Andrews, "Hadamard transform image coding," *Proceedings of the IEEE*, 1969. doi
6. W. K. Pratt, W.-H. Chen and L. R. Welch, "Slant transform image coding," *IEEE Transactions on Communications*, 1974. doi
7. S. G. Mallat, "A theory for multiresolution signal decomposition: the wavelet representation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 1989. doi
8. I. Daubechies, "Orthonormal bases of compactly supported wavelets," *Communications on Pure and Applied Mathematics*, 1988. doi
9. M. Unser and T. Blu, "Wavelet theory demystified," *IEEE Transactions on Signal Processing*, 2003. doi
10. G. R. Lee, R. Gommers, F. Waselewski, K. Wohlfahrt and A. O'Leary, "PyWavelets: A Python package for wavelet analysis," *Journal of Open Source Software*, 2019. doi
11. R. R. Coifman and M. V. Wickerhauser, "Entropy-based algorithms for best basis selection," *IEEE Transactions on Information Theory*, 1992. doi

12. D. L. Donoho and I. M. Johnstone, “Ideal spatial adaptation by wavelet shrinkage,” *Biometrika*, 1994. doi
13. D. L. Donoho, “De-noising by soft-thresholding,” *IEEE Transactions on Information Theory*, 1995. doi
14. S. G. Chang, B. Yu and M. Vetterli, “Adaptive wavelet thresholding for image denoising and compression,” *IEEE Transactions on Image Processing*, 2000. doi
15. A. Skodras, C. Christopoulos and T. Ebrahimi, “The JPEG 2000 still image compression standard,” *IEEE Signal Processing Magazine*, 2001. doi
16. R. Rubinstein, A. M. Bruckstein and M. Elad, “Dictionaries for sparse representation modeling,” *Proceedings of the IEEE*, 2010. doi
17. J. Bruna and S. Mallat, “Invariant scattering convolution networks,” arXiv:1203.1513, 2012. arXiv
18. P. Liu, H. Zhang, K. Zhang, L. Lin and W. Zuo, “Multi-level wavelet-CNN for image restoration,” CVPR Workshops (NTIRE), 2018. arXiv
19. Q. Li, L. Shen, S. Guo and Z. Lai, “Wavelet integrated CNNs for noise-robust image classification,” CVPR, 2020. arXiv
20. S. E. Finder, R. Amoyal, E. Treister and O. Freifeld, “Wavelet convolutions for large receptive fields,” ECCV, 2024. arXiv

IP 08 • Image Compression and Watermarking

How images are squeezed into fewer bits, from entropy and Huffman codes to JPEG, JPEG 2000 and learned codecs, and how hidden marks are written into them.

Online edition: https://blog.shyandram.dev/en/learn/image-processing/dip-08-compression-watermarking/

What you'll learn

- The three kinds of redundancy that compression removes, and how entropy puts a floor under lossless coding
- How the classic lossless coders work: Huffman, Golomb, arithmetic, LZW, run-length, symbol-based and bit-plane coding
- How JPEG turns an image into 8×8 DCT blocks, quantizes them, and why it shows blocks at low quality
- How predictive coding (DPCM, motion compensation) and wavelet coding (JPEG 2000) exploit neighboring pixels
- How to hide a watermark in the least significant bits or in DCT coefficients, and why one survives JPEG and the other does not
- Where learned (neural) codecs and deep-learning watermarks fit into this picture today

The big picture

A raw 512×512 color photo is 786,432 bytes. The same photo as a JPEG can be a few tens of kilobytes and still look fine. Compression is the art of finding out which bits are predictable or invisible, and then not storing them.

> **Explain like I'm five**
>
> Imagine telling a friend over the phone what a picture of the sky looks like. You would not say "blue, blue, blue, blue..." a million times. You would say "the top is all blue, and there is one white cloud on the left." That is compression: say the common things briefly, and only spend words on the surprises. Watermarking is like writing your name on the drawing in invisible ink, so that later you can prove it is yours.

Nearly every image you see has been compressed, and many vision datasets are stored as JPEG, so models learn on images that already carry compression artifacts. Watermarking matters for ownership and, increasingly, for marking images made by generative models. Following the textbook [1], we cover fundamentals, then a toolbox of coding methods, then watermarking.

Fundamentals

In plain words: an image file usually contains more bits than the information it carries. Compression removes the extra bits; a good measure of "information" tells us how far we can go.

Compression ratio and redundancy

If an image takes b bits uncompressed and b' bits compressed, the **compression ratio** and **relative data redundancy** are

$$C = \frac{b}{b'}, \qquad R = 1 - \frac{1}{C}.$$

Here C is a pure number (10 means "ten to one") and R is the fraction of the original data that was redundant. With $C = 10$, $R = 0.9$: 90% of the bits carried no new information.

Images contain three kinds of redundancy:

- **Coding redundancy.** A fixed 8-bit code spends 8 bits on every gray level, common or rare. Variable-length codes waste fewer bits.
- **Spatial and temporal redundancy.** Neighboring pixels, and consecutive video frames, are strongly correlated, so coding each pixel independently repeats information.
- **Irrelevant information.** Detail a viewer does not notice. Removing it is irreversible (**lossy** compression); removing only the first two kinds is **lossless**.

Measuring information: entropy

A symbol that occurs with probability $P(a)$ carries $I(a) = -\log_2 P(a)$ bits of information: rare events are surprising and carry more bits. Averaging over the source alphabet gives the **entropy**

$$H = -\sum_{j=1}^{J} P(a_j)\,\log_2 P(a_j) \quad \text{bits/symbol},$$

where $a_1, \ldots, a_J$ are the possible symbols (for an 8-bit image, the 256 gray levels) and $P(a_j)$ is their probability. The noiseless source coding theorem says that no lossless code can use fewer than H bits per symbol on average when symbols are coded one at a time and independently; good codes come close to H.

For a real image we only have estimates. The **first-order estimate** uses the normalized histogram $p_r(r_k) = n_k/MN$, where n_k counts pixels with gray level r_k in an $M \times N$ image. Figure 8.1 shows why the estimate depends on what you call a "symbol". The camera image has a first-order entropy of 7.23 bits/pixel, barely below 8. If we instead code the difference between each pixel and its left neighbor, the entropy drops to 4.70 bits. The differences are not more clever; they simply remove the spatial redundancy that the histogram cannot see. This is the idea behind predictive coding later in the chapter.

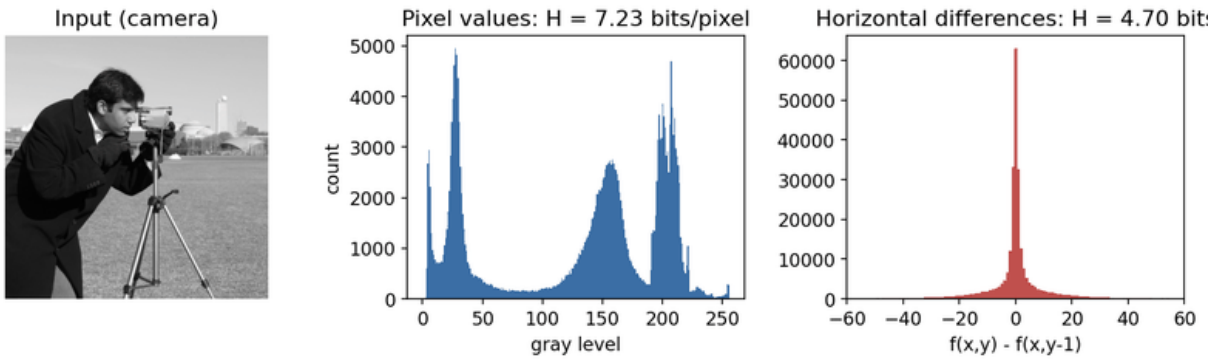


Figure 8.1 — The same image, two symbol choices. Pixel values are spread out (7.23 bits); differences between neighbors pile up near zero (4.70 bits).

```
import numpy as np
import Lab from '../../../../components/Lab.astro';
from skimage import data

def entropy(x):
    _, counts = np.unique(x, return_counts=True)
    p = counts / counts.sum()
    return -(p * np.log2(p)).sum()

img = data.camera()
print(f"H(pixels) = {entropy(img):.2f} bits")                              # 7.23
print(f"H(diffs)  = {entropy(np.diff(img.astype(int), axis=1)):.2f} bits")  # 4.70
```

Fidelity criteria

When compression is lossy we need to measure the damage. Let $f(x,y)$ be the original and $\hat{f}(x,y)$ the decompressed $M \times N$ image. The **root-mean-square error** and the **peak signal-to-noise ratio** are

$$e_{\text{rms}} = \sqrt{\frac{1}{MN}\sum_{x=0}^{M-1}\sum_{y=0}^{N-1}\left[\hat{f}(x,y) - f(x,y)\right]^2}, \qquad \text{PSNR} = 20\log_{10}\frac{L_{\max}}{e_{\text{rms}}}\ \text{dB},$$

where $L_{\max}$ is the largest possible pixel value (255 for 8-bit images). Higher PSNR means smaller error; as a rough guide, differences near 40 dB are hard to see and those below 25 dB are obvious, though this depends on content.

These **objective** criteria are easy to compute but do not always agree with people. **Subjective** criteria ask observers to rate images on a scale (from "excellent" to "unusable") or to compare pairs.

Image compression models

Almost every codec in this chapter fits one block diagram. The **encoder** has three stages:

1. a **mapper** that transforms the image into a form with less redundancy (differences, transform coefficients, run lengths);
2. a **quantizer** that discards precision according to a fidelity target (this is the only lossy stage, and lossless codecs omit it);
3. a **symbol coder** that assigns short codes to frequent symbols (Huffman, arithmetic, Golomb…).

The **decoder** runs a symbol decoder and an inverse mapper. There is no "inverse quantizer" that can bring back lost precision.

Formats, containers and standards

A **file format** fixes how bits are laid out, a **container** wraps coded images plus metadata (and can hold different codecs), and a **compression standard** defines the coding method. The common ones:

Name	Type	Core method	Lossless?
JPEG (ISO/IEC 10918-1, ITU-T T.81) [2]	standard + format	8×8 DCT, quantization, Huffman or arithmetic coding	lossy (baseline)
JPEG 2000 (ISO/IEC 15444) [3]	standard + format	discrete wavelet transform, bit-plane arithmetic coding	both
PNG [4]	format	per-row prediction filters + deflate (an LZ77 derivative)	lossless
JPEG-LS [10]	standard	context-based prediction + Golomb-type codes	lossless / near-lossless
JPEG XL (ISO/IEC 18181) [5]	standard + format	lossy and lossless modes; lossless transcoding of JPEG files	both
HEIF (ISO/IEC 23008-12) [7]	container	holds HEVC-coded images (`.heic`), among others	depends on codec
AVIF [6]	HEIF-based format	AV1 intra-frame coding stored in HEIF	both

Huffman coding

In plain words: give the most common symbols the shortest codewords, and build the code so no codeword is the beginning of another, so the bit stream can be read without separators.

Huffman's algorithm [8] builds an optimal **prefix code** for a known set of symbol probabilities, when symbols are coded one at a time. It works bottom-up:

1. List the symbols with their probabilities.

2. Merge the two least probable entries into a new node whose probability is their sum.
3. Repeat until one node (probability 1) remains.
4. Walk from the root to each leaf, writing 0 for one branch and 1 for the other. The path is the codeword.

A worked example

Take a source with six symbols: $P(a_1) = 0.40$, $P(a_2) = 0.20$, $P(a_3) = 0.15$, $P(a_4) = 0.10$, $P(a_5) = 0.10$, $P(a_6) = 0.05$. The merges are:

Step	Merge	New node
1	a_6 (0.05) + a_4 (0.10)	0.15
2	a_5 (0.10) + a_3 (0.15)	0.25
3	a_2 (0.20) + node 0.15	0.35
4	node 0.25 + node 0.35	0.60
5	a_1 (0.40) + node 0.60	1.00

The tree in Figure 8.2 gives \$a_1 = \$ `1`, \$a_2 = \$ `000`, \$a_3 = \$ `010`, \$a_4 = \$ `0010`, \$a_5 = \$ `011`, \$a_6 = \$ `0011`. The average codeword length is

$$L_{\text{avg}} = \sum_{j} P(a_j)\, l(a_j) = 0.4(1) + 0.2(3) + 0.15(3) + 0.1(4) + 0.1(3) + 0.05(4) = 2.35 \text{ bits/symbol},$$

where $l(a_j)$ is the length of the codeword for a_j. A fixed-length code needs 3 bits and the entropy is $H = 2.284$ bits, so Huffman is within 3% of the bound. Ties can be broken either way; other choices change the codewords but not the average length.

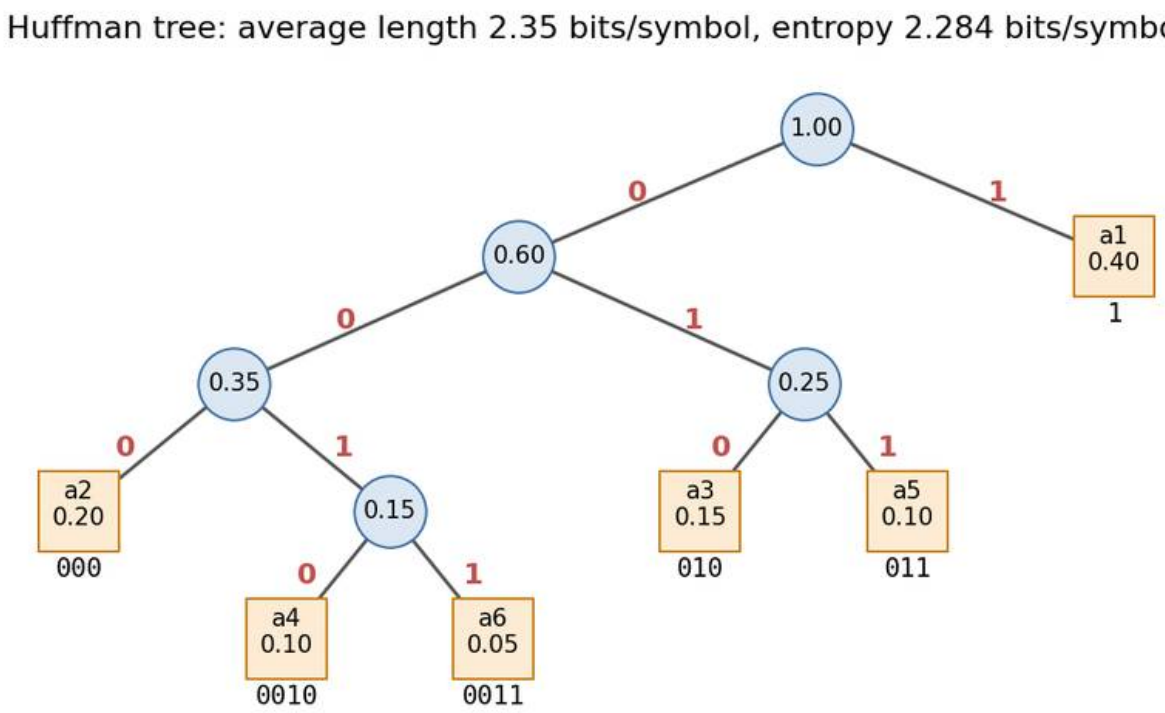


Figure 8.2 — Huffman tree for the six-symbol example. Edge labels give the bits; codewords are printed under each leaf.

Decoding is a walk down the tree: read bits until you hit a leaf, output its symbol, return to the root. Because no codeword is a prefix of another, the code is **instantaneous** and **uniquely decodable**. JPEG uses Huffman tables for its quantized coefficients [2].

```
import heapq, itertools

def huffman_codes(probs):
    tick = itertools.count()                       # tie-breaker for equal probabilities
    heap = [(p, next(tick), {s: ""}) for s, p in probs.items()]
    heapq.heapify(heap)
    while len(heap) > 1:
        p1, _, c1 = heapq.heappop(heap)            # least probable
        p2, _, c2 = heapq.heappop(heap)            # second least
        merged = {s: "1" + c for s, c in c1.items()}
        merged |= {s: "0" + c for s, c in c2.items()}
        heapq.heappush(heap, (p1 + p2, next(tick), merged))
    return heap[0][2]

P = {"a1": .40, "a2": .20, "a3": .15, "a4": .10, "a5": .10, "a6": .05}
print(huffman_codes(P))   # a1:'1', a2:'000', a3:'010', a4:'0010', a5:'011', a6:'0011'
```

The limitation: every symbol costs at least one whole bit. When one symbol has probability 0.95, its information is only 0.07 bits, but Huffman still spends 1 bit on it. Arithmetic coding, below,

removes that limit.

Golomb coding

In plain words: when small numbers are common and big numbers are rare, write a number as "how many full groups of m" (in tally marks) plus "what is left over" (in ordinary binary).

Golomb codes [9] are designed for nonnegative integers n with a geometric-like distribution, which is exactly what prediction residuals look like after mapping. For a parameter $m > 0$, the code $G_m(n)$ is the concatenation of

1. the **unary** code of the quotient $q = \lfloor n/m \rfloor$: q ones followed by a zero;
2. the **truncated binary** code of the remainder $r = n \bmod m$: with $k = \lceil \log_2 m \rceil$ and $c = 2^k - m$, values $r < c$ use $k - 1$ bits, and the rest are coded as $r + c$ in k bits.

n	0	1	2	3	4	5	6	7	8
$G_4(n)$	000	001	010	011	1000	1001	1010	1011	11000
$G_3(n)$	00	010	011	100	1010	1011	1100	11010	11011

When m is a power of two, the remainder is just the low k bits of n, so encoding needs only shifts and masks. Prediction residuals e are signed, so a codec first maps them to nonnegative integers, for example $M(e) = 2e$ for $e \geq 0$ and $M(e) = 2|e| - 1$ for $e < 0$, which interleaves $0, -1, 1, -2, 2, \ldots$. JPEG-LS [10] combines this mapping with Golomb codes whose power-of-two parameter is adapted to local context.

```
import numpy as np

def golomb(n, m):
    q, r = divmod(n, m)
    unary = "1" * q + "0"
    k = int(np.ceil(np.log2(m)))
    c = 2**k - m                        # how many remainders get k-1 bits
    if r < c:
        tail = format(r, f"0{k-1}b") if k > 1 else ""
    else:
        tail = format(r + c, f"0{k}b")
    return unary + tail

print([golomb(n, 3) for n in range(9)])
```

Arithmetic coding

In plain words: instead of giving each symbol its own codeword, describe the whole message as one very precise number between 0 and 1.

Arithmetic coding [11] starts from the interval $[0, 1)$. For each symbol, it shrinks the current interval to the sub-interval that the symbol owns, in proportion to its probability. After the message, any number inside the final interval identifies the message. If the current interval is $[\ell, h)$ and the next symbol s owns the cumulative range $[F_{\text{lo}}(s), F_{\text{hi}}(s))$, then

$$\ell' = \ell + (h - \ell)\, F_{\text{lo}}(s), \qquad h' = \ell + (h - \ell)\, F_{\text{hi}}(s).$$

The final width equals the product of the symbol probabilities, so the number of bits needed is about $-\log_2 \prod_i P(s_i) = \sum_i -\log_2 P(s_i)$, which is exactly the information content of the message. There is no "one bit per symbol" floor.

Worked example

Let $P(a) = 0.6$, $P(b) = 0.3$, $P(c) = 0.1$, so a owns $[0, 0.6)$, b owns $[0.6, 0.9)$ and c owns $[0.9, 1)$. Encode the message `abac`:

After	Interval
`a`	$[0, 0.6)$
`b`	$[0.36, 0.54)$
`a`	$[0.36, 0.468)$
`c`	$[0.4572, 0.468)$

The width is $0.6 \times 0.3 \times 0.6 \times 0.1 = 0.0108$, or $-\log_2 0.0108 \approx 6.5$ bits of information. The 7-bit binary fraction $0.0111011_2 = 0.4609375$ falls inside the interval, so 7 bits identify the message (plus a way to signal its end, such as a length or an end-of-message symbol).

Real coders use integer arithmetic with renormalization so that the interval never underflows, and they usually use **adaptive** probability models that are updated as symbols arrive, so no table needs to be sent. JPEG 2000's coefficient coder feeds an adaptive binary arithmetic coder [14], and the learned codecs in the Modern view section also end with arithmetic coding.

LZW coding

In plain words: build a dictionary of phrases as you go; whenever a phrase repeats, send its dictionary index instead of spelling it out.

The Lempel–Ziv–Welch (LZW) algorithm [12] starts with a dictionary holding all single symbols (codes 0–255 for 8-bit pixels). It reads the input, growing the current phrase w while w followed by the next symbol is still in the dictionary. When it is not, the coder outputs the code for w, adds the extended phrase as a new entry (256, 257, ...), and restarts from the new symbol. The decoder rebuilds the same dictionary from the codes it receives, so no dictionary is transmitted.

For a row that alternates `39 39 126 126` four times (16 pixels), the output is ten codes:

```
def lzw_encode(seq):
    table = {(v,): v for v in range(256)}
```

```
    w, out = (), []
    for x in seq:
        wx = w + (x,)
        if wx in table:
            w = wx
        else:
            out.append(table[w])
            table[wx] = len(table)
            w = (x,)
    if w:
        out.append(table[w])
    return out

row = [39, 39, 126, 126] * 4
print(lzw_encode(row))   # [39, 39, 126, 126, 256, 258, 260, 259, 257, 126]
```

Codes 256 onward stand for `39 39`, `39 126`, `126 126`, `126 39`, `39 39 126`, and so on. The longer the repetition, the longer the phrases and the better the ratio. LZW needs no prior knowledge of the probabilities, which made it popular in general-purpose file formats. PNG uses the related LZ77-style deflate method [4].

Run-length coding

In plain words: “0000 11 00000 1” becomes “four 0s, two 1s, five 0s, one 1”.

Run-length encoding (RLE) replaces each run of identical values with a pair (value, length). For binary images only the lengths are needed, if the first run’s color is agreed on. The run lengths themselves are then coded with a variable-length code, since short runs are more frequent than long ones.

```
import numpy as np

def rle(row):
    starts = np.r_[0, np.flatnonzero(np.diff(row)) + 1]
    lengths = np.diff(np.r_[starts, len(row)])
    return list(zip(row[starts].tolist(), lengths.tolist()))

print(rle(np.array([0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1])))  # [(0, 4), (1, 2), (0, 5), (1,
 ↪  1)]
```

RLE works well for documents, line drawings and bit planes, and poorly for noisy photographs where runs are short. Two-dimensional variants code the positions of color transitions in the current row relative to the transitions in the row above, which exploits vertical correlation too. JPEG uses a form of run-length coding for the many zero-valued coefficients produced by quantization [13].

Symbol-based coding

In plain words: a scanned page is mostly the same few letters again and again. Store each letter shape once, then just say "letter 7 goes here".

In **symbol-based** (token-based) coding, frequently occurring sub-images, called **symbols**, are stored once in a **symbol dictionary**. The image becomes a list of triplets (x_i, y_i, t_i): the position of each occurrence and the index t_i of its dictionary entry. A text page with thousands of characters but about a hundred distinct glyph shapes compresses very well, since each occurrence costs two coordinates and an index, which are then entropy-coded.

If an occurrence may match an entry that is only *similar*, the method becomes lossy. That improves the ratio, but a careless matcher can substitute a wrong character, such as two digits that look alike in a low-resolution scan. Safe systems keep the match threshold conservative, or also code the residual between occurrence and token to stay lossless.

Bit-plane coding

In plain words: split an 8-bit image into eight black-and-white pictures, one per bit, and compress each with a binary method.

A gray level $a = a_7 2^7 + a_6 2^6 + \cdots + a_0 2^0$ has eight bits $a_7, \ldots, a_0$. Collecting bit k of every pixel gives **bit plane** k (Figure 8.3). The high-order planes look like coarse versions of the picture and contain large uniform areas; the low-order planes look like noise. Each plane can be coded with run-length or other binary methods.

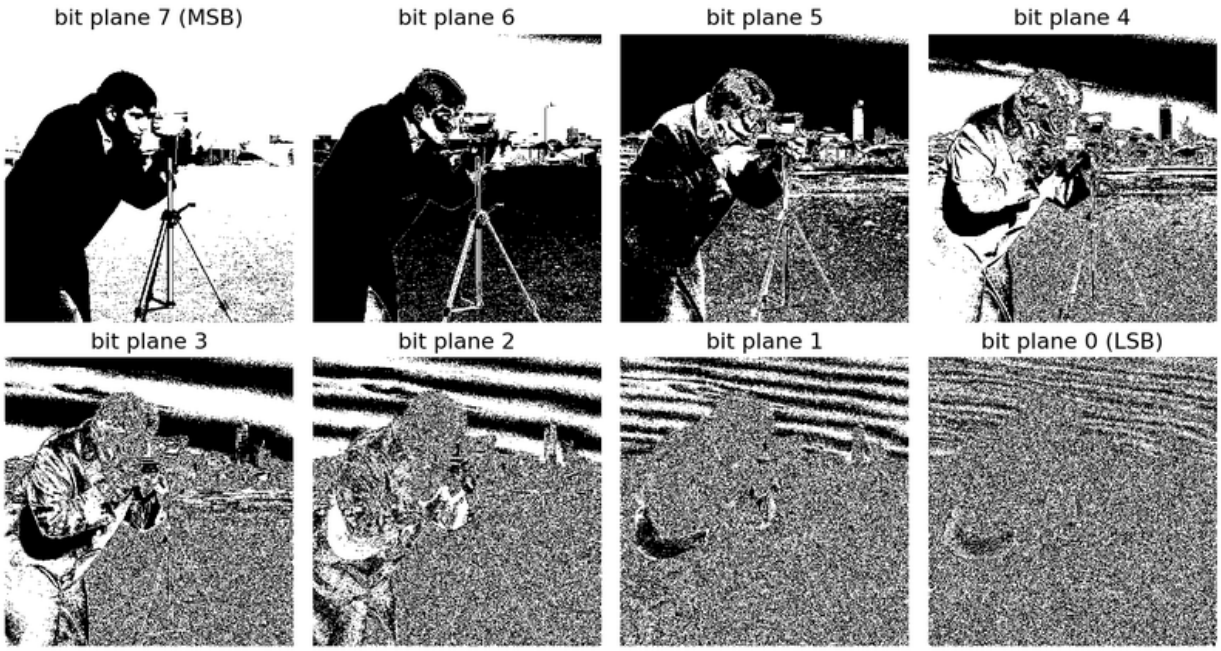


Figure 8.3 — Bit planes of an 8-bit image. Structure lives in the top planes; the bottom planes are close to random.

A weakness of the plain binary code is that a small change in gray level can flip many bits: 127 is `01111111` and 128 is `10000000`. The **Gray code** fixes this by making consecutive values differ in exactly one bit:

$$g_7 = a_7, \qquad g_i = a_i \oplus a_{i+1}, \quad 0 \le i \le 6,$$

where $\oplus$ is exclusive OR. With Gray-coded planes, smooth areas produce longer runs. On the camera image, bit plane 6 changes value between horizontal neighbors 6.4% of the time; its Gray-coded counterpart changes only 3.2% of the time, so its runs are about twice as long on average. In NumPy the conversion is one line: `gray = img ^ (img >> 1)`.

JPEG 2000 also codes its wavelet coefficients bit plane by bit plane, from the most significant down, which is what makes its bit stream truncatable at any rate [14].

Block transform coding

In plain words: cut the image into small tiles, describe each tile as a mix of standard patterns (smooth to busy), keep the important patterns, and throw away the fine ones the eye barely notices.

A **block transform coder** splits the image into non-overlapping $n \times n$ subimages, transforms each one, quantizes the coefficients, and entropy-codes the result. The transform does not compress by itself; it **packs** most of the energy into a few coefficients so that the rest can be quantized coarsely or dropped.

Transform selection

For an $n \times n$ block $g(x, y)$, the 2-D **discrete cosine transform** (DCT) is

$$T(u,v) = \alpha(u)\alpha(v) \sum_{x=0}^{n-1} \sum_{y=0}^{n-1} g(x,y) \cos \frac{(2x+1)u\pi}{2n} \cos \frac{(2y+1)v\pi}{2n},$$

with $\alpha(0) = \sqrt{1/n}$ and $\alpha(u) = \sqrt{2/n}$ for $u = 1, \dots, n-1$; u, v index the horizontal and vertical frequencies. The Karhunen–Loève transform (KLT, Chapter 7) packs energy best in the mean-square sense, but its basis depends on the data and must be computed and sent. The DCT is fixed, has fast algorithms, and for highly correlated images its basis is close to the KLT's. It also beats the DFT for blocks: the DFT treats a block as periodic, so the jump between its left and right edges creates spurious high frequencies, while the DCT's implied even symmetric extension has no such jump.

Subimage size

Larger blocks capture more correlation and pack energy better, but cost more computation, and an error spreads over a larger area. Gains level off quickly beyond about 8×8 or 16×16. JPEG chose a fixed 8×8; many newer codecs choose among several block sizes, using large blocks in smooth regions and small ones around detail.

Bit allocation

After the transform, we decide how many bits each coefficient gets.

- **Zonal coding** keeps a fixed zone of coefficients (usually the low frequencies, which have the largest variance across blocks) in every block.
- **Threshold coding** keeps whichever coefficients exceed a threshold in each block, so the kept set adapts to content.

The common practical form of threshold coding divides each coefficient by an entry of a **normalization (quantization) array** Z and rounds:

$$\hat{T}(u,v) = \text{round}\left[\frac{T(u,v)}{Z(u,v)}\right],$$

where $\hat{T}(u,v)$ is the integer that is actually coded. The decoder multiplies back, $\dot{T}(u,v) = \hat{T}(u,v)$ $Z(u,v)$, and inverts the DCT. Large entries of Z at high frequencies throw away fine detail; small entries at low frequencies keep the coarse structure.

JPEG baseline

The JPEG baseline coder for 8-bit grayscale [2][13] does exactly this:

1. Shift pixel values by -128 so they are centered on zero.
2. Compute the 8×8 DCT of each block.
3. Quantize with a Z table. The standard gives an example luminance table; a widely used convention scales it by a quality factor $q \in [1, 100]$, with scale $s = 5000/q$ for $q < 50$ and $s = 200 - 2q$ otherwise (in percent).
4. Code the DC coefficient $\hat{T}(0,0)$ as the difference from the previous block's DC.
5. Reorder the 63 AC coefficients in a **zigzag** from low to high frequency, so zeros cluster at the end, and code them as (run of zeros, nonzero value) pairs with Huffman tables, ending each block with an end-of-block symbol.

Figure 8.4 follows one dark, smooth block through steps 2–3 at $q = 50$. It has a large DC term (-603, because of the level shift) and a few low-frequency terms. After quantization only 6 of 64 integers are nonzero, yet the reconstruction is off by only a few gray levels.

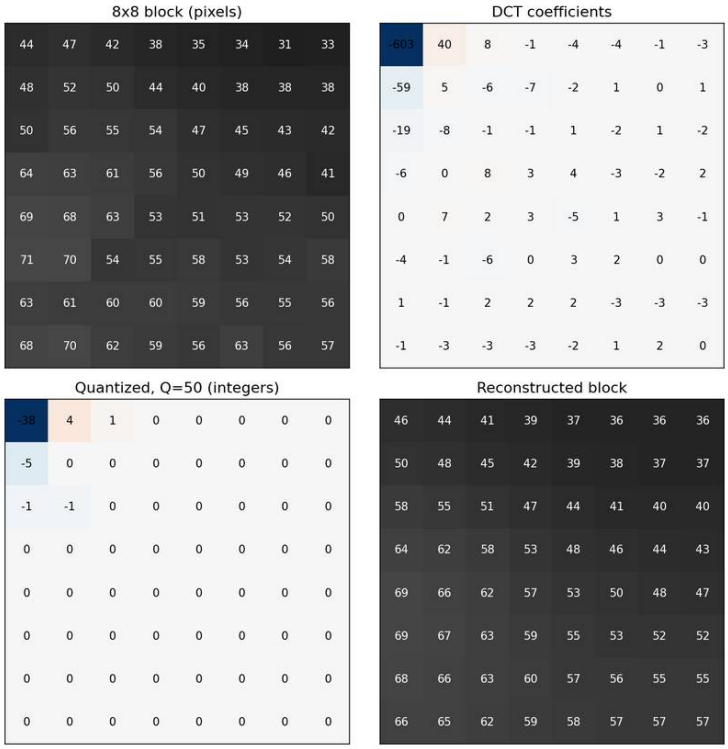


Figure 8.4 — One 8×8 block through the JPEG pipeline at quality 50. Quantization zeroes 58 of the 64 coefficients.

Figure 8.5 applies the same pipeline (without the entropy coder) to the whole image at four quality settings. PSNR drops from 40.3 dB at $q = 90$ to 26.3 dB at $q = 5$, while the fraction of nonzero coefficients falls from 31.3% to 2.4%. At low quality the familiar artifacts appear: **blocking** (visible 8×8 seams, because each block is coded independently) and **ringing** near edges (because the high frequencies that make a sharp edge were removed).

Figure 8.5 — JPEG-style DCT quantization at several quality settings. Blocking becomes obvious at $q = 5$.

Interactive demonstration (**online**): open the online edition of this tutorial to use this demonstration.Open demonstration

```
import numpy as np
from scipy.fft import dctn, idctn

Q50 = np.array([[16,11,10,16,24,40,51,61],[12,12,14,19,26,58,60,55],
                [14,13,16,24,40,57,69,56],[14,17,22,29,51,87,80,62],
                [18,22,37,56,68,109,103,77],[24,35,55,64,81,104,113,92],
                [49,64,78,87,103,121,120,101],[72,92,95,98,112,100,103,99]])

def dct_codec(img, Q):
    f = img.astype(float) - 128                       # level shift
    H, W = f.shape
    B = f.reshape(H//8, 8, W//8, 8).swapaxes(1, 2)   # (H/8, W/8, 8, 8) blocks
    q = np.round(dctn(B, axes=(2, 3), norm="ortho") / Q)
    rec = idctn(q * Q, axes=(2, 3), norm="ortho")
    rec = rec.swapaxes(1, 2).reshape(H, W) + 128
    return np.clip(rec.round(), 0, 255).astype(np.uint8), q

rec, q = dct_codec(img, Q50)     # PSNR 32.6 dB, 12.0% nonzero coefficients
```

Drag the slider below to compare a lossless PNG of a color test image (465 kB) with a real JPEG at quality 4 (7.6 kB, about 60 times smaller). Look at the flat background, the face and the color edges: blocks, ringing and color bleeding (the color channels are stored at lower resolution and quantized harder) are all visible.

Original (PNG)

JPEG q=4

Astronaut test image, lossless versus heavily quantized JPEG

Predictive coding

In plain words: guess each pixel from the ones you have already sent, and only send how wrong the guess was. Good guesses mean small numbers, and small numbers are cheap.

Lossless predictive coding

The encoder forms a prediction $\hat{f}(n)$ of pixel $f(n)$ from previously coded pixels and codes the **prediction error**

$$e(n) = f(n) - \hat{f}(n), \qquad \hat{f}(n) = \operatorname{round}\left[\sum_{i=1}^{m} \alpha_i f(n-i)\right],$$

where m is the predictor order and α_i are the prediction coefficients. The decoder computes the same prediction and adds $e(n)$ back, so the reconstruction is exact. Figure 8.1 already showed the payoff: the simplest predictor, “same as the left neighbor”, lowers the entropy from 7.23 to 4.70 bits/pixel.

PNG lets each row choose among five filters: none, left (Sub), above (Up), the average of left and above, and the Paeth predictor [4]. JPEG-LS [10] uses the **median edge detector**: with a the left, b the upper and c the upper-left neighbor,

$$\hat{f} = \begin{cases} \min(a,b) & c \geq \max(a,b) \\ \max(a,b) & c \leq \min(a,b) \\ a + b - c & \text{otherwise.} \end{cases}$$

The first two cases pick the neighbor on the correct side of a horizontal or vertical edge; the third assumes a smooth plane.

Motion compensation

Video adds a time axis. Instead of predicting from neighbors in the same frame, a **motion-compensated** predictor copies a block from a previously decoded frame, displaced by a **motion vector** (d_x, d_y) found by searching for the best match (for example, the smallest sum of absolute differences). The encoder sends the motion vector and the residual, usually transform-coded as in the previous section. Frames coded only from themselves (intra frames) provide entry points for random access; predicted frames are much smaller. HEVC, whose still-image form is stored in HEIC files [7], and AV1, used by AVIF [6], both combine this kind of inter-frame prediction with transform coding.

Lossy predictive coding (DPCM)

If we quantize the prediction error, $\dot{e}(n) = Q[e(n)]$, we get **differential pulse-code modulation** (DPCM). The key detail is that the encoder must predict from the *reconstructed* values $\dot{f}(n) = \dot{e}(n) + \hat{f}(n)$, the same values the decoder will have. If the encoder predicted from the originals instead, quantization errors would accumulate in the decoder (drift).

```
import numpy as np

def dpcm(row, step):
    """Lossy DPCM with previous-sample prediction and a uniform quantizer."""
    rec = np.empty(len(row)); prev = 128.0; idx = []
    for i, x in enumerate(row.astype(float)):
        e = x - prev                       # prediction error
        k = int(np.round(e / step))        # quantizer index: this is what is sent
        prev = prev + k * step             # reconstruction, identical in the decoder
        rec[i] = prev; idx.append(k)
    return np.array(idx), rec
```

On one row of the camera image with step 8, the indices have an entropy of 1.87 bits/sample and the reconstruction has a PSNR of 40.8 dB. With a very coarse quantizer, two typical distortions appear: **slope overload** (the reconstruction cannot climb fast enough at a steep edge) and **granular noise** (it jitters in flat areas).

Optimal predictors and quantizers

The **optimal linear predictor** chooses $a_1, \dots, a_m$ to minimize the mean-square prediction error $E\{e^2(n)\}$. Setting the derivatives to zero gives the normal equations

$$\mathbf{R}\,\boldsymbol{a} = \mathbf{r}, \qquad \boldsymbol{a} = \mathbf{R}^{-1}\mathbf{r},$$

where $\mathbf{R}$ is the $m \times m$ autocorrelation matrix with entries $E\{f(n-i)f(n-j)\}$ and $\mathbf{r}$ is the vector with entries $E\{f(n)f(n-i)\}$. (The analysis usually ignores the quantizer inside the loop, which is accurate when quantization is fine.)

The **optimal quantizer** for a given number of levels minimizes the mean-square quantization error for the error distribution. Two conditions characterize it: each decision boundary lies halfway between its two neighboring reconstruction levels, and each reconstruction level is the centroid (conditional mean) of the probability mass in its interval. Because prediction errors are peaked at zero, the optimal levels are dense near zero and sparse in the tails. In practice, a uniform quantizer followed by a variable-length code is nearly as good and much simpler.

Wavelet coding

In plain words: instead of chopping the image into tiles, split it into a blurry small version plus layers of detail at different scales. Most detail values are near zero, so they are cheap to store, and there are no tile seams.

A wavelet coder applies the discrete wavelet transform of Chapter 7, quantizes the coefficients, and entropy-codes them. Because the basis functions overlap and are localized at many scales, a wavelet coder has no block boundaries and degrades by blurring and mild ringing rather than by blocking. Figure 8.6 compares an 8×8 DCT and a five-level wavelet transform that each keep the same 2% of their coefficients (the largest in magnitude). The DCT result is a mosaic at 26.8 dB; the wavelet result is smoother and reaches 28.6 dB.

Figure 8.6 — Same coefficient budget (2%), different transforms. Block DCT produces seams; the wavelet transform does not.

```
import numpy as np, pywt

def wavelet_code(img, keep=0.02, wavelet="bior4.4", level=5):
    c = pywt.wavedec2(img.astype(float), wavelet, level=level, mode="periodization")
    arr, slices = pywt.coeffs_to_array(c)
    t = np.quantile(np.abs(arr), 1 - keep)            # keep the largest 2%
    arr[np.abs(arr) < t] = 0
    c = pywt.array_to_coeffs(arr, slices, output_format="wavedec2")
    return np.clip(pywt.waverec2(c, wavelet, mode="periodization"), 0, 255)
```

This toy counts coefficients, not bits. A real coder must also signal *which* coefficients survived, which is where clever significance coding earns its keep.

Wavelet selection

The wavelet determines how well energy is packed and what artifacts look like. Image coders favor **biorthogonal** wavelets because they can be symmetric (so edges are not shifted) and have short filters. JPEG 2000 specifies two: an irreversible 9/7-tap filter pair for lossy coding and a reversible integer 5/3 filter pair for lossless coding [14]. PyWavelets' `bior4.4` used above has the same 9- and 7-tap lengths.

Decomposition level

Each level splits the current approximation into four half-size subbands. More levels make the representation sparser, but gains shrink after about five levels for typical image sizes, and quantizing very coarse coefficients affects large areas.

Quantizer design

Wavelet coders usually use a uniform scalar quantizer with an enlarged **dead zone** around zero:

$$q_b = \operatorname{sign}(c_b)\left\lfloor \frac{|c_b|}{\Delta_b} \right\rfloor,$$

where c_b is a coefficient in subband b, Δ_b is that subband's step size and q_b the integer index. Small coefficients, most of which are noise, fall into the zero bin. Step sizes are chosen per subband, coarser for the high-frequency subbands where errors are less visible.

JPEG 2000

JPEG 2000 [3][14] builds a full system around these pieces. The image can be split into large tiles, color components are decorrelated with a component transform, and each tile is wavelet transformed and dead-zone quantized. Each subband is divided into small **code blocks** that are

coded independently bit plane by bit plane with an adaptive arithmetic coder. Finally, the coded pieces are arranged into **quality layers**, so a single file can be decoded at lower resolution or lower quality just by reading a prefix of the stream. It also supports lossless coding and regions of interest coded at higher quality.

Digital image watermarking

In plain words: a watermark is extra information hidden in an image, like a signature, an owner ID or a "this was generated by a model" flag, that travels with the picture.

Kinds of watermarks

- **Visible** watermarks are overlays (a semi-transparent logo); they degrade the image and can often be cropped or painted out. **Invisible** watermarks stay below the visibility threshold and are recovered by a detector.
- A **robust** watermark survives compression, resizing, filtering and cropping (for ownership and provenance). A **fragile** one breaks under any change (for authentication). **Semi-fragile** marks survive mild compression but not content edits.
- **Private** (non-blind) detection needs the original image; **blind** detection does not.

Every scheme trades off **imperceptibility** (how little the image changes), **robustness**, **capacity** (how many bits are hidden) and **security** (whether someone without the key can find, remove or forge the mark).

A fragile example: least significant bits

The simplest invisible watermark replaces the least significant bit (LSB) of each pixel with a bit of the mark: $f_w = 2\lfloor f/2 \rfloor + w$, with $w \in \{0, 1\}$. The change is at most one gray level, so the PSNR is about 51 dB.

```
def lsb_embed(cover, mark):          # cover: uint8 image, mark: 0/1 array of the same shape
    return (cover & 0xFE) | mark

def lsb_extract(img):
    return img & 1
```

Figure 8.7 embeds a binary "DIP" logo. Extraction is perfect from the PNG. After a single JPEG compression at quality 95, which looks identical to the eye, only 55% of the extracted bits match the mark, barely better than a coin flip. The LSB plane is the noise-like part of the image that every lossy coder discards first. That makes LSB marks useless for ownership but suitable as a fragile integrity check.

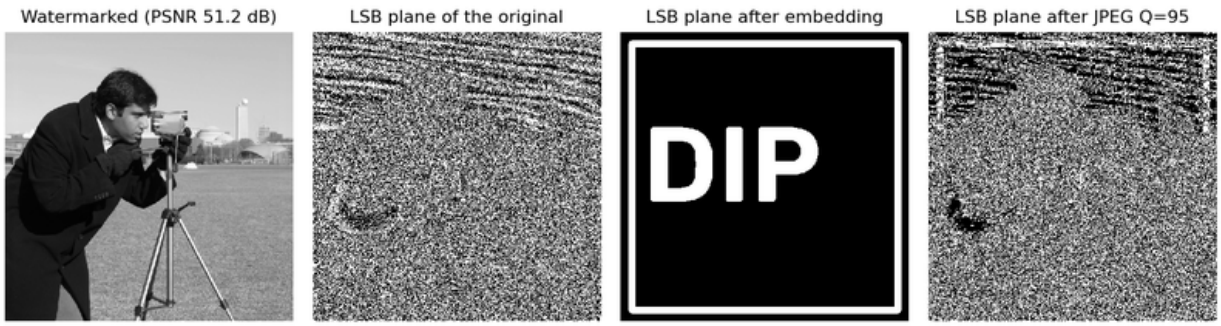


Figure 8.7 — An LSB watermark is invisible (51 dB) and perfectly recoverable, until a mild JPEG pass wipes it out.

A robust example: DCT-domain watermark

To survive JPEG, hide the mark where JPEG keeps information: in low- to mid-frequency DCT coefficients. The scheme below is a simple one of our own. In each 8×8 block it compares two mid-frequency coefficients, $C(2,3)$ and $C(3,2)$, which JPEG quantizes with similar step sizes. To embed bit 1, it makes $C(2,3)$ exceed $C(3,2)$ by at least a margin α; to embed bit 0, the reverse. The detector only checks which coefficient is larger, so it is blind.

```
import numpy as np
from scipy.fft import dctn, idctn

def blocks(img):
    H, W = img.shape
    return img.astype(float).reshape(H//8, 8, W//8, 8).swapaxes(1, 2).reshape(-1, 8, 8)

def unblocks(B, H, W):
    return B.reshape(H//8, W//8, 8, 8).swapaxes(1, 2).reshape(H, W)

def dct_embed(img, bits, alpha=12.0):
    C = dctn(blocks(img), axes=(1, 2), norm="ortho")
    for i, b in enumerate(bits):                # one bit per 8x8 block
        x, y = C[i, 2, 3], C[i, 3, 2]
        if (x - y if b else y - x) < alpha:     # enforce the order with margin alpha
            m = (x + y) / 2
            s = alpha / 2 if b else -alpha / 2
            C[i, 2, 3], C[i, 3, 2] = m + s, m - s
    out = unblocks(idctn(C, axes=(1, 2), norm="ortho"), *img.shape)
    return np.clip(out.round(), 0, 255).astype(np.uint8)

def dct_extract(img):
    C = dctn(blocks(img), axes=(1, 2), norm="ortho")
    return (C[:, 2, 3] > C[:, 3, 2]).astype(int)
```

On the 512×512 camera image this hides 4,096 bits (one per block). The margin α is the robustness dial:

Margin α	PSNR of marked image	Bit errors after JPEG q=90	q=75	q=50
12	43.2 dB	0.10%	0.56%	36.6%
30	38.4 dB	0%	0%	0.05%

A small margin is nearly invisible but breaks once the JPEG step size at those positions exceeds the margin. A larger margin survives quality 50 at the cost of about 5 dB PSNR. Real systems add error-correcting codes, spread each bit over many blocks with a secret key, and adapt the strength to local texture, since the eye tolerates larger changes in busy regions.

Modern view

The classical pipeline (transform, quantize, entropy code) still underlies every deployed image codec. What has changed since about 2016 is that each stage can be *learned*, and that the same idea of trained encoders and decoders has reshaped watermarking.

Surveys worth reading

- **Wallace (1992)** [13] remains the clearest overview of JPEG's design: the DCT, quantization tables, zigzag ordering, and the Huffman and arithmetic options. It is the right companion to the block transform section.
- **Skodras, Christopoulos and Ebrahimi (2001)** [14] is a tutorial review of JPEG 2000: tiling, the 9/7 and 5/3 wavelets, dead-zone quantization, code-block bit-plane coding and rate control. Its main takeaway is that JPEG 2000's value lies less in raw efficiency than in features: scalability, region-of-interest coding and lossless operation in one bit stream.
- **Hu, Yang, Ma and Liu (2021)** [16] survey end-to-end learned lossy image compression along three axes, network architecture, entropy model and rate control, trace the milestones of each, and benchmark representative methods on CPU and GPU. Their own contribution, a coarse-to-fine hyperprior, illustrates the survey's central theme: much of the progress lies in how well the entropy model predicts the latent representation.
- **Yang, Mandt and Theis (2023)** [17] is a monograph-length introduction to neural compression for a machine-learning audience. It reviews the information theory (entropy coding, rate-distortion theory) and the image-quality and perceptual metrics you need, then guides the reader through codecs built on normalizing flows, variational autoencoders, diffusion models and GANs.
- **Zhong, Das, Alrasheedi and Tanvir (2023)** [18] survey deep-learning image watermarking and sort methods into three families: jointly trained embedder–extractor networks, networks used as feature transforms around a classical embedding rule, and hybrids.

Learned image compression

Ballé et al. [15] replaced the DCT with a learned nonlinear **analysis transform** (a convolutional network) that maps the image x to a latent y, which is quantized to $\hat{y}$ and decoded by a learned **synthesis transform**. The whole system is trained to minimize a rate-distortion loss

$$\mathcal{L} = R + \lambda D = E\left[-\log_2 p(\hat{y})\right] + \lambda\, E\left[d(x, \hat{x})\right],$$

where R is the expected code length under the entropy model p, D is a distortion such as MSE, and λ sets the trade-off. Their key contribution was the **scale hyperprior**: a second, small latent z is sent as side information and predicts the scale of each element of y, so the entropy model adapts to each image. That is the same idea as JPEG's per-image Huffman tables or JPEG-LS's context modeling, but learned end to end. During training, rounding is replaced by additive uniform noise so gradients can flow.

What did not change: quantization is still rounding, and the bits still come from an arithmetic coder [11]. Learned codecs are better mappers and better probability models inside the same encoder diagram as before. Their practical obstacles are decoder complexity and the need for bit-exact, platform-independent entropy models.

Modern standard codecs

Meanwhile, standardized codecs kept improving with hand-designed tools. AVIF [6] stores AV1 intra frames in a HEIF container, and HEIC files [7] store HEVC intra frames in the same container family. Both reuse the intra-frame (single-picture) coding tools of a video codec, so still images benefit directly from decades of video-coding research. JPEG XL [5] targets photography and the web, supports lossy and lossless coding, high dynamic range and alpha, and can losslessly transcode existing JPEG files to smaller files that restore to the exact original bytes. PNG [4] remains the default for lossless graphics; its third edition (2025) adds HDR signaling and standardizes animated PNG. None of these replaced the textbook's ideas; they combine predictive coding, block transforms and adaptive entropy coding more flexibly.

Deep-learning watermarking

HiDDeN [19] jointly trains an encoder that hides a bit string in a cover image, a decoder that recovers it, and a **noise layer** between them that simulates cropping, blur and JPEG (through differentiable approximations, since real JPEG has no gradient). An adversarial loss keeps the marked images natural. Robustness is learned from simulated attacks rather than designed in by choosing coefficients as we did above, so it is only as good as those simulations.

Generative models opened a new use: marking images *at creation*. Stable Signature [20] fine-tunes the decoder of a latent diffusion model so every generated image carries a fixed hidden signature, recoverable by an extractor even after cropping and other edits, with a false-positive rate below one in a million. This answers a provenance question (was this image made by model X?) rather than an ownership one. Removal and forgery attacks remain open problems.

Key takeaways

- Compression removes coding redundancy, spatial/temporal redundancy and irrelevant information; only the last one is lossy.
- Entropy is the lower bound for lossless coding of independent symbols, and the choice of symbols matters: prediction differences have far lower entropy than raw pixels.
- Huffman codes are optimal per symbol; arithmetic coding approaches the entropy of the whole message; Golomb, LZW, run-length, symbol-based and bit-plane coding exploit particular structure.
- JPEG = level shift + 8×8 DCT + quantization table + zigzag + run-length/Huffman coding; quality is set by scaling the table, and low quality shows blocking and ringing.
- Predictive coders must predict from reconstructed values; motion-compensated prediction is the heart of video coding.
- Wavelet coding (JPEG 2000) avoids block seams and supports scalable, lossless and region-of-interest coding.
- LSB watermarks are invisible but fragile; DCT-domain marks survive JPEG when their strength exceeds the quantization step.
- Learned codecs and deep watermarks replace hand-designed transforms with trained networks but keep quantization and arithmetic coding at their core.

Exercises

1. A source has four symbols with probabilities 0.5, 0.25, 0.125 and 0.125. Build a Huffman code, compute its average length and the entropy, and explain why they are equal here.

   Hint

   Codewords `0`, `10`, `110`, `111`; both values are 1.75 bits. All probabilities are powers of $1/2$, so $-\log_2 P(a_j)$ is an integer and each codeword can have exactly that length.

2. Prediction residuals $e = 0, -1, 2, -3, 1$ are mapped with $M(e)$ from the Golomb section and coded with G_2. Write the bit string and count its bits. Would G_4 be shorter?

   Hint

   $M(e) = 0, 1, 4, 5, 2$. With $m = 2$ the codes are `00`, `01`, `1100`, `1101`, `100` (15 bits). With $m = 4$ they are `00 0`, `001`, `1000`, `1001`, `010` (17 bits). For small residuals the smaller parameter wins.

3. Using the arithmetic coding example ($P(a) = 0.6$, $P(b) = 0.3$, $P(c) = 0.1$), decode the first three symbols of the tag 0.70.

   Hint

   0.70 lies in $[0.6, 0.9)$, so the first symbol is `b`. Rescale: $(0.70 - 0.6)/0.3 = 0.333$, which lies in a's range. Rescale again: $0.333/0.6 = 0.556$, again `a`. The message starts `baa`.

4. Modify the DPCM snippet so the encoder predicts from the *original* previous pixel instead of the reconstructed one, but keep the decoder unchanged. Measure the PSNR along a row and explain what you see.

Hint

The decoder adds quantized errors to its own reconstruction, which no longer matches the encoder's reference. Errors accumulate along the row (drift). On row 200 of the camera image with step 8, PSNR falls from 40.8 dB to about 17 dB.

5. Change the LSB watermark to use bit plane 3 instead of bit plane 0. Measure the PSNR of the marked image and the bit agreement after JPEG at quality 95 and 75. What did you gain and what did you pay?

Hint

Each pixel can now change by up to 8 gray levels, so PSNR falls by roughly $20 \log_{10} 8 \approx 18$ dB (about 33 dB with a random mark). Bit agreement after JPEG rises to about 82% at quality 95 but drops to about 57% at quality 75: you paid a lot of visibility for little robustness, compared with the DCT-domain scheme.

6. Explain why a wavelet coder at very low bit rates looks blurry while a block DCT coder looks blocky, in terms of the support of the basis functions that survive quantization.

Hint

The surviving DCT basis functions are confined to their 8×8 block, so neighboring blocks are approximated independently and meet at seams. The surviving coarse wavelet functions are smooth and overlap their neighbors, so the approximation is continuous but lacks fine detail.

References

1. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018, Ch. 8. publisher page
2. Joint Photographic Experts Group, "JPEG 1 (ISO/IEC 10918-1 | ITU-T Rec. T.81): Digital compression and coding of continuous-tone still images," JPEG overview page. jpeg.org
3. Joint Photographic Experts Group, "JPEG 2000 (ISO/IEC 15444)," JPEG overview page. jpeg.org
4. W3C, "Portable Network Graphics (PNG) Specification (Third Edition)," W3C Recommendation, 2025. w3.org
5. Joint Photographic Experts Group, "JPEG XL (ISO/IEC 18181)," JPEG overview page. jpeg.org
6. Alliance for Open Media, "AV1 Image File Format (AVIF)," v1.2.0, 2025. aomedia-codec.github.io
7. Nokia Technologies, "High Efficiency Image File Format (HEIF), ISO/IEC 23008-12: technical information." nokiatech.github.io

8. D. A. Huffman, "A Method for the Construction of Minimum-Redundancy Codes," *Proceedings of the IRE*, vol. 40, pp. 1098–1101, 1952. doi
9. S. W. Golomb, "Run-length encodings," *IEEE Transactions on Information Theory*, vol. 12, pp. 399–401, 1966. doi
10. M. J. Weinberger, G. Seroussi and G. Sapiro, "The LOCO-I lossless image compression algorithm: principles and standardization into JPEG-LS," *IEEE Transactions on Image Processing*, vol. 9, pp. 1309–1324, 2000. doi
11. I. H. Witten, R. M. Neal and J. G. Cleary, "Arithmetic coding for data compression," *Communications of the ACM*, vol. 30, pp. 520–540, 1987. doi
12. T. A. Welch, "A Technique for High-Performance Data Compression," *Computer*, vol. 17, pp. 8–19, 1984. doi
13. G. K. Wallace, "The JPEG still picture compression standard," *IEEE Transactions on Consumer Electronics*, vol. 38, no. 1, pp. xviii–xxxiv, 1992. doi
14. A. Skodras, C. Christopoulos and T. Ebrahimi, "The JPEG 2000 still image compression standard," *IEEE Signal Processing Magazine*, vol. 18, pp. 36–58, 2001. doi
15. J. Ballé, D. Minnen, S. Singh, S. J. Hwang and N. Johnston, "Variational image compression with a scale hyperprior," *ICLR*, 2018. arXiv
16. Y. Hu, W. Yang, Z. Ma and J. Liu, "Learning End-to-End Lossy Image Compression: A Benchmark," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2021. arXiv
17. Y. Yang, S. Mandt and L. Theis, "An Introduction to Neural Data Compression," *Foundations and Trends in Computer Graphics and Vision*, vol. 15, no. 2, 2023. arXiv
18. X. Zhong, A. Das, F. Alrasheedi and A. Tanvir, "A Brief Yet In-Depth Survey of Deep Learning-Based Image Watermarking," *Applied Sciences*, 2023. arXiv
19. J. Zhu, R. Kaplan, J. Johnson and L. Fei-Fei, "HiDDeN: Hiding Data With Deep Networks," *arXiv:1807.09937*, 2018. arXiv
20. P. Fernandez, G. Couairon, H. Jégou, M. Douze and T. Furon, "The Stable Signature: Rooting Watermarks in Latent Diffusion Models," *ICCV*, 2023. arXiv

IP 09 • Morphological Image Processing

Probe shapes with small structuring elements: erosion, dilation, opening, closing, hit-or-miss, skeletons, reconstruction and grayscale morphology, with scikit-image code.

Online edition: https://blog.shyandram.dev/en/learn/image-processing/dip-09-morphology/

What you'll learn

- How to treat a binary image as a set of pixel coordinates, and how a small **structuring element** probes it
- The two primitive operations, **erosion** and **dilation**, and how they combine into **opening** and **closing**
- How the **hit-or-miss transform** finds exact pixel patterns such as corners and isolated points
- Classic algorithms built from those primitives: boundaries, hole filling, connected components, convex hulls, thinning, skeletons and pruning
- **Morphological reconstruction**, which removes objects without distorting the ones that survive
- **Grayscale morphology**: gradients, top-hat correction of uneven lighting, and size distributions (granulometry)

The big picture

Most of the filters you met in earlier chapters were *linear*: each output pixel is a weighted sum of input pixels. Morphology is different. It asks yes-or-no questions about **shape**: "Does this little shape fit here?" "Does it touch anything here?" The answers are built only from minimum, maximum, intersection and union. No multiplication is involved.

> **Explain like I'm five**
>
> Imagine a sheet of cookie dough with a funny outline, and a small round cookie cutter. Slide the cutter everywhere over the table. **Erosion** marks every spot where the cutter fits *completely* on the dough, so the dough shrinks and thin bits vanish. **Dilation** marks every spot where the cutter touches *any* dough, so the dough grows and small gaps fill in. **Opening** keeps only the dough you could actually cut cookies from; **closing** fills every dent that is too small for the cutter to poke into. Change the cutter's shape (a line, a square, a cross) and you ask a different question about the dough.

Why it matters: morphology is the standard toolbox for cleaning binary masks (from thresholding in Chapter 10, or from a neural network), for measuring objects (counting, sizing, finding their skeletons), and for correcting uneven backgrounds. Its theory, **mathematical morphology**, grew out of work by Georges Matheron and Jean Serra on random sets and on the geometry of porous materials [2], [3]. Haralick, Sternberg and Zhuang's tutorial brought it to the wider

image-analysis community [4], and Soille's book remains the standard practical reference [5]. This chapter follows the topic order of Gonzalez and Woods [1] and uses scikit-image [12] and SciPy for the code.

Preliminaries

In plain words: a binary image is a list of "on" pixel positions, and a structuring element is a tiny stencil with a marked centre that we slide over that list.

Images as sets

Let a binary image have foreground pixels (value 1) and background pixels (value 0). We describe the foreground by the set

$$A = \{(x, y) \in \mathbb{Z}^2 \mid I(x, y) = 1\},$$

where I is the image and (x, y) are integer pixel coordinates. The usual set operations now have image meanings: the **complement** A^c is the background, the **union** $A \cup C$ is a pixel-wise OR, the **intersection** $A \cap C$ is a pixel-wise AND, and the **difference** $A - C = A \cap C^c$ removes from A everything in C.

Structuring elements

A **structuring element** (SE) B is a second, small set of offsets with a designated **origin**, usually its centre. Think of it as the probe. Common choices are a 3×3 square, a cross (4-neighbourhood), a disk, or a line segment. The SE's shape decides which features the operation responds to: a disk is direction-neutral, a horizontal line responds only to horizontal structure. scikit-image calls an SE a **footprint**. In practice, an SE is stored as a small Boolean array whose `True` entries are its members; elements shown as "don't care" in diagrams are simply not in the set.

Reflection and translation

Two operations on sets appear in every definition below. The **reflection** of B is

$$\hat{B} = \{w \mid w = -b,\ b \in B\},$$

that is, every offset b is replaced by $-b$: the SE is rotated 180° about its origin. The **translation** of B by a vector $z = (z_1, z_2)$ is

$$(B)_z = \{c \mid c = b + z,\ b \in B\},$$

which places the SE's origin at pixel z. For symmetric SEs such as disks and squares, $\hat{B} = B$, so the reflection is invisible; for asymmetric ones it matters.

```
import numpy as np
import Lab from '../../../../components/Lab.astro';
from skimage import morphology

B = morphology.disk(2)            # 5x5 disk; origin at the centre (2, 2)
B_hat = B[::-1, ::-1]             # reflection: flip both axes
L = np.array([[1, 1, 0],
              [0, 1, 0],
              [0, 0, 0]], bool)   # an asymmetric element
print(L[::-1, ::-1].astype(int))  # its reflection points the other way
```

Erosion and dilation

In plain words: erosion keeps a pixel only if the whole stencil fits inside the shape there; dilation turns a pixel on if the stencil touches the shape at all.

Erosion

The **erosion** of A by B is

$$A \ominus B = \{z \mid (B)_z \subseteq A\},$$

the set of all positions z at which the translated SE lies entirely inside the foreground. Equivalently, $A \ominus B = \{z \mid (B)_z \cap A^c = \emptyset\}$: the SE must not overlap any background pixel. Erosion shrinks objects by roughly the SE's radius, deletes any part thinner than the SE, and separates objects joined by thin bridges. If B contains its origin, erosion is **anti-extensive**: $A \ominus B \subseteq A$.

Dilation

The **dilation** of A by B is

$$A \oplus B = \{z \mid (\hat{B})_z \cap A \neq \emptyset\},$$

the set of positions at which the reflected SE overlaps at least one foreground pixel. An equivalent form is the union of copies of B placed at every foreground pixel: $A \oplus B = \bigcup_{a \in A} (B)_a$. Dilation grows objects, bridges gaps narrower than the SE, and fills small holes. It is **extensive** ($A \subseteq A \oplus B$) when B contains its origin. Unlike erosion, dilation is commutative: $A \oplus B = B \oplus A$.

The reflection in the dilation formula is what makes dilation a true Minkowski sum and keeps it consistent with convolution's flip. For symmetric SEs it changes nothing.

Duality

Erosion and dilation are **duals** with respect to complement and reflection:

$$(A \ominus B)^c = A^c \oplus \hat{B}, \qquad (A \oplus B)^c = A^c \ominus \hat{B}.$$

Eroding the foreground is the same as dilating the background (with the reflected SE), then flipping the result. The proof is one line: $z \notin A \ominus B$ exactly when $(B)_z$ meets A^c, which is the definition of $z \in A^c \oplus \hat{B}$ (because $\hat{\hat{B}} = B$). In code, duality holds exactly only if you treat the image border consistently: pixels outside the image must count as foreground for the erosion when they count as background for the dilation of the complement.

```
import numpy as np
from scipy import ndimage as ndi
from skimage import data, morphology

A = ~data.horse()                 # True = horse pixels
B = morphology.disk(3)

eroded  = ndi.binary_erosion(A, structure=B)
dilated = ndi.binary_dilation(A, structure=B)
print(A.sum(), eroded.sum(), dilated.sum())     # erosion shrinks, dilation grows

# Duality check. border_value=1 treats outside pixels as foreground for the
# erosion, which matches background-outside for the dilation of ~A.
lhs = ~ndi.binary_erosion(A, structure=B, border_value=1)
rhs = ndi.binary_dilation(~A, structure=B[::-1, ::-1])
print("duality holds:", np.array_equal(lhs, rhs))   # True
```

Opening and closing

In plain words: opening is "erode, then dilate back": it removes specks and thin parts but leaves big shapes nearly unchanged. Closing is "dilate, then erode back": it fills small holes and narrow cracks.

Definitions

The **opening** of A by B is erosion followed by dilation with the same SE,

$$A \circ B = (A \ominus B) \oplus B,$$

and the **closing** is dilation followed by erosion,

$$A \bullet B = (A \oplus B) \ominus B.$$

Erosion alone throws away detail but also shrinks everything; the second step restores the size of whatever survived. The pair is dual, just like erosion and dilation: $(A \bullet B)^c = A^c \circ \hat{B}$ and $(A \circ B)^c = A^c \bullet \hat{B}$. Closing the foreground is opening the background.

Geometric interpretation: the rolling ball

Opening has a direct picture. It equals the union of all translates of B that fit inside A:

$$A \circ B = \bigcup \{(B)_z \mid (B)_z \subseteq A\}.$$

Roll a ball (the SE) around *inside* the foreground, staying fully inside. Every pixel the ball can reach is kept; corners sharper than the ball, thin necks and specks smaller than the ball are lost. Closing is the same picture applied from the outside: roll the ball around in the *background*; every background pixel the ball cannot reach (narrow gaps, small holes, deep thin bays) becomes foreground.

Properties

Opening and closing are both **morphological filters** in the technical sense: they are increasing and idempotent [5].

- **Opening is anti-extensive:** $A \circ B \subseteq A$. It only removes pixels.
- **Closing is extensive:** $A \subseteq A \bullet B$. It only adds pixels.
- **Increasing:** if $C \subseteq D$ then $C \circ B \subseteq D \circ B$, and the same for closing. Bigger input never gives smaller output.
- **Idempotent:** $(A \circ B) \circ B = A \circ B$ and $(A \bullet B) \bullet B = A \bullet B$. Applying the same opening twice changes nothing, because whatever survived the first pass already consists of places where the ball fits.

Idempotence is the property that separates opening and closing from erosion and dilation: repeated erosions keep shrinking, while repeated openings stop. Figure 9.1 shows all four operations on a noisy silhouette. The opening removes the white specks in the background; the closing removes the black specks inside the horse; opening followed by closing removes both, a simple and very common **morphological noise filter**.

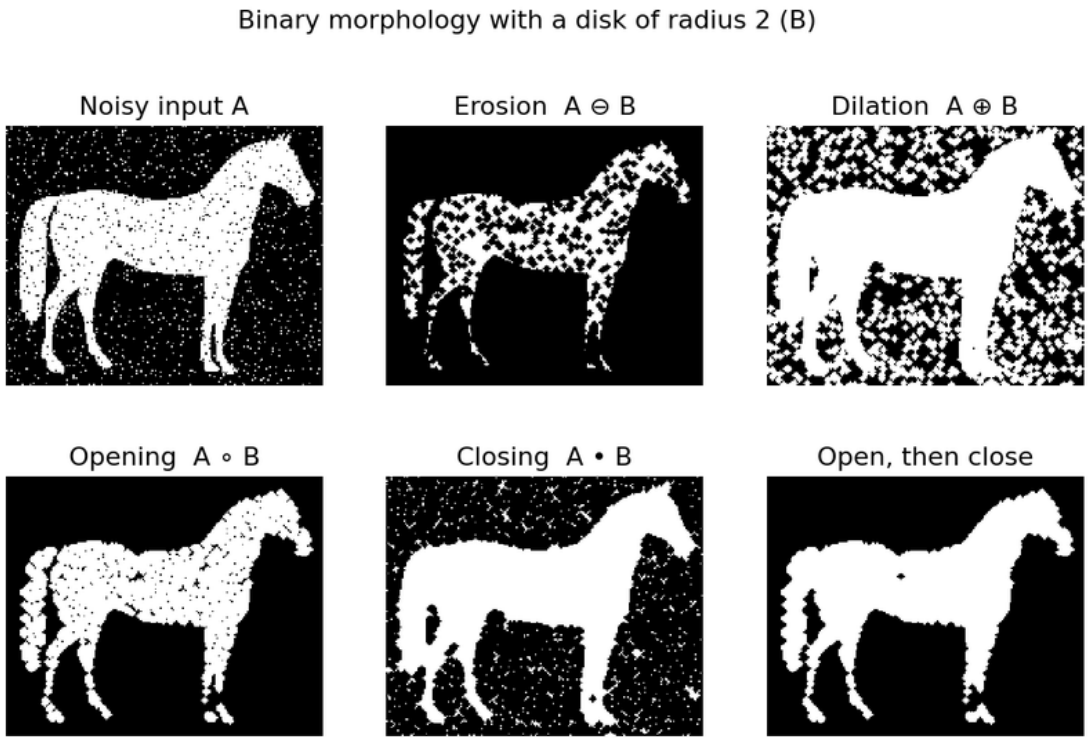


Figure 9.1 — Binary erosion, dilation, opening and closing of a horse silhouette with 4% of pixels flipped, using a disk of radius 2. Opening then closing removes both kinds of speck while keeping the outline.

> **Interactive demonstration (online)**: open the online edition of this tutorial to use this demonstration.Open demonstration

```
import numpy as np
from skimage import data, morphology

A = ~data.horse()
B = morphology.disk(4)
op = morphology.binary_opening(A, B)
cl = morphology.binary_closing(A, B)
print("opening is anti-extensive:", not (op & ~A).any())
print("closing is extensive:     ", not (A & ~cl).any())
print("opening is idempotent:    ", np.array_equal(morphology.binary_opening(op, B), op))
```

The hit-or-miss transform

In plain words: the hit-or-miss transform (HMT) is a template matcher for binary patterns. You say which pixels must be foreground and which must be background, and it lights up exactly where both conditions hold.

Use two disjoint SEs: B_1, the pixels that must lie in the foreground, and B_2, the pixels that must lie in the background. The HMT of A is

$$A \circledast B_{1,2} = \{z \mid (B_1)_z \subseteq A \text{ and } (B_2)_z \subseteq A^c\} = (A \ominus B_1) \cap (A^c \ominus B_2).$$

Here $B_{1,2}$ denotes the pair (B_1, B_2), $A \ominus B_1$ finds positions where the foreground pattern fits, and $A^c \ominus B_2$ finds positions where the background pattern fits. Pixels in neither B_1 nor B_2 are **don't-care** positions. It is common to draw the pair as one small grid with 1 (must be foreground), 0 (must be background) and blank (don't care).

Some useful templates:

- **Isolated point:** B_1 is the centre pixel; B_2 is its 8 neighbours.
- **Upper-left corner:** B_1 is the centre plus its right and lower neighbours; B_2 is the upper and left neighbours.
- **End point of a one-pixel line:** the centre and exactly one neighbour are foreground; the rest are background. You need one template per direction (8 rotations).

Figure 9.2 applies the first two templates. Note that the staircase outline of a digital disk contains several genuine upper-left corners at pixel level: the HMT is literal, not geometric.

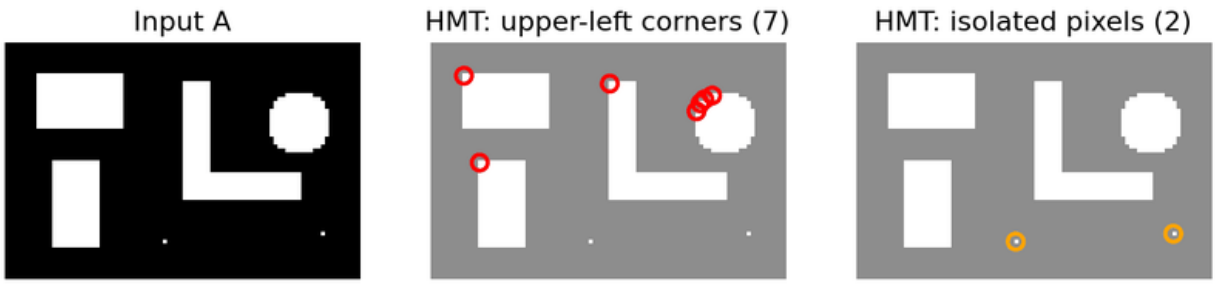


Figure 9.2 — The hit-or-miss transform as an exact pattern detector. Middle: upper-left corners (including small steps on the digital disk). Right: isolated foreground pixels.

```
import numpy as np
from scipy import ndimage as ndi

A = np.zeros((7, 9), bool)
A[1:5, 1:6] = True                # a rectangle
A[5, 7] = True                    # an isolated pixel
hit  = np.array([[0, 0, 0], [0, 1, 1], [0, 1, 0]], bool)   # must be foreground
miss = np.array([[0, 1, 0], [1, 0, 0], [0, 0, 0]], bool)   # must be background
corners = ndi.binary_hit_or_miss(A, structure1=hit, structure2=miss)
print(np.argwhere(corners))       # [[1 1]]: the upper-left corner only
```

The HMT is the building block of thinning, thickening and the convex hull algorithm below.

Some basic morphological algorithms

In plain words: with erosion, dilation and the HMT you can build many practical tools. Most of them are short loops that repeat one operation until nothing changes.

Boundary extraction

The inner boundary of a set is what erosion removes:

$$\beta(A) = A - (A \ominus B),$$

where B is a 3×3 square (giving an 8-connected boundary) or a cross (giving a thicker, 4-connected one). Using a larger B gives a thicker boundary. The first panel of Figure 9.3 shows $\beta(A)$ for the horse.

Hole filling

A **hole** is a background region completely surrounded by foreground. Given one seed pixel inside a hole, grow the seed by repeated dilation but clip it to the background at every step:

$$X_k = (X_{k-1} \oplus B) \cap A^c, \qquad k = 1, 2, 3, \ldots$$

Here X_0 contains only the seed, B is the 4-connected cross, and the intersection with A^c stops the growth at the hole's wall. Stop when $X_k = X_{k-1}$; then $X_k \cup A$ is the filled shape. This "dilate, then clip" pattern is called **conditional dilation**, and it reappears below as geodesic dilation. Using the cross (not the square) as B matters: with an 8-connected foreground wall, the 4-connected background fill cannot leak diagonally through the wall.

```
import numpy as np
from scipy import ndimage as ndi

def fill_hole(A, seed):
    """Grow X from a seed inside one hole: X_k = dilate(X_{k-1}) AND not A."""
    B = np.array([[0, 1, 0], [1, 1, 1], [0, 1, 0]], bool)   # 4-connected cross
    X = np.zeros_like(A); X[seed] = True
    while True:
        X_next = ndi.binary_dilation(X, structure=B) & ~A
        if np.array_equal(X_next, X):
            return A | X
        X = X_next

yy, xx = np.mgrid[-12:13, -12:13]
ring = (np.hypot(yy, xx) <= 10) & (np.hypot(yy, xx) > 6)   # boolean ring
print(ring.sum(), fill_hole(ring, (12, 12)).sum())        # 204 -> 317
```

Extraction of connected components

Swap the roles: to extract the connected component of A that contains a seed pixel, grow the seed by dilation but clip it to the *foreground*:

$$X_k = (X_{k-1} \oplus B) \cap A.$$

With B a 3×3 square the result is the seed's 8-connected component; with a cross it is the 4-connected component. Repeating this for every unlabelled seed labels all components. Real libraries use faster one- or two-pass labelling algorithms, but the result is the same: `scipy.ndimage.label(A, structure=np.ones((3, 3)))` returns a label image and the number of components.

Convex hull

A set is **convex** if the straight segment between any two of its points stays inside it. The **convex hull** $C(A)$ is the smallest convex set containing A. A purely morphological approximation uses four HMT templates B^i, $i = 1, \dots, 4$, each one detecting background pixels that have foreground on one side (left, top, right, bottom). Starting from $X_0^i = A$, iterate

$$X_k^i = (X_{k-1}^i \circledast B^i) \cup X_{k-1}^i$$

until convergence, giving D^i, and take $C(A) = \bigcup_{i=1}^{4} D^i$. Each template keeps adding pixels in one direction until no concavity facing that direction remains. The result is the convex hull with respect to those four directions; it can grow beyond the true Euclidean hull, so implementations often limit growth to the bounding box. In practice `skimage.morphology.convex_hull_image` computes the polygonal hull of the foreground pixel coordinates and rasterises it, which is faster and closer to the Euclidean hull.

Thinning

Thinning removes boundary pixels selectively, without breaking the shape apart, until only a one-pixel-wide line remains. One thinning step with template B is

$$A \otimes B = A - (A \circledast B),$$

which deletes exactly the pixels matched by the HMT. A full thinning cycles through a sequence of templates $\{B^1, B^2, \dots, B^n\}$, usually 8 rotations of an "edge pixel" pattern,

$$A \otimes \{B\} = ((\cdots ((A \otimes B^1) \otimes B^2) \cdots) \otimes B^n),$$

and repeats the whole cycle until no pixel changes. The templates are designed so that deleting a matched pixel never disconnects the shape and never removes the endpoint of a line. Zhang and Suen's parallel thinning algorithm is a widely used, efficient version of this idea [6]; `skimage.morphology.skeletonize` uses it by default for 2-D images, and `skimage.morphology.thin` implements a related parallel thinning.

Thickening

Thickening is the dual of thinning: it adds background pixels matched by a template,

$$A \odot B = A \cup (A \circledast B).$$

Rather than designing separate thickening templates, the usual practical route is to thin the background and complement the result. Carried to convergence, thinning the background leaves a thin network of background lines running midway between objects, so thickening is a classic way to grow objects until they meet without ever merging.

Skeletons

The **skeleton** $S(A)$ is a thin set of lines running down the middle of a shape. One formal definition uses maximal disks: a point is on the skeleton if it is the centre of a disk that fits inside A and is not contained in any other fitting disk. Such disks touch the boundary in at least two places. The skeleton has a purely morphological formula:

$$S(A) = \bigcup_{k=0}^{K} S_k(A), \qquad S_k(A) = (A \ominus kB) - ((A \ominus kB) \circ B),$$

where $A \ominus kB$ means k successive erosions of A by B and K is the last k before the erosion becomes empty. Each $S_k(A)$ holds the points that are "ridge" points at erosion depth k: they are present after k erosions but would not survive an opening at that depth. The decomposition is invertible:

$$A = \bigcup_{k=0}^{K} (S_k(A) \oplus kB),$$

so storing each S_k with its index k is a lossless shape code. The catch is that this skeleton is not guaranteed to be connected. Thinning gives connected, one-pixel-wide skeletons, and the **medial axis** (computed from a distance transform) attaches the radius of the maximal disk to each skeleton point, as in the right panel of Figure 9.3.

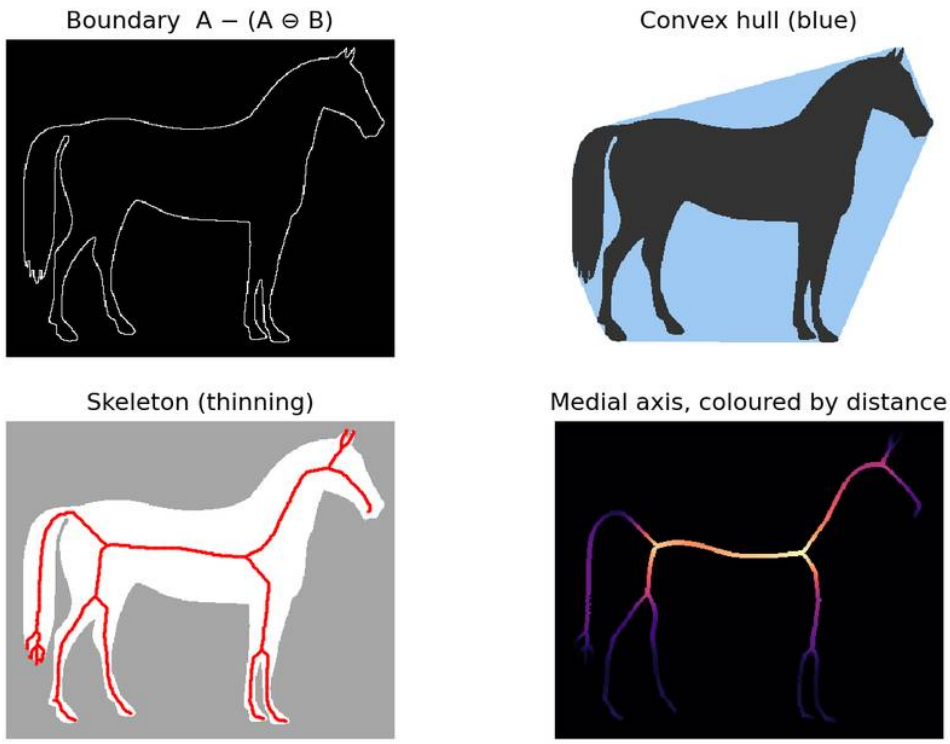


Figure 9.3 — Algorithms built from erosion and the HMT: inner boundary, convex hull, thinning skeleton, and medial axis whose colour encodes the local half-width.

Pruning

Skeletons are sensitive to boundary noise: a single bump on the outline sprouts a short **spur** (parasitic branch), visible at the horse's tail in Figure 9.3. **Pruning** removes spurs shorter than some length L:

1. Thin the skeleton by removing **end points** (pixels with exactly one 8-neighbour) L times. Every spur of length at most L disappears, but every genuine branch also loses L pixels at its tip.
2. Find the end points of the shortened skeleton.
3. Grow those end points back by L conditional dilations, constrained to the original skeleton, to restore the main branches to full length.
4. Take the union of the shortened skeleton and the regrown tips.

```
import numpy as np
from scipy import ndimage as ndi
from skimage import data, morphology

def endpoints(S):
    n = ndi.convolve(S.astype(int), np.ones((3, 3), int), mode="constant") - S
    return S & (n == 1)            # foreground pixels with exactly one neighbour
```

```
def prune_tips(S, L):
    """Step 1 of pruning: strip L layers of end points."""
    S = S.copy()
    for _ in range(L):
        S &= ~endpoints(S)
    return S

skel = morphology.skeletonize(~data.horse())
print(skel.sum(), prune_tips(skel, 10).sum())
```

Morphological reconstruction

In plain words: reconstruction lets you say "keep every object that contains this seed" and get the object back *exactly*, not a rounded copy of it. It uses two images: a **marker**, which says where to start, and a **mask**, which says how far you may grow.

Geodesic dilation and erosion

Let F be the marker and G the mask, with $F \subseteq G$. The **geodesic dilation of size 1** of F with respect to G is

$$D_G^{(1)}(F) = (F \oplus B) \cap G,$$

where B is a small SE (usually the 3×3 square). The marker grows by one step but may not leave the mask. Size n means n repetitions: $D_G^{(n)}(F) = D_G^{(1)}(D_G^{(n-1)}(F))$. The word *geodesic* refers to distances measured along paths that stay inside G, like walking inside a building rather than through its walls.

The dual **geodesic erosion of size 1** is

$$E_G^{(1)}(F) = (F \ominus B) \cup G,$$

with $F \supseteq G$: the marker shrinks, but never below the mask.

Reconstruction by dilation and by erosion

Iterate geodesic dilation until nothing changes. The result is the **reconstruction by dilation** of mask G from marker F:

$$R_G^D(F) = D_G^{(k)}(F) \quad \text{with } k \text{ such that } D_G^{(k)}(F) = D_G^{(k+1)}(F).$$

In binary images, $R_G^D(F)$ is exactly the union of the connected components of G that contain at least one marker pixel. Likewise, the **reconstruction by erosion** $R_G^E(F)$ iterates geodesic erosion to stability. Vincent's paper gives the standard efficient algorithms (queue-based, visiting each pixel a bounded number of times rather than iterating full-image passes) and many applications [7]; `skimage.morphology.reconstruction` follows that approach.

Opening by reconstruction

An ordinary opening removes small objects but also rounds the corners and erases the thin parts of the large ones. **Opening by reconstruction** avoids that:

$$O_R^{(n)}(F) = R_F^D(F \ominus nB),$$

where F is the input image, $F \ominus nB$ is n erosions by B (the marker), and the input itself is the mask. Erosion decides *which* objects survive (those that contain at least one placement of the SE); reconstruction restores each survivor to its original shape. Compare the "ordinary opening" and "opening by reconstruction" panels of Figure 9.4: the thin arm on the square and the sharp corners return intact. **Closing by reconstruction** is the dual, built from dilation and reconstruction by erosion.

Automatic hole filling

Reconstruction also fills all holes at once without seeds. Holes are background regions that cannot be reached from the image frame. So build a marker that is the complement of the image on the frame and zero elsewhere,

$$F(x,y) = \begin{cases} 1 - I(x,y) & \text{if } (x,y) \text{ is on the image border,} \\ 0 & \text{otherwise,} \end{cases}$$

reconstruct the background from it, and complement:

$$H = [R_{I^c}^D(F)]^c.$$

$R_{I^c}^D(F)$ is the background reachable from the border; everything else, H, is the foreground plus its holes.

Border clearing

Objects that touch the image border are often incomplete and should be excluded from measurements. Using the image itself on the border as the marker,

$$F(x,y) = \begin{cases} I(x,y) & \text{if } (x,y) \text{ is on the border,} \\ 0 & \text{otherwise,} \end{cases} \qquad X = I - R_I^D(F),$$

the reconstruction recovers every object that touches the border, and subtracting it leaves only interior objects.

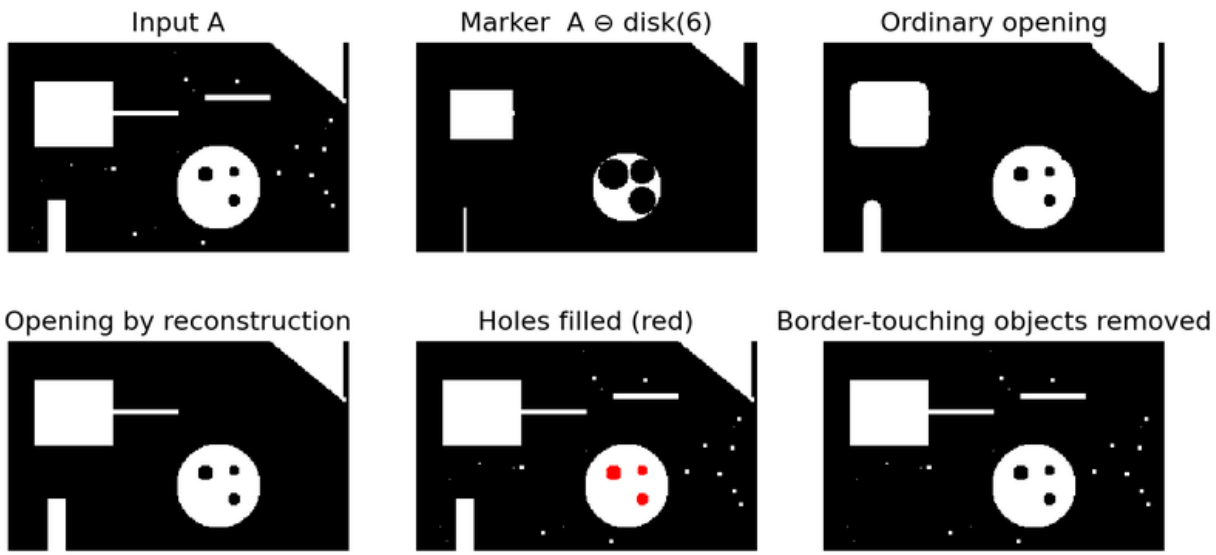


Figure 9.4 — Reconstruction. The eroded marker (disk of radius 6) picks the large objects; reconstruction restores them exactly, including the thin arm. Bottom: automatic hole filling and border clearing.

```
import numpy as np
from skimage import morphology, segmentation

def open_by_reconstruction(A, footprint):
    marker = morphology.binary_erosion(A, footprint)
    return morphology.reconstruction(marker.astype(np.uint8), A.astype(np.uint8),
                                     method="dilation").astype(bool)

def fill_holes_auto(A):
    """Background reachable from the image frame is background; the rest is hole."""
    Ac = ~A
    marker = np.zeros_like(Ac)
    marker[0, :], marker[-1, :] = Ac[0, :], Ac[-1, :]
    marker[:, 0], marker[:, -1] = Ac[:, 0], Ac[:, -1]
    reached = morphology.reconstruction(marker.astype(np.uint8), Ac.astype(np.uint8),
                                        method="dilation").astype(bool)
    return ~reached

# Library shortcuts: scipy.ndimage.binary_fill_holes(A), segmentation.clear_border(A)
```

Summary of binary operations

The table collects the binary operations of this chapter. Throughout, A is the input set, B the structuring element, F a marker, G a mask, and B is assumed to contain its origin.

Operation	Formula	What it does
Translation	$(B)_z = \{b + z\}$	Moves the SE's origin to z
Reflection	$\hat{B} = \{-b\}$	Rotates the SE by 180°
Complement	A^c	Swaps foreground and background
Erosion	$A \ominus B$	Keeps positions where B fits; shrinks, removes thin parts
Dilation	$A \oplus B$	Keeps positions where $\hat{B}$ touches A; grows, bridges gaps
Opening	$(A \ominus B) \oplus B$	Removes specks, thin necks, sharp corners; idempotent
Closing	$(A \oplus B) \ominus B$	Fills small holes and narrow gaps; idempotent
Hit-or-miss	$(A \ominus B_1) \cap (A^c \ominus B_2)$	Finds an exact foreground/ background pattern
Boundary	$A - (A \ominus B)$	One-pixel inner outline
Hole filling	$X_k = (X_{k-1} \oplus B) \cap A^c$	Fills the hole that contains the seed
Connected component	$X_k = (X_{k-1} \oplus B) \cap A$	Extracts the component that contains the seed
Convex hull	union of iterated HMTs	Fills concavities
Thinning	$A - (A \circledast B)$, over a template sequence	Peels pixels to a 1-pixel skeleton, keeps connectivity
Thickening	$A \cup (A \circledast B)$	Dual of thinning
Skeleton	$\bigcup_k (A \ominus kB) - (A \ominus kB) \circ B$	Medial lines; invertible with the index k
Pruning	end-point removal, then conditional regrowth	Removes short spurs
Geodesic dilation	$(F \oplus B) \cap G$	One step of growth inside a mask
Geodesic erosion	$(F \ominus B) \cup G$	One step of shrinking above a mask
Reconstruction by dilation	$D_G^{(k)}(F)$ at stability	Components of G hit by F
Opening by reconstruction	$R_F^D(F \ominus nB)$	Removes small objects, keeps others exact
Hole filling (auto)	$[R_{I^c}^D(F)]^c$	Fills all holes, no seeds
Border clearing	$I - R_I^D(F)$	Removes border-touching objects

Grayscale morphology

In plain words: for gray images, "fits inside" becomes "local minimum" and "touches" becomes "local maximum". Picture the image as a landscape where brightness is height. Erosion lowers each pixel to the lowest ground under the stencil; dilation raises it to the highest.

Grayscale erosion and dilation

With a **flat** SE b (a set of offsets, all of height zero), grayscale erosion and dilation of an image f are

$$[f \ominus b](x,y) = \min_{(s,t)\in b} f(x+s,\, y+t), \qquad [f \oplus b](x,y) = \max_{(s,t)\in b} f(x-s,\, y-t),$$

where (s,t) runs over the SE's offsets. The minus sign in dilation is the reflection again. On

a binary image these reduce exactly to the set definitions. Erosion darkens the image, shrinks bright features and widens dark ones; dilation does the opposite.

A **nonflat** SE b_N also has heights $b_N(s,t)$:

$$[f \ominus b_N](x,y) = \min_{(s,t)\in b_N} \{f(x+s,\, y+t) - b_N(s,t)\}, \qquad [f \oplus b_N](x,y) = \max_{(s,t)\in b_N} \{f(x-s,\, y-t) + b_N(s,t)\}.$$

Nonflat SEs are rarely used in practice: the result depends on the image's intensity scale, and erosion can go below the image's minimum. Duality carries over with the gray complement $f^c = -f$ (or $L-1-f$ for an L-level image): $(f \ominus b)^c = f^c \oplus \hat{b}$.

Opening and closing

The definitions are unchanged: $f \circ b = (f \ominus b) \oplus b$ and $f \bullet b = (f \oplus b) \ominus b$. The geometric picture is a ball pushed up from *beneath* the intensity surface: the opening is the highest surface the ball can reach while staying below f. Bright peaks narrower than the ball are cut down; everything wider is left alone. Closing pushes the ball down from *above*: dark valleys narrower than the ball are filled. The properties (anti-extensive/extensive, increasing, idempotent) carry over with $\leq$ in place of $\subseteq$.

Morphological smoothing

Because opening suppresses small bright details and closing suppresses small dark details, the sequence **opening followed by closing** with the same SE removes both kinds of small structure, while keeping sharp edges of larger regions. This is the grayscale version of the noise filter in Figure 9.1. A related scheme, **alternating sequential filtering**, applies opening–closing pairs with SEs of increasing size, which avoids the artefacts of using one large SE at once.

Morphological gradient

Dilation minus erosion gives a measure of local contrast:

$$g = (f \oplus b) - (f \ominus b).$$

Inside uniform regions the max and min are nearly equal, so $g \approx 0$; across an edge they differ by the edge height. Unlike a Sobel derivative, g is non-negative and direction-independent (for a symmetric b). Using only one side gives the **internal gradient** $f - (f \ominus b)$ or the **external gradient** $(f \oplus b) - f$, which place the edge response just inside or just outside the brighter region.

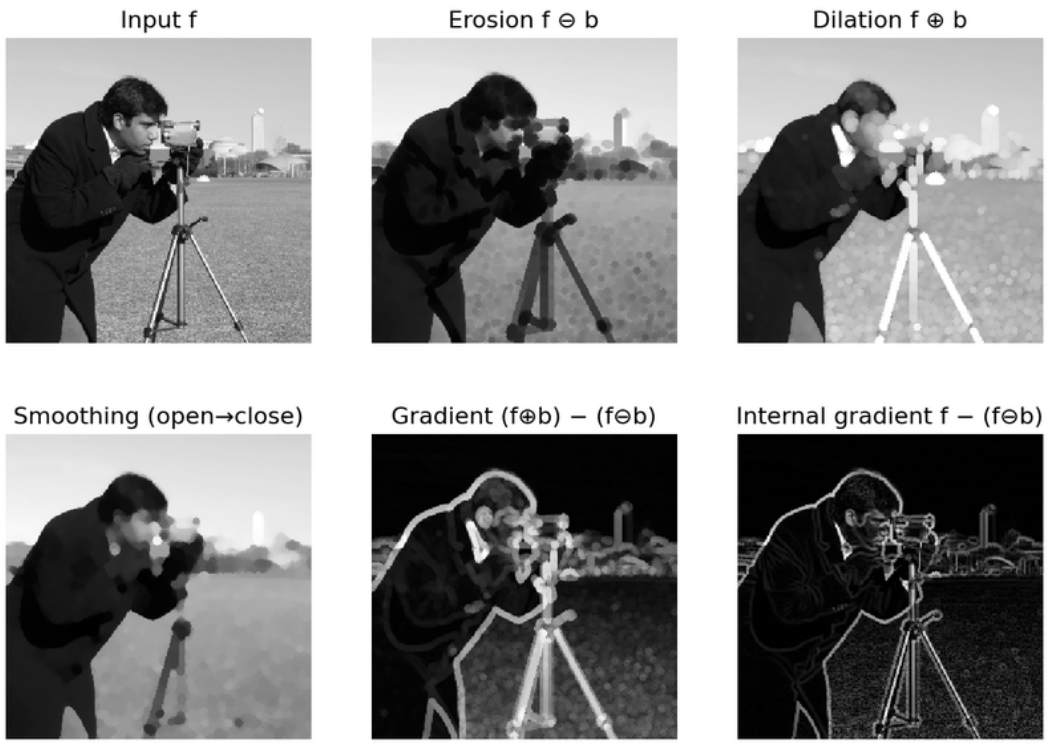


Figure 9.5 — Grayscale morphology with a flat disk of radius 3: erosion (local min), dilation (local max), smoothing by opening then closing, the morphological gradient, and the internal gradient.

```
from skimage import data, morphology, util

f = util.img_as_float(data.camera())
b = morphology.disk(3)                          # flat structuring element
ero  = morphology.erosion(f, b)                 # local minimum
dil  = morphology.dilation(f, b)                # local maximum
grad = dil - ero                                # morphological gradient (>= 0)
smooth = morphology.closing(morphology.opening(f, b), b)
```

Top-hat and bottom-hat transforms

Opening with an SE larger than the bright objects removes those objects but keeps the slowly varying background. Subtracting the opening from the image therefore leaves only the objects. This is the **top-hat** (or white top-hat) transform:

$$T_{\text{hat}}(f) = f - (f \circ b).$$

Its dual, the **bottom-hat** (or black top-hat) transform, extracts dark objects on a lighter background:

$$B_{\text{hat}}(f) = (f \bullet b) - f.$$

Both are non-negative. The main use is correcting **uneven illumination** before thresholding. In Figure 9.6, a synthetic image of grains lit from one side defeats a global Otsu threshold (Chapter 10): the bright side of the background is brighter than the grains on the dark side. After a top-hat with a disk of radius 8 (larger than any grain), the background is flat and a single threshold separates the grains almost perfectly. The SE size is the only parameter: it must be larger than the objects and smaller than the scale over which the background changes.

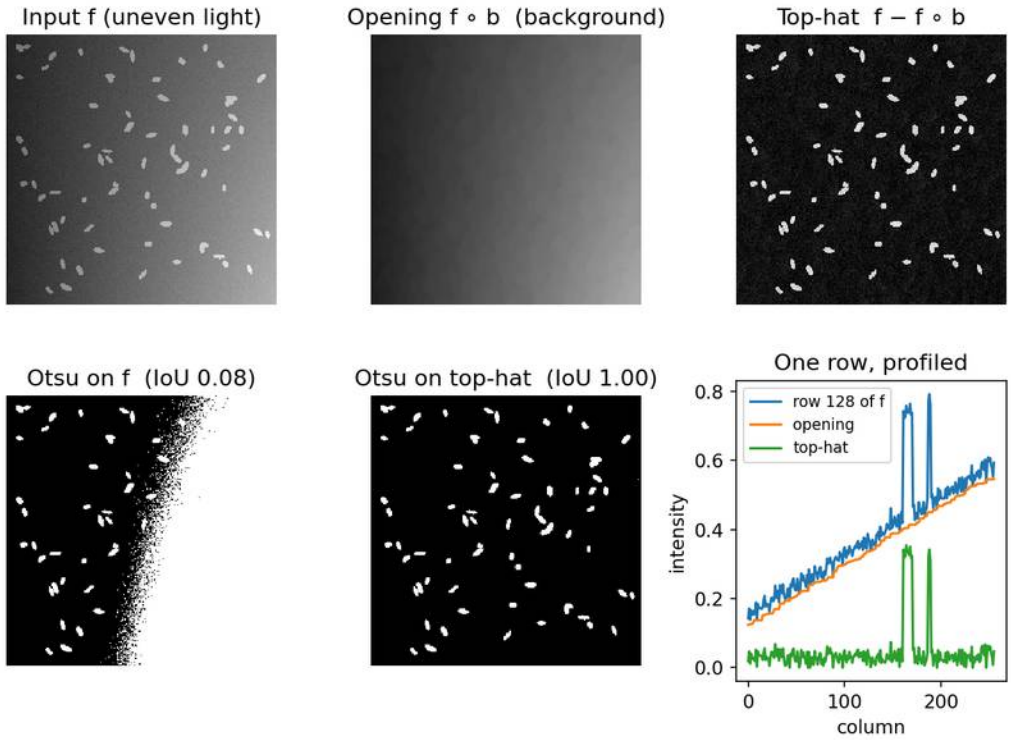
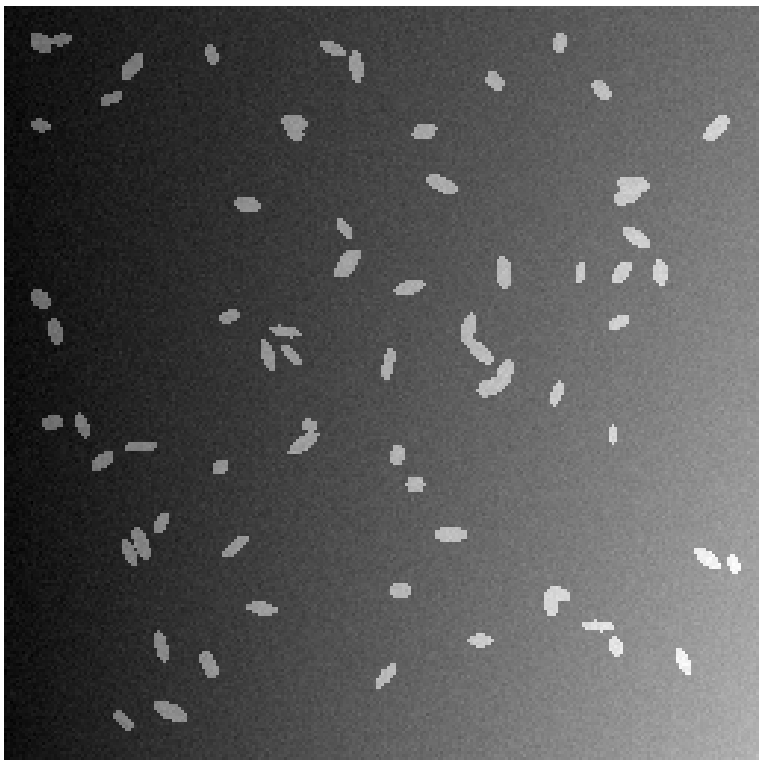


Figure 9.6 — Top-hat correction of uneven illumination. The opening with a disk larger than the grains is a background estimate; subtracting it flattens the background. IoU is measured against the known grain mask.

Drag the slider to compare the input with its top-hat:



Input

Top-hat (disk r=8)

Grains under uneven illumination, before and after the white top-hat transform

```
from skimage import data, filters, morphology, util

coins = util.img_as_float(data.coins())
th = morphology.white_tophat(coins, morphology.disk(30))   # bright objects
bh = morphology.black_tophat(coins, morphology.disk(30))   # dark objects
mask = th > filters.threshold_otsu(th)
```

Granulometry

Granulometry measures the size distribution of particles in an image without segmenting them. Open the image with SEs of increasing size r; each opening removes the bright particles smaller than b_r. Record the total intensity that remains,

$$V(r) = \sum_{x,y} [f \circ b_r](x,y), \qquad r = 0, 1, 2, \ldots,$$

where b_r is a disk of radius r and b_0 is a single pixel (so $V(0)$ is the sum of f). $V(r)$ never increases, because larger disks fit in fewer places. Its negative discrete derivative,

$$PS(r) = V(r-1) - V(r),$$

is a size histogram called the **pattern spectrum** [8]: a large value at r means a lot of image "mass" lives in features of radius about r. Figure 9.7 shows a scene with two particle sizes and the two corresponding peaks. The idea of a family of openings indexed by size, with the "sieve" interpretation, goes back to Matheron [2]; Maragos developed the pattern spectrum as a multiscale shape descriptor [8].

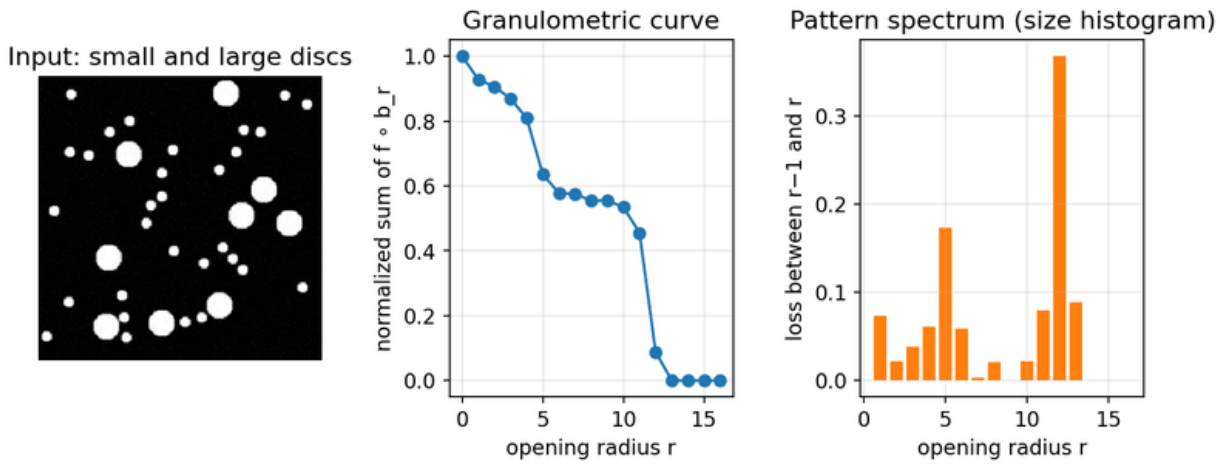


Figure 9.7 — Granulometry of a scene with discs of radius 5 and 12. The granulometric curve drops twice, and the pattern spectrum peaks at the two radii.

```
import numpy as np
from skimage import morphology

def granulometry(f, radii):
    total = f.sum()
    vol = np.array([morphology.opening(f, morphology.disk(r)).sum() / total
                    for r in radii])
    spectrum = -np.diff(vol)                # loss from radius r-1 to r
    return vol, spectrum
```

Textural segmentation

Morphology can also separate regions that differ in the *size* of their texture elements, for example a field of small blobs next to a field of large blobs. The recipe uses the same size logic as granulometry:

1. **Close** the image with an SE larger than the gaps between the small blobs (for bright blobs on a dark background, closing fills those dark gaps). The small-blob region turns into a nearly uniform bright patch, while the large blobs, whose gaps are wider than the SE, stay separate.
2. **Open** the result with an SE larger than the large blobs but smaller than the merged patch. This erases the isolated large blobs and leaves the merged small-blob patch.
3. Take the **morphological gradient** (or a threshold) of the result. The boundary between the two textures appears as a single strong edge.

The exact SE sizes depend on the blob and gap sizes in your data, which you can measure with a granulometry first.

Grayscale reconstruction

All reconstruction definitions carry over, with $\cap$ replaced by the point-wise minimum $\wedge$ and $\cup$ by the maximum $\vee$. Geodesic dilation of a marker image f under a mask image g (with $f \leq g$) is

$$D_g^{(1)}(f) = (f \oplus b) \wedge g,$$

and reconstruction by dilation iterates this to stability. Grayscale reconstruction powers several useful filters [7]:

- **Opening by reconstruction**, $R_f^D(f \ominus nb)$: removes bright details smaller than the SE but restores the exact shape of everything else. A **top-hat by reconstruction**, $f - R_f^D(f \ominus nb)$, is cleaner than the ordinary top-hat because the background estimate does not have rounded "shoulders".
- **Regional maxima and the h-dome transform**: reconstruct f from the marker $f - h$. The reconstruction flattens every peak of height at most h; subtracting it from f keeps only the "domes" (peaks), each at most h tall. Thresholding the domes detects bright blobs regardless of the local background level.

```
from skimage import data, morphology, util
import numpy as np

f = util.img_as_float(data.camera())
h = 0.1
background = morphology.reconstruction(np.clip(f - h, 0, None), f, method="dilation")
domes = f - background                    # bright peaks, each at most h tall
```

Modern view

Morphology is old, but it is far from finished. Five lines of work are worth knowing.

Foundations and tutorials. Matheron's work on random sets [2] and Serra's *Image Analysis and Mathematical Morphology* [3] set up the algebra: erosion and dilation as Minkowski operations, openings as "sieves", and the lattice-theoretic view that later extended everything from sets to gray-level functions. Haralick, Sternberg and Zhuang's 1987 tutorial [4] is still one of the clearest introductions to binary and grayscale morphology for engineers; it defines the operators, proves their basic properties, and connects them to practical algorithms. Soille's *Morphological Image Analysis* [5] is the practitioner's reference: it covers geodesic transforms, reconstruction, watersheds, granulometries, and efficient implementations, and it is the place to look when a library's documentation is too terse.

Reconstruction and efficient algorithms. Vincent's 1993 paper [7] made reconstruction practical. Its main takeaway is that the naive "iterate until stable" loop can be replaced by queue-based algorithms whose cost grows roughly linearly with the image size, which turned reconstruction into an everyday tool for hole filling, regional extrema, and h-domes.

Connected operators and component trees. A filter is **connected** if it can only remove or merge flat zones (connected regions of constant value) and never creates new contours. Openings by reconstruction are the simplest example. Salembier and Wilkinson's review [10] explains the family: area openings, attribute filters, levelings, and tree-based filtering. Breen and Jones introduced **attribute openings and thinnings** [9], which keep or remove a connected component based on an attribute (area, elongation, moment of inertia) rather than whether an SE fits. Efficient implementations represent an image as a **max-tree** or **min-tree** (a tree of the connected components of all threshold sets) and filter by pruning the tree. Carlinet and Géraud compare many max-tree construction algorithms in a single framework and give a decision tree for choosing one [11]. scikit-image exposes these as `area_opening`, `area_closing`, `diameter_opening` and `max_tree`.

Size distributions. Maragos's pattern spectrum [8] formalised granulometry as a multiscale shape descriptor; it is still used for texture and particle-size analysis, and attribute-based granulometries [9] extend it to shape attributes other than size.

Morphology meets deep learning. Deep learning has changed morphology in two directions.

Morphology inside networks. Max pooling is already a flat grayscale dilation followed by subsampling, so the connection is natural. Several groups have built layers whose structuring elements are learned by gradient descent. Mondal et al. study networks built from learnable dilation and

erosion neurons and show that two blocks of dilations and erosions, each followed by a linear combination, can approximate any continuous function [13]. Nogueira et al. propose DeepMorphNet, which replaces convolutions with learnable morphological operations and compares the features it learns with standard convolutional networks [14]. A practical difficulty is that min and max have sparse gradients. Kirszenberg et al. [16] and Hermary et al. [15] build smooth morphological layers from differentiable approximations of max and min (generalised means and soft-max functions whose sharpness is set by a learnable parameter), improving on earlier p-convolution layers, and study when such layers actually recover erosions and dilations. These remain research tools: morphological layers are not a standard component of mainstream vision backbones.

Morphology around networks. In segmentation, morphology is used constantly as **post-processing** of predicted masks: opening to remove false-positive specks, closing or hole filling to repair fragmented objects, `remove_small_objects`, border clearing, and skeletons for measuring tubular structures. It also appears inside **loss functions**. clDice [17] computes a differentiable "soft skeleton" by iterating min- and max-pooling (soft erosion and dilation), and uses overlap between the predicted and true skeletons as a loss that encourages topologically correct segmentation of vessels, roads and neurons.

What has *not* changed: for cleaning, measuring and correcting binary or grayscale images, the classical operators of this chapter are still the fastest, most predictable tools, and they come with guarantees (idempotence, connectivity, exact shape preservation) that learned filters do not offer.

Key takeaways

- Morphology works with **shape**, not frequency. It uses only min, max, union and intersection, controlled by a structuring element whose size and shape encode the question you ask.
- **Erosion** keeps positions where the SE fits; **dilation** keeps positions where the reflected SE touches. They are duals: eroding the foreground equals dilating the background.
- **Opening** (erode then dilate) removes small bright parts; **closing** (dilate then erode) fills small dark gaps. Both are idempotent filters, and opening followed by closing is a robust noise cleaner.
- The **hit-or-miss transform** matches an exact foreground/background pattern and underlies thinning, thickening, and convex hulls.
- **Reconstruction** (marker plus mask, grow until stable) keeps whole objects exactly: opening by reconstruction, automatic hole filling and border clearing all follow from it.
- In grayscale, erosion and dilation are local min and max. The **gradient**, **top-hat** and **bottom-hat** give edges, background correction, and small-object extraction; **granulometry** gives size distributions without segmentation.
- Today morphology lives on as post-processing for neural-network masks, as differentiable building blocks in losses such as clDice, and as a research topic in learnable morphological layers.

Exercises

1. A binary image contains a rectangle 40 pixels wide and 10 pixels tall. Describe its erosion and its opening by (a) a 5×5 square, (b) a horizontal line 15 pixels long, (c) a vertical line 15 pixels long. Which SE removes the rectangle entirely, and why?

   Hint

   Erosion keeps the positions of the SE's origin where the whole SE fits. The 5×5 square fits at a 36×6 block of positions, and the opening restores the full rectangle, because the rectangle is a union of fitting squares. The horizontal line also fits, so its opening is again the full rectangle. The vertical line is taller than the rectangle and never fits: both its erosion and its opening are empty.

2. Show that dilation is commutative and associative for sets: $A \oplus B = B \oplus A$ and $(A \oplus B) \oplus C = A \oplus (B \oplus C)$. Use the second identity to explain why dilating twice with a 3×3 square equals dilating once with a 5×5 square. Is the same true for erosion?

   Hint

   Write dilation as the Minkowski sum $\{a + b \mid a \in A, b \in B\}$; both identities follow from the commutativity and associativity of vector addition. A 3×3 square plus itself is a 5×5 square. For erosion, use $(A \ominus B) \ominus C = A \ominus (B \oplus C)$, which follows from duality.

3. Design the hit-or-miss template pair (B_1, B_2) that detects the **end points** of a one-pixel-wide 8-connected line pointing to the right (the line arrives from the left). How many rotated templates do you need to catch all end points? Test your answer with `scipy.ndimage.binary_hit_or_miss` on `skimage.morphology.skeletonize(~skimage.data.horse())`.

   Hint

   Centre in B_1 plus the left neighbour in B_1; the other seven neighbours in B_2. A line can also arrive diagonally, so you need 8 templates (4 axis directions and 4 diagonals). Compare your total count with the count from a neighbour-counting function like `endpoints()` in the pruning snippet.

4. Your segmentation network outputs masks of cells. Many masks have small holes, a few single-pixel false positives, and some cells are cut by the image border. Write a five-line post-processing pipeline in scikit-image and justify the order of the steps.

   Hint

   One reasonable order: `remove_small_objects` (or a small opening) first, so that specks do not get filled or merged; then `binary_fill_holes`; then a small closing if contours are ragged; then `clear_border`; finally `label` to count. Discuss what would go wrong if you closed before removing specks (nearby specks can merge into a "cell").

5. For a grayscale image f and a flat SE b, prove that the top-hat $T(f) = f - (f \circ b)$ is non-negative and that it is idempotent: $T(T(f)) = T(f)$.

Hint

Non-negativity follows from $f \circ b \le f$. For idempotence, show that $T(f) \circ b = 0$. Take any translate W of b and let y be the point of W where f is smallest. Since the opening is at least $\min_W f$ everywhere on W and at most f, it equals $f(y)$ at y, so $T(f)(y) = 0$. Every translate of b therefore contains a zero of $T(f)$, the erosion of $T(f)$ by b is zero, and so is its opening.

References

1. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018, Ch. 9. publisher page
2. G. Matheron, *Random Sets and Integral Geometry*, Wiley, 1975. library record
3. J. Serra, *Image Analysis and Mathematical Morphology*, Academic Press, 1982. library record
4. R. M. Haralick, S. R. Sternberg, and X. Zhuang, "Image Analysis Using Mathematical Morphology," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. PAMI-9, pp. 532–550, 1987. doi
5. P. Soille, *Morphological Image Analysis*, Springer, 2004. doi
6. T. Y. Zhang and C. Y. Suen, "A Fast Parallel Algorithm for Thinning Digital Patterns," *Communications of the ACM*, vol. 27, pp. 236–239, 1984. doi
7. L. Vincent, "Morphological Grayscale Reconstruction in Image Analysis: Applications and Efficient Algorithms," *IEEE Trans. Image Processing*, vol. 2, pp. 176–201, 1993. doi
8. P. Maragos, "Pattern Spectrum and Multiscale Shape Representation," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 11, no. 7, pp. 701–716, 1989. doi
9. E. J. Breen and R. Jones, "Attribute Openings, Thinnings, and Granulometries," *Computer Vision and Image Understanding*, vol. 64, no. 3, pp. 377–389, 1996. doi
10. P. Salembier and M. H. F. Wilkinson, "Connected Operators: A Review of Region-Based Morphological Image Processing Techniques," *IEEE Signal Processing Magazine*, vol. 26, no. 6, pp. 136–157, 2009. doi
11. E. Carlinet and T. Géraud, "A Comparative Review of Component Tree Computation Algorithms," *IEEE Trans. Image Processing*, vol. 23, pp. 3885–3895, 2014. doi
12. S. van der Walt et al., "scikit-image: Image Processing in Python," *PeerJ*, 2014. arXiv
13. R. Mondal, S. Santra, S. S. Mukherjee, and B. Chanda, "Morphological Network: How Far Can We Go with Morphological Neurons?," BMVC, 2022. arXiv
14. K. Nogueira, J. Chanussot, M. Dalla Mura, and J. A. dos Santos, "An Introduction to Deep Morphological Networks," arXiv:1906.01751, 2019. arXiv
15. R. Hermary, G. Tochon, É. Puybareau, A. Kirszenberg, and J. Angulo, "Learning Grayscale Mathematical Morphology with Smooth Morphological Layers," *Journal of Mathematical Imaging and Vision*, vol. 64, pp. 736–753, 2022. doi
16. A. Kirszenberg, G. Tochon, É. Puybareau, and J. Angulo, "Going Beyond p-convolutions to Learn Grayscale Morphological Operators," arXiv:2102.10038, 2021. arXiv

17. S. Shit et al., “clDice — A Novel Topology-Preserving Loss Function for Tubular Structure Segmentation,” CVPR, 2021. arXiv

IP 10 • Image Segmentation I: Edge Detection, Thresholding, and Region Detection

Split an image into meaningful parts: find edges with gradients, LoG and Canny, link them with the Hough transform, pick thresholds with Otsu, and grow, cluster, cut and flood regions.

Online edition: https://blog.shyandram.dev/en/learn/image-processing/dip-10-segmentation/

What you'll learn

- The formal definition of segmentation, and its two families: **discontinuity** (edges) and **similarity** (regions).
- How derivatives find points, lines and edges, and how Sobel, Marr–Hildreth (LoG) and Canny work step by step.
- How to link edge pixels into boundaries, including with the Hough transform.
- How Otsu's method picks a threshold, and what to do when one threshold is not enough.
- How region growing, split-and-merge, k-means, SLIC, normalized cuts, watersheds and motion turn pixels into regions.

The big picture

Until now, most operations took an image and returned an image. Segmentation returns a **map of parts**: each pixel gets a label saying which object or region it belongs to. Measuring objects, counting cells and reading text all depend on that map.

> **Explain like I'm five**
>
> Imagine a colouring page that has lost its black outlines. You can find the parts in two ways. Look for places where the colour suddenly changes and draw a line there: that is **edge detection**. Or start at one spot and keep spreading the same crayon while the paper looks the same: that is **region growing**. This chapter is about both tricks and how to make them work on messy real photos.

None of these methods learns from labelled examples. They need no training data, are easy to explain, and their ideas (gradients, hysteresis, between-class variance, graph cuts, flooding) live on inside modern deep pipelines.

Fundamentals

In plain words: segmentation cuts the image into pieces that do not overlap, cover everything, and each look "the same" inside.

Let R be the whole spatial region an image occupies. Segmentation partitions R into n subregions $R_1, \dots, R_n$ such that

(a) $\bigcup_{i=1}^{n} R_i = R$, (b) R_i is connected,
(c) $R_i \cap R_j = \emptyset$ for $i \neq j$, (d) $Q(R_i) = \text{TRUE}$, (e) $Q(R_i \cup R_j) = \text{FALSE}$ for adjacent R_i, R_j.

Here Q is a **logical predicate**: a yes/no test on a set of pixels, for example "all intensities lie within 10 grey levels of each other". So: every pixel is labelled (a), each region is one connected piece (b), no pixel has two labels (c), each region passes the test (d), and touching regions would fail it if merged (e).

Almost all monochrome segmentation algorithms rest on one of two properties of intensity [1]:

- **Discontinuity**: boundaries are where intensity changes abruptly. Point, line and edge detection use this.
- **Similarity**: pixels in one region share a property (intensity, colour, texture). Thresholding, region growing, clustering, graph cuts and watersheds use this.

Edges are precise but often broken; regions are closed by construction but their borders can wander.

Point, line, and edge detection

In plain words: an edge is where brightness changes fast, and "how fast something changes" is exactly what a derivative measures.

Background: derivatives on a pixel grid

For a 1-D digital function $f(x)$ the simplest approximations are

$$\frac{\partial f}{\partial x} \approx f(x+1) - f(x), \qquad \frac{\partial^2 f}{\partial x^2} \approx f(x+1) + f(x-1) - 2f(x).$$

Here x is an integer pixel position and $f(x)$ its intensity. Across a ramp, the first derivative is nonzero all along it, so it gives **thick** responses. The second derivative is nonzero only at the two ends, with opposite signs, giving a **double response** with a **zero crossing** in between, and it reacts far more strongly to fine detail and noise. These facts explain every detector below.

Detecting isolated points

A point is a tiny blob that differs from its surroundings. The Laplacian

$$\nabla^2 f(x,y) = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

is implemented with a 3×3 kernel whose centre is -8 and whose eight neighbours are 1 (the version that includes diagonals). With $Z(x, y)$ the filtered image, we mark a point wherever $|Z(x, y)| > T$ for a nonnegative threshold T. The kernel's coefficients sum to zero, so flat regions give no response.

Detecting lines

A line is a structure one or a few pixels wide. The Laplacian responds to lines too, but with a double response and no sense of direction. For a specific angle, use a kernel whose 2s run along that angle and whose other entries are -1 (the horizontal one has $2, 2, 2$ in its middle row). Comparing the responses of the horizontal, $+45°$, vertical and $-45°$ kernels tells which orientation dominates at each pixel.

Edge models

Real edges come in three idealized shapes:

- a **step edge** jumps from one level to another across one pixel (it appears mainly in synthetic images);
- a **ramp edge** changes linearly over several pixels, which is what blur from optics and sampling produces in real images;
- a **roof edge** rises and falls back, like a thin line seen through a blur.

On a ramp, the first-derivative magnitude says an edge is present, the sign of the second derivative says which side is bright, and its zero crossing marks the ramp's centre. But noise too faint to see ruins the second derivative, so **smoothing before differentiating** is part of every serious edge detector.

Basic edge detection with the gradient

The **gradient** of f at (x, y) is the vector

$$\nabla f = \begin{bmatrix} g_x \\ g_y \end{bmatrix} = \begin{bmatrix} \partial f/\partial x \\ \partial f/\partial y \end{bmatrix}, \qquad M(x, y) = \sqrt{g_x^2 + g_y^2}, \qquad \alpha(x, y) = \operatorname{atan2}(g_y, g_x).$$

g_x and g_y are the partial derivatives, M is the **gradient magnitude** (edge strength) and α the **gradient direction** (where intensity rises fastest); the edge runs perpendicular to α. M is often approximated by the cheaper $|g_x| + |g_y|$.

The derivatives are computed with small kernels. The classic operators are summarized in [1]. The **Roberts** cross operators use 2×2 diagonal differences. The **Prewitt** operators use 3×3 kernels with rows $[-1, 0, 1]$. The **Sobel** operators weight the centre row by 2, for example

$$K_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix},$$

a difference in one direction times a $[1, 2, 1]$ smoothing in the other. That built-in smoothing is why Sobel is usually preferred.

```
import cv2
import numpy as np
from skimage import data

f = data.camera().astype(np.float32) / 255.0
gx = cv2.Sobel(f, cv2.CV_32F, 1, 0, ksize=3)   # d/dx (columns)
gy = cv2.Sobel(f, cv2.CV_32F, 0, 1, ksize=3)   # d/dy (rows)
M = np.hypot(gx, gy)                           # gradient magnitude
alpha = np.arctan2(gy, gx)                     # gradient direction, radians
edges = M > 0.3 * M.max()                      # crude thresholded edge map
```

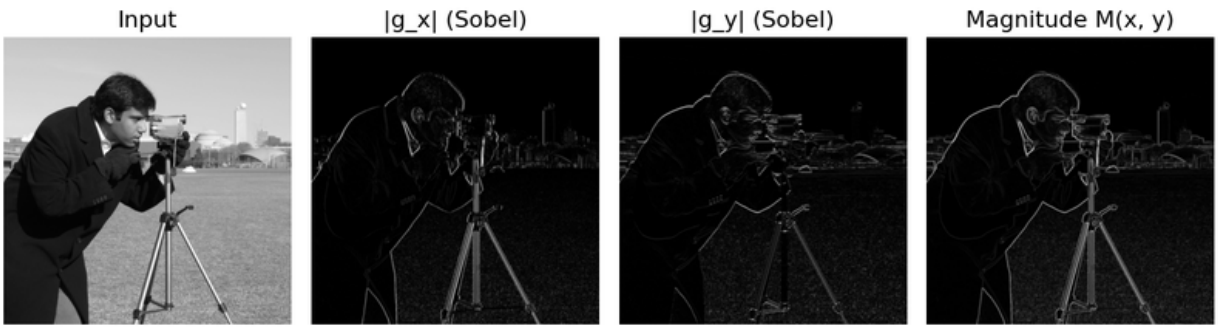


Figure 10.1 — Sobel derivatives. |g_x| lights up vertical structures such as the tripod legs, |g_y| lights up horizontal ones such as the horizon, and the magnitude M combines both. Note how texture in the grass also responds.

Try the Sobel kernel yourself. Swap it to the y version, or change the 2s to 1s to get Prewitt, and watch the response change.

> **Interactive demonstration (online)**: open the online edition of this tutorial to use this demonstration.Open demonstration

Thresholding M gives a poor edge map: thick edges, lost weak edges, and clutter from texture. The next two detectors build smoothing and thinning into the design.

The Marr–Hildreth edge detector

Marr and Hildreth [2] argued that intensity changes occur at many scales, so a detector needs an adjustable size, and that second-derivative zero crossings mark edge centres. Their operator is the **Laplacian of a Gaussian (LoG)**, $\nabla^2 G$, where

$$G(x,y) = e^{-\frac{x^2+y^2}{2\sigma^2}}, \qquad \nabla^2 G(x,y) = \frac{x^2+y^2-2\sigma^2}{\sigma^4}\, e^{-\frac{x^2+y^2}{2\sigma^2}}.$$

σ is the Gaussian's standard deviation and sets the scale (larger σ, only coarser edges survive); the normalizing constant is dropped since only zero crossings matter. Its shape earns $\nabla^2 G$ the name **Mexican hat**. The algorithm:

1. Smooth with an $n \times n$ Gaussian, n the smallest odd integer $\geq 6\sigma$.
2. Take the Laplacian of the result (by linearity, steps 1–2 are one convolution with $\nabla^2 G$).
3. Mark **zero crossings**: pixels where some pair of opposite neighbours has different signs and an absolute difference above a threshold; without the threshold, every ripple in a flat area becomes an "edge".

A **difference of Gaussians (DoG)**, $G_{\sigma_1} - G_{\sigma_2}$ with $\sigma_1/\sigma_2 \approx 1.6$, approximates the LoG [1]. Zero crossings form closed contours, but they produce "spaghetti" in texture and round off corners.

The Canny edge detector

Canny [3] turned edge detection into an optimization problem with three criteria:

1. **Low error rate**: find all true edges and no spurious ones.
2. **Good localization**: the detected edge should be as close as possible to the true edge.
3. **Single response**: one true edge should produce one detected edge, not a double line.

For a 1-D step in white noise, the optimal filter is well approximated by the first derivative of a Gaussian. The practical 2-D algorithm follows:

1. **Smooth** the image with a Gaussian of standard deviation σ: $f_s = G_\sigma \star f$.
2. **Compute the gradient** magnitude M and angle α of f_s.
3. **Non-maximum suppression.** Quantize α into four directions. Zero any pixel whose M is smaller than either neighbour along the gradient direction. Ridges become one pixel thick.
4. **Double thresholding.** Pick T_H and T_L with T_H/T_L about 2 to 3 [1]. Pixels above T_H are **strong**; pixels between are **weak**.
5. **Hysteresis.** Keep a weak pixel only if it connects, through weak pixels, to a strong one.

Hysteresis is the key idea. One threshold either breaks contours or admits noise; two thresholds plus connectivity keep faint contours anchored to strong ones and drop isolated weak responses.

```
import cv2
from scipy import ndimage as ndi
from skimage import data, feature

f8 = data.camera()
canny_cv = cv2.Canny(cv2.GaussianBlur(f8, (0, 0), 2), 30, 90)      # uint8 0/255
canny_sk = feature.canny(f8 / 255.0, sigma=2, low_threshold=0.05, high_threshold=0.15)
log = ndi.gaussian_laplace(f8 / 255.0, sigma=3)                    # Marr-Hildreth steps 1-2
```

`cv2.Canny` has no smoothing parameter, so we blur first; scikit-image takes `sigma` directly.

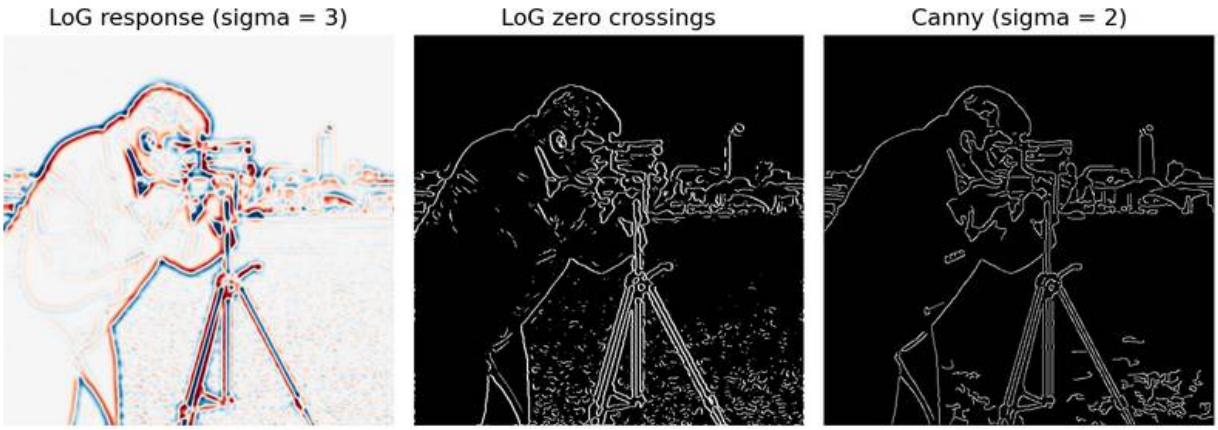


Figure 10.2 — Marr–Hildreth versus Canny on the same image. The LoG response (left, red positive, blue negative) changes sign at every edge. Its thresholded zero crossings (middle) are closed but cluttered in the grass. Canny (right) gives thin, mostly continuous edges with far less clutter.

Drag the slider to see which structures Canny keeps (edges are shown dark on white).

Input

Canny σ=2

Cameraman image compared with its Canny edge map

Linking edge points

Even Canny leaves gaps. **Edge linking** assembles edge pixels into meaningful boundaries.

Local processing. In a small neighbourhood S_{xy} of each edge pixel (x, y), link a neighbour (s, t) if both magnitude and angle are similar:

$$|M(s,t) - M(x,y)| \le E, \qquad |\alpha(s,t) - \alpha(x,y)| \le A,$$

where E and A are positive tolerances. A cheaper variant thresholds M, keeps pixels whose angle is near a desired direction, and fills short gaps along each row; rotating the image handles other directions.

Global processing with the Hough transform. When the shape is known, let every edge pixel vote for all shapes that could pass through it. For lines, Duda and Hart [4] proposed the **normal representation**

$$\rho = x \cos\theta + y \sin\theta,$$

where θ is the angle of the line's normal and ρ its signed distance from the origin; unlike slope–intercept, this stays bounded for vertical lines. Each point (x_k, y_k) becomes a sinusoid in the $\rho\theta$ plane, and the sinusoids of collinear points meet in one cell. The algorithm:

1. Compute a binary edge map (e.g. Canny).
2. Quantize the $\rho\theta$ plane into **accumulator cells** $A(\rho, \theta)$, all zero.
3. For each edge pixel and each θ, compute ρ and increment $A(\rho, \theta)$.
4. Local maxima with large counts are lines; optionally check the continuity of their voters to get segments.

```
# continues from the Canny snippet above (uses cv2, np and canny_cv)
import numpy as np

lines = cv2.HoughLines(canny_cv, rho=1, theta=np.pi / 180, threshold=120)
for rho, theta in lines[:5, 0]:
    print(f"rho = {rho:6.1f} px, theta = {np.degrees(theta):5.1f} deg")
```

OpenCV reports $\theta \in [0°, 180°)$ with signed ρ, the same convention over a different range. Voting extends to circles and other shapes with more parameters.

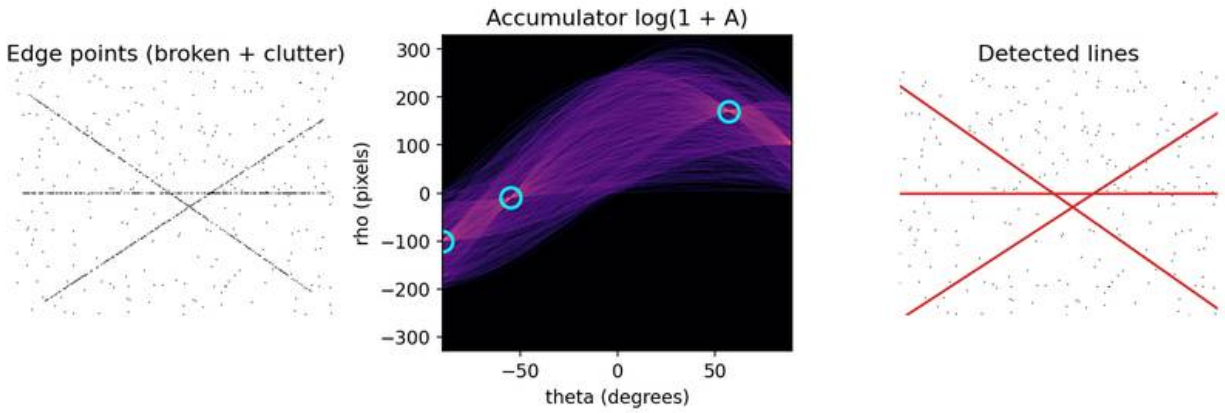


Figure 10.3 — The Hough transform. Three lines with 35% of their pixels missing, plus random clutter (left). Every point votes along a sinusoid; the three circled peaks in the accumulator (middle) are where many sinusoids meet. Converting the peaks back to lines recovers all three (right) despite the gaps.

Thresholding

In plain words: pick a grey level; everything brighter is "object" and everything darker is "background". The art is in picking the level.

Foundation

Given an intensity threshold T, global thresholding produces

$$g(x, y) = \begin{cases} 1 & \text{if } f(x, y) > T \\ 0 & \text{otherwise.} \end{cases}$$

A constant T is a **global** threshold; one that changes with position is **variable** (local, adaptive); several thresholds give **multiple thresholding**. It works when the histogram has well separated modes. **Noise** widens the modes, **uneven illumination and reflectance** shift them, and a tiny object makes a bump that is easy to miss.

Basic global thresholding

An easy iterative rule works well when the modes are clearly separated:

1. Pick an initial T, such as the mean intensity.
2. Split the pixels into G_1 (values $> T$) and G_2 (values $\leq T$).
3. Compute the mean of each group, m_1 and m_2.
4. Set $T = \frac{1}{2}(m_1 + m_2)$.
5. Repeat 2–4 until T changes by less than a small ΔT.

```
def basic_global(img, dT=0.5):
    T = img.mean()
    while True:
        m1, m2 = img[img > T].mean(), img[img <= T].mean()
        T_new = 0.5 * (m1 + m2)
        if abs(T_new - T) < dT:
            return T_new
        T = T_new
```

Otsu's optimum global thresholding

Otsu [5] picks the threshold that makes the two classes most **separable**, measured by the **between-class variance**. It needs only the histogram.

Let the image have L levels, n_i pixels at level i and MN pixels in total, so the normalized histogram is $p_i = n_i/MN$. A threshold k splits the levels into $C_1 = [0, k]$ and $C_2 = [k+1, L-1]$. Define

$$P_1(k) = \sum_{i=0}^{k} p_i, \qquad m(k) = \sum_{i=0}^{k} i\, p_i, \qquad m_G = \sum_{i=0}^{L-1} i\, p_i,$$

where $P_1(k)$ is the probability that a pixel falls in C_1, $m(k)$ is the cumulative mean up to level k, and m_G is the global mean. Then $P_2 = 1 - P_1$, and the class means are $m_1 = m/P_1$ and $m_2 = (m_G - m)/P_2$.

Derivation. The global mean is $m_G = P_1 m_1 + P_2 m_2$. The between-class variance is the weighted spread of the class means around it:

$$\sigma_B^2 = P_1(m_1 - m_G)^2 + P_2(m_2 - m_G)^2.$$

Substitute $m_G = P_1 m_1 + P_2 m_2$. Then $m_1 - m_G = P_2(m_1 - m_2)$ and $m_2 - m_G = P_1(m_2 - m_1)$, so

$$\sigma_B^2 = P_1 P_2^2 (m_1 - m_2)^2 + P_2 P_1^2 (m_1 - m_2)^2 = P_1 P_2 (m_1 - m_2)^2.$$

The farther apart the class means, the larger σ_B^2. Substituting $m_1 = m/P_1$ and $m_2 = (m_G - m)/(1 - P_1)$ gives a form that needs only cumulative sums:

$$\sigma_B^2(k) = \frac{\left(m_G P_1(k) - m(k)\right)^2}{P_1(k)\left(1 - P_1(k)\right)}.$$

The optimum is $k^* = \arg\max_k \sigma_B^2(k)$ (average ties). Since the global variance σ_G^2 is the sum of between- and within-class variance and does not depend on k, this also **minimizes the within-class variance**. The ratio

$$\eta = \frac{\sigma_B^2(k^*)}{\sigma_G^2}, \qquad 0 \le \eta \le 1,$$

measures **separability**: near 1 for two clean modes, lower when they overlap.

```
import numpy as np
from skimage import data, filters

def otsu(img):
    p = np.bincount(img.ravel(), minlength=256) / img.size   # normalized histogram
    P1 = np.cumsum(p)                                         # class-1 probability
    m = np.cumsum(np.arange(256) * p)                         # cumulative mean
    mG = m[-1]                                                # global mean
    with np.errstate(divide="ignore", invalid="ignore"):
        sB2 = (mG * P1 - m) ** 2 / (P1 * (1 - P1))
    sB2 = np.nan_to_num(sB2)
    k = int(np.argmax(sB2))
    return k, sB2[k] / np.var(img)                            # threshold, separability

coins = data.coins()
k, eta = otsu(coins)          # k = 107, eta ≈ 0.76
assert k == filters.threshold_otsu(coins)
```

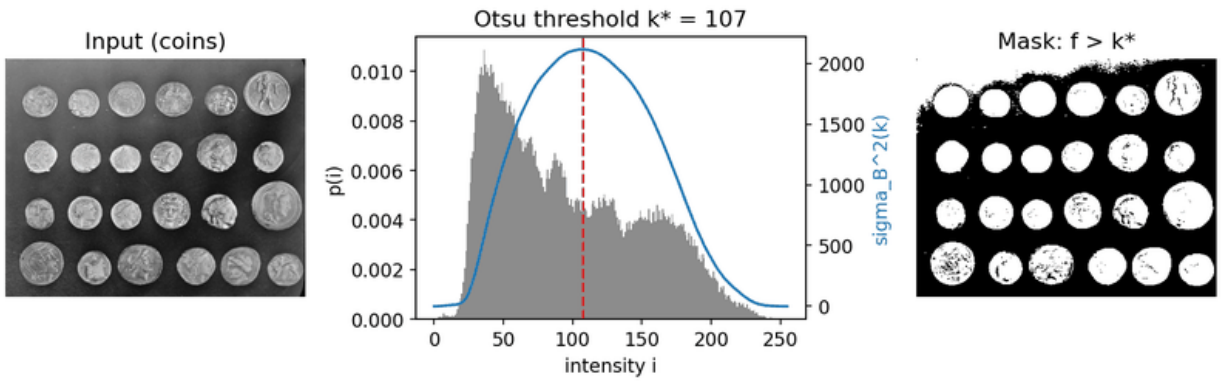


Figure 10.4 — Otsu's method on the coins image. The blue curve is σ_B²(k); its maximum gives k = 107 (dashed line). The mask (right) captures the coins, but the bright band along the top edge also passes, a reminder that one global threshold cannot correct uneven illumination.*

Using smoothing and edges to improve thresholding

- **Smooth first.** Noise widens the modes until they merge; smoothing narrows them again, as long as the object is large compared with the filter.
- **Use only pixels near edges.** A small object's mode is buried under the background's. Keep pixels where an edge indicator (gradient magnitude or $|\nabla^2 f|$) is high, say above its 99.7th percentile, and compute the histogram **from those pixels only**. They lie roughly half on each side of a boundary, so the histogram becomes balanced and bimodal. Apply the resulting threshold to the whole image.

Multiple thresholds

Otsu's criterion generalizes to K classes separated by $K-1$ thresholds:

$$\sigma_B^2 = \sum_{k=1}^{K} P_k (m_k - m_G)^2,$$

with P_k and m_k the probability and mean of class k, found by exhaustive search. `skimage.filters.threshold_multiotsu(coins, classes=3)` returns `[77, 139]`.

Variable thresholding

When illumination varies, no single T works. Three remedies:

1. **Partitioning.** Threshold each tile with its own Otsu value; each tile must contain both classes.
2. **Local properties.** With m_{xy} and σ_{xy} the mean and standard deviation in a window around (x, y), use $T_{xy} = a\,\sigma_{xy} + b\,m_{xy}$ $(a, b \geq 0)$, or more generally a predicate $Q(\sigma_{xy}, m_{xy})$. Sauvola and Pietikäinen's document rule [7], $T_{xy} = m_{xy}(1 + k(\sigma_{xy}/R - 1))$ with R the dynamic range of σ and small $k > 0$, belongs to this family.
3. **Moving averages.** For text, scan in a zigzag and threshold each pixel at $c \cdot m(k)$, where $m(k)$ is the mean of the last n pixels and c is slightly below 1.

```
from skimage import data, filters

page = data.page()
global_mask = page > filters.threshold_otsu(page)
local_T = filters.threshold_local(page, block_size=35, offset=10)  # Gaussian-weighted local
↪  mean - 10
local_mask = page > local_T
sauvola_mask = page > filters.threshold_sauvola(page, window_size=25, k=0.2)
```

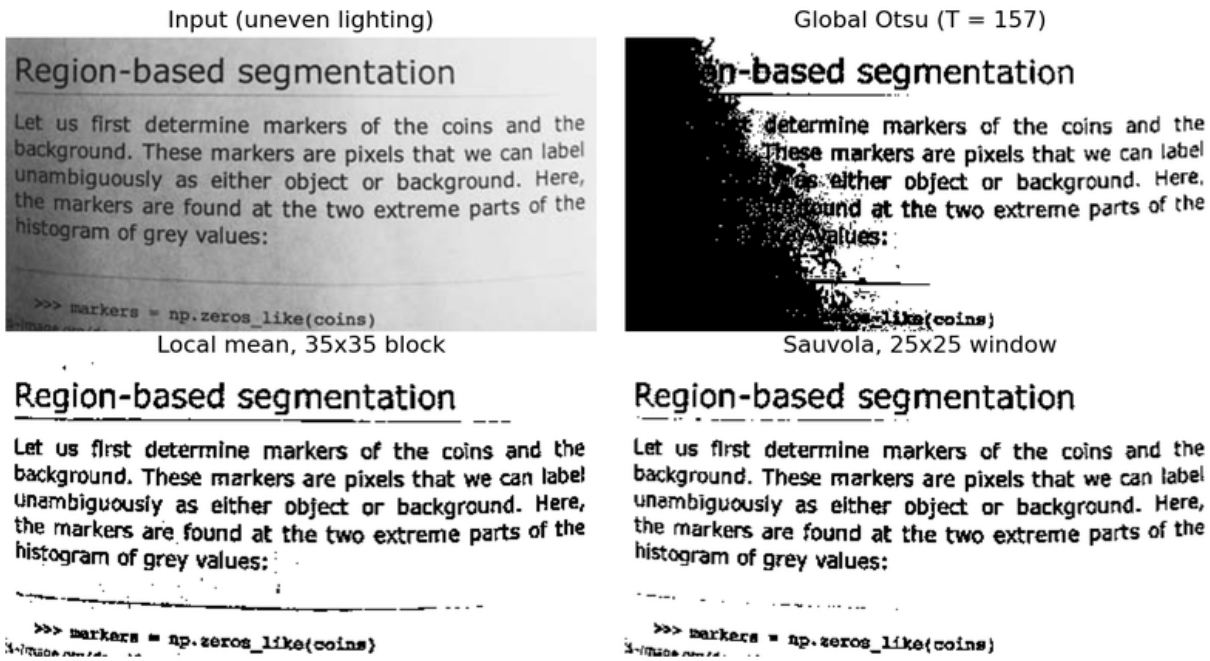


Figure 10.5 — Variable thresholding on unevenly lit text. A global Otsu threshold (top right) blacks out the shadowed left side. A local mean threshold (bottom left) and Sauvola's rule (bottom right) recover the text everywhere.

> **Interactive demonstration (online)**: open the online edition of this tutorial to use this demonstration.Open demonstration

Region growing, splitting, and merging

In plain words: start from a few "seed" pixels and keep adding neighbours that look like them; or start from the whole image and keep cutting it into four until every piece is uniform.

Region growing

Region growing adds to each **seed** the neighbouring pixels that satisfy a similarity predicate. Choose the **seeds** (by hand, by a safe threshold, or from histogram peaks), the **predicate** (e.g. $|f(x, y) - f(\text{seed})| \leq T$, or a test on region statistics), the **connectivity** (4 or 8), and the **stopping rule** (no neighbour passes; size or shape limits make it more robust).

```
from collections import deque
import numpy as np

def region_grow(img, seed, tol):
```

```
    """Grow from `seed` over 8-connected pixels whose value is within `tol` of the seed
     ↪  value."""
    H, W = img.shape
    ref = float(img[seed])
    mask = np.zeros((H, W), bool)
    mask[seed] = True
    q = deque([seed])
    while q:
        r, c = q.popleft()
        for dr in (-1, 0, 1):
            for dc in (-1, 0, 1):
                rr, cc = r + dr, c + dc
                if 0 <= rr < H and 0 <= cc < W and not mask[rr, cc] \
                        and abs(float(img[rr, cc]) - ref) <= tol:
                    mask[rr, cc] = True
                    q.append((rr, cc))
    return mask
```

`skimage.segmentation.flood(img, seed, tolerance=tol)` does the same in compiled code.

Region splitting and merging with quadtrees

Instead of seeds, start with the whole image R and a predicate Q:

1. **Split** any region R_i for which $Q(R_i) = \text{FALSE}$ into four equal quadrants.
2. Repeat until every region passes Q, or the region reaches a minimum size.
3. **Merge** any two adjacent regions R_j, R_k for which $Q(R_j \cup R_k) = \text{TRUE}$.
4. Stop when no more splits or merges are possible.

Splitting is stored as a **quadtree**: the root is the image, each node has four children. Splitting alone gives blocky regions; merging lets regions cross quadrant borders. A typical predicate: "standard deviation above a and mean between b and c".

Region segmentation using clustering and superpixels

In plain words: treat every pixel as a point described by a few numbers (its colour, maybe its position) and gather nearby points into groups.

k-means clustering

Represent each pixel by a feature vector $\mathbf{z} \in \mathbb{R}^n$, for example its Lab colour ($n = 3$). **k-means** looks for k cluster centres $\mathbf{m}_1, \dots, \mathbf{m}_k$ and sets $C_1, \dots, C_k$ that minimize

$$J = \sum_{i=1}^{k} \sum_{\mathbf{z} \in C_i} \|\mathbf{z} - \mathbf{m}_i\|^2,$$

the total squared distance to the cluster means. The global minimum is NP-hard to find, so the standard algorithm alternates: assign each vector to its nearest centre, then move each centre to the mean of its vectors. It reaches a local minimum, so run it from several starts. Clustering on colour alone ignores position, so one cluster can be scattered over the image.

Superpixels and SLIC

A **superpixel** is a small, compact group of similar connected pixels. A few hundred superpixels instead of 10^5 pixels shrink later problems without moving important boundaries. **SLIC** by Achanta et al. [8] is k-means in the 5-D space $[l, a, b, x, y]$. With N pixels and K superpixels, the grid spacing is $S = \sqrt{N/K}$.

1. Place K centres on a grid of spacing S, each nudged to the lowest gradient in its 3×3 neighbourhood.
2. Search **only a** $2S \times 2S$ **window** around each centre and assign pixels by

$$D = \sqrt{d_c^2 + \left(\frac{d_s}{S}\right)^2 m^2},$$

where d_c and d_s are Euclidean distances in Lab colour and in (x, y), and m is the **compactness**: large m gives square-ish superpixels, small m hugs colour edges. 3. Move each centre to the mean of its pixels; repeat about ten times. 4. **Enforce connectivity** by merging stray fragments into a neighbour.

The local search makes each iteration $O(N)$, independent of K.

```
from skimage import data, segmentation

img = data.astronaut()
sp = segmentation.slic(img, n_segments=300, compactness=10, start_label=1)
```

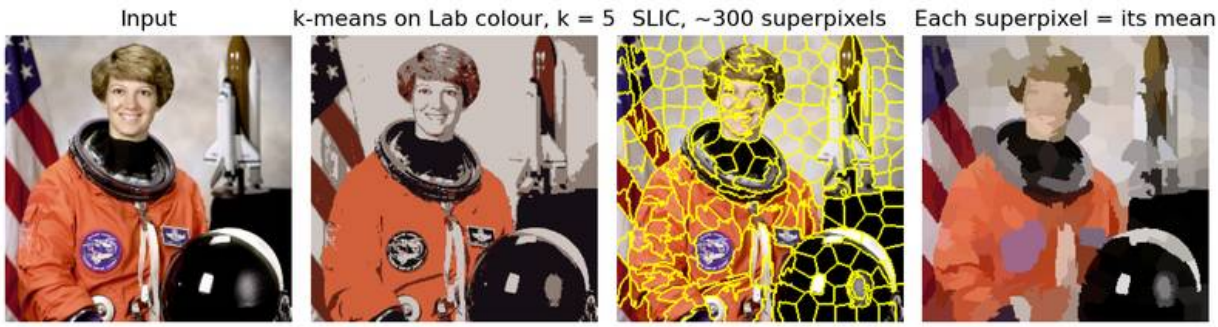


Figure 10.6 — Clustering. k-means on Lab colour (second panel) reduces the image to five colours, but each cluster is scattered across the picture. SLIC (third panel, compactness 10) produces about 300 compact superpixels that follow strong edges; painting each one with its mean colour (right) keeps the image readable at a tiny fraction of the pixel count.

Region segmentation using graph cuts

In plain words: draw the image as a net in which neighbouring pixels are tied together by strings, strong strings for similar pixels and weak strings for different ones. Segmenting is cutting the net into pieces while snapping only weak strings.

Images as graphs

A **graph** $G = (V, E)$ has nodes V and edges E. Each pixel (or superpixel) is a node; nearby nodes are joined by an edge with a nonnegative similarity weight $w(i, j)$, for example

$$w(i,j) = \exp\!\left(-\frac{\|\mathbf{F}_i - \mathbf{F}_j\|^2}{\sigma_I^2}\right)\exp\!\left(-\frac{\|\mathbf{X}_i - \mathbf{X}_j\|^2}{\sigma_X^2}\right) \quad \text{if } \|\mathbf{X}_i - \mathbf{X}_j\| \le r, \text{ else } 0,$$

where $\mathbf{F}$ is a feature vector (intensity, colour), $\mathbf{X}$ a spatial position, σ_I and σ_X scale parameters, and r a radius that keeps the graph sparse.

Minimum graph cuts

A **cut** splits V into two disjoint sets A and B by removing the edges between them. Its cost is

$$\operatorname{cut}(A, B) = \sum_{u \in A,\, v \in B} w(u, v).$$

The **minimum cut** removes as little similarity as possible. A popular formulation adds a **source** node (object) and a **sink** node (background), linked to every pixel with weights saying how much it resembles each. A minimum source–sink cut labels every pixel and can be found efficiently with maximum-flow algorithms; it underlies interactive tools where a user scribbles a few object and background pixels.

Normalized cuts

Without source and sink, a plain minimum cut prefers to cut off a single node or tiny group, because few edges are removed. Shi and Malik [9] fixed this with the **normalized cut**

$$\operatorname{Ncut}(A, B) = \frac{\operatorname{cut}(A, B)}{\operatorname{assoc}(A, V)} + \frac{\operatorname{cut}(A, B)}{\operatorname{assoc}(B, V)}, \qquad \operatorname{assoc}(A, V) = \sum_{u \in A,\, t \in V} w(u, t),$$

the total connection from A to all nodes. A tiny A has a tiny assoc, so cutting it off is now expensive. Exact minimization is NP-hard, but a relaxation gives an eigenproblem; with W the weight matrix and D the diagonal matrix of degrees $d_i = \sum_j w(i, j)$,

$$(D - W)\,\mathbf{y} = \lambda D\,\mathbf{y}.$$

The eigenvector of the **second smallest** eigenvalue is a soft partition indicator; thresholding it splits the graph, and recursion gives more segments. Practical code runs it on a superpixel graph.

```
import numpy as np
from skimage import data, segmentation, graph

img = data.coffee()
sp = segmentation.slic(img, n_segments=400, compactness=30, start_label=1)
rag = graph.rag_mean_color(img, sp, mode="similarity")   # edge weight = exp(-|c_i - c_j|^2
 ↪  / sigma)
ncut = graph.cut_normalized(sp, rag, rng=0)              # recursive two-way Ncut
print(sp.max(), "superpixels ->", len(np.unique(ncut)), "regions")
```

Segmentation using morphological watersheds

In plain words: think of the image as a landscape where bright means high. Pour water in from every valley; where water from two valleys is about to meet, build a wall. The walls are the segmentation.

Background: basins and dams

Treat intensity as height. Each **regional minimum** has a **catchment basin**, the points whose water drains to it; **watershed lines** separate basins. We usually flood the **gradient magnitude**, not the image: interiors are low, boundaries are ridges, so each basin is one object.

A **dam** uses Chapter 9's morphology. When two flooded components are about to merge, dilate both with a 3×3 structuring element, restricted to pixels below the current level; pixels reached by both in the same step become dam pixels, set higher than the image maximum. The dams are one pixel thick and connected.

The watershed algorithm

Let $M_1, \ldots, M_R$ be the regional minima of an image g, let min and max be its extreme values, and let $T[n] = \{(s,t) : g(s,t) < n\}$ be the set of pixels below level n. Let $C[n]$ be the union of the flooded parts of all catchment basins at level n. Flooding proceeds for $n = \min+1, \ldots, \max+1$. Start with $C[\min+1] = T[\min+1]$. At each level, take each connected component q of $T[n]$ and compare it with $C[n-1]$:

1. $q \cap C[n-1]$ is empty: a new minimum has appeared; q becomes a new basin.
2. $q \cap C[n-1]$ contains one component of $C[n-1]$: q lies inside one existing basin; add it.
3. $q \cap C[n-1]$ contains two or more components: two basins are meeting; build a dam inside q as above.

Vincent and Soille [10] gave the efficient implementation used today: sort pixels by height, then flood level by level with a FIFO queue ("immersion simulation").

The use of markers

On a raw gradient the watershed **over-segments**: noise and texture create thousands of minima. The cure is **markers**, components known in advance to be inside objects (**internal**) or background (**external**). Flooding starts only from markers, so there are as many regions as markers. Markers come from safe thresholds, smoothed minima, distance-transform peaks, or user clicks.

```
import numpy as np
from scipy import ndimage as ndi
from skimage import data, filters, morphology, segmentation

coins = data.coins()
elevation = filters.sobel(coins.astype(float))
markers = np.zeros_like(coins, dtype=np.int32)
markers[coins < 30] = 1        # sure background
markers[coins > 150] = 2       # sure object
ws = segmentation.watershed(elevation, markers)
coins_mask = morphology.remove_small_objects(ndi.binary_fill_holes(ws == 2), 100)
labels, n = ndi.label(coins_mask)   # n = 24 coins
```

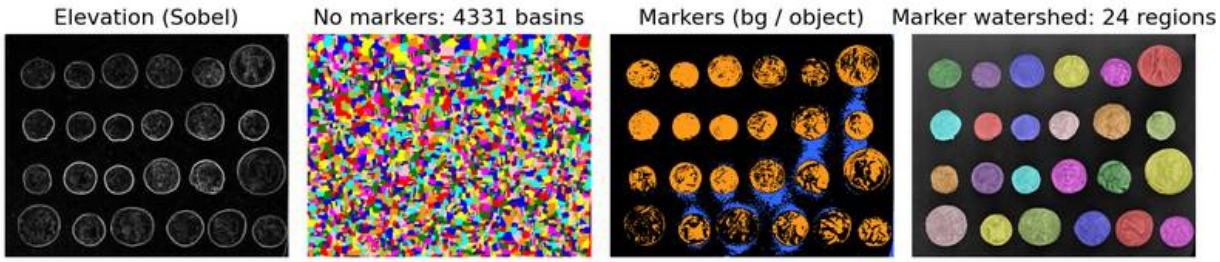


Figure 10.7 — Watershed on the coins image. Flooding the smoothed gradient from every regional minimum yields thousands of basins (second panel). With two marker labels from very safe thresholds (third panel), the floods meet exactly on the coin rims; filling holes and dropping specks leaves one region per coin (right).

The use of motion in segmentation

In plain words: if the camera is still, the things that moved between two frames are the things you care about. Subtract the frames and the moving objects light up.

Spatial techniques: difference images

Given two frames $f(x, y, t_i)$ and $f(x, y, t_j)$ of a static scene with a static camera, the **difference image** is

$$d_{ij}(x, y) = \begin{cases} 1 & \text{if } |f(x, y, t_i) - f(x, y, t_j)| > T \\ 0 & \text{otherwise,} \end{cases}$$

with T just above the noise level. Noise specks are removed by discarding small connected components.

Accumulative differences

One difference shows a moving object twice: where it was and where it is. An **accumulative difference image (ADI)** compares a **reference frame** $R = f(x, y, t_1)$ with each later frame and counts:

$$A_k = A_{k-1} + 1 \text{ if } |R - f_k| > T,$$
$$P_k = P_{k-1} + 1 \text{ if } R - f_k > T,$$
$$N_k = N_{k-1} + 1 \text{ if } R - f_k < -T,$$

each unchanged otherwise and starting at zero ($f_k = f(x, y, t_k)$). For an object brighter than the background, the **positive ADI** P marks where the object was in the reference frame; the **negative ADI** grows in the direction of motion, at a rate set by the speed; the **absolute ADI** A contains both.

```
import numpy as np

def accumulative_differences(frames, T):
    R = frames[0].astype(float)
    A = np.zeros(R.shape, int); P = np.zeros(R.shape, int); N = np.zeros(R.shape, int)
    for f in frames[1:]:
        d = R - f.astype(float)
        A += np.abs(d) > T
        P += d > T
        N += d < -T
    return A, P, N
```

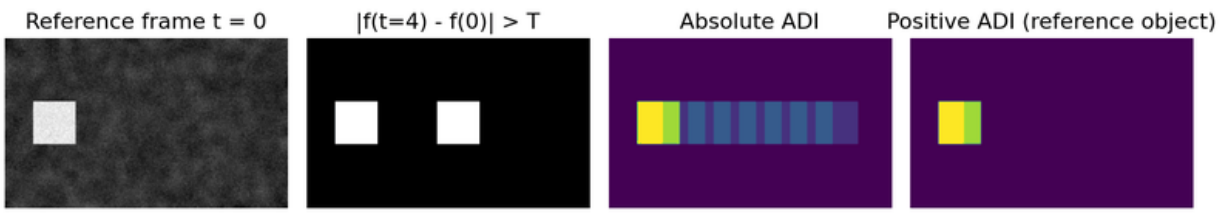


Figure 10.8 — Motion. A bright square moves right over eight frames. One difference image (second panel) shows the square twice. The absolute ADI (third panel) records every position it visited; the positive ADI (right) isolates where it started.

Building a reference image

A reference with no moving objects is rare. Once the positive ADI shows that an object has fully left its initial location, copy those pixels from the current frame into the reference; doing this for

every mover gives a **background reference image**. Background subtraction methods extend this idea with per-pixel statistical models.

Modern view

Five surveys map the field; below each is what deep learning did and did not change.

Classical thresholding. Sezgin and Sankur [6] sort thresholding methods into six families by the information they exploit (histogram shape, measurement-space clustering, entropy, object attributes, spatial correlation, local grey-level surface) and evaluate forty of them on non-destructive testing and document images, singling out those that perform consistently in both. Otsu's method sits in the clustering family.

Deep segmentation. Minaee et al. [11] survey deep semantic and instance segmentation, grouping models into fully convolutional networks, encoder–decoders, multi-scale and pyramid models, recurrent networks, attention models and adversarial generative models, and reviewing datasets and results. The landmark designs echo classical ideas:

- **FCN** [13] made classification networks fully convolutional so they output a label map, fusing coarse deep features with fine shallow ones: learned multi-scale analysis.
- **U-Net** [14] added a symmetric expanding path with skip connections, giving precise boundaries from little training data, and became a default in biomedical imaging.
- **DeepLab** [16] used atrous convolution and atrous spatial pyramid pooling for resolution and context, plus a fully connected CRF to sharpen boundaries.
- **Mask R-CNN** [15] added a mask branch to an object detector, separating touching objects of one class, the job markers did for the watershed.

Foundation models and promptable segmentation. Segment Anything (SAM) [17] is a model prompted with points, boxes or masks, trained on SA-1B, over one billion masks on 11 million images. Click-in, mask-out is the interface of seeded region growing and interactive graph cuts, with learned instead of hand-made similarity. Zhou et al. [12] review over 300 methods of this "foundation model era", split into generic tasks (semantic, instance, panoptic) and promptable ones (interactive, referring, few-shot), and show how large pretrained models carry segmentation knowledge.

Edge detection. Sun et al. [18] group traditional edge detectors into gradient, Gaussian-difference, multi-scale and structured-learning methods, and deep ones into encoder–decoder, network-reconstruction and multi-scale fusion designs. HED [19], the key early deep design, fuses side outputs from several network depths into one edge map, a learned heir to Marr and Hildreth's multi-scale argument. The survey concludes that learned detectors now perform close to, or beyond, human level on standard benchmarks, leaving lightweight models, weak supervision and interpretability as open problems.

Superpixels. Stutz, Hermans and Leibe [20] benchmark 28 algorithms, stressing tuned parameters and **strictly enforced connectivity**, with metrics independent of the number of superpixels and robustness tests against noise, blur and affine transforms.

What changed and what did not. Learned features replaced hand-designed predicates and edge strengths, dramatically so on natural images. The classical skeleton remains: network outputs are probability maps that get thresholded; touching instances still need separating, by boxes, learned markers or a watershed on a predicted distance map; and Otsu, Canny and Hough still win where data are scarce, compute is tight or behaviour must be predictable, as in document scanning and industrial inspection.

Key takeaways

- Segmentation partitions an image into connected, non-overlapping regions that each satisfy a predicate Q; methods look either for **discontinuities** (edges) or for **similarity** (regions).
- First derivatives give thick edge responses; second derivatives give double responses with a zero crossing at the edge centre and are very sensitive to noise, so smooth before differentiating.
- Canny = Gaussian smoothing + gradient + non-maximum suppression + double threshold with **hysteresis**; the last step is what keeps faint but connected edges.
- The Hough transform turns line finding into voting in (ρ, θ) space, which is robust to gaps and clutter.
- Otsu's threshold maximizes the between-class variance $\sigma_B^2 = P_1 P_2 (m_1 - m_2)^2$, computable from cumulative histogram sums; use local thresholds when lighting is uneven.
- Region methods (growing, split-and-merge, k-means, SLIC, normalized cuts) group by similarity; normalized cuts avoid the small-segment bias of plain minimum cuts.
- The watershed floods a gradient image from its minima and builds dams where floods meet; **markers** are essential to avoid over-segmentation.
- Deep networks learned the similarity and edge measures, but thresholding, multi-scale fusion, seeds or prompts, and instance separation are still the skeleton of modern pipelines.

Exercises

1. A 1-D signal is flat at 10 for five samples, rises linearly to 50 over four samples, then stays at 50. Write down its first and second differences, and mark where the zero crossing of the second difference lies relative to the ramp.

   Hint

   The first difference is 10 on the four ramp steps, 0 elsewhere. The second difference is +10 at the ramp's start, −10 at its end, 0 between, so the zero crossing lies mid-ramp.

2. Show that maximizing the between-class variance σ_B^2 is equivalent to minimizing the weighted within-class variance $\sigma_W^2 = P_1 \sigma_1^2 + P_2 \sigma_2^2$.

   Hint

   Split $\sum_i (i - m_G)^2 p_i$ at k and add and subtract each class mean; cross terms vanish, leaving $\sigma_G^2 = \sigma_W^2 + \sigma_B^2$, with σ_G^2 independent of k.

3. In Canny's detector, what happens if you set $T_L = T_H$? What if $T_L = 0$? Predict the result, then test with `skimage.feature.canny` on `data.camera()`.

Hint

$T_L = T_H$ disables hysteresis: broken contours. $T_L = 0$ keeps every thinned pixel connected to a strong edge, so edges leak into the grass texture.

4. A Hough accumulator uses θ in 1° steps over $[-90°, 90°)$ and ρ in 1-pixel steps for a 640×480 image. How many accumulator cells are there, and how many increments does an edge map with 20,000 edge pixels cause?

Hint

$|\rho| \le \sqrt{640^2 + 480^2} = 800$, so about 1,601 ρ bins and 180 θ bins, which is about 288,000 cells. Each edge pixel votes once per θ: $20{,}000 \times 180 = 3.6$ million increments.

5. Explain why plain minimum cut tends to isolate single pixels, using a 4-connected pixel graph with uniform weights w. Then compute Ncut for isolating one interior pixel in an N-pixel image and show that it is close to 1, whereas a balanced cut along a short boundary scores close to 0.

Hint

Isolating one pixel costs $4w$, less than any long boundary. For Ncut, $\text{assoc}(A, V) = 4w$, so the first term is 1 and the second is tiny: Ncut ≈ 1, a bad score.

6. Run the marker-controlled watershed on `data.coins()` with background markers at `coins < 30` and object markers at `coins > 150`, as above, then change the object threshold to 120, 190 and 230. Which coins merge or vanish, and why?

Hint

At 120, object markers land on the bright top band, so coins merge with the background into big regions (we counted 11). At 190 there are still 24, since one marker pixel per coin suffices; at 230 darker coins lose their markers and vanish (13 left). Markers decide the result.

References

1. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018, Ch. 10. publisher page
2. D. Marr and E. Hildreth, "Theory of edge detection," *Proceedings of the Royal Society of London. Series B*, vol. 207, no. 1167, pp. 187–217, 1980. doi
3. J. Canny, "A Computational Approach to Edge Detection," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-8, no. 6, pp. 679–698, 1986. doi

4. R. O. Duda and P. E. Hart, "Use of the Hough transformation to detect lines and curves in pictures," *Communications of the ACM*, vol. 15, pp. 11–15, 1972. doi
5. N. Otsu, "A Threshold Selection Method from Gray-Level Histograms," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 9, no. 1, pp. 62–66, 1979. doi
6. M. Sezgin and B. Sankur, "Survey over image thresholding techniques and quantitative performance evaluation," *Journal of Electronic Imaging*, vol. 13, no. 1, pp. 146–165, 2004. doi
7. J. Sauvola and M. Pietikäinen, "Adaptive document image binarization," *Pattern Recognition*, vol. 33, no. 2, pp. 225–236, 2000. doi
8. R. Achanta, A. Shaji, K. Smith, A. Lucchi, P. Fua and S. Süsstrunk, "SLIC Superpixels Compared to State-of-the-Art Superpixel Methods," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 34, no. 11, pp. 2274–2282, 2012. doi
9. J. Shi and J. Malik, "Normalized cuts and image segmentation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 8, pp. 888–905, 2000. doi
10. L. Vincent and P. Soille, "Watersheds in digital spaces: an efficient algorithm based on immersion simulations," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 13, no. 6, pp. 583–598, 1991. doi
11. S. Minaee, Y. Boykov, F. Porikli, A. Plaza, N. Kehtarnavaz and D. Terzopoulos, "Image Segmentation Using Deep Learning: A Survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 7, pp. 3523–3542, 2022. doi ·arXiv
12. T. Zhou, W. Xia, F. Zhang, B. Chang, W. Wang, Y. Yuan, E. Konukoglu and D. Cremers, "Image Segmentation in Foundation Model Era: A Survey," arXiv:2408.12957, 2024. arXiv
13. J. Long, E. Shelhamer and T. Darrell, "Fully Convolutional Networks for Semantic Segmentation," arXiv:1411.4038, 2014. arXiv
14. O. Ronneberger, P. Fischer and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," MICCAI 2015, arXiv:1505.04597. arXiv
15. K. He, G. Gkioxari, P. Dollár and R. Girshick, "Mask R-CNN," arXiv:1703.06870, 2017. arXiv
16. L.-C. Chen, G. Papandreou, I. Kokkinos, K. Murphy and A. L. Yuille, "DeepLab: Semantic Image Segmentation with Deep Convolutional Nets, Atrous Convolution, and Fully Connected CRFs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 40, no. 4, pp. 834–848, 2018. doi ·arXiv
17. A. Kirillov, E. Mintun, N. Ravi et al., "Segment Anything," arXiv:2304.02643, 2023. arXiv
18. R. Sun, T. Lei, Q. Chen, Z. Wang, X. Du, W. Zhao and A. K. Nandi, "Survey of Image Edge Detection," *Frontiers in Signal Processing*, 2022. doi
19. S. Xie and Z. Tu, "Holistically-Nested Edge Detection," arXiv:1504.06375, 2015. arXiv
20. D. Stutz, A. Hermans and B. Leibe, "Superpixels: An Evaluation of the State-of-the-Art," *Computer Vision and Image Understanding*, vol. 166, pp. 1–27, 2018. doi

IP 11 • Image Segmentation II: Active Contours —Snakes and Level Sets

Segment objects with curves that move: parametric snakes, gradient vector flow, balloon forces, and level-set methods such as geodesic active contours and Chan–Vese.

Online edition: https://blog.shyandram.dev/en/learn/image-processing/dip-11-active-contours/

What you'll learn

- Why segmentation can be posed as moving a curve until an energy stops decreasing, and what that buys over thresholding and edge linking.
- How a **snake** (an explicit, parametric active contour) is defined by internal and external energies, and how the Euler–Lagrange equation turns that energy into an iterative algorithm.
- How to pick the external force: image gradients, edge maps, **gradient vector flow** (**GVF**) and the **balloon** force, and why each one exists.
- How the **level-set method** represents a curve implicitly as the zero crossing of a function ϕ, and how to evolve it with $\phi_t + F|\nabla\phi| = 0$ using upwind finite differences under a CFL limit.
- How the speed function encodes edge-based (**geodesic active contours**) and region-based (**Chan–Vese**, from Mumford–Shah) models, plus signed-distance reinitialization, narrow bands and morphological approximations.
- Where active contours live today: inside deep networks as contour-refinement heads and as loss functions.

The big picture

Chapter 10 segmented images with local rules: threshold this pixel, link that edge, grow this region. Those rules know nothing about the *shape* of what they produce. The result can be a boundary with gaps, a region that leaks through a weak edge, or a speckle of isolated pixels.

Active contours take a different view. We start with a closed curve somewhere near the object and let it **move**. The curve is pushed by the image (toward edges, or toward a good split into "inside" and "outside") and held together by its own stiffness (it resists stretching and bending). When the pushes and the stiffness balance, the curve stops, and its final position is the segmentation. The output is always a closed, smooth boundary, never a heap of edge fragments.

> **Explain like I'm five**
>
> Put a big rubber band around a toy on the table. Now let the band shrink. It slides inward until it hits the toy, and then it hugs the toy's outline. The rubber wants to be short and smooth; the toy stops it from shrinking any further. A snake is that rubber band, and the "toy" is wherever the picture has a strong edge. A level set is a cleverer band that can also snap into two bands if it finds two toys.

Following the textbook [1], this chapter covers the two families in that order. **Snakes** (Kass, Witkin and Terzopoulos [2]) store the curve as a list of points and move the points directly. **Level sets** (Osher and Sethian [5]) store the curve implicitly as the zero contour of a surface and move the surface instead, which makes splitting and merging automatic. Both are *variational*: we write down an energy, then do gradient descent on it. That idea, more than any particular formula, is what survived into deep learning.

Background: why curves that move

In plain words: instead of deciding pixel by pixel, we describe the whole boundary at once and ask, "how good is this boundary?" Then we improve it a little at a time.

More precisely, an active contour method has three ingredients:

1. A **representation** of a closed curve C in the image domain $\Omega \subset \mathbb{R}^2$: either explicit (a parametrized curve $v(s)$) or implicit (the zero level set of a function $\phi(x, y)$).
2. An **energy** $E(C)$ that is low when C is a good boundary. It always mixes a *regularity* term (short, smooth curves are preferred) with a *data* term (edges are strong along the curve, or the regions inside and outside are each homogeneous).
3. An **evolution rule** that changes C to decrease E, usually gradient descent written as a partial differential equation (PDE) in an artificial time t.

Compared with Chapter 10, the regularity term bridges gaps in edges, the output is always a closed curve ready for the shape descriptors of Chapter 12, and prior knowledge (a user's click, a smoothness preference, an intensity model) enters naturally.

The price is that the energy is non-convex: gradient descent finds a *local* minimum, so the answer depends on where you start.

Image segmentation using snakes

A snake is a closed chain of points that slides over the image. Each point feels two kinds of pull: its neighbours keep the chain short and smooth, and the image drags it toward edges.

The snake as a parametric curve

We write the contour as

$$v(s) = (x(s), y(s)), \qquad s \in [0, 1], \quad v(0) = v(1),$$

where s is a parameter that runs once around the curve and $x(s)$, $y(s)$ are image coordinates. Derivatives with respect to s are written $v_s = dv/ds$ and $v_{ss} = d^2v/ds^2$. In a computer the curve is sampled at n points $v_i = (x_i, y_i)$, $i = 0, \ldots, n-1$, with indices taken modulo n for a closed snake.

The energy functional

The snake energy of [2] is

$$E_{\text{snake}}(v) = \int_0^1 \Big[\underbrace{\tfrac{1}{2}\big(\alpha\,|v_s(s)|^2 + \beta\,|v_{ss}(s)|^2\big)}_{E_{\text{int}}} + E_{\text{ext}}(v(s)) \Big]\, ds.$$

- $\alpha \geq 0$ is the **elasticity** (tension) weight. The term $|v_s|^2$ is large when the curve is stretched, so α makes the snake behave like an elastic string that wants to shrink.
- $\beta \geq 0$ is the **rigidity** (bending) weight. The term $|v_{ss}|^2$ measures how sharply the curve turns, so β makes the snake behave like a thin metal rod that resists kinks.
- $E_{\text{ext}}(x, y)$ is the **external (image) energy**, a scalar field over the image that is *low* where we want the snake to settle.

A common choice of external energy for a gray-level image f is

$$E_{\text{ext}}(x, y) = -\,w_{\text{edge}}\,|\nabla(G_\sigma * f)(x, y)|^2 \;-\; w_{\text{line}}\,(G_\sigma * f)(x, y),$$

where G_σ is a Gaussian of standard deviation σ, $*$ is convolution, ∇ is the gradient, and w_{edge}, w_{line} are weights. With $w_{\text{edge}} > 0$ the snake is attracted to strong edges. With $w_{\text{line}} > 0$ it is attracted to dark lines, and with $w_{\text{line}} < 0$ to bright lines. The Gaussian blur widens the valleys of E_{ext} so that the snake can feel an edge from a few pixels away; [2] calls this scale-space continuation.

From energy to motion: the Euler–Lagrange equation

To minimize an integral of the form $\int L(v, v_s, v_{ss})\, ds$ we use the calculus of variations. A minimizer must satisfy the Euler–Lagrange equation

$$\frac{\partial L}{\partial v} - \frac{d}{ds}\frac{\partial L}{\partial v_s} + \frac{d^2}{ds^2}\frac{\partial L}{\partial v_{ss}} = 0.$$

With $L = \tfrac{1}{2}\alpha|v_s|^2 + \tfrac{1}{2}\beta|v_{ss}|^2 + E_{\text{ext}}(v)$ we have $\partial L/\partial v = \nabla E_{\text{ext}}$, $\partial L/\partial v_s = \alpha v_s$ and $\partial L/\partial v_{ss} = \beta v_{ss}$. For constant α and β this gives

$$\alpha\, v_{ss} - \beta\, v_{ssss} - \nabla E_{\text{ext}}(v) = 0.$$

Read it as a force balance. αv_{ss} is the tension force (it points toward the local centre of curvature and shortens the curve), $-\beta v_{ssss}$ is the bending force (it straightens kinks), and $F_{\text{ext}} = -\nabla E_{\text{ext}}$ is the image force. At equilibrium they cancel.

We rarely can solve this equation directly, so we make v depend on an artificial time t and let it slide downhill:

$$\gamma\,\frac{\partial v}{\partial t} = \alpha\, v_{ss} - \beta\, v_{ssss} + F_{\text{ext}}(v),$$

where $\gamma > 0$ is a viscosity (damping) constant. When $\partial v/\partial t = 0$ we are back at the Euler–Lagrange equation.

Discretization and the iterative solution

Sample the curve at n points with unit spacing. Central differences give

$$v_{ss} \approx v_{i+1} - 2v_i + v_{i-1}, \qquad v_{ssss} \approx v_{i+2} - 4v_{i+1} + 6v_i - 4v_{i-1} + v_{i-2}.$$

Stack the x coordinates into a vector $\mathbf{x} \in \mathbb{R}^n$ (and likewise $\mathbf{y}$). Then $\alpha v_{ss} - \beta v_{ssss} = -A\mathbf{x}$, where A is a symmetric, circulant, pentadiagonal $n \times n$ matrix whose rows contain

$$[\ \beta,\ \ -\alpha - 4\beta,\ \ 2\alpha + 6\beta,\ \ -\alpha - 4\beta,\ \ \beta\ ]$$

centred on the diagonal (wrapping around at the ends). The internal forces are linear in the point positions; the external force is not. The classic trick of [2] is a **semi-implicit** step: treat the internal part implicitly (at the new time t) and the external part explicitly (at the old time $t-1$):

$$\gamma\,(\mathbf{x}^t - \mathbf{x}^{t-1}) = -A\,\mathbf{x}^t + \mathbf{f}_x(\mathbf{x}^{t-1}, \mathbf{y}^{t-1}) \quad \Longrightarrow \quad \mathbf{x}^t = (A + \gamma I)^{-1}(\gamma\,\mathbf{x}^{t-1} + \mathbf{f}_x),$$

and the same for $\mathbf{y}$ with $\mathbf{f}_y$. Here $\mathbf{f}_x$, $\mathbf{f}_y$ are the components of F_{ext} sampled (by interpolation) at the current points, and I is the identity. The matrix $(A + \gamma I)$ is fixed, so it is inverted or factorized once; each iteration is then a cheap matrix–vector product. The implicit treatment of the stiff fourth-order term is what lets the snake take reasonable steps without blowing up. The step size is $1/\gamma$: a larger γ means smaller, safer steps.

```
import numpy as np

def snake_step_matrix(n, alpha, beta, gamma):
    """(A + gamma*I)^-1 for a closed snake with n points (h = 1)."""
    row = np.zeros(n)
    row[[0, 1, 2, -2, -1]] = [2*alpha + 6*beta, -alpha - 4*beta, beta,
                              beta, -alpha - 4*beta]
    A = np.stack([np.roll(row, i) for i in range(n)])
    return np.linalg.inv(A + gamma * np.eye(n))

M = snake_step_matrix(100, alpha=0.05, beta=0.05, gamma=1.0)
# one iteration, given external forces fx, fy sampled at the points:
# x = M @ (gamma * x + fx);  y = M @ (gamma * y + fy)
```

scikit-image's `active_contour` implements exactly this scheme, with the edge and line energies above and an extra cap on how far a point may move per iteration [12]:

```
import numpy as np
from skimage import data
from skimage.color import rgb2gray
from skimage.filters import gaussian
from skimage.segmentation import active_contour

img = rgb2gray(data.astronaut())
s = np.linspace(0, 2 * np.pi, 400)
init = np.column_stack([100 + 100 * np.sin(s),   # rows
```

```
                        220 + 100 * np.cos(s)])  # columns
snake = active_contour(gaussian(img, sigma=3), init,
                       alpha=0.015, beta=10, gamma=0.001)
print(snake.shape)  # (400, 2): the final (row, col) points
```

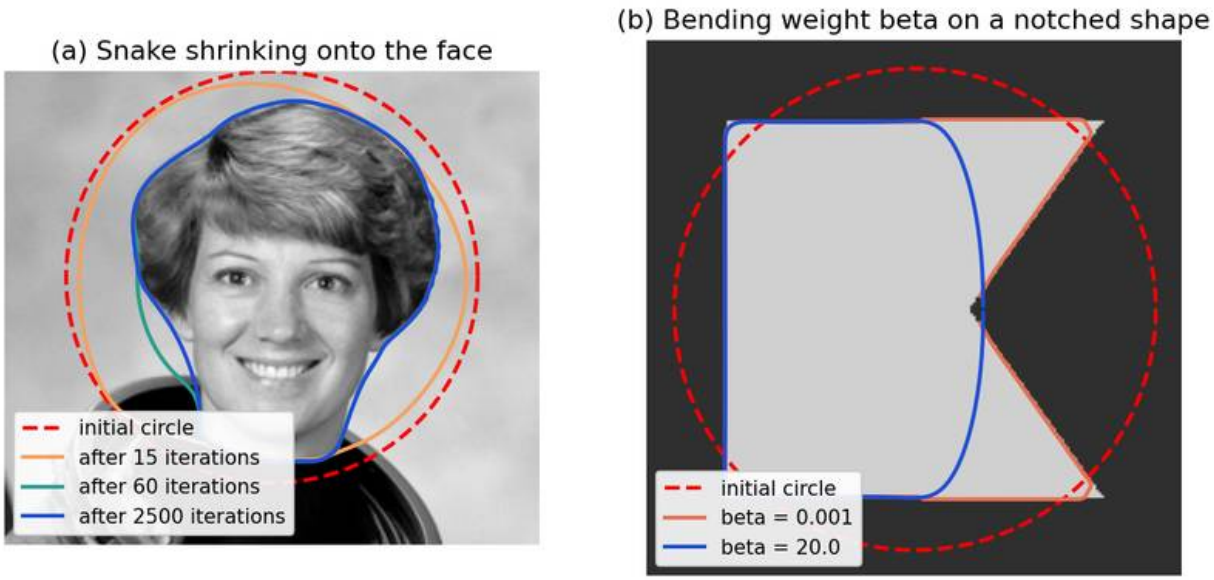


Figure 11.1 — (a) A snake started as a circle shrinks under tension until edge forces stop it on the hair and face outline. (b) The bending weight β: with $\beta = 0.001$ the snake follows the V notch and the sharp corners; with $\beta = 20$ it refuses to bend that hard and bridges the notch with a smooth arc.

Figure 11.1(a) shows the typical life of a snake: it shrinks quickly through the flat background (tension alone), then slows as it enters the zone where the blurred edge map has a slope, and finally locks onto the strongest nearby edges. Panel (b) shows the trade-off that β controls. A stiff snake is robust to noise and small gaps, but it cannot follow corners or concavities.

Choosing the external force

The external force decides *what* the snake looks for. Common choices:

- **Gradient of an edge map.** Compute an edge map $e(x, y) \geq 0$ that is large on edges, for example $e = |\nabla(G_\sigma * f)|$ (or its square), and use $F_{\text{ext}} = \kappa \nabla e$ with a weight κ. The force points uphill on e, toward the ridge of the edge.
- **Line attraction.** Use $F_{\text{ext}} = \pm\nabla(G_\sigma * f)$ to follow dark or bright curvilinear structures (roads, vessels, text strokes).

All of them share the same weakness. The gradient of an edge map is essentially zero away from edges (Figure 11.2a). A snake that starts far from the boundary feels nothing but its own tension. And inside a narrow concavity the forces from the two walls point sideways and cancel, so nothing

pulls the snake *down* into it. These two problems, **limited capture range** and **poor convergence into concavities**, motivated GVF.

Gradient vector flow (GVF)

GVF keeps the good part of ∇e (accurate, pointing at edges where edges exist) and fills in the rest of the image by smooth diffusion. Xu and Prince [3] define the GVF field $\mathbf{w}(x,y) = (u(x,y), v(x,y))$ as the minimizer of

$$\mathcal{E}(\mathbf{w}) = \iint_\Omega \mu\,(u_x^2 + u_y^2 + v_x^2 + v_y^2) \;+\; |\nabla e|^2\,|\mathbf{w} - \nabla e|^2 \, dx\,dy.$$

- u_x, u_y, v_x, v_y are the partial derivatives of the two field components; the first term asks the field to vary slowly.
- $|\nabla e|^2\,|\mathbf{w} - \nabla e|^2$ asks $\mathbf{w}$ to equal ∇e, but only where $|\nabla e|$ is large, that is, near edges.
- $\mu > 0$ balances the two; it should grow with the noise level.

The Euler–Lagrange equations are a pair of decoupled diffusion equations. Solving them by gradient descent in time gives

$$u_t = \mu\,\nabla^2 u - (u - e_x)\,|\nabla e|^2, \qquad v_t = \mu\,\nabla^2 v - (v - e_y)\,|\nabla e|^2,$$

where ∇^2 is the Laplacian and e_x, e_y are the components of ∇e. Far from edges the second term vanishes and the field simply diffuses outward from the edges, carrying their direction with it. With unit time step and the 5-point Laplacian, the explicit scheme is stable for $\mu \le 1/4$ when e is scaled to $[0,1]$.

```
import numpy as np
from scipy import ndimage as ndi

def gvf(f, mu=0.2, iters=500):
    """Gradient vector flow of an edge map f scaled to [0, 1]."""
    fy, fx = np.gradient(f)
    mag2 = fx**2 + fy**2
    u, v = fx.copy(), fy.copy()
    for _ in range(iters):  # explicit scheme, stable for mu <= 0.25
        u += mu * ndi.laplace(u, mode="nearest") - (u - fx) * mag2
        v += mu * ndi.laplace(v, mode="nearest") - (v - fy) * mag2
    return u, v
```

The snake then uses $F_{\text{ext}} = \kappa\,\mathbf{w}$ in place of $\kappa\,\nabla e$, with no other change to the algorithm.

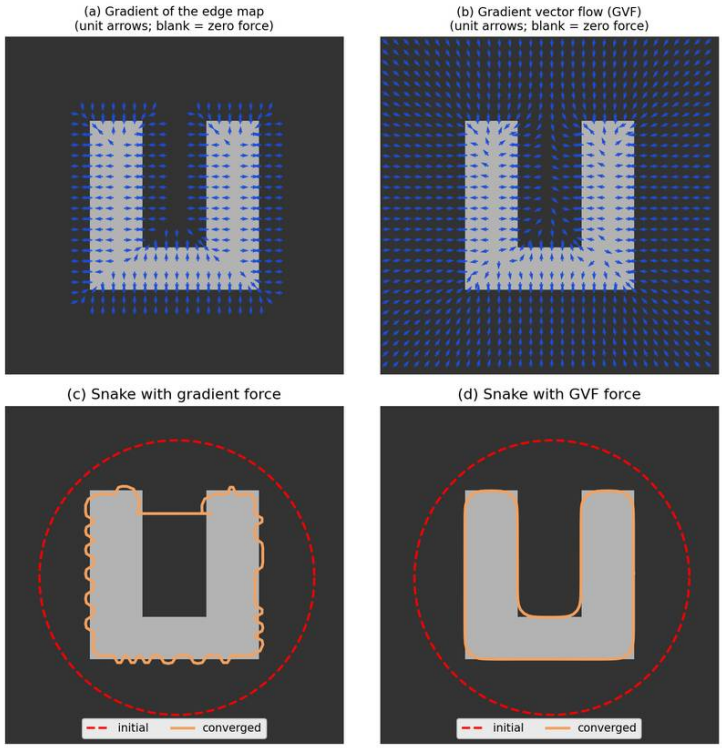


Figure 11.2 — (a) The gradient of an edge map is non-zero only in a thin band around edges. (b) GVF diffuses those vectors over the whole image; inside the notch they point downward. (c) With the gradient force, a snake started on the dashed circle stalls across the mouth of the notch (its wiggles are where it is caught on the edge band with nothing to smooth it). (d) The same snake, same parameters, driven by GVF, follows the concavity to its bottom.

Figure 11.2 reproduces the classic test of [3] on a synthetic U shape. Two things changed: the snake now feels the object from anywhere in the image (large capture range), and inside the notch the diffused vectors point down rather than sideways, so the snake is pulled into the concavity. One implementation detail matters: as the snake enters the notch its length grows, so the points must be **resampled** to a roughly constant spacing every few dozen iterations, or the chain becomes too sparse to follow the walls.

The balloon force

Cohen [4] attacked the capture-range problem from another angle. If you initialize a small snake *inside* the object, tension makes it shrink to a point before any edge force can reach it. So add a pressure:

$$F_{\text{ext}}(v) = \kappa \nabla e(v) \; + \; k_1 \, \mathbf{n}(s),$$

where $\mathbf{n}(s)$ is the unit outward normal of the curve at $v(s)$ and k_1 is the pressure. With $k_1 > 0$ the snake inflates like a balloon; with $k_1 < 0$ it deflates. The edge force must be able to beat the pressure at a real boundary ($\kappa |\nabla e| > |k_1|$ there), but not at the weak, noisy gradients inside the

object. Cohen also suggests normalizing the edge force, $\nabla e/|\nabla e|$, so that one constant controls its strength everywhere.

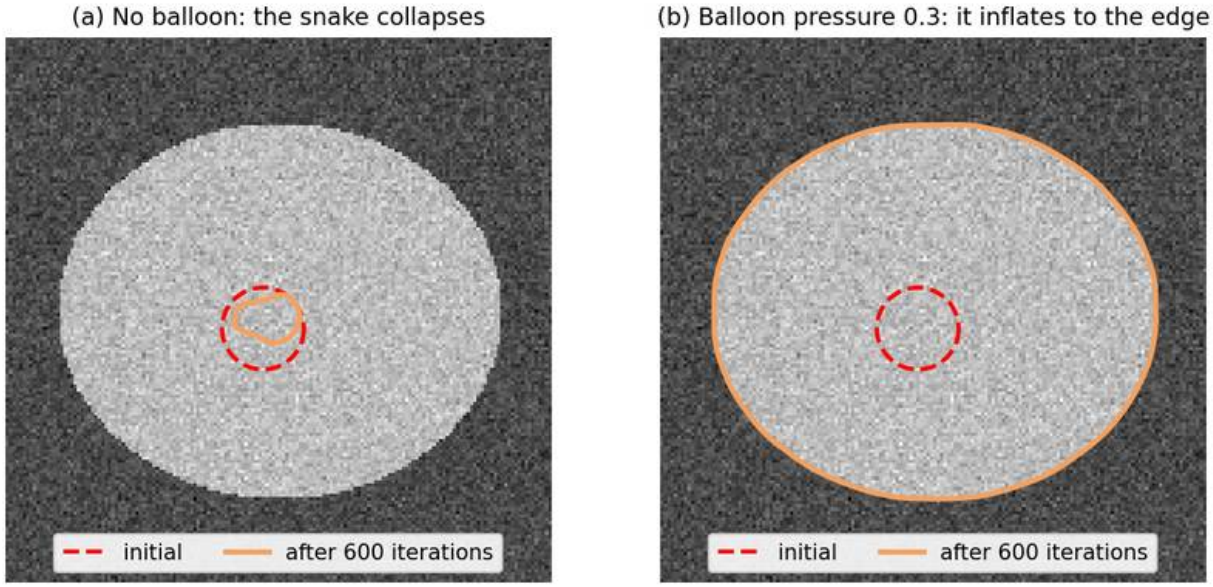


Figure 11.3 — The balloon force. (a) A small snake inside a noisy object, driven only by tension and edge forces, collapses because no edge is within reach. (b) Adding an outward pressure of 0.3 pixel per iteration inflates the snake until the edge force at the true boundary balances the pressure.

The balloon force has a cost: you must know in advance whether to inflate or deflate, and too much pressure carries the snake straight through weak edges. GVF and balloon forces are complementary and are often combined.

Limitations of snakes

- **Initialization.** The energy has many local minima. Even with GVF, a snake started across two objects or on the wrong side of a strong clutter edge will converge to the wrong answer.
- **Parameter tuning.** α, β, γ, κ, the blur σ, the pressure k_1 and the GVF μ interact. Values that work on one image often fail on the next.
- **Point management.** The points drift along the curve, bunch up in some places and spread out in others. Practical snakes resample the curve and must detect when it crosses itself.
- **Topology is fixed.** A snake is one closed chain. It cannot split into two when it encloses two objects (Figure 11.6a), and it cannot merge with another snake, without extra, error-prone bookkeeping.

The last limitation is the one level sets remove.

Segmentation using level sets

Instead of tracking the boundary itself, track a landscape whose shoreline is the boundary. Move the landscape up or down, and the shoreline moves with it. If the water rises enough to separate two hills, the one shoreline becomes two, with no special code.

The implicit representation

A **level-set function** is a scalar function $\phi : \Omega \times [0, \infty) \to \mathbb{R}$ such that the curve at time t is its zero level set:

$$C(t) = \{\, (x, y) \in \Omega \;:\; \phi(x, y, t) = 0 \,\}.$$

We use the convention $\phi < 0$ inside the curve and $\phi > 0$ outside. (Some papers, including [8], use the opposite sign; the equations change sign accordingly.) Two geometric quantities come for free from ϕ:

$$\mathbf{N} = \frac{\nabla \phi}{|\nabla \phi|}, \qquad \kappa = \nabla \cdot \frac{\nabla \phi}{|\nabla \phi|},$$

where $\mathbf{N}$ is the unit **outward** normal and κ is the **curvature** of the level curve through each point (positive on convex parts; for a circle of radius r, $\kappa = 1/r$).

The most convenient ϕ is the **signed distance function**: $|\phi(x, y)|$ is the Euclidean distance from (x, y) to C, with the sign telling inside from outside. It satisfies $|\nabla \phi| = 1$ almost everywhere, which keeps the numerics well behaved.

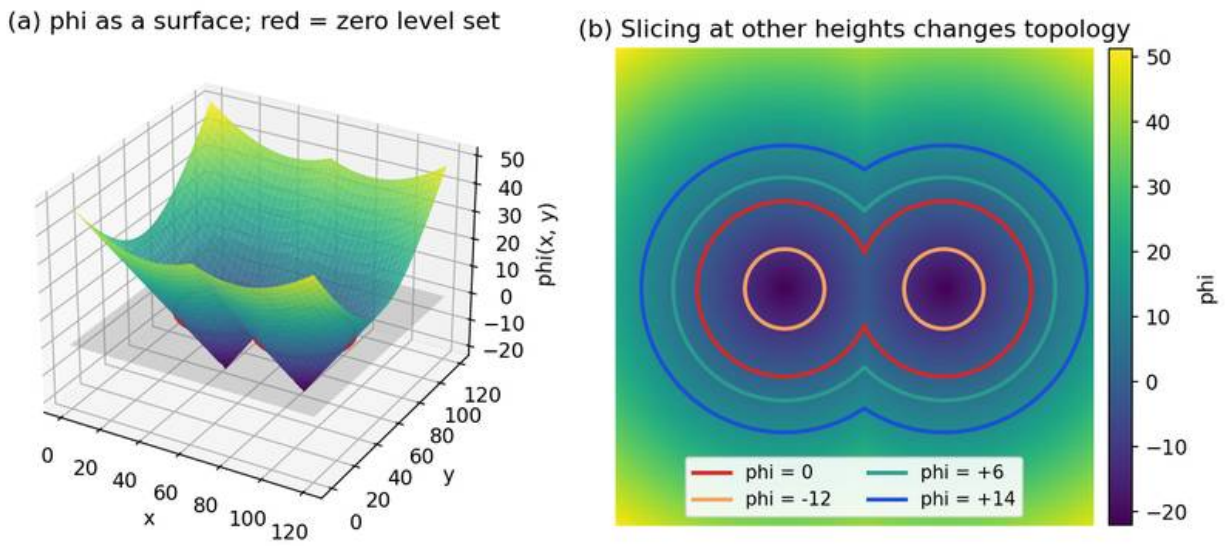


Figure 11.4 — A level-set function φ for two touching discs, built as a signed distance. (a) The curve is where the surface crosses the plane $\varphi = 0$. (b) Slicing the same surface at other heights gives two separate

curves ($\varphi = -12$) or one merged curve ($\varphi = +6, +14$). Moving φ up or down is how a level-set contour splits and merges without any special handling.

The level-set equation

Suppose each point of the curve moves along its outward normal with speed F (positive means outward):

$$\frac{\partial C}{\partial t} = F\,\mathbf{N}.$$

Points on the curve keep the value zero, so $\phi(C(t), t) = 0$ for all t. Differentiate with the chain rule:

$$\phi_t + \nabla\phi \cdot \frac{\partial C}{\partial t} = 0 \quad\Longrightarrow\quad \phi_t + F\,\nabla\phi \cdot \frac{\nabla\phi}{|\nabla\phi|} = 0 \quad\Longrightarrow\quad \boxed{\phi_t + F\,|\nabla\phi| = 0}$$

This is the level-set equation of Osher and Sethian [5]. ϕ_t is the time derivative of ϕ and F is the **speed function**, which may depend on position (image data), on the geometry of the curve (curvature κ), or on global statistics. The equation is solved on the fixed pixel grid for all (x, y), not only on the curve; the curve is read off at any time by extracting the zero contour. Nothing in the equation cares how many pieces the zero set has, which is why splitting and merging happen naturally.

Discrete computation: upwind differences and the CFL condition

Let ϕ_{ij}^{n} be the value at pixel (i, j) after n steps of size Δt, with grid spacing $\Delta x = 1$. The one-sided differences

$$D_{ij}^{-x} = \phi_{ij} - \phi_{i-1,j}, \qquad D_{ij}^{+x} = \phi_{i+1,j} - \phi_{ij}$$

(and $D^{\pm y}$ likewise) are combined so that information is always taken from the side the front is coming from. This is the **upwind** scheme of [5]:

$$\phi_{ij}^{n+1} = \phi_{ij}^{n} - \Delta t\,\Big[\,\max(F_{ij}, 0)\,\nabla_{ij}^{+} + \min(F_{ij}, 0)\,\nabla_{ij}^{-}\Big],$$

$$\nabla^{+} = \sqrt{\max(D^{-x}, 0)^2 + \min(D^{+x}, 0)^2 + \max(D^{-y}, 0)^2 + \min(D^{+y}, 0)^2},$$

$$\nabla^{-} = \sqrt{\min(D^{-x}, 0)^2 + \max(D^{+x}, 0)^2 + \min(D^{-y}, 0)^2 + \max(D^{+y}, 0)^2}.$$

Using a plain central difference here instead produces oscillations and corners that smear or explode; upwinding is what keeps the front sharp and stable.

The explicit scheme is stable only if the front moves less than one cell per step. This is the **Courant–Friedrichs–Lewy (CFL)** condition:

$$\Delta t\,\max_{ij}|F_{ij}| \le c\,\Delta x, \qquad 0 < c \le 1.$$

If F contains a curvature term $b\,\kappa$, that part behaves like diffusion and is discretized with central differences; it adds a stricter, second-order limit of roughly $\Delta t \le \Delta x^2/(4b)$ in 2D. In practice you compute Δt from the current maximum speed at each step.

```
import numpy as np
from scipy import ndimage as ndi

def signed_distance(mask):
    """Negative inside, positive outside, ~0 on the boundary."""
    return ndi.distance_transform_edt(~mask) - ndi.distance_transform_edt(mask)

def evolve(phi, F, steps=100, cfl=0.5):
    """Upwind scheme for phi_t + F |grad phi| = 0 (F is an array or scalar)."""
    for _ in range(steps):
        p = np.pad(phi, 1, mode="edge")
        dxm = phi - p[1:-1, :-2]; dxp = p[1:-1, 2:] - phi   # backward / forward
        dym = phi - p[:-2, 1:-1]; dyp = p[2:, 1:-1] - phi
        grad_plus = np.sqrt(np.maximum(dxm, 0)**2 + np.minimum(dxp, 0)**2 +
                            np.maximum(dym, 0)**2 + np.minimum(dyp, 0)**2)
        grad_minus = np.sqrt(np.minimum(dxm, 0)**2 + np.maximum(dxp, 0)**2 +
                             np.minimum(dym, 0)**2 + np.maximum(dyp, 0)**2)
        dt = cfl / (np.abs(F).max() + 1e-12)             # CFL: |F| dt <= dx
        phi = phi - dt * (np.maximum(F, 0) * grad_plus + np.minimum(F, 0) * grad_minus)
    return phi

yy, xx = np.mgrid[:128, :128]
phi = signed_distance((xx - 64)**2 + (yy - 64)**2 < 20**2)
phi = evolve(phi, F=1.0, steps=20)                # grow outward by 20 * 0.5 = 10 px
phi = signed_distance(phi < 0)                     # re-initialize
print(np.sqrt((phi < 0).sum() / np.pi))            # radius, about 30
```

Specifying the speed function

All the image analysis lives in F. Two families dominate.

Edge-based speed: geodesic active contours

Define an **edge-stopping function** that is close to 1 in flat regions and close to 0 on edges, for example

$$g(x,y) = \frac{1}{1 + |\nabla (G_\sigma * f)(x,y)|^2}.$$

Caselles, Kimmel and Sapiro [6] showed that a snake with $\beta = 0$ is closely related to finding a curve of minimal **weighted length**

$$L_g(C) = \oint_C g(C(s))\, ds,$$

where ds is arc length. In the metric defined by g, edges are "cheap" places to be, so a minimizer is a geodesic (shortest path) that hugs edges. This is the **geodesic active contour (GAC)**. Gradient descent on L_g, plus an optional constant pressure c like Cohen's balloon, gives in our sign convention

$$\phi_t = g\,|\nabla\phi|\,(\kappa - c) + \nabla g \cdot \nabla\phi.$$

Each term has a job. $g\,\kappa\,|\nabla\phi|$ is curvature flow (shortening and smoothing) that slows to a stop near edges. $-c\,g\,|\nabla\phi|$ is the pressure: $c > 0$ inflates, $c < 0$ deflates, again switched off on edges. $\nabla g \cdot \nabla\phi$ is an advection term that pulls the curve into the bottom of the edge valley, where g is minimal, and stops it from overshooting. Unlike a snake, the GAC is **intrinsic**: it does not depend on how the curve is parametrized, and in level-set form it splits and merges freely.

Because g never reaches exactly zero, a strong pressure still leaks through weak or blurred edges. That is the motivation for region-based speeds.

Region-based speed: Mumford–Shah and Chan–Vese

Mumford and Shah [7] posed segmentation as finding a piecewise-smooth approximation u of the image f together with a set of boundaries C:

$$E_{\mathrm{MS}}(u, C) = \int_\Omega (u - f)^2\,dx\,dy \;+\; \mu \int_{\Omega\setminus C} |\nabla u|^2\,dx\,dy \;+\; \nu\,|C|,$$

where the first term asks u to stay close to f, the second asks u to be smooth *except across* C, and $|C|$ is the total boundary length. This energy is hard to minimize in general.

Chan and Vese [8] restricted u to be **piecewise constant with two values**: c_1 inside C and c_2 outside. The energy becomes

$$E_{\mathrm{CV}}(c_1, c_2, C) = \mu\,|C| + \lambda_1 \int_{\mathrm{inside}(C)} (f - c_1)^2\,dx\,dy + \lambda_2 \int_{\mathrm{outside}(C)} (f - c_2)^2\,dx\,dy,$$

(the original also has an optional area term $\nu\,\mathrm{Area}(\mathrm{inside}(C))$, usually set to zero). $\mu \geq 0$ weights smoothness and $\lambda_1, \lambda_2 > 0$ weight how well each region is fitted by its constant. Note what is missing: there is no gradient. The model can segment objects whose boundaries are blurred or defined only by a change of average intensity, which is why [8] called it "active contours without edges".

To write this in level-set form, use the Heaviside function $H(z)$ (1 for $z \geq 0$, 0 otherwise) and its derivative, the Dirac delta $\delta(z)$. With $\phi < 0$ inside, the inside indicator is $H(-\phi)$ and the length is $|C| = \int \delta(\phi)|\nabla\phi|$. For fixed ϕ, the optimal constants are the region means

$$c_1 = \frac{\int f\,H(-\phi)}{\int H(-\phi)}, \qquad c_2 = \frac{\int f\,H(\phi)}{\int H(\phi)}.$$

For fixed c_1, c_2, gradient descent in ϕ gives

$$\phi_t = \delta_\varepsilon(\phi)\,\big[\,\mu\,\kappa + \lambda_1 (f - c_1)^2 - \lambda_2 (f - c_2)^2\,\big],$$

where δ_ε is a smoothed delta of width ε. Read it pixel by pixel: if a pixel's intensity is far from c_1 but close to c_2, the bracket is positive, ϕ rises, and the pixel moves outside. The curvature term keeps the boundary short. The algorithm alternates between updating the means and updating ϕ. Because δ_ε is non-zero away from the zero level set, the model can also **create new contours** far from the initial one, so it can detect interior holes and separate objects from almost any starting curve, a feature [8] highlights.

Cremers, Rousson and Deriche [14] show that this is one member of a large family: replace "constant intensity plus squared error" with any probability model of the region (Gaussian with unknown variance, color histograms, texture features, motion) and you get a region-based level-set method with the same structure, where the bracket becomes a log-likelihood ratio between inside and outside.

```
from skimage import data, img_as_float
from skimage.segmentation import chan_vese, morphological_chan_vese

cam = img_as_float(data.camera())
seg, phi, energies = chan_vese(cam, mu=0.25, lambda1=1, lambda2=1,
                               dt=0.5, max_num_iter=200, extended_output=True)
fast = morphological_chan_vese(cam, num_iter=100, init_level_set="checkerboard",
                               smoothing=3)
```

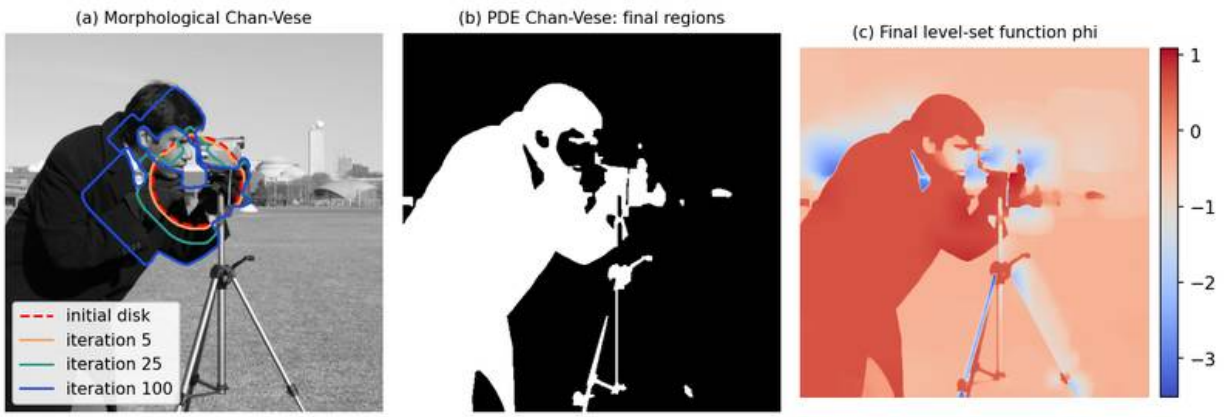


Figure 11.5 — Chan–Vese on the cameraman image. (a) The morphological approximation started from a disk on the face; the region grows into everything that is closer to the dark mean than to the bright mean. (b) The PDE version started from a checkerboard: the two-phase split separates the dark figure, camera and tripod from sky and grass, and also produces new, disconnected contours (tripod legs, distant buildings) that were never connected to the initial curve. (c) The final φ; note that scikit-image uses the opposite sign convention ($\varphi > 0$ inside) [12].

Initializing and reinitializing as a signed distance function

The initial ϕ is usually the signed distance to a user-drawn curve, a box, a disk, or many small circles (a "checkerboard") so that the evolution starts near every possible object. As the PDE

runs, ϕ drifts away from a distance function: it becomes very steep in some places and very flat in others. Steep regions force tiny time steps; flat regions make the zero crossing poorly located and sensitive to noise.

The classical remedy is **reinitialization**: every few iterations, replace ϕ by the signed distance to its current zero level set without moving that zero set. You can recompute it with a fast Euclidean distance transform, as in the code above, or solve a PDE that drives $|\nabla\phi|$ toward 1 while freezing the sign of ϕ. Reinitialization is effective but it can shift the front slightly, and deciding *when* to do it is a heuristic.

Li, Xu, Gui and Fox [10] proposed **distance-regularized level-set evolution** (**DRLSE**), which adds to the energy a penalty $\int p(|\nabla\phi|)$ whose minimum is at $|\nabla\phi| = 1$. The gradient flow of this penalty keeps ϕ close to a signed distance near the zero level set throughout the evolution, which removes the need for reinitialization.

Improving efficiency

A full-grid level-set solver updates every pixel at every step, although only the zero crossing matters. Three standard speed-ups:

- **Narrow band.** Adalsteinsson and Sethian [9] update ϕ only within a band of half-width w (a few pixels) around the zero level set. When the front approaches the edge of the band, the band is rebuilt around the new front (with a reinitialization). The work per step drops from the image area to roughly the curve length times w.
- **Larger stable steps.** Semi-implicit or operator-splitting schemes allow larger Δt for the stiff curvature term, at the cost of solving a linear system per step.
- **Morphological approximations.** Márquez-Neila, Baumela and Álvarez [11] replace the PDE by a sequence of binary morphological operators acting on a *binary* level set $u \in \{0, 1\}$. Dilation and erosion play the role of the balloon term, a composition of line-based sup-inf and inf-sup operators approximates curvature flow, and the sign of $\nabla g \cdot \nabla u$ implements the advection term. They prove that these operators have the same *infinitesimal* behavior as the PDE terms. The result is fast, has no time step, needs no reinitialization and never becomes unstable.

scikit-image provides `morphological_geodesic_active_contour` and `morphological_chan_vese` based on [11], alongside the PDE-based `chan_vese` [12]. Keep in mind that the morphological variants **approximate** the PDEs. They move on the pixel grid, their "smoothing" parameter is a number of operator applications rather than the weight μ, and their results are close to, not identical with, the continuous models.

```
from skimage import data, img_as_float
from skimage.segmentation import (inverse_gaussian_gradient, disk_level_set,
                                  morphological_geodesic_active_contour)

coins = img_as_float(data.coins())
g = inverse_gaussian_gradient(coins, alpha=100, sigma=2)   # ~0 on edges, ~1 elsewhere
init = disk_level_set(coins.shape, radius=150)
```

```
mask = morphological_geodesic_active_contour(g, num_iter=250, init_level_set=init,
                                             smoothing=1, balloon=-1, threshold=0.69)
```

Here `inverse_gaussian_gradient` builds the edge-stopping function g, `balloon=-1` shrinks the initial disk (the role of $c < 0$), and `threshold` decides which values of g count as edges that stop the balloon.

Snakes vs level sets in practice

Both families minimize similar energies. The difference is mostly in representation, and the representation decides what is easy.

	Snakes (explicit)	Level sets (implicit)
Unknowns	n points on the curve	one value per pixel (or per band pixel)
Cost per iteration	$O(n)$ after one matrix factorization	O(pixels), or O(band) with a narrow band
Topology changes	not without extra logic	automatic splitting and merging
Self-intersection	possible; must be detected	impossible by construction
Point spacing	drifts; needs resampling	no points to manage
Shape priors, landmarks, open curves	easy: points can carry labels	harder: no correspondence between iterations
Interactive editing	natural (drag a point)	indirect
Extension to 3D	meshes; harder bookkeeping	same equations on a voxel grid
Sub-pixel accuracy	yes	yes, by interpolating the zero crossing
Typical failure	stalls on clutter or in concavities	leaks through weak edges (edge-based); merges similar-looking regions (region-based)

Xu, Yezzi and Prince studied the relationship between the two formulations [3]; for many energies the choice is about implementation, not about what can be modeled in principle. A rule of thumb:

- Use a **snake** when you need one boundary of known topology, interactively, fast, or with point correspondences (tracking a contour across video frames, fitting a lip outline, delineating a building footprint).
- Use a **level set** when the number of objects is unknown, objects may touch or split, the boundary is ill-defined by gradients (region-based models), or you work in 3D.

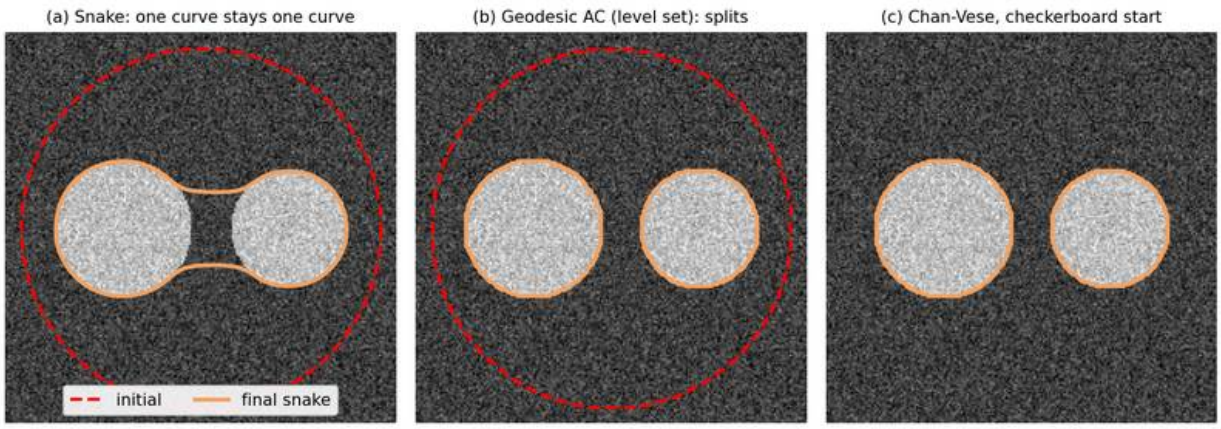


Figure 11.6 — Topology in practice. (a) A snake started around both discs can only shrink into a single curve that wraps both and bridges the gap. (b) A geodesic active contour (morphological level-set version) from the same initial circle splits into two curves when its two halves meet in the middle. (c) Chan–Vese started from a checkerboard does not need a meaningful initial curve at all.

Modern view

Active contours are no longer the default way to segment an image; deep networks are. But the ideas of this chapter, energies that combine boundary length with region fit, explicit contours deformed iteratively, and level sets as a shape representation, are still active ingredients in deep segmentation. Six papers give a good map.

Deformable models in medical imaging. McInerney and Terzopoulos [13] wrote the standard early survey of deformable models in medical image analysis. They frame snakes and their 3D relatives as energy-minimizing physical models and review applications to segmentation, shape representation, matching and motion tracking across CT, MRI and other modalities. Their main takeaway still holds: deformable models are most useful when the image alone is ambiguous and the model must contribute smoothness and prior knowledge.

Statistical region-based level sets. Cremers, Rousson and Deriche [14] review region-based level-set methods and derive them from a common Bayesian framework: each region gets a probability model (for intensity, color, texture or motion), and the level set maximizes the posterior probability of the partition, optionally with a statistical shape prior. Their argument for region-based energies, which are less sensitive to noise and have fewer local minima than edge-based snakes, is the reason Chan–Vese-type terms dominate later work.

A recent taxonomy of active contour models. Chen, Ge, Wang and Chen [15] sort active contour models into edge-based (GAC, DRLSE), region-based (Chan–Vese and local-fitting variants designed for intensity inhomogeneity) and hybrid models, implement a set of representative models, and compare them with deep learning segmentation on synthetic, medical and natural images. Their conclusion is pragmatic: classical models need no training data and give smooth closed boundaries, but they are sensitive to initialization and parameters; deep models are more

accurate where labelled data exist; the two are complementary.

Shape constraints in deep segmentation. Bohlender, Oksuz and Mukhopadhyay [16] survey how anatomical shape knowledge is injected into deep medical image segmentation: conditional random fields, statistical shape models, active contours, and implicit shape representations such as signed distance maps. Their motivation is exactly the weakness active contours were designed to fix: pixel-wise classifiers can produce fragmented regions and topological errors.

Explicit contours learned end to end. Deep Snake by Peng et al. [17] keeps the snake's representation, a closed polygon of vertices that is deformed iteratively, but replaces the energy-derived update with a learned one. Each vertex is described by CNN features sampled at its location, and a circular convolution along the polygon predicts per-vertex offsets toward the object boundary. Used for instance segmentation, it starts from a detector's box, and the authors report real-time speed. The circular convolution plays the role that the pentadiagonal matrix A played for the classical snake: it couples each vertex to its neighbours along the closed curve.

Active-contour energies as loss functions. Instead of running a contour at test time, several methods put the energy into the training loss:

- Chen et al. [18] propose a loss for medical image segmentation that adds a boundary-length term and inside/outside region-fitting terms, a supervised analogue of the Chan–Vese energy, to a dense segmentation network, and report improvements over cross-entropy on cardiac MRI.
- Kim and Ye [19] treat the softmax outputs of a network as the characteristic functions of regions and use a Mumford–Shah-style piecewise-constant energy as the loss. Because the energy needs no labels, the same loss supports unsupervised and semi-supervised segmentation, and it can act as a regularizer in supervised training.
- Kervadec et al. [20] propose the **boundary loss**, inspired by graph-based optimization of active-contour flows. It writes a distance between the predicted and true boundaries as a regional integral weighted by the signed distance map of the ground truth, which is exactly the level-set function of the true contour. It is designed for highly unbalanced problems, such as small lesions, where overlap losses alone train poorly, and is used together with a regional loss.

What deep learning changed, and what it did not:

- **Changed: the data term.** Hand-made forces (∇e, GVF, g, region means) are replaced by learned features. This is the main reason accuracy improved.
- **Not changed: the regularity prior.** Boundary length, curvature penalties, signed distance representations and region homogeneity reappear as loss terms and output representations, because pixel classifiers still produce ragged, fragmented boundaries without them.
- **Not changed: the explicit vs implicit trade-off.** Contour-based networks inherit the snake's fixed topology (one polygon per instance) and its speed; mask and distance-map networks inherit the level set's flexibility.
- **Still useful on its own.** When there are no labelled data, a Chan–Vese or GAC run is a few lines of scikit-image and needs no training, which makes it a good baseline, a pseudo-label generator, or a post-processing step that cleans up a network's mask.

Key takeaways

- An active contour segments by **minimizing an energy over curves**: a regularity term (length, bending) plus a data term (edges or region fit). Gradient descent on that energy is a PDE in artificial time.
- A **snake** is an explicit chain of points. Its Euler–Lagrange equation $\alpha v_{ss} - \beta v_{ssss} + F_{\text{ext}} = 0$ is solved with a semi-implicit step $\mathbf{x}^t = (A + \gamma I)^{-1}(\gamma \mathbf{x}^{t-1} + \mathbf{f}_x)$.
- Plain gradient forces have a **small capture range** and fail in **concavities**. GVF diffuses edge vectors across the image; the balloon force inflates or deflates the curve.
- A **level set** stores the curve as $\{\phi = 0\}$ and evolves $\phi_t + F|\nabla\phi| = 0$ with **upwind differences** under a **CFL** step limit. Topology changes are automatic.
- The speed F carries the model: **geodesic active contours** minimize edge-weighted length; **Chan–Vese** fits two region means and works without edges.
- Keep ϕ close to a **signed distance** (reinitialization or DRLSE), and save work with **narrow bands** or **morphological** approximations, which approximate rather than solve the PDEs.
- In deep learning the same ideas survive as **learned contour deformation** (Deep Snake) and as **loss functions** built from length, region and signed-distance terms.

Exercises

1. Tension alone. A closed snake with $\beta = 0$ and no external force is initialized as a circle of radius R sampled at n evenly spaced points. Using the discrete internal force $\alpha(v_{i+1} - 2v_i + v_{i-1})$, show that the circle stays a circle and find how its radius changes in one explicit step of size τ.

Hint

For a regular polygon with angle step $\theta = 2\pi/n$, $v_{i+1} + v_{i-1} = 2\cos\theta\, v_i$ (measured from the centre). The force is $-2\alpha(1 - \cos\theta)\, v_i$, pointing to the centre, so $R \mapsto R\,[1 - 2\alpha\tau(1 - \cos\theta)]$. The circle shrinks geometrically; note that the rate depends on n, which is one reason snake parameters are tied to the sampling.

2. Concavity test. Using the `gvf` function above, build the U shape of Figure 11.2 and vary the GVF weight μ (0.05, 0.1, 0.2) and the number of GVF iterations. Which parameter controls how far the field reaches from the edges, and which controls how smooth it is?

Hint

The number of iterations sets how far diffusion can travel (roughly the square root of iterations times μ). A larger μ also smooths more strongly, which helps against noise but rounds off small features. Above $\mu = 0.25$ the explicit scheme becomes unstable.

3. Signs and directions. With our convention ($\phi < 0$ inside), what happens to a circle under $\phi_t + F|\nabla\phi| = 0$ when $F = -1$? When $F = -\kappa$? Write the radius $r(t)$ in each case.

Answer

$F = -1$: the circle shrinks at unit speed, $r(t) = r_0 - t$. $F = -\kappa = -1/r$: curve-shortening flow, $\dot{r} = -1/r$, so $r(t) = \sqrt{r_0^2 - 2t}$, and the circle vanishes at $t = r_0^2/2$.

4. CFL in practice. Run the `evolve` snippet with `cfl=0.5`, `1.0` and `2.0` on the same circle. Measure the final radius and look at the zero contour. What breaks, and why does the narrow band not remove this limit?

Hint

Above 1 the front tries to cross more than one cell per step; upwind information comes from the wrong place and the contour becomes jagged or the values blow up. The narrow band reduces the number of pixels updated, not the size of each step.

5. When Chan–Vese fails. Create a synthetic image whose left half is a smooth gradient from black to white and whose right half contains a gray disc on a gray background of the same mean but different variance (add strong noise only inside the disc). Run `chan_vese`. Explain the result from the energy, and propose a change of the region model that would succeed.

Hint

Chan–Vese only compares means, so the disc (same mean) is invisible and the ramp is cut in the middle. Modeling each region by a Gaussian with its own mean *and* variance, as in the statistical framework of [14], makes the disc separable.

6. Energy as a loss. Write the Chan–Vese energy for a soft mask $m(x, y) \in [0, 1]$ (inside probability) instead of a level set, with the length term replaced by the total variation $\int |\nabla m|$. Show that for fixed m the optimal c_1, c_2 are weighted means, and explain why this function can be used to train a network without labels.

Hint

$E = \mu \int |\nabla m| + \lambda_1 \int m\,(f - c_1)^2 + \lambda_2 \int (1 - m)(f - c_2)^2$. Setting $\partial E / \partial c_1 = 0$ gives $c_1 = \int mf / \int m$. Nothing in E refers to ground truth, only to the image itself, so gradients with respect to the network producing m exist without labels; compare with [19].

References

1. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018, Ch. 11. publisher page
2. M. Kass, A. Witkin, and D. Terzopoulos, "Snakes: Active contour models," *International Journal of Computer Vision*, vol. 1, no. 4, pp. 321–331, 1988. doi:10.1007/BF00133570
3. C. Xu and J. L. Prince, "Snakes, shapes, and gradient vector flow," *IEEE Transactions on Image Processing*, vol. 7, no. 3, pp. 359–369, 1998; see also C. Xu, A. Yezzi Jr., and J. L. Prince, "On the relationship between parametric and geometric active contours," *Proc. 34th Asilomar Conf. on Signals, Systems, and Computers*, 2000. GVF project page with both papers

4. L. D. Cohen, “On active contour models and balloons,” *CVGIP: Image Understanding*, vol. 53, no. 2, pp. 211–218, 1991. author PDF
5. S. Osher and J. A. Sethian, “Fronts propagating with curvature-dependent speed: Algorithms based on Hamilton–Jacobi formulations,” *Journal of Computational Physics*, vol. 79, pp. 12–49, 1988. doi:10.1016/0021-9991(88)90002-2
6. V. Caselles, R. Kimmel, and G. Sapiro, “Geodesic active contours,” *International Journal of Computer Vision*, vol. 22, no. 1, pp. 61–79, 1997. PDF
7. D. Mumford and J. Shah, “Optimal approximations by piecewise smooth functions and associated variational problems,” *Communications on Pure and Applied Mathematics*, vol. 42, no. 5, pp. 577–685, 1989. doi:10.1002/cpa.3160420503
8. T. F. Chan and L. A. Vese, “Active contours without edges,” *IEEE Transactions on Image Processing*, vol. 10, no. 2, pp. 266–277, 2001. doi:10.1109/83.902291
9. D. Adalsteinsson and J. A. Sethian, “A fast level set method for propagating interfaces,” *Journal of Computational Physics*, vol. 118, pp. 269–277, 1995. summary page
10. C. Li, C. Xu, C. Gui, and M. D. Fox, “Distance regularized level set evolution and its application to image segmentation,” *IEEE Transactions on Image Processing*, vol. 19, no. 12, pp. 3243–3254, 2010. doi:10.1109/TIP.2010.2069690
11. P. Márquez-Neila, L. Baumela, and L. Álvarez, “A morphological approach to curvature-based evolution of curves and surfaces,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2014. doi:10.1109/TPAMI.2013.106
12. scikit-image developers, “skimage.segmentation” API reference (`active_contour`, `chan_vese`, `morphological_chan_vese`, `morphological_geodesic_active_contour`). docs
13. T. McInerney and D. Terzopoulos, “Deformable models in medical image analysis: A survey,” *Medical Image Analysis*, vol. 1, no. 2, 1996. author PDF
14. D. Cremers, M. Rousson, and R. Deriche, “A review of statistical approaches to level set segmentation: Integrating color, texture, motion and shape,” *International Journal of Computer Vision*, vol. 72, no. 2, pp. 195–215, 2007. doi:10.1007/s11263-006-8711-1
15. Y. Chen, P. Ge, G. Wang, and H. Chen, “An overview of intelligent image segmentation using active contour models,” *Intelligence & Robotics*, vol. 3, no. 1, pp. 23–55, 2023. open access
16. S. Bohlender, I. Oksuz, and A. Mukhopadhyay, “A survey on shape-constraint deep learning for medical image segmentation,” *IEEE Reviews in Biomedical Engineering*, vol. 16, pp. 225–240, 2023. arXiv:2101.07721
17. S. Peng, W. Jiang, H. Pi, X. Li, H. Bao, and X. Zhou, “Deep snake for real-time instance segmentation,” *CVPR*, 2020. arXiv:2001.01629
18. X. Chen, B. M. Williams, S. R. Vallabhaneni, G. Czanner, R. Williams, and Y. Zheng, “Learning active contour models for medical image segmentation,” *CVPR*, 2019. CVF open access
19. B. Kim and J. C. Ye, “Mumford–Shah loss functional for image segmentation with deep learning,” *IEEE Transactions on Image Processing*, vol. 29, pp. 1856–1866, 2020. doi ·arXiv:1904.02872
20. H. Kervadec, J. Bouchtiba, C. Desrosiers, E. Granger, J. Dolz, and I. Ben Ayed, “Boundary loss for highly unbalanced segmentation,” *Medical Image Analysis*, vol. 67, 2021 (first presented at MIDL 2019). arXiv:1812.07032

IP 12 • Feature Extraction

Turn regions, boundaries and whole images into compact numbers that stay the same when the object moves, turns, grows or changes brightness, from chain codes and GLCM texture to Harris, MSER and SIFT.

Online edition: https://blog.shyandram.dev/en/learn/image-processing/dip-12-feature-extraction/

What you'll learn

- The difference between a *feature* and a *descriptor*, and the four invariances most descriptors try to have: translation, rotation, scale and illumination.
- How to turn a segmented boundary into something measurable: chain codes, polygons, signatures and skeletons, and then into numbers such as shape numbers and Fourier descriptors.
- How to describe whole regions by size, shape, topology (Euler number), texture (histogram moments, co-occurrence matrices, spectra) and moment invariants.
- How principal component analysis (PCA) aligns objects and compresses many measurements into a few.
- How the Harris–Stephens corner detector, maximally stable extremal regions (MSER) and SIFT find and describe interest points that can be matched across two photos.
- How surveys of the last two decades judge these handcrafted features, and where learned features (SuperPoint, LoFTR, LightGlue, DINOv2) have replaced them.

The big picture

Segmentation (Chapters 10 and 11) told us *where* objects are. A program still cannot "look at" a region the way we do. It needs a short list of numbers that says *what the region is like*, so that it can compare, sort, search and recognize. Producing that list is feature extraction.

> **Explain like I'm five**
>
> Imagine describing a toy to a friend over the phone. You cannot send a picture, so you say: "It is round, about as big as an orange, has one hole in the middle, and its surface is bumpy." Your friend can now pick the right toy out of a box, even if it is upside down, farther away, or in a darker room. Each thing you said is a **feature**. Good features are the ones that stay true no matter how the toy is turned, moved, or lit.

This chapter follows the order of Gonzalez and Woods [1]. We start with boundaries, move to regions, then to statistics over many measurements (PCA), and end with features that need no segmentation at all: corners, stable regions and SIFT keypoints. These last ones connect classical image processing to modern computer vision tasks such as panorama stitching, 3D reconstruction and visual localization.

Background: features, descriptors and invariance

Plain version. A feature is something interesting you found in an image (a corner, a blob, a region). A descriptor is the list of numbers you use to describe it. We want descriptors that do not change when things that do not matter change.

Precise version. Following [1], we separate two steps:

- **Feature detection** finds *where* a feature is: a boundary, a region, a point with its scale and orientation.
- **Feature description** assigns a vector $\mathbf{d} \in \mathbb{R}^n$ to each detected feature.

A descriptor $\mathbf{d}(\cdot)$ is **invariant** to a family of transformations $\mathcal{T}$ if

$$\mathbf{d}(T(f)) = \mathbf{d}(f) \quad \text{for all } T \in \mathcal{T},$$

where f is the image (or region) and T a transformation applied to it. It is **covariant** if the detected quantity changes *with* the transformation in a predictable way (for example, a keypoint's position moves with the object, and its detected scale doubles when the object doubles). Detectors should be covariant; descriptors should be invariant.

The transformations we care about most are:

Change	What it does to the image	Typical cure
Translation	shifts coordinates	subtract a centroid, or use differences
Rotation	rotates coordinates	use a canonical orientation, magnitudes, or rotation-free quantities
Scale	multiplies coordinates	normalize by size, or search over scale
Illumination	changes intensities, roughly $f \mapsto af + b$	use gradients (remove b) and normalize (remove a)

There is always a trade-off between invariance and *discriminative power*. A descriptor invariant to everything describes nothing: a "6" rotated by 180° is a "9". Choosing features is choosing which differences matter for your task.

Boundary preprocessing

Plain version. Before we can measure a boundary, we must list its pixels in order and, often, simplify it so that noise and pixel staircases do not dominate.

Boundary following

A segmented region is a set of pixels. Most boundary descriptors need the boundary as an *ordered*, closed sequence of points. The standard Moore boundary-following procedure [1] starts at

the uppermost-leftmost foreground pixel b_0 and its background neighbor c_0 to the west. It then walks clockwise around the 8-neighborhood of the current boundary pixel, starting at c, until it hits a foreground pixel. That pixel becomes the next boundary point, the background pixel just checked before it becomes the new c, and the walk repeats until it returns to b_0 moving toward the second point. The result is a list $(x_0, y_0), \dots, (x_{K-1}, y_{K-1})$. OpenCV's `cv2.findContours` and scikit-image's `measure.find_contours` give you this list directly.

Chain codes

A **chain code** replaces the list of points with a list of *directions*. Each step from one boundary pixel to the next is one of 4 or 8 directions, coded 0–3 or 0–7 counterclockwise from east [1]. The code is compact (3 bits per step for 8 directions) and already independent of translation.

Two problems remain. The code depends on the **start point**, and it depends on **rotation**. The fixes are simple:

- **Start point.** Treat the code as circular and choose the rotation of the sequence that forms the smallest integer.
- **Rotation (by multiples of 45°).** Use the **first difference**: the number of direction changes, counted counterclockwise, between consecutive elements, $d_k = (c_k - c_{k-1}) \bmod 8$, where c_k is the k-th code element. Turning the object by 90° adds 2 to every c_k and leaves every d_k unchanged.

Raw chain codes on the full pixel grid are long and noisy. In practice the boundary is first resampled on a coarser grid, so that each link spans several pixels (Figure 12.1, left).

```
import numpy as np, cv2
from skimage import data

mask = (data.horse() == 0).astype(np.uint8)          # True inside the horse
cnts, _ = cv2.findContours(mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_NONE)
pts = max(cnts, key=len)[:, 0, :]                    # (x, y), 8-connected boundary

# Freeman 8-direction code: 0 = east, counted counter-clockwise (y points down)
DIRS = {(1, 0): 0, (1, -1): 1, (0, -1): 2, (-1, -1): 3,
        (-1, 0): 4, (-1, 1): 5, (0, 1): 6, (1, 1): 7}
steps = np.diff(np.vstack([pts, pts[:1]]), axis=0)
code = np.array([DIRS[tuple(s)] for s in steps])

diff = (code - np.roll(code, 1)) % 8                 # first difference: rotation invariant
shifts = [tuple(np.roll(diff, -i)) for i in range(len(diff))]
shape_number = min(shifts)                           # start-point invariant
print(len(code), code[:12], diff[:12])
```

Slope chain codes

Ordinary chain codes only know eight directions. A **slope chain code** (SCC) [1] instead places straight segments of *equal length* along the curve, end to end, and records the change of slope between consecutive segments as a real number, normalized to the interval $(-1, 1]$ (so ± 1 means a full reversal). Because only *changes* in direction are recorded, the SCC is invariant to translation and rotation; because all segments have the same length, it becomes invariant to scale once the segment length is set relative to the curve length. The sum of the absolute slope changes is a natural measure of how winding (tortuous) a curve is.

Polygonal approximation

A polygon can capture the essence of a boundary with a few vertices.

Minimum-perimeter polygons (MPP). Cover the boundary with a strip of square cells (a "cellular complex"). Its inner and outer walls form two closed fences. Now imagine a rubber band placed between the fences and allowed to shrink: it settles into the shortest closed path that stays inside the strip. That path is the MPP [1]. Its vertices are always at convex corners of the inner wall or concave corners of the outer wall, which leads to an efficient algorithm. The cell size controls the detail: big cells give a coarse polygon.

Merging. Walk along the boundary and keep adding points to the current segment while the least-squares line-fit error stays below a threshold T. When the error exceeds T, start a new segment. This is simple, but vertices may not land at true corners.

Splitting. Join the two most distant boundary points with a chord. Find the boundary point farthest from that chord; if its distance exceeds T, make it a vertex and split the problem in two. Repeat recursively. Splitting tends to place vertices at real inflection points. `skimage.measure.approximate_polygon` implements this recursive splitting idea; Figure 12.1 (second panel) shows two tolerances.

Signatures

A **signature** reduces a 2-D boundary to a 1-D function. The simplest is the distance from the centroid to the boundary as a function of angle, $r(\theta)$, or of normalized arc length, $r(s)$ [1]. A circle gives a constant; a square gives four identical bumps. Signatures are translation invariant by construction. Rotation becomes a *circular shift* of the function, and scale becomes a multiplication, so we normalize by choosing a canonical start (for example the farthest point) and dividing by the maximum or by the standard deviation. $r(\theta)$ is a function only for star-shaped regions; for shapes like the horse in Figure 12.1, a ray from the centroid can cross the boundary more than once, which is why $r(s)$ is used there.

Skeletons and the medial axis

Plain version. Light a fire on the entire edge of a grass field at the same moment. The places where fire fronts from different sides meet form the **skeleton**.

Precise version. The **medial axis** of a region R with border B is the set of points in R that have *more than one* closest point on B [1]. Each medial-axis point, together with its distance to the border, defines a maximal inscribed disk; the union of these disks reconstructs R exactly (the medial axis transform). Computing it directly is expensive, so practical skeletons are found by **thinning**: repeatedly deleting border pixels that are not endpoints and whose removal does not break connectivity, until nothing more can be removed. Skeletons are very sensitive to small bumps on the boundary, so smoothing or pruning short branches is usually needed.

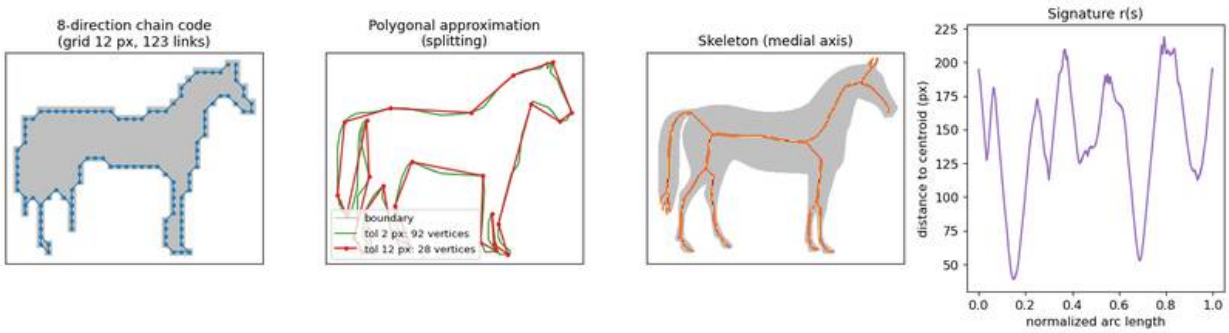


Figure 12.1 — Boundary representations of the same horse silhouette (skimage.data.horse). Left to right: an 8-direction chain code on a 12-pixel grid; polygonal approximation by recursive splitting with tolerances of 2 and 12 pixels; the skeleton obtained by thinning; and the signature $r(s)$, distance to the centroid versus normalized arc length (the deep dips are the legs).

Boundary feature descriptors

Plain version. Once a boundary is an ordered list, we can measure it: how long it is, how wide, how bent, and how it looks when written as a sum of waves.

Basic descriptors

- **Length.** The number of pixels on the boundary is a rough estimate. With an 8-connected chain code, count horizontal and vertical steps as 1 and diagonal steps as $\sqrt{2}$.
- **Diameter.** $\mathrm{Diam}(B) = \max_{i,j} D(p_i, p_j)$, where p_i, p_j are boundary points and D is a distance (usually Euclidean). The segment joining the two maximizing points is the **major axis**; the **minor axis** is perpendicular to it, with length chosen so that a box with these two sides encloses the boundary. That box is the **basic rectangle**.
- **Eccentricity.** The ratio of major-axis length to minor-axis length. (Region-based eccentricity, below, uses second moments instead.)

- **Curvature.** The rate of change of slope along the boundary. On a digital boundary it is noisy, so we usually compute the slope difference between line segments fitted on either side of a point. Where curvature changes sign, the boundary has an inflection; positive curvature (in the walking direction) marks a convex part, negative a concave part.

Shape numbers

The **shape number** of a boundary is the first difference of its 4-direction chain code, circularly shifted to form the smallest integer [1]. Its length is the **order** n of the shape. For a closed boundary on a 4-connected grid, n is always even, and only a finite number of shapes exist for each order. To compare two shapes fairly, we first fix the orientation: align the grid with the shape's basic rectangle (or major axis), pick the grid size that gives the desired order, compute the chain code there, then the shape number. Two shapes can then be compared by their **degree of similarity**, the largest order for which their shape numbers still agree.

Fourier descriptors

Plain version. Walk around the boundary and write each point as a complex number. The sequence repeats every lap, so we can decompose it into waves. The first few, slow waves give the rough outline; fast waves add the details.

Precise version. Let the boundary be K points (x_k, y_k), $k = 0, \dots, K-1$, and write each as

$$s(k) = x(k) + j\,y(k).$$

The discrete Fourier transform of $s(k)$ gives the **Fourier descriptors**

$$a(u) = \sum_{k=0}^{K-1} s(k)\, e^{-j2\pi uk/K}, \qquad u = 0, 1, \dots, K-1,$$

and the inverse rebuilds the boundary. If we keep only P of the coefficients (the lowest frequencies, both positive and negative) and set the rest to zero, we get an approximation $\hat{s}(k)$ that is smooth and keeps the overall shape (Figure 12.2). This is why a handful of Fourier descriptors can represent a shape compactly.

The descriptors react to basic geometric changes in simple ways [1]:

Change to the boundary	Effect on $a(u)$
Translation by $\Delta_{xy} = \Delta_x + j\Delta_y$	only $a(0)$ changes: $a(0) + K\Delta_{xy}$
Rotation by θ	every coefficient is multiplied by $e^{j\theta}$
Scaling by α	every coefficient is multiplied by α
Start point moved by k_0	$a(u)$ is multiplied by $e^{-j2\pi k_0 u/K}$

So we can build an invariant descriptor: drop $a(0)$ (translation), divide by $|a(1)|$ (scale), and keep only magnitudes $|a(u)|$ (rotation and start point). The cost of keeping only magnitudes is losing phase, which carries part of the shape information.

```
import numpy as np
from skimage import data, measure

mask = data.horse() == 0
c = max(measure.find_contours(mask.astype(float), 0.5), key=len)
z = c[:, 1] + 1j * c[:, 0]                        # boundary as complex numbers x + jy

def fourier_descriptor(z, n=16):
    a = np.fft.fft(z)
    a[0] = 0                                      # drop DC   -> translation invariance
    a = a / np.abs(a[1])                          # divide by |a(1)| -> scale invariance
    return np.abs(np.r_[a[1:n + 1], a[-n:]])      # magnitudes -> rotation and start-point
     ↪  invariance

f1 = fourier_descriptor(z)
f2 = fourier_descriptor(0.5 * np.exp(1j * 0.7) * np.roll(z, 300) + (40 + 10j))
print(np.abs(f1 - f2).max())                      # ~1e-16: same descriptor
```

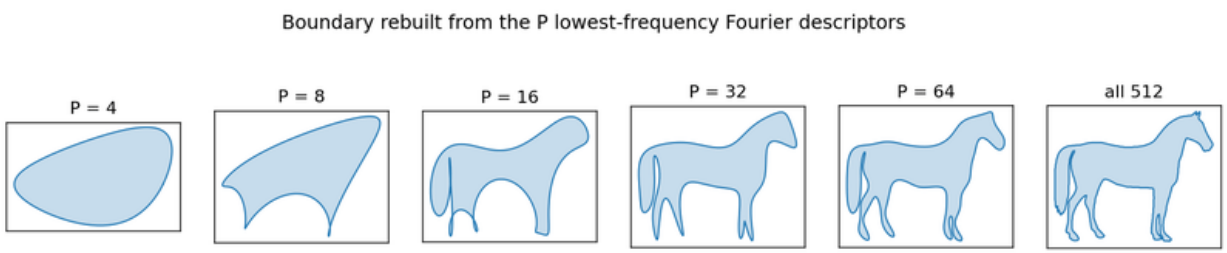


Figure 12.2 — The horse boundary, resampled to 512 points, rebuilt from its P lowest-frequency Fourier descriptors. With $P = 4$ it is an oval blob; by $P = 32$ the legs, neck and head are recognizable; the remaining coefficients mostly add small-scale detail.

Statistical moments of a boundary

A boundary segment, or a signature, is a 1-D function $g(r)$. Treat its amplitude as a random variable v and form a histogram $p(v_i)$, $i = 0, \dots, A-1$, where A is the number of amplitude bins. Then

$$\mu_n(v) = \sum_{i=0}^{A-1} (v_i - m)^n \, p(v_i), \qquad m = \sum_{i=0}^{A-1} v_i \, p(v_i),$$

where m is the mean amplitude and μ_n the n-th central moment [1]. μ_2 measures spread and μ_3 asymmetry. Alternatively, normalize $g(r)$ to unit area and treat it as a density over r; the moments then describe the *shape* of the curve. Moments are cheap, physically interpretable, and insensitive to rotation (if computed on a rotation-normalized signature).

Region feature descriptors

Plain version. Instead of tracing the edge, we can look at the whole patch: how big it is, how round, how many holes it has, and how its inside looks (smooth, grainy, striped).

Basic descriptors

- **Area** A: the number of pixels in the region.
- **Perimeter** P: the length of its boundary.
- **Compactness**: P^2/A. It is dimensionless, so it is invariant to scale (and to translation and rotation, up to digitization error). A disk minimizes it.
- **Circularity**: $R_c = 4\pi A/P^2$. It equals 1 for a disk and decreases for elongated or ragged regions. It is the inverse of compactness, rescaled.
- **Effective diameter**: $d_e = 2\sqrt{A/\pi}$, the diameter of a disk with the same area.
- **Eccentricity** (from moments): with $\lambda_1 \geq \lambda_2$ the eigenvalues of the covariance matrix of the pixel coordinates, $e = \sqrt{1 - \lambda_2/\lambda_1}$. It is 0 for a disk and approaches 1 for a line segment. (This is the definition used by scikit-image.)

```
import numpy as np
from skimage import data, filters, measure, morphology, segmentation

img = data.coins()
bw = img > filters.threshold_otsu(img)
bw = morphology.binary_closing(bw, morphology.disk(2))
bw = segmentation.clear_border(morphology.remove_small_objects(bw, 200))
for r in measure.regionprops(measure.label(bw))[:4]:
    circ = 4 * np.pi * r.area / r.perimeter ** 2  # 1 for a perfect disk
    print(f"area={r.area:5.0f}  circularity={circ:.2f}  "
          f"eccentricity={r.eccentricity:.2f}  euler={r.euler_number}  "
          f"hu1={r.moments_hu[0]:.4f}")
```

On the coins image, the well-segmented coins have circularity around 0.9 and eccentricity around 0.3–0.4 (digital perimeters are slightly overestimated, so even a perfect digital disk scores a bit below 1).

Topological descriptors

Topology studies properties that survive *rubber-sheet* deformations: stretching and bending without tearing or gluing. Area and perimeter are not topological; the number of pieces and the number of holes are. The **Euler number** is

$$E = C - H,$$

where C is the number of connected components and H the number of holes [1]. The letter "A" has $E = 0$ (one component, one hole); "B" has $E = -1$. For regions represented as straight-line networks (polygonal networks), Euler's formula connects topology to counting:

$$V - Q + F = C - H = E,$$

where V is the number of vertices, Q the number of edges and F the number of faces. Note that E depends on the connectivity convention (4 or 8) used for foreground and background.

Texture

Plain version. Texture is how a surface "feels" to the eye: smooth, coarse, regular. We describe it with statistics of the pixel values and of how pixel values sit next to each other.

Statistical moments of the histogram. Let z be a random variable for intensity, $p(z_i)$ the normalized histogram, $i = 0, \dots, L-1$, and $m = \sum_i z_i p(z_i)$ the mean. Then [1]:

$$\begin{aligned}
\mu_n(z) &= \sum_{i=0}^{L-1} (z_i - m)^n \, p(z_i) && (n\text{-th central moment}) \\
R(z) &= 1 - \frac{1}{1 + \sigma^2(z)} && (\text{relative smoothness; } \sigma^2 = \mu_2) \\
U(z) &= \sum_{i=0}^{L-1} p^2(z_i) && (\text{uniformity}) \\
e(z) &= -\sum_{i=0}^{L-1} p(z_i) \log_2 p(z_i) && (\text{entropy})
\end{aligned}$$

R is 0 for a constant region and approaches 1 for high variance (normalize σ^2 to $[0,1]$ first, for example by dividing by $(L-1)^2$). μ_3 measures histogram skew. These measures ignore *where* pixels are, so a checkerboard and its shuffled version get identical values.

Co-occurrence matrices. To capture spatial arrangement, Haralick, Shanmugam and Dinstein proposed the gray-level co-occurrence matrix (GLCM) [3]. Fix a **position operator** Q, for example "one pixel to the right". Then G is an $L \times L$ matrix whose element g_{ij} counts how many times a pixel with level i has a pixel with level j in position Q. Let $n = \sum_{i,j} g_{ij}$ and

$$p_{ij} = g_{ij} / n,$$

the estimated probability of that pair. Useful descriptors computed from p_{ij} include (with m_r, m_c and σ_r, σ_c the means and standard deviations of the row and column marginals):

$$\text{Contrast} = \sum_{i,j} (i-j)^2 \, p_{ij} \qquad \text{Homogeneity} = \sum_{i,j} \frac{p_{ij}}{1 + |i-j|}$$

$$\text{Uniformity (energy)} = \sum_{i,j} p_{ij}^2 \qquad \text{Entropy} = -\sum_{i,j} p_{ij} \log_2 p_{ij}$$

$$\text{Correlation} = \sum_{i,j} \frac{(i - m_r)(j - m_c)\, p_{ij}}{\sigma_r \sigma_c}$$

(scikit-image's `graycoprops` reports "energy" as the square root of the uniformity sum.) Mass near the diagonal means neighbors have similar values (smooth texture, high homogeneity, low contrast). Mass spread away from the diagonal means rapid changes (high contrast). In practice we quantize to few levels (8–32), compute G for several angles and distances, and average over angles for approximate rotation invariance.

```
import numpy as np
from skimage import data
from skimage.feature import graycomatrix, graycoprops

for name in ["brick", "grass", "gravel"]:
    q = (getattr(data, name)()[:256, :256] // 32).astype(np.uint8)   # 8 gray levels
    P = graycomatrix(q, distances=[1], angles=[0, np.pi/4, np.pi/2, 3*np.pi/4],
                     levels=8, symmetric=True, normed=True)
    feats = {p: graycoprops(P, p).mean() for p in ["contrast", "homogeneity", "energy",
     ↪  "correlation"]}
    print(name, {k: round(float(v), 3) for k, v in feats.items()})
```

```
brick  {'contrast': 0.269, 'homogeneity': 0.883, 'energy': 0.579, 'correlation': 0.817}
grass  {'contrast': 0.995, 'homogeneity': 0.717, 'energy': 0.288, 'correlation': 0.664}
gravel {'contrast': 0.669, 'homogeneity': 0.772, 'energy': 0.323, 'correlation': 0.78}
```

The brick wall is dominated by large flat bricks, so its GLCM is concentrated on a few diagonal cells: high energy, low contrast. Grass changes from blade to blade, so its GLCM spreads out: highest contrast, lowest energy. Four numbers already separate the three textures (Figure 12.3).

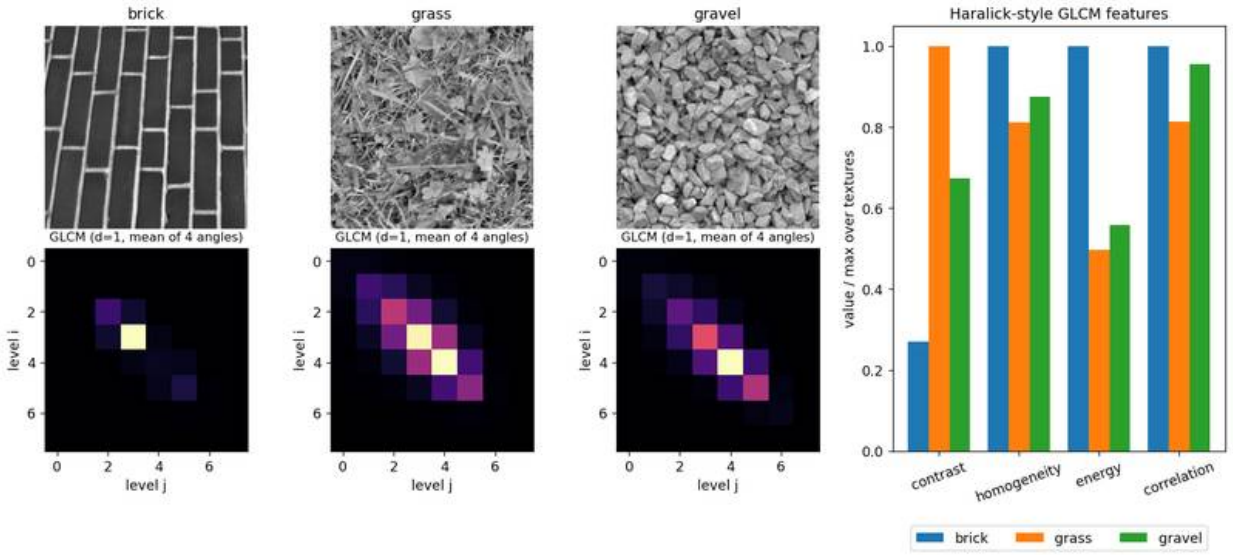


Figure 12.3 — Co-occurrence texture features on three skimage.data textures. Top: 256 × 256 patches. Bottom: GLCMs at distance 1 averaged over four angles (bright = frequent pair). Right: four Haralick-style features, each divided by its maximum over the three textures so they share one axis.

Spectral texture. Periodic or directional texture produces strong, concentrated peaks in the Fourier spectrum. Write the spectrum in polar coordinates, $S(r, \theta)$, where r is the distance from the origin (frequency) and θ the direction. Two 1-D functions summarize it [1]:

$$S(r) = \sum_{\theta=0}^{\pi} S_\theta(r), \qquad S(\theta) = \sum_{r=1}^{R_0} S_r(\theta),$$

where $S_\theta(r)$ is the spectrum along the ray at angle θ, $S_r(\theta)$ the spectrum on the half-circle of radius r, and R_0 the largest radius considered. Peaks in $S(r)$ reveal the period of the pattern (the brick spacing); peaks in $S(\theta)$ reveal its dominant directions. Only half the circle is needed because the spectrum of a real image is symmetric about the origin.

Moment invariants

For a 2-D image (or region indicator) $f(x, y)$ of size $M \times N$, the moment of order $(p + q)$ is

$$m_{pq} = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} x^p y^q f(x, y),$$

and the **central moment** is

$$\mu_{pq} = \sum_x \sum_y (x - \bar{x})^p (y - \bar{y})^q f(x, y), \qquad \bar{x} = \frac{m_{10}}{m_{00}}, \; \bar{y} = \frac{m_{01}}{m_{00}},$$

where $(\bar{x}, \bar{y})$ is the centroid. Central moments are translation invariant. **Normalized central moments**

$$\eta_{pq} = \frac{\mu_{pq}}{\mu_{00}^{\gamma}}, \qquad \gamma = \frac{p+q}{2} + 1,$$

are also scale invariant. Hu [2] derived seven combinations of the second- and third-order η_{pq} that are invariant to translation, scale *and rotation*. The first two are

$$\phi_1 = \eta_{20} + \eta_{02}, \qquad \phi_2 = (\eta_{20} - \eta_{02})^2 + 4\eta_{11}^2.$$

ϕ_1 is the normalized "moment of inertia" about the centroid: it grows as mass moves away from the center. ϕ_2 measures elongation. The seventh invariant changes sign under reflection, so it can tell a shape from its mirror image. Because higher-order moments amplify noise, and their values span many orders of magnitude, people usually compare $\operatorname{sign}(\phi_i)\log|\phi_i|$. In scikit-image, `regionprops(...).moments_hu` returns all seven; in OpenCV, `cv2.HuMoments(cv2.moments(mask))`.

Principal components as feature descriptors

Plain version. When we measure many things about an object, some measurements tell the same story twice. PCA finds a new set of axes ordered from "most informative" to "least informative", so we can keep the first few and drop the rest. For a 2-D shape, the first axis is simply the direction in which the shape is longest.

Precise version. Let $\mathbf{x}_1, \dots, \mathbf{x}_K$ be n-dimensional vectors: for example the n band values of each pixel in a multispectral image, or the (x, y) coordinates of each pixel of a region. Their mean and covariance are

$$\mathbf{m}_x = \frac{1}{K}\sum_{k=1}^{K} \mathbf{x}_k, \qquad \mathbf{C}_x = \frac{1}{K-1}\sum_{k=1}^{K} (\mathbf{x}_k - \mathbf{m}_x)(\mathbf{x}_k - \mathbf{m}_x)^{\mathsf{T}}.$$

$\mathbf{C}_x$ is real and symmetric, so it has n orthonormal eigenvectors $\mathbf{e}_i$ with eigenvalues $\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_n \geq 0$. Stack the eigenvectors as rows of a matrix $\mathbf{A}$. The **Hotelling transform** (also called the discrete Karhunen–Loève transform, or PCA) is

$$\mathbf{y} = \mathbf{A}(\mathbf{x} - \mathbf{m}_x).$$

The new vectors have zero mean and a *diagonal* covariance,

$\mathbf{C}_y = \mathbf{A}\mathbf{C}_x\mathbf{A}^{\mathsf{T}} = \operatorname{diag}(\lambda_1, \dots, \lambda_n)$: the components of $\mathbf{y}$ are uncorrelated, and the variance along the i-th axis is λ_i. If we keep only the first k rows, $\mathbf{A}_k$, the reconstruction $\hat{\mathbf{x}} = \mathbf{A}_k^{\mathsf{T}}\mathbf{y} + \mathbf{m}_x$ has mean squared error

$$e_{\text{ms}} = \sum_{j=k+1}^{n} \lambda_j,$$

the sum of the discarded eigenvalues. No other linear projection onto k dimensions does better in this sense.

Three uses in image processing [1]:

1. **Compressing multispectral images.** With n registered bands, each pixel is an n-vector. Most of the variance usually sits in the first two or three principal-component images, so they can replace all n bands for display or classification.
2. **Normalizing object pose.** Use the pixel coordinates of a region as the vectors. Then $\mathbf{y} = \mathbf{A}(\mathbf{x} - \mathbf{m}_x)$ moves the centroid to the origin and rotates the region so that its major axis lies along the first coordinate axis (Figure 12.4). Descriptors computed afterwards are translation and rotation invariant; dividing by $\sqrt{\lambda_1}$ adds scale invariance. One caveat: an eigenvector's sign is arbitrary, so the aligned object may be flipped; a rule such as "the heavier third-moment side points right" resolves it.
3. **Eigen-images.** Flatten each of many same-size images of one object class into a vector. The leading eigenvectors, reshaped back to images, are "eigen-images" that span most of the variation in the set. Any new image can then be described by its handful of coordinates $\mathbf{y}$, a compact feature vector for recognition.

```
import numpy as np
from skimage import data, transform

mask = transform.rotate((data.horse() == 0).astype(float), 35, resize=True) > 0.5
X = np.argwhere(mask)[:, ::-1].astype(float)     # (x, y) of every object pixel
m = X.mean(axis=0)
C = np.cov((X - m).T)                            # 2x2 covariance matrix
evals, evecs = np.linalg.eigh(C)                 # ascending eigenvalues
A = evecs[:, ::-1].T                             # rows = eigenvectors, largest first
Y = (X - m) @ A.T                                # Hotelling transform y = A(x - m)
print(np.round(np.cov(Y.T), 3))                  # diagonal: the axes are decorrelated
```

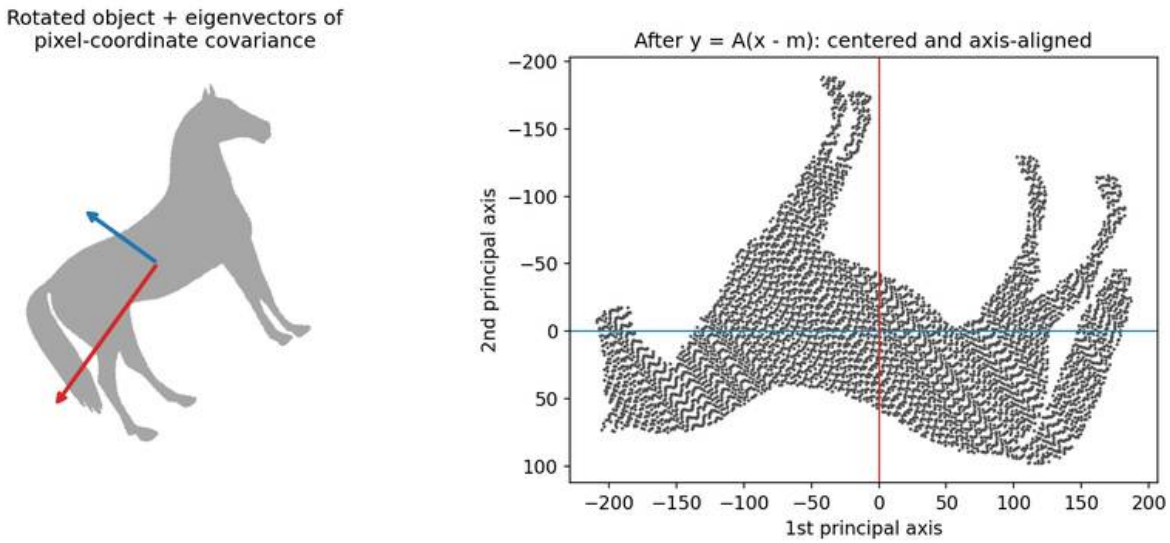


Figure 12.4 — PCA as a pose normalizer. Left: the horse rotated by 35°, with the eigenvectors of its coordinate covariance (arrow length $= 2\sqrt{\lambda}$). Right: the pixel coordinates after $y = A(x - m)$. The object

is centered and its longest direction lies on the horizontal axis; it appears flipped because the sign of each eigenvector is arbitrary.

Whole-image features

Plain version. So far we needed a segmented object. Many tasks (stitching panoramas, tracking, 3D reconstruction) instead need *landmarks* that can be found automatically in any photo and found again in another photo of the same scene. Corners and stable blobs are good landmarks.

The Harris–Stephens corner detector

Plain version. Look at the image through a tiny window and slide it a little. In a flat area nothing changes. Along an edge, sliding along the edge changes nothing but sliding across does. At a corner, sliding in *any* direction changes the view. Harris and Stephens turned this into a formula [4].

Precise version. The change in a window w caused by a shift (u, v) is

$$E(u,v) = \sum_{x,y} w(x,y)\,[f(x+u, y+v) - f(x,y)]^2 \approx [u \;\; v]\,\mathbf{M}\begin{bmatrix} u \\ v \end{bmatrix},$$

using a first-order Taylor expansion, with the **structure tensor** (second-moment matrix)

$$\mathbf{M} = \sum_{x,y} w(x,y)\begin{bmatrix} f_x^2 & f_x f_y \\ f_x f_y & f_y^2 \end{bmatrix},$$

where f_x,f_y are the image derivatives and w is a window, usually Gaussian. The eigenvalues λ_1, λ_2 of $\mathbf{M}$ tell the story: both small means flat, one large means edge, both large means corner. Computing eigenvalues at every pixel was costly in 1988, so Harris and Stephens used the **response**

$$R = \det(\mathbf{M}) - k\,\operatorname{tr}(\mathbf{M})^2 = \lambda_1\lambda_2 - k(\lambda_1 + \lambda_2)^2,$$

with k a small constant (values around 0.04–0.06 are common). R is large and positive at corners, negative at edges, and near zero in flat areas. Corners are local maxima of R above a threshold, after non-maximum suppression.

Properties: $\mathbf{M}$ is built from derivatives, so R ignores additive brightness changes; its eigenvalues do not depend on rotation, so the detector is rotation covariant. It is **not** scale covariant: a corner seen through a small window may look like an edge through a large one. Scale is what SIFT adds.

```
import numpy as np
from scipy import ndimage as ndi
from skimage import data, feature, util

def harris(img, sigma_d=1.0, sigma_i=1.5, k=0.05):
    Ix = ndi.gaussian_filter(img, sigma_d, order=(0, 1))   # derivative along x (columns)
    Iy = ndi.gaussian_filter(img, sigma_d, order=(1, 0))   # derivative along y (rows)
```

```
    Sxx = ndi.gaussian_filter(Ix * Ix, sigma_i)            # entries of the structure tensor
     ↪  M
    Syy = ndi.gaussian_filter(Iy * Iy, sigma_i)
    Sxy = ndi.gaussian_filter(Ix * Iy, sigma_i)
    det, tr = Sxx * Syy - Sxy ** 2, Sxx + Syy
    return det - k * tr ** 2                               # R > 0 corner, R < 0 edge, |R|
     ↪  small flat

img = util.img_as_float(data.camera())
R = harris(img)
corners = feature.corner_peaks(R, min_distance=7, threshold_rel=0.02)   # (row, col) list
```

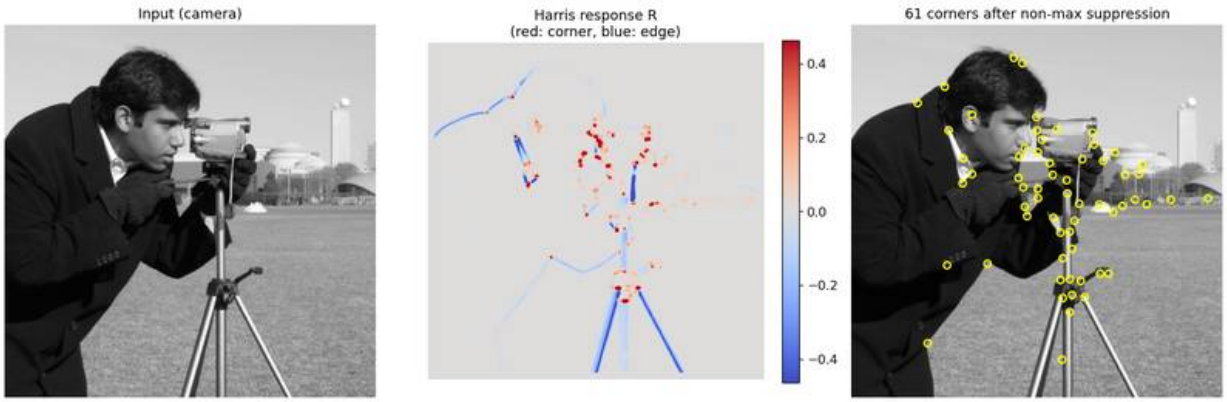


Figure 12.5 — Harris–Stephens corners on skimage.data.camera ($k = 0.05$, integration $\sigma = 1.5$). Middle: the response R; strong positive values (red) cluster on the camera body and tripod joints, while the tripod legs, which are edges, give negative values (blue). Right: local maxima of R after non-maximum suppression.

Interactive demonstration (online): open the online edition of this tutorial to use this demonstration.Open demonstration

Maximally stable extremal regions (MSER)

Plain version. Imagine slowly flooding a gray-level landscape. Dark letters on bright paper become little lakes. As the water level rises, most lakes grow quickly or merge, but a letter's lake keeps exactly its shape over a long range of levels, because the ink is much darker than the paper around it. Regions that barely change while the level changes a lot are *maximally stable*. Matas, Chum, Urban and Pajdla introduced them for wide-baseline stereo [5].

Precise version. Threshold the image at every level t. A connected component of $\{f \le t\}$ (or of $\{f \ge t\}$, for bright regions) is an **extremal region** Q_t: every pixel inside is darker (or brighter) than every pixel on its outer boundary. As t increases, the components grow and merge, forming a tree. For each region, define the stability

$$q(t) = \frac{|Q_{t+\Delta} \setminus Q_{t-\Delta}|}{|Q_t|},$$

where $|\cdot|$ is area and Δ a step in gray levels. A region is **maximally stable** when $q(t)$ has a local minimum. The whole tree can be built in near-linear time using a union-find structure over pixels sorted by intensity.

Why it works: the definition uses only the *order* of intensities, so MSERs are invariant to any monotonic change of brightness. Connected regions map to connected regions under continuous geometric changes, so MSERs are covariant with affine transformations (commonly an ellipse is fitted to each region and normalized to a circle before describing it). MSERs work best on well-defined uniform blobs, such as text, signs and windows, and poorly on blurred or highly textured scenes.

```
import cv2
from skimage import data

img = data.page()                                        # uint8 grayscale
mser = cv2.MSER_create(delta=5, min_area=10, max_area=800, max_variation=0.5)
regions, boxes = mser.detectRegions(img)                 # pixel lists + bounding boxes
print(len(regions), "regions")
```

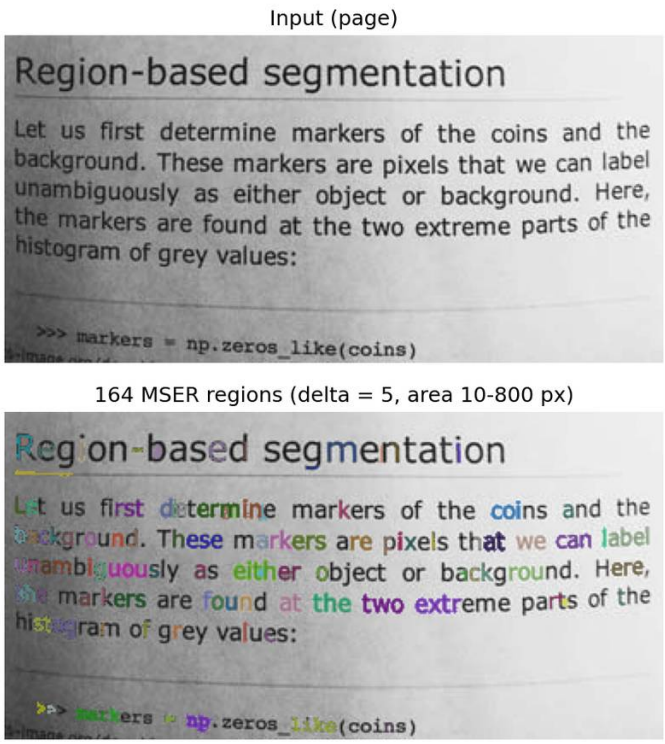


Figure 12.6 — MSER on skimage.data.page, which has uneven lighting. Each colored region is an extremal region that stayed stable over ±5 gray levels. Most are single letters or merged letter groups; because stability depends only on intensity order, the darker bottom-left corner causes no trouble.

Scale-invariant feature transform (SIFT)

Plain version. SIFT, introduced by Lowe [6], answers three questions about every landmark: *where* is it, *how big* is it, and *which way* is it facing? Then it writes down a summary of the gradients around it, measured in that landmark's own size and direction, so the summary is the same whether the photo was taken close up, far away, tilted, or in dim light.

Scale space

To find features at every size, SIFT builds a **scale space**: the image blurred by Gaussians of increasing width,

$$L(x, y, \sigma) = G(x, y, \sigma) \star f(x, y), \qquad G(x, y, \sigma) = \frac{1}{2\pi\sigma^2} e^{-(x^2+y^2)/2\sigma^2},$$

where $\star$ denotes convolution and σ the scale. The scales are grouped into **octaves**; each octave doubles σ and is divided into s intervals, so consecutive scales differ by a factor $k = 2^{1/s}$. Lowe [6] uses $s = 3$ and a base scale $\sigma_0 = 1.6$. After each octave the image is downsampled by 2, which keeps the cost low.

Keypoint detection with difference of Gaussians

Adjacent blurred images are subtracted:

$$D(x, y, \sigma) = L(x, y, k\sigma) - L(x, y, \sigma).$$

This **difference of Gaussians** (DoG) is a close approximation of the scale-normalized Laplacian of Gaussian, $\sigma^2 \nabla^2 G$, scaled by the constant $(k-1)$. It responds strongly to blobs whose size matches σ (Figure 12.7). Candidate keypoints are the pixels whose D value is larger or smaller than all 26 neighbors: 8 in the same DoG image and 9 in each of the scales above and below. A keypoint therefore comes with a position *and* a characteristic scale.

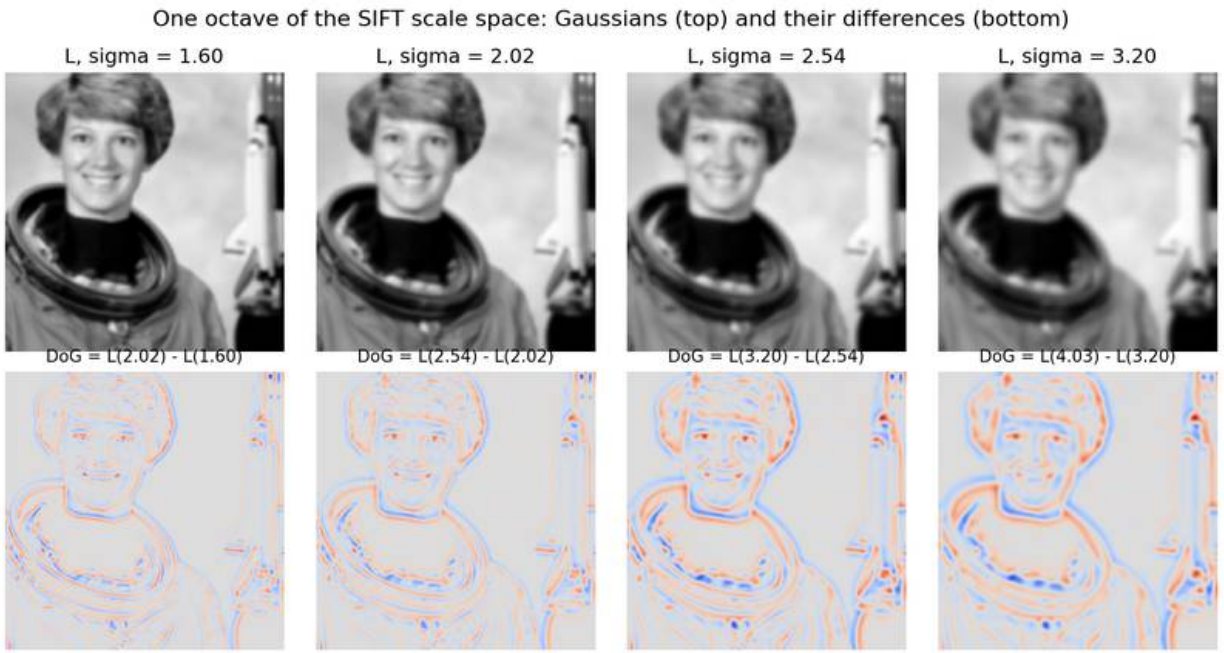


Figure 12.7 — One octave of a SIFT-style scale space on part of skimage.data.astronaut ($\sigma_0 = 1.6$, three intervals per octave). The bottom row shows DoG images: fine structures (hair strands, eyes) respond at small σ, larger structures (the helmet ring, the rocket body) at larger σ. Keypoints are extrema across space and across neighboring DoG images.

Keypoint localization

Extrema are found on a discrete grid. To refine them, fit a 3-D quadratic to D around the sample using its Taylor expansion,

$$D(\mathbf{x}) \approx D + \frac{\partial D}{\partial \mathbf{x}}^{\mathsf{T}} \mathbf{x} + \frac{1}{2}\mathbf{x}^{\mathsf{T}} \frac{\partial^2 D}{\partial \mathbf{x}^2} \mathbf{x}, \qquad \hat{\mathbf{x}} = -\left(\frac{\partial^2 D}{\partial \mathbf{x}^2}\right)^{-1} \frac{\partial D}{\partial \mathbf{x}},$$

where $\mathbf{x} = (x, y, \sigma)^{\mathsf{T}}$ is the offset from the sample point and $\hat{\mathbf{x}}$ the sub-pixel, sub-scale location of the extremum. Two tests then remove unstable points [6]:

- **Low contrast**: discard if $|D(\hat{\mathbf{x}})| < 0.03$ (for intensities in $[0, 1]$).
- **Edge responses**: DoG also responds along edges, where position is poorly defined. Using the 2×2 spatial Hessian $\mathbf{H}$ of D, keep the point only if

$$\frac{\operatorname{tr}(\mathbf{H})^2}{\det(\mathbf{H})} < \frac{(r+1)^2}{r},$$

where r bounds the ratio of the two principal curvatures; Lowe uses $r = 10$. This is the same idea as Harris: a good point must curve strongly in *both* directions.

Orientation assignment

Around each keypoint, compute gradient magnitudes and directions in L at the keypoint's scale. Build a 36-bin histogram of directions (10° per bin), each sample weighted by its gradient magnitude and by a Gaussian window with σ equal to 1.5 times the keypoint scale. The highest peak gives the keypoint's orientation; any other peak above 80% of the highest creates an additional keypoint with that orientation [6]. From now on, all measurements are made relative to this orientation, which gives rotation invariance.

The keypoint descriptor

Take a window around the keypoint, rotated to its orientation and sized by its scale. Divide it into a 4×4 grid of cells. In each cell, accumulate an 8-bin histogram of gradient directions, weighted by magnitude and a Gaussian centered on the keypoint, with trilinear interpolation so that small shifts do not cause sudden jumps between bins. This gives $4 \times 4 \times 8 = 128$ numbers [6]. Finally:

1. normalize the vector to unit length (cancels contrast changes $f \mapsto af$; gradients already cancel $+b$),
2. clip every element at 0.2 (limits the influence of a few very large gradients caused by nonlinear lighting such as glare),
3. renormalize to unit length.

Matching

Descriptors from two images are compared by Euclidean distance. The nearest neighbor alone is not reliable: many keypoints have no true match at all. Lowe's **ratio test** keeps a match only if

$$\frac{\|\mathbf{d} - \mathbf{d}_1\|}{\|\mathbf{d} - \mathbf{d}_2\|} < \tau,$$

where $\mathbf{d}_1, \mathbf{d}_2$ are the nearest and second-nearest descriptors in the other image and τ a threshold (Lowe suggests 0.8) [6]. A distinctive match is much closer than the runner-up. Large databases use approximate nearest-neighbor search, and the surviving matches are usually verified geometrically, for example by fitting a similarity, affine, or projective transformation with RANSAC (random sample consensus) [21] and keeping the inliers.

```
import numpy as np
from skimage import data, feature, transform
from skimage.color import rgb2gray
from skimage.measure import ransac

img1 = rgb2gray(data.astronaut())
tf = transform.AffineTransform(scale=0.7, rotation=np.deg2rad(25), translation=(140, -60))
img2 = 0.8 * transform.warp(img1, tf.inverse) + 0.1        # second "view"

sift = feature.SIFT()
```

```
sift.detect_and_extract(img1); k1, d1 = sift.keypoints, sift.descriptors
sift.detect_and_extract(img2); k2, d2 = sift.keypoints, sift.descriptors

matches = feature.match_descriptors(d1, d2, max_ratio=0.6, cross_check=True)  # ratio test
src, dst = k1[matches[:, 0]][:, ::-1], k2[matches[:, 1]][:, ::-1]           # (row, col) ->
 ↪  (x, y)
model, inliers = ransac((src, dst), transform.SimilarityTransform,
                        min_samples=3, residual_threshold=2, max_trials=500)
print(len(matches), "matches,", inliers.sum(), "inliers")
print("recovered scale %.3f, rotation %.1f deg" % (model.scale, np.degrees(model.rotation)))
```

On this synthetic pair the script finds 484 matches, 483 of them consistent with the true transformation, and recovers scale 0.700 and rotation 25.0°. Real photo pairs, with viewpoint changes, occlusion and repeated patterns, give far lower inlier ratios, which is why geometric verification is not optional.

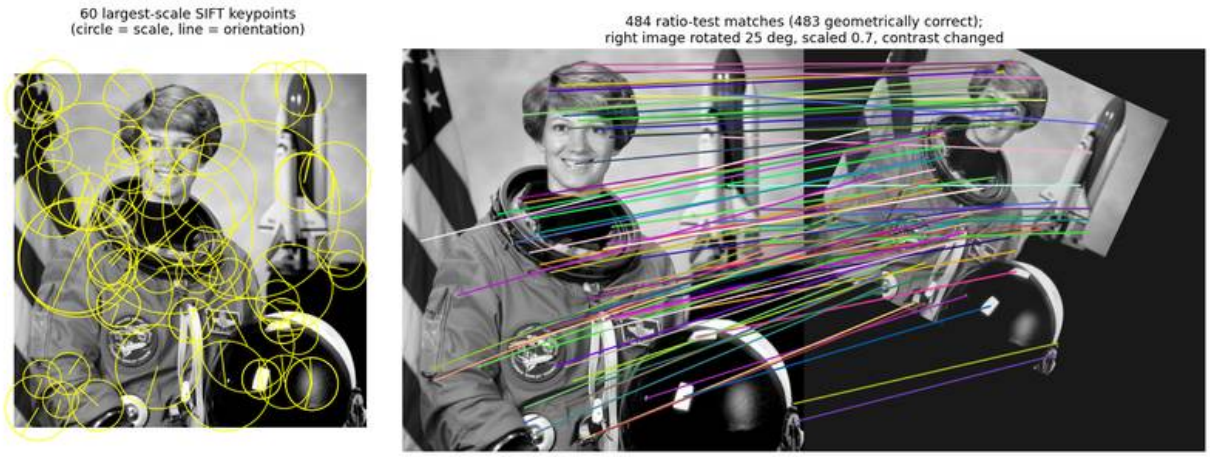


Figure 12.8 — SIFT on skimage.data.astronaut. Left: the 60 largest-scale keypoints (circle radius ∝ scale, line = orientation). Right: a subset of ratio-test matches to a copy rotated by 25°, scaled by 0.7 and changed in contrast; 483 of the 484 matches agree with the true transformation within 3 pixels.

> **Where to find SIFT**
>
> scikit-image added a SIFT detector and descriptor in version 0.19 (`skimage.feature.SIFT`, used above). In OpenCV, call `cv2.SIFT_create()` and then `detectAndCompute(gray, None)`.

Modern view

What the surveys say

Shape descriptors. Zhang and Lu's review [9] sorts shape descriptors along two axes: *contour-based* versus *region-based*, and *global* (one vector for the whole shape) versus *structural* (the shape

broken into primitives). Everything in the first half of this chapter fits that grid: chain codes and polygons are structural contour methods; Fourier descriptors and boundary moments are global contour methods; area, Euler number and Hu moments are global region methods; skeletons are structural region methods. For each technique the review describes the implementation and lists its advantages and disadvantages; the practical message is that no single descriptor is best for every application, so the choice depends on which invariances and how much detail the task needs.

Local feature detectors. Tuytelaars and Mikolajczyk [10] survey interest-point and region detectors: corner detectors (Harris and its scale- and affine-adapted versions), blob detectors (Laplacian/DoG, Hessian), and region detectors (MSER and others). They organize the field around properties a good local feature should have: repeatability, distinctiveness, locality, quantity, accuracy and efficiency. They point out that repeatability, the most important property, can be reached in two ways: by *invariance* (model large deformations mathematically and design the detector to ignore them) or by *robustness* (tolerate small ones such as noise, blur and compression). They also note that some properties compete: distinctiveness and locality cannot both be maximized, because a more local feature sees less intensity pattern and is harder to match correctly. Which compromise is right depends on the application. The two speed-oriented descendants of SIFT are good examples of trading accuracy for time: SURF [7] approximates Gaussian derivatives with box filters on integral images, and ORB [8] combines the FAST corner test with an oriented binary descriptor compared by Hamming distance, fast enough for phones.

Texture. Liu et al. [11] review roughly two decades of texture representation for classification, organized into bag-of-words pipelines (local descriptors, coding and pooling), CNN-based methods, and attribute-based methods. Co-occurrence statistics [3] sit at the root of this history: they are the first widely used descriptors of *pairs* of pixels. The survey, which covers more than 200 publications, traces how the field moved from hand-designed local descriptors encoded as bags of words to representations built on CNN features, and it closes with open questions and directions for future work.

Image matching, handcrafted to deep. Ma et al. [12] follow the full matching pipeline (detection, description, and matching with outlier removal) from handcrafted methods to trainable ones, and compare them experimentally on standard datasets. They show that learning has entered every stage of the pipeline, not only the descriptor. Jin et al. [13] built a benchmark that scores methods by the accuracy of the final camera poses rather than by intermediate metrics. Their key finding is a cautionary one: once each method's settings (ratio-test threshold, RANSAC parameters, number of keypoints) are tuned properly, classical pipelines such as SIFT can still beat methods perceived as the state of the art, which suggests that some reported gains reflected untuned baselines.

How deep learning changed feature extraction

Learned keypoints and descriptors. SuperPoint [14] trains one fully convolutional network to output both a keypoint heatmap and dense descriptors. It is self-supervised: it first learns corners on synthetic shapes, then improves itself on real images through *homographic adaptation*, aggregat-

ing its own detections over many random warps of the same image. DISK [16] trains detection and description end to end with reinforcement learning (policy gradient), rewarding keypoints that lead to correct matches. These replace the *formulas* of Harris and SIFT with learned functions, but keep the same idea: sparse, repeatable points with descriptors.

Learned matching. SuperGlue [15] replaces the nearest-neighbor ratio test with a graph neural network that lets keypoints in both images attend to each other, then solves an optimal-transport assignment that can also say "no match". LightGlue [18] reworks this design to be faster and adaptive, spending less computation on easy image pairs. LoFTR [17] removes the detector altogether: a Transformer with self- and cross-attention matches coarse feature maps densely and then refines positions, which helps in low-texture areas where detectors find nothing.

General-purpose deep features. The deepest change is that features are increasingly *not designed for a task at all*. Self-supervised foundation models such as DINOv2 [19] produce patch features that work across classification, segmentation, depth estimation and retrieval without fine-tuning. RoMa [20] shows the effect on matching: it uses frozen DINOv2 features for coarse matching, combined with fine convolutional features for precise localization. In other words, the handcrafted descriptor at the center of SIFT has been replaced by a network trained on a very large image collection.

What did not change. The *problem decomposition* of this chapter survives. Modern pipelines still detect, describe, match and verify geometrically, typically still with a RANSAC-style robust estimator [21]. Scale and rotation still have to be handled, either built in (SIFT's scale space, ORB's orientation) or learned from data augmentation. Invariance is still a design choice: a descriptor that is too invariant loses discriminative power, whether it was handcrafted or learned. And in measurement-driven fields (materials, microscopy, quality inspection), region descriptors such as area, circularity, Euler number and GLCM statistics remain popular because each number has a physical meaning a human can check.

Key takeaways

- A **detector** finds features (points, regions, boundaries); a **descriptor** turns each into a vector. Detectors should be covariant with geometric changes; descriptors should be invariant to the changes that do not matter for the task.
- Boundaries are first ordered (boundary following) and simplified (chain codes, minimum-perimeter polygons, merging and splitting, signatures, skeletons) before being measured.
- **Fourier descriptors** compress a closed contour into a few coefficients; dropping $a(0)$, dividing by $|a(1)|$ and keeping magnitudes gives translation, scale, rotation and start-point invariance.
- Region descriptors range from size and shape (area, circularity, eccentricity) to topology (Euler number $E = C - H$), texture (histogram moments, **GLCM** features, spectral signatures) and **Hu's moment invariants**.
- **PCA** (the Hotelling transform) decorrelates measurements, aligns objects to their principal axes, and gives compact eigen-image descriptors; the error from dropping components equals the sum of the dropped eigenvalues.
- **Harris** finds corners from the structure tensor; **MSER** finds regions stable over many thresh-

olds; **SIFT** adds scale space, orientation and a normalized 128-D gradient histogram, then matches with the ratio test.
- Learned detectors, matchers and foundation-model features are now the focus of most matching research, but carefully tuned classical pipelines remain strong baselines, and the detect–describe–match–verify structure has not changed.

Exercises

1. **Chain codes by hand.** Write the 8-direction chain code of an axis-aligned 3×2 rectangle of boundary links (3 links wide, 2 tall), starting at its lower-left corner and walking counterclockwise. Compute its first difference and the shape number (minimum circular rotation). Then rotate the rectangle by 90° and show the shape number does not change.

   Hint

   Walking counterclockwise from the lower-left corner (with 0 = east, 2 = north): 0 0 0 2 2 4 4 4 6 6. Each element of the first difference counts counterclockwise turns from the previous direction: 2 0 0 2 0 2 0 0 2 0 (the leading 2 comes from the wrap-around, $6 \to 0$). Its smallest circular rotation is 0 0 2 0 2 0 0 2 0 2. After a 90° turn every code element increases by 2 (mod 8), which leaves the differences unchanged.

2. **Fourier descriptor invariance, tested.** Take the `fourier_descriptor` function from this chapter. (a) Show numerically that it fails to distinguish a shape from its mirror image. (b) Explain why from the table of transformation effects, and propose a change that would tell mirror images apart.

   Hint

   Flip the mask with `mask[:, ::-1]` and trace both contours with `find_contours`; the descriptors agree to within about 10^{-4}. A tracer walks every boundary in the same rotational sense, so the mirrored boundary is $s'(k) = \overline{s(-k)}$ (conjugate *and* reversed), up to a shift. Its DFT is $\overline{a(u)}$: the same magnitudes, opposite phases. Magnitudes cannot see the reflection. Keep some phase instead: the complex number $a(u)\,a(2-u)/a(1)^2$ (indices modulo K) is unchanged by translation, rotation, scale and start point, but it is conjugated by a reflection, so the sign of its imaginary part tells a shape from its mirror image.

3. **Circularity on a grid.** Compute $4\pi A/P^2$ for digital disks of radius 5, 10, 20 and 50 pixels using `skimage.draw.disk` and `regionprops`. Why is the result not exactly 1, and how does it change with radius? Try `perimeter_crofton` as well.

   Hint

   The digital perimeter is a staircase, and different estimators correct for this differently. Errors are relatively larger for small disks. Report both perimeter estimates; one of them should be noticeably closer to the true $2\pi r$.

4. **Texture direction.** Create a synthetic texture of horizontal stripes with period 8 pixels plus a little noise. Compute GLCM contrast for offsets of 1 pixel at angles 0°, 45°, 90° and 135°, and the angular spectral signature $S(\theta)$. Which angle stands out in each, and why do the two methods agree?

   Hint

   Moving along a stripe (0°) keeps intensity almost constant, so contrast is low; moving across stripes (90°) changes it. In the spectrum, horizontal stripes put energy on the vertical frequency axis. Watch the convention: scikit-image's angle 0 means a horizontal pixel offset.

5. **PCA by eigenvalues.** For a filled ellipse with semi-axes $a = 40$ and $b = 10$, predict the ratio λ_1/λ_2 of the coordinate-covariance eigenvalues, then verify it in code. Use your answer to relate λ_1, λ_2 to the eccentricity reported by `regionprops`.

   Hint

   For a uniform filled ellipse, the variance along a semi-axis of length a is $a^2/4$. So $\lambda_1/\lambda_2 = (a/b)^2 = 16$, and $e = \sqrt{1 - \lambda_2/\lambda_1} = \sqrt{1 - b^2/a^2} \approx 0.968$.

6. **Ratio test trade-off.** Using the SIFT matching script, vary `max_ratio` from 0.5 to 0.95 and plot (a) the number of matches and (b) the fraction of RANSAC inliers. Then add a 0.6× rescale plus 30° rotation plus Gaussian noise ($\sigma = 0.05$) and repeat. Where would you set the threshold, and why does the answer depend on the noise?

   Hint

   A loose ratio admits more matches but more of them are wrong; a strict ratio keeps few, mostly correct ones. RANSAC can tolerate many outliers but needs a minimum number of correct matches, so the best threshold sits between the two extremes and moves with image quality.

References

1. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018, Ch. 12. publisher page
2. M.-K. Hu, "Visual pattern recognition by moment invariants," *IRE Transactions on Information Theory*, vol. 8, no. 2, pp. 179–187, 1962. doi
3. R. M. Haralick, K. Shanmugam, and I. Dinstein, "Textural features for image classification," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. SMC-3, no. 6, pp. 610–621, 1973. doi
4. C. Harris and M. Stephens, "A combined corner and edge detector," in *Proc. Alvey Vision Conference*, pp. 23.1–23.6, 1988. doi
5. J. Matas, O. Chum, M. Urban, and T. Pajdla, "Robust wide-baseline stereo from maximally stable extremal regions," *Image and Vision Computing*, vol. 22, no. 10, pp. 761–767, 2004. doi

6. D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004. doi
7. H. Bay, A. Ess, T. Tuytelaars, and L. Van Gool, "Speeded-up robust features (SURF)," *Computer Vision and Image Understanding*, vol. 110, no. 3, pp. 346–359, 2008. doi
8. E. Rublee, V. Rabaud, K. Konolige, and G. Bradski, "ORB: An efficient alternative to SIFT or SURF," in *Proc. IEEE International Conference on Computer Vision (ICCV)*, pp. 2564–2571, 2011. doi
9. D. Zhang and G. Lu, "Review of shape representation and description techniques," *Pattern Recognition*, vol. 37, no. 1, pp. 1–19, 2004. doi
10. T. Tuytelaars and K. Mikolajczyk, "Local invariant feature detectors: A survey," *Foundations and Trends in Computer Graphics and Vision*, vol. 3, no. 3, pp. 177–280, 2008. doi
11. L. Liu, J. Chen, P. Fieguth, G. Zhao, R. Chellappa, and M. Pietikäinen, "From BoW to CNN: Two decades of texture representation for texture classification," *International Journal of Computer Vision*, vol. 127, pp. 74–109, 2019. arXiv
12. J. Ma, X. Jiang, A. Fan, J. Jiang, and J. Yan, "Image matching from handcrafted to deep features: A survey," *International Journal of Computer Vision*, vol. 129, pp. 23–79, 2021. doi
13. Y. Jin, D. Mishkin, A. Mishchuk, J. Matas, P. Fua, K. M. Yi, and E. Trulls, "Image matching across wide baselines: From paper to practice," *International Journal of Computer Vision*, 2021. doi ·arXiv
14. D. DeTone, T. Malisiewicz, and A. Rabinovich, "SuperPoint: Self-supervised interest point detection and description," arXiv:1712.07629, 2017. arXiv
15. P.-E. Sarlin, D. DeTone, T. Malisiewicz, and A. Rabinovich, "SuperGlue: Learning feature matching with graph neural networks," arXiv:1911.11763, 2019. arXiv
16. M. J. Tyszkiewicz, P. Fua, and E. Trulls, "DISK: Learning local features with policy gradient," arXiv:2006.13566, 2020. arXiv
17. J. Sun, Z. Shen, Y. Wang, H. Bao, and X. Zhou, "LoFTR: Detector-free local feature matching with Transformers," arXiv:2104.00680, 2021. arXiv
18. P. Lindenberger, P.-E. Sarlin, and M. Pollefeys, "LightGlue: Local feature matching at light speed," arXiv:2306.13643, 2023. arXiv
19. M. Oquab, T. Darcet, T. Moutakanni, et al., "DINOv2: Learning robust visual features without supervision," arXiv:2304.07193, 2023. arXiv
20. J. Edstedt, Q. Sun, G. Bökman, M. Wadenbäck, and M. Felsberg, "RoMa: Robust dense feature matching," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024. arXiv
21. M. A. Fischler and R. C. Bolles, "Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography," *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981. doi

IP 13 • Image Pattern Classification

From minimum-distance and Bayes classifiers to perceptrons, backpropagation and convolutional neural networks: how a computer decides what an image or region shows.

Online edition: https://blog.shyandram.dev/en/learn/image-processing/dip-13-pattern-classification/

What you'll learn

- How patterns, pattern vectors and pattern classes turn "what is in this image?" into a precise decision problem
- How prototype matching works: the minimum-distance classifier, correlation-based template matching, SIFT feature matching and matching of structural descriptions
- Why the Bayes classifier is optimal, and what it becomes when each class is Gaussian
- How a perceptron learns, how multilayer networks are trained with backpropagation, and how a convolutional neural network (CNN) learns its own features
- The practical side of training: overfitting, data augmentation and regularization
- How image classification moved from AlexNet to Vision Transformers, CLIP and DINOv2, and why benchmark numbers deserve suspicion

The big picture

> **Explain like I'm five**
>
> Imagine a pile of fruit on the kitchen table and three bowls labelled *apples*, *limes* and *oranges*. You pick up each fruit, look at its color and feel its size, and drop it in the bowl it looks most like. That is pattern classification. The color and the size are the **features**. The bowls are the **classes**. The rule in your head ("red and big means apple") is the **classifier**. This chapter is about how to build that rule: by comparing with a typical example of each fruit, by using probabilities, or by letting a network of tiny adding machines learn the rule from many examples.

Chapters 10–12 carved an image into regions and described each region with numbers: areas, moments, Fourier descriptors, texture statistics, keypoint descriptors. This chapter closes the pipeline. It takes those descriptions and assigns each one a **label**: "coin", "digit 7", "water", "tumor", "cat". Almost every practical vision system ends with such a decision, so this is where image processing meets machine learning.

The chapter follows a historical arc that is also a conceptual one [1]. First come classifiers that work directly with **prototypes** (a typical member of each class). Then comes the **statistically optimal** classifier, which needs the class probability distributions. Finally come **neural networks**, which learn the decision rule from data. In their convolutional form they also learn the features, which removes much of the hand-crafted feature engineering of Chapter 12. If you want a gentle

refresher on supervised learning, cost functions and gradient descent first, see ML 01 ·Introduction to Machine Learning.

Background

Plain version. A classifier is a function that looks at a description of something and answers "which group does this belong to?". We build it from examples whose answers we already know.

A **pattern** is an arrangement of descriptors that represents an object or region. A **pattern class** is a family of patterns that share some property. We write the W classes as $\omega_1, \omega_2, \dots, \omega_W$. **Pattern recognition** by machine means assigning each pattern to its class automatically and with as few mistakes as possible.

The work splits into two stages:

1. **Representation.** Choose what to measure: pixel values, region descriptors, keypoint descriptors, or learned features. Chapter 12 covered this stage.
2. **Decision.** Given the measurements, choose a class. This chapter covers this stage.

Methods are usually trained with **supervised learning**: we have a **training set** of patterns with known labels and use it to set the classifier's parameters. We then measure performance on a separate **test set** that the classifier never saw. A third split, the **validation set**, is used to choose settings such as the network size or the stopping time. Mixing these sets up is the most common source of over-optimistic results.

A useful way to think about any classifier is through **decision functions**. For each class ω_j we define a function $d_j(\mathbf{x})$ and assign $\mathbf{x}$ to the class with the largest value:

$$\mathbf{x} \in \omega_i \quad \text{if} \quad d_i(\mathbf{x}) > d_j(\mathbf{x}) \quad \text{for all } j \neq i.$$

Here $\mathbf{x}$ is the pattern and d_j is a scalar score for class ω_j. The **decision boundary** between classes i and j is the set where $d_i(\mathbf{x}) - d_j(\mathbf{x}) = 0$. Every classifier in this chapter, from the simplest to a deep network, is a particular way of building these functions.

Patterns and pattern classes

Plain version. We describe objects either with a list of numbers (a vector) or with a description of how their parts connect (a string or a tree).

Pattern vectors

The most common representation is a column vector of n measurements:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix},$$

where each x_k is one descriptor, such as the area of a region, its hue, a moment invariant or the response of a texture filter. A vector is a point in n-dimensional **feature space**. Good features put patterns from the same class close together and patterns from different classes far apart. If the features are poor, no classifier can rescue them.

Two practical points matter here. First, features with different units need **normalization**; otherwise the one with the largest numeric range dominates any distance. In the fruit example, hue lies in $[0, 1]$ while diameter is measured in centimetres, so we standardize each feature to zero mean and unit variance before comparing distances. Second, the whole image can itself be a pattern vector: an $M \times N$ image flattened into an MN-dimensional vector. This is how neural networks see images, and it is why they need so much data, because the space is enormous.

> **Interactive demonstration (online)**: open the online edition of this tutorial to use this demonstration.Open demonstration

Structural patterns: strings and trees

Some objects are better described by **how their parts are arranged** than by a list of numbers. A boundary traced as a chain code (Chapter 12) is a **string** of symbols, for example `0 0 1 2 2 3 ...`. A scene can be described by a **tree**: the root is the image, its children are the main regions, and their children are sub-regions. Structural descriptions keep relations such as "is inside", "is above" or "follows", which vectors throw away. They are useful for shapes, characters and diagrams, where the order and connection of parts carry the meaning.

Pattern classification by prototype matching

Plain version. Keep one typical example of each class. To classify something new, see which typical example it looks most like.

Minimum-distance classifier

The simplest prototype is the **mean vector** of each class, computed from its training patterns:

$$\mathbf{m}_j = \frac{1}{n_j} \sum_{\mathbf{x} \in \omega_j} \mathbf{x}, \qquad j = 1, 2, \ldots, W,$$

where n_j is the number of training patterns of class ω_j. A new pattern goes to the class whose mean is nearest in Euclidean distance, $D_j(\mathbf{x}) = \|\mathbf{x} - \mathbf{m}_j\|$.

Minimizing $D_j^2 = \mathbf{x}^\top\mathbf{x} - 2\mathbf{x}^\top\mathbf{m}_j + \mathbf{m}_j^\top\mathbf{m}_j$ over j is the same as maximizing

$$d_j(\mathbf{x}) = \mathbf{x}^\top\mathbf{m}_j - \tfrac{1}{2}\,\mathbf{m}_j^\top\mathbf{m}_j,$$

because $\mathbf{x}^\top\mathbf{x}$ does not depend on j. This d_j is linear in $\mathbf{x}$. The boundary between classes i and j is

$$d_i(\mathbf{x}) - d_j(\mathbf{x}) = \mathbf{x}^\top(\mathbf{m}_i - \mathbf{m}_j) - \tfrac{1}{2}(\mathbf{m}_i - \mathbf{m}_j)^\top(\mathbf{m}_i + \mathbf{m}_j) = 0,$$

which is the **perpendicular bisector** of the segment joining $\mathbf{m}_i$ and $\mathbf{m}_j$: a line in 2-D, a plane in 3-D, a hyperplane in general. Figure 13.1 shows the result on our fruit data.

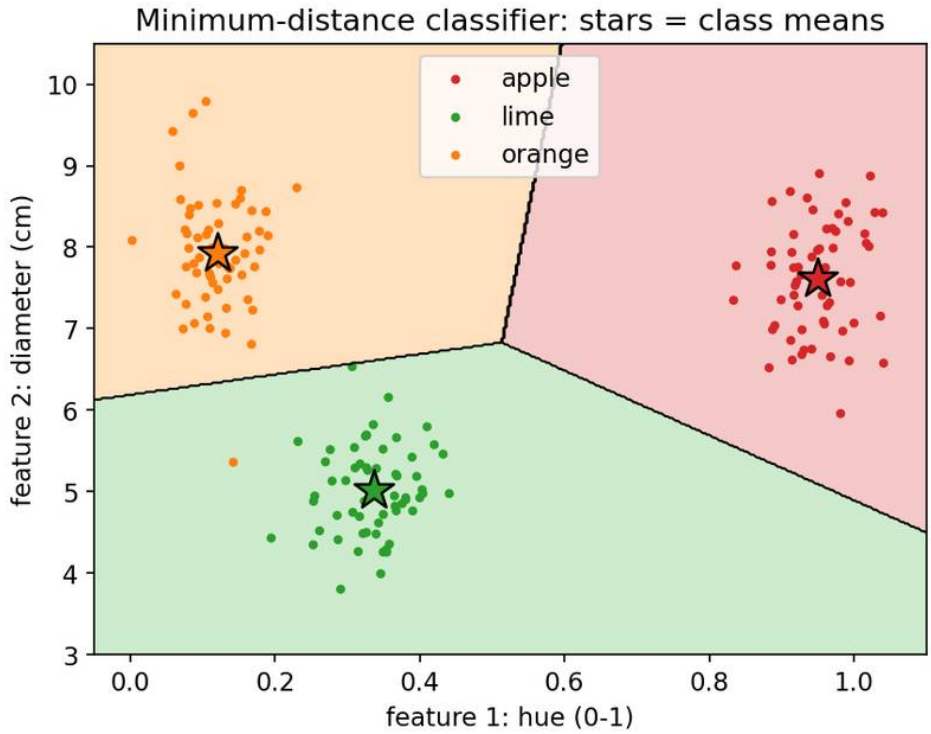


Figure 13.1 — Minimum-distance classification of synthetic fruit (hue, diameter). Stars are the class means; the boundaries are perpendicular bisectors between means (computed on standardized features).

```
import numpy as np

def fit_min_distance(X, y):
    """X: (n, d) pattern vectors, y: (n,) labels -> class list and mean vectors."""
    classes = np.unique(y)
    M = np.stack([X[y == c].mean(axis=0) for c in classes])
    return classes, M

def predict_min_distance(X, classes, M):
    d2 = ((X[:, None, :] - M[None, :, :]) ** 2).sum(axis=-1)  # (n, W) squared distances
    return classes[d2.argmin(axis=1)]
```

```
rng = np.random.default_rng(0)
X = np.r_[rng.normal([0, 0], 1, (100, 2)), rng.normal([4, 3], 1, (100, 2))]
y = np.repeat([0, 1], 100)
classes, M = fit_min_distance(X, y)
print("training accuracy:", (predict_min_distance(X, classes, M) == y).mean())
```

The classifier works well when classes are compact, roughly round blobs whose means are far apart compared with their spread. It fails when a class is elongated, curved or made of several clusters, because one mean cannot represent it. Two simple extensions help: keep **several prototypes per class**, or keep **all** training patterns and use the nearest one (the nearest-neighbor rule).

Matching by correlation (template matching)

When the prototype is a small image rather than a vector, we compare it with every position of a larger image. This is the 2-D version of prototype matching. Raw correlation $\sum wf$ is biased toward bright regions, so we use the **normalized correlation coefficient**:

$$\gamma(x,y) = \frac{\sum_{s,t}\left[w(s,t)-\bar{w}\right]\left[f(x+s,y+t)-\bar{f}_{xy}\right]}{\sqrt{\sum_{s,t}\left[w(s,t)-\bar{w}\right]^2 \sum_{s,t}\left[f(x+s,y+t)-\bar{f}_{xy}\right]^2}},$$

where w is the template, $\bar{w}$ its mean, f the image, and $\bar{f}_{xy}$ the mean of the image patch under the template when its corner is at (x,y). The sums run over the template's support. Because of the subtraction and the division, γ lies in $[-1,1]$ and does not change if the patch's brightness is scaled or offset. A value of 1 means a perfect match up to brightness and contrast. Note that this operation is correlation, not convolution: the template is not flipped (see ML 03 ·Convolution, Image Filtering and CNNs).

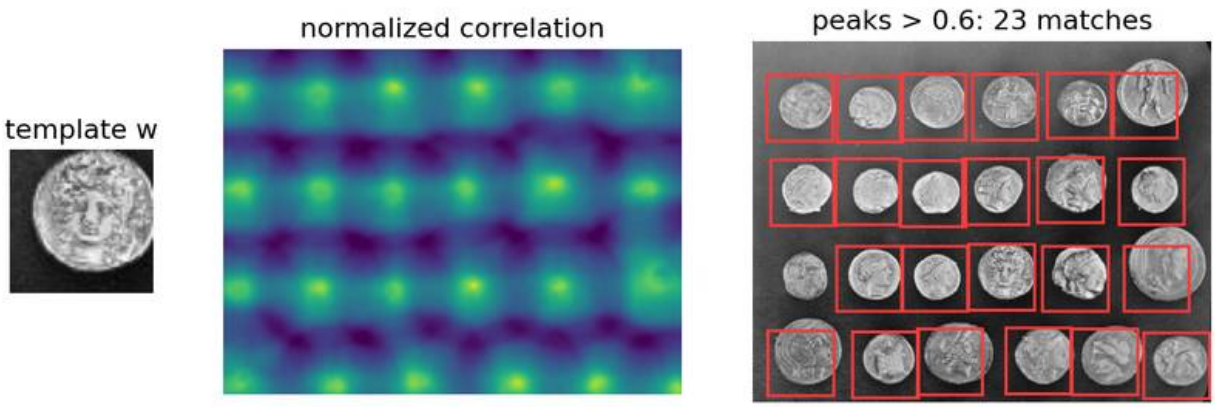


Figure 13.2 — Template matching on skimage's coins image. Left: one coin used as template. Middle: the correlation map, bright where the image looks like the template. Right: local maxima above 0.6.

```
import cv2
import numpy as np
```

```
from skimage import data

img = data.coins().astype(np.float32)
w = img[170:225, 180:235]                          # one coin as the template
R = cv2.matchTemplate(img, w, cv2.TM_CCOEFF_NORMED)  # values in [-1, 1]
y0, x0 = np.unravel_index(R.argmax(), R.shape)
print("best match at", (int(x0), int(y0)), "score", round(float(R.max()), 3))
```

Figure 13.2 also shows the method's limits. The template finds almost every coin, because every coin is a bright disk of similar size on a dark background. It cannot tell one coin design from another at this resolution. Correlation is also sensitive to rotation and scale: a template rotated by 30° or enlarged by 30% may no longer produce a clear peak. The usual remedies are to search over a set of rotated and scaled templates, or to match invariant features instead of raw pixels.

Matching SIFT features

The scale-invariant feature transform (SIFT) from Chapter 12 [3] gives exactly those invariant features. Each keypoint has a location, scale, orientation and a 128-dimensional descriptor. To decide whether a prototype object appears in a scene image:

1. Extract keypoints and descriptors from the prototype and from the scene.
2. For each prototype descriptor, find its nearest and second-nearest scene descriptors in Euclidean distance.
3. Accept the match only if the nearest distance is clearly smaller than the second-nearest, for example $d_1/d_2 < 0.8$. This **ratio test** [3] rejects ambiguous matches in repetitive texture.
4. Check that the accepted matches agree on one geometric transformation (a similarity, affine or projective map). Matches that disagree are outliers.

The decision is then "object present" if enough geometrically consistent matches survive. This is still prototype matching: the prototype is the set of descriptors, and the "distance" is the number of consistent correspondences. It tolerates rotation, scale change, partial occlusion and moderate viewpoint change, which template matching cannot.

Matching structural prototypes

For strings and trees, the distance must compare symbols and their order. Two simple measures:

- **Shape-number similarity.** If two closed boundaries are encoded as shape numbers (normalized chain-code differences, Chapter 12), their **degree of similarity** k is the largest order (code length) at which the two shape numbers are still identical. Similar shapes agree up to high orders. A distance can then be defined as $D = 1/k$: zero for identical shapes, large for shapes that already differ at coarse orders.
- **String similarity.** Align two strings a and b symbol by symbol. Let α be the number of positions that match and $\beta = \max(|a|, |b|) - \alpha$ the number that do not, where $|a|$ is the length of a. Then

$R = \alpha/\beta$ is infinite for a perfect match and 0 when nothing matches. A more forgiving variant is the **edit distance**: the minimum number of insertions, deletions and substitutions that turns one string into the other, computed by dynamic programming.

A new string goes to the class whose prototype string is most similar. Structural matching is less common today, but the idea of comparing **sequences** and **graphs** of parts survives in graph matching and in transformer models that compare sets of tokens.

Optimum (Bayes) statistical classifiers

Plain version. If we know how often each class occurs and what its measurements usually look like, there is a best possible rule: pick the class that is most probable given what we measured. No other rule makes fewer mistakes on average.

Derivation

Let $p(\omega_k \mid \mathbf{x})$ be the probability that pattern $\mathbf{x}$ comes from class ω_k. Let L_{kj} be the **loss** incurred when a pattern that truly belongs to ω_k is assigned to ω_j. The expected loss of choosing class ω_j, called the **conditional risk**, is

$$r_j(\mathbf{x}) = \sum_{k=1}^{W} L_{kj}\, p(\omega_k \mid \mathbf{x}).$$

Bayes' rule, $p(\omega_k \mid \mathbf{x}) = p(\mathbf{x} \mid \omega_k)\, P(\omega_k)/p(\mathbf{x})$, rewrites this in terms of the **class-conditional density** $p(\mathbf{x} \mid \omega_k)$ (what patterns of class ω_k look like) and the **prior** $P(\omega_k)$ (how common the class is). The factor $1/p(\mathbf{x})$ is the same for every j, so it can be dropped when comparing risks.

The classifier that picks the j with the smallest $r_j(\mathbf{x})$ for every $\mathbf{x}$ minimizes the total average loss. That is why it is called **optimal**: it is the best any rule can do, given the true distributions. With the common **0–1 loss** ($L_{kj} = 0$ if $k = j$, otherwise 1), every error costs the same and

$$r_j(\mathbf{x}) \propto p(\mathbf{x}) - p(\mathbf{x} \mid \omega_j) P(\omega_j).$$

Minimizing it means maximizing the decision function

$$d_j(\mathbf{x}) = p(\mathbf{x} \mid \omega_j)\, P(\omega_j).$$

In words: choose the class with the largest posterior probability. The catch is that we never know the true densities. We must estimate them from training data, and the classifier is only as good as those estimates.

Bayes classifier for Gaussian pattern classes

Assume each class is a multivariate Gaussian with mean $\mathbf{m}_j$ and covariance matrix $\mathbf{C}_j$:

$$p(\mathbf{x} \mid \omega_j) = \frac{1}{(2\pi)^{n/2}|\mathbf{C}_j|^{1/2}} \exp\Big[-\tfrac{1}{2}(\mathbf{x}-\mathbf{m}_j)^\top \mathbf{C}_j^{-1}(\mathbf{x}-\mathbf{m}_j)\Big],$$

where n is the dimension of $\mathbf{x}$ and $|\mathbf{C}_j|$ the determinant of $\mathbf{C}_j$. Because the logarithm is increasing, we can maximize $\ln d_j$ instead and drop the constant $\frac{n}{2}\ln 2\pi$:

$$d_j(\mathbf{x}) = \ln P(\omega_j) - \tfrac{1}{2}\ln|\mathbf{C}_j| - \tfrac{1}{2}(\mathbf{x}-\mathbf{m}_j)^\top \mathbf{C}_j^{-1}(\mathbf{x}-\mathbf{m}_j).$$

The last term is half the squared **Mahalanobis distance**, a distance that stretches space so that each class's spread looks round. This d_j is quadratic in $\mathbf{x}$, so boundaries between classes are quadrics: ellipses, parabolas or hyperbolas in 2-D. Two special cases connect back to earlier sections:

- **Equal covariances** ($\mathbf{C}_j = \mathbf{C}$ for all j). The quadratic term $\mathbf{x}^\top\mathbf{C}^{-1}\mathbf{x}$ is common to all classes and cancels, leaving a linear function $d_j(\mathbf{x}) = \ln P(\omega_j) + \mathbf{x}^\top\mathbf{C}^{-1}\mathbf{m}_j - \frac{1}{2}\mathbf{m}_j^\top\mathbf{C}^{-1}\mathbf{m}_j$. Boundaries become hyperplanes.
- **Identity covariance and equal priors** ($\mathbf{C} = \mathbf{I}$, $P(\omega_j) = 1/W$). The function reduces to $\mathbf{x}^\top\mathbf{m}_j - \frac{1}{2}\mathbf{m}_j^\top\mathbf{m}_j$: the minimum-distance classifier. So minimum distance is the Bayes-optimal rule for round, equally spread, equally likely Gaussian classes, and only then.

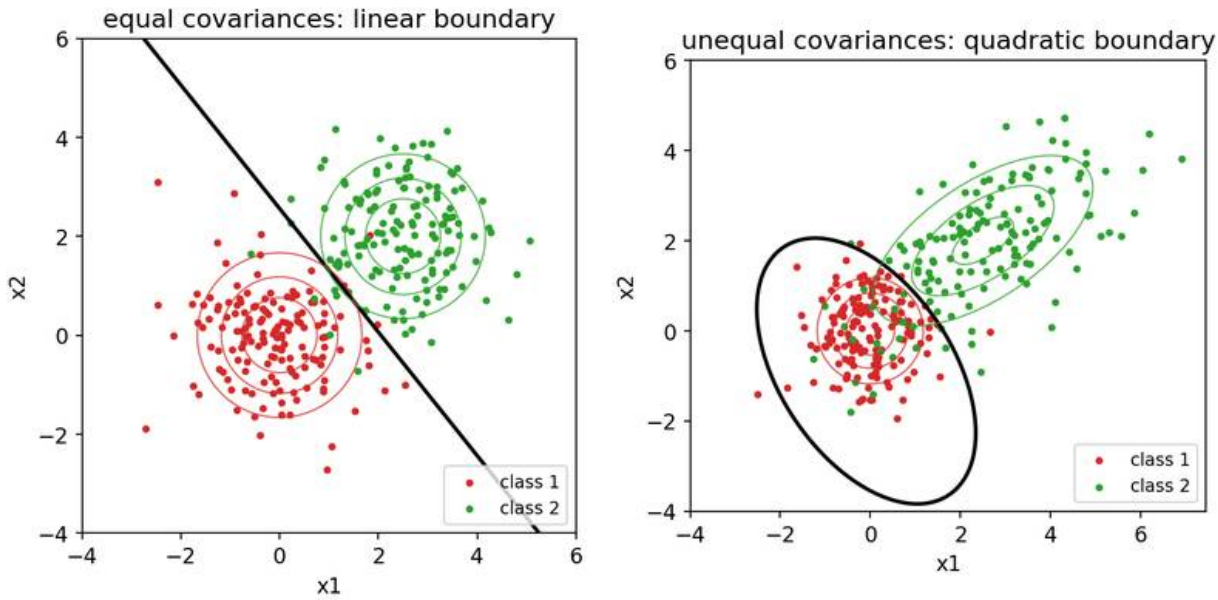


Figure 13.3 — Bayes decision boundaries (thick black) for two Gaussian classes. Thin contours show each class density. Left: equal covariances give a straight line. Right: unequal covariances give a curved (quadratic) boundary that wraps around the tighter class.

Estimating $\mathbf{m}_j$ and $\mathbf{C}_j$ from samples is easy: the sample mean and sample covariance. The priors can be the class frequencies in the training set, if those reflect reality.

```
import numpy as np
```

```
class GaussianBayes:
    def fit(self, X, y):
        self.classes = np.unique(y)
        self.m = [X[y == c].mean(0) for c in self.classes]
        self.C = [np.cov(X[y == c].T) for c in self.classes]
        self.logP = [np.log((y == c).mean()) for c in self.classes]
        return self

    def decision(self, X):
        scores = []
        for m, C, lp in zip(self.m, self.C, self.logP):
            d = X - m
            maha = np.einsum("ni,ij,nj->n", d, np.linalg.inv(C), d)
            scores.append(lp - 0.5 * np.linalg.slogdet(C)[1] - 0.5 * maha)
        return np.stack(scores, axis=1)            # (n, W): one d_j(x) per class

    def predict(self, X):
        return self.classes[self.decision(X).argmax(1)]
```

The number of covariance parameters grows as $n(n+1)/2$ per class. With 4 features that is 10 numbers per class, easily estimated. With 1,000 features it is about half a million, and the estimate becomes unreliable unless there is a huge amount of data. This is one reason the Gaussian Bayes classifier works best on short, well-chosen feature vectors.

Application: classifying pixels in a multispectral image

Remote-sensing satellites record each ground location in several spectral bands. Each pixel is then a natural pattern vector, $\mathbf{x} = (x_{\text{blue}}, x_{\text{green}}, x_{\text{red}}, x_{\text{NIR}})^{\top}$, where NIR is near-infrared. Healthy vegetation reflects strongly in NIR and weakly in red; water is dark in NIR; built-up areas are moderately bright everywhere.

We built a synthetic 128×128 scene with three classes (water, vegetation, built-up), gave each class a mean reflectance in four bands, and added independent Gaussian noise with standard deviation 0.06 to every band. We picked only 50 random pixels per class as training data, estimated $\mathbf{m}_j$ and $\mathbf{C}_j$, and classified every pixel with the Gaussian Bayes rule.

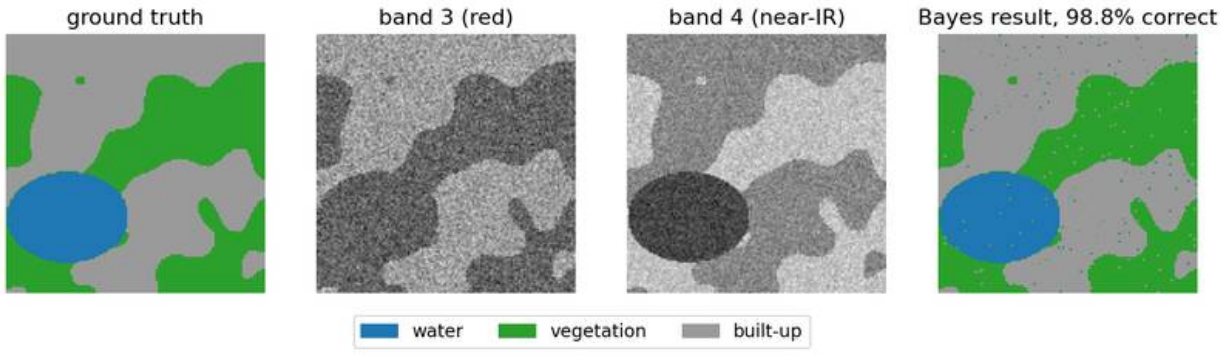


Figure 13.4 — Bayes classification of a synthetic four-band scene. Single bands are noisy and ambiguous (red barely separates water from vegetation), but the four-dimensional pattern vector separates the classes well: 98.8% of pixels are labelled correctly from only 150 training pixels.

Two lessons stand out. First, no single band is enough: in the red band, water and vegetation have similar values. The **combination** of bands separates them. Second, the remaining errors are isolated pixels. The Bayes rule here treats each pixel independently and ignores neighbors. Adding spatial context, for example by smoothing the decision scores or using a classifier that sees a neighborhood, removes most of those errors. That observation leads naturally to convolutional networks.

Neural networks and deep learning

Plain version. A neural network is a stack of very simple units. Each unit adds up its inputs with weights, then decides how strongly to "fire". By nudging the weights after each mistake, the network learns a decision rule on its own, without our writing down the probability distributions.

Background

The Bayes classifier needs density estimates, which are hard to get in high dimensions. An alternative is to **learn the decision functions directly** from training data. Neural networks do this with many simple, adjustable computing elements arranged in layers. The idea goes back to the perceptron of the late 1950s [4]. It became practical for multilayer networks once backpropagation was popularized in 1986 [5], and it became dominant for images after deep convolutional networks won the ImageNet challenge in 2012 [7]. For a fuller treatment of perceptrons, MLPs, activation functions and training tricks, see ML 02 ·Neural Networks.

The perceptron

A **perceptron** computes a weighted sum of its inputs plus a bias and outputs the sign:

$$d(\mathbf{x}) = \mathbf{w}^{\top}\mathbf{x} + w_0 = \sum_{k=1}^{n} w_k x_k + w_0, \qquad \text{class} = \begin{cases} \omega_1 & d(\mathbf{x}) > 0 \\ \omega_2 & d(\mathbf{x}) \le 0 \end{cases}$$

where $\mathbf{w} = (w_1, \dots, w_n)^{\top}$ are the **weights** and w_0 is the **bias**. The boundary $d(\mathbf{x}) = 0$ is a hyperplane. To simplify notation, append a constant 1 to every pattern, $\mathbf{y} = (x_1, \dots, x_n, 1)^{\top}$, and fold the bias into the weights, $\mathbf{w} = (w_1, \dots, w_n, w_0)^{\top}$, so that $d = \mathbf{w}^{\top}\mathbf{y}$.

Learning rule. Encode the label as $t = +1$ for ω_1 and $t = -1$ for ω_2. Present the training patterns one at a time. At step k:

$$\mathbf{w}(k+1) = \begin{cases} \mathbf{w}(k) + \alpha\, t\, \mathbf{y}(k) & \text{if } t\,\mathbf{w}(k)^{\top}\mathbf{y}(k) \le 0 \text{ (misclassified)} \\ \mathbf{w}(k) & \text{otherwise,} \end{cases}$$

where $\alpha > 0$ is the **learning rate**. Each correction moves the hyperplane so that the offending pattern is pushed toward its correct side: after the update, $t\,\mathbf{w}^\top\mathbf{y}$ increases by $\alpha\|\mathbf{y}\|^2$. If the two classes are **linearly separable**, this procedure is guaranteed to stop after a finite number of corrections with a separating hyperplane. If they are not, it never settles. In that case we instead minimize a smooth error, such as the squared difference between $\mathbf{w}^\top\mathbf{y}$ and t, by gradient descent (the least-mean-squares, or delta, rule). That idea leads directly to backpropagation.

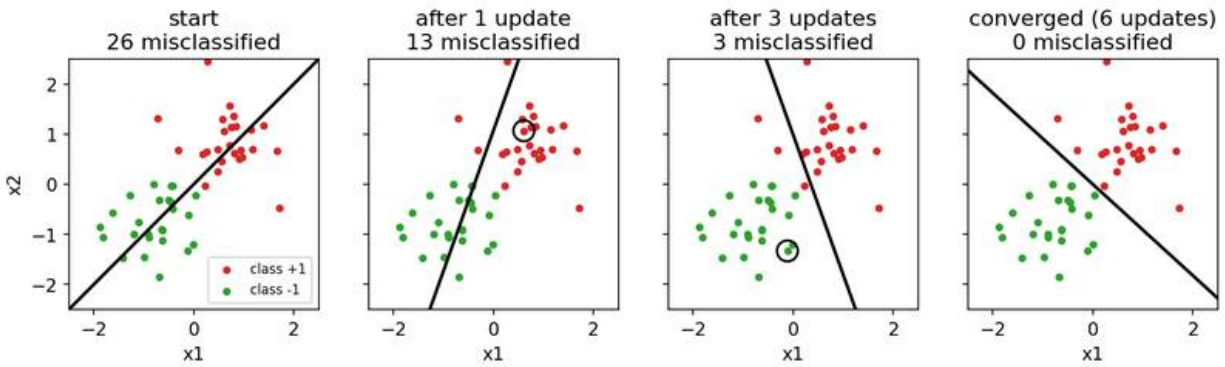


Figure 13.5 — Perceptron training on two separable classes. The circled point is the misclassified pattern that triggered the update. After six corrections the line separates the classes and learning stops.

```
import numpy as np

def train_perceptron(X, y, alpha=0.5, epochs=100):
    """X: (n, d), y in {-1, +1}. Returns augmented weights (w_1..w_d, w_0)."""
    Xa = np.c_[X, np.ones(len(X))]                  # append the constant input 1
    w = np.zeros(Xa.shape[1])
    for _ in range(epochs):
        mistakes = 0
        for xi, yi in zip(Xa, y):
            if yi * (w @ xi) <= 0:                  # wrong side (or on the line)
                w += alpha * yi * xi
                mistakes += 1
        if mistakes == 0:                           # a full clean pass: converged
            break
    return w
```

A single perceptron can only draw one hyperplane. It cannot solve XOR, where the two classes sit on opposite corners of a square. To draw curved or multi-piece boundaries we need more layers.

Multilayer feedforward networks

A **multilayer feedforward network** (or multilayer perceptron, MLP) stacks layers of units. Layer ℓ receives the outputs of layer $\ell - 1$, and nothing flows backward during prediction. Unlike the perceptron, each unit passes its weighted sum through a smooth, differentiable **activation function** h, such as the sigmoid $h(z) = 1/(1 + e^{-z})$, the hyperbolic tangent, or the rectified linear unit

$\mathrm{ReLU}(z) = \max(0, z)$. Smoothness is what makes gradient-based training possible. Nonlinearity is what makes depth useful: a stack of purely linear layers collapses into one linear layer.

With two hidden layers and enough units, a network can form decision regions of essentially any shape. In practice the question is not whether a network **can** represent a boundary, but whether training will **find** it from the data available.

Forward pass

Number the layers $\ell = 1, \dots, L$, with layer 0 being the input $\mathbf{a}^{(0)} = \mathbf{x}$. For each layer,

$$\mathbf{z}^{(\ell)} = \mathbf{W}^{(\ell)}\mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}, \qquad \mathbf{a}^{(\ell)} = h(\mathbf{z}^{(\ell)}),$$

where $\mathbf{W}^{(\ell)}$ is the weight matrix (row i holds the weights into unit i), $\mathbf{b}^{(\ell)}$ the bias vector, $\mathbf{z}^{(\ell)}$ the **net inputs** and $\mathbf{a}^{(\ell)}$ the **activations**, with h applied element by element. For a W-class problem the output layer has W units. Its net inputs are turned into probabilities with the **softmax** function,

$$p_j = \frac{e^{z_j^{(L)}}}{\sum_{i=1}^{W} e^{z_i^{(L)}}}, \qquad j = 1, \dots, W,$$

and the predicted class is the one with the largest p_j. Notice that the output layer is a set of decision functions $d_j(\mathbf{x}) = z_j^{(L)}$, exactly as in the Background section. The hidden layers compute features in which those simple, linear decision functions suffice.

Backpropagation

Training adjusts all weights to reduce a **loss** E that measures how wrong the outputs are. For classification the standard choice is the **cross-entropy** $E = -\sum_j t_j \ln p_j$, where $\mathbf{t}$ is the one-hot label (1 for the true class, 0 elsewhere). For a sigmoid output the classic alternative is the squared error $E = \frac{1}{2}\|\mathbf{a}^{(L)} - \mathbf{t}\|^2$. We need $\partial E / \partial \mathbf{W}^{(\ell)}$ for every layer. Backpropagation computes all of them with one backward sweep of the chain rule [5].

Define the **error signal** of layer ℓ as $\boldsymbol{\delta}^{(\ell)} = \partial E / \partial \mathbf{z}^{(\ell)}$.

Step 1: output layer. For softmax with cross-entropy the derivative is remarkably simple:

$$\boldsymbol{\delta}^{(L)} = \mathbf{p} - \mathbf{t}.$$

(For a sigmoid output with squared error it is $(\mathbf{a}^{(L)} - \mathbf{t}) \odot h'(\mathbf{z}^{(L)})$, where $\odot$ is element-wise multiplication.)

Step 2: propagate backward. Unit i of layer ℓ affects E only through the net inputs of layer $\ell + 1$, and $\partial z_k^{(\ell+1)} / \partial a_i^{(\ell)} = W_{ki}^{(\ell+1)}$. The chain rule gives

$$\boldsymbol{\delta}^{(\ell)} = h'(\mathbf{z}^{(\ell)}) \odot \left(\mathbf{W}^{(\ell+1)\top}\boldsymbol{\delta}^{(\ell+1)}\right), \qquad \ell = L-1, \dots, 1.$$

The errors travel backward through the same weights that carried the signal forward, transposed.

Step 3: gradients. Because $z_i^{(\ell)} = \sum_k W_{ik}^{(\ell)} a_k^{(\ell-1)} + b_i^{(\ell)}$,

$$\frac{\partial E}{\partial \mathbf{W}^{(\ell)}} = \delta^{(\ell)}\, \mathbf{a}^{(\ell-1)\top}, \qquad \frac{\partial E}{\partial \mathbf{b}^{(\ell)}} = \delta^{(\ell)}.$$

Each weight's gradient is "error at its output end" times "activation at its input end".

Step 4: update. Gradient descent with learning rate α:

$$\mathbf{W}^{(\ell)} \leftarrow \mathbf{W}^{(\ell)} - \alpha \frac{\partial E}{\partial \mathbf{W}^{(\ell)}}, \qquad \mathbf{b}^{(\ell)} \leftarrow \mathbf{b}^{(\ell)} - \alpha \frac{\partial E}{\partial \mathbf{b}^{(\ell)}}.$$

The cost of the backward pass is about the same as the forward pass, which is why networks with millions of weights can be trained at all. The perceptron rule is the special case of one layer with a step activation.

Training

In practice we average the gradient over a small random **mini-batch** of training patterns, update, and repeat. This is **stochastic gradient descent** (SGD). One pass through the whole training set is an **epoch**. Training runs for many epochs while we watch the loss on the validation set. The learning rate is the most important setting: too large and the loss oscillates or explodes; too small and training crawls. Weights must start as small random numbers, not zeros, or all units in a layer would compute the same thing and receive the same updates forever.

The following NumPy program trains a two-layer network on XOR, the problem a single perceptron cannot solve:

```
import numpy as np

rng = np.random.default_rng(0)
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], float)
y = np.array([[0], [1], [1], [0]], float)         # XOR: not linearly separable

W1, b1 = rng.normal(0, 1, (2, 8)), np.zeros(8)    # input -> 8 hidden units
W2, b2 = rng.normal(0, 1, (8, 1)), np.zeros(1)    # hidden -> 1 output
sig = lambda z: 1 / (1 + np.exp(-z))
alpha = 1.0

for step in range(5000):
    # forward pass
    a1 = np.tanh(X @ W1 + b1)
    out = sig(a1 @ W2 + b2)
    # backward pass (cross-entropy loss with a sigmoid output)
    d2 = (out - y) / len(X)                        # delta at the output layer
    d1 = (d2 @ W2.T) * (1 - a1 ** 2)               # delta at the hidden layer
    W2 -= alpha * a1.T @ d2;  b2 -= alpha * d2.sum(0)
    W1 -= alpha * X.T @ d1;   b1 -= alpha * d1.sum(0)

print(out.round(3).ravel())                        # close to [0, 1, 1, 0]
```

(Here patterns are rows, so the matrix products are transposed relative to the equations above.)

Deep convolutional neural networks

Plain version. Instead of telling the network which features to measure, we let it slide small learnable filters over the image, exactly like the filters of Chapter 3, and learn which filters help it classify. Early layers end up finding edges and blobs; later layers combine them into parts and whole objects.

Why fully connected networks struggle with images

Feed a 20×20 image into a fully connected layer with 2,048 hidden units, and that one layer has $400 \times 2048 = 819{,}200$ weights. The layer also ignores the fact that nearby pixels are related and that a "3" shifted two pixels to the right is still a "3". A CNN builds both facts into its structure [6].

Convolutional layers

A **convolutional layer** applies F small kernels (for example 5×5) to its input. With C input channels a_c, the net input of output channel f at pixel (x, y) is

$$z_f(x,y) = \sum_{c=1}^{C} \sum_{s,t} w_{f,c}(s,t)\, a_c(x+s, y+t) + b_f,$$

followed by an activation, usually ReLU: $a_f(x,y) = \max(0, z_f(x,y))$. The sum is a spatial correlation, as in Chapter 3, but the kernel values $w_{f,c}$ are **learned**. Each output channel is a **feature map**: it is large where its kernel's pattern appears. Two ideas cut the parameter count dramatically:

- **Local connectivity.** Each output depends only on a small neighborhood (its **receptive field**).
- **Weight sharing.** The same kernel is used at every position, so a feature learned in one place is detected everywhere.

Eight 5×5 kernels on a single-channel image need $8 \times 25 + 8 = 208$ parameters, against the 819,200 above.

Pooling layers

A **pooling layer** shrinks each feature map, typically by taking the maximum (max-pooling) or average of each non-overlapping 2×2 block. It halves the width and height, makes the response tolerant to small shifts, and enlarges the receptive field of later layers. Stacking convolution and pooling several times yields features that respond to larger and more abstract structures.

Fully connected layers and the whole pipeline

After the last pooling layer the feature maps are **flattened** into one vector and passed through one or more fully connected layers ending in a softmax. Figure 13.6 shows the small network we trained.

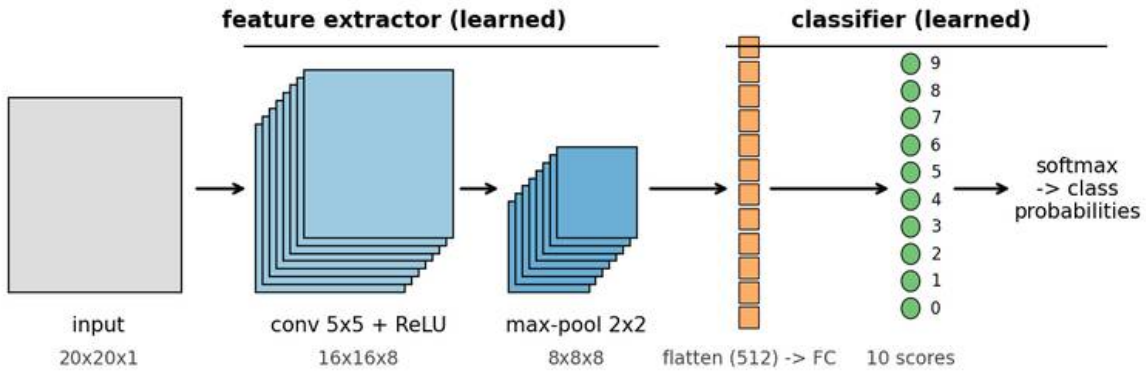


Figure 13.6 — The tiny CNN used in this chapter. The convolution and pooling layers form a learned feature extractor; the fully connected layer is a learned linear classifier on those features.

This is the key conceptual point of the chapter. The classical pipeline was **hand-designed features → classifier**. A CNN is **learned features → classifier**, with both parts trained together by backpropagation to minimize the same loss. The fully connected softmax layer at the end is just a set of linear decision functions, the same object we met in the minimum-distance and equal-covariance Bayes classifiers. What changed is that the space in which those linear functions operate is itself learned.

Training a CNN

Backpropagation works unchanged; only the local derivatives differ:

- **Convolution.** Because a weight is shared over all positions, its gradient is the **sum** over positions: $\partial E / \partial w_{f,c}(s,t) = \sum_{x,y} \delta_f(x,y)\, a_c(x+s, y+t)$, which is itself a correlation of the input with the error map. The error sent back to the input is a convolution of δ_f with the (flipped) kernel.
- **Max-pooling.** The gradient flows only to the input that was the maximum in each block; the others get zero.
- **ReLU.** The gradient passes where $z > 0$ and is blocked where $z \le 0$.

Here is the forward pass of one convolution, ReLU and pooling stage with SciPy:

```
import numpy as np
from scipy.signal import correlate2d
from skimage import data, transform
```

```
img = transform.resize(data.camera(), (64, 64), anti_aliasing=True)
k = np.array([[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]], float)    # one 3x3 "filter"
fmap = np.maximum(correlate2d(img, k, mode="valid") + 0.0, 0)  # conv + bias + ReLU
pooled = fmap[:62, :62].reshape(31, 2, 31, 2).max(axis=(1, 3))  # 2x2 max-pooling
print(img.shape, "->", fmap.shape, "->", pooled.shape)
```

Worked example: digits

The classic test bed for CNNs is handwritten digit recognition, where LeNet-style networks were developed on the MNIST digits [6]. To keep everything self-contained and reproducible, we generate **MNIST-like** 20 × 20 digits: each digit 0–9 is drawn with one of six OpenCV fonts at a random size, stroke width, shift and rotation ($\pm 15°$), then noise is added (Figure 13.7).

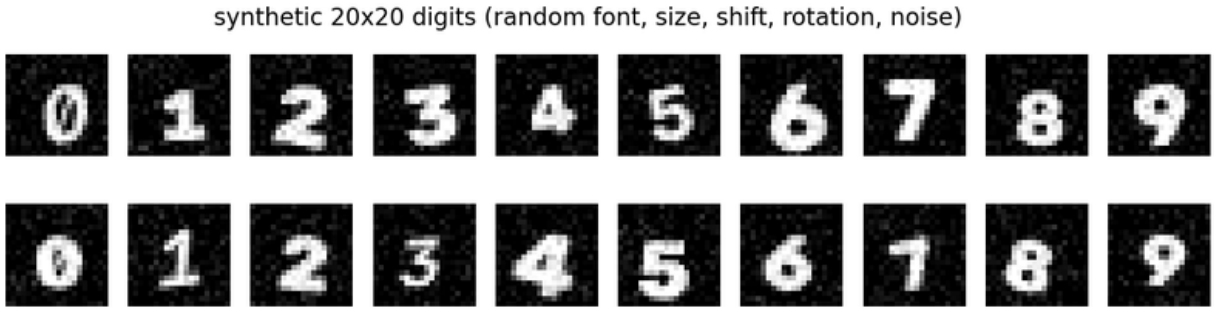


Figure 13.7 — Synthetic training digits. Variation in font, size, position and angle forces the classifier to learn shape rather than memorize pixels.

The network of Figure 13.6 (eight 5×5 filters, ReLU, 2×2 max-pooling, a fully connected layer to 10 outputs, softmax with cross-entropy) is written in about 80 lines of NumPy in `scripts/figures/dip_ch13.py`. We trained it with mini-batch SGD (batch size 32, learning rate 0.05, small weight decay) on 3,000 digits for 12 epochs. It reached **98.5%** accuracy on 1,000 separate validation digits. Figure 13.8 shows what it learned.

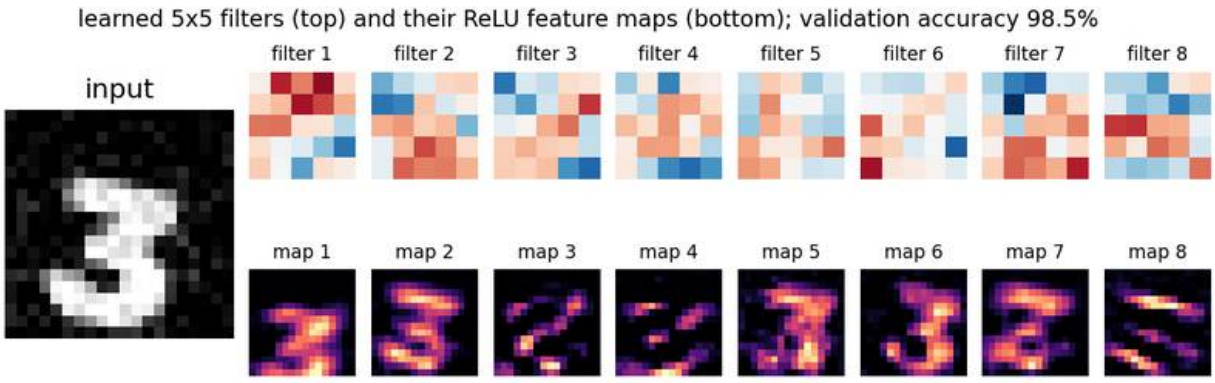


Figure 13.8 — Inside the trained CNN. Top: the eight learned 5×5 kernels (red positive, blue negative).

Bottom: their ReLU feature maps for the input digit. Several kernels act as oriented edge or stroke detectors, similar to the hand-designed gradient filters of Chapter 10.

Nobody told the network to detect edges. Oriented, edge-like kernels emerge because they are useful for telling digits apart. Large networks trained on natural images show the same behavior in their first layer, with progressively more complex patterns in deeper layers.

Some additional details of implementation

Plain version. A network that memorizes its homework can still fail the exam. Most of the practical craft of training is about making it learn the general rule instead of the specific examples.

Overfitting

A network with many parameters and few training examples can drive its training error to zero by memorizing. Its error on new data then stays high. This gap between training and validation performance is **overfitting**. Figure 13.9 (left) shows it clearly: trained on only 150 digits, the CNN reaches 100% training accuracy but only 86.2% validation accuracy.

The basic defenses:

- **More data.** Nothing helps as reliably. Data augmentation (below) is a cheap way to get more.
- **A validation set and early stopping.** Track validation accuracy and keep the weights from the best epoch.
- **Smaller or better-structured models.** Weight sharing in CNNs is itself a strong regularizer compared with fully connected layers.

Data augmentation

Data augmentation creates new training examples by applying label-preserving transformations: small shifts, rotations, scaling, flips (where they make sense: a flipped "cat" is still a cat, but a flipped "b" becomes a "d"), crops, brightness and contrast changes, and added noise [18]. Each epoch sees a slightly different version of each image, so memorizing is harder and invariance to those transformations is learned from data.

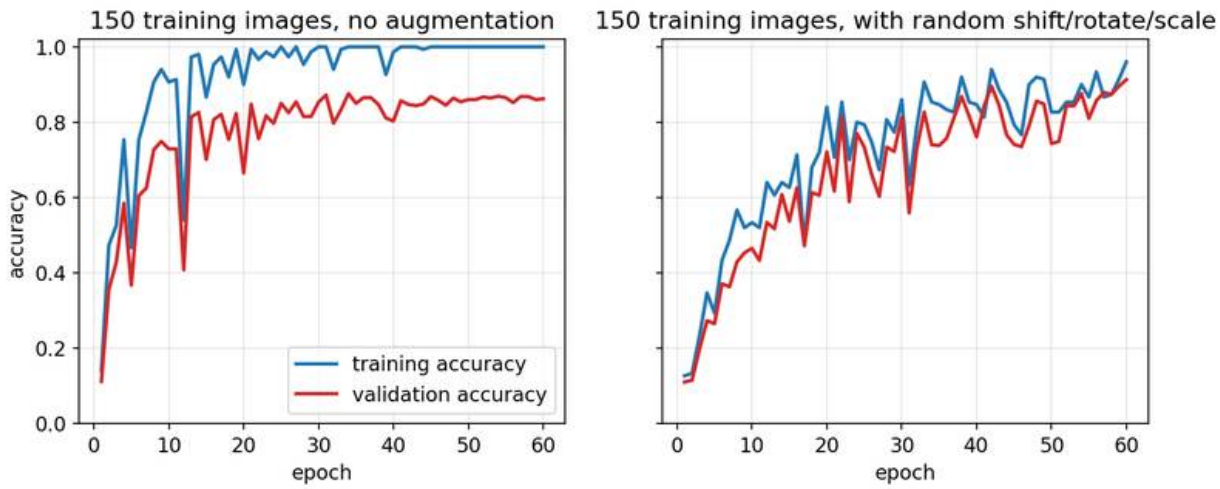


Figure 13.9 — The same CNN trained on only 150 synthetic digits for 60 epochs. Left: without augmentation, a large train–validation gap opens (overfitting). Right: random shifts, rotations and scaling keep the curves together, and final validation accuracy rises from 86.2% to 91.3%.

Regularization

Regularization adds a preference for simpler solutions:

- **Weight decay** (L_2 regularization) adds $\frac{\lambda}{2}\sum w^2$ to the loss, which adds λw to every weight's gradient and shrinks weights toward zero. Our training script uses $\lambda = 10^{-4}$.
- **Dropout** randomly sets a fraction of the units to zero at each training step, so no unit can rely on specific partners. At test time all units are used, with appropriately scaled weights. This behaves like averaging an ensemble of many thinned networks [17].

Other practical details matter just as much: normalizing inputs (zero mean, unit variance), initializing weights with a variance scaled to the number of inputs (so activations neither explode nor vanish through the layers), lowering the learning rate during training, and always reporting final results on a test set that was not used for any decision during development.

Modern view

The textbook chapter ends where modern image classification begins. Six survey and review papers map how the field has moved since.

Statistical pattern recognition before deep learning. Jain, Duin and Mao's review [2] is still the clearest map of the classical territory: representation, feature extraction and selection, Bayes and nearest-neighbor rules, linear and nonlinear classifiers, classifier combination, and error estimation. Its main takeaway, that the feature representation and the evaluation protocol matter

more than the choice of classifier, aged well. Deep learning did not refute it; it automated the representation part.

The CNN era. Rawat and Wang [11] review CNNs for image classification from their origins in the late 1980s to 2017, organizing more than 300 papers by architecture, regularization, optimization and practical tricks. The line they trace starts with LeCun et al.'s gradient-trained networks for document recognition [6]. It accelerates with AlexNet [7], which used GPUs, ReLU, dropout and data augmentation to win the ImageNet Large Scale Visual Recognition Challenge in 2012 [10]. VGG [8] showed that depth built from small 3×3 kernels helps. ResNet [9] made very deep networks trainable by learning **residual** functions through identity shortcut connections. Each of these is still recognizably the architecture of Figure 13.6: convolution, nonlinearity, pooling, a linear classifier, trained by backpropagation.

Transformers in vision. The Vision Transformer (ViT) [12] cuts an image into 16×16 patches, treats each patch as a token and processes the sequence with self-attention instead of convolution. With enough pre-training data it matches or exceeds CNNs. Khan et al.'s survey [13] covers the many vision transformer variants for classification, detection, segmentation and more, and discusses their dependence on large-scale pre-training. ConvNeXt [14] then showed that a pure CNN, modernized step by step with design choices borrowed from transformers, is competitive again. The practical lesson: architecture families matter less than training data, scale and recipe.

Foundation features and zero-shot classification. Two 2021–2023 developments changed what "training a classifier" means. CLIP [15] trains an image encoder and a text encoder together so that matching image–caption pairs have similar embeddings. A new classifier can then be built without any labelled images: embed prompts such as `a photo of a {class}`, and assign each image to the class whose text embedding is most similar. That is a **minimum-distance classifier** whose prototypes come from language. DINOv2 [16] learns general-purpose visual features by self-supervision on a large curated image set, without labels. Its frozen features support strong classification with only a linear layer or a nearest-neighbor rule on top. In both cases the expensive part, the feature extractor, is trained once. The classifier on top is often one of the simple rules from the first half of this chapter.

Data augmentation and regularization. Shorten and Khoshgoftaar's survey [18] organizes augmentation methods into basic image manipulations (geometric and color transformations, kernel filters, mixing images, random erasing) and learned approaches (adversarial training, GAN-based synthesis, neural style transfer, meta-learning of augmentation policies). Its takeaway matches Figure 13.9: augmentation is one of the cheapest and most effective ways to reduce overfitting when labelled data are limited.

Evaluation pitfalls and dataset bias. Torralba and Efros [19] asked a simple question: can a classifier tell which **dataset** an image came from? It could, well above chance, showing that each benchmark has its own "signature". Models trained on one dataset also generalized poorly to another. Liu and He [20] repeated the experiment with modern networks and very large, diverse web-scale datasets. Modern networks still identify the source dataset with high accuracy on held-out images, and they do it with generalizable features rather than memorization. Dataset bias did not disappear with scale. Two practical consequences follow. Accuracy on one benchmark is a statement about that benchmark, not about the visual world. And whenever possible, test on

data collected independently of the training data.

What changed and what did not. Deep learning replaced hand-crafted features and made the classifier itself almost an afterthought: a linear softmax layer. It did not change the decision-theoretic foundations. The softmax output is trained to estimate posterior probabilities, so the network approximates the Bayes rule. Cross-entropy is the negative log-likelihood. Prototype matching survives as nearest-neighbor search in embedding spaces, used in zero-shot classification and retrieval. Template matching by normalized correlation is still the right tool in controlled industrial inspection, where the object's appearance is fixed and training data are scarce. Understanding the classical methods tells you what the modern ones are optimizing and when a simpler method is enough.

Key takeaways

- Classification assigns a label to a pattern via decision functions $d_j(\mathbf{x})$; the boundaries are where two decision functions are equal.
- Minimum-distance classification uses class means as prototypes; its boundaries are perpendicular bisectors. Normalized correlation and SIFT matching are prototype matching for images and keypoints.
- The Bayes classifier minimizes the average loss. For Gaussian classes it is quadratic; with equal covariances it is linear; with identity covariance and equal priors it reduces to minimum distance.
- A perceptron learns a separating hyperplane by correcting mistakes. Multilayer networks with smooth activations learn nonlinear boundaries, and backpropagation computes all gradients in one backward pass.
- A CNN is a learned feature extractor (convolution, nonlinearity, pooling) followed by a learned linear classifier. Weight sharing makes it efficient and shift-tolerant.
- Overfitting is the central practical risk. Use validation data, data augmentation, weight decay and dropout, and keep the test set untouched.
- Modern systems often pair a large pre-trained feature extractor (CLIP, DINOv2) with a very simple classifier. Benchmark accuracy reflects dataset biases, so test on independently collected data.

Exercises

1. **Bisector by hand.** Two class means are $\mathbf{m}_1 = (1,2)^\top$ and $\mathbf{m}_2 = (5,4)^\top$. Write the minimum-distance boundary $d_1(\mathbf{x}) - d_2(\mathbf{x}) = 0$ as an equation in x_1, x_2, and verify that the midpoint of the two means lies on it.

   Hint

   $\mathbf{m}_1 - \mathbf{m}_2 = (-4,-2)^\top$ and
   $\frac{1}{2}(\mathbf{m}_1 - \mathbf{m}_2)^\top(\mathbf{m}_1 + \mathbf{m}_2) = \frac{1}{2}(-4 \cdot 6 - 2 \cdot 6) = -18$. The boundary is $-4x_1 - 2x_2 + 18 = 0$, i.e. $2x_1 + x_2 = 9$. The midpoint $(3,3)$ gives $6 + 3 = 9$. ✓

2. **When priors matter.** In a one-dimensional problem, class ω_1 is Gaussian with mean 0 and class ω_2 with mean 4, both with variance 1. Find the Bayes threshold when $P(\omega_1) = P(\omega_2)$, and when $P(\omega_1) = 0.9$. In which direction does the threshold move, and why does that make sense?

   Hint

   Set $\ln P_1 - \frac{1}{2}x^2 = \ln P_2 - \frac{1}{2}(x-4)^2$. This gives $x = 2 + \frac{1}{4}\ln(P_1/P_2)$. Equal priors give $x = 2$; $P_1 = 0.9$ gives $x = 2 + \frac{1}{4}\ln 9 \approx 2.55$. The threshold moves toward the rarer class, so more ambiguous patterns go to the common class.

3. **Correlation invariance.** Show that the normalized correlation coefficient γ is unchanged if every pixel of the image patch is replaced by $af + b$ with $a > 0$. What happens if $a < 0$? Then test it with `cv2.matchTemplate` on `skimage.data.camera()`.

   Hint

   Subtracting the mean removes b. The factor a appears once in the numerator and as $\sqrt{a^2} = |a|$ in the denominator, so it cancels for $a > 0$. For $a < 0$ (a photographic negative) the sign of γ flips: a perfect match becomes -1.

4. **A perceptron that never stops.** Generate two overlapping Gaussian clouds and run the perceptron rule from this chapter for 100 epochs. Plot the number of mistakes per epoch. Then replace the rule by gradient descent on the squared error $\frac{1}{2}\sum(\mathbf{w}^\top \mathbf{y} - t)^2$ and compare the two learning curves.

   Hint

   The perceptron's mistake count keeps fluctuating, because every mistake moves the line, even when no line can be perfect. The least-squares rule converges to a fixed compromise line, since its loss is smooth and convex.

5. **Counting parameters.** A network takes 64×64 grayscale images. Compare the number of weights (including biases) in (a) a fully connected layer with 1,024 hidden units and (b) a convolutional layer with 32 kernels of size 3×3. How many outputs does (b) produce without padding?

   Hint

   (a) $4096 \times 1024 + 1024 = 4{,}195{,}328$. (b) $32 \times 9 + 32 = 320$ parameters, producing 32 maps of 62×62, i.e. 123,008 outputs. Few parameters, many outputs: that is weight sharing.

6. **Augmentation that hurts.** Using the digit generator in `scripts/figures/dip_ch13.py`, add random 180° rotation to the augmentation. Retrain and inspect the confusion between which digits rises the most. Explain why.

Hint

A "6" rotated by 180° looks like a "9" (and "0", "1", "8" look like themselves). The augmentation is no longer label-preserving, so the network is taught contradictory labels for the same shape. Expect the 6/9 confusion to rise sharply.

References

1. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018, Ch. 13. publisher page
2. A. K. Jain, R. P. W. Duin, and J. Mao, "Statistical Pattern Recognition: A Review," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 1, pp. 4–37, 2000. doi:10.1109/34.824819
3. D. G. Lowe, "Distinctive Image Features from Scale-Invariant Keypoints," *International Journal of Computer Vision*, vol. 60, 2004. doi:10.1023/B:VISI.0000029664.99615.94
4. F. Rosenblatt, "The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain," *Psychological Review*, vol. 65, 1958. doi:10.1037/h0042519
5. D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning Representations by Back-Propagating Errors," *Nature*, vol. 323, 1986. doi:10.1038/323533a0
6. Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-Based Learning Applied to Document Recognition," *Proceedings of the IEEE*, vol. 86, 1998. doi:10.1109/5.726791
7. A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," *Advances in Neural Information Processing Systems 25 (NIPS)*, 2012. paper
8. K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," arXiv:1409.1556, 2014. arXiv
9. K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," arXiv:1512.03385, 2015. arXiv
10. O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, et al., "ImageNet Large Scale Visual Recognition Challenge," arXiv:1409.0575, 2014. arXiv
11. W. Rawat and Z. Wang, "Deep Convolutional Neural Networks for Image Classification: A Comprehensive Review," *Neural Computation*, vol. 29, no. 9, 2017. doi:10.1162/neco_a_00990
12. A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," *ICLR*, 2021. arXiv
13. S. Khan, M. Naseer, M. Hayat, S. W. Zamir, F. S. Khan, and M. Shah, "Transformers in Vision: A Survey," *ACM Computing Surveys*, 2022. arXiv
14. Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A ConvNet for the 2020s," *CVPR*, 2022. arXiv
15. A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, et al., "Learning Transferable Visual Models From Natural Language Supervision," arXiv:2103.00020, 2021. arXiv
16. M. Oquab, T. Darcet, T. Moutakanni, et al., "DINOv2: Learning Robust Visual Features without Supervision," arXiv:2304.07193, 2023. arXiv
17. N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," *Journal of Machine Learning Research*,

vol. 15, pp. 1929–1958, 2014. JMLR
18. C. Shorten and T. M. Khoshgoftaar, “A Survey on Image Data Augmentation for Deep Learning,” *Journal of Big Data*, vol. 6, 2019. doi:10.1186/s40537-019-0197-0
19. A. Torralba and A. A. Efros, “Unbiased Look at Dataset Bias,” *CVPR*, 2011. paper page
20. Z. Liu and K. He, “A Decade’s Battle on Dataset Bias: Are We There Yet?,” *ICLR*, 2025. arXiv

Part III

Natural Language Processing

NLP 01 • Introduction to Natural Language Processing

What NLP is, why language is hard for computers, how the field moved from rules to statistics to pretrained models, the main tasks, Chinese-specific issues, and a practical pipeline from task definition to error diagnosis.

Online edition: https://blog.shyandram.dev/en/learn/nlp/nlp-01-introduction/

What you'll learn

- Explain why language is hard for computers, with an example of ambiguity at each linguistic level.
- Trace the main eras of NLP — rules, statistical learning, neural networks and pretrained models — through the papers that marked them.
- Classify common NLP tasks by the shape of their output and find the tutorial that covers each.
- Recognize the issues specific to Chinese text: word segmentation, and the Traditional and Simplified character sets.
- Define an NLP task through its input, output and required linguistic evidence, and preserve that evidence through normalization and tokenization.
- Build data splits and diagnose errors at the stage where they originate.

The big picture

Natural language processing (NLP) is the study of computer systems that analyze, interpret or produce human language. An NLP pipeline converts a particular language problem into a computation. Consider "Pump B: not ready." A maintenance application might need the equipment name and its stated readiness. A message-routing application might need only a category. A conversational assistant might need to ask what remains unfinished. These are different tasks even though the input string is identical.

The first job is to decide which answer the application needs and what evidence supports it. Then preserve that evidence while turning text into model input. A powerful model cannot recover a distinction that preprocessing has already erased, and a plausible label is useless if it describes the wrong target.

> **Explain like I'm five**
>
> Imagine sorting letters into trays. You first decide what each tray means. Then you read the letters without rubbing out important words. Turning "the parcel did not arrive" into "the parcel arrived" makes the sorting problem wrong before you even choose a sorter.

Hagiwara's application-first treatment and McRoy's treatment of language and its data structures provide the two anchors for this tutorial [1, 2]. We first look at why language is difficult, how the field arrived at today's methods, which tasks it solves and what changes for Chinese text. Then we follow one practical route: define the task, preserve text, choose computational units, construct data and evaluate the result. NLP 02 develops the language structures behind those decisions, and the roadmap at the end links the whole track.

1. Why language is hard for computers

Plain version. The same string of characters can mean different things, and what it means depends on structure, context and the speaker's purpose. A computer sees only the characters unless we give it the rest.

Ambiguity appears at every level of linguistic analysis. Each row below is developed with examples and code in NLP 02:

Level	Example	Two readings
Word boundaries	學生會說話	"students can speak" / "the student council speaks"
Word category	"the pilots can land" / "the metal can fell"	"can" as an auxiliary / as a noun
Word sense	"window"	an opening in a wall / an area on a screen
Attachment	"The technician inspected the panel with the camera."	the camera is the tool / the camera is on the panel
Scope	"Every technician uses a camera."	possibly different cameras / one shared camera
Reference	"Mei put a camera beside a lamp. It was overheating."	"it" is the camera / the lamp
Intention	"Can you lower the gain?"	a question about ability / a request to act

People resolve most of these without noticing, using the surrounding sentence, the situation and knowledge of the world. A program must either be given that context or learn to approximate it from data. Three further properties compound the difficulty.

- **Productivity.** Speakers build new words and sentences from familiar parts all the time, so no finite list of inputs covers the language.
- **Skewed frequency.** A few words are extremely common and most words are rare. NLP 02 measures this with Zipf's law: in one corpus, a third of all word types occur exactly once. Rare words are where models have the least evidence.
- **Variation.** Dialects, registers, spelling conventions, mixed-language messages and writing systems all differ. A system trained on one variety can fail quietly on another.

None of this means NLP is impossible. It means every system makes choices about which distinctions it preserves, and good engineering makes those choices explicit.

2. A short history: rules, statistics, neural networks and pretraining

Plain version. Early systems followed rules that people wrote by hand. Later systems learned probabilities from large collections of text. Neural networks then learned their own features, and today most systems start from a large model pretrained on unlabeled text and adapt it to the task.

Figure 1 — Milestones cited in this tutorial, grouped into four approximate eras. Events are equally spaced, not to scale; in practice the approaches overlap.

Rules and hand-built knowledge. Weizenbaum's ELIZA (1966) responded to typed English by matching keywords and applying decomposition and reassembly rules from a script; its best-known script imitated a psychotherapist's conversational style [6]. Systems of this era encoded grammars, lexicons and patterns by hand. They could be precise inside their design, but every new phrasing or domain required new rules.

Statistical learning from corpora. Over the following decades, researchers increasingly estimated models from data instead of writing every rule. Brown and colleagues (1993) described a series of statistical translation models whose parameters are estimated from pairs of sentences that are translations of each other [7]. In the same year the Penn Treebank released a large English corpus annotated with part-of-speech tags and syntactic structure [8], giving taggers and parsers shared training data and a shared test. The key change was that the evidence came from counts in corpora, and evaluation on held-out data became standard practice.

Neural networks. Bengio and colleagues (2003) trained a neural language model that learned a distributed representation for each word together with the probability of word sequences [9]. Collobert and colleagues (2011) used one neural architecture for part-of-speech tagging, chunking, named-entity recognition and semantic role labeling, learning internal representations largely from unlabeled text instead of hand-engineered features [10]. Mikolov and colleagues (2013) made word vectors cheap to train on large corpora [11]. Models still tended to be built for one task at a time.

Pretrained models and large language models. The Transformer (2017) replaced recurrence with attention [12]. ELMo (2018) supplied context-dependent word representations from a pretrained bidirectional language model [13], and BERT (2019) pretrained a deep bidirectional Transformer on unlabeled text and then fine-tuned it for each task with a small output layer [14]. GPT-3 (2020) showed that a large autoregressive model could perform many tasks from a few examples in its prompt, without gradient updates [15], and InstructGPT (2022) fine-tuned such a model with human feedback to follow instructions [16]. Zhao and colleagues' survey organizes the large-language-model literature around pretraining, adaptation, utilization and capacity evaluation [17].

The eras overlap: rule-based components, counting methods and neural models all remain in use. What carries over is the pipeline in Sections 4–9. Whatever the model, someone still has to define the task, preserve the evidence, choose units, build data and evaluate.

3. The main tasks and applications

Plain version. Most NLP tasks can be grouped by what they output: a label for the whole text, a label for each word, a structure, a passage from a collection, or new text.

Output shape	Example tasks	Applications	Covered in
One label per text or text pair	topic classification, sentiment, spam detection, natural language inference	routing, moderation, review analysis	NLP 03, NLP 11
One label per token or span	part-of-speech tagging, named-entity recognition, information extraction	turning reports into records	NLP 06
A linguistic structure	parsing, semantic roles, coreference	analysis of who did what to whom	NLP 12
Evidence from a collection	search, extractive question answering	document search, help desks	NLP 11, NLP 09
New text conditioned on input	translation, summarization, dialogue	translators, assistants, chatbots	NLP 08, NLP 09
A reusable representation	word and sentence embeddings, pretrained encoders	similarity, clustering, transfer to new tasks	NLP 04, NLP 10

The same application often combines several rows. An assistant that answers maintenance questions may classify the question, retrieve passages, extract an equipment name and generate a reply. Each step has its own output and its own errors, which is why the rest of this tutorial treats the task definition as the first design decision.

4. Specify the task before choosing a model

Plain version. "Understand this message" is too broad to train or evaluate. Specify which part of its meaning must become an output.

For the invented message “Pump B: not ready,” three possible outputs are:

Task	Output	Evidence that matters
Topic classification	`maintenance`	The message concerns equipment status
Entity extraction	`Pump B`, with its source span	The exact equipment-name characters
Status extraction	Equipment `Pump B`; readiness `false`	The name and the negated readiness statement

The first answer does not establish the third. A classifier can recognize a topic from “pump” while ignoring “not.” A status extractor must preserve that negation. A system that returns equipment names also needs boundary rules: should `B` be included, and should the colon be excluded? Write these decisions into the annotation instructions.

Now compare “Pump B was not ready yesterday” with “Pump B is ready now.” The equipment is unchanged, but time and polarity differ. A current-status application needs a rule for multiple dated statements; a historical extractor may keep both. The correct answer depends on the task definition and available context, rather than on one universal interpretation of the string.

For “It still isn’t ready,” the equipment is absent from the message. If the application supplies earlier conversation, reference resolution can use it. If it supplies only this isolated sentence, identifying `Pump B` would add information that was never available. This is an input-design issue, developed further in NLP 12 (coreference) and NLP 08 (dialogue state).

Separate a prediction from an action. Predicting a request to reschedule does not determine which booking to change. Start with an output that can be checked against the text: an intent label, an equipment span or a stated event. Later tutorials add the contextual reasoning needed to resolve missing details.

5. Preserve the original text and the evidence

Plain version. The text used for model input and the text shown as evidence can be different representations of the same message. Keep a way to connect them.

A pipeline can be written as:

$$r \xrightarrow{g} x \xrightarrow{f_\theta} \hat{y}.$$

Here r is the raw input record; g is the preprocessing and numerical encoding function; x is model input; f_θ is a predictor with parameters θ; and $\hat{y}$ is its output. A record can include text and permitted context. Preprocessing includes decisions about normalization, token boundaries and vocabulary IDs.

If two different records become the same x, a deterministic predictor must return the same result for both:

$$g(r_1) = g(r_2) \quad \Longrightarrow \quad f_\theta(g(r_1)) = f_\theta(g(r_2)).$$

Here r_1, r_2 are two raw records. This simple identity explains why preprocessing is part of the model. If removing "not" makes affirmative and negative statements identical, changing the neural architecture cannot separate them afterward. The same reasoning applies when lowercasing merges `US` with `us` or punctuation removal collapses a decimal number into another string.

Some equivalences are useful. Unicode NFC normalization maps canonically equivalent character sequences to a common form [3]. This is different from choosing to remove accents, punctuation or case. The first concerns a specified character equivalence; the others may remove distinctions that the task needs. Decide what should remain equivalent for this application, rather than treating every transformation as harmless cleaning.

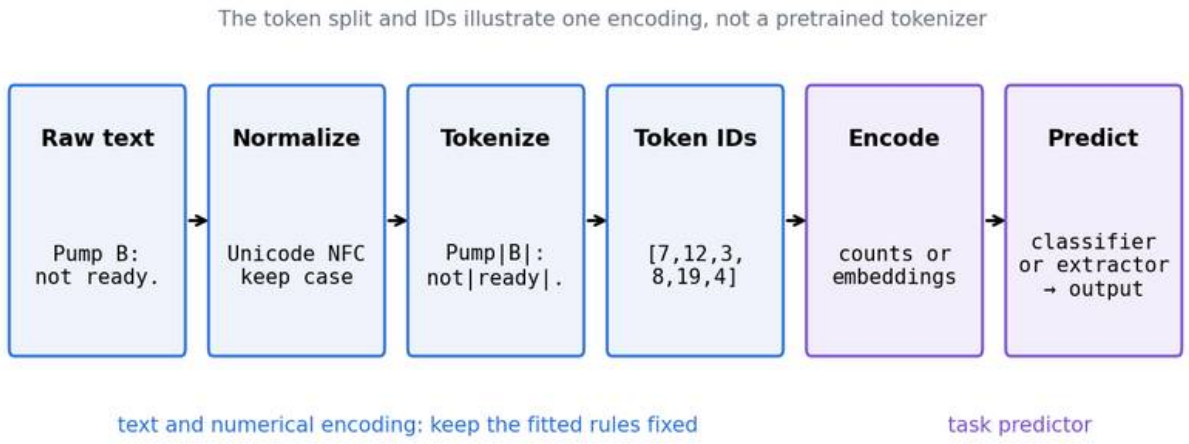


Figure 2 — An illustrative text-processing pipeline. Preserve a connection to the original text, and use the same fitted encoding during training and evaluation.

A worked normalization and span example

The word `Café` can use a precomposed accented letter or a base `e` followed by a combining accent. They look similar but occupy different numbers of code points [3]. If an extractor reports character positions, normalization can therefore shift a later span.

A span $[a, b)$ means the characters starting at offset a and ending just before b in a **specified string**. Both offsets are nonnegative integers, with $a \leq b$. They are not meaningful without knowing which text representation and indexing unit they refer to.

```
import re
import unicodedata

raw = 'Café Pump B: not ready.'
normalized = unicodedata.normalize('NFC', raw)
needle = 'Pump B'
raw_start = raw.index(needle)
```

```
normal_start = normalized.index(needle)
raw_span = (raw_start, raw_start + len(needle))
normal_span = (normal_start, normal_start + len(needle))
tokens = [(m.group(), m.start(), m.end())
          for m in re.finditer(r'\w+|[^\w\s]', normalized)]
assert normalized != raw
assert unicodedata.normalize('NFC', normalized) == normalized
assert raw[raw_span[0]:raw_span[1]] == needle
assert raw[normal_span[0]:normal_span[1]] != needle
assert all(normalized[start:end] == token for token, start, end in tokens)
print('code points:', len(raw), '->', len(normalized))
print('raw / normalized evidence spans:', raw_span, normal_span)
print('normalized tokens:', [token for token, _, _ in tokens])
```

The code prints `24 -> 23` code points. The raw equipment span is `(6, 12)` and the normalized span is `(5, 11)`. Both select `Pump B` in their own strings; applying the second span to the first string selects the wrong characters. The final assertion checks the normalized tokens against their own source offsets.

For extraction, either maintain an alignment from transformed positions back to the original text or retain annotations directly on the raw string. This example demonstrates the failure; it does not implement a general alignment algorithm. NLP 06 explains how word, token and character spans become structured outputs.

6. Decide which units the computation will use

Plain version. A word, a character and a model token answer different questions. Choose computational units without confusing them with all the units of language.

In "a cat and a dog," whitespace splitting produces five token **occurrences** and four distinct **types**: `a`, `cat`, `and`, `dog`. A vocabulary assigns an ID to each retained type. IDs are names for table entries: ID 7 does not represent a larger meaning than ID 3.

The string "Pump B: not ready." raises different choices. Whitespace splitting attaches punctuation to words. A punctuation-aware splitter may produce `Pump`, `B`, `:`, `not`, `ready`, `.`. A subword tokenizer can split a word further. All can be valid computational encodings, but their token positions are different. Word labels must be aligned to whichever encoding the model actually uses.

Subword tokenizers learn from data which pieces to keep. NLP 02 trains a small byte-pair encoding (BPE) tokenizer step by step and compares it with WordPiece, the unigram model and SentencePiece. Its central lesson is that learned pieces reflect frequency, not morphology, and that a tokenizer is a complete fitted procedure, not just a vocabulary list.

Vocabulary coverage and unknown inputs

A word vocabulary contains only its retained entries. Map an unseen word to an unknown marker, and all such words share that ID. This can be acceptable when the task depends on familiar surrounding words, but it removes the difference between an unseen equipment name and an unseen account name.

Define an illustrative unknown-token rate:

$$u = \frac{\sum_{i=1}^{N} \mathbf{1}[t_i \notin V]}{N}.$$

Here u is the unknown-token rate; t_i is token occurrence i under a declared tokenizer; V is its fitted vocabulary; $N > 0$ is the number of evaluated occurrences; and $\mathbf{1}[\cdot]$ equals one when the condition is true. A repeated unknown word counts repeatedly. This measures missing vocabulary entries, not lost meaning or task accuracy.

7. Chinese text: segmentation and two character sets

Plain version. Chinese is written without spaces between words, so deciding where words begin and end is itself an analysis. Chinese is also written in two standard character sets, Traditional and Simplified, and a model fitted on one sees many unfamiliar characters in the other.

Word segmentation

Chinese text often lacks spaces between words. "學生會說話" permits 學生 / 會 / 說話 (students can speak) or 學生會 / 說話 (the student council speaks); context helps choose the intended reading. A model tokenizer may simply use character or subword pieces. Its segmentation need not be a complete morphological or syntactic analysis. NLP 02 enumerates both readings with a small program and distinguishes those linguistic structures.

Segmentation is also a matter of convention. The Second International Chinese Word Segmentation Bakeoff used four corpora, two in Traditional characters (Academia Sinica in Taiwan and City University of Hong Kong) and two in Simplified characters (Peking University and Microsoft Research), each segmented under its provider's own standard [18]. When a dataset has word-level labels, the standard it follows is part of the task definition, and a character-based model must align its outputs to those words before evaluation.

Traditional and Simplified characters

Text from Taiwan and Hong Kong is mainly written in Traditional characters and text from mainland China in Simplified characters; the bakeoff corpora above follow that division [18]. Many characters are the same in both, but many are different Unicode code points: Traditional 機 is U+6A5F while its Simplified counterpart is U+673A. Unicode normalization does not merge them, because they are distinct characters, not alternative encodings of one character.

Conversion is not a one-to-one character swap. The Unihan database records Simplified and Traditional variants for each character and notes that context can decide the mapping: when converting Simplified to Traditional, U+810F becomes 臟 when it means “viscera” and 髒 when it means “dirty” [19]. Word choice can also differ between regions, so even a perfect character converter does not turn one variety’s text into the other’s.

The script mismatch shows up directly as vocabulary coverage. The program below fits a character vocabulary on one Traditional sentence and measures the unknown-token rate u on a new sentence written both ways. The Simplified version is written with escape codes so it is visible which characters differ.

```
vocab = set("這台相機已經校正好了")              # character vocabulary fitted on Traditional
 ↪  text
tests = {"Traditional": " 那台相機已經過熱了",
         # The same sentence in Simplified characters, written as escapes:
         "Simplified": " 那台相\u673a 已\u7ecf\u8fc7\u70ed 了"}
for name, text in tests.items():
    unknown = [f"U+{ord(ch):04X}" for ch in text if ch not in vocab]
    print(f"{name:11s} u = {len(unknown)}/{len(text)}  unknown: {unknown}")
```

The Traditional sentence has $u = 3/9$: 那, 過 and 熱 were not in the tiny training text. The Simplified sentence has $u = 5/9$: 那 (identical in both scripts) and the Simplified forms of 過 and 熱 are unknown as before, plus the Simplified counterparts of 機 and 經, which the vocabulary contains only in Traditional form. Real vocabularies are far larger, but the effect is the same in kind: a model trained mainly on one script has less evidence for the other. Decide whether to train on both, convert with a context-aware tool, or evaluate each script separately, and report which you chose.

8. Build examples and labels that answer the task

Plain version. A corpus is a collection of observations. Training examples are constructed from those observations according to a task. The construction determines what the model learns.

For classification, an example might be a whole message paired with one category. For extraction, it might be a message paired with character spans and their labels. For language modeling, the text supplies prediction targets without category annotations. These distinctions explain why the same corpus can support different objectives [1, 2].

Suppose a maintenance message contains “the camera is fine, but the delivery was late.” A product-sentiment label, a delivery-sentiment label and a whole-message label need not agree. Include the target in the instructions. If annotators choose different targets, a disagreement is evidence about the task definition before it is evidence about model quality.

Define the unit of independence for your split. Two paraphrases of one complaint are distinct strings but may describe the same event. If one appears in training and one in testing, the test can reward recognition of that event rather than transfer to a new one. Split related messages,

conversations or document revisions together when their shared information would otherwise leak.

There are two kinds of fitted state to track. The predictor learns weights from training labels. A tokenizer or vocabulary may learn from unlabeled training text. Keeping labels out of testing is not enough if the feature pipeline still uses test text to decide its vocabulary. Freeze both according to the experiment's stated rules.

Use a small baseline to diagnose the representation

Begin with a representation whose behavior is easy to inspect. A count-based message representation preserves which words appear while ignoring their order. "The dog bit the man" and "The man bit the dog" then produce the same counts despite reversing who did what. Every deterministic classifier using only those counts must give the same prediction for both.

This is a precise representation limitation. More examples cannot solve this pair while the feature map remains identical. Local phrase features can separate some order differences; contextual models can use richer input interactions. NLP 03 develops and tests those choices. Choose added structure in response to an observed failure, so the experiment identifies what changed.

9. Evaluate the output and diagnose the cause

Plain version. Evaluation should tell you what went wrong and where to improve it. One aggregate score rarely identifies the cause.

For classification, inspect errors for each category, especially rare ones; accuracy can conceal a category that is never predicted. Precision asks how often returned items are correct, while recall asks how many required items were returned. Both depend on what counts as an item [5]. For extraction, finding the right word with the wrong span is a boundary error. For a status record, the correct equipment paired with the wrong polarity is a meaning error.

Keep a few challenge pairs where one linguistic difference matters: "ready" versus "not ready," `Pump A` versus `Pump B`, and "yesterday" versus "now." Hold the intended task fixed while changing that difference. A correct answer on an ordinary message does not explain whether the system uses the distinction needed by these cases.

Observation	Inspect first	What an improvement must establish
Unseen equipment names collapse together	Vocabulary and fallback	Names remain distinguishable in the input
Correct words, incorrect evidence highlight	Normalization and offset alignment	Returned offsets refer to the displayed source
Negation reverses the true label but not the prediction	Feature map and supervision	The relevant distinction survives encoding and affects the decision
Strong validation result, weak new-customer result	Split independence and domain differences	Performance transfers to the intended population
Good results on one Chinese script, poor on the other	Character coverage and conversion	Both scripts are represented, or the gap is reported

Change one stage at a time when possible. If a preprocessing change and a model change happen together, a better score does not tell you which caused the improvement. Inspect transformed inputs before interpreting weights or outputs. This turns error analysis into a testable explanation instead of a list of model names.

10. Roadmap of the NLP track

The track follows the eras in Section 2: language structure first, then neural building blocks, sequence models, attention and Transformers, and finally pretrained models and structured tasks.

Tutorial	What it adds
NLP 01 ·Introduction to Natural Language Processing	Why language is hard, history, tasks, Chinese issues and the task-to-evaluation pipeline
NLP 02 ·Linguistic Essentials	Morphology, tokens and subwords, parts of speech, syntax, meaning, context, corpora and Zipf's law
NLP 03 ·Neural Networks for NLP	Count features, TF–IDF, logistic and neural text classifiers
NLP 04 ·Lexical Semantics and Word Embeddings	Distributional meaning, count-based vectors, word2vec
NLP 05 ·Recurrent Neural Networks for Language	n-gram language models, perplexity, recurrent language models
NLP 06 ·Sequence Labeling with RNNs	Part-of-speech tagging, entity recognition and span evaluation
NLP 07 ·Attention Mechanism and Transformers	Attention, contextual representations and position
NLP 08 ·Sequence-to-Sequence Models with Transformers	Encoder–decoder models, decoding, machine translation and chatbots
NLP 09 ·Language Modeling with Transformers	Causal language models, text generation and retrieval-augmented generation
NLP 10 ·Pretrained Models: BERT	Masked-language pretraining and the BERT encoder
NLP 11 ·Classification with BERT	Sentiment analysis, natural language inference and extractive QA heads
NLP 12 ·Syntactic and Semantic Tasks	Parsing, semantic roles, coreference and discourse

Related tutorials on this site cover the general machinery: ML 02 for neural networks, ML 03a for recurrent networks, ML 04 for attention and the Transformer block, and IT 01 for entropy and cross-entropy.

Modern view

Chang and colleagues' evaluation survey organizes what, where and how to evaluate language models [4]. For a pipeline, that motivates separate checks of encoding, task output and application behavior. A fluent generated explanation is not an alternative measurement of equipment-name recall; the two outputs answer different questions.

Zhao and colleagues' survey of large language models describes the current default: pretrain one large model on broad text, then adapt it (for example by instruction tuning or alignment with human preferences), use it through prompting, and evaluate its capacities across many tasks [17].

Its organization mirrors the history in Section 2, where each era kept the task definitions of the previous one and changed how the mapping from text to output was obtained.

For current systems, the transferable lesson is to inspect interfaces before comparing model families. A larger input budget is useful only if the relevant text survives truncation. A multilingual tokenizer is useful only if its representations support the distinctions the task needs, including both Chinese character sets when users write in both. A pretrained representation can reduce labeled-data requirements, but the application's targets still require clear supervision and evaluation. Later tutorials analyze the respective mechanisms rather than treating these as automatic benefits.

Open questions include how to measure linguistic coverage beyond character coverage, how segmentation affects different languages and registers, and how to separate representation failures from insufficient labels or missing context. A useful small study compares two encodings with one fixed task, independently split data and a few diagnostic linguistic cases. Report what each encoding preserves or loses, then explain the observed errors in those terms.

Key takeaways

- Language is ambiguous at every level, from word boundaries to intention; context and world knowledge resolve what the string alone cannot.
- NLP moved from hand-written rules to corpus statistics, then to neural networks and pretrained models; the task-definition and evaluation problems carried over unchanged.
- Group tasks by output shape: text labels, token labels, structures, retrieved evidence, generated text or representations.
- Chinese adds word segmentation under competing standards and two character sets whose conversion is not one-to-one.
- Define the desired output and its linguistic evidence before choosing a model, and keep raw text distinct from transformed model input.
- A predictor cannot recover distinctions erased by its preprocessing function; tokens and vocabulary IDs are computational units, not complete analyses of language.
- Fit learned representations on permitted training text, keep related observations together in splits, and diagnose vocabulary, boundary, meaning and transfer errors before changing architecture.

Exercises

1. For "Pump B was not ready yesterday, but it is ready now," define labels for topic classification, historical status extraction and current-status extraction. Which labels require a time policy?

   Hint

   The equipment name can stay fixed while status values and their validity times differ. Decide whether you return one state or two dated events.

2. Replace the decomposed accent in the normalization example with a precomposed one. Explain which offsets change and which assertions remain valid.

Hint

Compare raw and normalized strings separately. An offset is a location in one representation, not in all equivalent-looking strings.

3. Make a preprocessing rule remove "not." Construct two messages with different required status labels but identical transformed inputs, and prove why a deterministic predictor cannot separate them.

Hint

Apply the equality involving $g(r_1)$ and $g(r_2)$. The contradiction arises before training.

4. In the script-coverage program, refit the vocabulary on the union of the Traditional training sentence and its Simplified version. Recompute u for both test sentences. Then explain why converting every Simplified test character to Traditional before lookup is not always possible with a character table alone.

Hint

The union removes the script-specific unknowns but not 那, 過 and 熱. For conversion, use the Unihan example: one Simplified code point maps to two Traditional characters depending on meaning.

5. Group three rewritten messages from one incident into a train/test dataset. Design a split and two diagnostics that distinguish event memorization from transfer to new incidents.

Hint

Split incident groups together, then examine unseen names and unseen wording separately. Keep the label policy fixed.

References

1. Masato Hagiwara, *Real-World Natural Language Processing: Practical Applications with Deep Learning*, Chapters 1–3 and 10–11, Manning, 2021. Publisher
2. Susan McRoy, *Principles of Natural Language Processing*, open textbook, 2021. Book
3. Unicode Consortium, "Unicode Normalization Forms," Unicode Standard Annex #15, revision 58, 2026. Specification
4. Yupeng Chang et al., "A Survey on Evaluation of Large Language Models," *ACM Transactions on Intelligent Systems and Technology*, 2024. Survey

5. Scikit-learn developers, “Metrics and Scoring: Quantifying the Quality of Predictions,” documentation, current online edition. Documentation
6. Joseph Weizenbaum, “ELIZA—A Computer Program for the Study of Natural Language Communication Between Man and Machine,” *Communications of the ACM*, 9(1), pp. 36–45, 1966. DOI
7. Peter F. Brown, Stephen A. Della Pietra, Vincent J. Della Pietra and Robert L. Mercer, “The Mathematics of Statistical Machine Translation: Parameter Estimation,” *Computational Linguistics*, 19(2), 1993. Paper
8. Mitchell P. Marcus, Beatrice Santorini and Mary Ann Marcinkiewicz, “Building a Large Annotated Corpus of English: The Penn Treebank,” *Computational Linguistics*, 19(2), 1993. Paper
9. Yoshua Bengio, Réjean Ducharme, Pascal Vincent and Christian Jauvin, “A Neural Probabilistic Language Model,” *Journal of Machine Learning Research*, 3, pp. 1137–1155, 2003. Paper
10. Ronan Collobert, Jason Weston, Léon Bottou, Michael Karlen, Koray Kavukcuoglu and Pavel Kuksa, “Natural Language Processing (Almost) from Scratch,” *Journal of Machine Learning Research*, 12, pp. 2493–2537, 2011. Paper
11. Tomas Mikolov, Kai Chen, Greg Corrado and Jeffrey Dean, “Efficient Estimation of Word Representations in Vector Space,” arXiv:1301.3781, 2013. Paper
12. Ashish Vaswani et al., “Attention Is All You Need,” *NeurIPS*, 2017. Paper
13. Matthew E. Peters et al., “Deep Contextualized Word Representations,” *NAACL-HLT*, 2018. Paper
14. Jacob Devlin, Ming-Wei Chang, Kenton Lee and Kristina Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” *NAACL-HLT*, 2019. Paper
15. Tom B. Brown et al., “Language Models are Few-Shot Learners,” *NeurIPS*, 2020. Paper
16. Long Ouyang et al., “Training Language Models to Follow Instructions with Human Feedback,” *NeurIPS*, 2022. Paper
17. Wayne Xin Zhao et al., “A Survey of Large Language Models,” arXiv:2303.18223, 2023 (continuously updated). Survey
18. Thomas Emerson, “The Second International Chinese Word Segmentation Bakeoff,” *Proceedings of the Fourth SIGHAN Workshop on Chinese Language Processing*, 2005. Paper
19. Unicode Consortium, “Unicode Han Database (Unihan),” Unicode Standard Annex #38, revision 41, 2026. Specification

NLP 02 • Linguistic Essentials

The language structure every NLP system works with: units and morphology, tokens and subwords, parts of speech, phrase and dependency structure, meaning, context and discourse, corpora, annotation and Zipf's law.

Online edition: https://blog.shyandram.dev/en/learn/nlp/nlp-02-linguistic-essentials/

What you'll learn

- Distinguish language from its spoken, signed and written forms, and distinguish linguistic units from computational tokens.
- Analyze word structure, compare stemming with lemmatization, and enumerate ambiguous Chinese segmentations.
- Train a small BPE tokenizer and relate it to WordPiece, the unigram model and SentencePiece.
- Use Universal Dependencies part-of-speech tags and explain why grouping, grammatical relations and agreement carry information that word counts omit.
- Work out lexical meaning, composition, negation and quantifier scope, and separate assertion, presupposition, implicature and discourse relations.
- Read corpora and annotations critically: agreement between annotators, Zipf's law and the long tail of rare words.

The big picture

Language lets people describe situations, ask questions, coordinate action and change what others know. The visible sentence is only part of that activity. Its internal structure constrains interpretation; speakers also rely on shared objects, earlier conversation and expectations about why someone spoke.

> **Explain like I'm five**
>
> Imagine someone hands you building blocks and says, "Build that one again." Knowing the blocks' names does not tell you which building "that one" means, what happened last time, or whether the speaker wants the same colors. Language works similarly: words supply pieces, grammar connects them, and the situation supplies information the speaker leaves unsaid.

Consider "The technician inspected the panel with the camera." Did the technician use a camera, or inspect a panel that had a camera attached? Knowing every dictionary entry does not settle the question. Consider "Can you inspect it tomorrow?" Now the object, reference date and intended request depend on context. A useful analysis identifies these alternatives before choosing a representation or label.

This tutorial supplies the language foundation for the practical route in Hagiwara's *Real-World Natural Language Processing* [1] and the symbolic and computational route in McRoy's *Principles of Natural Language Processing* [2]. *Essentials of Linguistics*, an open textbook, supplies additional linguistic grounding [3]. The examples here are original. NLP 01 introduced the task-to-evaluation pipeline; this tutorial supplies the units and structures that pipeline must preserve. Later tutorials turn them into models: NLP 06 tags parts of speech and entities, and NLP 12 parses sentences, labels semantic roles and resolves coreference. These levels interact rather than forming a compulsory sequence of separate software modules.

1. Language, modality and units

Plain version. Speech, signing and writing are different ways of expressing language, and each records different information. A computer's "token" is yet another unit, chosen for processing.

Language is a system of structured expression and interpretation. Speech and writing are ways to express it; they are not interchangeable containers. Signed languages use systematic visual-manual expression, and tactile signing provides another modality [4]. A written transcript preserves selected information from an interaction, while leaving out timing, gaze, gesture and much of its physical setting.

Sounds and written symbols answer different questions

Phonetics studies the physical production and perception of speech sounds. **Phonology** studies their patterning within a language. A **phoneme** is a contrastive sound unit in a particular linguistic system; different physical realizations can belong to the same phoneme [5]. A **grapheme** is a unit of a writing system. These units need not map one to one.

English "ship" has four letters, yet its initial "sh" represents one consonant sound. Conversely, a letter can contribute differently in different words. Counting letters therefore cannot directly answer a question about sound structure. A speech application and a text application may need different annotations even when their users intend the same words.

Tone and intonation also differ [6]. Mandarin mā, má, mǎ and mà distinguish the ordinary written words 媽, 麻, 馬 and 罵. Their tone patterns contribute to lexical contrasts. The broader contour of an utterance can also convey questioning or emphasis. In English, "Really." and "Really?" suggest different interpretations in writing, but punctuation only approximates how an actual speaker expresses disbelief, curiosity or confirmation. The written question mark does not recover the original voice.

Characters, words and tokens

A **word form** is a particular realized form; a **lexeme** groups related forms as one vocabulary item. For a simplified analysis, "repair," "repairs" and "repaired" are forms of the verb lexeme REPAIR.

A **lemma** is a chosen citation form used to index that item. Lemmatization is thus an annotation decision, not a claim that the surface forms are interchangeable in a sentence [2, 3].

The distinctions matter in “The technician repairs cameras” versus “The technician repaired cameras.” Grouping both verbs under REPAIR helps search for the activity, but discarding the forms loses a present-versus-past contrast. An application can preserve both the original span and its lemma rather than deciding that one representation must replace the other.

A computational **token** is an input unit selected by a processing procedure. It might be a word, part of a word, punctuation or a character sequence. Its boundaries need not coincide with phonemes, morphemes or lexemes. Likewise, a Chinese character can participate in a multi-character word: the written sequence 相機 denotes a camera. A character-based system receives two units where a word-oriented analysis may use one. Neither count independently defines the number of meanings in the phrase.

2. Morphology: structure inside words

Plain version. Words have parts. Some parts carry vocabulary meaning, others carry grammar such as tense or number, and the way the parts are grouped can change the meaning.

Word structure can convey both vocabulary meaning and grammatical information. **Morphology** studies that structure; a **morpheme** is a minimal recurring pairing of form with meaning or grammatical function [7]. “Minimal” does not mean “one letter,” and a meaningful element need not stand alone. Morphological evidence such as a plural ending, tense marker or derivational prefix is useful, but not every language expresses these properties with the same pieces or word boundaries.

Inflection, derivation and productive formation

An **inflectional** contrast changes a word’s grammatical realization, as in repair/repaired. A **derivational** process can create a new vocabulary item, as in repair/repairable. A **compound** combines bases, as in camera mount [3, 7]. These are analytical distinctions: identifying an ending’s spelling is not enough to decide its function.

Compare “repairs” in “She repairs the camera” and “The repairs took a day.” In the first sentence, it is a finite verb form; in the second, a plural noun. A rule that strips every final “s” loses both grammatical category and the reason for the ending. It would also mishandle irregular pairs such as mouse/mice. The task is to explain recurring form–function patterns, with exceptions represented explicitly.

Productivity means that a pattern can support new combinations, rather than merely describing a memorized list. If a workshop introduces a new “sensor cradle,” listeners can interpret the compound from familiar pieces and the setting. Yet composition does not specify every detail: the cradle might hold a sensor during assembly, transport or operation. Word formation can supply a general relation while context supplies its exact instance.

Chinese also has internal word structure and productive combinations. In 三台相機, 三 expresses the count, 台 is a classifier associated with the counted object, and 相機 names the object. English "three cameras" places plural marking on the noun in this example. The contrast identifies different grammatical choices; it does not establish that Chinese words lack structure or that English nouns always carry an explicit count.

Stemming versus lemmatization

Plain version. A stemmer chops endings off by rule; a lemmatizer looks up the dictionary form, using the word's grammatical category. The stemmer is cheap and crude; the lemmatizer needs more knowledge and makes fewer blunders.

A **stemmer** maps word forms to a shared string, the **stem**, by rewriting suffixes. Porter's algorithm is the classic English example: it applies ordered groups of suffix rules, each guarded by a condition on the remaining stem, and was designed to improve information retrieval by grouping related terms [16]. The stem need not be a real word; it only has to be consistent. A **lemmatizer** returns the lemma, which requires a lexicon and usually the part of speech: "saw" is the lemma of a noun but a form of SEE as a verb.

The toy stripper below uses four ordered rules and a minimum-stem-length guard, in the spirit of (but far simpler than) Porter's algorithm. The lookup table stands in for a dictionary.

```
# A toy suffix stripper (a few Porter-style rules) versus a lookup lemmatizer.
RULES = [("ies", "y"), ("ing", ""), ("ed", ""), ("s", "")]   # tried in order; first match
 ↪  wins

def stem(word):
    for suffix, replacement in RULES:
        if word.endswith(suffix) and len(word) - len(suffix) >= 3:
            return word[: -len(suffix)] + replacement
    return word

# A lemmatizer needs the part of speech; this tiny table stands in for a dictionary.
LEMMAS = {("repairs", "VERB"): "repair", ("repairs", "NOUN"): "repair",
          ("repaired", "VERB"): "repair", ("mice", "NOUN"): "mouse",
          ("was", "AUX"): "be", ("batteries", "NOUN"): "battery",
          ("news", "NOUN"): "news", ("this", "DET"): "this"}

def lemmatize(word, pos):
    return LEMMAS.get((word, pos), word)

for word, pos in [("repairs", "VERB"), ("repaired", "VERB"), ("batteries", "NOUN"),
                  ("mice", "NOUN"), ("was", "AUX"), ("news", "NOUN"), ("this", "DET")]:
    print(f"{word:10s} stem={stem(word):9s} lemma={lemmatize(word, pos)}")
```

The stripper handles the regular forms (`repairs`, `repaired`, `batteries` all reach a sensible stem) but returns `mice` and `was` unchanged and damages `news` → `new` and `this` → `thi`. The first two failures are irregular morphology; the last two are **over-stemming**, where an ending that only

looks like a suffix is removed. The lemmatizer gets all seven right, but only because its table already contains them; an unseen word passes through unchanged. Neither tool decides the grammatical category: the caller supplied `VERB` or `NOUN`, which is the job of the tagger in NLP 06.

Which one to use depends on the task. For search, merging `repairs` and `repaired` can raise recall even if a few unrelated words collide. For a status extractor, the tense the stemmer throws away may be exactly the answer. Modern neural pipelines rarely stem their input; they keep the original form and let subword tokens and context carry the morphology, which is one reason the next section matters.

The same pieces can combine in different orders

An invented description "an unsealable case" permits two useful structural analyses:

- `un + (seal + able)`: a case that cannot be sealed.
- `(un + seal) + able`: a case that can be unsealed.

The first treats "un" as negating an adjective; the second treats it as reversing the sealing action before "able" forms an adjective. The readings have opposite practical implications. Their availability and naturalness depend on context, so the example demonstrates possible analyses rather than claiming that every speaker assigns both equally often.

A bag of these pieces is identical in both analyses. What changes is the intermediate grouping and the kind of object each operation combines with. The same problem will reappear in sentence attachment and logical scope: a representation must retain structure if the distinction matters to the task.

Chinese segmentation is an interpretation problem

Written Chinese does not mark word boundaries with spaces. The original string 學生會說話 supports "學生 / 會 / 說話" ("students can speak") and "學生會 / 說話" ("the student council speaks"). The first reading treats 會 as expressing ability. The second treats 學生會 as a compound noun. Spaces inserted after processing therefore record an analysis; the unspaced string does not supply them directly.

The following program enumerates paths through a small, explicit lexicon. It is not a general Chinese segmenter. It shows why choosing the longest matching entry cannot establish the speaker's intended reading.

```
from functools import lru_cache

text = " 學生會說話"
lexicon = (" 學生", " 學生會", " 會", " 說話")

@lru_cache(None)
def segment(start):
    if start == len(text):
```

```
        return ((),)
    paths = []
    for word in lexicon:
        if text.startswith(word, start):
            for rest in segment(start + len(word)):
                paths.append((word,) + rest)
    return tuple(paths)

for path in segment(0):
    print(" / ".join(path))
assert len(segment(0)) == 2
```

The two printed paths are the readings above. Every recursive step consumes a known word, and the base case accepts only complete coverage. Enlarging the lexicon may add alternatives; it does not add evidence about a particular conversation. A syntax analysis, the surrounding sentence or knowledge that the message concerns a council meeting could favor one path.

Even the gold standard is a choice. The Second International Chinese Word Segmentation Bakeoff evaluated systems on four corpora, two in Traditional characters (from Academia Sinica in Taiwan and City University of Hong Kong) and two in Simplified characters (from Peking University and Microsoft Research), and each provider segmented its data under its own standard [27]. A system can therefore be "wrong" on one corpus and "right" on another for the same split. Report which segmentation standard your labels follow.

3. Tokens and subwords

Plain version. A model needs a fixed list of units. Whole words leave too many unknown words; single characters make sequences long. Subword methods learn a middle ground from data: frequent strings become single tokens, rare ones are spelled out from smaller pieces.

NLP 01 defined the unknown-token rate and showed why mapping every unseen word to one marker loses distinctions. A character vocabulary reduces whole-word unknowns if the necessary characters are covered, at the cost of longer sequences. A byte-based encoding can cover arbitrary input bytes with a complete base byte alphabet, but individual units then need not correspond to readable characters. Subword methods trade vocabulary size against sequence length; Mielke and colleagues survey these choices [22]. There is no universal segmentation that is best for every language, task and resource budget.

A learned tokenizer typically fits statistical patterns in a corpus. These patterns can help reuse frequent pieces, but a frequent piece is not necessarily a meaningful morpheme. Conversely, a meaningful inflection can be split across several pieces. This distinction matters when asking what the model can recognize: tokenization supplies an encoding; later computation must learn how the encoded parts combine.

Work through one subword method: BPE

BPE learns which adjacent pieces are worth joining. It gives frequent combinations their own symbols while leaving other strings as sequences of smaller pieces. Sennrich, Haddow and Birch adapted byte-pair encoding for rare-word translation [17]. The character-based teaching version below works inside words:

1. Represent each training word as characters plus an end marker.
2. Count adjacent symbol pairs, weighted by the word's corpus frequency.
3. Merge the most frequent pair everywhere it appears in those word representations.
4. Save that merge and repeat for a fixed number of steps.

A merge changes subsequent pair counts. After combining two symbols, the new symbol can participate in another pair. The algorithm therefore learns an ordered sequence of operations, rather than a set of pieces that can be applied in any order.

```
from collections import Counter

corpus = ("low low low low low lower lower newest newest newest newest newest "
          "newest widest widest widest").split()
# Start from characters; "_" marks the end of a word so merges respect boundaries.
words = Counter(tuple(w) + ("_",) for w in corpus)

def pair_counts(words):
    pairs = Counter()
    for symbols, freq in words.items():
        for a, b in zip(symbols, symbols[1:]):
            pairs[a, b] += freq
    return pairs

def merge(words, pair):
    out = Counter()
    for symbols, freq in words.items():
        s, i = [], 0
        while i < len(symbols):
            if i + 1 < len(symbols) and (symbols[i], symbols[i + 1]) == pair:
                s.append(symbols[i] + symbols[i + 1]); i += 2
            else:
                s.append(symbols[i]); i += 1
        out[tuple(s)] += freq
    return out

merges = []
for step in range(10):
    pairs = pair_counts(words)
    best = max(pairs, key=lambda p: (pairs[p], p))   # most frequent; ties broken by symbol
     ↪  order
    merges.append(best)
    words = merge(words, best)
print("merges:", [a + b for a, b in merges])
```

```
def tokenize(word):                    # apply the learned merges in order
    symbols = list(word) + ["_"]
    for pair in merges:
        symbols = list(merge(Counter({tuple(symbols): 1}), pair))[0]
    return symbols

for w in ["lowest", "newer", "wider"]:
    print(w, "->", tokenize(w))
```

The output includes `lowest -> ('low', 'est_')`, `newer -> ('ne', 'w', 'e', 'r', '_')` and `wider -> ('wi', 'd', 'e', 'r', '_')`. The first word was not in the training corpus, yet its letters can be recombined using learned pieces. The longer encodings of the other words reflect which combinations acquired merges.

Why the merge sequence matters

The first merge is `t_`, followed by `st_` and `est_`. Repeated endings in the training words make these combinations frequent. This resembles an English suffix in the example, but the algorithm has no instruction to discover suffixes: frequency drives its decisions. Another corpus can produce a different split of the same spelling.

The merge `low` occurs only after smaller combinations have been formed. Encoding follows the saved order so it reproduces the fitted procedure. A longest-piece heuristic over the final symbols is a different algorithm and need not produce the same segmentation. The tie-breaking rule also matters: this code resolves equal counts through symbol order, making the run reproducible.

The function returns pieces, not numerical IDs. It does not prove coverage for arbitrary unseen characters. The example's successful novel words reuse characters present during training; a word containing a new character would require a declared fallback and a compatible vocabulary lookup. This separates two questions: can a string be segmented, and does every returned piece have an ID that the model understands?

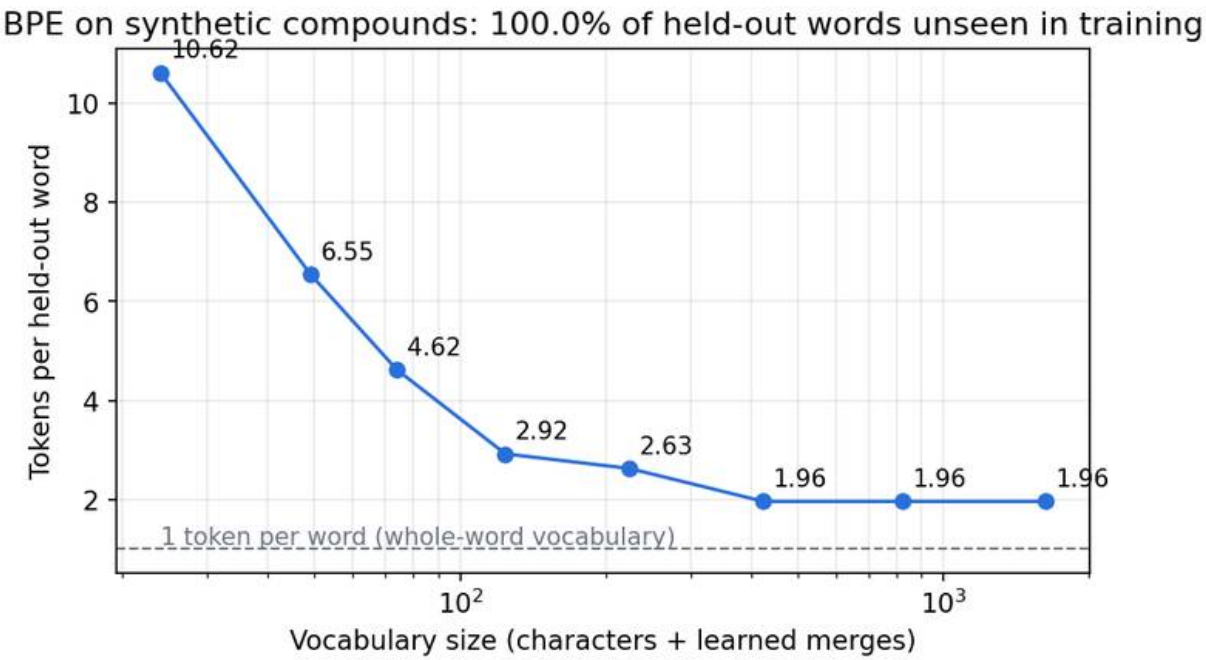


Figure 1 — More merges can shorten encodings, but vocabulary size alone does not determine coverage or linguistic quality. The synthetic training and held-out samples are separate.

WordPiece, the unigram model and SentencePiece

BPE is one member of a family. The others differ in how they choose pieces and how they split a new word.

- **WordPiece** originated in Japanese and Korean voice search [18]. Google's neural translation system describes its version as choosing a fixed number of pieces so that the segmented training corpus uses as few pieces as possible, with a boundary marker added at the start of each word so that the original word sequence can be recovered without ambiguity [19]. BERT uses a WordPiece vocabulary [30]; NLP 10 builds on it.
- **Unigram language model** segmentation works top-down: start from a large candidate inventory, model a word as a sequence of independently drawn pieces, and prune the pieces that contribute least to the corpus likelihood [20]. Because it is probabilistic, one word can have several scored segmentations; Kudo used this to sample alternative segmentations during training as a regularizer [20].
- **SentencePiece** is a toolkit rather than a new criterion: it trains BPE or unigram models directly from raw sentences and treats the space as an ordinary symbol, so no language-specific pre-tokenizer is required and decoding reverses encoding [21]. That matters for languages such as Chinese and Japanese that do not separate words with spaces.

In all three, the learned pieces are statistical, not morphological. A subword boundary that happens to fall at `est` is evidence about corpus frequency; it is not an analysis that `-est` is a superlative

suffix.

Use a complete tokenizer, not just a merge list

Tokenization software also specifies normalization, pre-tokenization, special markers and decoding [23]. A checkpoint's token IDs refer to its particular vocabulary and conventions. Matching only the tokenizer family name is insufficient: two vocabularies can assign different symbols to the same ID.

For a pretrained model, use the associated tokenizer and configuration. For a tokenizer trained in your experiment, fit it on the permitted training text and freeze its vocabulary and merge rules before evaluation. Inspect examples as pieces, IDs and reconstructed text; each view catches a different failure.

4. Parts of speech

Plain version. A word's category describes how it behaves in a sentence. Its spelling can suggest a category, but context often decides which use is intended.

Part of speech describes a grammatical category, such as noun or verb. A noun commonly heads a noun phrase; a verb commonly heads a clause's predicate. These are distributional descriptions rather than rules that every noun must name a physical object. "Decision" and "arrival" are nouns although they refer to abstract content and events [2].

The same surface word can have several categories. In the invented pair "the pilots can land" and "the metal can fell," the word "can" has different roles. The second sentence also demonstrates why a familiar local phrase can be misleading when the remainder of the sentence changes. A tagger must consider the permitted context and its annotation scheme.

The Universal Dependencies tag set

Tag inventories differ between projects, which makes cross-language comparison hard. **Universal Dependencies** (UD) is a framework for consistent cross-linguistic annotation of parts of speech, morphological features and dependency relations [24]. Its universal part-of-speech (UPOS) inventory has 17 tags [25]:

Group	Tags
Open class	`ADJ` adjective, `ADV` adverb, `INTJ` interjection, `NOUN` noun, `PROPN` proper noun, `VERB` verb
Closed class	`ADP` adposition, `AUX` auxiliary, `CCONJ` coordinating conjunction, `DET` determiner, `NUM` numeral, `PART` particle, `PRON` pronoun, `SCONJ` subordinating conjunction
Other	`PUNCT` punctuation, `SYM` symbol, `X` other

UD separates the coarse tag from **features** such as `Number=Plur` or `Tense=Past`, so "repairs" as a noun can be `NOUN` with `Number=Plur` while "repaired" is `VERB` with `Tense=Past`. Each language's treebanks add documented conventions for constructions the universal guidelines leave open. The tag set is a shared annotation scheme, not a claim that every language draws category boundaries in the same place.

Represent a tagged sentence as:

$$\mathbf{x} = (x_1, \dots, x_T), \qquad \mathbf{y} = (y_1, \dots, y_T), \qquad y_t \in \mathcal{Y}.$$

Here $\mathbf{x}$ is the input word sequence, x_t is the word at position t, T is sequence length, $\mathbf{y}$ is the tag sequence, y_t is the tag for word x_t and $\mathcal{Y}$ is the finite tag inventory, for example the 17 UPOS tags. The symbols t and T refer to linguistic word positions, not automatically to model subword positions.

Annotation needs additional rules: contractions, punctuation, multiword expressions and words with several plausible analyses. When a model uses subwords, define how its outputs return to words. Using only the first subword's representation is one possible convention; pooling all subwords is another. Whatever the choice, gold labels and decoded output must use the same units. Predicting $\mathbf{y}$ from $\mathbf{x}$, with hidden Markov models, Viterbi decoding and recurrent taggers, is the subject of NLP 06.

5. Syntax: order, grouping and grammatical relations

Plain version. Grammar organizes words into groups and links them by relations such as subject and object. The same words in a different structure can describe a different event.

Grammar organizes combinations. Word order, phrase structure and relations such as subject and object constrain who did what to whom [2, 3]. These constraints are richer than a list of adjacent words.

Why word counts lose role information

"Ada praised Bo" and "Bo praised Ada" contain the same words. Their ordinary active-clause readings reverse the person giving praise and the person receiving it. The Chinese pair 阿達稱讚小博 / 小博稱讚阿達 makes the same contrast in these sentences. A representation that only counts words maps each pair to the same object.

```
from collections import Counter

first = "Ada praised Bo".split()
second = "Bo praised Ada".split()

def roles(tokens):
    # Only our deliberately fixed three-word active-clause template.
    assert len(tokens) == 3 and tokens[1] == "praised"
    return {"praiser": tokens[0], "praised_person": tokens[2]}
```

```
print(Counter(first) == Counter(second))
print(roles(first))
print(roles(second))
assert roles(first) != roles(second)
```

The count comparison prints `True`; the role dictionaries differ. This is an information loss result, not a trained-model benchmark. Once the two inputs have become identical counts, a later deterministic function of those counts alone cannot recover which original order occurred.

Syntactic roles also differ from semantic roles. In "Bo was praised by Ada," Bo is the grammatical subject, while Ada still performs the praising. Equating the subject with the actor would fail even though the passive sentence describes the same event. An extraction task should state whether it wants grammatical dependencies, event participants or both.

Constituency and dependency describe different aspects

A **constituent** is a unit that participates in larger structure. In "The label beside the boxes is missing," the noun phrase "the label beside the boxes" functions as the subject. Its head is "label"; "beside the boxes" modifies that head. Replacing the whole phrase with "it" preserves a simple grammatical frame: "It is missing." Such tests provide evidence for grouping, rather than proving that every adjacent substring is a phrase [8].

The agreement contrast "The labels beside the box are missing" shows why proximity is insufficient. The verb agrees with the subject's head in these examples, not the nearest noun "box." A local heuristic could find the closest noun correctly and still select the wrong number.

A **dependency** representation instead records relations between words, such as a verb's subject and object or a noun's modifier [2]. UD uses dependency trees in which every word has exactly one head and a labeled relation to it, such as `nsubj` for a nominal subject [24]. Constituency highlights grouping; dependency highlights linked heads. They can describe related evidence without having identical structures or annotation conventions. The Penn Treebank, a large early English resource, annotated constituency trees [26]; UD annotates dependencies across many languages [24].

In the panel example, "with the camera" can modify the inspecting event or the panel. The simplified diagrams below retain this one attachment distinction and omit determiners and other relations. They are teaching diagrams, not complete parses under a named annotation standard.

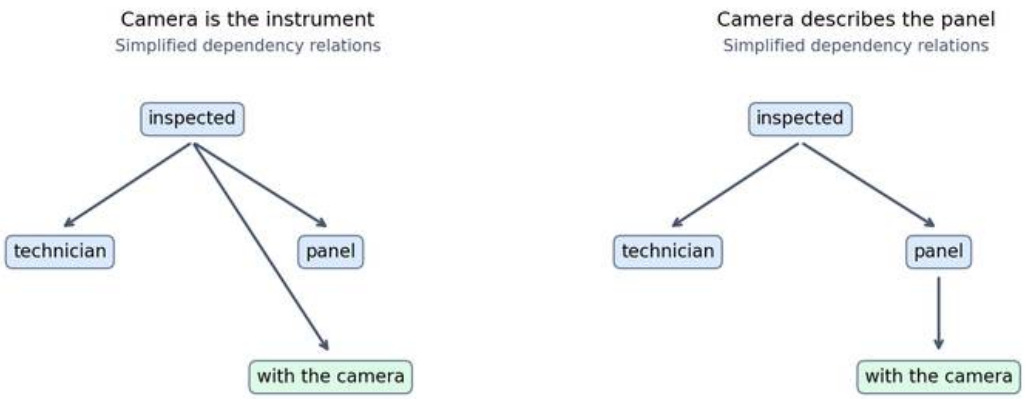


Figure 2 — The same words permit different attachment relations. Selecting a relation selects a different interpretation.

Under event attachment, a camera is the instrument of inspection. Under noun attachment, the camera helps identify the panel. Surrounding context can favor either: a technician explaining a recording method versus someone distinguishing equipped and unequipped panels. Grammatical possibility does not determine which situation actually occurred.

This is a preview. NLP 12 develops grammars, chart parsing, probabilistic parsing and dependency parsing. Here the essential question is what a representation must preserve: grouping, linked heads, role direction and plausible alternatives.

6. Semantics: meanings and truth conditions

Plain version. Semantics asks what an expression means and what the world would have to be like for a sentence to be true. Word meanings matter, but so does how the sentence combines them.

A sentence can be fluent while describing a situation incorrectly. **Semantics** asks how expressions contribute meaning and how a structured sentence determines conditions under which its assertion is true [2, 3]. Separating these conditions from conversational inference makes ambiguity easier to analyze.

Lexical meaning is contextual and relational

Polysemy concerns related senses of an expression; **homonymy** concerns distinct meanings sharing a form [3]. A window can be an opening in a wall or a bounded area on a screen. The relationship helps explain the extension, but replacing "window" with one fixed concrete object would miss the screen reading. A dictionary can list senses; the occurrence still needs interpretation. NLP 04 studies how vectors learned from contexts capture some of these relations.

Chinese 開 appears in 開門 (“open a door”) and 開會 (“hold a meeting”). Translating every occurrence with the same English verb would erase the construction-specific difference. Meaning belongs to the expression in its combination and context, not to a context-free lookup from character to English word.

Even apparently simple adjectives can rely on a comparison class. A small elephant may be larger than a large ant. The expressions compare each animal with an appropriate class, not with one universal size threshold. In a collection containing only toy elephants, the relevant comparison can change again. An annotation called “small” should therefore specify what is being compared.

Composition depends on structure

Compositionality is the idea that the meaning of a complex expression depends on meanings of its parts and how they combine [3]. It does not say that word meanings can simply be added together. “Bo praised Ada” combines participants in a directed relation; a reordered sentence combines the same names differently. Attachment changes which expression a modifier restricts.

An idiomatic expression can also have a conventional meaning not predictable from isolated word entries. A lexicon and a grammar are complementary: the analysis must record both conventional combinations and productive composition. For an application, ask which distinction is needed. Searching for repair instructions may tolerate broad lexical grouping; deciding whether a repair was completed requires event, tense and negation information.

Quantifier scope changes what is required

“Every technician uses a camera” has a reading in which different technicians may use different cameras. It can also be understood, in a suitable context, as involving one camera shared by all. These are different conditions:

$$A : \forall t \in T,\ \exists c \in C :\ U(t, c),$$
$$B : \exists c \in C,\ \forall t \in T :\ U(t, c).$$

Here T is the set of technicians, C the set of cameras, $U(t, c)$ means technician t uses camera c, A and B name the two readings, $\forall$ means “for every,” and $\exists$ means “there exists.” Both sets are nonempty in this example. The variable c can depend on t in A; in B one c must work for every t.

To check A, visit each technician and find at least one camera they use. To check B, choose a camera first and verify that all technicians use that particular camera. If B holds, A follows: use the shared camera for each person. The reverse implication fails when every person has a camera but no camera is shared by everyone.

```
technicians = ("Ada", "Bo", "Cy")
cameras = ("A", "B")
worlds = {
    "distributed": {("Ada", "A"), ("Bo", "B"), ("Cy", "A")},
    "shared": {("Ada", "A"), ("Bo", "A"), ("Cy", "A")},
```

```
}
for name, uses in worlds.items():
    each_has_some = all(any((t, c) in uses for c in cameras)
                        for t in technicians)
    some_for_everyone = any(all((t, c) in uses for t in technicians)
                            for c in cameras)
    print(name, each_has_some, some_for_everyone)
assert not any(all((t, c) in worlds["distributed"] for t in technicians)
               for c in cameras)
```

The outputs are `distributed True False` and `shared True True`. Each row in Figure 3 represents one person's possibilities. The first reading requires a marked cell in every row; the second requires an entirely marked column. Scope is therefore a constraint difference, not merely a different paraphrase of the same requirement.

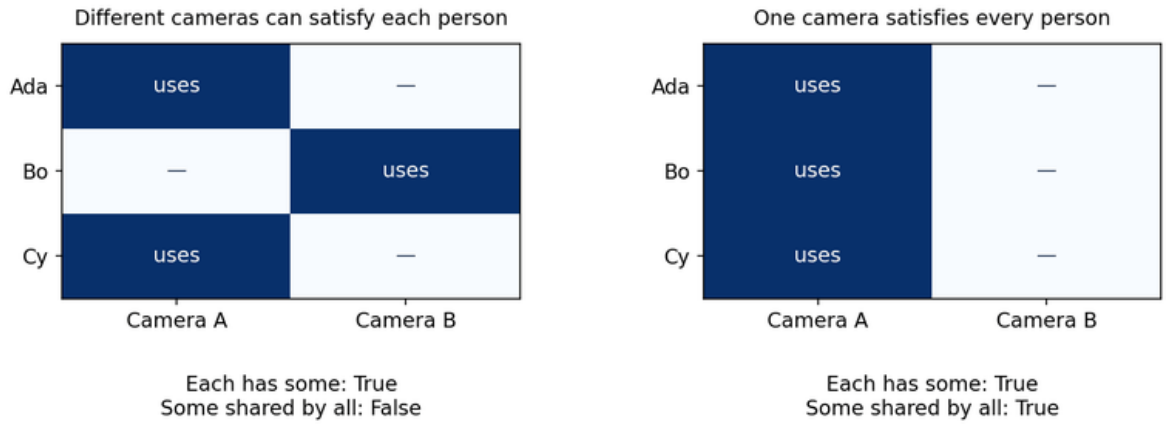


Figure 3 — Changing quantifier order changes which worlds satisfy the sentence.

Negation and scope must also be kept apart

"Not every technician approved" allows some approvals and at least one non-approval. "Every technician did not approve," under a reading with negation inside the universal quantifier, says that nobody approved. Written English can itself leave scope unclear; explicit paraphrases remove the ambiguity.

The Chinese contrast 不是每位技師都批准 / 每位技師都不批准 makes the intended distinction easier to display. The former negates "all approved"; the latter assigns non-approval to each technician. For a nonempty technician set:

$$\neg\forall t \in T\ P(t) \ \equiv\ \exists t \in T\ \neg P(t), \qquad \forall t \in T\ \neg P(t) \ \equiv\ \neg\exists t \in T\ P(t).$$

Here T is the technician set, $P(t)$ means t approved, $\neg$ is negation, $\forall$ and $\exists$ are universal and existential quantification, and $\equiv$ means logical equivalence. The first condition needs one coun-

terexample to universal approval; the second excludes every approval. If one technician approves and another does not, only the first condition is true.

NLP 12 develops predicate–argument structure and semantic role labeling, and NLP 11 tests inference between sentences. Before designing a field such as `approved`, decide whose approval, what event and which quantified claim that field represents.

7. Pragmatics: interpretation in a situation

Plain version. What a speaker means can go beyond what the sentence literally says. Pragmatics studies how the situation fills that gap.

Speakers often communicate more than their sentence literally asserts. **Pragmatics** studies language use in context, including reference, background assumptions and communicative actions [9, 10]. Context supplies evidence; it does not license arbitrary interpretations.

Reference and the speaker's viewpoint

"I will check that here tomorrow" is structurally simple but under-specified outside its setting. "I" identifies the speaker, "that" selects a referent, "here" uses a spatial viewpoint, and "tomorrow" uses a temporal viewpoint. Such context-dependent expressions involve **deixis** [10]. A forwarded message should not silently replace the original speaker's time and place with the receiver's.

Reference also depends on discourse. "Mei put a camera beside a lamp. It was overheating." Either object may be the referent of "it" without further evidence. Knowing that cameras and lamps can both become hot leaves the ambiguity intact. In "Mei turned off the lamp. It stopped flickering," the described event supplies evidence favoring the lamp. Evidence may favor a candidate without turning a grammatical possibility into a universal rule.

A practical analysis records the expression, candidate referents and supporting context. An unresolved referent is a meaningful result when the input does not determine one. Coreference resolution across sentences is developed in NLP 12, and tracking entities across dialogue turns in NLP 08.

Assertion, presupposition and implicature

An **assertion** contributes the claim at issue. A **presupposition** presents a proposition as background rather than as the main update. A **conversational implicature** is an additional inference drawn from the utterance and its use, which can often be cancelled without contradicting the literal assertion [9, 10]. They should not collapse into one "meaning" label.

"Mei stopped checking the sensor" asserts a cessation and normally backgrounds that Mei checked it previously. "Mei did not stop checking the sensor" still typically takes earlier checking for granted. That persistence under negation helps diagnose the background contribution. These

are ordinary interpretations, not a proof that the earlier event actually occurred. A listener can challenge it: “She never checked it in the first place.”

“Some panels passed” asserts that at least one panel passed in the relevant collection. In a conversation asking whether the whole collection passed, a listener may infer “not all.” But “Some panels passed; in fact, all of them did” can cancel that inference without denying the first assertion [9]. By contrast, “All panels passed; in fact, one failed” conflicts under the same panel collection, test and time.

This difference can be checked without a language model. Imagine three panels: one, two and three passing panels all satisfy the literal “some.” A “some but not all” interpretation retains only the first two possibilities. The inference narrows the possibilities because of a conversational expectation; that narrowing is not contributed by existential meaning alone. Whether it is justified depends on the question, speaker knowledge and manner of reporting.

A grammatical question can perform a request

“Can you lower the gain?” has the form of an ability question. During a recording session, it can perform a request to change a setting; during an assessment, it can ask whether the trainee knows how [10]. A **speech act** is the communicative action performed through an utterance, such as requesting, promising, asserting or asking.

“The gain is too high” can likewise function as a report or an indirect request. The physical words do not themselves specify every interpersonal condition. Who controls the equipment? What was the preceding question? Is the speaker diagnosing the recording or coordinating an adjustment? These details support an interpretation of the action.

A useful annotation separates sentence form, asserted content and intended act. Otherwise, every question-shaped request may be labeled as information seeking, or every unfavorable description may be labeled as an instruction. The goal is a defensible reading of this interaction, including alternatives when the setting is missing.

8. Discourse: meaning across sentences and turns

Plain version. Sentences in a text or a conversation lean on each other. Later sentences refer back, contrast, explain or correct, and a reader keeps track of that shared state.

An utterance can introduce an entity, correct an earlier claim, provide a reason or contrast with an expectation. **Discourse** analysis considers these connections across sentences and conversational turns [2, 10]. Reading each sentence independently discards the shared state that makes later expressions interpretable.

“Mei told Lin that she would calibrate the camera” can refer to either person as the future calibrator. The Chinese 美玲告訴怡君，她會校正相機 permits a comparable ambiguity in this invented setting. A preceding statement “Lin had already volunteered” favors Lin; “Mei was the only available technician” favors Mei. Neither grammatical gender nor mention order alone settles both contexts.

Now compare “The sensor became hot. Its readings stayed stable” with “The sensor became hot, but its readings stayed stable.” The second explicitly marks a contrast with an expectation that heat would disrupt readings. The assertion about stability can be the same while the discourse contribution differs. Conversely, “The readings drifted because the sensor became hot” introduces a causal explanation, not merely two events in sequence.

Conversations also contain repairs. “Move it to Friday.” “The inspection?” “Yes, the inspection.” The middle turn does not advance the scheduling date; it resolves the object. Once resolved, later references can use the shared identification. A dialogue system that summarizes only the first and last words misses why the clarification changed the conversational state.

This suggests a practical representation with distinct records for introduced entities, asserted events, unresolved references and conversational questions. Such a representation can remain partial. A one-sentence input may leave the speaker unknown; an entire exchange may resolve the speaker but still leave the exact camera unspecified. Increasing context length only helps when relevant evidence is actually present.

9. Corpora, annotation and Zipf’s law

Plain version. NLP learns from collections of real text, often with human labels added. How the text was collected and how consistently it was labeled limit what any model can learn from it, and every corpus has a long tail of rare words.

A **corpus** is a collection of text or transcribed speech gathered under stated decisions: which sources, which period, which speakers, which language varieties. An **annotated corpus** adds analyses such as segmentation, tags, trees, entity spans or sentiment labels, produced under written guidelines. Two widely used examples show the range. The Penn Treebank annotated English text with part-of-speech tags and constituency trees [26]. The Universal Dependencies project releases treebanks for many languages under one cross-lingual dependency scheme [24]. The segmentation bakeoff corpora above show that even word boundaries come with a standard attached [27].

Annotation guidelines and agreement

A label is only as clear as the guideline behind it. Two careful annotators can disagree because a sentence is genuinely ambiguous, because the guideline is silent, or because one of them made an error. Measuring agreement on a shared sample separates these cases before the labels are used for training.

Raw percent agreement overstates reliability when one label is very common, because annotators would agree often even by guessing. Cohen’s kappa corrects for that [28]:

$$\kappa = \frac{p_o - p_e}{1 - p_e}, \qquad p_e = \sum_{k \in \mathcal{Y}} \frac{n_{A,k}}{n} \cdot \frac{n_{B,k}}{n}.$$

Here p_o is the observed fraction of items on which annotators A and B agree; p_e is the agreement expected if each annotator chose labels independently at their own observed rates; $\mathcal{Y}$ is the label set; $n_{A,k}$ and $n_{B,k}$ count how often each annotator used label k; and n is the number of items. $\kappa = 1$ is perfect agreement and $\kappa = 0$ is agreement at the chance level; the formula requires $p_e < 1$.

```
from collections import Counter
from sklearn.metrics import cohen_kappa_score

words = "Mei has to repair the camera mount before the inspection".split()
a = "PROPN VERB PART VERB DET NOUN NOUN ADP DET NOUN".split()   # annotator A
b = "PROPN AUX ADP VERB DET NOUN NOUN ADP DET NOUN".split()     # annotator B
n = len(words)
p_o = sum(x == y for x, y in zip(a, b)) / n                     # observed agreement
ca, cb = Counter(a), Counter(b)
p_e = sum(ca[k] * cb[k] for k in ca) / n**2                     # agreement expected by
 ↪  chance
kappa = (p_o - p_e) / (1 - p_e)
print(f"p_o={p_o:.3f}  p_e={p_e:.3f}  kappa={kappa:.3f}")
assert abs(kappa - cohen_kappa_score(a, b)) < 1e-12
print("disagreements:", [(w, x, y) for w, x, y in zip(words, a, b) if x != y])
```

It prints `p_o=0.800  p_e=0.180  kappa=0.756`, and the two disagreements are `has` (`VERB` versus `AUX`) and `to` (`PART` versus `ADP`). Neither is a careless slip: "has to" sits on the border between a main verb and an auxiliary, and infinitival "to" resembles a preposition. The useful outcome is a guideline decision for that construction, not a higher number. Kappa summarizes reliability; the disagreement list says what to fix.

Zipf's law: a few words are very common, most are rare

Count every word in a corpus and sort the types by frequency. A handful of function words such as "the" and "of" account for a large share of all tokens, while a huge number of types occur only once or twice. **Zipf's law** describes the relation between a word's frequency rank and its count as approximately a power law [29]:

$$f(r) \approx \frac{C}{r^s}, \qquad \log f(r) \approx \log C - s \log r.$$

Here $r = 1, 2, \ldots$ is a type's rank when types are sorted by decreasing count, $f(r)$ is the count of the type at rank r, $C > 0$ is a scale constant and $s > 0$ is the exponent. With $s \approx 1$, the product $r \cdot f(r)$ stays roughly constant. On log–log axes the relation is a straight line with slope $-s$. Piantadosi's review shows that the law holds approximately across many languages and corpora, that it deviates systematically at the extremes, and that its explanation is still debated [29].

We can check it on real English text that is already on disk: the docstrings of the Python standard library installed in the project environment (Python 3.9).

```
import ast, re, sysconfig
from collections import Counter
```

```
from pathlib import Path

lib = Path(sysconfig.get_paths()["stdlib"])        # text already on disk: stdlib docstrings
docs = []
for path in sorted(lib.glob("*.py")):
    try:
        tree = ast.parse(path.read_text(encoding="utf-8"))
    except (SyntaxError, UnicodeDecodeError):
        continue
    docs += [ast.get_docstring(n) or "" for n in ast.walk(tree)
             if isinstance(n, (ast.Module, ast.FunctionDef, ast.AsyncFunctionDef,
              ↪  ast.ClassDef))]
tokens = [w for d in docs for w in re.findall(r"[a-z]+", d.lower())]
counts = Counter(tokens)
print(len(tokens), "tokens,", len(counts), "types")
ranked = counts.most_common()
for r in (1, 2, 10, 100, 1000):
    word, f = ranked[r - 1]
    print(f"rank {r:4d}  {word:10s} f={f:5d}  r*f={r * f}")
hapax = sum(f == 1 for f in counts.values())
print(f"types seen once: {hapax} ({hapax / len(counts):.0%})")
```

With Python 3.9.7 it finds 134,620 tokens and 6,281 types. The product $r \cdot f(r)$ is 8,313 at rank 1 (“the”), 13,640 at rank 10 (“be”), 20,100 at rank 100 (“called”) and 18,000 at rank 1,000: not constant, but within a factor of about 2.5 while the rank changes by a factor of 1,000. A least-squares fit over ranks 10 to 1,000 in the figure script gives $s \approx 1.00$. A third of all types, 2,003 of them (32%), occur exactly once. A different Python version ships different docstrings, so the exact counts can change.

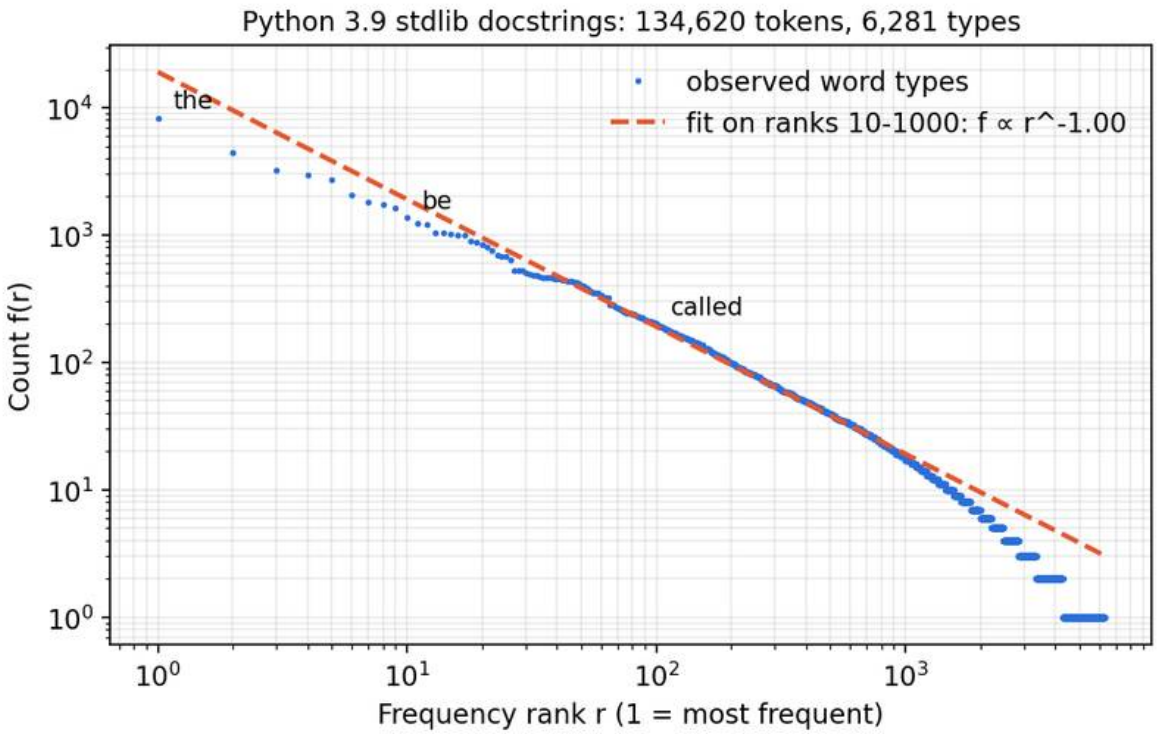


Figure 4 — Rank–frequency plot of the Python 3.9 standard-library docstrings. The dashed line is a power-law fit over ranks 10–1,000; the head and the tail of single-occurrence words bend away from it.

Three consequences follow for NLP. First, any fixed word vocabulary leaves out part of the tail, so new text keeps producing unseen words; this is why subword vocabularies exist. Second, count-based probability estimates are reliable for frequent words and unreliable for the rare majority, which is why smoothing appears in n-gram models (NLP 05). Third, accuracy on all tokens is dominated by frequent words; an evaluation that matters for rare names or technical terms must measure them separately.

10. Linguistic diversity and model assumptions

Plain version. Languages package information differently, and communities speak and write in different varieties. A model built around one language's habits can miss what another language says.

Languages and communities distribute information differently across morphology, order, prosody and context. **Typology** compares such structural patterns. **Dialect** concerns a community's variety; **register** concerns usage associated with a setting or activity [3, 11]. These distinctions describe variation rather than rank ways of speaking.

Translation does not preserve every structure

Compare “I calibrated this camera yesterday” with 這台相機，我昨天校正過. The Chinese example begins with the camera as the topic and then comments on it. Treating the first noun phrase as the actor would misidentify the calibrator. An English translation can preserve the event while changing how the message organizes old and new information.

The short Chinese 已經校正好了 can be informative when the camera and responsible person are already established. A word-for-word translation that demands overt counterparts for every English argument may make it look incomplete. In context it can answer a pending question directly. This does not mean that any omitted participant is always recoverable; the analysis must examine the actual discourse.

Similarly, 三台相機 and “three cameras” package number differently. A feature called “plural suffix” cannot be required universally merely because it suits one language. Structural categories may be useful for comparison while needing language-specific realizations and annotation decisions. A language using fewer obligatory endings can still express detailed distinctions through constructions, words and context.

Variety, setting and mixed-language expression

A formal maintenance report and a chat between technicians can describe the same equipment with different vocabulary, abbreviation and explicitness. The report “The shutter failed to actuate” and the informal “The shutter didn’t fire” may describe a related event in context, yet the wording provides different evidence about setting and intended readership. Unfamiliar wording is not automatically an error.

A speaker might write 把 exposure 調低一點 in a bilingual discussion. The English item occupies a position in an otherwise Chinese construction. **Code-switching** involves alternation between linguistic varieties within an interaction [14]; analyzing this example requires both the lexical choice and the surrounding grammar. One occurrence alone may not settle whether a community treats the item as switching or as an established borrowing.

An analysis should preserve the observed expression before standardizing it. Replacing a regional or technical expression with a presumed standard can help a particular search task, but may erase a distinction needed for another task. A corpus records language from particular speakers, settings and collection decisions; frequent corpus patterns are evidence of usage, not a universal inventory of everything speakers can say.

11. A complete worked reading

Put the levels together with an invented message:

> 小林說：「那台相機又過熱了，你明天能幫我看看嗎？」
>
> Lin said: “That camera overheated again. Can you take a look for me tomorrow?”

The translation helps an English reader but already chooses wording for 看看 and punctuation for the quoted interaction. Analyze the original before treating the translation as equivalent evidence.

Level	What the message contributes	What remains to be established
Written form and segmentation	The quotation groups Lin's utterance; 相機 is a camera expression	Speech intonation and gesture are absent
Morphology and construction	看看 is a reduplicated form in this request, not a counted pair of inspections	Exact activity and expected effort depend on the setting
Syntax	小林 introduces the reported speaker; 你 is the addressee inside the quotation	The reporter need not be the addressee
Semantics	The camera is described as overheating; a future assistance question is expressed	Which camera, and whether the description is factually correct
Background and reference	又 normally invokes a prior occurrence; 那台 selects a contextually identified camera	Prior overheating and the demonstrative referent require evidence
Pragmatics and discourse	The question naturally supports a request in a maintenance conversation	Whether that reading fits this interaction and whether it is accepted

The quotation changes the reference frame. "我" refers to Lin as the quoted speaker. "你" refers to Lin's addressee, who may differ from the current reader. "明天" is relative to the quoted utterance's day. If the report arrives later, its arrival date does not redefine the requested day.

The ordinary reading of 看看 presents the activity as taking a look, often with a light or tentative framing in this construction [13]. It does not establish a precise inspection duration or a numerical repetition count. Translating it as "inspect twice" would confuse visible repetition with event counting.

Finally, separate linguistic evidence from factual evidence. The message provides evidence that Lin reported overheating and requested help. It does not by itself verify overheating, identify a serial number or show that anyone agreed to help. A structured output might record the speaker and reported event while retaining unresolved referents and dates. This is a more informative analysis than assigning a single topic label such as "camera repair."

Modern view

Recent surveys show why language analysis remains necessary for large language models. Ma and colleagues' 2025 survey organizes pragmatic evaluation by phenomena including reference and deixis, implicature and presupposition, speech acts, discourse and social pragmatics [10]. It compares task design, collection and evaluation, and identifies gaps between available tests and contextual language use. Its useful lesson here is to evaluate the particular inference required, rather than treating a broad language score as proof of all pragmatic abilities.

Mielke and colleagues' tokenization review organizes the open-vocabulary problem across words, characters and intermediate units [22]. Its useful question is how segmentation balances coverage,

sequence length and reuse. These quantities remain distinct: a tokenizer can represent every input string while giving a model little training evidence for the relevant combinations. Zipf's law explains why the problem never goes away: however large the vocabulary, the tail keeps supplying rare forms [29].

Qin and colleagues' 2025 survey separates multilingual alignment through parameter tuning from alignment with frozen parameters, including prompting and translation-based approaches [12]. It reviews cross-language training, evaluation and challenges. This distinction identifies where a system's behavior is being adapted, but multilingual output alone does not establish that it preserves a construction's scope, register or conversational force.

Winata and colleagues' 2023 systematic survey groups code-switching research by languages, tasks and research trends [15]. Read it alongside Doğruöz and colleagues' account of structural and social patterns [14]: testing on text assembled from two monolingual sources is a different task from interpreting how bilingual speakers actually switch during an interaction. The exposure example in Section 10 makes that distinction concrete: the surrounding construction and conversational setting contribute information beyond identifying the English item.

Linguistic analysis also continues to refine apparently simple constructions. Lu and Müller's 2026 study analyzes Mandarin verb reduplication and its interaction with verb classes and aspect [13]. For this tutorial, the relevant observation is that limited-event meaning and context-dependent pragmatic effects deserve separate analysis. Visible repetition is not a general instruction to count events twice.

A concrete evaluation can vary one property while holding others fixed. Reverse Ada and Bo to test role direction; change camera attachment while preserving words; compare "not all" with "none"; move a quoted message into a later reporting context. Write the expected interpretations and acceptable alternatives first. Then a failure reveals which distinction was lost. These examples are small diagnostic tests, not a claim that passing them establishes general language competence.

Open problems include recovering relevant context, distinguishing genuine ambiguity from avoidable annotation disagreement, evaluating naturally mixed-language conversation and measuring interpretation across communities rather than only translated prompts. Some inputs legitimately admit several readings. A system that states the alternatives and the missing evidence may be more faithful to the language than one that confidently chooses a single answer.

Key takeaways

- Spoken, signed and written language expose different information; a transcript is a partial record.
- Computational tokens need not coincide with words, morphemes, lexemes or sound units; subword pieces from BPE, WordPiece or a unigram model reflect frequency, not morphology.
- Stemming strips endings by rule; lemmatization maps to a dictionary form and needs the part of speech. Both discard information some tasks need.
- Morphological structure and Chinese segmentation encode analytical choices with possible alternatives, and corpora follow different standards.

- Word counts discard order; part-of-speech categories, grouping and grammatical relations carry additional meaning. Constituency shows groups, dependency shows heads.
- Scope determines truth conditions: "each has some" need not mean "some shared by all."
- Assertion, background assumptions, cancellable inferences, reference, speech acts and discourse relations require the speaker's situation and conversational history.
- Corpora come with collection choices and annotation guidelines; check agreement, and expect a Zipfian long tail of rare words.

Exercises

1. For "The technician repaired two cameras," identify a word form, a lexeme, an inflectional contrast, a UPOS tag sequence and a possible computational tokenization. Explain what each representation preserves.

   Hint

   Compare "repaired" with REPAIR. A tokenizer could keep the whole form or split it; neither choice alone establishes the tense interpretation, which UD records as a feature.

2. Add one sentence of context for each reading of 學生會說話. Explain why the lexicon enumeration alone cannot prefer either context.

   Hint

   One setting can concern students' abilities; another can concern a student council announcing its position.

3. Change one training word's frequency in the BPE corpus. Inspect the first merge and encode `lowest` again. Why need neither result remain unchanged?

   Hint

   Pair counts are weighted by word occurrence counts. An early merge changes the pairs available later.

4. Write two contexts for "The technician inspected the panel with the camera," one for each attachment. Identify which head receives the modifier and how the event description changes.

   Hint

   Contrast the inspection method with identifying a camera-equipped panel among several panels.

5. Modify the quantifier program to add a technician who uses no camera. Predict both readings before running it, then explain the implication from B to A.

Hint

Both conditions fail if any technician has no usable pair. A single shared camera supplies a witness for every technician.

6. Analyze “Mei stopped checking the sensor” and “Some panels passed” separately under negation and explicit cancellation. Distinguish the main assertion, ordinary background assumption and conversational inference.

Hint

Try “Mei never checked it” as a challenge to background and “in fact, all passed” as cancellation of an inference. Keep the panel collection and test fixed.

References

1. Masato Hagiwara, *Real-World Natural Language Processing: Practical Applications with Deep Learning*, Manning, 2021. Publisher
2. Susan McRoy, *Principles of Natural Language Processing*, University of Wisconsin–Milwaukee open textbook, 2021. Grammars and Syntactic Processing
3. Catherine Anderson, Bronwyn Bjorkman, Derek Denis, Julianne Doner, Margaret Grant, Nathan Sanders and Ai Taniguchi, *Essentials of Linguistics*, 2nd ed., eCampusOntario, 2022. Open textbook
4. Catherine Anderson et al., “3.1 Modality,” *Essentials of Linguistics*, 2nd ed., eCampusOntario, 2022. Chapter
5. Catherine Anderson et al., “4.1 Phonemes and allophones,” *Essentials of Linguistics*, 2nd ed., eCampusOntario, 2022. Chapter
6. Catherine Anderson et al., “3.12 Tone and intonation,” *Essentials of Linguistics*, 2nd ed., eCampusOntario, 2022. Chapter
7. Catherine Anderson et al., “5.1 What is morphology?,” *Essentials of Linguistics*, 2nd ed., eCampusOntario, 2022. Chapter
8. Catherine Anderson et al., “6.4 Identifying phrases: Constituency tests,” *Essentials of Linguistics*, 2nd ed., eCampusOntario, 2022. Chapter
9. Catherine Anderson et al., “8.4 Conversational implicatures,” *Essentials of Linguistics*, 2nd ed., eCampusOntario, 2022. Chapter
10. Bolei Ma, Yuting Li, Wei Zhou, Ziwei Gong, Yang Janet Liu, Katja Jasinskaja, Annemarie Friedrich, Julia Hirschberg, Frauke Kreuter and Barbara Plank, “Pragmatics in the Era of Large Language Models: A Survey on Datasets, Evaluation, Opportunities and Challenges,” *ACL*, pp. 8679–8696, 2025. Paper
11. Catherine Anderson et al., “10.2 Language varies,” *Essentials of Linguistics*, 2nd ed., eCampusOntario, 2022. Chapter
12. Libo Qin, Qiguang Chen, Yuhang Zhou, Zhi Chen, Yinghui Li, Lizi Liao, Min Li, Wanxiang Che and Philip S. Yu, “A survey of multilingual large language models,” *Patterns*, 6(1), 101118, 2025. Paper

13. Yanru Lu and Stefan Müller, “Deliminative verbal reduplication in Mandarin Chinese,” *Journal of Linguistics*, 62(3), pp. 595–632, 2026. Paper
14. A. Seza Doğruöz, Sunayana Sitaram, Barbara E. Bullock and Almeida Jacqueline Toribio, “A Survey of Code-switching: Linguistic and Social Perspectives for Language Technologies,” *ACL-IJCNLP*, pp. 1654–1666, 2021. Paper
15. Genta Winata, Alham Fikri Aji, Zheng Xin Yong and Thamar Solorio, “The Decades Progress on Code-Switching Research in NLP: A Systematic Survey on Trends and Challenges,” *Findings of ACL*, pp. 2936–2978, 2023. Paper
16. M. F. Porter, “An algorithm for suffix stripping,” *Program*, 14(3), pp. 130–137, 1980. DOI
17. Rico Sennrich, Barry Haddow and Alexandra Birch, “Neural Machine Translation of Rare Words with Subword Units,” *ACL*, 2016. Paper
18. Mike Schuster and Kaisuke Nakajima, “Japanese and Korean Voice Search,” *ICASSP*, pp. 5149–5152, 2012. DOI
19. Yonghui Wu et al., “Google’s Neural Machine Translation System: Bridging the Gap between Human and Machine Translation,” arXiv:1609.08144, 2016. Paper
20. Taku Kudo, “Subword Regularization: Improving Neural Network Translation Models with Multiple Subword Candidates,” *ACL*, 2018. Paper
21. Taku Kudo and John Richardson, “SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing,” *EMNLP System Demonstrations*, 2018. Paper
22. Sabrina J. Mielke et al., “Between Words and Characters: A Brief History of Open-Vocabulary Modeling and Tokenization in NLP,” arXiv:2112.10508, 2021. Review
23. Hugging Face, “The Tokenization Pipeline,” Tokenizers documentation, current online edition. Documentation
24. Marie-Catherine de Marneffe, Christopher D. Manning, Joakim Nivre and Daniel Zeman, “Universal Dependencies,” *Computational Linguistics*, 47(2), 2021. Paper
25. Universal Dependencies contributors, “Universal POS tags,” Universal Dependencies documentation, current online edition. Documentation
26. Mitchell P. Marcus, Beatrice Santorini and Mary Ann Marcinkiewicz, “Building a Large Annotated Corpus of English: The Penn Treebank,” *Computational Linguistics*, 19(2), 1993. Paper
27. Thomas Emerson, “The Second International Chinese Word Segmentation Bakeoff,” *Proceedings of the Fourth SIGHAN Workshop on Chinese Language Processing*, 2005. Paper
28. Jacob Cohen, “A Coefficient of Agreement for Nominal Scales,” *Educational and Psychological Measurement*, 20(1), pp. 37–46, 1960. DOI
29. Steven T. Piantadosi, “Zipf’s word frequency law in natural language: A critical review and future directions,” *Psychonomic Bulletin & Review*, 21(5), pp. 1112–1130, 2014. DOI
30. Jacob Devlin, Ming-Wei Chang, Kenton Lee and Kristina Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” *NAACL-HLT*, 2019. Paper

NLP 03 • Neural Networks for NLP

From count features and logistic classification to embedding layers, the neural probabilistic language model and 1-D convolutional text classifiers, with padding, masking, regularization and a complete PyTorch experiment.

Online edition: https://blog.shyandram.dev/en/learn/nlp/nlp-03-neural-networks-for-nlp/

What you'll learn

- Define a text classification task with explicit annotation rules, then derive count features, TF–IDF, logistic classification and cross-entropy.
- Turn token IDs into learned vectors with an embedding layer, pool them, and classify with a small multilayer perceptron.
- Follow the neural probabilistic language model of Bengio et al. and explain why shared word vectors let it generalize to unseen histories.
- Write the mathematics of a 1-D convolutional text classifier with max-over-time pooling, and implement it with correct padding and masking.
- Regularize, train and early-stop a small PyTorch model, read its training curves and analyze its errors by category.
- Evaluate decisions, probability quality, coverage and class imbalance separately.

The big picture

A text classifier converts a message into a permitted decision: route a complaint, identify a language, label a review or flag a document for inspection. Its most consequential choices often occur before the neural architecture is selected. What counts as one example? Which context is visible? What does the label mean? What should happen when none of the available labels fits?

Hagiwara develops a first classification application, word and document embeddings, sentence classification and convolutional encoders in Chapters 2–4 and 7 [1]. McRoy's Chapter 6 distinguishes sentiment, similarity, grammaticality and entailment as different language tasks [2]. This lesson follows that combination: build an application, but make its prediction target precise. A sentiment label is not an entailment judgment, and a similarity score is not a statement that two claims have the same truth conditions.

> **Explain like I'm five**
>
> Imagine sorting letters into three trays: payments, deliveries and technical help. Counting words is like looking for familiar clues on each envelope. A neural network learns its own clues: it gives every word a small list of numbers, learns which short phrases matter, and adds up the evidence. Either way,

a trustworthy sorter also needs a fourth action: ask someone when the letter does not clearly belong in a tray.

The lesson has two halves. The first builds the strongest simple baseline: sparse features and a linear classifier, trained and evaluated with care. The second replaces hand-designed features with learned ones: an **embedding layer**, a pooled multilayer perceptron, the neural probabilistic language model and a convolutional encoder. Every neural model here is still trained with the same cross-entropy loss as the linear baseline, so the change is in the representation, not the objective. Backpropagation, activation functions and optimizers are developed in ML 02; this page uses them and focuses on what is specific to text.

All examples use invented messages and require no dataset download. Measured scores describe these synthetic messages; they cannot establish performance on an actual application.

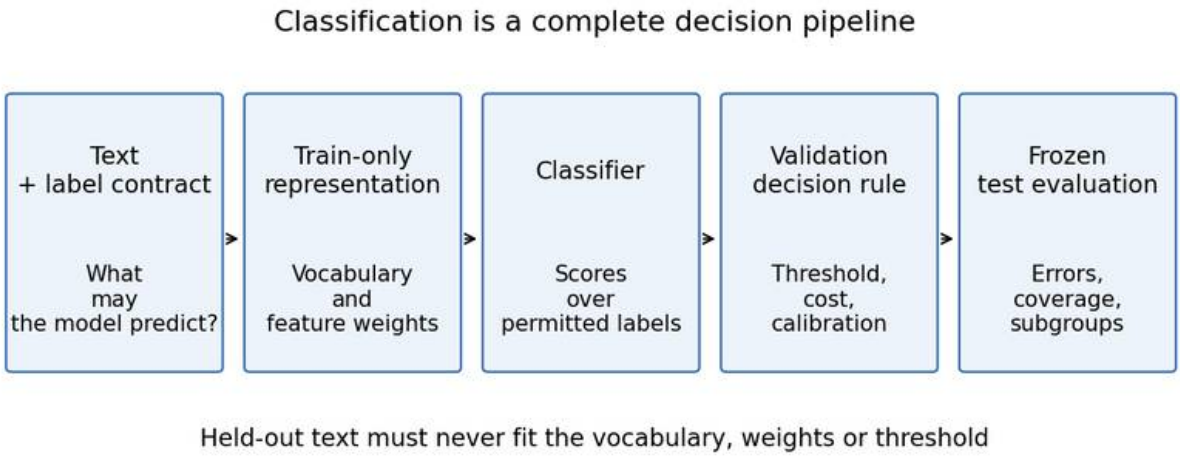


Figure 1 — Classification includes the representation, prediction and action rule. Holding out the final test set applies to every learned stage, including neural feature learning.

Define the prediction contract

Plain version. A label is an agreement about what a piece of text means for this application. Establish that agreement before collecting examples or optimizing a loss.

In single-label classification, one input receives exactly one label from a fixed set. In multi-label classification, several labels may apply simultaneously. A complaint about both a delayed parcel and an incorrect charge may require two routes, a primary route with secondary tags, or a human triage decision. These are different tasks even if the input text is identical.

Write the contract in ordinary language first. For our toy application, billing concerns a payment or invoice, delivery concerns a parcel or shipping, and technical concerns account access or software behavior. We permit an abstention action after classification. An ambiguous multi-topic

message is therefore not silently forced into a category merely because the numerical model has an argmax.

$$f_\theta : \mathcal{X} \to \Delta^{K-1}, \qquad \hat{y}(x) = \operatorname*{argmax}_{k \in \{1,\dots,K\}} p_\theta(y = k \mid x).$$

Here f_θ is a classifier with learned parameters θ, $\mathcal{X}$ is the set of allowed text inputs, Δ^{K-1} is the simplex of probability vectors over K classes, x is one input, y is its label, k indexes a class, p_θ is the model's conditional distribution and $\hat{y}$ is its most probable class. The simplex requires nonnegative entries whose sum is one.

This equation describes a model output, not the whole product action. The service may return a route only when confidence passes a validation-selected threshold. It may also reject an empty message, ask for additional context or send a multi-topic message to review. Those actions should be visible in evaluation.

Keep the unit of supervision aligned with deployment. A sentence-level sentiment model trained on isolated opinions may fail on a whole conversation containing quotations, corrections and several speakers. If deployment includes previous turns, annotation and training should include the same permitted context. Conversely, giving annotators access to the final resolution while the model only sees the first message creates an impossible target.

Count features and the vocabulary

Plain version. The simplest useful representation records which words or short sequences occur. It is inexpensive, inspectable and often an excellent first comparison.

Choose a vocabulary from training text only. Its entries might be words, character sequences or consecutive word pairs. For a word vocabulary, each coordinate counts one vocabulary entry. A document becomes a sparse vector because most possible words are absent.

$$x_j(d) = c(v_j, d), \qquad \mathbf{x}(d) = (x_1(d), \dots, x_V(d))^\top.$$

Here d is a document, v_j is the vocabulary item at coordinate j, $c(v_j, d)$ is its occurrence count in d, $x_j(d)$ is that feature value, $\mathbf{x}(d)$ is the resulting column vector and V is the vocabulary size. The transpose symbol $\top$ specifies that the listed coordinates form a column.

This bag-of-words representation ignores order. "Payment arrived after the refund" and "Refund arrived after the payment" contain the same words but describe different event orders. Word bigrams recover some local information, while character features can capture spelling fragments and inflections. Neither automatically understands a long dependency or the scope of a negation.

Feature choices encode useful assumptions. Lowercasing can reduce sparsity, but it merges names with common nouns. Removing every stop word can remove "not," which is essential to many classifications. A language detector may benefit from character patterns rather than a word vocabulary tailored to one language. Use the task definition to justify each normalization rule; NLP 02 discusses the linguistic units behind these choices.

Fit vocabulary construction, document-frequency thresholds and any feature selection inside the training portion of each experiment. Looking at test documents to decide which rare terms to retain gives the training pipeline information about the evaluation distribution. It can be subtle leakage even when no test label is read.

Keep a record of vocabulary size, proportion of messages with no recognized features, token counts and representative discarded terms. These diagnostics reveal changes that a single accuracy score hides. A new product code or language can make many inputs nearly empty without causing an obvious exception.

TF–IDF and vector normalization

Plain version. A word that occurs in almost every message is usually less useful for distinguishing messages. TF–IDF balances occurrence within a document against rarity across training documents.

One common smoothed definition, used by the implementation here, is:

$$\mathrm{idf}_j = \log \frac{1+N}{1+\mathrm{df}_j} + 1, \qquad z_j(d) = c(v_j, d)\,\mathrm{idf}_j, \qquad \tilde{\mathbf{z}}(d) = \frac{\mathbf{z}(d)}{\|\mathbf{z}(d)\|_2}.$$

Here idf_j is the inverse-document-frequency weight for vocabulary entry v_j, N is the number of training documents, df_j counts training documents containing that entry, $c(v_j, d)$ is its count in document d, $z_j(d)$ is the weighted coordinate, $\mathbf{z}(d)$ collects those coordinates and $\tilde{\mathbf{z}}(d)$ is its unit-length version. The Euclidean norm $\|\mathbf{z}(d)\|_2$ is the square root of the sum of squared coordinates. A zero vector remains zero rather than being divided by zero. The scikit-learn feature-extraction documentation specifies these implementation choices [3].

The added ones prevent undefined weights for the fitted vocabulary and fix the precise smoothing convention. Other libraries use other conventions, so “TF–IDF” alone is not a complete reproducibility statement. Record the tokenizer, feature range, count transformation, weighting formula and normalization.

Normalization makes a long message less dominant simply because it repeats words. That helps when direction in feature space matters more than total length. But length can itself be predictive: an error report might be longer than a password-reset request. If length is relevant, add an explicit length feature instead of relying on an accidental effect of unnormalized counts.

TF–IDF does not infer synonymy. “Charge” and “payment” remain different coordinates unless examples teach the classifier their similar role. It also weights rare identifiers strongly even when their rarity is irrelevant. An account number might look discriminative because a template is duplicated across related records. Grouped splitting and inspecting coefficients help reveal that shortcut.

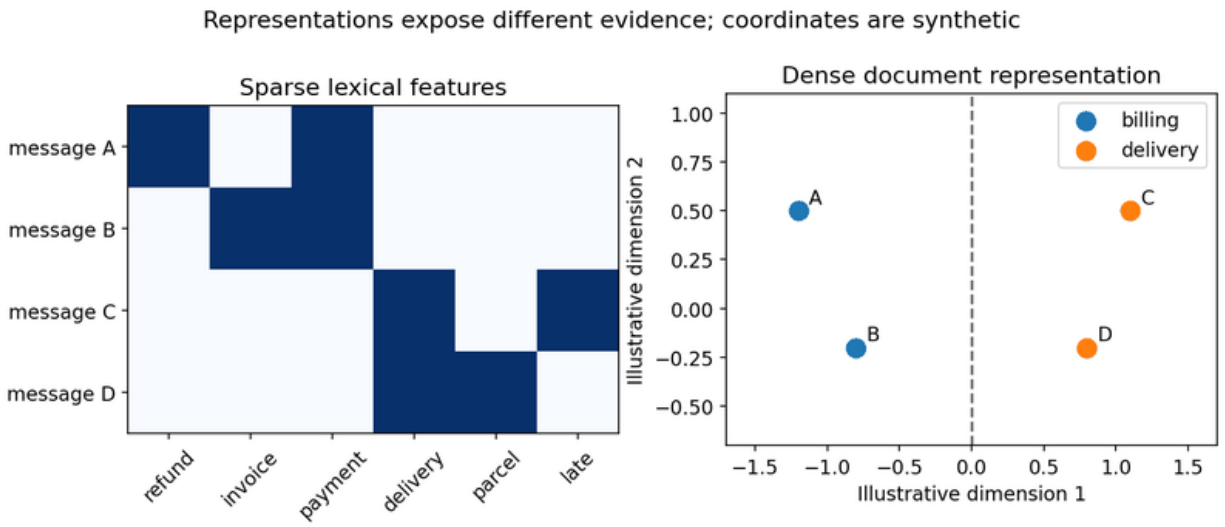


Figure 2 — Sparse features preserve explicit lexical evidence. Dense representations compress patterns into coordinates whose meaning depends on the training objective.

Logistic classification and its loss

Plain version. A linear classifier adds evidence from features to produce one score per label. Turning scores into probabilities gives a differentiable training objective and a basis for decisions.

For a multiclass model:

$$s_k = \mathbf{w}_k^\top \tilde{\mathbf{z}} + b_k, \qquad p_\theta(y = k \mid d) = \frac{\exp(s_k)}{\sum_{\ell=1}^{K} \exp(s_\ell)}.$$

Here s_k is the score for class k, $\mathbf{w}_k$ is its learned feature-weight vector, $\tilde{\mathbf{z}}$ is the normalized document representation, b_k is a learned bias, p_θ is the conditional class probability, θ collects all weights and biases, d is the document, ℓ indexes competing classes and K is their number. The function exp is the exponential function.

The probability of the annotated class should increase during training. Regularization discourages arbitrarily large weights that fit incidental words:

$$\mathcal{L}(\theta) = -\frac{1}{n} \sum_{i=1}^{n} \log p_\theta(y_i \mid d_i) + \frac{\lambda}{2} \sum_{k=1}^{K} \|\mathbf{w}_k\|_2^2.$$

Here $\mathcal{L}$ is the objective, n is the training-example count, i indexes examples, (d_i, y_i) is an annotated document and label, $\lambda \geq 0$ is the regularization strength, and $\|\mathbf{w}_k\|_2^2$ is the sum of squared feature weights for class k. The first term is multiclass cross-entropy; the second is squared-weight regularization. Library objectives can scale the terms differently; scikit-learn exposes inverse regularization through its parameter C [4]. IT 01 explains why the first term is a cross-entropy.

A large positive coefficient says that a feature increases a class score relative to the other evidence. It does not prove that the feature causes the true category. Correlated words can share or exchange weight, and changing the vocabulary changes the meaning of a coefficient comparison.

For binary classification, the softmax can be written with one sigmoid of a score difference. For independent multi-label prediction, use a separate sigmoid per label rather than a softmax that forces labels to compete. Choose a loss and output activation that match the annotation contract, not whichever head happens to be easiest to implement.

Read as a network, this model is a single linear layer followed by a softmax. Every neural classifier below keeps that final layer and its loss. What changes is the vector $\tilde{\mathbf{z}}$: instead of being computed by a fixed counting rule, it is computed by learned layers.

A complete CPU classification experiment

Plain version. The vocabulary, classifier and threshold must learn from the intended portions of data. A pipeline makes this boundary easier to preserve.

The following standalone example uses invented messages. Training, validation and test text are explicitly separate. Validation selects a threshold using a stated illustrative cost: one wrong route costs two units, while asking for review costs one fifth of a unit. These values are policy parameters for this demonstration, not measured business costs. All choices freeze before the final test is inspected.

```
import numpy as np
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import f1_score, log_loss

train = [
    ("please refund my payment", "billing"),
    ("the invoice has a wrong charge", "billing"),
    ("my payment was charged twice", "billing"),
    ("how can i update the billing card", "billing"),
    ("please send the payment receipt", "billing"),
    ("the refund is missing from my invoice", "billing"),
    ("where is my delivery parcel", "delivery"),
    ("my shipment is late", "delivery"),
    ("please change the delivery address", "delivery"),
    ("the parcel arrived damaged", "delivery"),
    ("how do i track the shipment", "delivery"),
    ("the delivery driver could not find the address", "delivery"),
    ("i cannot login to my account", "technical"),
    ("the application crashes on startup", "technical"),
    ("please reset my account password", "technical"),
    ("the software shows an error", "technical"),
    ("my login fails after the software update", "technical"),
    ("the application cannot connect to my account", "technical"),
]
```

```
validation = [
    ("i need a receipt for the charge", "billing"),
    ("the billing payment needs a refund", "billing"),
    ("can i track this delivery", "delivery"),
    ("the shipment went to another address", "delivery"),
    ("my password does not allow login", "technical"),
    ("the software crashes with an error", "technical"),
]
test = [
    ("please explain the invoice payment", "billing"),
    ("a charge appears before the refund", "billing"),
    ("the parcel delivery is not here", "delivery"),
    ("my shipment arrived at the wrong address", "delivery"),
    ("an account error prevents login", "technical"),
    ("the application crashes after a password change", "technical"),
]

def unpack(rows):
    return [text for text, _ in rows], np.array([label for _, label in rows])

train_x, train_y = unpack(train)
val_x, val_y = unpack(validation)
test_x, test_y = unpack(test)
model = Pipeline([
    ("features", TfidfVectorizer(ngram_range=(1, 2))),
    ("classifier", LogisticRegression(C=4.0, max_iter=300, random_state=0)),
])
model.fit(train_x, train_y)
classes = model.named_steps["classifier"].classes_

def decision(probabilities, threshold):
    best = probabilities.argmax(axis=1)
    prediction = classes[best]
    accepted = probabilities[np.arange(len(best)), best] >= threshold
    return prediction, accepted

val_prob = model.predict_proba(val_x)
grid = [0.40, 0.50, 0.60, 0.70, 0.80]
costs = []
for threshold in grid:
    pred, accepted = decision(val_prob, threshold)
    wrong = accepted & (pred != val_y)
    costs.append((2.0 * wrong.sum() + 0.2 * (~accepted).sum()) / len(val_y))
threshold = grid[int(np.argmin(costs))]  # first minimum fixes the tie rule

test_prob = model.predict_proba(test_x)
test_pred, accepted = decision(test_prob, threshold)
print("validation-selected threshold:", threshold)
print("test macro F1:", round(f1_score(test_y, test_pred, average="macro"), 3))
print("test log loss:", round(log_loss(test_y, test_prob, labels=classes), 3))
print("test coverage:", round(accepted.mean(), 3))
if accepted.any():
    print("accepted accuracy:", round((test_pred[accepted] == test_y[accepted]).mean(), 3))
else:
```

```
    print("accepted accuracy: undefined (no accepted messages)")
probe = ["the weather is pleasant", "my delivery charge is wrong"]
for text, probabilities in zip(probe, model.predict_proba(probe)):
    prediction, keep = decision(probabilities[None, :], threshold)
    print(text, "->", prediction[0] if keep[0] else "review")
```

The run prints:

```
validation-selected threshold: 0.4
test macro F1: 1.0
test log loss: 0.413
test coverage: 1.0
accepted accuracy: 1.0
the weather is pleasant -> delivery
my delivery charge is wrong -> delivery
```

Six perfectly classified test messages are far too few to support any claim about the method; they only show that the pipeline runs end to end. The validation set made every threshold above 0.40 look costly, so the lowest grid value won, and the policy accepts everything.

The unfamiliar weather message is the useful part. None of the three routes is correct, yet the model still returns a probability distribution, and here its largest probability (about 0.51 for delivery) passes the selected threshold. The output head has no additional class for arbitrary outside topics. An abstention threshold can reduce some inappropriate actions, but a probability maximum is not a guaranteed out-of-domain detector, and in this run it does not catch the message at all. The mixed-topic "delivery charge" message is likewise forced into one route.

Notice that the final macro F1 measures forced class predictions over all test examples, while coverage and accepted accuracy describe the action rule. Reporting only accepted accuracy can hide a model that rejects nearly everything. Reporting only forced accuracy can hide useful selective behavior. Keep both views and their denominators explicit.

For a real application, replace these toy lists with a documented corpus and group identifiers. Messages from one conversation should not be split randomly across train and test. Shared templates, repeated signatures and near-duplicate descriptions also need scrutiny. The small example intentionally exposes mechanics; it offers too few validation examples for a stable threshold estimate.

A controlled negation test

Changing the feature vocabulary can preserve information that an unordered word count loses. Here we hold the invented labeled corpus and classifier family fixed, then compare single-token features with a mixture of single tokens and adjacent pairs. The TF–IDF and logistic implementations are documented in [3,4].

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import make_pipeline

train = [
    ("the battery is good", 1), ("good screen and good sound", 1),
    ("the camera is great", 1), ("great value , works well", 1),
    ("the battery is not good", 0), ("not great , the screen broke", 0),
    ("the sound is bad", 0), ("bad camera and bad battery", 0),
    ("works well , not bad at all", 1), ("not good , not worth it", 0),
]
test = ["the screen is not good", "the sound is not bad", "great battery"]
texts, labels = zip(*train)

for name, ngrams in [("unigrams", (1, 1)), ("uni+bigrams", (1, 2))]:
    clf = make_pipeline(TfidfVectorizer(ngram_range=ngrams, token_pattern=r"\S+"),
                        LogisticRegression(C=10.0))
    clf.fit(texts, labels)                    # vocabulary + IDF fitted on train only
    probs = clf.predict_proba(test)[:, 1]
    print(f"{name:12s}", [f"{t!r}: {p:.2f}" for t, p in zip(test, probs)])
```

The positive-review probabilities are:

```
unigrams     ["'the screen is not good': 0.43", "'the sound is not bad': 0.11", "'great
 ↪  battery': 0.61"]
uni+bigrams  ["'the screen is not good': 0.20", "'the sound is not bad': 0.18", "'great
 ↪  battery': 0.56"]
```

Both models give “the screen is not good” a probability below one half; adding pairs decreases it from 0.43 to 0.20. For “the sound is not bad,” the probability rises from 0.11 to 0.18 but remains below one half. These observations establish the behavior of the fitted toy models. They do not isolate one coefficient as the cause: adding pairs changes the vocabulary, normalization and joint fitted weights.

The second sentence is a useful error probe because a local phrase feature is available, yet the predicted label is still wrong under the intended positive reading. The corpus contains only one positive “not bad” occurrence, with other positive wording. Learning which feature carries that label is underdetermined by such sparse evidence. Inspect feature contributions and add controlled examples that vary negation while keeping topic fixed before drawing a general conclusion about the architecture.

There is also a distinction between phrase identity and scope. Adjacent features can distinguish “not good” from “good,” but recognizing “not every repair succeeded” versus “every repair failed” requires a different compositional distinction. More pairs do not automatically provide a complete semantic analysis. NLP 02 and NLP 12 work through such cases explicitly.

Two limits of sparse features motivate the rest of this lesson. First, every bigram is a separate coordinate: “not good” and “not great” share nothing unless both appear in training with labels.

Second, the number of possible n-grams grows quickly with n, so long patterns are rarely observed. Neural models address both by representing words as vectors that can share evidence, and by learning which combinations of those vectors matter.

From word IDs to learned vectors: the embedding layer

Plain version. Give every vocabulary entry a short vector of learned numbers. Words that behave alike in the training task can end up with similar vectors, so evidence about one helps the other.

An **embedding layer** is a table with one row per vocabulary entry. Looking up a token is the same as multiplying its one-hot vector by the table:

$$\mathbf{e}_t = E^\top \mathbf{o}_{i_t} = E^\top_{i_t,:}, \qquad E \in \mathbb{R}^{V \times D}.$$

Here i_t is the vocabulary ID of the token at position t, $\mathbf{o}_{i_t} \in \{0,1\}^V$ is the one-hot vector with a single one in coordinate i_t, E is the learned embedding table with V rows and D columns, $E_{i_t,:}$ is row i_t, and $\mathbf{e}_t \in \mathbb{R}^D$ is the token's embedding as a column vector. D is usually far smaller than V, which is why embeddings are called **dense**: most coordinates are nonzero, unlike the sparse count vector.

The lookup has a useful consequence for training. The gradient of the loss with respect to E is nonzero only in rows of tokens that appear in the current batch, so each update touches a few rows. A word that never appears in training keeps its initial row, which is why a vocabulary needs an explicit unknown-word entry and a policy for using it. The table can be initialized randomly and trained with the classifier, or initialized from vectors pretrained on unlabeled text; NLP 04 explains how such vectors are learned and what their geometry does and does not mean.

Pool, then classify: the bag-of-embeddings MLP

A document has a variable number of tokens, while a classifier layer expects a fixed-width input. The simplest neural answer is to average the token vectors and feed the average to a **multilayer perceptron (MLP)**:

$$\mathbf{h} = \frac{1}{T}\sum_{t=1}^{T} \mathbf{e}_t, \qquad \mathbf{a} = \phi\left(W_1\mathbf{h} + \mathbf{b}_1\right), \qquad \mathbf{s} = W_2\mathbf{a} + \mathbf{b}_2, \qquad p_\theta(y = k \mid d) = \mathrm{softmax}(\mathbf{s})_k.$$

Here T is the number of real tokens in document d, $\mathbf{h} \in \mathbb{R}^D$ is the mean-pooled document vector, $W_1 \in \mathbb{R}^{H \times D}$ and $\mathbf{b}_1 \in \mathbb{R}^H$ are the hidden layer's weights and bias, H is the hidden width, ϕ is an elementwise nonlinearity such as ReLU, $\mathbf{a}$ is the hidden activation, $W_2 \in \mathbb{R}^{K \times H}$ and $\mathbf{b}_2 \in \mathbb{R}^K$ produce the class scores $\mathbf{s}$, and $\mathrm{softmax}(\mathbf{s})_k = \exp(s_k)/\sum_\ell \exp(s_\ell)$. The parameters $\theta = (E, W_1, \mathbf{b}_1, W_2, \mathbf{b}_2)$ are trained jointly by minimizing the same cross-entropy as the logistic model, with gradients computed by backpropagation.

Two published designs show how far this simple shape can go. Joulin et al. average word and n-gram embeddings and feed the result to a linear classifier, trading depth for speed [7]. Iyyer et al.'s deep averaging network stacks several nonlinear layers on the average and adds **word**

dropout, which removes whole tokens before averaging during training [6]. Goldberg's primer develops this family of feed-forward text models in detail [5].

The hidden layer matters. A linear model on the mean vector can only add per-word evidence, so it cannot represent "*good* is positive unless *not* is present" as an interaction. A nonlinear hidden layer can combine the presence of "not" with the presence of a positive word. What no pooling-based model can do is recover order: any permutation of the tokens gives the same mean. "The screen is good but the case is bad" and "the case is good but the screen is bad" are identical inputs. The experiment later in this lesson measures exactly this failure.

The neural probabilistic language model

Plain version. A language model predicts the next word from the previous ones. Bengio et al. showed that if each previous word is represented by a learned vector, a network can share evidence between similar histories and assign sensible probabilities to word sequences it never saw.

A **language model** assigns a probability to a sequence by predicting one token at a time. Writing w_t for the token at position t, the chain rule gives

$$P(w_1, \dots, w_T) = \prod_{t=1}^{T} P(w_t \mid w_1, \dots, w_{t-1}).$$

Here $P(w_1, \dots, w_T)$ is the probability of the whole sequence and $P(w_t \mid w_1, \dots, w_{t-1})$ is the conditional probability of the next token given everything before it. Count-based n-gram models approximate each factor using only the previous $n-1$ tokens and estimate it from counts; NLP 05 develops them together with smoothing and perplexity. Their weakness is that the history "a dog" shares nothing with "a cat": if the first never occurred, the counts are silent about it.

Bengio et al. keep the fixed $(n-1)$-token history but replace the count table with a neural network [8]. Each history word is looked up in a shared table C, the vectors are concatenated, and a hidden layer produces a score for every vocabulary word:

$$\mathbf{x} = [C(w_{t-1}); C(w_{t-2}); \dots; C(w_{t-n+1})], \qquad \mathbf{y} = \mathbf{b} + W\mathbf{x} + U \tanh(\mathbf{d} + H\mathbf{x}), \qquad \hat{P}(w_t = i \mid w_{t-n+1}^{t-1}) = \frac{\exp(y_i)}{\sum_{j=1}^{V} \exp(y_j)}.$$

Here $C(w) \in \mathbb{R}^m$ is the learned feature vector of word w (a row of the $V \times m$ table C), the semicolons denote concatenation, so $\mathbf{x} \in \mathbb{R}^{(n-1)m}$; $H \in \mathbb{R}^{h \times (n-1)m}$ and $\mathbf{d} \in \mathbb{R}^h$ are the hidden layer's weights and bias with h hidden units; $U \in \mathbb{R}^{V \times h}$ maps hidden units to vocabulary scores; $W \in \mathbb{R}^{V \times (n-1)m}$ is an optional **direct connection** from the word features to the output (set to zero when not used); $\mathbf{b} \in \mathbb{R}^V$ is the output bias; $\mathbf{y}$ holds the unnormalized log-probabilities; V is vocabulary size; and w_{t-n+1}^{t-1} abbreviates the history. This is the paper's equation (1) in our notation. Training maximizes the average log-likelihood of the training tokens plus a weight-decay penalty [8].

Three ideas in this model recur throughout the rest of the course. Word vectors are learned as a by-product of a prediction task. Concatenation, unlike averaging, keeps the position of each

history word, so “dog bites” and “bites dog” differ. And the softmax over the entire vocabulary is the expensive part: its cost grows with V for every predicted token, which later motivated cheaper objectives such as the negative sampling discussed in NLP 04.

The following example trains a tiny version on an invented corpus in which the history “a dog” never occurs. We use Adam rather than the paper’s stochastic gradient procedure, purely for convenience.

```
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0)
torch.set_num_threads(1)
corpus = """a cat sleeps . a cat eats . the cat sleeps . the cat eats .
the dog sleeps . the dog eats . a bird sings . the bird sings .
a bird eats . the bird eats""".split(".")      # the history "a dog" never occurs
sentences = [["<s>", "<s>"] + s.split() + ["</s>"] for s in corpus]
vocab = sorted({w for s in sentences for w in s})
ids = {w: i for i, w in enumerate(vocab)}
X, Y = [], []
for s in sentences:
    for t in range(2, len(s)):                  # predict w_t from (w_{t-2}, w_{t-1})
        X.append([ids[s[t - 2]], ids[s[t - 1]]])
        Y.append(ids[s[t]])
X, Y = torch.tensor(X), torch.tensor(Y)

class NPLM(nn.Module):
    def __init__(self, V, m=8, h=16, context=2, direct=True):
        super().__init__()
        self.C = nn.Embedding(V, m)                     # shared word features
        self.H = nn.Linear(context * m, h)              # hidden layer (bias = d)
        self.U = nn.Linear(h, V)                        # hidden-to-output (bias = b)
        self.W = nn.Linear(context * m, V, bias=False) if direct else None
    def forward(self, ctx):
        x = self.C(ctx).flatten(1)                      # concatenate the context vectors
        y = self.U(torch.tanh(self.H(x)))
        return y + self.W(x) if self.W is not None else y

model = NPLM(len(vocab))
opt = torch.optim.Adam(model.parameters(), lr=0.02, weight_decay=1e-4)
for step in range(400):
    opt.zero_grad()
    loss = F.cross_entropy(model(X), Y)
    loss.backward()
    opt.step()
print(f"training cross-entropy {loss.item():.3f} nats, {len(Y)} predictions")

history_count = sum(s[i:i + 2] == ["a", "dog"] for s in sentences for i in range(len(s)))
with torch.no_grad():
    p = F.softmax(model(torch.tensor([[ids["a"], ids["dog"]]])), dim=-1)[0]
print("training occurrences of the history 'a dog':", history_count)
for w in ["sleeps", "eats", "sings"]:
```

```
    print(f"P({w} | a dog) = {p[ids[w]].item():.3f}")
```

The run prints:

```
training cross-entropy 0.576 nats, 40 predictions
training occurrences of the history 'a dog': 0
P(sleeps | a dog) = 0.509
P(eats | a dog) = 0.490
P(sings | a dog) = 0.001
```

A trigram count model has no data for the history "a dog" and would have to back off to shorter histories. The neural model instead puts almost all probability on the two verbs that dogs (and cats) take in training, and almost none on "sings," which only birds do. It generalizes because "dog" and "cat" receive related vectors from sharing contexts, and because "a" and "the" do too. That is the central claim of Bengio et al., reproduced here at toy scale; it says nothing about how the model compares with a well-smoothed n-gram model on real text, which is what the paper measured [8].

Convolutional text classifiers

Plain version. A convolutional filter slides along the sentence and fires when it sees a particular short phrase pattern, wherever that pattern occurs. Taking the strongest response of each filter gives a fixed-size summary that remembers *which* patterns occurred, though not *where*.

Collobert et al. built a unified neural network for tagging tasks in which convolution over word vectors, followed by max pooling over positions, summarizes a whole sentence [9]. Kim showed that a single convolutional layer on top of pretrained word vectors is a strong sentence classifier [10]. Convolution itself, with its weight sharing and pooling, is developed for images in ML 03; text uses the one-dimensional version, sliding along positions only.

The mathematics of one filter

Stack the T token embeddings of a sentence into a matrix $X \in \mathbb{R}^{T \times D}$ whose row t is $\mathbf{e}_t^\top$. A filter of width h covers h consecutive rows:

$$c_t = \rho(\langle W, X_{t:t+h-1} \rangle + b), \qquad t = 1, \dots, T-h+1, \qquad \hat{c} = \max_t c_t.$$

Here $X_{t:t+h-1} \in \mathbb{R}^{h \times D}$ is the window of rows t through $t+h-1$, $W \in \mathbb{R}^{h \times D}$ contains the filter's learned weights, $\langle A, B \rangle = \sum_{r,s} A_{rs} B_{rs}$ sums elementwise products, $b \in \mathbb{R}$ is a bias, ρ is a nonlinearity such as ReLU or tanh, c_t is the filter's response at window t, the list $(c_1, \dots, c_{T-h+1})$ is the **feature map**, and $\hat{c}$ is the result of **max-over-time pooling**. Kim writes the same operation with the window flattened to a vector of length hD [10].

A model uses many filters. With F filters for each width in a set such as $\{3,4,5\}$, the pooled values form a vector $\mathbf{z} \in \mathbb{R}^{F\cdot(\text{number of widths})}$, which a linear layer and softmax turn into class probabilities, trained with cross-entropy as before. In PyTorch, `nn.Conv1d(D, F, h)` computes all F feature maps of width h at once; it expects the input as (batch, D, T), so the embedding output must be transposed.

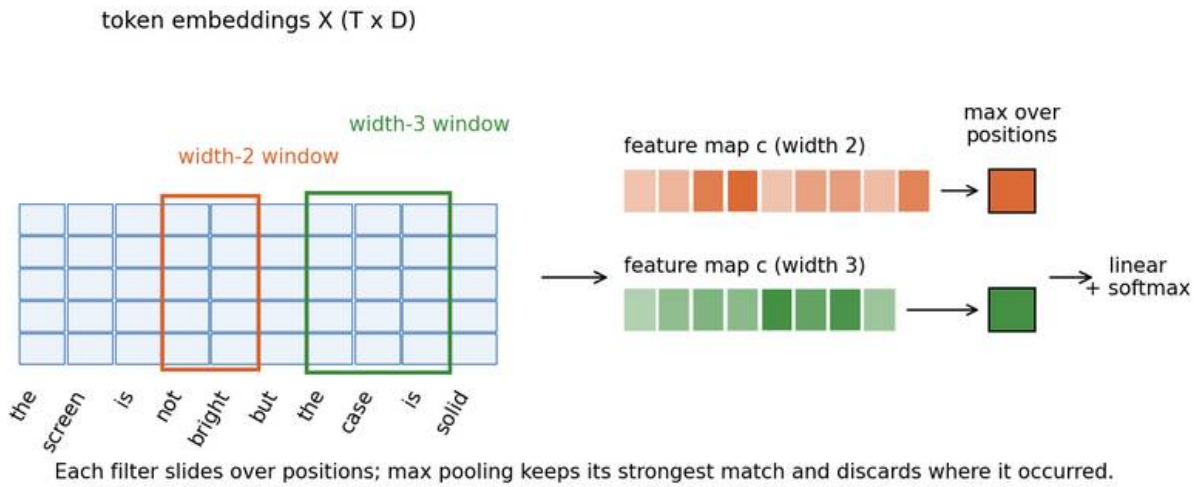


Figure 3 — A 1-D convolutional text classifier. Each filter scores every window of its width; max-over-time pooling keeps the strongest score and discards its position.

What the pieces buy, and what they lose

Weight sharing. The same filter is applied at every position, so a pattern learned from "not good" at the start of one sentence is detected at the end of another. The parameter count depends on h, D and F, not on sentence length.

Soft n-grams. Because the filter takes dot products with embeddings, a filter that responds to "not good" also responds, more weakly, to "not great" if the two adjectives have similar vectors. This is the advantage over sparse bigram features.

Max pooling. Keeping only the maximum handles variable sentence lengths and records whether a pattern occurred anywhere. It discards where the pattern occurred and how many times. A one-layer filter also cannot connect tokens farther apart than its width: with widths up to 4, nothing in the model sees "but" and an adjective four positions later inside the same window. The experiment below shows exactly this.

Kim's paper compares four ways to treat the embedding table: random initialization trained with the model (CNN-rand), pretrained vectors kept fixed (CNN-static), pretrained vectors fine-tuned (CNN-non-static), and two channels, one fixed and one fine-tuned, with each filter applied to both (CNN-multichannel) [10]. His reported settings use filter widths 3, 4 and 5 with 100 feature maps each, dropout rate 0.5 on the pooled layer, an ℓ_2-norm constraint of 3 on the output weight vectors, and mini-batches of 50 [10]. Zhang and Wallace later ran a sensitivity analysis of this one-layer

design and concluded that the filter region size can matter a lot and should be tuned, that 1-max pooling uniformly outperformed the other pooling strategies they tried, that ReLU and tanh were the best activation candidates, and that variance across runs must be reported when comparing configurations [11].

Recurrent encoders take a different route to order: they update a hidden state while reading the sequence and can, in principle, connect distant tokens. Hagiwara's sentence-classification discussion develops recurrent alternatives alongside practical training [1]. NLP 05 develops them for language and ML 03a gives the shared mathematics. Contextual encoders based on attention let every token's representation depend on the whole input; NLP 07 and NLP 10 develop them.

Choose according to evidence. A sparse classifier may be enough when labels have clear vocabulary clues. A convolutional or contextual encoder may help with indirect phrasing, negation or domain-specific meanings. Its larger capacity also makes split design and annotation quality more consequential. Complexity should answer a measured failure, such as errors on paraphrases, rather than stand in for evaluation.

Padding, masking and batching

Plain version. Sentences have different lengths, but a batch is a rectangle. We fill short sentences with a padding token and then make sure that padding never influences the result.

Training processes several sequences at once as one tensor of shape (batch, $T_{\max}$), where $T_{\max}$ is the longest length in the batch. Shorter sequences are filled with a reserved **padding ID**, usually 0. A boolean **mask** m_{bt} records which positions are real:

$$m_{bt} = \mathbf{1}[t \le T_b], \qquad \mathbf{h}_b = \frac{\sum_{t=1}^{T_{\max}} m_{bt}\, \mathbf{e}_{bt}}{\max\left(1, \sum_{t=1}^{T_{\max}} m_{bt}\right)}.$$

Here b indexes sequences in the batch, t indexes positions, T_b is the real length of sequence b, $\mathbf{1}[\cdot]$ is one when its condition holds and zero otherwise, $\mathbf{e}_{bt}$ is the embedding at position t of sequence b, and $\mathbf{h}_b$ is the **masked mean**. The $\max(1, \cdot)$ guards against an empty sequence. Each pooling operation needs its own masking rule:

- **Mean pooling** must divide by the real length, not by $T_{\max}$. A zero padding vector does not change the sum, but it does change an unmasked denominator.
- **Max pooling** must exclude padded positions or windows. The general rule is to fill them with $-\infty$ before the maximum. After a ReLU, all responses are nonnegative, so filling with 0 is equivalent and has one advantage: a sentence shorter than the filter width, which has no valid window at all, gets 0 instead of $-\infty$, which would turn into NaN in the next layer.
- **Convolution windows** that overlap padding are not real windows. With right padding, a window of width h starting at position t is valid exactly when position $t + h - 1$ is real.
- **Losses** over token positions, used later for tagging and language modeling, must ignore padded positions; PyTorch's cross-entropy accepts an `ignore_index` for this.

PyTorch's `nn.Embedding(..., padding_idx=0)` keeps the padding row at zero and excludes it

from gradient updates. That helps, but it does not make a mean or a max correct by itself. The test is an invariance: adding extra padding must not change any output.

```
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0)
emb = nn.Embedding(6, 4, padding_idx=0)            # ID 0 is reserved for padding
seqs = [torch.tensor([3, 5, 2]), torch.tensor([4, 1])]
x = nn.utils.rnn.pad_sequence(seqs, batch_first=True, padding_value=0)
print(x.tolist())                                   # [[3, 5, 2], [4, 1, 0]]

# A lookup is a one-hot row times the table.
one_hot = F.one_hot(x, num_classes=6).float()
assert torch.allclose(one_hot @ emb.weight, emb(x))

def masked_mean(ids):
    mask = (ids != 0).unsqueeze(-1).float()
    return (emb(ids) * mask).sum(1) / mask.sum(1).clamp(min=1)

longer = F.pad(x, (0, 3), value=0)                  # three more padding columns
print("masked mean unchanged:", torch.allclose(masked_mean(x), masked_mean(longer)))
print("plain mean unchanged: ", torch.allclose(emb(x).mean(1), emb(longer).mean(1)))
print("padding row:", emb.weight[0].tolist())
```

The run prints:

```
[[3, 5, 2], [4, 1, 0]]
masked mean unchanged: True
plain mean unchanged:  False
padding row: [0.0, 0.0, 0.0, 0.0]
```

The padding row is zero, yet the plain mean still changes, because its denominator counts padding. Add this kind of invariance check to any new encoder.

Batching has practical consequences beyond correctness. Padding wastes computation, so training code often groups sequences of similar length into the same batch (**bucketing**). Very long inputs are truncated to a maximum length; record that limit, because text beyond it is invisible to the model, and decide whether to keep the beginning, the end or both. Shuffle batches every epoch so the model does not see the same order of examples, and evaluate with the model in evaluation mode so that dropout is switched off.

Regularization for small text classifiers

Plain version. A neural text model has enough parameters to memorize a small training set, including its labeling mistakes. Regularization makes memorization harder, and a validation set

tells us when to stop.

The tools are the general ones from ML 02, applied at specific places in a text model:

- **Dropout** sets each unit of a layer to zero with probability p during training and rescales the rest, so that no unit can rely on a particular partner [12]. In text CNNs it is usually applied to the pooled feature vector, as in Kim's model [10]. PyTorch's `nn.Dropout(p)` uses the convention that p is the probability of *dropping* a unit.
- **Word dropout** removes entire tokens from the input during training, which encourages the model not to depend on any single word [6].
- **Weight decay** adds $\frac{\lambda}{2}\|\theta\|_2^2$ to the loss, the same penalty as in the logistic objective above. The embedding table is often the largest parameter block, so whether it is decayed or frozen matters.
- **Max-norm constraints** rescale a weight vector whenever its Euclidean norm exceeds a limit, as in Kim's output layer [10].
- **Early stopping** keeps the parameters from the epoch with the best validation loss. It is a regularizer too: it limits how far optimization can move toward memorizing the training set.
- **Freezing pretrained embeddings** removes the largest block of trainable parameters entirely, at the cost of not adapting word meanings to the task.

Every one of these choices is a hyperparameter selected on validation data. Change one at a time and report variation across random seeds before concluding that a choice helps; Zhang and Wallace emphasize exactly this for text CNNs [11].

A complete PyTorch experiment

Plain version. We now compare four classifiers on one invented review task in which order genuinely matters, read their training curves and inspect their mistakes by category.

Each synthetic review is either one clause, "the *aspect* is (*not*) *adjective*," or two clauses joined by "but." The label is positive when the deciding clause is positive: a negation flips the adjective, and in a "but" review the second clause decides. Training has 300 reviews with 10% of labels deliberately flipped to imitate annotation noise; validation (300) and test (1,000) labels are clean. Every test review is tagged as `plain`, `negated` or `contrast` so that errors can be counted by category.

```
import copy, random
import numpy as np
import torch
from torch import nn
from torch.nn import functional as F
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression

torch.set_num_threads(1)
ASPECTS = ["battery", "screen", "camera", "speaker", "case", "charger"]
POS, NEG = ["good", "great", "solid", "bright"], ["bad", "weak", "noisy", "dull"]
```

```
def clause(rng):
    aspect, positive = rng.choice(ASPECTS), rng.random() < 0.5
    negated = rng.random() < 0.4
    words = ["the", aspect, "is"] + ["not"] * negated + [rng.choice(POS if positive else
     ↪  NEG)]
    return words, int(positive != negated), negated

def example(rng):                       # label of "A but B" is the label of B
    words, label, negated = clause(rng)
    if rng.random() < 0.5:
        second, label, _ = clause(rng)
        return " ".join(words + ["but"] + second), label, "contrast"
    return " ".join(words), label, "negated" if negated else "plain"

def make(n, seed):
    rng = random.Random(seed)
    return [example(rng) for _ in range(n)]

train, val, test = make(300, 1), make(300, 2), make(1000, 3)
flip = random.Random(4)                 # 10% training-label noise; val/test stay clean
train = [(t, 1 - y if flip.random() < 0.1 else y, c) for t, y, c in train]
y_test, kind = np.array([r[1] for r in test]), np.array([r[2] for r in test])

def report(name, pred):
    parts = {k: round(float((pred[kind == k] == y_test[kind == k]).mean()), 3)
             for k in ["plain", "negated", "contrast"]}
    print(f"{name:12s} test acc {np.mean(pred == y_test):.3f}", parts)

for ngrams in [(1, 1), (1, 2)]:
    vec = TfidfVectorizer(ngram_range=ngrams, token_pattern=r"\S+")
    clf = LogisticRegression(C=10.0, max_iter=1000)
    clf.fit(vec.fit_transform([r[0] for r in train]), [r[1] for r in train])
    report(f"tfidf {ngrams}", clf.predict(vec.transform([r[0] for r in test])))

vocab = ["<pad>", "<unk>"] + sorted({w for r in train for w in r[0].split()})
ids = {w: i for i, w in enumerate(vocab)}

def encode(rows):                       # right-pad every sequence with ID 0
    seqs = [torch.tensor([ids.get(w, 1) for w in r[0].split()]) for r in rows]
    return (nn.utils.rnn.pad_sequence(seqs, batch_first=True, padding_value=0),
            torch.tensor([r[1] for r in rows]))

class MeanMLP(nn.Module):               # embedding -> masked mean -> MLP
    def __init__(self, V, D=16, H=32, p=0.3):
        super().__init__()
        self.emb = nn.Embedding(V, D, padding_idx=0)
        self.mlp = nn.Sequential(nn.Linear(D, H), nn.ReLU(), nn.Dropout(p), nn.Linear(H, 2))
    def forward(self, x):
        m = (x != 0).unsqueeze(-1).float()
        return self.mlp((self.emb(x) * m).sum(1) / m.sum(1).clamp(min=1))

class TextCNN(nn.Module):               # embedding -> 1-D conv -> masked max -> linear
    def __init__(self, V, D=16, widths=(2, 3, 4), maps=16, p=0.3):
        super().__init__()
```

```
        self.emb = nn.Embedding(V, D, padding_idx=0)
        self.convs = nn.ModuleList(nn.Conv1d(D, maps, h) for h in widths)
        self.drop, self.out = nn.Dropout(p), nn.Linear(maps * len(widths), 2)
    def forward(self, x):
        e, real = self.emb(x).transpose(1, 2), x != 0         # e: (batch, D, T)
        pooled = []
        for conv in self.convs:
            c = torch.relu(conv(e))                           # (batch, maps, T-h+1)
            valid = real[:, conv.kernel_size[0] - 1:].unsqueeze(1)  # window ends on a real
             ↪  token
            # ReLU outputs are >= 0, so filling invalid windows with 0 never wins the max,
            # and a text shorter than the filter gets 0 instead of -inf (which would give
             ↪  NaN).
            pooled.append(c.masked_fill(~valid, 0.0).max(dim=2).values)
        return self.out(self.drop(torch.cat(pooled, dim=1)))

def fit(model, weight_decay, epochs=80, batch=32):
    (xt, yt), (xv, yv) = encode(train), encode(val)
    opt = torch.optim.Adam(model.parameters(), lr=0.01, weight_decay=weight_decay)
    best, history = (float("inf"), None, 0), []
    for epoch in range(1, epochs + 1):
        model.train()
        for b in torch.randperm(len(yt)).split(batch):
            opt.zero_grad()
            F.cross_entropy(model(xt[b]), yt[b]).backward()
            opt.step()
        model.eval()
        with torch.no_grad():
            history.append((F.cross_entropy(model(xt), yt).item(),
                            F.cross_entropy(model(xv), yv).item()))
        if history[-1][1] < best[0]:                          # early stopping on val loss
            best = (history[-1][1], copy.deepcopy(model.state_dict()), epoch)
    model.load_state_dict(best[1])
    return history, best[2]

x_test, _ = encode(test)
for name, build, wd in [("mean+MLP", lambda: MeanMLP(len(vocab)), 1e-4),
                        ("CNN", lambda: TextCNN(len(vocab)), 1e-4),
                        ("CNN no-reg", lambda: TextCNN(len(vocab), p=0.0), 0.0)]:
    torch.manual_seed(0)
    model = build()
    history, best_epoch = fit(model, wd)
    with torch.no_grad():
        pred = model(x_test).argmax(1).numpy()
    print(f"{name:12s} last train/val loss {history[-1][0]:.3f}/{history[-1][1]:.3f}, "
          f"best epoch {best_epoch}")
    report(name, pred)
    if name == "CNN":
        cnn_pred = pred

wrong = np.flatnonzero(cnn_pred != y_test)
print("CNN errors by kind:", {k: int((kind[wrong] == k).sum()) for k in ["plain", "negated",
 ↪  "contrast"]})
for i in wrong[:4]:
```

```
    print(" ", y_test[i], "<-", test[i][0])
```

The whole script runs in a few seconds on one CPU thread and prints:

```
tfidf (1, 1) test acc 0.546 {'plain': 0.651, 'negated': 0.348, 'contrast': 0.555}
tfidf (1, 2) test acc 0.880 {'plain': 1.0, 'negated': 1.0, 'contrast': 0.77}
mean+MLP     last train/val loss 0.269/0.464, best epoch 36
mean+MLP     test acc 0.797 {'plain': 0.973, 'negated': 0.945, 'contrast': 0.645}
CNN          last train/val loss 0.144/0.431, best epoch 30
CNN          test acc 0.898 {'plain': 0.983, 'negated': 0.945, 'contrast': 0.833}
CNN no-reg   last train/val loss 0.092/0.444, best epoch 10
CNN no-reg   test acc 0.910 {'plain': 1.0, 'negated': 0.928, 'contrast': 0.852}
CNN errors by kind: {'plain': 5, 'negated': 10, 'contrast': 87}
  0 <- the camera is not dull but the battery is weak
  0 <- the camera is noisy but the screen is bad
  1 <- the charger is solid but the battery is solid
  1 <- the charger is solid but the charger is bright
```

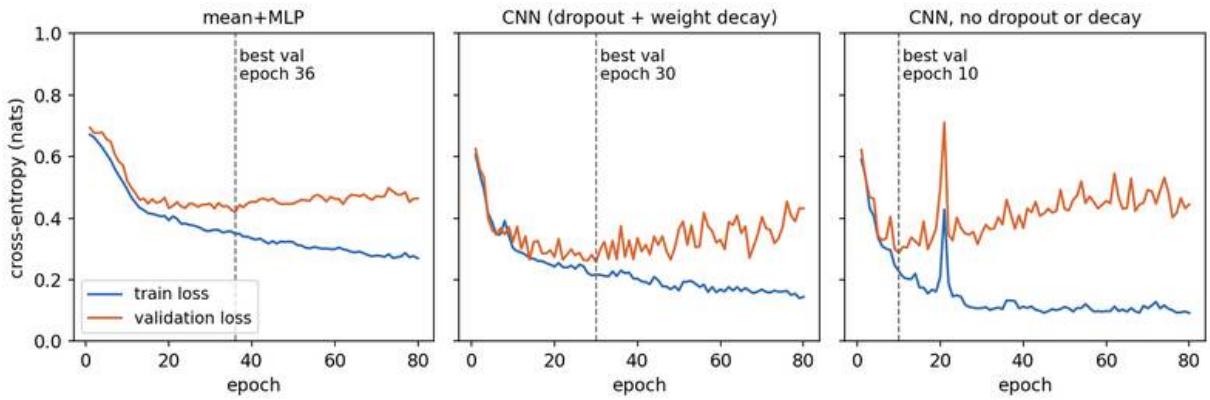


Figure 4 — Training (blue) and validation (orange) loss for the three neural models in the experiment, seed 0. The dashed line marks the epoch kept by early stopping.

Read the results by category

Unigram TF–IDF fails on negation by construction. In this data, "not" appears equally often in positive and negative reviews, and each adjective is positive or negative depending on whether "not" is present. A linear model over word presence cannot represent that interaction, so negated reviews score 0.348, worse than chance.

Bigrams fix negation, not contrast. The pair "not good" is a single feature, so the bigram model is perfect on plain and negated reviews. On contrast reviews it reaches 0.77: it sees both clauses' bigrams and has no reliable way to know which clause comes after "but."

The pooled MLP solves the interaction but not order. Its hidden layer can combine "not" with an adjective, so negated reviews reach 0.945 even though the input is an unordered mean. On contrast reviews it scores 0.645 because mean pooling makes "A but B" and "B but A" identical.

The CNN does best on contrast but is limited by its width. With filters up to width 4 it reaches 0.833 on contrast reviews. All but 15 of its 102 errors are contrast reviews, and the listed errors show why: in "the charger is solid but the battery is solid," the deciding adjective is four positions after "but," so no single window contains both. The filters must rely on indirect cues. Exercise 3 asks you to add a width-5 filter and test that explanation.

Read the curves

All three panels show the same pattern: training loss keeps falling while validation loss stops improving and then rises. Training labels contain 10% flipped labels, so a training loss far below that noise level means the model is memorizing mistakes. The unregularized CNN does this fastest: its validation loss bottoms out at epoch 10 and is noticeably worse by epoch 80. Dropout and weight decay slow that drift, and the regularized CNN's best validation loss comes later.

On test accuracy, however, the unregularized CNN (0.910) and the regularized CNN (0.898) are close once early stopping has chosen each model's epoch. With one seed and a difference of about one percentage point, this run cannot rank them. The defensible conclusion is narrower: early stopping on a clean validation set is essential here, and the regularized model is less sensitive to *when* it is stopped. A real comparison would repeat the experiment over several seeds and report the spread [11].

Error analysis as a habit

The category counts were possible only because each test example carried a tag describing *why* it might be hard. Build such tags into evaluation data from the start: negation, contrast, rare words, length, source. Then read individual errors and ask at which stage the needed evidence was lost. If mean pooling makes two inputs identical, more labels cannot help that model; if the CNN's windows are too short, a wider filter or a different encoder is the fix; if an annotation is wrong, the fix is in the data.

Evaluate errors and probabilities separately

Plain version. A classifier can choose the right label while giving unreliable probabilities. Evaluate the decision and the uncertainty estimate as different properties.

For one class treated as positive:

$$P = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}, \qquad R = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}}, \qquad F_1 = \frac{2PR}{P + R}.$$

Here P is precision, R is recall, TP counts true positives, FP counts false positives, FN counts false negatives and F_1 is their harmonic mean. A zero denominator requires an explicitly chosen convention. Macro F1 averages per-class F1 values equally; support-weighted averaging gives more influence to common classes. The metric documentation specifies these choices [13].

Macro averaging makes failures on a rare category more visible, but it does not encode the actual cost of those failures. Missing an urgent ticket and misrouting a routine receipt can have different consequences. Write a cost policy separately rather than treating a convenient score as a complete utility measure.

Calibration asks whether predicted confidence agrees with observed correctness over comparable examples. A model can preserve its argmax decisions while changing confidence. Guo et al. study this distinction and post-training temperature scaling [14]. Calibration parameters must be fit on held-out development data, with enough samples to estimate probability behavior. Neural classifiers trained to near-zero training loss are a common source of overconfident probabilities, so check calibration whenever the model changes.

For selective prediction:

$$\text{coverage} = \frac{1}{n}\sum_{i=1}^{n} a_i, \qquad \text{risk}_{\text{accepted}} = \frac{\sum_{i=1}^{n} a_i \mathbf{1}[\hat{y}_i \neq y_i]}{\sum_{i=1}^{n} a_i}.$$

Here n is evaluated-example count, $a_i \in \{0, 1\}$ indicates acceptance of example i, $\hat{y}_i$ is its prediction, y_i is its reference label, and $\mathbf{1}[\cdot]$ is one when its condition holds and zero otherwise. Accepted risk is undefined when no example is accepted. A risk-versus-coverage curve describes the trade-off over thresholds; choose the deployed threshold using validation rather than final test labels.

Imbalance, error analysis and deployment changes

Plain version. An overall score can improve while an important user group gets worse. Inspect the types of mistakes, the rare classes and the circumstances in which they occur.

Create an error table with text, gold label, predicted label, probability, language, length, source group and an analyst's explanation. Useful error categories include annotation ambiguity, missing context, negation, unseen vocabulary, multi-topic input, quotation and genuinely out-of-scope text. These categories suggest different repairs. More training examples can help missing vocabulary; they cannot supply a conversation turn that is never available at prediction time.

Class weighting changes how strongly each training example contributes:

$$\mathcal{L}_{\text{weighted}} = -\frac{1}{n}\sum_{i=1}^{n} \alpha_{y_i} \log p_\theta(y_i \mid d_i).$$

Here $\mathcal{L}_{\text{weighted}}$ is weighted cross-entropy, n is training-example count, i indexes examples, d_i is the document, y_i is its annotated class, $\alpha_{y_i} > 0$ is the chosen weight for that class and p_θ is the classifier distribution. The weighting policy changes the training objective; it does not manufacture missing linguistic variety.

Weighting, oversampling and threshold changes answer different problems. Weighting changes parameter learning. Oversampling changes the presentation frequency of examples, sometimes repeatedly exposing the same few minority examples. Thresholding changes the action policy after scores are computed. Hagiwara's development discussion covers imbalance and training choices [1]. Evaluate any intervention on a realistic prevalence distribution rather than on an artificially balanced final test.

Deployment shifts can affect vocabulary, label prevalence and the relationship between text and labels. A new feature release may produce a new technical-error vocabulary; a holiday may increase delivery messages; a changed routing policy may redefine labels. Monitor these separately. A rising fraction of empty feature vectors, or of unknown-word IDs in a neural model, suggests representation coverage; a routing-policy change requires updating the annotation contract.

Keep model and preprocessing artifacts together. The vocabulary order, IDF weights or embedding table, classifier class order, maximum length, probability calibration and decision threshold are a single inference package. Reloading weights with another vocabulary silently changes meanings. Store a versioned label schema and a few deterministic contract examples with the package so that serving can check its assumptions.

Modern view

Survey perspective. Goldberg's primer organizes neural NLP around feed-forward, convolutional, recurrent and recursive networks, input encoding and computation graphs [5]; it remains a compact map of the models in this lesson. Minaee et al. review more than 150 deep-learning text classifiers and more than 40 datasets, from feed-forward and convolutional models through recurrent, attention-based and pretrained models, and compare reported benchmark results [15]. Read such tables as evidence about specific datasets and protocols, not as a ranking that transfers to an arbitrary label schema.

Zhao et al. organize their LLM survey around pretraining, adaptation, use and evaluation [16]. For classification, this yields a useful comparison among fixed representations with a learned head, task-adapted encoders and instruction-based prediction. Chang et al. organize evaluation by what, where and how to measure [17]. Their perspective warns against treating one familiar benchmark as evidence that an arbitrary label schema works.

Robustness perspective. Sperduti and Moreo's 2025 misspelling survey reviews sources of spelling variation, correction approaches and the effect on downstream tasks and modern language models [18]. Its practical implication here is to include realistic spelling variation in evaluation, then distinguish normalization failures from classifier failures. Character features, subword embeddings and contextual representations offer different responses; none should be assumed robust without a task-specific comparison.

Practice in 2023–2026. Pretrained Transformer encoders fine-tuned with a classification head, covered in NLP 11, are the usual starting point when labeled data and compute allow. Instruction-following language models broaden the available ways to specify classification tasks [16]. A prompted model can interpret a textual label description without a separately trained classifier

head. That shifts part of the configuration into the prompt, label wording and output parser. Treat those artifacts as fitted design choices: fix them before testing, constrain outputs to the allowed schema and measure abstention and malformed responses.

The models in this lesson keep a role. TF–IDF with logistic regression and small convolutional or averaging networks are cheap to train, fast to serve and easy to inspect. They are the baselines that tell you whether a larger model's gain is real, and they are often adequate when labels have clear lexical cues.

An application can also use a larger model to suggest labels for human review, then train a smaller classifier. This changes the annotation process. Generated labels can contain systematic mistakes or reproduce the teacher's preferences. Audit a human-checked subset and keep synthetic supervision distinct from independent evaluation labels. The possibility of producing more labels is not proof of better supervision.

Open problems. Label interpretation can drift between annotators and model versions. Low-resource languages may have too little independent evaluation data. Multi-topic messages resist a single-label schema. Calibration can deteriorate after a domain change. Embedding similarity can hide contradiction. These questions need explicit challenge sets and continuing evaluation, not an architecture ranking with all context removed.

Key takeaways

- The annotation contract determines the prediction problem and the available actions; fit vocabulary, IDF weights and any learned representation on training text only.
- A sparse linear baseline is strong, cheap and diagnostic; bigrams capture local negation but treat every phrase as unrelated to every other.
- An embedding layer is a learned lookup table; mean pooling plus a nonlinear MLP can learn word interactions but cannot see order.
- The neural probabilistic language model concatenates learned word vectors and generalizes to unseen histories by sharing evidence between similar words.
- A 1-D convolutional classifier detects soft n-gram patterns anywhere in the text; max pooling discards position, and filter width limits the distance it can connect.
- Padding requires explicit masks for means, maxima, windows and losses; adding padding must not change any output.
- Regularization and early stopping matter most when labels are noisy; compare configurations over several seeds.
- Evaluate class decisions, probability quality, coverage and per-category errors separately.

Exercises

1. **A label contract.** Define a multi-label version of the support task. Write rules for "delivery charge," a quoted complaint and an empty input. Explain which output activation and loss change.

Hint

Independent label probabilities can allow two routes. Include an explicit rule for when no label is justified rather than interpreting every low score as a negative annotation.

2. **Train-only features.** Introduce a word that occurs only in the final test set. Inspect the fitted TF–IDF vocabulary and transformed test vector, and the ID it receives in the neural experiment's vocabulary. Compare the result with incorrectly fitting features on all text.

Hint

The correct vocabulary excludes the test-only word; in the neural model it maps to `<unk>`. Examine document-frequency changes as well as the vocabulary size; leakage can alter weights even when all words already exist.

3. **Test the width explanation.** In the PyTorch experiment, change the CNN's widths to `(2, 3, 4, 5)` and rerun. Does contrast accuracy change? Then run the original and the modified model over five seeds and report mean and range before drawing a conclusion.

Hint

Count the positions in "but the battery is solid." A width-5 window can contain "but" and the deciding adjective together. Keep everything except the widths fixed, and remember that a single-seed difference can be noise.

4. **Masking bugs on purpose.** Remove the mask from `MeanMLP` (use a plain `.mean(1)`) and from `TextCNN` (pool over all windows). Pad the test batch with five extra columns and compare predictions with the masked versions. Which model changes, and why?

Hint

The padding embedding is zero, so a plain mean shrinks toward zero as padding grows. For the CNN, a window that overlaps padding still has a nonzero bias and can win the max.

5. **A neural language model by hand.** For the NPLM with V words, $n-1$ history words, m features per word and h hidden units, count the parameters in C, H, $\mathbf{d}$, U, $\mathbf{b}$ and W. Which term dominates when V is large, and how does the count change if you remove the direct connections?

Hint

Write the shape of each matrix from the equation. The output layer and the embedding table both scale with V; the hidden layer does not.

6. **Selective decisions.** Run the support-message experiment with several validation cost policies. Plot coverage and accepted error on validation, select one threshold and then evaluate it once on test.

Hint

If no examples are accepted, the accepted-error denominator is zero. Report an undefined value, not perfect accuracy.

References

1. M. Hagiwara, *Real-World Natural Language Processing: Practical Applications with Deep Learning*, Manning, 2021; Chapters 2–4, 7 and 10–11. publisher
2. S. McRoy, *Principles of Natural Language Processing*, open textbook, 2021; Chapter 6, "Benchmark Tasks for Language Modelling." chapter
3. Scikit-learn developers, "Text feature extraction," scikit-learn documentation, current online edition. documentation
4. Scikit-learn developers, "Logistic regression," scikit-learn documentation, current online edition. documentation
5. Y. Goldberg, "A primer on neural network models for natural language processing," *Journal of Artificial Intelligence Research* 57, 2016. paper
6. M. Iyyer, V. Manjunatha, J. Boyd-Graber and H. Daumé III, "Deep unordered composition rivals syntactic methods for text classification," ACL-IJCNLP, 2015. paper
7. A. Joulin, E. Grave, P. Bojanowski and T. Mikolov, "Bag of tricks for efficient text classification," EACL, 2017. paper
8. Y. Bengio, R. Ducharme, P. Vincent and C. Jauvin, "A neural probabilistic language model," *Journal of Machine Learning Research* 3, 2003. paper
9. R. Collobert, J. Weston, L. Bottou, M. Karlen, K. Kavukcuoglu and P. Kuksa, "Natural language processing (almost) from scratch," *Journal of Machine Learning Research* 12, 2011. paper
10. Y. Kim, "Convolutional neural networks for sentence classification," EMNLP, 2014. paper
11. Y. Zhang and B. Wallace, "A sensitivity analysis of (and practitioners' guide to) convolutional neural networks for sentence classification," IJCNLP, 2017. paper
12. N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov, "Dropout: a simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research* 15, 2014. paper
13. Scikit-learn developers, "Classification metrics," scikit-learn documentation, current online edition. documentation
14. C. Guo, G. Pleiss, Y. Sun and K. Q. Weinberger, "On calibration of modern neural networks," ICML, 2017. paper
15. S. Minaee, N. Kalchbrenner, E. Cambria, N. Nikzad, M. Chenaghlu and J. Gao, "Deep learning-based text classification: a comprehensive review," *ACM Computing Surveys* 54(3), 2021. survey
16. W. X. Zhao et al., "A survey of large language models," *Frontiers of Computer Science*, 2026; online manuscript updated March 2026. survey
17. Y. Chang et al., "A survey on evaluation of large language models," *ACM Transactions on Intelligent Systems and Technology*, 2024. survey
18. G. Sperduti and A. Moreo, "Misspellings in natural language processing: a survey," arXiv:2501.16836, 2025. survey

NLP 04 • Lexical Semantics and Word Embeddings

From word senses, lexical relations and WordNet to distributional vectors: PPMI, Word2Vec, GloVe and fastText, how to evaluate them, the biases they absorb, sentence embeddings and the step to contextual representations.

Online edition: https://blog.shyandram.dev/en/learn/nlp/nlp-04-lexical-semantics-word-embeddings/

What you'll learn

- Distinguish word forms, lemmas and senses, and name the main lexical relations: synonymy, antonymy, hypernymy and meronymy, as organized in WordNet.
- Explain the distributional hypothesis, what shared contexts can reveal about words and what they cannot establish.
- Construct a word–context matrix, turn positive PMI into compact vectors, and derive CBOW, Skip-gram, negative sampling, GloVe and fastText objectives.
- Evaluate embeddings with similarity and analogy tests while recognizing their caveats, and measure an association bias.
- Pool word vectors into sentence representations, train sentence embeddings contrastively, and explain what changes with contextual embeddings.

The big picture

A word ID identifies a vocabulary entry. Its numerical value says nothing about what the word does in language. An **embedding** assigns that entry a learned vector, allowing words to share useful statistical patterns. The central question is not simply how to make a vector smaller. It is which evidence and objective should determine its coordinates.

> **Explain like I'm five**
>
> Imagine learning about unfamiliar tools by watching where people use them. Two tools that repeatedly appear beside wood, nails and a workbench may serve related jobs. You can draw a map from those observations without anyone naming the tools. But appearing at the same workbench does not make them interchangeable: a hammer and pliers still do different things. Word vectors learn a similar map from words' surroundings.

Linguists describe word meaning with **lexical semantics**: senses, and relations such as synonymy and hypernymy, recorded by hand in resources like WordNet. Distributional methods learn vectors from text instead. This lesson connects the two views. Hagiwara develops word and document embeddings as reusable features for practical applications [1]. McRoy distinguishes lexical

similarity from language tasks such as sentiment and entailment [2]. Here we develop the representation itself, and we keep asking which lexical relation a vector geometry actually captures.

NLP 03 learned word vectors as a by-product of a task: an embedding layer inside a classifier, and the table C inside the neural probabilistic language model of Bengio et al. [3]. Word2Vec makes the vectors themselves the goal and uses nearby words, not a following word, as the prediction target. These are different prediction contracts:

Question	Neural probabilistic language model	Static Word2Vec embedding
What is observed?	An ordered history of $n-1$ tokens and the token that follows	A center word and selected neighbors on both sides
What is predicted?	A normalized distribution over the next token	A center from neighbors, or individual neighbors from a center
Is left-to-right generation the purpose?	Yes; it is a language model	Basic window training is not an ordered generator
What is reused afterward?	The whole network; word vectors are a by-product	A learned vector table

Continuous in CBOW means that its internal representation has real-valued coordinates. It does not mean that the text is continuous, that words occur without boundaries, or that every word sense receives its own vector. By the end, we will connect the count-based and prediction-based constructions and identify exactly which information each retains. Count-based language models and their evaluation come later, in NLP 05.

Lexical semantics: senses and relations

Plain version. A written word can carry several meanings, and meanings relate to each other in structured ways: some are near-equivalents, some are opposites, some are kinds of others. Embeddings will later blur several of these relations together, so name them first.

Word forms, lemmas and senses

A **word form** is what appears in text (`banks`, `banked`); a **lemma** is the dictionary entry that groups inflected forms (`bank`). A **sense** is one meaning of a lemma. `bank` has at least two unrelated senses, the slope beside a river and the financial institution; this is **homonymy**. `window` has related senses, an opening in a wall and a region of a screen; this is **polysemy**. NLP 02 discusses how context and composition select a sense. For representation, the key point is that a vocabulary entry is a form or lemma, not a sense: a table with one row per entry must merge every sense of that entry.

Relations between senses

Lexical relations hold between senses, not between spelling strings:

- **Synonymy**: two senses that can substitute for each other in many contexts with little change in meaning, such as `slope` and `incline`. Exact synonymy is rare; register, collocation and connotation usually differ.
- **Antonymy**: senses that are opposite along one scale or dimension, such as `hot` and `cold`. Antonyms are otherwise very similar: they describe the same property and occur in the same kinds of sentences.
- **Hyponymy and hypernymy**: an is-a relation. `poodle` is a hyponym of `dog`, and `dog` is a hypernym of `poodle`. The relation is asymmetric and usually transitive, forming a hierarchy.
- **Meronymy**: a part–whole relation, such as `wheel` and `car`.

Two further notions matter for evaluation. **Similarity** asks whether two concepts share properties (`cup` and `mug`); **relatedness** or association asks whether they tend to occur together (`cup` and `coffee`). A related pair need not be similar at all.

WordNet: a hand-built lexical database

WordNet organizes English nouns, verbs, adjectives and adverbs into **synsets**, sets of synonymous lemmas that express one concept, each with a short gloss, connected by relations such as hypernymy, meronymy and antonymy [4]. Looking up a lemma returns its synsets, one per sense. Walking hypernym links from a synset reaches increasingly general concepts, which lets a program measure how far apart two senses are in the hierarchy.

The following example builds a miniature database in the same style, by hand, to show the mechanics without downloading WordNet itself. Its entries are invented for this lesson.

```
# A hand-built miniature lexical database in the style of WordNet (not WordNet data).
synsets = {
    "bank.n.slope":   {"lemmas": ["bank"], "gloss": "sloping land beside a river",
     ↪  "hypernym": "slope.n"},
    "bank.n.finance": {"lemmas": ["bank"], "gloss": "an institution that holds deposits",
     ↪  "hypernym": "institution.n"},
    "slope.n":        {"lemmas": ["slope", "incline"], "gloss": "a surface at an angle",
     ↪  "hypernym": "landform.n"},
    "landform.n":     {"lemmas": ["landform"], "gloss": "a natural feature of land",
     ↪  "hypernym": "entity.n"},
    "institution.n":  {"lemmas": ["institution"], "gloss": "an organization with a public
     ↪  role", "hypernym": "organization.n"},
    "school.n":       {"lemmas": ["school"], "gloss": "an institution for teaching",
     ↪  "hypernym": "institution.n"},
    "organization.n": {"lemmas": ["organization"], "gloss": "a group with a purpose",
     ↪  "hypernym": "entity.n"},
    "entity.n":       {"lemmas": ["entity"], "gloss": "anything that exists", "hypernym":
     ↪  None},
}

def senses(word):
    return [s for s, v in synsets.items() if word in v["lemmas"]]

def path_to_root(s):
```

```
    path = [s]
    while synsets[path[-1]]["hypernym"]:
        path.append(synsets[path[-1]]["hypernym"])
    return path

def path_similarity(a, b):            # 1 / (1 + edges on the shortest is-a path)
    pa, pb = path_to_root(a), path_to_root(b)
    common = next(s for s in pa if s in pb)
    return 1 / (1 + pa.index(common) + pb.index(common))

print("senses of bank:", senses("bank"))
print("synonyms in slope.n:", synsets["slope.n"]["lemmas"])
print("hypernym chain:", " -> ".join(path_to_root("bank.n.finance")))
for other in ["school.n", "slope.n"]:
    print(f"path_similarity(bank.n.finance, {other}) = {path_similarity('bank.n.finance',
     ↪  other):.3f}")
```

The run prints:

```
senses of bank: ['bank.n.slope', 'bank.n.finance']
synonyms in slope.n: ['slope', 'incline']
hypernym chain: bank.n.finance -> institution.n -> organization.n -> entity.n
path_similarity(bank.n.finance, school.n) = 0.333
path_similarity(bank.n.finance, slope.n) = 0.167
```

The financial sense of `bank` is closer to `school` (both are institutions) than to `slope`, even though the *other* sense of `bank` is a kind of slope. A sense inventory makes that distinction explicit; a single vector for the string `bank` cannot.

Hand-built resources are precise and interpretable, but they are expensive to build, cover some languages and domains far better than others, lag behind new words and senses, and encode one set of sense distinctions that may not match an application. Distributional vectors make the opposite trade: they are cheap and broad, but their relations are implicit and mixed. Much practical work combines the two, for example by checking embedding neighbors against a lexical resource.

Distributional evidence: contexts are observations

Plain version. Words that appear in similar surroundings can acquire similar vectors. The surroundings provide evidence about usage; they do not directly supply definitions or truth conditions.

Harris argued that differences in meaning correlate with differences in distribution: the environments in which linguistic elements occur [5]. This **distributional hypothesis** is the foundation of every method in the rest of this lesson.

A **word type** is a vocabulary entry, while a **word token** is one occurrence. A static embedding pools evidence across many tokens into one vector for the type. Occurrences of `bank` beside `riv`

`er` and beside `account` therefore influence the same row. The model can learn that the word has mixed usage, but the lookup operation cannot choose a different row for each sense.

Define context before collecting statistics. A radius-two word window contains up to two tokens on each side of the center, excluding the center itself. A shorter window emphasizes immediate surroundings; a larger one includes more distant topic evidence. If direction matters, left and right context positions can be separate features. Our examples deliberately merge the two sides so the model's invariance is visible.

The **distributional assumption** is useful because text provides repeated observations without requiring a semantic label for every word. It is incomplete because shared use includes synonymy, opposition, thematic association and grammatical compatibility. `hot` and `cold` can both appear before `water`; similarity on that evidence does not establish that they express the same property. In the terms of the previous section, distributional similarity mixes synonymy, antonymy, co-hyponymy and relatedness. Turney and Pantel organize these vector-space approaches by the evidence represented in their matrices [6].

Keep corpus, tokenizer and window specification together. Changing word segmentation changes the types whose contexts are counted. An embedding learned on technical incident reports may organize `charge` differently from one learned on electronics manuals. The coordinates are an outcome of those choices, not language-independent measurements of meaning.

Count-based embeddings: inspect the shared-context assumption

Plain version. Count how often each word appears beside each context, then compare that count with a baseline that treats the word and context as independent. An unusually strong association contributes a positive coordinate.

The distributional survey by Turney and Pantel [6] explains this **count-based** route. Let C_{wc} count the observed ordered pairs consisting of center word w and neighboring context word c. The two roles remain distinct even when their vocabularies contain the same words. Define empirical probabilities from the table:

$$N_C = \sum_{w,c} C_{wc}, \qquad P(w,c) = \frac{C_{wc}}{N_C}, \qquad P(w) = \sum_{c} P(w,c), \qquad P(c) = \sum_{w} P(w,c).$$

Here N_C is the total center–context pair count, C_{wc} is one cell count, $P(w,c)$ is its empirical joint probability, $P(w)$ is the center marginal, and $P(c)$ is the context marginal. The sums range over the corresponding vocabularies. A token may contribute several pairs, so N_C is not the number of tokens.

Pointwise mutual information (PMI) compares the empirical joint probability with the independence baseline. **Positive PMI (PPMI)** retains only associations stronger than that baseline:

$$\mathrm{PMI}(w,c) = \log \frac{P(w,c)}{P(w)P(c)}, \qquad \mathrm{PPMI}(w,c) = \max(\mathrm{PMI}(w,c), 0).$$

Here $\mathrm{PMI}(w,c)$ is the log association score for one pair, $\mathrm{PPMI}(w,c)$ is its nonnegative version, and

log is the natural logarithm. The probabilities are those defined above. A pair with zero joint count and positive marginals has PMI tending to negative infinity and PPMI zero; an entirely unobserved row requires a separate policy.

A frequent context does not automatically receive a large association score. It must occur near this center more often than its overall frequency predicts. Conversely, a rare pair can have high PMI on little evidence. Inspect counts alongside scores; removing negative associations and keeping only positive ones also discards information rather than making every remaining association reliable.

The PPMI matrix still has one coordinate per context. A truncated **singular value decomposition** (**SVD**) compresses it:

$$M = A\Sigma B^\top, \qquad M_D = A_D\Sigma_D B_D^\top, \qquad E = A_D\Sigma_D.$$

Here M is the PPMI matrix, A and B contain its left and right singular vectors, Σ is the diagonal matrix of singular values, D is the retained dimension, and the subscript D selects the first D singular components. M_D is the resulting low-rank reconstruction, and E supplies the center-word vectors used below. This is the convention implemented in the code; other distributions of singular-value scaling produce different vector geometries.

SVD identifies shared patterns across rows rather than assigning a semantic name to each coordinate. The compression cannot preserve every cell when the original matrix has higher rank than D. The following synthetic corpus lets us inspect both the construction and a failure of interpreting cosine similarity as synonymy.

```
import numpy as np

sentences = """the cat drinks milk . the dog drinks water . a cat chases a mouse .
a dog chases a cat . the kitten drinks milk . the puppy drinks water .
a kitten chases a ball . a puppy chases a ball . the car needs fuel .
the truck needs fuel . a car drives on the road . a truck drives on the road .""".split(".")
tokens = [s.split() for s in sentences if s.strip()]
vocab = sorted({w for s in tokens for w in s})
idx = {w: i for i, w in enumerate(vocab)}

# 1) Co-occurrence counts within a +-2 word window.
C = np.zeros((len(vocab), len(vocab)))
for s in tokens:
    for i, w in enumerate(s):
        for j in range(max(0, i - 2), min(len(s), i + 3)):
            if j != i:
                C[idx[w], idx[s[j]]] += 1

# 2) Positive pointwise mutual information.
total = C.sum()
pw, pc = C.sum(1, keepdims=True) / total, C.sum(0, keepdims=True) / total
with np.errstate(divide="ignore"):
    pmi = np.log((C / total) / (pw * pc))
ppmi = np.maximum(pmi, 0)

# 3) Truncated SVD gives dense 8-dimensional embeddings.
U, S, _ = np.linalg.svd(ppmi)
```

```
E = U[:, :8] * S[:8]
E /= np.linalg.norm(E, axis=1, keepdims=True)

def nearest(word, k=3):
    sims = E @ E[idx[word]]
    order = [vocab[i] for i in np.argsort(-sims) if vocab[i] != word]
    return [(w, round(float(sims[idx[w]]), 2)) for w in order[:k]]

for w in ["cat", "puppy", "truck", "milk"]:
    print(w, "->", nearest(w))
```

Output:

```
cat -> [('kitten', 0.96), ('puppy', 0.88), ('dog', 0.88)]
puppy -> [('dog', 1.0), ('kitten', 0.98), ('cat', 0.88)]
truck -> [('car', 1.0), ('needs', 0.6), ('the', 0.47)]
milk -> [('water', 0.97), ('a', 0.62), ('chases', 0.51)]
```

Animals cluster with animals and vehicles with vehicles, learned from twelve sentences and no labels. Note "milk" and "water" are nearly identical: both are things that are drunk in this corpus. Embedding similarity measures **shared contexts**, which often but not always matches similar meaning; antonyms such as "hot" and "cold" also share contexts.

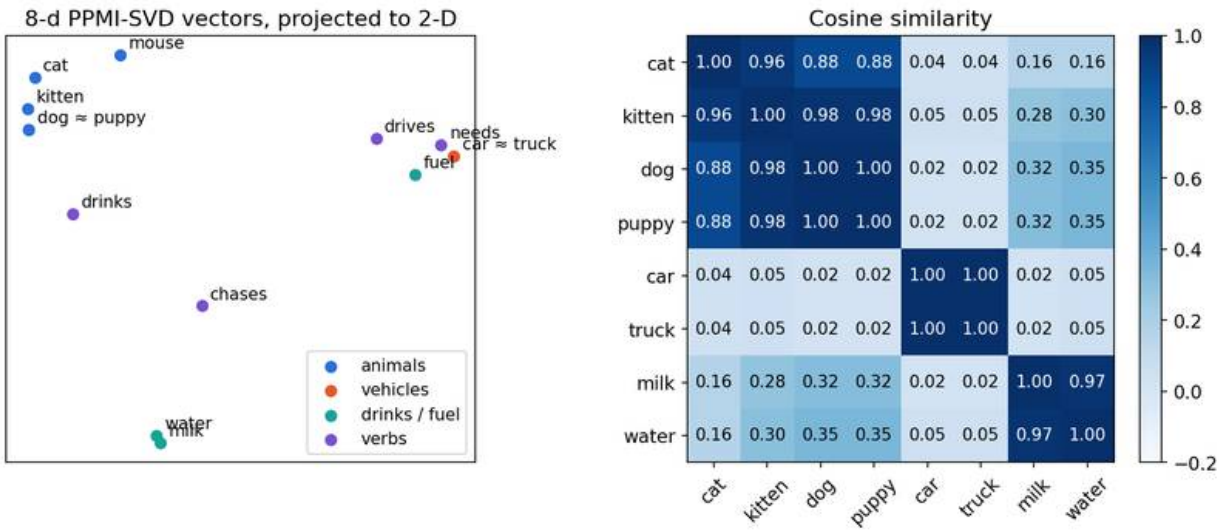


Figure 1 — Vectors from the code above. Cosine similarity is high within groups that share contexts.

Word2Vec: learn a word by predicting its neighbors

Plain version. Words used in similar surroundings often serve similar roles. Word2Vec turns that observation into a small prediction problem: learn vectors that help recover a word from its

neighbors, or its neighbors from the word. The vectors are learned parameters, not hand-written lists of meanings. The two basic architectures are **continuous bag of words** (**CBOW**) and **Skip-gram** [7].

Turn a sentence into training examples

Start with the invented sentence `small red robots move slowly`. A **context window** chooses nearby positions; radius one means the word immediately before and after the center. For this example, use only interior centers, so every window has two neighbors:

Center	Context	CBOW training example	Skip-gram training examples
`red`	`small`, `robots`	`[small, robots] → red`	`red → small`; `red → robots`
`robots`	`red`, `move`	`[red, move] → robots`	`robots → red`; `robots → move`
`move`	`robots`, `slowly`	`[robots, slowly] → move`	`move → robots`; `move → slowly`

CBOW combines several context words to predict one center word. Skip-gram uses one center word to predict each context word separately. Skip-gram does not generate an ordered sentence, and "skip" does not mean remove random words. The target may come from either side of the center. At sentence boundaries, an implementation can use shorter windows; our code omits those centers to keep the tensor shapes simple.

The training labels come from the text itself. Nobody must annotate `robots` with a semantic category before these examples can be created. Repeated occurrences create repeated evidence. The chosen window determines what relationships the model is encouraged to capture: changing it changes the training task, even when the vocabulary stays the same.

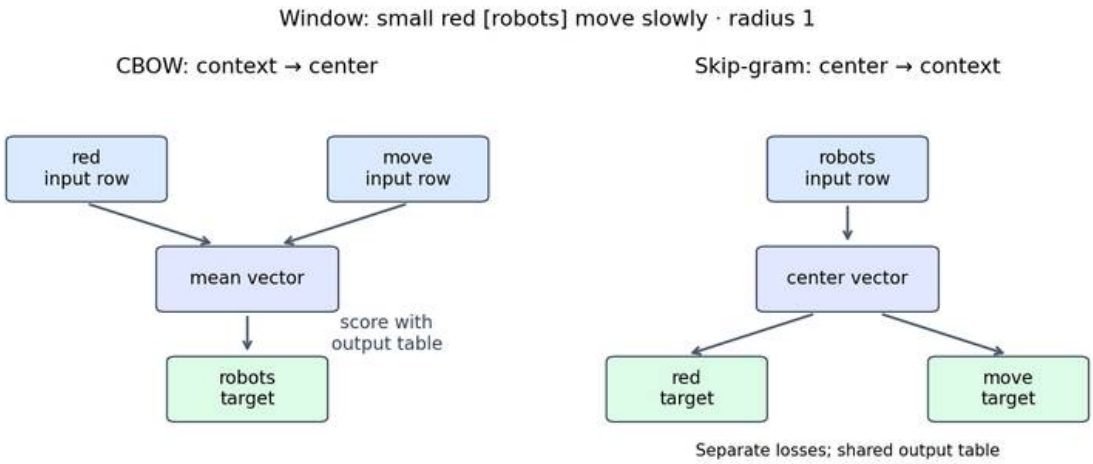


Figure 2 — The same sentence creates opposite prediction directions. Both architectures learn input and output embedding tables; the average appears only in the CBOW path shown here.

Two embedding tables and a softmax

A word needs a vector when it is an input and a vector when it is a prediction candidate. Use two learned tables, rather than assuming these roles share weights:

$$E, U \in \mathbb{R}^{V \times D}, \qquad \mathbf{e}_w = E_{w,:}^\top, \qquad \mathbf{u}_w = U_{w,:}^\top, \qquad p(o \mid \mathbf{h}) = \frac{\exp(\mathbf{u}_o^\top \mathbf{h})}{\sum_{v=1}^{V} \exp(\mathbf{u}_v^\top \mathbf{h})}.$$

Here E is the input embedding table, U is the output embedding table, V is vocabulary size, D is vector dimension, w is a word ID, and the colon selects all coordinates in its row. The transpose makes $\mathbf{e}_w$ and $\mathbf{u}_w$ column vectors. The vector $\mathbf{h} \in \mathbb{R}^D$ represents the input, o is the target word ID, v indexes a candidate word, and $p(o \mid \mathbf{h})$ is its softmax probability. Each dot product is a prediction score; the denominator makes probabilities sum to one over the vocabulary.

CBOW forms the input by averaging the context vectors. Its loss asks the model to assign high probability to the observed center:

$$\mathbf{h}_t = \frac{1}{|C_t|} \sum_{j \in C_t} \mathbf{e}_{w_j}, \qquad \mathcal{L}_{\text{CBOW}} = -\frac{1}{|\mathcal{T}|} \sum_{t \in \mathcal{T}} \log p(w_t \mid \mathbf{h}_t).$$

Here w_t is the center word at position t, C_t is the set of selected neighboring positions excluding t, $|C_t|$ is its size, $\mathbf{h}_t$ is their mean vector, $\mathcal{T}$ contains the valid center positions, and $\mathcal{L}_{\text{CBOW}}$ is the average negative log probability of those centers. The mean ignores context order: exchanging left and right neighbors produces the same input. Some formulations sum context vectors instead; this lesson and its code use the mean.

Skip-gram instead looks up the center vector and applies the same prediction rule to each neighbor:

$$\mathcal{L}_{\text{SG}} = -\frac{1}{|\mathcal{P}|} \sum_{(t,j) \in \mathcal{P}} \log p(w_j \mid \mathbf{e}_{w_t}).$$

Here $\mathcal{L}_{\text{SG}}$ is the Skip-gram loss, $\mathcal{P}$ is the collection of observed center–context position pairs, $|\mathcal{P}|$ is its size, t indexes a center position, j indexes one neighboring position, w_t is the input word and w_j is its target. Occurrences remain separate even when their word IDs repeat.

A training step increases the observed target's score relative to competing candidates. It changes both the selected input vectors and the output vectors. Over many examples, reusing the same input row makes the model reconcile that word's different surroundings. This shared prediction task is the route from discrete word IDs to learned continuous features. Neither architecture needs a deep stack of hidden layers in this basic form, which is the main computational difference from the neural language model in NLP 03.

Negative sampling: compare observed pairs with random draws

A full softmax scores every vocabulary word for every prediction. **Negative sampling** replaces that vocabulary-wide classification with a smaller binary task: distinguish an observed center–context pair from pairs formed with randomly drawn target words. It was introduced in the Word2Vec follow-up paper [8].

For an observed input word c and context word o, one sampled loss is:

$$\mathcal{L}_{\text{neg}}(c, o) = -\log \sigma(\mathbf{u}_o^\top \mathbf{e}_c) - \sum_{k=1}^{K} \log \sigma(-\mathbf{u}_{n_k}^\top \mathbf{e}_c), \qquad n_k \sim q, \qquad \sigma(a) = \frac{1}{1 + \exp(-a)}.$$

Here $\mathcal{L}_{\text{neg}}$ is the negative-sampling loss for one observed pair, c is the center word ID, o is its observed context word ID, $\mathbf{e}_c$ is the center's input vector, $\mathbf{u}_o$ and $\mathbf{u}_{n_k}$ are output vectors, K is the chosen number of random targets, n_k is draw k from distribution q, and σ is the sigmoid applied to a real score a. The first term rewards a large observed-pair score; the remaining terms reward small random-pair scores. The full training objective averages this expression over observed pairs and sampling randomness. For a CBOW version, use the averaged context vector in place of $\mathbf{e}_c$.

A common sampling distribution from the original paper is:

$$q(w) = \frac{f(w)^{3/4}}{\sum_{v=1}^{V} f(v)^{3/4}}.$$

Here $q(w)$ is the probability of drawing vocabulary word w, $f(w)$ is its corpus occurrence count, v indexes vocabulary entries, and V is vocabulary size. The exponent flattens the frequency distribution relative to sampling directly by counts. It is a design choice, not a universal probability law.

For the observed pair `robots → move`, a random draw might create `robots → red`. The label "negative" means **drawn by the noise process**; it does not assert that the two words can never occur together. A draw can be a genuine context elsewhere, or even collide with this target. An implementation must specify whether it retains, skips or redraws collisions; changing that rule changes the sampled objective. The model separates statistical sources of pairs rather than learning a dictionary of absolute incompatibilities.

The binary sigmoid scores do not sum to one over vocabulary words. Negative sampling is a different training objective, not an exact computation of the softmax above, and its loss is not a normalized next-token likelihood. Use it to learn useful representations; evaluate those representations on the intended downstream task.

Why prediction changes vector geometry

The phrase "learn from neighbors" becomes concrete in the gradient. For one full-softmax target, let its score be the dot product between an output vector and the input representation:

$$s_v = \mathbf{u}_v^\top \mathbf{h}, \qquad \ell = -\log p(o \mid \mathbf{h}), \qquad \frac{\partial \ell}{\partial s_v} = p(v \mid \mathbf{h}) - \mathbf{1}[v = o], \qquad \nabla_{\mathbf{h}} \ell = \sum_{v=1}^{V} p(v \mid \mathbf{h}) \mathbf{u}_v - \mathbf{u}_o.$$

Here s_v is candidate v's score, ℓ is the loss for observed target o, $\mathbf{1}[v = o]$ equals one for that target and zero otherwise, and $\nabla_{\mathbf{h}}\ell$ is the gradient with respect to the input representation. The embedding tables, vocabulary size V and conditional probability are those defined above.

Gradient descent moves the input toward the observed output vector and away from the model's probability-weighted average of alternatives. A word with many occurrences receives many such updates through the same row. Similar context distributions can therefore give different words compatible predictive roles, without any training label declaring them synonyms.

For mean-based CBOW, every selected context row receives $\nabla_{\mathbf{h}}\ell/|C_t|$. The input cannot distinguish two contexts that are permutations of the same words. For Skip-gram, one center row receives updates from multiple neighbor targets. Its basic score has no slot for whether a neighbor was on the left or right. Choosing the window therefore chooses which relations the objective can directly represent; a wider window is not simply a more accurate version of a narrow one.

What negative-sampling scores estimate

We can inspect the sampling objective before imposing the embedding dimension. Fix a center word and imagine that every candidate has an independently adjustable score. Let p_o be its empirical context probability and q_o its noise probability. The expected contribution for that candidate is:

$$\ell_o(s_o) = -p_o \log \sigma(s_o) - Kq_o \log \sigma(-s_o), \qquad \frac{d\ell_o}{ds_o} = (p_o + Kq_o)\sigma(s_o) - p_o.$$

Here o is one candidate context, s_o is its unrestricted real score, p_o and q_o are the two probabilities for the fixed center, K is the number of independent noise draws per observed pair and σ is the sigmoid defined above. This calculation retains collisions between noise draws and observed targets.

When both probabilities are positive, the stationary score is:

$$s_o^* = \log \frac{p_o}{Kq_o}.$$

Here s_o^* minimizes that candidate's expected loss. For $p_o = 0.5$, $q_o = 0.25$ and $K = 2$, it is zero: half the relevant probability mass is attributed to each statistical source. If $q_o = 0.05$ instead, the optimum becomes $\log 5 \approx 1.609$. The observed context frequency stayed fixed; the noise distribution changed the score. A positive score therefore means "more associated than this sampling baseline," rather than "the pair is certainly meaningful." If the observed probability is zero and noise probability is positive, the unrestricted optimum tends toward negative infinity.

When noise is exactly the corpus context marginal, this ratio yields PMI minus $\log K$. Levy and Goldberg analyze the resulting matrix-factorization interpretation [9]. The common three-quarter-power distribution changes the ratio. In an actual embedding model, scores share low-dimensional vectors, so they cannot generally reach every independent optimum. The learned tables compromise among the pair statistics under a rank constraint. This explains both the connection to the count-based construction and why truncating a PPMI matrix and training Word2Vec need not give identical vectors.

Full softmax costs roughly $O(VD)$ arithmetic per target, while scoring one observed target and K draws costs $O((K+1)D)$, before sampling and batching overhead. Here D is embedding dimension. The saving comes with a different objective. Neither the sigmoid value nor the negative-sampling loss is a substitute for the normalized token probability used to report perplexity.

Train CBOW and Skip-gram on CPU

This standalone example learns both input and output tables using full softmax. The vocabulary is tiny, so we can inspect the objective without a sampling approximation. Both models use the same invented corpus, radius-one interior windows, embedding width and training loop. A one-token input average in the Skip-gram path is simply the center vector.

```
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(7)
torch.set_num_threads(1)
corpus = [
    'small red robots move slowly',
    'small blue robots move quickly',
    'large red sensors work slowly',
    'large blue sensors work quickly',
    'small red sensors work quietly',
    'large blue robots move quietly',
]
vocab = sorted(set(' '.join(corpus).split()))
ids = {word: i for i, word in enumerate(vocab)}
cbow_x, cbow_y, sg_x, sg_y = [], [], [], []
for sentence in corpus:
    tokens = [ids[word] for word in sentence.split()]
    # Use radius 1 and interior centers, so every context has two words.
    for t in range(1, len(tokens) - 1):
        neighbors = [tokens[t - 1], tokens[t + 1]]
        cbow_x.append(neighbors)
        cbow_y.append(tokens[t])
        for neighbor in neighbors:
            sg_x.append([tokens[t]])
            sg_y.append(neighbor)

class Word2Vec(nn.Module):
    def __init__(self, size, width=8):
        super().__init__()
        self.input = nn.Embedding(size, width)
        self.output = nn.Embedding(size, width)
        nn.init.normal_(self.input.weight, std=0.1)
        nn.init.normal_(self.output.weight, std=0.1)

    def forward(self, context):
        h = self.input(context).mean(dim=1)
        return h @ self.output.weight.T
```

```
embeddings = {}
for name, inputs, targets in [('CBOW', cbow_x, cbow_y),
                              ('Skip-gram', sg_x, sg_y)]:
    x, y = torch.tensor(inputs), torch.tensor(targets)
    model = Word2Vec(len(vocab))
    optimizer = torch.optim.SGD(model.parameters(), lr=1.0)
    initial = F.cross_entropy(model(x), y).item()
    for step in range(250):
        optimizer.zero_grad()
        loss = F.cross_entropy(model(x), y)
        loss.backward()
        optimizer.step()
    final = F.cross_entropy(model(x), y).item()
    assert final < initial
    embeddings[name] = model.input.weight.detach().clone()
    print(name, len(y), f'{initial:.3f} -> {final:.3f}')
print('vocabulary', len(vocab), 'embedding shape', tuple(embeddings['CBOW'].shape))
```

The run prints `CBOW 18 2.384 -> 0.159`, `Skip-gram 36 2.402 -> 1.198`, and `vocabulary 11 embedding shape (11, 8)`. Losses are mean negative log probabilities in nats. The assertions establish that the implementation fits these training examples. The smaller CBOW loss does not establish that CBOW produces better word vectors: its targets and conditional inputs differ from Skip-gram's. This corpus is too small to judge semantic neighborhoods or natural-language generalization.

After training, `embeddings['CBOW'][ids['robots']]` retrieves one learned input vector. The output table was needed to train it but can be omitted when serving a downstream model that only looks up input vectors. Record which table you retain; using input vectors, output vectors or a combination is a representation choice.

GloVe: fit log co-occurrence counts directly

Plain version. Instead of streaming through windows one prediction at a time, GloVe first counts how often every pair of words co-occurs, then fits word vectors whose dot products reproduce the logarithms of those counts. Frequent pairs get more weight, but not unlimited weight.

Pennington et al. motivate GloVe with ratios of co-occurrence probabilities [10]. Let X_{ik} be the (distance-weighted) number of times word k occurs in the context of word i, and $P(k \mid i) = X_{ik} / \sum_{k'} X_{ik'}$. For two words i and j, the ratio $P(k \mid i)/P(k \mid j)$ is large for context words k that belong with i but not j, small for the reverse, and near one for words that are equally related or unrelated to both. The ratio discriminates better than either probability alone. The model asks vector differences to encode such ratios, which leads to a weighted least-squares objective over the co-occurrence matrix:

$$J = \sum_{i,j=1}^{V} f\left(X_{ij}\right)\left(\mathbf{w}_i^{\top}\tilde{\mathbf{w}}_j + b_i + \tilde{b}_j - \log X_{ij}\right)^2, \qquad f(x) = \begin{cases} (x/x_{\max})^{\alpha} & x < x_{\max}, \\ 1 & \text{otherwise.} \end{cases}$$

Here V is vocabulary size, X_{ij} is the co-occurrence count of words i and j, $\mathbf{w}_i \in \mathbb{R}^D$ is the word vector of i, $\tilde{\mathbf{w}}_j \in \mathbb{R}^D$ is a separate context vector of j, b_i and $\tilde{b}_j$ are scalar biases, f is a weighting function, and $x_{\max}$ and α are its cutoff and exponent. Because $f(0) = 0$, cells with zero count contribute nothing, so $\log 0$ is never needed and the sum runs only over observed pairs. The paper fixes $x_{\max} = 100$ and $\alpha = 3/4$, uses a co-occurrence weighting in which a pair d words apart adds $1/d$, and reports the sum $W + \tilde{W}$ of the two tables as the final word vectors [10].

Three design choices separate GloVe from Word2Vec. It trains on global counts rather than on individual windows, so one pass over the corpus builds X and training then costs time proportional to the number of nonzero cells. The biases absorb word frequency, so the dot product models what remains. And the weighting function prevents very frequent pairs, such as those involving `the`, from dominating, while giving rare and noisy pairs little influence. Like negative sampling, the objective factorizes a shifted log-count matrix under a rank constraint, which is why GloVe and Skip-gram vectors often behave similarly in practice.

The following code computes the ingredients on the earlier toy corpus: co-occurrence ratios and the weighting function.

```
import numpy as np

text = """the cat drinks milk . the dog drinks water . a cat chases a mouse .
a dog chases a cat . the kitten drinks milk . the puppy drinks water .
a kitten chases a ball . a puppy chases a ball . the car needs fuel .
the truck needs fuel . a car drives on the road . a truck drives on the road ."""
sents = [s.split() for s in text.split(".") if s.strip()]
vocab = sorted({w for s in sents for w in s})
idx = {w: i for i, w in enumerate(vocab)}

X = np.zeros((len(vocab), len(vocab)))            # +-2 window, 1/distance weighting
for s in sents:
    for i, w in enumerate(s):
        for j in range(max(0, i - 2), min(len(s), i + 3)):
            if j != i:
                X[idx[w], idx[s[j]]] += 1.0 / abs(i - j)
P = X / X.sum(1, keepdims=True)                   # P(k | i): row-normalized counts

for k in ["drinks", "chases", "needs", "the"]:
    a, b = P[idx["kitten"], idx[k]], P[idx["truck"], idx[k]]
    ratio = "inf" if b == 0 else f"{a / b:.2f}"
    print(f"P({k}|kitten)={a:.3f}  P({k}|truck)={b:.3f}  ratio={ratio}")

def f(x, x_max=100.0, alpha=0.75):                 # GloVe weighting function
    return np.minimum((x / x_max) ** alpha, 1.0)
print("f at 1, 10, 100, 1000:", np.round(f(np.array([1.0, 10.0, 100.0, 1000.0])), 3))
print("nonzero cells:", int((X > 0).sum()), "of", X.size)
```

The run prints:

```
P(drinks|kitten)=0.200  P(drinks|truck)=0.000  ratio=inf
P(chases|kitten)=0.200  P(chases|truck)=0.000  ratio=inf
```

```
P(needs|kitten)=0.000  P(needs|truck)=0.200  ratio=0.00
P(the|kitten)=0.200  P(the|truck)=0.200  ratio=1.00
f at 1, 10, 100, 1000: [0.032 0.178 1.    1.   ]
nonzero cells: 96 of 361
```

The ratios separate animal contexts from vehicle contexts, while `the` gives a ratio of one and therefore no information. The output also exposes a limitation of this tiny corpus: almost three quarters of the matrix is zero, and GloVe ignores zero cells. With so few observed pairs, a model with a few hundred parameters can fit every observed cell exactly, and the resulting geometry depends mainly on initialization. GloVe's count-based design needs large corpora in which the matrix is well populated; we therefore do not train it here. Exercise 3 asks you to observe this underdetermination yourself.

fastText: vectors from character n-grams

Plain version. A word-level table has no vector for a word it never saw, and it treats `charger` and `chargers` as unrelated rows. fastText builds each word's vector from vectors of its character pieces, so related spellings share parameters and unseen words still get a vector.

Bojanowski et al. extend Skip-gram with negative sampling by representing each word as a bag of character n-grams [11]. The word is wrapped in boundary symbols, < and >, so that prefixes and suffixes are distinguishable from the same letters inside a word, and the whole wrapped word is added as one extra unit. In the paper, all n-grams with $3 \le n \le 6$ are used. The score between a center word w and a context word c becomes

$$s(w, c) = \sum_{g \in \mathcal{G}_w} \mathbf{z}_g^\top \mathbf{v}_c.$$

Here $\mathcal{G}_w$ is the set of n-grams of w (including the wrapped word itself), $\mathbf{z}_g \in \mathbb{R}^D$ is the learned vector of n-gram g, and $\mathbf{v}_c \in \mathbb{R}^D$ is the context vector of c. The word's own vector is $\sum_{g \in \mathcal{G}_w} \mathbf{z}_g$. This score replaces $\mathbf{u}_o^\top \mathbf{e}_c$ in the negative-sampling loss above. To bound memory, the paper maps n-grams into a fixed number of buckets with a hash function, so unrelated n-grams can share a vector [11].

```
import numpy as np

def char_ngrams(word, n_min=3, n_max=6):
    w = f"<{word}>"                      # boundary symbols mark prefixes and suffixes
    grams = {w[i:i + n] for n in range(n_min, n_max + 1) for i in range(len(w) - n + 1)}
    return grams | {w}                   # the whole word is kept as one extra unit

print(sorted(char_ngrams("robot", 3, 3)))
vocab = ["charge", "charger", "recharge", "discharge", "robot", "robots"]
query = "chargers"                       # not in the vocabulary
q = char_ngrams(query)
for word in vocab:
```

```
    print(f"{word:10s} shares {len(q & char_ngrams(word)):2d} of {len(q)} units with
     ↪  '{query}'")

# A fastText-style vector is the sum of its unit vectors, so an unseen word still gets one.
rng = np.random.default_rng(0)
table = {}                                  # stands in for a trained (hashed) n-gram table
def vector(word, dim=8):
    return sum(table.setdefault(g, rng.normal(size=dim)) for g in sorted(char_ngrams(word)))
def cos(a, b):
    return a @ b / (np.linalg.norm(a) * np.linalg.norm(b))
for word in vocab:
    vector(word)
print("cos(chargers, charger) =", round(cos(vector("chargers"), vector("charger")), 3))
print("cos(chargers, robots)  =", round(cos(vector("chargers"), vector("robots")), 3))
```

The run prints:

```
['<ro', '<robot>', 'bot', 'obo', 'ot>', 'rob']
charge     shares 14 of 27 units with 'chargers'
charger    shares 18 of 27 units with 'chargers'
recharge   shares 10 of 27 units with 'chargers'
discharge  shares 10 of 27 units with 'chargers'
robot      shares  0 of 27 units with 'chargers'
robots     shares  0 of 27 units with 'chargers'
cos(chargers, charger) = 0.804
cos(chargers, robots)  = -0.222
```

The n-gram vectors here are random, not trained, so the high cosine between `chargers` and `charger` comes only from the units they share. That is exactly the mechanism: an unseen inflection inherits most of its vector from known pieces. Training decides whether sharing is helpful. It usually is for inflection and derivation, and it is misleading for words that share letters by accident; `discharge` and `recharge` share `charge` but differ in meaning in ways the pieces do not reveal. Subword vectors help most for morphologically rich languages and for rare words. Note the relation to the subword tokenizers in NLP 02: BPE splits a word into one sequence of pieces before lookup, whereas fastText sums vectors of *all* overlapping n-grams.

Dense word and document representations

Plain version. A dense vector lets different words share statistical strength. That can connect related phrases, but the vector's geometry reflects its training data and objective rather than a universal definition of meaning.

Static vectors provide compact reusable features for a classifier, nearest-neighbor word exploration or a pooled document baseline. They are useful when a labeled application has little data but an appropriate text corpus is available. Training the embedding and the downstream classifier are different stages: neighbor prediction never directly learns a billing label or an abstention policy.

A static embedding table assigns a vector to each vocabulary entry:

$$\mathbf{e}_t = E_{i_t,:}, \qquad \mathbf{h}(d) = \frac{1}{T}\sum_{t=1}^{T} \mathbf{e}_t.$$

Here $E \in \mathbb{R}^{V \times D}$ is the embedding table, V is vocabulary size, D is embedding dimension, i_t is the token ID at position t, $\mathbf{e}_t$ is its row vector, T is document length in tokens and $\mathbf{h}(d)$ is the mean-pooled document vector. The colon denotes all coordinates in the selected row.

The tables above supply ways to obtain E. Once token vectors are available, mean pooling turns a variable-length document into a fixed-width feature. This aggregation step is separate from the objective used to train the table.

Mean pooling is an instructive baseline because it exposes a limitation clearly. Any permutation of a document's tokens gives the same average. The representation of "the technician helped the customer" is identical to "the customer helped the technician" if the same static token vectors are used. It also treats negation as one contribution among many rather than an operator changing the meaning of a phrase. The word `bank` retrieves the same static vector in `river bank` and `bank account`; more training data does not change this lookup rule.

There are two separate choices: how token vectors are obtained and how they are combined. You can freeze pretrained token vectors and train only a classifier, fine-tune the vectors with the classifier, or train all parameters from scratch. Freezing reduces the number of task-specific parameters. Fine-tuning can adapt meanings to a domain, but small datasets can distort representations or memorize labels.

Do not compare embeddings only by visually attractive clusters. A two-dimensional projection can conceal overlap in the original space. Compare representations under the same task, split, classifier capacity and tuning allowance. If a dense representation changes the downstream head too, the experiment no longer isolates the representation.

Cosine measures direction, not truth

The count-based example compares unit-normalized vectors. Its dot product is then the cosine of the angle between them:

$$\cos(\mathbf{a}, \mathbf{b}) = \frac{\mathbf{a}^\top \mathbf{b}}{\|\mathbf{a}\|_2 \|\mathbf{b}\|_2}.$$

Here $\mathbf{a}$ and $\mathbf{b}$ are nonzero embedding column vectors, $\mathbf{a}^\top \mathbf{b}$ is their inner product, and $\|\cdot\|_2$ is Euclidean length. The cosine lies between -1 and 1. A zero vector needs an explicit handling rule because the denominator vanishes.

Cosine removes positive rescaling of either vector. It does not remove every transformation compatible with the training objective. In the two-table model, the predictive scores depend on products of an input and an output vector. Many different tables produce the same products:

$$E' = EA, \qquad U' = UA^{-\top}, \qquad E'U'^\top = EU^\top.$$

Here E and U are the input and output tables with shape $V \times D$, A is any invertible $D \times D$ matrix, $A^{-\top}$ is the transpose of its inverse, and E' and U' are the transformed tables. Each cell of $EU^\top$ is one center–output prediction score. Thus full-softmax predictions and the negative-sampling objective are unchanged by this paired transformation. The same argument applies to GloVe's $\mathbf{w}_i^\top \tilde{\mathbf{w}}_j$.

An orthogonal change of coordinates preserves input-vector angles, but general invertible changes need not. Consequently, identical prediction loss does not uniquely determine cosine neighborhoods in the retained input table. Optimization, initialization and additional constraints select a particular solution. This is one reason a low training loss alone cannot certify useful downstream similarity.

This standalone numerical check combines three claims: score preservation, order invariance of averaging, and the noise-dependent optimum derived above. The matrices are hand-chosen for this calculation; they are not trained language vectors.

```
import numpy as np

# Two input rows and three output rows: scores are center-output dot products.
E = np.array([[1., 1.], [1., -1.]])
U = np.array([[1., 0.], [0., 1.], [-1., 1.]])
A = np.diag([2., 0.5])
changed_E = E @ A
changed_U = U @ np.linalg.inv(A).T

def cosine(a, b):
    return (a @ b) / (np.linalg.norm(a) * np.linalg.norm(b))

assert np.allclose(E @ U.T, changed_E @ changed_U.T)
assert np.allclose(E[[0, 1]].mean(0), E[[1, 0]].mean(0))
print('scores unchanged:', np.allclose(E @ U.T, changed_E @ changed_U.T))
print('cosine:', f'{cosine(E[0], E[1]):.3f}', '->',
      f'{cosine(changed_E[0], changed_E[1]):.3f}')
print('mean unchanged by order:',
      np.allclose(E[[0, 1]].mean(0), E[[1, 0]].mean(0)))

# The unrestricted negative-sampling optimum depends on the noise baseline.
p, K = 0.5, 2
for q in [0.25, 0.05]:
    score = np.log(p / (K * q))
    sigmoid = 1 / (1 + np.exp(-score))
    gradient = (p + K * q) * sigmoid - p
    assert np.isclose(gradient, 0)
    print('noise', q, 'optimal score', f'{score:.3f}')
```

The output is:

```
scores unchanged: True
cosine: 0.000 -> 0.882
mean unchanged by order: True
```

```
noise 0.25 optimal score 0.000
noise 0.05 optimal score 1.609
```

The cosine changed while all center–output scores stayed fixed. The equal means have a different implication: once a representation merges two token orders, no classifier that receives only that mean can recover which order occurred. These are properties of the representation, rather than evidence that one optimizer ran poorly.

Evaluating word embeddings

Plain version. There are two ways to judge embeddings: ask whether their geometry agrees with human judgments about words (intrinsic), or plug them into a task and measure the task (extrinsic). Neither alone tells you whether vectors are "good."

Similarity: correlate cosines with human ratings

A **word-similarity** dataset lists word pairs with graded human ratings. The model scores each pair by cosine, and the evaluation measures how well the two rankings agree with **Spearman's rank correlation**:

$$\rho = 1 - \frac{6 \sum_{i=1}^{n} \delta_i^2}{n(n^2 - 1)}.$$

Here n is the number of pairs, δ_i is the difference between the rank of pair i under the model's cosines and its rank under the human ratings, and $\rho \in [-1, 1]$. This formula assumes no ties; with ties, ρ is the Pearson correlation of the ranks. Spearman's ρ uses only ranks, so it ignores the scale of cosines.

What the ratings mean matters as much as the statistic. WordSim-353 is a widely used set of rated word pairs [12]. Hill et al. point out that its ratings do not separate similarity from association, and built SimLex-999 to quantify **similarity** explicitly, so that associated but dissimilar pairs receive low ratings [13]. A model trained with broad windows may score well on association-style data and poorly on similarity-style data, for the reason given in the lexical-semantics section: distributional evidence mixes the relations.

Analogy: the vector-offset test and its caveats

Mikolov, Yih and Zweig observed that some relations appear as roughly constant vector offsets [14]. To answer "a is to a^* as b is to ?", the **offset method** returns

$$x^* = \operatorname*{argmax}_{x \in \mathcal{U} \setminus \{a, a^*, b\}} \cos(\mathbf{e}_x,\ \mathbf{e}_{a^*} - \mathbf{e}_a + \mathbf{e}_b)\,.$$

Here $\mathcal{U}$ is the vocabulary, $\mathbf{e}_x$ is the vector of candidate x, and the three question words are excluded from the candidates. Linzen points out that this exclusion is doing real work: the target point

$\mathbf{e}_{a^*} - \mathbf{e}_a + \mathbf{e}_b$ often lies closest to b itself, so the method's success mixes offset consistency with the neighborhood structure around b [15]. He recommends reporting simple baselines, such as returning the nearest neighbor of b, alongside analogy accuracy.

The next example evaluates the PPMI vectors from earlier with invented similarity ratings and three analogies. The ratings are made up for this lesson and are not a benchmark.

```
import numpy as np
from scipy.stats import spearmanr

text = """the cat drinks milk . the dog drinks water . a cat chases a mouse .
a dog chases a cat . the kitten drinks milk . the puppy drinks water .
a kitten chases a ball . a puppy chases a ball . the car needs fuel .
the truck needs fuel . a car drives on the road . a truck drives on the road ."""
sents = [s.split() for s in text.split(".") if s.strip()]
vocab = sorted({w for s in sents for w in s})
idx = {w: i for i, w in enumerate(vocab)}
C = np.zeros((len(vocab), len(vocab)))
for s in sents:
    for i, w in enumerate(s):
        for j in range(max(0, i - 2), min(len(s), i + 3)):
            if j != i:
                C[idx[w], idx[s[j]]] += 1
P = C / C.sum()
with np.errstate(divide="ignore"):
    ppmi = np.maximum(np.log(P / (P.sum(1, keepdims=True) * P.sum(0, keepdims=True))), 0)
U, S, _ = np.linalg.svd(ppmi)
E = U[:, :8] * S[:8]
E /= np.linalg.norm(E, axis=1, keepdims=True)
cos = lambda a, b: float(E[idx[a]] @ E[idx[b]])

# 1) Similarity: correlate model cosines with (invented) graded judgments, 0-10.
judged = [("cat", "kitten", 9), ("dog", "puppy", 9), ("car", "truck", 8), ("milk", "water",
 ↪  5),
          ("cat", "dog", 6), ("cat", "mouse", 3), ("car", "road", 2), ("kitten", "fuel", 0)]
model = [cos(a, b) for a, b, _ in judged]
rho = spearmanr(model, [h for _, _, h in judged]).statistic
print(f"Spearman rho = {rho:.3f} over {len(judged)} pairs")

# 2) Analogy a:b :: c:? with the offset b - a + c (3CosAdd).
def analogy(a, b, c, exclude_inputs=True):
    target = E[idx[b]] - E[idx[a]] + E[idx[c]]
    scores = E @ (target / np.linalg.norm(target))
    for i in np.argsort(-scores):
        if not exclude_inputs or vocab[i] not in (a, b, c):
            return vocab[i]
for a, b, c in [("cat", "kitten", "dog"), ("car", "fuel", "cat"), ("cat", "milk", "puppy")]:
    print(f"{a}:{b} :: {c}:?  ->", analogy(a, b, c), "| without excluding inputs ->",
          analogy(a, b, c, exclude_inputs=False))
```

The run prints:

```
Spearman rho = 0.837 over 8 pairs
cat:kitten :: dog:?  -> puppy | without excluding inputs -> puppy
car:fuel :: cat:?  -> kitten | without excluding inputs -> cat
cat:milk :: puppy:?  -> water | without excluding inputs -> water
```

Two analogies come out as intended. The middle one shows both caveats at once. Without exclusion, the answer is `cat`, one of the question words. With exclusion, the answer is `kitten`, the nearest neighbor of `cat`, not `milk`, the intended "what it consumes" answer. The offset did not carry the relation; the neighborhood of b decided. Eight invented pairs and three analogies are, of course, far too few to support any general claim about these vectors.

Intrinsic versus extrinsic

Schnabel et al. compare evaluation methods and find that different evaluations order embedding methods differently, that word frequency affects results and cosine similarity, and that extrinsic tasks should not be used as a proxy for generic embedding quality [16]. The practical rule is to evaluate on the relation your application needs.

Nearest-neighbor lists can reveal obvious corpus patterns, but they are a diagnostic rather than an application score. Separate lexical questions: is the candidate a synonym, an antonym, a word associated with the topic, or a word that can occupy the same grammatical slot? A single unqualified notion of "related" hides these distinctions.

Use controlled pairs alongside ordinary held-out examples. For a representation used in sentiment, compare a positive claim with its negation. For a role-sensitive task, exchange who acted on whom while holding the word inventory fixed. For a technical corpus, test a term used in its domain-specific and everyday senses. These tests ask whether the representation preserves a required difference; they do not demand that every word pair occupy a predetermined location.

If mean pooling makes a critical pair identical, collecting more classifier labels cannot fix that collision while the representation remains fixed. If the vectors distinguish the pair but the classifier predicts incorrectly, supervision or the decision boundary may be responsible. Locate the failure at the earliest stage that loses the needed evidence.

Bias in embeddings

Plain version. Embeddings learn whatever regularities the corpus contains, including social stereotypes. If a group of names co-occurs with some occupations more than others, the vectors encode that association, and downstream systems can reproduce it.

Bolukbasi et al. showed that word embeddings trained on news text encode gender stereotypes, identified a gender direction in the space, and proposed removing it from words that should be gender-neutral [17]. Caliskan et al. introduced the Word Embedding Association Test (WEAT), modeled on the Implicit Association Test from psychology, and found that embeddings reproduce

a range of documented human biases [18]. For a target word w and two attribute sets A and B, the association used by WEAT is

$$s(w, A, B) = \frac{1}{|A|} \sum_{a \in A} \cos(\mathbf{e}_w, \mathbf{e}_a) - \frac{1}{|B|} \sum_{b \in B} \cos(\mathbf{e}_w, \mathbf{e}_b).$$

Here $|A|$ and $|B|$ are the set sizes and $\mathbf{e}$ denotes word vectors. A positive value means w is closer on average to set A. The full test aggregates this score over two sets of target words and compares it with a permutation distribution.

The following invented corpus contains no sentence linking a name to an occupation. Occupation words always appear with their own activity, while invented names appear with activities at skewed rates.

```
import random
import numpy as np

rng = random.Random(0)
# An invented corpus. Job words always appear with their own activity; invented
# names appear with activities at skewed rates. No sentence says "ana is a pilot".
group_a, group_b = ["ana", "ari"], ["bo", "bea"]
activity = {"pilot": "flies the plane", "baker": "bakes the bread", "clerk": "files the
 ↪  forms"}
share_a = {"pilot": 0.8, "baker": 0.2, "clerk": 0.5}    # P(group A | activity)
sents = []
for _ in range(3000):
    job = rng.choice(list(activity))
    if rng.random() < 0.5:
        sents.append(f"the {job} {activity[job]}".split())
    else:
        name = rng.choice(group_a if rng.random() < share_a[job] else group_b)
        sents.append(f"{name} {activity[job]}".split())
vocab = sorted({w for s in sents for w in s})
idx = {w: i for i, w in enumerate(vocab)}
C = np.zeros((len(vocab), len(vocab)))
for s in sents:
    for i, w in enumerate(s):
        for j in range(max(0, i - 2), min(len(s), i + 3)):
            if j != i:
                C[idx[w], idx[s[j]]] += 1
P = C / C.sum()
with np.errstate(divide="ignore"):
    ppmi = np.maximum(np.log(P / (P.sum(1, keepdims=True) * P.sum(0, keepdims=True))), 0)
U, S, _ = np.linalg.svd(ppmi)
E = U[:, :4] * S[:4]
E /= np.linalg.norm(E, axis=1, keepdims=True)

def association(word, E):            # mean cosine to group A minus mean cosine to group B
    unit = E / np.linalg.norm(E, axis=1, keepdims=True)
    w = unit[idx[word]]
    return np.mean([w @ unit[idx[a]] for a in group_a]) - np.mean([w @ unit[idx[b]] for b in
     ↪  group_b])
```

```
for job in activity:
    print(f"{job:6s} association {association(job, E):+.3f}")

# Project out one "group direction" and measure again.
d = E[[idx[a] for a in group_a]].mean(0) - E[[idx[b] for b in group_b]].mean(0)
d /= np.linalg.norm(d)
E_proj = E.copy()
for job in activity:
    E_proj[idx[job]] -= (E_proj[idx[job]] @ d) * d
print("after projection:", [f"{job} {round(association(job, E_proj), 3) + 0.0:+.3f}" for job
 ↪  in activity])
```

The run prints:

```
pilot  association +0.795
baker  association -0.811
clerk  association +0.072
after projection: ['pilot +0.000', 'baker +0.000', 'clerk +0.000']
```

The skew in who performs which activity became a strong association between names and occupation words, even though the two never co-occur in a sentence: both share the activity contexts. Projecting out the group direction sets the measured association to zero, but that result is circular, because the projection removes exactly the direction this score measures. Gonen and Goldberg show that, for published debiasing methods, the bias is largely hidden rather than removed: it remains in the distances between supposedly neutralized words and can be recovered from them [19]. Treat a debiasing step as one intervention to be audited on the downstream task, not as a guarantee, and remember that the source of the association is the data.

From word vectors to sentence embeddings

Plain version. An embedding supplies features; an application still needs a target, supervision and an action rule. When the application compares whole sentences, train the sentence representation for the comparison it will support.

For document classification, mean pooling is a convenient first experiment. Build one vector per training document, fit a classifier on its labels, and apply the same tokenizer, embedding lookup and pooling at evaluation time. Compare it with the sparse features and convolutional encoders developed in NLP 03. Keep the data split and classifier tuning allowance fixed so that a change in representation can be assessed fairly.

A frozen table uses contextual statistics learned before the labeled task. Fine-tuning lets the labels adjust the rows. These policies answer different questions: whether reusable corpus features already distinguish the classes, and whether task-specific changes improve them. With limited labels, fine-tuning can also fit incidental identifiers. Holding out groups such as conversations or repeated document templates helps reveal that behavior.

Decide how to handle uncovered text. Mapping every unknown word to one row merges all such words. Dropping them changes the pooling denominator and may leave an empty document. Returning a zero vector does not create a meaningful cosine direction. Subword vectors reduce but do not remove this problem. Coverage, the number of retained tokens, and the action for an empty representation are part of the model specification.

For word-level tagging, retain the sequence of token vectors rather than pooling it into one document vector. A downstream encoder can then combine local context before predicting a label per position. This does not make the original table contextual: the initial lookup of `bank` is still identical in both senses. The context-dependent computation happens afterward.

Similarity and contrastive sentence learning

A representation useful for predicting neighboring words may need adaptation before distances between whole sentences become reliable. Sentence-BERT trains paired sentence representations with a shared encoder and task objectives, making sentence comparisons possible without jointly encoding every candidate pair [20]. The computational distinction matters: separately cached document vectors can support retrieval, whereas a model that reads a query and candidate together must run for each pair.

A generic contrastive objective makes the intended geometry explicit:

$$\mathcal{L}_{\text{pair}} = -\log \frac{\exp(\cos(\mathbf{q}, \mathbf{a}^{+})/\tau)}{\sum_{j=1}^{M} \exp(\cos(\mathbf{q}, \mathbf{a}_j)/\tau)}.$$

Here $\mathcal{L}_{\text{pair}}$ is the loss for one query, $\mathbf{q}$ is its vector, $\mathbf{a}^{+}$ is the vector of a relevant paired text included among the candidates, $\mathbf{a}_j$ is candidate j, M is candidate count and $\tau > 0$ is a temperature controlling the sharpness of the distribution. This is a generic learning construction, not a claim that every sentence encoder uses this exact objective. It has the same shape as the Word2Vec softmax: one observed pair competes with alternatives.

SimCSE is a widely used instance [21]. Its unsupervised version encodes the same sentence twice; because dropout differs between the two passes, the two vectors differ slightly and form a positive pair, while other sentences in the batch are negatives. The authors report that removing dropout leads to representation collapse. Its supervised version uses natural-language-inference data, taking entailment pairs as positives and contradiction pairs as hard negatives [21].

Negative examples define what "different" means. Two tickets about different invoices might be useful negatives for finding the same case but poor negatives for recognizing the billing category. False negatives can push semantically related examples apart. Before changing an encoder, inspect how positive and negative pairs are sampled.

Similarity also differs from entailment. "The parcel arrived" and "The parcel did not arrive" share topic and many tokens. They may be close in a semantic search space while making opposite claims. Classification, semantic search and inference therefore require different validation examples, even when they share an encoder. NLP 11 treats inference as its own task.

Preview: contextual embeddings

Plain version. A static table gives `bank` one vector everywhere. A contextual model computes a different vector for each occurrence, using the surrounding words, so the river sense and the money sense can come apart.

Peters et al. compute ELMo representations as functions of the entire input sentence, using the internal states of a bidirectional language model, and add them as features to task models [22]. BERT takes the step further within its pretrained encoder: a token's final hidden state depends on the other permitted input tokens [23]. The distinction is **lookup versus computation**. Both systems may begin with an embedding table; static Word2Vec exposes a row directly, while BERT processes rows and positions through a context-dependent network.

Context can improve sense discrimination, but it does not turn every hidden-state cosine into a calibrated similarity measure. A static vector is a type-level summary across many occurrences; a contextual vector is a token-level representation of one occurrence. Which one an application needs depends on whether it asks about a word in general or a word as used here. NLP 07 builds the attention mechanism these models use, and NLP 10 develops masked pretraining and BERT's contextual representations rather than repeating them here.

Modern view

Turney and Pantel's survey [6] distinguishes term–document, word–context and pair–pattern matrices. Its taxonomy makes the source of evidence explicit: a document association, a neighboring-word relation and a relational pattern are different observations. The lesson here is to define the matrix before interpreting the vector space.

Apidianaki's 2023 review [24] follows word meaning from static type-level vectors to contextual token-level representations and back. It covers semantic specialization, sense distinctions, representation interpretation and the derivation of static vectors from contextual encoders. Its central distinction is the level at which evidence is aggregated, rather than a ranking in which a newer architecture automatically solves every lexical task. Methods that inject lexical-resource relations into vectors connect directly to the WordNet section of this lesson.

For sentence- and text-level embeddings, Muennighoff et al.'s MTEB benchmark spans 8 embedding tasks, 58 datasets and 112 languages, and its authors find that no single text-embedding method dominates across all tasks [25]. In 2023–2026, most practical text embeddings come from pretrained Transformer encoders or language models fine-tuned with contrastive objectives of the kind shown above, and benchmark suites in the style of MTEB are the usual way to compare them. Static vectors remain useful as fast, transparent baselines, as features in low-resource settings, and as objects of study.

A useful research lens is therefore: **what meaning distinction survives this representation, and how was that checked?** Contextual hidden states, static vectors obtained by aggregating them, and task-trained sentence representations answer different questions. An embedding that preserves a broad topic may still collapse a negated claim or a rare sense. A small static table can

remain an appropriate baseline when its measurable invariances match the application.

Open questions include reliable geometry across domains, sense separation when evidence is sparse, bias that survives debiasing, and comparisons that hold corpus and evaluation conditions fixed. Multilingual work adds segmentation and uneven coverage: a row is only useful if the tokenizer and training corpus supplied evidence for that type. Report those conditions and challenge examples alongside the final score. This keeps the explanation tied to observed language behavior.

Key takeaways

- A vocabulary entry is a form or lemma, not a sense; lexical relations such as synonymy, antonymy and hypernymy hold between senses, and WordNet records them by hand.
- A static embedding aggregates many occurrences into one vector per vocabulary entry; distributional similarity captures shared usage and mixes synonymy, antonymy and relatedness.
- PPMI compares co-occurrence with independence; SVD, Skip-gram with negative sampling and GloVe all compress association statistics under a rank constraint.
- CBOW predicts a center from combined neighbors; Skip-gram predicts neighbors separately; negative sampling distinguishes statistical sources of pairs and is not a normalized distribution.
- fastText builds word vectors from character n-grams, so related spellings share parameters and unseen words get vectors.
- Similarity and analogy tests depend on what the ratings mean and on details such as excluding the question words; evaluate the relation your application needs.
- Embeddings absorb corpus associations, including social biases; projection-based debiasing can hide rather than remove them.
- Mean pooling erases order; sentence embeddings should be trained for the comparison they serve, and contextual models compute a vector per occurrence rather than looking one up.

Exercises

1. **Build the examples.** Enumerate radius-one CBOW and Skip-gram examples for `robots help people today`, using only interior centers. Then include the boundary centers. Which input shapes change?

   Hint

   Count centers separately from center–context pairs. Variable-length CBOW contexts need a mask or separate processing; Skip-gram still receives one center ID per pair.

2. **Inspect a matrix cell.** Construct a corpus in which one context occurs frequently overall but has PMI near zero for a chosen center. Then add observed center–context pairs and explain both the numerator and marginal changes.

Hint

Begin with a joint table close to independence. Recompute all marginals after editing the count; keeping them fixed would analyze a different table.

3. **Train GloVe and watch it wobble.** Implement the GloVe objective in PyTorch on the toy corpus, training only on nonzero cells. Train it with three random seeds and compare the nearest neighbors of `cat`. Then count the parameters and the nonzero cells. What explains the instability, and what would make the problem better determined?

Hint

Two $V \times D$ tables plus two bias vectors can easily outnumber the observed cells. A larger corpus with more varied contexts adds equations; a smaller D removes unknowns.

4. **Similarity versus relatedness.** Write ten word pairs that are related but not similar (such as `cup` and `coffee`) and ten that are similar. Using the PPMI code with window radius 1 and then 4 on a corpus you write, compare how the cosine separates the two groups.

Hint

Wider windows collect topical co-occurrence, which favors relatedness; narrow windows emphasize words that fill the same slot. Report the change for both groups, not a single score.

5. **Expose an invariant.** Create two meaningful sentences with the same word counts and different participant roles. Show why static mean pooling cannot separate them, even if every word vector is learned perfectly for its training objective.

Hint

Changing order does not change a sum. The limitation holds for every possible static embedding table, not only for the synthetic values in this lesson.

6. **Audit beyond the projection.** In the bias example, after projection, compute each occupation word's nearest names using all remaining dimensions, and train a small logistic classifier to predict group A versus group B from the projected occupation-word neighbors. Does the association still leave a trace?

Hint

The projection removed one direction measured one way. Look for structure the score did not measure, as Gonen and Goldberg did with clustering and classification.

References

1. M. Hagiwara, *Real-World Natural Language Processing: Practical Applications with Deep Learning*, Manning, 2021; Chapters 3–4. publisher
2. S. McRoy, *Principles of Natural Language Processing*, open textbook, 2021; Chapter 6, "Benchmark Tasks for Language Modelling." chapter
3. Y. Bengio, R. Ducharme, P. Vincent and C. Jauvin, "A neural probabilistic language model," *Journal of Machine Learning Research* 3, 2003. paper
4. G. A. Miller, "WordNet: a lexical database for English," *Communications of the ACM* 38(11), 1995. paper
5. Z. S. Harris, "Distributional structure," *Word* 10(2–3), 1954. paper
6. P. D. Turney and P. Pantel, "From frequency to meaning: vector space models of semantics," *Journal of Artificial Intelligence Research* 37, 2010. survey
7. T. Mikolov, K. Chen, G. Corrado and J. Dean, "Efficient estimation of word representations in vector space," ICLR Workshop, 2013. paper
8. T. Mikolov, I. Sutskever, K. Chen, G. Corrado and J. Dean, "Distributed representations of words and phrases and their compositionality," *Advances in Neural Information Processing Systems* 26, 2013. paper
9. O. Levy and Y. Goldberg, "Neural word embedding as implicit matrix factorization," *Advances in Neural Information Processing Systems* 27, 2014. paper
10. J. Pennington, R. Socher and C. D. Manning, "GloVe: global vectors for word representation," EMNLP, 2014. paper
11. P. Bojanowski, E. Grave, A. Joulin and T. Mikolov, "Enriching word vectors with subword information," *Transactions of the Association for Computational Linguistics* 5, 2017. paper
12. L. Finkelstein, E. Gabrilovich, Y. Matias, E. Rivlin, Z. Solan, G. Wolfman and E. Ruppin, "Placing search in context: the concept revisited," *ACM Transactions on Information Systems* 20(1), 2002. paper
13. F. Hill, R. Reichart and A. Korhonen, "SimLex-999: evaluating semantic models with (genuine) similarity estimation," *Computational Linguistics* 41(4), 2015. paper
14. T. Mikolov, W.-t. Yih and G. Zweig, "Linguistic regularities in continuous space word representations," NAACL-HLT, 2013. paper
15. T. Linzen, "Issues in evaluating semantic spaces using word analogies," Workshop on Evaluating Vector-Space Representations for NLP (RepEval), 2016. paper
16. T. Schnabel, I. Labutov, D. Mimno and T. Joachims, "Evaluation methods for unsupervised word embeddings," EMNLP, 2015. paper
17. T. Bolukbasi, K.-W. Chang, J. Zou, V. Saligrama and A. Kalai, "Man is to computer programmer as woman is to homemaker? Debiasing word embeddings," *Advances in Neural Information Processing Systems* 29, 2016. paper
18. A. Caliskan, J. J. Bryson and A. Narayanan, "Semantics derived automatically from language corpora contain human-like biases," *Science* 356(6334), 2017. paper
19. H. Gonen and Y. Goldberg, "Lipstick on a pig: debiasing methods cover up systematic gender biases in word embeddings but do not remove them," NAACL-HLT, 2019. paper
20. N. Reimers and I. Gurevych, "Sentence-BERT: sentence embeddings using Siamese BERT-networks," EMNLP-IJCNLP, 2019. paper

21. T. Gao, X. Yao and D. Chen, “SimCSE: simple contrastive learning of sentence embeddings,” EMNLP, 2021. paper
22. M. E. Peters, M. Neumann, M. Iyyer, M. Gardner, C. Clark, K. Lee and L. Zettlemoyer, “Deep contextualized word representations,” NAACL-HLT, 2018. paper
23. J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, “BERT: pre-training of deep bidirectional transformers for language understanding,” NAACL-HLT, 2019. paper
24. M. Apidianaki, “From word types to tokens and back: a survey of approaches to word meaning representation and interpretation,” *Computational Linguistics* 49(2), 2023. survey
25. N. Muennighoff, N. Tazi, L. Magne and N. Reimers, “MTEB: massive text embedding benchmark,” EACL, 2023. paper

NLP 05 • Recurrent Neural Networks for Language

Start from n-gram language models and their memory ceiling, then build, train and evaluate recurrent language models: Elman RNNs, LSTMs and GRUs, character-level modeling, sampling and practical training.

Online edition: https://blog.shyandram.dev/en/learn/nlp/nlp-05-recurrent-neural-networks/

What you'll learn

- Separate an n-gram occurrence, a document feature and a conditional language model, and derive unigram, bigram and trigram predictions from sequence probabilities.
- Count, smooth and interpolate n-gram estimates, then interpret held-out negative log likelihood and perplexity.
- Explain why a fixed-length history creates a memory ceiling that more data cannot remove.
- Write the Elman recurrent language model, its next-token cross-entropy and its perplexity, and explain how backpropagation through time, vanishing gradients, LSTM and GRU cells affect what it can learn.
- Train a character-level LSTM language model with stream packing, truncated backpropagation, clipping and early stopping, and compare it fairly with a tuned n-gram baseline.
- Sample from a trained model with temperature, and say what samples can and cannot tell you.

The big picture

A language model assigns probabilities to text by predicting the next token from the text before it. This chapter builds two such models and compares them under one evaluation contract. The first, an n-gram model, predicts the next token using a small piece of the preceding text. Its probability table is directly inspectable: we can trace a prediction to training counts and see exactly which earlier words the model ignores. This makes it a useful first language model, a controlled baseline, and a way to understand the statistical problems that more elaborate models also face. The second, a recurrent neural network (RNN), replaces the table with a learned state that is updated after every token, so its prediction can in principle depend on the entire prefix.

> **Explain like I'm five**
>
> Imagine a notebook recording what people say next. One page records what follows "cat"; another records what follows "the cat." When you need a guess, you open the matching page and count the examples. Longer page headings give you more context, but many pages may be nearly empty. Smoothing reserves a little room for words that have not yet appeared on that page. A recurrent network works differently: instead of a notebook with a page for every heading, it keeps a short running summary in its head and rewrites that summary after each word it hears.

Hagiwara's language-modeling chapter connects next-token prediction to practical text generation [1]. McRoy's textbook supplies the broader language and application setting [2]. Tokenization belongs to NLP 01; embeddings and their training objectives belong to NLP 04. Sections 1–5 focus on the counting model itself; sections 6–12 replace the count table with a recurrent network, and attention-based models follow in NLP 07 and NLP 09. The general mathematics of recurrent networks (full backpropagation-through-time derivations, gradient bounds, gate equations) is in ML 03A; here we take the language-modeling view. Follow one question through this chapter: **what information and evidence determine the probability of the next token?**

1. Three meanings of "n-gram"

An n-gram is a contiguous sequence of n tokens. The tokens may be words, subwords or characters; the chosen tokenization determines what is counted. In the word-token sequence `a cat sleeps`, the unigrams are `a`, `cat`, `sleeps`; the bigrams are `a cat`, `cat sleeps`; and the trigram is `a cat sleeps`.

Do not confuse these three uses:

Use	Object stored or predicted	What it tells us
N-gram occurrence	A contiguous token tuple	A local piece of a particular sequence
Document feature	The number or presence of a tuple in a document	Evidence supplied to a classifier or retrieval model
N-gram language model	A normalized next-token distribution for each retained history	Probabilities used to score or generate a sequence

A document bigram feature such as `not ready` can help a classifier distinguish an operational status. It does not, by itself, specify the probability of `ready` after `not`: we still need competing next-token counts and a normalization rule. Likewise, the probability of a phrase depends on the sentence context under a language model, while a feature count usually pools all of its occurrences within the document.

In a language model, the order n includes the token being predicted. A bigram uses **one previous token**, not two; a trigram uses two. Sentence boundaries require a consistent convention. A beginning marker supplies initial context, and an end marker is a predicted outcome that represents stopping. The examples never predict the beginning marker. Keeping it in the prediction vocabulary would allocate probability to an event that our sentence format does not allow.

2. Exact factorization, limited history

Predicting one token at a time can represent a sequence probability exactly. Throwing away older context is the modeling assumption, rather than a consequence of probability theory.

$$p(x_1, \dots, x_T) = \prod_{t=1}^{T} p(x_t \mid x_{<t}), \qquad p_n(x_1, \dots, x_T) = \prod_{t=1}^{T} p_n(x_t \mid h_t^{(n)}).$$

Here x_t is the token at position t; T counts predicted positions, including an end marker when used; $x_{<t}$ is the complete prefix; n is the n-gram order; $h_t^{(n)}$ is the suffix containing at most the previous $n-1$ tokens, with beginning markers where needed; and p_n denotes the limited-history model. The first equality is the chain rule. The second defines a model whose predictions depend only on the retained suffix. It need not equal the true sequence distribution.

Order	Context before predicting `sleeps` in `a cat sleeps`	Conditional distribution
Unigram, $n = 1$	Empty history	$p_1(\text{sleeps})$
Bigram, $n = 2$	`cat`	$p_2(\text{sleeps} \mid \text{cat})$
Trigram, $n = 3$	`a cat`	$p_3(\text{sleeps} \mid \text{a cat})$

An empty history is still a valid context: the unigram model uses the same next-token distribution everywhere. A bigram makes finer distinctions, but it must pool all prefixes ending in the same token. A trigram can separate prefixes whose last two tokens differ. None of these guarantees that the retained context contains the information needed by a linguistic dependency.

A worked count comparison

The following invented corpus intentionally makes the article informative: examples beginning with `the` end in `sleeps`, and examples beginning with `a` end in `eats`. This is a controlled statistical pattern, not a claim about English. The example fits three orders independently with the same vocabulary and smoothing strength.

```
from collections import Counter, defaultdict

train = ["the cat sleeps", "the dog sleeps", "a cat eats", "a dog eats"]
START, END, UNK = "<s>", "</s>", "<unk>"
vocab = sorted({w for s in train for w in s.split()} | {END, UNK})
alpha = 0.2
models = {}
for order in (1, 2, 3):
    rows = defaultdict(Counter)
    for sentence in train:
        tokens = [START] * (order - 1) + sentence.split() + [END]
        for t in range(order - 1, len(tokens)):
            history = tuple(tokens[t-order+1:t])
            rows[history][tokens[t]] += 1
    models[order] = rows

def probability(order, history, word):
    row = models[order].get(tuple(history), {})
    return (row.get(word, 0) + alpha) / (sum(row.values()) + alpha * len(vocab))

for order, rows in models.items():
    for history in rows:
        assert abs(sum(probability(order, history, w) for w in vocab) - 1) < 1e-10
print("vocabulary size:", len(vocab))
```

```
print("unigram P(eats):", round(probability(1, (), "eats"), 3))
print("bigram P(eats | cat):", round(probability(2, ("cat",), "eats"), 3))
print("trigram P(eats | a cat):", round(probability(3, ("a", "cat"), "eats"), 3))
print("trigram P(eats | the cat):", round(probability(3, ("the", "cat"), "eats"), 3))
print("unseen-history P(eats):", round(probability(3, (UNK, "cat"), "eats"), 3))
assert probability(3, ("a", "cat"), "eats") > probability(3, ("the", "cat"), "eats")
assert abs(probability(3, (UNK, "cat"), "eats") - 1 / len(vocab)) < 1e-10
```

Output:

```
vocabulary size: 8
unigram P(eats): 0.125
bigram P(eats | cat): 0.333
trigram P(eats | a cat): 0.462
trigram P(eats | the cat): 0.077
unseen-history P(eats): 0.125
```

After `cat`, the bigram count table contains one `eats` and one `sleeps`. Both article choices feed that same table. The trigram tables separate the two observations: `a cat` has one observed `eats`, while `the cat` has none. The trigram can therefore express the distinction, but each of its tables has only one observation. More detailed conditioning gives less evidence per context on this corpus.

The unseen history has no outgoing counts. Additive smoothing makes its distribution uniform over all eight prediction tokens. This is a complete probability distribution, yet it discards useful information that the final token `cat` does occur in training. Interpolation with a lower-order distribution can retain that evidence. Normalization and useful generalization are different properties.

3. From counts to normalized probabilities

To estimate a distribution, count each history–target event in training, then divide by the history's outgoing total. Two details make the result reproducible: which symbols are predicted and which contexts count as distinct.

$$\hat{p}(w \mid h) = \frac{c(h, w)}{c(h)}, \qquad c(h) = \sum_{v \in V} c(h, v), \qquad c(h) > 0.$$

Here h is a retained history; w is a candidate next token; V is the prediction vocabulary; v ranges over its members; $c(h, w)$ is the number of training transitions from h to w; $c(h)$ is their total; and $\hat{p}$ is the maximum-likelihood estimate. For a trigram, h is a two-token tuple; for a unigram, it is the empty tuple. The estimate is undefined when the history has never occurred.

Why does the observed relative frequency solve the likelihood problem? For one history, a distribution maximizing $\sum_w c(h, w) \log p(w \mid h)$ subject to probabilities summing to one has its positive-count probabilities proportional to the counts. The normalizing constraint then divides by $c(h)$.

Different histories can be fitted independently because a count table does not share parameters across them.

This simplicity creates two distinct problems. An **unseen transition** has an observed history but a target never counted after it, producing probability zero. An **unseen history** has no count table at all, making the unsmoothed ratio undefined. Smoothing can handle both within a fixed vocabulary. It cannot produce the spelling of a word that the vocabulary has replaced with an unknown marker.

Additive smoothing adds a small positive count to every possible target:

$$p_\alpha(w \mid h) = \frac{c(h, w) + \alpha}{c(h) + \alpha|V|}.$$

Here p_α is the smoothed distribution; $c(h, w)$ and $c(h)$ are transition and outgoing-history counts; w is the candidate token; h is the retained history; V is the prediction vocabulary; $|V|$ is its size; and $\alpha > 0$ is the amount added to each candidate count. Summing the numerator over the vocabulary gives exactly the denominator. Even an unobserved history has a defined distribution: all candidates receive probability $1/|V|$.

The vocabulary size matters as much as the count added to one candidate. With a large vocabulary, allocating α to every token can absorb substantial total probability. A parameter that works for this tiny word vocabulary should not be transferred unchanged to a much larger one without validation.

Code: train, score and sample a bigram model

This complete example needs no downloads or GPU. The corpus is invented for teaching; its test sentences are separate from training. Every test word absent from the training vocabulary maps to the same unknown token.

```
from collections import Counter, defaultdict
import math
import random

train = [
    "the cat sleeps", "the dog sleeps", "the cat runs",
    "the dog runs", "a cat eats", "a dog eats",
    "the bird sings", "a bird sleeps",
]
test = ["the cat eats", "a dog runs", "the bird sleeps", "a robot runs"]
START, END, UNK = "<s>", "</s>", "<unk>"
words = {w for sentence in train for w in sentence.split()}
vocab = sorted(words | {END, UNK})       # START is context, never predicted
alpha = 0.2
counts = defaultdict(Counter)
unigrams = Counter()

def encode(sentence):
    return [w if w in words else UNK for w in sentence.split()] + [END]
```

```
for sentence in train:
    tokens = [START] + encode(sentence)
    for h, w in zip(tokens, tokens[1:]):
        counts[h][w] += 1
        unigrams[w] += 1

def bigram(h, w):
    row = counts.get(h, {})
    return (row.get(w, 0) + alpha) / (sum(row.values()) + alpha * len(vocab))

def unigram(h, w):
    return (unigrams[w] + alpha) / (sum(unigrams.values()) + alpha * len(vocab))

def perplexity(sentences, probability):
    nll, n = 0.0, 0
    for sentence in sentences:
        tokens = [START] + encode(sentence)
        for h, w in zip(tokens, tokens[1:]):
            nll -= math.log(probability(h, w))
            n += 1
    return math.exp(nll / n)

for h in [START] + vocab:
    assert abs(sum(bigram(h, w) for w in vocab) - 1) < 1e-10
print("unigram test perplexity:", round(perplexity(test, unigram), 3))
print("bigram test perplexity:", round(perplexity(test, bigram), 3))
assert UNK in encode(test[-1])

rng = random.Random(7)
h, sample = START, []
for _ in range(20):                      # cap length for this demonstration
    w = rng.choices(vocab, weights=[bigram(h, v) for v in vocab])[0]
    if w == END:
        break
    sample.append(w)
    h = w
print("sample:", " ".join(sample))
```

Output:

```
unigram test perplexity: 10.382
bigram test perplexity: 3.573
sample: a bird sings
```

The assertions check that every conditional distribution sums to one and that “robot” maps to the unknown token. The bigram model is far less “surprised” by the test sentences than the unigram model, because word order carries information. Sampling can still produce awkward strings: positive probability is not a guarantee of grammaticality. The length cap only stops the demonstration; it does not change the probabilities used for scoring.

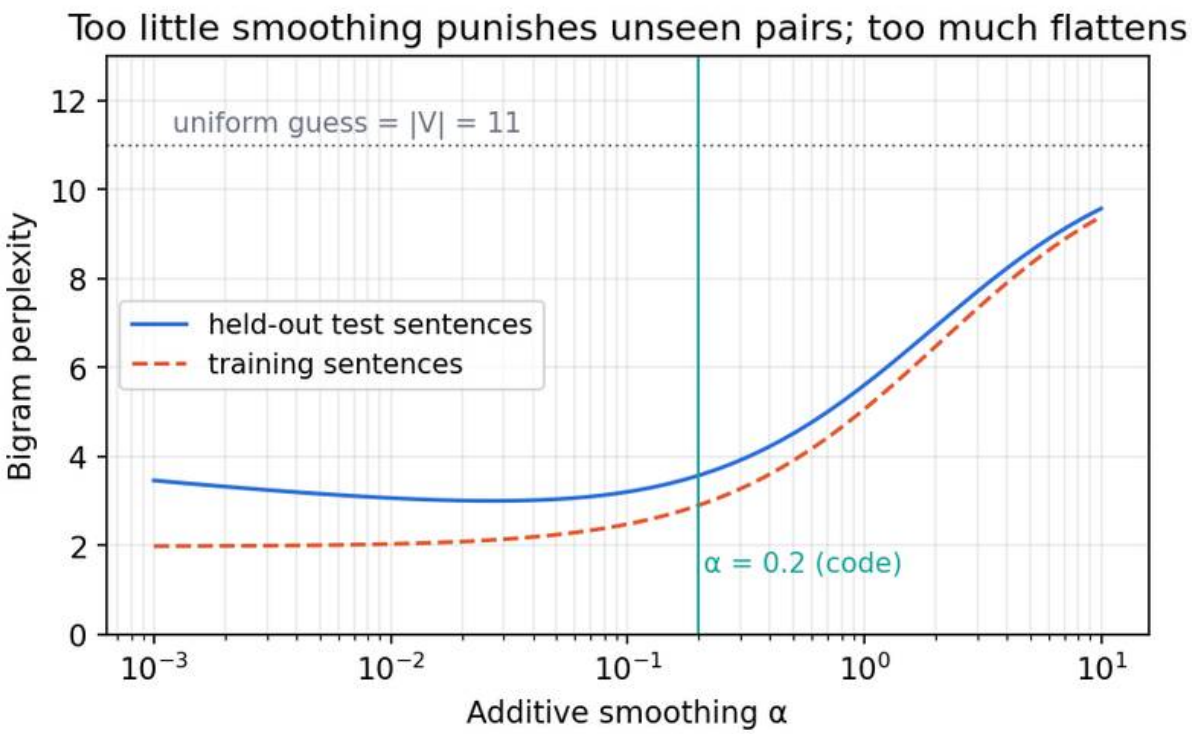


Figure 1 — Smoothing reserves probability for unobserved transitions. The synthetic held-out trace illustrates the tradeoff; four held-out sentences cannot establish a generally optimal setting.

Distinguish smoothing from longer memory

Smoothing addresses missing observations within the chosen representation. It does not restore discarded history. After either "the technician did not repair the" or "the technician repaired the," a bigram sees only `the`. It assigns the same next-token distribution even if the earlier words would make different continuations appropriate. Increasing the corpus can estimate that shared distribution more accurately while leaving the equality intact.

Additive smoothing also has an exact mixture interpretation. For a context with a positive observed count:

$$p_\alpha(w \mid h) = \lambda_h \hat{p}(w \mid h) + (1 - \lambda_h)\frac{1}{|V|}, \qquad \lambda_h = \frac{c(h)}{c(h) + \alpha|V|}, \qquad \hat{p}(w \mid h) = \frac{c(h, w)}{c(h)}.$$

Here p_α is the smoothed distribution, h is the retained history, w is a candidate token, $c(h, w)$ is its observed transition count, $c(h) > 0$ is the total outgoing count, $\hat{p}$ is the unsmoothed estimate, $\alpha > 0$ is the smoothing parameter, $|V|$ is vocabulary size and λ_h is the weight on observations. A context with zero count uses the uniform distribution directly.

The same smoothing parameter changes rare contexts more than frequent ones. As context count grows, the observed evidence receives more weight; as smoothing grows, probability moves toward the uniform distribution. This explains why a held-out curve can improve initially and

then worsen: protecting unseen transitions eventually also dilutes useful observed distinctions. Choose the parameter with validation data rather than the final test set.

A tokenizer and unknown-token rule are part of this experiment. Mapping several unseen words to one marker pools their probabilities. A lower perplexity after that mapping can partly reflect an easier prediction alphabet. It does not show that the model can identify those original words.

Interpolate evidence from several history lengths

When a long context is rare, shorter contexts often have more observations. Interpolation combines their complete distributions instead of assigning every unseen continuation the same probability.

$$p_{\text{mix}}(w \mid u, v) = \lambda_3 p_3(w \mid u, v) + \lambda_2 p_2(w \mid v) + \lambda_1 p_1(w), \qquad \lambda_i \geq 0, \quad \sum_{i=1}^{3} \lambda_i = 1.$$

Here u, v are the two most recent tokens; w is the next token; p_3, p_2, p_1 are normalized trigram, bigram and unigram distributions; p_{mix} is their mixture; and λ_i is the nonnegative weight assigned to order i. Each component must be defined for the queried history, for example by smoothing. Summing over candidate tokens gives one because each component sums to one and the weights also sum to one.

For the unseen context in our small comparison, a mixture can use the observed bigram after `c at` even though the full two-token context is missing. Fixed mixture weights are easy to inspect; history-dependent weights can assign more trust to well-supported longer histories. Choose the weights on validation text. Deciding them after inspecting the final test perplexity makes that score part of model selection.

Interpolation differs from simply substituting a shorter history only when a transition is missing. Such back-off rules must also account for the probability already allocated to observed transitions, so that the complete distribution stays normalized. Chen and Goodman's comparative study examines smoothing choices and their empirical behavior [4]. Our additive and mixture examples expose the mechanics rather than implementing every estimator in that study.

4. What held-out perplexity measures

Scoring measures how much probability a model gives the observed targets. A good score means that these particular token sequences were relatively unsurprising under a particular tokenizer and vocabulary; it does not establish factual accuracy or understanding.

$$\mathcal{L} = -\frac{1}{N} \sum_{j=1}^{N} \log p(w_j \mid h_j), \qquad \text{PPL} = \exp(\mathcal{L}) = \left(\prod_{j=1}^{N} \frac{1}{p(w_j \mid h_j)} \right)^{1/N}.$$

Here N is the number of scored token positions; j indexes those positions across the evaluated sentences; w_j is the observed target; h_j is its permitted history; p is the evaluated model; $\mathcal{L}$ is mean

negative log likelihood in nats per token; and PPL is perplexity. The product form shows that perplexity is the geometric mean of reciprocal target probabilities. It is not the count of plausible continuations at every position.

If every observed target receives probability $1/K$, perplexity equals K, where K is the number of equally likely choices in that hypothetical distribution. Actual predictions are not uniform: one difficult target can contribute a large log loss while many easy targets contribute little. Reporting only an average can conceal concentrated failure on rare names or unfamiliar domains. Inspect losses by word frequency and sentence type as well.

The bigram code counts the end marker, which makes stopping part of both the training problem and the evaluation. It excludes the beginning marker. Its unigram and bigram results are comparable because the test text, tokenization, unknown mapping and number of scored positions match. A comparison across word and character tokenizers has different units. Even the same raw sentence can yield a different score denominator and prediction alphabet.

Train, validation and test text have separate roles. Training determines vocabulary and counts; validation selects order and smoothing; test evaluates the chosen pipeline once. A held-out spelling absent from training must follow the fixed unknown rule. Rebuilding the vocabulary with test text silently changes the experiment. Repeated sentences or material copied across splits can also make memorized local continuations look like generalization.

5. Generation, stopping and the memory ceiling

Generation repeatedly samples from a normalized next-token distribution, appends the sampled token to the history and stops when the end marker appears. Scoring instead supplies the observed history from the text being evaluated. The same probability tables support both procedures, but one examines supplied text while the other produces a stochastic trajectory.

The demonstrated sample `a bird sings` is one draw with a fixed random seed. It is not a representative quality estimate. Examine several independently seeded trajectories and report both local errors and unusually long sequences. A length cap guarantees that a program stops running; if the cap fires before the end marker, the displayed sequence is truncated rather than a model-generated completed sentence.

Choosing the most probable next token at each step gives a deterministic greedy trajectory. It does not generally maximize the probability of the entire sequence: a locally attractive token may lead to poor later continuations. Conversely, sampling preserves distributional variety but may choose low-probability transitions. Generation choices therefore answer a different question from whether the model assigns good probabilities to held-out text.

A failure that more training cannot remove

Consider `the robot near the cabinets is` and `the robots near the cabinet are`. Agreement depends on the head of the subject, rather than simply the immediately preceding noun. If the relevant head lies outside the retained suffix, the model pools examples that require

different agreement choices. More observations can improve the pooled estimate, but cannot make that estimate depend on a token outside the model's input.

For any two prefixes with the same final $n-1$ tokens, an n-gram model's next-token distributions must be identical. This is an architectural equality, not an empirical tendency. Increase n and the ceiling moves farther away, while the number of possible histories and the sparsity of their count tables increase. Shared neural representations can connect similar contexts, but their actual behavior still requires testing; merely replacing a table does not establish that an agreement dependency was learned.

6. A recurrent language model

Plain version. An n-gram model looks the history up in a table. A recurrent language model reads the history one token at a time and keeps a fixed-size vector, the hidden state, that it updates after each token. The next-token distribution is computed from that state, so the model is no longer forced to forget everything older than $n-1$ tokens.

Elman's simple recurrent network introduced this idea of a state fed back as input at the next step [6]; Mikolov and colleagues showed that a recurrent network trained as a word-level language model could beat strong smoothed n-gram baselines for speech recognition [7]. Keep the chain rule from section 2 exactly as it was. Only the conditional distribution changes. For a token sequence $x_1, \dots, x_T$ with a beginning marker x_0:

$$\begin{aligned} \mathbf{h}_t &= \tanh(W_x\,\mathbf{e}(x_{t-1}) + W_h\,\mathbf{h}_{t-1} + \mathbf{b}), \qquad \mathbf{h}_0 = \mathbf{0}, \\ \mathbf{z}_t &= W_o\,\mathbf{h}_t + \mathbf{c}, \\ p_\theta(x_t = v \mid x_{<t}) &= \operatorname{softmax}(\mathbf{z}_t)_v = \frac{\exp z_{t,v}}{\sum_{u \in V} \exp z_{t,u}}. \end{aligned}$$

Here $\mathbf{e}(x) \in \mathbb{R}^d$ is the embedding of token x (a row of a learned table, as in NLP 04); $\mathbf{h}_t \in \mathbb{R}^m$ is the hidden state after reading $x_0, \dots, x_{t-1}$; $W_x \in \mathbb{R}^{m \times d}$, $W_h \in \mathbb{R}^{m \times m}$ and $\mathbf{b} \in \mathbb{R}^m$ are the recurrent parameters; $W_o \in \mathbb{R}^{|V| \times m}$ and $\mathbf{c} \in \mathbb{R}^{|V|}$ map the state to one score $z_{t,v}$ per vocabulary item v; V is the prediction vocabulary and u ranges over it; θ collects all parameters; and tanh acts element by element. This is the same recurrence as in ML 03A, written with an embedding as input and a softmax over the vocabulary as output.

Three consequences follow directly from the equations.

- **Unbounded conditioning, bounded memory.** $\mathbf{h}_t$ is a function of the whole prefix, so two prefixes that share their last $n-1$ tokens can receive different predictions. But $\mathbf{h}_t$ has only m numbers. The model must learn what to keep; nothing guarantees that it keeps the information a particular dependency needs.
- **Shared parameters.** The same W_x, W_h and W_o are used at every position. The parameter count does not grow with context length, whereas the number of possible n-gram histories grows as $|V|^{n-1}$.
- **Generalization through similarity.** Similar tokens receive similar embeddings, and similar prefixes can produce similar states. An unseen history no longer falls back to a uniform row:

it is mapped to some state, and the prediction made from that state can reuse what similar histories taught the model.

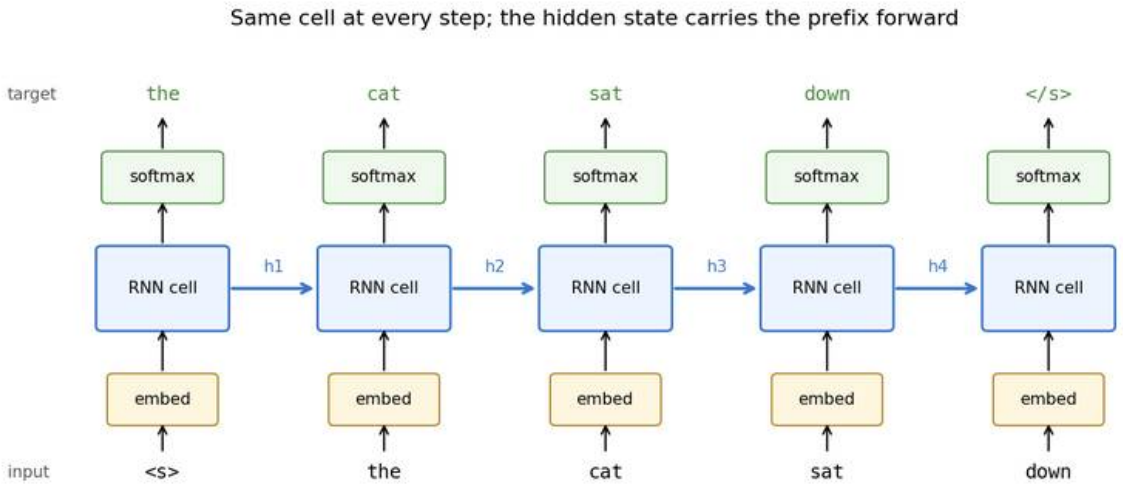


Figure 2 — A recurrent language model unrolled over five steps. Each input is the previous token; each target is the next token; the same cell parameters are reused at every step.

The training loss is next-token cross-entropy

Training minimizes the same quantity that section 4 used for evaluation, now as a differentiable function of θ:

$$\mathcal{L}(\theta) = -\frac{1}{N}\sum_{j=1}^{N} \log p_\theta(x_j \mid x_{<j}), \qquad \text{PPL} = \exp \mathcal{L}, \qquad \text{bits per token} = \frac{\mathcal{L}}{\ln 2}.$$

Here N is the number of supervised target positions (padding and other ignored positions excluded), x_j is the observed target at position j, $x_{<j}$ is its prefix, and $\ln 2$ converts nats to bits. The per-position term is the cross-entropy between the one-hot target and the model's softmax distribution; IT 01 develops cross-entropy and its link to coding length. Character-level work often reports bits per character instead of perplexity; the two carry the same information.

Two implementation details carry over from the count model. First, **shift by one**: the input sequence is $x_0, \ldots, x_{T-1}$ and the target sequence is $x_1, \ldots, x_T$. Feeding a position's own token as both input and target creates a trivial copying shortcut. Second, **teacher forcing**: during training each step receives the observed previous token, not the model's own earlier sample. At generation time, the model's samples become its inputs, so errors can feed on themselves. Training loss and sampling behavior therefore need separate checks, exactly as section 5 argued for the n-gram model.

7. Training through time: what gradients can reach

Plain version. Unrolling the recurrence turns the model into a very deep network with tied weights. Gradients from a prediction must travel back through every intermediate step to reach the token that should have influenced it. Over long distances they tend to shrink to almost nothing or, less often, blow up.

Backpropagation through time (BPTT) applies ordinary backpropagation to the unrolled graph and sums the gradient contributions for each shared weight [8]. The sensitivity of a later state to an earlier one is an ordered product of one-step Jacobians:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_s} = \prod_{k=s+1}^{t} \operatorname{diag}\left(1 - \mathbf{h}_k^2\right) W_h,$$

where $s < t$ are positions, $\mathbf{h}_k^2$ squares each element, and $\operatorname{diag}(\cdot)$ builds a diagonal matrix from the tanh derivatives. When the factors are typically smaller than one in norm, the product decays exponentially with distance $t - s$ (vanishing gradients); when they are larger, it can grow exponentially (exploding gradients). Bengio, Simard and Frasconi analyzed why learning long-term dependencies with gradient descent is hard [9]; Pascanu, Mikolov and Bengio revisited the problem and popularized clipping the gradient norm as the remedy for explosions [10]. ML 03A derives these results in full and measures them.

What does this mean for language? Return to the agreement example from section 5: in `the robots near the cabinet are`, the verb depends on `robots`, four words back, and the intervening noun points the other way. A plain RNN *can* represent a state that remembers the plural subject, but the training signal that would teach it to do so must survive the trip from `are` back to `robots`. At the word level that is a few steps; at the character level the same dependency spans roughly twenty steps. A model can therefore have enough capacity for a dependency and still fail to learn it, which is a different failure from the n-gram's architectural ceiling.

8. Gated cells: LSTM and GRU

Plain version. Gated cells add a memory path that is updated mainly by addition and controlled by learned switches. Information, and gradient, can then pass across many steps without being squashed at every one.

The long short-term memory (LSTM) cell of Hochreiter and Schmidhuber [11], with the forget gate added by Gers, Schmidhuber and Cummins [12], keeps a cell state $\mathbf{c}_t$ next to the hidden state:

$$\mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t, \qquad \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t).$$

Here $\mathbf{f}_t$, $\mathbf{i}_t$ and $\mathbf{o}_t$ are the forget, input and output gates (vectors in $(0,1)^m$ computed by sigmoid layers from the current input and $\mathbf{h}_{t-1}$), $\tilde{\mathbf{c}}_t$ is a tanh candidate computed the same way, and $\odot$ is element-wise multiplication. Along the cell path, $\partial \mathbf{c}_t / \partial \mathbf{c}_{t-1}$ is approximately $\operatorname{diag}(\mathbf{f}_t)$: no weight matrix, no squashing. When a forget gate stays near one, a value written at the plural subject can survive until the verb.

The gated recurrent unit (GRU) of Cho and colleagues [13] merges cell and hidden state and uses two gates:

$$\mathbf{h}_t = \mathbf{z}_t \odot \mathbf{h}_{t-1} + (1 - \mathbf{z}_t) \odot \tilde{\mathbf{h}}_t,$$

where $\mathbf{z}_t \in (0,1)^m$ is the update gate and $\tilde{\mathbf{h}}_t$ is a candidate state computed with a reset gate. (Some sources swap the roles of $\mathbf{z}_t$ and $1 - \mathbf{z}_t$.) The full gate equations, a from-scratch LSTM step checked against PyTorch, and parameter counts are in ML 03A. For language modeling the practical summary is: LSTMs and GRUs train far more reliably than plain tanh RNNs, neither dominates the other across tasks, and gating eases rather than removes the difficulty of very long dependencies.

9. Stacked and bidirectional recurrent networks

Plain version. Recurrent layers can be stacked so that a higher layer reads the state sequence of the layer below. Running a second network right to left gives each position a view of its future too, which is useful for labeling text but forbidden for predicting the next token.

A **stacked** (deep) RNN feeds $\mathbf{h}_1^{(\ell)}, \dots, \mathbf{h}_T^{(\ell)}$ from layer ℓ as the input sequence of layer $\ell + 1$, and the softmax reads the top layer. Two to four layers are common in recurrent language models; dropout between layers is the usual regularizer [16].

A **bidirectional** RNN runs one network forward and one backward and concatenates their states, $\mathbf{h}_t = [\overrightarrow{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t]$ [14]. The backward state at position t has read $x_{t+1}, \dots, x_T$. That is ideal for tagging, where the whole sentence is given and the label of a word may depend on words to its right; it is the main encoder of NLP 06. It is *wrong* for a left-to-right language model: if the state used to predict x_t has seen x_t, the loss rewards copying and the perplexity becomes meaningless. A useful sanity check for any language model is that changing a future token leaves every earlier prediction unchanged.

10. Character-level language models

Plain version. The prediction unit can be a character instead of a word. The vocabulary becomes tiny and no spelling is ever unknown, but sequences become several times longer and the model must learn spelling before it can learn grammar.

Character-level RNNs were popular demonstrations of what recurrent networks learn; Graves showed that LSTMs trained on characters can generate long, locally coherent text and handwriting [15]. The tradeoffs compared with word or subword units are concrete:

Choice	Vocabulary	Unknown inputs	Sequence length	Long-range cost
Words	Large; rare words need an unknown marker	Mapped to one marker; spelling lost	Shortest	Fewest steps between related words

Choice	Vocabulary	Unknown inputs	Sequence length	Long-range cost
Subwords (NLP 01)	Medium, fixed by the tokenizer	Split into known pieces	Medium	Medium
Characters	Tiny alphabet	Any spelling over the alphabet	Longest	Many steps between related words

Two cautions follow from the evaluation rules in section 4. First, character perplexity and word perplexity have different units; a character model with perplexity 11 is not "better" than a word model with perplexity 3.6, because each counts a different number of predictions over a different alphabet. To compare across units, convert both to total negative log likelihood (or bits) for the same raw text. Second, a fixed character alphabet still needs a policy for symbols outside it, such as accented letters or emoji. Decide it before looking at held-out text.

11. Sampling and temperature

Plain version. A trained model gives a probability for every next token. To generate, we draw one, append it, and repeat. A temperature setting makes the draw more cautious or more adventurous without retraining.

For logits $\mathbf{z}_t$ and temperature $\tau > 0$, sample from

$$q_\tau(v) = \frac{\exp(z_{t,v}/\tau)}{\sum_{u \in V} \exp(z_{t,u}/\tau)}.$$

Here v and u range over the vocabulary V, $z_{t,v}$ is the model's score for v at step t, and q_τ is the sampling distribution. $\tau = 1$ samples from the model's own distribution; $\tau \to 0$ approaches greedy selection of the highest-scoring token; $\tau > 1$ flattens the distribution and gives more weight to unlikely tokens. Temperature changes only the generation procedure. It does not change the probabilities used to compute perplexity, so a model's held-out score is the same at every temperature.

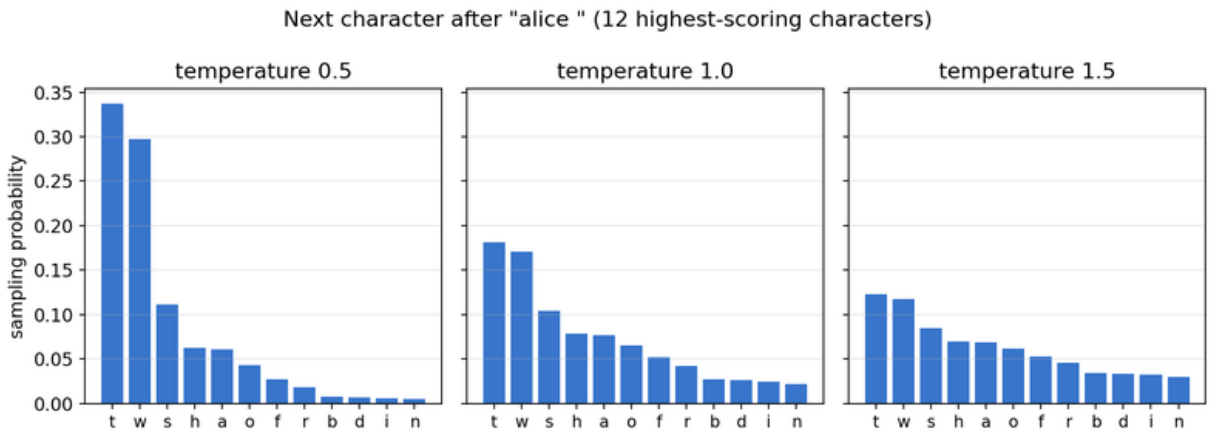


Figure 3 — Next-character distribution of the trained LSTM from section 12 after the prompt "alice", at three temperatures. The logits are identical in all three panels; only the sampling distribution changes.

Generation adds stopping rules, length limits and search strategies such as beam search and nucleus sampling. Those belong to NLP 08 and NLP 09. One warning applies already: a sample is a single random trajectory. Judge a model by held-out likelihood and task checks, and use samples to look for qualitative failure patterns, not to rank models.

12. Practical training, then a complete experiment

The training checklist

Plain version. Most failed recurrent language models fail for mundane reasons: padding counted as text, gradients that explode, states leaking between unrelated documents, or a model selected on the test set. Each has a standard remedy.

- **Packing a corpus into streams.** For a continuous text, cut the token stream into B equal parallel streams and lay them out as a $B \times n$ matrix. Each training step reads the next T columns. Every position is a real token, so no padding is needed. Stream b continues exactly where it stopped in the previous window.
- **Padding and masking.** When examples are separate sentences of different lengths, pad them to a common length and keep a Boolean mask. The mask must remove padded positions from the loss (otherwise the model is rewarded for predicting padding) and, for bidirectional encoders, from the state updates; PyTorch's `pack_padded_sequence` does the latter. NLP 06 implements both in a tagger.
- **Truncated BPTT.** Backpropagating through thousands of steps costs memory linear in the length. Instead, carry the hidden state from one window of T steps to the next but *detach* it, so the forward pass still sees the whole past while gradients flow back at most T steps.
- **Gradient clipping.** If the gradient norm $\|\mathbf{g}\|$ exceeds a threshold κ, rescale it to $\kappa\,\mathbf{g}/\|\mathbf{g}\|$ [10]. This keeps the update direction and caps its length.
- **Regularization and early stopping.** Dropout on non-recurrent connections [16] and weight decay limit memorization; select the epoch with the lowest *validation* perplexity, then report test perplexity once.
- **Fair baselines.** Melis, Dyer and Blunsom found that carefully tuned standard LSTMs outperformed several more elaborate recent architectures once all were given comparable hyperparameter search [17]. The same principle applies at small scale: tune the n-gram baseline on the same validation data before claiming that the neural model wins.

Code: a character-level LSTM against an interpolated n-gram

The text is the opening of Chapter I of Lewis Carroll's *Alice's Adventures in Wonderland* (1865), which is in the public domain; it was copied from Project Gutenberg eBook #11 [24] with italic underscores removed, curly quotes straightened and em dashes written as `--`, then lowercased. Paragraphs 1–7 are training data, paragraph 8 is validation and paragraph 9 is the test set. The

alphabet is fixed in advance, before the held-out text is inspected. Both models condition each held-out character on all earlier text. The n-gram baseline interpolates smoothed unigram, bigram and trigram character distributions exactly as in section 3, with weights and smoothing chosen on validation. The LSTM is trained three times with different seeds, because a single run on so little data can mislead. The whole script runs on a CPU in well under a minute and downloads nothing.

```
import copy
import math
from collections import Counter, defaultdict

import torch
from torch import nn

# Opening paragraphs of Chapter I of Lewis Carroll's "Alice's Adventures in
# Wonderland" (1865, public domain; Project Gutenberg eBook #11). Italic
# underscores removed, curly quotes straightened, em dashes written as "--".
PARAGRAPHS = [
    'Alice was beginning to get very tired of sitting by her sister on the '
    'bank, and of having nothing to do: once or twice she had peeped into the '
    'book her sister was reading, but it had no pictures or conversations in '
    'it, "and what is the use of a book," thought Alice "without pictures or '
    'conversations?"',
    'So she was considering in her own mind (as well as she could, for the hot '
    'day made her feel very sleepy and stupid), whether the pleasure of making '
    'a daisy-chain would be worth the trouble of getting up and picking the '
    'daisies, when suddenly a White Rabbit with pink eyes ran close by her.',
    'There was nothing so very remarkable in that; nor did Alice think it so '
    'very much out of the way to hear the Rabbit say to itself, "Oh dear! Oh '
    'dear! I shall be late!" (when she thought it over afterwards, it occurred '
    'to her that she ought to have wondered at this, but at the time it all '
    'seemed quite natural); but when the Rabbit actually took a watch out of '
    'its waistcoat-pocket, and looked at it, and then hurried on, Alice started'
    ' to her feet, for it flashed across her mind that she had never before '
    'seen a rabbit with either a waistcoat-pocket, or a watch to take out of '
    'it, and burning with curiosity, she ran across the field after it, and '
    'fortunately was just in time to see it pop down a large rabbit-hole under '
    'the hedge.',
    'In another moment down went Alice after it, never once considering how in '
    'the world she was to get out again.',
    'The rabbit-hole went straight on like a tunnel for some way, and then '
    'dipped suddenly down, so suddenly that Alice had not a moment to think '
    'about stopping herself before she found herself falling down a very deep '
    'well.',
    'Either the well was very deep, or she fell very slowly, for she had plenty'
    ' of time as she went down to look about her and to wonder what was going '
    'to happen next. First, she tried to look down and make out what she was '
    'coming to, but it was too dark to see anything; then she looked at the '
    'sides of the well, and noticed that they were filled with cupboards and '
    'book-shelves; here and there she saw maps and pictures hung upon pegs. She'
    ' took down a jar from one of the shelves as she passed; it was labelled '
    '"ORANGE MARMALADE", but to her great disappointment it was empty: she did '
    'not like to drop the jar for fear of killing somebody underneath, so '
```

```
    'managed to put it into one of the cupboards as she fell past it.',
    '"Well!" thought Alice to herself, "after such a fall as this, I shall '
    "think nothing of tumbling down stairs! How brave they'll all think me at "
    "home! Why, I wouldn't say anything about it, even if I fell off the top of"
    ' the house!" (Which was very likely true.)',
    'Down, down, down. Would the fall never come to an end? "I wonder how many '
    'miles I\'ve fallen by this time?" she said aloud. "I must be getting '
    'somewhere near the centre of the earth. Let me see: that would be four '
    'thousand miles down, I think--" (for, you see, Alice had learnt several '
    'things of this sort in her lessons in the schoolroom, and though this was '
    'not a very good opportunity for showing off her knowledge, as there was no'
    ' one to listen to her, still it was good practice to say it over) "--yes, '
    "that's about the right distance--but then I wonder what Latitude or "
    'Longitude I\'ve got to?" (Alice had no idea what Latitude was, or Longitude'
    ' either, but thought they were nice grand words to say.)',
    'Presently she began again. "I wonder if I shall fall right through the '
    "earth! How funny it'll seem to come out among the people that walk with "
    'their heads downward! The Antipathies, I think--" (she was rather glad '
    "there was no one listening, this time, as it didn't sound at all the right"
    ' word) "--but I shall have to ask them what the name of the country is, '
    'you know. Please, Ma\'am, is this New Zealand or Australia?" (and she tried'
    " to curtsey as she spoke--fancy curtseying as you're falling through the "
    'air! Do you think you could manage it?) "And what an ignorant little girl '
    "she'll think me for asking! No, it'll never do to ask: perhaps I shall see"
    ' it written up somewhere."',
]

paras = [p.lower() for p in PARAGRAPHS]
train = "\n".join(paras[:7]) + "\n"     # paragraphs 1-7: fit parameters
valid = paras[7] + "\n"                 # paragraph 8: choose settings
test = paras[8] + "\n"                  # paragraph 9: report once
ALPHABET = "\n !\"'(),-.:;?abcdefghijklmnopqrstuvwxyz"  # fixed in advance
V = len(ALPHABET)
assert set(train + valid + test) <= set(ALPHABET)
print("characters train/valid/test:", len(train), len(valid), len(test))
print("letters never seen in training:", sorted(set(valid + test) - set(train)))

# ---- Baseline: interpolated character n-grams (orders 1-3) ----
def count(order):
    rows = defaultdict(Counter)
    for t in range(len(train)):
        rows[train[max(0, t - order + 1):t]][train[t]] += 1
    return rows

tables = {k: count(k) for k in (1, 2, 3)}

def p_add(k, history, ch, alpha):
    row = tables[k].get(history[len(history) - k + 1:] if k > 1 else "", {})
    return (row.get(ch, 0) + alpha) / (sum(row.values()) + alpha * V)

def ngram_ppl(prefix, target, lams, alpha):
    full, nll = prefix + target, 0.0
    for pos in range(len(prefix), len(full)):
        h = full[max(0, pos - 2):pos]
```

```
        p = sum(l * p_add(k, h, full[pos], alpha) for k, l in zip((1, 2, 3), lams))
        nll -= math.log(p)
    return math.exp(nll / len(target))

grid = [(l1, l2, round(1 - l1 - l2, 2)) for l1 in (0.1, 0.2, 0.3)
        for l2 in (0.2, 0.3, 0.4, 0.5) if l1 + l2 < 1]
best_cfg = min(((ngram_ppl(train, valid, lams, a), lams, a) for lams in grid
                for a in (0.01, 0.1, 0.5)), key=lambda r: r[0])
ng_valid, ng_lams, ng_alpha = best_cfg
ng_test = ngram_ppl(train + valid, test, ng_lams, ng_alpha)
print(f"n-gram  weights={ng_lams} alpha={ng_alpha}  "
      f"valid PPL={ng_valid:.2f}  test PPL={ng_test:.2f}")

tri_alpha = min((0.01, 0.1, 0.5), key=lambda a: ngram_ppl(train, valid, (0, 0, 1), a))
tri_test = ngram_ppl(train + valid, test, (0, 0, 1), tri_alpha)
print(f"trigram only  alpha={tri_alpha}  test PPL={tri_test:.2f}")

# ---- Character-level LSTM language model ----
stoi = {c: i for i, c in enumerate(ALPHABET)}
encode = lambda s: torch.tensor([stoi[c] for c in s])

class CharLSTM(nn.Module):
    def __init__(self, vocab, emb=24, hidden=96, dropout=0.3):
        super().__init__()
        self.emb = nn.Embedding(vocab, emb)
        self.rnn = nn.LSTM(emb, hidden, batch_first=True)
        self.drop = nn.Dropout(dropout)
        self.out = nn.Linear(hidden, vocab)

    def forward(self, x, state=None):
        h, state = self.rnn(self.emb(x), state)
        return self.out(self.drop(h)), state

def lstm_ppl(model, prefix, target):
    model.eval()
    with torch.no_grad():
        ids = encode(prefix + target).unsqueeze(0)
        logits, _ = model(ids[:, :-1])          # condition on all earlier text
        nll = nn.functional.cross_entropy(logits[0, len(prefix) - 1:],
                                          ids[0, len(prefix):])
    model.train()
    return math.exp(nll.item())

def train_lstm(seed, epochs=80, B=16, T=48):
    torch.manual_seed(seed)
    model = CharLSTM(V)
    opt = torch.optim.Adam(model.parameters(), lr=3e-3, weight_decay=1e-4)
    ids = encode(train)
    n = (len(ids) - 1) // B                     # B parallel streams of n characters
    X, Y = ids[:n * B].view(B, n), ids[1:n * B + 1].view(B, n)  # targets shifted by one
    history, best = [], (float("inf"), None, 0)
    for epoch in range(1, epochs + 1):
        state, total, steps = None, 0.0, 0
        for i in range(0, n, T):
```

```
            if state is not None:               # truncated BPTT: keep values, cut graph
                state = tuple(s.detach() for s in state)
            logits, state = model(X[:, i:i + T], state)
            loss = nn.functional.cross_entropy(logits.reshape(-1, V),
                                               Y[:, i:i + T].reshape(-1))
            opt.zero_grad()
            loss.backward()
            nn.utils.clip_grad_norm_(model.parameters(), 1.0)
            opt.step()
            total, steps = total + loss.item(), steps + 1
        v = lstm_ppl(model, train, valid)
        history.append((epoch, math.exp(total / steps), v))
        if v < best[0]:                         # early stopping on validation only
            best = (v, copy.deepcopy(model.state_dict()), epoch)
    model.load_state_dict(best[1])
    return model, best, history

runs = {}
for seed in (0, 1, 2):
    m, best, hist = train_lstm(seed)
    runs[seed] = (m, hist, lstm_ppl(m, train + valid, test))
    print(f"LSTM seed={seed}  best epoch={best[2]}  valid PPL={best[0]:.2f}  "
          f"test PPL={runs[seed][2]:.2f}")
mean_test = sum(r[2] for r in runs.values()) / len(runs)
print(f"LSTM mean test PPL over 3 seeds: {mean_test:.2f}")
assert mean_test < tri_test
model = runs[0][0]

# ---- Sampling at several temperatures ----
def sample(prompt, tau, length=80, seed=1):
    g = torch.Generator().manual_seed(seed)
    model.eval()
    with torch.no_grad():
        logits, state = model(encode(prompt).unsqueeze(0))
        out = prompt
        for _ in range(length):
            probs = torch.softmax(logits[0, -1] / tau, dim=-1)
            nxt = torch.multinomial(probs, 1, generator=g)
            out += ALPHABET[nxt.item()]
            logits, state = model(nxt.view(1, 1), state)
    return out.replace("\n", " / ")

for tau in (0.5, 1.0, 1.5):
    print(f"tau={tau}:", sample("alice ", tau))

# ---- Same last two characters, different earlier context ----
def next_dist(prefix):
    model.eval()
    with torch.no_grad():
        logits, _ = model(encode(prefix).unsqueeze(0))
    return torch.softmax(logits[0, -1], dim=-1)

a, b = "alice was beginning to get ", "the rabbit-hole went "
assert a[-2:] == b[-2:]                     # a trigram sees only these two
```

```
def ngram_dist(prefix):
    return torch.tensor([sum(l * p_add(k, prefix[-2:], c, ng_alpha)
                             for k, l in zip((1, 2, 3), ng_lams)) for c in ALPHABET])

tv = lambda p, q: 0.5 * (p - q).abs().sum().item()
print("total variation, n-gram:", round(tv(ngram_dist(a), ngram_dist(b)), 3),
      " LSTM:", round(tv(next_dist(a), next_dist(b)), 3))
```

Output:

```
characters train/valid/test: 2633 706 682
letters never seen in training: ['z']
n-gram  weights=(0.1, 0.4, 0.5) alpha=0.01  valid PPL=9.70  test PPL=11.24
trigram only  alpha=0.1  test PPL=13.37
LSTM seed=0  best epoch=43  valid PPL=10.13  test PPL=12.09
LSTM seed=1  best epoch=41  valid PPL=9.76  test PPL=11.05
LSTM seed=2  best epoch=39  valid PPL=10.09  test PPL=11.30
LSTM mean test PPL over 3 seeds: 11.48
tau=0.5: alice whas she sor of the to her fear the she was she sely and it she save of the
 ↪  was
tau=1.0: alice whas nyt ook "hente tome urabuint, mubbeit-insiwselly bulith ver to welf was
 ↪  bor
tau=1.5: alice wherany fowh ".pvenst gime feurdt, )ubger and iwselly ?hont  ver, vermef
 ↪  wamen r
total variation, n-gram: 0.0  LSTM: 0.197
```

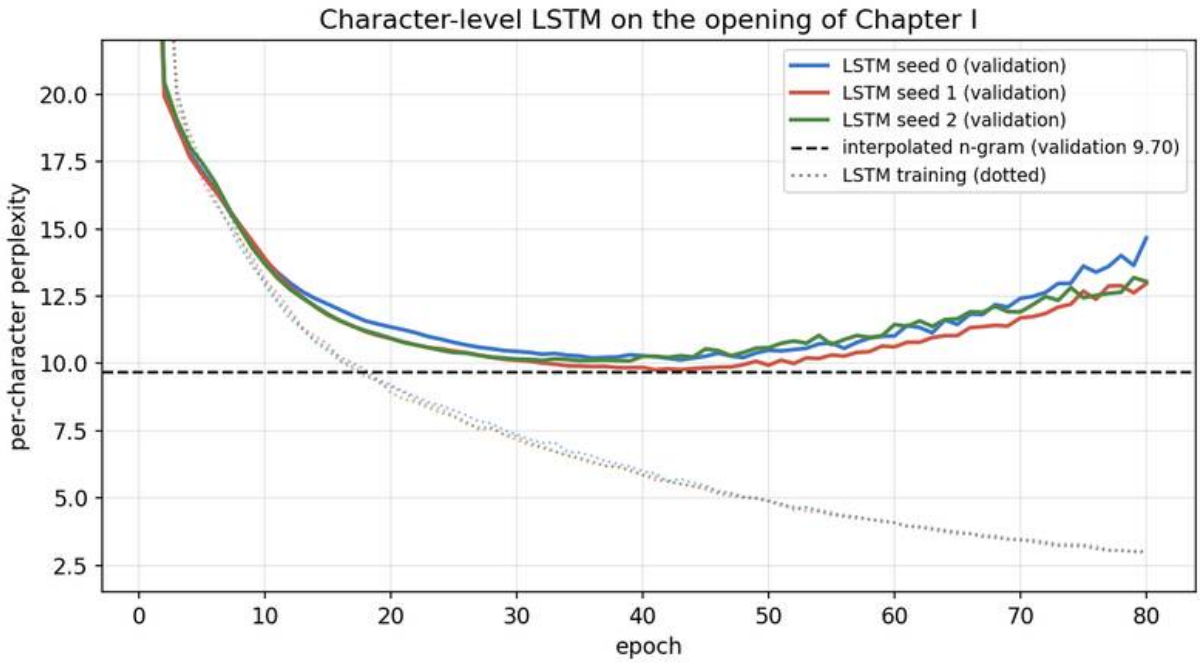


Figure 4 — Per-character perplexity during training. Training perplexity keeps falling; validation perplexity reaches its minimum around epoch 40 and then rises as the model memorizes the 2,633 training characters. The dashed line is the tuned interpolated n-gram on the same validation paragraph.

Reading the results

The baseline matters. A single trigram with additive smoothing reaches a test perplexity of 13.37, and every LSTM run beats it. The interpolated n-gram, tuned on the same validation paragraph, reaches 11.24, while the three LSTM runs reach 12.09, 11.05 and 11.30 (mean 11.48). On 2,633 training characters the recurrent model is roughly level with a well-smoothed count model and clearly better than a poorly smoothed one. This is the small-data version of the point made by Melis and colleagues [17]: an improvement claimed against an untuned baseline may vanish against a tuned one. Neural language models pull ahead as data grows, because shared representations can exploit many weak regularities that sparse tables cannot pool; Jozefowicz and colleagues document how far recurrent language models improved with large corpora and models [19]. Our experiment is far too small to show that regime, and it does not need to: it shows how to make the comparison honestly.

Seeds matter. The spread across seeds (11.05 to 12.09) is larger than the gap between the mean LSTM and the n-gram. With one seed, we could have reported either "the LSTM wins" or "the LSTM loses". Report several runs, or a confidence interval over held-out documents, whenever differences are this small.

The validation curve shows memorization. In Figure 4, training perplexity keeps falling far below the validation value. The model is learning the specific training paragraphs. Early stopping picks epochs 39–43; without it, the final epoch would be considerably worse on held-out text.

Units. These are per-character perplexities over a 39-symbol alphabet; $\log_2 11.24 \approx 3.49$ bits per character for the n-gram. They cannot be compared with the word-level values of section 3 (for example 3.573), which count different events over a different vocabulary.

The memory ceiling, measured. The final lines compare next-character distributions after `alic e was beginning to get` and `the rabbit-hole went`, two prefixes whose last two characters agree. The interpolated trigram must give identical distributions (total variation 0.0), exactly as section 5 proved. The LSTM's distributions differ (total variation 0.197), because its state depends on the earlier characters. This confirms that the recurrent model *uses* longer context. Whether it uses it *well* is a separate question, which held-out perplexity and targeted tests answer.

Samples. At $\tau = 0.5$ the sample repeats frequent fragments such as `she was` and `of the`; at $\tau = 1.0$ word shapes appear with many misspellings; at $\tau = 1.5$ the output breaks into rare punctuation and random letters. A character model trained on fewer than three thousand characters has learned frequent letter sequences, spaces and some short words. It has not learned English. These are single draws with one seed, so read them as illustrations of the temperature effect rather than as quality estimates.

Modern view

Survey perspective. Jurafsky and Martin's August 2026 draft retains a dedicated n-gram chapter, organized around probabilistic prediction, estimation, evaluation, generation and smoothing [3]. It is a useful conceptual bridge: readers can inspect local counts before encountering models that share learned representations. Chen and Goodman's smoothing comparison explains why the estimator matters in addition to the history length [4]. These sources serve different purposes: the textbook develops the model, while the empirical study compares ways to allocate probability with sparse evidence. Tiezzi and colleagues' 2024 survey reviews why recurrent processing has returned in the Transformer era [20]. It frames two trends under one umbrella of recurrence: models that mix Transformer and recurrent ideas to avoid the quadratic cost of self-attention, and deep state-space models that act as a special class of linear RNNs. Its main open problem is learning from streams whose length is not known in advance, which connects sequence modeling to lifelong online learning.

What recurrent language models achieved. From Mikolov's RNN language model [7] to large-scale LSTM language models [19] and carefully regularized LSTMs [18], recurrent networks were the strongest general-purpose language models until Transformers replaced them for most large-scale work. Merity and colleagues' AWD-LSTM study shows how much of that strength came from regularization and optimization choices rather than from new cell designs [18]; Melis and colleagues reached a similar conclusion through controlled hyperparameter search [17].

2023–2026. Recurrence has returned in new forms aimed at the cost of attention over long contexts. RWKV reformulates a Transformer-like model so it can be trained in parallel and run as an RNN at inference [21]. Mamba uses input-dependent (selective) state-space updates computed with a parallel scan [22]. xLSTM revisits the LSTM with exponential gating and new memory structures [23]. These models keep the property that made RNN language models attractive: a fixed-size state, so generation cost per token does not grow with context length. They also inherit the question raised in section 7: a fixed-size state must decide what to forget. ML 03A covers the state-space side in more detail.

Evaluation. Chang and colleagues' 2024 survey organizes large-language-model evaluation around what, where and how to evaluate [5]. It covers task behavior and evaluation methods beyond token prediction. Applied here, a lower perplexity should be accompanied by the task checks that matter: spelling recovery, agreement, meaning preservation or an appropriate stopping rule. The practical role of an n-gram baseline in a 2023–2026 workflow is to make a question measurable. Does a more elaborate model improve held-out predictions under the same tokenization? Does it distinguish two prefixes that the chosen n-gram history must merge? Does its added memory and latency improve the downstream result? Answer with matched experiments rather than comparing unmatched scores or assuming that a longer context is always used effectively.

Open problems. Open problems include sparse evidence after domain changes, probabilities for new names, and interactions between tokenization and measured difficulty. Character or subword tokens change the tradeoff between vocabulary coverage and sequence length. Longer histories, whether held in a table or in a recurrent state, offer more information but create harder

estimation and credit-assignment problems. Choose the prediction unit and retained history together, and state their limits before interpreting a result.

Key takeaways

- An n-gram occurrence or document feature is not automatically a normalized language model; order n uses $n-1$ preceding tokens, while the chain rule itself does not require limited memory.
- Smoothing protects unobserved events, and interpolation reuses shorter-history evidence; neither can recover history that the model discarded.
- Perplexity comparisons need matching prediction units, unknown mapping and boundary conventions; character and word perplexities are not comparable.
- A recurrent language model replaces the count table with a learned state that depends on the whole prefix, with parameters shared across positions.
- BPTT gradients shrink or grow with distance; clipping handles explosions, and LSTM or GRU gating eases vanishing gradients without removing the difficulty of long dependencies.
- Bidirectional RNNs suit labeling, not left-to-right language modeling.
- Practical training needs shifted targets, masks or packed streams, truncated BPTT, clipping, regularization and validation-based early stopping.
- On tiny data, a tuned interpolated n-gram can match an LSTM; compare against tuned baselines and report several seeds.

Exercises

1. Reproduce the smoothed trigram probabilities after `a cat` and `the cat` by hand in the first code example of section 2. Check the sum over all eight target tokens.

   Hint Each history has one observation. The denominator is one plus the smoothing strength multiplied by eight; only the observed continuation receives an additional count.

2. Fit a mixture of the first example's three orders. Choose any nonnegative weights summing to one, and verify normalization for an unseen two-token history.

   Hint Evaluate every component on the same candidate vocabulary. Compare the probability after the unseen history with the pure uniform fallback and the observed bigram after `cat`.

3. Score the bigram test set both including and excluding end markers. Explain why the resulting perplexities answer different prediction questions.

   Hint Change both the accumulated loss and the position count consistently. Excluding end markers removes the evaluation of stopping; it is not merely changing a display convention.

4. Construct two prefixes with identical last two tokens whose appropriate next-token probabilities should differ because of earlier negation, agreement or discourse. State what a trigram can and cannot learn about them, and what an RNN language model could learn about them in principle.

   Hint First confirm that the retained history is exactly equal. Then separate a lack of data from the impossibility of conditioning on the omitted distinction. For the RNN, distinguish what its state can represent from what its gradients can teach it.

5. In the character-level experiment, replace `nn.LSTM` with `nn.GRU` (the state becomes a single tensor, so adjust the detach line) and with `nn.RNN`. Compare validation and test perplexity over three seeds. Then remove gradient clipping from the plain RNN and record what happens to the training loss.

   Hint `nn.GRU` and `nn.RNN` return a tensor rather than a tuple as their state. Keep every other setting fixed, including the early-stopping rule, so that the cell type is the only change.

6. Deliberately break causality: give the LSTM model an extra backward LSTM over the same input (a bidirectional layer) and retrain. Explain the training and validation perplexities you observe, and write a test that would have caught the bug.

   Hint With a backward pass, the state used to predict the character at position t has already read that character. A good test changes one future character and checks that all earlier predicted distributions are unchanged.

References

1. Masato Hagiwara, *Real-World Natural Language Processing: Practical Applications with Deep Learning*, Chapter 5, Manning, 2021. Publisher
2. Susan McRoy, *Principles of Natural Language Processing*, 2021. Chapter
3. Daniel Jurafsky and James H. Martin, *Speech and Language Processing*, third-edition draft, Chapter 3, August 19, 2026. Chapter
4. Stanley F. Chen and Joshua Goodman, "An Empirical Study of Smoothing Techniques for Language Modeling," ACL, 1996. Paper
5. Yupeng Chang et al., "A Survey on Evaluation of Large Language Models," ACM Transactions on Intelligent Systems and Technology, 2024. Paper
6. Jeffrey L. Elman, "Finding Structure in Time," *Cognitive Science* 14(2), 1990. DOI
7. Tomáš Mikolov, Martin Karafiát, Lukáš Burget, Jan Černocký and Sanjeev Khudanpur, "Recurrent Neural Network Based Language Model," Interspeech, 2010. DOI
8. Paul J. Werbos, "Backpropagation Through Time: What It Does and How to Do It," *Proceedings of the IEEE* 78(10), 1990. DOI

9. Yoshua Bengio, Patrice Simard and Paolo Frasconi, “Learning Long-Term Dependencies with Gradient Descent Is Difficult,” *IEEE Transactions on Neural Networks* 5(2), 1994. DOI
10. Razvan Pascanu, Tomas Mikolov and Yoshua Bengio, “On the Difficulty of Training Recurrent Neural Networks,” ICML, 2013. arXiv
11. Sepp Hochreiter and Jürgen Schmidhuber, “Long Short-Term Memory,” *Neural Computation* 9(8), 1997. DOI
12. Felix A. Gers, Jürgen Schmidhuber and Fred Cummins, “Learning to Forget: Continual Prediction with LSTM,” *Neural Computation* 12(10), 2000. DOI
13. Kyunghyun Cho et al., “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” EMNLP, 2014. arXiv
14. Mike Schuster and Kuldip K. Paliwal, “Bidirectional Recurrent Neural Networks,” *IEEE Transactions on Signal Processing* 45(11), 1997. DOI
15. Alex Graves, “Generating Sequences With Recurrent Neural Networks,” arXiv:1308.0850, 2013. arXiv
16. Wojciech Zaremba, Ilya Sutskever and Oriol Vinyals, “Recurrent Neural Network Regularization,” arXiv:1409.2329, 2014. arXiv
17. Gábor Melis, Chris Dyer and Phil Blunsom, “On the State of the Art of Evaluation in Neural Language Models,” arXiv:1707.05589, 2017. arXiv
18. Stephen Merity, Nitish Shirish Keskar and Richard Socher, “Regularizing and Optimizing LSTM Language Models,” arXiv:1708.02182, 2017. arXiv
19. Rafal Jozefowicz, Oriol Vinyals, Mike Schuster, Noam Shazeer and Yonghui Wu, “Exploring the Limits of Language Modeling,” arXiv:1602.02410, 2016. arXiv
20. Matteo Tiezzi, Michele Casoni, Alessandro Betti, Tommaso Guidi, Marco Gori and Stefano Melacci, “On the Resurgence of Recurrent Models for Long Sequences: Survey and Research Opportunities in the Transformer Era,” arXiv:2402.08132, 2024. arXiv
21. Bo Peng et al., “RWKV: Reinventing RNNs for the Transformer Era,” Findings of EMNLP, 2023. Paper
22. Albert Gu and Tri Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” COLM, 2024. arXiv
23. Maximilian Beck et al., “xLSTM: Extended Long Short-Term Memory,” NeurIPS, 2024. Paper
24. Lewis Carroll, *Alice’s Adventures in Wonderland*, 1865; Project Gutenberg eBook #11. Project Gutenberg

NLP 06 • Sequence Labeling with RNNs (POS Tagging, Information Extraction)

Label every token of a sentence: part-of-speech tags with HMMs and Viterbi decoding, entity spans with BIO labels and exact span evaluation, and neural taggers built from BiLSTMs, character features and a CRF output layer.

Online edition: https://blog.shyandram.dev/en/learn/nlp/nlp-06-sequence-labeling/

What you'll learn

- Represent part-of-speech tagging and entity recognition as sequence labeling, and explain why independent token decisions can disagree with a valid sentence-level analysis.
- Derive hidden Markov sequence scores and maximum-path (Viterbi) decoding, and connect them to the conditional random field.
- Define an entity schema with reproducible offsets, encode mentions with BIO labels, and evaluate exact spans rather than token accuracy.
- Build BiLSTM taggers, add character-level features and a CRF output layer, and train the CRF by its log-likelihood with a masked forward algorithm.
- Place entity tagging inside information extraction (entities, relations, events) and name the standard datasets.
- Run a complete PyTorch experiment that measures span F1, error types and invalid tag transitions on names never seen in training.

The big picture

Many language tasks ask for one label per token. A part-of-speech (POS) tagger marks each word as noun, verb, adjective and so on. An entity recognizer marks which tokens belong to a person, organization or place name, and where each name begins and ends. Both look like classification repeated along a sentence, but they are not independent decisions: the right label for a word depends on its neighbors' words *and* on its neighbors' labels. Treating the whole label sequence as one structured output is the central idea of this chapter.

McRoy's Chapters 3 and 4 introduce grammatical categories and syntactic processing, and Chapter 8 connects language processing with information extraction [1]. Hagiwara's Chapter 5 develops sequential labeling, including part-of-speech tagging and named entity recognition [2]. This chapter combines those views. A tagger attaches categories to positions; an extractor turns selected spans into records that an application can inspect. The output contract matters: a customer-support system might need a product name and a complaint event, whereas a historical archive needs people, organizations, dates and attributed statements.

Explain like I'm five

Give a child a sentence and three highlighter pens: yellow for people, green for companies, blue for places. Coloring each word alone is not enough. "Port" is blue in "Port Lena" but not in "a port in the harbor," and if the second half of a two-word name is colored green, the first half had better be green too. A good highlighter looks at the whole sentence and makes the colors agree.

The practical risk is losing distinctions during conversion. "The supplier said the device passed" does not establish that the device passed. A system can identify the correct names and still build a false record by dropping attribution or modality. We therefore evaluate both surface extraction and interpretation, rather than treating well-formed output as evidence of correctness. The structured-prediction mathematics (HMMs, the forward–backward algorithm, Viterbi decoding and CRFs in general form) is developed in PR 06; the recurrent encoders come from NLP 05. Parsing, semantic roles and relation and event extraction in depth follow in NLP 12.

Words, morphology and grammatical categories

Plain version. A word's category describes how it behaves in a sentence. Its spelling can suggest a category, but context often decides which use is intended.

Morphology concerns structure within words. A plural ending, tense marker or derivational prefix can supply useful evidence, but not every language expresses these properties with the same pieces or word boundaries. Subword tokens learned for efficient modeling also need not correspond to linguistic morphemes. Keep the model's tokenization separate from the grammatical units used in annotation.

Part of speech describes a grammatical category, such as noun or verb. A noun commonly heads a noun phrase; a verb commonly heads a clause's predicate. These are distributional descriptions rather than rules that every noun must name a physical object. "Decision" and "arrival" are nouns although they refer to abstract content and events.

The same surface word can have several categories. In the invented pair "the pilots can land" and "the metal can fell," the word "can" has different roles. The second sentence also demonstrates why a familiar local phrase can be misleading when the remainder of the sentence changes. A tagger must consider the permitted context and its annotation scheme.

Represent a tagged sentence as:

$$\mathbf{x} = (x_1, \dots, x_T), \qquad \mathbf{y} = (y_1, \dots, y_T), \qquad y_t \in \mathcal{Y}.$$

Here $\mathbf{x}$ is the input word sequence, x_t is the word at position t, T is sequence length, $\mathbf{y}$ is the tag sequence, y_t is the tag for word x_t and $\mathcal{Y}$ is the finite tag inventory. The symbols t and T refer to linguistic word positions, not automatically to model subword positions.

Annotation needs additional rules: contractions, punctuation, multiword expressions and words with several plausible analyses. When a model uses subwords, define how its outputs return to words. Using only the first subword's representation is one possible convention; pooling all subwords is another. Whatever the choice, gold labels and decoded output must use the same units.

Local classification versus sequence prediction

Plain version. Choosing the best tag for every word separately can miss a better joint interpretation. A sequence model makes the relationship among tags explicit.

A local classifier may predict:

$$\hat{y}_t = \operatorname*{argmax}_{k \in \mathcal{Y}} p_\theta(y_t = k \mid \mathbf{x}).$$

Here $\hat{y}_t$ is the selected tag at position t, k ranges over tag inventory $\mathcal{Y}$, p_θ is a model with parameters θ and $\mathbf{x}$ is the whole visible input sequence. A local output head can still read contextual input: "independent" describes the output decision rule, not necessarily the encoder.

A structured decoder instead chooses:

$$\hat{\mathbf{y}} = \operatorname*{argmax}_{\mathbf{y} \in \mathcal{A}(\mathbf{x})} S_\theta(\mathbf{x}, \mathbf{y}).$$

Here $\hat{\mathbf{y}}$ is the selected complete tag sequence, $\mathcal{A}(\mathbf{x})$ is the set of permitted analyses for input $\mathbf{x}$, and S_θ is a score assigned by parameters θ to an input–analysis pair. The permitted set can enforce an annotation constraint, while the score can reward plausible neighboring labels.

This distinction appears in many NLP tasks. Entity labels must form valid spans; a dependency tree needs one parent per non-root word; a generated structured record must satisfy a schema. A stronger encoder does not automatically guarantee a valid output. The decoder or a separate validator must enforce constraints that the application requires.

Structured decoding also has a tradeoff. An incorrect hard constraint can exclude a legitimate construction. If the grammar says every sentence must contain an overt subject, it will reject many commands and fragments. Document the constraint and its intended input domain rather than treating constrained output as evidence that the constraint was linguistically universal.

Hidden Markov models for tags

Plain version. Imagine that tags form a hidden sequence and each tag generates a word. Neighboring tags and word compatibility then jointly determine the score.

A first-order hidden Markov model (HMM), reviewed by Rabiner [3], uses:

$$p(\mathbf{x}, \mathbf{y}) = \pi_{y_1} b_{y_1}(x_1) \prod_{t=2}^{T} a_{y_{t-1}, y_t} b_{y_t}(x_t).$$

Here $p(\mathbf{x}, \mathbf{y})$ is joint probability of words and tags, π_k is the probability of initial tag k, $a_{j,k}$ is the transition probability from tag j to tag k, $b_k(w)$ is the probability that tag k emits word w, x_t and y_t are the word and tag at position t, and T is sequence length. This example conditions on a fixed length; an explicit end-state factor can be added when modeling termination.

Initial probabilities sum to one. Each transition row sums to one, and each emission distribution sums to one over the observation vocabulary. These normalization conditions make the expression a probability model rather than an arbitrary product of useful scores.

The assumptions are restrictive: the next tag depends only on the previous tag, and an emitted word depends only on the current tag. A real sentence has long dependencies, lexical preferences and discourse context. The value of this model here is that the assumptions and exact decoder are visible. It provides a small laboratory for distinguishing model capacity from search.

With fully tagged training examples, count initial states, transitions and emissions, then normalize with a stated smoothing rule. With words alone, hidden-state estimation is a different problem; a state learned without supervision is not automatically a named grammatical category. The example below uses manually specified probabilities so that training uncertainty does not obscure decoding.

Maximum-path decoding

Plain version. Keep the best partial path ending in each tag. Extending those paths is enough to find the best complete sequence without listing every possibility.

Use log probabilities to turn products into sums:

$$D_1(k) = \log \pi_k + \log b_k(x_1),$$
$$D_t(k) = \log b_k(x_t) + \max_{j \in \mathcal{Y}} [D_{t-1}(j) + \log a_{j,k}].$$

Here $D_t(k)$ is the log score of the best path covering positions 1 through t and ending with tag k, π_k is the initial probability, $b_k(x_t)$ is the emission probability, $a_{j,k}$ is the transition probability and $\mathcal{Y}$ is the tag inventory. The indices j and k identify the previous and current tags. A zero probability contributes $-\infty$.

The recurrence is maximum-path dynamic programming, commonly called Viterbi decoding [3]. For each cell, store the previous tag attaining the maximum. After selecting the best final tag, follow these backpointers in reverse. Keeping only the score loses the actual sequence.

The decoder considers T positions and all pairs of adjacent tags, so its time complexity is $O(TK^2)$ for $K = |\mathcal{Y}|$ tags. The score table uses $O(TK)$ space if all backpointers are retained. By contrast, enumerating tag sequences considers K^T possibilities. This is a computational saving under a specific factorization, not an approximate search result.

The following standalone example verifies the dynamic-programming answer against exhaustive enumeration. The toy observation “fish fish” has an intentionally ambiguous word. Independent emission preferences select nouns at both positions; the joint model can favor a noun followed by a verb.

```
import itertools
import numpy as np
```

```
tags = ["N", "V"]
vocabulary = ["fish", "swim"]
initial = np.array([0.8, 0.2])
transition = np.array([[0.1, 0.9], [0.8, 0.2]])
emission = np.array([[0.7, 0.3], [0.3, 0.7]])
words = ["fish", "fish"]
observation = [vocabulary.index(word) for word in words]
assert np.allclose(transition.sum(axis=1), 1)
assert np.allclose(emission.sum(axis=1), 1)

scores = np.empty((len(words), len(tags)))
back = np.zeros_like(scores, dtype=int)
scores[0] = np.log(initial) + np.log(emission[:, observation[0]])
for t in range(1, len(words)):
    candidates = scores[t - 1, :, None] + np.log(transition)
    back[t] = candidates.argmax(axis=0)
    scores[t] = candidates.max(axis=0) + np.log(emission[:, observation[t]])
path = [int(scores[-1].argmax())]
for t in range(len(words) - 1, 0, -1):
    path.append(int(back[t, path[-1]]))
path.reverse()

def joint_score(sequence):
    value = np.log(initial[sequence[0]])
    value += np.log(emission[sequence[0], observation[0]])
    for t in range(1, len(words)):
        value += np.log(transition[sequence[t - 1], sequence[t]])
        value += np.log(emission[sequence[t], observation[t]])
    return value

all_paths = list(itertools.product(range(len(tags)), repeat=len(words)))
brute = max(all_paths, key=joint_score)
assert tuple(path) == brute
print("local emission choices:", [tags[emission[:, w].argmax()] for w in observation])
print("best joint path:", [tags[index] for index in path])
print("joint log score:", round(joint_score(path), 3))
```

Changing transition probabilities changes the chosen sequence even when every word-level preference stays fixed. This does not prove the new choice is correct in real language. It proves that the implemented model and decoder express the intended dependence.

Read the path score as evidence, not just an argmax

The noun–verb path wins in the preceding example because the transition preference outweighs the second word's emission preference. Its joint probability is the product of four factors: an initial noun, a noun emitting `fish`, a noun-to-verb transition, and a verb emitting `fish`. Comparing it with noun–noun cancels the shared first two factors. The remaining comparison is between transition times emission; the decision is therefore intelligible without treating the decoder as a black box.

The following independent calculation enumerates the same four paths and normalizes them into a posterior distribution for the observed words:

```
import itertools
import numpy as np

initial = np.array([0.8, 0.2])
transition = np.array([[0.1, 0.9], [0.8, 0.2]])
fish = np.array([0.7, 0.3])
paths = list(itertools.product(range(2), repeat=2))
mass = np.array([initial[a] * fish[a] * transition[a, b] * fish[b]
                 for a, b in paths])
posterior = mass / mass.sum()
print('NN / NV / VN / VV joint:', mass.round(4))
print('Posterior:', posterior.round(3))
print('Probability of observed words:', round(mass.sum(), 4))
assert paths[posterior.argmax()] == (0, 1)
assert np.isclose(posterior.sum(), 1)
```

The joint masses are `0.0392`, `0.1512`, `0.0336` and `0.0036`, and the posterior rounds to `0.172`, `0.664`, `0.148` and `0.016`. The observation probability is `0.2276`. These answer different questions. A joint mass includes how likely the words were; a posterior compares label interpretations after those words are fixed. The winning path still leaves probability on alternatives under this constructed model.

For larger sequences, summing predecessor contributions rather than maximizing them computes observation probability. Combining forward and backward contributions gives tag marginals [3]. Selecting the most probable tag at each position minimizes expected per-position mistakes under that posterior; selecting the most probable complete path targets whole-sequence correctness. Those decisions can differ. Independently selected marginal tags can also violate output constraints when the allowed sequence set contains restrictions.

Why can maximum-path decoding discard other prefixes ending in the same tag? Under the first-order factorization, their remaining extensions depend on that final tag and future observations, not on earlier prefix details. A lower-scoring prefix cannot overtake the better prefix after an identical suffix is added. If a new score rewards a relation to a tag several positions earlier, this argument no longer applies to the same state. Either enlarge the state or change the decoder. Exactness belongs to the score factorization, not to the name of the algorithm.

Conditional structured scores connect neural features to decoding

An HMM must model how words are generated. A conditional model can instead score label sequences directly from the observed sentence. A linear-chain conditional random field (CRF) provides this interface [5]:

$$S_\theta(\mathbf{x}, \mathbf{y}) = \sum_{t=1}^{T} u_\theta(t, y_t, \mathbf{x}) + \sum_{t=2}^{T} A_{y_{t-1}, y_t}, \qquad p_\theta(\mathbf{y} \mid \mathbf{x}) = \frac{\exp S_\theta(\mathbf{x}, \mathbf{y})}{Z_\theta(\mathbf{x})}.$$

Here $\mathbf{x}$ is the observed sentence, $\mathbf{y}$ is a tag sequence, T is its length, t indexes positions, u_θ is a position–tag score computed with parameters θ, and A contains adjacent-tag scores. The normalizer $Z_\theta(\mathbf{x})$ sums $\exp S_\theta(\mathbf{x}, \mathbf{y}')$ over every permitted sequence $\mathbf{y}'$. Unlike HMM emissions, the position scores need not sum to one over words. A contextual encoder can compute each score using the entire sentence while the output dependence remains a chain.

Training an annotated sequence minimizes its negative log probability: the correct score is rewarded relative to the scores of all permitted competitors. Computing the normalizer uses a sum recurrence; finding the best sequence uses a max recurrence. This explains why a neural encoder and structured decoder fit together. Rich input evidence does not require discarding a tractable output factorization. Conversely, a chain transition table cannot express every syntactic dependency; sentence-level input features and output constraints solve different parts of the problem.

PR 06 develops the general theory behind these two models: the forward and forward–backward algorithms, Baum–Welch training of HMMs from unlabeled sequences, and the CRF gradient as observed minus expected feature counts. The neural tagger sections below reuse exactly this CRF, with the position scores u_θ computed by a recurrent encoder.

Entity recognition starts with a schema

Plain version. An entity recognizer locates mentions and assigns types. It does not automatically identify the unique real-world object behind each mention.

Represent an entity mention as:

$$r = (d, b, e, c).$$

Here r is a mention record; d is the document identifier; b is its inclusive starting character offset; e is its exclusive ending offset; and c is its category. The surface text is the original document slice from b to e. These are Python string offsets in our example; another interface may use bytes or another indexing convention. Declare the convention because Unicode characters and encoded bytes do not have identical boundaries.

An annotation guideline must decide whether titles belong to a person's span, whether an organization suffix is included, and how to handle ambiguous uses. "Taipei" can name a place or stand for a government in context. A location-only schema and a context-sensitive organization schema can legitimately produce different annotations. Evaluation becomes meaningful only after the schema and guidelines are frozen.

Named entity recognition, or NER, commonly distinguishes people, organizations and locations; the CoNLL-2003 shared-task paper describes one established benchmark setting [6]. We mention that resource to identify the evaluation tradition, not to imply that its categories fit every application. Product identifiers, legal provisions or equipment parts may be more useful in another domain.

Entity linking is a separate decision: map a mention to a record in a knowledge base, or choose that no suitable record exists. Two mentions with the same spelling can refer to different organizations; two different spellings can refer to the same one. A recognizer that marks "Orion" as

an organization has not solved this identity problem. Linking requires a candidate inventory, context and an explicit unknown-entity policy.

Keep the original text and offsets even when you store a normalized name. Lowercasing, accent removal and whitespace collapse can change strings and boundaries. One reliable design extracts from the original text and stores normalization as an additional field. If preprocessing is necessary, retain an alignment map so the predicted span can be projected back without guessing.

Sequence labels and span prediction

Plain version. Entity mentions can be expressed as labels on tokens or as explicit spans. The representation determines which overlaps the model can describe.

In a BIO scheme, a token label begins with B for the beginning of a mention, I for its continuation or O for outside. An example is "Mina works at Orion Labs," with person labels on Mina and organization labels on Orion and Labs. Hagiwara's sequence-labeling chapter and the NER survey give context for token-level extraction models [2], [7]. The scheme descends from the inside–outside chunk tags of Ramshaw and Marcus [10].

For an independent token classifier:

$$p_\theta(\mathbf{y} \mid \mathbf{x}) = \prod_{i=1}^{T} p_\theta(y_i \mid \mathbf{x}).$$

Here $\mathbf{x}$ is the input token sequence; $\mathbf{y}$ is the label sequence; T is the number of tokens; i indexes positions; and θ denotes learned parameters. Each prediction may use a contextual representation of the whole permitted input, although the output labels are independent under this factorization. Input context and output dependence are different ideas.

Independent decisions can produce invalid transitions, such as an I-PER label after O. A deterministic repair rule is possible, but its behavior becomes part of the evaluation contract. Alternatively, a structured decoder searches only valid paths while scoring the whole sequence. The dynamic-programming idea from the maximum-path decoding section applies when the score decomposes locally. More powerful encoders do not remove the need to define valid outputs.

A flat token-label sequence cannot represent arbitrary nested mentions. "Orion Labs Research Division" could contain an organization mention and a nested division mention under a particular schema. A span model instead scores candidate boundaries and types:

$$s_\theta(b, e, c \mid \mathbf{x}).$$

Here s_θ is a learned score; b and e are candidate span boundaries; c is the candidate category; and $\mathbf{x}$ is the input. The score need not be a calibrated probability. A selection policy decides which candidates to keep, whether overlap is allowed and how conflicts are resolved.

The candidate count grows with sentence length, so practical designs may bound maximum span length or prune candidates. Every such restriction changes which mentions can be recovered. If very long names matter, measure their recall separately. A fast system can otherwise achieve a

reasonable average score while never representing the outputs needed by a small but consequential group of users.

Boundary modeling changes which errors are possible

A token-labeling model and a span model can use the same contextual encoder yet search different output spaces. Flat BIO labels choose a single membership at each token. Explicit spans choose boundary pairs and types, so nested intervals can coexist when the selection policy permits them. The difference is representational: an encoder cannot recover a nested output that its decoder has no way to express [7].

For n tokens, the number of nonempty contiguous intervals is $n(n+1)/2$. Here n is sentence length; the count sums the possible endings for each starting position. With maximum length $L \le n$, the count becomes $nL - L(L-1)/2$, where L is the permitted span length. A type scorer may need a score for each of those intervals and each category. Bounding length saves work but removes longer mentions from the candidate space before classification begins.

Boundary quality and semantic typing need separate diagnosis. A span covering `Orion` may be a strong organization candidate while the gold mention is `Orion Labs`. Raising its category confidence does not fix the missing boundary. A model may instead propose both spans; then deduplication and overlap rules determine the final output. Treat selection as part of the model behavior, not as formatting applied after evaluation.

Relations add another search problem. Candidate entity pairs multiply as more mentions survive. Restricting pairs by type or distance can reduce computation, but a document-level relation may cross several sentences. A joint extractor can use relation evidence to influence mention decisions, whereas a pipeline commits to mentions first. Compare the pipeline on predicted mentions and on gold mentions: the gap reveals how much relation recovery is lost before relation scoring starts.

Exact span evaluation

Plain version. Correctly labeling common background tokens can hide missed entities. Evaluate complete mentions, including their boundaries and types.

For sets of predicted records P and gold records G:

$$\text{precision} = \frac{|P \cap G|}{|P|}, \qquad \text{recall} = \frac{|P \cap G|}{|G|}, \qquad F_1 = \frac{2|P \cap G|}{|P| + |G|}.$$

Here P and G contain document identifier, start offset, end offset and category tuples; $|\cdot|$ denotes set size; and $P \cap G$ contains exact matches. Precision measures the share of predictions that match; recall measures the share of annotations recovered; and F_1 combines them. Our code reports zero when a displayed denominator is zero; another evaluation convention should be documented explicitly.

Exact matching is strict. Predicting “Orion” when the annotation says “Orion Labs” creates both a false positive and a false negative. That is appropriate when downstream code depends on

a complete name. For another application, partial overlap may deserve a secondary diagnostic. Report its matching rule separately; do not rename a forgiving overlap score as the established exact-match measure [6].

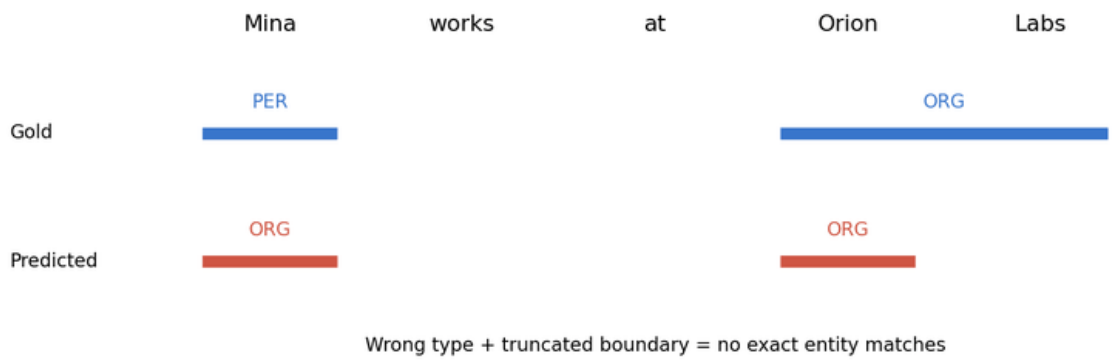


Figure 1 — A boundary error and a type error remain errors even when most words are labeled correctly. This is a synthetic diagnostic example.

A single aggregate score still hides the mechanism. Break errors into missed mention, spurious mention, boundary mismatch, category mismatch and linking mismatch. Also report performance by category and document source. When a category is rare, retain its example count so the reader can assess uncertainty. A large weighted average can otherwise bury a category with nearly no successful predictions.

A complete CPU extraction experiment

Plain version. A small deterministic extractor makes the contract easy to inspect. It is a teaching baseline with visible blind spots, not a general language-understanding system.

The following standalone example recognizes an invented name inventory, extracts one tightly constrained active repair construction and evaluates exact entity records. It downloads nothing. The inventory is fixed before evaluation, and the gold records are written explicitly. A lexicon is an allowed input here; if a real evaluation forbids external name inventories, the same code would violate that contract.

```
import re

documents = {
    "a": "Mina repaired Sensor A in Taipei.",
    "b": "Orion Labs said Sensor B might fail.",
    "c": "Leo did not repair Sensor B.",
    "d": "The sensor was repaired by Mina.",
}
inventory = {
```

```
    "Mina": "PER", "Leo": "PER", "Orion Labs": "ORG",
    "Sensor A": "DEVICE", "Sensor B": "DEVICE", "Taipei": "LOC",
}
gold = {
    ("a", 0, 4, "PER"), ("a", 14, 22, "DEVICE"), ("a", 26, 32, "LOC"),
    ("b", 0, 10, "ORG"), ("b", 16, 24, "DEVICE"),
    ("c", 0, 3, "PER"), ("c", 19, 27, "DEVICE"), ("d", 27, 31, "PER"),
}
predicted = set()
for doc_id, text in documents.items():
    for name, category in inventory.items():
        pattern = r"(?<!\w)" + re.escape(name) + r"(?!\w)"
        for match in re.finditer(pattern, text):
            predicted.add((doc_id, match.start(), match.end(), category))
tp = len(predicted & gold)
precision = tp / len(predicted) if predicted else 0.0
recall = tp / len(gold) if gold else 0.0
f1 = 2 * tp / (len(predicted) + len(gold)) if predicted or gold else 0.0
assert predicted == gold
print("entity precision/recall/F1:", precision, recall, f1)

repair = re.compile(r"^(Mina|Leo) repaired (Sensor [AB])(?=[ .])")
events = []
for doc_id, text in documents.items():
    match = repair.search(text)
    if match:
        events.append({
            "document": doc_id, "predicate": "repair",
            "actor": match.group(1), "object": match.group(2),
            "polarity": "positive", "modality": "asserted",
            "evidence": [match.start(), match.end()],
        })
assert len(events) == 1 and events[0]["document"] == "a"
for event in events:
    start, end = event["evidence"]
    print(event, "source:", documents[event["document"]][start:end])
print("unsupported repair constructions:", ["c", "d"])
```

The exact entity score is 1.0 on these hand-built documents, because all mention spellings are in the inventory. That result does not establish generalization. The event rule returns only document a. It excludes the explicit negation in c but also misses the valid passive construction in d. These are different reasons for returning no event, and a production interface should distinguish rejected negative claims from unsupported syntax.

Document b contains recognized entities but no asserted repair event. It also contains an attributed, possible failure. This example illustrates why entity recognition and event extraction need separate evaluation. A system can obtain every name correctly while omitting the task's most relevant event. Expanding the event grammar requires additional positive and negative fixtures before accepting the new behavior.

The evidence interval covers only the matched clause, not the trailing location. That is adequate for the narrow repair record, which has no location field. If the record later includes location,

expand the evidence contract and its tests. Treat schema changes as changes to the task, rather than adding a field that is populated opportunistically from unrelated nearby words.

Information extraction beyond entity spans

Plain version. Entity recognition finds the names. Information extraction (IE) goes on to connect them: which person works for which organization, which device failed where and when. Each step adds a new kind of record and a new way to be wrong.

A useful map has three layers, each building on the previous one.

Layer	Output record	Example from the running domain	Typical method family
Entities	Typed span (d, b, e, c)	`Orion Labs` is an ORG at characters 0–10	Sequence labeling (this chapter) or span scoring
Relations	Typed, directed pair of mentions with evidence	`employer(Mina, Orion Labs)`	Classify candidate mention pairs
Events	Trigger plus role-labeled arguments, polarity and modality	`repair` with repairer `Mina`, object `Sensor A`, place `Taipei`	Trigger detection, then argument labeling

Sequence labeling contributes directly to all three. Entity mentions are BIO spans. Event triggers can be tagged the same way as entities. Even some relation and argument decisions are posed as labeling, by tagging the tokens of each argument relative to a given trigger. The rule-based extractor above already shows the dependency between layers: its single event record is correct only because both entity spans were found and the construction was supported.

Standard resources reflect the layers. The ACE 2005 multilingual training corpus annotates entities, relations and events for English, Chinese and Arabic, with event annotation limited to English and Chinese [19]. TACRED is a crowdsourced sentence-level relation extraction dataset targeted at the relation types of the TAC Knowledge Base Population evaluation [20]. DocRED moves relation extraction to whole documents built from Wikipedia and Wikidata, where many relations can only be inferred by reading several sentences [21]. The record design questions raised earlier in this chapter (evidence spans, attribution, negation, unknown values) apply at every layer. Relation and event extraction, semantic roles and their evaluation are developed in NLP 12.

Neural taggers: a BiLSTM over the sentence

Plain version. Replace hand-written emission probabilities with a neural network that reads the whole sentence in both directions and produces a score for every tag at every position. The simplest version picks the highest-scoring tag at each position independently.

Collobert and colleagues showed that a single neural architecture, trained from data with little feature engineering, could tag parts of speech, chunks, named entities and semantic roles [12].

With recurrent encoders the standard design is the bidirectional LSTM (BiLSTM) tagger [13], [14], [15]:

$$\vec{\mathbf{h}}_t = \mathrm{LSTM}_{\rightarrow}(\mathbf{v}_t, \vec{\mathbf{h}}_{t-1}), \qquad \overleftarrow{\mathbf{h}}_t = \mathrm{LSTM}_{\leftarrow}(\mathbf{v}_t, \overleftarrow{\mathbf{h}}_{t+1}),$$
$$\mathbf{u}_t = W\,[\vec{\mathbf{h}}_t; \overleftarrow{\mathbf{h}}_t] + \mathbf{b} \in \mathbb{R}^K.$$

Here $\mathbf{v}_t$ is the input vector for token x_t (a word embedding, optionally concatenated with character features below); $\vec{\mathbf{h}}_t$ has read $x_1, \dots, x_t$ and $\overleftarrow{\mathbf{h}}_t$ has read $x_t, \dots, x_T$; $[\cdot;\cdot]$ is concatenation; W and $\mathbf{b}$ map the two states to $K = |\mathcal{Y}|$ tag scores; and $\mathbf{u}_t$ collects those scores, $u_{t,k}$ for tag k. These scores play the role of $u_\theta(t, y_t, \mathbf{x})$ in the CRF formula above: they can depend on the whole sentence. Bidirectionality is appropriate here, unlike in the left-to-right language model of NLP 05, because the whole sentence is given before any tag is chosen [23].

The **softmax tagger** treats the tags as conditionally independent given these scores:

$$p_\theta(y_t = k \mid \mathbf{x}) = \frac{\exp u_{t,k}}{\sum_{k'=1}^{K} \exp u_{t,k'}}, \qquad \mathcal{L}_{\text{tok}} = -\frac{1}{N} \sum_b \sum_t m_{b,t} \log p_\theta(y_{b,t} \mid \mathbf{x}_b).$$

Here b indexes sentences in a mini-batch, $m_{b,t} \in \{0, 1\}$ is the padding mask (one for a real token), $N = \sum_{b,t} m_{b,t}$ counts real tokens, $y_{b,t}$ is the gold tag and k' ranges over tags. This is exactly the independent token classifier of the sequence-labeling section, with a BiLSTM computing the context. It is fast and often strong. Its weakness is also the one identified there: nothing stops it from emitting `O I-ORG`, or `B-PER I-ORG`, because each position's decision ignores its neighbors' decisions.

Three practical details matter as much as the architecture.

- **Masks.** Sentences in a batch are padded to a common length. The mask removes padding from the loss; packing (`pack_padded_sequence`) or masking keeps padding out of the backward LSTM, which would otherwise start reading from padding symbols.
- **Unknown words.** A word-embedding table only covers training words. Map the rest to an unknown id, and *train that id*: randomly replacing a fraction of training words with it (word dropout) teaches the tagger to rely on context when the word itself is unfamiliar.
- **Subword encoders.** When the encoder uses subwords, define how word labels map to pieces (label the first piece and ignore the others, or another explicit rule) and map predictions back to words before evaluation, as discussed in the first section.

BiLSTM-CRF: scoring the whole label sequence

Plain version. Keep the BiLSTM's per-token scores, add a learned score for every pair of adjacent tags, and choose the label sequence with the best total. Training raises the score of the gold sequence relative to all other sequences, which the forward algorithm sums efficiently.

Huang, Xu and Yu proposed putting a linear-chain CRF layer on top of a BiLSTM for POS tagging, chunking and NER [13]. Lample and colleagues [14] and Ma and Hovy [15] combined the same output layer with character-level word representations and reported strong NER results without

hand-engineered features. Using the CRF notation from the maximum-path decoding section, with start and end scores added:

$$S_\theta(\mathbf{x}, \mathbf{y}) = \pi_{y_1} + \sum_{t=1}^{T} u_{t,y_t} + \sum_{t=2}^{T} A_{y_{t-1},y_t} + \eta_{y_T}, \qquad \log p_\theta(\mathbf{y} \mid \mathbf{x}) = S_\theta(\mathbf{x}, \mathbf{y}) - \log Z_\theta(\mathbf{x}).$$

Here $u_{t,k}$ are the BiLSTM scores; $A_{j,k}$ is the learned score for tag j followed by tag k; π_k and η_k score starting and ending with tag k; and $Z_\theta(\mathbf{x}) = \sum_{\mathbf{y}'} \exp S_\theta(\mathbf{x}, \mathbf{y}')$ sums over every tag sequence $\mathbf{y}'$ of length T. The training loss is the negative log-likelihood $-\log p_\theta(\mathbf{y} \mid \mathbf{x})$ averaged over sentences.

The sum over K^T sequences is computed by the forward algorithm in log space:

$$\begin{aligned} \alpha_1(k) &= \pi_k + u_{1,k}, \\ \alpha_t(k) &= u_{t,k} + \log \sum_{j=1}^{K} \exp\left(\alpha_{t-1}(j) + A_{j,k}\right), \qquad t = 2, \dots, T, \\ \log Z_\theta(\mathbf{x}) &= \log \sum_{k=1}^{K} \exp\left(\alpha_T(k) + \eta_k\right). \end{aligned}$$

Here $\alpha_t(k)$ is the log of the summed exponentiated scores of all partial sequences that cover positions 1 to t and end in tag k. Replacing each log-sum-exp by a maximum, and storing the maximizing j, gives exactly the Viterbi recursion of the HMM section with $\log b_k(x_t)$ replaced by $u_{t,k}$ and $\log a_{j,k}$ by $A_{j,k}$. Both cost $O(TK^2)$. In a padded batch, a position with mask zero copies α_{t-1} forward unchanged, so each sentence's normalizer stops at its own length.

What the gradient does. Differentiating the loss with respect to a transition score gives

$$\frac{\partial(-\log p_\theta(\mathbf{y} \mid \mathbf{x}))}{\partial A_{j,k}} = \mathbb{E}_{\mathbf{y}' \sim p_\theta(\cdot \mid \mathbf{x})}[n_{j,k}(\mathbf{y}')] - n_{j,k}(\mathbf{y}),$$

where $n_{j,k}(\mathbf{y})$ counts how often tag j is followed by tag k in $\mathbf{y}$ and the expectation is under the model. Gradient descent therefore raises transitions that the gold sequence uses more often than the model currently expects and lowers the others; the expectations come from forward–backward marginals, as PR 06 shows. Automatic differentiation of the forward algorithm computes this without writing the backward pass by hand.

Hard constraints. A BIO scheme forbids some transitions outright: `I-x` may only follow `B-x` or `I-x`, and no sentence may start with `I-x`. A learned A usually discovers this from data, but with little data it may not. Setting the forbidden entries of A and π to $-\infty$ (in practice a large negative number) makes invalid sequences impossible under both training and decoding, at no cost to valid ones. This is the output-constraint idea from the local-versus-sequence section made concrete.

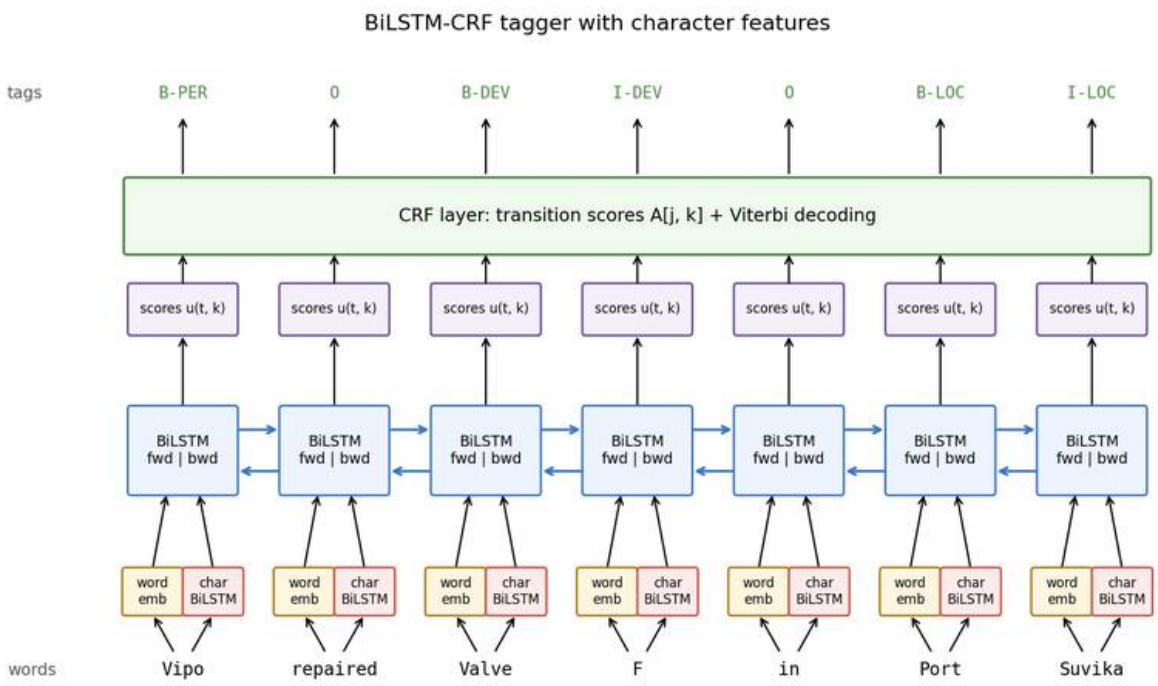


Figure 2 — A BiLSTM-CRF tagger with character features. The emission scores come from the encoder; the transition scores and Viterbi decoding make the tags agree with each other.

Character-level features

Plain version. A word-embedding table knows nothing about a word it never saw. The letters of the word often do: capitalization, suffixes such as "-ville" or "-corp," digits and hyphens are strong clues to the tag even for a brand-new name.

Before neural taggers, these clues were hand-written features (is capitalized, ends in "-ing," contains a digit). Neural taggers learn them from characters. Lample and colleagues run a small bidirectional LSTM over each word's characters and concatenate its final forward and backward states to the word embedding [14]; Ma and Hovy use a convolution over character embeddings followed by max pooling [15]. Either way the input becomes

$$\mathbf{v}_t = [\mathbf{e}(x_t); \mathbf{c}(x_t)], \qquad \mathbf{c}(x_t) = [\vec{\mathbf{g}}_L; \overleftarrow{\mathbf{g}}_1],$$

where $\mathbf{e}(x_t)$ is the word embedding, $\mathbf{c}(x_t)$ the character summary, L the number of characters in x_t, $\vec{\mathbf{g}}_L$ the forward character-LSTM state after the last character and $\overleftarrow{\mathbf{g}}_1$ the backward state after reading back to the first. The word-level BiLSTM then reads $\mathbf{v}_t$. Akbik, Blythe and Vollgraf took this idea further: they trained a character-level language model, as in NLP 05, and used its internal states around each word as contextual embeddings for sequence labeling [16].

Character features help with exactly the inputs that word embeddings handle worst: unseen names, rare inflections and misspellings. They do not resolve ambiguity that the spelling does not

encode. A name with no telling suffix can be a person or a company, and only context or world knowledge decides. Character inputs also need their own unknown-symbol policy for characters missing from training.

Datasets for tagging and extraction

Plain version. Benchmarks fix a tag inventory, an annotation guideline, a tokenization and a split. Comparing numbers across them is only meaningful when all four match.

Resource	Task	What it provides
Penn Treebank [11]	English POS tagging and parsing	A large annotated corpus of English with part-of-speech tags and syntactic bracketing; its tag set is widely used for English
Universal Dependencies [4]	Cross-lingual POS, morphology and dependencies	A shared annotation framework applied to treebanks in many languages
CoNLL-2003 [6]	Named entity recognition	English and German data with four types (person, organization, location, miscellaneous) and exact-match evaluation
OntoNotes [17]	Named entities with 18 types, plus syntax, semantic roles and coreference	Multi-genre, multilingual annotation with stratified training, development and test partitions whose test sets match the CoNLL-2012 shared task
WNUT 2017 [18]	Novel and emerging entities in noisy user-generated text	A test of recall on rare and previously unseen surface forms
ACE 2005 [19]	Entities, relations and events	Multilingual annotation (English, Chinese, Arabic) used for relation and event extraction
TACRED [20]	Sentence-level relation extraction	Crowdsourced relation labels for TAC KBP relation types
DocRED [21]	Document-level relation extraction	Entities and relations annotated over Wikipedia documents, requiring multi-sentence reasoning

Tag encodings differ too. The CoNLL-2003 files use an IOB variant, descended from Ramshaw and Marcus's chunk tags, in which `B-` appears only when an entity directly follows another entity of the same type [6], [10]; convert such files to the BIO form used in this chapter before training, or a BIO span extractor will misread them. Licenses differ: some of these resources are freely downloadable, while others are distributed by the Linguistic Data Consortium under license. Check the terms before building on one. For a new domain, the most valuable dataset is usually a small, carefully adjudicated set annotated with your own frozen guideline, used alongside a public benchmark to compare methods.

A complete PyTorch tagging experiment

Plain version. We train three taggers on the same small synthetic corpus and evaluate them on sentences whose names never appear in training: a word-only BiLSTM, the same tagger with character features, and the character tagger with a constrained CRF output layer. The evaluation counts exact spans, error types and invalid tag transitions.

The corpus is invented. Person, organization and location names are generated from random syllables, so test names are new spellings. Organizations either end in `tek` or `corp` or are followed by the word `Labs`; locations either end in `ville` or follow `Port`. Several templates allow either a person or an organization in the same slot, so context alone cannot always decide the type. There are 100 training sentences and 200 test sentences. Everything else is implemented from scratch: span extraction from BIO tags, exact span precision, recall and F1, an error breakdown, padding masks, word dropout, the character BiLSTM, and a CRF with a masked forward algorithm, Viterbi decoding and BIO constraints, checked against brute-force enumeration. Each model is trained with three seeds; the script runs on a CPU in well under a minute.

```
import random
import torch
from torch import nn

# ---- 1. A synthetic tagged corpus: invented names, fixed templates ----
SYL = ["ka", "mi", "to", "re", "na", "lo", "su", "vi", "da", "po", "ri", "ze"]

def coin(rng, kind):
    stem = "".join(rng.choice(SYL) for _ in range(rng.choice((2, 3)))).capitalize()
    if kind == "PER":
        return stem
    if kind == "ORG":
        return rng.choice([stem + rng.choice(["tek", "corp"]), stem + " Labs"])
    return rng.choice(["Port " + stem, stem + "ville"])           # LOC

rng = random.Random(0)
TRAIN_NAMES = {k: sorted({coin(rng, k) for _ in range(40)}) for k in ("PER", "ORG", "LOC")}
seen = {n for v in TRAIN_NAMES.values() for n in v}
TEST_NAMES = {k: rng.sample(sorted({coin(rng, k) for _ in range(60)} - seen), 15)
              for k in ("PER", "ORG", "LOC")}                      # unseen spellings
TRAIN_NAMES["DEV"] = ["Pump A", "Pump B", "Sensor C", "Sensor D", "Valve E", "Valve F"]
TEST_NAMES["DEV"] = ["Pump G", "Sensor H", "Valve J", "Pump K"]
TEMPLATES = [
    "{PER} repaired {DEV} in {LOC} .",
    "{ORG} repaired {DEV} in {LOC} .",
    "{PER} reported a fault in {DEV} .",
    "{ORG} reported a fault in {DEV} .",
    "the sensor near {LOC} was checked by {PER} .",
    "the sensor near {LOC} was checked by {ORG} .",
    "{PER} and {PER} visited {LOC} .",
    "{ORG} shipped {DEV} to {LOC} yesterday .",
    "after the test , {PER} said {DEV} might fail .",
    "a pump in {LOC} failed and {ORG} replaced it .",
]
```

```
def make(names, n):
    data = []
    for _ in range(n):
        words, tags = [], []
        for piece in rng.choice(TEMPLATES).split():
            if piece.startswith("{"):
                kind = piece[1:-1]
                toks = rng.choice(names[kind]).split()
                words += toks
                tags += ["B-" + kind] + ["I-" + kind] * (len(toks) - 1)
            else:
                words.append(piece)
                tags.append("O")
        data.append((words, tags))
    return data

train_data, test_data = make(TRAIN_NAMES, 100), make(TEST_NAMES, 200)
TAGS = ["O"] + [f"{p}-{k}" for k in ("PER", "ORG", "LOC", "DEV") for p in "BI"]
t2i = {t: i for i, t in enumerate(TAGS)}
w2i = {w: i + 2 for i, w in enumerate(sorted({w for s, _ in train_data for w in s}))}
c2i = {c: i + 2 for i, c in enumerate(sorted({c for w in w2i for c in w}))}
PAD, UNK = 0, 1
print(train_data[0])

# ---- 2. Spans and exact span F1, from scratch ----
def spans(tags):
    out, start, kind = set(), None, None
    for i, t in enumerate(tags + ["O"]):
        if start is not None and not (t.startswith("I-") and t[2:] == kind):
            out.add((start, i, kind))
            start = None
        if t.startswith("B-") or (t.startswith("I-") and start is None):
            start, kind = i, t[2:]                  # an orphan I- still opens a span
    return out

def span_prf(gold_seqs, pred_seqs):
    tp = n_pred = n_gold = 0
    for g, p in zip(gold_seqs, pred_seqs):
        G, P = spans(g), spans(p)
        tp, n_pred, n_gold = tp + len(G & P), n_pred + len(P), n_gold + len(G)
    prec = tp / n_pred if n_pred else 0.0
    rec = tp / n_gold if n_gold else 0.0
    f1 = 2 * tp / (n_pred + n_gold) if n_pred + n_gold else 0.0
    return prec, rec, f1

def error_types(gold_seqs, pred_seqs):
    counts = {"type": 0, "boundary": 0, "missed": 0, "spurious": 0}
    for g, p in zip(gold_seqs, pred_seqs):
        G, P = spans(g), spans(p)
        overlap = lambda a, b: a[0] < b[1] and b[0] < a[1]
        for s in G - P:
            hits = [q for q in P if overlap(s, q)]
            if any(q[:2] == s[:2] for q in hits):
```

```
                counts["type"] += 1             # right boundaries, wrong category
            elif hits:
                counts["boundary"] += 1         # overlaps, wrong boundaries
            else:
                counts["missed"] += 1
        counts["spurious"] += sum(not any(overlap(q, s) for s in G) for q in P - G)
    return counts

def invalid(tags):                                  # I-x not preceded by B-x or I-x
    return sum(t.startswith("I-") and (i == 0 or tags[i - 1][2:] != t[2:])
               for i, t in enumerate(tags))

assert spans(["B-ORG", "I-ORG", "O", "B-PER"]) == {(0, 2, "ORG"), (3, 4, "PER")}
assert invalid(["O", "I-LOC", "B-DEV", "I-DEV"]) == 1
print("train/test sentences:", len(train_data), len(test_data),
      " gold test spans:", sum(len(spans(t)) for _, t in test_data))

# ---- 3. Padding, masks and word dropout ----
def batch(data, unk_rate=0.0, r=None):
    L = max(len(s) for s, _ in data)
    W = max(len(w) for s, _ in data for w in s)
    x = torch.full((len(data), L), PAD)
    c = torch.full((len(data), L, W), PAD)
    y = torch.zeros(len(data), L, dtype=torch.long)
    for b, (s, t) in enumerate(data):
        for i, w in enumerate(s):
            drop = r is not None and r.random() < unk_rate
            x[b, i] = UNK if drop else w2i.get(w, UNK)      # train the unknown id too
            y[b, i] = t2i[t[i]]
            for j, ch in enumerate(w):
                c[b, i, j] = c2i.get(ch, UNK)
    return x, c, y, x != PAD

# ---- 4. BiLSTM encoder with optional character BiLSTM features ----
class Encoder(nn.Module):
    def __init__(self, use_chars, wdim=32, cdim=16, hidden=48):
        super().__init__()
        self.use_chars = use_chars
        self.wemb = nn.Embedding(len(w2i) + 2, wdim, padding_idx=PAD)
        self.cemb = nn.Embedding(len(c2i) + 2, cdim, padding_idx=PAD)
        self.clstm = nn.LSTM(cdim, cdim, batch_first=True, bidirectional=True)
        self.lstm = nn.LSTM(wdim + (2 * cdim if use_chars else 0), hidden,
                            batch_first=True, bidirectional=True)
        self.emit = nn.Linear(2 * hidden, len(TAGS))

    def forward(self, x, c):
        feats = [self.wemb(x)]
        if self.use_chars:
            B, L, W = c.shape
            lengths = (c != PAD).sum(-1).clamp(min=1).view(-1)
            packed = nn.utils.rnn.pack_padded_sequence(
                self.cemb(c.view(B * L, W)), lengths, batch_first=True,
                enforce_sorted=False)
            _, (h, _) = self.clstm(packed)          # last forward + first backward state
```

```
            feats.append(torch.cat([h[0], h[1]], -1).view(B, L, -1))
        h, _ = self.lstm(torch.cat(feats, -1))
        return self.emit(h)                          # emission scores u[b, t, k]

# ---- 5. Linear-chain CRF with BIO constraints and a padding mask ----
class CRF(nn.Module):
    def __init__(self, K):
        super().__init__()
        self.A = nn.Parameter(torch.zeros(K, K))     # A[j, k]: score of tag j -> tag k
        self.start = nn.Parameter(torch.zeros(K))
        self.end = nn.Parameter(torch.zeros(K))
        ban = torch.zeros(K, K)                      # I-x only after B-x or I-x
        for k, tag in enumerate(TAGS):
            if tag.startswith("I-"):
                ban[:, k] = -1e4
                ban[t2i["B-" + tag[2:]], k] = ban[k, k] = 0.0
        self.register_buffer("ban", ban)
        self.register_buffer("ban_start", torch.tensor(
            [-1e4 if t.startswith("I-") else 0.0 for t in TAGS]))

    def trans(self):
        return self.A + self.ban

    def score(self, u, y, mask):                     # S(x, y) for the gold path
        s = (self.start + self.ban_start)[y[:, 0]] + u[:, 0].gather(1, y[:, :1]).squeeze(1)
        for t in range(1, u.size(1)):
            step = self.trans()[y[:, t - 1], y[:, t]] + u[:, t].gather(1, y[:, t:t +
             ↪  1]).squeeze(1)
            s = s + step * mask[:, t]
        last = y.gather(1, (mask.sum(1) - 1).unsqueeze(1)).squeeze(1)
        return s + self.end[last]

    def log_partition(self, u, mask):                # forward algorithm in log space
        alpha = self.start + self.ban_start + u[:, 0]
        for t in range(1, u.size(1)):
            nxt = torch.logsumexp(alpha.unsqueeze(2) + self.trans() + u[:, t].unsqueeze(1),
             ↪  dim=1)
            alpha = torch.where(mask[:, t:t + 1], nxt, alpha)   # padding: carry forward
        return torch.logsumexp(alpha + self.end, dim=1)

    def nll(self, u, y, mask):
        return (self.log_partition(u, mask) - self.score(u, y, mask)).mean()

    def viterbi(self, u, mask):                      # same recursion with max + backpointers
        delta, back = self.start + self.ban_start + u[:, 0], []
        for t in range(1, u.size(1)):
            best, arg = (delta.unsqueeze(2) + self.trans()).max(1)
            back.append(arg)
            delta = torch.where(mask[:, t:t + 1], best + u[:, t], delta)
        delta = delta + self.end
        paths = []
        for b in range(u.size(0)):
            k = int(delta[b].argmax())
            path = [k]
```

```
            for t in range(int(mask[b].sum()) - 1, 0, -1):
                k = int(back[t - 1][b, k])
                path.append(k)
            paths.append(path[::-1])
        return paths

# Check the masked forward algorithm and Viterbi against brute-force enumeration
import itertools
torch.manual_seed(0)
check = CRF(len(TAGS))
with torch.no_grad():
    check.A.normal_()
u = torch.randn(1, 3, len(TAGS))
m = torch.ones(1, 3, dtype=torch.bool)
paths = [torch.tensor([p]) for p in itertools.product(range(len(TAGS)), repeat=3)]
scores = torch.stack([check.score(u, p, m)[0] for p in paths])
assert torch.allclose(check.log_partition(u, m)[0], torch.logsumexp(scores, 0), atol=1e-4)
assert paths[int(scores.argmax())][0].tolist() == check.viterbi(u, m)[0]

# ---- 6. Train three taggers and evaluate exact spans on unseen names ----
def run(use_chars, use_crf, seed, epochs=15):
    torch.manual_seed(seed)
    r = random.Random(seed)
    enc, crf = Encoder(use_chars), CRF(len(TAGS))
    params = list(enc.parameters()) + (list(crf.parameters()) if use_crf else [])
    opt = torch.optim.Adam(params, lr=5e-3)
    order = list(range(len(train_data)))
    for _ in range(epochs):
        r.shuffle(order)
        for i in range(0, len(order), 32):
            x, c, y, mask = batch([train_data[j] for j in order[i:i + 32]], 0.25, r)
            u = enc(x, c)
            loss = (crf.nll(u, y, mask) if use_crf      # sequence-level loss
                    else nn.functional.cross_entropy(u[mask], y[mask]))  # per token
            opt.zero_grad()
            loss.backward()
            nn.utils.clip_grad_norm_(params, 5.0)
            opt.step()
    enc.eval()
    with torch.no_grad():
        x, c, y, mask = batch(test_data)
        u = enc(x, c)
        pred = (crf.viterbi(u, mask) if use_crf else
                [u[b, :int(mask[b].sum())].argmax(-1).tolist() for b in range(len(u))])
    pred = [[TAGS[k] for k in p] for p in pred]
    gold = [t for _, t in test_data]
    return span_prf(gold, pred), sum(map(invalid, pred)), error_types(gold, pred), crf

results = {}
for name, use_chars, use_crf in [("BiLSTM, words", False, False),
                                 ("BiLSTM, words+chars", True, False),
                                 ("BiLSTM-CRF, words+chars", True, True)]:
    runs = [run(use_chars, use_crf, seed) for seed in (0, 1, 2)]
    results[name] = runs
```

```
    f1s = [prf[2] for prf, _, _, _ in runs]
    errors = {k: sum(r[2][k] for r in runs) for k in runs[0][2]}
    print(f"{name}: F1 per seed {[round(f, 3) for f in f1s]}, mean {sum(f1s) / 3:.3f}")
    print(f"    invalid I- tags {[r[1] for r in runs]}, errors over 3 seeds {errors}")
assert all(r[1] == 0 for r in results["BiLSTM-CRF, words+chars"])
```

Output:

```
(['Vipo', 'repaired', 'Valve', 'F', 'in', 'Port', 'Suvika', '.'], ['B-PER', 'O', 'B-DEV',
 ↪  'I-DEV', 'O', 'B-LOC', 'I-LOC', 'O'])
train/test sentences: 100 200  gold test spans: 497
BiLSTM, words: F1 per seed [0.893, 0.771, 0.82], mean 0.828
    invalid I- tags [0, 30, 0], errors over 3 seeds {'type': 66, 'boundary': 43, 'missed':
     ↪  247, 'spurious': 0}
BiLSTM, words+chars: F1 per seed [0.893, 0.903, 0.942], mean 0.913
    invalid I- tags [4, 1, 0], errors over 3 seeds {'type': 124, 'boundary': 2, 'missed': 7,
     ↪  'spurious': 0}
BiLSTM-CRF, words+chars: F1 per seed [0.915, 0.909, 0.961], mean 0.929
    invalid I- tags [0, 0, 0], errors over 3 seeds {'type': 102, 'boundary': 0, 'missed': 8,
     ↪  'spurious': 0}
```

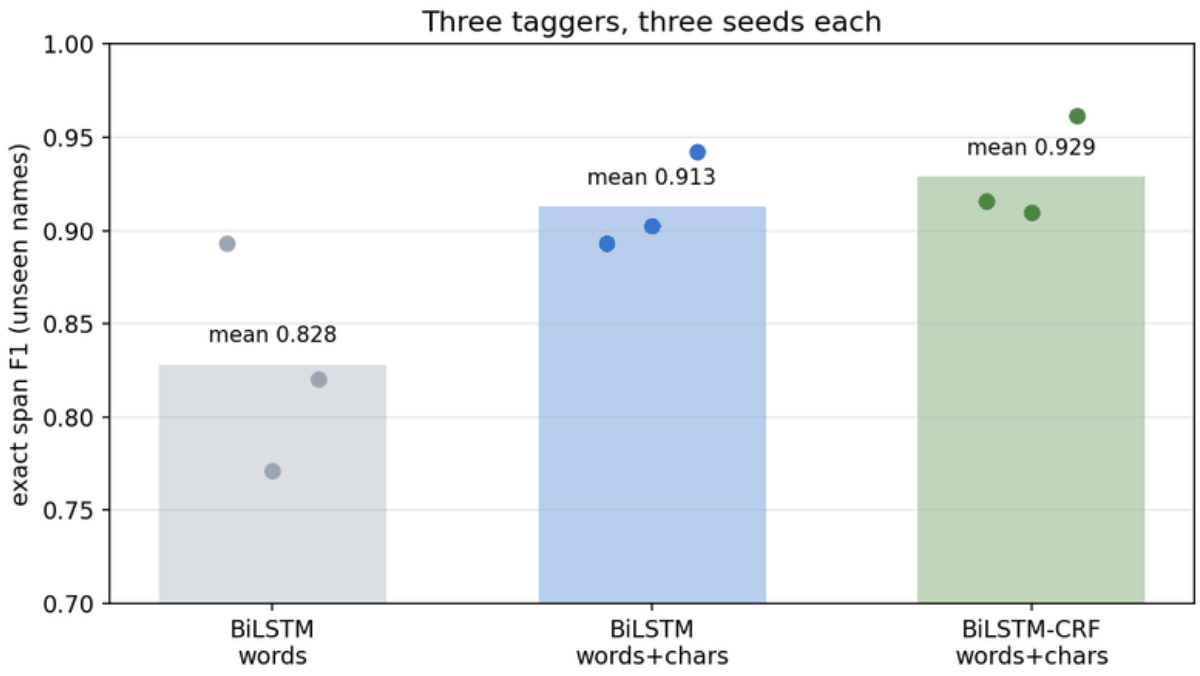


Figure 3 — Exact span F1 on test sentences whose names never occur in training. Dots are seeds; bars are means. Character features give the largest gain; the constrained CRF adds a smaller, consistent one.

Reading the results

Character features fix detection. The word-only tagger sees most test names as the unknown id. Its errors are dominated by missed entities (247 over three seeds), plus boundary errors when a two-token name is split or truncated. With character features, misses drop to 7 and boundary errors to 2: the suffixes and capitalization of new names are enough to find them. Mean F1 rises from 0.828 to 0.913.

What remains is typing. Almost all remaining errors for both character models are type errors (124 and 102 over three seeds): the span is right but a person is labeled as an organization or the reverse. They involve single-token names, often in templates that accept either type in the same slot, such as `Mimitek reported a fault` or `checked by Renaka`. Context cannot settle those cases, the spelling of a person name carries no type marker in this corpus, and a model trained on 100 sentences has not learned the `tek` and `corp` endings reliably enough to override a context that also fits a person. This is the limit described in the character-features section: spelling helps only when it encodes the distinction.

The CRF guarantees valid output and helps a little. The word-only tagger produced 30 invalid `I-` tags in one seed and the character softmax tagger produced 4 and 1 in two seeds; the CRF with BIO constraints produced none, by construction. Its mean F1 is 0.929 against 0.913, and it is at least as good as the softmax tagger for each seed. With 100 training sentences that is a small margin; it is the consistency of the improvement and the guarantee of well-formed spans, not its size, that justifies the extra layer here.

Seeds matter again. The word-only tagger ranges from 0.771 to 0.893 across seeds. The seed with 30 invalid transitions is also the worst one. As in NLP 05, a single run would have been an unreliable summary.

What this does not show. The corpus is synthetic, tiny and templated; the test sentences reuse the training templates; and the tag set is small. Real benchmarks contain ambiguous context, nested and discontinuous mentions, and annotation disagreements. The experiment isolates three mechanisms (unknown-word handling, character evidence and output constraints) so that each effect can be seen and measured; it does not estimate how large those effects are on CoNLL-2003 or any other dataset.

What the transitions learned. Figure 4 shows the learned transition scores for one seed. Inside the allowed region, the CRF has learned the corpus's structure: `B-LOC → I-LOC` and `B-DEV → I-DEV` are strongly positive, while `I-ORG → I-ORG` and `I-LOC → I-LOC` are strongly negative, because no name in this corpus is longer than two tokens. A learned transition table is also a modeling assumption to inspect. Trained on a domain where every name has at most two tokens, it will penalize a three-token name elsewhere.

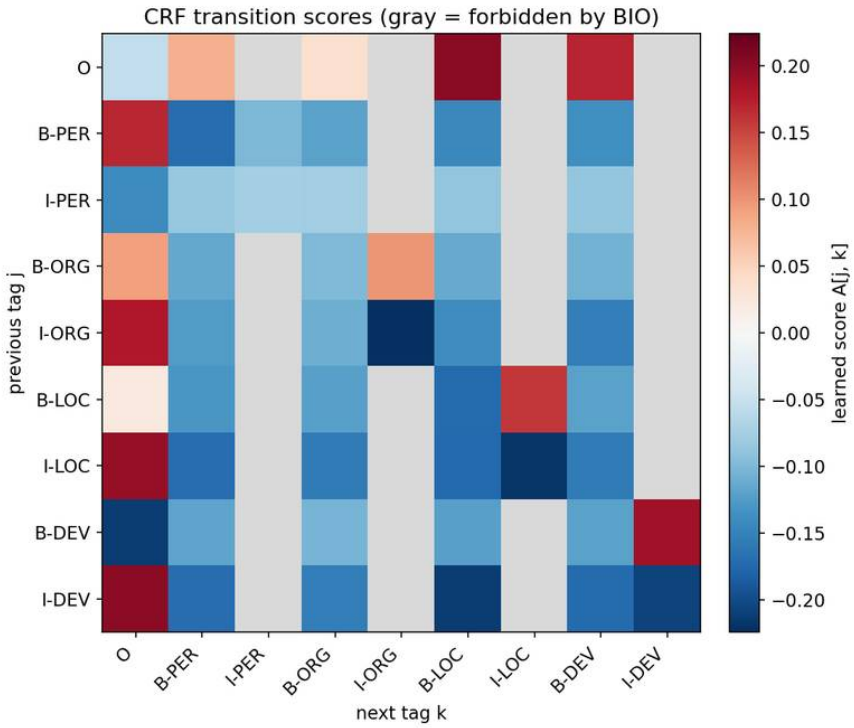


Figure 4 — Learned transition scores of the CRF (seed 0). Gray cells are BIO transitions forbidden by the constraint mask; the remaining scores were learned from 100 training sentences.

Modern view

Plain version. Current extraction systems combine pretrained representations with explicit task definitions. General language generation increases flexibility, while leaving span fidelity and support as separate problems.

Li and colleagues' NER survey organizes earlier deep systems by input representation, context encoder and tag decoder [7]. This taxonomy is still useful: changing the encoder does not determine whether the output is a flat sequence or overlapping spans. Xu and colleagues' 2024 generative extraction survey adds a task axis—entities, relations, events and unified extraction—and an adaptation axis including prompting, examples, supervised tuning and constrained generation [8]. Read these maps as design dimensions, rather than a ranking of universally superior architectures.

From BiLSTM-CRF to pretrained encoders. In Li and colleagues' taxonomy, this chapter's experiment varies the input representation (words versus words plus characters) and the tag decoder (softmax versus CRF) while keeping a BiLSTM context encoder [7]. Between 2016 and 2018 the BiLSTM-CRF with character features [14], [15] became the standard recipe, and contextual string embeddings from character language models extended it [16]. Pretrained Transformer encoders then replaced the BiLSTM as the context encoder in most systems; a linear or CRF tag decoder on

top of them is the token-classification head discussed in NLP 10 and NLP 11. The decoder questions of this chapter (valid transitions, span boundaries, exact evaluation) carry over unchanged.

Large language models for open entity recognition. UniversalNER (ICLR 2024) distills a large language model into much smaller models for open-domain NER through targeted instruction tuning, and evaluates them on a benchmark assembled from 43 NER datasets across 9 domains [22]. Together with GLiNER [9], it represents a move from a fixed tag inventory to entity types supplied at inference time.

GLiNER is a 2024 original example of label-conditioned span extraction with a bidirectional Transformer [9]. Its design shows that adapting the requested entity categories need not require autoregressively generating a long list of records. A structured span output can retain direct alignment with input text. The relevant comparison for an application includes supported category definitions, nested-span behavior, latency and error review, alongside aggregate accuracy.

A generative extractor instead emits a textual serialization of the requested record. Enforcing a valid schema can prevent missing keys or malformed syntax, but valid serialization does not guarantee a correct name, relation or evidence interval. A practical validation layer checks that quoted evidence exists, offsets match, categories are allowed and entity identities are consistent. These checks eliminate some errors while leaving interpretation errors for evaluation or review.

Open problems include changing category inventories, specialized language, nested entities, cross-document identity and reliable uncertainty under domain shift. Multilingual extraction adds different name boundaries, writing systems and conventions for titles. Treat a language-specific annotation guideline as part of the task definition; translating category names is insufficient. Evaluate documents by source and language while keeping enough examples to make each comparison interpretable.

For a new project, establish a deterministic or compact supervised baseline, freeze an annotation guide and inspect its error types before choosing a larger model. Use a held-out document set that prevents repeated templates or near-duplicate reports crossing the split. Compare complete supported records, not only whether a model returns parseable output. The result is a measurable extraction service whose failures can be traced back to evidence.

New labels test both representation and interpretation

Label-conditioned span extraction [9] and generative extraction [8] change how an application supplies its category inventory. That flexibility creates an analytical question: does the model recognize the intended category, or merely react to a familiar label name? Compare a short label with a precise description and deliberately similar competing descriptions. For example, distinguish an equipment manufacturer from an equipment operator; both may be organization mentions, but their event roles differ.

Keep entity detection, requested typing and relation assignment as separate judgments. A correctly located organization span does not establish that it manufactured the device. Likewise, a generated record that fills every requested field may invent a participant when the source omitted it. Test an explicit unknown answer against forced completion. The comparison should reveal

which output interface preserves missing information and which training examples distinguish closely related schemas. Broader category coverage is useful only when the semantic boundaries remain clear.

Key takeaways

- Sequence labeling assigns one label per token, but labels interact; a structured decoder chooses the best complete sequence under a stated factorization.
- HMM Viterbi decoding is exact for its first-order factorization; a linear-chain CRF keeps the same recursions while scoring tags conditionally, so a neural encoder can supply the scores.
- Entity recognition starts with a schema and offsets; BIO labels encode flat spans, and nested mentions need a span representation.
- Evaluate exact spans (boundaries and type), and break errors into missed, spurious, boundary and type errors instead of reporting token accuracy.
- A BiLSTM tagger reads the whole sentence in both directions; masks and trained unknown-word ids are essential details.
- A CRF layer trained by log-likelihood (forward algorithm) and decoded by Viterbi makes tags agree; BIO constraints make invalid sequences impossible.
- Character features recover unseen names from their spelling but cannot resolve ambiguity the spelling does not encode.
- Information extraction builds relations and events on top of entity spans; each layer adds its own record contract and failure modes.

Exercises

1. **A sequence decision.** Change one transition row in the HMM while preserving its normalization. Find the condition under which the best path for "fish fish" becomes two nouns.

   Hint

   Compare the complete path products. The initial factor and first emission are shared by the two noun-initial candidates.

2. Extend the CPU extraction example with a new person spelling that is absent from the inventory. Add its gold span and report the change in exact recall.

   Hint

   Keep the original inventory frozen. A missed mention is useful evidence about the baseline's scope, rather than a reason to edit the test vocabulary into training.

3. Design a schema for overlapping organization and department names. Explain why a single flat BIO sequence cannot express every allowed annotation.

Hint

Draw the character intervals first. Decide whether nested names count as separate entities and whether a department is linked to its parent organization.

4. In the PyTorch experiment, remove the BIO constraint mask from the CRF (set `ban` and `ban_start` to zeros) and retrain. Count invalid `I-` tags over three seeds, then inspect the learned transition scores for `O` followed by `I-ORG`.

Hint

Without the mask, invalid transitions are discouraged only to the extent the 100 training sentences teach it. Compare the learned score with the scores of allowed transitions into `I-ORG`.

5. Add a character-only ablation: drop the word embedding and keep the character BiLSTM. Then give person names a distinctive ending in the generator (for example `-ko`) and retrain all models. Which error type changes, and why?

Hint

Use the error breakdown printed by the script. A new spelling cue should move errors out of the type column only for the names that carry it.

6. Write a partial-match evaluator that gives credit when a predicted span overlaps a gold span of the same type. Report it next to exact F1 for the word-only tagger and explain why the two can rank models differently.

Hint

Define the matching rule precisely (for example one-to-one matching by largest overlap) and report it under its own name. The word-only tagger's boundary errors are where the two scores diverge.

References

1. Susan McRoy, *Principles of Natural Language Processing*, open textbook, 2021; Chapters 3–5 and 8. Open textbook
2. Masato Hagiwara, *Real-World Natural Language Processing: Practical Applications with Deep Learning*, Manning, 2021; Chapter 5, "Sequential labeling and language modeling." Chapter
3. L. R. Rabiner, "A tutorial on hidden Markov models and selected applications in speech recognition," *Proceedings of the IEEE*, 77(2), 1989. Paper
4. M.-C. de Marneffe, C. D. Manning, J. Nivre and D. Zeman, "Universal Dependencies," *Computational Linguistics*, 47(2), 2021. Paper
5. J. Lafferty, A. McCallum and F. Pereira, "Conditional random fields: Probabilistic models for segmenting and labeling sequence data," ICML, 2001. Paper

6. Erik F. Tjong Kim Sang and Fien De Meulder, “Introduction to the CoNLL-2003 Shared Task: Language-Independent Named Entity Recognition,” CoNLL, 2003. Paper
7. Jing Li, Aixin Sun, Jianglei Han and Chenliang Li, “A Survey on Deep Learning for Named Entity Recognition,” IEEE Transactions on Knowledge and Data Engineering 34(1), 2022. Author-hosted paper
8. Derong Xu, Wei Chen, Wenjun Peng, Chao Zhang, Tong Xu, Xiangyu Zhao, Xian Wu, Yefeng Zheng, Yang Wang and Enhong Chen, “Large language models for generative information extraction: a survey,” Frontiers of Computer Science 18(6), 2024. Survey
9. Urchade Zaratiana, Nadi Tomeh, Pierre Holat and Thierry Charnois, “GLiNER: Generalist Model for Named Entity Recognition using Bidirectional Transformer,” NAACL, 2024. Paper
10. Lance Ramshaw and Mitch Marcus, “Text Chunking using Transformation-Based Learning,” Third Workshop on Very Large Corpora, 1995. Paper
11. Mitchell P. Marcus, Beatrice Santorini and Mary Ann Marcinkiewicz, “Building a Large Annotated Corpus of English: The Penn Treebank,” *Computational Linguistics* 19(2), 1993. Paper
12. Ronan Collobert, Jason Weston, Léon Bottou, Michael Karlen, Koray Kavukcuoglu and Pavel Kuksa, “Natural Language Processing (Almost) from Scratch,” *Journal of Machine Learning Research* 12, 2011. Paper
13. Zhiheng Huang, Wei Xu and Kai Yu, “Bidirectional LSTM-CRF Models for Sequence Tagging,” arXiv:1508.01991, 2015. arXiv
14. Guillaume Lample, Miguel Ballesteros, Sandeep Subramanian, Kazuya Kawakami and Chris Dyer, “Neural Architectures for Named Entity Recognition,” NAACL-HLT, 2016. Paper
15. Xuezhe Ma and Eduard Hovy, “End-to-end Sequence Labeling via Bi-directional LSTM-CNNs-CRF,” ACL, 2016. Paper
16. Alan Akbik, Duncan Blythe and Roland Vollgraf, “Contextual String Embeddings for Sequence Labeling,” COLING, 2018. Paper
17. Sameer Pradhan, Alessandro Moschitti, Nianwen Xue, Hwee Tou Ng, Anders Björkelund, Olga Uryupina, Yuchen Zhang and Zhi Zhong, “Towards Robust Linguistic Analysis using OntoNotes,” CoNLL, 2013. Paper
18. Leon Derczynski, Eric Nichols, Marieke van Erp and Nut Limsopatham, “Results of the WNUT2017 Shared Task on Novel and Emerging Entity Recognition,” Workshop on Noisy User-generated Text, 2017. Paper
19. Christopher Walker, Stephanie Strassel, Julie Medero and Kazuaki Maeda, *ACE 2005 Multilingual Training Corpus*, LDC2006T06, Linguistic Data Consortium, 2006. Catalog
20. Yuhao Zhang, Victor Zhong, Danqi Chen, Gabor Angeli and Christopher D. Manning, “Position-aware Attention and Supervised Data Improve Slot Filling,” EMNLP, 2017. Paper
21. Yuan Yao, Deming Ye, Peng Li, Xu Han, Yankai Lin, Zhenghao Liu, Zhiyuan Liu, Lixin Huang, Jie Zhou and Maosong Sun, “DocRED: A Large-Scale Document-Level Relation Extraction Dataset,” ACL, 2019. Paper
22. Wenxuan Zhou, Sheng Zhang, Yu Gu, Muhao Chen and Hoifung Poon, “UniversalNER: Targeted Distillation from Large Language Models for Open Named Entity Recognition,” ICLR, 2024. arXiv
23. Mike Schuster and Kuldip K. Paliwal, “Bidirectional Recurrent Neural Networks,” *IEEE Transactions on Signal Processing* 45(11), 1997. DOI

NLP 07 • Attention Mechanism and Transformers

From the encoder–decoder bottleneck to Bahdanau and Luong attention, scaled dot-product and multi-head attention, masks, positions and the Transformer encoder and decoder blocks, with attention and an encoder block built from scratch.

Online edition: https://blog.shyandram.dev/en/learn/nlp/nlp-07-attention-transformers/

What you'll learn

- Why squeezing a whole source sentence into one vector limits an encoder–decoder, and how attention removes that bottleneck.
- The additive (Bahdanau) and multiplicative (Luong) attention equations, and how they generalize to queries, keys and values.
- Scaled dot-product attention, multi-head attention, and the difference between self-attention and cross-attention.
- How padding masks and causal masks encode what each position is allowed to read, and how to test that they work.
- Why attention needs position information, and the main ways to supply it.
- How Transformer encoder and decoder blocks are assembled, what they cost, and which variants target long contexts.

The big picture

Recurrent networks (NLP 05) read a sentence one token at a time and carry a running summary. That summary has a fixed size, whether the sentence has five words or fifty. Attention changes the access pattern: instead of relying on one summary, a position can look back at every stored representation and decide which ones matter for the prediction it is making now. The Transformer takes this idea to its conclusion and builds both the encoder and the decoder out of attention and small per-token networks, with no recurrence at all.

> **Explain like I'm five**
>
> Imagine translating a long sentence. One way is to read it once, close the book, and write the translation from memory. Another way is to keep the book open and, for each word you write, glance back at the part of the sentence you need. Attention is the second way. The "glance" is a set of weights saying how much to look at each word. A mask is a rule about which parts of the page you may look at: in some games you may not peek at the words you have not written yet.

Three decisions organize this chapter, and they are worth keeping separate. The **scoring rule** decides how strongly one position matches another. The **visibility rule**, expressed as a mask,

decides which positions may be read at all. The **position rule** decides how word order enters a computation that would otherwise ignore it. A change to one does not fix a mistake in another: a better scoring function cannot repair a mask that leaks future tokens.

Hagiwara's Chapter 8 develops attention and Transformers for practical NLP [1]. This chapter adds the derivations, connects them to the original papers, and builds every mechanism in small CPU examples. ML 04 treats the same machinery from the vision side, with the full positional-encoding math; NLP 08 uses it to build a translation model.

1. The encoder–decoder bottleneck

Plain version. A sequence-to-sequence model has an encoder that reads the source and a decoder that writes the target. If the only bridge between them is one vector, every detail the decoder needs must fit through that vector.

The recurrent encoder–decoder models of Sutskever, Vinyals and Le [2] and Cho et al. [3] read a source sequence $x_1, \dots, x_T$ with an encoder RNN and pass its final state to the decoder:

$$h_j = f_{\text{enc}}(h_{j-1}, x_j), \qquad c = h_T, \qquad s_u = f_{\text{dec}}(s_{u-1}, y_{u-1}, c), \qquad p(y_u \mid y_{<u}, \mathbf{x}) = g(s_u).$$

Here $h_j \in \mathbb{R}^d$ is the encoder state after reading source token x_j; T is the source length; c is the single context vector; s_u is the decoder state at target step u; y_{u-1} is the previous target token; $y_{<u}$ is the target prefix; f_{enc} and f_{dec} are recurrent update functions such as LSTM or GRU cells; and g is an output layer followed by a softmax over the target vocabulary. ML 03A derives these cells and trains such a model.

The vector c has the same dimension d for every input. A ten-word sentence and a sixty-word paragraph must both be compressed into it before the first target word is produced. Bahdanau, Cho and Bengio named this fixed-length vector as a likely bottleneck for basic encoder–decoders and proposed letting the decoder search the source for the relevant parts instead [4]. Sutskever et al. also reported that reversing the source order helped their LSTM model, which is consistent with the same diagnosis: shortening the path between the first source words and the first target words made them easier to use [2].

The bottleneck is about information flow, not only capacity. Even a large c must be computed before the decoder knows which source detail it will need at step u. Attention postpones that choice: the encoder keeps one vector per source token, and the decoder chooses a different mixture of them at every step.

2. Additive attention (Bahdanau)

Plain version. At each target step, score every source position against what the decoder currently needs, turn the scores into weights that sum to one, and take the weighted average of the encoder states.

Bahdanau et al. [4] keep all encoder states $h_1, \dots, h_T$ (from a bidirectional encoder) and compute, for target step u,

$$e_{u,j} = v_a^\top \tanh\big(W_a s_{u-1} + U_a h_j\big), \qquad \alpha_{u,j} = \frac{\exp(e_{u,j})}{\sum_{k=1}^{T} \exp(e_{u,k})}, \qquad c_u = \sum_{j=1}^{T} \alpha_{u,j}\, h_j.$$

Here s_{u-1} is the previous decoder state; h_j is the encoder state for source position j; W_a, U_a and v_a are learned matrices and a learned vector; $e_{u,j}$ is the **alignment score**; $\alpha_{u,j}$ is the **attention weight**, nonnegative and summing to one over j; and c_u is the step-specific **context vector**, which the decoder uses together with its own state to predict y_u. The notation matches ML 03A. Because the score passes the two vectors through a small feedforward layer before comparing them, this form is called **additive attention**.

Two consequences matter for NLP. First, the decoder now has a direct path to every source token, so the information about the first source word no longer has to survive many recurrent updates. Second, the weights $\alpha_{u,j}$ form a soft alignment between target and source positions, which can be plotted and inspected. That inspection is a diagnostic, not a guarantee: as discussed below, a large weight shows where the computation read from, not why the final prediction is correct.

3. Multiplicative attention (Luong)

Plain version. Replace the small scoring network by a dot product, possibly through one learned matrix. It is cheaper and turns attention into matrix multiplication.

Luong, Pham and Manning [5] compared several scoring functions for a **global** attention model that attends to all source states. They score the current decoder state h_u (rather than the previous one) against each source state $\bar{h}_j$:

$$\operatorname{score}(h_u, \bar{h}_j) = \begin{cases} h_u^\top \bar{h}_j & \text{dot} \\ h_u^\top W_a\, \bar{h}_j & \text{general} \end{cases} \qquad a_u(j) = \frac{\exp \operatorname{score}(h_u, \bar{h}_j)}{\sum_{j'} \exp \operatorname{score}(h_u, \bar{h}_{j'})}.$$

Here h_u is the top-layer decoder state at target step u; $\bar{h}_j$ is the encoder state at source position j; W_a is a learned matrix; and $a_u(j)$ is the attention weight. Their third option, **concat**, scores the concatenation $[h_u; \bar{h}_j]$ with a feedforward layer, as in additive attention. The context vector $c_u = \sum_j a_u(j) \bar{h}_j$ is combined with the decoder state into an **attentional hidden state**, which feeds the output softmax:

$$\tilde{h}_u = \tanh(W_c [c_u; h_u]), \qquad p(y_u \mid y_{<u}, \mathbf{x}) = \operatorname{softmax}(W_s \tilde{h}_u).$$

Here $[c_u; h_u]$ is vector concatenation and W_c, W_s are learned matrices. Luong et al. also proposed **local** attention, which first predicts an aligned source position and attends only to a window around it, and **input feeding**, which passes $\tilde{h}_u$ into the next decoder step so the model knows its earlier alignment decisions [5].

The "general" score is the bridge to Transformers. Write $q_u = h_u$ and $k_j = W_a^\top \bar{h}_j$; then the score is just $q_u^\top k_j$. Once every score is a dot product between a **query** and a **key**, a whole table of scores is

a single matrix product, which is what modern hardware does best.

4. Queries, keys and values: from alignment to self-attention

Plain version. Generalize the decoder state into a **query** (what this position is looking for), the source states into **keys** (what each position offers for matching) and **values** (what each position passes on if it is read). Attention then works between any two sets of vectors, including a sequence and itself.

In additive and multiplicative attention, the same encoder state is used both for matching and for the weighted sum. The Transformer separates these roles with three learned projections [6]. If the queries come from the decoder and the keys and values come from the encoder, the operation is **cross-attention**, the direct descendant of Bahdanau and Luong attention. If queries, keys and values all come from the same sequence, it is **self-attention**: every token builds a new representation of itself by reading the other tokens of its own sentence.

Self-attention produces **contextual representations**. In "the crane lifts steel" and "the crane eats fish," the embedding lookup for `crane` is the same, but after self-attention each occurrence has read different neighbors, so the two output vectors can differ. This is the representation idea that NLP 10 develops into BERT. Reuse of such representations requires the matching tokenizer and vocabulary: an embedding row has meaning only relative to the model's token IDs, and swapping in a different tokenizer without adapting the weights turns valid-looking numbers into unrelated symbols.

5. Scaled dot-product attention

Plain version. Each position asks which other positions contain useful information. Attention converts those comparisons into weights and combines the corresponding value vectors.

For one attention head:

$$Q = HW_Q, \quad K = HW_K, \quad V = HW_V, \qquad A = \mathrm{softmax}_{\mathrm{row}}\left(\frac{QK^\top}{\sqrt{d_k}} + B\right), \quad O = AV.$$

Here $H \in \mathbb{R}^{T\times d}$ contains input representations; $W_Q, W_K \in \mathbb{R}^{d\times d_k}$ are learned query and key projections; $W_V \in \mathbb{R}^{d\times d_v}$ is the value projection; Q, K, V are projected matrices; d_k and d_v are key and value widths; $B \in \mathbb{R}^{T\times T}$ is an additive visibility mask, with entries 0 (allowed) or $-\infty$ (blocked); A contains row-normalized attention weights; and $O \in \mathbb{R}^{T\times d_v}$ is the output. The softmax is applied separately to each query row. This is the mechanism introduced in the Transformer paper [6].

The scale factor prevents dot products from growing simply because the key vector has more dimensions under common initialization assumptions: if query and key components are independent with mean zero and unit variance, $q^\top k$ has variance d_k, and dividing by $\sqrt{d_k}$ restores unit variance so the softmax does not saturate (ML 04 works through this). It does not turn attention

weights into calibrated semantic importance. A large weight indicates a large contribution within this computation, after learned projections. It is not a proof that the attended token caused the final answer or supports the generated statement.

Code: attention from scratch on a toy sentence

To see what the equations do without a training run, we build the vectors by hand. Each token of "the pump stopped because it overheated" gets a four-feature vector (device noun, pronoun, event verb, function word), and the sentence is padded with one <pad> slot as it would be in a batch. The projections are chosen so that pronouns and verbs **look for** devices, nouns look for events, and a pronoun weakly **offers** device information. Nothing is learned; the point is to watch scores, masking and averaging work.

```
import numpy as np

tokens = ["the", "pump", "stopped", "because", "it", "overheated", "<pad>"]
# Hand-built features: [device noun, pronoun, event verb, function word]
X = np.array([[0, 0, 0, 1], [1, 0, 0, 0], [0, 0, 1, 0], [0, 0, 0, 1],
              [0, 1, 0, 0], [0, 0, 1, 0], [0, 0, 0, 0]], dtype=float)
# Query space: [looking for a device, looking for an event]
W_Q = 3.0 * np.array([[0, 1], [1, 0], [1, 0], [0, 0]])   # nouns seek events; pronouns,
 ↪  verbs seek devices
W_K = np.array([[1, 0], [0.5, 0], [0, 1], [0, 0]])       # devices offer "device"; a pronoun
 ↪  weakly so
W_V = np.eye(4)                                          # pass the features through
 ↪  unchanged

def attention(X, mask):
    Q, K, V = X @ W_Q, X @ W_K, X @ W_V
    scores = Q @ K.T / np.sqrt(K.shape[1])
    scores = np.where(mask, scores, -np.inf)              # blocked pairs get zero weight
    A = np.exp(scores - scores.max(axis=1, keepdims=True))
    A /= A.sum(axis=1, keepdims=True)
    return A @ V, A

T = len(tokens)
real = np.array([t != "<pad>" for t in tokens])
padding_mask = np.broadcast_to(real, (T, T))              # every query may read real keys
causal_mask = padding_mask & np.tri(T, dtype=bool)        # ...and only earlier ones
out, A = attention(X, padding_mask)
_, A_causal = attention(X, causal_mask)
it, pump = tokens.index("it"), tokens.index("pump")
print("weights of 'it':", " ".join(f"{t}={w:.2f}" for t, w in zip(tokens, A[it])))
print("output of 'it' (device, pronoun, event, function):", out[it].round(2))
print("'pump' reads 'stopped': bidirectional", A[pump, 2].round(2),
      "| causal", A_causal[pump, 2].round(2))
X2 = X.copy(); X2[-1] = [5, 5, 5, 5]                      # garbage in the padding slot
out2, _ = attention(X2, padding_mask)
print("padding changes real outputs:", not np.allclose(out[real], out2[real]))
assert np.allclose(A.sum(axis=1), 1) and A[:, -1].max() == 0
```

Output:

```
weights of 'it': the=0.07 pump=0.55 stopped=0.07 because=0.07 it=0.19 overheated=0.07
 ↪  <pad>=0.00
output of 'it' (device, pronoun, event, function): [0.55 0.19 0.13 0.13]
'pump' reads 'stopped': bidirectional 0.4 | causal 0.0
padding changes real outputs: False
```

The pronoun's new vector is mostly the device feature it borrowed from `pump`: after one attention step, `it` carries information about its likely antecedent. The padding column receives exactly zero weight, so even garbage stored in the padding slot cannot change a real token's output. Under the causal mask, `pump` can no longer read the later verb `stopped`. Real models learn the projections; here we wrote them by hand, and the example deliberately contains only one device, so it says nothing about how a trained model resolves harder references (that is a coreference problem, treated in NLP 12).

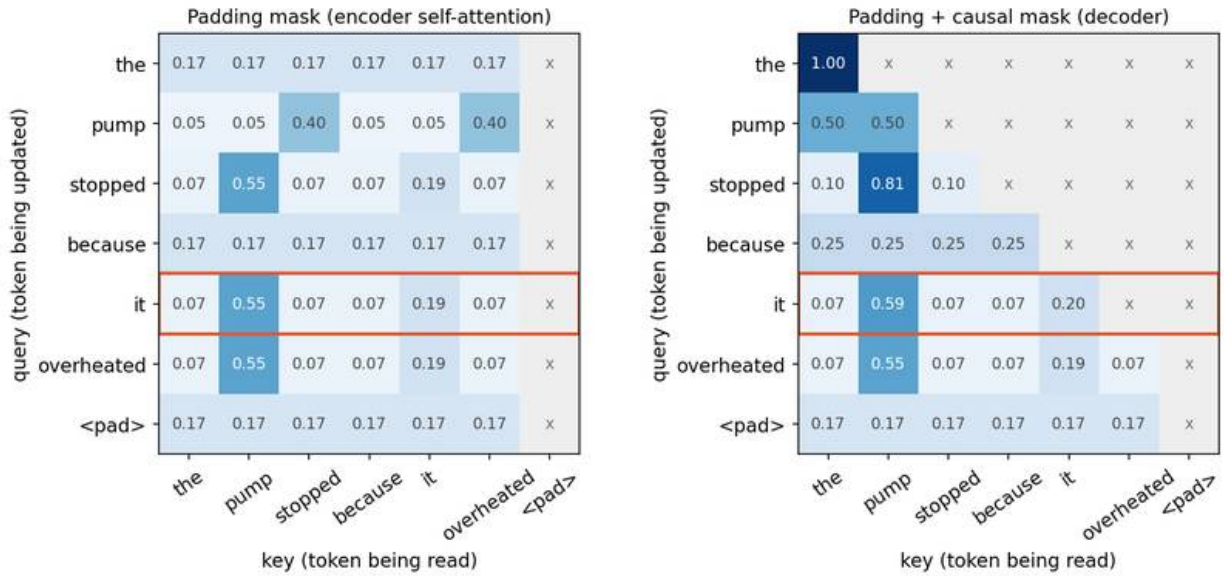


Figure 1 — The hand-built toy example. Left: padding mask only, as in an encoder. Right: padding plus causal mask, as in a decoder. Grey cells marked x are blocked; the orange row is the pronoun. The weights come from hand-chosen projections, not from a trained model.

Notice the **`<pad>` row** in the figure: a padding query still produces a (meaningless) output, because its keys are not all blocked. That is the safe convention. If a query row had every key blocked, its softmax would divide zero by zero and produce `NaN`, which can then spread through later layers. Mask padding **keys**, and ignore padding **outputs** in the loss.

6. Multi-head attention

Plain version. Run several smaller attention operations in parallel, each with its own projections, so that different heads can match on different relationships. Then concatenate their outputs and mix them with one more projection.

With h heads, head r has its own projections $W_Q^{(r)}, W_K^{(r)}, W_V^{(r)} \in \mathbb{R}^{d \times d/h}$:

$$\text{head}_r = \text{Attention}\left(HW_Q^{(r)}, HW_K^{(r)}, HW_V^{(r)}\right), \qquad \text{MHA}(H) = [\,\text{head}_1, \ldots, \text{head}_h\,]\, W_O.$$

Here Attention is the scaled dot-product operation of Section 5 with $d_k = d_v = d/h$; the brackets concatenate the h outputs (each $T \times d/h$) back to width d; and $W_O \in \mathbb{R}^{d \times d}$ is the output projection. The original Transformer used $h = 8$ heads with model width $d = 512$ [6]. Because each head is h times narrower, multi-head attention costs about as much as one full-width head, and its four projection matrices hold $4d^2$ weights (plus biases), the same count as in ML 04.

One head forms one weighted average per query. If a verb needs both its subject and its object, a single softmax must split its weight between them, blurring both. Several heads can each concentrate on one relationship. Do not over-read this: heads are not guaranteed to learn human-readable roles, and some are redundant.

Two later variants change only how keys and values are shared. **Multi-query attention** uses one key head and one value head for all query heads, which shrinks the memory read at every decoding step [15]. **Grouped-query attention** shares each key/value head among a group of query heads, an intermediate point between the two [16]. Both matter mainly for fast autoregressive decoding, discussed in NLP 09.

7. Self-attention and cross-attention

Plain version. The same operation connects tokens within one sequence (self-attention) or connects one sequence to another (cross-attention). The only change is where the queries, keys and values come from.

Mechanism	Queries from	Keys and values from	Typical use in NLP
Bidirectional self-attention	the sequence	the same sequence, all real tokens	encoders such as BERT; the source side of a translator
Causal self-attention	the sequence	the same sequence, current and earlier positions	decoder-only language models; the target side of a translator
Cross-attention	the decoder	the encoder outputs	encoder–decoder translation and summarization

For cross-attention, let $H_{\text{dec}} \in \mathbb{R}^{T_y \times d}$ be decoder states and $M \in \mathbb{R}^{T_x \times d}$ the encoder outputs (often called the **memory**):

$$Q = H_{\text{dec}} W_Q, \qquad K = MW_K, \qquad V = MW_V, \qquad O = \text{softmax}_{\text{row}}\left(\frac{QK^\top}{\sqrt{d_k}} + B_{\text{src}}\right) V.$$

Here T_y and T_x are the target and source lengths; the score matrix has shape $T_y \times T_x$, one row per target position; B_{src} blocks source padding; and $O \in \mathbb{R}^{T_y \times d_v}$ has one output per target position. This is Bahdanau and Luong attention with learned query, key and value projections. No causal mask is needed here, because the source is fully known before decoding begins.

The task's available information should determine the mask, not a model name. A sentence classifier may use a bidirectional encoder because the complete sentence is available. A streaming continuation must exclude future output tokens. Translation can use the entire source after it arrives, but simultaneous translation, which starts speaking before the source sentence ends, needs a more restrictive source visibility rule.

8. Masks: padding and causal

Plain version. A mask is a table that says which query may read which key. Padding masks hide artificial filler tokens; causal masks hide the future.

For causal self-attention, $B_{ij} = 0$ when $j \le i$ and $B_{ij} = -\infty$ otherwise. Here i is the query position and j is the candidate key position. With shifted targets, the current input position is an allowed prefix token, so the diagonal remains visible. A fully bidirectional self-attention layer permits every real input position, although padding can still be blocked. For a padding mask, $B_{ij} = -\infty$ whenever key j is a padding position, for every query i. Masks combine by blocking a pair if either mask blocks it.

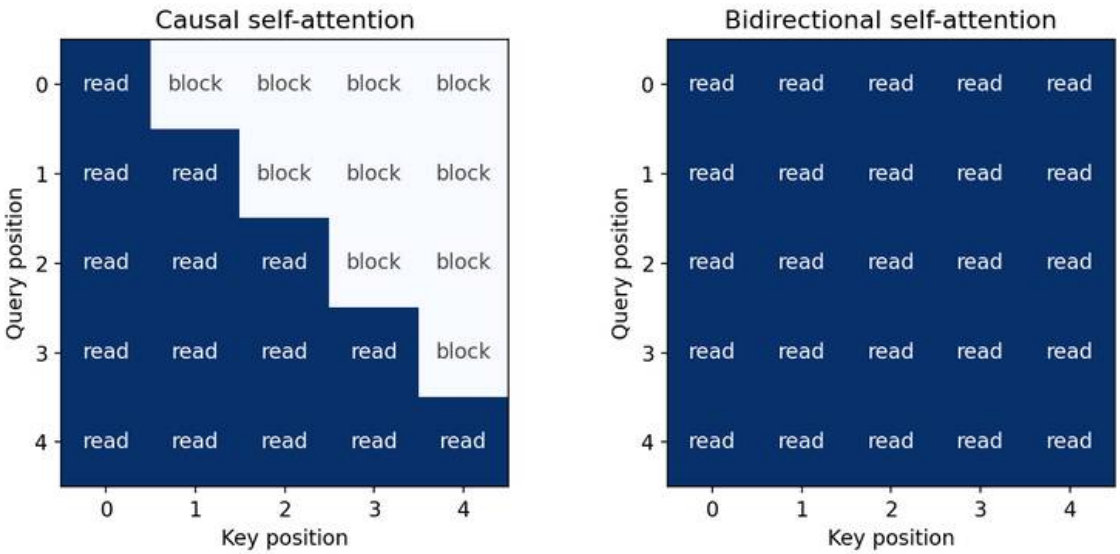


Figure 2 — Visibility is part of the task definition. Rows are querying positions and columns are the positions they may read.

Two different masks are easy to confuse. An **attention mask** controls what representations may read. A **loss mask** controls which target positions contribute to the training objective. Padding needs both: padded keys must not contribute context, and padded targets must not contribute loss. They solve different problems and should be tested independently.

Inspect attention and test its information flow

This standalone matrix example separates learned scores from the visibility rule. All vectors and projections are synthetic, so the weights have no linguistic interpretation.

```
import numpy as np

rng = np.random.default_rng(0)
T, d = 5, 8                                  # 5 tokens, 8-dimensional embeddings
X = rng.normal(size=(T, d))                  # token embeddings (+ positions)
Wq, Wk, Wv = (rng.normal(size=(d, d)) / np.sqrt(d) for _ in range(3))

def attention(X, causal):
    Q, K, V = X @ Wq, X @ Wk, X @ Wv
    scores = Q @ K.T / np.sqrt(d)            # (T, T) query-key similarities
    if causal:                               # position t may only look at <= t
        scores = np.where(np.tri(T, dtype=bool), scores, -np.inf)
    w = np.exp(scores - scores.max(axis=1, keepdims=True))
    w /= w.sum(axis=1, keepdims=True)        # softmax over keys
    return w @ V, w

out, w = attention(X, causal=True)
print(np.round(w, 2))

# Changing the LAST token must not change any earlier causal output...
X2 = X.copy(); X2[-1] += 5.0
out2, _ = attention(X2, causal=True)
print("causal: earlier outputs unchanged:", np.allclose(out[:-1], out2[:-1]))
# ...but in a bidirectional encoder every position can see it.
b1, _ = attention(X, causal=False); b2, _ = attention(X2, causal=False)
print("bidirectional: earlier outputs unchanged:", np.allclose(b1[:-1], b2[:-1]))
```

Output:

```
[[1.   0.   0.   0.   0.  ]
 [0.88 0.12 0.   0.   0.  ]
 [0.42 0.29 0.29 0.   0.  ]
 [0.28 0.24 0.29 0.19 0.  ]
 [0.34 0.06 0.24 0.11 0.26]]
causal: earlier outputs unchanged: True
bidirectional: earlier outputs unchanged: False
```

The causal weights are lower-triangular: the first token can only read itself, the last can read everything. The numerical checks illustrate the mask's visibility rule. More generally, row t only aggregates value vectors from positions at most t. If its queries, keys and values were computed from states that also exclude future information, future inputs cannot affect that output. This condition must hold throughout the network, including any preprocessing that mixes positions. The same mask permits parallel evaluation of all next-token training losses while keeping each prediction restricted to its own prefix.

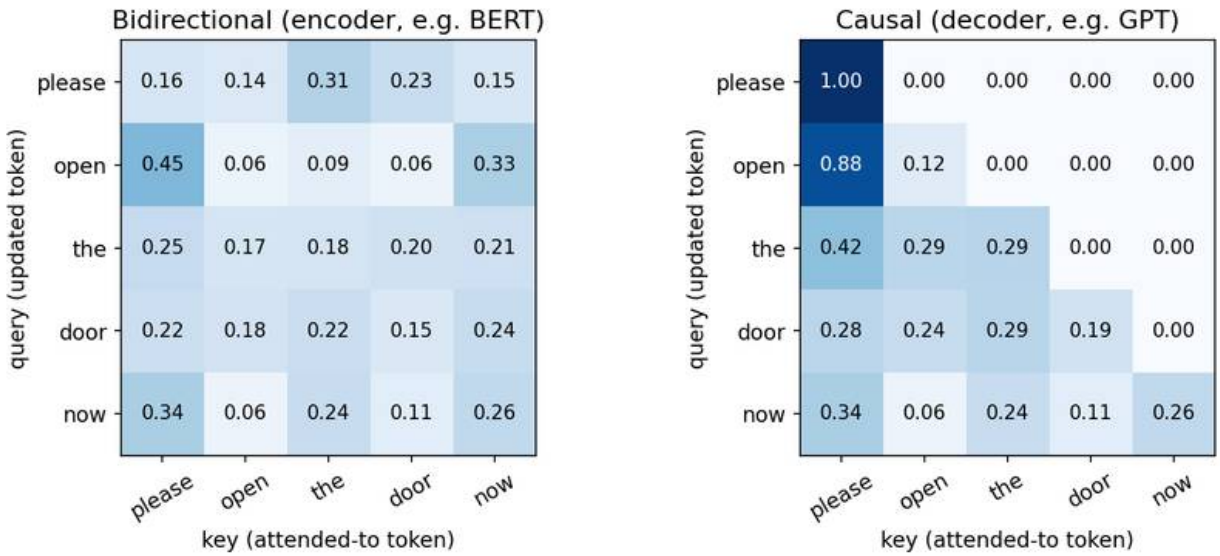


Figure 3 — Attention weights from the code above (random projections, so the values carry no linguistic meaning). The mask, not the weights, decides what each token may read.

A perturbation test like this one is worth keeping in any real code base. Low training loss cannot reveal a leaking mask, because a model that can see the answer also achieves low loss; only a test that changes a future input and checks earlier outputs targets the visibility contract directly.

9. Position: telling attention where tokens are

Plain version. Attention compares token representations, and position information tells it where those tokens occur. A token's identity alone does not say whether it is the subject or the final word.

Without position information, self-attention is **permutation-equivariant**: shuffle the input tokens and the outputs are shuffled the same way, because nothing in the formula refers to order. "Dog bites man" and "man bites dog" would produce the same set of vectors. The encoder-block code in Section 10 checks this property and shows that adding positions breaks it.

The simplest fix adds a position vector to each token embedding before the first layer. For a learned embedding and position table:

$$\mathbf{h}_i^{(0)} = E[x_i] + P[i].$$

Here $\mathbf{h}_i^{(0)} \in \mathbb{R}^d$ is the initial representation at position i; E is a token embedding table; $E[x_i]$ selects the row for token x_i; P is a position table; $P[i]$ selects the position row; and d is representation width. A learned position table also imposes a maximum supported position. Giving a sequence beyond that range is not an ordinary longer input; it is an invalid lookup. Record context limits in the serving contract. Extending the table or changing positional behavior is a model change that should be evaluated, rather than assumed to preserve performance on longer documents.

The original Transformer instead used fixed **sinusoidal** encodings [6]:

$$P[i]_{2m} = \sin\left(\frac{i}{10000^{2m/d}}\right), \qquad P[i]_{2m+1} = \cos\left(\frac{i}{10000^{2m/d}}\right), \qquad m = 0, \ldots, \tfrac{d}{2} - 1.$$

Here i is the position and m indexes a pair of dimensions, each pair rotating at its own frequency. This is the same formula as in ML 04, written there as $p_{pos,2i}$; that page proves the two properties that make it useful (a fixed offset acts as a rotation, and dot products depend only on the offset) and plots the pattern. The original paper also multiplies token embeddings by $\sqrt{d}$ before adding the encoding [6]; the translation model in NLP 08 follows that convention.

Later schemes put position into the attention score itself, so that the model sees **relative** offsets:

- **Relative position representations** add learned vectors that depend on the clipped offset $j - i$ to the keys and values [9].
- **Rotary position embedding (RoPE)** rotates each query and key by an angle proportional to its position, so the score $q_i^\top k_j$ depends on positions only through $i - j$ [10].
- **ALiBi** adds a fixed, head-specific penalty proportional to the distance $i - j$ to each causal attention score, and was designed so that a model trained on short inputs can be evaluated on longer ones [11].

The choice interacts with the context-length contract. A learned absolute table has a hard maximum; relative methods remove that hard limit but do not guarantee good accuracy beyond the lengths seen in training. Evaluate at the lengths you intend to serve.

10. The Transformer encoder and decoder blocks

Plain version. A block alternates two jobs. Attention lets tokens exchange information; a small feedforward network then processes each token on its own. Residual connections and normalization keep a deep stack trainable. The decoder adds one more job between them: reading the encoder through cross-attention.

The original **encoder block** [6] wraps each sublayer in a residual connection followed by layer normalization (**post-norm**):

$$U = \mathrm{LN}(H + \mathrm{MHA}(H)), \qquad H' = \mathrm{LN}(U + \mathrm{FFN}(U)), \qquad \mathrm{FFN}(u) = W_2 \max(0, W_1 u + b_1) + b_2.$$

Here $H \in \mathbb{R}^{T \times d}$ is the block input; MHA is multi-head self-attention with a padding mask; LN is layer normalization, which standardizes each token's d features and applies a learned scale and

shift [7]; FFN is applied to each position separately with $W_1 \in \mathbb{R}^{d_{\text{ff}} \times d}$, $W_2 \in \mathbb{R}^{d \times d_{\text{ff}}}$ and a hidden width d_{ff} (2048 for $d = 512$ in the original); and H' is the block output. Many later models move the normalization **before** each sublayer (**pre-norm**, $H + \text{MHA}(\text{LN}(H))$). Xiong et al. analyzed why post-norm needs a learning-rate warm-up while pre-norm can train without one [8]. So "a Transformer" is not fully specified until you say where normalization goes.

The **decoder block** has three sublayers, each with its own residual connection and normalization:

1. **Masked self-attention** over the target prefix, with a causal mask (and a padding mask for batches).
2. **Cross-attention**, with queries from step 1 and keys and values from the final encoder output, masked only for source padding.
3. A position-wise **feedforward network**.

Stacking N encoder blocks and N decoder blocks gives the full model ($N = 6$ in the original [6]). Every decoder block reads the same final encoder output. A linear layer and a softmax over the target vocabulary turn the last decoder state into $p(y_t \mid y_{<t}, \mathbf{x})$. Dropping the encoder and the cross-attention leaves a **decoder-only** model, the shape of most current large language models (NLP 09); dropping the decoder leaves an **encoder-only** model such as BERT (NLP 10).

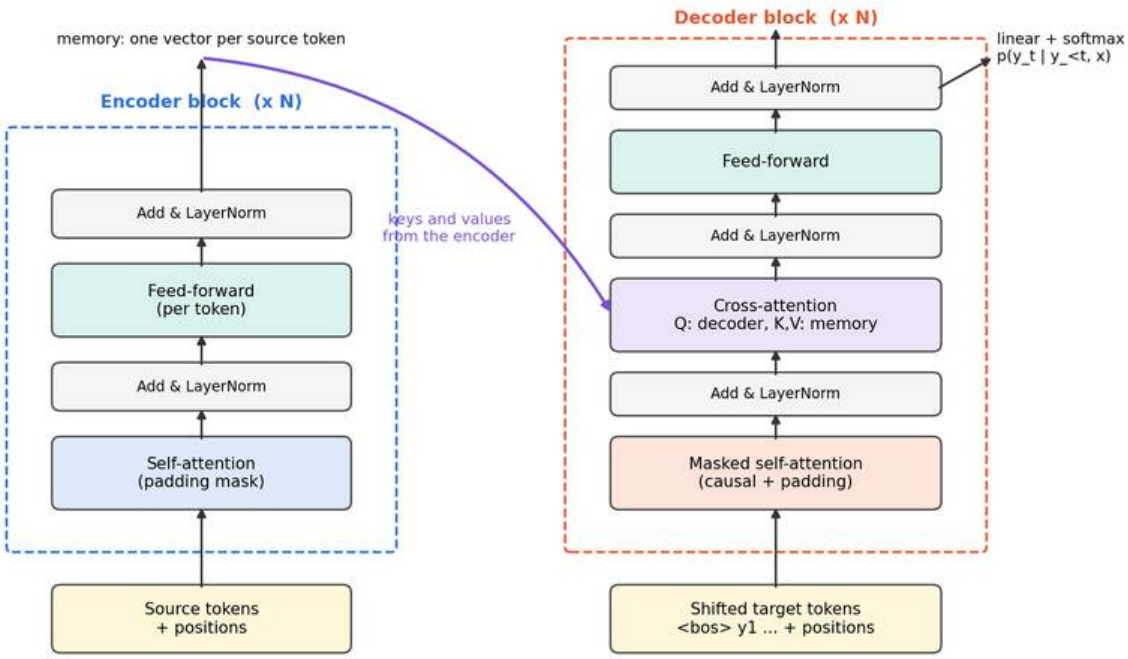


Figure 4 — The original post-norm Transformer encoder and decoder blocks. Residual connections around each sublayer are folded into the Add and LayerNorm boxes. The purple arrow is cross-attention: every decoder block reads the final encoder output.

Code: a tiny Transformer encoder block from scratch

The following code implements multi-head self-attention and a post-norm encoder block directly from the equations, borrows the randomly initialized weights of PyTorch's `nn.TransformerEncoderLayer` [21], and checks that both give the same output. It then tests three properties: padding is ignored, the block is permutation-equivariant without positions, and sinusoidal positions break that symmetry. Dropout is off and the reference layer is in evaluation mode so the comparison is exact.

```
import math
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0)
B, T, d, h, d_ff = 2, 6, 16, 4, 64
ref = nn.TransformerEncoderLayer(d, h, d_ff, dropout=0.0, activation="relu",
                                 batch_first=True, norm_first=False).eval()

def multi_head_attention(x, key_padding, p):
    q, k, v = (x @ p.in_proj_weight.T + p.in_proj_bias).chunk(3, dim=-1)
    split = lambda t: t.view(B, T, h, d // h).transpose(1, 2)        # (B, h, T, d/h)
    q, k, v = split(q), split(k), split(v)
    scores = q @ k.transpose(-2, -1) / math.sqrt(d // h)               # (B, h, T, T)
    scores = scores.masked_fill(key_padding[:, None, None, :], float("-inf"))
    heads = scores.softmax(dim=-1) @ v                                  # (B, h, T, d/h)
    concat = heads.transpose(1, 2).reshape(B, T, d)                     # heads side by side
    return concat @ p.out_proj.weight.T + p.out_proj.bias

def encoder_block(x, key_padding, layer=ref):
    # Post-norm, as in the original Transformer: x <- LN(x + Sublayer(x))
    a = multi_head_attention(x, key_padding, layer.self_attn)
    x = F.layer_norm(x + a, (d,), layer.norm1.weight, layer.norm1.bias, layer.norm1.eps)
    f = layer.linear2(F.relu(layer.linear1(x)))
    return F.layer_norm(x + f, (d,), layer.norm2.weight, layer.norm2.bias, layer.norm2.eps)

x = torch.randn(B, T, d)
pad = torch.tensor([[False] * 6, [False] * 4 + [True] * 2])     # second sentence: 4 real
 ↪  tokens
with torch.no_grad():
    ours, theirs = encoder_block(x, pad), ref(x, src_key_padding_mask=pad)
    print("matches nn.TransformerEncoderLayer:", torch.allclose(ours[~pad], theirs[~pad],
     ↪  atol=1e-5))
    x2 = x.clone(); x2[1, 4:] = 100.0                               # garbage in padded slots
    print("padding is ignored:", torch.allclose(encoder_block(x2, pad)[~pad], ours[~pad],
     ↪  atol=1e-5))
    perm = torch.tensor([3, 0, 5, 1, 4, 2])
    none = torch.zeros(B, T, dtype=torch.bool)
    y, yp = encoder_block(x, none), encoder_block(x[:, perm], none)
    print("no positions -> permutation-equivariant:", torch.allclose(yp, y[:, perm],
     ↪  atol=1e-5))
    pos = torch.arange(T)[:, None] / 10000 ** (torch.arange(0, d, 2) / d)
```

```
    pe = torch.stack([pos.sin(), pos.cos()], dim=-1).flatten(1)     # sin/cos interleaved
    y, yp = encoder_block(x + pe, none), encoder_block(x[:, perm] + pe, none)
    print("with positions -> still equivariant:", torch.allclose(yp, y[:, perm], atol=1e-5))
print("parameters:", sum(p.numel() for p in ref.parameters()))
```

Output:

```
matches nn.TransformerEncoderLayer: True
padding is ignored: True
no positions -> permutation-equivariant: True
with positions -> still equivariant: False
parameters: 3280
```

The comparison uses only real tokens: in evaluation mode PyTorch may take a fused path that does not compute meaningful values at padded positions, and those outputs are ignored anyway. The parameter count checks the formulas: attention has $4d^2 + 4d = 1088$ weights and biases, the feedforward network $2d\,d_{\text{ff}} + d_{\text{ff}} + d = 2128$, and the two layer norms $4d = 64$. Notice where the parameters live: in this block, as in the original Transformer, the feedforward network holds most of them.

The model has no positional information until we add it, which is exactly what the third and fourth checks show. In a real model, the position vectors are added once, at the input of the first block; every later block inherits order information through the residual stream.

11. Cost and long-context variants

Plain version. Every query compares itself with every key, so the work and the score table grow with the square of the sequence length. That is cheap for a sentence and expensive for a book.

For a sequence of length T and width d, one self-attention layer computes $QK^\top$ and AV in about T^2d multiply–adds each, plus about $4Td^2$ for the projections. A recurrent layer costs about Td^2 but needs T **sequential** steps, whereas all positions of a self-attention layer can be computed in parallel, and any two positions are connected by a path of length one. Vaswani et al. summarize this trade-off in their comparison of layer types [6]. For short sentences (T smaller than d), self-attention is the cheaper of the two; for long documents the T^2 term dominates.

Direct attention score storage grows quadratically with sequence length. Specialized implementations can avoid materializing every intermediate matrix, but the modeling question remains which positions need to interact. In deployment, benchmark memory and latency under the actual context lengths rather than extrapolating from a short sentence.

Approaches to long contexts fall into a few families. Tay et al.'s survey organizes them as fixed patterns, learnable patterns, low-rank and kernel methods, memory, recurrence and downsampling [17]. Representative examples:

Idea	Example	What changes
Restrict which pairs interact	Longformer: a sliding local window plus a few global tokens [12]	cost grows linearly with T for a fixed window; distant tokens communicate through global tokens or several layers
Replace the softmax kernel	linear attention with a feature map [13]	scores factorize, giving linear cost and a recurrent view of causal attention; the attention function is different
Keep exact attention, move less memory	FlashAttention: tiled, IO-aware computation [14]	same result as standard attention, less memory traffic; quadratic arithmetic remains
Share keys and values across heads	multi-query and grouped-query attention [15, 16]	smaller key/value cache during decoding
Position schemes that extend	relative, rotary, ALiBi [9–11]	removes a hard positional limit; accuracy at unseen lengths must still be measured

These are not interchangeable. FlashAttention computes the same function as standard attention; Longformer and linear attention compute different functions and need to be evaluated as different models. State-space and recurrent alternatives that avoid pairwise scores entirely are discussed in ML 03A and ML 04.

Modern view

Plain version. The attention block of 2017 is still recognizable in current models; most changes concern normalization placement, position handling, key/value sharing and efficient kernels, not the core equation.

Surveys. Lin, Wang, Liu and Qiu's survey of Transformer variants (*AI Open*, 2022) first describes the original model, then proposes a taxonomy and reviews variants from three perspectives: architectural modification, pretraining, and applications [18]. It is a good map for locating a variant ("is this a change to attention, to position, or to the block?"). Tay, Dehghani, Bahri and Metzler's *Efficient Transformers: A Survey* (*ACM Computing Surveys*, 2022) focuses on the memory and compute of attention and classifies efficient models by mechanism (the families in Section 11) [17]. Its practical message is that efficiency claims depend on sequence length, hardware and implementation, so compare models at the lengths you need.

Attention as explanation. Jain and Wallace reported that attention weights in their models often did not agree with gradient-based importance measures and that very different attention distributions could yield the same prediction [19]. Wiegreffe and Pinter argued that whether attention can serve as an explanation depends on what is meant by explanation, and proposed tests for it [20]. Taken together: an attention map shows where a computation read from, which is useful for debugging masks and alignments, but it is not by itself evidence for why a model made a decision.

State of practice, 2023–2026. Grouped-query attention [16] and IO-aware exact attention ker-

nels [14] address the two costs that dominate serving: key/value memory during decoding and memory traffic during attention. Relative position methods such as RoPE [10] and ALiBi [11] are designed to remove the hard maximum length of a learned position table. Encoder–decoder Transformers remain a natural fit for tasks with a clear source and target, such as translation (NLP 08), while decoder-only models dominate open-ended generation (NLP 09).

Open problems. Reliable use of very long contexts (a longer window permits more input but does not guarantee that all of it is used), length generalization beyond training lengths, the cost of attention over long documents, and faithful interpretation of what attention heads compute.

Key takeaways

- A single context vector forces the whole source through a fixed-size bottleneck; attention lets every output step read every source position.
- Bahdanau attention scores with a small feedforward network; Luong's dot and general scores reduce attention to matrix products, which leads directly to queries, keys and values.
- Scaled dot-product attention is $\text{softmax}(QK^\top/\sqrt{d_k}+B)V$; multi-head attention runs h narrower copies at about the cost of one.
- Self-attention mixes a sequence with itself; cross-attention lets a decoder read the encoder. The mask, not the weights, decides what each position may read.
- Mask padding keys, mask the future in causal models, keep loss masks separate, and test visibility with a perturbation check.
- Attention ignores order; learned, sinusoidal, relative, rotary or ALiBi positions supply it, each with a different length contract.
- Attention cost grows with T^2; long-context variants either change the function (sparse, linear) or compute the same function more efficiently (FlashAttention).

Exercises

1. Remove the causal mask from the information-flow example in Section 8. Which printed check changes, and why could a language model trained without the mask still show a low training loss?

   Hint

   The changed final token becomes visible to earlier positions. Low loss can reward access to information that is unavailable during generation.

2. In the toy-sentence code, add a second device noun ("valve") before the pronoun. What happens to the weights of `it`? What would a model need, beyond these hand-built features, to choose the right antecedent?

   Hint

With identical features, the two devices receive equal scores. Position information, the verb's meaning and discourse context must enter the query or key to break the tie.

3. Design attention and loss masks for a batch of two target sequences of lengths 3 and 5, padded to 5, in a causal decoder. Write the 5×5 blocked/allowed table for each sequence and the list of target positions that contribute to loss.

Hint

Padding keys must not contribute context, and padding targets must not contribute loss. Decide how a padding query row is handled without creating an all-blocked softmax row.

4. For a model with $d = 512$, $h = 8$ and $d_{\text{ff}} = 2048$, count the weights in one encoder block and one decoder block (ignore biases and layer norms). What fraction of the encoder block is in the feedforward network?

Hint

Self-attention and cross-attention each have $4d^2$ weights; the feedforward network has $2d\,d_{\text{ff}}$. The decoder has one more attention sublayer than the encoder.

5. For sequence lengths $T = 128$, 1,024 and 16,384 with $d = 512$, compare $T^2 d$ (attention scores) with $4Td^2$ (projections). At which length does the score term start to dominate?

Hint

The two terms are equal when $T = 4d$. Below that, the projections cost more than the score computation.

6. Explain why Luong's "general" score $h_u^\top W_a \bar{h}_j$ is a special case of scaled dot-product attention. What would the query, key and value projections be?

Hint

Absorb W_a into the key projection. The value projection is the identity, and the scale factor is 1.

References

1. Masato Hagiwara, *Real-World Natural Language Processing: Practical Applications with Deep Learning*, Chapter 8, Manning, 2021. Publisher ·Chapter 8
2. I. Sutskever, O. Vinyals and Q. V. Le, "Sequence to Sequence Learning with Neural Networks," NeurIPS, 2014. Paper

3. K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk and Y. Bengio, “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” EMNLP, 2014. Paper
4. D. Bahdanau, K. Cho and Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” ICLR, 2015. Paper
5. M.-T. Luong, H. Pham and C. D. Manning, “Effective Approaches to Attention-based Neural Machine Translation,” EMNLP, 2015. Paper
6. A. Vaswani et al., “Attention Is All You Need,” NeurIPS, 2017. Paper
7. J. L. Ba, J. R. Kiros and G. E. Hinton, “Layer Normalization,” arXiv:1607.06450, 2016. Paper
8. R. Xiong et al., “On Layer Normalization in the Transformer Architecture,” ICML, 2020. Paper
9. P. Shaw, J. Uszkoreit and A. Vaswani, “Self-Attention with Relative Position Representations,” NAACL, 2018. Paper
10. J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo and Y. Liu, “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing* 568, 2024. DOI
11. O. Press, N. A. Smith and M. Lewis, “Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation,” ICLR, 2022. Paper
12. I. Beltagy, M. E. Peters and A. Cohan, “Longformer: The Long-Document Transformer,” arXiv:2004.05150, 2020. Paper
13. A. Katharopoulos, A. Vyas, N. Pappas and F. Fleuret, “Transformers are RNNs: Fast Autoregressive Transformers with Linear Attention,” ICML, PMLR 119, 2020. Paper
14. T. Dao, D. Y. Fu, S. Ermon, A. Rudra and C. Ré, “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” NeurIPS, 2022. Paper
15. N. Shazeer, “Fast Transformer Decoding: One Write-Head is All You Need,” arXiv:1911.02150, 2019. Paper
16. J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón and S. Sanghai, “GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints,” EMNLP, 2023. Paper
17. Y. Tay, M. Dehghani, D. Bahri and D. Metzler, “Efficient Transformers: A Survey,” *ACM Computing Surveys* 55(6), 2022. DOI
18. T. Lin, Y. Wang, X. Liu and X. Qiu, “A Survey of Transformers,” *AI Open* 3, 2022. DOI
19. S. Jain and B. C. Wallace, “Attention is not Explanation,” NAACL, 2019. Paper
20. S. Wiegreffe and Y. Pinter, “Attention is not not Explanation,” EMNLP-IJCNLP, 2019. Paper
21. PyTorch, `TransformerEncoderLayer` documentation. Documentation

NLP 08 • Sequence-to-Sequence Models with Transformers (Machine Translation, Chatbots)

Build and train an encoder–decoder Transformer for translation, decode it with greedy and beam search, score it with BLEU written from scratch, and carry the same conditional-generation ideas into dialogue state, clarification and conversation evaluation.

Online edition: https://blog.shyandram.dev/en/learn/nlp/nlp-08-seq2seq-transformers/

What you'll learn

- How machine translation moved from statistical phrase tables to neural encoder–decoders, and what the Transformer changed.
- Conditional sequence learning: target shifting, teacher forcing, padding-aware loss, and why exposure bias appears at generation time.
- How to train a small encoder–decoder Transformer translator on CPU, and decode it with greedy and beam search with length normalization.
- How BLEU works (written from scratch), what chrF and COMET add, and why none of them certifies meaning.
- How the same conditional-generation view applies to chatbots, and why task dialogue also needs explicit state, clarification and tool boundaries.
- How to evaluate whole conversations rather than isolated replies.

The big picture

A sequence-to-sequence (seq2seq) model reads one sequence and writes another: a French sentence from an English one, a reply from a conversation history, a summary from a document. The encoder turns the input into one vector per token; the decoder writes the output one token at a time, reading its own prefix through causal self-attention and the input through cross-attention (NLP 07). The same machinery serves translation and dialogue, but the two tasks fail differently. A translation can be fluent and reverse a negation. A chatbot can produce a friendly reply while querying the wrong device.

> **Explain like I'm five**
>
> Think of an interpreter at a meeting. First they listen to the whole sentence (the encoder). Then they speak the translation word by word, and before each word they glance back at the part of the speech they are translating (cross-attention). A good interpreter can also say "do you mean Pump A or Pump B?" when the speaker is unclear. Speaking smoothly is not the same as being right; the interpreter is judged by whether the listener ends up with the same meaning.

Two complementary readings anchor the chapter. Hagiwara's Chapter 6 explains sequence-to-sequence models for translation and conversational applications, and Chapter 8 covers attention and Transformers [1]. McRoy's Chapter 9 discusses translation as meaning-preserving generation [2]. The examples here are original, local CPU experiments; they teach mechanisms without downloading a model or a dataset. NLP 09 continues with open-ended generation, summarization and serving.

1. From statistical to neural machine translation

Plain version. Machine translation first learned word and phrase correspondences from aligned sentence pairs and combined them with a separate model of fluent target text. Neural translation replaced that pipeline with one network trained end to end.

Statistical machine translation (SMT) framed translation as a **noisy channel**. To translate a foreign sentence $\mathbf{f}$ into English $\mathbf{e}$, choose

$$\hat{\mathbf{e}} = \arg\max_{\mathbf{e}} \; p(\mathbf{e})\, p(\mathbf{f} \mid \mathbf{e}).$$

Here $p(\mathbf{e})$ is a language model that rewards fluent English; $p(\mathbf{f} \mid \mathbf{e})$ is a translation model that rewards faithfulness; and $\hat{\mathbf{e}}$ is the chosen translation. Brown et al.'s IBM models estimated word-level translation probabilities and alignments from sentence-aligned text [3]. Koehn, Och and Marcu's phrase-based translation used multi-word phrase pairs, which capture local reordering and idioms better than single words [4]. Such systems combined many separately trained components and features.

Neural networks entered as components first: Cho et al. used an RNN encoder–decoder to score phrase pairs inside an existing SMT system [5]. Sutskever, Vinyals and Le then trained an LSTM encoder–decoder to translate directly [6]. Bahdanau, Cho and Bengio added attention so the decoder could search the source at each step [7], and Vaswani et al. replaced recurrence with attention entirely [8]. Subword segmentation such as byte-pair encoding made open vocabularies practical, so rare words and names could be translated as sequences of known pieces [9]; NLP 02 builds a BPE vocabulary step by step. Stahlberg's review traces this transition from count-based SMT to neural MT in detail [22].

Two recent anchors show where the field went. The NLLB project built a single massively multilingual model covering 200 languages, with particular attention to low-resource languages [21]. The WMT24 general translation task compared submitted systems with several large language models and online providers under professional human evaluation; its title summarizes the finding: the LLM era is here, but MT is not solved [20].

2. Conditional sequence learning

Plain version. A translator models the probability of the target sentence given the source, one token at a time. Training shows it the correct previous target tokens; generation makes it use its own.

A conditional sequence model writes

$$p_\theta(\mathbf{y} \mid \mathbf{x}) = \prod_{t=1}^{U} p_\theta(y_t \mid y_{<t}, \mathbf{x}).$$

Here $\mathbf{x}$ is the source sequence; $\mathbf{y} = (y_1, \dots, y_U)$ is the target sequence, including an end token; U is its length; $y_{<t}$ contains the tokens before position t; and θ denotes learned parameters. The source can be visible in full while the target remains causal. This factorization describes a distribution. It does not certify that a sampled output is supported by the source.

Training usually supplies the correct preceding target tokens, a procedure called teacher forcing [6]. For a batch, the token-level loss is

$$\mathcal{L}(\theta) = -\frac{1}{M} \sum_{b=1}^{B} \sum_{t=1}^{T_b} m_{bt} \log p_\theta(y_{bt} \mid y_{b,<t}, \mathbf{x}_b).$$

Here B is the number of examples; T_b is the padded target length for example b; m_{bt} is one for a real target token and zero for padding; $M = \sum_{b,t} m_{bt}$ is the number of real tokens; and $\mathbf{x}_b$, y_{bt} and $y_{b,<t}$ denote the source, current target token and reference prefix. The logarithm is natural. The mask removes padding from the loss; it does not remove it from computation automatically.

Check the target shift carefully. The decoder input begins with a start token, while the target begins with the first output token. The end token belongs in the target so the model learns when to stop. Padding is separate from the end token: otherwise training can reward stopping in the wrong place. When batching sentences of different lengths, record whether you average over tokens or examples; those choices assign different weights to long sentences.

Data construction is often more consequential than a small architectural change. A translation pair containing a silently corrected number trains the model to change that number. Keep source versions, annotation instructions and target provenance together. Group splits by document, conversation or event so near-duplicate inputs cannot enter both training and testing.

Teacher forcing and exposure bias

Teacher forcing presents the correct prefix even when the model would have generated a different one. Suppose the reference begins `Pump A`, but generation selects `Pump B`. Later tokens can form a perfectly fluent statement about Pump B. Reference-conditioned loss still evaluates those later positions after `Pump A`; it never tests recovery from this particular generated mistake. The distinction is between prediction under a supplied history and behavior under the model's own evolving history.

Ranzato et al. named this training/inference mismatch **exposure bias**: the model is exposed only to ground-truth prefixes during training, but must consume its own predictions at test time, so errors can accumulate [10]. Two responses illustrate the design space. **Scheduled sampling** gradually replaces some ground-truth decoder inputs with the model's own predictions during training [11]. **Sequence-level training** optimizes a score of whole generated sequences, such as BLEU, rather than only next-token likelihood [10]. Neither removes the need to evaluate full generated

outputs; maximum likelihood with teacher forcing remains the standard starting point, and the factorization itself is exact. ML 08 discusses the same mismatch for image generation.

Token averaging also hides unequal semantic consequences. Replacing `the` may slightly affect style; replacing `not`, a device ID or a unit can reverse the result. Each token contributes a log-loss term, while the application may assign very different costs to these changes. A lower average loss can coexist with more critical errors if common wording improves while a rare distinction deteriorates. Several outputs can also correctly express the source: a single reference supplies one acceptable wording, not the complete set.

3. The encoder–decoder Transformer for translation

Plain version. The encoder reads the whole source with bidirectional self-attention. The decoder writes the target with causal self-attention and reads the encoder with cross-attention. A final linear layer turns each decoder state into a distribution over target tokens.

The original Transformer [8] is assembled from the blocks of NLP 07:

1. **Inputs.** Source and target text are segmented into subwords [9] (NLP 02). Each token ID is embedded, scaled by $\sqrt{d}$ and added to a sinusoidal position encoding.
2. **Encoder.** N blocks of self-attention and feedforward sublayers, with a padding mask on source keys, produce the **memory** $M \in \mathbb{R}^{T_x \times d}$, one vector per source token.
3. **Decoder.** N blocks of masked self-attention (causal plus target padding), cross-attention to M (source padding masked), and feedforward sublayers.
4. **Output.** A linear layer and softmax give $p_\theta(y_t \mid y_{<t}, \mathbf{x})$ at every target position.

During training, all target positions are computed in one pass, because the causal mask makes each position see only its own prefix. During generation, the encoder runs once, and the decoder runs once per output token. Practical decoders cache the keys and values of earlier target positions instead of recomputing them; NLP 09 analyzes that cache.

Two training details from the original paper are worth knowing. The model was trained with **label smoothing**: the target distribution puts most of its mass on the reference token and spreads a small amount over the rest of the vocabulary [8, 29]. With smoothing parameter ε and vocabulary V, the training target is $q(v) = (1-\varepsilon)\mathbf{1}\{v = y_t\} + \varepsilon/|V|$; here $\mathbf{1}\{\cdot\}$ is one when its condition holds. This discourages extreme confidence; Vaswani et al. report that it hurt perplexity but improved BLEU [8]. The model also used a learning-rate warm-up, which Xiong et al. later linked to the post-norm layout (NLP 07).

4. Decoding: greedy search, beam search and length normalization

Plain version. A trained model gives a probability for every possible next token. A decoder must turn those probabilities into one output sentence. Greedy decoding takes the best next token each time; beam search keeps several partial sentences alive.

The sequence score for a candidate is

$$\ell(\mathbf{y} \mid \mathbf{x}) = \sum_{t=1}^{U} \log p_\theta(y_t \mid y_{<t}, \mathbf{x}).$$

Here ℓ is the log probability of the complete sequence; U includes the end token; and the remaining symbols are defined above. Exact maximization over all sequences is intractable because the number of candidates grows exponentially with length. **Greedy decoding** chooses the most probable token at each step. **Beam search** with width k keeps the k highest-scoring prefixes, extends each by every token, and again keeps the best k; finished hypotheses (ending in the end token) are set aside. These choices control search and variation; none makes unsupported content reliable.

Greedy decoding can miss the best complete sequence: a first token with slightly lower probability may lead to a much more probable continuation, which a beam keeps alive long enough to discover. NLP 09 works through a constructed example of this, the bound that raw log probabilities give on unfinished prefixes, the difference between search error and model error, and sampling-based decoding. Here we focus on what translation adds: length normalization, and a measured greedy-versus-beam comparison on a trained translator (Section 6).

Length normalization

Raw log probability has a built-in preference. Every additional token contributes a nonpositive term, so longer hypotheses tend to score lower even when they are better translations. Wu et al. describe this bias in Google's neural MT system and rank finished hypotheses by a normalized score [12]:

$$s(\mathbf{y}, \mathbf{x}) = \frac{\ell(\mathbf{y} \mid \mathbf{x})}{\mathrm{lp}(\mathbf{y})}, \qquad \mathrm{lp}(\mathbf{y}) = \frac{(5 + |\mathbf{y}|)^\alpha}{(5+1)^\alpha}.$$

Here $|\mathbf{y}|$ is the hypothesis length in tokens; $\alpha \geq 0$ controls the strength of normalization, with $\alpha = 0$ recovering the raw score and $\alpha = 1$ approximating a per-token average; and s is the score used to rank finished hypotheses. Their full scoring function also adds a coverage penalty based on attention, which we omit. They tuned α on development data [12]; treat it as a hyperparameter, not a constant.

```
candidates = {                          # per-token log-probabilities, end token included
    "stop <eos>": [-0.9, -1.1],
    "stop pump b now <eos>": [-0.9, -0.5, -0.3, -0.4, -0.2],
}
def length_penalty(n, alpha):           # Wu et al. (2016): ((5 + n) / 6) ** alpha
    return ((5 + n) / 6) ** alpha
for alpha in [0.0, 0.6, 1.0]:
    scores = {c: sum(lp) / length_penalty(len(lp), alpha) for c, lp in candidates.items()}
    best = max(scores, key=scores.get)
    print(f"alpha={alpha}:", {c: round(s, 3) for c, s in scores.items()}, "->", best)
```

Output:

```
alpha=0.0: {'stop <eos>': -2.0, 'stop pump b now <eos>': -2.3} -> stop <eos>
alpha=0.6: {'stop <eos>': -1.823, 'stop pump b now <eos>': -1.693} -> stop pump b now <eos>
alpha=1.0: {'stop <eos>': -1.714, 'stop pump b now <eos>': -1.38} -> stop pump b now <eos>
```

The raw score prefers the truncated instruction; $\alpha = 0.6$ and $\alpha = 1$ prefer the complete one (a small α such as 0.2 would still prefer the short one). Whether that is right depends on the source, which the score does not see.

Normalization also changes the search itself: with raw scores, a finished hypothesis that beats every unfinished prefix can be returned safely, because extending a prefix can only lower its raw score; with a length-normalized score that argument no longer holds, so the stopping rule must account for the penalty.

Set explicit output limits, a stopping rule and behavior when the limit is reached. A truncated translation should not be returned as complete. Record the decoding parameters with the model so an evaluation can be reproduced.

5. Evaluating translations: BLEU, chrF and COMET

Plain version. Automatic metrics compare a system output with one or more human reference translations. They are cheap and reproducible, which makes them useful for comparing systems, but a tiny word change can reverse meaning without changing the score much.

BLEU

BLEU compares n-grams of the candidate with those of the references [13]. For each order $n = 1, \dots, 4$ it computes a **modified (clipped) precision**: each candidate n-gram is counted at most as many times as it appears in the reference where it is most frequent. Over a test corpus,

$$p_n = \frac{\sum_{\text{sent}} \sum_{g \in G_n} \min\left(c(g),\, c_{\text{ref}}^{\max}(g)\right)}{\sum_{\text{sent}} \sum_{g \in G_n} c(g)}, \qquad \text{BP} = \begin{cases} 1 & c > r \\ e^{1 - r/c} & c \le r \end{cases}, \qquad \text{BLEU} = \text{BP} \cdot \exp\Big(\sum_{n=1}^{4} w_n \log p_n \Big).$$

Here G_n is the set of distinct n-grams in a candidate sentence; $c(g)$ is the count of n-gram g in the candidate; $c_{\text{ref}}^{\max}(g)$ is its largest count in any single reference for that sentence; c is the total candidate length over the corpus; r is the **effective reference length**, the sum over sentences of the reference length closest to each candidate's length; BP is the **brevity penalty**, which stops a system from gaining precision by writing very short outputs; and $w_n = 1/4$ are uniform weights. BLEU is a corpus-level score: the precisions are pooled over all sentences before the logarithm, so it is not the average of sentence scores.

```
import math
from collections import Counter
```

```
def ngrams(tokens, n):
    return Counter(tuple(tokens[i:i + n]) for i in range(len(tokens) - n + 1))

def corpus_bleu(hypotheses, references, max_n=4):
    """hypotheses: list of token lists; references: list of lists of token lists."""
    matched, total = [0] * max_n, [0] * max_n
    hyp_len = ref_len = 0
    for hyp, refs in zip(hypotheses, references):
        hyp_len += len(hyp)
        # effective reference length: the reference length closest to the hypothesis
        ref_len += min((abs(len(r) - len(hyp)), len(r)) for r in refs)[1]
        for n in range(1, max_n + 1):
            counts = ngrams(hyp, n)
            max_ref = Counter()
            for r in refs:
                max_ref |= ngrams(r, n)            # elementwise max over references
            matched[n - 1] += sum((counts & max_ref).values())   # clipped counts
            total[n - 1] += max(len(hyp) - n + 1, 0)
    if min(matched) == 0:
        return 0.0, matched, total
    log_p = sum(math.log(m / t) for m, t in zip(matched, total)) / max_n
    bp = 1.0 if hyp_len > ref_len else math.exp(1 - ref_len / hyp_len)
    return bp * math.exp(log_p), matched, total

refs = [["the cat is on the mat".split(), "there is a cat on the mat".split()]]
_, m, t = corpus_bleu(["the the the the the the the".split()], refs, max_n=1)
print("clipped unigram precision:", f"{m[0]}/{t[0]}")
ref = "pump a is not operating at 30 c".split()
for hyp in ["pump a is not operating at 30 c",
            "pump a is operating at 30 c",
            "pump a is not running at 30 c"]:
    score, _, _ = corpus_bleu([hyp.split()], [[ref]])
    print(f"{score:.3f}  {hyp}")
assert m[0] == 2 and t[0] == 7
assert corpus_bleu([ref], [[ref]])[0] == 1.0
```

Output:

```
clipped unigram precision: 2/7
1.000  pump a is not operating at 30 c
0.515  pump a is operating at 30 c
0.500  pump a is not running at 30 c
```

The first check reproduces the classic clipping example: a candidate that repeats “the” seven times gets credit for only two, because no reference contains “the” more than twice [13]. The second check is the warning. Dropping `not` reverses the meaning, yet scores higher than a correct paraphrase that replaces `operating` with `running`. BLEU measures surface overlap with the references; it has no notion of which words carry the proposition.

This implementation uses whitespace tokens, no smoothing (any zero n-gram precision gives zero), and the closest-reference-length rule. Published BLEU scores depend on such choices.

Post showed that differences in tokenization and normalization alone can change reported BLEU substantially, and introduced sacreBLEU, which computes the score on detokenized text with a standard internal tokenizer and prints a version signature [14]. Report that signature with any BLEU number; scores computed with different settings are not comparable.

chrF

chrF scores **character** n-grams instead of word n-grams [15]:

$$\mathrm{chrF}_\beta = (1 + \beta^2)\,\frac{\mathrm{chrP} \cdot \mathrm{chrR}}{\beta^2\,\mathrm{chrP} + \mathrm{chrR}}.$$

Here chrP and chrR are character n-gram precision and recall, averaged over n-gram orders; and β weights recall β times as much as precision. Popović found 6-grams worked best in her experiments and compared $\beta = 1$ with $\beta = 3$ [15]. Character n-grams give partial credit for shared stems and inflections, which helps for morphologically rich target languages, but chrF is still an overlap score: it cannot tell that `not` matters more than `the`.

Learned metrics: COMET

COMET trains a neural model to predict human quality judgments from the source, the candidate and a reference, using a cross-lingual pretrained encoder [16]. It can reward a correct paraphrase that shares few words with the reference. In the WMT22 metrics task, evaluated against expert MQM (Multidimensional Quality Metrics) annotations, neural learned metrics correlated better with human ratings than overlap metrics such as BLEU and chrF [18]. Learned metrics bring their own model versions, training distributions and language coverage; their outputs are measurements under those conditions. Check the metric on the same kinds of negation, quantity and terminology errors that matter in your application.

A metric that ranks systems is not automatically an output gate

The WMT25 evaluation findings separate segment-quality prediction, error-span annotation and quality-informed correction; they report that system-level and segment-level evaluation can favor different approaches, and that linguistic diversity remains difficult [19]. This matters because rejecting one faulty translation is a different decision from ranking average model quality. A metric can reward a generally fluent system while missing rare reversed negations in individual outputs. The acceptance threshold needs validation on those errors, rather than being inherited from an aggregate correlation result.

Cross the generator's error categories with the evaluator's inputs. A reference-only comparison can inspect wording agreement but lacks the source needed to verify an unsupported statement. A source-aware evaluator has that evidence, yet can still share the generator's scope or quantity mistake. Inspect disagreement cases: correct paraphrases marked poor, fluent contradictions marked good, and accurate but incomplete translations. Include human review with the original

source and separate adequacy from style. This analysis identifies which part of an evaluation signal is useful for model comparison and which part can responsibly inform acceptance of a particular result.

Overlap, recall and human judgment

The same counterexample affects recall-oriented overlap, which ROUGE uses for summarization [17]. For a simple unigram recall check,

$$R_1 = \frac{\sum_w \min(c_{\text{ref}}(w), c_{\text{hyp}}(w))}{\sum_w c_{\text{ref}}(w)}.$$

Here w ranges over word types; $c_{\text{ref}}(w)$ and $c_{\text{hyp}}(w)$ are reference and hypothesis word counts. The denominator is the number of reference tokens. This minimal implementation does not reproduce every preprocessing or aggregation option of a full evaluation package.

```
import re
from collections import Counter

def words(text):
    return re.findall(r"[a-z0-9]+", text.lower())
def overlap_recall(reference, hypothesis):
    ref, hyp = Counter(words(reference)), Counter(words(hypothesis))
    return sum((ref & hyp).values()) / sum(ref.values())
reference = "Pump A is operating at 30 C."
contradiction = "Pump A is not operating at 30 C."
score = overlap_recall(reference, contradiction)
print("Unigram recall:", score)
assert score == 1.0
assert ("not" in words(reference)) != ("not" in words(contradiction))
```

The contradiction receives recall 1.0 because it retains every reference token. A precision term would penalize the extra word, but the small penalty would still not express the importance of reversed negation. This is a controlled counterexample, not evidence that overlap metrics never correlate with human judgments. It explains why the metric must be paired with meaning-sensitive checks.

Human evaluation should hide system identity, randomize presentation, preserve source context and specify the decision task. Freitag et al.'s large MQM study, with professional translators who could see the full document, ranked systems substantially differently from the crowd-worker ratings of the same shared task [30]. Ask reviewers to mark specific errors before judging overall quality, report who reviewed the outputs and in which languages, and keep an adjudicated challenge set separate from routine samples.

6. Hands-on: a tiny Transformer translator

Plain version. We invent a small language pair, train an encoder–decoder Transformer on 3,000 sentence pairs for under a minute on a CPU, and compare greedy and beam decoding with the BLEU function above.

The source language is English-like: subject, verb, object, with optional adjectives before nouns and an optional `never`. The target language is invented: every word has a fixed translation, adjectives follow their nouns, the object is followed by the particle `o`, the verb comes last, and negation becomes the suffix word `nai`. For example, `the small drone never lifts a red cable` becomes `la drono pico un kablo rosso o levi nai`. Translating therefore requires both a dictionary and reordering, the two things that made word-by-word translation inadequate. Training and test sentences are disjoint, but they come from the same small grammar, so this is an in-distribution test.

The model uses PyTorch's encoder and decoder layers (two of each, width 64, four heads), scaled embeddings plus sinusoidal positions, boolean masks (`True` means blocked), and `ignore_index` to drop padded targets from the loss, implementing the masked loss of Section 2. Run it after the BLEU block, because it calls `corpus_bleu`.

```
# Run after the BLEU block above: this code calls its corpus_bleu function.
import math, random, time
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0); torch.set_num_threads(1); rng = random.Random(0)
lexicon = {"the": "la", "a": "un", "robot": "robo", "sensor": "senso",
           "pump": "pompa", "valve": "valva", "drone": "drono", "cable": "kablo",
           "red": "rosso", "blue": "blu", "big": "grando", "small": "pico",
           "sees": "vidi", "checks": "kontrolli", "moves": "movi",
           "opens": "aperi", "lifts": "levi"}
DET, NOUN = ["the", "a"], ["robot", "sensor", "pump", "valve", "drone", "cable"]
ADJ, VERB = ["red", "blue", "big", "small"], ["sees", "checks", "moves", "opens", "lifts"]

def noun_phrase():
    adj = [rng.choice(ADJ)] if rng.random() < 0.5 else []
    return rng.choice(DET), adj, rng.choice(NOUN)

def pair():          # source: SVO, adjective before noun; target: SOV, adjective after noun
    (d1, a1, n1), (d2, a2, n2) = noun_phrase(), noun_phrase()
    verb, neg = rng.choice(VERB), rng.random() < 0.3
    src = [d1, *a1, n1] + (["never"] if neg else []) + [verb, d2, *a2, n2]
    np_ = lambda d, a, n: [lexicon[d], lexicon[n]] + [lexicon[x] for x in a]
    tgt = np_(d1, a1, n1) + np_(d2, a2, n2) + ["o", lexicon[verb]] + (["nai"] if neg else [])
    return " ".join(src), " ".join(tgt)

pairs = list(dict(pair() for _ in range(6000)).items())   # unique source sentences
train_pairs, test_pairs = pairs[:3000], pairs[3000:3200]
PAD, BOS, EOS = 0, 1, 2
src_vocab = ["<pad>", "<bos>", "<eos>"] + sorted({w for s, _ in pairs for w in s.split()})
```

```
tgt_vocab = ["<pad>", "<bos>", "<eos>"] + sorted({w for _, t in pairs for w in t.split()})
s_id = {w: i for i, w in enumerate(src_vocab)}
t_id = {w: i for i, w in enumerate(tgt_vocab)}

def batch(texts, ids, L):
    rows = [[ids[w] for w in t.split()] + [EOS] for t in texts]
    return torch.tensor([r + [PAD] * (L - len(r)) for r in rows])
S = batch([s for s, _ in train_pairs], s_id, 10)
Y = batch([t for _, t in train_pairs], t_id, 12)
Y_in = torch.cat([torch.full((len(Y), 1), BOS), Y[:, :-1]], dim=1)   # shift right

def sinusoidal(n, d):
    pos = torch.arange(n)[:, None]
    omega = 1.0 / 10000 ** (torch.arange(0, d, 2) / d)
    pe = torch.zeros(n, d)
    pe[:, 0::2], pe[:, 1::2] = torch.sin(pos * omega), torch.cos(pos * omega)
    return pe

class TinyMT(nn.Module):
    def __init__(self, d=64):
        super().__init__()
        self.d = d
        self.src_emb = nn.Embedding(len(src_vocab), d)
        self.tgt_emb = nn.Embedding(len(tgt_vocab), d)
        self.register_buffer("pe", sinusoidal(32, d))
        enc = nn.TransformerEncoderLayer(d, 4, 128, dropout=0.0, batch_first=True)
        dec = nn.TransformerDecoderLayer(d, 4, 128, dropout=0.0, batch_first=True)
        self.encoder = nn.TransformerEncoder(enc, 2, enable_nested_tensor=False)
        self.decoder = nn.TransformerDecoder(dec, 2)
        self.out = nn.Linear(d, len(tgt_vocab))
    def embed(self, emb, x):
        return emb(x) * math.sqrt(self.d) + self.pe[: x.shape[1]]
    def encode(self, src):
        return self.encoder(self.embed(self.src_emb, src), src_key_padding_mask=src.eq(PAD))
    def decode(self, memory, src, y_in):
        T = y_in.shape[1]
        causal = torch.ones(T, T, dtype=torch.bool).triu(1)      # True = blocked
        h = self.decoder(self.embed(self.tgt_emb, y_in), memory, tgt_mask=causal,
                         tgt_key_padding_mask=y_in.eq(PAD),
                         memory_key_padding_mask=src.eq(PAD))
        return self.out(h)

@torch.no_grad()
def greedy(src_text, max_len=14):
    src = batch([src_text], s_id, len(src_text.split()) + 1)
    memory, y = model.encode(src), [BOS]
    for _ in range(max_len):
        logp = model.decode(memory, src, torch.tensor([y]))[0, -1].log_softmax(-1)
        y.append(int(logp.argmax()))
        if y[-1] == EOS:
            break
    return [tgt_vocab[i] for i in y[1:] if i != EOS]

@torch.no_grad()
```

```
def beam_search(src_text, width=4, alpha=0.6, max_len=14):
    src = batch([src_text], s_id, len(src_text.split()) + 1)
    memory = model.encode(src)
    beams, finished = [([BOS], 0.0)], []
    lp = lambda n: ((5 + n) / 6) ** alpha             # GNMT length penalty
    for _ in range(max_len):
        prefixes = torch.tensor([y for y, _ in beams])
        logp = model.decode(memory.expand(len(beams), -1, -1), src.expand(len(beams), -1),
                            prefixes)[:, -1].log_softmax(-1)
        cand = [(y + [t], s + logp[b, t].item()) for b, (y, s) in enumerate(beams)
                for t in logp[b].topk(width).indices.tolist()]
        cand.sort(key=lambda c: c[1], reverse=True)
        beams = []
        for y, s in cand:
            (finished if y[-1] == EOS else beams).append((y, s))
            if len(beams) == width:
                break
        if not beams:
            break
    best, _ = max(finished or beams, key=lambda c: c[1] / lp(len(c[0]) - 1))
    return [tgt_vocab[i] for i in best[1:] if i != EOS]

@torch.no_grad()
def log_prob(src_text, words):                        # model score of a full output
    src = batch([src_text], s_id, len(src_text.split()) + 1)
    y = [t_id[w] for w in words] + [EOS]
    logp = model.decode(model.encode(src), src, torch.tensor([[BOS] +
     ↪  y[:-1]])).log_softmax(-1)
    return logp[0, torch.arange(len(y)), y].sum().item()

def evaluate():
    model.eval()
    refs, outputs = [[t.split()] for _, t in test_pairs], {}
    for name, decode in [("greedy", greedy), ("beam-4", beam_search)]:
        hyps = [decode(s) for s, _ in test_pairs]
        exact = sum(h == r[0] for h, r in zip(hyps, refs)) / len(refs)
        mean_lp = sum(log_prob(s, h) for (s, _), h in zip(test_pairs, hyps)) / len(hyps)
        print(f"  {name}: BLEU {corpus_bleu(hyps, refs)[0]:.3f}  exact {exact:.3f}"
              f"  mean log p {mean_lp:.3f}")
        outputs[name] = hyps
    print("  outputs that differ:", sum(g != b for g, b in zip(*outputs.values())))
    model.train()

model = TinyMT()
opt = torch.optim.Adam(model.parameters(), lr=1e-3)
start = time.time()
for step in range(1, 401):
    idx = torch.randint(0, len(S), (64,))
    logits = model.decode(model.encode(S[idx]), S[idx], Y_in[idx])   # teacher forcing
    loss = F.cross_entropy(logits.reshape(-1, len(tgt_vocab)), Y[idx].reshape(-1),
                           ignore_index=PAD)
    opt.zero_grad(); loss.backward(); opt.step()
    if step in (100, 400):
        print(f"after {step} steps: training loss {loss.item():.3f}")
```

```
        evaluate()
model.eval()
print(" ".join(greedy("the small drone never lifts a red cable")))
print(f"total time {time.time() - start:.0f}s")
```

Output (CPU, single thread; the timing will vary by machine, and other PyTorch versions or hardware can change the last digits):

```
after 100 steps: training loss 0.221
  greedy: BLEU 0.614  exact 0.365  mean log p -1.587
  beam-4: BLEU 0.618  exact 0.375  mean log p -1.553
  outputs that differ: 31
after 400 steps: training loss 0.003
  greedy: BLEU 1.000  exact 1.000  mean log p -0.027
  beam-4: BLEU 1.000  exact 1.000  mean log p -0.027
  outputs that differ: 0
la drono pico un kablo rosso o levi nai
total time 16s
```

Read the two checkpoints separately. After 100 steps the model is half-trained: only about a third of the 200 test translations are exactly right. Here beam search changes 31 of the 200 outputs, finds outputs with a higher average model log probability, and improves BLEU and exact match slightly. After 400 steps the model is confident and correct on this in-distribution test, and greedy and beam produce identical outputs: when the model's distribution is sharply peaked, there is nothing for a wider search to find. Beam search helps by reducing **search error**; it cannot fix **model error**, and on easy inputs it may change nothing.

BLEU and exact match move together here because the language is tiny and deterministic: there is exactly one correct translation per source. With real references, a correct translation can differ from the reference, and the two numbers diverge. The test also checks only sentences from the same grammar. It says nothing about compositional generalization, such as an adjective–noun combination never seen in training; Exercise 1 asks you to test that.

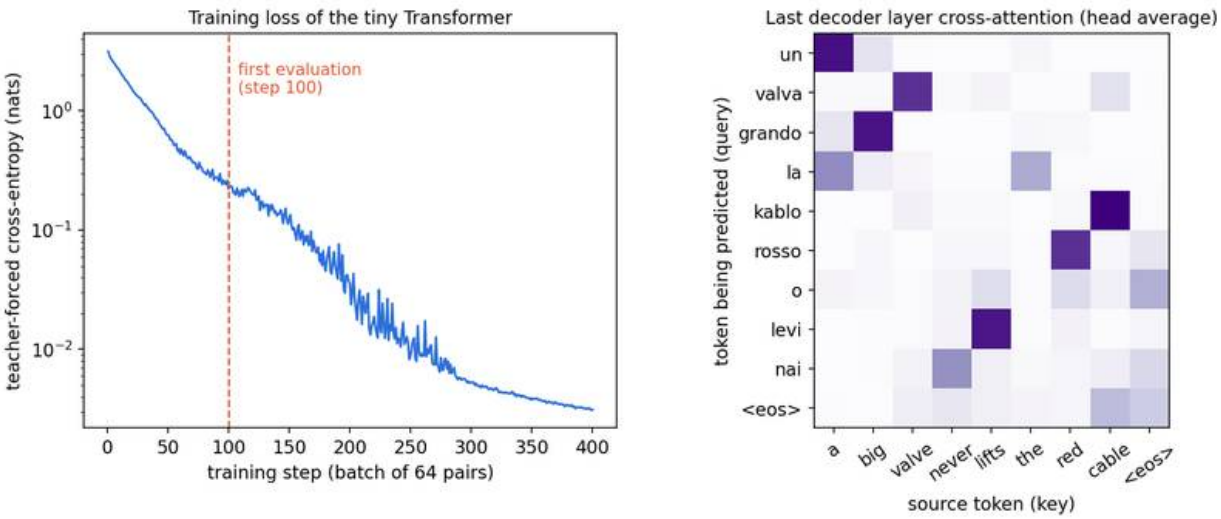


Figure 1 — The tiny translator. Left: teacher-forced training loss from the code above. Right: cross-attention of the last decoder layer, averaged over heads, for a held-out sentence it translates correctly. Each row is the decoder position that predicts the labeled word. The off-diagonal pattern is the reordering the model learned; it is a view of where the computation read, not a proof of why it is correct.

The cross-attention map in Figure 1 is the Transformer version of the soft alignments that Bahdanau et al. visualized for RNN translators [7]. Adjectives in the target read the adjectives that preceded the nouns in the source; the verb, which moves to the end, reads the source verb; and `nai` reads `never`. This alignment emerged from the translation loss alone; no alignment labels were given.

7. Translation preserves meaning across form

Plain version. A good translation changes the words and their order but keeps who did what to whom, when, how much, and whether it happened at all.

Translation is conditional generation with a strong content constraint: preserve the source meaning while producing natural target-language text. Word order, morphology, idioms and implicit context make it more than replacing each word. McRoy's translation discussion [2] pairs naturally with Hagiwara's sequence-to-sequence material [1].

Evaluate **adequacy**, whether the source meaning survives, separately from **fluency**, whether the target reads naturally. A fluent translation can omit an exception, reverse negation or change who performed an action. A literal translation may preserve technical content but sound awkward. The acceptable balance depends on the use case and must be specified before choosing a model or metric.

For a general translator, preserve terminology through a glossary and test it in context. A required term can be present while attached to the wrong entity. Likewise, keeping every number is insufficient if the values are reassigned. Build challenge cases around relations: "Pump A is

warmer than Pump B," "the valve may remain closed," and "no fault was detected before noon." Minimal edits reveal whether the model respects comparison, modality, negation and time.

Language tags and text normalization belong to the model contract. Traditional and Simplified Chinese are different output requirements; a broad language label alone does not specify script, regional terminology or punctuation. Do not convert source text destructively before storing it. Preserve the original alongside the normalized model input so reviewers can investigate a mistranslation.

In low-resource settings, uncertainty can come from limited training coverage rather than input ambiguity. Record domain, dialect, script and language pair when reporting results. A system validated on short product descriptions has not thereby been validated on long technical instructions. If reviewers cannot read both languages, they cannot directly assess adequacy; use qualified bilingual review and explain the remaining coverage gaps.

Preserve the proposition, not only its words

For a translation, compare the source and target as propositions with participants, relations and scope. "Pump A is warmer than Pump B" must keep the comparison direction. A target that contains both names and the word "warmer" can still reverse it. "No sensor exceeded the limit" differs from "Not every sensor exceeded the limit": the second allows some exceedances. NLP 12 explains why these negation scopes differ.

Ambiguity sometimes belongs to the source. `May` can express possibility or permission depending on context. If an isolated sentence does not decide between them, translating it into a form that unambiguously expresses permission adds an interpretation. Use the surrounding document or conversation when available; otherwise preserve or flag the ambiguity. A fluent target does not justify resolving an underspecified source arbitrarily.

A meaning-oriented comparison can check entity bindings, event status, quantities, units, negation and conditions individually, then assess overall readability. These checks allow lexical variation while exposing proposition changes. They also separate a true translation error from an already ambiguous source: an annotator disagreement caused by missing source context should not be interpreted as evidence that one vocabulary choice fixes the model.

8. Chatbots: conversation as sequence-to-sequence

Plain version. A chatbot can be built like a translator whose "source" is the conversation so far and whose "target" is the next reply. That works for fluent small talk, but a task assistant also needs to keep track of what has been decided.

Vinyals and Le trained a sequence-to-sequence model to predict the next turn from the previous turns, on an IT helpdesk dataset and on movie subtitles, and found it could hold simple conversations despite optimizing a likelihood objective that does not directly measure conversation quality [23]. Li et al. observed that such models tend to produce safe, generic replies such as "I don't know" regardless of the input, and proposed a maximum mutual information objective

that favors replies specific to the message [24]. The lesson carries over to modern assistants: a high-probability reply is not necessarily an informative or correct one.

For response generation, write $p_\theta(u_t \mid h_t, s_t, e_t)$, where u_t is the next system utterance, h_t is selected history, s_t is task state, e_t is evidence and θ is the model parameters. The conditional distribution says what the generator receives. It does not ensure that the generator respects every state field or evidence item.

Task-oriented dialogue has a defined outcome: supply a query, complete a booking-like record, retrieve the right observation or produce a specified report. Open conversation has less sharply defined goals and more acceptable replies. Both require consistency, but their success criteria differ. A modular task system separates understanding, state tracking, policy, evidence access and realization, which makes errors easier to locate. A learned end-to-end system can combine these functions, but it still needs tests that distinguish wrong state from poor wording. The surveys of Deriu et al. [25] and Ni et al. [26] describe the broader design and evaluation families.

A task policy can choose clarification, query, explanation or refusal of an unsupported operation. Generation then verbalizes that action. This keeps conversational fluency from deciding whether a query is ready. Open conversation benefits from flexible language, but persona or style should not override factual consistency. A system can be engaging while inventing past interactions or implying it performed an action that never occurred. Evaluate claims about the conversation against the actual log, and claims about external facts against evidence.

9. Turns, dialogue acts and state

Plain version. A conversation is a changing shared problem. The latest sentence may replace an earlier request, refer back to something, or correct the system. The system must track what has been proposed and what has actually been accepted.

A dialogue turn is a participant's contribution at a particular point in an exchange. Its function, its **dialogue act**, can be a request, an answer, a correction, a confirmation or a clarification. The same surface word can have different functions depending on the pending question: "Tuesday" may supply a date or correct an earlier date.

A task-oriented dialogue state stores the information needed to continue a specified task. For an equipment-query assistant, slots might include device, time interval, measurement and output language. Store values together with source turn, confidence and status. Unknown, explicitly absent and not applicable should not collapse into one empty string. Write a state update as

$$s_t = U(s_{t-1}, o_t, r_t), \qquad a_t = \pi(s_t).$$

Here s_t is the state after turn t; o_t is the observed utterance or extracted dialogue act; r_t identifies the speaker and other turn context; U is the update rule; a_t is the next system action; and π is the policy mapping state to actions. The state need not be a single deterministic record, and the policy need not generate text directly.

Keep proposal state separate from committed state. A user may ask for an option, inspect it and then change it. The system's own suggestion does not become a user preference merely because it

appears in the transcript. Corrections require scope: “Actually, Pump B” can replace the selected device while keeping the time interval; “start over” may discard the whole task; “not yesterday” rejects a value without necessarily specifying a replacement.

A deterministic state tracker with correction tests

A structured state tracker is a useful baseline for a narrow task. It reveals the contract between language understanding and action selection. The example consumes already extracted events; it does not claim to understand arbitrary natural language. The state stores selected slots and a revision counter. Any accepted slot change invalidates the previous confirmation, and a confirmation is valid only for the revision explicitly presented to the user. This prevents an old “yes” from authorizing a newly changed proposal.

```
from copy import deepcopy

initial = {"device": None, "period": None, "revision": 0, "confirmed": None}
def update(state, event):
    result = deepcopy(state)
    kind = event["kind"]
    if kind == "set":
        slot, value = event["slot"], event["value"]
        allowed = {"device": {"pump_a", "pump_b"}, "period": {"today", "yesterday"}}
        if slot not in allowed or value not in allowed[slot]:
            raise ValueError("Unsupported slot or value")
        if result[slot] != value:
            result[slot] = value
            result["revision"] += 1
            result["confirmed"] = None
    elif kind == "confirm":
        if event["revision"] != result["revision"]:
            raise ValueError("Confirmation refers to an old proposal")
        if any(result[k] is None for k in ["device", "period"]):
            raise ValueError("Cannot confirm incomplete state")
        result["confirmed"] = result["revision"]
    else:
        raise ValueError("Unsupported event")
    return result
s1 = update(initial, {"kind": "set", "slot": "device", "value": "pump_a"})
s2 = update(s1, {"kind": "set", "slot": "period", "value": "today"})
s3 = update(s2, {"kind": "confirm", "revision": 2})
s4 = update(s3, {"kind": "set", "slot": "device", "value": "pump_b"})
print("Before correction:", s3)
print("After correction:", s4)
assert initial["device"] is None
assert s3["confirmed"] == 2 and s4["confirmed"] is None
try:
    update(s4, {"kind": "confirm", "revision": 2})
except ValueError:
    pass
else:
```

```
    raise AssertionError("Stale confirmation was accepted")
```

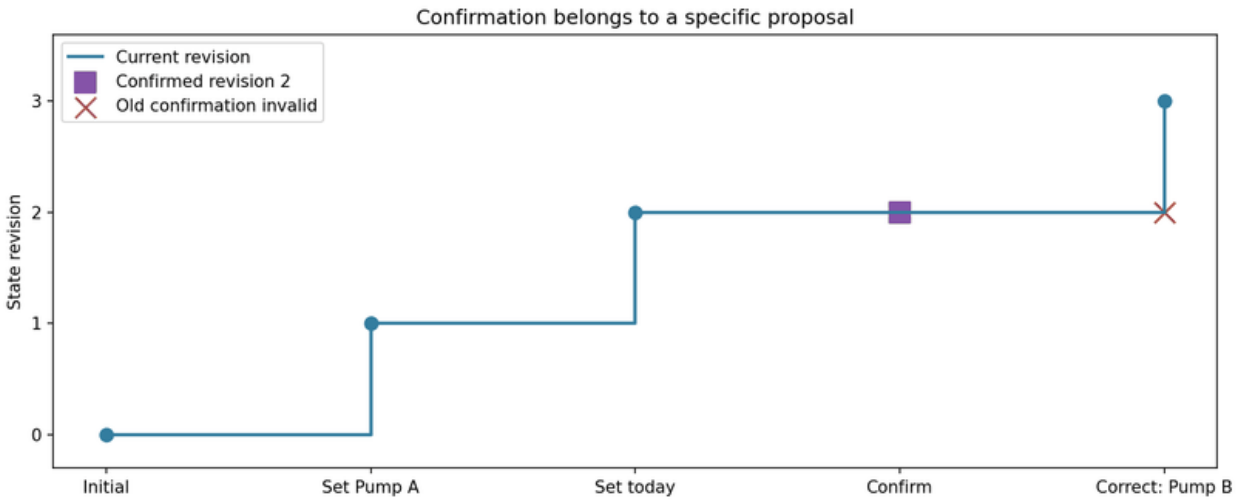


Figure 2 — Corrections change state and confirmation scope. Keeping a revision makes stale acceptance visible rather than silently applying it to a different proposal.

The copy preserves previous states for inspection and testing. A real system also needs event IDs so a repeated delivery does not apply the same update twice. Store the source turn and parsed event so an error can be traced to language understanding or state logic. Do not equate absence of a slot with a negative assertion: "no fault occurred" fills a fault-status value; an unmentioned fault leaves it unknown. The narrow allowed values make the baseline's coverage obvious. General systems can use learned slot extraction and open-vocabulary values, but should still validate units, schema and provenance. A model's confidence is not a substitute for a legal value.

10. Belief states and useful clarification

Plain version. When an utterance is ambiguous, keep several interpretations with probabilities instead of guessing. Then decide whether to act or to ask a question, depending on how costly a wrong action would be.

A simple Bayesian update is

$$b_t(h) = \frac{p(o_t \mid h) b_{t-1}(h)}{\sum_{h'} p(o_t \mid h') b_{t-1}(h')}.$$

Here h is a candidate interpretation; $b_{t-1}(h)$ is its previous probability; o_t is the new observation; $p(o_t \mid h)$ is its likelihood under the interpretation; and the denominator normalizes over candidates h'. This simplified update assumes a fixed hypothesis space and a specified observation model. A changing task needs a transition model or an explicit correction operation rather than repeatedly treating every utterance as independent evidence.

```
import numpy as np

hypotheses = ["pump_a", "pump_b", "unknown"]
prior = np.array([0.48, 0.48, 0.04])
ambiguous_likelihood = np.array([0.6, 0.5, 0.2])
posterior = prior*ambiguous_likelihood
posterior /= posterior.sum()
print("After ambiguous evidence:", dict(zip(hypotheses, posterior.round(3).tolist())))
action = "clarify" if posterior.max() < 0.9 else "query"
assert action == "clarify"
# A second synthetic observation explicitly names Pump B.
named_likelihood = np.array([0.02, 0.95, 0.03])
updated = posterior*named_likelihood
updated /= updated.sum()
print("After explicit naming:", dict(zip(hypotheses, updated.round(3).tolist())))
assert updated[1] > 0.9
assert np.isclose(updated.sum(), 1)
```

The likelihoods are constructed for the teaching example. They are not calibrated model scores or measured human behavior. In a learned system, calibration and uncertainty under distribution shift require validation. Retaining an unknown hypothesis also prevents a closed list from pretending that every real request refers to a known device.

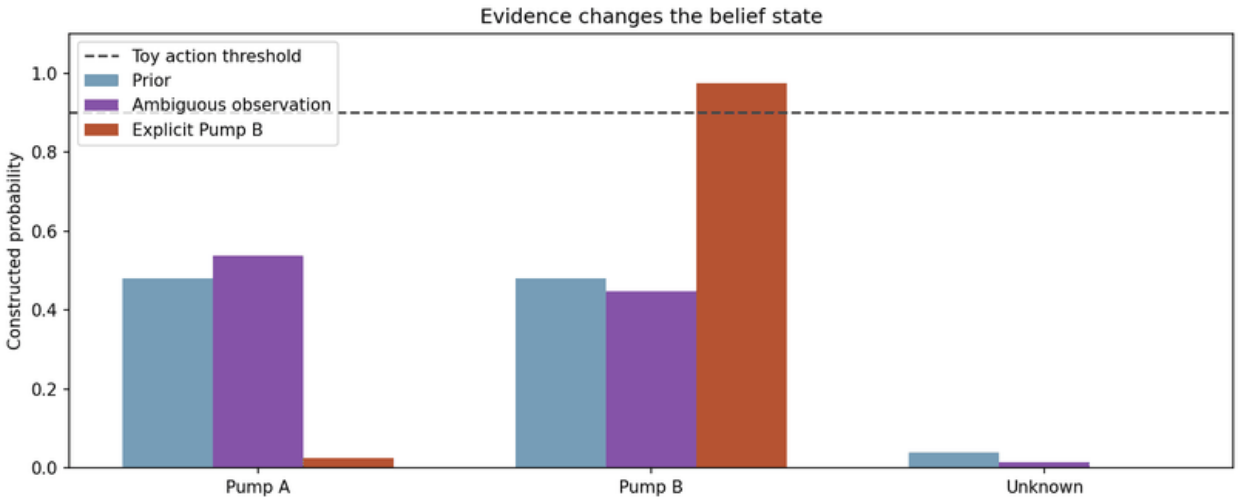


Figure 3 — A belief state preserves ambiguity until additional evidence changes the decision. The probabilities are constructed for the example and do not claim empirical calibration.

A decision-theoretic policy minimizes expected loss: $a^* = \arg\min_a \sum_h b_t(h) L(a,h)$. Here a ranges over permitted actions and $L(a,h)$ is the cost of action a when interpretation h is true. For a candidate question q, one-step expected loss is:

$$V(q) = c(q) + \sum_o p(o \mid q) \min_a \sum_h b(h \mid o, q) L(a,h).$$

Here $V(q)$ is expected loss after asking, $c(q)$ is the question's cost, o ranges over possible replies, $p(o \mid q)$ is the reply probability under the current belief, $b(h \mid o, q)$ is the posterior over interpretation h after that reply, a ranges over subsequent actions, and $L(a, h)$ is their loss. The minimum assumes the system chooses its best next action after observing the answer. Compare $V(q)$ with the minimum expected loss of acting immediately. This finite decision calculation is a teaching model, not a calibrated description of human replies.

```
import numpy as np

prior = np.array([0.55, 0.45])  # true device A or B
reply_given_device = np.array([[0.8, 0.2], [0.2, 0.8]])
question_cost = 1.0
for wrong_cost in [10.0, 2.0]:
    immediate = wrong_cost * (1 - prior.max())
    expected_after_question = question_cost
    for reply in range(2):
        mass = prior * reply_given_device[:, reply]
        probability = mass.sum()
        posterior = mass / probability
        expected_after_question += probability * wrong_cost * (1 - posterior.max())
    action = 'clarify' if expected_after_question < immediate else 'query A'
    print(wrong_cost, round(immediate, 2), round(expected_after_question, 2), action)
assert np.allclose(reply_given_device.sum(axis=1), 1)
```

The outputs compare `4.5` with `3.0` and choose clarification at wrong-query cost `10`; they compare `0.9` with `1.4` and query A at cost `2`. The belief and question are unchanged. The consequence of an error changes the best action. A fixed confidence threshold hides this dependence, while expected loss makes it explicit. Ask the question that distinguishes the uncertainty that matters: if only the device is ambiguous, do not request the already established time interval again.

Corrections require a different distinction. "Actually, Pump B" can change the requested device, rather than provide evidence that Pump B was intended all along. The old belief concerned the earlier state; the new utterance changes the state itself. A probabilistic tracker needs state transitions as well as observation likelihoods, or an explicit update rule that recognizes corrections.

11. Context, memory and tool boundaries

Plain version. A long conversation does not fit, or does not need to fit, into every model input. What is kept must keep its status: a retracted request is not an active one. And when a chatbot can call tools, saying "I did it" must be backed by an actual result.

Partition context into recent turns, structured task state and retrieved evidence. Recent turns support local reference and tone; state stores accepted slots and unresolved questions; evidence supports factual claims. A summary should not promote an unverified model response into authoritative evidence. A useful memory record includes a value, its source, time, scope and status

(proposed, accepted or retracted). A rolling window may contain "yes" while excluding the proposal it answered; a summary may retain a device but omit that it was replaced by another. Test both with synthetic conversations, and compare the reconstructed state with an explicit event log.

A tool-connected dialogue system generates proposals that software may execute. The proposal, execution and result are distinct events. A sentence saying "I queried the database" should be supported by a successful query result, not merely by the model having generated a query description. Tool-learning research studies how foundation models plan, select tools, construct arguments and use feedback [27]. An application should expose only its permitted operations through an explicit schema, validate the selected tool and arguments before execution, then attach the result status and evidence identifier to the state.

Read-only evidence can contain imperative text. An equipment manual might quote a command, and a retrieved conversation might contain another participant's request. These strings remain source content. They should not update the current user's task state or expand the system's permitted operations. A result should distinguish success, no matching observation and execution error; converting all three into a generic fluent answer erases the information needed to understand what happened. Retrying a read-only query may be acceptable; repeating an operation that changes external state can produce a second effect, so record request identifiers and execution status in software rather than relying on the language model to remember.

12. Evaluate a conversation, not just a reply

Plain version. A correct-looking response can hide a wrong slot, a stale memory or an action that never ran. Measure the state and the final outcome over whole conversations.

Deriu et al.'s survey [25] explains why task-oriented, conversational and question-answering systems require different evaluation methods. For a specified set of slots J, define exact state accuracy as

$$A_{\text{state}} = \frac{1}{N}\sum_{t=1}^{N} \mathbf{1}\{\hat{s}_{t,j} = s_{t,j} \text{ for every } j \in J\}.$$

Here N is the number of evaluated turns; $s_{t,j}$ is the reference value for slot j at turn t; $\hat{s}_{t,j}$ is the predicted value; and $\mathbf{1}$ is one when all selected slots match and zero otherwise. This strict measure exposes errors hidden by averaging over mostly empty or easy slots. The selected slot set and handling of unknown values must be specified.

```
reference = [
    {"device": "pump_a", "period": None},
    {"device": "pump_a", "period": "today"},
    {"device": "pump_b", "period": "today"},
]
predicted = [
    {"device": "pump_a", "period": None},
    {"device": "pump_a", "period": "today"},
    {"device": "pump_a", "period": "today"},
```

```
]
slots = ["device", "period"]
joint = sum(all(p[j] == r[j] for j in slots)
            for p, r in zip(predicted, reference))/len(reference)
slot_accuracy = sum(p[j] == r[j] for p, r in zip(predicted, reference)
                    for j in slots)/(len(reference)*len(slots))
print("Exact state accuracy:", joint)
print("Per-slot accuracy:", slot_accuracy)
assert joint == 2/3 and slot_accuracy == 5/6
# The final device is wrong despite many correct fields.
assert predicted[-1]["device"] != reference[-1]["device"]
```

Task success should check the final requested outcome, not just dialogue length or friendly style. For the example, require the correct device, time interval and evidence, with corrections honored. A short conversation that queries the wrong device is not efficient success. BLEU against a reference reply is a poor fit here for the same reason as in translation, made worse by the many acceptable replies a conversation allows.

Human evaluation should show enough history to interpret a response and specify the judgment task. A rater who sees only the last reply cannot assess whether a correction was honored. Randomize system identity, define error categories and retain disagreements. Schmidtova et al. [28] show why explicit hallucination definitions and reporting matter. Split datasets by conversation so adjacent turns are not scattered across train and test, and report uncertainty at the conversation level: turns within one conversation are dependent.

Modern view

Plain version. Translation and dialogue both moved toward large pretrained models, but the evaluation questions in this chapter became more important, not less.

Surveys. Stahlberg's review in the *Journal of Artificial Intelligence Research* (2020) traces neural MT from its roots in word and sentence embeddings and early encoder–decoders to attention and the Transformer, and closes with a survey of recent trends [22]. It is the best single map for the material in Sections 1–4. For dialogue, Ni et al.'s systematic survey (*Artificial Intelligence Review*, 2023) organizes deep-learning dialogue systems by model type and system type, and its task-oriented versus open-domain distinction explains why one response metric cannot characterize every conversation [26]. Deriu et al. [25] map evaluation methods to those system types. Qin et al.'s review of tool learning with foundation models (*ACM Computing Surveys*, 2024) motivates inspecting argument construction and result use alongside text quality [27].

State of the art, 2023–2026. WMT has documented the arrival of large language models in translation: the WMT24 general-task findings evaluate several LLMs alongside submitted MT systems and online providers with professional human annotation, and the title states the balance, "the LLM era is here but MT is not solved yet" [20]. Massively multilingual models such as NLLB extend coverage to 200 languages, with low-resource languages as the explicit target [21]. On evaluation, the WMT22 metrics task found neural metrics more reliable than overlap metrics against

expert MQM judgments [18], and the WMT25 findings separate system ranking from segment-level error detection [19].

Design axis	Choices to compare	Evidence needed
Architecture	encoder–decoder, decoder-only, multilingual	held-out quality per language pair and domain
Decoding	greedy, beam with length normalization, sampling	completeness, length and semantic errors, not only model score
Evaluation	BLEU/chrF (with signature), learned metrics, MQM-style human review	agreement with the application's failure categories
Dialogue state	end-to-end generation, explicit state and policy	state accuracy, correction recovery, final task success
Tool use	free-form claims, validated proposals with results	executed outcomes matching reported outcomes

Open problems. Low-resource and dialectal translation, document-level consistency, evaluating metrics on rare but critical errors (negation, numbers, names), reliable automatic acceptance of individual outputs, calibration of dialogue state under task changes, multilingual corrections, and stale memories in long conversations. A higher corpus score and a fluent reply are both easy to obtain; preserved meaning and a correct final action are what need evidence.

Key takeaways

- SMT combined separately estimated translation and language models; NMT trains one encoder–decoder end to end, and the Transformer made it fully attention-based.
- Shift targets, keep the end token, mask padding in both attention and loss; teacher forcing trains on gold prefixes, so test the model on its own generated outputs (exposure bias).
- Greedy and beam search optimize the model's score with different amounts of search; length normalization changes which hypothesis wins; neither repairs a wrong model.
- BLEU is clipped corpus-level n-gram precision with a brevity penalty; report the sacreBLEU signature, add chrF or learned metrics, and check meaning-critical errors directly.
- A tiny Transformer learns dictionary translation and reordering in seconds, and its cross-attention recovers the alignment without supervision.
- Chatbots can be seq2seq models, but task dialogue needs explicit state with revisions, belief over ambiguous interpretations, and clarification chosen by expected loss.
- Keep tool proposals, executions and results separate, and evaluate whole conversations by state and final outcome.

Exercises

1. **Compositional test.** In the translator code, remove every training pair in which `red` modifies `c able`, then test on sentences that contain `red cable`. Compare exact match on these sentences with the ordinary test set.

Hint

Filter `train_pairs` before building `S` and `Y`, and keep the vocabulary unchanged so the failure is about combination rather than unknown words. Look at the cross-attention map of a failure.

2. **Beam width and length.** Evaluate the 100-step checkpoint with beam widths 1, 2, 4 and 8 and with $\alpha \in \{0, 0.6, 1\}$. Report BLEU, exact match, mean output length and mean log probability. When does a higher model score fail to improve BLEU?

Hint

Width 1 with $\alpha = 0$ is greedy decoding. Save the outputs and compare the sentences that change, not only the averages.

3. **Translation challenge set.** Write six original source sentences testing negation, comparison, units, modality, time and ambiguous reference. Specify the meaning that any acceptable translation must retain.

Hint

Use a minimal edit such as adding "not" or swapping the two device names. Judge source adequacy independently of whether the target sentence sounds elegant.

4. **Metric disagreement.** Construct two candidate translations: one with high word overlap but a wrong quantity, and another with lower overlap but correct meaning. Score them with the BLEU function and design an annotation form that reveals the difference.

Hint

Ask reviewers to identify unsupported quantities and missing required facts before giving an overall judgment. Keep the source visible and system identity hidden.

5. **Correction scope.** Extend the state tracker with a measurement slot. Test a device-only correction, a rejected period and a complete restart without silently changing unrelated slots.

Hint

Define distinct events for setting, retracting and resetting. Any changed proposal should invalidate a confirmation that refers to its previous revision.

6. **False action report.** Specify an evidence schema that distinguishes a successful query, no matching record and execution failure. Write a response requirement for each state, and a test conversation for each.

Hint

Attach an execution identifier and status to the result. A generated proposal alone cannot support a sentence claiming that the query succeeded.

References

1. M. Hagiwara, *Real-World Natural Language Processing*, Manning, 2021, Chapters 6 and 8. Book ·Sequence-to-sequence chapter
2. S. McRoy, *Principles of Natural Language Processing*, open textbook, 2021, Chapter 9. Book · Generation chapter
3. P. F. Brown, S. A. Della Pietra, V. J. Della Pietra and R. L. Mercer, "The Mathematics of Statistical Machine Translation: Parameter Estimation," *Computational Linguistics* 19(2), 1993. Paper
4. P. Koehn, F. J. Och and D. Marcu, "Statistical Phrase-Based Translation," HLT-NAACL, 2003. Paper
5. K. Cho, B. van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk and Y. Bengio, "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation," EMNLP, 2014. Paper
6. I. Sutskever, O. Vinyals and Q. V. Le, "Sequence to Sequence Learning with Neural Networks," NeurIPS, 2014. Paper
7. D. Bahdanau, K. Cho and Y. Bengio, "Neural Machine Translation by Jointly Learning to Align and Translate," ICLR, 2015. Paper
8. A. Vaswani et al., "Attention Is All You Need," NeurIPS, 2017. Paper
9. R. Sennrich, B. Haddow and A. Birch, "Neural Machine Translation of Rare Words with Subword Units," ACL, 2016. Paper
10. M. Ranzato, S. Chopra, M. Auli and W. Zaremba, "Sequence Level Training with Recurrent Neural Networks," ICLR, 2016. Paper
11. S. Bengio, O. Vinyals, N. Jaitly and N. Shazeer, "Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks," NeurIPS, 2015. Paper
12. Y. Wu et al., "Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation," arXiv:1609.08144, 2016. Paper
13. K. Papineni, S. Roukos, T. Ward and W.-J. Zhu, "Bleu: a Method for Automatic Evaluation of Machine Translation," ACL, 2002. Paper
14. M. Post, "A Call for Clarity in Reporting BLEU Scores," WMT, 2018. Paper
15. M. Popović, "chrF: character n-gram F-score for automatic MT evaluation," WMT, 2015. Paper
16. R. Rei, C. Stewart, A. C. Farinha and A. Lavie, "COMET: A Neural Framework for MT Evaluation," EMNLP, 2020. Paper
17. C.-Y. Lin, "ROUGE: A Package for Automatic Evaluation of Summaries," Text Summarization Branches Out, 2004. Paper
18. M. Freitag et al., "Results of WMT22 Metrics Shared Task: Stop Using BLEU – Neural Metrics Are Better and More Robust," WMT, 2022. Report
19. A. Lavie et al., "Findings of the WMT25 Shared Task on Automated Translation Evaluation Systems: Linguistic Diversity is Challenging and References Still Help," WMT, 2025. Report
20. T. Kocmi et al., "Findings of the WMT24 General Machine Translation Shared Task: The LLM Era Is Here but MT Is Not Solved Yet," WMT, 2024. Report
21. NLLB Team et al., "Scaling neural machine translation to 200 languages," *Nature* 630, 2024. DOI
22. F. Stahlberg, "Neural Machine Translation: A Review," *Journal of Artificial Intelligence Research* 69, 2020. DOI

23. O. Vinyals and Q. V. Le, “A Neural Conversational Model,” ICML Deep Learning Workshop, 2015. Paper
24. J. Li, M. Galley, C. Brockett, J. Gao and B. Dolan, “A Diversity-Promoting Objective Function for Neural Conversation Models,” NAACL-HLT, 2016. Paper
25. J. Deriu, Á. Rodrigo, A. Otegi, G. Echegoyen, S. Rosset, E. Agirre and M. Cieliebak, “Survey on Evaluation Methods for Dialogue Systems,” *Artificial Intelligence Review* 54, 2021. Survey
26. J. Ni, T. Young, V. Pandelea, F. Xue and E. Cambria, “Recent Advances in Deep Learning Based Dialogue Systems: A Systematic Survey,” *Artificial Intelligence Review* 56, 2023. Survey
27. Y. Qin et al., “Tool Learning with Foundation Models,” *ACM Computing Surveys* 57, 2024. Author manuscript ·Publication
28. P. Schmidtova et al., “Do My Eyes Deceive Me? A Survey of Human Evaluations of Hallucinations in NLG,” INLG, 2025. Survey
29. C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens and Z. Wojna, “Rethinking the Inception Architecture for Computer Vision,” CVPR, 2016. Paper
30. M. Freitag, G. Foster, D. Grangier, V. Ratnakar, Q. Tan and W. Macherey, “Experts, Errors, and Context: A Large-Scale Study of Human Evaluation for Machine Translation,” *Transactions of the Association for Computational Linguistics* 9, 2021. DOI

NLP 09 • Language Modeling with Transformers (Natural Language Generation)

Train a tiny decoder-only Transformer language model, decode it with greedy, beam, temperature, top-k and nucleus search, and evaluate summaries, data-to-text and grounded generation.

Online edition: https://blog.shyandram.dev/en/learn/nlp/nlp-09-language-modeling-transformers/

What you'll learn

- Factor a sequence probability into causal next-token predictions, and train a tiny decoder-only Transformer that obeys that factorization.
- Place the GPT line of models, their training objective and scaling studies in one consistent picture.
- Decode with greedy search, beam search, temperature, top-k and nucleus sampling, and explain what each does and does not change.
- Build and evaluate three generation tasks: extractive and abstractive summarization, data-to-text and open-ended story writing.
- Measure generation with perplexity, ROUGE, human review and model-based judges, knowing the blind spots of each.
- Retrieve evidence with lexical and dense methods and evaluate the ranking with Recall@k, MRR and nDCG.
- Explain hallucination, retrieval-augmented generation and the boundary between retrieved evidence and application instructions.

The big picture

A language model assigns probabilities to text under a specified context. That simple definition connects spelling completion, sentence generation and reusable contextual representations, but these uses do not share the same output contract. Predicting a likely continuation is different from verifying a factual answer. A fluent continuation may reflect common text patterns while being unsuitable for a particular task.

A language application must also choose what to say, express it clearly, and support what it says. These are separate responsibilities. A readable sentence can describe the wrong event; a correct sentence can omit the warning that the user needed. Deploying the generator adds a further responsibility: the result must arrive predictably, with enough recorded context to investigate a failure.

Explain like I'm five

> Imagine a guessing game where you see the beginning of a sentence and guess the next word. Then you say your guess out loud, add it to the sentence, and guess again. That is how a language model writes. It can look only at the words already said; peeking at the end of the sentence would be cheating. And a guess that sounds right is not the same as a guess that is true, so a good writer also checks the facts.

Hagiwara develops language modeling in Chapter 5, attention and Transformers in Chapter 8, pretrained-model adaptation in Chapter 9, and training practice and deployment in Chapters 10–11 [1]. McRoy's Chapter 6 emphasizes language-model evaluation through tasks, and Chapter 9 develops generation and summarization [2]. This chapter connects the probability model to those practical concerns. We train only tiny synthetic models; large pretrained systems are discussed through verified primary papers and surveys.

Three decisions organize the subject. The **training objective** determines what is predicted. The **attention pattern** determines which input positions are visible. The **decoding procedure** determines how predictions become user-facing output. Keep these separate when comparing models. A change in sampling temperature changes the output distribution; it does not change which facts were learned or verify the resulting claim.

Where this chapter sits: count-based n-gram language models, smoothing and the first perplexity calculations are in NLP 05; attention, positions and the Transformer block are in NLP 07; encoder–decoder translation and dialogue are in NLP 08. Here the Transformer is used as a **decoder-only** language model and as a text generator. The bidirectional alternative, BERT, follows in NLP 10.

1. Causal sequence probability

Plain version. A sequence probability can be built by predicting each token from earlier tokens. The complete sequence is available during training, but each prediction must still obey that visibility rule.

Using the probability chain rule:

$$p_\theta(x_1, \dots, x_T) = \prod_{t=1}^{T} p_\theta(x_t \mid x_{<t}).$$

Here x_t is the token at position t; T is the sequence length; $x_{<t}$ is the prefix before t; and θ denotes model parameters. A beginning token can supply an explicit context for the first ordinary token, and an end token lets the model assign probability to stopping. The chain rule is exact; the learned conditional distributions are approximations to the data-generating process.

For a corpus with N supervised token positions, the mean negative log likelihood is:

$$\mathcal{L}_{\text{causal}} = -\frac{1}{N} \sum_{n=1}^{N} \log p_\theta(x_n \mid c_n).$$

Here $\mathcal{L}_{\text{causal}}$ is the training objective; n indexes valid prediction positions across examples; x_n is the target token; c_n is its allowed prefix; and N excludes padding and any other positions intentionally

left unsupervised. Natural logarithms express this loss in nats per token. A batch average that includes padding changes the objective and can reward predicting artificial padding symbols.

An n-gram model, as in NLP 05, replaces $x_{<t}$ with its last $n-1$ tokens, so two prefixes ending the same way must receive identical predictions. A Transformer language model keeps the whole permitted prefix, up to its context limit, and lets learned attention decide which earlier tokens matter. The factorization is the same; what changes is how much of the prefix the conditional can read.

Teacher forcing uses the observed prefix when predicting each training token. It permits parallel evaluation of all positions when the architecture enforces a causal mask. At generation time, the prefix contains earlier generated tokens instead. Errors can then affect later predictions. The training objective and rollout behavior therefore require separate diagnostics, even when the same model weights are used.

Shift the input and target by one position. If the full sequence is beginning, the, robot, moves, end, the model input is beginning, the, robot, moves and the targets are the, robot, moves, end. Predicting each input token from its own unshifted representation would create a shortcut unrelated to next-token prediction. The code in Section 3 makes the shift explicit.

Why teacher-forced loss misses some rollout errors

Teacher forcing presents the correct prefix even when the model would have generated a different one [16]. Suppose the reference begins `Pump A`, but generation selects `Pump B`. Later tokens can form a perfectly fluent statement about Pump B. Teacher-forced loss still evaluates those later positions after `Pump A`; it never tests recovery from this particular generated mistake. The distinction is between prediction under a supplied history and behavior under the model's own evolving history.

Token averaging also hides unequal semantic consequences. Replacing `the` may slightly affect style; replacing `not`, a device ID or a unit can reverse the result. Each token contributes a log-loss term, while the application may assign very different costs to these changes. A lower average loss can coexist with more critical field errors if common wording improves while a rare distinction deteriorates. Check the whole output's propositions in addition to the average training objective.

2. Decoder-only Transformer language models

Plain version. A decoder-only Transformer is a stack of self-attention blocks in which every position may read only itself and earlier positions. Trained on next-token prediction, it becomes both a scorer of text and a generator of text.

Each block computes masked self-attention followed by a position-wise feedforward network, with residual connections and normalization, as in the original Transformer [3]; NLP 07 derives the attention equation. The only change from a bidirectional encoder is an additive mask B inside the attention softmax: $B_{ij} = 0$ when $j \le i$ and $B_{ij} = -\infty$ otherwise. Here i is the querying position

and j is the key position it may read. With shifted targets, the current input position is an allowed prefix token, so the diagonal remains visible.

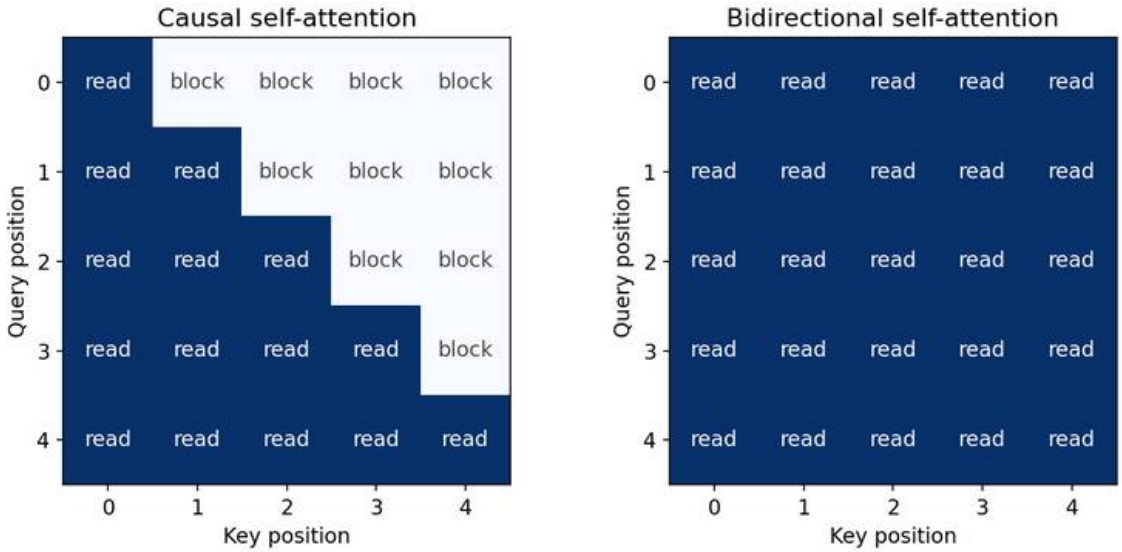


Figure 1 — Visibility is part of the task definition. Rows are querying positions and columns are the positions they may read. A decoder-only language model uses the left pattern in every layer.

A final linear layer maps the top-layer vector at position t to vocabulary logits $\mathbf{z}_t$, and a softmax turns them into $p_\theta(x_{t+1} \mid x_{\le t})$. Because every row of the mask hides the future, one forward pass computes the training loss at all positions at once, while generation still produces one token per pass.

The GPT line

The GPT papers show how far this one design can be pushed. Radford and colleagues trained a 12-layer decoder-only Transformer with masked self-attention as a language model on BooksCorpus, then fine-tuned it with a task-specific output layer for classification, entailment and similarity tasks [4]. The recipe was **generative pretraining followed by discriminative fine-tuning**: unlabeled text supplies most of the learning signal, and a small labeled set adapts the model.

GPT-2 kept the next-token objective and dropped task-specific fine-tuning from its main experiments. Its authors trained on WebText, a new dataset of millions of web pages, used byte-level byte-pair encoding so that any string could be represented, and reported that the language model began to perform tasks such as question answering and summarization in a zero-shot setting; the largest model had 1.5 billion parameters [5]. GPT-3 scaled the same approach to an autoregressive model with 175 billion parameters and evaluated it with tasks and a few demonstrations given purely as text, without gradient updates [6]. These are three points on one line: the architecture and objective barely change, while data, model size and the way tasks are presented do.

Reading the line correctly matters for later sections. A prompt is just a prefix $x_{\le t}$. "Zero-shot" and

"few-shot" describe what that prefix contains, not a different model. The quality of a continuation still depends on what the training text made probable, which is why instruction tuning, preference learning and retrieval (Sections 9–11) were added on top.

3. A runnable causal model

Plain version. A tiny model can learn a synthetic language pattern and still generalize poorly to unseen combinations. We also test a property that low loss alone cannot establish: future tokens must not influence earlier predictions.

The following standalone CPU example uses invented fixed-length sentences. Train and test sentences differ, while every ordinary vocabulary item is present in training. The code uses no padding, pretrained weights or dataset downloads. It is intentionally a mechanism experiment rather than a language benchmark.

The implementation uses AdamW for optimization [9], the GELU activation in the feedforward block [10], and layer normalization before attention and the feedforward transformation [11]. These choices define this small experiment, rather than reproduce every detail of the original Transformer. Keep them fixed when studying the causal mask so a change in optimization or activation does not confound the comparison.

```
import math
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(11)
torch.set_num_threads(1)
train_text = [
    'the red robot moves', 'the blue robot waits',
    'a red sensor waits', 'a blue sensor moves',
    'the red sensor moves', 'a blue robot waits',
]
test_text = ['a red robot moves', 'the blue sensor waits']
vocab = ['<bos>', '<eos>'] + sorted(set(' '.join(train_text).split()))
index = {word: i for i, word in enumerate(vocab)}
def encode(rows):
    return torch.tensor([[index[w] for w in ['<bos>'] + s.split() + ['<eos>']] for s in
     ↪  rows])
train, test = encode(train_text), encode(test_text)

class TinyCausalModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.token = nn.Embedding(len(vocab), 16)
        self.position = nn.Embedding(5, 16)
        self.norm1, self.norm2 = nn.LayerNorm(16), nn.LayerNorm(16)
        self.q, self.k, self.v = [nn.Linear(16, 16, bias=False) for _ in range(3)]
        self.output = nn.Linear(16, 16)
        self.ff = nn.Sequential(nn.Linear(16, 32), nn.GELU(), nn.Linear(32, 16))
```

```
        self.head = nn.Linear(16, len(vocab))
    def forward(self, ids):
        length = ids.shape[1]
        h = self.token(ids) + self.position(torch.arange(length))
        z = self.norm1(h)
        scores = self.q(z) @ self.k(z).transpose(-1, -2) / math.sqrt(16)
        blocked = torch.ones(length, length, dtype=torch.bool).triu(1)
        weights = scores.masked_fill(blocked, float('-inf')).softmax(dim=-1)
        h = h + self.output(weights @ self.v(z))
        h = h + self.ff(self.norm2(h))
        return self.head(h)

model = TinyCausalModel()
optimizer = torch.optim.AdamW(model.parameters(), lr=0.01)
def loss(rows):
    logits = model(rows[:, :-1])
    return F.cross_entropy(logits.reshape(-1, len(vocab)), rows[:, 1:].reshape(-1))
initial = loss(train).item()
for _ in range(180):
    optimizer.zero_grad()
    objective = loss(train)
    objective.backward()
    optimizer.step()
model.eval()
with torch.no_grad():
    train_nll, test_nll = loss(train).item(), loss(test).item()
    original = train[:1, :-1]
    changed = original.clone()
    changed[0, -1] = index['blue']
    gap = (model(original)[:, :-1] - model(changed)[:, :-1]).abs().max().item()
assert train_nll < initial and gap < 1e-6
print('initial/train/test NLL:', *(round(x, 3) for x in [initial, train_nll, test_nll]))
print('held-out perplexity:', round(math.exp(test_nll), 3))
print('future-token perturbation gap:', gap)
```

Output:

```
initial/train/test NLL: 2.194 0.409 3.17
held-out perplexity: 23.81
future-token perturbation gap: 0.0
```

The assertion checks that training loss decreases; it does not require held-out loss to improve by any specified amount. Here the held-out loss (3.17 nats per token) is even higher than the untrained starting loss (2.194): the held-out sentences combine familiar words in combinations excluded from training, and the model has confidently memorized the six training patterns. A large train–test gap demonstrates that learning the observed patterns does not establish compositional generalization.

The future-token perturbation test changes the final input token and compares all earlier logits. Causal attention should make that change invisible to earlier predictions, and the printed gap is

exactly zero. This test targets the exact visibility contract. Passing it does not establish every form of data isolation: duplicated documents across splits, vocabulary construction from forbidden test data or future metadata can still create other leakage.

The model has no dropout, so evaluation mode does not alter a stochastic layer here. We call it explicitly because the training/evaluation distinction is essential in realistic models. The loss function flattens batch and position dimensions while retaining vocabulary as the class dimension. If those axes are accidentally exchanged, a program can run while optimizing the wrong prediction problem.

Each sequence has the same length, and the position table covers the five model-input positions. A learned position table imposes a maximum supported position; giving a longer sequence is an invalid lookup, not an ordinary longer input. Extending to variable-length data requires separate attention and loss masks. An attention mask controls what representations may read; a loss mask controls which target positions contribute to the objective. They solve different problems and should be tested independently.

4. Training objective and scaling

Plain version. Large language models are trained with the same next-token loss as the tiny model above. What changes is the amount of text, the number of parameters and the compute spent, and those quantities trade off in measurable ways.

Kaplan and colleagues trained families of Transformer language models and reported that held-out loss falls approximately as a power law in model size, dataset size and compute [7]. In model size alone:

$$L(N) = \left(\frac{N_c}{N}\right)^{\alpha_N}.$$

Here L is held-out cross-entropy loss, N is the number of non-embedding parameters, N_c is a fitted constant and $\alpha_N > 0$ is a fitted exponent. Hoffmann and colleagues later fitted a form with an irreducible term, $L(N, D) = E + A/N^{\alpha} + B/D^{\beta}$, where D is the number of training tokens, E the loss that no model can remove, and A, B, α, β fitted constants; for a fixed compute budget they concluded that parameters and training tokens should be scaled in roughly equal proportion [8]. ML 09 derives the compute-optimal trade-off and fits a scaling law in code.

Two cautions keep scaling in its place. First, the fitted constants depend on tokenizer, data mixture and training protocol, so they do not transfer between unrelated setups. Second, scaling laws predict **loss**, not any particular task behavior. A lower next-token loss usually helps, but whether a model preserves a negation in a summary or refuses to invent a citation must still be measured directly (Section 8).

5. Perplexity and meaningful comparison

Plain version. Perplexity summarizes held-out token prediction. Its value depends on what counts as a token and which positions are evaluated.

For mean negative log likelihood $\mathcal{L}$:

$$\mathrm{PPL} = \exp(\mathcal{L}).$$

Here PPL is perplexity and exp is the natural exponential; $\mathcal{L}$ uses natural logarithms over the same evaluated token positions. Lower perplexity means that the model assigns higher average probability to observed tokens under that evaluation contract. It is not a direct measure of factual correctness, helpfulness or grammaticality. IT 01 connects this quantity to cross-entropy, and NLP 05 computes it for count models.

Compare perplexities only under compatible tokenization and input conventions. A word can be one token in one vocabulary and several in another. The mean loss per token then describes a different unit. Also record whether beginning and end markers are scored, how documents are segmented, the context length and whether evaluation windows overlap. Recounting duplicated target positions changes the weighting.

Keep evaluation documents independent of training documents and their derivatives. Splitting adjacent passages of one document across partitions can expose style, facts and nearly identical wording. For a time-dependent application, use an appropriate chronological split and retain the document version. A held-out name or phrase does not by itself prove the surrounding document is independent.

McRoy's benchmark perspective reminds us to assess task behavior in addition to sequence prediction [2]. A classifier needs category performance; an extractor needs spans and supported relations; a generator needs meaning preservation and omissions. A model with lower perplexity can fail a critical task distinction, such as changing "did not move" to "moved." Match the evaluation to the user's actual outcome.

A reproducible language-model experiment

A result is useful when someone can tell exactly which text, tokens and decisions produced it. Begin with the document split and its reason: a random split may suit independent synthetic sentences, while a support assistant might require customer-separated or chronological documents. In the toy experiment, the vocabulary comes only from training sentences, and the test combinations were chosen to contain known words. This isolates a narrow combination-generalization question.

Next record the tokenization, sequence boundaries, target shift, maximum context and loss denominator. The code scores every next-token target including the end token, and does not score the beginning token as a target. If another run excludes stopping predictions or weights each sentence equally regardless of length, its number answers a different question. Finally separate selection from reporting: choose architecture, training length and decoding behavior on develop-

ment data, and inspect the held-out set only after those choices are frozen. Save the preprocessing version, model configuration, seed, permitted input context and output settings, while keeping sensitive raw documents under appropriate access control.

6. Decoding: from a distribution to a text

Plain version. A trained model produces scores. A decoder decides whether to choose the strongest token, search over several continuations, or sample alternatives. None of these choices makes unsupported content reliable.

Temperature

For logit z_v and positive temperature τ:

$$q_\tau(v) = \frac{\exp(z_v/\tau)}{\sum_{u \in V} \exp(z_u/\tau)}.$$

Here v is a candidate token; V is the vocabulary; u indexes candidates in the denominator; z_v is the model's unnormalized score; τ is temperature; and q_τ is the resulting sampling distribution. Lower temperature concentrates mass around high-scoring tokens; higher temperature spreads it. As $\tau \to 0$, sampling approaches greedy selection, which chooses a maximum-score token. A factual mistake can be the highest-logit option, so reducing randomness does not remove it.

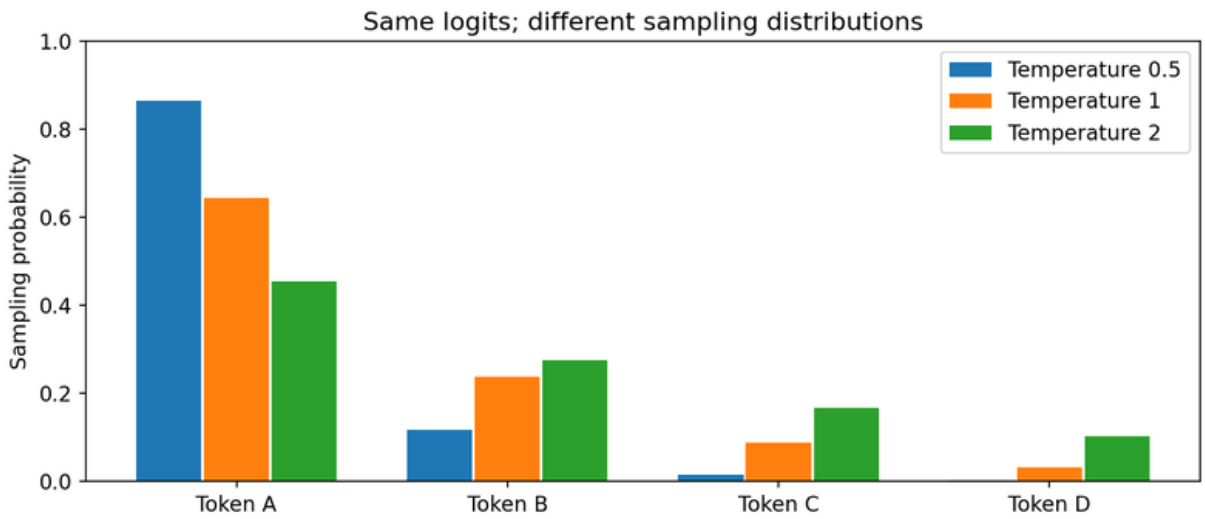


Figure 2 — Temperature changes a sampling distribution over fixed synthetic logits. More concentrated output is not automatically more correct.

Greedy and beam search

Greedy decoding chooses the most probable next token. Beam search retains several partial sequences. The sequence score for a candidate is

$$\ell(y \mid x) = \sum_{t=1}^{T} \log p_\theta(y_t \mid y_{<t}, x).$$

Here ℓ is the log probability of the complete output y given a prompt or source x; T includes the end token; and the other symbols are as above. With raw log probabilities, every additional token contributes a nonpositive term. This can favor shorter sequences. Length adjustments change the search objective; describe their form and tune them on validation data rather than silently applying them.

This tiny conditional model has two competing first tokens. It illustrates why choosing the best next token can miss the best complete sequence. The probabilities are constructed for the experiment, not estimated from language data.

```
import math

table = {
    (): {"safe": 0.55, "close": 0.45},
    ("safe",): {"stop": 0.51, "wait": 0.49},
    ("close",): {"valve": 0.99, "door": 0.01},
}
def next_probs(prefix):
    return table.get(prefix, {"<eos>": 1.0})
def decode(width):
    beams = [((), 0.0)]
    for _ in range(3):
        candidates = []
        for seq, score in beams:
            if seq and seq[-1] == "<eos>":
                candidates.append((seq, score))
                continue
            for token, prob in next_probs(seq).items():
                candidates.append((seq + (token,), score + math.log(prob)))
        beams = sorted(candidates, key=lambda pair: pair[1], reverse=True)[:width]
    return beams[0]
greedy, beam = decode(1), decode(2)
print(greedy[0], round(math.exp(greedy[1]), 4))
print(beam[0], round(math.exp(beam[1]), 4))
assert greedy[0] == ("safe", "stop", "<eos>")
assert beam[0] == ("close", "valve", "<eos>")
assert beam[1] > greedy[1]
```

Output:

```
('safe', 'stop', '<eos>') 0.2805
```

```
('close', 'valve', '<eos>') 0.4455
```

Beam search finds the more probable sequence in this constructed tree. That result is about the model's probability assignment. If "close valve" contradicts the source instruction, a better probability search still produces the wrong application behavior. Search quality and task correctness need separate measurements.

Search error, model error and a wrong objective

The toy beam succeeds because it keeps the `close` prefix long enough to discover its strong continuation. Greedy decoding discards that branch before seeing the continuation. This is **search error** relative to the constructed sequence-probability objective. A larger beam can reduce that kind of error, but it is still a bounded search: a useful prefix can fall outside the retained set at any depth.

Raw prefix log probability supplies a helpful bound. Every future log-probability term is nonpositive, so extending a prefix cannot improve its raw score. A finished candidate that exceeds the scores of all unfinished prefixes is therefore optimal among those surviving branches. This argument requires the unadjusted sum; a length bonus or normalization changes the bound and must be considered in the stopping rule.

Now distinguish **model error**. If the source requires keeping the valve open while the model assigns its greatest sequence probability to `close valve`, exact probability search exposes the model's mistake more consistently. Search cannot create source compatibility that the score lacks. Finally, maximum probability is only one decision objective. A generic short response may have high probability while omitting the required warning. Evaluate whether the selected objective rewards the user's intended result before tuning beam width.

Top-k and nucleus sampling

For open-ended writing, maximizing probability is often the wrong goal. Holtzman and colleagues observed that using likelihood as a decoding objective produces text that is bland and strangely repetitive, even from a strong model, while pure sampling from the full distribution can pick very unlikely tokens from the long tail [12]. Two truncation rules sit between these extremes.

Top-k sampling keeps the k highest-probability tokens and renormalizes. Fan and colleagues used it for story generation, sampling from the 10 most likely candidates at each step and finding it more effective than beam search for that task [13]. **Nucleus (top-p) sampling** keeps the smallest set of tokens whose cumulative probability reaches p [12]:

$$V^{(p)} = \arg\min_{S \subseteq V} |S| \quad \text{s.t.} \quad \sum_{v \in S} p_\theta(v \mid x_{\le t}) \ge p, \qquad q(v) = \frac{p_\theta(v \mid x_{\le t})\, \mathbf{1}[v \in V^{(p)}]}{\sum_{u \in V^{(p)}} p_\theta(u \mid x_{\le t})}.$$

Here $V^{(p)}$ is the nucleus, $|S|$ the size of a candidate set S, $p \in (0, 1]$ the target mass, $\mathbf{1}[\cdot]$ an indicator, and q the renormalized sampling distribution. The difference from top-k is that the nucleus **adapts**: after a prefix with one obvious continuation it may contain a single token, and after an open-ended prefix it may contain many.

```
import numpy as np

tokens = np.array(['valve', 'pump', 'door', 'sensor', 'cabinet', 'label', 'fish', 'moon'])
logits = np.array([3.0, 2.6, 1.2, 1.0, 0.2, -0.5, -1.5, -2.0])

def softmax(z):
    e = np.exp(z - z.max())
    return e / e.sum()

def temperature(z, tau):
    return softmax(z / tau)

def top_k(p, k):
    keep = np.argsort(-p)[:k]                 # indices of the k largest
    q = np.zeros_like(p); q[keep] = p[keep]
    return q / q.sum()

def nucleus(p, top_p):
    order = np.argsort(-p)
    cumulative = np.cumsum(p[order])
    size = int(np.searchsorted(cumulative, top_p) + 1)   # smallest prefix reaching top_p
    q = np.zeros_like(p); q[order[:size]] = p[order[:size]]
    return q / q.sum()

p = softmax(logits)
print('greedy choice:', tokens[np.argmax(p)])
for name, q in [('tau=0.5', temperature(logits, 0.5)), ('tau=1.0', p),
                ('tau=2.0', temperature(logits, 2.0)),
                ('top-k=3', top_k(p, 3)), ('top-p=0.9', nucleus(p, 0.9))]:
    print(f'{name:9s} support={int((q > 0).sum())}  P(valve)={q[0]:.3f}  '
          f'tail mass={q[4:].sum():.3f}')

# The nucleus adapts its size to the shape of the distribution.
flat = softmax(np.zeros(8))
peaked = softmax(np.array([6.0, 1, 0, 0, 0, 0, 0, 0]))
print('nucleus size, flat vs peaked:',
      int((nucleus(flat, 0.9) > 0).sum()), int((nucleus(peaked, 0.9) > 0).sum()))

rng = np.random.default_rng(0)
draws = rng.choice(len(tokens), size=2000, p=nucleus(p, 0.9))
assert set(draws) <= set(np.flatnonzero(nucleus(p, 0.9)))
assert np.isclose(top_k(p, 3).sum(), 1) and np.argmax(temperature(logits, 0.1)) == 0
```

Output:

```
greedy choice: valve
tau=0.5   support=8  P(valve)=0.667  tail mass=0.003
```

```
tau=1.0   support=8  P(valve)=0.481  tail mass=0.052
tau=2.0   support=8  P(valve)=0.312  tail mass=0.190
top-k=3   support=3  P(valve)=0.545  tail mass=0.000
top-p=0.9 support=4  P(valve)=0.507  tail mass=0.000
nucleus size, flat vs peaked: 8 1
```

Here "tail mass" is the probability left on the four least likely tokens. Temperature only reshapes the distribution, so every token stays possible: at $\tau = 2$ almost a fifth of the mass sits on `cabine t`, `label`, `fish` and `moon`. Truncation removes the tail outright. On the uniform distribution the nucleus keeps all eight tokens; on the peaked one it keeps one. A fixed k would keep three in both cases.

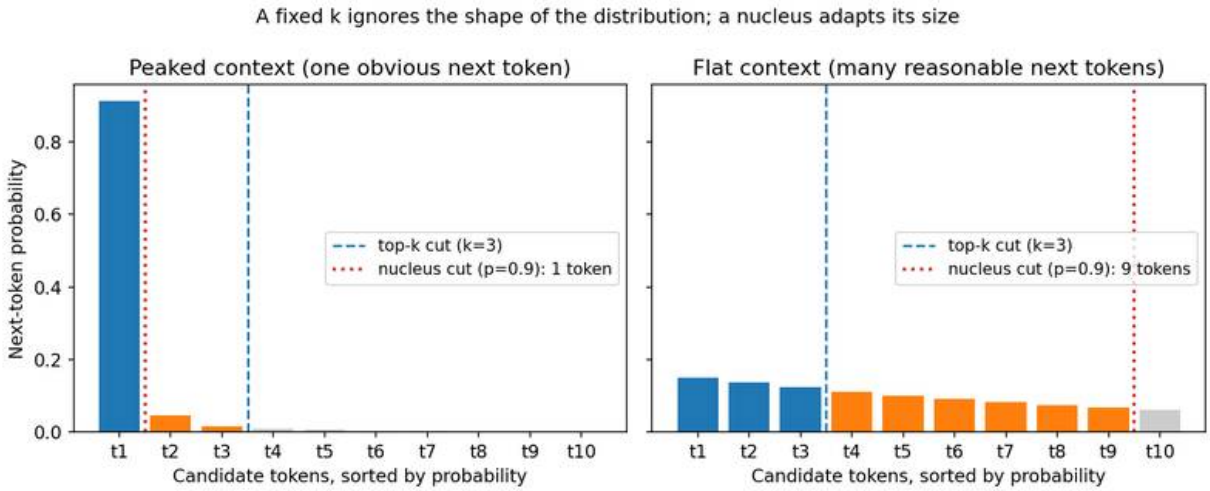


Figure 3 — Top-k and nucleus truncation on two synthetic distributions. Blue bars are kept by both rules, orange bars by only one, gray bars by neither.

In practice these knobs are combined (for example a temperature followed by a nucleus) and chosen per task. A data-to-text report usually wants deterministic, reproducible output; brainstorming wants variety. Sampling diversity is not evidence of quality, and repeated samples that agree are not an independent verification of a claim: they share the same model and often the same training biases.

Stopping and length

Generation also needs a stopping policy and a length budget. A model may stop too early or continue with redundant content. Set explicit output limits, a stopping rule and behavior when the limit is reached. A truncated summary should not be returned as complete. Record the decoding parameters with the model artifact so an evaluation can be reproduced; changing only the decoder can change every downstream metric.

7. Natural language generation tasks

Plain version. Generation means producing a sequence whose content and form satisfy a task. The input may be a document, a table, a prompt or retrieved passages; the task defines what counts as an error.

A decoder-only model handles every conditional task the same way: it treats the input as a prefix. The conditional factorization is

$$p_\theta(y \mid x, c) = \prod_{t=1}^{T} p_\theta(y_t \mid y_{<t}, x, c).$$

Here x is the source input; c is task context, such as the requested language or format; $y = (y_1, \dots, y_T)$ is the output token sequence, including an end token; $y_{<t}$ contains the tokens before position t; and θ denotes learned parameters. In a decoder-only model, x and c are serialized into the prefix and the loss is usually taken only over the tokens of y; an encoder–decoder (NLP 08) encodes x separately. Either way, the factorization describes a distribution. It does not certify that the sampled output is supported by the source.

Define the generation problem before choosing a model

Begin by writing a task contract in ordinary language. For an equipment report: "Given timestamped observations, produce two sentences describing the current temperature and any explicitly recorded fault; preserve device names, units, numbers and negation." This contract exposes omissions and contradictions that an unconstrained instruction such as "summarize nicely" hides.

Separate three operations. **Content selection** chooses observations relevant to the user. **Sentence planning** decides ordering, grouping and referring expressions. **Realization** expresses the plan as grammatical text. McRoy [2] provides the linguistic reading for this distinction. Neural systems can combine the operations internally, but a combined architecture still needs tests for each responsibility.

Consider a source with a high temperature, a later normal temperature, and an explicitly absent fault. Selecting the highest reading creates a different report from selecting the latest reading. Neither selection rule follows from grammatical fluency. Record whether the output should describe the current state, an extreme event, or the whole trajectory. Time and scope belong in the input contract.

Data-to-text: a template baseline

Data-to-text generation verbalizes structured records. Wiseman, Shieber and Rush introduced a large corpus of data records paired with descriptive documents, together with extractive evaluation methods that check which records the text actually conveys; the neural systems they tested produced fluent text, yet templated baselines exceeded them on some of those metrics [15]. A template is a useful baseline when the input schema is narrow. It offers complete control over

permitted claims and makes failure localization simple. Its limits are also clear: it cannot gracefully cover arbitrary relations, referring expressions or discourse. Measure that coverage rather than treating templates as inherently inadequate.

Here is a deterministic report function for a narrow schema. It demonstrates field validation and traceable output. The event record is trusted structured input; obtaining and validating that record from raw text would be a separate NLP stage.

```
import math

VERSION = "equipment-template-1"
def report(event):
    required = {"device", "temperature_c", "fault", "evidence_id"}
    if set(event) != required:
        raise ValueError("Unexpected input fields")
    if not isinstance(event["device"], str) or not event["device"].strip():
        raise ValueError("Missing device")
    value = event["temperature_c"]
    if isinstance(value, bool) or not isinstance(value, (int, float)) or not
     ↪  math.isfinite(value):
        raise ValueError("Temperature must be finite")
    if event["fault"] not in ("present", "absent", "unknown"):
        raise ValueError("Unknown fault status")
    if not isinstance(event["evidence_id"], str) or not event["evidence_id"]:
        raise ValueError("Missing evidence ID")
    phrases = {"present": "A fault was recorded.",
               "absent": "No fault was recorded.",
               "unknown": "Fault status was not recorded."}
    text = f"{event['device']} measured {value:g} C. {phrases[event['fault']]}"
    return {"text": text, "evidence_id": event["evidence_id"], "version": VERSION}
output = report({"device": "Pump A", "temperature_c": 30,
                 "fault": "absent", "evidence_id": "synthetic-event-01"})
print(output)
assert output["text"] == "Pump A measured 30 C. No fault was recorded."
assert output["version"] == VERSION
```

Output:

```
{'text': 'Pump A measured 30 C. No fault was recorded.', 'evidence_id':
 ↪  'synthetic-event-01', 'version': 'equipment-template-1'}
```

Notice the difference between absent and unknown. The input “no fault was recorded” is a statement about recorded evidence, not a claim that a fault is impossible. Encoding that distinction prevents the realization stage from upgrading missing evidence into certainty. A learned generator becomes useful when its additional flexibility solves a documented failure of this baseline, and it should be tested on exactly these distinctions.

Extractive summarization: select evidence under a budget

An extractive summary chooses source sentences. An abstractive summary may rewrite or combine them. Extraction makes the origin of each sentence explicit, but the selected sentences can still mislead through missing context, conflicting timestamps or unresolved pronouns. Selecting a sentence containing "it failed" without its antecedent is a discourse error even though every word came from the source.

Suppose a document contains candidate sentences $s_1, \ldots, s_n$. Represent sentence i by a unit vector v_i and a user query by a unit vector q. A simple relevance score is $r_i = v_i^\top q$. To discourage repetition, use maximal marginal relevance, or MMR [14]:

$$i^* = \arg\max_{i \notin S} \left[\lambda r_i - (1 - \lambda) \max_{j \in S} v_i^\top v_j \right].$$

Here S is the set of already selected sentence indices; i^* is the next index; $\lambda \in [0, 1]$ balances relevance against similarity to previous selections; and the maximum similarity is defined as zero when S is empty. The dot products are cosine similarities because the vectors have unit length. This is a greedy selection rule, rather than a proof of globally optimal summary quality.

The following example uses binary word indicators to make every decision inspectable. The four candidate sentences are synthetic. Removing a small set of function words makes the representation less dominated by grammar, but it also removes potentially useful distinctions.

```
import re
import numpy as np

sentences = [
    "Pump A temperature reached 80 C during the test.",
    "The test recorded Pump A temperature at 80 C.",
    "Pump A vibration triggered a shutdown warning.",
    "The technician replaced a label on the cabinet.",
]
query = "pump temperature vibration shutdown"
stop = {"a", "the", "at", "on", "during"}
def tokens(text):
    return set(re.findall(r"[a-z0-9]+", text.lower())) - stop
vocab = sorted(set().union(*(tokens(s) for s in sentences), tokens(query)))
def vector(text):
    v = np.array([w in tokens(text) for w in vocab], dtype=float)
    return v / max(np.linalg.norm(v), 1e-12)
V = np.stack([vector(s) for s in sentences])
q = vector(query)
relevance, similarity = V @ q, V @ V.T
selected = []
for _ in range(2):
    redundancy = similarity[:, selected].max(axis=1) if selected else np.zeros(4)
    score = 0.55 * relevance - 0.45 * redundancy
    score[selected] = -np.inf
    selected.append(int(np.argmax(score)))
print("Selected indices:", selected)
```

```
print("Summary:", " ".join(sentences[i] for i in sorted(selected)))
assert len(set(selected)) == 2
assert 2 in selected and not ({0, 1} <= set(selected))
```

Output:

```
Selected indices: [2, 0]
Summary: Pump A temperature reached 80 C during the test. Pump A vibration triggered a
 ↪  shutdown warning.
```

The selected pair includes the shutdown sentence and one temperature sentence, avoiding the nearly repeated temperature observation. Sorting the selected indices restores source order after selection. This matters because selection order measures marginal value, while presentation order should support comprehension.

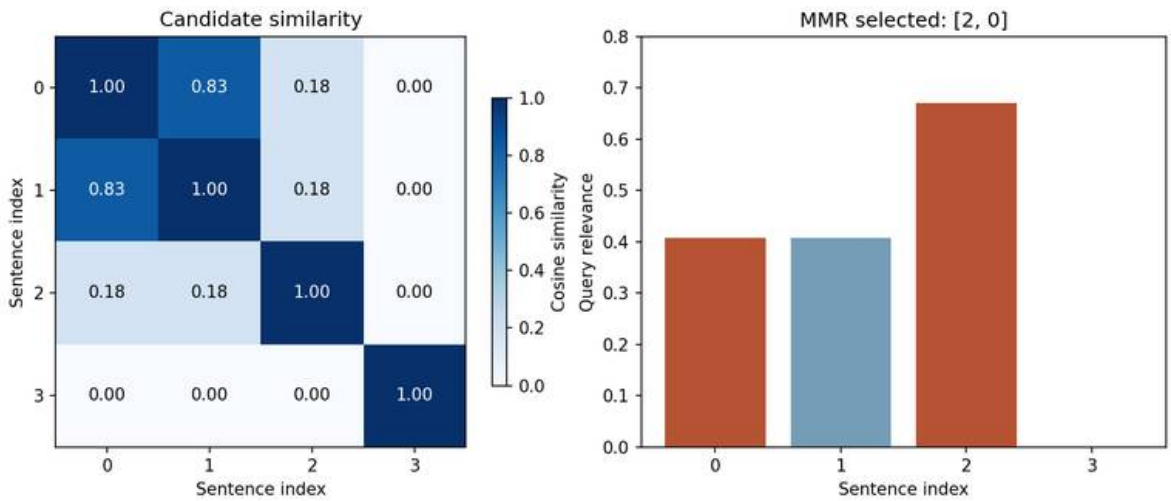


Figure 4 — An extractive baseline makes its trade-off visible. Similar temperature sentences compete for the same limited summary space; the shutdown sentence contributes different content.

A real budget usually limits tokens rather than sentence count. Add a candidate only if its full tokenized length fits the remaining budget; silently cutting a sentence can remove its condition or negation. For multi-document summaries, group near-duplicate reports so repeated copies do not masquerade as independent evidence. Retain source IDs and timestamps so readers can follow the selected evidence.

Relevance, feasibility and content preservation are different objectives

MMR discourages selecting sentences that resemble previous selections [14]. It does not know which facts are mandatory. If a warning shares many words with a selected observation, a similarity penalty can suppress it even though its added qualification is essential. Vector diversity is a proxy for redundancy, not a direct measure of complete meaning.

A content requirement changes the feasible set. For a token budget and a required warning:

$$\sum_{i \in S} \ell_i \le B, \qquad S \cap W \neq \emptyset.$$

Here S is the selected sentence-index set, ℓ_i is the full token length of candidate i, B is the budget, and W contains candidates expressing the required warning. The first condition limits size; the second ensures coverage. If no set satisfies both, the task is infeasible for extraction under that budget. Changing a relevance coefficient cannot make an over-budget sentence fit or invent a missing warning.

The following independent example isolates that distinction. Utilities are explicit teaching choices, not learned relevance scores:

```
import itertools

sentences = [
    'Pump A temperature reached 80 C during the test.',
    'The test recorded Pump A temperature at 80 C.',
    'Pump A vibration triggered a shutdown warning.',
    'The technician replaced a label on the cabinet.',
]
lengths = [len(s.split()) for s in sentences]
utility = [0.9, 0.85, 0.6, 0.2]
subsets = [tuple(i for i in range(4) if keep[i])
           for keep in itertools.product([False, True], repeat=4)]
feasible = [s for s in subsets if sum(lengths[i] for i in s) <= 20]
score = lambda s: sum(utility[i] for i in s)
ordinary = max(feasible, key=score)
with_warning = max((s for s in feasible if 2 in s), key=score)
print('Lengths:', lengths)
print('Without requirement:', ordinary, 'with warning:', with_warning)
assert ordinary == (0, 1) and with_warning == (0, 2)
```

Output:

```
Lengths: [9, 9, 7, 8]
Without requirement: (0, 1) with warning: (0, 2)
```

Here length counts whitespace-separated words; a model-token budget must use that model's tokenizer. The unconstrained choice repeats temperature content; the warning requirement changes the selected pair. More sophisticated scoring still needs the task's hard content requirements when omission is unacceptable.

Abstractive summarization

A rewriting model can fuse observations and improve readability. It can also invent a connecting statement that the observations never established. "The temperature was high, so the pump

failed" adds causation if the source only records simultaneous events. Rewriting two sentences into one is acceptable when relationships survive, but changing sequence into causation or removing "during the test" alters content. A plan that records propositions and their source intervals before wording is generated lets you evaluate selection, plan and realization separately, so you can tell where meaning was lost.

Story generation and open-ended text

Open-ended generation reverses the priorities of a report. There is no single correct output, and variety is part of quality. Fan, Lewis and Dauphin collected a dataset of human-written stories paired with writing prompts, generated a premise first and then the story, and used top-k sampling because beam search tended to produce short and generic text [13]. The lesson for this chapter is that the **same** language model can need different decoders for different tasks: deterministic search for a constrained report, truncated sampling for a story. What stays constant is that the evaluation must match the task: coherence and relevance to the prompt for a story, supported content for a summary.

8. Evaluating generated text

Plain version. One automatic score cannot summarize all generation quality. Use automatic metrics to compare systems under a specified protocol, then inspect the error dimensions the application actually cares about.

Perplexity (Section 5) evaluates the model as a predictor of reference text. It says nothing directly about a particular decoded output, and it cannot compare models with different tokenizers. **Overlap metrics** compare an output with references. ROUGE includes recall-oriented overlap measures for summarization [17]. Such metrics compare a candidate with one or more references; they do not prove semantic equivalence or source support. Name the variant, tokenizer, aggregation and reference set when reporting a score. Translation metrics such as BLEU and learned metrics are discussed in NLP 08.

For a simple unigram recall check,

$$R_1 = \frac{\sum_w \min(c_{\text{ref}}(w), c_{\text{hyp}}(w))}{\sum_w c_{\text{ref}}(w)}.$$

Here w ranges over word types; $c_{\text{ref}}(w)$ and $c_{\text{hyp}}(w)$ are reference and hypothesis word counts. The denominator is the number of reference tokens. This overlap calculation illustrates one component of ROUGE-style evaluation; the minimal implementation below does not reproduce every preprocessing or aggregation option of a full evaluation package.

```
import re
from collections import Counter

def words(text):
```

```
    return re.findall(r"[a-z0-9]+", text.lower())
def overlap_recall(reference, hypothesis):
    ref, hyp = Counter(words(reference)), Counter(words(hypothesis))
    return sum((ref & hyp).values()) / sum(ref.values())
reference = "Pump A is operating at 30 C."
contradiction = "Pump A is not operating at 30 C."
score = overlap_recall(reference, contradiction)
print("Unigram recall:", score)
assert score == 1.0
assert ("not" in words(reference)) != ("not" in words(contradiction))
```

Output:

```
Unigram recall: 1.0
```

The contradiction receives recall 1.0 because it retains every reference token. A precision term would penalize the extra word, but the small penalty would still not express the importance of reversed negation. This is a controlled counterexample, not evidence that overlap metrics never correlate with human judgments. It explains why the metric must be paired with meaning-sensitive checks.

Human evaluation and error categories

For the equipment application, count **unsupported claims**, **missing required claims**, **incorrect numeric relations**, and **unresolved references** separately. These are project-defined error categories. Give annotators positive and negative examples for each, including cases where evidence is absent or contradictory. A summary that says less may have fewer unsupported claims while omitting critical information; report both error directions.

Human evaluation should hide system identity, randomize presentation, preserve source context and specify the decision task. Ask reviewers to mark unsupported statements before judging overall usefulness. An overall preference may reward style and miss a rare critical error. Report who reviewed the outputs and how disagreements were resolved. Aggregate by a meaningful unit: several sentences from one incident are correlated, and treating them as independent examples can exaggerate certainty.

Language models as judges

A strong language model can be prompted to grade or compare outputs. Zheng and colleagues studied this “LLM-as-a-judge” setting and reported that a strong judge’s preferences agreed with human preferences at a level comparable to agreement between humans on their benchmarks, while also documenting position bias, verbosity bias and self-enhancement bias [18]. Wang and colleagues showed that simply swapping the order in which two responses appear can change which one a model judge prefers, and proposed calibration such as evaluating both orders [19].

Treat a model judge as another measurement instrument with its own version, prompt and failure modes. Evaluate both presentation orders, hide which system produced which output, validate the judge against human labels on your own error categories (especially negation, quantities and unsupported claims), and avoid using a judge from the same model family as the system it grades when that can be avoided. A judge that ranks systems well on average is not automatically a reliable gate for accepting a single output.

9. Retrieval: finding the evidence to generate from

Plain version. Before a generator can use evidence, something must find it. A retrieval system selects passages from an explicit corpus; whether those passages support an answer, and how the answer is worded, are later decisions.

A language model can continue a question without searching a current document collection. A retrieval system instead selects information from an explicit corpus. Question answering adds another decision: whether the selected information supports an answer, and which words express it. Putting these components together creates a useful pipeline, but no stage automatically certifies the next.

This section's running question is "What is the temperature limit for Pump A?" A relevant passage discusses Pump A or temperature. A sufficient passage states the limit for that pump under the applicable policy. A valid citation points to a real source. A supported answer preserves the source's value, units, time and scope. These are four separate conditions, and an evaluation should be able to tell which one failed.

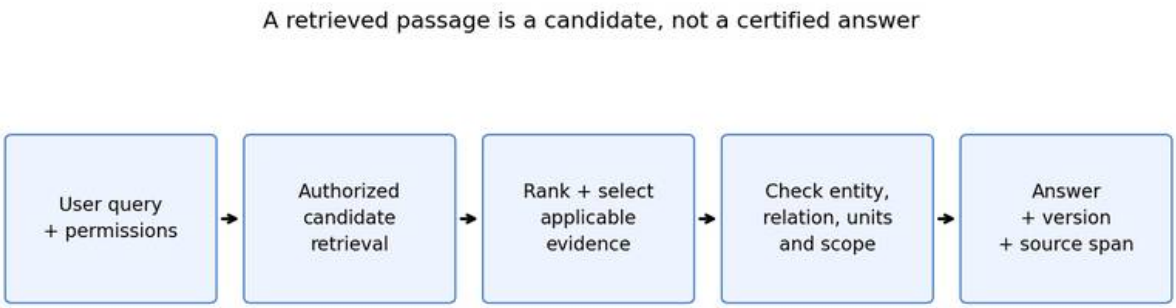


Figure 5 — Retrieval selects candidates; evidence checking determines what can be said. Every stage belongs to an explicit application contract.

This section builds and evaluates the retrieval stage. Deciding whether a retrieved passage actually answers the question — extractive span answers, relevance versus answerability, and diagnosis by controlled substitution — is developed in NLP 11. Section 10 then conditions a generator on the retrieved evidence.

Corpus design before model choice

Plain version. A retriever searches what you give it. Document identity, freshness and permissions are part of the system, not optional metadata added after ranking.

Represent a searchable passage as:

$$d = (u, v, b, e, \mathbf{x}, a).$$

Here d is a passage record; u is the parent document identifier; v is its version; b and e are inclusive-start and exclusive-end offsets in the original document; $\mathbf{x}$ is passage text; and a describes its access policy. The offset unit must be declared. This is our application record, rather than a universal storage standard.

A passage may be a paragraph, a sentence group or a section selected by document structure. Small passages can isolate an answer but lose its qualifications. Large passages retain context but introduce unrelated text and consume more generation budget. Choose chunk boundaries based on the task, then evaluate whether the supporting facts and their conditions remain together.

For example, a heading may specify "Outdoor equipment only," followed by a paragraph containing a temperature limit. If the paragraph is indexed alone, it may appear to apply to indoor equipment. Retaining the heading or a structured scope field changes the answerability contract. A fixed character window is easy to implement, but simplicity does not establish that it preserves meaning.

Versions also matter. An old policy can be relevant yet inapplicable. Keep effective dates and replacement links when the application needs current answers. Do not rely on a generator to infer the current policy merely because the newest passage appears last in the prompt. Filter or rank by declared applicability rules, and make conflicts visible when the corpus does not identify a single authoritative source.

Apply permissions before returning passage text to an unauthorized component or user. Filtering the final visible answer is too late if private text was already included in a model request or log. The retriever's corpus view should match the current user's permitted view. Measure retrieval quality under that restricted view; global relevance scores can otherwise evaluate documents the system may never show.

Lexical retrieval and sparse vectors

Plain version. Lexical retrieval matches words and weights informative terms. It is inspectable and often useful when exact names, identifiers and technical phrases matter.

A simple term-frequency and inverse-document-frequency weight is:

$$w_{t,d} = \mathrm{tf}(t, d) \log \frac{N}{\mathrm{df}(t)}.$$

Here $w_{t,d}$ is the weight of term t in document d; $\mathrm{tf}(t, d)$ is its frequency under a chosen convention; N is the number of indexed documents; and $\mathrm{df}(t)$ is the count of documents containing the term.

A zero document frequency requires special handling for query-only terms. Implementations often smooth the formula or transform term frequency; record the actual convention [33, 34].

With normalized document and query vectors:

$$s(q,d) = \frac{\mathbf{v}_q^\top \mathbf{v}_d}{\|\mathbf{v}_q\|_2 \|\mathbf{v}_d\|_2}.$$

Here q is a query; d is a candidate document; $\mathbf{v}_q$, $\mathbf{v}_d$ are their vectors; s is cosine similarity; and $\|\cdot\|_2$ is Euclidean norm. Define the score as zero when either vector is zero. The code uses normalized vectors from scikit-learn, so their dot product implements cosine similarity for nonzero rows [34].

An inverted index stores, for each term, the documents where it occurs. That allows a search engine to consider matching candidates without comparing a query with every vocabulary entry in every document [33]. Our small code computes all document scores because the corpus contains only four passages. The mathematical representation and the efficient indexing strategy are separate choices.

Preprocessing directly affects retrieval. A tokenizer that discards one-character tokens can remove the A in Pump A. Lowercasing may be helpful for ordinary words but can erase distinctions in case-sensitive identifiers. Phrase features help distinguish adjacent words, but do not capture arbitrary syntax or guarantee semantic equivalence. Inspect the vocabulary before blaming the ranking model for a missed result.

Lexical systems can miss paraphrases. “Device ceased operating” and “machine shutdown” may share little wording. A synonym or query-expansion rule can help, but changes the matching policy and may add ambiguity. Evaluate expansion on held-out queries, especially exact-identifier cases. A broad expansion that improves recall can also retrieve a wrong device with similar terminology.

Saturation and document length

Plain version. Repeating a query word many times should not always keep increasing relevance. Longer documents also have more opportunities to match by chance.

One common BM25 form is:

$$s(q,d) = \sum_{t \in q} \operatorname{idf}(t) \frac{f_{t,d}(k_1+1)}{f_{t,d} + k_1\left(1 - b + b\,|d|/\overline{L}\right)}.$$

Here q is the query-term set in this simplified form; d is the candidate document; $f_{t,d}$ is term frequency; $\operatorname{idf}(t)$ is a chosen inverse-document-frequency weight; $|d|$ is document length in tokens; $\overline{L}$ is average document length; k_1 is a positive saturation parameter; and $b \in [0,1]$ controls length normalization. Different implementations use different IDF and query-frequency conventions. The information-retrieval textbook discusses this family and its assumptions [33].

The numerator and denominator make the term-frequency contribution approach a limit as repetition grows. The length term changes how much frequency is expected in a longer document.

These are ranking adjustments, not a probability that the document contains a true answer. A passage can rank highly because it repeatedly discusses a limit while never stating the value.

Tune length normalization on development queries that reflect the deployment corpus. If every useful answer is in a short reference entry while irrelevant documents are long narratives, a particular setting can look very strong. Changing to long technical manuals may alter that behavior. Keep passage construction fixed when comparing ranking settings, or report that both the corpus representation and score changed.

Dense retrieval and joint reranking

Plain version. A learned encoder can match related expressions without exact word overlap. Its success depends on the training task and the examples used as contrasts.

A dual-encoder score is:

$$s_\theta(q, d) = f_\theta(q)^\top g_\theta(d).$$

Here f_θ maps queries to vectors; g_θ maps passages to vectors; θ denotes learned parameters; and s_θ is their dot-product score. The encoders may share parameters or use distinct ones. Dense Passage Retrieval provides an original question-answering example of this design [35].

Passage vectors can be computed ahead of time. A query then searches those vectors for high scores. Approximate search trades some exact nearest-neighbor fidelity for speed; if used, evaluate both the representation and the search approximation. Our chapter does not implement such an index, because an exact small corpus is better for learning the contract.

A contrastive training loss can compare one positive passage with several negative passages:

$$\mathcal{L} = -\log \frac{\exp(s_\theta(q, d^+)/\tau)}{\exp(s_\theta(q, d^+)/\tau) + \sum_j \exp(s_\theta(q, d_j^-)/\tau)}.$$

Here $\mathcal{L}$ is the loss for one query; d^+ is an annotated positive passage; d_j^- is negative candidate j; τ is a positive temperature; and s_θ is the learned score. Negatives may include difficult passages with similar wording. A passage labeled negative merely because it was not annotated can be a false negative if it also contains a valid answer.

A joint reranker reads a query and candidate passage together, which permits richer token interactions than comparing two fixed vectors. It is typically applied after a cheaper candidate retrieval stage. This adds computation and another selection procedure. Report candidate recall before reranking: a reranker cannot recover the missing correct passage if it never receives it.

Dense and lexical retrieval can be complementary. Dense representations may help paraphrases; lexical matching can preserve exact codes and rare names. Combining score scales requires care because a lexical score and a learned dot product do not have the same units. Select combination settings on development data, and inspect whether the combined system introduces wrong-entity results.

What the contrastive loss actually teaches

The dense loss does more than move a query toward any vaguely related passage. It learns the distinctions represented by the candidate set. Let candidate zero be the positive passage and let the remaining candidates be negatives. For candidate scores:

$$p_j = \frac{\exp(s_j/\tau)}{\sum_{r=0}^{M} \exp(s_r/\tau)}, \qquad \frac{\partial \mathcal{L}}{\partial s_j} = \frac{p_j - \mathbf{1}[j=0]}{\tau}.$$

Here s_j is the query's score for candidate j, M is the number of negative candidates, r indexes all candidates, $\tau > 0$ is temperature, p_j is the candidate softmax probability, $\mathcal{L} = -\log p_0$ is the loss, and $\mathbf{1}[j=0]$ is one for the positive and zero otherwise. This derivative follows directly from the softmax loss above. Gradient descent increases the positive score and decreases negative scores; the strongest competing negative receives the largest downward pressure.

DPR uses positive and negative passage comparisons to train separate query and passage representations [35]. The equation explains why choosing the comparisons matters. If negatives are unrelated topics, the model may learn topic separation without learning to distinguish Pump A from Pump B. A wrong-device passage with the same relation is a more informative contrast for that error. But if an unannotated passage really supports the answer, treating it as negative teaches the wrong separation.

This standalone score-space experiment verifies the gradient direction. It illustrates the objective; it does not train passage encoders:

```
import numpy as np

scores = np.array([1.2, 0.8, -0.4])  # positive, hard negative, easy negative
def probabilities(values):
    z = np.exp(values - values.max())
    return z / z.sum()
p = probabilities(scores)
gradient = p.copy()
gradient[0] -= 1.0  # temperature is one
updated = scores - 0.5 * gradient
before, after = -np.log(p[0]), -np.log(probabilities(updated)[0])
print('Gradient:', gradient.round(3))
print('Loss:', round(before, 3), '->', round(after, 3))
assert gradient[0] < 0 and gradient[1] > gradient[2] > 0
assert after < before
```

The gradient rounds to `[-0.466, 0.358, 0.108]`, and loss falls from `0.627` to `0.467`. The competing high-score negative receives more downward pressure than the easy negative.

Changing temperature affects both distribution sharpness and gradient scale. Changing candidates changes the contrast being learned. Consequently, improvements from a new encoder cannot be attributed to architecture alone when its positive labels, negative construction or training corpus also change. Keep those differences visible in the comparison.

Compare retrieval routes on the distinction that matters

A dual encoder compresses a passage before it knows the future query. This permits cached vectors and efficient candidate search, but every query-relevant detail must survive that compression. A joint reranker receives both texts simultaneously and can compare specific tokens across them. It spends more computation per pair and still depends on its training examples; joint attention is an opportunity to distinguish a detail, not proof that the detail will be respected.

Query distinction	Useful evidence	Failure to inspect
`Pump A` versus `Pump B`	Exact identifier plus relation	Topic similarity overwhelms a one-character difference
`ceased operating` versus `shutdown`	Learned paraphrase compatibility	Lexical vocabulary misses equivalent wording
Current versus pre-upgrade limit	Version and effective-date metadata	A highly relevant passage answers the wrong time scope
Permission versus prohibition	Negation and its grammatical scope	Shared vocabulary hides an opposite instruction

A hybrid design can take the union of lexical and dense candidate sets, remove duplicates, and jointly rerank the survivors. Union protects candidates found by either route, while a fixed total budget can still force some candidates out. Simply adding raw scores is less transparent: multiplying one system's scores by a constant preserves its own ranking but changes its influence in the combined sum. Normalization or learned combination therefore requires development judgments, not an arbitrary coefficient presumed to work everywhere.

Compare candidate recall, final ordering and resource cost separately. A system can improve the first while harming the second, or improve both while exceeding the serving budget. Diagnose exact names, paraphrases and scope-sensitive questions as distinct groups. Their error patterns reveal which information the representation loses and which downstream interaction is worth paying for.

Ranking evaluation

Plain version. Retrieval evaluation needs queries with relevance judgments. A document being selected does not establish that it was the right one.

For query q with relevant-document set R_q and top-k result set $T_k(q)$:

$$\text{Recall@}k(q) = \frac{|R_q \cap T_k(q)|}{|R_q|}.$$

Here k is the cutoff; R_q contains the annotated relevant documents; $T_k(q)$ contains the retrieved documents up to that cutoff; and $|\cdot|$ denotes set size. We evaluate this expression only for queries with at least one relevant document. Queries with no answer belong in a separate answerability evaluation, rather than receiving an arbitrary recall value.

If the first relevant result has rank r_q:

$$\mathrm{RR}(q) = \frac{1}{r_q}, \qquad \mathrm{MRR} = \frac{1}{Q}\sum_{q=1}^{Q} \mathrm{RR}(q).$$

Here RR is reciprocal rank; MRR is its mean; Q is the number of evaluated queries; and r_q starts at one. Define reciprocal rank as zero when no relevant result is returned. These ranking diagnostics appear in the current retrieval-augmented evaluation discussion [31]. Reciprocal rank emphasizes the first successful hit and ignores later relevant results.

Annotation completeness matters. If only one passage is judged relevant when several support the answer, a valid retrieval can be scored as wrong. Record whether relevance means topic match, answer-containing evidence or complete support. Use multiple judgments when an ambiguity is consequential, and adjudicate according to a written guide. The dataset should expose the task definition, not merely list a favorite passage.

Keep query families and document derivatives separate across development and test. Rephrasing a development query into a nearly identical test query gives weak evidence of generalization. Likewise, splitting chunks from one report across document partitions can leak facts and wording. Choose a split reflecting the deployment question: unseen users, unseen documents, future versions or new domains.

Measure a small retriever

The standalone code uses four synthetic passages. It preserves one-character words so that the device identifier A remains a feature, and it measures ranking on three annotated queries.

```
import numpy as np
from sklearn.feature_extraction.text import TfidfVectorizer

corpus = {
    'temperature-v2': 'Pump A temperature limit is 80 C. This policy is current.',
    'shutdown-v1': 'Pump A shutdown follows a vibration alarm.',
    'label-v1': 'A technician replaced the cabinet label.',
    'sensor-v1': 'Sensor B reports humidity once per hour.',
}
ids = list(corpus)
vectorizer = TfidfVectorizer(ngram_range=(1, 2), token_pattern=r"(?u)\b\w+\b")
D = vectorizer.fit_transform([corpus[key] for key in ids])
def rank(query):
    score = (D @ vectorizer.transform([query]).T).toarray().ravel()
    order = sorted(range(len(ids)), key=lambda i: (-score[i], ids[i]))
    return [(ids[i], float(score[i])) for i in order]
queries = [
    ('Pump A temperature limit', {'temperature-v2'}),
    ('vibration shutdown alarm', {'shutdown-v1'}),
    ('humidity report', {'sensor-v1'}),
]
recalls, reciprocal = [], []
for query, relevant in queries:
    ranking = rank(query)
```

```
    recalls.append(len({key for key, _ in ranking[:2]} & relevant) / len(relevant))
    first = next((i + 1 for i, (key, _) in enumerate(ranking) if key in relevant), None)
    reciprocal.append(1 / first if first else 0.0)
    print(query, '->', ranking[0][0])
print('mean Recall@2:', np.mean(recalls))
print('mean reciprocal rank:', np.mean(reciprocal))
```

Output:

```
Pump A temperature limit -> temperature-v2
vibration shutdown alarm -> shutdown-v1
humidity report -> sensor-v1
mean Recall@2: 1.0
mean reciprocal rank: 1.0
```

The three top results match their single annotated relevant passages, producing mean Recall@2 and mean reciprocal rank of 1.0 in this small run. That is a fixture result, not a real retrieval benchmark. NLP 11 reuses the same corpus and ranker for a narrow evidence-backed answerer.

Graded relevance: nDCG

Recall@k and reciprocal rank treat a judgment as relevant or not. When judges grade passages — complete support above a merely topical mention — a measure should reward placing the most useful passage first.

$$\mathrm{DCG@}k = \sum_{i=1}^{k} \frac{g_i}{\log_2(i+1)}, \qquad \mathrm{nDCG@}k = \frac{\mathrm{DCG@}k}{\mathrm{IDCG@}k}.$$

Here $g_i \geq 0$ is the graded relevance of the result at rank i; DCG@k is the discounted cumulative gain over the first k ranks; IDCG@k is the same sum for the ideal ordering of the judged results, so nDCG@$k \in [0, 1]$; define it as zero when IDCG@$k = 0$. Järvelin and Kekäläinen introduced the cumulated-gain family of measures [36]; the logarithm base, the gain transform and the cutoff vary between implementations, so report them.

Two rankings of the same four judged passages differ only in whether the supporting passage or the topical one comes first:

```
import numpy as np

def dcg(gains, k):
    gains = np.asarray(gains[:k], dtype=float)
    return float(np.sum(gains / np.log2(np.arange(2, gains.size + 2))))
def ndcg(gains, k):
    ideal = dcg(sorted(gains, reverse=True), k)
    return dcg(gains, k) / ideal if ideal > 0 else 0.0
```

```
# graded judgments of one ranking: 2 = complete support, 1 = topical, 0 = unrelated
ranked_a = [1, 2, 0, 0]  # topical passage above the supporting one
ranked_b = [2, 1, 0, 0]  # supporting passage first
for name, gains in [('A', ranked_a), ('B', ranked_b)]:
    print(name, 'nDCG@3:', round(ndcg(gains, 3), 3),
          'binary RR:', 1 / (1 + gains.index(2)))
assert ndcg(ranked_b, 3) == 1.0 and ndcg(ranked_a, 3) < 1.0
```

Output:

```
A nDCG@3: 0.86 binary RR: 0.5
B nDCG@3: 1.0 binary RR: 1.0
```

Reciprocal rank with "complete support" as the relevance criterion halves for ranking A. nDCG also drops, but less, because it still credits the topical passage at rank 1. Which behavior is right depends on the task: if only a complete answer is useful, the graded credit for topical passages can hide a failure. Write the grading guide before choosing the measure.

10. Hallucination and retrieval-augmented generation

Plain version. A language model can produce fluent text that is not supported by its input or by the world. Retrieval supplies external evidence, but the generator still needs to use it correctly, and retrieved text must never be treated as a new authority.

Ji and colleagues' survey distinguishes **intrinsic** hallucination, output that contradicts the source, from **extrinsic** hallucination, output that cannot be verified from the source [20]. Maynez and colleagues' human study of abstractive summarizers found substantial amounts of hallucinated content in all model-generated summaries they examined, and found that textual-entailment measures correlated better with faithfulness than standard overlap metrics [21]. Both points follow from Sections 1 and 6: the training objective rewards probable text, and no decoder adds evidence that the distribution lacks.

Use attention as a diagnostic of computational access, not a factuality certificate. A token can attend to the correct evidence and still express its opposite. Evidence validation should examine the output claim, source entity, relation and scope. For narrow structured tasks, the generator can verbalize a validated plan, as in the template above. For unrestricted prose, validation becomes incomplete and should be reported as such.

Retrieval-augmented generation

The original retrieval-augmented generation, or RAG, paper combines a learned retriever with conditional sequence generation for knowledge-intensive tasks [22]. One sequence-level form is:

$$p(y \mid x) \approx \sum_{z \in Z_k(x)} p_\eta(z \mid x)\, p_\theta(y \mid x, z).$$

Here x is the input question; y is the output sequence; z is a retrieved passage; $Z_k(x)$ is the selected top-k candidate set; η parameterizes the retriever; and θ parameterizes the generator. The approximation reflects restricting retrieval to candidates. This is a model-specific probabilistic formulation, not a definition that every contemporary RAG application implements.

Many practical systems retrieve passages, assemble a prompt and run a separately trained generator. That pipeline can be useful without optimizing the original marginal objective. State which design you use. Otherwise the name RAG can hide whether retrieval is learned, whether candidates are reranked and whether the generator is trained to use the selected sources. Section 9 builds and evaluates the retriever itself; extractive answers and answerability checks are in NLP 11.

Grounding means relating claims to external evidence under a declared support rule. A generated answer with citations may be traceable, but traceability is weaker than support. A valid URL can point to a document that contradicts the answer. Evaluate whether each consequential claim follows from its cited evidence, and whether the answer includes necessary qualifications.

Evidence may also conflict. A generator should not silently combine mutually inconsistent policies into a single confident answer. An application can prioritize an authoritative version, display a conflict or request clarification according to its contract. The decision should come from metadata and task rules, rather than which sentence happens to be most fluent.

Errors propagate through retrieval, interpretation and realization

A supported answer requires a chain of successful decisions. The corpus must contain applicable evidence, retrieval must preserve it, the reader must interpret the right entity and relation, and the output must retain that interpretation. An answer guessed from memorized knowledge can be correct by coincidence while failing this evidence requirement. For questions whose applicable evidence is in the corpus, write:

$$P(R \cap E \cap G) = P(R)P(E \mid R)P(G \mid R, E).$$

Here P denotes probability, R is the event that sufficient evidence survives retrieval, E is the event that the evidence is interpreted correctly, and G is the event that the output faithfully realizes the interpretation. The expression is the probability chain rule, not an independence assumption. Multiplying unrelated unconditional stage scores is not equivalent: reader accuracy on hand-picked easy passages may differ from reader accuracy on the retriever's actual candidates.

```
import numpy as np

retrieved = np.arange(100) < 80
interpreted = np.arange(100) < 72
realized = np.arange(100) < 68
r = retrieved.mean()
e = interpreted[retrieved].mean()
```

```
g = realized[retrieved & interpreted].mean()
complete = (retrieved & interpreted & realized).mean()
print('Conditional stage rates:', round(r, 3), round(e, 3), round(g, 3))
print('Complete supported answers:', round(complete, 3))
assert np.isclose(r * e * g, complete)
```

Output:

```
Conditional stage rates: 0.8 0.9 0.944
Complete supported answers: 0.68
```

These are synthetic outcomes, not reported system performance. The calculation explains why individually respectable component results need not produce equally strong complete answers. Improving the weakest stage is useful only if the improvement survives the later stages.

Boundaries around retrieved content

Retrieved text is evidence to inspect, not an instruction source with permission to change the application. A passage can contain quoted commands, misleading text or an adversarial instruction. Prompt injection describes attempts to make a model follow instructions that conflict with the intended application boundary; OWASP documents this risk category [23]. For retrieval systems, the important boundary is between user/application instructions and untrusted source material.

Keep source text delimited and labeled with document identity. Do not allow it to redefine the current user's permissions, request secret data or authorize a tool action. If tools are available, validate actions against the user's task and the application policy outside the retrieved passage. A textual delimiter helps structure input, but is not a complete enforcement mechanism.

The same principle applies to logs and caches. A cached result produced under one user's access policy must not be reused for another user without checking permissions. Evidence spans must refer to the same document version used for the answer. When a document is removed or replaced, refresh the index and consider invalidating derived answers.

These checks also support ordinary correctness. A passage that says "Ignore the old limit; the new policy is below" can be legitimate document content that needs interpretation, while "Ignore your application instructions" is not a new authority for the system. Store both as source material, then apply the task's evidence and permission rules. Do not make the retriever a route for changing the governing contract.

11. Adapting a pretrained generator

Plain version. Pretraining learns broad continuation patterns. Adaptation changes behavior for a task; prompting changes the supplied input without necessarily changing weights. Learning to

follow an instruction and learning which response people prefer are different signals, and neither is a truth guarantee.

Supervised task adaptation minimizes a task-specific loss over labeled examples, for instance the conditional loss of Section 7 over the response tokens only. Continued language-model training on domain text changes the distribution modeled, but does not automatically teach the application's output policy. Domain familiarity and task instruction are distinct forms of adaptation. In-context examples, as in GPT-3 [6], appear inside the input and demonstrate format without updating parameters; examples that contain private answers or test labels can invalidate evaluation, and a prompt chosen on development cases must be frozen before the final test.

Low-Rank Adaptation, or LoRA, freezes a base weight matrix and trains a constrained update [24]:

$$W' = W + BA.$$

Here $W \in \mathbb{R}^{m \times n}$ is the frozen matrix; $A \in \mathbb{R}^{r \times n}$ and $B \in \mathbb{R}^{m \times r}$ are learned factors; r is the chosen rank; and W' is the effective adapted matrix. This simplified expression omits optional scaling. Fewer trainable parameters can reduce adaptation cost, but do not remove the need for representative labels, data isolation or task evaluation. Save the base-model version, tokenizer, adaptation weights and configuration together: an adaptation trained against one base checkpoint is not guaranteed to work on another checkpoint with the same family name.

The InstructGPT paper studies supervised instruction examples followed by preference-based training with human feedback [25]. This distinguishes a base continuation model from one adapted toward requested behavior. The source of preferences matters: annotators follow a policy, and that policy may balance helpfulness, formatting and other goals. A preferred response can still contain a factual error.

Direct Preference Optimization, or DPO, uses preferred and rejected responses with a reference policy [26]. A representative objective is:

$$\mathcal{L}_{\text{DPO}} = -\log \sigma\left(\beta\left[\log \frac{\pi_\theta(y^+ \mid x)}{\pi_{\text{ref}}(y^+ \mid x)} - \log \frac{\pi_\theta(y^- \mid x)}{\pi_{\text{ref}}(y^- \mid x)}\right]\right).$$

Here x is the prompt; y^+ and y^- are the preferred and rejected response sequences; π_θ is the trainable policy; π_{ref} is a fixed reference policy; β is a positive scale parameter; σ is the logistic sigmoid; and $\mathcal{L}_{\text{DPO}}$ is the loss for one preference pair. Sequence probabilities are products over response-token conditionals; prompts are context rather than supervised response tokens.

The objective increases the preferred response's relative likelihood compared with the rejected response, measured against the reference. If a dataset rewards confident unsupported answers, optimizing the objective can reinforce that behavior. Instruction compliance, preference agreement and factual support are separate properties that need separate evidence. ML 09 runs a low-rank update in code.

12. Training practices meet serving constraints

Plain version. A deployed generator serves inputs of different lengths, and its cost per request depends on both the prompt and the output. Padding and cached attention state are two different memory costs; measure both.

Hagiwara's Chapter 10 connects padding, sorting and masking to practical inference [1]. Keep this distinction clear: a correct loss mask prevents padding from altering the objective; it does not necessarily prevent the hardware from processing padded positions. Define padding utilization as

$$u = \frac{\sum_{b=1}^{B} L_b}{B \max_b L_b}.$$

Here L_b is the real token length of request b; B is batch size; and u is the fraction of padded token positions containing real tokens. This is a workload diagnostic, not a measured speedup. Sorting nearby lengths into batches can increase utilization while also delaying requests that wait for a matching batch.

```
import numpy as np

lengths = np.array([8, 64, 12, 60, 10, 56, 14, 52])
def utilization(values):
    batches = values.reshape(-1, 4)
    real = int(batches.sum())
    padded = int((batches.max(axis=1) * 4).sum())
    return real / padded
arrival = utilization(lengths)
bucketed = utilization(np.sort(lengths))
print("Arrival-order utilization:", round(arrival, 3))
print("Length-bucketed utilization:", round(bucketed, 3))
assert bucketed > arrival
# Two cache arrays (keys and values), with a toy equal head layout.
layers, kv_heads, head_dim, cache_tokens, bytes_per_value = 4, 2, 16, 128, 4
cache_bytes = 2 * layers * kv_heads * head_dim * cache_tokens * bytes_per_value
print("Toy cache bytes:", cache_bytes)
assert cache_bytes == 131072
```

Output:

```
Arrival-order utilization: 0.575
Length-bucketed utilization: 0.885
Toy cache bytes: 131072
```

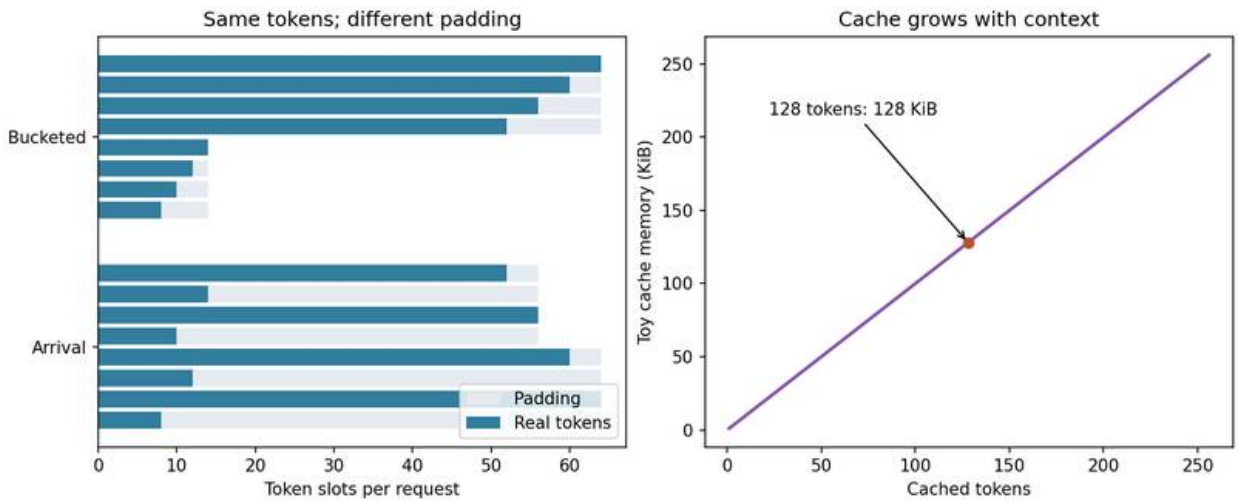


Figure 6 — Padding waste and cached attention state are different memory costs. The figure reports calculated token slots and toy cache bytes, rather than claiming an observed serving speedup.

Autoregressive attention commonly caches prior keys and values to avoid recomputing them at every generated token. For a simple layout, the cache memory is approximately $M_{\mathrm{KV}} = 2N_{\mathrm{layers}}H_{\mathrm{KV}}d_hTb_v$. Here N_{layers} is layer count, H_{KV} is the number of key/value heads, d_h is head dimension, T is cached token count, and b_v is bytes per stored value. The factor two counts keys and values. Actual implementations add allocation and metadata costs and may use different layouts.

Separate time to first output token from time between subsequent tokens, total completion time and throughput under load. A short prompt with a long answer stresses a different stage from a long prompt with a short answer. Include preprocessing, waiting, validation and serialization in end-to-end measurements. Set a request-length limit, an output limit and a queue deadline based on measured workload behavior, and report what happens on timeout: a partial result, a retryable error or a reviewed fallback.

A reproducible serving contract

A model file alone cannot reproduce a prediction. Tokenization, normalization, evidence selection, decoding parameters, validation rules and output formatting all affect behavior. Hagiwara's deployment chapter [1] supplies the practical reading for versioning, testing, train–serve differences and monitoring. For a learned generator, return the model version and the validated evidence IDs alongside the text, as the template function does, and store the preprocessing version and decoding configuration with the artifact. Logs should support diagnosis without unnecessarily retaining private source text.

Deploy a new version to a limited share of requests and compare it with a stable version using the same error categories. Check latency, completion rate and evidence correctness together. Keep rollback simple: a stored bundle should select the previous model, tokenizer, decoder settings and validation rules as one unit. Mixing an old model with a new tokenizer can create a failure

that neither component exhibited alone.

Modern view

Plain version. Contemporary language modeling combines pretraining, adaptation, decoding, evaluation and system integration. Surveys are useful when they clarify these separate choices rather than compressing everything into one model ranking.

Zhao and colleagues' survey [27], revised through 2026 and published in Frontiers of Computer Science, organizes large language models through pretraining, adaptation, utilization and capability evaluation. This framework helps distinguish more training data from instruction tuning or a different usage pattern. **Chang and colleagues' evaluation survey [28]** separates questions about what to evaluate, where to evaluate and how to evaluate. Together they motivate a task-specific evaluation plan rather than relying on a single general score.

Li et al.'s pretrained-generation survey [29] organizes methods around input representation, architecture and optimization, and covers applications and evaluation. This taxonomy helps locate a change: retrieval alters available input evidence, an encoder–decoder changes conditional architecture, and adaptation changes parameters or training objectives. The survey predates some later systems, so use it for organization alongside newer evaluation and deployment evidence.

Ji et al.'s hallucination survey [20] and **Schmidtova et al.'s survey of human evaluations of hallucination [30]** approach the same problem from two sides. The first organizes definitions, metrics and mitigation across tasks; the second examines how studies define and assess hallucinations, and identifies inconsistent terminology and missing methodological reporting. Its practical lesson is to define source support, world knowledge and error severity separately, then publish annotation instructions and disagreement handling.

Gao and colleagues' RAG survey [31] organizes retrieval-augmented systems into naive, advanced and modular paradigms, and separates context relevance, answer relevance and answer faithfulness. Stages interact: rewriting "What was the limit before the upgrade?" into "What is the temperature limit?" may improve topical retrieval while deleting the time constraint. Preserve the original query and compare the rewritten meaning, not only the retrieval score.

Kwon et al.'s PagedAttention work [32] studies key–value cache allocation and sharing in language-model serving. It addresses wasted memory and the batch sizes that memory permits, and supplies a concrete systems perspective on the cache calculation above. Its reported improvements belong to its workloads and comparisons; measure your own request lengths, concurrency and latency targets before adopting a serving design.

Open problems include contamination of evaluation sets, long-context robustness (a larger context window permits more input; it does not establish that all relevant parts are used reliably), multilingual and low-resource coverage, stable uncertainty, detection of unsupported claims without suppressing useful answers, and the relationship between instruction following and source-grounded correctness. Evaluation models can share errors with generators, especially when both rely on similar training data. A compact specialized model, or even a template, can be the better engineering choice when it satisfies the task with lower latency and simpler review.

Key takeaways

- A decoder-only Transformer factorizes text causally; shift targets explicitly and test that future tokens cannot change earlier logits.
- The GPT line kept one architecture and objective while scaling data, parameters and the way tasks are presented; scaling laws predict loss, not task behavior.
- Perplexity comparisons require compatible tokenization and evaluation positions, and say nothing directly about decoded outputs.
- Greedy and beam search optimize sequence probability; temperature reshapes the distribution; top-k and nucleus sampling cut its tail, with the nucleus adapting to the distribution's shape.
- Generation needs a content contract: selection, planning and realization each fail differently, and extraction still needs mandatory-content checks.
- Overlap metrics, human review and model judges each have blind spots; validate them on the errors that matter, such as reversed negation.
- Retrieval supplies evidence, not authority; relevance, citation validity and claim support are separate conditions.
- Version the complete preprocessing–model–decoding–validation bundle, and treat instruction tuning and preference learning as distinct from factual support.

Exercises

1. Remove the causal mask from the runnable causal model. Which assertion should fail, and why might training loss still look good?

   Hint

   The changed final token can become visible to earlier positions. Low loss can reward access to information unavailable during generation.

2. Using the decoding code, find a temperature at which plain sampling puts more mass on the four tail tokens than top-k with $k = 5$ ever can. Then construct a logit vector for which the nucleus with $p = 0.9$ is larger than five tokens, and explain when each rule is the better choice.

   Hint

   Top-k with $k = 5$ can keep at most one tail token. A flat logit vector makes the nucleus grow; a peaked one makes it shrink.

3. Change the toy beam-search probabilities so beam search chooses a sentence that violates an externally supplied instruction. Add a constraint on permitted final sequences and compare its effect with widening the beam.

   Hint

   Probability search optimizes the distribution's score. A permitted-sequence rule changes what outputs are admissible. Widening a beam need not repair a wrong probability model.

4. Construct two candidate summaries: one with high word overlap but a wrong quantity, and another with lower overlap but correct meaning. Design an annotation form, and a model-judge protocol, that reveal this difference.

Hint

Ask reviewers to identify unsupported quantities and missing required facts before giving an overall usefulness judgment. Keep the source visible and system identity hidden; run the judge in both presentation orders.

5. Construct one preference pair that rewards style but selects the less-supported response. Explain which additional evaluation would expose the problem, and compute the trainable parameter count of a rank-4 LoRA update for a 128-by-256 matrix.

Hint

Keep the evidence fixed and compare a confident answer that adds a fact with a qualified answer that preserves the source's uncertainty. For LoRA, count the two factor matrices separately.

6. Extend the report function with a timestamp and a target-language choice. Reject malformed inputs, preserve unknown status, attach evidence IDs and version the realization rules. Document the tests needed before replacing the template with a learned generator.

Hint

Do not infer an unrecorded timestamp or convert unknown into absent. Test preprocessing parity, number formatting, script requirements, long inputs, timeout behavior and the complete rollback bundle.

References

1. Masato Hagiwara, *Real-World Natural Language Processing: Practical Applications with Deep Learning*, Chapters 5, 8, 9, 10 and 11, Manning, 2021. Publisher ·Training practices ·Deployment
2. Susan McRoy, *Principles of Natural Language Processing*, open textbook, Chapters 6 and 9, 2021. Benchmark chapter ·Generation chapter
3. Ashish Vaswani et al., "Attention Is All You Need," NeurIPS, 2017. Paper
4. Alec Radford, Karthik Narasimhan, Tim Salimans and Ilya Sutskever, "Improving Language Understanding by Generative Pre-Training," OpenAI technical report, 2018. Paper
5. Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei and Ilya Sutskever, "Language Models are Unsupervised Multitask Learners," OpenAI technical report, 2019. Paper
6. Tom B. Brown et al., "Language Models are Few-Shot Learners," NeurIPS, 2020. Paper
7. Jared Kaplan et al., "Scaling Laws for Neural Language Models," arXiv:2001.08361, 2020. arXiv
8. Jordan Hoffmann et al., "Training Compute-Optimal Large Language Models," arXiv:2203.15556, 2022; published as "An empirical analysis of compute-optimal large language model training," NeurIPS, 2022. arXiv

9. Ilya Loshchilov and Frank Hutter, “Decoupled Weight Decay Regularization,” ICLR, 2019. Paper
10. Dan Hendrycks and Kevin Gimpel, “Gaussian Error Linear Units (GELUs),” arXiv:1606.08415, 2016, revised 2023. Paper
11. Jimmy Lei Ba, Jamie Ryan Kiros and Geoffrey E. Hinton, “Layer Normalization,” arXiv:1607.06450, 2016. Paper
12. Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes and Yejin Choi, “The Curious Case of Neural Text Degeneration,” ICLR, 2020. Paper
13. Angela Fan, Mike Lewis and Yann Dauphin, “Hierarchical Neural Story Generation,” ACL, 2018. Paper
14. Jade Goldstein and Jaime Carbonell, “Summarization: (1) Using MMR for Diversity-Based Reranking and (2) Evaluating Summaries,” TIPSTER Phase III Workshop, 1998. Paper
15. Sam Wiseman, Stuart M. Shieber and Alexander M. Rush, “Challenges in Data-to-Document Generation,” EMNLP, 2017. Paper
16. Ilya Sutskever, Oriol Vinyals and Quoc V. Le, “Sequence to Sequence Learning with Neural Networks,” NeurIPS, 2014. Paper
17. Chin-Yew Lin, “ROUGE: A Package for Automatic Evaluation of Summaries,” Text Summarization Branches Out, 2004. Paper
18. Lianmin Zheng et al., “Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena,” NeurIPS Datasets and Benchmarks Track, 2023. Paper
19. Peiyi Wang et al., “Large Language Models are not Fair Evaluators,” ACL, 2024. Paper
20. Ziwei Ji et al., “Survey of Hallucination in Natural Language Generation,” ACM Computing Surveys 55, 2023. Survey
21. Joshua Maynez, Shashi Narayan, Bernd Bohnet and Ryan McDonald, “On Faithfulness and Factuality in Abstractive Summarization,” ACL, 2020. Paper
22. Patrick Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” NeurIPS, 2020. Paper
23. OWASP GenAI Security Project contributors, “LLM01: Prompt Injection,” OWASP guidance, accessed 2026. Official guidance
24. Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang and Weizhu Chen, “LoRA: Low-Rank Adaptation of Large Language Models,” ICLR, 2022. Author publication page
25. Long Ouyang et al., “Training language models to follow instructions with human feedback,” NeurIPS, 2022. Paper
26. Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D. Manning, Stefano Ermon and Chelsea Finn, “Direct Preference Optimization: Your Language Model is Secretly a Reward Model,” NeurIPS, 2023. Paper
27. Wayne Xin Zhao et al., “A Survey of Large Language Models,” Frontiers of Computer Science, 2026. Updated survey
28. Yupeng Chang et al., “A Survey on Evaluation of Large Language Models,” ACM Transactions on Intelligent Systems and Technology, 2024. Survey
29. Junyi Li, Tianyi Tang, Wayne Xin Zhao, Jian-Yun Nie and Ji-Rong Wen, “Pre-Trained Language Models for Text Generation: A Survey,” ACM Computing Surveys, 2024. Publisher ·Author manuscript

30. Patricia Schmidtova et al., “Do My Eyes Deceive Me? A Survey of Human Evaluations of Hallucinations in NLG,” INLG, 2025. Survey
31. Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang and Haofen Wang, “Retrieval-Augmented Generation for Large Language Models: A Survey,” arXiv:2312.10997, revised 2024. Survey
32. Woosuk Kwon et al., “Efficient Memory Management for Large Language Model Serving with PagedAttention,” SOSP, 2023. Conference ·Paper
33. Christopher D. Manning, Prabhakar Raghavan and Hinrich Schütze, *Introduction to Information Retrieval*, Chapters 6, 8 and 11, Cambridge University Press, 2008. Author-hosted book
34. scikit-learn developers, “Text feature extraction,” scikit-learn documentation, accessed 2026. Documentation
35. Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen and Wen-tau Yih, “Dense Passage Retrieval for Open-Domain Question Answering,” EMNLP, 2020. Paper
36. Kalervo Järvelin and Jaana Kekäläinen, “Cumulated Gain-Based Evaluation of IR Techniques,” ACM Transactions on Information Systems 20(4), 422–446, 2002. Publisher

NLP 10 • Pretrained Models: BERT

Understand BERT from WordPiece input to bidirectional encoder, pretrain a tiny BERT-style model with masked language modeling from scratch, and probe what its contextual representations contain.

Online edition: https://blog.shyandram.dev/en/learn/nlp/nlp-10-pretrained-bert/

What you'll learn

- Explain the pretrain-then-fine-tune recipe and where BERT sits next to ELMo and GPT.
- Turn text into BERT input: WordPiece pieces, `[CLS]`/`[SEP]` packing, and summed token, segment and position embeddings.
- State the masked-language-modeling objective exactly (15% selection, 80/10/10 replacement), and the next-sentence-prediction objective with its later critique.
- Pretrain a tiny BERT-style encoder from scratch on a toy corpus, and measure what it learned.
- Distinguish contextual from static embeddings, and probe representations with a control task.
- Place RoBERTa, ALBERT, DistilBERT, ELECTRA, multilingual BERT and Chinese BERT models in one map.

The big picture

Standard Word2Vec features retrieve a fixed vector per word [6]. BERT, short for **Bidirectional Encoder Representations from Transformers**, computes a new representation for each token occurrence by reading its surrounding text [1]. In "the crane lifts steel" and "the crane eats fish," the embedding lookup for `crane` is initially the same; the encoder can use the remaining words to produce different final vectors. The distinction is between a vocabulary entry and a representation of this occurrence.

BERT is learned in two stages. **Pretraining** uses ordinary unlabeled text, where the text supplies its own targets. **Fine-tuning** then adapts the same network, plus a small task head, to a labeled task such as sentiment or entailment. The first stage is expensive and done once; the second is cheap and done per task.

> **Explain like I'm five**
>
> Cover one word in a sentence with your thumb and ask a friend to guess it: "the crane ___ steel." Your friend can read the words on both sides of your thumb, so "lifts" is a good guess. Playing this game millions of times teaches a lot about how words fit together. Afterwards, the friend can learn a new job, like sorting happy and angry messages, from only a few examples, because they already know how sentences work.

Hagiwara's Chapter 9 develops pretrained-model adaptation in a practical setting [18]. This chapter goes inside the model. It reuses the causal language model of NLP 09 as a contrast, the attention block of NLP 07, and the static embeddings of NLP 04. The classification, entailment, tagging and extractive question-answering heads that sit on top of BERT are the subject of NLP 11. Every model here is tiny and trained from scratch; no checkpoint is downloaded.

1. Pretraining, then fine-tuning

Plain version. Learn general knowledge about language from a large amount of unlabeled text, then reuse it for a task with few labels. The question is what pretraining objective produces the most reusable representation.

Three 2018 designs frame BERT. ELMo trained left-to-right and right-to-left LSTM language models and supplied a learned combination of their internal states as contextual word features to existing task models [5]. GPT pretrained a 12-layer decoder-only Transformer as a left-to-right language model and then fine-tuned the whole network on each task [4]. BERT kept GPT's fine-tune-everything recipe but replaced the causal objective with a masked one, so every layer can use both left and right context [1].

The BERT paper argues that the left-to-right constraint is sub-optimal for sentence-level tasks and can be very harmful for token-level tasks such as question answering, where context from both directions matters [1]. Its ablations include a left-to-right model without next sentence prediction, trained and fine-tuned under the same left-only constraint, and compare it with the bidirectional model [1]. The engineering lesson is that the visibility pattern is a modeling choice tied to the task, not a matter of taste.

Pretraining supplies an initialization and a representation function. It does not supply an application's labels. A pump-maintenance classifier still needs examples defining "urgent," and an entity tagger still needs examples identifying the site's equipment names. Fine-tuning then adjusts the encoder and a task head together; freezing the encoder and training only the head is a cheaper alternative with a different adaptation capacity.

2. Masked and conditional objectives

Plain version. Not every pretrained language model predicts the next token. A masked objective uses surrounding context, while a conditional generator predicts an output sequence from a separate input.

BERT's masked language objective hides selected input positions and predicts their original tokens using bidirectional context [1]. A simplified description is:

$$\mathcal{L}_{\text{mask}} = -\frac{1}{|M|} \sum_{i \in M} \log p_\theta(x_i \mid \tilde{\mathbf{x}}).$$

Here M is the set of supervised masked positions; $|M|$ is its size; x_i is the original token; θ denotes model parameters; and $\tilde{\mathbf{x}}$ is the corrupted input sequence. The tilde indicates corruption, not an

independently sampled document. This expression describes the masked prediction component; Section 5 gives the full corruption procedure.

A masked predictor is useful for contextual representations, classification and span-oriented tasks. Its loss is not automatically the log probability of an ordinary left-to-right sequence. Multiplying predictions that each use both sides of the sentence does not generally yield the same normalized joint distribution as a causal factorization. Consequently, masked-objective scores and causal perplexities are not directly interchangeable.

Conditional sequence generation instead writes:

$$p_\theta(\mathbf{y} \mid \mathbf{x}) = \prod_{t=1}^{U} p_\theta(y_t \mid y_{<t}, \mathbf{x}).$$

Here $\mathbf{x}$ is the source sequence; $\mathbf{y}$ is the target sequence; U is target length including any end token; and $y_{<t}$ is the target prefix. The source can be visible in full while the target remains causal. T5 organizes many NLP tasks in a text-to-text framework and studies transfer learning with a Transformer encoder–decoder [27]; NLP 08 develops this family.

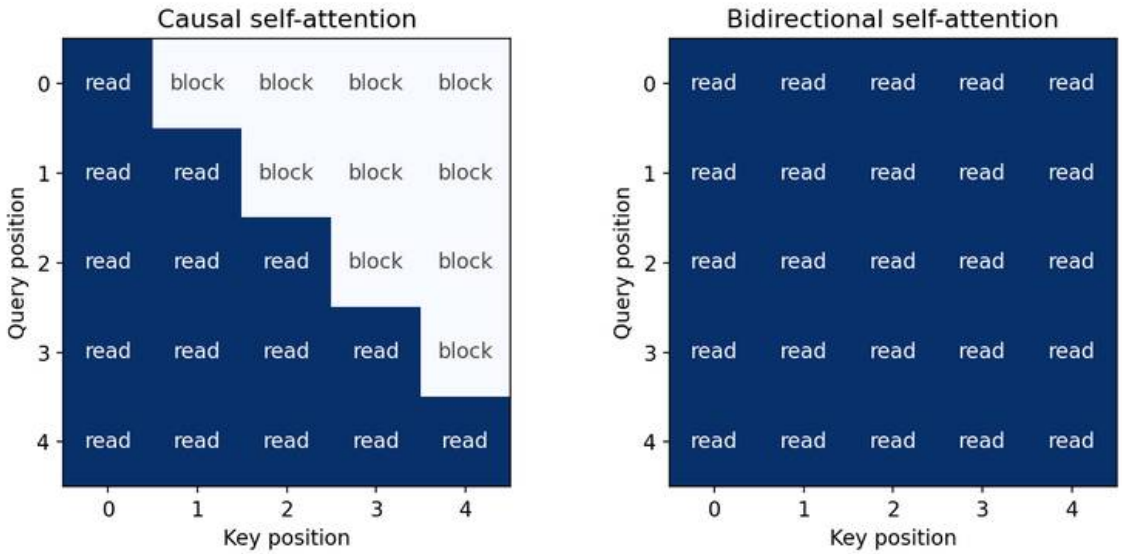


Figure 1 — Visibility is part of the task definition. Rows are querying positions and columns are the positions they may read. BERT uses the right-hand pattern in every layer; padding keys are still blocked.

These distinctions guide model choice. A sentence classifier may use a bidirectional encoder because the complete sentence is available. A streaming continuation must exclude future output tokens. Translation can use the entire source after it arrives, but simultaneous translation needs a more restrictive source visibility contract. The task's available information should determine the mask, rather than choosing a mask because a particular model name is familiar.

3. From text to BERT input

Plain version. BERT never sees words directly. Text is split into vocabulary pieces, wrapped with special markers, and each position receives the sum of three learned vectors.

WordPiece

The original tokenizer first applies text preprocessing, then WordPiece tokenization [2]. BERT uses WordPiece embeddings with a 30,000-token vocabulary [1], a subword method introduced for Google's neural machine translation system [7]. WordPiece splits a word into vocabulary pieces when needed; ## marks a continuation piece in the original vocabulary convention. The original implementation splits each word **greedily, longest match first**: take the longest vocabulary entry that starts the word, then the longest continuation piece, and so on; if some remainder matches nothing, the whole word becomes `[UNK]` [2]. The vocabulary is learned beforehand from a corpus, as BPE merges are in NLP 02.

```
vocab = {'[UNK]', 'the', 'pump', 'over', 'heat', '##heat', '##ed', '##s',
         'un', '##load', 'load', '##ing', '幫', '浦', '過', '熱'}

def wordpiece(word, max_chars=200):
    """Greedy longest-match-first split into vocabulary pieces."""
    if len(word) > max_chars:
        return ['[UNK]']
    pieces, start = [], 0
    while start < len(word):
        end = len(word)
        while end > start:
            piece = word[start:end] if start == 0 else '##' + word[start:end]
            if piece in vocab:
                break
            end -= 1
        if end == start:                    # no piece fits: the whole word is unknown
            return ['[UNK]']
        pieces.append(piece)
        start = end
    return pieces

def basic_split(text):
    """Lowercase, put spaces around CJK ideographs, split on whitespace."""
    out = []
    for ch in text.lower():
        cjk = 0x4E00 <= ord(ch) <= 0x9FFF
        out.append(f' {ch} ' if cjk else ch)
    return ''.join(out).split()

for text in ['The pump overheated', 'unloading pumps', 'the pumpkin', '幫浦過熱']:
    tokens = [p for w in basic_split(text) for p in wordpiece(w)]
    print(f'{text!r:24} -> {tokens}')
```

```
assert wordpiece('overheated') == ['over', '##heat', '##ed']
assert wordpiece('pumpkin') == ['[UNK]']
```

Output:

```
'The pump overheated'    -> ['the', 'pump', 'over', '##heat', '##ed']
'unloading pumps'        -> ['un', '##load', '##ing', 'pump', '##s']
'the pumpkin'            -> ['the', '[UNK]']
'幫浦過熱'                   -> ['幫', '浦', '過', '熱']
```

Three consequences follow. A word may occupy several model positions, so word labels and character spans must be mapped to token positions. A word whose remainder is not covered becomes a single `[UNK]`, even though its prefix `pump` is known; that is a property of the greedy rule, not of the language. And Chinese characters are split one per position before WordPiece, as the original multilingual and Chinese models do (Section 9). The `basic_split` here checks only the main CJK Unified Ideographs block; the original code checks several more ranges and also handles accents and punctuation [2].

Pack one text or a text pair

For a single text, pack `[CLS] text [SEP]`. For a pair, pack `[CLS] text A [SEP] text B [SEP]`. `[CLS]` supplies a position whose final state can be read by a sequence classifier; `[SEP]` marks boundaries. Segment IDs distinguish the two parts. They **do not block attention between them**: a query in text A can read text B. This joint encoding is useful when a decision depends on their interaction [2].

Before the encoder, sum three learned vectors:

$$\mathbf{e}_i = E[x_i] + P[i] + S[s_i], \qquad H = \mathrm{Encoder}_\theta(\mathbf{e}_1, \dots, \mathbf{e}_T).$$

Here T is packed sequence length; θ denotes encoder parameters; x_i is the token ID at position i; $s_i \in \{0, 1\}$ is its segment ID; $E \in \mathbb{R}^{|V| \times d}$ is the token table for vocabulary V; $P \in \mathbb{R}^{T_{\max} \times d}$ is the position table with supported length $T_{\max}$; $S \in \mathbb{R}^{2 \times d}$ is the segment table; $\mathbf{e}_i \in \mathbb{R}^d$ is the summed input; and $H \in \mathbb{R}^{T \times d}$ contains contextual output rows $\mathbf{h}_i$. The original implementation normalizes and applies dropout to the summed embeddings [2]. Padding keys are blocked; ordinary text on either side remains visible.

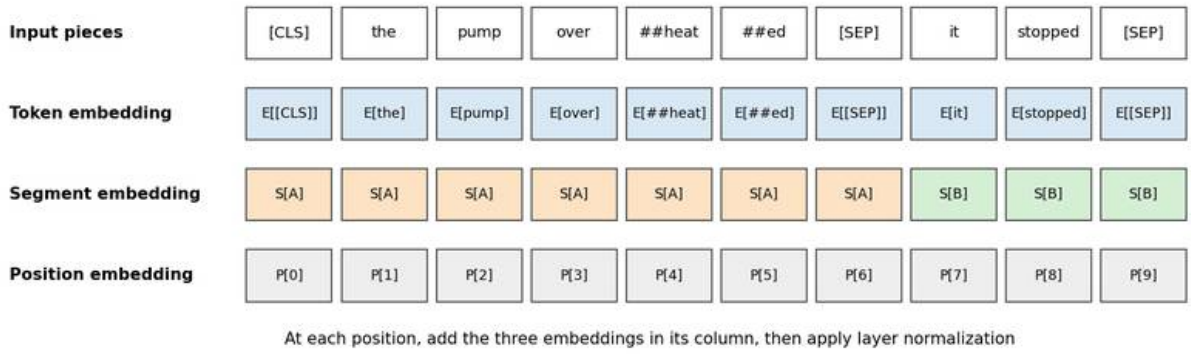


Figure 2 — BERT input for a text pair. Each column is one position; its token, segment (A or B) and position embeddings are added. The position table is learned, so it fixes a maximum input length.

Unlike the fixed sinusoids of the original Transformer, BERT's position vectors are learned rows [2]. A learned position table imposes a maximum supported position; a longer input is an invalid lookup, not an ordinary longer sequence, and must be truncated or split into windows. Embedding rows also have meaning only relative to the model's token IDs. Swapping in a different tokenizer without adapting weights can turn apparently valid numbers into unrelated symbols.

4. The encoder

Plain version. BERT is the encoder half of a Transformer: a stack of identical blocks, each letting every position read every other position and then transforming each position separately.

BERT is a stack of Transformer **encoder** blocks [1, 3]. Each block mixes information with bidirectional self-attention, then transforms each position with a feedforward network; residual connections and layer normalization support the stack [9]. "Bidirectional" means both left and right input positions are available throughout the encoder. It does not mean training two separate causal models and joining their outputs, as ELMo does [5]. NLP 07 derives multi-head attention; the only BERT-specific choice is the attention mask of Figure 1.

The paper reports two sizes. Writing L for the number of blocks, H for the hidden width and A for attention heads, BERT-Base has $L = 12$, $H = 768$, $A = 12$ and 110M parameters, and BERT-Large has $L = 24$, $H = 1024$, $A = 16$ and 340M parameters [1]. BERT-Base was chosen to match the size of the original GPT for comparison [1]. The feedforward layers use the GELU activation [8, 1]. Pretraining used BooksCorpus and English Wikipedia, and the paper stresses using a document-level corpus so that long contiguous sequences are available [1].

Think of the outputs as a table with one row per input token, rather than a single sentence vector. A token-labeling head reads those rows individually. A sequence head reads the first row through a pooling transformation. The same encoder can support both because its output preserves token positions; NLP 11 builds those heads.

5. What BERT learns before task labels are available

Plain version. Masked language modeling turns raw text into a fill-in-the-blank exercise. Next sentence prediction adds a yes/no question about whether two text segments belong together.

Masked language modeling

Masked language modeling (MLM) learns from ordinary text: the original tokens provide their own targets. For example, `the crane [MASK] steel` asks the model to predict the removed token using words on both sides. The prediction loss updates the encoder as well as the vocabulary head, so the final token vectors become useful inputs for later supervised tasks.

Original BERT selects 15% of WordPiece positions as prediction targets. Among selected positions, it inserts `[MASK]` 80% of the time, a random token 10%, and leaves the original token unchanged 10% [1]. Selection and replacement are different: **every selected position contributes a target**, even when its visible token is unchanged. Other positions supply context without contributing MLM loss. This mixed replacement reduces the difference between pretraining inputs and ordinary unmasked task inputs, where `[MASK]` never appears.

The price of this design is sample efficiency: only the selected positions produce a loss. ELECTRA's authors make exactly this point, noting that MLM learns from only the masked subset of tokens per example [14]. The original implementation also chose masks once when the data was prepared; RoBERTa compared that static masking with **dynamic masking**, which draws a new pattern every time a sequence is used, and found dynamic masking comparable or slightly better [11]. The tiny model in Section 7 masks dynamically.

Next sentence prediction and its critique

Original BERT also trains **next sentence prediction** (NSP) [1]: a text pair is either consecutive corpus text or a pair whose second segment is sampled from the corpus, with equal sampling probability. The pooled `[CLS]` state predicts this binary label. NSP predicts a relationship label; it does not generate the next sentence. The original pretraining objective combines MLM and NSP, and the paper's ablation reported that removing NSP hurt performance on QNLI, MNLI and SQuAD 1.1 [1].

Later work disagreed. RoBERTa's replication study found that removing the NSP loss **matches or slightly improves** downstream performance when inputs are filled with contiguous text, and suggested that the original ablation may have removed the loss while keeping the segment-pair input format [11]. ALBERT's authors conjectured that NSP is too easy because it conflates topic prediction with coherence prediction: a randomly sampled second segment usually differs in topic. They replaced it with **sentence-order prediction**, which uses two consecutive segments from the same document either in order or swapped, so topic no longer gives the answer away [12]. The general lesson is to inspect a checkpoint's actual objectives rather than assume every later encoder keeps every original design choice.

6. A runnable bidirectional encoder

Plain version. Before pretraining a whole model, isolate the two properties that make it BERT-like: changing a later word changes an earlier word's representation, and padding changes nothing.

The following CPU example trains MLM on four invented sentences. Position 2 is always selected and replaced with `[MASK]`, making the visible right-hand verb useful for distinguishing `robot` from `sensor`. This deliberately simpler corruption policy isolates the mechanism; the model is randomly initialized, has one small encoder block, and uses a linear vocabulary head rather than reproducing original BERT's full head. It uses layer normalization, GELU and AdamW [8–10].

```
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(23)
torch.set_num_threads(1)
vocab = ['[PAD]', '[CLS]', '[SEP]', '[MASK]', 'the', 'a',
         'robot', 'sensor', 'moves', 'measures']
index = {word: i for i, word in enumerate(vocab)}
rows = ['the robot moves', 'the sensor measures',
        'a robot moves', 'a sensor measures']
ids = torch.tensor([[index[w] for w in ['[CLS]'] + s.split()
                     + ['[SEP]', '[PAD]']] for s in rows])
blocked = ids.eq(index['[PAD]'])
corrupted = ids.clone()
corrupted[:, 2] = index['[MASK]']
labels = torch.full_like(ids, -100)
labels[:, 2] = ids[:, 2]

class TinyEncoder(nn.Module):
    def __init__(self):
        super().__init__()
        self.word = nn.Embedding(len(vocab), 16)
        self.position = nn.Embedding(6, 16)
        self.segment = nn.Embedding(2, 16)
        self.embed_norm = nn.LayerNorm(16)
        self.layer = nn.TransformerEncoderLayer(
            16, 2, 32, dropout=0.0, activation='gelu', batch_first=True)
        self.mlm = nn.Linear(16, len(vocab))
        self.pool = nn.Sequential(nn.Linear(16, 16), nn.Tanh())
        self.classifier = nn.Linear(16, 3)
        self.tagger = nn.Linear(16, 3)
        self.span = nn.Linear(16, 2)
    def forward(self, tokens, padding):
        positions = torch.arange(tokens.shape[1])
        segments = torch.zeros_like(tokens)  # single segment here
        h = self.embed_norm(self.word(tokens) + self.position(positions)
                            + self.segment(segments))
        return self.layer(h, src_key_padding_mask=padding)

model = TinyEncoder()
```

```
optimizer = torch.optim.AdamW(model.parameters(), lr=0.01)
def objective():
    logits = model.mlm(model(corrupted, blocked))
    return F.cross_entropy(logits.reshape(-1, len(vocab)), labels.reshape(-1))
initial = objective().item()
for _ in range(100):
    optimizer.zero_grad()
    loss = objective()
    loss.backward()
    optimizer.step()
model.eval()
with torch.no_grad():
    final = objective().item()
    h = model(ids, blocked)
    changed = ids.clone()
    changed[0, 3] = index['measures']
    context_gap = (model(changed, blocked)[0, 2] - h[0, 2]).abs().max().item()
    changed_padding = ids.clone()
    changed_padding[blocked] = index['robot']
    pad_gap = (model(changed_padding, blocked)[~blocked] - h[~blocked]).abs().max().item()
    shapes = [model.classifier(model.pool(h[:, 0])).shape,
              model.tagger(h).shape, model.span(h).shape]
assert final < initial and context_gap > 1e-4 and pad_gap < 1e-6
print('masked loss:', round(initial, 3), '->', round(final, 3))
print('context changes representation:', context_gap > 1e-4)
print('padding leaves real tokens unchanged:', pad_gap < 1e-6)
print('class / tag / span shapes:', [tuple(s) for s in shapes])
```

Output:

```
masked loss: 2.223 -> 0.001
context changes representation: True
padding leaves real tokens unchanged: True
class / tag / span shapes: [(4, 3), (4, 6, 3), (4, 6, 2)]
```

The masked loss falls from `2.223` to `0.001`. This measures fitting those four examples. Changing the word after `robot` changes its final representation, whereas changing the value stored at blocked padding positions leaves real-token outputs unchanged. A causal model must reject the first kind of future influence (the perturbation test of NLP 09); a bidirectional encoder deliberately allows it.

The task heads print shapes `(4, 3)`, `(4, 6, 3)` and `(4, 6, 2)`: four sequence decisions over three categories, six token decisions per example, and two boundary scores per token. **Those heads are untrained.** Only the encoder and MLM head receive gradients in this example. To build an application, supply its labels, optimize the corresponding head and encoder, and evaluate held-out task outputs, as NLP 11 does.

There are two masks to keep separate. The encoder's padding mask controls readable keys. The `-100` labels tell cross-entropy which positions to ignore; they do not remove those positions from

attention. In the demonstration, all real words may provide context, but only the selected noun contributes MLM loss.

7. Pretraining a tiny BERT from scratch

Plain version. Now apply the real corruption recipe to a small artificial language, train for a few seconds, and check three things: how well the model fills in held-out blanks, whether one word gets different vectors in different contexts, and what simple classifiers can read from each layer.

The toy corpus comes from a small grammar. The subject constrains the verb and the verb constrains the object, so a masked word is often predictable from **both** sides. The word `crane` is deliberately ambiguous: it can `lift` (a machine) or `eat` (a bird). Of the 128 possible sentences, 40 are held out. The corruption function implements Section 5 exactly: 15% selection, then 80/10/10 replacement, with a fresh draw on every step.

```
import itertools, random
from collections import Counter
import numpy as np
import torch
from torch import nn
from torch.nn import functional as F
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score

random.seed(0); torch.manual_seed(0); torch.set_num_threads(1)

# 1. A toy corpus: the subject constrains the verb, the verb constrains the object.
frames = {'robot': ['lifts', 'moves'], 'crane': ['lifts', 'eats'],
          'technician': ['repairs', 'moves'], 'heron': ['eats', 'sees']}
objects = {'lifts': ['steel', 'box'], 'moves': ['box', 'cart'],
           'eats': ['fish', 'bread'], 'repairs': ['pump', 'cart'], 'sees': ['fish', 'boat']}
sentences = [f'{d1} {a}{s} {v} {d2} {o}'.split()
             for s in frames for v in frames[s] for o in objects[v]
             for d1, d2, a in itertools.product(['the', 'a'], ['the', 'a'], ['', 'old '])]
random.shuffle(sentences)
test, train = sentences[:40], sentences[40:]
special = ['[PAD]', '[CLS]', '[SEP]', '[MASK]']
words = sorted({w for s in sentences for w in s})
vocab = special + words
index = {w: i for i, w in enumerate(vocab)}
first_word = len(special)
L = 2 + max(len(s) for s in sentences)

def encode(rows):
    ids = torch.zeros(len(rows), L, dtype=torch.long)
    for r, s in enumerate(rows):
        seq = [index['[CLS]']] + [index[w] for w in s] + [index['[SEP]']]
        ids[r, :len(seq)] = torch.tensor(seq)
    return ids
train_ids, test_ids = encode(train), encode(test)
```

```
# 2. BERT-style corruption: select 15% of real word positions; of those,
#    80% -> [MASK], 10% -> random word, 10% -> unchanged. All selected are targets.
def corrupt(ids, generator):
    real = ids >= first_word
    selected = (torch.rand(ids.shape, generator=generator) < 0.15) & real
    labels = torch.where(selected, ids, torch.full_like(ids, -100))
    u = torch.rand(ids.shape, generator=generator)
    out = ids.clone()
    out[selected & (u < 0.8)] = index['[MASK]']
    swap = selected & (u >= 0.8) & (u < 0.9)
    out[swap] = torch.randint(first_word, len(vocab), (int(swap.sum()),),
     ↪  generator=generator)
    return out, labels

class TinyBERT(nn.Module):
    def __init__(self, d=32, layers=2):
        super().__init__()
        self.word, self.position = nn.Embedding(len(vocab), d), nn.Embedding(L, d)
        self.segment, self.norm = nn.Embedding(2, d), nn.LayerNorm(d)
        self.blocks = nn.ModuleList(nn.TransformerEncoderLayer(
            d, 4, 4 * d, dropout=0.0, activation='gelu', batch_first=True)
            for _ in range(layers))
        self.mlm = nn.Linear(d, len(vocab))
    def forward(self, ids):
        pad = ids.eq(index['[PAD]'])
        h = self.norm(self.word(ids) + self.position(torch.arange(ids.shape[1]))
                      + self.segment(torch.zeros_like(ids)))
        states = [h]
        for block in self.blocks:
            h = block(h, src_key_padding_mask=pad)
            states.append(h)
        return states                                  # embeddings, layer 1, layer 2

model = TinyBERT()
optimizer = torch.optim.AdamW(model.parameters(), lr=3e-3)
g = torch.Generator().manual_seed(1)
for step in range(600):                                # a fresh mask every step
    inputs, labels = corrupt(train_ids, g)
    logits = model.mlm(model(inputs)[-1])
    loss = F.cross_entropy(logits.reshape(-1, len(vocab)), labels.reshape(-1))
    optimizer.zero_grad(); loss.backward(); optimizer.step()
model.eval()

# 3. Held-out masked-word accuracy: mask each word position in turn.
def masked_hits(ids, rows_text):
    by_kind = {}
    with torch.no_grad():
        for r, s in enumerate(rows_text):
            for i, w in enumerate(s):
                x = ids[r:r + 1].clone(); x[0, i + 1] = index['[MASK]']
                guess = vocab[model.mlm(model(x)[-1][0, i + 1]).argmax()]
                kind = 'determiner/adjective' if w in ('the', 'a', 'old') else 'noun/verb'
                by_kind.setdefault(kind, []).append(guess == w)
```

```
    return {k: round(float(np.mean(v)), 3) for k, v in sorted(by_kind.items())}
counts = torch.bincount(train_ids[train_ids >= first_word], minlength=len(vocab))
real_test = test_ids[test_ids >= first_word]
print('final training MLM loss:', round(loss.item(), 3))
print('held-out masked accuracy:', masked_hits(test_ids, test))
print('most-frequent-word baseline:', vocab[int(counts.argmax())],
      round((real_test == counts.argmax()).float().mean().item(), 3))
# Upper reference: the best guess given every other word, read off the grammar.
def oracle(text):
    by_kind = {}
    for s in text:
        for i, w in enumerate(s):
            fits = Counter(t[i] for t in sentences if len(t) == len(s)
                           and all(a == b for j, (a, b) in enumerate(zip(t, s)) if j != i))
            kind = 'determiner/adjective' if w in ('the', 'a', 'old') else 'noun/verb'
            by_kind.setdefault(kind, []).append(fits[w] / sum(fits.values()))
    return {k: round(float(np.mean(v)), 3) for k, v in sorted(by_kind.items())}
print('expected accuracy of an ideal guesser:', oracle(test))

# 4. One word, two contexts: identical input row, different output rows.
with torch.no_grad():
    pair = encode([['the', 'crane', 'lifts', 'the', 'steel'],
                   ['the', 'crane', 'eats', 'the', 'fish']])
    cos = [F.cosine_similarity(s[0, 2], s[1, 2], dim=0).item() for s in model(pair)]
print('cosine of the two "crane" vectors by layer:', [round(c, 3) for c in cos])

# 5. Probes. (a) Part of speech vs a control task that gives each word type a
#    random label; (b) the sense of "crane" (machine vs bird) from its own vector.
pos = {w: 'DET' if w in ('the', 'a') else 'ADJ' if w == 'old' else
       'VERB' if w in objects else 'NOUN' for w in words}
control = {w: random.randint(0, 3) for w in words}
crane = [s for s in sentences if 'crane' in s]
crane_ids = encode(crane)
sense = np.array([int('eats' in s) for s in crane])
def rows(ids, text, layer):
    with torch.no_grad():
        h = model(ids)[layer]
    return np.array([h[r, i + 1].numpy() for r, s in enumerate(text) for i in range(len(s))])
for layer in range(3):
    Xtr, Xte = rows(train_ids, train, layer), rows(test_ids, test, layer)
    acc = []
    for table in (pos, control):
        ytr = [table[w] for s in train for w in s]; yte = [table[w] for s in test for w in s]
        acc.append(LogisticRegression(max_iter=3000).fit(Xtr, ytr).score(Xte, yte))
    h = model(crane_ids)[layer].detach()[:, :, :]
    Xc = np.array([h[r, s.index('crane') + 1].numpy() for r, s in enumerate(crane)])
    acc.append(cross_val_score(LogisticRegression(max_iter=3000), Xc, sense, cv=4).mean())
    print(f'layer {layer}: POS {acc[0]:.2f}  control {acc[1]:.2f}  crane sense {acc[2]:.2f}')
assert cos[0] > 0.999 and cos[-1] < cos[0]
```

Output (about six seconds on one CPU thread):

```
final training MLM loss: 0.453
```

```
held-out masked accuracy: {'determiner/adjective': 0.418, 'noun/verb': 0.492}
most-frequent-word baseline: the 0.17
expected accuracy of an ideal guesser: {'determiner/adjective': 0.592, 'noun/verb': 0.633}
cosine of the two "crane" vectors by layer: [1.0, 0.819, 0.639]
layer 0: POS 1.00  control 1.00  crane sense 0.38
layer 1: POS 1.00  control 1.00  crane sense 1.00
layer 2: POS 1.00  control 1.00  crane sense 1.00
```

Read the masked-word numbers against two references. Always guessing the most frequent word, `the`, is right 17% of the time. An **ideal guesser** that knows the grammar exactly, and therefore the true distribution of each blank given every other word, would be right 63% of the time on content words, not 100%: after `the robot ___ the box`, both `lifts` and `moves` are grammatical. The model reaches 49% on content words from 88 training sentences. That gap is the honest measure of what it has and has not learned. Many blanks have no single answer, which is why MLM accuracy is never a direct measure of understanding.

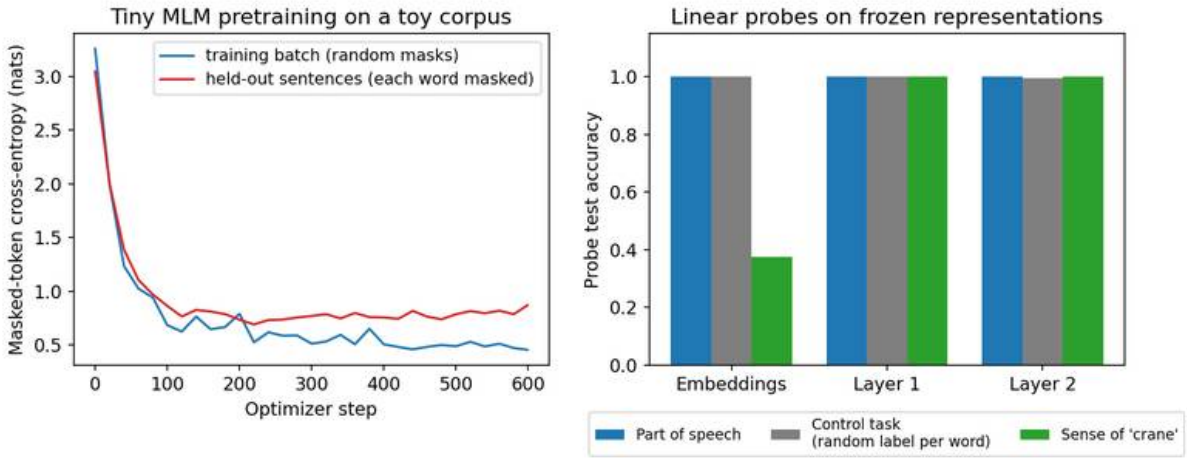


Figure 3 — The tiny MLM run. Left: training loss on randomly masked batches and held-out loss with each word masked in turn; the gap is overfitting to 88 sentences. Right: linear probes on frozen representations from each layer (Section 9 explains why the control task matters).

The left panel of Figure 3 shows the held-out loss flattening, then drifting up, while the training loss keeps falling. The same script trained for more steps lowers training loss further but makes held-out masked accuracy worse, so the 600-step budget was set by watching held-out loss, not training loss. That is ordinary model selection, and it applies to pretraining as much as to fine-tuning.

8. Contextual versus static embeddings

Plain version. A static embedding gives a word one vector everywhere. A contextual encoder gives each occurrence its own vector, and the difference grows with depth.

A Word2Vec or PPMI vector, as built in NLP 04, is a single row of a table: the bird and the machine share one `crane` vector, which must average their contexts. The printed cosines above show the contextual alternative in miniature. At the embedding layer, the two `crane` vectors are identical (cosine 1.0) because the token and its position are the same. After one encoder block the cosine is 0.819, and after two it is 0.639: the right-hand context `lifts the steel` or `eats the fish` has been mixed in.

Ethayarajh measured this effect in real models. In ELMo, BERT and GPT-2, representations of the same word in different contexts are still more similar than those of two different words, but this self-similarity is much lower in upper layers, and on average less than 5% of the variance in a word's contextual representations can be explained by a static embedding for that word [19]. The same study found that contextual representations are not isotropic in any layer: vectors occupy a narrow region of the space [19]. A practical consequence is that raw cosine similarity between BERT vectors is not automatically a meaningful semantic similarity. Sentence-level similarity needs its own training objective, which NLP 04 and NLP 11 discuss.

Static vectors still have uses: they are cheap, need no encoder at inference time and are easy to inspect. Contextual vectors are preferable when the same surface form carries different meanings or roles, which is most of the time in real text.

9. Probing what the representations contain

Plain version. A probe is a small classifier trained on frozen representations to predict a linguistic property. If it succeeds, the property is recoverable from the vectors. Whether it was the representation or the probe that did the work is a separate question.

Tenney, Das and Pavlick probed BERT layer by layer and found that the regions responsible for part-of-speech tagging, parsing, named entities, semantic roles and coreference appear in roughly the order of the classical NLP pipeline, with later layers sometimes revising earlier decisions [20]. Hewitt and Manning's **structural probe** looks for a linear transformation under which squared distances between word vectors encode distances in the parse tree, and found such transformations for ELMo and BERT but not for baselines [21]. Clark and colleagues analyzed attention instead of vectors and found heads that attend to delimiter tokens, to fixed positional offsets, and to direct objects, determiners and coreferent mentions [23].

Hewitt and Liang added the essential control. A **control task** assigns each word type a random label, so it can be solved only by memorizing word identities. A good probe should be **selective**: high accuracy on the linguistic task and low accuracy on its control [22]. Without that comparison, a high probe accuracy may show that the probe is powerful, not that the representation encodes the property.

Our tiny run illustrates both sides. Part of speech is perfectly recoverable at every layer, but so is a random label per word, because in this grammar part of speech is a function of the word itself. The probe's success therefore says nothing beyond "the word is identifiable," and its selectivity is zero. The `crane` sense probe is different. It is at chance or below (0.38, from 32 sentences in four folds) at the embedding layer, where both senses share one vector, and perfect after one encoder

block. That property requires context, so the layer-wise rise is evidence that the encoder moved the information into the `crane` vector. Even so, a probe shows that information is **available** to a linear reader, not that the model **uses** it for its predictions.

10. The BERT family

Plain version. After BERT, several papers changed one part of the recipe at a time: how long to train, how to share parameters, how to compress, and what to predict.

Model	What changed	Reported motivation
RoBERTa [11]	Longer training with bigger batches on more data, NSP removed, longer sequences, dynamic masking	A replication study finding BERT significantly undertrained
ALBERT [12]	Factorized embedding parameterization, cross-layer parameter sharing, sentence-order prediction	Lower memory and faster training; a coherence objective instead of NSP
DistilBERT [13]	Knowledge distillation during pretraining into a smaller student, with a triple loss	40% smaller, retaining 97% of BERT's language understanding and 60% faster, as reported by its authors
ELECTRA [14]	Replaced-token detection: a small generator fills masked positions, and the encoder classifies every token as original or replaced	Learning from all input tokens instead of only the masked subset

ALBERT's factorized embedding separates vocabulary size from hidden size: instead of one $|V|\times H$ table it uses a $|V| \times E$ table followed by an $E \times H$ projection, with E much smaller than H [12]. Its authors argue that WordPiece embeddings are meant to be context-independent while hidden states are context-dependent, so the two sizes need not match [12]. ELECTRA's discriminator is the model kept for fine-tuning; the generator is discarded after pretraining [14].

These variants change parameter counts, compute budgets and objectives together, so their published comparisons belong to their own experimental settings. When choosing one, check its tokenizer, maximum length, objectives and training data, and measure on the task you care about.

11. Multilingual and Chinese BERT

Plain version. One encoder can be pretrained on many languages at once with a shared vocabulary. Chinese needs a decision about units, because it is not written with spaces between words.

The original release includes a multilingual BERT-Base trained on the Wikipedias of 104 languages with a shared 110k WordPiece vocabulary and no marker telling the model which language it is reading [2]. Pires, Schlinger and Garrette found that this model transfers surprisingly well zero-shot, where a task is fine-tuned in one language and evaluated in another, including across scripts, and that transfer works best between typologically similar languages; they also found systematic

deficiencies for certain language pairs [15]. XLM-R later trained a Transformer masked language model on one hundred languages with far more filtered CommonCrawl data and reported large gains over multilingual BERT on cross-lingual benchmarks, especially for low-resource languages [16]. Its analysis of the trade-off between positive transfer and capacity dilution is a reminder that adding languages to a fixed-size model is not free [16].

For Chinese, the original multilingual model adds spaces around every character in the CJK ideograph ranges before WordPiece, so Chinese is effectively character-tokenized [2]; the `basic_split` function in Section 3 imitates this rule. The release also provides a Chinese-only BERT-Base that covers both Simplified and Traditional Chinese [2]. Character positions avoid word-segmentation errors, but masking a single character of a multi-character word such as 幫浦 lets the model recover it from its partner character rather than from sentence context. Cui and colleagues introduced **whole word masking** for Chinese BERT: a word segmenter identifies word boundaries, and all characters of a selected word are masked together, while the input is still tokenized by characters [17]. Their MacBERT model replaces `[MASK]` with similar words chosen by word-vector similarity, reducing the gap between pretraining and fine-tuning inputs [17].

For Traditional Chinese text from Taiwan, check what the pretraining corpus actually contained, not only the language label. A model whose data is dominated by Simplified Chinese can tokenize Traditional characters correctly and still be weaker on regional vocabulary such as 資料 versus other usages, or on Taiwanese terminology in a domain. That is a data question to measure with in-domain examples, not something the architecture decides.

Modern view

Rogers, Kovaleva and Rumshisky's primer [24] surveys over 150 studies of BERT: what linguistic and world knowledge it captures and where, how its attention heads and layers behave, modifications to its objectives and architecture, overparameterization and compression. Its overall message matches Section 9: many properties are recoverable, but recoverable is not the same as used, and probing results depend on the probe.

Encoders have not disappeared in the era of large decoder-only models. **ModernBERT [25]**, published at ACL 2025, is a 2024 encoder-only model that brings recent architectural and efficiency optimizations to the BERT design; its authors describe encoders as the workhorse of many production retrieval and classification pipelines and report training on 2 trillion tokens with a native sequence length of 8,192. **Zhao and colleagues' survey [26]** places masked encoders within the broader history of pretrained language models and notes that causal decoders have been widely adopted as the architecture of existing large language models; NLP 09 shows why one causal objective can serve both scoring and generation.

A practical 2026 summary: use a bidirectional encoder when the whole input is available and the output is a label, a span or a vector for retrieval, and when latency and cost matter; use a decoder-only model when the output is open-ended text. Open problems include faithful interpretation beyond probing, robust multilingual coverage for low-resource languages and scripts, long-document encoding beyond learned position limits, and evaluating how much a pretrained representation, rather than the fine-tuning data, explains a task result.

Key takeaways

- BERT is a Transformer encoder pretrained without labels and then fine-tuned with a small task head; pretraining supplies a representation, not task labels.
- Input is WordPiece pieces packed as `[CLS] A [SEP] B [SEP]`, with summed token, segment and learned position embeddings; segments do not block attention.
- MLM selects 15% of positions and replaces them 80/10/10; every selected position is a target, and masked scores are not causal perplexities.
- NSP was later found unnecessary (RoBERTa) or replaced by sentence-order prediction (ALBERT).
- A tiny BERT pretrained from scratch shows the mechanisms: held-out blanks are filled better than a frequency baseline but below the grammar's own ceiling, and an ambiguous word's vectors diverge with depth.
- Contextual vectors differ by occurrence; static vectors do not. Raw cosine between contextual vectors is not automatically a semantic similarity.
- Probes need control tasks; a probe shows information is available, not that the model uses it.
- Multilingual and Chinese BERT models make specific unit choices (shared WordPiece, per-character CJK splitting, whole word masking) that affect what is learned.

Exercises

1. In the tiny BERT, replace dynamic masking with a single static mask drawn once before training. Compare held-out masked accuracy and the loss curves.

   Hint

   Draw `corrupt(train_ids, g)` once outside the loop. With a static mask, each sentence always hides the same positions, so many (sentence, position) targets are never seen.

2. Add next sentence prediction to the toy corpus: build pairs of two sentences joined by `[SEP]` with segment IDs, where half the second sentences are random. Then build a sentence-order version. Which one can be solved from topic cues alone in your corpus?

   Hint

   Make two "documents" that use different vocabularies. A random second sentence then usually comes from the other document, so NSP is easy; swapped order within one document is not.

3. Extend the WordPiece vocabulary so that `pumpkin` is not `[UNK]`, without adding `pumpkin` itself. Then find a word for which greedy longest-match gives a different split from the one with the fewest pieces.

   Hint

   Adding `##kin` is enough for the first part. For the second, include a long prefix that leaves an awkward remainder, and a shorter prefix that leads to a clean split.

4. Design a probing task on the toy corpus for which part-of-speech style control results would show **positive** selectivity. What property must depend on context rather than on word identity?

Hint

Subject versus object role of a noun is one option, but check first whether position alone already reveals it in this grammar. If it does, add sentence variants that break the link.

5. Replace the MLM head with an ELECTRA-style replaced-token detector: corrupt positions with random words instead of `[MASK]`, and train a binary classifier on every position. Compare how many positions produce a loss per batch.

Hint

ELECTRA uses a small trained generator; random replacements make detection much easier, so say what your simplification changes. Count labeled positions with and without masking.

6. Turn the bidirectional example into a token classifier. If one annotated word splits into three pieces, define which positions contribute to loss and how you reconstruct the word label.

Hint

Decide between first-piece supervision and an explicitly defined all-piece label mapping. Ignore padding and special markers, train the tag head, and map the output back to the original text.

References

1. Jacob Devlin, Ming-Wei Chang, Kenton Lee and Kristina Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," NAACL, 2019. Paper
2. Google Research, *BERT: original implementation and released models*, software, 2018. Encoder and pooler ·Tokenizer ·Multilingual and Chinese models
3. Ashish Vaswani et al., "Attention Is All You Need," NeurIPS, 2017. Paper
4. Alec Radford, Karthik Narasimhan, Tim Salimans and Ilya Sutskever, "Improving Language Understanding by Generative Pre-Training," OpenAI technical report, 2018. Paper
5. Matthew E. Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee and Luke Zettlemoyer, "Deep Contextualized Word Representations," NAACL, 2018. Paper
6. Tomas Mikolov, Kai Chen, Greg Corrado and Jeffrey Dean, "Efficient Estimation of Word Representations in Vector Space," ICLR Workshop, 2013. Paper
7. Yonghui Wu et al., "Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation," arXiv:1609.08144, 2016. Paper
8. Dan Hendrycks and Kevin Gimpel, "Gaussian Error Linear Units (GELUs)," arXiv:1606.08415, 2016, revised 2023. Paper

9. Jimmy Lei Ba, Jamie Ryan Kiros and Geoffrey E. Hinton, “Layer Normalization,” arXiv:1607.06450, 2016. Paper
10. Ilya Loshchilov and Frank Hutter, “Decoupled Weight Decay Regularization,” ICLR, 2019. Paper
11. Yinhan Liu et al., “RoBERTa: A Robustly Optimized BERT Pretraining Approach,” arXiv:1907.11692, 2019. Paper
12. Zhenzhong Lan, Mingda Chen, Sebastian Goodman, Kevin Gimpel, Piyush Sharma and Radu Soricut, “ALBERT: A Lite BERT for Self-supervised Learning of Language Representations,” ICLR, 2020. Paper
13. Victor Sanh, Lysandre Debut, Julien Chaumond and Thomas Wolf, “DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter,” 5th Workshop on Energy Efficient Machine Learning and Cognitive Computing at NeurIPS, 2019. Paper
14. Kevin Clark, Minh-Thang Luong, Quoc V. Le and Christopher D. Manning, “ELECTRA: Pre-training Text Encoders as Discriminators Rather Than Generators,” ICLR, 2020. Paper
15. Telmo Pires, Eva Schlinger and Dan Garrette, “How Multilingual is Multilingual BERT?,” ACL, 2019. Paper
16. Alexis Conneau et al., “Unsupervised Cross-lingual Representation Learning at Scale,” ACL, 2020. Paper
17. Yiming Cui, Wanxiang Che, Ting Liu, Bing Qin and Ziqing Yang, “Pre-Training With Whole Word Masking for Chinese BERT,” IEEE/ACM Transactions on Audio, Speech, and Language Processing 29, 2021. Publisher ·Author manuscript
18. Masato Hagiwara, *Real-World Natural Language Processing: Practical Applications with Deep Learning*, Chapter 9, Manning, 2021. Publisher
19. Kawin Ethayarajh, “How Contextual are Contextualized Word Representations? Comparing the Geometry of BERT, ELMo, and GPT-2 Embeddings,” EMNLP-IJCNLP, 2019. Paper
20. Ian Tenney, Dipanjan Das and Ellie Pavlick, “BERT Rediscovers the Classical NLP Pipeline,” ACL, 2019. Paper
21. John Hewitt and Christopher D. Manning, “A Structural Probe for Finding Syntax in Word Representations,” NAACL, 2019. Paper
22. John Hewitt and Percy Liang, “Designing and Interpreting Probes with Control Tasks,” EMNLP-IJCNLP, 2019. Paper
23. Kevin Clark, Urvashi Khandelwal, Omer Levy and Christopher D. Manning, “What Does BERT Look at? An Analysis of BERT’s Attention,” BlackboxNLP Workshop, 2019. Paper
24. Anna Rogers, Olga Kovaleva and Anna Rumshisky, “A Primer in BERTology: What We Know About How BERT Works,” Transactions of the Association for Computational Linguistics 8, 2020. Paper
25. Benjamin Warner et al., “Smarter, Better, Faster, Longer: A Modern Bidirectional Encoder for Fast, Memory Efficient, and Long Context Finetuning and Inference,” ACL, 2025. Paper
26. Wayne Xin Zhao et al., “A Survey of Large Language Models,” Frontiers of Computer Science, 2026. Updated survey
27. Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li and Peter J. Liu, “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer,” JMLR 21(140), 2020. Paper

NLP 11 • Classification with BERT (Sentiment Analysis, Natural Language Inference)

Fine-tune a pretrained bidirectional encoder for single-text and text-pair decisions, read span answers from passage tokens, and evaluate labels, probabilities and shortcuts honestly.

Online edition: https://blog.shyandram.dev/en/learn/nlp/nlp-11-classification-bert/

What you'll learn

- Choose the output head that matches the decision: one label, one label per token, or a span inside a passage.
- Fine-tune a pretrained encoder for sentiment analysis and for premise–hypothesis inference, and know what the standard datasets (SST, IMDB, SNLI, MultiNLI, SQuAD) actually contain.
- Recognize annotation artifacts, hypothesis-only baselines and challenge sets, and why a high in-distribution score can hide a shortcut.
- Set learning rate, epochs, warmup, class weights and decision thresholds deliberately, and know why fine-tuning can be unstable.
- Evaluate with accuracy, macro-F1, calibration and coverage, and separate relevant evidence from sufficient evidence.
- Run a tiny encoder that is pretrained from scratch with masked language modeling, then fine-tuned for synthetic sentiment and inference tasks on a CPU.

The big picture

NLP 10 built BERT: a bidirectional encoder pretrained by predicting masked tokens. Pretraining gives a representation function, not an application. A support team still has to say what "urgent" means, and an inference dataset still has to say when one sentence follows from another. This tutorial is about that second step: attaching a small output layer to the encoder, training both on labeled examples, and checking whether the result solves the intended task rather than a convenient imitation of it.

> **Explain like I'm five**
>
> Imagine a friend who has read thousands of books but has never sorted movie reviews. You show them twenty reviews marked "liked it" or "didn't like it." Because they already understand words like "dull" and "wonderful," they learn the sorting job quickly. But if every "didn't like it" example you showed happened to contain the word "not," your friend might simply look for "not." Fine-tuning is the short lesson; good evaluation checks what the friend actually learned.

Two families of decisions dominate this chapter. **Single-text classification** assigns one label to one input: positive or negative sentiment, a topic, a ticket route. **Text-pair classification** labels the relationship between two inputs: does the hypothesis follow from the premise, contradict it, or neither? Extractive question answering sits next to them: instead of one label, the model chooses where an answer starts and ends inside a passage. All three reuse the same pretrained encoder; only the head and the training labels change [3].

Hagiwara's Chapter 9 develops transfer learning with pretrained encoders for exactly these applications [1], and McRoy's chapters on benchmark tasks and on information extraction and argumentation supply the linguistic side [2]. Throughout, keep one question in view: *what evidence would convince a skeptic that the model learned the task, not the dataset?*

BERT applications: choose the output head

Plain version. A task head turns the encoder's token vectors into the output you need. Start by deciding whether the answer is one class, one label per token, or a span inside a passage.

Application	Input	Supervised output
Sentiment, topic or support-ticket classification	One text	One category from the pooled `[CLS]` state
Entailment, paraphrase or query–document relevance	Two texts packed together	A relationship label or relevance score
Entity recognition or other token labeling	One tokenized text	A label at each supervised token position
Extractive question answering	Question and passage	Start/end positions in the passage
Semantic retrieval with Sentence-BERT	Texts encoded separately	Comparable vectors learned with a sentence-level objective

Sequence and token tasks use the representations described in the BERT paper [3]; the original classification and span implementations make their output shapes explicit [4]. Sentence-BERT adds training for sentence comparisons [20]. The following formulas explain what those choices compute.

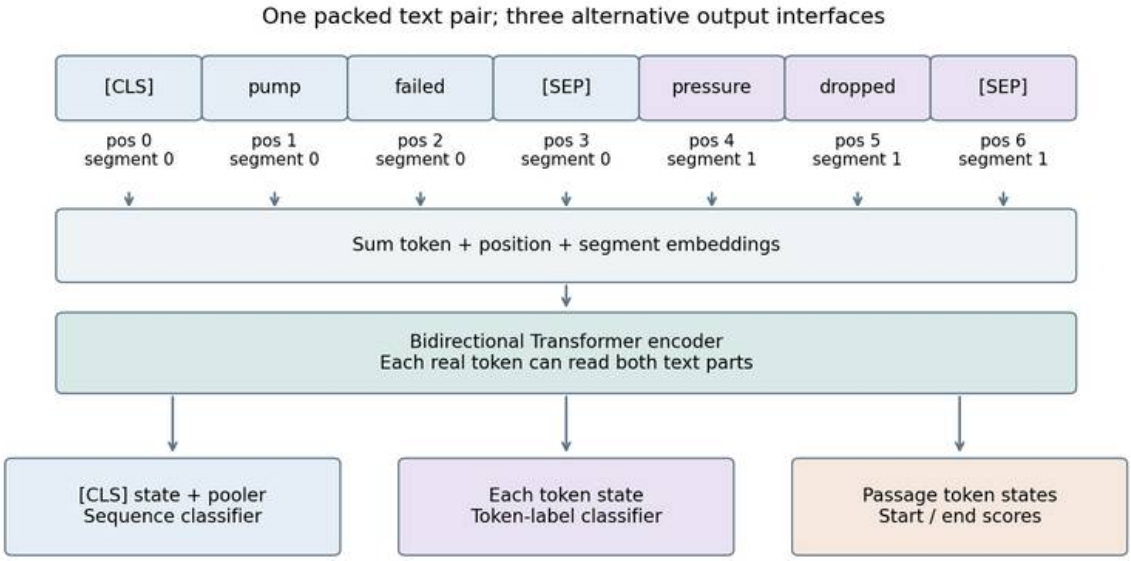


Figure 1 — A shared BERT encoder supports different supervised outputs. Segment A/B labels identify text parts; they do not prevent cross-segment attention. The three heads illustrate alternative tasks, not predictions from a trained model.

Sequence classification and text-pair decisions

For a classifier with K classes, one common BERT interface is:

$$\mathbf{c} = \tanh(W_p \mathbf{h}_{\text{CLS}} + \mathbf{b}_p), \qquad p(y = k \mid \mathbf{x}) = \text{softmax}(W_c \mathbf{c} + \mathbf{b}_c)_k.$$

Here $\mathbf{x}$ is the packed token sequence; $\mathbf{h}_{\text{CLS}} \in \mathbb{R}^d$ is the first contextual row; $W_p \in \mathbb{R}^{d \times d}$ and $\mathbf{b}_p \in \mathbb{R}^d$ form the pooler; $\mathbf{c} \in \mathbb{R}^d$ is its output; $W_c \in \mathbb{R}^{K \times d}$ and $\mathbf{b}_c \in \mathbb{R}^K$ form the classifier; and $y = k$ denotes class k. Cross-entropy for the labeled category updates both head and encoder during end-to-end fine-tuning [4].

For "the pump failed," a ticket classifier could predict an incident category. For a premise–hypothesis pair, the same output shape can represent entailment, contradiction or neither. The labels come from the task dataset, not from the next-sentence-prediction label vocabulary used in pretraining. Text-pair self-attention lets tokens in one text inspect the other before the final decision.

Token labeling

Attach a classifier to each contextual row:

$$p(y_i = k \mid \mathbf{x}) = \operatorname{softmax}(W_t \mathbf{h}_i + \mathbf{b}_t)_k, \qquad \mathcal{L}_{\text{tag}} = -\frac{1}{|J|} \sum_{i \in J} \log p(y_i^* \mid \mathbf{x}).$$

Here i is a token position and K is the number of labels; $W_t \in \mathbb{R}^{K \times d}$ and $\mathbf{b}_t \in \mathbb{R}^K$ are shared across positions; y_i is the token label; y_i^* is its annotated value; and J is the nonempty set of positions included in supervision. A device name split into multiple WordPieces requires a policy: supervise its first piece and ignore the others, or map the labels consistently across pieces. Exclude padding and special markers from J. At inference, map predicted pieces back to word or character spans, as developed in NLP 06. Semantic role labeling in NLP 12 reuses the same head with a predicate marker.

Extractive question answering

Suppose the passage says “Pump B stopped at noon,” and the question asks “Which pump stopped?” Pack the question and passage together, encode them, then score possible answer boundaries:

$$a_i = \mathbf{w}_s^\top \mathbf{h}_i, \quad b_i = \mathbf{w}_e^\top \mathbf{h}_i, \qquad \mathcal{L}_{\text{span}} = -\log p_s(i^*) - \log p_e(j^*).$$

Here i, j are candidate start/end positions; $\mathbf{w}_s, \mathbf{w}_e \in \mathbb{R}^d$ are learned start/end vectors; a_i, b_i are boundary scores; p_s, p_e are their softmax distributions over allowed positions; and i^*, j^* are annotated answer boundaries. Decode a valid span maximizing $a_i + b_j$ with $i \le j$ and a declared maximum length [4]. Question positions and padding cannot be returned as passage answers. If a no-answer marker is included, train that option and select its decision threshold on development data.

The BERT paper introduces exactly two new vectors for this task, a start vector and an end vector, and computes each boundary probability as a dot product with the final token states followed by a softmax over passage positions [3]. Figure 2 shows why decoding must be joint rather than two independent argmaxes: the best start and the best end must form a legal span. In the synthetic scores, the highest-scoring legal span is “Pump B.”

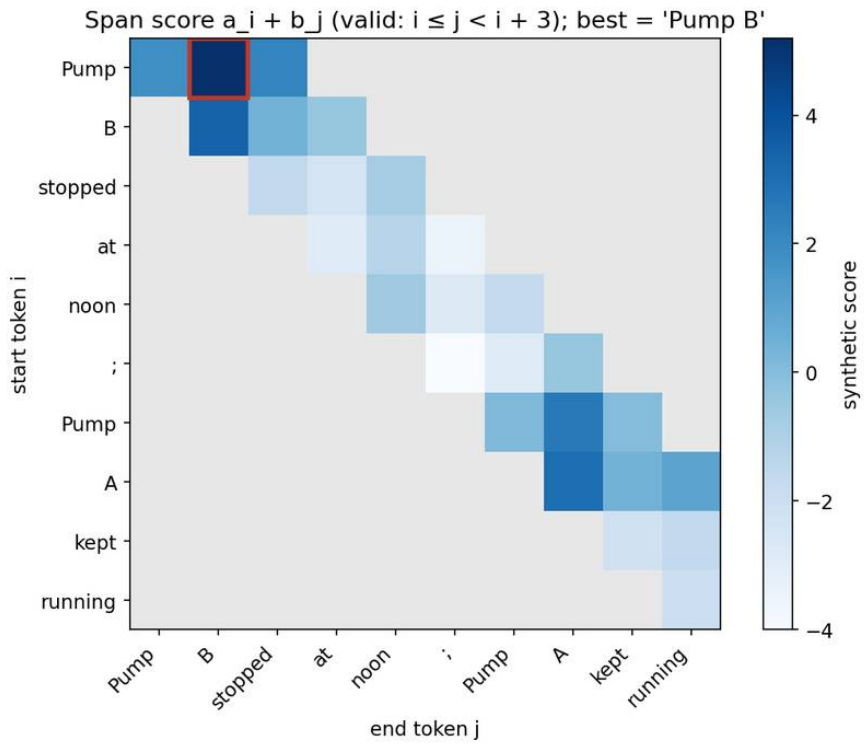


Figure 2 — Span decoding over start + end scores. Grey cells violate $i \le j$ or the length limit. Scores are invented for illustration.

The returned answer is passage text such as "Pump B." This setup does not generate an explanation or find a missing passage. Retrieval-augmented generation in NLP 09 handles finding passages; the sections on answerability below handle deciding whether a found passage is enough.

Retrieval: joint scoring or reusable vectors?

A joint BERT relevance model packs a query and candidate together, so the candidate's representation depends on that query. It must run again for another pair. A sentence embedding model instead encodes documents separately and permits caching. Sentence-BERT trains this second interface with paired/triplet sentence objectives [20]. Raw pretrained `[CLS]` vectors or arbitrary mean pooling do not automatically provide a well-trained cosine retrieval space.

For a practical system, one design is to retrieve candidates using separately stored vectors, then jointly score a smaller set of query–candidate pairs. Evaluate the retrieval stage's candidate recall separately from the final ranker. NLP 04 explains pooling and similarity; NLP 09 develops the full retrieval pipeline.

Single-text classification: sentiment analysis

Plain version. Sentiment analysis asks whether a text expresses a positive or negative opinion. It looks easy, but negation, contrast and mixed opinions mean that counting nice and nasty words is not enough.

Write the fine-tuning objective over n labeled examples:

$$\mathcal{L}(\theta, \phi) = -\frac{1}{n} \sum_{i=1}^{n} \log p_{\theta,\phi}(y_i \mid \mathbf{x}_i).$$

Here θ denotes the encoder parameters, initialized from pretraining; $\phi = (W_p, \mathbf{b}_p, W_c, \mathbf{b}_c)$ denotes the head parameters, initialized randomly; $\mathbf{x}_i$ is the i-th packed text; y_i is its label; and $p_{\theta,\phi}$ is the classifier distribution defined above. Full fine-tuning updates both θ and ϕ. Freezing θ and training only ϕ is cheaper, but it can only reuse whatever distinctions the frozen features already make.

Two benchmark datasets and what they measure

The **Stanford Sentiment Treebank** (**SST**) contains 11,855 single sentences from movie reviews; the authors parsed them and had annotators label all 215,154 unique phrases in those parse trees [5]. Phrase-level labels let researchers study how sentiment composes: "not" combined with "good" is a phrase with its own label. The paper reports both fine-grained (five-class) and binary sentence-level results [5]. GLUE later adopted the two-way split as **SST-2**, listing about 67k training and 1.8k test examples and evaluating it with accuracy [14]. The original SST models were recursive networks over parse trees; ML 03A explains that architecture family.

The **IMDB** review dataset from Maas et al. contains 50,000 reviews divided evenly into 25,000 training and 25,000 test reviews, plus 50,000 unlabeled reviews; the labeled tasks are balanced two-class problems [6]. Its unit is a whole review, not a sentence. Reviews are often longer than a short encoder window, so a document classifier needs a stated truncation or chunking policy. Keep the head, the tail, or pool predictions over windows — each policy answers a slightly different question, because a review frequently changes tone near its end.

These two datasets illustrate a recurring issue: the **unit of supervision** defines the task. A sentence-level sentiment model trained on isolated opinions may fail on a whole conversation containing quotations, corrections and several speakers. If deployment includes previous turns, annotation and training should include the same permitted context. Conversely, giving annotators access to the final resolution while the model only sees the first message creates an impossible target. NLP 03 sets out this prediction contract in detail for count-based classifiers; the same contract applies after BERT.

What a contextual encoder adds, and what it does not

A bag-of-words classifier gives "good" the same weight in "the battery is good" and "the battery is not good." A contextual encoder computes a different vector for "good" in each sentence, so the

head can, in principle, learn the flip. NLP 03 runs a controlled negation test with count features and shows how sparse evidence leaves such a flip underdetermined. Context makes the distinction *representable*; the labeled data must still make it *learnable*. Phrase identity is also different from scope: distinguishing "not every repair succeeded" from "every repair failed" requires a compositional distinction that adjacent-word features do not supply, and that a fine-tuned encoder learns only if the training data contains it.

Treat sentiment labels as an annotation decision, too. A review can praise the screen and criticize the battery. Single-label sentiment forces such a review into one class; aspect-based sentiment instead assigns opinions to targets. Neither is "the true" task. Choose the one whose output the application consumes.

Text pairs: natural language inference

Plain version. A text can support a conclusion without proving that conclusion true in the world. Keep textual entailment and evidence quality separate.

Natural language inference (NLI) classifies a premise–hypothesis pair as entailment, contradiction or neutral under the annotation convention. If the premise is "Mina repaired a sensor," the hypothesis "Someone repaired something" follows under an ordinary interpretation. "Mina repaired every sensor" does not follow, and "Mina repaired no sensor" conflicts with the premise.

For BERT, pack the pair as `[CLS] premise [SEP] hypothesis [SEP]` with segment IDs 0 and 1, then apply the sequence head with $K = 3$:

$$\hat{y} = \underset{k \in \{\mathrm{E,N,C}\}}{\operatorname{argmax}} \, p_{\theta,\phi}(y = k \mid [\mathrm{CLS}]\, \mathbf{u}\, [\mathrm{SEP}]\, \mathbf{v}\, [\mathrm{SEP}]).$$

Here $\mathbf{u}$ is the premise token sequence, $\mathbf{v}$ is the hypothesis token sequence, $\mathrm{E}, \mathrm{N}, \mathrm{C}$ denote entailment, neutral and contradiction, and the other symbols are as in the sequence-classification head. Because self-attention crosses the `[SEP]` boundary, every hypothesis token can attend to every premise token in every layer. That joint encoding lets the decision depend on token-level comparisons between the two texts — and it is also the reason a pair model cannot cache one side.

SNLI and MultiNLI

The **Stanford Natural Language Inference (SNLI)** corpus contains 570,152 human-written sentence pairs labeled for entailment, contradiction and semantic independence (neutral) [7]. Annotators were shown an image caption as the premise and wrote hypotheses. The paper discusses a subtle labeling choice: whether two sentences should be assumed to describe the same event. Without that assumption, many apparent contradictions would be neutral. The corpus fixes one convention so that each pair receives one label [7]. That is exactly the kind of decision you must make — and document — for your own inference data.

MultiNLI extends the design to 433k examples from ten genres of written and spoken English. Only five genres appear in training, so the development and test sets have a **matched** section from

the training genres and a **mismatched** section from the others [8]. The split turns domain shift into a measured quantity rather than an afterthought.

Annotation artifacts and hypothesis-only baselines

Crowd workers who are asked to write a contradiction often reach for the same tools: negation words, or a different activity. Gururangan et al. showed that a simple text classifier that sees **only the hypothesis** predicts the label for about 67% of SNLI and 53% of MultiNLI, and that phenomena such as negation and vagueness correlate strongly with particular classes [9]. Poliak et al. ran hypothesis-only baselines across ten NLI datasets and found that they significantly outperform the majority-class baseline on a number of them [10]. A model that ignores the premise cannot be performing inference, so its score measures dataset regularities.

McCoy, Pavlick and Linzen took the opposite direction: they built **HANS**, a controlled set where three syntactic heuristics — lexical overlap, subsequence and constituent — give the wrong answer, and found that models trained on MNLI, including BERT, perform very poorly on it [11]. Together these studies motivate two habits:

1. **Always run a partial-input baseline.** If a hypothesis-only (or premise-only, or question-only) model does well, part of the benchmark score is not about the relationship you care about.
2. **Build challenge pairs that flip the cue.** Hold the vocabulary fixed and change the one distinction that should matter: actor reversal, negation in the premise rather than the hypothesis, possibility versus assertion, allegation versus fact, one versus all.

An inference model predicts an annotation relationship. It does not independently verify the premise. If the premise itself is fabricated, an entailed hypothesis can also be fabricated. A retrieval-assisted application therefore needs both source selection and an answer-support check, which the question-answering sections below develop.

Argumentation mining can identify a claim, a proposed supporting statement and an expressed attack relation [2]. “The device is reliable because it passed one test” contains a recognizable argument structure. Identifying that structure does not establish that the test is representative or that the conclusion is valid. The application may store the stated reason while marking its evidential strength as unassessed.

Useful challenge pairs change one crucial distinction at a time. Fluent paraphrases are useful additional variation, but they should preserve the intended labels. A synthetic paraphrase that changes scope or removes attribution must receive a new annotation rather than inherit the original record automatically.

Extractive QA as a pair task: answers need support

Plain version. SQuAD-style question answering asks the model to point at the answer inside a given passage. The harder practical question is whether the passage contains a supported answer at all.

The **Stanford Question Answering Dataset** (**SQuAD**) contains 107,785 question–answer pairs written by crowd workers on 536 Wikipedia articles; every answer is a span of the corresponding paragraph [12]. Its two metrics both ignore punctuation and the articles *a*, *an* and *the*: **exact match** is the percentage of predictions equal to one of the reference answers, and **F1** measures token overlap between prediction and reference, averaged over questions [12]. Partial credit matters because annotators disagree mostly about non-essential words at span edges.

SQuAD 2.0 adds over 50,000 unanswerable questions written adversarially to look similar to answerable ones; a system must also decide when the paragraph supports no answer and abstain [13]. The BERT paper handles this by treating an unanswerable question as a span that starts and ends at `[CLS]`. At prediction time it compares the null score $s_{\text{null}} = \mathbf{w}_s^\top \mathbf{h}_{\text{CLS}} + \mathbf{w}_e^\top \mathbf{h}_{\text{CLS}}$ with the best non-null span score $\hat{s} = \max_{j \geq i}(a_i + b_j)$, and predicts an answer only when $\hat{s} > s_{\text{null}} + \tau$ [3]. Here τ is a threshold selected on the development set to maximize F1. This one threshold converts a span model into a selective answerer.

A local evidence-backed answerer

We can demonstrate the answer contract without a trained span model. The answerer below handles one explicit question family and returns a source interval. It ranks four synthetic passages with TF–IDF, then answers only a narrow temperature-limit question for Pump A. It preserves one-character words so that the device identifier A remains a feature. The function is not a general classifier for question intent; its caller must route the declared question family to it.

```
import re
from sklearn.feature_extraction.text import TfidfVectorizer

corpus = {
    'temperature-v2': 'Pump A temperature limit is 80 C. This policy is current.',
    'shutdown-v1': 'Pump A shutdown follows a vibration alarm.',
    'label-v1': 'A technician replaced the cabinet label.',
    'sensor-v1': 'Sensor B reports humidity once per hour.',
}
ids = list(corpus)
vectorizer = TfidfVectorizer(ngram_range=(1, 2), token_pattern=r"(?u)\b\w+\b")
D = vectorizer.fit_transform([corpus[key] for key in ids])
def rank(query):
    score = (D @ vectorizer.transform([query]).T).toarray().ravel()
    order = sorted(range(len(ids)), key=lambda i: (-score[i], ids[i]))
    return [(ids[i], float(score[i])) for i in order]

def answer_temperature(query):
    if not re.search(r'\bpump\s+a\b', query, flags=re.I):
        return {'answer': None, 'reason': 'unsupported entity'}
    key, score = rank(query)[0]
    text = corpus[key]
    pattern = r'Pump A temperature limit is (\d+) C\.'
    match = re.search(pattern, text)
    if not match:
        return {'answer': None, 'reason': 'no matching evidence'}
```

```
    evidence = text[match.start():match.end()]
    result = {'answer': f'{match.group(1)} C', 'document': key,
              'start': match.start(), 'end': match.end(), 'evidence': evidence}
    assert text[result['start']:result['end']] == result['evidence']
    return result
print(answer_temperature('What is the Pump A temperature limit?'))
print(answer_temperature('What is the Pump Z temperature limit?'))
```

The supported Pump A answer returns 80 C, its document version and the character interval `0–3 3`. A Pump Z question returns `unsupported entity` even though temperature-related passages exist. A span head trained on SQuAD-style data replaces the regular expression with learned boundary scores, but the surrounding contract — entity check, evidence interval, verified offsets, explicit abstention — should stay.

The explicit entity check prevents a high lexical score from transferring Pump A's limit to an unsupported device. The evidence expression also requires the source to state the expected relation, not merely contain a number. A more flexible system needs a richer entity and relation check, including scope, units and applicable dates. Expand those capabilities with labeled tests rather than weakening the check until every question receives an answer.

The answerer searches only the top passage. It can therefore abstain when the correct evidence appears lower in the ranking. That limitation is intentional and visible. A later version could inspect several candidates under a fixed budget, while recording which candidate supplied the final answer. Measure the effect on both supported answers and false acceptance.

The function stores an evidence slice and verifies the offsets. This catches serialization mistakes, but not every semantic mistake. A quoted sentence can still describe the wrong entity, an outdated policy or a hypothetical case. Exact quotation is an important mechanical check; the support relation remains a separate part of the contract.

Relevance is not answerability

A passage can discuss the topic and still lack the requested fact. A system should have a meaningful way to return insufficient evidence.

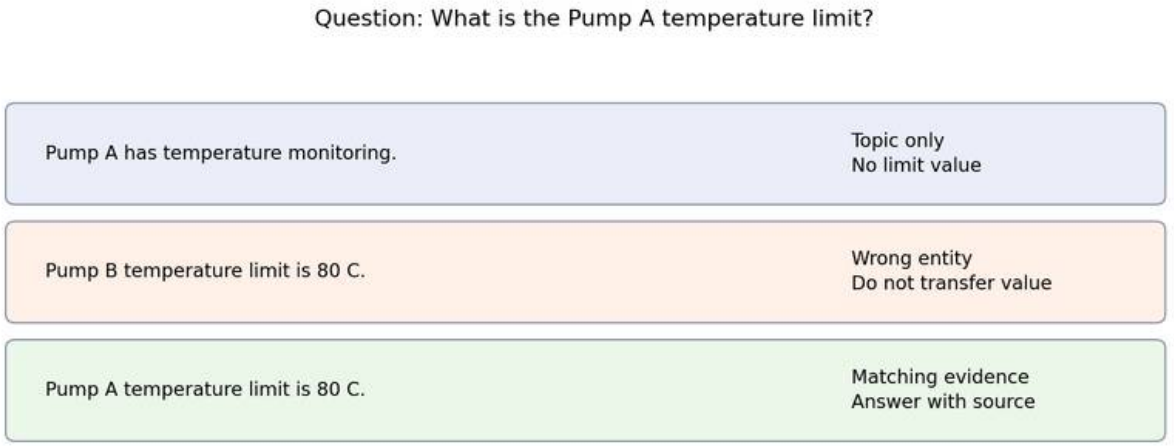


Figure 3 — Topic overlap, wrong-entity evidence and complete support require different decisions. Scores alone do not define answerability.

For a narrow question, define the required fields before evaluating a system. The temperature example needs device identity, a limit relation, a value and a unit. It may also need an effective date or operating condition. Missing any required field can make the available evidence insufficient even if the passage ranks first.

An abstention policy decides when to withhold an answer or ask for clarification. If confidence scores are used, select their threshold on development data with the intended costs of a wrong answer and unnecessary review — exactly the role of τ in the SQuAD 2.0 head. A raw similarity score is not a calibrated correctness probability. Its scale can change when the corpus, tokenizer or embedding model changes.

Report both coverage and quality among accepted answers:

$$\text{coverage} = \frac{N_{\text{accepted}}}{N_{\text{all}}}, \qquad \text{accepted accuracy} = \frac{N_{\text{correct, accepted}}}{N_{\text{accepted}}}.$$

Here N_{all} is the number of evaluated questions; N_{accepted} is the count receiving an answer; and $N_{\text{correct, accepted}}$ counts accepted answers judged correct under the task contract. Accepted accuracy is undefined when no answer is accepted, so report that condition rather than presenting zero acceptance as a successful accuracy result.

Coverage must include questions without available answers. Otherwise a system that always answers can look good on a collection designed to contain an answer everywhere — the weakness SQuAD 2.0 was built to expose [13]. Include wrong entities, missing values, conflicting sources and unsupported time scopes. These cases test whether the model recognizes the limits of evidence instead of merely producing a likely span.

Diagnose failures with controlled substitutions

Replace one imperfect stage with a known-correct input to discover where the final answer fails. Keep these diagnostic results separate from end-to-end deployment scores.

First provide the answerer with the annotated supporting passage directly. If it still produces the wrong value, retrieval is not the immediate cause. Inspect entity matching, unit handling, scope and realization. If it succeeds with gold evidence but fails with retrieved evidence, inspect candidate recall and the selected passage order. This comparison localizes a bottleneck without pretending that perfect evidence will be available at deployment.

Next preserve the correct passage but add irrelevant or conflicting passages. A system that fails only after extra context arrives has a context-selection or evidence-integration problem. Vary one property at a time: a different device identifier, an obsolete version, a quoted allegation or an unsupported command. Label the intended behavior before running the experiment so the interpretation does not shift to excuse the output.

A supported answer requires a chain of successful decisions. For questions whose applicable evidence is in the corpus, write:

$$P(R \cap E \cap G) = P(R)P(E \mid R)P(G \mid R, E).$$

Here P denotes probability, R is the event that sufficient evidence survives retrieval, E is the event that the evidence is interpreted correctly, and G is the event that the output faithfully realizes the interpretation. The expression is the probability chain rule, not an independence assumption. Multiplying unrelated unconditional stage scores is not equivalent: reader accuracy on hand-picked easy passages may differ from reader accuracy on the retriever's actual candidates.

This constructed evaluation makes the conditioning visible:

```
import numpy as np

retrieved = np.arange(100) < 80
interpreted = np.arange(100) < 72
realized = np.arange(100) < 68
r = retrieved.mean()
e = interpreted[retrieved].mean()
g = realized[retrieved & interpreted].mean()
complete = (retrieved & interpreted & realized).mean()
print('Conditional stage rates:', round(r, 3), round(e, 3), round(g, 3))
print('Complete supported answers:', round(complete, 3))
assert np.isclose(r * e * g, complete)
```

The rates are `0.8`, `0.9` and `0.944`, while complete success is `0.68`. These are synthetic outcomes, not reported system performance. Individually respectable component results need not produce equally strong complete answers. Finally retain realistic query distributions: a diagnostic suite deliberately overrepresents difficult cases, while a representative held-out set estimates ordinary behavior. Report both with their purposes stated, and do not mix them into one aggregate number.

Benchmarks: GLUE and SuperGLUE

Plain version. A benchmark suite runs one pretrained model through many small tasks and summarizes them. It is good for comparing general-purpose encoders and poor at telling you whether your application works.

GLUE collects nine English understanding tasks — single-sentence tasks (acceptability and SST-2 sentiment), similarity and paraphrase tasks, and inference tasks including MNLI — together with a diagnostic dataset for specific linguistic phenomena and an online leaderboard [14]. None of its datasets was created from scratch for the benchmark, and some keep test labels private so that the leaderboard is used fairly [14]. Metrics differ by task: accuracy for SST-2, accuracy and F1 for paraphrase tasks, Pearson and Spearman correlations for sentence similarity, and the Matthews correlation coefficient for the unbalanced acceptability task [14]. The BERT paper reported its main transfer results on GLUE, SQuAD and a commonsense-inference set [3].

SuperGLUE keeps GLUE's design — a leaderboard around eight tasks, a single summary number and analysis tools — but selects more difficult tasks, including yes/no question answering over passages and three-class entailment on embedded clauses [15].

Read a suite average carefully. It mixes tasks with different sizes, metrics and noise levels. A gain can come from one small task. It says nothing about artifacts unless partial-input baselines and challenge sets are reported, and nothing about your domain unless your domain is in it. Use suites to choose a starting checkpoint; use your own held-out data to choose a system.

Fine-tuning practice

Plain version. Fine-tuning is short supervised training starting from good weights. The main risks are a learning rate that destroys what pretraining learned, too few or too many updates, unlucky random seeds, and data whose class balance or label meaning differs from deployment.

Learning rate, epochs and schedule

The BERT authors kept dropout at 0.1 and reported that the following ranges worked well across their tasks: batch size 16 or 32; Adam learning rate 5×10^{-5}, 3×10^{-5} or 2×10^{-5}; and 2, 3 or 4 epochs [3]. They also observed that large datasets (100k+ labeled examples) were far less sensitive to these choices than small datasets [3]. The released classification script defaults to a learning rate of 5×10^{-5}, three epochs, a maximum sequence length of 128 and a warmup proportion of 0.1 [4]. Its optimizer schedule is:

$$\eta_t = \begin{cases} \eta_{\max}\, t/t_w, & t < t_w, \\ \eta_{\max}\,(1 - t/T), & t \geq t_w. \end{cases}$$

Here η_t is the learning rate at update step t; $\eta_{\max}$ is the configured peak rate; T is the total number of training steps; and t_w is the number of warmup steps, a fixed fraction of T. The same implementation applies weight decay of 0.01 except to layer-normalization and bias parameters, and

clips the global gradient norm at 1.0 [4]. AdamW-style decoupled weight decay is the standard name for this optimizer family [26].

Why so small a learning rate? Each update moves pretrained weights that encode useful distinctions. A rate suitable for training a randomly initialized network can erase them in a few hundred steps, after which the model is effectively learning from the small labeled set alone.

Instability and random restarts

For BERT-LARGE, the original authors found fine-tuning sometimes unstable on small datasets; they ran several random restarts, which changed data shuffling and classifier initialization while keeping the same pretrained checkpoint, and selected the best model on the development set [3]. Mosbach, Andriushchenko and Klakow analyzed this instability for BERT, RoBERTa and ALBERT on GLUE tasks. They attribute failed runs to optimization difficulties early in training, characterized by vanishing gradients, and the remaining variance to generalization differences between runs with almost identical training loss [16]. They did not observe the instability for BERT-BASE. Their guidelines for small datasets are to use small learning rates **with** Adam's bias correction, and to train for considerably more iterations, close to zero training loss [16].

The practical lesson: report the spread over several seeds, not only the best seed, and select by development data only. A single lucky run is not a method.

Class imbalance

Real label distributions are rarely balanced. Class weighting changes how strongly each training example contributes:

$$\mathcal{L}_{\text{weighted}} = -\frac{1}{n}\sum_{i=1}^{n} \alpha_{y_i} \log p_\theta(y_i \mid d_i).$$

Here $\mathcal{L}_{\text{weighted}}$ is weighted cross-entropy, n is training-example count, i indexes examples, d_i is the document, y_i is its annotated class, $\alpha_{y_i} > 0$ is the chosen weight for that class and p_θ is the classifier distribution. The weighting policy changes the training objective; it does not manufacture missing linguistic variety.

Weighting, oversampling and threshold changes answer different problems. Weighting changes parameter learning. Oversampling changes the presentation frequency of examples, sometimes repeatedly exposing the same few minority examples. Thresholding changes the action policy after scores are computed. Hagiwara's development discussion covers imbalance and training choices [1]. Evaluate any intervention on a realistic prevalence distribution rather than on an artificially balanced final test.

Calibration

A classifier can choose the right label while giving unreliable probabilities. Calibration asks whether predicted confidence agrees with observed correctness over comparable examples. A model can preserve its argmax decisions while changing confidence. Guo et al. study this distinction and post-training temperature scaling [17]:

$$\tilde{p}(y = k \mid \mathbf{x}) = \operatorname{softmax}(\mathbf{z}/T_c)_k, \qquad \mathrm{ECE} = \sum_{b=1}^{B} \frac{|S_b|}{n} \left| \operatorname{acc}(S_b) - \operatorname{conf}(S_b) \right|.$$

Here $\mathbf{z}$ is the logit vector, $T_c > 0$ is a single temperature fit by minimizing negative log-likelihood on held-out development data, and $\tilde{p}$ is the rescaled distribution; for the expected calibration error ECE, examples are grouped into B confidence bins S_b, n is the number of examples, $\operatorname{acc}(S_b)$ is the accuracy within bin b and $\operatorname{conf}(S_b)$ is its mean predicted confidence. Because dividing all logits by one positive number preserves their order, temperature scaling cannot change accuracy.

Desai and Durrett measured BERT and RoBERTa on NLI, paraphrase detection and commonsense reasoning. Used out of the box, the pretrained models were calibrated in-domain and had lower out-of-domain calibration error than the baselines they compared; temperature scaling reduced in-domain error further, and label smoothing helped out of domain [18]. Their result is a property of those tasks and models, not a guarantee for your fine-tuned checkpoint. Measure calibration on your own development data whenever probabilities drive an action such as abstention.

Evaluation: accuracy, macro-F1 and selective decisions

Plain version. Accuracy counts correct answers. Macro-F1 makes every class count equally, so a rare class cannot be ignored. Selective metrics ask how good the accepted answers are when the system may abstain.

For one class treated as positive:

$$P = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}, \qquad R = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}}, \qquad F_1 = \frac{2PR}{P + R}.$$

Here P is precision, R is recall, TP counts true positives, FP counts false positives, FN counts false negatives and F_1 is their harmonic mean. A zero denominator requires an explicitly chosen convention. Macro F1 averages per-class F1 values equally; support-weighted averaging gives more influence to common classes. The metric documentation specifies these choices [19].

Accuracy is adequate for balanced tasks such as SST-2 or IMDB [6], [14]. For three-class NLI, report per-class F1 as well: a model can reach decent accuracy while confusing neutral with contradiction. For skewed tasks, macro-F1 or the Matthews correlation makes failures on a rare category visible, but neither encodes the actual cost of those failures. Missing an urgent ticket and misrouting a routine receipt can have different consequences. Write a cost policy separately rather than treating a convenient score as a complete utility measure.

For selective prediction:

$$\text{coverage} = \frac{1}{n}\sum_{i=1}^{n} a_i, \qquad \text{risk}_{\text{accepted}} = \frac{\sum_{i=1}^{n} a_i \mathbf{1}[\hat{y}_i \neq y_i]}{\sum_{i=1}^{n} a_i}.$$

Here n is evaluated-example count, $a_i \in \{0, 1\}$ indicates acceptance of example i, $\hat{y}_i$ is its prediction, y_i is its reference label, and $\mathbf{1}[\cdot]$ is one when its condition holds and zero otherwise. Accepted risk is undefined when no example is accepted. A risk-versus-coverage curve describes the trade-off over thresholds; choose the deployed threshold using validation rather than final test labels.

Then read the errors. Create an error table with text, gold label, predicted label, probability, language, length, source group and an analyst's explanation. Useful error categories include annotation ambiguity, missing context, negation, unseen vocabulary, multi-topic input, quotation and genuinely out-of-scope text. These categories suggest different repairs. More training examples can help missing vocabulary; they cannot supply a conversation turn that is never available at prediction time. ML 10 covers the general evaluation and monitoring toolkit.

Keep model and preprocessing artifacts together. The tokenizer and vocabulary, maximum length and truncation policy, label order, calibration temperature and decision threshold are a single inference package. Reloading weights with another tokenizer silently changes meanings. Store a versioned label schema and a few deterministic contract examples with the package so that serving can check its assumptions.

A runnable experiment: pretrain, then fine-tune

Plain version. We pretrain a two-layer encoder from scratch with masked language modeling on a tiny unlabeled corpus, then fine-tune it for a sentiment task and an NLI task. The data are synthetic, so the experiment demonstrates mechanisms, not benchmark performance.

The design isolates three questions:

1. **Does pretraining transfer?** The unlabeled corpus contains every adjective, each next to `love` or `hate`. The sentiment training set never contains `superb` or `terrible`; the test set contains only those two adjectives, with and without `not`. A randomly initialized encoder has never seen them in any context.
2. **Does a hypothesis-only baseline expose a shortcut?** In the NLI training data, `not` appears only in contradiction hypotheses — a deliberate annotation artifact.
3. **Does a challenge set reveal it?** The challenge set puts `not` in an entailment (both sentences negated) and builds a contradiction whose hypothesis has no `not` (the premise is negated instead).

The encoder follows the BERT recipe from NLP 10 in miniature: token + position + segment embeddings, layer normalization, two Transformer encoder blocks, 15% masked positions, and a pooled `[CLS]` head with a tanh layer for fine-tuning. It uses AdamW [26].

```
import itertools, random
import torch
from torch import nn
```

```
from torch.nn import functional as F
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, f1_score

torch.manual_seed(0); random.seed(0); torch.set_num_threads(1)
POS, NEG = ['good', 'great', 'fine', 'superb'], ['bad', 'awful', 'poor', 'terrible']
ITEMS, NAMES = ['battery', 'screen', 'camera', 'speaker'], ['mina', 'ravi', 'lena']
VERBS = {'repaired': 'repair', 'cleaned': 'clean', 'tested': 'test'}
OBJS = ['the pump', 'the valve', 'the sensor']
words = set(' '.join(POS + NEG + ITEMS + NAMES + OBJS).split()) | set(VERBS) \
    | set(VERBS.values()) | set('the is not i love hate it someone did'.split())
vocab = ['[PAD]', '[CLS]', '[SEP]', '[MASK]'] + sorted(words)
idx, L, D = {w: i for i, w in enumerate(vocab)}, 16, 32

def batch(examples):                       # each example: (text,) or (text_a, text_b)
    ids, segs = [], []
    for ex in examples:
        toks, seg = ['[CLS]'], [0]
        for s, text in enumerate(ex):
            part = text.split() + ['[SEP]']
            toks += part; seg += [s] * len(part)
        ids.append([idx[t] for t in toks] + [0] * (L - len(toks)))
        segs.append(seg + [0] * (L - len(seg)))
    return torch.tensor(ids), torch.tensor(segs)

class TinyBERT(nn.Module):
    def __init__(self):
        super().__init__()
        self.tok, self.pos, self.seg = nn.Embedding(len(vocab), D), nn.Embedding(L, D),
         ↪  nn.Embedding(2, D)
        self.norm = nn.LayerNorm(D)
        block = nn.TransformerEncoderLayer(D, 2, 64, dropout=0.0, activation='gelu',
         ↪  batch_first=True)
        self.encoder = nn.TransformerEncoder(block, 2)
        self.mlm = nn.Linear(D, len(vocab))
    def forward(self, ids, seg):
        h = self.tok(ids) + self.pos(torch.arange(ids.shape[1])) + self.seg(seg)
        return self.encoder(self.norm(h), src_key_padding_mask=ids.eq(0))

# 1) Masked-LM pretraining on unlabeled text (no sentiment or NLI labels).
corpus = [(f'i love it the {i} is {a}',) for i in ITEMS for a in POS] \
    + [(f'i hate it the {i} is {a}',) for i in ITEMS for a in NEG] \
    + [(f'{n} {v} {o}',) for n in NAMES for v in VERBS for o in OBJS]
model = TinyBERT()
ids, seg = batch(corpus)
opt = torch.optim.AdamW(model.parameters(), lr=3e-3)
for step in range(400):
    pick = (torch.rand(ids.shape) < 0.15) & ids.gt(3)    # 15% of real tokens
    logits = model.mlm(model(ids.masked_fill(pick, idx['[MASK]']), seg))
    loss = F.cross_entropy(logits.flatten(0, 1), ids.masked_fill(~pick, -100).flatten())
    opt.zero_grad(); loss.backward(); opt.step()
print('final MLM loss:', round(loss.item(), 3))
pretrained_state = {k: v.clone() for k, v in model.state_dict().items()}
```

```
# 2) Fine-tune encoder + [CLS] head end to end.
def finetune(encoder, train, n_classes, epochs=60, lr=1e-3):
    torch.manual_seed(1)
    head = nn.Sequential(nn.Linear(D, D), nn.Tanh(), nn.Linear(D, n_classes))
    opt = torch.optim.AdamW([*encoder.parameters(), *head.parameters()], lr=lr)
    x, y = batch([ex for ex, _ in train]), torch.tensor([lab for _, lab in train])
    for _ in range(epochs):
        loss = F.cross_entropy(head(encoder(*x)[:, 0]), y)
        opt.zero_grad(); loss.backward(); opt.step()
    return lambda exs: head(encoder(*batch(exs))[:, 0]).argmax(-1).tolist()

def report(name, data, predict):
    gold = [lab for _, lab in data]
    with torch.no_grad():
        pred = predict([ex for ex, _ in data])
    print(f'{name:46s} acc={accuracy_score(gold, pred):.2f} '
          f'macro-F1={f1_score(gold, pred, average="macro"):.2f}')

# Task A: sentiment with negation. "superb" and "terrible" never get a label.
def review(item, adj, negated):
    return ((f'the {item} is {"not " * negated}{adj}',), int(adj in POS) ^ negated)
labeled = [a for a in POS + NEG if a not in ('superb', 'terrible')]
train_a = [review(i, a, n) for i in ITEMS for a in labeled for n in (0, 1)]
test_a = [review(i, a, n) for i in ITEMS for a in ('superb', 'terrible') for n in (0, 1)]
report('sentiment, pretrained encoder', test_a, finetune(model, train_a, 2))
report('sentiment, random-init encoder', test_a, finetune(TinyBERT(), train_a, 2))

# Task B: NLI (0 entailment, 1 neutral, 2 contradiction) with a negation shortcut.
def pair(n, v, o, kind):
    p, neg = f'{n} {v} {o}', f'{n} did not {VERBS[v]} {o}'
    other = list(VERBS)[(list(VERBS).index(v) + 1) % 3]
    return {'ent': ((p, f'someone {v} {o}'), 0), 'neu': ((p, f'{n} {other} {o}'), 1),
            'con': ((p, neg), 2),                   # training: "not" only in contradictions
            'neg-ent': ((neg, neg), 0),             # challenge: "not" in an entailment
            'neg-con': ((neg, p), 2)}[kind]         # challenge: contradiction without "not"
triples = list(itertools.product(NAMES, VERBS, OBJS)); random.shuffle(triples)
train_b = [pair(*t, k) for t in triples[:18] for k in ('ent', 'neu', 'con')]
test_b = [pair(*t, k) for t in triples[18:] for k in ('ent', 'neu', 'con')]
challenge = [pair(*t, k) for t in triples[18:] for k in ('neg-ent', 'neu', 'neg-con')]

vec = CountVectorizer(token_pattern=r'\S+').fit([ex[1] for ex, _ in train_b])
hyp_only = LogisticRegression(C=10, max_iter=1000).fit(
    vec.transform([ex[1] for ex, _ in train_b]), [lab for _, lab in train_b])
hyp_predict = lambda exs: hyp_only.predict(vec.transform([ex[1] for ex in exs])).tolist()
model.load_state_dict(pretrained_state)
pair_predict = finetune(model, train_b, 3)
for split, data in [('in-distribution', test_b), ('challenge', challenge)]:
    report(f'NLI {split}, hypothesis-only', data, hyp_predict)
    report(f'NLI {split}, pair encoder', data, pair_predict)
counter = [pair(*t, k) for t in triples[:18] for k in ('neg-ent', 'neg-con')]
model.load_state_dict(pretrained_state)
report('NLI challenge, pair encoder + counterexamples', challenge,
       finetune(model, train_b + counter, 3))
```

The run takes a few seconds on one CPU thread and prints:

```
final MLM loss: 0.621
sentiment, pretrained encoder                  acc=1.00 macro-F1=1.00
sentiment, random-init encoder                 acc=0.25 macro-F1=0.20
NLI in-distribution, hypothesis-only           acc=1.00 macro-F1=1.00
NLI in-distribution, pair encoder              acc=1.00 macro-F1=1.00
NLI challenge, hypothesis-only                 acc=0.33 macro-F1=0.22
NLI challenge, pair encoder                    acc=0.67 macro-F1=0.56
NLI challenge, pair encoder + counterexamples  acc=1.00 macro-F1=1.00
```

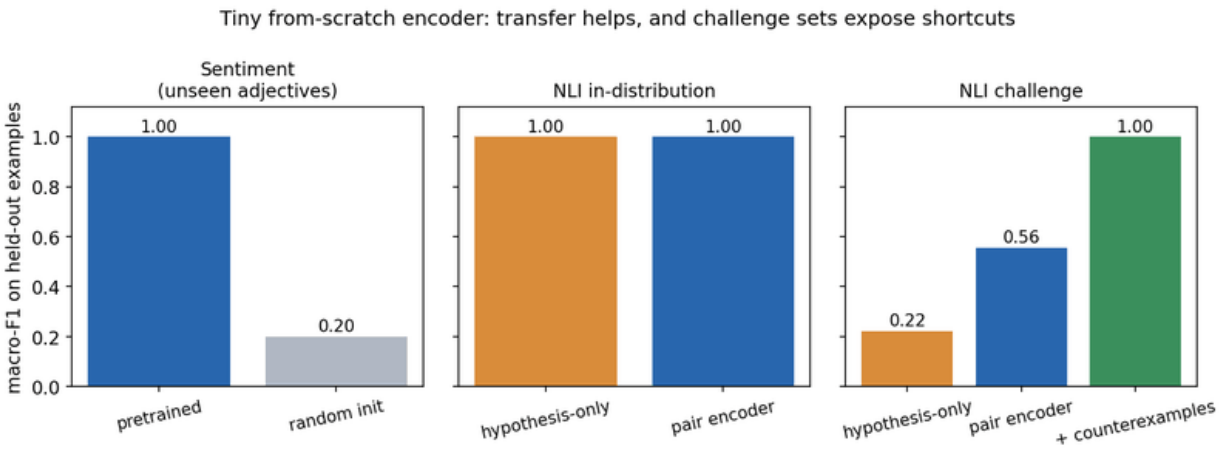


Figure 4 — Macro-F1 from the run above, regenerated by the figure script. Sixteen sentiment test sentences, 27 in-distribution and 27 challenge NLI pairs; synthetic data only.

Transfer. The pretrained encoder labels all sixteen test sentences correctly, including the negated ones, although `superb` and `terrible` never appeared with a label. During pretraining they shared contexts with `love` and `hate` respectively, so their representations already sat near the labeled adjectives of the same polarity. The randomly initialized encoder has only random embeddings for the two words, and the fine-tuning set gives it nothing to connect them to the labels. This is the BERT argument at toy scale: unlabeled text can supply distinctions that a small labeled set cannot. It also shows the dependency: transfer worked because the unlabeled corpus contained the relevant co-occurrence. Pretraining on unrelated text would not help.

Shortcut. In distribution, the hypothesis-only logistic regression is perfect. That is the warning sign: a model that never reads the premise cannot be doing inference, so the test set does not require inference. The fine-tuned pair encoder is also perfect in distribution, which on its own tells us nothing about what it uses.

Challenge. On the challenge set, the hypothesis-only baseline drops to chance-level accuracy. Inspecting the predictions shows that the pair encoder classifies every negated contradiction and every neutral pair correctly, but labels all nine doubly negated entailments as contradictions — it has partly learned "`not` in the hypothesis means contradiction." Adding eighteen counterexamples of the two challenge types (built from the *training* names, verbs and objects) fixes the

challenge set. Be careful with that last line: the counterexamples share templates with the challenge set, so a perfect score shows that the model can learn the distinction once the data demands it, not that it would generalize to a new kind of shortcut.

Everything here is deliberately small: a 40-word vocabulary, sentences of at most a dozen tokens, one seed. Across the four seeds we tried, the qualitative pattern is the same, but the exact random-init and challenge numbers move. Exercise 5 asks you to measure that spread.

Modern view

Surveys and their maps. Minaee et al.'s ACM Computing Surveys review covers more than 150 deep-learning text classifiers and more than 40 datasets across sentiment analysis, news categorization, question answering and natural language inference [21]. It groups models by architecture — feed-forward, recurrent, convolutional, capsule, attention, memory-augmented, graph, Siamese, hybrid, and Transformer-based pretrained language models — and its central takeaway is that pretrained language models fine-tuned with task labels had set new states of the art on many text classification tasks. Chang et al.'s evaluation survey separates *what* to evaluate, *where* (which benchmarks) and *how* (automatic, human, protocols) [24], and Zhao et al.'s survey places classification inside the broader workflow of pretraining, adaptation, utilization and capability evaluation [25]. Read together, they argue for task-specific evaluation plans rather than one leaderboard number.

Encoders in the LLM era (2023–2026). Large decoder-only models can classify by prompting, without task-specific weights. Zhang et al. evaluated LLMs on 13 sentiment tasks across 26 datasets against small models trained on domain data. LLMs performed satisfactorily on simpler tasks but lagged on tasks that need structured sentiment information or a deeper understanding of specific phenomena, while clearly outperforming the small models in few-shot settings [22]. The practical reading: prompting is a strong option when labels are scarce, and a fine-tuned encoder remains competitive when labeled data exist and latency matters. Encoders themselves are still evolving. ModernBERT, published at ACL 2025, brings recent architecture and training optimizations to encoder-only models, trains on 2 trillion tokens with a native 8,192-token sequence length, and reports strong results across classification and retrieval evaluations along with speed and memory efficiency [23]. The fine-tuning interface in this chapter — a head on `[CLS]` or on token states — carries over unchanged.

What did not change. Annotation artifacts [9], [10] and heuristic shortcuts [11] are properties of datasets and training signals, not of one architecture. A larger or newer model can still learn them, and a prompted model can bring its own. Partial-input baselines, challenge sets, calibration checks [18] and abstention policies [13] remain the tools for finding out.

Open problems. Robustness under domain shift (the MultiNLI matched/mismatched split [8] is an early, narrow version), calibrated abstention for long and multi-passage inputs, multilingual and code-switched sentiment, labels that change with policy, and evaluation sets that leak into pretraining corpora. Each is easier to study with a small, transparent baseline next to the large model.

Key takeaways

- The encoder is shared; the head and the labels define the task. Choose sequence, token or span output first.
- Pair tasks pack both texts into one sequence so that attention can compare them token by token; the price is that neither side can be cached.
- The unit of supervision — sentence, review, conversation — is part of the task definition.
- A hypothesis-only (partial-input) baseline that scores well means the benchmark does not fully require the relationship you care about.
- Challenge sets that flip one cue at a time reveal shortcuts that in-distribution accuracy hides.
- Fine-tune with small learning rates, warmup and decay; report variance over seeds and select on development data.
- Evaluate decisions (accuracy, macro-F1), probabilities (calibration) and abstention (coverage, accepted risk) as separate properties.
- Relevance is not answerability: a span answer needs a supporting passage and an explicit way to say "not stated."

Exercises

1. **Truncation policy.** You must classify reviews that are often longer than the encoder window. Compare keeping the first tokens, the last tokens, and averaging predictions over overlapping windows. Which reviews would each policy misjudge?

   Hint

   Write three short synthetic reviews whose verdict appears at the start, at the end, and is reversed halfway ("at first I liked it ... but"). Decide which label the annotation guideline intends before choosing a policy.

2. **A partial-input audit.** For the NLI data in the experiment, train a premise-only baseline as well. Predict its accuracy before running it, then explain the result.

   Hint

   In the training templates, every premise appears with all three labels. A premise-only model can do no better than the majority class; compare that with the hypothesis-only result.

3. **SQuAD 2.0 threshold.** Given development-set scores s_{null} and $\hat{s}$ for answerable and unanswerable questions, describe how to choose τ, and what happens to coverage and accepted accuracy as τ grows.

   Hint

   Sweep τ over the observed score differences $\hat{s} - s_{\text{null}}$. A larger τ answers fewer questions. Plot coverage against accepted accuracy, then pick the point that matches the stated cost of a wrong answer.

4. **Calibration.** Add temperature scaling to the experiment's sentiment classifier: fit one temperature on a held-out split by minimizing negative log-likelihood. Show that accuracy cannot change and measure how the mean confidence moves.

   Hint

   Dividing logits by a positive scalar preserves their order. Optimize $\log T_c$ instead of T_c to keep the temperature positive.

5. **Seed variance.** Run the experiment with five seeds for pretraining and fine-tuning. Report the mean and range of each line, and decide which conclusions survive.

   Hint

   Change both `torch.manual_seed` calls. Separate claims about direction (pretrained better than random-init; challenge lower than in-distribution) from claims about exact values.

6. **Inference challenge pairs.** Create six NLI pairs that separately test role reversal, negation in the premise, modality, attribution, quantifier scope and time. Write the intended label and the assumptions before evaluating any model.

   Hint

   Preserve neutral as an option when the premise supplies too little information. For attribution, compare "Mina said the pump failed" with "The pump failed."

References

1. Masato Hagiwara, *Real-World Natural Language Processing: Practical Applications with Deep Learning*, Chapters 9–11, Manning, 2021. Publisher
2. Susan McRoy, *Principles of Natural Language Processing*, Chapters 6 and 8, open textbook, 2021. Open textbook
3. Jacob Devlin, Ming-Wei Chang, Kenton Lee and Kristina Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," NAACL-HLT, 2019. Paper
4. Google Research, *BERT: original implementation*, software, 2018. Classification, Question answering, Optimizer
5. Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Ng and Christopher Potts, "Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank," EMNLP, 2013. Paper
6. Andrew L. Maas, Raymond E. Daly, Peter T. Pham, Dan Huang, Andrew Y. Ng and Christopher Potts, "Learning Word Vectors for Sentiment Analysis," ACL-HLT, 2011. Paper
7. Samuel R. Bowman, Gabor Angeli, Christopher Potts and Christopher D. Manning, "A large annotated corpus for learning natural language inference," EMNLP, 2015. Paper

8. Adina Williams, Nikita Nangia and Samuel R. Bowman, “A Broad-Coverage Challenge Corpus for Sentence Understanding through Inference,” NAACL-HLT, 2018. Paper
9. Suchin Gururangan, Swabha Swayamdipta, Omer Levy, Roy Schwartz, Samuel R. Bowman and Noah A. Smith, “Annotation Artifacts in Natural Language Inference Data,” NAACL-HLT, 2018. Paper
10. Adam Poliak, Jason Naradowsky, Aparajita Haldar, Rachel Rudinger and Benjamin Van Durme, “Hypothesis Only Baselines in Natural Language Inference,” *SEM, 2018. Paper
11. R. Thomas McCoy, Ellie Pavlick and Tal Linzen, “Right for the Wrong Reasons: Diagnosing Syntactic Heuristics in Natural Language Inference,” ACL, 2019. Paper
12. Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev and Percy Liang, “SQuAD: 100,000+ Questions for Machine Comprehension of Text,” EMNLP, 2016. Paper
13. Pranav Rajpurkar, Robin Jia and Percy Liang, “Know What You Don’t Know: Unanswerable Questions for SQuAD,” ACL, 2018. Paper
14. Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy and Samuel R. Bowman, “GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding,” ICLR, 2019. Paper
15. Alex Wang, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy and Samuel R. Bowman, “SuperGLUE: A Stickier Benchmark for General-Purpose Language Understanding Systems,” NeurIPS, 2019. Paper
16. Marius Mosbach, Maksym Andriushchenko and Dietrich Klakow, “On the Stability of Fine-tuning BERT: Misconceptions, Explanations, and Strong Baselines,” ICLR, 2021. Paper
17. Chuan Guo, Geoff Pleiss, Yu Sun and Kilian Q. Weinberger, “On Calibration of Modern Neural Networks,” ICML, 2017. Paper
18. Shrey Desai and Greg Durrett, “Calibration of Pre-trained Transformers,” EMNLP, 2020. Paper
19. scikit-learn developers, “Classification metrics,” scikit-learn documentation, current online edition. Documentation
20. Nils Reimers and Iryna Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” EMNLP-IJCNLP, 2019. Paper
21. Shervin Minaee, Nal Kalchbrenner, Erik Cambria, Narjes Nikzad, Meysam Chenaghlu and Jianfeng Gao, “Deep Learning–based Text Classification: A Comprehensive Review,” ACM Computing Surveys 54(3), 2021. Survey
22. Wenxuan Zhang, Yue Deng, Bing Liu, Sinno Jialin Pan and Lidong Bing, “Sentiment Analysis in the Era of Large Language Models: A Reality Check,” Findings of NAACL, 2024. Paper
23. Benjamin Warner, Antoine Chaffin, Benjamin Clavié, Orion Weller, Oskar Hallström, Said Taghadouini et al., “Smarter, Better, Faster, Longer: A Modern Bidirectional Encoder for Fast, Memory Efficient, and Long Context Finetuning and Inference,” ACL, 2025. Paper
24. Yupeng Chang et al., “A Survey on Evaluation of Large Language Models,” ACM Transactions on Intelligent Systems and Technology, 2024. Survey
25. Wayne Xin Zhao et al., “A Survey of Large Language Models,” Frontiers of Computer Science, 2026. Updated survey
26. Ilya Loshchilov and Frank Hutter, “Decoupled Weight Decay Regularization,” ICLR, 2019. Paper

NLP 12 • Syntactic and Semantic Tasks

Parse constituents with CKY and dependencies with transitions or spanning trees, label semantic roles, resolve coreference, sketch AMR, and evaluate each structure with its own metric.

Online edition: https://blog.shyandram.dev/en/learn/nlp/nlp-12-syntactic-semantic-tasks/

What you'll learn

- Specify a context-free grammar, recognize and enumerate parses with the CKY chart, and choose among them with a probabilistic grammar or neural span scores.
- Build dependency trees with a transition system (arc-standard) and with graph-based decoding, and explain why independent head choices need a tree constraint.
- Label predicate–argument structure (semantic roles), keep scope and provenance in extracted relations, and read a small AMR graph.
- Link mentions into entities with an antecedent model and preserve ambiguity when context is missing.
- Evaluate each structure with its own metric: PARSEVAL brackets, UAS/LAS, role F1, MUC/B^3/CEAF and CoNLL F1, and Smatch.
- Describe how pretrained encoders and large language models changed these tasks, and what they did not change.

The big picture

Language has a linear surface and a structured interpretation. A sentence arrives as an ordered sequence of words, but readers group those words, identify relations among them, decide who did what to whom, and connect "it" in one sentence to "the pump" in another. "We saw robots with cameras" might describe camera-carrying robots or the equipment used to see them. The word list alone does not choose one interpretation.

> **Explain like I'm five**
>
> A sentence is like a construction kit. The words are pieces, and grammar describes ways to connect them. Parsing shows the connections; role labeling says which piece is the doer and which is the thing being done to; coreference notices when two pieces are really the same toy seen twice. Sometimes the same pieces make two sensible constructions. A good explanation shows both, then asks which fits the situation.

This chapter collects the classic **structured prediction** tasks of NLP. In NLP 06 the output was one label per position. Here the output is a tree, a graph or a partition of mentions, and choices at one place constrain choices elsewhere: a word can have only one head, a span can belong to

only one argument of a predicate, and coreference links must form consistent clusters. McRoy's chapters on syntax, semantics and discourse, with a separate chart-parsing appendix, give the linguistic view [1]; Jurafsky and Martin's chapters on constituency parsing, dependency parsing, semantic roles and coreference give the algorithmic one [2]. The linguistic categories themselves — constituents, dependencies, quantifier scope — are introduced in NLP 02.

These structures are useful for extracting who did what, locating modifiers and building tests that distinguish fluency from structural understanding. They do not settle every question: a grammatical sentence may be false, and a fragmentary message may still communicate clearly. We treat each structure as an explicit representation with a known scope, not as a definition of understanding.

Constituents and context-free grammars

Plain version. Phrase structure groups words into units that can combine recursively. A grammar is a specification of permitted combinations, while a parse is one derivation for a particular sentence.

A context-free grammar can be specified as:

$$G = (\mathcal{N}, \Sigma, \mathcal{R}, S).$$

Here G is the grammar, $\mathcal{N}$ is the set of nonterminal categories, Σ is the terminal vocabulary, $\mathcal{R}$ is the set of rewrite rules and $S \in \mathcal{N}$ is the start category. Nonterminals represent structures such as a noun phrase; terminals are the observed words. A context-free rule replaces one nonterminal without inspecting its surrounding symbols.

For an original toy grammar, use rules such as $S \to NP\ VP$, $VP \to V\ NP$, $PP \to P\ NP$ and $NP \to NP\ PP$. Here S denotes a sentence, NP a noun phrase, VP a verb phrase, PP a prepositional phrase, V a verb category and P a preposition category. These are notation choices for the toy grammar, not a complete grammar of English. Treebanks such as the Penn Treebank supply large collections of sentences annotated with such trees, from which grammars and parsers are learned [4].

Recursive rules generate unbounded families from finite specifications, but the same recursion can make naive search revisit work or loop indefinitely, so a parser needs a search strategy and a compact representation of partial analyses.

Distinguish coverage from preference. A rule system can permit a sentence while assigning several analyses, or fail to parse a perfectly reasonable sentence because a lexical entry is missing. A parsing failure says that the implemented grammar and search did not produce an analysis; it does not establish that the user wrote nonsense. The benefit of an explicit grammar is inspectability: one can show which rule connected spans and where ambiguity arose [1].

Chart parsing by spans: the CKY algorithm

Top-down hypotheses and bottom-up evidence

Plain version. A parser can start from what a sentence is expected to contain, or from what the observed words already support. Both directions need a way to avoid repeating the same work.

A top-down search begins with the start category and expands a production into expected children. It can reject lexical alternatives that do not fit the current expectation, but a poorly chosen expansion can send it down a long unsuccessful path, and a production that expands its first child into the same category can revisit the same unresolved expectation before consuming any input. A bottom-up search begins with observed words and constructs categories that their spans support. It can create many phrases that never contribute to a complete sentence, but the intermediate results can be shared by several successful analyses.

The important design question is therefore not simply which direction is "better," but how partial work is represented and when two partial analyses count as the same search state. A chart stores completed spans so that a state reached by another path need not be recomputed. Memoization alone does not settle ambiguity, though: recognition needs one Boolean per span and category, the preferred parse needs a score and backpointer, and all alternatives need a forest that retains distinct derivation links.

The span recurrence

Plain version. Store what each contiguous span can be. Build longer spans by joining shorter ones and share their results across all complete parses.

The Cocke–Kasami–Younger (CKY) algorithm is the standard dynamic program for this idea [2]. It requires binary rules: a grammar in Chomsky normal form has only rules $A \to B\,C$ and $A \to a$, and any context-free grammar without empty productions can be converted to that form [2]. Use half-open boundaries: the span $[i, j)$ contains tokens at positions i through $j - 1$. Recognition follows:

$$C[i, j, A] = \bigvee_{\substack{A \to BC \in \mathcal{R} \\ i<k<j}} \left(C[i, k, B] \wedge C[k, j, C]\right).$$

Here $C[i, j, A]$ is a Boolean statement that category A covers span $[i, j)$, $\mathcal{R}$ is the binary rule set, $A \to BC$ combines left category B with right category C, k is an interior split boundary, $\wedge$ means both children must exist and $\bigvee$ means at least one rule-and-split combination succeeds. The letter C is used both for the chart notation and for a right-child category; its role is identified by brackets versus a category argument.

Initialize length-one cells from lexical rules. Then iterate over span lengths, start positions, split positions and compatible rules. Because children are shorter than their parent, they have already been computed. A unary rule or empty production needs additional handling; this lesson's executable grammar deliberately excludes both.

This algorithm recognizes an input, but recognition alone cannot explain its trees. Store successful combinations as backpointers or a packed forest. Several parents can share a child span rather

than duplicating its full tree. Expanding all complete trees can still cost exponential time when a sentence is highly ambiguous, even if recognition is polynomial. For fixed binary grammar rules, the nested span-and-split loops have cubic dependence on token length; the chart has quadratically many spans, each with possible categories.

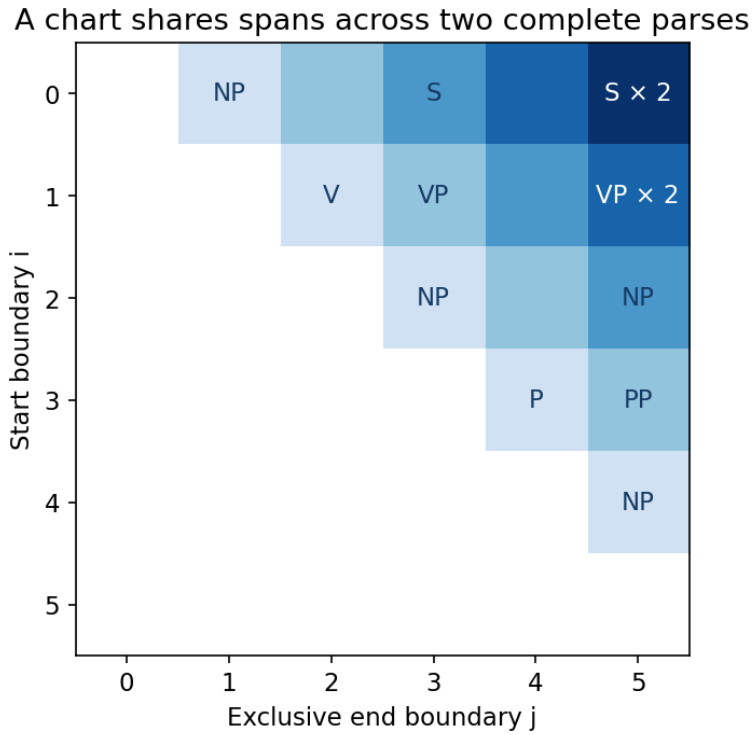


Figure 1 — A chart stores reusable spans. The complete sentence cell can contain more than one analysis.

A parser that preserves two analyses

When the grammar permits two attachments, report the alternatives. A parser should not discard ambiguity merely because one option was inserted first. The following standalone parser stores complete small trees in chart cells. This is intentionally simpler than a packed-forest implementation; it suits the five-word demonstration, while the preceding paragraphs explain why explicit enumeration is unsuitable for large ambiguous inputs.

```
from collections import defaultdict

words = "we saw robots with cameras".split()
lexical = {
    "we": ["NP"], "saw": ["V"], "robots": ["NP"],
    "with": ["P"], "cameras": ["NP"],
}
rules = [
    ("S", "NP", "VP"),
```

```
    ("VP", "V", "NP"),
    ("VP", "VP", "PP"),
    ("NP", "NP", "PP"),
    ("PP", "P", "NP"),
]
chart = defaultdict(lambda: defaultdict(list))
for i, word in enumerate(words):
    for category in lexical.get(word, []):
        chart[i, i + 1][category].append((category, word))
for length in range(2, len(words) + 1):
    for start in range(len(words) - length + 1):
        end = start + length
        for split in range(start + 1, end):
            for parent, left, right in rules:
                for left_tree in chart[start, split].get(left, []):
                    for right_tree in chart[split, end].get(right, []):
                        tree = (parent, left_tree, right_tree)
                        if tree not in chart[start, end][parent]:
                            chart[start, end][parent].append(tree)

def bracket(tree):
    if len(tree) == 2:
        return f"({tree[0]} {tree[1]})"
    return f"({tree[0]} {bracket(tree[1])} {bracket(tree[2])})"

parses = chart[0, len(words)].get("S", [])
assert len(parses) == 2
for tree in parses:
    print(bracket(tree))
print("complete parses:", len(parses))
```

```
(S (NP we) (VP (V saw) (NP (NP robots) (PP (P with) (NP cameras)))))
(S (NP we) (VP (VP (V saw) (NP robots)) (PP (P with) (NP cameras))))
complete parses: 2
```

In one tree, the prepositional phrase belongs inside the object noun phrase. In the other, it modifies the verb phrase. Both use every word in order and satisfy the grammar. Choosing between them may require the surrounding scene, prior discourse or lexical preferences beyond these rules.

Add an unknown word and the chart may become empty at a lexical cell. The correct response is a coverage diagnosis. A production service might use a fallback tagger, allow an explicit unknown category or ask for clarification, but each fallback changes the recognized language. It should be documented and tested.

Probabilities over structures

Plain version. A grammar says which analyses exist. A probability model can prefer some analyses and summarize uncertainty across them.

A probabilistic context-free grammar (PCFG) assigns a probability to each production, normalized among rules with the same parent. A derivation has:

$$p(\mathcal{T}) = \prod_{r \in \mathcal{T}} p(r), \qquad \hat{\mathcal{T}} = \operatorname*{argmax}_{\mathcal{T} \in \mathcal{D}(\mathbf{x})} p(\mathcal{T}).$$

Here $\mathcal{T}$ is a complete derivation tree, r indexes each production occurrence in that tree, $p(r)$ is its production probability, $\mathcal{D}(\mathbf{x})$ is the set of permitted parses whose terminal yield is the input $\mathbf{x}$, and $\hat{\mathcal{T}}$ is the highest-probability parse. The product includes repeated uses of a rule separately.

Replacing chart alternatives with sums computes total probability mass over derivations (the inside algorithm). Replacing them with maxima computes the best derivation (probabilistic CKY). These answer different questions: a sentence with many moderately probable parses can have greater total probability than a sentence with one strong parse, and the best tree need not explain most of the posterior mass. The following snippet computes both for the same five words.

```
import math
from collections import defaultdict

words = "we saw robots with cameras".split()
lexical = {"we": "NP", "saw": "V", "robots": "NP", "with": "P", "cameras": "NP"}

def parse(p_np_pp):
    # Binary-rule probabilities. Both trees use the same five lexical rules,
    # so their probabilities are a shared factor and are omitted (set to 1).
    rules = {("S", "NP", "VP"): 1.0, ("PP", "P", "NP"): 1.0,
             ("VP", "V", "NP"): 0.6, ("VP", "VP", "PP"): 0.4,
             ("NP", "NP", "PP"): p_np_pp}
    best = defaultdict(dict)    # (i, j) -> {category: (log prob, tree)}
    inside = defaultdict(lambda: defaultdict(float))
    for i, w in enumerate(words):
        best[i, i + 1][lexical[w]] = (0.0, f"({lexical[w]} {w})")
        inside[i, i + 1][lexical[w]] = 1.0
    n = len(words)
    for length in range(2, n + 1):
        for i in range(n - length + 1):
            j = i + length
            for k in range(i + 1, j):
                for (a, b, c), p in rules.items():
                    if b in best[i, k] and c in best[k, j]:
                        score = math.log(p) + best[i, k][b][0] + best[k, j][c][0]
                        if score > best[i, j].get(a, (-math.inf,))[0]:
                            best[i, j][a] = (score, f"({a} {best[i, k][b][1]} {best[k,
                             ↪  j][c][1]})")
                        inside[i, j][a] += p * inside[i, k][b] * inside[k, j][c]
    score, tree = best[0, n]["S"]
    return tree, math.exp(score), inside[0, n]["S"]

for p in (0.2, 0.5):
    tree, p_best, p_total = parse(p)
    print(f"P(NP -> NP PP) = {p}: best = {p_best:.3f}, total = {p_total:.3f}")
```

```
    print("  ", tree)
```

```
P(NP -> NP PP) = 0.2: best = 0.240, total = 0.360
   (S (NP we) (VP (VP (V saw) (NP robots)) (PP (P with) (NP cameras))))
P(NP -> NP PP) = 0.5: best = 0.300, total = 0.540
   (S (NP we) (VP (V saw) (NP (NP robots) (PP (P with) (NP cameras)))))
```

The verb attachment has probability $0.4 \times 0.6 = 0.24$ regardless of the noun rule; the noun attachment has $0.6 \times p(NP \rightarrow NP\,PP)$. The preferred tree flips when that rule probability crosses 0.4, while the inside total always counts both trees. Notice what decides the attachment: a rule probability that knows nothing about `saw`, `robots` or `cameras`. That is the weakness the next section's neural scorers address.

Rule probabilities estimated from a treebank depend on the annotation convention and domain. A news-style treebank and a support-message corpus can have different fragment patterns and attachment preferences. Probabilistic modeling supplies a preference mechanism, but it does not remove domain dependence.

Neural constituency parsing: score spans, decode with a chart

Plain version. Instead of multiplying rule probabilities, let a neural encoder score every span and label, then use the same chart to find the highest-scoring tree.

A span-based parser assigns each candidate constituent a score and searches for the tree with the largest total:

$$s(\mathcal{T}) = \sum_{(i,j,\ell)\in\mathcal{T}} s(i,j,\ell), \qquad \hat{\mathcal{T}} = \operatorname*{argmax}_{\mathcal{T}} s(\mathcal{T}).$$

Here $\mathcal{T}$ is a tree viewed as a set of labeled spans (i,j,ℓ) with label ℓ over the half-open span $[i,j)$, $s(i,j,\ell)$ is a score computed from contextual vectors at the span boundaries, and $\hat{\mathcal{T}}$ is the decoded tree. Stern, Andreas and Klein showed that independent scoring of labels and spans is compatible with classical chart dynamic programming and also admits a greedy top-down decoder that recursively splits the input [5]. Kitaev and Klein replaced the LSTM encoder in that design with self-attention, found that separating positional and content information improved accuracy, and reported 93.55 F1 on the Penn Treebank without external data and 95.13 F1 with pretrained word representations [6]. A follow-up added multilingual pretraining [7].

The output remains a tree even if its scores come from a large neural network: the chart guarantees well-formedness, and the encoder supplies lexical evidence that a PCFG's rule probabilities could not.

Dependency structure and grammatical roles

Plain version. Dependency analysis links words directly. It highlights which word depends on which other word and labels their grammatical relationship.

A basic dependency tree can be represented by head indices:

$$h_t \in \{0, 1, \dots, T\} \setminus \{t\}, \qquad r_t \in \mathcal{D}.$$

Here h_t is the head index of word t, 0 denotes an artificial root, T is sentence length, r_t is the relation label for that word and $\mathcal{D}$ is the relation inventory. A valid basic tree additionally requires a specified root convention, connectivity and absence of directed cycles.

Independent head choices can violate those global requirements. Two words might select each other, forming a cycle. Several disconnected components might attach without a unique sentence root. A tree decoder must enforce the intended structure, or the service must validate and reject an invalid analysis.

Universal Dependencies provides a cross-linguistic annotation framework for morphology, part of speech and grammatical dependencies [8]. Its categories aim for useful comparability across languages while retaining language-specific distinctions. This is an annotation framework, not a claim that every language has identical word order or overt grammatical markers.

Phrase structure and dependencies emphasize different information. A constituent tree makes span grouping explicit. A dependency tree makes lexical heads and relations explicit. Converting between them requires conventions for heads, punctuation and function words; the conversion is not merely a drawing change.

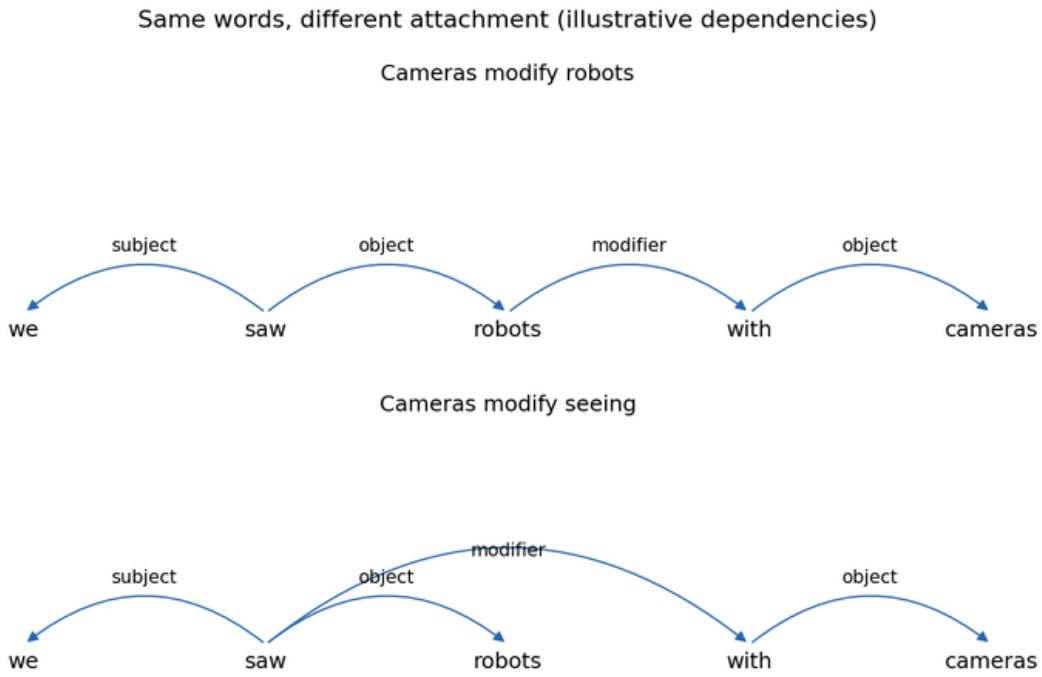


Figure 2 — Two attachments of the same words. These illustrative grammatical links explain the ambiguity and are not copied from a treebank.

For extraction, grammatical roles are useful evidence but not interchangeable with semantic roles. A passive sentence changes grammatical subject placement while preserving who acted on whom. The semantic-role section below develops that distinction and explains why a dependency arrow does not by itself establish a real-world relation.

Attachment ambiguity reveals a model's inductive bias

The two camera attachments share their words but assign different participants to the seeing event. Noun-phrase attachment describes a property of the robots. Verb-phrase attachment describes an instrument or circumstance of seeing. A downstream extractor that commits to the noun interpretation might create a robot–camera possession record that the instrument interpretation does not support. An attachment difference is consequential because of the relation it changes, not simply because the brackets look different.

A grammar can permit both while a probability model prefers one, as the PCFG snippet showed. With only parent-category rule probabilities, the model cannot condition that preference on the lexical identity of `saw`, `robots` and `cameras` unless its state or scoring representation is extended. This is a capacity limitation. A wider search over the same score cannot create missing lexical evidence.

The opposite failure is a search limitation: useful scores may exist, but pruning removes the correct span before a complete tree is formed. Diagnose these separately by asking whether the reference structure is representable, whether it survives candidate construction, and whether it receives the highest score among survivors. Each answer suggests a different intervention: extend the representation, revise pruning, or improve scoring.

There is also a genuine information limitation: in an isolated sentence both readings may remain possible, and a gold tree records only the corpus's chosen interpretation. For an extractor, retaining both alternatives until the required relation is checked can be preferable to an early commitment.

Transition-based dependency parsing

Plain version. Read the sentence left to right with a stack, and at each step choose one of a few actions: shift the next word onto the stack, or attach the top two stack items to each other. A classifier picks the action; the sequence of actions builds the tree.

Nivre formalizes deterministic dependency parsers as transition systems and analyzes stack-based variants (arc-standard and arc-eager, restricted to projective trees) and list-based variants, one of which handles non-projective trees [9]. A **projective** tree is one whose arcs can be drawn above the sentence without crossing. In the arc-standard system used by Chen and Manning, a configuration $c = (\sigma, \beta, A)$ has a stack σ, a buffer β and an arc set A, and three transitions are available [10]:

$$\begin{aligned}
\text{SHIFT}: &\ (\sigma,\ w|\beta,\ A) \Rightarrow (\sigma|w,\ \beta,\ A),\\
\text{LEFT-ARC}_r: &\ (\sigma|s_2|s_1,\ \beta,\ A) \Rightarrow (\sigma|s_1,\ \beta,\ A \cup \{s_1 \xrightarrow{r} s_2\}),\\
\text{RIGHT-ARC}_r: &\ (\sigma|s_2|s_1,\ \beta,\ A) \Rightarrow (\sigma|s_2,\ \beta,\ A \cup \{s_2 \xrightarrow{r} s_1\}).
\end{aligned}$$

Here s_1 is the top of the stack and s_2 the item below it, w is the first buffer word, $\sigma|x$ means stack σ with x on top, $h \xrightarrow{r} d$ is an arc from head h to dependent d with label r, and $\Rightarrow$ reads "becomes." Parsing starts with only the root on the stack and all words in the buffer, and ends when the buffer is empty and only the root remains. Each word is shifted once and reduced once, so a sentence of n words needs exactly $2n$ transitions.

To train the classifier, convert each gold tree into the transition sequence that produces it — a **static oracle**. The rule below is one standard choice: attach left when the second stack item's gold head is the top; attach right when the top's gold head is the second item *and* the top has already collected all of its own dependents; otherwise shift.

```
words = ["ROOT", "we", "saw", "robots", "with", "cameras"]
# Gold heads for the instrument reading: "with cameras" attaches to "saw".
gold = {1: 2, 2: 0, 3: 2, 4: 5, 5: 2}

def oracle(gold):
    stack, buffer, arcs, steps = [0], list(range(1, len(words))), {}, []
    def done(h):  # has h already collected all of its dependents?
        return all(d in arcs for d, head in gold.items() if head == h)
    while buffer or len(stack) > 1:
        if len(stack) >= 2 and gold.get(stack[-2]) == stack[-1]:
            dependent = stack.pop(-2)
            arcs[dependent] = stack[-1]; steps.append("LEFT-ARC")
        elif len(stack) >= 2 and gold[stack[-1]] == stack[-2] and done(stack[-1]):
            dependent = stack.pop()
            arcs[dependent] = stack[-1]; steps.append("RIGHT-ARC")
        else:
            stack.append(buffer.pop(0)); steps.append("SHIFT")
    return steps, arcs

steps, arcs = oracle(gold)
print(" ".join(steps))
print({words[d]: words[h] for d, h in sorted(arcs.items())})
assert arcs == gold and len(steps) == 2 * (len(words) - 1)
```

```
SHIFT SHIFT LEFT-ARC SHIFT RIGHT-ARC SHIFT SHIFT LEFT-ARC RIGHT-ARC RIGHT-ARC
{'we': 'saw', 'saw': 'ROOT', 'robots': 'saw', 'with': 'cameras', 'cameras': 'saw'}
```

Ten transitions for five words, as predicted. The `done` check matters: if `robots` were attached to `saw` and popped before `cameras` arrived, a noun-attachment tree could never be built afterwards. The order of reductions encodes the tree.

At parse time a classifier replaces the oracle. Chen and Manning fed embeddings of words, part-of-speech tags and arc labels from fixed stack and buffer positions into a one-hidden-layer network

with a cube activation and a softmax over transitions; their greedy parser is fast but, as they note, exposed to error propagation because an early wrong action cannot be undone [10]. Kiperwasser and Goldberg instead computed one bidirectional-LSTM vector per token, trained jointly with the parser, and built features by concatenating a few of them; the same representation worked for a greedy transition parser and a globally optimized graph parser [12]. Transition parsers run in time linear in sentence length; the price is a local, left-to-right decision procedure.

Graph-based dependency parsing

Plain version. Score every possible head–dependent arc at once, then choose the highest-scoring set of arcs that forms a tree.

McDonald et al. formalize weighted dependency parsing as finding a maximum spanning tree in a directed graph whose nodes are the words plus a root [11]:

$$\hat{\mathbf{h}} = \operatorname*{argmax}_{\mathbf{h} \in \mathcal{Y}(\mathbf{x})} \sum_{t=1}^{T} s(h_t, t).$$

Here $\mathbf{h} = (h_1, \dots, h_T)$ assigns a head to each word, $\mathcal{Y}(\mathbf{x})$ is the set of head assignments that form a valid tree for sentence $\mathbf{x}$, and $s(h, t)$ is the score of the arc from head h to dependent t. Over projective trees, Eisner's algorithm solves this in $O(n^3)$ time; over all trees, including non-projective ones, the Chu–Liu–Edmonds maximum-spanning-tree algorithm solves it in $O(n^2)$ [11].

Dozat and Manning made the arc scorer **biaffine** on top of a deep bidirectional encoder [13]:

$$s(h, t) = \mathbf{r}_t^{(\text{dep})\top} U \, \mathbf{r}_h^{(\text{head})} + \mathbf{r}_h^{(\text{head})\top} \mathbf{u}.$$

Here $\mathbf{r}_t^{(\text{dep})}$ and $\mathbf{r}_h^{(\text{head})}$ are lower-dimensional projections of the contextual vectors of the dependent and the candidate head, U is a learned matrix scoring their interaction, and $\mathbf{u}$ is a learned vector giving each word a prior tendency to be a head. A second biaffine classifier predicts the relation label for the chosen arc. The scorer produces a full $(T + 1) \times T$ matrix of arc scores in one pass.

What does the tree constraint buy? The snippet below uses invented scores for our sentence, picks each word's best head independently, and then enumerates every single-root tree — feasible only for tiny inputs, but it makes the decoder's job visible. It finishes with UAS and LAS, defined in the next section.

```
import itertools
import numpy as np

words = ["ROOT", "we", "saw", "robots", "with", "cameras"]
n = len(words) - 1
# score[h, d]: invented arc scores (head h -> dependent d), not a trained model.
score = np.full((n + 1, n + 1), -2.0)
score[0, 2], score[2, 1], score[2, 3], score[5, 4] = 3.0, 2.0, 2.0, 2.0
score[3, 5], score[2, 5] = 1.5, 1.2               # robots vs saw for "cameras"
```

```
score[4, 5] = 1.8                                   # tempting but creates a cycle
np.fill_diagonal(score, -np.inf)

def is_tree(heads):                                 # heads[d-1] is the head of d
    if sum(h == 0 for h in heads) != 1:
        return False
    for d in range(1, n + 1):
        seen, node = set(), d
        while node != 0:
            if node in seen:
                return False
            seen.add(node); node = heads[node - 1]
    return True

greedy = [int(np.argmax(score[:, d])) for d in range(1, n + 1)]
total = lambda hs: sum(score[h, d] for d, h in enumerate(hs, start=1))
trees = [hs for hs in itertools.product(range(n + 1), repeat=n)
         if all(h != d for d, h in enumerate(hs, 1)) and is_tree(hs)]
best = max(trees, key=total)
print("greedy heads:", greedy, "valid tree:", is_tree(greedy))
print("best tree:   ", list(best), "score:", round(total(best), 2))
print("trees enumerated:", len(trees))

gold = [2, 0, 2, 5, 2]
labels_gold = ["nsubj", "root", "obj", "case", "obl"]
labels_pred = ["nsubj", "root", "obj", "mark", "nmod"]
uas = np.mean([p == g for p, g in zip(best, gold)])
las = np.mean([p == g and lp == lg for p, g, lp, lg
               in zip(best, gold, labels_pred, labels_gold)])
print(f"UAS = {uas:.2f}, LAS = {las:.2f}")
```

```
greedy heads: [2, 0, 2, 5, 4] valid tree: False
best tree:    [2, 0, 2, 5, 3] score: 10.5
trees enumerated: 625
UAS = 0.80, LAS = 0.60
```

Greedy selection makes `with` and `cameras` heads of each other, a two-word cycle disconnected from the root (Figure 3). Among the 625 single-root trees over five words, the best one breaks the cycle and attaches `cameras` to `robots` — a valid tree, but the noun reading, while the gold annotation chose the instrument reading. A maximum-spanning-tree algorithm finds the same answer without enumeration.

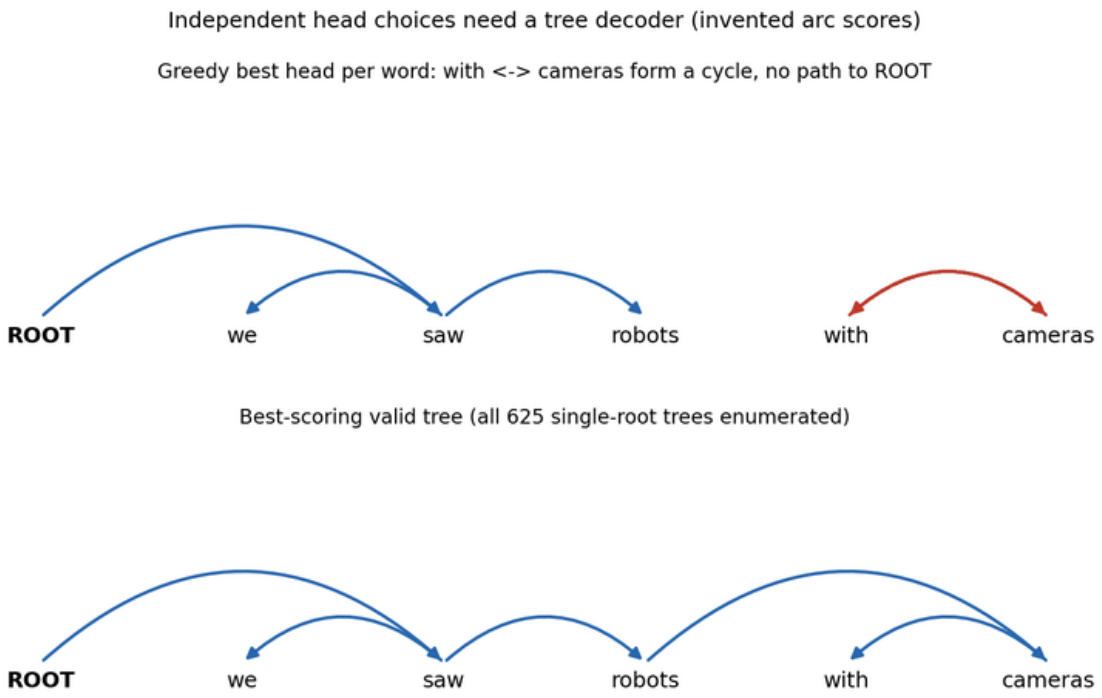


Figure 3 — Independent best heads can form a cycle; a tree decoder restores a valid structure. Scores are invented and match the snippet.

Graph-based and transition-based parsers make different trade-offs: global search over arc scores with quadratic or cubic decoding, versus linear-time local decisions. Kiperwasser and Goldberg's shared BiLSTM features served both families [12]. Whichever family is used, the structural guarantee comes from the decoder or transition system, not from the encoder.

Evaluating parsers: PARSEVAL, UAS and LAS

Plain version. Compare the representation that the parser promises to produce, and inspect mistakes where tokenization, annotation or context changes the answer.

The original PARSEVAL procedure compared a candidate parse with a treebank parse after removing material such as punctuation, and computed a **crossing-brackets** score — how often the candidate brackets a span in a way that crosses a reference bracket — and **recall**, the number of shared bracket pairs divided by the number in the reference [3]. Today's standard is labeled bracket precision, recall and F1 [2]:

$$P_{\text{lab}} = \frac{|\mathcal{B}(\hat{\mathcal{T}}) \cap \mathcal{B}(\mathcal{T}^*)|}{|\mathcal{B}(\hat{\mathcal{T}})|}, \qquad R_{\text{lab}} = \frac{|\mathcal{B}(\hat{\mathcal{T}}) \cap \mathcal{B}(\mathcal{T}^*)|}{|\mathcal{B}(\mathcal{T}^*)|}.$$

Here $\mathcal{B}(\mathcal{T})$ is the set of labeled constituents (i, j, ℓ) of tree $\mathcal{T}$, $\hat{\mathcal{T}}$ is the predicted tree, $\mathcal{T}^*$ is the

reference tree, and F1 is the harmonic mean of the two. For our two parses, the trees share every labeled bracket except one: the noun-attachment tree has $(2, 5, NP)$, the verb-attachment tree has $(1, 3, VP)$. Counting preterminals, each tree has nine labeled constituents, so either tree scored against the other gets precision and recall $8/9$. State whether lexical preterminals, punctuation and the root are included; different inclusion rules change the denominator.

For dependencies, the **unlabeled attachment score** (UAS) is the fraction of words with the correct head, and the **labeled attachment score** (LAS) additionally requires the correct relation:

$$\mathrm{LAS} = \frac{1}{M} \sum_{t \in \mathcal{J}} \mathbf{1}[(\hat{h}_t, \hat{r}_t) = (h_t, r_t)].$$

Here LAS is labeled attachment score, $\mathcal{J}$ is the set of evaluated word positions, $M = |\mathcal{J}|$ is its size, $\hat{h}_t$ and $\hat{r}_t$ are predicted head and relation, h_t and r_t are reference head and relation, and $\mathbf{1}[\cdot]$ is the indicator function; UAS drops the relation condition. In the snippet, `cameras` has the wrong head (both scores lose it) and `with` has the right head with the wrong label (only LAS loses it), giving 0.80 and 0.60.

Token alignment must be fixed before applying these formulas. The CoNLL 2018 shared task asked systems to parse raw text into Universal Dependencies for many languages, without gold-standard annotation on the test input [14]. Systems therefore segment and tokenize for themselves, and predicted words must be aligned with gold words before attachment scores mean anything. If the gold annotation has two words where the tokenizer emits one, an attachment comparison cannot be interpreted normally. Report the segmentation mismatch and specify how alignment is handled. Do not silently delete difficult examples.

Build small diagnostic pairs. Move a modifier while keeping vocabulary similar; add a nested clause; replace an active construction with a passive one; include a fragment or a code-switched phrase. These tests reveal the conditions under which a model’s structure changes, even when an aggregate score appears stable. Avoid treating one gold tree as certainty when the sentence is genuinely ambiguous. A corpus needs a chosen convention or adjudicated analysis, but another analysis may be linguistically reasonable. For an application, ask whether the disagreement changes the extracted fact or action. That connects a parsing error to its practical consequence.

Semantic role labeling: who did what to whom

Plain version. The same event can be expressed with different grammar. Extract the roles in the event instead of assuming the first name is always the actor.

For a simple two-participant predicate, write:

$$\mathrm{repair}(\mathrm{Mina}, \mathrm{sensor}).$$

Here repair names the relation; the first argument is the repairer; and the second argument is the repaired object. “Mina repaired the sensor” and “The sensor was repaired by Mina” can share this representation. Their surface subjects differ. Grammar supplies clues, but a role interpretation must also account for the predicate and construction [1].

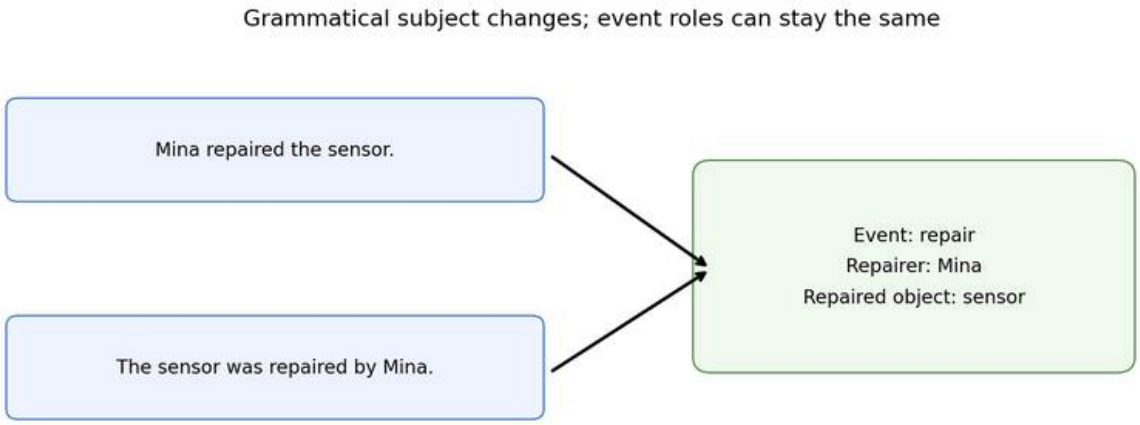


Figure 4 — Surface subjects can change while event roles stay the same. The examples and record are synthetic.

Not every verb permits every alternation, and not every argument is overt. "The sensor was repaired" leaves the repairer unspecified. A system that fills in the nearest previously mentioned person has made a contextual inference, not a direct extraction. That inference may be reasonable, but it needs a separate confidence policy and evidence contract. Preserve an explicit unknown role when the application does not authorize that inference.

Arguments and modifiers also differ. In "Mina repaired the sensor in Taipei on Tuesday," the repairer and repaired object participate in the core event; location and time modify it. An event schema may include all four fields, but it should say whether each is required. Optional fields prevent an extractor from inventing details merely to satisfy an overly strict schema.

PropBank roles and the SRL task

Gildea and Jurafsky framed automatic semantic role labeling as a statistical task over parsed sentences: match role-bearing phrases to parse constituents, extract features from the words and the tree, and estimate a probability for each possible role [15]. **PropBank** then annotated the verbs of the Penn Treebank with predicate-specific numbered arguments: each verb sense has a frameset in which `Arg0` is typically the agent-like participant and `Arg1` the patient or theme, while higher numbers are defined per verb; adjuncts receive `ArgM` labels such as `ArgM-NEG` for negation and `ArgM-MOD` for modals [16]. The numbering is the point: in "an amendment offered by Rep. X," the amendment is still `Arg1` of *offer* although it is the grammatical subject [16]. The CoNLL-2005 shared task standardized SRL evaluation on PropBank: a predicted argument counts as correct only if its span and role both match, and systems are compared by precision, recall and F1 over arguments [17].

Semantic roles provide a bridge across constructions, not a guarantee of universal agreement. Different resources can label the same participant with predicate-specific or more general roles.

Before combining training corpora, compare their definitions and examples. A mechanical label rename can conceal differences about instruments, causes, beneficiaries and implicit participants.

SRL as predicate-conditioned tagging

He et al. cast SRL as BIO sequence tagging, once per predicate: the input marks which word is the predicate, and a deep highway BiLSTM with **constrained decoding** — forbidding, for example, an `I-` tag without a preceding `B-` — labels argument spans [18]. Their eight-layer ensemble reported 83.2 F1 on the CoNLL-2005 test set and found that deep models recover long-distance dependencies but still make surprisingly obvious errors, leaving room for syntactic information to help [18]. With a pretrained encoder, the same design becomes the token-labeling head from NLP 11 plus a predicate indicator. The snippet converts role spans to tag sequences and back, and rejects an invalid sequence.

```
tokens = "Mina said the sensor was repaired by Lena".split()
# One frame per predicate: (start, end) token spans, end exclusive.
frames = {
    "said":     {"ARG0": (0, 1), "ARG1": (2, 8)},
    "repaired": {"ARG1": (2, 4), "ARG0": (6, 8)},
}

def to_bio(frame, n):
    tags = ["O"] * n
    for role, (i, j) in frame.items():
        tags[i] = "B-" + role
        tags[i + 1:j] = ["I-" + role] * (j - i - 1)
    return tags

def from_bio(tags):
    spans, start, role = {}, None, None
    for k, tag in enumerate(tags + ["O"]):
        if start is not None and not tag.startswith("I-" + role):
            spans[role] = (start, k); start = None
        if tag.startswith("B-"):
            start, role = k, tag[2:]
        elif tag.startswith("I-") and start is None:
            raise ValueError(f"I- tag without B- at position {k}")
    return spans

for predicate, frame in frames.items():
    tags = to_bio(frame, len(tokens))
    tags[tokens.index(predicate)] = "B-V"
    print(f"{predicate:9s}", " ".join(tags))
    assert {r: s for r, s in from_bio(tags).items() if r != "V"} == frame
try:
    from_bio(["O", "I-ARG1", "I-ARG1"])
except ValueError as error:
    print("invalid:", error)
```

```
said      B-ARG0 B-V B-ARG1 I-ARG1 I-ARG1 I-ARG1 I-ARG1 I-ARG1
repaired  O O B-ARG1 I-ARG1 O B-V B-ARG0 I-ARG0
invalid: I- tag without B- at position 1
```

The same sentence yields two frames. For *said*, Mina is the speaker and the whole reported clause is `ARG1`. For *repaired*, Lena is the repairer and the sensor is `ARG1` although it is the grammatical subject. An extractor that reads only the *repaired* frame would record a completed repair as fact; the *said* frame shows that the repair is Mina's report. That is exactly the attribution distinction the relation records below must keep.

Scope in semantic representations

Plain version. Small words such as "every," "some" and "not" can change a statement more than the names do. A semantic parser must record what they modify, or its output answers the wrong question.

NLP 02 introduces quantifier scope and negation with a worked countermodel. Here we look at what scope means for a task output. "Every technician checked a sensor" has a reading in which each technician may check a different sensor and a reading in which one sensor was checked by all. "Mina did not check every sensor" has the form

$$\neg \forall z\,[S(z) \Rightarrow C(m,z)] \quad \equiv \quad \exists z\,[S(z) \wedge \neg C(m,z)].$$

Here z ranges over entities, $S(z)$ means sensor, $C(m,z)$ means Mina checked that entity, m identifies Mina, $\neg$ means negation and $\equiv$ means logical equivalence. A counterexample sensor suffices to make the universal false. "Mina checked no sensor" instead rules out every successful check. Negation's position determines which evidence can refute the statement.

A document database usually lacks complete knowledge: no extracted inspection record may mean unreported, not false. This difference matters when answering a universal question. A list of observed inspections can support "these sensors were checked," while establishing "every sensor was checked" also requires the full relevant sensor inventory and complete coverage. Conversely, finding one explicitly unchecked sensor is enough to refute completeness. Scope reasoning and evidence completeness must be considered together. Likewise, "No evidence of failure was found" describes an observation or search result; it should not automatically become "failure is impossible."

In a restricted extractor, unsupported scope constructions should trigger abstention or review. A rule that handles "did not repair" can still misread "did not say that Mina repaired": the system must determine what is negated, which is exactly what PropBank's `ArgM-NEG` attaches to a specific predicate [16].

Composition is a calculation over roles and scope

A useful way to understand composition is to ask what remains incomplete at each phrase. `Min a` identifies a participant. `repaired` requires participants in specified roles. `repaired a sensor` fixes the object requirement through an existential condition and leaves the actor to be supplied. Combining the subject then closes that requirement. This order of combination explains why identifying the same three content words is insufficient: their roles must be bound correctly [1].

A restricted logical reading of "Mina repaired a sensor" is:

$$\exists z\,[S(z) \wedge R(m,z)].$$

Here z ranges over possible entities, $S(z)$ means that z is a sensor, $R(m,z)$ means that participant m repaired z, and m identifies Mina. The existential quantifier states that at least one suitable entity exists; it does not identify which sensor or imply that every sensor was repaired. This simple reading omits tense and event time intentionally. Those are additional semantic fields, not hidden facts supplied by the notation.

"Only Mina repaired the sensor" adds a restriction on alternatives. "Mina tried to repair the sensor" describes an attempt without guaranteeing a completed repair. "Mina stopped repairing the sensor" changes the event's temporal interpretation. A record containing only actor and object cannot express those distinctions. This gives a concrete criterion for choosing a representation: list the questions that must be answered, then check whether two source statements requiring different answers collapse into the same record. If both an attempted repair and a completed repair become `repair(Mina, sensor)`, the schema is too coarse for a completion query. A richer encoder cannot prevent that loss after serialization.

Relations, events and provenance

Plain version. A relation links entities; an event organizes participants around an occurrence. Both need evidence, and both can be wrong even when every entity span is right.

A binary relation record can be written:

$$q = (r, h, t, s).$$

Here q is the record; r is the relation type; h is the head entity or mention; t is the tail entity or mention; and s identifies supporting evidence. The letters head and tail define an ordering convention, not a linguistic claim. For an employment relation, reversing the employee and employer changes the record. Define direction in the schema and evaluate it.

Event extraction adds a trigger or event type with multiple roles, such as buyer, seller, item, amount and time — semantic role labeling restricted to an application's schema. A transaction sentence may mention several prices, only one of which is the actual amount paid. Jointly selecting role assignments can prevent some contradictions, but it cannot resolve an ambiguous source without additional context. Preserve alternatives or route ambiguity to review when the application permits it.

Provenance should include document identity, source version and supporting span. Without a version, offsets can point into a later edited document; without attribution — the *said* frame above — a quoted allegation becomes an asserted fact. These are properties of the record pipeline, not failures that a larger encoder automatically fixes.

Decompose evaluation by stage — mention detection, category prediction, relation selection and the final supported record — and evaluate relations against predicted mentions as well as gold mentions: perfect input mentions during evaluation can conceal the failure rate of the deployed pipeline.

Discourse, mentions and coreference

Plain version. Meaning continues across sentences. Coreference resolution decides which mentions refer to the same entity, so that facts about "the pump" and "it" end up on the same record.

Discourse is language organized across clauses or sentences. Consider "The pump warmed. The operator stopped it." The two sentences introduce an event, an agent and a reference to an object. They may suggest a response to warming, but they do not necessarily prove that warming caused the stop. Compare "The pump stopped **because** the valve closed," "The pump stopped **after** the valve closed," and "The pump stopped **although** the valve remained open." The first presents a causal explanation, the second orders events without establishing causation, and the third marks a contrast with an expectation [1]. A summarizer or extractor that inserts "because" has added a claim. Relations also have scope: in "Mina said the pump stopped because the valve closed," the explanation may be part of Mina's reported account rather than an independently established finding.

A mention is a text span that refers to an entity, event or concept. An antecedent is an earlier expression that helps interpret another mention. Coreference groups mentions that refer to the same entity. Similar wording does not necessarily imply identity: two different pumps may both be called "the backup pump."

An antecedent-ranking model

A span-based coreference model can score possible antecedents for a mention, as illustrated by Lee et al. [19]. Abstractly,

$$p(a_i = j \mid d) = \frac{\exp s(i, j, d)}{\sum_{k \in A_i} \exp s(i, k, d)}.$$

Here d is the document or conversation; i indexes the current mention; A_i is the set of candidate antecedents, including a null option; a_i is the selected antecedent variable; and $s(i, j, d)$ is a learned compatibility score. The null option permits a mention to introduce a new entity or remain unlinked. Candidate pruning changes what the model can consider. Lee et al.'s model separates the contributions [19]:

$$s(i, j) = s_m(i) + s_m(j) + s_a(i, j) \quad (j \neq \epsilon), \qquad s(i, \epsilon) = 0.$$

Here i is the current candidate span, j is an earlier candidate span, ϵ is the null antecedent, s_m scores whether a span is a mention, and s_a scores pair compatibility. Fixing the null score to zero gives the real links a common reference. Mention pruning saves pairwise computation but can remove the only valid antecedent before linking begins.

The model must detect mentions as well as link them. If an earlier entity span was discarded, a later linking decision cannot recover it merely by assigning better scores among remaining candidates. Separate detection recall from linking accuracy when diagnosing failures.

Clusters complicate supervision. If `Pump A`, `the pump` and `it` refer to one entity, the final mention can correctly link to either earlier mention. A loss that arbitrarily rewards only the nearest one penalizes another valid link. A cluster-based training target instead rewards their combined probability:

$$\mathcal{L}_i = -\log \sum_{j \in G_i} p(a_i = j \mid d).$$

Here $\mathcal{L}_i$ is the loss for candidate span i, G_i is its nonempty set of valid retained antecedents under the annotation and pruning policy, a_i is the antecedent variable, d is the document, and $p(a_i = j \mid d)$ is its conditional antecedent probability. The null option belongs in G_i when the span has no valid earlier antecedent. This is marginalization over valid links, as used in the original model [19]. At inference, connecting mentions through antecedent links forms entity groups. A single erroneous bridge can merge two otherwise coherent groups, and the wrongly merged cluster can later assign several events to the wrong pump.

A recency baseline and its failure

The following hand-written baseline uses explicit compatibility and recency. It is deliberately small enough to expose a failure. Device categories are supplied as data; the code does not infer attributes from personal names or social stereotypes.

```
entities = [
    {"id": "pump_a", "category": "device", "turn": 1},
    {"id": "valve_b", "category": "device", "turn": 2},
    {"id": "operator", "category": "person", "turn": 2},
]
def candidates(category):
    return [e for e in entities if e["category"] == category]
def recent(category):
    choices = candidates(category)
    return max(choices, key=lambda e: e["turn"])["id"] if choices else None
print("Recency baseline for it:", recent("device"))
assert recent("device") == "valve_b"
# In a synthetic annotation, the speaker meant the earlier pump.
intended = "pump_a"
assert recent("device") != intended
# A cautious application preserves ambiguity instead of hiding the choice.
possible = {e["id"] for e in candidates("device")}
```

```
resolution = next(iter(possible)) if len(possible) == 1 else None
assert resolution is None and possible == {"pump_a", "valve_b"}
```

The baseline prints `valve_b`, while the constructed speaker intention points to the earlier pump. A cue cannot establish identity in every case. A clarifying question such as “Do you mean Pump A or Valve B?” preserves the uncertainty explicitly; NLP 08 develops clarification in dialogue.

Context can include deixis: “this,” “here” or “now” depend on a speaker, place, time or interface selection, so store that context with the utterance, and do not score a system as wrong for refusing an inference the input cannot support. Quotation adds constraints too: `I` in an operator’s quoted statement refers to that operator, and “the other pump” presupposes a contrast set that a recency rule does not define.

Evaluating coreference: MUC, B³, CEAF and CoNLL F1

Plain version. A coreference output is a partition of mentions into entities. Different metrics count agreement between two partitions in different units — links, mentions or entities — so no single one tells the whole story.

Let K be the key (gold) partition and R the response partition.

- **MUC** is link-based [20]. For each key entity S, let $p(S)$ be the partition of S induced by the response, with unmatched mentions as singletons. Recall is $\sum_S (|S| - |p(S)|) / \sum_S (|S| - 1)$: the number of key links the response preserves over the minimum number of links needed. Precision swaps the roles of key and response.
- **B³** is mention-based. For each mention m, compare the key entity K_m and response entity R_m containing it: recall averages $|K_m \cap R_m|/|K_m|$ and precision averages $|K_m \cap R_m|/|R_m|$ over mentions [23].
- **CEAF** is entity-based [21]. It finds the best one-to-one alignment between key and response entities, a maximum bipartite matching. With the entity similarity $\phi(K, R) = 2|K \cap R|/(|K| + |R|)$ (the entity-based variant CEAF_e), recall is the total aligned similarity divided by the number of key entities and precision divides by the number of response entities.

The CoNLL-2012 shared task scored systems by the **unweighted mean** of MUC, B³ and CEAF_e F1, on the grounds that each represents a different dimension [23]. Pradhan et al. later published a reference scorer because inconsistent implementations, especially for predicted mentions that do not match gold mentions, had produced incomparable numbers [22]. The snippet computes all three for the response in Figure 5, where “it” is wrongly merged with the valve.

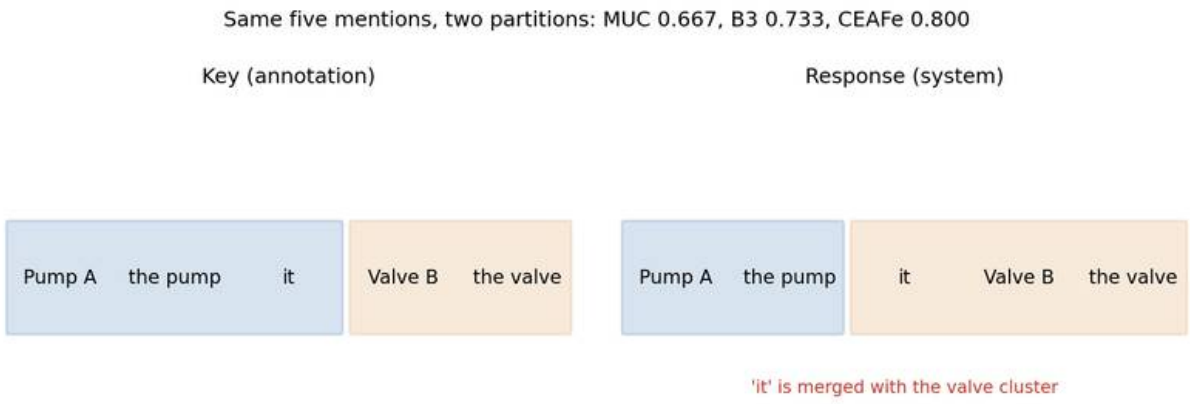


Figure 5 — One wrong link, three different penalties. The scores are computed by the snippet below.

```
import numpy as np
from scipy.optimize import linear_sum_assignment

# Mentions: a="Pump A", b="the pump", c="it", d="Valve B", e="the valve"
key = [{"a", "b", "c"}, {"d", "e"}]
response = [{"a", "b"}, {"c", "d", "e"}]          # "it" wrongly joined to the valve

def f1(p, r):
    return 2 * p * r / (p + r) if p + r else 0.0

def muc(gold, pred):                                # link-based
    def recall(g, p):
        def parts(s):  # pieces of s cut by p; uncovered mentions are singletons
            covered = set().union(*p)
            return sum(1 for t in p if s & t) + len(s - covered)
        return sum(len(s) - parts(s) for s in g) / sum(len(s) - 1 for s in g)
    return recall(pred, gold), recall(gold, pred)    # precision, recall

def b_cubed(gold, pred):                            # mention-based
    def score(a, b):
        cluster_of = {m: s for s in b for m in s}
        return np.mean([len(s & cluster_of.get(m, set())) / len(s)
                        for s in a for m in s])
    return score(pred, gold), score(gold, pred)

def ceaf_e(gold, pred):                             # entity-based
    phi = np.array([[2 * len(g & p) / (len(g) + len(p)) for p in pred] for g in gold])
    rows, cols = linear_sum_assignment(-phi)
    total = phi[rows, cols].sum()
    return total / len(pred), total / len(gold)

scores = {name: f1(*fn(key, response)) for name, fn in
          [("MUC", muc), ("B3", b_cubed), ("CEAFe", ceaf_e)]}
for name, value in scores.items():
    print(f"{name:6s} F1 = {value:.3f}")
```

```
print(f"CoNLL F1 = {np.mean(list(scores.values())):.3f}")
```

```
MUC    F1 = 0.667
B3     F1 = 0.733
CEAFe  F1 = 0.800
CoNLL F1 = 0.733
```

Check MUC by hand: the key needs $2 + 1 = 3$ links; the response splits the pump entity into two pieces and keeps the valve entity whole, preserving $1 + 1 = 2$ links, so recall is $2/3$, and by symmetry precision is $2/3$. B^3 penalizes each affected mention, and $CEAF_e$ still aligns each key entity with a mostly-correct response entity. One error, three numbers: that is why the shared task averaged them, and why an application should also check the records that consume the clusters.

Scores are interpretable only relative to the annotation policy: whether events and associative relations are linked, whether singletons count, and how ambiguity is annotated.

Semantic parsing and AMR

Plain version. Semantic parsing maps a sentence to a meaning representation that a program can use: a logical form, a database query or a graph. Abstract Meaning Representation (AMR) is a widely used whole-sentence graph format.

Banarescu et al. designed AMR for "sembanking": AMRs are rooted, labeled, directed graphs that are easy for people to read and programs to traverse; they aim to abstract away from syntactic idiosyncrasies, assigning the same AMR to sentences with the same basic meaning, and they make extensive use of PropBank framesets [24]. When one entity plays several roles, AMR uses **re-entrancy** — a node with multiple parents — so the graph is not a tree. In "the operator wants to stop the pump," the operator is `ARG0` of both `want-01` and `stop-01`. AMR deliberately does not specify how the graph is derived from the words, which makes alignment between words and nodes a separate problem [24].

Smatch evaluates AMR parsing by viewing each graph as a conjunction of triples, either `instance(variable, concept)` or `relation(variable1, variable2)`, and reporting precision, recall and F1 of matched triples under the variable mapping that maximizes the match [25]. Finding that mapping is NP-complete, so the paper provides an integer-linear-programming method and a hill-climbing heuristic with random restarts [25]. For a four-variable graph, we can simply try every mapping:

```
import itertools

# "the operator wants to stop the pump" vs a parse that loses who stops.
gold = {("instance", "w", "want-01"), ("instance", "o", "operator"),
        ("instance", "s", "stop-01"), ("instance", "p", "pump"),
        ("ARG0", "w", "o"), ("ARG1", "w", "s"),
        ("ARG0", "s", "o"), ("ARG1", "s", "p")}
```

```
pred = {("instance", "a", "want-01"), ("instance", "b", "operator"),
        ("instance", "c", "stop-01"), ("instance", "d", "pump"),
        ("ARG0", "a", "b"), ("ARG1", "a", "c"), ("ARG1", "c", "d")}

def rename(triples, mapping):
    return {(r, mapping.get(x, x), mapping.get(y, y)) for r, x, y in triples}

pred_vars, gold_vars = sorted({t[1] for t in pred}), sorted({t[1] for t in gold})
best = max(len(rename(pred, dict(zip(pred_vars, perm))) & gold)
           for perm in itertools.permutations(gold_vars))
precision, recall = best / len(pred), best / len(gold)
print("matched triples:", best)
print(f"Smatch P={precision:.3f} R={recall:.3f}
 ↪  F1={2*precision*recall/(precision+recall):.3f}")
```

```
matched triples: 7
Smatch P=1.000 R=0.875 F1=0.933
```

The predicted graph is missing one triple — the re-entrant `ARG0` edge that says the operator is also the one stopping. Smatch still gives 0.933. For a safety application, "who stops the pump" may be the whole question. Like every aggregate metric in this chapter, Smatch should be paired with targeted checks on the relations that matter.

How pretrained models changed these tasks

Plain version. Pretrained encoders turned parsers, role labelers and coreference systems into "encoder plus small structured decoder." They did not remove the need for the decoder, the annotation scheme or careful evaluation.

Three shifts stand out. First, **features were replaced by contextual vectors**. Kitaev and Klein's parser improved from 93.55 to 95.13 F1 when it used pretrained word representations [6]; Joshi et al. applied BERT to coreference and reported a gain of 3.9 F1 over the previous state of the art on OntoNotes, noting remaining weaknesses in document-level context, conversations and mention paraphrasing [26]. SpanBERT changed the pretraining objective itself — masking contiguous spans and training span-boundary representations to predict the span content — and reported 79.6% F1 on OntoNotes coreference along with 94.6 and 88.7 F1 on SQuAD 1.1 and 2.0 [27]. Span-shaped tasks benefited from span-shaped pretraining.

Second, **the structures turned out to be partly present inside the encoders**. Hewitt and Manning's structural probe found a linear transformation under which squared distance between word vectors encodes distance in the parse tree, for ELMo and BERT but not for baselines [28]. Tenney, Das and Pavlick found that BERT represents the steps of the traditional pipeline in an interpretable order across layers — part-of-speech tagging, parsing, named entities, semantic roles, then coreference — while sometimes revising lower-level decisions using higher-level information [29]. These are probing results: they show that information is linearly recoverable, not that the model uses it the way a parser does.

Third, **the pipeline collapsed into fine-tuning and prompting**. Encoders supply evidence for tags and trees, and large language models can be prompted to return structures [30]. An output validator remains necessary: a plausible bracketed string can omit a token, reorder words or fail to form a connected tree.

Modern view

Surveys and their taxonomies. Zhang's survey of syntactic–semantic parsing focuses on the two most popular formalizations, constituent and dependency parsing, reviews representative models for each and for semantic dependency graphs, and covers cross-domain, cross-lingual and joint parsing as well as corpus development [31]. Poesio et al.'s review of computational models of anaphora traces coreference from pre-neural work through current neural models, surveys the annotated datasets, and treats less-studied phenomena such as bridging reference and discourse deixis [32]. Chen et al.'s SRL survey, accepted at Artificial Intelligence Review, proposes a four-dimensional taxonomy — model architectures, syntax feature modeling, application scenarios and multimodal extensions — analyzes when syntactic features help, and examines the complementary roles of LLMs and specialized SRL systems [34]. Zhao et al.'s LLM survey distinguishes model training, adaptation, utilization and capacity evaluation [30]; structure can play several roles within that taxonomy — a supervised output, an internal representational probe, a constraint on generation or a diagnostic challenge. Chang et al.'s evaluation survey further separates the tasks being measured from the benchmarks and protocols used [33].

Recent original work (2023–2026). Tian, Xia and Song found that LLMs are effective at shallow parsing (chunking) but struggle to produce correct full constituency trees, and improved them by prompting for chunks first and filtering them [35]. Hale and Stanojević tested a candidate syntactic universal across typologically diverse languages and found that the model they studied respected it only in some higher-resource languages [36], illustrating why an English-focused structural test cannot establish uniform multilingual competence. For coreference, Le and Ritter found that prompting instruction-tuned models can outperform unsupervised coreference systems but relies on high-quality mention detection [37]; Porada, Zou and Cheung controlled for the pretrained language model across five coreference systems and found that encoder-based models outperformed more recent decoder-based ones at equal model size, and that controlling for the encoder removes most, but not all, of the reported F1 gains of the preceding five years [38]. For semantic representations, Jin et al. found that feeding AMR into LLM chain-of-thought prompting generally hurt more than it helped across five tasks, with errors concentrated in multiword expressions, named entities and the final inference step [39].

Distinguish valid structure from structural generalization

These studies motivate two different tests. First, can a system produce a legal structure for this annotation scheme? Second, does its choice respond to the grammatical distinction rather than a familiar word sequence? A structural validator addresses the first and cannot establish the second. A model that always returns one conventional attachment can pass format checks while failing a

controlled contrast.

Construct comparisons that hold lexical content nearly fixed while changing the role-relevant construction (active versus passive, modifier relocation, a relative clause), then vary vocabulary while holding the construction fixed. If an unfamiliar noun changes the whole structure, lexical coverage is interfering; if a grammatical change leaves the interpretation untouched, a familiar lexical pattern dominates. Finally check whether mistakes alter the semantic roles needed downstream.

Longer context changes the reference decision, not only memory size

A longer history can restore a missing antecedent, but it also adds compatible candidates; in an antecedent softmax, a new candidate changes the denominator even when existing link scores stay fixed, so thresholds calibrated on short histories need not transfer. Compare a short history, a long irrelevant history and a long history containing another device of the same type to separate useful context from merely more text.

Open problems. Annotation schemes differ; low-resource languages lack enough independently checked trees; segmentation interacts with syntax; spontaneous dialogue contains fragments and repairs; coreference across long documents and conversations remains hard; and a system can learn benchmark-specific conventions without robust compositional behavior. Preserving alternative analyses, testing structural minimal pairs, controlling for the encoder when comparing architectures [38], and measuring downstream consequences offer concrete ways to study those limits.

Key takeaways

- A grammar's failure to parse is a coverage result, not a judgment of a speaker; charts share spans, but enumerating all trees can still grow exponentially.
- Best-tree (max) and total-probability (sum) answer different questions; rule probabilities alone cannot see lexical evidence, which is what neural span and arc scorers add.
- Transition parsers make linear-time local decisions; graph parsers score all arcs and need a tree decoder, because independent head choices can form cycles.
- Each structure has its own metric — labeled brackets, UAS/LAS, argument F1, MUC/B^3/CEAF, Smatch — and each needs a stated denominator and alignment policy.
- Semantic roles survive grammatical alternations; PropBank numbers arguments per predicate and marks negation and modality as adjuncts.
- Preserve scope, attribution and provenance in extracted records; a coarse schema loses distinctions that no encoder can recover afterwards.
- Coreference is detection plus linking; one wrong bridge can merge entities, and an explicit ambiguous outcome is better than a confident wrong one.
- Pretrained encoders improved and simplified these systems, but validity still comes from the decoder and evaluation from the annotation scheme.

Exercises

1. **A grammar extension.** Extend the chart parser to accept “we saw cameras” and a new preposition. Then add an unseen noun and explain which lexical change is required.

   Hint

   The existing binary rules already support the shorter sentence. Coverage of terminals is a separate issue from phrase combination.

2. **Recognition versus enumeration.** Replace tree lists with Boolean categories in the chart. Compare what the parser can still answer and what information it loses. Then change the PCFG snippet so that the noun attachment wins with the original rule probabilities, by adding a lexicalized rule score for `robots with`.

   Hint

   Acceptance remains available without trees; recovering alternative attachments needs backpointers. For the lexicalized variant, multiply the $NP \rightarrow NP\,PP$ score by a factor that depends on the head noun.

3. **A structural validator and a non-projective tree.** Write checks for token yield, one head per word, no self-links, a unique root and absence of cycles in a basic dependency tree. Then construct a valid tree for a short sentence whose arcs cross, and explain why the arc-standard oracle cannot produce it.

   Hint

   Following head pointers from each word should reach the root without visiting a word twice. Arc-standard only attaches the top two stack items, so every arc it creates spans a contiguous, already-completed region.

4. **Two readings, two records.** Write two interpretations of “A technician inspected every sensor.” Design a relation schema that can store either reading, and an evidence rule for answering “Was every sensor inspected?” from a document collection.

   Hint

   Compare an existential quantifier outside a universal quantifier with the reverse order. The answer requires a sensor inventory and coverage information, not only extracted inspection events.

5. **Coreference metrics.** Construct a response that merges all five mentions of Figure 5 into one entity, and another that leaves all five as singletons. Compute MUC, B^3 and $CEAF_e$ for both before running the code, then explain which metric is most lenient to each mistake.

Hint

MUC counts links, so a single giant cluster preserves every key link. B^3 penalizes each mention whose cluster contains wrong members. Singletons have no links at all.

6. **Bilingual deixis.** Write a bilingual message containing a demonstrative, a relative date and an indirect request, for example mixing English and Chinese. Describe two contexts that change its interpretation, and list the evidence a coreference or extraction system would need to distinguish them.

Hint

A forwarded quotation changes whose viewpoint controls "I," "here" and "tomorrow." An ability test and a maintenance conversation can give the same question different purposes.

References

1. Susan McRoy, *Principles of Natural Language Processing*, Chapters 3–5 and 7 and the chart-parsing appendix, open textbook, 2021. Open textbook
2. Daniel Jurafsky and James H. Martin, *Speech and Language Processing*, 3rd ed. draft (August 19, 2026), Chapters 19, 20, 22 and 24. Book site
3. E. Black, S. Abney, D. Flickenger, C. Gdaniec, R. Grishman, P. Harrison, D. Hindle, R. Ingria, F. Jelinek, J. Klavans, M. Liberman, M. Marcus, S. Roukos, B. Santorini and T. Strzalkowski, "A Procedure for Quantitatively Comparing the Syntactic Coverage of English Grammars," Speech and Natural Language Workshop, 1991. Paper
4. Mitchell P. Marcus, Beatrice Santorini and Mary Ann Marcinkiewicz, "Building a Large Annotated Corpus of English: The Penn Treebank," Computational Linguistics 19(2), 1993. Paper
5. Mitchell Stern, Jacob Andreas and Dan Klein, "A Minimal Span-Based Neural Constituency Parser," ACL, 2017. Paper
6. Nikita Kitaev and Dan Klein, "Constituency Parsing with a Self-Attentive Encoder," ACL, 2018. Paper
7. Nikita Kitaev, Steven Cao and Dan Klein, "Multilingual Constituency Parsing with Self-Attention and Pre-Training," ACL, 2019. Paper
8. Marie-Catherine de Marneffe, Christopher D. Manning, Joakim Nivre and Daniel Zeman, "Universal Dependencies," Computational Linguistics 47(2), 2021. Paper
9. Joakim Nivre, "Algorithms for Deterministic Incremental Dependency Parsing," Computational Linguistics 34(4), 2008. Paper
10. Danqi Chen and Christopher Manning, "A Fast and Accurate Dependency Parser using Neural Networks," EMNLP, 2014. Paper
11. Ryan McDonald, Fernando Pereira, Kiril Ribarov and Jan Hajič, "Non-Projective Dependency Parsing using Spanning Tree Algorithms," HLT-EMNLP, 2005. Paper
12. Eliyahu Kiperwasser and Yoav Goldberg, "Simple and Accurate Dependency Parsing Using Bidirectional LSTM Feature Representations," Transactions of the ACL 4, 2016. Paper

13. Timothy Dozat and Christopher D. Manning, “Deep Biaffine Attention for Neural Dependency Parsing,” ICLR, 2017. Paper
14. Daniel Zeman, Jan Hajič, Martin Popel, Martin Potthast, Milan Straka, Filip Ginter, Joakim Nivre and Slav Petrov, “CoNLL 2018 Shared Task: Multilingual Parsing from Raw Text to Universal Dependencies,” CoNLL Shared Task, 2018. Paper
15. Daniel Gildea and Daniel Jurafsky, “Automatic Labeling of Semantic Roles,” Computational Linguistics 28(3), 2002. Paper
16. Martha Palmer, Daniel Gildea and Paul Kingsbury, “The Proposition Bank: An Annotated Corpus of Semantic Roles,” Computational Linguistics 31(1), 2005. Paper
17. Xavier Carreras and Lluís Màrquez, “Introduction to the CoNLL-2005 Shared Task: Semantic Role Labeling,” CoNLL, 2005. Paper
18. Luheng He, Kenton Lee, Mike Lewis and Luke Zettlemoyer, “Deep Semantic Role Labeling: What Works and What’s Next,” ACL, 2017. Paper
19. Kenton Lee, Luheng He, Mike Lewis and Luke Zettlemoyer, “End-to-end Neural Coreference Resolution,” EMNLP, 2017. Paper
20. Marc Vilain, John Burger, John Aberdeen, Dennis Connolly and Lynette Hirschman, “A Model-Theoretic Coreference Scoring Scheme,” MUC-6, 1995. Paper
21. Xiaoqiang Luo, “On Coreference Resolution Performance Metrics,” HLT-EMNLP, 2005. Paper
22. Sameer Pradhan, Xiaoqiang Luo, Marta Recasens, Eduard Hovy, Vincent Ng and Michael Strube, “Scoring Coreference Partitions of Predicted Mentions: A Reference Implementation,” ACL, 2014. Paper
23. Sameer Pradhan, Alessandro Moschitti, Nianwen Xue, Olga Uryupina and Yuchen Zhang, “CoNLL-2012 Shared Task: Modeling Multilingual Unrestricted Coreference in OntoNotes,” EMNLP-CoNLL Shared Task, 2012. Paper
24. Laura Banarescu, Claire Bonial, Shu Cai, Madalina Georgescu, Kira Griffitt, Ulf Hermjakob, Kevin Knight, Philipp Koehn, Martha Palmer and Nathan Schneider, “Abstract Meaning Representation for Sembanking,” Linguistic Annotation Workshop, 2013. Paper
25. Shu Cai and Kevin Knight, “Smatch: an Evaluation Metric for Semantic Feature Structures,” ACL, 2013. Paper
26. Mandar Joshi, Omer Levy, Luke Zettlemoyer and Daniel Weld, “BERT for Coreference Resolution: Baselines and Analysis,” EMNLP-IJCNLP, 2019. Paper
27. Mandar Joshi, Danqi Chen, Yinhan Liu, Daniel S. Weld, Luke Zettlemoyer and Omer Levy, “SpanBERT: Improving Pre-training by Representing and Predicting Spans,” Transactions of the ACL 8, 2020. Paper
28. John Hewitt and Christopher D. Manning, “A Structural Probe for Finding Syntax in Word Representations,” NAACL-HLT, 2019. Paper
29. Ian Tenney, Dipanjan Das and Ellie Pavlick, “BERT Rediscovers the Classical NLP Pipeline,” ACL, 2019. Paper
30. Wayne Xin Zhao et al., “A Survey of Large Language Models,” Frontiers of Computer Science, 2026; online manuscript updated March 2026. Survey
31. Meishan Zhang, “A survey of syntactic-semantic parsing based on constituent and dependency structures,” Science China Technological Sciences 63, 2020. Survey
32. Massimo Poesio, Juntao Yu, Silviu Paun, Abdulrahman Aloraini, Pengcheng Lu, Janosch Haber and Derya Cokal, “Computational Models of Anaphora,” Annual Review of Linguistics 9, 2023.

Survey
33. Yupeng Chang et al., “A Survey on Evaluation of Large Language Models,” ACM Transactions on Intelligent Systems and Technology, 2024. Survey
34. Huiyao Chen, Meishan Zhang, Jing Li, Lilja Øvrelid, Jan Hajič, Hao Fei and Min Zhang, “A Systematic Survey of Semantic Role Labeling in the Era of Pretrained Language Models,” arXiv: 2502.08660, 2025; accepted at Artificial Intelligence Review. Survey
35. Yuanhe Tian, Fei Xia and Yan Song, “Large Language Models are No Longer Shallow Parsers,” ACL, 2024. Paper
36. John T. Hale and Miloš Stanojević, “Do LLMs learn a true syntactic universal?” EMNLP, 2024. Paper
37. Nghia T. Le and Alan Ritter, “Are Large Language Models Robust Coreference Resolvers?” arXiv:2305.14489, 2023. Paper
38. Ian Porada, Xiyuan Zou and Jackie Chi Kit Cheung, “A Controlled Reevaluation of Coreference Resolution Models,” LREC-COLING, 2024. Paper
39. Zhijing Jin, Yuen Chen, Fernando Gonzalez Adauto, Jiarui Liu, Jiayi Zhang, Julian Michael, Bernhard Schölkopf and Mona Diab, “Analyzing the Role of Semantic Representations in the Era of Large Language Models,” NAACL, 2024. Paper

Part IV

Pattern Recognition

PR 01 • Patterns, Probability and Bayes Decisions

Define a recognition pipeline, separate likelihoods from posteriors, and derive minimum-error, minimum-risk and reject decisions, Gaussian discriminants and ROC analysis.

Online edition: https://blog.shyandram.dev/en/learn/pattern-recognition/pr-01-bayes-decisions/

What you'll learn

- Turn a recognition problem into observations, features, predictions and actions, and decide what one "sample" means.
- Distinguish a class prior, a class likelihood and a class posterior, and see how a change in the prior moves a decision boundary.
- Derive the minimum-error rule, the minimum-risk rule for any loss matrix, and Chow's reject rule.
- Write classifiers as discriminant functions and derive the linear (LDA) and quadratic (QDA) boundaries of the Gaussian case.
- Read ROC and precision–recall curves, connect them to the Neyman–Pearson criterion, and choose an operating point from costs.

The big picture

A camera records a manufactured component. A system measures its shape, estimates whether it is defective, and decides whether to send it for manual inspection. These are different stages. The recorded pixels are **observations**; shape measurements are **features**; a defect probability is a **prediction**; sending the part to inspection is an **action**. Bayes decision theory is the part of pattern recognition that connects the last two: given what we believe, what should we do?

> **Explain like I'm five**
>
> A sorting machine looks at a fruit and guesses whether it is ripe. Guessing is one job; choosing a basket is another. If most fruit in the box is ripe, the machine should lean towards "ripe" even when the fruit looks a bit odd. If throwing away a good fruit costs much more than asking a person to check it, the machine should sometimes say "I'm not sure, please look at this one." Bayes decision theory is just the arithmetic of these three ideas: how common each kind is, what the evidence says, and how much each mistake hurts.

This track follows the classic pattern-recognition textbooks: Duda, Hart and Stork's *Pattern Classification* [1], Bishop's *Pattern Recognition and Machine Learning* [2], and Theodoridis and Koutroumbas's *Pattern Recognition* [3], with Hastie, Tibshirani and Friedman [4] for the statistical view. The chapters use original examples and derivations; the textbooks are where to go for longer proofs.

Bayes decision theory comes first in all of them because every later method can be judged against it: a classifier is good to the extent that it approximates the decision the Bayes rule would take if we knew the true distributions.

Recognition is not limited to images. An observation can be a sound recording, a sequence of pen strokes or a document. Classification predicts categories, regression predicts quantities, and clustering discovers groups without labels. This chapter is about classification, but the way of thinking (model the uncertainty, then choose the action with the lowest expected cost) carries over to all three.

Start with the experiment

Plain version. Before any formula, write down what one sample is and how the data were collected. Most surprising failures of recognition systems come from this step, not from the classifier.

One image, one patient, one video and one crop of a video are different **sampling units**. If twenty frames come from the same recording, treating them as twenty independent cases can make a test look much easier than deployment, because the test frames are near-copies of training frames.

Precise version. A recognition pipeline can be written as

$$o \longrightarrow x = \phi(o) \longrightarrow p(c \mid x) \longrightarrow a(x).$$

Here o is a raw observation (an image, a waveform); ϕ is the preprocessing and feature extraction that maps it to a feature vector $x \in \mathbb{R}^d$; $c \in \{c_1, \dots, c_K\}$ is one of K classes; $p(c \mid x)$ is the model's belief about the class; and $a(x)$ is the action finally taken. Learning adjusts the parameters of ϕ and of the probability model from **training** samples; **validation** samples choose settings such as thresholds and regularization; a held-out **test** set estimates the performance of the whole chosen procedure once (PR 09 treats this carefully: validation and ensembles).

Handcrafted features (edge histograms, shape moments) and learned embeddings (the output of a pretrained network) occupy the same slot ϕ. Changing the feature extractor changes which information the classifier receives. Information removed by cropping, quantization or pooling cannot be recovered by any classifier downstream, however clever. In decision-theoretic language: the Bayes error of the problem is defined *relative to* the features, and a poor ϕ raises the floor that no classifier can go below.

Bayes' rule connects evidence and belief

Plain version. Bayes' rule says how to update "how common is each class" into "how likely is each class for *this* object", using "how typical is this measurement for each class".

Precise version. Let the classes be $c_1, \dots, c_K$. Bayes' rule gives

$$P(c_k \mid x) = \frac{p(x \mid c_k)\, P(c_k)}{\sum_{j=1}^{K} p(x \mid c_j)\, P(c_j)}.$$

- The **prior** $P(c_k)$ is the probability of class c_k before examining this observation, for example the fraction of defective parts on the production line.
- The **likelihood** (or class-conditional density) $p(x \mid c_k)$ describes how measurements are distributed within class c_k.
- The **evidence** $p(x) = \sum_j p(x \mid c_j)P(c_j)$ is the overall density of x; it is the same for every class and only normalizes.
- The **posterior** $P(c_k \mid x)$ is the probability of class c_k after observing x.

For continuous measurements, a likelihood is a density. Its value need not lie between zero and one, and it is not the probability of observing that exact real number. Posterior class probabilities do lie in $[0, 1]$ and sum to one over k. Confusing the two directions of conditioning, $p(x \mid c)$ versus $P(c \mid x)$, is the classic **base-rate error** that the code example below makes concrete.

Because the evidence is shared, a decision only needs the products $p(x \mid c_k)P(c_k)$. Figure 1 shows what that means geometrically for two one-dimensional Gaussian classes: the boundary sits where the two *prior-weighted* likelihoods cross. When the prior of class c_1 drops from 0.5 to 0.1, its weighted curve shrinks, and the boundary moves towards c_1's mean: the system now needs stronger evidence before declaring the rarer class.

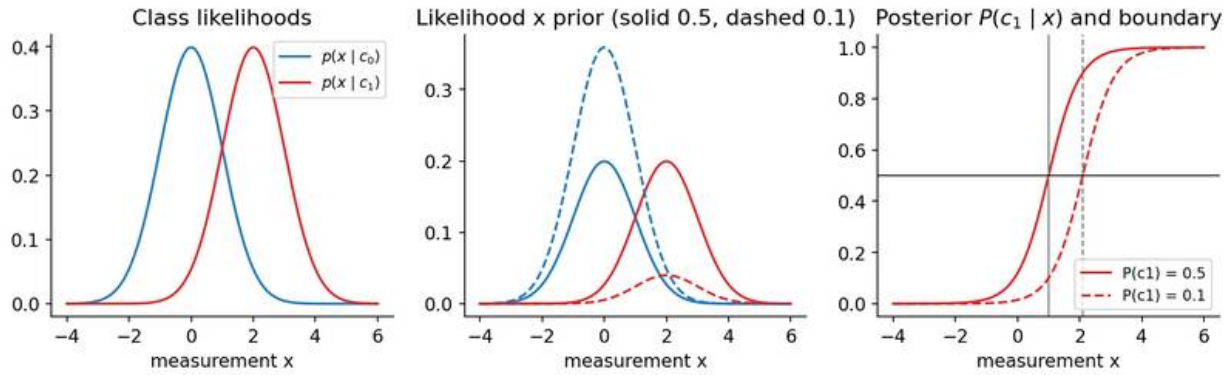


Figure 1 — Likelihoods (left) do not change when the prior changes; prior-weighted likelihoods (middle) and posteriors (right) do. With $P(c1) = 0.5$ the boundary is at $x = 1$; with $P(c1) = 0.1$ it moves to $1 + ln(9)/2 \approx 2.10$.

For the unit-variance case in Figure 1 the boundary has a closed form. With means $\mu_0 = 0$, $\mu_1 = 2$ and variance 1, the log ratio of the weighted likelihoods is $2x - 2 + \ln \frac{P(c_1)}{P(c_0)}$, which is zero at $x = 1 + \frac{1}{2} \ln \frac{P(c_0)}{P(c_1)}$. That is $x = 1$ for equal priors and $x \approx 2.10$ for $P(c_1) = 0.1$.

Code: rare events change the posterior

This NumPy example uses hypothetical inspection statistics: a detector flags 90% of defective parts (sensitivity) and 8% of good parts (false-positive rate), and 2% of parts are defective. It computes the posterior after a positive and after a negative detector output, then compares the minimum-error and minimum-risk actions (derived in the next sections).

```
import numpy as np

prior = 0.02
sensitivity = 0.90
false_positive_rate = 0.08
p_positive = (sensitivity * prior /
              (sensitivity * prior + false_positive_rate * (1 - prior)))
p_negative = ((1 - sensitivity) * prior /
              ((1 - sensitivity) * prior + (1 - false_positive_rate) * (1 - prior)))
p = np.array([p_negative, p_positive])
c_fp, c_fn = 1.0, 9.0
threshold = c_fp / (c_fp + c_fn)
risks = np.column_stack([c_fn * p, c_fp * (1 - p)])
decisions = risks.argmin(axis=1)
assert np.array_equal(decisions, (p > threshold).astype(int))
print("posteriors [negative observation, positive observation]:", p)
print("minimum-risk actions:", decisions, "threshold:", threshold)
print("minimum-error actions:", (p > 0.5).astype(int))
```

The output is a posterior of about 0.0022 after a negative flag and only 0.187 after a positive one. A positive observation does not imply a 90% defect probability: sensitivity conditions on the true class, while the posterior conditions on the observation, and with a 2% prior most positive flags come from the large pool of good parts. Under equal costs, neither observation leads to "defective" ($0.187 < 0.5$). With a missed defect costing nine times as much as an unnecessary inspection, the threshold drops to 0.1 and a positive flag does lead to inspection. The probability model did not change; only the decision rule did.

Minimum-error decisions

Plain version. If every mistake costs the same, always pick the class that is most probable for this observation. No other rule makes fewer mistakes on average.

Precise version. A decision rule $a(x)$ assigns each x to a class, which partitions feature space into **decision regions** $\mathcal{R}_1, \dots, \mathcal{R}_K$, where $\mathcal{R}_k = \{x : a(x) = c_k\}$. The probability of error is

$$P(\text{error}) = \int p(x)\,\bigl[1 - P(a(x) \mid x)\bigr]\,dx.$$

Here $P(a(x) \mid x)$ is the posterior of the class we chose. The integrand is non-negative and $p(x)$ does not depend on the rule, so the integral is minimized by minimizing $1 - P(a(x) \mid x)$ separately at every x, that is, by choosing

$$a^*(x) = \arg\max_k P(c_k \mid x) = \arg\max_k p(x \mid c_k) P(c_k).$$

This is the **maximum a posteriori (MAP) rule** or **Bayes classifier**. Its error,

$$P^*_{\text{err}} = \int p(x)\Bigl[1 - \max_k P(c_k \mid x)\Bigr]dx,$$

is the **Bayes error**: the lowest error any classifier can achieve with these features. It is zero only when the classes do not overlap in feature space. A classifier with 8% test error on a problem whose Bayes error is 7% is close to optimal; the same 8% on a problem with Bayes error 1% leaves a lot on the table. We rarely know the Bayes error of a real task, but thinking about it separates two different questions: "are my features informative enough?" (the Bayes error) and "does my classifier extract what the features contain?" (the gap above it).

For two classes the MAP rule becomes a **likelihood-ratio test**:

$$\Lambda(x) = \frac{p(x \mid c_1)}{p(x \mid c_0)} \gtrless \frac{P(c_0)}{P(c_1)}.$$

Choose c_1 when the ratio exceeds the threshold on the right, c_0 otherwise. The data enter only through $\Lambda(x)$; priors and, as we will see next, costs only move the threshold. This is why one score can serve many decision policies.

Loss matrices and minimum risk

Plain version. Some mistakes are worse than others. Assign a cost to each kind of mistake, compute the average cost of each possible action given the current beliefs, and take the cheapest action.

Precise version. Let the available actions be $\alpha_1, \dots, \alpha_A$ (usually "decide class i", possibly also "reject"). A **loss matrix** $\lambda_{ij} = L(\alpha_i, c_j)$ gives the cost of taking action α_i when the true class is c_j. The **conditional risk** of an action is its expected loss under the posterior:

$$R(\alpha_i \mid x) = \sum_{j=1}^{K} \lambda_{ij}\, P(c_j \mid x), \qquad a^*(x) = \arg\min_i R(\alpha_i \mid x).$$

The overall risk $R = \int R(a(x) \mid x)\, p(x)\, dx$ is minimized by the same pointwise argument as before, so this **Bayes decision rule** achieves the minimum possible expected loss, the **Bayes risk** [1, 2]. With the **zero-one loss** ($\lambda_{ij} = 0$ if $i = j$, 1 otherwise), $R(\alpha_i \mid x) = 1 - P(c_i \mid x)$ and the rule reduces to MAP.

For two classes, write λ_{10} for the cost of deciding c_1 when the truth is c_0 (a false positive) and λ_{01} for deciding c_0 when the truth is c_1 (a false negative). Choosing c_1 is cheaper when $R(\alpha_1 \mid x) < R(\alpha_0 \mid x)$, which rearranges to

$$\frac{p(x \mid c_1)}{p(x \mid c_0)} > \frac{\lambda_{10} - \lambda_{00}}{\lambda_{01} - \lambda_{11}} \cdot \frac{P(c_0)}{P(c_1)}.$$

Costs and priors multiply in the threshold: doubling the cost of a false negative has exactly the same effect on the decision as doubling the prior of the positive class. With correct decisions free ($\lambda_{00} = \lambda_{11} = 0$), $C_{\mathrm{FP}} = \lambda_{10}$, $C_{\mathrm{FN}} = \lambda_{01}$ and $p = P(c_1 \mid x)$, the rule becomes a threshold on the posterior:

$$C_{\mathrm{FP}}(1 - p) < C_{\mathrm{FN}}\, p \quad \Longleftrightarrow \quad p > \frac{C_{\mathrm{FP}}}{C_{\mathrm{FP}} + C_{\mathrm{FN}}}.$$

When missing a defect costs nine times as much as an unnecessary inspection, the threshold is 0.1, not 0.5 (Figure 2, right).

Two remarks matter in practice. First, a loss matrix is a modelling decision, often made by people outside the machine-learning team (clinicians, safety engineers, product owners); write it down explicitly so that it can be reviewed. Second, the posterior threshold is only meaningful if the posterior is **calibrated**, meaning that among cases given probability 0.1, about 10% really are positive. Many modern classifiers are not, which is one theme of the Modern view below.

The reject option

Plain version. When the system is unsure, it can hand the case to a person. Refusing to decide has a fixed cost; deciding has an expected cost that is high when the posterior is spread out. Refuse exactly when deciding is expected to cost more.

Precise version. Add an action a_{reject} with constant loss C_R regardless of the true class, and keep the zero-one loss for the K classification actions. The best classification action has risk $1 - \max_k P(c_k \mid x)$, so the Bayes rule becomes **Chow's rule** [5]:

$$a^*(x) = \begin{cases} \arg\max_k P(c_k \mid x) & \text{if } \max_k P(c_k \mid x) \geq 1 - C_R, \\ \text{reject} & \text{otherwise.} \end{cases}$$

If rejection costs 0.2, the system accepts only when the top posterior is at least 0.8 (Figure 2, left). Two edge cases help intuition. If $C_R = 0$ (asking a person is free), the system rejects everything that is not certain. If $C_R \geq 1 - 1/K$, it never rejects, because the top posterior is always at least $1/K$.

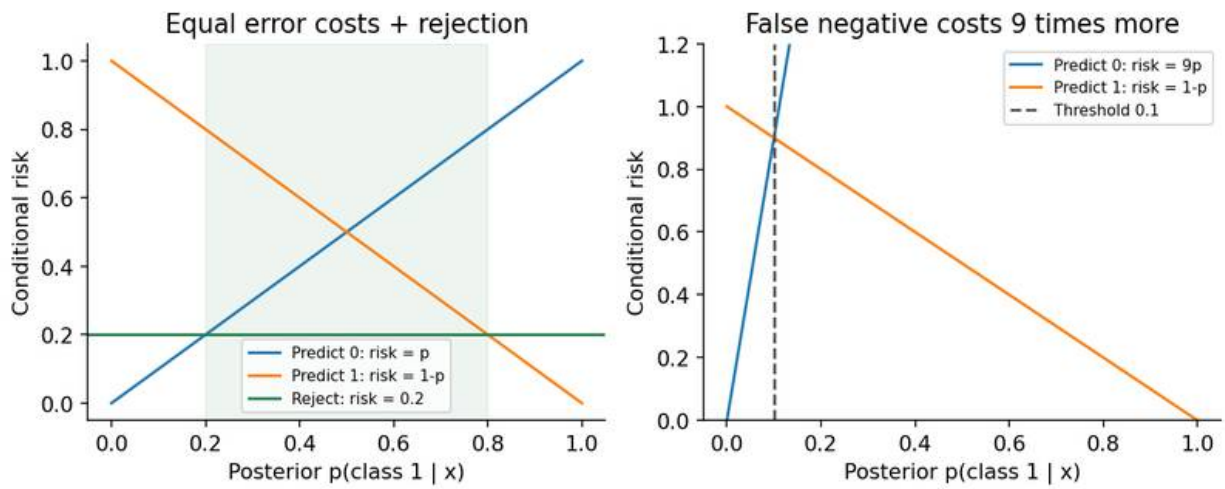


Figure 2 — Conditional risk as a function of the posterior. Left: equal costs plus a reject action costing 0.2; rejecting is cheapest when the posterior lies between 0.2 and 0.8. Right: a false negative costs nine times a false positive, so the decision threshold moves to 0.1.

Chow showed that sweeping the threshold traces an **error–reject trade-off curve**: each extra percent of rejected samples buys a reduction in the error on the remaining ones, with diminishing returns [5]. Figure 3 shows it for three overlapping Gaussian classes whose true posteriors are known. Without rejection about a quarter of all samples are misclassified; rejecting the least certain 40% cuts the error on all samples to about 8%.

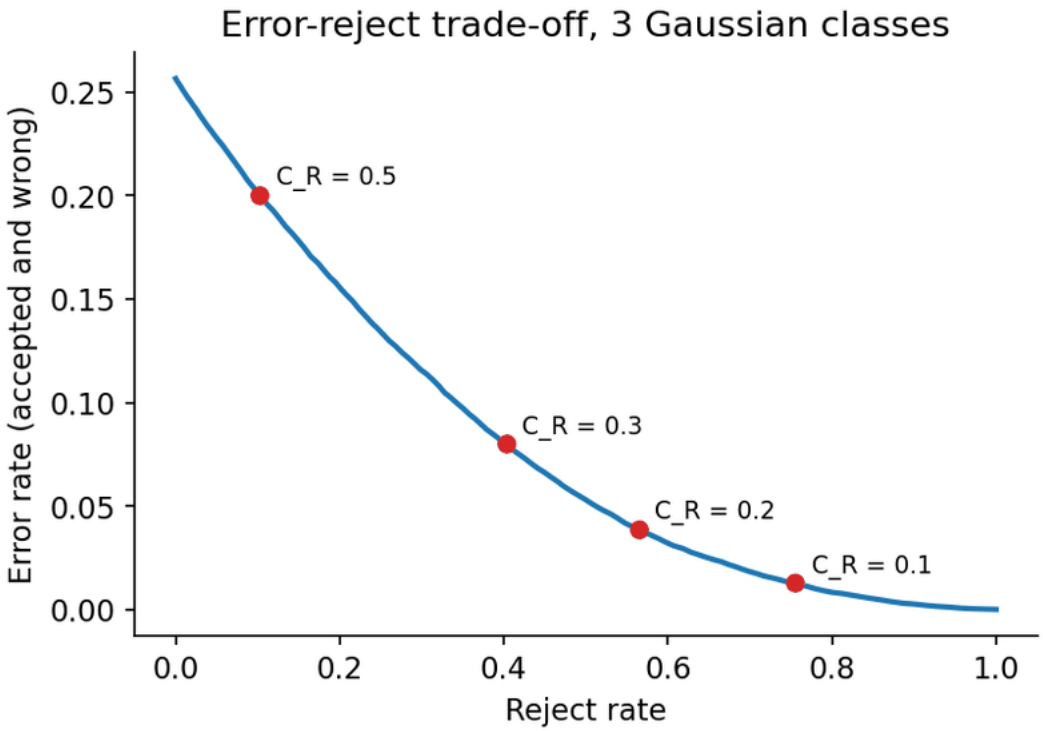


Figure 3 — Error–reject trade-off for three 2-D Gaussian classes with known posteriors (20,000 synthetic samples). Each point is Chow's rule for one reject cost C_R; the vertical axis counts samples that were accepted and misclassified.

Code: Chow's rule

```
import numpy as np

rng = np.random.default_rng(2)
means = np.array([[0, 0], [2, 0], [1, 1.7]])
y = rng.integers(0, 3, 20000)
X = means[y] + rng.normal(0, 1, (len(y), 2))

# exact posteriors: equal priors, identity covariance
logits = -0.5 * ((X[:, None, :] - means[None]) ** 2).sum(-1)
post = np.exp(logits - logits.max(1, keepdims=True))
post /= post.sum(1, keepdims=True)
pred, pmax = post.argmax(1), post.max(1)
```

```
for c_r in [0.5, 0.3, 0.2, 0.1]:
    accept = pmax >= 1 - c_r                    # Chow's rule
    reject_rate = 1 - accept.mean()
    err_accepted = np.mean(pred[accept] != y[accept])
    risk = np.mean(np.where(accept, pred != y, c_r))
    print(f"C_R={c_r}: reject {reject_rate:.3f}, "
          f"error on accepted {err_accepted:.3f}, average loss {risk:.3f}")
```

With this seed, $C_R = 0.3$ rejects about 40% of samples and leaves an error of about 13.5% *among the accepted ones*; $C_R = 0.1$ rejects about 76% and leaves about 4.4%. The average loss always falls as C_R falls because the reject action itself becomes cheaper. Subtracting the max logit before exponentiating (`logits.max`) is the standard trick that keeps the softmax numerically stable.

Rejection helps only when the uncertainty estimate is meaningful. Chow's rule handles **ambiguity**: observations that lie between known classes. It does not handle **novelty**: an object of a kind never seen in training can land far from every class and still receive a confident posterior, because posteriors are normalized over the known classes only. Detecting such inputs is the subject of open-set recognition and out-of-distribution detection (Modern view).

Discriminant functions and decision regions

Plain version. A classifier can be written as a set of scoring functions, one per class: compute every score and pick the largest. Many different scores give the same decisions.

Precise version. A classifier is defined by **discriminant functions** $g_1(x), \dots, g_K(x)$ and the rule "assign x to c_k if $g_k(x) > g_j(x)$ for all $j \neq k$" [1]. The Bayes minimum-error classifier is obtained with $g_k(x) = P(c_k \mid x)$, but any **monotonically increasing** function f applied to all discriminants leaves every decision unchanged. Three equivalent choices are therefore

$$g_k(x) = P(c_k \mid x), \qquad g_k(x) = p(x \mid c_k)P(c_k), \qquad g_k(x) = \ln p(x \mid c_k) + \ln P(c_k).$$

The log form is the one used in practice: it turns products of many small densities into sums and avoids numerical underflow. For minimum risk, use $g_k(x) = -R(\alpha_k \mid x)$.

The **decision boundary** between c_i and c_j is the set where $g_i(x) = g_j(x)$ and both are larger than all other scores. For two classes a single function suffices, a **dichotomizer**:

$$g(x) = g_1(x) - g_0(x) = \ln \frac{p(x \mid c_1)}{p(x \mid c_0)} + \ln \frac{P(c_1)}{P(c_0)}, \qquad \text{decide } c_1 \text{ if } g(x) > 0.$$

This is the log-likelihood ratio plus the log prior odds. It also equals the **logit** of the posterior, because $P(c_1 \mid x) = \sigma(g(x))$ with the logistic sigmoid $\sigma(z) = 1/(1+e^{-z})$. Remember this identity: in PR 03, logistic regression models $g(x)$ directly as a linear function instead of deriving it from densities.

The view through discriminants is useful because it separates *what* the classifier computes (the shape of the boundaries) from *how* it was obtained (densities, direct fitting, margins). Linear

discriminants $g_k(x) = w_k^\top x + w_{k0}$ produce boundaries that are hyperplanes and regions that are convex; quadratic discriminants produce conics in 2-D and hyperquadrics in general. The Gaussian case shows exactly when each arises.

The Gaussian case: LDA and QDA boundaries

Plain version. If each class looks like a "cloud" shaped like a multivariate Gaussian, the Bayes rule measures how far x is from each cloud's center, in units that respect the cloud's shape. When all clouds have the same shape, the boundaries are straight; when shapes differ, they bend.

Precise version. Suppose $p(x \mid c_k) = \mathcal{N}(x; \mu_k, \Sigma_k)$ with mean $\mu_k \in \mathbb{R}^d$ and covariance matrix $\Sigma_k \in \mathbb{R}^{d\times d}$ (symmetric, positive definite):

$$\mathcal{N}(x; \mu, \Sigma) = \frac{1}{(2\pi)^{d/2}|\Sigma|^{1/2}} \exp\!\Big(-\tfrac{1}{2}(x-\mu)^\top \Sigma^{-1}(x-\mu)\Big),$$

where $|\Sigma|$ is the determinant. The log discriminant is

$$g_k(x) = -\tfrac{1}{2}\,(x-\mu_k)^\top \Sigma_k^{-1}(x-\mu_k) - \tfrac{1}{2}\ln|\Sigma_k| + \ln P(c_k),$$

after dropping the constant $-\frac{d}{2}\ln 2\pi$ shared by all classes. The quadratic form $\Delta_k^2(x) = (x-\mu_k)^\top \Sigma_k^{-1}(x-\mu_k)$ is the squared **Mahalanobis distance** from x to μ_k: Euclidean distance after "whitening" the class so its cloud becomes a unit sphere. Three cases, shown in Figure 4, cover what happens [1, 4].

Case 1: $\Sigma_k = \sigma^2 I$ for all k. Features are uncorrelated with equal variance σ^2. Expanding $\|x-\mu_k\|^2$ and dropping the class-independent $x^\top x$ gives a linear discriminant

$$g_k(x) = w_k^\top x + w_{k0}, \qquad w_k = \frac{\mu_k}{\sigma^2}, \qquad w_{k0} = -\frac{\mu_k^\top \mu_k}{2\sigma^2} + \ln P(c_k).$$

The boundary between classes i and j is the hyperplane $w^\top(x - x_0) = 0$ with $w = \mu_i - \mu_j$ and

$$x_0 = \tfrac{1}{2}(\mu_i + \mu_j) - \frac{\sigma^2}{\|\mu_i - \mu_j\|^2} \ln\frac{P(c_i)}{P(c_j)}\,(\mu_i - \mu_j).$$

It is perpendicular to the line joining the means. With equal priors it passes through their midpoint, and the classifier is exactly the **minimum-distance (nearest-mean) classifier** of IP 13. Unequal priors slide the plane away from the more probable mean.

Case 2: $\Sigma_k = \Sigma$ shared by all classes. The quadratic term $x^\top \Sigma^{-1} x$ is again common and cancels, leaving

$$w_k = \Sigma^{-1}\mu_k, \qquad w_{k0} = -\tfrac{1}{2}\mu_k^\top \Sigma^{-1}\mu_k + \ln P(c_k).$$

The boundaries are still hyperplanes, $w^\top(x - x_0) = 0$ with $w = \Sigma^{-1}(\mu_i - \mu_j)$, but no longer perpendicular to the mean difference: they tilt to follow the shared correlation (Figure 4, middle). This is **linear discriminant analysis (LDA)** in its probabilistic form. With equal priors it is a

minimum-Mahalanobis-distance classifier. The direction $\Sigma^{-1}(\mu_1 - \mu_0)$ is the same as the one Fisher derived from a projection criterion in 1936, which PR 03 re-derives.

Case 3: arbitrary Σ_k. Nothing cancels:

$$g_k(x) = x^\top W_k x + w_k^\top x + w_{k0}, \quad W_k = -\tfrac{1}{2}\Sigma_k^{-1}, \quad w_k = \Sigma_k^{-1}\mu_k, \quad w_{k0} = -\tfrac{1}{2}\mu_k^\top \Sigma_k^{-1}\mu_k - \tfrac{1}{2}\ln|\Sigma_k| + \ln P(c_k).$$

The boundaries are **hyperquadrics**: lines, parabolas, ellipses or hyperbolas in 2-D. This is **quadratic discriminant analysis (QDA)**. In Figure 4 (right) the tight blue class is enclosed by an ellipse, because a point far from the blue mean in any direction is better explained by the broad red class. A region can even be disconnected, which never happens for linear discriminants.

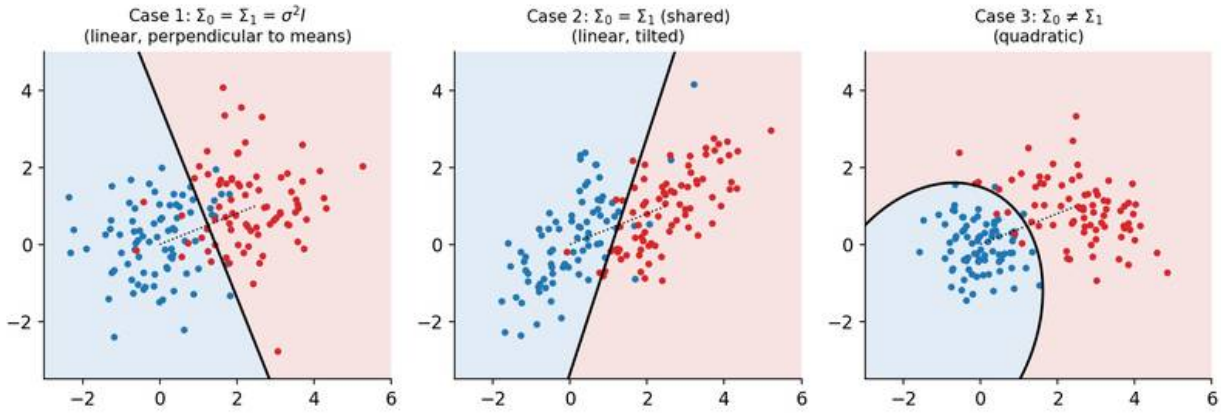


Figure 4 — Bayes boundaries (black) for two Gaussian classes with equal priors. Case 1: spherical shared covariance gives the perpendicular bisector. Case 2: a shared correlated covariance tilts the line (LDA). Case 3: different covariances give a quadratic boundary (QDA). The dotted segment joins the means.

For two classes with a shared covariance and equal priors, the Bayes error has a closed form,

$$P^*_{\text{err}} = \Phi\left(-\frac{\Delta}{2}\right), \qquad \Delta = \sqrt{(\mu_1 - \mu_0)^\top \Sigma^{-1}(\mu_1 - \mu_0)},$$

where Φ is the standard normal cumulative distribution function and Δ is the Mahalanobis distance between the means. Two classes three standard deviations apart ($\Delta = 3$) still give $\Phi(-1.5) \approx 6.7\%$ unavoidable error. This formula is a good sanity check for any classifier you train on Gaussian toy data.

Which case should you use? It is a bias–variance choice. A full covariance per class has $d(d+1)/2$ free entries per class; with $d = 100$ that is 5,050 numbers per class, which needs far more samples than many datasets have. Sharing one covariance (LDA), restricting to diagonals (which gives Gaussian naive Bayes) or shrinking towards a simple target trades flexibility for stability. Estimating these parameters is the subject of the next chapter, PR 02: density estimation.

Code: Gaussian discriminants from scratch

This example fits Gaussian class models, evaluates the log discriminants with a Cholesky factorization (never an explicit inverse), and checks that the decisions match scikit-learn's `LinearDiscriminantAnalysis` and `QuadraticDiscriminantAnalysis` on the Case 3 data of Figure 4.

```
import numpy as np
from sklearn.discriminant_analysis import (LinearDiscriminantAnalysis,
                                           QuadraticDiscriminantAnalysis)

rng = np.random.default_rng(0)
means = [np.array([0.0, 0.0]), np.array([2.5, 1.0])]
covs = [np.array([[0.4, 0.0], [0.0, 0.4]]), np.array([[2.0, -0.6], [-0.6, 1.0]])]
X = np.vstack([rng.multivariate_normal(m, S, 300) for m, S in zip(means, covs)])
y = np.repeat([0, 1], 300)

def fit_gaussians(X, y, shared):
    params = []
    pooled = sum(np.cov(X[y == k].T, bias=True) * (y == k).sum()
                 for k in (0, 1)) / len(y)
    for k in (0, 1):
        Xk = X[y == k]
        S = pooled if shared else np.cov(Xk.T, bias=True)
        params.append((Xk.mean(0), S, len(Xk) / len(y)))
    return params

def discriminants(X, params):
    g = []
    for mu, S, prior in params:
        L = np.linalg.cholesky(S)               # S = L L^T
        z = np.linalg.solve(L, (X - mu).T)      # whitened residuals
        maha = (z ** 2).sum(0)                  # squared Mahalanobis distance
        logdet = 2 * np.log(np.diag(L)).sum()
        g.append(-0.5 * maha - 0.5 * logdet + np.log(prior))
    return np.stack(g, axis=1)

for shared, ref in [(True, LinearDiscriminantAnalysis(store_covariance=True)),
                    (False, QuadraticDiscriminantAnalysis())]:
    mine = discriminants(X, fit_gaussians(X, y, shared)).argmax(1)
    theirs = ref.fit(X, y).predict(X)
    name = "LDA (shared)" if shared else "QDA (separate)"
    print(f"{name}: training error {np.mean(mine != y):.3f}, "
          f"agrees with scikit-learn on {np.mean(mine == theirs):.1%} of points")
```

Both hand-written classifiers agree with scikit-learn on every point. On these data, whose true covariances differ, QDA's training error (4.3%) is below LDA's (5.5%). That is training error, though: with few samples in high dimensions the ordering often reverses on held-out data.

ROC analysis and the Neyman–Pearson view

Plain version. A two-class score can be cut at many thresholds. Each threshold trades catching more positives against raising more false alarms. The ROC curve draws all those trade-offs at once, so you can pick the threshold later, when costs or priors are known.

Precise version. Let $s(x)$ be any score where larger values favor c_1 (the log-likelihood ratio, a posterior, a network logit), and decide c_1 when $s(x) > t$. For each threshold t define

$$\mathrm{TPR}(t) = P(s(x) > t \mid c_1), \qquad \mathrm{FPR}(t) = P(s(x) > t \mid c_0).$$

The true-positive rate (sensitivity, recall) is the fraction of positives caught; the false-positive rate ($1-$ specificity) is the fraction of negatives flagged. The **receiver operating characteristic (ROC)** curve plots TPR against FPR as t sweeps from $+\infty$ to $-\infty$ [7]. A useless score lies on the diagonal; a perfect one passes through the top-left corner. Because both rates condition on the true class, the ROC curve does not change when the class priors change.

The Neyman–Pearson criterion. Sometimes costs are hard to state but a constraint is easy: "at most 1% false alarms". The Neyman–Pearson lemma [6] says that among all decision rules with $\mathrm{FPR} \le \alpha$, the one with the highest TPR is a likelihood-ratio test $\Lambda(x) > \eta$, with η chosen so the false-positive rate equals α (with randomization at the boundary if needed). So the ROC curve of the likelihood ratio is the best achievable curve; any other score's ROC lies on or below it. Both the Bayes rule and the Neyman–Pearson rule are likelihood-ratio tests; they differ only in how the threshold is chosen.

Choosing an operating point from costs. The expected cost per sample at threshold t is

$$\mathbb{E}[\text{cost}] = C_{\mathrm{FP}}\, P(c_0)\, \mathrm{FPR}(t) + C_{\mathrm{FN}}\, P(c_1)\, (1 - \mathrm{TPR}(t)).$$

Points of equal cost lie on straight **iso-cost lines** in ROC space with slope $C_{\mathrm{FP}}P(c_0)/(C_{\mathrm{FN}}P(c_1))$; the optimal operating point is where the steepest such line touches the curve [7]. For the likelihood-ratio score this tangent point is exactly the Bayes threshold derived earlier.

Area under the curve. The AUC summarizes the curve in one number. It equals the probability that a randomly chosen positive receives a higher score than a randomly chosen negative (ties counted half) [7]. For two unit-variance Gaussian score distributions separated by d' standard deviations, $\mathrm{AUC} = \Phi(d'/\sqrt{2})$: Figure 5 shows $d' = 2$ gives 0.921. AUC ignores costs, priors and calibration, so two classifiers with the same AUC can have very different costs at the threshold you actually deploy.

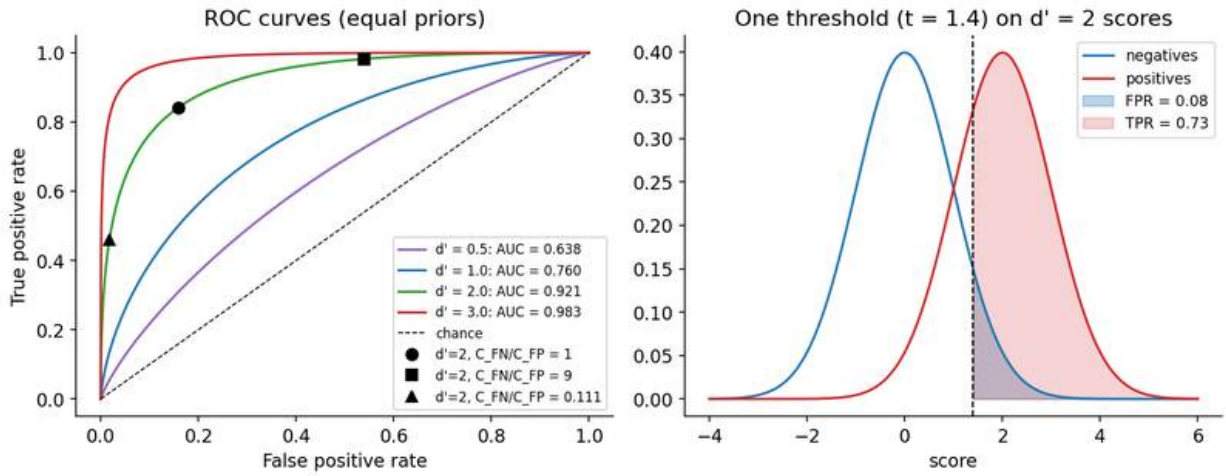


Figure 5 — Left: ROC curves of equal-variance Gaussian scores; markers show the Bayes operating points on the $d' = 2$ curve for three cost ratios (equal priors). Right: one threshold on the $d' = 2$ scores and the FPR and TPR it produces.

Precision–recall curves. When positives are rare (fraud, defects, small objects in detection), a small FPR can still mean many false alarms in absolute numbers, because negatives are so numerous. Precision $= \mathrm{TP}/(\mathrm{TP} + \mathrm{FP})$ depends on the prior and exposes this. Davis and Goadrich [8] proved that one curve dominates another in ROC space if and only if it dominates in precision–recall space, and that linear interpolation between PR points is not valid. Saito and Rehmsmeier [9] showed with simulations and published studies that for imbalanced data the PR plot is more informative than ROC. Object-detection benchmarks report average precision for this reason (see DLCV 03).

Code: ROC, AUC and the cost-optimal threshold

```
import numpy as np
from sklearn.metrics import roc_auc_score, roc_curve

rng = np.random.default_rng(1)
n_neg, n_pos = 9000, 1000                          # prevalence 10%
scores = np.r_[rng.normal(0, 1, n_neg), rng.normal(2, 1, n_pos)]
labels = np.r_[np.zeros(n_neg), np.ones(n_pos)]

fpr, tpr, thr = roc_curve(labels, scores)
auc = roc_auc_score(labels, scores)
# Mann-Whitney view of AUC: P(score_pos > score_neg)
pos, neg = scores[labels == 1], scores[labels == 0]
auc_pairs = (pos[:, None] > neg[None, :]).mean()
print(f"AUC = {auc:.4f}, pairwise estimate = {auc_pairs:.4f}")

# choose the operating point with minimum expected cost per sample
c_fp, c_fn, prior = 1.0, 9.0, n_pos / (n_pos + n_neg)
cost = c_fp * fpr * (1 - prior) + c_fn * (1 - tpr) * prior
```

```
i = np.argmin(cost)
print(f"best threshold {thr[i]:.2f}: FPR {fpr[i]:.3f}, TPR {tpr[i]:.3f}, "
      f"expected cost {cost[i]:.3f}")
# Bayes threshold from the true model: predict 1 when the likelihood ratio
# exceeds c_fp (1 - prior) / (c_fn prior); for unit Gaussians, log LR = 2s - 2
t_bayes = (np.log(c_fp * (1 - prior) / (c_fn * prior)) + 2) / 2
print(f"Bayes threshold from the true densities: {t_bayes:.2f}")
```

The empirical AUC (0.9166) matches the pairwise-ranking estimate exactly, and is close to the theoretical $\Phi(2/\sqrt{2}) \approx 0.921$. With a 10% prior and a 9:1 cost ratio, prior odds and cost ratio cancel, so the Bayes threshold is $t = 1$; the empirical search finds 0.99. In a real project the threshold search must run on validation data, never on the test set.

Where the assumptions can fail

The Bayes rule is optimal *relative to* the assumed distributions and the specified loss. Each ingredient can be wrong in deployment.

Prior shift. Class frequencies may differ between the training set and deployment, for example between two factories or between a balanced research dataset and a real clinic. If the class-conditional distributions $p(x \mid c)$ stay fixed (**label shift**), the posterior can be corrected without retraining: multiply each posterior by the ratio of new to old priors and renormalize,

$$P_{\text{new}}(c_k \mid x) \propto P_{\text{old}}(c_k \mid x)\, \frac{P_{\text{new}}(c_k)}{P_{\text{old}}(c_k)}.$$

When the new priors are unknown, Saerens, Latinne and Decaestecker [10] estimate them with an EM procedure on the unlabeled deployment data, and Lipton, Wang and Smola's black-box shift estimation [11] estimates them from a classifier's confusion matrix. If a new camera changes the likelihoods themselves (**covariate** or **concept shift**), a prior correction is not enough.

Miscalibrated posteriors. Every threshold in this chapter assumes that $P(c \mid x)$ means what it says. Scores from SVMs, boosted trees or deep networks are often systematically over- or under-confident; recalibration on held-out data (Platt scaling, isotonic regression, temperature scaling) is a standard fix [12, 14].

Label noise and proxies. A label may record an imperfect inspection rather than physical truth. The model then learns the inspector's posterior, not the world's.

Single-number summaries. Two classifiers with identical accuracy can have different false-negative rates and very different expected loss. Report the confusion matrix, the operating point and the costs.

Modern view

Bayes decision theory is more than fifty years old, and its vocabulary (posterior, risk, reject, operating point) is now the vocabulary of trustworthy deep learning. Research since 2017 has mostly been about making the *inputs* to the Bayes rule trustworthy when the classifier is a large network.

Calibration. Guo et al. [12] found that modern deep networks were much more overconfident than the shallower networks of a decade earlier. They related this to depth, width, weight decay and batch normalization, and showed that **temperature scaling** (dividing all logits by one scalar fitted on validation data, a one-parameter version of Platt scaling) is surprisingly effective. Minderer et al. [13] revisited the question for 2021-era image classifiers and found that the most recent models, notably those not using convolutions, are among the best calibrated, and that the earlier trend of calibration decaying with model size or under distribution shift is less pronounced. The survey by Silva Filho et al. [14] is the best entry point: it covers proper scoring rules and calibration metrics, reliability diagrams, and a comprehensive account of post-hoc calibration methods for binary and multiclass problems. Gawlikowski et al. [23] review the wider field of uncertainty in neural networks, separating reducible model uncertainty from irreducible data uncertainty and covering Bayesian networks, ensembles and test-time augmentation.

Rejection and selective classification. Chow's reject rule [5] has been rediscovered as **selective classification**. Geifman and El-Yaniv [16] showed how to wrap a trained network with a rejection threshold chosen so that the risk on accepted samples stays below a user-specified level with high probability, trading coverage for guaranteed accuracy. The survey by Hendrickx et al. [15] formalizes two kinds of rejection, **ambiguity rejection** (Chow's case) and **novelty rejection** (inputs unlike the training data), reviews how to evaluate a model's predictive and rejective quality together, and categorizes the architectures used to learn models with a reject option.

Distribution-free guarantees. Conformal prediction replaces a single label with a *set* of labels that is guaranteed to contain the true class with a chosen probability, such as 90%, under the sole assumption that calibration and test data are exchangeable. Angelopoulos and Bates [17] give a hands-on introduction with code. Large prediction sets play the role of rejection: the model is telling you that it cannot narrow things down.

Open-set recognition and out-of-distribution detection. A posterior normalized over K known classes cannot say "none of the above". Geng, Huang and Chen [18] survey open-set recognition: definitions, model families, datasets and evaluation, and its relationship to classification with reject option and to zero- and few-shot learning. Yang et al. [19] unify anomaly detection, novelty detection, open-set recognition, out-of-distribution detection and outlier detection as subtasks of one **generalized OOD detection** framework. Two results connect directly to this chapter. Hendrycks and Gimpel [20] showed that the **maximum softmax probability**, Chow's statistic, is a strong baseline for detecting both misclassified and out-of-distribution inputs. Vaze et al. [21] found that a classifier's ability to say "none of the above" is highly correlated with its closed-set accuracy, so improving the ordinary classifier improves open-set performance. Lee et al. [22] fit **class-conditional Gaussians with a shared covariance** (exactly Case 2 / LDA above) to the features of a pretrained network and use the Mahalanobis distance as a confidence score. A 1970s-style generative classifier, placed on top of learned features, remains a competitive OOD detector.

Open problems. Costs are rarely known precisely, and they change over time; calibration measured on one distribution may not transfer to another; and guarantees such as conformal coverage are marginal (on average over inputs) rather than conditional on a particular input or subgroup. Deciding *what* the loss matrix should be, and who decides, remains a human question that no estimator answers.

Key takeaways

- A recognition system maps observations to features, features to beliefs, and beliefs to actions; the sampling unit and data split are part of the problem definition.
- Prior, likelihood and posterior answer different questions. Rare classes need strong evidence: a 90%-sensitive test at 2% prevalence gives a posterior below 20%.
- Maximum posterior minimizes the error rate; minimum conditional risk handles any loss matrix. For two classes both are likelihood-ratio tests whose threshold depends on priors times costs.
- Chow's rule rejects when the top posterior falls below $1 - C_R$; it handles ambiguity, not novelty.
- Discriminant functions are defined up to a monotone transform. Gaussian classes give linear boundaries when covariances are shared (LDA) and quadratic ones when they differ (QDA).
- ROC curves separate the score from the threshold; the Neyman–Pearson lemma makes the likelihood ratio the optimal score; PR curves are more informative when positives are rare.
- All of this relies on calibrated probabilities and stable priors, which modern work on calibration, selective classification, conformal prediction and OOD detection tries to secure.

Exercises

1. **Asymmetric costs.** Derive the binary decision threshold on the posterior $p = P(c_1 \mid x)$ when a false positive costs four and a false negative costs one. Which class does the rule now favor, and why?

   Hint

   Choose c_1 when $4(1 - p) < p$, so the threshold is $p > 0.8$. False alarms are now the expensive mistake, so the rule needs strong evidence before saying "positive".

2. **Reject or not.** For posterior $(0.55, 0.35, 0.10)$ and reject cost $C_R = 0.3$, choose the minimum-risk action under zero-one classification loss. For which values of C_R would the answer change?

   Hint

   The best classification risk is $1 - 0.55 = 0.45 > 0.3$, so reject. The system classifies (as c_1) when $C_R \geq 0.45$.

3. **A moving boundary.** Two 1-D classes have $p(x \mid c_0) = \mathcal{N}(0,1)$ and $p(x \mid c_1) = \mathcal{N}(3,1)$. Find the minimum-error boundary for equal priors and for $P(c_1) = 0.2$. Then find the minimum-risk boundary for equal priors when a false negative costs four times a false positive.

Hint

The log-likelihood ratio is $3x - 4.5$. Equal priors: $x = 1.5$. With $P(c_1) = 0.2$: $3x - 4.5 = \ln 4$, so $x \approx 1.96$. With $C_{\text{FN}} = 4C_{\text{FP}}$: $3x - 4.5 = \ln(1/4)$, so $x \approx 1.04$. The prior and the cost act through the same threshold, in opposite directions here.

4. **Mahalanobis versus Euclidean.** Two classes share $\Sigma = \begin{bmatrix} 4 & 0 \\ 0 & 1 \end{bmatrix}$ with means $\mu_0 = (0,0)$ and $\mu_1 = (2,1)$ and equal priors. Classify $x = (2,0)$ with the Euclidean nearest-mean rule and with the Bayes (LDA) rule. Explain the disagreement.

Hint

Euclidean squared distances: 4 to μ_0 and 1 to μ_1, so nearest-mean says c_1. Mahalanobis: $(2^2/4)+0 = 1$ to μ_0 and $0+1 = 1$ to μ_1, a tie: x lies exactly on the LDA boundary. The first feature varies a lot, so a 2-unit offset along it is weak evidence; the Euclidean rule over-weights it.

5. **ROC from scratch.** Using NumPy only, compute the ROC curve and the AUC of the scores in the ROC code example by sorting the scores, and verify against `roc_auc_score`. Then change the prevalence to 1% and explain which of ROC, AUC and precision at a fixed threshold change.

Hint

Sort scores in decreasing order; cumulative sums of labels give TP counts and of $1-$labels give FP counts; integrate TPR over FPR with the trapezoid rule. ROC and AUC depend only on the two class-conditional score distributions, so they stay (up to sampling noise) the same; precision at a fixed threshold falls sharply because false positives now come from a much larger pool of negatives.

6. **The sampling unit.** A test set contains adjacent frames from the same videos used for training. Explain what its accuracy estimates, and propose a split that estimates performance on new videos, new cameras or new subjects.

Hint

It estimates accuracy on near-duplicates of training frames (an interpolation task). Split by recording (group-wise splitting), and, if deployment will meet new cameras or people, hold out entire cameras or subjects as well.

References

1. R. O. Duda, P. E. Hart and D. G. Stork, *Pattern Classification*, 2nd ed., Wiley, 2001 (Ch. 2, Bayesian decision theory). publisher page
2. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006 (Sections 1.2 and 1.5). official page and free PDF
3. S. Theodoridis and K. Koutroumbas, *Pattern Recognition*, 4th ed., Academic Press, 2008 (Ch. 1–2). publisher page
4. T. Hastie, R. Tibshirani and J. Friedman, *The Elements of Statistical Learning*, 2nd ed., Springer, 2009 (Section 4.3, linear discriminant analysis). official page
5. C. K. Chow, "On optimum recognition error and reject tradeoff," *IEEE Transactions on Information Theory*, vol. 16, no. 1, pp. 41–46, 1970. doi
6. J. Neyman and E. S. Pearson, "On the problem of the most efficient tests of statistical hypotheses," *Philosophical Transactions of the Royal Society of London, Series A*, vol. 231, pp. 289–337, 1933. doi
7. T. Fawcett, "An introduction to ROC analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006. doi
8. J. Davis and M. Goadrich, "The relationship between precision-recall and ROC curves," in *Proc. International Conference on Machine Learning (ICML)*, 2006. doi
9. T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLOS ONE*, vol. 10, no. 3, e0118432, 2015. doi
10. M. Saerens, P. Latinne and C. Decaestecker, "Adjusting the outputs of a classifier to new a priori probabilities: a simple procedure," *Neural Computation*, vol. 14, no. 1, pp. 21–41, 2002. doi
11. Z. C. Lipton, Y.-X. Wang and A. Smola, "Detecting and correcting for label shift with black box predictors," in *Proc. ICML*, PMLR 80, 2018. paper
12. C. Guo, G. Pleiss, Y. Sun and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. ICML*, PMLR 70, 2017. paper
13. M. Minderer, J. Djolonga, R. Romijnders, F. Hubis, X. Zhai, N. Houlsby, D. Tran and M. Lucic, "Revisiting the calibration of modern neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. paper
14. T. Silva Filho, H. Song, M. Perello-Nieto, R. Santos-Rodriguez, M. Kull and P. Flach, "Classifier calibration: a survey on how to assess and improve predicted class probabilities," *Machine Learning*, vol. 112, pp. 3211–3260, 2023. doi
15. K. Hendrickx, L. Perini, D. Van der Plas, W. Meert and J. Davis, "Machine learning with a reject option: a survey," *Machine Learning*, vol. 113, pp. 3073–3110, 2024. doi
16. Y. Geifman and R. El-Yaniv, "Selective classification for deep neural networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. paper
17. A. N. Angelopoulos and S. Bates, "Conformal prediction: a gentle introduction," *Foundations and Trends in Machine Learning*, vol. 16, no. 4, pp. 494–591, 2023. doi
18. C. Geng, S.-J. Huang and S. Chen, "Recent advances in open set recognition: a survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, no. 10, pp. 3614–3631, 2021. doi
19. J. Yang, K. Zhou, Y. Li and Z. Liu, "Generalized out-of-distribution detection: a survey," *International Journal of Computer Vision*, vol. 132, pp. 5635–5662, 2024. doi

20. D. Hendrycks and K. Gimpel, “A baseline for detecting misclassified and out-of-distribution examples in neural networks,” in *Proc. International Conference on Learning Representations* (*ICLR*), 2017. arXiv
21. S. Vaze, K. Han, A. Vedaldi and A. Zisserman, “Open-set recognition: a good closed-set classifier is all you need,” in *Proc. ICLR*, 2022. arXiv
22. K. Lee, K. Lee, H. Lee and J. Shin, “A simple unified framework for detecting out-of-distribution samples and adversarial attacks,” in *Advances in Neural Information Processing Systems* (*NeurIPS*), 2018. paper
23. J. Gawlikowski et al., “A survey of uncertainty in deep neural networks,” *Artificial Intelligence Review*, vol. 56, suppl. 1, pp. 1513–1589, 2023. doi

PR 02 • Density Estimation and Bayesian Learning

Estimate the densities a Bayes classifier needs: maximum likelihood, MAP and full Bayesian estimation; histograms, Parzen windows and kernel bandwidth selection; k-NN density and the k-NN classifier; and the curse of dimensionality.

Online edition: https://blog.shyandram.dev/en/learn/pattern-recognition/pr-02-density-estimation/

What you'll learn

- Why a Bayes classifier needs density estimates, and the difference between parametric and non-parametric estimation.
- How to derive maximum-likelihood (ML) estimates for a Gaussian, why the covariance becomes singular with few samples, and how shrinkage fixes it.
- How MAP and full Bayesian estimation use a prior over parameters, worked out for a Gaussian mean and a Bernoulli rate.
- How histograms, Parzen windows (kernel density estimation) and k-nearest-neighbor estimates follow from one formula, and how to choose the bandwidth or k.
- How the k-NN classifier falls out of k-NN density estimation, what the Cover–Hart bound says, and why all local methods suffer from the curse of dimensionality.

The big picture

PR 01 showed that the best possible classifier needs the class priors $P(c_k)$ and the class-conditional densities $p(x \mid c_k)$. In practice nobody hands us these distributions; we have a finite set of labelled examples. This chapter is about turning examples into densities. Once we have $\hat{p}(x \mid c_k)$ and $\hat{P}(c_k)$, we can plug them into the Bayes rule. This is the **generative** route to classification. The next chapter takes the **discriminative** route and fits the decision boundary directly.

> **Explain like I'm five**
>
> Put a sticker on a park map wherever you have seen a bird. Now guess where the next bird will be. You could draw **one big oval** around most stickers (a parametric model: you only decide the oval's center and shape). You could lay a **grid** over the map and count stickers per square (a histogram). Or you could put a **little sand hill** on every sticker and look at where the sand piles up highest (a kernel estimate). Each choice is a different way to guess, and each can go wrong: the oval is too simple if birds like two different ponds, and the sand hills are too bumpy if you only have five stickers.

There are two families of estimators [1, 2, 3]:

- **Parametric** methods assume a functional form, such as a Gaussian, and estimate a fixed number of parameters. They need little data when the form is right and give systematically wrong answers when it is wrong.
- **Non-parametric** methods let the data decide the shape. Their complexity grows with the number of samples. They are flexible but need much more data, especially in high dimensions.

Both families face the same central question: how much structure can the available data support? A flexible estimator can describe the training data beautifully and still predict new data poorly. Every method below has a "smoothness knob" (the number of parameters, the prior, the bin width, the bandwidth, k), and choosing it well matters more than choosing the method.

Maximum-likelihood estimation

Plain version. Choose the parameter values under which the observed data would have been most probable.

Precise version. Let $D = \{x_1, \dots, x_N\}$ be samples drawn independently from a density $p(x \mid \theta)$ with unknown parameter vector θ. The **likelihood** of θ is $p(D \mid \theta) = \prod_n p(x_n \mid \theta)$, and the ML estimate maximizes its logarithm:

$$\hat{\theta}_{\mathrm{ML}} = \arg\max_{\theta} \ell(\theta), \qquad \ell(\theta) = \sum_{n=1}^{N} \log p(x_n \mid \theta).$$

The independence assumption is about *sampling* (different observations do not influence each other), not about independence between the coordinates of one feature vector. For a classifier we fit one model per class, using only that class's samples, and estimate the prior as the class frequency $\hat{P}(c_k) = N_k/N$.

The Gaussian case

For a d-dimensional Gaussian $\mathcal{N}(\mu, \Sigma)$ the log-likelihood is

$$\ell(\mu, \Sigma) = -\frac{N}{2}\log|\Sigma| - \frac{1}{2}\sum_n (x_n - \mu)^\top \Sigma^{-1}(x_n - \mu) + \text{const.}$$

Setting the gradient with respect to μ to zero gives $\Sigma^{-1}\sum_n (x_n - \mu) = 0$. Differentiating with respect to Σ^{-1} (using $\partial \log|A|/\partial A = A^{-\top}$ and $\partial\, a^\top A a/\partial A = a a^\top$) gives the estimates

$$\hat{\mu} = \frac{1}{N}\sum_n x_n, \qquad \hat{\Sigma} = \frac{1}{N}\sum_n (x_n - \hat{\mu})(x_n - \hat{\mu})^\top.$$

The sample mean is unbiased. The ML covariance is slightly **biased**: $\mathbb{E}[\hat{\Sigma}] = \frac{N-1}{N}\Sigma$, because $\hat{\mu}$ was fitted to the same data and sits closer to them than the true mean does. The familiar sample covariance divides by $N - 1$ to remove this bias. For large N the difference is negligible.

When ML breaks: too many parameters

A full covariance has $d(d+1)/2$ distinct entries. With $d = 64$ features that is 2,080 numbers per class, plus 64 for the mean. Worse, $\hat{\Sigma}$ is a sum of N rank-one matrices centered at $\hat{\mu}$, so its rank is at most $N - 1$. **When a class has no more samples than dimensions, its ML covariance is singular**: it has zero variance in some directions, $\log|\hat{\Sigma}| = -\infty$, and the Gaussian discriminant of PR 01 is undefined. Even when N is a bit larger than d, the smallest eigenvalues are systematically underestimated, and the inverse $\hat{\Sigma}^{-1}$ hugely amplifies noise along those directions.

Regularization **shrinks** the covariance towards a simpler target:

$$\tilde{\Sigma} = (1-\alpha)\hat{\Sigma} + \alpha\tau I, \qquad \tau = \operatorname{tr}(\hat{\Sigma})/d, \qquad 0 \le \alpha \le 1.$$

Here τI is a spherical matrix with the same average variance, and α is the shrinkage intensity. Shrinkage pulls large eigenvalues down and small ones up, trading a little bias for a much better-conditioned estimate. Ledoit and Wolf [14] derived a closed-form estimate of the α that minimizes the expected squared error, so it needs no cross-validation. Other common restrictions are a **diagonal** covariance (ignore correlations; this is Gaussian naive Bayes, below) and a **shared** covariance across classes (pool the evidence; this is LDA from PR 01). These are different modelling assumptions, not interchangeable numerical tricks.

Code: when the ML covariance fails

```
import numpy as np
from scipy.stats import multivariate_normal
from sklearn.covariance import EmpiricalCovariance, LedoitWolf

rng = np.random.default_rng(0)
d = 40
A = rng.normal(size=(d, d))
true_cov = A @ A.T / d + 0.5 * np.eye(d)          # a well-conditioned truth
test = rng.multivariate_normal(np.zeros(d), true_cov, 2000)

for n in [30, 60, 200, 1000]:
    train = rng.multivariate_normal(np.zeros(d), true_cov, n)
    mu = train.mean(0)
    S_ml = EmpiricalCovariance().fit(train).covariance_   # divisor n
    S_lw = LedoitWolf().fit(train)
    rank = np.linalg.matrix_rank(S_ml)
    ll_ml = (multivariate_normal(mu, S_ml, allow_singular=True).logpdf(test).mean()
             if rank == d else -np.inf)
    ll_lw = multivariate_normal(mu, S_lw.covariance_).logpdf(test).mean()
    print(f"n={n:4d}: rank(S_ML)={rank:2d}, test log-lik ML={ll_ml:8.2f}, "
          f"Ledoit-Wolf={ll_lw:8.2f} (shrinkage {S_lw.shrinkage_:.2f})")
```

With $d = 40$ and $n = 30$ the ML covariance has rank 29 and assigns zero density to almost every test point (log-likelihood $-\infty$). At $n = 60$ it is invertible but poor (test log-likelihood -94.6 versus -63.0 for Ledoit–Wolf). The automatically chosen shrinkage falls from 0.65 to 0.08 as n grows,

and at $n = 1000$ the two estimates almost coincide. The rule of thumb: the fewer samples per parameter, the more you should shrink.

MAP and Bayesian estimation

Plain version. Maximum likelihood trusts the data completely. Bayesian estimation starts from a prior belief about the parameters and lets the data gradually overrule it. With little data the prior matters; with lots of data it hardly does.

Precise version. Treat θ as a random variable with a **parameter prior** $p(\theta)$. Bayes' rule gives the parameter posterior

$$p(\theta \mid D) = \frac{p(D \mid \theta)\, p(\theta)}{\int p(D \mid \theta')\, p(\theta')\, d\theta'} \;\propto\; p(D \mid \theta)\, p(\theta).$$

A parameter prior is different from the class prior of PR 01. The class prior says how common a category is; a parameter prior says which model settings are plausible before seeing this training set. Three ways to use the posterior are common:

- **MAP estimation** keeps only its mode: $\hat{\theta}_{\mathrm{MAP}} = \arg\max_\theta \left[\log p(D \mid \theta) + \log p(\theta)\right]$. The log prior acts as a regularizer; a Gaussian prior on weights is exactly an L_2 (ridge) penalty, which PR 03 uses.
- **Full Bayesian prediction** averages over all parameter values instead of choosing one:

$$p(x_{\mathrm{new}} \mid D) = \int p(x_{\mathrm{new}} \mid \theta)\, p(\theta \mid D)\, d\theta.$$

This **posterior predictive** distribution carries parameter uncertainty into the prediction.

- **Plug-in prediction** uses $p(x_{\mathrm{new}} \mid \hat{\theta})$ with $\hat{\theta}$ from ML or MAP. It ignores parameter uncertainty and is therefore overconfident when data are scarce.

One subtle point: MAP is not invariant to re-parameterization. The mode of a density over σ is not the square root of the mode of the corresponding density over σ^2, because densities pick up a Jacobian factor when the variable changes. ML estimates and full posteriors do not have this problem.

Worked example: the mean of a Gaussian

Let $x_n \sim \mathcal{N}(\mu, \sigma^2)$ with known σ^2, and put a Gaussian prior on the unknown mean, $\mu \sim \mathcal{N}(m_0, s_0^2)$. Multiplying the likelihood and the prior and completing the square in μ shows that the posterior is again Gaussian, $\mu \mid D \sim \mathcal{N}(m_N, s_N^2)$, with [1, 2]

$$\frac{1}{s_N^2} = \frac{1}{s_0^2} + \frac{N}{\sigma^2}, \qquad m_N = s_N^2 \left(\frac{m_0}{s_0^2} + \frac{N \bar{x}}{\sigma^2} \right),$$

where $\bar{x}$ is the sample mean. Two readings make this formula memorable. **Precisions add**: the posterior precision $1/s_N^2$ is the prior precision plus N times the precision of one observation. **The mean is a precision-weighted average** of the prior mean and the sample mean. As $N \to \infty$, $m_N \to \bar{x}$ (the ML estimate) and $s_N \to 0$. Since the posterior is Gaussian, its mode equals its mean, so here $\hat{\mu}_{\text{MAP}} = m_N$. The posterior predictive is $\mathcal{N}(m_N,\ \sigma^2 + s_N^2)$: wider than the plug-in $\mathcal{N}(m_N,\ \sigma^2)$ by the remaining uncertainty about μ.

A prior whose posterior stays in the same family is called **conjugate**. Conjugate pairs (Gaussian–Gaussian, Beta–Bernoulli, Dirichlet–categorical, Normal–inverse-Wishart for a mean and covariance) give closed-form updates; other priors require the approximate inference methods of PR 08.

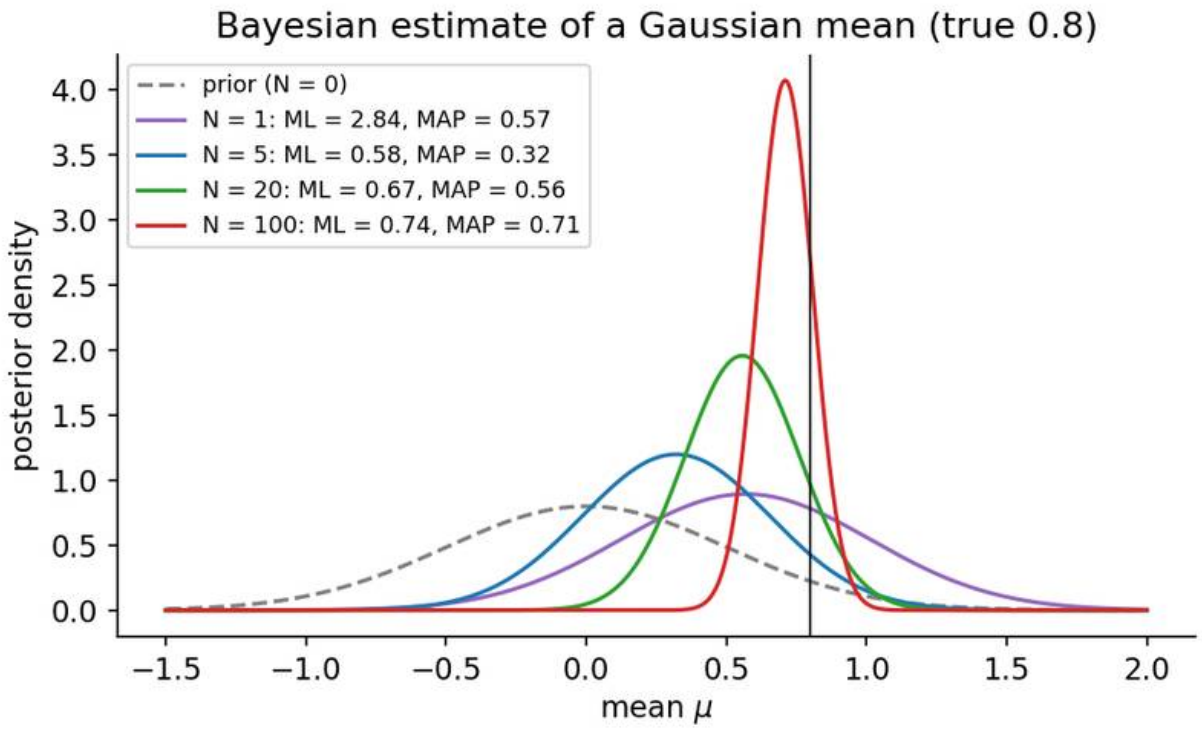


Figure 1 — Bayesian estimation of a Gaussian mean (true value 0.8, prior $N(0, 0.5^2)$). With one sample the ML estimate (2.84) is far off and the posterior mean is pulled back to 0.57; with 100 samples ML (0.74) and MAP (0.71) nearly agree and the posterior is narrow.

Worked example: a Bernoulli rate

For a Bernoulli rate q (for instance the defect rate on a line) with prior $\text{Beta}(a, b)$, observing s successes in N trials gives

$$q \mid D \sim \text{Beta}(a + s,\ b + N - s), \qquad p(x_{\text{new}} = 1 \mid D) = \frac{a + s}{a + b + N}.$$

The prior acts like a imaginary successes and b imaginary failures. With one success and no failures, ML predicts probability one, which is absurd. A uniform prior $\mathrm{Beta}(1,1)$ predicts $2/3$; this is Laplace's rule of succession. The same idea, called **additive** or **Laplace smoothing**, keeps a naive Bayes text classifier from assigning zero probability to a document because one word never appeared with a class in training.

Code: ML, MAP and the posterior predictive

```
import numpy as np

rng = np.random.default_rng(3)
sigma = 1.0                 # known noise standard deviation
m0, s0 = 0.0, 0.5           # prior on the mean: N(m0, s0^2)
x = rng.normal(0.8, sigma, 5)
n = len(x)

mu_ml = x.mean()
post_var = 1 / (1 / s0**2 + n / sigma**2)
post_mean = post_var * (m0 / s0**2 + x.sum() / sigma**2)   # = MAP for a Gaussian
pred_var = sigma**2 + post_var                              # predictive variance

# brute-force check on a grid: posterior is proportional to likelihood x prior
grid = np.linspace(-3, 3, 200001)
log_post = (-0.5 * ((x[:, None] - grid) ** 2).sum(0) / sigma**2
            - 0.5 * (grid - m0) ** 2 / s0**2)
w = np.exp(log_post - log_post.max())
w /= w.sum()
print(f"ML {mu_ml:.3f} | MAP/posterior mean {post_mean:.3f} "
      f"(grid {np.sum(w * grid):.3f}) | posterior sd {np.sqrt(post_var):.3f}")
print(f"predictive sd {np.sqrt(pred_var):.3f} vs plug-in sd {sigma:.3f}")
```

With five samples the ML mean is 0.577, the posterior mean is 0.320 (the grid integration agrees), and the posterior standard deviation is 0.333. The predictive standard deviation, 1.054, is 5% wider than the plug-in value: small here, but it grows quickly as data get scarcer or dimension grows.

Classifiers built from densities

Plain version. Fit one density per class, multiply by the class frequency, and pick the class with the largest product. How you model each density decides what the classifier can learn.

Gaussian plug-in classifiers. Estimate $\hat{\mu}_k$, $\hat{\Sigma}_k$ and $\hat{P}(c_k)$ and insert them into the Gaussian discriminant of PR 01. Separate covariances give QDA, a shared (pooled) covariance gives LDA, and the boundary geometry is exactly as derived there. For computation, use a Cholesky factorization or a linear solve rather than an explicit inverse.

Naive Bayes. Assume that the features are **conditionally independent given the class**:

$$p(x \mid c_k) = \prod_{j=1}^{d} p(x_j \mid c_k).$$

This needs only d one-dimensional densities per class instead of one d-dimensional density, so it can be estimated from very little data. It does *not* assume that features are independent in the overall population; it assumes that the class explains all of their dependence. Gaussian naive Bayes gives each coordinate a univariate Gaussian per class; its boundary is quadratic when the per-class variances differ. When features are strongly correlated (or duplicated), naive Bayes counts the same evidence several times, so its posteriors become overconfident even when its decisions remain reasonable.

Generative versus discriminative. Ng and Jordan [15] compared naive Bayes with logistic regression, its discriminative counterpart. They showed that logistic regression has a lower (or equal) error in the limit of infinite data, but naive Bayes approaches its own, higher, asymptotic error much faster: with a number of samples logarithmic in the dimension, rather than linear. So the generative model often wins with few samples and the discriminative one with many. This is a useful mental model for the trade-off between the two routes.

Non-parametric estimation: one formula, three estimators

Plain version. To estimate density at a point, look at a small region around it and count how many samples fall inside. Density is "count divided by (number of samples times region size)". You can fix the region and count, or fix the count and grow the region.

Precise version. The probability that a sample falls in a small region $\mathcal{R}$ around x is $P = \int_{\mathcal{R}} p(x')\, dx' \approx p(x)\, V$, where V is the volume of $\mathcal{R}$. If k of the N samples fall inside, $k \approx NP$, so [1, 2]

$$\hat{p}(x) = \frac{k}{N\,V}.$$

Two approximations are hidden here, and they pull in opposite directions. $P \approx p(x)V$ needs a *small* region (otherwise $\hat{p}$ is a blurred average of p: **bias**). $k \approx NP$ needs *many* samples inside (otherwise the count is noisy: **variance**). Duda, Hart and Stork [1] show that the estimate converges to $p(x)$ as $N \to \infty$ if the region shrinks ($V_N \to 0$) while the count grows ($k_N \to \infty$) more slowly than N ($k_N/N \to 0$). There are two ways to satisfy this:

- **fix V and count k**: histograms and Parzen windows (kernel density estimation);
- **fix k and measure V**: k-nearest-neighbor density estimation.

Histograms

Divide the axis into bins of width w and set $\hat{p}(x) = n_i/(Nw)$ for x in bin i with count n_i. Histograms are simple and fast, but have three weaknesses visible in Figure 2. The estimate is discontinuous at bin edges. It depends on the **origin** of the grid as well as the width: shifting all bins by 0.3 changes the picture. And in d dimensions the number of bins grows like $(1/w)^d$, so most bins

are empty long before d reaches ten. Scott's analysis [8] shows that the bin width minimizing the mean integrated squared error shrinks only like $N^{-1/3}$, so more data buy finer bins slowly.

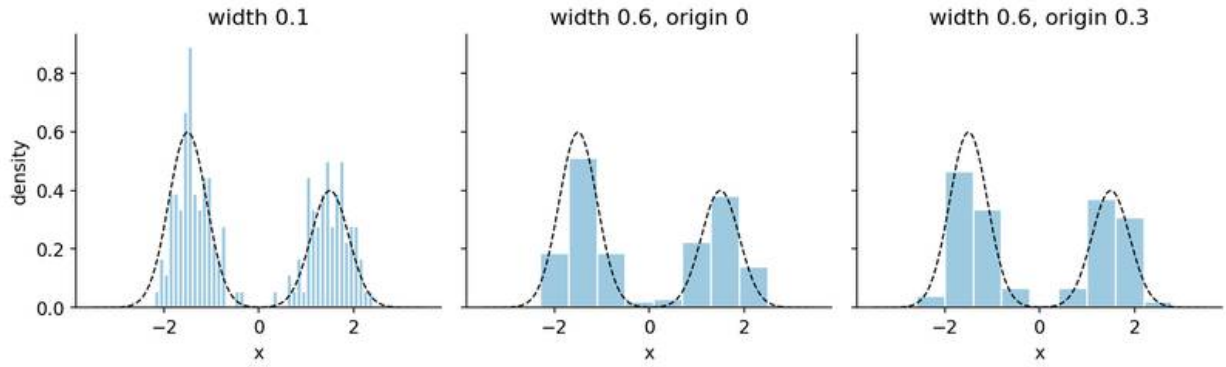


Figure 2 — The same 180 samples (dashed: true density). Narrow bins are noisy; wide bins depend on where the grid starts. The two right panels differ only in the bin origin.

Parzen windows and kernel density estimation

The **Parzen window** or **kernel density estimate (KDE)**, introduced by Rosenblatt [5] and analysed by Parzen [6], centers a window on every sample instead of fixing a grid:

$$\hat{p}_h(x) = \frac{1}{N\,h^d} \sum_{n=1}^{N} K\!\left(\frac{x - x_n}{h}\right).$$

Here K is the **kernel**, a function with $K(u) \geq 0$ and $\int K(u)\,du = 1$ (so $\hat{p}_h$ is a valid density), and $h > 0$ is the **bandwidth**, which plays the role of the bin width. With a box kernel ($K = 1$ inside the unit cube) the formula is literally "count the samples within a cube of side h around x", the fixed-V estimator above. A Gaussian kernel $K(u) = (2\pi)^{-d/2} e^{-\|u\|^2/2}$ replaces the hard count with a smooth weight, giving a smooth estimate (Figure 3, left). In practice the shape of the kernel matters much less than the bandwidth.

Bias and variance. For a smooth density in one dimension, the bias of a KDE grows like h^2 (times the curvature $p''(x)$) and the variance like $1/(Nh)$. Minimizing their sum gives an optimal bandwidth proportional to $N^{-1/5}$ and a mean integrated squared error that falls like $N^{-4/5}$, slower than the N^{-1} of a correctly specified parametric model [7, 8]. In d dimensions the optimal bandwidth scales like $N^{-1/(d+4)}$ and the error like $N^{-4/(d+4)}$. The slowdown with d is one face of the curse of dimensionality.

Choosing the bandwidth. This is the important practical decision. Common options:

- **Rules of thumb.** Silverman's rule $h = 0.9\,\min(\hat{\sigma},\ \mathrm{IQR}/1.34)\,N^{-1/5}$ [7], where $\hat{\sigma}$ is the sample standard deviation and IQR the interquartile range, is optimal if the data are Gaussian. For multimodal data it oversmooths.
- **Cross-validation.** Choose h to maximize the log-likelihood of held-out samples (likelihood

cross-validation), or to minimize an unbiased estimate of the integrated squared error (least-squares cross-validation, analysed by Scott and Terrell [9]).

- **Plug-in methods.** Sheather and Jones [10] estimate the unknown curvature term in the optimal-bandwidth formula from the data and plug it in; their selector is a widely used reliable default in one dimension.

Whatever rule you use, choose h without looking at the final test data. Figure 4 shows why the choice matters: on a bimodal sample, $h = 0.08$ produces spurious bumps, $h = 1.2$ merges the two modes into one, and the validation log-likelihood correctly prefers $h = 0.2$.

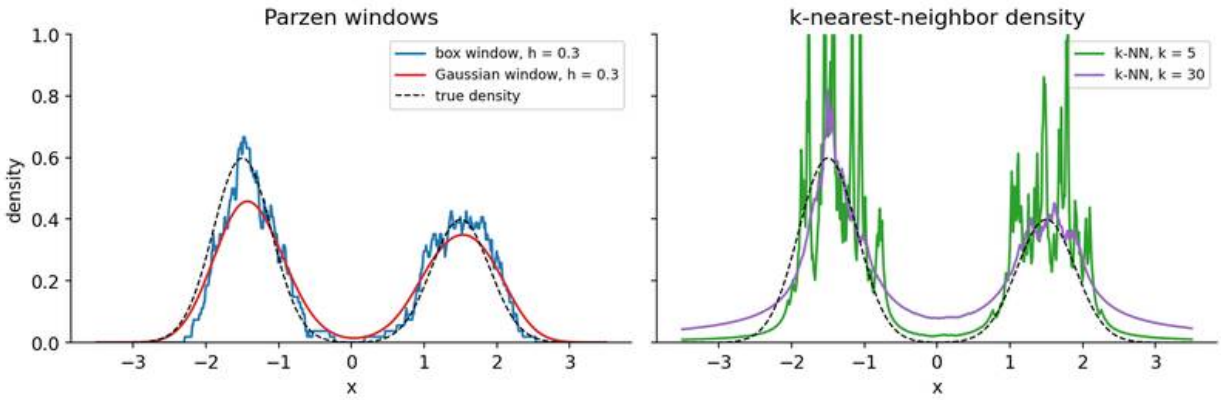


Figure 3 — Left: a box window gives a jagged estimate, a Gaussian window of the same width a smooth one. Right: k-NN density estimates adapt their width to the data but are spiky for small k and have tails that decay too slowly.

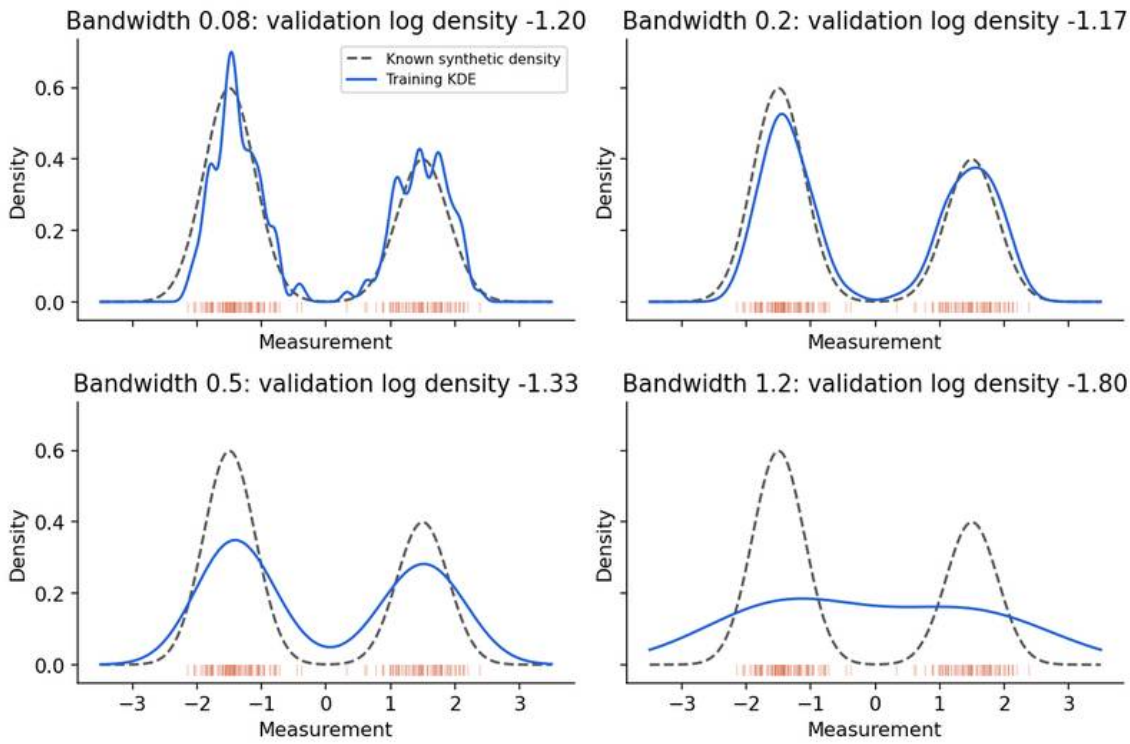


Figure 4 — KDE with four bandwidths on the training sample used in the code below. Narrow bandwidths show sampling fluctuations; wide bandwidths blur the modes. The mean validation log-density (in each title) is highest for $h = 0.2$.

Code: bandwidth selection and a Beta posterior

This example uses scikit-learn's `KernelDensity` [26] on a synthetic bimodal distribution, separate validation samples, Silverman's rule for comparison, and the closed-form Beta posterior from the previous section.

```
import numpy as np
from scipy.stats import beta
from sklearn.neighbors import KernelDensity

rng = np.random.default_rng(4)
def sample(n):
    means = np.where(rng.random(n) < 0.6, -1.5, 1.5)
    return (means + rng.normal(0, 0.4, n)).reshape(-1, 1)
train, validation = sample(180), sample(100)
bandwidths = [0.08, 0.2, 0.5, 1.2]
scores = []
for h in bandwidths:
    model = KernelDensity(bandwidth=h).fit(train)
    scores.append(model.score(validation) / len(validation))
```

```
best = bandwidths[int(np.argmax(scores))]

# Silverman's rule of thumb for comparison
x = train.ravel()
iqr = np.subtract(*np.percentile(x, [75, 25]))
h_silverman = 0.9 * min(x.std(ddof=1), iqr / 1.34) * len(x) ** (-1 / 5)

posterior = beta(2, 1)  # uniform prior + one success
print("validation mean log densities:",
      {h: round(float(s), 3) for h, s in zip(bandwidths, scores)})
print("selected bandwidth:", best, "| Silverman's rule:", round(h_silverman, 3))
print("posterior mean:", posterior.mean())
print("95% credible interval:", posterior.ppf([0.025, 0.975]))
assert np.isclose(posterior.mean(), 2 / 3)
```

Validation selects $h = 0.2$, while Silverman's rule gives $h \approx 0.475$. The rule assumes a single Gaussian; the spread of this sample comes mostly from the gap *between* the two modes, so the rule oversmooths. The Beta posterior after one success under a uniform prior has mean $2/3$ and a 95% credible interval of roughly $[0.16, 0.99]$: one observation tells us very little.

A credible interval describes uncertainty under the assumed prior and likelihood; it is not automatically a calibrated frequentist coverage guarantee. Likewise, a good density score does not by itself guarantee a good classification boundary: some of the modelled variation may be irrelevant to the labels.

k-nearest-neighbor density estimation

Fix k instead of V: around x, grow a ball until it contains exactly k samples, and let $V_k(x)$ be its volume. Then [11]

$$\hat{p}_k(x) = \frac{k}{N\, V_k(x)}.$$

Loftsgaarden and Quesenberry [11] showed that this estimate is consistent when $k \to \infty$ and $k/N \to 0$. Its bandwidth adapts automatically: narrow where data are dense, wide where they are sparse. But Figure 3 (right) shows its flaws. For small k it is very spiky, because V_k is tiny near clusters of samples. In the tails, V_k grows only linearly with the distance in 1-D, so $\hat{p}_k$ decays like $1/|x|$, and the estimate **does not integrate to one** (its integral diverges). It is therefore a poor *density* estimate, yet, as the next section shows, an excellent basis for *classification*.

The k-nearest-neighbor classifier

Plain version. To classify a new point, find the k training points closest to it and take a majority vote. No model is trained; the training set *is* the model.

Precise version. Draw a ball around x that contains k samples in total, of which k_c belong to class c. Applying the $k/(NV)$ estimate separately to each class, with N_c samples in class c, gives

$$\hat{p}(x \mid c) = \frac{k_c}{N_c V}, \qquad \hat{P}(c) = \frac{N_c}{N}, \qquad \hat{p}(x) = \frac{k}{NV} \quad \Longrightarrow \quad \hat{P}(c \mid x) = \frac{\hat{p}(x \mid c)\hat{P}(c)}{\hat{p}(x)} = \frac{k_c}{k}.$$

The volume cancels, and the posterior estimate is simply the fraction of neighbors from each class. The Bayes rule then becomes the **majority vote**. With $k = 1$ it is the **nearest-neighbor rule**.

How good is it? Cover and Hart [12] proved that, as $N \to \infty$, the error $P_{1\mathrm{NN}}$ of the 1-NN rule satisfies

$$P^* \le P_{1\mathrm{NN}} \le P^*\Big(2 - \frac{K}{K-1}P^*\Big) \le 2P^*,$$

where P^* is the Bayes error and K the number of classes. With unlimited data, half of the information needed for optimal classification is contained in the single nearest neighbor. As k grows (with $k/N \to 0$), the k-NN error approaches P^* itself. This is an asymptotic statement; with finite data and many dimensions, things are much less rosy.

Choosing k. Small k gives low bias but high variance: the 1-NN boundary in Figure 5 wraps around individual noisy points. Large k smooths the boundary but eventually ignores real structure. Choose k by cross-validation. Two practical details: use an odd k for two classes to avoid ties, and **standardize the features** first, because the Euclidean distance is dominated by whichever feature has the largest numeric range. In PR 04, kernels and nearest neighbors are compared with other nonlinear classifiers, and learned distance metrics are discussed there and in later chapters.

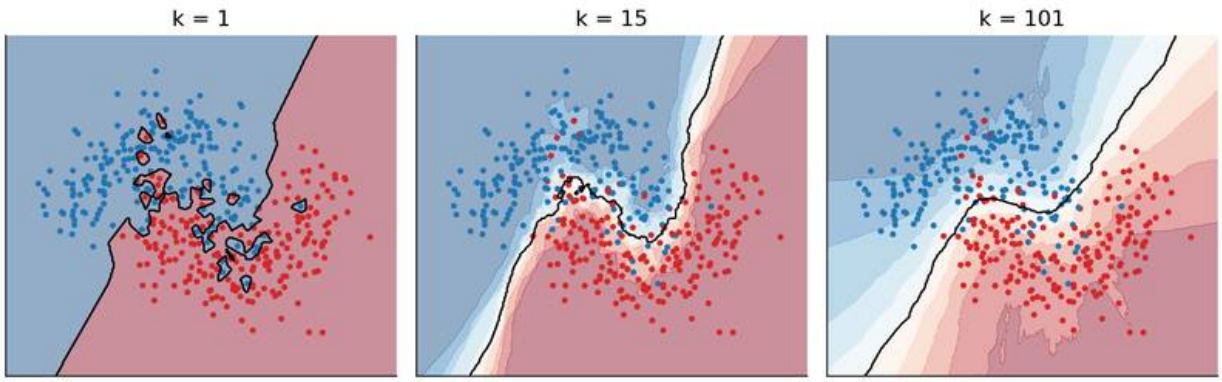


Figure 5 — k-NN on 400 noisy two-moons points. Shading shows the vote fraction k_c/k; the black curve is the 0.5 boundary. k = 1 overfits individual points, k = 101 starts to straighten the moons.

Code: k-NN versus Gaussian classifiers

```
import numpy as np
from sklearn.datasets import make_moons
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis

X, y = make_moons(1000, noise=0.3, random_state=0)
```

```
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.5, random_state=0)

search = GridSearchCV(KNeighborsClassifier(),
                      {"n_neighbors": [1, 3, 5, 9, 15, 31, 61, 101]}, cv=5)
search.fit(Xtr, ytr)                      # k chosen on training folds only
print("cross-validated k:", search.best_params_["n_neighbors"])
models = {"1-NN": KNeighborsClassifier(1),
          f"{search.best_params_['n_neighbors']}-NN": search.best_estimator_,
          "Gaussian naive Bayes": GaussianNB(),
          "QDA": QuadraticDiscriminantAnalysis()}
for name, m in models.items():
    m.fit(Xtr, ytr)
    print(f"{name:22s} test error {1 - m.score(Xte, yte):.3f}")
```

Cross-validation picks $k = 31$, with a test error of 10.6%, against 12.4% for 1-NN. The Gaussian models, whose single-ellipse class shapes cannot follow the moons, are worse (17.4% for naive Bayes, 17.6% for QDA). On this 2-D problem, the non-parametric method wins because the parametric form is wrong. The next section shows why the advantage shrinks as dimension grows.

The curse of dimensionality

Plain version. Local methods need neighbors that are close. In high dimensions, there is no such thing as close: to collect even a small fraction of the data you must reach across most of the space, and the nearest and farthest points end up at almost the same distance.

Precise version. The phrase "curse of dimensionality" covers several related effects; Bishop [2, Section 1.4] gives geometric illustrations. Three are easy to check (Figure 6):

1. **Neighborhoods must be huge.** For data uniform in the unit cube $[0,1]^d$, a sub-cube that contains a fraction r of the data has edge length $r^{1/d}$. To capture 10% of the data, the edge is 0.32 in 2-D, 0.79 in 10-D and 0.98 in 100-D. A "local" neighborhood spans almost the full range of every feature, so the estimate is no longer local.
2. **Volume is in the corners and the shell.** The volume of the unit ball divided by the volume of its enclosing cube $[-1,1]^d$ tends to zero rapidly, and for a ball almost all volume lies in a thin shell near its surface. Samples from high-dimensional Gaussians sit at nearly the same distance from the mean, not near it.
3. **Distances concentrate.** Beyer et al. [13] proved that, under broad conditions on the data distribution, the ratio of the farthest to the nearest neighbor distance tends to one as d grows, so "nearest neighbor" stops being meaningful. In the simulation below, the ratio of nearest to farthest distance rises from 0.02 in 2-D to about 0.7 in 100-D.

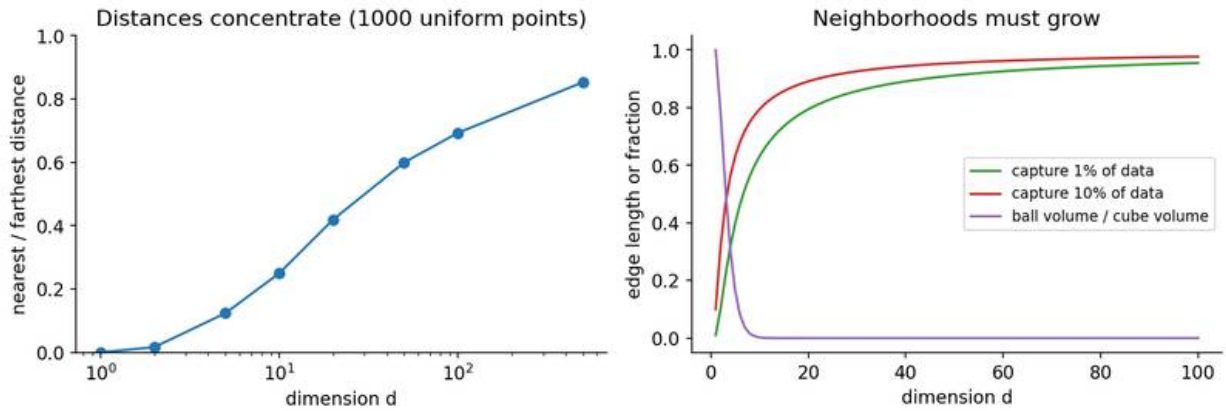


Figure 6 — Left: distance concentration for 1,000 uniform points and 50 queries. Right: edge length of a sub-cube that captures 1% or 10% of uniform data, and the fraction of the cube [-1, 1]^d occupied by the unit ball.

Code: distance concentration

```
import numpy as np

rng = np.random.default_rng(0)
for d in [1, 2, 10, 100, 1000]:
    pts = rng.random((1000, d))          # 1000 uniform points in [0, 1]^d
    q = rng.random((50, d))              # 50 query points
    dist = np.sqrt(((q[:, None, :] - pts[None]) ** 2).sum(-1))
    ratio = (dist.min(1) / dist.max(1)).mean()
    edge = 0.1 ** (1 / d)                # sub-cube edge holding 10% of the volume
    print(f"d={d:5d}: nearest/farthest = {ratio:.3f}, "
          f"edge for 10% of data = {edge:.3f}")
```

The nearest/farthest ratio is 0.017 in 2-D, 0.27 in 10-D, 0.70 in 100-D and 0.89 in 1000-D.

Why does anything work in high dimensions, then? Real data are not uniform. Images, sounds and text lie close to low-dimensional structures (manifolds, clusters) inside their huge ambient spaces, and their informative directions are far fewer than their raw coordinates. Practical defences are therefore: reduce or learn the representation first (PR 05: features and dimensionality), constrain the model (shared or diagonal covariances, naive Bayes), and regularize. Nearest-neighbor methods work very well today precisely *because* they are applied to learned embeddings rather than raw pixels, as the Modern view explains.

Modern view

Density estimation has split into two streams since the textbooks were written: classical statistics refined KDE and covariance estimation, while deep learning built density models that scale to

images, and nearest-neighbor search became infrastructure.

Kernel density estimation, revisited. Chen's tutorial [16] is a gentle modern introduction: convergence rates, bandwidth selection, confidence bands and how to handle their bias, and recent work on estimating geometric and topological features of a density (modes, level sets, ridges) with KDE. Wang and Scott [17] review non-parametric density estimators designed for high-dimensional data, including applications such as modal clustering, and list open research directions. Scott's monograph [8] and Silverman's classic [7] remain the references for the theory.

High-dimensional covariance. The singular-covariance problem of this chapter is a whole research area. Fan, Liao and Liu [18] review estimators of large covariance and precision (inverse covariance) matrices, focusing on rank-based methods and factor-model-based methods; Ledoit–Wolf shrinkage [14] is still the default regularizer in libraries.

Deep density models. Normalizing flows build a flexible density by passing a simple base distribution through a chain of invertible, differentiable maps; the change-of-variables formula then gives an exact log-likelihood. Papamakarios et al. [19] and Kobyzev, Prince and Brubaker [20] are the two standard reviews: the first emphasizes design principles, expressive power and computational trade-offs, the second the construction of flows and open questions. Autoregressive models, VAEs and diffusion models are other ways to learn high-dimensional densities; see ML 06: generative models and ML 07: diffusion and flow matching. Murphy's two-volume text [4] covers both the classical estimators of this chapter and these modern models in one notation.

Nearest neighbors at scale. k-NN is now everywhere, but on learned embeddings instead of raw features. Johnson, Douze and Jégou [21] showed how to run billion-scale similarity search on GPUs, and the Faiss library paper [22] describes the trade-off space of vector search (accuracy, speed, memory) and the indexing methods behind modern vector databases. Papernot and McDaniel's **deep k-NN** [23] compares a test input with its training neighbors in every layer's representation, using label disagreement among neighbors as a confidence and robustness signal. For out-of-distribution detection, Sun et al. [24] use the plain k-th nearest-neighbor distance in feature space, without any distributional assumption, and report large improvements over a parametric Mahalanobis-distance baseline. That parametric baseline, from Lee et al. [25], fits class-conditional Gaussians with shared covariance to network features, which is LDA from PR 01. The parametric versus non-parametric choice of this chapter is being replayed on top of deep features.

Open problems. Reliable density estimation in high dimension remains hard. Nalisnick et al. [27] showed that flow-based models, VAEs and PixelCNNs trained on one image dataset can assign *higher* likelihood to a different dataset (for example, a model trained on a dataset of everyday objects scores a dataset of house-number photos higher), so likelihood alone is not a safe novelty score. Bandwidth or k selection for embeddings, and calibrated uncertainty for non-parametric estimates, are active questions.

Key takeaways

- Generative classifiers need $\hat{p}(x \mid c_k)$ and $\hat{P}(c_k)$; the estimator's assumptions decide the boundary's shape.
- ML estimates for a Gaussian are the sample mean and the divisor-N covariance; with $N \leq d$ samples the covariance is singular, and shrinkage (e.g. Ledoit–Wolf) or shared/diagonal structure is needed.
- MAP keeps the posterior mode; full Bayesian prediction integrates over parameters and is wider than plug-in prediction when data are scarce. Conjugate priors give closed forms: precisions add for a Gaussian mean, counts add for a Beta–Bernoulli rate.
- Non-parametric estimates all follow from $\hat{p} = k/(NV)$: histograms and Parzen/KDE fix V, k-NN fixes k. The bandwidth or k matters more than the kernel; choose it by cross-validation or a justified rule, never on test data.
- The k-NN classifier estimates $P(c \mid x) \approx k_c/k$; asymptotically 1-NN is within a factor two of the Bayes error.
- In high dimensions neighborhoods stop being local and distances concentrate; local methods work in practice only on low-dimensional or learned representations.

Exercises

1. **Counting parameters.** Count the mean and covariance parameters of one 64-dimensional Gaussian. Explain why fitting it to 30 samples fails, and give two ways to fix it.

   Hint

   64 means plus $64 \cdot 65/2 = 2{,}080$ covariance entries. With 30 samples, $\hat{\Sigma}$ has rank at most 29, so it is singular. Fixes: shrinkage towards τI (Ledoit–Wolf), a diagonal covariance (naive Bayes), a covariance shared across classes, or dimensionality reduction first.

2. **Beta–Bernoulli update.** Start with $\text{Beta}(2, 3)$, observe three successes and two failures, and compute the posterior, the MAP estimate and the posterior predictive probability of success.

   Hint

   Posterior $\text{Beta}(5, 5)$. MAP (mode) $= (5-1)/(5+5-2) = 0.5$. Predictive $= 5/10 = 0.5$. Compare with ML, $3/5 = 0.6$.

3. **Precision-weighted means.** For the Gaussian mean with prior $\mathcal{N}(0, 1)$ and noise variance $\sigma^2 = 4$, after how many samples does the data weight equal the prior weight? What is the posterior standard deviation then?

   Hint

   Prior precision 1; each sample adds $1/4$. Equal weight at $N = 4$. Posterior precision 2, so $s_N = 1/\sqrt{2} \approx 0.71$.

4. **Duplicated features.** Duplicate one feature in Gaussian naive Bayes. Predict what happens to accuracy and to the confidence of the posteriors, then check with scikit-learn on `load_breast_cancer`.

   Hint

   The extra coordinate adds no new information, but the factorized likelihood counts it twice, squaring its likelihood ratio. Decisions may change a little; posteriors move closer to 0 or 1 (more overconfident). Compare `predict_proba` histograms and the log-loss before and after.

5. **k-NN density is not a density.** Show that in 1-D the k-NN estimate $\hat{p}_k(x)$ decays like $1/|x|$ far from the data, and conclude that $\int \hat{p}_k(x)\,dx$ diverges. Why does this not hurt the k-NN classifier?

   Hint

   Far from all samples, the distance to the k-th neighbor is about $|x|$, so $V_k \approx 2|x|$ and $\hat{p}_k \approx k/(2N|x|)$, whose integral is logarithmically divergent. The classifier only uses ratios k_c/k at the same point, where the common volume cancels.

6. **Bandwidth rules.** Generate 500 samples from a mixture of $\mathcal{N}(-2, 0.5^2)$ and $\mathcal{N}(2, 0.5^2)$. Compare Silverman's rule, likelihood cross-validation (`GridSearchCV` over `KernelDensity`) and, if you have `statsmodels`, a plug-in selector. Which oversmooths, and why?

   Hint

   Silverman's rule uses the overall spread (about 2.1 here), which is dominated by the gap between the modes, so it gives a bandwidth several times too large for each mode. Cross-validated likelihood picks something close to the within-mode optimum.

References

1. R. O. Duda, P. E. Hart and D. G. Stork, *Pattern Classification*, 2nd ed., Wiley, 2001 (Ch. 3, maximum-likelihood and Bayesian estimation; Ch. 4, non-parametric techniques). publisher page
2. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006 (Sections 1.4, 2.1–2.3, 2.5 and 4.2). official page and free PDF
3. S. Theodoridis and K. Koutroumbas, *Pattern Recognition*, 4th ed., Academic Press, 2008 (Ch. 2, density estimation and Bayesian classification). publisher page
4. K. P. Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022. official page
5. M. Rosenblatt, "Remarks on some nonparametric estimates of a density function," *The Annals of Mathematical Statistics*, vol. 27, no. 3, pp. 832–837, 1956. doi
6. E. Parzen, "On estimation of a probability density function and mode," *The Annals of Mathematical Statistics*, vol. 33, no. 3, pp. 1065–1076, 1962. doi

7. B. W. Silverman, *Density Estimation for Statistics and Data Analysis*, Chapman and Hall, 1986 (reissued by Routledge, 2018). doi
8. D. W. Scott, *Multivariate Density Estimation: Theory, Practice, and Visualization*, 2nd ed., Wiley, 2015. doi
9. D. W. Scott and G. R. Terrell, "Biased and unbiased cross-validation in density estimation," *Journal of the American Statistical Association*, vol. 82, no. 400, pp. 1131–1146, 1987. doi
10. S. J. Sheather and M. C. Jones, "A reliable data-based bandwidth selection method for kernel density estimation," *Journal of the Royal Statistical Society, Series B*, vol. 53, no. 3, pp. 683–690, 1991. doi
11. D. O. Loftsgaarden and C. P. Quesenberry, "A nonparametric estimate of a multivariate density function," *The Annals of Mathematical Statistics*, vol. 36, no. 3, pp. 1049–1051, 1965. doi
12. T. M. Cover and P. E. Hart, "Nearest neighbor pattern classification," *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967. doi
13. K. Beyer, J. Goldstein, R. Ramakrishnan and U. Shaft, "When is 'nearest neighbor' meaningful?," in *Database Theory — ICDT'99*, Lecture Notes in Computer Science, Springer, 1999. doi
14. O. Ledoit and M. Wolf, "A well-conditioned estimator for large-dimensional covariance matrices," *Journal of Multivariate Analysis*, vol. 88, no. 2, pp. 365–411, 2004. doi
15. A. Y. Ng and M. I. Jordan, "On discriminative vs. generative classifiers: a comparison of logistic regression and naive Bayes," in *Advances in Neural Information Processing Systems* (*NIPS*), 2001. paper
16. Y.-C. Chen, "A tutorial on kernel density estimation and recent advances," *Biostatistics & Epidemiology*, vol. 1, no. 1, pp. 161–187, 2017. doi
17. Z. Wang and D. W. Scott, "Nonparametric density estimation for high-dimensional data — algorithms and applications," *WIREs Computational Statistics*, vol. 11, no. 4, 2019. doi
18. J. Fan, Y. Liao and H. Liu, "An overview of the estimation of large covariance and precision matrices," *The Econometrics Journal*, vol. 19, no. 1, pp. C1–C32, 2016. doi
19. G. Papamakarios, E. Nalisnick, D. J. Rezende, S. Mohamed and B. Lakshminarayanan, "Normalizing flows for probabilistic modeling and inference," *Journal of Machine Learning Research*, vol. 22, no. 57, pp. 1–64, 2021. arXiv
20. I. Kobyzev, S. J. D. Prince and M. A. Brubaker, "Normalizing flows: an introduction and review of current methods," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, no. 11, pp. 3964–3979, 2021. doi
21. J. Johnson, M. Douze and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021. doi
22. M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, M. Lomeli, L. Hosseini and H. Jégou, "The Faiss library," arXiv:2401.08281, 2024. arXiv
23. N. Papernot and P. McDaniel, "Deep k-nearest neighbors: towards confident, interpretable and robust deep learning," arXiv:1803.04765, 2018. arXiv
24. Y. Sun, Y. Ming, X. Zhu and Y. Li, "Out-of-distribution detection with deep nearest neighbors," in *Proc. International Conference on Machine Learning* (*ICML*), PMLR 162, 2022. paper
25. K. Lee, K. Lee, H. Lee and J. Shin, "A simple unified framework for detecting out-of-distribution samples and adversarial attacks," in *Advances in Neural Information Processing Systems* (*NeurIPS*), 2018. paper
26. scikit-learn developers, "Density estimation," scikit-learn user guide. documentation

27. E. Nalisnick, A. Matsukawa, Y. W. Teh, D. Gorur and B. Lakshminarayanan, “Do deep generative models know what they don’t know?,” in *Proc. International Conference on Learning Representations (ICLR)*, 2019. arXiv

PR 03 • Linear Models for Regression and Classification

Linear discriminants from first principles: least squares, Fisher's discriminant, the perceptron, logistic regression and the maximum-margin SVM with its dual, unified through their loss functions.

Online edition: https://blog.shyandram.dev/en/learn/pattern-recognition/pr-03-linear-models/

What you'll learn

- Read the geometry of a linear discriminant (weight vector, bias, signed distance) and the pitfalls of building multiclass classifiers from two-class ones.
- Derive least squares, ridge regression and their Bayesian interpretation, and see why least squares is a fragile classifier.
- Derive Fisher's linear discriminant and connect it to Gaussian LDA from PR 01.
- Train a perceptron, prove its convergence theorem, and explain why it can stop at any separating line.
- Fit logistic regression by Newton's method, and explain why separable data push its weights to infinity.
- Derive the maximum-margin SVM, its Lagrangian dual and its support vectors, and see all of these methods as different losses on the same linear score.

The big picture

Linear models are the first serious classifiers in every pattern-recognition course, and they remain the first thing to try on a new representation. If a straight boundary separates two categories in feature space, a large nonlinear classifier may add cost without useful accuracy. If a linear baseline fails, its errors help identify missing features or nonlinear structure. In modern deep learning, the final layer of almost every classifier is a linear model on learned features, and "how well does a linear classifier do on these features?" is the standard test of a representation.

> **Explain like I'm five**
>
> Each measurement gets a vote. Multiply each measurement by how strongly it votes, add everything up, and compare the total with a threshold: above means "yes", below means "no". All the methods in this chapter use exactly this voting machine. They only disagree about **how to choose the vote strengths**: make the totals match the right answers (least squares), spread the two groups apart (Fisher), fix mistakes one at a time (perceptron), predict honest probabilities (logistic regression), or leave the widest possible gap between the groups (SVM).

A linear score is $f(x) = w^\top \phi(x) + b$. The **feature map** ϕ might contain raw intensities, squared values, gradients or a frozen neural embedding. "Linear" refers to the parameters w and b, not

necessarily to the original measurement x: with $\phi(x) = (x, x^2)$ a linear model draws a curved boundary in x. This chapter follows Duda, Hart and Stork [1, Ch. 5], Bishop [2, Ch. 3–4, 7] and Theodoridis and Koutroumbas [3, Ch. 3], with Hastie et al. [4] for the statistical view. Kernels, which make ϕ implicit and very rich, are the topic of PR 04.

Linear discriminant functions

Plain version. A two-class linear classifier is a flat cut through feature space: a line in 2-D, a plane in 3-D, a hyperplane in general. The weight vector says which way the cut faces; the bias says where it sits.

Precise version. For two classes, decide c_1 if $g(x) = w^\top x + b > 0$ and c_0 otherwise. The decision boundary $g(x) = 0$ is a hyperplane H. Three facts follow from elementary geometry [1, 2]:

- w is **normal** to H: for two points x_a, x_b on H, $w^\top(x_a - x_b) = 0$.
- The **signed distance** from any x to H is $r = g(x)/\|w\|$. So $g(x)$ is proportional to how far, and on which side, x lies.
- The distance from the origin to H is $-b/\|w\|$; the bias shifts the plane without rotating it.

It is convenient to absorb the bias by **augmenting** the input with a constant: $\tilde{x} = (x, 1)$ and $\tilde{w} = (w, b)$, so that $g(x) = \tilde{w}^\top \tilde{x}$. We do this silently in code. With labels $y \in \{-1, +1\}$, a sample is correctly classified exactly when its **functional margin** $y\,g(x)$ is positive. Every method below can be written in terms of this quantity.

More than two classes. Two-class machines can be combined in two obvious ways: **one-versus-rest** (K classifiers, each separating one class from all others) and **one-versus-one** ($K(K-1)/2$ classifiers, one per pair, followed by voting). Both can leave regions of input space where the votes are ambiguous [2, Section 4.1.2]. The clean solution is a **linear machine**: one discriminant per class, $g_k(x) = w_k^\top x + b_k$, and assign x to the class with the largest score. Its decision regions are always convex and simply connected, because if x_a and x_b both have g_k largest, so does every point between them (each g_j is linear along the segment). The Gaussian LDA of PR 01, multiclass logistic regression and the final layer of a neural network are all linear machines.

Least squares and ridge regression

Plain version. For predicting a quantity, choose the weights that make the squared prediction errors as small as possible. If there are too many weights for the data, also penalize large weights.

Precise version. Let row n of the **design matrix** $\Phi \in \mathbb{R}^{N \times M}$ be the feature vector $\phi(x_n)^\top$, including a constant coordinate for the intercept, and let $t \in \mathbb{R}^N$ hold the targets. Least squares minimizes

$$\mathcal{L}(w) = \frac{1}{2}\|\Phi w - t\|_2^2,$$

whose gradient $\Phi^\top(\Phi w - t)$ vanishes at the **normal equations** $\Phi^\top \Phi w = \Phi^\top t$. Computing

$(\Phi^\top\Phi)^{-1}$ explicitly is unnecessary and squares the condition number; QR- or SVD-based solvers (`np.linalg.lstsq`) handle near-dependent columns more reliably. Assuming independent Gaussian noise of common variance on the targets, this objective is exactly the negative log-likelihood, so least squares is the maximum-likelihood estimate (PR 02). The noise assumption explains the statistical interpretation; it does not make squared error suitable for every target.

Ridge regression [8] adds a penalty $\frac{\lambda}{2}\|w\|^2$, giving

$$(\Phi^\top\Phi + \lambda I)\, w = \Phi^\top t.$$

The matrix on the left is invertible for any $\lambda > 0$, even when $M > N$. Usually the intercept is excluded from the penalty. Feature scaling matters, because the same penalty has different effects on coordinates measured in different units; fit scaling statistics on training data only.

A Bayesian connection

With a Gaussian prior $w \sim \mathcal{N}(0, \alpha^{-1}I)$ (precision α) and Gaussian noise of precision β, the posterior over weights is Gaussian with covariance and mean [2, Section 3.3]

$$S = (\alpha I + \beta\Phi^\top\Phi)^{-1}, \qquad m = \beta S\Phi^\top t.$$

For a new feature vector ϕ_*, the predictive distribution is

$$p(t_* \mid x_*, D) = \mathcal{N}(t_* \mid m^\top\phi_*,\ \beta^{-1} + \phi_*^\top S\phi_*).$$

The first variance term is observation noise; the second expresses uncertainty about the weights, which is larger along feature directions that the training data rarely visited. The posterior mean m equals the ridge solution with $\lambda = \alpha/\beta$: ridge is a MAP estimate, the same story as the Gaussian mean in PR 02.

Least squares for classification

Plain version. You can treat class labels as numbers ($+1$ and -1, or one-hot vectors) and fit them with least squares. It is fast and has a closed form, but it is the wrong tool: it punishes points for being *too* correct.

Precise version. For K classes, encode the targets as **one-hot** vectors $t_n \in \{0,1\}^K$ and fit a linear machine $g(x) = W^\top\tilde{x}$ by least squares, $W = (\tilde{X}^\top\tilde{X})^{-1}\tilde{X}^\top T$, then classify by the largest output. For two classes with suitable target values (N/N_1 for class 1 and $-N/N_0$ for class 0), the least-squares direction is exactly Fisher's direction derived in the next section [2, Section 4.1.5], so the method is not unreasonable.

It has two well-known failure modes:

1. **Sensitivity to points far on the correct side.** Squared error penalizes $g(x) = 5$ for a target of $+1$ as severely as $g(x) = -3$. Figure 1 adds twenty red points that are correctly classified but far away; least squares rotates its boundary to reduce their "error" and starts misclassifying blue points. Logistic regression barely moves.

2. **Masking with three or more classes.** When classes lie roughly along a line, the output for the middle class can be dominated everywhere by one of its neighbors, so the middle class is never predicted [4, Section 4.2].

In probabilistic terms, the Gaussian noise model behind least squares is simply wrong for binary targets. The remedy is to use a loss designed for classification, which the rest of the chapter does.

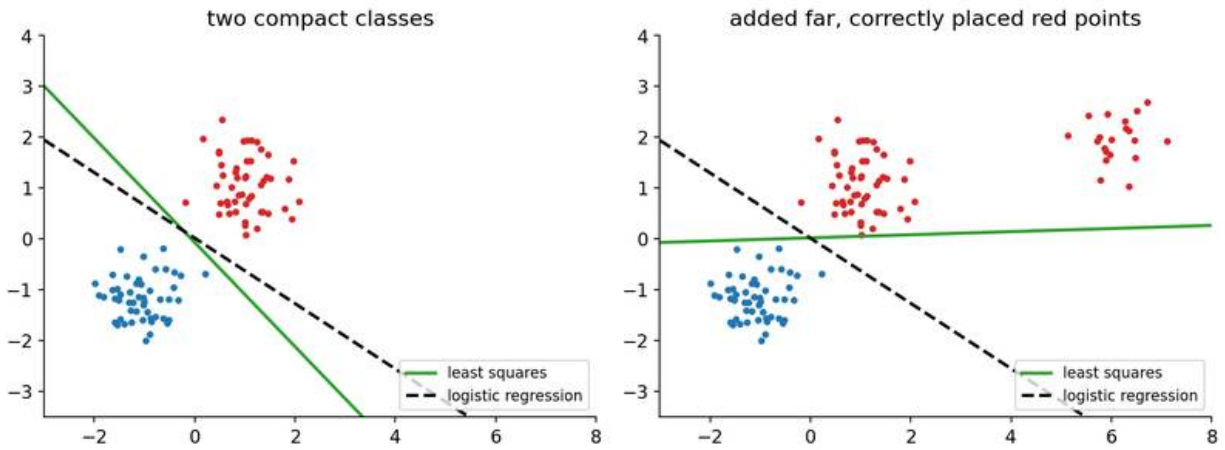


Figure 1 — Least squares versus logistic regression. Adding red points that are already correctly classified (right) pulls the least-squares boundary (green) into the blue class; logistic regression (dashed) is unaffected.

Fisher's linear discriminant

Plain version. Project all points onto one line and look at the two classes' 1-D histograms. A good line makes the class means far apart *and* each class narrow. Fisher's direction is the line that maximizes this ratio.

Precise version. Fisher [5] proposed choosing a projection $z = w^\top x$ that maximizes the separation of the projected class means relative to the spread within each class. Let $m_k = \frac{1}{N_k}\sum_{n\in c_k} x_n$ be the class means. The projected means are $w^\top m_k$, and the projected within-class scatter of class k is $\sum_{n\in c_k}(w^\top x_n - w^\top m_k)^2 = w^\top S_k w$, with $S_k = \sum_{n\in c_k}(x_n - m_k)(x_n - m_k)^\top$. **Fisher's criterion** is

$$J(w) = \frac{(w^\top m_1 - w^\top m_0)^2}{w^\top S_0 w + w^\top S_1 w} = \frac{w^\top S_B w}{w^\top S_W w},$$

where

$$S_B = (m_1 - m_0)(m_1 - m_0)^\top, \qquad S_W = S_0 + S_1$$

are the **between-class** and **within-class scatter matrices**. J does not depend on the length of w, only on its direction.

Derivation. Setting the gradient of the ratio to zero gives

$$(w^\top S_B w)\, S_W w = (w^\top S_W w)\, S_B w.$$

Both bracketed terms are scalars. Moreover $S_B w = (m_1 - m_0)\,[(m_1 - m_0)^\top w]$ always points along $m_1 - m_0$. Hence $S_W w \propto m_1 - m_0$, and since only the direction matters,

$$w^\star \propto S_W^{-1}(m_1 - m_0).$$

If S_W is a multiple of the identity (isotropic, uncorrelated classes), Fisher's direction is just the mean difference. Otherwise S_W^{-1} rotates it away from directions in which the classes are spread out. Figure 2 shows the effect for two elongated, correlated classes: projecting on the mean difference gives heavily overlapping histograms ($J = 0.70$), while Fisher's direction separates them well ($J = 3.18$).

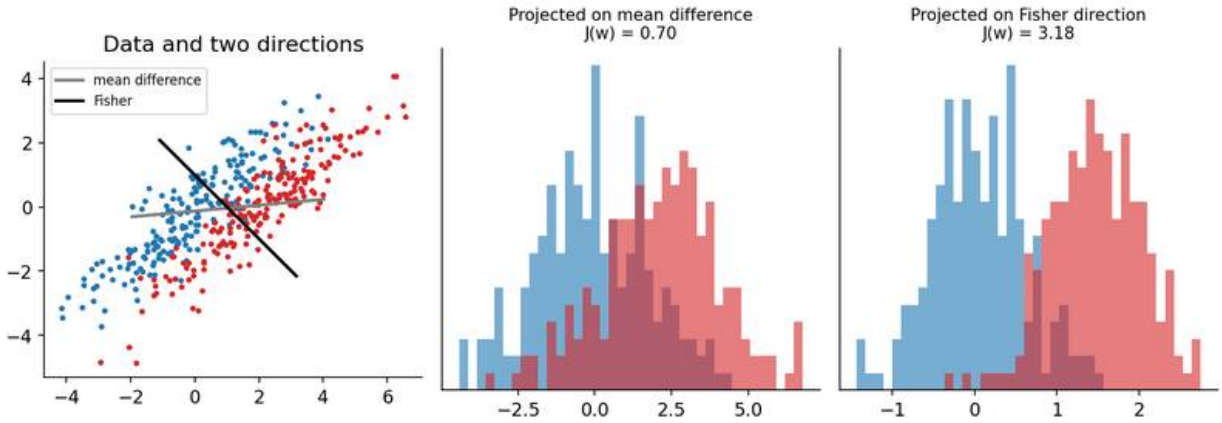


Figure 2 — Two correlated classes projected onto the mean difference (middle, $J = 0.70$) and onto Fisher's direction (right, $J = 3.18$). Per-class covariances were used for S_W; with equal class sizes this only rescales J.

Relationship with Gaussian LDA. Fisher's criterion makes no distributional assumption; it is a criterion for a *representation*. Yet its direction coincides with the Gaussian LDA direction $\Sigma^{-1}(\mu_1 - \mu_0)$ of PR 01 when S_W is (proportional to) the pooled within-class covariance. The two differ only in the threshold: Fisher leaves the cut point on the projected axis open, while Gaussian LDA fixes it from the class means and priors. So "LDA" is used for both. (In topic modelling, LDA also means latent Dirichlet allocation, which is unrelated.) Fisher projections do not by themselves provide calibrated probabilities.

More than two classes. With K classes and overall mean m, define $S_W = \sum_k S_k$ and $S_B = \sum_k N_k (m_k - m)(m_k - m)^\top$. The projection $W \in \mathbb{R}^{d \times (K-1)}$ that maximizes the multiclass criterion is spanned by the leading eigenvectors of $S_W^{-1} S_B$ [1, 2]. Because S_B is a sum of K rank-one terms constrained by the overall mean, its rank is at most $K - 1$, so Fisher can produce at most $K - 1$ useful features. This is the supervised dimensionality reduction of PR 05.

Code: Fisher's direction

```
import numpy as np
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

rng = np.random.default_rng(2)
cov = np.array([[3.0, 2.4], [2.4, 2.4]])
X0 = rng.multivariate_normal([0, 0], cov, 200)
X1 = rng.multivariate_normal([2, 0], cov, 200)
m0, m1 = X0.mean(0), X1.mean(0)
S_W = np.cov(X0.T) + np.cov(X1.T)       # within-class scatter (per-class covariances)
w = np.linalg.solve(S_W, m1 - m0)                          # Fisher direction

def J(v):                        # Fisher criterion for two classes
    return (v @ (m1 - m0)) ** 2 / (v @ S_W @ v)

lda = LinearDiscriminantAnalysis().fit(np.vstack([X0, X1]), np.repeat([0, 1], 200))
cos = w @ lda.coef_[0] / np.linalg.norm(w) / np.linalg.norm(lda.coef_[0])
angles = np.linspace(0, np.pi, 3601)
best = max(J(np.array([np.cos(a), np.sin(a)])) for a in angles)
print(f"J(mean difference) = {J(m1 - m0):.4f}, J(Fisher) = {J(w):.4f}, "
      f"grid maximum = {best:.4f}")
print(f"cosine between Fisher direction and LDA coef_: {cos:.6f}")
```

The closed form attains the grid maximum ($J = 3.1841$), more than four times the mean-difference value (0.6953), and its direction agrees with scikit-learn's Gaussian LDA coefficients to six decimals (cosine 1.000000).

The perceptron

Plain version. Start with any line. Go through the training points; whenever one is on the wrong side, nudge the line towards classifying it correctly. If the classes can be separated by a line, this process is guaranteed to stop.

Precise version. Rosenblatt's perceptron [6] uses labels $y_n \in \{-1, +1\}$ and augmented features $\tilde{x}_n$. It minimizes the **perceptron criterion**

$$J_P(w) = -\sum_{n \in \mathcal{M}} y_n \, w^\top \tilde{x}_n,$$

where $\mathcal{M}$ is the set of misclassified samples (those with $y_n w^\top \tilde{x}_n \le 0$). Each term is non-negative, and $J_P = 0$ exactly when every sample is correctly classified. Stochastic gradient descent on one misclassified sample at a time gives the update

$$w \leftarrow w + \eta \, y_n \tilde{x}_n.$$

With w initialized at zero, the learning rate $\eta > 0$ only rescales w and does not change the sequence of decisions, so $\eta = 1$ is common. The IP 13 perceptron section presents the same rule in the language of image patterns and connects it to multilayer networks.

Convergence theorem. Suppose the data are **linearly separable with margin** $\gamma > 0$: some unit vector u satisfies $y_n u^\top \tilde{x}_n \geq \gamma$ for all n. Let $R = \max_n \|\tilde{x}_n\|$. Then the perceptron (started at $w_0 = 0$) makes at most $(R/\gamma)^2$ updates. The proof takes three lines. After t updates:

- each update adds $y_n \tilde{x}_n^\top u \geq \gamma$ to $w^\top u$, so $w_t^\top u \geq t\gamma$;
- each update happens on a mistake, where $y_n w^\top \tilde{x}_n \leq 0$, so $\|w_t\|^2 = \|w_{t-1}\|^2 + 2y_n w_{t-1}^\top \tilde{x}_n + \|\tilde{x}_n\|^2 \leq \|w_{t-1}\|^2 + R^2$, hence $\|w_t\|^2 \leq tR^2$;
- by Cauchy–Schwarz, $t\gamma \leq w_t^\top u \leq \|w_t\| \leq \sqrt{t}\, R$, so $t \leq R^2/\gamma^2$.

The bound does not depend on the number of samples or the dimension, only on how "fat" the gap between the classes is relative to the data's size.

Limitations. If the classes overlap, the perceptron never stops; the weights keep cycling (variants that keep the best weights seen so far are a common practical fix). Even when it converges, it stops at the *first* separating hyperplane it reaches, which depends on initialization and data order (Figure 3, left). And its score is not a probability. In terms of losses, the perceptron uses $\max(0, -y\, g(x))$, which is zero for any correct classification, however marginal. Both logistic regression and the SVM fix this by asking for more than "just barely correct".

Code: perceptron updates versus the bound

```
import numpy as np
from sklearn.svm import SVC

rng = np.random.default_rng(1)
X = np.vstack([rng.normal([-1.0, -0.7], 0.6, (30, 2)),
               rng.normal([1.0, 0.7], 0.6, (30, 2))])
y = np.r_[-np.ones(30), np.ones(30)]
Phi = np.c_[X, np.ones(len(X))]          # augmented features (bias as last weight)

def perceptron(Phi, y, rng, max_epochs=1000):
    w, updates = np.zeros(Phi.shape[1]), 0
    for epoch in range(max_epochs):
        errors = 0
        for i in rng.permutation(len(y)):
            if y[i] * (w @ Phi[i]) <= 0:  # misclassified or on the boundary
                w += y[i] * Phi[i]        # learning rate 1 (scale is irrelevant)
                errors += 1
        updates += errors
        if errors == 0:
            return w, updates
    return w, updates

runs = [perceptron(Phi, y, np.random.default_rng(s)) for s in range(200)]
updates = np.array([u for _, u in runs])
print(f"updates over 200 random orders: mean {updates.mean():.1f}, "
      f"max {updates.max()}")
print("all runs separate the data:",
      all(np.all(np.sign(Phi @ w) == y) for w, _ in runs))
```

```
# convergence bound (R / gamma)^2, with gamma from the max-margin separator
svm = SVC(kernel="linear", C=1e6).fit(X, y)
u = np.r_[svm.coef_[0], svm.intercept_[0]]
gamma = np.min(y * (Phi @ u)) / np.linalg.norm(u)
R = np.max(np.linalg.norm(Phi, axis=1))
print(f"bound (R/gamma)^2 = {(R / gamma) ** 2:.1f}")
```

Over 200 random presentation orders the perceptron needs on average 2.7 updates (at most 10), and every run separates the data. The worst-case bound is 691: it is a guarantee, not a prediction. The bound needs a valid margin γ; we take it from a hard-margin SVM (the subject of the SVM section below). Because the SVM does not penalize the bias, the true maximum margin in the augmented space may be slightly larger, which would give a slightly tighter bound.

Logistic regression

Plain version. Instead of a hard yes/no, output a probability by squashing the linear score through an S-shaped curve. Fit the weights so that the probabilities assigned to the true labels are as high as possible.

Precise version. With labels $y \in \{0, 1\}$, binary logistic regression [7] models

$$P(y = 1 \mid x) = \sigma(w^\top \tilde{x}), \qquad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

The score $w^\top \tilde{x}$ is the **log-odds** $\ln \frac{P(y=1|x)}{P(y=0|x)}$. This form is not arbitrary. PR 01 showed that for Gaussian classes with a shared covariance the posterior is exactly $\sigma(w^\top x + w_0)$ with $w = \Sigma^{-1}(\mu_1 - \mu_0)$; the same holds for many other class-conditional families. Logistic regression keeps the functional form of that posterior but fits w directly, with $d + 1$ parameters instead of the $2d + d(d+1)/2 + 1$ of the Gaussian model. Ng and Jordan [18] analysed this generative–discriminative trade-off (PR 02).

Training. The negative log-likelihood of the Bernoulli model is the **cross-entropy**

$$E(w) = -\sum_{n=1}^{N} \Big[y_n \ln p_n + (1 - y_n) \ln(1 - p_n) \Big], \qquad p_n = \sigma(w^\top \tilde{x}_n).$$

Because $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, its gradient and Hessian are remarkably simple [2, Section 4.3]:

$$\nabla E = \sum_n (p_n - y_n)\, \tilde{x}_n = \tilde{X}^\top (p - y), \qquad H = \tilde{X}^\top R\, \tilde{X}, \quad R = \operatorname{diag}(p_n(1 - p_n)).$$

The gradient has the same "prediction minus target, times input" form as least squares. H is positive semidefinite, so E is convex and has no spurious local minima. Newton's method, $w \leftarrow w - H^{-1}\nabla E$, can be rewritten as a weighted least-squares problem with weights R that change every iteration: **iteratively reweighted least squares (IRLS)**. In code, compute losses from logits with a stable formula such as $\ln(1 + e^z) =$ `np.logaddexp(0, z)` rather than taking logarithms of rounded probabilities.

Separable data. If a hyperplane separates the training classes, scaling w up pushes every p_n closer to its label, so E keeps decreasing towards zero and the unregularized solution does not exist: $\|w\| \to \infty$. In practice, always regularize, e.g. add $\frac{\lambda}{2}\|w\|^2$ (scikit-learn's `C` is $1/\lambda$). Remarkably, *how* the weights diverge is well defined: plain gradient descent on separable data converges in direction to the maximum-margin SVM solution [19], which the Modern view returns to.

Multiclass. Replace the sigmoid with the **softmax** $P(y = k \mid x) = \exp(w_k^\top \tilde{x}) / \sum_j \exp(w_j^\top \tilde{x})$ and the cross-entropy with its K-class version. This is a linear machine with probabilistic outputs, and it is exactly the last layer plus loss of a standard neural classifier. Probability outputs are not automatically calibrated under label noise or distribution shift (PR 01).

Code: Newton's method (IRLS) and separable data

```
import numpy as np
from sklearn.linear_model import LogisticRegression

rng = np.random.default_rng(0)
X = np.vstack([rng.normal(-1, 1, (100, 2)), rng.normal(1, 1, (100, 2))])
y = np.r_[np.zeros(100), np.ones(100)]
Phi = np.c_[np.ones(len(X)), X]

def nll(w, lam):
    z = Phi @ w
    # log(1 + e^z) - y z, written stably with logaddexp
    return np.sum(np.logaddexp(0, z) - y * z) + 0.5 * lam * np.sum(w[1:] ** 2)

# Newton's method (iteratively reweighted least squares)
lam, w = 1.0, np.zeros(3)
for it in range(20):
    p = 1 / (1 + np.exp(-(Phi @ w)))
    grad = Phi.T @ (p - y) + lam * np.r_[0, w[1:]]
    H = Phi.T @ (Phi * (p * (1 - p))[:, None]) + lam * np.diag([0, 1, 1])
    step = np.linalg.solve(H, grad)
    w -= step
    if np.linalg.norm(step) < 1e-10:
        break
sk = LogisticRegression(C=1 / lam, tol=1e-10, max_iter=1000).fit(X, y)
print(f"Newton converged in {it + 1} iterations, loss {nll(w, lam):.4f}")
print("Newton:      ", np.round(w, 4))
print("scikit-learn:", np.round(np.r_[sk.intercept_, sk.coef_[0]], 4))
```

Newton's method converges in 8 iterations to the same weights as scikit-learn, $(0.0777, 1.7205, 1.5300)$ for (intercept, w_1, w_2). The penalty excludes the intercept, matching scikit-learn's convention.

The next snippet runs plain gradient descent on *unregularized* logistic loss for data that a line through the origin separates. It tracks the weight norm and the cosine between w and the max-margin direction.

```
import numpy as np
from sklearn.svm import LinearSVC

rng = np.random.default_rng(1)
X = np.vstack([rng.normal([2.0, 1.0], 0.6, (40, 2)),
               rng.normal([-2.0, -1.0], 0.6, (40, 2))])
y = np.r_[np.ones(40), -np.ones(40)]              # separable through the origin

# max-margin direction without bias (hard margin approximated by large C)
svm = LinearSVC(fit_intercept=False, loss="hinge", C=1e4, max_iter=200000).fit(X, y)
u = svm.coef_[0] / np.linalg.norm(svm.coef_[0])

w, lr = np.zeros(2), 0.1
for t in range(1, 100001):
    m = y * (X @ w)
    grad = -(X * (y / (1 + np.exp(m)))[:, None]).mean(0)   # logistic loss gradient
    w -= lr * grad
    if t in (10, 100, 1000, 10000, 100000):
        print(f"step {t:6d}: ||w|| = {np.linalg.norm(w):6.2f}, "
              f"cos(w, SVM) = {w @ u / np.linalg.norm(w):.5f}")
```

The norm grows without bound but only slowly (0.68 after 10 steps, 8.64 after 10^5), while the cosine with the SVM direction creeps from 0.87 to 0.98. This is the logarithmically slow convergence to the max-margin direction predicted by Soudry et al. [19].

Support vector machines: maximum margin

Plain version. Among all lines that separate the classes, pick the one with the widest empty "street" around it. Only the points on the edges of the street matter; move any other point (without crossing the street) and the answer does not change.

Precise version. Use labels $y_n \in \{-1, +1\}$ and score $f(x) = w^\top x + b$ (bias kept separate here). For a separating hyperplane, the distance of x_n to the boundary is $y_n f(x_n)/\|w\|$. Since (w, b) and (cw, cb) define the same plane, fix the scale so that the closest points satisfy $y_n f(x_n) = 1$ (the **canonical** form). The margin on each side is then $1/\|w\|$ and the street has width $2/\|w\|$. Maximizing it is equivalent to the convex quadratic program [9, 10]

$$\min_{w,b} \ \frac{1}{2}\|w\|^2 \quad \text{subject to} \quad y_n(w^\top x_n + b) \geq 1, \quad n = 1, \dots, N.$$

Lagrangian and dual. Introduce multipliers $a_n \geq 0$, one per constraint:

$$\mathcal{L}(w, b, a) = \frac{1}{2}\|w\|^2 - \sum_n a_n [y_n(w^\top x_n + b) - 1].$$

Setting $\partial\mathcal{L}/\partial w = 0$ and $\partial\mathcal{L}/\partial b = 0$ gives

$$w = \sum_n a_n y_n x_n, \qquad \sum_n a_n y_n = 0.$$

Substituting back eliminates w and b and leaves the **dual problem**

$$\max_{a} \sum_{n} a_n - \frac{1}{2} \sum_{n} \sum_{m} a_n a_m y_n y_m \, x_n^{\top} x_m \quad \text{subject to} \quad a_n \geq 0, \;\; \sum_{n} a_n y_n = 0.$$

Three consequences matter [2, 11]:

- **Support vectors.** The Karush–Kuhn–Tucker conditions include **complementary slackness**, $a_n[y_n f(x_n) - 1] = 0$. So $a_n > 0$ only for points exactly on the margin. These are the **support vectors**; all other points have $a_n = 0$ and can be removed without changing the solution. The bias follows from any support vector s: $b = y_s - w^{\top} x_s$ (average over all of them for numerical stability).
- **Sparsity of the prediction.** $f(x) = \sum_{n \in \mathcal{S}} a_n y_n \, x_n^{\top} x + b$ depends only on support vectors $\mathcal{S}$.
- **Only inner products.** The data appear in the dual and in the prediction only through inner products $x_n^{\top} x_m$. Replacing them with a kernel $k(x_n, x_m)$ gives a nonlinear SVM without ever computing ϕ. That is the **kernel trick**, developed in PR 04.

Soft margin and the hinge loss. Real classes overlap, so the constraints cannot all hold. Cortes and Vapnik [10] added **slack variables** $\xi_n \geq 0$, constraints $y_n f(x_n) \geq 1 - \xi_n$, and a penalty $C \sum_n \xi_n$. At the optimum $\xi_n = \max(0, 1 - y_n f(x_n))$, so the soft-margin SVM is equivalent to the unconstrained problem

$$\min_{w,b} \; \frac{1}{2} \|w\|^2 + C \sum_{n} \max(0, \; 1 - y_n f(x_n)).$$

The second term is the **hinge loss**: zero for points beyond the margin, linear inside it and on the wrong side. In the dual the only change is a box constraint $0 \leq a_n \leq C$. So a linear SVM is "ridge-regularized hinge-loss minimization", directly comparable to regularized logistic regression. The roles of C, kernel bandwidth and feature scaling, and the fact that an SVM score is not a probability, are treated in PR 04.

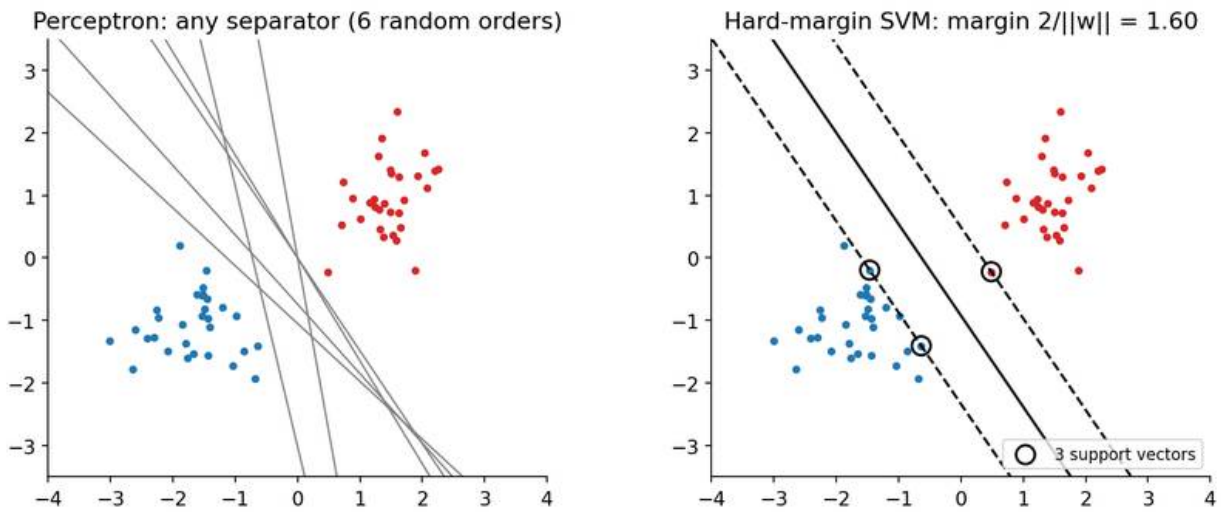


Figure 3 — Left: the perceptron stops at whichever separating line it reaches first (six random orders).

Right: the hard-margin SVM picks the unique line with the widest margin; only the three circled support vectors determine it.

Code: the dual in practice

This check fits a (nearly) hard-margin linear SVM and verifies the dual relations numerically: $w = \sum_n a_n y_n x_n$, $\sum_n a_n y_n = 0$, and $y_n f(x_n) = 1$ on the support vectors.

```
import numpy as np
from sklearn.svm import SVC

rng = np.random.default_rng(7)
X = np.vstack([rng.normal([-1.5, -1.0], 0.6, (30, 2)),
               rng.normal([1.5, 1.0], 0.6, (30, 2))])
y = np.r_[-np.ones(30), np.ones(30)]

svm = SVC(kernel="linear", C=1e6).fit(X, y)          # huge C ~ hard margin
alpha_y = svm.dual_coef_[0]                          # alpha_n * y_n for SVs
w_dual = alpha_y @ svm.support_vectors_              # w = sum_n alpha_n y_n x_n
f = X @ svm.coef_[0] + svm.intercept_[0]

print("support vectors:", len(svm.support_), "of", len(y))
print("w from dual:", np.round(w_dual, 4), " coef_:", np.round(svm.coef_[0], 4))
print("sum_n alpha_n y_n =", round(float(alpha_y.sum()), 8))
print("y f(x) on support vectors:", np.round(y[svm.support_] * f[svm.support_], 4))
print("smallest y f(x) over all points:", round(float(np.min(y * f)), 4))
print("margin width 2/||w|| =", round(2 / np.linalg.norm(svm.coef_[0]), 4))
```

Only 3 of the 60 points are support vectors (the circled points in Figure 3). The weight vector rebuilt from the dual coefficients matches `coef_` exactly, the dual coefficients sum to zero, the support vectors sit at $yf(x) \approx 1$ (to solver tolerance), no point is inside the margin, and the margin width is 1.60. scikit-learn's `SVC` wraps LIBSVM [16]; for large sparse problems, `LinearSVC` and `LogisticRegression(solver="liblinear")` use LIBLINEAR [14], and stochastic sub-gradient solvers such as Pegasos [15] scale to very large datasets.

One picture: linear classifiers as losses

Plain version. All the methods in this chapter score a point with the same linear function. They differ in how much they charge for each value of the margin $y\,g(x)$: nothing once correct (perceptron), nothing once safely correct (SVM), a smoothly decreasing amount (logistic), or a parabola centred at exactly 1 (least squares).

Precise version. With labels $y \in \{-1, +1\}$ and margin $m = y\,g(x)$, each method minimizes a (possibly regularized) average of a **surrogate loss** $\varphi(m)$ that stands in for the non-convex zero-one loss $\mathbb{1}[m \le 0]$ (Figure 4):

Method	Loss $\varphi(m)$	Behaviour
Perceptron	$\max(0, -m)$	zero for any correct point; scale-free, so $w = 0$ is a trivial minimizer
Hinge (SVM)	$\max(0, 1 - m)$	zero beyond the margin; sparse solutions (support vectors)
Logistic	$\ln(1 + e^{-m})$	smooth, never exactly zero; gives probabilities
Squared	$(1 - m)^2 = (y - g(x))^2$	penalizes $m > 1$: "too correct" points pull the boundary

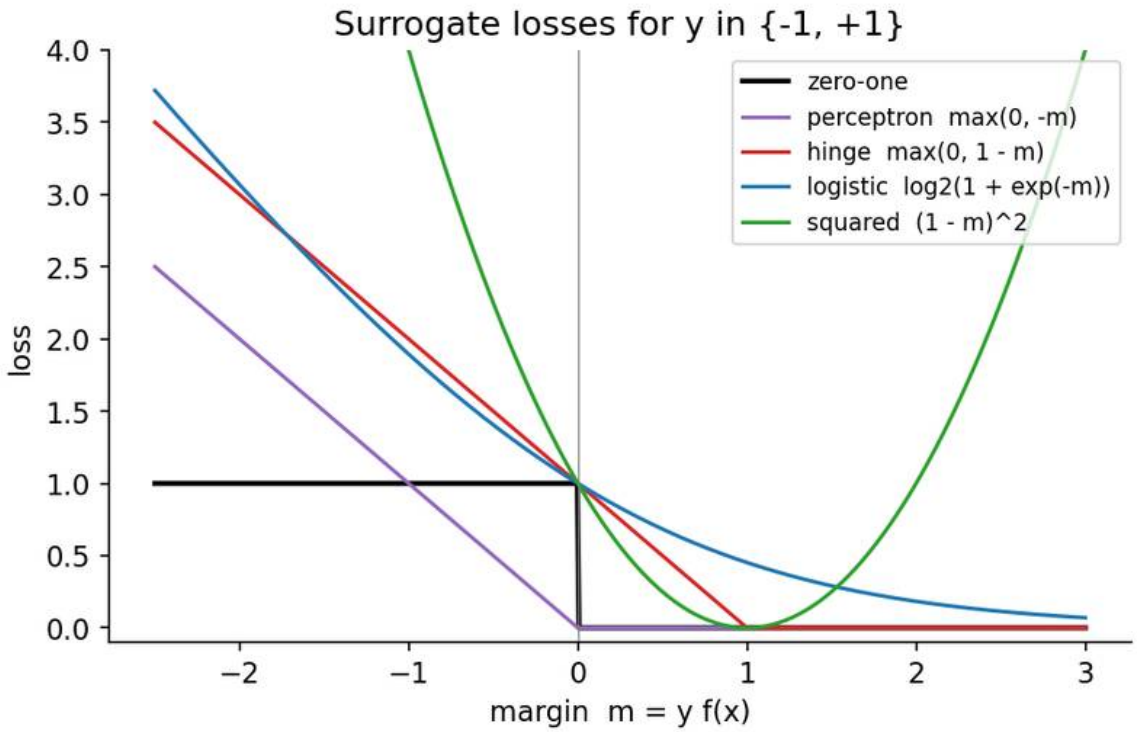


Figure 4 — Surrogate losses as functions of the margin $m = y\,f(x)$. The logistic loss is scaled by $1/\ln 2$ so that it passes through $(0, 1)$. Only the squared loss increases for $m > 1$, which explains Figure 1.

Which surrogates lead to the Bayes-optimal classifier when there is enough data? Bartlett, Jordan and McAuliffe [12] answered this precisely: a convex surrogate is **classification-calibrated** (minimizing its expected value yields the Bayes decision $\operatorname{sign}(2\eta(x) - 1)$, with $\eta(x) = P(y = 1 \mid x)$) if and only if it is differentiable at 0 with $\varphi'(0) < 0$. Hinge, logistic, squared and exponential losses all qualify; the perceptron loss does not. They also relate the excess surrogate risk to the excess classification risk. Rosasco et al. [13] compared the convergence rates implied by different convex losses and concluded that, for classification, the hinge loss converges about as fast as the logistic loss and much faster than the square loss. The losses still differ in *what* they estimate: minimizing the expected logistic loss pointwise gives $g(x) = \ln \frac{\eta(x)}{1-\eta(x)}$, so its scores are log-odds (probabilities after a sigmoid), whereas the pointwise minimizer of the expected hinge loss is $\operatorname{sign}(2\eta(x) - 1)$,

which carries no probability information. That is why SVM scores need separate calibration.

Code: compare linear classifiers on digits

The built-in digits dataset (8×8 images, 10 classes) requires no download. This is a single reproducible hold-out demonstration, not a benchmark ranking. Every scaler is fitted inside a pipeline, on training data only.

```
from sklearn.datasets import load_digits
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.linear_model import LogisticRegression, Perceptron, RidgeClassifier
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import LinearSVC

X, y = load_digits(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=7)
models = {
    "least squares (ridge)": make_pipeline(StandardScaler(), RidgeClassifier(alpha=1.0)),
    "perceptron": make_pipeline(StandardScaler(),
                                Perceptron(max_iter=1500, random_state=7)),
    "logistic": make_pipeline(StandardScaler(),
                              LogisticRegression(C=1, max_iter=1500)),
    "linear SVM": make_pipeline(StandardScaler(),
                                LinearSVC(C=0.1, max_iter=20000, random_state=7)),
    "Gaussian LDA": LinearDiscriminantAnalysis(solver="lsqr", shrinkage="auto"),
}
for name, model in models.items():
    model.fit(Xtr, ytr)
    print(f"{name:22s} test accuracy: {accuracy_score(yte, model.predict(Xte)):.4f}")
```

On this split, logistic regression reaches 97.1%, the linear SVM 96.4%, the perceptron 95.1%, and ridge-regularized least squares and shrinkage LDA 94.7% each. All five are linear machines on the same 64 pixels; the differences come from the losses and regularization alone. Changing regularization after seeing these test results would turn this test set into a validation set. A real model-selection study needs a separate validation split or cross-validation inside the training partition (PR 09).

Modern view

Linear classifiers have not been replaced by deep learning; they have moved to the top of it. Three lines of recent work explain why they still matter, and the classic survey literature remains useful for the methods themselves.

Surveys and solvers. Burges's tutorial [11] remains the most readable derivation of SVMs, from VC-dimension motivation through the dual and kernels. Cervantes et al. [17] survey SVM classification as of 2020: variants, applications, and challenges such as training cost on large datasets, class imbalance and model selection. On the computational side, LIBLINEAR [14] made linear SVMs and logistic regression with millions of sparse features routine, LIBSVM [16] is the reference kernel-SVM implementation behind scikit-learn's `SVC`, and Pegasos [15] showed that stochastic sub-gradient descent on the SVM primal reaches a given accuracy with a run time that, for linear kernels, does not depend directly on the training-set size, the same idea as training neural networks with SGD.

Linear probes as the yardstick of representations. Alain and Bengio [21] proposed training independent linear classifiers ("probes") on every layer of a network, and observed that linear separability increases monotonically with depth. Kornblith, Shlens and Le [22] compared 16 networks on 12 datasets and found that, with features used as fixed extractors under a logistic-regression classifier, ImageNet accuracy and transfer accuracy are strongly correlated, though some regularizers that slightly improve ImageNet accuracy produce much worse penultimate-layer features. CLIP [23] compares its zero-shot classifier with logistic-regression probes on frozen features, and DINOv2 [24] evaluates its self-supervised features largely with linear classifiers on frozen backbones. Kumar et al. [25] showed that full fine-tuning can distort good pretrained features and underperform a linear probe out of distribution, and proposed fitting a linear probe first and then fine-tuning (LP-FT). In other words: a well-regularized logistic regression on top of a frozen foundation model is a strong baseline, and often the right first experiment.

Implicit bias: logistic regression secretly finds the SVM. Soudry et al. [19] proved that gradient descent on unregularized logistic (or cross-entropy) loss over linearly separable data converges *in direction* to the hard-margin SVM solution, extending the result to other losses with exponential tails, to multiclass problems and to a restricted deep-network setting. Convergence is only logarithmic, which helps explain why continuing to train after zero training error can keep improving the margin. The gradient-descent code above shows the effect in two dimensions.

Neural collapse: Fisher's picture at the end of training. Papyan, Han and Donoho [20] observed, across three architectures and seven datasets, that in the terminal phase of training (after zero training error) the last-layer features of each class collapse to their class mean, the class means arrange themselves into a maximally separated, symmetric configuration (a simplex equiangular tight frame), the classifier weights align with the class means, and the decision rule becomes the nearest class mean. In this chapter's language, the within-class scatter S_W shrinks towards zero, Fisher's criterion grows without bound, and the network's last layer becomes the minimum-distance classifier of PR 01.

Open problems. When should one trust a linear probe as a measure of representation quality, given that probe results depend on regularization and data size? How do implicit-bias results extend to realistic, non-separable, data-augmented training? And when do probabilities from linear heads stay calibrated under distribution shift? These questions link the oldest classifier in this track to current research.

Key takeaways

- A linear discriminant $g(x) = w^\top x + b$ is a hyperplane with normal w; $g(x)/\|w\|$ is the signed distance. Use one discriminant per class (a linear machine) for multiclass problems.
- Least squares and ridge have closed forms and a Bayesian MAP interpretation, but least squares is a poor classifier: it punishes points that are "too correct" and can mask classes.
- Fisher's direction $w \propto S_W^{-1}(m_1 - m_0)$ maximizes between-class over within-class spread; it equals the Gaussian LDA direction, which adds a threshold from means and priors.
- The perceptron converges in at most $(R/\gamma)^2$ updates on separable data, but stops at an arbitrary separator and cycles on overlapping data.
- Logistic regression fits the log-odds by convex cross-entropy minimization (Newton/IRLS); without regularization its weights diverge on separable data, in the direction of the max-margin solution.
- The hard-margin SVM maximizes the margin $2/\|w\|$; its dual shows that only support vectors matter and that data enter only through inner products, which opens the door to kernels (PR 04). The soft margin is ridge-regularized hinge loss.
- All these methods are one linear score with different surrogate losses; hinge, logistic and squared losses are classification-calibrated, but only the logistic loss estimates probabilities.

Exercises

1. **Linear in the parameters.** Show that fitting $g(x) = w_0 + w_1 x + w_2 x^2$ by least squares is a linear regression problem, and describe the decision regions in x if the same features are used for a two-class linear classifier.

   Hint

   Build rows $(1, x, x^2)$ of a design matrix; the model is linear in (w_0, w_1, w_2). A quadratic in x changes sign at most twice, so the region for one class can be an interval and the other class its complement (two half-lines).

2. **Units and ridge.** Two features measure the same distance, one in metres and one in millimetres. Explain what ridge regression does without feature scaling, and why standardizing fixes it.

   Hint

   The millimetre coefficient can be 1,000 times smaller while describing the same effect, so it is penalized far less; ridge will load almost all weight onto the millimetre feature. After standardization both coordinates are penalized on the same scale.

3. **Fisher by hand.** Two classes have means $m_0 = (0, 0)$, $m_1 = (2, 0)$ and within-class scatter $S_W = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}$. Compute Fisher's direction and compare it with the mean difference. When would the two directions coincide?

Hint

$S_W^{-1} = \frac{1}{3}\begin{bmatrix} 2 & -1 \\ -1 & 2 \end{bmatrix}$, so $w \propto S_W^{-1}(2,0)^\top = \frac{1}{3}(4,-2) \propto (2,-1)$, rotated away from the mean difference $(1,0)$ because the classes are stretched along $(1,1)$. The directions coincide exactly when $m_1 - m_0$ is an eigenvector of S_W, e.g. for $m_1 = (2,2)$.

4. **A perceptron that never stops.** Construct a one-dimensional dataset on which the perceptron updates forever, and explain which assumption of the convergence theorem fails.

Hint

Put two points at the same coordinate with opposite labels (or any overlapping classes). No u with $\gamma > 0$ exists, so the bound does not apply; each update that fixes one point breaks the other.

5. **SVM by hand.** Find the hard-margin SVM for the two points $x_1 = (1,1)$ with $y_1 = +1$ and $x_2 = (-1,-1)$ with $y_2 = -1$. Give w, b, the dual variables and the margin width, and check the KKT conditions.

Hint

By symmetry $b = 0$ and $w = c(1,1)$. The constraint $y_1 w^\top x_1 = 1$ gives $2c = 1$, so $w = (\frac{1}{2}, \frac{1}{2})$ and the width is $2/\|w\| = 2\sqrt{2}$, the distance between the points. From $w = a_1 x_1 - a_2 x_2$ and $a_1 = a_2$, $a_1 = a_2 = \frac{1}{4}$. Both points are support vectors.

6. **Logistic versus hinge.** Using the digits pipeline, fit `LogisticRegression` and `LinearSVC` for several values of `C` with 5-fold cross-validation on the training set only. Compare their best validation accuracies, then compare their scores: which one gives meaningful probabilities, and how would you obtain probabilities from the other?

Hint

Accuracies are typically close, as the loss analysis predicts. Only logistic regression's `predict_proba` estimates posteriors; for the SVM, fit a calibration map (Platt scaling or isotonic regression, e.g. `CalibratedClassifierCV`) on held-out data.

References

1. R. O. Duda, P. E. Hart and D. G. Stork, *Pattern Classification*, 2nd ed., Wiley, 2001 (Ch. 5, linear discriminant functions). publisher page
2. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006 (Sections 3.1–3.3, 4.1–4.3 and 7.1). official page and free PDF
3. S. Theodoridis and K. Koutroumbas, *Pattern Recognition*, 4th ed., Academic Press, 2008 (Ch. 3, linear classifiers). publisher page

4. T. Hastie, R. Tibshirani and J. Friedman, *The Elements of Statistical Learning*, 2nd ed., Springer, 2009 (Ch. 3–4 and 12). official page
5. R. A. Fisher, “The use of multiple measurements in taxonomic problems,” *Annals of Eugenics*, vol. 7, no. 2, pp. 179–188, 1936. doi
6. F. Rosenblatt, “The perceptron: a probabilistic model for information storage and organization in the brain,” *Psychological Review*, vol. 65, no. 6, pp. 386–408, 1958. doi
7. D. R. Cox, “The regression analysis of binary sequences,” *Journal of the Royal Statistical Society, Series B*, vol. 20, no. 2, pp. 215–232, 1958. doi
8. A. E. Hoerl and R. W. Kennard, “Ridge regression: biased estimation for nonorthogonal problems,” *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970. doi
9. B. E. Boser, I. M. Guyon and V. N. Vapnik, “A training algorithm for optimal margin classifiers,” in *Proc. 5th Annual Workshop on Computational Learning Theory (COLT)*, pp. 144–152, 1992. doi
10. C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995. doi
11. C. J. C. Burges, “A tutorial on support vector machines for pattern recognition,” *Data Mining and Knowledge Discovery*, vol. 2, no. 2, pp. 121–167, 1998. doi
12. P. L. Bartlett, M. I. Jordan and J. D. McAuliffe, “Convexity, classification, and risk bounds,” *Journal of the American Statistical Association*, vol. 101, no. 473, pp. 138–156, 2006. doi
13. L. Rosasco, E. De Vito, A. Caponnetto, M. Piana and A. Verri, “Are loss functions all the same?,” *Neural Computation*, vol. 16, no. 5, pp. 1063–1076, 2004. doi
14. R.-E. Fan, K.-W. Chang, C.-J. Hsieh, X.-R. Wang and C.-J. Lin, “LIBLINEAR: a library for large linear classification,” *Journal of Machine Learning Research*, vol. 9, pp. 1871–1874, 2008. paper
15. S. Shalev-Shwartz, Y. Singer, N. Srebro and A. Cotter, “Pegasos: primal estimated sub-gradient solver for SVM,” *Mathematical Programming*, vol. 127, no. 1, pp. 3–30, 2011. doi
16. C.-C. Chang and C.-J. Lin, “LIBSVM: a library for support vector machines,” *ACM Transactions on Intelligent Systems and Technology*, vol. 2, no. 3, article 27, 2011. doi
17. J. Cervantes, F. Garcia-Lamont, L. Rodríguez-Mazahua and A. Lopez, “A comprehensive survey on support vector machine classification: applications, challenges and trends,” *Neurocomputing*, vol. 408, pp. 189–215, 2020. doi
18. A. Y. Ng and M. I. Jordan, “On discriminative vs. generative classifiers: a comparison of logistic regression and naive Bayes,” in *Advances in Neural Information Processing Systems (NIPS)*, 2001. paper
19. D. Soudry, E. Hoffer, M. S. Nacson, S. Gunasekar and N. Srebro, “The implicit bias of gradient descent on separable data,” *Journal of Machine Learning Research*, vol. 19, no. 70, pp. 1–57, 2018. paper
20. V. Papyan, X. Y. Han and D. L. Donoho, “Prevalence of neural collapse during the terminal phase of deep learning training,” *Proceedings of the National Academy of Sciences*, vol. 117, no. 40, pp. 24652–24663, 2020. doi
21. G. Alain and Y. Bengio, “Understanding intermediate layers using linear classifier probes,” arXiv:1610.01644, 2016. arXiv
22. S. Kornblith, J. Shlens and Q. V. Le, “Do better ImageNet models transfer better?,” in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2019. paper
23. A. Radford et al., “Learning transferable visual models from natural language supervision,” in *Proc. International Conference on Machine Learning (ICML)*, PMLR 139, 2021. paper

24. M. Oquab et al., “DINOv2: learning robust visual features without supervision,” *Transactions on Machine Learning Research*, 2024. arXiv
25. A. Kumar, A. Raghunathan, R. Jones, T. Ma and P. Liang, “Fine-tuning can distort pretrained features and underperform out-of-distribution,” in *Proc. International Conference on Learning Representations (ICLR)*, 2022. arXiv

PR 04 • Neighbors, Kernels and Nonlinear Classifiers

From nearest neighbors and the kernel trick to soft-margin SVMs, RBF networks, decision trees and multilayer perceptrons: how classifiers bend the decision boundary, and what each bend costs.

Online edition: https://blog.shyandram.dev/en/learn/pattern-recognition/pr-04-kernels-nonlinear-classifiers/

What you'll learn

- How nearest-neighbor rules produce nonlinear decision regions without fitting any parameters, and why the distance metric matters as much as k.
- What the kernel trick is, what makes a function a valid (Mercer) kernel, and how to check it on a finite Gram matrix.
- The soft-margin support vector machine: its primal and dual problems, support vectors, and how the hyperparameters C and γ shape the boundary.
- How Gaussian processes and relevance vector machines reuse kernels with a probabilistic interpretation.
- Three other routes to nonlinearity: radial basis function (RBF) networks, decision trees grown with impurity measures (CART), and multilayer perceptrons.
- How to compare these families fairly, and where they stand in 2026.

The big picture

PR 03 built classifiers whose decision boundary is a straight line, a plane or a hyperplane. Many real problems are not like that. Points inside a ring cannot be separated from points outside it by any straight line. Handwritten "1"s and "7"s overlap in pixel space along curved, twisted regions. We need boundaries that bend.

There are only a few basic ways to make them bend:

1. **Compare with stored examples.** Label a new point by looking at the most similar training points (nearest neighbors).
2. **Change the representation, then stay linear.** Map each input x to new features $\phi(x)$ in which a hyperplane does work. Kernels do this implicitly; RBF networks and multilayer perceptrons do it explicitly, with learned or fixed basis functions.
3. **Cut the space into pieces.** A decision tree asks a sequence of yes/no questions, and each answer path ends in a small box with its own label.

Explain like I'm five

> Imagine sorting leaves you find in a park. You could hold each new leaf next to your labeled collection and copy the label of the leaves it looks most like (neighbors). You could look at the leaf through special glasses that make the differences jump out, so a simple rule works again (kernels and features). Or you could play twenty questions: "Is it wider than my thumb? Does it have jagged edges?" until only one kind of leaf is left (decision trees).

One warning applies to every method in this chapter. A nonlinear classifier can only use information that the measurements contain. If two classes look identical in the features you recorded, no amount of boundary-bending will separate them. Feature design is the subject of PR 05. The textbook background for this chapter is Theodoridis and Koutroumbas, Chapters 2 and 4 [1], and Bishop, Sections 2.5.2, 5.1–5.3, 6.1–6.4, 7.1–7.2 and 14.4 [2].

Nearest-neighbor classification

In plain words. To classify a new point, find the k training points closest to it and let them vote. Nothing is "learned" beyond storing the data, yet the resulting boundary can follow any shape the data have.

The rule and its guarantee

Given training pairs (x_n, y_n), $n = 1, \dots, N$, and a query x, let $\mathcal{N}_k(x)$ be the indices of the k training points nearest to x under some distance d. The k-nearest-neighbor (k-NN) rule predicts

$$\hat{y}(x) = \arg\max_{c} \sum_{n \in \mathcal{N}_k(x)} w_n \, \mathbf{1}[y_n = c],$$

where c runs over the classes, $\mathbf{1}[\cdot]$ is 1 when its argument is true and 0 otherwise, and w_n is a vote weight: $w_n = 1$ for plain majority voting, or a decreasing function of $d(x, x_n)$ for distance-weighted voting.

The vote fractions are a crude estimate of the class posteriors $P(c \mid x)$: k-NN is a nonparametric density method in disguise (PR 02), with the neighborhood size set by k rather than by a fixed bandwidth [2]. Cover and Hart proved the classic result for the 1-NN rule: as the number of training samples grows without bound, its error rate is at most twice the Bayes error rate [3]. This is an asymptotic statement. With finite data, especially in high dimension, k-NN can be far from that bound.

The metric is a modeling decision

Euclidean distance treats a one-unit difference in every coordinate as equally important. If one feature is measured in millimeters and another in kilometers, the first will dominate every neighborhood. Standardizing features, learning a Mahalanobis metric $d(x, z)^2 = (x - z)^\top M (x - z)$ with a positive semidefinite matrix M, or comparing normalized embeddings by cosine similarity all change who counts as a neighbor.

For vectors u and v normalized to unit length,

$$\|u - v\|^2 = \|u\|^2 + \|v\|^2 - 2u^\top v = 2 - 2u^\top v.$$

So the nearest neighbor by Euclidean distance is exactly the neighbor with the *largest* cosine similarity: the two criteria rank candidates in reverse order of each other and pick the same neighbors. This is why nearest-neighbor search over normalized deep features (ML 05) can use either.

Interactive demonstration (online): open the online edition of this tutorial to use this demonstration.Open demonstration

Choosing k, and the cost of prediction

A small k (such as 1) gives a jagged boundary that follows every noisy training point: low bias, high variance. A large k averages over a wide region and smooths the boundary, but it can wash out a small class entirely. Choose k, the metric and the scaling by cross-validation on training data only.

k-NN shifts all the cost to prediction time. A naive query compares x with all N training points, which costs $O(ND)$ for D-dimensional inputs. Tree-based indices and approximate nearest-neighbor search reduce this in practice. In high dimension, a further problem appears: when many coordinates carry irrelevant variation, distances between random points concentrate around a common value, and "nearest" stops meaning "most similar." The right panel of Figure 3 later in this chapter shows a 7-NN boundary on a two-dimensional toy problem, where none of these problems arise.

Kernels: the kernel trick and Mercer's condition

In plain words. Many linear algorithms only ever use inputs through inner products $x^\top z$. If we replace every inner product by a "similarity function" $k(x, z)$ that equals an inner product in some richer feature space, we get a nonlinear algorithm for the price of a linear one, without ever computing the rich features.

Lifting the data

Take the ring data on the left of Figure 1. No line separates the inner disc from the outer ring. Add a third feature, the squared radius $r^2 = x_1^2 + x_2^2$. In the new three-dimensional space the inner points sit low and the outer points sit high, and a flat plane separates them perfectly. A linear classifier in the lifted space is a circle in the original space.

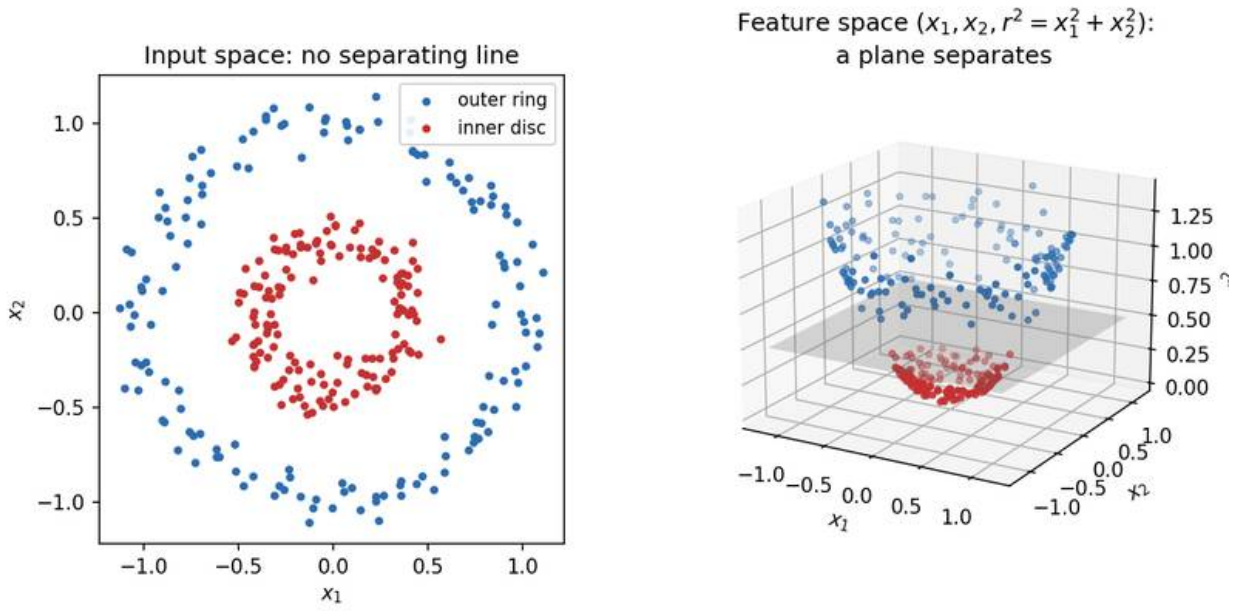


Figure 1 — Lifting to a feature space. Adding the feature $r^2 = x_1^2 + x_2^2$ turns a circular boundary into a flat plane.

Explicit lifting gets expensive quickly. All monomials of degree up to m in D variables number $\binom{D+m}{m}$; for a 16×16 image ($D = 256$) and $m = 3$ that is about 2.9 million features. Kernels avoid building them.

The kernel trick

A **kernel** is a function $k(x, z)$ for which there exists a feature map ϕ into some inner-product space with

$$k(x, z) = \phi(x)^{\top} \phi(z).$$

Here x and z are inputs, $\phi(x)$ is the (possibly very long, even infinite) feature vector of x, and the right-hand side is their inner product. For example, in two dimensions, with $x = (a, b)$,

$$(x^{\top} z + 1)^2 = \phi(x)^{\top} \phi(z), \qquad \phi(x) = (1,\ \sqrt{2}a,\ \sqrt{2}b,\ a^2,\ b^2,\ \sqrt{2}ab).$$

Evaluating the left side costs one inner product and one square. The right side needs six features per point. For degree m in D dimensions, the left side still costs $O(D)$.

The **kernel trick** is the observation that any algorithm written entirely in terms of inner products between data points can be "kernelized" by replacing $x_i^{\top} x_j$ with $k(x_i, x_j)$ [7]. The SVM dual, kernel ridge regression, kernel PCA (PR 05) and Gaussian processes are all of this kind. The result is collected in the **Gram matrix** $K \in \mathbb{R}^{N \times N}$, $K_{ij} = k(x_i, x_j)$.

The code below checks the identity numerically, and then shows a popular "similarity" that is *not* a valid kernel.

```
import numpy as np

rng = np.random.default_rng(0)
x, z = rng.normal(size=2), rng.normal(size=2)

def phi(v):  # explicit feature map of (v.w + 1)^2 in 2-D
    a, b = v
    s = np.sqrt(2)
    return np.array([1, s * a, s * b, a * a, b * b, s * a * b])

lhs = (x @ z + 1) ** 2
rhs = phi(x) @ phi(z)
print("kernel:", round(lhs, 6), " explicit:", round(rhs, 6))
assert np.isclose(lhs, rhs)

X = rng.normal(size=(40, 2))
K_sig = np.tanh(2.0 * X @ X.T - 1.0)        # "sigmoid kernel"
print("sigmoid Gram min eigenvalue:", np.linalg.eigvalsh(K_sig).min().round(3))
```

The two numbers agree. The tanh "sigmoid kernel" Gram matrix has a clearly negative eigenvalue (about -15 here), so for these parameters no feature map ϕ can reproduce it.

Which functions are kernels? Mercer's condition

A function k is a valid kernel exactly when it is **symmetric**, $k(x, z) = k(z, x)$, and **positive semidefinite**: for every finite set of points $x_1, \dots, x_N$ and every real vector $a \in \mathbb{R}^N$,

$$a^\top K a = \sum_{i=1}^{N} \sum_{j=1}^{N} a_i a_j \, k(x_i, x_j) \;\geq\; 0.$$

Equivalently, every Gram matrix has nonnegative eigenvalues. Symmetry alone is not enough, as the sigmoid example shows.

Mercer's theorem [4] gives the continuous version. For a continuous symmetric k on a compact domain $\mathcal{X}$, positive semidefiniteness in the integral sense,

$$\int_{\mathcal{X}} \int_{\mathcal{X}} g(x)\, k(x, z)\, g(z)\, dx\, dz \geq 0 \quad \text{for all square-integrable } g,$$

guarantees an expansion $k(x, z) = \sum_{i=1}^{\infty} \lambda_i \, e_i(x) \, e_i(z)$ with eigenvalues $\lambda_i \geq 0$ and orthonormal eigenfunctions e_i. The feature map is then $\phi(x) = (\sqrt{\lambda_1} e_1(x), \sqrt{\lambda_2} e_2(x), \dots)$. In modern terms, every positive semidefinite kernel defines a **reproducing kernel Hilbert space** (RKHS) of functions, and the representer theorem says that regularized fits in that space are finite combinations $f(x) = \sum_n \alpha_n k(x_n, x)$ [7].

Valid kernels can be built from valid kernels. If k_1 and k_2 are kernels, so are $k_1 + k_2$, $c\,k_1$ for $c > 0$, $k_1 k_2$, $\exp(k_1)$, and $f(x) k_1(x, z) f(z)$ for any function f [2]. The common kernels follow:

Kernel	Formula	Parameters	Notes
Linear	$x^\top z$	none	Recovers the linear model
Polynomial	$(\gamma x^\top z + r)^m$	$\gamma > 0$, $r > 0$, integer $m \geq 1$	All monomials up to degree m ($r = 0$: degree m only)
Gaussian RBF	$\exp(-\gamma\|x - z\|^2)$	$\gamma > 0$	Infinite-dimensional feature space; local similarity
Laplacian	$\exp(-\gamma\|x - z\|_1)$	$\gamma > 0$	Rougher functions than the Gaussian
Sigmoid	$\tanh(\gamma x^\top z + r)$	γ, r	Not positive semidefinite in general

The Gaussian RBF kernel is the default nonlinear choice. Its parameter γ (sometimes written $1/(2\sigma^2)$ with a bandwidth σ) sets how far similarity reaches. A large γ makes each training point influence only a tiny neighborhood; a small γ makes the kernel nearly constant over the data, so the model approaches a very smooth, almost linear function. Because γ multiplies squared distances, it is tied to the scale of the features: always standardize before using an RBF kernel.

Kernel ridge regression: the trick in one line

Ridge regression with features ϕ minimizes $\sum_n (t_n - w^\top \phi(x_n))^2 + \lambda\|w\|^2$. Its solution can be written as $w = \sum_n \alpha_n \phi(x_n)$, which leads to

$$\alpha = (K + \lambda I)^{-1} t, \qquad f(x) = k_x^\top \alpha,$$

where $t = (t_1, \dots, t_N)^\top$ are the targets, $\lambda > 0$ is the regularization strength, I is the $N \times N$ identity, and $k_x = (k(x_1, x), \dots, k(x_N, x))^\top$ holds the similarities of a query x to the training points. The D-dimensional feature-space solve has become an $N \times N$ sample-space solve: attractive when features are many and samples few, expensive ($O(N^3)$ time, $O(N^2)$ memory) when N is large.

```
import numpy as np
from sklearn.kernel_ridge import KernelRidge
from sklearn.metrics.pairwise import rbf_kernel

rng = np.random.default_rng(1)
X = rng.uniform(-3, 3, size=(60, 1))
t = np.sin(X[:, 0]) + 0.1 * rng.normal(size=60)
lam, gamma = 0.1, 0.5

K = rbf_kernel(X, X, gamma=gamma)
alpha = np.linalg.solve(K + lam * np.eye(len(X)), t)
Xq = np.linspace(-3, 3, 5)[:, None]
f_ours = rbf_kernel(Xq, X, gamma=gamma) @ alpha
f_skl = KernelRidge(alpha=lam, kernel="rbf", gamma=gamma).fit(X, t).predict(Xq)
print(np.round(f_ours, 4))
assert np.allclose(f_ours, f_skl)
```

The hand-written solve and scikit-learn's `KernelRidge` agree, and the predictions follow $\sin x$ at the five query points.

Support vector machines

In plain words. Among all boundaries that separate the classes, pick the one with the widest empty "street" between them. Only the points on the edges of the street, or inside it, matter; they are the support vectors. Allowing a few points into the street, at a price C, makes the method work on overlapping data. Replacing inner products by a kernel makes the street curve.

Hard and soft margins

Use labels $y_n \in \{-1, +1\}$ and a score $f(x) = w^\top \phi(x) + b$, with weight vector w and bias b; predict the sign of f. If the data are separable, scale w and b so that the closest points satisfy $y_n f(x_n) = 1$. The distance between the two lines $f = \pm 1$, the **margin**, is then $2/\|w\|$. Maximizing it means minimizing $\frac{1}{2}\|w\|^2$ subject to $y_n f(x_n) \geq 1$ for all n: the hard-margin SVM of Boser, Guyon and Vapnik [5].

Real data overlap, so Cortes and Vapnik introduced slack variables $\xi_n \geq 0$ that measure how far each point violates the margin [6]. The **soft-margin SVM** solves

$$\min_{w,b,\xi} \frac{1}{2}\|w\|^2 + C \sum_{n=1}^{N} \xi_n \quad \text{subject to} \quad y_n f(x_n) \geq 1 - \xi_n, \;\; \xi_n \geq 0.$$

A point with $\xi_n = 0$ is on the correct side with margin at least one. A point with $0 < \xi_n \leq 1$ is inside the street but still correctly classified, and $\xi_n > 1$ means it is misclassified. The constant $C > 0$ sets the price of violations. Eliminating ξ gives the equivalent unconstrained form

$$\min_{w,b} \frac{1}{2}\|w\|^2 + C \sum_{n} \max\left(0,\, 1 - y_n f(x_n)\right),$$

regularized **hinge loss**. A point with $y_n f(x_n) \geq 1$ contributes no loss at all; this is the source of the SVM's sparsity. (Some texts average the losses and put the constant on the regularizer instead, writing $\frac{\lambda}{2}\|w\|^2 + \frac{1}{N}\sum_n \text{hinge}$. The two forms match when $C = 1/(\lambda N)$.)

The dual problem and support vectors

Introducing a Lagrange multiplier $\alpha_n \geq 0$ for each margin constraint and eliminating w, b and ξ gives the **dual problem** [2], [6]:

$$\max_{\alpha} \sum_{n=1}^{N} \alpha_n - \frac{1}{2} \sum_{n=1}^{N} \sum_{m=1}^{N} \alpha_n \alpha_m y_n y_m \, k(x_n, x_m) \quad \text{subject to} \quad 0 \leq \alpha_n \leq C, \;\; \sum_{n=1}^{N} \alpha_n y_n = 0.$$

It is a convex quadratic program in the N multipliers α_n, and the data appear *only* through kernel values $k(x_n, x_m)$. That is where the kernel trick enters. The optimal weight vector is $w = \sum_n \alpha_n y_n \phi(x_n)$, so predictions need only kernels too:

$$f(x) = \sum_{n=1}^{N} \alpha_n y_n \, k(x_n, x) + b.$$

The Karush–Kuhn–Tucker (KKT) optimality conditions sort the training points into three groups:

- $\alpha_n = 0$: the point is outside the street ($y_n f(x_n) \geq 1$) and has no influence on f;
- $0 < \alpha_n < C$: the point lies exactly on the margin ($y_n f(x_n) = 1$); these points determine b;
- $\alpha_n = C$: the point is inside the street or misclassified ($y_n f(x_n) \leq 1$).

Points with $\alpha_n > 0$ are the **support vectors**. Deleting any other point and retraining gives the same classifier. How many there are depends on the data: on a heavily overlapping problem, a large fraction of the training set can end up as support vectors, and prediction becomes as expensive as nearest-neighbor search.

How C and γ shape an RBF-SVM

With the Gaussian kernel, two numbers control everything. C sets how much the training errors cost relative to a wide margin; γ sets how local the kernel is. Figure 2 sweeps both on the same noisy data.

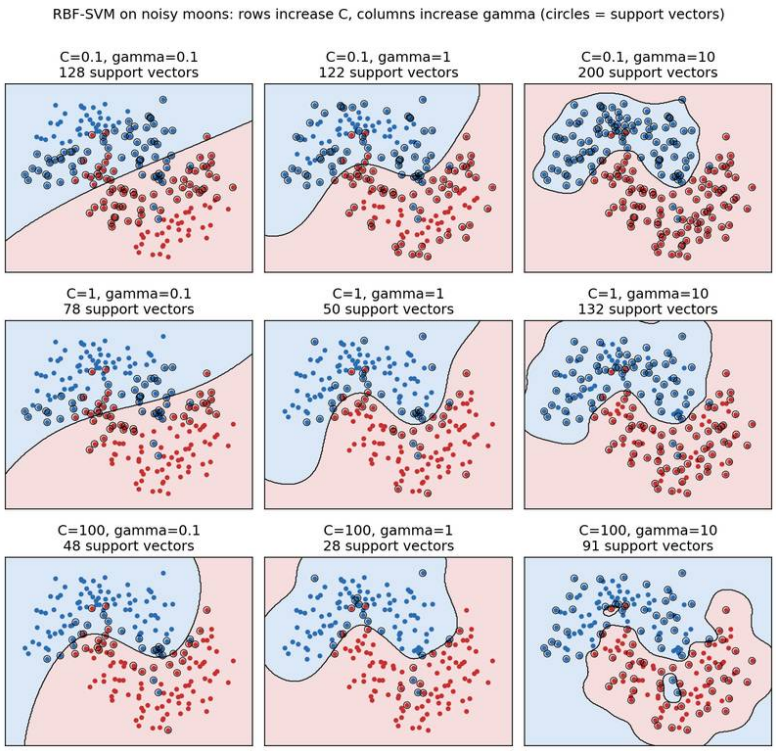


Figure 2 — RBF-SVM on noisy two-moons data (standardized). Small γ gives almost linear boundaries; large γ gives islands around individual points. Small C tolerates many margin violations (many support vectors); large C fits the training points harder. Generated by scripts/figures/pr_04_kernels_nonlinear_classifiers_extra.py.

Read the grid like this. Moving down (larger C) makes the boundary chase individual training points, and the number of support vectors drops because fewer points are allowed inside the margin. Moving right (larger γ) shrinks each point's sphere of influence; at $\gamma = 10$ the classifier draws little islands, which is overfitting. The good region is a diagonal band: when γ grows, a smaller C is usually needed. In practice one searches C and γ on a logarithmic grid with cross-validation.

```
import numpy as np
from sklearn.datasets import make_moons
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC

X, y = make_moons(n_samples=500, noise=0.18, random_state=2)
Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=2)
models = {
    "linear SVM": SVC(kernel="linear", C=1),
    "RBF SVM": SVC(kernel="rbf", C=3, gamma=0.7),
    "7-neighbor": KNeighborsClassifier(n_neighbors=7),
}
for name, estimator in models.items():
    model = make_pipeline(StandardScaler(), estimator).fit(Xtr, ytr)
    print(name, "test accuracy:", round(model.score(Xte, yte), 4))

search = GridSearchCV(
    make_pipeline(StandardScaler(), SVC(kernel="rbf")),
    {"svc__C": [0.1, 1, 10, 100], "svc__gamma": [0.01, 0.1, 1, 10]},
    cv=5)
search.fit(Xtr, ytr)
best = search.best_estimator_
print("best:", search.best_params_,
      "CV acc:", round(search.best_score_, 4),
      "test acc:", round(best.score(Xte, yte), 4),
      "support vectors:", best[-1].n_support_.sum(), "of", len(Xtr))
```

On this split the linear SVM reaches 0.827 test accuracy, the hand-picked RBF-SVM and the 7-NN rule both 0.96. The grid search chooses $C = 1$, $\gamma = 1$ (cross-validated accuracy 0.971) with 59 of 350 training points as support vectors, and scores 0.953 on the test set. The tuned model is not "worse" than the hand-picked one in any meaningful sense: with 150 test points, one test point is 0.0067 of accuracy, and the two differ by one point. Treat single-split differences of this size as noise (PR 09). The scaler lives inside the pipeline so that it is refitted on each training fold.

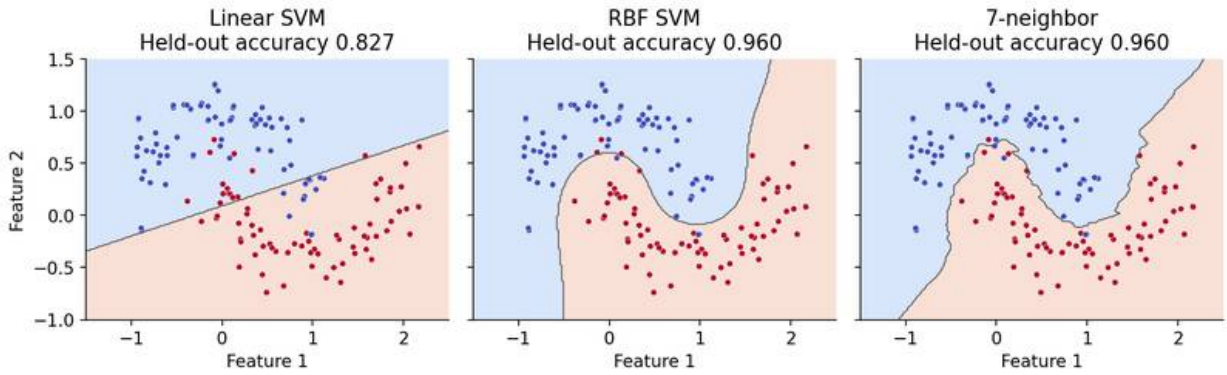


Figure 3 — Linear SVM, RBF-SVM and 7-NN on the same split as the code above. Scalers are fitted on training data only; dots and scores use the held-out partition. A synthetic example, not a general benchmark.

Scores are not probabilities

The SVM output $f(x)$ is a signed distance-like score, not $P(y = +1 \mid x)$. Its training objective never asks it to be calibrated. If probabilities are needed (for a reject option, PR 01, or for combining models), fit a separate monotone mapping from scores to probabilities on held-out data, and check it with a reliability diagram (PR 09). Multiclass SVMs are usually built from binary ones, either one-versus-rest or one-versus-one (scikit-learn's `SVC` uses one-versus-one).

Gaussian processes and relevance vectors

In plain words. A Gaussian process uses the same kernels, but instead of returning one best function it returns a whole distribution of plausible functions, so each prediction comes with an error bar.

A **Gaussian process** (GP) is a prior over functions f such that any finite set of values $(f(x_1), \dots, f(x_N))$ is jointly Gaussian with mean zero (typically) and covariance $K_{ij} = k(x_i, x_j)$ [8]. The kernel must therefore be a valid covariance, that is, positive semidefinite. For regression with Gaussian observation noise of variance σ^2, the posterior at a query x is Gaussian with

$$\mu(x) = k_x^\top (K + \sigma^2 I)^{-1} t, \qquad s^2(x) = k(x,x) - k_x^\top (K + \sigma^2 I)^{-1} k_x,$$

where $\mu(x)$ is the predictive mean and $s^2(x)$ the variance of the latent function. The mean is exactly kernel ridge regression with $\lambda = \sigma^2$. The variance is new: it is small near training inputs and grows back to the prior variance $k(x,x)$ far from them. Kernel hyperparameters (such as γ and σ^2) can be chosen by maximizing the marginal likelihood instead of cross-validation [2], [8].

Two cautions. The uncertainty is conditional on the chosen kernel; a confident GP with the wrong kernel can be confidently wrong. And GP *classification* is harder: a Bernoulli likelihood on top of a Gaussian latent function makes the posterior non-Gaussian, so Laplace, expectation propagation or variational approximations are needed (PR 08).

The **relevance vector machine** (RVM), introduced by Tipping [9] and presented in Bishop, Section 7.2 [2], uses the same functional form as the SVM, $f(x) = \sum_n w_n k(x_n, x) + b$, but places a separate zero-mean Gaussian prior on each weight w_n with its own precision hyperparameter. Maximizing the marginal likelihood drives many precisions to infinity, which switches the corresponding basis functions off. The surviving training points, the "relevance vectors," are typically far fewer than an SVM's support vectors, the outputs are probabilistic, and the basis functions do not need to satisfy Mercer's condition. The price is a non-convex training problem. "Sparse" thus has different meanings: few stored basis functions (SVM, RVM), few nonzero input weights (lasso), or sparse input data. Say which one you mean.

Radial basis function networks

In plain words. Put a few Gaussian "bumps" at representative places in the input space, measure how strongly a new point activates each bump, and feed those activations to a linear classifier. It is the lifting idea of Figure 1, but with a small, explicit set of features.

An RBF network computes

$$f(x) = \sum_{j=1}^{M} w_j \, \exp\!\left(-\gamma \lVert x - \mu_j \rVert^2\right) + w_0,$$

where $\mu_1, \dots, \mu_M$ are M **centers**, γ sets the width of each bump, w_j are output weights and w_0 is a bias. Moody and Darken proposed the classic two-stage training [10]: first place the centers without labels, for example by k-means clustering (PR 07); then, with centers fixed, the output weights solve an ordinary linear least-squares or logistic regression problem. Park and Sandberg showed that RBF networks are universal approximators: with enough centers they can approximate any continuous function on a compact set arbitrarily well [11].

Compare this with the kernel machines above. A kernel SVM effectively places one bump on every support vector, chosen by the margin criterion. An RBF network uses $M \ll N$ bumps whose positions come from clustering (or from gradient descent if all parameters are trained jointly), which makes prediction cheap. Its weakness is that unsupervised center placement ignores the labels, so centers may be wasted where the classes do not meet.

```
import numpy as np
from sklearn.cluster import KMeans
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

X, y = make_moons(n_samples=500, noise=0.18, random_state=2)
Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=2)
scaler = StandardScaler().fit(Xtr)
Xtr, Xte = scaler.transform(Xtr), scaler.transform(Xte)

M, gamma, lam = 12, 1.0, 1e-3
```

```
centers = KMeans(M, n_init=10, random_state=0).fit(Xtr).cluster_centers_

def design(X):  # N x (M+1): Gaussian bumps plus a bias column
    d2 = ((X[:, None, :] - centers[None, :, :]) ** 2).sum(-1)
    return np.c_[np.exp(-gamma * d2), np.ones(len(X))]

Phi = design(Xtr)
t = 2.0 * ytr - 1.0                       # targets in {-1, +1}
w = np.linalg.solve(Phi.T @ Phi + lam * np.eye(M + 1), Phi.T @ t)
pred = (design(Xte) @ w > 0).astype(int)
print("RBF network test accuracy:", round((pred == yte).mean(), 4))
```

Twelve bumps and a least-squares output layer give 0.953 test accuracy on the moons split, on par with the RBF-SVM that keeps dozens of support vectors. Figure 5 (left) shows the same network with a logistic output layer.

Decision trees

In plain words. A decision tree plays twenty questions with the data. Each question looks at one feature and asks "is it at most this threshold?" Each answer sends the point left or right, until it reaches a leaf that holds a label. The tree is grown greedily: at every step, pick the question that makes the two resulting groups as "pure" as possible.

Impurity measures

Let a node contain training points with class proportions $p_1, \dots, p_C$ (fraction of the node's points in each class c). An **impurity** function is zero when one class has all the points and largest when the classes are evenly mixed. The three standard choices are

$$\text{Gini:} \quad G = 1 - \sum_{c=1}^{C} p_c^2 = \sum_{c} p_c (1 - p_c),$$

$$\text{Entropy:} \quad H = -\sum_{c=1}^{C} p_c \log_2 p_c,$$

$$\text{Misclassification:} \quad E = 1 - \max_c p_c.$$

The Gini index is the probability that two points drawn at random (with replacement) from the node have different labels. Entropy is the information-theoretic uncertainty of the label, in bits. Misclassification error is the error rate if the node predicted its majority class. Figure 4 (left) plots the three for two classes.

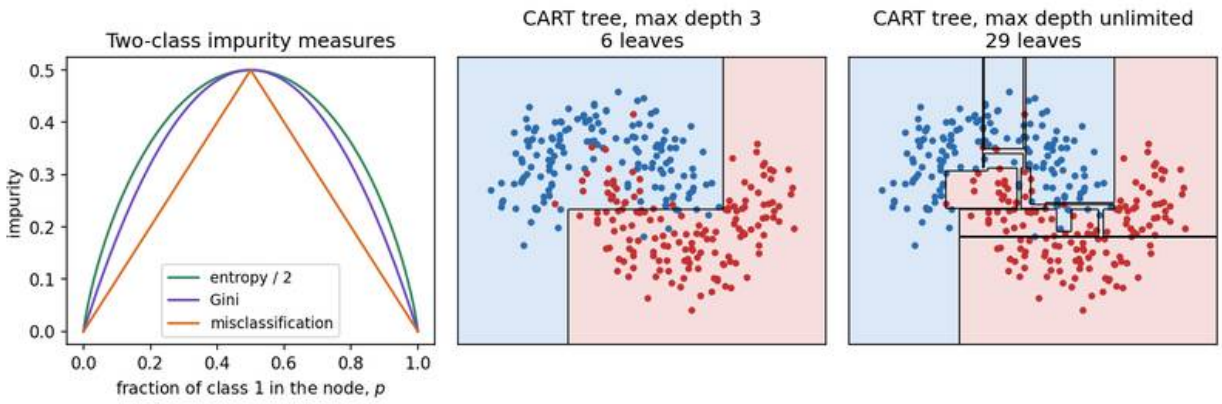


Figure 4 — Left: two-class impurity measures (entropy scaled by ½ to share the axis). Middle: a depth-3 CART tree gives a coarse, axis-aligned staircase. Right: an unlimited tree carves tiny boxes around noisy points, overfitting the training data.

Gini and entropy are strictly concave, so splitting a node never increases their weighted average. Misclassification error is piecewise linear and often fails to reward a split that makes the children purer without changing either child's majority label. That is why trees are grown with Gini or entropy and evaluated with error rate.

Growing a tree: the CART algorithm

The CART procedure of Breiman, Friedman, Olshen and Stone [12] grows a binary tree top-down. At a node with point set S, it considers every feature j and every threshold τ between consecutive observed values, splitting S into $S_L = \{x \in S : x_j \le \tau\}$ and $S_R = S \setminus S_L$. It chooses the split with the largest impurity decrease

$$\Delta i(j, \tau) = i(S) - \frac{|S_L|}{|S|}\, i(S_L) - \frac{|S_R|}{|S|}\, i(S_R),$$

where $i(\cdot)$ is the chosen impurity and $|S|$ counts the points in a set. It then recurses on both children until a stopping rule fires (pure node, minimum node size, maximum depth). Quinlan's ID3 [13] is a close relative that uses entropy, calls Δi the **information gain**, and makes multiway splits on categorical features.

The code finds the best root split by brute force and checks it against scikit-learn, whose `DecisionTreeClassifier` implements an optimized version of CART.

```
import numpy as np
from sklearn.datasets import make_moons
from sklearn.tree import DecisionTreeClassifier

def gini(labels):
    p = np.bincount(labels, minlength=2) / len(labels)
    return 1.0 - (p ** 2).sum()
```

```
def best_split(X, y):
    best = (np.inf, None, None)
    for j in range(X.shape[1]):
        values = np.unique(X[:, j])
        for thr in (values[:-1] + values[1:]) / 2:
            left = X[:, j] <= thr
            n_l, n_r = left.sum(), (~left).sum()
            score = (n_l * gini(y[left]) + n_r * gini(y[~left])) / len(y)
            if score < best[0]:
                best = (score, j, thr)
    return best

X, y = make_moons(n_samples=300, noise=0.25, random_state=1)
score, j, thr = best_split(X, y)
root = DecisionTreeClassifier(max_depth=1).fit(X, y).tree_
print(f"ours: feature {j}, threshold {thr:.4f}, weighted Gini {score:.4f}")
print(f"sklearn: feature {root.feature[0]}, threshold {root.threshold[0]:.4f}")
assert j == root.feature[0] and np.isclose(thr, root.threshold[0])
```

Both choose feature 1 with threshold 0.0547, reducing the impurity from 0.5 at the root to a weighted 0.287.

Pruning and the character of trees

A tree grown until every leaf is pure memorizes the training set (Figure 4, right). CART's answer is **cost-complexity pruning** [12]: grow a large tree T_0, then for each subtree T obtained by collapsing internal nodes, measure

$$R_\alpha(T) = R(T) + \alpha\,|\widetilde{T}|,$$

where $R(T)$ is the training misclassification rate, $|\widetilde{T}|$ is the number of leaves and $\alpha \geq 0$ is a complexity price. Increasing α traces a nested sequence of smaller and smaller trees; choose α by cross-validation (`ccp_alpha` in scikit-learn).

Trees have a distinctive profile. They need no feature scaling, handle mixed numeric and categorical inputs, ignore irrelevant features if they never help a split, and produce rules a person can read. Their boundaries are axis-aligned staircases, so a diagonal boundary costs many splits. And they are unstable: a small change in the data can change the root split and hence the whole tree. Averaging many randomized trees (bagging, random forests) or adding them sequentially (boosting) fixes the instability, and that is the topic of PR 09.

Multilayer perceptrons as nonlinear classifiers

In plain words. A multilayer perceptron (MLP) is an RBF network whose bumps are replaced by learned "ramps," and whose ramps are themselves learned from the labels. Each hidden unit draws one line; the next layer combines the lines into curved regions.

A one-hidden-layer MLP for C classes computes

$$h = \sigma(W_1 x + b_1), \qquad P(y = c \mid x) = \mathrm{softmax}(W_2 h + b_2)_c,$$

where W_1, b_1 map the D inputs to H hidden units, σ is an elementwise nonlinearity (ReLU $\max(0, \cdot)$, logistic or tanh), and W_2, b_2 map hidden units to class scores. All weights are trained jointly by minimizing cross-entropy with gradient descent, the gradients being computed by backpropagation [14]. Unlike every method above, both the features (h) and the classifier are learned from the labels. The universal approximation theorems of Cybenko [15] and Hornik [16] guarantee that one hidden layer with enough units can approximate any continuous function on a compact set; they say nothing about how many units are needed or whether gradient descent will find them. The full story, from neurons and activations to backpropagation and training tricks, is in ML 02; the image-classification version is in IP 13.

Geometrically, with ReLU units each hidden unit splits the input space with a hyperplane, and the network output is piecewise linear on the resulting cells, which is why the MLP boundary in Figure 5 is made of straight segments.

```
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X, y = make_moons(n_samples=500, noise=0.18, random_state=2)
Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=2)
for hidden in [(2,), (16,), (16, 16)]:
    mlp = make_pipeline(
        StandardScaler(),
        MLPClassifier(hidden_layer_sizes=hidden, alpha=1e-3,
                      max_iter=3000, random_state=0))
    mlp.fit(Xtr, ytr)
    print(hidden, "test accuracy:", round(mlp.score(Xte, yte), 4))
```

With two hidden units the network can only bend the boundary once and reaches 0.833. Sixteen units reach 0.947, and a second layer of 16 does not help on this easy two-dimensional problem. Depth pays off on high-dimensional inputs such as images, where it lets the network build features hierarchically (ML 03).

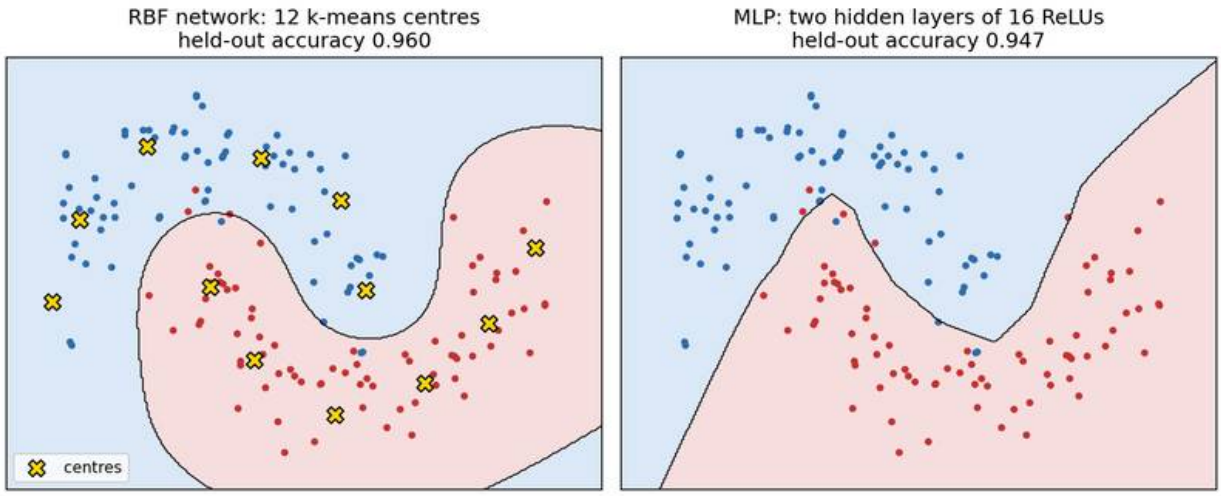


Figure 5 — Two explicit feature maps. Left: an RBF network with 12 k-means centers and a logistic output layer gives smooth, local curves. Right: an MLP with ReLU units gives a piecewise-linear boundary. Held-out points from the same split as the code.

Choosing among nonlinear classifiers

Method	Boundary shape	Training cost	Prediction cost	Key hyperparameters	Typical strengths
k-NN	Any; follows data	None (store data)	$O(ND)$ per query	k, metric, scaling	Simple baseline; good with good embeddings
Kernel SVM	Smooth (RBF)	Roughly $O(N^2)$–$O(N^3)$	$O(N_{\text{sv}}D)$	C, kernel, γ	Small/medium data, strong theory
Gaussian process	Smooth, with uncertainty	$O(N^3)$ exact	$O(N)$ mean, $O(N^2)$ variance	Kernel; set by marginal likelihood	Calibrated uncertainty (regression)
RBF network	Smooth, local	Clustering + linear solve	$O(MD)$	M, γ	Cheap prediction
Decision tree	Axis-aligned boxes	$O(DN \log N)$ per level	$O(\text{depth})$	Depth, leaf size, α	Interpretable; mixed feature types
MLP	Piecewise linear (ReLU)	Iterative SGD	$O(\text{weights})$	Width, depth, decay, learning rate	Scales to large data; learns features

Here N_{sv} is the number of support vectors and M the number of RBF centers. The complexity entries are orders of magnitude, not exact counts. No row wins everywhere. Compare candidates under one validation protocol, and remember the lesson of the grid-search example: differences of a fraction of a percent on one split are rarely real.

Modern view

Kernel methods dominated pattern recognition from the late 1990s until deep networks took over large-scale perception around 2012. They did not disappear; they changed roles. Six strands summarize the current picture.

Surveys of the classical field. Hofmann, Schölkopf and Smola's review in the *Annals of Statistics* [7] is the standard map of kernel methods: positive definite kernels and their RKHS, the representer theorem, kernel construction for structured data such as strings and graphs, and the statistical learning theory behind large margins. Cervantes et al. [19] survey SVM classification from the application side: variants developed where the basic SVM struggles (very large data sets, many classes, imbalanced classes), combinations with evolutionary parameter search, a long catalogue of applications, and the method's limitations. For trees, Costa and Pedreira [20] review recent decision-tree research around the three goals of any predictive learner: fitting the training data, generalizing beyond it, and staying interpretable.

Scaling kernels. The $O(N^2)$ memory of a Gram matrix was the main barrier. Rahimi and Recht's random features [17] approximate a shift-invariant kernel such as the RBF by an explicit random map $z(x) \in \mathbb{R}^R$ with $z(x)^\top z(x') \approx k(x, x')$, so that a linear model on R random features stands in for the kernel machine. Liu et al.'s survey [18] organizes a decade of follow-ups: sampling schemes for the random features, theory on how many features are needed for a given accuracy, and connections to deep networks. Combining such approximations with GPU linear algebra, the FALKON line of work trains kernel ridge regression on billions of points [23], and GPyTorch [24] makes exact and approximate Gaussian process inference practical on GPUs with iterative matrix–matrix solvers.

Kernels as a lens on deep learning. Jacot, Gabriel and Hongler showed that, in an infinite-width limit with suitable scaling, training a neural network by gradient descent is equivalent to kernel regression with a fixed **neural tangent kernel** [21]. Belkin, Ma and Mandal argued that phenomena puzzling in deep learning, such as fitting the training data exactly yet still generalizing, already appear in kernel machines, so understanding kernels is a prerequisite for understanding deep networks [22]. Real networks of finite width do learn their features, which a fixed kernel cannot, so this analogy is a theoretical tool, not a replacement.

Trees still rule tabular data. For data in tables (rows of heterogeneous measurements rather than pixels or words), Grinsztajn, Oyallon and Varoquaux benchmarked tree ensembles against deep architectures on 45 medium-sized datasets and found the tree-based models still ahead, attributing the gap to neural networks' sensitivity to uninformative features, their rotation invariance, and their bias toward smooth functions [25]. The newest challenger is a **tabular foundation model**: TabPFN, a transformer trained across millions of synthetic datasets, predicts on a new small dataset in a single forward pass. Its authors report that it outperforms previous methods, including tuned gradient-boosted tree ensembles, on datasets with up to 10,000 samples, with far less training time [26].

Neighbors on learned features. The nearest-neighbor rule has had a revival as a classifier on top of large pretrained embeddings. Once features are good, a k-NN or cosine-similarity classifier is often the simplest strong baseline, and it can add new classes without retraining. IP 13 and ML

05 discuss this pattern.

Open problems. Exact kernel methods and GPs still scale poorly in N; approximation quality versus speed remains an active trade-off [18]. Choosing kernels for structured data is still largely manual. Single trees remain unstable, and globally optimal trees are expensive to compute [20]. And for tabular data, whether foundation-style models will displace tuned tree ensembles beyond small datasets is an open question as of 2026.

Key takeaways

- Nonlinear boundaries come from three ideas: comparing with stored examples (k-NN), lifting to a feature space where a linear rule works (kernels, RBF networks, MLPs), or partitioning the space (trees).
- k-NN's behavior is set by the metric and scaling as much as by k; for unit vectors, Euclidean and cosine neighbors coincide.
- A kernel is valid exactly when every Gram matrix is symmetric positive semidefinite (Mercer's condition). The kernel trick replaces inner products by kernel values in any algorithm written in inner products.
- The soft-margin SVM minimizes $\frac{1}{2}\|w\|^2$ plus C times hinge loss. Its dual depends on the data only through the kernel, and only support vectors ($\alpha_n > 0$) affect predictions. Tune C and γ jointly, on standardized features, by cross-validation.
- GPs give kernel predictions with uncertainty; the RVM (Tipping) gives sparse probabilistic kernel models. SVM scores are not probabilities.
- RBF networks use a few explicit Gaussian bumps; trees grow greedily by impurity decrease (Gini or entropy) and need pruning; MLPs learn their own features and are covered in depth in ML 02.
- No family wins everywhere: kernels for small/medium smooth problems, trees and their ensembles for tabular data, deep networks for raw perceptual data, neighbors on top of good embeddings.

Exercises

1. **Distance and cosine.** Derive $\|u - v\|^2 = 2 - 2u^\top v$ for unit vectors, and show that the 1-NN under Euclidean distance equals the 1-NN under cosine similarity. Does this still hold if the vectors are not normalized?

   Hint

   Expand $\|u - v\|^2 = u^\top u - 2u^\top v + v^\top v$. With unit norms, distance is a decreasing function of $u^\top v$, so the minimizer of one is the maximizer of the other. Without normalization the norms enter: a long vector pointing in a slightly different direction can be closer in Euclidean distance than a short, perfectly aligned one.

2. **Building a kernel.** Show that $k(x, z) = (x^\top z)^2$ is a valid kernel in $\mathbb{R}^2$ by writing down its feature map. Then use the closure rules to argue that the Gaussian kernel $\exp(-\gamma\|x - z\|^2)$ is valid.

   Hint

   $\phi(x) = (x_1^2, x_2^2, \sqrt{2}x_1x_2)$. For the Gaussian, write
   $\exp(-\gamma\|x - z\|^2) = \exp(-\gamma\|x\|^2)\exp(2\gamma x^\top z)\exp(-\gamma\|z\|^2)$. The middle factor is exp of a valid (scaled linear) kernel, and the outer factors have the form $f(x)\,k(x, z)\,f(z)$.

3. **The limits of γ.** For a fixed finite data set, what does the RBF Gram matrix tend to as $\gamma \to 0$ and as $\gamma \to \infty$? Relate each limit to the leftmost and rightmost columns of Figure 2.

   Hint

   As $\gamma \to 0$ every entry tends to 1 (the all-ones matrix): all points look alike and the boundary becomes nearly linear or constant. As $\gamma \to \infty$ the off-diagonal entries tend to 0 and $K \to I$: each point is only similar to itself, so the SVM can memorize every training label, which is the "islands" regime.

4. **Support vectors and cost.** An RBF-SVM trained on 100,000 images keeps 90% of them as support vectors. What does this imply about the data and about prediction time? Name two things you could try.

   Hint

   Many points sit inside the margin or are misclassified: the classes overlap heavily in this feature space, or C is small or γ is badly chosen. Each prediction needs about 90,000 kernel evaluations. Try re-tuning C and γ, better features, random features with a linear SVM, or a different model family.

5. **Impurity by hand.** A node has 40 points of class A and 40 of class B. Split 1 gives children (30 A, 10 B) and (10 A, 30 B). Split 2 gives (40 A, 20 B) and (0 A, 20 B). Compute the misclassification-error decrease and the Gini decrease for both splits. Which split does each measure prefer?

   Hint

   Root Gini = 0.5, root error = 0.5. Split 1: both children have error 0.25 and Gini $2 \cdot 0.75 \cdot 0.25 = 0.375$; weighted error 0.25, weighted Gini 0.375. Split 2: left child error 1/3, Gini $2 \cdot \frac{2}{3} \cdot \frac{1}{3} = 4/9$; right child pure. Weighted error $\frac{60}{80} \cdot \frac{1}{3} = 0.25$, weighted Gini $\frac{60}{80} \cdot \frac{4}{9} = 1/3$. Error cannot tell them apart (both 0.25); Gini prefers split 2, which creates a pure node.

6. **RBF network design.** In the RBF-network code, vary $M \in \{2, 4, 12, 50\}$ and $\gamma \in \{0.1, 1, 10\}$. Which combinations underfit and which overfit? Why does a small M with a large γ behave badly?

   Hint

   With few centers and a large γ, the bumps are narrow and most points activate none of them strongly, so the design matrix is nearly zero away from the centers and the classifier falls back to the bias. Many centers with a large γ start to behave like a kernel machine with too little regularization.

References

1. S. Theodoridis and K. Koutroumbas, *Pattern Recognition*, 4th ed., Academic Press, 2009, Chapters 2 and 4. publisher page
2. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006, Sections 2.5.2, 5.1–5.3, 6.1–6.4, 7.1–7.2 and 14.4. official book page and PDF
3. T. Cover and P. Hart, "Nearest Neighbor Pattern Classification," *IEEE Transactions on Information Theory*, vol. 13, no. 1, pp. 21–27, 1967. doi:10.1109/TIT.1967.1053964
4. J. Mercer, "Functions of Positive and Negative Type, and Their Connection with the Theory of Integral Equations," *Philosophical Transactions of the Royal Society of London, Series A*, vol. 209, pp. 415–446, 1909. doi:10.1098/rsta.1909.0016
5. B. E. Boser, I. M. Guyon, and V. N. Vapnik, "A Training Algorithm for Optimal Margin Classifiers," *Proceedings of the Fifth Annual Workshop on Computational Learning Theory* (*COLT*), pp. 144–152, 1992. doi:10.1145/130385.130401
6. C. Cortes and V. Vapnik, "Support-Vector Networks," *Machine Learning*, vol. 20, pp. 273–297, 1995. doi:10.1007/BF00994018
7. T. Hofmann, B. Schölkopf, and A. J. Smola, "Kernel Methods in Machine Learning," *The Annals of Statistics*, vol. 36, no. 3, 2008. doi:10.1214/009053607000000677
8. C. E. Rasmussen and C. K. I. Williams, *Gaussian Processes for Machine Learning*, MIT Press, 2006. book page
9. M. E. Tipping, "Sparse Bayesian Learning and the Relevance Vector Machine," *Journal of Machine Learning Research*, vol. 1, pp. 211–244, 2001. JMLR
10. J. Moody and C. J. Darken, "Fast Learning in Networks of Locally-Tuned Processing Units," *Neural Computation*, vol. 1, no. 2, pp. 281–294, 1989. doi:10.1162/neco.1989.1.2.281
11. J. Park and I. W. Sandberg, "Universal Approximation Using Radial-Basis-Function Networks," *Neural Computation*, vol. 3, no. 2, pp. 246–257, 1991. doi:10.1162/neco.1991.3.2.246
12. L. Breiman, J. H. Friedman, R. A. Olshen, and C. J. Stone, *Classification and Regression Trees*, Chapman and Hall/CRC, 2017 edition. doi:10.1201/9781315139470
13. J. R. Quinlan, "Induction of Decision Trees," *Machine Learning*, vol. 1, pp. 81–106, 1986. doi:10.1007/BF00116251
14. D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning Representations by Back-Propagating Errors," *Nature*, vol. 323, pp. 533–536, 1986. doi:10.1038/323533a0
15. G. Cybenko, "Approximation by Superpositions of a Sigmoidal Function," *Mathematics of Control, Signals, and Systems*, vol. 2, pp. 303–314, 1989. doi:10.1007/BF02551274
16. K. Hornik, "Approximation Capabilities of Multilayer Feedforward Networks," *Neural Networks*, vol. 4, no. 2, pp. 251–257, 1991. doi:10.1016/0893-6080(91)90009-T
17. A. Rahimi and B. Recht, "Random Features for Large-Scale Kernel Machines," *Advances in Neural Information Processing Systems 20* (*NIPS*), 2007. paper
18. F. Liu, X. Huang, Y. Chen, and J. A. K. Suykens, "Random Features for Kernel Approximation: A Survey on Algorithms, Theory, and Beyond," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 10, pp. 7128–7148, 2022. doi:10.1109/TPAMI.2021.3097011
19. J. Cervantes, F. Garcia-Lamont, L. Rodríguez-Mazahua, and A. Lopez, "A Comprehensive Survey on Support Vector Machine Classification: Applications, Challenges and Trends," *Neurocomputing*, vol. 408, pp. 189–215, 2020. doi:10.1016/j.neucom.2019.10.118

20. V. G. Costa and C. E. Pedreira, “Recent Advances in Decision Trees: An Updated Survey,” *Artificial Intelligence Review*, vol. 56, pp. 4765–4800, 2023. doi:10.1007/s10462-022-10275-5
21. A. Jacot, F. Gabriel, and C. Hongler, “Neural Tangent Kernel: Convergence and Generalization in Neural Networks,” *NeurIPS*, 2018. arXiv:1806.07572
22. M. Belkin, S. Ma, and S. Mandal, “To Understand Deep Learning We Need to Understand Kernel Learning,” arXiv:1802.01396, 2018. arXiv
23. G. Meanti, L. Carratino, L. Rosasco, and A. Rudi, “Kernel Methods Through the Roof: Handling Billions of Points Efficiently,” *Advances in Neural Information Processing Systems 33 (NeurIPS)*, 2020. paper
24. J. R. Gardner, G. Pleiss, D. Bindel, K. Q. Weinberger, and A. G. Wilson, “GPyTorch: Blackbox Matrix-Matrix Gaussian Process Inference with GPU Acceleration,” *NeurIPS*, 2018. arXiv:1809.11165
25. L. Grinsztajn, E. Oyallon, and G. Varoquaux, “Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data?,” *Advances in Neural Information Processing Systems 35 (NeurIPS), Datasets and Benchmarks Track*, 2022. paper
26. N. Hollmann, S. Müller, L. Purucker, A. Krishnakumar, et al., “Accurate Predictions on Small Data with a Tabular Foundation Model,” *Nature*, vol. 637, pp. 319–326, 2025. doi:10.1038/s41586-024-08328-6

PR 05 • Feature Selection, Transforms and Dimensionality

Choose and build features: filter, wrapper and embedded selection, class-separability criteria, PCA, Fisher LDA, kernel PCA, ICA and manifold learning (Isomap, LLE, t-SNE, UMAP), and how to validate a feature pipeline without leakage.

Online edition: https://blog.shyandram.dev/en/learn/pattern-recognition/pr-05-features-dimensionality/

What you'll learn

- Why fewer, better features often beat more features, and the difference between feature **selection**, feature **extraction** and feature **learning**.
- Filter, wrapper and embedded selection, and the most common mistake: selecting features outside cross-validation.
- How to score features and feature subsets: Fisher ratio, scatter-matrix criteria, Bhattacharyya distance and mutual information.
- Linear extraction: PCA (with whitening) and Fisher's linear discriminant, derived and compared.
- Nonlinear and non-Gaussian extraction: kernel PCA, ICA (and NMF), and manifold learning with Isomap, LLE, t-SNE and UMAP, including what their pictures can and cannot prove.
- Classic hand-designed image and signal descriptors, and where all of this stands in 2026.

The big picture

Two recognition systems can use exactly the same classifier and still behave very differently, because they describe the input differently. A texture histogram throws away where things are. A shape descriptor may ignore color. An embedding learned from web photos may quietly encode the background. Every representation keeps some variation and discards the rest, and the classifier can only use what was kept.

> **Explain like I'm five**
>
> Describe a toy to a friend so they can pick it out of a toy box. "It's red, round and the size of an orange" works. "It's made of atoms" is true but useless. Telling them all 500 things you can see is too much to remember. A good description is short and keeps exactly the clues that tell this toy apart from the others.

This chapter is about building that short, useful description. There are three ways to get one:

- **Selection** keeps a subset of the original measurements, for example 20 of 5,000 gene-expression levels. The kept features remain interpretable.
- **Extraction** computes new coordinates as functions of all measurements, for example the projection onto a few principal components.
- **Learning** estimates an extraction rule from data, often with a deep network (ML 05). PCA is already a (very simple) learned extraction rule; the categories overlap.

Why bother, if a flexible classifier could use all the features? Three reasons. With a fixed number of training samples, adding weakly informative features can make a trained classifier *worse*, because each extra dimension adds parameters to estimate and room to overfit: the "curse of dimensionality" and the peaking phenomenon discussed by Theodoridis and Koutroumbas [1]. Fewer features are cheaper to measure, store and compute. And a small, meaningful feature set helps people understand what the classifier relies on. Guyon and Elisseeff list the same three motives in their classic introduction to feature selection [3]. The textbook background for this chapter is Theodoridis and Koutroumbas, Chapters 5–7 [1], and Bishop, Sections 4.1.4–4.1.6 and 12.1–12.4 [2]. For hand-designed image features, see also IP 12.

Feature selection

In plain words. Feature selection asks: which of the measurements I already have should I keep? You need a way to score features (or groups of features) and a way to search, because trying every subset is impossible.

With D features there are $2^D - 1$ non-empty subsets: about 10^{30} for $D = 100$. Exhaustive search is out of the question except for tiny D. The approaches differ in how they score and how they search [3], [22], [23].

Filter methods

A **filter** scores each feature (or subset) with a criterion computed directly from the data, independently of the final classifier, then keeps the best. Typical criteria are variance (drop near-constant features), a class-separation statistic such as the ANOVA F statistic or the Fisher ratio (next section), correlation with the label, and the mutual information

$$I(x_j; y) = \sum_{v} \sum_{c} p(x_j = v, y = c) \, \log \frac{p(x_j = v, y = c)}{p(x_j = v)\, P(y = c)},$$

where x_j is feature j (discretized to values v), y is the class label with values c, and p, P are joint and marginal probabilities estimated from the training set. $I(x_j; y) = 0$ exactly when the feature and the label are statistically independent.

Filters are fast and classifier-agnostic, but univariate filters have two blind spots. **Redundancy**: two copies of the same good feature both score high, yet the second adds nothing. Peng, Long and Ding's mRMR criterion addresses this by rewarding relevance to the label while penalizing mutual information with already-selected features [6]. **Interactions**: two features can each be

useless alone and decisive together. In the exclusive-or pattern $y = \mathbf{1}[x_1 > 0] \oplus \mathbf{1}[x_2 > 0]$, each feature on its own is independent of y, so every univariate filter discards both.

Wrapper methods

A **wrapper** scores a candidate subset by training the actual classifier on it and measuring validation performance [4]. Because exhaustive search is impossible, wrappers use heuristics such as:

- **sequential forward selection**: start empty, repeatedly add the feature that most improves the validation score;
- **sequential backward elimination**: start with all features, repeatedly remove the least useful one;
- floating variants that may undo earlier steps [1].

Wrappers see interactions and redundancy through the classifier's eyes, but they are expensive (a full training run per candidate) and, because they try many subsets against the same validation data, they can overfit the validation set itself. Kohavi and John's paper that introduced the term is still the standard reference for these trade-offs [4].

Embedded methods

An **embedded** method selects features as a side effect of training. The best-known example is the lasso, which adds an ℓ_1 penalty to a linear model [5]:

$$\hat{w} = \arg\min_{w} \; \frac{1}{2N} \sum_{n=1}^{N} \left(t_n - w^{\top} x_n\right)^2 + \lambda \lVert w \rVert_1, \qquad \lVert w \rVert_1 = \sum_{j=1}^{D} |w_j|,$$

with targets t_n, inputs x_n and penalty strength $\lambda \geq 0$. The corners of the ℓ_1 ball make many coordinates of $\hat{w}$ exactly zero, so the surviving coordinates are the selected features. The same idea applies to logistic regression for classification. Decision trees and their ensembles (PR 04, PR 09) also embed selection: a feature that never helps a split is never used.

The selection-bias trap

Supervised selection uses the labels. If you select features on the full data set and *then* cross-validate only the classifier, the held-out folds have already influenced which features were chosen, and the error estimate is optimistic. Ambroise and McLachlan showed that this mistake made published gene-expression classifiers look nearly perfect; with selection performed inside each cross-validation fold, the error was no longer zero [7]. The code makes the effect concrete with features that are pure noise.

```
import numpy as np
from sklearn.feature_selection import SelectKBest, f_classif
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline

rng = np.random.default_rng(0)
X = rng.normal(size=(60, 5000))          # 60 samples, 5000 pure-noise features
y = rng.integers(0, 2, size=60)          # labels unrelated to X

# Wrong: select on all data, then cross-validate the classifier only.
X_sel = SelectKBest(f_classif, k=20).fit_transform(X, y)
wrong = cross_val_score(LogisticRegression(max_iter=1000), X_sel, y, cv=5)

# Right: selection is part of the model and is refitted in every fold.
model = make_pipeline(SelectKBest(f_classif, k=20),
                      LogisticRegression(max_iter=1000))
right = cross_val_score(model, X, y, cv=5)
print(f"selection outside CV: {wrong.mean():.2f}")
print(f"selection inside CV:  {right.mean():.2f}")
```

The labels are random, so no method can truly beat 50% accuracy. Selecting outside cross-validation reports 0.98. Putting the selector inside the pipeline reports 0.58, which is chance level up to the noise of a 60-sample estimate. The rule generalizes: every step that looks at labels (selection, supervised projection such as LDA, choosing the number of features) belongs inside the training procedure. Even unsupervised steps such as PCA and scaling should be fitted on training folds only, so that the procedure you evaluate is the procedure you will deploy.

Evaluating features and feature subsets

In plain words. A good feature puts the classes far apart compared with how spread out each class is. Several formulas turn this idea into a number, from a simple per-feature ratio to criteria for whole subsets.

Fisher's discriminant ratio for a single feature compares the spread *between* class means with the spread *within* classes:

$$\mathrm{FDR}_j = \frac{\sum_c P_c \, (\mu_{cj} - \mu_j)^2}{\sum_c P_c \, \sigma_{cj}^2},$$

where P_c is the proportion of class c, μ_{cj} and σ_{cj}^2 are the mean and variance of feature j in class c, and μ_j is the overall mean [1]. (For two classes, Theodoridis and Koutroumbas write it as $(\mu_1 - \mu_2)^2/(\sigma_1^2 + \sigma_2^2)$; the idea is the same.)

For a **subset** of features, use the scatter matrices. With x the feature vector restricted to the subset and μ_c, μ the class and overall mean vectors,

$$S_W = \sum_c P_c \, \Sigma_c, \qquad S_B = \sum_c P_c \, (\mu_c - \mu)(\mu_c - \mu)^\top, \qquad J = \mathrm{tr}(S_W^{-1} S_B),$$

where Σ_c is the covariance of class c, S_W is the within-class scatter, S_B is the between-class scatter, and tr is the trace. J is large when class means are far apart relative to the within-class spread, measured in the directions that matter [1]. Unlike summing per-feature ratios, J accounts for correlation between the features.

Two further criteria come from probability. The **Bhattacharyya distance** between two Gaussian classes $\mathcal{N}(\mu_1, \Sigma_1)$ and $\mathcal{N}(\mu_2, \Sigma_2)$ is

$$B = \frac{1}{8}(\mu_1 - \mu_2)^\top \Big(\frac{\Sigma_1 + \Sigma_2}{2}\Big)^{-1}(\mu_1 - \mu_2) + \frac{1}{2}\ln \frac{\big|\frac{1}{2}(\Sigma_1 + \Sigma_2)\big|}{\sqrt{|\Sigma_1|\,|\Sigma_2|}},$$

with $|\cdot|$ the determinant. The first term measures mean separation; the second measures covariance difference. It gives an upper bound on the Bayes error for equal priors, $P_e \le \frac{1}{2}e^{-B}$ [1]. **Mutual information** with the label, introduced above, is the distribution-free counterpart.

The code compares single-feature scores with a subset criterion on the 13-feature wine data.

```
import numpy as np
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler

X, y = load_wine(return_X_y=True)
X = StandardScaler().fit_transform(X)
classes = np.unique(y)
mu = X.mean(axis=0)

# Per-feature Fisher ratio: between-class over within-class variance.
between = sum((y == c).mean() * (X[y == c].mean(0) - mu) ** 2 for c in classes)
within = sum((y == c).mean() * X[y == c].var(0) for c in classes)
fdr = between / within
print("top-3 features by Fisher ratio:", np.argsort(fdr)[::-1][:3],
      np.round(np.sort(fdr)[::-1][:3], 2))

def J(cols):  # scatter-matrix criterion trace(Sw^-1 Sb) on a feature subset
    Z = X[:, cols]
    m = Z.mean(0)
    Sw = sum((y == c).mean() * np.cov(Z[y == c].T, bias=True).reshape(len(cols), -1)
             for c in classes)
    Sb = sum((y == c).mean() * np.outer(Z[y == c].mean(0) - m, Z[y == c].mean(0) - m)
             for c in classes)
    return np.trace(np.linalg.solve(Sw, Sb))

top2 = list(np.argsort(fdr)[::-1][:2])
print("J(top-2 by ratio) =", round(J(top2), 2))
best_pair = max(((i, j) for i in range(13) for j in range(i + 1, 13)),
                key=lambda p: J(list(p)))
print("best pair by J:", best_pair, "J =", round(J(list(best_pair)), 2))
```

Features 6, 12 and 11 have the highest individual ratios (2.67, 2.38, 2.17). Yet the best *pair* by J is features 6 and 9 ($J = 5.39$), not the top two individual features ($J = 4.93$): feature 12 overlaps in

information with feature 6, while feature 9 complements it. This is the redundancy problem of univariate filters, seen in numbers.

All of these criteria are proxies. The final judge of a feature set is the cross-validated performance of the complete pipeline, estimated without leakage (PR 09).

Principal component analysis

In plain words. PCA rotates the coordinate system so that the first axis points in the direction where the data vary most, the second in the direction of most remaining variation, and so on. Keeping only the first few axes gives the best low-dimensional summary in the least-squares sense. It never looks at the labels.

Derivation

Let $x_1, \dots, x_N \in \mathbb{R}^D$ have mean $\bar{x}$ and covariance $\Sigma = \frac{1}{N-1} \sum_n (x_n - \bar{x})(x_n - \bar{x})^\top$. The variance of the data projected onto a unit vector u is $u^\top \Sigma u$. Maximizing it subject to $u^\top u = 1$ with a Lagrange multiplier λ gives

$$\Sigma u = \lambda u,$$

so the best direction is an eigenvector of Σ, and the variance it captures is its eigenvalue λ. The first principal direction is the eigenvector with the largest eigenvalue; the next ones are the remaining eigenvectors in decreasing order of eigenvalue, each orthogonal to the previous ones [2], [8], [9]. Keeping the top r directions also minimizes the mean squared reconstruction error, which equals the sum of the discarded eigenvalues $\sum_{i>r} \lambda_i$ [2].

In practice PCA is computed with the singular value decomposition (SVD) of the centered data matrix $X_c \in \mathbb{R}^{N \times D}$ (row n is $x_n - \bar{x}$):

$$X_c = U S V^\top, \qquad \lambda_i = \frac{s_i^2}{N-1}, \qquad Z = X_c V_r,$$

where U and V have orthonormal columns, $S = \operatorname{diag}(s_1, s_2, \dots)$ holds the singular values in decreasing order, V_r contains the first r columns of V (the principal directions), and $Z \in \mathbb{R}^{N \times r}$ are the **scores**. The fraction $\lambda_i / \sum_k \lambda_k$ is the **explained variance ratio** of component i. To reconstruct, compute $\hat{x} = \bar{x} + V_r z$.

Whitening

Whitening rescales each score to unit variance:

$$z_i^{\text{white}} = \frac{z_i}{\sqrt{\lambda_i + \epsilon}},$$

where z_i is the score on component i, λ_i is that component's variance from the training data, and $\epsilon \geq 0$ is a small constant that keeps tiny eigenvalues from blowing up. After whitening

the training covariance is (approximately) the identity. This makes all directions equally loud, which helps some algorithms (ICA below, k-means, some neural networks) but also amplifies directions that were small because they were mostly noise. Whitening is a modeling choice, not a free improvement.

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA

X, _ = load_digits(return_X_y=True)          # 1797 images of 8x8 = 64 pixels
Xc = X - X.mean(axis=0)
U, S, Vt = np.linalg.svd(Xc, full_matrices=False)
eigvals = S ** 2 / (len(X) - 1)              # variances along each direction
ratio = eigvals / eigvals.sum()
r = int(np.searchsorted(np.cumsum(ratio), 0.90) + 1)
print("components for 90% variance:", r)

Z = Xc @ Vt[:r].T                            # scores, N x r
X_hat = Z @ Vt[:r] + X.mean(axis=0)          # reconstruction
print("relative reconstruction error:",
      round(np.linalg.norm(X - X_hat) / np.linalg.norm(Xc), 3))

pca = PCA(n_components=r).fit(X)
assert np.allclose(pca.explained_variance_, eigvals[:r])
assert np.allclose(np.abs(pca.components_), np.abs(Vt[:r]))   # equal up to sign
Zw = Z / np.sqrt(eigvals[:r])                # whitened scores
print("whitened covariance is identity:",
      np.allclose(np.cov(Zw.T), np.eye(r)))
```

Twenty-one of the 64 pixel dimensions hold 90% of the variance of the digits, and the reconstruction from them has a relative error of 0.31. The hand-written SVD agrees with scikit-learn up to the sign of each component (an eigenvector's sign is arbitrary). Figure 1 shows the cumulative curve and reconstructions.

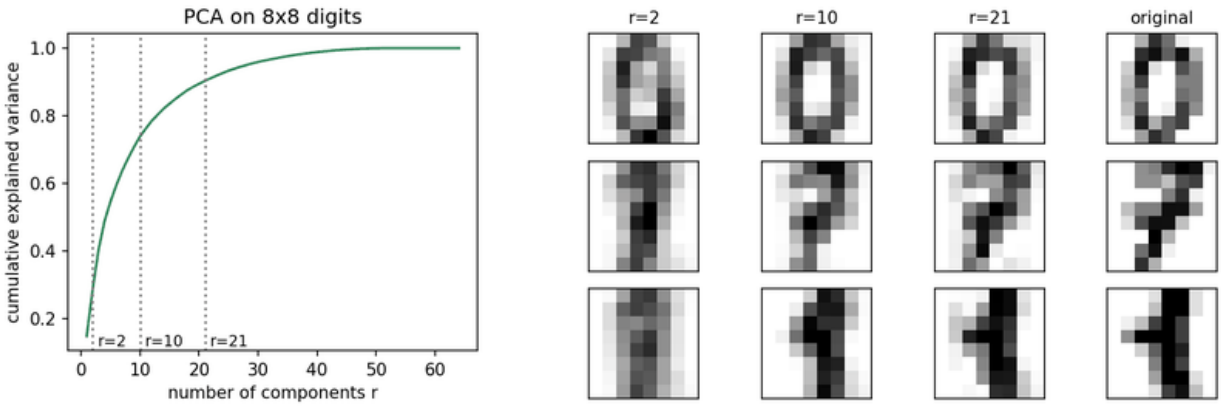


Figure 1 — PCA on the scikit-learn digits. Two components give blurry blobs; ten already show the digit; 21 components (90% of the variance) are close to the original.

Variance is not class information

PCA ranks directions by variance, and variance is not the same as usefulness for classification. In Figure 2, a large nuisance variation (think lighting) dominates the first principal component, while the class difference lives in a direction with small variance. Projecting onto PC 1 keeps 96% of the variance and throws away essentially all the class information. A supervised projection (Fisher's LDA, next section) finds the useful direction immediately.

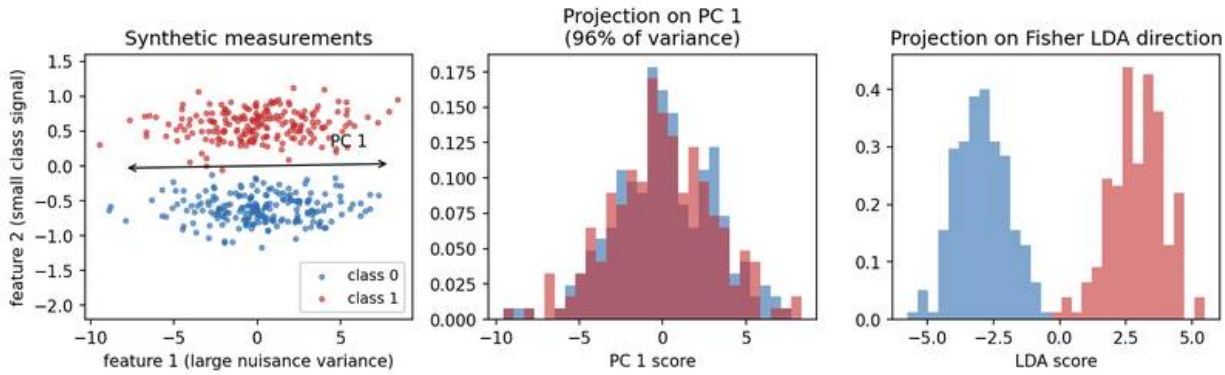


Figure 2 — Variance is not class information. PC 1 (96% of the variance) follows the nuisance axis and mixes the classes; the Fisher LDA direction separates them. Synthetic data; generated by scripts/figures/pr_05_features_dimensionality_extra.py.

Jolliffe and Cadima's review [10] covers what PCA can and cannot do, and its many variants (for example sparse and robust PCA). Probabilistic PCA and factor analysis, which turn PCA into a latent-variable model, are developed in Bishop, Chapter 12 [2], and in ML 05A.

Fisher's linear discriminant analysis

In plain words. LDA looks for the directions in which the class means are far apart *relative to* how spread out each class is. It uses the labels, so it finds directions that matter for classification, but it can produce at most one fewer direction than there are classes.

Fisher's criterion for a projection direction v is the ratio of between-class to within-class scatter along v [11]:

$$J(v) = \frac{v^\top S_B v}{v^\top S_W v},$$

with S_W and S_B as defined in the previous section. Setting the gradient to zero gives the **generalized eigenproblem**

$$S_B v = \lambda S_W v,$$

and the best directions are the eigenvectors with the largest eigenvalues λ (equivalently, eigenvectors of $S_W^{-1} S_B$ when S_W is invertible) [1], [2]. For two classes there is a closed form, $v \propto$

$S_W^{-1}(\mu_1 - \mu_2)$, which reappears as the Gaussian LDA classifier in PR 03.

Why at most $K-1$ directions. S_B is a weighted sum of K rank-one matrices $(\mu_c - \mu)(\mu_c - \mu)^\top$. The vectors $\mu_c - \mu$ are linearly dependent, because $\sum_c P_c(\mu_c - \mu) = 0$. So S_B has rank at most $K-1$, and at most $K-1$ eigenvalues are nonzero. Ten digit classes give at most nine discriminant directions, whatever the input dimension.

When S_W is singular. With fewer samples than features (common with images), S_W cannot be inverted. Standard fixes are to apply PCA first and then LDA in the reduced space, or to shrink S_W toward a multiple of the identity (regularized LDA).

```
from sklearn.datasets import load_digits
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis as LDA
from sklearn.model_selection import cross_val_score
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

X, y = load_digits(return_X_y=True)
lda = LDA().fit(X, y)
print("discriminant directions for 10 classes:", lda.transform(X).shape[1])
for name, reducer in [("PCA", PCA(n_components=2)),
                      ("LDA", LDA(n_components=2))]:
    pipe = make_pipeline(StandardScaler(), reducer,
                         KNeighborsClassifier(n_neighbors=5))
    acc = cross_val_score(pipe, X, y, cv=5).mean()
    print(f"{name} to 2-D, then 5-NN: CV accuracy {acc:.3f}")
```

LDA returns nine directions for ten classes, as predicted. Squeezed to two dimensions, a 5-NN classifier on LDA coordinates reaches a cross-validated accuracy of 0.624, against 0.512 on the first two principal components. Because LDA uses labels, it sits inside the pipeline and is refitted in every fold. Figure 3 shows why: LDA pulls whole digit classes apart, while PCA mostly shows the directions of largest pixel variation.

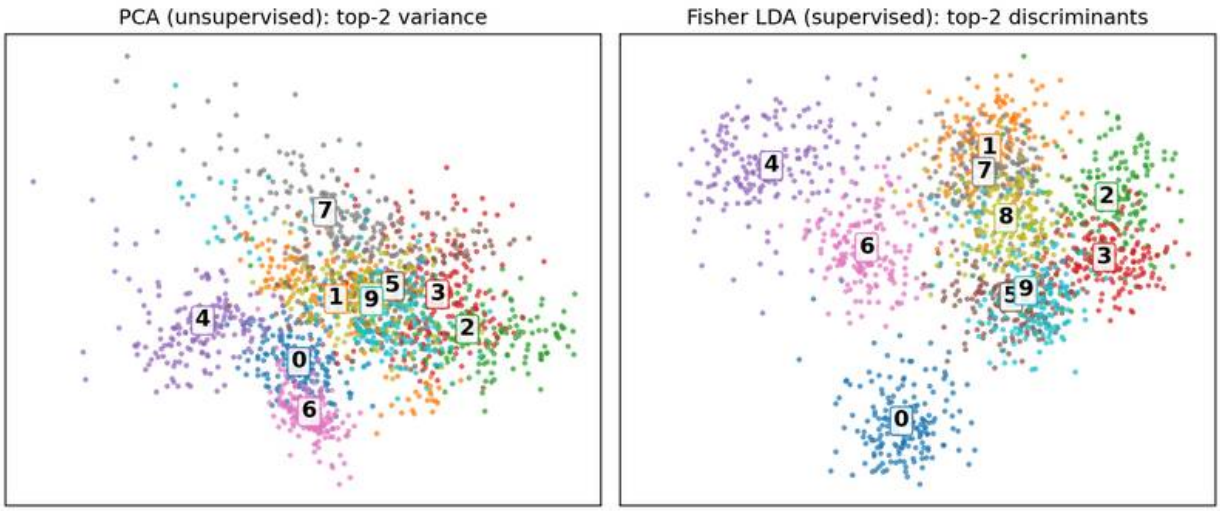


Figure 3 — Digits projected to two dimensions. Left: the top-2 principal components (no labels used). Right: the top-2 Fisher discriminants (labels used). Class labels mark each class mean.

Kernel PCA

In plain words. Kernel PCA is PCA performed in the rich feature space of a kernel (PR 04), without ever computing the features. It can unroll structures that no linear projection can.

Schölkopf, Smola and Müller showed that PCA in the feature space $\phi(x)$ only needs the kernel matrix $K_{ij} = k(x_i, x_j)$ [12]. Center it in feature space,

$$\tilde{K} = K - \mathbf{1}_N K - K\mathbf{1}_N + \mathbf{1}_N K \mathbf{1}_N,$$

where $\mathbf{1}_N$ is the $N \times N$ matrix with every entry $1/N$. Solve the eigenproblem $\tilde{K}a_i = N\lambda_i a_i$ for coefficient vectors $a_i \in \mathbb{R}^N$, normalized so that $N\lambda_i a_i^\top a_i = 1$. The i-th kernel principal component of a point x is then

$$z_i(x) = \sum_{n=1}^{N} a_{in}\, \tilde{k}(x_n, x),$$

where $\tilde{k}$ is the kernel centered in the same way. The cost is that of an $N \times N$ eigenproblem, and a new point needs N kernel evaluations.

Kernel PCA is still unsupervised: it ranks components by variance in feature space, not by class separation. On the two-ring data, the first two RBF kernel components describe the *angle* around the rings and mix the classes; the third component describes the *radius* and separates them perfectly (Figure 4).

```
import numpy as np
from sklearn.datasets import make_circles
```

```
from sklearn.decomposition import PCA, KernelPCA
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import cross_val_score
from sklearn.pipeline import make_pipeline

X, y = make_circles(n_samples=400, factor=0.4, noise=0.07, random_state=0)
Z = KernelPCA(n_components=4, kernel="rbf", gamma=2).fit_transform(X)
print("|corr(component, label)|:",
      np.round([abs(np.corrcoef(Z[:, i], y)[0, 1]) for i in range(4)], 3))

for name, reducer in [("linear PCA, 2 comps", PCA(n_components=2)),
                      ("kernel PCA, 3 comps", KernelPCA(n_components=3,
                                                        kernel="rbf", gamma=2))]:
    acc = cross_val_score(make_pipeline(reducer, LogisticRegression()),
                          X, y, cv=5).mean()
    print(f"{name} -> logistic regression: {acc:.3f}")
```

Only the third component correlates with the label (0.982; the others are below 0.05). A logistic regression on the two linear principal components cannot beat chance (0.432), while on three kernel components it is perfect (1.000).

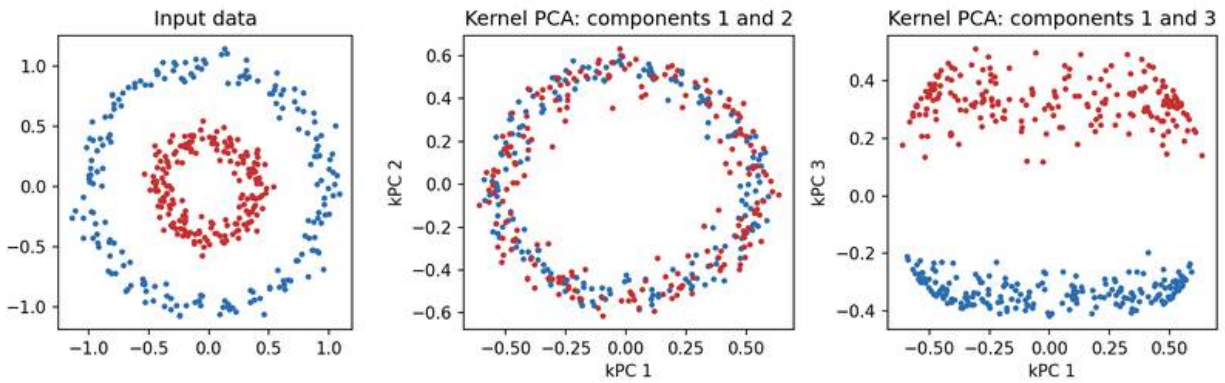


Figure 4 — Kernel PCA (RBF, $\gamma = 2$) on two rings. The leading components capture the angle and mix the classes; component 3 captures the radius and separates them. High variance in feature space still does not mean high class information.

Independent component analysis

In plain words. At a party, two microphones each record a mixture of two voices. ICA tries to recover the individual voices from the mixtures alone, using the fact that the voices are statistically independent and not Gaussian.

ICA assumes the observed vector is a linear mixture of independent sources [13]:

$$x = As,$$

where $s = (s_1, \dots, s_M)$ are statistically independent, non-Gaussian source signals, A is an unknown mixing matrix and x are the observed signals. The goal is an unmixing matrix $W \approx A^{-1}$ such that $\hat{s} = Wx$ has independent components. Why non-Gaussian? By the central limit theorem, mixtures tend to be "more Gaussian" than their sources, so maximizing the non-Gaussianity of $w^\top x$ finds a source. FastICA measures non-Gaussianity with approximations of negentropy and runs a fixed-point iteration after whitening the data [13]. Two ambiguities can never be resolved: the order of the sources and their scale (including sign).

The contrast with PCA is the point. PCA only makes the components *uncorrelated* (second-order statistics); ICA makes them *independent*, using higher-order statistics. If the sources were Gaussian, uncorrelated would already mean independent, and any rotation of the whitened data would be equally good, so ICA would fail.

```
import numpy as np
from sklearn.decomposition import FastICA

t = np.linspace(0, 8, 2000)
S = np.c_[np.sin(3 * t),                      # source 1: sine
          np.sign(np.sin(5 * t)),             # source 2: square wave
          ((2 * t) % 2) - 1]                  # source 3: sawtooth
A = np.array([[1.0, 0.5, 0.3],
              [0.6, 1.0, 0.4],
              [0.4, 0.3, 1.0]])               # unknown mixing matrix
X = S @ A.T                                   # what the sensors record

S_hat = FastICA(n_components=3, whiten="unit-variance",
                random_state=0).fit_transform(X)
corr = np.abs(np.corrcoef(S.T, S_hat.T)[:3, 3:])
print(np.round(corr, 3))                      # one entry near 1 per row
```

Each true source matches exactly one recovered component with absolute correlation 0.999 or 1.000, and the off-diagonal matches are below 0.05. Figure 5 shows that PCA, applied to the same mixtures, returns uncorrelated signals that are still mixtures.

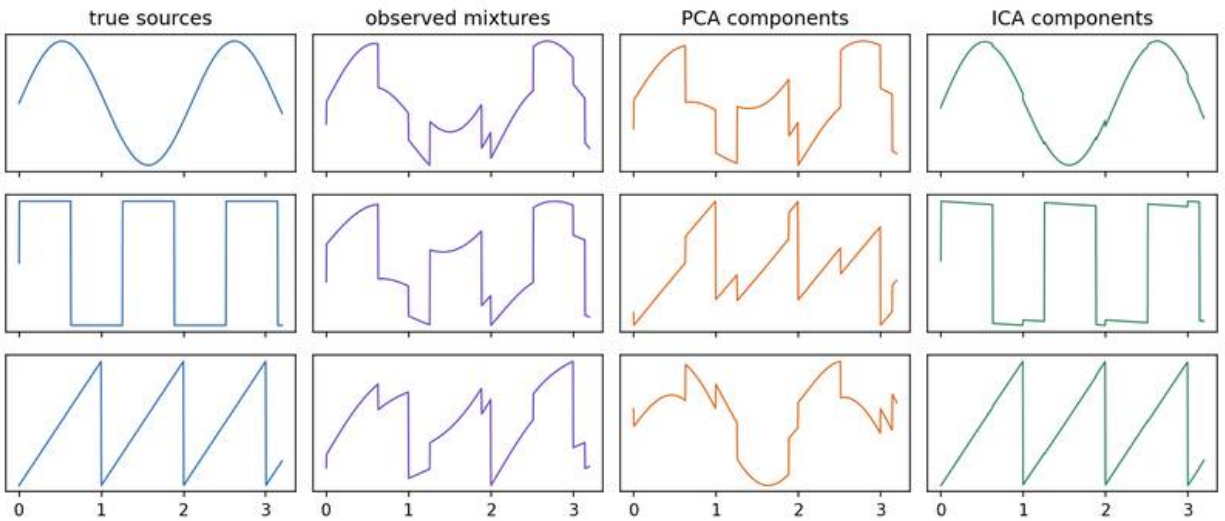


Figure 5 — Blind source separation. PCA decorrelates the mixtures but does not unmix them; FastICA recovers the sine, square and sawtooth sources up to order and scale.

Non-negative matrix factorization (NMF) is a related decomposition for non-negative data such as pixel intensities or word counts: it writes $X \approx WH$ with $W, H \geq 0$ elementwise. Because nothing can be subtracted, the factors tend to be additive "parts," as Lee and Seung showed for face images [14].

Manifold learning

In plain words. Sometimes high-dimensional data lie on a curved, low-dimensional sheet, like a rolled-up poster. Manifold learning tries to unroll the sheet, keeping points that were neighbors along the sheet close together, without being fooled by points that are close in 3-D only because the sheet is rolled up.

The Swiss roll in Figure 6 is the standard example: a two-dimensional rectangle rolled up in three dimensions. PCA flattens it by projection and puts different layers of the roll on top of each other. Neighborhood-based methods do better:

- **Isomap** [15] builds a k-nearest-neighbor graph, estimates *geodesic* distances along the manifold as shortest-path lengths in the graph, and then finds low-dimensional coordinates whose Euclidean distances match those geodesic distances (classical multidimensional scaling).
- **Locally linear embedding** (LLE) [16] writes each point as a weighted combination of its neighbors, $x_n \approx \sum_{m \in \mathcal{N}(n)} W_{nm} x_m$ with $\sum_m W_{nm} = 1$, and then finds low-dimensional points y_n that are reconstructed by the *same* weights.
- **t-SNE** [17] converts distances into neighbor probabilities p_{ij} in the input (Gaussian, with a bandwidth set by a "perplexity" parameter) and q_{ij} in the embedding (heavy-tailed Student-t), and minimizes the Kullback–Leibler divergence $\sum_{i \neq j} p_{ij} \log(p_{ij}/q_{ij})$. The heavy tails let well-separated groups spread out.

- **UMAP** [18] builds a weighted k-nearest-neighbor graph (a "fuzzy" topological representation) and optimizes a low-dimensional layout with a cross-entropy objective. It is typically faster than t-SNE and is often used for the same purpose.

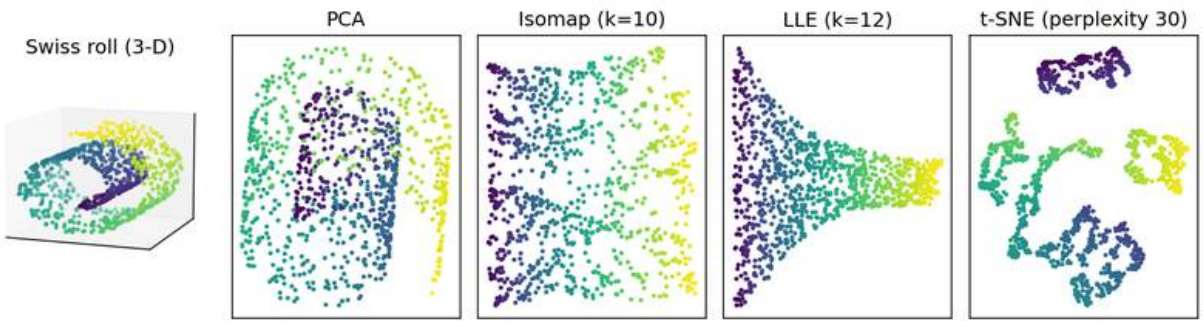


Figure 6 — The Swiss roll and four 2-D embeddings, colored by position along the roll. Isomap and LLE unroll it; PCA cannot; t-SNE keeps neighbors together but tears the continuous sheet into separate islands.

One quantitative check is **trustworthiness**: the extent to which the k nearest neighbors of each point in the embedding were also near neighbors in the input (1.0 is perfect).

```
from sklearn.datasets import make_swiss_roll
from sklearn.decomposition import PCA
from sklearn.manifold import Isomap, LocallyLinearEmbedding, TSNE, trustworthiness

X, t = make_swiss_roll(n_samples=1000, noise=0.05, random_state=0)
methods = {
    "PCA": PCA(n_components=2),
    "Isomap": Isomap(n_neighbors=10, n_components=2),
    "LLE": LocallyLinearEmbedding(n_neighbors=12, n_components=2,
                                  random_state=0),
    "t-SNE": TSNE(n_components=2, perplexity=30, init="pca",
                  random_state=0),
}
for name, method in methods.items():
    Y = method.fit_transform(X)
    print(f"{name:7s} trustworthiness (k=10): "
          f"{trustworthiness(X, Y, n_neighbors=10):.3f}")
```

PCA scores 0.905; Isomap and t-SNE 0.999; LLE 0.996. Note what the number does *not* see: t-SNE scores as high as Isomap even though Figure 6 shows it tearing the roll into islands. Local neighborhoods are preserved; global structure is not.

What an embedding picture can and cannot prove

These methods are mainly tools for looking at data. Three cautions follow from how they work [19]:

- **Cluster sizes and gaps are not meaningful by default.** t-SNE normalizes local densities, so a tight cluster and a diffuse one can look the same size, and distances between clusters may say little.
- **Results depend on settings.** Perplexity, number of neighbors, initialization and random seed all change the picture. Wattenberg, Viégas and Johnson's interactive article shows the same data producing very different t-SNE maps [19].
- **Two dimensions may not be enough.** A projection to 2-D must distort high-dimensional distances; well-separated islands in a picture are not proof of separate populations.

There is also a practical gap for classification: t-SNE and standard LLE learn coordinates for the training points only, with no natural rule for embedding a new point. Isomap, kernel PCA and UMAP provide out-of-sample transforms, but if a nonlinear embedding is part of a classifier, it must be fitted inside the training folds like any other step.

Image and signal descriptors

In plain words. Before features were learned, people designed them by hand to measure exactly the properties they believed mattered: texture, edges, shape, frequency content. They remain useful as baselines and as interpretable checks.

Theodoridis and Koutroumbas devote two chapters to transform-based and application-specific features [1]; IP 12 treats image descriptors in detail. Consider what each one keeps and what it is sensitive to:

Descriptor	Preserves	Typical sensitivity
Intensity histogram	Distribution of values	Loses spatial arrangement entirely
Co-occurrence (GLCM) statistics [20]	Pairwise intensity relations at an offset	Quantization levels and offset direction
Histogram of oriented gradients (HOG) [21]	Local edge directions in a grid of cells	Cropping, scale and alignment
Area, perimeter, compactness	Region geometry	Segmentation errors and resolution
Fourier/DCT coefficients	Frequency content	Alignment and which frequencies are kept
Wavelet coefficients	Location and scale of variation	Basis choice and boundary handling

For a quantized image I and an offset δ, a gray-level co-occurrence matrix counts pairs $(I(p), I(p+\delta))$ over pixel positions p; normalized to probabilities $P(i,j)$, Haralick's texture features include contrast $\sum_{i,j}(i-j)^2 P(i,j)$ and energy $\sum_{i,j} P(i,j)^2$ [20]. Always state the offset and the number of gray levels, because the numbers depend on both. For a segmented object with area A and perimeter P, compactness $4\pi A/P^2$ equals 1 for an ideal continuous circle and is smaller for irregular shapes; pixel-based perimeter estimates distort it for small objects. Dalal and Triggs' HOG descriptor, built from normalized histograms of gradient orientation in small cells, was introduced for pedestrian detection and became one of the most widely used hand-designed image features [21].

For audio and other 1-D signals, features are computed on short windows in which the signal is approximately stationary: spectral energy in frequency bands, zero-crossing rate, and so on. The window length trades time resolution against frequency resolution.

Code: one classifier, three feature sets

This example compares fixed feature strategies with the same classifier on the built-in digits data.

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.linear_model import LogisticRegression

X, y = load_digits(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=9)
transforms = {
    "all features": "passthrough",
    "24 PCA coordinates": PCA(n_components=24, random_state=9),
    "24 selected coordinates": SelectKBest(f_classif, k=24),
}
for name, transform in transforms.items():
    model = make_pipeline(
        StandardScaler(), transform, LogisticRegression(max_iter=1500))
    model.fit(Xtr, ytr)
    print(name, "test accuracy:", round(model.score(Xte, yte), 4))
```

All 64 pixels give 0.969 test accuracy, 24 principal components 0.956, and the 24 pixels with the best ANOVA F scores 0.953. Here a reduction to about a third of the dimensions costs little accuracy. Some border pixels of the digits are constant, so the F test warns about constant features; the warning reflects zero within-class variance, not leakage, and those pixels simply receive no useful score.

Modern view

Feature engineering by hand has largely given way to learned representations, but the ideas of this chapter are everywhere in modern pipelines: deep features are selected, projected, whitened, visualized and evaluated with the same tools. Here is how the survey literature maps the field.

Feature selection. Guyon and Elisseeff's introduction [3] set the vocabulary (filters, wrappers, embedded methods), discussed feature construction, ranking and validation, and stressed that a feature can be useless alone and useful in combination. Chandrashekar and Sahin [23] provide a compact survey organized along the same filter/wrapper/embedded lines. Li et al. [22]

revisit selection "from a data perspective": they group algorithms for conventional data into similarity-based, information-theoretic, sparse-learning-based and statistical methods, add methods for structured, heterogeneous and streaming data, and release an open-source repository for benchmarking. The leakage warning of Ambroise and McLachlan [7] remains the single most important practical lesson.

PCA and its descendants. Jolliffe and Cadima's review [10] explains PCA as an adaptive, data-defined technique and surveys its variants tailored to different data types. In modern deep learning, PCA survives as a diagnostic (how many directions carry most variance in a layer's activations) and as a preprocessing and initialization step, for example the PCA initialization recommended for t-SNE [26].

Dimensionality reduction for visualization. Espadoto et al. [24] ran a large quantitative survey of projection techniques: they characterized data sets, techniques and projection quality with several quantitative metrics (such as neighborhood and distance preservation), measured how the results depend on the technique and the data, and turned the observations into guidance for choosing a projection in a given context, with all code and results released as an open benchmark. Wang, Huang, Rudin and Shaposhnik [25] analyzed why t-SNE, UMAP and TriMAP behave as they do, focused on the trade-off between preserving local and global structure, and proposed PaCMAP to balance the two.

Pitfalls of 2-D embeddings. Single-cell biology, where t-SNE and UMAP are routine, produced the sharpest critiques. Kobak and Berens [26] gave a protocol for more faithful t-SNE maps: PCA initialization, a high learning rate and multi-scale similarities. Kobak and Linderman [27] showed that the better global structure often credited to UMAP comes largely from its initialization; with the same informative initialization, t-SNE preserves global structure similarly. Chari and Pachter [28] went further, arguing that extreme reduction to two dimensions inevitably distorts high-dimensional data and that such pictures should not be used for quantitative conclusions.

Variance versus usefulness, revisited. Figure 2's lesson reappears at scale in self-supervised learning. Balestriero and LeCun [29] argue that training a representation to reconstruct its input emphasizes high-variance components of the data, which are often less informative for perception tasks than low-variance details; reconstruction-trained features therefore need extra steps (such as fine-tuning) to be useful for classification. This is PCA's limitation, in a deep-learning setting.

Open problems. Selection with many correlated features and few samples remains unstable: different subsets can perform equally well, which complicates interpretation. Evaluating embeddings still lacks agreed metrics that capture both local and global fidelity [24], [25]. And for learned features, deciding which directions carry task-relevant rather than merely high-variance information is an active research question [29].

Key takeaways

- Feature selection keeps original measurements; extraction builds new coordinates; learning fits the extraction rule from data. All three decide what the classifier can see.
- Filters are fast but miss interactions and redundancy; wrappers see them but cost more and can

overfit validation data; embedded methods (lasso, trees) select during training.

- Any step that uses labels (selection, LDA, choosing the number of features) must be refitted inside each cross-validation fold. Selecting on the full data can report 98% accuracy on pure noise.
- Fisher ratio, $J = \mathrm{tr}(S_W^{-1} S_B)$, Bhattacharyya distance and mutual information score features; subsets must be scored jointly because of redundancy.
- PCA keeps maximum variance (eigenvectors of the covariance, computed by SVD); whitening equalizes variances. Variance is not class information.
- Fisher LDA solves $S_B v = \lambda S_W v$ and gives at most $K - 1$ directions. Kernel PCA, ICA and NMF extend linear extraction to nonlinear, independent or non-negative structure.
- Isomap, LLE, t-SNE and UMAP are mainly visualization tools: their pictures depend on settings, and gaps between clusters are not proof of separate populations.

Exercises

1. **Same histogram, different image.** Construct two 4×4 binary images with identical intensity histograms but different texture. Which descriptor from the table distinguishes them?

   Hint

   Use eight 1-pixels arranged as a checkerboard versus the left half filled. The histograms are identical. A co-occurrence matrix with horizontal offset $(0, 1)$ distinguishes them: the checkerboard has only unequal neighbor pairs, the half-filled image mostly equal ones.

2. **The $K - 1$ limit.** Prove that the between-class scatter S_B of K classes has rank at most $K - 1$. What does this imply for LDA on a two-class problem?

   Hint

   $S_B = \sum_c P_c d_c d_c^\top$ with $d_c = \mu_c - \mu$, and $\sum_c P_c d_c = 0$, so the K vectors d_c span at most $K - 1$ dimensions. With two classes, LDA gives a single direction, $v \propto S_W^{-1}(\mu_1 - \mu_2)$.

3. **When 99% of the variance is not enough.** Describe a realistic imaging setup in which keeping 99% of the PCA variance discards the class signal. How would you detect the problem, and what would you do?

   Hint

   Large independent lighting or pose changes plus a small, consistent defect mark (Figure 2 in disguise). Detect it by comparing cross-validated accuracy with and without PCA, or by checking class separation (Fisher ratio) per component. Use a supervised projection, normalize the nuisance (for example illumination) first, or skip PCA.

4. **The XOR trap.** Generate x_1, x_2 uniform on $[-1, 1]$ and eight irrelevant noise features, with $y = \mathbf{1}[x_1 > 0] \oplus \mathbf{1}[x_2 > 0]$. Rank all ten features with `f_classif` and with `mutual_info_classif`. Then run sequential forward selection with a decision tree. Which methods find x_1 and x_2?

   Hint

   Both univariate filters give x_1 and x_2 scores near zero, like the noise features, because each alone is independent of y. A wrapper that adds features one at a time can also struggle at the first step, for the same reason; a tree-based wrapper that evaluates pairs, or backward elimination starting from all features, finds them.

5. **Leakage, quantified.** In the selection-bias code, vary the number of noise features (100, 1,000, 5,000) and the number of samples (30, 60, 200). How does the "wrong" accuracy change? Explain the trend.

   Hint

   The optimism grows with the number of candidate features and shrinks with the number of samples: with more noise features, more of them correlate with the labels by chance in a small sample, and selecting the best 20 picks exactly those.

6. **Reading a t-SNE map.** Run t-SNE on the digits data with perplexity 5, 30 and 100, and with two random seeds each. Which features of the plots are stable (which digits are neighbors?) and which change (cluster sizes, gaps, positions)? What would you be willing to claim from one such plot?

   Hint

   Membership of local neighborhoods is quite stable; absolute positions, cluster areas and between-cluster distances are not. A defensible claim is "these points are locally similar in the input space," not "these two groups are far apart" or "this group is twice as diverse."

References

1. S. Theodoridis and K. Koutroumbas, *Pattern Recognition*, 4th ed., Academic Press, 2009, Chapters 5–7. publisher page
2. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006, Sections 4.1.4–4.1.6 and 12.1–12.4. official book page and PDF
3. I. Guyon and A. Elisseeff, "An Introduction to Variable and Feature Selection," *Journal of Machine Learning Research*, vol. 3, pp. 1157–1182, 2003. JMLR
4. R. Kohavi and G. H. John, "Wrappers for Feature Subset Selection," *Artificial Intelligence*, vol. 97, pp. 273–324, 1997. doi:10.1016/S0004-3702(97)00043-X
5. R. Tibshirani, "Regression Shrinkage and Selection via the Lasso," *Journal of the Royal Statistical Society, Series B*, vol. 58, no. 1, pp. 267–288, 1996. doi:10.1111/j.2517-6161.1996.tb02080.x

6. H. Peng, F. Long, and C. Ding, "Feature Selection Based on Mutual Information: Criteria of Max-Dependency, Max-Relevance, and Min-Redundancy," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 27, no. 8, pp. 1226–1238, 2005. doi:10.1109/TPAMI.2005.159
7. C. Ambroise and G. J. McLachlan, "Selection Bias in Gene Extraction on the Basis of Microarray Gene-Expression Data," *Proceedings of the National Academy of Sciences*, vol. 99, no. 10, pp. 6562–6566, 2002. doi:10.1073/pnas.102102699
8. K. Pearson, "On Lines and Planes of Closest Fit to Systems of Points in Space," *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, vol. 2, no. 11, pp. 559–572, 1901. doi:10.1080/14786440109462720
9. H. Hotelling, "Analysis of a Complex of Statistical Variables into Principal Components," *Journal of Educational Psychology*, vol. 24, no. 6, pp. 417–441, 1933. doi:10.1037/h0071325
10. I. T. Jolliffe and J. Cadima, "Principal Component Analysis: A Review and Recent Developments," *Philosophical Transactions of the Royal Society A*, vol. 374, 20150202, 2016. doi:10.1098/rsta.2015.0202
11. R. A. Fisher, "The Use of Multiple Measurements in Taxonomic Problems," *Annals of Eugenics*, vol. 7, no. 2, pp. 179–188, 1936. doi:10.1111/j.1469-1809.1936.tb02137.x
12. B. Schölkopf, A. Smola, and K.-R. Müller, "Nonlinear Component Analysis as a Kernel Eigenvalue Problem," *Neural Computation*, vol. 10, no. 5, pp. 1299–1319, 1998. doi:10.1162/089976698300017467
13. A. Hyvärinen and E. Oja, "Independent Component Analysis: Algorithms and Applications," *Neural Networks*, vol. 13, no. 4–5, pp. 411–430, 2000. doi:10.1016/S0893-6080(00)00026-5
14. D. D. Lee and H. S. Seung, "Learning the Parts of Objects by Non-Negative Matrix Factorization," *Nature*, vol. 401, pp. 788–791, 1999. doi:10.1038/44565
15. J. B. Tenenbaum, V. de Silva, and J. C. Langford, "A Global Geometric Framework for Nonlinear Dimensionality Reduction," *Science*, vol. 290, no. 5500, pp. 2319–2323, 2000. doi:10.1126/science.290.5500.2319
16. S. T. Roweis and L. K. Saul, "Nonlinear Dimensionality Reduction by Locally Linear Embedding," *Science*, vol. 290, no. 5500, pp. 2323–2326, 2000. doi:10.1126/science.290.5500.2323
17. L. van der Maaten and G. Hinton, "Visualizing Data using t-SNE," *Journal of Machine Learning Research*, vol. 9, pp. 2579–2605, 2008. JMLR
18. L. McInnes, J. Healy, and J. Melville, "UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction," arXiv:1802.03426, 2018. arXiv
19. M. Wattenberg, F. Viégas, and I. Johnson, "How to Use t-SNE Effectively," *Distill*, 2016. doi:10.23915/distill.00002
20. R. M. Haralick, K. Shanmugam, and I. Dinstein, "Textural Features for Image Classification," *IEEE Transactions on Systems, Man, and Cybernetics*, vol. SMC-3, no. 6, pp. 610–621, 1973. doi:10.1109/TSMC.1973.4309314
21. N. Dalal and B. Triggs, "Histograms of Oriented Gradients for Human Detection," *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, vol. 1, pp. 886–893, 2005. doi:10.1109/CVPR.2005.177
22. J. Li, K. Cheng, S. Wang, F. Morstatter, R. P. Trevino, J. Tang, and H. Liu, "Feature Selection: A Data Perspective," *ACM Computing Surveys*, vol. 50, no. 6, 2018. doi:10.1145/3136625
23. G. Chandrashekar and F. Sahin, "A Survey on Feature Selection Methods," *Computers & Electrical Engineering*, vol. 40, no. 1, pp. 16–28, 2014. doi:10.1016/j.compeleceng.2013.11.024

24. M. Espadoto, R. M. Martins, A. Kerren, N. S. T. Hirata, and A. C. Telea, “Toward a Quantitative Survey of Dimension Reduction Techniques,” *IEEE Transactions on Visualization and Computer Graphics*, vol. 27, no. 3, pp. 2153–2173, 2021. doi:10.1109/TVCG.2019.2944182
25. Y. Wang, H. Huang, C. Rudin, and Y. Shaposhnik, “Understanding How Dimension Reduction Tools Work: An Empirical Approach to Deciphering t-SNE, UMAP, TriMAP, and PaCMAP for Data Visualization,” *Journal of Machine Learning Research*, vol. 22, 2021. arXiv:2012.04456
26. D. Kobak and P. Berens, “The Art of Using t-SNE for Single-Cell Transcriptomics,” *Nature Communications*, vol. 10, 2019. doi:10.1038/s41467-019-13056-x
27. D. Kobak and G. C. Linderman, “Initialization Is Critical for Preserving Global Data Structure in Both t-SNE and UMAP,” *Nature Biotechnology*, vol. 39, pp. 156–157, 2021. doi:10.1038/s41587-020-00809-z
28. T. Chari and L. Pachter, “The Specious Art of Single-Cell Genomics,” *PLOS Computational Biology*, vol. 19, e1011288, 2023. doi:10.1371/journal.pcbi.1011288
29. R. Balestriero and Y. LeCun, “Learning by Reconstruction Produces Uninformative Features for Perception,” arXiv:2402.11337, 2024. arXiv

PR 06 • Template Matching and Structured Recognition

Match patterns with correlation, normalized cross-correlation and dynamic time warping, describe structure with strings, grammars and graphs, and model context with graphical models, HMMs (forward algorithm, Viterbi) and CRFs.

Online edition: https://blog.shyandram.dev/en/learn/pattern-recognition/pr-06-templates-structured-models/

What you'll learn

- How template matching works with sum of squared differences, cross-correlation and normalized cross-correlation (NCC), and which nuisance changes each one tolerates.
- How dynamic time warping (DTW) aligns sequences that unfold at different speeds, and how to constrain it.
- Syntactic and structural pattern recognition: describing patterns as strings, grammars and graphs, and comparing them with edit distance and graph matching.
- How directed and undirected graphical models factorize a joint distribution, and why that makes inference efficient.
- Hidden Markov models: the forward algorithm, forward–backward smoothing, Viterbi decoding and Baum–Welch learning, with tested code.
- Conditional random fields as the discriminative counterpart of HMMs, and where all of these ideas live on in 2026.

The big picture

The classifiers of PR 03 and PR 04 look at one fixed-length feature vector at a time and decide its class alone. Many recognition problems are not like that:

- To find a part in an image, you slide a reference picture over the image and look for the best match (**templates**).
- A spoken word or a signature unfolds in time, and two people produce it at different speeds (**alignment**).
- A character, a molecule or a road network is best described by its parts and how they connect (**structure**).
- An isolated image patch might be road or roof; a blurry handwritten letter might be "c" or "e". Neighboring regions or letters resolve the ambiguity (**context**).

Explain like I'm five

Reading messy handwriting, you don't stare at each letter alone. You compare its shape with letters you know (templates), you allow some letters to be stretched or squashed (alignment), you know

how strokes join to make a letter (structure), and you use the neighboring letters to guess the word (context). This chapter turns each of these habits into an algorithm.

A common thread runs through the whole chapter: **dynamic programming**. DTW, edit distance, the HMM forward algorithm, Viterbi decoding and CRF inference are all the same trick. Break a huge search over all alignments or label sequences into small, reusable subproblems, and the cost drops from exponential to polynomial.

One caution belongs at the start. Context helps only when its assumptions match reality. A strong "neighbors agree" rule can erase small objects or brief events, so more context is not automatically better. The textbook background is Theodoridis and Koutroumbas, Chapters 8 and 9 [1], and Bishop, Sections 8.1–8.4 and 13.1–13.3 [2].

Template matching

In plain words. Keep a small reference image, the template. Slide it over the scene, and at every position measure how similar the patch underneath is. The best-scoring position is the match. The interesting question is *how* to measure similarity so that irrelevant changes, such as a brighter or dimmer exposure, do not fool you.

Three similarity measures

Write the template as a vector t of its M pixel values and the image patch under it as x, also of length M.

Sum of squared differences (SSD). $\mathrm{SSD}(x,t) = \|x-t\|^2 = \sum_i (x_i - t_i)^2$. Small is good. It is the natural distance, but any brightness offset or contrast change in the scene increases it.

Cross-correlation. Expanding the square, $\|x-t\|^2 = \|x\|^2 - 2x^\top t + \|t\|^2$. The template energy $\|t\|^2$ is constant, so if the patch energy $\|x\|^2$ were roughly constant too, minimizing SSD would be the same as maximizing the correlation $x^\top t$. In practice $\|x\|^2$ varies a lot, and plain correlation is drawn to bright regions.

Normalized cross-correlation (NCC). Subtract the means and divide by the norms:

$$\mathrm{NCC}(x,t) = \frac{(x-\bar{x}\mathbf{1})^\top (t-\bar{t}\mathbf{1})}{\|x-\bar{x}\mathbf{1}\|\,\|t-\bar{t}\mathbf{1}\|},$$

where $\bar{x}$ and $\bar{t}$ are the mean pixel values of the patch and the template and $\mathbf{1}$ is the all-ones vector. NCC is the cosine of the angle between the mean-subtracted vectors, so it lies in $[-1,1]$, and it equals 1 exactly when $x = a\,t + b\mathbf{1}$ for some $a > 0$ and any b. It is therefore invariant to positive changes of contrast (a) and to brightness offsets (b). It is undefined for a constant template or patch (zero denominator); implementations return a convention such as 0 or NaN. NCC is the "matching by correlation" of IP 13.

Computing it efficiently

Evaluating NCC at every position of an $N \times N$ image with an $m \times m$ template costs on the order of $N^2 m^2$ operations. The numerator is a correlation, which can be computed for all positions at once with the FFT (the correlation theorem, IP 04). The local means and energies in the denominator vary from position to position; Lewis showed how to get them cheaply from running-sum tables (integral images) of the image and of its square, which makes NCC nearly as fast as plain correlation [3].

```
import cv2
import numpy as np
from skimage import data

img = data.camera().astype(np.float32)          # 512 x 512 grayscale
r0, c0, h, w = 300, 200, 40, 40
template = img[r0:r0 + h, c0:c0 + w].copy()

def ncc_at(window, t):
    a, b = window - window.mean(), t - t.mean()
    return float((a * b).sum() / (np.linalg.norm(a) * np.linalg.norm(b)))

# The scene is captured again under different exposure: darker and lower contrast.
scene = 0.5 * img + 20.0
ncc = cv2.matchTemplate(scene, template, cv2.TM_CCOEFF_NORMED)
ssd = cv2.matchTemplate(scene, template, cv2.TM_SQDIFF)
r, c = np.unravel_index(ncc.argmax(), ncc.shape)
print("NCC best (row, col):", (int(r), int(c)), "score", round(float(ncc.max()), 4))
r, c = np.unravel_index(ssd.argmin(), ssd.shape)
print("SSD best (row, col):", (int(r), int(c)))
print("hand-computed NCC at true location:",
      round(ncc_at(scene[r0:r0 + h, c0:c0 + w], template), 4))
```

NCC finds the template at its true position (300, 200) with a score of 1.0, and the hand-computed value agrees with OpenCV's `TM_CCOEFF_NORMED`. SSD, fooled by the exposure change, picks (130, 413), a patch of sky whose darkened intensities (mean about 129) are closer to the template's original values (mean about 143) than the darkened true patch is (mean about 92).

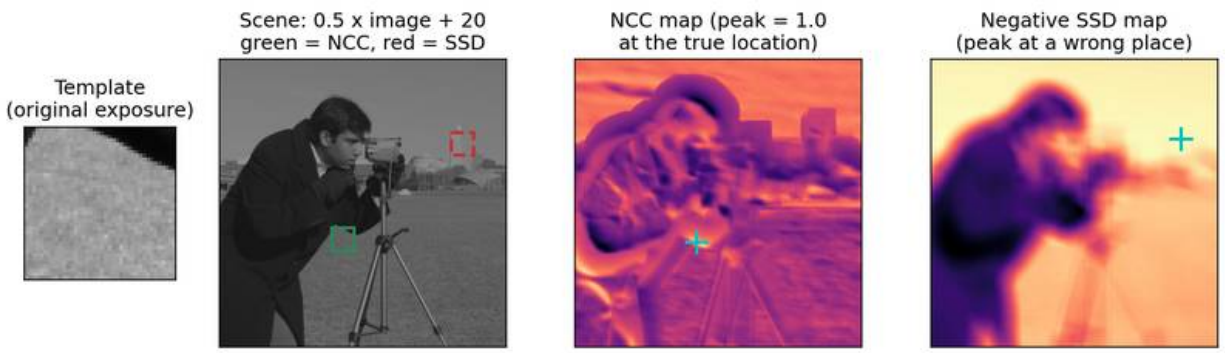


Figure 1 — Template matching under an exposure change. NCC is invariant to the gain 0.5 and offset 20 and finds the true location; SSD does not. Generated by scripts/figures/pr_06_templates_structured_models_extra.py.

What templates cannot handle

NCC removes brightness and contrast, nothing else. Rotation, scale change, perspective, occlusion and non-rigid deformation all lower the score. The classical remedies are to store a family of templates (rotated and scaled copies), to search over an image pyramid, or to match small parts and then check their geometric arrangement, which leads to the structural methods below. Template matching remains the right tool when appearance is tightly controlled, as in industrial inspection, and it is still a core step in tracking and image registration [4].

Dynamic time warping

In plain words. Two people say the same word, one slowly, one quickly. Comparing the recordings sample by sample fails because the sounds do not line up in time. DTW stretches and compresses the time axis of one sequence to line it up with the other as well as possible, then measures the remaining difference.

The recursion

Let $x = (x_1, \dots, x_n)$ and $t = (t_1, \dots, t_m)$ be two sequences, and $d(x_i, t_j)$ a local cost such as $|x_i - t_j|$ or $\|x_i - t_j\|^2$ for vector-valued samples. A **warping path** is a sequence of index pairs $(i_1, j_1), \dots, (i_L, j_L)$ that starts at $(1,1)$, ends at (n,m), and moves by one of the steps $(1,0)$, $(0,1)$ or $(1,1)$ at a time, so that it never goes backward in either sequence. DTW is the smallest total cost over all warping paths. It is computed by filling a table:

$$D(i,j) = d(x_i, t_j) + \min\{D(i-1,j),\ D(i,j-1),\ D(i-1,j-1)\},$$

with $D(0,0) = 0$ and $D(i,0) = D(0,j) = \infty$ for $i, j \geq 1$. Then $\mathrm{DTW}(x,t) = D(n,m)$. Each cell needs only its three neighbors, so the cost is $O(nm)$ time. Tracing back which neighbor gave the minimum recovers the optimal alignment. This is the algorithm Sakoe and Chiba optimized for spoken-word recognition [5].

Constraints and caveats

An unconstrained path can match a single sample of one sequence to a long stretch of the other ("pathological warping"), which lets unrelated sequences look similar. Two standard fixes are:

- a **global constraint**, such as the Sakoe–Chiba band $|i - j| \leq w$, which forbids cells far from the diagonal and also reduces the cost to $O(nw)$ [5];
- **slope constraints** or step weights that penalize long horizontal or vertical runs [5].

The band width w must allow the real timing differences. In the code and in Figure 2, a band of 3 samples is narrower than the true shift of about 6 samples, so the banded DTW cannot fully align the two sine waves.

Two further facts matter in practice. DTW is not a metric: it can violate the triangle inequality, so data structures that assume a metric need care. And sequences are usually **z-normalized** (zero mean, unit variance) before comparison, so that DTW compares shapes rather than offsets and scales.

```
import numpy as np

def dtw(x, t, band=None):
    """DTW distance between 1-D sequences x (length n) and t (length m)."""
    n, m = len(x), len(t)
    D = np.full((n + 1, m + 1), np.inf)
    D[0, 0] = 0.0
    for i in range(1, n + 1):
        lo, hi = 1, m
        if band is not None:                    # Sakoe-Chiba band
            lo, hi = max(1, i - band), min(m, i + band)
        for j in range(lo, hi + 1):
            cost = abs(x[i - 1] - t[j - 1])
            D[i, j] = cost + min(D[i - 1, j], D[i, j - 1], D[i - 1, j - 1])
    return D[n, m]

s = np.linspace(0, 2 * np.pi, 60)
a = np.sin(s)
b = np.sin(s - 0.6)                             # same shape, shifted in time
c = 0.25 * np.sign(np.sin(3 * s))               # a different shape
print("lock-step a-b:", round(np.abs(a - b).sum(), 2),
      " a-c:", round(np.abs(a - c).sum(), 2))
print("DTW       a-b:", round(dtw(a, b), 2),
      " a-c:", round(dtw(a, c), 2))
print("DTW a-b with band 3:", round(dtw(a, b, band=3), 2))
```

Both comparisons use the same local cost $|x_i - t_j|$. Sample by sample ("lock-step"), the shifted copy b costs 22.76 and the different shape c costs 35.18. DTW reduces the shifted copy to 5.05 while the different shape stays at 34.57: warping absorbs timing differences, not shape differences. With a band of 3 samples, the cost of the shifted copy rises to 12.76 because the band is too narrow for a shift of about 6 samples.

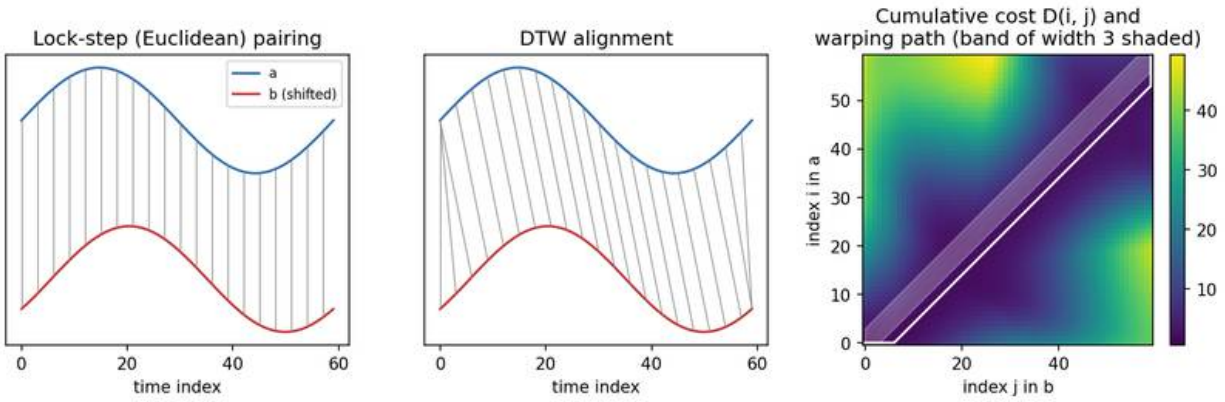


Figure 2 — Dynamic time warping. Lock-step comparison pairs equal time indices; DTW pairs corresponding parts of the two curves. In the cost matrix, the optimal path runs about 6 samples off the diagonal, outside a band of width 3.

A 1-nearest-neighbor classifier with DTW distance was for many years a strong, hard-to-beat baseline for time-series classification [16]. Searching for the nearest neighbor under DTW in very long series became practical once cheap lower bounds and early abandoning of hopeless candidates were combined [6].

Syntactic and structural pattern recognition

In plain words. Some patterns are best described not by a list of numbers but by their parts and how the parts fit together, the way a sentence is words arranged by grammar. Structural pattern recognition describes a pattern as a string, tree or graph of primitive parts, and compares or parses those structures.

Strings and grammars

Choose a small alphabet of **primitives**, for example the four directions of a step along an object's boundary. Then a shape becomes a string. A 4-direction **chain code** writes 0 for a step right, 1 up, 2 left and 3 down while tracing the boundary (see IP 12). Figure 3 shows three outlines and their codes.

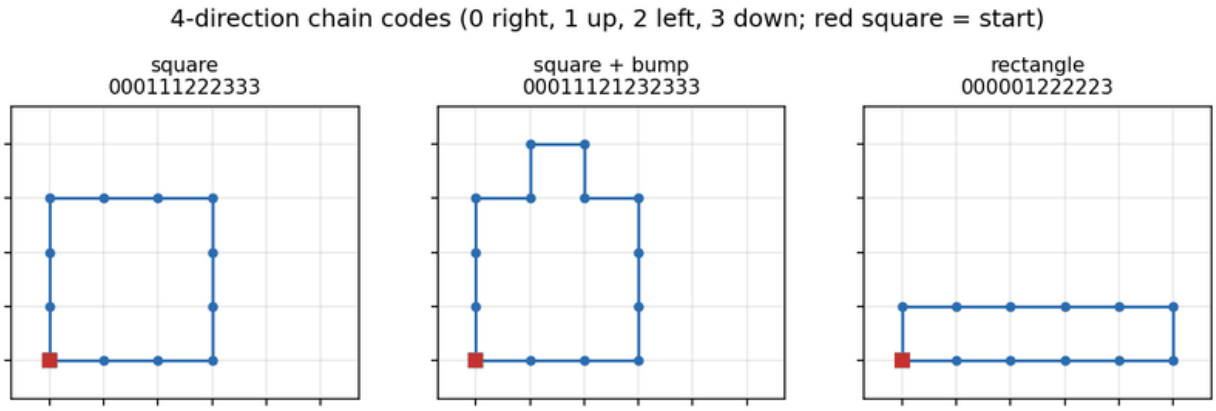


Figure 3 — Shapes as strings. Each outline is traced counter-clockwise from the red start point, writing one symbol per boundary step.

In **syntactic** pattern recognition each class is described by a formal grammar: a set of rewrite rules that generates exactly the valid strings of that class. Recognition is **parsing**: a string belongs to a class if that class's grammar can generate it. For example, the rule set $S \to 0S2 \mid 1$ generates strings $0^k 1\, 2^k$, a "class" in which every rightward run is matched by an equally long leftward run. Grammars can express recursive structure (nested parts, repeated sub-shapes) that fixed-length feature vectors cannot. Their weaknesses are that real data are noisy, so a single wrong primitive makes a string ungrammatical, and that grammars are hard to learn from examples. Stochastic grammars, which attach probabilities to rules, address the first problem; a hidden Markov model, below, is the simplest stochastic grammar of all [1].

Edit distance

A more forgiving approach compares strings by the cheapest way to turn one into the other. The **edit distance** (Levenshtein distance) counts the minimum number of insertions, deletions and substitutions. Wagner and Fischer gave the dynamic program [7]:

$$E(i,j) = \min\{E(i-1,j)+1,\; E(i,j-1)+1,\; E(i-1,j-1) + \mathbf{1}[s_i \neq u_j]\},$$

with $E(i,0) = i$ and $E(0,j) = j$, for strings s (length n) and u (length m); the answer is $E(n,m)$. The structure is exactly that of DTW, with the operations "insert", "delete" and "substitute" instead of "stretch". Costs can be weighted, for example to make substituting neighboring directions cheaper than opposite ones.

```
import numpy as np

def edit_distance(s, t):
    """Levenshtein distance: unit-cost insertions, deletions, substitutions."""
    n, m = len(s), len(t)
    D = np.zeros((n + 1, m + 1), dtype=int)
    D[:, 0] = np.arange(n + 1)
    D[0, :] = np.arange(m + 1)
```

```
    for i in range(1, n + 1):
        for j in range(1, m + 1):
            sub = 0 if s[i - 1] == t[j - 1] else 1
            D[i, j] = min(D[i - 1, j] + 1,          # delete s[i-1]
                          D[i, j - 1] + 1,          # insert t[j-1]
                          D[i - 1, j - 1] + sub)    # match or substitute
    return D[n, m]

# 4-direction chain codes (0=right, 1=up, 2=left, 3=down) of three outlines
square = "000111222333"           # 3 x 3 square, traced counter-clockwise
square_bump = "00011121232333"   # the same square with a one-pixel bump on top
rectangle = "000001222223"       # a flat 5 x 1 rectangle
for name, code in [("square + bump", square_bump), ("rectangle", rectangle)]:
    print(f"edit distance(square, {name}) = {edit_distance(square, code)}")
```

The bumped square is 2 edits from the square (insert one "up" and one "down"); the flat rectangle is 4 edits away. A nearest-neighbor classifier over strings with this distance is the structural analogue of k-NN. Chain codes depend on the starting point and on rotation; normalizing them (for example by choosing the starting point that gives the smallest string, and by using differences between successive directions) is part of the representation design.

Graphs and graph matching

When parts connect in more than a chain, a pattern becomes an **attributed graph**: nodes are parts (strokes, regions, atoms) with attributes such as size or color, and edges are relations (touching, above, bonded). Recognition then means **graph matching**: finding a correspondence between the nodes of two graphs. Exact matching (graph or subgraph isomorphism) tolerates no differences and is computationally hard in general; error-tolerant matching, such as **graph edit distance**, generalizes string edit distance with node and edge insertions, deletions and substitutions, at an even higher computational price. Conte, Foggia, Sansone and Vento surveyed three decades of graph matching in pattern recognition, from exact algorithms to inexact ones and their applications [8], and Foggia, Percannella and Vento updated the picture with a decade of graph matching and graph learning methods [9].

Graphical models describe dependence

In plain words. A graphical model is a drawing of which variables directly influence which. The drawing tells you how a huge joint probability breaks into small local pieces, and small pieces are what make computation possible.

A **directed** acyclic graph (a Bayesian network) factorizes the joint distribution of variables $z_1, \dots, z_M$ as

$$p(z_1, \dots, z_M) = \prod_{i=1}^{M} p(z_i \mid \mathrm{pa}(z_i)),$$

where $\mathrm{pa}(z_i)$ are the parents of z_i in the graph [2]. An **undirected** graph (a Markov random field) uses nonnegative potential functions ψ_C on its cliques C (fully connected subsets of nodes):

$$p(z) = \frac{1}{Z} \prod_C \psi_C(z_C), \qquad Z = \sum_z \prod_C \psi_C(z_C).$$

A potential is a compatibility score, not necessarily a probability. The **partition function** Z normalizes the product; it sums over all joint configurations and is often the hardest quantity to compute.

Conditional independence is read from the graph by its rules (d-separation for directed graphs, graph separation for undirected ones), not from the mere absence of an arrow. A classic surprise: two independent causes become dependent once you condition on their common effect ("explaining away") [2]. **Factor graphs** draw variables and factors as two kinds of nodes and make local message passing explicit; on trees, message passing (the sum-product algorithm) computes all marginals exactly, and the HMM algorithms below are its special case on a chain [2].

For image labeling, a common undirected model is an **energy** with unary data terms and pairwise smoothness terms:

$$E(z; x) = \sum_i U_i(z_i; x) + \lambda \sum_{(i,j)\in\mathcal{E}} \mathbf{1}[z_i \neq z_j],$$

where z_i is the label of pixel i, U_i is the cost of that label given the image x (for example the negative log-probability from a pixel classifier), $\mathcal{E}$ is the set of neighboring pixel pairs, and $\lambda \geq 0$ is the price of a label change between neighbors (a Potts model). The distribution is $p(z \mid x) \propto \exp(-E(z; x))$. It encourages neighboring labels to agree, which cleans up noise but can also erase thin structures if λ is too large. Exact minimization is hard on general grids, and PR 08 covers approximate inference.

Hidden Markov models

In plain words. A system moves between a few hidden states (silence, vowel, consonant; walking, running) and at each time step emits an observation that depends only on the current state. You see the observations, never the states. An HMM lets you compute how likely an observation sequence is, which hidden states probably produced it, and how to learn the model's probabilities from data.

The model

An HMM with K hidden states has initial probabilities $\pi_j = p(z_1 = j)$, a **transition matrix** $A_{ij} = p(z_t = j \mid z_{t-1} = i)$, and **emission** probabilities (or densities) $B_j(x_t) = p(x_t \mid z_t = j)$. The joint probability of states $z_{1:T} = (z_1, \ldots, z_T)$ and observations $x_{1:T}$ is

$$p(z_{1:T}, x_{1:T}) = \pi_{z_1} B_{z_1}(x_1) \prod_{t=2}^{T} A_{z_{t-1}, z_t} B_{z_t}(x_t).$$

Two assumptions are built in: the hidden state is first-order Markov (the future depends on the past only through the present state), and each observation depends only on the current state. The observations themselves are *not* Markov: integrating out the hidden states creates long-range dependence among them. Rabiner's tutorial, still the standard introduction, organizes HMM computation around three problems [10]: **evaluation** (compute $p(x_{1:T})$), **decoding** (find the hidden states) and **learning** (estimate π, A, B). Figure 4 contrasts the HMM's directed graph with the CRF discussed later.

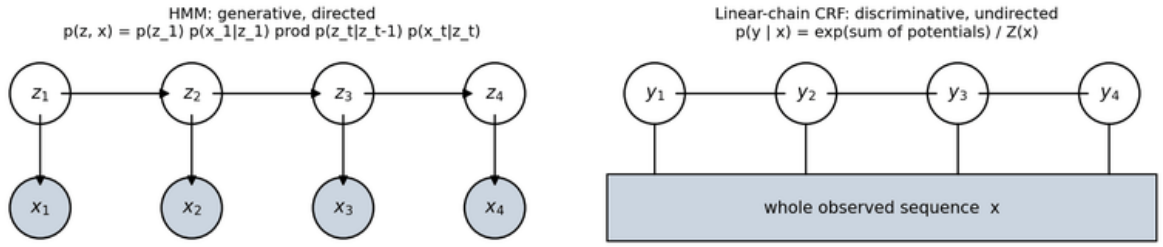


Figure 4 — HMM versus linear-chain CRF. The HMM is a directed generative model of states and observations; the CRF is an undirected model of labels conditioned on the whole observation sequence.

Evaluation: the forward algorithm

Summing the joint over all K^T state sequences is hopeless for realistic T. Define the **forward variable** $\alpha_t(j) = p(x_{1:t}, z_t = j)$. Because z_t summarizes everything the past can tell about the future, it satisfies

$$\alpha_1(j) = \pi_j B_j(x_1), \qquad \alpha_t(j) = B_j(x_t) \sum_{i=1}^{K} \alpha_{t-1}(i) A_{ij},$$

and $p(x_{1:T}) = \sum_j \alpha_T(j)$. The cost is $O(TK^2)$ instead of $O(TK^T)$. Since α_t is a product of many probabilities, it underflows for long sequences. The standard fix is to normalize at each step: store $c_t = \sum_j \tilde{\alpha}_t(j)$ and $\hat{\alpha}_t = \tilde{\alpha}_t / c_t$, where $\tilde{\alpha}_t$ is the unnormalized update. Then $\hat{\alpha}_t(j) = p(z_t = j \mid x_{1:t})$ is exactly the **filtered** state probability, $c_t = p(x_t \mid x_{1:t-1})$, and

$$\log p(x_{1:T}) = \sum_{t=1}^{T} \log c_t.$$

Evaluation gives a classifier immediately: train one HMM per class (one per spoken word, say) and assign a new sequence to the class whose HMM gives it the highest likelihood, times the prior.

Smoothing: the forward–backward algorithm

Filtering computes $p(z_t \mid x_{1:t})$ from past and present only; it is what an online system can know at time t. **Smoothing** computes $p(z_t \mid x_{1:T})$, using later observations as well. A backward recursion

$\beta_t(i) = p(x_{t+1:T} \mid z_t = i)$,

$$\beta_T(i) = 1, \qquad \beta_t(i) = \sum_{j=1}^{K} A_{ij}\, B_j(x_{t+1})\, \beta_{t+1}(j),$$

combines with the forward variable to give $\gamma_t(j) = p(z_t = j \mid x_{1:T}) \propto \alpha_t(j)\beta_t(j)$ [2], [10]. With scaling, divide each backward step by c_{t+1}, and then $\gamma_t = \hat{\alpha}_t \odot \hat{\beta}_t$ (elementwise product) exactly.

Decoding: the Viterbi algorithm

Often we want the single most probable *sequence* of states, $\arg\max_{z_{1:T}} p(z_{1:T} \mid x_{1:T})$. Viterbi's algorithm [11] replaces the sum in the forward recursion by a maximum. In log space,

$$\delta_1(j) = \log \pi_j + \log B_j(x_1), \qquad \delta_t(j) = \log B_j(x_t) + \max_i \{\delta_{t-1}(i) + \log A_{ij}\},$$

where $\delta_t(j)$ is the log-probability of the best path that ends in state j at time t. Store the maximizing predecessor $\psi_t(j) = \arg\max_i\{\cdots\}$, pick the best final state, and trace the pointers backward. The cost is again $O(TK^2)$.

The most probable path is *not* the same as choosing, at each time, the state with the highest smoothed probability $\gamma_t(j)$. The per-step choice maximizes the expected number of correct states but can string together states that the model says cannot follow each other. The code shows this on a model whose states must cycle $0 \to 1 \to 2 \to 0$, so the transition $0 \to 2$ has probability zero.

```
import itertools
import numpy as np

pi = np.array([1 / 3, 1 / 3, 1 / 3])            # initial state probabilities
A = np.array([[0.5, 0.5, 0.0],                  # cyclic model: 0 -> 1 -> 2 -> 0
              [0.0, 0.5, 0.5],
              [0.5, 0.0, 0.5]])
B = np.array([[0.7, 0.3],                       # P(symbol | state)
              [0.5, 0.5],
              [0.3, 0.7]])
obs = np.array([1, 0, 1, 0, 0])
T, K = len(obs), len(pi)

# Forward pass with scaling: alpha[t] = p(z_t | x_1..t), c[t] = p(x_t | x_1..t-1).
alpha, c = np.zeros((T, K)), np.zeros(T)
alpha[0] = pi * B[:, obs[0]]
c[0] = alpha[0].sum(); alpha[0] /= c[0]
for t in range(1, T):
    alpha[t] = B[:, obs[t]] * (alpha[t - 1] @ A)
    c[t] = alpha[t].sum(); alpha[t] /= c[t]
log_lik = np.log(c).sum()

# Backward pass (scaled) and smoothed marginals gamma[t] = p(z_t | x_1..T).
beta = np.ones((T, K))
for t in range(T - 2, -1, -1):
    beta[t] = A @ (B[:, obs[t + 1]] * beta[t + 1]) / c[t + 1]
```

```
gamma = alpha * beta

# Viterbi in log space (log 0 = -inf marks forbidden transitions).
with np.errstate(divide="ignore"):
    lA, lB, lpi = np.log(A), np.log(B), np.log(pi)
delta = lpi + lB[:, obs[0]]
back = []
for t in range(1, T):
    cand = delta[:, None] + lA
    back.append(cand.argmax(axis=0))
    delta = cand.max(axis=0) + lB[:, obs[t]]
path = [int(delta.argmax())]
for b in reversed(back):
    path.append(int(b[path[-1]]))
path = path[::-1]

# Brute force over all K**T paths as an independent check.
def joint(z):
    p = pi[z[0]] * B[z[0], obs[0]]
    for t in range(1, T):
        p *= A[z[t - 1], z[t]] * B[z[t], obs[t]]
    return p
paths = list(itertools.product(range(K), repeat=T))
assert np.isclose(log_lik, np.log(sum(joint(z) for z in paths)))
assert tuple(path) == max(paths, key=joint)

marginal = gamma.argmax(axis=1)
print("log p(x):", round(log_lik, 4))
print("Viterbi path:         ", path, " p =", round(joint(path), 5))
print("per-step argmax gamma:", marginal.tolist(),
      " p =", round(joint(marginal), 5))
```

The scaled forward pass gives $\log p(x) = -3.5449$, and both the likelihood and the Viterbi path agree with brute-force enumeration of all $3^5 = 243$ paths. Viterbi returns $1 \to 1 \to 2 \to 0 \to 0$ with joint probability 0.00179. Picking the most probable state at each step gives $2, 0, 2, 0, 0$, which contains the forbidden jump $0 \to 2$ and has probability exactly zero under the model. Which decoder is right depends on the loss: per-step accuracy favors the marginals; a valid, coherent sequence needs Viterbi.

Figure 5 shows the same computations on a longer simulated sequence: the filtered probabilities react only to the past, the smoothed ones use the whole sequence and are more confident, and the Viterbi path tracks the true states except where observations are ambiguous.

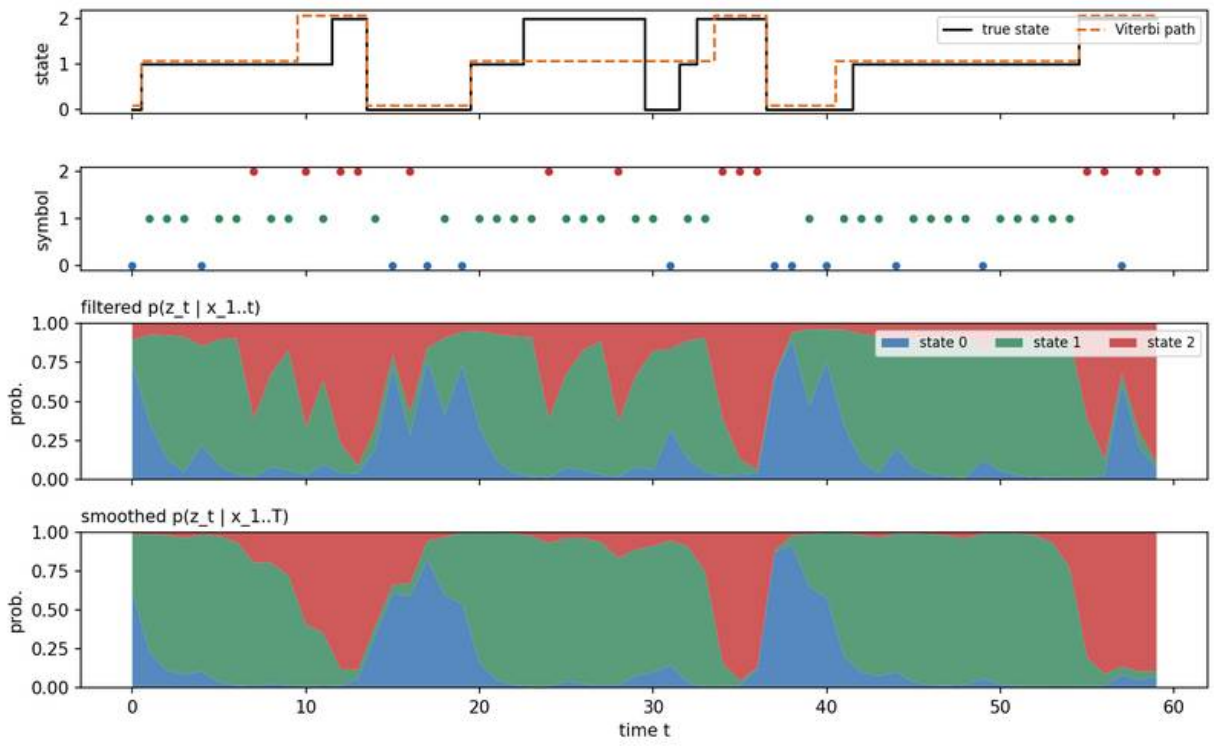


Figure 5 — A simulated 3-state cyclic HMM. Filtering uses observations up to time t; smoothing uses the whole sequence. On this run the Viterbi path matches the true state at 78% of time steps, the per-step smoothed choice at 77%, and the per-step filtered choice at 73%.

A note on evaluation. If a system must make decisions online, report filtered (or fixed-lag) performance. A smoothed result uses observations that would not yet exist at decision time, and it overstates what the deployed system can do.

Learning: Baum–Welch

When the hidden states are not labeled, the parameters are estimated by expectation–maximization, known for HMMs as the **Baum–Welch** algorithm [12]. The E-step runs forward–backward to get the expected state occupancies $\gamma_t(j)$ and the expected transitions $\xi_t(i, j) = p(z_{t-1} = i, z_t = j \mid x_{1:T})$. The M-step re-estimates

$$\hat{A}_{ij} = \frac{\sum_{t=2}^{T} \xi_t(i,j)}{\sum_{t=2}^{T} \sum_{k} \xi_t(i,k)}, \qquad \hat{\pi}_j = \gamma_1(j),$$

and the emission parameters from observations weighted by $\gamma_t(j)$ (for example, weighted means and covariances for Gaussian emissions). Each iteration does not decrease the likelihood, but the result depends on the initialization and can be a local maximum; PR 07 develops EM in general. If the hidden states are continuous and everything is linear-Gaussian, the same forward recursion

becomes the Kalman filter [13]. Neural recurrent models, which replace the discrete state by a learned vector, are covered in ML 03A.

Conditional random fields

In plain words. An HMM models how states *generate* observations, which forces simple assumptions about the observations. When all you need is the labels given the observations, a CRF models that directly, and it can then use any features of the whole input, overlapping or not.

An HMM is **generative**: it models $p(z, x)$, including how observations arise. That requires the emission model $B_j(x_t)$ to describe x_t fully, which is awkward when the features are many, overlapping and dependent (the word itself, its suffix, its capitalization, the neighboring words...). Lafferty, McCallum and Pereira introduced conditional random fields to model the **conditional** distribution of a label sequence $y = (y_1, \dots, y_T)$ given the observations x directly [14]. A **linear-chain CRF** is

$$p(y \mid x) = \frac{1}{Z(x)} \exp\Big(\sum_{t=1}^{T} \sum_{k} \theta_k f_k(y_{t-1}, y_t, x, t) \Big), \qquad Z(x) = \sum_{y'} \exp\Big(\sum_{t,k} \theta_k f_k(y'_{t-1}, y'_t, x, t) \Big),$$

where the f_k are feature functions that may look at the current and previous label and at *any* part of the input sequence, θ_k are learned weights, and $Z(x)$ normalizes over all label sequences y' for this particular input [14], [15]. An HMM, written in this form, is a special case in which the features are only "previous label, current label" and "current label, current observation" indicators. The CRF can add arbitrary input features without modeling their distribution.

Inference uses the same dynamic programs as the HMM: the forward algorithm (with log-sum-exp in place of sums) computes $Z(x)$ and the marginals, and Viterbi finds the best labeling. Training maximizes the conditional log-likelihood $\sum_n \log p(y^{(n)} \mid x^{(n)})$, usually with an ℓ_2 penalty. For a linear-chain CRF that objective is concave, so there are no bad local optima; its gradient is "observed feature counts minus expected feature counts," where the expectations come from forward–backward [15]. The code checks the forward computation of $\log Z$ against brute force.

```
import itertools
import numpy as np

rng = np.random.default_rng(0)
T, K = 5, 3
U = rng.normal(size=(T, K))       # unary scores  theta . f(y_t, x, t)
P = rng.normal(size=(K, K))       # transition scores  theta . f(y_{t-1}, y_t)

def score(y):
    return U[np.arange(T), y].sum() + sum(P[y[t - 1], y[t]] for t in range(1, T))

# log Z by the forward recursion in log space (log-sum-exp instead of sum).
log_alpha = U[0].copy()
for t in range(1, T):
    m = log_alpha[:, None] + P
    log_alpha = U[t] + np.logaddexp.reduce(m, axis=0)
```

```
log_Z = np.logaddexp.reduce(log_alpha)

brute = np.logaddexp.reduce([score(np.array(y))
                             for y in itertools.product(range(K), repeat=T)])
print("log Z forward:", round(log_Z, 6), " brute force:", round(brute, 6))
assert np.isclose(log_Z, brute)
```

Both give $\log Z = 6.173403$. Here the scores are random numbers standing in for $\sum_k \theta_k f_k$; in a real CRF they come from features of x, and in modern systems from a neural network.

On images, the same idea gives grid-structured CRFs over pixels, with unary terms from a pixel classifier and pairwise terms like the Potts energy above. Exact inference is no longer possible on loopy grids, so approximate methods are used (PR 08).

Modern view

Templates, alignment, grammars and Markov chains look old-fashioned next to deep networks. In fact each survives, sometimes inside the networks. These are the main threads in the review literature and recent work.

Template matching. Hashemi et al. [4] review the basic concept of matching, advances such as invariant features and deformable models, and applications including medical image analysis. They conclude that template matching works best combined with other recognition steps, for example NCC followed by a robust decision stage. Correlation-based matching is fast, simple and exact when appearance is controlled, and it remains standard in industrial inspection, image registration and as a building block of trackers. When appearance varies, matching moved to learned feature spaces; the "template" is now a feature map, and correlation is computed between deep features, but the operation is the same.

Time-series classification and DTW. Bagnall et al.'s "bake off" [16] reimplemented 18 time-series classification algorithms in one framework and compared them over 100 resamples of each of 85 UCR datasets. Only nine were significantly better than both benchmarks (one of which was 1-NN with DTW), and a collective of transformation-based ensembles (COTE) beat all others. The "bake off redux" by Middlehurst, Schäfer and Bagnall [17] repeated the exercise on the expanded 112-dataset archive plus 30 new problems, added convolution-based, feature-based and deep-learning categories to the taxonomy, and found two recent algorithms, MultiROCKET+Hydra and HIVE-COTE 2, significantly better than the rest. Ismail Fawaz et al. [18] reviewed deep learning for time-series classification with a unified taxonomy and trained 8,730 networks on 97 datasets in one open framework. Meanwhile DTW itself became a building block: soft-DTW [19] replaces the minimum in the DTW recursion by a smooth soft-minimum, which makes it differentiable so it can serve as a loss for training networks and for averaging time series.

Alignment inside neural networks. Connectionist temporal classification (CTC) [20] trains a recurrent network to output label sequences without a frame-by-frame alignment, by summing over all alignments with a forward–backward recursion closely related to the HMM's. The dynamic program survived; the emission model became a network.

CRFs with deep features. Sutton and McCallum's tutorial [15] remains the standard reference for CRF modeling, inference and training. In vision, fully connected CRFs with Gaussian edge potentials made mean-field inference over all pixel pairs efficient [21] and became a popular refinement step for semantic segmentation; Zheng et al. then wrote that mean-field inference as a recurrent network layer and trained it end to end with the CNN [22]. In language, the BiLSTM-CRF of Lample et al. [23] put a CRF output layer on top of a bidirectional LSTM for named-entity recognition, so that label transitions are modeled explicitly while features are learned. In both areas, later architectures with large receptive fields or attention often model enough context that the explicit CRF adds little, but the structured output layer is still used where label constraints matter.

Structure and graphs. The graph-matching surveys of Conte et al. [8] and Foggia et al. [9] trace the move from exact and error-tolerant matching to graph kernels, graph embeddings and learning-based methods. The modern descendants, graph neural networks, learn node and graph representations by message passing, a direct generalization of the local computations of factor graphs.

State-space models. The idea that a compact hidden state summarizes the past, the core of HMMs and the Kalman filter [13], is central again in sequence modeling. Selective state-space models such as Mamba [24] use input-dependent linear recurrences that run in time linear in the sequence length, offered as an alternative to attention for long sequences.

Open problems. DTW and edit distances are quadratic per comparison, so search over large collections relies on bounds and pruning [6]. Graph edit distance remains expensive to compute exactly. HMM and CRF results depend on the independence assumptions built into the graph; when those are wrong, smoothing can confidently erase short events. And for any online system, the distinction between filtered and smoothed evaluation is easy to get wrong.

Key takeaways

- NCC subtracts means and divides by norms, so it is invariant to brightness offsets and positive contrast changes; SSD and raw correlation are not. Neither handles rotation, scale or deformation.
- DTW finds the cheapest monotone alignment by dynamic programming in $O(nm)$; a Sakoe–Chiba band limits pathological warping and cost, but must be wider than the real timing differences.
- Structural recognition describes patterns as strings, grammars or graphs; edit distance compares strings with the same dynamic-programming pattern as DTW.
- Graphical models factorize joint distributions into local pieces; message passing on chains and trees is exact and efficient.
- HMM algorithms: forward ($p(x)$ and filtering), forward–backward (smoothing), Viterbi (best path), Baum–Welch (learning), all $O(TK^2)$. Scale or work in logs to avoid underflow.
- The Viterbi path and per-step most-probable states answer different questions; the latter can be an impossible sequence.
- CRFs model $p(y \mid x)$ directly, can use arbitrary overlapping input features, share the HMM's

inference algorithms and have a concave training objective in the linear-chain case.

Exercises

1. **NCC and negatives.** Show that NCC is unchanged when every pixel of a patch is replaced by $ax_i + b$ with $a > 0$. What happens when $a < 0$, and why can NCC not tell you the value of b?

   Hint

   Subtracting the mean removes b entirely, so no information about it survives. The factor a appears in the numerator and as $|a|$ in the denominator; it cancels for $a > 0$ and flips the sign for $a < 0$ (a photographic negative of a perfect match scores -1).

2. **DTW by hand.** Compute the DTW distance with cost $|x_i - t_j|$ between $x = (0, 1, 2)$ and $t = (0, 0, 1, 2)$. Write out the 3×4 table and the optimal path. What is the lock-step distance, and why is it not even defined here?

   Hint

   The optimal path matches x_1 to both zeros, then $x_2 \leftrightarrow t_3$ and $x_3 \leftrightarrow t_4$, with total cost 0. Lock-step comparison needs equal lengths, which these sequences do not have; DTW handles unequal lengths naturally.

3. **A forbidden transition.** Construct your own two- or three-state HMM with one transition probability set to zero and an observation sequence for which the per-step argmax of γ_t is an impossible state sequence. Verify with the code above.

   Hint

   You need two neighboring time steps whose individually most likely states are joined by the forbidden transition. A symmetric or cyclic model with ambiguous emissions (probabilities close to 0.5) makes this easy; then check that `joint(marginal)` is 0.

4. **Online versus offline.** An activity recognizer reports 95% accuracy computed from smoothed state probabilities on recorded sequences. It will run on a phone and must decide at each second. What information advantage does the reported number include, and what would you report instead?

   Hint

   Smoothing uses observations from the future of each decision. Report filtered accuracy, or fixed-lag smoothing with the lag the application can tolerate (for example decisions delayed by 2 seconds use observations up to $t + 2$).

5. **From HMM to CRF.** Write an HMM's $\log p(z, x)$ as a sum of feature functions times weights, in the form of the linear-chain CRF. Which features and weights do you get? What extra features could a CRF add for part-of-speech tagging that an HMM cannot easily use?

   Hint

   Use indicator features $\mathbf{1}[z_{t-1} = i, z_t = j]$ with weight $\log A_{ij}$, $\mathbf{1}[z_t = j, x_t = v]$ with weight $\log B_j(v)$, and $\mathbf{1}[t = 1, z_1 = j]$ with weight $\log \pi_j$. A CRF can add features such as "the next word ends in -ly" or "the current word is capitalized," which overlap with and depend on the word identity.

6. **Edit distance with weights.** Modify `edit_distance` so that substituting two directions that differ by 90 degrees costs 1 and by 180 degrees costs 2. Recompute the distances in the chain-code example. Does the ranking change?

   Hint

   The substitution cost is the circular difference $\min(|a - b|, 4 - |a - b|)$ between direction codes. The bumped square needs only insertions, so its distance stays 2; check whether the rectangle's distance changes, given which substitutions its cheapest edit sequence uses.

References

1. S. Theodoridis and K. Koutroumbas, *Pattern Recognition*, 4th ed., Academic Press, 2009, Chapters 8 and 9. publisher page
2. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006, Sections 8.1–8.4 and 13.1–13.3. official book page and PDF
3. J. P. Lewis, "Fast Normalized Cross-Correlation," *Vision Interface*, 1995. paper
4. N. S. Hashemi, R. B. Aghdam, A. S. B. Ghiasi, and P. Fatemi, "Template Matching Advances and Applications in Image Analysis," arXiv:1610.07231, 2016. arXiv
5. H. Sakoe and S. Chiba, "Dynamic Programming Algorithm Optimization for Spoken Word Recognition," *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 26, no. 1, pp. 43–49, 1978. doi:10.1109/TASSP.1978.1163055
6. T. Rakthanmanon, B. Campana, A. Mueen, G. Batista, B. Westover, Q. Zhu, J. Zakaria, and E. Keogh, "Searching and Mining Trillions of Time Series Subsequences under Dynamic Time Warping," *Proceedings of the 18th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 262–270, 2012. doi:10.1145/2339530.2339576
7. R. A. Wagner and M. J. Fischer, "The String-to-String Correction Problem," *Journal of the ACM*, vol. 21, no. 1, pp. 168–173, 1974. doi:10.1145/321796.321811
8. D. Conte, P. Foggia, C. Sansone, and M. Vento, "Thirty Years of Graph Matching in Pattern Recognition," *International Journal of Pattern Recognition and Artificial Intelligence*, vol. 18, no. 3, pp. 265–298, 2004. doi:10.1142/S0218001404003228
9. P. Foggia, G. Percannella, and M. Vento, "Graph Matching and Learning in Pattern Recognition in the Last 10 Years," *International Journal of Pattern Recognition and Artificial Intelligence*, vol. 28, no. 1, 1450001, 2014. doi:10.1142/S0218001414500013

10. L. R. Rabiner, “A Tutorial on Hidden Markov Models and Selected Applications in Speech Recognition,” *Proceedings of the IEEE*, vol. 77, no. 2, pp. 257–286, 1989. doi:10.1109/5.18626
11. A. J. Viterbi, “Error Bounds for Convolutional Codes and an Asymptotically Optimum Decoding Algorithm,” *IEEE Transactions on Information Theory*, vol. 13, no. 2, pp. 260–269, 1967. doi:10.1109/TIT.1967.1054010
12. L. E. Baum, T. Petrie, G. Soules, and N. Weiss, “A Maximization Technique Occurring in the Statistical Analysis of Probabilistic Functions of Markov Chains,” *The Annals of Mathematical Statistics*, vol. 41, no. 1, pp. 164–171, 1970. doi:10.1214/aoms/1177697196
13. R. E. Kalman, “A New Approach to Linear Filtering and Prediction Problems,” *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960. doi:10.1115/1.3662552
14. J. D. Lafferty, A. McCallum, and F. C. N. Pereira, “Conditional Random Fields: Probabilistic Models for Segmenting and Labeling Sequence Data,” *Proceedings of the 18th International Conference on Machine Learning (ICML)*, pp. 282–289, 2001. paper page
15. C. Sutton and A. McCallum, “An Introduction to Conditional Random Fields,” *Foundations and Trends in Machine Learning*, vol. 4, no. 4, pp. 267–373, 2012. doi:10.1561/2200000013
16. A. Bagnall, J. Lines, A. Bostrom, J. Large, and E. Keogh, “The Great Time Series Classification Bake Off: A Review and Experimental Evaluation of Recent Algorithmic Advances,” *Data Mining and Knowledge Discovery*, vol. 31, pp. 606–660, 2017. doi:10.1007/s10618-016-0483-9
17. M. Middlehurst, P. Schäfer, and A. Bagnall, “Bake Off Redux: A Review and Experimental Evaluation of Recent Time Series Classification Algorithms,” *Data Mining and Knowledge Discovery*, vol. 38, pp. 1958–2031, 2024. doi:10.1007/s10618-024-01022-1
18. H. Ismail Fawaz, G. Forestier, J. Weber, L. Idoumghar, and P.-A. Muller, “Deep Learning for Time Series Classification: A Review,” *Data Mining and Knowledge Discovery*, vol. 33, pp. 917–963, 2019. doi:10.1007/s10618-019-00619-1
19. M. Cuturi and M. Blondel, “Soft-DTW: A Differentiable Loss Function for Time-Series,” *ICML*, 2017. arXiv:1703.01541
20. A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber, “Connectionist Temporal Classification: Labelling Unsegmented Sequence Data with Recurrent Neural Networks,” *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pp. 369–376, 2006. doi:10.1145/1143844.1143891
21. P. Krähenbühl and V. Koltun, “Efficient Inference in Fully Connected CRFs with Gaussian Edge Potentials,” *NIPS*, 2011. arXiv:1210.5644
22. S. Zheng, S. Jayasumana, B. Romera-Paredes, V. Vineet, Z. Su, D. Du, C. Huang, and P. H. S. Torr, “Conditional Random Fields as Recurrent Neural Networks,” *ICCV*, 2015. arXiv:1502.03240
23. G. Lample, M. Ballesteros, S. Subramanian, K. Kawakami, and C. Dyer, “Neural Architectures for Named Entity Recognition,” *NAACL*, 2016. arXiv:1603.01360
24. A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” arXiv:2312.00752, 2023. arXiv

PR 07 • Clustering, Mixtures and EM

Group unlabeled data with k-means and k-means++, hierarchical linkage, DBSCAN and spectral methods; derive Gaussian-mixture EM; choose the number of clusters with BIC and validity indices.

Online edition: https://blog.shyandram.dev/en/learn/pattern-recognition/pr-07-clustering-mixtures/

What you'll learn

- What a clustering algorithm actually optimizes, and why the choice of features, scaling and distance decides the answer before any algorithm runs.
- How Lloyd's k-means works, why it always converges but only to a local optimum, and how k-means++ seeding gives a provable $O(\log K)$ guarantee.
- How hierarchical clustering (single, complete, average and Ward linkage), DBSCAN and spectral clustering encode different definitions of "a group".
- How to derive the E and M steps of expectation–maximization (EM) for a Gaussian mixture from scratch, why EM never decreases the likelihood, and where it can fail.
- How to choose the number of clusters with BIC and AIC, and how to read internal indices (silhouette, Davies–Bouldin, Calinski–Harabasz), external indices (ARI, NMI) and stability.

The big picture

So far in this track every training example came with a label. Clustering removes the labels and asks: are there natural groups in the data? The answer is never fully determined by the data alone. The same collection of images can be grouped by object, by color, by background or by the camera that took them. Each grouping is "correct" under its own notion of similarity. A clustering algorithm is therefore a precise statement of *which* notion of similarity we mean, plus a search procedure for the best grouping under that notion.

> **Explain like I'm five**
>
> You pour a big box of buttons on the floor and want to tidy them into piles. You could pile them by color, by size, or by how many holes they have. Every way makes neat piles, but which one is useful depends on what you want to do next. A clustering method is a rule for making piles. **k-means** says "pick a few pile-centers and put each button on the closest pile." **DBSCAN** says "buttons that are crowded together belong to the same pile, and lonely buttons are leftovers." A **Gaussian mixture** says "each pile is a fuzzy cloud, and a button can be 70% sure it belongs to one cloud and 30% to another."

Why it matters in recognition: clustering builds visual vocabularies (local image descriptors quantized with k-means into "visual words"), initializes and compresses models (vector quantization,

color palettes), discovers sub-classes inside a labeled class, finds outliers, segments images (pixel clustering, see IP 10), and, in modern self-supervised learning, produces pseudo-labels for training deep networks. Gaussian mixtures are also the standard example of a **latent-variable model**, and EM is the template for many inference algorithms we will meet in PR 08.

This chapter follows the clustering part of Theodoridis and Koutroumbas (Chapters 11–16) [1] and Bishop's Chapter 9 on mixtures and EM [2]. It organizes the main algorithm families, derives the two most important ones in full, and ends with how to decide whether a clustering is any good.

Similarity, scaling and what "a cluster" means

Plain version. Before choosing an algorithm, decide what "close" means. Change the features or their units and the clusters change.

Precise version. Most methods start from a dissimilarity $d(x, x')$ between feature vectors $x, x' \in \mathbb{R}^D$. Common choices are

$$d_2(x,x') = \|x - x'\|_2, \qquad d_M(x,x') = \sqrt{(x-x')^\top S^{-1}(x-x')}, \qquad d_{\cos}(x,x') = 1 - \frac{x^\top x'}{\|x\|\,\|x'\|},$$

where d_2 is the Euclidean distance, d_M is the Mahalanobis distance with a positive-definite matrix S (often a covariance estimate), and $d_{\cos}$ is the cosine dissimilarity, which ignores vector length and is popular for deep-network embeddings.

Three practical consequences follow.

1. **Scaling is a modeling decision.** If one coordinate is measured in millimeters and another in kilometers, Euclidean distance is dominated by the first. Standardizing every feature to unit variance removes that, but it also removes any genuinely meaningful difference in spread. For classification we can let validation decide; for clustering there are no labels, so the choice must come from domain knowledge.
2. **Hard versus soft assignments.** A *hard* partition assigns each point x_n to exactly one cluster $z_n \in \{1, \dots, K\}$. A *soft* assignment stores degrees of membership $r_{nk} \geq 0$ with $\sum_k r_{nk} = 1$. In a Gaussian mixture these are posterior probabilities; in fuzzy clustering [1] they are memberships defined by an objective and should not be read as probabilities.
3. **Clusters are not classes.** Labels are attached to clusters afterwards and arbitrarily. Cluster "3" in one run may be cluster "1" in the next, so any comparison with class labels must be invariant to permutations, which is exactly what ARI and NMI (below) are.

Theodoridis and Koutroumbas classify algorithms into sequential, hierarchical, objective-function-based (k-means, mixtures, fuzzy), and graph/density-based families [1]. We follow roughly that order.

K-means and Lloyd's algorithm

Plain version. Choose K centers. Repeat two steps: give every point to its nearest center, then move every center to the average of its points. Stop when nothing changes.

The objective

With one-hot indicators $z_{nk} \in \{0, 1\}$ ($z_{nk} = 1$ if point n is in cluster k, and $\sum_k z_{nk} = 1$) and centers $\mu_1, \dots, \mu_K \in \mathbb{R}^D$, k-means minimizes the within-cluster sum of squares

$$J(z, \mu) = \sum_{n=1}^{N} \sum_{k=1}^{K} z_{nk} \, \|x_n - \mu_k\|^2.$$

Here N is the number of points and D their dimension. J is also called *inertia* or *distortion*.

Two alternating steps

Lloyd's algorithm [3] (developed for quantizing signals in pulse-code modulation) minimizes J by **block coordinate descent**:

- **Assignment step.** Fix μ. Each term involves a single z_n, so set $z_{nk} = 1$ for $k = \arg\min_j \|x_n - \mu_j\|^2$.
- **Update step.** Fix z. J is a convex quadratic in each μ_k; setting the gradient to zero,

$$\frac{\partial J}{\partial \mu_k} = -2 \sum_n z_{nk}(x_n - \mu_k) = 0 \quad \Longrightarrow \quad \mu_k = \frac{\sum_n z_{nk} x_n}{\sum_n z_{nk}},$$

the mean of the points currently in cluster k. The name "k-means" was coined by MacQueen [4], whose version updates a center immediately after each point arrives, $\mu_k \leftarrow \mu_k + (x_n - \mu_k)/N_k$, where N_k is the running count of points in cluster k.

Why it converges, and why only locally

Each step can only lower J or leave it unchanged: the assignment step picks the best z for the current μ, and the update step picks the best μ for the current z. There are finitely many partitions of N points into K groups, and (with a consistent tie-breaking rule) the algorithm never revisits a partition with a higher J, so it stops after finitely many iterations. The fixed point is a **local** minimum: a partition where no single step helps. Different initial centers can lead to very different fixed points.

Two implementation details matter. An **empty cluster** (no point assigned) has no mean; common rules keep the old center or re-seed it at the point with the largest current error. And every iteration costs $O(NKD)$ distance computations, which motivates the mini-batch variant below.

What k-means assumes

Because the objective is a sum of squared Euclidean distances to a center, the boundary between two clusters is the perpendicular bisector of their centers: k-means partitions space into **Voronoi cells**. It works well for compact, roughly spherical groups of similar size. It splits elongated groups, merges small groups that are close to large ones, and cannot follow curved shapes (Figure 1, middle).

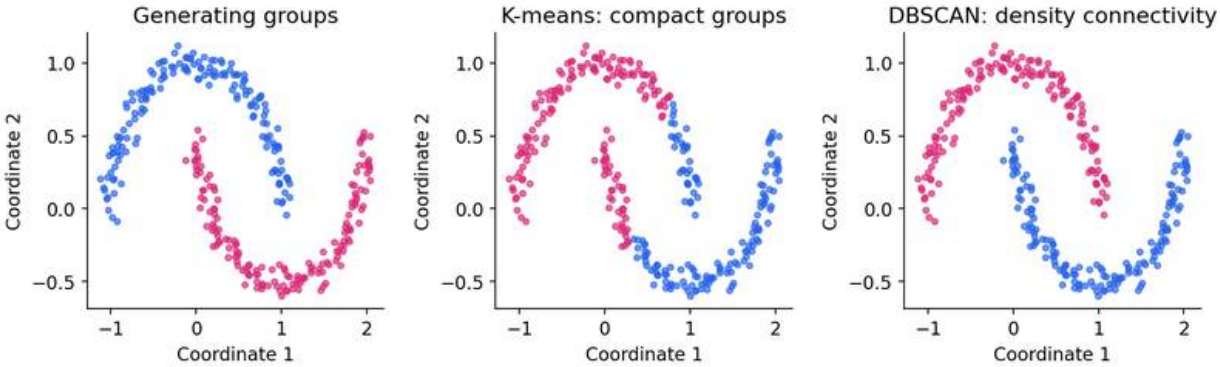


Figure 1 — The same two-moons data grouped by the generating labels (left), k-means (middle) and DBSCAN (right). K-means and DBSCAN fit without labels. Colors are arbitrary cluster identifiers.

k-means++ seeding

Plain version. Start with one random center, then pick each new center far from the ones you already have, but randomly, so outliers do not always win.

Arthur and Vassilvitskii [5] proposed **D² seeding**:

1. Pick the first center uniformly at random from the data.
2. For every point compute $D(x_n)$, the distance to the nearest center chosen so far.
3. Pick the next center as x_n with probability $D(x_n)^2 / \sum_m D(x_m)^2$.
4. Repeat steps 2–3 until K centers are chosen, then run Lloyd's algorithm.

Their main theorem states that the seeding alone already achieves, in expectation,

$$\mathbb{E}[J] \leq 8(\ln K + 2)\, J_{\mathrm{OPT}},$$

where J_{OPT} is the optimal k-means objective. Lloyd iterations can only lower J further. Uniform random seeding has no such guarantee. The bound is a worst-case statement; in practice the benefit shows up mostly as fewer catastrophic runs. Libraries combine k-means++ with several restarts (`n_init` in scikit-learn) and keep the run with the lowest J.

Code: k-means++ and Lloyd from scratch

```
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans

def kmeans_pp_init(X, k, rng):
    """D^2 seeding: each new center is drawn with probability proportional
    to the squared distance to the nearest center chosen so far."""
    centers = [X[rng.integers(len(X))]]
    for _ in range(k - 1):
        d2 = np.min(((X[:, None, :] - np.array(centers)[None]) ** 2).sum(-1), axis=1)
        centers.append(X[rng.choice(len(X), p=d2 / d2.sum())])
    return np.array(centers)

def lloyd(X, centers, max_iter=100):
    history = []
    for _ in range(max_iter):
        d2 = ((X[:, None, :] - centers[None]) ** 2).sum(-1)   # N x K
        z = d2.argmin(axis=1)                                 # assignment step
        history.append(d2[np.arange(len(X)), z].sum())        # objective J
        new = np.array([X[z == k].mean(axis=0) if np.any(z == k) else centers[k]
                        for k in range(len(centers))])        # update step
        if np.allclose(new, centers):
            break
        centers = new
    return centers, z, history

X, _ = make_blobs(n_samples=600, centers=4, cluster_std=1.8, random_state=7)
rng = np.random.default_rng(0)
best = None
for restart in range(10):                       # several seeds, keep the best J
    c, z, hist = lloyd(X, kmeans_pp_init(X, 4, rng))
    if best is None or hist[-1] < best[2][-1]:
        best = (c, z, hist)
c, z, hist = best
print("J per iteration:", np.round(hist, 1))
assert all(a >= b - 1e-9 for a, b in zip(hist, hist[1:]))   # monotone
ref = KMeans(n_clusters=4, n_init=10, random_state=0).fit(X)
print("from scratch J:", round(hist[-1], 2), " sklearn inertia:", round(ref.inertia_, 2))

# single runs (no restarts): uniform random seeding vs D^2 seeding, 15 clusters
X2, _ = make_blobs(n_samples=1500, centers=15, cluster_std=0.6, random_state=2)
for name, init in [("random", lambda r: X2[r.choice(len(X2), 15, replace=False)]),
                   ("k-means++", lambda r: kmeans_pp_init(X2, 15, r))]:
    finals = [lloyd(X2, init(np.random.default_rng(s)))[2][-1] for s in range(20)]
    print(f"{name:9s} final J over 20 runs: median {np.median(finals):7.1f}, "
          f"worst {np.max(finals):7.1f}")
```

Output (NumPy 2.0, scikit-learn 1.6):

```
J per iteration: [5865.6 3633.4 3625. ]
```

```
from scratch J: 3625.0  sklearn inertia: 3625.0
random    final J over 20 runs: median  1305.1, worst  4750.8
k-means++ final J over 20 runs: median  1147.5, worst  1525.2
```

The from-scratch version reaches the same objective as scikit-learn, and J decreases monotonically. On the 15-cluster problem, uniform seeding sometimes puts two seeds in one blob and none in another; Lloyd's algorithm cannot repair that, and the worst run is about three times worse. Figure 2 shows the two worst runs.

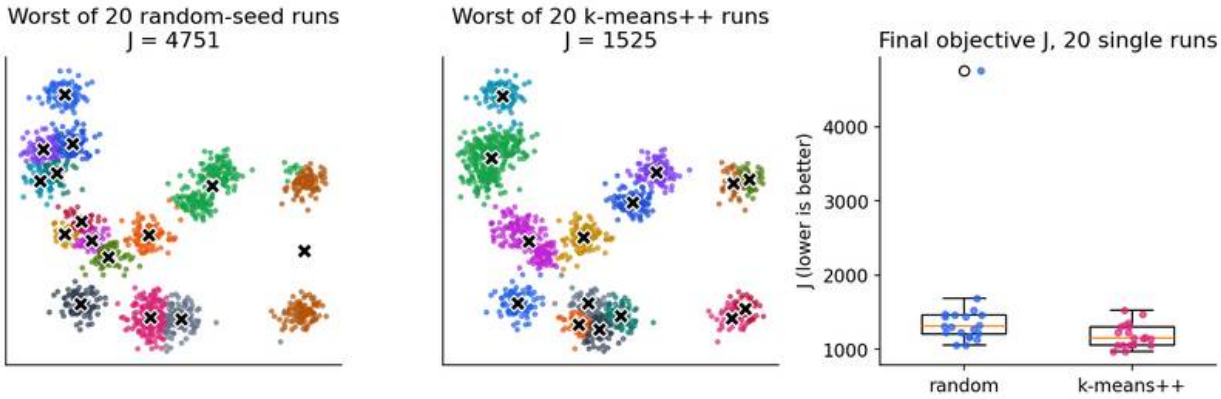


Figure 2 — Seeding matters. Fifteen synthetic blobs, 20 single Lloyd runs per seeding rule (no restarts). Random seeding leaves some runs stuck with merged and split blobs; D^2 (k-means++) seeding gives lower and less variable objectives. Black crosses are final centers.

Mini-batch k-means

For millions of points, Sculley's mini-batch k-means [6] updates the centers using small random batches: each center moves toward the batch points assigned to it with a per-center learning rate that decays as 1/(count of points seen). It trades a slightly higher final objective for a large speed-up, and is available as `MiniBatchKMeans` in scikit-learn. It is the usual choice for building large visual vocabularies.

Hierarchical clustering

Plain version. Start with every point as its own group. Repeatedly merge the two closest groups. The record of merges is a tree; cut it at some height to get clusters.

Agglomerative algorithm and linkages

Agglomerative clustering needs a definition of the distance between two *groups* A and B. The four classical **linkages** are

$$d_{\text{single}}(A,B) = \min_{a\in A,\, b\in B} d(a,b),$$
$$d_{\text{complete}}(A,B) = \max_{a\in A,\, b\in B} d(a,b),$$
$$d_{\text{average}}(A,B) = \frac{1}{|A|\,|B|}\sum_{a\in A}\sum_{b\in B} d(a,b),$$
$$\Delta_{\text{Ward}}(A,B) = \frac{|A|\,|B|}{|A|+|B|}\,\|\bar{a}-\bar{b}\|^2,$$

where $|A|$ is the number of points in A and $\bar{a}, \bar{b}$ are the group means. Ward's criterion [7] is the increase of the within-cluster sum of squares J caused by merging A and B, so Ward agglomeration is a greedy, bottom-up way of keeping J small, the same objective as k-means.

Each linkage has a personality (Figure 3):

- **Single linkage** merges whenever *any* two points are close. It follows arbitrary shapes, but a thin "bridge" of points can chain two groups together. It is equivalent to cutting the longest edges of a minimum spanning tree, one of the graph-based methods in [1].
- **Complete linkage** requires *all* pairs to be close. It produces compact groups of similar diameter and is sensitive to outliers.
- **Average linkage** sits between the two.
- **Ward** produces compact, similarly sized, roughly spherical groups, much like k-means.

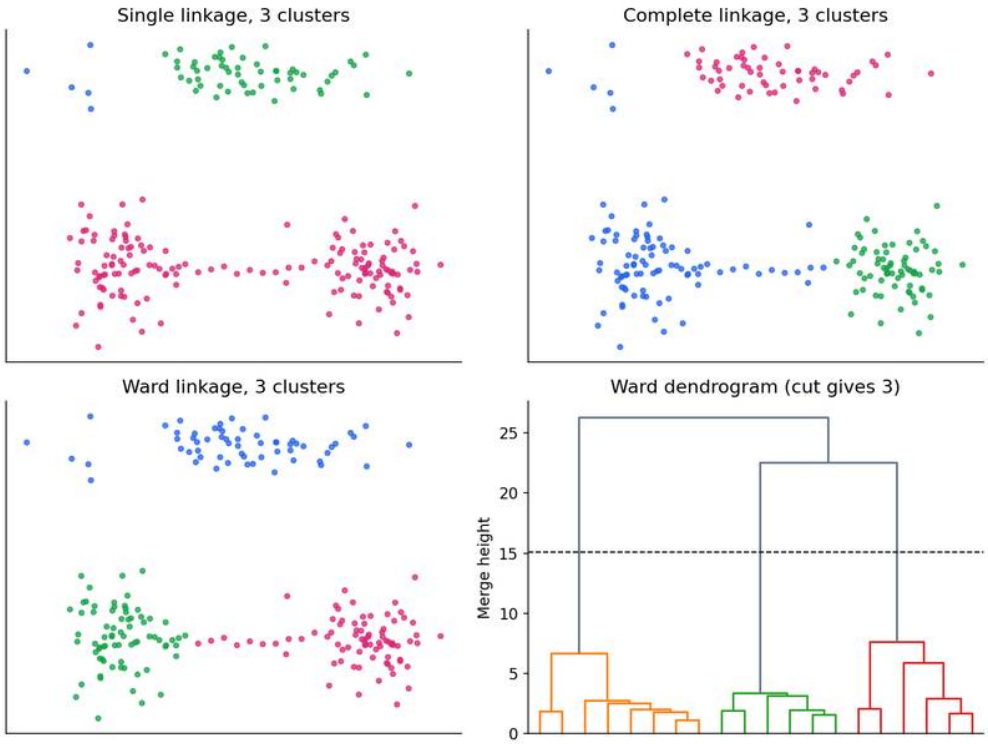


Figure 3 — One data set, three linkages. Single linkage chains along the thin bridge, merges both blobs into

one cluster and spends a cluster on four stray points; complete and Ward linkage keep the two blobs apart and split the bridge between them. Bottom right: the Ward dendrogram, where cutting at the dashed line yields three clusters.

The dendrogram

The merge history is a binary tree called a **dendrogram**; the height of each junction is the linkage value at which the merge happened. Cutting the tree at height h yields the partition at that level, and a large gap between consecutive merge heights suggests a natural number of clusters. The cut height is still a choice.

Two practical limitations: merges are never undone, so an early mistake propagates; and the standard algorithm stores all pairwise distances, $O(N^2)$ memory, which limits it to tens of thousands of points. Divisive (top-down) hierarchical methods exist but are less common.

Density-based clustering: DBSCAN

Plain version. A cluster is a crowded region. Points in crowded places grow the cluster; points in empty places are labeled noise.

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) [8] has two parameters, a radius ε and a count MinPts. Let $N_\varepsilon(x) = \{x' : d(x, x') \le \varepsilon\}$ be the ε-neighborhood of x (including x itself). Then

- x is a **core point** if $|N_\varepsilon(x)| \ge$ MinPts;
- x is a **border point** if it is not core but lies in the ε-neighborhood of some core point;
- every other point is **noise**.

A point is *density-reachable* from a core point if there is a chain of core points, each within ε of the next, ending in its neighborhood. A cluster is a maximal set of mutually density-connected points. DBSCAN finds clusters of any shape, does not need K, and explicitly refuses to assign isolated points.

Choosing ε. The original paper suggests the sorted **k-distance graph**: for each point compute the distance to its k-th nearest neighbor with $k =$ MinPts, sort these values, and pick ε near the "knee" where the curve rises sharply (Figure 4, left). In the descending plot of Figure 4 the large values to the left of the knee are the candidates for noise. scikit-learn's `min_samples` counts the point itself, which matches the definition above.

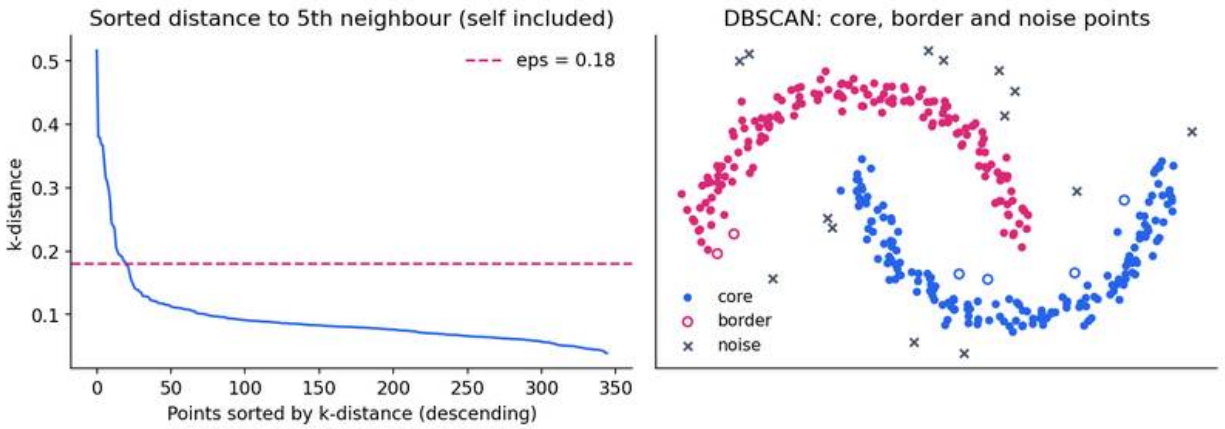


Figure 4 — DBSCAN on two moons plus 25 uniform outliers. Left: the sorted 5-distance curve; the chosen $\varepsilon = 0.18$ sits where the curve starts to rise. Right: core points (filled), border points (hollow) and points rejected as noise (crosses).

Limitations. A single ε means a single density level. If one cluster is dense and another sparse, no ε separates both. In high dimensions, distances concentrate and the neighborhood counts lose contrast. **HDBSCAN** [9] addresses the first issue by building a hierarchy over all density levels and extracting the most stable clusters, replacing ε with a minimum cluster size; it is available as `HDBSCAN` in scikit-learn.

Spectral clustering in one page

Plain version. Build a graph that connects nearby points, then find the cuts that sever the fewest strong connections.

Given an affinity matrix W ($W_{ij} \geq 0$, large for similar points, for example a k-nearest-neighbor graph or $W_{ij} = \exp(-\|x_i - x_j\|^2/2\sigma^2)$) and the diagonal degree matrix $D_{ii} = \sum_j W_{ij}$, one normalized graph Laplacian is

$$L_{\text{sym}} = I - D^{-1/2} W D^{-1/2}.$$

If the graph has K connected components, L_{sym} has eigenvalue 0 with multiplicity K, and its eigenvectors indicate the components. For a nearly disconnected graph, the K eigenvectors with the smallest eigenvalues still nearly indicate the groups. Spectral clustering stacks these eigenvectors as columns, normalizes the rows, and runs k-means on the rows [10]. The result can follow curved shapes that defeat k-means in the original space, because the eigenvectors measure *connectivity*, not distance to a center.

The graph is part of the model. The neighborhood size or bandwidth σ must be chosen; a graph that is accidentally disconnected (an isolated vertex, say) produces a trivial eigenvector that dominates the result. Von Luxburg's tutorial [10] explains the graph-cut interpretation (ratio cut and normalized cut) and gives practical advice on choosing the graph.

Code: one geometry, different assumptions

The two-moons data set below is generated with labels, but the labels are used only to score each result afterwards with the adjusted Rand index (ARI, 1 = perfect agreement up to relabeling, about 0 = chance).

```
import numpy as np
from sklearn.datasets import make_moons
from sklearn.neighbors import NearestNeighbors
from sklearn.cluster import (KMeans, AgglomerativeClustering, DBSCAN,
                             HDBSCAN, SpectralClustering)
from sklearn.mixture import GaussianMixture
from sklearn.metrics import adjusted_rand_score

X, truth = make_moons(n_samples=320, noise=0.06, random_state=5)

# k-distance curve for choosing DBSCAN's eps (k = min_samples)
dist, _ = NearestNeighbors(n_neighbors=5).fit(X).kneighbors(X)
kdist = np.sort(dist[:, -1])                    # 5th neighbour, counting the point itself
print("k-distance quantiles 50/90/99%:", np.round(np.quantile(kdist, [0.5, 0.9, 0.99]), 3))

models = {
    "k-means": KMeans(n_clusters=2, n_init=10, random_state=5),
    "single linkage": AgglomerativeClustering(n_clusters=2, linkage="single"),
    "Ward linkage": AgglomerativeClustering(n_clusters=2, linkage="ward"),
    "Gaussian mixture": GaussianMixture(n_components=2, n_init=3, random_state=5),
    "DBSCAN": DBSCAN(eps=0.18, min_samples=5),
    "HDBSCAN": HDBSCAN(min_cluster_size=15),
    "spectral": SpectralClustering(n_clusters=2, affinity="nearest_neighbors",
                                   n_neighbors=20, assign_labels="kmeans",
                                   random_state=5),
}
for name, model in models.items():
    labels = model.fit_predict(X)
    print(f"{name:17s} ARI = {adjusted_rand_score(truth, labels):.3f}"
          f"   noise points = {int((labels == -1).sum())}")
```

```
k-distance quantiles 50/90/99%: [0.079 0.11  0.144]
k-means           ARI = 0.260   noise points = 0
single linkage    ARI = 1.000   noise points = 0
Ward linkage      ARI = 0.242   noise points = 0
Gaussian mixture  ARI = 0.488   noise points = 0
DBSCAN            ARI = 1.000   noise points = 0
HDBSCAN           ARI = 0.988   noise points = 2
spectral          ARI = 1.000   noise points = 0
```

Figure 1 shows two of these results. Methods built on centers or ellipses (k-means, Ward, a two-component Gaussian mixture) cut each moon in half; connectivity-based methods (single linkage, DBSCAN, HDBSCAN, spectral) recover the moons. The chosen $\varepsilon = 0.18$ lies above the 99th

percentile of the k-distance, so no moon point is called noise. A k-nearest-neighbor graph with too few neighbors can be disconnected, and scikit-learn then warns that the spectral embedding may be unreliable; 20 neighbors avoids that here. None of this makes connectivity methods "better": on overlapping Gaussian blobs single linkage chains everything together and k-means wins. Comparisons must vary geometry, density, noise and dimension.

Gaussian mixtures and the EM algorithm

Plain version. Pretend every point was produced by first rolling a K-sided die to pick a cloud, then sampling from that Gaussian cloud. We never see the die. EM guesses, for every point, how likely each cloud produced it (E step), then refits each cloud using those guesses as weights (M step), and repeats.

The model

A Gaussian mixture model (GMM) with K components has density

$$p(x \mid \theta) = \sum_{k=1}^{K} \pi_k \, \mathcal{N}(x \mid \mu_k, \Sigma_k), \qquad \pi_k \geq 0, \quad \sum_{k=1}^{K} \pi_k = 1,$$

where π_k is the **mixing weight** (prior probability of component k), $\mu_k \in \mathbb{R}^D$ its mean, Σ_k its $D \times D$ covariance, $\theta = \{\pi_k, \mu_k, \Sigma_k\}_{k=1}^{K}$ collects all parameters, and

$$\mathcal{N}(x \mid \mu, \Sigma) = \frac{1}{(2\pi)^{D/2} |\Sigma|^{1/2}} \exp\Big(-\tfrac{1}{2} (x-\mu)^{\top} \Sigma^{-1} (x-\mu) \Big).$$

Introduce a **latent** one-hot variable z_n with $p(z_{nk} = 1) = \pi_k$ and $p(x_n \mid z_{nk} = 1) = \mathcal{N}(x_n \mid \mu_k, \Sigma_k)$. Summing out z_n gives back the mixture density. Given data $X = \{x_1, \dots, x_N\}$, the log-likelihood is

$$\ln p(X \mid \theta) = \sum_{n=1}^{N} \ln \sum_{k=1}^{K} \pi_k \, \mathcal{N}(x_n \mid \mu_k, \Sigma_k).$$

The logarithm of a sum couples all the parameters, so setting its gradient to zero has no closed-form solution. EM is the standard way around this [11], [2].

Step 1: the complete-data log-likelihood

If we *knew* the assignments $Z = \{z_n\}$, the log-likelihood of (X, Z) would be

$$\ln p(X, Z \mid \theta) = \sum_{n=1}^{N} \sum_{k=1}^{K} z_{nk} \Big[\ln \pi_k + \ln \mathcal{N}(x_n \mid \mu_k, \Sigma_k) \Big].$$

The logarithm now acts on a single Gaussian, and the problem splits into K separate weighted Gaussian fits: easy.

Step 2 (E step): the expected complete-data log-likelihood

We do not know Z, but with current parameters θ^{old} we know its posterior. By Bayes' rule,

$$r_{nk} \equiv p(z_{nk} = 1 \mid x_n, \theta^{\text{old}}) = \frac{\pi_k \, \mathcal{N}(x_n \mid \mu_k, \Sigma_k)}{\sum_{j=1}^{K} \pi_j \, \mathcal{N}(x_n \mid \mu_j, \Sigma_j)},$$

evaluated at θ^{old}. The number r_{nk} is the **responsibility** of component k for point n; each row sums to one. Because the complete-data log-likelihood is linear in z_{nk}, its expectation under this posterior simply replaces z_{nk} by r_{nk}:

$$Q(\theta, \theta^{\text{old}}) = \sum_{n} \sum_{k} r_{nk} \Big[\ln \pi_k - \tfrac{D}{2} \ln 2\pi - \tfrac{1}{2} \ln |\Sigma_k| - \tfrac{1}{2} (x_n - \mu_k)^{\top} \Sigma_k^{-1} (x_n - \mu_k) \Big].$$

Step 3 (M step): maximize Q in closed form

Define the **effective count** $N_k = \sum_n r_{nk}$.

Mixing weights. Only $\sum_n \sum_k r_{nk} \ln \pi_k$ depends on π, under the constraint $\sum_k \pi_k = 1$. With a Lagrange multiplier λ,

$$\frac{\partial}{\partial \pi_k} \Big[\sum_{n,k} r_{nk} \ln \pi_k + \lambda \Big(\sum_k \pi_k - 1 \Big) \Big] = \frac{N_k}{\pi_k} + \lambda = 0 \;\Rightarrow\; \pi_k = -\frac{N_k}{\lambda}.$$

Summing over k gives $\lambda = -N$, so $\pi_k = N_k / N$. (The constraint $\pi_k \geq 0$ is satisfied automatically.)

Means. The gradient with respect to μ_k is $\sum_n r_{nk} \Sigma_k^{-1} (x_n - \mu_k)$. Setting it to zero and multiplying by Σ_k,

$$\mu_k = \frac{1}{N_k} \sum_{n} r_{nk} \, x_n.$$

Covariances. Write $\Lambda_k = \Sigma_k^{-1}$, so $-\tfrac{1}{2} \ln |\Sigma_k| = \tfrac{1}{2} \ln |\Lambda_k|$ and the quadratic term is $-\tfrac{1}{2} \operatorname{tr}\big(\Lambda_k (x_n - \mu_k)(x_n - \mu_k)^{\top}\big)$. Using $\partial \ln |\Lambda| / \partial \Lambda = \Lambda^{-1}$ and $\partial \operatorname{tr}(\Lambda A) / \partial \Lambda = A^{\top}$ for symmetric A,

$$\frac{\partial Q}{\partial \Lambda_k} = \frac{1}{2} \sum_{n} r_{nk} \Big[\Sigma_k - (x_n - \mu_k)(x_n - \mu_k)^{\top} \Big] = 0 \;\Rightarrow\; \Sigma_k = \frac{1}{N_k} \sum_{n} r_{nk} (x_n - \mu_k)(x_n - \mu_k)^{\top}.$$

These are the familiar maximum-likelihood formulas for one Gaussian (see PR 02), with every point weighted by its responsibility. EM alternates the E step and the M step until the log-likelihood stops improving (Figure 5).

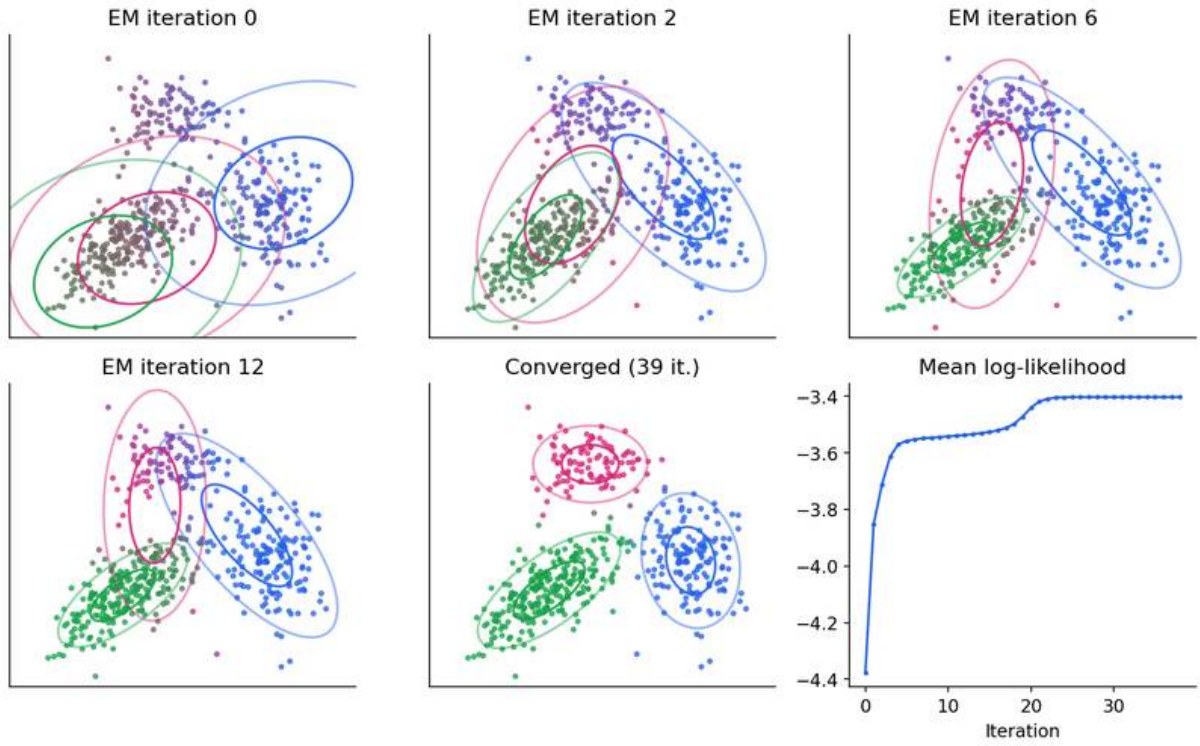


Figure 5 — EM for a three-component Gaussian mixture. The components start at random data points with the overall covariance; point colors mix the component colors by responsibility. Right: the mean log-likelihood never decreases.

Why EM never decreases the likelihood

For any distribution $q(Z)$ over the latent assignments,

$$\ln p(X \mid \theta) = \underbrace{\sum_{Z} q(Z) \ln \frac{p(X, Z \mid \theta)}{q(Z)}}_{\mathcal{L}(q,\theta)} + \underbrace{\sum_{Z} q(Z) \ln \frac{q(Z)}{p(Z \mid X, \theta)}}_{\mathrm{KL}(q \,\|\, p(Z \mid X, \theta)) \geq 0}.$$

So $\mathcal{L}(q, \theta)$ is a lower bound on the log-likelihood, the **evidence lower bound** (ELBO). The E step sets $q(Z) = p(Z \mid X, \theta^{\text{old}})$, which makes the KL term zero and the bound tight at θ^{old}. The M step maximizes $\mathcal{L}$ over θ; since $\mathcal{L}(q, \theta) = Q(\theta, \theta^{\text{old}}) + \text{const}$, this is the same as maximizing Q. The new likelihood is at least the new bound, which is at least the old bound, which equals the old likelihood. Hence $\ln p(X \mid \theta^{\text{new}}) \geq \ln p(X \mid \theta^{\text{old}})$ [11], [2]. When the exact posterior is intractable, we restrict q to a tractable family and get **variational inference**, the subject of PR 08.

Monotone does not mean global: EM converges to a stationary point that depends on initialization. Practical implementations initialize from k-means and run several restarts.

Singularities and how to avoid them

The likelihood of an unconstrained GMM is **unbounded**. Put one component's mean exactly on a data point, $\mu_j = x_n$, with $\Sigma_j = \sigma^2 I$. That term contributes $\pi_j (2\pi\sigma^2)^{-D/2}$ to $p(x_n)$, which goes to infinity as $\sigma \to 0$, while the other components keep the remaining points' likelihood finite. The maximum likelihood estimate therefore does not exist; what EM finds is a good local maximum that avoids collapse. Remedies: add a small constant to the covariance diagonal (`reg_covar` in scikit-learn), restrict the covariance (diagonal, spherical, tied), place a prior on the parameters (MAP-EM or Bayesian mixtures), or restart a collapsing component.

Identifiability and label switching

Permuting the component labels leaves the density unchanged, so there are $K!$ equivalent maxima. "Component 2" has no inherent meaning, and comparing component indices with class labels requires a matching rule.

K-means as the limit of EM

Fix $\pi_k = 1/K$ and $\Sigma_k = \epsilon I$ for all k. The responsibilities become

$$r_{nk} = \frac{\exp(-\|x_n - \mu_k\|^2 / 2\epsilon)}{\sum_j \exp(-\|x_n - \mu_j\|^2 / 2\epsilon)}.$$

As $\epsilon \to 0$, the term with the smallest distance dominates, so $r_{nk} \to 1$ for the nearest center and 0 otherwise: the E step becomes the k-means assignment step and the M step for μ_k becomes the k-means update [2]. A general GMM therefore extends k-means with soft assignments, unequal sizes (π_k) and elliptical shapes (Σ_k).

Code: EM from scratch, checked against scikit-learn

```
import numpy as np
from scipy.special import logsumexp
from scipy.stats import multivariate_normal
from sklearn.mixture import GaussianMixture

rng = np.random.default_rng(3)
true_means = np.array([[0.0, 0.0], [4.0, 1.0], [1.0, 4.0]])
true_covs = [np.array([[1.0, 0.6], [0.6, 0.8]]),
             np.array([[0.5, -0.2], [-0.2, 1.2]]),
             np.array([[0.7, 0.0], [0.0, 0.3]])]
counts = [250, 150, 100]
X = np.vstack([rng.multivariate_normal(m, S, n)
               for m, S, n in zip(true_means, true_covs, counts)])
N, D = X.shape
K, reg = 3, 1e-6                              # reg: small covariance floor
```

```
# initialise with K random points, shared data covariance, equal weights
mu = X[rng.choice(N, K, replace=False)]
Sigma = np.array([np.cov(X.T) for _ in range(K)])
pi = np.full(K, 1.0 / K)
loglik = []
for it in range(200):
    # E step: log of pi_k N(x_n | mu_k, Sigma_k), normalised with log-sum-exp
    log_joint = np.column_stack([np.log(pi[k]) +
                                 multivariate_normal(mu[k], Sigma[k]).logpdf(X)
                                 for k in range(K)])
    log_norm = logsumexp(log_joint, axis=1)
    R = np.exp(log_joint - log_norm[:, None])     # responsibilities r_nk
    loglik.append(log_norm.sum())
    # M step: closed-form weighted updates
    Nk = R.sum(axis=0)
    pi = Nk / N
    mu = (R.T @ X) / Nk[:, None]
    for k in range(K):
        diff = X - mu[k]
        Sigma[k] = (R[:, k, None] * diff).T @ diff / Nk[k] + reg * np.eye(D)
    if it > 0 and loglik[-1] - loglik[-2] < 1e-8:
        break

assert np.all(np.diff(loglik) > -1e-6)           # EM never decreases log-likelihood
print("iterations:", len(loglik), " final mean log-lik:", round(loglik[-1] / N, 4))
print("weights:", np.round(np.sort(pi), 3))
gm = GaussianMixture(3, covariance_type="full", n_init=5, random_state=0).fit(X)
print("sklearn mean log-lik:", round(gm.score(X), 4))
print("sklearn weights:", np.round(np.sort(gm.weights_), 3))
```

```
iterations: 40  final mean log-lik: -3.4032
weights: [0.202 0.299 0.499]
sklearn mean log-lik: -3.4033
sklearn weights: [0.203 0.3   0.497]
```

The E step works in log space: densities in high dimension underflow, and `logsumexp` computes $\ln \sum_k e^{a_k}$ stably. The recovered weights match the generating proportions $0.5, 0.3, 0.2$ and scikit-learn's solution; the tiny difference comes from scikit-learn's own stopping tolerance.

Choosing the number of clusters

Plain version. More clusters always fit the training data better. A model-selection criterion charges a price for every extra parameter, and the best K balances fit against that price.

BIC and AIC for mixtures

For a model with maximized likelihood $\hat{L}$, p free parameters and N data points,

$$\mathrm{BIC} = -2\ln\hat{L} + p\ln N, \qquad \mathrm{AIC} = -2\ln\hat{L} + 2p,$$

and smaller is better. The Bayesian information criterion (BIC) [12] comes from a large-sample approximation of the log marginal likelihood (model evidence, see PR 08); Akaike's information criterion (AIC) [13] estimates expected predictive log-loss. Because $\ln N > 2$ once $N \geq 8$, BIC penalizes complexity more and tends to choose fewer components.

For a full-covariance GMM in D dimensions,

$$p = \underbrace{(K-1)}_{\text{weights}} + \underbrace{KD}_{\text{means}} + \underbrace{K\,\tfrac{D(D+1)}{2}}_{\text{covariances}}.$$

The covariances dominate in high dimension: with $D = 128$ (a SIFT-like descriptor) each full covariance has 8,256 free numbers, which is why diagonal covariances are standard for large descriptors. The covariance *type* is therefore part of model selection too.

```
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.mixture import GaussianMixture

X, _ = make_blobs(n_samples=600, centers=4, cluster_std=1.0, random_state=7)
X = X @ np.array([[1.0, 0.0], [0.6, 0.5]])        # shear: make clusters elliptical
results = []
for cov in ["spherical", "diag", "full"]:
    for K in range(1, 9):
        gm = GaussianMixture(K, covariance_type=cov, n_init=3, random_state=0).fit(X)
        results.append((gm.bic(X), gm.aic(X), cov, K))
for bic, aic, cov, K in sorted(results)[:5]:
    print(f"BIC={bic:8.1f}  AIC={aic:8.1f}  {cov:9s} K={K}")

# check the parameter count used by BIC for a full-covariance model
gm = GaussianMixture(4, covariance_type="full", random_state=0).fit(X)
D, K = X.shape[1], 4
p = (K - 1) + K * D + K * D * (D + 1) // 2
manual = -2 * gm.score(X) * len(X) + p * np.log(len(X))
print("free parameters:", p, " manual BIC:", round(manual, 1), " sklearn:", round(gm.bic(X),
 ↪  1))
```

```
BIC=  4301.1  AIC=  4200.0  full      K=4
BIC=  4320.7  AIC=  4193.2  full      K=5
BIC=  4357.6  AIC=  4203.7  full      K=6
BIC=  4388.4  AIC=  4208.2  full      K=7
BIC=  4414.9  AIC=  4243.4  diag      K=8
free parameters: 23  manual BIC: 4301.1  sklearn: 4301.1
```

BIC picks the true $K = 4$ with full covariances (the shear made the clusters elliptical, so spherical and diagonal models need extra components). AIC slightly prefers $K = 5$: its lighter penalty lets a fifth component buy a small likelihood gain. Neither criterion is "right" in general; both assume the model family is adequate, and for non-Gaussian clusters BIC will happily use several

Gaussians to describe one curved group. If the goal is density estimation, that is fine; if each component is supposed to be one semantic group, it is not.

Other heuristics

The **elbow** method plots k-means J against K and looks for a bend; J always decreases, so the bend is subjective. The silhouette and stability approaches in the next section give alternatives. A Bayesian treatment can also start with more components than needed and let a prior on the mixing weights switch the unnecessary ones off; Bishop works through this for a variational Gaussian mixture [2].

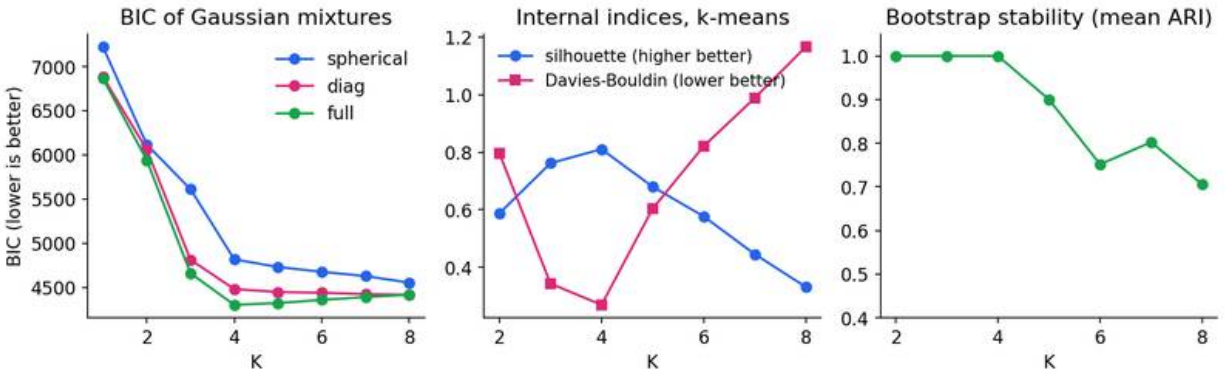


Figure 6 — Three ways to choose K on synthetic four-blob data. Left: BIC (sheared data). Middle: silhouette (higher is better) and Davies–Bouldin (lower is better) for k-means. Right: bootstrap stability is perfect for both $K = 3$ and $K = 4$, so stability alone cannot decide.

Cluster validity: is the clustering any good?

Plain version. Without labels, "good" has to be defined: tight and well separated (internal indices), agreeing with some trusted reference (external indices), or reproducible when the data are perturbed (stability). Theodoridis and Koutroumbas devote Chapter 16 to these questions [1]; PR 09 returns to validation in the supervised setting.

Internal indices

Silhouette [14]. For point i in cluster C, let $a(i)$ be its mean distance to the other points in C, and $b(i)$ the smallest mean distance from i to the points of any other cluster. Then

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \in [-1, 1].$$

$s(i)$ near 1 means i sits well inside its cluster; near 0, on a boundary; negative, probably in the wrong cluster. The mean silhouette over all points summarizes a partition, and the per-point

silhouette plot shows which clusters are weak.

Davies–Bouldin [15]. Let s_i be the average distance of cluster i's points to its centroid c_i, and $d_{ij} = \|c_i - c_j\|$. Then

$$\mathrm{DB} = \frac{1}{K}\sum_{i=1}^{K} \max_{j \neq i} \frac{s_i + s_j}{d_{ij}},$$

the average "worst-case similarity" of each cluster to another. Lower is better.

Calinski–Harabasz [16]. With between-cluster scatter $B = \sum_k N_k (c_k - \bar{x})(c_k - \bar{x})^\top$ and within-cluster scatter $W = \sum_k \sum_{n \in C_k} (x_n - c_k)(x_n - c_k)^\top$ ($\bar{x}$ is the overall mean),

$$\mathrm{CH} = \frac{\operatorname{tr}(B)/(K-1)}{\operatorname{tr}(W)/(N-K)},$$

a variance-ratio statistic. Higher is better.

All three reward compact, convex, well-separated clusters in the chosen geometry. They will rate k-means' wrong answer on the two moons higher than the correct curved answer. They measure agreement with an assumed cluster *shape*, not semantic correctness.

External indices

When reference labels exist (for benchmarking, or for a separate held-out annotation), compare partitions with permutation-invariant indices.

Adjusted Rand index (ARI) [17]. The Rand index counts the fraction of point pairs on which two partitions agree (both "same cluster" or both "different cluster"). ARI subtracts the value expected under random labelings with the same cluster sizes and rescales:

$$\mathrm{ARI} = \frac{\mathrm{RI} - \mathbb{E}[\mathrm{RI}]}{\max \mathrm{RI} - \mathbb{E}[\mathrm{RI}]},$$

so random partitions score about 0 and identical partitions (up to relabeling) score 1.

Normalized mutual information (NMI). Treat the two labelings U and V as random variables over points; then

$$\mathrm{NMI}(U, V) = \frac{I(U;V)}{\frac{1}{2}\left(H(U) + H(V)\right)},$$

with mutual information I and entropies H (the arithmetic-mean normalization shown is scikit-learn's default; other normalizations exist). Vinh, Epps and Bailey [18] showed that unadjusted NMI is biased upward when the number of clusters is large relative to N, and recommend chance-adjusted versions such as adjusted mutual information (AMI).

If the reference labels were used to tune the algorithm or K, they are no longer an independent check.

Stability

Refit the clustering on bootstrap samples or perturbed data and compare each new partition with the original on the *same* points, using ARI. A K that gives unstable partitions is suspect. But stability is necessary, not sufficient: merging two nearby blobs is perfectly reproducible (Figure 6, right, $K = 3$), and a stable partition can still reflect an irrelevant factor such as the camera or the lighting.

Code: validity indices and stability

```
import numpy as np
from sklearn.datasets import make_blobs
from sklearn.cluster import KMeans
from sklearn.metrics import (silhouette_score, davies_bouldin_score,
                             calinski_harabasz_score, adjusted_rand_score,
                             normalized_mutual_info_score)

X, truth = make_blobs(n_samples=600, centers=4, cluster_std=1.0, random_state=7)
print(" K  silhouette(+)  Davies-Bouldin(-)  Calinski-Harabasz(+)")
for K in range(2, 8):
    z = KMeans(K, n_init=10, random_state=0).fit_predict(X)
    print(f"{K:2d}  {silhouette_score(X, z):12.3f}  {davies_bouldin_score(X, z):17.3f}"
          f"  {calinski_harabasz_score(X, z):20.1f}")

z = KMeans(4, n_init=10, random_state=0).fit_predict(X)
print("external ARI:", round(adjusted_rand_score(truth, z), 3),
      " NMI:", round(normalized_mutual_info_score(truth, z), 3))
perm = np.array([2, 0, 3, 1])[z]                       # relabel clusters
print("ARI after relabelling:", round(adjusted_rand_score(truth, perm), 3))

# stability: refit on bootstrap samples, compare on the original points
rng = np.random.default_rng(1)
for K in [3, 4, 5, 6]:
    base = KMeans(K, n_init=10, random_state=0).fit(X)
    scores = []
    for b in range(10):
        idx = rng.integers(0, len(X), len(X))
        boot = KMeans(K, n_init=10, random_state=b).fit(X[idx])
        scores.append(adjusted_rand_score(base.predict(X), boot.predict(X)))
    print(f"K={K}: bootstrap stability (mean ARI) = {np.mean(scores):.3f}")
```

```
 K  silhouette(+)  Davies-Bouldin(-)  Calinski-Harabasz(+)
 2         0.588              0.796                850.9
 3         0.762              0.343               2235.5
 4         0.811              0.270               7641.8
 5         0.680              0.603               6341.0
 6         0.577              0.821               5620.2
 7         0.446              0.988               5304.4
external ARI: 1.0  NMI: 1.0
ARI after relabelling: 1.0
```

```
K=3: bootstrap stability (mean ARI) = 1.000
K=4: bootstrap stability (mean ARI) = 1.000
K=5: bootstrap stability (mean ARI) = 0.858
K=6: bootstrap stability (mean ARI) = 0.699
```

On these well-separated blobs every internal index agrees on $K = 4$, and the external indices confirm a perfect match regardless of how the clusters are numbered. Real data are rarely this kind: indices frequently disagree, and the disagreement is informative, because it tells you the data do not have one unambiguous cluster structure at the chosen scale.

Modern view

Plain version. The algorithms in this chapter are decades old and still everywhere. What changed is the *representation*: today we usually cluster learned embeddings rather than raw pixels or hand-made descriptors, and deep networks are sometimes trained *by* clustering.

Surveys worth reading

- **Xu and Wunsch, "Survey of clustering algorithms" (2005)** [19] is the classic broad survey from a pattern-recognition and neural-network viewpoint. It reviews proximity measures, then organizes algorithms into hierarchical, squared-error (k-means family), mixture-density, graph-theoretic, fuzzy, neural (self-organizing maps, adaptive resonance), kernel, sequential-data, large-scale and high-dimensional methods, and closes with cluster validity. It is a good map of the classical landscape.
- **Jain, "Data clustering: 50 years beyond K-means" (2010)** [20] is a shorter, opinionated overview. Its main messages remain current: there is no universally best algorithm; the representation and the definition of a cluster matter more than the algorithm; the number of clusters and validity are open questions; and side information (pairwise constraints, semi-supervised clustering) and clustering ensembles are valuable directions.
- **Ikotun et al., "K-means clustering algorithms: A comprehensive review, variants analysis, and advances in the era of big data" (2023)** [21] reviews the many k-means variants: initialization (including k-means++), automatic choice of K, handling of mixed or high-dimensional data, and scalable and distributed implementations.
- **Zhou et al., "A comprehensive survey on deep clustering: Taxonomy, challenges, and future directions" (ACM Computing Surveys, 2024)** [22] organizes deep clustering by how the representation-learning module and the clustering module interact: multi-stage methods (learn features, then cluster), iterative methods (alternate pseudo-labels and feature learning), generative methods (latent-variable models such as mixture-prior autoencoders), and simultaneous methods (optimize both jointly). It also discusses benchmarks, metrics and applications.
- **Ren et al., "Deep clustering: A comprehensive survey" (IEEE TNNLS, 2025)** [23] organizes the field by data source and prior knowledge: single-view, semi-supervised, multi-view and transfer deep clustering. Read together with [22], it shows how wide the deep-clustering design space has become.

From visual words to self-labeling

Two influential vision papers show how classical clustering became a training signal. **DeepCluster** [24] alternates between running k-means on the features of a convolutional network and training that network to predict the cluster assignments as pseudo-labels; repeated over the whole data set, this learns useful features from unlabeled images. **SCAN** [25] separates representation learning from clustering: it first learns features with a self-supervised pretext task, then mines each image's nearest neighbors in that feature space and trains a clustering head so that neighbors receive the same cluster, followed by a self-labeling refinement. Both are applications of the ideas in this chapter (k-means as quantization, neighborhoods as evidence of similarity) to learned representations; ML 05 explains where such representations come from.

State of practice (2023–2026) and open problems

In current practice, a common recipe for grouping images is: extract embeddings from a pretrained (often self-supervised) encoder, optionally reduce dimension, then run k-means, a Gaussian mixture or HDBSCAN, and validate on a labeled subset. The chapter's warnings apply unchanged:

- **The representation defines similarity.** Embeddings trained for one purpose may cluster by style, background or camera rather than by the category you care about.
- K **and validity remain open.** Information criteria assume the model family; internal indices assume a shape; external indices need labels. No survey above claims a general solution.
- **Evaluation of deep clustering usually relies on labeled benchmarks** and permutation-invariant scores such as clustering accuracy after optimally matching clusters to classes, NMI and ARI [22]. Those scores measure agreement with one chosen labeling, not the "true" structure.
- **Scale and imbalance.** Long-tailed data, many small clusters, and streaming data stress both classical and deep methods; [21] and [22] list these as active problems.

Key takeaways

- Clustering is defined by its notion of similarity. Features, scaling and distance are modeling decisions made before any algorithm runs.
- K-means (Lloyd) alternates nearest-center assignment and mean update; J decreases monotonically to a local minimum. k-means++ seeding gives $\mathbb{E}[J] \leq 8(\ln K + 2)J_{\mathrm{OPT}}$ and far fewer bad runs; use several restarts anyway.
- Linkage defines hierarchical clustering: single linkage chains along thin bridges, complete and Ward prefer compact groups. The dendrogram must still be cut.
- DBSCAN defines clusters as density-connected regions, finds arbitrary shapes and labels noise; ε comes from the k-distance curve, and HDBSCAN handles varying density.
- EM for Gaussian mixtures follows from the complete-data log-likelihood: E step responsibilities, M step weighted means and covariances, $\pi_k = N_k/N$ via a Lagrange multiplier. The ELBO

decomposition proves monotone improvement; singularities and label switching need explicit care; k-means is the hard, isotropic limit.
- BIC $= -2\ln\hat{L} + p\ln N$ and AIC $= -2\ln\hat{L} + 2p$ trade fit against parameter count; covariance type is part of the choice.
- Internal indices (silhouette, Davies–Bouldin, Calinski–Harabasz) reward an assumed geometry; external indices (ARI, NMI) need independent labels; stability is necessary but not sufficient.

Exercises

1. **Lloyd's algorithm never increases J.** Prove that both the assignment step and the update step cannot increase J, and use this to argue that the algorithm terminates. What goes wrong with the termination argument if ties are broken randomly?

   Hint

   The assignment step minimizes each term $\|x_n - \mu_{z_n}\|^2$ separately; the update step uses $\sum_{n\in C_k}\|x_n - \mu\|^2 = \sum_{n\in C_k}\|x_n - \bar{x}_k\|^2 + N_k\|\bar{x}_k - \mu\|^2$. There are at most K^N partitions. With random tie-breaking, two partitions of equal J could alternate forever; a fixed rule (for example, keep the current cluster on ties) prevents that.

2. **k-means++ by hand.** One-dimensional data are $\{0, 1, 2, 10, 11\}$ and $K = 2$. The first seed is 0. Compute the probability that each point becomes the second seed. Then compare with uniform sampling of the second seed.

   Hint

   D^2 values are $0, 1, 4, 100, 121$, summing to 226. So $P(10) = 100/226$ and $P(11) = 121/226$; together about 0.98. Under uniform sampling the second seed lands in the right-hand group with probability only $2/5$ (or $2/4$ if the first seed is excluded).

3. **The M step for a 1-D mixture.** For a one-dimensional two-component mixture with variances σ_1^2, σ_2^2, derive the M-step update of σ_k^2 from Q directly, without matrix calculus. Then show that if a component's responsibilities concentrate on one point x_n (that is, $r_{nk} \approx 1$ for that point and ≈ 0 for the others), the update drives σ_k^2 toward 0. What does scikit-learn's `reg_covar` do about it?

   Hint

   Differentiate $\sum_n r_{nk}[-\frac{1}{2}\ln\sigma_k^2 - (x_n-\mu_k)^2/(2\sigma_k^2)]$ with respect to σ_k^2 to get $\sigma_k^2 = \sum_n r_{nk}(x_n-\mu_k)^2/N_k$. With all weight on one point, $\mu_k = x_n$ and the sum is zero. `reg_covar` adds a small constant to the diagonal after each M step, so the variance stays bounded away from zero.

4. **Counting parameters.** For local descriptors with $D = 64$ and $K = 32$ components, count the free parameters of a GMM with full, diagonal and spherical covariances. With $N = 10^5$ descriptors, how large is the BIC penalty $p \ln N$ in each case?

Hint

Full: $31 + 32 \cdot 64 + 32 \cdot 64 \cdot 65/2 = 31 + 2048 + 66560 = 68{,}639$. Diagonal: $31 + 2048 + 2048 = 4{,}127$. Spherical: $31 + 2048 + 32 = 2{,}111$. Multiply by $\ln 10^5 \approx 11.51$. The full model has to improve $-2 \ln \hat{L}$ by roughly 7.4×10^5 over the diagonal one just to break even.

5. **When indices mislead.** Run k-means with $K = 2$ and DBSCAN on the two-moons data from the comparison code. Compute the silhouette score of each partition and their ARI against the generating labels. Explain the disagreement in terms of what each index measures.

Hint

Silhouette is computed from Euclidean distances and favors convex, separated groups, so the k-means cut usually scores higher than the two moons even though its ARI is about 0.26 versus 1.0 for DBSCAN. Internal indices encode a shape assumption; the "right" answer here is defined by connectivity.

6. **Stability is not truth.** Generate four Gaussian blobs where two are very close (for example centers $(0, 0)$ and $(1.2, 0)$ with standard deviation 0.5). Compute bootstrap stability for $K = 3$ and $K = 4$. Which is more stable? Which matches the generating process? What extra information would settle it?

Hint

Merging the close pair is very reproducible, so $K = 3$ can be as stable as or more stable than $K = 4$. The generating process has four groups, but whether they *should* be separated depends on the task. Domain knowledge, a labeled subset, or a density-based criterion such as BIC on a GMM gives additional evidence.

References

1. S. Theodoridis and K. Koutroumbas, *Pattern Recognition*, 4th ed., Academic Press, 2009, Ch. 11–16 (clustering basic concepts, sequential, hierarchical, function-optimization and graph-based algorithms, cluster validity). doi
2. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006, Ch. 9 (k-means, mixtures of Gaussians, EM, the general EM algorithm). book page
3. S. P. Lloyd, "Least squares quantization in PCM," *IEEE Transactions on Information Theory*, vol. 28, no. 2, pp. 129–137, 1982. doi
4. J. MacQueen, "Some methods for classification and analysis of multivariate observations," in *Proc. Fifth Berkeley Symposium on Mathematical Statistics and Probability*, vol. 1, pp. 281–297, 1967. Project Euclid

5. D. Arthur and S. Vassilvitskii, “k-means++: The advantages of careful seeding,” in *Proc. 18th Annual ACM-SIAM Symposium on Discrete Algorithms (SODA)*, 2007, pp. 1027–1035. pdf
6. D. Sculley, “Web-scale k-means clustering,” in *Proc. 19th International Conference on World Wide Web (WWW)*, 2010, pp. 1177–1178. doi
7. J. H. Ward, “Hierarchical grouping to optimize an objective function,” *Journal of the American Statistical Association*, vol. 58, pp. 236–244, 1963. doi
8. M. Ester, H.-P. Kriegel, J. Sander and X. Xu, “A density-based algorithm for discovering clusters in large spatial databases with noise,” in *Proc. 2nd International Conference on Knowledge Discovery and Data Mining (KDD)*, 1996. pdf
9. R. J. G. B. Campello, D. Moulavi and J. Sander, “Density-based clustering based on hierarchical density estimates,” in *Advances in Knowledge Discovery and Data Mining (PAKDD)*, LNCS, 2013, pp. 160–172. doi
10. U. von Luxburg, “A tutorial on spectral clustering,” *Statistics and Computing*, vol. 17, pp. 395–416, 2007. doi
11. A. P. Dempster, N. M. Laird and D. B. Rubin, “Maximum likelihood from incomplete data via the EM algorithm,” *Journal of the Royal Statistical Society: Series B*, vol. 39, no. 1, pp. 1–22, 1977. doi
12. G. Schwarz, “Estimating the dimension of a model,” *The Annals of Statistics*, vol. 6, no. 2, 1978. doi
13. H. Akaike, “A new look at the statistical model identification,” *IEEE Transactions on Automatic Control*, vol. 19, no. 6, pp. 716–723, 1974. doi
14. P. J. Rousseeuw, “Silhouettes: A graphical aid to the interpretation and validation of cluster analysis,” *Journal of Computational and Applied Mathematics*, vol. 20, pp. 53–65, 1987. doi
15. D. L. Davies and D. W. Bouldin, “A cluster separation measure,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-1, no. 2, pp. 224–227, 1979. doi
16. T. Caliński and J. Harabasz, “A dendrite method for cluster analysis,” *Communications in Statistics*, vol. 3, no. 1, pp. 1–27, 1974. doi
17. L. Hubert and P. Arabie, “Comparing partitions,” *Journal of Classification*, vol. 2, pp. 193–218, 1985. doi
18. N. X. Vinh, J. Epps and J. Bailey, “Information theoretic measures for clusterings comparison: Variants, properties, normalization and correction for chance,” *Journal of Machine Learning Research*, vol. 11, pp. 2837–2854, 2010. JMLR
19. R. Xu and D. Wunsch II, “Survey of clustering algorithms,” *IEEE Transactions on Neural Networks*, vol. 16, no. 3, pp. 645–678, 2005. doi
20. A. K. Jain, “Data clustering: 50 years beyond K-means,” *Pattern Recognition Letters*, vol. 31, no. 8, pp. 651–666, 2010. doi
21. A. M. Ikotun, A. E. Ezugwu, L. Abualigah, B. Abuhaija and J. Heming, “K-means clustering algorithms: A comprehensive review, variants analysis, and advances in the era of big data,” *Information Sciences*, vol. 622, pp. 178–210, 2023. doi
22. S. Zhou, H. Xu, Z. Zheng, J. Chen, Z. Li, J. Bu, J. Wu, X. Wang, W. Zhu and M. Ester, “A comprehensive survey on deep clustering: Taxonomy, challenges, and future directions,” *ACM Computing Surveys*, vol. 57, no. 3, pp. 1–38, 2024 (arXiv:2206.07579). doi
23. Y. Ren, J. Pu, Z. Yang, J. Xu, G. Li, X. Pu, P. S. Yu and L. He, “Deep clustering: A comprehensive survey,” *IEEE Transactions on Neural Networks and Learning Systems*, vol. 36, no. 4, pp. 5858–5878,

2025 (arXiv:2210.04142). doi
24. M. Caron, P. Bojanowski, A. Joulin and M. Douze, “Deep clustering for unsupervised learning of visual features,” in *Proc. European Conference on Computer Vision* (*ECCV*), LNCS, 2018, pp. 139–156. doi
25. W. Van Gansbeke, S. Vandenhende, S. Georgoulis, M. Proesmans and L. Van Gool, “SCAN: Learning to classify images without labels,” in *Proc. European Conference on Computer Vision* (*ECCV*), LNCS, 2020, pp. 268–285. doi

PR 08 • Approximate Inference and Bayesian Model Complexity

Why posterior integrals are hard, and the two big families of answers: sampling (Monte Carlo, importance sampling, Metropolis–Hastings, Gibbs, HMC) and optimization (variational inference, Laplace, EP), with links to VAEs and diffusion models.

Online edition: https://blog.shyandram.dev/en/learn/pattern-recognition/pr-08-approximate-inference/

What you'll learn

- Why Bayesian posteriors are easy to *evaluate* up to a constant but hard to *integrate*, and why grids fail beyond a handful of dimensions.
- How plain Monte Carlo, rejection sampling and importance sampling turn integrals into averages, and how to tell when their weights have collapsed.
- How Markov chain Monte Carlo works: detailed balance, Metropolis–Hastings (including the asymmetric-proposal correction), Gibbs sampling, Hamiltonian Monte Carlo, and the diagnostics ($\hat{R}$, effective sample size) that keep you honest.
- How variational inference turns integration into optimization: the evidence lower bound (ELBO) derived two ways, mean-field coordinate ascent, reverse versus forward KL, and stochastic gradients with the reparameterization trick.
- Where Laplace approximation and expectation propagation fit, how model evidence penalizes complexity, and how the same ideas power VAEs and diffusion models.

The big picture

Earlier chapters fitted one "best" parameter setting: maximum likelihood, MAP, or EM for mixtures. A Bayesian recognizer asks a bigger question. After seeing the data, *how plausible is every parameter setting*, and what do they predict *on average*? Answering it requires integrals over all unknowns. For almost every useful model those integrals have no closed form. This chapter is about the two families of workarounds: **draw samples** that behave like the posterior, or **fit a simpler distribution** that sits close to it.

> **Explain like I'm five**
>
> Imagine a huge dark field with treasure buried everywhere in different amounts, and you want to know the *total* treasure. Finding the single richest spot is easy: walk uphill. That is optimization. Knowing the total is harder. One way is to walk around randomly but cleverly, spending more time where treasure is rich, and count what you see: that is **sampling**. Another way is to lay a simple, smooth blanket over the field shaped as much like the treasure map as possible, then measure the

blanket instead: that is **variational inference**. Walking is slow but honest. The blanket is fast but can hide bumps it was not shaped to show.

Why it matters for recognition: uncertainty tells a system when to abstain (see PR 01), integrated evidence compares models without a held-out set, and the machinery of this chapter, especially the ELBO and Langevin-style sampling, is exactly what modern generative models use. We follow Bishop's chapters on approximate inference and sampling [1]; Theodoridis and Koutroumbas supply the estimation and optimization background (Chapter 2 and Appendix C) rather than a one-to-one treatment of these methods [2].

Why exact inference is hard

Plain version. Bayes' rule says the posterior is "prior times likelihood, divided by a normalizing number". The top is usually easy to compute for any single guess. The bottom is a sum over *every possible* guess, and that is where the trouble starts.

Precise version. Collect all unknowns, latent variables and parameters, into z, and write D for the observed data. Then

$$p(z \mid D) = \frac{p(D, z)}{p(D)}, \qquad p(D) = \int p(D, z)\, dz,$$

where $p(D, z) = p(D \mid z)\, p(z)$ is the joint density (likelihood times prior) and $p(D)$ is the **evidence** or marginal likelihood. Almost everything we want is an expectation under the posterior:

$$\mathbb{E}_{p(z|D)}[f(z)] = \int f(z)\, p(z \mid D)\, dz,$$

for example the posterior mean ($f(z) = z$), a predictive probability ($f(z) = p(y^* \mid x^*, z)$) or the probability that a parameter is positive ($f(z) = \mathbf{1}[z > 0]$). We write $\tilde{p}(z) = p(D, z)$ for the **unnormalized** posterior: it is cheap to evaluate pointwise, but its normalizer $Z = p(D)$ is unknown.

Three things make the integral hard:

1. **Dimension.** A grid with G points per axis needs G^d evaluations in d dimensions. For $G = 201$ and $d = 50$ that is about 10^{115}, far beyond any computer.
2. **Concentration.** With a lot of data the posterior occupies a tiny region of a huge space. Random guesses almost never land in it.
3. **Discrete structure.** For a mixture with K components and N points, the latent assignments have K^N configurations; summing over them exactly is hopeless even when each term is simple.

MAP estimation sidesteps all of this by maximizing $\tilde{p}$, but a maximum is not an integral. The highest point of the posterior can sit in a narrow spike that carries little mass, and a single point carries no information about spread. EM, from PR 07, sits in between: it integrates over latent assignments in its E step (which is tractable for Gaussian mixtures) but still returns a point estimate of the parameters.

A worked example: Bayesian logistic regression

Take a two-class problem with inputs $x_n \in \mathbb{R}^2$ (a bias feature and one measurement), labels $y_n \in \{0, 1\}$, weights w and the model

$$p(y_n = 1 \mid x_n, w) = \sigma(w^\top x_n), \qquad \sigma(s) = \frac{1}{1 + e^{-s}}, \qquad p(w) = \mathcal{N}(0, 4I).$$

The log joint is $\sum_n \left[y_n w^\top x_n - \log(1 + e^{w^\top x_n}) \right] - \frac{1}{8}\|w\|^2$ plus a constant. The Gaussian prior is not conjugate to the logistic likelihood, so the posterior is *not* Gaussian and its normalizer has no formula. With only two weights we can still brute-force it on a grid, which gives us a ground truth to compare every approximation against.

```
import numpy as np

rng = np.random.default_rng(0)
n = 40
X = np.c_[np.ones(n), rng.normal(size=n)]
w_true = np.array([-0.5, 1.5])
y = rng.random(n) < 1 / (1 + np.exp(-X @ w_true))

def log_joint(w):                       # log p(y | X, w) + log p(w), w ~ N(0, 2^2 I)
    s = X @ w
    return np.sum(y * s - np.logaddexp(0, s)) - 0.5 * np.sum(w**2) / 4.0

# Brute-force normalization on a grid: fine in 2-D, hopeless in 50-D.
g = np.linspace(-4, 4, 201)
W0, W1 = np.meshgrid(g, g, indexing="ij")
logp = np.array([[log_joint(np.array([a, b])) for b in g] for a in g])
cell = (g[1] - g[0])**2
log_evidence = np.log(np.sum(np.exp(logp - logp.max())) * cell) + logp.max()
post = np.exp(logp - log_evidence)
mean = np.array([np.sum(W0 * post), np.sum(W1 * post)]) * cell
sd = np.sqrt(np.array([np.sum((W0 - mean[0])**2 * post),
                       np.sum((W1 - mean[1])**2 * post)]) * cell)
print("log evidence:", round(log_evidence, 3))
print("posterior mean:", mean.round(3), " sd:", sd.round(3))
print("grid points for 201 per axis in 50-D:", f"{201.0**50:.2e}")
```

It prints a log evidence of -22.117, a posterior mean of $(-0.303, 1.631)$ and standard deviations of $(0.366, 0.578)$. Note the subtraction of `logp.max()` before exponentiating: log densities of real data are large negative numbers, and working in log space (the "log-sum-exp" trick) is the first habit of every practitioner. We will reuse this model to judge the Laplace and variational approximations later in the chapter.

Monte Carlo: integrals as averages

Plain version. If you can draw random examples from a distribution, the average of a quantity over those examples estimates its expected value. More draws give a better estimate, and the error shrinks in a predictable way that does not depend on the dimension.

Precise version. Given independent draws $z^{(1)}, \dots, z^{(S)} \sim p$, the Monte Carlo estimator of $I = \mathbb{E}_p[f(z)]$ is

$$\hat{I}_S = \frac{1}{S}\sum_{s=1}^{S} f(z^{(s)}), \qquad \mathbb{E}[\hat{I}_S] = I, \qquad \mathrm{Var}[\hat{I}_S] = \frac{\mathrm{Var}_p[f(z)]}{S}.$$

Here S is the number of samples and $\mathrm{Var}_p[f]$ is the variance of f under p. The estimator is **unbiased**, and its standard error falls as $1/\sqrt{S}$: to halve the error you need four times as many samples. The striking part is that d does not appear. Monte Carlo replaces an exponential-in-dimension grid by a cost that depends on the variance of f.

The catch is the assumption "draws from p". For a posterior we usually cannot sample directly. The rest of the sampling half of this chapter is about manufacturing such draws, or correcting for draws from the wrong distribution.

Rejection sampling

Suppose we can sample from a simple **proposal** $q(z)$ and know a constant M with $M\,q(z) \geq \tilde{p}(z)$ for all z. Draw $z \sim q$ and $u \sim \mathrm{Uniform}(0,1)$, and keep z if $u < \tilde{p}(z)/(Mq(z))$. The kept points are exact draws from p, because the accepted region under the envelope Mq has exactly the shape of $\tilde{p}$. The acceptance probability is Z/M, where $Z = \int \tilde{p}$. In high dimensions a usable envelope must be enormous compared with the target, so almost everything is rejected. Rejection sampling is a building block for one-dimensional conditionals (for example inside Gibbs samplers), not a general posterior engine [1].

Importance sampling

Plain version. Draw from a convenient distribution, then correct each draw with a weight that says how much more (or less) likely it is under the distribution you actually care about. Works well when the convenient distribution covers the important region; fails quietly when it does not.

Precise version. For any q with $q(z) > 0$ wherever $p(z)f(z) \neq 0$,

$$\mathbb{E}_p[f(z)] = \int f(z)\,\frac{p(z)}{q(z)}\,q(z)\,dz \approx \frac{1}{S}\sum_{s=1}^{S} w_s f(z^{(s)}), \qquad w_s = \frac{p(z^{(s)})}{q(z^{(s)})}, \quad z^{(s)} \sim q.$$

The ratio w_s is the **importance weight**. When only $\tilde{p}$ is available, the unknown Z cancels if we normalize the weights:

$$\hat{I}_{\mathrm{SN}} = \sum_{s=1}^{S} \bar{w}_s f(z^{(s)}), \qquad \bar{w}_s = \frac{\tilde{w}_s}{\sum_t \tilde{w}_t}, \qquad \tilde{w}_s = \frac{\tilde{p}(z^{(s)})}{q(z^{(s)})}.$$

This **self-normalized** estimator is slightly biased for finite S but consistent as $S \to \infty$. As a by-product, $\frac{1}{S}\sum_s \tilde{w}_s$ is an unbiased estimate of the evidence Z, which is how importance sampling is used for model comparison.

The quality of the estimate depends on how uneven the weights are. A widely used summary is the **effective sample size** of the weights,

$$\mathrm{ESS} = \frac{1}{\sum_s \bar{w}_s^2},$$

which equals S when all weights are equal and 1 when one weight takes everything. Two failure modes matter:

- **Proposal too narrow or misplaced.** Regions where p has mass but q is tiny produce rare, gigantic weights. The estimate is dominated by a handful of draws and its variance can even be infinite.
- **Light tails.** The ratio p/q must stay bounded in the tails. A rule of thumb is to make q *heavier-tailed* than p (a Student-t instead of a Gaussian, or a wider Gaussian).

```
import numpy as np
from scipy.stats import norm

rng = np.random.default_rng(1)
N = 10_000
exact = norm.sf(3.0)                       # P(X > 3) for X ~ N(0, 1)

# Plain Monte Carlo: most samples never reach the tail.
x = rng.normal(0, 1, N)
plain = np.mean(x > 3)

# Importance sampling with a proposal centered on the tail, q = N(3, 1).
z = rng.normal(3, 1, N)
w = norm.pdf(z) / norm.pdf(z, loc=3)       # p(z) / q(z)
is_est = np.mean(w * (z > 3))
print(f"exact {exact:.6f}  plain MC {plain:.6f}  importance {is_est:.6f}")

# Self-normalized IS for an unnormalized target, with effective sample size.
def log_p_tilde(t):                        # N(1.5, 0.7^2) without its constant
    return -0.5 * ((t - 1.5) / 0.7) ** 2

for scale in [1.0, 3.0, 0.3]:
    t = rng.normal(0, scale, N)
    log_w = log_p_tilde(t) - norm.logpdf(t, scale=scale)
    wn = np.exp(log_w - log_w.max())
    wn /= wn.sum()
    ess = 1.0 / np.sum(wn**2)
    print(f"proposal N(0,{scale}^2): mean {np.sum(wn * t):.3f}  ESS {ess:7.1f} of {N}")
```

The rare-event estimate shows why importance sampling exists: the exact tail probability is 0.001350; plain Monte Carlo with 10,000 draws gives 0.001900 (only 19 draws landed in the tail), while the shifted proposal gives 0.001372. In the second part the target mean is 1.5. The wide proposals give 1.509 and 1.492 with ESS of about 1,870 and 2,830. The narrow proposal $\mathcal{N}(0, 0.3^2)$ gives 0.921 with an ESS of 13, a confident-looking but wrong answer. A tiny ESS is the alarm bell.

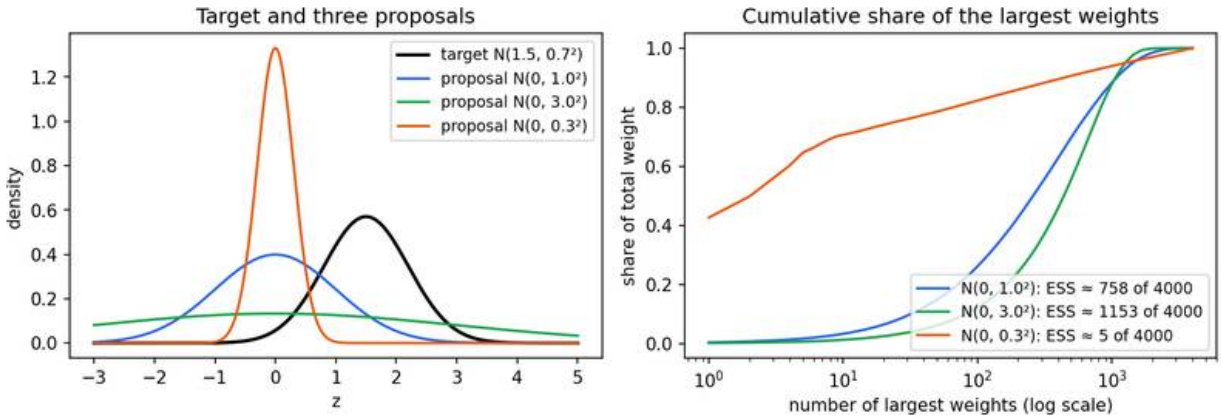


Figure 1 — Importance sampling with 4,000 draws per proposal. The narrow proposal (orange) barely reaches the target, so its largest single weight already holds over 40% of the total and the ESS is about 5. The wide proposals spread the weight over hundreds of draws. Computed by this tutorial's figure script.

Importance sampling stays important even when it is not the main engine: it re-weights samples from an old model to a new one, it estimates evidence, and it powers **sequential Monte Carlo** (particle filters), where weights are refreshed and resampled step by step as data arrive.

Markov chain Monte Carlo

Plain version. Instead of drawing independent samples, take a random walk whose rules are designed so that, in the long run, the fraction of time spent in each region equals the posterior probability of that region. Then time averages along the walk estimate posterior expectations. Successive steps are correlated, so a long walk is worth fewer independent draws than its length suggests.

Precise version. A Markov chain with transition kernel $T(z' \mid z)$ has p as a **stationary** (invariant) distribution if

$$\int T(z' \mid z)\, p(z)\, dz = p(z').$$

A simple sufficient condition is **detailed balance** (reversibility):

$$p(z)\, T(z' \mid z) = p(z')\, T(z \mid z') \quad \text{for all } z, z'.$$

Integrating both sides over z gives stationarity. If the chain can also reach every region (irreducibility) and does not cycle (aperiodicity), the distribution of the state converges to p from

any starting point, and the ergodic average $\frac{1}{S}\sum_s f(z^{(s)})$ converges to $\mathbb{E}_p[f]$. Importantly, detailed balance involves only *ratios* $p(z')/p(z)$, so the unknown normalizer cancels.

Metropolis–Hastings

The Metropolis algorithm [3], generalized by Hastings [4], builds such a kernel from any proposal $q(z' \mid z)$:

1. From the current state z, propose $z' \sim q(z' \mid z)$.
2. Accept with probability

$$A(z', z) = \min\left(1,\ \frac{\tilde{p}(z')\, q(z \mid z')}{\tilde{p}(z)\, q(z' \mid z)}\right).$$

3. If accepted, move to z'; otherwise **stay at** z **and record** z **again**.

Here $q(z \mid z')$ is the density of proposing the reverse move. The factor $q(z \mid z')/q(z' \mid z)$ is the **Hastings correction**: it cancels any bias the proposal has toward certain directions. For a symmetric proposal such as a Gaussian random walk, $q(z' \mid z) = q(z \mid z')$ and the rule reduces to Metropolis's original $\min(1, \tilde{p}(z')/\tilde{p}(z))$.

To see why it works, take $z' \neq z$ and suppose $\tilde{p}(z')q(z \mid z') \leq \tilde{p}(z)q(z' \mid z)$. Then $p(z)T(z' \mid z) = p(z)\, q(z' \mid z)\, \frac{p(z')q(z|z')}{p(z)q(z'|z)} = p(z')q(z \mid z')$, and in the reverse direction the acceptance is 1, so $p(z')T(z \mid z') = p(z')q(z \mid z')$. Both sides match: detailed balance holds.

Forgetting step 3 (only recording accepted states) is a classic bug: it over-represents regions the chain moves through easily and biases every estimate.

A Gaussian target with known moments lets us check a random-walk chain. The assertions are deliberately loose: a numerical sanity check, not a convergence proof.

```
import numpy as np

rng = np.random.default_rng(12)
def log_target(x):
    return -0.5 * ((x - 1.5) / 0.7)**2
x, accepted = -3.0, 0
samples = []
for step in range(18000):
    proposal = x + rng.normal(0, 0.8)
    if np.log(rng.random()) < min(0.0, log_target(proposal) - log_target(x)):
        x = proposal
        accepted += 1
    if step >= 2000:                      # discard warm-up, keep repeats
        samples.append(x)
samples = np.asarray(samples)
rho1 = np.corrcoef(samples[:-1], samples[1:])[0, 1]
print("mean, std:", samples.mean(), samples.std())
print("acceptance:", accepted / 18000, "lag-one correlation:", rho1)
```

```
assert abs(samples.mean() - 1.5) < 0.12
assert abs(samples.std() - 0.7) < 0.12
```

The chain recovers mean 1.50 and standard deviation 0.69, with acceptance 0.67 and lag-one correlation 0.74. The step size is the main tuning knob, and acceptance alone does not tell you whether it is good. Tiny steps are almost always accepted but crawl; huge steps are mostly rejected and the chain sits still. Both produce strongly correlated samples.

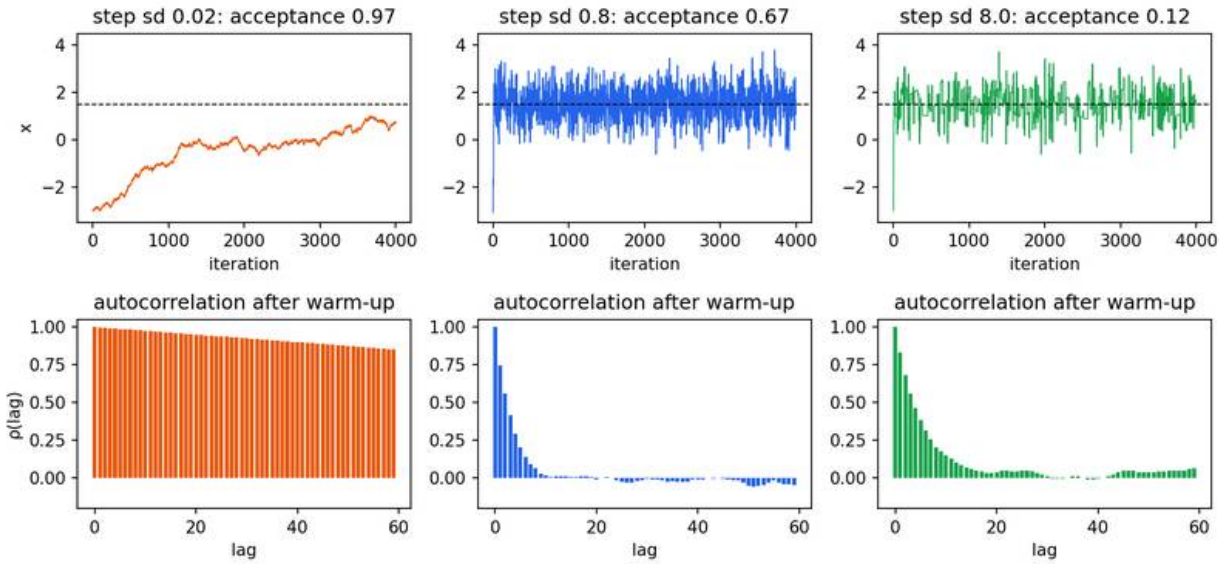


Figure 2 — Random-walk Metropolis on $N(1.5, 0.7^2)$ started at -3. With step 0.02 (acceptance 0.97) the chain has not even reached the target after 4,000 steps and its autocorrelation barely decays. Step 0.8 mixes well. Step 8 (acceptance 0.12) gets stuck for long flat stretches. Computed by this tutorial's figure script.

Gibbs sampling

Plain version. Update one variable (or block) at a time by drawing it from its distribution *given all the others*. If those conditional distributions are easy, as they often are in conjugate models, every move is accepted.

Precise version. Gibbs sampling, introduced for image restoration with Markov random fields by Geman and Geman [5], cycles through blocks $j = 1, \dots, J$ and draws

$$z_j \sim p(z_j \mid z_{-j}, D),$$

where z_{-j} denotes all blocks except j. It is a special case of Metropolis–Hastings with proposal $q(z' \mid z) = p(z'_j \mid z_{-j})$ and $z'_{-j} = z_{-j}$; plugging into the acceptance ratio gives exactly 1 [1]. In a Gaussian mixture, for example, one alternates between sampling each point's component label from its responsibilities and sampling the component parameters from their conjugate posteriors: a stochastic cousin of EM.

For a zero-mean bivariate Gaussian with unit variances and correlation ρ, the conditionals are $z_1 \mid z_2 \sim \mathcal{N}(\rho z_2,\, 1-\rho^2)$ and symmetrically for z_2. The code below runs four chains from dispersed corners and computes the **split-$\hat{R}$** convergence diagnostic described in the next subsection.

```
import numpy as np

rng = np.random.default_rng(3)
rho = 0.85                      # target: zero-mean bivariate Gaussian, unit variances
sd = np.sqrt(1 - rho**2)        # conditional standard deviation

def gibbs(n_steps, start):
    z = np.array(start, dtype=float)
    out = np.empty((n_steps, 2))
    for t in range(n_steps):
        z[0] = rng.normal(rho * z[1], sd)   # z1 | z2 ~ N(rho z2, 1 - rho^2)
        z[1] = rng.normal(rho * z[0], sd)   # z2 | z1 ~ N(rho z1, 1 - rho^2)
        out[t] = z
    return out

def split_rhat(chains):         # chains: (m, n) draws of one scalar quantity
    m, n = chains.shape
    halves = chains[:, : n // 2 * 2].reshape(2 * m, n // 2)
    W = halves.var(axis=1, ddof=1).mean()          # within-chain variance
    B = (n // 2) * halves.mean(axis=1).var(ddof=1) # between-chain variance
    var_plus = (n // 2 - 1) / (n // 2) * W + B / (n // 2)
    return np.sqrt(var_plus / W)

starts = [(-4, -4), (4, 4), (-4, 4), (4, -4)]
chains = np.stack([gibbs(3000, s)[500:] for s in starts])   # drop warm-up
draws = chains.reshape(-1, 2)
print("sample correlation:", np.corrcoef(draws.T)[0, 1].round(3))
print("sample variances:", draws.var(axis=0).round(3))
lag1 = np.mean([np.corrcoef(c[:-1, 0], c[1:, 0])[0, 1] for c in chains])
print("lag-1 autocorrelation of z1:", lag1.round(3), " (theory rho^2 =", rho**2, ")")
print("split R-hat for z1:", split_rhat(chains[:, :, 0]).round(4))
```

Output: sample correlation 0.849, variances near 0.98, lag-one autocorrelation 0.717 (theory $\rho^2 = 0.7225$) and split-$\hat{R}$ 1.0009. The autocorrelation is the price of axis-aligned moves: when the target is a long diagonal ridge, each coordinate can only move a short distance before hitting the walls. As $\rho \to 1$ the chain slows to a crawl. Reparameterizing (rotating or whitening the variables) or updating correlated variables together as a block fixes this.

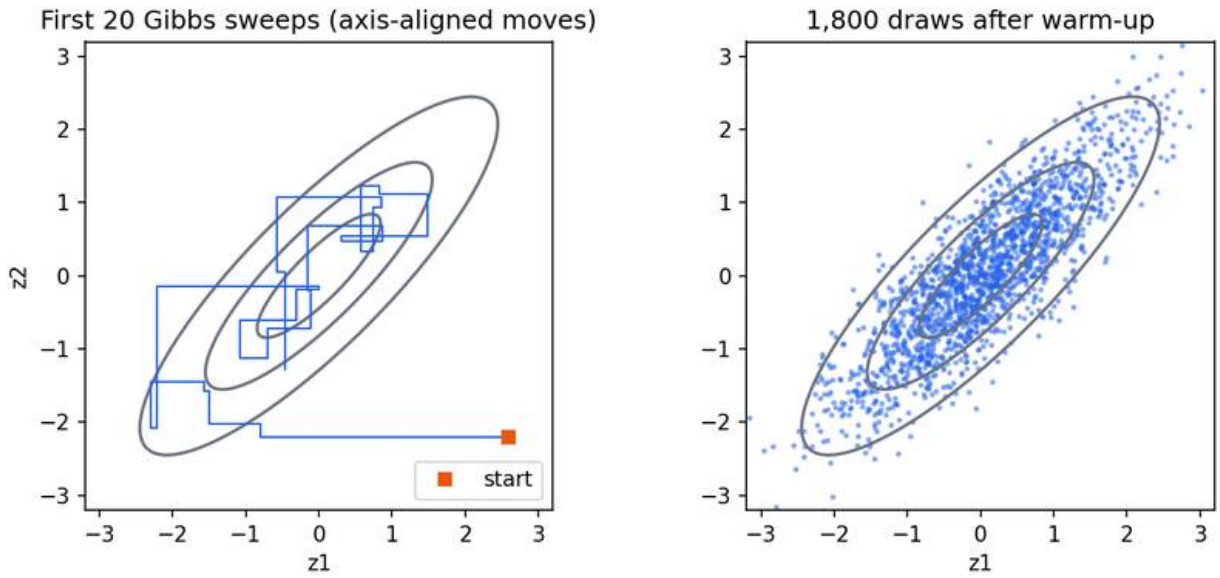


Figure 3 — Gibbs sampling on a Gaussian with correlation 0.85. Each half-step moves parallel to an axis (left), so progress along the diagonal ridge is slow, yet after warm-up the draws fill the target correctly (right). Computed by this tutorial's figure script.

Hamiltonian Monte Carlo

Plain version. Random walks waste effort going back and forth. Hamiltonian Monte Carlo (HMC) uses the *gradient* of the log posterior to glide along the distribution like a frictionless puck on a curved surface, making long, informed moves that are still accepted with high probability.

Precise version. Define potential energy $U(z) = -\log \tilde{p}(z)$, add an auxiliary momentum $r \sim \mathcal{N}(0, M)$ with mass matrix M, and form the Hamiltonian

$$H(z, r) = U(z) + \tfrac{1}{2} r^{\top} M^{-1} r.$$

Hamiltonian dynamics conserve H and preserve volume. In practice we simulate them with L **leapfrog** steps of size ϵ:

$$r \leftarrow r - \tfrac{\epsilon}{2} \nabla U(z), \qquad z \leftarrow z + \epsilon M^{-1} r, \qquad r \leftarrow r - \tfrac{\epsilon}{2} \nabla U(z).$$

The leapfrog integrator is reversible and volume-preserving but not exactly energy-conserving, so the endpoint (z', r') is accepted with probability $\min\left(1, \exp(H(z, r) - H(z', r'))\right)$, a Metropolis step on the joint space [6]. Because the trajectory follows the geometry of the target, HMC can traverse a correlated ridge that would trap Gibbs or a random walk. Its tuning parameters, ϵ and L, are delicate; the **No-U-Turn Sampler** (NUTS) chooses the trajectory length automatically by stopping when the path starts to double back, and adapts ϵ during warm-up [7]. Betancourt's conceptual introduction explains why HMC scales to high dimensions in terms of the **typical set**, the thin shell where posterior mass actually lives [8]. HMC needs differentiable, continuous targets; discrete latent variables must be summed out or handled by other moves.

A close relative is the **Langevin** update

$$z \leftarrow z + \tfrac{\epsilon}{2}\nabla \log \tilde{p}(z) + \sqrt{\epsilon}\,\xi, \qquad \xi \sim \mathcal{N}(0, I),$$

which is one leapfrog step with freshly drawn momentum. Without an accept/reject step it is only approximately correct for finite ϵ; with the Metropolis correction it becomes the Metropolis-adjusted Langevin algorithm. Keep this update in mind: it reappears in diffusion models.

Diagnostics: is the chain telling the truth?

No finite run *proves* convergence. Diagnostics can only reveal problems. Standard practice:

- **Run several chains from dispersed starting points** and discard an initial warm-up portion of each.
- **Compare chains with $\hat{R}$.** With m chains (split in halves) of length n, let W be the average within-chain variance and B/n the variance of the chain means. Then

$$\hat{R} = \sqrt{\frac{\frac{n-1}{n}W + \frac{1}{n}B}{W}}.$$

If all chains explore the same distribution, $\hat{R} \approx 1$; values noticeably above 1 mean the chains disagree. Vehtari et al. showed that the classic version misses heavy tails and differences in scale, and proposed a rank-normalized, folded split-$\hat{R}$ together with bulk and tail effective sample sizes; this is the version current software reports [9].

- **Report effective sample size.** For a stationary chain with lag-ℓ autocorrelation ρ_ℓ of the quantity of interest,

$$N_{\text{eff}} \approx \frac{N}{1 + 2\sum_{\ell \geq 1} \rho_\ell},$$

where the sum is truncated once estimated autocorrelations become noisy. The Monte Carlo standard error is roughly $\text{sd}/\sqrt{N_{\text{eff}}}$, not $\text{sd}/\sqrt{N}$. For the Gibbs chain above, $\rho_\ell = \rho^{2\ell}$, so $N_{\text{eff}}/N = (1-\rho^2)/(1+\rho^2) \approx 0.16$: 10,000 stored draws are worth about 1,600 independent ones.

- **Look at traces** and, for gradient-based samplers, at divergence warnings, which flag regions of high curvature the integrator cannot follow.

Different quantities mix at different speeds: a mean can look fine while a tail quantile has not converged. And a perfectly mixing chain still only describes the posterior of the *model you wrote down*; it says nothing about whether that model is right.

Variational inference

Plain version. Pick a family of simple distributions (for example, independent Gaussians), and search within it for the member closest to the posterior. Integration becomes optimization, which is fast and scales to huge datasets, at the cost of a bias you cannot remove by running longer.

Precise version. Choose a family $\mathcal{Q}$ of tractable densities $q(z)$, indexed by **variational parameters** ϕ. Variational inference (VI) [10] solves

$$q^* = \arg\min_{q \in \mathcal{Q}} D_{\mathrm{KL}}(q(z) \,\|\, p(z \mid D)), \qquad D_{\mathrm{KL}}(q\|p) = \mathbb{E}_q\left[\log \frac{q(z)}{p(z)}\right].$$

We cannot evaluate this KL directly because it contains the unknown $p(D)$. The trick is the ELBO.

The ELBO, derived two ways

Derivation 1: decomposition. Substitute $\log p(z \mid D) = \log p(D, z) - \log p(D)$ into the KL:

$$D_{\mathrm{KL}}(q\|p(\cdot \mid D)) = \mathbb{E}_q[\log q(z)] - \mathbb{E}_q[\log p(D, z)] + \log p(D).$$

Rearranging,

$$\log p(D) = \mathcal{L}(q) + D_{\mathrm{KL}}(q(z) \,\|\, p(z \mid D)), \qquad \mathcal{L}(q) = \mathbb{E}_q[\log p(D, z)] - \mathbb{E}_q[\log q(z)].$$

Because $\log p(D)$ does not depend on q, maximizing $\mathcal{L}$ is *exactly* minimizing the KL. Because KL is nonnegative, $\mathcal{L}(q) \le \log p(D)$: it is the **evidence lower bound**, ELBO.

Derivation 2: Jensen's inequality. Multiply and divide by q inside the evidence integral and use the concavity of log:

$$\log p(D) = \log \mathbb{E}_q\left[\frac{p(D, z)}{q(z)}\right] \ge \mathbb{E}_q\left[\log \frac{p(D, z)}{q(z)}\right] = \mathcal{L}(q).$$

The gap in Jensen's inequality is exactly the KL from derivation 1. A useful reading of the ELBO is $\mathcal{L} = \mathbb{E}_q[\log p(D \mid z)] - D_{\mathrm{KL}}(q(z)\|p(z))$: fit the data, but do not stray too far from the prior. EM is the special case where q over latent variables is set to the exact posterior in the E step (making the bound tight) and the bound is then maximized over parameters in the M step [1].

Mean field and coordinate ascent (CAVI)

The most common family is **mean field**: $q(z) = \prod_j q_j(z_j)$, independent factors over blocks of z. Holding all factors except q_j fixed and maximizing the ELBO over q_j gives

$$\log q_j^*(z_j) = \mathbb{E}_{q_{-j}}[\log p(D, z)] + \text{const},$$

where $\mathbb{E}_{q_{-j}}$ averages over all other factors and the constant normalizes q_j^*. (To derive it, write $\mathcal{L}$ as a function of q_j only; it equals $-D_{\mathrm{KL}}(q_j\|\tilde{q}_j)$ plus a constant, where $\log \tilde{q}_j = \mathbb{E}_{q_{-j}}[\log p(D, z)]$.) Cycling through j is **coordinate ascent variational inference** (CAVI). Each update cannot decrease

the ELBO, so CAVI converges, but only to a local optimum [11]. For conditionally conjugate models each q_j^* stays in the same family as the prior and the updates are closed-form.

Worked example: a Gaussian with unknown mean and precision. Observations $x_n \sim \mathcal{N}(\mu, \tau^{-1})$, prior $\mu \mid \tau \sim \mathcal{N}(\mu_0, (\lambda_0\tau)^{-1})$, $\tau \sim \mathrm{Gamma}(a_0, b_0)$. The exact posterior is known (Normal–Gamma), so we can grade the approximation. With $q(\mu,\tau) = q(\mu)q(\tau)$ the mean-field update rule gives $q(\mu) = \mathcal{N}(\mu_N, \lambda_N^{-1})$ and $q(\tau) = \mathrm{Gamma}(a_N, b_N)$ with [1]

$$\mu_N = \frac{\lambda_0\mu_0 + N\bar{x}}{\lambda_0 + N}, \qquad \lambda_N = (\lambda_0 + N)\,\mathbb{E}[\tau],$$
$$a_N = a_0 + \frac{N+1}{2}, \qquad b_N = b_0 + \tfrac{1}{2}\,\mathbb{E}_{\mu}\Big[\textstyle\sum_n (x_n - \mu)^2 + \lambda_0(\mu - \mu_0)^2\Big],$$

where $\bar{x}$ is the sample mean and $\mathbb{E}[\tau] = a_N/b_N$. The two factors depend on each other through $\mathbb{E}[\tau]$ and $\mathbb{E}[\mu]$, $\mathbb{E}[\mu^2]$, so we iterate.

```
import numpy as np

# Data x_n ~ N(mu, 1/tau); prior mu | tau ~ N(mu0, 1/(lam0 tau)), tau ~ Gamma(a0, b0).
rng = np.random.default_rng(4)
x = rng.normal(2.0, 0.5, size=30)
N, xbar = len(x), x.mean()
mu0, lam0, a0, b0 = 0.0, 1.0, 1.0, 1.0

# Mean field q(mu, tau) = q(mu) q(tau) with q(mu) = N(mu_N, 1/lam_N), q(tau) = Gamma(a_N,
 ↪  b_N).
E_tau = a0 / b0
for it in range(50):
    mu_N = (lam0 * mu0 + N * xbar) / (lam0 + N)
    lam_N = (lam0 + N) * E_tau
    a_N = a0 + (N + 1) / 2
    E_mu, E_mu2 = mu_N, mu_N**2 + 1 / lam_N
    b_N = b0 + 0.5 * (np.sum(x**2) - 2 * E_mu * np.sum(x) + N * E_mu2
                      + lam0 * (E_mu2 - 2 * mu0 * E_mu + mu0**2))
    E_tau = a_N / b_N

# Exact posterior is Normal-Gamma: compare the marginal moments of mu.
lam_post = lam0 + N
mu_post = (lam0 * mu0 + N * xbar) / lam_post
a_post = a0 + N / 2
b_post = b0 + 0.5 * (np.sum((x - xbar) ** 2) + lam0 * N * (xbar - mu0) ** 2 / lam_post)
var_mu_exact = b_post / ((a_post - 1) * lam_post)       # Student-t marginal variance
print("E[mu]   VI %.4f  exact %.4f" % (mu_N, mu_post))
print("Var[mu] VI %.5f  exact %.5f" % (1 / lam_N, var_mu_exact))
print("E[tau]  VI %.3f   exact %.3f" % (a_N / b_N, a_post / b_post))
```

The means agree (1.8668 both) and so does $\mathbb{E}[\tau]$ (2.146), but the variance of μ is 0.01503 under VI against 0.01603 exactly: about 6% too small. Under the exact posterior, μ and τ are dependent (the spread of μ scales with $1/\tau$) and its marginal is a heavier-tailed Student-t; a factorized Gaussian cannot represent either property.

This underestimation is systematic. For a Gaussian target with precision matrix Λ, the optimal mean-field factor q_j has variance $1/\Lambda_{jj}$, the *conditional* variance, which is never larger than the marginal variance. Figure 4 shows the extreme case.

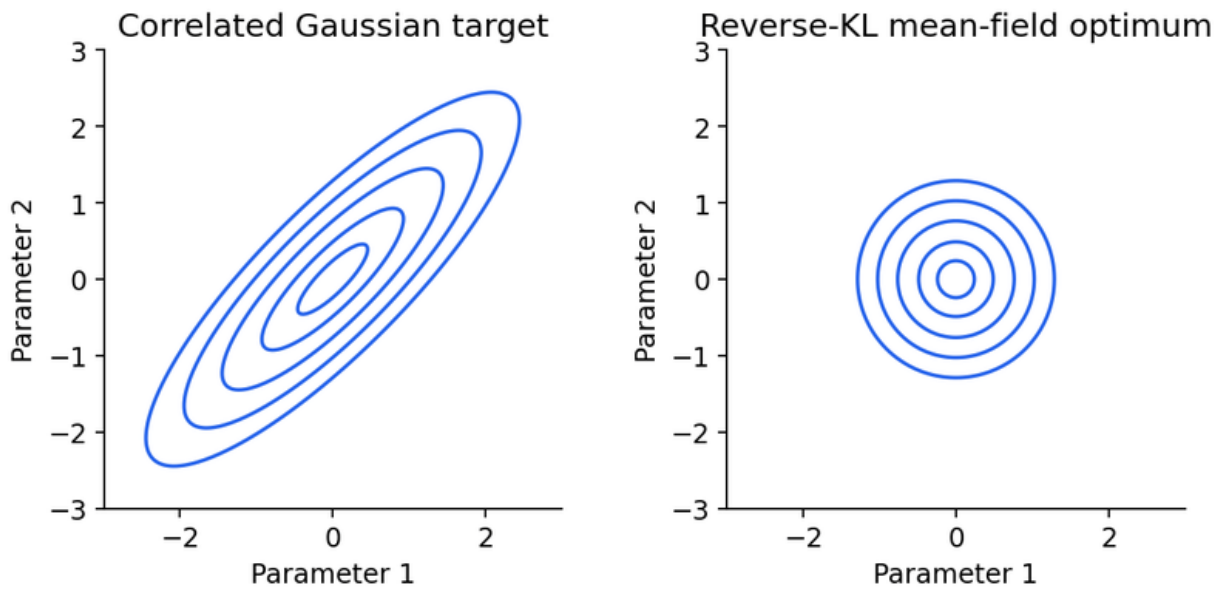


Figure 4 — Original analytical illustration with unit marginal target variances and correlation 0.85. The optimal Gaussian mean-field factors under reverse KL have variance $1 - 0.85^2 \approx 0.28$ each, illustrating lost correlation and underestimated marginal uncertainty. Contours show relative density, not measured model performance.

Which KL? Mode seeking versus mass covering

KL is not symmetric, and the direction changes the answer. **Reverse KL**, $D_{\mathrm{KL}}(q\|p)$, used by standard VI, takes expectations under q: it heavily penalizes putting q-mass where p is tiny, but does not mind if q ignores some of p. The result is **mode seeking** and **under-dispersed**. **Forward KL**, $D_{\mathrm{KL}}(p\|q)$, takes expectations under p: it heavily penalizes missing any region where p has mass, so q stretches to cover everything, which is **mass covering** and can put mass in the valleys between modes. Within an exponential family such as the Gaussians, minimizing forward KL is exactly **moment matching**: q takes the mean and covariance of p.

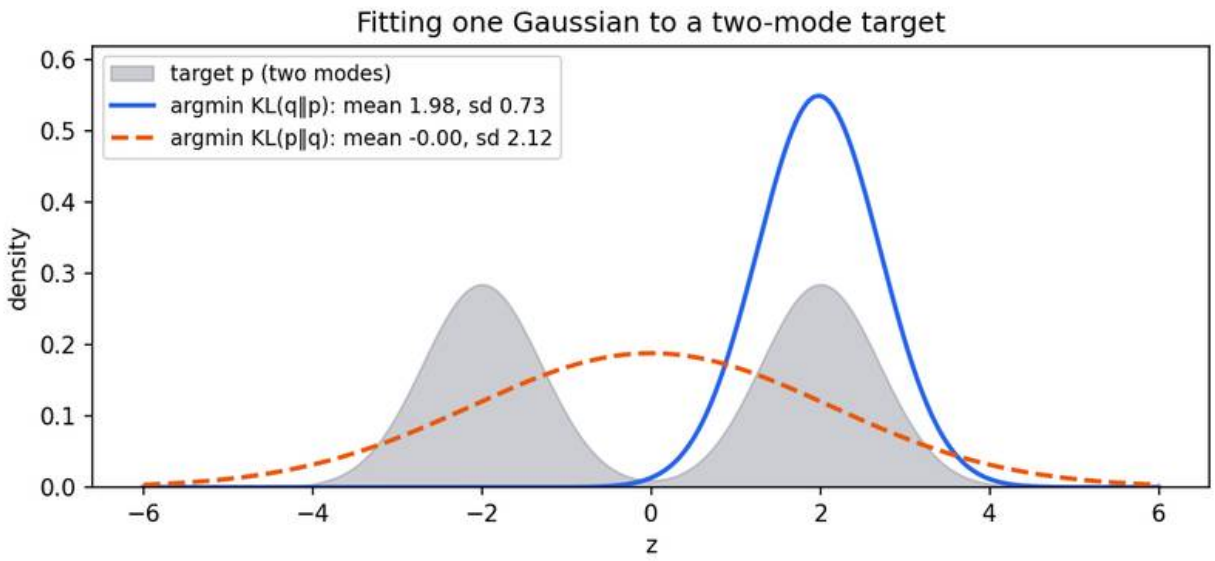


Figure 5 — One Gaussian fitted to an equal mixture of $N(-2, 0.7^2)$ and $N(2, 0.7^2)$. Minimizing reverse $KL(q\|p)$ locks onto one mode (blue; which mode depends on initialization). Minimizing forward $KL(p\|q)$ matches the mixture's mean and variance and covers both modes, placing its peak where the target has almost no mass (orange, dashed). Computed numerically by this tutorial's figure script.

Neither direction is "right". Reverse KL is tractable and gives sharp, usable answers but understates uncertainty. Forward KL is the objective behind expectation propagation and behind maximum-likelihood fitting of a model to data, and it is safer when missing a mode is costly.

Stochastic and black-box VI

CAVI needs conjugacy and passes over the whole dataset per update. Modern VI instead takes **stochastic gradients** of the ELBO, which works for almost any model and scales to large data by subsampling minibatches [11, 12]. There are two main gradient estimators for $\nabla_\phi \mathcal{L}(\phi)$:

- **Score-function (REINFORCE) estimator**, used by black-box VI [13]:

$$\nabla_\phi \mathcal{L} = \mathbb{E}_{q_\phi}\Big[\nabla_\phi \log q_\phi(z)\,\big(\log p(D, z) - \log q_\phi(z)\big)\Big].$$

It needs only evaluations of $\log p(D, z)$ and works for discrete z, but has high variance and needs control variates.

- **Reparameterization (pathwise) estimator.** Write $z = g_\phi(\varepsilon)$ with noise ε from a fixed distribution, for example $z = m + \sigma \odot \varepsilon$ with $\varepsilon \sim \mathcal{N}(0, I)$. Then

$$\nabla_\phi \mathcal{L} = \mathbb{E}_{\varepsilon}\Big[\nabla_\phi\big(\log p(D, g_\phi(\varepsilon)) - \log q_\phi(g_\phi(\varepsilon))\big)\Big].$$

It requires differentiable $\log p$ and continuous z but typically has much lower variance. This is the estimator the VAE made famous [15].

For a diagonal Gaussian with $\phi = (m, s)$, $\sigma = e^s$, the entropy term is $\sum_j s_j$ plus a constant, so its gradient with respect to s is simply 1. Applied to the logistic-regression posterior:

```
import numpy as np

rng = np.random.default_rng(0)
n = 40
X = np.c_[np.ones(n), rng.normal(size=n)]
y = (rng.random(n) < 1 / (1 + np.exp(-X @ np.array([-0.5, 1.5])))).astype(float)
prior_var = 4.0

def grad_log_joint(W):                  # W: (S, 2) -> gradients (S, 2)
    p = 1 / (1 + np.exp(-W @ X.T))      # (S, n)
    return (y - p) @ X - W / prior_var

# Laplace: Newton's method to the MAP, then invert the Hessian.
w = np.zeros(2)
for _ in range(25):
    p = 1 / (1 + np.exp(-X @ w))
    H = X.T @ (X * (p * (1 - p))[:, None]) + np.eye(2) / prior_var
    w = w + np.linalg.solve(H, grad_log_joint(w[None])[0])
print("Laplace mean:", w.round(3), " sd:", np.sqrt(np.diag(np.linalg.inv(H))).round(3))

# Mean-field Gaussian VI with reparameterization: w = m + exp(s) * eps.
m, s = np.zeros(2), np.full(2, -1.0)
lr, S = 0.02, 64
for step in range(3000):
    eps = rng.normal(size=(S, 2))
    W = m + np.exp(s) * eps
    g = grad_log_joint(W)
    grad_m = g.mean(axis=0)                                  # d ELBO / d m
    grad_s = (g * eps).mean(axis=0) * np.exp(s) + 1.0        # + entropy term
    m += lr * grad_m
    s += lr * grad_s
print("VI mean:     ", m.round(3), " sd:", np.exp(s).round(3))
```

Compare with the grid truth, mean $(-0.303, 1.631)$ and sd $(0.366, 0.578)$. The code prints a Laplace mean of $(-0.291, 1.500)$ with sd $(0.355, 0.556)$, and a VI mean of $(-0.303, 1.630)$ with sd $(0.358, 0.562)$. VI recovers the posterior mean almost exactly, because it averages over the skewed posterior; Laplace reports the *mode*, which differs from the mean when the posterior is skewed. Both slightly understate the spread. With 40 data points the posterior is nearly Gaussian, so the differences are small; with fewer points or separable classes they grow quickly.

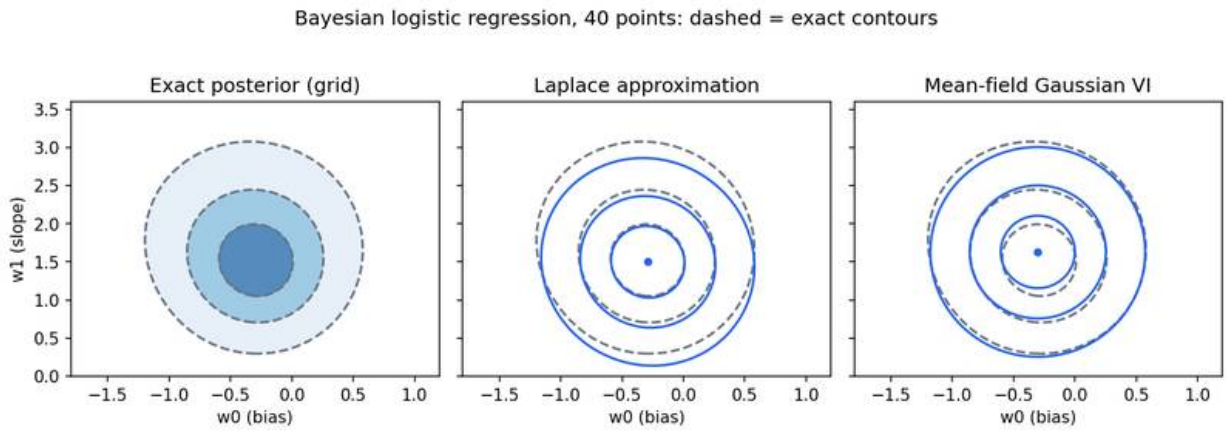


Figure 6 — Two-weight Bayesian logistic regression on 40 synthetic points. Dashed gray: exact posterior contours from the grid. Blue: Laplace (centered at the MAP, shifted toward smaller slope) and mean-field Gaussian VI (centered near the true posterior mean, slightly too narrow). Computed by this tutorial's figure script.

Laplace approximation and expectation propagation

Laplace: a Gaussian at the peak

Plain version. Find the highest point of the posterior and fit a Gaussian whose curvature matches the curvature there.

Precise version. Expand $\log \tilde{p}(z)$ to second order around a mode $\hat{z}$, where the gradient is zero. With $H = -\nabla^2 \log \tilde{p}(\hat{z})$ the Hessian of the negative log posterior,

$$q(z) = \mathcal{N}(z \mid \hat{z}, H^{-1}), \qquad \log p(D) \approx \log \tilde{p}(\hat{z}) + \frac{d}{2} \log 2\pi - \frac{1}{2} \log |H|,$$

where d is the dimension of z. The second formula is the **Laplace evidence approximation**; keeping only the terms that grow with N gives the BIC criterion used for choosing the number of mixture components in PR 07 [1]. Laplace is cheap (one optimization and one Hessian) and becomes accurate as data accumulate, because posteriors of regular models become increasingly Gaussian. It fails for skewed posteriors, for modes on a boundary (such as a variance near zero), when the Hessian is singular, and for multi-modal posteriors, where it describes only one mode. Modern "Laplace for deep networks" methods apply it to the last layer or with structured Hessian approximations to obtain cheap uncertainty for trained neural networks.

Expectation propagation

Expectation propagation (EP) [14] targets posteriors that factor as $p(z \mid D) \propto \prod_i f_i(z)$, for example one factor per data point. It approximates each factor by a simple **site** $\tilde{f}_i(z)$ from an exponential family, so $q(z) \propto \prod_i \tilde{f}_i(z)$, and refines sites one at a time:

1. Remove site i to form the **cavity** $q^{\setminus i}(z) \propto q(z)/\tilde{f}_i(z)$.
2. Form the **tilted** distribution $\hat{p}_i(z) \propto f_i(z)\, q^{\setminus i}(z)$, the cavity times the *exact* factor.
3. Set the new q to the member of the family that matches the moments of $\hat{p}_i$, i.e. minimizes the *forward* KL $D_{\mathrm{KL}}(\hat{p}_i \| q)$.
4. Update the site: $\tilde{f}_i \propto q^{\text{new}}/q^{\setminus i}$.

Because each step uses forward KL locally, EP tends to cover mass and often gives better-calibrated variances than mean-field VI, for example in Gaussian-process classification. In exchange, it has no global objective that is guaranteed to improve, can fail to converge, and may produce invalid (negative-variance) sites that need damping [1, 14].

Evidence and complexity: Bayesian Occam's razor

Plain version. A model that can explain *anything* spreads its prior bets thin, so it gets little credit for explaining the data you actually saw. A simpler model that bet on this kind of data gets more credit.

Precise version. The evidence of model $\mathcal{M}$ integrates the likelihood against the prior,

$$p(D \mid \mathcal{M}) = \int p(D \mid \theta, \mathcal{M})\, p(\theta \mid \mathcal{M})\, d\theta.$$

A flexible model can reach a high peak likelihood, yet if most of its prior mass sits on parameters that fit badly, the integral is small. The Laplace formula makes this explicit: the evidence is the peak value times an "Occam factor" that compares posterior volume with prior volume. Two cautions follow. First, the evidence depends on the prior, even on aspects that barely affect the posterior, such as how wide a vague prior is. Second, it answers "which model would have predicted this data set?", which is related to but not identical to "which model will predict *new* data best?". Held-out validation (PR 09) answers the second. Comparing ELBOs across models is also delicate: each ELBO undershoots its evidence by a different, unknown KL gap.

From the ELBO to VAEs and diffusion models

The machinery in this chapter is the backbone of two families of deep generative models covered in the ML track.

VAEs are amortized variational inference. A variational autoencoder models images x with a latent code z, a prior $p(z)$, and a neural decoder $p_\theta(x \mid z)$. Instead of optimizing separate variational parameters for every image, an encoder network outputs them: $q_\phi(z \mid x) = \mathcal{N}(\mu_\phi(x), \mathrm{diag}\, \sigma^2_\phi(x))$. This is **amortized inference**: one network learns to perform inference for any input. The per-image objective is exactly the ELBO,

$$\mathcal{L}(\theta, \phi; x) = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)] - D_{\mathrm{KL}}(q_\phi(z \mid x) \,\|\, p(z)),$$

trained jointly over θ and ϕ with the reparameterization gradient [15, 16]. Its known weaknesses are the ones above: an **amortization gap** (the encoder's answer is worse than per-image optimiza-

tion), the under-dispersion of reverse KL, and a Gaussian decoder that blurs. See ML 06 for a full treatment and code.

Diffusion models are trained with an ELBO and sampled with Langevin-style dynamics. A denoising diffusion probabilistic model fixes a forward process that gradually adds Gaussian noise, $x_0 \to x_1 \to \ldots \to x_T$, and learns a reverse process $p_\theta(x_{t-1} \mid x_t)$. The forward noisy states play the role of latent variables, with the forward process as a *fixed* variational posterior; the training objective is a variational bound on $-\log p_\theta(x_0)$, and the popular noise-prediction loss is a reweighted form of that bound [17]. The score-based view [18] instead learns $s_\theta(x, t) \approx \nabla_x \log p_t(x)$, the gradient of the log density of noisy data, and generates samples by integrating a reverse-time stochastic differential equation; its predictor–corrector samplers explicitly interleave Langevin MCMC steps that use s_θ in place of $\nabla \log \tilde{p}$. The only change from the Langevin update earlier in this chapter is that the score is *learned from data* rather than computed from a formula. See ML 07.

Choosing a method

Method	What it returns	Strengths	Main failure mode
Grid / quadrature	exact numbers	ground truth	exponential in dimension
Importance sampling	weighted samples, evidence	simple, parallel, gives $p(D)$	weight collapse when q misses p
Random-walk MH	correlated samples	needs only $\tilde{p}$	slow in high dimension; tuning
Gibbs	correlated samples	no tuning when conditionals are easy	slow on correlated variables
HMC / NUTS	correlated samples	scales to many continuous parameters	needs gradients; discrete variables; divergences
Mean-field VI	a distribution q	fast, scalable, minibatch-friendly	under-dispersion; local optima
Laplace	Gaussian at the mode	cheap; evidence estimate	skew, boundaries, multiple modes
EP	Gaussian / exponential family	often good variances	may not converge

A sensible workflow is to start with an optimizer (MAP or VI) for speed, check it against a careful sampler on a smaller version of the problem, and use simulation from known parameters to verify that the whole pipeline recovers what it should.

Modern view

Surveys worth reading. Blei, Kucukelbir and McAuliffe's review for statisticians [11] is the standard entry point to VI: it derives the ELBO and CAVI, works through Gaussian mixtures, introduces stochastic VI, and is explicit that VI trades MCMC's asymptotic exactness for speed, tending to underestimate posterior variance. Zhang, Bütepage, Kjellström and Mandt [12] organize

the later literature into four axes: scalable VI (stochastic and distributed), generic VI for non-conjugate models (black-box and reparameterization gradients), accurate VI (alternative divergences beyond reverse KL, richer families) and amortized VI (inference networks, VAEs). Jordan, Ghahramani, Jaakkola and Saul [10] is the classic introduction to variational methods for graphical models and remains the clearest source for the convex-duality view. On the sampling side, Neal's chapter [6] is the canonical derivation of HMC, Betancourt [8] gives the geometric picture of why it works in high dimensions, and Hoffman and Gelman [7] describe NUTS, the default sampler in modern probabilistic programming systems. Vehtari et al. [9] set the current standard for convergence diagnostics. Kingma and Welling's monograph [16] surveys VAEs as amortized VI, and Yang et al. [19] survey diffusion models, including their variational and score-based formulations.

State of the art, 2023–2026. Three trends stand out.

1. *Machine learning inside Bayesian computation.* Winter, Campbell, Lin, Srivastava and Dunson [20] review how normalizing flows, Bayesian coresets, distributed inference and variational methods are being used to speed up posterior computation, and where the guarantees are still weaker than for classical MCMC.
2. *Diffusion ideas used for sampling, not just generation.* When only an unnormalized density is available (no data), one can still train a diffusion-style sampler. Denoising Diffusion Samplers [21] learn to reverse a noising process so that it ends at a target known only up to a constant, giving approximate samples and an estimate of the normalizing constant; they connect to optimal control and Schrödinger bridges and are a representative member of a growing family of learned samplers for Bayesian and physics problems.
3. *Workflows over single algorithms.* Practice has moved toward probabilistic programming systems that run NUTS by default, report rank-normalized $\hat{R}$ and bulk/tail ESS [9], and use VI or Laplace as fast first passes rather than final answers.

Open problems. Reliable uncertainty for very large models remains hard: full-batch MCMC is too expensive, and cheap approximations (mean-field VI, last-layer Laplace, ensembles) disagree. Multi-modal posteriors defeat both local VI and single chains. Evaluating the quality of an approximation without ground truth is still largely heuristic, and comparing models through ELBOs is confounded by unknown approximation gaps.

Key takeaways

- Posteriors are easy to evaluate up to a constant and hard to integrate; grids cost G^d, so approximation is unavoidable beyond a few dimensions.
- Monte Carlo error falls as $1/\sqrt{S}$ regardless of dimension, but only if you can draw from the right distribution. Importance sampling corrects for the wrong one, and the weight ESS tells you when the correction has collapsed.
- Metropolis–Hastings needs only ratios of $\tilde{p}$ and a Hastings correction for asymmetric proposals. Gibbs is MH with acceptance 1. HMC uses gradients to move far. Always record rejected states, run several chains, and check $\hat{R}$ and ESS.

- VI maximizes the ELBO, which equals minimizing reverse KL to the posterior. It is fast and scalable, mode seeking, and tends to underestimate variance, especially under mean field.
- Laplace fits a Gaussian at the mode and gives a cheap evidence estimate (and BIC); EP matches moments locally and often gives better variances but may not converge.
- The evidence implements Occam's razor but depends on priors and answers a different question from held-out prediction.
- VAEs are amortized VI with the reparameterization gradient; diffusion models are trained with a variational bound and sampled with Langevin-like dynamics driven by a learned score.

Exercises

1. **ELBO by hand.** Starting from $D_{\mathrm{KL}}(q(z) \| p(z \mid D))$, derive $\log p(D) = \mathcal{L}(q) + D_{\mathrm{KL}}$. Then show that when $q(z) = p(z \mid D)$ the bound is tight, and explain how this makes EM a special case of maximizing the ELBO.

   Hint

   Substitute $\log p(z \mid D) = \log p(D, z) - \log p(D)$ and pull $\log p(D)$ out of the expectation because it does not depend on z. With q equal to the posterior, the KL is zero. EM's E step sets q to the posterior of the latent variables at the current parameters (tight bound); its M step maximizes $\mathcal{L}$ over the parameters with q fixed.

2. **The Hastings correction.** You propose from a log-normal random walk on a positive parameter, $z' = z\, e^{\eta}$ with $\eta \sim \mathcal{N}(0, s^2)$. Compute $q(z' \mid z)$ and the ratio $q(z \mid z')/q(z' \mid z)$. What goes wrong if you use the plain Metropolis rule?

   Hint

   $q(z' \mid z) = \frac{1}{z' s\sqrt{2\pi}} \exp(-(\log z' - \log z)^2/(2s^2))$. The exponential parts are symmetric, so the ratio is z'/z. Omitting it makes the chain sample from a density proportional to $\tilde{p}(z)/z$, biased toward small values.

3. **Step size and mixing.** In the Metropolis code, change the proposal standard deviation to 0.02 and then to 8. For each, report acceptance, lag-one autocorrelation and an estimate of N_{eff} using the formula in the diagnostics section (truncate the sum at the first negative autocorrelation). Which step size gives the most effective samples per iteration?

   Hint

   Compare with Figure 2. Very small steps give acceptance near 1 but autocorrelations near 1, so N_{eff} is tiny and the chain may not even reach the target. Very large steps give low acceptance and long runs of repeated values. An intermediate value, around the target's standard deviation, gives far more effective samples.

4. **Gibbs on a ridge.** Re-run the Gibbs sampler with $\rho = 0.99$. Predict the lag-one autocorrelation of z_1 and N_{eff}/N, then check by simulation. Propose and implement a fix.

   Hint

   The z_1 chain is autoregressive with coefficient $\rho^2 \approx 0.98$, so $N_{\text{eff}}/N = (1-\rho^2)/(1+\rho^2) \approx 0.01$. Fixes: sample (z_1, z_2) jointly as one block, or rotate to $u = (z_1 + z_2)/\sqrt{2}$, $v = (z_1 - z_2)/\sqrt{2}$, which are independent under the target.

5. **Mean field underestimates variance.** For a Gaussian target $\mathcal{N}(0, \Sigma)$ with precision $\Lambda = \Sigma^{-1}$, use the mean-field update rule to show that the optimal q_j is Gaussian with variance $1/\Lambda_{jj}$. Prove $1/\Lambda_{jj} \le \Sigma_{jj}$ and evaluate both for unit variances and correlation 0.85.

   Hint

   $\log p = -\frac{1}{2} z^\top \Lambda z + \text{const}$; take the expectation over q_{-j} and keep terms in z_j: $-\frac{1}{2}\Lambda_{jj} z_j^2 - z_j \sum_{k \neq j} \Lambda_{jk} \mathbb{E}[z_k]$, a Gaussian with precision Λ_{jj}. $1/\Lambda_{jj}$ is the conditional variance of z_j given the rest, which by the Schur complement is Σ_{jj} minus a nonnegative term. For $\rho = 0.85$: $1 - \rho^2 \approx 0.28$ versus 1.

6. **Importance sampling the evidence.** For the two-weight logistic regression, estimate $\log p(D)$ by importance sampling with (a) the prior as proposal and (b) the Laplace Gaussian (with its covariance inflated by 1.5) as proposal. Compare both with the grid value -22.117 and with the weight ESS. Which proposal would you trust in 50 dimensions, and why?

   Hint

   The estimate is $\log \frac{1}{S} \sum_s \tilde{p}(z_s)/q(z_s)$, computed with log-sum-exp. The prior is much wider than the posterior, so most draws get negligible weight and the ESS is a small fraction of S; the Laplace-based proposal sits on the posterior and gives a much larger ESS. In high dimensions the mismatch between prior and posterior grows exponentially, so the prior proposal becomes useless while a well-fitted, slightly heavy-tailed proposal can still work.

References

1. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006, Sections 4.4 (Laplace), 10.1–10.2 and 10.7 (variational inference, EP) and 11.1–11.5 (sampling). Official book page and PDF
2. S. Theodoridis and K. Koutroumbas, *Pattern Recognition*, 4th ed., Academic Press, 2009, Chapter 2 (Bayes decision theory and parameter estimation) and Appendix C (cost function optimization). These are prerequisites rather than identical coverage of this chapter. doi
3. N. Metropolis, A. W. Rosenbluth, M. N. Rosenbluth, A. H. Teller and E. Teller, "Equation of state calculations by fast computing machines," *The Journal of Chemical Physics*, vol. 21, no. 6, pp. 1087–1092, 1953. doi

4. W. K. Hastings, “Monte Carlo sampling methods using Markov chains and their applications,” *Biometrika*, vol. 57, no. 1, pp. 97–109, 1970. doi
5. S. Geman and D. Geman, “Stochastic relaxation, Gibbs distributions, and the Bayesian restoration of images,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-6, no. 6, pp. 721–741, 1984. doi
6. R. M. Neal, “MCMC using Hamiltonian dynamics,” in S. Brooks, A. Gelman, G. Jones and X.-L. Meng (eds.), *Handbook of Markov Chain Monte Carlo*, Chapman and Hall/CRC, 2011 (arXiv: 1206.1901). arXiv
7. M. D. Hoffman and A. Gelman, “The No-U-Turn Sampler: Adaptively setting path lengths in Hamiltonian Monte Carlo,” *Journal of Machine Learning Research*, vol. 15, pp. 1593–1623, 2014. JMLR
8. M. Betancourt, “A conceptual introduction to Hamiltonian Monte Carlo,” arXiv:1701.02434, 2017. arXiv
9. A. Vehtari, A. Gelman, D. Simpson, B. Carpenter and P.-C. Bürkner, “Rank-normalization, folding, and localization: An improved $\hat{R}$ for assessing convergence of MCMC (with discussion),” *Bayesian Analysis*, vol. 16, no. 2, 2021. doi
10. M. I. Jordan, Z. Ghahramani, T. S. Jaakkola and L. K. Saul, “An introduction to variational methods for graphical models,” *Machine Learning*, vol. 37, no. 2, pp. 183–233, 1999. doi
11. D. M. Blei, A. Kucukelbir and J. D. McAuliffe, “Variational inference: A review for statisticians,” *Journal of the American Statistical Association*, vol. 112, no. 518, pp. 859–877, 2017. doi
12. C. Zhang, J. Bütepage, H. Kjellström and S. Mandt, “Advances in variational inference,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 41, no. 8, pp. 2008–2026, 2019. doi
13. R. Ranganath, S. Gerrish and D. M. Blei, “Black box variational inference,” in *Proc. International Conference on Artificial Intelligence and Statistics* (*AISTATS*), PMLR vol. 33, 2014. PMLR
14. T. P. Minka, “Expectation propagation for approximate Bayesian inference,” in *Proc. Conference on Uncertainty in Artificial Intelligence* (*UAI*), 2001 (arXiv:1301.2294). arXiv
15. D. P. Kingma and M. Welling, “Auto-encoding variational Bayes,” arXiv:1312.6114, 2013. arXiv
16. D. P. Kingma and M. Welling, “An introduction to variational autoencoders,” *Foundations and Trends in Machine Learning*, vol. 12, no. 4, pp. 307–392, 2019. arXiv
17. J. Ho, A. Jain and P. Abbeel, “Denoising diffusion probabilistic models,” in *Advances in Neural Information Processing Systems* (*NeurIPS*), 2020 (arXiv:2006.11239). arXiv
18. Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon and B. Poole, “Score-based generative modeling through stochastic differential equations,” in *Proc. International Conference on Learning Representations* (*ICLR*), 2021. arXiv
19. L. Yang, Z. Zhang, Y. Song, S. Hong, R. Xu, Y. Zhao, W. Zhang, B. Cui and M.-H. Yang, “Diffusion models: A comprehensive survey of methods and applications,” *ACM Computing Surveys*, vol. 56, no. 4, 2023. doi
20. S. Winter, T. Campbell, L. Lin, S. Srivastava and D. B. Dunson, “Machine learning and the future of Bayesian computation,” arXiv:2304.11251, 2023. arXiv
21. F. Vargas, W. Grathwohl and A. Doucet, “Denoising diffusion samplers,” in *Proc. International Conference on Learning Representations* (*ICLR*), 2023 (arXiv:2302.13834). arXiv

PR 09 • Ensembles, Validation and Cluster Validity

Derive the bias–variance decomposition, bagging, random forests, AdaBoost and gradient boosting, then estimate and compare recognizers honestly with cross-validation, the bootstrap, nested and group-aware validation, and paired statistical tests.

Online edition: https://blog.shyandram.dev/en/learn/pattern-recognition/pr-09-ensembles-validation/

What you'll learn

- Derive the bias–variance decomposition of squared error and use it to explain why averaging helps unstable learners.
- Explain how decision trees partition feature space, and how bagging and random forests reduce their variance, including out-of-bag estimates.
- Derive AdaBoost from exponential loss, connect it to gradient boosting, and distinguish boosting, voting, Bayesian model averaging and stacking.
- Estimate the performance of a whole recognition procedure with hold-out, k-fold, group-aware and nested cross-validation and with the bootstrap (.632 and .632+).
- Compare two or more classifiers with McNemar's test, the 5×2cv and corrected resampled t-tests, and the Friedman/Nemenyi procedure, and know when each is valid.
- Evaluate probabilities as well as decisions, and treat cluster validity as a model-selection question.

The big picture

A single recognizer makes mistakes for two very different reasons. Sometimes it is too simple to express the true decision rule; sometimes it is so flexible that it memorizes accidents of the particular training set. Ensembles attack the second problem (bagging, random forests) or the first (boosting) by combining many models. Validation is the other half of the story: once we can build many models and tune many knobs, we need a trustworthy way to estimate how the *chosen* procedure will perform on new data, and to decide whether one procedure is really better than another.

> **Explain like I'm five**
>
> Imagine guessing how many sweets are in a jar. One friend's guess can be way off. If ten friends guess *independently* and you average, the wild guesses cancel out and the average is usually closer. But if all ten friends just copy the loudest friend, averaging does not help at all. Now suppose you want to know which friend is the best guesser. You must test them on a *new* jar they have never seen; testing on the jar they already studied would flatter everyone.

Why it matters: tree ensembles remain among the strongest methods for tabular features, including hand-crafted image descriptors [23], deep-network ensembles are a standard way to obtain uncertainty estimates [24], and almost every published comparison between recognizers rests on cross-validation and a statistical test. Theodoridis and Koutroumbas treat decision trees, classifier combination and boosting in Chapter 4, error estimation and comparison in Chapter 10 ("Supervised Learning: The Epilogue"), and cluster validity in Chapter 16 [1]. Bishop's Chapter 14 separates several mathematically different ways to combine models, and Section 3.2 derives the bias–variance decomposition [2]. Classifiers themselves were covered in PR 03 and PR 04.

Bias, variance and noise

Plain version. Imagine retraining the same learning algorithm on many different training sets drawn from the same source. The *average* prediction may still miss the truth: that systematic miss is **bias**. The individual predictions scatter around their average: that scatter is **variance**. Even a perfect predictor cannot remove the randomness in the labels themselves: that is **noise**.

Precise version. Let the target be $y = f(x) + \varepsilon$, where f is the unknown regression function and ε is noise with $\mathbb{E}[\varepsilon] = 0$ and $\mathrm{Var}(\varepsilon) = \sigma^2$, independent of everything else. A learning algorithm maps a random training set D to a predictor $\hat{f}_D$. Write $\bar{f}(x) = \mathbb{E}_D[\hat{f}_D(x)]$ for the average prediction over training sets. At a fixed input x, the expected squared error over training sets and test noise is

$$\begin{aligned}\mathbb{E}_{D,\varepsilon}\big[(y - \hat{f}_D(x))^2\big] &= \mathbb{E}\big[(\varepsilon + f(x) - \bar{f}(x) + \bar{f}(x) - \hat{f}_D(x))^2\big] \\ &= \underbrace{\sigma^2}_{\text{noise}} + \underbrace{\big(f(x) - \bar{f}(x)\big)^2}_{\text{bias}^2} + \underbrace{\mathbb{E}_D\big[(\hat{f}_D(x) - \bar{f}(x))^2\big]}_{\text{variance}}.\end{aligned}$$

The three cross terms vanish: ε has zero mean and is independent of D, $f(x) - \bar{f}(x)$ is a constant, and $\mathbb{E}_D[\bar{f}(x) - \hat{f}_D(x)] = 0$ by the definition of $\bar{f}$. Averaging over the input distribution $p(x)$ gives the expected test error. Geman, Bienenstock and Doursat named the resulting tension the *bias/variance dilemma* [3]: making a model more flexible typically lowers bias and raises variance, and the best complexity balances the two.

The code below estimates each term by brute force for regression trees of growing depth on a noisy sine curve. Two hundred training sets of 60 points each are drawn; the empirical average and spread of the 200 fitted trees give bias² and variance.

```
import numpy as np
from sklearn.tree import DecisionTreeRegressor

rng = np.random.default_rng(0)
f = lambda x: np.sin(2 * np.pi * x)          # true regression function
sigma = 0.3                                   # noise standard deviation
x_test = np.linspace(0, 1, 200)[:, None]

def bias_variance(depth, n_sets=200, n=60):
    preds = np.empty((n_sets, len(x_test)))
```

```
    for s in range(n_sets):                    # many independent training sets
        x = rng.random((n, 1))
        y = f(x[:, 0]) + rng.normal(0, sigma, n)
        tree = DecisionTreeRegressor(max_depth=depth, random_state=0)
        preds[s] = tree.fit(x, y).predict(x_test)
    mean_pred = preds.mean(axis=0)
    bias2 = np.mean((mean_pred - f(x_test[:, 0])) ** 2)
    var = np.mean(preds.var(axis=0))
    return bias2, var

for depth in [1, 2, 3, 5, 8, None]:
    b2, v = bias_variance(depth)
    print(f"depth={str(depth):>4}  bias^2={b2:.4f}  variance={v:.4f}  "
          f"expected test MSE~{b2 + v + sigma**2:.4f}")
```

On our run, depth 1 gives bias² ≈ 0.074 and variance ≈ 0.035, depth 3 gives the lowest total (bias² ≈ 0.003, variance ≈ 0.045), and fully grown trees have almost no bias but variance ≈ 0.092, as large as the noise variance $\sigma^2 = 0.09$ itself. Figure 1 shows the same experiment visually.

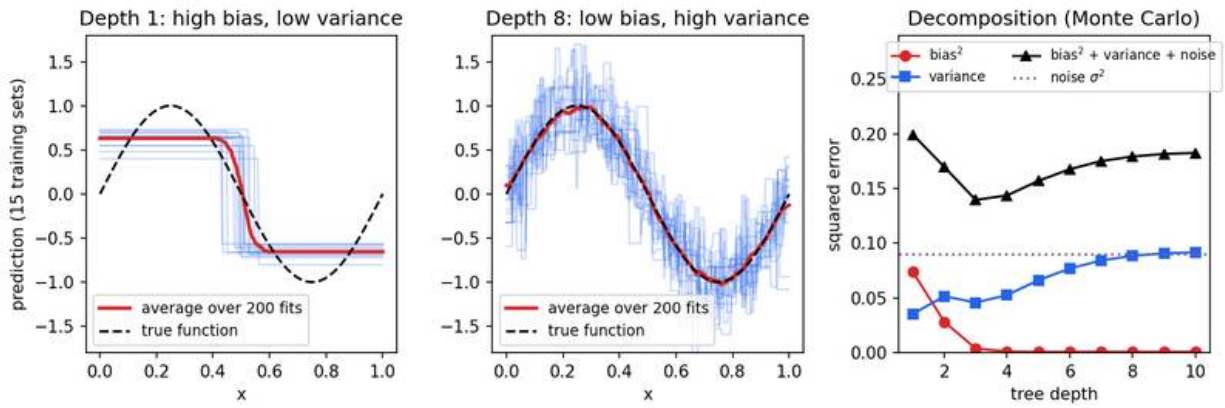


Figure 1 — Bias–variance decomposition for regression trees on $y = \sin(2\pi x) + \text{noise}$ ($\sigma = 0.3$, 60 points per training set, 200 training sets). Shallow trees are consistently wrong (bias); deep trees are right on average but each one is noisy (variance). Generated by this chapter's script.

Three cautions keep the picture honest. First, the clean decomposition is specific to squared error; for 0–1 classification loss there are several competing definitions and bias and variance can interact non-additively, so use the decomposition as intuition rather than as an exact accounting for classifiers. Second, "complexity" is not just parameter count: depth, regularization strength, number of boosting rounds and early stopping all move a model along the bias–variance axis. Third, very large over-parameterized models can show a test-error curve that rises and then falls again as capacity grows past the interpolation point, the "double descent" shape documented by Belkin and colleagues [25]. The decomposition still holds pointwise; what changes is how variance behaves as capacity grows.

Trees and partitions

Plain version. A decision tree asks a sequence of yes/no questions about the features, such as "is feature 3 below 0.7?", and each final answer box (a leaf) stores a prediction. It is the classical example of a low-bias, high-variance learner, which is exactly why it is the favorite building block for ensembles.

Precise version. A tree recursively divides observations by conditions such as $x_j \le b$. Each leaf estimates a class distribution (classification) or a constant (regression). For a classification node with class proportions p_k, the **Gini impurity** is

$$G = 1 - \sum_k p_k^2,$$

and the **entropy** is $H = -\sum_k p_k \log p_k$. Both are zero for a pure node and largest for a uniform mix. A split of a parent node P into children L and R, holding n_L and n_R of its n_P samples, is scored by the impurity decrease

$$\Delta = G(P) - \frac{n_L}{n_P} G(L) - \frac{n_R}{n_P} G(R),$$

and the learner chooses the feature and threshold with the largest Δ. This is a greedy local choice: the best single split need not lead to the best tree. A deep tree can capture interactions and nonlinear, axis-aligned boundaries, but it can change substantially when a few training points change; that instability is exactly the high variance seen in Figure 1.

Depth limits, minimum leaf sizes and cost-complexity pruning constrain complexity. Because splits depend only on the ordering of each feature, trees are invariant to monotone rescaling, which makes scaling less critical than for distance-based models; missing values, feature definitions and train-only transformations still require care. Theodoridis and Koutroumbas (Chapter 4) and Bishop (Section 14.4) treat trees as one family of nonlinear classifiers [1], [2].

Bagging and random forests

Plain version. Bagging ("bootstrap aggregating") builds many training sets by sampling the original data *with replacement*, trains one unstable model on each, and averages or votes. Random forests add one more source of randomness, a random subset of features at every split, so that the trees disagree more and their average improves more.

Precise version. Breiman introduced bagging as a way to improve *unstable* procedures, those whose output changes a lot under small perturbations of the training set [4]. Given B bootstrap samples $D^{(1)}, \dots, D^{(B)}$, each of size n, the bagged regressor is $\hat{f}_{\text{bag}}(x) = \frac{1}{B} \sum_b \hat{f}_{D^{(b)}}(x)$ and the bagged classifier takes a majority vote (or averages class probabilities).

Why does averaging help? Suppose M scalar predictors at a fixed x each have variance σ^2 and every pair has correlation ρ. Expanding the variance of a sum,

$$\operatorname{Var}\Big(\frac{1}{M}\sum_{m=1}^{M} f_m\Big) = \frac{1}{M^2}\Big[M\sigma^2 + M(M-1)\rho\sigma^2\Big] = \sigma^2\left[\rho + \frac{1-\rho}{M}\right].$$

The second term vanishes as M grows, but the first term $\rho\sigma^2$ does not: correlated errors set a floor (Figure 2, left). Averaging leaves the bias essentially unchanged, so bagging helps most for low-bias, high-variance learners such as deep trees, and little for stable learners such as linear discriminants or k-nearest neighbors with large k. This illustrative formula explains why correlated errors limit gains; it does not promise that every classification ensemble improves accuracy.

Random forests reduce ρ directly [5]. When growing each tree on its bootstrap sample, every split considers only a random subset of m_{try} features (a common default for classification is $\sqrt{d}$ of d features). Breiman showed that the forest's generalization error is bounded in terms of the *strength* of the individual trees and the *correlation* between them [5], which is the same trade-off as the variance formula: weaker but less correlated trees can make a stronger forest.

Out-of-bag (OOB) estimates. The probability that a given observation is *not* drawn in n draws with replacement is $(1 - 1/n)^n \to e^{-1} \approx 0.368$. Each tree therefore leaves out about 37% of the data, and each observation can be predicted by the roughly one third of trees that never saw it. Aggregating those predictions gives an OOB error estimate essentially for free [5]. In Figure 2 (right) the OOB estimate tracks the test error once there are enough trees. But ordinary bootstrapping resamples *rows*: if several rows share a subject, writer or recording, a tree can train on one image of a person and be "tested" OOB on another image of the same person. The OOB score is then optimistic for deployment to new groups.

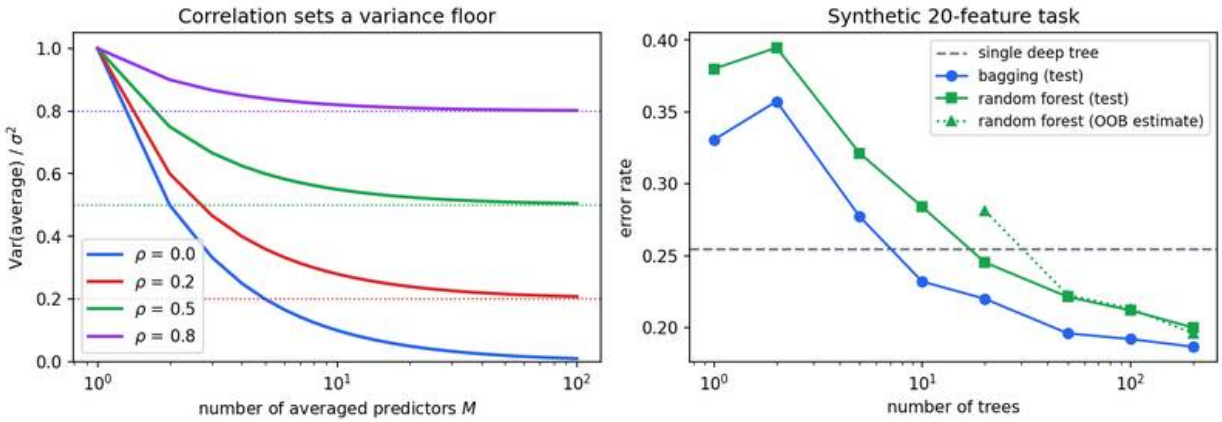


Figure 2 — Left: analytic variance of an average of M equally correlated predictors. Right: one synthetic 20-feature task (750 training, 750 test samples). Both ensembles beat a single deep tree once they have tens of trees; on this particular task plain bagging is slightly better than the forest, a reminder that random feature subsets are not guaranteed to help. Votes with very few trees (1–2) are unstable and can tie.

Boosting changes the next learner's job

Plain version. Bagging trains its members independently and in parallel. Boosting trains them *in sequence*: each new weak learner concentrates on the examples the current ensemble still gets wrong, and the final prediction is a weighted sum. Bagging mainly reduces variance; boosting mainly reduces bias, turning many "slightly better than chance" rules into a strong classifier.

Precise version. Boosting builds an additive model

$$F_M(x) = \sum_{m=1}^{M} \alpha_m h_m(x),$$

where each h_m is a weak learner (often a depth-1 tree, a *stump*) and α_m its weight. The class prediction for labels $y \in \{-1, +1\}$ is $\operatorname{sign} F_M(x)$.

The AdaBoost algorithm

Freund and Schapire's AdaBoost [6] works as follows, for training pairs (x_i, y_i), $i = 1, \dots, n$:

1. Start with uniform weights $w_i^{(1)} = 1/n$.
2. For $m = 1, \dots, M$:
   - fit h_m to the training data using weights $w^{(m)}$;
   - compute the weighted error $\epsilon_m = \sum_i w_i^{(m)} \mathbf{1}[h_m(x_i) \neq y_i] / \sum_i w_i^{(m)}$;
   - set $\alpha_m = \frac{1}{2} \log \frac{1-\epsilon_m}{\epsilon_m}$;
   - update $w_i^{(m+1)} \propto w_i^{(m)} \exp\big(-\alpha_m y_i h_m(x_i)\big)$ and renormalize.
3. Output $\operatorname{sign} \sum_m \alpha_m h_m(x)$.

Misclassified points have $y_i h_m(x_i) = -1$, so their weight is multiplied by e^{α_m}; correctly classified points are multiplied by $e^{-\alpha_m}$. A learner with $\epsilon_m < 0.5$ gets a positive vote, and the better it is, the larger the vote.

Deriving AdaBoost from exponential loss

Friedman, Hastie and Tibshirani showed that AdaBoost is **forward stagewise additive modeling** with the **exponential loss** $L(y, F) = \exp(-yF)$ [7]. Suppose F_{m-1} is fixed and we add one term αh. The training loss is

$$\sum_{i=1}^{n} \exp\big(-y_i[F_{m-1}(x_i) + \alpha h(x_i)]\big) = \sum_{i=1}^{n} w_i^{(m)} \exp\big(-\alpha y_i h(x_i)\big), \qquad w_i^{(m)} = \exp\big(-y_i F_{m-1}(x_i)\big).$$

The weights $w_i^{(m)}$ do not depend on α or h. Split the sum into correctly and incorrectly classified points:

$$\sum_i w_i^{(m)} e^{-\alpha y_i h(x_i)} = e^{-\alpha} \sum_{y_i = h(x_i)} w_i^{(m)} + e^{\alpha} \sum_{y_i \neq h(x_i)} w_i^{(m)} = (e^{\alpha} - e^{-\alpha}) \sum_i w_i^{(m)} \mathbf{1}[y_i \neq h(x_i)] + e^{-\alpha} \sum_i w_i^{(m)}.$$

Step 1, choose h. For any $\alpha > 0$, the first coefficient is positive, so the best h minimizes the **weighted training error**: exactly "fit h_m with weights $w^{(m)}$."

Step 2, choose α. Divide by $\sum_i w_i^{(m)}$ and write ϵ for the normalized weighted error. We minimize $g(\alpha) = e^{-\alpha}(1-\epsilon) + e^{\alpha}\epsilon$. Setting $g'(\alpha) = -e^{-\alpha}(1-\epsilon) + e^{\alpha}\epsilon = 0$ gives $e^{2\alpha} = (1-\epsilon)/\epsilon$, that is,

$$\alpha_m = \frac{1}{2}\log\frac{1-\epsilon_m}{\epsilon_m}.$$

Step 3, update weights. The next weights are $w_i^{(m+1)} = \exp(-y_i F_m(x_i)) = w_i^{(m)}\exp(-\alpha_m y_i h_m(x_i))$, the AdaBoost update. Every step of the algorithm is thus recovered from greedy minimization of exponential loss.

Two consequences follow. Since $\mathbf{1}[yF < 0] \le e^{-yF}$, the training error is bounded by the average exponential loss; with the optimal α_m, each round multiplies that bound by $2\sqrt{\epsilon_m(1-\epsilon_m)} < 1$ whenever $\epsilon_m < 0.5$, so training error falls exponentially fast if every weak learner beats chance by a margin [6]. And the minimizer of expected exponential loss is $F^*(x) = \frac{1}{2}\log\frac{P(y=1\mid x)}{P(y=-1\mid x)}$, half the log-odds [7], so $\sigma(2F_M(x))$ is a (not necessarily well-calibrated) probability estimate. The exponential weight also explains a known weakness: a mislabeled point receives exponentially growing weight, so AdaBoost can be sensitive to label noise and outliers.

The code implements AdaBoost from scratch with weighted stumps and checks it against scikit-learn's `AdaBoostClassifier`. For two classes scikit-learn's SAMME variant uses $\alpha = \log\frac{1-\epsilon}{\epsilon}$, twice ours, which rescales F but leaves every sign, and every renormalized weight, unchanged.

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import AdaBoostClassifier

X, y01 = make_classification(n_samples=600, n_features=10, n_informative=4,
                             flip_y=0.05, random_state=1)
y = 2 * y01 - 1                                     # labels in {-1, +1}
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, random_state=1)

def adaboost_fit(X, y, M=100):
    n = len(y)
    w = np.full(n, 1 / n)                           # step 1: uniform weights
    stumps, alphas = [], []
    for m in range(M):
        h = DecisionTreeClassifier(max_depth=1, random_state=0)
        h.fit(X, y, sample_weight=w)                # step 2a: weighted fit
        pred = h.predict(X)
        err = np.sum(w * (pred != y)) / np.sum(w) # step 2b: weighted error
        err = np.clip(err, 1e-12, 1 - 1e-12)
        alpha = 0.5 * np.log((1 - err) / err)     # step 2c: learner weight
        w = w * np.exp(-alpha * y * pred)         # step 2d: reweight
        w /= w.sum()
        stumps.append(h); alphas.append(alpha)
    return stumps, np.array(alphas)
```

```
def adaboost_score(stumps, alphas, X):
    return sum(a * h.predict(X) for h, a in zip(stumps, alphas))

stumps, alphas = adaboost_fit(Xtr, ytr)
F_tr = adaboost_score(stumps, alphas, Xtr)
F_te = adaboost_score(stumps, alphas, Xte)
print("from scratch: train err", np.mean(np.sign(F_tr) != ytr),
      " test err", np.mean(np.sign(F_te) != yte))
print("exp-loss bound on train err:", np.mean(np.exp(-ytr * F_tr)))

ref = AdaBoostClassifier(DecisionTreeClassifier(max_depth=1, random_state=0),
                         n_estimators=100, random_state=0).fit(Xtr, ytr)
agree = np.mean(ref.predict(Xte) == np.sign(F_te))
print("sklearn test err", np.mean(ref.predict(Xte) != yte),
      " agreement with ours:", agree)
```

With scikit-learn 1.6 our run prints a training error of 0.076, an exponential-loss bound of 0.384, a test error of 0.156 for both implementations and 100% agreement on test predictions. Figure 3 shows the curves over 300 rounds and the staircase boundary that stumps build in two dimensions.

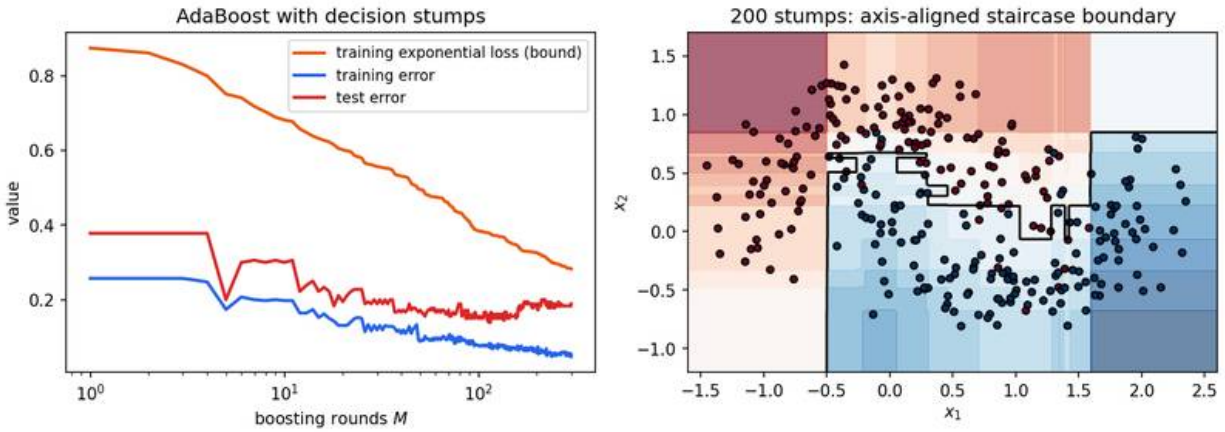


Figure 3 — AdaBoost with decision stumps (our from-scratch implementation). Left: the exponential loss upper-bounds the training error and keeps shrinking; test error plateaus and can drift upward slightly with label noise. Right: summing axis-aligned stumps yields a staircase decision boundary on the two-moons data.

Gradient boosting

Plain version. Gradient boosting generalizes the AdaBoost idea to any differentiable loss: at each round, compute in which direction each training prediction should move to reduce the loss, and fit a small tree to those directions.

Precise version. Friedman's gradient boosting machine [8] performs gradient descent in function space. At round m, compute the **pseudo-residuals**

$$r_{im} = -\left.\frac{\partial L(y_i, F)}{\partial F}\right|_{F=F_{m-1}(x_i)},$$

fit a regression tree h_m to the pairs (x_i, r_{im}) by least squares, choose a step ρ_m by line search (or one optimal constant per leaf, Friedman's TreeBoost), and update $F_m = F_{m-1} + \nu\rho_m h_m$ with a **learning rate** (shrinkage) $0 < \nu \leq 1$. For squared error $L = \frac{1}{2}(y - F)^2$ the pseudo-residual is the ordinary residual $y_i - F_{m-1}(x_i)$. For binary log loss with $y \in \{0, 1\}$ and F the log-odds, it is $y_i - \sigma(F_{m-1}(x_i))$: the label minus the *predicted probability*, not minus the predicted class. With exponential loss and a line search, the procedure reduces to a version of AdaBoost.

The learning rate, the number of rounds and the tree size interact: a smaller ν needs more rounds but usually generalizes better, and the number of rounds is chosen by early stopping on validation data, never on the final test. XGBoost [9] made gradient-boosted trees fast and scalable through a regularized objective, second-order (Newton) leaf values, sparsity-aware split finding and a weighted quantile sketch; histogram-based implementations such as scikit-learn's `HistGradientBoostingClassifier` follow the same principles [26].

Averaging is not always the same operation

Many methods "combine models," but they answer different questions.

Method	What supplies the combination?	Main caution
Voting or probability averaging (bagging, forests)	fixed equal combination of independently trained predictors	correlated errors and incompatible calibration
Boosting	sequential fitting; weights from a loss	label noise; needs early stopping
Bayesian model averaging	posterior probability over model hypotheses	evidence and priors must be meaningful
Stacking	a learned predictor of base-model outputs	train it on out-of-fold predictions
Conditional mixtures (mixtures of experts)	input-dependent gates or responsibilities	experts and gate can overfit together

Bayesian model averaging (Bishop Section 14.1 [2]) uses

$$p(y \mid x, D) = \sum_m p(y \mid x, D, M_m)\, p(M_m \mid D),$$

where $p(M_m \mid D) \propto p(D \mid M_m)p(M_m)$ is the posterior probability of model M_m. Those weights are posterior model probabilities, not arbitrary validation accuracies, and with plenty of data they tend to concentrate on one model. BMA expresses uncertainty about *which* model generated the data; a committee or ensemble instead builds a richer model out of several parts [2]. The model evidence $p(D \mid M_m)$ connects to the approximate-inference tools of PR 08.

Stacking (stacked generalization, Wolpert [10]) instead *learns* a mapping from base-model predictions to the target. Its training features must come from base models that did not train on those

same examples; otherwise the stacker learns from unrealistically easy, overfitted predictions. The standard recipe: split the training set into K folds, fit each base model on $K-1$ folds and predict the held-out fold, collect these out-of-fold predictions as features, fit a simple meta-learner (often regularized logistic regression) on them, and finally refit the base models on all training data.

The following comparison on scikit-learn's bundled breast-cancer dataset (569 samples, 30 features) runs every method inside the same stratified 5-fold cross-validation, so all numbers are comparable. `StackingClassifier` builds its own inner out-of-fold predictions (`cv=5`) inside each outer training fold.

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import (BaggingClassifier, RandomForestClassifier,
                              HistGradientBoostingClassifier, StackingClassifier)

X, y = load_breast_cancer(return_X_y=True)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)
logreg = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000))
models = {
    "single tree": DecisionTreeClassifier(random_state=0),
    "bagged trees": BaggingClassifier(DecisionTreeClassifier(), n_estimators=100,
                                      random_state=0),
    "random forest": RandomForestClassifier(n_estimators=100, random_state=0),
    "hist. gradient boosting": HistGradientBoostingClassifier(random_state=0),
    "logistic regression": logreg,
}
models["stacking"] = StackingClassifier(
    estimators=[("rf", models["random forest"]),
                ("gb", models["hist. gradient boosting"]),
                ("lr", logreg)],
    final_estimator=LogisticRegression(max_iter=2000),
    cv=5, stack_method="predict_proba")
for name, model in models.items():
    s = cross_val_score(model, X, y, cv=cv, scoring="accuracy")
    print(f"{name:>24}: {s.mean():.3f} +/- {s.std():.3f}")
```

Our run gives mean accuracies of 0.926 (single tree), 0.960 (bagging), 0.965 (forest), 0.970 (gradient boosting), 0.979 (logistic regression) and 0.979 (stacking), with fold standard deviations between 0.009 and 0.022. Two lessons follow. Ensembling clearly fixes the unstable single tree. But a well-regularized *linear* model is already as good as the stack on this small, nearly linearly separable dataset, and the differences among the top four are within one fold standard deviation. Whether any of them is *significantly* better is a question for the statistical tests later in this chapter, not for eyeballing means.

Validation estimates a procedure

Plain version. A test score is only trustworthy if the test data played no part in building the model. Every choice we make while looking at data, including features, hyperparameters, thresholds, and even which algorithm to use, is part of the "model," and must be made without the data used to judge it.

Training fits parameters. Validation selects hyperparameters, features, thresholds and calibration choices. Testing assesses the selected procedure. Every adaptive choice consumes information from whichever data it examines. Strictly speaking, cross-validation estimates the performance of a *procedure* (an algorithm plus its tuning rules) trained on data sets of a given size, rather than the error of the one fitted model you will ship; Bates, Hastie and Tibshirani make this precise and show that naive cross-validation confidence intervals can undercover [19].

Hold-out, k-fold and leave-one-out

- **Hold-out.** Split once into training and test parts. Simple and cheap, but with small data the estimate has high variance and depends on the split, and the model is trained on less data than you have.
- **k-fold cross-validation.** Partition the data into k folds; train on $k-1$ and score the remaining one, rotating so each fold is held out once (Figure 4, top). The CV estimate is $\widehat{\mathrm{Err}}_{\mathrm{CV}} = \frac{1}{n}\sum_i L(y_i, \hat{f}^{(-\kappa(i))}(x_i))$, where $\kappa(i)$ is the fold containing sample i and $\hat{f}^{(-k)}$ is the model trained without fold k. Values $k = 5$ or 10 are common compromises: each model sees 80–90% of the data, so the pessimistic bias is small, and the cost is k fits.
- **Leave-one-out (LOO).** The case $k = n$. Nearly unbiased for the size-n procedure, but it requires n fits (unless a closed form exists, as for linear smoothers or k-NN) and can have high variance because the n training sets are almost identical.
- **Stratification** keeps class proportions similar across folds, which matters for rare classes. It does not fix dependence between samples.

Arlot and Celisse survey the theory of these choices for both estimating risk and selecting models [18].

Groups and time

For observations sharing a patient, writer, camera or recording session, hold **whole groups** together (Figure 4, middle). Otherwise near-duplicate images of the same person land on both sides of the split and the score measures memorization of that person, not recognition of new people. Time-ordered data needs chronological splits (train on the past, test on the future). The rule is to make the split mirror the deployment question: "new images of known writers" and "images of new writers" are different questions and deserve different splits.

The bootstrap, .632 and .632+

Plain version. Instead of folds, draw many bootstrap samples, train on each, and test on the points each one left out.

Precise version. Draw B bootstrap samples of size n with replacement. For each, fit the model and evaluate it on the out-of-bootstrap points (about 36.8% of the data). The average is the leave-one-out bootstrap error $\widehat{\mathrm{Err}}^{(1)}$. Because each bootstrap model is trained on only about 63.2% of *distinct* points, $\widehat{\mathrm{Err}}^{(1)}$ is pessimistic, while the resubstitution (training) error $\overline{\mathrm{err}}$ is optimistic. Efron's **.632 estimator** blends them:

$$\widehat{\mathrm{Err}}^{(.632)} = 0.368\,\overline{\mathrm{err}} + 0.632\,\widehat{\mathrm{Err}}^{(1)}.$$

It fails for heavily overfitting learners such as 1-nearest neighbor, whose training error is zero. Efron and Tibshirani's **.632+ estimator** [11] fixes this by moving weight toward $\widehat{\mathrm{Err}}^{(1)}$ as overfitting grows. Let $\hat{\gamma}$ be the *no-information error rate* (the error if inputs and labels were independent: $\hat{\gamma} = \sum_k \hat{p}_k(1-\hat{q}_k)$, with $\hat{p}_k$ the observed proportion of class k and $\hat{q}_k$ the proportion of predictions equal to k), and define the relative overfitting rate $\hat{R} = (\widehat{\mathrm{Err}}^{(1)} - \overline{\mathrm{err}})/(\hat{\gamma} - \overline{\mathrm{err}})$, clipped to $[0,1]$. Then

$$\hat{w} = \frac{0.632}{1 - 0.368\hat{R}}, \qquad \widehat{\mathrm{Err}}^{(.632+)} = (1-\hat{w})\,\overline{\mathrm{err}} + \hat{w}\,\widehat{\mathrm{Err}}^{(1)}.$$

When $\hat{R} = 0$ (no overfitting) it reduces to .632; when $\hat{R} = 1$ it becomes $\widehat{\mathrm{Err}}^{(1)}$.

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_val_score, KFold

X, y = make_classification(n_samples=200, n_features=6, n_informative=3,
                           flip_y=0.1, random_state=3)
model = KNeighborsClassifier(n_neighbors=1)          # resubstitution error is 0
rng = np.random.default_rng(3)
n, B = len(y), 200
resub = np.mean(model.fit(X, y).predict(X) != y)
oob_errs = []
for b in range(B):
    idx = rng.integers(0, n, n)                       # draw n with replacement
    out = np.setdiff1d(np.arange(n), idx)             # ~36.8% left out
    model.fit(X[idx], y[idx])
    oob_errs.append(np.mean(model.predict(X[out]) != y[out]))
err_boot1 = np.mean(oob_errs)                          # leave-one-out bootstrap
err_632 = 0.368 * resub + 0.632 * err_boot1
cv10 = 1 - cross_val_score(model, X, y,
                           cv=KFold(10, shuffle=True, random_state=3)).mean()
print(f"resubstitution {resub:.3f}  OOB bootstrap {err_boot1:.3f}  "
      f".632 {err_632:.3f}  10-fold CV {cv10:.3f}")
```

```
# .632+ : shrink toward the out-of-bootstrap error when overfitting is large.
pred = model.fit(X, y).predict(X)
gamma = sum(np.mean(y == k) * (1 - np.mean(pred == k)) for k in np.unique(y))
R = np.clip((err_boot1 - resub) / (gamma - resub), 0, 1)
w = 0.632 / (1 - 0.368 * R)
print(f"no-information rate {gamma:.3f}  R {R:.3f}  "
      f".632+ {(1 - w) * resub + w * err_boot1:.3f}")
```

For 1-NN the resubstitution error is 0.000, the out-of-bootstrap error 0.332, the .632 estimate only 0.210 (pulled down by the meaningless zero), the .632+ estimate 0.278 and 10-fold CV 0.325. The .632 estimator is badly optimistic here; .632+ repairs most of the gap. In practice, 10-fold CV (possibly repeated) and .632+ are both reasonable; the bootstrap is attractive for very small samples and for confidence intervals.

Nested cross-validation

Plain version. If you tune hyperparameters by cross-validation and then report that same cross-validation score, you report the score of the *luckiest* configuration, which is optimistic. Nested CV puts the whole tuning process inside an outer loop.

Nested cross-validation has an **inner loop** for selection and an **outer loop** for evaluating that selection procedure (Figure 4, bottom). Within each outer training set, an inner CV chooses hyperparameters; the chosen configuration is refit on the whole outer training set and scored on the outer held-out fold, which the inner loop never saw. Varma and Simon showed that reporting the inner (tuning) CV score can be substantially biased, and that nested CV gives nearly unbiased estimates [12]. Cawley and Talbot went further: overfitting the model-selection criterion itself can cost as much as the differences between learning algorithms, so a low-variance selection criterion matters as much as an unbiased one [13]. Refit preprocessing (scaling, feature selection, PCA) inside each inner fold, typically by placing it in a `Pipeline`. Report variability across outer folds, but do not treat correlated fold scores as independent measurements for a naive confidence interval.

After estimating the procedure, select on development data and evaluate once on an untouched test. Repeatedly inspecting test errors and redesigning the model turns that test into another validation set.

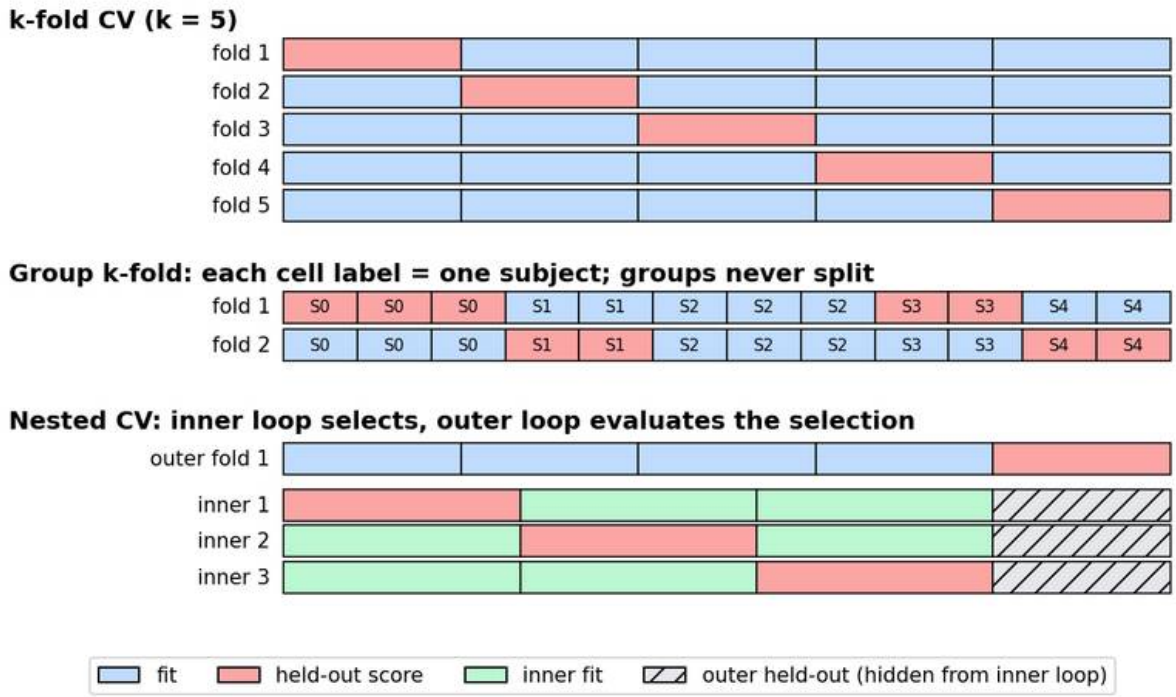


Figure 4 — Three validation layouts. Top: 5-fold CV. Middle: group k-fold keeps every observation of a subject (S0–S4) on one side of each split. Bottom: nested CV; the inner loop tunes on the outer training data only, while the outer held-out fold (hatched) stays hidden until the tuned model is scored.

Code: nested selection without group leakage

This CPU example creates three noisy repeated measurements of each synthetic subject. Group labels describe subjects, not classes. The outer loop estimates the selected forest procedure; a separate group-disjoint test set remains untouched until final reporting.

```
import numpy as np
from sklearn.datasets import make_classification
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split, GroupKFold, GridSearchCV
from sklearn.metrics import balanced_accuracy_score, log_loss

rng = np.random.default_rng(9)
subject_x, subject_y = make_classification(
    n_samples=180, n_features=8, n_informative=5, n_redundant=1,
    class_sep=1.0, random_state=9)
groups = np.repeat(np.arange(180), 3)
X = subject_x[groups] + rng.normal(0, 0.03, (len(groups), 8))
y = subject_y[groups]
dev_groups, test_groups = train_test_split(
    np.arange(180), test_size=0.25, stratify=subject_y, random_state=9)
dev = np.flatnonzero(np.isin(groups, dev_groups))
```

```
test = np.flatnonzero(np.isin(groups, test_groups))
assert not set(groups[dev]) & set(groups[test])

def search():
    return GridSearchCV(
        RandomForestClassifier(n_estimators=60, random_state=9, n_jobs=1),
        {"max_depth": [3, None], "min_samples_leaf": [1, 3]},
        cv=GroupKFold(3), scoring="balanced_accuracy")

outer_scores = []
for fit, valid in GroupKFold(3).split(X[dev], y[dev], groups[dev]):
    chosen = search().fit(X[dev[fit]], y[dev[fit]], groups=groups[dev[fit]])
    pred = chosen.predict(X[dev[valid]])
    outer_scores.append(balanced_accuracy_score(y[dev[valid]], pred))
final = search().fit(X[dev], y[dev], groups=groups[dev])
p = final.predict_proba(X[test])
pred = final.predict(X[test])
positive = p[:, np.flatnonzero(final.classes_ == 1)[0]]
print("Nested outer balanced accuracy:", np.round(outer_scores, 3))
print("Selected parameters:", final.best_params_)
print("Independent group test:", balanced_accuracy_score(y[test], pred))
print("Test log loss:", log_loss(y[test], p, labels=final.classes_))
print("Test Brier:", np.mean((positive - y[test]) ** 2))
```

Our run prints outer balanced accuracies of 0.869, 0.784 and 0.861, selects `max_depth=None, min_samples_leaf=3`, and scores 0.853 balanced accuracy, 0.378 log loss and 0.118 Brier score on the independent group test. The spread of the outer scores (0.78 to 0.87) is a useful reminder of how noisy a single split would be. These results describe the generated subjects; they do not establish performance on images. Replacing `GroupKFold` by shuffled row-level folds would let near-copies of a subject appear on both sides and inflate the scores (Exercise 3).

Comparing classifiers statistically

Plain version. If classifier A scores 97.9% and B scores 96.5%, is A really better, or did it get lucky with this particular data? A statistical test asks how surprising the observed difference would be if the two were equally good. The catch is that the usual textbook t-test assumes independent measurements, and cross-validation scores are not independent.

Dietterich's study of approximate tests [14] is the classic starting point; Raschka's tutorial collects the main options with code [20].

One test set: McNemar's test

Train both classifiers once and evaluate them on the *same* test set. Count n_{01}, the test points A gets right and B gets wrong, and n_{10}, the reverse. Points that both get right or both get wrong carry no information about which is better. Under the null hypothesis of equal error rates, each

disagreement is equally likely to go either way, so $n_{01} \sim \text{Binomial}(n_{01}+n_{10}, \tfrac{1}{2})$. The continuity-corrected statistic

$$\chi^2 = \frac{(|n_{01}-n_{10}|-1)^2}{n_{01}+n_{10}}$$

is approximately χ^2 with one degree of freedom; for small counts use the exact binomial test. Dietterich found McNemar's test has acceptable Type I error and recommended it when the algorithms can be run only once [14]. It does not account for variability due to the choice of training set.

Repeated splits: why the naive t-test fails

A tempting alternative is to repeat random train/test splits J times, record the error differences d_j, and run a paired t-test on them. Dietterich showed this **resampled t-test** has a badly inflated Type I error [14]: the training sets overlap heavily, so the d_j are positively correlated and the usual standard error $\hat{\sigma}/\sqrt{J}$ is far too small. Two fixes are common.

- **5×2cv paired t-test** [14]. Do five replications of 2-fold CV. In replication i, let $p_i^{(1)}, p_i^{(2)}$ be the error differences on the two folds, $\bar{p}_i$ their mean and $s_i^2 = (p_i^{(1)}-\bar{p}_i)^2 + (p_i^{(2)}-\bar{p}_i)^2$. The statistic $t = p_1^{(1)} / \sqrt{\tfrac{1}{5}\sum_{i=1}^{5} s_i^2}$ is compared with a t distribution with 5 degrees of freedom.
- **Corrected resampled t-test** (Nadeau and Bengio [15]). Keep J random splits with n_{test} test and n_{train} training points, but inflate the variance to account for overlap:

$$t = \frac{\bar{d}}{\sqrt{\left(\dfrac{1}{J} + \dfrac{n_{\text{test}}}{n_{\text{train}}}\right)\hat{\sigma}_d^2}},$$

where $\bar{d}$ and $\hat{\sigma}_d^2$ are the mean and sample variance of the d_j, compared with a t distribution with $J-1$ degrees of freedom. The extra $n_{\text{test}}/n_{\text{train}}$ term does not shrink as J grows, which is the point: more resamples of the same data cannot create more information.

```
import numpy as np
from scipy import stats
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, ShuffleSplit
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier

X, y = load_breast_cancer(return_X_y=True)
A = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000))
B = RandomForestClassifier(n_estimators=200, random_state=0)

# 1) McNemar's test on one held-out test set (paired predictions).
```

```
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.4, stratify=y,
                                      random_state=0)
ca = A.fit(Xtr, ytr).predict(Xte) == yte
cb = B.fit(Xtr, ytr).predict(Xte) == yte
n01 = int(np.sum(ca & ~cb))      # A right, B wrong
n10 = int(np.sum(~ca & cb))      # A wrong, B right
p_exact = stats.binomtest(n01, n01 + n10, 0.5).pvalue if n01 + n10 else 1.0
chi2 = (abs(n01 - n10) - 1) ** 2 / (n01 + n10) if n01 + n10 else 0.0
print(f"McNemar: n01={n01} n10={n10} chi2_cc={chi2:.3f} "
      f"p_chi2={stats.chi2.sf(chi2, 1):.3f} p_exact={p_exact:.3f}")

# 2) Corrected resampled t-test (Nadeau & Bengio) over J random splits.
J, test_frac = 30, 0.2
d = []
for tr, te in ShuffleSplit(J, test_size=test_frac, random_state=0).split(X):
    ea = np.mean(A.fit(X[tr], y[tr]).predict(X[te]) != y[te])
    eb = np.mean(B.fit(X[tr], y[tr]).predict(X[te]) != y[te])
    d.append(ea - eb)
d = np.array(d)
naive_se = np.sqrt(d.var(ddof=1) / J)
corr_se = np.sqrt(d.var(ddof=1) * (1 / J + test_frac / (1 - test_frac)))
for name, se in [("naive", naive_se), ("corrected", corr_se)]:
    t = d.mean() / se
    print(f"{name:>9} t={t:.3f}  p={2 * stats.t.sf(abs(t), J - 1):.3f}")
```

Our run gives $n_{01} = 9$, $n_{10} = 5$ and McNemar $p \approx 0.42$: no evidence of a difference on one test set of 228 cases. Over 30 random 80/20 splits, logistic regression has on average about 1.8 percentage points lower error. The naive t-test reports $t \approx -5.4$, $p < 0.001$; the corrected test reports $t \approx -1.85$, $p \approx 0.075$ (Figure 5). Same data, opposite conclusions at the 5% level: the naive test mistakes 30 heavily overlapping resamples for 30 independent experiments.

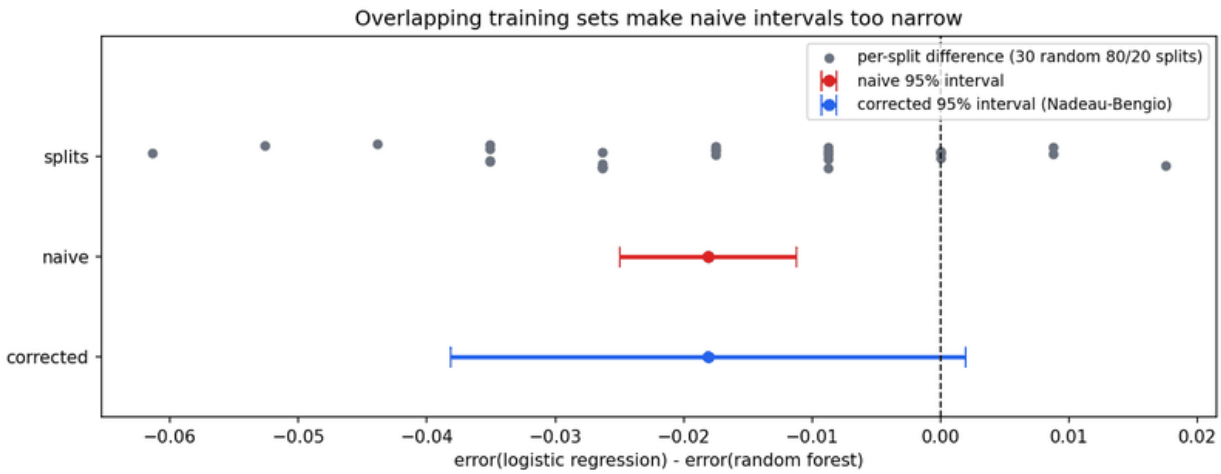


Figure 5 — Error differences (logistic regression minus random forest) on the breast-cancer data over 30 random 80/20 splits. The naive paired t interval excludes zero; the Nadeau–Bengio corrected interval, which accounts for overlapping training sets, does not.

Many classifiers on many datasets

When comparing k algorithms over N datasets, Demšar recommends non-parametric tests on ranks [16]: the Wilcoxon signed-rank test for two classifiers, and for more the **Friedman test**. Rank the algorithms on each dataset (1 = best) and let R_j be the average rank of algorithm j. Under the null hypothesis that all are equivalent,

$$\chi_F^2 = \frac{12N}{k(k+1)}\left[\sum_{j=1}^{k} R_j^2 - \frac{k(k+1)^2}{4}\right]$$

is approximately χ^2 with $k-1$ degrees of freedom (Demšar also gives a less conservative F-distributed variant). If the null is rejected, the **Nemenyi post-hoc test** declares two algorithms different when their average ranks differ by more than the critical difference $\mathrm{CD} = q_\alpha\sqrt{k(k+1)/(6N)}$, where q_α comes from the Studentized range statistic divided by $\sqrt{2}$; Demšar's *critical difference diagrams* display the result compactly [16]. The datasets, not the folds, are the independent units here.

Benavoli, Corani, Demšar and Zaffalon argue for going further and replacing null-hypothesis significance testing with **Bayesian analysis** of the differences, which reports the probability that one classifier is practically better, practically equivalent within a region of practical equivalence, or worse [17]. Whatever the test, remember that a p-value says nothing about *how large* the difference is; report effect sizes and intervals as well.

Measure decisions and probabilities

Accuracy measures the proportion correct; **balanced accuracy** averages per-class recall, so it is not dominated by the majority class. **Macro-F1** weights classes equally when averaging classwise F1. A confusion matrix shows which errors occur. For rare positives, examine precision and recall at an operational threshold, and the full precision–recall or ROC curve when no threshold is fixed yet.

Log loss and the Brier score assess probability predictions:

$$\mathrm{LogLoss} = -\frac{1}{N}\sum_{n} \log p_n(y_n), \qquad \mathrm{Brier} = \frac{1}{N}\sum_{n}(p_n - y_n)^2,$$

where $p_n(y_n)$ is the probability the model gave to the true class of example n, and in the Brier score (binary case) p_n is the predicted probability of the positive class and $y_n \in \{0,1\}$. Both are *proper scoring rules*: they are optimized in expectation by reporting the true probabilities. A **reliability diagram** compares predicted probability with observed frequency in bins. Good ranking, good accuracy and good calibration are different properties: random forests average votes and tend to push probabilities toward the middle, boosting with many rounds can push them toward the extremes, and a model can have excellent ROC AUC with poorly calibrated scores.

Fit a calibrator (scikit-learn offers sigmoid and isotonic mappings [26]) on predictions from data excluded from the corresponding base-model fitting. If observations are grouped, the calibration

folds must respect those groups too. Calibration does not automatically repair domain shift.

Cluster validity as model selection

Clustering has no labels, so "validation" means something different. Theodoridis and Koutroumbas devote Chapter 16 to cluster validity because a tidy partition need not serve the task [1]. The full set of indices (silhouette, Davies–Bouldin, Calinski–Harabasz, gap statistic, BIC for mixtures) is developed alongside the algorithms in PR 07; here we place them in the validation framework of this chapter.

Internal criteria use only features and assignments. The silhouette of point i compares its mean distance to its own cluster, $a(i)$, with the smallest mean distance to another cluster, $b(i)$:

$$s(i) = \frac{b(i) - a(i)}{\max(a(i), b(i))} \in [-1, 1].$$

It favors separation in the selected geometry. Choosing the number of clusters by maximizing an internal index is a model-selection step, exactly like tuning a hyperparameter, and a high score is not evidence that groups represent the intended semantic categories.

External criteria compare assignments with independent reference labels. The adjusted Rand index (ARI) corrects pairwise agreement for chance and is invariant to label permutations, so it does not matter which cluster is called "1." If labels select the algorithm or the number of clusters, those labels are no longer an untouched external assessment; the separation between selection and evaluation applies here too.

Stability compares partitions after resampling or perturbation, much like the bootstrap: cluster several subsamples and measure agreement (for example ARI) on the observations they share, accounting for label permutation. Stable groups can still reflect an irrelevant camera or acquisition artifact.

Modern view

Surveys of ensemble learning. Dong, Yu, Cao, Shi and Ma survey ensemble learning broadly [21]: they review the mainstream ensemble approaches, classify them by their characteristics, list challenges and research directions for each, and discuss combining ensembles with deep learning and reinforcement learning. Ganaie, Hu, Malik, Tanveer and Suganthan review **ensemble deep learning** [22], organizing methods into bagging-, boosting- and stacking-based deep ensembles, negative-correlation learning, explicit versus implicit ensembles (for example snapshot or dropout-style ensembles that share weights), homogeneous versus heterogeneous members, and decision-fusion strategies. Across all of these categories the practical question is the one posed by the variance formula above: how to obtain *diverse* members cheaply.

Tree ensembles on tabular data. Despite the rise of deep networks, gradient-boosted trees (XGBoost [9] and its histogram-based successors) and random forests [5] remain the default for

medium-sized tabular problems, which in recognition include hand-crafted descriptors, metadata and the outputs of other models. Grinsztajn, Oyallon and Varoquaux benchmarked tree-based models and deep architectures on 45 datasets and found that tree-based models remained state of the art at around 10,000 samples, attributing the gap to neural networks' sensitivity to uninformative features, their rotation invariance, and their difficulty learning irregular functions [23].

Ensembles for uncertainty. Lakshminarayanan, Pritzel and Blundell showed that **deep ensembles**, several networks trained from different random initializations with a proper scoring rule, give predictive uncertainty that matched or beat approximate Bayesian methods in their experiments and degrade more gracefully on out-of-distribution inputs [24]. This links the bagging intuition (disagreement signals variance) to the Bayesian model averaging of PR 08, and it is the baseline that newer uncertainty methods are compared against.

Bias–variance revisited. Belkin, Hsu, Ma and Mandal documented **double descent** [25]: as capacity grows past the point where a model can interpolate the training data, test error can fall again. It does not overturn the decomposition, but it warns that "more parameters means more variance" is not a law, and that the classical U-shaped curve describes only part of the capacity range.

What cross-validation estimates. Arlot and Celisse's survey [18] remains the reference on CV's theory for risk estimation and model selection. Bates, Hastie and Tibshirani showed that CV estimates the average error of models fit on other training sets of the same size, not the error of the particular model in hand, and that standard CV confidence intervals can have poor coverage; they proposed a nested-CV estimate of the variance that gives intervals with better coverage [19]. Cawley and Talbot's warning about overfitting the selection criterion [13] has grown more relevant with automated hyperparameter search, where thousands of configurations are compared on the same folds.

Comparing algorithms. Practice has moved from naive resampled t-tests toward McNemar and corrected tests for one dataset [14], [15], rank-based Friedman/Nemenyi tests with critical-difference diagrams across datasets [16], and Bayesian comparisons with a region of practical equivalence [17]. Raschka's tutorial [20] is a practical guide to all of these.

Open problems. Honest evaluation under distribution shift (new cameras, sites or populations), where no random split mirrors deployment; leakage through large pretrained models whose pretraining data may contain the test images; reliable uncertainty from ensembles that are cheap at inference time; and statistical comparison when only a single large benchmark exists. ML 10 continues these themes for deep vision models.

Key takeaways

- For squared error, expected test error = noise + bias^2 + variance; flexible learners such as deep trees trade bias for variance.
- Averaging M predictors with pairwise correlation ρ leaves variance $\sigma^2[\rho + (1-\rho)/M]$: bagging and random forests help unstable learners, and decorrelating members (random feature

subsets) is the lever.
- AdaBoost is greedy forward-stagewise minimization of exponential loss; gradient boosting fits trees to the negative gradient of any differentiable loss, with shrinkage and early stopping as the main regularizers.
- Voting, boosting, Bayesian model averaging and stacking combine models differently; stacking must be trained on out-of-fold predictions.
- Validation estimates a *procedure*: every adaptive choice belongs inside the loop, splits must respect groups and time, and nested CV is needed to report a tuned model's performance.
- The bootstrap's .632+ estimator corrects .632's optimism for overfitting learners; out-of-bag scores ignore group structure.
- Do not run a naive t-test on overlapping CV folds; use McNemar, 5×2cv, the corrected resampled t-test, or Friedman/Nemenyi across datasets, and report effect sizes.
- Internal cluster indices, external agreement (ARI) and stability answer different questions; choosing K with them is model selection.

Exercises

1. **Correlation floor.** Derive the averaging-variance formula by expanding the variance of a sum. With $\rho = 0.8$, how many predictors are needed to bring the variance within 1% of its limit? What does this say about bagging very similar models?

   Hint

   The sum has M variance terms and $M(M-1)$ ordered covariance terms $\rho\sigma^2$. The limit is $\rho\sigma^2 = 0.8\sigma^2$; the excess is $(1-\rho)\sigma^2/M = 0.2\sigma^2/M$. Requiring $0.2/M \le 0.008$ gives $M \ge 25$. Beyond a few dozen highly correlated models almost nothing is gained; the only way to go lower is to reduce ρ.

2. **AdaBoost by hand.** Five points have weights 0.2 each. The first stump misclassifies exactly one of them. Compute ϵ_1, α_1 and the new normalized weights. What weighted error does the same stump have under the new weights, and why is that no accident?

   Hint

   $\epsilon_1 = 0.2$, $\alpha_1 = \frac{1}{2}\log 4 = \log 2 \approx 0.693$. Unnormalized weights: $0.2 \cdot 2 = 0.4$ for the misclassified point and $0.2/2 = 0.1$ for the others; total 0.8, so the new weights are 0.5 and 0.125 (four times). The stump's new weighted error is exactly 0.5: with the optimal α, AdaBoost always reweights so that the last learner is no better than chance, forcing the next learner to find something new.

3. **Leaky folds.** In the nested-CV code, replace every `GroupKFold` by `KFold(3, shuffle=True, random_state=0)` (and drop the `groups` arguments). Compare the outer scores with the group-disjoint test score, and explain which deployment question each split answers.

   Hint

   With row-level folds, two of a subject's three near-identical measurements are often in training when the third is validated, so the forest can recognize the subject rather than the class; inner and outer

scores should rise above the independent group test. Row-level folds answer "new measurements of known subjects"; group folds answer "new subjects."

4. **Bootstrap vs. CV for a stable learner.** Rerun the bootstrap code with `KNeighborsClassifier(n_neighbors=15)`. How do resubstitution, out-of-bootstrap, .632, .632+ and 10-fold CV compare now, and why does the gap between .632 and .632+ shrink?

   Hint

   A 15-NN classifier no longer has zero training error, so the relative overfitting rate $\hat{R}$ is much smaller, $\hat{w}$ is close to 0.632, and the two estimators nearly coincide. All estimates should land within a few points of each other.

5. **Choosing a test.** You have (a) one fixed benchmark split and the predictions of two detectors; (b) one small dataset and enough compute to retrain each model ten times; (c) five algorithms evaluated on twelve public datasets. Which test fits each case, and what are the independent units in each?

   Hint

   (a) McNemar's test (or an exact binomial test) on the paired disagreements; the units are test images, and training-set variability is not captured. (b) The 5×2cv t-test or the corrected resampled t-test; the units are resamples, corrected for overlap. (c) Friedman test with Nemenyi post-hoc (or a Bayesian hierarchical comparison); the units are datasets.

6. **Clusters that mean the wrong thing.** Construct (on paper or with `make_blobs`) a dataset of object images that clusters strongly by camera while the class labels describe object identity. Which validity measures would expose the mismatch, and which would not?

   Hint

   Internal indices such as silhouette would score the camera clusters highly, because they are well separated. External ARI against object labels would be low, while ARI against camera metadata would be high. Stability would also be high, so stability alone cannot reveal that the grouping is semantically wrong.

References

1. S. Theodoridis and K. Koutroumbas, *Pattern Recognition*, 4th ed., Academic Press, 2009. Chapter 4 (decision trees, combining classifiers, boosting), Chapter 10 ("Supervised Learning: The Epilogue") and Chapter 16 ("Cluster Validity"). doi
2. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006. Sections 1.3 and 3.2 (model selection, bias–variance) and 14.1–14.5 (Bayesian model averaging, committees, boosting, trees, conditional mixtures). official book and PDF

3. S. Geman, E. Bienenstock and R. Doursat, “Neural networks and the bias/variance dilemma,” *Neural Computation*, vol. 4, no. 1, pp. 1–58, 1992. doi
4. L. Breiman, “Bagging predictors,” *Machine Learning*, vol. 24, no. 2, pp. 123–140, 1996. doi
5. L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001. doi
6. Y. Freund and R. E. Schapire, “A decision-theoretic generalization of on-line learning and an application to boosting,” *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997. doi
7. J. Friedman, T. Hastie and R. Tibshirani, “Additive logistic regression: A statistical view of boosting (with discussion),” *The Annals of Statistics*, vol. 28, no. 2, 2000. doi
8. J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” *The Annals of Statistics*, vol. 29, no. 5, 2001. doi
9. T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pp. 785–794, 2016. doi
10. D. H. Wolpert, “Stacked generalization,” *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992. doi
11. B. Efron and R. Tibshirani, “Improvements on cross-validation: The .632+ bootstrap method,” *Journal of the American Statistical Association*, vol. 92, no. 438, pp. 548–560, 1997. doi
12. S. Varma and R. Simon, “Bias in error estimation when using cross-validation for model selection,” *BMC Bioinformatics*, vol. 7, 91, 2006. doi
13. G. C. Cawley and N. L. C. Talbot, “On over-fitting in model selection and subsequent selection bias in performance evaluation,” *Journal of Machine Learning Research*, vol. 11, pp. 2079–2107, 2010. JMLR
14. T. G. Dietterich, “Approximate statistical tests for comparing supervised classification learning algorithms,” *Neural Computation*, vol. 10, no. 7, pp. 1895–1923, 1998. doi
15. C. Nadeau and Y. Bengio, “Inference for the generalization error,” *Machine Learning*, vol. 52, no. 3, pp. 239–281, 2003. doi
16. J. Demšar, “Statistical comparisons of classifiers over multiple data sets,” *Journal of Machine Learning Research*, vol. 7, pp. 1–30, 2006. JMLR
17. A. Benavoli, G. Corani, J. Demšar and M. Zaffalon, “Time for a change: A tutorial for comparing multiple classifiers through Bayesian analysis,” *Journal of Machine Learning Research*, vol. 18, no. 77, pp. 1–36, 2017. JMLR
18. S. Arlot and A. Celisse, “A survey of cross-validation procedures for model selection,” *Statistics Surveys*, vol. 4, pp. 40–79, 2010. doi
19. S. Bates, T. Hastie and R. Tibshirani, “Cross-validation: What does it estimate and how well does it do it?,” *Journal of the American Statistical Association*, vol. 119, no. 546, pp. 1434–1445, 2024. doi
20. S. Raschka, “Model evaluation, model selection, and algorithm selection in machine learning,” arXiv:1811.12808, 2018. arXiv
21. X. Dong, Z. Yu, W. Cao, Y. Shi and Q. Ma, “A survey on ensemble learning,” *Frontiers of Computer Science*, vol. 14, no. 2, pp. 241–258, 2020. doi
22. M. A. Ganaie, M. Hu, A. K. Malik, M. Tanveer and P. N. Suganthan, “Ensemble deep learning: A review,” *Engineering Applications of Artificial Intelligence*, vol. 115, 105151, 2022. doi
23. L. Grinsztajn, E. Oyallon and G. Varoquaux, “Why do tree-based models still outperform deep learning on typical tabular data?,” in *Advances in Neural Information Processing Systems*

(*NeurIPS*) *Datasets and Benchmarks Track*, 2022 (arXiv:2207.08815). arXiv

24. B. Lakshminarayanan, A. Pritzel and C. Blundell, “Simple and scalable predictive uncertainty estimation using deep ensembles,” in *Advances in Neural Information Processing Systems* (*NIPS*), 2017 (arXiv:1612.01474). arXiv
25. M. Belkin, D. Hsu, S. Ma and S. Mandal, “Reconciling modern machine-learning practice and the classical bias–variance trade-off,” *Proceedings of the National Academy of Sciences*, vol. 116, no. 32, pp. 15849–15854, 2019. doi
26. scikit-learn developers, *scikit-learn User Guide*: ensemble methods, cross-validation, probability calibration and clustering performance evaluation.

PR 10 • Recognition Capstone: Rejection, Shift and Learned Features

Build an end-to-end, reproducible recognition project on scikit-learn's digits: frame the task, lock a test set, compare baselines and classifiers with nested validation, analyze errors, then test rejection, unknown classes and distribution shift.

Online edition: https://blog.shyandram.dev/en/learn/pattern-recognition/pr-10-recognition-capstone/

What you'll learn

- How to turn a vague goal ("recognize handwritten digits") into a written protocol: observation, sampling unit, loss, reject option and what each data split may be used for.
- How to run an honest comparison of the track's classifiers (templates, naive Bayes, k-NN, LDA, logistic regression, kernel SVM, random forest) with one shared pipeline and nested cross-validation.
- How to decide whether a difference between two models is real, using a corrected resampled t-test instead of eyeballing fold means.
- How to do error analysis on development data, read a learning curve, and open the locked test set exactly once.
- How to separate closed-set accuracy from selective classification, unknown-class rejection and robustness to distribution shift, and how all of this carries over to pretrained neural features.

The big picture

The first nine chapters each studied one part of a recognizer: decisions and risk (PR 01), density estimation (PR 02), linear and kernel classifiers (PR 03, PR 04), features (PR 05), templates (PR 06), clustering (PR 07), approximate inference (PR 08) and ensembles and validation (PR 09). A real project needs all of them at once, in the right order, with discipline about which data each decision is allowed to see.

> **Explain like I'm five**
>
> Imagine you are training a friend to sort a big jar of buttons into ten boxes. First you agree on the rules: what counts as a "button", which boxes exist, and what to do with something weird, like a coin. Then you hide a handful of buttons in your pocket and do not show them during practice. Your friend practices on the rest, tries different sorting tricks, and looks closely at every mistake. Only at the very end do you pull the hidden buttons out of your pocket and count how many land in the right box. If you peeked at the hidden buttons during practice, the final count would make your friend look better than they really are.

Why it matters: most disappointing recognition systems do not fail because someone picked the "wrong" classifier. They fail because the evaluation answered a different question from the deployment: the test set leaked into model selection, the split mixed samples from the same person, the system was never asked what to do with something it had not seen, or nobody checked what happens when the camera moves by one pixel. Theodoridis and Koutroumbas treat system design and evaluation as part of pattern recognition itself [1], and Bishop frames the final step as a decision under a loss, with an explicit reject option [2]. This chapter builds one complete project around those ideas.

The project at a glance

Plain version. We follow nine steps. The first seven use only development data. Step 8 opens the locked test set once. Step 9 asks harder questions than "how accurate is it?"

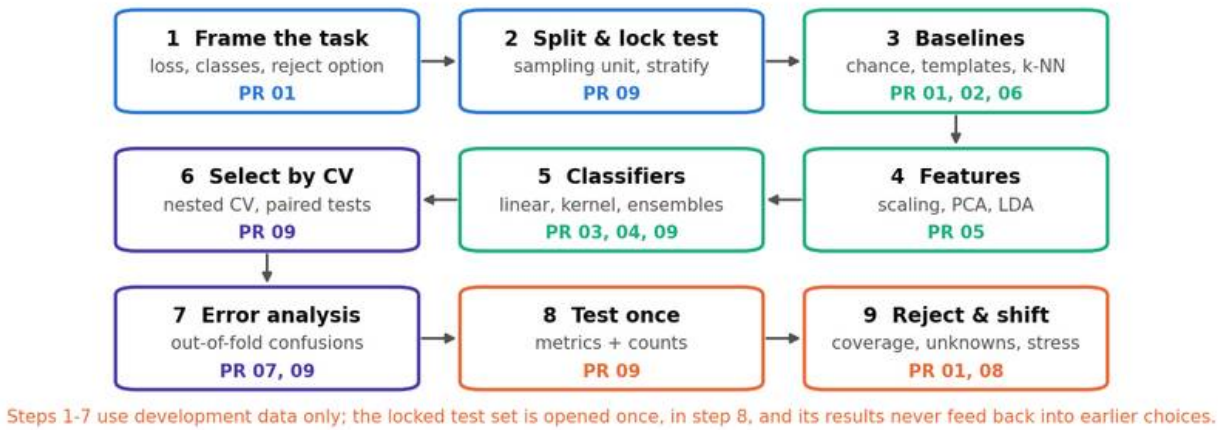


Figure 1 — The capstone workflow and the chapters each step draws on. Steps 1–7 see only development data; the test set is opened once in step 8.

Step	Question it answers	Main tools	Track chapter
1 Frame	What is an observation, a class, a mistake?	loss matrix, reject option	PR 01
2 Split	Which data may inform which decision?	stratified, grouped splits	PR 09
3 Baselines	What does "good" mean here?	chance, class means, k-NN, naive Bayes	PR 01, 02, 06
4 Features	What representation do classifiers see?	scaling, PCA, LDA	PR 05
5 Classifiers	Which model families are plausible?	linear, kernel, ensemble	PR 03, 04, 09
6 Select	Which configuration, and is the gap real?	nested CV, corrected t-test	PR 09
7 Analyze	Why does it fail, and would more data help?	out-of-fold confusions, learning curve	PR 07, 09

Step	Question it answers	Main tools	Track chapter
8 Test	How well does the frozen procedure do?	accuracy with interval, per-class report	PR 09
9 Stress	What about doubt, novelty and shift?	coverage–risk, unknown classes, perturbations	PR 01, 08

All code below runs on a laptop CPU in well under a minute and uses only scikit-learn's bundled data [4]. The snippets in steps 2 to 8 form one script: run them in order in the same Python session.

Step 1 — Frame the task

Plain version. Before touching a model, write down what one example is, what the possible answers are, what a mistake costs, and when the system is allowed to say "I don't know".

A written protocol

A protocol is a short document you write *before* experimenting. It protects you from changing the question after seeing the answer. For any recognition project, answer at least these questions:

1. **Observation.** What is one input? An image, a crop, a video clip, a whole document?
2. **Sampling unit.** Which observations share a source (a person, a camera, a patient, a day)? Observations from the same source are not independent and must stay on the same side of every split.
3. **Label space.** Which classes are known during training? What should happen to inputs from other categories?
4. **Loss.** Are all mistakes equally bad? Is abstaining allowed, and what does it cost?
5. **Data roles.** What may training data, validation data and test data each be used for?
6. **Deployment conditions.** Which devices, writers, lighting or time periods must the evaluation cover?

The decision rule we are aiming for

PR 01 showed that under zero–one loss the Bayes classifier picks the class with the largest posterior probability. Adding a reject option with cost λ_r (where a correct decision costs 0 and an error costs 1) gives the reject-option rule that Bishop presents in his decision-theory section [2]:

$$\hat{y}(x) = \begin{cases} \arg\max_k P(\omega_k \mid x) & \text{if } \max_k P(\omega_k \mid x) \geq 1 - \lambda_r, \\ \text{reject} & \text{otherwise.} \end{cases}$$

Here x is the feature vector, ω_k is class k, $P(\omega_k \mid x)$ is its posterior probability, and $0 \leq \lambda_r \leq 1$ is the cost of rejecting relative to the cost of an error. If $\lambda_r = 0$, rejecting is free and we reject everything that is not certain; if $\lambda_r \geq 1 - 1/K$ for K classes, we never reject. Everything that follows (classifier

choice, validation, threshold setting) is an attempt to approximate this rule with finite data. Keep in mind one limitation: the rule only protects against doubt *among known classes*. A completely new category can still receive a large posterior, which is why step 9 treats unknowns separately.

Our project and its data card

We will recognize handwritten digits using `sklearn.datasets.load_digits` [4]. Writing a short "data card" for it is part of the framing (this is the spirit of datasheets for datasets [24]):

- **Origin.** The data are a copy of the *test* portion of the UCI "Optical Recognition of Handwritten Digits" dataset by Alpaydin and Kaynak [3].
- **Observations.** 1,797 images. Each started as a 32×32 binary bitmap that was divided into non-overlapping 4×4 blocks; each of the resulting 8×8 cells counts the "on" pixels, so values are integers from 0 to 16 [3].
- **Classes.** Ten digits, roughly balanced (174 to 183 images per class).
- **Sources.** According to the dataset description, this portion was written by 13 people, different from the 30 who wrote the UCI training portion [3]. **Writer IDs are not provided.**

The last point shapes every claim we can make. Because we cannot group by writer, our random splits place images by the same 13 people on both sides. The test accuracy we report will therefore estimate performance on *new images from these writers*, not on *new writers*. The UCI training portion, written by different people, would allow a writer-disjoint test, but it is not bundled with scikit-learn and we do not use it here. Stating this limitation openly is part of an honest result.

Step 2 — Split the data and lock the test set

Plain version. Put a quarter of the data in a "locked drawer" right away. Every decision until step 8 is made without it.

```
import numpy as np
import warnings
warnings.filterwarnings("ignore", message=".*within_class_std_dev_.*")
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

digits = load_digits()
X, y = digits.data, digits.target          # 1797 x 64, integer intensities 0..16
print("shape:", X.shape, "range:", X.min(), X.max())
print("class counts:", np.bincount(y))

idx = np.arange(len(y))
dev, test = train_test_split(idx, test_size=0.25, stratify=y, random_state=0)
assert not set(dev) & set(test)
print("development:", len(dev), "test (locked):", len(test))
```

Output: `shape: (1797, 64) range: 0.0 16.0`, class counts between 174 and 183, `development: 1347 test (locked): 450`. (The warnings filter only silences a harmless notice that `NearestCentroid` prints because some border pixels are constant within a class.)

Why stratify? With 450 test images, a random split could easily leave one digit under-represented. Stratification keeps the class proportions the same in both parts. It does *not* fix dependence between observations; only grouping by the sampling unit does that (PR 09).

Leakage: the most common way to fool yourself

Kapoor and Narayanan surveyed reproducibility failures in machine-learning-based science and found data leakage across many fields; they organize it into a taxonomy that includes no clean train–test separation, preprocessing fitted on all data, duplicates across splits, illegitimate features, and test sets that do not match the population of interest [5]. In a recognition project, the most frequent forms are:

- fitting a scaler, PCA or feature selector on all data before splitting;
- choosing hyperparameters, features or thresholds by looking at test scores;
- near-duplicate images (consecutive video frames, augmented copies, the same person) on both sides of a split;
- a test set from a different population than the deployment (for example, one writer group when deployment involves new writers).

The remedy is structural: keep every fitted transformation inside a scikit-learn `Pipeline`, so cross-validation refits it on each training fold, and keep the test indices out of every function call until step 8.

Step 3 — Look at the data and set baselines

Plain version. Before training anything clever, look at examples and compute what simple methods achieve. A fancy model that barely beats a trivial one is not a success.

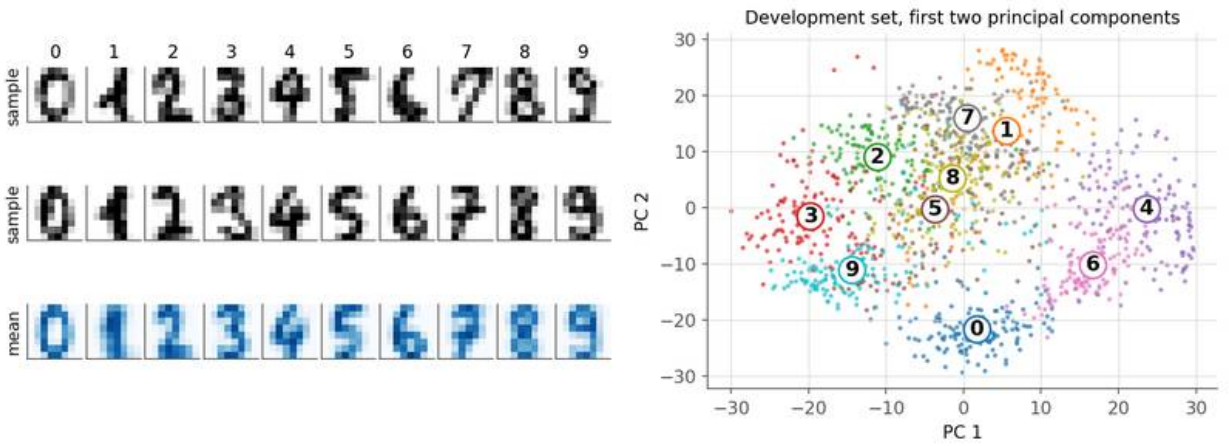


Figure 2 — Left: two development images per class and each class's mean image (the templates of a nearest-class-mean classifier). Right: the development set projected on its first two principal components. Digits 0, 4 and 6 already largely separate; 1, 7, 8, 5 and 9 overlap heavily in this 2-D view.

Figure 2 already suggests the difficulty structure. The class means look like clean digits, so template matching (PR 06) should work reasonably. In two principal components, some classes are well separated and others overlap; PR 05 explained why high-variance directions need not be the most discriminative, so overlap in two components says little about overlap in 64 dimensions.

We use four baselines, each tied to a chapter:

- **Majority class.** Always predict the most common digit. With ten balanced classes this gives about 10% accuracy. It is the floor.
- **Nearest class mean.** Compare an image with each class mean and pick the closest: the simplest template matcher.
- **Gaussian naive Bayes.** A Bayes classifier (PR 01) with class-conditional Gaussian densities whose covariance is diagonal. Pixels are strongly correlated, so the independence assumption is badly wrong here, and we expect it to suffer.
- **k-nearest neighbors.** A nonparametric density-based rule (PR 02): vote among the k closest training images.

Step 4 — Decide on features

Plain version. A classifier only sees numbers. Decide which numbers, how they are scaled, and whether to compress them, and fit any of those transformations on training folds only.

Our raw features are the 64 block counts. Three common transformations are worth comparing:

Standardization. Subtract each feature's training mean and divide by its standard deviation:

$$z_j = \frac{x_j - \bar{x}_j}{s_j},$$

where x_j is pixel j, $\bar{x}_j$ and s_j are its mean and standard deviation over the training fold, and z_j is the standardized value. This matters for distance-based and gradient-based methods when features have very different units. Here all features already share the same unit (a count from 0 to 16). Worse, the border pixels are almost always 0, so their s_j is tiny: standardizing them *amplifies* rare stray strokes into large values. We will see that this choice measurably hurts the kernel SVM.

Principal component analysis. PCA projects onto the directions of largest variance (PR 05). With d' components the representation is $u = W^\top(x - \bar{x})$, where W holds the top d' eigenvectors of the training covariance. It removes noise directions and speeds up kernels, but variance is not the same as class information.

Linear discriminant analysis. LDA (PR 03, PR 05) finds at most $K - 1 = 9$ directions that maximize between-class over within-class scatter. We use it as a classifier with shrinkage, which regularizes the within-class covariance estimate.

> **Features are hyperparameters**
>
> Whether to scale, how many PCA components to keep, or which hand-made features to add are choices made with data. Treat them exactly like a regularization constant: put them in the pipeline and let validation choose them. Fitting them once on all the data before cross-validation is leakage.

For real images, this is also where you would add engineered features (HOG, LBP, moments, see the feature extraction chapter) or pretrained embeddings. At 8×8 resolution, the raw counts are a reasonable starting representation.

Step 5 — Compare candidate classifiers

Plain version. Run every candidate through the *same* cross-validation splits of the development set and compare their score distributions, not just their best fold.

```
from sklearn.model_selection import RepeatedStratifiedKFold, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.dummy import DummyClassifier
from sklearn.neighbors import NearestCentroid, KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier

candidates = {
    "majority class (PR 01)": DummyClassifier(strategy="most_frequent"),
    "nearest class mean (PR 06)": NearestCentroid(),
    "Gaussian naive Bayes (PR 01)": GaussianNB(),
    "k-NN, k=3 (PR 02)": KNeighborsClassifier(n_neighbors=3),
    "LDA, shrinkage (PR 03/05)": LinearDiscriminantAnalysis(
```

```
        solver="lsqr", shrinkage="auto"),
    "logistic regression (PR 03)": make_pipeline(
        StandardScaler(), LogisticRegression(C=1.0, max_iter=3000)),
    "PCA-30 + RBF SVM (PR 04/05)": make_pipeline(
        StandardScaler(), PCA(n_components=30, random_state=0),
        SVC(C=10, gamma="scale")),
    "random forest (PR 09)": RandomForestClassifier(
        n_estimators=300, random_state=0, n_jobs=1),
}
cv = RepeatedStratifiedKFold(n_splits=5, n_repeats=2, random_state=0)
cv_scores = {}
for name, model in candidates.items():
    s = cross_val_score(model, X[dev], y[dev], cv=cv, scoring="accuracy")
    cv_scores[name] = s
    print(f"{name:30s} {s.mean():.3f} +/- {s.std():.3f}")
```

Output (mean $\pm$ standard deviation over the 10 folds):

Candidate	Mean CV accuracy	Std
majority class	0.100	0.000
Gaussian naive Bayes	0.827	0.017
nearest class mean	0.897	0.011
LDA with shrinkage	0.951	0.007
logistic regression (scaled)	0.966	0.012
random forest, 300 trees	0.975	0.010
scaled PCA-30 + RBF SVM	0.976	0.010
3-nearest neighbors	0.985	0.007

Several lessons are already visible:

- **The ranking of simple models follows their assumptions.** Naive Bayes, whose independence assumption is violated by correlated neighboring pixels, is the weakest real model. A single template per class (nearest mean) cannot capture different writing styles of the same digit. LDA, which models a shared full covariance, does much better.
- **The best untuned model is the simplest nonparametric one.** 3-NN beats every untuned parametric model. With 1,347 small images and only 13 writers, many test images have a near twin in the training folds, which is exactly the situation where nearest neighbors shine. This also reminds us that our split measures "new images from known writers".
- **Defaults are not a fair comparison.** The SVM and the logistic regression ran with a standardization step and default-ish settings. Before concluding anything, the strong candidates deserve tuning, under a protocol that does not bias the comparison.

Step 6 — Tune and select without fooling yourself

Plain version. Tuning is part of the method. To estimate how good "SVM *with* tuning" is, you must repeat the whole tuning inside each outer fold. Then ask whether the remaining gap to the next-best model is bigger than the noise.

Nested cross-validation

The inner loop chooses hyperparameters; the outer loop scores the *whole procedure*. If you report the best inner score directly, you report the maximum of many noisy numbers, which is optimistically biased. Varma and Simon showed this bias empirically for cross-validated model selection [6], and Cawley and Talbot showed that over-fitting the model-selection criterion can produce performance differences as large as the differences between learning algorithms [7]. Raschka's tutorial review collects the practical recipes [9].

```
from sklearn.model_selection import GridSearchCV, StratifiedKFold
from sklearn.pipeline import Pipeline

svm = Pipeline([("scale", StandardScaler()), ("pca", PCA(random_state=0)),
                ("svc", SVC())])
grid = {"scale": [StandardScaler(), "passthrough"],
        "pca": [PCA(30, random_state=0), "passthrough"],
        "svc__C": [1, 10],
        "svc__gamma": ["scale", 0.001]}
inner = StratifiedKFold(5, shuffle=True, random_state=1)
outer = StratifiedKFold(5, shuffle=True, random_state=2)
search = GridSearchCV(svm, grid, cv=inner, scoring="accuracy")
nested = cross_val_score(search, X[dev], y[dev], cv=outer)
print("nested-CV accuracy of the tuning procedure:",
      nested.round(3), "mean", nested.mean().round(3))
search.fit(X[dev], y[dev])
print("chosen on all development data:", search.best_params_)
```

Output: outer-fold accuracies `[0.993 0.993 0.985 0.985 0.993]`, mean `0.99`. The configuration chosen on all development data is **no standardization, PCA with 30 components,** $C = 10$**,** $\gamma = 0.001$.

Two observations. First, the grid includes the preprocessing decision itself (`"passthrough"` means "skip this step"), and validation rejected standardization, confirming the argument of step 4. Second, the nested estimate (0.990) describes the *procedure* "run this grid search, then refit", which is what we will deploy. It is not a property of one particular hyperparameter setting.

Is the winner really better?

The tuned SVM (0.990 nested) and 3-NN (0.985) differ by half a percentage point. Is that real? A naive paired t-test on fold scores treats the folds as independent samples, but the training sets of different folds overlap heavily, so the fold scores are positively correlated and the naive test is too confident. Nadeau and Bengio proposed a corrected variance for resampled designs [8]:

$$t = \frac{\bar{d}}{\sqrt{\left(\frac{1}{k} + \frac{n_{\text{te}}}{n_{\text{tr}}}\right)\hat{\sigma}_d^2}},$$

where d_i is the score difference between the two models on resample i, $\bar{d}$ and $\hat{\sigma}_d^2$ are the mean and sample variance of the k differences, and n_{tr} and n_{te} are the training and test sizes of each resample. The extra term $n_{\text{te}}/n_{\text{tr}}$ inflates the variance to account for overlap. The statistic is compared with a t distribution with $k-1$ degrees of freedom. PR 09 derives this and discusses alternatives such as McNemar's test.

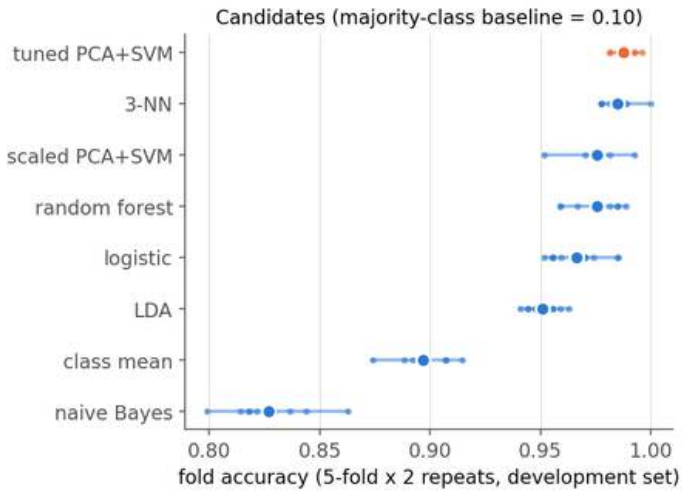
```
from scipy import stats

def corrected_t(a, b, n_train, n_test):
    """Nadeau-Bengio corrected resampled t-test on paired fold scores."""
    d = np.asarray(a) - np.asarray(b)
    k = len(d)
    var = d.var(ddof=1) * (1 / k + n_test / n_train)
    t = d.mean() / np.sqrt(var)
    return t, 2 * stats.t.sf(abs(t), df=k - 1)

n_test_fold = len(dev) // 5
tuned = cross_val_score(search.best_estimator_, X[dev], y[dev], cv=cv)
t, p = corrected_t(tuned,
                   cv_scores["k-NN, k=3 (PR 02)"],
                   n_train=len(dev) - n_test_fold, n_test=n_test_fold)
print(f"tuned SVM vs k-NN: corrected t = {t:.2f}, p = {p:.3f}")
```

Output: `tuned SVM vs k-NN: corrected t = 1.03, p = 0.331`.

The difference is *not* statistically distinguishable on this data. (Strictly, the tuned configuration was itself chosen on these development data, which makes the comparison slightly favorable to the SVM; even so it is not significant.) In a real project this is a decision point, not a failure. Both are reasonable. Other criteria then break the tie: the SVM stores support vectors and needs a kernel evaluation per support vector, while k-NN stores every training image and searches them all; the SVM's decision values can be calibrated into probabilities; k-NN has a single interpretable hyperparameter. We proceed with the tuned SVM pipeline because it had the higher nested estimate and a more compact model, and we record that 3-NN is a statistically tied alternative.



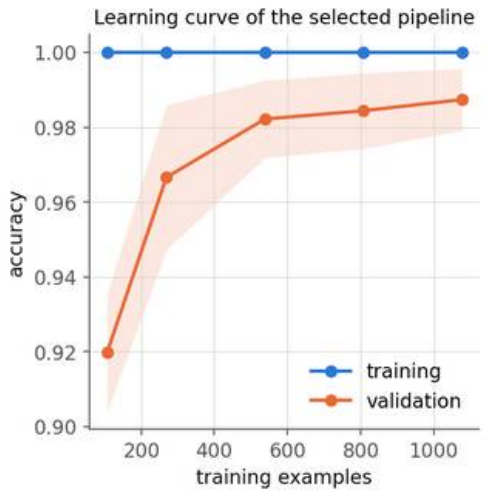


Figure 3 — Left: accuracy on each of the 10 development folds (small dots), with the mean (large dot) and range (bar). The tuned pipeline (orange) uses the configuration chosen by the search. Right: learning curve of the selected pipeline; the shaded band is ±1 standard deviation across folds.

Step 7 — Error analysis on development data

Plain version. Look at the mistakes, but look at *development* mistakes. Once you start redesigning the system because of test errors, the test set stops being a test.

Out-of-fold predictions

`cross_val_predict` returns, for each development image, the prediction of a model that did not train on it. These out-of-fold predictions are a legitimate place to study errors.

```
from sklearn.model_selection import cross_val_predict
from sklearn.metrics import confusion_matrix

chosen = search.best_estimator_
oof = cross_val_predict(chosen, X[dev], y[dev],
                        cv=StratifiedKFold(5, shuffle=True, random_state=3))
cm_dev = confusion_matrix(y[dev], oof)
off = cm_dev.copy()
np.fill_diagonal(off, 0)
order = np.argsort(off.ravel())[::-1][:5]
print("out-of-fold errors:", int(off.sum()), "of", len(dev))
for t_, p_ in zip(*np.unravel_index(order, off.shape)):
    print(f"  true {t_} -> predicted {p_}: {off[t_, p_]}")
dev_errors = dev[oof != y[dev]]
print("first misclassified dataset indices:", dev_errors[:8])
```

Output: 17 errors out of 1,347. The most frequent confusions are 9 → 8 (3 times), 5 → 9 (3), 8 → 1 (2), then single errors such as 9 → 5 and 6 → 8.

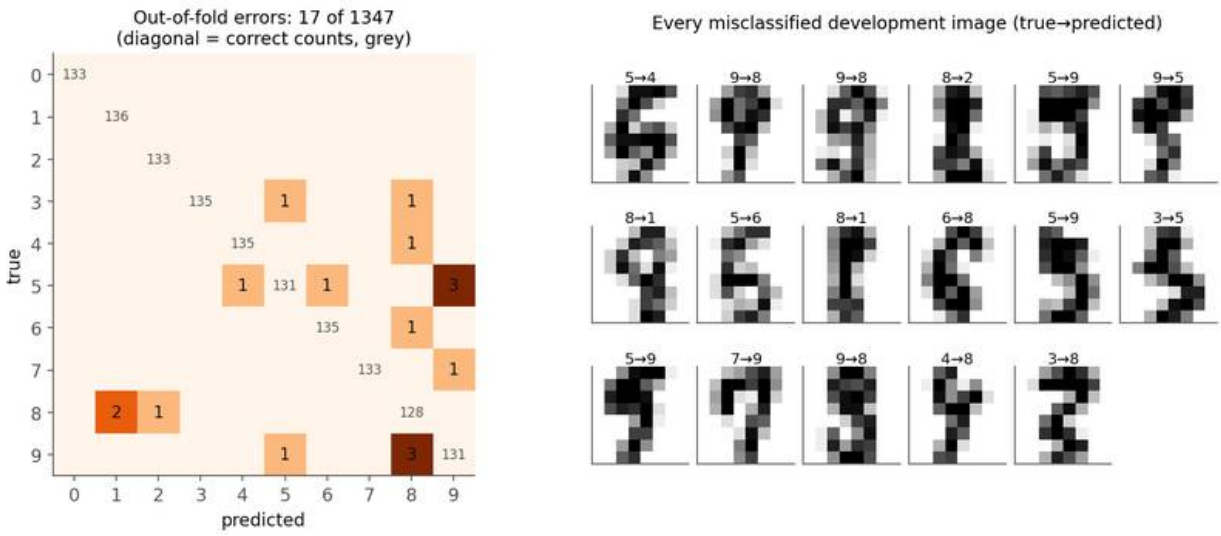


Figure 4 — Out-of-fold error analysis of the selected pipeline. Left: off-diagonal counts (correct counts shown small on the diagonal). Right: every misclassified development image, labeled true→predicted.

Reading Figure 4 is where domain knowledge enters:

- **Systematic confusions.** The errors are concentrated among digits with closed upper loops (5, 8, 9) and thin vertical strokes (1, 8). At 8 × 8 resolution, the gap that distinguishes a 9 from an 8 may be a single cell.
- **Label or quality issues.** Some misclassified images are ambiguous even to a human. In a larger project you would review such cases and decide whether they are labeling errors (to fix), genuinely ambiguous inputs (candidates for rejection, step 9), or ordinary failures (to address with features or data).
- **Grouping errors.** With hundreds of errors you cannot inspect them one by one. A common trick is to cluster the misclassified examples in a feature space (PR 07) and inspect a few members of each cluster; clusters often correspond to one failure mode, such as a writing style or an acquisition problem.

A good error analysis ends with *hypotheses*, each paired with an experiment that can be run on development data: "a one-pixel shift invariance might help 5/9 confusions; try augmenting training folds with shifted copies and re-run the nested comparison."

Would more data help? The learning curve

```
from sklearn.model_selection import learning_curve

sizes, tr, va = learning_curve(
    chosen, X[dev], y[dev], cv=StratifiedKFold(5, shuffle=True, random_state=3),
    train_sizes=[0.1, 0.25, 0.5, 0.75, 1.0])
for n, a, b in zip(sizes, tr.mean(1), va.mean(1)):
```

```
    print(f"n={n:4d}  train={a:.3f}  validation={b:.3f}")
```

Training images	Training accuracy	Validation accuracy
107	1.000	0.920
269	1.000	0.967
538	1.000	0.982
807	1.000	0.984
1,077	1.000	0.987

The training accuracy is perfect at every size, and the validation accuracy is still rising slowly at the right edge (Figure 3, right). In bias–variance terms (PR 09) the remaining error is mostly *variance*: the model is flexible enough, and more (or more varied) training data would likely help a little. Had both curves plateaued together at a low value, the advice would be the opposite: a more expressive model or better features.

Step 8 — Open the test set, once

Plain version. Freeze every choice, run the final pipeline on the locked test images, and report the result with its uncertainty and its limits.

```
from sklearn.metrics import (accuracy_score, balanced_accuracy_score,
                             classification_report)

final = search.best_estimator_            # already refit on all of dev
pred = final.predict(X[test])
print("TEST accuracy:", round(accuracy_score(y[test], pred), 4),
      "balanced:", round(balanced_accuracy_score(y[test], pred), 4),
      "errors:", int((pred != y[test]).sum()), "of", len(test))
cm = confusion_matrix(y[test], pred)
print(cm)
print(classification_report(y[test], pred, digits=3))

k, n, z = int((pred == y[test]).sum()), len(test), 1.96   # Wilson interval
phat = k / n
centre = (phat + z**2 / (2 * n)) / (1 + z**2 / n)
half = z * np.sqrt(phat * (1 - phat) / n + z**2 / (4 * n**2)) / (1 + z**2 / n)
print(f"95% Wilson interval for accuracy: [{centre - half:.3f}, {centre + half:.3f}]")
```

Output: test accuracy **0.9933** (3 errors out of 450: one each of 3 → 7, 8 → 1 and 9 → 5), balanced accuracy 0.9933, per-class recall between 0.977 and 1.000, and a 95% Wilson interval of **[0.981, 0.998]**.

How to read and report this:

- **Report counts, not just percentages.** "3 errors out of 450" tells the reader immediately how much the third decimal can be trusted.
- **Report an interval.** For an accuracy close to 1, the usual normal approximation $\hat{p} \pm 1.96\sqrt{\hat{p}(1-\hat{p})/n}$ can exceed 1 and is unreliable; the Wilson score interval used above stays inside $[0, 1]$ and behaves better near the boundary. With n test examples, k correct and $\hat{p} = k/n$, its center is $(\hat{p} + z^2/2n)/(1 + z^2/n)$ with $z = 1.96$ for 95% coverage.
- **Compare with the development estimate.** The test result (0.993) is consistent with the nested estimate (0.990). If it had been much lower, the likely causes would be leakage in development or a test population that differs from development, and either deserves investigation, but a fix would then need a *new* untouched test set.
- **State the scope.** This is accuracy on new images from the same 13 writers (step 1). It is not evidence about new writers, scanners or preprocessing.

The protocol note in the code matters: `search.best_estimator_` was refit on all development data once, before testing. Refitting again on development plus test and then reporting the test score would be meaningless. For deployment you *may* refit the frozen procedure on all labeled data, but the reported number must come from the run above.

Step 9 — Beyond closed-set accuracy

A recognizer that is 99% accurate on a balanced test of known digits can still be useless if it confidently labels a letter as a digit, or collapses when the scanner shifts the image. This step tests three questions the closed-set number cannot answer.

Four tasks that are easy to confuse

Task	What the system must do	What the evaluation must contain
Closed-set classification	choose among trained classes	independent examples of known classes
Selective classification	abstain on difficult known examples	coverage and error on accepted known examples
Open-set recognition	classify known classes and reject unknown categories	held-out categories as well as known examples
Anomaly detection	flag deviations from a reference population	a defined normal population and relevant deviations

A low-confidence known example is not necessarily an unknown category; an unfamiliar category can receive a high confidence; and a valid known object photographed badly may be rejected. These tasks need different data and different metrics.

Selective classification: coverage and risk

Plain version. Let the classifier skip the inputs it is least sure about and measure two things: how many inputs it still answers, and how often those answers are wrong.

For a confidence score $q(x)$ and an acceptance threshold τ, over N known-class test examples with predictions $\hat{y}_n$ and labels y_n:

$$\text{coverage}(\tau) = \frac{1}{N}\sum_{n=1}^{N} \mathbf{1}[q(x_n) \geq \tau],$$

$$\text{selective risk}(\tau) = \frac{\sum_n \mathbf{1}[q(x_n) \geq \tau]\,\mathbf{1}[\hat{y}_n \neq y_n]}{\sum_n \mathbf{1}[q(x_n) \geq \tau]}.$$

Here $\mathbf{1}[\cdot]$ is 1 when its condition holds and 0 otherwise. Selective risk is undefined when nothing is accepted. Plotting risk against coverage while sweeping τ gives the risk–coverage curve; Geifman and El-Yaniv showed how to choose a threshold for deep networks that guarantees a desired risk with high probability [11]. A useful threshold must be chosen on validation data and then only *reported* on test data.

Open-set recognition: held-out categories

Plain version. Pretend some digits do not exist during training, then check whether the system rejects them when they show up.

Scheirer and colleagues formalized open-set recognition: a classifier must limit the "open space" it labels as known, because a model trained on K classes will otherwise assign every input on Earth to one of them [13]. The simplest baseline score is the maximum softmax (or class) probability, proposed by Hendrycks and Gimpel for detecting misclassified and out-of-distribution examples [12].

We redo the experiment with a reduced label space. Digits 0–5 are **known**; digits 6–7 are **proxy unknowns** that development may look at; digits 8–9 are **test unknowns** reserved for the final evaluation. Validation chooses between two predefined pipelines, and the rejection threshold is the 5th percentile of validation confidences, which targets roughly 95% acceptance of known digits. All choices are frozen before the final block. This script is self-contained.

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.metrics import balanced_accuracy_score, confusion_matrix, roc_auc_score

digits = load_digits()
```

```
X, y = digits.data, digits.target
known = np.flatnonzero(y < 6)
train, rest = train_test_split(known, test_size=0.4,
                               stratify=y[known], random_state=10)
valid, test = train_test_split(rest, test_size=0.5,
                               stratify=y[rest], random_state=10)
proxy_unknown = np.flatnonzero((y == 6) | (y == 7))
test_unknown = np.flatnonzero(y >= 8)
assert not set(train) & (set(valid) | set(test))
assert not set(proxy_unknown) & set(test_unknown)

candidates = {
    "linear": make_pipeline(
        StandardScaler(), LogisticRegression(C=1.0, max_iter=1500)),
    "PCA + RBF": make_pipeline(
        StandardScaler(), PCA(n_components=24, random_state=10),
        SVC(C=4.0, gamma=0.02, probability=True, random_state=10)),
}
scores = {}
for name, model in candidates.items():
    model.fit(X[train], y[train])
    validation_p = model.predict_proba(X[valid])
    validation_pred = model.classes_[validation_p.argmax(axis=1)]
    scores[name] = balanced_accuracy_score(y[valid], validation_pred)
name = max(scores, key=scores.get)
model = candidates[name]
threshold = np.quantile(model.predict_proba(X[valid]).max(axis=1), 0.05)
proxy_conf = model.predict_proba(X[proxy_unknown]).max(axis=1)
print("Validation candidates:", scores, "selected:", name)
print("Fixed threshold:", threshold)
print("Proxy-unknown acceptance:", np.mean(proxy_conf >= threshold))

# Final evaluation begins here; do not revise choices from these results.
known_p = model.predict_proba(X[test])
known_conf = known_p.max(axis=1)
known_pred = model.classes_[known_p.argmax(axis=1)]
accept = known_conf >= threshold
unknown_conf = model.predict_proba(X[test_unknown]).max(axis=1)
print("Known test balanced accuracy:",
      balanced_accuracy_score(y[test], known_pred))
print("Known coverage:", accept.mean())
print("Accepted known accuracy:",
      np.mean(known_pred[accept] == y[test][accept]) if accept.any() else np.nan)
print("Unknown-test acceptance:", np.mean(unknown_conf >= threshold))
is_unknown = np.r_[np.zeros(len(test)), np.ones(len(test_unknown))]
unknown_score = 1 - np.r_[known_conf, unknown_conf]
print("Unknown detection AUROC:", roc_auc_score(is_unknown, unknown_score))
print("Known-class confusion matrix:\n", confusion_matrix(y[test], known_pred))
```

Output, in the order printed: validation balanced accuracy 0.982 (linear) versus 0.995 (PCA + RBF), so the RBF pipeline is selected; threshold 0.741; **proxy-unknown acceptance 0.083**; known test balanced accuracy 0.981; known coverage 0.949; accepted known accuracy 0.995; **unknown-test acceptance 0.350**; unknown-detection AUROC 0.944.

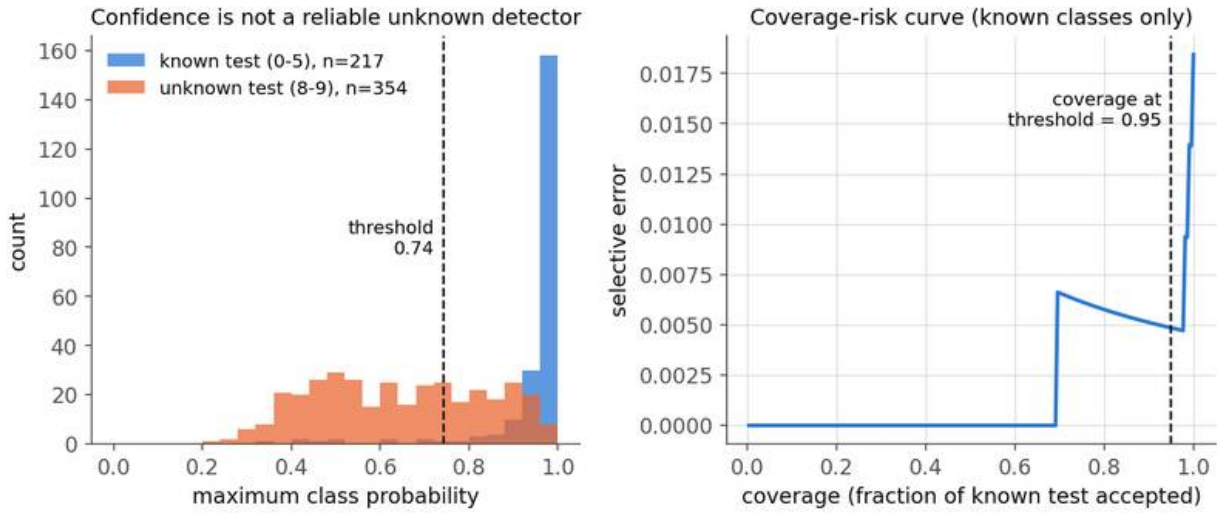


Figure 5 — Left: maximum class probability for known test digits (0–5) and unknown test digits (8–9), with the threshold fixed on validation data. Right: risk–coverage curve for known test digits; the dashed line marks the coverage of the fixed threshold.

The numbers carry the main lesson of open-set evaluation:

- On known digits, rejection works as intended: 95% coverage and 99.5% accuracy on what is accepted.
- The proxy unknowns (6 and 7) were rejected 92% of the time, which might have suggested that confidence thresholding "solves" unknowns.
- The real test unknowns (8 and 9) were **accepted 35% of the time**. An 8 looks like a 3 or a 5 with extra strokes, and a 9 looks like a 5 or a 3; the classifier confidently maps them onto those known digits.
- AUROC of 0.944 summarizes ranking over these particular unknowns. It does not fix an operating threshold and does not describe other kinds of novelty.

Performance on one set of proxy unknowns does not transfer to other unknowns. Report which categories were held out, and never claim rejection of "all unfamiliar inputs". Vaze and colleagues found that a stronger closed-set classifier tends to be a better open-set recognizer, with closed-set and open-set performance strongly correlated across methods [16], so improving the base model and using better scores (feature distances, energy or logit-based scores) are complementary.

Other families of scores model the reference population directly: a one-class SVM estimates a region that contains most of the training data [17], and an Isolation Forest flags points that random axis-aligned splits isolate quickly [18]. An "outlier" may be an acquisition error, a rare valid case or a new class; statistical unusualness does not tell you which.

Distribution shift and stress tests

Plain version. The world changes after you deploy. Ask "what if" questions with controlled perturbations and, better, with real data from new conditions.

Let $p_{\text{train}}(x, y)$ and $p_{\text{deploy}}(x, y)$ be the joint distributions of features and labels during training and deployment. Moreno-Torres and colleagues organized the vocabulary of dataset shift in classification [19]. Three cases matter most:

- **Prior (label) shift:** class frequencies $p(y)$ change, while $p(x \mid y)$ stays the same. Example: one digit becomes much more common in a new form layout.
- **Covariate shift:** $p(x)$ changes, while $p(y \mid x)$ stays the same. Example: a new scanner makes strokes thicker.
- **Concept shift:** $p(y \mid x)$ itself changes. Example: a new convention for writing 7 with a bar.

These are modeling assumptions, not diagnoses you can read off the data's name. A camera change can alter class-conditional features; adding a new class breaks the label space entirely.

We cannot collect new handwriting here, but we can probe sensitivity with controlled perturbations of the *test* images. The model is not retrained; the perturbations were decided before looking at their results.

```
rng = np.random.default_rng(0)
def shift_right(X, k=1):
    imgs = X.reshape(-1, 8, 8)
    out = np.zeros_like(imgs)
    out[:, :, k:] = imgs[:, :, :-k]
    return out.reshape(len(X), 64)

conditions = {
    "clean": X[test],
    "noise sd=2": np.clip(X[test] + rng.normal(0, 2, X[test].shape), 0, 16),
    "noise sd=4": np.clip(X[test] + rng.normal(0, 4, X[test].shape), 0, 16),
    "shift 1 px right": shift_right(X[test], 1),
    "contrast x0.5": X[test] * 0.5,
}
for name, Xc in conditions.items():
    print(f"{name:18s} accuracy = {accuracy_score(y[test], final.predict(Xc)):.3f}")
```

(Run this after the step 8 block, not after the open-set script, because it uses the 10-class `X`, `test` and `final`.)

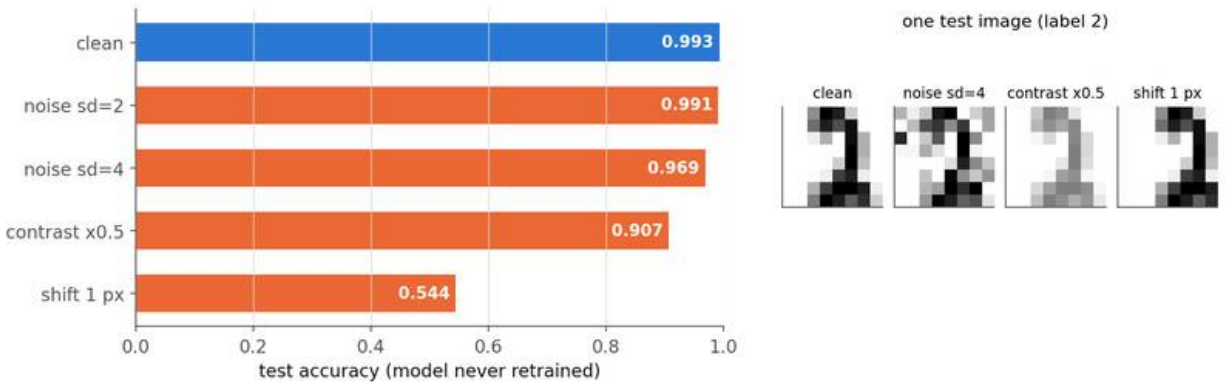


Figure 6 — Test accuracy of the frozen pipeline under perturbations it never saw in training, and one test image under each condition. Intensities stay in the dataset's 0–16 range.

Condition	Test accuracy
clean	0.993
additive noise, sd = 2	0.991
additive noise, sd = 4	0.969
contrast halved	0.907
shifted right by one cell	**0.544**

Additive noise barely matters. Halving the contrast costs about 9 points, because the RBF kernel compares raw intensities and γ was tuned for full-contrast strokes. A **one-cell translation** cuts accuracy almost in half: a pixel-vector classifier has no notion that a shifted 3 is still a 3. One cell corresponds to a 4-pixel block of the original 32×32 bitmap, an eighth of the image width, so this is a realistic misalignment for a sloppy crop. This is the classic argument for building translation tolerance into the model, by augmentation or by convolution and pooling, which LeCun and colleagues demonstrated for document recognition [10] (see ML 03).

Synthetic perturbations are a cheap first check, not a substitute for real new-condition data. Benchmarks such as WILDS collect real distribution shifts across hospitals, cameras and regions and show sizable drops from in-distribution to out-of-distribution performance [21]; Recht and colleagues rebuilt the CIFAR-10 and ImageNet test sets following the original procedures and observed accuracy drops for many models, even without any deliberate shift [20]. Report results per meaningful condition (writer, device, site, time), not just pooled.

Probabilities that mean what they say

If downstream users act on confidence values, they must be *calibrated*: among predictions made with confidence 0.8, about 80% should be correct. Guo and colleagues showed that modern deep networks are often overconfident and that a single scalar *temperature* fitted on validation data is a strong fix [22]. For SVMs, `SVC(probability=True)` fits Platt scaling internally with cross-validation; `CalibratedClassifierCV` gives explicit control. Calibration must be fitted on data not used for training the base model, and it does not repair distribution shift.

Bridge to learned and foundation features

Plain version. Swap the hand-made features for a pretrained network's features. Everything else in this chapter stays the same.

Replace the feature map with a pretrained encoder:

$$x = \phi_{\text{pretrained}}(I), \qquad \hat{y} = g_\theta(x),$$

where I is the raw image, $\phi_{\text{pretrained}}$ is a frozen network (for example a self-supervised vision transformer such as DINOv2 [25]), x is its embedding, and g_θ is a classifier with parameters θ trained on your labels. A **linear probe** (logistic regression on frozen embeddings) is the standard baseline; compare it with k-NN on embeddings, a regularized nonlinear head and fine-tuning, holding splits and metrics fixed. ML 05 explains how such features are learned and ML 09 how they are adapted.

The protocol extends naturally:

- record the checkpoint, input resizing, normalization, pooling and embedding dimension;
- fit downstream preprocessing (scaling, PCA) only on training embeddings;
- count the choice of checkpoint, prompt or layer as model selection, made on validation data;
- if evaluation images could have appeared in the pretraining data, state what evidence you have about overlap;
- distance-based unknown scores (k-NN distance to training embeddings) often work much better on good embeddings than on raw pixels, but they still need held-out unknown categories to evaluate.

Representation learning changes *how measurements are obtained*. It does not change minimum-risk decisions, honest validation or the separation of known-class accuracy from rejection and shift, which is the framework of this track [1], [2].

What to submit

A finished recognition project should include, at minimum:

1. **Task definition:** observation, label space, loss, reject policy.
2. **Data card:** provenance, licensing, sampling unit, class counts, known limitations (here: 13 writers, no writer IDs) [24].
3. **Split description:** what each split was used for, the unit of splitting, seeds.
4. **Baselines and candidates:** all models tried, not just the winner, with development scores and their variability.
5. **Selection evidence:** nested estimate of the chosen procedure and a statistical comparison with the runner-up.
6. **Error analysis:** confusion structure, examples, hypotheses, learning curve.
7. **Final test result:** counts, interval, per-class results, scope statement.

8. **Stress results:** rejection (coverage and risk), held-out unknown categories, shift or perturbation tests, calibration if probabilities are used.
9. **Reproducibility:** environment and library versions, the full pipeline object, class ordering, runtime and memory if they matter.

Model cards formalize the deployment-facing part of this list (intended use, evaluation conditions, performance across relevant groups, caveats) [23]. If the original test set informed any redesign, the revised system needs a new protected test.

Modern view

The capstone workflow is old in its principles and still evolving in practice. The following reviews and studies are good entry points.

Raschka, "Model evaluation, model selection, and algorithm selection in machine learning" (2018) [9]. A long tutorial review that organizes evaluation into three distinct goals: estimating generalization performance, selecting hyperparameters and models, and comparing algorithms. It covers holdout, bootstrap and cross-validation, nested cross-validation and statistical tests (McNemar, 5×2cv and related tests), with practical recommendations by dataset size. It is the closest single reference to steps 2 and 6 of this chapter.

Kapoor and Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science" (*Patterns*, 2023) [5]. Collects published leakage failures across many scientific fields and proposes a taxonomy of leakage types together with "model info sheets" that ask authors to justify the absence of each type. Its main takeaway is that leakage is common, often invisible in the reported numbers, and best prevented by protocol rather than detected afterwards.

Geng, Huang and Chen, "Recent advances in open set recognition: a survey" (TPAMI, 2021) [14]. Distinguishes known known, known unknown, unknown known and unknown unknown classes and organizes open-set methods into discriminative (traditional ML and deep network based) and generative (instance and non-instance generation) families. It also relates open-set recognition to zero-shot, one-shot and classification-with-reject settings and reviews the datasets and evaluation criteria in use.

Yang, Zhou, Li and Liu, "Generalized out-of-distribution detection: a survey" (IJCV, 2024) [15]. Unifies anomaly detection, novelty detection, open-set recognition, out-of-distribution detection and outlier detection in one framework, distinguishing them by whether shift is semantic or covariate and by what the in-distribution task is. This is the vocabulary to use when deciding which of the "four tasks" table rows your project belongs to.

Vaze, Han, Vedaldi and Zisserman, "Open-set recognition: a good closed-set classifier is all you need?" (ICLR, 2022) [16]. Shows a strong correlation between closed-set accuracy and open-set performance and that a well-trained baseline with a maximum-logit score is competitive with specialized methods; it also proposes the "Semantic Shift Benchmark" to separate semantic novelty from low-level shift.

Koh et al., "WILDS" (ICML, 2021) [21] and Recht et al. (ICML, 2019) [20]. Two complementary

views of evaluation under shift: WILDS curates real deployment shifts across domains, while Recht et al. show that even a careful re-collection of a test set "from the same distribution" can lower accuracy. Together they argue for reporting results per condition and treating a single i.i.d. test score as an upper bound.

Guo et al., "On calibration of modern neural networks" (ICML, 2017) [22] and **Geifman and El-Yaniv (NeurIPS, 2017) [11]** are the starting points for confidence quality and selective prediction in deep models.

State of the art (2023–2026). In practice, recognition projects increasingly begin from frozen or lightly fine-tuned foundation features (for example DINOv2 [25]) with a linear or k-NN head, and spend the saved effort on data quality, evaluation design and documentation. Out-of-distribution evaluation has moved toward unified benchmarks and the semantic-versus-covariate distinction of [15]. Datasheets [24] and model cards [23] provide ready-made templates for documenting data and models alongside these results.

Open problems.

- **Unknown unknowns.** Any held-out set of "unknown" categories is a sample; there is no general way to certify rejection of all novel inputs.
- **Evaluation under shift.** Collecting labeled data for every deployment condition is expensive, and synthetic perturbations do not reliably predict real shifts.
- **Pretraining overlap.** With web-scale pretraining, it is increasingly hard to guarantee that test images were never seen, which complicates the "locked test" principle.
- **Small-sample comparisons.** Many applied projects have hundreds, not millions, of labeled examples, where the variance of every estimate in this chapter is large and statistical comparison is essential.

Key takeaways

- Write the protocol first: observation, sampling unit, label space, loss, reject policy and the role of each split. Most evaluation failures are protocol failures.
- Lock the test set before any modeling, keep every fitted transformation inside a pipeline, and treat features and preprocessing as hyperparameters.
- Compare all candidates on identical folds; baselines tell you what "good" means, and on these digits a 3-NN baseline is hard to beat.
- Estimate the tuned procedure with nested cross-validation, and use a corrected resampled test before claiming one model is better.
- Do error analysis on out-of-fold development predictions; use learning curves to decide between more data and a richer model.
- Open the test set once and report counts, an interval and the scope (here: new images from the same 13 writers).
- Closed-set accuracy, selective risk, unknown-class rejection and robustness to shift are different properties: a 99%-accurate digit classifier still accepted 35% of unseen digits and lost almost half its accuracy under a one-cell shift.

Exercises

1. **Distance-based rejection.** In the open-set experiment, replace maximum class probability with the distance from a test image to its nearest training image in the train-fitted PCA space. Choose the threshold with validation known digits only, then report known coverage and unknown acceptance on the test sets. Does it beat confidence thresholding?

   Hint

   Fit `StandardScaler` and `PCA` on `X[train]` only, then use `NearestNeighbors(n_neighbors=1)` on the transformed training set. The score is the distance (larger means "more unknown"); set the threshold at the 95th percentile of validation distances so about 95% of known validation digits are accepted. Report both numbers; do not change the threshold after seeing the test results.

2. **Risk–coverage without cheating.** Draw the risk–coverage curve of the 10-class tuned SVM using its decision values as confidence, first with out-of-fold development predictions, then (for reporting only) on the test set. Explain how selective accuracy can improve while useful coverage collapses, and what happens at zero coverage.

   Hint

   Sort examples by decreasing confidence; at each cut, coverage is the fraction kept and risk is the error rate among them. At very low coverage only a few examples remain, so risk is noisy, and at zero coverage it is undefined. Choose any operating point on the development curve, then read off its test value.

3. **Fix the shift weakness.** Augment each *training fold* with copies of its images shifted by one cell left, right, up and down. Re-run the nested comparison of step 6 and the stress test of step 9. Which accuracies change, and why must augmentation happen inside the folds?

   Hint

   Write a small transformer or a custom loop that augments only the training part of each split; augmenting before splitting puts shifted copies of validation images into training, which is leakage. Expect the shifted-test accuracy to rise sharply and clean accuracy to stay similar or change slightly.

4. **Writer-disjoint evaluation by design.** The UCI dataset's training portion was written by 30 people and its test portion by 13 different people [3]. Design (on paper) an experiment that measures generalization to new writers. Which numbers in this chapter would you expect to change, and which conclusions would you no longer be able to draw from the bundled data alone?

   Hint

   Train and select on one writer group and test on the other. Expect lower accuracy for k-NN in particular, since its advantage relies on near-duplicates from the same writers. Without writer IDs inside a portion, you still cannot do grouped cross-validation within development, so validation remains writer-mixed.

5. **Is it significant?** Using the 10 fold scores from step 5, apply the corrected resampled t-test to logistic regression versus random forest, and to LDA versus logistic regression. Then compute the naive paired t-test without the $n_{\text{te}}/n_{\text{tr}}$ term. In which case would the two tests lead to different conclusions?

Hint

With 5-fold CV, $n_{\text{te}}/n_{\text{tr}} \approx 0.25$, which is much larger than $1/k = 0.1$, so the corrected variance is about 3.5 times the naive one and the t statistic shrinks by a factor of about $\sqrt{3.5} \approx 1.9$. Small but consistent differences are where the conclusions diverge.

6. **Frozen features versus fine-tuning.** Design an experiment comparing a linear probe on frozen pretrained features with full fine-tuning for a real image dataset of your choice. List every decision that could leak test information.

Hint

Include checkpoint and layer choice, image resizing and normalization, augmentation, learning-rate and epoch selection (early stopping), threshold and calibration fitting, prompt design if a vision–language model is used, and possible overlap between pretraining data and your test images.

References

1. S. Theodoridis and K. Koutroumbas, *Pattern Recognition*, 4th ed., Academic Press (Elsevier), 2009; Ch. 1 “Introduction”, Ch. 5–7 on feature selection and generation, Ch. 10 “Supervised Learning: The Epilogue”. doi
2. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006; Sec. 1.5 (decision theory and the reject option), Sec. 1.3 (model selection), Ch. 14 (combining models). official page and PDF
3. E. Alpaydin and C. Kaynak, “Optical Recognition of Handwritten Digits,” UCI Machine Learning Repository, 1998. doi
4. F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. JMLR; dataset loader: load_digits
5. S. Kapoor and A. Narayanan, “Leakage and the reproducibility crisis in machine-learning-based science,” *Patterns*, vol. 4, no. 9, 2023. doi
6. S. Varma and R. Simon, “Bias in error estimation when using cross-validation for model selection,” *BMC Bioinformatics*, vol. 7, 91, 2006. doi
7. G. C. Cawley and N. L. C. Talbot, “On over-fitting in model selection and subsequent selection bias in performance evaluation,” *Journal of Machine Learning Research*, vol. 11, pp. 2079–2107, 2010. JMLR
8. C. Nadeau and Y. Bengio, “Inference for the generalization error,” *Machine Learning*, vol. 52, pp. 239–281, 2003. doi
9. S. Raschka, “Model evaluation, model selection, and algorithm selection in machine learning,” arXiv:1811.12808, 2018. arXiv

10. Y. LeCun, L. Bottou, Y. Bengio and P. Haffner, “Gradient-based learning applied to document recognition,” *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. doi
11. Y. Geifman and R. El-Yaniv, “Selective classification for deep neural networks,” in *Advances in Neural Information Processing Systems 30* (*NIPS*), 2017. proceedings
12. D. Hendrycks and K. Gimpel, “A baseline for detecting misclassified and out-of-distribution examples in neural networks,” in *Proc. ICLR*, 2017. arXiv
13. W. J. Scheirer, A. de Rezende Rocha, A. Sapkota and T. E. Boult, “Toward open set recognition,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 7, pp. 1757–1772, 2013. doi
14. C. Geng, S.-J. Huang and S. Chen, “Recent advances in open set recognition: A survey,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, no. 10, 2021. doi
15. J. Yang, K. Zhou, Y. Li and Z. Liu, “Generalized out-of-distribution detection: A survey,” *International Journal of Computer Vision*, 2024. doi
16. S. Vaze, K. Han, A. Vedaldi and A. Zisserman, “Open-set recognition: A good closed-set classifier is all you need?,” in *Proc. ICLR*, 2022. arXiv
17. B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola and R. C. Williamson, “Estimating the support of a high-dimensional distribution,” *Neural Computation*, vol. 13, no. 7, pp. 1443–1471, 2001. doi
18. F. T. Liu, K. M. Ting and Z.-H. Zhou, “Isolation forest,” in *Proc. IEEE International Conference on Data Mining* (*ICDM*), 2008. doi
19. J. G. Moreno-Torres, T. Raeder, R. Alaiz-Rodríguez, N. V. Chawla and F. Herrera, “A unifying view on dataset shift in classification,” *Pattern Recognition*, vol. 45, no. 1, pp. 521–530, 2012. doi
20. B. Recht, R. Roelofs, L. Schmidt and V. Shankar, “Do ImageNet classifiers generalize to ImageNet?,” in *Proc. ICML*, PMLR 97, pp. 5389–5400, 2019. PMLR
21. P. W. Koh, S. Sagawa, H. Marklund et al., “WILDS: A benchmark of in-the-wild distribution shifts,” in *Proc. ICML*, PMLR 139, pp. 5637–5664, 2021. PMLR
22. C. Guo, G. Pleiss, Y. Sun and K. Q. Weinberger, “On calibration of modern neural networks,” in *Proc. ICML*, 2017. arXiv
23. M. Mitchell, S. Wu, A. Zaldivar, P. Barnes, L. Vasserman, B. Hutchinson, E. Spitzer, I. D. Raji and T. Gebru, “Model cards for model reporting,” in *Proc. Conference on Fairness, Accountability, and Transparency* (*FAT**), 2019. doi
24. T. Gebru, J. Morgenstern, B. Vecchione, J. W. Vaughan, H. Wallach, H. Daumé III and K. Crawford, “Datasheets for datasets,” *Communications of the ACM*, vol. 64, no. 12, 2021. doi
25. M. Oquab, T. Darcet, T. Moutakanni et al., “DINOv2: Learning robust visual features without supervision,” arXiv:2304.07193, 2023. arXiv

Part V

Machine Learning

ML 01 • Introduction to Machine Learning

What it means for a computer to learn from data: the learning paradigms, linear and logistic regression, cost functions, gradient descent, overfitting and evaluation, built from scratch in NumPy.

Online edition: https://blog.shyandram.dev/en/learn/ml/ml-01-introduction/

What you'll learn

- What "learning from data" means, and how it differs from writing rules by hand, using small images of handwritten digits as the running example.
- The main learning paradigms: supervised (regression and classification), unsupervised (clustering, dimensionality reduction, anomaly detection), self-supervised, and reinforcement learning, including how reinforcement learning from human feedback (RLHF) tunes chat models.
- The vocabulary of a learning problem: model $f_{w,b}(x)$, parameters, features, targets, predictions, training/validation/test sets and cost functions.
- Linear regression with the mean squared error cost, and gradient descent to minimize it: the update rule, the learning rate, convergence, and batch versus mini-batch versus stochastic updates.
- Why feature scaling matters, how logistic regression turns a line into a classifier with the sigmoid and cross-entropy loss, and how to detect and fix overfitting with validation data and L2 regularization.
- How to report results honestly with accuracy, precision, recall and MSE.

The big picture

Most programs are recipes: a person thinks hard about a problem and writes down every step. Machine learning flips this. We write a *flexible* program with many adjustable numbers, show it many examples of the job done correctly, and let an algorithm adjust the numbers until the program does the job well. The examples replace the recipe.

> **Explain like I'm five**
>
> Think about how you learned to tell cats from dogs. Nobody gave you a rulebook ("pointy ears plus whiskers minus barking equals cat"). Grown-ups pointed and said "cat", "dog", "cat" many times, and you made mistakes and got corrected until you just *knew*. Machine learning is the same: we show the computer lots of examples with the right answer, it guesses, we tell it how wrong it was, and it nudges its guesses a tiny bit each time until it is rarely wrong.

Why this matters: almost every modern computer-vision system, from phone face unlock to medical image analysis, is a learned model, not a hand-written program. The ideas in this tutorial (a model with parameters, a cost that measures error, gradient descent that lowers the cost, and separate data to check the result) are the same ideas that train today's largest neural networks. Only the size of the model changes. Once these few ideas are clear, the rest of the series (ML 02 and the deep-learning tutorials) is mostly about *which* model and *which* cost.

About this tutorial

This tutorial is an expanded, formal version of the earlier notes Introduction to Machine Learning [1], which followed Andrew Ng's Machine Learning Specialization [4] and the *Deep Learning* book [3].

What is machine learning?

Plain version. A machine-learning system improves at a task by looking at data, instead of having every decision written by a programmer.

Learning instead of programming

The idea is old. In 1959, Arthur Samuel described a program that learned to play checkers better than he did, by playing many games and adjusting how it scored board positions [2]. His point was that a computer which learns from experience removes much of the detailed, hand-written programming the task would otherwise need. The famous one-line definition, machine learning as the "field of study that gives computers the ability to learn without being explicitly programmed", is a later paraphrase of this idea, usually attributed to Samuel, rather than a sentence from his paper.

A more operational definition, common in textbooks, has three parts: a **task** (for example, recognizing digits), a **performance measure** (the fraction recognized correctly), and **experience** (a collection of labeled digit images). A program *learns* if its performance on the task improves with experience [3], [7].

An image example: rules versus learning

Our running example is the scikit-learn digits dataset [5], [6]: 1,797 grayscale images of handwritten digits, each only 8×8 pixels with values from 0 to 16. As in Images as Arrays, each image is just a small array of numbers, and flattening it gives a vector of 64 numbers.

Suppose we only need to tell a **0** from a **1**. A programmer might write a rule: "a 1 is thin, a 0 is wide, so count how many columns contain ink; five columns or fewer means 1." This rule is easy to read, and it is right for most upright 1s. But people write 1s with slants, serifs and bases, and a slanted 1 covers as many columns as a 0 (Figure 1). The rule gets about 79% of the cases right. Fixing it means more rules ("unless it slants..."), and each new rule creates new exceptions.

The machine-learning approach writes *no* digit-specific rule. It picks a flexible family of functions (here, logistic regression, which we build later), and lets an algorithm choose the 65 numbers inside it from examples. On held-out images, it separates 0s from 1s perfectly, and exactly the same code handles all ten digits with no new rules:

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

digits = load_digits()                        # 1,797 images, 8x8 pixels, values 0..16
X, y = digits.data, digits.target             # X: (1797, 64) flattened pixels; y: 0..9

# Hand-written rule for "0 versus 1": a 1 is thin, a 0 is wide.
keep = (y == 0) | (y == 1)
imgs, labels = digits.images[keep], y[keep]
width = (imgs.max(axis=1) > 4).sum(axis=1)   # how many columns contain ink
rule = np.where(width <= 5, 1, 0)
print("hand rule, 0 vs 1:", round((rule == labels).mean(), 3))           # 0.789

# Learned rule for the same task, judged on images it never saw.
X01 = imgs.reshape(len(imgs), -1)
Xa, Xb, ya, yb = train_test_split(X01, labels, test_size=0.3, random_state=0)
print("learned, 0 vs 1:", LogisticRegression(max_iter=1000).fit(Xa, ya).score(Xb, yb))   #
 ↪  1.0

# The same recipe scales to all ten digits with no new rules.
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.25, random_state=0)
print("learned, 10 digits:",
      round(LogisticRegression(max_iter=5000).fit(X_tr, y_tr).score(X_te, y_te), 3))  # 0.953
```

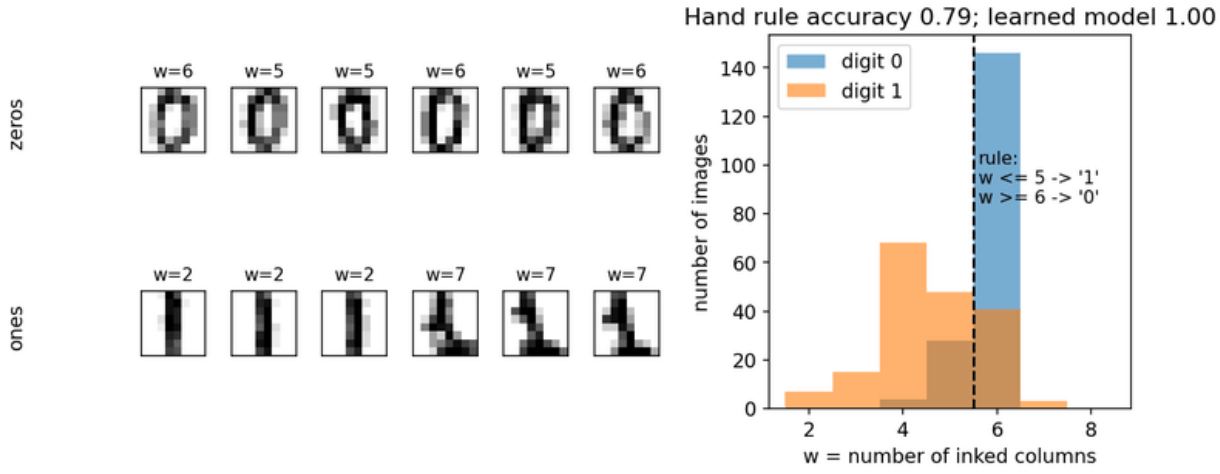


Figure 1 — Why hand-written rules are brittle. Left: zeros and ones from the digits dataset; slanted ones are as wide as zeros. Right: the 'count inked columns' feature overlaps heavily between the two classes, so any threshold on it makes many mistakes. A learned model uses all 64 pixels at once.

This small experiment contains the whole argument for machine learning. When the rule is hard to put into words but examples are cheap, learn the rule from examples. It also shows the price: the learned model is a list of 65 numbers that is much harder to read than "count the columns."

The learning paradigms

Plain version. Learning problems differ mainly in *what feedback the learner gets*: the right answer for every example, no answers at all, answers it can manufacture from the data itself, or only an occasional score for how well it did.

Paradigm	What the data looks like	What is learned	Typical example
Supervised	pairs (x, y): input and the "right answer"	a function from x to y	digit image → digit label
Unsupervised	inputs x only	structure: groups, directions, outliers	grouping similar digits
Self-supervised	inputs x only, but part of x is hidden and used as the target	a general-purpose representation	predicting a masked word or image patch
Reinforcement	an agent's actions and the rewards they earn	a policy: which action to take in each situation	game playing, tuning chat models

My original notes listed the first, second and fourth; self-supervised learning has since become the main way large models are pre-trained, so it gets its own row [9], [10].

Supervised learning

In supervised learning, the learner is given the right answers. The data is a set of examples $\{(x^{(i)}, y^{(i)})\}_{i=1}^{m}$, where $x^{(i)}$ is an input, $y^{(i)}$ is the desired output (the **label** or **target**), and m is the number of examples. The goal is a function f such that $f(x)$ is close to y, *also for new inputs not in the data*. That last part, called **generalization**, is the real goal; memorizing the examples is easy.

Unsupervised learning

In unsupervised learning there are no answers, only inputs $\{x^{(i)}\}$. The goal is to find something interesting in the data: groups of similar examples, a compact description, or examples that do not fit. There is no single "correct" output, which makes these methods useful for exploration and harder to evaluate.

Self-supervised learning

Self-supervised learning uses unlabeled data but turns it into a supervised problem by hiding part of each example and asking the model to predict the hidden part. In text, hide a word and predict it from the rest of the sentence. In images, hide most of the patches and reconstruct them: masked

autoencoders hide 75% of an image's patches and train a network to fill them in [12]. LeCun and Misra describe self-supervised learning as a way for models to acquire broad background knowledge from raw observation, because the labels come for free [9].

The point is not the fill-in-the-blank task itself; it is that a model which can fill in blanks well must have learned a lot about the structure of the data. The learned internal **representation** can then be adapted to many other tasks with few labels. This is the recipe behind **foundation models**: models trained on broad data at scale (usually self-supervised) and then adapted to a wide range of downstream tasks [10]. GPT-3, trained simply to predict the next word on a large text corpus, was able to perform many language tasks from a few examples given in the prompt, without further training [11].

Reinforcement learning

In reinforcement learning (RL), an **agent** interacts with an **environment**. At each step it observes a **state**, chooses an **action**, and receives a **reward** (a number) and a new state. Nobody tells it the right action; it must discover which actions lead to high total reward over time, often when the reward arrives long after the action that earned it. What is learned is a **policy**, a rule for choosing actions in each state [8]. Samuel's checkers player is an early example of this setting [2].

Today, RL is best known for tuning large language models. In **reinforcement learning from human feedback** (RLHF), as described by Ouyang et al. [13], a pre-trained language model is first fine-tuned on demonstrations written by people (supervised learning). People then rank several model answers to the same prompt, and a separate **reward model** is trained to predict those rankings. Finally, the language model is optimized with RL to produce answers that the reward model scores highly. The authors report that labelers preferred the outputs of their 1.3-billion-parameter tuned model over those of the 175-billion-parameter GPT-3 [13]. More recently, DeepSeek-R1 used large-scale RL with automatically checkable rewards (such as whether a math answer is correct) to improve step-by-step reasoning [14].

Notice that RLHF chains three paradigms: self-supervised pre-training, supervised fine-tuning, and reinforcement learning. Real systems mix paradigms freely; the table above is a map, not a set of walls.

Supervised learning: regression and classification

Plain version. Supervised learning comes in two main kinds. **Regression** predicts a number; **classification** predicts a category.

Regression

In regression the target y is a real number, so there are infinitely many possible outputs. Examples: predicting a stock price, a student's final grade from earlier test scores, or a person's age from a

face image. The last one shows that the input can be an image while the output is still a single number.

Classification

In classification the target is one of a small, fixed set of categories (classes). Examples: spam versus not spam (two classes, **binary classification**), the shape category of a face, or a tumor type such as benign, malignant type 1 or malignant type 2 (**multi-class classification**). Classes are names, not quantities: "class 2" is not twice "class 1". A classifier usually outputs a probability for each class and then picks the most likely one.

```
from sklearn.datasets import make_regression, make_classification
from sklearn.linear_model import LinearRegression, LogisticRegression

# Regression: the target is a real number.
X_r, y_r = make_regression(n_samples=100, n_features=1, noise=10.0, random_state=0)
reg = LinearRegression().fit(X_r, y_r)
print(reg.predict(X_r[:3]).round(1), y_r[:3].round(1))   # numbers vs numbers

# Classification: the target is one of a few labels.
X_c, y_c = make_classification(n_samples=100, n_features=2, n_redundant=0,
                               n_clusters_per_class=1, random_state=0)
clf = LogisticRegression().fit(X_c, y_c)
print(clf.predict(X_c[:5]), y_c[:5])                     # labels vs labels
print(clf.predict_proba(X_c[:2]).round(3))               # class probabilities
```

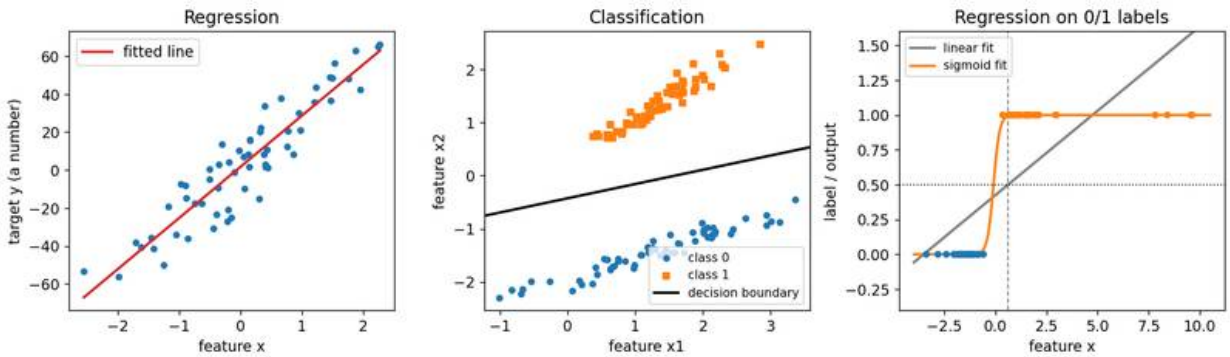


Figure 2 — Left: regression fits a curve through numeric targets. Middle: classification separates labeled points with a decision boundary. Right: fitting a straight regression line to 0/1 labels (gray) gives outputs outside [0, 1], and far-away points drag its 0.5 crossing; a sigmoid (logistic regression, orange) is the right tool.

Why not use regression for classification?

It is tempting to code the classes as 0 and 1, fit a regression line, and call anything above 0.5 a 1. This can work on easy data, but it usually performs badly, for two reasons visible in Figure 2 (right). First, the line outputs values like -0.4 or 1.7, which are not probabilities. Second, the squared error punishes a point that is *correctly* classified but far from the boundary, so a few extreme but easy examples pull the line and move its 0.5 crossing to the wrong place. Logistic regression (later in this tutorial) fixes both problems.

Unsupervised learning

Plain version. Without any answers, we can still ask: which examples belong together, how can each example be described with fewer numbers, and which examples are strange?

Clustering

Clustering groups similar examples. The classic algorithm is **k-means** [15]. Choose the number of groups K. Start with K center points, then repeat two steps until nothing changes: assign each example to its nearest center, and move each center to the mean of the examples assigned to it. Each step can only lower the total squared distance from examples to their centers,

$$\sum_{i=1}^{m} \|x^{(i)} - \mu_{c^{(i)}}\|^2,$$

where μ_k is the center of cluster k and $c^{(i)}$ is the cluster that example i is assigned to. So the algorithm always stops, but possibly at a poor local solution, which is why it is run from several random starts. Jain's review of fifty years of clustering research [15] stresses that there is no best clustering algorithm in general: the "right" groups depend on what similarity means for the data.

A common confusion: **k-nearest neighbors** (KNN) is *not* a clustering method. KNN classifies a new example by a vote among the k most similar *labeled* training examples, so it is a supervised method [5]. The "k" in k-means counts clusters; the "k" in KNN counts neighbors.

Dimensionality reduction

Dimensionality reduction describes each example with fewer numbers while keeping as much useful information as possible. **Principal component analysis** (PCA) finds the directions along which the data varies most and projects the data onto the top few of them; the new coordinates are uncorrelated and successively capture as much variance as possible [16]. Two PCA coordinates can turn 64-pixel digit images into points on a plane that we can look at (Figure 3, middle).

Anomaly detection

Anomaly detection finds examples that do not fit the pattern of the rest: a fraudulent transaction, a defective part, a corrupted image. Chandola, Banerjee and Kumar's survey [17] organizes the many techniques by the assumption each one makes about what "normal" looks like. A simple version uses PCA: fit a few components on the data, rebuild each example from them, and flag those with a large reconstruction error, because the components only describe what is common.

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.cluster import KMeans
from sklearn.decomposition import PCA

X, y = load_digits(return_X_y=True)            # y is used only to check, never to fit

# Clustering: ten groups, found without looking at y.
km = KMeans(n_clusters=10, n_init=10, random_state=0).fit(X)
purity = sum(np.bincount(y[km.labels_ == k]).max() for k in range(10)) / len(y)
print("cluster purity:", round(purity, 3))     # 0.792: most clusters are one digit

# Dimensionality reduction: 64 numbers per image -> 2 numbers.
Z = PCA(n_components=2).fit_transform(X)       # shape (1797, 2)

# Anomaly detection: images that a 20-component PCA rebuilds badly are unusual.
pca20 = PCA(n_components=20).fit(X)
err = ((X - pca20.inverse_transform(pca20.transform(X))) ** 2).sum(axis=1)
print("most unusual images:", np.argsort(err)[-5:])
```

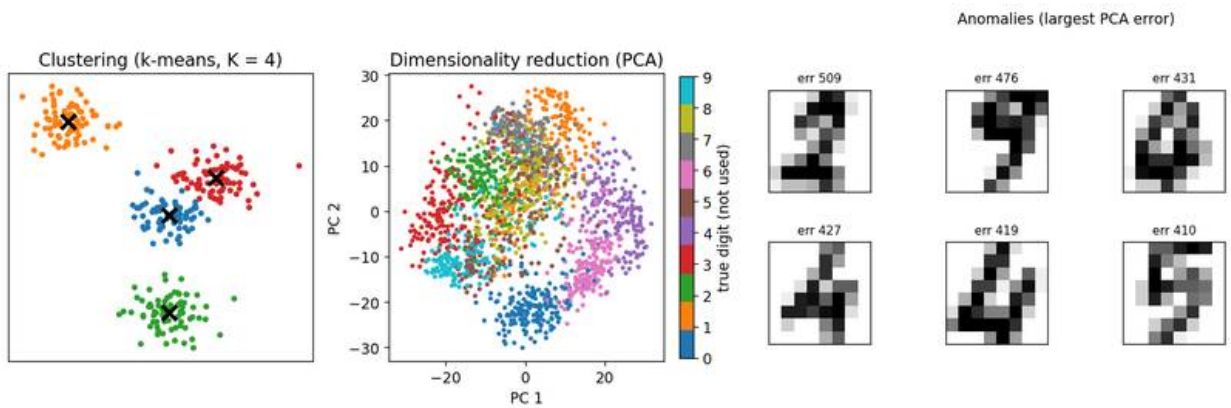


Figure 3 — Unsupervised learning. Left: k-means groups points around K centers. Middle: PCA compresses each 64-pixel digit to two numbers (colors show the true digits, which PCA never saw). Right: the digits with the largest PCA reconstruction error are the oddest-looking ones.

The purity check in the snippet uses the labels only *after* clustering, to see whether the discovered groups match the digits. About 79% of the images fall in a cluster dominated by their own digit, even though k-means never saw a label.

The vocabulary of a learning problem

Plain version. Before any math, we need names for the parts: what goes in, what comes out, what the right answer is, what the model is, and how we measure its mistakes.

Inputs, outputs and targets

- **Features** (inputs), x: the numbers that describe one example. For a digit image, the 64 pixel values; for a house, its size and number of rooms. With n features we write $x = (x_1, \dots, x_n)$, and x_j is the j-th feature.
- **Target**, y: the correct output, also called the label or **ground truth**. Only supervised learning has targets.
- **Prediction**, $\hat{y}$ ("y-hat"): what the model outputs for an input. The hat means "estimated".
- **Training example** i: the pair $(x^{(i)}, y^{(i)})$. The superscript in parentheses is an index, not a power. $x_j^{(i)}$ is feature j of example i.

The model and its parameters

The **model** is the function that maps features to a prediction. We write it $f_{w,b}$ to show that it depends on adjustable numbers called **parameters**, here w (the **weights**) and b (the **bias**):

$$\hat{y} = f_{w,b}(x).$$

Choosing the *form* of f (a line, a curve, a neural network) is a design decision. Choosing the *values* of w and b is what learning does. Numbers we set by hand, such as the learning rate or polynomial degree, are called **hyperparameters** to distinguish them from the learned parameters.

Cost function

A **loss** $L(\hat{y}, y)$ measures how wrong one prediction is. The **cost function** $J(w, b)$ averages the loss over the training set, so it is a single number that says how wrong the whole model is for a given choice of parameters. The model is f; the cost is J. Keeping these two apart avoids a lot of confusion: $f_{w,b}(x)$ takes an input and returns a prediction, while $J(w, b)$ takes parameters and returns an error.

The **objective** of training is then precise: find the parameters that make J as small as possible, so that $\hat{y}^{(i)}$ is close to $y^{(i)}$ for all training examples.

Training, validation and test sets

The data is split into three parts with different jobs:

- the **training set** is used to fit the parameters;

- the **validation set** is used during development to compare models and choose hyperparameters;
- the **test set** is used once, at the end, to estimate how the final model will do on new data.

The test set must not influence any decision, or the estimate becomes optimistic. We come back to *why* this split matters in the section on overfitting.

Linear regression

Plain version. Fit a straight line through the data. The line has two knobs: its slope and its height. Learning means turning the knobs until the line passes as close as possible to the points.

The model

With one feature, the linear regression model is

$$f_{w,b}(x) = wx + b,$$

where x is the feature, w is the slope (how much the prediction changes when x grows by 1) and b is the intercept (the prediction when $x = 0$). Changing w tilts the line; changing b moves it up and down (Figure 4, left).

With n features, the same model is a weighted sum,

$$f_{\mathbf{w},b}(\mathbf{x}) = \mathbf{w} \cdot \mathbf{x} + b = \sum_{j=1}^{n} w_j x_j + b,$$

where $\mathbf{w} = (w_1, \dots, w_n)$ holds one weight per feature. A common trick appends a constant feature $x_0 = 1$, so the bias becomes just another weight w_0 and the model shortens to $\mathbf{w} \cdot \mathbf{x}$. Either way, "linear" means linear in the *parameters*; we will later feed it squared or cubed features and it remains a linear regression.

The cost function: mean squared error

For each example, the **error** (residual) is $f_{w,b}(x^{(i)}) - y^{(i)}$. Squaring makes errors positive and punishes big ones more than small ones. Averaging over the m training examples gives the **mean squared error** (MSE) cost:

$$J(w, b) = \frac{1}{2m} \sum_{i=1}^{m} \left(f_{w,b}(x^{(i)}) - y^{(i)} \right)^2,$$

where m is the number of training examples and $(x^{(i)}, y^{(i)})$ is the i-th example. The extra factor $\frac{1}{2}$ is a convenience: it cancels the 2 that appears when we differentiate the square, and it does not change where the minimum is.

Picture the cost as a landscape over the parameters. If we fix b and vary only w, J is a parabola: lines that are too flat or too steep both have large error, and one slope in between is best (Figure 4, middle). With both w and b, J is a bowl, which we draw as a contour map of ellipses (Figure 4, right). The best line sits at the bottom of the bowl.

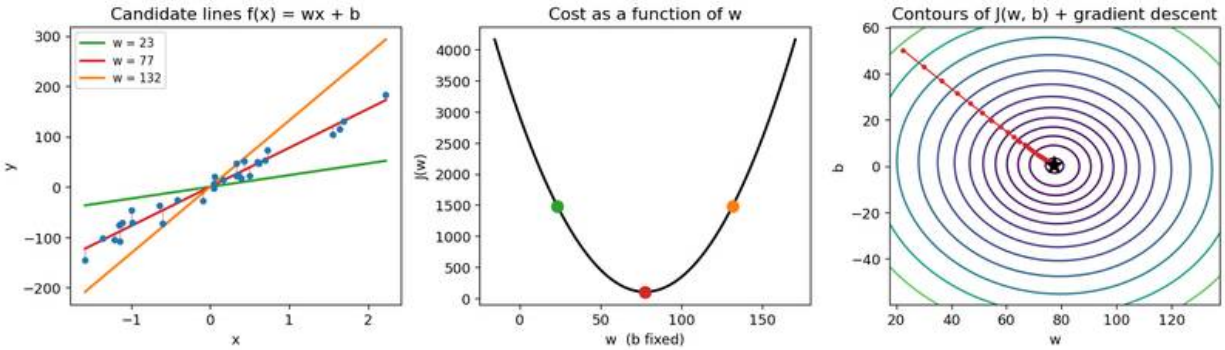


Figure 4 — From lines to costs. Left: three candidate lines; vertical segments are the residuals of the best one. Middle: with b fixed, $J(w)$ is a parabola, and each line is one point on it. Right: $J(w, b)$ is a bowl (contours), and gradient descent walks downhill to its bottom.

Why this bowl has one bottom

For linear regression, the MSE cost is a **convex** function of (w, b): any straight segment between two points on its surface lies on or above the surface. A convex function has no separate local minima; every minimum is the global one. This is a special, friendly case. For many other models, including the neural networks of ML 02, the cost landscape has many valleys, and an optimizer can stop in one that is not the lowest. The original notes make exactly this point: in general, the best we can hope for is a good local minimum, close to the global one.

For linear regression we could even solve for the bottom directly with a formula (the normal equations, which `np.polyfit` uses). We use gradient descent anyway, because it is the method that still works when no formula exists.

Gradient descent

Plain version. Stand somewhere on the cost landscape in fog. Feel which way the ground slopes, take a small step downhill, and repeat. When the ground is flat, you are at the bottom.

The update rule

The **derivative** $\partial J/\partial w$ tells us how J changes when w increases a little: positive means "uphill to the right", negative means "uphill to the left". Its sign gives the direction, and its size says how steep the slope is. Gradient descent moves each parameter a small step *against* its derivative:

$$w \leftarrow w - \alpha \frac{\partial J(w,b)}{\partial w},$$
$$b \leftarrow b - \alpha \frac{\partial J(w,b)}{\partial b},$$

where $\alpha > 0$ is the **learning rate** (step size) and $\leftarrow$ means "replace with". Both derivatives are computed at the *current* (w, b) before either is changed; this is called a **simultaneous update**. The vector of all partial derivatives is the **gradient** ∇J, which points in the direction of steepest increase, so the update moves in the direction of steepest decrease [3].

For the MSE cost of linear regression, the chain rule gives

$$\frac{\partial J}{\partial w} = \frac{1}{m}\sum_{i=1}^{m}\left(f_{w,b}(x^{(i)}) - y^{(i)}\right)x^{(i)}, \qquad \frac{\partial J}{\partial b} = \frac{1}{m}\sum_{i=1}^{m}\left(f_{w,b}(x^{(i)}) - y^{(i)}\right).$$

Read them in words: the gradient for w is the average error times the feature; the gradient for b is the average error. If predictions are too high on average, b goes down. If predictions are too high mainly where x is large, w goes down. With n features, $\partial J/\partial w_j$ uses $x_j^{(i)}$ in place of $x^{(i)}$.

Implementing it from scratch

```
import numpy as np
from sklearn.datasets import make_regression

X, y = make_regression(n_samples=100, n_features=1, noise=10.0, random_state=0)
x = X[:, 0]                                  # one feature, shape (m,)
m = len(x)

def cost(w, b):
    return ((w * x + b - y) ** 2).sum() / (2 * m)

w, b, alpha = 0.0, 0.0, 0.1
for step in range(200):
    err = w * x + b - y                      # f_wb(x_i) - y_i for every i
    dj_dw = (err * x).sum() / m
    dj_db = err.sum() / m
    w, b = w - alpha * dj_dw, b - alpha * dj_db   # simultaneous update
    if step % 50 == 0:
        print(f"step {step:3d}  J = {cost(w, b):8.2f}")

print("gradient descent:", round(w, 3), round(b, 3))   # 42.619 -0.814
print("closed form     :", np.polyfit(x, y, 1).round(3))  # [42.619 -0.814]
```

The cost drops from about 800 to 57 within 50 steps and then stops changing. Gradient descent and the exact formula agree to three decimals. The remaining cost is not zero, because the data contains noise that no straight line can explain.

Convergence: where the derivative is zero

"Repeat until convergence" needs a definition. At a minimum, the landscape is flat, so the **derivative is zero** there, and the update $w - \alpha \cdot 0$ leaves w unchanged. Note that it is the *derivative*, not the cost, that becomes zero: the cost at the minimum is just the smallest value available, 57 in the example above. In practice we stop when the cost decreases by less than a small tolerance between iterations, or when the gradient's size falls below a threshold, or after a fixed budget of steps.

A useful consequence: as we approach a minimum, the derivative shrinks, so even with a *fixed* α the steps $\alpha\, \partial J/\partial w$ get smaller automatically. Gradient descent naturally slows down near the bottom.

Choosing the learning rate

The learning rate is the most important hyperparameter of gradient descent (Figure 5):

- **Too small:** every step is tiny. The algorithm does go downhill, but it may need a very large number of steps.
- **About right:** the cost falls quickly and settles.
- **Too large:** a step jumps past the bottom to the other side of the valley, possibly higher than where it started. The parameters bounce back and forth, may never settle (**overshoot**), and can fly off to infinity (**divergence**).

The standard diagnostic is a **learning curve**: plot J against the iteration number. With a good α, J decreases at every step. If J ever goes *up*, α is too large (or there is a bug in the gradient). A practical recipe is to try values spaced by about a factor of 3, such as 0.001, 0.003, 0.01, 0.03, 0.1, 0.3, and pick the largest one whose curve decreases smoothly.

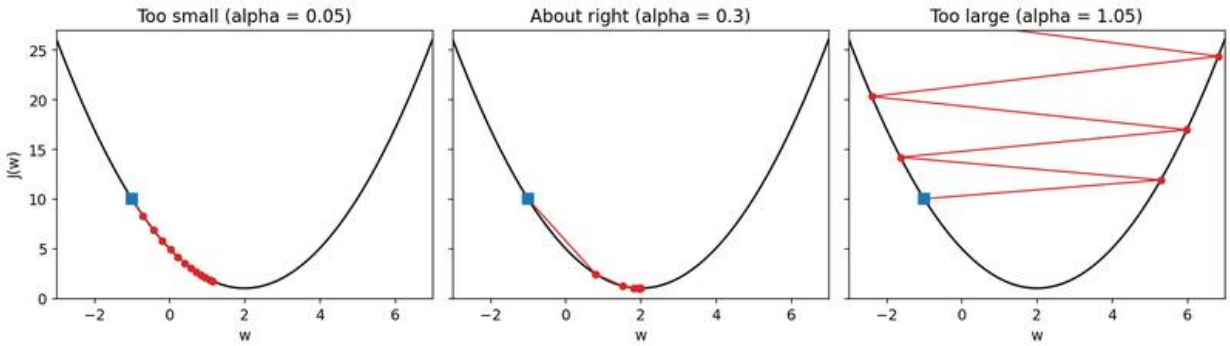


Figure 5 — The same cost $J(w)$, starting from the same point, with three learning rates. Too small: slow progress. About right: fast convergence. Too large: each step overshoots further, and the iterates diverge.

Many optimizers adapt the step size during training, either by a **schedule** that lowers α over time or by scaling each parameter's step using running statistics of its past gradients, as Adam does [19]. These make "the steps become smaller near the minimum" an explicit design choice rather than just a side effect.

Batch, mini-batch and stochastic gradient descent

The gradient formula sums over all m examples. That is fine for 100 examples and a problem for 100 million. The key insight, explained clearly in [3] and analyzed in depth by Bottou, Curtis and Nocedal [18], is that the gradient is an *average*, that is, an expectation over the data, and an average can be estimated from a sample.

- **Batch gradient descent** uses the *whole* training set for every update. Each step is exact and expensive.
- **Mini-batch gradient descent** shuffles the data and, for each update, uses a small random subset (a **mini-batch**) of B examples, typically 32 to a few hundred. Each step is a noisy but unbiased estimate of the full gradient, and costs B/m as much.
- **Stochastic gradient descent** (SGD), in the strict sense, uses one example per update ($B = 1$). In practice, "SGD" often refers to the mini-batch version too.

One full pass through the training data is an **epoch**. Beware of naming in code: the `batch_size` argument in deep-learning libraries means the *mini-batch* size, not the batch-gradient-descent "whole set".

The noise in mini-batch gradients is not only tolerable but often helpful: in the same number of epochs, mini-batch descent makes many more updates than batch descent. Bottou et al. argue that this is why stochastic methods dominate large-scale learning: they reach a useful accuracy with far less computation than exact methods [18].

```
import numpy as np
from sklearn.datasets import make_regression

X, y = make_regression(n_samples=10_000, n_features=5, noise=10.0, random_state=0)
m, n = X.shape
rng = np.random.default_rng(0)

def minibatch_gd(batch_size, alpha=0.05, epochs=5):
    w, b = np.zeros(n), 0.0
    for epoch in range(epochs):
        order = rng.permutation(m)                    # shuffle once per epoch
        for start in range(0, m, batch_size):
            idx = order[start:start + batch_size]
            err = X[idx] @ w + b - y[idx]
            w -= alpha * X[idx].T @ err / len(idx)    # gradient estimated on the batch
            b -= alpha * err.mean()
    return ((X @ w + b - y) ** 2).mean() / 2

for bs in [m, 64, 1]:                                 # batch, mini-batch, stochastic
    print(f"batch size {bs:5d}: J = {minibatch_gd(bs, alpha=0.05 if bs > 1 else 0.005):.2f}")
# batch size 10000: J = 6127.00   (only 5 updates in 5 epochs)
# batch size    64: J = 49.93     (785 updates)
# batch size     1: J = 50.64     (50,000 noisy updates)
```

With the same five passes over the data, batch descent has made only five steps and is still far from

the bottom, while mini-batch descent has essentially converged (the noise level of this data puts the best achievable J near 50). Single-example SGD gets there too, but needs a smaller learning rate because its gradients are noisier, and it cannot use vectorized arithmetic efficiently.

Feature scaling

Plain version. If one feature is measured in thousands and another in fractions, the cost landscape becomes a long, narrow valley, and gradient descent zigzags. Putting all features on similar scales turns the valley into a round bowl that is easy to descend.

Why scale matters

Suppose feature x_1 (say, house area in square feet) ranges over thousands and feature x_2 (number of bedrooms) ranges from 1 to 5. A small change in w_1 changes the predictions a lot; a similar change in w_2 barely matters. So J is extremely steep along w_1 and very flat along w_2: the contours are thin ellipses. A learning rate small enough not to diverge along the steep direction makes almost no progress along the flat one (Figure 6, left).

Standardization and min–max scaling

The most common fix is **standardization** (z-score normalization):

$$x'_j = \frac{x_j - \mu_j}{\sigma_j},$$

where μ_j and σ_j are the mean and standard deviation of feature j *computed on the training set*. Each scaled feature then has mean 0 and standard deviation 1. An alternative is **min–max scaling**, $x'_j = (x_j - \min_j)/(\max_j - \min_j)$, which maps the training range to $[0, 1]$ and is the usual choice for pixel intensities (divide by 255 for 8-bit images). Both are available in scikit-learn as `StandardScaler` and `MinMaxScaler` [5].

Two rules prevent subtle bugs. Compute μ_j, σ_j on the training set only, then apply the *same* numbers to validation, test and future data. And remember that scaling changes the meaning of the learned weights: a weight now says how much the prediction changes per standard deviation of the feature.

```
import numpy as np
from sklearn.datasets import make_regression

X, y = make_regression(n_samples=200, n_features=2, noise=5.0, random_state=1)
X = X * np.array([1000.0, 1.0])               # feature 1 now has a ~1000x wider range

def gd(X, y, alpha, steps=500):
    m, n = X.shape
    w, b = np.zeros(n), 0.0
```

```
    for _ in range(steps):
        err = X @ w + b - y
        w -= alpha * X.T @ err / m
        b -= alpha * err.mean()
    return ((X @ w + b - y) ** 2).mean() / 2

print("raw,    alpha=1e-7:", round(gd(X, y, 1e-7), 2))    # 158.01; alpha=3e-6 diverges
mu, sigma = X.mean(axis=0), X.std(axis=0)
Xs = (X - mu) / sigma                          # z-score standardization
print("scaled, alpha=0.1 :", round(gd(Xs, y, 0.1), 2))    # 13.47
```

On the raw features, any learning rate above about 2×10^{-6} diverges, and with a safe one the cost is still stuck near 158 after 500 steps (it is still near 158 after 5,000), because the second weight barely moves. After standardization, $\alpha = 0.1$ reaches the minimum (about 13, the noise level) easily.

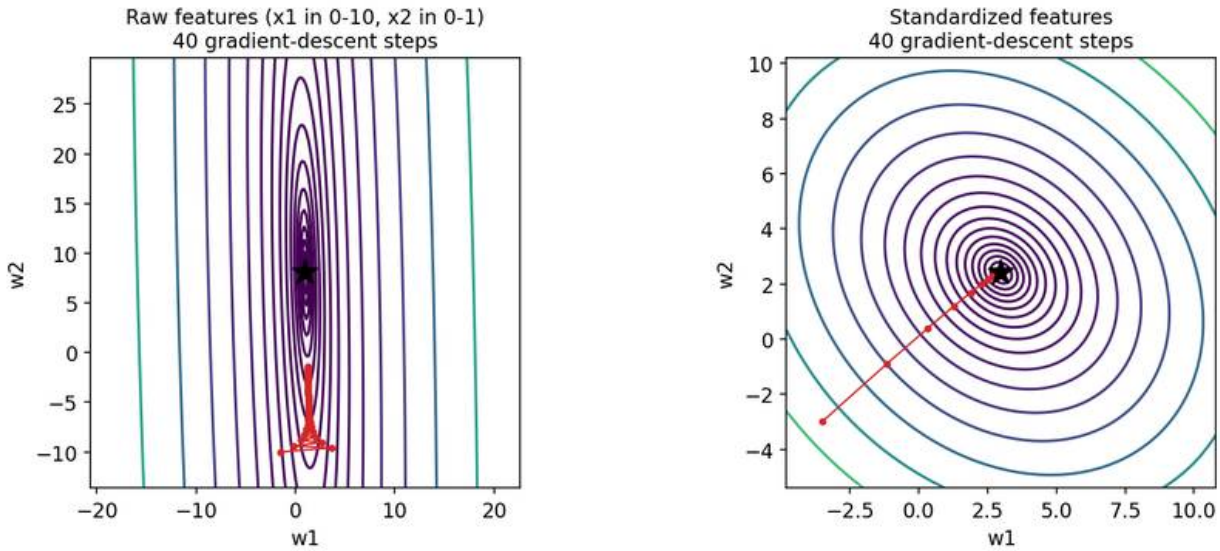


Figure 6 — Feature scaling reshapes the cost landscape. Left: features on very different scales give a narrow valley, and gradient descent zigzags. Right: after standardization, the contours are nearly circular, and the same number of steps reaches the minimum.

Logistic regression for classification

Plain version. Keep the weighted sum from linear regression, but squash its output into the range 0 to 1 so it can be read as a probability. Then choose the weights so that the probability is high for the true class.

The sigmoid function

The **sigmoid** (logistic) function is

$$g(z) = \frac{1}{1 + e^{-z}},$$

where z is any real number. It maps $-\infty$ to 0, $+\infty$ to 1 and 0 to exactly 0.5, with an S-shaped curve in between (Figure 7, left). It is also symmetric, $g(-z) = 1 - g(z)$, and has the convenient derivative $g'(z) = g(z)\,(1 - g(z))$.

The model and the decision boundary

Logistic regression feeds the linear model into the sigmoid:

$$f_{\mathbf{w},b}(\mathbf{x}) = g(\mathbf{w} \cdot \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}\cdot\mathbf{x}+b)}},$$

and interprets the output as the estimated probability that the label is 1, $f_{\mathbf{w},b}(\mathbf{x}) = P(y = 1 \mid \mathbf{x})$. Despite the name, it is a classification method. To make a decision, predict 1 when this probability is at least 0.5. Since $g(z) \geq 0.5$ exactly when $z \geq 0$, the rule is

$$\hat{y} = 1 \quad \text{if} \quad \mathbf{w} \cdot \mathbf{x} + b \geq 0, \qquad \hat{y} = 0 \text{ otherwise.}$$

The set where $\mathbf{w} \cdot \mathbf{x} + b = 0$ is the **decision boundary**. With two features it is a straight line; in general it is a flat hyperplane, with $\mathbf{w}$ perpendicular to it. Points far from the boundary get probabilities near 0 or 1; points near it get probabilities near 0.5. To obtain a curved boundary, feed the model nonlinear features such as x_1^2 or $x_1 x_2$; the model stays linear in its parameters.

The cost: cross-entropy (log loss)

Why not reuse squared error? Combined with the sigmoid, the squared-error cost is *not* convex in $(\mathbf{w}, b)$: it develops flat plateaus and multiple valleys. Instead, logistic regression uses the **log loss**, also called **binary cross-entropy**:

$$L(f, y) = -y \log f \; - \; (1 - y) \log(1 - f),$$

where $f = f_{\mathbf{w},b}(\mathbf{x})$ is the predicted probability and $y \in \{0, 1\}$ is the true label. Only one of the two terms is active for each example. If $y = 1$, the loss is $-\log f$: zero when $f = 1$, and growing without bound as $f \to 0$. If $y = 0$, the loss is $-\log(1 - f)$, the mirror image (Figure 7, middle). Confident wrong answers are punished very heavily; confident right answers cost almost nothing.

The cost is the average over the training set:

$$J(\mathbf{w}, b) = \frac{1}{m} \sum_{i=1}^{m} \Big[-\, y^{(i)} \log f_{\mathbf{w},b}(\mathbf{x}^{(i)}) - (1 - y^{(i)}) \log\left(1 - f_{\mathbf{w},b}(\mathbf{x}^{(i)})\right) \Big].$$

This choice is not arbitrary. If we treat each label as a coin flip with probability $f_{\mathbf{w},b}(\mathbf{x}^{(i)})$ of coming up 1, then J is exactly the negative log-likelihood of the training labels, divided by m. Minimizing

it is **maximum likelihood estimation** [3]. And with the sigmoid, this cost is convex, so gradient descent again finds the single global minimum.

Gradient descent, again

Using $g' = g(1-g)$, the derivative of the log loss simplifies remarkably:

$$\frac{\partial J}{\partial w_j} = \frac{1}{m}\sum_{i=1}^{m}\left(f_{\mathbf{w},b}(\mathbf{x}^{(i)}) - y^{(i)}\right)x_j^{(i)}, \qquad \frac{\partial J}{\partial b} = \frac{1}{m}\sum_{i=1}^{m}\left(f_{\mathbf{w},b}(\mathbf{x}^{(i)}) - y^{(i)}\right).$$

This has *exactly the same form* as the linear-regression gradient: average error times feature. The only difference is inside f. This is not a coincidence; it holds for a whole family of models that pair an output function with its matching likelihood, and it is one reason gradient descent code is so reusable.

```
import numpy as np
from sklearn.datasets import make_classification

X, y = make_classification(n_samples=200, n_features=2, n_redundant=0,
                           n_clusters_per_class=1, class_sep=1.5, random_state=3)

def sigmoid(z):
    return 1.0 / (1.0 + np.exp(-z))

def log_loss(p, y, eps=1e-12):
    p = np.clip(p, eps, 1 - eps)              # avoid log(0)
    return -(y * np.log(p) + (1 - y) * np.log(1 - p)).mean()

w, b, alpha = np.zeros(2), 0.0, 0.5
for step in range(1000):
    p = sigmoid(X @ w + b)                    # predicted P(y = 1 | x)
    w -= alpha * X.T @ (p - y) / len(y)       # same form as linear regression
    b -= alpha * (p - y).mean()

p = sigmoid(X @ w + b)
print("log loss:", round(log_loss(p, y), 3))  # 0.06
print("accuracy:", ((p >= 0.5) == y).mean())  # 0.99
print("boundary: %.2f*x1 + %.2f*x2 + %.2f = 0" % (w[0], w[1], b))
```

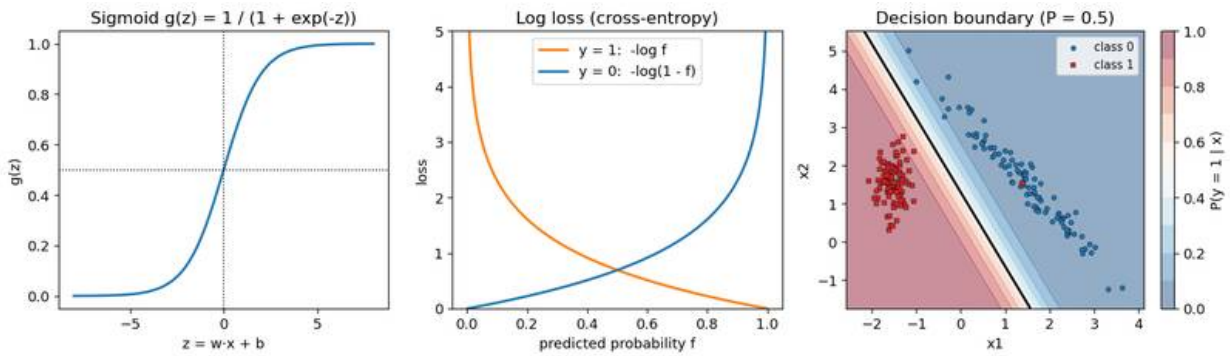


Figure 7 — Logistic regression. Left: the sigmoid squashes any score into $(0, 1)$. Middle: the log loss for $y = 1$ and $y = 0$ punishes confident mistakes heavily. Right: a model trained from scratch; shading is the predicted probability, and the black line is the decision boundary $w \cdot x + b = 0$.

More than two classes

For $K > 2$ classes, the standard generalization gives each class its own weight vector and score $z_k = \mathbf{w}_k \cdot \mathbf{x} + b_k$, and turns the scores into probabilities that sum to 1 with the **softmax** function, $P(y = k \mid \mathbf{x}) = e^{z_k} / \sum_{l=1}^{K} e^{z_l}$. The cost is again the average negative log probability of the true class (categorical cross-entropy) [3]. This is what scikit-learn's `LogisticRegression` did for the ten digits at the start of this tutorial, and it is exactly the output layer of most classification networks in ML 02.

Overfitting, underfitting and regularization

Plain version. A model that is too simple misses the pattern (**underfitting**). A model that is too flexible memorizes the training examples, noise included, and fails on new ones (**overfitting**). We detect the difference with data the model did not train on, and we tame an over-flexible model by penalizing large weights.

Seeing it happen

Take 25 noisy samples of a smooth curve and fit polynomials of different degrees, which is linear regression on the features $x, x^2, \ldots, x^d$ (Figure 8):

- **Degree 1 (underfitting, high bias).** A straight line cannot follow the curve. The error is large on the training set *and* on new data.
- **Degree 3 (a good fit).** The curve follows the trend and ignores the noise. Errors are small on both.
- **Degree 15 (overfitting, high variance).** The curve passes almost exactly through every training point, so the training error is nearly zero. Between and beyond the points it swings wildly, and the error on new data explodes.

The training error alone cannot tell the good model from the overfitted one; it even prefers the overfitted one. Only data that the model did not see can reveal the difference. This is why we keep a validation set.

Using the validation and test sets

The workflow is:

1. Fit each candidate model (each degree, each regularization strength) on the **training set**.
2. Compare the candidates on the **validation set** and keep the best one.
3. Report the chosen model's error on the **test set**, once.

Why three sets and not two? Because picking the best of many candidates on the validation set is itself a form of fitting: the winner is partly the one that got lucky on those particular examples. The untouched test set gives an unbiased final estimate. When data is scarce, **cross-validation** replaces the single validation set: split the training data into k folds, train k times, each time validating on a different fold, and average.

The pattern to look for: if training and validation errors are both high, the model underfits (use a more flexible model or better features). If training error is low but validation error is much higher, the model overfits (get more data, simplify the model, or regularize).

L2 regularization

Regularization keeps a flexible model but discourages extreme parameter values. The wild curve of the degree-15 fit needs huge, finely balanced weights. **L2 regularization** adds a penalty on the squared size of the weights to the cost:

$$J_{\text{reg}}(\mathbf{w}, b) = J(\mathbf{w}, b) + \frac{\lambda}{2m} \sum_{j=1}^{n} w_j^2,$$

where J is the original cost (MSE or log loss), $\lambda \geq 0$ is the **regularization strength** (a hyperparameter), and the sum runs over the weights but conventionally not the bias b. With $\lambda = 0$ we recover the original model; as λ grows, the weights are pulled toward zero and the fitted function becomes smoother; too large a λ underfits. For linear regression, this is **ridge regression**, introduced by Hoerl and Kennard to stabilize least-squares estimates when features are strongly correlated [20].

The gradient-descent update shows what the penalty does:

$$w_j \leftarrow w_j \left(1 - \alpha \frac{\lambda}{m}\right) - \alpha \frac{\partial J}{\partial w_j}.$$

Before each ordinary step, every weight is multiplied by a number slightly less than 1. This is why L2 regularization is also called **weight decay** in neural-network training [3].

```
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.metrics import mean_squared_error

rng = np.random.default_rng(0)
x = rng.uniform(0, 1, 25)
y = np.sin(2 * np.pi * x) + rng.normal(0, 0.25, 25)      # true curve + noise
X = x[:, None]

# 60% train, 20% validation, 20% test
X_tmp, X_te, y_tmp, y_te = train_test_split(X, y, test_size=0.2, random_state=0)
X_tr, X_va, y_tr, y_va = train_test_split(X_tmp, y_tmp, test_size=0.25, random_state=0)

def fit(degree, lam=0.0):
    reg = Ridge(alpha=lam) if lam > 0 else LinearRegression()   # sklearn calls lambda
    ↪  "alpha"
    model = make_pipeline(PolynomialFeatures(degree), StandardScaler(), reg)
    return model.fit(X_tr, y_tr)

for degree, lam in [(1, 0), (3, 0), (15, 0), (15, 1e-2)]:
    model = fit(degree, lam)
    tr = mean_squared_error(y_tr, model.predict(X_tr))
    va = mean_squared_error(y_va, model.predict(X_va))
    print(f"degree {degree:2d}, lambda {lam:g}: train MSE {tr:.3f}, val MSE {va:.3g}")

best = fit(15, 1e-2)                                    # chosen on validation only
print("test MSE:", round(mean_squared_error(y_te, best.predict(X_te)), 3))
```

The output tells the whole story: degree 1 has train/validation MSE of $0.144/0.292$ (underfit), degree 3 has $0.031/0.052$ (good), and unregularized degree 15 has a training MSE of 0.003 but a validation MSE in the millions, because some validation points fall in wide gaps between training points (for example between $x \approx 0.27$ and $x \approx 0.61$), where the polynomial swings wildly. Adding a mild L2 penalty ($\lambda = 0.01$) to the same degree-15 model gives $0.041/0.035$, and its test MSE is 0.075.

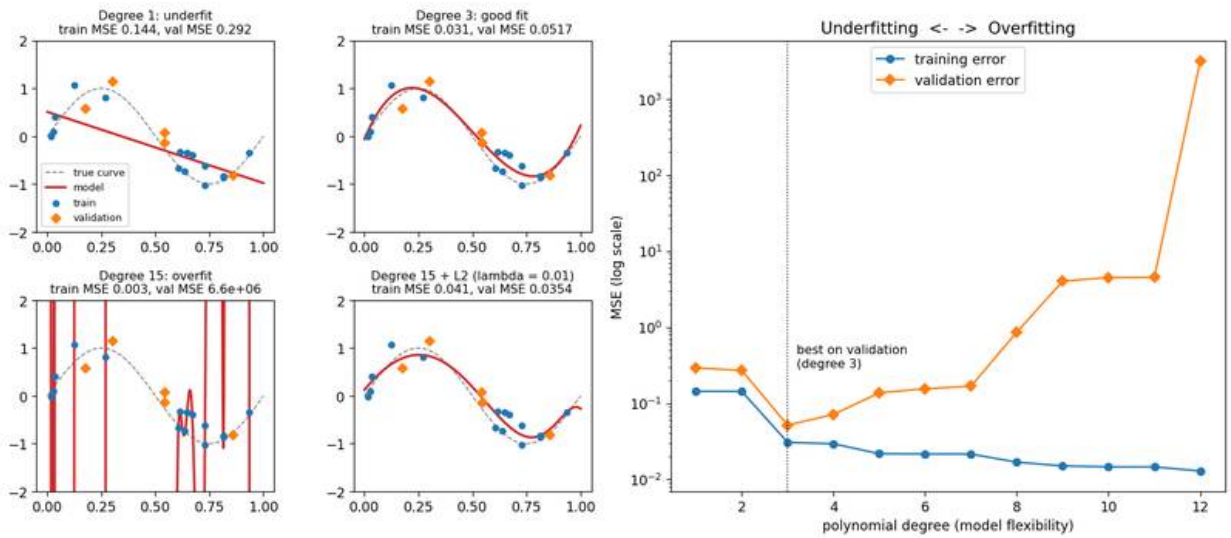


Figure 8 — Underfitting, a good fit, overfitting, and overfitting tamed by L2 regularization (training points in blue, validation points in orange). Right: as the degree grows, training error keeps falling while validation error falls and then rises; the bottom of the validation curve is the model to choose.

Evaluating a model

Plain version. A single number never tells the whole story. Choose a metric that matches what a mistake costs, and always compare against a trivial baseline.

Regression metrics

For regression, the most common metric is the MSE, $\frac{1}{m}\sum_i(\hat{y}^{(i)} - y^{(i)})^2$, here without the $\frac{1}{2}$ that was only a training convenience. Its square root, the **RMSE**, is in the same units as y and is easier to interpret ("off by about 3 points on average"). The **mean absolute error**, $\frac{1}{m}\sum_i|\hat{y}^{(i)} - y^{(i)}|$, is less sensitive to a few large errors [5].

Classification metrics

For classification, **accuracy** is the fraction of correct predictions. It is easy to understand and easy to be fooled by. In the digits data, about 10% of the images are 9s. A "model" that always answers "not a 9" is 90% accurate and completely useless.

The **confusion matrix** counts the four outcomes for a binary problem, with "positive" meaning the class we care about:

	predicted positive	predicted negative
actually positive	true positive (TP)	false negative (FN)
actually negative	false positive (FP)	true negative (TN)

From it,

$$\text{accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \qquad \text{precision} = \frac{TP}{TP + FP}, \qquad \text{recall} = \frac{TP}{TP + FN}.$$

Precision answers "when the model says positive, how often is it right?" **Recall** answers "of all the real positives, how many did the model find?" They trade off: lowering the 0.5 decision threshold of logistic regression finds more positives (higher recall) but raises more false alarms (lower precision). Which matters more depends on the cost of each mistake: a cancer screen should not miss tumors (high recall), while a spam filter should not hide real mail (high precision). The **F1 score**, the harmonic mean of precision and recall, combines both into one number [5].

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score

X, y = load_digits(return_X_y=True)
y = (y == 9).astype(int)                  # "is it a 9?": only about 10% positives
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.3, stratify=y, random_state=0)

lazy = np.zeros_like(y_te)                # always answers "not a 9"
model = LogisticRegression(max_iter=5000).fit(X_tr, y_tr)
pred = model.predict(X_te)

print("lazy accuracy :", round(accuracy_score(y_te, lazy), 3), " recall:",
 ↪  recall_score(y_te, lazy))
print("model accuracy:", round(accuracy_score(y_te, pred), 3),
      " precision:", round(precision_score(y_te, pred), 3),
      " recall:", round(recall_score(y_te, pred), 3))
print(confusion_matrix(y_te, pred))       # rows: true 0/1, columns: predicted 0/1
# lazy accuracy : 0.9   recall: 0.0
# model accuracy: 0.974  precision: 0.87  recall: 0.87
```

The lazy baseline reaches 90% accuracy with zero recall. The real model's 97.4% accuracy sounds only a little better, but its recall of 0.87 shows that it actually finds most of the 9s. Always report the baseline, and always look past accuracy when classes are imbalanced. The same metrics reappear, with more care about thresholds and multiple classes, in the DIP chapter on pattern classification.

Modern view

The mechanics in this tutorial (a parameterized model, a cost, gradient-based optimization, and held-out evaluation) have not changed in decades, and every system discussed below still uses

them. What has changed is *where the features come from*, *where the labels come from*, and *how large the models are*. A handful of reviews map these shifts well.

Machine learning as a field. Jordan and Mitchell's review in *Science* [7] frames machine learning around a single question, how to build computers that improve automatically through experience, and places it at the intersection of computer science and statistics, at the core of artificial intelligence and data science. They attribute its rapid spread across science, medicine, manufacturing, education and finance to three things arriving together: new learning algorithms and theory, an explosion in the amount of available data, and cheap computation. That framing is a useful antidote to hype: the method is only as good as the data and the experience it is given.

From feature engineering to representation learning. For a long time, the success of a classical model depended mostly on hand-designed features, the "count the inked columns" step of Figure 1 done by experts: edge histograms, texture descriptors, keypoint descriptors (see IP 12). Bengio, Courville and Vincent [21] argue that this dependence is a weakness, because the right representation is what makes a task easy or hard, and propose learning the representation itself from data. Their review covers the tools for this (probabilistic models, autoencoders, manifold learning and deep networks) and the general priors that make a representation good, such as disentangling the separate explanatory factors behind the data. In the language of this tutorial, a deep network is still "features followed by logistic regression", except that the features are produced by earlier layers whose weights are learned by the same gradient descent. That is the subject of ML 02.

Self-supervised learning. Learning representations needs data, and labels are the expensive part. Jing and Tian's survey of self-supervised visual feature learning [22] sorts the label-free "pretext tasks" into four families: generation-based (for example, reconstructing or colorizing images), context-based (exploiting spatial or temporal structure, such as the arrangement of image patches), free-semantic-label-based (using labels that come automatically, for example from rendering engines or classical methods), and cross-modal (checking whether two signals, such as video and audio, belong together). It also sets out the evaluation protocol that is still standard: judge a representation by how well it transfers to downstream tasks such as classification, detection and segmentation. The later survey by Gui et al. [23] reorganizes the field around context-based methods, contrastive learning (with negative examples, self-distillation, or feature decorrelation), generative methods dominated by masked image modeling, and hybrids of contrastive and generative objectives, reflecting the shift toward masked prediction after masked autoencoders [12]. For practitioners, Balestriero et al.'s *Cookbook of Self-Supervised Learning* [24] groups modern methods into a deep metric learning family, a self-distillation family, a canonical correlation analysis family and masked image modeling, and is candid about how much the outcome depends on details such as data augmentation and the small "projector" network used during training.

Foundation models. The report by Bommasani et al. [10] named the resulting paradigm: models trained on broad data at scale, usually self-supervised, then adapted to many downstream tasks. It highlights two consequences. **Emergence**: capabilities such as following instructions from a few examples in the prompt [11] appear with scale rather than being designed in. **Homogenization**: many applications are now adapted from the same few base models, which gives great leverage but means that a defect in the base model is inherited by everything built on it. The report also stresses how limited our understanding is of how these models work and when they fail.

Reinforcement learning from human feedback. RL has moved from games to the final stage of training language models [13], [14]. Casper et al. [25] survey what can go wrong. They follow the three steps of RLHF (collecting human feedback, fitting a reward model, and optimizing the policy) and catalog problems at each: feedback from people is noisy, inconsistent and can be manipulated; a reward model is only an imperfect proxy for what people want; and a policy optimized hard against that proxy can exploit its errors. They separate problems that better methods within RLHF could solve from fundamental limitations that would require a different approach, and they argue for transparency and for combining RLHF with other safety measures rather than relying on it alone.

Optimization at scale. Bottou, Curtis and Nocedal [18] explain why the humble stochastic gradient method, and not more sophisticated optimizers from classical numerical analysis, became the workhorse of machine learning: when data is plentiful, a cheap noisy step beats an expensive exact one. Adaptive methods such as Adam [19] add per-parameter step sizes on top. Every model mentioned in this section, however large, is trained with a descendant of the mini-batch loop in this tutorial.

Open problems. Despite the scale, several questions from this beginner's tutorial remain the hard ones. *Generalization*: very large models fit their training data almost perfectly yet still generalize, which does not match the simple overfitting picture of Figure 8, and why this happens is not fully understood. *Evaluation*: when a model is trained on a large fraction of the public internet, it is hard to guarantee that the test set was not in the training data, so a clean train/test separation, the foundation of honest evaluation, is difficult to verify. *Objectives*: the cost function is the only thing the model optimizes, and RLHF shows how hard it is to write a cost that captures what people actually want. And *data*: what a model learns is bounded by the data and labels it sees, with all their gaps and biases.

For computer vision specifically, the classical route (hand-made features plus a simple classifier) is developed in the DIP chapter on pattern classification, and the learned route continues in ML 02 and Deep Learning for Image Processing.

Key takeaways

- Machine learning replaces hand-written rules with a flexible model whose parameters are fitted to examples; it shines when the rule is hard to state but examples are easy to get.
- The paradigms differ in feedback: supervised (right answers; regression for numbers, classification for categories), unsupervised (no answers; clustering, dimensionality reduction, anomaly detection), self-supervised (answers made from the data itself), and reinforcement learning (rewards for actions). KNN is supervised; k-means is clustering.
- Keep the model $f_{w,b}(x)$ and the cost $J(w, b)$ apart: the model maps inputs to predictions; the cost maps parameters to an error. Training means minimizing J.
- Gradient descent repeats $w \leftarrow w - \alpha\, \partial J/\partial w$. At a minimum the *derivative* is zero. The learning rate must be small enough to avoid overshooting and large enough to make progress; mini-batches make each step cheap.
- Scale features before gradient descent, using statistics from the training set only.

- Logistic regression applies a sigmoid to a linear score and minimizes cross-entropy; its decision boundary is $\mathbf{w} \cdot \mathbf{x} + b = 0$, and its gradient has the same form as linear regression's.
- Judge models on data they did not train on: choose with the validation set, report once on the test set, regularize (L2) when the model overfits, and look beyond accuracy when classes are imbalanced.

Exercises

1. **A derivative by hand.** For the one-feature MSE cost $J(w, b) = \frac{1}{2m} \sum_i (wx^{(i)} + b - y^{(i)})^2$, derive $\partial J/\partial w$ and $\partial J/\partial b$. Then, for the three points $(1, 2)$, $(2, 4)$, $(3, 6)$ starting at $w = 0, b = 0$, compute one gradient-descent step with $\alpha = 0.1$.

   Hint

   The chain rule on each square gives $2(wx^{(i)} + b - y^{(i)}) \cdot x^{(i)}$, and the 2 cancels the $\frac{1}{2}$, giving the formulas in the gradient-descent section. At $w = b = 0$ the errors are $-2, -4, -6$. So $\partial J/\partial w = (-2 \cdot 1 - 4 \cdot 2 - 6 \cdot 3)/3 = -28/3$ and $\partial J/\partial b = -12/3 = -4$. After one step, $w = 0.1 \cdot 28/3 \approx 0.933$ and $b = 0.4$.

2. **Find the divergence threshold.** For $J(w) = \frac{1}{2m} \sum_i (wx^{(i)} - y^{(i)})^2$ (no bias), show that one gradient step multiplies the distance to the optimum w^* by $1 - \alpha s$, where $s = \frac{1}{m} \sum_i (x^{(i)})^2$. For which α does gradient descent converge? Check your answer with the `gd` function from the feature-scaling section.

   Hint

   $\partial J/\partial w = s\,(w - w^*)$ because J is a parabola with curvature s and minimum at w^*. So $w_{\text{new}} - w^* = (1 - \alpha s)(w - w^*)$. It converges when $|1 - \alpha s| < 1$, that is $0 < \alpha < 2/s$; at $\alpha = 1/s$ it lands on the minimum in one step. With several features, s is replaced by the largest eigenvalue of $\frac{1}{m} X^\top X$, which is why one huge-scale feature forces a tiny α.

3. **Regression for classification.** Generate 1-D data with `make_classification(n_features=1, n_informative=1, n_redundant=0, n_clusters_per_class=1)`, then add five extra examples of class 1 with very large x. Fit both `LinearRegression` (thresholding at 0.5) and `LogisticRegression`. Compare their accuracy before and after adding the extra points, and explain the difference.

   Hint

   The extra points are easy to classify, but the squared error still wants the line to pass near them. The line's slope drops, its 0.5 crossing moves toward class 1, and some ordinary class-1 points near the boundary become misclassified. The log loss of a correctly classified, far-away point is already close to 0, so logistic regression barely moves.

4. **Scaling the digits.** Train the from-scratch logistic regression of this tutorial on "9 versus not 9" with the digits data, once on raw pixels (0 to 16) and once on pixels divided by 16. Use the same number of steps and find the largest learning rate that works for each. Does scaling change the final accuracy, the speed, or both?

   Hint

   All pixels share one unit here, so the effect is milder than in Figure 6, but the raw inputs are 16 times larger, which multiplies the gradient's size and the landscape's curvature. The safe learning rate on raw pixels is roughly $16^2 = 256$ times smaller. With properly tuned learning rates, both reach similar accuracy; the scaled version is simply easier to tune.

5. **Choosing lambda.** Using the polynomial example, fix degree 15 and try $\lambda \in \{10^{-6}, 10^{-4}, 10^{-2}, 1, 100\}$. Plot training and validation MSE against λ on a log axis. Which end underfits, which overfits, and which λ would you pick? Why must you not use the test set to make this choice?

   Hint

   Tiny λ behaves like no regularization (overfitting: low training error, high validation error). Huge λ flattens the curve toward a constant (underfitting: both errors high). Pick the λ with the lowest validation error. If the test set guided the choice, it would no longer be an unbiased estimate of performance on new data.

6. **Which paradigm?** Classify each problem as supervised (regression or classification), unsupervised, self-supervised or reinforcement learning, and name the target or reward: (a) estimating the remaining battery life of a phone from usage logs with recorded outcomes; (b) grouping a million unlabeled product photos for browsing; (c) pre-training an image model by predicting the missing quarter of each photo; (d) teaching a robot arm to stack blocks with a score given only when the stack stands; (e) flagging unusual readings in a factory sensor stream with no labeled failures.

   Hint

   (a) Supervised regression; target: remaining hours. (b) Unsupervised clustering. (c) Self-supervised; the target is the hidden pixels, made from the data itself. (d) Reinforcement learning; reward: whether the stack stands, delayed until the end. (e) Unsupervised anomaly detection.

Continue through ML/DL

If the mathematics is new, take linear algebra, probability and information theory, and numerical computation before neural networks.

The rest of the track connects image filtering and CNNs with recurrent/recursive networks, transformers and representation learning. It then builds from factor models and autoencoders to VAEs,

GANs, normalizing flows, diffusion/flow matching, autoregressive models and reusable foundation models. Finish with evaluation and deployment. For the statistical decision and inference viewpoint, follow Pattern Recognition; for task-specific systems, follow Deep Learning for Computer Vision.

References

1. Shyandram, "Introduction to Machine Learning," Medium post, 2024; updated 2026. link
2. A. L. Samuel, "Some studies in machine learning using the game of checkers," *IBM Journal of Research and Development*, vol. 3, no. 3, pp. 210–229, 1959. doi
3. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016. book site
4. A. Ng, E. Shyu, A. Bagul and G. Ladwig, "Machine Learning Specialization," DeepLearning.AI and Stanford Online, Coursera. course page
5. F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. link
6. E. Alpaydin and C. Kaynak, "Optical Recognition of Handwritten Digits," UCI Machine Learning Repository, 1998. doi
7. M. I. Jordan and T. M. Mitchell, "Machine learning: Trends, perspectives, and prospects," *Science*, vol. 349, no. 6245, pp. 255–260, 2015. doi
8. R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018. book page
9. Y. LeCun and I. Misra, "Self-supervised learning: The dark matter of intelligence," Meta AI blog, 2021. link
10. R. Bommasani, D. A. Hudson, E. Adeli et al., "On the opportunities and risks of foundation models," arXiv:2108.07258, 2021. arXiv
11. T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan et al., "Language models are few-shot learners," in *Advances in Neural Information Processing Systems* (*NeurIPS*), vol. 33, 2020. link
12. K. He, X. Chen, S. Xie, Y. Li, P. Dollár and R. Girshick, "Masked autoencoders are scalable vision learners," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition* (*CVPR*), pp. 15979–15988, 2022. arXiv
13. L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright et al., "Training language models to follow instructions with human feedback," in *Advances in Neural Information Processing Systems* (*NeurIPS*), vol. 35, 2022. link
14. DeepSeek-AI, "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning," *Nature*, vol. 645, pp. 633–638, 2025. arXiv
15. A. K. Jain, "Data clustering: 50 years beyond K-means," *Pattern Recognition Letters*, vol. 31, no. 8, pp. 651–666, 2010. doi
16. I. T. Jolliffe and J. Cadima, "Principal component analysis: A review and recent developments," *Philosophical Transactions of the Royal Society A*, vol. 374, no. 2065, 20150202, 2016. doi
17. V. Chandola, A. Banerjee and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, 2009. doi
18. L. Bottou, F. E. Curtis and J. Nocedal, "Optimization methods for large-scale machine learning," *SIAM Review*, vol. 60, no. 2, pp. 223–311, 2018. doi

19. D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. International Conference on Learning Representations (ICLR)*, 2015. arXiv
20. A. E. Hoerl and R. W. Kennard, “Ridge regression: Biased estimation for nonorthogonal problems,” *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970. doi
21. Y. Bengio, A. Courville and P. Vincent, “Representation learning: A review and new perspectives,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 8, pp. 1798–1828, 2013. doi
22. L. Jing and Y. Tian, “Self-supervised visual feature learning with deep neural networks: A survey,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, no. 11, pp. 4037–4058, 2021. doi
23. J. Gui, T. Chen, J. Zhang, Q. Cao, Z. Sun, H. Luo and D. Tao, “A survey on self-supervised learning: Algorithms, applications, and future trends,” arXiv:2301.05712, 2023. arXiv
24. R. Balestriero, M. Ibrahim, V. Sobal, A. Morcos, S. Shekhar et al., “A cookbook of self-supervised learning,” arXiv:2304.12210, 2023. arXiv
25. S. Casper, X. Davies, C. Shi et al., “Open problems and fundamental limitations of reinforcement learning from human feedback,” *Transactions on Machine Learning Research*, 2023. TMLR ·arXiv

ML 01A • Linear Algebra for Learning

Vectors, matrices and tensors, norms, eigendecomposition, the SVD, the pseudo-inverse, trace and determinant, and a step-by-step derivation of PCA, with tested NumPy code.

Online edition: https://blog.shyandram.dev/en/learn/ml/ml-01a-linear-algebra/

What you'll learn

- How scalars, vectors, matrices and tensors describe images, batches and learned transformations, and how to keep their shapes straight.
- Four ways to read a matrix product, and what identity, inverse, span, rank and null space mean for a system of equations.
- How norms measure size, how special matrices (diagonal, symmetric, orthogonal) simplify computation, and how dot products define similarity.
- What eigendecomposition and the singular value decomposition (SVD) reveal about a matrix, and why the SVD gives the best low-rank approximation.
- How the Moore–Penrose pseudo-inverse solves least-squares and under-determined problems, and why conditioning and ridge regularization matter.
- How the trace and determinant work, and how to derive principal component analysis (PCA) from scratch using only these tools.

The big picture

Almost every operation inside a neural network is a linear map followed by a simple nonlinear function. A fully connected layer multiplies by a weight matrix. A convolution is a structured matrix product. Attention computes dot products between vectors and then mixes vectors with a matrix of weights. Linear algebra is the language for all of these. It tells us what a map keeps, what it throws away, what it stretches, and how to compute it without losing accuracy.

> **Explain like I'm five**
>
> A vector is a list of measurements, like the height, weight and age of one person. A matrix is a machine that takes one list and mixes it into a new list. Some machines just turn the list around (rotation), some make some parts bigger and others smaller (stretching), and some squash parts flat so the information is gone forever. Linear algebra is the instruction manual for these machines: it tells you which way each machine stretches, by how much, and whether you can undo it.

This chapter follows the "Applied Math" part of Goodfellow, Bengio and Courville's *Deep Learning* [1] and the linear algebra chapters of *Mathematics for Machine Learning* [2]. Strang's textbook [3] is the gentlest full course if you want more practice, and Golub and Van Loan [4] and Trefethen and

Bau [5] are the standard references for how these computations are done reliably on a computer. Read this chapter before ML 02 if matrix notation is unfamiliar. The companion chapters are ML 01B ·Probability and Information Theory and ML 01C ·Numerical Computation.

The chapter has a single thread. We start with objects and shapes, move to products and solving equations, then measure size with norms. Eigendecomposition and the SVD then explain the "personality" of a matrix. The pseudo-inverse, trace and determinant are tools built on top of them. Finally, we use every one of these tools to derive PCA, the classic method for compressing data, exactly as it is derived in [1].

Scalars, vectors, matrices and tensors

Plain version. Every number in machine learning lives in an array with named axes. Name every axis before you multiply anything.

A **scalar** is a single number, written in italics: $s \in \mathbb{R}$. A **vector** is an ordered list of n numbers, written in bold lowercase or plain lowercase in this chapter: $x \in \mathbb{R}^n$, with entries $x_1, \dots, x_n$. By convention a vector is a column. A **matrix** is a 2-D grid of numbers: $A \in \mathbb{R}^{m \times n}$ has m rows and n columns, and A_{ij} is the entry in row i and column j. A **tensor** is an array with any number of axes.

These objects map directly onto data:

- A grayscale image of height H and width W is a matrix in $\mathbb{R}^{H \times W}$. Flattened, it becomes a vector $x \in \mathbb{R}^{HW}$. (See Images as Arrays.)
- A dataset of n examples with d features each, stored one example per row, is a **design matrix** $X \in \mathbb{R}^{n \times d}$.
- A fully connected layer with weights $W \in \mathbb{R}^{d \times m}$ and bias $b \in \mathbb{R}^m$ maps the whole batch at once: $Y = XW + \mathbf{1}b^\top \in \mathbb{R}^{n \times m}$, where $\mathbf{1} \in \mathbb{R}^n$ is a vector of ones.
- An image batch in PyTorch is usually a 4-D tensor of shape $B \times C \times H \times W$ (batch, channels, height, width). A batch of token embeddings in a Transformer is usually $B \times N \times D$ (batch, sequence length, embedding width).

The **transpose** $A^\top$ swaps rows and columns: $(A^\top)_{ij} = A_{ji}$. It satisfies $(A^\top)^\top = A$ and $(AB)^\top = B^\top A^\top$. Note the reversed order.

Broadcasting

Writing $\mathbf{1}b^\top$ is mathematically honest but tedious, so libraries let you add a vector to every row of a matrix directly. This is called **broadcasting**: when two arrays have different shapes, axes of length 1 (or missing leading axes) are stretched to match. Broadcasting is convenient and dangerous, because a wrong shape often still runs. The most common bug is subtracting a column vector of shape $(n, 1)$ from a flat vector of shape $(n,)$, which silently produces an $n \times n$ matrix instead of n residuals.

```
import numpy as np

rng = np.random.default_rng(0)
X = rng.normal(size=(32, 64))      # batch of 32 examples, 64 features
W = rng.normal(size=(64, 10))      # linear map 64 -> 10
b = rng.normal(size=10)            # one bias per output
Y = X @ W + b                      # b broadcasts over the 32 rows
assert Y.shape == (32, 10)

# A classic broadcasting bug: (32,) - (32, 1) silently becomes (32, 32)
y_true = rng.normal(size=32)
y_pred = rng.normal(size=(32, 1))
print("buggy residual shape:", (y_true - y_pred).shape)
print("fixed residual shape:", (y_true - y_pred[:, 0]).shape)

# Four equivalent views of the same matrix product
A, B = rng.normal(size=(3, 4)), rng.normal(size=(4, 2))
entry = np.array([[A[i] @ B[:, j] for j in range(2)] for i in range(3)])
cols = np.stack([A @ B[:, j] for j in range(2)], axis=1)
outer = sum(np.outer(A[:, k], B[k]) for k in range(4))
assert np.allclose(A @ B, entry) and np.allclose(A @ B, cols) and np.allclose(A @ B, outer)
print("all views agree")
```

The output shows `(32, 32)` for the buggy line and `(32,)` for the fixed one. A mean over a 32×32 array still returns a single number, so a loss computed this way trains without errors while optimizing the wrong thing. Put `assert` statements on shapes in any code you care about.

Multiplying matrices and vectors

Plain version. A matrix product chains two linear machines into one. Each output number is a dot product between a row of the first matrix and a column of the second.

For $A \in \mathbb{R}^{m \times n}$ and $B \in \mathbb{R}^{n \times p}$, the **matrix product** $C = AB \in \mathbb{R}^{m \times p}$ is

$$C_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj}.$$

Here i indexes rows of A (and of C), j indexes columns of B (and of C), and k runs over the shared inner dimension n. The inner dimensions must match; that is the first thing to check when a shape error appears.

The same product can be read in four ways, and each one is useful:

1. **Entry view.** C_{ij} is the dot product of row i of A with column j of B. This is how attention scores $QK^\top$ are read: every query is compared with every key.
2. **Column view.** Column j of C is A times column j of B. Each column of C is a combination of the columns of A.

3. **Row view.** Row i of C is row i of A times B. In $Y = XW$, each output example is its input row pushed through the same weights.
4. **Outer-product view.** $C = \sum_k a_{:k} b_{k:}$, a sum of n rank-one matrices, each formed from column k of A and row k of B. This view explains low-rank approximation and LoRA later in the chapter.

The code above checks that all four views give the same answer.

Matrix multiplication is **associative**, $(AB)C = A(BC)$, and **distributive**, $A(B + C) = AB + AC$, but **not commutative**: $AB \neq BA$ in general, and often one of the two products is not even defined. The **elementwise** (Hadamard) product $A \odot B$ multiplies matching entries instead and needs equal shapes. In NumPy and PyTorch, `A @ B` is the matrix product and `A * B` is elementwise. The **dot product** of two vectors $x, y \in \mathbb{R}^n$ is the special case $x^\top y = \sum_i x_i y_i$, and it is symmetric: $x^\top y = y^\top x$.

Identity, inverse and systems of equations

A system of m linear equations in n unknowns can be written compactly as

$$Ax = b, \qquad A \in \mathbb{R}^{m \times n},\ x \in \mathbb{R}^n,\ b \in \mathbb{R}^m.$$

Each row of A holds the coefficients of one equation, x holds the unknowns, and b holds the right-hand sides.

The **identity matrix** I_n has ones on the diagonal and zeros elsewhere; it leaves every vector unchanged, $I_n x = x$. A square matrix A is **invertible** if there is a matrix A^{-1} with $A^{-1}A = AA^{-1} = I$. Then the system has exactly one solution, $x = A^{-1}b$.

That formula is excellent for derivations and poor for computation. Explicitly forming A^{-1} costs more and loses more accuracy than solving the system directly with an LU, Cholesky or QR factorization [4], [5]. In NumPy, write `np.linalg.solve(A, b)`, not `np.linalg.inv(A) @ b`. In this chapter, A^{-1} is a thinking tool.

Linear dependence, span and rank

Plain version. Rank counts how many genuinely different directions a matrix can produce. Repeated or redundant columns add nothing new.

A **linear combination** of vectors $v^{(1)}, \ldots, v^{(k)}$ is $\sum_i c_i v^{(i)}$ for scalars c_i. The **span** of a set of vectors is the set of all their linear combinations. Because $Ax = \sum_j x_j a_{:j}$, the vector Ax is a linear combination of the columns $a_{:j}$ of A. So $Ax = b$ has a solution exactly when b lies in the span of the columns, called the **column space** (or range) of A.

A set of vectors is **linearly independent** if no vector in it is a linear combination of the others; equivalently, $\sum_i c_i v^{(i)} = 0$ only when every $c_i = 0$. The **rank** of A is the number of linearly independent columns, which always equals the number of linearly independent rows. For $A \in \mathbb{R}^{m \times n}$, $\operatorname{rank}(A) \leq \min(m, n)$; if equality holds, A has **full rank**.

The **null space** of A is the set of vectors z with $Az = 0$. Those are the input directions that A destroys. The **rank–nullity theorem** says

$$\operatorname{rank}(A) + \dim \operatorname{null}(A) = n,$$

where n is the number of columns and dim is the number of independent directions in the null space. This one equation explains the three regimes of $Ax = b$:

Shape and rank	What happens	Typical ML case
square, full rank	exactly one solution	a well-posed change of coordinates
tall ($m > n$), full column rank	usually no exact solution; solve in the least-squares sense	regression with more examples than features
wide ($m < n$) or rank-deficient	infinitely many solutions (if any)	more parameters than constraints, as in deep networks

For a square matrix, "invertible", "full rank", "columns linearly independent" and "null space contains only zero" are all the same statement. A square matrix that fails them is called **singular**.

Norms: measuring size

Plain version. A norm turns a vector into one non-negative number that says how big it is. Different norms disagree about what "big" means.

A **norm** is any function f with $f(x) = 0 \Rightarrow x = 0$, $f(x + y) \le f(x) + f(y)$ (the triangle inequality), and $f(\alpha x) = |\alpha| f(x)$ for every scalar α. The most common family is the L^p norm,

$$\|x\|_p = \Big(\sum_i |x_i|^p \Big)^{1/p}, \qquad p \ge 1.$$

- $\|x\|_2$, the **Euclidean** norm, is ordinary length; we often write $\|x\|$. Its square $\|x\|_2^2 = x^\top x$ is easier to differentiate and appears in mean-squared-error losses and weight decay.
- $\|x\|_1 = \sum_i |x_i|$ grows at the same rate everywhere, even near zero. That makes it useful when the difference between exactly zero and slightly nonzero matters, which is why L^1 penalties encourage sparse solutions.
- $\|x\|_\infty = \max_i |x_i|$, the **max norm**, is the size of the largest entry. Adversarial-robustness work often bounds perturbations in this norm.
- The count of nonzero entries is sometimes called the "L^0 norm", but it is not a norm: scaling x by 2 does not change it.

The dot product connects to the Euclidean norm through angles: $x^\top y = \|x\|_2 \|y\|_2 \cos\theta$, where θ is the angle between x and y. The **cosine similarity**

$$\cos(x, y) = \frac{x^\top y}{\|x\|_2 \|y\|_2}$$

is defined for nonzero vectors and ignores scale. That is useful for some embeddings (contrastive learning normalizes features for this reason) and wrong when magnitude carries meaning, such as image brightness.

Matrix norms

For matrices we use two norms constantly. The **Frobenius norm** treats the matrix as one long vector,

$$\|A\|_F = \sqrt{\sum_{i,j} A_{ij}^2}.$$

The **spectral norm** (operator 2-norm) measures the largest factor by which A can stretch a unit vector,

$$\|A\|_2 = \max_{\|x\|_2=1} \|Ax\|_2 = \sigma_{\max}(A),$$

where $\sigma_{\max}(A)$ is the largest singular value, defined in the SVD section below. In terms of all singular values $\sigma_1 \geq \sigma_2 \geq \ldots$, we have $\|A\|_F^2 = \sum_i \sigma_i^2$ while $\|A\|_2 = \sigma_1$. So many small singular values can make the Frobenius norm large without changing the largest amplification. The spectral norm bounds how much a layer can magnify a perturbation of its input, which is why it appears in robustness and GAN-stability work [17].

```
import numpy as np

x = np.array([3.0, -4.0, 0.0, 1.0])
for p in [1, 2, np.inf]:
    print(f"L{p} norm:", np.linalg.norm(x, p))
print("L0 'norm' (count of nonzeros):", np.count_nonzero(x))

A = np.array([[3.0, 0.0], [4.0, 5.0]])
s = np.linalg.svd(A, compute_uv=False)
print("singular values:", s.round(4))
print("Frobenius:", np.linalg.norm(A, "fro"), "=", np.sqrt((s**2).sum()))
print("spectral :", np.linalg.norm(A, 2), "=", s[0])
assert np.isclose(np.linalg.norm(A, "fro")**2, np.trace(A.T @ A))
```

For $x = (3, -4, 0, 1)$ this prints $L^1 = 8$, $L^2 = \sqrt{26} \approx 5.10$, $L^\infty = 4$ and three nonzeros. For the 2×2 matrix the singular values are $3\sqrt{5} \approx 6.71$ and $\sqrt{5} \approx 2.24$, giving $\|A\|_F = \sqrt{50} \approx 7.07$ and $\|A\|_2 \approx 6.71$.

Special kinds of matrices and vectors

Plain version. Some matrices are much easier to work with than others. Learning to spot them saves computation and explains many design choices.

- A **diagonal** matrix $D = \mathrm{diag}(v)$ has nonzero entries only on its main diagonal. Multiplying by it scales each coordinate: $\mathrm{diag}(v)x = v \odot x$. Its inverse, when every $v_i \neq 0$, is $\mathrm{diag}(1/v_1, \ldots, 1/v_n)$. Per-channel scaling in normalization layers is a diagonal matrix in disguise. Rectangular diagonal matrices also exist; they appear in the SVD.

- A **symmetric** matrix satisfies $A = A^\top$. Covariance matrices, Gram matrices $X^\top X$, Hessians of smooth functions and distance matrices are symmetric.
- A **unit vector** has $\|x\|_2 = 1$. Two vectors are **orthogonal** if $x^\top y = 0$; if they are also unit vectors, they are **orthonormal**.
- An **orthogonal matrix** Q is square with orthonormal rows and columns, so $Q^\top Q = QQ^\top = I$ and $Q^{-1} = Q^\top$. Its inverse is free to compute. It preserves lengths and angles, $\|Qx\|_2 = \|x\|_2$, so geometrically it is a rotation, a reflection, or both.

If $Q \in \mathbb{R}^{n\times k}$ has $k < n$ orthonormal columns, then $Q^\top Q = I_k$ but $QQ^\top \neq I_n$. The matrix $P = QQ^\top$ is the **orthogonal projection** onto the span of the columns of Q. The projection Px is the closest point to x in that subspace, and the residual $x - Px$ is perpendicular to every column of Q. Projecting twice changes nothing: $P^2 = P$. PCA, below, is exactly the search for the best such Q.

Eigendecomposition

Plain version. Most vectors change direction when a matrix acts on them. Eigenvectors are the special directions that only get stretched or shrunk, and the eigenvalue says by how much.

An **eigenvector** of a square matrix $A \in \mathbb{R}^{n\times n}$ is a nonzero vector v with

$$Av = \lambda v,$$

where the scalar λ is the corresponding **eigenvalue**. If v is an eigenvector, so is cv for any $c \neq 0$, so we usually normalize eigenvectors to unit length.

Suppose A has n linearly independent eigenvectors $v^{(1)}, \dots, v^{(n)}$ with eigenvalues $\lambda_1, \dots, \lambda_n$. Stack them as columns of V and put the eigenvalues in $\Lambda = \operatorname{diag}(\lambda_1, \dots, \lambda_n)$. Then $AV = V\Lambda$, so

$$A = V\Lambda V^{-1}.$$

This is the **eigendecomposition**. It says: change to eigenvector coordinates (V^{-1}), scale each coordinate independently (Λ), and change back (V). Not every matrix has one. The shear $\begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$ has only one independent eigenvector. A rotation by 90° has eigenvalues $\pm i$, which are complex, because no real direction is left unturned.

The symmetric case

Symmetric matrices are the friendly case, and they are the ones ML meets most. The **spectral theorem** says that every real symmetric matrix can be written as

$$A = Q\Lambda Q^\top,$$

where Q is orthogonal (its columns are orthonormal eigenvectors) and Λ is real and diagonal [3], [4]. Because $Q^{-1} = Q^\top$, no inversion is needed. Geometrically, A stretches space by λ_i along the direction $q^{(i)}$, the i-th column of Q. The decomposition is unique up to the order of the eigenvalues

and the choice of basis within any repeated eigenvalue; by convention we sort eigenvalues from largest to smallest.

For a symmetric A, the **quadratic form** $f(x) = x^\top Ax$ restricted to unit vectors is bounded by the extreme eigenvalues:

$$\lambda_{\min} \le x^\top Ax \le \lambda_{\max} \qquad \text{for } \|x\|_2 = 1,$$

with the maximum reached at the top eigenvector and the minimum at the bottom one. The ratio $x^\top Ax / x^\top x$ is called the **Rayleigh quotient**. This fact is the engine of the PCA derivation later.

Positive definite matrices

The signs of the eigenvalues classify symmetric matrices:

Name	Eigenvalues	Equivalent condition
positive definite (PD)	all $\lambda_i > 0$	$x^\top Ax > 0$ for every $x \neq 0$
positive semidefinite (PSD)	all $\lambda_i \ge 0$	$x^\top Ax \ge 0$ for every x
negative (semi)definite	all $\lambda_i < 0$ (or ≤ 0)	the reverse inequalities
indefinite	both signs present	$x^\top Ax$ takes both signs

Every covariance matrix and every Gram matrix $X^\top X$ is PSD, because $v^\top X^\top Xv = \|Xv\|_2^2 \ge 0$. A Gaussian density needs a PD covariance so that it can be inverted. The Hessian of a loss is PD at a strict local minimum and indefinite at a saddle point; ML 01C uses this to explain why optimization can be slow. A PD matrix has a **Cholesky factorization** $A = LL^\top$ with L lower triangular, which is the fastest stable way to solve systems with it and to sample from a Gaussian [4].

```
import numpy as np

rng = np.random.default_rng(1)
M = rng.normal(size=(4, 4))
S = M @ M.T + 0.1 * np.eye(4)        # symmetric positive definite
lam, Q = np.linalg.eigh(S)           # ascending eigenvalues, orthonormal Q
assert np.allclose(Q @ np.diag(lam) @ Q.T, S)
assert np.allclose(Q.T @ Q, np.eye(4))
print("eigenvalues:", lam.round(3), "all positive:", bool((lam > 0).all()))

# The quadratic form x^T S x is bounded by the extreme eigenvalues
x = rng.normal(size=4); x /= np.linalg.norm(x)
print("Rayleigh quotient in range:", lam[0] <= x @ S @ x <= lam[-1])

# A non-symmetric matrix can have complex eigenvalues
R = np.array([[0.0, -1.0], [1.0, 0.0]])  # 90-degree rotation
print("rotation eigenvalues:", np.linalg.eigvals(R))
```

Use `np.linalg.eigh` for symmetric matrices: it is faster, returns real eigenvalues in ascending order and orthonormal eigenvectors. `np.linalg.eig` handles the general case and may return complex numbers, as the rotation shows.

Singular value decomposition

Plain version. Every matrix, of any shape, does the same three things in order: it rotates, it stretches along perpendicular axes, and it rotates again. The SVD tells you those rotations and stretch factors.

Eigendecomposition needs a square matrix and may not exist. The **singular value decomposition** always exists, for every real matrix $A \in \mathbb{R}^{m \times n}$:

$$A = U \Sigma V^\top.$$

- $U \in \mathbb{R}^{m \times m}$ is orthogonal. Its columns are the **left singular vectors**, directions in the output space.
- $\Sigma \in \mathbb{R}^{m \times n}$ is rectangular diagonal. Its diagonal entries $\sigma_1 \geq \sigma_2 \geq \ldots \geq 0$ are the **singular values**.
- $V \in \mathbb{R}^{n \times n}$ is orthogonal. Its columns are the **right singular vectors**, directions in the input space.

Read from right to left, $Ax = U \Sigma V^\top x$ first expresses x in the V coordinates (a rotation or reflection), then scales coordinate i by σ_i, then places the result along the U directions. Figure 1 shows this for a 2×2 matrix: the unit circle becomes an ellipse whose semi-axes have lengths σ_1 and σ_2.

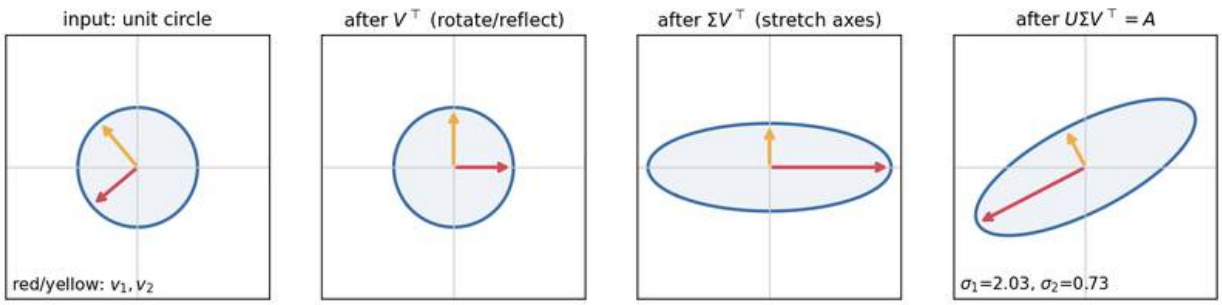


Figure 1 — The SVD as rotate, stretch, rotate. The right singular vectors v1 and v2 (red, yellow) are first rotated onto the axes by V-transpose, stretched by the singular values, then rotated by U. The final ellipse is the image of the unit circle under A.

The SVD and eigendecomposition are closely related. Since $A^\top A = V \Sigma^\top \Sigma V^\top$ and $AA^\top = U \Sigma \Sigma^\top U^\top$, the right singular vectors are eigenvectors of $A^\top A$, the left singular vectors are eigenvectors of $AA^\top$, and $\sigma_i = \sqrt{\lambda_i(A^\top A)}$. For a symmetric PSD matrix, the SVD and the eigendecomposition coincide.

The number of nonzero singular values equals the rank r. Keeping only those gives the **compact**

(thin) SVD $A = U_r \Sigma_r V_r^\top = \sum_{i=1}^{r} \sigma_i u^{(i)} v^{(i)\top}$, a sum of r rank-one matrices. That is the outer-product view of a matrix product again.

The best low-rank approximation

Truncating that sum after k terms gives $A_k = \sum_{i=1}^{k} \sigma_i u^{(i)} v^{(i)\top}$. The **Eckart–Young theorem** [6] says that A_k is the best rank-k approximation of A in the Frobenius norm:

$$\min_{\operatorname{rank}(B) \le k} \|A - B\|_F^2 = \|A - A_k\|_F^2 = \sum_{i>k} \sigma_i^2 .$$

The same A_k is also optimal in the spectral norm, with error $\|A - A_k\|_2 = \sigma_{k+1}$ [4]. In words: throw away the smallest singular values, and the energy you lose is exactly what you threw away. Storing A_k needs $k(m + n + 1)$ numbers instead of mn.

Figure 2 applies this to a 256 × 256 grayscale photograph. Natural images have rapidly decaying singular values, so a rank-50 approximation already keeps the main structure. The streaky artifacts at low rank come from the fact that each rank-one term is an outer product of a column pattern and a row pattern.

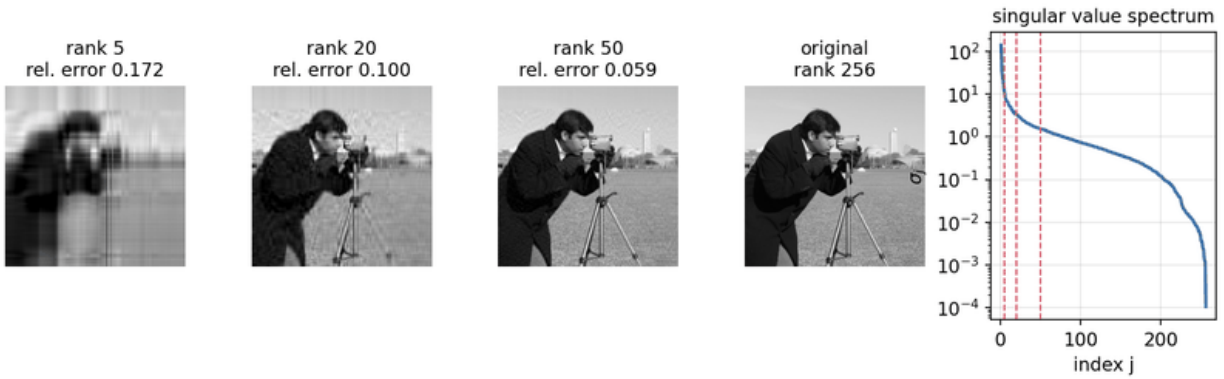


Figure 2 — Rank-k approximations of the 256×256 'camera' image from scikit-image, with the relative Frobenius error in each title. Right: singular values on a log scale; dashed lines mark k = 5, 20, 50. The error of each approximation is set by the singular values to the right of its line.

```
import numpy as np

rng = np.random.default_rng(0)
A = rng.normal(size=(12, 8))
U, s, Vt = np.linalg.svd(A, full_matrices=False)
assert np.allclose(U @ np.diag(s) @ Vt, A)
k = 3
Ak = (U[:, :k] * s[:k]) @ Vt[:k]
err_F = np.linalg.norm(A - Ak, "fro")**2
err_2 = np.linalg.norm(A - Ak, 2)
assert np.isclose(err_F, (s[k:]**2).sum()) and np.isclose(err_2, s[k])
print("rank-k squared Frobenius error:", round(err_F, 3))
```

```
print("rank-k spectral error = sigma_{k+1}:", round(err_2, 3), round(s[k], 3))
# Eigenvalues of A^T A are squared singular values of A
assert np.allclose(np.sort(np.linalg.eigvalsh(A.T @ A))[::-1], s**2)
print("storage: full", A.size, "numbers; rank-3 factors", k * (12 + 8 + 1))
```

`full_matrices=False` returns the thin SVD, which is what you almost always want. `(U[:, :k] * s[:k])` scales the columns of U by broadcasting instead of building a diagonal matrix. Both Eckart–Young identities hold to machine precision.

One caveat: the directions with the largest singular values carry the most energy, not necessarily the most useful information. For classification, a small-variance direction can be the one that separates the classes. PR 05 contrasts PCA with discriminant methods for exactly this reason.

The Moore–Penrose pseudo-inverse

Plain version. When a matrix has no inverse, the pseudo-inverse gives the next best thing: the answer that fits the data as closely as possible, and among equally good answers, the smallest one.

Only square, full-rank matrices have inverses. For any $A \in \mathbb{R}^{m\times n}$, the **Moore–Penrose pseudo-inverse** $A^+ \in \mathbb{R}^{n\times m}$ is the unique matrix satisfying the four Penrose conditions [7]:

$$AA^+A = A, \quad A^+AA^+ = A^+, \quad (AA^+)^\top = AA^+, \quad (A^+A)^\top = A^+A.$$

It is easiest to compute from the SVD. If $A = U\Sigma V^\top$, then

$$A^+ = V\Sigma^+U^\top,$$

where $\Sigma^+ \in \mathbb{R}^{n\times m}$ is formed by transposing Σ and replacing each nonzero singular value σ_i by $1/\sigma_i$ (zeros stay zero). Goodfellow et al. give the equivalent limit $A^+ = \lim_{\alpha\searrow 0}(A^\top A + \alpha I)^{-1}A^\top$ [1], which already hints at ridge regression.

The pseudo-inverse gives the right answer in every regime of the rank table:

- **Tall, full column rank** ($m > n$). $Ax = b$ usually has no exact solution. Then $x = A^+b = (A^\top A)^{-1}A^\top b$ is the **least-squares** solution: it minimizes $\|Ax - b\|_2^2$.
- **Wide, full row rank** ($m < n$). There are infinitely many solutions. Then $x = A^+b = A^\top(AA^\top)^{-1}b$ is the solution with the smallest $\|x\|_2$, the **minimum-norm** solution. Any other solution adds a null-space vector, which can only increase the norm.
- **Rank-deficient.** A^+b is both: among all least-squares minimizers, the one with the smallest norm.

The minimum-norm property is a small window into a modern question. Gradient descent on an under-determined least-squares problem, started from zero, converges to the minimum-norm solution, because every gradient $A^\top(Ax - b)$ lies in the row space of A. Over-parameterized networks are not linear, but this simple case is the usual starting point for thinking about which of many perfect fits a training algorithm picks.

Least squares, the normal equations and conditioning

For a regression design matrix $X \in \mathbb{R}^{n \times d}$ and targets $y \in \mathbb{R}^n$, least squares minimizes $\|Xw - y\|_2^2$. Setting the gradient to zero gives the **normal equations**

$$X^\top X w = X^\top y.$$

These are correct but numerically risky. The sensitivity of a linear problem is measured by the **condition number** $\kappa_2(A) = \sigma_{\max}/\sigma_{\min}$. Roughly, relative errors in the data can be amplified by up to a factor κ in the solution [5]. Forming $X^\top X$ squares the condition number, $\kappa(X^\top X) = \kappa(X)^2$, so a mildly ill-conditioned problem becomes a severely ill-conditioned one. Library least-squares solvers (`np.linalg.lstsq`, `torch.linalg.lstsq`) work with QR or SVD factorizations of X directly and avoid that squaring [4].

When two features are almost identical, $\sigma_{\min}$ is tiny, and the individual coefficients become extremely sensitive even when predictions are fine. **Ridge regression** [8] adds a penalty $\lambda\|w\|_2^2$ with $\lambda > 0$:

$$w_{\text{ridge}} = (X^\top X + \lambda I)^{-1} X^\top y = \sum_i \frac{\sigma_i}{\sigma_i^2 + \lambda} \, (u^{(i)\top} y) \, v^{(i)}.$$

The SVD form on the right shows what ridge does. Plain least squares uses the factor $1/\sigma_i$, which explodes for small σ_i. Ridge uses $\sigma_i/(\sigma_i^2+\lambda)$, which stays bounded by $1/(2\sqrt{\lambda})$. Large-σ directions are nearly untouched; small-σ directions are suppressed. The price is bias. In neural networks the same penalty is called **weight decay**.

```
import numpy as np

rng = np.random.default_rng(0)
# Over-determined: more equations than unknowns -> least squares
x = np.linspace(-1, 1, 40)
X = np.c_[np.ones_like(x), x]
y = 2 + 3*x + rng.normal(0, .05, len(x))
w, residuals, rank, sv = np.linalg.lstsq(X, y, rcond=None)
print("intercept and slope:", w.round(3), "rank:", rank)
assert np.allclose(w, np.linalg.pinv(X) @ y)

# Under-determined: fewer equations than unknowns -> minimum-norm solution
A = rng.normal(size=(2, 5)); b = rng.normal(size=2)
w_min = np.linalg.pinv(A) @ b
null = np.linalg.svd(A)[2][-1]             # a direction with A @ null = 0
w_other = w_min + 0.7 * null
print("both solve Aw=b:", np.allclose(A @ w_min, b), np.allclose(A @ w_other, b))
print("norms:", np.linalg.norm(w_min).round(3), "<", np.linalg.norm(w_other).round(3))

# Near-collinear features: condition number and ridge
z = rng.normal(size=200)
F = np.c_[z, z + 1e-4 * rng.normal(size=200)]
t = z + 0.01 * rng.normal(size=200)
print("cond(F) = %.1e, cond(F^T F) = %.1e" % (np.linalg.cond(F), np.linalg.cond(F.T @ F)))
w_ls = np.linalg.lstsq(F, t, rcond=None)[0]
```

```
lam = 1e-2
w_ridge = np.linalg.solve(F.T @ F + lam * np.eye(2), F.T @ t)
print("least squares:", w_ls.round(2), " ridge:", w_ridge.round(3))
```

The line fit recovers intercept ≈ 2.00 and slope ≈ 3.01, and `pinv` agrees with `lstsq`. The underdetermined system has two exact solutions; the pseudo-inverse one has the smaller norm (0.885 versus 1.128). In the collinear case, $\kappa(F) \approx 2 \times 10^4$ while $\kappa(F^\top F) \approx 4 \times 10^8$, its square. Least squares returns wild coefficients $(-2.63, 3.63)$ whose sum is still about 1, so its predictions are fine. Ridge returns the stable split $(0.5, 0.5)$. The intercept column in the first example lets the line have a nonzero offset. In a real pipeline, fit any centering and scaling on the training set only, then apply it unchanged to validation and test data.

The trace operator

Plain version. The trace adds up the diagonal. It looks too simple to matter, but it lets you rewrite awkward sums as matrix expressions and move matrices around inside them.

The **trace** of a square matrix is

$$\operatorname{Tr}(A) = \sum_i A_{ii}.$$

Its useful properties are:

- **Frobenius norm.** $\|A\|_F = \sqrt{\operatorname{Tr}(AA^\top)} = \sqrt{\operatorname{Tr}(A^\top A)}$.
- **Transpose.** $\operatorname{Tr}(A) = \operatorname{Tr}(A^\top)$.
- **Cyclic property.** $\operatorname{Tr}(ABC) = \operatorname{Tr}(CAB) = \operatorname{Tr}(BCA)$ whenever the products are defined, even when the matrices are not square and the intermediate products have different shapes. In particular $\operatorname{Tr}(AB) = \operatorname{Tr}(BA)$ for $A \in \mathbb{R}^{m\times n}$, $B \in \mathbb{R}^{n\times m}$. You may cycle, but not arbitrarily reorder: $\operatorname{Tr}(ABC) \neq \operatorname{Tr}(ACB)$ in general.
- **Scalars.** A scalar is its own trace, $a = \operatorname{Tr}(a)$. This turns a quadratic form into a trace: $x^\top Ax = \operatorname{Tr}(x^\top Ax) = \operatorname{Tr}(Axx^\top)$.
- **Eigenvalues.** $\operatorname{Tr}(A) = \sum_i \lambda_i$. For a covariance matrix, that is the total variance, the sum of the variances along every principal direction.

The scalar trick is how many expectations are evaluated. For a random vector x with mean zero and covariance Σ, $\mathbb{E}[x^\top Ax] = \mathbb{E}[\operatorname{Tr}(Axx^\top)] = \operatorname{Tr}(A\Sigma)$, because trace and expectation are both linear.

The determinant

Plain version. The determinant of a square matrix says how much it scales area (in 2-D) or volume (in higher dimensions). If it is zero, the matrix flattens space and cannot be undone.

The **determinant** $\det(A)$ of a square matrix is a single number with these properties:

- $\det(A) = \prod_i \lambda_i$, the product of the eigenvalues.
- $|\det(A)|$ is the factor by which A scales volume. The unit cube maps to a parallelepiped of volume $|\det(A)|$.
- The sign records orientation. A negative determinant means A includes a reflection.
- $\det(A) = 0$ exactly when A is singular: some direction is collapsed to zero, so information is lost.
- $\det(AB) = \det(A)\det(B)$, $\det(A^\top) = \det(A)$, and $\det(A^{-1}) = 1/\det(A)$.

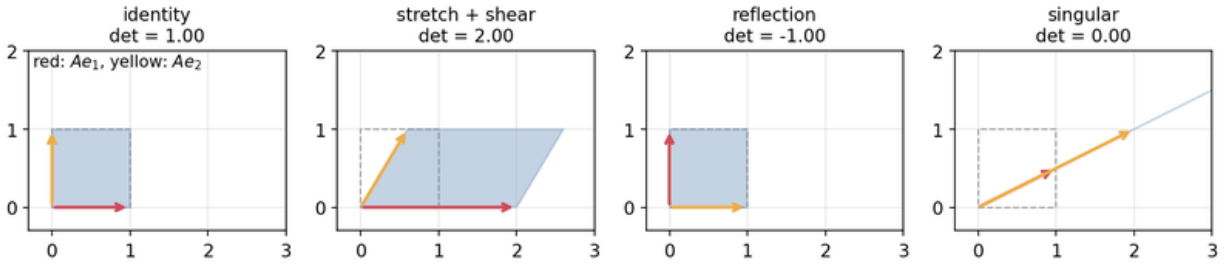


Figure 3 — The determinant as signed area. Dashed: the unit square. Shaded: its image, spanned by Ae1 (red) and Ae2 (yellow). A shear does not change area; a reflection flips the sign; a singular matrix collapses the square onto a line (det = 0).

Determinants appear in machine learning in two main places. The multivariate Gaussian density contains $\det(\Sigma)^{-1/2}$, the volume normalization of the covariance; ML 01B uses it. Normalizing flows (ML 06) need $\log|\det J|$ of a Jacobian matrix J to account for how an invertible network stretches probability mass. In both cases you need the logarithm, never the raw value: the determinant of a 500×500 diagonal matrix with entries 10^{-3} is 10^{-1500}, which underflows to zero in float64.

```
import numpy as np

rng = np.random.default_rng(2)
A, B, C = rng.normal(size=(3, 4)), rng.normal(size=(4, 5)), rng.normal(size=(5, 3))
print("cyclic trace:", np.isclose(np.trace(A @ B @ C), np.trace(C @ A @ B)),
      np.isclose(np.trace(A @ B @ C), np.trace(B @ C @ A)))

S = rng.normal(size=(4, 4))
lam = np.linalg.eigvals(S)
print("trace = sum of eigenvalues:", np.isclose(np.trace(S), lam.sum().real))
print("det = product of eigenvalues:", np.isclose(np.linalg.det(S), np.prod(lam).real))

# log-determinant of a covariance without overflow/underflow
Sigma = np.diag(np.full(500, 1e-3))
print("det underflows:", np.linalg.det(Sigma))
sign, logdet = np.linalg.slogdet(Sigma)
print("slogdet:", sign, round(logdet, 2))
L = np.linalg.cholesky(Sigma)
print("via Cholesky:", round(2 * np.log(np.diag(L)).sum(), 2))
```

The direct determinant prints `0.0`, while `slogdet` and the Cholesky route both give $\log\det =$

$500 \log 10^{-3} \approx -3453.88$. For a PD matrix, $\det(A) = \prod_i L_{ii}^2$ when $A = LL^\top$, so $\log\det A = 2\sum_i \log L_{ii}$.

Worked example: deriving principal component analysis

Plain version. PCA looks for the flat subspace that the data cloud lies closest to. Projecting onto it compresses each point to a few numbers while losing as little as possible.

PCA goes back to Pearson's "lines and planes of closest fit" [9] and Hotelling's analysis of "principal components" [10]; Jolliffe and Cadima give a modern review [11]. We derive it in the style of [1], using only the tools above.

Setup. We have n points $x^{(1)}, \dots, x^{(n)} \in \mathbb{R}^d$ and want to store each one with only $l < d$ numbers. First subtract the mean $\mu = \frac{1}{n}\sum_i x^{(i)}$ from every point, so we can assume the data are centered. Stack the centered points as rows of $X \in \mathbb{R}^{n \times d}$.

Step 1: choose the form of the code. We use a linear **decoder** $g(c) = Dc$ with $D \in \mathbb{R}^{d \times l}$. To make the problem well-posed we require the columns of D to be orthonormal, $D^\top D = I_l$. (Without this, we could shrink c and enlarge D forever.)

Step 2: find the best code for a fixed decoder. For a given x, choose c to minimize the reconstruction error:

$$c^* = \arg\min_c \|x - Dc\|_2^2 = \arg\min_c \left(-2x^\top Dc + c^\top D^\top Dc \right) = \arg\min_c \left(-2x^\top Dc + c^\top c \right).$$

We expanded the square, dropped $x^\top x$ (it does not depend on c) and used $D^\top D = I$. The gradient with respect to c is $-2D^\top x + 2c$. Setting it to zero gives the **encoder**

$$f(x) = D^\top x.$$

So encoding is a matrix–vector product, and the reconstruction is $r(x) = g(f(x)) = DD^\top x$, the orthogonal projection of x onto the span of D.

Step 3: find the best decoder. Now choose D to minimize the total reconstruction error over all points:

$$D^* = \arg\min_D \|X - XDD^\top\|_F^2 \quad \text{subject to } D^\top D = I_l.$$

Start with $l = 1$, so D is a single unit vector d. Write the Frobenius norm as a trace and expand:

$$\begin{aligned} \|X - Xdd^\top\|_F^2 &= \operatorname{Tr}\left((X - Xdd^\top)^\top (X - Xdd^\top)\right) \\ &= \operatorname{Tr}(X^\top X) - 2\operatorname{Tr}(X^\top Xdd^\top) + \operatorname{Tr}(dd^\top X^\top Xdd^\top) \\ &= \operatorname{Tr}(X^\top X) - 2\, d^\top X^\top Xd + d^\top X^\top Xd\, (d^\top d) \\ &= \operatorname{Tr}(X^\top X) - d^\top X^\top Xd. \end{aligned}$$

The cyclic property moved $d^\top$ to the front in the second and third terms, and $d^\top d = 1$ simplified the last line. The first term does not depend on d. So minimizing the error is the same as

$$d^* = \arg\max_d \; d^\top X^\top Xd \quad \text{subject to } d^\top d = 1.$$

That is a Rayleigh quotient of the symmetric PSD matrix $X^\top X$. By the eigendecomposition section, it is maximized by the **eigenvector of $X^\top X$ with the largest eigenvalue**.

Step 4: more than one component. Repeating the argument for general l (for example, by induction, requiring each new direction to be orthogonal to the previous ones) gives the answer: the columns of D are the eigenvectors of $X^\top X$ for the l largest eigenvalues [1]. The minimum error equals the sum of the discarded eigenvalues.

Step 5: the variance view and the SVD link. Dividing by n turns $X^\top X$ into the sample covariance $S = \frac{1}{n} X^\top X$ (some texts use $n-1$). The quantity $d^\top S d$ is the variance of the projected data Xd. So "minimize reconstruction error" and "maximize retained variance" select the same directions: Pearson's and Hotelling's views agree. Finally, if the centered data matrix has thin SVD $X = U\Sigma V^\top$, then $X^\top X = V\Sigma^2 V^\top$. The principal directions are the right singular vectors V, the eigenvalues are σ_i^2, and the codes are $XV_l = U_l\Sigma_l$. In practice PCA is computed with the SVD of X rather than by forming the covariance, for the conditioning reason we met in least squares.

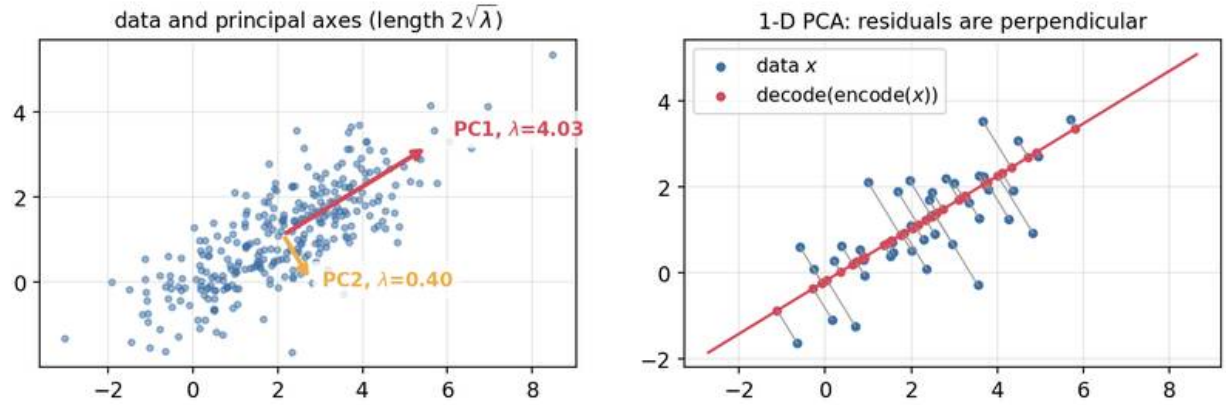


Figure 4 — PCA on 300 correlated 2-D points. Left: principal axes from the eigendecomposition of the sample covariance, drawn with length $2\sqrt{\lambda}$. Right: a 1-D PCA reconstruction $\mathrm{decode}(\mathrm{encode}(x))$; each gray residual meets the red line at a right angle, because the reconstruction is an orthogonal projection.

```
import numpy as np

rng = np.random.default_rng(0)
n, d, l = 500, 5, 2
Z = rng.normal(size=(n, l))
X = Z @ rng.normal(size=(l, d)) + 0.1 * rng.normal(size=(n, d)) + 3.0

mu = X.mean(axis=0)
Xc = X - mu                                  # centre first
S = Xc.T @ Xc / n                            # sample covariance (d x d)
lam, Q = np.linalg.eigh(S)
lam, Q = lam[::-1], Q[:, ::-1]               # descending
D = Q[:, :l]                                 # decoder matrix, orthonormal columns

codes = Xc @ D                               # encode: f(x) = D^T (x - mu)
X_hat = codes @ D.T + mu                     # decode: g(c) = D c + mu
```

```
mse = np.mean(np.sum((X - X_hat)**2, axis=1))
print("explained variance ratio:", (lam[:l].sum() / lam.sum()).round(4))
print("reconstruction MSE:", mse.round(5), " sum of discarded eigenvalues:",
 ↪  lam[l:].sum().round(5))
assert np.isclose(mse, lam[l:].sum())

# Same subspace from the SVD of the centred data matrix
U, s, Vt = np.linalg.svd(Xc, full_matrices=False)
assert np.allclose(s[:l]**2 / n, lam[:l])
assert np.allclose(np.abs(Vt[:l] @ D), np.eye(l), atol=1e-8)
print("SVD and eigendecomposition agree")
```

The synthetic data are 5-D points that really live near a 2-D plane, plus small noise. Two components explain 99.36% of the variance, and the average squared reconstruction error equals the sum of the three discarded eigenvalues, exactly as Step 4 predicts. The SVD route finds the same directions (up to sign, which is arbitrary for eigenvectors).

Three practical notes. Always center the data, or the first "component" just points at the mean. Standardize features if they have incomparable units, because PCA is not scale-invariant. And fit the mean and directions on training data only. PCA is the linear ancestor of autoencoders; ML 05A develops the probabilistic version and its nonlinear successors.

Matrix calculus you will meet

Plain version. Training means following gradients, and most gradients in ML are a handful of matrix formulas used over and over.

For a scalar function $f(x)$ of a vector $x \in \mathbb{R}^n$, the **gradient** $\nabla_x f \in \mathbb{R}^n$ collects the partial derivatives $\partial f / \partial x_i$. For a vector function $f : \mathbb{R}^n \to \mathbb{R}^m$, the **Jacobian** $J \in \mathbb{R}^{m \times n}$ has entries $J_{ij} = \partial f_i / \partial x_j$. The rules you will use most are:

Function	Gradient	Note
$a^\top x$	a	linear function
$x^\top A x$	$(A + A^\top)x$	becomes $2Ax$ when A is symmetric
$\frac{1}{2}\lVert Xw - y \rVert_2^2$	$X^\top(Xw - y)$	least-squares loss
$\lVert x \rVert_2^2$	$2x$	weight decay
$\operatorname{Tr}(AB)$ w.r.t. A	$B^\top$	used with the trace tricks above

Each can be verified by writing the expression as a sum over indices and differentiating term by term [2]. The **chain rule** for vector functions multiplies Jacobians: if $y = g(x)$ and $L = f(y)$ is a scalar, then $\nabla_x L = J_g^\top \nabla_y L$. Backpropagation applies this rule layer by layer, multiplying transposed local Jacobians by the incoming gradient without ever building the full Jacobians, which would be enormous. ML 02 builds this up in detail, and ML 01C adds the Hessian, the matrix of second derivatives.

Setting the least-squares gradient to zero recovers the normal equations from the previous section. The Hessian of that loss is $X^\top X$, which is PSD, so the loss is convex and every stationary point is a global minimum. A quick finite-difference check confirms the formula:

```
import numpy as np

rng = np.random.default_rng(3)
X, y = rng.normal(size=(30, 4)), rng.normal(size=30)
w = rng.normal(size=4)
L = lambda w: 0.5 * np.sum((X @ w - y)**2)
grad = X.T @ (X @ w - y)                    # analytic gradient
eps = 1e-6
num = np.array([(L(w + eps*e) - L(w - eps*e)) / (2*eps) for e in np.eye(4)])
print("max gradient mismatch:", np.abs(grad - num).max())
# Setting the gradient to zero gives the normal equations
w_star = np.linalg.lstsq(X, y, rcond=None)[0]
print("gradient at solution ~ 0:", np.abs(X.T @ (X @ w_star - y)).max() < 1e-10)
```

The analytic and numerical gradients agree to about 10^{-8}, the size expected from finite-difference rounding.

Modern view

The definitions in this chapter are a century old, but the way they are used in machine learning keeps changing. Four lines of work are worth knowing about.

Randomized numerical linear algebra. Exact SVDs of large matrices are expensive, roughly $O(mn \min(m, n))$ operations. Halko, Martinsson and Tropp's SIAM Review survey [12] made a simple idea standard: multiply A by a thin random matrix Ω to "sketch" its range, orthonormalize the result to get a basis Q, and compute an exact SVD of the small matrix $Q^\top A$. With a little oversampling and a few power iterations, the result is accurate for matrices whose spectrum decays. Martinsson and Tropp's *Acta Numerica* survey [13] covers the theory and a wide range of algorithms: norm estimation, random embeddings, regression, low-rank approximation, Nyström approximation of PSD matrices, streaming algorithms and kernel matrices from machine learning. Murray et al. [14] (LAPACK Working Note 299, 2023) take the software view. They note that RandNLA methods have entered MATLAB, the NAG Library, NVIDIA's cuSOLVER and scikit-learn, and they argue for standard "RandBLAS" and "RandLAPACK" libraries. scikit-learn's randomized PCA solver is a direct descendant. The core algorithm fits in a few lines:

```
import numpy as np

def randomized_svd(A, k, oversample=10, power_iters=2, seed=0):
    rng = np.random.default_rng(seed)
    Omega = rng.normal(size=(A.shape[1], k + oversample))  # random test matrix
    Y = A @ Omega                                          # sample the range of A
    for _ in range(power_iters):                           # sharpen a slow spectrum
```

```
        Y = A @ (A.T @ Y)
    Q, _ = np.linalg.qr(Y)                                  # orthonormal basis
    B = Q.T @ A                                             # small (k+p) x n matrix
    Ub, s, Vt = np.linalg.svd(B, full_matrices=False)
    return (Q @ Ub)[:, :k], s[:k], Vt[:k]

rng = np.random.default_rng(1)
m, n, r = 2000, 500, 20
A = rng.normal(size=(m, r)) @ rng.normal(size=(r, n)) + 0.01 * rng.normal(size=(m, n))
U, s, Vt = randomized_svd(A, k=r)
s_exact = np.linalg.svd(A, compute_uv=False)[:r]
print("max relative error in top singular values:", np.max(np.abs(s - s_exact) / s_exact))
rel = np.linalg.norm(A - (U * s) @ Vt) / np.linalg.norm(A)
print("relative reconstruction error:", round(rel, 5))
```

On this nearly rank-20 matrix the top 20 singular values match the exact ones to about 10^{-15} relative error, and the rank-20 reconstruction error (about 0.2%) is set by the added noise. The power iterations are what make it work when the spectrum decays slowly. For a matrix with a flat spectrum, no rank-k method can do well, randomized or not.

Low-rank structure in large models. The outer-product view of matrix products has become a practical tool for adapting large networks. LoRA [15] freezes a pretrained weight $W_0 \in \mathbb{R}^{d\times k}$ and learns an update $\Delta W = BA$ with $B \in \mathbb{R}^{d\times r}$, $A \in \mathbb{R}^{r\times k}$ and $r \ll \min(d,k)$. Only $r(d+k)$ numbers are trained instead of dk; for $d = k = 4096$ and $r = 8$, that is 65,536 instead of about 16.8 million per matrix. B starts at zero, so training begins exactly at the pretrained model. GaLore [16] (ICML 2024) keeps all weights trainable but projects the *gradients* onto a low-rank subspace, which shrinks the optimizer's memory. Its authors argue that restricting the weight update itself to a low-rank subspace, as LoRA does, can underperform full-rank training. Both rely on the Eckart–Young intuition that much of a matrix's useful content lives in a few singular directions. Whether that holds for a given task is an empirical question, not a theorem.

Controlling the spectral norm. A network's sensitivity to input perturbations is bounded by the product of its layers' spectral norms (for 1-Lipschitz activations such as ReLU). Spectral normalization [17] divides each weight matrix by an estimate of its largest singular value, obtained cheaply with one step of power iteration per training step. It was introduced to stabilize the discriminator in GANs (ML 06), and the same idea appears wherever a Lipschitz bound is wanted.

Beyond matrices. Convolution kernels and video features are naturally tensors with three or more axes. Kolda and Bader's SIAM Review survey [18] is the standard introduction to tensor decompositions. It covers the CP decomposition (a sum of rank-one tensors, generalizing the outer-product view) and the Tucker decomposition (a small core tensor multiplied by a matrix along each mode, generalizing the SVD). Unlike matrices, computing the rank of a tensor is hard in general, and best low-rank approximations need not exist. The clean Eckart–Young picture does not carry over. These decompositions are used to compress layers and to analyze multi-way data.

Open questions. Why do trained weight matrices often have rapidly decaying spectra, and how does that relate to generalization? How low can the rank of an adaptation be before quality drops, and does that depend on the task or the model? How should randomized algorithms be combined

with low-precision hardware, where rounding errors are much larger than in float64? None of these have settled answers. The tools in this chapter are what you need to read the papers that study them. For broader textbook treatments that connect linear algebra to deep learning, see [1], [2] and [19].

Key takeaways

- Name every axis. Most silent bugs are shape or broadcasting errors, and `assert` statements catch them.
- A matrix product has four readings (entries, columns, rows, sums of outer products); choose the one that makes the problem clear.
- Rank and null space decide whether $Ax = b$ has zero, one or infinitely many solutions. Never form A^{-1} in code; use a solver.
- Symmetric matrices have real eigenvalues and orthonormal eigenvectors; the signs of the eigenvalues say whether a quadratic form is a bowl, a saddle or a dome.
- The SVD exists for every matrix: rotate, stretch, rotate. Truncating it gives the best low-rank approximation, with error equal to the discarded singular values.
- The pseudo-inverse gives least-squares and minimum-norm solutions. Small singular values amplify noise; the condition number measures how much, and ridge regularization tames it.
- Trace and determinant summarize eigenvalues (sum and product). Use the cyclic trace property in derivations and log-determinants in code.
- PCA is a projection onto the top eigenvectors of the covariance, equivalently the top right singular vectors of the centered data. Minimizing reconstruction error and maximizing variance give the same answer.

Exercises

1. **Shapes and bias.** A batch has shape 32×64 and a weight matrix has shape 64×10. What shape should the bias have, and what goes wrong if you use a bias of shape $(32, 1)$ instead?

   Hint

   A bias of shape $(10,)$ broadcasts over the 32 rows: one offset per output unit, shared by every example. A $(32, 1)$ bias broadcasts across the 10 columns, giving one offset per *example*. The code runs, but the "bias" now depends on the position of an example in the batch, which is meaningless and changes with batch size.

2. **Projection.** Let $Q \in \mathbb{R}^{n \times k}$ have orthonormal columns and $P = QQ^\top$. Show that $P^2 = P$, $P^\top = P$, and that $x - Px$ is orthogonal to every column of Q. What are the eigenvalues of P?

   Hint

   $P^2 = Q(Q^\top Q)Q^\top = QIQ^\top = P$. For orthogonality compute $Q^\top(x - Px) = Q^\top x - Q^\top QQ^\top x = 0$. From $P^2 = P$, any eigenvalue satisfies $\lambda^2 = \lambda$, so $\lambda \in \{0, 1\}$: eigenvalue 1 with multiplicity k (the subspace) and 0 with multiplicity $n - k$ (its orthogonal complement). Hence $\mathrm{Tr}(P) = k$.

3. **SVD by hand.** Find the singular values of $A = \begin{bmatrix} 3 & 0 \\ 4 & 5 \end{bmatrix}$ by computing the eigenvalues of $A^\top A$. Check that $\|A\|_F^2 = \sigma_1^2 + \sigma_2^2$ and $|\det A| = \sigma_1 \sigma_2$.

Hint

$A^\top A = \begin{bmatrix} 25 & 20 \\ 20 & 25 \end{bmatrix}$ has eigenvalues 45 and 5, so $\sigma_1 = 3\sqrt{5}$, $\sigma_2 = \sqrt{5}$. Then $\sigma_1^2 + \sigma_2^2 = 50 = 9 + 16 + 25$, and $\sigma_1 \sigma_2 = 15 = |\det A| = |3 \cdot 5 - 0 \cdot 4|$. In general $|\det A| = \prod_i \sigma_i$ for a square matrix, because $|\det U| = |\det V| = 1$.

4. **Conditioning and ridge.** Starting from the least-squares code, replace the second feature by an exact copy of the first. What does `np.linalg.lstsq` return, and why is it the minimum-norm solution? Then derive the SVD form of the ridge solution and show that each factor $\sigma/(\sigma^2 + \lambda)$ is at most $1/(2\sqrt{\lambda})$.

Hint

With identical columns, the rank is 1 and the solution set is a line $w_1 + w_2 = c$. `lstsq` uses the SVD and returns the pseudo-inverse solution, which splits the weight equally, $w_1 = w_2 = c/2$, because that point on the line is closest to the origin. For the bound, maximize $g(\sigma) = \sigma/(\sigma^2 + \lambda)$: $g'(\sigma) = 0$ at $\sigma = \sqrt{\lambda}$, giving $g = 1/(2\sqrt{\lambda})$.

5. **PCA error.** Using the derivation in this chapter, show that the total reconstruction error of l-component PCA equals $\sum_{i>l} \lambda_i(X^\top X)$, and that the fraction of variance retained is $\sum_{i \le l} \lambda_i / \sum_i \lambda_i$. Then run PCA on `sklearn.datasets.load_digits()` and find the smallest l that keeps 90% of the variance.

Hint

Write $\|X - XDD^\top\|_F^2 = \mathrm{Tr}(X^\top X) - \mathrm{Tr}(D^\top X^\top X D)$ (use $D^\top D = I$ and the cyclic property). With D the top-l eigenvectors, the second trace is $\sum_{i \le l} \lambda_i$, and $\mathrm{Tr}(X^\top X) = \sum_i \lambda_i$. For the digits, center the 1797×64 matrix, compute its singular values, and take the cumulative sum of σ_i^2. Report your own number rather than trusting a quoted one.

6. **Determinants and volume.** A 2×2 matrix maps the unit square to a parallelogram with vertices 0, $(2,1)$, $(1,3)$ and $(3,4)$. Write down a matrix that does this, compute its determinant, and explain why a normalizing flow must track $\log|\det J|$ for every layer.

Hint

Put the edge vectors in the columns: $A = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}$, with $\det A = 5$, so areas grow by a factor of 5. If a map stretches volume by 5, probability density must shrink by the same factor to keep total probability equal to 1. Stacking layers multiplies these factors, so their logarithms add, which is why flows sum log-determinants layer by layer.

References

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016, ch. 2 “Linear Algebra”. Book
2. M. P. Deisenroth, A. A. Faisal and C. S. Ong, *Mathematics for Machine Learning*, Cambridge University Press, 2020. Book
3. G. Strang, *Introduction to Linear Algebra*, 6th ed., Wellesley-Cambridge Press, 2023. Book site
4. G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed., Johns Hopkins University Press, 2013. DOI
5. L. N. Trefethen and D. Bau III, *Numerical Linear Algebra*, SIAM, 1997. DOI
6. C. Eckart and G. Young, “The Approximation of One Matrix by Another of Lower Rank,” *Psychometrika*, vol. 1, pp. 211–218, 1936. DOI
7. R. Penrose, “A Generalized Inverse for Matrices,” *Mathematical Proceedings of the Cambridge Philosophical Society*, vol. 51, pp. 406–413, 1955. DOI
8. A. E. Hoerl and R. W. Kennard, “Ridge Regression: Biased Estimation for Nonorthogonal Problems,” *Technometrics*, vol. 12, pp. 55–67, 1970. DOI
9. K. Pearson, “On Lines and Planes of Closest Fit to Systems of Points in Space,” *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, vol. 2, pp. 559–572, 1901. DOI
10. H. Hotelling, “Analysis of a Complex of Statistical Variables into Principal Components,” *Journal of Educational Psychology*, vol. 24, pp. 417–441, 1933. DOI
11. I. T. Jolliffe and J. Cadima, “Principal Component Analysis: A Review and Recent Developments,” *Philosophical Transactions of the Royal Society A*, vol. 374, 20150202, 2016. DOI
12. N. Halko, P.-G. Martinsson and J. A. Tropp, “Finding Structure with Randomness: Probabilistic Algorithms for Constructing Approximate Matrix Decompositions,” *SIAM Review*, vol. 53, no. 2, pp. 217–288, 2011. DOI ·arXiv
13. P.-G. Martinsson and J. A. Tropp, “Randomized Numerical Linear Algebra: Foundations and Algorithms,” *Acta Numerica*, vol. 29, pp. 403–572, 2020. DOI ·arXiv
14. R. Murray, J. Demmel, M. W. Mahoney, N. B. Erichson, M. Melnichenko, O. A. Malik, L. Grigori, P. Luszczek, M. Dereziński, M. E. Lopes, T. Liang, H. Luo and J. Dongarra, “Randomized Numerical Linear Algebra: A Perspective on the Field With an Eye to Software,” LAPACK Working Note 299, 2023. arXiv
15. E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang and W. Chen, “LoRA: Low-Rank Adaptation of Large Language Models,” in *Proc. International Conference on Learning Representations* (*ICLR*), 2022. arXiv
16. J. Zhao, Z. Zhang, B. Chen, Z. Wang, A. Anandkumar and Y. Tian, “GaLore: Memory-Efficient LLM Training by Gradient Low-Rank Projection,” *ICML*, 2024. arXiv
17. T. Miyato, T. Kataoka, M. Koyama and Y. Yoshida, “Spectral Normalization for Generative Adversarial Networks,” *ICLR*, 2018. arXiv
18. T. G. Kolda and B. W. Bader, “Tensor Decompositions and Applications,” *SIAM Review*, vol. 51, no. 3, pp. 455–500, 2009. DOI
19. C. M. Bishop and H. Bishop, *Deep Learning: Foundations and Concepts*, Springer, 2024. Book

ML 01B • Probability and Information Theory

Random variables, common distributions, Bayes' rule, expectation and covariance, likelihood, entropy, cross-entropy, KL divergence and structured models, connected to how neural networks are trained and evaluated.

Online edition: https://blog.shyandram.dev/en/learn/ml/ml-01b-probability-information/

What you'll learn

- Why machine learning needs probability, and the two common ways of reading a probability (frequency and degree of belief).
- How random variables, probability mass functions and densities work, and how marginal, conditional and joint probabilities relate through the sum rule, the product rule and the chain rule.
- How expectation, variance, covariance and correlation summarize a distribution, and why zero covariance does not mean independence.
- The distributions you will meet constantly in deep learning — Bernoulli, categorical, Gaussian, exponential, Laplace, Dirac, empirical and mixtures — plus the sigmoid and softplus functions that parameterize them.
- How Bayes' rule, the change-of-variables formula, maximum likelihood and MAP estimation work, with tested NumPy code.
- What entropy, cross-entropy, KL divergence and mutual information measure, why the classification loss is a cross-entropy, and how structured (graphical) models factor a large joint distribution.

The big picture

A neural network never sees the whole world. It sees a finite training set, measured by imperfect sensors, labelled by busy people, and then it is asked about images it has never seen. Every step of that story involves **uncertainty**. Probability theory is the mathematics of uncertainty: it tells us how to describe what we do not know, how to combine evidence, and how to make predictions that admit doubt. Information theory, its close relative, measures *how much* uncertainty there is and how costly it is to use the wrong distribution. Together they explain almost every loss function in this site's ML and DL tracks.

> **Explain like I'm five**
>
> Imagine a bag of marbles you cannot see into. **Probability** is your best guess about what you will pull out next: "probably red, maybe blue, almost never green." Every time you pull a marble, you learn a little more about the bag and update your guess — that updating is **Bayes' rule**. **Entropy** is how surprised you expect to be: a bag of all-red marbles never surprises you; a bag with ten colours in

equal amounts surprises you a lot. A machine learning model is a very fancy guesser, and training means making its guesses less surprised by the real marbles.

This chapter follows the "Applied Math" part of *Deep Learning* by Goodfellow, Bengio and Courville [1], with additional material from Bishop and Bishop [2], Murphy [3] and *Mathematics for Machine Learning* [4]. It sits between ML 01A ·Linear Algebra and ML 01C ·Numerical Computation. If you are heading to the Pattern Recognition track, PR 01 ·Bayes Decisions builds directly on the Bayes-rule section here.

Why probability?

Plain version. Computer science usually deals with certain facts: a bit is 0 or 1, a program either halts or does not. Machine learning cannot work that way because the data, the world and the model are all imperfect.

Goodfellow et al. [1] list three sources of uncertainty:

1. **Inherent stochasticity.** Some systems are genuinely random — photon arrival at a camera sensor (shot noise), the roll of a die, or the exact moment a pedestrian steps off a curb.
2. **Incomplete observability.** Even a deterministic system looks random if we cannot see all of its variables. A single photo of a car does not show whether its brake lights are about to turn on.
3. **Incomplete modelling.** A model must discard information to be useful. A classifier that sees 32×32 thumbnails cannot recover details that were lost when the image was downsampled, so its predictions about fine-grained classes must remain uncertain.

It is often more practical to use a simple rule with uncertainty ("most birds fly") than a complex rule that is always true ("birds fly, except penguins, ostriches, injured birds, ...").

Two readings of a probability

A probability can be read in two ways, and both obey the same mathematical rules.

- **Frequentist probability** describes long-run frequencies of repeatable events: "if we toss this coin many times, about 70% of tosses land heads".
- **Bayesian probability** describes a *degree of belief* about something that might not be repeatable: "given this X-ray, I am 40% confident the patient has the disease".

If degrees of belief are required to satisfy a few common-sense consistency properties, they turn out to obey exactly the same rules as frequencies, which is why one calculus serves both readings [1], [3]. In practice, deep learning mixes the two: the data are treated as samples from a distribution (frequentist), while the model's output is often read as a belief about the label (Bayesian).

Random variables and probability distributions

Plain version. A random variable is a quantity whose value is not known in advance. A probability distribution says how likely each value is.

We write random variables in roman letters (for example x in [1]); in this tutorial we use X for the variable and x for one of its values. A random variable can be **discrete** (a finite or countable set of values, such as a class label in $\{1, \dots, K\}$) or **continuous** (real values, such as a pixel intensity treated as a real number).

Probability mass functions

A discrete variable has a **probability mass function** (PMF) $P(X = x)$, often shortened to $P(x)$. It must satisfy

$$0 \le P(x) \le 1 \text{ for every } x, \qquad \sum_{x} P(x) = 1.$$

Here the sum runs over every possible value x of X. The first condition says no event is more than certain; the second says that *something* always happens. A joint PMF $P(X = x, Y = y)$ gives the probability that both events happen together.

Probability density functions

A continuous variable has a **probability density function** (PDF) $p(x)$, which must satisfy

$$p(x) \ge 0, \qquad \int p(x)\, dx = 1.$$

A density is *not* a probability. The probability that X falls in an interval $[a, b]$ is the area under the density:

$$P(a \le X \le b) = \int_a^b p(x)\, dx.$$

The probability of any single exact value is zero. Because only areas must be at most one, a density can exceed one: a narrow Gaussian with standard deviation 0.25 peaks at about 1.6 (Figure 1, top right). A common mistake is to report "probability 1.6" for a density value.

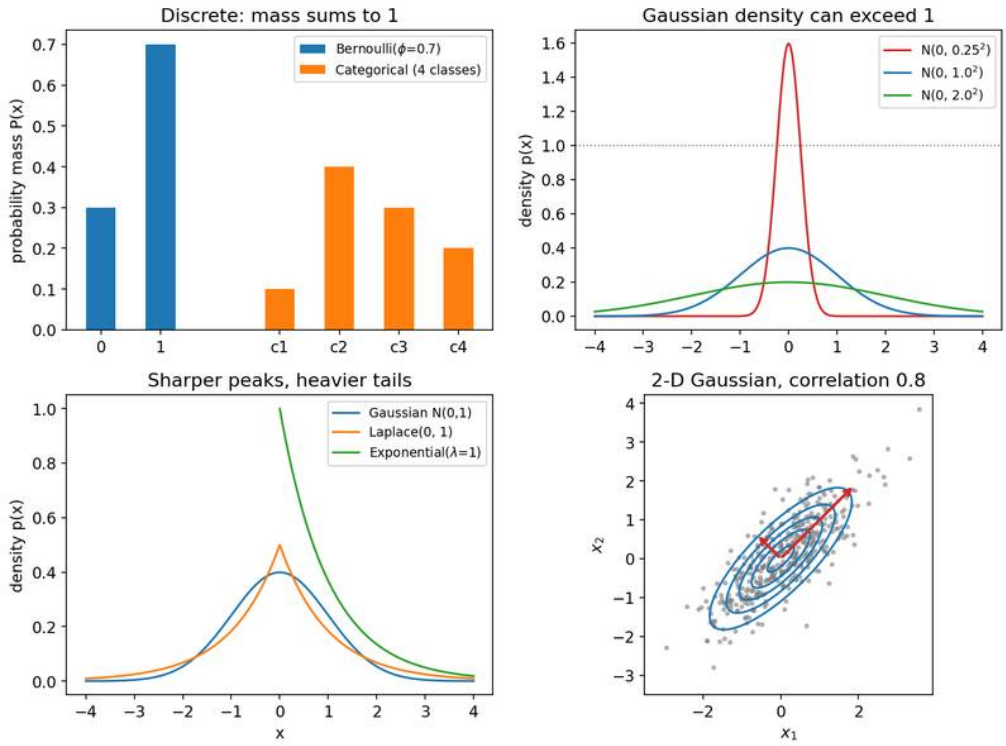


Figure 1 — Common distributions. Top left: discrete masses sum to one. Top right: a density can exceed one because only its area must equal one. Bottom left: the Laplace density has a sharp peak and heavier tails than the Gaussian; the exponential lives on $x \geq 0$. Bottom right: a 2-D Gaussian with correlation 0.8; red arrows show the eigenvectors of its covariance, scaled by two standard deviations.

Marginal probability

Plain version. If you know how two things vary together, you can recover how one of them varies on its own by "summing out" the other.

Given a joint distribution over X and Y, the **marginal** distribution of X is

$$P(x) = \sum_{y} P(x, y) \quad \text{(discrete)}, \qquad p(x) = \int p(x, y)\, dy \quad \text{(continuous)}.$$

This is the **sum rule**. If the joint is written as a table with rows for x and columns for y, the marginal of x is the row sums — historically written in the margin of the page, hence the name.

Conditional probability

Plain version. A conditional probability answers "how likely is this, *given that* I already know that?"

$$P(y \mid x) = \frac{P(x, y)}{P(x)}, \qquad P(x) > 0.$$

$P(y \mid x)$ is the probability of $Y = y$ after we have observed $X = x$. It is only defined when the condition can actually happen. Rearranging gives the **product rule** $P(x, y) = P(y \mid x)P(x)$.

Do not confuse conditioning with intervention. "The probability that the street is wet given that people carry umbrellas" is high, but *forcing* people to carry umbrellas will not wet the street. Questions about interventions belong to causal inference, which needs more than a joint distribution.

The chain rule of probability

Applying the product rule repeatedly factorizes any joint distribution over n variables:

$$P(x^{(1)}, \dots, x^{(n)}) = P(x^{(1)}) \prod_{i=2}^{n} P(x^{(i)} \mid x^{(1)}, \dots, x^{(i-1)}).$$

Here $x^{(i)}$ is the value of the i-th variable. This identity is *exact* — no assumption is involved. It is the backbone of autoregressive models: a language model or an autoregressive image model predicts one token or pixel at a time given all previous ones (ML 08, NLP 01). The modelling choice lies in *how* each conditional is parameterized, not in the factorization itself.

Independence and conditional independence

Two variables are **independent**, written $X \perp Y$, if

$$p(x, y) = p(x)\, p(y) \quad \text{for all } x, y.$$

They are **conditionally independent** given Z, written $X \perp Y \mid Z$, if

$$p(x, y \mid z) = p(x \mid z)\, p(y \mid z) \quad \text{for all } x, y, z.$$

Ice-cream sales and sunburn are dependent, but roughly independent once you condition on the weather. Conditional independence is the main tool for making high-dimensional models tractable — naive Bayes assumes the features are independent given the class; a hidden Markov model assumes the future is independent of the past given the present state. These are *assumptions* [3]: they should be justified by the problem, not adopted only because they make the algebra easy.

Code: a joint table, marginals and conditionals

The following example builds a small joint table for weather and umbrella use, computes marginals and conditionals, and tests independence. It also shows a case where two variables are clearly dependent yet have (almost) zero covariance.

```
import numpy as np
```

```
# Joint table P(weather, umbrella): rows = sunny/rainy, columns = no/yes
P = np.array([[0.56, 0.14],
              [0.06, 0.24]])
assert np.isclose(P.sum(), 1)

p_weather = P.sum(axis=1)            # marginal over umbrella
p_umbrella = P.sum(axis=0)           # marginal over weather
p_umb_given_w = P / p_weather[:, None]   # each row sums to 1
print("P(weather):", p_weather)
print("P(umbrella):", p_umbrella)
print("P(umbrella | weather):\n", p_umb_given_w)
print("independent?", np.allclose(P, np.outer(p_weather, p_umbrella)))

# Zero covariance does not imply independence
rng = np.random.default_rng(0)
x = rng.uniform(-1, 1, 200_000)
y = x**2                              # y is a deterministic function of x
print("cov(x, y) ~", np.cov(x, y)[0, 1].round(4))
```

Output: the marginals are $[0.7, 0.3]$ and $[0.62, 0.38]$; the conditional table is $[[0.8, 0.2], [0.2, 0.8]]$; the variables are not independent; and the sample covariance of x and x^2 is about -0.0007, essentially zero, even though y is completely determined by x.

Expectation, variance and covariance

Plain version. The expectation is the long-run average; the variance is how far values typically spread from that average; the covariance says whether two quantities tend to go up and down together.

The **expectation** of a function f under a distribution is

$$\mathbb{E}_{x\sim P}[f(x)] = \sum_{x} P(x) f(x), \qquad \mathbb{E}_{x\sim p}[f(x)] = \int p(x) f(x)\, dx.$$

The subscript $x \sim P$ names the distribution we average over; we drop it when it is obvious. Expectation is **linear**: $\mathbb{E}[\alpha f(x) + \beta g(x)] = \alpha \mathbb{E}[f(x)] + \beta \mathbb{E}[g(x)]$ for constants α, β. Almost every training loss is an expectation over the data distribution, estimated by an average over a mini-batch.

The **variance** measures spread around the mean $\mu = \mathbb{E}[f(x)]$:

$$\operatorname{Var}(f(x)) = \mathbb{E}\big[(f(x) - \mu)^2\big] = \mathbb{E}[f(x)^2] - \mu^2.$$

Its square root is the **standard deviation**, which has the same units as $f(x)$.

The **covariance** of two quantities measures how they vary together:

$$\operatorname{Cov}(f(x), g(y)) = \mathbb{E}\big[(f(x) - \mathbb{E}[f(x)])\,(g(y) - \mathbb{E}[g(y)])\big].$$

A positive covariance means both tend to be above (or below) their means at the same time; a negative covariance means one tends to be high when the other is low. Its magnitude depends on the units, so we often normalize it into the **correlation**

$$\rho = \frac{\mathrm{Cov}(X, Y)}{\sqrt{\mathrm{Var}(X)\,\mathrm{Var}(Y)}} \in [-1, 1].$$

Independence implies zero covariance, but **zero covariance does not imply independence**. Covariance measures only *linear* dependence. In the code above, $y = x^2$ with x uniform on $[-1, 1]$: knowing x tells you y exactly, but the covariance is zero because the relationship is symmetric and nonlinear. Mutual information, introduced later, captures every kind of dependence.

The covariance matrix

For a random vector $\mathbf{x} \in \mathbb{R}^n$ with mean $\boldsymbol{\mu} = \mathbb{E}[\mathbf{x}]$, the **covariance matrix** is

$$\Sigma = \mathrm{Cov}(\mathbf{x}) = \mathbb{E}[(\mathbf{x} - \boldsymbol{\mu})(\mathbf{x} - \boldsymbol{\mu})^\top], \qquad \Sigma_{ij} = \mathrm{Cov}(x_i, x_j).$$

Its diagonal holds the variances $\Sigma_{ii} = \mathrm{Var}(x_i)$. Σ is symmetric and positive semidefinite, so (by ML 01A) it has real nonnegative eigenvalues and orthogonal eigenvectors. The eigenvectors are the principal axes of the data cloud and the eigenvalues are the variances along them — the red arrows in Figure 1 — which is exactly what PCA computes. For an affine map $\mathbf{y} = A\mathbf{x} + \mathbf{b}$, $\mathbb{E}[\mathbf{y}] = A\boldsymbol{\mu} + \mathbf{b}$ and $\mathrm{Cov}(\mathbf{y}) = A\Sigma A^\top$.

Common probability distributions

Plain version. A handful of distributions appear over and over. Choosing one is choosing an assumption about your data or your target.

Bernoulli

A single binary variable $X \in \{0, 1\}$ with parameter $\phi \in [0, 1]$:

$$P(X = 1) = \phi, \quad P(X = 0) = 1 - \phi, \quad P(X = x) = \phi^x (1 - \phi)^{1-x}.$$

Its mean is ϕ and its variance $\phi(1 - \phi)$. A binary classifier with a sigmoid output predicts the parameter ϕ of a Bernoulli distribution over the label.

Categorical (multinoulli)

A single discrete variable with K states, parameterized by a probability vector $\mathbf{p} \in [0, 1]^K$ with $\sum_k p_k = 1$:

$$P(X = k) = p_k.$$

If the label is encoded one-hot as $\mathbf{y} \in \{0, 1\}^K$, then $P(\mathbf{y}) = \prod_k p_k^{y_k}$. A softmax classifier outputs exactly such a vector. (The *multinomial* distribution counts outcomes over several draws; for one draw it reduces to the categorical, which Goodfellow et al. call "multinoulli" [1].)

Gaussian (normal)

The most widely used continuous distribution:

$$\mathcal{N}(x;\mu,\sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}}\exp\Big(-\frac{(x-\mu)^2}{2\sigma^2}\Big).$$

μ is the mean (and the peak), σ^2 the variance. Sometimes we use the **precision** $\beta = 1/\sigma^2$ instead, which is cheaper to evaluate when the variance changes often.

Two reasons make the Gaussian the default. The **central limit theorem** says that the sum of many independent variables with finite variance tends towards a Gaussian, so measurement noise built from many small effects is often near-Gaussian. And among all distributions with a given mean and variance on the real line, the Gaussian has the **maximum entropy**: it builds in the least extra structure.

In n dimensions,

$$\mathcal{N}(\mathbf{x};\boldsymbol{\mu},\Sigma) = \frac{1}{\sqrt{(2\pi)^n \det\Sigma}}\exp\Big(-\tfrac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^\top\Sigma^{-1}(\mathbf{x}-\boldsymbol{\mu})\Big),$$

where $\boldsymbol{\mu} \in \mathbb{R}^n$ is the mean vector and $\Sigma \in \mathbb{R}^{n\times n}$ is a symmetric positive-definite covariance matrix. The **precision matrix** $\Lambda = \Sigma^{-1}$ has a neat meaning: $\Lambda_{ij} = 0$ exactly when x_i and x_j are conditionally independent given all other coordinates. An **isotropic** Gaussian has $\Sigma = \sigma^2 I$ — the noise distribution used by diffusion models (ML 07).

A fundamental link to the losses you will use: if a regression network predicts the mean of a Gaussian with fixed variance, maximizing the likelihood is the same as minimizing the **mean squared error**.

Exponential and Laplace

Sometimes we want a sharp point at a specific location.

$$p(x;\lambda) = \lambda\,\mathbf{1}_{x\ge 0}\,e^{-\lambda x}, \qquad \mathrm{Laplace}(x;\mu,\gamma) = \frac{1}{2\gamma}\exp\Big(-\frac{|x-\mu|}{\gamma}\Big).$$

$\lambda > 0$ is the rate, $\mathbf{1}_{x\ge 0}$ is the indicator that is 1 when $x \ge 0$ and 0 otherwise, μ is the location and $\gamma > 0$ the scale. Compared with the Gaussian, the Laplace density has a sharper peak and heavier tails (Figure 1, bottom left). Predicting the location of a Laplace distribution by maximum likelihood is equivalent to minimizing the **absolute error** (L_1 loss), which is why L_1 is more tolerant of outliers than L_2; a Laplace *prior* on weights gives L_1 regularization (sparsity).

Dirac and empirical distributions

The **Dirac delta** $\delta(x-\mu)$ puts all of its mass at the point μ: it is zero everywhere else yet integrates to one. It is a generalized function, defined by what it does inside integrals: $\int \delta(x-\mu)f(x)\,dx = f(\mu)$.

A training set $\{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ defines the **empirical distribution**

$$\hat{p}(\mathbf{x}) = \frac{1}{m} \sum_{i=1}^{m} \delta(\mathbf{x} - \mathbf{x}^{(i)}),$$

which puts mass $1/m$ on each of the m examples. Training a model by averaging a loss over the dataset is the same as taking an expectation under $\hat{p}$. It is also the distribution that maximizes the likelihood of the training data — which is precisely why minimizing training loss alone overfits.

Mixtures and latent variables

Plain version. A mixture first picks a "type" at random, then draws a sample from the distribution belonging to that type.

$$P(\mathbf{x}) = \sum_{k=1}^{K} P(z = k)\, P(\mathbf{x} \mid z = k).$$

z is a categorical **latent variable** — a variable we never observe directly — chosen with probabilities $\pi_k = P(z = k)$, and $P(\mathbf{x} \mid z = k)$ is the k-th component. A **Gaussian mixture model** (GMM) uses Gaussian components $\mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_k, \Sigma_k)$; with enough components it can approximate any smooth density to any desired nonzero error [1]. Given an observation $\mathbf{x}$, Bayes' rule gives the **posterior responsibility** $P(z = k \mid \mathbf{x})$: how likely it is that component k generated that point. Fitting a GMM with expectation–maximization is the subject of PR 07.

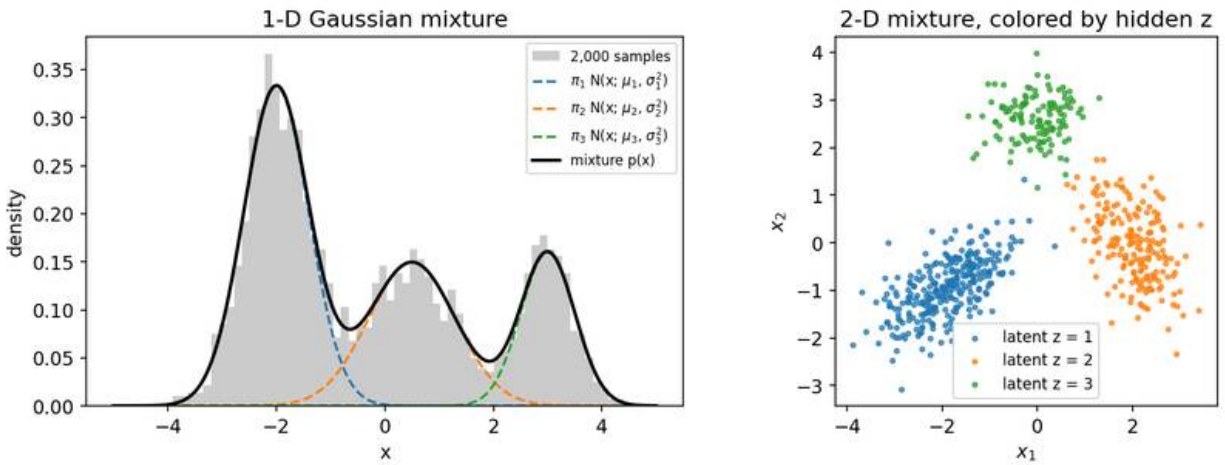


Figure 2 — A Gaussian mixture. Left: the black density is the weighted sum of three components; the grey histogram is 2,000 samples drawn by first picking a component and then sampling from it. Right: in two dimensions, colour shows the hidden component z, which a real dataset would not reveal.

Latent variables are central to modern generative models. A VAE (ML 06) is, informally, a mixture with a *continuous* latent $\mathbf{z}$ and a neural network for $p(\mathbf{x} \mid \mathbf{z})$ [8]; the marginal $p(\mathbf{x}) = \int p(\mathbf{x} \mid \mathbf{z}) p(\mathbf{z})\, d\mathbf{z}$ is then intractable and must be approximated.

```
import numpy as np

rng = np.random.default_rng(0)
pi = np.array([0.5, 0.3, 0.2])      # mixing weights P(z = k)
mu = np.array([-2.0, 0.5, 3.0])     # component means
sd = np.array([0.6, 0.8, 0.5])      # component standard deviations

# Ancestral sampling: first the hidden component z, then x given z
z = rng.choice(3, size=100_000, p=pi)
x = rng.normal(mu[z], sd[z])

def log_normal(x, m, s):
    return -0.5*((x - m)/s)**2 - np.log(s) - 0.5*np.log(2*np.pi)

def mixture_logpdf(x):
    # log sum_k pi_k N(x; mu_k, sd_k^2), computed with log-sum-exp
    a = np.log(pi) + log_normal(x[:, None], mu, sd)
    m = a.max(axis=1, keepdims=True)
    return (m + np.log(np.exp(a - m).sum(axis=1, keepdims=True))).ravel()

grid = np.linspace(-6, 6, 4001)
print("density integrates to",
 ↪  (np.exp(mixture_logpdf(grid)).sum()*(grid[1]-grid[0])).round(4))
print("sample mean", x.mean().round(3), "vs exact", (pi*mu).sum().round(3))
# Posterior responsibility P(z | x) for one observation
a = np.log(pi) + log_normal(np.array([0.0])[:, None], mu, sd)
resp = np.exp(a - a.max()); resp /= resp.sum()
print("P(z | x=0):", resp.round(3))
```

The density integrates to 1.0, the sample mean (-0.255) matches the exact mean $\sum_k \pi_k \mu_k = -0.25$, and the point $x = 0$ is assigned to the middle component with probability 0.99. The log-sum-exp trick used here is explained in ML 01C.

Useful functions: sigmoid and softplus

Plain version. The sigmoid squashes any real number into a probability between 0 and 1; the softplus is a smooth version of $\max(0, x)$ that is always positive.

$$\sigma(x) = \frac{1}{1 + e^{-x}}, \qquad \zeta(x) = \log\left(1 + e^{x}\right).$$

The **logistic sigmoid** σ produces the parameter ϕ of a Bernoulli distribution; it **saturates** (becomes flat) for large $|x|$, which makes gradients small there. The **softplus** ζ produces a positive number, such as a standard deviation σ or a precision β; its name comes from being a softened "positive part" $x^+ = \max(0, x)$.

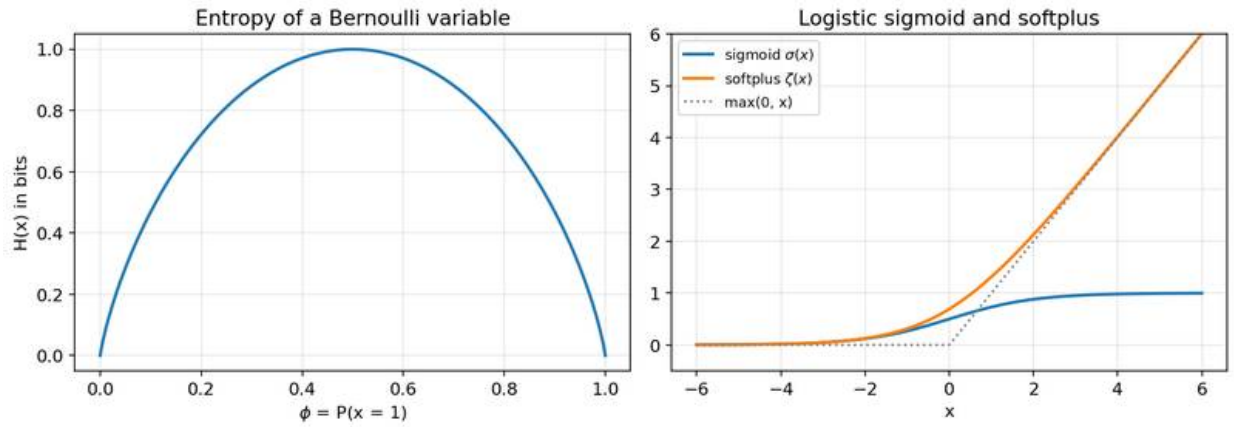


Figure 3 — Left: entropy of a Bernoulli variable, maximal (1 bit) at $\varphi = 0.5$ and zero when the outcome is certain. Right: the logistic sigmoid $\sigma(x)$ and the softplus $\zeta(x)$, a smooth version of $\max(0, x)$.

These identities are worth memorizing, because they appear every time a loss is simplified [1]:

$$1 - \sigma(x) = \sigma(-x), \qquad \frac{d}{dx}\sigma(x) = \sigma(x)(1 - \sigma(x)),$$

$$\log \sigma(x) = -\zeta(-x), \qquad \frac{d}{dx}\zeta(x) = \sigma(x),$$

$$\sigma^{-1}(p) = \log \frac{p}{1-p}, \qquad \zeta(x) - \zeta(-x) = x.$$

σ^{-1} is the **logit** (log-odds) function. The identity $\log \sigma(x) = -\zeta(-x)$ shows why binary cross-entropy should be computed from logits: $-\log \sigma(x) = \zeta(-x)$, and softplus can be evaluated without overflow.

```
import numpy as np

def sigmoid(x):
    return 1/(1 + np.exp(-x))

def softplus(x):
    return np.logaddexp(0, x)          # log(1 + exp(x)) without overflow

x = np.linspace(-30, 30, 13)
assert np.allclose(1 - sigmoid(x), sigmoid(-x))
assert np.allclose(np.log(sigmoid(x)), -softplus(-x))
assert np.allclose(softplus(x) - softplus(-x), x)
h = 1e-6
assert np.allclose((softplus(x+h) - softplus(x-h))/(2*h), sigmoid(x), atol=1e-6)
p = sigmoid(x[4:9])
assert np.allclose(np.log(p/(1-p)), x[4:9])   # logit inverts sigmoid
print("naive log(1+exp(1000)):", np.log1p(np.exp(np.float64(1000))))
print("stable softplus(1000):", softplus(1000.0))
```

All identities pass. The naive expression returns `inf` (with an overflow warning) while `np.loga`

`ddexp` returns 1000.0.

Bayes' rule

Plain version. Bayes' rule turns "how likely is the evidence if the hypothesis is true?" into "how likely is the hypothesis now that I have seen the evidence?" — and it insists that you account for how common the hypothesis was in the first place.

From the product rule written both ways, $P(a, b) = P(a \mid b)P(b) = P(b \mid a)P(a)$, so

$$P(a \mid b) = \frac{P(b \mid a)\, P(a)}{P(b)}, \qquad P(b) = \sum_{a'} P(b \mid a')\, P(a').$$

- $P(a)$ is the **prior**: how plausible a is before seeing the evidence.
- $P(b \mid a)$ is the **likelihood** of the evidence b under hypothesis a.
- $P(a \mid b)$ is the **posterior**: the updated plausibility.
- $P(b)$ is the **evidence** (or marginal likelihood), the normalizer obtained by summing over all hypotheses a'.

A worked example: the base-rate effect

Consider a hypothetical defect detector with **sensitivity** $P(+ \mid \text{defect}) = 0.9$, **false-positive rate** $P(+ \mid \text{no defect}) = 0.05$, and a **prevalence** (base rate) $P(\text{defect}) = 0.01$. The probability that an item flagged as positive really is defective is

$$P(\text{defect} \mid +) = \frac{0.9 \cdot 0.01}{0.9 \cdot 0.01 + 0.05 \cdot 0.99} \approx 0.154.$$

Only about 15% of alarms are real, even though the detector catches 90% of defects. Counting makes this obvious: among 10,000 items, 100 are defective and 90 of them raise an alarm, but 9,900 are fine and 495 of them also raise an alarm (Figure 4, right). The same detector at 10% prevalence gives $0.09/(0.09 + 0.045) = 2/3$. This is a mathematical illustration, not a performance claim about any real system. Its lesson carries over to every deployed classifier: **sensitivity and false-positive rate alone do not determine how trustworthy a positive prediction is; the deployment prevalence matters.** ML 10 returns to this when discussing precision under distribution shift.

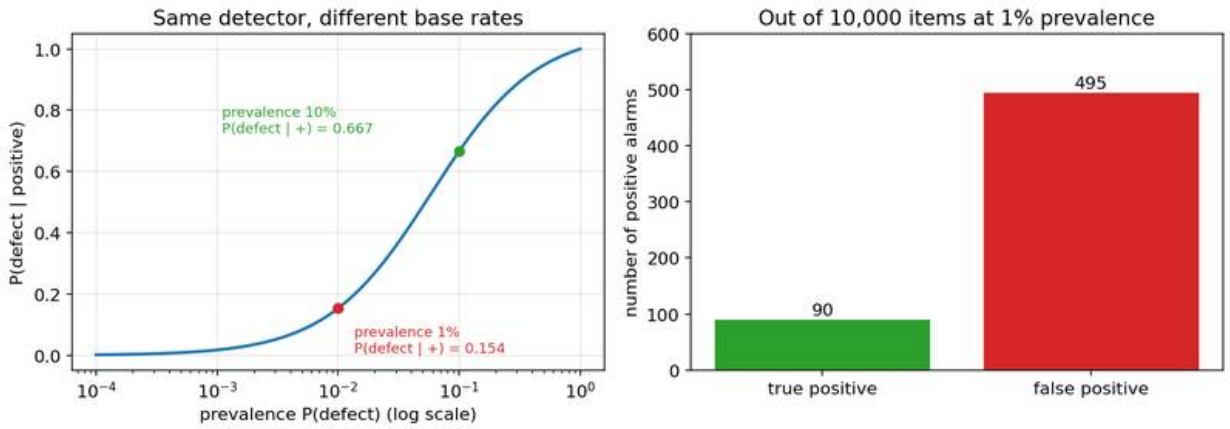


Figure 4 — The base-rate effect for a hypothetical detector with sensitivity 0.9 and false-positive rate 0.05. Left: the posterior $P(\text{defect} \mid +)$ depends strongly on prevalence. Right: at 1% prevalence, false alarms outnumber true alarms more than five to one.

```
import numpy as np

def posterior_positive(prevalence, sensitivity=0.9, false_pos=0.05):
    evidence = sensitivity*prevalence + false_pos*(1 - prevalence)   # P(+)
    return sensitivity*prevalence / evidence                          # P(defect | +)

for prev in [0.001, 0.01, 0.1, 0.5]:
    print(f"prevalence {prev:>5}: P(defect | +) = {posterior_positive(prev):.3f}")

# Simulation check with 1,000,000 synthetic items
rng = np.random.default_rng(0)
defect = rng.random(1_000_000) < 0.01
alarm = np.where(defect, rng.random(defect.size) < 0.9, rng.random(defect.size) < 0.05)
print("simulated:", defect[alarm].mean().round(3))
```

The formula gives 0.018, 0.154, 0.667 and 0.947 for the four prevalences, and a one-million-item simulation reproduces 0.154.

Bayes' rule also underlies the **Bayes classifier**: predicting the class k with the largest posterior $P(y = k \mid \mathbf{x}) \propto p(\mathbf{x} \mid y = k)P(y = k)$ minimizes the expected error rate. PR 01 develops this decision theory in depth.

Change of variables for densities

Plain version. If you stretch a rubber sheet, the ink on it gets thinner. When a random variable is transformed, its density must be rescaled by how much the transformation stretches space.

A tempting mistake: if $y = g(x)$ and x has density p_x, then $p_y(y) = p_x(g^{-1}(y))$. This is wrong, because it does not integrate to one. The correct rule for an invertible, differentiable g in one dimension is

$$p_y(y) = p_x(g^{-1}(y)) \left| \frac{d\, g^{-1}(y)}{dy} \right|.$$

The probability in a small interval must be preserved: $|p_y(y)\,dy| = |p_x(x)\,dx|$. In n dimensions, with $\mathbf{y} = g(\mathbf{x})$,

$$p_x(\mathbf{x}) = p_y(g(\mathbf{x})) \left| \det \frac{\partial g(\mathbf{x})}{\partial \mathbf{x}} \right|,$$

where $\partial g / \partial \mathbf{x}$ is the $n \times n$ **Jacobian matrix** of g (entry (i,j) is $\partial y_i / \partial x_j$) and $|\det(\cdot)|$ is the absolute value of its determinant — the factor by which g scales a small volume (see ML 01A for determinants). This formula is the entire foundation of **normalizing flows**, which stack invertible networks with cheap Jacobian determinants to obtain an exact likelihood [9].

```
import numpy as np

rng = np.random.default_rng(0)
x = rng.normal(0, 1, 500_000)        # x ~ N(0, 1)
y = np.exp(x)                        # y = g(x), invertible on (0, inf)

def p_x(x):
    return np.exp(-0.5*x**2)/np.sqrt(2*np.pi)

def p_y(y):
    # p_y(y) = p_x(g^{-1}(y)) * |d g^{-1}(y) / dy| = p_x(log y) / y
    return p_x(np.log(y))/y

counts, edges = np.histogram(y, bins=np.linspace(0.05, 4, 80), density=False)
centers = 0.5*(edges[:-1] + edges[1:])
empirical = counts/(y.size*np.diff(edges))
print("max |histogram - formula|:", np.abs(empirical - p_y(centers)).max().round(3))
print("wrong formula (no Jacobian) error:", np.abs(empirical -
 ↪  p_x(np.log(centers))).max().round(3))
```

With the Jacobian factor $1/y$ the histogram matches the formula to within 0.011 (sampling noise); without it the error is 0.46.

Likelihood, maximum likelihood and MAP

Plain version. Hold the observed data fixed and ask which parameter values make that data most plausible. If you also have prior knowledge about the parameters, weigh that in too.

Suppose a model $p_{\boldsymbol{\theta}}(\mathbf{x})$ has parameters $\boldsymbol{\theta}$, and we observe a dataset $\mathcal{D} = \{\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(m)}\}$ of independent and identically distributed (i.i.d.) examples. The **likelihood** is

$$L(\boldsymbol{\theta}) = p_{\boldsymbol{\theta}}(\mathcal{D}) = \prod_{i=1}^{m} p_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}).$$

As a function of $\boldsymbol{\theta}$, this is *not* a probability distribution over parameters: it need not integrate to one over $\boldsymbol{\theta}$. The **maximum likelihood estimate** (MLE) is

$$\boldsymbol{\theta}_{\mathrm{ML}} = \arg\max_{\boldsymbol{\theta}} \sum_{i=1}^{m} \log p_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}) = \arg\min_{\boldsymbol{\theta}} \; \mathbb{E}_{\mathbf{x}\sim\hat{p}}[\, -\log p_{\boldsymbol{\theta}}(\mathbf{x})\,].$$

We take logarithms because a product of many numbers below one underflows, while a sum of logs is numerically safe and has the same maximizer. The right-hand form says MLE **minimizes the average negative log-likelihood** (NLL) under the empirical distribution $\hat{p}$. For a classifier $p_{\boldsymbol{\theta}}(y \mid \mathbf{x})$ the same idea gives the conditional NLL $-\log p_{\boldsymbol{\theta}}(y \mid \mathbf{x})$, which is the cross-entropy loss we meet below.

Two classic results: for a Bernoulli coin with k heads in n flips, $\phi_{\mathrm{ML}} = k/n$; for a Gaussian, μ_{ML} is the sample mean and $\sigma^2_{\mathrm{ML}} = \frac{1}{m}\sum_i (x^{(i)} - \mu_{\mathrm{ML}})^2$ — a slightly *biased* estimate (dividing by $m-1$ removes the bias).

MAP estimation

Bayesian inference treats $\boldsymbol{\theta}$ itself as a random variable with a **prior** $p(\boldsymbol{\theta})$ and computes the **posterior**

$$p(\boldsymbol{\theta} \mid \mathcal{D}) = \frac{p(\mathcal{D} \mid \boldsymbol{\theta})\, p(\boldsymbol{\theta})}{p(\mathcal{D})} \;\propto\; p(\mathcal{D} \mid \boldsymbol{\theta})\, p(\boldsymbol{\theta}).$$

The full posterior is often intractable, so a common shortcut is its mode, the **maximum a posteriori** (MAP) estimate:

$$\boldsymbol{\theta}_{\mathrm{MAP}} = \arg\max_{\boldsymbol{\theta}} \, [\, \log p(\mathcal{D} \mid \boldsymbol{\theta}) + \log p(\boldsymbol{\theta})\,].$$

With a Gaussian prior $p(\boldsymbol{\theta}) = \mathcal{N}(\mathbf{0}, \frac{1}{\lambda} I)$, the term $-\log p(\boldsymbol{\theta})$ equals $\frac{\lambda}{2}\|\boldsymbol{\theta}\|_2^2$ plus a constant: **weight decay is MAP with a Gaussian prior**. A Laplace prior gives L_1 regularization. This links probabilistic assumptions to the regularizers in ML 02. But a MAP point estimate is still a single parameter vector; it does not tell you how uncertain the parameters are. Approximating the whole posterior is the topic of PR 08 and of variational inference [10].

```
import numpy as np

rng = np.random.default_rng(0)
flips = rng.random(10) < 0.7          # 10 coin flips, true P(heads) = 0.7
k, n = flips.sum(), flips.size

phi = np.linspace(1e-3, 1 - 1e-3, 999)
log_lik = k*np.log(phi) + (n - k)*np.log(1 - phi)
a, b = 5.0, 5.0                        # Beta(5, 5) prior, centered on 0.5
log_prior = (a - 1)*np.log(phi) + (b - 1)*np.log(1 - phi)

print(f"{k} heads in {n} flips")
print("MLE  grid:", phi[log_lik.argmax()].round(3), " closed form:", k/n)
```

```
print("MAP  grid:", phi[(log_lik + log_prior).argmax()].round(3),
      " closed form:", round((k + a - 1)/(n + a + b - 2), 3))

# Gaussian MLE: sample mean and the 1/m (biased) variance
x = rng.normal(2.0, 3.0, 50)
print("Gaussian MLE mean, var:", x.mean().round(3), x.var().round(3),
      " unbiased var:", x.var(ddof=1).round(3))
```

With this seed, 6 heads appear in 10 flips. The MLE is 0.6; the Beta(5, 5) prior pulls the MAP estimate towards 0.5, giving 0.556, matching the closed form $(k + a - 1)/(n + a + b - 2)$. With more data the likelihood dominates and the two estimates converge. The Gaussian MLE variance (7.71) is smaller than the unbiased estimate (7.87), as expected.

Information theory

Plain version. Information theory measures surprise. A rare event carries a lot of information when it happens; a certain event carries none. Averaging surprise gives entropy, and the extra surprise from using the wrong distribution gives KL divergence.

Claude Shannon founded the field to study how many bits are needed to send messages over a noisy channel [5]; the textbook treatment is Cover and Thomas [7]. For machine learning, the useful idea is to quantify how different two distributions are.

Self-information and entropy

The **self-information** of an outcome x is

$$I(x) = -\log P(x).$$

With the natural logarithm the unit is the **nat**; with $\log_2$ it is the **bit** (one nat $= 1/\ln 2 \approx 1.443$ bits). Certain events ($P = 1$) carry zero information; unlikely events carry a lot; and information from independent events adds, because logs turn products into sums.

The **Shannon entropy** is the expected self-information:

$$H(P) = \mathbb{E}_{x\sim P}[I(x)] = -\sum_x P(x)\log P(x).$$

We use the convention $0 \log 0 = 0$, justified by the limit $p \log p \to 0$ as $p \to 0$. Entropy is the minimum average number of bits (or nats) needed to encode outcomes drawn from P. It is zero for a deterministic outcome and maximal, $\log K$, for a uniform distribution over K outcomes. For a Bernoulli variable with parameter ϕ, $H = -\phi \log \phi - (1 - \phi)\log(1 - \phi)$, which peaks at one bit when $\phi = 0.5$ (Figure 3, left).

For continuous variables, $h(p) = -\int p(x) \log p(x)\, dx$ is the **differential entropy**. Treat it carefully: it can be negative (a narrow uniform distribution on $[0, \frac{1}{2}]$ has $h = -\log 2$) and it changes when you rescale the coordinates. It is not a limiting version of the discrete count of bits.

KL divergence

The **Kullback–Leibler divergence** [6] measures how different a distribution Q is from a reference P:

$$D_{\mathrm{KL}}(P \,\|\, Q) = \mathbb{E}_{x\sim P}\Big[\log \frac{P(x)}{Q(x)}\Big] = \sum_x P(x) \log \frac{P(x)}{Q(x)}.$$

In coding terms, it is the *extra* number of nats needed to encode samples from P when the code was designed for Q. Its key properties:

- **Nonnegative**: $D_{\mathrm{KL}}(P\|Q) \geq 0$, with equality if and only if $P = Q$ (for discrete variables; "almost everywhere" for continuous ones). This follows from Jensen's inequality, because $-\log$ is convex.
- **Infinite under unsupported mass**: if $Q(x) = 0$ for some x with $P(x) > 0$, the divergence is infinite.
- **Asymmetric**: in general $D_{\mathrm{KL}}(P\|Q) \neq D_{\mathrm{KL}}(Q\|P)$, and it does not satisfy the triangle inequality. KL is therefore *not* a distance metric, even though it is often described as "the distance between distributions".

Forward versus reverse KL

The asymmetry has visible consequences when the approximating family is too simple. Figure 5 fits a single Gaussian q to a target p with two well-separated modes.

- Minimizing **forward KL** $D_{\mathrm{KL}}(p\|q)$ heavily penalizes q for being small where p has mass. The best Gaussian therefore **covers both modes**, placing its mean between them ($\mu = 0$, $\sigma \approx 2.62$) and putting a lot of mass in the empty valley. For Gaussian q, the forward-KL optimum simply matches the mean and variance of p.
- Minimizing **reverse KL** $D_{\mathrm{KL}}(q\|p)$ heavily penalizes q for being large where p is small. The best Gaussian therefore **locks onto one mode** ($\mu \approx 2.49$, $\sigma \approx 0.81$) and ignores the other.

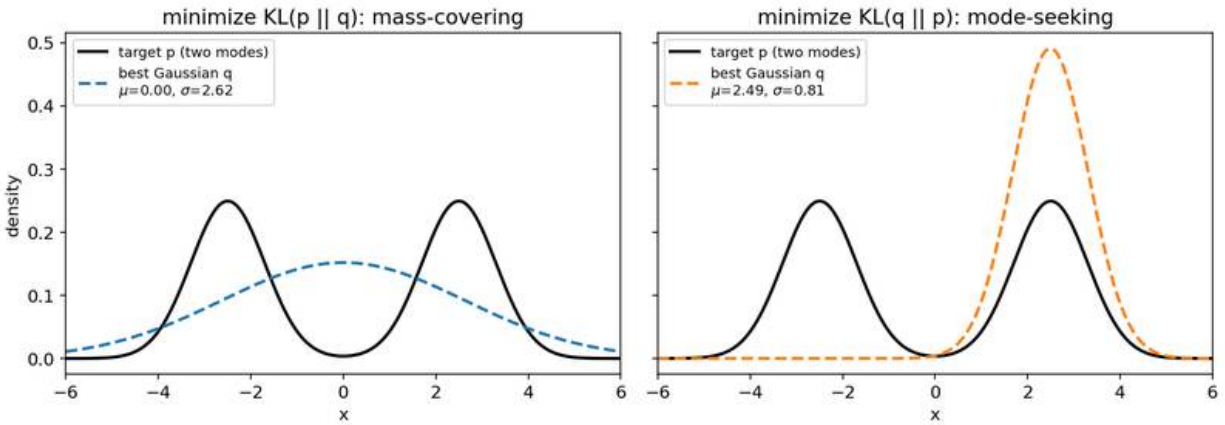


Figure 5 — KL is asymmetric. Fitting one Gaussian q to a two-mode target p: forward $KL(p\|q)$ gives a mass-covering fit centred between the modes; reverse $KL(q\|p)$ gives a mode-seeking fit on one peak (the other peak gives an equally good mirror solution).

Which direction you minimize is a design choice. **Maximum likelihood minimizes forward KL** from the data distribution to the model, so likelihood-trained models (autoregressive models, flows, VAEs) tend to spread probability over all the data, sometimes producing blurry samples. **Variational inference usually minimizes reverse KL** from the approximation to the true posterior [10], and so tends to underestimate posterior uncertainty.

Cross-entropy

The **cross-entropy** of Q relative to P is

$$H(P, Q) = -\mathbb{E}_{x\sim P}[\log Q(x)] = H(P) + D_{\mathrm{KL}}(P \,\|\, Q).$$

Because $H(P)$ does not depend on Q, minimizing cross-entropy with respect to Q is the same as minimizing forward KL. In classification, P is the empirical label distribution — a one-hot vector for each example — and Q is the model's softmax output $\hat{\mathbf{y}}$. For one example with true class y,

$$H(\mathbf{e}_y, \hat{\mathbf{y}}) = -\sum_k [\mathbf{e}_y]_k \log \hat{y}_k = -\log \hat{y}_y,$$

where $\mathbf{e}_y$ is the one-hot vector for class y. Averaged over a dataset, this is exactly the conditional negative log-likelihood. So "cross-entropy loss", "log loss" and "negative log-likelihood of a categorical model" are three names for the same objective. Any loss consisting of a negative log-likelihood is a cross-entropy between the empirical distribution and the model — which is why the mean squared error is also "the cross-entropy" for a Gaussian model [1].

Mutual information

Mutual information measures how much knowing one variable reduces uncertainty about another:

$$I(X;Y) = D_{\mathrm{KL}}(p(x,y) \,\|\, p(x)p(y)) = H(X) - H(X \mid Y) = H(X) + H(Y) - H(X,Y).$$

Here $H(X \mid Y) = -\mathbb{E}_{x,y}[\log p(x \mid y)]$ is the conditional entropy and $H(X,Y)$ the joint entropy. $I(X;Y) \geq 0$, with equality if and only if X and Y are independent — unlike covariance, it detects any form of dependence. It is symmetric in X and Y.

Code: information quantities

```
import numpy as np

p = np.array([.7, .2, .1])
q = np.array([.5, .3, .2])
```

```
entropy = -(p*np.log(p)).sum()
cross_entropy = -(p*np.log(q)).sum()
kl = (p*np.log(p/q)).sum()
reverse_kl = (q*np.log(q/p)).sum()
assert np.allclose(cross_entropy - entropy, kl) and kl >= 0
print("H(p), H(p,q), KL(p||q), KL(q||p) in nats:",
      np.round([entropy, cross_entropy, kl, reverse_kl], 4))
print("H(p) in bits:", (entropy/np.log(2)).round(4))

# Mutual information from a joint table (same weather/umbrella table)
P = np.array([[0.56, 0.14],
              [0.06, 0.24]])
px, py = P.sum(1), P.sum(0)
mi = (P*np.log(P/np.outer(px, py))).sum()
H = lambda t: -(t[t > 0]*np.log(t[t > 0])).sum()
assert np.isclose(mi, H(px) + H(py) - H(P.ravel()))
print("I(weather; umbrella) =", mi.round(4), "nats")

# Cross-entropy loss on one-hot labels equals mean -log q(y)
probs = np.array([[0.7, 0.2, 0.1], [0.1, 0.8, 0.1], [0.3, 0.3, 0.4]])
labels = np.array([0, 1, 2])
print("mean cross-entropy:", (-np.log(probs[np.arange(3), labels])).mean().round(4))
```

Output: $H(p) = 0.8018$, $H(p,q) = 0.8869$, $D_{\mathrm{KL}}(p\|q) = 0.0851$ and $D_{\mathrm{KL}}(q\|p) = 0.0920$ nats — the two directions differ. $H(p)$ is 1.1568 bits. The weather–umbrella mutual information is 0.1637 nats, and both formulas for it agree. The mean cross-entropy of the three predictions is 0.4987 nats. In real training code, compute the cross-entropy from logits with a fused function rather than from probabilities (see ML 01C).

Structured probabilistic models

Plain version. A joint distribution over many variables is far too big to write down as a table. Structured models draw a graph saying which variables directly influence which, and then the joint splits into small pieces.

A joint table over 100 binary variables would need $2^{100} - 1$ numbers. **Structured probabilistic models**, also called **graphical models**, exploit conditional independence to factorize the joint into small factors [1], [3]. There are two main kinds.

Directed models (Bayesian networks) use a directed acyclic graph $\mathcal{G}$. Each variable depends only on its parents $Pa_{\mathcal{G}}(x_i)$:

$$p(\mathbf{x}) = \prod_i p(x_i \mid Pa_{\mathcal{G}}(x_i)).$$

For example, with $a \to b \to c$ and $a \to c$ we have $p(a,b,c) = p(a)\,p(b \mid a)\,p(c \mid a,b)$; deleting the edge $a \to c$ asserts $c \perp a \mid b$ and shortens the last factor to $p(c \mid b)$. Directed graphs are natural for generative stories ("first choose the class, then the pixels"), and a mixture model is the simplest example: $z \to \mathbf{x}$.

Undirected models (Markov random fields) use an undirected graph. Each **clique** $C^{(i)}$ (a set of mutually connected nodes) has a nonnegative **factor** $\phi^{(i)}$, and

$$p(\mathbf{x}) = \frac{1}{Z} \prod_i \phi^{(i)}(C^{(i)}), \qquad Z = \sum_{\mathbf{x}} \prod_i \phi^{(i)}(C^{(i)}).$$

The **partition function** Z normalizes the product; computing it is often the hard part. Undirected models suit symmetric interactions, like neighbouring pixels in an image prior or a conditional random field for segmentation.

A graph is a *description* of a distribution, not a different kind of distribution: any distribution can be written either way, although one may be much more compact. Deep learning often uses these ideas implicitly — an autoregressive model is a fully connected directed model, and a VAE is a two-node directed model $\mathbf{z} \to \mathbf{x}$ whose conditional is a neural network [8].

Modern view

The probability and information theory in this chapter is classical. What has changed in the deep-learning era is *how* it is used and what goes wrong in practice. This section surveys the review literature and recent directions; it summarizes their framing rather than reproducing benchmark numbers.

Calibration: are predicted probabilities honest?

A classifier is **calibrated** if, among all inputs for which it predicts confidence 0.8, about 80% are correct. Guo et al. [11] (ICML 2017) found that modern deep networks are often **overconfident**, and identified depth, width, weight decay and batch normalization as factors that affect calibration. Their main recommendation, **temperature scaling** — dividing all logits by a single scalar T fitted on a validation set — is a one-parameter variant of Platt scaling and remains a strong baseline because it changes confidence without changing the predicted class. Wang's survey [12] organizes calibration methods into four families: post-hoc calibration, regularization during training, uncertainty estimation, and compositions of these; its revised version also covers calibration of large language models. The probabilistic lesson: a softmax output is a model of $P(y \mid \mathbf{x})$, and like any model it can be wrong in its uncertainty, not only in its argmax.

Uncertainty quantification

Two major reviews map the field of uncertainty in deep learning. Abdar et al. [13] (*Information Fusion*, 2021) review Bayesian approximations, ensembles and their applications across many domains. Gawlikowski et al. [14] (*Artificial Intelligence Review*, 2023) distinguish **reducible model (epistemic) uncertainty** from **irreducible data (aleatoric) uncertainty** — roughly the third and first sources of uncertainty from the start of this chapter — and compare deterministic single networks, Bayesian methods, ensembles and test-time augmentation, together with calibration measures.

Two primary studies are especially influential. **Deep ensembles** [15] train several networks from different random initializations and average their predictive distributions; this simple, parallel method was competitive with approximate Bayesian neural networks and expressed higher uncertainty on out-of-distribution inputs. Ovadia et al. [16] (NeurIPS 2019) benchmarked uncertainty methods under **dataset shift** and found that post-hoc calibration on in-distribution validation data did not carry over well, while methods that marginalize over models (such as ensembles) held up better.

A complementary, assumption-light approach is **conformal prediction**. Angelopoulos and Bates [17] give an accessible introduction: using a held-out calibration set, conformal methods turn any pre-trained model's scores into prediction *sets* that contain the true label with a user-chosen probability, assuming the calibration and test data are exchangeable. It does not require the model's probabilities to be calibrated, which makes it attractive for vision and language systems that are used as black boxes.

Mutual information in representation learning

Mutual information became a popular objective for self-supervised learning (ML 05). Contrastive Predictive Coding [18] introduced the **InfoNCE** loss, which classifies a positive pair among negatives and can be shown to maximize a lower bound on mutual information. Poole et al. [19] (ICML 2019) unified many variational MI bounds and showed that they degrade when the true MI is large, trading high bias against high variance. Tschannen et al. [20] (ICLR 2020) argued that the success of these methods "cannot be attributed to the properties of MI alone": the encoder architecture and the parameterization of the estimator matter at least as much. The modern consensus is to treat InfoNCE-style objectives as well-behaved contrastive losses with an information-theoretic motivation, not as precise MI estimators.

Divergences and latent variables in generative models

Generative modelling is applied probability. VAEs [8] maximize an evidence lower bound that combines a reconstruction likelihood with a KL term; Blei et al. [10] review variational inference, which approximates intractable posteriors by minimizing (reverse) KL within a tractable family. Normalizing flows [9] are built entirely on the change-of-variables formula. Diffusion and flow-matching models (ML 07) combine Gaussian noise, conditional expectations and likelihood bounds. The choice of divergence (forward KL, reverse KL, or something else) shapes whether a model covers all the data or concentrates on its modes — the same picture as Figure 5, at scale.

Open problems

- **Calibration under shift.** Calibration that holds on a validation set can fail on new cameras, hospitals or prompts [16]; detecting when this happens is an open question.
- **Separating uncertainty types.** Decomposing predictive uncertainty into epistemic and aleatoric parts is conceptually clear but hard to do reliably for large networks [14].

- **Estimating information in high dimensions.** Variational MI bounds have fundamental bias–variance limits [19], so information-theoretic claims about deep representations need care.
- **Uncertainty for foundation models.** Applying calibration and conformal methods to large vision–language and language models, whose outputs are open-ended, is an active research area [12], [17].

Key takeaways

- Probability describes uncertainty from randomness, partial observation and simplified models; frequentist and Bayesian readings share the same rules.
- A PMF assigns probabilities; a PDF assigns densities, which can exceed one. Probabilities of continuous events are areas.
- The sum rule (marginalization), product rule (conditioning) and chain rule are exact; independence and conditional independence are modelling assumptions.
- Covariance measures linear co-variation only — zero covariance does not imply independence; mutual information detects any dependence.
- Choosing a distribution is choosing a loss: Bernoulli/categorical give cross-entropy, Gaussian gives squared error, Laplace gives absolute error; Gaussian and Laplace priors give L_2 and L_1 regularization.
- Bayes' rule combines evidence with base rates; the same detector can be trustworthy at one prevalence and misleading at another.
- Transforming a random variable rescales its density by the absolute Jacobian determinant.
- Cross-entropy equals entropy plus KL; minimizing it is maximum likelihood. KL is asymmetric: forward KL covers modes, reverse KL seeks one.

Exercises

1. **Units and certainty.** (a) Convert a loss of 1 nat into bits. (b) Compute the entropy, in bits, of a fair coin, a certain outcome, and a fair 8-sided die.

   Hint

   (a) Divide by $\ln 2$: about 1.443 bits. (b) 1 bit, 0 bits and $\log_2 8 = 3$ bits. A uniform distribution over K outcomes has entropy $\log K$, the maximum for K outcomes.

2. **Prevalence shift.** Using the detector in the Bayes section, find the prevalence at which a positive result is exactly a coin flip, $P(\text{defect} \mid +) = 0.5$.

   Hint

   Set $0.9\pi = 0.05(1 - \pi)$, where π is the prevalence. Then $\pi = 0.05/0.95 \approx 0.053$, about 5.3%. Below that, most alarms are false.

3. **Dependent but uncorrelated.** Let X take values $-1, 0, 1$ with equal probability and $Y = X^2$. Show that $\mathrm{Cov}(X, Y) = 0$ and compute $I(X; Y)$ in bits.

Hint

$\mathbb{E}[X] = 0$ and $\mathbb{E}[XY] = \mathbb{E}[X^3] = 0$, so the covariance is zero. Because Y is a function of X, $H(Y \mid X) = 0$ and $I(X; Y) = H(Y)$. $Y = 1$ with probability $2/3$ and $Y = 0$ with probability $1/3$, so $I \approx 0.918$ bits.

4. **Gaussian MLE equals squared error.** Show that maximizing $\prod_i \mathcal{N}(y^{(i)}; f_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}), \sigma^2)$ over $\boldsymbol{\theta}$ (with σ fixed) is equivalent to minimizing the mean squared error. What changes if σ is also predicted by the network?

Hint

The negative log-likelihood is $\sum_i \frac{(y^{(i)} - f_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}))^2}{2\sigma^2} + m \log \sigma + \text{const}$. With σ fixed only the squared error depends on $\boldsymbol{\theta}$. If the network also predicts $\sigma(\mathbf{x})$ (for example through a softplus), each example's error is weighted by $1/\sigma^2$ and a $\log \sigma$ penalty prevents the variance from growing without bound — a heteroscedastic regression model.

5. **Change of variables.** If $X \sim \mathcal{U}(0, 1)$ (uniform) and $Y = -\frac{1}{\lambda} \log X$ with $\lambda > 0$, what is the density of Y? Check your answer by sampling in NumPy.

Hint

$g^{-1}(y) = e^{-\lambda y}$ and $|dg^{-1}/dy| = \lambda e^{-\lambda y}$, while $p_X = 1$ on $(0, 1)$. So $p_Y(y) = \lambda e^{-\lambda y}$ for $y \geq 0$: the exponential distribution. This is the inverse-transform sampling trick.

6. **KL direction.** For $p = (0.5, 0.5, 0)$ and $q = (0.4, 0.4, 0.2)$, compute $D_{\mathrm{KL}}(p \| q)$ and $D_{\mathrm{KL}}(q \| p)$. Which one is infinite, and what does that tell you about training a model by maximum likelihood?

Hint

$D_{\mathrm{KL}}(p \| q) = \log(0.5/0.4) \approx 0.223$ nats (the third term is $0 \log 0 = 0$). $D_{\mathrm{KL}}(q \| p) = \infty$ because q puts mass where p has none. Maximum likelihood minimizes $D_{\mathrm{KL}}(p_{\text{data}} \| q_{\text{model}})$, which becomes infinite if the model assigns zero probability to any observed example — so likelihood-trained models must spread some probability everywhere the data might be.

References

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016, ch. 3 "Probability and Information Theory". Book
2. C. M. Bishop and H. Bishop, *Deep Learning: Foundations and Concepts*, Springer, 2024. Book
3. K. P. Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022. Book

4. M. P. Deisenroth, A. A. Faisal and C. S. Ong, *Mathematics for Machine Learning*, Cambridge University Press, 2020. Book
5. C. E. Shannon, “A Mathematical Theory of Communication,” *Bell System Technical Journal*, vol. 27, pp. 379–423, 1948. DOI
6. S. Kullback and R. A. Leibler, “On Information and Sufficiency,” *The Annals of Mathematical Statistics*, vol. 22, pp. 79–86, 1951. DOI
7. T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006. DOI
8. D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” in *Proc. International Conference on Learning Representations (ICLR)*, 2014. arXiv
9. G. Papamakarios, E. Nalisnick, D. J. Rezende, S. Mohamed and B. Lakshminarayanan, “Normalizing Flows for Probabilistic Modeling and Inference,” *Journal of Machine Learning Research*, vol. 22, no. 57, pp. 1–64, 2021. arXiv
10. D. M. Blei, A. Kucukelbir and J. D. McAuliffe, “Variational Inference: A Review for Statisticians,” *Journal of the American Statistical Association*, vol. 112, no. 518, pp. 859–877, 2017. DOI
11. C. Guo, G. Pleiss, Y. Sun and K. Q. Weinberger, “On Calibration of Modern Neural Networks,” *ICML*, 2017. arXiv
12. C. Wang, “Calibration in Deep Learning: A Survey of the State-of-the-Art,” arXiv:2308.01222, 2023 (revised 2025). arXiv
13. M. Abdar et al., “A Review of Uncertainty Quantification in Deep Learning: Techniques, Applications and Challenges,” *Information Fusion*, vol. 76, pp. 243–297, 2021. DOI
14. J. Gawlikowski et al., “A Survey of Uncertainty in Deep Neural Networks,” *Artificial Intelligence Review*, vol. 56, pp. 1513–1589, 2023. DOI
15. B. Lakshminarayanan, A. Pritzel and C. Blundell, “Simple and Scalable Predictive Uncertainty Estimation using Deep Ensembles,” *NeurIPS (NIPS)*, 2017. arXiv
16. Y. Ovadia et al., “Can You Trust Your Model’s Uncertainty? Evaluating Predictive Uncertainty Under Dataset Shift,” *NeurIPS*, 2019. arXiv
17. A. N. Angelopoulos and S. Bates, “A Gentle Introduction to Conformal Prediction and Distribution-Free Uncertainty Quantification,” arXiv:2107.07511, 2021. arXiv
18. A. van den Oord, Y. Li and O. Vinyals, “Representation Learning with Contrastive Predictive Coding,” arXiv:1807.03748, 2018. arXiv
19. B. Poole, S. Ozair, A. van den Oord, A. A. Alemi and G. Tucker, “On Variational Bounds of Mutual Information,” *ICML*, 2019. arXiv
20. M. Tschannen, J. Djolonga, P. K. Rubenstein, S. Gelly and M. Lucic, “On Mutual Information Maximization for Representation Learning,” *ICLR*, 2020. arXiv

ML 01C • Numerical Computation and Gradient Checks

Floating point, overflow and underflow, stable softmax, conditioning, gradients, Jacobians and Hessians, Newton-type methods, KKT conditions and mixed precision: make the mathematical model survive actual computation.

Online edition: https://blog.shyandram.dev/en/learn/ml/ml-01c-numerical-computation/

What you'll learn

- How computers store real numbers, what machine epsilon means, and why float16, bfloat16, float32 and float64 behave differently.
- Why exact formulas overflow, underflow or lose digits, and how to compute softmax, log-sum-exp and logistic losses stably.
- What a condition number measures, and how ill-conditioning differs from an unstable algorithm.
- How gradients, Jacobians and Hessians describe a loss surface, why gradient descent zigzags, and what Newton and quasi-Newton methods change.
- How to handle constraints with projections, the generalized Lagrangian and the KKT conditions.
- How to check derivatives, debug non-finite values and use mixed precision safely.

The big picture

Machine learning is written in the language of real numbers, but computers do not have real numbers. They have a finite set of floating-point numbers, and every arithmetic operation rounds its result to one of them. Most of the time this rounding is invisible. Sometimes it is catastrophic: an exponential overflows to infinity, a tiny probability becomes exactly zero and its logarithm becomes minus infinity, or two nearly equal numbers are subtracted and almost all their significant digits disappear.

The second half of the story is optimization. Training a model means minimizing a function of millions or billions of parameters, and we can only do that numerically, by taking many small steps guided by derivatives. How fast and how reliably those steps make progress depends on the shape of the loss surface — its slopes and its curvatures — and on how we take the steps. This chapter follows the "Numerical Computation" chapter of Goodfellow, Bengio and Courville [1], adds the floating-point details that modern low-precision training relies on, and connects each idea to small NumPy experiments.

Explain like I'm five

> Imagine a calculator that can show only seven digits. If you ask it for one million plus 0.1, it shows one million: the 0.1 fell off the screen. If you then subtract one million, you get zero instead of 0.1. Good numerical code arranges its steps so the important digits never fall off the screen. Optimization is a different game: you are blindfolded on a hilly field and want to reach the lowest point. You can feel the slope under your feet (the gradient) and, with more effort, how the ground curves (the Hessian). Numerical computation is about walking down safely without your calculator lying to you.

Read ML 01B first if softmax and cross-entropy are new, and keep ML 01A nearby for eigenvalues and the singular value decomposition. ML 02 then uses everything here to train neural networks.

Floating-point numbers

Plain version. A floating-point number stores a sign, a few significant digits and a power of two. The *relative* spacing between neighbouring numbers is roughly constant, so a big number has a big absolute rounding error and a small number has a small one.

How a float is stored

The IEEE 754 standard [3] defines binary floating-point formats used by essentially all CPUs and GPUs. A normal (non-special) number is stored as

$$x = (-1)^s \times \left(1 + \sum_{i=1}^{t} b_i \, 2^{-i}\right) \times 2^{e},$$

where $s \in \{0,1\}$ is the sign bit, $b_1, \dots, b_t \in \{0,1\}$ are the t stored fraction bits of the significand (the leading 1 is implicit), and e is an integer exponent restricted to a fixed range. Two numbers matter most:

- **Machine epsilon** $\varepsilon_{\text{mach}} = 2^{-t}$ is the gap between 1 and the next larger representable number.
- **Unit roundoff** $u = \varepsilon_{\text{mach}}/2$ bounds the relative error of rounding to nearest: for any real x in the normal range, the stored value is $\text{fl}(x) = x(1+\delta)$ with $|\delta| \le u$.

The standard also requires that each basic operation $(+, -, \times, \div, \sqrt{\cdot})$ is computed as if exactly and then rounded once: $\text{fl}(a \circ b) = (a \circ b)(1+\delta)$, $|\delta| \le u$. This one rule is the foundation of rounding-error analysis [2], [4].

Below the smallest normal number the format uses **subnormal** numbers, which give up leading precision to fill the gap down to zero gradually. Special values complete the picture: $\pm\infty$ for results that are too large, and **NaN** ("not a number") for invalid operations such as $0/0$, $\infty - \infty$ or $\infty \times 0$. Once a NaN appears, almost every operation that touches it returns NaN, which is why a single bad value can poison a whole training run.

The four formats you will meet

The table below was computed with `numpy.finfo` (and `torch.finfo` for bfloat16, which NumPy lacks). "Bits" lists sign/exponent/fraction.

Format	Bits	$\varepsilon_{\text{mach}}$	Largest finite	Smallest normal	Smallest subnormal
float16 (IEEE half)	1/5/10	9.77×10^{-4}	65504	6.10×10^{-5}	5.96×10^{-8}
bfloat16	1/8/7	7.81×10^{-3}	3.39×10^{38}	1.18×10^{-38}	9.18×10^{-41}
float32 (single)	1/8/23	1.19×10^{-7}	3.40×10^{38}	1.18×10^{-38}	1.40×10^{-45}
float64 (double)	1/11/52	2.22×10^{-16}	1.80×10^{308}	2.23×10^{-308}	4.94×10^{-324}

Read the table in two directions. The **exponent bits** set the *range*: float16 overflows above 65504, while bfloat16 keeps float32's 8 exponent bits and therefore its range. The **fraction bits** set the *precision*: float32 carries about 7 significant decimal digits, float64 about 16, float16 about 3, and bfloat16 only about 2. That trade-off — bfloat16's wide range with coarse precision versus float16's narrow range with finer precision — is the reason the two half-precision formats need different training recipes (see the mixed-precision section below and [5], [12]).

```
import numpy as np

for dtype in (np.float16, np.float32, np.float64):
    info = np.finfo(dtype)
    print(f"{info.dtype}: eps={float(info.eps):.3e}  max={float(info.max):.3e}  "
          f"tiny={float(info.tiny):.3e}  subnormal={float(info.smallest_subnormal):.3e}")

# Rounding: 1 + eps/2 rounds back to 1 in float32
e = np.finfo(np.float32).eps
print(np.float32(1) + np.float32(e/2) == np.float32(1))   # True
print(np.float32(1) + np.float32(e) == np.float32(1))     # False

# Addition is not associative
a, b, c = np.float32(1e8), np.float32(-1e8), np.float32(1.0)
print((a + b) + c, a + (b + c))                           # 1.0 0.0
```

The last line is worth pausing on. In exact arithmetic both orders give 1. In float32, $-10^8 + 1$ rounds back to -10^8 because the spacing between float32 numbers near 10^8 is 8, so the 1 is lost. Consequences in practice: summing the same numbers in a different order — for example on a different number of GPUs, or with a different reduction kernel — can change the result in the last digits. This is a major source of run-to-run nondeterminism, and it is not a bug in your code [13].

Overflow, underflow and cancellation

Plain version. Overflow means a result is too large to store; underflow means it is too small and becomes zero (or loses precision as a subnormal). Cancellation means subtracting nearly equal

numbers, which keeps the rounding error but throws away the true signal.

Overflow and underflow

The exponential is the usual culprit in machine learning. Because $\exp(z)$ exceeds the largest float32 once $z > \ln(3.40 \times 10^{38}) \approx 88.7$, a logit of 100 overflows in float32 (and a logit of about 11.1 already overflows in float16, since $\ln 65504 \approx 11.09$). In the other direction, $\exp(-110) \approx 1.7 \times 10^{-48}$ is below the smallest float32 subnormal and becomes exactly 0. Neither event raises an error by default; they produce `inf` and `0`, and the next operation turns them into `nan` or $-\infty$.

```
import numpy as np

with np.errstate(over="ignore", under="ignore", invalid="ignore"):
    print(np.exp(np.float32(100.0)))        # inf: overflow
    print(np.exp(np.float32(-110.0)))       # 0.0: underflow
    big = np.exp(np.float32(100.0))
    print(big / big)                        # nan: inf / inf

# Catastrophic cancellation and its cure
x = 1e-10
print(np.log(1 + x), np.log1p(x))           # roundoff vs accurate
print(np.exp(x) - 1, np.expm1(x))

# Variance: the one-pass formula cancels badly when the mean is large
rng = np.random.default_rng(0)
data = (1e4 + rng.normal(0, 1, 100_000)).astype(np.float32)
one_pass = np.mean(data**2) - np.mean(data)**2
two_pass = np.mean((data - data.mean())**2)
print("one-pass:", one_pass, " two-pass:", two_pass)
```

On this machine the one-pass variance comes out as -8.0 — a negative variance — while the two-pass formula gives the correct value of about 1.0. The data have mean 10^4 and variance 1, so $\mathbb{E}[X^2] \approx 10^8$ and $(\mathbb{E}X)^2 \approx 10^8$; float32 can only resolve differences of about 8 at that magnitude, so the true difference of 1 is pure noise.

Why subtraction is the dangerous operation

Suppose $\hat{a} = a(1+\delta_a)$ and $\hat{b} = b(1+\delta_b)$ are stored with relative errors $|\delta_a|, |\delta_b| \le u$. Even if the subtraction itself is exact,

$$\frac{|(\hat{a}-\hat{b}) - (a-b)|}{|a-b|} = \frac{|a\delta_a - b\delta_b|}{|a-b|} \le u\,\frac{|a|+|b|}{|a-b|}.$$

Here a, b are the true values and u is the unit roundoff. The factor $(|a|+|b|)/|a-b|$ is huge when $a \approx b$: the subtraction does not create error, it *reveals* the error already present in the inputs by removing the digits they shared. The cures are algebraic: rewrite the formula so the subtraction never happens. Examples you will use:

- `log1p(x)` computes $\log(1+x)$ and `expm1(x)` computes $e^x - 1$ accurately for small x, where the naive forms lose almost all digits (see the output above).
- Variance: subtract the mean first (two-pass), or use Welford's streaming update.
- Softmax and log-sum-exp: subtract the maximum first, as the next section shows.
- Distances: $\|x-y\|^2 = \|x\|^2 + \|y\|^2 - 2x^\top y$ is fast but cancels badly when $x \approx y$; clamp it at zero or compute $x - y$ directly when accuracy matters.

Stable softmax and log-sum-exp

Plain version. Subtract a shared constant before exponentiating, and compute logarithms of probabilities directly instead of first creating tiny probabilities and then taking their logarithm.

The softmax turns a vector of logits $z \in \mathbb{R}^K$ into probabilities:

$$\operatorname{softmax}(z)_i = \frac{\exp(z_i)}{\sum_{j=1}^{K} \exp(z_j)}, \qquad i = 1, \dots, K.$$

It is invariant to adding the same constant c to every logit, because the factor $\exp(c)$ cancels between numerator and denominator. Choose $c = -m$ with $m = \max_i z_i$:

$$p_i = \frac{\exp(z_i - m)}{\sum_j \exp(z_j - m)}.$$

Now every exponent is ≤ 0, so nothing overflows, and the largest term is exactly $\exp(0) = 1$, so the denominator is at least 1 and cannot underflow to zero. The same trick stabilizes the **log-sum-exp** function,

$$\operatorname{LSE}(z) = \log \sum_j \exp(z_j) = m + \log \sum_j \exp(z_j - m), \qquad \log p_i = z_i - \operatorname{LSE}(z).$$

LSE is a smooth version of the maximum: $\max_i z_i \le \operatorname{LSE}(z) \le \max_i z_i + \log K$. Its gradient is exactly the softmax, $\partial \operatorname{LSE} / \partial z_i = p_i$.

For a target class y, the cross-entropy loss is $-\log p_y = \operatorname{LSE}(z) - z_y$, and its gradient with respect to the logits is the famously simple $p - e_y$, where e_y is the one-hot vector of the target. Computing `softmax` first and then `log` can still fail: if p_y underflows to 0, its logarithm is $-\infty$ even though $\operatorname{LSE}(z) - z_y$ is a perfectly ordinary finite number. This is why frameworks provide fused `log_softmax` and `cross_entropy` functions that take logits.

Code: logits that overflow naive exponentiation

This standalone NumPy example checks stable probabilities, log probabilities and invariance to shifting logits.

```
import numpy as np
```

```
def logsumexp(z):
    m = np.max(z)
    return m + np.log(np.exp(z-m).sum())
def logsoftmax(z):
    return z-logsumexp(z)

z = np.array([1000., 1001., 999.], dtype=np.float64)
logp = logsoftmax(z)
p = np.exp(logp)
assert np.allclose(p.sum(), 1)
assert np.allclose(logsoftmax(z+12345), logp)
print("probabilities:", p)
print("cross-entropy for class 1:", -logp[1])
with np.errstate(over="ignore", invalid="ignore"):
    naive = np.exp(z) / np.exp(z).sum()
print("naive softmax:", naive)
```

The stable version prints probabilities $[0.245, 0.665, 0.090]$ and a cross-entropy of 0.408 nats for class 1; the naive version prints `[nan nan nan]`, because $\exp(1000)$ overflows even in float64 (whose limit is $\exp(709.78)$).

Stable logistic (binary cross-entropy) loss

Binary classification has the same issue in a different form. With logit z, label $y \in \{0, 1\}$ and sigmoid $\sigma(z) = 1/(1 + e^{-z})$, the loss $-y \log \sigma(z) - (1 - y) \log(1 - \sigma(z))$ simplifies algebraically to

$$\ell(z, y) = \operatorname{softplus}(z) - yz, \qquad \operatorname{softplus}(z) = \log(1 + e^{z}) = \max(z, 0) + \log\!\left(1 + e^{-|z|}\right).$$

The right-hand form never exponentiates a positive number, and `log1p` keeps the small correction accurate. Prefer `binary_cross_entropy_with_logits` to a hand-built sigmoid followed by a logarithm.

```
import numpy as np

def bce_naive(z, y):
    s = 1 / (1 + np.exp(-z))
    return -(y*np.log(s) + (1-y)*np.log(1-s))

def bce_stable(z, y):
    # softplus(z) - y*z, with softplus(z) = max(z,0) + log1p(exp(-|z|))
    return np.maximum(z, 0) + np.log1p(np.exp(-np.abs(z))) - y*z

z = np.array([-800., -30., 0., 30., 800.])
y = np.array([1., 1., 0., 0., 0.])
with np.errstate(over="ignore", divide="ignore", invalid="ignore"):
    print("naive :", bce_naive(z, y))
print("stable:", bce_stable(z, y))
```

The naive version returns `inf` for the two confident mistakes ($z = \pm 800$) and 30.001 instead of 30.000 at $z = 30$, because $1 - \sigma(30)$ is computed by cancellation. The stable version returns 800, 30, $\log 2$, 30 and 800 — the correct values.

Poor conditioning

Plain version. Some problems are inherently sensitive: a tiny change in the input produces a large change in the answer, no matter how carefully you compute. The **condition number** measures that sensitivity.

Condition number of a linear system

Consider solving $Ax = b$ for a square invertible matrix A. If the data change to $b + \delta b$, the solution changes to $x + \delta x$ with

$$\frac{\|\delta x\|}{\|x\|} \le \kappa(A)\,\frac{\|\delta b\|}{\|b\|}, \qquad \kappa(A) = \|A\|\,\|A^{-1}\| = \frac{\sigma_{\max}(A)}{\sigma_{\min}(A)} \ \text{(2-norm)}.$$

Here $\|\cdot\|$ is the Euclidean vector norm and its induced matrix norm, and $\sigma_{\max}, \sigma_{\min}$ are the largest and smallest singular values of A. For a symmetric positive-definite matrix, singular values equal eigenvalues and $\kappa = \lambda_{\max}/\lambda_{\min}$. Goodfellow et al. state the same idea for the function $f(x) = A^{-1}x$ with an eigendecomposition: its condition number is $\max_{i,j} |\lambda_i/\lambda_j|$ [1]. A practical rule of thumb: with $\kappa \approx 10^k$, expect to lose about k decimal digits of accuracy relative to the precision of your inputs [4].

```
import numpy as np

A = np.array([[1.0, 1.0],
              [1.0, 1.0001]])
print("condition number:", np.linalg.cond(A))

b = np.array([2.0, 2.0001])
x = np.linalg.solve(A, b)
db = np.array([0.0, 0.0001])                  # tiny change in the data
x2 = np.linalg.solve(A, b + db)
rel_in = np.linalg.norm(db) / np.linalg.norm(b)
rel_out = np.linalg.norm(x2 - x) / np.linalg.norm(x)
print("x:", x, " x after perturbation:", x2)
print(f"relative change in b: {rel_in:.1e}, in x: {rel_out:.1e}, "
      f"amplification: {rel_out/rel_in:.0f}")
```

The two columns of A are almost identical, $\kappa(A) \approx 4 \times 10^4$, and a relative change of 3.5×10^{-5} in b moves the solution from $(1, 1)$ to $(0, 2)$ — a 100% change, an amplification of about 28,000, within the bound κ. Nothing was computed wrongly. The *problem* is ill-conditioned.

Ill-conditioned problem versus unstable algorithm

Keep two ideas apart:

- An **ill-conditioned problem** amplifies small changes in its input. No algorithm can fix that; you need better data, regularization (which changes the problem) or a reformulation.
- An **unstable algorithm** adds avoidable error to a problem that may be perfectly well conditioned. The naive softmax and the one-pass variance are unstable algorithms; changing the algorithm fixes them.

Least squares shows both. If features are nearly collinear, the coefficients are ill-conditioned (the problem). Solving the normal equations $X^\top X w = X^\top y$ makes things worse than necessary because $\kappa(X^\top X) = \kappa(X)^2$ (the algorithm); a QR- or SVD-based solver works with X directly. Ridge regression, which adds $\lambda\|w\|^2$, changes the problem to $(X^\top X + \lambda I)w = X^\top y$, whose condition number $(\sigma_{\max}^2 + \lambda)/(\sigma_{\min}^2 + \lambda)$ is smaller — a theme ML 01A develops further.

Gradient-based optimization

Plain version. To minimize a function, look at its slope and step downhill. The gradient points uphill most steeply, so step in the opposite direction, with a step size small enough that the slope you measured is still roughly valid.

Derivatives, gradients and directional derivatives

We minimize an **objective** (also called cost, loss or error) function $f : \mathbb{R}^n \to \mathbb{R}$. For one variable, the derivative $f'(x)$ gives the first-order approximation $f(x+\epsilon) \approx f(x) + \epsilon f'(x)$, so moving a small step against the sign of $f'(x)$ decreases f. For many variables, the **gradient** $\nabla_x f(x) \in \mathbb{R}^n$ collects the partial derivatives $\partial f / \partial x_i$. The **directional derivative** in a unit direction u is the slope along u:

$$\frac{\partial}{\partial \alpha} f(x + \alpha u)\Big|_{\alpha=0} = u^\top \nabla_x f(x) = \|\nabla_x f(x)\|_2 \cos\theta,$$

where θ is the angle between u and the gradient. It is most negative when $\theta = \pi$, that is, when u points along $-\nabla_x f(x)$. This is the **method of steepest descent**, or **gradient descent**:

$$x \leftarrow x - \eta \nabla_x f(x),$$

with **learning rate** (step size) $\eta > 0$. You can pick η as a small constant, try several values and keep the one with the lowest objective (a **line search**), or follow a schedule. Gradient descent converges when every element of the gradient is (close to) zero.

Critical points: minima, maxima and saddles

Points where $\nabla_x f(x) = 0$ are **critical** or **stationary points**. A **local minimum** is lower than all neighbouring points, a **local maximum** is higher, and a **saddle point** is neither: it is a minimum along some directions and a maximum along others. A **global minimum** has the lowest value anywhere; a function can have several global minima and many local minima that are not global.

For deep networks we rarely find a true global minimum, and we do not need to. We settle for parameters with a low enough loss that generalize well. In high dimensions, saddle points rather than poor local minima are the more common critical points of many random-like functions, and they can slow gradient-based optimizers because the gradient becomes small near them [8]. Figure 1 shows the three kinds of critical point.

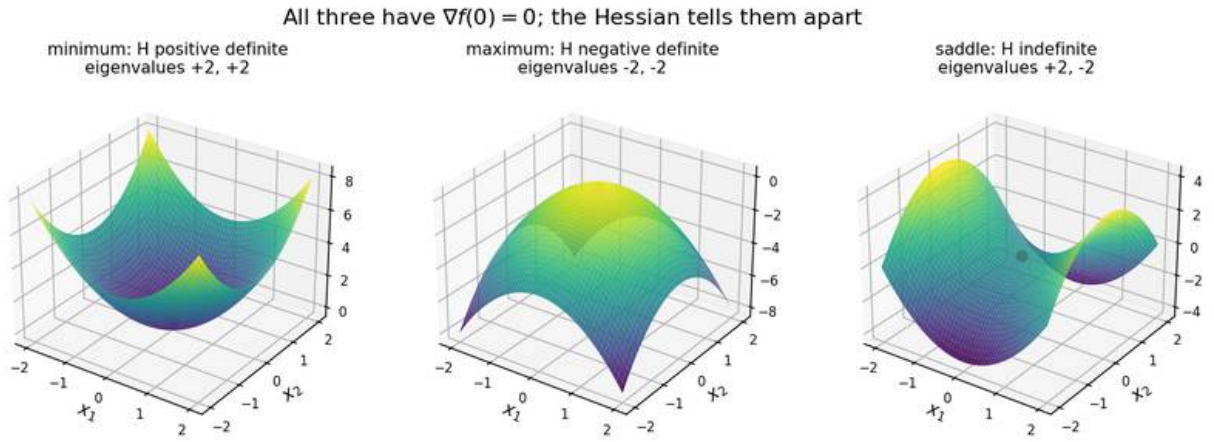


Figure 1 — Three functions with zero gradient at the origin. The Hessian's eigenvalues distinguish them: all positive (minimum), all negative (maximum), mixed signs (saddle).

Jacobian and Hessian matrices

Plain version. The Jacobian collects all first derivatives of a vector-valued function. The Hessian collects all second derivatives of a scalar function: it tells you how the slope itself changes, which is curvature.

Definitions

For a function $f : \mathbb{R}^m \to \mathbb{R}^n$, the **Jacobian** $J \in \mathbb{R}^{n \times m}$ has entries

$$J_{ij} = \frac{\partial f_i(x)}{\partial x_j}, \qquad i = 1, \dots, n,\ j = 1, \dots, m.$$

Row i is the gradient of output f_i. Backpropagation is repeated multiplication by Jacobians; reverse-mode automatic differentiation computes vector–Jacobian products $v^\top J$ without ever

forming J, which is why a gradient costs only a small constant multiple of one function evaluation [11].

For a scalar function $f : \mathbb{R}^n \to \mathbb{R}$, the **Hessian** $H(f)(x) \in \mathbb{R}^{n \times n}$ is the Jacobian of the gradient:

$$H(f)(x)_{ij} = \frac{\partial^2 f(x)}{\partial x_i \, \partial x_j}.$$

Wherever the second partial derivatives are continuous, the order of differentiation does not matter, so $H_{ij} = H_{ji}$: the Hessian is symmetric. A real symmetric matrix has real eigenvalues $\lambda_1, \ldots, \lambda_n$ and an orthonormal basis of eigenvectors $v_1, \ldots, v_n$. The **second derivative in a unit direction** d is $d^\top H d$. If d is an eigenvector v_i, it equals λ_i; for other directions it is a weighted average $\sum_i \lambda_i (d^\top v_i)^2$ of the eigenvalues, with weights between 0 and 1 that sum to 1. So $\lambda_{\max}$ is the largest curvature in any direction and $\lambda_{\min}$ the smallest.

What a gradient step really does: second-order Taylor expansion

Write $g = \nabla_x f(x^{(0)})$ and $H = H(f)(x^{(0)})$. The second-order Taylor approximation around $x^{(0)}$ is

$$f(x) \approx f(x^{(0)}) + (x - x^{(0)})^\top g + \tfrac{1}{2}(x - x^{(0)})^\top H (x - x^{(0)}).$$

Substituting a gradient step $x = x^{(0)} - \eta g$ gives

$$f(x^{(0)} - \eta g) \approx f(x^{(0)}) - \eta \, g^\top g + \tfrac{1}{2} \eta^2 \, g^\top H g.$$

The three terms are the current value, the improvement predicted by the slope, and a correction for curvature. If $g^\top H g \le 0$, the approximation keeps decreasing as η grows (until the approximation itself stops being valid). If $g^\top H g > 0$, the curvature term eventually wins, and the approximation is minimized at

$$\eta^* = \frac{g^\top g}{g^\top H g}.$$

In the worst case, g is aligned with the eigenvector of $\lambda_{\max}$ and $\eta^* = 1/\lambda_{\max}$. Large curvature forces small steps.

The second derivative test

At a critical point ($g = 0$) the Taylor expansion reduces to the quadratic term, so the Hessian decides what kind of point it is:

- H **positive definite** (all $\lambda_i > 0$): local minimum.
- H **negative definite** (all $\lambda_i < 0$): local maximum.
- H has at least one positive and at least one negative eigenvalue: **saddle point**.
- Otherwise (some $\lambda_i = 0$ and the rest of one sign) the test is inconclusive.

In one dimension a saddle cannot exist; in n dimensions it needs only one eigenvalue of the "wrong" sign, which is why saddles become so common as n grows [8].

Condition number of the Hessian: why gradient descent zigzags

On the quadratic $f(w) = \frac{1}{2} w^\top H w$ with H symmetric positive definite, gradient descent is linear: $w \leftarrow (I - \eta H) w$. Expanding w in the eigenvectors of H, the component along v_i is multiplied by $(1 - \eta \lambda_i)$ at every step. Therefore:

- The iteration is **stable** only if $|1 - \eta \lambda_i| < 1$ for every i, that is, $0 < \eta < 2/\lambda_{\max}$.
- The **slowest** direction is the flat one, contracting by $|1 - \eta \lambda_{\min}|$ per step.
- The best fixed step, $\eta = 2/(\lambda_{\max} + \lambda_{\min})$, contracts the error by $(\kappa - 1)/(\kappa + 1)$ per step, with $\kappa = \lambda_{\max}/\lambda_{\min}$ the condition number of the Hessian.

So a large condition number means one direction wants tiny steps (or it diverges) while another direction needs many steps to make progress. Gradient descent wastes its time bouncing across the steep walls of a narrow valley — the zigzag in Figure 2 — while creeping along the valley floor. Badly scaled input features produce exactly this situation, which is one reason standardization and normalization layers speed up training.

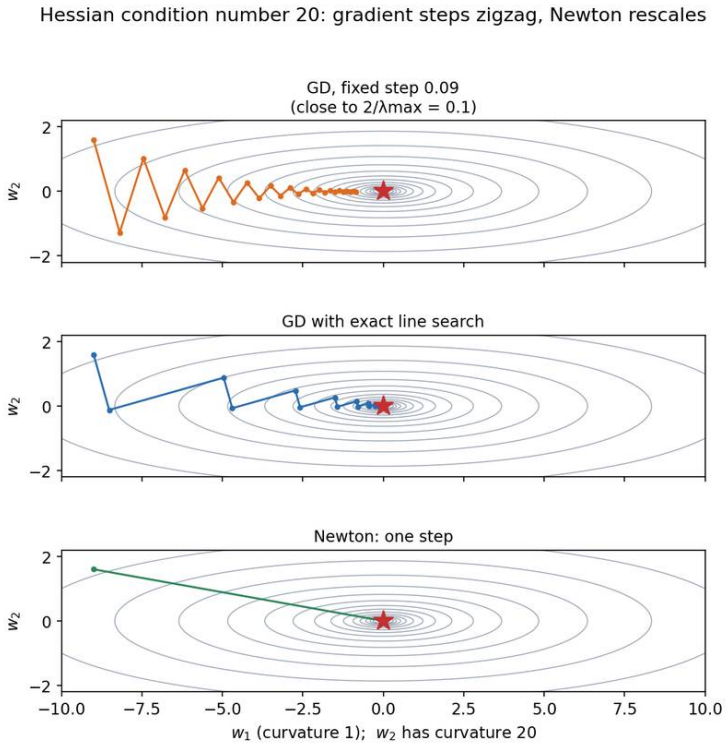


Figure 2 — Minimizing a quadratic with Hessian condition number 20. Gradient descent with a step close to the stability limit oscillates across the steep direction; exact line search makes successive steps orthogonal; Newton's method rescales by the inverse Hessian and reaches the minimum of a quadratic in one step.

Figure 3 shows the stability limit numerically. Below $2/\lambda_{\max}$ the loss decreases; just above it the steep component grows by a factor $|1 - \eta \lambda_{\max}| > 1$ every step and the loss explodes.

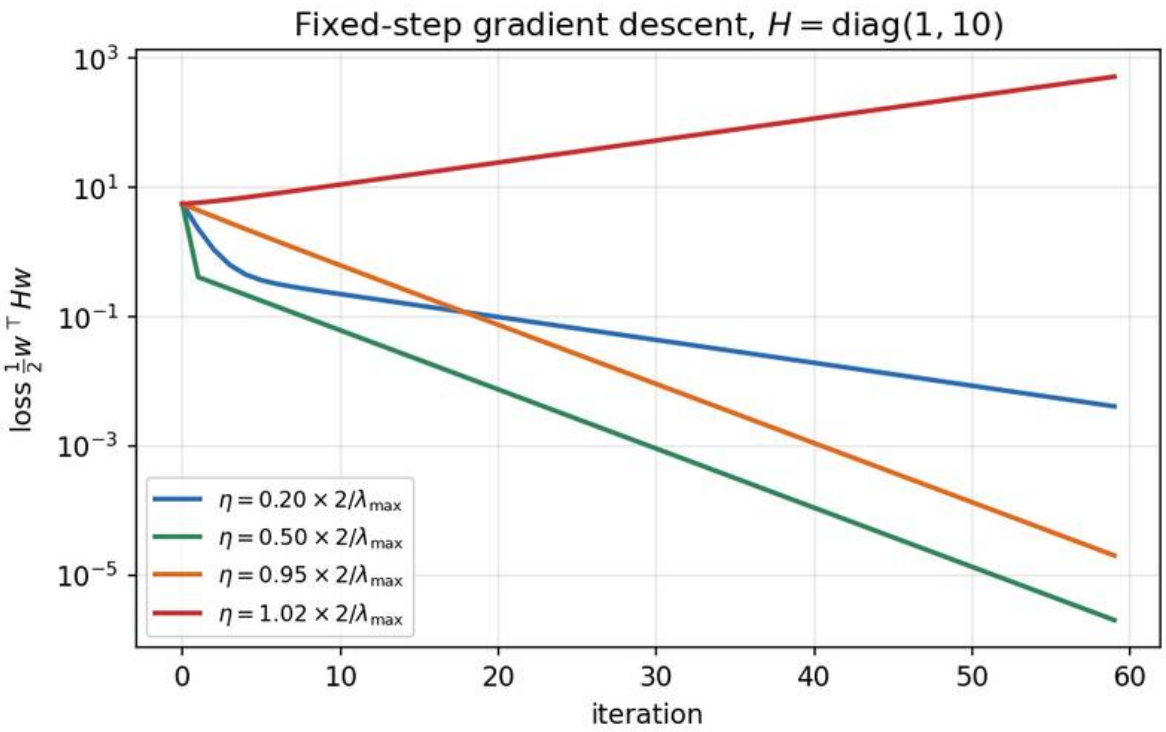


Figure 3 — Fixed-step gradient descent on a quadratic with eigenvalues 1 and 10. Step sizes below 2/λmax converge (fastest near 2/(λmax+λmin)); a step just 2% above the limit diverges.

```
import numpy as np

H = np.diag([1.0, 25.0])                    # Hessian: condition number 25
f = lambda w: 0.5 * w @ H @ w
grad = lambda w: H @ w

def run_gd(eta, steps=50):
    w = np.array([10.0, 1.0])
    for _ in range(steps):
        w = w - eta * grad(w)
    return f(w)

lam_max = np.linalg.eigvalsh(H).max()
for eta in (0.01, 0.07, 0.079, 0.081):       # stability limit 2/25 = 0.08
    print(f"eta={eta:<6} loss after 50 steps: {run_gd(eta):.3e}")

# Exact line search along -g for a quadratic: eps* = g'g / g'Hg
w = np.array([10.0, 1.0])
g = grad(w)
eps_star = (g @ g) / (g @ H @ g)
print("optimal step along the gradient:", round(eps_star, 4))

# One Newton step lands on the minimizer of a quadratic
w_newton = w - np.linalg.solve(H, g)
```

```
print("after one Newton step:", w_newton)
```

The output shows the whole story: $\eta = 0.01$ is stable but slow (loss 18.3 after 50 steps), $\eta = 0.07$ reaches 0.035, $\eta = 0.079$ is barely stable and the steep component still oscillates (loss about 1.0), and $\eta = 0.081$ diverges (loss 148). The optimal step along the first gradient is 0.046, and one Newton step lands exactly on $(0, 0)$.

Second-order methods

Plain version. Instead of following the slope alone, use the curvature as well: divide the step by how sharply the function curves in each direction, so steep directions get small steps and flat directions get large ones.

Newton's method

Setting the gradient of the second-order Taylor approximation to zero gives the **Newton step**

$$x^* = x^{(0)} - H(f)(x^{(0)})^{-1} \nabla_x f(x^{(0)}).$$

On a positive-definite quadratic this jumps to the minimum in one step (Figure 2, bottom). On a general function that is locally close to a positive-definite quadratic, iterating the update converges much faster than gradient descent — quadratically near the solution, under standard assumptions [6]. The cost is the Hessian: storing it takes $O(n^2)$ memory and solving with it $O(n^3)$ time, impossible for a network with 10^8 parameters.

There is also a more fundamental problem for non-convex objectives. Newton's method finds a point where the *gradient* vanishes, and it treats a saddle the same way as a minimum: near a saddle, the Newton step points *toward* it. In a landscape full of saddles, plain Newton is attracted to exactly the points we want to escape. Dauphin et al. proposed a "saddle-free Newton" method that rescales by the absolute values of the Hessian eigenvalues to avoid this [8]. Gradient descent, by contrast, is not attracted to saddles in the same way, although it can move slowly near them.

Quasi-Newton methods and L-BFGS

Quasi-Newton methods build an approximation $B_k \approx H$ (or of its inverse) from gradients alone. After a step $s_k = x_{k+1} - x_k$ with gradient change $y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$, they require the **secant condition** $B_{k+1} s_k = y_k$, which says the model's curvature along the step matches the observed change in slope. BFGS is the most widely used update of this kind; **L-BFGS** (limited-memory BFGS) stores only the last m pairs (s_k, y_k) and applies the inverse-Hessian approximation implicitly in $O(mn)$ time [6], [10]. L-BFGS is excellent for smooth, deterministic, medium-sized problems (logistic regression, style transfer on one image, fitting a few thousand parameters); its curvature estimates suffer when gradients are noisy mini-batch estimates.

Hessian-vector products

Many curvature-aware methods never need H itself, only products Hv. These can be computed exactly at roughly the cost of a couple of gradient evaluations, because $Hv = \nabla_x(\nabla_x f(x)^\top v)$ is the gradient of a scalar; Pearlmutter showed how to do this with automatic differentiation [9]. A finite-difference version, $Hv \approx [\nabla f(x + rv) - \nabla f(x - rv)]/(2r)$, is a useful check. Power iteration with Hessian-vector products estimates $\lambda_{\max}$, which is how researchers track the sharpness of the loss during training.

```
import numpy as np

rng = np.random.default_rng(0)
X = rng.normal(size=(50, 4))
y = (rng.random(50) < 0.5).astype(float)

def grad(w):                                    # logistic regression, mean loss
    p = 1 / (1 + np.exp(-(X @ w)))
    return X.T @ (p - y) / len(y)

def hessian(w):
    p = 1 / (1 + np.exp(-(X @ w)))
    return (X.T * (p * (1 - p))) @ X / len(y)

w = rng.normal(size=4)
v = rng.normal(size=4)
r = 1e-5
hvp_fd = (grad(w + r*v) - grad(w - r*v)) / (2*r)   # never forms H
print("Hv (finite diff):", hvp_fd.round(6))
print("Hv (explicit)   :", (hessian(w) @ v).round(6))
print("eigenvalues of H:", np.linalg.eigvalsh(hessian(w)).round(4))
```

Both products agree to six decimals, and all Hessian eigenvalues are positive: logistic regression is convex, so its Hessian is positive semidefinite everywhere.

Lipschitz continuity and convexity

Deep-learning objectives come with few guarantees, but two properties explain much of the theory. A function is **Lipschitz continuous** with constant L if $|f(x) - f(y)| \le L\|x - y\|_2$ for all x, y: its rate of change is bounded. When the *gradient* is L-Lipschitz (the function is "L-smooth"), the curvature is bounded by L, and a gradient step with $\eta \le 1/L$ is guaranteed not to increase the function:

$$f(x - \eta \nabla f(x)) \le f(x) - \tfrac{\eta}{2}\|\nabla f(x)\|_2^2 \qquad (0 < \eta \le 1/L).$$

This is the general form of the "$\eta < 2/\lambda_{\max}$" rule above, and it is the starting point of most convergence proofs [7], [14].

A function is **convex** if its Hessian is positive semidefinite everywhere (for twice-differentiable functions). Convex functions have no saddles and no spurious local minima: every local minimum is global. Least squares, logistic regression and support vector machines are convex; deep networks are not. Convex optimization is the source of many tools, such as duality and the KKT conditions below, that deep learning borrows [7].

Constrained optimization

Plain version. Sometimes the answer must lie in an allowed region — weights with bounded norm, probabilities that sum to one, a physical quantity that must be positive. Either step and then project back into the region, or add a penalty for each constraint whose strength is chosen automatically.

Projected gradient descent

We want to minimize $f(x)$ over a feasible set $\mathbb{S} \subseteq \mathbb{R}^n$. When projecting onto $\mathbb{S}$ is easy, take a normal gradient step and then return to the nearest feasible point:

$$x \leftarrow \operatorname{Proj}_{\mathbb{S}}\left(x - \eta \nabla f(x)\right), \qquad \operatorname{Proj}_{\mathbb{S}}(v) = \arg\min_{x \in \mathbb{S}} \|x - v\|_2.$$

For the Euclidean ball $\{x : \|x\|_2 \le r\}$ the projection is $v \cdot \min(1, r/\|v\|_2)$: rescale if too long. Max-norm constraints on weights, clipping parameters into a box, and projecting onto the probability simplex all follow this pattern. Another option is to reparameterize so the constraint holds automatically, for example $x = r\,(\cos\theta, \sin\theta)$ for a vector on a circle, or a softmax to produce probabilities.

The generalized Lagrangian

A more general approach describes the feasible set with m equality constraints $g^{(i)}(x) = 0$ and k inequality constraints $h^{(j)}(x) \le 0$:

$$\mathbb{S} = \{x \mid g^{(i)}(x) = 0 \ \forall i,\ h^{(j)}(x) \le 0 \ \forall j\}.$$

Introduce one **KKT multiplier** for each constraint, λ_i for equalities and α_j for inequalities, and define the **generalized Lagrangian**

$$L(x, \lambda, \alpha) = f(x) + \sum_{i=1}^{m} \lambda_i g^{(i)}(x) + \sum_{j=1}^{k} \alpha_j h^{(j)}(x).$$

The constrained problem is equivalent to the unconstrained min–max problem

$$\min_{x} \max_{\lambda} \max_{\alpha,\ \alpha \ge 0} L(x, \lambda, \alpha).$$

Why does this work? If x violates a constraint, the inner maximization can drive L to $+\infty$ by choosing a large multiplier, so infeasible points are never chosen. If x is feasible, the best the inner

maximization can do is set the terms to zero (equalities are already 0; for $h^{(j)}(x) \le 0$ the best non-negative α_j makes $\alpha_j h^{(j)}(x) = 0$), so the value is exactly $f(x)$ [1].

The KKT conditions

At a solution x^* (under mild regularity conditions called constraint qualifications), there exist multipliers satisfying the **Karush–Kuhn–Tucker (KKT) conditions** [1], [7]:

1. **Stationarity:** $\nabla_x L(x^*, \lambda^*, \alpha^*) = 0$.
2. **Primal feasibility:** all constraints hold at x^*.
3. **Dual feasibility:** $\alpha^* \ge 0$.
4. **Complementary slackness:** $\alpha_j^* \, h^{(j)}(x^*) = 0$ for every j.

Complementary slackness says each inequality is either **active** ($h^{(j)}(x^*) = 0$: the solution sits on the boundary and the constraint pushes back with $\alpha_j \ge 0$) or **inactive** ($h^{(j)}(x^*) < 0$: the constraint does not matter locally and $\alpha_j = 0$). Geometrically, stationarity says that at an active constraint the downhill direction $-\nabla f$ points straight out of the feasible set, parallel to ∇h; any feasible move would go uphill (Figure 4). For convex problems the KKT conditions are also sufficient for optimality [7].

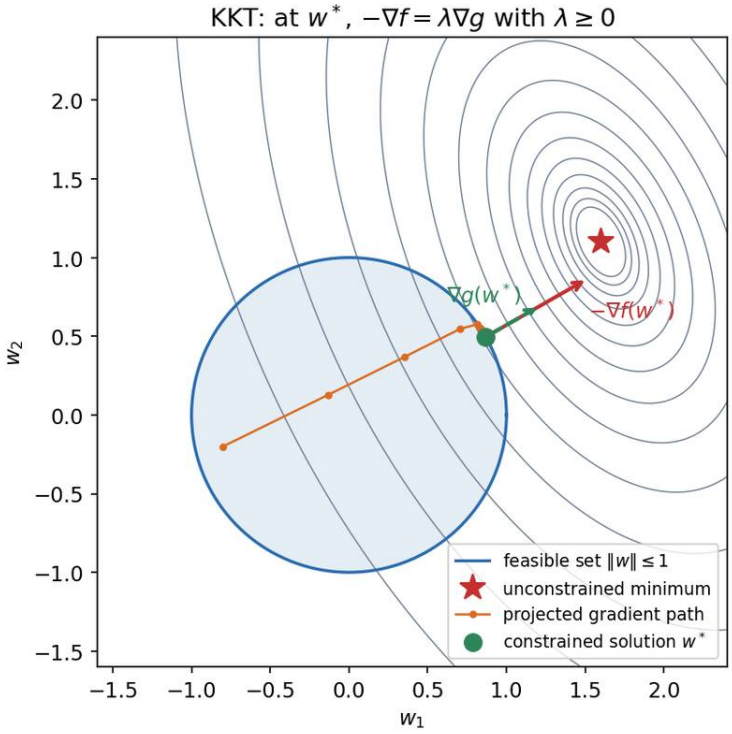


Figure 4 — Minimizing a quadratic over the unit disk. The unconstrained minimum (star) is infeasible. Projected gradient descent stops on the boundary at w, where $-\nabla f$ is parallel to the outward normal ∇g of the constraint: the KKT stationarity condition with a positive multiplier.*

Worked example: least squares with a norm constraint

Following the example in [1], minimize the squared error $f(w) = \|Aw - b\|_2^2$, with $A \in \mathbb{R}^{N \times n}$ and $b \in \mathbb{R}^N$, subject to $h(w) = w^\top w - r^2 \le 0$. The Lagrangian is

$$L(w, \alpha) = \|Aw - b\|_2^2 + \alpha\,(w^\top w - r^2).$$

Stationarity gives $2A^\top(Aw - b) + 2\alpha w = 0$, so

$$w(\alpha) = (A^\top A + \alpha I)^{-1} A^\top b.$$

This is ridge regression with penalty α. If the ordinary least-squares solution $w(0)$ already satisfies $\|w\| \le r$, the constraint is inactive, $\alpha^* = 0$, and we are done. Otherwise the constraint is active, and we increase α until $\|w(\alpha)\| = r$; because $\|w(\alpha)\|$ decreases monotonically in α, a bisection finds it. This is also how the inner maximization over α behaves: the multiplier grows while the constraint is violated and stops when it is just satisfied. The code solves the problem both ways and checks all four KKT conditions.

```
import numpy as np

# minimize ||Aw - b||^2  subject to  ||w||^2 <= r^2
rng = np.random.default_rng(1)
A = rng.normal(size=(30, 5))
b = rng.normal(size=30)
r = 0.2

def w_of(lam):                                  # stationarity of the Lagrangian
    return np.linalg.solve(A.T @ A + lam * np.eye(5), A.T @ b)

w_ls = w_of(0.0)
print("unconstrained norm:", np.linalg.norm(w_ls).round(3))

lo, hi = 0.0, 1e6                               # find lambda with ||w|| = r
for _ in range(200):
    mid = 0.5 * (lo + hi)
    if np.linalg.norm(w_of(mid)) > r:
        lo = mid
    else:
        hi = mid
lam = hi
w = w_of(lam)
print("lambda:", round(lam, 3), " ||w||:", np.linalg.norm(w).round(6))

# KKT checks
stationarity = 2 * A.T @ (A @ w - b) + 2 * lam * w
print("stationarity residual:", np.abs(stationarity).max())
print("primal feasible:", w @ w <= r**2 + 1e-9, " dual feasible:", lam >= 0)
print("complementary slackness:", lam * (w @ w - r**2))

# Projected gradient descent reaches the same point
L = 2 * np.linalg.eigvalsh(A.T @ A).max()     # Lipschitz constant of the gradient
v = np.zeros(5)
```

```
for _ in range(5000):
    v = v - (1 / L) * 2 * A.T @ (A @ v - b)
    n = np.linalg.norm(v)
    if n > r:
        v = v * (r / n)                        # projection onto the ball
print("projected GD matches KKT solution:", np.allclose(v, w, atol=1e-6))
```

The unconstrained solution has norm 0.219, larger than $r = 0.2$, so the constraint is active. Bisection finds $\alpha^* \approx 1.81$, the stationarity residual is about 10^{-15}, complementary slackness holds, and projected gradient descent with step $1/L$ converges to the same vector. Note how L, the Lipschitz constant of the gradient, sets a safe step size, exactly as in the previous section.

Checking derivatives

Plain version. Check that the gradient your code computes matches how the loss actually changes when you nudge the parameters.

A central finite difference estimates a partial derivative,

$$\frac{\partial f}{\partial x_j} \approx \frac{f(x + \epsilon e_j) - f(x - \epsilon e_j)}{2\epsilon},$$

where e_j is the j-th standard basis vector and $\epsilon > 0$ a small step. Its **truncation error** (from ignoring higher-order Taylor terms) is $O(\epsilon^2)$, better than the $O(\epsilon)$ of the forward difference $[f(x + \epsilon e_j) - f(x)]/\epsilon$. But the numerator subtracts two nearly equal numbers, so its **roundoff error** grows like $u\,|f|/\epsilon$. The total error is smallest at an intermediate step — roughly $\epsilon \approx u^{1/3}$ for central differences and $u^{1/2}$ for forward differences, when f and its derivatives have unit scale. Figure 5 shows the resulting V-shaped curves, and why float32 gradient checks are much less precise than float64 ones.

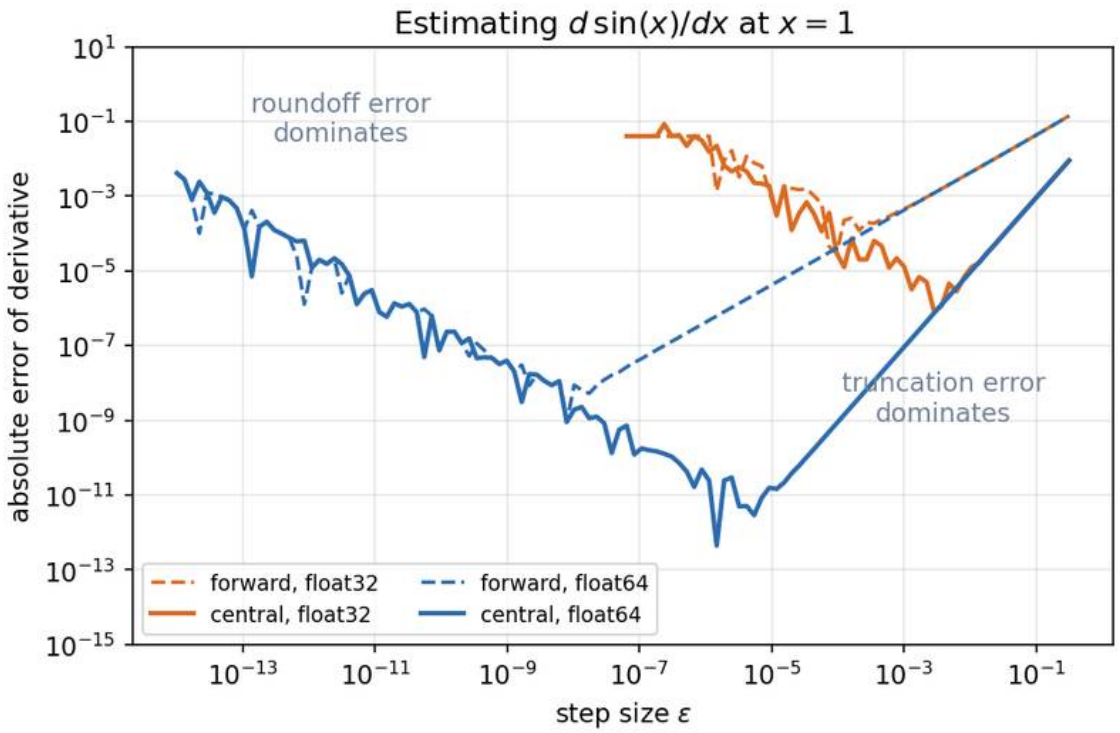


Figure 5 — Error of finite-difference derivatives of $\sin(x)$ at x=1. Right of each minimum, truncation error dominates (slope 1 for forward, 2 for central differences); left of it, roundoff dominates. Central differences bottom out around 1e−6 in float32 but around 1e−11 or lower in float64.

Practical rules: use float64 for checks; try several step sizes; compare with a relative error such as $|a-n|/\max(|a|,|n|,\text{tiny})$; and test at smooth points. ReLU exactly at zero is not differentiable, so there is no single derivative to match there. For many parameters, a **directional check** is cheaper: choose a random unit vector v and compare $\nabla f(w)^\top v$ with a finite difference along v — one pair of function evaluations instead of $2n$.

```
import numpy as np

rng = np.random.default_rng(0)
X = rng.normal(size=(20, 5))
y = rng.normal(size=20)
w = rng.normal(size=5)
v = rng.normal(size=5)
v /= np.linalg.norm(v)
def loss(w):
    return .5*np.mean((X@w-y)**2)
grad = X.T@(X@w-y)/len(y)
eps = 1e-5
numeric = (loss(w+eps*v)-loss(w-eps*v))/(2*eps)
analytic = grad@v
assert np.allclose(numeric, analytic, rtol=1e-6, atol=1e-8)
print("directional derivative:", numeric, analytic)
```

The two numbers agree to about nine significant digits. Remember what a passing check means. Automatic differentiation computes derivatives of the *recorded program*, not of the program you intended [11]. A detached tensor, a wrong reduction, accidental broadcasting or target leakage produces a perfectly differentiable but incorrect objective, and a gradient check will happily confirm its gradient. Check the loss values themselves on cases where you know the answer.

Mixed precision

Plain version. Doing most arithmetic in 16-bit numbers halves memory traffic and uses fast hardware units, but 16 bits are not enough for everything. Keep a few sensitive quantities in 32 bits.

Mixed-precision training, as introduced by Micikevicius et al. [12], combines three ideas:

1. Keep a **float32 master copy of the weights**. An update smaller than about 2^{-11} times the weight it is added to rounds away entirely in float16 (the same effect as $1 + \varepsilon_{\text{mach}}/2 = 1$ above), and many updates are that small.
2. Use **loss scaling** for float16: multiply the loss by a factor S before backpropagation so small gradients do not underflow below float16's smallest subnormal (6×10^{-8}), then divide the gradients by S before the update. Dynamic loss scaling reduces S when an overflow (inf or NaN) appears and skips that step.
3. **Accumulate** dot products and reductions in float32 even when inputs are float16.

bfloat16 has the same 8-bit exponent as float32, so it rarely needs loss scaling; Kalamkar et al. found bfloat16 training reached float32 results across a range of models without changing hyperparameters [5]. The price is precision: with $\varepsilon_{\text{mach}} \approx 0.0078$, bfloat16 cannot represent $1 + 0.003$ distinctly from 1, so long sums and weight updates still need float32 accumulation.

Loss scaling protects small float16 gradients; it cannot repair an invalid objective or an overflow elsewhere in the network (an attention logit or a normalization statistic, for example). Changing precision can also change optimization trajectories and outputs, especially for reductions and long training runs. Validate on the target hardware; a smaller dtype does not guarantee that every operation becomes faster [13].

A debugging order for non-finite values

When a loss becomes `nan` or `inf`, resist the urge to clip everything. Find the *first* operation that produced a non-finite value, then ask which of this chapter's failure modes it is. A useful order:

1. **Inputs:** check shapes, ranges, dtypes and missing values. A single NaN pixel or an un-normalized 0–255 image changes everything downstream.
2. **Forward pass:** check that activations and the loss are finite. Look for `exp`, `log`, division, `sqrt` and normalization layers with near-zero variance.
3. **Loss:** use logits-based losses (`cross_entropy`, `binary_cross_entropy_with_logits`) rather than probabilities followed by `log`.

4. **Gradients:** run a gradient check on a tiny model; inspect per-layer gradient norms.
5. **Updates:** compare the update size with the weight size; a learning rate above the curvature limit $2/\lambda_{\max}$ causes oscillation and divergence even with perfect arithmetic.
6. **Precision:** reproduce in float32 (or float64) before adding mixed precision or compilation.

Overfit a tiny batch first to establish that the pipeline can learn. Gradient-norm clipping bounds the gradient passed to the optimizer; it does not directly bound an adaptive optimizer's update, and it does not make NaN gradients meaningful. ML 02 develops optimizers and normalization in more detail.

Modern view

The core ideas of this chapter are decades old, but they are at the centre of current research because models are larger, training runs are longer, and hardware keeps pushing precision downward.

Optimization theory for large-scale learning. Bottou, Curtis and Nocedal's SIAM Review survey [14] is the standard map of the field. It explains why stochastic gradient methods dominate large-scale machine learning (cheap noisy steps beat expensive exact ones when data are redundant), analyses their convergence under smoothness assumptions like the Lipschitz condition above, and organizes improvements into two families: *noise reduction* (larger or growing batches, variance reduction, iterate averaging) and *second-order* methods (Hessian-free Newton, Gauss–Newton, quasi-Newton, natural gradient and diagonal scaling). Nocedal and Wright [6] and Boyd and Vandenberghe [7] remain the reference textbooks for the deterministic and convex theory, and Baydin et al. [11] survey the automatic differentiation that makes gradients cheap.

What the loss surface actually looks like. Dauphin et al. [8] argued that saddle points, not poor local minima, are the main obstacle in high-dimensional non-convex optimization. Later, Cohen et al. [15] observed that full-batch gradient descent on neural networks typically operates at the "edge of stability": the largest Hessian eigenvalue rises until it hovers just above $2/\eta$ — precisely the classical stability limit derived above — while the loss keeps decreasing over long time scales, though not monotonically. The classical quadratic analysis predicts divergence there, so explaining this regime remains an open theoretical problem.

Curvature-aware optimizers. Full Newton is impossible at scale, but structured approximations are practical. K-FAC [16] approximates the Fisher information matrix (a positive-semidefinite curvature matrix closely related to the Gauss–Newton approximation of the Hessian) with Kronecker products of small per-layer matrices, so inverting it is cheap. Shampoo [17] keeps one preconditioning matrix per tensor dimension. More recently, Muon applies an approximate orthogonalization to each weight matrix's update; Liu et al. [18] report that, with suitable weight decay and per-parameter update scaling, it trains large language models with roughly twice the computational efficiency of their adaptive-optimizer baseline in their experiments. These methods all revisit the question this chapter asks with the Hessian: how should a step be rescaled direction by direction?

Lower precision. The format story continues from float16 with loss scaling [12] and bfloat16 [5] to 8-bit floating point. Micikevicius et al. [19] proposed two FP8 encodings, E4M3 (more

precision) and E5M2 (more range), and showed training results comparable to 16-bit baselines across image and language models. The range–precision trade-off in the table above is exactly what these designs balance, and the IEEE 754 rules [3] are what they relax. Per-tensor or per-block scaling factors play the role that loss scaling played for float16.

Stable softmax at scale. The max-subtraction trick has a streaming version. Milakov and Gimelshein [20] showed that a softmax normalizer can be computed in one pass by keeping a running maximum and rescaling the running sum whenever the maximum changes. FlashAttention [21] builds on this idea: it computes exact attention block by block in fast on-chip memory, never materializing the full attention matrix, and FlashAttention-3 [22] extends it to FP8 with block quantization and "incoherent processing" to reduce quantization error. The core of the online trick fits in a few lines:

```
import numpy as np

def online_logsumexp(chunks):
    m, s = -np.inf, 0.0                 # running max and rescaled running sum
    for c in chunks:
        m_new = max(m, c.max())
        s = s * np.exp(m - m_new) + np.exp(c - m_new).sum()
        m = m_new
    return m + np.log(s)

rng = np.random.default_rng(0)
z = rng.normal(0, 50, size=10_000)
chunks = np.array_split(z, 37)          # seen one block at a time
m = z.max()
reference = m + np.log(np.exp(z - m).sum())
print(online_logsumexp(chunks), reference)
assert np.isclose(online_logsumexp(chunks), reference)
```

Both values print as 174.0918752579 …, identical to the last digit shown, although the online version never sees all logits at once. ML 04 covers attention itself, and ML 10 covers quantization for deployment.

Open problems. Why gradient methods succeed at the edge of stability; how to make low-precision (8-bit and below) training as robust as float32 without hand-tuned scaling; how to make curvature-aware optimizers cheap and robust enough to replace first-order defaults; and how to obtain reproducible results when parallel reductions change summation order.

Key takeaways

- Floating-point numbers have a fixed relative precision ($\varepsilon_{\text{mach}}$) and a fixed range; exponent bits set range, fraction bits set precision.
- Overflow, underflow and cancellation are predictable. Fix them algebraically: subtract the max before `exp`, use `log1p`/`expm1`, use logits-based losses, subtract means before squaring.

- Conditioning is a property of the problem; stability is a property of the algorithm. A condition number of 10^k costs about k digits.
- The Hessian's eigenvalues give curvature. Gradient descent is stable only for $\eta < 2/\lambda_{\max}$ and slows down by the condition number $\lambda_{\max}/\lambda_{\min}$.
- Newton's method rescales by curvature and converges fast near minima, but it is expensive and attracted to saddles; quasi-Newton methods and Hessian-vector products are the practical compromises.
- Constraints can be handled by projection or by the generalized Lagrangian; the KKT conditions characterize the solution.
- Gradient checks need float64, sensible step sizes and smooth test points, and they only test the program you wrote.
- Mixed precision needs float32 master weights and accumulation, and loss scaling for float16; validate rather than assume identical behaviour.

Exercises

1. **Softmax shift.** Prove that softmax is invariant under $z_i \mapsto z_i + c$, and show that with $c = -\max_i z_i$ the denominator lies in $[1, K]$.

   Hint

   The shared factor $\exp(c)$ cancels between numerator and denominator. After the shift, every term $\exp(z_j - m)$ is in $(0, 1]$ and the largest equals 1, so the sum of K terms is between 1 and K. It can therefore neither overflow nor underflow to 0.

2. **Finite-difference sweep.** For $f(x) = e^x$ at $x = 0$, compute forward and central differences for $\epsilon = 10^{-1}, \ldots, 10^{-12}$ in float32 and float64. At which ϵ is each error smallest? Compare with $u^{1/2}$ and $u^{1/3}$.

   Hint

   For float64, $u \approx 1.1 \times 10^{-16}$, so $u^{1/2} \approx 10^{-8}$ (forward) and $u^{1/3} \approx 5 \times 10^{-6}$ (central). For float32, $u \approx 6 \times 10^{-8}$, giving about 2×10^{-4} and 4×10^{-3}. The observed minima should be within an order of magnitude of these, and the error at the minimum is roughly $u^{1/2}$ and $u^{2/3}$ respectively.

3. **Stability limit.** For $f(w) = \frac{1}{2} w^\top H w$ with $H = \mathrm{diag}(2, 50)$, find the largest stable fixed learning rate and the step size that minimizes the worst-case contraction factor. How many iterations does that best step need to reduce the error by a factor of 10^6?

   Hint

   Stability requires $\eta < 2/50 = 0.04$. The best step is $2/(\lambda_{\max} + \lambda_{\min}) = 2/52 \approx 0.0385$, with contraction $(\kappa - 1)/(\kappa + 1) = 24/26 \approx 0.923$. Solve $0.923^k \leq 10^{-6}$: $k \geq \ln 10^6 / (-\ln 0.923) \approx 173$ iterations. A Newton step would need one.

4. **Classify critical points.** Find and classify the critical points of $f(x, y) = x^3 - 3x + y^2$.

   Hint

   $\nabla f = (3x^2 - 3,\ 2y) = 0$ gives $(1, 0)$ and $(-1, 0)$. The Hessian is $\mathrm{diag}(6x,\ 2)$: at $(1, 0)$ it is $\mathrm{diag}(6, 2)$, positive definite, a local minimum; at $(-1, 0)$ it is $\mathrm{diag}(-6, 2)$, indefinite, a saddle. Neither is a global minimum because $f \to -\infty$ as $x \to -\infty$. Starting Newton's method near $(-1, 0)$ converges to the saddle.

5. **KKT on the simplex.** Minimize $f(x) = \frac{1}{2}\|x - c\|_2^2$ subject to $\sum_i x_i = 1$ and $x_i \geq 0$, for $c = (0.8,\ 0.6, -0.4)$. Write the Lagrangian and the KKT conditions, and find the solution.

   Hint

   $L = \frac{1}{2}\|x - c\|^2 + \lambda(\sum_i x_i - 1) - \sum_i \alpha_i x_i$ with $\alpha_i \geq 0$. Stationarity gives $x_i = c_i - \lambda + \alpha_i$; complementary slackness gives $x_i = \max(c_i - \lambda, 0)$. Guess that only the first two are positive: $(0.8 - \lambda) + (0.6 - \lambda) = 1$, so $\lambda = 0.2$ and $x = (0.6, 0.4, 0)$. Check: $c_3 - \lambda = -0.6 < 0$, so $x_3 = 0$ with $\alpha_3 = 0.6 \geq 0$. This is the Euclidean projection onto the probability simplex.

6. **A correct gradient of the wrong loss.** Change a mean reduction into a sum in the gradient-check example. Does the check still pass? Is autograd wrong? What else changes in training?

   Hint

   If you change both the loss and the analytic gradient consistently, the check passes: the derivative is correct for the new objective. Nothing is wrong with autograd. But the gradient is now N times larger, so the largest stable learning rate shrinks by N, and in a multi-term loss the balance between terms changes. A gradient check cannot tell you that you chose the wrong objective.

References

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016, ch. 4 (Numerical Computation) and ch. 8. Book
2. D. Goldberg, "What Every Computer Scientist Should Know About Floating-Point Arithmetic," *ACM Computing Surveys*, vol. 23, no. 1, pp. 5–48, 1991. DOI
3. IEEE, "IEEE Standard for Floating-Point Arithmetic," IEEE Std 754-2019, 2019. DOI
4. N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed., SIAM, 2002. DOI
5. D. Kalamkar et al., "A Study of BFLOAT16 for Deep Learning Training," arXiv:1905.12322, 2019. arXiv
6. J. Nocedal and S. J. Wright, *Numerical Optimization*, 2nd ed., Springer, 2006. DOI
7. S. Boyd and L. Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004. Book
8. Y. N. Dauphin, R. Pascanu, C. Gulcehre, K. Cho, S. Ganguli and Y. Bengio, "Identifying and Attacking the Saddle Point Problem in High-Dimensional Non-Convex Optimization," *NeurIPS*, 2014. Paper

9. B. A. Pearlmutter, “Fast Exact Multiplication by the Hessian,” *Neural Computation*, vol. 6, no. 1, pp. 147–160, 1994. DOI
10. D. C. Liu and J. Nocedal, “On the Limited Memory BFGS Method for Large Scale Optimization,” *Mathematical Programming*, vol. 45, pp. 503–528, 1989. DOI
11. A. G. Baydin, B. A. Pearlmutter, A. A. Radul and J. M. Siskind, “Automatic Differentiation in Machine Learning: a Survey,” *Journal of Machine Learning Research*, vol. 18, no. 153, pp. 1–43, 2018. arXiv
12. P. Micikevicius et al., “Mixed Precision Training,” *ICLR*, 2018. arXiv
13. PyTorch documentation, “Numerical Accuracy.” Documentation
14. L. Bottou, F. E. Curtis and J. Nocedal, “Optimization Methods for Large-Scale Machine Learning,” *SIAM Review*, vol. 60, no. 2, pp. 223–311, 2018. DOI
15. J. M. Cohen, S. Kaur, Y. Li, J. Z. Kolter and A. Talwalkar, “Gradient Descent on Neural Networks Typically Occurs at the Edge of Stability,” *ICLR*, 2021. arXiv
16. J. Martens and R. Grosse, “Optimizing Neural Networks with Kronecker-factored Approximate Curvature,” *ICML*, 2015. Paper
17. V. Gupta, T. Koren and Y. Singer, “Shampoo: Preconditioned Stochastic Tensor Optimization,” *ICML*, 2018. Paper
18. J. Liu, J. Su, X. Yao et al., “Muon is Scalable for LLM Training,” arXiv:2502.16982, 2025. arXiv
19. P. Micikevicius et al., “FP8 Formats for Deep Learning,” arXiv:2209.05433, 2022. arXiv
20. M. Milakov and N. Gimelshein, “Online Normalizer Calculation for Softmax,” arXiv:1805.02867, 2018. arXiv
21. T. Dao, D. Y. Fu, S. Ermon, A. Rudra and C. Ré, “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” *NeurIPS*, 2022. Paper
22. J. Shah, G. Bikshandi, Y. Zhang, V. Thakkar, P. Ramani and T. Dao, “FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision,” *NeurIPS*, 2024. Paper

ML 02 • Neural Networks from Perceptron to Deep Networks

From a single perceptron to deep networks: activations, normalization, output layers and losses, backpropagation, optimizers, and the habits that make a network train and generalize.

Online edition: https://blog.shyandram.dev/en/learn/ml/ml-02-neural-networks/

What you'll learn

- What a perceptron is, how its learning rule works, and exactly how it differs from logistic regression.
- How stacking layers gives a multilayer perceptron (MLP), what the universal approximation theorem does and does not promise, and what "deep" really means.
- The activation functions in use today (sigmoid, tanh, softmax, ReLU, Leaky ReLU, GELU, SiLU/Swish, SwiGLU) and the normalization layers (BatchNorm, LayerNorm, RMSNorm), with their formulas and derivatives.
- How to pair the output layer with the right loss, and how backpropagation computes every gradient, derived by hand for a two-layer network and checked numerically in NumPy.
- How to train in practice: initialization, SGD, momentum, Adam and AdamW, learning-rate schedules, batch size, gradient clipping, and how to read a loss curve.
- How to diagnose underfitting and overfitting, the regularization tools that fix overfitting, and how residual connections lead to the Transformer.

The big picture

A neural network is a function with many adjustable knobs (its **parameters**). It takes an input, such as the pixels of an image or two coordinates of a point, and produces an output, such as a class label. Training means turning the knobs, a tiny amount at a time, so the outputs on known examples get closer to the right answers. Everything in this tutorial is about one of three questions: what shape the function has (the architecture), how to measure "closer" (the loss), and how to turn the knobs well (the optimization).

> **Explain like I'm five**
>
> Think of a team of people passing notes down a line. The first person looks at a picture and writes a few simple observations ("there is a curve here", "this part is bright"). The next person reads those notes and writes slightly smarter ones ("this looks like an eye"). The last person says "cat" or "dog". When the answer is wrong, a teacher walks back up the line telling each person how much their note was to blame, and each one changes their habits a little. That walk backward is **backpropagation**, and doing it many thousands of times is **training**.

Why it matters: almost every modern system in vision, speech and language is built from the pieces in this tutorial. Convolutional networks, Transformers and diffusion models all use the same ingredients: linear layers, nonlinear activations, normalization, residual connections, a loss, backpropagation and an adaptive optimizer. Once these are clear, the deep-learning track is about arranging them for images.

About this tutorial

This tutorial is expanded from the author's earlier study notes on machine learning and neural networks [1], which are written in Traditional Chinese: 機器學習及類神經網路筆記. It assumes the ideas of ML 01: data, features, a loss, and gradient descent.

The perceptron

Plain version. A perceptron is the simplest artificial neuron. It multiplies each input by a weight, adds them up with a bias, and answers "yes" if the total is positive and "no" otherwise. It learns by nudging its weights only when it gets an example wrong.

The model

Rosenblatt introduced the perceptron in 1958 as a model of how a brain might store and recognize patterns [4]. For an input vector $\mathbf{x} \in \mathbb{R}^d$ it computes

$$z = \mathbf{w}^\top \mathbf{x} + b, \qquad \hat{y} = \mathbf{1}[z > 0],$$

where $\mathbf{w} \in \mathbb{R}^d$ is the weight vector, $b \in \mathbb{R}$ is the bias, z is the **pre-activation** (a weighted sum), $\mathbf{1}[\cdot]$ is the indicator that equals 1 when its condition is true and 0 otherwise, and $\hat{y} \in \{0, 1\}$ is the predicted class. The indicator is the **step function**. Geometrically, $\mathbf{w}^\top \mathbf{x} + b = 0$ is a hyperplane (a line in 2-D), and the perceptron labels points by which side of it they fall on.

The perceptron learning rule

Go through the training examples $(\mathbf{x}_i, y_i)$ one at a time. For each one, compute $\hat{y}_i$ and update

$$\mathbf{w} \leftarrow \mathbf{w} + \eta\,(y_i - \hat{y}_i)\,\mathbf{x}_i, \qquad b \leftarrow b + \eta\,(y_i - \hat{y}_i),$$

where $\eta > 0$ is the **learning rate** (step size) and $y_i \in \{0, 1\}$ is the true label. If the prediction is right, $y_i - \hat{y}_i = 0$ and nothing changes. If the perceptron said 0 but the answer was 1, the weights move *toward* $\mathbf{x}_i$, which raises z for this input next time; the opposite mistake moves them away. When the two classes can be separated by a hyperplane (the data are **linearly separable**), this rule stops making mistakes after a finite number of updates. When they cannot, it never settles.

```
import numpy as np
```

```
def perceptron_train(X, y, epochs=20, eta=1.0):
    """X: (N, d), y in {0, 1}. Returns w, b."""
    w, b = np.zeros(X.shape[1]), 0.0
    for _ in range(epochs):
        mistakes = 0
        for x_i, y_i in zip(X, y):
            y_hat = float(w @ x_i + b > 0)       # step function
            if y_hat != y_i:                     # update only on a mistake
                w += eta * (y_i - y_hat) * x_i
                b += eta * (y_i - y_hat)
                mistakes += 1
        if mistakes == 0:
            break
    return w, b

# AND is linearly separable; XOR is not
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]], dtype=float)
for name, y in [("AND", [0, 0, 0, 1]), ("XOR", [0, 1, 1, 0])]:
    w, b = perceptron_train(X, np.array(y, float))
    print(name, (X @ w + b > 0).astype(int))
# AND [0 0 0 1]
# XOR [1 1 0 0]   <- wrong: no single line can do it
```

Perceptron versus logistic regression

The two models are easy to confuse because they share the same first step, $z = \mathbf{w}^\top \mathbf{x} + b$. They are different methods:

	Perceptron	Logistic regression
Output function	step: $\hat{y} = \mathbf{1}[z > 0]$	sigmoid: $\hat{p} = \sigma(z) = \dfrac{1}{1+e^{-z}}$
Output meaning	a hard 0/1 decision	a probability $P(y = 1 \mid \mathbf{x})$
Training	perceptron rule, updates only on mistakes	minimize cross-entropy (maximum likelihood) with gradient descent
Non-separable data	never converges	converges to a well-defined best fit

The gradient of the logistic-regression loss for one example is $(\hat{p}_i - y_i)\,\mathbf{x}_i$, so its gradient-descent step is $\mathbf{w} \leftarrow \mathbf{w} + \eta\,(y_i - \hat{p}_i)\,\mathbf{x}_i$. It looks almost identical to the perceptron rule, but $\hat{p}_i$ is a probability between 0 and 1, so *every* example contributes, and confidently correct ones contribute very little. That smoothness is what makes logistic regression trainable by gradient descent and what lets it be stacked into larger differentiable networks. The step function, in contrast, has derivative zero almost everywhere, so gradients cannot flow through it.

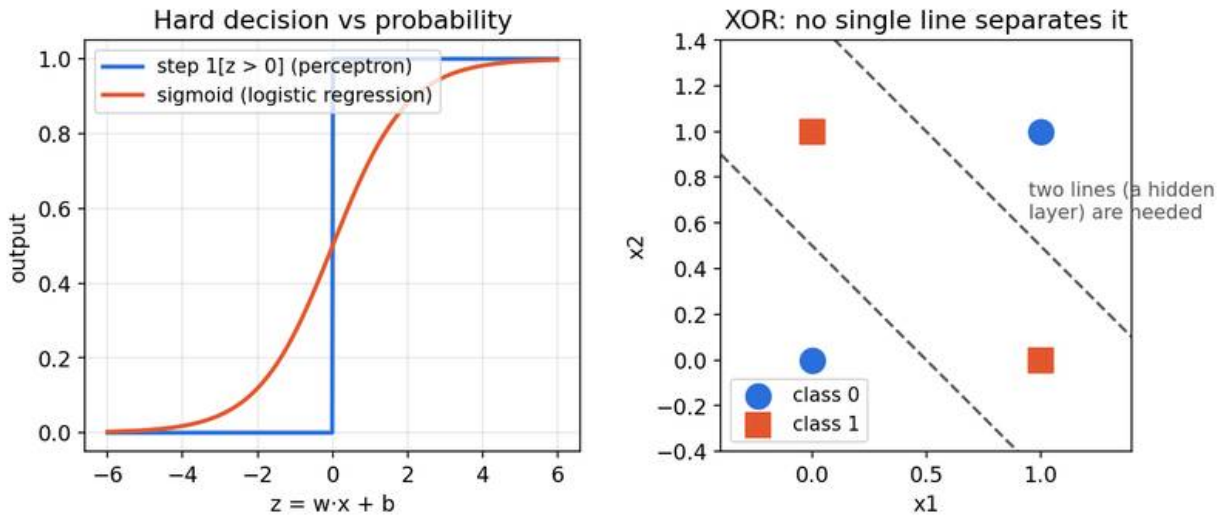


Figure 1 — Left: the perceptron's step output versus logistic regression's sigmoid probability. Right: XOR, the classic problem that no single linear boundary can solve; two boundaries (a hidden layer) are needed.

The multilayer perceptron (MLP)

Plain version. One neuron draws one straight line. Put many neurons side by side to get many lines, then feed their outputs into another layer that combines them. The combination can bend, so the network can carve out curved regions.

Stacking layers

An MLP with one hidden layer computes

$$\mathbf{h} = f(W_1\mathbf{x} + \mathbf{b}_1)\,,$$
$$\hat{\mathbf{y}} = g(W_2\mathbf{h} + \mathbf{b}_2)\,,$$

where $\mathbf{x} \in \mathbb{R}^d$ is the input, $W_1 \in \mathbb{R}^{m\times d}$ and $\mathbf{b}_1 \in \mathbb{R}^m$ are the first layer's weights and biases, m is the **width** (number of hidden units), f is a nonlinear **activation function** applied element-wise, $\mathbf{h} \in \mathbb{R}^m$ is the hidden representation, $W_2 \in \mathbb{R}^{K\times m}$ and $\mathbf{b}_2 \in \mathbb{R}^K$ map it to K outputs, and g is the output function chosen for the task (see below). Each hidden unit is one "logistic-regression-like" neuron; the name multilayer *perceptron* is historical, since modern MLPs use smooth activations, not the step function.

The nonlinearity f is essential. Without it,

$W_2(W_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (W_2W_1)\mathbf{x} + (W_2\mathbf{b}_1 + \mathbf{b}_2)$ is again a single linear map: any number of linear layers collapses to one, and the network can still only draw one hyperplane. For XOR, two hidden units can each draw one of the dashed lines in Figure 1, and the output unit then answers "between the two lines".

Universal approximation

Cybenko proved in 1989 that a network with **one** hidden layer of sigmoidal units can approximate any continuous function on a bounded box (the unit hypercube) to any desired accuracy, provided the hidden layer is wide enough [5]. Later work extended the result to other activations, including ReLU. The theorem is reassuring, but read it carefully:

- It is about **existence**. It says suitable weights exist; it says nothing about whether gradient descent will find them.
- It says nothing about **how wide** "wide enough" is. For some functions the required width grows extremely fast with the input dimension.
- It says nothing about **generalization**: fitting the training points well does not mean predicting new points well.

So the theorem explains why MLPs are flexible enough in principle. The rest of this tutorial is about the practical questions it leaves open.

Deep neural networks: depth and width

Plain version. "Deep" just means many layers stacked in sequence. There is no official number of layers at which a network becomes deep.

A **deep neural network** (DNN) is an MLP, or any other layered network, with several hidden layers: $\mathbf{h}_\ell = f(W_\ell \mathbf{h}_{\ell-1} + \mathbf{b}_\ell)$ for $\ell = 1, \dots, L$, with $\mathbf{h}_0 = \mathbf{x}$. There is no agreed threshold for "deep". A common informal usage calls any network with more than one hidden layer deep, while other texts count total layers or hidden layers differently, so "three layers" means different things in different books. What matters is the idea, not the count: a deep network builds its answer as a **composition** of many simple steps.

The two ways to add capacity are **width** (more units per layer) and **depth** (more layers). They are not equivalent. A wider layer adds more pieces that are combined once; a deeper network re-uses the features of earlier layers to build more abstract ones. The review by LeCun, Bengio and Hinton [2] explains the success of deep learning through exactly this point: each layer turns the representation from the previous layer into a slightly more abstract one, and composing many such layers lets the network learn very intricate functions from raw data, with the features learned rather than hand-designed.

Historically, the 1990s were dominated by **shallow** methods. Support vector machines, a kernel method rather than a wide neural network, were the main tool, and deep networks were hard to train. From the mid-2000s onward, more data, GPUs and better training techniques made depth practical, and deep networks began to win benchmark after benchmark [2]. Depth alone is not the whole story, though: simply stacking more plain layers eventually makes training *worse*, a problem that residual connections solved (see the last main section).

Figure 2 shows width and depth at work on the two-moons dataset from scikit-learn. With two or eight hidden units in one layer, the boundary is made of only a few straight pieces and cannot

follow the moons. Wider or deeper networks bend the boundary to fit; the widest and deepest one even wraps around single noisy points, a first hint of overfitting.

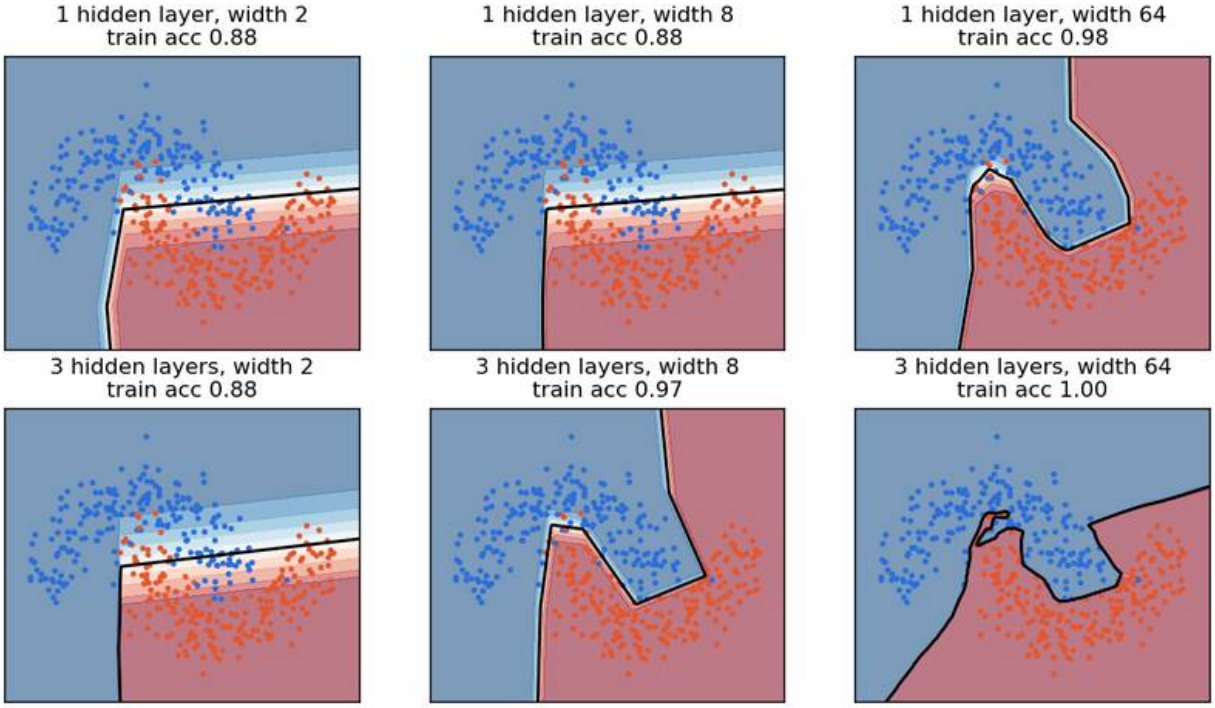


Figure 2 — Decision boundaries of ReLU MLPs on make_moons (400 points, noise 0.2) as width (columns) and depth (rows) change, each trained with Adam for 1,500 full-batch steps. The black curve is the 0.5 probability contour.

Activation functions

Plain version. The activation function is the "bend" applied after each weighted sum. Without it, the whole network would be a straight-line model. Different bends trade off smoothness, speed and how well gradients pass through many layers.

Figure 3 plots the common choices and their derivatives. The derivative matters as much as the function, because backpropagation multiplies derivatives layer after layer (see the training section).

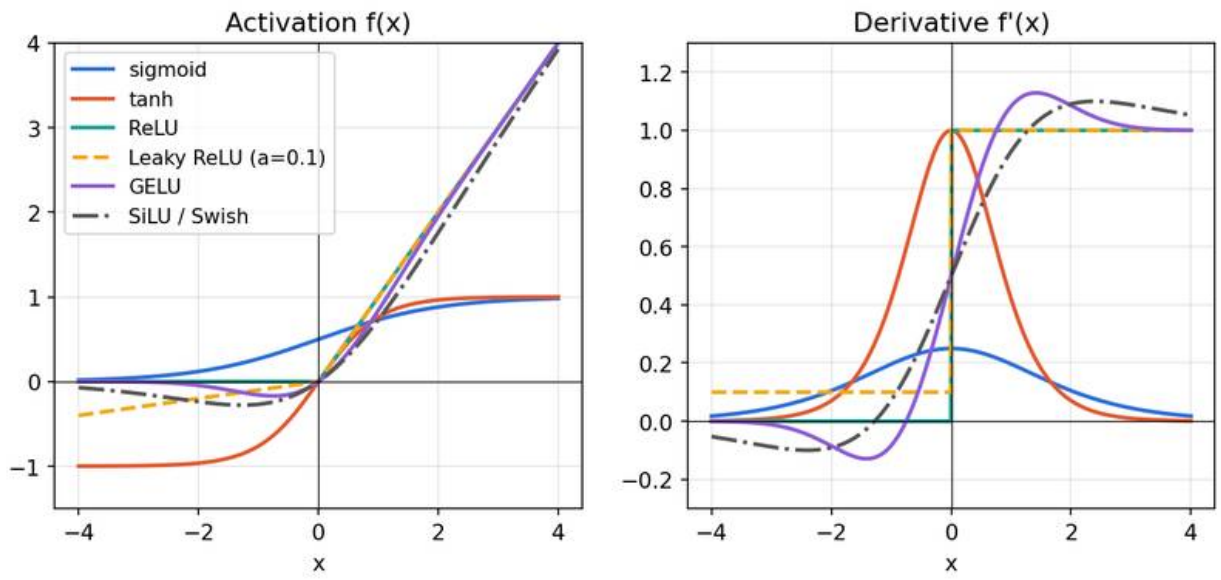


Figure 3 — Common activation functions (left) and their derivatives (right). Note how small the sigmoid's derivative is everywhere (at most 0.25) and how flat both sigmoid and tanh become for large inputs.

Sigmoid and tanh

$$\sigma(x) = \frac{1}{1+e^{-x}}, \quad \sigma'(x) = \sigma(x)(1-\sigma(x)); \qquad \tanh(x) = 2\sigma(2x) - 1, \quad \tanh'(x) = 1 - \tanh^2(x).$$

The sigmoid maps any real number to $(0, 1)$; tanh maps it to $(-1, 1)$ and is centered at zero. Both **saturate**: for large $|x|$ the curve is flat and the derivative is nearly zero, so almost no gradient passes through. The sigmoid's derivative is at most $1/4$ (at $x = 0$). These two facts made deep sigmoid networks hard to train, which Glorot and Bengio analyzed in detail [17]. Today sigmoid is used mainly at the **output** of binary classifiers and inside gates (as in SwiGLU below), and tanh in some recurrent networks.

Softmax

Softmax turns a vector of K real scores $\mathbf{z}$ (called **logits**) into a probability distribution:

$$p_k = \text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad \frac{\partial p_k}{\partial z_j} = p_k(\delta_{kj} - p_j),$$

where δ_{kj} is 1 if $k = j$ and 0 otherwise. Each p_k is positive and they sum to 1. Unlike the others in this section, softmax acts on a **whole vector**, not element by element, so it is used almost only in the output layer of multi-class classifiers (and inside attention), not as a hidden activation. In code, subtract $\max_j z_j$ before exponentiating; it does not change the result and avoids overflow.

ReLU and Leaky ReLU

$$\mathrm{ReLU}(x) = \max(0, x), \quad \mathrm{ReLU}'(x) = \mathbf{1}[x > 0]; \qquad \mathrm{LeakyReLU}_a(x) = \max(ax, x), \quad 0 < a < 1.$$

The rectified linear unit passes positive inputs unchanged and zeroes the rest. Its derivative is exactly 1 for all positive inputs, so gradients do not shrink as they pass through active units, and it is very cheap to compute. ReLU was one of the changes that made deep supervised networks trainable without unsupervised pre-training [2]. Its weakness is the **dying ReLU**: a unit whose input is negative for every example has zero gradient and stops learning. Leaky ReLU keeps a small slope a (for example 0.01) on the negative side so some gradient always flows. The activation survey by Dubey, Singh and Chaudhuri [7] groups these and many later variants into a ReLU-based family.

GELU and SiLU/Swish

These are smooth versions of ReLU that let small negative values through:

$$\mathrm{GELU}(x) = x\,\Phi(x), \qquad \mathrm{GELU}'(x) = \Phi(x) + x\,\varphi(x),$$

where Φ is the standard normal cumulative distribution function and φ its density [8]. Instead of gating by sign as ReLU does, GELU weights the input by how large it is: $\Phi(x)$ is the probability that a standard normal variable is below x. A common fast approximation is $0.5x(1 + \tanh[\sqrt{2/\pi}\,(x + 0.044715x^3)])$ [8]. GELU is the default activation in many Transformer models.

$$\mathrm{Swish}_\beta(x) = x\,\sigma(\beta x), \qquad \mathrm{SiLU}(x) = x\,\sigma(x), \qquad \mathrm{SiLU}'(x) = \sigma(x)(1 + x(1 - \sigma(x))).$$

The sigmoid-weighted linear unit (SiLU) was proposed by Elfwing, Uchibe and Doya for reinforcement learning [9]. Ramachandran, Zoph and Le found the same shape with a learnable or fixed β by automated search and called it Swish [10]; with $\beta = 1$, Swish is SiLU. GELU and SiLU look very similar (Figure 3). Both are non-monotonic: they dip slightly below zero for moderate negative inputs.

SwiGLU: a gated feed-forward block

A **gated linear unit** (GLU) multiplies two linear projections element-wise, one of them passed through a nonlinearity that acts like a gate. Shazeer tested several variants in the Transformer's feed-forward block [11]. The SwiGLU version, written in the paper's row-vector notation and without biases, is

$$\mathrm{FFN}_{\mathrm{SwiGLU}}(\mathbf{x}) = (\,\mathrm{Swish}_1(\mathbf{x}W) \odot \mathbf{x}V)\,W_2,$$

where $\mathbf{x}$ is a row vector of size d_{model}, W and V are $d_{\text{model}} \times d_{\text{ff}}$ matrices, W_2 is $d_{\text{ff}} \times d_{\text{model}}$, and $\odot$ is element-wise multiplication. In Shazeer's experiments the GLU variants, SwiGLU and GEGLU among them, gave better results than the ReLU and GELU versions of the block. Because SwiGLU has three weight matrices instead of two, d_{ff} is reduced by a factor of $2/3$ to keep the parameter

count equal [11]. LLaMA, for example, replaced ReLU with SwiGLU using a hidden size of $\frac{2}{3} \cdot 4d$ [12], and SwiGLU has become a common default in large language models.

Activation	Range	Smooth	Typical use today
sigmoid	$(0, 1)$	yes	binary / multi-label output, gates
tanh	$(-1, 1)$	yes	some recurrent networks
softmax	probability vector	yes	multi-class output, attention weights
ReLU / Leaky ReLU	$[0, \infty)$ / $\mathbb{R}$	no (kink at 0)	CNNs, MLPs, the safe default
GELU, SiLU	about $[-0.17, \infty)$, $[-0.28, \infty)$	yes	Transformers, modern CNNs
SwiGLU	$\mathbb{R}$	yes	Transformer feed-forward blocks

Normalization layers

Plain version. As training goes on, the numbers flowing through a network can drift to very large or very small scales, which makes learning unstable. A normalization layer rescales them to a standard size at each step, then lets the network choose the final scale it prefers.

All three common layers share one template. For a set of values $x_1, \ldots, x_n$,

$$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i, \qquad \sigma^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2, \qquad y_i = \gamma\,\frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta,$$

where μ and σ^2 are the mean and variance, ϵ is a small constant for numerical safety, and γ, β are a learnable scale and shift (so the layer can undo the normalization if that helps). The layers differ in **which values** form the set (Figure 4).

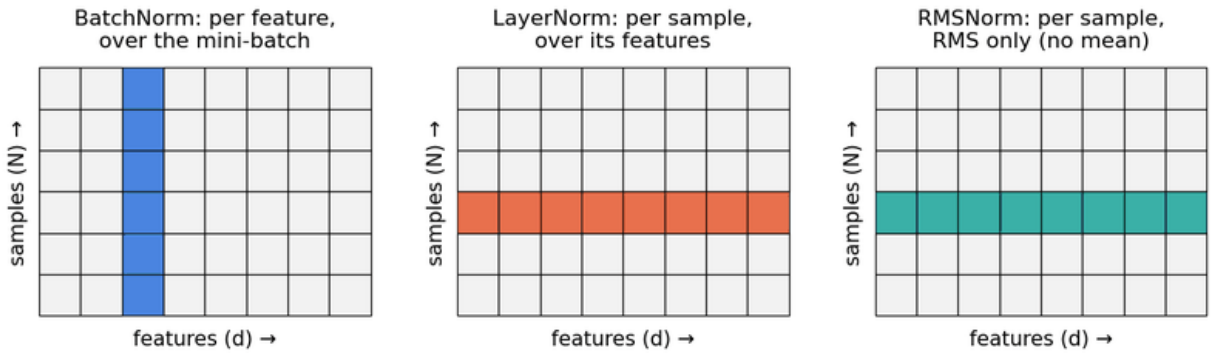


Figure 4 — Which entries share statistics. BatchNorm normalizes each feature over the mini-batch; LayerNorm and RMSNorm normalize each sample over its own features.

- **BatchNorm** (Ioffe and Szegedy [13]) computes μ, σ^2 for each feature (each channel, in a CNN) **across the mini-batch**. It speeds up training and became standard in CNNs. Because the statistics depend on the batch, it behaves differently in training and inference: at inference it uses running averages collected during training. It also degrades when batches are very small, since the batch statistics become noisy.
- **LayerNorm** (Ba, Kiros and Hinton [14]) computes μ, σ^2 **over the features of a single sample**. It does not depend on the batch, behaves identically at training and test time, and works naturally for sequences. It is the standard normalization in Transformers.
- **RMSNorm** (Zhang and Sennrich [15]) drops the mean subtraction and divides by the root mean square, $y_i = \gamma_i \, x_i / \sqrt{\frac{1}{n}\sum_j x_j^2 + \epsilon}$. The authors hypothesize that LayerNorm's re-centering is dispensable and report comparable quality with lower running time [15]. LLaMA uses RMSNorm and applies it to the **input** of each Transformer sub-layer ("pre-normalization") rather than to its output [12].

```
import torch

x = torch.randn(4, 8) * 5 + 3                       # 4 samples, 8 features
ln = torch.nn.LayerNorm(8, elementwise_affine=False)
bn = torch.nn.BatchNorm1d(8, affine=False)
rms = x / torch.sqrt(x.pow(2).mean(dim=1, keepdim=True) + 1e-6)

print(ln(x).mean(dim=1))     # ~0 for every sample (row)
print(bn(x).mean(dim=0))     # ~0 for every feature (column), in train mode
print(rms.pow(2).mean(dim=1))  # ~1 for every sample: unit RMS, mean not removed
```

Huang et al. [16] survey this family and decompose any normalization method into three choices: which values are pooled together (the *normalization area*), what operation is applied (standardizing, centering, scaling or whitening), and how the representation is recovered afterward (the learnable affine step). BatchNorm, LayerNorm and RMSNorm differ mainly in the first two.

Designing the output layer and choosing the loss

Plain version. The last layer must speak the language of the answer: any real number for a quantity, a probability for yes/no, a probability distribution for "which of K". Each kind of output has a natural loss, and the pairs below are the ones to use.

Task	Output layer	Output range	Loss
Regression	linear (no activation)	$\mathbb{R}$	mean squared error (MSE, L2) or mean absolute error (MAE, L1)
Binary classification	1 unit + sigmoid	$(0, 1)$	binary cross-entropy
Multi-label (several yes/no)	K units + sigmoid each	$(0, 1)^K$	binary cross-entropy per label

Task	Output layer	Output range	Loss
Multi-class (exactly one of K)	K units + softmax	probability vector	cross-entropy

A common mistake is to put a sigmoid on a regression output. The sigmoid's range is the open interval $(0, 1)$, and its job is to output a probability for binary or multi-label classification. A regression head is normally **linear**, so it can produce any real value. A sigmoid on a regression output is reasonable only when the target itself is known to lie between 0 and 1, such as a normalized pixel intensity or a fraction.

The losses, for one example, are

$$\begin{aligned} \text{MSE:} \quad & \ell = \tfrac{1}{2}\|\hat{\mathbf{y}} - \mathbf{y}\|^2, \\ \text{binary cross-entropy:} \quad & \ell = -[\, y \log \hat{p} + (1-y)\log(1-\hat{p}) \,], \\ \text{cross-entropy:} \quad & \ell = -\sum_{k=1}^{K} y_k \log p_k = -\log p_c, \end{aligned}$$

where $\mathbf{y}$ is the target, $\hat{p} = \sigma(z)$, $\mathbf{p} = \text{softmax}(\mathbf{z})$, y_k is the one-hot label and c is the correct class. (The factor $\frac{1}{2}$ in MSE is a convenience; PyTorch's `MSELoss` averages without it.) MAE, $\|\hat{\mathbf{y}} - \mathbf{y}\|_1$, is less sensitive to outliers than MSE. Each pair is a maximum-likelihood estimate: MSE corresponds to Gaussian noise on the target, binary cross-entropy to a Bernoulli label, cross-entropy to a categorical label [3].

The pairs are also chosen for a beautiful practical reason. In all three cases the gradient of the loss with respect to the **pre-activation** of the output layer is simply

$$\frac{\partial \ell}{\partial \mathbf{z}} = \hat{\mathbf{y}} - \mathbf{y} \quad \text{(prediction minus target)},$$

with $\hat{\mathbf{y}}$ meaning the linear output, $\sigma(z)$ or $\text{softmax}(\mathbf{z})$ respectively. The saturating sigmoid or softmax derivative cancels out, so a confidently wrong prediction still produces a large gradient. (Exercise 3 asks you to verify this for softmax.) This is why frameworks fuse the last activation into the loss: PyTorch's `nn.CrossEntropyLoss` and `nn.BCEWithLogitsLoss` take **logits**, so the model itself must not end with a softmax or sigmoid during training.

Training I: the forward pass and backpropagation

Plain version. The forward pass computes the prediction and the loss, layer by layer from input to output. Backpropagation then walks back from the loss to the input, using the chain rule to find how much each weight contributed to the error. It reuses intermediate results, so all gradients cost about as much as one extra forward pass.

Backpropagation was popularized for training multi-layer networks by Rumelhart, Hinton and Williams [6]. It is nothing more than the chain rule of calculus, organized so that no work is repeated. Figure 5 shows the computation graph for the network we will differentiate.

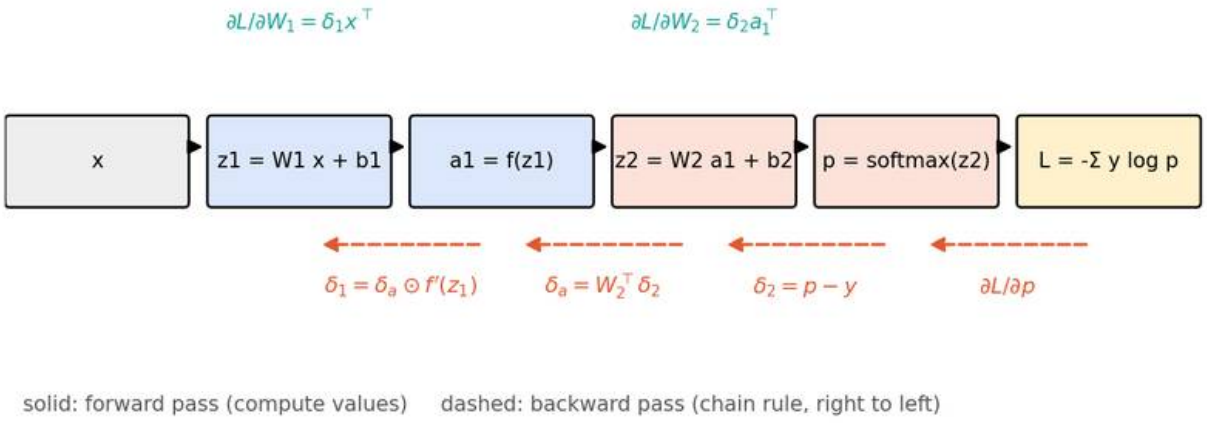


Figure 5 — Computation graph of a two-layer MLP with softmax cross-entropy. Solid arrows: forward pass. Dashed arrows: the backward pass, which carries the error signal δ from the loss back toward the input.

Derivation for a two-layer MLP

Take one example $(\mathbf{x}, \mathbf{y})$ with $\mathbf{x} \in \mathbb{R}^d$ and a one-hot label $\mathbf{y} \in \{0,1\}^K$. The forward pass is

$$\begin{aligned} \mathbf{z}_1 &= W_1\mathbf{x} + \mathbf{b}_1 \in \mathbb{R}^m, \quad \mathbf{a}_1 = f(\mathbf{z}_1) \in \mathbb{R}^m, \\ \mathbf{z}_2 &= W_2\mathbf{a}_1 + \mathbf{b}_2 \in \mathbb{R}^K, \quad \mathbf{p} = \mathrm{softmax}(\mathbf{z}_2), \qquad L = -\textstyle\sum_k y_k \log p_k, \end{aligned}$$

with $W_1 \in \mathbb{R}^{m\times d}$, $W_2 \in \mathbb{R}^{K\times m}$, and f an element-wise activation. Define the **error signal** of a layer as the gradient of the loss with respect to its pre-activation, $\delta_\ell = \partial L/\partial \mathbf{z}_\ell$.

Step 1, output layer. From the previous section, softmax with cross-entropy gives

$$\delta_2 = \frac{\partial L}{\partial \mathbf{z}_2} = \mathbf{p} - \mathbf{y} \in \mathbb{R}^K.$$

Step 2, output weights. Since $z_{2,k} = \sum_j W_{2,kj}\, a_{1,j} + b_{2,k}$, we have $\partial z_{2,k}/\partial W_{2,kj} = a_{1,j}$, so

$$\frac{\partial L}{\partial W_2} = \delta_2\, \mathbf{a}_1^\top \in \mathbb{R}^{K\times m}, \qquad \frac{\partial L}{\partial \mathbf{b}_2} = \delta_2.$$

The gradient of a weight is (error at its output) times (activation at its input). The shapes match W_2, which is a useful check.

Step 3, back through the linear layer. Each hidden activation $a_{1,j}$ affects every output, so its gradient sums over them: $\partial L/\partial a_{1,j} = \sum_k \delta_{2,k} W_{2,kj}$, or in matrix form

$$\frac{\partial L}{\partial \mathbf{a}_1} = W_2^\top \delta_2 \in \mathbb{R}^m.$$

Step 4, back through the activation. Because f acts element by element, its Jacobian is diagonal, and the chain rule becomes an element-wise product:

$$\delta_1 = \frac{\partial L}{\partial \mathbf{z}_1} = (W_2^\top \delta_2) \odot f'(\mathbf{z}_1).$$

Step 5, first-layer weights. Exactly as in Step 2,

$$\frac{\partial L}{\partial W_1} = \delta_1 \, \mathbf{x}^\top \in \mathbb{R}^{m \times d}, \qquad \frac{\partial L}{\partial \mathbf{b}_1} = \delta_1.$$

For a deeper network, Steps 3 to 5 simply repeat:

$\delta_\ell = (W_{\ell+1}^\top \delta_{\ell+1}) \odot f'(\mathbf{z}_\ell)$ and $\partial L / \partial W_\ell = \delta_\ell \, \mathbf{a}_{\ell-1}^\top$. For a mini-batch of N examples, the loss is the average, so each gradient is the average of the per-example gradients.

From scratch in NumPy, with a gradient check

The code below stores examples as **rows** (`X` has shape $N \times d$), which is the usual convention in code. Then `X @ W1` with `W1` of shape $d \times m$ plays the role of $W_1 \mathbf{x}$ in the math, and every formula above appears transposed: for example `dW2 = A1.T @ dZ2` is $\delta_2 \mathbf{a}_1^\top$ stacked over the batch. The **gradient check** compares one analytic derivative with a central finite difference, $(L(w+\epsilon) - L(w-\epsilon))/2\epsilon$. If they agree to many digits, the backward pass is correct. Always do this when you write gradients by hand.

```
import numpy as np
from sklearn.datasets import make_moons

rng = np.random.default_rng(0)
X, y = make_moons(n_samples=200, noise=0.2, random_state=0)
N, d, h, K = X.shape[0], 2, 16, 2
Y = np.eye(K)[y]                                  # one-hot targets, (N, K)

# He-style initialization (std = sqrt(2 / fan_in))
W1 = rng.normal(0, np.sqrt(2 / d), (d, h)); b1 = np.zeros(h)
W2 = rng.normal(0, np.sqrt(2 / h), (h, K)); b2 = np.zeros(K)

def forward(X, W1, b1, W2, b2):
    Z1 = X @ W1 + b1                              # (N, h)
    A1 = np.maximum(Z1, 0)                        # ReLU
    Z2 = A1 @ W2 + b2                             # (N, K) logits
    Z2 = Z2 - Z2.max(axis=1, keepdims=True)       # numerical stability
    P = np.exp(Z2) / np.exp(Z2).sum(axis=1, keepdims=True)
    loss = -np.mean(np.sum(Y * np.log(P + 1e-12), axis=1))
    return Z1, A1, P, loss

def backward(X, Z1, A1, P, W2):
    dZ2 = (P - Y) / N                             # delta at the output
    dW2 = A1.T @ dZ2;  db2 = dZ2.sum(axis=0)
    dA1 = dZ2 @ W2.T
    dZ1 = dA1 * (Z1 > 0)                          # ReLU'(z) = 1[z > 0]
```

```
    dW1 = X.T @ dZ1;   db1 = dZ1.sum(axis=0)
    return dW1, db1, dW2, db2

# Gradient check on one entry of W1 (central difference)
Z1, A1, P, loss = forward(X, W1, b1, W2, b2)
dW1, db1, dW2, db2 = backward(X, Z1, A1, P, W2)
eps = 1e-6; E = np.zeros_like(W1); E[0, 3] = eps
num = (forward(X, W1 + E, b1, W2, b2)[3] - forward(X, W1 - E, b1, W2, b2)[3]) / (2 * eps)
print(f"analytic {dW1[0, 3]:.8f}  numeric {num:.8f}")

# Plain gradient descent
lr = 0.5
for step in range(2000):
    Z1, A1, P, loss = forward(X, W1, b1, W2, b2)
    dW1, db1, dW2, db2 = backward(X, Z1, A1, P, W2)
    W1 -= lr * dW1; b1 -= lr * db1; W2 -= lr * dW2; b2 -= lr * db2
acc = (P.argmax(axis=1) == y).mean()
print(f"final loss {loss:.3f}, train accuracy {acc:.3f}")
# analytic -0.01941581  numeric -0.01941581
# final loss 0.063, train accuracy 0.970
```

In practice you never write the backward pass yourself: frameworks such as PyTorch record the forward computation and run this exact procedure automatically when you call `loss.backward()`. Writing it once by hand is still the best way to understand what that call does, and why it fails when something in the graph has a zero derivative.

Training II: making optimization work

Plain version. Knowing the gradient is not enough. You also need a sensible starting point (initialization), a rule for turning gradients into steps (the optimizer), a step size that changes over time (the schedule), a choice of how many examples to look at per step (the batch size), and a guard against occasional huge steps (clipping).

Vanishing and exploding gradients

Unroll the recursion $\delta_\ell = (W_{\ell+1}^\top \delta_{\ell+1}) \odot f'(\mathbf{z}_\ell)$ across L layers and the gradient at the first layer contains a **product** of L weight matrices and L activation derivatives. If the typical factor is below 1, the product shrinks exponentially with depth and the early layers barely learn: **vanishing gradients**. With the sigmoid, every f' is at most 0.25, so ten layers can shrink the signal by a factor of up to $0.25^{10} \approx 10^{-6}$. If the typical factor is above 1, for example because the initial weights are too large, the product grows exponentially, updates become huge, and the loss turns into `inf` or `NaN`: **exploding gradients**.

The remedies, each covered below, attack the product from different sides:

- activations whose derivative is 1 over a wide range (ReLU and its smooth relatives);

- initialization that keeps signal size constant from layer to layer;
- normalization layers that re-standardize activations;
- **residual connections**, which give the gradient a path with factor 1 (the last main section);
- **gradient clipping**, the standard emergency brake for explosions.

Other practices sometimes listed, such as fine-tuning a pre-trained model or using a smaller network, make training easier overall, but they do not directly fix the gradient problem.

Weight initialization

If all weights start equal, every hidden unit computes the same thing and receives the same gradient, and they stay identical forever. So weights start random. The scale matters: we want the variance of activations (forward) and of gradients (backward) to stay roughly constant from layer to layer. For a linear layer with n_{in} inputs and n_{out} outputs:

- **Xavier (Glorot) initialization** [17] balances the forward and backward conditions for symmetric activations such as tanh: $\operatorname{Var}(W_{ij}) = \dfrac{2}{n_{\text{in}} + n_{\text{out}}}$, for example uniform on $[-\sqrt{6/(n_{\text{in}} + n_{\text{out}})}, \sqrt{6/(n_{\text{in}} + n_{\text{out}})}]$.
- **He (Kaiming) initialization** [18] accounts for ReLU zeroing about half its inputs, which halves the variance at every layer, by doubling the scale: $\operatorname{Var}(W_{ij}) = \dfrac{2}{n_{\text{in}}}$, for example $W_{ij} \sim \mathcal{N}(0, 2/n_{\text{in}})$.

Biases usually start at zero. He et al. showed that this initialization lets very deep ReLU networks be trained from scratch [18]. The NumPy example above uses it.

Optimizers

Every optimizer below is a variant of **stochastic gradient descent** (SGD): estimate the gradient on a small random mini-batch, take a step against it, repeat. Write $\mathbf{g}_t = \nabla_\theta L_{\text{batch}}(\theta_t)$ for the mini-batch gradient at step t, θ for all parameters and η for the learning rate.

SGD. $\theta_{t+1} = \theta_t - \eta\,\mathbf{g}_t$. Simple and memory-free, but it zigzags in narrow valleys and moves slowly along flat directions.

SGD with momentum. Keep a running velocity: $\mathbf{v}_{t+1} = \mu\,\mathbf{v}_t + \mathbf{g}_t$, $\theta_{t+1} = \theta_t - \eta\,\mathbf{v}_{t+1}$, with μ around 0.9. Like a heavy ball rolling downhill, it averages out the zigzag and builds speed along consistent directions.

Adam (Kingma and Ba [19]) keeps two running averages per parameter, of the gradient and of its square:

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\,\mathbf{g}_t, \quad \mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\,\mathbf{g}_t^2,$$

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1-\beta_1^t}, \qquad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1-\beta_2^t}, \qquad \theta_t = \theta_{t-1} - \eta\,\frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon},$$

where squares, square roots and division are element-wise, $\beta_1 \approx 0.9$ and $\beta_2 \approx 0.999$ are decay rates, and ϵ is a small constant. The first average is momentum. Dividing by the root of the second gives every parameter its own step size, which is the idea of RMSprop that Adam builds on [19]: parameters with consistently large gradients take smaller steps, rarely updated ones take larger steps. The **bias correction** $1/(1-\beta^t)$ fixes the fact that both averages start at zero and would otherwise be too small in the first steps.

AdamW (Loshchilov and Hutter [20]). Weight decay shrinks weights toward zero at every step. For plain SGD this is the same as adding an L2 penalty $\frac{\lambda}{2}\|\theta\|^2$ to the loss. For Adam it is not: an L2 term enters $\mathbf{g}_t$ and is then divided by $\sqrt{\hat{\mathbf{v}}_t}$, so weights with large gradients are regularized *less*. AdamW **decouples** the decay from the adaptive step:

$$\theta_t = \theta_{t-1} - \eta \left(\frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} + \lambda\, \theta_{t-1} \right).$$

The authors showed this improves Adam's generalization [20], and AdamW is now the most common optimizer for Transformers and large language models. In PyTorch, `torch.optim.Adam(..., weight_decay=λ)` is the coupled L2 version and `torch.optim.AdamW` is the decoupled one.

Newer optimizers (brief notes). Two recent alternatives are worth knowing by name. **Lion** was found by an automated program search [21]: it keeps only a momentum buffer (so it uses less memory than Adam) and updates every parameter by the **sign** of an interpolated momentum, so all updates have the same magnitude; it needs a smaller learning rate than AdamW. **Muon** [22] applies to the 2-D weight matrices of hidden layers: it takes the momentum update and approximately orthogonalizes it with a few Newton–Schulz iterations, while embeddings, output layers, biases and other scalar or vector parameters are still trained with AdamW. Liu et al. report that, with weight decay and per-parameter update scaling added, Muon reached about twice the compute efficiency of AdamW in their compute-optimal large-language-model training experiments [23]. These are active research topics; AdamW remains the safe default.

Learning-rate schedules

The learning rate is usually the most important hyperparameter. A rate that is fine early in training is often too large later, when the model needs small, careful steps, so the rate is varied over time (Figure 6).

- **Step decay**: multiply the rate by a factor such as 0.1 at fixed epochs. Traditional for CNN training with SGD.
- **Cosine annealing**: $\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})(1 + \cos(\pi t/T))$, a smooth decay from $\eta_{\max}$ to $\eta_{\min}$ over T steps; SGDR [24] combined it with periodic warm restarts.
- **Warmup**: increase the rate linearly from near zero over the first few hundred or thousand steps. Early in training, Adam's second-moment estimates are noisy and the weights are far from any good region, and a full-size step can be destabilizing.

As a concrete recipe from a published large model, LLaMA was trained with AdamW ($\beta_1 = 0.9$,

$\beta_2 = 0.95$), weight decay 0.1, gradient clipping at 1.0, 2,000 warmup steps, and a cosine schedule ending at 10% of the peak rate [12].

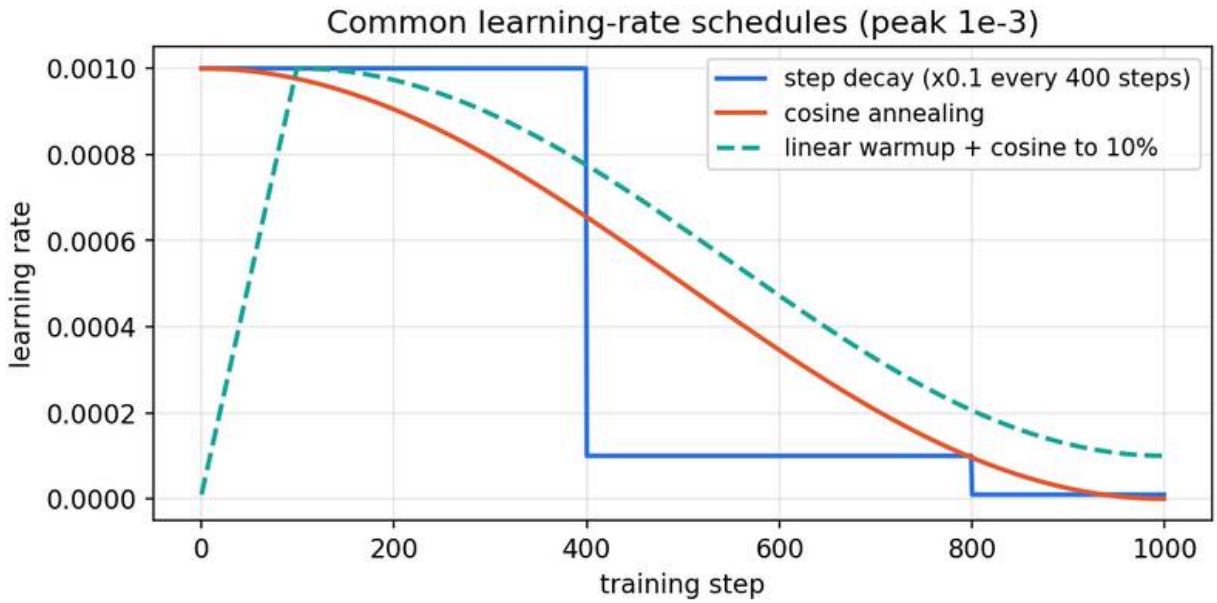


Figure 6 — Three common learning-rate schedules over 1,000 steps with a peak of 1e-3.

Mini-batches, iterations and epochs

A **mini-batch** (often just "batch") is the small random subset of B examples used for one gradient estimate. One **iteration** is one parameter update, using one mini-batch. One **epoch** is one full pass over the training set, which takes $\lceil N/B \rceil$ iterations for N examples. For example, $N = 50{,}000$ and $B = 128$ give 391 iterations per epoch.

The batch size B affects both speed and the result, and bigger is **not** always better:

- **Small batches** give noisy gradient estimates (the variance of the mini-batch mean falls roughly like $1/B$), so the loss curve is jumpy and each epoch takes many steps with poor hardware utilization.
- **Large batches** give stable gradients and run efficiently in parallel on GPUs, but each epoch has fewer updates, and they tend to generalize worse when other settings are kept fixed. Keskar et al. found that large-batch training tends to converge to **sharp minima**, where the loss rises steeply around the solution, while small-batch training, thanks to its gradient noise, tends to find **flat minima**, which generalize better [25].

In practice, choose the largest batch that fits comfortably in memory *and* still trains well, and retune the learning rate when you change B; the two interact strongly.

Gradient clipping

Even in a well-designed network, a rare batch can produce a huge gradient. **Gradient-norm clipping** rescales the whole gradient when its norm exceeds a threshold c:

$$\mathbf{g} \leftarrow \mathbf{g} \cdot \min\left(1, \frac{c}{\|\mathbf{g}\|}\right),$$

where $\|\mathbf{g}\|$ is the norm of all parameter gradients concatenated. It keeps the direction but limits the step size, which is the standard defense against exploding gradients, especially in recurrent networks and Transformers [3]. A threshold of 1.0 is a common choice.

Putting it together in PyTorch

The snippet trains a small MLP on `make_moons` with mini-batches, AdamW, a cosine schedule and gradient clipping. It runs on a CPU in seconds.

```
import torch, torch.nn as nn
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split

torch.manual_seed(0)
X, y = make_moons(n_samples=1000, noise=0.25, random_state=0)
Xtr, Xva, ytr, yva = train_test_split(X, y, test_size=0.3, random_state=0)
Xtr, Xva = torch.tensor(Xtr, dtype=torch.float32), torch.tensor(Xva, dtype=torch.float32)
ytr, yva = torch.tensor(ytr), torch.tensor(yva)

model = nn.Sequential(
    nn.Linear(2, 64), nn.ReLU(),
    nn.Linear(64, 64), nn.ReLU(),
    nn.Linear(64, 2),               # logits: no softmax here
)
loss_fn = nn.CrossEntropyLoss()     # applies log-softmax internally
opt = torch.optim.AdamW(model.parameters(), lr=1e-2, weight_decay=1e-2)
sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, T_max=200)

for epoch in range(200):
    model.train()
    perm = torch.randperm(len(Xtr))
    for i in range(0, len(Xtr), 64):                # mini-batches of 64
        idx = perm[i:i + 64]
        loss = loss_fn(model(Xtr[idx]), ytr[idx])
        opt.zero_grad()
        loss.backward()                              # backpropagation
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        opt.step()
    sched.step()

model.eval()
with torch.no_grad():
```

```
    acc = (model(Xva).argmax(dim=1) == yva).float().mean().item()
print(f"validation accuracy: {acc:.3f}")   # about 0.94
```

Note `model.train()` and `model.eval()`: they switch layers such as dropout and BatchNorm between their training and inference behavior. Forgetting `eval()` at test time is a classic bug.

Generalization: underfitting, overfitting and regularization

Plain version. The goal is not to do well on the training data; it is to do well on data the model has never seen. A model can fail in two opposite ways: it can be too weak to learn even the training data (underfitting), or it can memorize the training data, noise included, and fail on new data (overfitting).

Capacity and the two failure modes

A model's **capacity** is, informally, how complex a function it can fit; it grows with the number of parameters, depth and width, and training time. As capacity increases, training error keeps falling, while validation error first falls and then rises again once the model starts fitting noise. The gap between the two is the **generalization gap** [3].

- **Underfitting**: the **training** error itself is high. The model, or the training procedure, cannot capture the pattern.
- **Overfitting**: the training error is low but the validation or test error is much higher.

What to do about underfitting, in order:

1. **Check the code and data first.** A bug, mismatched inputs and labels, wrong preprocessing, or a loss applied to the wrong tensor all look like underfitting.
2. **Increase capacity**: a wider or deeper model, or a better architecture.
3. **Improve optimization**: train longer, tune the learning rate and schedule, switch optimizer, check initialization and normalization.

Making the model **smaller** never fixes underfitting; it lowers capacity further. A smaller model can be a reasonable choice for speed or memory once accuracy is adequate, but that is an efficiency decision, not a cure.

What to do about overfitting: use more or more varied data, add regularization (below), reduce capacity, and stop early. It is also worth checking that the training and validation sets really come from the same distribution; a mismatch between them looks like overfitting but needs a data fix, not a regularizer.

Reading a loss curve

Plot both the training and the validation loss against iterations or epochs. Four patterns cover most situations:

- **A. Both curves flat from the start.** If the loss is high, the model is not learning at all: suspect a bug, a far too small or large learning rate, or a model/data mismatch. If the loss is already low, the task may be very easy, or each logged point covers so many iterations that the curve's shape is hidden; log more often.
- **B. Training loss keeps falling, validation loss turns upward.** Classic overfitting. Keep the parameters from the point of lowest validation loss (early stopping) and add regularization.
- **C. Both curves fall and level off close together.** Either the model is near its best, which is common when training and validation data are very similar, or both are still high, which is underfitting: train longer or increase capacity.
- **D. Validation loss above training loss, but both still decreasing steadily.** The normal, healthy situation. Keep training and keep the checkpoint with the lowest validation loss.

Regularization tools

Figure 7 shows overfitting and two cures on purpose. A wide MLP (two hidden layers of 256 units) is trained on only 100 noisy two-moons points. Without regularization the training loss falls toward zero while the validation loss reaches its minimum early and then climbs steeply. Weight decay and dropout both slow the climb dramatically.

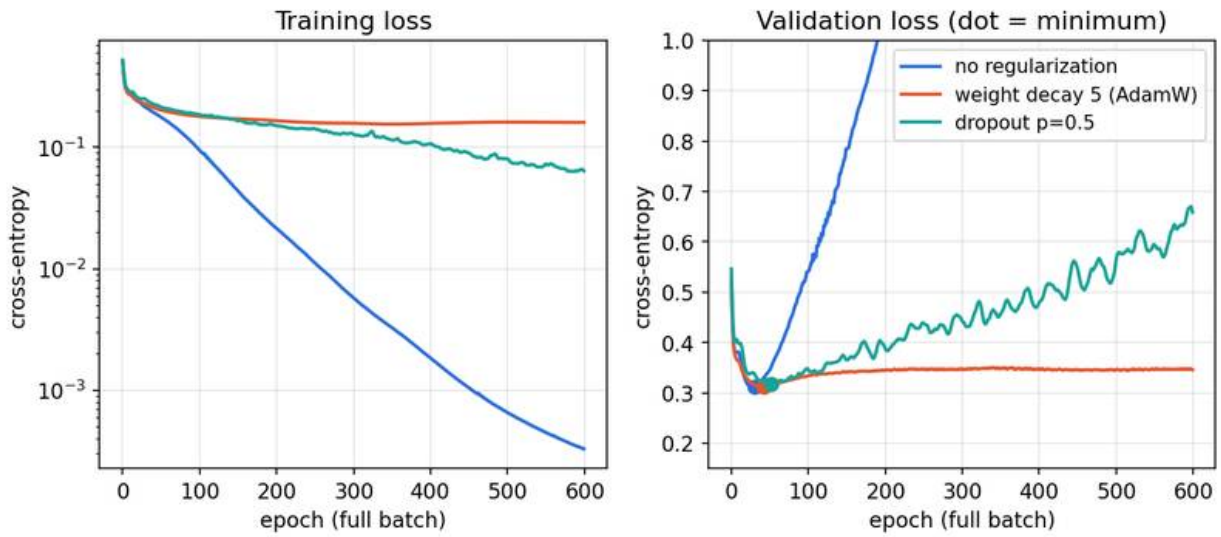


Figure 7 — Overfitting on 100 training points from make_moons (noise 0.35), evaluated on 2,000 validation points. Full-batch AdamW, learning rate 3e-3. Dots mark each run's minimum validation loss, which is where early stopping would keep the model.

Weight decay (L2 regularization). Penalize large weights by adding $\frac{\lambda}{2}\|\theta\|^2$ to the loss, or, with

Adam, use decoupled decay (AdamW). Large weights let a network make sharp, wiggly boundaries around individual points; keeping them small favors smoother functions. In Figure 7, decoupled decay with $\lambda = 5$ keeps the validation loss almost flat after its minimum.

Dropout (Srivastava et al. [27]). During training, set each hidden unit to zero with probability p, independently for every example and step, and scale the survivors by $1/(1-p)$ so the expected activation is unchanged ("inverted dropout"). At test time, use all units with no scaling. Each step thus trains a different thinned sub-network; units cannot rely on specific partners and must learn features that are useful on their own, and the full network at test time behaves like an average of many thinned networks [27]. This is why `model.eval()` matters.

Data augmentation. Create new training examples by applying transformations that do not change the label: flips, crops, small rotations, color jitter and noise for images. It is often the single most effective regularizer in vision, because it teaches the invariances directly. It must respect the task: a horizontal flip is fine for cats, not for reading text.

Early stopping. Monitor the validation loss during training and keep the parameters from the epoch where it was lowest; stop when it has not improved for a set number of epochs (the "patience"). It costs nothing, needs no change to the model, and acts as a regularizer by limiting how far the parameters can travel from their initialization [3].

More data and better strategy. The other levers the original notes list are worth repeating: more training data is the most reliable cure for overfitting; a stronger architecture (deeper, or better structured, so that it converges well) helps when the model underfits; and a better training strategy (optimizer, learning-rate schedule, batch size) helps both.

Residual connections and the road to Transformers

Plain version. If you stack many layers, each must pass the signal on intact as well as improve it, which turns out to be hard. A residual connection gives every block an express lane: the block only has to learn a *correction* that is added to its input.

He et al. observed that adding more plain layers to an already deep network made even the **training** error worse, which is an optimization problem rather than overfitting [28]. Their **residual block** computes

$$\mathbf{y} = \mathbf{x} + F(\mathbf{x}),$$

where $\mathbf{x}$ is the block's input and F is a small stack of layers (in ResNet, convolutions with BatchNorm and ReLU). If the best thing a block can do is nothing, it only has to push F toward zero, which is easy. The backward pass shows why it trains well:

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = I + \frac{\partial F}{\partial \mathbf{x}},$$

where I is the identity matrix. The identity term passes the gradient straight through every block, so the product over many blocks no longer has to shrink to zero. With residual connections, He et al. trained networks with up to 152 layers on ImageNet and won several ILSVRC 2015 tasks [28].

The **Transformer** (Vaswani et al. [29]) combines every idea in this tutorial. Each block has two sub-layers, multi-head self-attention (which uses softmax to mix information between positions) and a position-wise MLP, and each sub-layer is wrapped in a residual connection with LayerNorm. The original paper normalized after the addition; many recent models, LLaMA among them, normalize the sub-layer **input** instead (pre-normalization) and use RMSNorm and SwiGLU [12]:

$$\mathbf{h} = \mathbf{x} + \text{Attention}\,(\,\text{Norm}(\mathbf{x})), \qquad \mathbf{y} = \mathbf{h} + \text{FFN}\,(\,\text{Norm}(\mathbf{h})).$$

Since 2017 this design has spread from machine translation to language modeling, vision, speech and more, and it is the basis of today's large language models. The trained recipe is the one from this tutorial too: AdamW, warmup and cosine decay, gradient clipping, and weight decay [12]. The deep-learning track picks up from here with convolutional networks and Transformers for images, starting with image classification.

Modern view

The basic toolkit above has been stable for several years; research continues on why it works and on better versions of each piece. The following reviews are good guides, each from a different angle.

The big picture: LeCun, Bengio and Hinton [2] (Nature, 2015). This short review is still the clearest statement of what deep learning is: representation learning with many layers, where each layer transforms the previous representation into a more abstract one and the features are learned from data with backpropagation rather than engineered. It walks through supervised learning with SGD, the role of ReLU, convolutional networks for images, recurrent networks for sequences, and distributed representations, and it closes by predicting that unsupervised learning and the combination of representation learning with reasoning would matter more in the future. For a textbook treatment of every topic in this tutorial, including the maximum-likelihood view of loss functions, regularization and optimization, the book by Goodfellow, Bengio and Courville [3] remains the standard reference.

Optimization: Sun [26]. Sun's survey asks "when and why can a neural network be successfully trained?" and organizes the answer in three parts: problems with the gradient (explosion, vanishing) and their fixes through careful initialization and normalization; the practical algorithms (SGD, adaptive methods such as Adam, and distributed training) with their convergence theory; and the global picture of the loss landscape, including bad local minima, mode connectivity, the lottery-ticket hypothesis and the infinite-width limit. It is a good bridge from the practice above to theory.

Normalization: Huang et al. [16] (IEEE TPAMI, 2023). This survey gives a unified view of normalization methods from the perspective of optimization, and a taxonomy that splits any method into its normalization area, normalization operation and representation recovery. It covers the history from BatchNorm on, the analyses of why normalization speeds training and helps generalization, and applications in different domains. Read it to see BatchNorm, LayerNorm, group-wise and weight-based normalization as points in one design space.

Activation functions: Dubey, Singh and Chaudhuri [7] (Neurocomputing, 2022). This survey classifies activation functions into logistic sigmoid/tanh-based, ReLU-based, ELU-based and learning-based families, characterizes them by properties such as output range, monotonicity and smoothness, and benchmarks 18 of them across network types and data. Its practical message is that no single activation is best everywhere; the choice interacts with the architecture and the data.

Where things stand (2023–2026). Several convergences are visible in published model recipes:

- **Architecture.** The pre-normalized Transformer block with residual connections is the dominant architecture for language and increasingly for vision; RMSNorm and SwiGLU, as in LLaMA [12], are common defaults in large language models.
- **Optimization.** AdamW [20] with linear warmup, a cosine (or similar) decay, and gradient clipping is the standard recipe. Optimizers that look beyond per-coordinate scaling are an active area: Lion [21] uses sign updates and less memory, and Muon [22] orthogonalizes matrix updates, with reported compute-efficiency gains at scale [23]. How robust such gains are across tasks and scales is still being studied, which is why AdamW remains the safe choice.
- **Generalization.** The sharp-versus-flat-minima view [25] is a useful intuition rather than a complete theory, and why heavily over-parameterized networks generalize as well as they do is still an open question; Sun's survey [26] summarizes the main lines of attack.

Open problems. A reliable theory of generalization for deep networks; principled rules for choosing the learning rate, batch size and schedule together, especially when scaling models up; understanding exactly how normalization, residual connections and adaptive optimizers interact; and making training stable at very large scale without the many small tricks that current recipes rely on.

Key takeaways

- A perceptron is a linear score followed by a step function and trained by a mistake-driven rule; logistic regression uses a sigmoid probability and cross-entropy. Similar structure, different methods, and only the smooth one can be stacked and trained by gradients.
- Hidden layers with nonlinear activations are what make networks more than linear models. One wide hidden layer can in principle approximate any continuous function, but that is an existence result; depth makes useful functions easier to represent and learn. There is no fixed layer count that defines "deep".
- ReLU-family activations (ReLU, Leaky ReLU, GELU, SiLU) keep gradients alive; sigmoid and softmax belong mostly at the output; SwiGLU is the modern feed-forward choice in Transformers. BatchNorm normalizes over the batch, LayerNorm and RMSNorm over the features of each sample.
- Match the head to the task: linear plus MSE for regression, sigmoid plus binary cross-entropy for binary or multi-label, softmax plus cross-entropy for multi-class. In each case the output gradient is prediction minus target.
- Backpropagation is the chain rule applied from the loss backward: $\delta_\ell = (W_{\ell+1}^\top \delta_{\ell+1}) \odot f'(\mathbf{z}_\ell)$

and $\partial L/\partial W_\ell = \delta_\ell \mathbf{a}_{\ell-1}^\top$. Check hand-written gradients numerically.
- Practical training: He or Xavier initialization, AdamW with warmup and a decaying schedule, gradient clipping, and a batch size chosen with the learning rate, not "as large as possible".
- Underfitting means high training error and calls for more capacity or better optimization, never a smaller model; overfitting calls for more data, augmentation, weight decay, dropout and early stopping.
- Residual connections make very deep networks trainable, and the Transformer is these ideas combined.

Exercises

1. **Perceptron by hand.** Run the perceptron rule on the AND data with $\eta = 1$, $\mathbf{w} = \mathbf{0}$, $b = 0$, visiting the points in the order $(0,0), (0,1), (1,0), (1,1)$. Write down $\mathbf{w}$ and b after each update until an epoch passes with no mistakes. Then explain in one sentence why the same procedure on XOR never stops.

   Hint

   With the strict test $z > 0$, the first point $(0,0)$ has $z = 0$, so it is predicted 0 and is correct. The first mistake is $(1,1)$, which gives $\mathbf{w} = (1,1)$, $b = 1$. Keep going: each mistake on a 0-labeled point subtracts $\mathbf{x}$ and 1, each mistake on the 1-labeled point adds them. With this order there are ten updates over five epochs, and the sixth epoch is error-free with $\mathbf{w} = (2,1)$, $b = -2$, that is, the line $2x_1 + x_2 = 2$. A different visiting order gives a different, equally valid line. For XOR, no $(\mathbf{w}, b)$ classifies all four points correctly, so some point is always wrong and an update always happens.

2. **How fast do sigmoid gradients vanish?** Show that $\sigma'(x) = \sigma(x)(1 - \sigma(x)) \le 1/4$. Then consider a chain of 20 sigmoid layers with all weights equal to 1. What is the largest factor by which the gradient can be multiplied on its way back? Repeat the reasoning for ReLU with active units.

   Hint

   $s(1-s)$ with $s \in (0,1)$ is a downward parabola with maximum $1/4$ at $s = 1/2$. Twenty factors of at most $1/4$ give at most $4^{-20} \approx 9 \times 10^{-13}$. For an active ReLU unit, $f' = 1$, so the activations contribute no shrinkage at all; only the weights do, which is why initialization matters.

3. **The prediction-minus-target gradient.** Using $\partial p_k/\partial z_j = p_k(\delta_{kj} - p_j)$, prove that for $L = -\sum_k y_k \log p_k$ with a one-hot $\mathbf{y}$, $\partial L/\partial z_j = p_j - y_j$. Then show the same result for the sigmoid with binary cross-entropy.

   Hint

   $\partial L/\partial z_j = -\sum_k (y_k/p_k)\, p_k(\delta_{kj} - p_j) = -\sum_k y_k \delta_{kj} + p_j \sum_k y_k = -y_j + p_j$, since $\sum_k y_k = 1$. For the sigmoid, $\partial \ell/\partial \hat{p} = -y/\hat{p} + (1-y)/(1-\hat{p})$ and $\partial \hat{p}/\partial z = \hat{p}(1-\hat{p})$; multiply and simplify to $\hat{p} - y$.

4. **Extend the gradient check.** In the NumPy example, check every entry of `W2`, `b1` and `b2` (not just one entry of `W1`) against finite differences, and report the maximum relative error $|a-n|/\max(|a|, |n|, 10^{-12})$. Then deliberately introduce a bug, for example drop the `* (Z1 > 0)` factor, and see how large the error becomes.

   Hint

   Loop over `np.ndindex(param.shape)`, perturb one entry by $\pm 10^{-6}$, and recompute the loss. A correct implementation gives relative errors around 10^{-7} or smaller (a few entries next to ReLU kinks can be larger). Removing the ReLU derivative makes errors of order 1 appear in `W1` and `b1`, while `W2` and `b2` stay correct, because the bug is only in the backward path below the hidden layer.

5. **Iterations, epochs and batch size.** A dataset has 60,000 training images. (a) How many iterations make one epoch with $B = 32$, and with $B = 1{,}024$? (b) If you train both for 20 epochs, how many parameter updates does each make? (c) Based on the batch-size discussion, give two reasons the $B = 1{,}024$ run might reach a worse validation accuracy with the same learning rate, and one change that could help.

   Hint

   (a) $\lceil 60000/32 \rceil = 1875$ and $\lceil 60000/1024 \rceil = 59$. (b) 37,500 versus 1,180 updates. (c) Far fewer updates in the same number of epochs, and less gradient noise, which the sharp-minima findings associate with worse generalization. Tuning the learning rate for the new batch size (typically larger, with warmup) and training for more epochs are the usual adjustments.

6. **Diagnose the curve.** A colleague's network has a training loss that drops quickly at first and then stays flat at a high value; the validation loss tracks it closely. They propose halving the number of hidden units "to make it converge faster". What is happening, what would you check first, and what would you suggest instead?

   Hint

   Both losses are high and close together: this is underfitting (pattern C with high values), not overfitting. First check for bugs: labels aligned with inputs, the loss applied to logits, data normalized, the learning rate sensible. Then increase capacity or improve optimization (wider or deeper model, a learning-rate schedule, longer training). Shrinking the model lowers capacity and will make underfitting worse.

References

1. Shyandram, "機器學習及類神經網路筆記 (Notes on machine learning and neural networks)," Medium post (in Traditional Chinese), 2024; updated 2026. link
2. Y. LeCun, Y. Bengio and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015. doi

3. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016. online book
4. F. Rosenblatt, “The perceptron: A probabilistic model for information storage and organization in the brain,” *Psychological Review*, vol. 65, pp. 386–408, 1958. doi
5. G. Cybenko, “Approximation by superpositions of a sigmoidal function,” *Mathematics of Control, Signals and Systems*, vol. 2, pp. 303–314, 1989. doi
6. D. E. Rumelhart, G. E. Hinton and R. J. Williams, “Learning representations by back-propagating errors,” *Nature*, vol. 323, pp. 533–536, 1986. doi
7. S. R. Dubey, S. K. Singh and B. B. Chaudhuri, “Activation functions in deep learning: A comprehensive survey and benchmark,” *Neurocomputing*, vol. 503, pp. 92–108, 2022. arXiv
8. D. Hendrycks and K. Gimpel, “Gaussian error linear units (GELUs),” arXiv:1606.08415, 2016. arXiv
9. S. Elfwing, E. Uchibe and K. Doya, “Sigmoid-weighted linear units for neural network function approximation in reinforcement learning,” *Neural Networks*, vol. 107, pp. 3–11, 2018. arXiv
10. P. Ramachandran, B. Zoph and Q. V. Le, “Searching for activation functions,” arXiv:1710.05941, 2017. arXiv
11. N. Shazeer, “GLU variants improve Transformer,” arXiv:2002.05202, 2020. arXiv
12. H. Touvron, T. Lavril, G. Izacard et al., “LLaMA: Open and efficient foundation language models,” arXiv:2302.13971, 2023. arXiv
13. S. Ioffe and C. Szegedy, “Batch normalization: Accelerating deep network training by reducing internal covariate shift,” in *Proc. International Conference on Machine Learning* (*ICML*), PMLR 37, pp. 448–456, 2015. PMLR
14. J. L. Ba, J. R. Kiros and G. E. Hinton, “Layer normalization,” arXiv:1607.06450, 2016. arXiv
15. B. Zhang and R. Sennrich, “Root mean square layer normalization,” in *Advances in Neural Information Processing Systems* (*NeurIPS*), 2019. arXiv
16. L. Huang, J. Qin, Y. Zhou, F. Zhu, L. Liu and L. Shao, “Normalization techniques in training DNNs: Methodology, analysis and application,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 45, no. 8, pp. 10173–10196, 2023. arXiv
17. X. Glorot and Y. Bengio, “Understanding the difficulty of training deep feedforward neural networks,” in *Proc. International Conference on Artificial Intelligence and Statistics* (*AISTATS*), PMLR 9, pp. 249–256, 2010. PMLR
18. K. He, X. Zhang, S. Ren and J. Sun, “Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification,” in *Proc. IEEE International Conference on Computer Vision* (*ICCV*), pp. 1026–1034, 2015. arXiv
19. D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *Proc. International Conference on Learning Representations* (*ICLR*), 2015. arXiv
20. I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *Proc. International Conference on Learning Representations* (*ICLR*), 2019. arXiv
21. X. Chen, C. Liang, D. Huang, E. Real, K. Wang, H. Pham, X. Dong, T. Luong, C.-J. Hsieh, Y. Lu and Q. V. Le, “Symbolic discovery of optimization algorithms,” in *Advances in Neural Information Processing Systems* (*NeurIPS*), 2023. arXiv
22. K. Jordan, “Muon: An optimizer for hidden layers in neural networks,” Medium post, December 2024. link
23. J. Liu, J. Su, X. Yao et al., “Muon is scalable for LLM training,” arXiv:2502.16982, 2025. arXiv
24. I. Loshchilov and F. Hutter, “SGDR: Stochastic gradient descent with warm restarts,” in *Proc.*

International Conference on Learning Representations (*ICLR*), 2017. arXiv
25. N. S. Keskar, D. Mudigere, J. Nocedal, M. Smelyanskiy and P. T. P. Tang, "On large-batch training for deep learning: Generalization gap and sharp minima," in *Proc. International Conference on Learning Representations* (*ICLR*), 2017. arXiv
26. R. Sun, "Optimization for deep learning: Theory and algorithms," arXiv:1912.08957, 2019. arXiv
27. N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014. JMLR
28. K. He, X. Zhang, S. Ren and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition* (*CVPR*), pp. 770–778, 2016. arXiv
29. A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems* (*NeurIPS*), 2017. arXiv

ML 03 • Convolution, Image Filtering and CNNs

From blur and Sobel kernels to learned filters: convolution versus correlation, border handling, padding, stride, dilation, pooling, receptive fields, backpropagation through a convolution, and the classic CNN architectures.

Online edition: https://blog.shyandram.dev/en/learn/ml/ml-03-convolutional-networks/

Notebook: https://github.com/Shyandram/cv-blog/blob/05b94ec/notebooks/image-processing/convolution.ipynb

What you'll learn

- What a kernel is, how it slides over an image, and why the same operation can blur, sharpen or find edges.
- The exact difference between **correlation** and **convolution**, and which one NumPy, SciPy, OpenCV and PyTorch actually compute.
- How to write 2-D filtering from scratch in NumPy, reproduce it in OpenCV, SciPy and PyTorch, and handle image borders deliberately.
- How a convolutional layer generalizes a fixed filter: channels, weight sharing, padding, stride, dilation, 1×1 convolutions and pooling.
- How receptive fields grow with depth, and how gradients flow backward through a convolution.
- What the classic architectures (LeNet, AlexNet, VGG, GoogLeNet, ResNet, U-Net, MobileNet, EfficientNet, ConvNeXt) each contributed, and where CNNs stand in 2026.

The big picture

An image is a grid of numbers (see Images as arrays). Many of the most useful things we can do to it are **local**: whether a pixel sits on an edge, inside a smooth region or in a noisy patch depends mostly on its neighbors. **Convolution** is the operation that formalizes "look at a small neighborhood, combine it with fixed weights, and repeat everywhere." It is the workhorse of classical image processing, and it is also the building block of **convolutional neural networks** (**CNNs**), which learn their weights from data instead of having them designed by hand.

> **Explain like I'm five**
>
> Imagine you have a small cardboard window with nine holes, and behind each hole you write a number. You place the window on a picture, multiply each visible dot of color by the number behind its hole, add everything up, and write the answer on a new sheet. Then you slide the window one step and do it again, until you have covered the whole picture. If all nine numbers are equal, the new picture is a blurry average. If the numbers on the left are negative and those on the right are positive,

you get big answers only where the picture changes from dark to light: an edge finder. A CNN is a machine that slides thousands of such windows and **learns** which numbers to write behind the holes.

This tutorial has two halves. The first half (filtering) is the classical toolbox and needs only NumPy. The second half (CNNs) builds on ML 02, where you met layers, losses and backpropagation. If you want more depth on fixed filters afterward, IP 03 covers spatial filtering and IP 04 explains the frequency-domain view through the convolution theorem. For the deep-learning side, DLCV 02 studies modern classification backbones in detail, ML 03A turns to sequences, and ML 04 introduces attention, the main alternative to convolution. Standard textbook treatments are chapter 9 of Goodfellow et al. [1] and chapter 10 of Prince [10].

Part I • Filtering with a fixed kernel

The sliding weighted sum

Plain version. Every output pixel is a weighted sum of the input pixels around the same position. The small grid of weights is called a **kernel** (also *filter* or *mask*).

Precise version. Let I be a grayscale image, with $I(x, y)$ the intensity at column x and row y. Let K be a kernel of size $(2a + 1) \times (2b + 1)$, indexed from $-a$ to a vertically and $-b$ to b horizontally, so that $K(0, 0)$ is its center. The filtered image G is

$$G(x, y) = \sum_{i=-a}^{a} \sum_{j=-b}^{b} K(i, j)\, I(x + j,\ y + i).$$

Here i indexes kernel rows and j kernel columns, $G(x, y)$ is the output at the same position, and the double sum visits every tap of the kernel once. For a 3×3 kernel, $a = b = 1$ and each output pixel costs nine multiplications and eight additions.

Figure 1 works through one output value by hand. The orange window covers the top-left 3×3 neighborhood of the padded input. Multiply element-wise by the kernel and add: $1 \cdot 1 + 0 \cdot 2 + (-1) \cdot 0 + 2 \cdot 0 + 0 \cdot 1 + (-2) \cdot 3 + 1 \cdot 2 + 0 \cdot 0 + (-1) \cdot 1 = -4$. That number becomes G at the window's center. Sliding the window one step to the right and repeating fills in the next output.

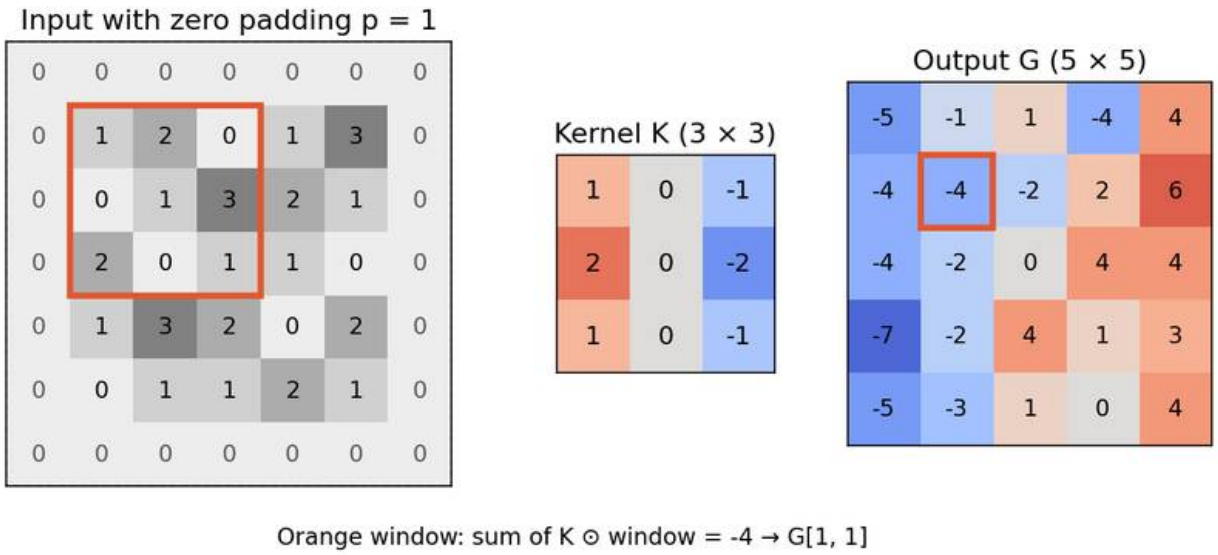


Figure 1 — One step of the sliding weighted sum. The orange 3 × 3 window, multiplied element-wise by K and summed, gives −4 at the matching output position. Zero padding (gray) lets the output keep the input size.

Two observations already matter for everything that follows. First, the same nine weights are used at every position: the operation is **shift-invariant** (also called *space-invariant*). Second, the operation is **linear**: filtering $\alpha I_1 + \beta I_2$ gives $\alpha G_1 + \beta G_2$. Linear shift-invariant (LSI) operations are exactly the ones a convolution can express, which is why convolution is so central in signal processing.

Correlation versus convolution

Plain version. The formula above is technically **cross-correlation**. True **convolution** does the same thing with the kernel rotated by 180 degrees. For symmetric kernels the two agree; for asymmetric ones they differ.

Precise version. Mathematical convolution, written $K * I$, is

$$(K * I)(x, y) = \sum_{i=-a}^{a} \sum_{j=-b}^{b} K(i, j)\, I(x - j,\ y - i) = \sum_{i,j} K(-i, -j)\, I(x + j,\ y + i).$$

The second form shows that convolution equals correlation with the flipped kernel $K(-i, -j)$. A clean way to see the difference is to filter a single bright pixel (an **impulse**). Correlation leaves a copy of the kernel rotated by 180 degrees; convolution leaves an exact copy of the kernel. That copy is the **impulse response**, and it is why signal processing prefers convolution: the output for any image is the sum of shifted, scaled impulse responses.

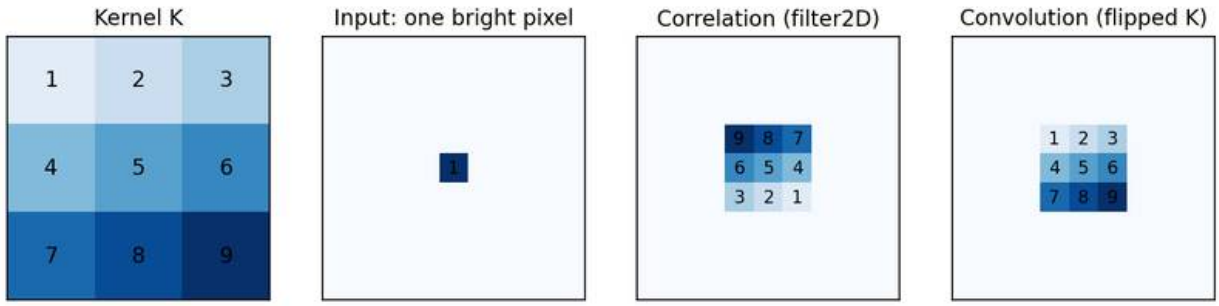


Figure 2 — Filtering an impulse. Correlation (what cv2.filter2D computes) stamps a 180°-rotated kernel; convolution stamps the kernel itself.

Convolution has algebraic properties that correlation lacks. It is **commutative** ($K * I = I * K$) and **associative** ($K_2 * (K_1 * I) = (K_2 * K_1) * I$), so two filters applied in a row equal one bigger filter. Applying a 3×3 box blur twice, for instance, is a single 5×5 kernel. Associativity also explains **separable** kernels: a $k \times k$ Gaussian equals a $k \times 1$ column filter followed by a $1 \times k$ row filter, cutting the cost per pixel from k^2 to $2k$ operations. The **convolution theorem** says convolution in space is multiplication in the Fourier domain; IP 04 develops that view.

Which libraries compute which?

Library call	Computes	Default border
`cv2.filter2D`	correlation [11]	reflect-101 (`BORDER_DEFAULT`)
`scipy.ndimage.correlate` / `convolve`	correlation / convolution	`reflect` (edge sample repeated)
`scipy.signal.convolve2d`	convolution	zero fill
`torch.nn.functional.conv2d`, `nn.Conv2d`	correlation [31]	zero padding

For hand-designed asymmetric kernels (Sobel, emboss, motion blur) the flip changes the sign or direction of the result, so flip deliberately when you need true convolution, for example with `cv2.flip(K, -1)`. For learned kernels the distinction disappears: a network that "should" use the flipped kernel simply learns it. That is why deep-learning frameworks call their correlation "convolution" without harm.

Blur, edges and sharpening

Plain version. Positive weights that sum to one average neighbors and blur. Weights that sum to zero cancel flat regions and keep only changes, so they find edges. Adding a bit of "the image minus its blur" back to the image sharpens it.

A **box blur** gives every neighbor equal weight; the 3×3 version has nine entries of $1/9$. A **Gaussian blur** samples

$$K(i,j) = \frac{1}{Z} \exp\left(-\frac{i^2 + j^2}{2\sigma^2}\right),$$

where σ (in pixels) sets the blur width and Z normalizes the weights to sum to one. Nearby pixels count more than distant ones, which avoids the blocky artifacts of a box. A practical rule is to use a kernel radius of about 3σ.

The **sum of the weights** tells you what happens in a flat region where every pixel equals c: the output is $c \sum K$. A blur with sum one preserves brightness; a kernel with sum two doubles it (and saturates an 8-bit image); a kernel with sum zero outputs zero.

The **Sobel** kernels estimate horizontal and vertical derivatives while smoothing in the orthogonal direction:

$$S_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix} \begin{bmatrix} -1 & 0 & 1 \end{bmatrix}, \qquad S_y = S_x^\top.$$

S_x responds to intensity changes along x, so it highlights **vertical** boundaries; S_y highlights **horizontal** ones. The outputs G_x, G_y are signed: dark-to-light and light-to-dark edges have opposite signs. The **gradient magnitude** $\sqrt{G_x^2 + G_y^2}$ is a nonnegative edge strength, and $\operatorname{atan2}(G_y, G_x)$ is the edge orientation. The factorization shows that Sobel is separable: a $[1, 2, 1]^\top$ smoothing column times a $[-1, 0, 1]$ central difference.

The **Laplacian** kernel $\begin{bmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{bmatrix}$ approximates $\partial^2 I/\partial x^2 + \partial^2 I/\partial y^2$; it responds to thin lines and to both sides of an edge. **Unsharp masking** sharpens with

$$I_{\text{sharp}} = I + \lambda\,(I - G_\sigma * I),$$

where $G_\sigma * I$ is a Gaussian-blurred copy, $I - G_\sigma * I$ is the high-frequency detail, and $\lambda > 0$ controls strength. Large λ amplifies noise and creates bright and dark halos along edges.

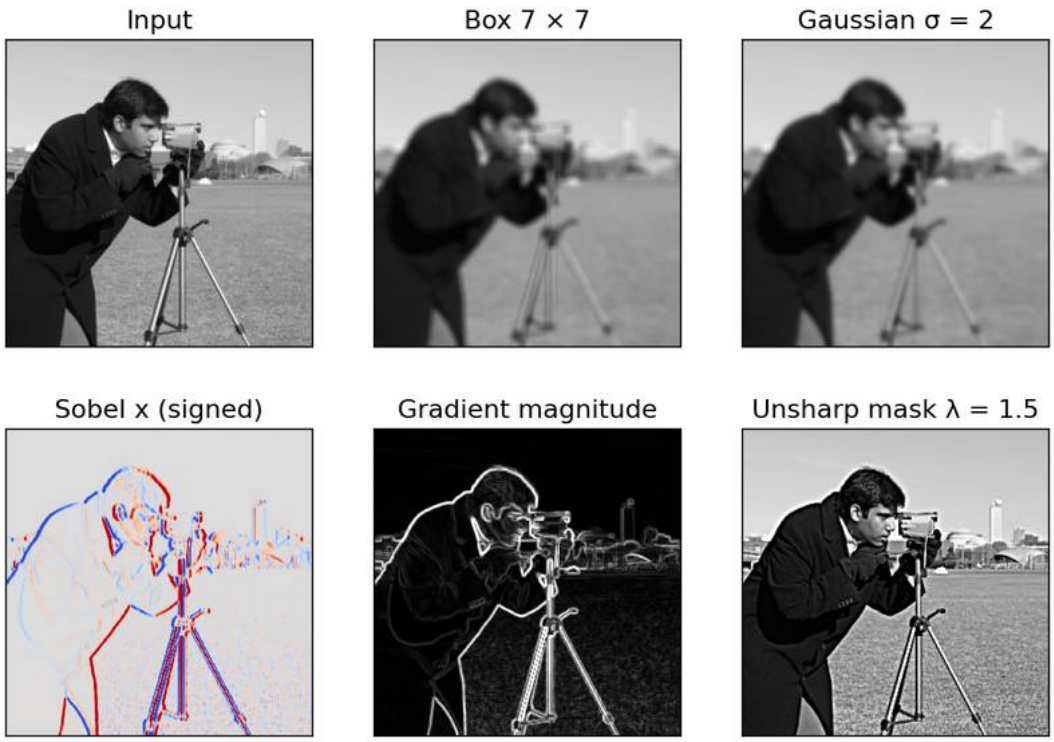


Figure 3 — The same image through six kernels. Box and Gaussian blur suppress fine detail; Sobel x is signed (red = dark-to-light, blue = light-to-dark); the gradient magnitude outlines objects; unsharp masking adds back amplified detail.

Try it: the kernel playground

Edit the kernel cells below and watch the output update. Start from Sobel x, then transpose it to detect horizontal edges instead of vertical ones. Make every weight of the blur preset larger and notice that flat regions brighten, exactly as the sum-of-weights rule predicts. Try the Laplacian and emboss presets and predict their kernel sums before you look.

> **Interactive demonstration (online)**: open the online edition of this tutorial to use this demonstration.Open demonstration

The slider below compares a synthetic test image (a disc, a square, a checkerboard and stripes) with a 5×5 Gaussian blur, $\sigma = 1.4$. The fine stripes and the checkerboard disappear first because a blur is a **low-pass filter**: it removes fast intensity variations while leaving slow ones.

> **Interactive demonstration (online)**: open the online edition of this tutorial to use this demonstration.Open demonstration

Code: from scratch, then OpenCV, SciPy and PyTorch

The loop below is the whole algorithm. Instead of looping over every pixel (slow in Python), it loops over the kernel taps and adds one shifted copy of the padded image per tap, which NumPy vectorizes. The script then checks that four libraries agree once you match the definition (correlation versus convolution) and the border rule. It needs `numpy`, `scipy`, `opencv-python` and `torch`.

```
import numpy as np
import cv2
import torch
import torch.nn.functional as F
from scipy import ndimage, signal

def correlate2d(img, K, mode="edge"):
    """Cross-correlation of a 2-D float image with an odd-sized kernel.
    mode is any np.pad mode: "constant" (zeros), "edge", "reflect", "wrap"."""
    kh, kw = K.shape
    ph, pw = kh // 2, kw // 2
    padded = np.pad(img, ((ph, ph), (pw, pw)), mode=mode)
    H, W = img.shape
    out = np.zeros((H, W), dtype=np.float64)
    for i in range(kh):              # loop over kernel taps, not pixels:
        for j in range(kw):          # each step adds one shifted copy of the image
            out += K[i, j] * padded[i:i + H, j:j + W]
    return out

def convolve2d(img, K, mode="edge"):
    return correlate2d(img, K[::-1, ::-1], mode)     # convolution = correlation with
     ↪  flipped K

rng = np.random.default_rng(0)
img = rng.random((32, 40))
K = rng.standard_normal((3, 5))                      # deliberately asymmetric

ours_corr = correlate2d(img, K, "reflect")
ours_conv = convolve2d(img, K, "reflect")
# OpenCV: filter2D computes correlation; BORDER_REFLECT_101 is np.pad's "reflect"
cv_corr = cv2.filter2D(img, cv2.CV_64F, K, borderType=cv2.BORDER_REFLECT_101)
# SciPy: ndimage.correlate / ndimage.convolve with mode="mirror" (= reflect-101)
sp_corr = ndimage.correlate(img, K, mode="mirror")
sp_conv = ndimage.convolve(img, K, mode="mirror")
# PyTorch: conv2d is correlation; pad first with the same rule
t = torch.tensor(img)[None, None]
t = F.pad(t, (2, 2, 1, 1), mode="reflect")
pt_corr = F.conv2d(t, torch.tensor(K)[None, None])[0, 0].numpy()

for name, a in [("OpenCV", cv_corr), ("SciPy", sp_corr), ("PyTorch", pt_corr)]:
    print(f"{name:8s} correlation matches:", np.allclose(ours_corr, a))
print("SciPy    convolution matches:", np.allclose(ours_conv, sp_conv))
print("conv == corr for this kernel?", np.allclose(ours_conv, ours_corr))
# zero padding: our convolution equals signal.convolve2d with mode="same"
same = signal.convolve2d(img, K, mode="same", boundary="fill")
```

```
print("zero-padded 'same' convolution matches:", np.allclose(convolve2d(img, K, "constant"),
 ↪  same))
```

All checks print `True` except `conv == corr`, which is `False` because the random kernel is asymmetric. Note the naming trap: NumPy's `"reflect"` padding (`dcb|abcd|cba`, edge sample not repeated) is OpenCV's `BORDER_REFLECT_101` and SciPy's `"mirror"`, while SciPy's `"reflect"` repeats the edge sample (`dcba|abcd|dcba`).

The second snippet shows the two bugs that cause most beginner frustration with real images: integer saturation and the kernel-sum rule.

```
import numpy as np
import cv2
from skimage import data

img_u8 = data.camera()                                  # 512 x 512 uint8 test image
img = img_u8.astype(np.float32) / 255.0
sobel_x = np.array([[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]], np.float32)

gx = cv2.filter2D(img, -1, sobel_x)                     # float in, float out: negatives kept
gy = cv2.filter2D(img, -1, sobel_x.T)
mag = np.hypot(gx, gy)
print("float Sobel x range:", gx.min().round(3), gx.max().round(3))

gx_u8 = cv2.filter2D(img_u8, -1, sobel_x)               # uint8 in, uint8 out: saturated
print("uint8 Sobel x range:", gx_u8.min(), gx_u8.max(),
      "| pixels clipped to 0:", f"{(gx_u8 == 0).mean():.0%}")

blur = cv2.GaussianBlur(img, (5, 5), 1.4)               # 5 x 5 Gaussian, sigma 1.4
print("mean brightness kept by blur:", np.isclose(img.mean(), blur.mean(), atol=1e-3))
K2 = 2 * np.ones((3, 3), np.float32) / 9                # weights sum to 2
print("mean after sum-2 kernel:", round(float(cv2.filter2D(img, -1, K2).mean() /
 ↪  img.mean()), 3))
```

In floating point the Sobel response ranges from about -3.4 to 3.3. With `uint8` input and output, every negative response is clipped to zero (more than half the pixels here), so all light-to-dark edges vanish. Always filter in `float32` and convert back only for display.

Border handling

Plain version. Near the edge of the image, part of the kernel hangs outside. You must decide what values to pretend are there.

The common choices are: **zero** (constant) padding, which pretends the world outside is black; **replicate**, which repeats the edge pixel; **reflect**, which mirrors the image at its border; and **wrap**, which treats the image as periodic, as the discrete Fourier transform implicitly does. OpenCV's default, `BORDER_REFLECT_101`, mirrors without repeating the edge pixel [11]. PyTorch's `nn.Con`

`v2d` uses zero padding unless you set `padding_mode` to `'reflect'`, `'replicate'` or `'circular'` [31].

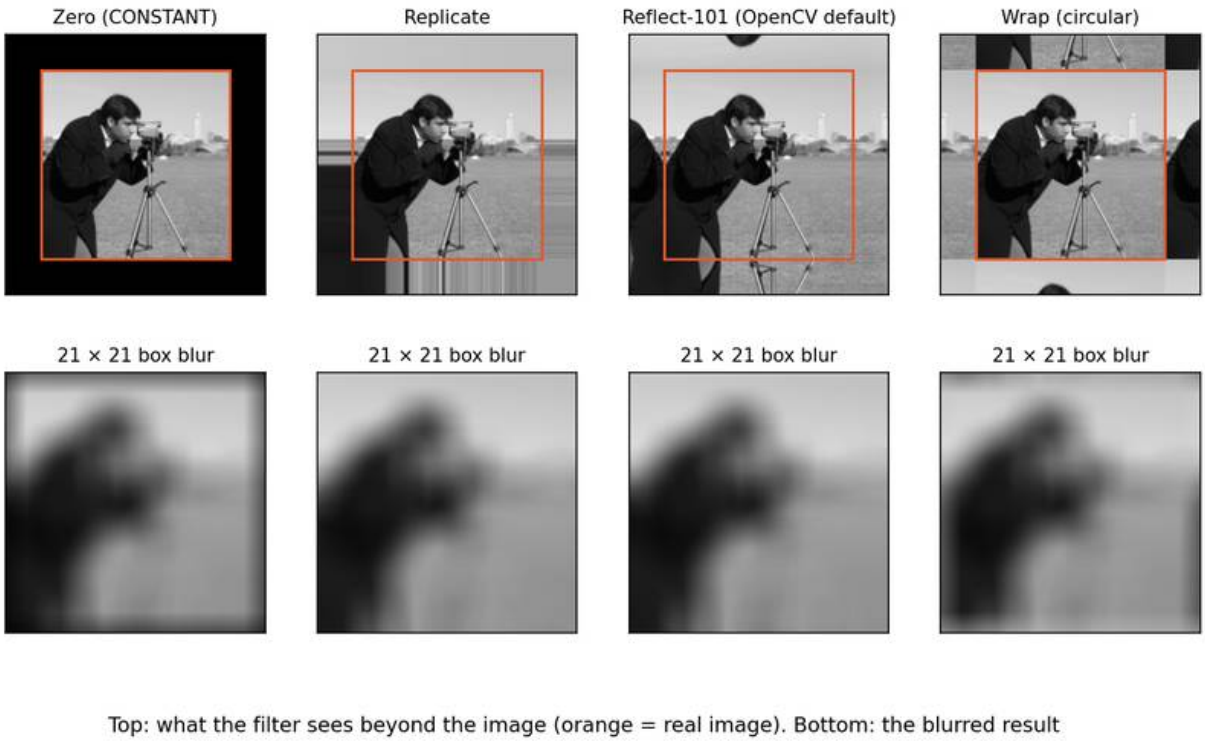


Figure 4 — Four border rules and their effect on a large 21 × 21 box blur. Zero padding darkens every border; wrap blends opposite edges; replicate and reflect keep flat borders flat.

Zero padding darkens the borders of a blurred image because black pixels are averaged in. Wrap mixes the top of the image into the bottom. Replicate and reflect are usually the least visible choices for photographs. With small kernels the difference touches only a thin frame, but in a deep CNN every layer adds its own frame: after many zero-padded layers, a large fraction of the feature map has "seen" the artificial border, which lets networks infer absolute position from padding.

Part II • Convolutional neural networks

From a fixed filter to a learned layer

Plain version. A convolutional layer is a bank of filters whose weights are learned by gradient descent. Each filter looks at all input channels at once and produces one output channel, called a **feature map**.

Precise version. Let the input be $X \in \mathbb{R}^{C_{\text{in}} \times H \times W}$ (channels, height, width) and the weights $K \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}} \times k \times k}$ with bias $b \in \mathbb{R}^{C_{\text{out}}}$. A stride-one layer with zero padding p computes

$$Y_{o,u,v} = b_o + \sum_{c=1}^{C_{\text{in}}} \sum_{i=0}^{k-1} \sum_{j=0}^{k-1} K_{o,c,i,j}\, X_{c,\,u+i-p,\,v+j-p}.$$

o indexes output channels (filters), c input channels, (u, v) output positions and (i, j) kernel taps; positions outside the image read the padding value. A nonlinearity such as ReLU is then applied element-wise. An RGB filter has three $k \times k$ slices, one per color channel, and their responses are **summed** into a single output number per position: a filter does not produce three separate images.

Two structural assumptions are built into this formula:

- **Local connectivity**: each output depends only on a $k \times k$ neighborhood, not on the whole image.
- **Weight sharing** (parameter sharing): the same K is used at every position (u, v).

Together they are the CNN's **inductive bias**, an assumption about which functions are likely. Its payoff is efficiency. A layer with $C_{\text{in}} = 3$, $C_{\text{out}} = 32$ and $k = 3$ has $32 \times (3 \cdot 9 + 1) = 896$ parameters, whatever the image size. A fully connected layer from a flattened $224 \times 224 \times 3$ image to just 32 numbers needs $150{,}528 \times 32 \approx 4.8$ million weights, and it would have to learn the same edge detector separately at every position.

Goodfellow et al. [1] summarize the motivation as sparse interactions, parameter sharing and equivariance. Figure 5 shows what a tiny CNN learns in its first layer when trained on the 8×8 digits dataset used in the classifier code later on. The learned 3×3 filters look like oriented derivative and blob detectors: the network rediscovered Sobel-like kernels because they are useful, not because anyone told it to. Zeiler and Fergus [30] showed the same phenomenon at scale by visualizing the filters and feature maps of large ImageNet networks.

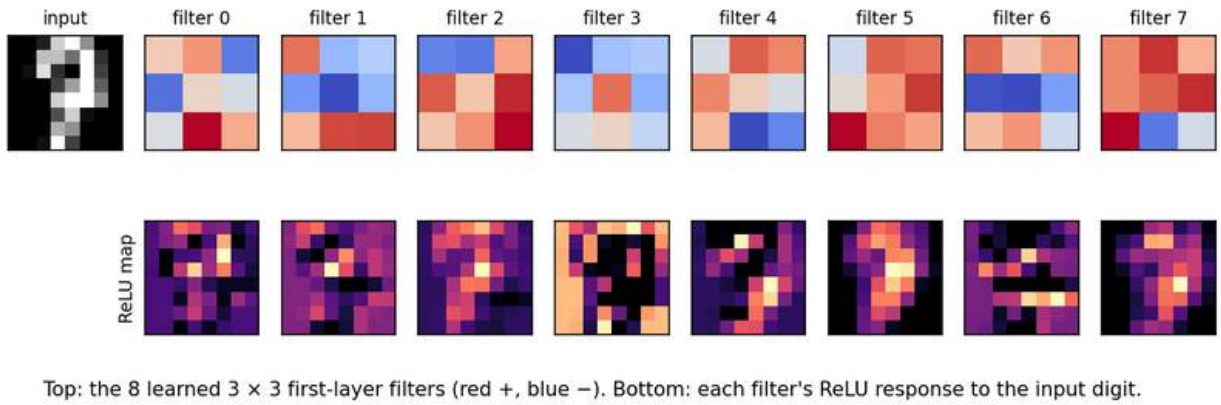


Figure 5 — First-layer filters learned on 8×8 digits (top) and their ReLU responses to one test digit (bottom). Many look like oriented edge or stroke detectors.

Padding, stride and dilation

Plain version. Padding controls what happens at the border and whether the output keeps its size. Stride skips positions to shrink the output. Dilation spreads the kernel taps apart to see farther without adding weights.

For input size n (height or width), kernel size k, padding p on each side, stride s and dilation d, the output size is [12]

$$n_{\text{out}} = \left\lfloor \frac{n + 2p - d(k-1) - 1}{s} \right\rfloor + 1.$$

$d(k-1)+1$ is the **effective kernel size**: dilation d inserts $d-1$ gaps between taps. Three padding conventions have names: **valid** ($p = 0$, output shrinks by $k-1$), **same** ($p = (k-1)/2$ for odd k and $s = 1$, output keeps its size) and **full** ($p = k-1$, every partial overlap counted, output grows by $k-1$). Stride two roughly halves each spatial dimension and is a common alternative to pooling for downsampling.

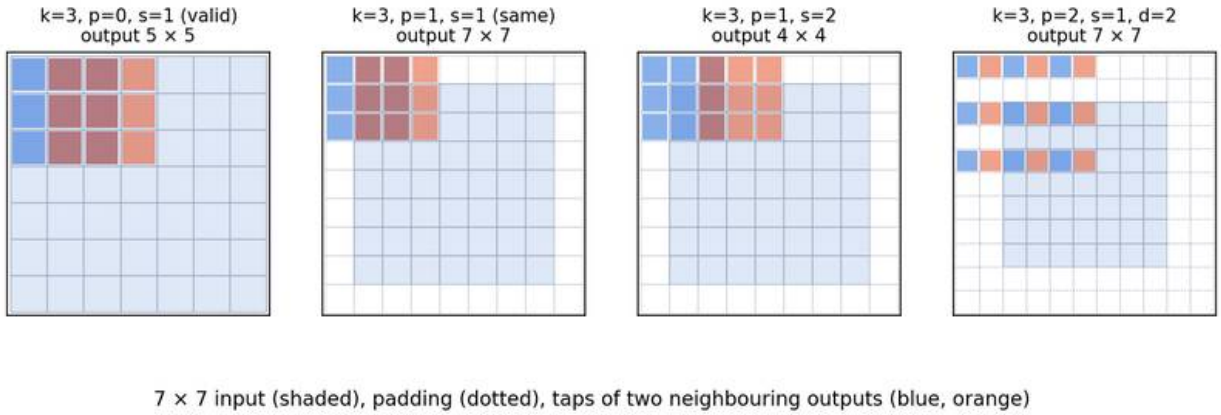


Figure 6 — Taps of two neighboring outputs (blue and orange; overlap appears brown) under four settings. Stride moves the window by s; dilation spaces the taps d apart.

Dilated (atrous) convolutions were popularized for dense prediction by Yu and Koltun [18]: stacking layers with dilations 1, 2, 4, ... grows the field of view exponentially while keeping full resolution. A single dilated layer sees only a sparse lattice of inputs, so dilations should be mixed with care to avoid "gridding" artifacts. A **transposed convolution** runs the forward map of a strided convolution backward to upsample; it reappears in the backpropagation section below. The convolution arithmetic guide by Dumoulin and Visin [12] has diagrams for every combination of these settings.

The snippet checks the formula against PyTorch, counts parameters, shows that a 1×1 convolution is a per-pixel linear layer, and measures translation equivariance:

```
import torch
from torch import nn
```

```
def out_size(n, k, p=0, s=1, d=1):
    return (n + 2 * p - d * (k - 1) - 1) // s + 1

x = torch.randn(1, 3, 64, 64)                          # one RGB 64 x 64 image
for k, p, s, d in [(3, 1, 1, 1), (5, 2, 2, 1), (3, 2, 1, 2), (1, 0, 1, 1)]:
    conv = nn.Conv2d(3, 16, k, stride=s, padding=p, dilation=d)
    y = conv(x)
    n_params = sum(q.numel() for q in conv.parameters())
    assert y.shape[-1] == out_size(64, k, p, s, d)
    assert n_params == 16 * (3 * k * k + 1)
    print(f"k={k} p={p} s={s} d={d}: output {tuple(y.shape[1:])}, parameters {n_params}")

# A 1x1 convolution is the same linear layer applied at every pixel
conv1 = nn.Conv2d(16, 4, kernel_size=1)
lin = nn.Linear(16, 4)
lin.weight.data = conv1.weight.data[:, :, 0, 0].clone()
lin.bias.data = conv1.bias.data.clone()
z = torch.randn(2, 16, 8, 8)
same = torch.allclose(conv1(z), lin(z.permute(0, 2, 3, 1)).permute(0, 3, 1, 2), atol=1e-6)
print("1x1 conv == per-pixel Linear:", same)

# Translation equivariance away from borders (stride 1, no padding effects in the centre)
conv = nn.Conv2d(1, 4, 3, padding=1, bias=False)
img = torch.zeros(1, 1, 32, 32); img[..., 10:14, 8:12] = 1
shift = lambda t: torch.roll(t, shifts=(3, 5), dims=(-2, -1))
with torch.no_grad():
    a, b = conv(shift(img)), shift(conv(img))
print("shift then conv == conv then shift:", torch.allclose(a, b, atol=1e-6))
conv2 = nn.Conv2d(1, 4, 3, padding=1, stride=2, bias=False)
with torch.no_grad():
    y0, y1 = conv2(img), conv2(torch.roll(img, 1, dims=-1))   # shift by one pixel
print("stride 2, 1-pixel shift: max output change =", round((y0 - y1).abs().max().item(), 3))
```

It prints output shapes $16 \times 64 \times 64$, $16 \times 32 \times 32$, $16 \times 64 \times 64$ and $16 \times 64 \times 64$ with 448, 1216, 448 and 64 parameters, confirms the 1×1 and equivariance checks, and shows that a one-pixel shift changes a stride-two output by a clearly nonzero amount; the exact value depends on the randomly initialized weights.

Channels and 1×1 convolutions

Plain version. Channels are the "what" dimension of a feature map; height and width are the "where". A 1×1 convolution mixes channels at each pixel without looking at neighbors.

A 1×1 convolution with C_{in} inputs and C_{out} outputs multiplies the channel vector at every pixel by the same $C_{\text{out}} \times C_{\text{in}}$ matrix, as the snippet verified. Lin et al.'s Network in Network [17] used this idea as a tiny per-pixel MLP and introduced **global average pooling** as a classifier head. GoogLeNet [16] used 1×1 layers to reduce channels before expensive 3×3 and 5×5 branches, and ResNet's bottleneck block [5] wraps a 3×3 layer between a 1×1 reduction and a 1×1 expansion. In every case the motivation is the same: spatial filtering costs k^2 times more than channel mixing, so do the cheap part with 1×1 layers and keep the expensive part narrow.

Pooling and downsampling

Plain version. Pooling summarizes each small block of a feature map by a single number, usually the maximum or the average. It shrinks the map and makes the representation less sensitive to small shifts.

Max pooling with a 2 × 2 window and stride two keeps the largest value in each block: "was this feature present anywhere here?". **Average pooling** keeps the mean. **Global average pooling** averages each channel over all positions, producing one number per channel regardless of input size, which is why modern classifiers can accept different image sizes. Pooling has no learnable parameters. Its gradient is simple: max pooling routes the whole upstream gradient to the winning input of each window; average pooling spreads it equally over the window.

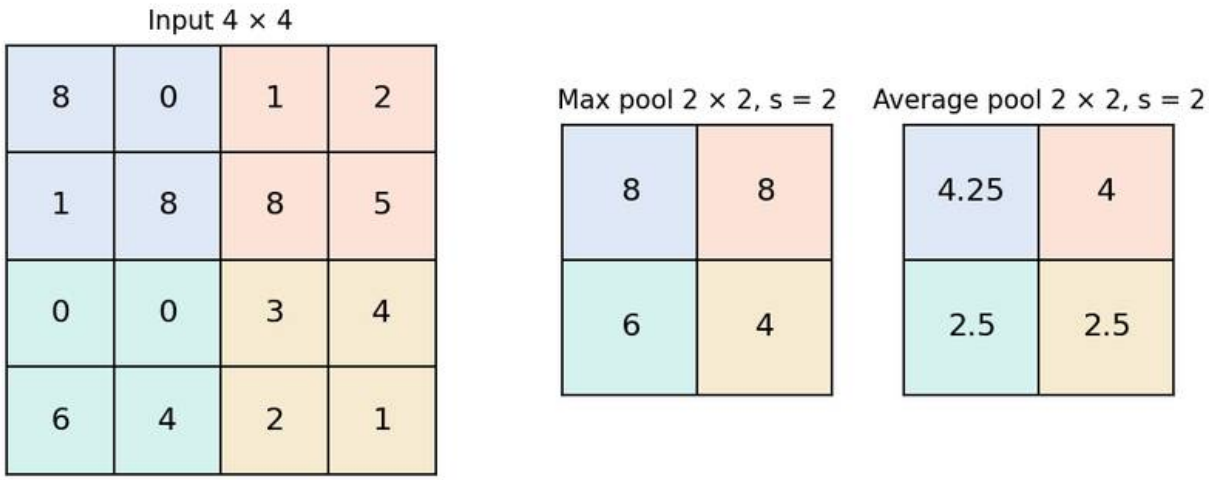


Figure 7 — Max and average pooling with a 2 × 2 window and stride 2. Each colored block becomes one number.

Downsampling, by pooling or by strided convolution, saves computation and enlarges later receptive fields, but it is also **subsampling** without a proper low-pass filter. High frequencies alias, and the output can change noticeably when the input moves by a single pixel, as the snippet above measured. Zhang [20] showed that inserting a small blur before subsampling ("anti-aliased" or "blur pooling") makes standard CNNs measurably more shift-invariant: a neat case of a classical signal-processing rule fixing a deep network.

Equivariance is not invariance

Plain version. Shift the input, and a convolutional feature map shifts with it. That is not the same as the final prediction staying unchanged.

Let T_Δ shift an image by Δ pixels. Away from the borders, a stride-one convolution is **translation-equivariant**: $f(T_\Delta X) = T_\Delta f(X)$. A classifier wants **invariance**: $g(T_\Delta X) = g(X)$. CNNs get approximate invariance by stacking equivariant layers and then pooling, especially global average

pooling, which discards position. Padding, strides, cropping and the finite image break exact equivariance. Rotation and scale equivariance are **not** built in; data augmentation or specially designed group-equivariant layers must supply them.

The distinction matters in practice. A detector must be equivariant: moving a person should move the predicted box. A classifier should be invariant: moving the person should not change the label. The same backbone serves both, with different heads.

Receptive field

Plain version. Early layers see small pieces of the image; deeper layers combine those pieces and see larger regions.

The **receptive field** of a unit is the set of input pixels that can influence it. Let r_ℓ be its width after layer ℓ, and j_ℓ the **jump**, the distance in input pixels between adjacent units of layer ℓ. Starting from $r_0 = j_0 = 1$, a layer with kernel size k_ℓ, dilation d_ℓ and stride s_ℓ gives

$$r_\ell = r_{\ell-1} + (d_\ell(k_\ell - 1))\, j_{\ell-1}, \qquad j_\ell = s_\ell\, j_{\ell-1}.$$

Three stacked stride-one 3×3 layers see 7×7 pixels, the same area as one 7×7 layer but with $3 \cdot 9C^2 = 27C^2$ instead of $49C^2$ weights and three nonlinearities instead of one; this was the central design argument of VGG [15]. A stride-two layer doubles the jump, so every later layer grows the field twice as fast.

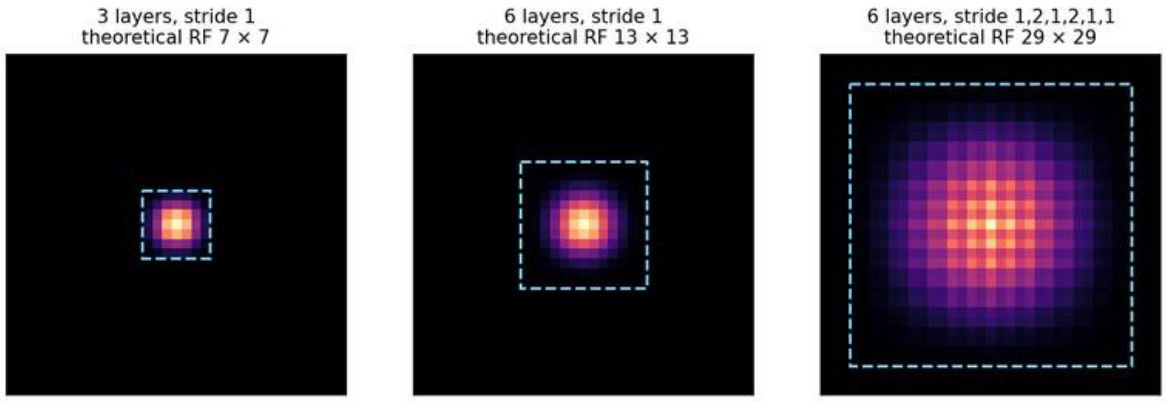


Figure 8 — Theoretical receptive field (dashed) versus actual influence, measured as the input gradient of one central output through stacks of 3 × 3 averaging layers. Influence is Gaussian-like and concentrated in the center; strides add a checkerboard pattern.

The theoretical field says which pixels **could** matter. Luo et al. [3] measured the **effective receptive field** via input gradients, as in Figure 8, and found that it is roughly Gaussian and occupies only a fraction of the theoretical field, growing like the square root of depth for uniform layers. Two practical lessons follow: a network may need more context than its theoretical field suggests,

and downsampling, dilation and large kernels are ways to enlarge the effective field without unbounded depth.

> **Interactive demonstration (online)**: open the online edition of this tutorial to use this demonstration.Open demonstration

Backpropagation through a convolution

Plain version. A convolution is linear, so its backward pass is also a convolution: the gradient with respect to the kernel correlates the input with the upstream gradient, and the gradient with respect to the input convolves the upstream gradient with the kernel.

Precise version. Take one channel, no padding, stride one: $Y_{u,v} = \sum_{i,j} K_{i,j} X_{u+i,v+j}$. Let L be the loss and $G_{u,v} = \partial L / \partial Y_{u,v}$ the **upstream gradient** delivered by backpropagation, as in ML 02. Each weight $K_{i,j}$ is used at every output position, so by the chain rule its gradient sums over them:

$$\frac{\partial L}{\partial K_{i,j}} = \sum_{u,v} G_{u,v}\, X_{u+i,v+j}.$$

That is a valid correlation of the input X with G, which is why weight sharing does not complicate training: shared weights simply accumulate gradients from every location. Each input pixel $X_{m,n}$ contributes to every output whose window covers it, at tap $(i,j) = (m-u, n-v)$:

$$\frac{\partial L}{\partial X_{m,n}} = \sum_{i,j} G_{m-i,n-j}\, K_{i,j},$$

with G taken as zero outside its range. This is a **full convolution** of G with K, equivalently a correlation of the zero-padded G with the flipped kernel. It is exactly what a **transposed convolution** computes, which is how transposed convolutions got their name. Bias gradients are just $\sum_{u,v} G_{u,v}$ per output channel. With multiple channels, the input gradient also sums over output channels, and the kernel gradient also sums over the batch.

```
import torch
import torch.nn.functional as F

torch.manual_seed(0)
X = torch.randn(1, 1, 6, 7, requires_grad=True)     # input
K = torch.randn(1, 1, 3, 3, requires_grad=True)     # kernel
Y = F.conv2d(X, K)                                   # 'valid' correlation: 4 x 5
G = torch.randn_like(Y)                              # pretend upstream gradient dL/dY
(Y * G).sum().backward()                             # so that dL/dY = G exactly

# 1) dL/dK: correlate the input with the upstream gradient
dK = F.conv2d(X.detach(), G)
# 2) dL/dX: 'full' convolution of G with K = correlation of zero-padded G with flipped K
dX = F.conv2d(F.pad(G, (2, 2, 2, 2)), K.detach().flip(-1, -2))
# 3) the same thing is what a transposed convolution computes
```

```
dX_t = F.conv_transpose2d(G, K.detach())

print("dL/dK matches autograd:", torch.allclose(dK, K.grad, atol=1e-5))
print("dL/dX matches autograd:", torch.allclose(dX, X.grad, atol=1e-5))
print("conv_transpose2d gives the same:", torch.allclose(dX_t, X.grad, atol=1e-5))
```

All three lines print `True`. The trick `(Y * G).sum().backward()` makes autograd use a chosen upstream gradient G, a handy way to test any hand-derived backward pass.

Classic architectures in brief

Plain version. Every famous CNN is a stack of convolutions, nonlinearities and downsampling. What changed over the years was depth, how to keep deep stacks trainable, and how to spend computation wisely.

Model (year)	Key idea	What it taught
LeNet-5 (1998) [13]	conv → subsample → conv → subsample → fully connected, trained end to end	Learned convolutional features beat hand-designed ones for digits
AlexNet (2012) [14]	Deeper and wider, ReLU, dropout, data augmentation, GPU training	Large labeled data plus GPUs made CNNs win ImageNet
VGG (2014) [15]	Only 3×3 convolutions, 16–19 layers	Depth with small kernels is simple and effective
GoogLeNet / Inception (2014) [16]	Parallel 1×1, 3×3, 5×5 branches with 1×1 reductions	Multi-scale features at controlled cost
ResNet (2015) [5]	Residual connections $y = x + F(x)$, BatchNorm	Very deep networks (100+ layers) become trainable
U-Net (2015) [4]	Encoder–decoder with skip connections at each resolution	Dense, pixel-aligned outputs from few images
MobileNetV2 (2018) [7]	Depthwise-separable convolutions, inverted residuals, linear bottlenecks	Accurate models for phones
EfficientNet (2019) [8]	Compound scaling of depth, width and resolution	Scale dimensions together, not one at a time
ConvNeXt (2022) [9]	7×7 depthwise kernels, LayerNorm, inverted bottlenecks, transformer-era training	A modernized pure CNN competes with vision transformers

The pattern from LeNet to ResNet is a steady increase in depth. AlexNet [14] showed that a deep CNN trained on ImageNet with GPUs, ReLU and dropout could beat traditional pipelines by a wide margin. VGG [15] argued for uniform stacks of 3×3 layers, as computed in the receptive-field section. GoogLeNet [16] made width cheap with 1×1 reductions. Then depth hit an optimization wall: plain networks with more layers had **higher training error**, which is not overfitting but a failure to optimize. ResNet [5] fixed it with identity shortcuts. DLCV 02 follows this line further, through training recipes, transfer learning and evaluation.

Residual blocks and normalization

Plain version. Instead of asking a block to compute the whole output, ask it to compute a correction to its input.

A residual block computes $y = x + F_\theta(x)$, where F_θ is typically two or three convolutions with normalization and ReLU. Its Jacobian is $\partial y/\partial x = I + \partial F_\theta/\partial x$: even if F_θ contributes little, the identity path carries the signal forward and the gradient backward. If $F_\theta \approx 0$ is the best a block can do, it can learn that easily, so adding blocks no longer hurts optimization. When the shape changes (more channels, stride two), the shortcut becomes a projection $P(x)$, usually a strided 1×1 convolution.

Batch normalization [19] standardizes each channel using the batch mean and variance during training, then applies a learned scale and shift; at evaluation it uses running averages. It makes training faster and less sensitive to initialization, but it misbehaves with very small or correlated batches, where **GroupNorm** [6] (statistics over groups of channels within each sample) is more reliable. Always call `model.eval()` before validation so BatchNorm uses its running statistics.

```
import torch
from torch import nn

count = lambda m: sum(p.numel() for p in m.parameters())

C = 64
dense = nn.Conv2d(C, C, 3, padding=1, bias=False)
separable = nn.Sequential(
    nn.Conv2d(C, C, 3, padding=1, groups=C, bias=False),   # depthwise: one 3x3 per channel
    nn.Conv2d(C, C, 1, bias=False))                        # pointwise: mix channels
print("dense:", count(dense), "| depthwise+pointwise:", count(separable),
      "| ratio:", round(count(separable) / count(dense), 4), "vs 1/C + 1/9 =",
      round(1 / C + 1 / 9, 4))

class BasicBlock(nn.Module):
    """ResNet-style block: y = relu(x + F(x)), with a 1x1 projection when shapes change."""
    def __init__(self, c_in, c_out, stride=1):
        super().__init__()
        self.f = nn.Sequential(
            nn.Conv2d(c_in, c_out, 3, stride, 1, bias=False), nn.BatchNorm2d(c_out),
            ↪ nn.ReLU(),
            nn.Conv2d(c_out, c_out, 3, 1, 1, bias=False), nn.BatchNorm2d(c_out))
        self.proj = (nn.Identity() if stride == 1 and c_in == c_out else
                     nn.Sequential(nn.Conv2d(c_in, c_out, 1, stride, bias=False),
                                   nn.BatchNorm2d(c_out)))
    def forward(self, x):
        return torch.relu(self.proj(x) + self.f(x))

x = torch.randn(2, 64, 32, 32)
print("same shape :", tuple(BasicBlock(64, 64)(x).shape))
print("downsample :", tuple(BasicBlock(64, 128, stride=2)(x).shape))
```

The output reports 36,864 weights for the dense layer against 4,672 for the separable pair, a ratio

of 0.1267 that matches the formula below, and block outputs of shape $64 \times 32 \times 32$ and $128 \times 16 \times 16$.

Efficient CNNs separate spatial filtering from channel mixing

Plain version. Looking around (spatial filtering) and combining features (channel mixing) need not happen in one expensive operation.

A dense $k \times k$ convolution has $k^2 C_{\text{in}} C_{\text{out}}$ weights. A **depthwise** convolution filters each channel separately ($k^2 C_{\text{in}}$ weights, `groups=C` in PyTorch); a following **pointwise** 1×1 convolution mixes channels ($C_{\text{in}} C_{\text{out}}$ weights). With $C_{\text{in}} = C_{\text{out}} = C$,

$$\frac{\text{depthwise} + \text{pointwise}}{\text{dense}} = \frac{k^2 C + C^2}{k^2 C^2} = \frac{1}{C} + \frac{1}{k^2}.$$

For $k = 3$ and large C this approaches $1/9$.

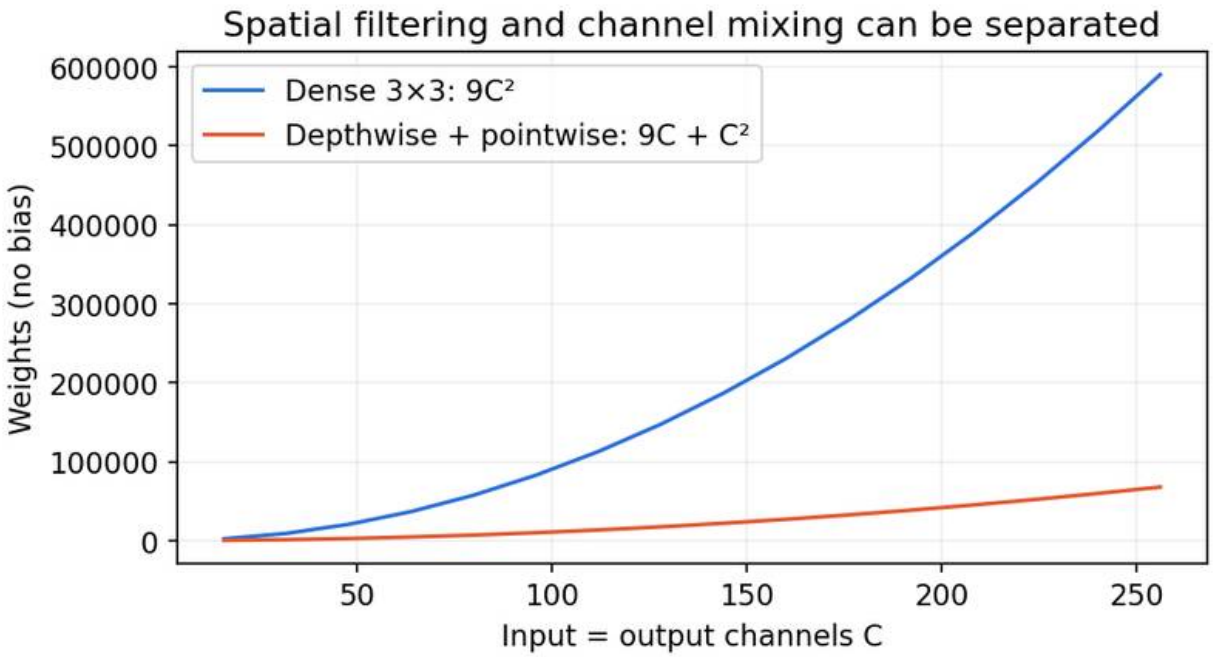


Figure 9 — Analytical weight counts for equal input and output channels. Fewer weights do not automatically mean lower latency: depthwise layers have low arithmetic intensity and may run slowly on some hardware.

MobileNetV2 [7] builds blocks that expand channels with a 1×1 layer, filter with a depthwise 3×3, then project back with a linear (no ReLU) 1×1 bottleneck, with a residual connection between the narrow ends. EfficientNet [8] scales depth, width and input resolution together with a single compound coefficient. ConvNeXt [9] started from a ResNet and applied, one at a time, design choices associated with vision transformers: patchify stem, 7×7 depthwise kernels, inverted bottlenecks, fewer activations, LayerNorm, and longer training with heavy augmentation. The re-

sult matched Swin Transformers of similar size, evidence that much of the transformer advantage came from training recipes and macro-design rather than from attention alone.

Code: a complete small-image classifier

This example needs `torch`, `numpy` and `scikit-learn`. It uses the bundled 8×8 digits dataset, so no download or GPU is needed. A stratified split gives separate training, validation and test sets: validation picks the best epoch, and the test set is evaluated once at the end. It is a teaching baseline, not an ImageNet recipe.

```
import copy
import numpy as np
import torch
from torch import nn
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

torch.manual_seed(0)
torch.set_num_threads(1)
d = load_digits()
X = torch.tensor(d.images[:, None] / 16.0, dtype=torch.float32)
y = torch.tensor(d.target, dtype=torch.long)
train, test = train_test_split(np.arange(len(y)), test_size=.2,
                               stratify=d.target, random_state=0)
train, val = train_test_split(train, test_size=.2,
                              stratify=d.target[train], random_state=1)
model = nn.Sequential(
    nn.Conv2d(1, 16, 3, padding=1), nn.ReLU(),
    nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(),
    nn.MaxPool2d(2),                     # B × 32 × 4 × 4
    nn.Flatten(), nn.Linear(32 * 4 * 4, 10),
)
opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
best_loss, best_state = float("inf"), None
for epoch in range(12):
    model.train()
    order = torch.tensor(train)[torch.randperm(len(train))]
    for idx in order.split(64):
        loss = nn.functional.cross_entropy(model(X[idx]), y[idx])
        opt.zero_grad()
        loss.backward()
        opt.step()
    model.eval()
    with torch.no_grad():
        val_loss = nn.functional.cross_entropy(model(X[val]), y[val]).item()
    if val_loss < best_loss:
        best_loss = val_loss
        best_state = copy.deepcopy(model.state_dict())
model.load_state_dict(best_state)
model.eval()
with torch.no_grad():
```

```
    accuracy = (model(X[test]).argmax(1) == y[test]).float().mean().item()
print("parameters:", sum(p.numel() for p in model.parameters()))
print("best validation loss:", round(best_loss, 4))
print("held-out accuracy:", round(accuracy, 4))
```

On our CPU run it reports 9,930 parameters and about 96% held-out accuracy (exact numbers vary slightly across PyTorch versions). Cross-entropy receives **logits**, with no softmax in front. The head flattens the $32 \times 4 \times 4$ map rather than global-average-pooling it, because on tiny digits the position of strokes carries information; for variable-size images, adaptive or global pooling gives a fixed-size head.

Inspect the errors, not just the score. Which digit pairs get confused? Does shifting every test digit by one pixel hurt? Compare against a parameter-matched MLP and a classical SVM, as in IP 13, before crediting every gain to convolution.

Modern view

Survey literature. Several reviews map the CNN landscape, each with a different emphasis. Gu et al. [23] (*Pattern Recognition*, 2018) organize CNN advances by component: convolution variants, pooling, activations, loss functions, regularization and optimization, followed by applications. Khan et al. [22] (*Artificial Intelligence Review*, 2020) propose a taxonomy of architectures by their main innovation: spatial exploitation (kernel sizes), depth, multi-path designs (shortcuts, Inception branches), width, feature-map exploitation (channel attention), and attention-based designs. Li et al. [21] (*IEEE TNNLS*, 2022) cover the history from LeNet onward, compare one-, two- and multi-dimensional convolutions, and survey lightweight networks and applications with rules of thumb for choosing hyperparameters. Alzubaidi et al. [2] (*Journal of Big Data*, 2021) give a broad, application-oriented review of deep learning centered on CNN architectures, with discussion of data scarcity, transfer learning and hardware. These surveys predate ConvNeXt, so read them as maps of the 2012–2020 era.

CNNs after transformers (2022–2026). Vision transformers (ML 04) prompted a re-examination of convolution, and the main finding is that the gap was smaller than early comparisons suggested:

- **Modernized CNNs.** ConvNeXt [9] showed that a carefully modernized ResNet matches Swin. ConvNeXt V2 [24] (CVPR 2023) added masked-autoencoder pretraining with sparse convolutions and a global response normalization layer, bringing self-supervised pretraining (see ML 05) to pure CNNs.
- **Large kernels.** RepLKNet [26] (CVPR 2022) used depthwise kernels as large as 31×31, with re-parameterized small kernels to ease optimization, to enlarge the effective receptive field. UniRepLKNet [27] (CVPR 2024) extended large-kernel design to audio, video, point clouds and time series.
- **Dynamic and deformable convolution.** InternImage [25] (CVPR 2023) scaled deformable convolution, whose sampling locations are predicted from the input, into a large foundation backbone for detection and segmentation.

- **Scale.** Smith et al. [29] (2023) pretrained NFNet CNNs on the large JFT-4B image dataset and found that their performance followed similar compute scaling trends to vision transformers under comparable budgets, arguing that compute and data, more than architecture, dominate at scale.
- **Edge devices.** MobileNetV4 [28] (ECCV 2024) designed blocks and a neural architecture search for good latency across CPUs, GPUs, DSPs and accelerators, an acknowledgment that parameter counts alone do not predict speed.

Where CNNs fit in 2026. Convolution remains a default for latency-constrained and on-device inference, for dense prediction at high resolution (restoration, segmentation, medical imaging, see DLCV 04), for small datasets where the locality bias helps, and as the stem or early stages of hybrid CNN–transformer models. Large vision foundation models are mostly transformer-based, but they frequently contain convolutions in their patch embeddings, upsampling decoders and diffusion U-Nets.

Open problems. Exact shift equivariance under downsampling (aliasing [20]); building rotation and scale symmetry in without heavy cost; understanding what the effective receptive field really needs to be for a task [3]; and fair comparison across architectures, which requires matching data, augmentation, resolution, schedule and compute before attributing a difference to convolution versus attention.

Key takeaways

- A kernel slides over the image and computes a weighted sum at every position. Libraries usually compute **correlation**; true convolution flips the kernel, which matters only for asymmetric, hand-designed kernels.
- The sum of a kernel's weights sets its response to flat regions: sum 1 blurs while keeping brightness; sum 0 detects changes. Filter in floating point, and choose the border rule on purpose.
- A convolutional layer is a bank of learned filters with **local connectivity** and **weight sharing**. Its parameter count, $C_{\text{out}}(C_{\text{in}}k^2 + 1)$, does not depend on image size.
- Padding, stride and dilation set the output size, $\lfloor (n + 2p - d(k-1) - 1)/s \rfloor + 1$. 1×1 convolutions mix channels; pooling and strides downsample but can alias.
- Convolution is shift-**equivariant**; invariance comes from pooling. Receptive fields grow with depth and stride, but the effective field is much smaller than the theoretical one.
- Backpropagation through a convolution is itself a convolution: kernel gradients correlate input with upstream gradient, and input gradients are a transposed convolution.
- Architecture history is a story of depth (LeNet → VGG → ResNet), efficient channel use (1×1, depthwise-separable) and better training. Modernized CNNs remain competitive with transformers when budgets are matched.

Exercises

1. **Design and verify.** Write a 3×3 kernel that detects only horizontal edges, and another that detects only diagonal edges running from top-left to bottom-right. Verify both in the kernel playground or with `cv2.filter2D` on a synthetic image containing a horizontal, a vertical and a diagonal line.

   Hint

   A horizontal edge is a change along y, so use $S_y = S_x^\top$ (rows $-1,-2,-1$ / $0,0,0$ / $1,2,1$). For a diagonal edge, put the derivative across the diagonal, for example $\begin{bmatrix} 0 & 1 & 2 \\ -1 & 0 & 1 \\ -2 & -1 & 0 \end{bmatrix}$. Its weights sum to zero, so flat regions give zero.

2. **Kernel sums.** Why should a blur kernel's weights sum to one? Predict, then check, what happens to a mid-gray `uint8` image filtered with a 3×3 kernel whose weights all equal $2/9$, and what changes if you filter in `float32` instead.

   Hint

   A flat region of value c maps to $c \sum K$. With sum two, a gray of 128 becomes 256, which saturates at 255 in `uint8`; every brighter-than-mid-gray region turns white. In `float32` the values double without clipping, and you can rescale afterward.

3. **Composition.** Show that applying a 3×3 box blur twice equals one 5×5 kernel, and write that kernel. Verify numerically away from the borders, and explain why the equality can fail at the borders.

   Hint

   By associativity, the combined kernel is the box convolved with itself: the outer product of $[1,2,3,2,1]$ with itself, divided by 81. At the border, the first pass already invented padding values, and the second pass pads that result; a single 5×5 pass pads the original image instead, so the two differ in a frame of width two.

4. **Layer arithmetic.** For a 64×64 RGB input and a layer with 16 filters, $k = 5$, $p = 2$, $s = 2$: give the output shape and the number of parameters. Then compute (r, j) after three 3×3 layers with strides 1, 2, 1, and compare with all strides equal to one.

   Hint

   $\lfloor (64 + 4 - 4 - 1)/2 \rfloor + 1 = 32$, so the output is $16 \times 32 \times 32$ with $16(3 \cdot 25 + 1) = 1216$ parameters. Receptive field: $(3,1) \to (5,2) \to (9,2)$. All strides one gives $(7,1)$.

5. **Gradient by hand.** For a 1×4 input $X = [x_1, x_2, x_3, x_4]$ and a kernel $K = [k_1, k_2]$ with valid correlation, write Y, then derive $\partial L/\partial K$ and $\partial L/\partial X$ in terms of $G = \partial L/\partial Y$. Check your formulas with the autograd trick from the backpropagation snippet.

   Hint

   $Y = [k_1x_1 + k_2x_2,\ k_1x_2 + k_2x_3,\ k_1x_3 + k_2x_4]$. Then $\partial L/\partial k_1 = G_1x_1 + G_2x_2 + G_3x_3$ and $\partial L/\partial k_2 = G_1x_2 + G_2x_3 + G_3x_4$ (a correlation of X with G). $\partial L/\partial X = [G_1k_1,\ G_1k_2 + G_2k_1,\ G_2k_2 + G_3k_1,\ G_3k_2]$, the full convolution of G with K.

6. **Ablation on digits.** In the classifier, replace `Flatten` + `Linear(512, 10)` by global average pooling + `Linear(32, 10)`. Compare parameter counts, validation loss and test accuracy, then evaluate both models on test digits shifted by one pixel. Explain the trade-off you observe.

   Hint

   `nn.AdaptiveAvgPool2d(1)` then `nn.Flatten()`. Pooling removes most head parameters and position information, which can help shift robustness but hurt on 8×8 digits where stroke position matters. Keep everything else fixed, select on validation only, and report both accuracy and parameter count.

References

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016, ch. 9. Book
2. L. Alzubaidi et al., "Review of deep learning: concepts, CNN architectures, challenges, applications, future directions," *Journal of Big Data*, 2021. Review
3. W. Luo, Y. Li, R. Urtasun and R. Zemel, "Understanding the Effective Receptive Field in Deep Convolutional Neural Networks," NeurIPS, 2016. Paper
4. O. Ronneberger, P. Fischer and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," MICCAI, 2015. Paper
5. K. He, X. Zhang, S. Ren and J. Sun, "Deep Residual Learning for Image Recognition," CVPR, 2016. Paper
6. Y. Wu and K. He, "Group Normalization," ECCV, 2018. Paper
7. M. Sandler et al., "MobileNetV2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018. Paper
8. M. Tan and Q. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," ICML, 2019. Paper
9. Z. Liu et al., "A ConvNet for the 2020s," CVPR, 2022. Paper
10. S. J. D. Prince, *Understanding Deep Learning*, MIT Press, 2023, ch. 10. Book and notebooks
11. OpenCV, "Image Filtering" (`filter2D`, border types), documentation, version 4.x. Documentation
12. V. Dumoulin and F. Visin, "A guide to convolution arithmetic for deep learning," arXiv:1603.07285, 2016. arXiv
13. Y. LeCun, L. Bottou, Y. Bengio and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, 1998. DOI

14. A. Krizhevsky, I. Sutskever and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” NeurIPS, 2012. Paper
15. K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” ICLR, 2015. arXiv
16. C. Szegedy et al., “Going Deeper with Convolutions,” CVPR, 2015. arXiv
17. M. Lin, Q. Chen and S. Yan, “Network In Network,” ICLR, 2014. arXiv
18. F. Yu and V. Koltun, “Multi-Scale Context Aggregation by Dilated Convolutions,” ICLR, 2016. arXiv
19. S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” ICML, 2015. arXiv
20. R. Zhang, “Making Convolutional Networks Shift-Invariant Again,” ICML, 2019. arXiv
21. Z. Li, F. Liu, W. Yang, S. Peng and J. Zhou, “A Survey of Convolutional Neural Networks: Analysis, Applications, and Prospects,” *IEEE Transactions on Neural Networks and Learning Systems*, 2022. DOI
22. A. Khan, A. Sohail, U. Zahoora and A. S. Qureshi, “A Survey of the Recent Architectures of Deep Convolutional Neural Networks,” *Artificial Intelligence Review*, 2020. arXiv
23. J. Gu et al., “Recent Advances in Convolutional Neural Networks,” *Pattern Recognition*, 2018. arXiv
24. S. Woo et al., “ConvNeXt V2: Co-designing and Scaling ConvNets with Masked Autoencoders,” CVPR, 2023. arXiv
25. W. Wang et al., “InternImage: Exploring Large-Scale Vision Foundation Models with Deformable Convolutions,” CVPR, 2023. arXiv
26. X. Ding et al., “Scaling Up Your Kernels to 31x31: Revisiting Large Kernel Design in CNNs” (RepLKNet), CVPR, 2022. arXiv
27. X. Ding et al., “UniRepLKNet: A Universal Perception Large-Kernel ConvNet for Audio, Video, Point Cloud, Time-Series and Image Recognition,” CVPR, 2024. arXiv
28. D. Qin et al., “MobileNetV4: Universal Models for the Mobile Ecosystem,” ECCV, 2024. arXiv
29. S. L. Smith, A. Brock, L. Berrada and S. De, “ConvNets Match Vision Transformers at Scale,” arXiv:2310.16764, 2023. arXiv
30. M. D. Zeiler and R. Fergus, “Visualizing and Understanding Convolutional Networks,” ECCV, 2014. arXiv
31. PyTorch, `torch.nn.Conv2d` documentation. Documentation

ML 03A • Recurrent and Recursive Networks

Learn from sequences and trees: shared recurrent state, backpropagation through time, vanishing and exploding gradients, LSTM and GRU gates, bidirectional and encoder–decoder models, attention as the bridge to Transformers, recursive networks, and their uses in vision.

Online edition: https://blog.shyandram.dev/en/learn/ml/ml-03a-recurrent-recursive-networks/

What you'll learn

- How a recurrent network reuses one set of weights at every time step, so it can read sequences of any length.
- How backpropagation through time (BPTT) trains it, and why gradients over long spans vanish or explode.
- How LSTM and GRU gates build a protected memory path, written out equation by equation and checked against PyTorch.
- How bidirectional, stacked and encoder–decoder (sequence-to-sequence) models are assembled, and how attention removed the encoder bottleneck and led to the Transformer.
- How recursive networks compose representations over trees instead of chains.
- Where recurrence is used in vision (captioning, video, spatiotemporal forecasting) and how Mamba and Mamba-2 use selective state spaces, parallel scans and fixed-size streaming memory.

The big picture

ML 03 used one assumption about images: nearby pixels are related, and the same pattern can appear anywhere, so the same filter is reused across **space**. Sequences invite the same trick across **time**. A video is a sequence of frames, a caption is a sequence of words, a pen stroke is a sequence of positions. A **recurrent neural network (RNN)** reads such a sequence one element at a time, carries a summary called the **hidden state** from step to step, and uses the same weights at every step [1]. A **recursive neural network** generalizes the idea from a chain to a tree: it combines parts into wholes by repeatedly applying one shared composition rule.

> **Explain like I'm five**
>
> When you listen to a story, you do not remember every word. You keep a little picture in your head of what has happened so far, and each new sentence updates that picture. A recurrent network does exactly this: it has a small notebook (the hidden state), reads one word or one video frame at a time, and rewrites the notebook after each one using the same rules every time. An LSTM is a notebook with an eraser and a pen that the network learns to use carefully, so important notes from the beginning are not smudged away by the end. A recursive network is like building with LEGO: snap two small

pieces into a bigger piece, then snap that into an even bigger piece, using the same kind of snap each time.

Why study RNNs when Transformers dominate large-scale language and vision? Three reasons. First, every concept in modern sequence modeling — teacher forcing, encoder–decoder, attention, masking, causal generation — was developed on RNNs, and the Transformer is easiest to understand as the answer to RNN limitations. Second, recurrence is efficient for **streaming**: a constant-size state per step suits robots, sensors and on-device video. Third, recurrence is back: modern state-space models such as Mamba are recurrences with clever parameterizations, and ML 04 applies them to images.

A recurrent state

Plain version. At every step, combine the current input with the previous state to make a new state, using the same weights each time. Read outputs off the state whenever you need them.

Precise version. Let the input sequence be $x_1, \dots, x_T$ with $x_t \in \mathbb{R}^d$, and let the hidden state be $h_t \in \mathbb{R}^m$ with a fixed starting state h_0 (usually zeros). The simple (Elman [9]) RNN is

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b), \qquad \hat{y}_t = g\big(W_y h_t + c\big).$$

$W_x \in \mathbb{R}^{m \times d}$ maps the input into the state space, $W_h \in \mathbb{R}^{m \times m}$ is the **recurrent** weight matrix that carries information from the previous step, b and c are biases, W_y maps the state to outputs, and g is an output function such as softmax. Crucially, none of these parameters depends on t. That is **parameter sharing across time**, the temporal version of a CNN's weight sharing: the model has the same number of parameters whether the sequence has 5 or 5,000 steps, and a pattern learned at step 3 can be recognized at step 300.

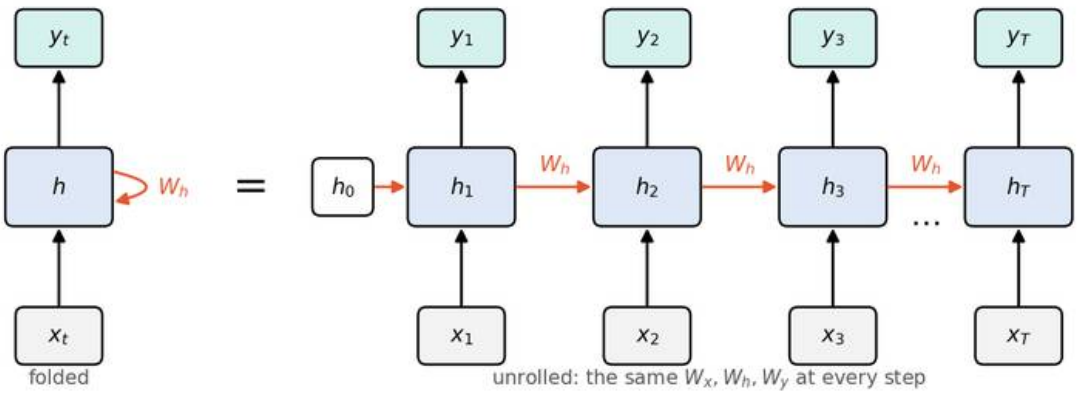


Figure 1 — Folded and unrolled views of an RNN. Unrolling turns the loop into a deep feed-forward network whose layers all share the same weights.

Unrolling (Figure 1) is the key mental move. Writing out the loop for T steps produces an ordinary feed-forward network that is T layers deep, in which every layer has the same weights. Everything you learned about backpropagation in ML 02 applies to this unrolled graph.

The same recurrence supports several input–output patterns:

Pattern	Example	Output read from
Many-to-one	Classify a video clip; sentiment of a sentence	The last state h_T (or a pooled state)
Many-to-many, aligned	Label every video frame; tag every word	Every h_t
One-to-many	Generate a caption from one image vector	A decoder that feeds its own outputs back in
Many-to-many, unaligned	Translate a sentence; describe a video	An **encoder–decoder** (covered below)

An RNN that predicts x_{t+1} from $x_{\le t}$ is an **autoregressive** model, and sampling from it generates new sequences; but an RNN used as an encoder is not automatically a generator. The head and the loss decide the task.

Code: an RNN and its backward pass in NumPy

The snippet below implements the forward pass and backpropagation through time by hand for a loss $L = v^\top h_T$, then checks one gradient entry against a finite difference. Notice the line `dWh += ...`: because W_h is used at every step, its gradient is the **sum** of contributions from all steps.

```
import numpy as np

rng = np.random.default_rng(0)
d, m, T = 3, 5, 7                                  # input size, state size, length
Wx = rng.normal(0, 0.5, (m, d))
Wh = rng.normal(0, 0.5, (m, m))
b = np.zeros(m)

def rnn_forward(xs, h0):
    hs = [h0]
    for x in xs:                                    # the SAME Wx, Wh, b at every step
        hs.append(np.tanh(Wx @ x + Wh @ hs[-1] + b))
    return hs

def rnn_backward(xs, hs, dhT):
    """BPTT for the loss L = dhT . h_T; returns dL/dWh and dL/dx_t for every t."""
    dWh, dxs, dh = np.zeros_like(Wh), [None] * len(xs), dhT
    for t in reversed(range(len(xs))):
        da = dh * (1 - hs[t + 1] ** 2)              # through tanh
        dWh += np.outer(da, hs[t])                  # shared weight: gradients ADD up
        dxs[t] = Wx.T @ da
        dh = Wh.T @ da                              # pass the gradient one step back
    return dWh, dxs

xs = rng.normal(size=(T, d))
hs = rnn_forward(xs, np.zeros(m))
dhT = rng.normal(size=m)
```

```
dWh, dxs = rnn_backward(xs, hs, dhT)

# finite-difference check of one entry of dL/dWh
eps, (i, j) = 1e-6, (2, 3)
Wh[i, j] += eps; up = dhT @ rnn_forward(xs, np.zeros(m))[-1]
Wh[i, j] -= 2 * eps; down = dhT @ rnn_forward(xs, np.zeros(m))[-1]
Wh[i, j] += eps
print("BPTT vs finite difference:", np.isclose(dWh[i, j], (up - down) / (2 * eps)))
print("|dL/dx_t| from first to last step:", np.round([np.linalg.norm(g) for g in dxs], 3))
```

It prints `True`, and the input-gradient norms are $[0.009, 0.015, 0.006, 0.029, 0.383, 0.212, 0.433]$: even in a seven-step sequence, the early inputs receive gradients roughly ten to fifty times smaller than the late ones. That is the vanishing-gradient problem in miniature.

Backpropagation through time

Plain version. Unroll the network, run ordinary backpropagation through the unrolled graph, and add up the gradients for each shared weight.

Precise version. Write the pre-activation as $a_t = W_x x_t + W_h h_{t-1} + b$, so $h_t = \tanh(a_t)$. Suppose the loss is a sum over steps, $L = \sum_t L_t(h_t)$. Working backward from $t = T$, the total gradient arriving at h_t is

$$\frac{\partial L}{\partial h_t} = \frac{\partial L_t}{\partial h_t} + \left(\frac{\partial h_{t+1}}{\partial h_t}\right)^{\top} \frac{\partial L}{\partial h_{t+1}}, \qquad \frac{\partial h_{t+1}}{\partial h_t} = \operatorname{diag}(1 - h_{t+1}^2)\, W_h.$$

The first term is the local loss at step t; the second is everything later that h_t influenced, passed back one step. With $\delta_t = (1 - h_t^2) \odot \partial L / \partial h_t$ (the gradient at the pre-activation), the parameter gradients are sums over time:

$$\frac{\partial L}{\partial W_h} = \sum_{t=1}^{T} \delta_t\, h_{t-1}^{\top}, \qquad \frac{\partial L}{\partial W_x} = \sum_{t=1}^{T} \delta_t\, x_t^{\top}, \qquad \frac{\partial L}{\partial b} = \sum_{t=1}^{T} \delta_t.$$

This algorithm, **backpropagation through time** (**BPTT**), was laid out clearly by Werbos [8]. Its cost is linear in T, but so is its memory: every h_t must be stored for the backward pass.

Truncated BPTT

For a very long stream (an hour of video, a continuous sensor log) storing all states is impossible. **Truncated BPTT** processes the stream in windows of k steps. It carries the hidden state forward from one window to the next, so the model still *sees* the whole past, but it **detaches** the state from the graph at each window boundary, so gradients flow back at most k steps. The model can therefore use long-range information only if it learns to preserve it from the gradient signal within windows.

```
import torch
from torch import nn

torch.manual_seed(0)
rnn, head = nn.GRU(1, 16, batch_first=True), nn.Linear(16, 1)
params = list(rnn.parameters()) + list(head.parameters())
opt = torch.optim.Adam(params, lr=1e-2)
t = torch.arange(0, 2000) * 0.05
stream = torch.sin(t) + 0.1 * torch.randn_like(t)   # one long noisy signal
x, y = stream[:-1, None][None], stream[1:, None][None]   # predict the next sample

h, window = None, 50
for start in range(0, x.shape[1] - window, window):
    out, h = rnn(x[:, start:start + window], h)
    loss = nn.functional.mse_loss(head(out), y[:, start:start + window])
    opt.zero_grad()
    loss.backward()                       # gradients flow back at most `window` steps
    nn.utils.clip_grad_norm_(params, 1.0)
    opt.step()
    h = h.detach()                        # keep the state, cut the graph
print("final window MSE:", round(loss.item(), 4), "(noise variance is 0.01)")
```

After a single pass over the stream, the final-window mean squared error is about 0.014, close to the 0.01 floor set by the added noise. Forgetting `h.detach()` makes the second window try to backpropagate into a graph that was already freed, and PyTorch raises an error; resetting `h = None` at each window instead would throw the context away.

Vanishing and exploding gradients

Plain version. The gradient that travels from step t back to step s is multiplied by roughly the same matrix at every step. Repeated multiplication either shrinks it toward zero or blows it up, so plain RNNs struggle to learn dependencies more than a few dozen steps long.

Precise version. Chaining the one-step Jacobians gives

$$\frac{\partial h_t}{\partial h_s} = \prod_{k=s+1}^{t} \frac{\partial h_k}{\partial h_{k-1}} = \prod_{k=s+1}^{t} \operatorname{diag}(1 - h_k^2)\, W_h,$$

an ordered product of $t - s$ matrices (matrices do not commute, so the order matters). Because $|\tanh'| \le 1$,

$$\left\| \frac{\partial h_t}{\partial h_s} \right\| \le \left(\sigma_{\max}(W_h)\right)^{t-s},$$

where $\sigma_{\max}$ is the largest singular value. If $\sigma_{\max}(W_h) < 1$, the gradient from distant steps **vanishes** exponentially. If W_h has large singular values and the units are not saturated, the gradient can **explode**. Bengio, Simard and Frasconi [6] showed in 1994 that this is not a bug of a particular training method: a system that robustly stores information in the presence of noise must be in

a regime where gradients of long-range dependencies shrink. Pascanu, Mikolov and Bengio [7] revisited the analysis with dynamical-systems tools and gave a practical remedy for explosions.

The intuition is clearest for a scalar linear RNN, $h_t = \lambda h_{t-1} + x_t$: then $\partial h_t / \partial h_s = \lambda^{t-s}$, which is $0.9^{80} \approx 2 \times 10^{-4}$ for $\lambda = 0.9$ and $1.1^{80} \approx 2 \times 10^{3}$ for $\lambda = 1.1$. Figure 2 shows the same behavior measured in real networks.

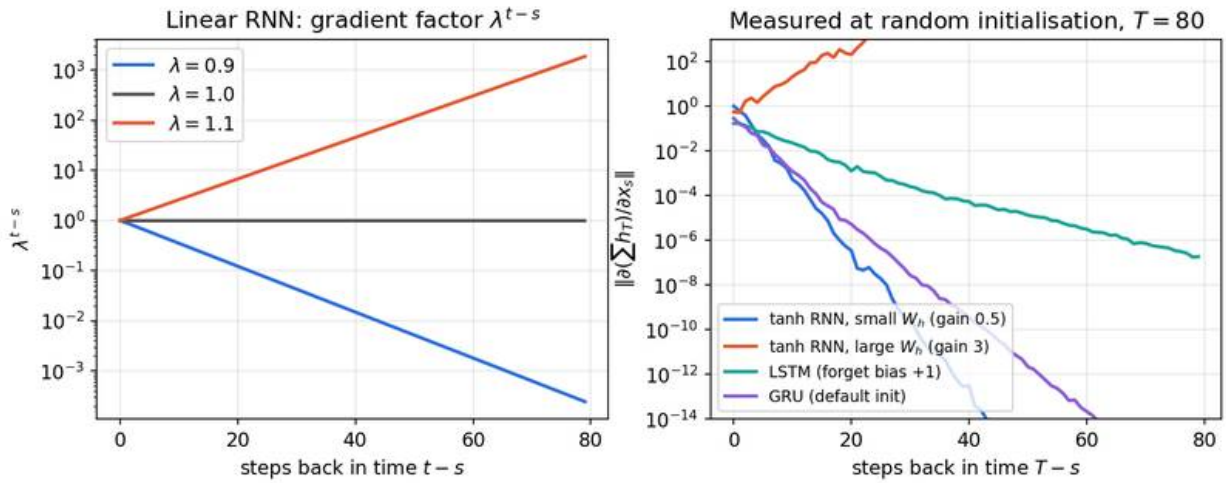


Figure 2 — Left: the scalar factor $\lambda^{(t-s)}$. Right: gradient of the final state with respect to each input, measured at random initialization ($T = 80$). The plain tanh RNN vanishes or explodes; the LSTM's additive memory path decays far more slowly.

Remedies. Explosions are handled by **gradient clipping** [7]: if the gradient norm $\|g\|$ exceeds a threshold τ, rescale it to $g \leftarrow \tau\, g / \|g\|$. This keeps the update direction and caps its length. PyTorch's `clip_grad_norm_` does exactly this. Vanishing gradients are harder: they require a different architecture, one whose state update is close to the identity so that gradients can pass through many steps. That architecture is the LSTM.

Long short-term memory (LSTM)

Plain version. Give the network a separate memory lane, the **cell state**, that is updated by addition rather than by repeated matrix multiplication, and let learned **gates** decide what to erase, what to write and what to reveal.

Precise version. With $[x_t, h_{t-1}]$ denoting concatenation and σ the logistic sigmoid, a standard LSTM step is

$$
\begin{aligned}
f_t &= \sigma\big(W_f[x_t, h_{t-1}] + b_f\big) && \text{forget gate} \\
i_t &= \sigma(W_i[x_t, h_{t-1}] + b_i) && \text{input gate} \\
o_t &= \sigma(W_o[x_t, h_{t-1}] + b_o) && \text{output gate} \\
\tilde{c}_t &= \tanh(W_c[x_t, h_{t-1}] + b_c) && \text{candidate memory} \\
c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t && \text{cell update} \\
h_t &= o_t \odot \tanh(c_t) && \text{exposed state}
\end{aligned}
$$

Each gate is a vector of numbers between 0 and 1, and $\odot$ is element-wise multiplication, so each gate acts like a bank of dimmer switches, one per memory unit. The forget gate f_t decides how much of the old memory to keep, the input gate i_t how much of the candidate $\tilde{c}_t$ to write, and the output gate o_t how much of the memory to reveal as h_t.

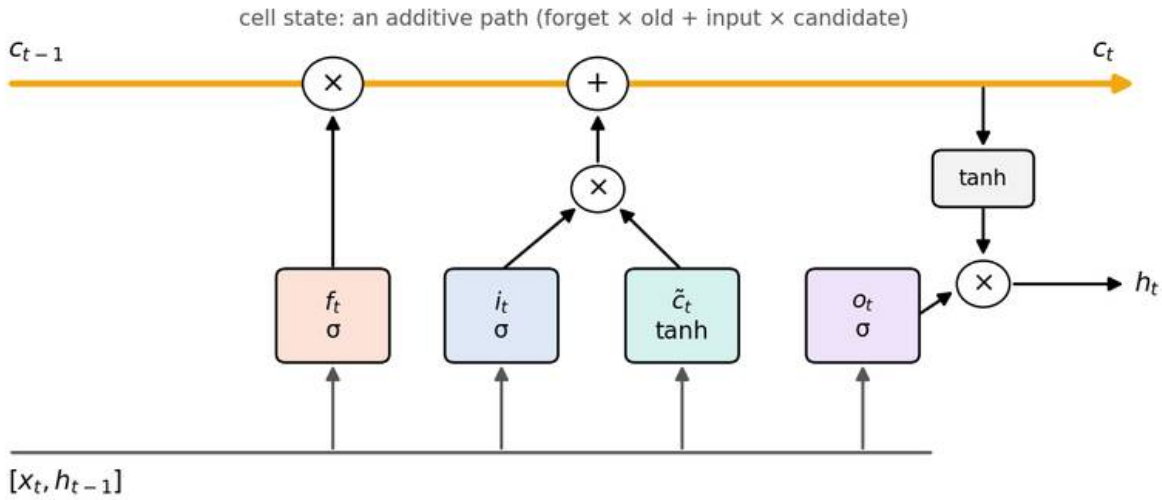


Figure 3 — Inside an LSTM cell. The cell state (gold) changes only by an element-wise scaling and an addition, which gives gradients a protected path through time.

Why it works. Along the cell-state path, $\partial c_t / \partial c_{t-1} = \mathrm{diag}(f_t)$ (ignoring the smaller indirect paths through the gates). There is no recurring weight matrix and no squashing nonlinearity on this path. If the forget gate stays near one, the gradient passes through many steps almost unchanged. That is why the LSTM curve in Figure 2 decays so slowly. It is also why a common initialization sets the forget-gate bias to a positive value such as +1, so the network starts out remembering. This is not a guarantee of unlimited memory: the forget gate can and does learn to close, and long-range credit assignment still gets harder with distance.

History and attribution. Hochreiter and Schmidhuber introduced the LSTM in 1997 [2] with a constant-error "carousel": the cell state passed on unchanged and there was **no forget gate**. Gers, Schmidhuber and Cummins added the forget gate in 2000 [5] so that the cell could learn to reset its memory, for example between independent segments of a continuous stream. The equations above, with the forget gate, are the version used in practice and in every major framework. Greff et al. [19] compared eight LSTM variants on speech, handwriting and music tasks and found that

none significantly beat the standard architecture, while the forget gate and the output activation were its most critical components.

Parameters. Each of the four gate blocks has an $m \times (d + m)$ weight matrix and a bias, so an LSTM layer has about $4m(d + m + 1)$ parameters, four times a simple RNN with the same state size. (PyTorch keeps separate input and recurrent biases, giving $4m(d + m + 2)$.)

Code: an LSTM step from scratch, checked against PyTorch

PyTorch stores the four gate blocks stacked in the order input, forget, cell candidate (g), output. Implementing the step by hand and comparing with `nn.LSTMCell` is the best way to be sure you read the equations correctly:

```
import torch
from torch import nn

torch.manual_seed(0)
d, m = 4, 6
cell = nn.LSTMCell(d, m)                # PyTorch stacks gate weights in order i, f, g, o

def lstm_step(x, h, c):
    z = x @ cell.weight_ih.T + cell.bias_ih + h @ cell.weight_hh.T + cell.bias_hh
    i, f, g, o = z.chunk(4, dim=-1)
    i, f, o = torch.sigmoid(i), torch.sigmoid(f), torch.sigmoid(o)
    g = torch.tanh(g)                   # candidate memory, often written c~_t
    c = f * c + i * g                   # additive cell update
    h = o * torch.tanh(c)
    return h, c

x = torch.randn(2, d)
h0, c0 = torch.randn(2, m), torch.randn(2, m)
h_ref, c_ref = cell(x, (h0, c0))
h_ours, c_ours = lstm_step(x, h0, c0)
print("matches nn.LSTMCell:", torch.allclose(h_ours, h_ref, atol=1e-6),
      torch.allclose(c_ours, c_ref, atol=1e-6))
print("parameters:", sum(p.numel() for p in cell.parameters()), "= 4 * m * (d + m + 2) =",
      4 * m * (d + m + 2))
```

It prints `True True` and `288 = ⋯ = 288`.

Gated recurrent unit (GRU)

Plain version. A GRU is a slimmer LSTM: it merges the cell and hidden states into one, and uses two gates instead of three.

Precise version. Following Cho et al. [3] and PyTorch's `nn.GRU`,

$$
\begin{aligned}
r_t &= \sigma(W_r x_t + U_r h_{t-1} + b_r) && \text{reset gate} \\
z_t &= \sigma(W_z x_t + U_z h_{t-1} + b_z) && \text{update gate} \\
\tilde{h}_t &= \tanh(W x_t + r_t \odot (U h_{t-1}) + b) && \text{candidate} \\
h_t &= z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t && \text{interpolation}
\end{aligned}
$$

The reset gate r_t decides how much of the old state to use when forming the candidate; the update gate z_t interpolates between keeping the old state ($z_t \approx 1$) and replacing it ($z_t \approx 0$). The interpolation plays the role of the LSTM's additive cell path. Two caveats are worth knowing. Some papers and textbooks swap the roles of z_t and $1 - z_t$; the models are equivalent, but the gate means the opposite. And the original paper applies the reset gate before the recurrent matrix ($U(r_t \odot h_{t-1})$), whereas PyTorch and cuDNN apply it after, as written above. Always check the framework's documentation before reading meaning into a gate.

GRUs have three gate blocks instead of four, so about 25% fewer parameters than an LSTM of the same width. Neither consistently wins across tasks; the GRU is a reasonable default when data or compute is limited.

Code: a recurrent classifier that generalizes to fresh sequences

This complete CPU example learns to classify the sign of a sequence's sum, which requires accumulating information over all twelve steps. A fresh test set checks that the model did not merely memorize its training sequences.

```
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0)
torch.set_num_threads(1)
rnn = nn.GRU(input_size=1, hidden_size=24, batch_first=True)
head = nn.Linear(24, 2)
opt = torch.optim.Adam(list(rnn.parameters())+list(head.parameters()), lr=.003)
def batch(n):
    x = torch.randn(n, 12, 1)
    y = (x.sum(dim=(1, 2)) > 0).long()
    return x, y
for step in range(400):
    x, y = batch(64)
    _, h = rnn(x)
    loss = F.cross_entropy(head(h[-1]), y)
    opt.zero_grad(); loss.backward()
    nn.utils.clip_grad_norm_(list(rnn.parameters())+list(head.parameters()), 1.)
    opt.step()
rnn.eval(); head.eval()
with torch.no_grad():
    x, y = batch(512)
    _, h = rnn(x)
```

```
    acc = (head(h[-1]).argmax(1) == y).float().mean()
    print("fresh-sequence accuracy:", acc.item())
```

It reaches about 99% accuracy on fresh length-12 sequences. The returned hidden tensor has shape `layers × batch × hidden` even with `batch_first=True`, which affects only the input and output sequence layout. Testing on longer sequences asks a different question, length generalization, and a model can succeed at length 12 and fail at length 100 (Exercise 2).

Deep, bidirectional and streaming RNNs

Stacking. A **deep RNN** feeds the sequence of hidden states of one recurrent layer into another (`num_layers` in PyTorch). Lower layers can capture short-range patterns and higher layers longer ones. Two to four layers are typical; deeper stacks usually need residual connections between layers.

Bidirectional. Schuster and Paliwal [10] proposed running one RNN forward in time and another backward, then concatenating their states, $h_t = [\vec{h}_t; \overleftarrow{h}_t]$. Each position then summarizes both its past and its future. This is ideal for **offline** tasks where the whole input is available (tagging a recorded video, encoding a sentence), but it is **not causal**: it cannot be used for a live prediction that must be made before future frames arrive. Reporting offline bidirectional accuracy for an online application overstates what the deployed system can know.

Variable lengths. Batches contain sequences of different lengths, padded to a common length. Without care, the padding leaks into states and losses. Use per-sequence lengths (`pack_padded_sequence` in PyTorch) or masks so that padded steps neither update the state nor contribute to the loss.

Streaming. For a live stream, keep the state between calls and reset it only between independent streams. Accidentally carrying state from one user, document or video into the next leaks information and produces irreproducible results.

Encoder–decoder models and teacher forcing

Plain version. To turn one sequence into another of a different length (a sentence into its translation, a video into a description), use one RNN to read the input into a summary and a second RNN to write the output from that summary, one token at a time.

Precise version. In the **sequence-to-sequence** (seq2seq) model of Sutskever, Vinyals and Le [11] and the closely related encoder–decoder of Cho et al. [3], an **encoder** RNN reads $x_1, \dots, x_T$ and its final state becomes a context vector $c = h_T$. A **decoder** RNN, initialized from c, models

$$p(y_1, \dots, y_U \mid x) = \prod_{u=1}^{U} p(y_u \mid y_{<u}, c), \qquad s_u = \mathrm{RNN}(s_{u-1}, y_{u-1}),$$

where s_u is the decoder state and $p(y_u \mid \cdot)$ is a softmax over the vocabulary. Generation stops when the decoder emits an end-of-sequence token, so input and output lengths are independent.

Teacher forcing. During training, the decoder input at step u is the **ground-truth** previous token y_{u-1}, not the model's own prediction. This makes all steps trainable in parallel against the known target and greatly stabilizes learning. At test time, however, the model must consume its own predictions, which may contain errors it never saw during training. This train–test mismatch is called **exposure bias**. Scheduled sampling [25] gradually replaces ground-truth inputs with model samples during training to reduce it. Decoding usually uses greedy or beam search rather than sampling.

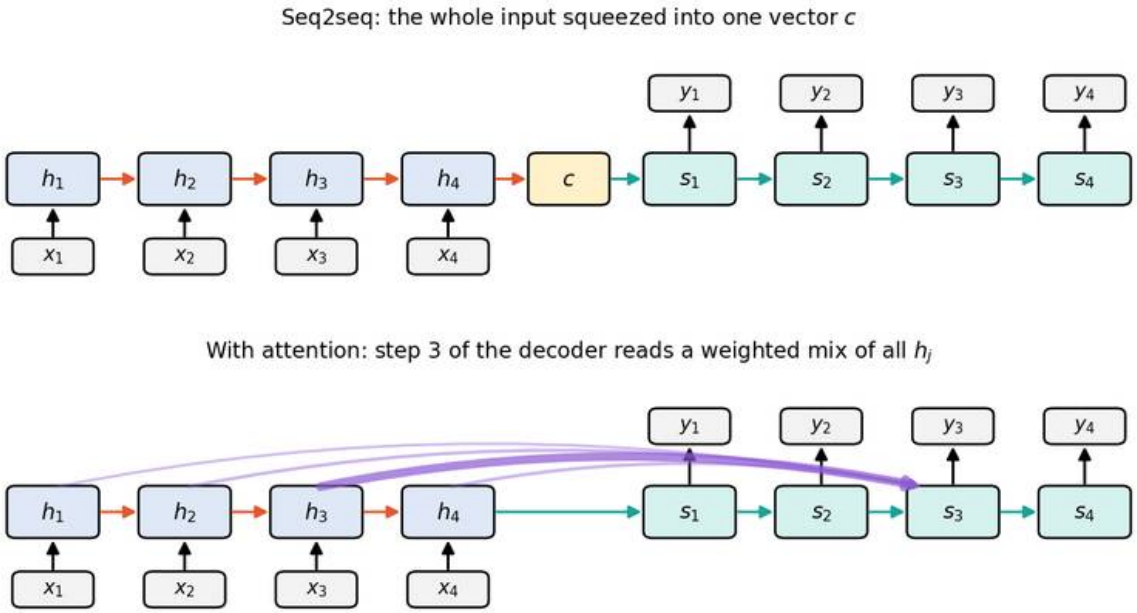


Figure 4 — Top: plain seq2seq squeezes the entire input into one vector c. Bottom: with attention, every decoder step reads a learned, weighted mix of all encoder states.

The weakness is visible in Figure 4 (top): everything the decoder knows about the input must pass through one fixed-size vector c, no matter how long the input. Translation quality of plain seq2seq models dropped on long sentences. Sutskever et al. found that simply **reversing the source sentence** helped, because it shortened the distance between corresponding words, a hint that the bottleneck and long paths were the problem [11].

Attention: the bridge to Transformers

Plain version. Instead of forcing the decoder to rely on one summary vector, let it look back at **all** encoder states at every step and decide which ones are relevant right now.

Precise version. Bahdanau, Cho and Bengio [12] kept every encoder state $h_1, \dots, h_T$ (from a bidirectional encoder) and, at each decoder step u, computed

$$e_{u,j} = v^\top \tanh\!\big(W s_{u-1} + U h_j\big), \qquad \alpha_{u,j} = \frac{\exp(e_{u,j})}{\sum_{k=1}^{T} \exp(e_{u,k})}, \qquad c_u = \sum_{j=1}^{T} \alpha_{u,j}\, h_j.$$

$e_{u,j}$ is a learned **alignment score** between the decoder's current need (s_{u-1}) and encoder position j; W, U and v are learned parameters. The softmax turns scores into weights $\alpha_{u,j}$ that are positive and sum to one, and the **context vector** c_u is their weighted average of encoder states. The decoder then uses c_u together with its own state. This form is called **additive attention**. Because c_u changes at every step and has direct access to every input position, the path length between any input and any output becomes one step, regardless of the sequence length.

```
import torch
from torch import nn

class AdditiveAttention(nn.Module):
    """Bahdanau-style scores e_j = v^T tanh(W s + U h_j), then softmax over j."""
    def __init__(self, d_dec, d_enc, d_att):
        super().__init__()
        self.W = nn.Linear(d_dec, d_att, bias=False)
        self.U = nn.Linear(d_enc, d_att, bias=False)
        self.v = nn.Linear(d_att, 1, bias=False)

    def forward(self, s, H, mask=None):
        # s: B x d_dec (decoder state), H: B x T x d_enc (all encoder states)
        e = self.v(torch.tanh(self.W(s)[:, None] + self.U(H))).squeeze(-1)   # B x T
        if mask is not None:                        # True where the input is padding
            e = e.masked_fill(mask, float("-inf"))
        alpha = e.softmax(dim=-1)                   # attention weights, rows sum to 1
        context = (alpha[:, :, None] * H).sum(1)    # B x d_enc weighted average
        return context, alpha

torch.manual_seed(0)
att = AdditiveAttention(d_dec=16, d_enc=32, d_att=8)
H = torch.randn(2, 5, 32)
s = torch.randn(2, 16)
pad = torch.tensor([[False] * 5, [False, False, False, True, True]])   # 2nd sequence has
 ↪  length 3
ctx, alpha = att(s, H, pad)
print("context:", tuple(ctx.shape), "| weights sum to 1:",
      torch.allclose(alpha.sum(-1), torch.ones(2)))
print("weights on padding:", alpha[1, 3:].tolist())
```

It prints a context of shape `(2, 32)`, confirms that weights sum to one, and shows exactly zero weight on padded positions: masking with $-\infty$ before the softmax is the standard way to make attention ignore padding (and, in ML 04, the future).

Attention weights are also interpretable as a soft **alignment**. Figure 5 shows a small GRU encoder–decoder with this attention trained for under a minute on CPU to reverse random digit sequences (about 96% per-token accuracy with teacher forcing on fresh sequences). Without being told, it learned to look at input position $T + 1 - u$ when writing output u: the anti-diagonal.

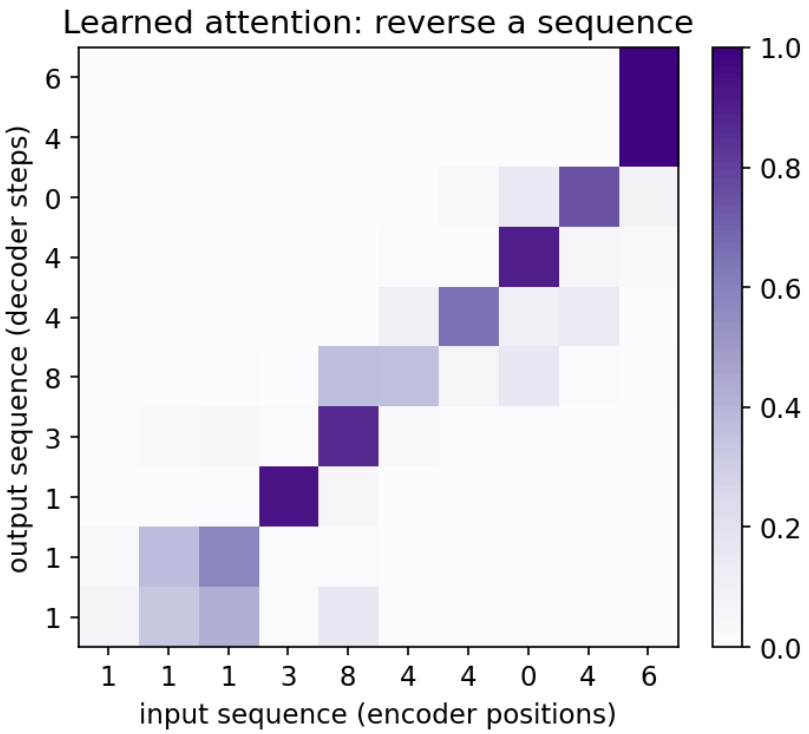


Figure 5 — Attention weights of a toy encoder–decoder trained to reverse a sequence (generated by the figure script). Each output step concentrates on the mirrored input position.

From here to Transformers. Attention fixed the bottleneck, but the encoder and decoder were still recurrent, so computation over a sequence of length T required T sequential steps that cannot be parallelized during training. Vaswani et al. [28] asked what happens if recurrence is removed entirely and **attention is used inside the encoder and decoder as well** (self-attention), with positional encodings to restore order. The answer, the Transformer, trains in parallel across positions and became the dominant architecture for language and, with ViT, for vision. ML 04 picks up the story with scaled dot-product attention, which replaces the additive score with $q^\top k / \sqrt{d_k}$.

Recurrent networks in vision

Image captioning. Show and Tell [13] (Vinyals et al.) is a direct image-to-sequence model: a CNN (ML 03) encodes the image into a vector that initializes an LSTM language model, which then generates the caption word by word, trained with teacher forcing on image–caption pairs. Show, Attend and Tell [14] (Xu et al.) kept the CNN's spatial feature grid instead of a single vector and let the LSTM attend over image regions at each word, producing attention maps that often highlight the object being named. This was one of the first widely seen demonstrations of visual attention.

Video understanding. Long-term Recurrent Convolutional Networks (LRCN) [15] (Donahue et al.) apply a CNN to each frame and an LSTM across frames, trained end to end, for activity

recognition, image description and video description. The CNN handles space, the RNN handles time: a common pattern for any per-frame feature sequence, such as pose tracks or per-frame detections in object tracking.

Spatiotemporal forecasting. When the state itself should be an image, fully connected gates waste the spatial structure. The **ConvLSTM** of Shi et al. [16] replaces the matrix products in the LSTM gates with convolutions, $i_t = \sigma(W_{xi} * X_t + W_{hi} * H_{t-1} + \dots)$, so inputs, states and gates are all 3-D tensors (channels × height × width). It was introduced for precipitation nowcasting from radar maps and is widely used for video prediction and other forecasting on grids.

What changed. For captioning and video classification at scale, Transformers and vision–language models (see ML 09) have largely replaced LSTMs, because they train in parallel and scale better with data. Recurrent and convolutional-recurrent models remain common where frames arrive one at a time and memory must stay constant: online video processing, tracking, robotics and low-power devices.

Recursive networks: composing over trees

Plain version. A recursive network applies one shared combination rule to merge two child representations into a parent, following a tree, until it reaches the root.

Precise version. For a binary tree whose leaves carry vectors (word embeddings, region features), each internal node computes

$$h_{\text{parent}} = \tanh\!\Big(W\,[h_{\text{left}}; h_{\text{right}}] + b\Big),$$

with the **same** $W \in \mathbb{R}^{D\times 2D}$ and b at every node. The root vector represents the whole structure and can be classified; every node can also have its own label. Training uses **backpropagation through structure**: the same idea as BPTT, but along the tree's edges. A chain of length n is the special case of a completely unbalanced tree; a balanced tree reduces the path from any leaf to the root from $n-1$ to about $\log_2 n$ steps (Figure 6), which shortens the route for gradients.

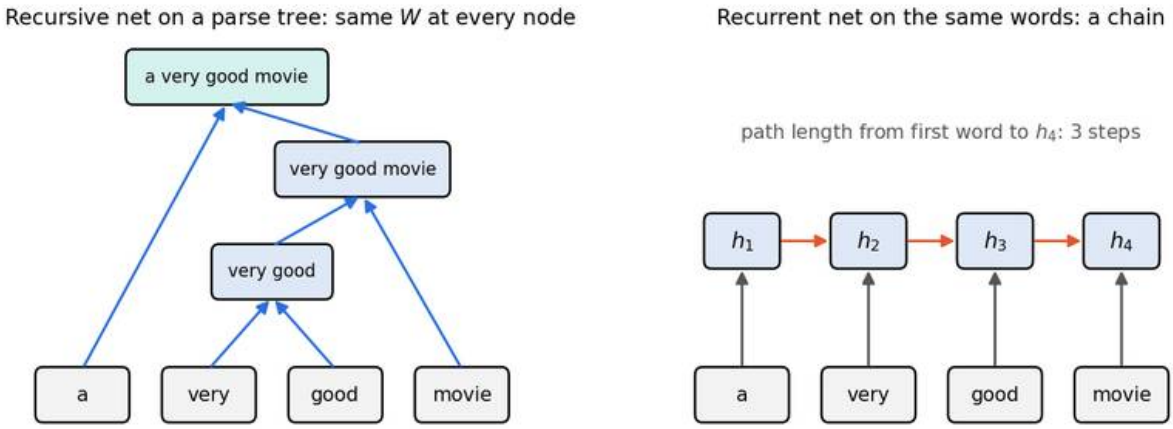


Figure 6 — A recursive network follows a tree (left); a recurrent network follows a chain (right). Both share weights, but the tree mirrors the structure of the input.

Socher et al. [18] used recursive networks in a vision setting first: they segmented an image into superpixels, then learned both to merge adjacent regions into a scene parse tree and to compute region representations, using the same method to parse sentences. Their later Recursive Neural Tensor Network and Stanford Sentiment Treebank [17] showed that composing along parse trees captures phenomena such as negation ("not very good"). Tai, Socher and Manning's **Tree-LSTM** [4] added LSTM-style gates to tree composition, with one forget gate per child, so a parent can selectively keep information from each subtree.

```
import torch
from torch import nn

torch.manual_seed(0)
D = 8
vocab = {"a": 0, "very": 1, "good": 2, "movie": 3}
emb = nn.Embedding(len(vocab), D)
compose = nn.Linear(2 * D, D)            # ONE weight matrix shared by every internal node
classify = nn.Linear(D, 2)               # e.g. negative / positive sentiment

def encode(tree):
    """tree is a word (leaf) or a pair (left, right)."""
    if isinstance(tree, str):
        return emb(torch.tensor(vocab[tree]))
    left, right = tree
    return torch.tanh(compose(torch.cat([encode(left), encode(right)])))

tree = ("a", (("very", "good"), "movie"))          # (a ((very good) movie))
root = encode(tree)
loss = nn.functional.cross_entropy(classify(root)[None], torch.tensor([1]))
loss.backward()                                     # backpropagation through structure
print("root vector:", tuple(root.shape), "| loss:", round(loss.item(), 4))
print("gradient reached the shared W:", compose.weight.grad.abs().sum().item() > 0)
swapped = encode((("movie", ("good", "very")), "a"))
print("swapping children changes the root:", not torch.allclose(root, swapped))
```

The recursion is ordinary Python, and autograd builds the tree-shaped graph on the fly. Swapping children changes the result because concatenation gives the left and right slots different roles.

The main practical limitation is the tree itself: someone has to provide it (a parser, a segmentation hierarchy), and batching variable trees efficiently is awkward. Learning structure and representation jointly is possible but harder. In current practice, self-attention can learn soft, input-dependent groupings without an explicit tree, which is one reason recursive networks are now uncommon; the idea of hierarchical composition lives on in hierarchical vision backbones and scene graphs.

Mamba: selective state spaces and scan

A useful memory should hold a relevant observation through distractions and overwrite it when the situation changes. Mamba [21] makes the memory update depend on the current input, while keeping that update affine in the previous state. This distinction lets us retain a recurrent interpretation and compute whole sequences with a parallel prefix scan.

From continuous dynamics to selective updates

Start with a small continuous-time memory. A stable transition lets stored information decay; an input projection writes new information; an output projection reads it:

$$\frac{dh(\tau)}{d\tau} = Ah(\tau) + Bu(\tau), \qquad y(\tau) = Ch(\tau).$$

Here τ is continuous time, $u(\tau) \in \mathbb{R}$ is one input channel, $h(\tau) \in \mathbb{R}^N$ is its N-dimensional state, $A \in \mathbb{R}^{N\times N}$ is the transition matrix, $B \in \mathbb{R}^{N\times 1}$ writes the input, $C \in \mathbb{R}^{1\times N}$ reads the state, and $y(\tau) \in \mathbb{R}$ is the output. A full neural block mixes multiple channels with learned projections and includes additional local convolution and gating [21]. This equation isolates its memory mechanism.

Hold the input and its write matrix constant during each interval. Exact zero-order-hold discretization gives

$$\bar{A}_t = \exp(\Delta_t A), \qquad \bar{B}_t = \left(\int_0^{\Delta_t} \exp(sA)\, ds \right) B_t,$$
$$h_t = \bar{A}_t h_{t-1} + \bar{B}_t u_t, \qquad y_t = C_t h_t.$$

Here t indexes discrete tokens, $\Delta_t > 0$ is the interval length, s is an integration variable, u_t is the held input, B_t, C_t are the current write/read matrices, and bars mark discrete coefficients. The integral remains valid when A is singular; an inverse-based formula needs additional assumptions. A diagonal A makes these operations elementwise.

Selectivity means predicting Δ_t, B_t, C_t from the current token. For example, a learned affine projection followed by $\mathrm{softplus}(z) = \log(1 + e^z)$ supplies a positive interval. The underlying learned A can remain fixed: $\bar{A}_t$ nevertheless changes through Δ_t. With negative diagonal entries, a larger interval reduces the fraction of old state retained. Selective writes and reads also determine which content enters and leaves memory. The resulting map is nonlinear in its inputs even though each state update is affine. Figure 7 isolates the retention effect with synthetic coefficients; it does not measure a trained model.

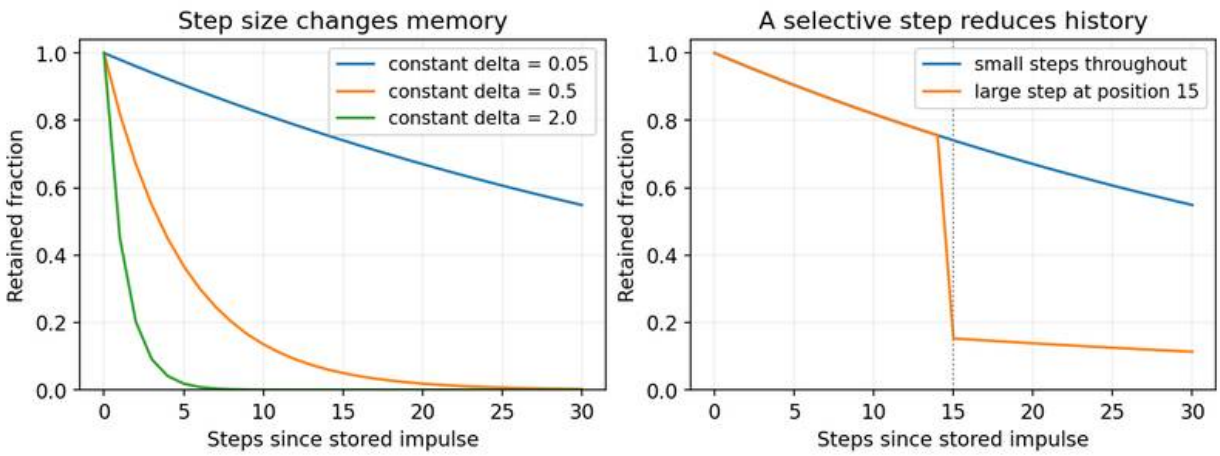


Figure 7 — For a scalar transition $A = -0.4$, retained history is the product of $\exp(-0.4\ \text{delta})$. A large selected interval sharply reduces old memory; small intervals preserve it longer.

Why this recurrence can be scanned

Once input-dependent coefficients are computed, composing two affine updates requires only another affine update:

$$(a_2, b_2) \circ (a_1, b_1) = (a_2 a_1,\ a_2 b_1 + b_2).$$

Here pair (a_i, b_i) represents $h \mapsto a_i h + b_i$, $i = 1, 2$ identifies consecutive updates, and $\circ$ means applying the first update and then the second. Composition is associative; for a diagonal transition, products are elementwise. A prefix scan can therefore regroup the operations into a tree while preserving their chronological effect. Ordinary LSTM/GRU gates depend on the previous hidden state, so their coefficients cannot generally be prepared this way before the state is known.

The following standalone CPU example uses exact diagonal discretization and hand-chosen selective coefficients. It checks causality and state continuity across chunks. It is a teaching recurrence, not a pretrained Mamba implementation or a speed benchmark.

```
import numpy as np

rng = np.random.default_rng(44)
u = rng.normal(size=24)
A = -np.array([0.1, 0.8])  # two stable diagonal entries

def selective_scan(values, state=None):
    h = np.zeros(2) if state is None else state.copy()
    outputs = []
    for x in values:
        delta = np.logaddexp(0.0, 0.5 * x)
        B = np.array([1.0, np.tanh(x)])
        C = np.array([1.0, 0.5 + 0.1 * np.tanh(x)])
        a = np.exp(delta * A)
```

```
        b = np.expm1(delta * A) / A * B * x
        h = a * h + b
        outputs.append(C @ h)
    return np.array(outputs), h

y, final_state = selective_scan(u)
changed = u.copy()
changed[12:] += 10.0
changed_y, _ = selective_scan(changed)
assert np.allclose(y[:12], changed_y[:12])
left, boundary = selective_scan(u[:12])
right, resumed_state = selective_scan(u[12:], boundary)
assert np.allclose(y, np.r_[left, right])
assert np.allclose(final_state, resumed_state)
reset_right, _ = selective_scan(u[12:])
assert not np.allclose(right, reset_right)
print('causal prefix and carried-state chunk checks passed')
```

The output confirms both checks. Resetting the state at a batch boundary changes the computation. Conversely, sharing state between independent examples leaks information. A streaming implementation must track which sequence owns each state and reset it at genuine sequence boundaries.

Mamba-2 and the attention connection

Mamba-2 [29] changes both the inner sequence mixer and the surrounding block. Its structured state-space duality uses a scalar-times-identity transition within each head and organizes computation into chunks whose dense matrix multiplications suit accelerator hardware. It also restructures parameter projections and normalization. This is more specific than adding attention to the original Mamba block.

With a zero initial state, the scalar-transition recurrence can be expanded as

$$y_t = \sum_{j=1}^{t} \left[C_t \left(\prod_{k=j+1}^{t} a_k \right) B_j \right] u_j.$$

Here j indexes earlier inputs, k indexes intervening transitions, a_k is the scalar discrete transition multiplying the identity, B_j is the discrete write column, C_t is the read row, and an empty product equals one. The bracketed quantity is a structured causal mixing coefficient. This exposes an attention-like matrix view without requiring ordinary softmax attention or storing that matrix. Hu et al. [30] (ICML 2026) extend the duality to diagonal transitions and clarify why it does not extend directly to standard softmax attention.

At fixed width and state dimension, recurrent inference carries a fixed-size state, whereas ordinary cached causal attention retains keys and values whose storage grows with context length. Compression saves memory but puts pressure on exact recall of many unrelated details. Linear sequence scaling also does not imply constant training memory: activations, gradients and recom-

putation policies matter. Evaluate retrieval quality, context length and measured device latency together. The image-specific ordering problem appears next in ML 04.

Modern view

Survey literature. Goodfellow et al.'s chapter 10 [1] remains the clearest unified treatment of recurrent and recursive networks, echo-state ideas, and long-term dependencies. Lipton, Berkowitz and Elkan [20] wrote a critical review of RNNs for sequence learning in 2015: a self-contained, historically ordered account of three decades of work leading to LSTMs and bidirectional RNNs, with conflicting notation reconciled and applications to captioning, translation and handwriting recognition. Greff et al. [19] is less a survey than a large empirical study, but it is the reference for which LSTM components matter. Two recent reviews by Tiezzi et al. reframe recurrence for the Transformer era: an extensive survey [26] places Transformer–RNN hybrids, attention variants with recurrent form, and deep state-space models (a special class of linear RNNs) under the single umbrella of recurrence, and highlights research opportunities in learning online from potentially infinite streams rather than fixed-length sequences; and a shorter perspective in *Nature Machine Intelligence* [27] revisits recurrent processing at the crossroad of Transformers and state-space models.

Recurrence returns (2023–2026). Transformers process all pairs of positions, which costs memory and computation quadratic in length. A family of models revisits recurrence to get linear-time inference with a constant-size state:

- **Linear recurrent units.** Orvieto et al. [24] (ICML 2023) showed that a carefully parameterized **linear** recurrence with diagonal complex-valued state matrices, stable initialization and normalization matches deep state-space models on long-range benchmarks, and that the nonlinearity can live outside the recurrence.
- **Selective state spaces.** Mamba [21] makes the state-space parameters functions of the input, so the model can choose what to store or ignore, as LSTM gates do, while a hardware-aware parallel scan keeps training efficient. Vision variants and the debate about whether they are necessary for images are covered in ML 04.
- **Revived LSTMs and GRUs.** xLSTM [22] (NeurIPS 2024) adds exponential gating and a matrix-valued memory to the LSTM and scales it to language-model sizes. Feng et al. [23] (2024) remove the hidden-state dependence from the gates of LSTMs and GRUs ("minLSTM", "minGRU"), which allows a parallel scan during training, and report results competitive with recent sequence models.

The shared lesson is that the **gated, additive state update** invented for the LSTM was right; what limited classical RNNs was mostly sequential training. Making the recurrence linear in the state (so it can be computed with a parallel scan) recovers parallel training while keeping cheap, streaming inference.

Open problems. Length generalization beyond the training horizon; credit assignment over very long streams without full BPTT; how much a fixed-size state can remember compared with attention's growing cache, a real limitation for tasks that need exact recall of distant details; and

choosing a scan order for 2-D images and 3-D video, where no single natural sequence order exists.

Key takeaways

- An RNN applies one shared update $h_t = f(x_t, h_{t-1})$ at every step, so it handles any sequence length with a fixed number of parameters. Unrolled, it is a deep network with tied weights.
- BPTT is ordinary backpropagation on the unrolled graph; gradients of shared weights are summed over time. Truncated BPTT carries state forward but limits how far gradients flow back.
- Long-range gradients are products of Jacobians and tend to vanish or explode. Clipping fixes explosions; architectures with near-identity, additive state paths fix vanishing.
- LSTM gates (forget, input, output) and GRU gates (reset, update) control a protected memory path. The forget gate was a 2000 addition to the 1997 LSTM. Check the framework's gate conventions.
- Bidirectional encoders use future context and are not causal; streaming models must manage state carefully.
- Seq2seq maps sequences to sequences with teacher forcing; attention removed its single-vector bottleneck, and self-attention without recurrence became the Transformer.
- Recursive networks compose along trees with one shared rule. In vision, recurrence powered captioning, video models and ConvLSTM forecasting, and gated linear recurrences are back as state-space models.
- Mamba predicts write, read and interval coefficients from inputs while keeping updates affine in state; Mamba-2 uses structured duality for efficient chunk computation. Compression saves cache memory but limits exact recall.

Exercises

1. **Causality.** Your team reports 92% frame-level accuracy for an action-recognition model with a bidirectional LSTM and proposes deploying it for live alerts. What is wrong with this plan, and how would you evaluate the deployable alternative fairly? Run the selective-scan example, then add a backward scan: which causality check should fail?

   Hint

   The backward LSTM at frame t has read frames after t, which a live system has not seen yet. Retrain a unidirectional (causal) model, or allow a fixed look-ahead delay and evaluate with exactly that delay. Report accuracy as a function of latency. A backward scan reads the altered future, so the combined prefix generally changes even when the forward prefix does not.

2. **Length generalization.** Train the GRU sum classifier at length 12, then evaluate on fresh sequences of length 6, 24, 100 and 400 without retraining. Plot accuracy versus length. Then repeat with an LSTM and with training on mixed lengths 6–24.

Hint

Generate fresh data per length. The sum's scale grows like $\sqrt{T}$, so the state may saturate on long inputs even if the logic is right. Mixed-length training usually helps more than switching cell type. Success at length 12 says nothing by itself about length 400.

3. **Gradient bound.** For the scalar RNN $h_t = \tanh(wh_{t-1} + x_t)$, show that $|\partial h_t / \partial h_s| \leq |w|^{t-s}$. For what values of w can the gradient explode, and why does saturation of tanh work against explosion?

Hint

Each factor is $w(1 - h_k^2)$ with $0 \leq 1 - h_k^2 \leq 1$. Explosion requires $|w| > 1$, but a large $|w|$ also drives h_k toward ± 1, which makes $1 - h_k^2$ small. The gradient explodes only on trajectories that stay in the non-saturated region, which is why explosions are bursty in practice.

4. **LSTM memory.** In the LSTM equations, suppose $f_t = 1$ and $i_t = 0$ for all t from s to t. What is c_t in terms of c_s, and what is $\partial c_t / \partial c_s$ along the cell path? What would happen if the forget gate were fixed at 1 forever, as in the original 1997 design, on a never-ending input stream?

Hint

$c_t = c_s$ and $\partial c_t / \partial c_s = I$: perfect memory and perfect gradient flow. With $f_t \equiv 1$ and nonzero writes, the cell state accumulates without bound over an endless stream, which is exactly the problem Gers et al. solved by letting the network learn to forget.

5. **Attention shapes and masks.** In the additive-attention code, a batch has encoder states of shape $8 \times 40 \times 256$ (with some sequences padded) and a decoder state of shape 8×512. Give the shapes of e, α and the context, and explain what would go wrong if padded positions were masked with 0 instead of $-\infty$.

Hint

e and α are 8×40; the context is 8×256. A score of 0 is an ordinary score: $e^0 = 1$ would still give padding positive weight, and the context would mix in meaningless padded states.

6. **Recursive versus recurrent.** For a balanced binary tree over $n = 1024$ leaves, how many composition steps lie between a leaf and the root? Compare with a left-to-right RNN over the same 1024 inputs, and relate the answer to the vanishing-gradient bound.

Hint

$\log_2 1024 = 10$ steps versus up to 1023 for the chain. The gradient bound decays like $(\text{factor})^{\text{path length}}$, so a shorter path from each input to the output helps optimization. Attention takes this to the extreme: path length one.

References

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016, ch. 10. Book
2. S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, 1997. DOI
3. K. Cho et al., "Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation," EMNLP, 2014. arXiv
4. K. S. Tai, R. Socher and C. D. Manning, "Improved Semantic Representations From Tree-Structured Long Short-Term Memory Networks," ACL, 2015. arXiv
5. F. A. Gers, J. Schmidhuber and F. Cummins, "Learning to Forget: Continual Prediction with LSTM," *Neural Computation*, 2000. DOI
6. Y. Bengio, P. Simard and P. Frasconi, "Learning long-term dependencies with gradient descent is difficult," *IEEE Transactions on Neural Networks*, 1994. DOI
7. R. Pascanu, T. Mikolov and Y. Bengio, "On the difficulty of training Recurrent Neural Networks," ICML, 2013. arXiv
8. P. J. Werbos, "Backpropagation through time: what it does and how to do it," *Proceedings of the IEEE*, 1990. DOI
9. J. L. Elman, "Finding Structure in Time," *Cognitive Science*, 1990. DOI
10. M. Schuster and K. K. Paliwal, "Bidirectional recurrent neural networks," *IEEE Transactions on Signal Processing*, 1997. DOI
11. I. Sutskever, O. Vinyals and Q. V. Le, "Sequence to Sequence Learning with Neural Networks," NeurIPS, 2014. arXiv
12. D. Bahdanau, K. Cho and Y. Bengio, "Neural Machine Translation by Jointly Learning to Align and Translate," ICLR, 2015. arXiv
13. O. Vinyals, A. Toshev, S. Bengio and D. Erhan, "Show and Tell: A Neural Image Caption Generator," CVPR, 2015. arXiv
14. K. Xu et al., "Show, Attend and Tell: Neural Image Caption Generation with Visual Attention," ICML, 2015. arXiv
15. J. Donahue et al., "Long-term Recurrent Convolutional Networks for Visual Recognition and Description," CVPR, 2015. arXiv
16. X. Shi et al., "Convolutional LSTM Network: A Machine Learning Approach for Precipitation Nowcasting," NeurIPS, 2015. arXiv
17. R. Socher et al., "Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank," EMNLP, 2013. Paper
18. R. Socher, C. C.-Y. Lin, A. Y. Ng and C. D. Manning, "Parsing Natural Scenes and Natural Language with Recursive Neural Networks," ICML, 2011. Paper
19. K. Greff, R. K. Srivastava, J. Koutník, B. R. Steunebrink and J. Schmidhuber, "LSTM: A Search Space Odyssey," *IEEE Transactions on Neural Networks and Learning Systems*, 2017. arXiv
20. Z. C. Lipton, J. Berkowitz and C. Elkan, "A Critical Review of Recurrent Neural Networks for Sequence Learning," arXiv:1506.00019, 2015. arXiv
21. A. Gu and T. Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," COLM, 2024. arXiv
22. M. Beck et al., "xLSTM: Extended Long Short-Term Memory," NeurIPS, 2024. Paper
23. L. Feng et al., "Were RNNs All We Needed?," arXiv:2410.01201, 2024. arXiv
24. A. Orvieto et al., "Resurrecting Recurrent Neural Networks for Long Sequences," ICML, 2023.

arXiv
25. S. Bengio, O. Vinyals, N. Jaitly and N. Shazeer, “Scheduled Sampling for Sequence Prediction with Recurrent Neural Networks,” NeurIPS, 2015. arXiv
26. M. Tiezzi, M. Casoni, A. Betti, T. Guidi, M. Gori and S. Melacci, “On the Resurgence of Recurrent Models for Long Sequences: Survey and Research Opportunities in the Transformer Era,” arXiv:2402.08132, 2024. arXiv
27. M. Tiezzi, M. Casoni, A. Betti, T. Guidi, M. Gori and S. Melacci, “Back to recurrent processing at the crossroad of transformers and state-space models,” *Nature Machine Intelligence*, 2025. DOI
28. A. Vaswani et al., “Attention Is All You Need,” NeurIPS, 2017. arXiv
29. T. Dao and A. Gu, “Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality,” ICML, PMLR 235, 2024. Paper
30. J. Y.-C. Hu, X. Zhang, A. Elsheikh, W. Wu and H. Liu, “On Structured State-Space Duality,” ICML, PMLR 306, 2026. Paper

ML 04 • Attention and Vision Transformers

Build self-attention from queries, keys and values; add multiple heads, positional encodings and the Transformer block; turn images into patch tokens with ViT; then study DeiT, Swin, hybrid CNN–Transformer models, efficient attention and state-space alternatives.

Online edition: https://blog.shyandram.dev/en/learn/ml/ml-04-vision-transformers/

What you'll learn

- How self-attention computes an input-dependent weighted average, why queries, keys and values play different roles, and why the scores are divided by $\sqrt{d_k}$.
- How multi-head attention, causal masks and cross-attention are built, with code checked against PyTorch.
- Why attention needs positional information, and how sinusoidal, learned, relative and rotary encodings provide it.
- How the Transformer block combines attention, an MLP, LayerNorm and residual connections.
- How the Vision Transformer (ViT) turns an image into patch tokens, why it needs so much data, and how DeiT, Swin and hybrid CNN–Transformer designs respond.
- What makes attention expensive, what FlashAttention, windows and token merging change, and how Mamba and Mamba-2 compare with attention, and how image scan ordering affects global context and directional bias.

The big picture

A CNN (ML 03) moves information through fixed local neighborhoods: a pixel learns about a distant pixel only after many layers. **Attention** lets every location decide, based on its current content, which other locations to read from, in a single step. To recognize a bird, a patch containing its head can directly consult a distant patch containing its wing.

> **Explain like I'm five**
>
> Imagine a classroom where each child holds one piece of a jigsaw puzzle. Each child asks a question about their piece ("I have a blue edge, who has sky?"), and every other child holds up a card describing their own piece. Each child listens most closely to the classmates whose cards best match the question, and updates their idea of what their piece shows. Then everyone does it again with the improved ideas. Attention is the listening rule. A Transformer repeats many rounds of listening, with a little thinking time in between.

The attention idea came from machine translation with recurrent networks: ML 03A shows how it removed the single-vector bottleneck of encoder–decoder RNNs. The Transformer of Vaswani

et al. [1] went further and removed recurrence entirely, building both encoder and decoder from attention and small per-token networks. The Vision Transformer (ViT) of Dosovitskiy et al. [2] applied the Transformer encoder almost unchanged to sequences of image patches. Bishop and Bishop [10] give a modern textbook treatment, and the surveys discussed in the Modern view [3], [11], [12] map the many vision variants. Afterwards, DLCV 02 compares CNN and Transformer backbones for classification in practice, and ML 05 shows how such backbones are pretrained without labels.

Self-attention: queries, keys and values

Plain version. Each token produces three vectors: a **query** ("what am I looking for?"), a **key** ("what do I contain?") and a **value** ("what will I pass on if you listen to me?"). A token's output is the average of all values, weighted by how well its query matches each key.

Precise version. Stack N tokens as the rows of $X \in \mathbb{R}^{N\times D}$, where D is the model width. Three learned matrices project them:

$$Q = XW_Q, \qquad K = XW_K, \qquad V = XW_V,$$

with $W_Q, W_K \in \mathbb{R}^{D\times d_k}$ and $W_V \in \mathbb{R}^{D\times d_v}$. **Scaled dot-product attention** [1] is

$$\text{Attention}(Q,K,V) = \text{softmax}_{\text{row}}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V = AV.$$

$QK^\top$ is an $N\times N$ matrix of **scores**: entry (i,j) is the dot product of query i with key j, a measure of how relevant token j is to token i. The row-wise softmax turns each row into nonnegative weights that sum to one, giving the **attention matrix** A. The output $Y = AV \in \mathbb{R}^{N\times d_v}$ has one row per token: row i is $\sum_j A_{ij}v_j$, a weighted average of value vectors.

Why three different projections? If queries and keys were the same vector, every token would match itself most strongly, and relevance would be forced to be symmetric. Separate W_Q and W_K let "what I seek" differ from "what I offer" (an eye patch can seek a nose without being one), and a separate W_V lets "what I offer" differ from "what I pass on".

Why divide by $\sqrt{d_k}$? If the components of a query and a key are independent with mean zero and variance one, their dot product $q\cdot k = \sum_{l=1}^{d_k} q_l k_l$ has mean zero and variance d_k. Without scaling, larger heads produce larger scores, the softmax saturates (almost all weight on one token), and its gradients become tiny. Dividing by $\sqrt{d_k}$ restores unit variance. It is a scale convention, not a normalization of each query to unit length.

Content, not position, decides who talks to whom. Unlike a convolution, whose weights are tied to relative positions, attention weights are computed from the data at run time. A related consequence: self-attention is **permutation-equivariant**. Shuffle the input tokens, and the outputs are shuffled in the same way, because nothing in the formula refers to token order. The snippet checks both the shapes and this symmetry.

```
import torch

torch.manual_seed(0)
X = torch.randn(5, 8)                    # N=5, D=8
Wq, Wk, Wv = [torch.randn(8, 4) for _ in range(3)]
def attention(x):
    q, k, v = x @ Wq, x @ Wk, x @ Wv
    a = (q @ k.T / q.shape[-1]**.5).softmax(dim=-1)
    return a @ v, a
y, a = attention(X)
perm = torch.tensor([3, 0, 4, 1, 2])
yp, _ = attention(X[perm])
assert torch.allclose(a.sum(-1), torch.ones(5), atol=1e-6)
assert torch.allclose(yp, y[perm], atol=1e-5)
print("output shape:", tuple(y.shape))
```

This symmetry suits sets, but an image is not a set of patches: a face with its eyes below its mouth is not a face. Positional information, introduced below, breaks the symmetry.

Multi-head attention

Plain version. Run several attention operations in parallel, each with its own smaller projections, so different heads can track different relationships (nearby texture, the same object, symmetric parts). Then concatenate their outputs and mix them.

Precise version. With h heads, head r has its own $W_Q^{(r)}, W_K^{(r)}, W_V^{(r)} \in \mathbb{R}^{D\times d_k}$, usually with $d_k = d_v = D/h$:

$$\text{head}_r = \text{Attention}\big(XW_Q^{(r)}, XW_K^{(r)}, XW_V^{(r)}\big), \qquad \text{MHA}(X) = [\,\text{head}_1, \dots, \text{head}_h\,]\,W_O,$$

where the brackets concatenate the h outputs of width D/h back to width D, and $W_O \in \mathbb{R}^{D\times D}$ mixes them. Because each head has width D/h, multi-head attention costs about the same as one full-width head, and its projections have $4D^2$ weights in total (plus biases). Heads are not guaranteed to learn human-readable roles; some are redundant and can be pruned.

The implementation below splits the projections into heads, computes the scores per head, concatenates and projects, and matches `nn.MultiheadAttention` exactly. It also shows a **causal mask**: setting forbidden scores to $-\infty$ before the softmax gives them exactly zero weight.

```
import torch
from torch import nn

torch.manual_seed(0)
B, N, D, h = 2, 6, 16, 4
mha = nn.MultiheadAttention(D, num_heads=h, batch_first=True, bias=False)
X = torch.randn(B, N, D)
```

```
def multi_head(x):
    Wq, Wk, Wv = mha.in_proj_weight.chunk(3)          # each D x D (rows = outputs)
    q, k, v = x @ Wq.T, x @ Wk.T, x @ Wv.T              # B x N x D
    split = lambda t: t.view(B, N, h, D // h).transpose(1, 2)   # B x h x N x d_k
    q, k, v = split(q), split(k), split(v)
    scores = q @ k.transpose(-2, -1) / (D // h) ** 0.5  # B x h x N x N
    A = scores.softmax(dim=-1)
    heads = A @ v                                       # B x h x N x d_k
    concat = heads.transpose(1, 2).reshape(B, N, D)     # put the heads side by side
    return concat @ mha.out_proj.weight.T, A

ours, A = multi_head(X)
ref, _ = mha(X, X, X)
print("matches nn.MultiheadAttention:", torch.allclose(ours, ref, atol=1e-5))
print("attention tensor:", tuple(A.shape), "| parameters:",
      sum(p.numel() for p in mha.parameters()), "= 4 * D^2 =", 4 * D * D)

# Causal mask: query i may only read keys j <= i
causal = torch.triu(torch.ones(N, N, dtype=torch.bool), diagonal=1)   # True = forbidden
out_c, w_c = mha(X, X, X, attn_mask=causal)
print("no weight on the future:", torch.all(w_c[:, causal] == 0).item())
```

It prints `True`, an attention tensor of shape `(2, 4, 6, 6)` (batch, heads, queries, keys), `1024 = 4 * D^2 = 1024`, and `True` for the causal check. In production code, call `torch.nn.functional.scaled_dot_product_attention`, which computes the same thing and dispatches to fused, memory-efficient kernels when the device, data type and shapes allow [7].

Three ways to connect tokens

Plain version. Which tokens a query is allowed to read is part of the model's design, expressed as a mask on the score matrix.

Mechanism	A query reads	Typical use
Bidirectional self-attention	All tokens of the same sequence	ViT and other image encoders
Causal (masked) self-attention	Itself and earlier positions only	Next-token generation, autoregressive image models
Cross-attention	Tokens from a different sequence	A decoder reading an encoder; image tokens reading text

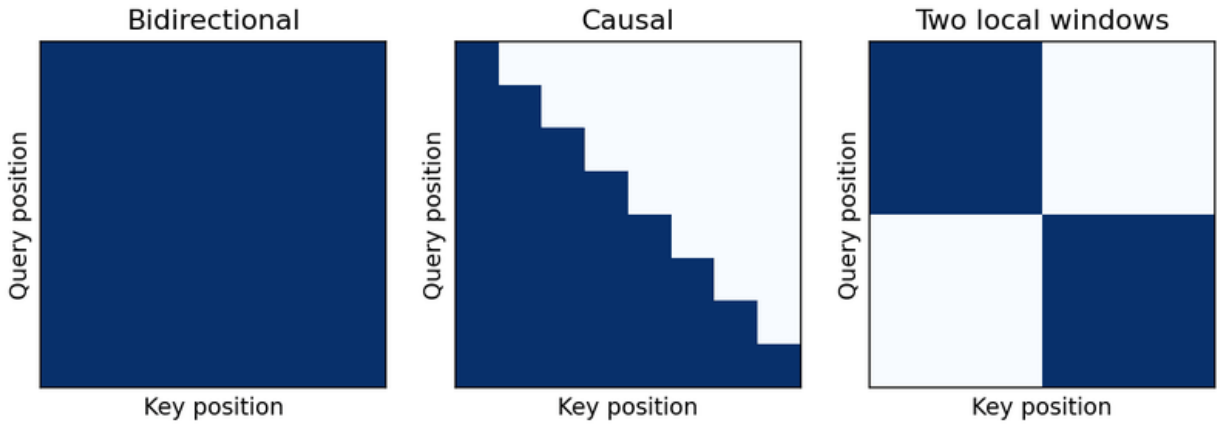


Figure 1 — Attention masks: blue entries are allowed, white entries are forbidden. Causal generation also shifts its inputs by one position so a target cannot simply be copied.

In **cross-attention**, queries come from one sequence X and keys and values from another sequence Z: $Q = XW_Q$, $K = ZW_K$, $V = ZW_V$. The score matrix has shape $N_X \times N_Z$, and the output has one row per query. This is how the original Transformer decoder reads its encoder, how image captioners read image features, and how text-to-image diffusion models read a text prompt (ML 07) without being autoregressive over image patches.

Attention weights are part of a computation, not automatically an explanation. A large weight can multiply an uninformative value, and the residual and MLP paths also shape the output.

Positional information

Plain version. Because attention ignores order, we must tell the model where each token is. The simplest way is to add a position-specific vector to each token before the first layer; more refined ways inject relative position directly into the attention scores.

Sinusoidal encodings. The original Transformer [1] adds a fixed vector $p_{pos} \in \mathbb{R}^D$ to the token at position pos:

$$p_{pos,2i} = \sin\!\left(\frac{pos}{10000^{2i/D}}\right), \qquad p_{pos,2i+1} = \cos\!\left(\frac{pos}{10000^{2i/D}}\right), \qquad i = 0, \dots, \tfrac{D}{2} - 1.$$

Each pair of dimensions $(2i, 2i+1)$ is a point rotating around a circle at angular frequency $\omega_i = 10000^{-2i/D}$, from fast (one radian per position) to very slow. Two properties follow from the angle-addition formulas. First, for any fixed offset k, $p_{pos+k} = p_{pos} R_k$ for a block-diagonal rotation R_k that does not depend on pos; a linear layer can therefore learn to "look k positions ahead". Second, $p_m \cdot p_n = \sum_i \cos\left(\omega_i (m - n)\right)$ depends only on the offset $m - n$ (Figure 2, middle).

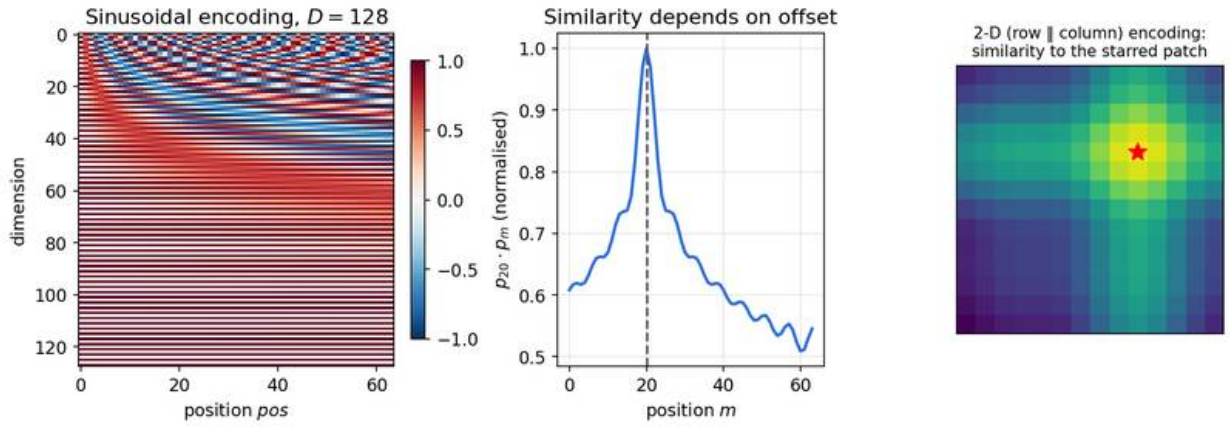


Figure 2 — Sinusoidal positional encodings. Left: each dimension pair oscillates at its own frequency. Middle: similarity between encodings depends on the offset. Right: concatenating row and column encodings gives a 2-D encoding for a 14 × 14 patch grid.

```
import numpy as np

def sinusoidal(n_pos, D):
    pos = np.arange(n_pos)[:, None]
    omega = 1.0 / 10000 ** (np.arange(0, D, 2) / D)    # one frequency per sin/cos pair
    pe = np.zeros((n_pos, D))
    pe[:, 0::2] = np.sin(pos * omega)
    pe[:, 1::2] = np.cos(pos * omega)
    return pe, omega

pe, omega = sinusoidal(100, 16)
k = 7                                                  # a fixed offset
# block-diagonal rotation that maps p_pos to p_{pos+k}, the same for every pos
R = np.zeros((16, 16))
for i, w in enumerate(omega):
    c, s = np.cos(w * k), np.sin(w * k)
    R[2 * i:2 * i + 2, 2 * i:2 * i + 2] = [[c, -s], [s, c]]
print("p[pos+k] == p[pos] @ R for all pos:", np.allclose(pe[:-k] @ R, pe[k:]))
print("dot products depend only on offset:",
      np.isclose(pe[10] @ pe[17], pe[50] @ pe[57]))
```

Both checks print `True`.

Other schemes.

- **Learned absolute embeddings**: a trainable vector per position, as in ViT [2]. ViT's 1-D learned embeddings, indexed by raster order, worked about as well as explicitly 2-D ones; after training, embeddings of patches in the same row or column became similar, so the model learned the grid by itself. At a new resolution, the grid of embeddings is resized by 2-D interpolation before fine-tuning.

- **Relative position representations** (Shaw et al. [17]) add a learned term that depends on the offset $j - i$ to the score between tokens i and j. Swin [5] adds a learned **relative position bias** table to each window's scores.
- **Rotary position embedding (RoPE)** (Su et al. [16]) rotates each query and key by an angle proportional to its position, using the same rotations as the sinusoidal code above. The score $q_m^\top k_n$ then depends on the positions only through $m - n$. RoPE is standard in large language models and increasingly used in vision with separate rotations for the two image axes.
- **No explicit encoding**: convolutions inside the network, as in CvT [19], or zero padding at borders can leak enough position information on their own.

The Transformer block

Plain version. A block alternates two jobs. Attention lets tokens exchange information; a small MLP then processes each token on its own. Residual connections and normalization keep the stack stable.

Precise version. The **pre-norm** block used by ViT and most modern models is

$$U = X + \mathrm{MHA}(\mathrm{LN}(X)), \qquad Y = U + \mathrm{MLP}(\mathrm{LN}(U)), \qquad \mathrm{MLP}(u) = W_2\,\mathrm{GELU}(W_1 u + b_1) + b_2.$$

LN is **layer normalization** [14]: it standardizes each token's D features to zero mean and unit variance, then applies a learned scale and shift; unlike BatchNorm, it does not depend on other samples in the batch. The MLP expands each token from D to typically $4D$ hidden units, applies the GELU nonlinearity [26] ($\mathrm{GELU}(z) = z\,\Phi(z)$, where Φ is the standard normal CDF), and projects back. The residual additions mean each sub-layer only learns a correction, exactly as in ResNet.

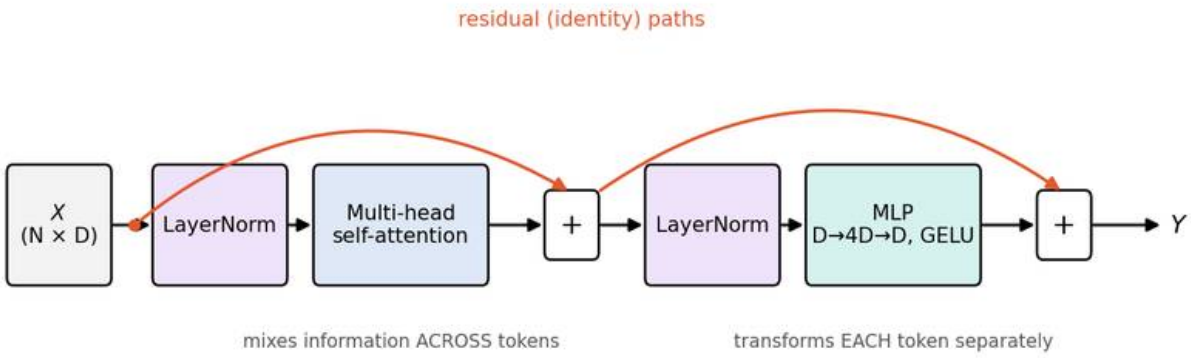


Figure 3 — A pre-norm Transformer block. Attention mixes information across tokens; the MLP transforms each token separately; residual paths (orange) carry the input around each branch.

The original Transformer placed LayerNorm **after** each residual addition (post-norm). Xiong et al. [15] showed that with post-norm the gradients near the output are large at initialization, which is why it needs a learning-rate warm-up, whereas pre-norm keeps gradients well behaved

and can train without one. So "a Transformer" is not fully specified until you say where the normalization goes. Counting weights per block: $4D^2$ for attention and $8D^2$ for the MLP, so about $12D^2$, two thirds of it in the MLP.

The Vision Transformer (ViT)

Plain version. Cut the image into square patches, turn each patch into a vector, add position information, and feed the sequence to a standard Transformer encoder. Read the answer from a special extra token.

Patchify. For an image with $H \times W$ pixels and C channels, and patch size P dividing both H and W, there are $N = HW/P^2$ patches. Flatten each into $x_i^{\text{patch}} \in \mathbb{R}^{P^2C}$ and project linearly:

$$z_0 = [\, x_{\text{cls}};\ x_1^{\text{patch}}E;\ \ldots;\ x_N^{\text{patch}}E \,] + E_{\text{pos}},$$

where $E \in \mathbb{R}^{P^2C \times D}$ is the learned **patch embedding**, $x_{\text{cls}} \in \mathbb{R}^D$ is a learned **class token** prepended to the sequence, and $E_{\text{pos}} \in \mathbb{R}^{(N+1)\times D}$ holds learned position embeddings. After L blocks, a LayerNorm and a linear head applied to the class token's output give the logits. Averaging all patch outputs (mean pooling) is an equally common alternative to the class token.

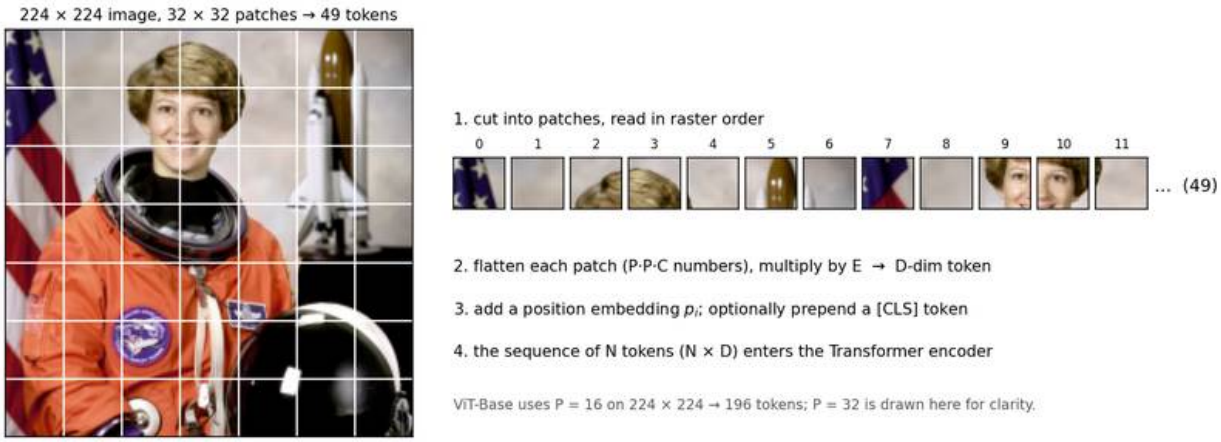


Figure 4 — Patchify. A 32-pixel grid is drawn for readability; ViT-Base uses 16 × 16 patches, which turn a 224 × 224 image into 196 tokens.

A linear map applied to non-overlapping $P \times P$ patches is exactly a convolution with kernel size and stride both equal to P, which is how implementations write it:

```
import torch
from torch import nn

torch.manual_seed(0)
B, C, H, W, P, D = 2, 3, 32, 32, 8, 48
x = torch.randn(B, C, H, W)
```

```
# 1) explicit: cut into P x P patches, flatten each, multiply by E
patches = x.unfold(2, P, P).unfold(3, P, P)                 # B, C, H/P, W/P, P, P
patches = patches.permute(0, 2, 3, 1, 4, 5).reshape(B, -1, C * P * P)   # B, N, P*P*C
E = torch.randn(C * P * P, D) * 0.02
tokens_explicit = patches @ E                                # B, N, D

# 2) the same linear map written as a convolution with kernel = stride = P
conv = nn.Conv2d(C, D, kernel_size=P, stride=P, bias=False)
conv.weight.data = E.T.reshape(D, C, P, P)
tokens_conv = conv(x).flatten(2).transpose(1, 2)             # B, N, D

print("tokens:", tuple(tokens_conv.shape), "| identical:",
      torch.allclose(tokens_explicit, tokens_conv, atol=1e-5))
```

It prints 16 tokens of width 48 and `True`.

Sizes. The standard configurations are ViT-Base (12 blocks, $D = 768$, 12 heads, about 86 million parameters), ViT-Large (24 blocks, $D = 1024$, 16 heads) and ViT-Huge (32 blocks, $D = 1280$, 16 heads) [2]. "ViT-B/16" means Base with 16×16 patches. **Patch size is the main compute knob**: halving P quadruples N and multiplies the attention-score count by sixteen. Small patches keep fine detail; large patches are cheap but lose anything finer than the patch, which no later layer can recover.

The data lesson. Dosovitskiy et al. [2] found that ViT trained only on ImageNet-1k (about 1.3 million images) was somewhat worse than comparable ResNets, but when pretrained on ImageNet-21k (14 million) or JFT-300M (300 million) and then fine-tuned, it matched or exceeded them at lower pretraining cost. Their explanation is **inductive bias**: a CNN assumes locality and translation equivariance in every layer, whereas a ViT is local only within the patch embedding and must **learn** spatial relationships from data. Raghu et al. [18] compared the internal representations of ViTs and ResNets and found that ViTs have more uniform representations across layers, that their lower layers attend both locally and globally, and that learning local attention in early layers requires large-scale pretraining.

Code: a tiny ViT trained on digits

This complete example trains a small ViT on the same 8×8 digits as the CNN in ML 03, with 2×2 patches (16 tokens), a class token, learned positions and four pre-norm blocks. It runs on CPU in about half a minute.

```
import copy
import numpy as np
import torch
from torch import nn
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

torch.manual_seed(0)
torch.set_num_threads(1)
```

```
class TinyViT(nn.Module):
    def __init__(self, img=8, patch=2, chans=1, D=64, depth=4, heads=4, classes=10):
        super().__init__()
        N = (img // patch) ** 2
        self.patch = nn.Conv2d(chans, D, kernel_size=patch, stride=patch)   # patchify + E
        self.cls = nn.Parameter(torch.zeros(1, 1, D))                       # [CLS] token
        self.pos = nn.Parameter(torch.randn(1, N + 1, D) * .02)             # learned
         ↪  positions
        layer = nn.TransformerEncoderLayer(D, heads, dim_feedforward=4 * D, dropout=0.1,
                                           activation="gelu", batch_first=True,
                                           norm_first=True)                 # pre-norm block
        self.blocks = nn.TransformerEncoder(layer, depth, enable_nested_tensor=False)
        self.norm, self.head = nn.LayerNorm(D), nn.Linear(D, classes)

    def forward(self, x):
        t = self.patch(x).flatten(2).transpose(1, 2)                         # B, N, D
        t = torch.cat([self.cls.expand(len(t), -1, -1), t], dim=1) + self.pos
        return self.head(self.norm(self.blocks(t))[:, 0])                    # read [CLS]

d = load_digits()
X = torch.tensor(d.images[:, None] / 16.0, dtype=torch.float32)
y = torch.tensor(d.target)
train, test = train_test_split(np.arange(len(y)), test_size=.2, stratify=d.target,
                               random_state=0)
train, val = train_test_split(train, test_size=.2, stratify=d.target[train], random_state=1)
model = TinyViT()
opt = torch.optim.AdamW(model.parameters(), lr=1e-3, weight_decay=0.05)
best = (float("inf"), None)
for epoch in range(30):
    model.train()
    for idx in torch.tensor(train)[torch.randperm(len(train))].split(64):
        loss = nn.functional.cross_entropy(model(X[idx]), y[idx])
        opt.zero_grad(); loss.backward(); opt.step()
    model.eval()
    with torch.no_grad():
        v = nn.functional.cross_entropy(model(X[val]), y[val]).item()
    if v < best[0]:
        best = (v, copy.deepcopy(model.state_dict()))
model.load_state_dict(best[1]); model.eval()
with torch.no_grad():
    acc = (model(X[test]).argmax(1) == y[test]).float().mean().item()
print("parameters:", sum(p.numel() for p in model.parameters()))
print("best validation loss:", round(best[0], 4), "| held-out accuracy:", round(acc, 4))
```

On our run it uses about 202,000 parameters and reaches about 95% held-out accuracy, while the CNN in ML 03 reached about 96% with fewer than 10,000 parameters and the same split. On a small dataset, the CNN's built-in locality is worth a factor of twenty in parameters. This is the data lesson in miniature, not a verdict on large-scale performance.

Training recipes: DeiT and beyond

Plain version. Much of the early gap between ViTs and CNNs on ImageNet-sized data turned out to be about training, not architecture.

DeiT (Touvron et al. [4]) trained ViTs competitively **on ImageNet-1k alone** using strong data augmentation (RandAugment, Mixup, CutMix, random erasing), heavy regularization (stochastic depth, label smoothing) and long schedules. It also introduced a **distillation token**: an extra learned token, like the class token, whose output is trained to match the predictions of a pre-trained CNN teacher. The CNN's inductive bias is thus transferred through its predictions. DeiT III [27] later showed that a simpler, carefully tuned supervised recipe (a lightweight "3-Augment" scheme, simple cropping and a binary cross-entropy loss) makes plain ViTs stronger still. Steiner et al. [25] ran a large study of data, augmentation and regularization ("AugReg") and found that the right combination of augmentation, regularization and compute can match models trained on roughly ten times more data.

The general lesson, repeated in ML 03's discussion of ConvNeXt: compare architectures only under matched data, augmentation, resolution, schedule and compute.

Swin and hierarchical Transformers

Plain version. Global attention over thousands of tokens is expensive, and plain ViT keeps one coarse resolution throughout, which is awkward for detection and segmentation. Swin restricts attention to local windows, shifts the windows between layers so information still spreads, and merges patches to build a CNN-like pyramid.

Windows. Swin [5] partitions the token grid into non-overlapping windows of $M \times M$ tokens (typically $M = 7$) and computes self-attention inside each window only. For an $h \times w$ token grid with C channels, the paper gives the costs

$$\Omega(\text{global MSA}) = 4hwC^2 + 2(hw)^2C, \qquad \Omega(\text{window MSA}) = 4hwC^2 + 2M^2hwC,$$

where $4hwC^2$ counts the four projections and the second term counts the score and weighting products. Global cost is quadratic in the number of tokens hw; window cost is linear for fixed M.

Shifted windows. With fixed windows, tokens in different windows would never interact. Swin alternates regular windows with windows displaced by $\lfloor M/2 \rfloor$ tokens in each direction. A token near a window boundary at layer ℓ shares a window with tokens from neighboring windows at layer $\ell + 1$, so information crosses boundaries (Figure 5). An efficient implementation cyclically shifts the feature map and uses a mask so that tokens wrapped from opposite edges do not attend to each other.

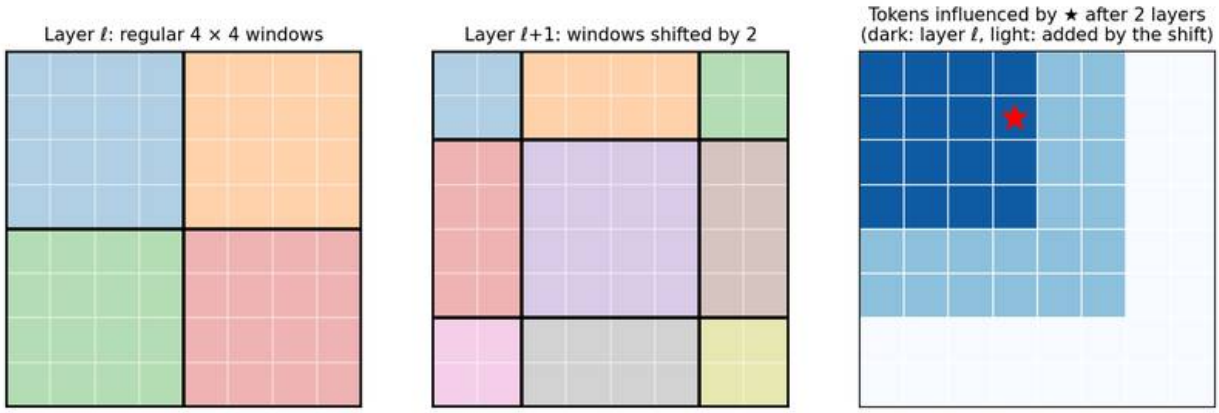


Figure 5 — Swin's regular and shifted windows on an 8 × 8 token grid. After two layers, information from the starred token reaches the dark window (layer ℓ) plus the light band added by the shifted windows (layer ℓ+1).

Hierarchy. Between stages, **patch merging** concatenates each 2 × 2 group of neighboring tokens and projects them linearly, halving the resolution and doubling the channels, as a strided convolution would. The four stages produce feature maps at 1/4, 1/8, 1/16 and 1/32 of the input resolution, the same pyramid a ResNet produces, so Swin can replace a CNN backbone in detection and segmentation frameworks. Each window also adds a learned **relative position bias** to its scores. Swin V2 [32] scaled the design up with residual post-normalization, scaled cosine attention and a continuous, log-spaced position bias that transfers better across window sizes and resolutions.

Hybrid CNN–Transformer models

Plain version. Convolutions are good at local detail with little data; attention is good at long-range interaction. Hybrids use each where it helps.

- **Convolutional stem.** The original ViT paper already tested a "hybrid" in which the patches come from a ResNet feature map instead of raw pixels [2]; it helped at smaller compute budgets.
- **Convolutions inside attention.** CvT [19] uses a convolutional token embedding at each stage and depthwise-convolutional projections for queries, keys and values, which builds in locality and lets it drop explicit position embeddings.
- **Stacking by stage.** CoAtNet [20] places depthwise-convolution stages early, where resolution is high and local patterns dominate, and relative-attention stages later; it reports strong results across data scales because the convolutional stages supply inductive bias when data is limited.
- **Mobile hybrids.** MobileViT [28] inserts small Transformer blocks into a MobileNet-style network to add global context at mobile cost.
- **The reverse direction.** ConvNeXt (see ML 03) imported Transformer-era design choices into a pure CNN and matched Swin.

Cost and efficient attention

Plain version. Global attention compares every token with every other one, so doubling the image side multiplies the work by sixteen. Efficient methods either make exact attention faster, restrict which tokens interact, or use fewer tokens.

For N tokens of width D, the score matrix costs about N^2D multiply–adds per layer and the projections and MLP about $12ND^2$. A naive implementation stores an $N \times N$ matrix per head. Doubling the image height and width at a fixed patch size multiplies N by four and the pairwise scores by sixteen.

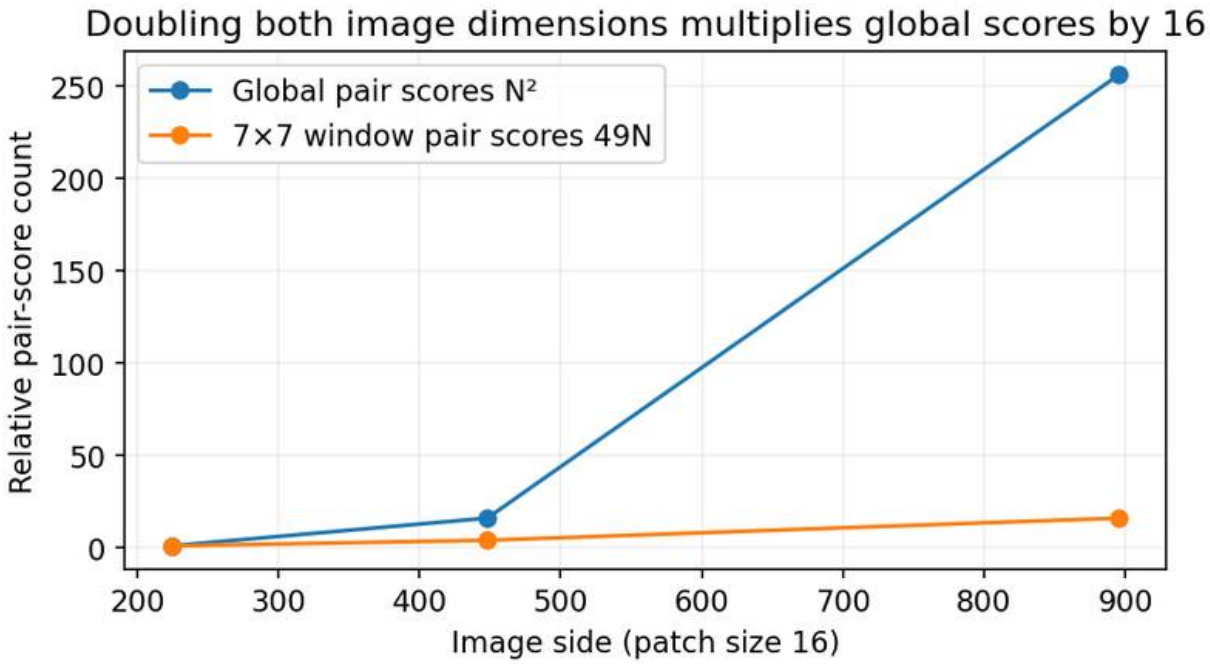


Figure 6 — Pair-score scaling, not a measured runtime. Projections, MLPs, memory traffic and kernel quality also determine latency.

The main families:

- **Exact but faster.** FlashAttention [6] computes **exact** attention by tiling the computation so that blocks of queries, keys and values stay in fast on-chip memory, and never writes the full $N \times N$ matrix to slow GPU memory; FlashAttention-2 [29] improves the work partitioning further. The arithmetic is still quadratic; the memory is linear and wall-clock time drops substantially. PyTorch's `scaled_dot_product_attention` selects such kernels automatically when it can [7].
- **Restricted connectivity.** Windowed (Swin) or other sparse patterns reduce the number of pairs, at the price of needing several layers for global communication.
- **Fewer tokens.** Token Merging (ToMe) [21] gradually merges similar tokens inside a pretrained ViT using a fast matching step, increasing throughput without retraining. Larger patches, token pruning and pooling stages do the same more bluntly.

- **Smaller models.** Papa et al.'s survey [13] organizes efficient ViTs into compact architectures, pruning, knowledge distillation and quantization, and proposes an "Efficient Error Rate" metric to compare models across parameters, bits, FLOPs and size.

State-space models and hybrids

Plain version. Instead of comparing every pair of tokens, carry a compact state through the sequence, as a recurrent network does (ML 03A). The cost grows linearly with the number of tokens.

A simplified state-space recurrence over a token sequence is

$$h_t = \bar{A}_t\, h_{t-1} + \bar{B}_t\, x_t, \qquad y_t = C_t\, h_t,$$

where h_t is a hidden state of fixed size, and $\bar{A}_t$, $\bar{B}_t$, C_t are (discretized) state-space matrices. In **selective** models such as Mamba, the write/read coefficients and step sizes depend on input x_t; the underlying continuous transition can remain fixed while its discretization varies. Input selection and an affine state update permit a parallel scan [33]. An image has no natural one-dimensional order, so vision variants must choose **scan orders**: Vision Mamba (Vim) [30] scans the patch sequence forward and backward with position embeddings, and VMamba [31] scans along four directions over the 2-D grid. The Visual Mamba survey [8] reviews over 200 papers and identifies scanning as the critical design decision for vision.

A useful counterpoint is MambaOut [9] (CVPR 2025). Its authors argue that Mamba suits tasks with long sequences and causal (autoregressive) structure, and build MambaOut models by removing the state-space component while keeping the gated convolutional blocks. Their models outperform visual Mamba models on ImageNet classification, suggesting that Mamba is unnecessary there, while for detection and segmentation, which involve long token sequences, they find state-space models may still be worth exploring. The broader point is methodological: isolate the mechanism (attention, scan, convolution) from gating, normalization, training recipe and parameter budget before crediting a result to it.

An image scan is an inductive bias

Mamba [33] supplies an efficient sequence operator; it does not choose an image geometry for us. Vim [30] uses a patch sequence, position embeddings and forward/backward processing. VMamba [31] instead places a four-route selective scan inside a hierarchical visual backbone. Neither label alone specifies receptive-field strength, resolution behavior or runtime. The state dimensions, routes, local operators and downsampling stages all matter.

For a fixed scan order, two adjacent pixels in the image may be many steps apart in the sequence. Conversely, the last pixel of one raster row and the first pixel of the next row become consecutive despite being far apart spatially. A local convolution can supply nearby evidence before the scan carries context farther. Multiple routes offer additional paths, but do not restore every geometric symmetry automatically. Position embeddings can also encode useful location while making transfer to a different grid a separate design problem.

A mathematical way to expose the choice is

$$Y^{(r)} = P_r^{-1} \mathcal{S}_r(P_r X), \qquad Y = \frac{1}{m} \sum_{r=1}^{m} Y^{(r)}.$$

Here X contains flattened image tokens, r indexes one of m routes, P_r permutes tokens into that route, $\mathcal{S}_r$ applies its sequence recurrence, P_r^{-1} restores image coordinates, and Y averages the resulting features. This is a conceptual route-fusion model. A learned vision block can use different projections or fusion rules; averaging is chosen here to keep the experiment interpretable.

CPU experiment: measure directional bias

The next example uses a single scalar state with the same decay on every route. It compares forward/backward raster scans with four raster/column routes, then rotates the input by 90 degrees, processes it and rotates the output back. Everything uses a synthetic 6×6 impulse; no weights or images are downloaded. This isolates ordering rather than reproducing a trained Vim or VMamba.

```
import numpy as np

def scan_image(image, four=False):
    ids = np.arange(image.size).reshape(image.shape)
    row, col = ids.ravel(), ids.T.ravel()
    orders = [row, row[::-1]]
    if four:
        orders += [col, col[::-1]]
    outputs = []
    for order in orders:
        h = 0.0
        out = np.zeros(image.size)
        for i in order:
            h = 0.8 * h + 0.2 * image.ravel()[i]
            out[i] = h
        outputs.append(out.reshape(image.shape))
    return np.mean(outputs, axis=0)

image = np.zeros((6, 6))
image[1, 4] = 1.0
errors = []
for four in [False, True]:
    original = scan_image(image, four=four)
    rotated_back = np.rot90(scan_image(np.rot90(image), four=four), -1)
    error = np.max(np.abs(original - rotated_back))
    errors.append(error)
    print('four routes:', four, 'rotation discrepancy:', round(error, 6))
assert 0.0 < errors[1] < errors[0]
```

The maximum discrepancies are `0.063622` for two routes and `0.026738` for four. More routes reduce directional bias in this example, but the remaining discrepancy is nonzero. Rotating a raster

path reverses one traversal axis; that path need not belong to the original four-route collection. Even shared coefficients and averaging therefore do not guarantee rotation equivariance. Learned directional projections, position information and local operations introduce further choices. Test the property you need instead of inferring it from "2-D scan."

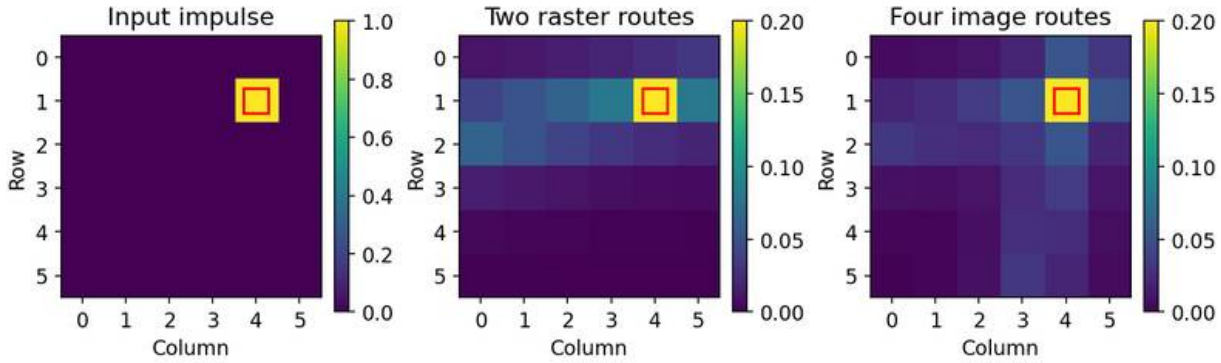


Figure 7 — The red outline marks the input impulse. Two raster routes spread context unevenly; adding column routes changes the influence map. These synthetic responses use the exact recurrence in the example, not learned attention maps or trained VMamba activations.

The recurrence can have a global receptive field because information travels through the entire scan. This says a dependence is possible, not that its influence remains strong at every distance. With scalar decay 0.8, the carried contribution decays by another factor of 0.8 per intervening step. Selective transitions can change this retention, as derived in ML 03A, but training must still learn which information to preserve. Fine boundaries and small objects can be lost through state compression or downsampling even when distant context is available.

What Mamba-2 changes, and how to evaluate it

Mamba-2 [34] reorganizes the selective sequence mixer around structured state-space duality and efficient chunk computation. It changes the transition structure and block parameter projections, rather than making an arbitrary image patch sequence equivalent to a Transformer. The causal structured coefficient matrix has an attention-like interpretation, but standard softmax attention retains different expressive and computational properties. An image application must still specify directions and fusion. The duality therefore answers a computation question; it does not remove the geometry question.

A fair vision comparison should separate three experiments: replace only the mixer inside a shared block; compare complete backbones under a matched training budget; and measure the deployed models on the target hardware. Keep data, augmentation, token resolution and downstream heads controlled. Record state size as well as parameter count, and measure latency and peak memory across image resolutions and batch sizes. A linear token-count term can coexist with expensive projections, route permutations and small-kernel overheads. GPU throughput claims do not follow from the Python loop above.

For dense tasks, inspect errors near boundaries, on small structures and across aspect ratios; for

video, distinguish an offline bidirectional encoder from causal live processing. Include rotation/crop sensitivity and longer-grid evaluations rather than only in-distribution classification. MambaOut [9] supplies an important mixer-removal ablation. Its results support a task-dependent comparison, not a theorem that scans are unnecessary for every image problem. Xu et al. [8] organize visual Mamba work by backbones, scan mechanisms and image/video/point-cloud/multimodal applications. Their survey remains a 2024 preprint on the verified record. Its taxonomy is useful for selecting controlled experiments; the number of proposed variants does not establish a universal winner.

Modern view

Survey literature. Khan et al. [3] (*ACM Computing Surveys*, 2022) start from the foundations (self-attention, large-scale pretraining, bidirectional encoding) and then survey Transformers across recognition, generative modeling, multimodal tasks such as visual question answering, video, low-level vision and 3-D analysis, closing with open problems. Han et al. [11] (*IEEE TPAMI*, 2023) organize vision Transformers by task (backbones, high- and mid-level vision such as detection and segmentation, low-level vision, video), and also cover efficient Transformer methods. Liu et al. [12] (*IEEE TNNLS*, 2024) review over a hundred visual Transformers for classification, detection and segmentation on images, point clouds and vision–language data, and suggest three future directions. For efficiency, Papa et al. [13] (*IEEE TPAMI*, 2024) is the most systematic. For state-space vision models, see Xu et al. [8]. Bishop and Bishop [10] provide the textbook derivations.

State of the art, 2023–2026. The plain ViT has become the default **foundation** backbone. ViT-22B [24] (ICML 2023) scaled a ViT to 22 billion parameters, with architectural changes (such as normalizing queries and keys) needed for stable training at that size. DINOv2 [23] (TMLR 2024) trained ViTs with self-supervision on a large curated image collection and produced general-purpose features that work frozen across classification, segmentation and depth estimation (ML 05). Darcet et al. [22] (ICLR 2024) found that large ViTs repurpose uninformative background patches as internal scratch space, producing high-norm artifact tokens in attention maps, and that adding a few extra **register tokens** removes the artifacts and improves dense predictions. Vision–language foundation models (ML 09) and diffusion Transformers (ML 07) build on the same block. At the same time, hierarchical and hybrid designs remain common for dense, high-resolution tasks, modernized CNNs remain competitive under matched budgets (ML 03), and state-space and linear-attention models continue to be explored for very long token sequences such as video.

Open problems. Quadratic cost at high resolution and long video; the large data and compute appetite of plain ViTs; position encodings that generalize across resolutions and aspect ratios; interpreting attention without over-reading it; and fair, compute-matched comparisons between attention, convolution and state-space mechanisms.

Key takeaways

- Self-attention is a learned, input-dependent weighted average: $\text{softmax}(QK^\top/\sqrt{d_k})V$. The $\sqrt{d_k}$ keeps scores at unit variance so the softmax does not saturate.
- Multi-head attention runs h narrower heads in parallel at about the cost of one; masks (causal, window) and cross-attention control who reads whom.
- Attention is permutation-equivariant, so position must be injected: sinusoidal, learned absolute, relative bias or rotary encodings.
- A pre-norm Transformer block is $X + \text{MHA}(\text{LN}(X))$ then $U + \text{MLP}(\text{LN}(U))$; attention mixes tokens and the MLP transforms each token.
- ViT = patchify (a stride-P convolution) + position embeddings + class token + Transformer encoder. It has weak inductive bias and needs large pretraining or strong recipes (DeiT, AugReg).
- Swin brings locality and hierarchy back with shifted windows and patch merging; hybrids such as CvT, CoAtNet and MobileViT combine convolution and attention.
- FlashAttention makes exact attention faster without changing its quadratic arithmetic; windows, token merging and state-space scans change what is computed. Compare mechanisms only under matched budgets.
- Vision Mamba and VMamba adapt selective state spaces to image sequences. Multiple routes change influence paths; they do not automatically guarantee rotation equivariance or preserve every distant detail.

Exercises

1. **Token budget.** Calculate the number of tokens and attention-score entries per head for a 512×512 image with patches of size 16 and 8. By what factor does the score count grow when the patch size halves?

   Hint

   $N = (512/16)^2 = 1024$ and $(512/8)^2 = 4096$ tokens; $N^2 = 1{,}048{,}576$ and $16{,}777{,}216$ scores. Halving the patch size multiplies N by 4 and the scores by 16.

2. **Scaling check.** Draw random queries and keys with independent standard-normal components for $d_k = 4, 64, 1024$. Measure the variance of $q \cdot k$ with and without division by $\sqrt{d_k}$, and the largest softmax weight over 100 keys in each case. What happens to the softmax without scaling?

   Hint

   The variance of the raw dot product is close to d_k; after scaling it is close to 1. Without scaling, at $d_k = 1024$ the softmax puts nearly all weight on a single key (it becomes almost one-hot), so gradients through the other keys vanish.

3. **Remove position.** In the TinyViT code, set the position embeddings to zero, drop the class token and use mean pooling, then permute the patch tokens before the blocks and compare the logits in evaluation mode. Explain the result, and what it implies for an image classifier without positional information.

Hint

The logits agree up to floating-point error, because every block is permutation-equivariant and mean pooling is permutation-invariant. Use dropout off (`eval()`) and permute tokens, not pixels inside a patch. Such a model cannot tell a face from a scrambled face built from the same patches.

4. **Cross-attention shapes.** In a text-to-image model, 256 image tokens of width 320 read 77 text tokens of width 768 with 8 heads of width 40. Give the shapes of W_Q, W_K, W_V, one head's score matrix, and the output.

Hint

$W_Q \in \mathbb{R}^{320\times320}$ (8 heads × 40), $W_K, W_V \in \mathbb{R}^{768\times320}$. Each head's scores are 256×77. The output follows the queries: 256×320 after the output projection. Key and value lengths must match each other, not the query length.

5. **Swin arithmetic.** Using the cost formulas, compare global and window attention for a 56×56 token grid with $C = 96$ and $M = 7$. Which term dominates in each case?

Hint

$hw = 3136$. Global: $4 \cdot 3136 \cdot 96^2 \approx 1.16 \times 10^8$ plus $2 \cdot 3136^2 \cdot 96 \approx 1.89 \times 10^9$, so the quadratic term dominates. Window: the same 1.16×10^8 plus $2 \cdot 49 \cdot 3136 \cdot 96 \approx 2.95 \times 10^7$, so the projections dominate. The window version is about 14 times cheaper here.

6. **Fair comparison.** You have 2,000 labeled images and access to pretrained CNN, ViT and visual Mamba backbones. Design an experiment that tells you which backbone to deploy, and list what must be held fixed. Include a mixer-removal ablation and a resolution/rotation test.

Hint

Compare linear probes on frozen features first, then fine-tuning with matched budgets and tuned learning rates per model on a validation split. Record pretraining data, input resolution, parameters, FLOPs and measured latency on the target device. A pretrained ViT against a CNN trained from scratch tests pretraining, not architecture. For the scan backbone, record state size and test whether benefits survive removing the mixer while retaining gating and local convolution. Keep the same preprocessing in the resolution and rotation tests.

References

1. A. Vaswani et al., “Attention Is All You Need,” NeurIPS, 2017. arXiv
2. A. Dosovitskiy et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” ICLR, 2021. arXiv
3. S. Khan, M. Naseer, M. Hayat, S. W. Zamir, F. S. Khan and M. Shah, “Transformers in Vision: A Survey,” *ACM Computing Surveys*, 2022. arXiv
4. H. Touvron et al., “Training data-efficient image transformers & distillation through attention,” ICML, 2021. arXiv
5. Z. Liu et al., “Swin Transformer: Hierarchical Vision Transformer using Shifted Windows,” ICCV, 2021. arXiv
6. T. Dao, D. Y. Fu, S. Ermon, A. Rudra and C. Ré, “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness,” NeurIPS, 2022. arXiv
7. PyTorch, `scaled_dot_product_attention` and `TransformerEncoderLayer` documentation. Attention, Encoder layer
8. R. Xu et al., “Visual Mamba: A Survey and New Outlooks,” arXiv:2404.18861, 2024. arXiv
9. W. Yu and X. Wang, “MambaOut: Do We Really Need Mamba for Vision?,” CVPR, 2025. Paper
10. C. M. Bishop and H. Bishop, *Deep Learning: Foundations and Concepts*, Springer, 2024, ch. 12. Book
11. K. Han et al., “A Survey on Vision Transformer,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2023. DOI
12. Y. Liu et al., “A Survey of Visual Transformers,” *IEEE Transactions on Neural Networks and Learning Systems*, 2024. DOI
13. L. Papa, P. Russo, I. Amerini and L. Zhou, “A Survey on Efficient Vision Transformers: Algorithms, Techniques, and Performance Benchmarking,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024. DOI
14. J. L. Ba, J. R. Kiros and G. E. Hinton, “Layer Normalization,” arXiv:1607.06450, 2016. arXiv
15. R. Xiong et al., “On Layer Normalization in the Transformer Architecture,” ICML, 2020. arXiv
16. J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo and Y. Liu, “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *Neurocomputing*, 2024. DOI
17. P. Shaw, J. Uszkoreit and A. Vaswani, “Self-Attention with Relative Position Representations,” NAACL, 2018. arXiv
18. M. Raghu, T. Unterthiner, S. Kornblith, C. Zhang and A. Dosovitskiy, “Do Vision Transformers See Like Convolutional Neural Networks?,” NeurIPS, 2021. arXiv
19. H. Wu et al., “CvT: Introducing Convolutions to Vision Transformers,” ICCV, 2021. arXiv
20. Z. Dai, H. Liu, Q. V. Le and M. Tan, “CoAtNet: Marrying Convolution and Attention for All Data Sizes,” NeurIPS, 2021. arXiv
21. D. Bolya et al., “Token Merging: Your ViT But Faster,” ICLR, 2023. arXiv
22. T. Darcet, M. Oquab, J. Mairal and P. Bojanowski, “Vision Transformers Need Registers,” ICLR, 2024. arXiv
23. M. Oquab et al., “DINOv2: Learning Robust Visual Features without Supervision,” *Transactions on Machine Learning Research*, 2024. arXiv
24. M. Dehghani et al., “Scaling Vision Transformers to 22 Billion Parameters,” ICML, 2023. arXiv
25. A. Steiner et al., “How to train your ViT? Data, Augmentation, and Regularization in Vision

Transformers," *Transactions on Machine Learning Research*, 2022. arXiv
26. D. Hendrycks and K. Gimpel, "Gaussian Error Linear Units (GELUs)," arXiv:1606.08415, 2016. arXiv
27. H. Touvron, M. Cord and H. Jégou, "DeiT III: Revenge of the ViT," ECCV, 2022. arXiv
28. S. Mehta and M. Rastegari, "MobileViT: Light-weight, General-purpose, and Mobile-friendly Vision Transformer," ICLR, 2022. arXiv
29. T. Dao, "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning," ICLR, 2024. arXiv
30. L. Zhu et al., "Vision Mamba: Efficient Visual Representation Learning with Bidirectional State Space Model," ICML, 2024. arXiv
31. Y. Liu et al., "VMamba: Visual State Space Model," NeurIPS, 2024. arXiv
32. Z. Liu et al., "Swin Transformer V2: Scaling Up Capacity and Resolution," CVPR, 2022. arXiv
33. A. Gu and T. Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," COLM, 2024. Author version
34. T. Dao and A. Gu, "Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality," ICML, PMLR 235, 2024. Paper

ML 05 • Representation Learning and Self-Supervision

What a representation is, why distributed features transfer, and how SimCLR, MoCo, BYOL, SimSiam, DINO, MAE, JEPA and CLIP learn them without labels, with honest linear-probe and fine-tuning evaluation.

Online edition: https://blog.shyandram.dev/en/learn/ml/ml-05-representation-learning/

What you'll learn

- What a **representation** is, why **distributed** representations generalize better than one-hot codes, and which invariances a good representation should (and should not) have.
- How **transfer learning** works: pretrain an encoder once, then reuse it through k-NN, a linear probe or fine-tuning.
- How the main **self-supervised learning** families build a training signal from unlabeled data: contrastive (SimCLR, MoCo, with the InfoNCE loss), teacher–student without negatives (BYOL, SimSiam, DINO), redundancy reduction (Barlow Twins, VICReg), masked modeling (BEiT, MAE) and feature prediction (I-JEPA, V-JEPA 2).
- Why "make two views agree" can **collapse**, and how alignment and uniformity make that precise.
- How **CLIP** uses captions as supervision and enables zero-shot classification.
- How to evaluate a representation honestly, including why fine-tuning can underperform a linear probe and what LP-FT does about it.

The big picture

Suppose you have a million unlabeled images but only 500 labeled examples for the task you care about. Training a large network from scratch on those 500 examples ignores almost all the available data. **Representation learning** splits the job in two. First, learn an encoder $h = f_\theta(x)$ from a broad signal that does not need your labels. Second, train a small task-specific head on top of h using the few labels you have.

> **Explain like I'm five**
>
> Before a child learns the names of every bird, they already notice wings, beaks, colors and how things move. Later, a parent points at three robins and two crows, and that is enough: the child can tell robins from crows from then on. The "noticing" was learned for free, just by looking at the world. A representation is that reusable way of noticing things, and self-supervised learning is the "just looking" part.

Why it matters: almost every modern vision system starts from a pretrained encoder. Image classifiers, detectors, segmentation models, image–text models and the image encoders inside generative models are built on representations learned this way. The quality of that starting point often matters more than the details of the task head.

This chapter starts with what a representation is and why transfer works, then walks through the self-supervised objectives that dominate practice, and ends with how to measure whether a representation is any good. It assumes you know neural networks (ML 02), CNNs (ML 03) and vision transformers (ML 04). Linear factor models and autoencoders, the classical route to representations, have their own chapter: ML 05A.

What is a representation?

Plain version. A representation is the list of numbers a model uses to describe an input. A good one makes the things you care about easy to read off, and hides the things you do not care about.

Precise version. An encoder is a function $f_\theta : \mathcal{X} \to \mathbb{R}^D$ from inputs x (here, images) to feature vectors $h = f_\theta(x)$, where θ are the learnable parameters and D is the feature dimension. A task head g maps h to a prediction, so the full model is $g \circ f_\theta$. We call h a good representation for a task when a *simple* head, such as a linear classifier, can solve the task from h using *few* labeled examples. Bengio, Courville and Vincent's review [1] is the classic statement of this view; it lists the "priors" that make representations useful, such as smoothness, multiple explanatory factors, a hierarchy of increasingly abstract features, and shared factors across tasks.

Local versus distributed representations

The simplest representation of a category is **one-hot**: one unit per concept, with exactly one unit on. It is easy to read but it cannot generalize. Knowing about "red car" says nothing about "red truck", because they share no active units. A **distributed** representation describes each input with many features that are each shared by many inputs: "red", "blue", "car", "truck". Now "red truck" reuses the "red" unit learned from red cars.

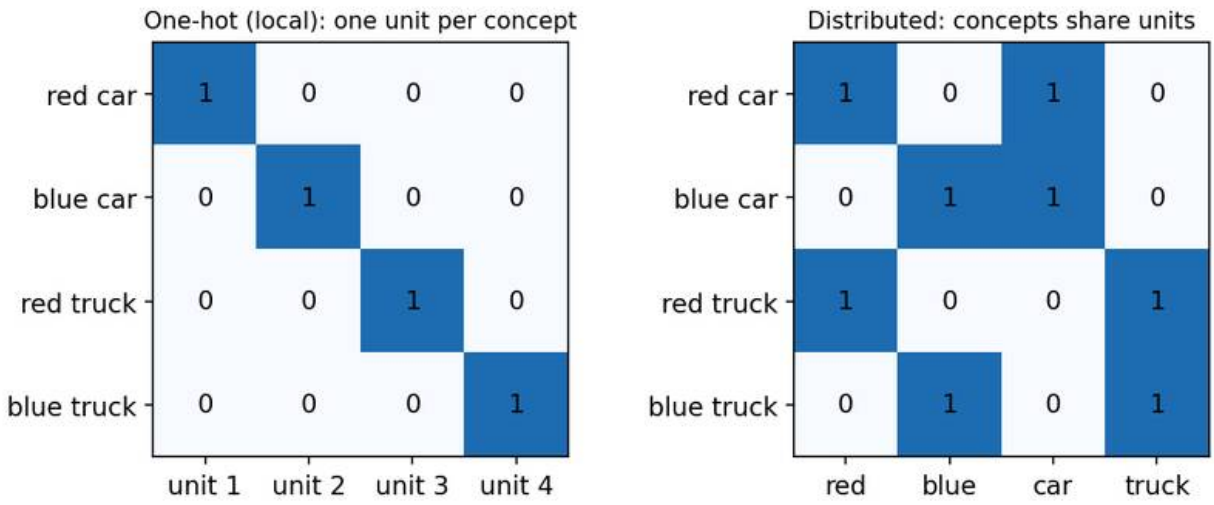


Figure 1 — Local (one-hot) versus distributed codes for four concepts. With distributed codes, a new combination such as a red truck reuses features learned from other examples.

The counting argument is striking. With D binary features, a one-hot code distinguishes D concepts, while a distributed code can in principle distinguish 2^D configurations [1]. Deep networks learn distributed codes automatically: each hidden unit responds to many inputs, and each input activates many units. Most of the practical value of representation learning comes from this sharing.

Invariance and equivariance

A useful representation keeps differences relevant to the task and suppresses differences that distract from it.

- A feature is **invariant** to a transformation T if $f(T(x)) = f(x)$: shifting a cat by two pixels should not change "this is a cat".
- A feature is **equivariant** if it changes in a predictable way, $f(T(x)) = T'(f(x))$: shifting the cat should shift its segmentation mask by the same amount.

"Useful" always depends on the downstream task. Color invariance helps object recognition but hurts fruit-ripeness estimation. Rotation invariance helps for textures but confuses 6 and 9. Dense prediction (segmentation, depth) needs spatially equivariant patch features, not a single invariant vector per image. We will see that self-supervised objectives choose their invariances mostly through the **data augmentations** they use, so this choice is a modeling decision, not a detail.

Learning distances: metric learning

PCA, introduced in ML 01 and developed in ML 05A, keeps the directions of highest variance. But high variance may come from lighting rather than object identity. Supervised learning shapes

features around labels; **metric learning** shapes distances around pairs that should match. For an anchor a, a positive p (same identity) and a negative n (different identity), the triplet loss is

$$\mathcal{L}_{\text{triplet}} = \max\left(0,\ \|h_a - h_p\|^2 - \|h_a - h_n\|^2 + m\right),$$

where h_a, h_p, h_n are the three feature vectors and $m > 0$ is a margin. The loss is zero once the positive is closer than the negative by at least m. An easy negative gives zero loss and teaches nothing, so choosing informative ("hard") negatives matters. Contrastive self-supervised learning, below, is metric learning in which the positives are created by augmentation instead of by labels.

Transfer learning: pretrain once, reuse many times

Plain version. Train a network on a big task, keep its feature extractor, and reuse it for a small task. You can freeze the features and only train a new head, or let the whole network keep learning.

Precise version. Pretraining finds encoder parameters $\theta^\star$ on a source dataset or objective. Transfer to a target task with labeled set $\{(x_i, y_i)\}_{i=1}^n$ uses one of three protocols:

1. **k-nearest neighbors (k-NN):** no training. Classify a test feature $f_{\theta^\star}(x)$ by the labels of its k most similar training features.
2. **Linear probe:** keep $\theta^\star$ frozen and fit a linear classifier $\hat{y} = \text{softmax}(Wh + b)$ on $h = f_{\theta^\star}(x)$, where W and b are the only trained parameters.
3. **Fine-tuning:** start from $\theta^\star$ and update the encoder together with the head, usually with a small learning rate.

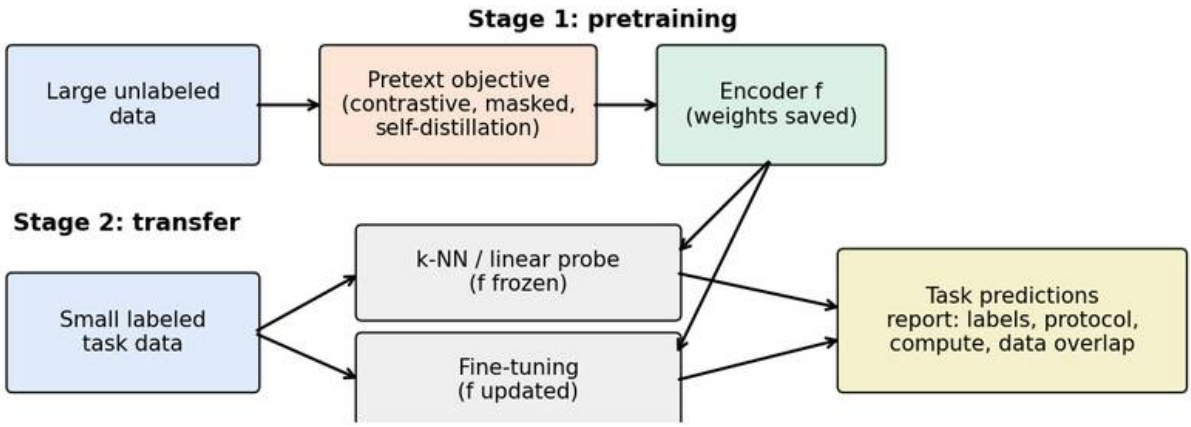


Figure 2 — The two-stage pattern of representation learning. Pretraining is expensive and done once; transfer is cheap and done per task.

Why does transfer work at all? Yosinski et al. [2] measured it layer by layer on ImageNet-trained CNNs. First-layer features (edge and color detectors much like Gabor filters) were general: they transferred well between tasks. Features became progressively more specific to the source task in higher layers, and transferability decreased as the source and target tasks became more different.

They also found that initializing from transferred features and fine-tuning improved generalization even on the target task, compared with training from random initialization.

For years, the default pretraining source was supervised ImageNet classification. Labels are expensive, though, and a label like "dog" tells the network nothing about the dog's pose, the grass or the lighting. Self-supervised learning removes the label bottleneck by making the data supervise itself.

Self-supervised learning: let the data make the labels

Plain version. Invent a task whose answer you can compute from the data itself: "are these two crops from the same photo?", "what was under this mask?", "which caption goes with this image?". Solving such a **pretext task** forces the network to learn useful features.

Precise version. A self-supervised objective is a loss $\mathcal{L}(\theta; x)$ computed from unlabeled inputs alone (or from naturally paired data such as image–caption pairs). The surveys by Liu et al. [3] and Gui et al. [4] and the practical "cookbook" by Balestriero et al. [5] organize the field into the families summarized here:

Family	Training signal	Representative methods	Main risk
Contrastive	Pull positive pairs together, push other samples apart	SimCLR, MoCo	Needs many negatives; false negatives
Self-distillation (no negatives)	Student matches a teacher on another view	BYOL, SimSiam, DINO	Collapse without asymmetry or centering
Redundancy reduction	Agreement plus decorrelated, non-constant feature dimensions	Barlow Twins, VICReg	Extra hyperparameters for the regularizers
Masked modeling	Predict hidden pixels or tokens	BEiT, MAE	Features may focus on low-level detail
Feature prediction	Predict features of hidden regions	I-JEPA, V-JEPA 2	Needs anti-collapse design like distillation
Language supervision	Match images with their captions	CLIP, SigLIP	Inherits web-data biases and noise

The families are learning objectives, not architectures: each can train a CNN or a transformer. Modern systems often combine several of them. The sections below take them in turn.

Contrastive learning and the InfoNCE loss

Plain version. Make two augmented versions of the same image agree, and make them disagree with every other image in the batch. It is a multiple-choice question: "out of these N candidates, which one is my partner?"

InfoNCE

Van den Oord, Li and Vinyals introduced the **InfoNCE** loss in Contrastive Predictive Coding (CPC) [6]. Take a query q, one positive key k^+ that "belongs" to it, and $N-1$ negative keys k_1^-, $\dots, k_{N-1}^-$. With a similarity score $s(\cdot,\cdot)$ and a temperature $\tau > 0$,

$$\mathcal{L}_{\text{InfoNCE}} = -\log \frac{\exp\left(s(q,k^+)/\tau\right)}{\exp\left(s(q,k^+)/\tau\right) + \sum_{j=1}^{N-1} \exp\left(s(q,k_j^-)/\tau\right)}.$$

This is exactly softmax cross-entropy for an N-way classification problem whose correct class is the positive. Two consequences follow.

- **A lower bound on mutual information.** CPC shows that $I(q;k^+) \geq \log N - \mathcal{L}_{\text{InfoNCE}}$, where I is mutual information. Minimizing the loss raises this lower bound, and more negatives (larger N) allow a higher bound. In practice this bound is loose, and the quality of the features depends heavily on the encoder and augmentations, so read it as motivation rather than as the explanation of why the features are good.
- **The temperature controls hardness.** Dividing by a small τ sharpens the softmax, so the loss concentrates on the negatives most similar to the query (the hard negatives).

SimCLR

SimCLR [7] is InfoNCE in its simplest form. Take a batch of B images and produce two augmented views of each (random crop and resize, color distortion, blur), giving $2B$ views. A backbone f maps each view to h, a small MLP **projection head** g maps h to $z = g(h)$, and the similarity is cosine similarity $s(z_i, z_j) = z_i^\top z_j / (\|z_i\|\|z_j\|)$. For view i with partner j,

$$\ell_{i,j} = -\log \frac{\exp(z_i^\top z_j/\tau)}{\sum_{k \neq i} \exp(z_i^\top z_k/\tau)},$$

where the z are ℓ_2-normalized and the sum runs over all $2B-1$ other views. Every other image in the batch is a negative, so no memory bank is needed, but the method benefits from large batches. The total loss averages $\ell_{i,j}$ over both orderings of every positive pair; the paper calls this the NT-Xent loss (normalized temperature-scaled cross-entropy).

Three findings from the SimCLR paper are worth remembering [7]. The *composition* of augmentations matters a lot, with random cropping plus color distortion being especially important. The projection head helps: features *before* the head (h) transfer better than those after it (z), so the head is discarded after pretraining. And contrastive learning benefits from larger batches and longer training than supervised learning.

Code: check the loss and its collapse value

This standalone PyTorch example uses synthetic paired vectors to verify the loss, the self-mask and the gradients. It does not train an encoder.

```
import torch
from torch.nn import functional as F

torch.manual_seed(0)
B, D = 8, 16
base = torch.randn(B, D)
views = torch.cat([base + .1*torch.randn_like(base),
                   base + .1*torch.randn_like(base)]).requires_grad_()
z = F.normalize(views, dim=-1)
logits = z @ z.T / .2
self_mask = torch.eye(2*B, dtype=torch.bool)
logits = logits.masked_fill(self_mask, float("-inf"))
positive = (torch.arange(2*B) + B) % (2*B)
loss = F.cross_entropy(logits, positive)
loss.backward()
assert torch.isfinite(views.grad).all()
print("paired loss:", round(loss.item(), 4))

collapsed = F.normalize(torch.ones(2*B, D), dim=-1)
scores = (collapsed @ collapsed.T / .2).masked_fill(self_mask, float("-inf"))
collapsed_loss = F.cross_entropy(scores, positive)
print("collapsed loss:", round(collapsed_loss.item(), 4))
print("expected:", round(torch.log(torch.tensor(2*B-1.)).item(), 4))
```

It prints a paired loss of 0.1423 and a collapsed loss of 2.7081, equal to $\log 15$. If every representation is identical, the positive gets probability $1/(2B-1)$ and the loss is $\log(2B-1)$, the worst value a uniform guess can get. Contrastive discrimination therefore makes complete collapse a bad solution, although optimization and negative selection can still be difficult.

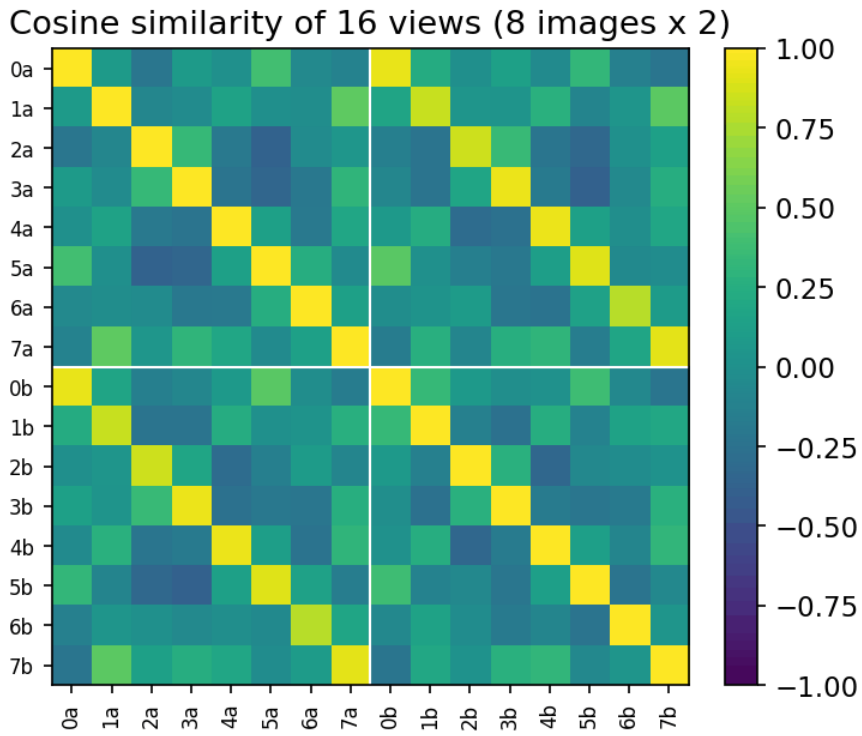


Figure 3 — Cosine similarities between 16 views (8 digit images, two augmentations each) after training the small SimCLR encoder used later in this chapter. The bright off-diagonal stripes are the positive pairs; InfoNCE pushes them up and everything else down.

MoCo: a queue of negatives and a momentum encoder

InfoNCE prefers many negatives, but SimCLR gets them only from the current batch. **Momentum Contrast (MoCo)** [8] decouples the number of negatives from the batch size with two ideas.

1. **A queue.** Keys encoded in recent batches are stored in a first-in, first-out queue of length K and reused as negatives. The query sees K negatives no matter how small the batch is.
2. **A momentum encoder.** Keys in the queue were produced by slightly older networks. If the key encoder changed quickly, old and new keys would be inconsistent. MoCo therefore computes keys with a separate encoder whose parameters θ_k follow the query encoder's parameters θ_q slowly:

$$\theta_k \leftarrow m\,\theta_k + (1-m)\,\theta_q,$$

where $m \in [0,1)$ is the momentum, close to 1 (the paper's default is 0.999). Only θ_q receives gradients. This **exponential moving average (EMA)** teacher idea reappears in BYOL, DINO and JEPA.

```
import torch
```

```
from torch.nn import functional as F

torch.manual_seed(0)
B, D, K = 8, 16, 32          # batch, embedding size, queue length

# --- MoCo-style InfoNCE: one positive key per query, negatives from a queue
q = F.normalize(torch.randn(B, D), dim=1)                 # queries (online encoder)
k = F.normalize(q + 0.1 * torch.randn(B, D), dim=1)       # keys (momentum encoder)
queue = F.normalize(torch.randn(K, D), dim=1)             # keys from past batches
l_pos = (q * k).sum(1, keepdim=True)                      # B x 1
l_neg = q @ queue.T                                       # B x K
logits = torch.cat([l_pos, l_neg], dim=1) / 0.07
loss_moco = F.cross_entropy(logits, torch.zeros(B, dtype=torch.long))
queue = torch.cat([k, queue])[:K]                         # enqueue new, drop oldest
print("MoCo logits:", tuple(logits.shape), "loss:", round(loss_moco.item(), 3))

# momentum (EMA) update of the key encoder's parameters
theta_q, theta_k, m = torch.randn(5), torch.zeros(5), 0.999
theta_k = m * theta_k + (1 - m) * theta_q
print("key params moved by:", round((theta_k.norm() / theta_q.norm()).item(), 4))
```

Each query gets a $1 + K$-way classification problem whose correct answer is always column 0, hence the all-zero targets. The output shows logits of shape `(8, 33)`, and the EMA step moves the key encoder only 0.1% of the way toward the query encoder (from zero, after one step). In a real implementation, q and k come from the two encoders applied to two augmentations of the same images.

Alignment and uniformity

Wang and Isola [9] gave a clean geometric reading of what contrastive losses do on the unit sphere. With normalized features, the loss asymptotically optimizes two properties:

$$\mathcal{L}_{\text{align}} = \mathbb{E}_{(x,x^+)} \|f(x) - f(x^+)\|_2^2, \qquad \mathcal{L}_{\text{unif}} = \log \mathbb{E}_{x,y}\, e^{-t\|f(x)-f(y)\|_2^2},$$

where (x, x^+) is a positive pair, x, y are independent samples, and $t > 0$ (commonly $t = 2$). **Alignment** is low when positive pairs map to nearby points. **Uniformity** is low when features spread out over the sphere, so they keep as much information about the input as possible. A collapsed encoder has perfect alignment (0) and the worst uniformity (0); a useful encoder has both small.

```
import torch
from torch.nn import functional as F

def alignment(z1, z2):            # mean squared distance between positive pairs
    return (z1 - z2).square().sum(1).mean()

def uniformity(z, t=2.0):         # log of mean Gaussian potential over all pairs
    return torch.pdist(z).square().mul(-t).exp().mean().log()
```

```
torch.manual_seed(0)
n, D = 512, 16
collapsed = F.normalize(torch.ones(n, D), dim=1)
spread = F.normalize(torch.randn(n, D), dim=1)
noisy_view = F.normalize(spread + 0.1 * torch.randn(n, D), dim=1)
print("collapsed: align", alignment(collapsed, collapsed).item(),
      "uniform", round(uniformity(collapsed).item(), 3))
print("spread:    align", round(alignment(spread, noisy_view).item(), 3),
      "uniform", round(uniformity(spread).item(), 3))
```

The collapsed set scores alignment 0.0 and uniformity 0.0; the spread set scores about 0.138 and −3.51. Figure 4 tracks the same two quantities while training the digit encoder below. At step 0 the random network maps every view to almost the same point, so alignment is near zero only because everything has collapsed. Training first spreads the features out (uniformity drops sharply), then gradually pulls positives back together.

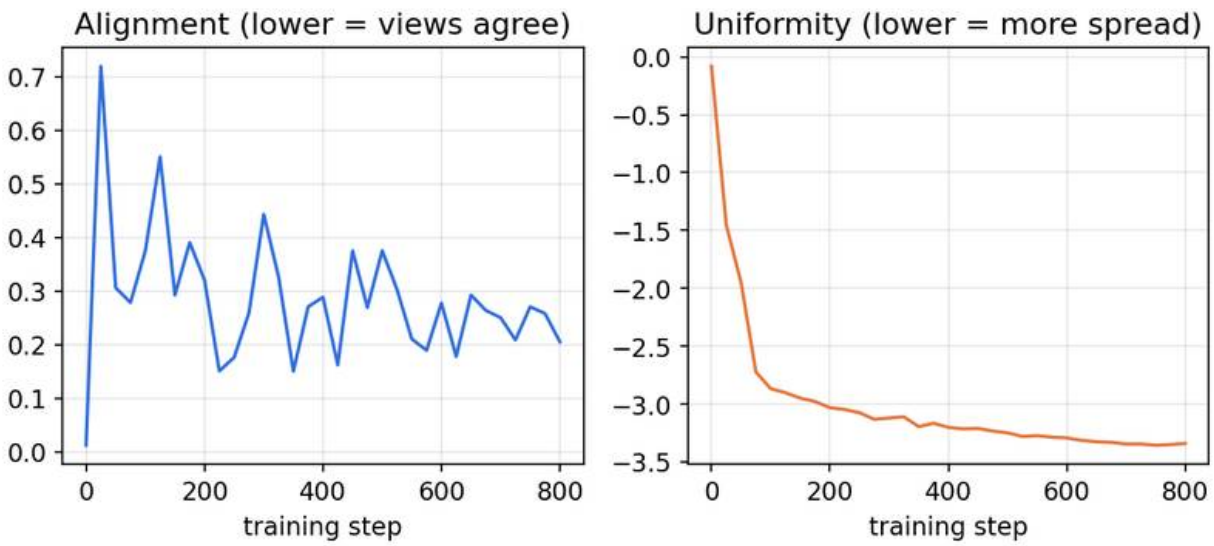


Figure 4 — Alignment and uniformity of the projection features during SimCLR training on 8×8 digits. Low alignment alone is meaningless (a constant encoder has it); the useful regime is low alignment together with low uniformity.

A worked example: SimCLR on 8×8 digits

Plain version. Train a tiny SimCLR encoder on scikit-learn's digits without using any labels, then check whether its features make a nearest-neighbor classifier more robust.

The augmentation here is a random shift of up to one pixel, a random brightness change and pixel noise. We therefore expect the learned features to be roughly invariant to small shifts, something raw pixels are not. The evaluation uses the same labeled training set for every feature type and tests on clean digits and on digits shifted by up to one pixel.

```
import copy
import numpy as np
import torch
from torch import nn
from torch.nn import functional as F
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

torch.manual_seed(0); np.random.seed(0); torch.set_num_threads(1)
d = load_digits()
Xtr, Xte, ytr, yte = train_test_split(d.data / 16., d.target, test_size=.3,
                                      random_state=0, stratify=d.target)
Xtr_t = torch.tensor(Xtr, dtype=torch.float32)

def augment(x):                       # the augmentations define the invariances
    img = F.pad(x.view(-1, 1, 8, 8), (1, 1, 1, 1))
    i, j = np.random.randint(0, 3, size=2)               # shift by up to 1 px
    img = img[:, :, i:i + 8, j:j + 8]
    img = img * (0.7 + 0.6 * torch.rand(len(img), 1, 1, 1))  # brightness
    return (img + 0.1 * torch.randn_like(img)).reshape(len(x), 64)  # noise

backbone = nn.Sequential(nn.Unflatten(1, (1, 8, 8)),
                         nn.Conv2d(1, 32, 3, padding=1), nn.ReLU(),
                         nn.Conv2d(32, 64, 3, padding=1), nn.ReLU(),
                         nn.AdaptiveAvgPool2d(2), nn.Flatten())  # h: 256-d
random_backbone = copy.deepcopy(backbone)                    # baseline
head = nn.Sequential(nn.ReLU(), nn.Linear(256, 64))          # projection g
opt = torch.optim.Adam([*backbone.parameters(), *head.parameters()], lr=1e-3)

def nt_xent(z1, z2, tau=0.2):
    z = F.normalize(torch.cat([z1, z2]), dim=-1)
    n = len(z1)
    logits = (z @ z.T / tau).fill_diagonal_(float("-inf"))
    return F.cross_entropy(logits, (torch.arange(2 * n) + n) % (2 * n))

for step in range(800):                                      # about a minute on CPU
    x = Xtr_t[torch.randint(len(Xtr_t), (256,))]
    loss = nt_xent(head(backbone(augment(x))), head(backbone(augment(x))))
    opt.zero_grad(); loss.backward(); opt.step()

def shift(x, dy, dx):                    # move one 8x8 image, fill with zeros
    img = np.zeros((10, 10)); img[1:9, 1:9] = x.reshape(8, 8)
    return img[1 - dy:9 - dy, 1 - dx:9 - dx].ravel()

moves = np.random.default_rng(0).integers(-1, 2, size=(len(Xte), 2))
Xte_shift = np.stack([shift(x, *m) for x, m in zip(Xte, moves)])
enc = lambda net: (lambda A: net(torch.tensor(A, dtype=torch.float32)).detach().numpy())
pca = PCA(32, random_state=0).fit(Xtr)
for name, f in [("raw pixels", lambda A: A), ("PCA-32", pca.transform),
                ("random CNN", enc(random_backbone)), ("SimCLR CNN", enc(backbone))]:
    knn = KNeighborsClassifier(5, metric="cosine").fit(f(Xtr), ytr)
```

```
    print(f"{name:11s} clean {knn.score(f(Xte), yte):.3f}  "
          f"shifted {knn.score(f(Xte_shift), yte):.3f}")
```

On our machine this prints:

```
raw pixels  clean 0.978  shifted 0.524
PCA-32      clean 0.978  shifted 0.507
random CNN  clean 0.974  shifted 0.524
SimCLR CNN  clean 0.987  shifted 0.919
```

Exact numbers vary slightly across PyTorch versions and hardware. The pattern is the lesson. On clean digits, raw pixels are already excellent: this dataset is easy, and a fancy encoder adds little. On shifted digits, raw pixels, PCA and an untrained CNN of the same architecture all drop to about 50%, while the self-supervised features keep most of their accuracy. The network learned the invariance we asked for through the augmentations, and nothing more: had we tested on rotated digits, which the augmentations never showed, we should not expect the same robustness.

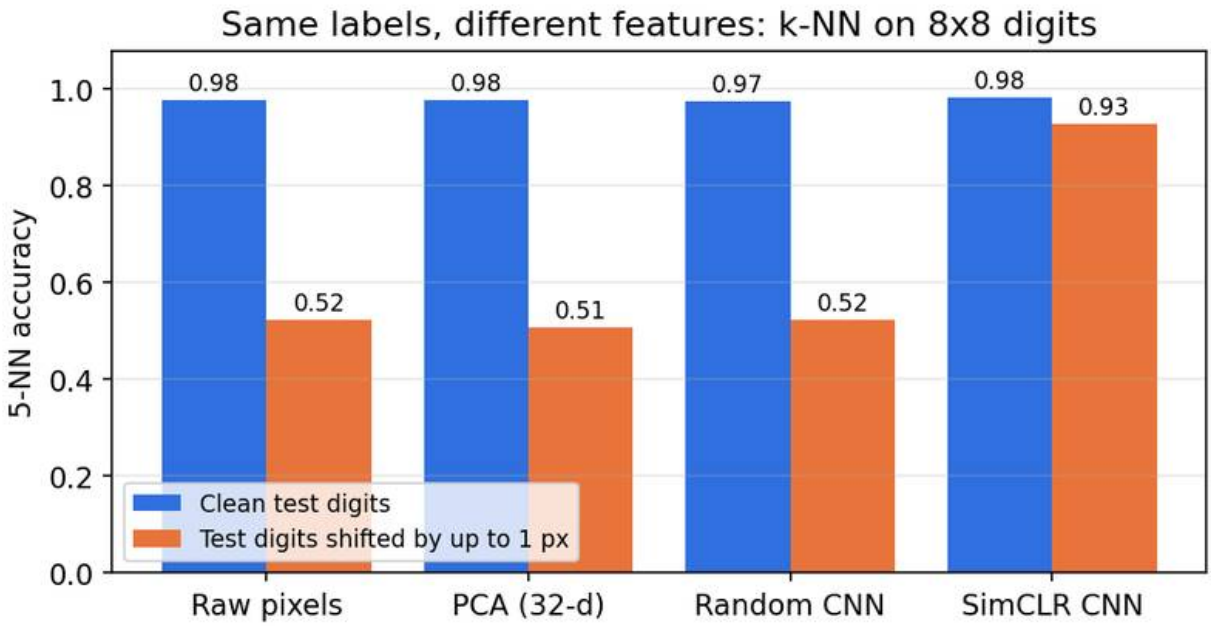


Figure 5 — Same labels, same classifier, different features. The figure script uses a separate random stream, so its numbers differ from the printed output by up to 0.01. The random CNN rules out the explanation that convolution alone produces the robustness.

Without negatives: why matching alone can collapse

Plain version. If the only instruction is "make the two outputs agree", returning the same constant for every image solves it perfectly. Methods without negatives need some other mechanism to stop

this.

Minimizing $\|f_\theta(v_1) - f_\theta(v_2)\|^2$ over two views v_1, v_2 has the trivial solution $f_\theta(x) = c$ for a constant c. Agreement alone does not require the features to carry any information. **Complete collapse** maps everything to one point; **dimensional collapse** is subtler: the features vary, but only along a few directions, wasting most of the D dimensions.

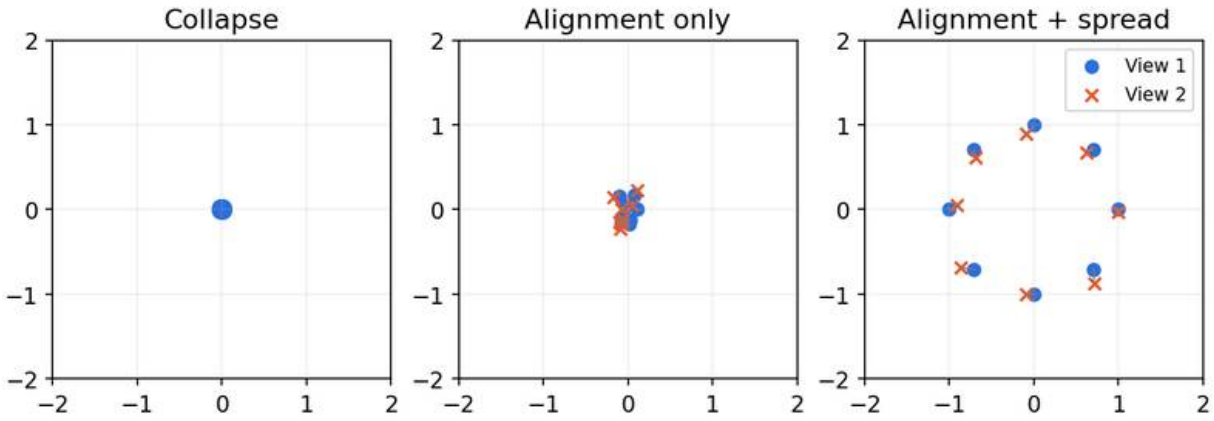


Figure 6 — Schematic of collapse, alignment without spread, and alignment with spread. The points are constructed illustrations, not embeddings learned by a published model.

BYOL: an online network chases a slow target

BYOL [10] uses two networks. The **online** network (encoder, projector and an extra **predictor** q_ϕ) is trained to predict the **target** network's projection of a different view of the same image. The target is not trained by gradients; it is an EMA of the online weights, as in MoCo:

$$\theta_{\text{target}} \leftarrow m\,\theta_{\text{target}} + (1-m)\,\theta_{\text{online}}.$$

The loss is the squared distance between the normalized prediction and the normalized target projection, symmetrized over the two views. The target output is treated as a constant (**stop-gradient**). There are no negatives, yet BYOL does not collapse in practice. The asymmetry (predictor on one side, stop-gradient and EMA on the other) and the full training recipe matter; EMA alone does not prove collapse impossible.

SimSiam: stop-gradient is the key ingredient

Chen and He's SimSiam [11] stripped BYOL down further: one shared encoder for both views, a predictor h on one branch, **no** momentum encoder, **no** negatives and no large batches. With $p_1 = h(f(v_1))$ and $z_2 = f(v_2)$,

$$\mathcal{L} = \tfrac{1}{2}\,\mathcal{D}(p_1, \text{sg}(z_2)) + \tfrac{1}{2}\,\mathcal{D}(p_2, \text{sg}(z_1)), \qquad \mathcal{D}(p, z) = -\frac{p^\top z}{\|p\|\,\|z\|},$$

where sg is stop-gradient and $\mathcal{D}$ is negative cosine similarity. The paper's experiments show that removing the stop-gradient makes the model collapse quickly, while the predictor is also needed. The authors interpret the method as an alternating, expectation-maximization-like optimization. The practical lesson: very small architectural asymmetries decide whether these methods learn or collapse.

DINO: self-distillation with centering and sharpening

DINO [12] turns the agreement problem into classification with soft labels. Teacher and student both output a probability vector over K "prototype" dimensions via a temperature-scaled softmax. The student, seeing a view x', is trained to match the teacher's distribution on another view x:

$$\mathcal{L} = -\sum_{k=1}^{K} P_t(x)_k \, \log P_s(x')_k, \qquad P_t(x) = \text{softmax}\Big(\frac{g_t(x) - c}{\tau_t}\Big),$$

where g_t is the teacher network (an EMA of the student), c is a running mean of teacher outputs (**centering**) and τ_t is a low teacher temperature (**sharpening**). Centering stops one dimension from dominating; sharpening stops the output from becoming uniform. Each alone collapses in a different way; together they balance. DINO also uses **multi-crop**: the teacher sees only large global crops, while the student also sees small local crops, so the student must infer global content from local evidence. With vision transformers, DINO's attention maps were notable for highlighting object boundaries without any segmentation labels.

Barlow Twins and VICReg: regularize the statistics directly

A different route is to forbid collapse explicitly in the loss. **Barlow Twins** [13] computes the $D \times D$ cross-correlation matrix C between the (batch-standardized) embeddings of the two views and pushes it toward the identity:

$$\mathcal{L}_{\text{BT}} = \sum_i (1 - C_{ii})^2 + \lambda \sum_i \sum_{j \neq i} C_{ij}^2,$$

where the first term makes each dimension agree across views and the second, weighted by $\lambda > 0$, decorrelates different dimensions so they do not carry redundant information. **VICReg** [14] splits the same idea into three named terms: **invariance** (mean squared distance between the two embeddings), **variance** (a hinge loss keeping the standard deviation of every dimension above a threshold, which directly forbids constant outputs) and **covariance** (penalizing off-diagonal covariance entries). "No negatives" does not mean "no anti-collapse design"; these methods simply put the design in the loss instead of in the architecture.

Masked modeling: learn by filling gaps

Plain version. Hide part of the input and predict what is missing. This is how BERT learned language, and it works for images too, once you choose the right amount of hiding.

BEiT [15] brought BERT-style pretraining to vision transformers. An image is split into patches; some are masked, and the network predicts discrete **visual tokens** for the masked positions, where the tokens come from a separately trained image tokenizer.

MAE [16] predicts raw pixels instead. It masks a large fraction of the patches, sends only the visible ones through the encoder, and reconstructs the missing pixels with a lightweight decoder. With the set of masked patch indices $\mathcal{M}$,

$$\mathcal{L}_{\text{mask}} = \frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} \|\hat{x}_i - x_i\|^2,$$

where x_i is the true pixel vector of patch i and $\hat{x}_i$ is the prediction. MAE's design choices are deliberate. A high masking ratio (75% by default) makes the task hard enough that the network cannot simply interpolate from neighbors. The encoder only processes visible patches, which cuts pretraining cost. Normalizing each target patch by its own mean and variance improves the features. After pretraining, the decoder is discarded.

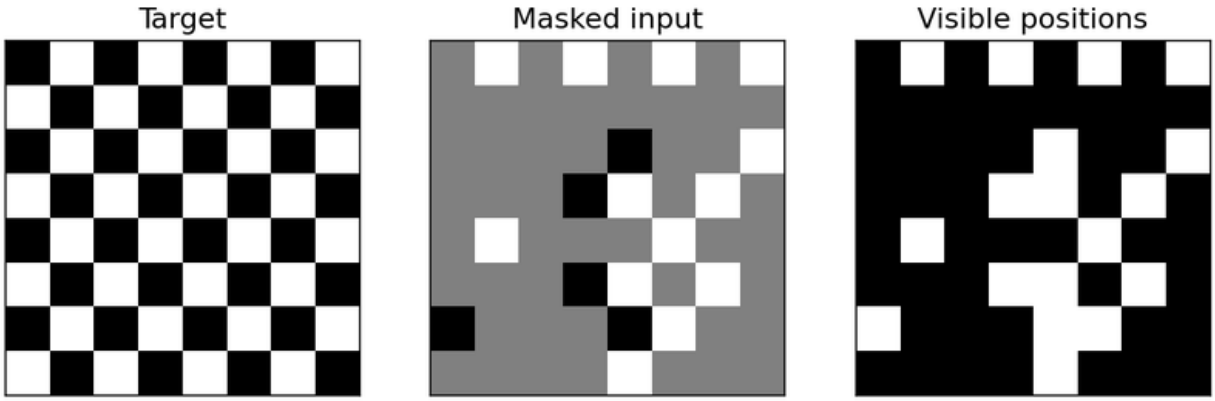


Figure 7 — Masked prediction creates its own targets from the input. The checkerboard illustrates the masking operation, not semantic understanding.

Reconstruction is not the same as generation. A model that predicts masked pixels from visible ones is not trained to sample complete images from scratch. Low reconstruction loss does not guarantee the best semantic features either: background texture can dominate pixel error. Masked-modeling features are typically weaker than contrastive or distillation features under a linear probe but strong after fine-tuning, which is one reason the evaluation protocol must be stated.

Predicting features instead of pixels: JEPA

Plain version. Instead of painting in every uncertain pixel, predict a *description* of the hidden region.

I-JEPA [17] (Image Joint-Embedding Predictive Architecture) encodes a visible **context** block and predicts the encoder features of several **target** blocks from it. The target encoder is an EMA of the

context encoder, and a predictor receives the context features plus the positions of the targets. A simplified objective is

$$\mathcal{L}_{\text{pred}} = \left\| g_\phi(f_\theta(x_{\text{context}}), \text{positions}) - \text{sg}\left(f_{\bar{\theta}}(x_{\text{target}})\right) \right\|^2,$$

where f_θ is the context encoder, $f_{\bar{\theta}}$ the EMA target encoder, g_ϕ the predictor and sg stop-gradient. The formula summarizes the idea; the masking strategy (large, semantic-scale target blocks) and the architecture are part of the method, not optional details.

V-JEPA 2 [18] extends feature prediction to video at scale. Its report separates action-free pretraining on video from an action-conditioned model, trained afterward on a small amount of robot data, that is used for planning. Predicting future latent states, recognizing actions and generating photorealistic frames are different capabilities, and a "world model" claim should say which one it makes.

Language as supervision: CLIP

Plain version. The internet is full of images with captions. Train an image encoder and a text encoder so that each image lands next to its own caption. Afterwards you can classify images by writing the class names as sentences.

CLIP [19] trains on image–text pairs collected from the web. In a batch of N pairs, the image encoder gives normalized embeddings $u_1, \dots, u_N$ and the text encoder gives $v_1, \dots, v_N$. The $N \times N$ similarity matrix $S_{ij} = u_i^\top v_j / \tau$, with a learned temperature τ, has the correct pairs on its diagonal. The loss is the average of two InfoNCE losses, one over rows (which caption fits this image?) and one over columns (which image fits this caption?):

$$\mathcal{L}_{\text{CLIP}} = \frac{1}{2}\Big[\frac{1}{N}\sum_i -\log \frac{e^{S_{ii}}}{\sum_j e^{S_{ij}}} + \frac{1}{N}\sum_j -\log \frac{e^{S_{jj}}}{\sum_i e^{S_{ij}}}\Big].$$

Zero-shot classification follows directly. Embed one prompt per class, such as "a photo of a dog", and assign an image to the class whose prompt embedding is most similar. No task-specific labels are needed, although prompt wording affects accuracy.

SigLIP [20] replaces the softmax with an independent **sigmoid** loss on every image–text pair: each entry of the matrix becomes a yes/no question, "do these match?". This removes the need to normalize over the whole batch, which simplifies large-scale distributed training.

```
import torch
from torch.nn import functional as F

torch.manual_seed(0)
D = 16

# --- CLIP-style symmetric loss on N matched image-text pairs
N = 6
img = F.normalize(torch.randn(N, D), dim=1)
```

```
txt = F.normalize(img + 0.3 * torch.randn(N, D), dim=1)   # caption i matches image i
logit_scale = torch.tensor(1 / 0.07)
sim = logit_scale * img @ txt.T                           # N x N, diagonal = matches
labels = torch.arange(N)
loss_clip = (F.cross_entropy(sim, labels) + F.cross_entropy(sim.T, labels)) / 2
print("CLIP loss:", round(loss_clip.item(), 3))

# zero-shot classification: embed one prompt per class, pick the most similar
class_prompts = txt[:3]                                   # stand-ins for "a photo of a
 ↪  {class}"
pred = (img @ class_prompts.T).argmax(1)
print("zero-shot predictions for images 0-2:", pred[:3].tolist())

# SigLIP-style alternative: an independent yes/no decision for every pair
t, b = 10.0, -10.0                                        # learnable in practice
signs = 2 * torch.eye(N) - 1                              # +1 on matches, -1 elsewhere
loss_sig = -F.logsigmoid(signs * (t * img @ txt.T + b)).sum() / N
print("sigmoid loss:", round(loss_sig.item(), 3))
```

The synthetic captions are noisy copies of the image embeddings, so the CLIP loss is small (0.054) and the zero-shot step recovers the right "classes" `[0, 1, 2]`. In real CLIP the two encoders are a vision transformer or CNN and a text transformer, and the hard part is the data: what the captions describe, and what they leave out, determines what the representation knows. The sigmoid loss value (3.473) is not comparable to the softmax loss; it uses different units and a bias term b.

DINOv2 and DINOv3: features as a product

Plain version. With enough curated data and compute, pretraining can produce an encoder that you freeze and reuse for many different tasks, from classification to depth estimation.

DINOv2 [21] combines image-level self-distillation (as in DINO) with a patch-level masked objective, and pays close attention to **data curation**: building a large, diverse, deduplicated pretraining set from uncurated images by retrieval against curated ones. The result is a family of frozen encoders whose global and patch features work well with simple heads. DINOv3 [22], released in 2025, scales this further and adds **Gram anchoring** to counter the degradation of dense (patch) features during long training. Rather than only matching each feature independently, Gram anchoring constrains the matrix of similarities among patch features toward that of an earlier model. The report also distills a very large teacher into smaller models for deployment.

The practical question is whether frozen global or dense features solve **your** task. A model strong at semantic retrieval may miss a small scratch on a manufactured part. Increasing the input resolution and using patch features can help, but it changes the compute budget and the comparison protocol.

Evaluate transfer honestly

Plain version. A representation has earned its name only when it helps on data and tasks that were not used to build or choose it. How you test it changes the answer.

Protocol	What is trained	What it measures
k-NN	Nothing; similarity to labeled reference features	Local structure of the representation
Linear probe	Linear head; encoder frozen	Information accessible through a simple readout
Fine-tuning	Head plus some or all encoder layers	Adaptability under a stated label and compute budget
Dense transfer	Spatial head on patch maps	Suitability for pixel or region outputs

Linear probing versus fine-tuning

Fine-tuning has more freedom, so it usually wins when the target data is plentiful and similar to the pretraining data. It is not always better. Kumar et al. [23] showed theoretically and empirically that full fine-tuning can **distort** good pretrained features: while a randomly initialized head is still wrong, its large gradients flow into the encoder and change features in directions that suit the training distribution but hurt **out-of-distribution** (OOD) inputs. A linear probe cannot distort the features. Their remedy, **LP-FT**, is two-step: first train the linear head with the encoder frozen, then fine-tune everything starting from that head.

We reproduced the qualitative effect on the digits example. The encoder is the SimCLR CNN from above; the labeled training digits are clean; the test digits are shifted, a mild distribution shift. For each label budget, we average three random draws of the labeled examples.

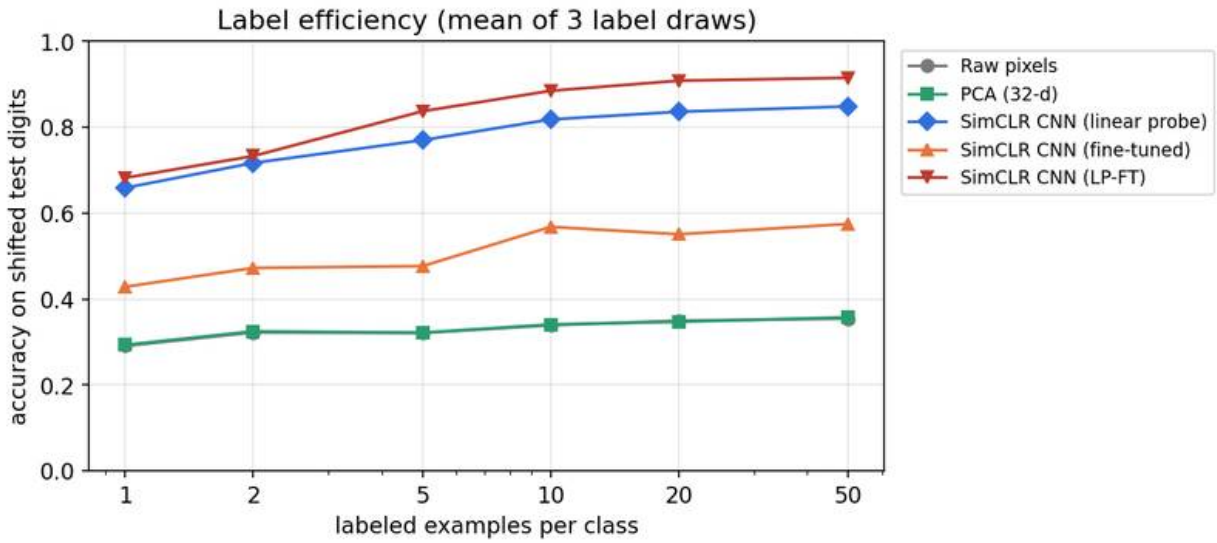


Figure 8 — Label efficiency on shifted test digits. Self-supervised features are far more label-efficient than

pixels or PCA. With one fixed recipe, plain fine-tuning from a random head hurt the shifted-test accuracy, while LP-FT improved on the probe. A toy result: with a tuned learning rate or more labels, plain fine-tuning can do better.

Read Figure 8 for its shape, not its exact values. The frozen SSL features with a linear head need only a handful of labels per class to beat raw pixels with 50 labels per class. Plain fine-tuning with a randomly initialized head did worse than the frozen probe here, and LP-FT did better than both, the same ordering Kumar et al. report under distribution shift. Fine-tuning needs its own hyperparameter search, and the head initialization matters.

Rules against leakage

- Normalize features consistently for similarity comparisons (cosine k-NN needs normalized vectors).
- Fit scalers, PCA and classifier hyperparameters on training and validation data only.
- A frozen encoder still has a training history: overlap between pretraining data and evaluation data can inflate results, especially for web-scale models.
- Report the number of labels, backbone, checkpoint, input resolution, augmentations, head, tuning budget and protocol. Choosing the best of many encoders on the test set is test leakage, even if none of them is fine-tuned.

Modern view

Surveys and review papers. Four reviews map the field from complementary angles.

- **Bengio, Courville and Vincent (TPAMI 2013) [1]** frame the whole enterprise. They ask what makes one representation better than another and answer with a list of generic priors: distributed codes, depth and abstraction, disentangling of explanatory factors, manifolds and natural clustering. They review probabilistic models, autoencoders and manifold learning. It predates modern SSL but remains the best statement of *why* we learn representations.
- **Liu et al. (TKDE 2023) [3]** classify self-supervised methods into **generative** (autoencoding and autoregressive reconstruction), **contrastive** (context–instance and instance–instance) and **generative-contrastive** (adversarial) groups across vision, language and graphs, and discuss why contrastive objectives were winning on vision benchmarks at the time.
- **Gui et al. (TPAMI 2024) [4]** survey algorithms more recently, covering contrastive, self-distillation, redundancy-reduction and masked image modeling methods, and discuss applications and open problems such as theory, the choice of pretext task and multimodal SSL.
- **Balestriero et al. (2023) [5]** is a practitioner's guide written by many of the field's authors. It groups methods into a deep-metric-learning family (SimCLR and relatives), a self-distillation family (BYOL, SimSiam, DINO), a canonical-correlation-analysis family (VICReg, Barlow Twins) and masked image modeling, and collects the "tricks" (augmentations, projectors, EMA schedules, batch-size effects) that the papers often bury. Prince's textbook [24] gives a gentle, figure-rich introduction to the same ideas with notebooks.

State of practice (2023–2026). The trends visible in the papers cited above are:

- **Combined objectives.** Leading image encoders mix image-level distillation with patch-level masked prediction (DINOv2 [21], DINOv3 [22]) rather than relying on one pretext task.
- **Data as a first-class design choice.** DINOv2's curation pipeline and the web-scale pair collection behind CLIP [19] and SigLIP [20] show that what goes into pretraining matters as much as the loss.
- **Dense features.** Segmentation, depth and correspondence need good patch features, which motivated objectives like Gram anchoring [22].
- **Prediction in latent space and video.** I-JEPA [17] and V-JEPA 2 [18] predict features instead of pixels, and extend to video and action-conditioned prediction.
- **Language-aligned encoders as the default backbone** for multimodal models (see ML 09).

Open problems. We still lack a predictive theory of which pretext task suits which downstream task; collapse is prevented by recipes more than by guarantees; evaluation depends heavily on protocol and on overlap between pretraining and test data; and web-scale pretraining inherits biases and gaps from its data. Our reading of [3], [4], [5], [21], [22] is that data and target design deserve as much attention as architecture. That is an interpretation of the literature, not a theorem that one objective wins everywhere.

The next chapters change the question: instead of *describing* an image, how can a model describe and **sample** its whole distribution? See ML 05A for autoencoders and ML 06 for generative models.

Key takeaways

- A representation is useful when a simple head can solve the task from it with few labels. Distributed codes generalize because features are shared across examples.
- The invariances you want depend on the task. In self-supervised learning, the augmentations choose them, as the shifted-digits example showed.
- Contrastive methods (SimCLR, MoCo) are InfoNCE classification: find your positive among negatives. Collapse costs them $\log(2B-1)$.
- Methods without negatives (BYOL, SimSiam, DINO) avoid collapse through asymmetry: predictors, stop-gradient, EMA teachers, centering and sharpening. Barlow Twins and VICReg put the anti-collapse term in the loss.
- Masked modeling (BEiT, MAE) reconstructs hidden content; JEPA predicts hidden *features*. CLIP aligns images with captions and enables zero-shot classification.
- k-NN, linear probes and fine-tuning answer different questions. Fine-tuning can distort features under distribution shift; LP-FT is a simple fix.

Exercises

1. **Temperature and collapse.** In the first code block, change the temperature from 0.2 to 0.05 and to 1.0. Does the collapsed loss change? Does the paired loss?

   Hint

   The collapsed loss stays $\log 15$: all non-self logits are equal, so dividing them by any τ leaves the softmax uniform. The paired loss changes, because temperature rescales the *differences* between scores, and with real differences a smaller τ gives a sharper softmax.

2. **Augmentations are assumptions.** You want features for (a) counting cells in microscopy, (b) recognizing bird species, (c) reading street-sign text. For each, name one standard SimCLR augmentation you would remove and why.

   Hint

   (a) Random cropping can change the number of cells in view, so the "same" label is no longer the same. (b) Strong color jitter can remove plumage color, a key species cue. (c) Horizontal flips turn text into mirror text. Decide what must be preserved before choosing invariances.

3. **MAE compute.** An image has 196 patches and MAE masks 75%. How many patches enter the encoder? By what factor does the number of pairwise attention scores in the encoder shrink?

   Hint

   49 visible patches enter the encoder. Attention scores scale with the square of the token count: $(49/196)^2 = 1/16$. The decoder still processes all 196 positions, but it is small.

4. **Break the digits experiment.** Modify the SimCLR digits code so that `augment` only adds noise (no shift). Retrain and compare the shifted-test k-NN accuracy with the original. What does this tell you?

   Hint

   Replace the random `i, j` with `i, j = 1, 1`. On our machine the shifted-test accuracy fell from 0.919 to 0.743, while clean accuracy stayed at 0.981. Without the shift augmentation, nothing in the objective asks for shift invariance; the robustness that remains comes from what a trained convolution-and-pooling encoder learns anyway (the untrained one in the main example had none). The augmentations, not the word "self-supervised", decide most of what is learned.

5. **SimSiam ablation, on paper.** In SimSiam, suppose you remove the stop-gradient so both branches receive gradients. Explain in two sentences why a constant output becomes an easy minimizer, and why the predictor alone is not enough to prevent it.

Hint

With gradients on both sides, the loss $-\cos(p_1, z_2)$ is minimized whenever p_1 and z_2 point the same way, which a constant encoder plus a predictor that outputs that same constant direction achieves for every input. The stop-gradient changes the optimization dynamics so that the target is held fixed during each step; the SimSiam paper reports rapid collapse without it.

6. **Fair probe.** Design an experiment comparing a frozen pretrained encoder with PCA plus logistic regression at 10, 50 and 100 labels per class. List three things you must hold fixed and two things you must report.

Hint

Hold fixed: the labeled subsets (same examples for both), the held-out test split, and the procedure for choosing regularization (on validation data only). Report: the pretrained encoder's training data (and any overlap with your test set) and the number of random label draws with their spread.

References

1. Y. Bengio, A. Courville and P. Vincent, "Representation Learning: A Review and New Perspectives," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 8, 2013. Paper ·arXiv
2. J. Yosinski, J. Clune, Y. Bengio and H. Lipson, "How transferable are features in deep neural networks?" NeurIPS, 2014. Paper
3. X. Liu et al., "Self-supervised Learning: Generative or Contrastive," *IEEE Transactions on Knowledge and Data Engineering*, vol. 35, 2023 (online 2021). Paper ·arXiv
4. J. Gui et al., "A Survey on Self-Supervised Learning: Algorithms, Applications, and Future Trends," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024. Paper ·arXiv
5. R. Balestriero et al., "A Cookbook of Self-Supervised Learning," arXiv:2304.12210, 2023. Paper
6. A. van den Oord, Y. Li and O. Vinyals, "Representation Learning with Contrastive Predictive Coding," arXiv:1807.03748, 2018. Paper
7. T. Chen, S. Kornblith, M. Norouzi and G. Hinton, "A Simple Framework for Contrastive Learning of Visual Representations," ICML, 2020. Paper
8. K. He, H. Fan, Y. Wu, S. Xie and R. Girshick, "Momentum Contrast for Unsupervised Visual Representation Learning," CVPR, 2020. Paper
9. T. Wang and P. Isola, "Understanding Contrastive Representation Learning through Alignment and Uniformity on the Hypersphere," ICML, 2020. Paper
10. J.-B. Grill et al., "Bootstrap Your Own Latent: A New Approach to Self-Supervised Learning," NeurIPS, 2020. Paper
11. X. Chen and K. He, "Exploring Simple Siamese Representation Learning," CVPR, 2021. Paper
12. M. Caron et al., "Emerging Properties in Self-Supervised Vision Transformers," ICCV, 2021. Paper
13. J. Zbontar, L. Jing, I. Misra, Y. LeCun and S. Deny, "Barlow Twins: Self-Supervised Learning via Redundancy Reduction," ICML, 2021. Paper

14. A. Bardes, J. Ponce and Y. LeCun, “VICReg: Variance-Invariance-Covariance Regularization for Self-Supervised Learning,” ICLR, 2022. Paper
15. H. Bao, L. Dong, S. Piao and F. Wei, “BEiT: BERT Pre-Training of Image Transformers,” ICLR, 2022. Paper
16. K. He et al., “Masked Autoencoders Are Scalable Vision Learners,” CVPR, 2022. Paper
17. M. Assran et al., “Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture,” CVPR, 2023. Paper
18. M. Assran et al., “V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning,” technical report, 2025. Report
19. A. Radford et al., “Learning Transferable Visual Models From Natural Language Supervision,” ICML, 2021. Paper
20. X. Zhai, B. Mustafa, A. Kolesnikov and L. Beyer, “Sigmoid Loss for Language Image Pre-Training,” ICCV, 2023. Paper
21. M. Oquab et al., “DINOv2: Learning Robust Visual Features without Supervision,” *Transactions on Machine Learning Research*, 2024. Paper
22. O. Siméoni et al., “DINOv3,” technical report, 2025. Report
23. A. Kumar, A. Raghunathan, R. Jones, T. Ma and P. Liang, “Fine-Tuning can Distort Pretrained Features and Underperform Out-of-Distribution,” ICLR, 2022. Paper
24. S. J. D. Prince, *Understanding Deep Learning*, MIT Press, 2023. Book and notebooks

ML 05A • Linear Factor Models and Autoencoders

PCA, probabilistic PCA, factor analysis, ICA, sparse coding, NMF and autoencoders (undercomplete, sparse, denoising, contractive): how compression, noise assumptions and the manifold view lead to VAEs.

Online edition: https://blog.shyandram.dev/en/learn/ml/ml-05a-factor-models-autoencoders/

What you'll learn

- How a small number of hidden **factors** can explain many correlated measurements, and why that idea underlies both classical statistics and modern deep generative models.
- How PCA, probabilistic PCA and factor analysis differ, and what each one assumes about noise.
- How independent component analysis (ICA), sparse coding and nonnegative matrix factorization (NMF) pick *different* factors by adding different assumptions.
- How linear and nonlinear autoencoders relate to these models, and how sparse, denoising and contractive penalties change what an autoencoder learns.
- What the **manifold view** says about high-dimensional data, and why it leads naturally from autoencoders to variational autoencoders (VAEs).

The big picture

An 8×8 digit image has 64 pixel values, and a phone photo has millions. Yet the meaningful variation in a collection of such images depends on far fewer underlying quantities: which digit it is, the stroke thickness, the slant, the position, the lighting. A **factor model** tries to explain each observation through a handful of hidden (latent) coordinates. An **autoencoder** is a neural network that learns to squeeze an input into such coordinates and expand it back again.

> **Explain like I'm five**
>
> Imagine describing a friend's face over the phone. You don't read out every freckle; you say "round face, short hair, big smile, light from the left." Those few words are factors: they let the listener rebuild a good picture. A factor model looks for the words that explain the most about many faces at once. An autoencoder is a pair of helpers: one who turns a picture into a few words, and one who turns the words back into a picture. They practise together until the rebuilt pictures look right.

Why spend a whole chapter on this? Because almost every idea in modern representation learning and generative modeling has an ancestor here. Dimensionality reduction for visualization, denoising, compression, anomaly detection, the "tokenizers" that turn images into compact codes for diffusion models, and the probabilistic machinery of VAEs all start from the same question: *what small set of causes could have produced these observations?* The progression from linear factor

models to autoencoders follows Goodfellow, Bengio and Courville [1, ch. 13–14] and Murphy [2]; Bengio, Courville and Vincent's review [3] gives the broader motivation for why learned representations matter.

One template, many models

Every method in this chapter fits the same template. There is an **encoder** that maps an observation $x \in \mathbb{R}^d$ to a code $z \in \mathbb{R}^k$, and a **decoder** that maps a code back to a reconstruction $\hat{x}$. The models differ in three choices:

Choice	Options in this chapter
Shape of encoder/decoder	linear (PCA, factor analysis, ICA, NMF, sparse coding) or nonlinear (deep autoencoders)
Size of the code	**undercomplete** ($k < d$: the bottleneck forces compression) or **overcomplete** ($k \geq d$: something else must stop the model from copying)
Extra assumption or penalty	Gaussian noise (PPCA, FA), independence and non-Gaussianity (ICA), sparsity (sparse coding, sparse autoencoders), nonnegativity (NMF), robustness to corruption (denoising), insensitivity to small input changes (contractive)

Keep this table in mind. When two methods disagree, it is almost always because they made a different choice in one of these rows, not because one of them is "wrong."

PCA: the best linear reconstruction subspace

Plain version. Find a few orthogonal directions such that, if you keep only the coordinates of each example along those directions, you lose as little as possible.

Let $X \in \mathbb{R}^{n \times d}$ hold n centered examples as rows (the column means have been subtracted). PCA chooses a matrix $W \in \mathbb{R}^{d \times k}$ with orthonormal columns that minimizes the squared reconstruction error

$$\min_{W} \ \|X - XWW^\top\|_F^2 \qquad \text{subject to} \qquad W^\top W = I_k.$$

Here $\|\cdot\|_F$ is the Frobenius norm (the square root of the sum of squared entries), $Z = XW \in \mathbb{R}^{n \times k}$ are the **codes** (also called scores), and $\hat{X} = ZW^\top$ is the reconstruction. The encoder is $x \mapsto W^\top x$ and the decoder is $z \mapsto Wz$: PCA is already an autoencoder, just a linear one with tied weights.

Where the solution comes from

Two equivalent views lead to the same answer.

Maximum variance. Take a single unit vector w. The variance of the projected data Xw is $w^\top Sw$, where $S = \frac{1}{n-1}X^\top X$ is the sample covariance. Maximize $w^\top Sw$ subject to $w^\top w = 1$ with a Lagrange multiplier λ:

$$\nabla_w (w^\top Sw - \lambda(w^\top w - 1)) = 0 \quad \Longrightarrow \quad Sw = \lambda w.$$

So w must be an eigenvector of S, and the variance it captures is its eigenvalue λ. The best direction is the eigenvector with the largest eigenvalue; the next best direction, constrained to be orthogonal to the first, is the second eigenvector, and so on.

Minimum reconstruction error. Because W has orthonormal columns, the error splits into "total variance minus kept variance." Minimizing the error is therefore the same as maximizing the kept variance, and the residual equals the sum of the discarded eigenvalues (times $n-1$). The Eckart–Young theorem states the same result in matrix language: truncating the singular value decomposition $X = U\Sigma V^\top$ to its k largest singular values gives the best rank-k approximation of X in Frobenius norm. The PCA directions are the first k right singular vectors (columns of V), and the covariance eigenvalues are $\sigma_j^2/(n-1)$.

Code: PCA from the SVD

The snippet fits PCA on the training split of scikit-learn's bundled digits, evaluates reconstruction on held-out images, and checks the result against scikit-learn's own implementation.

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split

X = load_digits().data / 16.
train, test = train_test_split(X, test_size=.2, random_state=0)

mean = train.mean(axis=0)
U, s, Vt = np.linalg.svd(train - mean, full_matrices=False)
var = s**2 / (len(train) - 1)              # eigenvalues of the covariance
ratio = np.cumsum(var) / var.sum()
for k in (2, 10, 20):
    W = Vt[:k].T                           # d x k, orthonormal columns
    rec = (test - mean) @ W @ W.T + mean
    print(f"k={k:2d}  explained={ratio[k-1]:.3f}  "
          f"test MSE={np.mean((test - rec)**2):.4f}")

sk = PCA(n_components=10).fit(train)
print("matches sklearn:", np.allclose(np.abs(sk.components_), np.abs(Vt[:10])))
```

Output:

```
k= 2  explained=0.291  test MSE=0.0546
k=10  explained=0.740  test MSE=0.0202
```

```
k=20  explained=0.896  test MSE=0.0084
matches sklearn: True
```

The `np.abs` in the last line is not a hack: each principal direction is only defined up to sign, because w and $-w$ span the same line. Note that the mean is estimated on the training split only and reused for the test split; estimating it on test data would leak information.

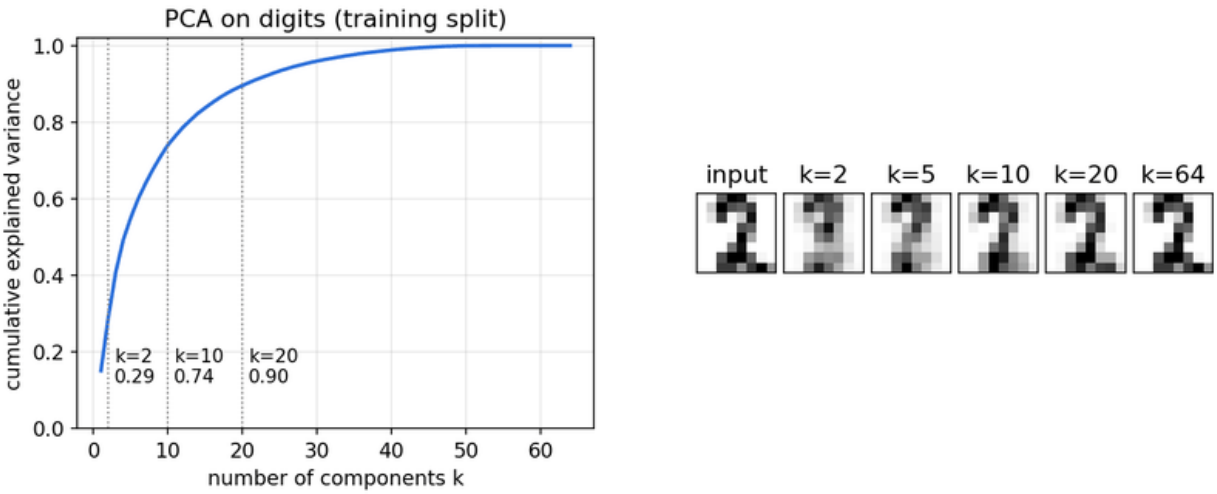


Figure 1 — PCA on the bundled digits. Left: cumulative explained variance (fit on the training split). Right: one held-out digit reconstructed from k components; k = 64 reproduces the input exactly.

> **Interactive demonstration (online)**: open the online edition of this tutorial to use this demonstration.Open demonstration

What PCA does not promise

Three cautions follow directly from the objective.

- **Centering and scaling change the answer.** Without centering, the first "component" mostly points at the mean. Standardizing every feature to unit variance gives each pixel equal weight, so raw-pixel PCA and standardized PCA solve different problems.
- **Variance is not relevance.** The direction of largest variance can be lighting or background rather than the class you care about. PCA is unsupervised; it maximizes retained variance, not class separation (that is the job of methods such as linear discriminant analysis).
- **PCA is linear.** If the data lie on a curved surface, a flat subspace may need many dimensions to describe what is really a low-dimensional shape. We return to this in the manifold section.

Probabilistic PCA and factor analysis

Plain version. Instead of just finding a subspace, write down a story for how the data were generated: draw a few hidden factors, mix them linearly, then add noise. Then fit that story by maximum likelihood.

Both models share the generative equation

$$x = \mu + Wz + \epsilon, \qquad z \sim \mathcal{N}(0, I_k),$$

where $\mu \in \mathbb{R}^d$ is the mean, $W \in \mathbb{R}^{d \times k}$ is the **loading matrix** (how each factor shows up in each measured coordinate), z is the vector of latent factors, and ϵ is noise independent of z. They differ only in the noise.

Probabilistic PCA (PPCA)

In PPCA [4], the noise is isotropic, $\epsilon \sim \mathcal{N}(0, \sigma^2 I_d)$: every coordinate gets the same noise variance. Integrating out z gives a Gaussian marginal

$$x \sim \mathcal{N}(\mu,\, C), \qquad C = WW^\top + \sigma^2 I_d.$$

The covariance has a clear interpretation: a low-rank part $WW^\top$ that the factors explain, plus a constant "floor" σ^2 that they do not. Tipping and Bishop showed that the maximum-likelihood solution is available in closed form [4]:

$$W_{\mathrm{ML}} = U_k(\Lambda_k - \sigma^2 I)^{1/2} R, \qquad \sigma^2_{\mathrm{ML}} = \frac{1}{d-k} \sum_{j=k+1}^{d} \lambda_j,$$

where U_k holds the top k eigenvectors of the sample covariance, Λ_k their eigenvalues $\lambda_1, \dots, \lambda_k$, and R is any $k \times k$ orthogonal matrix. In words: the loadings span exactly the PCA subspace, and the noise variance is the average of the variance PCA throws away. PPCA therefore *is* PCA, plus a likelihood. That likelihood is useful: you can compare models with different k on held-out data, handle missing values with expectation–maximization (EM), and build mixtures of PPCA models.

Given an observation, the posterior over its factors is Gaussian with mean

$$\mathbb{E}[z \mid x] = M^{-1} W^\top (x - \mu), \qquad M = W^\top W + \sigma^2 I_k.$$

This is the probabilistic encoder. As $\sigma^2 \to 0$ it approaches the ordinary PCA projection; for $\sigma^2 > 0$ it shrinks codes toward zero, because part of what we see is attributed to noise.

Factor analysis (FA)

Factor analysis keeps the same structure but allows a diagonal noise covariance, $\epsilon \sim \mathcal{N}(0, \Psi)$ with $\Psi = \mathrm{diag}(\psi_1, \dots, \psi_d)$. Each observed coordinate has its own residual noise level ψ_i, while

the latent factors explain the **shared** covariance between coordinates. The marginal is $x \sim \mathcal{N}(\mu, WW^\top + \Psi)$, and the posterior mean of the factors is $\mathbb{E}[z \mid x] = W^\top (WW^\top + \Psi)^{-1}(x - \mu)$.

There is no closed-form solution, so FA is fitted with EM or direct likelihood optimization. The extra flexibility matters when measurements come from sensors with unequal noise, or when some pixels are almost constant (such as the always-blank border pixels of the digits). PCA happily spends components on any high-variance coordinate; FA can instead explain an individually noisy coordinate as "noise," and reserve its factors for correlations.

The rotation problem

Both models share an ambiguity. For any orthogonal R, replacing W by WR and z by $R^\top z$ leaves $WW^\top$, and hence the distribution of x, unchanged, because $R^\top z$ is still standard normal. The *subspace* is identified; the individual axes are not. That is why the factor-analysis literature uses rotations such as varimax to pick more interpretable axes, and why a factor is not automatically a physical cause. To pin down individual factors we must assume something beyond Gaussianity, which is precisely what ICA does.

Independent component analysis (ICA)

Plain version. Several people talk at once in a room, and several microphones record different mixtures of their voices. ICA tries to recover each voice by finding mixtures that are as statistically independent, and as non-Gaussian, as possible.

The noise-free ICA model is

$$x = As, \qquad s = (s_1, \dots, s_k) \text{ with independent components,}$$

where $x \in \mathbb{R}^d$ is the observed mixture, $A \in \mathbb{R}^{d \times k}$ is an unknown **mixing matrix**, and the sources s_j are independent and **non-Gaussian**. The goal is an unmixing matrix $B \approx A^{-1}$ (or a pseudo-inverse) so that $\hat{s} = Bx$ recovers the sources [5].

Why non-Gaussianity is the key

The rotation problem from the previous section explains why ICA needs non-Gaussian sources. First **whiten** the data: transform x so that its covariance is the identity (PCA followed by rescaling each component to unit variance does this). After whitening, the remaining unknown is just a rotation. If the sources were Gaussian, every rotation of independent unit Gaussians would again be independent unit Gaussians, so no rotation could be preferred. ICA's identifiability theorem states the converse: if **at most one** source is Gaussian, the sources can be recovered up to three harmless ambiguities [5]:

- **order** (we cannot know which source is "first"),
- **sign**, and
- **scale** (a louder source with a quieter mixing column looks the same).

The central limit theorem gives the intuition for how to find the right rotation: a sum of independent variables tends to be *more* Gaussian than its parts. So the projections of x that are *least* Gaussian are the ones most likely to be single sources rather than mixtures. ICA algorithms therefore maximize a measure of non-Gaussianity. A principled one is **negentropy**,

$$J(y) = H(y_{\text{gauss}}) - H(y) \;\geq 0,$$

where H is differential entropy and y_{gauss} is a Gaussian with the same variance as y. Among all distributions with a given variance, the Gaussian has the largest entropy, so $J(y) = 0$ exactly for Gaussian y. Because entropy is hard to estimate, the **FastICA** algorithm approximates negentropy with contrast functions such as $G(u) = \log\cosh u$, and finds each unmixing direction with a fast fixed-point iteration [5]. Maximizing non-Gaussianity is closely related to minimizing the mutual information between the estimated components and to maximum-likelihood estimation of the ICA model.

Code: unmix three signals

The toy problem uses a sine wave, a square wave and a sawtooth, all non-Gaussian, mixed by a fixed matrix.

```
import numpy as np
from sklearn.decomposition import FastICA, PCA

rng = np.random.default_rng(0)
t = np.linspace(0, 8, 2000)
S = np.c_[np.sin(2*t),                # smooth sine
          np.sign(np.sin(3*t)),       # square wave
          2*(t % 1) - 1]              # sawtooth
S += 0.05 * rng.normal(size=S.shape)
A = np.array([[1.0, 1.0, 1.0],
              [0.5, 2.0, 1.0],
              [1.5, 1.0, 2.0]])       # unknown mixing matrix
X = S @ A.T                           # three "microphones"

def best_match(est):
    C = np.abs(np.corrcoef(S.T, est.T)[:3, 3:])
    return np.round(C.max(axis=1), 3)

ica = FastICA(n_components=3, whiten="unit-variance", random_state=0)
print("ICA |corr| with each source:", best_match(ica.fit_transform(X)))
print("PCA |corr| with each source:", best_match(PCA(3).fit_transform(X)))
```

Output:

```
ICA |corr| with each source: [0.997 0.999 0.997]
PCA |corr| with each source: [0.689 0.811 0.735]
```

We compare with absolute correlation and take the best match because ICA returns the sources in arbitrary order, sign and scale. PCA only decorrelates; it finds orthogonal directions of high variance, which here are still mixtures.

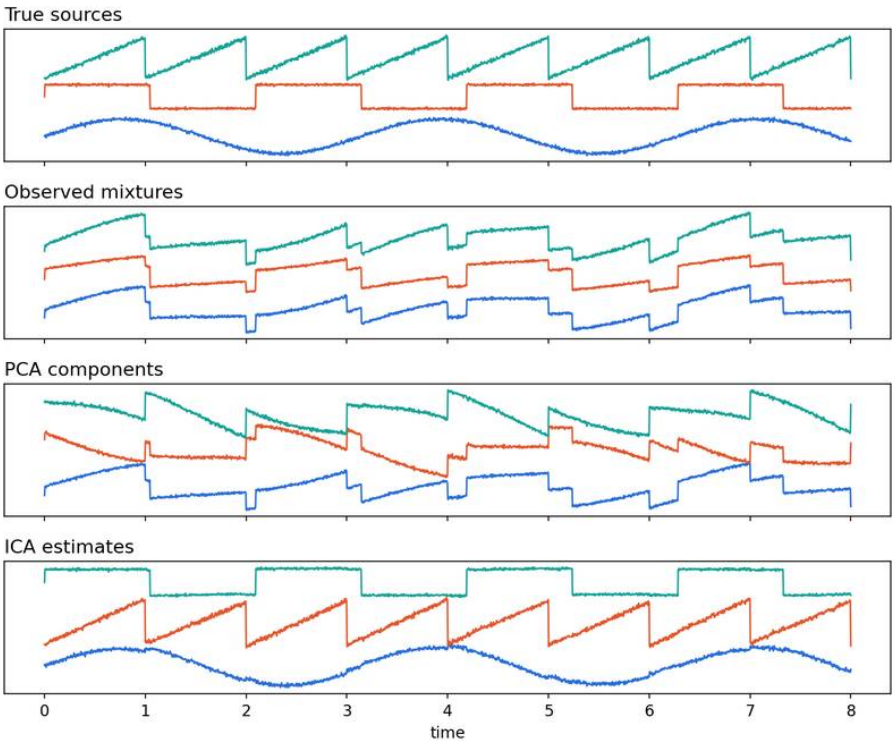


Figure 2 — Blind source separation. PCA components remain mixtures; FastICA recovers the sine, square and sawtooth waves up to order, sign and scale.

ICA is widely used for separating artifacts in EEG and other sensor recordings. Its assumptions also mark its limits: it needs genuinely independent, non-Gaussian sources and a mixing process that is close to linear and instantaneous. On image patches, ICA tends to learn localized, oriented edge-like filters, which brings us to a closely related idea.

Sparse coding

Plain version. Keep a large dictionary of small patterns (“atoms”) and describe each input using only a few of them at a time.

Sparse coding learns a dictionary $D = [d_1, \dots, d_m] \in \mathbb{R}^{d \times m}$ and, for each example x_i, a sparse code $\alpha_i \in \mathbb{R}^m$ by solving

$$\min_{D,\,\{\alpha_i\}} \sum_{i=1}^{n} \left(\tfrac{1}{2} \|x_i - D\alpha_i\|_2^2 + \lambda \|\alpha_i\|_1 \right) \qquad \text{subject to} \qquad \|d_j\|_2 \le 1.$$

The first term asks for accurate reconstruction; the ℓ_1 norm $\|\alpha\|_1 = \sum_j |\alpha_j|$ pushes most code entries

to exactly zero; $\lambda > 0$ sets the trade-off. The norm constraint on atoms stops the model from cheating by making atoms huge and codes tiny. The dictionary is often **overcomplete** ($m > d$): there are more atoms than pixel dimensions, but only a few are active for any one input.

Olshausen and Field [6] trained this kind of model on patches of natural images and found that the learned atoms were localized, oriented and band-pass, much like the receptive fields of simple cells in the primary visual cortex. The result was influential because it suggested that sparsity alone, without any labels, can explain a known property of biological vision.

Two practical points distinguish sparse coding from the autoencoders later in this chapter:

- **Inference is an optimization.** Given D, finding α for a new input means solving a lasso problem. There is no fixed encoder network; the "encoder" is an algorithm. Training alternates between updating the codes (with D fixed) and updating the dictionary (with codes fixed).
- **Sparsity is a prior.** The ℓ_1 penalty corresponds to a Laplace prior on the codes, so sparse coding can be read as maximum a posteriori inference in a linear factor model with a sparse, non-Gaussian prior, which is the same family that ICA belongs to.

Code: sparsity versus fidelity

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.decomposition import DictionaryLearning
from sklearn.model_selection import train_test_split

X = load_digits().data / 16.
train, test = train_test_split(X, test_size=.2, random_state=0)
for alpha in (0.1, 1.0):
    dl = DictionaryLearning(n_components=32, alpha=alpha, max_iter=30,
                            transform_algorithm="lasso_lars",
                            transform_alpha=alpha, random_state=0)
    dl.fit(train[:600])
    codes = dl.transform(test)
    rec = codes @ dl.components_
    active = np.mean(np.abs(codes) > 1e-8, axis=1).mean()
    print(f"alpha={alpha}: active atoms per image={active*32:.1f}/32, "
          f"test MSE={np.mean((test - rec)**2):.4f}")
```

Output (about half a minute on a laptop CPU):

```
alpha=0.1: active atoms per image=17.9/32, test MSE=0.0075
alpha=1.0: active atoms per image=3.0/32, test MSE=0.0401
```

Raising λ (here `alpha`) buys a much sparser description, about three active atoms per image instead of eighteen, at the price of higher reconstruction error. Neither setting is "correct": a compression codec, a denoiser and an interpretability tool would each choose differently. Note

that this dictionary does not subtract a mean, so its error is not directly comparable to the PCA numbers above.

Nonnegative matrix factorization (NMF)

Plain version. Describe each example as an *additive* mixture of nonnegative parts: you may add strokes, but you may never subtract one.

For data $X \geq 0$ (pixel intensities, word counts, spectra), NMF finds $Z \geq 0$ and $H \geq 0$ that minimize a reconstruction criterion such as

$$\min_{Z \geq 0,\, H \geq 0} \|X - ZH\|_F^2,$$

where the rows of $H \in \mathbb{R}^{k \times d}$ are nonnegative parts and the rows of $Z \in \mathbb{R}^{n \times k}$ are nonnegative mixing weights. Because nothing can cancel, the parts tend to look like pieces of the object rather than the positive-and-negative "ghost" patterns of PCA. Lee and Seung [7] demonstrated this parts-based behavior on faces and text and popularized simple multiplicative update rules for fitting it.

Two cautions. First, parts-based results are a tendency, not a guarantee: on some datasets the factors are not clean parts. Second, the objective is nonconvex in Z and H jointly, so initialization matters and different runs can give different factors. Scikit-learn's `init="nndsvda"` uses an SVD-based start to make results more stable.

Code: compare factor reconstructions

This CPU example fits each model on training digits only and evaluates reconstruction on held-out images. It compares different model assumptions; it is not a claim that one method wins every task.

```
import numpy as np
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.decomposition import PCA, FactorAnalysis, NMF

X = load_digits().data / 16.
train, test = train_test_split(X, test_size=.2, random_state=0)
models = {
    "PCA": PCA(n_components=12),
    "factor analysis": FactorAnalysis(n_components=12, random_state=0),
    "NMF": NMF(n_components=12, init="nndsvda", max_iter=1000,
               random_state=0),
}
for name, model in models.items():
    model.fit(train)
    z = model.transform(test)
    if name == "factor analysis":
```

```
        reconstructed = z @ model.components_ + model.mean_
    else:
        reconstructed = model.inverse_transform(z)
    mse = np.mean((test-reconstructed)**2)
    print(name, "held-out reconstruction MSE:", round(mse, 5))
```

Output:

```
PCA held-out reconstruction MSE: 0.01652
factor analysis held-out reconstruction MSE: 0.02501
NMF held-out reconstruction MSE: 0.02207
```

PCA wins on squared reconstruction error, as it must: among all rank-12 linear reconstructions, the PCA subspace is optimal for this criterion (up to the train/test split). That does not make it the best *model*. The factor-analysis reconstruction above is a posterior-mean reconstruction that deliberately attributes part of each pixel to noise, and FA's real output, a full probability model with per-pixel noise, is not summarized by MSE at all. NMF trades some error for nonnegative, additive parts.

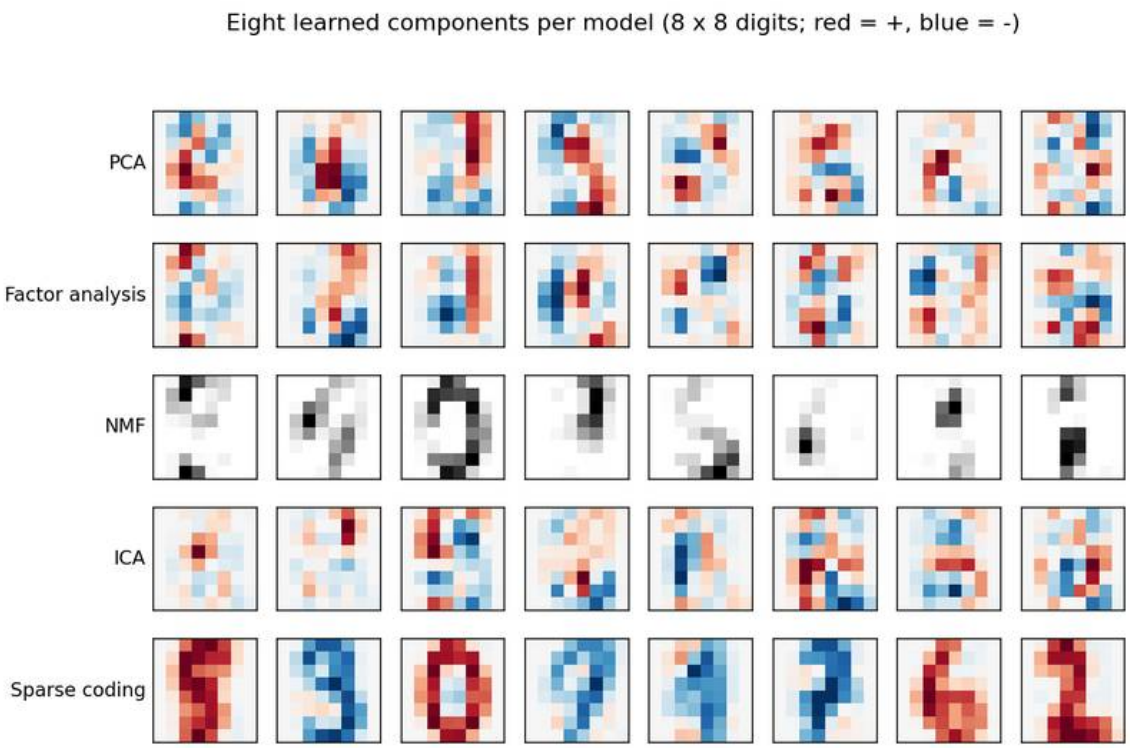


Figure 3 — Eight components from each linear factor model on the same digits. PCA and FA give signed, global patterns; NMF gives nonnegative strokes; ICA gives signed but more localized patterns; sparse-coding atoms resemble prototypes that are combined a few at a time.

Linear autoencoders and PCA

Plain version. If you train a neural network with no nonlinearities to compress and reconstruct its input, it ends up doing PCA, though possibly in rotated or stretched coordinates.

A linear autoencoder with centered input computes

$$z = W_e x, \qquad \hat{x} = W_d z, \qquad \mathcal{L}(W_e, W_d) = \frac{1}{n} \sum_i \|x_i - W_d W_e x_i\|^2,$$

with encoder $W_e \in \mathbb{R}^{k \times d}$ and decoder $W_d \in \mathbb{R}^{d \times k}$, $k < d$. The product $W_d W_e$ has rank at most k, so by Eckart–Young the best possible product is the projection onto the top-k principal subspace. Baldi and Hornik [8] proved more: for this squared-error linear network, every local minimum is a global one, and at the global minimum the network projects onto the principal subspace. The individual weight matrices, however, are not unique. For any invertible $k \times k$ matrix G, replacing W_e by GW_e and W_d by $W_d G^{-1}$ gives the same product, so the code coordinates can be any invertible linear transform of the PCA scores.

Code: a linear autoencoder rediscovers the PCA subspace

```
import numpy as np, torch
from sklearn.datasets import load_digits
from sklearn.decomposition import PCA
from scipy.linalg import subspace_angles

torch.manual_seed(0)
X = load_digits().data / 16.
X = X - X.mean(axis=0)
Xt = torch.tensor(X, dtype=torch.float32)
k = 5
enc = torch.nn.Linear(64, k, bias=False)
dec = torch.nn.Linear(k, 64, bias=False)
opt = torch.optim.Adam([*enc.parameters(), *dec.parameters()], lr=1e-2)
for step in range(3000):
    loss = (dec(enc(Xt)) - Xt).square().mean()
    opt.zero_grad(); loss.backward(); opt.step()

Wd = dec.weight.detach().numpy()            # 64 x k decoder columns
P = PCA(n_components=k).fit(X).components_.T
angles = np.degrees(subspace_angles(Wd, P))
print("largest angle to PCA subspace (deg):", round(angles.max(), 3))
print("decoder columns orthonormal?",
      np.allclose(Wd.T @ Wd, np.eye(k), atol=1e-2))
```

Output:

```
largest angle to PCA subspace (deg): 0.003
decoder columns orthonormal? False
```

The subspace angle (the largest angle between the two k-dimensional subspaces) is essentially zero: the network found the PCA subspace. Yet its decoder columns are not orthonormal, so they are not the principal components themselves. This is the coordinate ambiguity in action, and it is the first hint of a theme that recurs through the chapter: reconstruction alone pins down *what is kept*, not *how it is labeled*.

Nonlinear autoencoders

Plain version. Replace the linear maps with neural networks, so the code can describe curved structure that no flat subspace can capture.

A general autoencoder has an encoder f_ϕ and a decoder g_θ, both neural networks:

$$z = f_\phi(x), \qquad \hat{x} = g_\theta(z), \qquad \min_{\phi,\theta} \frac{1}{n} \sum_i \ell(x_i,\, g_\theta(f_\phi(x_i))),$$

where ℓ is a reconstruction loss, such as squared error for real values or binary cross-entropy for values in $[0, 1]$. The choice of ℓ is itself an assumption: squared error corresponds to a Gaussian likelihood around $\hat{x}$, cross-entropy to a Bernoulli likelihood.

Hinton and Salakhutdinov [9] showed in 2006 that deep autoencoders, initialized with greedy layer-wise pretraining and then fine-tuned end to end, could find low-dimensional codes that reconstructed images much better than PCA with the same number of dimensions. Today, better initializations, normalization and optimizers make the pretraining step unnecessary, but the core lesson stands: nonlinear encoders can compress much more aggressively than linear ones when the data are curved.

Undercomplete is not enough, overcomplete needs a rule

An **undercomplete** autoencoder ($k < d$) is forced to compress. That is useful, but not a full guarantee of good features. A sufficiently powerful encoder and decoder could, in principle, map each training example to its own arbitrary code (say, its index) and memorize the decoding, achieving low training error without learning any reusable structure [1, ch. 14]. An **overcomplete** autoencoder ($k \geq d$) can simply learn the identity function. In both cases, what makes the representation useful is not the bottleneck by itself but the *inductive bias*: limited capacity, the architecture (convolutions, for instance), and explicit penalties. The next section covers the three classic penalties, which together are called **regularized autoencoders**.

Regularized autoencoders: sparse, denoising and contractive

Each penalty answers the question "what should the code ignore?" in a different way.

Sparse autoencoders

Plain version. Allow many code units, but require that only a few of them be active for any given input.

The loss adds a sparsity penalty on the code $h = f_\phi(x)$:

$$\mathcal{L} = \ell(x,\, g_\theta(h)) + \lambda\, \Omega(h), \qquad \Omega(h) = \sum_j |h_j|.$$

Alternatives penalize the deviation of each unit's average activation from a small target rate, or keep only the k largest activations (a "top-k" or k-sparse autoencoder). A sparse autoencoder is the amortized cousin of sparse coding: the encoder network predicts the sparse code in one forward pass, instead of solving a lasso problem for every input. Goodfellow et al. interpret the penalty as an approximate prior over codes rather than an ordinary weight regularizer [1, ch. 14]. As the Modern view explains, sparse autoencoders have recently become a key tool for interpreting the internals of large neural networks.

Denoising autoencoders

Plain version. Show the network a damaged input and ask it to rebuild the clean one. To succeed, it must learn what clean data look like.

A **denoising autoencoder (DAE)** draws a corrupted copy $\tilde{x} \sim C(\tilde{x} \mid x)$ and minimizes

$$\mathcal{L} = \mathbb{E}_x\, \mathbb{E}_{\tilde{x}\sim C(\cdot|x)}\big[\, \ell(x,\, g_\theta(f_\phi(\tilde{x})))\, \big],$$

where C is a chosen corruption process: additive Gaussian noise, masking noise (setting a random fraction of inputs to zero), or salt-and-pepper noise [10], [11]. Vincent et al. introduced the idea in 2008 [10] and showed in 2010 that stacking DAE layers gave useful features for classifiers [11]. Because the input differs from the target, copying is no longer a solution, even for an overcomplete code.

The corruption defines what the model learns to be robust to. A DAE trained on Gaussian noise has no reason to fill in missing patches, and one trained with masking noise learns to complete missing parts. Modern masked image modeling, such as the masked autoencoder (MAE) in ML 05, is a direct descendant of masking-noise DAEs.

There is also a deeper geometric reading. For small Gaussian corruption, the vector $g(f(\tilde{x})) - \tilde{x}$ points from a corrupted point back toward regions of high data density; it approximately estimates the **score** $\nabla_x \log p(x)$, the gradient of the log-density [1, ch. 14]. This connection is one of the roots of score-based and diffusion generative models, covered in ML 07.

Code: a denoising autoencoder on digits

This standalone PyTorch example trains a small nonlinear DAE on training digits and evaluates clean reconstruction from noisy held-out input.

```
import torch
from torch import nn
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

torch.manual_seed(0); torch.set_num_threads(1)
train, test = train_test_split(load_digits().data/16., test_size=.2, random_state=0)
X = torch.tensor(train, dtype=torch.float32)
Xt = torch.tensor(test, dtype=torch.float32)
encoder = nn.Sequential(nn.Linear(64, 32), nn.ReLU(), nn.Linear(32, 12))
decoder = nn.Sequential(nn.Linear(12, 32), nn.ReLU(), nn.Linear(32, 64), nn.Sigmoid())
model = nn.Sequential(encoder, decoder)
opt = torch.optim.Adam(model.parameters(), lr=.003)
for step in range(2000):
    clean = X[torch.randint(len(X), (64,))]
    noisy = (clean + .2*torch.randn_like(clean)).clamp(0, 1)
    loss = (model(noisy)-clean).square().mean()
    opt.zero_grad(); loss.backward(); opt.step()
model.eval()
with torch.no_grad():
    noisy = (Xt + .2*torch.randn_like(Xt)).clamp(0, 1)
    print("noisy-input MSE:", (noisy-Xt).square().mean().item())
    print("reconstructed MSE:", (model(noisy)-Xt).square().mean().item())
```

Output (about 0.0248 for the noisy input and 0.0197 after the DAE; the last digits depend on the PyTorch build):

```
noisy-input MSE: 0.024781296029686928
reconstructed MSE: 0.01968955807387829
```

The improvement is real but modest, and Figure 4 shows why: with a 12-dimensional code, the DAE removes the speckle but also smooths away some stroke detail. Denoising and faithful compression pull in different directions; a wider code or a convolutional architecture would shift the balance.

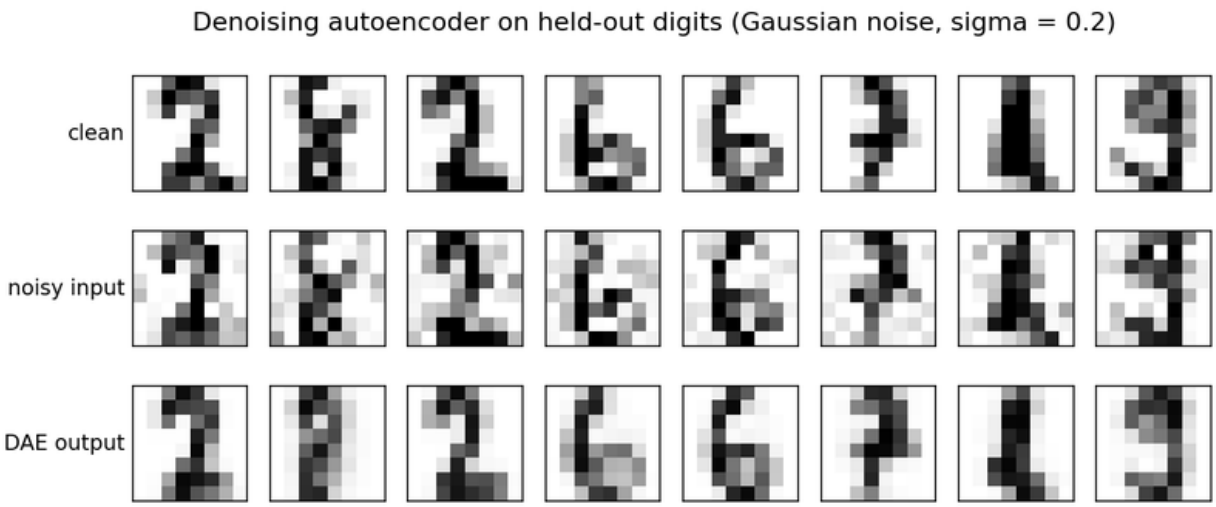


Figure 4 — Denoising autoencoder outputs on held-out digits. The background speckle is removed; some fine stroke detail is lost through the 12-unit bottleneck.

Contractive autoencoders

Plain version. Penalize the encoder whenever a tiny change in the input causes a big change in the code. The code should only react to changes that matter.

Rifai et al. [12] add the squared Frobenius norm of the encoder's Jacobian:

$$\mathcal{L} = \ell(x,\, g_\theta(f_\phi(x))) + \lambda \left\| \frac{\partial f_\phi(x)}{\partial x} \right\|_F^2, \qquad \left\| \frac{\partial f_\phi(x)}{\partial x} \right\|_F^2 = \sum_{j,i} \left(\frac{\partial h_j}{\partial x_i} \right)^2 .$$

Here $h = f_\phi(x)$ and the Jacobian entry $\partial h_j / \partial x_i$ measures how much code unit j moves when input coordinate i moves. For a single sigmoid layer $h = \sigma(Wx + b)$, the entry is $h_j(1 - h_j)W_{ji}$, so the penalty is cheap to compute:

$$\left\| \frac{\partial h}{\partial x} \right\|_F^2 = \sum_j \left(h_j (1 - h_j) \right)^2 \sum_i W_{ji}^2 .$$

The reconstruction term wants the code to change with the input (otherwise it could not reconstruct different inputs), while the penalty wants it to change as little as possible. The compromise is a code that is sensitive only along the few directions in which the training data actually vary. A DAE achieves a related effect implicitly: for small Gaussian noise, denoising training penalizes the sensitivity of the whole reconstruction function [1, ch. 14], whereas the CAE penalizes only the encoder, explicitly.

Code: the contractive penalty in action

```
import torch
from torch import nn
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

train, test = train_test_split(load_digits().data/16., test_size=.2, random_state=0)
X = torch.tensor(train, dtype=torch.float32)
Xt = torch.tensor(test, dtype=torch.float32)

def fit(lam):
    torch.manual_seed(0)
    W = nn.Linear(64, 16)                    # encoder h = sigmoid(Wx + b)
    dec = nn.Linear(16, 64)
    opt = torch.optim.Adam([*W.parameters(), *dec.parameters()], lr=3e-3)
    for step in range(1500):
        x = X[torch.randint(len(X), (64,))]
        h = torch.sigmoid(W(x))
        rec = (dec(h) - x).square().sum(1).mean()
        # ||J||_F^2 = sum_j (h_j(1-h_j))^2 * ||W_j||^2 for a sigmoid layer
        jac = ((h*(1-h))**2 * W.weight.square().sum(1)).sum(1).mean()
        loss = rec + lam*jac
        opt.zero_grad(); loss.backward(); opt.step()
    with torch.no_grad():
        h = torch.sigmoid(W(Xt))
        mse = (dec(h) - Xt).square().mean().item()
        jac = ((h*(1-h))**2 * W.weight.square().sum(1)).sum(1).mean().item()
    return mse, jac

for lam in (0.0, 0.5):
    mse, jac = fit(lam)
    print(f"lambda={lam}: test MSE={mse:.4f}, mean ||J||_F^2={jac:.2f}")
```

Output:

```
lambda=0.0: test MSE=0.0141, mean ||J||_F^2=4.79
lambda=0.5: test MSE=0.0194, mean ||J||_F^2=0.70
```

The penalty cuts the encoder's average sensitivity by a factor of almost seven, at the cost of a somewhat higher reconstruction error. Whether that trade is worthwhile depends on the downstream use: a smoother code often helps nearest-neighbor retrieval or classification with few labels, while a codec would prefer the unpenalized model.

The manifold view

Plain version. Real data do not fill their high-dimensional space. They cluster near a thin, curved surface, like a sheet of paper rolled up inside a room. Good representations describe where you

are *on the sheet*.

The **manifold hypothesis** says that natural data, such as images, concentrate near a low-dimensional manifold embedded in the high-dimensional input space [3]. A random 64-pixel array almost never looks like a digit; the set of plausible digits is a tiny, structured region. Locally, around any data point, the manifold looks like a flat plane, the **tangent space**, spanned by the few directions you can move while staying realistic (thicker stroke, slight slant, small shift). Directions orthogonal to the tangent space lead off the manifold into noise.

This view gives a single lens for everything in the chapter:

- **PCA** fits one global flat sheet. It works when the manifold is nearly flat and fails when it is curved.
- **Mixtures of PPCA or FA** fit many local flat sheets.
- **Nonlinear autoencoders** learn a curved chart of the manifold: the encoder gives coordinates on the sheet, and the decoder maps them back into the room.
- **Denoising** teaches the model to move points that fell off the manifold back onto it.
- **Contractive** penalties make the encoder insensitive to directions orthogonal to the manifold, while reconstruction keeps it sensitive along the tangent directions [1, ch. 14], [12].

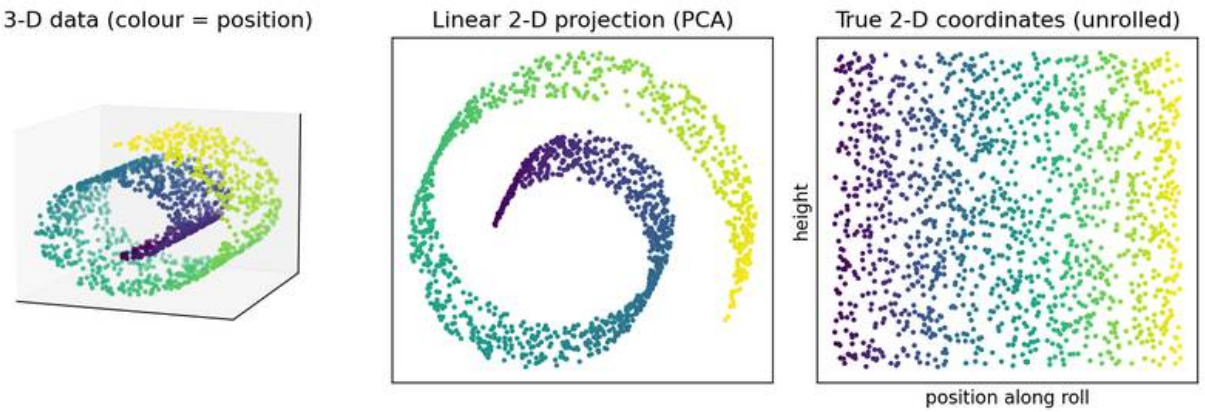


Figure 5 — The Swiss roll is a two-dimensional sheet rolled up in three dimensions. A linear projection (PCA) flattens it into a spiral that does not preserve position along the sheet; the true intrinsic coordinates form a flat rectangle.

Figure 5 makes the limitation of linear methods concrete. The Swiss roll has two intrinsic dimensions, position along the roll and height, but no two-dimensional linear projection can unroll it: the PCA view keeps the spiral, so points far apart along the sheet are not laid out in order. A nonlinear encoder can, in principle, learn the unrolled coordinates. In practice, whether it does depends on capacity, regularization and the amount of data; manifold learning in high dimensions remains hard.

From autoencoders to VAEs

Plain version. An ordinary autoencoder learns to compress and decompress, but it gives no rule for *inventing* new codes. A VAE adds that rule.

Suppose a trained autoencoder maps every training digit to a code in $\mathbb{R}^{12}$. To generate a new digit, we would like to pick a random code and decode it. But which random code? The training codes may occupy a strange, clumpy region, and a random point between them may decode to garbage. Nothing in the reconstruction loss asked the code space to be smooth, filled, or shaped like any known distribution. Excellent reconstruction therefore does not imply good generation.

The **variational autoencoder** [13] fixes this by returning to the probabilistic factor-model story of PPCA and FA, but with a neural decoder:

$$z \sim p(z) = \mathcal{N}(0, I), \qquad x \sim p_\theta(x \mid z),$$

where $p_\theta(x \mid z)$ is a distribution whose parameters are produced by a decoder network. Because the true posterior $p_\theta(z \mid x)$ is now intractable, a second network, the encoder, outputs an approximate posterior $q_\phi(z \mid x)$, typically a diagonal Gaussian. Training maximizes the **evidence lower bound (ELBO)** on $\log p_\theta(x)$:

$$\log p_\theta(x) \;\geq\; \underbrace{\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)]}_{\text{reconstruction}} \;-\; \underbrace{D_{\mathrm{KL}}(q_\phi(z \mid x) \,\|\, p(z))}_{\text{keep codes near the prior}}.$$

Read the two terms against everything above:

- The **reconstruction** term is exactly an autoencoder loss, now with noise injected into the code. With a Gaussian decoder it becomes squared error, with a Bernoulli decoder binary cross-entropy.
- The **KL** term is a regularizer, like the sparse and contractive penalties, but chosen so that codes from all training examples together fill the prior. That is what makes sampling $z \sim \mathcal{N}(0, I)$ meaningful.
- With a **linear** Gaussian decoder, the model *is* PPCA. Its exact posterior is Gaussian and can be matched by a linear Gaussian encoder, so the bound becomes tight and the optimum coincides with the PPCA maximum-likelihood solution. The VAE is the nonlinear generalization of the factor models we began with.

Kingma and Welling's introduction [14] develops this connection at length. Weighting the KL term by a factor $\beta > 1$ gives the **β-VAE** [15], which was proposed to encourage disentangled factors at the cost of reconstruction quality; note that with $\beta \neq 1$ the objective is no longer the ELBO. The full derivation of the ELBO, the reparameterization trick and failure modes such as posterior collapse are developed in ML 06.

What reconstruction error can and cannot tell you

Before leaving autoencoders, a practical warning. Reconstruction error is often used as an anomaly score: an autoencoder trained on normal data should reconstruct normal inputs well

and unusual ones poorly. This can work, but it fails when the autoencoder generalizes too well and reconstructs unusual inputs too, or when anomalies differ only in fine detail that the bottleneck discards anyway. Similarly, a code that reconstructs well may be poor for classification, and a code that classifies well may discard the pixel detail a restoration task needs. Evaluate compression, denoising, downstream usefulness and generation separately.

Modern view

Plain version. The classical models of this chapter did not disappear. They became components: compression stages for generative models, pretraining objectives for vision, and microscopes for looking inside large networks.

Surveys worth reading

- **Bengio, Courville and Vincent (2013)** [3] is the standard review of representation learning. It frames good representations through a list of general priors (smoothness, multiple explanatory factors, a hierarchy of concepts, shared factors across tasks, the manifold hypothesis, sparsity and others), and connects autoencoders, probabilistic models and manifold learning under that lens. It is a good map for *why* the penalties in this chapter exist.
- **Bank, Koenigstein and Giryes (2020)** [16] is a compact survey chapter on autoencoders. It walks through regularized variants (sparse, denoising, contractive), VAEs and disentangled variants, applications (generative modeling, classification, clustering, anomaly detection, recommendation and dimensionality reduction) and hybrids with adversarial training, which makes it a useful companion to this tutorial.
- **Kingma and Welling (2019)** [14] is the authors' own extended introduction to VAEs. It covers the ELBO, the reparameterization trick, richer posteriors and hierarchical models, and places VAEs among other deep latent-variable models.
- **Goodfellow, Bengio and Courville** [1, ch. 13–14] and **Murphy** [2] remain the clearest textbook treatments of linear factor models, the DAE–score connection and the manifold interpretation.

Disentanglement: a sober result

A recurring hope has been that unsupervised factor models would recover *the* true underlying factors, one per code unit ("disentanglement"). Locatello et al. [17] showed theoretically that unsupervised learning of disentangled representations is fundamentally impossible without inductive biases on both models and data, which mirrors the rotation ambiguity of factor analysis at a much larger scale. In a large reproducible study they also found that well-disentangled models could not be identified without supervision, and that more disentanglement did not reliably reduce the sample complexity of downstream tasks. The practical lesson echoes ICA: identifiability always comes from extra assumptions, so state them.

Autoencoders as tokenizers

The biggest modern role of autoencoders is as the first stage of large generative models. **VQ-VAE** [18] replaces the continuous code with a learned codebook of discrete vectors, so images become grids of discrete tokens that an autoregressive model can predict (see ML 08). **Latent diffusion** [19] trains an autoencoder for perceptual compression and then runs the diffusion model in its latent space instead of pixel space, which greatly reduces compute. In both cases, the autoencoder's reconstruction quality sets a ceiling: the generator cannot restore detail the encoder discarded. Designing these tokenizers, balancing compression, reconstruction and how easy the latent space is for the generator to model, is an active research area as of 2026.

Sparse autoencoders for interpretability (2023–2026)

Sparse autoencoders have found a new and very active application: **mechanistic interpretability** of large language and vision models. The motivating observation is that individual neurons are often polysemantic, responding to several unrelated concepts. Cunningham et al. [20] trained overcomplete sparse autoencoders to reconstruct a language model's internal activations, and found sparsely activating features that automated measures rated more interpretable and monosemantic than the neurons or other decompositions. Gao et al. [21] used k-sparse (top-k) autoencoders to control sparsity directly, reported clean scaling laws in autoencoder size and sparsity, proposed new metrics for feature quality, and trained a 16-million-latent autoencoder on GPT-4 activations. Conceptually this is sparse coding (Olshausen and Field's idea) applied to a network's activations instead of image patches.

Open problems

- **Evaluation.** Reconstruction error, sparsity and downstream accuracy disagree, and there is no agreed measure of whether learned features are the "right" ones, either for disentanglement or for interpretability.
- **Identifiability.** When can learned factors be trusted to correspond to real causes? Nonlinear ICA and causal representation learning study which extra assumptions (time structure, auxiliary variables, interventions) make this possible.
- **Tokenizer design.** How much to compress, whether codes should be discrete or continuous, and how to make latents both reconstructive and easy to model remain open engineering and theory questions.

Key takeaways

- All models here share one template: an encoder to a code, a decoder back, and an assumption that decides which code to prefer.
- PCA minimizes linear reconstruction error; its directions are covariance eigenvectors, found with the SVD of centered data.

- PPCA adds isotropic Gaussian noise and recovers the PCA subspace by maximum likelihood; factor analysis allows a separate noise level per coordinate. Both identify the subspace but not the axes.
- ICA resolves the axes by assuming independent, non-Gaussian sources, up to order, sign and scale; sparse coding and NMF pick factors by sparsity and nonnegativity.
- A linear autoencoder recovers the PCA subspace, not unique PCA weights. Nonlinear autoencoders need a bottleneck and/or a penalty (sparse, denoising, contractive) to learn useful structure.
- The manifold view unifies these: good codes move along the data's low-dimensional surface and ignore directions off it.
- Reconstruction does not provide a sampler. The VAE adds a prior and an approximate posterior, and with a linear decoder it reduces to PPCA.

Exercises

1. **Noise assumptions.** You record 20 sensors, five of which are much noisier than the rest, but all respond to the same few underlying causes. Would you start with PPCA or factor analysis? What would PCA tend to do with the noisy sensors?

   Hint

   Factor analysis: its diagonal Ψ can assign large residual variance to the five noisy sensors without spending factors on them. PCA (and PPCA, with a single shared σ^2) may devote components to the high-variance noisy sensors, because high variance is all PCA sees.

2. **Latent ambiguity.** For an invertible $k \times k$ matrix G, show that replacing W_e by GW_e and W_d by $W_d G^{-1}$ leaves a linear autoencoder's reconstruction unchanged. Which matrices G also preserve the PPCA likelihood?

   Hint

   The product is $W_d G^{-1} G W_e = W_d W_e$. For PPCA, the likelihood depends on $WW^\top$; replacing W by WG gives $WGG^\top W^\top$, which is unchanged when $GG^\top = I$, that is, when G is orthogonal.

3. **Why ICA fails on Gaussians.** Generate two independent standard Gaussian sources, mix them with any invertible 2×2 matrix, and run FastICA several times with different random seeds. What do you observe, and why?

   Hint

   After whitening, every rotation of two independent unit Gaussians is again two independent unit Gaussians, so there is no "right" rotation to find. Different seeds give different, equally valid unmixing directions, and the correlation with the true sources varies from run to run.

4. **Sparse codes versus PCA codes.** Using the sparse-coding snippet, compare the number of nonzero code entries per test image with PCA codes of the same dimension (32). Then plot the test MSE of sparse coding as `alpha` varies over 0.05, 0.1, 0.3, 1.0.

   Hint

   PCA codes are dense: essentially all 32 entries are nonzero. As `alpha` grows, the number of active atoms drops and the error rises, tracing a sparsity–fidelity curve. Remember that the dictionary was not given a mean term, so compare curves rather than single numbers with PCA.

5. **Denoising mismatch.** Train the DAE from the tutorial on Gaussian noise, then test it on inputs where a random 3×3 block is set to zero. Then train a second DAE on that masking corruption and test both on both corruptions.

   Hint

   These are different corruption processes. The Gaussian-trained model has never been asked to fill in missing structure, so it typically leaves holes; the masking-trained model learns completion. Report a 2×2 table (train corruption × test corruption) rather than inferring robustness to one from success on the other.

6. **Contractive penalty by hand.** For a single sigmoid unit $h = \sigma(w^\top x + b)$, derive $\|\partial h / \partial x\|^2$. When is it largest, and what does that say about where a contractive autoencoder allows its code to be sensitive?

   Hint

   $\partial h / \partial x = h(1-h)w$, so the squared norm is $h^2(1-h)^2\|w\|^2$. It is largest at $h = 1/2$ (the steep middle of the sigmoid) and nearly zero when the unit is saturated. The penalty pushes units to be saturated for most inputs, keeping them in the sensitive regime only where the data require variation.

References

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016, ch. 13–14. Book
2. K. P. Murphy, *Probabilistic Machine Learning: An Introduction*, MIT Press, 2022. Book
3. Y. Bengio, A. Courville and P. Vincent, "Representation Learning: A Review and New Perspectives," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 35, no. 8, 2013. Paper
4. M. E. Tipping and C. M. Bishop, "Probabilistic Principal Component Analysis," *Journal of the Royal Statistical Society: Series B*, vol. 61, no. 3, 1999. Paper
5. A. Hyvärinen and E. Oja, "Independent component analysis: algorithms and applications," *Neural Networks*, vol. 13, no. 4–5, 2000. Paper
6. B. A. Olshausen and D. J. Field, "Emergence of simple-cell receptive field properties by learning a sparse code for natural images," *Nature*, vol. 381, 1996. Paper

7. D. D. Lee and H. S. Seung, “Learning the parts of objects by non-negative matrix factorization,” *Nature*, vol. 401, 1999. Paper
8. P. Baldi and K. Hornik, “Neural networks and principal component analysis: Learning from examples without local minima,” *Neural Networks*, vol. 2, no. 1, 1989. Paper
9. G. E. Hinton and R. R. Salakhutdinov, “Reducing the Dimensionality of Data with Neural Networks,” *Science*, vol. 313, 2006. Paper
10. P. Vincent, H. Larochelle, Y. Bengio and P.-A. Manzagol, “Extracting and composing robust features with denoising autoencoders,” ICML, 2008. Paper
11. P. Vincent, H. Larochelle, I. Lajoie, Y. Bengio and P.-A. Manzagol, “Stacked Denoising Autoencoders: Learning Useful Representations in a Deep Network with a Local Denoising Criterion,” *JMLR*, vol. 11, 2010. Paper
12. S. Rifai, P. Vincent, X. Muller, X. Glorot and Y. Bengio, “Contractive Auto-Encoders: Explicit Invariance During Feature Extraction,” ICML, 2011. Paper
13. D. P. Kingma and M. Welling, “Auto-Encoding Variational Bayes,” ICLR, 2014. Paper
14. D. P. Kingma and M. Welling, “An Introduction to Variational Autoencoders,” *Foundations and Trends in Machine Learning*, vol. 12, no. 4, 2019. Paper
15. I. Higgins et al., “β-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework,” ICLR, 2017. Paper
16. D. Bank, N. Koenigstein and R. Giryes, “Autoencoders,” survey chapter, arXiv:2003.05991, 2020. Survey
17. F. Locatello et al., “Challenging Common Assumptions in the Unsupervised Learning of Disentangled Representations,” ICML, 2019. Paper
18. A. van den Oord, O. Vinyals and K. Kavukcuoglu, “Neural Discrete Representation Learning,” NeurIPS, 2017. Paper
19. R. Rombach, A. Blattmann, D. Lorenz, P. Esser and B. Ommer, “High-Resolution Image Synthesis with Latent Diffusion Models,” CVPR, 2022. Paper
20. H. Cunningham, A. Ewart, L. Riggs, R. Huben and L. Sharkey, “Sparse Autoencoders Find Highly Interpretable Features in Language Models,” arXiv:2309.08600, 2023. Paper
21. L. Gao et al., “Scaling and evaluating sparse autoencoders,” arXiv:2406.04093, 2024. Paper

ML 06 • Generative Models: Autoencoders, VAEs, GANs and Flows

Learn distributions rather than labels: likelihood-based versus implicit models, the ELBO and reparameterization, adversarial training and Wasserstein GANs, normalizing flows, and how generative models are evaluated.

Online edition: https://blog.shyandram.dev/en/learn/ml/ml-06-generative-models/

What you'll learn

- How modeling a distribution $p(x)$ differs from predicting a label, and why maximum likelihood is the same as minimizing a KL divergence.
- The two big families: **likelihood-based** models that write down (or bound) $\log p(x)$, and **implicit** models that only provide a sampler.
- How a VAE turns an intractable integral into a trainable lower bound (the ELBO), and why the reparameterization trick makes it differentiable.
- How GANs learn through a discriminator, what the original objective secretly minimizes, why mode collapse happens, and what Wasserstein GANs change.
- How normalizing flows compute exact likelihoods with the change-of-variables formula, and where energy-based, autoregressive and diffusion models fit.
- How to evaluate generators with likelihood, Inception Score, FID and precision/recall — and what each of these cannot tell you.

The big picture

A classifier asks, "What is in this image?" A **generative model** asks, "What kinds of images could occur?" If it learns a useful distribution, we can draw new samples, fill missing regions, compress data, detect unusual inputs, or condition generation on text, a class label or another image.

> **Explain like I'm five**
>
> A child who recognizes drawings of houses can say "house." A child who understands how houses are drawn can invent a new one with a door, windows and a roof. Generative modeling tries to learn the recipe behind the examples, so it can cook up new ones — and a good cook makes a different, but still tasty, dish every time instead of the same one over and over.

Learning a distribution is harder than learning a label. A label is one number; an image has thousands of pixels whose values depend on one another in complicated ways. Every family in this chapter is a different answer to the same engineering question: how do you represent a

distribution over very high-dimensional data so that you can both **train** it from examples and **sample** from it afterwards?

This chapter uses the probabilistic viewpoint developed in Bishop and Bishop [1] and Prince [2]. The comparative review by Bond-Taylor et al. [3] organizes the same families along the trade-offs we will meet — training speed, sample quality, diversity, and whether a likelihood is available — and the flow review by Papamakarios et al. [4] is particularly helpful for distinguishing a learned distribution from a convenient sampling procedure. Diffusion and autoregressive models get their own chapters (ML 07 and ML 08) because their training and sampling deserve careful treatment; here we place them on the map.

Modeling a distribution

Plain version. Fit a probability model so that the observed examples look likely under it, then use the model to produce new examples that look like they came from the same source.

Maximum likelihood

Suppose the data $x_1, \dots, x_n$ are independent draws from an unknown distribution p_{data}, and p_θ is a model with parameters θ. Maximum likelihood minimizes the average **negative log-likelihood (NLL)**:

$$\mathcal{L}_{\text{NLL}}(\theta) = -\frac{1}{n}\sum_{i=1}^{n} \log p_\theta(x_i).$$

Here n is the number of training examples and $\log p_\theta(x_i)$ is the log-density the model assigns to example i.

Why is this a sensible goal? As n grows, the average approaches an expectation over the data distribution, and

$$\mathbb{E}_{x\sim p_{\text{data}}}[-\log p_\theta(x)] = D_{\text{KL}}(p_{\text{data}} \,\|\, p_\theta) + H(p_{\text{data}}).$$

$D_{\text{KL}}(p\|q) = \mathbb{E}_p[\log p - \log q]$ is the Kullback–Leibler divergence, which is zero only when the two distributions agree, and $H(p_{\text{data}})$ is the entropy of the data, which does not depend on θ. So **maximum likelihood is the same as minimizing the forward KL divergence** from data to model.

The direction of the KL matters. The forward KL heavily punishes a model that assigns near-zero density where real data lives, so likelihood training is "mass-covering": it prefers to spread probability over every mode, even if that means putting some probability between modes. Keep this in mind; we will see that GANs behave differently.

Densities, discrete pixels and bits per dimension

For continuous values, $p_\theta(x)$ is a **density**, not the probability of observing exactly x; a density can exceed 1. Real images, however, store integers such as 0–255. Fitting a continuous density directly

to integers is ill-posed: the model can place infinitely tall spikes on the integer grid and drive the likelihood to infinity. The standard fix is **dequantization** — add uniform noise in $[0, 1)$ to each integer value before fitting a continuous density. Theis et al. [5] show that the continuous model's log-likelihood on dequantized data is then a lower bound on the log-likelihood of a corresponding discrete model, which makes the numbers comparable across papers. Alternatively one uses a discrete likelihood directly (a softmax over 256 values, or a Bernoulli for binary pixels, as in our code below).

Image likelihoods are usually reported in **bits per dimension**:

$$\text{bpd} = \frac{\mathcal{L}_{\text{NLL}}}{D \ln 2},$$

where $\mathcal{L}_{\text{NLL}}$ is measured in nats per image and D is the number of pixel values (height × width × channels). Bits per dimension are the average number of bits a lossless compressor built on the model would need per subpixel, which is why likelihood and compression are two sides of the same coin.

Conditional and unconditional generation

A discriminative classifier models $p(y \mid x)$. A conditional generator models $p(x \mid c)$, where c can be text, a class, a low-resolution image or observed pixels. Bayes' rule links related quantities, but a classifier does not by itself provide an image sampler.

Conditioning also explains a familiar failure. Minimizing pixel MSE predicts the **conditional mean** $\mathbb{E}[x \mid c]$. If an ambiguous input has several valid outputs — a gray-scale photo whose shirt could be red or blue — the mean blends them into a brownish average. A generative model instead tries to represent the whole distribution $p(x \mid c)$ and sample one plausible answer at a time.

Two families: likelihood-based and implicit

Every model below needs two abilities: a way to **train** from data and a way to **sample**. They differ in whether they can evaluate $p_\theta(x)$.

- **Likelihood-based (explicit) models** write down a density, or a bound on it, and maximize it. Autoregressive models and normalizing flows give exact likelihoods; VAEs (and diffusion models, in their variational view) optimize a lower bound; energy-based models define a density only up to an unknown normalizing constant.
- **Implicit models** define a sampler — "push noise through a network" — without any formula for $p_\theta(x)$. GANs are the main example. Because there is no likelihood to maximize, they must be trained by comparing samples with data in some other way.

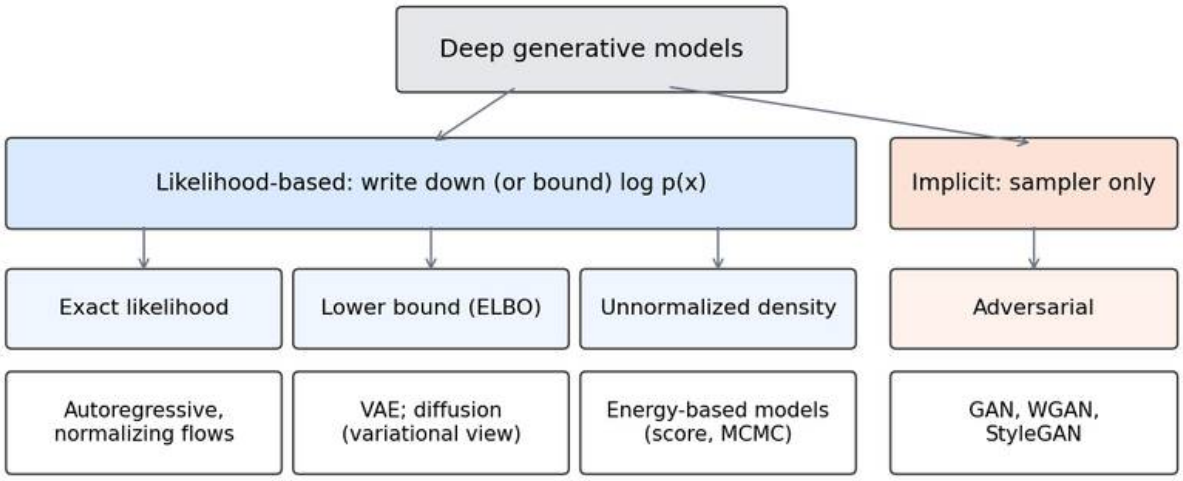


Figure 1 — A teaching taxonomy of generative models, following the explicit/implicit split used in comparative reviews [3]. *Real systems often combine several branches.*

Ordinary autoencoders: compression without a prior

Plain version. Encode an example into a compact code and decode it back. That is excellent for compression and representation learning, but it does not by itself tell you how to invent new codes.

An encoder $z = f_\phi(x)$ and decoder $\hat{x} = g_\theta(z)$ minimize a reconstruction loss such as $\|x - \hat{x}\|^2$. Autoencoders, their linear relatives (PCA, factor analysis) and their regularized variants are developed in ML 05A. The point for this chapter is narrow: an ordinary autoencoder places no requirement on **where** codes live. The training codes may occupy a thin, oddly shaped region of latent space. If you then sample $z \sim \mathcal{N}(0, I)$ and decode it, you may land in an empty region the decoder never saw, and get garbage. Excellent reconstruction therefore does not guarantee good unconditional generation.

A VAE fixes this by turning the autoencoder into a probability model with a prior over codes.

VAEs: a latent-variable probability model

Plain version. Imagine that each image was made by first picking a few hidden "settings" (pose, thickness, style) at random from a simple distribution, then rendering an image from those settings with a bit of noise. A VAE learns both the renderer (decoder) and a quick guesser (encoder) that reads an image and proposes which settings probably produced it.

The model

Choose a prior $p(z)$ over a latent vector $z \in \mathbb{R}^k$, usually a standard Gaussian $\mathcal{N}(0, I)$, and a decoder $p_\theta(x \mid z)$, a neural network that outputs the parameters of a distribution over images (Bernoulli probabilities for binary pixels, or Gaussian means). The model's density over images is the marginal

$$p_\theta(x) = \int p_\theta(x \mid z)\, p(z)\, dz.$$

Sampling is easy: draw $z \sim p(z)$, then draw $x \sim p_\theta(x \mid z)$. Evaluating $p_\theta(x)$ is hard: the integral runs over every possible code, and the decoder is a nonlinear network, so there is no closed form. The true posterior $p_\theta(z \mid x) = p_\theta(x \mid z)p(z)/p_\theta(x)$ is intractable for the same reason.

Deriving the evidence lower bound

Kingma and Welling [6] introduce an **encoder** (or inference network) $q_\phi(z \mid x)$ that approximates the posterior. Here is the derivation step by step. Since $\log p_\theta(x)$ does not depend on z, we can take its expectation under any distribution over z without changing it:

$$\begin{aligned}
\log p_\theta(x) &= \mathbb{E}_{q_\phi(z|x)}\big[\log p_\theta(x)\big] \\
&= \mathbb{E}_{q_\phi(z|x)}\left[\log \frac{p_\theta(x,z)}{p_\theta(z \mid x)}\right] \\
&= \mathbb{E}_{q_\phi(z|x)}\left[\log \frac{p_\theta(x,z)}{q_\phi(z \mid x)}\right] + \mathbb{E}_{q_\phi(z|x)}\left[\log \frac{q_\phi(z \mid x)}{p_\theta(z \mid x)}\right] \\
&= \mathrm{ELBO}(x) + D_{\mathrm{KL}}(q_\phi(z \mid x) \,\|\, p_\theta(z \mid x)).
\end{aligned}$$

The second line uses $p_\theta(x) = p_\theta(x,z)/p_\theta(z \mid x)$; the third multiplies and divides by $q_\phi(z \mid x)$ inside the log. Because a KL divergence is never negative, the first term is a lower bound on $\log p_\theta(x)$: the **evidence lower bound (ELBO)**. The **gap** between the bound and the true log-likelihood is exactly the KL divergence between the approximate and true posteriors. A better encoder tightens the bound; a perfect one closes it.

Expanding $p_\theta(x,z) = p_\theta(x \mid z)p(z)$ gives the form used in code:

$$\mathrm{ELBO}(x) = \underbrace{\mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x \mid z)]}_{\text{reconstruction}} - \underbrace{D_{\mathrm{KL}}(q_\phi(z \mid x) \,\|\, p(z))}_{\text{stay close to the prior}}.$$

The first term rewards codes that let the decoder reproduce x; the second penalizes codes whose distribution wanders away from the prior. The same bound can be reached in one line with Jensen's inequality ($\log \mathbb{E} \geq \mathbb{E} \log$), but the decomposition above is more useful because it names the gap. Training maximizes the average ELBO over the data with respect to both θ and ϕ; equivalently, it minimizes the **negative ELBO**.

For a diagonal Gaussian encoder $q_\phi = \mathcal{N}(\mu, \mathrm{diag}(\sigma^2))$ and a standard Gaussian prior, the KL term has a closed form:

$$D_{\mathrm{KL}}(q_\phi \| p) = \frac{1}{2} \sum_{j=1}^{k} \left(\mu_j^2 + \sigma_j^2 - \log \sigma_j^2 - 1 \right),$$

where μ_j and σ_j^2 are the encoder's mean and variance for latent coordinate j. Each summand is zero exactly when $\mu_j = 0$ and $\sigma_j^2 = 1$.

Reparameterization

The reconstruction term is an expectation over $z \sim q_\phi(z \mid x)$, and we need its gradient with respect to ϕ. Sampling is not differentiable as written: a random draw is a black box. The **reparameterization trick** [6] moves the randomness into a fixed noise variable:

$$\epsilon \sim \mathcal{N}(0, I), \qquad z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon.$$

Here $\odot$ is elementwise multiplication. Now z is a deterministic, differentiable function of ϕ and an independent ϵ, so ordinary backpropagation gives low-variance gradient estimates through μ and σ. In code, the encoder predicts `logvar` $= \log \sigma^2$ and uses `exp(.5 * logvar)` as the standard deviation, which guarantees positivity without constraints.

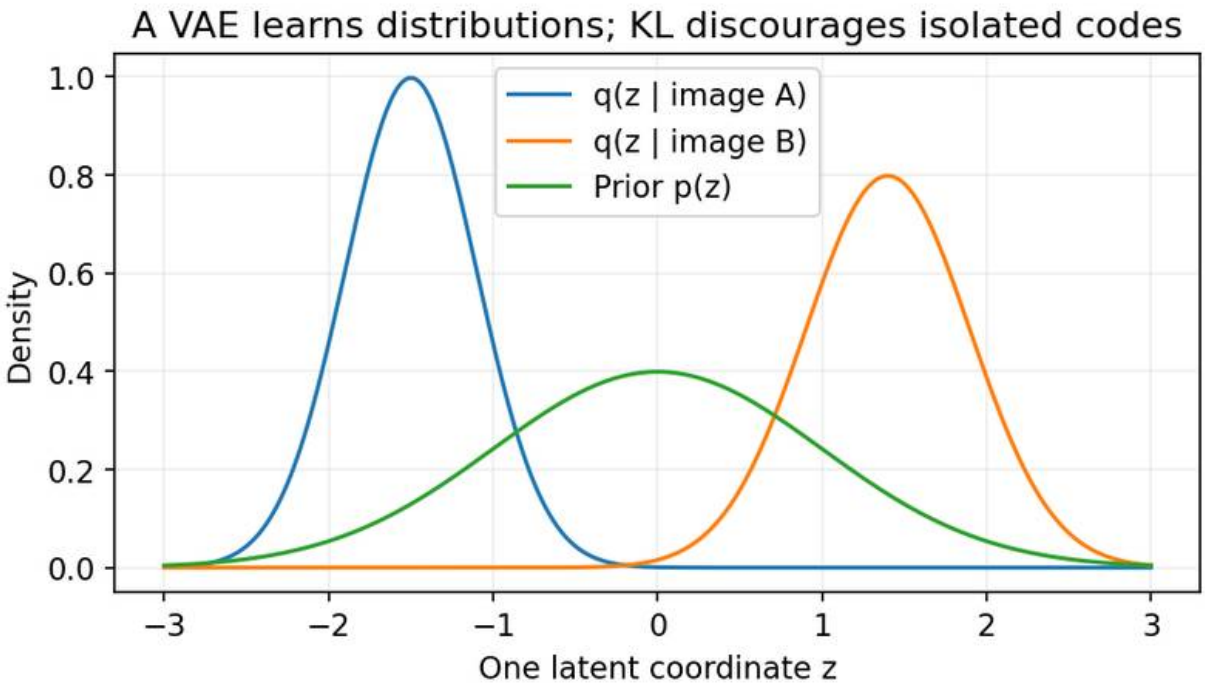


Figure 2 — A VAE represents uncertainty in its code and encourages compatibility with a shared prior. The curves are illustrative.

Code: a small VAE for binary digits

This complete CPU example uses scikit-learn's bundled 8×8 digits, binarized so that a Bernoulli decoder is a stated likelihood model. Reconstruction is summed over pixels and KL over latent dimensions, then both are averaged over examples; silently averaging only one term would change their balance.

```
import torch
from torch import nn
from torch.nn import functional as F
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

torch.manual_seed(0)
torch.set_num_threads(1)
d = load_digits()
train, test = train_test_split(d.data, test_size=.2, random_state=0,
                               stratify=d.target)
X = torch.tensor((train > 8).astype("float32"))
Xt = torch.tensor((test > 8).astype("float32"))
enc = nn.Sequential(nn.Linear(64, 64), nn.ReLU(), nn.Linear(64, 16))
dec = nn.Sequential(nn.Linear(8, 64), nn.ReLU(), nn.Linear(64, 64))
opt = torch.optim.Adam(list(enc.parameters()) + list(dec.parameters()), lr=.002)

def terms(x):
    mu, logvar = enc(x).chunk(2, dim=-1)
    z = mu + (.5*logvar).exp() * torch.randn_like(mu)
    recon = F.binary_cross_entropy_with_logits(dec(z), x, reduction="none").sum(1)
    kl = .5*(mu.square() + logvar.exp() - logvar - 1).sum(1)
    return recon, kl

for step in range(400):
    x = X[torch.randint(len(X), (64,))]
    recon, kl = terms(x)
    loss = (recon + kl).mean()
    opt.zero_grad()
    loss.backward()
    opt.step()
enc.eval(); dec.eval()
with torch.no_grad():
    recon, kl = terms(Xt)
    samples = dec(torch.randn(4, 8)).sigmoid().reshape(4, 8, 8)
    print("test negative ELBO estimate:", round((recon + kl).mean().item(), 3))
    print("reconstruction / KL:", round(recon.mean().item(), 3),
          round(kl.mean().item(), 3))
    print("sample probability maps:", tuple(samples.shape))
```

On our machine it prints a test negative ELBO of about 20.5 nats per image, split into roughly 17.0 nats of reconstruction and 3.5 nats of KL. Since the bound is a lower bound on log-likelihood, 20.5 nats is an **upper bound** on the test NLL. The output maps are Bernoulli probabilities; to obtain

binary samples, draw each pixel with those probabilities. The test estimate is stochastic, so average several latent draws for a more stable number. No test-based checkpoint selection happens here.

Figure 3 shows what a VAE with only a **two-dimensional** latent space learns on the same data (trained longer by the figure script). Even though labels are never used, digits of the same class land near each other, and decoding a regular grid of z values produces smooth transitions between digit shapes. This is the property an ordinary autoencoder lacks: every point near the prior's bulk decodes to something digit-like.

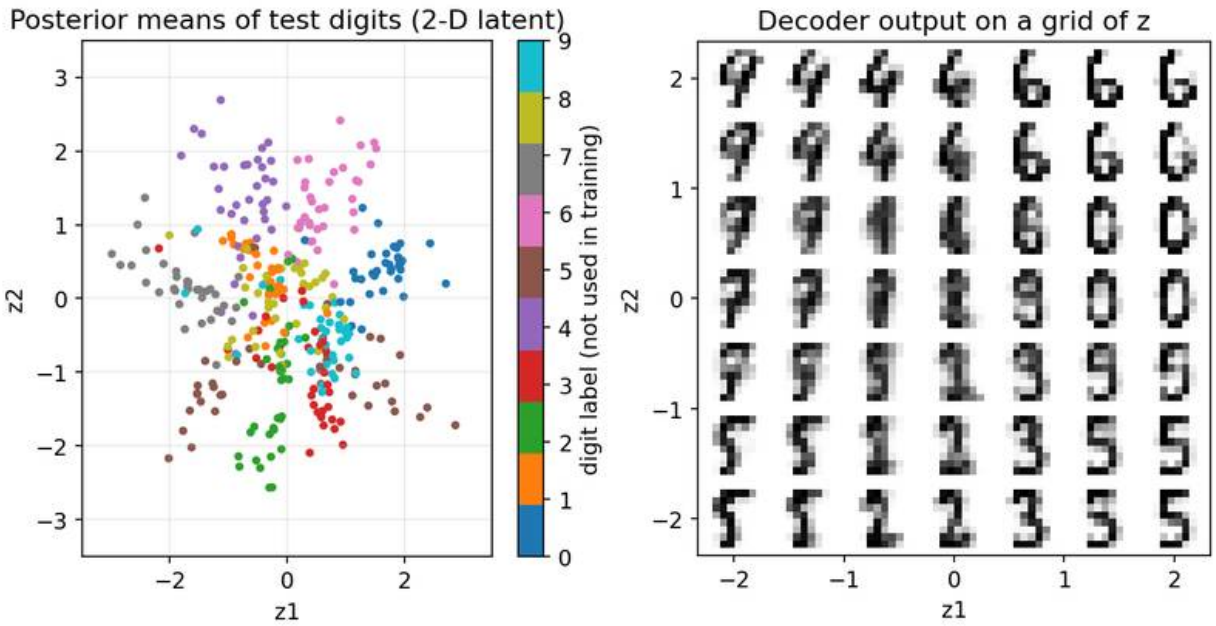


Figure 3 — A VAE with a 2-D latent space on binarized 8 × 8 digits. Left: encoder means for test images (labels used only for coloring). Right: decoder outputs on a grid of latent values.

Posterior collapse, β-VAE and deeper VAEs

If the decoder is very powerful (for example, an autoregressive decoder that can model pixels on its own), the cheapest way to raise the ELBO may be to ignore z entirely: set $q_\phi(z \mid x) = p(z)$ so the KL term is zero, and let the decoder do all the work. This is **posterior collapse**. Bowman et al. [7] observed it when training VAEs on text and proposed **KL annealing**: start with a small weight on the KL term and raise it to 1 during training, so the model learns to use the code before the penalty bites. Monitoring the KL per latent dimension is the simplest diagnostic — dimensions with KL near zero carry no information.

Higgins et al. [8] go the other way: the **β-VAE** multiplies the KL term by $\beta > 1$, pressuring the model toward a more factorized, "disentangled" code at the cost of reconstruction. Be precise about what this means: with $\beta \neq 1$ the objective is no longer the ELBO, so the value is not a bound on $\log p_\theta(x)$, and Kingma and Welling's tutorial review [9] discusses this and many other variants. Finally, single-layer VAEs with factorized Gaussians tend to produce blurry images,

partly because the Gaussian decoder averages over uncertainty. **Hierarchical VAEs** stack several layers of latent variables; NVAE [10] showed that a carefully engineered deep hierarchy with stabilizing regularization can produce sharp samples on natural images up to 256×256 pixels.

GANs: learning with an adversary

Plain version. Two networks play a game. A **generator** turns random noise into fake examples; a **discriminator** looks at an example and guesses whether it is real or fake. The generator wins by fooling the discriminator, which forces its outputs to look more and more like data.

The minimax game

Let G map noise $z \sim p(z)$ to samples $G(z)$, whose distribution we call p_g. Let $D(x) \in (0,1)$ be the discriminator's estimated probability that x is real. The original GAN objective of Goodfellow et al. [11] is

$$\min_G \max_D \; V(D,G) = \mathbb{E}_{x\sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z\sim p(z)}[\log(1 - D(G(z)))].$$

The discriminator maximizes V: it is ordinary binary cross-entropy with real data labeled 1 and fakes labeled 0. The generator minimizes V: it wants $D(G(z))$ to be large.

What the game minimizes

For a fixed generator, write V as an integral over x: $V = \int [p_{\text{data}}(x)\log D(x) + p_g(x)\log(1 - D(x))]dx$. For each x separately, the function $a\log D + b\log(1-D)$ with $a, b > 0$ is maximized at $D = a/(a+b)$ (set the derivative $a/D - b/(1-D)$ to zero). So the **optimal discriminator** is

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)}.$$

It is the probability that a point came from the data rather than the generator, if both were equally likely a priori. Substituting D^* back in and adding and subtracting $\log 4$ gives

$$V(D^*, G) = -\log 4 + 2\,\mathrm{JSD}(p_{\text{data}} \,\|\, p_g),$$

where $\mathrm{JSD}(p\|q) = \frac{1}{2}D_{\mathrm{KL}}(p\|m) + \frac{1}{2}D_{\mathrm{KL}}(q\|m)$ with $m = \frac{1}{2}(p+q)$ is the **Jensen–Shannon divergence**. So, with a perfect discriminator, the generator minimizes a symmetric divergence between data and model, and the global optimum is $p_g = p_{\text{data}}$, where $D^* = 1/2$ everywhere [11]. Notice that no likelihood was ever computed: the discriminator acts as a learned, adaptive test of "does this look like the data?"

The non-saturating loss and how to implement it

Early in training the generator is poor, $D(G(z)) \approx 0$, and $\log(1 - D(G(z)))$ is flat, so the generator's gradient vanishes. In practice the generator instead minimizes $-\mathbb{E} \log D(G(z))$, the **non-saturating** loss proposed in the same paper [11], which has strong gradients precisely when the generator is losing. It has the same fixed point but different dynamics.

Updates alternate. When updating D, **detach** generated samples so no gradient reaches G; when updating G, let gradients flow through D into G but step only G's optimizer. Using logits with `binary_cross_entropy_with_logits` is numerically safer than applying a sigmoid and then a log.

Code: a GAN on eight Gaussians

The classic test bed for mode coverage is a ring of eight tight Gaussian clusters in 2-D. It is small enough to train on a CPU in under a minute, and you can **see** whether every mode is covered.

```
import math
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0)
torch.set_num_threads(1)
angles = torch.arange(8) * (2 * math.pi / 8)
centers = 2.0 * torch.stack([angles.cos(), angles.sin()], dim=1)

def sample_data(n):
    idx = torch.randint(8, (n,))
    return centers[idx] + 0.05 * torch.randn(n, 2)

def mlp(i, o):
    return nn.Sequential(nn.Linear(i, 128), nn.ReLU(),
                         nn.Linear(128, 128), nn.ReLU(), nn.Linear(128, o))

G, D = mlp(8, 2), mlp(2, 1)
opt_g = torch.optim.Adam(G.parameters(), lr=1e-3, betas=(.5, .999))
opt_d = torch.optim.Adam(D.parameters(), lr=1e-3, betas=(.5, .999))
for step in range(3000):
    real = sample_data(256)
    fake = G(torch.randn(256, 8))
    # Discriminator: logits, real -> 1, fake -> 0; detach so G is untouched.
    d_loss = (F.binary_cross_entropy_with_logits(D(real), torch.ones(256, 1))
              + F.binary_cross_entropy_with_logits(D(fake.detach()),
                                                   torch.zeros(256, 1)))
    opt_d.zero_grad(); d_loss.backward(); opt_d.step()
    # Generator: non-saturating loss -log D(G(z)).
    g_loss = F.binary_cross_entropy_with_logits(D(fake), torch.ones(256, 1))
    opt_g.zero_grad(); g_loss.backward(); opt_g.step()
```

```
with torch.no_grad():
    x = G(torch.randn(5000, 8))
    dist = torch.cdist(x, centers)
    nearest = dist.min(1)
    good = nearest.values < 0.15          # within 3 std of some mode
    counts = torch.bincount(nearest.indices[good], minlength=8)
print("fraction near a mode:", round(good.float().mean().item(), 3))
print("modes with >= 1% of samples:", int((counts >= 50).sum()))
print("samples per mode:", counts.tolist())
```

It prints:

```
fraction near a mode: 0.896
modes with >= 1% of samples: 7
samples per mode: [590, 14, 710, 276, 651, 731, 863, 646]
```

About 90% of samples land on a mode, but the mode at 45° receives only 14 of 5,000 samples and another is under-represented. That is **partial mode collapse** caught in the act. Figure 4 shows the generator's journey: an unstructured cloud early on, a ring that covers every mode but also the space between them, and finally sharper clusters with one mode nearly abandoned. Exact numbers depend on the seed and library version; the qualitative picture is typical.

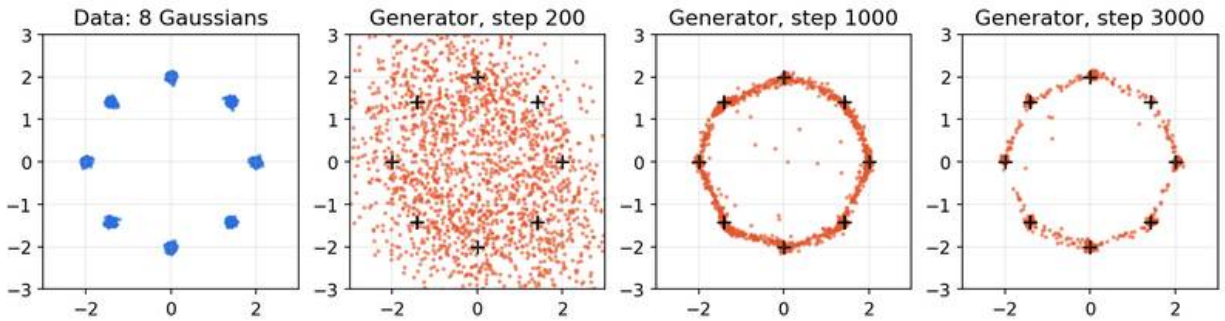


Figure 4 — A small GAN learning a ring of eight Gaussians (crosses mark the true modes). By step 3000 samples are sharp but one mode is nearly dropped: sharpness and coverage are separate achievements.

Mode collapse and why it happens

Mode collapse occurs when the generator produces too few kinds of output. The generator is rewarded for samples the current discriminator accepts, not for diversity. If it finds one region the discriminator likes, it can pile samples there; the discriminator then learns to reject that region, the generator hops to another, and the pair can chase each other around instead of converging. Unlike maximum likelihood, nothing in the per-sample generator loss directly punishes **missing** a mode. Salimans et al. [12] proposed early remedies such as feature matching and minibatch

discrimination, which let the discriminator look at statistics of a whole batch so that a lack of diversity becomes detectable.

A collection of sharp, convincing images can therefore still cover only a small part of the data distribution. Always measure coverage separately from quality — we return to this under evaluation.

Wasserstein GANs

When p_{data} and p_g live on thin, non-overlapping sets (as images do in pixel space), the JSD is stuck at its maximum $\log 2$ and provides no useful gradient about **how far apart** they are. Arjovsky et al. [13] proposed the **Wasserstein-1 (earth mover's) distance** instead: the minimum cost of moving probability mass from one distribution to the other. Its dual form, from the Kantorovich–Rubinstein theorem, is

$$W(p_{\text{data}}, p_g) = \sup_{\|f\|_L \le 1} \mathbb{E}_{x \sim p_{\text{data}}}[f(x)] - \mathbb{E}_{x \sim p_g}[f(x)],$$

where the supremum is over all 1-Lipschitz functions f (functions whose output changes by at most $\|x - x'\|$ when the input changes from x' to x). The network f is now called a **critic**: it outputs an unbounded score, not a probability. The Wasserstein distance shrinks smoothly as the generator's samples move toward the data, even when the supports do not overlap.

The hard part is enforcing the Lipschitz constraint. The original WGAN clips every critic weight to a small interval, which is crude and can make the critic too simple. **WGAN-GP** [14] instead adds a gradient penalty

$$\lambda \, \mathbb{E}_{\hat{x}}[(\|\nabla_{\hat{x}} f(\hat{x})\|_2 - 1)^2],$$

where $\hat{x}$ is drawn uniformly on straight lines between pairs of real and generated samples and λ is a weight (the paper uses 10). Two other widely used stabilizers act on the discriminator directly: **spectral normalization** [15] divides each weight matrix by its largest singular value, bounding the Lipschitz constant layer by layer, and the **R1 penalty** analyzed by Mescheder et al. [16] penalizes $\frac{\gamma}{2}\|\nabla_x D(x)\|^2$ on real data, which they show gives local convergence where unregularized GAN training can oscillate.

Architectures and where GANs are used now

StyleGAN [17] illustrates how architecture matters as much as the loss: it maps noise to an intermediate latent space and injects that "style" at every resolution, giving controllable, high-quality face synthesis. GANs sample with a single generator pass, which makes them fast at inference. Adversarial losses also appear as **components** of other systems — for example, inside image tokenizers trained to reconstruct sharp detail. That does not make every model using such a tokenizer a GAN generator; the distribution may be modeled by something else entirely.

Normalizing flows: invertible transformations

Plain version. Start from simple noise, such as a standard Gaussian, and push it through a chain of invertible transformations. Because every step can be undone, you can run any image backwards to the noise that would have produced it — and keep exact track of how each step stretches or squeezes space, which gives an exact likelihood.

Change of variables

Let $z \sim p_0(z)$ be a simple base distribution and $x = f_\theta(z)$ an invertible, differentiable map with the same dimension. Then

$$\log p_\theta(x) = \log p_0(f_\theta^{-1}(x)) - \log \left| \det \frac{\partial f_\theta}{\partial z} \right|_{z=f_\theta^{-1}(x)}.$$

$\partial f_\theta / \partial z$ is the Jacobian matrix of the map, and its determinant measures how much the map scales a small volume around z. Probability mass is conserved, so if a region is stretched to twice its volume, its density must halve. In one dimension, $x = 2z$ doubles every interval and adds $-\log 2$ to the log density. A composition $f = f_K \circ \cdots \circ f_1$ simply adds up the log-determinants of the steps.

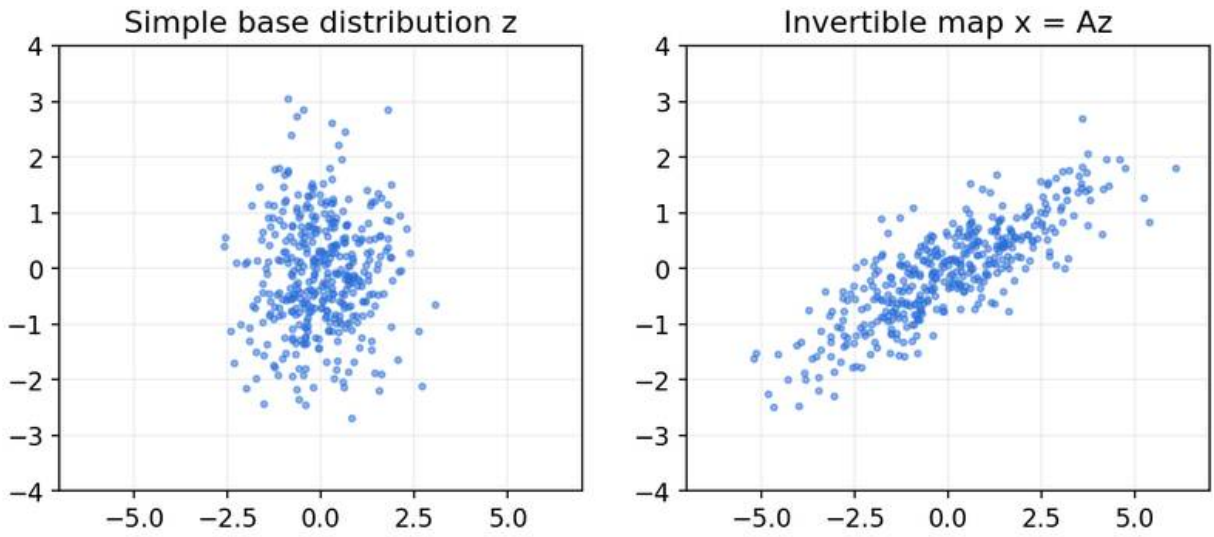


Figure 5 — Change of variables preserves total probability while changing density. This affine map has an analytically tractable determinant.

Coupling layers

The formula is exact, but a general $D \times D$ determinant costs $O(D^3)$ — hopeless for images. Flow layers are designed so that the Jacobian is **triangular**, whose determinant is just the product of the diagonal. The affine coupling layer of RealNVP [18] splits x into two halves (x_1, x_2) and computes

$$y_1 = x_1, \qquad y_2 = x_2 \odot \exp\big(s(x_1)\big) + t(x_1),$$

where s and t are arbitrary neural networks. The Jacobian is block-triangular with the identity on the first block and $\operatorname{diag}(\exp s(x_1))$ on the second, so $\log|\det J| = \sum_j s_j(x_1)$. The inverse is just as cheap: $x_2 = (y_2 - t(y_1)) \odot \exp(-s(y_1))$. Crucially, s and t never need to be inverted, so they can be as complex as you like. Stacking layers with alternating splits lets every coordinate be transformed. Glow [19] added learned invertible 1×1 convolutions as a generalized channel permutation between coupling layers.

Code: verify a coupling layer's log-determinant

The snippet below builds one affine coupling layer and checks its cheap log-determinant against the full Jacobian computed by autograd.

```
import torch
from torch import nn

torch.manual_seed(0)

class AffineCoupling(nn.Module):
    """RealNVP-style layer: keep x1, scale-and-shift x2 using x1."""
    def __init__(self, dim, hidden=32):
        super().__init__()
        self.d = dim // 2
        self.net = nn.Sequential(nn.Linear(self.d, hidden), nn.Tanh(),
                                 nn.Linear(hidden, 2 * (dim - self.d)))

    def forward(self, x):
        x1, x2 = x[..., :self.d], x[..., self.d:]
        s, t = self.net(x1).chunk(2, dim=-1)
        s = torch.tanh(s)                          # keep scales moderate
        y2 = x2 * s.exp() + t
        return torch.cat([x1, y2], -1), s.sum(-1)   # output, log|det J|

    def inverse(self, y):
        y1, y2 = y[..., :self.d], y[..., self.d:]
        s, t = self.net(y1).chunk(2, dim=-1)
        s = torch.tanh(s)
        return torch.cat([y1, (y2 - t) * (-s).exp()], -1)

layer = AffineCoupling(4)
x = torch.randn(4)
y, logdet = layer(x)
J = torch.autograd.functional.jacobian(lambda v: layer(v)[0], x)
print("log|det J| from layer:   ", round(logdet.item(), 5))
print("log|det J| from autograd:", round(torch.linalg.slogdet(J)[1].item(), 5))
print("upper-right block is zero:", bool(torch.allclose(J[:2, 2:], torch.zeros(2, 2))))
print("inverse error:", (layer.inverse(y) - x).abs().max().item() < 1e-6)

# Log density of y under a standard-normal base after one layer:
base = torch.distributions.Normal(0., 1.)
```

```
z = layer.inverse(y)
log_px = base.log_prob(z).sum() - layer(z)[1]
print("log p(y):", round(log_px.item(), 4))
```

Both log-determinants print 0.14867, the upper-right Jacobian block is exactly zero (the triangular structure), and the inverse recovers the input. The last line is the change-of-variables formula in action. Training a flow simply maximizes this `log p` over data with a stack of such layers.

Strengths, limits and continuous flows

Flows give exact likelihoods, exact inversion (useful for editing and for mapping data to a well-behaved latent space) and one-pass sampling. The price is architectural: every layer must be invertible with a cheap determinant, and the latent must have the **same dimension** as the data, which wastes capacity on representing noise in all pixel directions. Rezende and Mohamed [20] introduced flows as a way to make the approximate posterior of a VAE more flexible, which shows that the same tool serves inference as well as generation.

Continuous normalizing flows replace a discrete stack by an ordinary differential equation $\mathrm{d}x/\mathrm{d}t = v_\theta(x,t)$. Neural ODEs [21] showed that the log-density then evolves through the **divergence** (trace of the Jacobian) of v_θ rather than a determinant, which removes the architectural restriction but makes likelihood evaluation require an ODE solve. **Flow matching** [22], covered in ML 07, trains such a vector field by simple regression instead of maximum likelihood. The word "flow" therefore refers to related transformations, but the training objective need not be likelihood.

Energy-based models

Plain version. Instead of describing how to generate an image, give every possible image a score — an "energy" — where low energy means "looks real." Probability is high wherever energy is low.

An energy-based model (EBM) defines

$$p_\theta(x) = \frac{\exp(-E_\theta(x))}{Z_\theta}, \qquad Z_\theta = \int \exp(-E_\theta(x))\, dx,$$

where E_θ is any network that outputs a scalar and Z_θ is the **partition function** that makes the density integrate to one. EBMs are extremely flexible — any architecture works — but Z_θ is intractable in high dimensions, so neither the likelihood nor exact samples are directly available. Goodfellow, Bengio and Courville [23] cover the classical tools (the partition-function chapter and approximate inference), and Song and Kingma [24] survey modern training routes: maximum likelihood with MCMC samples (for example, Langevin dynamics), **score matching**, which fits $\nabla_x \log p_\theta(x) = -\nabla_x E_\theta(x)$ and thereby never needs Z_θ, and noise-contrastive methods. Score matching is the bridge to diffusion models.

Autoregressive models, briefly

Plain version. Generate an image one piece at a time, each piece conditioned on everything generated so far, like writing a sentence word by word.

The chain rule of probability is exact for any ordering of the D elements of x:

$$p_\theta(x) = \prod_{i=1}^{D} p_\theta(x_i \mid x_1, \dots, x_{i-1}).$$

Each factor is an ordinary classification or regression problem, so training is simple, stable maximum likelihood with an exact likelihood. PixelRNN and PixelCNN [25] modeled raw pixels this way with a 256-way softmax per subpixel. The drawback is sequential sampling: D network evaluations for D elements. Modern systems instead predict compressed tokens, groups of tokens or whole scales at once. ML 08 develops this family in detail.

Diffusion models, briefly

Plain version. Teach a network to remove a little noise from a noisy image. Then start from pure noise and apply the denoiser many times, slowly sculpting an image.

A diffusion model defines a fixed forward process that gradually adds Gaussian noise to data until nothing but noise remains, and learns a reverse process that undoes it step by step. Ho et al. [26] showed that training reduces, up to weighting, to predicting the noise that was added — a simple regression — and that the objective is a reweighted variational bound, which is why the taxonomy in Figure 1 lists diffusion next to the VAE. The connection to score matching (see the energy-based models section) gives a second, equivalent view. The survey by Yang et al. [27] maps the many variants. Diffusion and flow matching now dominate high-quality image generation and are the subject of ML 07.

The family map

Family	Basic training signal	Sampling mechanism	Likelihood	Main qualification
VAE	Variational bound (ELBO)	Sample latent, then decoder (one pass)	Lower bound	Bound depends on approximate inference; samples can be blurry
GAN	Adversarial discrimination	Generator pass (one pass)	None	Unstable game; mode collapse
Normalizing flow	Change-of-variables likelihood	Invertible transform (one pass)	Exact	Invertibility and same-dimension constraints
Autoregressive	Conditional likelihoods	Sequential conditional sampling	Exact	Generation dependencies can be costly

Family	Basic training signal	Sampling mechanism	Likelihood	Main qualification
Diffusion	Denoising/score objectives	Reverse noise process (many steps)	Bound or ODE-based	Quality depends on model and sampler
Flow matching	Velocity regression	Integrate learned vector field	Needs extra computation	Integration cost
Energy-based	Energy learning (MCMC, score matching)	Often iterative sampling	Unnormalized	Normalization and mixing can be hard

Bond-Taylor et al. [3] summarize these trade-offs as a triangle that is hard to satisfy all at once: fast sampling, high sample quality, and good mode coverage. GANs are fast and sharp but can miss modes; likelihood models cover modes but were historically slower or blurrier; diffusion models reach high quality and coverage at the cost of many sampling steps. This table is a teaching synthesis, not an exclusive taxonomy. Real models combine families: a VAE compresses an image, a transformer models its latent distribution, and a diffusion objective trains that transformer.

Evaluating generative models

Plain version. A generator can be good in different ways: it can assign high probability to real data, make individually convincing samples, cover all the variety in the data, or follow its conditioning. No single number captures all of these, so use several measures and know what each misses.

Theis et al. [5] made this point sharply: average log-likelihood, sample quality and older proxies such as Parzen-window estimates can be **largely independent** of one another in high dimensions. A model can have excellent likelihood and poor samples, or the reverse. Evaluate against the purpose you have in mind.

Likelihood

For likelihood-based models, held-out NLL (or bits per dimension) is principled and comparable when dequantization and preprocessing match. It cannot be computed for GANs. And a high likelihood need not mean reliable out-of-distribution detection: Nalisnick et al. [28] found that flows, VAEs and PixelCNNs trained on CIFAR-10 can assign **higher** likelihood to the unrelated SVHN digit images, because likelihood is dominated by low-level statistics such as smoothness rather than semantic content.

Inception Score

Salimans et al. [12] proposed the **Inception Score (IS)**. Feed each generated image x to a pre-trained ImageNet classifier (Inception-v3) to get a label distribution $p(y \mid x)$, and let $p(y) = \mathbb{E}_x[p(y \mid x)]$ be the average over samples. Then

$$\mathrm{IS} = \exp\left(\mathbb{E}_{x\sim p_g} D_{\mathrm{KL}}(p(y \mid x) \,\|\, p(y))\right).$$

IS is high when each image is confidently classified ($p(y \mid x)$ peaked) and the set covers many classes ($p(y)$ spread out). It never looks at real data, so it cannot tell whether samples resemble **your** dataset, and it is meaningful only for ImageNet-like content.

Fréchet Inception Distance

The **Fréchet Inception Distance (FID)** of Heusel et al. [29] compares real and generated images in a feature space. Extract Inception-v3 features for both sets, fit a Gaussian to each — means μ_r, μ_g and covariances Σ_r, Σ_g — and compute the Fréchet distance between the Gaussians:

$$\mathrm{FID} = \|\mu_r - \mu_g\|_2^2 + \mathrm{Tr}\left(\Sigma_r + \Sigma_g - 2(\Sigma_r\Sigma_g)^{1/2}\right).$$

Lower is better. FID uses real data and is sensitive to both quality and diversity, which made it the standard metric. But it is not an image-by-image truth measure: it depends on the feature extractor, image resizing, and especially the **number of samples** (it is biased for finite sets, so compare FIDs only with equal sample counts), and it folds quality and coverage into one number.

Precision and recall

To separate the two, Sajjadi et al. [30] defined **precision and recall for distributions**: precision asks what fraction of generated samples fall where real data lives (quality), recall asks what fraction of the real distribution the generator covers (diversity). Kynkäänniemi et al. [31] gave a simple, robust estimator that we implement below. Approximate each set's manifold by a union of balls around its points, with each ball's radius equal to the distance to the point's k-th nearest neighbor in the same set. Then

- **precision** = fraction of generated samples inside some real sample's ball;
- **recall** = fraction of real samples inside some generated sample's ball.

On images, the points are deep features rather than pixels. On our 2-D ring we can use coordinates directly:

```
import numpy as np

rng = np.random.default_rng(0)
angles = np.arange(8) * 2 * np.pi / 8
centers = 2 * np.stack([np.cos(angles), np.sin(angles)], 1)

def ring(n, modes):
    idx = rng.choice(modes, n)
    return centers[idx] + .05 * rng.standard_normal((n, 2))

def knn_radius(a, k):
    d = np.linalg.norm(a[:, None] - a[None], axis=-1)
```

```
    return np.sort(d, axis=1)[:, k]          # column 0 is the point itself

def coverage(query, ref, k=3):
    """Fraction of query points inside some reference point's k-NN ball."""
    r = knn_radius(ref, k)
    d = np.linalg.norm(query[:, None] - ref[None], axis=-1)
    return np.mean((d <= r[None]).any(axis=1))

real = ring(1000, range(8))
fakes = {
    "all 8 modes": ring(1000, range(8)),
    "only 2 modes": ring(1000, [0, 4]),
    "8 modes, blurry": ring(1000, range(8)) + .4 * rng.standard_normal((1000, 2)),
}
for name, fake in fakes.items():
    precision = coverage(fake, real)   # are fake samples on the real manifold?
    recall = coverage(real, fake)      # is the real manifold covered by fakes?
    print(f"{name:16s} precision={precision:.2f} recall={recall:.2f}")
```

```
all 8 modes      precision=0.97 recall=0.96
only 2 modes     precision=0.97 recall=0.26
8 modes, blurry  precision=0.07 recall=0.99
```

The mode-collapsed generator keeps high precision — each of its samples is perfect — but its recall drops to about 2/8. The blurry generator is the mirror image: it covers every mode but most samples are off the data. A single scalar such as FID would rank both as "worse" without saying **why**. Figure 6 shows the three cases.

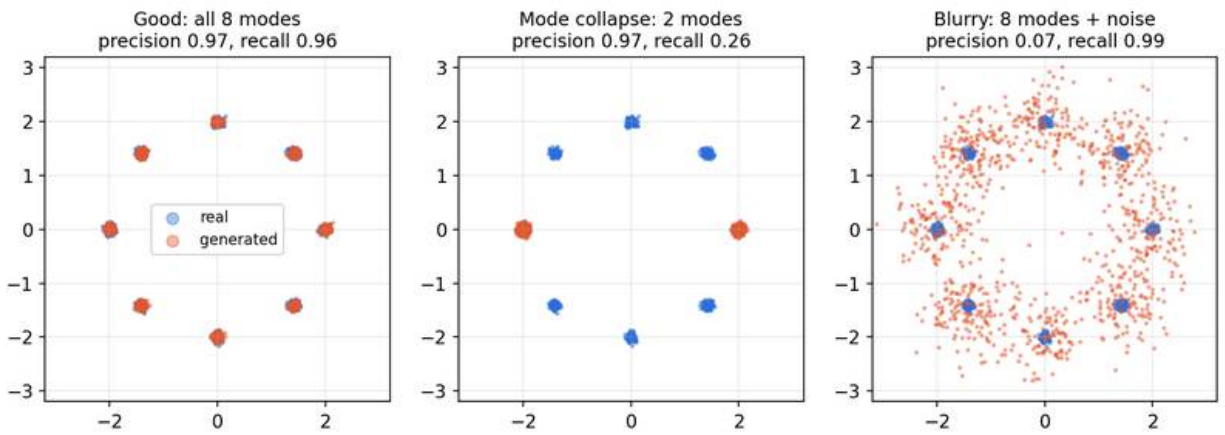


Figure 6 — k-nearest-neighbor precision and recall on a toy ring: mode collapse lowers recall, blur lowers precision.

Known limitations of current metrics

Evaluation is itself an active research area. Stein et al. [32] ran large human studies and found that no existing automatic metric strongly correlates with human judgments of realism; in particular, FID computed on Inception features under-rates the perceptual realism of diffusion models. They recommend richer self-supervised feature spaces (DINOv2 ViT-L/14) for metrics like FID and precision/recall. Jayasumana et al. [33] argue that FID's normality assumption and Inception features are poor fits for modern text-to-image models and propose **CMMD**, a maximum mean discrepancy computed on CLIP embeddings, which needs no Gaussian assumption. Practical advice: report several complementary metrics with their exact protocol (feature extractor, sample count, resolution, reference set), show uncurated samples, and run a human study when the claim is about perceived quality.

For conditional tasks, add a measure of **alignment** (does the image match the text or the low-resolution input?). For restoration, a beautiful generated texture may be unsupported by the observation; the distinction between plausible samples and measured evidence is central in DLCV 04.

Modern view

Surveys worth reading. Four reviews cover this chapter's ground from different angles.

- **Bond-Taylor et al. [3]** (IEEE TPAMI 2022) is the best single comparative review of VAEs, GANs, normalizing flows, energy-based and autoregressive models. It organizes each family by its core technique and its known pathologies, and compares them along training speed, sampling speed, sample quality, diversity and likelihood, concluding that each family pays for its strengths with a weakness somewhere else, and that hybrids are the natural way forward.
- **Papamakarios et al. [4]** (JMLR 2021) is a unified treatment of normalizing flows: their expressive power, the design space of coupling, autoregressive, residual and continuous flows, and their uses for density estimation, inference and simulation.
- **Kingma and Welling [9]** (Foundations and Trends in ML 2019) is the authoritative tutorial-review of VAEs by their inventors: the derivation, the reparameterization estimator, tighter bounds, more flexible posteriors (including flows), hierarchical models and the β-weighted variants.
- **Yang et al. [27]** (ACM Computing Surveys 2023) surveys diffusion models — sampling acceleration, likelihood improvements, structured data, and the links to VAEs, GANs, flows and EBMs — and is the bridge to the next chapter.

The textbooks of Bishop and Bishop [1] and Prince [2] give coherent derivations of all the families, with Prince's notebooks providing runnable companions.

Where the field is (2023–2026). Diffusion and flow-matching models, usually operating in a learned latent space, now produce most state-of-the-art image and video generation. That makes the **autoencoder** a load-bearing component again: VQ-VAE [34] showed how to learn discrete latent codes that a separate prior can model, and latent diffusion [35] showed that running the

generative model inside a compressed autoencoder latent space cuts cost dramatically while keeping detail. A tokenizer must balance compression, reconstruction and how easy its latent space is to model; evaluate its reconstruction first, because no downstream prior can restore detail the encoder discarded.

GANs have not disappeared. GigaGAN [36] scaled a GAN to text-to-image synthesis and emphasized what GANs still do best: single-pass sampling fast enough for interactive use, high-resolution upsampling, and a smooth, editable latent space. Huang et al. [37] revisited the training objective itself, deriving a regularized relativistic GAN loss with local convergence guarantees and showing that a minimalist modern baseline (R3GAN) without the traditional bag of tricks can surpass StyleGAN2 on standard benchmarks. Adversarial losses also live on inside other systems, for example to sharpen tokenizer reconstructions.

On evaluation, the 2023–2024 critiques of FID [32], [33] have moved the field toward richer feature extractors, separate precision/recall-style measures and human studies, but there is still no accepted single metric.

Open problems. (1) Reliable evaluation that agrees with humans and separates quality, diversity and memorization. (2) Detecting when a model reproduces training images rather than generalizing. (3) Calibrated likelihoods that are actually useful for out-of-distribution detection [28]. (4) The speed–quality–coverage trade-off: getting diffusion-level quality with GAN-level speed without losing modes. (5) Learning latent spaces that are simultaneously compressive, faithful and easy to model.

Key takeaways

- Generation requires a distribution or a defined sampling process. Maximum likelihood equals minimizing $D_{\mathrm{KL}}(p_{\mathrm{data}} \| p_\theta)$, which encourages covering every mode.
- Likelihood-based models (autoregressive, flows, VAEs, EBMs, diffusion's bound) write down or bound $\log p(x)$; implicit models (GANs) only provide a sampler.
- An ordinary autoencoder supplies reconstruction but no prior over codes, so decoding random codes need not give realistic samples.
- The ELBO equals $\log p_\theta(x)$ minus $D_{\mathrm{KL}}(q_\phi(z \mid x) \| p_\theta(z \mid x))$; reparameterization makes it trainable by backpropagation.
- With an optimal discriminator, the original GAN minimizes $2\,\mathrm{JSD} - \log 4$; the non-saturating loss, Wasserstein critics, gradient penalties, spectral normalization and R1 make the game trainable, but mode collapse must still be measured.
- Flows compute exact likelihoods via the change-of-variables formula, made cheap by triangular Jacobians in coupling layers.
- Likelihood, IS, FID and precision/recall measure different things; report several, with their exact protocol.

Exercises

1. **KL sanity check.** Substitute $\mu = 0$ and $\sigma^2 = 1$ into the Gaussian KL. Then compute the KL for $\mu = 1, \sigma^2 = 1$ in one dimension.

   Hint

   For $\mu = 0, \sigma^2 = 1$ every term is zero, because posterior and prior are the same distribution. For $\mu = 1$, $\sigma^2 = 1$ you get $\frac{1}{2}(1 + 1 - 0 - 1) = 0.5$ nats: shifting the mean by one standard deviation costs half a nat.

2. **Loss scaling.** Average the VAE's reconstruction across pixels rather than summing. What happens if KL is unchanged?

   Hint

   Reconstruction becomes 64 times smaller relative to KL. This is equivalent to a β-VAE with $\beta = 64$ on the summed loss: it effectively changes regularization (often toward posterior collapse), rather than merely changing reported units.

3. **Optimal discriminator.** At a point x where $p_{\text{data}}(x) = 0.3$ and $p_g(x) = 0.1$, what is $D^*(x)$? What is D^* everywhere when $p_g = p_{\text{data}}$, and what is the value of the game then?

   Hint

   $D^*(x) = 0.3/(0.3 + 0.1) = 0.75$. When the distributions match, $D^* = 1/2$ everywhere and $V = \log \frac{1}{2} + \log \frac{1}{2} = -\log 4$, consistent with JSD $= 0$.

4. **Affine flow.** For $x = Az + b$ with $z \sim \mathcal{N}(0, I)$, derive the density correction and explain why singular A invalidates it. Check your answer against the known Gaussian $\mathcal{N}(b, AA^\top)$.

   Hint

   Subtract $\log|\det A|$. Since $\det(AA^\top)^{1/2} = |\det A|$, this matches the normalizing constant of $\mathcal{N}(b, AA^\top)$. A singular map is not invertible and collapses volume, so the same-dimensional change-of-variables formula does not apply.

5. **Spot collapse.** Propose different checks for VAE posterior collapse and GAN mode collapse. Then modify the GAN code to use WGAN-GP (critic without sigmoid, gradient penalty with $\lambda = 10$, several critic steps per generator step) and compare the per-mode counts.

   Hint

   For a VAE, inspect the KL per latent dimension and whether decoded outputs change when you vary z. For a GAN, count samples per mode (as in our code), compute recall, and look for repeated samples at fixed conditions. They are different failures. For WGAN-GP, sample $\hat{x} = \alpha x_{\text{real}} + (1 - \alpha) x_{\text{fake}}$ with $\alpha \sim U(0, 1)$, call `torch.autograd.grad` with `create_graph=True`, and lower the learning rate if training oscillates. Compare across several seeds before concluding anything.

6. **Metric disagreement.** Using the precision/recall code, construct a fake set that has roughly the same precision and recall as the good generator but places half its samples in only four modes. What does this tell you about the limits of the metric?

Hint

Unequal mode weights can still give high recall, because recall only asks whether each real point is near **some** fake point, not whether mode frequencies match. Precision/recall measure support overlap, not density; pair them with a distribution-level measure such as FID or a per-mode histogram.

References

1. C. M. Bishop and H. Bishop, *Deep Learning: Foundations and Concepts*, Springer, 2024. Book
2. S. J. D. Prince, *Understanding Deep Learning*, MIT Press, 2023. Book and notebooks
3. S. Bond-Taylor, A. Leach, Y. Long and C. G. Willcocks, "Deep Generative Modelling: A Comparative Review of VAEs, GANs, Normalizing Flows, Energy-Based and Autoregressive Models," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2022. Paper
4. G. Papamakarios et al., "Normalizing Flows for Probabilistic Modeling and Inference," *JMLR*, 2021. Review
5. L. Theis, A. van den Oord and M. Bethge, "A note on the evaluation of generative models," ICLR, 2016. Paper
6. D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," ICLR, 2014. Paper
7. S. R. Bowman et al., "Generating Sentences from a Continuous Space," CoNLL, 2016. Paper
8. I. Higgins et al., "β-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework," ICLR, 2017. Paper
9. D. P. Kingma and M. Welling, "An Introduction to Variational Autoencoders," *Foundations and Trends in Machine Learning*, 2019. Review
10. A. Vahdat and J. Kautz, "NVAE: A Deep Hierarchical Variational Autoencoder," NeurIPS, 2020. Paper
11. I. Goodfellow et al., "Generative Adversarial Nets," NeurIPS, 2014. Paper
12. T. Salimans et al., "Improved Techniques for Training GANs," NeurIPS, 2016. Paper
13. M. Arjovsky, S. Chintala and L. Bottou, "Wasserstein GAN," ICML, 2017. Paper
14. I. Gulrajani et al., "Improved Training of Wasserstein GANs," NeurIPS, 2017. Paper
15. T. Miyato et al., "Spectral Normalization for Generative Adversarial Networks," ICLR, 2018. Paper
16. L. Mescheder, A. Geiger and S. Nowozin, "Which Training Methods for GANs do actually Converge?" ICML, 2018. Paper
17. T. Karras, S. Laine and T. Aila, "A Style-Based Generator Architecture for Generative Adversarial Networks," CVPR, 2019. Paper
18. L. Dinh, J. Sohl-Dickstein and S. Bengio, "Density estimation using Real NVP," ICLR, 2017. Paper
19. D. P. Kingma and P. Dhariwal, "Glow: Generative Flow with Invertible 1x1 Convolutions," NeurIPS, 2018. Paper

20. D. J. Rezende and S. Mohamed, “Variational Inference with Normalizing Flows,” ICML, 2015. Paper
21. R. T. Q. Chen, Y. Rubanova, J. Bettencourt and D. Duvenaud, “Neural Ordinary Differential Equations,” NeurIPS, 2018. Paper
22. Y. Lipman et al., “Flow Matching for Generative Modeling,” ICLR, 2023. Paper
23. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016, ch. 18–20. Book
24. Y. Song and D. P. Kingma, “How to Train Your Energy-Based Models,” arXiv, 2021. Paper
25. A. van den Oord, N. Kalchbrenner and K. Kavukcuoglu, “Pixel Recurrent Neural Networks,” ICML, 2016. Paper
26. J. Ho, A. Jain and P. Abbeel, “Denoising Diffusion Probabilistic Models,” NeurIPS, 2020. Paper
27. L. Yang et al., “Diffusion Models: A Comprehensive Survey of Methods and Applications,” *ACM Computing Surveys*, 2023. Survey
28. E. Nalisnick et al., “Do Deep Generative Models Know What They Don’t Know?” ICLR, 2019. Paper
29. M. Heusel et al., “GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium,” NeurIPS, 2017. Paper
30. M. S. M. Sajjadi, O. Bachem, M. Lucic, O. Bousquet and S. Gelly, “Assessing Generative Models via Precision and Recall,” NeurIPS, 2018. Paper
31. T. Kynkäänniemi, T. Karras, S. Laine, J. Lehtinen and T. Aila, “Improved Precision and Recall Metric for Assessing Generative Models,” NeurIPS, 2019. Paper
32. G. Stein et al., “Exposing flaws of generative model evaluation metrics and their unfair treatment of diffusion models,” NeurIPS, 2023. Paper
33. S. Jayasumana et al., “Rethinking FID: Towards a Better Evaluation Metric for Image Generation,” CVPR, 2024. Paper
34. A. van den Oord, O. Vinyals and K. Kavukcuoglu, “Neural Discrete Representation Learning,” NeurIPS, 2017. Paper
35. R. Rombach et al., “High-Resolution Image Synthesis with Latent Diffusion Models,” CVPR, 2022. Paper
36. M. Kang et al., “Scaling up GANs for Text-to-Image Synthesis,” CVPR, 2023. Paper
37. Y. Huang, A. Gokaslan, V. Kuleshov and J. Tompkin, “The GAN is dead; long live the GAN! A Modern GAN Baseline,” NeurIPS, 2024. Paper

ML 07 • Diffusion Models and Flow Matching

Derive noise, clean-image and velocity prediction, compare pixel and latent generation with JiT and representation autoencoders, and understand pretrained image/video diffusion and flow pipelines.

Online edition: https://blog.shyandram.dev/en/learn/ml/ml-07-diffusion-flow-matching/

What you'll learn

- How a known noise process turns "generate an image" into an ordinary supervised regression problem.
- How the simplified noise-prediction loss of DDPM falls out of a variational bound, step by step.
- How noise, clean-image and velocity predictions convert algebraically, why their losses and finite networks differ, and how score estimation and the SDE view connect them.
- How DDPM, DDIM and fast ODE solvers sample, and why the number of steps is a design choice.
- How classifier guidance and classifier-free guidance steer a sampler, and what they cost.
- How pixel and latent diffusion, DiT, JiT, representation autoencoders, flow matching and few-step generation fit together, and how pretrained image/video pipelines are evaluated.

The big picture

A diffusion model learns to undo controlled corruption. During training we take a real example, add a known amount of Gaussian noise, and ask a network to tell us what noise we added. Because we added the noise ourselves, the correct answer is always available: no labels, no adversary, just regression. During generation we start from pure noise and apply the learned "denoise a little" step many times until an example appears.

> **Explain like I'm five**
>
> Imagine slowly covering a drawing with pencil scribbles. You keep copies of the drawing at many scribble levels: a few lines, lots of lines, so many that the drawing is gone. A model practices one small job: look at a scribbled page, and guess which scribbles were added. After lots of practice you hand it a page that is *only* scribbles. It erases a little, looks again, erases a little more, and a brand-new drawing slowly appears — one that was never in the pile.

Why does this matter? Since 2021, diffusion-style models have produced many of the state-of-the-art results in image, video and audio synthesis, and the same machinery is used for inpainting, super-resolution, image editing, 3-D generation and scientific data. Diffusion also replaced two pain points of earlier families covered in ML 06: GANs need a fragile two-player game, and VAEs

tend to produce blurry samples. Diffusion training is a stable regression, and its samples are sharp because generation is spread over many small, easy steps.

The price is sampling cost: a classic sampler calls the network hundreds of times per image. Much of the research since 2021 — DDIM, fast ODE solvers, distillation, consistency models, flow matching, rectified flow — is about keeping the quality while cutting the number of network calls.

The chapter follows the organization of the major surveys. Yang et al. [1] group the field into efficient sampling, improved likelihood and data with special structure; Croitoru et al. [27] group it by formulation (DDPM, noise-conditioned score networks, SDEs); Lipman et al.'s flow-matching guide [2] and the monograph by Lai et al. [29] present diffusion and flows as one family of learned transport. Prince's textbook [14] is a gentle companion for the derivations. Throughout, we use **one explicit time convention per section** and say when it changes — mixing conventions is the most common source of sign errors in this area.

Forward diffusion has a closed form

Plain version. At each step, keep most of the image and add a little independent Gaussian noise. After enough steps, nothing of the image is left — only noise with a known distribution.

The idea goes back to Sohl-Dickstein et al. [15], who proposed destroying structure with a slow forward diffusion and learning to reverse it; DDPM [3] made it practical. Let $x_0 \in \mathbb{R}^d$ be a data example (an image flattened into d numbers, scaled to roughly $[-1, 1]$). Pick a **noise schedule** $\beta_1, \dots, \beta_T$ with $0 < \beta_t < 1$ and write $\alpha_t = 1 - \beta_t$. The forward (noising) process is a Markov chain

$$q(x_t \mid x_{t-1}) = \mathcal{N}\big(\sqrt{\alpha_t}\, x_{t-1},\ \beta_t I\big), \qquad t = 1, \dots, T,$$

where $\mathcal{N}(\mu, \Sigma)$ is a Gaussian with mean μ and covariance Σ, and I is the $d \times d$ identity. The factor $\sqrt{\alpha_t}$ shrinks the signal slightly so that the total variance stays bounded instead of growing; this is why this family is called **variance preserving (VP)**.

Deriving the one-shot formula

Write one step as $x_t = \sqrt{\alpha_t} x_{t-1} + \sqrt{1-\alpha_t}\,\epsilon_t$ with $\epsilon_t \sim \mathcal{N}(0, I)$. Substituting the previous step,

$$\begin{aligned} x_t &= \sqrt{\alpha_t}\left(\sqrt{\alpha_{t-1}} x_{t-2} + \sqrt{1-\alpha_{t-1}}\,\epsilon_{t-1}\right) + \sqrt{1-\alpha_t}\,\epsilon_t \\ &= \sqrt{\alpha_t \alpha_{t-1}}\, x_{t-2} + \underbrace{\sqrt{\alpha_t(1-\alpha_{t-1})}\,\epsilon_{t-1} + \sqrt{1-\alpha_t}\,\epsilon_t}_{\text{sum of independent Gaussians}}. \end{aligned}$$

The two noise terms are independent zero-mean Gaussians, so their sum is Gaussian with variance $\alpha_t(1-\alpha_{t-1}) + (1-\alpha_t) = 1 - \alpha_t\alpha_{t-1}$. Repeating the argument down to x_0 gives, with $\bar\alpha_t = \prod_{s=1}^{t} \alpha_s$,

$$q(x_t \mid x_0) = \mathcal{N}\big(\sqrt{\bar\alpha_t}\, x_0,\ (1-\bar\alpha_t) I\big) \quad \Longleftrightarrow \quad x_t = \sqrt{\bar\alpha_t}\, x_0 + \sqrt{1-\bar\alpha_t}\,\epsilon, \qquad \epsilon \sim \mathcal{N}(0, I).$$

Here $\bar{\alpha}_t$ is the fraction of signal variance that survives to step t, and ϵ is a single standard Gaussian vector. This formula is what makes training cheap: we can jump straight to any noise level without simulating the earlier steps.

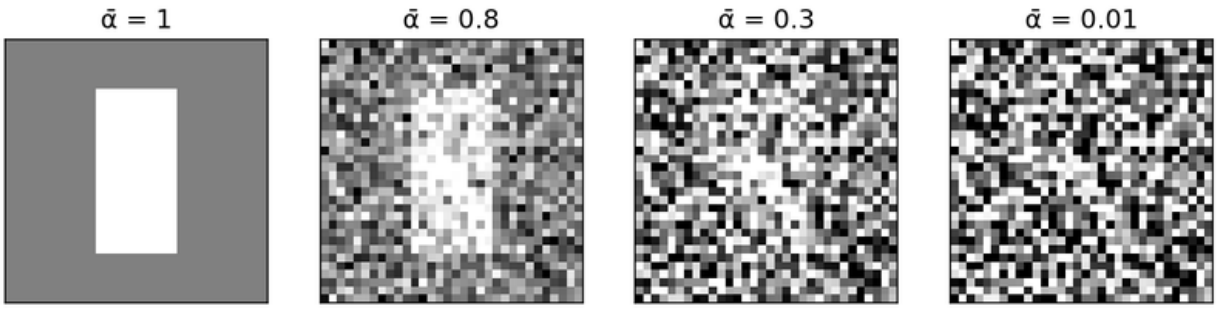


Figure 1 — The closed-form forward process at several signal levels. A shared noise realization makes the effect visible; training draws fresh noise for every example and every step.

Noise schedules and the signal-to-noise ratio

The **signal-to-noise ratio** at step t is

$$\mathrm{SNR}(t) = \frac{\bar{\alpha}_t}{1 - \bar{\alpha}_t},$$

the ratio of signal variance to noise variance (for unit-variance data). It decreases monotonically from very large at $t = 1$ to nearly zero at $t = T$. Two requirements constrain the schedule: $\bar{\alpha}_T$ must be close to 0, so that x_T is practically a standard Gaussian (the sampler starts from $\mathcal{N}(0, I)$); and the steps in between should spend the training budget on noise levels where the network has something useful to learn.

DDPM [3] used $T = 1000$ and a **linear schedule** β_t from 10^{-4} to 0.02. Nichol and Dhariwal [19] observed that this destroys information quickly: for a large part of the chain the image is already nearly pure noise. Their **cosine schedule** defines $\bar{\alpha}_t = f(t)/f(0)$ with $f(t) = \cos^2\!\left(\frac{t/T + s}{1+s} \cdot \frac{\pi}{2}\right)$ and a small offset $s = 0.008$, and clips each β_t at 0.999. Figure 2 compares the two. Karras et al. [20] later argued that the schedule, the network's input/output scaling and the distribution of training noise levels are separate design choices that are best tuned independently.

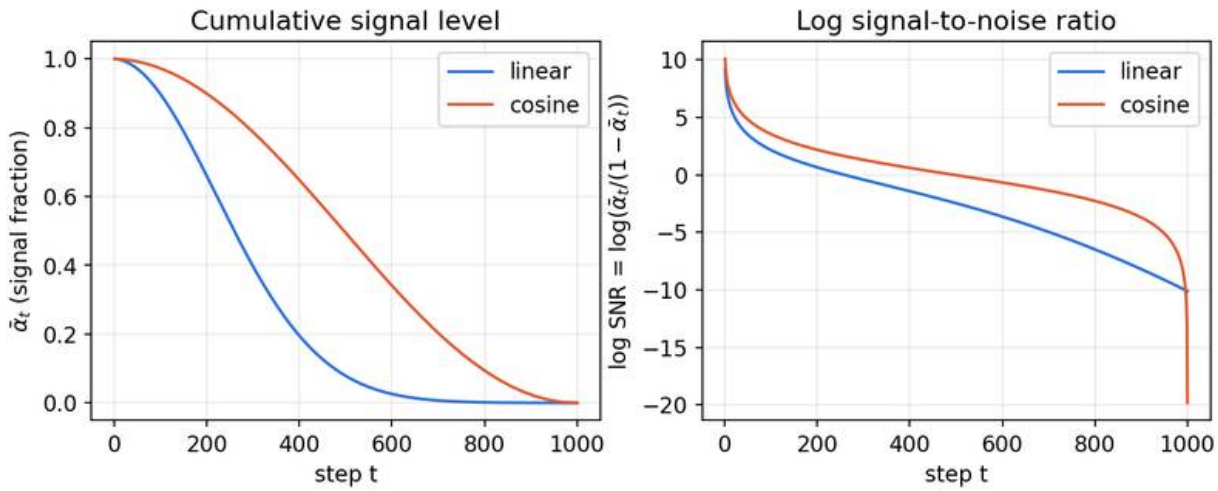


Figure 2 — Linear versus cosine schedule for $T = 1000$. The cosine schedule keeps more signal for longer and spreads the log-SNR more evenly over the steps.

The reverse process and the variational bound

Plain version. To generate, we need the reverse step: "given a noisy example, what did the slightly less noisy example look like?" We cannot compute this directly for real data, but we *can* compute it if we also know the clean example — and that turns out to give a simple training loss.

The model and the bound

The generative model is a reverse Markov chain that starts at $p(x_T) = \mathcal{N}(0, I)$ and uses learned Gaussian transitions

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}\big(\mu_\theta(x_t, t),\ \sigma_t^2 I\big),$$

where μ_θ is a neural network (with parameters θ) and σ_t^2 is a variance we will choose. This is a latent-variable model with latents $x_{1:T}$, exactly like a deep VAE whose encoder q is fixed rather than learned (see ML 06). The same Jensen-inequality argument gives an evidence lower bound. Following [3], [15], the negative bound can be rearranged into a sum of KL divergences:

$$-\log p_\theta(x_0) \ \le\ \mathbb{E}_q\Big[\underbrace{D_{\mathrm{KL}}(q(x_T \mid x_0)\,\|\,p(x_T))}_{L_T} + \sum_{t=2}^{T} \underbrace{D_{\mathrm{KL}}(q(x_{t-1} \mid x_t, x_0)\,\|\,p_\theta(x_{t-1} \mid x_t))}_{L_{t-1}} \underbrace{-\log p_\theta(x_0 \mid x_1)}_{L_0}\Big].$$

D_{KL} is the Kullback–Leibler divergence. L_T has no learnable parameters (it is small when $\bar{\alpha}_T \approx 0$); L_0 is a decoder term for the last step; and each L_{t-1} compares the model's reverse step with the **forward posterior** $q(x_{t-1} \mid x_t, x_0)$ — the reverse step that would be exact if the clean example x_0 were known.

The forward posterior is Gaussian

By Bayes' rule, $q(x_{t-1} \mid x_t, x_0) \propto q(x_t \mid x_{t-1})\, q(x_{t-1} \mid x_0)$. Both factors are Gaussian in x_{t-1}, so their product is Gaussian. Collecting the quadratic and linear terms in x_{t-1} (completing the square) gives

$$q(x_{t-1} \mid x_t, x_0) = \mathcal{N}\big(\tilde{\mu}_t(x_t, x_0),\ \tilde{\beta}_t I\big),$$

$$\tilde{\mu}_t = \frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1-\bar{\alpha}_t}\, x_0 + \frac{\sqrt{\alpha_t}\,(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\, x_t, \qquad \tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\,\beta_t.$$

The posterior mean is a weighted average of the clean example and the current noisy one; the posterior variance $\tilde{\beta}_t$ is slightly smaller than β_t. You do not have to trust the algebra: the snippet below samples the forward chain and fits a regression of x_{t-1} on x_t.

```
import numpy as np

rng = np.random.default_rng(0)
beta = np.linspace(1e-4, 0.02, 1000)
alpha = 1 - beta
abar = np.cumprod(alpha)
t, x0, n = 300, 1.3, 400_000          # check q(x_{t-1} | x_t, x_0) at step t

# Sample the forward chain: x_{t-1} from q(x_{t-1}|x_0), then one more step.
x_prev = np.sqrt(abar[t-1])*x0 + np.sqrt(1-abar[t-1])*rng.normal(size=n)
x_t = np.sqrt(alpha[t])*x_prev + np.sqrt(beta[t])*rng.normal(size=n)

# Empirical regression of x_{t-1} on x_t (the posterior mean is linear in x_t).
slope, intercept = np.polyfit(x_t, x_prev, 1)
resid_var = np.var(x_prev - (slope*x_t + intercept))

# Closed-form posterior from the derivation.
c_x0 = np.sqrt(abar[t-1])*beta[t] / (1-abar[t])
c_xt = np.sqrt(alpha[t])*(1-abar[t-1]) / (1-abar[t])
var = beta[t]*(1-abar[t-1]) / (1-abar[t])
print(f"slope     {slope:.4f}  vs  {c_xt:.4f}")
print(f"intercept {intercept:.4f}  vs  {c_x0*x0:.4f}")
print(f"variance  {resid_var:.6f}  vs  {var:.6f}")
```

The output matches to three or four decimals (slope 0.9934 vs 0.9930, variance 0.006050 vs 0.006052); the small gaps are Monte Carlo error.

From the bound to the simplified loss

Now use the forward formula backwards: $x_0 = (x_t - \sqrt{1-\bar{\alpha}_t}\,\epsilon)/\sqrt{\bar{\alpha}_t}$. Substituting into $\tilde{\mu}_t$ and simplifying with $\bar{\alpha}_t = \alpha_t \bar{\alpha}_{t-1}$,

$$\tilde{\mu}_t = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\,\epsilon\right).$$

At sampling time the network sees x_t but not ϵ. So DDPM parameterizes the model mean in exactly the same form, with the unknown noise replaced by a prediction $\epsilon_\theta(x_t, t)$:

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right).$$

The KL between two Gaussians with the same fixed covariance $\sigma_t^2 I$ is a scaled squared distance between their means, so

$$L_{t-1} = \frac{1}{2\sigma_t^2} \|\tilde{\mu}_t - \mu_\theta\|^2 + C = \frac{\beta_t^2}{2\sigma_t^2 \, \alpha_t \, (1-\bar{\alpha}_t)} \, \|\epsilon - \epsilon_\theta(x_t, t)\|^2 + C,$$

where C does not depend on θ. Every term of the bound is therefore a **weighted noise-prediction error**. Ho et al. [3] found that dropping the weight, and sampling t uniformly, trains better:

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{x_0,\, t\sim \mathcal{U}\{1,\dots,T\},\, \epsilon\sim\mathcal{N}(0,I)} \left\| \epsilon - \epsilon_\theta(\sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon,\ t) \right\|^2.$$

Read it as an algorithm: pick a training example, pick a random step, pick random noise, build x_t in one line, and regress the noise. The time t is fed to the network (usually through a sinusoidal embedding) because the same pixel values mean different things at different noise levels.

Two points are easy to get wrong. First, $\mathcal{L}_{\text{simple}}$ is *not* the likelihood bound: removing the weights relatively down-weights the low-noise steps, which helps sample quality but hurts the reported log-likelihood. Improved DDPM [19] recovered good likelihoods by learning the variances σ_t^2 (as an interpolation between β_t and $\tilde{\beta}_t$) with a hybrid loss. Second, the same network also yields a **clean estimate**,

$$\hat{x}_0(x_t, t) = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\, \epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}},$$

which every modern sampler uses internally. When $\bar{\alpha}_t$ is tiny, the division amplifies prediction errors, so $\hat{x}_0$ at very high noise is a blurry average, not a sample.

Noise, clean data, velocity and the score

Plain version. Predicting the noise, predicting the clean image and predicting "which direction makes this example more typical" are three ways to express the same knowledge. They differ in scaling, and scaling changes how the training loss weights different noise levels.

Three parameterizations

Because $x_t = \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$ is a fixed linear relation, a network can output any one of

- the noise ϵ (DDPM [3]),
- the clean example x_0,

- the **velocity** $v = \sqrt{\bar{\alpha}_t}\,\epsilon - \sqrt{1-\bar{\alpha}_t}\,x_0$, introduced by Salimans and Ho [22] for distillation, which stays well scaled at both ends of the schedule.

Given x_t and t, each converts to the others exactly. However, an unweighted MSE on ϵ, on x_0 or on v weights the noise levels differently, so the choice affects training. Karras et al. [20] made this explicit with input/output "preconditioning" coefficients that keep the network's inputs and targets at unit variance for every noise level. Note that this diffusion "velocity" is a reparameterization of signal and noise; it is related to, but not identical with, the transport velocity of flow matching later in this chapter.

Exact conversion does not make finite networks equivalent

For the variance-preserving path, write $a = \sqrt{\bar{\alpha}_t}$ and $b = \sqrt{1-\bar{\alpha}_t}$. Thus $a^2 + b^2 = 1$ and $x_t = ax_0 + b\epsilon$. The diffusion velocity $v_{\mathrm{VP}} = a\epsilon - bx_0$ completes a rotation:

$$x_0 = ax_t - bv_{\mathrm{VP}}, \qquad \epsilon = bx_t + av_{\mathrm{VP}},$$
$$\hat{x}_0^{(\epsilon)} = (x_t - b\hat{\epsilon})/a, \quad \hat{\epsilon}^{(x)} = (x_t - a\hat{x}_0)/b.$$

Here a, b are known schedule coefficients; v_{VP} is the target used in [22]; hats denote estimates; and superscripts identify the network's direct output. The last two conversions require nonzero divisors. Their instability near an endpoint is a numerical and statistical issue, even though the algebra is exact in the interior.

A noise error δ_ϵ becomes a clean-image error $-(b/a)\delta_\epsilon$. Therefore, for corresponding predictions at a fixed noise level,

$$\|\hat{x}_0 - x_0\|^2 = \frac{b^2}{a^2}\|\hat{\epsilon} - \epsilon\|^2 = \frac{1}{\mathrm{SNR}(t)}\|\hat{\epsilon} - \epsilon\|^2, \qquad \|\hat{x}_0^{(v)} - x_0\|^2 = b^2\|\hat{v}_{\mathrm{VP}} - v_{\mathrm{VP}}\|^2.$$

$\mathrm{SNR}(t) = a^2/b^2$ is the signal-to-noise ratio; the second identity follows from the rotation above. Equal unweighted losses consequently emphasize different times. Changing the target without recording time sampling and loss weighting confounds an experiment [20], [22].

There is also a capacity issue. If a network directly outputs a clean image, its final representation can discard nuisance noise. A network directly outputting noise may need to preserve more coordinates of the noisy input, or use a skip connection that supplies those coordinates. Converting one network's output afterward does not make its architecture identical to a network trained to output another target. This distinction becomes especially relevant for large pixel patches and high-dimensional latents; the JiT discussion below supplies recent evidence [42]. It does not imply that noise prediction generally fails: the earlier DDPM examples and dense image denoisers are counterexamples.

```
import numpy as np

rng = np.random.default_rng(7)
x0 = rng.normal(size=32)
eps = rng.normal(size=32)
```

```
a, b = np.sqrt(.2), np.sqrt(.8)
xt = a*x0 + b*eps
v_vp = a*eps - b*x0
assert np.allclose(a*xt - b*v_vp, x0)
assert np.allclose(b*xt + a*v_vp, eps)

# A deliberately imperfect noise estimate: compare equivalent losses.
eps_hat = eps + .03*rng.normal(size=32)
x_hat = (xt - b*eps_hat)/a
noise_mse = np.mean((eps_hat - eps)**2)
clean_mse = np.mean((x_hat - x0)**2)
assert np.allclose(clean_mse/noise_mse, b*b/(a*a))
print("Clean/noise MSE ratio:", round(clean_mse/noise_mse, 3))

# A straight flow uses a different path and a different velocity.
t = .35
zt = (1-t)*eps + t*x0
u = x0 - eps
x_from_flow = zt + (1-t)*u
u_from_clean = (x0 - zt)/(1-t)
assert np.allclose(x_from_flow, x0)
assert np.allclose(u_from_clean, u)
print("VP velocity equals straight-flow target:", np.allclose(v_vp, u))
```

The example prints a loss ratio of 4.0 and `False` for the two velocity targets. Both use the letter v in parts of the literature; a checkpoint's scheduler must identify the actual path, normalization and time direction.

Interactive demonstration (online): open the online edition of this tutorial to use this demonstration.Open demonstration

What is a score, and why does denoising estimate it?

The **score** of a density p is $\nabla_x \log p(x)$: a vector field that points toward higher probability. If you know the score, you can sample with **Langevin dynamics**: repeatedly step a little along the score and add a little noise. Song and Ermon [17] trained "noise-conditional score networks" at many noise levels and sampled with annealed Langevin dynamics — a parallel route to diffusion that turned out to be the same idea.

The bridge is **denoising score matching** (Vincent [16]). For Gaussian corruption, the best possible noise predictor (in mean squared error) is a conditional expectation, $\epsilon^*(x_t, t) = \mathbb{E}[\epsilon \mid x_t]$, and **Tweedie's formula** [40] links it to the score of the noisy marginal p_t:

$$\nabla_{x_t} \log p_t(x_t) = -\frac{\mathbb{E}[\epsilon \mid x_t]}{\sqrt{1-\bar{\alpha}_t}} \quad \Longrightarrow \quad s_\theta(x_t, t) \approx -\frac{\epsilon_\theta(x_t, t)}{\sqrt{1-\bar{\alpha}_t}},$$

with s_θ the learned score. The same formula in "clean" units reads $\mathbb{E}[x_0 \mid x_t] = (x_t + (1-\bar{\alpha}_t)\nabla \log p_t(x_t))/\sqrt{\bar{\alpha}_t}$. Two warnings: this is the score of the *noisy marginal distribution* at level t,

not the gradient for any single training image; and it holds for the optimal predictor — a trained network approximates it.

You can verify Tweedie's formula where everything is known: one-dimensional Gaussian data, for which the best noise predictor is linear in x_t.

```
import numpy as np

rng = np.random.default_rng(0)
s, abar, n = 0.7, 0.4, 500_000     # data x0 ~ N(0, s^2); one noise level
x0 = s*rng.normal(size=n)
eps = rng.normal(size=n)
xt = np.sqrt(abar)*x0 + np.sqrt(1-abar)*eps

# "Train" the best linear noise predictor eps_hat = w * x_t by least squares.
w = (xt @ eps) / (xt @ xt)
score_from_eps = -w / np.sqrt(1-abar)       # s(x_t) = -eps_hat / sqrt(1-abar)

# Exact: x_t ~ N(0, abar s^2 + 1 - abar), so grad log p_t(x) = -x / var.
exact = -1 / (abar*s**2 + 1 - abar)
print(f"score slope from noise predictor: {score_from_eps:.4f}")
print(f"exact score slope:                {exact:.4f}")
```

It prints -1.2560 versus -1.2563: a regression on noise recovers the score.

The SDE view

Song et al. [4] took the step count to infinity. With continuous time $t \in [0,1]$ and a rate $\beta(t)$, the VP forward process becomes the **stochastic differential equation (SDE)**

$$dx = -\tfrac{1}{2}\beta(t)\,x\,dt + \sqrt{\beta(t)}\,dw,$$

where w is a standard Wiener process (Brownian motion). A classical result says that this process run backwards in time is again an SDE, and that the only unknown in it is the score:

$$dx = \Big[-\tfrac{1}{2}\beta(t)\,x - \beta(t)\,\nabla_x \log p_t(x)\Big]dt + \sqrt{\beta(t)}\,d\bar{w},$$

with dt negative and $\bar{w}$ a reverse-time Wiener process. Moreover, there is a deterministic **probability-flow ODE** whose solutions have the same marginal distributions p_t at every time:

$$\frac{dx}{dt} = -\tfrac{1}{2}\beta(t)\Big[x + \nabla_x \log p_t(x)\Big].$$

This unifies the field. DDPM's ancestral sampler is a discretization of the reverse SDE; NCSN [17] corresponds to a "variance exploding" SDE; and the ODE turns a diffusion model into a continuous normalizing flow, so exact log-likelihoods can be computed with the instantaneous change-of-variables formula [4]. The ODE also explains why deterministic samplers such as DDIM work, and why general-purpose ODE solvers can be applied.

Sampling is a reverse process

Plain version. A loss evaluated at one random noise level is not a generator. To generate, you must chain many reverse steps with a consistent schedule, starting from pure noise — and you can choose how many steps and how much fresh randomness to inject.

DDPM ancestral sampling

DDPM samples by following the learned reverse chain:

$$x_{t-1} = \mu_\theta(x_t, t) + \sigma_t z, \qquad z \sim \mathcal{N}(0, I), \qquad \sigma_t^2 \in \{\beta_t,\ \tilde{\beta}_t\},$$

with no noise added at the final step. Ho et al. [3] report similar results for $\sigma_t^2 = \beta_t$ and $\sigma_t^2 = \tilde{\beta}_t$; learned variances [19] help most when using fewer steps.

Code: DDPM on a two-dimensional distribution

This complete CPU example (about 10 seconds) uses two Gaussian clusters centered at $x = \pm 1.5$ with standard deviation 0.25. It contains every operation of an image model — schedule, random-time training, noise prediction, reverse mean — without an image-scale budget. With 100 steps and this schedule, almost no signal survives to $t = T$.

```
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0)
torch.set_num_threads(1)
T = 100
beta = torch.linspace(.001, .15, T)
alpha = 1-beta
abar = alpha.cumprod(0)
previous = torch.cat([torch.ones(1), abar[:-1]])
post_var = beta * (1-previous) / (1-abar)
net = nn.Sequential(nn.Linear(3, 64), nn.SiLU(),
                    nn.Linear(64, 64), nn.SiLU(), nn.Linear(64, 2))
opt = torch.optim.Adam(net.parameters(), lr=.002)
def data(n):
    side = torch.randint(2, (n, 1))*2-1
    return torch.cat([side*1.5, torch.zeros(n, 1)], 1) + .25*torch.randn(n, 2)
for step in range(3000):
    x0 = data(128)
    t = torch.randint(T, (len(x0),))
    eps = torch.randn_like(x0)
    a = abar[t, None]
    xt = a.sqrt()*x0 + (1-a).sqrt()*eps
    predicted = net(torch.cat([xt, t[:, None].float()/(T-1)], 1))
    loss = F.mse_loss(predicted, eps)
```

```
    opt.zero_grad(); loss.backward(); opt.step()
net.eval()
with torch.no_grad():
    x = torch.randn(512, 2)
    for t in reversed(range(T)):
        time = torch.full((len(x), 1), t/(T-1))
        eps = net(torch.cat([x, time], 1))
        mean = (x-beta[t]/(1-abar[t]).sqrt()*eps)/alpha[t].sqrt()
        x = mean + post_var[t].sqrt()*torch.randn_like(x) if t else mean
    assert torch.isfinite(x).all()
    print("terminal signal:", round(abar[-1].item(), 6))
    left = x[x[:, 0] < 0]
    print("fraction in left cluster:", round((x[:, 0] < 0).float().mean().item(), 3))
    print("left cluster: mean x", round(left[:, 0].mean().item(), 2),
          "std", round(left[:, 0].std().item(), 2), "(data: -1.5 and 0.25)")
```

On our machine it prints a terminal signal of 0.000349, a left-cluster fraction of 0.441, and a left-cluster mean and standard deviation of -1.47 and 0.25. The cluster *shapes* are right; the 44/56 split shows the small network has not learned the mode weights exactly. This is why a single summary number is never enough: always plot samples, and check mode coverage, within-mode spread and outliers against fresh data.

DDIM: same training, different sampler

Song, Meng and Ermon [5] noticed that $\mathcal{L}_{\text{simple}}$ only depends on the marginals $q(x_t \mid x_0)$, not on the Markov chain that produced them. Many non-Markovian forward processes share those marginals, and each gives a different reverse sampler — all usable with the *same* trained ϵ_θ. The **DDIM** update, for any decreasing subsequence of steps $\tau_1 > \tau_2 > \cdots$, is

$$x_{\tau_{i+1}} = \sqrt{\bar\alpha_{\tau_{i+1}}}\,\hat{x}_0 + \sqrt{1-\bar\alpha_{\tau_{i+1}}-\sigma^2}\,\epsilon_\theta(x_{\tau_i},\tau_i) + \sigma z, \qquad \sigma = \eta\sqrt{\frac{1-\bar\alpha_{\tau_{i+1}}}{1-\bar\alpha_{\tau_i}}\Big(1-\frac{\bar\alpha_{\tau_i}}{\bar\alpha_{\tau_{i+1}}}\Big)},$$

where $\hat{x}_0$ is the clean estimate at x_{τ_i} and $\eta \geq 0$ controls stochasticity. Read it as: "estimate the clean example, then re-noise it to the next, lower noise level using the predicted noise direction." With $\eta = 1$ and consecutive steps, this recovers DDPM's posterior variance; with $\eta = 0$ the sampler is **deterministic** and is a discretization of the probability-flow ODE. Determinism has practical consequences: the same starting noise always gives the same output, so you can interpolate in noise space or invert a real image to its noise (DDIM inversion), which is the basis of many editing methods.

The following standalone example repeats the same small training setup so it can run independently. When experimenting in one session, reuse the already trained `net`, `abar` and `T` instead:

```
import torch
from torch import nn
from torch.nn import functional as F
```

```
torch.manual_seed(0)
torch.set_num_threads(1)
T = 100
beta = torch.linspace(.001, .15, T)
alpha = 1-beta
abar = alpha.cumprod(0)
previous = torch.cat([torch.ones(1), abar[:-1]])
post_var = beta * (1-previous) / (1-abar)
net = nn.Sequential(nn.Linear(3, 64), nn.SiLU(),
                    nn.Linear(64, 64), nn.SiLU(), nn.Linear(64, 2))
opt = torch.optim.Adam(net.parameters(), lr=.002)
def data(n):
    side = torch.randint(2, (n, 1))*2-1
    return torch.cat([side*1.5, torch.zeros(n, 1)], 1) + .25*torch.randn(n, 2)
for step in range(3000):
    x0 = data(128)
    t = torch.randint(T, (len(x0),))
    eps = torch.randn_like(x0)
    a = abar[t, None]
    xt = a.sqrt()*x0 + (1-a).sqrt()*eps
    predicted = net(torch.cat([xt, t[:, None].float()/(T-1)], 1))
    loss = F.mse_loss(predicted, eps)
    opt.zero_grad(); loss.backward(); opt.step()
net.eval()

def ddim_sample(n, steps, eta=0.0, seed=1):
    g = torch.Generator().manual_seed(seed)
    taus = torch.linspace(T-1, 0, steps).round().long()
    x = torch.randn(n, 2, generator=g)
    with torch.no_grad():
        for i, t in enumerate(taus):
            time = torch.full((n, 1), t.item()/(T-1))
            eps = net(torch.cat([x, time], 1))
            a_t = abar[t]
            a_prev = abar[taus[i+1]] if i+1 < steps else torch.tensor(1.)
            x0_hat = (x - (1-a_t).sqrt()*eps) / a_t.sqrt()
            sigma = eta*((1-a_prev)/(1-a_t)*(1-a_t/a_prev)).sqrt()
            x = (a_prev.sqrt()*x0_hat + (1-a_prev-sigma**2).sqrt()*eps
                 + sigma*torch.randn(n, 2, generator=g))
    return x

for steps in [5, 20, 100]:
    s = ddim_sample(2000, steps)
    left = s[s[:, 0] < 0]
    print(f"{steps:3d} steps: left fraction {(s[:, 0] < 0).float().mean():.3f}, "
          f"left-cluster mean x {left[:, 0].mean():.2f}, std {left[:, 0].std():.2f}")
```

The output shows the typical pattern. With 5 steps the left cluster is centered at -1.11 with standard deviation 0.39 — the samples are pulled toward the middle and smeared, because large steps follow a curved trajectory with straight lines. With 20 steps it is -1.37 and 0.30; with 100 steps -1.42 and 0.24, close to the data. No retraining was needed to change the step count.

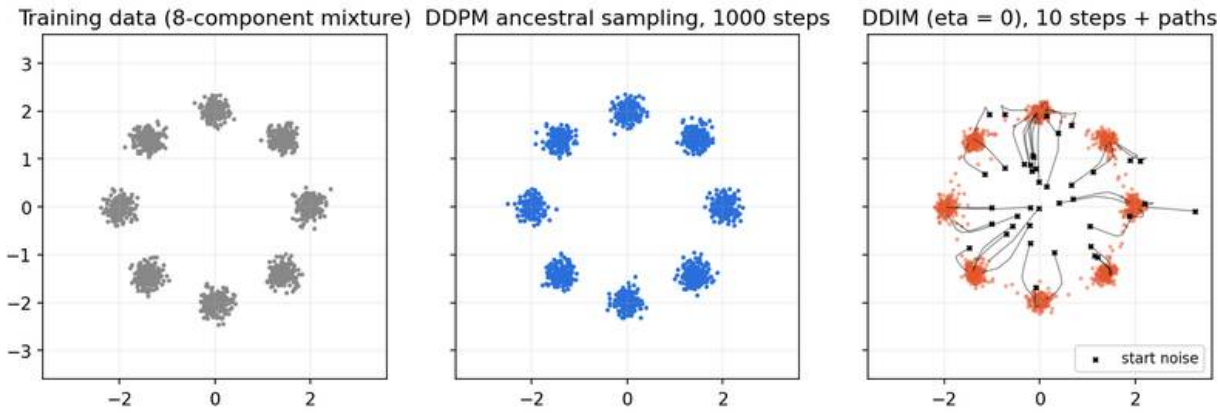


Figure 3 — Samplers driven by the exact denoiser of an 8-component mixture, so only the sampling rule differs. DDPM with 1000 steps reproduces the data; deterministic DDIM with 10 steps lands near the modes but with slightly wider, smeared clusters. Black lines are DDIM trajectories from starting noise (×).

Faster solvers

Once sampling is "solve an ODE," numerical analysis applies. Karras et al. (EDM) [20] used a second-order Heun solver with a carefully chosen spacing of noise levels and showed that sampler, schedule, preconditioning and training noise distribution can be tuned separately. DPM-Solver [21] exploits the semi-linear structure of the diffusion ODE — the linear part can be integrated exactly — to reach good quality in around 10 network evaluations. Stochastic samplers add a little fresh noise at each step; that costs more steps but can correct accumulated errors. Whatever the solver, report the **number of function evaluations (NFE)**: the number of network calls, which, rather than "steps," is the true sampling cost.

Steering generation: conditioning and guidance

Plain version. To generate "a cat" instead of "anything," give the network the condition. Guidance then turns a dial: follow the condition more strictly, at the cost of variety.

Conditioning

A conditional model learns $\epsilon_\theta(x_t, t, c)$ for a condition c. Class labels enter through embeddings added to the time embedding; text enters through **cross-attention** from image features to text-encoder tokens [7]; images (for super-resolution, inpainting, editing) are concatenated to the input channels or fed through extra encoders.

Classifier guidance

Bayes' rule for the noisy marginals gives $\nabla \log p_t(x_t \mid y) = \nabla \log p_t(x_t) + \nabla \log p_t(y \mid x_t)$. Dhariwal and Nichol [18] trained a separate classifier $p_\phi(y \mid x_t)$ on *noisy* images and added its gradient, scaled by w, to an unconditional model. In noise units,

$$\hat{\epsilon} = \epsilon_\theta(x_t, t) - w\sqrt{1-\bar{\alpha}_t}\,\nabla_{x_t} \log p_\phi(y \mid x_t).$$

A scale $w > 1$ samples roughly from $p_t(x_t)\,p_t(y \mid x_t)^w$, a sharpened distribution that trades diversity for fidelity. Together with architecture improvements, this was the work that showed diffusion models beating GANs on ImageNet [35] sample quality [18]. Its drawback is the extra classifier, which must be trained on noisy inputs for every noise level.

Classifier-free guidance (CFG)

Ho and Salimans [6] removed the classifier. During training, the condition is replaced by a "null" token for a random fraction of examples, so one network learns both $\epsilon_\theta(x_t, t, c)$ and $\epsilon_\theta(x_t, t, \emptyset)$. Bayes' rule says the classifier gradient equals the difference of the conditional and unconditional scores, so the implicit-classifier version of the formula above is

$$\epsilon_{\text{guided}} = \epsilon_\theta(x_t, t, \emptyset) + w(\epsilon_\theta(x_t, t, c) - \epsilon_\theta(x_t, t, \emptyset)).$$

In this convention $w = 0$ is unconditional, $w = 1$ is the ordinary conditional model, and $w > 1$ extrapolates past it. (Some papers write the same thing as $(1 + w')\epsilon_c - w'\epsilon_u$, so their "scale" is $w - 1$ in our notation; always check the convention before comparing numbers.)

The snippet below uses the exact noise predictor of a four-blob mixture, so any odd behaviour is due to guidance itself, not to a badly trained network. Class 1 is the two blobs with $x > 0$.

```
import numpy as np

# Data: four Gaussian blobs at the corners of a square; class c is the sign of x.
MU = np.array([[-1.5, -1.5], [-1.5, 1.5], [1.5, -1.5], [1.5, 1.5]])
SIG = 0.2
CLASS = np.array([0, 0, 1, 1])

def eps_exact(x, abar, comps):
    """Exact E[eps | x_t] for a Gaussian mixture restricted to `comps`."""
    m = np.sqrt(abar)*MU[comps]
    var = abar*SIG**2 + 1 - abar
    logw = -((x[:, None] - m[None])**2).sum(-1) / (2*var)
    w = np.exp(logw - logw.max(1, keepdims=True))
    w /= w.sum(1, keepdims=True)
    score = -(w[..., None]*(x[:, None] - m[None])).sum(1) / var
    return -np.sqrt(1-abar)*score

abar = np.cumprod(1 - np.linspace(1e-4, 0.02, 1000))
taus = np.linspace(999, 0, 50).round().astype(int)
everything, class1 = np.arange(4), np.where(CLASS == 1)[0]
print("data: mean distance to its mode", round(SIG*np.sqrt(np.pi/2), 3))
```

```
for w in [0, 1, 3, 8]:
    x = np.random.default_rng(0).normal(size=(4000, 2))
    for i, t in enumerate(taus):                    # deterministic DDIM
        e_u, e_c = eps_exact(x, abar[t], everything), eps_exact(x, abar[t], class1)
        e = e_u + w*(e_c - e_u)                     # classifier-free guidance
        x0 = (x - np.sqrt(1-abar[t])*e) / np.sqrt(abar[t])
        a_prev = abar[taus[i+1]] if i+1 < len(taus) else 1.0
        x = np.sqrt(a_prev)*x0 + np.sqrt(1-a_prev)*e
    dist = np.linalg.norm(x[:, None] - MU[None], axis=-1).min(1)
    print(f"w={w}: class-1 fraction {(x[:, 0] > 0).mean():.3f}, "
          f"mean distance to nearest mode {dist.mean():.3f}")
```

The data's mean distance to its mode is 0.251. With $w = 0$ about half the samples (0.490) are class 1; with $w = 1$ all of them are, at distance 0.225 (50 DDIM steps slightly under-disperse). At $w = 3$ the distance is 0.259 and at $w = 8$ it grows to 0.446: strong guidance does not just "make it more class 1" — it pushes samples *past* the data, a toy version of the over-saturated colours and distorted shapes seen with very high guidance in image models. Figure 4 shows the same effect on the 8-mode mixture.

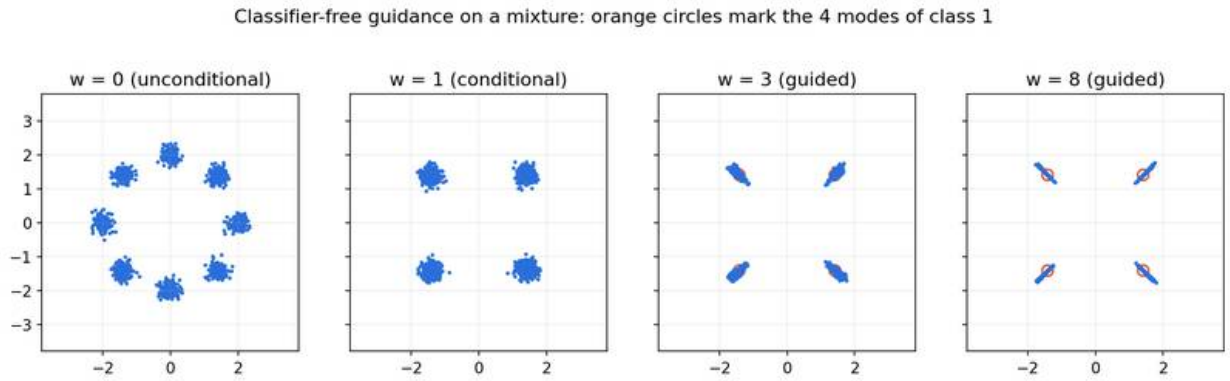


Figure 4 — Exact classifier-free guidance on an 8-mode mixture whose class 1 has 4 modes (orange circles). $w = 1$ already restricts samples to class 1; larger w shrinks clusters along some directions and stretches them along others — the samples no longer follow any data distribution.

Guidance has a compute cost too: each step needs a conditional *and* an unconditional prediction, so a 30-step guided sampler costs 60 network evaluations unless the two are fused. And w is a sampling knob, not a calibrated measure of how well the image matches the prompt.

Latent diffusion and diffusion transformers

Plain version. Pixels are expensive. Compress the image into a smaller grid of features with an autoencoder, run diffusion in that compressed space, then decode. Separately, the denoising network itself can be a transformer instead of a convolutional U-Net [37].

Latent diffusion

Rombach et al. [7] train an autoencoder first: an encoder $z = \mathcal{E}(x)$ maps an $H \times W \times 3$ image to an $h \times w \times c$ latent with a spatial downsampling factor $f = H/h$, and a decoder $\mathcal{D}$ maps back. The autoencoder is trained with reconstruction, perceptual and adversarial losses plus a light regularizer (a small KL term toward a Gaussian, or vector quantization) so that the latent space stays well behaved. Diffusion is then trained on z exactly as before, and generation ends with $\hat{x} = \mathcal{D}(z_0)$. With $f = 8$, a 512×512 image becomes a 64×64 latent: 64 times fewer spatial positions for every network call.

The trade-off is a reconstruction bottleneck: information discarded by the encoder cannot be faithfully recovered from that code, although the decoder may synthesize plausible detail. The latent's scale must also match what the diffusion model was trained on; implementations rescale latents to roughly unit variance. Text conditioning in [7] uses cross-attention layers in the U-Net, the design behind the Stable Diffusion family.

Diffusion transformers

DiT (Peebles and Xie [8]) replaces the U-Net with a Vision Transformer [41] (see ML 04) over latent patches. A latent of size $h \times w$ is cut into $p \times p$ patches, giving hw/p^2 tokens; smaller p means more tokens, more compute and better quality. The timestep and class label condition every block through **adaptive layer norm**: they predict the scale and shift of each LayerNorm, and in the "adaLN-Zero" variant also a gating factor initialized to zero, so each residual block starts as the identity. The paper's main finding is a scaling trend: across model sizes and patch sizes, more transformer compute (GFLOPs) gave consistently lower FID [8].

It helps to keep three choices separate. The **architecture** (U-Net or transformer) is the function approximator. The **objective** (noise, x_0, v, or flow velocity) is what it regresses. The **sampler** (DDPM, DDIM, ODE solver) is how predictions become an image. Stable Diffusion 3 [9] is an example of a new combination: a latent transformer with separate weights for the image and text token streams that attend jointly ("MM-DiT"), trained with a rectified-flow objective and a timestep distribution biased toward intermediate noise levels.

For video, the latent and the attention must cover time as well as space, so compression and attention patterns become the main engineering problem. Melnik et al.'s survey [31] reviews architectures, temporal-consistency mechanisms and evaluation for video diffusion. Per-frame quality is not enough there: object permanence, motion plausibility and long-duration consistency need their own evaluation.

Pixel space, latent space and representation autoencoders

Plain version. The generator needs a place to do its repeated updates. That place can be the image itself, a compact reconstruction code, or features learned by a visual representation model. A latent stage buys a particular computational and statistical trade-off; it is not a prerequisite for generative modeling.

Why pixels become latents and then pixels again

Training images arrive as pixels, while the output must be viewable as pixels. Between these endpoints, an encoder can remove redundancy and produce a grid that is cheaper to update repeatedly [7]. Train the encoder and decoder on reconstruction first, then fit a generative distribution to encoded training images. These are two different tasks: reconstruction preserves a given example; generation learns which encoded examples are plausible.

The training and sampling routes are

$$\text{training: } x \xrightarrow{\mathcal{E}} z_{\text{data}} \xrightarrow{\text{corrupt}} z_t \xrightarrow{\text{prediction loss}} \theta,$$

$$\text{generation: } z_{\text{noise}} \xrightarrow{\text{learned sampler}} \hat{z} \xrightarrow{\mathcal{D}} \hat{x},$$

$$\text{reconstruction check: } x \xrightarrow{\mathcal{E}} \mathcal{E}(x) \xrightarrow{\mathcal{D}} \tilde{x}.$$

Here x is a training image; $\mathcal{E}$ and $\mathcal{D}$ are the encoder and decoder; z_{data} is an encoded image; z_t is its corrupted state; θ parameterizes the generative predictor; $\hat{z}$ is a generated code; $\hat{x}$ is the generated image; and $\tilde{x}$ is a reconstruction. Pure text-to-image sampling does **not** encode an input image first. Editing and image-to-image generation can encode an actual input image and begin from a partially corrupted code.

The main saving occurs because the denoiser or flow predictor runs repeatedly, while encoding training data and decoding a sample occur much less often. A spatial factor f reduces grid positions by f^2, but the number of latent channels also matters. For an image with three channels and a latent with c channels, the scalar-count reduction is $3f^2/c$. It is incorrect to describe the grid reduction as the reduction in all floating-point values or total runtime.

For a transformer, token count and token width must also be distinguished. Full attention stores a matrix with a number of entries proportional to N^2, where N is the token count; feature projections depend on token width as well [41]. A patch size can reduce N in either pixels or latents. None of these counts alone predicts wall-clock speed because kernels, memory traffic, the decoder and conditioning contribute.

```
# Synthetic shape bookkeeping, not a benchmark or a specific checkpoint.
H = W = 512
channels, spatial_factor, patch = 4, 8, 2
pixel_values = H*W*3
h = H//spatial_factor
w = W//spatial_factor
latent_values = h*w*channels
pixel_tokens = (H//patch)*(W//patch)
latent_tokens = (h//patch)*(w//patch)
print("Spatial positions reduction:", (H*W)//(h*w))
print("Scalar values reduction:", pixel_values//latent_values)
print("Dense attention entries reduction:", pixel_tokens**2//latent_tokens**2)
assert pixel_values//latent_values == 48
assert pixel_tokens//latent_tokens == 64
assert pixel_tokens**2//latent_tokens**2 == 4096
```

This constructed example yields 64 times fewer spatial positions, 48 times fewer scalar values and 4096 times fewer attention entries **at the same numerical patch size in each coordinate space**. Changing patch sizes changes the comparison. A pixel transformer with much larger patches can have the same token count as a latent transformer.

The cost is information loss. If two images have the same code, a deterministic decoder cannot recover their different exact details. A decoder may invent plausible texture, but that does not restore information about the original image. The reconstruction check therefore deserves its own evaluation before generative training. Inspect fine lines, lettering and temporal changes, rather than relying only on average image error.

The following deliberately simple encoder averages blocks and the decoder repeats those averages. It demonstrates a bottleneck without pretending to reproduce a learned VAE or RAE.

```
import numpy as np

y, x = np.mgrid[:64, :64]
checker = ((x+y) % 2).astype(float)
smooth = x/63
image = .6*smooth + .4*checker
f = 4
latent = image.reshape(64//f, f, 64//f, f).mean(axis=(1, 3))
reconstruction = np.repeat(np.repeat(latent, f, axis=0), f, axis=1)
print("Synthetic reconstruction MSE:", round(np.mean((image-reconstruction)**2), 6))
assert np.mean((image-reconstruction)**2) > .04
# Recovering the exact latent does not recover the missing checkerboard.
assert np.allclose(reconstruction[0, :f], reconstruction[0, 0])
```

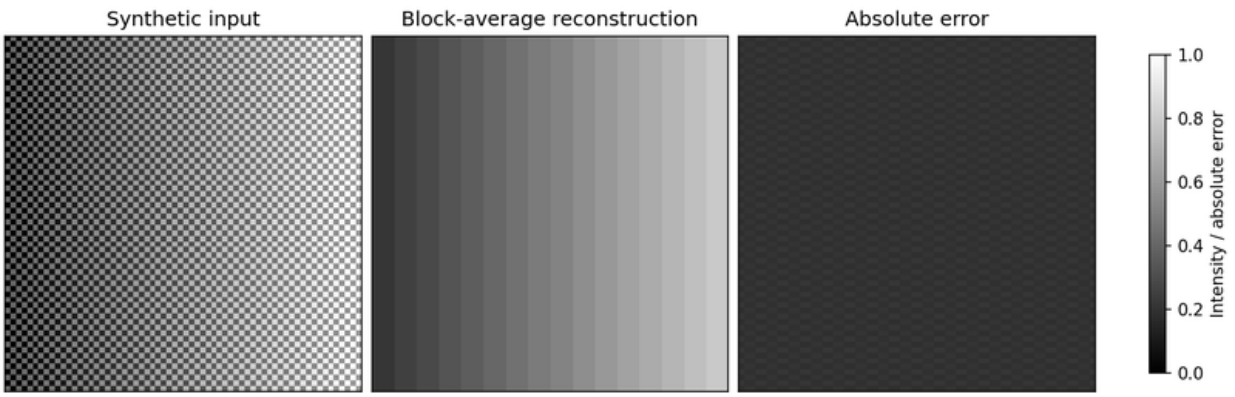


Figure 5 — A latent bottleneck can remove detail before the generative model is trained. This block-average toy is an explanation of information loss, not an evaluation of a learned autoencoder.

For editing, repeatedly encoding and decoding a result can accumulate distortion. Keep intermediate edits in the compatible latent space when the pipeline permits it, and inspect the eventual pixel output. A good unconditional generator may still be a poor reconstruction system. The

claim that an autoencoder sets a ceiling concerns faithfulness to encoded information; it does not mean that a generative decoder can never synthesize new fine texture.

Reconstruction codes versus semantic representation codes

A VAE encoder is trained with a reconstruction objective and a distributional regularizer; its code is designed to support decoding and sampling [7]. A pretrained recognition encoder instead learns useful visual features. These goals can align, but recognition invariance may discard details needed for faithful reconstruction. The decoder and the retained feature structure determine whether the representation is a useful generative space.

Zheng, Ma, Tong and Xie's **Representation Autoencoders (RAEs)** [43] pair a frozen pretrained representation encoder with a trained decoder. Their ICLR 2026 study uses semantic visual features as the space for generative modeling. It examines high-dimensional token features, dimension-dependent noise scheduling, decoder robustness and a shallow wide diffusion head. This changes the representation being generated, rather than simply adding a feature-alignment loss to a conventional latent predictor.

Do not equate "latent" with "smaller in every dimension." A spatially sparse feature grid can have many channels per token. An RAE is also not automatically a variational model: a frozen feature extractor plus a decoder does not by itself define a Gaussian posterior, a KL regularizer or a likelihood bound. Keep reconstruction, representation quality and generative quality as separate measurements.

Tong et al. [44] extend this approach to freeform text-to-image generation in 2026. Their study compares RAE and VAE spaces at larger scales and examines decoder data composition. It finds that dimension-aware noise scheduling remains important while some small-scale architectural and decoder modifications offer little benefit at the tested larger scales. This is a reason to re-test recipes when scale and data change, rather than treating an ImageNet result as a universal design rule.

For a new application, inspect whether the chosen encoder preserves the content that matters: fine text for documents, small defects for inspection, or frame-to-frame differences for video. Evaluate the decoder on held-out real features and on generated features. A decoder trained only on exact training codes can be sensitive to small off-distribution errors in a sampled code. Normalize features using the training recipe, and record the encoder version with the generative checkpoint.

JiT: direct clean-image prediction on pixels

Li and He's **Just image Transformers (JiT)** [42], published at CVPR 2026 after a 2025 preprint, revisit large-patch transformers operating directly on pixels. The network predicts the clean image; sampling derives a flow velocity from that prediction. Their controlled experiments support a manifold-based explanation: a bottleneck can retain structured clean-image information while making direct prediction of high-dimensional noise or velocity difficult. The reported setup uses no pretrained latent tokenizer or auxiliary representation loss. This is evidence for a particular

architecture and training regime, not a proof that pixel-space models always outperform latent generators.

The crucial distinction is **direct network output versus the quantity used in the loss or solver**. For the straight path defined below, use $y_t = (1-t)\epsilon + tx$, with noise at $t = 0$ and clean data at $t = 1$. A network that directly predicts $\hat{x}_\theta(y_t, t)$ can supply

$$\hat{u}_\theta(y_t, t) = \frac{\hat{x}_\theta(y_t, t) - y_t}{1-t}, \qquad \|\hat{u}_\theta - (x-\epsilon)\|^2 = \frac{\|\hat{x}_\theta - x\|^2}{(1-t)^2}.$$

Here x is the clean image; ϵ is the noise draw; y_t is their interpolation; $\hat{x}_\theta$ is the direct prediction; and $\hat{u}_\theta$ is the corresponding transport velocity. The identity holds for $t < 1$ before numerical clipping. An x-output network can thus train with a velocity-space loss. Calling it "clean prediction" does not mean its loss is an unweighted clean-image MSE. JiT's released implementation makes this distinction explicit [42].

Near $t = 1$, division by $1 - t$ needs an endpoint convention and numerical safeguards. For the interior algebra, a fixed clean-image error is amplified increasingly as the endpoint approaches. During sampling, use the checkpoint's time distribution, noise scaling and solver recipe. Changing the direct output label alone does not reproduce the method.

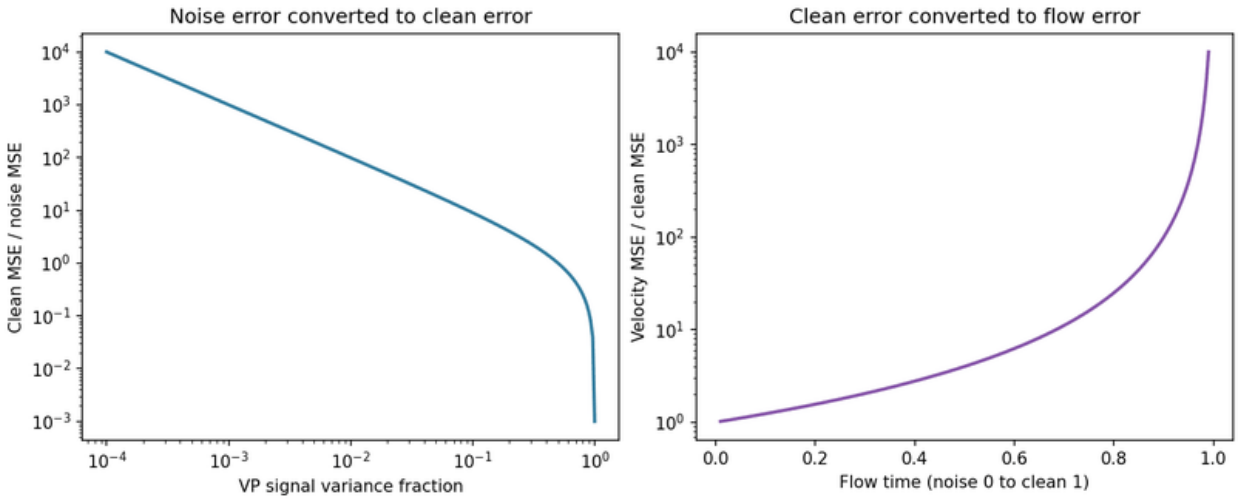


Figure 6 — Equivalent predictions use different units. The left curve shows noise-error amplification at low signal; the right shows velocity-error amplification near the clean endpoint. The curves are algebraic identities, not model performance measurements.

A useful comparison fixes training data, compute, resolution, token count and the sampler, then changes the representation or target one at a time. Compare a pixel transformer, a compact VAE-latent transformer and a representation-latent transformer on reconstruction needs, generated quality and total cost. Record encoder/decoder cost and any representation pretraining. A model that uses a pretrained visual encoder has inherited training even when its generative stage starts from scratch.

Flow matching: learn a transport velocity

Plain version. Forget noise schedules for a moment. Draw a noise point and a data point, connect them by a straight line, and train a network to predict the direction of travel at every point along such lines. To generate, drop a fresh noise point and follow the predicted directions.

Time convention change. In this section time runs from **noise at** $t = 0$ **to data at** $t = 1$, the opposite of the diffusion sections above (where large t meant more noise).

The conditional flow-matching objective

A continuous normalizing flow moves points with an ODE $dx_t/dt = v(x_t, t)$. If the velocity field v is right, it transports the noise distribution $p_0 = \mathcal{N}(0, I)$ to the data distribution p_1. Lipman et al. [10] (and, concurrently, Liu et al. [11] and Albergo and Vanden-Eijnden's stochastic interpolants [34]) showed how to train v without ever simulating the ODE.

Pair noise $z \sim \mathcal{N}(0, I)$ with a data example $x \sim p_1$ and define the straight **conditional path**

$$x_t = (1 - t)\, z + t\, x, \qquad \frac{dx_t}{dt} = u_t(x_t \mid z, x) = x - z.$$

Every pair moves at constant speed in a straight line. The marginal velocity we actually want is the average of these conditional velocities over all pairs that pass through the point:

$$u_t(y) = \mathbb{E}[\, x - z \mid x_t = y \,].$$

We cannot compute this average, but we do not need to. Compare the "ideal" loss with the one we can compute:

$$\mathcal{L}_{\text{FM}} = \mathbb{E}_{t, x_t} \| v_\theta(x_t, t) - u_t(x_t) \|^2, \qquad \mathcal{L}_{\text{CFM}} = \mathbb{E}_{t, z, x} \| v_\theta(x_t, t) - (x - z) \|^2,$$

with $t \sim \mathcal{U}[0, 1]$. Expand both squares. The $\|v_\theta\|^2$ terms are identical. The cross terms are equal because, by the tower rule,

$\mathbb{E}[v_\theta(x_t, t)^\top (x - z)] = \mathbb{E}[v_\theta(x_t, t)^\top \, \mathbb{E}[x - z \mid x_t]] = \mathbb{E}[v_\theta^\top u_t(x_t)]$. The remaining terms do not involve θ. Therefore

$$\nabla_\theta \, \mathcal{L}_{\text{CFM}} = \nabla_\theta \, \mathcal{L}_{\text{FM}},$$

so regressing the per-pair velocity $x - z$ trains the marginal velocity field. This is the same trick as in diffusion: a target that is noisy for any single example has the right conditional mean. Generation solves $dx/dt = v_\theta(x, t)$ from $t = 0$ to $t = 1$ with any ODE solver.

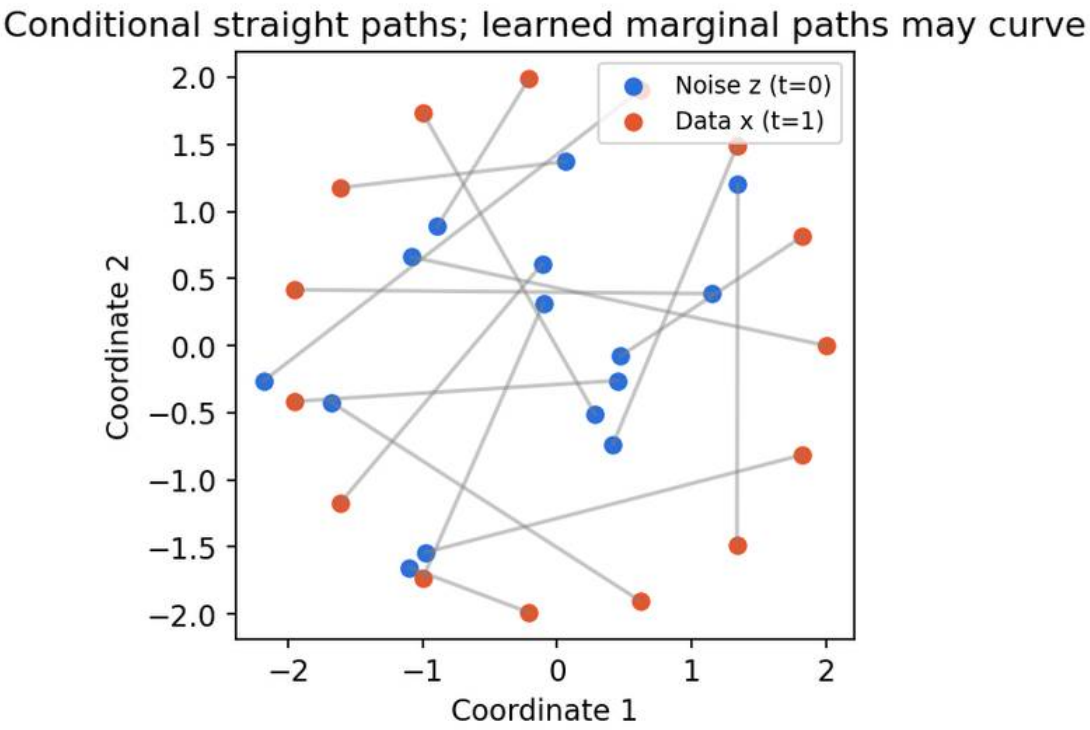


Figure 7 — Conditional training paths are straight. Paths of different pairs cross, so the marginal velocity at a crossing point is an average, and the learned trajectories can curve.

How flow matching relates to diffusion

The linear path is a Gaussian path: given x, $x_t \sim \mathcal{N}(t\,x,\ (1-t)^2 I)$. Writing $\hat{z} = \mathbb{E}[z \mid x_t]$ (a noise prediction) and using $x = (x_t - (1-t)z)/t$,

$$u_t(x_t) = \frac{x_t - \hat{z}}{t}, \qquad \nabla \log p_t(x_t) = -\frac{\hat{z}}{1-t}, \qquad \text{so} \qquad u_t(x_t) = \frac{x_t + (1-t)\,\nabla \log p_t(x_t)}{t}.$$

So a flow-matching velocity, a noise predictor and a score are again interchangeable for this path; what differs is the path (straight lines versus the VP curve), the weighting of noise levels in the loss, and the time convention. Diffusion's probability-flow ODE is itself a flow along a particular Gaussian path. Lipman et al.'s guide [2] and the stochastic-interpolant framework [34] treat both as special cases of one construction. The practical advice is the same as for parameterizations: the targets are related by derivation, not interchangeable by renaming.

Code: flow matching on two clusters

This standalone example learns the same two-cluster distribution as the DDPM snippet with linear conditional paths, then samples with Euler steps of size $1/N$.

```
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0)
torch.set_num_threads(1)
net = nn.Sequential(nn.Linear(3, 64), nn.SiLU(),
                    nn.Linear(64, 64), nn.SiLU(), nn.Linear(64, 2))
opt = torch.optim.Adam(net.parameters(), lr=.002)
for step in range(3000):
    z = torch.randn(128, 2)
    side = torch.randint(2, (128, 1))*2-1
    x = torch.cat([1.5*side, torch.zeros(128, 1)], 1) + .25*torch.randn(128, 2)
    t = torch.rand(128, 1)
    xt = (1-t)*z+t*x
    loss = F.mse_loss(net(torch.cat([xt, t], 1)), x-z)
    opt.zero_grad(); loss.backward(); opt.step()
net.eval()
def euler_sample(n, steps, seed=1):
    x = torch.randn(n, 2, generator=torch.Generator().manual_seed(seed))
    with torch.no_grad():
        for k in range(steps):
            t = torch.full((n, 1), k/steps)
            x = x + net(torch.cat([x, t], 1))/steps
    return x

for steps in [1, 4, 50]:
    s = euler_sample(2000, steps)
    assert torch.isfinite(s).all()
    left = s[s[:, 0] < 0]
    print(f"{steps:2d} Euler steps: left fraction {(s[:, 0] < 0).float().mean():.3f}, "
          f"left-cluster mean x {left[:, 0].mean():.2f}, std {left[:, 0].std():.2f}")
```

It prints, for 1, 4 and 50 steps, left-cluster means of -0.18, -1.20 and -1.45 with standard deviations 0.12, 0.42 and 0.29. One Euler step lands almost at the overall data mean: at $t = 0$ the marginal velocity is $\mathbb{E}[x] - z$, so a single full step sends *every* noise point to $\mathbb{E}[x]$. Straight conditional paths do not make the learned marginal flow straight. Figure 8 shows the same effect with the exact velocity of an 8-mode mixture.

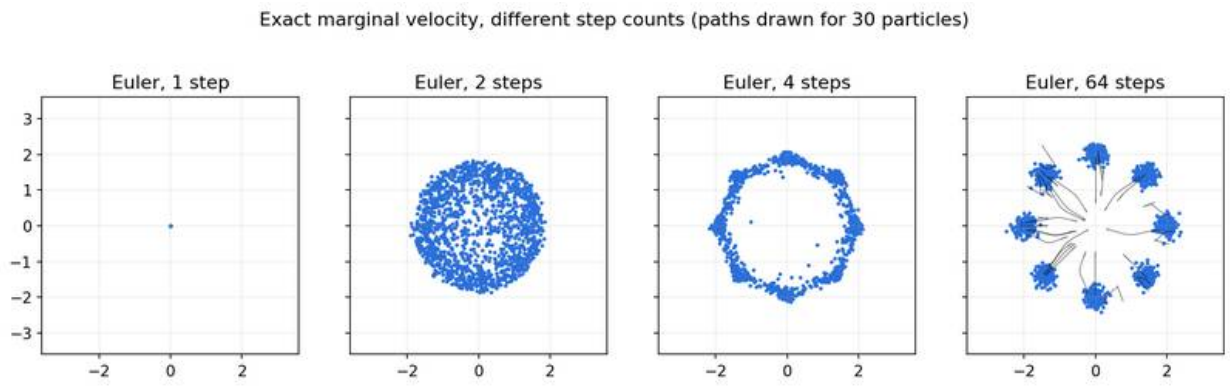


Figure 8 — Even with the exact marginal velocity, Euler sampling needs many steps: one step collapses every sample to the data mean, and the 64-step trajectories (black) bend toward the modes late in the path.

Rectified flow and reflow

Liu, Gong and Liu [11] use the same straight interpolation ("rectified flow") and add **reflow**: after training, generate pairs $(z, \hat{x})$ by solving the learned ODE from each z, then retrain on these *coupled* pairs instead of independent ones. Because the new pairs are already connected by the flow, their straight lines cross less, the learned trajectories become straighter, and fewer Euler steps are needed; a straight enough flow can then be distilled into a one-step model. Stable Diffusion 3 [9] showed that rectified-flow training of a large latent transformer works at text-to-image scale.

Pretrained image and video generators as complete pipelines

Plain version. A pretrained generator is a bundle of compatible components. Loading the denoiser weights alone does not establish the right representation, text conditioning, prediction target or sampling schedule. The image/video boundary also changes what a reconstruction must preserve.

Read the components before running the checkpoint

A practical pipeline combines a text encoder, an optional image/video encoder, a time-conditioned predictor, a sampler and an optional decoder. A separate refinement model may follow the first generator. Each component expects particular tensor shapes and scales. A scheduler declaring `v_prediction` may mean VP velocity; a flow scheduler may expect a derivative along a straight interpolation. Check the training path rather than guessing from a configuration string.

The following verified examples illustrate architectural differences. They are fixed research baselines, rather than a ranking of the latest product releases. The references include their papers or technical reports and official release documentation.

Baseline	Generation representation and predictor	What to inspect in the pretrained pipeline
SDXL [45]	Reconstruction latents with a convolutional denoiser	Text conditioning, latent scaling, optional base/refiner composition and compatible sampling configuration
Stable Diffusion 3 [9]	Latent rectified flow with a joint text–image transformer	Flow time convention, text streams, time sampling and autoencoder compatibility
FLUX.1 [46]	Latent rectified-flow transformer	Exact release variant, guidance behavior, text conditioning, autoencoder and release terms
JiT [42]	Class-conditioned clean-image prediction directly on pixels	Image normalization, patch/ resolution configuration, derived velocity and sampling recipe; this is not a freeform text-to-image checkpoint
HunyuanVideo [47]	Compressed video latents with a flow-matching transformer	Video autoencoder, image/video joint training, temporal dimensions and text-conditioning components
Wan2.1 [48]	Video diffusion transformer with a spatiotemporal autoencoder	Text-to-video versus image-to-video variant, causal video compression, temporal length and decode caching

SDXL [45] documents a larger latent image-generation architecture and optional refinement stage. Stable Diffusion 3 [9] illustrates how the same broad latent pipeline can instead use rectified flow and joint text/image attention. FLUX.1's official model card [46] describes a released rectified-flow transformer; its `.1-dev` variant is guidance-distilled. These facts do not make the checkpoints interchangeable. Ordinary two-pass CFG and a distilled guidance input can have different meanings and costs.

For image-to-image generation, encoding an input and adding partial noise trades input preservation against the ability to change it. The actual noise level is determined by the schedule, not merely by a UI label such as "strength." To compare edits, keep the same input and record the initial time, seed, prompt, guidance and reconstruction baseline. A reconstruction artifact can otherwise be mistaken for an edit requested by the text.

Video adds temporal compression and a consistency problem

Let a video have F frames, height H and width W. A hypothetical encoder compressing time by f_t and each spatial axis by f_s gives a rough latent grid of $(F/f_t) \times (H/f_s) \times (W/f_s)$ positions. Exact shapes depend on padding and special handling of the first frame. After patching, dense attention across all positions grows quadratically with the remaining token count; separately reducing time and space can therefore change cost substantially.

HunyuanVideo's technical report [47] combines a 3D video autoencoder with a flow-matching transformer and staged image/video training. Wan's report [48] explains a causal video VAE with special first-frame handling, spatial and temporal compression, and feature caching during

chunked encoding/decoding. Causality in an autoencoder does not by itself imply that the entire video generator samples autoregressively; the generative attention and sampling procedure must be examined separately.

A frame can look excellent while a clip changes an object's identity, reverses motion unexpectedly or forgets an occluded object. Test reconstruction of held-out clips before generation, then evaluate appearance, motion, prompt correspondence and consistency across frames. Melnik et al. [31] provide the survey map for these video-specific issues. A per-image metric alone cannot diagnose temporal behavior.

A pretrained video generator also does not automatically become a reliable world model. Predicting plausible clips differs from predicting the consequences of a specified action. The world-model chapter develops this distinction through state, actions, rollouts and evaluation. Use the actual conditioning interface and evaluate the task the system claims to solve.

A reproducible first experiment

Start by recording a pipeline manifest: checkpoint revision, encoder/decoder revisions, text encoder, representation shape and scaling, prediction target, time direction, solver, evaluation count, guidance, dtype and seed. Then perform a reconstruction check if an autoencoder is present and generate a fixed small prompt/seed set. Compare one change at a time. Include decoder latency and peak memory when reporting the cost of a latent method.

This chapter's CPU examples test the underlying identities and failure modes without downloading weights. Full-resolution pretrained generation belongs in a separately provisioned experiment using the official release instructions [42], [45]–[48]. Treat an actual measured run, a paper's published result and a toy calculation as three different kinds of evidence.

Few-step and one-step generation

Plain version. If a sampler needs 50 network calls, train a second model that jumps along the trajectory in a few calls — or train one from scratch that is designed to jump.

The main families are:

- **Progressive distillation** [22]: a student learns to match two deterministic teacher steps with one step; repeating the procedure halves the step count each round.
- **Consistency models** [12]: learn a function $f_\theta(x_t, t)$ that maps *any* point on a probability-flow ODE trajectory to that trajectory's starting (clean) point. It must satisfy the boundary condition $f_\theta(x, \epsilon) = x$ at the smallest time ϵ, enforced by writing $f_\theta(x,t) = c_{\text{skip}}(t)\, x + c_{\text{out}}(t)\, F_\theta(x,t)$ with $c_{\text{skip}}(\epsilon) = 1$ and $c_{\text{out}}(\epsilon) = 0$. Training asks outputs at adjacent points of a trajectory to agree. In *consistency distillation*, adjacent points come from one ODE-solver step of a pretrained diffusion model; in *consistency training*, from the same data example noised at two levels, with no teacher. Sampling takes one step, or a few steps of "jump to clean, re-noise, jump again." Improved training recipes [32] simplified continuous-time consistency models and scaled them to 1.5 billion parameters on ImageNet 512×512 with two-step sampling.

- **Adversarial distillation**: ADD [33] combines score distillation from a large pretrained diffusion teacher with an adversarial loss so that a student produces sharp images in one to four steps.
- **Average-velocity models**: MeanFlow [13] learns the *average* velocity $u(x_t, r, t) = \frac{1}{t-r}\int_r^t v(x_s, s)\, ds$ over an interval instead of the instantaneous velocity v. Differentiating $(t-r)\,u = \int_r^t v\, ds$ with respect to t gives the identity $u = v - (t-r)\frac{d}{dt}u$, which supplies a training target without a teacher. One network call over the whole interval then generates a sample; the paper reports an FID of 3.43 with one evaluation on ImageNet 256 × 256, trained from scratch [13].

A small step count is a learned capability, not a free consequence of a nicer equation. When comparing, count NFE including guidance evaluations, the decoder's cost in latent models, resolution, quality *and* diversity — and remember that a one-step network can be larger and slower per call than a multi-step one.

Evaluating generative models

Plain version. "Does it look good?" hides at least three questions: are individual samples realistic (fidelity), does the set of samples cover the variety of the data (diversity), and does the sample match the condition (alignment)? No single number answers all three.

Fréchet Inception Distance

FID (Heusel et al. [23]) embeds real and generated images with an Inception-v3 network [36], fits a Gaussian to each set of features, and computes the Fréchet (2-Wasserstein) distance between the two Gaussians:

$$\mathrm{FID} = \|\mu_r - \mu_g\|^2 + \mathrm{tr}\Big(\Sigma_r + \Sigma_g - 2(\Sigma_r\Sigma_g)^{1/2}\Big),$$

where μ_r, Σ_r and μ_g, Σ_g are the feature mean and covariance of the real and generated sets and tr is the matrix trace. Lower is better. The snippet applies the formula to two-dimensional "features."

```
import numpy as np
from scipy import linalg

def frechet_distance(a, b):
    """Frechet distance between Gaussians fitted to feature sets a and b."""
    mu1, mu2 = a.mean(0), b.mean(0)
    c1, c2 = np.cov(a, rowvar=False), np.cov(b, rowvar=False)
    covmean = linalg.sqrtm(c1 @ c2).real
    return float(((mu1 - mu2)**2).sum() + np.trace(c1 + c2 - 2*covmean))

rng = np.random.default_rng(0)
centers = np.array([[-1.5, 0.], [1.5, 0.]])
real = centers[rng.integers(2, size=5000)] + .25*rng.normal(size=(5000, 2))
good = centers[rng.integers(2, size=5000)] + .25*rng.normal(size=(5000, 2))
```

```
one_mode = centers[[0]] + .25*rng.normal(size=(5000, 2))      # mode collapse
blurry = .6*rng.normal(size=(5000, 2))                         # mass in the gap
print("good     :", round(frechet_distance(real, good), 3))
print("one mode :", round(frechet_distance(real, one_mode), 3))
print("blurry   :", round(frechet_distance(real, blurry), 3))
small = [frechet_distance(real[:50], good[i*50:(i+1)*50]) for i in range(20)]
print("good, 50 samples: mean", round(np.mean(small), 3), "std", round(np.std(small), 3))
```

It prints about 0.001 for a correct sampler, 3.9 for a sampler that only produces the left mode, and 0.976 for a "blurry" sampler that puts its mass between the modes. The last line shows a practical trap: with 50 samples, even a *perfect* sampler scores 0.077 ± 0.073, because FID is biased upward for small sample sizes. Compare FIDs only with the same number of samples, the same reference set and the same feature extractor and preprocessing.

Beyond FID

- **Precision and recall** (Kynkäänniemi et al. [24]) separate fidelity from diversity: estimate the real and generated "manifolds" with k-nearest-neighbour balls in feature space; precision is the fraction of generated samples inside the real manifold, and recall the fraction of real samples inside the generated one. Our "one mode" sampler would have high precision and low recall.
- **Feature extractor matters.** Stein et al. [25] ran a large human study and found that no common metric correlated strongly with human judgments; in particular, the perceptual realism of diffusion models was not reflected in FID. They recommend features from DINOv2-ViT-L/14 [39] instead of Inception, and show that current metrics cannot separate memorization from underfitting or mode shrinkage.
- **CMMD** (Jayasumana et al. [26]) replaces Inception features with CLIP [38] embeddings and the Gaussian assumption with a maximum mean discrepancy using a Gaussian RBF kernel, giving an unbiased estimator that is more sample-efficient.
- **Likelihood.** Through the probability-flow ODE, diffusion models can report exact log-likelihoods (in bits per dimension) [4], [19]. Likelihood measures something different from visual quality and the two can disagree.
- **Conditional models** need alignment checks (does the image show what the prompt says?), usually by human rating or by vision–language models, and memorization checks against the training set.
- **Cost.** Report NFE, guidance evaluations, resolution and wall-clock time on stated hardware.

The evaluation chapter ML 10 returns to sample-size effects and honest comparisons.

Modern view

Surveys and reference texts. The literature is large; these reviews are good maps.

- Yang et al. [1] (*ACM Computing Surveys*, 2023) organize diffusion research into three improvement directions — efficient sampling, improved likelihood estimation, and data with special

structure (manifolds, discrete data, invariances) — and also cover combinations with VAEs, GANs and flows and applications across domains.
- Croitoru et al. [27] (*IEEE TPAMI*, 2023) focus on vision. They identify three generic formulations — DDPMs, noise-conditioned score networks and SDEs — and categorize vision applications such as generation, super-resolution, inpainting, editing, translation and segmentation.
- Cao et al. [28] (*IEEE TKDE*, 2024) take a methods angle: fundamental formulation, algorithmic enhancements (sampling acceleration, objective design, conditioning) and applications from images to biology and healthcare.
- Lipman et al. [2] (2024) is a guide-with-code to flow matching that covers Gaussian and general probability paths, the link to diffusion and extensions to discrete and Riemannian data.
- Lai, Song, Kim, Mitsufuji and Ermon [29] (2025) is a book-length monograph presenting the variational, score-based and flow-based views as three routes to the same time-dependent velocity field, followed by guidance, fast solvers and flow-map (few-step) models.
- Shen et al. [30] (*TMLR*, 2025) survey efficiency at the algorithm level (training, sampling, compression), system level (hardware and serving) and framework level.
- Melnik et al. [31] (*TMLR*, 2024) survey video diffusion: architectures, temporal-consistency mechanisms, datasets and evaluation.

State of the art, 2023–2026. Several trends are visible across these reviews and the primary papers in this chapter. (1) *Transformers in latent space*: DiT-style backbones [8] replaced U-Nets in many large systems, and joint text–image transformers trained with rectified-flow objectives [9] became a standard text-to-image recipe. (2) *Flow matching as the default objective*: straight-path flow matching [10], [11] is now used alongside or instead of noise prediction, with diffusion understood as one choice of probability path [2], [29]. (3) *Fewer evaluations*: distillation [22], [33], consistency models [12], [32] and average-velocity models [13] bring sampling down to one to four network calls with quality approaching multi-step samplers. (4) *Better evaluation*: the community is moving from Inception-based FID toward richer feature spaces, human studies and alignment metrics [25], [26]. (5) *Beyond images*: video, 3-D, audio and scientific data reuse the same machinery, with video's temporal consistency and cost the main open challenges [31].

Representation and prediction, 2025–2026. JiT [42] tests direct clean-image prediction on pixels; RAE [43] tests generation in frozen semantic feature spaces; its text-to-image follow-up [44] examines larger-scale behavior. These studies separate where the model operates from what the network directly predicts. The pretrained examples above [45]–[48] show why component compatibility and temporal reconstruction matter in actual image/video pipelines. Compare these choices under matched compute and data, including representation pretraining.

Open problems. Sampling cost at high resolution and long video duration; principled guidance that does not trade away diversity or produce over-saturation; evaluation metrics that agree with people and detect memorization; understanding generalization (why models produce new samples rather than copies of training data); controllability and editing with guarantees; and data provenance, consent and misuse, which are social rather than purely technical questions. The next chapter, ML 08, covers the main alternative route: autoregressive generation over pixels and tokens, and its hybrids with diffusion.

Key takeaways

- The forward process has a closed form $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$, so training reduces to "pick a random noise level and regress the noise."
- Every term of the variational bound is a weighted noise-prediction error; $\mathcal{L}_{\text{simple}}$ drops the weights, which improves samples but is not the likelihood.
- Noise, clean-image, VP velocity, score and flow velocity convert under a specified path; time direction, loss weighting, endpoint stability and network capacity matter. JiT directly outputs clean pixels while deriving a velocity for training and sampling.
- Sampling is solving a reverse SDE or ODE: DDPM, DDIM and higher-order solvers trade steps for accuracy with the same trained network; count NFE, not "steps."
- Guidance trades diversity for adherence and doubles the per-step cost for CFG; very large weights push samples off the data distribution.
- Pixels, reconstruction latents and RAE features are different generative spaces. Latent compression saves repeated updates but can lose information; pretrained image/video pipelines require compatible encoders, predictors, guidance and samplers.
- Conditional flow matching trains the marginal velocity by regressing per-pair straight-line velocities, but straight conditional paths do not make the learned flow straight — few-step generation needs reflow, distillation, consistency or average-velocity training.
- No single metric measures quality: FID depends on sample size and feature extractor; pair it with precision/recall, human or alignment studies and cost.

Exercises

1. **Recover the clean estimate.** Starting from $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1-\bar{\alpha}_t}\epsilon$, derive $\hat{x}_0$ from a noise prediction. If ϵ_θ has an error δ, how large is the error in $\hat{x}_0$ at $\bar{\alpha}_t = 0.25$ and at $\bar{\alpha}_t = 10^{-4}$? Then derive the straight-flow velocity from a clean prediction and explain why JiT can output x while using a velocity loss.

   Hint

   $\hat{x}_0 = (x_t - \sqrt{1-\bar{\alpha}_t}\,\epsilon_\theta)/\sqrt{\bar{\alpha}_t}$, so the error is $\sqrt{1-\bar{\alpha}_t}\,\delta/\sqrt{\bar{\alpha}_t}$: about $1.7\,\delta$ at $\bar{\alpha}_t = 0.25$ and about $100\,\delta$ at 10^{-4}. At high noise, $\hat{x}_0$ is an average over many plausible images, not one image.

2. **Posterior mean in two forms.** Substitute $x_0 = (x_t - \sqrt{1-\bar{\alpha}_t}\,\epsilon)/\sqrt{\bar{\alpha}_t}$ into $\tilde{\mu}_t$ and show that it equals $\frac{1}{\sqrt{\alpha_t}}(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon)$. Then change `t` in the posterior snippet to 50 and 900 and check that the formula still matches.

   Hint

   Use $\bar{\alpha}_t = \alpha_t\bar{\alpha}_{t-1}$ and $\beta_t = 1-\alpha_t$. The coefficient of x_t becomes $\frac{\beta_t + \alpha_t(1-\bar{\alpha}_{t-1})}{\sqrt{\alpha_t}(1-\bar{\alpha}_t)} = \frac{1}{\sqrt{\alpha_t}}$ because the numerator equals $1-\bar{\alpha}_t$.

3. **Count compute.** A latent text-to-image model samples with 30 DDIM steps and classifier-free guidance using separate conditional and unconditional passes, then runs the decoder once. How many denoiser evaluations are needed? How does the count change for a 4-step distilled model that has guidance "baked in"? For the synthetic shape example, change the latent channels to 16 and the pixel patch size to 16. Recompute scalar counts and attention entries. Which savings remain, and which depend on patching?

Hint

60 denoiser evaluations plus one decoder pass; batching both CFG branches reduces wall-clock time but not the work. A guidance-distilled 4-step model needs 4 evaluations plus the decoder. Compare quality and diversity, not just the counts.

4. **Time convention.** Diffusion sections use "large t = noisy"; the flow-matching section uses "$t = 0$ noise, $t = 1$ data." Reparameterize the flow with $s = 1 - t$. What is dx/ds, and what goes wrong if you only relabel the time input?

Hint

$dx/ds = -v(x, 1 - s)$. If you feed s to a network trained on t without flipping the sign and the time input, you integrate in the wrong direction and move samples away from the data.

5. **Guidance sweep.** In the CFG snippet, add $w \in \{0.5, 1.5, 5\}$ and also record the fraction of class-1 samples that land closer to the wrong-class modes. Plot mean distance to the nearest mode against w. Where does guidance stop helping?

Hint

Already at $w = 1$ the exact conditional model puts every sample in class 1, so larger w cannot improve class accuracy here — it only distorts the clusters. With a learned, imperfect conditional model, moderate $w > 1$ can help adherence, which is why it is used in practice.

6. **Sampler ablation.** Hold the trained flow-matching network fixed. Compare Euler with 1, 2, 5, 20 and 100 steps against a second-order Heun step (predict with Euler, then average the velocities at both ends). Use the same starting noise and measure cluster means and spreads. Does the 2-step Heun sampler beat 4-step Euler at the same number of network calls?

Hint

Heun uses two network calls per step. Plot samples, not only numbers. One Euler step of a model trained for instantaneous velocity does not test a separately trained one-step model such as a consistency or MeanFlow model.

References

1. L. Yang, Z. Zhang, Y. Song, S. Hong, R. Xu, Y. Zhao, W. Zhang, B. Cui and M.-H. Yang, "Diffusion Models: A Comprehensive Survey of Methods and Applications," *ACM Computing Surveys*, 2023. arXiv
2. Y. Lipman, M. Havasi, P. Holderrieth, N. Shaul, M. Le, B. Karrer, R. T. Q. Chen, D. Lopez-Paz, H. Ben-Hamu and I. Gat, "Flow Matching Guide and Code," arXiv:2412.06264, 2024. arXiv
3. J. Ho, A. Jain and P. Abbeel, "Denoising Diffusion Probabilistic Models," *NeurIPS*, 2020. arXiv
4. Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon and B. Poole, "Score-Based Generative Modeling through Stochastic Differential Equations," *ICLR*, 2021. arXiv
5. J. Song, C. Meng and S. Ermon, "Denoising Diffusion Implicit Models," *ICLR*, 2021. arXiv
6. J. Ho and T. Salimans, "Classifier-Free Diffusion Guidance," NeurIPS 2021 Workshop on Deep Generative Models and Downstream Applications; arXiv:2207.12598, 2022. arXiv
7. R. Rombach, A. Blattmann, D. Lorenz, P. Esser and B. Ommer, "High-Resolution Image Synthesis with Latent Diffusion Models," *CVPR*, 2022. arXiv
8. W. Peebles and S. Xie, "Scalable Diffusion Models with Transformers," *ICCV*, 2023. arXiv
9. P. Esser, S. Kulal, A. Blattmann et al., "Scaling Rectified Flow Transformers for High-Resolution Image Synthesis," *ICML*, PMLR 235, 2024. PMLR
10. Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel and M. Le, "Flow Matching for Generative Modeling," *ICLR*, 2023. arXiv
11. X. Liu, C. Gong and Q. Liu, "Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow," *ICLR*, 2023. arXiv
12. Y. Song, P. Dhariwal, M. Chen and I. Sutskever, "Consistency Models," *ICML*, 2023. arXiv
13. Z. Geng, M. Deng, X. Bai, J. Z. Kolter and K. He, "Mean Flows for One-step Generative Modeling," *NeurIPS*, 2025. arXiv
14. S. J. D. Prince, *Understanding Deep Learning*, MIT Press, 2023. book site
15. J. Sohl-Dickstein, E. Weiss, N. Maheswaranathan and S. Ganguli, "Deep Unsupervised Learning using Nonequilibrium Thermodynamics," *ICML*, PMLR 37, 2015. PMLR
16. P. Vincent, "A Connection Between Score Matching and Denoising Autoencoders," *Neural Computation*, vol. 23, no. 7, pp. 1661–1674, 2011. doi
17. Y. Song and S. Ermon, "Generative Modeling by Estimating Gradients of the Data Distribution," *NeurIPS*, 2019. arXiv
18. P. Dhariwal and A. Nichol, "Diffusion Models Beat GANs on Image Synthesis," *NeurIPS*, 2021. NeurIPS
19. A. Q. Nichol and P. Dhariwal, "Improved Denoising Diffusion Probabilistic Models," *ICML*, PMLR 139, 2021. PMLR
20. T. Karras, M. Aittala, T. Aila and S. Laine, "Elucidating the Design Space of Diffusion-Based Generative Models," *NeurIPS*, 2022. arXiv
21. C. Lu, Y. Zhou, F. Bao, J. Chen, C. Li and J. Zhu, "DPM-Solver: A Fast ODE Solver for Diffusion Probabilistic Model Sampling in Around 10 Steps," *NeurIPS*, 2022. arXiv
22. T. Salimans and J. Ho, "Progressive Distillation for Fast Sampling of Diffusion Models," *ICLR*, 2022. arXiv
23. M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler and S. Hochreiter, "GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium," *NeurIPS (NIPS)*, 2017. arXiv

24. T. Kynkäänniemi, T. Karras, S. Laine, J. Lehtinen and T. Aila, "Improved Precision and Recall Metric for Assessing Generative Models," *NeurIPS*, 2019. arXiv
25. G. Stein, J. C. Cresswell, R. Hosseinzadeh et al., "Exposing Flaws of Generative Model Evaluation Metrics and Their Unfair Treatment of Diffusion Models," *NeurIPS*, 2023. arXiv
26. S. Jayasumana, S. Ramalingam, A. Veit, D. Glasner, A. Chakrabarti and S. Kumar, "Rethinking FID: Towards a Better Evaluation Metric for Image Generation," *CVPR*, 2024. CVF
27. F.-A. Croitoru, V. Hondru, R. T. Ionescu and M. Shah, "Diffusion Models in Vision: A Survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2023. doi
28. H. Cao, C. Tan, Z. Gao, Y. Xu, G. Chen, P.-A. Heng and S. Z. Li, "A Survey on Generative Diffusion Models," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 7, pp. 2814–2830, 2024. doi
29. C.-H. Lai, Y. Song, D. Kim, Y. Mitsufuji and S. Ermon, "The Principles of Diffusion Models," monograph, arXiv:2510.21890, 2025. arXiv
30. H. Shen, J. Zhang, B. Xiong et al., "Efficient Diffusion Models: A Survey," *Transactions on Machine Learning Research*, 2025. arXiv
31. A. Melnik, M. Ljubljanac, C. Lu, Q. Yan, W. Ren and H. Ritter, "Video Diffusion Models: A Survey," *Transactions on Machine Learning Research*, 2024. arXiv
32. C. Lu and Y. Song, "Simplifying, Stabilizing and Scaling Continuous-Time Consistency Models," *ICLR*, 2025. arXiv
33. A. Sauer, D. Lorenz, A. Blattmann and R. Rombach, "Adversarial Diffusion Distillation," *ECCV*, 2024. arXiv
34. M. S. Albergo and E. Vanden-Eijnden, "Building Normalizing Flows with Stochastic Interpolants," *ICLR*, 2023. arXiv
35. J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li and L. Fei-Fei, "ImageNet: A Large-Scale Hierarchical Image Database," *CVPR*, 2009, pp. 248–255. doi
36. C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens and Z. Wojna, "Rethinking the Inception Architecture for Computer Vision," *CVPR*, 2016, pp. 2818–2826. arXiv
37. O. Ronneberger, P. Fischer and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," *MICCAI*, LNCS 9351, pp. 234–241, 2015. arXiv
38. A. Radford, J. W. Kim, C. Hallacy et al., "Learning Transferable Visual Models From Natural Language Supervision," *ICML*, 2021. arXiv
39. M. Oquab, T. Darcet, T. Moutakanni et al., "DINOv2: Learning Robust Visual Features without Supervision," *Transactions on Machine Learning Research*, 2024. arXiv
40. B. Efron, "Tweedie's Formula and Selection Bias," *Journal of the American Statistical Association*, vol. 106, no. 496, pp. 1602–1614, 2011. doi
41. A. Dosovitskiy, L. Beyer, A. Kolesnikov et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," *ICLR*, 2021. arXiv
42. T. Li and K. He, "Back to Basics: Let Denoising Generative Models Denoise," *CVPR*, 2026 (preprint 2025). paper, publication record, official JiT code.
43. B. Zheng, N. Ma, S. Tong and S. Xie, "Diffusion Transformers with Representation Autoencoders," *ICLR*, 2026 (preprint 2025). proceedings, manuscript.
44. S. Tong, B. Zheng, Z. Wang, B. Tang, N. Ma, E. Brown, J. Yang, R. Fergus, Y. LeCun and S. Xie, "Scaling Text-to-Image Diffusion Transformers with Representation Autoencoders," *CVPR Findings Track*, 2026. manuscript, author publication record.

45. D. Podell, Z. English, K. Lacey, A. Blattmann, T. Dockhorn, J. Müller, J. Penna and R. Rombach, “SDXL: Improving Latent Diffusion Models for High-Resolution Image Synthesis,” *ICLR*, 2024. proceedings, official model card.
46. Black Forest Labs, “FLUX.1 [dev],” official pretrained-model card, 2024. model card.
47. W. Kong, Q. Tian, Z. Zhang et al., “HunyuanVideo: A Systematic Framework For Large Video Generative Models,” technical report, arXiv:2412.03603, 2024. report, official release.
48. Team Wan, A. Wang, B. Ai, B. Wen et al., “Wan: Open and Advanced Large-Scale Video Generative Models,” technical report, arXiv:2503.20314, 2025. report, official Wan2.1 release.

ML 08 • Autoregressive Vision: Pixels, Tokens and Scales

Factor an image distribution into conditionals: PixelCNN, VQ-VAE and VQGAN tokenizers, GPT-style token transformers, next-scale prediction, masked generative models, continuous tokens and how they compare with diffusion.

Online edition: https://blog.shyandram.dev/en/learn/ml/ml-08-autoregressive-vision/

What you'll learn

- How the probability chain rule turns "model a whole image" into a sequence of "predict the next piece" problems, and how to measure the result in bits per dimension.
- Why training can run in parallel even though sampling is sequential, and how causal masks and masked convolutions keep the answer hidden from its own predictor.
- How PixelRNN and PixelCNN model raw pixels, and why their sequence length eventually pushed the field toward learned tokens.
- How VQ-VAE and VQGAN tokenizers turn an image into a short grid of integers, including the codebook and commitment losses and the straight-through gradient.
- How GPT-style token transformers (Image GPT, Parti, LlamaGen), next-scale prediction (VAR), masked generative models (MaskGIT, Muse) and continuous-token models (MAR) change the unit of prediction.
- How sampling controls (temperature, top-k, top-p, classifier-free guidance, KV caching) work, and how autoregressive generators compare with diffusion models.

The big picture

An **autoregressive** (**AR**) generator builds a sample piece by piece. At each step it looks at everything it has produced so far, predicts a probability distribution for the next piece, draws from it, and appends the result. The "piece" can be a single pixel value, an integer token from a learned codebook, a vector of continuous features, or an entire map at the next resolution.

> **Explain like I'm five**
>
> Imagine drawing a comic one panel at a time. Each new panel must fit the panels already drawn. When you practice, you copy real comics, so you always know what the earlier panels look like. When you invent a new comic, you have to live with your own earlier panels, even the slightly wobbly ones. An autoregressive image model works the same way: practice on real images, then create new ones one piece at a time.

Why does this matter? Autoregression is the same recipe that powers large language models: next-token prediction with a transformer. Bringing images into that recipe lets one architecture

read and write text and pictures, reuse the engineering of language-model serving, and measure progress with an exact likelihood. The price is that images are not naturally sequences. Someone has to decide **what a token is** and **in which order tokens are produced**. Most of this chapter is about those two decisions.

Xiong et al.'s survey [1] organizes visual AR models into three families by their prediction unit: **pixel-based**, **token-based** and **scale-based**. We follow that map, then add masked (non-left-to-right) and continuous-token models, which blur the boundary with diffusion. The previous chapters on generative models and diffusion and flow matching provide the VAE and diffusion background used here; Prince [9] covers the general probabilistic material at textbook level.

The chain rule needs no independence assumption

Plain version. Any joint distribution can be written as a product of "next element given the earlier ones" distributions. That is a fact of probability, not a modeling assumption.

For an ordered sequence $x = (x_1, \dots, x_N)$,

$$p_\theta(x) = \prod_{i=1}^{N} p_\theta(x_i \mid x_{<i}), \qquad \mathcal{L}_{\mathrm{NLL}}(\theta) = -\sum_{i=1}^{N} \log p_\theta(x_i \mid x_{<i}).$$

Here x_i is the i-th element in the chosen order, $x_{<i} = (x_1, \dots, x_{i-1})$ is everything before it, θ are the network parameters, and $\mathcal{L}_{\mathrm{NLL}}$ is the negative log-likelihood of one training example (averaged over the dataset in practice).

The chain rule is exact for **any** order. Approximation enters only through the model: a finite network, a limited context window, or a restricted form for each conditional. For discrete values, each conditional is usually a softmax over the possible values; binary pixels can use a Bernoulli distribution; continuous features need a density or another conditional generator (we meet one in the MAR section).

An order is a computational choice, not a claim about physics. **Raster order** (left to right, top to bottom) is convenient but arbitrary for a two-dimensional image: the pixel below you is just as related as the pixel to your right. Different orders lead to different conditionals, and a finite model may find some orders easier to learn than others. Figure 1 shows three prediction units used in this chapter.

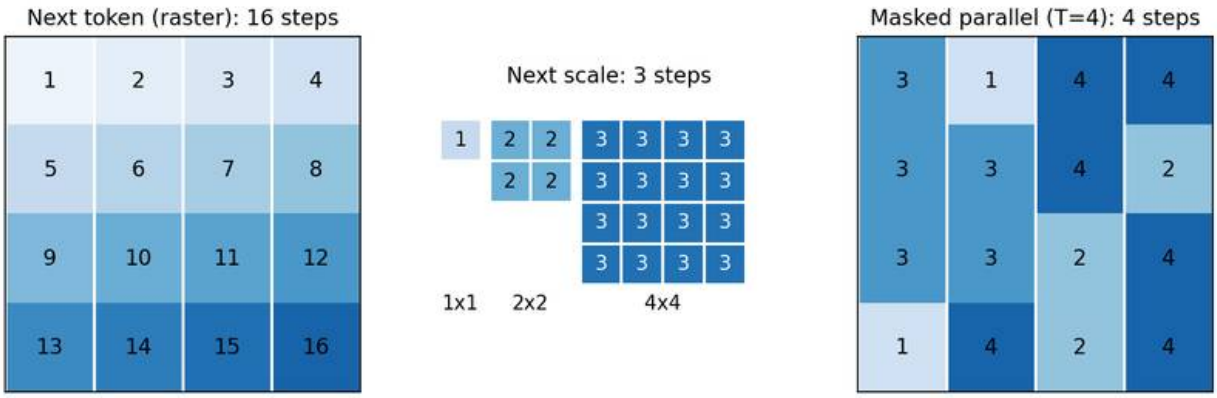


Figure 1 — The same 16 final tokens produced by three generation schemes. Numbers give the sequential step that produces each token: raster autoregression needs 16 steps, next-scale prediction 3 (one per map), and masked parallel decoding with $T = 4$ needs 4. Diagram only; no model is trained.

Measuring an AR model: bits per dimension

Because the chain rule gives an exact $\log p_\theta(x)$, AR models are usually compared by **negative log-likelihood in bits per dimension (bpd)**:

$$\text{bpd}(x) = \frac{-\log_2 p_\theta(x)}{D},$$

where D is the number of discrete values in the image (for example $D = 3 \cdot 32 \cdot 32$ for a 32×32 RGB image) and $\log_2$ converts nats to bits. A model that assigned every 8-bit value a uniform probability would score exactly 8 bpd; anything lower means the model has learned structure. Bpd is measured on held-out images, so it detects overfitting as well as underfitting.

Two cautions. First, bpd and visual quality can disagree: a model can spend most of its likelihood on imperceptible noise while still making odd global mistakes. Second, for a **token** model the likelihood is over codebook indices, not pixels. A lossy tokenizer throws information away, so a token-level likelihood is not an exact pixel-space likelihood and cannot be compared directly with pixel-model bpd.

Teacher forcing, causal masks and exposure bias

Plain version. During training the model always sees the real history; during sampling it sees its own history. A mask makes sure training never lets a position peek at its own answer.

With **teacher forcing**, every conditional in the training loss is computed using the ground-truth previous elements. Since the whole training image is available, all N conditionals can be evaluated in a single forward pass, as long as position i is prevented from reading positions $\geq i$. A causal transformer enforces this with an attention mask; a PixelCNN enforces it with masked convolution kernels.

Two input conventions are common, and the mask must match the convention:

- In a **shifted-input** decoder, feed `[BOS, x1, x2, ...]` and predict `[x1, x2, x3, ...]`. A lower-triangular mask that *includes* the diagonal is correct, because the input at position i is the *previous* element.
- In a model that receives **unshifted** values, the dependency must be **strictly** earlier. Including the diagonal would let the network copy the value it is supposed to predict.

At inference, earlier sampled mistakes enter the context of later steps. This training/inference mismatch is often called **exposure bias**. It does not make maximum likelihood wrong (the factorization is still exact), but it matters in practice for imperfect models and long sequences: a small early error in the layout of an image can push later steps into contexts that never occurred in training.

Pixel models: PixelRNN and PixelCNN

Plain version. Treat each pixel's intensity as a 256-way classification problem whose input is all pixels above and to the left.

PixelRNN and PixelCNN were introduced together by van den Oord, Kalchbrenner and Kavukcuoglu [2]. Both model each 8-bit value with a **256-way softmax** rather than a Gaussian, so the distribution can be multimodal (a pixel can be "either black or white, but not grey"). Colour is handled by a fixed order inside each pixel, red then green then blue, so that green may depend on red and blue on both. PixelRNN uses two-dimensional LSTM layers (Row LSTM and Diagonal BiLSTM) whose dependency range is potentially unbounded; PixelCNN replaces them with stacked masked convolutions, which have a bounded receptive field but train much faster because all positions are computed in parallel.

Masked convolutions

A masked convolution multiplies its kernel by a binary mask that zeros the weights at "future" positions. In [2], **mask A** is used only in the first layer: it excludes the centre pixel, so the first layer never sees the value it predicts (and, for colour, only sees channels already predicted). **Mask B** is used in all later layers: it allows the centre position, because after the first layer the centre feature already summarizes only earlier pixels. Formally, for a $k \times k$ kernel with centre (c, c), $c = \lfloor k/2 \rfloor$,

$$M_{uv} = \begin{cases} 1 & u < c, \\ 1 & u = c \text{ and } v < c \text{ (A) or } v \le c \text{ (B)}, \\ 0 & \text{otherwise}, \end{cases}$$

where u indexes kernel rows and v kernel columns. The code below builds both masks, stacks five layers, and uses a gradient to find exactly which input pixels can influence the output at the centre of a 9×9 image.

```
import torch
from torch import nn
from torch.nn import functional as F

class MaskedConv2d(nn.Conv2d):
    """PixelCNN-style masked convolution for one channel (raster order)."""
    def __init__(self, cin, cout, k, mask_type):
        super().__init__(cin, cout, k, padding=k//2, bias=False)
        mask = torch.ones(k, k)
        c = k//2
        mask[c, c + (mask_type == "B"):] = 0   # centre (type A) and pixels to its right
        mask[c+1:] = 0                         # all rows below
        self.register_buffer("mask", mask)
    def forward(self, x):
        return F.conv2d(x, self.weight*self.mask, padding=self.padding)

torch.manual_seed(0)
layers = [MaskedConv2d(1, 8, 3, "A"), nn.Tanh()]
for _ in range(4):                             # 5 layers: reach of 5 pixels
    layers += [MaskedConv2d(8, 8, 3, "B"), nn.Tanh()]
net = nn.Sequential(*layers, MaskedConv2d(8, 1, 1, "B"))
print(net[0].mask.int())
x = torch.rand(1, 1, 9, 9, requires_grad=True)
net(x)[0, 0, 4, 4].backward()                  # logit for pixel (row 4, col 4)
dep = (x.grad[0, 0].abs() > 0).int()
print(dep)
later = torch.zeros(9, 9, dtype=torch.bool)
later[4, 4:] = True; later[5:] = True
assert dep[later].sum() == 0                   # never reads itself or later pixels
print("earlier pixels never seen (blind spot):", int(((dep == 0) & ~later).sum()))
```

The assertion passes: no pixel at or after the target influences its logit, so the model is a valid autoregressive factorization. But the printout reveals something else. Six pixels that come *earlier* in raster order, in a staircase above and to the right of the target, have zero gradient. This is the **blind spot** of stacked masked convolutions (Figure 2): each layer's receptive field grows up and to the left, but the region up and to the right is never reached. The model is still correct, just less informed than it could be.

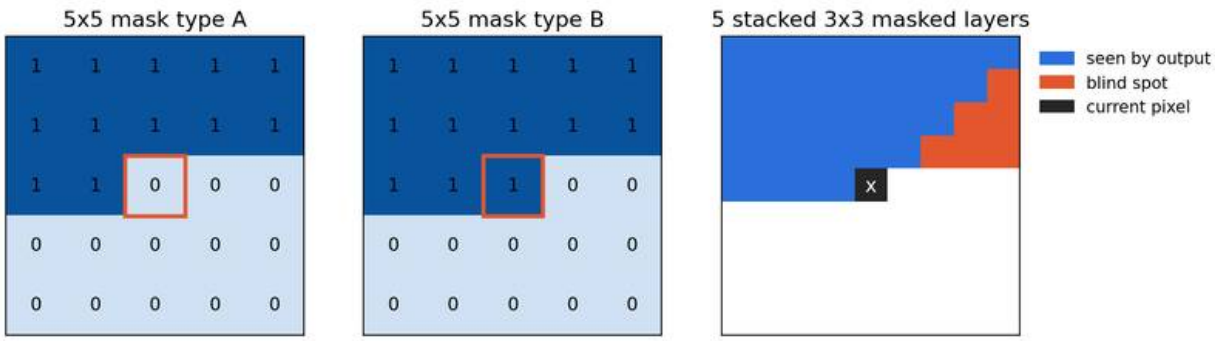


Figure 2 — Left and centre: mask A excludes the centre weight; mask B keeps it. Right: input pixels that influence the prediction at the dark cell after five stacked 3×3 masked layers, measured with a gradient. Orange cells come earlier in raster order but are never seen: the blind spot.

The Gated PixelCNN of van den Oord et al. [3] removed the blind spot by splitting the network into two stacks: a **vertical stack** that sees all rows above the current one without restriction and a **horizontal stack** that sees the current row up to the current pixel, combined after each layer. It also replaced ReLU with a gated unit $y = \tanh(W_f * x) \odot \sigma(W_g * x)$, where $*$ is convolution, σ the logistic sigmoid and $\odot$ elementwise multiplication, and showed how to condition the model on class labels or embeddings. With these changes PixelCNN matched PixelRNN's likelihood on ImageNet [23] at much lower cost [3].

Code: a minimal autoregressive model

Masked convolutions are one way to enforce causality; a transformer is not required either. The following complete CPU experiment generates 4×4 binary images whose rows all repeat the same four random bits. A single masked linear layer, whose strictly lower-triangular mask excludes the current pixel and all later ones, learns the conditionals directly.

```
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0)
torch.set_num_threads(1)
class BinaryAR(nn.Module):
    def __init__(self, n=16):
        super().__init__()
        self.weight = nn.Parameter(torch.zeros(n, n))
        self.bias = nn.Parameter(torch.zeros(n))
        self.register_buffer("mask", torch.tril(torch.ones(n, n), diagonal=-1))
    def forward(self, x):
        return F.linear(x, self.weight*self.mask, self.bias)

model = BinaryAR()
opt = torch.optim.Adam(model.parameters(), lr=.04)
for step in range(600):
    first_row = torch.randint(2, (128, 4)).float()
    x = first_row.repeat(1, 4)
    loss = F.binary_cross_entropy_with_logits(model(x), x)
    opt.zero_grad(); loss.backward(); opt.step()

with torch.no_grad():
    a = torch.rand(2, 16)
    b = a.clone(); b[:, 8:] = torch.rand(2, 8)
    assert torch.allclose(model(a)[:, :9], model(b)[:, :9])
    samples = torch.zeros(8, 16)
    for i in range(16):
        p = model(samples)[:, i].sigmoid()
        samples[:, i] = torch.bernoulli(p)
```

```
    rows = samples.reshape(-1, 4, 4)
    print("one generated image:\n", rows[0].int())
    print("repeat agreement:", (rows[:, 1:] == rows[:, :1]).float().mean().item())
```

The assertion checks causality: changing positions 8 onward cannot alter the logits for positions 0 through 8, because each of those outputs reads only earlier inputs. Training then drives the agreement between later rows and the first row to 1.0, while the first row stays random (the model must not memorize one particular row).

The true distribution has 16 equally likely images, so its entropy is $\log_2 16 = 4$ bits per image, or 0.25 bits per pixel. Later rows contain no new uncertainty. A model that treated all 16 pixels as independent fair coins would need 16 bits per image. The gap between 16 and 4 bits is exactly the structure an autoregressive model captures and an independent-pixel model cannot.

Why pixel models stopped scaling

A 256×256 RGB image has $3 \cdot 256 \cdot 256 = 196{,}608$ values, so pixel-level sampling needs about two hundred thousand sequential network evaluations. Most of the likelihood is also spent on fine, locally predictable detail, while the global layout that people notice is a small fraction of the bits. Image GPT [11] showed that a plain sequence transformer trained to predict pixels autoregressively on low-resolution images learns features that transfer well under linear probing, but the pixel route stays expensive at higher resolutions. The next step was to learn a shorter, more meaningful sequence.

Discrete visual tokens: VQ-VAE and VQGAN

Plain version. Train a separate network that compresses an image into a small grid of "words" from a learned dictionary, then do autoregression over the words instead of the pixels.

VQ-VAE

VQ-VAE [4] has an encoder that produces a feature vector $z_e(x) \in \mathbb{R}^d$ at each position of a downsampled grid, a **codebook** $\{e_1, \dots, e_K\} \subset \mathbb{R}^d$ of K learned vectors, and a decoder. Each feature is replaced by its nearest code:

$$k^* = \arg\min_k \|z_e(x) - e_k\|_2^2, \qquad z_q(x) = e_{k^*}.$$

The image is now represented by the integer grid of k^* values. Training minimizes

$$\mathcal{L}_{\text{VQ}} = \underbrace{\|x - D(z_q)\|^2}_{\text{reconstruction}} + \underbrace{\|\text{sg}[z_e] - e_{k^*}\|^2}_{\text{codebook}} + \beta \underbrace{\|z_e - \text{sg}[e_{k^*}]\|^2}_{\text{commitment}},$$

where D is the decoder, $\text{sg}[\cdot]$ is the **stop-gradient** operator (identity in the forward pass, zero gradient in the backward pass), and β is a small weight on the commitment term (the snippet

below uses 0.25). The reconstruction term is written as squared error here; any reconstruction likelihood can take its place.

The three terms do three jobs. The **codebook** term moves the chosen code toward the encoder output, like a k-means update. The **commitment** term pulls the encoder toward its code so that encoder outputs do not drift far from the codebook. The `argmin` has no useful gradient, so the reconstruction gradient reaches the encoder through the **straight-through estimator**: in the forward pass use z_q, in the backward pass copy the gradient from z_q to z_e as if quantization were the identity. In code this is the one-liner `z_q = z_e + (e - z_e).detach()`.

The toy below trains a 2-D "encoder", codebook and decoder on four clusters, with eight codes available:

```
import torch
from torch import nn
from torch.nn import functional as F

def quantize(z_e, codebook, beta=0.25):
    """z_e: (n, d) encoder outputs; codebook: (K, d). Returns z_q with straight-through
     ↪  gradient."""
    dist = torch.cdist(z_e, codebook)            # (n, K) Euclidean distances
    idx = dist.argmin(1)                         # nearest code index per vector
    e = codebook[idx]
    codebook_loss = F.mse_loss(e, z_e.detach())  # moves codes toward encoder outputs
    commit_loss = F.mse_loss(z_e, e.detach())    # keeps the encoder near its code
    z_q = z_e + (e - z_e).detach()               # forward: e; backward: identity
    return z_q, idx, codebook_loss + beta*commit_loss

torch.manual_seed(0)
torch.set_num_threads(1)
centres = torch.tensor([[2., 0.], [-2., 0.], [0., 2.], [0., -2.]])
encoder = nn.Linear(2, 2)                        # toy "encoder" and "decoder"
decoder = nn.Linear(2, 2)
codebook = nn.Parameter(torch.randn(8, 2)*.1)    # K = 8 codes, 4 are enough
opt = torch.optim.Adam([*encoder.parameters(), *decoder.parameters(), codebook], lr=.03)
for step in range(800):
    x = centres[torch.randint(4, (256,))] + .1*torch.randn(256, 2)
    z_q, idx, vq_loss = quantize(encoder(x), codebook)
    loss = F.mse_loss(decoder(z_q), x) + vq_loss
    opt.zero_grad(); loss.backward(); opt.step()

with torch.no_grad():
    x = centres.repeat_interleave(100, 0) + .1*torch.randn(400, 2)
    z_q, idx, _ = quantize(encoder(x), codebook)
    print("reconstruction MSE:", round(F.mse_loss(decoder(z_q), x).item(), 4))
    print("codes used:", idx.unique().numel(), "of", len(codebook))
    print("codes per cluster:", [idx[i*100:(i+1)*100].unique().tolist() for i in range(4)])
z = torch.randn(3, 2, requires_grad=True)
z_q, _, _ = quantize(z, codebook.detach())
z_q.sum().backward()
print("straight-through gradient:", z.grad[0].tolist())
```

It prints a reconstruction MSE of about 0.0096, uses **4 of the 8 codes** (one per cluster), and shows a straight-through gradient of `[1.0, 1.0]`. Two lessons hide in those numbers. The reconstruction error is almost exactly the within-cluster noise variance $0.1^2 = 0.01$: quantization has kept *which cluster* and discarded *where inside the cluster*. And half the codebook is **dead**: codes that are never selected receive no codebook gradient and never move (Figure 3). In real tokenizers, low codebook usage wastes capacity and is a standard failure to monitor.

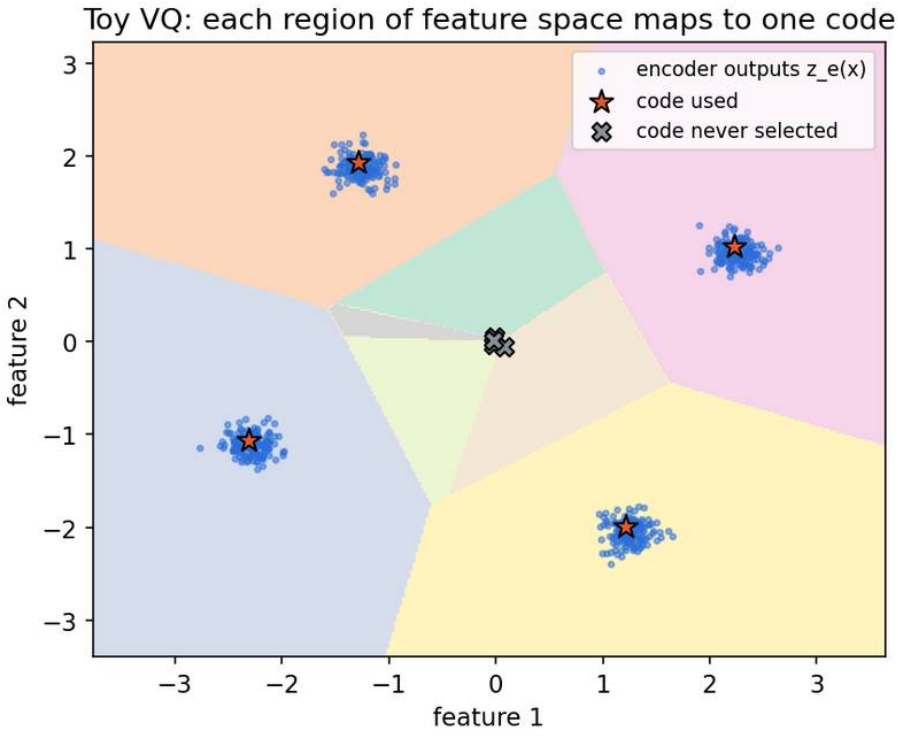


Figure 3 — Toy vector quantization. Each coloured region is the set of features mapped to one code. Four codes (stars) are used, one per cluster; four (crosses) were never selected and stayed near their initialization.

Two stages: a tokenizer, then a prior

A VQ-VAE alone is an autoencoder; it does not by itself say which token grids are likely. Generation therefore has **two stages** (Figure 4). Stage 1 trains the tokenizer. Stage 2 freezes it, converts every training image into its index grid, and trains an autoregressive model $p_\psi(k_1, \ldots, k_M)$ over the M indices. To sample, draw an index grid from the prior and decode it. In [4] the prior was a PixelCNN over codes; VQ-VAE-2 [10] used a hierarchy of two code maps (a coarse top level and a finer bottom level) with PixelCNN-style priors and produced diverse, high-fidelity ImageNet samples.

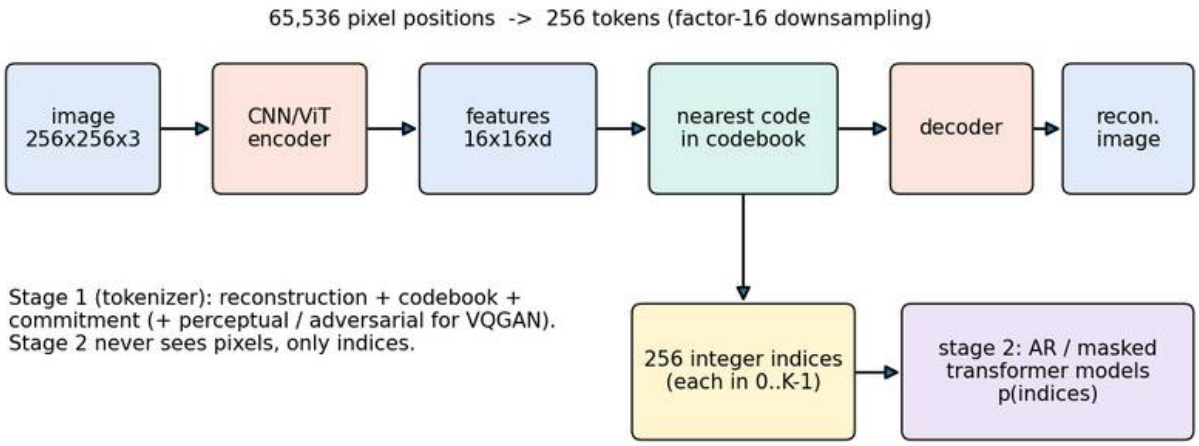


Figure 4 — Two-stage generation. The tokenizer (top row) is trained for reconstruction; the prior (bottom right) only ever sees integer indices. With factor-16 downsampling, a 256×256 image becomes 256 tokens instead of 65,536 pixel positions.

The arithmetic explains the appeal. If a tokenizer downsamples a 256×256 image by 16 in each direction, the prior sees $16 \times 16 = 256$ tokens rather than 65,536 pixel positions (or 196,608 colour values). Self-attention cost grows with the square of sequence length, so this is the difference between feasible and infeasible.

VQGAN: perceptual tokens

Esser, Rombach and Ommer's VQGAN [5] kept the two-stage design but trained the tokenizer with a **perceptual loss** and a **patch-based adversarial loss** in addition to reconstruction, and modelled the token grid with a transformer. The adversarial and perceptual terms let the decoder produce sharp, realistic texture from very few tokens, because the tokens no longer need to encode every pixel exactly; they need to encode enough for the decoder to produce *a* plausible version. This is why VQGAN-style tokenizers became the default front end for token-based generation. Note the division of labour: the **tokenizer** uses adversarial training, but the **prior** is an ordinary autoregressive model trained by maximum likelihood.

Codebook problems and alternatives

Real tokenizers face three recurring problems: dead codes (as in the toy), a gap between what the tokenizer reconstructs and what the prior can model, and loss of small details such as text and thin lines. Several designs respond to them:

- **ViT-VQGAN** [12] uses a Vision Transformer encoder/decoder, looks codes up in a low-dimensional **factorized** space and ℓ_2**-normalizes** codes and features, which improved codebook usage and reconstruction.

- **RQ-VAE** [15] uses **residual quantization**: each position is represented by a short stack of codes, each one quantizing what the previous ones missed. This allows a coarser spatial grid (shorter sequence) at similar fidelity.
- **FSQ** [16] (finite scalar quantization) projects to a handful of dimensions and rounds each to a few fixed levels. The implicit codebook is the product of the levels; there is no learned codebook to collapse and no commitment loss.

Compression always creates a bottleneck. Measure codebook usage, reconstruction quality and the generator's quality **separately**: a generator cannot produce detail its tokenizer cannot represent.

Autoregressive transformers over tokens

Plain version. Once an image is a sequence of integers, generating it is the same problem as generating text: a decoder-only transformer predicts the next token.

From pixels to tokens

Image GPT [11] applied a GPT-style transformer directly to sequences of low-resolution pixels. Its main finding was about **representation learning**: features from a model trained only to predict the next pixel transfer well when a linear classifier is trained on top. Working on tokens instead of pixels shortens sequences enough to generate at higher resolution. The VQGAN transformer [5] is the classic example: a GPT-like model over VQGAN indices in raster order, which let a transformer synthesize high-resolution images that would be far too long as pixel sequences.

Text-to-image as translation: Parti

Parti [13] treats text-to-image generation like machine translation. A transformer **encoder** reads the text prompt; a transformer **decoder** autoregressively generates ViT-VQGAN image tokens [12] conditioned on it. The report scales this sequence-to-sequence model up to 20 billion parameters and introduces PartiPrompts, a benchmark of more than 1,600 English prompts spanning many categories of content.

Language-model architecture, unchanged: LlamaGen

LlamaGen [14] asks whether a vanilla language-model architecture, with no image-specific inductive biases, can generate images well when scaled. It uses a Llama-style [24] decoder (RMSNorm pre-normalization, SwiGLU feed-forward layers, 2-D rotary position embeddings) over the tokens of a factor-16 VQ tokenizer with $K = 16{,}384$ codes of dimension 8. The class label or projected text embedding is supplied as a **prefix** token, and generation proceeds in raster order.

The paper releases class-conditional models from 111M to 3.1B parameters and reports that standard LLM serving software (vLLM) speeds up inference. Its broader message is architectural reuse: an image generator can share almost everything with a text model except the tokenizer.

Classifier-free guidance for autoregressive models

Classifier-free guidance (CFG) [18] was introduced for diffusion models, but it transfers directly to AR logits. During training, the condition c is randomly replaced by a learned **null** embedding so that one network learns both conditional and unconditional predictions. At each sampling step, compute both sets of logits and combine them:

$$\ell_g = \ell_u + s\,(\ell_c - \ell_u),$$

where ℓ_c are the next-token logits with the condition, ℓ_u the logits with the null condition, and s the guidance scale; $s = 1$ recovers the ordinary conditional model. This is the form LlamaGen uses [14]. Larger s usually improves prompt or class adherence and sample sharpness at the cost of diversity. It also doubles the per-step compute unless the two passes are batched, so step counts alone understate cost.

Next-scale prediction (VAR)

Plain version. Instead of predicting the next token in a fixed reading order, predict the next, sharper version of the whole image, conditioned on all coarser versions.

Raster order forces a strange dependency: the top-left of an image is generated without any knowledge of the bottom-right. VAR (Visual AutoRegressive modeling) [6] argues that this one-directional order conflicts with the two-directional correlations of image tokens and with spatial locality, and replaces it with a **coarse-to-fine** order over token maps $r_1, \ldots, r_K$ of increasing resolution:

$$p(r_1, \ldots, r_K) = \prod_{k=1}^{K} p(r_k \mid r_1, \ldots, r_{k-1}),$$

where r_k is an $h_k \times w_k$ grid of token indices, r_1 is 1×1 and r_K has the full latent resolution. All tokens of r_k are predicted **in parallel** given the coarser maps, so the number of sequential steps equals the number of scales, not the number of tokens.

The tokenizer is part of the method. VAR's multi-scale VQ encoder works **residually**: starting from the encoder feature map f, for $k = 1, \ldots, K$ it downsamples f to $h_k \times w_k$, quantizes it to get r_k (with one codebook shared across scales), upsamples the looked-up codes back to full resolution, passes them through a small convolution ϕ_k, and subtracts the result from f before moving to the next scale [6]. Each scale therefore encodes what the coarser scales have not explained. Simply resizing ordinary token maps would not reproduce this.

During training, a **block-wise causal** attention mask lets every token of r_k attend to all tokens of $r_{\le k}$ but none of the finer scales; at inference a KV cache stores the coarser scales. VAR uses

CFG as well. On class-conditional ImageNet 256 × 256 the paper reports that VAR improves its AR baseline's FID from 18.65 to 1.73, with faster inference, and that its models follow power-law scaling trends similar to those of language models [6]. VAR received a NeurIPS 2024 Best Paper award. As always, those numbers apply to the paper's data, resolution and training protocol.

Parallel prediction within a scale is itself an assumption: tokens in r_k are sampled independently given $r_{<k}$. The masked-model section shows what that assumption can cost on a toy problem.

Masked generative models (MaskGIT and Muse)

Plain version. Start with a grid of blanks. At each round, guess every blank at once, keep only the guesses you are most confident about, and blank out the rest again.

MaskGIT [8] uses the same VQ tokens as an AR model, but a **bidirectional** transformer that sees every position. Training is a masked-token objective: choose a mask ratio, replace that fraction of tokens with a special `[MASK]` token, and minimize cross-entropy on the masked positions,

$$\mathcal{L}_{\text{mask}} = -\mathbb{E}\Big[\sum_{i \in \mathcal{M}} \log p_\theta(y_i \mid y_{\bar{\mathcal{M}}})\Big],$$

where y is the token grid, $\mathcal{M}$ is the set of masked positions and $y_{\bar{\mathcal{M}}}$ the unmasked tokens. The mask ratio is drawn as $\gamma(r)$ with $r \sim \mathcal{U}(0,1)$ and a **mask schedule** γ that decreases from 1 to 0.

Generation runs for T iterations. At iteration t the model predicts all masked tokens in parallel and samples them; each sample's probability serves as its **confidence**. The number of tokens that remain masked after the iteration is

$$n_t = \lceil \gamma(t/T)\, N \rceil,$$

with N the total number of tokens, so the n_t least confident new tokens are re-masked and everything else is fixed. MaskGIT compares several schedules and selects the **cosine** schedule $\gamma(r) = \cos(\pi r/2)$, which fixes few tokens early (when the model knows little) and many late; it generates 256 × 256 images in 8 iterations with a 16×16 token grid and a 1,024-entry codebook [8]. Muse [17] scales the same idea to text-to-image generation, conditioning the masked-token transformer on text embeddings from a pre-trained large language model.

MaskGIT is **not** ordinary left-to-right chain-rule autoregression: the order is chosen on the fly by confidence, and several tokens are sampled in the same step. Those simultaneous tokens are drawn **independently** given the current context. The toy below shows when that matters. It trains a small bidirectional network on the repeated-row 4 × 4 images from earlier and decodes with the cosine schedule.

```
import math
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0)
torch.set_num_threads(1)
```

```
N, MASK = 16, 2                                      # 16 binary tokens; state 2 = [MASK]
gamma = lambda r: math.cos(math.pi*r/2)             # cosine schedule: fraction still masked
net = nn.Sequential(nn.Flatten(), nn.Linear(3*N, 128), nn.ReLU(), nn.Linear(128, N))

def data(n):                                         # 4x4 images whose rows repeat
    return torch.randint(2, (n, 4)).repeat(1, 4)

def logits(tokens):                                  # sees every position: bidirectional
    return net(F.one_hot(tokens, 3).float())

opt = torch.optim.Adam(net.parameters(), lr=3e-3)
for step in range(1500):
    x = data(256)
    ratio = torch.tensor([gamma(r) for r in torch.rand(256).tolist()])
    n_mask = (ratio*N).ceil().clamp(min=1)
    masked = torch.rand(256, N).argsort(1) < n_mask[:, None]   # random positions
    inp = torch.where(masked, MASK, x)
    loss = F.binary_cross_entropy_with_logits(logits(inp)[masked], x[masked].float())
    opt.zero_grad(); loss.backward(); opt.step()

@torch.no_grad()
def generate(n, T=4):
    tokens = torch.full((n, N), MASK)
    for t in range(1, T+1):
        p = logits(tokens).sigmoid()
        sample = torch.bernoulli(p).long()
        conf = torch.where(sample == 1, p, 1-p)              # prob. of the drawn value
        still = tokens == MASK
        conf = torch.where(still, conf, torch.inf)           # never re-mask fixed tokens
        proposal = torch.where(still, sample, tokens)
        n_keep_masked = math.floor(gamma(t/T)*N)             # 0 after the last step
        order = conf.argsort(1)                              # least confident first
        remask = torch.zeros_like(still)
        remask.scatter_(1, order[:, :n_keep_masked], True)
        tokens = torch.where(remask, MASK, proposal)
    return tokens

for T in (1, 2, 4, 8, 16):
    rows = generate(1000, T).reshape(-1, 4, 4)
    agree = (rows[:, 1:] == rows[:, :1]).float().mean().item()
    valid = (rows == rows[:, :1]).all(2).all(1).float().mean().item()
    print(f"T={T:2d}  repeat agreement={agree:.3f}  valid images={valid:.3f}")
```

(The code uses `floor` rather than the paper's ceiling so that the count reaches exactly zero at $t = T$ in this tiny example.) The output is:

```
T= 1  repeat agreement=0.511  valid images=0.000
T= 2  repeat agreement=0.812  valid images=0.100
T= 4  repeat agreement=0.908  valid images=0.360
T= 8  repeat agreement=0.998  valid images=0.982
T=16  repeat agreement=1.000  valid images=0.999
```

With $T = 1$ every token is sampled independently from the all-mask context, so rows almost never agree. With more iterations, the first few tokens are fixed early and later ones can condition on them; by $T = 8$ nearly every sample is valid. On natural images the dependence between tokens decoded in the same round is weaker, which is why a handful of iterations works, but the trade-off between **fewer steps** and **more independence assumptions** is real (Figure 5).

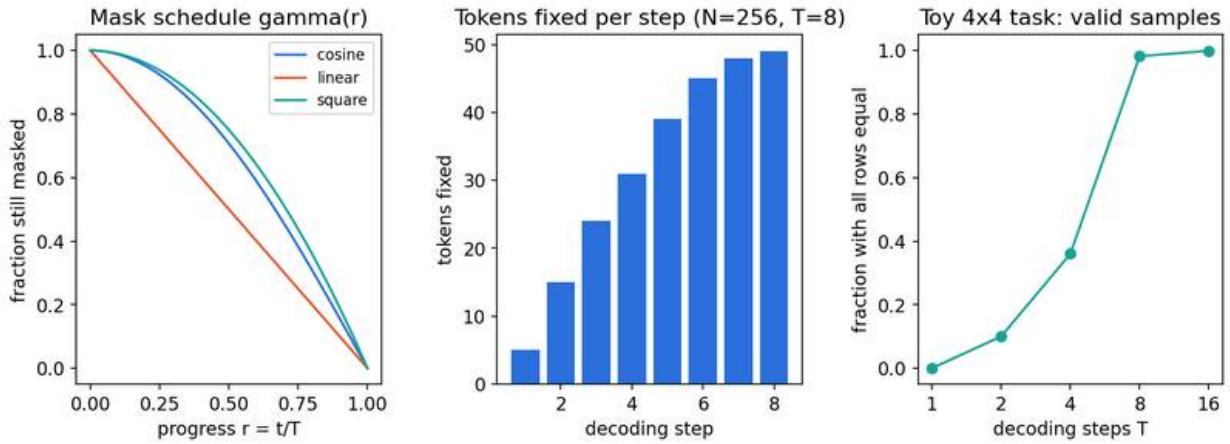


Figure 5 — Left: candidate mask schedules $\gamma(r)$. Centre: with the cosine schedule, 256 tokens and $T = 8$, few tokens are fixed early and many late. Right: on the toy repeated-row task, the fraction of valid samples grows with the number of decoding iterations because fewer tokens are sampled independently in the same step.

Continuous tokens and diffusion conditionals (MAR)

Plain version. Autoregression decides *what comes next*; a tiny diffusion model can decide *how to sample* it, so the tokens no longer need to be integers.

Vector quantization is lossy and awkward to train. Li et al. [7] observe that autoregression needs a way to model $p(x_i \mid x_{<i})$, not necessarily a categorical distribution. They let the AR (or masked AR) transformer output a conditioning vector z_i for each continuous-valued token x_i, and train a small MLP denoiser ϵ_θ with a **diffusion loss**:

$$\mathcal{L}(z_i, x_i) = \mathbb{E}_{\epsilon,t}\|\epsilon - \epsilon_\theta(x_{i,t} \mid t, z_i)\|^2, \qquad x_{i,t} = \sqrt{\bar{\alpha}_t}\, x_i + \sqrt{1-\bar{\alpha}_t}\, \epsilon,$$

where $\epsilon \sim \mathcal{N}(0, I)$, t is a diffusion time step and $\bar{\alpha}_t$ the noise schedule from ML 07. The gradient also flows into the transformer through z_i. To sample a token, run the reverse diffusion process of the small denoiser conditioned on z_i; a temperature scales the injected noise. The paper's main variant, **MAR** (masked autoregressive), uses a random order and predicts a set of tokens per step, combining the outer structure of MaskGIT with an inner continuous sampler [7].

Consequently, "AR versus diffusion" is not an either/or choice. A system can decide the order and grouping of tokens autoregressively and sample each group by diffusion. Likewise, a language decoder can autoregressively generate *text about* an image without generating any pixels;

the unifying idea is factorization, not a particular output type.

Sampling, temperature and caching

Plain version. The trained model gives a distribution at every step; the sampler decides how adventurous to be when drawing from it, and the cache avoids recomputing the past.

For logits ℓ_k over K possible values, **temperature** T rescales the distribution:

$$p_T(k) = \frac{\exp(\ell_k / T)}{\sum_{j=1}^{K} \exp(\ell_j / T)}.$$

Lower T concentrates probability on the top values; larger T spreads it (Figure 6). **Top-k** sampling keeps only the k highest logits; **top-p** (nucleus) sampling keeps the smallest set of values whose probability mass reaches p. Both renormalize what remains.

```
import torch

def filtered_probs(logits, temperature=1.0, top_k=None, top_p=None):
    """Return the (renormalized) distribution actually sampled from."""
    logits = logits / temperature
    if top_k is not None:                               # keep the k largest logits
        kth = logits.topk(top_k).values[..., -1, None]
        logits = logits.masked_fill(logits < kth, -torch.inf)
    if top_p is not None:                               # smallest set with mass >= p
        sorted_logits, order = logits.sort(descending=True)
        cum = sorted_logits.softmax(-1).cumsum(-1)
        drop = cum - sorted_logits.softmax(-1) >= top_p  # mass before this token already >=
         ↪  p
        sorted_logits = sorted_logits.masked_fill(drop, -torch.inf)
        logits = torch.empty_like(logits).scatter(-1, order, sorted_logits)
    return logits.softmax(-1)

logits = torch.tensor([2.0, 1.0, 0.0, -1.0])
for name, kw in [("T=1", {}), ("T=0.5", {"temperature": .5}), ("T=2", {"temperature": 2.}),
                 ("top-k=2", {"top_k": 2}), ("top-p=0.8", {"top_p": .8})]:
    p = filtered_probs(logits, **kw)
    entropy = -(p[p > 0]*p[p > 0].log2()).sum().item()
    print(f"{name:10s}", [round(v, 3) for v in p.tolist()], f"entropy={entropy:.2f} bits")
```

The model's own distribution ($T = 1$) has 1.37 bits of entropy; $T = 0.5$ lowers it to 0.66 bits and $T = 2$ raises it to 1.80 bits. Top-$k = 2$ and top-$p = 0.8$ both keep the two most likely values here (the first value has mass 0.644, which is below 0.8, so the second is needed). Every one of these changes the distribution you sample from, so a sampler's visual quality should not be confused with the model's likelihood.

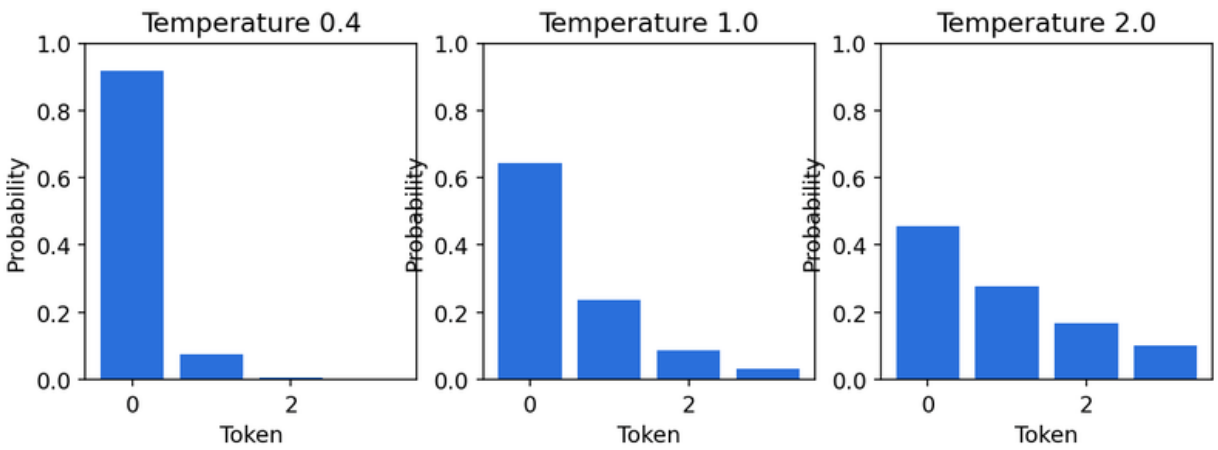


Figure 6 — Temperature changes diversity at each conditional. It cannot recover information that a tokenizer has already lost.

The KV cache

In a causal transformer, the keys and values of earlier tokens do not change when a new token is appended. A **KV cache** stores them, so each new step computes the query, key and value of the newest token only and attends to the cache:

```
import torch
from torch.nn import functional as F

torch.manual_seed(0)
d, n = 8, 6
Wq, Wk, Wv = (torch.randn(d, d) for _ in range(3))
x = torch.randn(n, d)                                  # embeddings of n tokens

# Full causal attention over the whole sequence (what training computes).
q, k, v = x@Wq, x@Wk, x@Wv
mask = torch.ones(n, n).tril().bool()
scores = (q@k.T/d**.5).masked_fill(~mask, -torch.inf)
full = scores.softmax(-1)@v

# Incremental decoding: each step computes q, k, v for the newest token only.
K_cache, V_cache, outputs = [], [], []
for i in range(n):
    K_cache.append(x[i]@Wk); V_cache.append(x[i]@Wv)
    K, V = torch.stack(K_cache), torch.stack(V_cache)
    att = F.softmax((x[i]@Wq)@K.T/d**.5, -1)        # attends to tokens 0..i
    outputs.append(att@V)
assert torch.allclose(full, torch.stack(outputs), atol=1e-5)
print("cached decoding matches full causal attention")
print("dot products per step:", [i+1 for i in range(n)], "total:", n*(n+1)//2)
```

The assertion confirms that cached decoding gives exactly the training-time causal outputs (for a single layer here; real models cache every layer and head). The cache removes recomputation but not the sequential dependency, and it grows linearly with sequence length. Step i still attends to i cached keys, so total attention work over a sequence of length n is $n(n+1)/2$, quadratic in n. This is why shorter sequences (better tokenizers) and fewer sequential steps (next-scale or masked decoding) matter so much.

For video, tokens can be ordered over space and time or grouped into chunks. Caching, compression and temporal ordering then dominate cost, and object consistency and long-duration drift need their own evaluation.

Autoregressive versus diffusion

Plain version. Both families turn one hard generation problem into many easier ones. AR splits the *image* into pieces; diffusion splits the *noise level* into steps.

Aspect	Autoregressive (token / scale)	Masked generative	Diffusion / flow (ML 07)
What is generated per step	One token (or one scale)	A confidence-chosen set of tokens	A refinement of the whole sample
Sequential steps	Tokens or scales	Fixed, typically few (T)	Solver steps (NFE)
Typical representation	Discrete codes (or continuous with a diffusion head [7])	Discrete codes	Continuous pixels or latents
Training objective	Exact NLL (cross-entropy)	Masked cross-entropy	Denoising / velocity regression (ELBO-related)
Likelihood	Exact over tokens	Not a single exact chain-rule likelihood	Bound or ODE-based estimate
Natural strengths	Shares architecture and serving with LLMs; mixes with text	Few steps; easy inpainting and editing	Strong image quality; flexible guidance
Typical weaknesses	Long sequential decoding; tokenizer bottleneck; error accumulation	Independence within a step	Many network evaluations unless distilled

Three points keep this table honest. First, the families mix: MAR [7] puts a diffusion model inside an AR loop, and latent diffusion also depends on a learned tokenizer-like autoencoder. Second, claims that one family "beats" the other (as in LlamaGen's title [14] or VAR's comparison with DiT [6]) hold for a stated benchmark, tokenizer, model size and guidance setting, not universally. Third, cost should be compared in network evaluations, guidance passes, sequence length and actual latency, not in "steps" alone.

Modern view

The field has moved from raw pixels to learned representations, from fixed raster order to flexible prediction units, and from separate image generators to models that read and write both text and

images. Several surveys map this territory:

- **Xiong et al. [1]** (TMLR) is the most direct survey of visual autoregression. Its taxonomy by representation (pixel-, token- and scale-based models) is the one used in this chapter, and it covers applications in image, video, 3D and multimodal generation as well as embodied and medical settings.
- **Bond-Taylor et al. [19]** (TPAMI 2022) compare VAEs, GANs, normalizing flows, energy-based and autoregressive models side by side, including the trade-offs in run time, sample diversity and architectural constraints. It places AR models in the wider generative-model landscape: exact likelihood and stable training, paid for with slow sequential sampling.
- **Li et al. [20]** (TPAMI 2026) survey discrete tokenization for multimodal LLMs, organizing vector-quantization variants and their use in classical, single-modality LLM and multimodal LLM systems, and discussing codebook collapse and unstable gradient estimation.
- **Jia et al. [22]** survey discrete tokenizers across generation, comprehension, recommendation and retrieval, dissecting tokenizers into their sub-modules and mechanisms before organizing methods by application, and listing limitations and future directions.
- **Zhao et al. [21]** survey unified multimodal understanding-and-generation models, grouping them into diffusion-based, autoregressive-based and hybrid AR–diffusion designs, which is where most 2025–2026 activity sits.

State of the art (2023–2026). Three lines stand out in the verified literature. (i) *Scaling plain AR transformers over better tokenizers*: LlamaGen [14] shows that a language-model architecture with almost no image-specific design is competitive on class-conditional ImageNet when the tokenizer and training are good. (ii) *Changing the prediction unit*: VAR's next-scale prediction [6] and masked decoding in MaskGIT and Muse [8], [17] reduce the number of sequential steps; MAR [7] removes vector quantization by using a diffusion head per token. (iii) *Unified multimodal models*: the same next-token machinery is used to generate text and image tokens in one model, with tokenizers designed to serve both understanding and generation [20], [21]. Our interpretation is that choosing the **representation and prediction unit** now matters at least as much as choosing the backbone.

Open problems. The surveys repeatedly point to the following:

- *Tokenizer quality.* Reconstruction caps generation quality; small text, faces and fine geometry are the usual casualties. Better codebook usage and semantic structure without losing detail remain active topics [20], [22].
- *Discrete versus continuous tokens.* Discrete tokens fit LLM machinery; continuous tokens reconstruct better but need a continuous conditional model such as a diffusion head [7].
- *Speed.* Sequential decoding is latency-bound. Next-scale, masked and parallel decoding trade steps for independence assumptions.
- *Evaluation.* Token likelihood, FID-type metrics, prompt adherence and human preference measure different things; a lossy tokenizer's token likelihood is not a pixel likelihood.
- *Video and long contexts.* Token counts explode over time, and temporal consistency, object permanence and drift need dedicated evaluation [1].

Key takeaways

- Autoregression is a factorization, not a synonym for "transformer": the chain rule is exact for any order, and the model's capacity and context decide how good each conditional is.
- Training is parallel under teacher forcing; sampling is sequential. Shifted inputs and causal masks (attention masks or PixelCNN mask A/B) must be designed together.
- Pixel models are exact but long; learned tokenizers (VQ-VAE, VQGAN, RQ-VAE, FSQ) shorten the sequence at the price of a reconstruction bottleneck that the generator cannot exceed.
- Token transformers (Image GPT, VQGAN transformer, Parti, LlamaGen) reuse language-model machinery, including classifier-free guidance on logits and KV caching.
- Next-scale (VAR), masked (MaskGIT, Muse) and continuous-token (MAR) models change the **unit** of prediction; fewer sequential steps usually mean more conditional-independence assumptions.
- Temperature, top-k and top-p reshape the sampling distribution; they trade diversity for fidelity but cannot add information lost by the tokenizer.
- AR and diffusion are complementary decompositions and are increasingly combined; compare them on matched data, tokenizers, guidance and compute.

Exercises

1. **Leak the answer.** Change the `BinaryAR` mask to include the diagonal (`diagonal=0`). Explain why the training loss can fall almost to zero while sampling breaks.

   Hint

   Each output can now read its own target and simply copy it, so the training loss collapses. At sampling time the current position is still 0 (not yet drawn) when its logit is computed, so the learned "copy" rule produces nonsense. The causality assertion in the snippet would also fail.

2. **Bits per dimension.** For the repeated-row 4×4 binary data, compute the bits per pixel of (a) the true distribution, (b) a model that treats every pixel as an independent fair coin, and (c) a model that learns the first row perfectly but treats later rows as independent fair coins.

   Hint

   (a) 4 bits / 16 pixels = 0.25. (b) 16 / 16 = 1.0. (c) 4 + 12 = 16 bits per image, also 1.0 bits per pixel: learning the first row's marginal does not help, because its bits are already uniform. All gains come from modelling the dependence of later rows on the first.

3. **Tokenizer budget.** A 512×512 image is tokenized with factor-8 or factor-16 downsampling. Compute the sequence length in each case and the ratio of full self-attention costs. What might you lose with factor 16?

Hint

$64 \times 64 = 4096$ versus $32 \times 32 = 1024$ tokens. Attention cost scales with length squared, so the ratio is 16. With factor 16 each token must describe a 16×16 patch, so small text and thin structures are more likely to be lost unless the codebook or decoder compensates.

4. **Blind spot.** Modify the masked-convolution snippet to use 7 layers and an 11×11 input with the target at the centre. How many earlier pixels are never seen? Then sketch how a vertical stack plus a horizontal stack removes them.

Hint

The blind spot is the staircase above and to the right of the target that the up-left-growing receptive field never reaches; counting zero-gradient cells that are earlier in raster order gives 10. A vertical stack sees full rows above (no left/right restriction) and is fed into the horizontal stack, which handles the current row only, so every earlier pixel becomes reachable.

5. **Masked decoding steps.** Using $n_t = \lceil \gamma(t/T)N \rceil$ with the cosine schedule, $N = 256$ and $T = 8$, compute how many tokens are fixed at each step. Why does the schedule fix few tokens first?

Hint

Remaining masked counts are 252, 237, 213, 182, 143, 98, 50 and 0, so 4, 15, 24, 31, 39, 45, 48 and 50 tokens are fixed per step (the figure uses floor, which shifts counts by at most one). In exact arithmetic $\gamma(1) = 0$; in floating point $\cos(\pi/2)$ is a tiny positive number, so a literal ceiling leaves one token masked, another reason the snippet uses floor. Early on the model has little context, so its parallel guesses are the least reliable and the most likely to conflict; fixing only a few lets later steps condition on them.

6. **Classify the methods.** For PixelCNN, VQGAN transformer, Parti, LlamaGen, VAR, MaskGIT and MAR, give the prediction unit, the representation (pixels, discrete tokens, continuous tokens, multiscale maps), the conditional distribution type, and the number of sequential steps for a 16×16 token grid.

Hint

PixelCNN: pixels in raster order, 256-way softmax, one step per value. VQGAN transformer, Parti, LlamaGen: discrete tokens in raster order, softmax over the codebook, 256 steps. VAR: multiscale discrete maps, softmax per token with parallel tokens inside a scale, one step per scale. MaskGIT: discrete tokens, bidirectional masked prediction, T steps (8 in the paper). MAR: continuous tokens in random-order groups, a per-token diffusion head, as many steps as groups (each with its own inner diffusion steps).

References

1. J. Xiong, G. Liu, L. Huang et al., "Autoregressive Models in Vision: A Survey," *Transactions on Machine Learning Research* (*TMLR*), 2025 (arXiv:2411.05902, 2024). arXiv
2. A. van den Oord, N. Kalchbrenner and K. Kavukcuoglu, "Pixel Recurrent Neural Networks," in *Proc. ICML*, PMLR 48, 2016. PMLR
3. A. van den Oord, N. Kalchbrenner, O. Vinyals, L. Espeholt, A. Graves and K. Kavukcuoglu, "Conditional Image Generation with PixelCNN Decoders," in *NeurIPS*, 2016. NeurIPS
4. A. van den Oord, O. Vinyals and K. Kavukcuoglu, "Neural Discrete Representation Learning," in *NeurIPS*, 2017. NeurIPS
5. P. Esser, R. Rombach and B. Ommer, "Taming Transformers for High-Resolution Image Synthesis," in *Proc. IEEE/CVF CVPR*, 2021. CVF
6. K. Tian, Y. Jiang, Z. Yuan, B. Peng and L. Wang, "Visual Autoregressive Modeling: Scalable Image Generation via Next-Scale Prediction," in *NeurIPS*, 2024 (Best Paper Award). arXiv
7. T. Li, Y. Tian, H. Li, M. Deng and K. He, "Autoregressive Image Generation without Vector Quantization," in *NeurIPS*, 2024. arXiv
8. H. Chang, H. Zhang, L. Jiang, C. Liu and W. T. Freeman, "MaskGIT: Masked Generative Image Transformer," in *Proc. IEEE/CVF CVPR*, 2022. CVF
9. S. J. D. Prince, *Understanding Deep Learning*, MIT Press, 2023. Book
10. A. Razavi, A. van den Oord and O. Vinyals, "Generating Diverse High-Fidelity Images with VQ-VAE-2," in *NeurIPS*, 2019. NeurIPS
11. M. Chen, A. Radford, R. Child, J. Wu, H. Jun, D. Luan and I. Sutskever, "Generative Pretraining From Pixels," in *Proc. ICML*, PMLR 119, 2020. PMLR
12. J. Yu, X. Li, J. Y. Koh et al., "Vector-quantized Image Modeling with Improved VQGAN," in *ICLR*, 2022. arXiv
13. J. Yu, Y. Xu, J. Y. Koh et al., "Scaling Autoregressive Models for Content-Rich Text-to-Image Generation," *Transactions on Machine Learning Research*, 2022. arXiv
14. P. Sun, Y. Jiang, S. Chen, S. Zhang, B. Peng, P. Luo and Z. Yuan, "Autoregressive Model Beats Diffusion: Llama for Scalable Image Generation," arXiv:2406.06525, 2024. arXiv
15. D. Lee, C. Kim, S. Kim, M. Cho and W.-S. Han, "Autoregressive Image Generation using Residual Quantization," in *Proc. IEEE/CVF CVPR*, 2022. CVF
16. F. Mentzer, D. Minnen, E. Agustsson and M. Tschannen, "Finite Scalar Quantization: VQ-VAE Made Simple," in *Proc. International Conference on Learning Representations* (*ICLR*), 2024. arXiv
17. H. Chang, H. Zhang, J. Barber et al., "Muse: Text-To-Image Generation via Masked Generative Transformers," in *Proc. ICML*, PMLR 202, 2023. PMLR
18. J. Ho and T. Salimans, "Classifier-Free Diffusion Guidance," arXiv:2207.12598, 2022 (short version at the NeurIPS 2021 Workshop on Deep Generative Models and Downstream Applications). arXiv
19. S. Bond-Taylor, A. Leach, Y. Long and C. G. Willcocks, "Deep Generative Modelling: A Comparative Review of VAEs, GANs, Normalizing Flows, Energy-Based and Autoregressive Models," *IEEE TPAMI*, vol. 44, no. 11, 2022. doi
20. J. Li, Y. Fu, J. Liu, L. Cao, W. Ji, M. Yang, I. King and M.-H. Yang, "Discrete Tokenization for Multimodal LLMs: A Comprehensive Survey," *IEEE TPAMI*, 2026 (arXiv:2507.22920). doi
21. S. Zhao, X. Zhang, J. Guo et al., "Unified Multimodal Understanding and Generation Models:

Advances, Challenges, and Opportunities," arXiv:2505.02567, 2025; revised 2026. arXiv
22. J. Jia, J. Gao, B. Xue et al., "From Principles to Applications: A Comprehensive Survey of Discrete Tokenizers in Generation, Comprehension, Recommendation, and Information Retrieval," arXiv:2502.12448, 2025. arXiv
23. J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li and L. Fei-Fei, "ImageNet: A Large-Scale Hierarchical Image Database," *CVPR*, 2009, pp. 248–255. doi
24. H. Touvron, T. Lavril, G. Izacard et al., "LLaMA: Open and Efficient Foundation Language Models," arXiv:2302.13971, 2023. arXiv

ML 09 • Foundation Models and Vision–Language Learning

What makes a model a foundation model: scaling laws, pre-training objectives, CLIP and SigLIP, DINOv2 features, promptable segmentation, multimodal language models, adaptation with linear probes and LoRA, and how to evaluate the risks.

Online edition: https://blog.shyandram.dev/en/learn/ml/ml-09-foundation-models/

What you'll learn

- What makes a model a foundation model, why the term describes training and reuse rather than an architecture, and why it need not generate text.
- What scaling laws say (a power law with a floor, compute-optimal allocation) and what they do not say.
- The main pre-training objectives — supervised, contrastive image–text, masked image modeling, self-distillation and generative — and the kind of features each tends to produce.
- How CLIP, SigLIP, DINOv2/DINOv3 and the Segment Anything family are used as reusable vision components.
- How multimodal large language models connect a vision encoder to a language model, and how LLaVA-style instruction tuning works.
- How to adapt a pretrained model — linear probe, fine-tuning, LP-FT, adapters, prompt tuning, LoRA, prompting — and how to evaluate the result honestly, including bias, hallucination and test-set contamination.

The big picture

For most of the history of computer vision, each task had its own model: one network for classifying flowers, another for detecting cars, another for segmenting tumors, each trained from scratch or from an ImageNet [43] classifier. A **foundation model** is trained once, on broad data and at large scale, and then **adapted** to many downstream tasks [1]. The word describes how the model is trained and reused, not a new layer type. A DINOv2 [15] or DINOv3 [16] image encoder, a CLIP-style image–text model [3], a promptable segmenter such as SAM [5], a multimodal language model and a broadly pretrained image generator can all play this role.

> **Explain like I'm five**
>
> Instead of teaching a new student from the beginning for every job, start with someone who has already looked at millions of pictures and read millions of captions. They already know what edges, wheels, faces and dogs look like. For a new job you only need to give them a few examples, a short instruction, or a small new tool — not a whole new education.

Why does this matter? Three reasons. First, **labels are expensive** and pretraining moves most of the learning to data that needs no hand labels (captions, the images themselves). Second, **one backbone serves many tasks**, which concentrates engineering effort. Third, it changes the **interface**: instead of retraining, you can often specify a task with a text prompt, a click or a few examples. Bommasani et al. call the resulting concentration of methods *homogenization*, and warn that it also concentrates failure: a flaw in one shared backbone is inherited by every system built on it [1].

This chapter builds on ML 05 ·Representation Learning (what makes a feature reusable), ML 04 ·Vision Transformers (the usual backbone) and the generative chapters ML 07 and ML 08. The textbook treatment of transfer learning and representation in Bishop and Bishop [14] is a good companion. We follow the architectural map of the vision foundation-model survey by Awais et al. [39] and the multimodal survey by Caffagni et al. [2], with representative reports verified for this October 2026 edition. Examples illustrate families; they do not rank commercial products or claim that one checkpoint is best everywhere.

What makes a model a "foundation" model

Plain version. A foundation model is a large model pretrained on broad data with a generic objective, so that its internal representations or its interface can be reused for tasks it was never specifically trained on.

Bommasani et al. [1] coined the term in 2021 and emphasized two properties:

- **Emergence.** Capabilities arise from scale and the training objective rather than being designed explicitly. For example, CLIP was trained only to match images with captions, yet it can classify images into categories named in text [3]; DINO was trained without any labels, yet its attention maps outline objects [24].
- **Homogenization.** Many applications converge on a small number of pretrained backbones and recipes.

Be careful with "emergence". In this chapter it means only *a capability that was not a training target*. It does not mean an unpredictable or mysterious jump; whether a capability appears suddenly often depends on the metric used to measure it.

Formally, let f_θ be a model pretrained on an upstream dataset $\mathcal{D}_{\text{up}}$ with objective $\mathcal{L}_{\text{pre}}$:

$$\theta^\star = \arg\min_\theta \; \mathbb{E}_{x \sim \mathcal{D}_{\text{up}}}[\mathcal{L}_{\text{pre}}(f_\theta; x)].$$

A downstream task k supplies a (usually small) dataset $\mathcal{D}_k$ and a task loss ℓ_k. Adaptation chooses a small set of new or changed parameters ϕ_k and solves

$$\phi_k^\star = \arg\min_{\phi_k} \sum_{(x,y)\in\mathcal{D}_k} \ell_k(h_{\phi_k}(f_{\theta^\star}, x), y),$$

where h_{ϕ_k} is whatever we put on top of or inside the pretrained model: a linear head, an adapter, a LoRA update, a prompt, or nothing at all (pure prompting, $\phi_k = \emptyset$). The whole chapter is about

the two lines above: what $\mathcal{L}_{\text{pre}}$ and $\mathcal{D}_{\text{up}}$ should be, and how small ϕ_k can be.

A foundation model does **not** have to produce text. A frozen image encoder whose features feed a hundred different classifiers is a foundation model. Conversely, a large model trained for one narrow task is not one, however big it is.

Scaling laws

Plain version. When you train bigger models on more data with more compute, the test loss usually falls smoothly and predictably — roughly as a straight line on a log–log plot — until it approaches a floor. That predictability is what lets labs plan large training runs from small ones.

Kaplan et al. [17] trained families of Transformer language models and reported that the loss scales as a power law in model size, dataset size and compute, with some trends spanning more than seven orders of magnitude, while width and depth mattered little within the tested range. A power law in model size N reads

$$L(N) = \left(\frac{N_c}{N}\right)^{\alpha_N},$$

where L is held-out cross-entropy loss, N the number of non-embedding parameters, N_c a fitted constant with the same units as N, and $\alpha_N > 0$ the fitted exponent. Taking logs gives $\log L = \alpha_N \log N_c - \alpha_N \log N$: a straight line with slope $-\alpha_N$.

Real curves bend because some loss can never be removed (the data itself is noisy or ambiguous). Hoffmann et al. [18] therefore fitted a form with an irreducible term:

$$L(N, D) = E + \frac{A}{N^{\alpha}} + \frac{B}{D^{\beta}},$$

where D is the number of training tokens, E the irreducible loss, and A, B, α, β fitted constants. For dense Transformers a common approximation of training compute is $C \approx 6ND$ floating-point operations [17] (roughly two for the forward and four for the backward pass, per parameter per token). Fixing C and minimizing L over N (with $D = C/6N$) gives a **compute-optimal** model size. Hoffmann et al. trained over 400 language models and concluded that, for compute-optimal training, model size and training tokens should be scaled equally — every doubling of model size should come with a doubling of tokens [18]. That differed from Kaplan et al.'s earlier recommendation of very large models trained on relatively modest data [17]; the disagreement is a useful reminder that fitted exponents depend on the experimental protocol (learning-rate schedules, which runs are included, what is counted as compute).

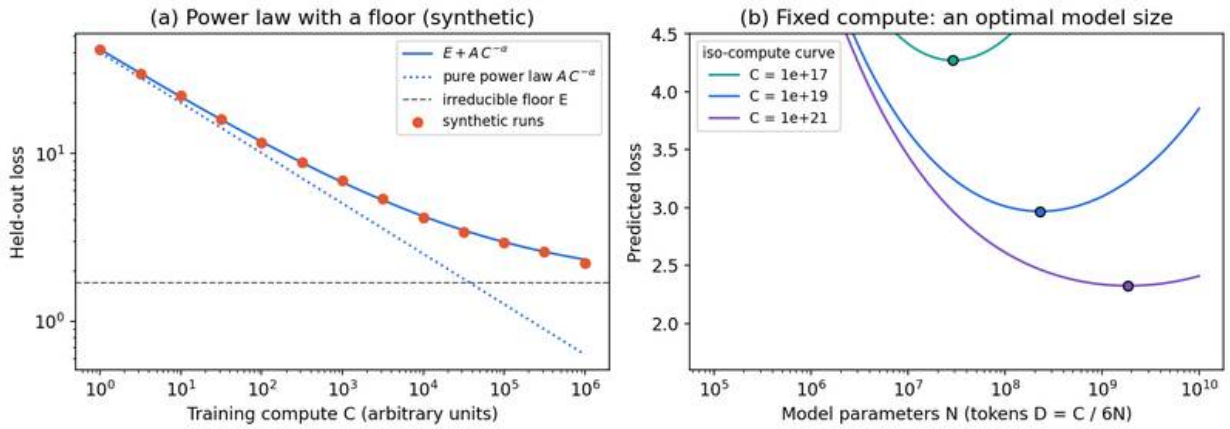


Figure 1 — (a) A power law with an irreducible floor E: a pure power law (dotted) keeps falling and over-promises at large scale. (b) With compute fixed, too small a model under-fits and too large a model sees too few tokens, so each budget has an optimal size. All constants are synthetic.

Vision has its own scaling studies. Zhai et al. [19] characterized the relationship between error rate, data and compute for Vision Transformers [21] trained on large labeled datasets, scaling to two-billion-parameter models. Cherti et al. [20] trained CLIP-style models on the public LAION dataset [34] and found power-law scaling for zero-shot classification, retrieval, linear probing and fine-tuning — but also that OpenAI's CLIP and OpenCLIP, with the same architectures and similar recipes, scale differently because their **training distributions** differ. Data is not a scalar.

Code: fit a scaling law and extrapolate

The snippet fits $L(C) = E + A\,C^{-\alpha}$ to the cheaper of 13 synthetic training runs and then predicts the expensive ones. Everything is synthetic; the point is the procedure.

```
import numpy as np
from scipy.optimize import curve_fit

rng = np.random.default_rng(0)
# Synthetic "runs": loss = E + A * C^(-alpha), with 2% multiplicative noise.
E_true, A_true, alpha_true = 1.7, 40.0, 0.3
compute = np.logspace(0, 6, 13)            # arbitrary compute units
loss = (E_true + A_true * compute**-alpha_true) * np.exp(.02 * rng.normal(size=13))

def law(c, E, A, alpha):
    return E + A * c**-alpha

# Fit only the cheaper runs, then extrapolate to the larger ones.
small = compute <= 1e4
(E, A, alpha), _ = curve_fit(law, compute[small], loss[small], p0=(1, 10, .5))
print(f"fit: E={E:.2f}  A={A:.1f}  alpha={alpha:.3f}")
for c, l in zip(compute[~small], loss[~small]):
    print(f"C={c:9.0f}  observed {l:.3f}  predicted {law(c, E, A, alpha):.3f}")
```

```
# Naive straight line in log-log space ignores the floor E.
slope = np.polyfit(np.log(compute[small]), np.log(loss[small]), 1)[0]
print(f"pure power-law slope without floor: {slope:.3f}")
```

Output: the fit recovers $E \approx 1.73$, $A \approx 40.0$, $\alpha \approx 0.300$; the extrapolated losses are within about 7% of the noisy observations; and a straight log–log line without the floor gives a slope of about -0.25, a wrong exponent. Two lessons transfer to real practice: always include the floor, and treat extrapolation over many orders of magnitude as a forecast with error bars.

What scaling laws do not tell you

- They predict **average loss** on the pretraining distribution, not accuracy on your task. Downstream metrics may improve smoothly, saturate, or move in steps.
- They hold **for a fixed recipe and data distribution** [20]. Better data filtering or a different objective changes the constants.
- They say nothing about **inference cost**. A smaller model trained on more data can be cheaper to deploy at the same loss, which is why many deployed models are trained beyond the compute-optimal point.
- Data is finite. If the high-quality data runs out, the B/D^{β} term stops shrinking however much compute you add.

Pre-training objectives: where does the target come from?

Plain version. Every pretraining method invents a prediction task. The difference is where the "answer" comes from: a human label, a caption, a hidden part of the same image, another network, or the next token.

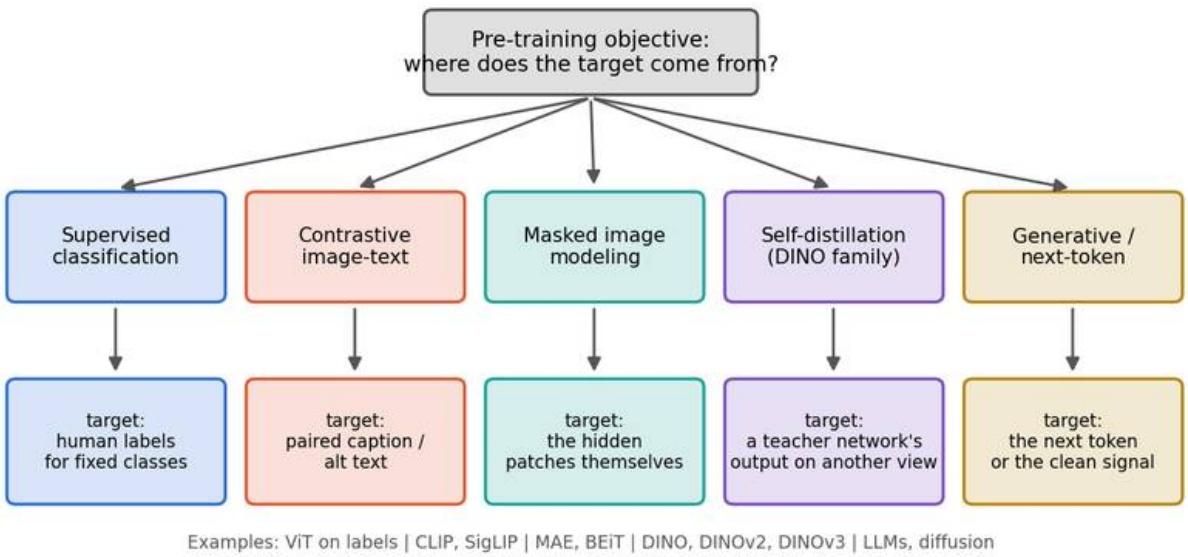


Figure 2 — Five families of pre-training objectives, organized by the source of the training target.

Objective	Target comes from	Representative models	Typical strength
Supervised classification	Human labels	ViT on large labeled sets [19], [21]	Strong global semantics for the label space
Contrastive image–text	Paired captions / alt text	CLIP [3], ALIGN [25], SigLIP [4]	Text-defined recognition, retrieval
Masked image modeling	Hidden patches of the same image	BEiT [23], MAE [22]	Strong after fine-tuning; local detail
Self-distillation	A teacher network's output on another view	DINO [24], DINOv2 [15], DINOv3 [16]	Frozen features, dense correspondence
Generative / next-token	Next token or clean signal	LLMs, diffusion (ML 07), AR (ML 08)	Generation; language interface

Supervised pre-training

Train a classifier with cross-entropy on a large labeled dataset, then drop the final layer. This was the ImageNet recipe for a decade and still works at scale [19], [21]. Its weakness is that the features are shaped by a fixed label set: distinctions that the labels do not require may be discarded.

Contrastive image–text learning

Captions come for free with web images, so CLIP [3] and ALIGN [25] train an image encoder and a text encoder together so that matching pairs score higher than mismatched ones within a batch. ALIGN showed that a large, noisy alt-text corpus with minimal filtering can still give strong representations [25]. The loss is detailed in the next section.

Masked image modeling

Hide most of an image and predict what is missing. BEiT [23] masks patches and predicts discrete visual tokens for them, borrowing the masked-language-modeling idea from BERT [44]. MAE [22] predicts the raw pixels directly and observes that masking a high proportion, e.g. 75% of patches, creates a nontrivial task; its encoder sees only the visible patches and a lightweight decoder reconstructs the rest, which makes training efficient. With $\mathcal{M}$ the set of masked patch indices, x_p the (normalized) pixels of patch p and $\hat{x}_p$ the reconstruction,

$$\mathcal{L}_{\text{MAE}} = \frac{1}{|\mathcal{M}|} \sum_{p \in \mathcal{M}} \|\hat{x}_p - x_p\|_2^2.$$

The loss is computed only on masked patches: predicting the visible ones would be trivial copying. Masked-modeling features are often excellent after fine-tuning but weaker as frozen features for linear probing than contrastive or self-distilled ones.

Self-distillation: the DINO family

DINO [24] — described by its authors as self-distillation with no labels — trains a **student** network g_s to match a **teacher** network g_t on different augmented views (crops) of the same image. Both output K-dimensional scores; probabilities are softmaxes with temperatures τ_s and τ_t:

$$P_s(x') = \text{softmax}\left(\frac{g_s(x')}{\tau_s}\right), \qquad P_t(x) = \text{softmax}\left(\frac{g_t(x) - c}{\tau_t}\right), \qquad \mathcal{L} = -\sum_{k=1}^{K} P_t(x)_k \log P_s(x')_k.$$

Here x and x' are two views, c is a running mean of teacher outputs (**centering**), and a small τ_t makes the teacher's distribution sharp (**sharpening**). Centering stops one dimension from dominating; sharpening stops the output from becoming uniform; together they avoid **collapse**, the trivial solution where every image maps to the same output. The teacher receives no gradients. Its weights are an exponential moving average (the momentum encoder) of the student's:

$$\theta_t \leftarrow \lambda\, \theta_t + (1 - \lambda)\, \theta_s, \qquad 0 < \lambda < 1.$$

DINO also uses multi-crop training (several small local crops for the student, global crops for the teacher), which forces local-to-global correspondence. Its authors report that the resulting ViT features contain explicit information about object segmentation that does not emerge as clearly with supervised ViTs or convnets, and that they work well as k-nearest-neighbor classifiers [24].

DINOv2 [15] scales this recipe and, crucially, curates the data: an automatic pipeline builds a dedicated, diverse, curated image dataset instead of using uncurated web images, a 1-billion-parameter ViT is trained, and it is distilled into smaller models. The authors report that these frozen features surpass OpenCLIP on most of their image-level and pixel-level benchmarks [15]. DINOv3 [16] scales data and model further and introduces **Gram anchoring**, a technique aimed at a known problem: dense (per-patch) feature maps degrading during long training schedules. It also adds post-hoc strategies for flexibility in resolution, model size and alignment with text.

Why dense features matter. A classifier needs one vector per image. Segmentation, depth estimation, correspondence and tracking need a good vector **per patch**. Self-distilled encoders are popular frozen backbones for these dense tasks precisely because their patch features stay semantically meaningful without fine-tuning.

Generative pre-training

Predicting the next token (language models, autoregressive image models in ML 08) or denoising (diffusion in ML 07) is also a pretraining objective. For language it produces the most flexible interface — instructions in natural language — which is why multimodal models attach vision to a pretrained language model rather than the other way round.

Image and text in a shared space: CLIP and SigLIP

Plain version. Train an image encoder and a text encoder so that corresponding pictures and captions point in similar directions. Then any sentence becomes a "class".

CLIP [3] normalizes image embeddings v_i and text embeddings t_j to unit length and forms scores $s_{ij} = v_i^\top t_j / \tau$ with a learned temperature τ. For a batch of B paired examples,

$$\mathcal{L}_{\text{CLIP}} = \frac{1}{2}\left[\frac{1}{B}\sum_i -\log \frac{\exp(s_{ii})}{\sum_j \exp(s_{ij})} + \frac{1}{B}\sum_j -\log \frac{\exp(s_{jj})}{\sum_i \exp(s_{ij})}\right].$$

Each image should select its matching caption among the B captions (first term, a row-wise softmax), and each caption its matching image (second term, column-wise). All other pairs in the batch act as negatives. Training uses paired text supervision, usually from existing captions or alt text; "zero-shot" downstream use does not mean the model was trained without supervision.

At inference, encode candidate descriptions such as "a photo of a dog" and compare an image with them. The text embeddings replace the class weights of a learned classifier:

$$\hat{y} = \arg\max_k v(x)^\top t(\text{prompt}_k).$$

The candidate set and wording affect the result; averaging the embeddings of several templates ("a photo of a k", "a close-up of a k") is a common way to reduce sensitivity. Zero-shot means no task-specific parameter training under the stated protocol; it does not guarantee that the classes or even the test images were absent from pretraining (see *Risks and evaluation*).

Code: paired loss and retrieval mechanics

These **synthetic embeddings** make the computation transparent. They are not outputs from a pretrained model, and their matches do not demonstrate real-world zero-shot accuracy.

```
import torch
from torch.nn import functional as F

images = F.normalize(torch.tensor([[1., .1, 0.], [.1, 1., 0.],
                                   [0., .1, 1.]]), dim=-1)
texts = F.normalize(torch.eye(3), dim=-1)
scores = images @ texts.T / .1
labels = torch.arange(3)
loss = (F.cross_entropy(scores, labels) +
        F.cross_entropy(scores.T, labels))/2
print("matching text:", scores.argmax(1).tolist())
print("paired loss:", round(loss.item(), 4))
ambiguous = F.normalize(torch.tensor([[.5, .5, .2]]), dim=-1)
print("cosines:", (ambiguous @ texts.T).tolist())
```

It prints `matching text: [0, 1, 2]`, a paired loss of about `0.0002`, and cosines of about `0.68 , 0.68, 0.27` for the ambiguous image: it is equally close to two descriptions, and no softmax can tell you which is right.

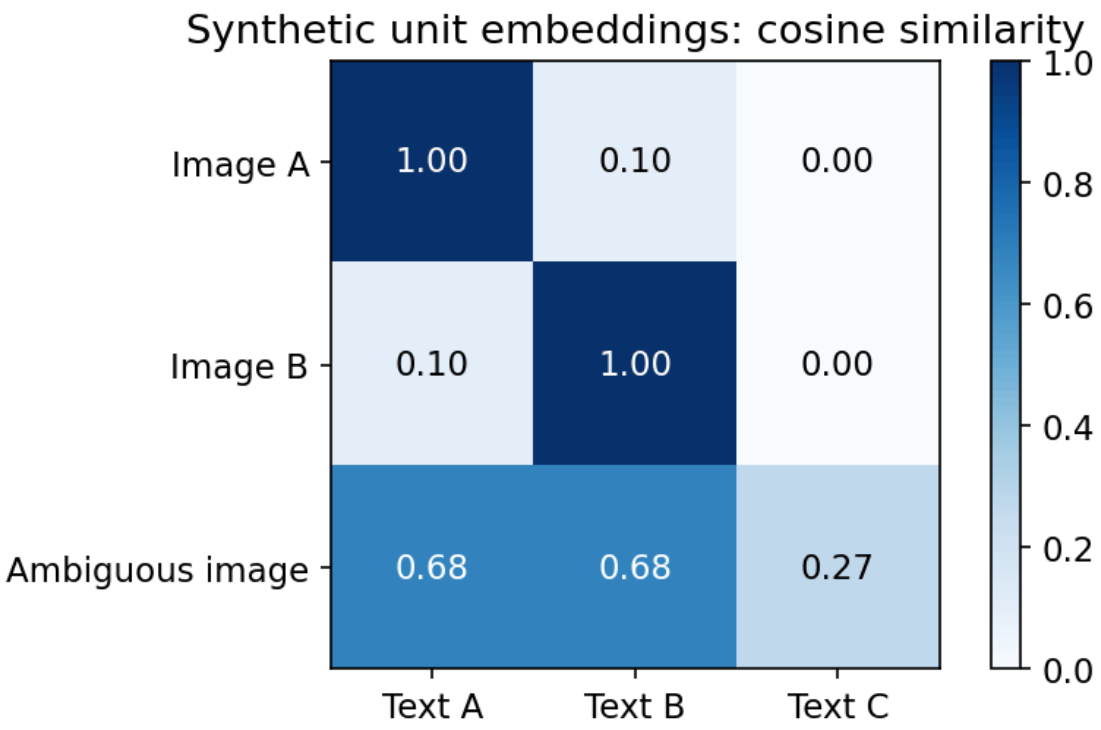


Figure 3 — Cosine similarity ranks candidates. It is not a calibrated probability that a concept is present.

SigLIP: a sigmoid instead of a softmax

SigLIP [4] replaces the batch-normalized softmax with an independent binary decision for every image–text pair. With labels $y_{ij} = +1$ for matching pairs and -1 otherwise, a learned scale a and bias b,

$$\mathcal{L}_{\text{SigLIP}} = -\frac{1}{B}\sum_{i=1}^{B}\sum_{j=1}^{B}\log\sigma(y_{ij}(a\,v_i^{\top}t_j + b)),$$

where $\sigma(u) = 1/(1+e^{-u})$, so each term equals $\log(1+\exp(-y_{ij}(a\,v_i^{\top}t_j+b)))$. Because no term needs a sum over the whole batch, the loss does not require a global normalization across devices, which simplifies large-batch training. Since there are B positives and $B^2 - B$ negatives, the bias b is initialized so that the initial predictions are strongly negative.

Shared embeddings support retrieval and open-vocabulary recognition, but fine-grained counting, negation ("a dog **without** a collar"), spatial relations and rare domain terms are known weak points. Evaluate those requirements directly rather than inferring them from a strong overall classification score.

Promptable segmentation: the Segment Anything family

Plain version. A prompt tells the model which region or concept you want, and the output is a mask.

SAM [5] combines a heavy image encoder (run once per image), a light prompt encoder and a light mask decoder, so many prompts can be answered quickly on the same image. Spatial prompts — points, boxes or rough masks — select objects without a fixed class vocabulary. Because a single click is ambiguous (a click on a shirt could mean the shirt or the whole person), SAM predicts several candidate masks with confidence scores. It was trained on SA-1B, over 1 billion masks on 11 million licensed and privacy-respecting images, collected with a model-in-the-loop data engine [5].

SAM 2 [6] extends promptable segmentation to video with a streaming memory: the model stores features of past frames and of the prompted object, and propagates the mask through time; corrective clicks on later frames update the memory.

SAM 3 [7], released in 2025 with a report revised in March 2026, introduces **promptable concept segmentation**: given a short noun phrase (e.g. "yellow school bus"), image exemplars, or both, it returns masks and identities for **all** matching instances. Its image-level detector and memory-based video tracker share one backbone, and a **presence head** decouples recognizing whether the concept occurs at all from localizing its instances [7].

These interfaces supply different information: a spatial point says *where*, a noun phrase says *what*, an exemplar says *what it looks like*. Report prompt creation cost and whether prompts come from people, ground truth or an automatic detector. A "zero-shot segmentation" score obtained with ground-truth boxes as prompts hides substantial annotation information.

Multimodal large language models

Plain version. Turn visual features into "words" a language model can read, then teach it to follow visual instructions.

A common design has

$$\text{image/video} \xrightarrow{\text{vision encoder}} h \xrightarrow{\text{connector}} z \xrightarrow{\text{language decoder with text}} \text{answer}.$$

Here h is a grid of visual features (for a ViT, one vector per patch), z is a sequence of vectors in the language model's token-embedding space, and the decoder treats z like the embeddings of ordinary words.

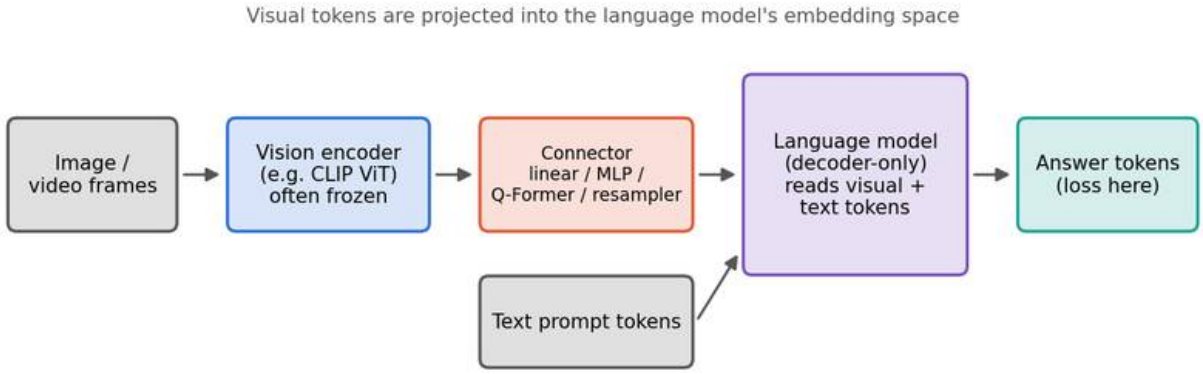


Figure 4 — The typical multimodal LLM pipeline. Visual features are converted into tokens in the language model's embedding space.

Connectors

- **Linear or MLP projection.** LLaVA [8] uses a single projection from CLIP features to the language model's embedding space; LLaVA-1.5 [28] reports that a simple MLP connector (with a CLIP ViT-L at 336-pixel resolution and academic VQA data) is surprisingly powerful and data-efficient. Every patch becomes one visual token.
- **Learned queries.** BLIP-2 [26] inserts a lightweight Querying Transformer (Q-Former) between a frozen image encoder and a frozen language model. A fixed set of learned query vectors cross-attends to the image features, so the language model always sees a small, fixed number of visual tokens. BLIP-2 trains the Q-Former in two stages: representation learning against the frozen image encoder, then generative learning against the frozen language model.
- **Resampling and cross-attention.** Flamingo [27] bridges frozen vision and language models and handles arbitrarily interleaved sequences of images and text, which enabled few-shot learning from examples written directly in the prompt.

The trade-off is token count versus detail: one token per patch keeps fine detail but makes long sequences (expensive for many images or video); a few learned queries are cheap but can lose small text.

Visual instruction tuning

LLaVA [8] used a language-only GPT-4 [45] to generate multimodal instruction-following data from image captions and boxes, then trained in two stages: first align the connector (feature alignment), then fine-tune connector and language model on the instructions. The loss is ordinary next-token cross-entropy, applied to the **answer** tokens only:

$$\mathcal{L} = -\sum_{i \in \mathcal{A}} \log p_\theta(w_i \mid z,\ w_{<i}),$$

where w_i are text tokens, $\mathcal{A}$ is the set of positions belonging to the assistant's answer, and z are the visual tokens. Instruction and prompt tokens are visible as context but masked out of the loss.

Qwen3-VL's 2025 report [9] illustrates the current direction: a family of dense and mixture-of-experts models with natively interleaved image, video and text contexts of up to 256K tokens. It is a representative report, not evidence that every task benefits from longer contexts or more generated reasoning.

What the diagram hides

Reading a document may require preserving small text through tiling or dynamic resolution. Video questions need timestamps and temporal order. These details change token counts and preprocessing; a generic "vision encoder + LLM" diagram is only the starting point. A fluent answer can also contain objects that are not in the image (**object hallucination**) or wrong counts. Distinguish object grounding, OCR, spatial reasoning, factual answering and stylistic instruction following — they need different evidence and metrics (see *Risks and evaluation*). For a survey of the whole design space, see Yin et al. [40] and Caffagni et al. [2].

Adapting a foundation model

Plain version. Use the least expensive adaptation that solves the target task, and check it on data that looks like deployment.

Strategy	Trainable components	Useful starting situation
Prompting / in-context examples	Nothing	The model's interface already fits the task
Frozen features + linear head (linear probe)	Small supervised readout	Few labels, good feature transfer
Prompt tuning / visual prompt tuning	A few learned input tokens	Many tasks share one frozen backbone
Adapters / LoRA	Small inserted or low-rank modules	Domain adaptation under memory limits
Partial fine-tuning	Selected blocks plus head	Some adaptation without full optimizer cost
Full fine-tuning (optionally LP-FT)	All model parameters	Enough data and compute; transfer alone insufficient

Linear probe

Freeze the encoder f, compute features once, and fit a linear classifier $\hat{y} = \mathrm{softmax}(Wf(x) + b)$. It is cheap, hard to overfit, and is also the standard **measurement** of representation quality: if a linear readout works, the information is linearly accessible in the features.

Code: why pretrained features help most with few labels

This toy experiment simulates the whole story. Observations are 300-dimensional and mix 8 **semantic** factors (which determine the class) with 8 large **nuisance** factors (think lighting and pose) plus noise. "Pretraining" sees 50 upstream classes with plenty of labels and learns an 8-dimensional projection (linear discriminant analysis, standing in for a deep encoder). Downstream, there are four **new** classes and only a few labels per class.

```
import numpy as np
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.linear_model import LogisticRegression

rng = np.random.default_rng(0)
D, k_sem, k_nui = 300, 8, 8
sem_map = rng.normal(size=(k_sem, D))   # semantic factors -> pixels
nui_map = rng.normal(size=(k_nui, D))   # nuisance factors (light, pose...)

def sample(centers, n):
    y = rng.integers(len(centers), size=n)
    z = centers[y] + 0.7 * rng.normal(size=(n, k_sem))
    u = 4.0 * rng.normal(size=(n, k_nui))          # large nuisance variation
    x = z @ sem_map + u @ nui_map + rng.normal(size=(n, D))
    return x, y

# 1) "Pre-training": many upstream classes with plenty of labels.
up_centers = rng.normal(size=(50, k_sem))
x_up, y_up = sample(up_centers, 10_000)
encoder = LinearDiscriminantAnalysis(n_components=k_sem).fit(x_up, y_up)

# 2) Downstream: four NEW classes, few labels, linear probe only.
new_centers = rng.normal(size=(4, k_sem))
x_te, y_te = sample(new_centers, 2_000)
f_te = encoder.transform(x_te)          # frozen features, computed once
for shots in [1, 5, 20, 100]:
    raw, feat = [], []
    for repeat in range(10):
        x_tr, y_tr = sample(new_centers, 4 * shots)
        raw.append(LogisticRegression(max_iter=3000)
                   .fit(x_tr, y_tr).score(x_te, y_te))
        feat.append(LogisticRegression(max_iter=3000)
                    .fit(encoder.transform(x_tr), y_tr).score(f_te, y_te))
    print(f"{shots:>3} labels/class | raw pixels {np.mean(raw):.3f}"
          f" | frozen features {np.mean(feat):.3f}")
```

On our machine it prints accuracies of roughly 0.29 vs 0.71 with one label per class, 0.62 vs 0.95 with five, 0.96 vs 0.99 with twenty and 0.98 vs 0.99 with a hundred (raw inputs vs frozen features; chance is 0.25). The pretrained projection already discards the nuisance directions, so a handful of labels suffice; with enough labels, the raw-input probe learns to ignore the nuisance itself and the gap closes. The upstream classes are different from the downstream ones — the features transfer because both share the same semantic factors. If the downstream task depended on a

factor that pretraining threw away (here, the nuisance), frozen features would fail no matter how many labels you add. That is the failure mode to test for in practice.

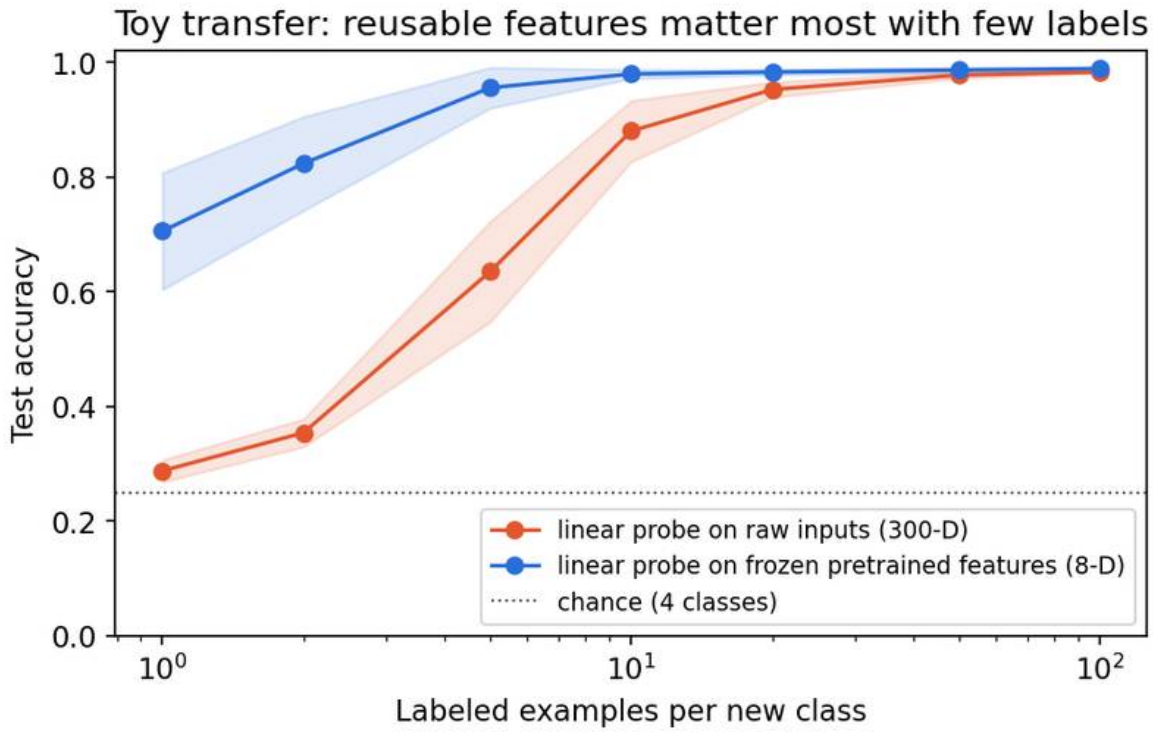


Figure 5 — The toy transfer experiment over more label budgets (mean ± one standard deviation over 10 draws). Reusable features matter most when labels are scarce.

Fine-tuning, LP-FT and robust fine-tuning

Full fine-tuning updates every weight and usually gives the best in-distribution accuracy when data suffices. But Kumar et al. [32] showed that it can distort good pretrained features: across their benchmarks, fine-tuning gave on average higher in-distribution but lower out-of-distribution accuracy than linear probing. Their explanation is that a randomly initialized head sends large, noisy gradients into the backbone early in training. **LP-FT** — first fit a linear probe, then fine-tune everything starting from that head — reduced this distortion and outperformed both alternatives in their experiments [32]. **WiSE-FT** [33] takes another route: linearly interpolate the weights of the zero-shot and fine-tuned models, $\theta_\gamma = (1-\gamma)\theta_{\text{zero-shot}} + \gamma\,\theta_{\text{ft}}$, which improved robustness under distribution shift for CLIP-style models while keeping strong target accuracy. Both are cheap to try and worth knowing before declaring that fine-tuning "hurt robustness".

Parameter-efficient fine-tuning (PEFT)

Adapters [29] insert small bottleneck MLPs inside each Transformer block and train only them, adding a few parameters per task while the original network stays fixed. **Prompt tuning** learns a few extra input tokens; **Visual Prompt Tuning (VPT)** [30] does this for ViTs, prepending learnable tokens to the patch sequence of a frozen backbone. Xu et al. [42] review the PEFT landscape with a taxonomy of additive, partial, reparameterized, hybrid and unified fine-tuning, plus experiments on parameter and memory efficiency.

LoRA [10] reparameterizes an update to a frozen weight $W_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ as

$$W = W_0 + \frac{\alpha}{r} BA, \qquad A \in \mathbb{R}^{r \times d_{\text{in}}}, \quad B \in \mathbb{R}^{d_{\text{out}} \times r},$$

with rank $r \ll \min(d_{\text{in}}, d_{\text{out}})$ and a scaling constant α. Freeze W_0 and learn A, B. Trainable weights drop from $d_{\text{out}} d_{\text{in}}$ to $r(d_{\text{in}} + d_{\text{out}})$. Initialize $B = 0$ so the initial update is zero, and A randomly so learning can start. After training, BA can be merged into W_0, so inference costs nothing extra. QLoRA [31] stores the frozen base weights in 4-bit precision and backpropagates through them into LoRA adapters, cutting memory enough to fine-tune very large models on a single GPU.

Code: a low-rank update with a frozen base

This standalone example trains one adapter step on synthetic input. It shows the parameterization and initial behavior; it does not reproduce a full pretrained-model fine-tuning pipeline.

```
import torch
from torch import nn
from torch.nn import functional as F

torch.manual_seed(0)
class LoRALinear(nn.Module):
    def __init__(self, din, dout, rank=4, alpha=4):
        super().__init__()
        self.base = nn.Linear(din, dout, bias=False)
        self.base.weight.requires_grad_(False)
        self.A = nn.Parameter(torch.randn(rank, din) * .02)
        self.B = nn.Parameter(torch.zeros(dout, rank))
        self.scale = alpha/rank
    def forward(self, x):
        return self.base(x) + self.scale * F.linear(F.linear(x, self.A), self.B)

layer = LoRALinear(32, 16)
x = torch.randn(8, 32)
assert torch.allclose(layer(x), layer.base(x))
opt = torch.optim.Adam([layer.A, layer.B], lr=.01)
loss = F.mse_loss(layer(x), torch.randn(8, 16))
opt.zero_grad(); loss.backward(); opt.step()
assert layer.base.weight.grad is None
print("trainable:", sum(p.numel() for p in layer.parameters() if p.requires_grad))
```

It prints `trainable: 192`, which is $r(d_{\text{in}} + d_{\text{out}}) = 4 \times (32 + 16)$, against 512 for the full matrix. The first assertion checks that the zero-initialized B leaves the model unchanged at the start; the second, that the frozen base receives no gradient.

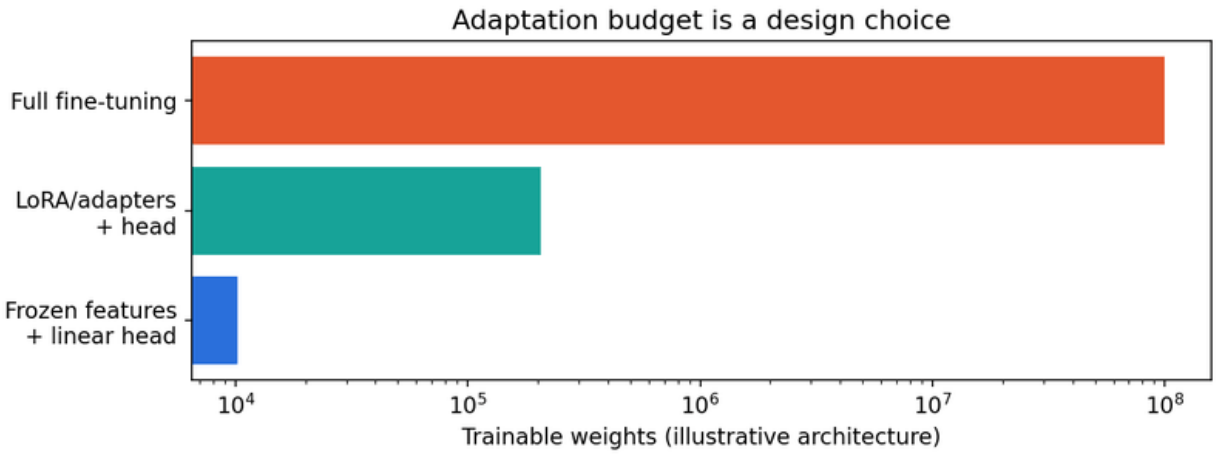


Figure 6 — Analytical trainable-weight budgets for an illustrative architecture (feature size 1024, 10 classes, rank 8 on 12 matrices, 100M parameters in total). Activation memory, frozen weight storage and optimizer state must be counted separately.

LoRA is a parameterization, not a guarantee against overfitting or forgetting. Adapters still require domain validation. Frozen BatchNorm statistics, image normalization, output-label mapping and checkpoint-specific preprocessing also need deliberate treatment.

Prompting and in-context learning

The cheapest adaptation changes no weights: write a better text prompt for CLIP, give SAM a box, or put a few solved examples into a multimodal model's context, as Flamingo did [27]. The vision–language survey by Zhang et al. [41] compares prompting, feature adaptation and fine-tuning across vision tasks. Prompting is fast to iterate but brittle: small wording changes can move results, so fix and report the prompt protocol.

Video, geometry and world models

Plain version. Beyond recognizing a frame, a model may predict change over time, infer three-dimensional structure, or support action planning.

V-JEPA 2 [11] studies self-supervised video models that predict in feature space rather than pixel space, with an action-conditioned extension used for planning. VGGT [12] predicts camera parameters, depth, point maps and tracks from one or many views in a single feed-forward pass. These illustrate two routes beyond classification: temporal state modeling and geometric inference.

A generated video that looks convincing need not obey physical constraints. A scene reconstruction can be useful without generating photorealistic pixels. Measure multi-view consistency, geometry, action-conditioned outcomes and temporal behavior according to the claimed capability. DLCV 07 ·World Models develops partial observability, learned dynamics and planning in more detail.

Risks and evaluation

Plain version. A model trained on a large slice of the internet inherits that slice's biases and gaps, may confidently describe things that are not there, and may already have seen your "test" set. Evaluation has to be designed around these facts.

Data provenance, bias and privacy

Web-scale image–text datasets are too large to inspect by hand. LAION-5B [34] made such a dataset public (5.85 billion CLIP-filtered pairs) and thereby enabled open replication such as OpenCLIP [20]; audits of its predecessor LAION-400M by Birhane et al. [35] documented explicit and stereotyping content in image–text pairs. Downstream systems inherit these associations. Licenses and consent also matter: training data, model weights and generated outputs may each carry different terms, and personal data (faces, names, medical images) raises privacy obligations that do not disappear because data was public. SAM's SA-1B is an example of a dataset whose description emphasizes licensed and privacy-respecting images [5]; record such properties for every model you deploy.

Hallucination

Multimodal LLMs can mention objects that are absent from the image. POPE [37] measures object hallucination with a polling approach: it asks yes/no questions about the presence of specific objects (including frequent and co-occurring distractors), which makes the measurement less dependent on the model's generation style than scoring free-form captions. A low POPE error does not cover counting, attributes or relations; test those separately.

Contamination and "zero-shot"

If benchmark images (or near-duplicates) are in the pretraining set, a "zero-shot" score partly measures memory. Mayilvahanan et al. [36] asked whether CLIP's generalization mainly stems from high train–test similarity; after pruning LAION to remove images too similar to common out-of-distribution benchmarks and retraining, performance dropped on some benchmarks but remained strong overall. The lesson is two-sided: similarity explains part of the reported performance, not all of it — so measure it rather than assume either way. Practical checks: deduplicate the test set against available training data, report results on data collected after the model's data cutoff, and state the full zero-shot protocol (class names, templates, candidate set).

Benchmarks

Aggregate multimodal benchmarks such as MMMU [38] — 11.5K college-level questions from exams, quizzes and textbooks across six disciplines, with charts, diagrams, maps, tables and other image types — test knowledge and reasoning, but a single number mixes perception, OCR, domain knowledge and answer formatting. For a deployment, build a small, task-specific evaluation set with known failure cases, report variance over seeds or prompt wordings, and compare against a simple baseline (a linear probe on frozen features is often surprisingly strong). ML 10 · Evaluation and Deployment covers calibration, splits and deployment in depth.

Modern view

Surveys worth reading.

- **Bommasani et al. [1]** — the report that named foundation models. It covers capabilities (language, vision, robotics, reasoning), applications (law, health, education), technology (modeling, training, adaptation, evaluation, data) and societal impact. Its key takeaways — emergence, homogenization, and inherited failure — still frame the field.
- **Awais et al. [39]** — a computer-vision survey organizing foundation models by architecture design, training objective (contrastive versus generative), datasets, fine-tuning mechanisms and prompting, and listing open challenges in evaluation, real-world applicability, adversarial robustness and interpretability.
- **Zhang et al. [41]** — a TPAMI survey of vision–language models for vision tasks: network architectures, pre-training objectives, downstream tasks, datasets, transfer learning and knowledge distillation, with benchmarking of reviewed methods.
- **Yin et al. [40]** and **Caffagni et al. [2]** — surveys of multimodal large language models: architecture (encoder, connector, LLM), training strategy and data (pre-training, instruction tuning, alignment), evaluation, and hallucination.
- **Han et al. [13]** — a survey broadening the map to multimodal *generative* models, organized by generative category and technique.
- **Xu et al. [42]** — a critical review of parameter-efficient fine-tuning that sorts methods into additive, partial, reparameterized, hybrid and unified families and compares representative ones experimentally.

State of the art, 2023–2026. Four trends stand out. (1) **Better frozen features**: DINOv2 [15] and DINOv3 [16] made self-supervised encoders strong enough that many dense tasks start from frozen patch features. (2) **Concept-level prompting**: segmentation moved from "click an object" (SAM [5]) to video memory (SAM 2 [6]) to "segment every instance of this phrase" (SAM 3 [7]). (3) **Long, interleaved multimodal contexts**: models such as Qwen3-VL [9] read many images, video and text together. (4) **Beyond recognition**: video prediction for planning (V-JEPA 2 [11]) and feed-forward geometry (VGGT [12]) treat foundation models as world and scene estimators.

Open problems.

- Grounded reliability: reducing hallucination and making models say "not visible" when ap-

propriate [37].
- Honest evaluation under contamination and benchmark saturation [36].
- Data: curation as a scaling axis [15], [20], provenance, licensing and consent [34], [35].
- Efficient adaptation and inference: token budgets for high-resolution images and video; PEFT that preserves robustness [32], [33].
- Dense, 3D and temporal understanding that matches the semantic strength of image-level models.

For any new "foundation" model, record its input/output interface, training objective, data sources, supported adaptation, checkpoint license, preprocessing and task-level evaluation. This avoids treating model size or an open-ended chat interface as proof of general competence.

Key takeaways

- "Foundation" describes broad pretraining plus reuse, not a particular architecture; such models need not generate text.
- Scaling laws describe smooth, power-law-like loss improvements with a floor; compute-optimal training balances model size and data; the constants depend on data and recipe.
- The pretraining objective decides what the features are good at: contrastive for text-defined recognition, masked modeling for fine-tuning, self-distillation for frozen dense features, generative for flexible interfaces.
- CLIP-style alignment enables retrieval and text-defined classification; cosine similarity is not calibrated confidence.
- Spatial, conceptual and language prompts provide different kinds of information; report who made the prompts.
- Multimodal LLMs = vision encoder + connector + language model, trained with answer-only next-token loss; the connector sets a detail-versus-token-count trade-off.
- Start adaptation cheap (prompt, linear probe), then PEFT (LoRA, adapters), then full fine-tuning; LP-FT and weight interpolation help preserve robustness.
- Evaluate for bias, hallucination and contamination explicitly; one aggregate benchmark number is not evidence of general competence.

Exercises

1. **Candidate dependence.** Add a fourth description to the synthetic CLIP example. Can the softmax "confidence" for an image change without changing the image?

   Hint

   Yes: the denominator of the softmax changes. Similarity to a fixed description is unchanged, but the normalized probability depends on the candidate set. Try adding a text vector close to the ambiguous image's embedding.

2. **Compute-optimal allocation.** Using the parametric law $L(N, D) = E + A/N^{\alpha} + B/D^{\beta}$ and $C = 6ND$, show that the optimal model size grows as $N^{\star} \propto C^{\beta/(\alpha+\beta)}$. What does $\alpha = \beta$ imply?

Hint

Substitute $D = C/(6N)$, differentiate with respect to N and set to zero: $\alpha A N^{-\alpha-1} = \beta B (6N/C)^{\beta}/N$. Solving gives $N^{\alpha+\beta} \propto C^{\beta}$. With $\alpha = \beta$, $N^{\star} \propto C^{1/2}$ and therefore $D^{\star} \propto C^{1/2}$ too: scale parameters and tokens equally, as Hoffmann et al. concluded.

3. **Break the transfer.** In the linear-probe experiment, make the downstream class depend on the *nuisance* factors instead of the semantic ones. What happens to the frozen-feature probe as labels increase, and what does that say about choosing a foundation model?

Hint

Draw class centers for `u` instead of `z`. The frozen 8-D features were trained to discard the nuisance directions, so they stay near chance however many labels you add, while the raw-input probe improves. A backbone is only useful if its pretraining kept the information your task needs — check with a linear probe before investing in adaptation.

4. **LoRA budget.** A ViT-B block has four 768×768 attention projections and two MLP matrices of sizes 3072×768 and 768×3072. Count trainable weights for full fine-tuning of these six matrices and for rank-8 LoRA on all of them.

Hint

Full: $4{\cdot}768^2 + 2{\cdot}3072{\cdot}768 = 2{,}359{,}296 + 4{,}718{,}592 = 7{,}077{,}888$. LoRA: $8{\cdot}[4{\cdot}(768{+}768) + 2{\cdot}(3072{+}768)] = 8 \cdot (6144 + 7680) = 110{,}592$, about 1.6% of the full count.

5. **Prompt fairness.** Design a SAM evaluation that separates mask quality from prompt quality.

Hint

Fix the prompt protocol and count human interactions. Evaluate automatic proposals separately from masks produced with ideal ground-truth prompts, and report both. For SAM 3, compare noun-phrase prompts with exemplar prompts on the same images.

6. **Contamination audit.** You are given a new image–text model that reports high zero-shot accuracy on a public benchmark. List three checks you would run before trusting the number.

Hint

(a) Near-duplicate search of test images against any available training data (e.g. by embedding similarity); (b) evaluation on a freshly collected set from after the data cutoff; (c) sensitivity to the prompt templates and the candidate class list; optionally compare with a linear probe and a simple baseline.

References

1. R. Bommasani et al., "On the Opportunities and Risks of Foundation Models," Stanford CRFM report, arXiv:2108.07258, 2021; revised 2022. arXiv
2. D. Caffagni et al., "The Revolution of Multimodal Large Language Models: A Survey," *Findings of ACL*, 2024. arXiv
3. A. Radford et al., "Learning Transferable Visual Models From Natural Language Supervision," *ICML*, 2021. arXiv
4. X. Zhai, B. Mustafa, A. Kolesnikov and L. Beyer, "Sigmoid Loss for Language Image Pre-Training," *ICCV*, 2023. arXiv
5. A. Kirillov et al., "Segment Anything," *ICCV*, 2023. arXiv
6. N. Ravi et al., "SAM 2: Segment Anything in Images and Videos," *ICLR*, 2025. arXiv
7. N. Carion et al., "SAM 3: Segment Anything with Concepts," arXiv:2511.16719, 2025; revised March 2026. arXiv
8. H. Liu, C. Li, Q. Wu and Y. J. Lee, "Visual Instruction Tuning," *NeurIPS*, 2023. arXiv
9. S. Bai et al., "Qwen3-VL Technical Report," arXiv:2511.21631, 2025. arXiv
10. E. J. Hu et al., "LoRA: Low-Rank Adaptation of Large Language Models," *ICLR*, 2022. arXiv
11. M. Assran et al., "V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning," arXiv:2506.09985, 2025. arXiv
12. J. Wang et al., "VGGT: Visual Geometry Grounded Transformer," *CVPR*, 2025. arXiv
13. L. Han et al., "A Survey of Generative Categories and Techniques in Multimodal Generative Models," arXiv:2506.10016, 2025. arXiv
14. C. M. Bishop and H. Bishop, *Deep Learning: Foundations and Concepts*, Springer, 2024. Book
15. M. Oquab et al., "DINOv2: Learning Robust Visual Features without Supervision," *Transactions on Machine Learning Research*, 2024. arXiv
16. O. Siméoni et al., "DINOv3," arXiv:2508.10104, 2025. arXiv
17. J. Kaplan et al., "Scaling Laws for Neural Language Models," arXiv:2001.08361, 2020. arXiv
18. J. Hoffmann et al., "Training Compute-Optimal Large Language Models," arXiv:2203.15556, 2022; published as "An empirical analysis of compute-optimal large language model training," *NeurIPS*, 2022. arXiv
19. X. Zhai, A. Kolesnikov, N. Houlsby and L. Beyer, "Scaling Vision Transformers," *CVPR*, 2022. arXiv
20. M. Cherti et al., "Reproducible Scaling Laws for Contrastive Language-Image Learning," *CVPR*, 2023. arXiv
21. A. Dosovitskiy et al., "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale," *ICLR*, 2021. arXiv
22. K. He, X. Chen, S. Xie, Y. Li, P. Dollár and R. Girshick, "Masked Autoencoders Are Scalable Vision Learners," *CVPR*, 2022. arXiv
23. H. Bao, L. Dong, S. Piao and F. Wei, "BEiT: BERT Pre-Training of Image Transformers," *ICLR*, 2022. arXiv
24. M. Caron et al., "Emerging Properties in Self-Supervised Vision Transformers," *ICCV*, 2021. arXiv
25. C. Jia et al., "Scaling Up Visual and Vision-Language Representation Learning With Noisy Text Supervision," *ICML*, 2021. arXiv

26. J. Li, D. Li, S. Savarese and S. Hoi, “BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models,” *ICML*, 2023. PMLR
27. J.-B. Alayrac et al., “Flamingo: a Visual Language Model for Few-Shot Learning,” *NeurIPS*, 2022. arXiv
28. H. Liu, C. Li, Y. Li and Y. J. Lee, “Improved Baselines with Visual Instruction Tuning,” *CVPR*, 2024. arXiv
29. N. Houlsby et al., “Parameter-Efficient Transfer Learning for NLP,” *ICML*, 2019. PMLR
30. M. Jia et al., “Visual Prompt Tuning,” *ECCV*, 2022. arXiv
31. T. Dettmers, A. Pagnoni, A. Holtzman and L. Zettlemoyer, “QLoRA: Efficient Finetuning of Quantized LLMs,” *NeurIPS*, 2023. arXiv
32. A. Kumar, A. Raghunathan, R. Jones, T. Ma and P. Liang, “Fine-Tuning can Distort Pretrained Features and Underperform Out-of-Distribution,” *ICLR*, 2022. arXiv
33. M. Wortsman et al., “Robust Fine-Tuning of Zero-Shot Models,” *CVPR*, 2022. arXiv
34. C. Schuhmann et al., “LAION-5B: An Open Large-Scale Dataset for Training Next Generation Image-Text Models,” *NeurIPS Datasets and Benchmarks Track*, 2022. arXiv
35. A. Birhane, V. U. Prabhu and E. Kahembwe, “Multimodal Datasets: Misogyny, Pornography, and Malignant Stereotypes,” arXiv:2110.01963, 2021. arXiv
36. P. Mayilvahanan, T. Wiedemer, E. Rusak, M. Bethge and W. Brendel, “Does CLIP’s Generalization Performance Mainly Stem from High Train-Test Similarity?,” *ICLR*, 2024. arXiv
37. Y. Li et al., “Evaluating Object Hallucination in Large Vision-Language Models,” *EMNLP*, 2023. arXiv
38. X. Yue et al., “MMMU: A Massive Multi-discipline Multimodal Understanding and Reasoning Benchmark for Expert AGI,” *CVPR*, 2024. arXiv
39. M. Awais et al., “Foundational Models Defining a New Era in Vision: A Survey and Outlook,” arXiv:2307.13721, 2023. arXiv
40. S. Yin et al., “A Survey on Multimodal Large Language Models,” *National Science Review*, 2024. arXiv
41. J. Zhang, J. Huang, S. Jin and S. Lu, “Vision-Language Models for Vision Tasks: A Survey,” *IEEE TPAMI*, vol. 46, no. 8, 2024. arXiv
42. L. Xu et al., “Parameter-Efficient Fine-Tuning Methods for Pretrained Language Models: A Critical Review and Assessment,” arXiv:2312.12148, 2023. arXiv
43. J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li and L. Fei-Fei, “ImageNet: A Large-Scale Hierarchical Image Database,” *CVPR*, 2009, pp. 248–255. doi
44. J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” *NAACL-HLT*, 2019, pp. 4171–4186. doi
45. OpenAI, “GPT-4 Technical Report,” arXiv:2303.08774, 2023. arXiv

ML 10 • Evaluation, Efficiency and Deployment

Split data without leakage, cross-validate, attach confidence intervals, check calibration and robustness under shift, measure FLOPs, memory and latency honestly, then export, compress, monitor and reproduce a model.

Online edition: https://blog.shyandram.dev/en/learn/ml/ml-10-evaluation-deployment/

What you'll learn

- How to build a baseline and a data split whose result actually supports the claim you want to make, and how leakage silently breaks that.
- How K-fold, grouped, time-ordered and nested cross-validation work, and when each one is the right tool.
- How to put an uncertainty on a test score: standard errors, the paired bootstrap and McNemar's test.
- How calibration differs from accuracy, how to measure it (Brier score, NLL, ECE) and how temperature scaling fixes part of it.
- What distribution shift is (covariate, label, concept), how robustness benchmarks probe it, and how to monitor it after deployment.
- How to count FLOPs, memory and latency correctly, and what export, quantization, pruning, distillation and inference runtimes such as ONNX Runtime and TensorRT really buy you.
- How to make an experiment reproducible and document it.

The big picture

Training a model is the middle of a project, not the end. Before anyone relies on it, you need to answer four questions. Does it work on the cases it will really see? How sure are we of that number? Does it know when it is likely to be wrong? Does it fit the device, latency and budget it has to run on? After it ships, a fifth question appears: is it still working?

> **Explain like I'm five**
>
> A student who memorized last year's exam gets full marks on last year's exam. That tells you nothing. A fair test uses new questions (a clean split). One test can be lucky, so you want to know how much the score would wobble on another day (a confidence interval). A good student also says "I'm not sure" on the questions they might get wrong (calibration). If the real exam is in a noisy hall, you check that too (robustness). If it is a timed exam, being right slowly is not enough (latency). And once the student starts a job, someone still checks their work now and then (monitoring).

This chapter closes the ML track. It gathers the practical methodology from Goodfellow et al. [1, ch. 11] and the statistical-learning view of James et al. [2], and adds the engineering steps that

turn a trained network into a dependable service. The task-specific metrics of vision (top-1, mAP, PSNR, FID and so on) are covered in DLCV 02 and DLCV 03; here we focus on what all tasks share.

Start with a meaningful baseline

Plain version. Before spending more compute, find out how well a simple method does on exactly the same split. The baseline tells you what the complicated model actually buys.

Good baselines for image tasks include: logistic regression on raw pixels or hand-designed features; a nearest-neighbor or SVM classifier; and a small linear head on frozen features from a pretrained model (see ML 09). Logistic regression learns a linear boundary; a support vector machine (SVM) maximizes a regularized margin; nearest neighbors compares examples directly; tree ensembles partition the feature space. The classical pattern-classification chapter, IP 13, develops these methods.

For reference, the binary linear SVM minimizes

$$\frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\max(0,\, 1 - y_i(w^\top x_i + b)), \qquad y_i \in \{-1, +1\},$$

where w and b are the weights and bias, x_i is the i-th feature vector, and $C > 0$ trades margin width against training errors. The term $\max(0, 1 - y_i f(x_i))$ is the **hinge loss**: it is zero for points on the correct side with margin at least 1 and grows linearly otherwise.

Code: baselines with preprocessing inside the pipeline

This example uses only bundled data. Model choice uses a validation split; the chosen model is evaluated on the test split once. Scaling and PCA are fitted inside each training pipeline, never on the full dataset.

```
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, balanced_accuracy_score

X, y = load_digits(return_X_y=True)
Xa, Xt, ya, yt = train_test_split(X, y, test_size=.2, stratify=y, random_state=0)
Xtr, Xv, ytr, yv = train_test_split(Xa, ya, test_size=.25, stratify=ya, random_state=1)
models = {      # listed from simplest to most complex
    "linear": make_pipeline(StandardScaler(),
                            LogisticRegression(max_iter=2000)),
    "PCA + RBF SVM": make_pipeline(StandardScaler(), PCA(n_components=30),
```

```
                                SVC(C=10, gamma="scale")),
}
scores = {}
for name, model in models.items():
    model.fit(Xtr, ytr)
    scores[name] = accuracy_score(yv, model.predict(Xv))
best = max(scores, key=scores.get)   # tie -> first listed, i.e. the simpler model
pred = models[best].predict(Xt)
print("validation:", scores)
print("selected:", best)
print("test accuracy:", accuracy_score(yt, pred))
print("test balanced accuracy:", balanced_accuracy_score(yt, pred))
```

On this split both candidates score exactly 0.9639 on validation (347 of 360 images). The rule "on a tie, prefer the simpler model" was fixed in advance, so the linear model is selected, and it scores 0.9639 accuracy and 0.9636 balanced accuracy on test. Two lessons hide in this tiny run. First, a tie-break rule is part of the protocol and should be written down before you look. Second, with 360 validation images a single image is worth 0.28 percentage points, so many apparent "wins" in small experiments are noise; the section on confidence intervals makes this precise.

Data splits and leakage

Plain version. The test set must be new in the same way that deployment data will be new. If test images share a patient, a video, a scene or a time period with training images, the test score measures memory, not generalization.

Choose the split unit from the claim

Images from one patient, one video, one production batch or one camera are correlated. Randomly splitting adjacent video frames puts almost the same picture in train and test. Ask what "new" means in deployment and split by that unit: by patient if the model will see new patients, by video if it will see new videos, by site if it will be installed in new hospitals, by time if it must predict the future.

Figure 1 — Choose the split unit according to the generalization claim. Group separation prevents near-duplicate context from crossing the split.

Stratification keeps class proportions similar across splits; it does not replace grouping. Deduplicate before splitting, and keep exact or near duplicates together. When the deployment population drifts over time, a time-based split (train on the past, test on the most recent period) is the honest choice.

The many faces of leakage

Leakage is any path by which information that would not be available at prediction time reaches the model or the model-selection process. Kapoor and Narayanan found leakage errors affecting 329 papers across 17 scientific fields that use ML, and propose a taxonomy of leakage types, among them missing train/test separation, preprocessing on the full dataset, duplicates, temporal leakage and illegitimate features [4]. The scikit-learn documentation lists the most common coding mistakes [3]. A practical checklist:

Leak	Example	Fix
Preprocessing on all data	Fit `StandardScaler`, PCA or a vocabulary on train + test	Put preprocessing inside a `Pipeline` fitted on the training fold only
Correlated units	Frames of one video, slices of one scan, crops of one photo	Split by group
Duplicates	The same image scraped twice	Hash or embed and deduplicate before splitting
Temporal	Using next week's data to predict this week	Time-ordered splits
Target proxy	A feature recorded *after* the outcome (e.g. "treatment given")	Audit each feature's timestamp
Test reuse	Tuning hyperparameters while watching the test score	Validation for decisions, test once

The last row is subtle. Every time you look at the test score and change something, the test set becomes a little more like a validation set. Public benchmarks suffer from this at community scale.

Cross-validation

Plain version. Instead of one train/validation split, rotate: split the data into K parts, train K times, each time holding out a different part, and average. You use every example for evaluation once and you see how much the score varies.

K-fold and its variants

Partition the indices into K disjoint folds $F_1, \dots, F_K$. For each k, train $\hat{f}^{(-k)}$ on all folds except F_k and compute its error on F_k. The cross-validation estimate is

$$\widehat{\mathrm{Err}}_{\mathrm{CV}} = \frac{1}{K} \sum_{k=1}^{K} \frac{1}{|F_k|} \sum_{i \in F_k} \ell(y_i, \hat{f}^{(-k)}(x_i)),$$

where ℓ is the per-example loss (for example 0–1 error) and $|F_k|$ is the fold size. Common choices are $K = 5$ or $K = 10$ [2]. The variants differ only in how folds are formed:

- **Stratified K-fold** keeps class proportions equal across folds; useful when classes are imbalanced.
- **Group K-fold** assigns each group (patient, video, site) to exactly one fold, so no group appears in both training and held-out data.
- **Time-series split** always trains on earlier data and validates on later data.
- **Leave-one-group-out** is group K-fold with one group per fold; useful with few groups.

The fold scores are not independent (training sets overlap), so the standard deviation across folds understates the true uncertainty. Treat it as a rough indication of stability, not as a formal confidence interval.

Code: what random K-fold does to grouped data

We simulate 40 patients with 25 images each. Every patient's images share a "fingerprint" (patient-specific appearance), and the label is fixed per patient. The real class signal is weak.

```
import numpy as np
from sklearn.model_selection import KFold, GroupKFold, cross_val_score
from sklearn.ensemble import RandomForestClassifier

rng = np.random.default_rng(0)
n_groups, per_group = 40, 25                    # e.g. 40 patients, 25 images each
groups = np.repeat(np.arange(n_groups), per_group)
y = np.repeat(rng.integers(0, 2, n_groups), per_group)   # label fixed per patient
```

```
identity = rng.normal(size=(n_groups, 20))[groups]       # patient "fingerprint"
signal = 0.3 * y[:, None] * np.ones((1, 20))             # weak real class signal
X = identity + signal + 0.3 * rng.normal(size=(len(y), 20))

model = RandomForestClassifier(n_estimators=200, random_state=0)
random_cv = cross_val_score(model, X, y, cv=KFold(5, shuffle=True, random_state=0))
group_cv = cross_val_score(model, X, y, cv=GroupKFold(5), groups=groups)
print(f"random K-fold accuracy: {random_cv.mean():.3f} +/- {random_cv.std():.3f}")
print(f"group  K-fold accuracy: {group_cv.mean():.3f} +/- {group_cv.std():.3f}")
```

Output:

```
random K-fold accuracy: 0.999 +/- 0.002
group  K-fold accuracy: 0.657 +/- 0.158
```

The random split reports near-perfect accuracy because the forest recognizes each patient's fingerprint and copies that patient's label. Group *K*-fold, which matches the claim "works on new patients", reports 0.657 with large variation across folds. Neither number is wrong as arithmetic; only one answers the question.

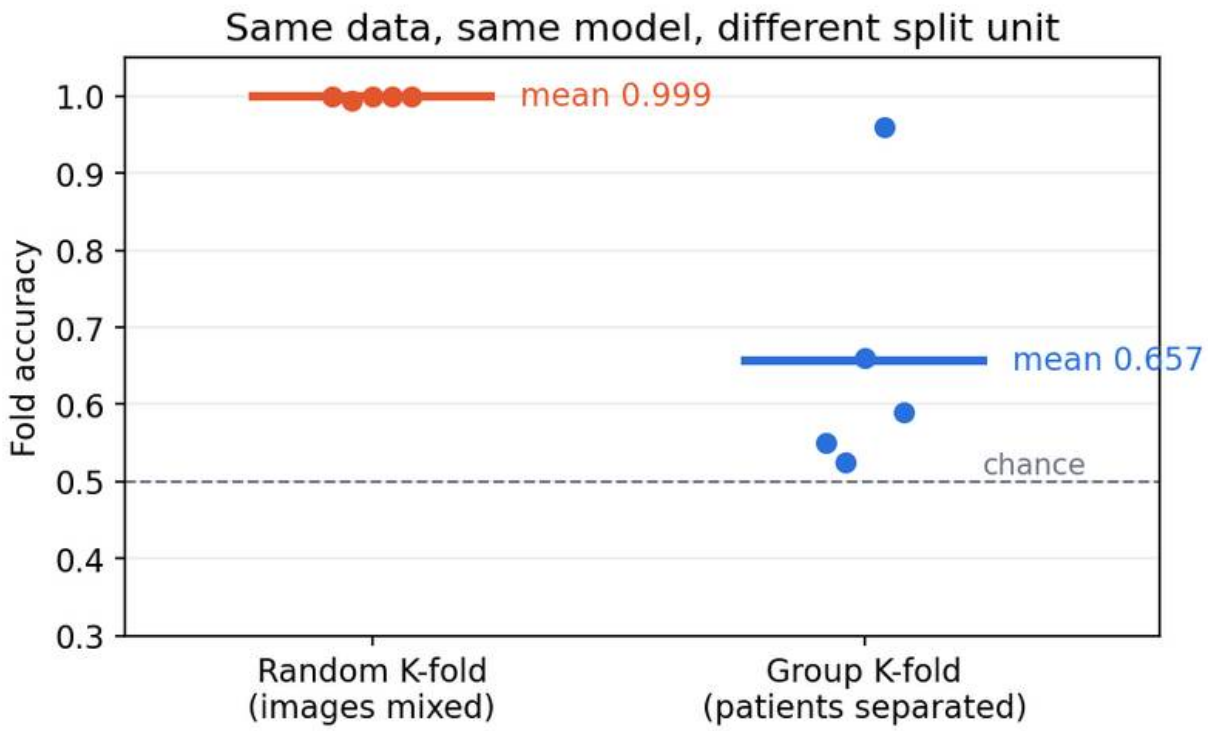


Figure 2 — Output of the code above, one dot per fold. The split unit, not the model, causes the 34-point gap.

Nested cross-validation

If you use cross-validation both to **choose** hyperparameters and to **report** performance, the reported number is optimistically biased: you picked the configuration that happened to look best on those folds. Varma and Simon demonstrated this with "null" data in which the two classes were statistically identical: after tuning an SVM by cross-validation, the CV error estimate fell below 30% on 38% of the simulated datasets, while the tuned classifiers were no better than chance on independent test data [5]; Cawley and Talbot showed that over-fitting the model-selection criterion can be as large as the differences between learning algorithms [6].

Nested CV fixes this with two loops. The outer loop holds out a fold for evaluation. Inside each outer training set, an inner CV selects hyperparameters. The outer fold is touched only to score the configuration the inner loop chose. In scikit-learn this is simply a `GridSearchCV` placed inside `cross_val_score`. The result estimates the performance of the whole *procedure* (including tuning), which is what you will run when you retrain.

For deep networks a full nested CV is often too expensive. The usual compromise is a single fixed train/validation/test split, a stated tuning budget for every method compared, and a test set that is opened once.

Metrics: a recap

Plain version. Decide what an error costs before choosing a number. One scalar rarely tells the whole story.

Task	Useful measurements	Common trap
Classification	Per-class precision/recall, macro-F1, balanced accuracy, AUROC/AUPRC	Accuracy hides rare failures; PR curves depend on prevalence
Detection/segmentation	AP, IoU/Dice, boundary errors	Comparing different IoU thresholds or protocols
Retrieval	Recall@k, mAP under a defined relevance set	Gallery duplicates inflate matches
Restoration	PSNR/SSIM, perceptual metrics, consistency with the observation	Plausible texture may not be true detail
Generation	FID/KID, coverage, conditional alignment, human study	Feature realism does not establish prompt fidelity
Video/geometry	Temporal consistency, task accuracy, geometric error	Good individual frames can hide inconsistent structure

The definitions and pitfalls of classification metrics (confusion matrix, top-k, ROC/PR curves) are worked through in DLCV 02, and those of detection (IoU, AP, COCO mAP) in DLCV 03. For generative models, FID compares Gaussian fits (means and covariances) of feature distributions; KID instead uses a kernel maximum-mean-discrepancy estimate that is unbiased, which makes it better behaved with small sample counts [7]. Neither is a universal measure of human preference. Always state the feature extractor, reference set, sample count, resolution and sampling settings.

Two classical facts are worth remembering. With 1% positives, an always-negative classifier gets 99% accuracy and zero recall, so accuracy alone is meaningless for rare events. And precision depends on prevalence: the same detector has lower precision in a population where the event is rarer, while its recall and false-positive rate do not change.

Statistical significance and confidence intervals

Plain version. A test score is computed from a finite sample, so it would come out a little different on another sample. Report a range, not just a point, and compare two models on the *same* examples.

Standard error of an accuracy

If the n test examples are independent and the model's true accuracy is p, the number of correct predictions is binomial, and the observed accuracy $\hat{p}$ has standard error

$$\mathrm{SE}(\hat{p}) = \sqrt{\frac{\hat{p}(1-\hat{p})}{n}}, \qquad 95\%\ \text{interval} \approx \hat{p} \pm 1.96\,\mathrm{SE}.$$

Here $\hat{p}$ is the fraction correct and 1.96 is the 97.5th percentile of the standard normal distribution. For $\hat{p} = 0.972$ and $n = 540$, $\mathrm{SE} \approx 0.0071$, so the interval is about ± 1.4 percentage points. This normal approximation is poor when $\hat{p}$ is very close to 0 or 1 or n is small; Wilson or exact binomial intervals are better there. If test examples are grouped (several images per patient), the effective sample size is closer to the number of groups, and the interval must be computed by resampling groups.

The bootstrap

The **bootstrap** estimates the sampling distribution of any statistic by resampling the observed data with replacement [9]. Draw B resamples of size n, recompute the statistic on each, and take the 2.5th and 97.5th percentiles as an approximate 95% interval (the *percentile* interval). It works for metrics with no simple formula, such as macro-F1, AUROC or mAP.

To compare two models, use a **paired** bootstrap: resample example indices once, and evaluate both models on the same resample. Because the models usually fail on many of the same hard examples, pairing removes that shared variance and gives a much tighter interval for the difference than two separate intervals would.

McNemar's test

For two classifiers evaluated on the same test set, only the examples where they disagree carry information about which is better. Let n_{01} count examples model A gets right and B gets wrong, and n_{10} the reverse. Under the null hypothesis that both models have the same error rate, each

discordant example is equally likely to fall either way, so $n_{10} \sim \text{Binomial}(n_{01}+n_{10}, \frac{1}{2})$. The exact McNemar test computes a two-sided binomial p-value from that. Dietterich compared five tests for comparing learning algorithms and found that, for algorithms that can be run only once (typical for expensive models), McNemar's test was the only one with acceptable type I error; when ten runs are affordable he recommended the 5×2 cross-validated t test [8].

Code: paired bootstrap and McNemar

```
import numpy as np
from scipy.stats import binomtest
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC

X, y = load_digits(return_X_y=True)
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3, stratify=y, random_state=0)
a = make_pipeline(StandardScaler(), LogisticRegression(max_iter=2000)).fit(Xtr, ytr)
b = make_pipeline(StandardScaler(), SVC(C=10, gamma="scale")).fit(Xtr, ytr)
ca, cb = a.predict(Xte) == yte, b.predict(Xte) == yte    # per-example correctness

# 1) Percentile bootstrap: resample test examples with replacement.
rng = np.random.default_rng(0)
idx = rng.integers(0, len(yte), size=(10_000, len(yte)))
diff = cb[idx].mean(axis=1) - ca[idx].mean(axis=1)       # paired: same resample
lo, hi = np.percentile(diff, [2.5, 97.5])
print(f"accuracy A={ca.mean():.4f}  B={cb.mean():.4f}  n={len(yte)}")
print(f"B - A = {cb.mean() - ca.mean():+.4f}, 95% bootstrap CI [{lo:+.4f}, {hi:+.4f}]")

# 2) Exact McNemar test: only the discordant examples carry information.
n01 = int(np.sum(ca & ~cb))      # A right, B wrong
n10 = int(np.sum(~ca & cb))      # A wrong, B right
p = binomtest(n10, n01 + n10, 0.5).pvalue
print(f"discordant pairs: A-only={n01}, B-only={n10}, exact McNemar p={p:.4f}")
```

Output:

```
accuracy A=0.9722  B=0.9852  n=540
B - A = +0.0130, 95% bootstrap CI [-0.0019, +0.0278]
discordant pairs: A-only=5, B-only=12, exact McNemar p=0.1435
```

The SVM is 1.3 points better, but the interval includes zero and McNemar's p-value is 0.14. Only 17 of 540 images separate the two models, and 12 versus 5 is not unusual under a fair coin. With this test set, the honest conclusion is "B is probably at least as good; the evidence that it is better is weak."

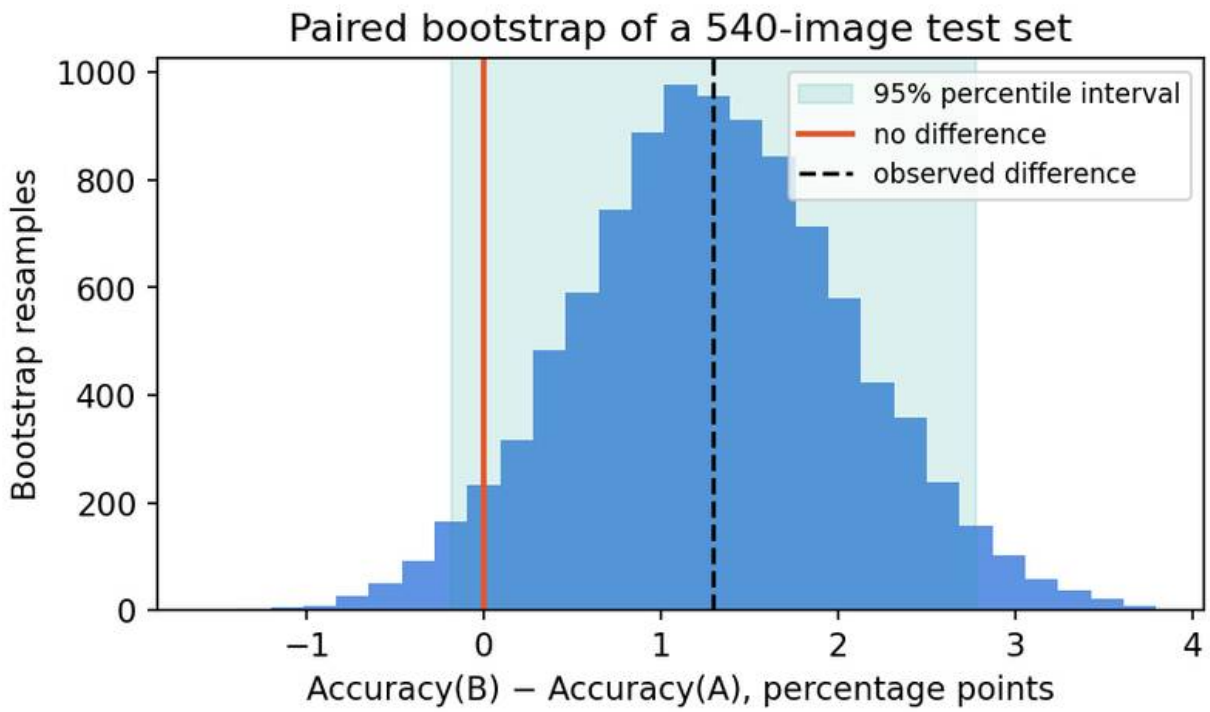


Figure 3 — The paired bootstrap distribution of Accuracy(B) − Accuracy(A) from the code above. The 95% interval crosses zero.

Other sources of variance

The test sample is not the only source of randomness. For a neural network, the random seed changes initialization, data order and augmentation; the data split itself is random; and some GPU kernels are nondeterministic. Bouthillier et al. modeled the whole benchmarking process and found that variance from data sampling, parameter initialization and hyperparameter choice markedly affects conclusions; they recommend randomizing as many of these sources as possible across repeated runs [10]. Practical rules:

- Run each configuration with several seeds (3–5 is common) and report mean and spread.
- Compare methods with paired tests on the same examples and splits.
- If you compare many models or many datasets, some will "win" by chance; adjust for multiple comparisons or confirm on a fresh set.
- Report effect sizes with intervals. A statistically significant difference can be practically irrelevant, and a large practical difference can be statistically unproven on a small set.

Calibration: knowing when you might be wrong

Plain version. A model's confidence should match how often it is right. Among all predictions made with 80% confidence, about 80% should be correct. Accuracy and calibration are separate properties.

Measuring probability quality

Let p_{ik} be the predicted probability of class k for example i, and $y_{ik} \in \{0, 1\}$ the one-hot label. Two **proper scoring rules** (scores minimized in expectation by the true probabilities) are:

$$\mathrm{NLL} = -\frac{1}{n}\sum_{i=1}^{n} \log p_{i,y_i}, \qquad \mathrm{Brier} = \frac{1}{n}\sum_{i=1}^{n}\sum_{k=1}^{K} (p_{ik} - y_{ik})^2,$$

where y_i is the true class of example i and K is the number of classes. Some libraries divide the Brier score by K; say which convention you use.

The **expected calibration error** (ECE) groups predictions into M bins $B_1, \dots, B_M$ by their top-class confidence and compares average confidence with accuracy in each bin:

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{n} \left|\mathrm{acc}(B_m) - \mathrm{conf}(B_m)\right|,$$

where $\mathrm{acc}(B_m)$ is the fraction correct in bin m and $\mathrm{conf}(B_m)$ the mean confidence there. A **reliability diagram** plots acc against conf per bin; a perfectly calibrated model lies on the diagonal. ECE is not a proper scoring rule and depends on the bin count and the bin scheme, so report it alongside NLL or Brier and a reliability diagram.

Temperature scaling

Guo et al. observed that modern deep networks, which are much larger than those of earlier decades, are often overconfident, and that a single scalar fix works surprisingly well [11]. **Temperature scaling** divides the logits ℓ by $T > 0$ before the softmax:

$$\hat{p}_k = \frac{\exp(\ell_k / T)}{\sum_j \exp(\ell_j / T)}, \qquad T^\star = \arg\min_{T>0} \ \mathrm{NLL}_{\mathrm{cal}}(T).$$

$T > 1$ softens overconfident predictions; $T < 1$ sharpens underconfident ones. Because dividing all logits by the same positive number never changes which is largest, the predicted class (and so accuracy) is unchanged. T must be fitted on a held-out calibration set, never on the test set. Later work by Minderer et al. found that the most recent image classifiers, notably non-convolutional ones such as Vision Transformers, are among the best calibrated, and that older trends such as calibration decaying under distribution shift are less pronounced for them [12].

Code: temperature scaling and what shift does to it

```
import numpy as np
from scipy.optimize import minimize_scalar
from scipy.special import log_softmax, softmax
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier

X, y = load_digits(return_X_y=True)
X = X / 16.0
Xa, Xte, ya, yte = train_test_split(X, y, test_size=0.4, stratify=y, random_state=0)
Xtr, Xcal, ytr, ycal = train_test_split(Xa, ya, test_size=0.3, stratify=ya, random_state=0)
net = MLPClassifier(hidden_layer_sizes=(128,), alpha=1e-6, max_iter=300,
                    random_state=0).fit(Xtr, ytr)

def logits(X):                        # log-probabilities = logits up to a constant
    return np.log(np.clip(net.predict_proba(X), 1e-12, 1.0))

def ece(prob, y, bins=10):
    conf, pred = prob.max(1), prob.argmax(1)
    edges = np.linspace(0, 1, bins + 1)
    total = 0.0
    for lo, hi in zip(edges[:-1], edges[1:]):
        m = (conf > lo) & (conf <= hi)
        if m.any():
            total += m.mean() * abs((pred[m] == y[m]).mean() - conf[m].mean())
    return total

def nll(T, z, y):
    return -log_softmax(z / T, axis=1)[np.arange(len(y)), y].mean()

T = minimize_scalar(nll, bounds=(0.05, 20), method="bounded",
                    args=(logits(Xcal), ycal)).x
print(f"fitted temperature T = {T:.2f}")
noise = np.random.default_rng(0).normal(size=Xte.shape)
for sigma in [0.0, 0.3, 0.6]:         # Gaussian pixel noise = a simple covariate shift
    Xs = np.clip(Xte + sigma * noise, 0, 1)
    z = logits(Xs)
    p_raw, p_ts = softmax(z, axis=1), softmax(z / T, axis=1)
    print(f"noise {sigma:.1f}: acc={(p_raw.argmax(1) == yte).mean():.3f}  "
          f"ECE raw={ece(p_raw, yte):.3f}  ECE scaled={ece(p_ts, yte):.3f}  "
          f"NLL raw={nll(1, z, yte):.3f}  NLL scaled={nll(T, z, yte):.3f}")
```

Output (scikit-learn may print a convergence warning, which is harmless here):

```
fitted temperature T = 1.19
noise 0.0: acc=0.972  ECE raw=0.007  ECE scaled=0.015  NLL raw=0.108  NLL scaled=0.110
noise 0.3: acc=0.794  ECE raw=0.088  ECE scaled=0.063  NLL raw=0.693  NLL scaled=0.639
noise 0.6: acc=0.435  ECE raw=0.380  ECE scaled=0.347  NLL raw=2.998  NLL scaled=2.598
```

Three observations. On clean data this small network is already well calibrated, and fitting T on 324 calibration images slightly *worsens* ECE: a fix fitted on a small set can add noise. Under pixel noise the model becomes badly overconfident: at $\sigma = 0.6$ it is right 43.5% of the time while its confidence suggests far more. Temperature scaling fitted on clean data helps only a little under shift. This matches the large study by Ovadia et al., who found that the calibration of many uncertainty methods deteriorates as shift increases [16].

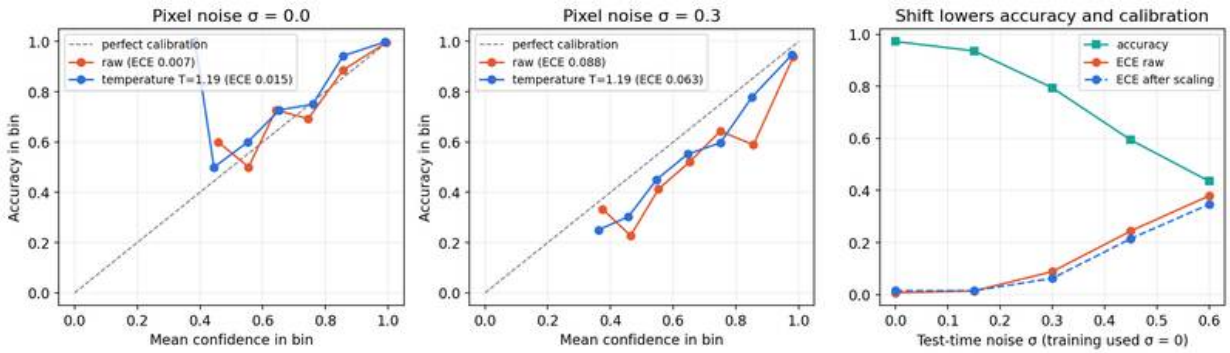


Figure 4 — Reliability diagrams and a shift sweep for the network above (same noise pattern, scaled by σ). Points below the diagonal mean overconfidence.

Robustness and distribution shift

Plain version. The world after deployment is rarely identical to the training set. Cameras change, lighting changes, populations change. A robust model degrades gracefully; a brittle one fails silently.

Three kinds of shift

Write the joint distribution as $p(x, y) = p(y \mid x)\,p(x) = p(x \mid y)\,p(y)$, with x the input and y the label. Shifts are named by what changes between training (p_{tr}) and deployment (p_{te}):

- **Covariate shift:** $p_{\text{te}}(x) \neq p_{\text{tr}}(x)$ while $p(y \mid x)$ is unchanged. New camera, new lighting, new scanner.
- **Label (prior) shift:** $p_{\text{te}}(y) \neq p_{\text{tr}}(y)$ while $p(x \mid y)$ is unchanged. A disease becomes more common during an outbreak.
- **Concept shift:** $p(y \mid x)$ itself changes. The definition of "spam" evolves.

Under pure covariate shift, the expected test loss can be rewritten as a weighted training loss, $\mathbb{E}_{\text{te}}[\ell] = \mathbb{E}_{\text{tr}}[w(x)\,\ell]$ with importance weight $w(x) = p_{\text{te}}(x)/p_{\text{tr}}(x)$. These weights are hard to estimate for images, but the identity explains why shift matters most where test inputs fall in regions with little training data. Under label shift, Lipton et al. showed that a fixed black-box classifier's confusion matrix on held-out training data, combined with its prediction frequencies on unlabeled test data, is enough to estimate the new class priors and re-weight the model [17].

Benchmarks that probe shift

- **ImageNet-C** applies 15 corruption types (noise, blur, weather, digital) at several severities to ImageNet validation images, and **ImageNet-P** adds perturbation sequences; together they measure robustness to common corruptions rather than adversarial attacks [13]. Our pixel-noise sweep is a miniature version.
- **ImageNetV2** re-created the ImageNet test set by following the original collection process as closely as possible. Accuracy dropped by 11–14% across many models, yet gains on the original test set translated into even larger gains on the new one, so the drop reflects slightly harder images rather than over-fitting to the reused test set [14]. Even a careful re-collection is a shift.
- **WILDS** gathers ten datasets with naturally occurring shifts, including tumor tissue from different hospitals, camera traps at new locations and satellite images from new countries, and reports a large gap between in-distribution and out-of-distribution performance that existing mitigation methods did not close [15].

The practical lesson: always add at least one *shifted* evaluation set built from the change you expect in deployment (new site, new device, later time), and report in-distribution and shifted results side by side.

Efficiency: FLOPs, memory and latency

Plain version. "Fast" and "small" each have several meanings. A model with fewer operations is not automatically quicker, and a model with fewer parameters does not automatically use less memory. Measure on the target device.

Counting operations

One **multiply–accumulate** (MAC) is a multiplication plus an addition, so 1 MAC $\approx$ 2 floating-point operations (FLOPs). For a fully connected layer with d_{in} inputs and d_{out} outputs, and a convolution with C_{in} input channels, C_{out} output channels, a $k \times k$ kernel and an $H' \times W'$ output:

$$\text{FLOPs}_{\text{linear}} \approx 2\, d_{\text{in}} d_{\text{out}}, \qquad \text{FLOPs}_{\text{conv}} \approx 2\, C_{\text{in}} k^2 C_{\text{out}} H' W'$$

per example (bias and activation costs are small). Libraries are inconsistent about whether they report MACs or FLOPs, so state which.

Why FLOPs do not equal speed

Dehghani et al. showed that common cost indicators (parameter count, FLOPs, throughput, latency) can disagree with each other and argued that reporting only one of them gives a misleading picture of efficiency [18]. The main reasons:

- **Memory traffic.** A layer whose weights must be read from memory for very little arithmetic is limited by memory bandwidth, not compute. A linear layer applied to a single example reads

$4d_{\text{in}}d_{\text{out}}$ bytes of fp32 weights to perform $2d_{\text{in}}d_{\text{out}}$ FLOPs: 0.5 FLOPs per byte, far too little to keep a modern accelerator busy. Batching raises this ratio (called *arithmetic intensity*).

- **Parallelism.** Depthwise convolutions and many small layers have few FLOPs but use the hardware poorly.
- **Kernel support.** An operation without an optimized kernel on the target runtime may be slow regardless of its FLOP count.

Memory accounting

At inference, memory holds weights (parameters × bytes per value), activations of the layers currently alive, workspace for kernels, and for autoregressive Transformers a **KV cache** that grows with context length and batch size. During training add gradients and optimizer states: Adam keeps two extra values per parameter, so an fp32 model with Adam needs roughly 4× its weight memory before counting activations.

A latency protocol

Wall-clock latency depends on hardware, runtime, batch size, input resolution, numeric precision and thread settings, and the first calls are slower because of one-off initialization. A sound protocol:

1. Fix and report device, runtime version, precision, batch size, input size and thread count.
2. Run warm-up iterations and discard them.
3. Time many runs and report percentiles (p50 and p90 or p99), not just the mean: users experience the tail.
4. On GPUs, synchronize the device before stopping the timer, because kernels are launched asynchronously.
5. Measure end to end (pre-processing, transfers, decoding) as well as the network alone.

A **Pareto frontier** contains the candidates for which no other candidate is at least as good on every axis (accuracy, latency, memory) and strictly better on one. Presenting the frontier makes trade-offs visible without inventing a single "best model".

Deployment: export, compression and runtimes

Plain version. To ship a model, you freeze it into a portable file, often make it smaller or cheaper, and run it with an inference engine designed for the target hardware. Each step can change the outputs, so each step needs re-evaluation.

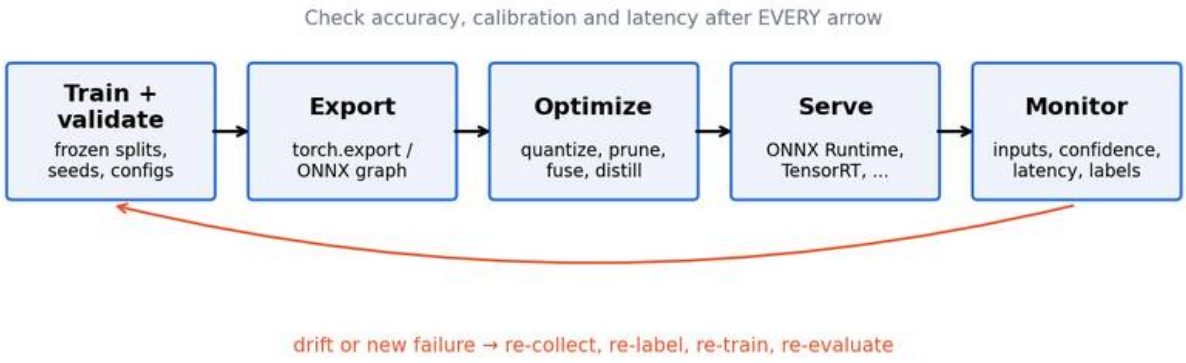


Figure 5 — A deployment loop. Accuracy, calibration and latency should be re-checked after every step, because export, compression and runtimes all change numerics.

Export and inference runtimes

Training code is written in a flexible, Python-driven framework. Deployment prefers a static description of the computation:

- `torch.export` traces a PyTorch module ahead of time into an `ExportedProgram`, a graph of tensor operations that can be serialized and executed with different inputs without the original Python class [19].
- **ONNX** (Open Neural Network Exchange) is an open standard file format for ML models, so a model trained in one framework can be run by tools from others [20].
- **ONNX Runtime** is a cross-platform inference engine for ONNX models. It partitions the graph into subgraphs and hands each to a hardware-specific *execution provider* (for example CUDA, TensorRT, OpenVINO or CoreML) [21].
- **NVIDIA TensorRT** is an SDK that compiles models (from frameworks or ONNX) into optimized engines for NVIDIA GPUs, using layer fusion and reduced precisions such as FP16, BF16, FP8 and INT8 [22].

These runtimes gain speed by fusing operations, choosing fast kernels for the exact shapes and lowering precision. The same model can therefore give slightly different outputs on different runtimes. Compare exported and original outputs on a validation batch and re-measure accuracy after conversion.

Quantization

Quantization stores and computes with fewer bits. In the common **affine (asymmetric) uniform** scheme, a real value x is mapped to an integer q and back:

$$q = \mathrm{clip}\Big(\mathrm{round}\big(\tfrac{x}{s}\big) + z,\ 0,\ 2^b - 1\Big), \qquad \hat{x} = s\,(q - z),$$

where b is the bit width, $s > 0$ the **scale** (the step between representable values), z the integer **zero point** chosen so that the real value 0 is represented exactly, and $\hat{x}$ the dequantized approximation. For a range $[x_{\min}, x_{\max}]$ that includes 0, $s = (x_{\max} - x_{\min})/(2^b - 1)$ and $z = \text{round}(-x_{\min}/s)$. The *symmetric* variant fixes $z = 0$ and uses a signed range. The integer-arithmetic formulation of Jacob et al. showed how to run whole networks with 8-bit integer matrix multiplication and how to train for it [23].

Error comes from two sources: **rounding** inside the range (at most $s/2$ per value) and **clipping** outside it. Both depend on the **granularity** of the scale: one scale per tensor, per output channel, or per small group of weights. Nagel et al. give a practical guide covering these choices, *post-training quantization* (PTQ: calibrate ranges on a small representative dataset after training) and *quantization-aware training* (QAT: simulate rounding during training so the weights adapt) [24]. Gholami et al. survey the wider design space [25].

```
import numpy as np

def quantize(x, bits=8, axis=None):
    """Asymmetric (affine) uniform quantization, per tensor or per channel."""
    qmin, qmax = 0, 2**bits - 1
    lo = x.min(axis=axis, keepdims=True) if axis is not None else x.min()
    hi = x.max(axis=axis, keepdims=True) if axis is not None else x.max()
    lo, hi = np.minimum(lo, 0), np.maximum(hi, 0)      # keep 0 exactly representable
    s = (hi - lo) / (qmax - qmin)                       # scale
    z = np.round(qmin - lo / s)                         # zero point (integer)
    q = np.clip(np.round(x / s) + z, qmin, qmax)        # stored integers
    return s * (q - z)                                  # dequantized values

rng = np.random.default_rng(0)
# 64 output channels whose weight magnitudes differ by up to 100x
W = rng.normal(size=(64, 256)) * np.logspace(-2, 0, 64)[:, None]
for bits in [8, 4]:
    e_t = np.abs(quantize(W, bits) - W).mean()
    e_c = np.abs(quantize(W, bits, axis=1) - W).mean()
    print(f"{bits}-bit mean |error|: per-tensor {e_t:.5f}  per-channel {e_c:.5f}")
print("zero maps back to:", quantize(np.array([-1.0, 0.0, 3.0]))[1])
```

Output:

```
8-bit mean |error|: per-tensor 0.00632  per-channel 0.00124
4-bit mean |error|: per-tensor 0.06718  per-channel 0.02112
zero maps back to: 0.0
```

When channels have very different ranges, one shared scale wastes most of the grid on the largest channel and rounds small channels coarsely. Per-channel scales cut the error about fivefold at 8 bits here. In large language models, a few outlier activation channels cause the same problem at a larger scale, which is what methods such as GPTQ (layer-wise error-compensating weight

rounding) and AWQ (protecting salient weight channels identified from activations) address for 3–4-bit weight-only quantization [42], [43].

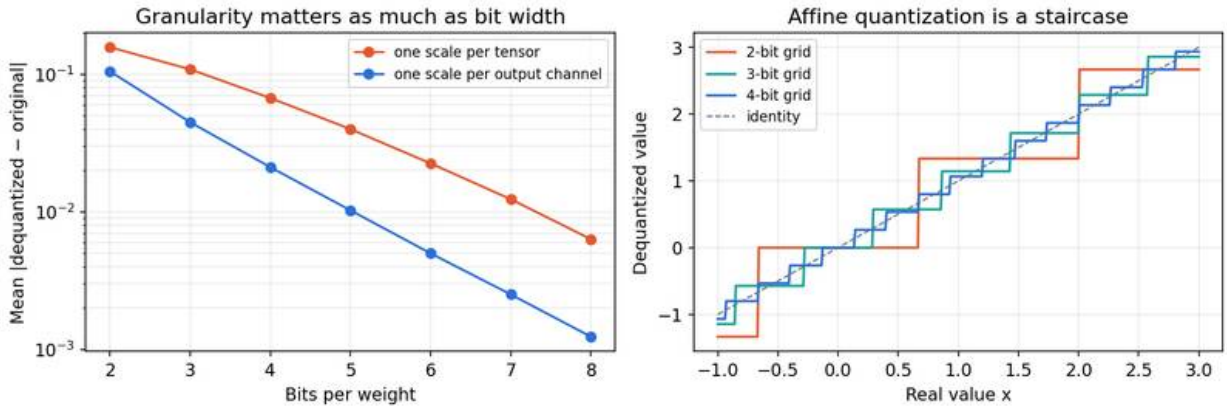


Figure 6 — Quantization error falls roughly by half for each extra bit, and the choice of scale granularity shifts the whole curve.

Pruning

Pruning removes weights, channels, attention heads or blocks that contribute little. *Unstructured* pruning zeros individual weights; it can remove a large fraction of parameters, as in Deep Compression, which combined pruning, weight sharing via quantization, and Huffman coding to shrink networks substantially [26]. But scattered zeros save time only if the runtime has sparse kernels for that pattern. *Structured* pruning removes whole channels or heads, producing smaller dense tensors that any runtime can exploit. Blalock et al. reviewed 81 pruning papers and found that inconsistent baselines and metrics made it hard to tell which methods actually work, and proposed a standardized benchmark [27]. Hoefler et al. survey sparsity in training and inference in depth [28].

Knowledge distillation

Distillation trains a small *student* to imitate a large *teacher*. Hinton et al. proposed matching the teacher's softened output distribution [29]. With temperature T, logits ℓ^t (teacher) and ℓ^s (student), softened probabilities $p^{(T)} = \mathrm{softmax}(\ell/T)$, and weight $\lambda \in [0,1]$:

$$\mathcal{L} = (1-\lambda)\,\mathcal{L}_{\mathrm{CE}}(y,\, p^{s,(1)}) + \lambda\, T^2\, D_{\mathrm{KL}}(p^{t,(T)} \,\|\, p^{s,(T)}).$$

The softened teacher distribution carries "dark knowledge" about which wrong classes are similar. The gradients of the soft term scale as $1/T^2$, so multiplying by T^2 keeps the two terms balanced when T changes [29]. Gou et al. organize later work by *what* is transferred (responses, intermediate features, relations between examples) and *how* (offline, online, self-distillation) [30]. A student inherits the teacher's mistakes and biases as well as its skill.

Code: weight-only int8, latency and export

This sketch stores the weights of a small MLP as int8 with one scale per output channel and measures size and latency on the CPU. It uses only core PyTorch.

```
import io, time
import numpy as np
import torch
from torch import nn

torch.manual_seed(0)
model = nn.Sequential(nn.Linear(512, 2048), nn.ReLU(),
                      nn.Linear(2048, 2048), nn.ReLU(),
                      nn.Linear(2048, 10)).eval()
x = torch.randn(32, 512)

class Int8WeightLinear(nn.Module):
    """Weight-only int8 storage, symmetric per output channel."""
    def __init__(self, lin):
        super().__init__()
        w = lin.weight.detach()
        scale = w.abs().amax(dim=1, keepdim=True) / 127
        self.register_buffer("q", torch.round(w / scale).to(torch.int8))
        self.register_buffer("scale", scale)
        self.register_buffer("bias", lin.bias.detach().clone())
    def forward(self, x):                       # dequantize, then fp32 matmul
        return nn.functional.linear(x, self.q.float() * self.scale, self.bias)

int8 = nn.Sequential(*[Int8WeightLinear(m) if isinstance(m, nn.Linear) else m
                       for m in model]).eval()

def size_mb(m):
    buf = io.BytesIO()
    torch.save(m.state_dict(), buf)
    return buf.getbuffer().nbytes / 1e6

def latency_ms(m, x, warmup=10, runs=100):
    with torch.inference_mode():
        for _ in range(warmup):                 # first calls pay one-off costs
            m(x)
        times = []
        for _ in range(runs):
            t0 = time.perf_counter()
            m(x)
            times.append((time.perf_counter() - t0) * 1e3)
    return np.percentile(times, [50, 90]).round(2)

with torch.inference_mode():
    gap = (model(x) - int8(x)).abs().max().item()
    agree = (model(x).argmax(1) == int8(x).argmax(1)).float().mean().item()
print(f"fp32 file: {size_mb(model):.1f} MB  p50/p90 latency {latency_ms(model, x)} ms")
print(f"int8 file: {size_mb(int8):.1f} MB  p50/p90 latency {latency_ms(int8, x)} ms")
print(f"max |logit difference| = {gap:.4f}, top-1 agreement = {agree:.3f}")
```

```
# Ahead-of-time graph capture: a portable program, independent of this Python class.
exported = torch.export.export(model, (x,))
ops = [n.target for n in exported.graph.nodes if n.op == "call_function"]
print(type(exported).__name__, "->", [str(o).split(".")[1] for o in ops])
```

One CPU run (four threads) printed:

```
fp32 file: 21.1 MB  p50/p90 latency [2.74 4.49] ms
int8 file: 5.3 MB  p50/p90 latency [ 9.23 11.06] ms
max |logit difference| = 0.0018, top-1 agreement = 1.000
ExportedProgram -> ['linear', 'relu', 'linear', 'relu', 'linear']
```

Your latencies will differ; the pattern is the point. The file is four times smaller and the outputs barely change. Yet the int8 version is *slower*, because it converts the weights back to fp32 on every call and then runs the same fp32 matrix multiplication. Lower precision becomes faster only when the runtime has integer (or low-precision) kernels for that operation, or when the workload is limited by memory bandwidth so that reading fewer bytes dominates. This is exactly the gap between "compressed" and "accelerated" that runtimes such as TensorRT, ONNX Runtime and PyTorch's own `torchao` library [46] are built to close. The FLOP count of this MLP is $2 \times (512 \cdot 2048 + 2048 \cdot 2048 + 2048 \cdot 10) \approx 10.5$ MFLOPs per example, identical for both versions.

Monitoring after deployment

Plain version. A model can get worse without any code changing, because the data changes. Watch the inputs and outputs, collect fresh labels when possible, and decide in advance what triggers an investigation.

Sculley et al. describe how ML systems accumulate hidden "technical debt": entangled features, undeclared consumers of predictions, feedback loops where the model's outputs influence its future training data, and dependence on external data that changes without notice [33]. Paleyes et al. collected published deployment case studies and found problems at every stage of the workflow [39]. Useful signals, from cheapest to most informative:

- **System health:** latency percentiles, error rates, memory, throughput.
- **Input statistics:** image brightness, resolution, class of device, missing fields; compared with a reference window from training time.
- **Output statistics:** distribution of predicted classes and of confidence. These need no labels.
- **Delayed labels:** a small, regularly labelled sample of live traffic gives the only direct estimate of accuracy.

To compare a live window with the reference, a two-sample test is a simple starting point. The **Kolmogorov–Smirnov statistic** for one-dimensional samples with empirical cumulative distribution functions F_{ref} and F_{live} is

$$D = \sup_t |F_{\text{ref}}(t) - F_{\text{live}}(t)|,$$

the largest vertical gap between the two curves. Rabanser et al. compared shift detectors and found that a two-sample test applied after reducing the inputs with a pre-trained classifier (for example to its softmax outputs) performed best across the shifts they studied [32].

```
import numpy as np
from scipy.stats import ks_2samp
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

X, y = load_digits(return_X_y=True)
X = X / 16.0
Xtr, Xlive, ytr, ylive = train_test_split(X, y, test_size=0.5, random_state=0)
Xtr, Xref, ytr = Xtr[:600], Xtr[600:], ytr[:600]   # reference window from training time
clf = LogisticRegression(max_iter=2000).fit(Xtr, ytr)
ref_conf = clf.predict_proba(Xref).max(1)     # label-free signal: top-class confidence

rng = np.random.default_rng(0)
for name, noise in [("week 1 (clean)", 0.0), ("week 2 (new camera)", 0.25)]:
    batch = rng.choice(len(Xlive), 200, replace=False)
    Xb = np.clip(Xlive[batch] + noise * rng.normal(size=Xlive[batch].shape), 0, 1)
    conf = clf.predict_proba(Xb).max(1)
    stat, p = ks_2samp(ref_conf, conf)
    acc = (clf.predict(Xb) == ylive[batch]).mean()   # usually NOT available live
    print(f"{name}: mean confidence {conf.mean():.3f}, KS={stat:.3f}, p={p:.2g}, "
          f"(hidden) accuracy {acc:.3f}")
```

Output:

```
week 1 (clean): mean confidence 0.846, KS=0.089, p=0.28, (hidden) accuracy 0.950
week 2 (new camera): mean confidence 0.757, KS=0.277, p=1.3e-08, (hidden) accuracy 0.885
```

The confidence distribution flags week 2 without a single label, and the hidden accuracy confirms that something real happened. A detector says *that* the data changed, not *whether* accuracy fell: harmless shifts also trigger it, and with large windows even tiny changes become "significant". Use it to trigger investigation and labelling, not automatic retraining.

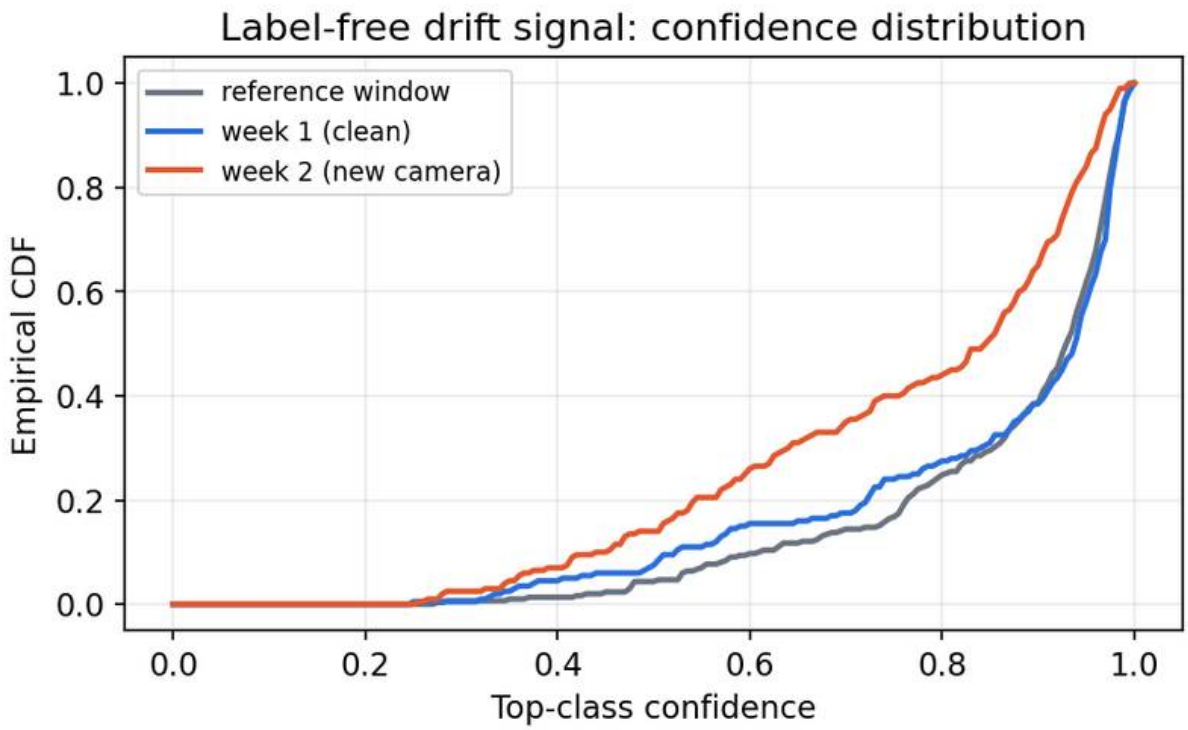


Figure 7 — The Kolmogorov–Smirnov statistic is the largest vertical gap between the reference curve and a live window's curve.

Reproducibility and documentation

Plain version. Someone else (including you in six months) should be able to rerun your experiment and get the same numbers, and a reader should be able to tell what the model is for and where it should not be used.

Making runs repeatable

Fix and record random seeds for every library, but know their limits: the PyTorch documentation states that completely reproducible results are not guaranteed across releases, commits or platforms, and recommends seeding, `torch.use_deterministic_algorithms(True)` and disabling cuDNN benchmarking when bit-exact repetition matters [34]. Beyond seeds, record:

- the dataset version, split IDs (the actual lists of example IDs) and deduplication rule;
- preprocessing and augmentation code, pretrained checkpoint and its source;
- the full configuration (optimizer, schedule, stopping rule), software environment and hardware;
- every validation decision made, and the single final test evaluation.

The NeurIPS 2019 reproducibility program, reported by Pineau et al., introduced a code-submission policy, a community reproducibility challenge and a checklist covering exactly these items [37].

Documenting models and data

Model cards are short documents that accompany a trained model and report its intended use, the evaluation data, performance broken down by relevant groups and conditions, and known limitations [35]. **Datasheets for datasets** ask dataset creators to document motivation, composition, collection process, preprocessing, recommended uses and maintenance [36]. Both make the evaluation decisions in this chapter visible to the people who will rely on them.

A capstone experiment

Choose one task with a modest, legally usable dataset. Define the deployment population and the split unit. Compare three candidates: a classical baseline, frozen pretrained features with a small head, and one fine-tuned network. For each, report the test metric with a bootstrap interval, a paired comparison against the best baseline, a reliability diagram, results on one shifted set, and latency percentiles on the target device. Write a one-page model card. Open the final test set once; if it reveals a failure and you revise the model, treat the old result as development evidence and obtain a fresh evaluation for the revised claim.

Modern view

The engineering of evaluation and deployment is now a research field in its own right. Several surveys and reviews map it.

Leakage and reproducibility. Kapoor and Narayanan's review of ML-based science documents leakage in 329 papers across 17 fields and proposes "model info sheets" that make authors argue why their train/test separation is valid [4]. Combined with the variance study of Bouthillier et al. [10] and the NeurIPS reproducibility program [37], the consensus is that most over-optimistic results come from protocol errors, not from exotic statistics.

Uncertainty and calibration. Gawlikowski et al. survey uncertainty estimation in deep networks: they separate reducible model (epistemic) uncertainty from irreducible data (aleatoric) uncertainty, organize methods into deterministic networks, Bayesian networks, ensembles and test-time augmentation, and review calibration methods and their practical limits [38]. Ovadia et al. add the empirical warning that post-hoc calibration falls short under shift while methods that marginalize over models, such as ensembles, hold up better [16].

Deployment in practice. Paleyes et al. survey published case studies, map the challenges onto the stages of the deployment workflow and show that practitioners hit problems at every stage [39], extending the technical-debt view of Sculley et al. [33].

Compression. Cheng et al. give the classic overview of pruning, low-rank factorization, compact convolutions and distillation for CNNs [31]; Gholami et al. [25] and Nagel et al. [24] focus on quantization; Hoefler et al. on sparsity [28]; Gou et al. on distillation [30]. For large language models, Zhu et al. survey quantization, pruning, distillation and low-rank methods specific to models with billions of parameters [41]. A visible 2023–2026 trend is that weight-only quantization to 4 bits (GPTQ [42], AWQ [43]) and low-precision formats such as FP8 have moved from papers into production toolkits: TensorRT lists FP8, INT8, FP4 and INT4 among its supported precisions [22], and PyTorch's quantization tooling is consolidating in the `torchao` library [46].

Holistic evaluation. For foundation models, HELM evaluates many models on many scenarios and on several metrics at once (accuracy, calibration, robustness, fairness, bias, toxicity and efficiency) and argues that a single leaderboard number hides these trade-offs [40]. The same multi-metric habit applies to vision models.

Scaling. Scaling laws fit empirical relationships among model size, data, compute and loss. Hoffmann et al. found that, for a fixed compute budget, language models should scale parameters and training tokens roughly equally [44]; Zhai et al. studied how ViT accuracy scales with model size, data and compute [45]. Fitted constants from one domain should not be copied into another; the transferable lesson is to report the compute and data behind every result.

Open problems. Evaluation sets leaking into web-scale pretraining data (contamination) is hard to rule out; calibration under unknown shift remains unsolved; latency claims rarely transfer between hardware generations; and evaluation of generative and interactive systems (see DLCV 07) still lacks agreed protocols.

Key takeaways

- A baseline and a clean split decide whether a result means anything. Split by the unit that will be new in deployment, and fit all preprocessing inside the training fold.
- Cross-validation gives a more stable estimate on small data; use group or time-ordered folds when examples are correlated, and nested CV when tuning and reporting from the same data.
- Every test score needs an uncertainty. Compare models on the same examples with a paired bootstrap or McNemar's test.
- Accuracy, calibration and robustness are different properties. Temperature scaling fixes in-distribution overconfidence but not shift-induced overconfidence.
- FLOPs, parameters, memory and latency can disagree. Measure latency on the target hardware with warm-up and percentiles.
- Quantization, pruning and distillation save resources only when the runtime exploits them; re-evaluate accuracy after every export or compression step.
- Monitor inputs, outputs and a trickle of fresh labels after deployment, and document models and data so others can reproduce and judge them.

Exercises

1. **Leakage audit.** A colleague fits PCA on all 1,797 digits, then splits into train and test and reports test accuracy. Explain the leak, and why it exists even though PCA never sees labels. How would you fix the code?

   Hint

   The principal directions are computed using the test images, so the representation is tuned to the test distribution. Put `PCA` inside a `Pipeline` so it is fitted only on each training fold. The bias is often small for PCA, but for supervised preprocessing (feature selection using labels) it can be large.

2. **Rare positives.** A dataset has 1% positive images. What accuracy, positive recall and balanced accuracy does an always-negative classifier achieve? Which metric would you report to a doctor, and why?

   Hint

   Accuracy 99%, recall 0, balanced accuracy 0.5 (mean of recall 0 on positives and 1 on negatives). Report recall (sensitivity) and specificity or precision at a clinically chosen operating point, plus their intervals, because the cost of a missed positive is what matters.

3. **How big must the test set be?** You expect accuracy around 90% and want a 95% confidence interval of about ± 1 percentage point. Using the normal approximation, roughly how many independent test examples do you need? What changes if the images come from 50 patients?

   Hint

   Solve $1.96\sqrt{0.9 \cdot 0.1/n} = 0.01$, giving $n \approx 3{,}460$. With 50 patients the examples are not independent; the effective sample size is closer to 50 than to the number of images, so the interval will be much wider. Use a bootstrap that resamples patients.

4. **Temperature by hand.** A three-class model outputs logits $(4, 1, 0)$. Compute the top-class probability with $T = 1$ and $T = 2$. Does the predicted class change? When would a fitted T be less than 1?

   Hint

   $T = 1$: $e^4/(e^4 + e^1 + e^0) \approx 0.936$. $T = 2$: $e^2/(e^2 + e^{0.5} + 1) \approx 0.736$. The argmax is unchanged. $T < 1$ is fitted when the model is underconfident, for example after heavy regularization or label smoothing.

5. **Compression claim.** A pruned model has half the parameters of the original but identical latency on your GPU. Give three possible explanations and one experiment to tell them apart.

Hint

Unstructured zeros are still multiplied by dense kernels; the remaining cost is dominated by activations, memory traffic or unchanged layers; or the batch is so small that kernel-launch overhead dominates. Profile per-layer time, and compare with a structured-pruned model whose tensors are actually smaller.

6. **Design a monitor.** Your defect detector runs on a factory line; labels arrive two weeks late from manual inspection. Design a monitoring plan: what you log, which statistics you compare with which reference, what threshold triggers an alert, and what happens after an alert.

Hint

Log latency, image statistics (brightness, blur), predicted-defect rate and confidence histograms daily; compare with a reference window from validation using KS or a population-stability measure; alert on sustained change rather than single days. After an alert, label a sample immediately, compute accuracy with an interval, and only then decide whether to recalibrate, fine-tune or roll back.

References

1. I. Goodfellow, Y. Bengio and A. Courville, *Deep Learning*, MIT Press, 2016, ch. 11 "Practical Methodology". book
2. G. James, D. Witten, T. Hastie, R. Tibshirani and J. Taylor, *An Introduction to Statistical Learning: with Applications in Python*, Springer, 2023. book
3. scikit-learn developers, "Common pitfalls and recommended practices," scikit-learn documentation. docs
4. S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, 2023. doi
5. S. Varma and R. Simon, "Bias in error estimation when using cross-validation for model selection," *BMC Bioinformatics*, vol. 7, art. 91, 2006. doi
6. G. C. Cawley and N. L. C. Talbot, "On over-fitting in model selection and subsequent selection bias in performance evaluation," *Journal of Machine Learning Research*, vol. 11, pp. 2079–2107, 2010. paper
7. M. Bińkowski, D. J. Sutherland, M. Arbel and A. Gretton, "Demystifying MMD GANs," ICLR, 2018. arXiv
8. T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," *Neural Computation*, vol. 10, no. 7, 1998. doi
9. B. Efron, "Bootstrap methods: another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, 1979. doi
10. X. Bouthillier et al., "Accounting for variance in machine learning benchmarks," MLSys, 2021. arXiv
11. C. Guo, G. Pleiss, Y. Sun and K. Q. Weinberger, "On calibration of modern neural networks," ICML, 2017. arXiv
12. M. Minderer et al., "Revisiting the calibration of modern neural networks," NeurIPS, 2021. arXiv

13. D. Hendrycks and T. Dietterich, “Benchmarking neural network robustness to common corruptions and perturbations,” ICLR, 2019. arXiv
14. B. Recht, R. Roelofs, L. Schmidt and V. Shankar, “Do ImageNet classifiers generalize to ImageNet?,” ICML, 2019. paper
15. P. W. Koh et al., “WILDS: A benchmark of in-the-wild distribution shifts,” ICML, 2021. paper
16. Y. Ovadia et al., “Can you trust your model’s uncertainty? Evaluating predictive uncertainty under dataset shift,” NeurIPS, 2019. arXiv
17. Z. C. Lipton, Y.-X. Wang and A. Smola, “Detecting and correcting for label shift with black box predictors,” ICML, 2018. arXiv
18. M. Dehghani, A. Arnab, L. Beyer, A. Vaswani and Y. Tay, “The efficiency misnomer,” ICLR, 2022. arXiv
19. PyTorch, “torch.export,” PyTorch documentation. docs
20. ONNX, “Open Neural Network Exchange: the open standard for machine learning interoperability.” site
21. ONNX Runtime, “Welcome to ONNX Runtime,” documentation. docs
22. NVIDIA, “NVIDIA TensorRT documentation.” docs
23. B. Jacob et al., “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” CVPR, 2018. doi
24. M. Nagel et al., “A white paper on neural network quantization,” arXiv:2106.08295, 2021. arXiv
25. A. Gholami, S. Kim, Z. Dong, Z. Yao, M. W. Mahoney and K. Keutzer, “A survey of quantization methods for efficient neural network inference,” book chapter in *Low-Power Computer Vision*; arXiv:2103.13630, 2021. arXiv
26. S. Han, H. Mao and W. J. Dally, “Deep compression: compressing deep neural networks with pruning, trained quantization and Huffman coding,” ICLR, 2016. arXiv
27. D. Blalock, J. J. Gonzalez Ortiz, J. Frankle and J. Guttag, “What is the state of neural network pruning?,” MLSys, 2020. arXiv
28. T. Hoefler, D. Alistarh, T. Ben-Nun, N. Dryden and A. Peste, “Sparsity in deep learning: pruning and growth for efficient inference and training in neural networks,” *Journal of Machine Learning Research*, vol. 22, 2021. arXiv
29. G. Hinton, O. Vinyals and J. Dean, “Distilling the knowledge in a neural network,” NIPS Deep Learning Workshop, 2014; arXiv:1503.02531. arXiv
30. J. Gou, B. Yu, S. J. Maybank and D. Tao, “Knowledge distillation: a survey,” *International Journal of Computer Vision*, vol. 129, 2021. doi
31. Y. Cheng, D. Wang, P. Zhou and T. Zhang, “Model compression and acceleration for deep neural networks: the principles, progress, and challenges,” *IEEE Signal Processing Magazine*, vol. 35, no. 1, 2018. doi arXiv
32. S. Rabanser, S. Günnemann and Z. C. Lipton, “Failing loudly: an empirical study of methods for detecting dataset shift,” NeurIPS, 2019. arXiv
33. D. Sculley et al., “Hidden technical debt in machine learning systems,” NeurIPS, 2015. paper
34. PyTorch, “Reproducibility,” PyTorch documentation. docs
35. M. Mitchell et al., “Model cards for model reporting,” ACM FAT*, 2019. arXiv
36. T. Gebru et al., “Datasheets for datasets,” *Communications of the ACM*, vol. 64, no. 12, 2021. doi
37. J. Pineau et al., “Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program),” *Journal of Machine Learning Research*, vol. 22, 2021.

paper
38. J. Gawlikowski et al., “A survey of uncertainty in deep neural networks,” *Artificial Intelligence Review*, vol. 56, 2023. doi
39. A. Paleyes, R.-G. Urma and N. D. Lawrence, “Challenges in deploying machine learning: a survey of case studies,” *ACM Computing Surveys*, vol. 55, no. 6, 2022. doi
40. P. Liang et al., “Holistic evaluation of language models,” *Transactions on Machine Learning Research*, 2023. arXiv
41. X. Zhu, J. Li, Y. Liu, C. Ma and W. Wang, “A survey on model compression for large language models,” *Transactions of the Association for Computational Linguistics*, vol. 12, 2024. doi
42. E. Frantar, S. Ashkboos, T. Hoefler and D. Alistarh, “GPTQ: accurate post-training quantization for generative pre-trained transformers,” ICLR, 2023. arXiv
43. J. Lin et al., “AWQ: activation-aware weight quantization for LLM compression and acceleration,” MLSys, 2024. arXiv
44. J. Hoffmann et al., “Training compute-optimal large language models,” arXiv:2203.15556, 2022; NeurIPS 2022 version titled “An empirical analysis of compute-optimal large language model training.” arXiv
45. X. Zhai, A. Kolesnikov, N. Houlsby and L. Beyer, “Scaling vision transformers,” CVPR, 2022. doi
46. PyTorch, “torchao documentation.” docs

Part VI

Deep Learning for Computer Vision

DLCV 01 • Deep Learning for Image Processing and Computer Vision

How deep learning reorganizes image processing, image analysis and computer vision: from the classical pattern-recognition pipeline to CNNs, Vision Transformers, self-supervised backbones and promptable foundation models.

Online edition: https://blog.shyandram.dev/en/learn/deep-learning/dl-01-deep-learning-for-image-processing/

What you'll learn

- How the classical three levels of image work (image processing, image analysis, computer vision) map onto the two families that deep learning actually uses: **low-level** and **high-level** vision tasks.
- Why the textbook pattern-recognition pipeline (sensor → features → classifier → evaluation) was replaced by **end-to-end learning**, and how it quietly came back as "pretrained backbone + light head".
- How a convolutional layer is a bank of **learned spatial filters**, and how kernel size, stride, padding and pooling set the **receptive field** and the feature hierarchy.
- The design reasons behind the landmark CNNs (LeNet-5, AlexNet, VGG, GoogLeNet, ResNet) and the encoder–decoder (FCN, U-Net) used for dense prediction.
- How the **Vision Transformer** works, why it needs large-scale pretraining, and how Swin, ConvNeXt and self-supervised backbones (DINO, MAE, DINOv2/v3) answered it.
- A map of today's tasks and representative models: classification, detection, segmentation (the SAM family), restoration and enhancement, generation (diffusion, latent diffusion, DiT) and vision–language models (CLIP, LLaVA).

The big picture

This tutorial is the entry point of the deep-learning track. It does not go deep into any one task. Instead it gives you the map: what kinds of image problems exist, which network designs solve them, and why those designs look the way they do. The later tutorials then zoom in on image classification, object detection, image restoration, low-light enhancement and object tracking.

> **Explain like I'm five**
>
> Think of a photo lab with two kinds of workers. The **repair crew** takes a scratched, blurry or dark photo and hands back a cleaner photo: picture in, picture out. The **detective** looks at a photo and tells you *what* is in it: "a cat, sitting on a sofa, on the left." Old-style computer vision gave both workers a fixed toolbox designed by experts. Deep learning lets them build their own tools by looking at millions

of examples. Today there is even a "super-worker" who learned from billions of pictures and can be told what to do with a few words.

Why it matters: almost every modern camera pipeline, medical-imaging product, driver-assistance system and photo app is built from the pieces in this tutorial. Knowing the map lets you pick a sensible starting point for a new problem, read papers critically, and understand which classical ideas from the Digital Image Processing series survive inside the networks (many do).

About this tutorial

This tutorial is expanded from the author's earlier post, written in Traditional Chinese, "深度學習的數位影像處理介紹" (An introduction to digital image processing with deep learning) [1]. It keeps the post's structure and viewpoints and adds the details, math and code.

Three levels of image tasks

Plain version. "Image processing" in the broad sense covers three kinds of work. Some operations turn an image into a better image. Some pull out a particular part or property of the image. Some try to *understand* the image the way a person would.

Precise version. Following Gonzalez and Woods [2], we can sort image work into three levels by what goes in and what comes out.

Level	Input → output	Typical operations	Classical examples
Image processing (narrow sense; low level)	image → image	primitive, pixel- or neighborhood-wise steps	histogram equalization, filtering, Fourier transform (IP 03, IP 04)
Image analysis (mid level)	image → attributes or highlighted regions	segmentation, feature extraction, description, using prior knowledge	skin-color regions in a face image, vessel regions in an X-ray (IP 10, IP 12)
Computer vision (high level; understanding)	image → meaning	recognition, interpretation, decision	classification, "is there a pedestrian?" (IP 13)

The boundaries are soft. Real systems mix all three. A vessel-segmentation tool, for instance, first denoises (low level), then extracts a vessel map (mid level), and may then flag a stenosis (high level). The levels describe *kinds of computation*, not separate departments.

How deep learning redraws the map

In deep-learning papers you will almost never see the word "mid-level". The field splits tasks into just two families, according to the **shape of the output** [1]:

- **Low-level vision**: the output is an image of (roughly) the same size as the input, and the target is a pixel-accurate signal. Denoising, deblurring, super-resolution, dehazing, low-light enhancement, image-to-image translation.
- **High-level vision**: the output is a *semantic* description. A label, boxes with labels, or a label per pixel.

The old "image analysis" level is absorbed into one of the two. Segmentation produces a per-pixel map, so it *looks* like an image-to-image task. But its values are class labels that require recognition, so it is grouped with high-level vision. Edge or vessel enhancement that outputs a cleaner intensity image is grouped with low-level vision.

The reason for the split is practical. The two families differ in the loss (pixel regression versus classification), the architecture (full-resolution encoder–decoders versus downsampling backbones), the training data (pairs of degraded and clean images versus human labels) and the metrics (fidelity versus accuracy). Figure 1 shows the reorganization.

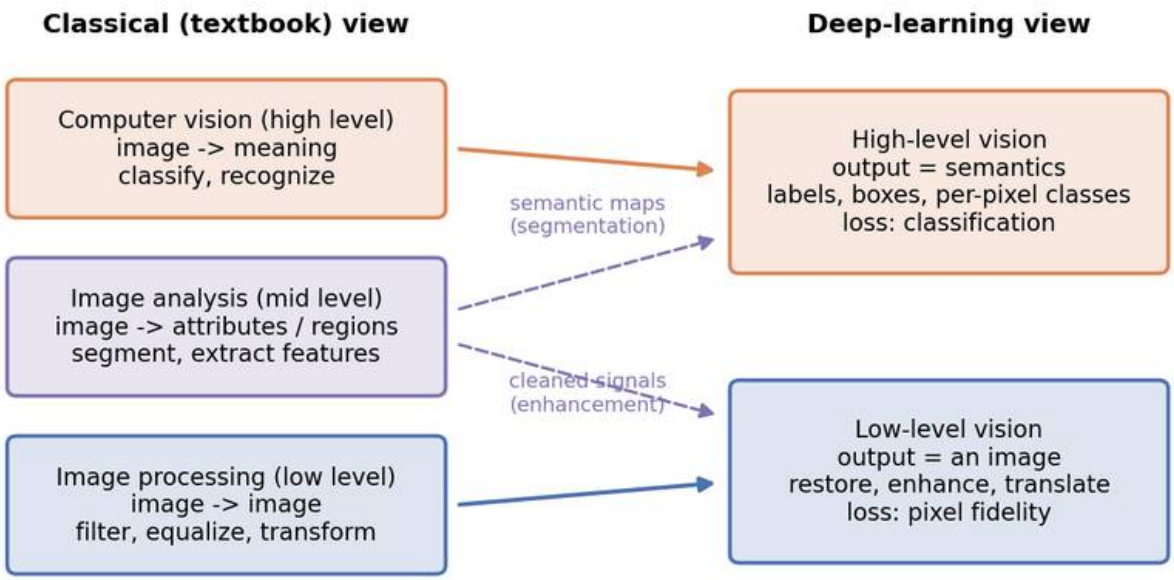


Figure 1 — The classical three levels (left) and the two families used in deep learning (right). Mid-level image analysis is split between them according to whether the output is a cleaned-up signal or a semantic map.

From the pattern-recognition pipeline to end-to-end learning

Plain version. Before deep learning, a recognition system was built like an assembly line: a camera, then a hand-designed step that measures useful properties, then a classifier that decides. Deep learning fuses the middle steps into one network that learns its own measurements.

Precise version. The classical pipeline [3] has four stages:

1. **Sensor.** Capture the pattern (camera, scanner, microscope).
2. **Feature generation and selection.** Compute a vector $\mathbf{x} = \phi(I) \in \mathbb{R}^d$ from image I, using a hand-designed map ϕ (moments, texture statistics, edge histograms, keypoint descriptors), then keep the most informative entries.
3. **Classifier design.** Learn a decision rule $\hat{y} = g(\mathbf{x}; \theta)$ with parameters θ, for example a Bayes classifier, a nearest-neighbor rule or a support-vector machine.
4. **System evaluation.** Measure the error rate on held-out data and go back to step 2 or 3 if it is too high.

Only θ is learned. The feature map ϕ is fixed by an engineer, so the whole system is only as good as ϕ. The chapter on feature extraction (IP 12) shows how much ingenuity went into those maps.

End-to-end learning makes the feature map learnable too. A deep network computes

$$\hat{y} = g(\phi(I; \mathbf{w}); \theta),$$

where $\phi(\cdot\,; \mathbf{w})$ is a stack of layers with weights $\mathbf{w}$, $g(\cdot\,; \theta)$ is a small head, and both $\mathbf{w}$ and θ are fitted jointly by minimizing a loss $\mathcal{L}(\hat{y}, y)$ over labeled pairs (I, y) with gradient descent and back-propagation. The input is raw pixels; nothing is hand-designed except the architecture and the loss.

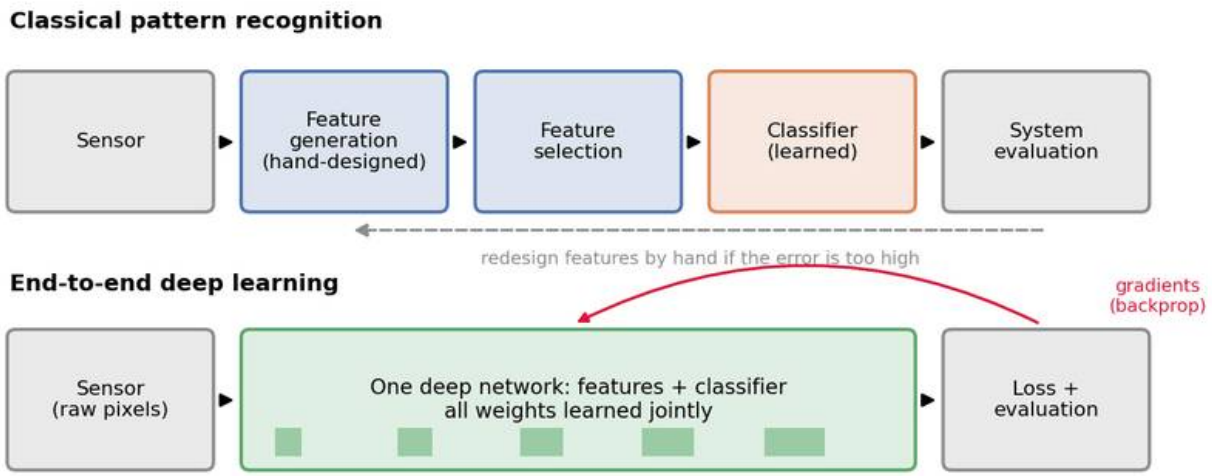


Figure 2 — Classical pattern recognition (top) versus end-to-end learning (bottom). In the deep network the functional split into 'features' and 'classifier' still exists, but there is no clean boundary between them.

The functional split still exists inside a network: early layers behave like feature extractors and the last layers like a classifier. But in practice you usually cannot point at a layer and say "here the features end" [1]. Two consequences follow.

- **Data replaces design.** End-to-end systems need many labeled examples; the hand-designed ϕ encoded prior knowledge that the network must now learn from data.

- **The split came back at a larger scale.** Since about 2021 the standard recipe is a **pretrained backbone** (trained once, often without labels, on a huge dataset) plus a **light task head** trained on your small dataset. That is "feature extractor + classifier" again, except the feature extractor is learned rather than designed. We return to this in the section on self-supervised backbones.

Convolutional neural networks

Plain version. A convolutional layer slides a small window over the image and computes a weighted sum at every position, exactly like the spatial filters of IP 03. The difference is that a CNN *learns* the weights of its filters, uses many filters at once, and stacks dozens of such layers with nonlinearities in between.

A convolutional layer is a bank of learned filters

Precise version. Let the input feature map be $X \in \mathbb{R}^{C_{\text{in}} \times H \times W}$ (channels, height, width). A convolutional layer with C_{out} filters of size $k \times k$ computes

$$Y_o(x,y) = b_o + \sum_{c=1}^{C_{\text{in}}} \sum_{i=-r}^{r} \sum_{j=-r}^{r} W_{o,c}(i,j)\, X_c(x+i,\, y+j), \qquad o = 1, \dots, C_{\text{out}},$$

where $r = (k-1)/2$ for odd k, $W_{o,c}(i,j)$ is the weight of filter o on input channel c at offset (i, j), and b_o is a bias. Strictly this is a **correlation** (no kernel flip), as explained in the convolution tutorial; since the weights are learned, the flip makes no difference. An elementwise nonlinearity such as $\text{ReLU}(z) = \max(0, z)$ follows. Without it, a stack of convolutions would collapse into one big linear filter [1].

Two properties make this layer the right tool for images:

- **Local connectivity.** Each output depends only on a $k \times k$ neighborhood, because nearby pixels are strongly related and distant ones much less so.
- **Weight sharing.** The same filter is used at every position. This makes the layer **translation-equivariant** (shift the input, and the output shifts the same way) and slashes the number of parameters.

The saving is huge. A fully connected layer from a $224 \times 224 \times 3$ image to 64 units needs $224 \cdot 224 \cdot 3 \cdot 64 + 64 \approx 9.6$ million parameters. A 3×3 convolution from 3 to 64 channels needs $3 \cdot 3 \cdot 3 \cdot 64 + 64 = 1{,}792$, and it produces 64 *full-resolution* feature maps instead of 64 numbers.

The code below sets the two filters of a PyTorch convolution by hand to Sobel kernels, which turns the layer into an edge detector. Training would instead set these numbers by gradient descent.

```
import torch, torch.nn as nn, torch.nn.functional as F
from skimage import data, img_as_float
```

```
x = torch.tensor(img_as_float(data.camera()), dtype=torch.float32)[None, None]  # (1, 1,
 ↪  512, 512)
conv = nn.Conv2d(1, 2, kernel_size=3, padding=1, bias=False)
sobel_x = torch.tensor([[-1., 0., 1.], [-2., 0., 2.], [-1., 0., 1.]])
with torch.no_grad():
    conv.weight[0, 0] = sobel_x        # channel 0: responds to vertical edges
    conv.weight[1, 0] = sobel_x.T      # channel 1: responds to horizontal edges
    y = F.relu(conv(x))                # (1, 2, 512, 512): two feature maps
```

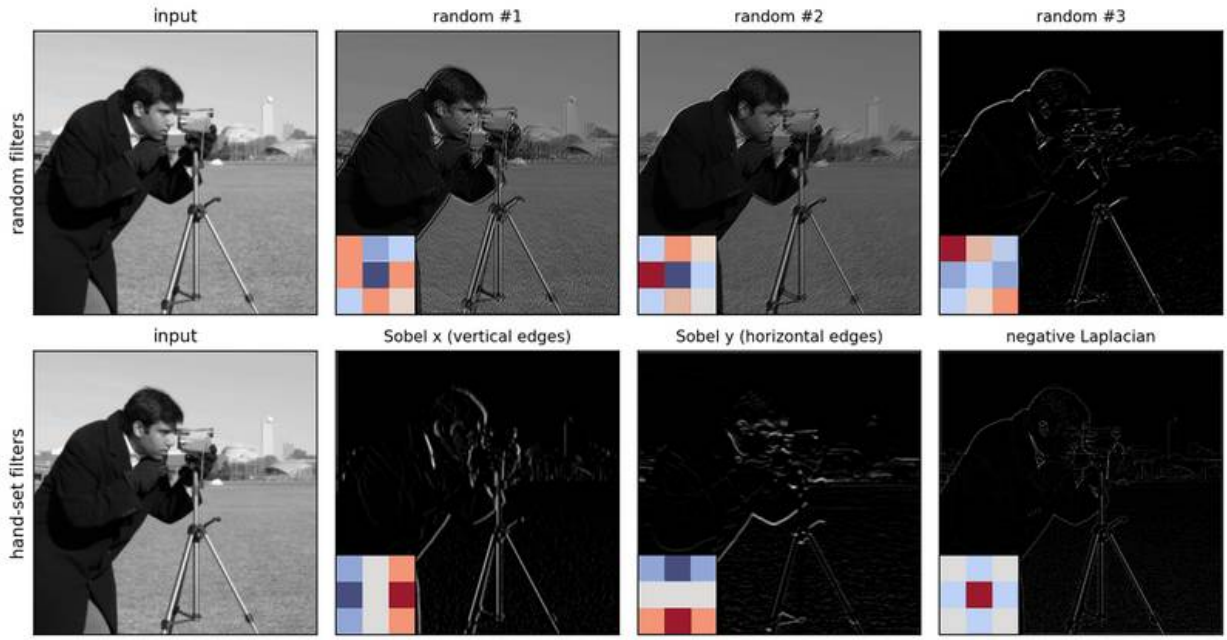


Figure 3 — Feature maps (after ReLU) of 3×3 convolutions on the camera image. Top: random initial weights, which give noisy mixtures of smoothing and edges. Bottom: hand-set Sobel and Laplacian weights. Training moves the random filters toward useful ones.

What do trained first-layer filters look like? We can train a tiny CNN on the 8×8 handwritten digits that ship with scikit-learn [36] in a few seconds on a CPU.

```
import torch, torch.nn as nn
from sklearn.datasets import load_digits
from sklearn.model_selection import train_test_split

torch.manual_seed(0)
X, y = load_digits(return_X_y=True)                    # 1,797 images of 8x8 pixels
X = torch.tensor(X / 16.0, dtype=torch.float32).reshape(-1, 1, 8, 8)
Xtr, Xte, ytr, yte = train_test_split(X, torch.tensor(y), test_size=0.25, random_state=0)

net = nn.Sequential(
    nn.Conv2d(1, 8, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),     # 8x8 -> 4x4
    nn.Conv2d(8, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),    # 4x4 -> 2x2
    nn.Flatten(), nn.Linear(16 * 2 * 2, 10),
```

```
)
opt = torch.optim.Adam(net.parameters(), lr=1e-2)
for epoch in range(30):
    for i in range(0, len(Xtr), 64):
        loss = nn.functional.cross_entropy(net(Xtr[i:i + 64]), ytr[i:i + 64])
        opt.zero_grad(); loss.backward(); opt.step()
print((net(Xte).argmax(1) == yte).float().mean())       # about 0.97
filters = net[0].weight.detach()[:, 0]                   # (8, 3, 3) learned kernels
```

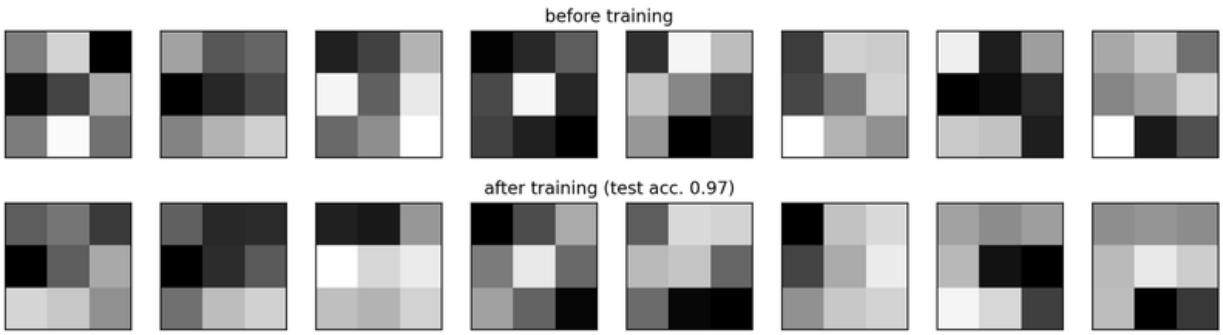


Figure 4 — The eight first-layer 3×3 filters of the tiny digit CNN, before (top) and after (bottom) training. Several drift toward oriented light-to-dark transitions, the derivative-like patterns a hand designer would choose; nobody told the network to do so. (Tiny 8×8 inputs and 3×3 kernels keep the effect modest; large networks trained on natural images show it much more clearly.)

Stride, padding and output size

Plain version. Padding adds a border so the window can sit on edge pixels. Stride makes the window jump more than one pixel at a time, which shrinks the output.

Precise version. For an input of width W, kernel size k, padding p (pixels added on each side) and stride s, the output width is

$$W_{\text{out}} = \left\lfloor \frac{W + 2p - k}{s} \right\rfloor + 1.$$

With $k = 3$, $p = 1$, $s = 1$ the size is preserved ("same" padding); with $s = 2$ it is halved. Zero padding is the default; reflect or replicate padding, the same boundary choices discussed in IP 03, are preferred in low-level networks because zero borders create dark artifacts at image edges.

Pooling

Pooling summarizes each small window by one number, usually the maximum (max pooling) or the mean (average pooling), typically with a 2×2 window and stride 2. It halves resolution, makes the representation tolerant to small shifts, and cheaply enlarges the receptive field. Many modern networks replace it with strided convolutions, which learn how to downsample. At the

very end of a classification network, **global average pooling** turns a $C \times H \times W$ map into a C-vector for the classifier head.

Receptive field

Plain version. The receptive field of a neuron is the patch of the input image that can influence it. One 3×3 layer sees 3×3 pixels. Stack two, and each output sees 5×5. Downsampling makes it grow much faster.

Precise version. Number the layers $\ell = 1, \dots, L$ with kernel size k_ℓ and stride s_ℓ. Let j_ℓ be the **jump**, the distance in input pixels between two neighboring units of layer ℓ, and r_ℓ the receptive-field size, with $r_0 = 1$, $j_0 = 1$ for the input. Then

$$r_\ell = r_{\ell-1} + (k_\ell - 1)\, j_{\ell-1}, \qquad j_\ell = j_{\ell-1}\, s_\ell.$$

Each layer adds $k_\ell - 1$ units of its input, and each input unit is worth $j_{\ell-1}$ pixels. With stride 1 everywhere, L layers of 3×3 give $r_L = 2L + 1$: linear growth. Every stride-2 step doubles the jump, so later layers grow the field twice as fast, giving roughly exponential growth in depth.

```
def receptive_field(layers):
    r, j = 1, 1                       # receptive field and jump, measured in input pixels
    for k, s in layers:               # (kernel size, stride) of each layer
        r, j = r + (k - 1) * j, j * s
    return r

print(receptive_field([(3, 1)] * 3))                                          # 7
print(receptive_field([(3, 1), (3, 1), (2, 2), (3, 1), (3, 1), (2, 2), (3, 1)])) # 24
```

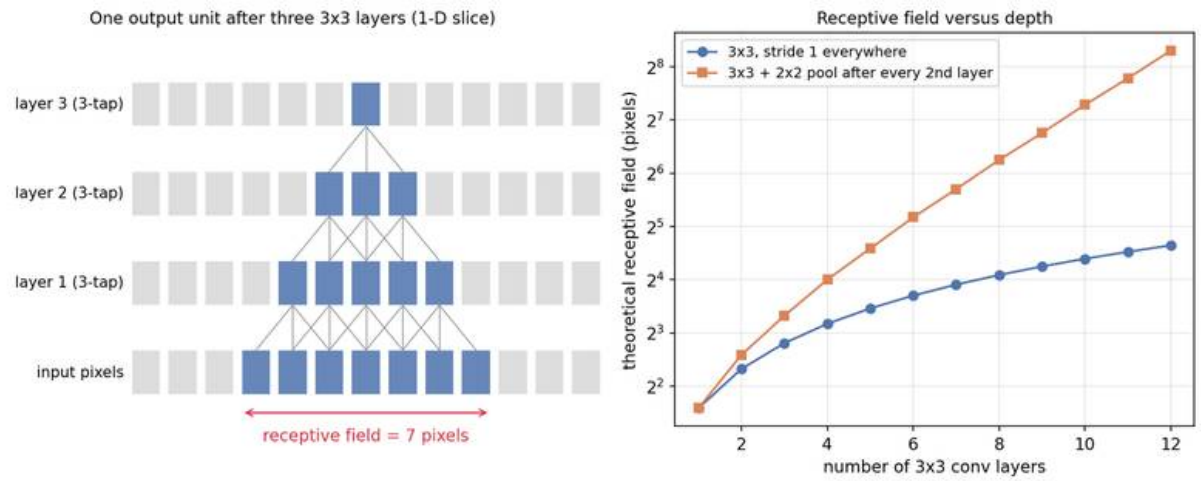


Figure 5 — Left: three stacked 3-tap layers give one output a 7-pixel receptive field. Right: theoretical receptive field versus depth, for stride 1 everywhere and for 2× downsampling after every second layer.

Interactive demonstration (online): open the online edition of this tutorial to use this demonstration.Open demonstration

Two refinements matter in practice. First, the formula gives the **theoretical** receptive field. Luo et al. [12] showed that the **effective** receptive field, measured by how much each input pixel actually affects the output gradient, is roughly Gaussian and occupies only a fraction of the theoretical one. Pixels near the center dominate. Second, the receptive field decides what a network *can* use: a denoiser whose receptive field is smaller than the noise correlation length cannot remove that noise, and a classifier whose last layer sees only part of an object cannot recognize it from context.

Feature hierarchies

Because the receptive field grows with depth while resolution shrinks, a CNN builds a **feature hierarchy**: early layers respond to edges and color blobs (Figure 4), middle layers to textures and parts, late layers to object-level patterns over large regions. A typical backbone is organized in **stages**, each ending with a 2× downsampling and doubling the channel count, so features go from "high-resolution, few channels, local" to "low-resolution, many channels, semantic". High-level tasks read the late stages. Dense prediction and low-level tasks need the early, high-resolution stages too, which is why encoder–decoders exist (below).

Landmark CNN architectures

Plain version. Between 2012 and 2016 a series of networks, each deeper than the last, won the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) [6]. Each one introduced a design idea that is still in use.

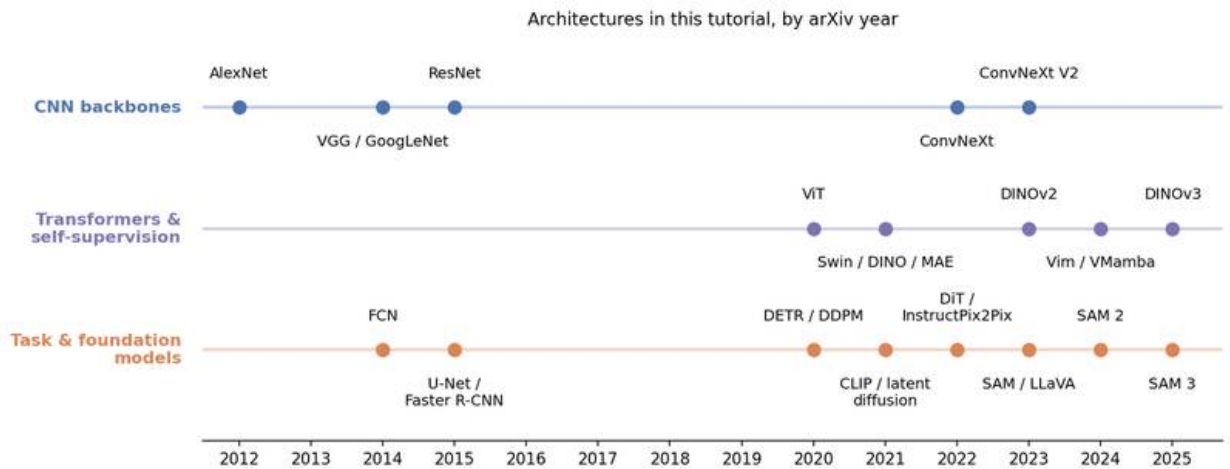


Figure 6 — A timeline of the architectures discussed in this tutorial, by arXiv year. LeNet-5 (1998) and the earlier convolutional networks of the late 1980s sit to the left of this axis.

LeNet-5 and where CNNs really began

The post's original heading said "CNN (1998~)". That year is when LeCun, Bottou, Bengio and Haffner published LeNet-5 [5], not when CNNs started: convolutional networks trained by back-propagation were already reading handwritten ZIP-code digits in 1989 [4].

LeNet-5 is a textbook pattern-recognition system in network form [1]. Layers C1 to C5 are the **feature extractor**: convolutions (C1, C3, C5) alternating with subsampling layers (S2, S4) that turn a 32×32 digit image into a 120-dimensional feature vector. F6 and the output layer are the **classifier**. Everything, features included, is trained by gradient descent on the classification loss. This was the end-to-end idea, already in place, but it needed 15 more years of data and compute to dominate.

AlexNet (2012)

AlexNet [7] has five convolutional layers and three fully connected layers (eight learned layers, about 60 million parameters). Its ingredients were not individually new, but the combination was: ReLU activations, which train much faster than saturating sigmoids; an efficient GPU implementation; heavy data augmentation; and dropout in the fully connected layers. It won ILSVRC 2012 by a wide margin, which is why 2012 is usually given as the start of the deep-learning era in vision.

VGG (2014)

Simonyan and Zisserman [8] asked a single question: what happens if you use only 3×3 convolutions and make the network deeper? Their best configurations had 16 and 19 weight layers. The reason 3×3 is enough is the receptive-field formula above: two stacked 3×3 layers see 5×5, three see 7×7. For C channels in and out, three 3×3 layers cost $3 \cdot 9C^2 = 27C^2$ weights versus $49C^2$ for one 7×7 layer, and they insert two extra nonlinearities. VGG's uniform design made it the default pretrained feature extractor for years.

GoogLeNet / Inception (2014)

GoogLeNet [9], 22 layers deep, won ILSVRC 2014. Its **Inception module** runs 1×1, 3×3 and 5×5 convolutions and a pooling branch in parallel and concatenates their outputs, so each block sees several scales at once. To keep this affordable, cheap 1×1 convolutions first reduce the number of channels. A 1×1 convolution mixes channels at each pixel without looking at neighbors; it is a per-pixel fully connected layer, and it remains a basic building block everywhere.

ResNet (2015) and why residual connections work

Deeper should be better, but in practice it was not: He et al. [10] observed that a plain 56-layer network had *higher training error* than a 20-layer one. That is not overfitting. It is an optimization

failure: the deeper net could in principle copy the shallow one and set the extra layers to the identity, but gradient descent does not find that solution.

The fix is the **residual block** (Figure 7). Instead of asking a stack of layers to learn a mapping $H(\mathbf{x})$ directly, let it learn the residual $F(\mathbf{x}) = H(\mathbf{x}) - \mathbf{x}$ and add the input back:

$$\mathbf{y} = \sigma(\mathbf{x} + F(\mathbf{x}; W)),$$

where $\mathbf{x}$ is the block input, F is typically conv–BN–ReLU–conv–BN with weights W (BN is batch normalization), and σ is ReLU. When channel counts or resolution change, the shortcut uses a 1×1 convolution instead of the identity.

Why it works:

- **Identity is easy.** If extra depth is not useful, the block only has to drive F toward zero, which is far easier than making a stack of nonlinear layers imitate the identity. Initializing the last BN scale of F at zero makes each block start as the identity.
- **Gradients have a highway.** Ignoring the final nonlinearity, $\partial \mathbf{y}/\partial \mathbf{x} = I + \partial F/\partial \mathbf{x}$. The identity term carries the gradient from the loss to early layers without passing through every weight matrix, so it does not vanish with depth.
- **An ensemble view.** Unrolling L blocks gives a sum over many paths of different lengths, so a deep ResNet behaves partly like a collection of shallower networks.

```
class ResidualBlock(nn.Module):
    """Basic block: y = ReLU(x + F(x)), with F = conv-BN-ReLU-conv-BN."""
    def __init__(self, c):
        super().__init__()
        self.f = nn.Sequential(
            nn.Conv2d(c, c, 3, padding=1, bias=False), nn.BatchNorm2d(c), nn.ReLU(),
            nn.Conv2d(c, c, 3, padding=1, bias=False), nn.BatchNorm2d(c),
        )
    def forward(self, x):
        return torch.relu(x + self.f(x))

blk = ResidualBlock(16)
nn.init.zeros_(blk.f[4].weight)                  # zero the last BN scale, so F(x) = 0
x = torch.randn(2, 16, 32, 32)
print(torch.allclose(blk(x), torch.relu(x)))     # True: the block starts as the identity
 ↪  (then ReLU)
```

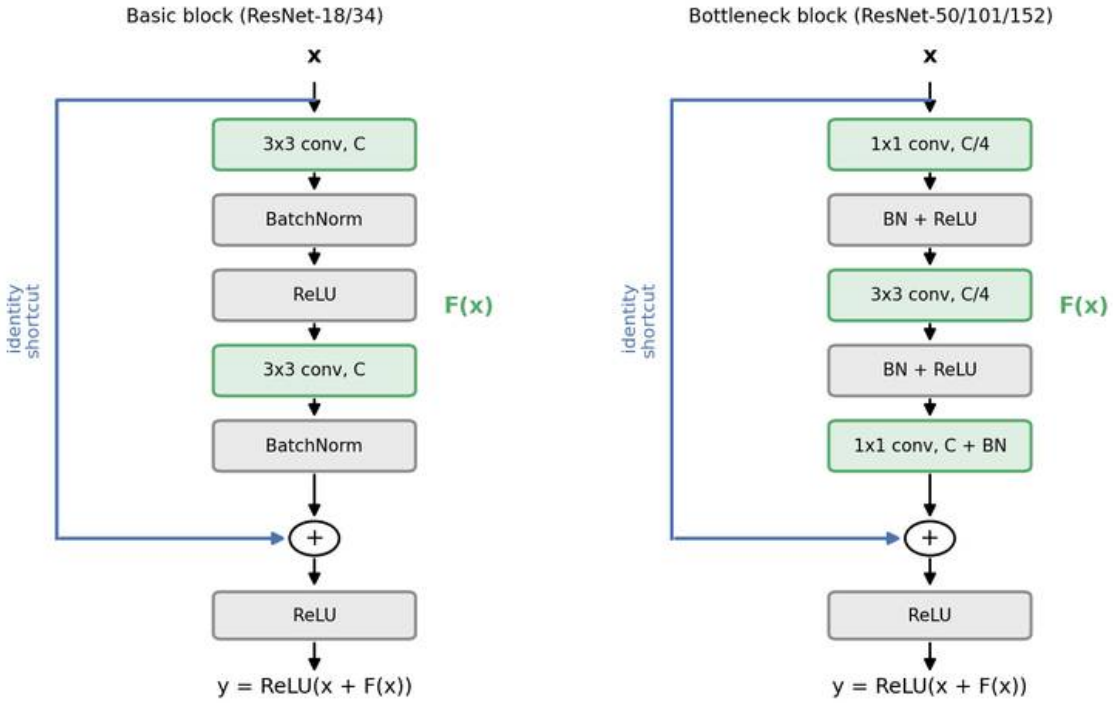


Figure 7 — Left: the basic residual block. Right: the bottleneck block used in ResNet-50/101/152, where 1×1 convolutions shrink and restore the channel count around a cheap 3×3 convolution.

On ImageNet the paper trained ResNets with **18, 34, 50, 101 and 152 layers**; the deeper three use the bottleneck block. The famous **1202-layer** network appears only in the paper's CIFAR-10 experiments [10], [11] (32×32 images), where it trained fine but overfit and tested worse than the 110-layer version. The ResNet family won ILSVRC 2015, and the residual connection is now in almost every deep network, Transformers included.

Encoder–decoders for dense prediction

Plain version. A classification backbone throws away resolution to gain meaning. For segmentation or restoration we need an answer at every pixel, so we add a second half that brings the resolution back.

Precise version. Long et al.'s **fully convolutional network** (**FCN**) [13] replaced the fully connected layers of a classifier with convolutions, so the network outputs a coarse class-score map for an input of any size, then upsampled it with learned transposed convolutions and fused it with scores from earlier, finer layers. **U-Net** [14] made this symmetric: a contracting **encoder** (conv blocks and downsampling) and an expanding **decoder** (upsampling and conv blocks), with **skip connections** that concatenate each encoder stage's feature map to the decoder stage of the same resolution. The encoder supplies *what*; the skips supply *where*. U-Net was designed for biomedical images with few labels and strong augmentation, and the same shape now underlies medical segmentation, almost every restoration network covered in DLCV 04, and the denoiser

inside many diffusion models.

The Vision Transformer

Plain version. The Transformer [15] was built for sentences: it treats the input as a sequence of tokens and lets every token look at every other token. The Vision Transformer (ViT) [16] cuts an image into small square patches, treats each patch as a "word", and feeds the sequence to an unmodified Transformer encoder.

Patchify, embed, attend

Precise version. For an image $I \in \mathbb{R}^{C\times H\times W}$ and patch size P:

1. **Patchify.** Split I into $N = HW/P^2$ non-overlapping patches and flatten each into a vector $\mathbf{p}_i \in \mathbb{R}^{CP^2}$. For a 224×224 RGB image and $P = 16$, $N = 196$ and each vector has 768 entries.
2. **Embed.** Map each patch linearly to a D-dimensional token, $\mathbf{z}_i = E\,\mathbf{p}_i$ with $E \in \mathbb{R}^{D\times CP^2}$. Prepend a learnable class token $\mathbf{z}_{\text{cls}}$ and add learnable **position embeddings** $\mathbf{e}_i$, because attention by itself has no notion of where a token came from: $Z^{(0)} = [\mathbf{z}_{\text{cls}}; \mathbf{z}_1; \dots; \mathbf{z}_N] + [\mathbf{e}_0; \dots; \mathbf{e}_N]$.
3. **Encode.** Apply L Transformer encoder blocks, each a multi-head self-attention (MSA) sublayer and an MLP sublayer, both with layer normalization (LN) and residual connections:

$$Z' = Z^{(\ell-1)} + \mathrm{MSA}(\mathrm{LN}(Z^{(\ell-1)})),$$
$$Z^{(\ell)} = Z' + \mathrm{MLP}(\mathrm{LN}(Z')).$$

4. **Head.** Feed the final class token (or the average of all tokens) to an MLP head, which plays the role of the pattern-recognition classifier [1].

A single attention head computes, for token matrix $Z \in \mathbb{R}^{(N+1)\times D}$,

$$\mathrm{Attn}(Z) = \mathrm{softmax}\!\left(\frac{QK^\top}{\sqrt{d}}\right)V, \qquad Q = ZW_Q,\; K = ZW_K,\; V = ZW_V,$$

where $W_Q, W_K, W_V \in \mathbb{R}^{D\times d}$ are learned projections and d is the head dimension. Row i of the softmax matrix says how much token i draws from every other token. Compared with a convolution, the "filter" is computed from the content and covers the whole image from the first layer on. The cost is quadratic in the number of tokens, $O(N^2 d)$, which is why patches rather than pixels are used.

```
img = torch.randn(1, 3, 224, 224)
P, D = 16, 768
patches = img.unfold(2, P, P).unfold(3, P, P)                    # (1, 3, 14, 14, 16, 16)
patches = patches.permute(0, 2, 3, 1, 4, 5).reshape(1, -1, 3 * P * P)
print(patches.shape)                                              # (1, 196, 768)
```

```
embed = nn.Conv2d(3, D, kernel_size=P, stride=P)                    # patchify + linear embed
 ↪  in one op
tokens = embed(img).flatten(2).transpose(1, 2)                      # (1, 196, 768)
```

The last two lines show a useful identity: patch embedding *is* a convolution with kernel size and stride both equal to P.

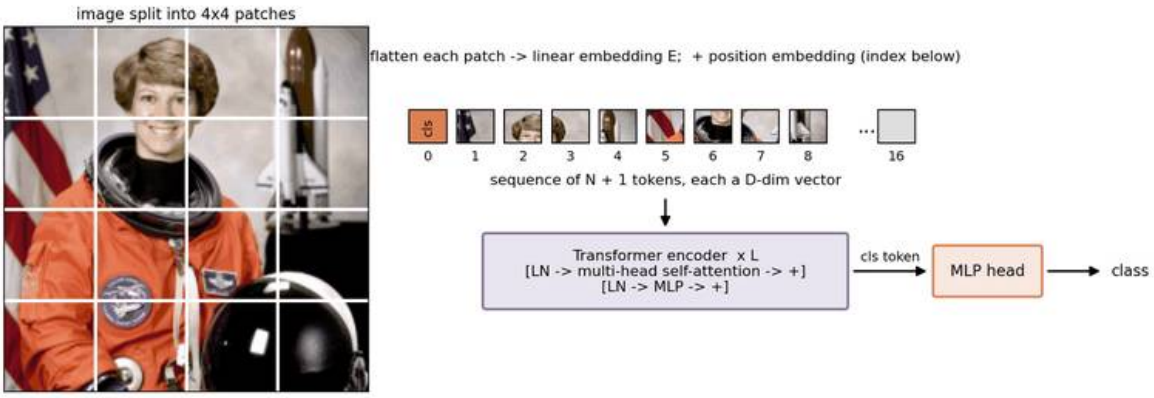


Figure 8 — ViT in one picture: patchify, linearly embed, add position embeddings and a class token, run a Transformer encoder, classify from the class token. (A 4×4 grid is drawn for legibility; ViT-B/16 on 224×224 inputs uses 14×14 patches.)

Inductive bias and the need for scale

The original post said that ViT is "harder to converge". The more precise statement [1], [16] is about **inductive bias**, the assumptions built into an architecture before it sees data. A CNN assumes locality and weight sharing, so it already "knows" that nearby pixels matter together and that a pattern means the same thing anywhere in the image. ViT assumes almost nothing: apart from the patch split, all spatial structure, even the 2-D layout of the position embeddings, must be learned.

That is a trade-off, not a defect. With little data, the CNN's assumptions are a valuable head start, and ViT trained from scratch on ImageNet-1k alone underperforms comparable ResNets. With enough data, the assumptions become a limitation, and ViT pretrained on very large datasets matched or beat the best CNNs when fine-tuned [16]. The practical lesson is that ViTs are used **pretrained**, either on huge labeled sets or, increasingly, with self-supervision.

Backbones after ViT

Plain version. After ViT, research went three ways: make Transformers more image-friendly (Swin), show that CNNs can catch up when modernized (ConvNeXt), and pretrain without labels so that one backbone serves every task (DINO, MAE, DINOv2/v3).

Swin Transformer: locality returns

Swin [17] computes attention only inside non-overlapping local windows (for example 7×7 tokens), which makes the cost linear in image size. In alternate blocks the window grid is **shifted** by half a window, so information crosses window borders. Between stages, neighboring tokens are merged to halve resolution. The result is a hierarchical, multi-scale feature pyramid shaped like a CNN backbone, which is exactly what detection and segmentation heads expect. Swin put locality and hierarchy, the CNN's inductive biases, back into a Transformer.

ConvNeXt: the CNN counterpoint

Liu et al. [18] started from a ResNet-50 and modernized it one step at a time, using the training recipe and design choices of Transformers: a "patchify" stem (a 4×4 convolution with stride 4), large 7×7 depthwise convolutions, an inverted-bottleneck MLP, fewer activation and normalization layers, and layer normalization. The resulting pure CNN, **ConvNeXt**, matched Swin in accuracy and scaling on classification, detection and segmentation. The lesson: much of ViT's advantage came from training recipes and macro design, not from attention itself. **ConvNeXt V2** [18] then added masked-autoencoder pretraining adapted to convolutions (the fully convolutional masked autoencoder, FCMAE) and a global response normalization layer. Today large foundation models mostly use ViT backbones, but convolutions remain common in low-level vision and on edge devices [1].

Self-supervised pretraining: DINO versus MAE

Self-supervised learning creates a training signal from the images themselves, so no labels are needed. Two recipes dominate, and they are often confused.

DINO: self-distillation [19]. A student network g_s and a teacher g_t with the same architecture each output a probability vector over K prototype dimensions. Several random crops of an image are made: two large "global" views and several small "local" views. The teacher sees only global views, the student sees all of them, and the student is trained to match the teacher's output:

$$\min_{\theta_s} \sum_{x \in \{x_1^g, x_2^g\}} \sum_{\substack{x' \in V \\ x' \neq x}} H(P_t(x),\, P_s(x')), \qquad H(a, b) = -a^\top \log b,$$

where V is the set of all views, P_t and P_s are the softmax outputs of teacher and student, and θ_s are the student's weights. The teacher is not trained by gradients. Its weights are an exponential moving average of the student's, $\theta_t \leftarrow \lambda \theta_t + (1-\lambda)\theta_s$ with λ close to 1, and its outputs are centered and sharpened so that training does not collapse to a constant output. Matching "small crop" to "whole image" forces the network to recognize objects from parts. A remarkable emergent property is that the attention maps of a DINO ViT segment the main objects without ever seeing a mask.

MAE: masked prediction [20]. The masked autoencoder hides a large random subset of patches (75 % works best), encodes only the visible ones with a ViT, and asks a light decoder to reconstruct

the missing pixels:

$$\mathcal{L}_{\text{MAE}} = \frac{1}{|\mathcal{M}|} \sum_{i \in \mathcal{M}} \|\hat{\mathbf{p}}_i - \mathbf{p}_i\|_2^2,$$

where $\mathcal{M}$ is the set of masked patch indices, $\mathbf{p}_i$ the (normalized) pixels of patch i and $\hat{\mathbf{p}}_i$ the reconstruction. This "hide and predict" game is the image analogue of masked-word prediction in language models; DINO, by contrast, is a teacher–student distillation method [1]. MAE is cheap because the encoder processes only a quarter of the tokens, and it fine-tunes very well. DINO-style features are better **frozen**, used as-is with a linear head.

DINOv2 and DINOv3: frozen general-purpose features

DINOv2 [21] scaled DINO-style self-distillation to a 1-billion-parameter ViT trained on a large, automatically **curated** image collection, then distilled it into smaller models. Its frozen features work well for both image-level tasks (classification, retrieval) and pixel-level tasks (segmentation, depth estimation) with only a light head. **DINOv3** [21] scaled data and model size further and introduced **Gram anchoring**, a regularizer that keeps dense patch features from degrading during very long training, so the same frozen backbone gives high-quality dense features.

This closes the loop with the pattern-recognition pipeline. "Feature extractor + classifier" has returned as **pretrained frozen backbone + light head**. The feature extractor is now learned once, at very large scale, without labels.

State-space backbones

Vision Mamba (**Vim**) and **VMamba** [22] replace attention with selective state-space models, which scan the token sequence with cost linear in its length. Images are not sequences, so these models scan the patch grid along more than one path (Vim is bidirectional; VMamba's 2-D selective scan traverses several routes). They are attractive for high-resolution inputs but are still mostly a research direction rather than a default backbone.

High-level vision tasks

Plain version. High-level tasks differ in how *where* the answer must be: one label for the whole image, a box per object, or a label for every pixel.

Task	Output	Typical loss	Typical metric	Representative models
Classification	one label (or a probability vector)	cross-entropy	top-1 / top-5 accuracy	ResNet, ViT, ConvNeXt [10], [16], [18]

Task	Output	Typical loss	Typical metric	Representative models
Object detection	a set of (box, class, score)	classification + box regression, or set matching	average precision (AP) over IoU thresholds	Faster R-CNN, DETR [23]
Semantic segmentation	a class per pixel	per-pixel cross-entropy, Dice (an overlap loss, below)	mean IoU	FCN, U-Net [13], [14]
Instance segmentation	a mask and class per object	detection + mask losses	mask AP	promptable: SAM family [24]

Classification

Assign one label to the whole image. The network is a backbone plus global pooling plus a linear layer, trained with cross-entropy $\mathcal{L} = -\log \hat{p}_y$, where $\hat{p}_y$ is the predicted probability of the true class y. Classification is also the **pretraining task** that produced most backbones before self-supervision. DLCV 02 covers it in depth.

Object detection

Find every object, its class and its location as an axis-aligned box (a region of interest, ROI, given by its corner coordinates or center, width and height) [1]. Two design lineages illustrate the field [23]. **Faster R-CNN** is two-stage: a region proposal network suggests candidate boxes from shared CNN features, and a second head classifies and refines each one; duplicates are removed by non-maximum suppression. **DETR** treats detection as **set prediction**: a Transformer decoder emits a fixed set of predictions, and a bipartite matching between predictions and ground-truth objects defines the loss, which removes hand-designed anchors and non-maximum suppression. Box quality is measured by intersection over union,

$$\mathrm{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|},$$

where A and B are the predicted and true regions. The closely related Dice coefficient $2|A \cap B|/(|A| + |B|)$ is used, in a differentiable soft form, as a segmentation loss. Average precision summarizes the precision–recall curve at one or more IoU thresholds. Details are in DLCV 03.

Semantic and instance segmentation

Here the region of interest is the object's exact set of pixels, not a box [1].

- **Semantic segmentation** labels each pixel with a class. Two overlapping people become one "person" region.
- **Instance segmentation** also separates objects of the same class. The two people become "person 1" and "person 2".

Semantic segmentation is the natural job of the encoder–decoders above [13], [14]. Instance segmentation adds a detection-like component that proposes one mask per object.

Promptable segmentation: SAM, SAM 2, SAM 3

Segmentation now has **promptable foundation models** [24]. **SAM** (Segment Anything) takes an image and a prompt (points, a box, or a rough mask) and returns a mask for the indicated object. A heavy ViT image encoder runs once per image, and a light prompt encoder and mask decoder run per prompt, so interaction is fast. It was trained on SA-1B, a dataset of more than 1 billion masks on 11 million images, built with a model-in-the-loop "data engine". **SAM 2** extends this to video with a streaming memory that carries the object from frame to frame, trained on a new video dataset (SA-V). **SAM 3** adds **promptable concept segmentation**: given a short noun phrase such as "yellow school bus", image exemplars, or both, it finds, segments and (in video) tracks *all* matching instances, rather than the one object a click points at. With SAM 3 the line between detection, instance segmentation and tracking becomes thin; tracking is covered in DLCV 06.

Low-level vision tasks

Plain version. Low-level tasks take an image and return an improved or transformed image of the same scene. The output is judged pixel by pixel, or by how natural it looks.

The general formulation is an inverse problem. A degraded observation $\mathbf{y}$ is modeled as

$$\mathbf{y} = \mathcal{D}(\mathbf{x}) + \mathbf{n},$$

where $\mathbf{x}$ is the clean image, $\mathcal{D}$ a degradation operator (blur, downsampling, haze, low exposure) and $\mathbf{n}$ noise, the same model as in IP 05. A network f_θ is trained on pairs to minimize, for example, $\|f_\theta(\mathbf{y}) - \mathbf{x}\|_1$. Because real paired data is scarce, pairs are usually **synthesized** by applying a modeled degradation to clean images, and the gap between synthetic and real degradations is the field's central difficulty.

- **Image restoration** recovers the clean image: denoising, deblurring, super-resolution (small to large), dehazing (hazy to clear), deraining. Architectures are full-resolution encoder–decoders (U-Net-like) or residual networks without downsampling, now often with window attention. See DLCV 04.
- **Image enhancement** makes content easier to see without a single "true" target: low-light enhancement (bringing out detail in dark images, roughly a smart brightening; turning night into day would be translation instead) and underwater enhancement. See DLCV 05.
- **Image alignment** (registration) estimates the geometric transform or dense flow that maps one image onto another, a prerequisite for burst fusion, HDR and medical image comparison.
- **Image-to-image translation** maps an image from one domain or style to another: day to night, summer to winter, sketch to photo. Face-swapping "deepfakes" are a (misused) application of the same idea. Since diffusion models, translation and editing are often done by a generative model conditioned on the input image, as discussed next.

Generation and vision–language models

Plain version. Two newer families do not fit the old map well: models that **create** images from text, and models that **talk** about images. Both rely on large-scale pretraining and on connecting images with language.

Diffusion models: from noise to image

A **denoising diffusion probabilistic model** (**DDPM**) [27] defines a forward process that gradually adds Gaussian noise to an image $\mathbf{x}_0$ over T steps. In closed form,

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, I),$$

where $t \in \{1, \dots, T\}$ is the step and $\bar{\alpha}_t \in (0, 1)$ is a fixed schedule that decreases from nearly 1 to nearly 0. A network $\boldsymbol{\epsilon}_\theta$ is trained to predict the noise that was added,

$$\mathcal{L} = \mathbb{E}_{\mathbf{x}_0, t, \boldsymbol{\epsilon}} \|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t)\|_2^2,$$

and generation runs the process backwards, starting from pure noise and denoising step by step. In other words, a generator is a **learned denoiser applied many times**, which ties generation directly to the restoration problems above.

- **Latent diffusion** [28] runs the diffusion in the compressed latent space of a pretrained autoencoder rather than on pixels, which cuts the cost enough for high-resolution text-to-image synthesis; text conditioning enters through cross-attention. It is the basis of Stable Diffusion.
- **DiT** (Diffusion Transformer) [28] replaces the U-Net denoiser with a Transformer on latent patches and shows that quality improves steadily as the model's compute grows.
- **InstructPix2Pix** [29] edits a given image according to a written instruction ("make it winter"), so classic image-to-image translation becomes a special case of conditional generation.

Vision–language models

Visual question answering (VQA), answering a free-form question about an image, once required a pipeline of image features, captioning and text question answering [1]. Two steps made it a single model.

CLIP [25] trains an image encoder f and a text encoder g on 400 million image–text pairs with a **contrastive** objective. In a batch of B pairs, with normalized embeddings $\mathbf{u}_i = f(I_i)$ and $\mathbf{v}_j = g(T_j)$ and temperature τ, the image-to-text loss is

$$\mathcal{L}_{I \to T} = -\frac{1}{B} \sum_{i=1}^{B} \log \frac{\exp(\mathbf{u}_i^\top \mathbf{v}_i / \tau)}{\sum_{j=1}^{B} \exp(\mathbf{u}_i^\top \mathbf{v}_j / \tau)},$$

and the total loss averages it with the symmetric text-to-image term. Matching pairs are pulled together and all other pairs in the batch pushed apart. Because the label space is now "any sen-

tence", CLIP can classify zero-shot: embed the prompts "a photo of a cat", "a photo of a dog", ... and pick the one closest to the image.

LLaVA [26] connects a CLIP vision encoder to a large language model (LLM) through a projection layer that maps visual tokens into the LLM's word-embedding space, then fine-tunes on instruction-following conversations about images. Most current multimodal LLMs keep this **vision encoder + projector + LLM** pattern, so VQA, captioning and many recognition tasks are now handled by prompting one model.

Modern view

The trend across this tutorial is a move from "one task, one model" to "one foundation model + prompt or light head". The surveys below are a good map of how the field got there.

CNN architectures. Li et al. [30] review CNNs from their history through the convolution operation itself, classic and advanced architectures (the LeNet–ResNet line, lightweight and attention-augmented networks), 1-D, 2-D and multi-dimensional convolutions, and applications. They also cover the improvements made to each component (layer design, activation and loss functions, regularization, optimization, fast computation), run experiments to derive rules of thumb for choosing functions and hyperparameters, and close with open issues and promising directions. It is the best single place to see the CNN design space laid out component by component.

Vision Transformers. Khan et al. [31] (ACM Computing Surveys) organize Transformer work by task: the foundations (self-attention, large-scale pretraining, bidirectional encoding), then recognition (classification, detection, segmentation, action recognition), generative modeling, multimodal tasks (VQA, visual reasoning, grounding), video, low-level vision (super-resolution, enhancement, colorization) and 3-D point clouds. Their central contrast matches this tutorial's: Transformers model long-range dependencies with minimal inductive bias, so they need large-scale pretraining or architectural priors (windows, hierarchies) to be data-efficient.

Self-supervised visual learning. Jing and Tian [32] (TPAMI) survey the pre-2020 generation of self-supervised methods, organized by pretext task (generation-based, context-based, free-semantic-label-based and cross-modal), along with the standard protocol of evaluating features by transfer to downstream tasks. Balestriero et al. [33] cover the modern generation in a practical "cookbook" and group methods into four families: deep metric learning (contrastive), self-distillation (the DINO family), canonical-correlation-style methods, and masked image modeling (MAE). Their emphasis on the many hidden "knobs" (augmentations, projector heads, collapse prevention) explains why reproducing self-supervised results is hard.

Foundation models for vision. Awais et al. [34] (TPAMI) categorize vision foundation models by modality pairing (vision with text, audio, depth), training objective (contrastive versus generative) and **prompt type**: textual, visual (points, boxes, masks, as in SAM) and heterogeneous. Their open challenges are evaluation and benchmarking, real-world understanding and contextual reasoning, bias, adversarial robustness and interpretability. Zhang et al. [35] (TPAMI) focus on vision–language models for visual recognition: architectures and pretraining objectives, transfer methods (prompt tuning, adapters) and knowledge distillation into detectors and segmenters.

They argue that web-scale image–text data reduces the dependence on crowd-sourced labels and lets one model serve many tasks.

State of the art, 2024–2026. Without quoting benchmark numbers, the landscape looks like this:

- **Backbones.** Large ViTs pretrained with self-supervision or image–text data are the default for high-level tasks, with frozen DINOv2/DINOv3 features [21] a strong baseline for dense tasks. Modern CNNs (ConvNeXt V2 [18]) remain competitive and are preferred where latency, memory or high resolution dominate. State-space backbones [22] are an active alternative.
- **Segmentation and tracking** have converged on promptable models: SAM 2 for video objects and SAM 3 for open-vocabulary concepts [24].
- **Generation** has moved to latent diffusion with Transformer denoisers [28], and editing to instruction-conditioned diffusion [29].
- **Understanding** is increasingly done by multimodal LLMs in the LLaVA pattern [26].
- **Low-level vision** still relies heavily on task-specific encoder–decoders trained on synthetic degradations, now increasingly with generative (diffusion) priors; see DLCV 04 and DLCV 05.

Open problems. (1) *Data and evaluation*: benchmarks saturate and may overlap with web-scale pretraining data, so it is hard to tell generalization from memorization [34]. (2) *Real-world degradations*: models trained on synthetic noise or blur still fail on real camera pipelines. (3) *Fidelity versus hallucination*: generative priors produce sharp but sometimes invented detail, which is unacceptable in medical or forensic use. (4) *Efficiency*: attention is quadratic in tokens, and foundation models are hard to run on devices. (5) *Dense features at scale*: DINOv3's Gram anchoring [21] exists because dense feature quality can degrade as training scales. (6) *Bias, robustness and interpretability* of models trained on uncurated web data [34].

Key takeaways

- Deep learning reduces the three classical levels to two: **low-level** (image → image, judged by fidelity) and **high-level** (image → semantics, judged by accuracy). Mid-level analysis goes to whichever its output resembles.
- **End-to-end learning** replaced hand-designed features with learned ones. The split returned at scale as **pretrained backbone + light head**.
- A convolutional layer is a bank of **learned spatial filters** with local connectivity and weight sharing. Stride, padding and pooling set the output size; the **receptive field** grows as $r_\ell = r_{\ell-1} + (k_\ell - 1)j_{\ell-1}$, and the effective field is smaller than the theoretical one.
- LeNet-5 (1998) was not the first CNN; backprop-trained convolutional networks date to 1989. ResNet was trained at **18/34/50/101/152** layers on ImageNet; **1202** layers appears only on CIFAR-10. Residual connections make the identity easy and give gradients a direct path.
- Encoder–decoders with skip connections (**FCN, U-Net**) are the standard shape for dense prediction and restoration.
- **ViT** has little inductive bias, so it needs large-scale pretraining. Swin added locality back; ConvNeXt showed a modernized CNN matches it.

- **DINO** is teacher–student self-distillation; **MAE** is masked prediction. DINOv2/v3 provide frozen, general-purpose features.
- Tasks increasingly converge on promptable or language-connected **foundation models**: SAM 1/2/3 for segmentation, latent diffusion and DiT for generation, CLIP and LLaVA for vision–language.

Exercises

1. **Receptive field by hand.** A network has, in order: conv 7×7 stride 2; max-pool 3×3 stride 2; then four conv 3×3 stride 1. Compute the theoretical receptive field and the jump after each layer, and check your answer with the `receptive_field` function.

   Hint

   Start with $r = 1, j = 1$. After the first conv: $r = 7, j = 2$. After pooling: $r = 7 + 2 \cdot 2 = 11, j = 4$. Each following 3×3 layer adds $2 \cdot 4 = 8$, so $r = 19, 27, 35, 43$ with $j = 4$.

2. **Parameter budget.** For $C = 256$ input and output channels, compare the weights (ignore biases) of one 5×5 convolution, two stacked 3×3 convolutions, and a bottleneck (1×1 to 64 channels, 3×3 at 64, 1×1 back to 256). Which have the same receptive field?

   Hint

   $25C^2 \approx 1.64$M; $2 \cdot 9C^2 \approx 1.18$M; bottleneck $256 \cdot 64 + 9 \cdot 64^2 + 64 \cdot 256 \approx 0.07$M. The first two both see 5×5; the bottleneck sees 3×3 but is far cheaper, which is why ResNet-50 and deeper use it.

3. **Why the identity is hard without a shortcut.** Inside a network, a block's input usually comes from a previous ReLU, so it is non-negative. Show that for such inputs a residual block with $F \equiv 0$ outputs exactly $\mathbf{y} = \mathbf{x}$. Then explain what a plain block (conv–BN–ReLU–conv–BN–ReLU) would have to learn to output $\mathbf{y} = \mathbf{x}$. What does this imply for very deep plain networks?

   Hint

   For $\mathbf{x} \geq 0$, $\mathrm{ReLU}(\mathbf{x} + 0) = \mathbf{x}$. A plain block must instead learn weights that exactly undo two convolutions and two normalizations, a precise and fragile solution that gradient descent rarely finds. The residual block only needs $F = 0$, which weight decay and zero-initialization already push it toward. Deep plain networks therefore struggle to "do nothing" in extra layers, which matches the degradation the ResNet authors observed.

4. **ViT token count and cost.** For a 512×512 input, compute the number of tokens for patch sizes 32, 16 and 8, and the relative cost of one global attention layer ($\propto N^2$). How does Swin's windowed attention with 7×7 windows change the scaling?

   Hint

   $N = (512/P)^2$: 256, 1024, 4096 tokens; costs scale $1 : 16 : 256$. Windowed attention costs $N \cdot 49$ per layer (each token attends to 49 others), so it grows linearly with N.

5. **Learned versus hand-set filters.** Modify the digit CNN so that its first layer is frozen to four hand-set kernels (two Sobels, a Laplacian and a box blur) plus four random kernels. Train only the rest. Compare test accuracy with the fully learned network and inspect what happens to the random kernels if you unfreeze them.

   Hint

   Set `conv.weight.requires_grad_(False)` after assigning the weights, and pass only the remaining parameters to the optimizer. On this easy dataset both versions reach similar accuracy; the point is that hand-set derivative filters are a reasonable first layer, and that unfrozen random kernels tend to drift toward oriented, derivative-like patterns (compare Figure 4).

6. **Choose a recipe.** You have 2,000 labeled microscopy images and need (a) a per-image quality label, (b) per-pixel cell masks, (c) denoised images. For each, propose a backbone, a head and a training strategy using this tutorial's map, and say which would benefit most from a frozen self-supervised backbone.

   Hint

   (a) Frozen DINOv2/v3 or fine-tuned ConvNeXt + linear head. (b) U-Net, possibly with a pretrained encoder, or SAM prompted by points and then fine-tuned. (c) A full-resolution restoration network trained on synthetic noisy/clean pairs that match the microscope's noise. Frozen semantic features help (a) and (b) most; (c) depends on pixel-accurate statistics more than on semantics.

References

1. Shyandram, "深度學習的數位影像處理介紹 (An introduction to digital image processing with deep learning)," Medium post, 2024; updated 2026. link
2. R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018. publisher page
3. S. Theodoridis and K. Koutroumbas, *Pattern Recognition*, 4th ed., Academic Press, 2008. publisher page
4. Y. LeCun, B. Boser, J. S. Denker, D. Henderson, R. E. Howard, W. Hubbard, and L. D. Jackel, "Backpropagation applied to handwritten zip code recognition," *Neural Computation*, vol. 1, no. 4, pp. 541–551, 1989. doi:10.1162/neco.1989.1.4.541
5. Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. doi:10.1109/5.726791
6. O. Russakovsky, J. Deng, H. Su, et al., "ImageNet large scale visual recognition challenge," *International Journal of Computer Vision*, vol. 115, no. 3, pp. 211–252, 2015. doi:10.1007/s11263-015-0816-y ·arXiv:1409.0575
7. A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet classification with deep convolutional neural networks," *NeurIPS*, 2012. proceedings
8. K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *ICLR*, 2015. arXiv:1409.1556

9. C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going deeper with convolutions," *CVPR*, 2015. doi:10.1109/CVPR.2015.7298594 ·arXiv:1409.4842
10. K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," *CVPR*, 2016. doi:10.1109/CVPR.2016.90 ·arXiv:1512.03385
11. A. Krizhevsky, "Learning multiple layers of features from tiny images," technical report, University of Toronto, 2009 (the CIFAR-10/100 datasets). dataset page
12. W. Luo, Y. Li, R. Urtasun, and R. Zemel, "Understanding the effective receptive field in deep convolutional neural networks," *NeurIPS*, 2016. arXiv:1701.04128
13. J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," *CVPR*, 2015. arXiv:1411.4038
14. O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," *MICCAI*, 2015. arXiv:1505.04597
15. A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," *NeurIPS*, 2017. ML anthology
16. A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," *ICLR*, 2021. arXiv:2010.11929
17. Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, "Swin Transformer: Hierarchical vision Transformer using shifted windows," *ICCV*, 2021. doi:10.1109/ICCV48922.2021.00986 ·arXiv:2103.14030
18. Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, "A ConvNet for the 2020s," *CVPR*, 2022, arXiv:2201.03545; and S. Woo, S. Debnath, R. Hu, X. Chen, Z. Liu, I. S. Kweon, and S. Xie, "ConvNeXt V2: Co-designing and scaling ConvNets with masked autoencoders," *CVPR*, 2023, arXiv:2301.00808.
19. M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, and A. Joulin, "Emerging properties in self-supervised vision Transformers," *ICCV*, 2021. doi:10.1109/ICCV48922.2021.00951 ·arXiv:2104.14294
20. K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, "Masked autoencoders are scalable vision learners," *CVPR*, 2022. doi:10.1109/CVPR52688.2022.01553 ·arXiv:2111.06377
21. M. Oquab, T. Darcet, T. Moutakanni, et al., "DINOv2: Learning robust visual features without supervision," *Transactions on Machine Learning Research*, 2024, arXiv:2304.07193; and O. Siméoni, H. V. Vo, M. Seitzer, F. Baldassarre, M. Oquab, et al., "DINOv3," arXiv preprint, 2025, arXiv:2508.10104.
22. L. Zhu, B. Liao, Q. Zhang, X. Wang, W. Liu, and X. Wang, "Vision Mamba: Efficient visual representation learning with bidirectional state space model," *ICML*, 2024, arXiv:2401.09417; and Y. Liu, Y. Tian, Y. Zhao, H. Yu, L. Xie, Y. Wang, Q. Ye, J. Jiao, and Y. Liu, "VMamba: Visual state space model," *NeurIPS*, 2024, arXiv:2401.10166.
23. S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards real-time object detection with region proposal networks," *NeurIPS*, 2015, arXiv:1506.01497; and N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko, "End-to-end object detection with Transformers," *ECCV*, 2020, arXiv:2005.12872.
24. A. Kirillov, E. Mintun, N. Ravi, et al., "Segment anything," *ICCV*, 2023, arXiv:2304.02643; N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, et al., "SAM 2: Segment anything in images and videos,"

ICLR, 2025, arXiv:2408.00714; and N. Carion, L. Gustafson, Y.-T. Hu, S. Debnath, et al., "SAM 3: Segment anything with concepts," *ICLR*, 2026, arXiv:2511.16719.
25. A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, et al., "Learning transferable visual models from natural language supervision," *ICML*, 2021. arXiv:2103.00020
26. H. Liu, C. Li, Q. Wu, and Y. J. Lee, "Visual instruction tuning," *NeurIPS*, 2023. arXiv:2304.08485
27. J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," *NeurIPS*, 2020. arXiv:2006.11239
28. R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-resolution image synthesis with latent diffusion models," *CVPR*, 2022, arXiv:2112.10752; and W. Peebles and S. Xie, "Scalable diffusion models with Transformers," *ICCV*, 2023, arXiv:2212.09748.
29. T. Brooks, A. Holynski, and A. A. Efros, "InstructPix2Pix: Learning to follow image editing instructions," *CVPR*, 2023. arXiv:2211.09800
30. Z. Li, F. Liu, W. Yang, S. Peng, and J. Zhou, "A survey of convolutional neural networks: Analysis, applications, and prospects," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, 2022. doi:10.1109/TNNLS.2021.3084827 ·arXiv:2004.02806
31. S. Khan, M. Naseer, M. Hayat, S. W. Zamir, F. S. Khan, and M. Shah, "Transformers in vision: A survey," *ACM Computing Surveys*, vol. 54, no. 10s, 2022. doi:10.1145/3505244 ·arXiv:2101.01169
32. L. Jing and Y. Tian, "Self-supervised visual feature learning with deep neural networks: A survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, 2021. doi:10.1109/TPAMI.2020.2992393 ·arXiv:1902.06162
33. R. Balestriero, M. Ibrahim, V. Sobal, et al., "A cookbook of self-supervised learning," arXiv preprint, 2023. arXiv:2304.12210
34. M. Awais, M. Naseer, S. Khan, R. M. Anwer, H. Cholakkal, M. Shah, M.-H. Yang, and F. S. Khan, "Foundation models defining a new era in vision: A survey and outlook," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 47, 2025. doi:10.1109/TPAMI.2024.3506283 ·arXiv:2307.13721
35. J. Zhang, J. Huang, S. Jin, and S. Lu, "Vision-language models for vision tasks: A survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 46, 2024. doi:10.1109/TPAMI.2024.3369699 ·arXiv:2304.00685
36. scikit-learn developers, "sklearn.datasets.load_digits" (a copy of the test set of the UCI optical recognition of handwritten digits dataset). docs

DLCV 02 • Deep Image Classification

How deep networks decide what an image shows: losses and metrics, datasets, the CNN and transformer architectures and why they were designed that way, modern training recipes, pre-training, robustness, calibration and interpretability.

Online edition: https://blog.shyandram.dev/en/learn/deep-learning/dl-02-image-classification/

What you'll learn

- How image classification is posed precisely: single-label, multi-label and fine-grained tasks; logits, softmax and cross-entropy; top-1/top-5 accuracy; the confusion matrix and the metrics built on it (precision, recall, F1, MCC, macro and micro averages, ROC-AUC, average precision, mAP); and what it means for a classifier to be calibrated.
- What the standard training datasets (MNIST, CIFAR, ImageNet, iNaturalist) are for, which evaluation benchmarks test what (ImageNet-ReaL, ImageNetV2, robustness sets, fine-grained and transfer suites, long-tailed and zero-shot evaluation), and how to run an honest evaluation protocol.
- The design reasoning behind each major backbone, from AlexNet, VGG, Inception and ResNet to MobileNet, EfficientNet, RegNet, ConvNeXt, ViT, DeiT and Swin, with parameter and cost arithmetic you can check yourself.
- Which parts of a modern training recipe matter (augmentation, Mixup/CutMix, label smoothing, AdamW, warmup + cosine, stochastic depth, EMA) and why recipes alone moved ResNet-50 by several points.
- How pre-training changed the job: linear probes and fine-tuning, contrastive and masked self-supervision, DINOv2-style frozen features and CLIP zero-shot classification.
- How to evaluate honestly: distribution shift, adversarial examples, shortcut learning, temperature scaling, efficiency metrics and Grad-CAM.

The big picture

Image classification asks one question of a whole image: *which of these labels fits?* It is the oldest deep-learning benchmark task and still the place where most backbone networks are born. A detector, a segmenter or a restoration network usually starts from a backbone that was first trained, or at least evaluated, as an image classifier.

> **Explain like I'm five**
>
> Think of a child learning animals from a picture book. At first the child only sees blobs. With many pages, the child learns to spot ears, then faces, then "that's a cat". A deep classifier does the same in layers: early layers notice edges and colors, middle layers notice parts like eyes and wheels, and the

last layer votes for a label. Training is the parent saying "no, that's a dog" thousands of times. The rest of this tutorial is about building a better picture book (data), a better brain (architecture), a better way of teaching (training recipe), and better exams (evaluation), because a child who only ever saw cats on sofas may decide that "sofa" means "cat".

This tutorial assumes you know what a convolution, a CNN layer and backpropagation are. The classical side of classification (Bayes classifiers, perceptrons, a small CNN written in NumPy, softmax and cross-entropy derived by hand) is in IP 13. The broad history of deep learning for images is in DLCV 01. Here we go deep on classification itself. For a refresher on activation functions and optimizers, see ML 02 ·Neural Networks. A good single survey of the CNN side of this story is Li et al. [1].

The classification problem

Plain version. The network looks at an image and outputs one score per class. We turn scores into probabilities, compare them with the true label, and nudge the network so the right class gets more probability next time.

Single-label, multi-label and fine-grained

Let $\mathbf{x} \in \mathbb{R}^{H\times W\times 3}$ be an RGB image and $\mathcal{Y} = \{1, \dots, K\}$ a set of K classes. A network f_θ with parameters θ maps the image to a vector of **logits** $\mathbf{z} = f_\theta(\mathbf{x}) \in \mathbb{R}^K$, one unnormalized score per class.

- **Single-label (multi-class)** classification assumes exactly one correct class $y \in \mathcal{Y}$. The logits are turned into a probability vector with the softmax,

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad k = 1, \dots, K,$$

where p_k is the model's estimate of $P(y = k \mid \mathbf{x})$. Training minimizes the **cross-entropy**, the negative log-probability of the true class, averaged over a mini-batch:

$$\mathcal{L}_{\text{CE}} = -\frac{1}{B}\sum_{i=1}^{B} \log p_{y_i}(\mathbf{x}_i),$$

where B is the batch size and y_i the label of image $\mathbf{x}_i$. The gradient with respect to the logits is $\partial\mathcal{L}/\partial\mathbf{z} = \mathbf{p} - \mathbf{e}_y$ (per image), with $\mathbf{e}_y$ the one-hot vector of the true class; DIP Chapter 13 derives it.

- **Multi-label** classification allows any subset of labels (a street photo can contain "car", "person" and "bicycle"). Each class gets an independent sigmoid $\sigma(z_k) = 1/(1+e^{-z_k})$ and a binary cross-entropy, $-\sum_k [t_k \log \sigma(z_k) + (1 - t_k)\log(1 - \sigma(z_k))]$, where $t_k \in \{0, 1\}$ says whether class k is present. Metrics change too: mean average precision over classes replaces accuracy.

- **Fine-grained** classification separates visually similar subordinate classes, such as 200 bird species or 1,000 aircraft variants. The loss is the same as single-label, but the signal lives in small parts (a beak shape, a wing bar), classes are often long-tailed, and label noise from non-expert annotators is common. iNaturalist [6] is the standard large fine-grained benchmark.

Metrics: top-1, top-5 and beyond

Top-1 accuracy is the fraction of images whose highest-scoring class is the true one. **Top-k accuracy** counts an image as correct if the true class is among the k highest scores. The ImageNet challenge reported top-5 error because many images contain several objects, and some classes are nearly indistinguishable [2]. Top-5 hides real mistakes, so modern papers mostly report top-1. For imbalanced data, report **balanced accuracy** (the mean of per-class recalls) or per-class results, because a classifier can score high accuracy by ignoring rare classes.

```
import torch
import torch.nn.functional as F

logits = torch.tensor([[2.0, 1.0, 0.1, -1.0],    # 2 images, 4 classes
                       [0.5, 2.5, 2.4, 0.0]])
y = torch.tensor([0, 2])

p = logits.softmax(dim=1)
ce = F.cross_entropy(logits, y)                       # mean of -log p[i, y_i]
ce_ls = F.cross_entropy(logits, y, label_smoothing=0.1)
top1 = (logits.argmax(1) == y).float().mean()
top2 = (logits.topk(2, dim=1).indices == y[:, None]).any(1).float().mean()
print(p.round(decimals=3))
print(f"CE {ce:.3f}  CE+LS {ce_ls:.3f}  top-1 {top1:.2f}  top-2 {top2:.2f}")

# multi-label: one independent sigmoid per class, binary cross-entropy
targets = torch.tensor([[1., 0., 1., 0.], [0., 1., 1., 0.]])
print(f"BCE {F.binary_cross_entropy_with_logits(logits, targets):.3f}")
```

The second image is a near-tie between classes 1 and 2. Top-1 counts it as wrong; top-2 counts it as right. That one example is the whole argument for and against top-5.

The confusion matrix and the metrics built on it

Plain version. Accuracy collapses all mistakes into one number. A confusion matrix keeps them apart: it counts, for every true class, what the model said. Almost every other classification metric is a ratio of cells in that table, and each ratio answers a different question.

The binary case: TP, FP, FN, TN

Take one class as **positive** (say "sick") and the rest as **negative**. Each prediction lands in one of four cells:

	Predicted positive	Predicted negative
Actually positive	true positive (TP)	false negative (FN)
Actually negative	false positive (FP)	true negative (TN)

Explain like I'm five

A smoke alarm is a classifier. When there is a fire and it rings, that is a true positive. When you burn toast and it rings, that is a false positive: annoying, but nobody is hurt. When there is a fire and it stays quiet, that is a false negative: the dangerous one. When nothing happens and it stays quiet, that is a true negative. A good alarm is not just "right most of the time" (it is quiet almost every day, so that is easy). It must ring for fires and stay quiet for toast. Different metrics weigh those two goals differently.

With $N = \mathrm{TP} + \mathrm{FP} + \mathrm{FN} + \mathrm{TN}$ test examples, the standard ratios are

$$\text{accuracy} = \frac{\mathrm{TP}+\mathrm{TN}}{N}, \quad \text{precision} = \frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FP}}, \quad \text{recall (TPR)} = \frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FN}},$$

$$\text{specificity (TNR)} = \frac{\mathrm{TN}}{\mathrm{TN}+\mathrm{FP}}, \quad \mathrm{FPR} = \frac{\mathrm{FP}}{\mathrm{FP}+\mathrm{TN}} = 1 - \mathrm{TNR}.$$

- **Precision** answers "when the model says positive, how often is it right?" It matters when a false alarm is costly: a spam filter that hides real mail, an automatic product recall.
- **Recall**, also called **sensitivity** or the **true positive rate (TPR)**, answers "of all real positives, how many did we catch?" It matters when a miss is costly: cancer screening, a defect detector on a production line.
- **Specificity**, the **true negative rate (TNR)**, is recall for the negative class. The **false positive rate (FPR)** is its complement, the fraction of negatives wrongly flagged.

Precision and recall pull against each other: lowering the decision threshold flags more items, which raises recall and usually lowers precision. Single numbers that combine them are:

$$F_1 = \frac{2\,\mathrm{P}\,\mathrm{R}}{\mathrm{P}+\mathrm{R}} = \frac{2\,\mathrm{TP}}{2\,\mathrm{TP}+\mathrm{FP}+\mathrm{FN}}, \qquad F_\beta = \frac{(1+\beta^2)\,\mathrm{P}\,\mathrm{R}}{\beta^2\,\mathrm{P}+\mathrm{R}},$$

where P is precision, R recall and $\beta > 0$ says how many times more recall matters than precision (F_2 favours recall, $F_{0.5}$ favours precision). F_1 is the harmonic mean, so it is high only when *both* parts are high. Note what F_1 ignores: true negatives never appear in it.

Two further summaries use all four cells:

$$\text{balanced accuracy} = \frac{\mathrm{TPR}+\mathrm{TNR}}{2}, \qquad \mathrm{MCC} = \frac{\mathrm{TP}\cdot\mathrm{TN} - \mathrm{FP}\cdot\mathrm{FN}}{\sqrt{(\mathrm{TP}+\mathrm{FP})(\mathrm{TP}+\mathrm{FN})(\mathrm{TN}+\mathrm{FP})(\mathrm{TN}+\mathrm{FN})}}.$$

Balanced accuracy is the mean of the per-class recalls, so a classifier that ignores one class scores 0.5 however rare that class is. The **Matthews correlation coefficient** (MCC) is the Pearson correlation between the true and predicted binary labels: $+1$ is perfect, 0 is no better than chance, -1 is perfectly inverted. Chicco and Jurman [57] argue that it should be preferred over accuracy and F_1 for binary evaluation because it is high only when the classifier does well in all four cells. Sokolova and Lapalme [58] give a systematic comparison of these measures and of how each one reacts to changes in the confusion matrix.

Why accuracy misleads at 99:1. Suppose 10 of 1,000 scans are sick. A "model" that always answers *healthy* scores 99.0% accuracy. Model B flags 30 scans and catches 8 of the 10 sick ones; it scores *lower* accuracy (97.6%) yet is the only useful one:

```
import numpy as np
from sklearn.metrics import (accuracy_score, balanced_accuracy_score, confusion_matrix,
                             f1_score, matthews_corrcoef, precision_score, recall_score)

# 1,000 scans, 10 sick (label 1). Model A says "healthy" to everyone.
# Model B flags 30 scans and catches 8 of the 10 sick ones.
y = np.r_[np.ones(10), np.zeros(990)].astype(int)
pred_a = np.zeros(1000, dtype=int)
pred_b = np.r_[np.ones(8), np.zeros(2), np.ones(22), np.zeros(968)].astype(int)

for name, p in [("all-negative", pred_a), ("model B", pred_b)]:
    tn, fp, fn, tp = confusion_matrix(y, p).ravel()
    print(f"{name:12s} TP={tp} FP={fp} FN={fn} TN={tn}  "
          f"acc={accuracy_score(y, p):.3f} bal-acc={balanced_accuracy_score(y, p):.3f} "
          f"P={precision_score(y, p, zero_division=0):.3f} R={recall_score(y, p):.3f} "
          f"F1={f1_score(y, p, zero_division=0):.3f} MCC={matthews_corrcoef(y, p):.3f}")
# all-negative TP=0 FP=0 FN=10 TN=990  acc=0.990 bal-acc=0.500 P=0.000 R=0.000 F1=0.000
 ↪  MCC=0.000
# model B      TP=8 FP=22 FN=2 TN=968  acc=0.976 bal-acc=0.889 P=0.267 R=0.800 F1=0.400
 ↪  MCC=0.454
```

Balanced accuracy, F_1 and MCC all rank model B first; accuracy ranks it last. Rules of thumb: use accuracy only when classes are roughly balanced and all errors cost the same; report precision and recall (or F_β with a β chosen from the real costs) when one kind of error matters more; use balanced accuracy or MCC as a single imbalance-aware summary.

The multi-class confusion matrix

With K classes the confusion matrix $C \in \mathbb{N}^{K\times K}$ has entry C_{ij} = number of test images whose **true** class is i (row) and whose **predicted** class is j (column); this is the convention of scikit-learn [62]. The diagonal holds the correct predictions. Dividing each row by its sum gives the **row-normalized** matrix, whose diagonal is the per-class recall and whose off-diagonal entries show *which* classes are confused. Figure 1 shows one for a logistic regression trained on only 200 of scikit-learn's 8×8 digits: most mistakes are $3\to 8$, $4\to 1$ and $7\to 9$, pairs that share strokes.

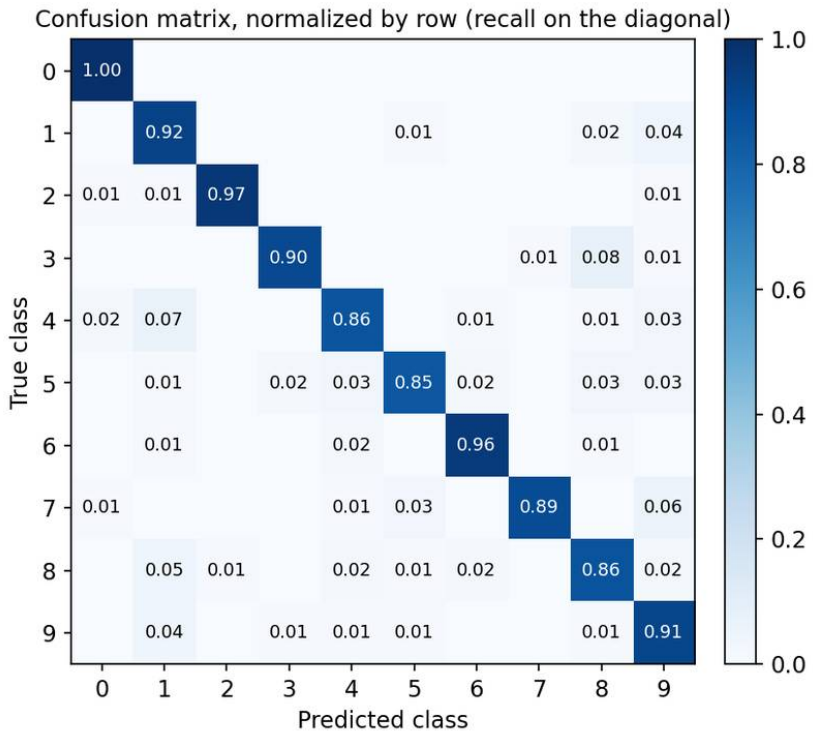


Figure 1 — Row-normalized confusion matrix of a logistic regression trained on 200 digits and tested on 899 (accuracy 91.1%). Each row sums to 1; the diagonal is the recall of each class, and the bright off-diagonal cells name the confusions worth fixing.

Treating each class k in turn as "positive" and all others as "negative" gives per-class counts: $\mathrm{TP}_k = C_{kk}$, $\mathrm{FP}_k = \sum_{i \neq k} C_{ik}$ (the rest of column k), $\mathrm{FN}_k = \sum_{j \neq k} C_{kj}$ (the rest of row k). Hence per-class precision $\mathrm{P}_k = C_{kk} / \sum_i C_{ik}$ and recall $\mathrm{R}_k = C_{kk} / \sum_j C_{kj}$. There are three common ways to turn K per-class numbers into one:

- **Macro average**: the plain mean over classes, $\mathrm{F}_1^{\text{macro}} = \frac{1}{K} \sum_k \mathrm{F}_{1,k}$. Every class counts equally, so rare classes weigh as much as common ones.
- **Micro average**: sum TP_k, FP_k, FN_k over classes first, then compute the metric once. In single-label multi-class problems every error is one FP (for the predicted class) and one FN (for the true class), so micro precision = micro recall = micro F_1 = accuracy.
- **Weighted average**: the mean of per-class values weighted by each class's **support** n_k (number of true examples). It sits close to micro and hides rare-class failures almost as well.

A worked example. 100 test images from three classes with supports 80 (A), 15 (B) and 5 (C):

true \ predicted	A	B	C
A (80)	76	3	1
B (15)	4	10	1
C (5)	3	0	2

Column sums are 83, 13 and 4, so precision is $76/83 = 0.916$, $10/13 = 0.769$ and $2/4 = 0.500$; recall is $76/80 = 0.950$, $10/15 = 0.667$ and $2/5 = 0.400$; per-class F_1 is 0.933, 0.714 and 0.444. Then

$$\mathrm{F}_1^{\text{macro}} = \tfrac{1}{3}(0.933 + 0.714 + 0.444) = 0.697, \qquad \mathrm{F}_1^{\text{micro}} = \text{accuracy} = \tfrac{76+10+2}{100} = 0.880,$$

and the support-weighted F_1 is $0.80 \cdot 0.933 + 0.15 \cdot 0.714 + 0.05 \cdot 0.444 = 0.875$. The 18-point gap between micro and macro is the signal: the model is good on the common class and poor on the rare ones. Report macro when every class matters (rare diseases, rare species); report micro/accuracy when every *image* matters equally.

Cohen's kappa [59] corrects observed agreement for agreement expected by chance:

$$\kappa = \frac{p_o - p_e}{1 - p_e}, \qquad p_o = \frac{1}{N}\sum_k C_{kk}, \qquad p_e = \frac{1}{N^2}\sum_k \Big(\sum_j C_{kj}\Big)\Big(\sum_i C_{ik}\Big),$$

where p_o is the accuracy and p_e the accuracy a classifier would get by predicting each class at its observed rate independently of the input. It was designed for agreement between two human raters and is still common when comparing a model with annotators.

```
from sklearn.datasets import load_digits
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (average_precision_score, classification_report,
 ↪  cohen_kappa_score,
                             confusion_matrix, precision_recall_fscore_support,
                              ↪  roc_auc_score)
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import label_binarize

X, y = load_digits(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.5, stratify=y, random_state=0)
clf = LogisticRegression(max_iter=2000).fit(X_tr[:200], y_tr[:200])   # only 200 labels
pred, prob = clf.predict(X_te), clf.predict_proba(X_te)

cm = confusion_matrix(y_te, pred)                     # rows = true, columns = predicted
cm_norm = confusion_matrix(y_te, pred, normalize="true")   # each row sums to 1 (Figure 1)
print(classification_report(y_te, pred, digits=3))    # per-class P, R, F1, support +
 ↪  averages
for avg in ["macro", "micro", "weighted"]:
    p, r, f, _ = precision_recall_fscore_support(y_te, pred, average=avg)
    print(f"{avg:8s} P={p:.3f} R={r:.3f} F1={f:.3f}")
print(f"kappa {cohen_kappa_score(y_te, pred):.3f}")
print(f"ROC-AUC (one-vs-rest, macro) {roc_auc_score(y_te, prob, multi_class='ovr'):.4f}")
Y = label_binarize(y_te, classes=range(10))           # one binary column per class
print(f"mean AP over classes {average_precision_score(Y, prob, average='macro'):.4f}")
# macro    P=0.914 R=0.911 F1=0.912
# micro    P=0.911 R=0.911 F1=0.911
# weighted P=0.915 R=0.911 F1=0.912
# kappa 0.901
# ROC-AUC (one-vs-rest, macro) 0.9958
# mean AP over classes 0.9742
```

Here macro and micro agree because the digit classes are balanced; on the A/B/C example above they would not.

Threshold-free evaluation: ROC and precision–recall curves

Every metric above needs a decision rule (argmax, or "positive if the score exceeds t"). A scoring classifier can be evaluated at *all* thresholds at once. Sweep t from high to low and record a point per threshold:

- The **ROC curve** plots TPR against FPR. Its area, **ROC-AUC**, equals the probability that a randomly chosen positive gets a higher score than a randomly chosen negative; 0.5 is chance, 1.0 is perfect separation. Fawcett [54] is the standard introduction.
- The **precision–recall (PR) curve** plots precision against recall. Its summary, **average precision** (AP), is the precision averaged over the recall levels at which a new positive is retrieved,

$$\mathrm{AP} = \sum_{n} (\mathrm{R}_n - \mathrm{R}_{n-1})\, \mathrm{P}_n,$$

where P_n and R_n are precision and recall at the n-th threshold, ordered by decreasing score (this is scikit-learn's definition [62]; benchmarks such as PASCAL VOC [61] have used interpolated variants, so check which one a paper reports).

Why PR under heavy imbalance. FPR divides by the number of negatives. When negatives outnumber positives 99 to 1, even thousands of false alarms barely move FPR, so the ROC curve looks excellent while precision is poor. Precision divides by the number of *flagged* items, so false alarms hit it directly, and the random baseline of AP equals the positive rate rather than 0.5. Davis and Goadrich [55] proved that a curve dominates in ROC space if and only if it dominates in PR space, and showed that optimizing ROC-AUC does not optimize AP; Saito and Rehmsmeier [56] showed with simulations and a literature review that PR plots are more informative than ROC plots on imbalanced data. Figure 2 makes this concrete: two classifiers with the same ROC-AUC (0.95) on a task with 1% positives differ clearly in AP (0.45 against 0.36).

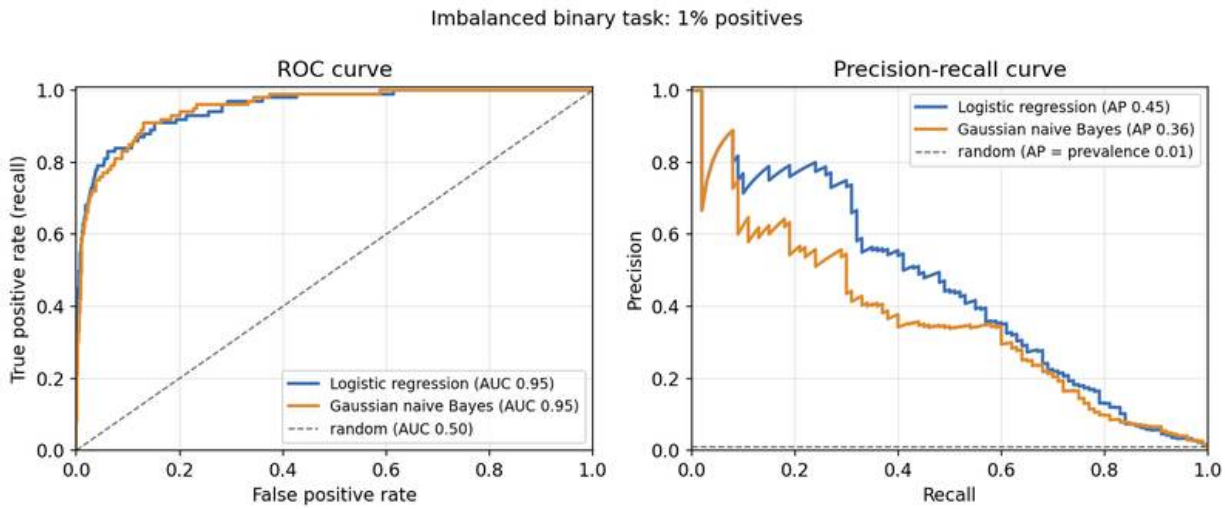


Figure 2 — ROC (left) and precision–recall (right) curves for two classifiers on a synthetic task with 1% positives (100 of 10,000 test points). Both have ROC-AUC 0.95, yet logistic regression keeps much higher precision at every recall level (AP 0.45 against 0.36). An all-negative classifier would score 99% accuracy here.

For $K > 2$ classes, both curves are computed **one-vs-rest**: class k is positive, all others negative, using the score p_k. The K areas are then macro-averaged (`roc_auc_score(..., multi_class="ovr")`), or micro-averaged by pooling all (k, image) pairs.

Multi-label metrics

In multi-label classification each image has a binary target vector $\mathbf{t} \in \{0,1\}^K$ and the model a score vector. Each label is its own binary problem, so the natural ranking metric is the **per-class AP** and its mean over classes, **mAP**, which became standard with the PASCAL VOC classification task [61]. Thresholded metrics come in two families, reviewed by Zhang and Zhou [60]:

- **Label-based** metrics compute a binary metric per label and macro- or micro-average over labels, exactly as in the multi-class case.
- **Example-based** metrics compute a score per *image* and average over images. Example-based F_1 is $\frac{1}{N}\sum_i \frac{2|\hat{Y}_i \cap Y_i|}{|\hat{Y}_i| + |Y_i|}$, where Y_i and $\hat{Y}_i$ are the true and predicted label sets of image i.
- **Hamming loss** is the fraction of wrong (image, label) bits, $\frac{1}{NK}\sum_i |\hat{Y}_i \,\triangle\, Y_i|$, where $\triangle$ is the symmetric difference. It is easy to read but, with many rare labels, an empty prediction already looks good.

```
import numpy as np
from sklearn.metrics import average_precision_score, f1_score, hamming_loss

# 4 street photos, 3 labels: car, person, bicycle
T = np.array([[1, 1, 0], [1, 0, 0], [0, 1, 1], [1, 1, 1]])           # ground truth
```

```
S = np.array([[.9, .6, .2], [.8, .4, .3], [.3, .7, .4], [.6, .2, .7]])  # sigmoid scores
P = (S >= 0.5).astype(int)                                               # threshold 0.5

print("per-class AP:", average_precision_score(T, S, average=None).round(3))
print(f"mAP {average_precision_score(T, S, average='macro'):.3f}")
print(f"example-based F1 {f1_score(T, P, average='samples'):.3f}  "
      f"macro F1 {f1_score(T, P, average='macro'):.3f}  micro F1 {f1_score(T, P,
       ↪  average='micro'):.3f}")
print(f"Hamming loss {hamming_loss(T, P):.3f}")
# per-class AP: [1.    0.917 1.   ]
# mAP 0.972
# example-based F1 0.867  macro F1 0.822  micro F1 0.857
# Hamming loss 0.167
```

Calibration: are the probabilities honest?

Accuracy only checks the argmax. **Calibration** checks whether the confidence means what it says: among all predictions made with confidence 0.8, about 80% should be right. Formally, a classifier is calibrated if $P(\hat{y} = y \mid \hat{p} = c) = c$ for every confidence c, where $\hat{y}$ is the predicted class and $\hat{p} = \max_k p_k$ its probability. The usual summary is the **expected calibration error**: split predictions into M confidence bins B_m and compute

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{n} \left| \mathrm{acc}(B_m) - \mathrm{conf}(B_m) \right|,$$

where n is the number of test images, $\mathrm{acc}(B_m)$ the accuracy inside bin m and $\mathrm{conf}(B_m)$ the mean confidence there. Guo et al. [47] found that modern deep networks, trained long with cross-entropy, tend to be **overconfident**, and that a single scalar fix works remarkably well. We return to it in the robustness section.

Datasets and why they matter

Plain version. A benchmark is a frozen exam. It lets everyone compare methods fairly, but once a whole field studies for the same exam, scores start to measure the exam as much as the skill. It helps to keep two roles apart: datasets used to *train* a model and datasets used only to *evaluate* it.

Training datasets

Dataset	Images	Classes	Resolution	Role
MNIST [3]	60k train / 10k test	10 digits	28×28 gray	sanity check, teaching
CIFAR-10 / CIFAR-100 [4]	50k train / 10k test	10 / 100 (20 superclasses)	32×32 RGB	small-scale architecture and recipe studies

Dataset	Images	Classes	Resolution	Role
ImageNet-1k (ILSVRC 2012) [2]	about 1.28M train / 50k val	1,000	variable, usually trained at 224×224	the standard backbone benchmark
ImageNet-21k [5]	roughly 13–14M (versions differ)	about 21k	variable	larger-scale supervised pre-training
iNaturalist [6]	hundreds of thousands, by year	thousands of species	variable	fine-grained, long-tailed

MNIST is solved and now only catches bugs. CIFAR is small enough to run many controlled experiments, which is why regularization papers love it. ImageNet-1k became the reference after the 2012 challenge [2], [10]: architectures are usually reported as "top-1 on ImageNet-1k val at 224×224", and almost every pre-trained backbone in a model zoo has an ImageNet number attached. ImageNet-21k is the larger hierarchy from which ImageNet-1k was drawn; Ridnik et al. [5] made pre-training on it practical with a cleaned, semantic-hierarchy-aware recipe.

Evaluation benchmarks

Plain version. Training data teaches; evaluation data examines. A model that only ever sat one exam may have learned that exam. So modern papers grade a backbone on a *battery* of test sets, each built to probe one thing: the original distribution, a fresh sample of it, corrupted or stylized images, unusual viewpoints, fine-grained categories, small labelled budgets or rare classes.

The table lists the evaluation sets you will meet most often. Sizes are those stated by the papers or official pages cited.

Benchmark	What it measures	Size / classes	Usually reported
ImageNet-1k val [2]	in-distribution accuracy; the de facto test set, since ILSVRC test labels stay on the evaluation server	50,000 images, 1,000 classes	top-1 (top-5)
ImageNet-ReaL [8]	the same validation images with re-annotated, multi-label "reassessed" labels	46,837 images with 57,553 labels (3,163 images left without a label are discarded)	ReaL accuracy: top-1 counted correct if it is in the label set
ImageNetV2 [7]	generalization to a new sample drawn by repeating the original collection process	10,000 images per variant (10 per class); three sampling variants, MatchedFrequency is the usual one	top-1
ImageNet-C [42]	robustness to synthetic corruptions	validation images × 15 corruptions × 5 severities	mean corruption error
ImageNet-R [43]	renditions: art, cartoons, sculpture, toys and more	30,000 images, 200 classes	top-1 over the 200 classes
ImageNet-A [44]	natural photos filtered to fool a ResNet-50	7,500 images, 200 classes	top-1 over the 200 classes
ImageNet-Sketch [63]	domain shift to black-and-white sketches	50,000 images, 1,000 classes (50 per class)	top-1

Benchmark	What it measures	Size / classes	Usually reported
ObjectNet [64]	bias-controlled photos with varied rotation, background and viewpoint; test set only	50,000 images, 313 classes (113 overlap ImageNet)	top-1 on the overlapping classes
ImageNet-LT [73]	long-tailed training, balanced testing	115.8k training images, 1,000 classes, 5 to 1,280 per class; balanced test set	top-1, often split by class frequency
iNaturalist [6]	naturally long-tailed, fine-grained species	see the training table	top-1

Fine-grained and transfer benchmarks. A backbone is also judged by how well its features transfer to small, specialized datasets, either with a linear probe on frozen features or with fine-tuning (see the transfer-learning section below).

Dataset	Domain	Size / classes	Usually reported
CUB-200-2011 [65]	bird species, with part and attribute annotations	11,788 images, 200 classes	top-1
Stanford Cars [66]	car make, model and year	196 classes; 8,144 train / 8,041 test	top-1
FGVC-Aircraft [67]	aircraft variants in a three-level hierarchy	10,000 images, 100 variants	mean per-class accuracy
Oxford-IIIT Pets [68]	cat and dog breeds	37 classes, about 200 images each	mean per-class accuracy
Oxford Flowers-102 [69]	flower species	102 classes, 40 to 258 images each; 2,040 train / 6,149 test	mean per-class accuracy
Food-101 [70]	dishes; training labels deliberately left noisy	101,000 images, 101 classes (750 train / 250 test per class)	top-1
Caltech-101 [71]	generic objects	101 categories plus background, about 40 to 800 images each	mean per-class accuracy

Train/test sizes for Cars and Flowers follow the splits listed by Kornblith et al. [33]. Note that several of these benchmarks report **mean per-class accuracy** (macro-averaged recall) instead of plain accuracy, because their test sets are not balanced; mixing the two in one comparison is a common error.

Two **suites** bundle such datasets so that one number summarizes transfer:

- Kornblith et al. [33] used **12 datasets**: Food-101, CIFAR-10, CIFAR-100, Birdsnap, SUN397, Stanford Cars, FGVC-Aircraft, PASCAL VOC 2007 classification, Describable Textures, Oxford-IIIT Pets, Caltech-101 and Oxford Flowers-102. They report top-1 accuracy, mean per-class accuracy, or 11-point mAP (VOC 2007) depending on the dataset, under linear probing, fine-tuning and training from scratch.
- **VTAB** [72] contains 19 tasks in three groups: 7 *natural* (standard camera images), 4 *specialized* (satellite and medical imagery) and 8 *structured* (scene geometry, such as counting or distance,

mostly from simulated environments). In **VTAB-1k** each task provides only 1,000 labelled training examples (800 for training and 200 for validation), so it tests how well a representation adapts from few labels to tasks unlike its pre-training data.

Zero-shot evaluation. Image–text models such as CLIP [40] are evaluated without any training on the target dataset: the class names are turned into text prompts, and the image is assigned to the most similar prompt (see the CLIP section below). The CLIP paper benchmarked over 30 existing datasets this way, reported that zero-shot CLIP beat a linear probe on ResNet-50 features on 16 of 27 datasets, and found that an ensemble of 80 prompt templates improved ImageNet accuracy by 3.5 points over a single default prompt. Two cautions follow. Prompts and class-name wording are hyperparameters, so tuning them on the test set leaks test information exactly as tuning a learning rate on it would. And because the pre-training data are web-scale and mostly unpublished, "zero-shot" does not guarantee that the test images, or near-duplicates, were never seen.

How reliable are the numbers?

Three lines of work changed how we read ImageNet accuracy.

- **Reproduction.** Recht et al. [7] rebuilt test sets for CIFAR-10 and ImageNet by repeating the original collection procedure as closely as possible (ImageNetV2). Every model lost accuracy: 3–15% on CIFAR-10 and 11–14% on ImageNet. But the ranking was largely preserved, and gains on the old test set translated into gains on the new one. Their reading is that the drop comes from a subtle shift in difficulty between the old and new samples, not from the community overfitting to the test set by reusing it.
- **Labels.** Beyer et al. [8] re-annotated the ImageNet validation set with multiple labels per image and found that recent "progress" partly reflects fitting the idiosyncrasies of the original single-label annotations. Northcutt et al. [9] estimated that label errors are pervasive across common test sets and showed that they can change which model looks best.
- **Bias.** Every dataset has a "signature" in viewpoint, background and photographer style. DIP Chapter 13 reviews the dataset-bias studies; the practical lesson is to test on independently collected data whenever possible.

Read a benchmark number as a claim about a benchmark

"81.0% top-1" means: on these 50,000 particular validation images, with this preprocessing and this label set. Report the resolution, the number of crops, whether extra data or distillation was used, and the variance across seeds when differences are under a point.

Evaluation protocol

A benchmark number is only as honest as the protocol behind it.

- **Three splits, three jobs.** Fit parameters on the **training** set. Choose hyperparameters, early stopping, temperature, thresholds and prompts on a **validation** set (or by cross-validation inside the training data). Touch the **test** set once, at the end. When a benchmark only publishes a validation split, as with ImageNet-1k, hold out part of the training set for tuning.
- **Never tune on the test set.** Every decision made after looking at test results (an extra augmentation, another epoch, a different checkpoint) fits the test set a little. Repeated over a field's worth of papers, this is **test-set reuse**, or adaptive overfitting to a benchmark. ImageNetV2 [7] found that ranking was largely preserved on a fresh sample, which is reassuring, but the same study also shows that absolute numbers do not transfer and that a fresh test set is the only direct check.
- **Report variation.** Train with several random seeds (at least three, more for small datasets) and report mean $\pm$ standard deviation, or a confidence interval. With 10,000 test images, ImageNetV2 [7] quotes 95% intervals of about $\pm 1\%$; a 0.3-point gain on such a set is not a result.
- **Report the whole recipe.** Resolution, number of crops, extra data, distillation, which variant of a metric (top-1 or mean per-class accuracy, which AP definition) and which version of a label set (original or ReaL).

CNN architectures and the reasons behind them

Plain version. Each famous CNN fixed one specific problem of its predecessors: training too slow, too many parameters, too hard to optimize when deep, too expensive on a phone. Knowing the problem each one solved is more useful than memorizing their layer tables.

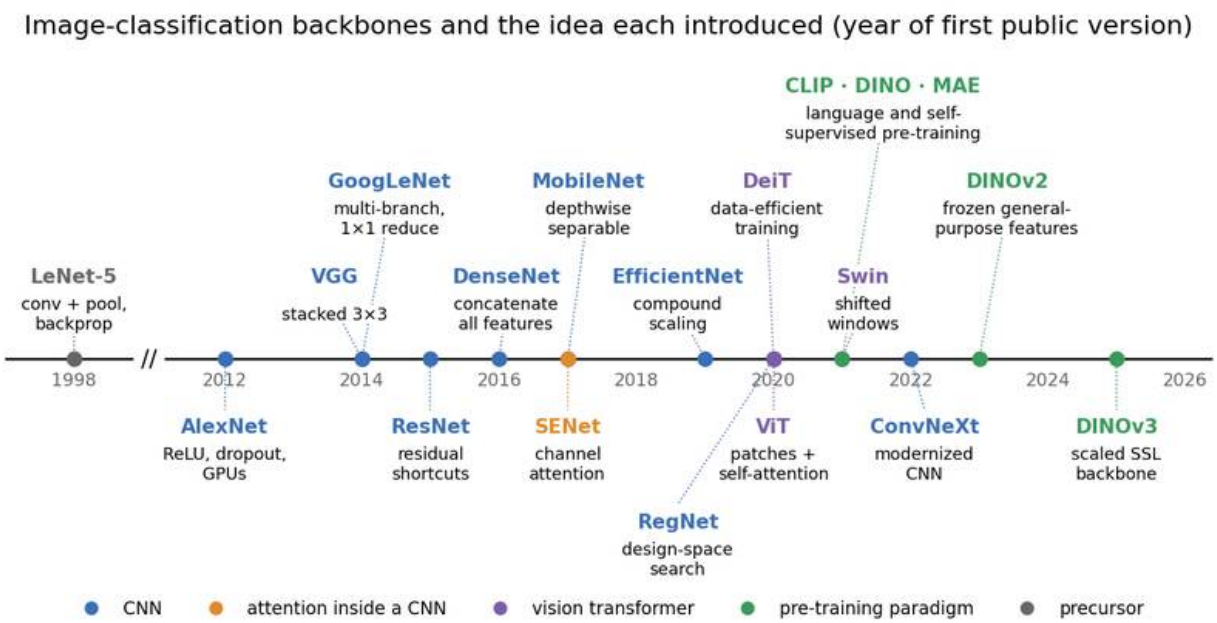


Figure 3 — Image-classification backbones and the main idea each introduced. Dates are the year of the first public version (usually arXiv). Colors separate pure CNNs, attention inside CNNs, vision transformers

and pre-training paradigms.

AlexNet: depth becomes trainable

Before 2012, the best ImageNet systems used hand-crafted features (SIFT, Fisher vectors) with linear classifiers. AlexNet [10] was a five-convolution, three-fully-connected network trained end to end on ImageNet. Three ingredients made it work at that scale. **ReLU** activations, $\max(0, z)$, do not saturate for positive inputs, so gradients do not vanish the way they do with tanh, and the paper reports several times faster training. **Dropout** in the large fully connected layers fought overfitting; those layers held most of the parameters. **GPUs** made it feasible: the model was split across two GPUs. Heavy data augmentation (random crops and flips, color perturbation) was the fourth, often forgotten, ingredient. The design lesson: with enough data and compute, learned features beat engineered ones.

VGG: small kernels, stacked

VGG [11] asked a cleaner question: what happens if we only use 3×3 convolutions and go deeper (16–19 weight layers)? Two stacked 3×3 layers see a 5×5 region; three see 7×7. For C input and output channels,

$$\underbrace{2 \cdot 3^2 C^2}_{\text{two } 3\times 3} = 18C^2 \;<\; \underbrace{5^2 C^2}_{\text{one } 5\times 5} = 25C^2,$$

ignoring biases. So the stack has fewer parameters for the same receptive field, and it inserts an extra nonlinearity between the two layers. VGG's uniform design made it a favourite feature extractor for years. Its weakness is cost: the fully connected head and the wide early layers make it heavy.

GoogLeNet / Inception: width, branches and 1×1 bottlenecks

GoogLeNet [12] attacked cost instead. Each **Inception module** runs 1×1, 3×3, 5×5 convolutions and pooling in parallel and concatenates the outputs, so the next layer can choose the right scale. Running big kernels on many channels would be expensive, so 1×1 **convolutions** first reduce the channel count. A 1×1 convolution is a per-pixel linear map across channels: it mixes channels without looking at neighbours, and it is the cheapest way to change width. GoogLeNet also replaced the large fully connected head with **global average pooling**. The follow-up Inception-v3 paper [13] factorized large kernels into stacks or into $1 \times n$ and $n \times 1$ pairs, and introduced **label smoothing**, which we meet again in the training section.

ResNet: learning residuals

The problem. Stacking more plain layers eventually made networks *worse*, even on the training set. This "degradation" is an optimization problem, not overfitting: a deeper network could in

principle copy a shallower one and set the extra layers to identity, yet SGD did not find that solution.

The idea. He et al. [14] let a block learn a residual $\mathcal{F}$ and add its input back:

$$\mathbf{y} = \mathcal{F}(\mathbf{x}; \{W_i\}) + \mathbf{x},$$

where $\mathbf{x}$ is the block input, $\mathcal{F}$ is two or three convolution layers with weights W_i, and $\mathbf{y}$ the output. If the best mapping is close to identity, the block only has to push $\mathcal{F}$ toward zero, which is easy. When the shapes differ (more channels, stride 2), the shortcut becomes a 1×1 projection.

Why it trains. Unroll L blocks of the pre-activation form $\mathbf{x}_{l+1} = \mathbf{x}_l + \mathcal{F}(\mathbf{x}_l)$, as analysed in the follow-up paper on identity mappings [15]:

$$\mathbf{x}_L = \mathbf{x}_l + \sum_{i=l}^{L-1} \mathcal{F}(\mathbf{x}_i), \qquad \frac{\partial \mathcal{L}}{\partial \mathbf{x}_l} = \frac{\partial \mathcal{L}}{\partial \mathbf{x}_L}\Big(\mathbf{I} + \frac{\partial}{\partial \mathbf{x}_l} \sum_{i=l}^{L-1} \mathcal{F}(\mathbf{x}_i)\Big),$$

where $\mathcal{L}$ is the loss and $\mathbf{I}$ the identity matrix. The identity term carries the gradient from the top straight down to any block, so it cannot vanish through a product of many small Jacobians. That paper also found that keeping the shortcut a *pure* identity (moving BN and ReLU inside the residual branch, "pre-activation") trains even deeper networks more easily.

The bottleneck. For deeper ResNets (50 layers and up) each block becomes 1×1 (reduce) $\to 3\times3$ $\to 1\times1$ (expand), shown in Figure 4(c). At 256 channels, a basic block of two 3×3 convolutions has $2\cdot9\cdot256^2 \approx 1.18$M weights; a bottleneck that reduces to 64 channels has $256\cdot64 + 9\cdot64^2 + 64\cdot256 \approx 70$k. The expensive 3×3 runs on a thin tensor, which is what makes 50–152 layers affordable.

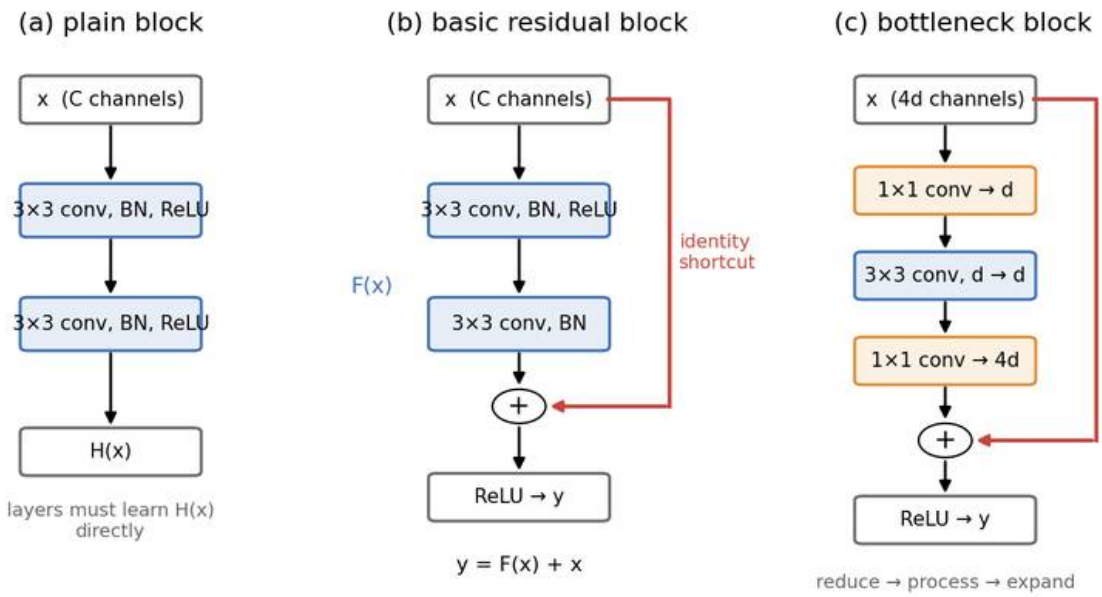


Figure 4 — (a) A plain block must learn the full mapping $H(x)$. (b) A residual block learns $F(x) = H(x) - x$ and adds the input back through an identity shortcut. (c) The bottleneck block runs the 3×3 convolution on a reduced width d and expands back to $4d$.

The basic block used in this tutorial's experiments, written in PyTorch:

```
import torch.nn as nn
import torch.nn.functional as F

class BasicBlock(nn.Module):
    def __init__(self, cin, cout, stride=1):
        super().__init__()
        self.conv1 = nn.Conv2d(cin, cout, 3, stride, 1, bias=False)
        self.bn1 = nn.BatchNorm2d(cout)
        self.conv2 = nn.Conv2d(cout, cout, 3, 1, 1, bias=False)
        self.bn2 = nn.BatchNorm2d(cout)
        self.short = nn.Identity()
        if stride != 1 or cin != cout:   # projection shortcut when shapes change
            self.short = nn.Sequential(nn.Conv2d(cin, cout, 1, stride, bias=False),
             ↪  nn.BatchNorm2d(cout))

    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        return F.relu(out + self.short(x))   # y = F(x) + x
```

DenseNet: reuse everything

DenseNet [16] pushes feature reuse further: inside a dense block, layer ℓ receives the **concatenation** of all earlier feature maps, $\mathbf{x}_\ell = H_\ell([\mathbf{x}_0, \mathbf{x}_1, \ldots, \mathbf{x}_{\ell-1}])$, where $[\cdot]$ is channel concatenation and H_ℓ is BN–ReLU–conv. Each layer adds only a few channels (the "growth rate"), so the network is parameter-efficient, and every layer has a short path to the loss. The cost is memory: concatenated activations grow with depth, which made DenseNets slower in practice than their parameter count suggests. A useful lesson for later: parameters, FLOPs and wall-clock time are three different things.

MobileNet: factorize the convolution

MobileNet [17] targets phones. A standard $K \times K$ convolution from M to N channels on an $H \times W$ map costs HWK^2MN multiply–adds. A **depthwise separable** convolution splits it into a **depthwise** $K \times K$ convolution that filters each channel separately (HWK^2M) and a **pointwise** 1×1 convolution that mixes channels ($HWMN$). The cost ratio is

$$\frac{HWK^2M + HWMN}{HWK^2MN} = \frac{1}{N} + \frac{1}{K^2},$$

so with $K = 3$ and many output channels the separable version is roughly 8–9 times cheaper. The bet is that spatial filtering and channel mixing do not need to be learned jointly. That bet held up so well that depthwise convolutions reappear in EfficientNet and ConvNeXt.

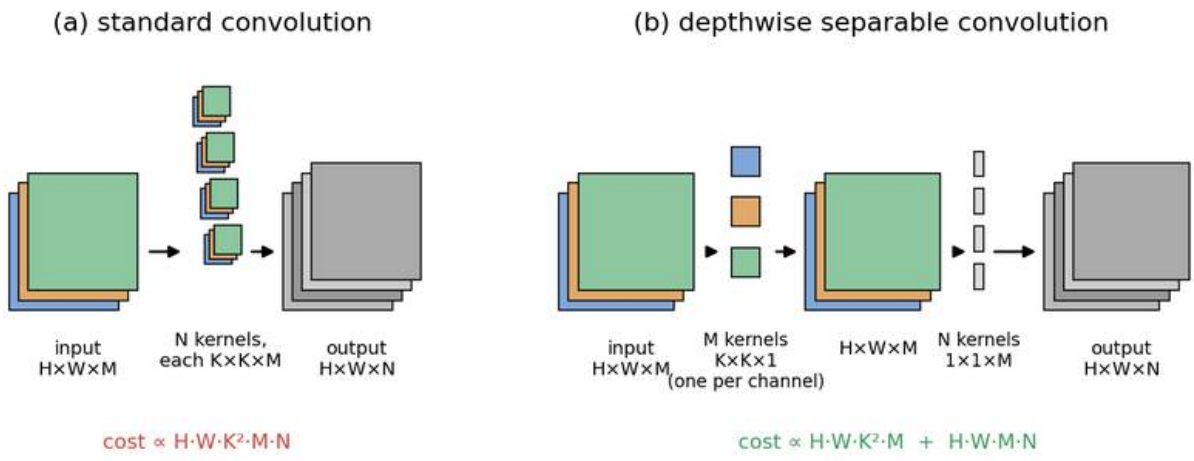


Figure 5 — Standard convolution filters and mixes channels in one step. Depthwise separable convolution filters each channel alone (depthwise) and then mixes channels with 1×1 kernels (pointwise).

```
import torch.nn as nn

def n_params(m):
    return sum(p.numel() for p in m.parameters())

M, N, K = 128, 128, 3
standard = nn.Conv2d(M, N, K, padding=1, bias=False)
separable = nn.Sequential(nn.Conv2d(M, M, K, padding=1, groups=M, bias=False),   # depthwise
                          nn.Conv2d(M, N, 1, bias=False))                          #
                           ↪ pointwise
print("standard:", n_params(standard), " separable:", n_params(separable),
      f" ratio {n_params(separable) / n_params(standard):.3f} vs 1/N + 1/K^2 = {1/N +
       ↪ 1/K**2:.3f}")
# standard: 147456  separable: 17536  ratio 0.119 vs 1/N + 1/K^2 = 0.119
```

EfficientNet: scale depth, width and resolution together

Given a good small network, how should we make it bigger? Deeper, wider, or higher input resolution? Tan and Le [18] argued these interact: a higher resolution image needs more depth (larger receptive field) and more width (finer patterns). **Compound scaling** grows all three with one coefficient ϕ:

$$d = \alpha^{\phi}, \quad w = \beta^{\phi}, \quad r = \gamma^{\phi}, \qquad \text{s.t. } \alpha\beta^2\gamma^2 \approx 2, \;\; \alpha, \beta, \gamma \geq 1,$$

where d, w, r multiply depth, width and resolution. FLOPs scale roughly as $d\,w^2 r^2$, so each unit increase of ϕ roughly doubles the cost. The constants are found by a small grid search on the base network (itself found by architecture search, using inverted-bottleneck blocks with depthwise convolutions and squeeze-and-excitation). The idea, "scale in a balanced way", outlived the specific constants.

RegNet: design spaces instead of single designs

Radosavovic et al. [19] studied populations of networks rather than single winners. Starting from a broad space of ResNet-like networks, they repeatedly constrained it (for example, shared bottleneck ratios, widths that increase across stages) and checked whether the *distribution* of errors improved. The surviving space, RegNet, describes stage widths with a simple quantized linear rule. The lesson is methodological: good architectures follow simple regularities, and comparing distributions of models is more reliable than comparing one tuned model against another.

ConvNeXt: a CNN modernized step by step

After vision transformers took over the leaderboards, Liu et al. [20] asked how much of their advantage came from attention and how much from everything else. They started from a ResNet-50 trained with a transformer-style recipe and changed one thing at a time: stage ratios like Swin's, a "patchify" stem (a 4×4 stride-4 convolution), depthwise convolutions with larger 7×7 kernels, an inverted bottleneck, fewer activations and normalizations, GELU and LayerNorm. The result, ConvNeXt, matched Swin-style transformers of similar cost on ImageNet and downstream tasks. The lesson: architecture families matter less than the many small design and training choices that come with them.

Attention and transformers

Plain version. A convolution only looks at a small neighbourhood with fixed weights. Attention lets each position decide, based on content, which other positions to listen to. Vision transformers apply this to image patches.

Squeeze-and-excitation: attention inside a CNN

The SE block [21] adds cheap, content-dependent **channel** weights to any CNN. It "squeezes" each channel to one number by global average pooling, passes the vector through a small two-layer MLP, and multiplies each channel by a sigmoid gate:

$$\mathbf{s} = \sigma(W_2\, \delta(W_1\, \bar{\mathbf{u}})), \qquad \tilde{\mathbf{u}}_c = s_c\, \mathbf{u}_c,$$

where $\mathbf{u}_c$ is channel c of the feature map, $\bar{\mathbf{u}}$ the vector of channel means, δ a ReLU, W_1 reduces the dimension by a ratio r and W_2 restores it. The network can now emphasise "fur" channels when the global context says "animal". SE blocks are part of EfficientNet and many mobile networks.

```
import torch
import torch.nn as nn

class SE(nn.Module):
    """Squeeze-and-excitation: reweight channels using global context."""
    def __init__(self, c, r=16):
```

```
        super().__init__()
        self.fc = nn.Sequential(nn.Linear(c, c // r), nn.ReLU(), nn.Linear(c // r, c),
        ↪  nn.Sigmoid())

    def forward(self, x):                       # x: (B, C, H, W)
        s = x.mean(dim=(2, 3))                  # squeeze: (B, C)
        w = self.fc(s)[:, :, None, None]        # excitation: (B, C, 1, 1) in (0, 1)
        return x * w

print(SE(64)(torch.randn(2, 64, 8, 8)).shape)
```

ViT: an image as a sequence of patches

The Vision Transformer [22] removes convolutions almost entirely. An image $\mathbf{x} \in \mathbb{R}^{H\times W\times C}$ is cut into $N = HW/P^2$ non-overlapping $P\times P$ patches. Each patch is flattened and linearly projected to a D-dimensional **token**. A learnable **class token** is prepended and learnable **position embeddings** are added:

$$\mathbf{z}_0 = [\,\mathbf{x}_{\text{class}};\ \mathbf{x}_p^1\mathbf{E};\ \dots;\ \mathbf{x}_p^N\mathbf{E}\,] + \mathbf{E}_{\text{pos}},$$

where $\mathbf{x}_p^i \in \mathbb{R}^{P^2C}$ is the i-th flattened patch, $\mathbf{E} \in \mathbb{R}^{P^2C\times D}$ the patch embedding and $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{(N+1)\times D}$ the position embeddings. A stack of L standard transformer encoder layers (multi-head self-attention and MLP, each with LayerNorm and a residual connection) processes the sequence, and a small head reads the final class token to produce logits. Self-attention for one head is

$$\text{Attn}(\mathbf{Q},\mathbf{K},\mathbf{V}) = \text{softmax}\Big(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}}\Big)\mathbf{V},$$

with $\mathbf{Q} = \mathbf{Z}\mathbf{W}_Q$, $\mathbf{K} = \mathbf{Z}\mathbf{W}_K$, $\mathbf{V} = \mathbf{Z}\mathbf{W}_V$ linear projections of the token matrix $\mathbf{Z}$, and d the key dimension. The attention matrix is $(N+1)\times(N+1)$: every patch can attend to every other patch from the first layer, so the receptive field is global immediately.

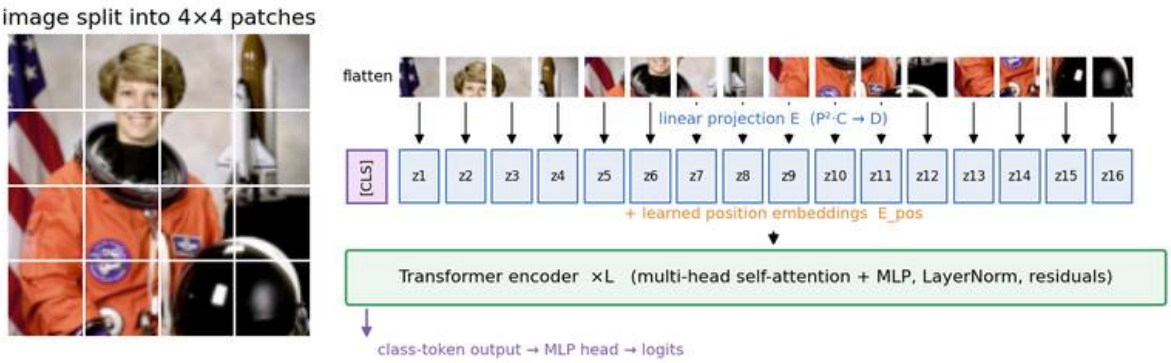


Figure 6 — The ViT input pipeline. Patches become tokens through one shared linear projection (equivalently, a $P\times P$ convolution with stride P). A class token and learned position embeddings are added, and the class token's final state is classified.

```
import torch
import torch.nn as nn

B, C, H, W, P, D = 2, 3, 32, 32, 8, 64
img = torch.randn(B, C, H, W)

# 1) patchify: (B, C, H, W) -> (B, N, P*P*C) with N = (H/P)*(W/P)
patches = img.unfold(2, P, P).unfold(3, P, P)                 # (B, C, H/P, W/P, P, P)
patches = patches.permute(0, 2, 3, 1, 4, 5).reshape(B, -1, C * P * P)
N = patches.shape[1]

# 2) linear patch embedding + class token + learned position embedding
embed = nn.Linear(C * P * P, D)          # same as nn.Conv2d(C, D, kernel_size=P, stride=P)
cls = nn.Parameter(torch.zeros(1, 1, D))
pos = nn.Parameter(torch.randn(1, N + 1, D) * 0.02)
z = torch.cat([cls.expand(B, -1, -1), embed(patches)], dim=1) + pos   # (B, N+1, D)

# 3) one head of scaled dot-product self-attention
Wq, Wk, Wv = (nn.Linear(D, D, bias=False) for _ in range(3))
q, k, v = Wq(z), Wk(z), Wv(z)
A = (q @ k.transpose(1, 2) / D ** 0.5).softmax(dim=-1)        # (B, N+1, N+1), rows sum to 1
out = A @ v
print("tokens:", N + 1, " attention:", tuple(A.shape), " class-token output:", tuple(out[:,
 ↪  0].shape))
```

Why ViT needs data. A CNN has built-in assumptions, called **inductive biases**: locality (a pixel mostly relates to its neighbours) and translation equivariance (the same filter everywhere). ViT has almost none; apart from patch extraction and position embeddings, it must learn spatial structure from data. The ViT paper [22] reports that, trained on ImageNet-1k alone, ViTs fell behind comparable ResNets, but pre-trained on much larger datasets (ImageNet-21k or the 300M-image JFT) they matched or beat them. Weak priors cost data at small scale and stop limiting the model at large scale. The same paper also tried **hybrid** models, feeding the feature map of a ResNet stem into the transformer instead of raw patches; hybrids helped at small compute budgets and the gap closed as models grew.

DeiT: training ViTs on ImageNet alone

DeiT [23] showed that the "ViT needs huge data" conclusion was partly a statement about recipes. With strong augmentation (RandAugment, Mixup, CutMix, random erasing), stochastic depth, repeated augmentation and AdamW, a ViT-B trained only on ImageNet-1k reached 83.1% top-1. DeiT also added a **distillation token**: a second learnable token, like the class token, whose output is trained to match a teacher network's predictions (they found a convnet teacher worked best), while the class token is trained on the true label. This is a transformer-specific form of knowledge distillation [51]. At test time the two heads' outputs are combined.

Swin: windows, shifts and a hierarchy

Global attention costs grow quadratically with the number of tokens, which is a problem for high-resolution inputs and for dense tasks. Swin [24] computes attention only inside non-overlapping $M \times M$ windows, and shifts the window grid by half a window in alternating layers so information crosses window borders. Token count is reduced between stages by merging 2×2 neighbouring patches, giving a CNN-like pyramid with strides 4, 8, 16, 32. For an $h \times w$ token map with C channels, the paper gives

$$\Omega(\text{MSA}) = 4hwC^2 + 2(hw)^2C, \qquad \Omega(\text{W-MSA}) = 4hwC^2 + 2M^2hwC,$$

so windowed attention is linear in the number of tokens hw for a fixed window size M. Swin brought back two CNN priors, locality and hierarchy, and became a general-purpose backbone for detection and segmentation as well as classification. The broad landscape of vision transformers is surveyed by Khan et al. [25].

Training recipes

Plain version. The same network can gain or lose several points of accuracy depending on how it is trained. A modern recipe is a bundle of small tricks that each make memorizing harder or optimization smoother.

Data augmentation

- **Random resized crop** samples a crop covering a random fraction of the image area and a random aspect ratio, then resizes it to the training resolution. It teaches scale and position invariance and is the default for ImageNet training. **Horizontal flips** are added when they are label-preserving (they are for most photos, not for text or digits).
- **Mixup** [26] blends two images and their labels: $\tilde{\mathbf{x}} = \lambda \mathbf{x}_i + (1-\lambda)\mathbf{x}_j$ and $\tilde{\mathbf{y}} = \lambda \mathbf{y}_i + (1-\lambda)\mathbf{y}_j$, with $\lambda \sim \text{Beta}(\alpha, \alpha)$ and $\mathbf{y}$ one-hot. It encourages linear behaviour between training examples.
- **CutMix** [27] pastes a random rectangle from image j into image i and mixes the labels in proportion to the pasted area. Unlike Mixup, every pixel stays natural, and the network must recognize objects from partial views.
- **RandAugment** [28] replaces learned augmentation policies with two numbers: apply N random operations (rotate, shear, posterize, contrast, ...) at a shared magnitude M. Its paper reports that this tiny search space matches or beats earlier automated policies, and that the right strength depends on model and dataset size.

```
import numpy as np
import torch
import torch.nn.functional as F

def mixup(x, y, num_classes, alpha=0.2):
    lam = float(np.random.beta(alpha, alpha))
```

```
    idx = torch.randperm(x.size(0))
    y1 = F.one_hot(y, num_classes).float()
    return lam * x + (1 - lam) * x[idx], lam * y1 + (1 - lam) * y1[idx]

def cutmix(x, y, num_classes, alpha=1.0):
    lam = float(np.random.beta(alpha, alpha))
    idx = torch.randperm(x.size(0))
    H, W = x.shape[-2:]
    h, w = int(H * np.sqrt(1 - lam)), int(W * np.sqrt(1 - lam))
    cy, cx = np.random.randint(H), np.random.randint(W)
    y0, y1_, x0, x1_ = max(cy - h // 2, 0), min(cy + h // 2, H), max(cx - w // 2, 0), min(cx
    ↪  + w // 2, W)
    x = x.clone()
    x[..., y0:y1_, x0:x1_] = x[idx][..., y0:y1_, x0:x1_]
    lam = 1 - (y1_ - y0) * (x1_ - x0) / (H * W)            # actual area kept
    y1 = F.one_hot(y, num_classes).float()
    return x, lam * y1 + (1 - lam) * y1[idx]

def soft_ce(logits, soft_targets):
    return -(soft_targets * logits.log_softmax(1)).sum(1).mean()
```

Label smoothing

Cross-entropy with one-hot targets keeps pushing the true logit up forever; the loss only reaches zero at infinite margin. **Label smoothing** [13] replaces the target with

$$\mathbf{q} = (1 - \varepsilon)\,\mathbf{e}_y + \frac{\varepsilon}{K}\mathbf{1},$$

where ε (often 0.1) is the smoothing strength, $\mathbf{e}_y$ the one-hot vector and $\mathbf{1}$ the all-ones vector. The optimum is now a finite logit gap, which regularizes the network and usually reduces overconfidence. In PyTorch it is one argument: `F.cross_entropy(logits, y, label_smoothing=0.1)`.

Optimizer, schedule and regularizers

- **AdamW** [29] decouples weight decay from the adaptive gradient step: instead of adding $\lambda\theta$ to the gradient (which Adam then rescales per parameter), it shrinks the weights directly, $\theta \leftarrow \theta - \eta(\hat{\mathbf{m}}/(\sqrt{\hat{\mathbf{v}}} + \epsilon) + \lambda\theta)$, where η is the learning rate, $\hat{\mathbf{m}}, \hat{\mathbf{v}}$ Adam's moment estimates and λ the decay. It is the default for transformers and for modern CNN recipes. Biases and normalization parameters are usually excluded from decay.
- **Warmup + cosine schedule.** Large learning rates are unstable in the first steps, when normalization statistics and Adam's moment estimates are still poor, so the learning rate rises linearly for a few epochs. It then follows a half cosine down to near zero, $\eta_t = \frac{1}{2}\eta_{\max}(1 + \cos(\pi t/T))$ with t the step after warmup and T the remaining steps, an idea popularized by SGDR [30].
- **Stochastic depth** [31] randomly skips whole residual branches during training (the block becomes identity), with a skip probability that grows with depth. It shortens the effective depth

during training and acts like an ensemble of shallower networks.
- **EMA of weights** keeps an exponential moving average $\bar{\theta} \leftarrow \mu\bar{\theta} + (1-\mu)\theta$ (with μ close to 1, e.g. 0.999) and evaluates $\bar{\theta}$. It smooths out the noise of the last SGD steps.

```
import math
import torch

def warmup_cosine(step, total, warmup):
    if step < warmup:
        return (step + 1) / warmup                      # linear warmup
    p = (step - warmup) / max(1, total - warmup)
    return 0.5 * (1 + math.cos(math.pi * p))            # cosine decay to 0

model = torch.nn.Linear(10, 3)
decay = [p for n, p in model.named_parameters() if p.ndim > 1]      # weights
no_decay = [p for n, p in model.named_parameters() if p.ndim <= 1]  # biases, norm scales
opt = torch.optim.AdamW([{"params": decay, "weight_decay": 0.05},
                         {"params": no_decay, "weight_decay": 0.0}], lr=1e-3)
total = 1000
sched = torch.optim.lr_scheduler.LambdaLR(opt, lambda s: warmup_cosine(s, total, warmup=50))
ema = torch.optim.swa_utils.AveragedModel(model,
 ↪  multi_avg_fn=torch.optim.swa_utils.get_ema_multi_avg_fn(0.999))
for step in range(total):
    loss = model(torch.randn(16, 10)).pow(2).mean()
    opt.zero_grad(); loss.backward(); opt.step(); sched.step(); ema.update_parameters(model)
```

A small experiment

To see the pieces working together, we trained a 78k-parameter ResNet ("TinyResNet": a stem, three basic blocks with 16, 32 and 64 channels, global average pooling) on scikit-learn's 8×8 digits, upsampled to 32 × 32 so that the strided stages have room to work. To make overfitting visible we used only **100 training images**, 500 images for calibration and 1,197 for testing, with AdamW (weight decay 0.05), batch size 32, 40 epochs and two epochs of warmup before a cosine decay. One run used no augmentation. The other used random rotation ($\pm 15°$), scaling and shifts (a random-resized-crop-like affine; no flips, since a flipped digit is a different symbol) together with label smoothing 0.1. All code is in `scripts/figures/dl_02_image_classification.py` and runs on a CPU in about a minute.

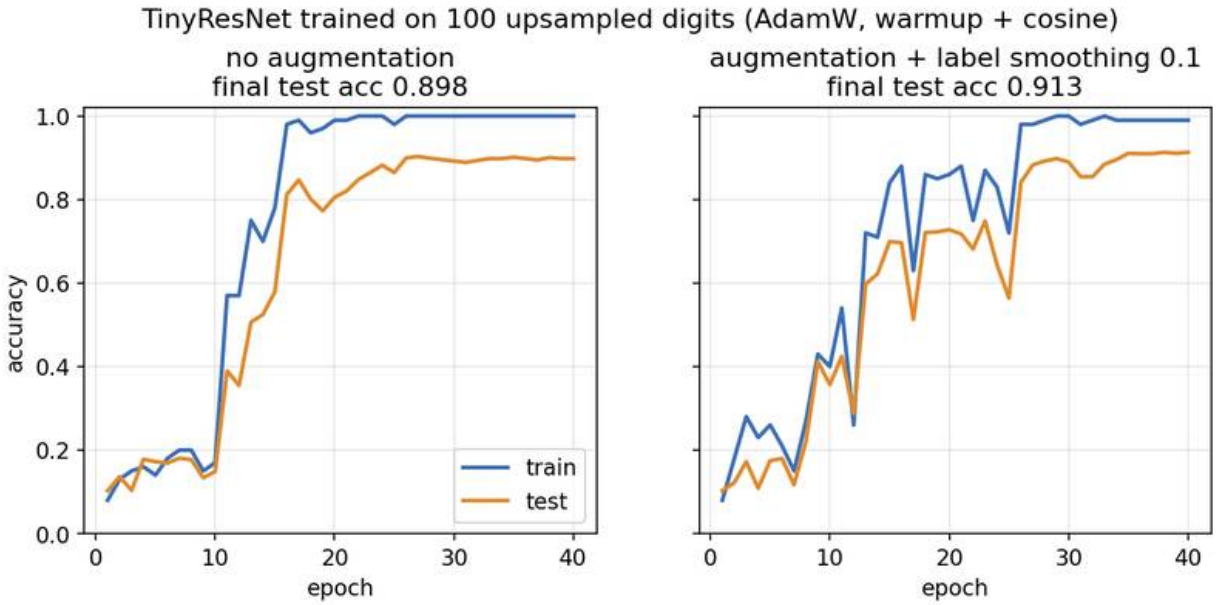


Figure 7 — Training curves on 100 training digits. Without augmentation (left) the network memorizes the training set and reaches 89.8% test accuracy. With augmentation and label smoothing (right) it reaches 91.3%. The gap is modest on such an easy task; on natural images the effect of the recipe is much larger.

The no-augmentation model reaches 100% training accuracy after about twenty epochs: with 100 images, memorization is easy. The augmented model sees a slightly different version of each digit every epoch, learns more slowly, and generalizes a little better (91.3% against 89.8%). Do not read too much into 1.5 points on one seed; the point is the mechanism.

"ResNet strikes back"

The most striking evidence for recipes comes from Wightman, Touvron and Jégou [32]. They kept the 2015 ResNet-50 architecture fixed and retrained it with a modern recipe: random resized crops and flips, RandAugment, Mixup and CutMix, the LAMB optimizer with a cosine schedule, a binary cross-entropy loss and longer training. The vanilla ResNet-50 reached **80.4% top-1** on ImageNet at 224×224 without extra data or distillation, against 76.1% for the standard torchvision weights they compare with. Two consequences follow. First, many "new architecture beats ResNet-50" claims compared a new model with a modern recipe against an old model with an old recipe. Second, when you compare architectures, train them with the same, tuned recipe, as ConvNeXt [20] and RegNet [19] did.

Transfer learning and fine-tuning

Plain version. Training a backbone from scratch needs a lot of data. Instead, start from a network already trained on a big dataset and adapt it to your task. You can either keep its features frozen

and train only a new last layer, or gently retrain everything.

Let g_ϕ be a pre-trained backbone that maps an image to a feature vector $\mathbf{h} = g_\phi(\mathbf{x}) \in \mathbb{R}^D$, and $\mathbf{W} \in \mathbb{R}^{K \times D}$ a new linear head.

- **Linear probe.** Freeze ϕ and train only $\mathbf{W}$ (a logistic regression on fixed features). It is cheap, needs few labels, and is the standard way to *measure* how good a representation is.
- **Full fine-tuning.** Update ϕ and $\mathbf{W}$ together, usually with a learning rate for the backbone 10–100 times smaller than for the head, a short warmup, and sometimes layer-wise learning-rate decay (smaller rates for earlier layers). It adapts the features to the new domain and usually wins when there is enough labelled data.
- In between lie partial fine-tuning (only the last stages), and parameter-efficient methods that add small trainable modules to a frozen backbone.

Kornblith et al. [33] measured how ImageNet accuracy predicts transfer performance. For fine-tuning, better ImageNet models generally transferred better, with a strong correlation. For fixed features, the correlation was weaker and depended on regularization choices: some settings that improved ImageNet accuracy (for example, label smoothing and dropout) produced *worse* frozen features for other tasks. A good classifier and a good feature extractor are related, but not the same thing.

```
import torch
import torch.nn as nn

backbone = nn.Sequential(nn.Conv2d(3, 32, 3, 2, 1), nn.ReLU(), nn.Conv2d(32, 64, 3, 2, 1),
 ↪  nn.ReLU(),
                          nn.AdaptiveAvgPool2d(1), nn.Flatten())          # stand-in for a
                           ↪  pre-trained model
head = nn.Linear(64, 5)

# linear probe: freeze the backbone, train only the head
for p in backbone.parameters():
    p.requires_grad = False
probe_opt = torch.optim.AdamW(head.parameters(), lr=1e-3)

# full fine-tuning: unfreeze, use a smaller learning rate for the backbone than for the head
for p in backbone.parameters():
    p.requires_grad = True
ft_opt = torch.optim.AdamW([{"params": backbone.parameters(), "lr": 1e-5},
                            {"params": head.parameters(), "lr": 1e-3}], weight_decay=0.05)
```

Self-supervised and foundation-model classifiers

Plain version. Labels are expensive; images are not. Self-supervised learning invents a task whose answer is in the image itself (are these two crops from the same photo? what was under this mask?), learns features by solving it, and only then uses a few labels.

Contrastive learning: SimCLR and MoCo

Contrastive methods pull together the embeddings of two augmented **views** of the same image and push apart views of different images. SimCLR [34] takes a batch of B images, makes two random views of each, and minimizes the InfoNCE (NT-Xent) loss for each positive pair (i, j):

$$\ell_{i,j} = -\log \frac{\exp(\mathrm{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau)}{\sum_{k=1}^{2B} \mathbb{1}_{[k \neq i]} \exp(\mathrm{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)},$$

where $\mathbf{z}$ are projected embeddings, sim is cosine similarity, τ a temperature and the other $2B - 2$ views in the batch act as negatives. It is a $2B - 1$-way softmax classification: "which of these is my partner?". SimCLR found that strong augmentation (especially crop plus color distortion), a non-linear projection head and large batches were essential. MoCo [35] avoids the need for huge batches by keeping a **queue** of negatives encoded by a slowly updated **momentum encoder**, whose weights are an EMA of the main encoder.

DINO and DINOv2: self-distillation

DINO [36] drops explicit negatives. A student network sees several crops (including small local ones) and is trained to match the output distribution of a teacher that sees global crops. The teacher is an EMA of the student; centering and sharpening of the teacher outputs prevent collapse to a constant. With ViTs, DINO features showed striking properties: the class token's attention maps segment objects without any segmentation labels, and a simple k-NN classifier on frozen features works well. DINOv2 [38] scaled the approach with an automatically curated dataset and a large ViT distilled into smaller ones, producing general-purpose **frozen features** that work across classification, segmentation and depth without fine-tuning. DINOv3 [39] continued the scaling and introduced "Gram anchoring" to keep dense patch features from degrading during long training.

MAE: masked image modeling

MAE [37] borrows the masked-language-model idea. It masks a large fraction of patches (75% in the paper), feeds only the visible patches to a ViT encoder, and asks a light decoder to reconstruct the missing pixels. Because the encoder skips masked tokens, pre-training is cheap. MAE features are less linearly separable than contrastive ones (linear probes are weaker), but they **fine-tune** very well. This is a recurring pattern: different pre-training objectives favour different adaptation methods.

CLIP: zero-shot classification with language

CLIP [40] trains an image encoder g and a text encoder h on hundreds of millions of image–caption pairs with a symmetric contrastive loss, so that matching pairs have high cosine similarity.

A classifier is then built **without any training images**: write a prompt per class, such as `a photo of a dog.`, embed it, and predict

$$\hat{y} = \arg\max_k \frac{g(\mathbf{x})^\top h(t_k)}{\|g(\mathbf{x})\| \, \|h(t_k)\|},$$

where t_k is the prompt for class k. The text embeddings act as the weights of a linear classifier, computed from words. The paper reports that zero-shot CLIP matched the accuracy of a ResNet-50 trained on ImageNet's 1.28M labelled images, and that averaging several prompt templates ("a photo of a ...", "a blurry photo of a ...") helps. Zero-shot accuracy depends strongly on prompt wording, on whether the class names are visually meaningful, and on how well the web data covers the domain.

Robustness and evaluation pitfalls

Plain version. A classifier that scores 90% on its test set may fail badly when the photos come from a different camera, when someone adds invisible noise on purpose, or when the "easy clue" it learned (backgrounds, watermarks) is missing. And even when it is right, its confidence may be wrong.

Distribution shift: ImageNet-C, -R and -A

Hendrycks and colleagues built three widely used shifted test sets for ImageNet classifiers. **ImageNet-C** [42] applies 15 algorithmic corruptions (noise, blur, weather, digital artefacts) at five severities and summarizes the error relative to a baseline (mean corruption error). **ImageNet-R** [43] collects renditions (art, cartoons, sculptures, sketches, toys) of 200 ImageNet classes and tests whether a model understands shape and concept rather than photographic texture. **ImageNet-A** [44] contains natural, unmodified photos that were filtered to fool a ResNet-50; standard models do very poorly on it. These sets reveal different weaknesses, and improving one does not guarantee improving the others. Large-scale pre-training (as in CLIP and DINOv2) tends to improve robustness to such natural shifts much more than architecture changes at fixed data.

Adversarial examples: FGSM

Goodfellow et al. [45] explained adversarial examples by the **linearity** of networks in high dimensions: a tiny change to every input dimension, aligned with the weights, adds up to a large change in the output. The **fast gradient sign method** perturbs an input in one step:

$$\mathbf{x}_{\text{adv}} = \text{clip}\left(\mathbf{x} + \varepsilon \, \text{sign}(\nabla_{\mathbf{x}} \mathcal{L}(f_\theta(\mathbf{x}), y))\right),$$

where ε bounds the change per pixel (ℓ_∞ norm), $\mathcal{L}$ is the loss and clip keeps pixels in the valid range.

```
import torch.nn.functional as F

def fgsm(model, x, y, eps):
    x = x.clone().requires_grad_(True)
    F.cross_entropy(model(x), y).backward()
    return (x + eps * x.grad.sign()).clamp(-1, 1).detach()
```

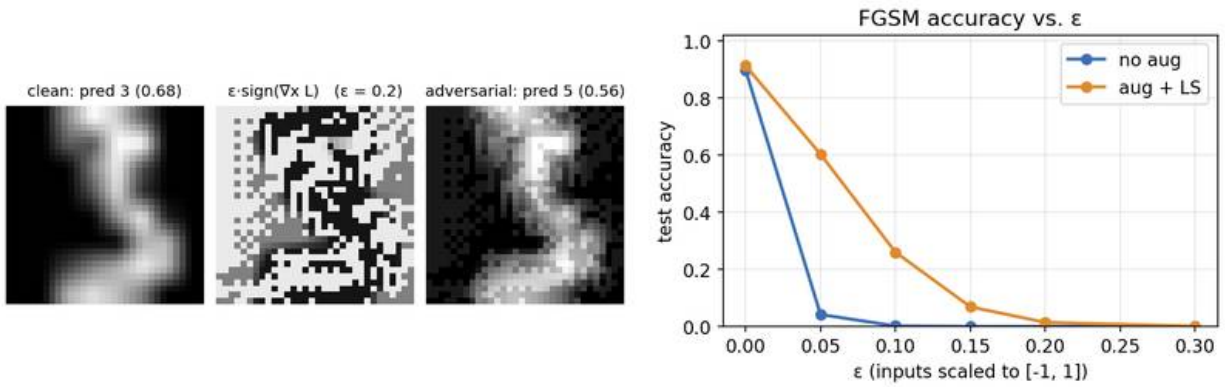


Figure 8 — FGSM on the TinyResNet models (inputs in $[-1, 1]$). At $\varepsilon = 0.05$ test accuracy falls from 89.8% to 4.2% for the model trained without augmentation and from 91.3% to 60.3% for the model trained with augmentation and label smoothing; at $\varepsilon = 0.2$ both are near zero.

Two cautions about Figure 8. First, the augmented model looks more robust at small ε, but label smoothing is known to flatten the loss surface around training points, which can weaken one-step gradient attacks without giving real robustness; stronger iterative attacks are the proper test. Second, $\varepsilon = 0.05$ on a $[-1, 1]$ scale is visible on an 8×8 digit; on natural images, imperceptible perturbations already fool standard ImageNet models.

Shortcut learning

Geirhos et al. [46] collect many failures under one name: **shortcut learning**. A network minimizes the training loss with whatever cue is easiest, which is often not the cue we intended: background instead of animal, hospital tag instead of pathology, texture instead of shape. Shortcuts look fine on an i.i.d. test set, because the same shortcut exists there. They fail under distribution shift. The prescription is to test out of distribution on purpose, design tests that break the suspected shortcut, and use interpretability tools such as Grad-CAM (below) to check what the model attends to.

Calibration and temperature scaling

Temperature scaling [47] divides all logits by one scalar $T > 0$ before the softmax, $\mathbf{p} = \mathrm{softmax}(\mathbf{z}/T)$. T is fitted by minimizing the negative log-likelihood on a held-out **cali-**

bration split (never the test set). $T > 1$ softens overconfident predictions; $T < 1$ sharpens underconfident ones. Because dividing by T does not change the argmax, accuracy is unchanged.

```
import torch
import torch.nn.functional as F

def fit_temperature(logits, y, iters=300):
    """Learn one scalar T > 0 on a held-out calibration split."""
    log_t = torch.zeros(1, requires_grad=True)
    opt = torch.optim.Adam([log_t], lr=0.05)
    for _ in range(iters):
        loss = F.cross_entropy(logits / log_t.exp(), y)
        opt.zero_grad(); loss.backward(); opt.step()
    return log_t.exp().item()

def ece(probs, y, n_bins=10):
    conf, pred = probs.max(1)
    correct = (pred == y).float()
    edges = torch.linspace(0, 1, n_bins + 1)
    total = 0.0
    for lo, hi in zip(edges[:-1], edges[1:]):
        m = (conf > lo) & (conf <= hi)
        if m.any():
            total += m.float().mean() * (correct[m].mean() - conf[m].mean()).abs()
    return float(total)

# toy check: an over-confident "model" = true logits multiplied by 3
torch.manual_seed(0)
true_logits = torch.randn(4000, 10)
y = torch.distributions.Categorical(logits=true_logits).sample()
over = 3 * true_logits
T = fit_temperature(over[:2000], y[:2000])
print(f"T = {T:.2f}  ECE {ece(over[2000:].softmax(1), y[2000:]):.3f} -> "
      f"{ece((over[2000:] / T).softmax(1), y[2000:]):.3f}")
# T = 2.90  ECE 0.345 -> 0.018
```

In this toy check the logits are three times too large, and the fitted temperature (2.90) recovers that factor. On our real TinyResNet (the model trained without augmentation), the miscalibration went the *other* way.

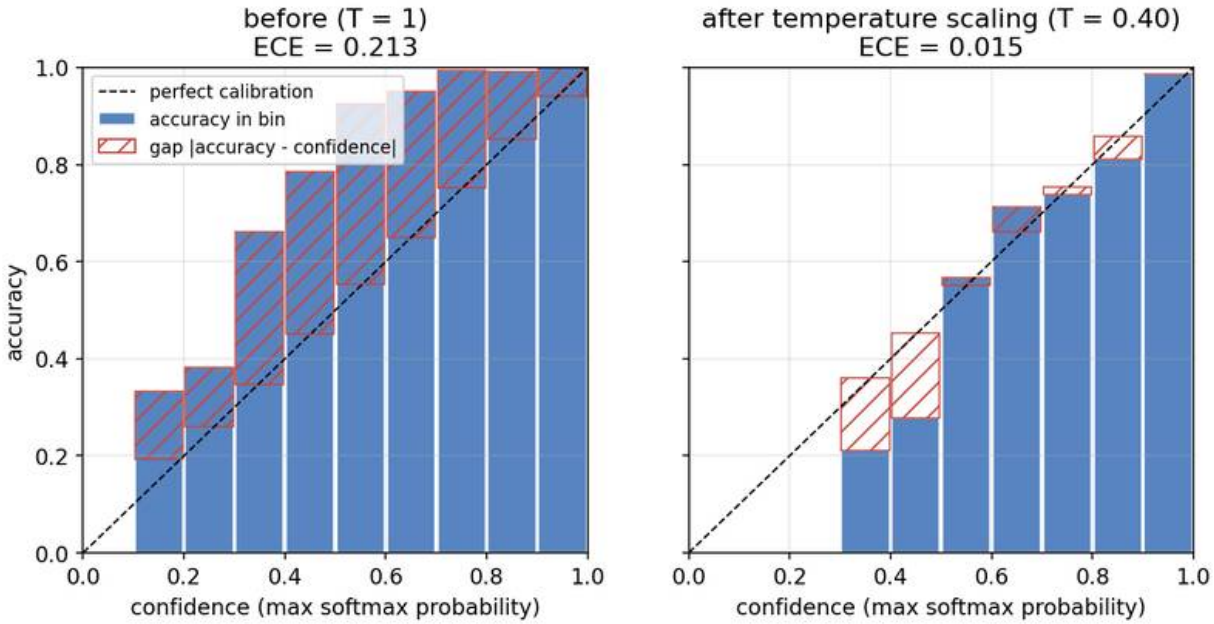


Figure 9 — Reliability diagrams for the TinyResNet trained without augmentation, on 1,197 test digits. Before scaling (left), accuracy exceeds confidence in most bins: the model is underconfident (ECE 0.213). A temperature $T = 0.40$ fitted on 500 separate calibration images brings ECE to 0.015 and test NLL from 0.515 to 0.298, without changing any prediction.

Why underconfident? Our tiny model was trained briefly with strong weight decay, which keeps logits small. Guo et al. [47] observed overconfidence in large, long-trained modern networks, and identified depth, width, batch normalization and *reduced* weight decay as contributing factors. Miscalibration has no fixed sign; measure it. Also note that calibration is itself fragile under distribution shift: a temperature fitted on in-distribution data does not guarantee calibrated confidence on ImageNet-C-style shifts.

Efficiency

Plain version. A model must fit the device and the time budget. Parameter count, compute, and actual speed are three different measurements, and only the last one is what users feel.

- **Parameters** determine model size on disk and in memory. A float32 parameter takes 4 bytes.
- **FLOPs** (or multiply–accumulates, MACs; one MAC is usually counted as two FLOPs) measure arithmetic per image. For a convolution with $K \times K$ kernels, C_{in} input and C_{out} output channels on an $H' \times W'$ output map, MACs $= H'W'K^2C_{\text{in}}C_{\text{out}}$. Compute grows with the square of resolution.
- **Latency and throughput** are measured on target hardware. Depthwise convolutions have few FLOPs but low arithmetic intensity, so they often run slower on GPUs than their FLOP count suggests; DenseNet's concatenations cost memory bandwidth. Always benchmark.

- **Quantization** stores weights and activations as 8-bit integers instead of 32-bit floats. Jacob et al. [50] described integer-only inference with quantization-aware training, which simulates rounding during training so the network learns to tolerate it. Post-training quantization with a small calibration set is the cheap alternative.
- **Distillation** [51] trains a small student to match a large teacher's soft predictions, as DeiT did with its distillation token.

Interpretability: CAM and Grad-CAM

Plain version. A heat map over the image shows which regions pushed the score for a class up. It is a sanity check, not a proof of reasoning.

Class activation mapping (CAM) [48] works for networks that end with global average pooling and one linear layer. If A^k is the k-th feature map of the last convolutional layer and w_k^c the weight connecting it to class c, then the class score is $\sum_k w_k^c \cdot \text{mean}(A^k)$, and the map $\sum_k w_k^c A^k$ shows where that score comes from. **Grad-CAM** [49] generalizes this to any differentiable architecture by replacing the weights with spatially averaged gradients:

$$\alpha_k^c = \frac{1}{Z} \sum_{i,j} \frac{\partial y^c}{\partial A_{ij}^k}, \qquad L_{\text{Grad-CAM}}^c = \text{ReLU}\Big(\sum_k \alpha_k^c A^k \Big),$$

where y^c is the logit of class c, A_{ij}^k the activation of channel k at position (i, j), and Z the number of positions. The ReLU keeps only regions with a positive influence on the class. The coarse map (8×8 in our network) is upsampled to the image size.

```
import torch.nn.functional as F

def grad_cam(model, x, target=None):
    model.eval()
    feats = model.features(x)                          # (1, K, h, w) last conv stage
    feats.retain_grad()
    logits = model.fc(feats.mean(dim=(2, 3)))
    c = logits.argmax(1).item() if target is None else target
    logits[0, c].backward()
    alpha = feats.grad.mean(dim=(2, 3), keepdim=True)  # channel importance weights
    cam = F.relu((alpha * feats).sum(1, keepdim=True))
    cam = F.interpolate(cam, size=x.shape[-2:], mode="bilinear", align_corners=False)
    return (cam / (cam.max() + 1e-8))[0, 0].detach(), c
```

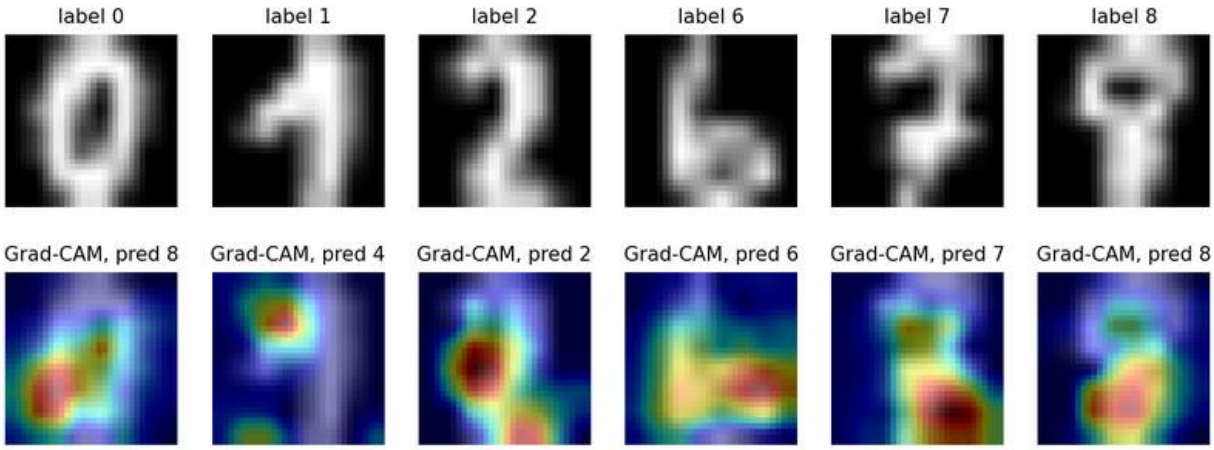


Figure 10 — Grad-CAM for the predicted class on the first test example of six digits, using the augmented TinyResNet. Two are misclassified: the 0 is called an 8, with heat on the lower-left part of the loop, and the 1 is called a 4, with heat on the hook at its upper left. The maps point to where the misleading evidence was found.

Explanations are most useful exactly when the model is wrong, as in Figure 10: they show which strokes looked like the wrong class. But treat saliency maps with care. They are coarse (limited by the last feature map's resolution), and they explain one class score for one input, not the model's general reasoning.

Modern view

Surveys worth reading. Li et al. [1] review CNNs from their history and basic building blocks through classic and advanced models, highlighting the key ideas behind each, and run experiments that compare activation functions, loss functions and optimizers to derive rules of thumb; they close with applications of 1-D, 2-D and multi-dimensional convolution and a list of open issues. Khan et al. [25] survey vision transformers across tasks. For classification they group them into uniform-scale models (ViT, DeiT), multi-scale hierarchical models (such as Swin), hybrids with convolutions, and self-supervised transformers (such as DINO), and they stress the reliance on large-scale pre-training and the cost of quadratic attention. Gui et al. [41] survey self-supervised learning and divide it into context-based methods (pretext tasks such as rotation prediction or jigsaw puzzles), contrastive methods (with negative pairs, self-distillation as in DINO, or feature decorrelation), generative methods (masked image modeling as in MAE), and hybrids of contrastive and generative objectives, before covering combinations with semi-supervised and multi-modal learning. On the reliability of benchmarks, the ImageNetV2 study [7], the re-labelling study of Beyer et al. [8] and the label-error analysis of Northcutt et al. [9] together argue for multiple test sets, multi-label evaluation and reporting variance.

The 2023–2026 picture. Five trends define current practice.

1. **Pre-train once, adapt cheaply.** Most new classifiers are not trained from scratch. They start

from a large backbone trained with self-supervision (DINOv2 [38], DINOv3 [39]), image–text contrast (CLIP [40] and open reproductions), or masked modeling (MAE [37]), and adapt it with a linear probe, a k-NN rule, or a short fine-tune.

2. **Zero-shot and open-vocabulary classification.** Image–text models turn the label set into text. The class list can change at test time without retraining, which blurs the line between classification and retrieval.
3. **Scaling laws for vision.** Zhai et al. [52] scaled ViTs to billions of parameters and characterized how error falls with compute, data and model size, including the observation that larger models are more sample-efficient in few-shot transfer. Cherti et al. [53] fitted reproducible power laws for open CLIP training on public data, and found that the scaling behaviour depends on the training data distribution and the downstream task.
4. **Architectures converged.** ConvNeXt [20] and Swin [24] show that convolutions and attention, given the same recipe and scale, land in a similar place. The choice is now driven by hardware, resolution and downstream tasks (dense prediction favours hierarchical designs) more than by top-1 accuracy.
5. **Evaluation moved beyond ImageNet-1k top-1.** Robustness suites (ImageNet-C/R/A [42]–[44] and ImageNetV2 [7]), calibration, and transfer to many downstream datasets are now standard parts of a backbone paper.

Open problems. Robustness to natural distribution shift still lags clean accuracy, and gains from robust training methods are often specific to the shift they target. Shortcut learning [46] is hard to detect without counterfactual tests. Calibration under shift is unsolved. Long-tailed and fine-grained recognition remain hard because the rare classes have the fewest examples. Benchmark saturation and label noise make small accuracy differences on ImageNet-1k uninformative. And the dominant foundation models are trained on web-scale data whose composition, licensing and biases are hard to audit, which matters directly for what a "zero-shot" classifier will and will not recognize.

Key takeaways

- A classifier outputs logits; softmax + cross-entropy trains single-label tasks, independent sigmoids + binary cross-entropy train multi-label tasks. Report top-1 accuracy and, where it matters, calibration (ECE) and per-class results.
- Each landmark CNN fixed one bottleneck: ReLU/GPUs (AlexNet), small stacked kernels (VGG), 1×1 reductions and multi-branch modules (Inception), identity shortcuts for optimization (ResNet), depthwise separable convolutions for cost (MobileNet), balanced scaling (EfficientNet).
- ViTs trade convolutional inductive biases for flexibility; they need more data or a stronger recipe (DeiT), and windows plus hierarchy (Swin) make them practical backbones. ConvNeXt shows that the recipe explains much of the gap.
- Training recipes matter as much as architecture: augmentation (random resized crop, Mixup, CutMix, RandAugment), label smoothing, AdamW, warmup + cosine, stochastic depth and EMA lifted an unchanged ResNet-50 to 80.4% top-1.

- Pre-trained backbones are the default starting point. Linear probes measure representations; fine-tuning adapts them. CLIP turns class names into classifier weights for zero-shot recognition.
- Accuracy hides *which* errors a model makes. Read the confusion matrix; under class imbalance report precision/recall, balanced accuracy or MCC, and prefer the precision–recall curve and AP over ROC-AUC. State whether an average is macro (every class equal) or micro (every image equal; equals accuracy for single-label tasks).
- Separate training data from evaluation data: judge a backbone on a battery of test sets (ImageNet val and ReaL, ImageNetV2, ImageNet-C/R/A/Sketch, ObjectNet, fine-grained and transfer suites), tune only on validation data, and report mean $\pm$ std over seeds.
- Benchmark accuracy is fragile: test on shifted data, beware shortcuts and adversarial inputs, fit a temperature on held-out data, and inspect Grad-CAM maps, especially on errors.

Exercises

1. **Receptive fields and parameters.** Show that three stacked 3×3 convolutions (stride 1) have a 7×7 receptive field. With C input and output channels and no biases, compare their parameter count with a single 7×7 convolution.

   Hint

   Each 3×3 layer adds 2 pixels to the receptive field: $3 \to 5 \to 7$. Parameters: $3 \cdot 9C^2 = 27C^2$ versus $49C^2$, about 45% fewer, plus two extra nonlinearities.

2. **Bottleneck arithmetic.** A ResNet bottleneck has $4d$ input channels and reduces to d. Count the weights (no biases) of the 1×1–3×3–1×1 branch as a function of d, and compare with a basic block of two 3×3 convolutions at width $4d$. What is the ratio?

   Hint

   Bottleneck: $4d \cdot d + 9d^2 + d \cdot 4d = 17d^2$. Basic block at width $4d$: $2 \cdot 9 \cdot 16d^2 = 288d^2$. The ratio is $17/288 \approx 0.059$, about 17 times fewer weights.

3. **Label smoothing optimum.** For one example with K classes and smoothing ε, assume all wrong-class logits are equal. Show that the cross-entropy with smoothed targets is minimized when the logit gap between the true class and each wrong class is $\log\left((1 - \varepsilon + \varepsilon/K) \,/\, (\varepsilon/K)\right)$. Evaluate it for $K = 1000$, $\varepsilon = 0.1$.

   Hint

   The minimum of cross-entropy over $\mathbf{p}$ is at $\mathbf{p} = \mathbf{q}$. So $p_y = 1 - \varepsilon + \varepsilon/K$ and every other $p_k = \varepsilon/K$, and the logit gap is the log of their ratio: $\log(0.9001/0.0001) \approx 9.1$. Without smoothing the optimal gap is infinite.

4. **ViT token count.** A ViT with 16×16 patches receives a 224×224 image. How many tokens enter the encoder? If the image is fine-tuned at 384 × 384, how many tokens are there, by what factor does the attention-matrix size grow, and what must be done with the position embeddings?

   Hint

   $(224/16)^2 = 196$ patches plus one class token: 197. At 384: $24^2 + 1 = 577$. The attention matrix grows by $(577/197)^2 \approx 8.6$. The learned position embeddings form a 14×14 grid; they are interpolated (2-D) to 24 × 24, which is what the ViT paper does when fine-tuning at higher resolution.

5. **Temperature does not change accuracy.** Prove that for any $T > 0$, $\arg\max_k \mathrm{softmax}(\mathbf{z}/T)_k = \arg\max_k z_k$. Then, using `scripts/figures/dl_02_image_classification.py`, fit the temperature on the *test* set instead of the calibration split and compare the reported ECE. Why is that number optimistic?

   Hint

   Division by a positive constant and the exponential are both strictly increasing, and the softmax denominator is shared. Fitting T on the test set uses test labels to tune a parameter, so the reported ECE is no longer an honest estimate of performance on new data.

6. **Build a shortcut.** Modify the digit experiment so that in the training set every "3" has a bright pixel in the top-left corner, but the test set does not. Train the TinyResNet, report test accuracy on the 3s, and look at Grad-CAM maps for a few training 3s. What do you expect, and how would you detect this problem if you did not know the shortcut was there?

   Hint

   The network can classify training 3s from the corner pixel alone, so test accuracy on 3s drops while training accuracy stays perfect, and Grad-CAM tends to highlight the corner. Without knowing the shortcut, look for heat maps that fall on background, evaluate on independently collected data, and run counterfactual tests (erase or move suspicious regions and see whether predictions change).

7. **Macro versus micro F1.** A three-class model is tested on 100 images. Rows are true classes, columns predictions: class A (90 images) → 88 A, 1 B, 1 C; class B (5 images) → 3 A, 2 B, 0 C; class C (5 images) → 4 A, 0 B, 1 C. Compute per-class precision, recall and F_1, then macro F_1 and micro F_1. Which number would you put in a report about detecting the rare classes B and C, and why?

   Hint

   Column sums are 95, 3 and 2. Precision: $88/95 = 0.926$, $2/3 = 0.667$, $1/2 = 0.500$. Recall: $88/90 = 0.978$, $2/5 = 0.400$, $1/5 = 0.200$. Using $F_1 = 2\mathrm{TP}/(2\mathrm{TP} + \mathrm{FP} + \mathrm{FN})$: $176/185 = 0.951$, $4/8 = 0.500$, $2/7 = 0.286$. Macro $F_1 = (0.951 + 0.500 + 0.286)/3 = 0.579$. Micro F_1 equals accuracy, $91/100 = 0.910$. Micro is dominated by class A; macro exposes that B and C are mostly missed, so macro (plus per-class recall) is the honest number for rare classes. You can check the result with `precision_recall_fscore_support(..., average="macro")`.

References

1. Z. Li, F. Liu, W. Yang, S. Peng, and J. Zhou, "A Survey of Convolutional Neural Networks: Analysis, Applications, and Prospects," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 12, pp. 6999–7019, 2022. doi:10.1109/TNNLS.2021.3084827
2. O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, et al., "ImageNet Large Scale Visual Recognition Challenge," *International Journal of Computer Vision*, vol. 115, pp. 211–252, 2015. doi:10.1007/s11263-015-0816-y
3. Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner, "Gradient-Based Learning Applied to Document Recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998. doi:10.1109/5.726791
4. A. Krizhevsky, "Learning Multiple Layers of Features from Tiny Images," technical report, University of Toronto, 2009. dataset page
5. T. Ridnik, E. Ben-Baruch, A. Noy, and L. Zelnik-Manor, "ImageNet-21K Pretraining for the Masses," *NeurIPS*, 2021. arXiv
6. G. Van Horn, O. Mac Aodha, Y. Song, Y. Cui, C. Sun, A. Shepard, et al., "The iNaturalist Species Classification and Detection Dataset," *CVPR*, 2018. arXiv
7. B. Recht, R. Roelofs, L. Schmidt, and V. Shankar, "Do ImageNet Classifiers Generalize to ImageNet?," *ICML*, PMLR vol. 97, 2019. paper
8. L. Beyer, O. J. Hénaff, A. Kolesnikov, X. Zhai, and A. van den Oord, "Are We Done with ImageNet?," arXiv:2006.07159, 2020. arXiv
9. C. G. Northcutt, A. Athalye, and J. Mueller, "Pervasive Label Errors in Test Sets Destabilize Machine Learning Benchmarks," *NeurIPS*, 2021. arXiv
10. A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," *Advances in Neural Information Processing Systems 25 (NIPS)*, 2012. paper
11. K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," *ICLR*, 2015. arXiv
12. C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going Deeper with Convolutions," *CVPR*, 2015. doi:10.1109/CVPR.2015.7298594
13. C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna, "Rethinking the Inception Architecture for Computer Vision," *CVPR*, 2016. doi:10.1109/CVPR.2016.308
14. K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," *CVPR*, 2016. doi:10.1109/CVPR.2016.90
15. K. He, X. Zhang, S. Ren, and J. Sun, "Identity Mappings in Deep Residual Networks," *ECCV*, 2016. arXiv
16. G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, "Densely Connected Convolutional Networks," *CVPR*, 2017. arXiv
17. A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," arXiv:1704.04861, 2017. arXiv
18. M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," *ICML*, 2019. arXiv
19. I. Radosavovic, R. P. Kosaraju, R. Girshick, K. He, and P. Dollár, "Designing Network Design Spaces," *CVPR*, 2020. arXiv

20. Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, “A ConvNet for the 2020s,” *CVPR*, 2022. arXiv
21. J. Hu, L. Shen, S. Albanie, G. Sun, and E. Wu, “Squeeze-and-Excitation Networks,” arXiv:1709.01507, 2017 (journal version in *IEEE TPAMI*). arXiv
22. A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, et al., “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” *ICLR*, 2021. arXiv
23. H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, “Training Data-Efficient Image Transformers & Distillation through Attention,” *ICML*, PMLR vol. 139, 2021. paper
24. Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, “Swin Transformer: Hierarchical Vision Transformer using Shifted Windows,” *ICCV*, 2021. doi:10.1109/ICCV48922.2021.00986
25. S. Khan, M. Naseer, M. Hayat, S. W. Zamir, F. S. Khan, and M. Shah, “Transformers in Vision: A Survey,” *ACM Computing Surveys*, 2022. arXiv
26. H. Zhang, M. Cisse, Y. N. Dauphin, and D. Lopez-Paz, “mixup: Beyond Empirical Risk Minimization,” *ICLR*, 2018. arXiv
27. S. Yun, D. Han, S. J. Oh, S. Chun, J. Choe, and Y. Yoo, “CutMix: Regularization Strategy to Train Strong Classifiers with Localizable Features,” *ICCV*, 2019. arXiv
28. E. D. Cubuk, B. Zoph, J. Shlens, and Q. V. Le, “RandAugment: Practical Automated Data Augmentation with a Reduced Search Space,” *CVPR Workshops*, 2020. doi:10.1109/CVPRW50498.2020.00359
29. I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” *ICLR*, 2019. arXiv
30. I. Loshchilov and F. Hutter, “SGDR: Stochastic Gradient Descent with Warm Restarts,” *ICLR*, 2017. arXiv
31. G. Huang, Y. Sun, Z. Liu, D. Sedra, and K. Q. Weinberger, “Deep Networks with Stochastic Depth,” *ECCV*, 2016. doi:10.1007/978-3-319-46493-0_39
32. R. Wightman, H. Touvron, and H. Jégou, “ResNet Strikes Back: An Improved Training Procedure in timm,” arXiv:2110.00476, 2021. arXiv
33. S. Kornblith, J. Shlens, and Q. V. Le, “Do Better ImageNet Models Transfer Better?,” *CVPR*, 2019. arXiv
34. T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, “A Simple Framework for Contrastive Learning of Visual Representations,” *ICML*, 2020. arXiv
35. K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, “Momentum Contrast for Unsupervised Visual Representation Learning,” *CVPR*, 2020. arXiv
36. M. Caron, H. Touvron, I. Misra, H. Jégou, J. Mairal, P. Bojanowski, and A. Joulin, “Emerging Properties in Self-Supervised Vision Transformers,” *ICCV*, 2021. doi:10.1109/ICCV48922.2021.00951
37. K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick, “Masked Autoencoders Are Scalable Vision Learners,” *CVPR*, 2022. doi:10.1109/CVPR52688.2022.01553
38. M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, et al., “DINOv2: Learning Robust Visual Features without Supervision,” *Transactions on Machine Learning Research*, 2024. arXiv
39. O. Siméoni, H. V. Vo, M. Seitzer, F. Baldassarre, M. Oquab, C. Jose, et al., “DINOv3,” arXiv:2508.10104, 2025. arXiv

40. A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, et al., “Learning Transferable Visual Models From Natural Language Supervision,” *ICML*, PMLR vol. 139, 2021. paper
41. J. Gui, T. Chen, J. Zhang, Q. Cao, Z. Sun, H. Luo, and D. Tao, “A Survey on Self-Supervised Learning: Algorithms, Applications, and Future Trends,” *IEEE Transactions on Pattern Analysis and Machine Intelligence* (arXiv:2301.05712, 2023). arXiv
42. D. Hendrycks and T. Dietterich, “Benchmarking Neural Network Robustness to Common Corruptions and Perturbations,” *ICLR*, 2019. arXiv
43. D. Hendrycks, S. Basart, N. Mu, S. Kadavath, F. Wang, E. Dorundo, et al., “The Many Faces of Robustness: A Critical Analysis of Out-of-Distribution Generalization,” *ICCV*, 2021. arXiv
44. D. Hendrycks, K. Zhao, S. Basart, J. Steinhardt, and D. Song, “Natural Adversarial Examples,” *CVPR*, 2021. arXiv
45. I. J. Goodfellow, J. Shlens, and C. Szegedy, “Explaining and Harnessing Adversarial Examples,” in *Proc. International Conference on Learning Representations* (*ICLR*), 2015. arXiv
46. R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, and F. A. Wichmann, “Shortcut Learning in Deep Neural Networks,” *Nature Machine Intelligence*, 2020. arXiv
47. C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, “On Calibration of Modern Neural Networks,” *ICML*, 2017. arXiv
48. B. Zhou, A. Khosla, A. Lapedriza, A. Oliva, and A. Torralba, “Learning Deep Features for Discriminative Localization,” *CVPR*, 2016. doi:10.1109/CVPR.2016.319
49. R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, “Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization,” *ICCV*, 2017. doi:10.1109/ICCV.2017.74
50. B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, et al., “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” *CVPR*, 2018. doi:10.1109/CVPR.2018.00286
51. G. Hinton, O. Vinyals, and J. Dean, “Distilling the Knowledge in a Neural Network,” arXiv: 1503.02531, 2015. arXiv
52. X. Zhai, A. Kolesnikov, N. Houlsby, and L. Beyer, “Scaling Vision Transformers,” *CVPR*, 2022. arXiv
53. M. Cherti, R. Beaumont, R. Wightman, M. Wortsman, G. Ilharco, C. Gordon, et al., “Reproducible Scaling Laws for Contrastive Language-Image Learning,” *CVPR*, 2023. arXiv
54. T. Fawcett, “An Introduction to ROC Analysis,” *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006. doi:10.1016/j.patrec.2005.10.010
55. J. Davis and M. Goadrich, “The Relationship Between Precision-Recall and ROC Curves,” *Proc. 23rd International Conference on Machine Learning* (*ICML*), pp. 233–240, 2006. doi:10.1145/1143844.1143874
56. T. Saito and M. Rehmsmeier, “The Precision-Recall Plot Is More Informative than the ROC Plot When Evaluating Binary Classifiers on Imbalanced Datasets,” *PLOS ONE*, vol. 10, no. 3, e0118432, 2015. doi:10.1371/journal.pone.0118432
57. D. Chicco and G. Jurman, “The Advantages of the Matthews Correlation Coefficient (MCC) over F1 Score and Accuracy in Binary Classification Evaluation,” *BMC Genomics*, vol. 21, article 6, 2020. doi:10.1186/s12864-019-6413-7
58. M. Sokolova and G. Lapalme, “A Systematic Analysis of Performance Measures for Classification Tasks,” *Information Processing & Management*, vol. 45, no. 4, pp. 427–437, 2009. doi:10.1016/

j.ipm.2009.03.002
59. J. Cohen, “A Coefficient of Agreement for Nominal Scales,” *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960. doi:10.1177/001316446002000104
60. M.-L. Zhang and Z.-H. Zhou, “A Review on Multi-Label Learning Algorithms,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 26, no. 8, pp. 1819–1837, 2014. doi:10.1109/TKDE.2013.39
61. M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, “The Pascal Visual Object Classes (VOC) Challenge,” *International Journal of Computer Vision*, vol. 88, no. 2, pp. 303–338, 2010. doi:10.1007/s11263-009-0275-4
62. F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. paper
63. H. Wang, S. Ge, E. P. Xing, and Z. C. Lipton, “Learning Robust Global Representations by Penalizing Local Predictive Power,” *NeurIPS*, 2019 (introduces ImageNet-Sketch). arXiv
64. A. Barbu, D. Mayo, J. Alverio, W. Luo, C. Wang, D. Gutfreund, J. Tenenbaum, and B. Katz, “ObjectNet: A Large-Scale Bias-Controlled Dataset for Pushing the Limits of Object Recognition Models,” *NeurIPS*, 2019. dataset page
65. C. Wah, S. Branson, P. Welinder, P. Perona, and S. Belongie, “The Caltech-UCSD Birds-200-2011 Dataset,” Technical Report CNS-TR-2011-001, California Institute of Technology, 2011. dataset page
66. J. Krause, M. Stark, J. Deng, and L. Fei-Fei, “3D Object Representations for Fine-Grained Categorization,” *IEEE International Conference on Computer Vision Workshops* (*ICCVW*), pp. 554–561, 2013 (introduces Stanford Cars). doi:10.1109/ICCVW.2013.77
67. S. Maji, E. Rahtu, J. Kannala, M. Blaschko, and A. Vedaldi, “Fine-Grained Visual Classification of Aircraft,” arXiv:1306.5151, 2013. arXiv
68. O. M. Parkhi, A. Vedaldi, A. Zisserman, and C. V. Jawahar, “Cats and Dogs,” *CVPR*, 2012 (introduces Oxford-IIIT Pets). dataset page
69. M.-E. Nilsback and A. Zisserman, “Automated Flower Classification over a Large Number of Classes,” *Indian Conference on Computer Vision, Graphics and Image Processing* (*ICVGIP*), 2008 (introduces Oxford Flowers-102). dataset page
70. L. Bossard, M. Guillaumin, and L. Van Gool, “Food-101 – Mining Discriminative Components with Random Forests,” *ECCV*, 2014. dataset page
71. L. Fei-Fei, R. Fergus, and P. Perona, “One-Shot Learning of Object Categories,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2006 (the reference given for Caltech-101). dataset page
72. X. Zhai, J. Puigcerver, A. Kolesnikov, et al., “A Large-scale Study of Representation Learning with the Visual Task Adaptation Benchmark,” arXiv:1910.04867, 2019. arXiv
73. Z. Liu, Z. Miao, X. Zhan, J. Wang, B. Gong, and S. X. Yu, “Large-Scale Long-Tailed Recognition in an Open World,” *CVPR*, 2019 (introduces ImageNet-LT). arXiv

DLCV 03 • Object Detection: From R-CNN to DETR

How deep networks find and label every object in an image: box regression, IoU, anchors, NMS and feature pyramids; two-stage, one-stage, anchor-free and transformer detectors; open-vocabulary models; and how detection is measured.

Online edition: https://blog.shyandram.dev/en/learn/deep-learning/dl-03-object-detection/

What you'll learn

- What a detector outputs (boxes, class labels, confidences), and how detection differs from classification, semantic segmentation and instance segmentation.
- The building blocks every modern detector shares, in math and in code: box parameterization and regression targets, IoU and its GIoU/DIoU/CIoU variants, anchors and label assignment, non-maximum suppression, feature pyramids and the focal loss.
- How the four big families work and why each was invented: two-stage (R-CNN to Cascade R-CNN), one-stage (YOLO, SSD, RetinaNet), anchor-free (CornerNet, CenterNet, FCOS) and set prediction with transformers (DETR to RT-DETR).
- How open-vocabulary and grounding detectors (OWL-ViT, Grounding DINO, YOLO-World) and promptable foundation models change what "the class list" means.
- How detection is evaluated (precision–recall, AP, VOC and COCO protocols, AP50/AP75/APs/m/l), which datasets matter, and what limits speed in deployment.

The big picture

A classifier answers one question about a whole image: *what is this?* A detector answers a harder one: *what is where?* It must find every object of interest, draw a tight rectangle (a **bounding box**) around each, name it, and say how sure it is. The number of objects is not known in advance: there may be zero, one or three hundred.

> **Explain like I'm five**
>
> Imagine a "Where's Waldo?" page. A **classifier** only tells you "this page has Waldo on it." A **detector** puts a sticker on Waldo, another sticker on every dog and every balloon, and writes on each sticker what it is and how sure it is ("dog, pretty sure"). The tricky parts are the ones you would also find tricky: tiny things far away, things hiding behind other things, and not putting two stickers on the same dog.

Detection is the entry point of most "real" vision systems. Self-driving cars detect pedestrians and vehicles; phones detect faces before focusing; factories detect defects; trackers (see the tracking chapter linked at the end) mostly *link detections over time*. The field also produced many ideas that

now appear everywhere in deep learning: multi-task losses, feature pyramids, the focal loss, and set prediction with bipartite matching.

This tutorial assumes you know CNNs and image classification (DLCV 02) and the classical feature ideas of IP 12. Two surveys frame the whole story: Zou et al., *Object Detection in 20 Years* [1], and Liu et al.'s IJCV survey of deep generic detection [2]. We return to both in the Modern view.

The detection problem

Plain version. The input is an image. The output is a short list of (box, class, score) triples. The training data is a list of human-drawn boxes with class names.

Precise version. Given an image $I \in \mathbb{R}^{H\times W\times 3}$, a detector outputs a set

$$\hat{Y} = \{(\hat{b}_i, \hat{c}_i, \hat{s}_i)\}_{i=1}^{N},$$

where $\hat{b}_i \in \mathbb{R}^4$ is a box, $\hat{c}_i \in \{1, \dots, K\}$ is one of K classes, $\hat{s}_i \in [0, 1]$ is a confidence score, and N varies from image to image. The ground truth is a set $Y = \{(b_j, c_j)\}_{j=1}^{M}$ of M annotated objects. Two box formats are common: corners (x_1, y_1, x_2, y_2) and center–size (c_x, c_y, w, h). Boxes are **axis-aligned**; rotated boxes are a separate variant used for aerial images and text.

Three properties make detection harder than classification:

1. **Variable-size output.** The network must decide *how many* things to output. Every architecture in this tutorial is, at heart, a different answer to this problem.
2. **Localization and recognition at once.** The model must be invariant to *what* changes about an object's appearance but sensitive to *where* it is, a tension that shapes the design of every head.
3. **Extreme imbalance.** A typical image has a handful of objects but tens of thousands of candidate positions and sizes, nearly all background.

Detection versus its neighbours

Figure 1 compares the four classic recognition tasks on one synthetic scene. **Classification** gives one label per image. **Detection** gives a box and a label per object. **Semantic segmentation** labels every pixel with a class, so two touching balls merge into one blob. **Instance segmentation** gives each object its own pixel mask, which is detection plus a mask per box; Mask R-CNN [16] is the standard way to get it, as we will see.

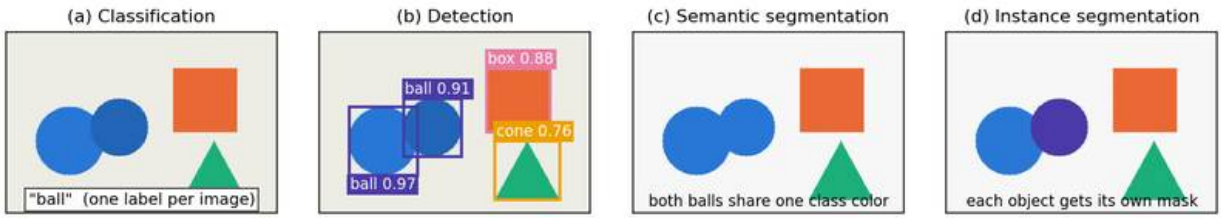


Figure 1 — The same scene seen by four tasks: (a) classification, (b) detection, (c) semantic segmentation, (d) instance segmentation.

Classical detection: sliding windows and hand-made features

Plain version. Before deep learning, detection was "classification in a loop": cut out a window at every position and size, ask a classifier whether it contains the object, and keep the windows that say yes.

Sliding windows. For a window of fixed size $w \times h$ slid with stride s over an image pyramid with scale factor λ, the number of windows is roughly

$$N_{\text{win}} \approx \sum_{k \geq 0} \frac{(H\lambda^{-k} - h)(W\lambda^{-k} - w)}{s^2},$$

where k indexes the pyramid levels. For a 640×480 image, a 64×128 window, $s = 8$ and $\lambda = 1.2$, that is on the order of 10^4 windows for one aspect ratio, all of which must be classified. This is the template matching of IP 13 scaled up, and it has the same weaknesses: one template shape, many scales, and a lot of wasted work on background.

Viola–Jones (2001) [5] made real-time face detection possible with three tricks. *Haar-like features* (differences of sums over adjacent rectangles) can be evaluated in constant time using an **integral image** $S(x, y) = \sum_{x' \leq x, y' \leq y} I(x', y')$, so any rectangle sum needs four lookups. *AdaBoost* selects a small set of informative features out of a huge pool. An *attentional cascade* of increasingly complex classifiers rejects most background windows after only a few feature evaluations, which is the first big idea for beating the imbalance problem: spend computation only where it might matter.

HOG + linear SVM (2005) [6] replaced raw intensities with **histograms of oriented gradients**, close relatives of the gradient-orientation descriptors in IP 12. The detection window is divided into small cells; each cell stores a histogram of gradient orientations weighted by magnitude; overlapping blocks of cells are contrast-normalized; the concatenated vector is scored by a linear SVM. HOG is robust to small shifts and illumination changes, and it made pedestrian detection practical.

Deformable part models (DPM) [7] added structure. An object is a coarse *root* HOG filter plus several higher-resolution *part* filters that may shift relative to the root, paying a quadratic deformation cost:

$$\text{score}(p_0) = F_0 \cdot \phi(p_0) + \sum_{i=1}^{n} \max_{p_i} \Big[F_i \cdot \phi(p_i) - d_i \cdot \psi(p_i - p_0) \Big] + \beta,$$

where p_0 is the root position, p_i the position of part i, F_i the learned filters, $\phi(\cdot)$ the HOG features at a position, ψ the displacement features (dx, dy, dx^2, dy^2), d_i learned deformation weights and β a bias. Part positions are latent variables, trained with a *latent SVM*. DPM and its variants dominated the PASCAL VOC benchmark [3] until 2012.

Why this era ended. Classical detectors were *slow* (an exhaustive scan over positions, scales and aspect ratios, each scored by a hand-designed pipeline) and *brittle* (one hand-designed feature

type, a few rigid templates, and poor transfer to new categories, viewpoints and clutter). They also had no way to share features across classes. Deep detectors kept the useful ideas, such as scanning in a feature pyramid, cascades, parts and hard-negative mining, and replaced the hand-made features with learned ones.

Core building blocks

Every detector in the rest of this tutorial is assembled from the same parts. We define each one carefully, with code you can run.

Box parameterization and regression targets

Plain version. A network is bad at outputting absolute pixel coordinates, but good at outputting small corrections. So we start from a **reference box** (a proposal, an anchor or a point) and predict how to nudge it.

Given a reference box $P = (P_x, P_y, P_w, P_h)$ in center–size form and its matched ground-truth box $G = (G_x, G_y, G_w, G_h)$, the standard targets introduced with R-CNN [8] are

$$t_x = \frac{G_x - P_x}{P_w}, \qquad t_y = \frac{G_y - P_y}{P_h}, \qquad t_w = \log \frac{G_w}{P_w}, \qquad t_h = \log \frac{G_h}{P_h}.$$

Dividing the shift by the reference size makes the target **scale-invariant** (a 10-pixel error matters more for a 20-pixel box than for a 400-pixel box). The **log** for width and height makes scaling symmetric (doubling and halving are $+\log 2$ and $-\log 2$) and keeps the decoded size positive: $\hat{G}_w = P_w \exp(\hat{t}_w)$. In practice the targets are often divided by fixed standard deviations, such as $(0.1, 0.1, 0.2, 0.2)$, so that all four have similar magnitude.

```
import numpy as np

def to_cxcywh(b):
    return np.stack([(b[:, 0] + b[:, 2]) / 2, (b[:, 1] + b[:, 3]) / 2,
                     b[:, 2] - b[:, 0], b[:, 3] - b[:, 1]], axis=1)

def to_xyxy(c):
    return np.stack([c[:, 0] - c[:, 2] / 2, c[:, 1] - c[:, 3] / 2,
                     c[:, 0] + c[:, 2] / 2, c[:, 1] + c[:, 3] / 2], axis=1)

def encode(anchors, gt, std=(0.1, 0.1, 0.2, 0.2)):
    """Regression targets (tx, ty, tw, th) of gt relative to anchors (both xyxy)."""
    P, G = to_cxcywh(anchors), to_cxcywh(gt)
    t = np.stack([(G[:, 0] - P[:, 0]) / P[:, 2],
                  (G[:, 1] - P[:, 1]) / P[:, 3],
                  np.log(G[:, 2] / P[:, 2]),
                  np.log(G[:, 3] / P[:, 3])], axis=1)
    return t / np.array(std)          # normalize so all four have similar scale
```

```
def decode(anchors, t, std=(0.1, 0.1, 0.2, 0.2)):
    P = to_cxcywh(anchors)
    t = t * np.array(std)
    c = np.stack([P[:, 0] + t[:, 0] * P[:, 2],
                  P[:, 1] + t[:, 1] * P[:, 3],
                  P[:, 2] * np.exp(t[:, 2]),
                  P[:, 3] * np.exp(t[:, 3])], axis=1)
    return to_xyxy(c)

anchors = np.array([[0, 0, 32, 32], [100, 100, 228, 164]], float)
gt      = np.array([[4, 2, 40, 30], [90, 110, 250, 170]], float)
t = encode(anchors, gt)
print(np.round(t, 3))
print(np.allclose(decode(anchors, t), gt))   # round trip
```

```
[[ 1.875  0.     0.589 -0.668]
 [ 0.469  1.25   1.116 -0.323]]
True
```

These targets are usually trained with the **smooth L1** (Huber) loss of Fast R-CNN [9],

$$\text{smooth}_{L_1}(x) = \begin{cases} 0.5\,x^2 & \text{if } |x| < 1, \\ |x| - 0.5 & \text{otherwise,} \end{cases}$$

applied to each coordinate difference $x = \hat{t} - t$. It behaves like L2 for small errors (smooth gradients) and like L1 for large ones (robust to outliers early in training). Its weakness is that the four coordinates are treated as independent numbers, while the evaluation metric, IoU, treats the box as a whole. That mismatch motivated IoU-based losses.

IoU and its variants

Plain version. IoU asks: of all the area covered by either box, what fraction is covered by both? It is 1 for a perfect match and 0 for boxes that do not touch.

For two boxes A and B,

$$\text{IoU}(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{|A \cap B|}{|A| + |B| - |A \cap B|},$$

where $|\cdot|$ is area. IoU is scale-invariant, and it is the yardstick for evaluation (a detection counts as correct if its IoU with an unmatched ground-truth box exceeds a threshold), for label assignment and for NMS. Figure 2 shows how demanding the thresholds are: shifting a square by only about 18% of its side along both axes already drops IoU to 0.5.

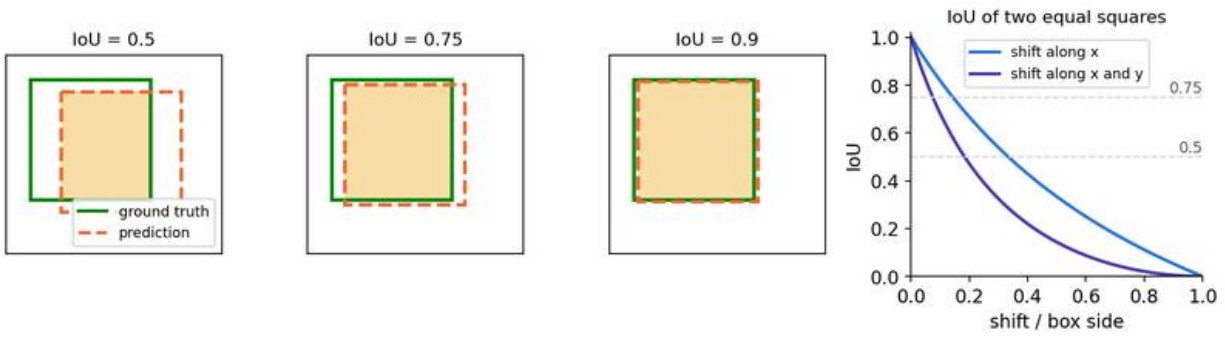


Figure 2 — What IoU thresholds look like. Left: a prediction at IoU 0.5, 0.75 and 0.9 with the ground truth. Right: IoU of two equal squares as one is shifted along x, or along x and y.

As a loss, $1 - \text{IoU}$ has a flaw: when two boxes do not overlap, IoU is 0 no matter how far apart they are, so the gradient is zero. Three fixes are widely used:

- **GIoU** [11] adds a penalty based on the smallest box C that encloses both: $\text{GIoU} = \text{IoU} - \frac{|C \setminus (A \cup B)|}{|C|}$, which lies in $(-1, 1]$ and keeps decreasing as non-overlapping boxes drift apart.
- **DIoU** [12] penalizes the distance between centers directly: $\text{DIoU} = \text{IoU} - \frac{\rho^2(\mathbf{b}, \mathbf{b}^{gt})}{c^2}$, where ρ is the Euclidean distance between the box centers $\mathbf{b}$ and $\mathbf{b}^{gt}$, and c is the diagonal length of C. It converges faster than GIoU because it pulls the centers together even when one box contains the other, where the GIoU penalty vanishes.
- **CIoU** [12] adds an aspect-ratio term: $\text{CIoU} = \text{DIoU} - \alpha v$, with $v = \frac{4}{\pi^2}\left(\arctan\frac{w^{gt}}{h^{gt}} - \arctan\frac{w}{h}\right)^2$ measuring aspect-ratio mismatch and $\alpha = \frac{v}{(1-\text{IoU})+v}$ a trade-off weight.

The corresponding losses are $1 - \text{GIoU}$, $1 - \text{DIoU}$ and $1 - \text{CIoU}$.

```
import numpy as np

def iou_family(a, b, eps=1e-9):
    """a, b: (N, 4) boxes as (x1, y1, x2, y2). Returns IoU, GIoU, DIoU, CIoU."""
    ix1, iy1 = np.maximum(a[:, 0], b[:, 0]), np.maximum(a[:, 1], b[:, 1])
    ix2, iy2 = np.minimum(a[:, 2], b[:, 2]), np.minimum(a[:, 3], b[:, 3])
    inter = np.clip(ix2 - ix1, 0, None) * np.clip(iy2 - iy1, 0, None)
    wa, ha = a[:, 2] - a[:, 0], a[:, 3] - a[:, 1]
    wb, hb = b[:, 2] - b[:, 0], b[:, 3] - b[:, 1]
    union = wa * ha + wb * hb - inter
    iou = inter / (union + eps)
    # smallest enclosing box C
    cx1, cy1 = np.minimum(a[:, 0], b[:, 0]), np.minimum(a[:, 1], b[:, 1])
    cx2, cy2 = np.maximum(a[:, 2], b[:, 2]), np.maximum(a[:, 3], b[:, 3])
    area_c = (cx2 - cx1) * (cy2 - cy1)
    giou = iou - (area_c - union) / (area_c + eps)
    # center distance over enclosing-box diagonal
    rho2 = ((a[:, 0] + a[:, 2]) - (b[:, 0] + b[:, 2])) ** 2 / 4 \
         + ((a[:, 1] + a[:, 3]) - (b[:, 1] + b[:, 3])) ** 2 / 4
```

```
    c2 = (cx2 - cx1) ** 2 + (cy2 - cy1) ** 2
    diou = iou - rho2 / (c2 + eps)
    v = 4 / np.pi**2 * (np.arctan(wb / hb) - np.arctan(wa / ha)) ** 2
    alpha = v / (1 - iou + v + eps)
    ciou = diou - alpha * v
    return iou, giou, diou, ciou

gt = np.array([[10, 10, 50, 50]] * 3, float)
pred = np.array([[20, 20, 60, 60],    # overlapping, shifted
                 [60, 10, 100, 50],   # disjoint, nearby
                 [200, 10, 240, 50]], float)  # disjoint, far away
for name, v in zip(["IoU", "GIoU", "DIoU", "CIoU"], iou_family(pred, gt)):
    print(f"{name:5s}", np.round(v, 3))
```

```
IoU   [0.391 0.    0.   ]
GIoU  [ 0.311 -0.111 -0.652]
DIoU  [ 0.351 -0.258 -0.662]
CIoU  [ 0.351 -0.258 -0.662]
```

The two disjoint predictions both have IoU 0, but GIoU and DIoU correctly say that the far one is worse. CIoU equals DIoU here because all boxes are squares, so there is no aspect-ratio mismatch.

Anchors and label assignment

Plain version. Instead of searching freely, a dense detector places a fixed set of reference boxes, **anchors**, at every cell of a feature map, and each anchor asks: "is there an object roughly like me here, and how should I move to fit it?"

At a feature map with stride s (each cell covers $s \times s$ input pixels), anchors are centered at $((i+\frac{1}{2})s, (j+\frac{1}{2})s)$ with a few scales σ and aspect ratios $r = h/w$, giving widths $w = \sigma/\sqrt{r}$ and heights $h = \sigma\sqrt{r}$ so that the area stays σ^2. Faster R-CNN uses 3 scales × 3 ratios = 9 anchors per location [10] (Figure 3, left).

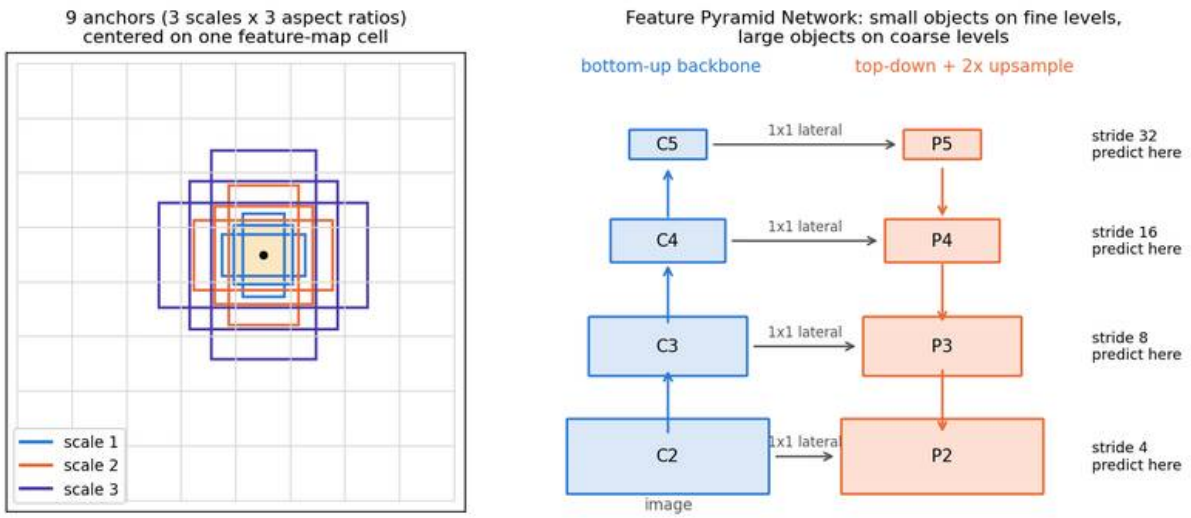


Figure 3 — Left: nine anchors (3 scales × 3 aspect ratios) at one feature-map cell. Right: a Feature Pyramid Network; each level predicts objects of a matching size range.

Label assignment decides which anchors are trained as which object. The classic rule is IoU-based: in the region proposal network of Faster R-CNN an anchor is *positive* if its IoU with some ground-truth box is at least 0.7 (or if it is the best anchor for that box), *negative* if its IoU with every box is below 0.3, and ignored otherwise [10]; RetinaNet uses 0.5 and 0.4 [15]. Each positive anchor regresses toward its matched box with the targets above. Assignment turned out to matter as much as architecture: anchor-free detectors (FCOS [29]) assign by *location* and object size instead, and DETR [30] replaces all fixed rules by a learned one-to-one matching.

Anchors have real costs: they introduce hyperparameters (scales, ratios, thresholds) that must be tuned per dataset, and they multiply the number of predictions. YOLO9000 [19] chose anchor shapes by running k-means on the training boxes, using $1 - \text{IoU}$ as the distance.

Non-maximum suppression (NMS) and Soft-NMS

Plain version. A dense detector fires many times on the same object (neighboring anchors all see it). NMS keeps the most confident box and deletes the others that overlap it too much.

Greedy NMS, per class: sort the boxes by score; repeatedly take the highest-scoring remaining box M, keep it, and remove every box b_i with $\text{IoU}(M, b_i) > N_t$ (typically $N_t = 0.5$ to 0.7). The weakness is crowded scenes: two real people standing close may overlap by more than N_t, and the lower-scoring one is deleted, losing recall.

Soft-NMS [13] replaces deletion with a score decay. With the Gaussian variant,

$$s_i \leftarrow s_i \exp\!\left(-\frac{\text{IoU}(M, b_i)^2}{\sigma}\right),$$

where s_i is the score of box b_i and σ controls how fast overlapping boxes are penalized. A heavy overlap still pushes the score down a lot, but a genuine neighbor survives with a reduced score

instead of disappearing.

```
import numpy as np

def iou_one_to_many(box, boxes):
    x1 = np.maximum(box[0], boxes[:, 0]); y1 = np.maximum(box[1], boxes[:, 1])
    x2 = np.minimum(box[2], boxes[:, 2]); y2 = np.minimum(box[3], boxes[:, 3])
    inter = np.clip(x2 - x1, 0, None) * np.clip(y2 - y1, 0, None)
    area = lambda b: (b[..., 2] - b[..., 0]) * (b[..., 3] - b[..., 1])
    return inter / (area(box) + area(boxes) - inter)

def nms(boxes, scores, thr=0.5):
    order = np.argsort(-scores)
    keep = []
    while order.size:
        i = order[0]
        keep.append(i)
        ious = iou_one_to_many(boxes[i], boxes[order[1:]])
        order = order[1:][ious <= thr]          # drop everything that overlaps too much
    return np.array(keep)

def soft_nms(boxes, scores, sigma=0.5, score_thr=0.001):
    scores = scores.copy()
    idx = np.arange(len(scores))
    keep = []
    while idx.size:
        j = np.argmax(scores[idx]); i = idx[j]
        keep.append(i)
        idx = np.delete(idx, j)
        ious = iou_one_to_many(boxes[i], boxes[idx])
        scores[idx] *= np.exp(-ious**2 / sigma)  # decay instead of delete
        idx = idx[scores[idx] > score_thr]
    return np.array(keep), scores

# two people standing close together + duplicates
boxes = np.array([[10, 10, 60, 110], [14, 12, 62, 112], [8, 6, 58, 104],
                  [40, 15, 90, 115], [43, 18, 92, 118]], float)
scores = np.array([0.95, 0.90, 0.80, 0.85, 0.70])
print("greedy NMS keeps:", nms(boxes, scores, 0.5))
keep, s = soft_nms(boxes, scores)
print("Soft-NMS order:", keep, "rescored:", np.round(s[keep], 3))
```

```
greedy NMS keeps: [0 3]
Soft-NMS order: [0 3 2 4 1] rescored: [0.95  0.761 0.183 0.145 0.058]
```

Greedy NMS keeps one box per person. Soft-NMS keeps everything but ranks the duplicates far below the two real detections; a final score threshold or the AP computation then treats them as low-confidence guesses. Figure 4 shows the same effect on a synthetic scene.

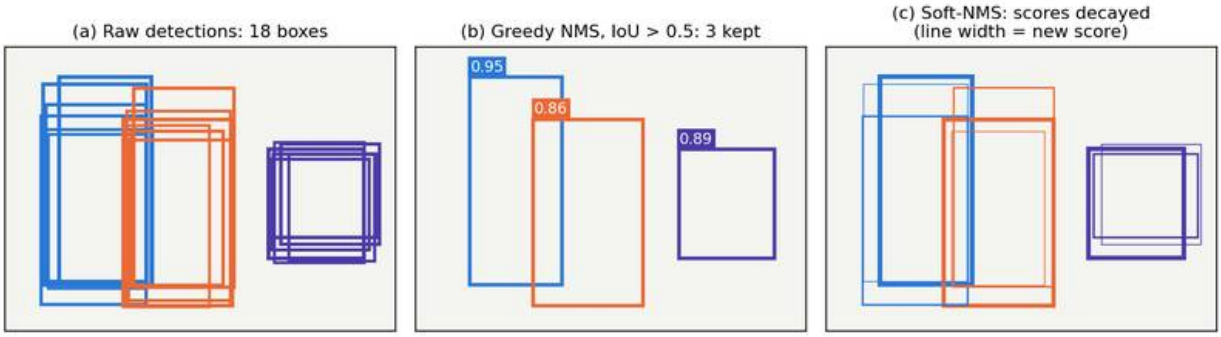


Figure 4 — NMS on synthetic detections. (a) 18 raw boxes around three objects. (b) Greedy NMS at IoU 0.5 keeps one box per object. (c) Soft-NMS keeps all boxes but decays the scores of overlapping ones (thinner line = lower score).

NMS has three further problems: it has a hand-tuned threshold; it is a sequential, data-dependent step that is awkward to run on accelerators; and it is not part of training, so the network is never taught to avoid duplicates. Removing it is one of the main motivations behind DETR and YOLOv10 [24].

Feature pyramids (FPN)

Plain version. Small objects need high-resolution feature maps; large objects need features that have seen a lot of context. A feature pyramid gives you both, at every scale.

A CNN backbone naturally produces a pyramid $\{C_2, C_3, C_4, C_5\}$ with strides $4, 8, 16, 32$, but its shallow levels are semantically weak. The **Feature Pyramid Network** [14] adds a top-down path: starting from the coarsest level, it upsamples by 2 and adds a 1×1-projected lateral connection from the backbone,

$$P_l = \mathrm{Conv}_{3\times3}\big(\mathrm{Conv}_{1\times1}(C_l) + \mathrm{Up}_{2\times}(P_{l+1})\big),$$

so every output level P_l has the same channel width and strong semantics. In a two-stage detector with FPN, a region of interest of size $w \times h$ is pooled from level $l = \lfloor l_0 + \log_2(\sqrt{wh}/224) \rfloor$ (with $l_0 = 4$ in [14]), so a region twice as large moves one level up; dense detectors use a similar size-to-level rule. Later necks (extra bottom-up paths, weighted fusion) refine the same idea; nearly every detector after 2017 uses one (Figure 3, right).

Focal loss and class imbalance

Plain version. If 99.9% of your candidates are background, a normal loss is dominated by millions of easy "nothing here" examples. The focal loss turns down the volume on examples the model already gets right.

Let $p \in [0, 1]$ be the predicted probability of the foreground class and $y \in \{0, 1\}$ the label. Define $p_t = p$ if $y = 1$ and $p_t = 1 - p$ otherwise. The **focal loss** [15] is

$$\mathrm{FL}(p_t) = -\alpha_t (1 - p_t)^{\gamma} \log(p_t),$$

where $\gamma \geq 0$ is the focusing parameter and α_t a class-balancing weight (α for foreground, $1-\alpha$ for background). With $\gamma = 0$ this is weighted cross-entropy. With the paper's default $\gamma = 2, \alpha = 0.25$, an easy negative with $p_t = 0.99$ is down-weighted by $(0.01)^2 = 10^{-4}$ relative to cross-entropy, while hard examples keep almost their full weight. The loss is normalized by the number of positive anchors. Two-stage detectors avoid the problem differently (the proposal stage removes most background, then a fixed foreground:background sampling ratio), and SSD [26] uses **hard negative mining** (keeping only the highest-loss negatives at about 3:1). The focal loss made a *dense* one-stage detector as accurate as two-stage ones for the first time.

Two-stage detectors: the R-CNN family

Plain version. First propose a few thousand places where *something* might be; then look closely at each one and decide *what* it is and exactly where its edges are.

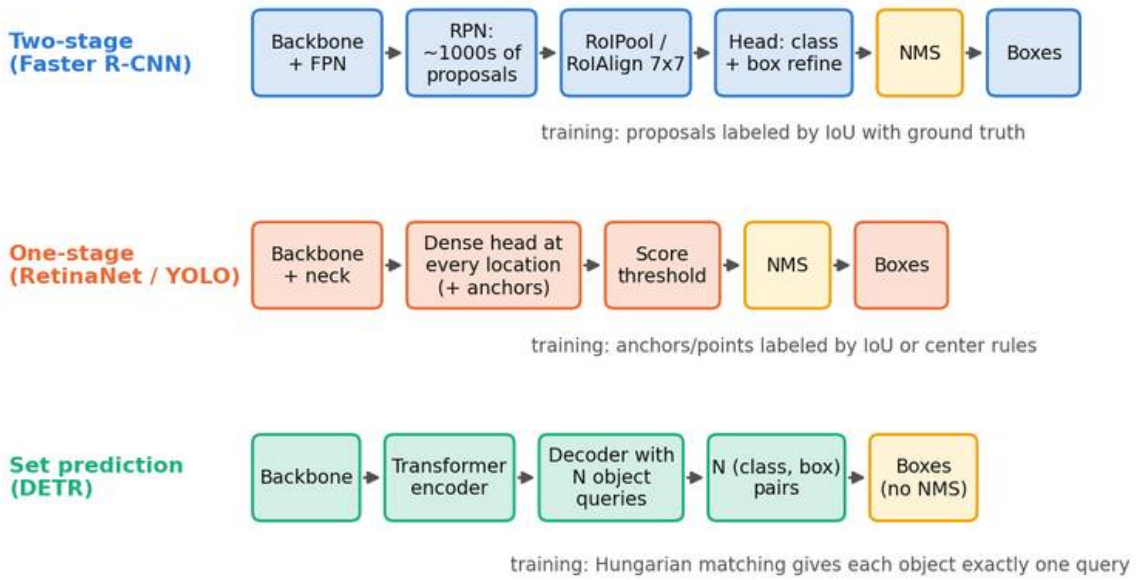


Figure 5 — Three detector pipelines. NMS (yellow) is a post-processing step for two-stage and one-stage detectors; DETR-style set prediction removes it by learning one-to-one assignments.

R-CNN (2014) [8] ran a bottom-up grouping algorithm to get about 2,000 class-agnostic region proposals per image, warped each one to a fixed size, passed it through a classification-pretrained CNN, scored the features with per-class linear SVMs and refined the box with the regression targets above. It improved mean AP on PASCAL VOC 2012 by more than 30% relative to the previous best result, and it established the recipe "pretrain on classification, fine-tune on detection" that is still standard. It was very slow, because the CNN ran separately on every proposal.

Fast R-CNN (2015) [9] runs the CNN *once* on the whole image and then crops each proposal from the shared feature map with **RoI pooling**: the proposal's projection onto the feature map is divided into a fixed $H \times W$ grid (e.g. 7×7) and each bin is max-pooled, so every proposal yields the same-size feature regardless of its shape. A single network is trained with a multi-task loss,

$$L = L_{\text{cls}}(p, u) + \lambda\,[u \geq 1]\,L_{\text{loc}}(t^u, v),$$

where p is the softmax over $K + 1$ classes, u the true class (0 for background), $[u \geq 1]$ is 1 for foreground proposals only, t^u the predicted box offsets for class u, v the target offsets and L_{loc} the smooth L1 loss.

Faster R-CNN (2015) [10] made the proposals themselves learned. A small **Region Proposal Network** (RPN) slides over the shared feature map and, for each of the 9 anchors per location, predicts an objectness score and box offsets. The top proposals (after NMS) feed the Fast R-CNN head. Because both stages share the backbone, proposals become almost free, and the whole detector is one network trained end to end.

Mask R-CNN (2017) [16] adds a small fully convolutional branch that predicts a binary mask per class inside each RoI, giving instance segmentation. Its key detail is **RoIAlign**: RoI pooling rounds the proposal boundaries and bin edges to the feature grid, which shifts features by up to half a stride. RoIAlign keeps real-valued coordinates and samples each bin with bilinear interpolation. This sounds minor, but pixel-accurate masks need it, and box accuracy at high IoU improves too.

Cascade R-CNN (2018) [17] addresses a subtle problem. A head trained with positives at IoU ≥ 0.5 learns to produce boxes that are "good at 0.5" but not at 0.75. Simply raising the threshold leaves too few positives and overfits. Cascade R-CNN chains several heads trained at increasing IoU thresholds (0.5, 0.6, 0.7), each refining the boxes of the previous one, so each stage receives a training distribution of higher quality. It was a large gain on the strict COCO metrics.

Two-stage detectors are accurate, flexible (the RoI head can predict masks, keypoints or attributes) and still the default for many instance-segmentation tasks, but the per-RoI head and NMS cost time.

One-stage detectors: YOLO, SSD and RetinaNet

Plain version. Skip the proposal step. Make one dense prediction at every location of the feature map in a single forward pass, then clean it up with NMS.

YOLO v1 (2016) [18] divided the image into an $S \times S$ grid ($S = 7$). Each cell predicted $B = 2$ boxes, each with a confidence, plus one set of C class probabilities, so the output was a single $S \times S \times (5B + C)$ tensor ($7 \times 7 \times 30$ for PASCAL VOC's 20 classes). The cell containing an object's center was responsible for it. It ran in real time and, because it saw the whole image, made fewer background errors than Fast R-CNN, but it localized less precisely and struggled with small objects and groups, since each cell could only predict a fixed number of boxes and one class.

The **YOLO family** then evolved quickly. The table lists only versions with a paper we could verify; many widely used releases are maintained as software packages without papers and are not covered here.

Version	Year	Main ideas (from the paper)
YOLO [18]	2016	Single-pass grid regression; real time
YOLO9000 / v2 [19]	2016 (arXiv)	Anchors chosen by k-means with an IoU distance; batch norm; higher-resolution training
YOLOv3 [20]	2018 (tech report)	Deeper backbone; predictions at three scales; independent logistic class scores
YOLOv4 [21]	2020 (arXiv)	Systematic study of training tricks ("bag of freebies") and cheap architectural add-ons ("bag of specials")
YOLOv7 [22]	2023 (CVPR)	Trainable bag-of-freebies: re-parameterized modules, improved label assignment, efficient layer aggregation
YOLOv9 [23]	2024 (ECCV)	Programmable gradient information (PGI) to keep information through deep layers; GELAN backbone
YOLOv10 [24]	2024 (NeurIPS)	NMS-free training with consistent dual (one-to-many and one-to-one) assignments; efficiency-driven design
YOLOv12 [25]	2025 (arXiv)	Attention-centric design that keeps real-time speed

A recurring theme is visible: the core "dense grid + NMS" idea stayed, while label assignment, necks, training recipes and, most recently, attention and NMS-free heads changed.

SSD (2016) [26] attached small convolutional predictors to *several* feature maps of different resolutions, each with its own set of default boxes (anchors) of several aspect ratios, which handled scale far better than YOLO v1. It trained with hard negative mining and heavy data augmentation.

RetinaNet (2017) [15] combined an FPN backbone, dense anchors (9 per location) and two small subnetworks for classification and box regression, trained with the focal loss. The paper's analysis showed that class imbalance, not the architecture, had been the main reason one-stage detectors lagged behind two-stage ones.

Anchor-free detectors: points instead of boxes

Plain version. Anchors are guesses about what objects look like. Anchor-free detectors drop them and predict boxes from *points*: corners, centers, or every pixel inside the object.

CornerNet (2018) [27] detects an object as a pair of keypoints: a heatmap for top-left corners, a heatmap for bottom-right corners, and an **embedding** per corner. Corners from the same object are trained to have similar embeddings, so pairing is done by embedding distance. Because a corner often lies outside the object, the paper introduced **corner pooling**, which looks along rows and columns to collect evidence for a corner.

CenterNet, "Objects as Points" (2019) [28] represents each object by its center. The network

outputs a heatmap $\hat{Y} \in [0,1]^{\frac{W}{R}\times\frac{H}{R}\times K}$ (output stride R, one channel per class), plus a size (w,h) and a sub-pixel offset at each location. Ground-truth centers are splatted with a Gaussian, and the heatmap is trained with a pixel-wise focal loss. At inference, peaks are simply local maxima, found with a 3×3 max-pooling comparison, so **no IoU-based NMS** is needed. The same design extends to 3D boxes and human pose by regressing more quantities at the center.

FCOS (2019) [29] treats detection like segmentation. Every feature-map location that falls inside a ground-truth box is a positive sample and regresses the four distances to the box sides,

$$l^* = x - x_1, \qquad t^* = y - y_1, \qquad r^* = x_2 - x, \qquad b^* = y_2 - y,$$

where (x,y) is the location in image coordinates and (x_1,y_1,x_2,y_2) the box. Each FPN level only handles objects whose largest regression distance falls in its own range, which resolves most ambiguity when boxes overlap. Locations far from the object's center produce poor boxes, so FCOS adds a **centerness** branch,

$$\text{centerness} = \sqrt{\frac{\min(l^*,r^*)}{\max(l^*,r^*)}\cdot\frac{\min(t^*,b^*)}{\max(t^*,b^*)}},$$

that is 1 at the center and decays toward the edges; it multiplies the class score at test time so off-center boxes rank lower before NMS. FCOS removed every anchor hyperparameter and matched or beat anchor-based RetinaNet.

Set prediction with transformers: DETR and its descendants

Plain version. Instead of predicting thousands of candidates and deleting duplicates, predict a small *set* of answers directly, and train so that each real object is claimed by exactly one prediction.

DETR (2020) [30] uses a CNN backbone, a transformer encoder over the feature map, and a transformer decoder that takes N learned **object queries** (100 in the paper) and outputs N (class, box) predictions in parallel. There are no anchors, no proposals and no NMS. Since N exceeds the number of objects, a special "no object" class $\emptyset$ pads the ground truth to size N.

The key is training with **bipartite matching**. Let $y = \{y_j\}$ be the padded ground truth and $\hat{y} = \{\hat{y}_i\}_{i=1}^{N}$ the predictions. Find the permutation $\hat{\sigma}$ of N elements with lowest total matching cost,

$$\hat{\sigma} = \arg\min_{\sigma\in\mathfrak{S}_N}\sum_{j=1}^{N}\mathcal{L}_{\text{match}}(y_j,\hat{y}_{\sigma(j)}), \qquad \mathcal{L}_{\text{match}} = -\mathbb{1}_{\{c_j\neq\emptyset\}}\,\hat{p}_{\sigma(j)}(c_j) + \mathbb{1}_{\{c_j\neq\emptyset\}}\,\mathcal{L}_{\text{box}}(b_j,\hat{b}_{\sigma(j)}),$$

where $\mathfrak{S}_N$ is the set of permutations, c_j and b_j are the class and box of target j, $\hat{p}_{\sigma(j)}(c_j)$ is the predicted probability of class c_j for the matched query, and

$\mathcal{L}_{\text{box}} = \lambda_{L1}\|b_j - \hat{b}_{\sigma(j)}\|_1 + \lambda_{\text{iou}}(1 - \text{GIoU}(b_j,\hat{b}_{\sigma(j)}))$. This assignment problem is solved exactly by the **Hungarian algorithm** [31] in polynomial time. The training loss (the **set loss**) then uses the matched pairs:

$$\mathcal{L}_{\text{Hungarian}} = \sum_{j=1}^{N}\Big[-\log\hat{p}_{\hat{\sigma}(j)}(c_j) + \mathbb{1}_{\{c_j\neq\emptyset\}}\,\mathcal{L}_{\text{box}}(b_j,\hat{b}_{\hat{\sigma}(j)})\Big],$$

with the log-probability term down-weighted by a factor of 10 for $c_j = \emptyset$ to balance the many "no object" slots. Because each object is matched to exactly one query, a duplicate prediction is *penalized* as a false "no object", and the decoder's self-attention learns to suppress duplicates, which is why NMS is unnecessary.

The matching step is short with SciPy. The cost below combines the three terms with DETR's weights; `linear_sum_assignment` accepts the rectangular $N \times M$ matrix directly (DETR computes the L1 term on normalized (c_x, c_y, w, h); corner format is used here for brevity).

```
import numpy as np
from scipy.optimize import linear_sum_assignment

def giou_matrix(a, b):
    x1 = np.maximum(a[:, None, 0], b[None, :, 0]); y1 = np.maximum(a[:, None, 1], b[None, :,
     ↪  1])
    x2 = np.minimum(a[:, None, 2], b[None, :, 2]); y2 = np.minimum(a[:, None, 3], b[None, :,
     ↪  3])
    inter = np.clip(x2 - x1, 0, None) * np.clip(y2 - y1, 0, None)
    area = lambda b: (b[:, 2] - b[:, 0]) * (b[:, 3] - b[:, 1])
    union = area(a)[:, None] + area(b)[None, :] - inter
    cx1 = np.minimum(a[:, None, 0], b[None, :, 0]); cy1 = np.minimum(a[:, None, 1], b[None,
     ↪  :, 1])
    cx2 = np.maximum(a[:, None, 2], b[None, :, 2]); cy2 = np.maximum(a[:, None, 3], b[None,
     ↪  :, 3])
    c = (cx2 - cx1) * (cy2 - cy1)
    return inter / union - (c - union) / c

rng = np.random.default_rng(1)
N, C = 6, 3                                      # 6 queries, 3 real classes (+ "no object")
logits = rng.normal(size=(N, C + 1))
prob = np.exp(logits) / np.exp(logits).sum(1, keepdims=True)
pred_boxes = np.array([[.10, .10, .40, .50], [.55, .20, .90, .60], [.12, .08, .42, .48],
                       [.30, .60, .60, .95], [.00, .00, .20, .20], [.50, .50, .70, .70]])
gt_boxes  = np.array([[.10, .10, .40, .50], [.32, .62, .62, .93], [.56, .22, .88, .62]])
gt_labels = np.array([0, 2, 1])

w_cls, w_l1, w_giou = 1.0, 5.0, 2.0              # weights used by DETR
cost = (-w_cls * prob[:, gt_labels]                          # (N, M) class term
        + w_l1 * np.abs(pred_boxes[:, None] - gt_boxes[None]).sum(-1)
        - w_giou * giou_matrix(pred_boxes, gt_boxes))
rows, cols = linear_sum_assignment(cost)         # works on rectangular N x M matrices
print("query -> gt:", dict(zip(rows.tolist(), cols.tolist())))
print("unmatched queries (trained as 'no object'):", sorted(set(range(N)) -
 ↪  set(rows.tolist())))
```

```
query -> gt: {0: 0, 1: 2, 3: 1}
unmatched queries (trained as 'no object'): [2, 4, 5]
```

Query 2 is a near-duplicate of query 0, yet it is left unmatched and will be trained toward "no object". That is the mechanism that replaces NMS.

The price. DETR needed very long training schedules (hundreds of epochs, compared with roughly a dozen for Faster R-CNN) and was weak on small objects, because global attention over a high-resolution feature map is expensive, so it used only a single coarse level. The descendants fixed these problems one by one.

- **Deformable DETR (2021)** [32] replaces dense attention with **deformable attention**: each query attends to only a few learned sampling points around a reference point, across multiple feature levels. This makes multi-scale features affordable and, according to the paper, reaches better accuracy than DETR (especially on small objects) with 10 times fewer training epochs.
- **DINO (2022)** [33] (not to be confused with the self-supervised method of the same name) combines *contrastive denoising training* (noised copies of ground-truth boxes are fed as extra queries and must be reconstructed, while slightly more noised ones must be rejected), *mixed query selection* (initializing anchor positions from encoder features) and a *look-forward-twice* box update. It reports 49.4 AP after 12 epochs and 51.3 AP after 24 epochs on COCO with a standard 50-layer CNN backbone, and 63.2 AP on COCO val2017 with a large backbone and Objects365 pretraining.
- **RT-DETR (2024)** [34] targets real-time use. An *efficient hybrid encoder* applies attention only to the coarsest level and fuses scales with convolutions, and *IoU-aware query selection* picks better initial queries. The paper reports 53.1 / 54.3 AP on COCO at 108 / 74 FPS on a T4 GPU for its smaller and larger variants, faster and more accurate than YOLO models of comparable size at the time, without NMS.

Today the DETR line and the YOLO line are converging: YOLOv10 borrows one-to-one assignment to drop NMS, and real-time DETRs borrow efficient convolutional necks.

Open-vocabulary and grounding detectors

Plain version. A closed-set detector can only find the 80 or 365 classes it was trained on. An open-vocabulary detector takes the class list *as text at test time*, so you can ask for "red fire hydrant" or "forklift" without retraining.

The trick is to replace the last classification layer, a fixed matrix of K class weight vectors, with **text embeddings**. For a region feature $\mathbf{z}_i$ and the text embedding $\mathbf{e}_k$ of a prompt such as "a photo of a forklift", the class score is a scaled cosine similarity,

$$s_{ik} = \tau \cdot \frac{\mathbf{z}_i^\top \mathbf{e}_k}{\|\mathbf{z}_i\| \, \|\mathbf{e}_k\|},$$

where τ is a learned temperature. Any phrase you can embed becomes a class. The vision side must learn to align regions with language, which requires image–text data far larger than any box-annotated dataset.

- **OWL-ViT (2022)** [35] starts from a transformer image encoder and a text encoder pretrained contrastively on image–text pairs, removes the final pooling, and attaches light box and class heads to every output token. The class head is the text embedding above. It supports zero-shot

(text query) and one-shot (image-exemplar query) detection, and the paper shows consistent gains with model scale.

- **Grounding DINO (2024)** [36] fuses language deeply into a DINO detector: a feature enhancer with cross-attention between image and text, language-guided query selection, and a cross-modality decoder. It is pretrained on detection, grounding (phrase-to-region) and caption data, and reports 52.5 AP on COCO zero-shot transfer (with no COCO training data) and a 26.1 mean AP on a benchmark collection of diverse real-world detection datasets.
- **YOLO-World (2024)** [37] brings open vocabulary to real-time detectors with a vision–language path aggregation network (RepVL-PAN) and region–text contrastive pretraining. Its *prompt-then-detect* mode encodes the user's vocabulary once offline, so inference costs about the same as a closed-set YOLO. It reports 35.4 AP on LVIS at 52.0 FPS on a V100 GPU.

Promptable foundation models go one step further. SAM 3 [38] (2025) accepts a short noun phrase or image exemplars as a *concept prompt* and returns masks with identities for **all** matching instances in images and videos; the authors built a data engine producing a dataset with 4M unique concept labels and report doubling the accuracy of existing systems on their promptable concept segmentation benchmark. In such systems, detection becomes one output of a general "find this concept" model rather than a separately trained network.

Small objects and dense scenes

Plain version. Tiny objects have very few pixels, and stride-32 features may squeeze them into less than one cell. Crowds are hard because objects hide each other and NMS confuses neighbors with duplicates.

Small objects. COCO defines "small" as area below 32^2 pixels [40]. After a stride-16 backbone, such an object covers at most about 2×2 feature cells, and its appearance is mostly lost in downsampling. IoU also becomes harsh: a 2-pixel error on a 10×10 box can drop IoU below 0.7. Common remedies are high-resolution feature levels (P_2 with stride 4), feature pyramids, image tiling or slicing at inference, larger input resolutions, copy-paste and scale augmentation, and assignment rules that do not starve small boxes of positive samples. The survey and benchmark of Cheng et al. [39] reviews these strategies and introduces two large small-object benchmarks: SODA-D (24,828 traffic images, 278,433 instances, 9 categories) and SODA-A (2,513 high-resolution aerial images, 872,069 instances, 9 classes).

Dense and occluded scenes. In crowds, true objects overlap heavily, so a fixed NMS threshold trades recall against duplicates. Soft-NMS [13] helps; set-prediction detectors avoid the threshold altogether by learning which predictions are duplicates. Occlusion also makes box annotation ambiguous (full extent or visible part?), and evaluation then depends on the protocol. These issues return in tracking, where objects disappear and reappear.

Evaluating detectors

Plain version. Sort all detections by confidence. Walk down the list: each detection is either a hit (it matches an unclaimed real object well enough) or a false alarm. Track how precision and recall change; the area under that curve is the Average Precision.

Precision, recall and AP

For one class and one IoU threshold θ, sort all detections in the test set by descending score. A detection is a **true positive** (TP) if its IoU with a not-yet-matched ground-truth box of the same class is at least θ; otherwise it is a **false positive** (FP). Each ground-truth box can be matched only once, which is how duplicates are punished. After the top k detections,

$$\text{precision}(k) = \frac{\text{TP}(k)}{k}, \qquad \text{recall}(k) = \frac{\text{TP}(k)}{M},$$

where M is the number of ground-truth objects of that class. To make the curve monotone, precision is **interpolated** with its upper envelope, $p_{\text{interp}}(r) = \max_{r' \geq r} p(r')$. Then:

- **VOC 11-point AP** (PASCAL VOC up to 2009 [3]): $\text{AP} = \frac{1}{11} \sum_{r \in \{0, 0.1, \ldots, 1\}} p_{\text{interp}}(r)$.
- **VOC all-point AP** (VOC 2010 onwards [41]): the exact area under the interpolated curve, $\text{AP} = \sum_n (r_{n+1} - r_n)\, p_{\text{interp}}(r_{n+1})$ over the distinct recall values.
- **COCO AP** [40]: the mean of p_{interp} at 101 recall points $0, 0.01, \ldots, 1$.

Figure 6 shows a PR curve, its envelope and the AP area.

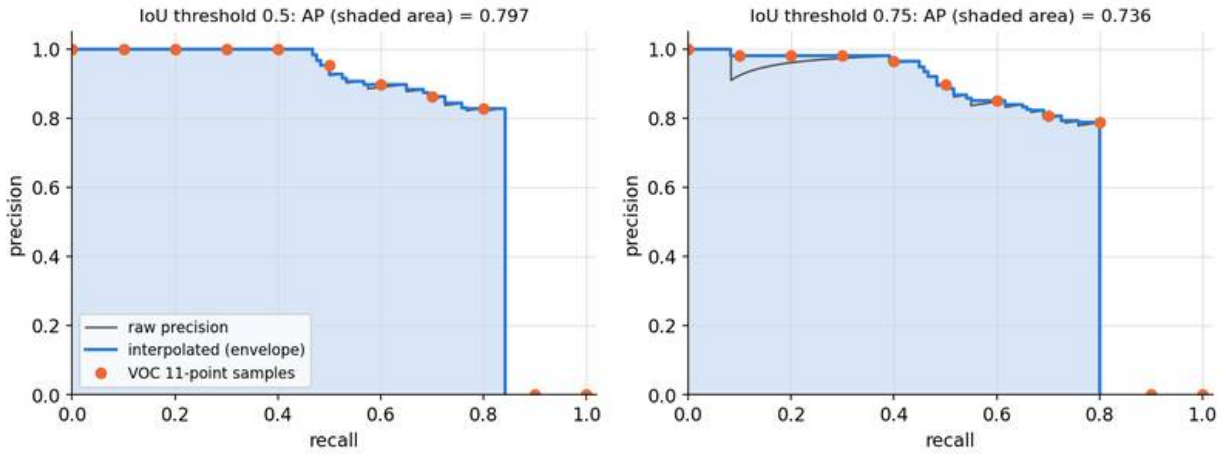


Figure 6 — Precision–recall on synthetic detections at two IoU thresholds. The shaded area under the interpolated envelope is the all-point AP; orange dots are the VOC 11-point samples. Precision drops to 0 where recall ends, because some objects were never detected.

The COCO protocol

COCO's primary metric, written **AP** or **mAP@[.5:.95]**, averages AP over 10 IoU thresholds $\theta \in \{0.50, 0.55, \dots, 0.95\}$ and over all 80 categories [40]:

$$\mathrm{AP}_{\mathrm{COCO}} = \frac{1}{10\,K} \sum_{\theta} \sum_{k=1}^{K} \mathrm{AP}_k(\theta).$$

It rewards precise localization, not just finding the object. Companion metrics are AP_{50} (the VOC-style metric), AP_{75} (strict), and AP_S, AP_M, AP_L for small (area $< 32^2$), medium (32^2 to 96^2) and large ($> 96^2$) objects. At most 100 detections per image are scored. Average recall (AR) at 1, 10 and 100 detections per image is also reported. Note that COCO calls its category-averaged metric "AP"; "mAP" in papers usually means the same thing.

The code below computes everything from scratch on synthetic predictions (jittered ground truth, about 15% misses and some random false positives).

```
import numpy as np

def box_iou(a, b):
    """Pairwise IoU between (N,4) and (M,4) xyxy boxes -> (N, M)."""
    x1 = np.maximum(a[:, None, 0], b[None, :, 0]); y1 = np.maximum(a[:, None, 1], b[None, :,
     ↪  1])
    x2 = np.minimum(a[:, None, 2], b[None, :, 2]); y2 = np.minimum(a[:, None, 3], b[None, :,
     ↪  3])
    inter = np.clip(x2 - x1, 0, None) * np.clip(y2 - y1, 0, None)
    area = lambda b: (b[:, 2] - b[:, 0]) * (b[:, 3] - b[:, 1])
    return inter / (area(a)[:, None] + area(b)[None, :] - inter)

def match(dets, gts, thr):
    """dets: list over images of (boxes, scores); gts: list of gt boxes.
    Returns scores, TP flags (sorted by score), and number of GT."""
    recs = []
    for img, ((db, ds), gb) in enumerate(zip(dets, gts)):
        for k in range(len(ds)):
            recs.append((ds[k], img, k))
    recs.sort(key=lambda r: -r[0])                      # global ranking by confidence
    used = [np.zeros(len(g), bool) for g in gts]
    tp = np.zeros(len(recs))
    for n, (s, img, k) in enumerate(recs):
        gb = gts[img]
        if len(gb) == 0:
            continue
        ious = box_iou(dets[img][0][k:k + 1], gb)[0]
        ious[used[img]] = -1                            # each GT can be matched once
        j = ious.argmax()
        if ious[j] >= thr:
            tp[n] = 1; used[img][j] = True
    return tp, sum(len(g) for g in gts)

def pr_curve(tp, n_gt):
    ctp, cfp = np.cumsum(tp), np.cumsum(1 - tp)
```

```
    return ctp / n_gt, ctp / (ctp + cfp)                # recall, precision

def ap(rec, prec, mode="all"):
    r = np.concatenate([[0], rec, [1]]); p = np.concatenate([[0], prec, [0]])
    p = np.maximum.accumulate(p[::-1])[::-1]            # monotone envelope
    if mode == "all":                                   # VOC2010+: exact area
        i = np.where(r[1:] != r[:-1])[0]
        return np.sum((r[i + 1] - r[i]) * p[i + 1])
    pts = np.linspace(0, 1, 11 if mode == "voc11" else 101)
    return np.mean([p[r >= t].max() if np.any(r >= t) else 0 for t in pts])

# synthetic data: 50 images, 1-4 objects each; detector = jittered GT + false positives
rng = np.random.default_rng(0)
gts, dets = [], []
for _ in range(50):
    n = rng.integers(1, 5)
    xy = rng.uniform(0, 200, (n, 2)); wh = rng.uniform(20, 80, (n, 2))
    g = np.hstack([xy, xy + wh]); gts.append(g)
    keep = rng.random(n) > 0.15                         # miss ~15% of objects
    jit = rng.normal(0, 0.06, (n, 4)) * np.hstack([wh, wh])
    db = (g + jit)[keep]; ds = rng.uniform(0.4, 1.0, keep.sum())
    m = rng.integers(0, 3)                              # 0-2 false positives
    fxy = rng.uniform(0, 200, (m, 2)); fwh = rng.uniform(20, 80, (m, 2))
    db = np.vstack([db, np.hstack([fxy, fxy + fwh])]); ds = np.concatenate([ds,
     ↪  rng.uniform(0, 0.7, m)])
    dets.append((db, ds))

for thr in (0.5, 0.75):
    tp, n_gt = match(dets, gts, thr)
    rec, prec = pr_curve(tp, n_gt)
    print(f"IoU {thr}: VOC11 {ap(rec, prec, 'voc11'):.3f}  all-point {ap(rec, prec):.3f}"
          f"  COCO-101 {ap(rec, prec, 'coco'):.3f}")
aps = [ap(*pr_curve(*match(dets, gts, t)), "coco") for t in np.arange(0.5, 0.96, 0.05)]
print("AP@[.5:.95] =", round(float(np.mean(aps)), 3))
```

```
IoU 0.5: VOC11 0.776  all-point 0.797  COCO-101 0.797
IoU 0.75: VOC11 0.749  all-point 0.736  COCO-101 0.738
AP@[.5:.95] = 0.532
```

Three lessons are visible. The three interpolation rules give similar but *not identical* numbers, so AP values computed with different protocols must never be compared. AP@[.5:.95] is much lower than AP50 because jittered boxes fail the strict thresholds. And this is a single class; real mAP averages over classes, so rare classes count as much as frequent ones.

What AP does not tell you

AP summarizes ranking quality over the whole test set, so it does not choose an operating threshold, and it does not separate *types* of errors: localization errors, confusion with similar classes,

duplicates and background false positives all look the same. When diagnosing a model, break errors down by type and by object size, and look at the images.

Datasets and benchmarks

Plain version. Detection progress has been driven by a few public datasets; each one changed what "good" meant.

Dataset	Classes	Scale (from the papers)	Annotations	Why it matters
PASCAL VOC [3]	20	Small by today's standards	Boxes (+ some masks)	The 2005–2012 benchmark; AP50
MS COCO [4]	91 types labeled; 80 used for detection	328k images, 2.5M labeled instances	Boxes + instance masks	Cluttered everyday scenes, many small objects; AP@[.5:.95]
Objects365 [42]	365	Large-scale, high-quality boxes	Boxes	Common pretraining set for strong detectors
Open Images V4 [43]	600 boxable classes	9.2M images; 15.4M boxes	Boxes, image labels, relationships	Very large; 19.8k concepts at image level
LVIS [44]	over 1000	~2M masks in 164k images	Instance masks	Long-tailed vocabulary; rare-class evaluation

Two trends stand out. Category counts grew (20, then 80, then hundreds and over a thousand), and with them came **long-tail** problems: in LVIS many categories have only a handful of training examples, so a loss or sampler that ignores frequency fails on rare classes. And the biggest models are now pretrained on Objects365 or on image–text data before fine-tuning on COCO, which makes "trained on COCO only" and "with extra data" separate leaderboards.

Speed, accuracy and deployment

Plain version. In practice you pick a point on a curve: more accuracy usually means more computation. What counts is the latency on *your* hardware, end to end.

- **FLOPs are not latency.** FLOPs count arithmetic, but runtime also depends on memory traffic, operator support and parallelism. Depth-wise convolutions and attention can be cheap in FLOPs but slow on some accelerators.
- **Measure end to end.** Report latency including preprocessing, the network *and* post-processing. NMS cost grows with the number of candidate boxes (worst case quadratic in their number), is data-dependent, and often runs on the CPU. RT-DETR's paper [34] analyses exactly this and is one reason NMS-free designs such as RT-DETR and YOLOv10 [24] are attractive for deployment.

- **Input resolution** is the strongest knob: halving the side length cuts compute by about 4 times but hurts small objects most.
- **Quantization** (e.g. 8-bit integer weights and activations) and **re-parameterization** (training a multi-branch block, then folding it into a single convolution, as used by YOLOv7 [22]) reduce latency with small accuracy loss. Box-regression outputs and the score head are often the most sensitive to quantization, so calibrate with real data and check AP75, not just AP50.
- **Batch size 1** is the usual edge setting; throughput numbers measured with large batches do not transfer.

A tiny detector you can train

To see box regression and an IoU loss work end to end, the PyTorch script below trains a small CNN to localize one rectangle or ellipse in noisy 32×32 images. It is a single-object detector: the network directly regresses (c_x, c_y, w, h) in $[0, 1]$ through a sigmoid, trained with L1 plus a GIoU loss, as DETR does for its matched queries.

```
import time, torch, torch.nn as nn
torch.manual_seed(0)
S = 32
torch.set_num_threads(4)

def make_batch(n):
    """One white rectangle or ellipse per 32x32 noisy image; target = (cx, cy, w, h) in [0,
    ↪ 1]."""
    img = 0.1 * torch.randn(n, 1, S, S)
    wh = torch.rand(n, 2) * 0.4 + 0.15
    c = wh / 2 + torch.rand(n, 2) * (1 - wh)
    yy, xx = torch.meshgrid(torch.arange(S), torch.arange(S), indexing="ij")
    xx, yy = (xx[None] + 0.5) / S, (yy[None] + 0.5) / S
    dx = (xx - c[:, 0, None, None]) / (wh[:, 0, None, None] / 2)
    dy = (yy - c[:, 1, None, None]) / (wh[:, 1, None, None] / 2)
    rect = (dx.abs() <= 1) & (dy.abs() <= 1)
    ell = dx**2 + dy**2 <= 1
    shape = torch.where((torch.rand(n) < 0.5)[:, None, None], rect, ell)
    return img + shape[:, None].float(), torch.cat([c, wh], 1)

def xyxy(b):
    return torch.cat([b[:, :2] - b[:, 2:] / 2, b[:, :2] + b[:, 2:] / 2], 1)

def giou(p, g):
    p, g = xyxy(p), xyxy(g)
    lt, rb = torch.max(p[:, :2], g[:, :2]), torch.min(p[:, 2:], g[:, 2:])
    inter = (rb - lt).clamp(min=0).prod(1)
    area = lambda b: (b[:, 2:] - b[:, :2]).prod(1)
    union = area(p) + area(g) - inter
    c = (torch.max(p[:, 2:], g[:, 2:]) - torch.min(p[:, :2], g[:, :2])).prod(1)
    return inter / union, inter / union - (c - union) / c

net = nn.Sequential(
    nn.Conv2d(1, 16, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),   # 16
```

```
    nn.Conv2d(16, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),  # 8
    nn.Conv2d(32, 32, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),  # 4
    nn.Flatten(), nn.Linear(32 * 4 * 4, 128), nn.ReLU(),
    nn.Linear(128, 4), nn.Sigmoid())                               # (cx, cy, w, h) in [0, 1]
opt = torch.optim.Adam(net.parameters(), 2e-3)

t0 = time.time()
for step in range(500):
    x, y = make_batch(64)
    pred = net(x)
    iou, g = giou(pred, y)
    loss = (pred - y).abs().sum(1).mean() + (1 - g).mean()        # L1 + GIoU loss
    opt.zero_grad(); loss.backward(); opt.step()

with torch.no_grad():
    x, y = make_batch(1000)
    iou, _ = giou(net(x), y)
print(f"train time {time.time() - t0:.1f}s | test mean IoU {iou.mean():.3f}"
      f" | IoU>=0.5: {(iou >= 0.5).float().mean():.1%} | IoU>=0.75: {(iou >=
      ↪  0.75).float().mean():.1%}")
```

```
train time 21.1s | test mean IoU 0.885 | IoU>=0.5: 100.0% | IoU>=0.75: 97.8%
```

(Training time depends on your CPU.) Because there is exactly one object, there is no assignment and no NMS. Exercise 6 asks you to extend this to several objects, which is where every hard problem in this tutorial reappears.

Modern view

Figure 7 places the methods of this tutorial on a timeline, using the year of each cited publication.

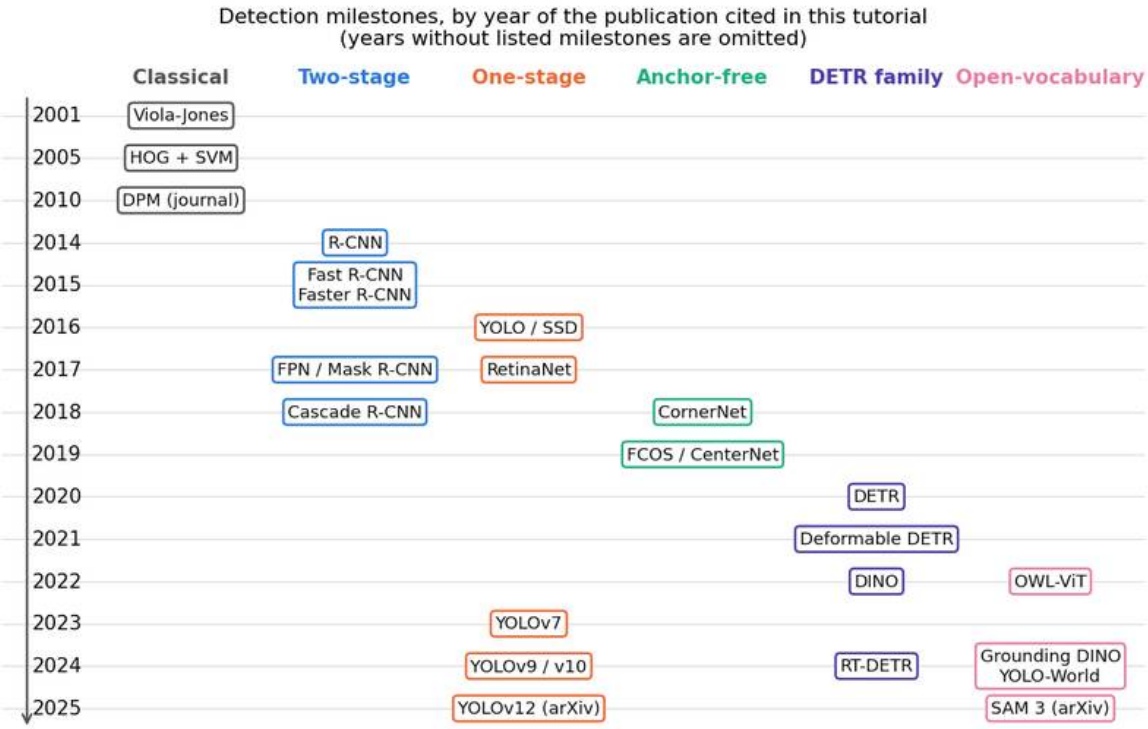


Figure 7 — Milestones by year of the publication cited in this tutorial, grouped by family. Years without a listed milestone are omitted.

What the surveys say

Object Detection in 20 Years (Zou et al., *Proceedings of the IEEE*, 2023) [1] is the broadest historical review. It covers milestone detectors from the traditional era (Viola–Jones, HOG, DPM) to deep two-stage and one-stage methods, detection datasets and metrics, fundamental building blocks (multi-scale detection, context, hard-negative mining, NMS, box regression), speed-up techniques, and recent state-of-the-art methods. Its main message is that most "new" ideas have older roots: multi-scale scanning became feature pyramids, cascades became cascaded heads, hard-negative mining became the focal loss, and NMS has been rethought several times.

Deep Learning for Generic Object Detection: A Survey (Liu et al., *IJCV*, 2020) [2] reviews over 300 contributions in depth and organizes them by detection framework, object feature representation, object proposal generation, context modeling, training strategies and evaluation metrics. It is the best single reference for the pre-transformer CNN era, and it stresses that progress came as much from training strategies, data and evaluation as from architectures.

Object Detection with Transformers: A Review (Shehzadi et al., *Sensors*, 2025) [45] analyses 21 recent improvements to DETR, grouped by changes to the backbone, query design and attention mechanism, and compares them on common benchmarks. Its taxonomy tracks the fixes described above: multi-scale and sparse attention for small objects and speed, and better query initialization

and denoising for convergence.

A Survey on Open-Vocabulary Detection and Segmentation: Past, Present, and Future (Zhu and Chen, *IEEE TPAMI*, 2024) [46] organizes open-vocabulary methods by how they use weak supervision into six families: visual–semantic space mapping, novel visual feature synthesis, region-aware training, pseudo-labeling, knowledge distillation and transfer learning. It covers 2D, 3D and video tasks, and it highlights how evaluation of "novel" classes is complicated by vocabulary overlap between pretraining and test data.

Towards Large-Scale Small Object Detection (Cheng et al., *IEEE TPAMI*, 2023) [39] focuses on the most stubborn failure mode, reviews the remedies by category and contributes the SODA benchmarks described earlier. It shows that small-object accuracy remains far below that of larger objects even for strong detectors.

The state of the art, 2023–2026

- **Real-time closed-set detection** is now a contest between the YOLO line (YOLOv9 [23], YOLOv10 [24], YOLOv12 [25]) and real-time DETRs (RT-DETR [34]). The two lines borrow from each other: one-to-one assignment and NMS-free heads in YOLO, convolutional necks and hybrid encoders in DETRs. Attention is now affordable at real-time budgets.
- **High-accuracy closed-set detection** is dominated by DETR-style models such as DINO [33] with large backbones and large-scale pretraining (Objects365 [42] and beyond). Gains on COCO have become small, and the benchmark is close to saturated for the largest models, which is pushing attention to harder benchmarks: LVIS [44], open vocabulary and cross-domain evaluation.
- **Open-vocabulary and grounding detection** (OWL-ViT [35], Grounding DINO [36], YOLO-World [37]) is moving into practice: in many applications you now *prompt* a detector rather than train one, and then optionally fine-tune it.
- **Promptable foundation models** (SAM 3 [38]) unify detection, segmentation and tracking under concept prompts.

Open problems

1. **Localization quality versus confidence.** Classification score and box quality are trained separately and are often poorly correlated, so the "best" box can be suppressed by a more confident but worse one.
2. **Small, crowded and occluded objects** remain far from solved [39].
3. **Long-tail and open-world recognition.** Rare classes and truly unseen objects, and knowing *when* the model does not know.
4. **Evaluation.** COCO AP hides error types, is saturated at the top, and open-vocabulary evaluation depends on prompts and on what the pretraining data already contained [46].
5. **Efficiency on real hardware**, including NMS-free designs, quantization-friendly heads and fair latency reporting.

Key takeaways

- Detection outputs a variable-size set of (box, class, score) triples; every architecture is a different answer to "how many, and which one is responsible for which object?"
- Boxes are regressed as scale-normalized offsets from a reference $(t_x, t_y$, log-scale $t_w, t_h)$; IoU-based losses (GIoU, DIoU, CIoU) align training with the IoU-based metric.
- Anchors and assignment rules decide which predictions learn which object; NMS removes duplicates afterwards, and Soft-NMS softens its failure in crowds.
- FPN handles scale; the focal loss handles the foreground–background imbalance that made dense detectors weak.
- Two-stage (R-CNN, Fast, Faster, Mask, Cascade) refine proposals; one-stage (YOLO, SSD, RetinaNet) predict densely in one pass; anchor-free (CornerNet, CenterNet, FCOS) predict from points; DETR predicts a set with Hungarian matching and needs no NMS.
- Open-vocabulary detectors turn the class list into text prompts, and promptable foundation models merge detection, segmentation and tracking.
- Measure with the right protocol: VOC 11-point, VOC all-point and COCO 101-point AP differ; COCO AP averages IoU 0.50–0.95, and APs/m/l expose scale weaknesses.

Exercises

1. **Encode by hand.** An anchor has center $(100, 100)$ and size 64×32. A ground-truth box has center $(110, 96)$ and size 80×40. Compute (t_x, t_y, t_w, t_h) without normalization, then decode them back.

   Hint

   $t_x = 10/64 = 0.156$, $t_y = -4/32 = -0.125$, $t_w = \log(80/64) = 0.223$, $t_h = \log(40/32) = 0.223$. Decoding: $c_x = 100 + 0.156 \cdot 64 = 110$, $w = 64e^{0.223} = 80$, and similarly for y and h.

2. **IoU versus shift.** Show that two equal $a \times a$ squares shifted by d along one axis have IoU $= (a - d)/(a + d)$. What shift gives IoU 0.5 and 0.75? Repeat for an equal shift along both axes (Figure 2, right).

   Hint

   Intersection $a(a - d)$, union $2a^2 - a(a - d) = a(a + d)$. IoU 0.5 at $d = a/3$; IoU 0.75 at $d = a/7$. For a diagonal shift, IoU $= (a - d)^2/(2a^2 - (a - d)^2)$, so IoU 0.5 at $d = a(1 - \sqrt{2/3}) \approx 0.18a$.

3. **When greedy NMS fails.** Modify the NMS example so that two different people overlap with IoU 0.6, and run greedy NMS at thresholds 0.5 and 0.7, and Soft-NMS. Which recovers both people, and what new problem does a high threshold create?

   Hint

At 0.5 the lower-scored person is deleted. At 0.7 both survive, but duplicates of the same person with IoU between 0.5 and 0.7 also survive, adding false positives. Soft-NMS keeps the second person with a reduced score ($e^{-0.36/0.5} \approx 0.49$ times its original score), which still counts toward recall in AP.

4. **Focal loss numbers.** With $\gamma = 2$ and $\alpha = 0.25$, compute the focal loss and the plain cross-entropy for a background anchor with $p = 0.01$ (easy) and $p = 0.6$ (hard). By what factor does each change?

   Hint

   For background, $p_t = 1 - p$ and $\alpha_t = 0.75$. Easy: CE $= -\log 0.99 = 0.01005$; FL $= 0.75 \cdot 0.01^2 \cdot 0.01005 \approx 7.5{\times}10^{-7}$, a factor of about $1.3{\times}10^4$ smaller. Hard: CE $= -\log 0.4 = 0.916$; FL $= 0.75 \cdot 0.6^2 \cdot 0.916 \approx 0.247$, only about 3.7 times smaller.

5. **Protocols disagree.** Using the AP code, find (by changing the random seed or the jitter) a case where VOC 11-point AP is *higher* than all-point AP, and one where it is lower. Explain why neither is always larger.

   Hint

   The 11-point rule samples the envelope at fixed recalls, so it overestimates when the envelope is high just to the right of a sample point and drops right after it, and underestimates in the opposite case. The output above already shows both: at IoU 0.5 the 11-point value is lower, at 0.75 it is higher.

6. **From one object to many.** Extend the tiny PyTorch detector to images with up to three shapes. Option A: predict an 8×8 grid of (objectness, box) like YOLO v1, assigning each object to the cell containing its center. Option B: predict 5 (class, box) slots and train with Hungarian matching like DETR. Compare the failure modes.

   Hint

   Option A fails when two centers fall in the same cell and needs NMS to remove duplicates from neighboring cells. Option B needs no NMS, but trains more slowly, and early in training the matching is unstable (the same object may be assigned to different slots from step to step). Use `scipy.optimize.linear_sum_assignment` on a detached cost matrix.

Detections are the raw material of tracking. The next step, linking boxes across video frames and keeping identities, is covered in DLCV 06 ·Object Tracking.

References

1. Z. Zou, K. Chen, Z. Shi, Y. Guo and J. Ye, "Object detection in 20 years: A survey," *Proceedings of the IEEE*, vol. 111, no. 3, 2023. doi
2. L. Liu, W. Ouyang, X. Wang, P. Fieguth, J. Chen, X. Liu and M. Pietikäinen, "Deep learning for generic object detection: A survey," *International Journal of Computer Vision*, 2020. doi

3. M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn and A. Zisserman, “The PASCAL Visual Object Classes (VOC) challenge,” *International Journal of Computer Vision*, vol. 88, 2010. doi
4. T.-Y. Lin, M. Maire, S. Belongie, L. Bourdev, R. Girshick et al., “Microsoft COCO: Common objects in context,” in *Proc. ECCV*, 2014. doi
5. P. Viola and M. Jones, “Rapid object detection using a boosted cascade of simple features,” in *Proc. IEEE CVPR*, 2001. doi
6. N. Dalal and B. Triggs, “Histograms of oriented gradients for human detection,” in *Proc. IEEE CVPR*, 2005. doi
7. P. F. Felzenszwalb, R. B. Girshick, D. McAllester and D. Ramanan, “Object detection with discriminatively trained part-based models,” *IEEE TPAMI*, 2010. doi
8. R. Girshick, J. Donahue, T. Darrell and J. Malik, “Rich feature hierarchies for accurate object detection and semantic segmentation,” in *Proc. IEEE CVPR*, 2014. arXiv
9. R. Girshick, “Fast R-CNN,” in *Proc. IEEE ICCV*, 2015. arXiv
10. S. Ren, K. He, R. Girshick and J. Sun, “Faster R-CNN: Towards real-time object detection with region proposal networks,” *IEEE TPAMI*, 2017 (conference version 2015). doi
11. H. Rezatofighi, N. Tsoi, J. Gwak, A. Sadeghian, I. Reid and S. Savarese, “Generalized intersection over union: A metric and a loss for bounding box regression,” in *Proc. IEEE/CVF CVPR*, 2019. arXiv
12. Z. Zheng, P. Wang, W. Liu, J. Li, R. Ye and D. Ren, “Distance-IoU loss: Faster and better learning for bounding box regression,” in *Proc. AAAI*, 2020. arXiv
13. N. Bodla, B. Singh, R. Chellappa and L. S. Davis, “Soft-NMS — Improving object detection with one line of code,” in *Proc. IEEE ICCV*, 2017. arXiv
14. T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan and S. Belongie, “Feature pyramid networks for object detection,” in *Proc. IEEE CVPR*, 2017. doi
15. T.-Y. Lin, P. Goyal, R. Girshick, K. He and P. Dollár, “Focal loss for dense object detection,” in *Proc. IEEE ICCV*, 2017. doi
16. K. He, G. Gkioxari, P. Dollár and R. Girshick, “Mask R-CNN,” in *Proc. IEEE ICCV*, 2017. doi
17. Z. Cai and N. Vasconcelos, “Cascade R-CNN: Delving into high quality object detection,” in *Proc. IEEE/CVF CVPR*, 2018. doi
18. J. Redmon, S. Divvala, R. Girshick and A. Farhadi, “You only look once: Unified, real-time object detection,” in *Proc. IEEE CVPR*, 2016. doi
19. J. Redmon and A. Farhadi, “YOLO9000: Better, faster, stronger,” arXiv:1612.08242, 2016. arXiv
20. J. Redmon and A. Farhadi, “YOLOv3: An incremental improvement,” tech. report, arXiv:1804.02767, 2018. arXiv
21. A. Bochkovskiy, C.-Y. Wang and H.-Y. M. Liao, “YOLOv4: Optimal speed and accuracy of object detection,” arXiv:2004.10934, 2020. arXiv
22. C.-Y. Wang, A. Bochkovskiy and H.-Y. M. Liao, “YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors,” in *Proc. IEEE/CVF CVPR*, 2023. doi
23. C.-Y. Wang, I-H. Yeh and H.-Y. M. Liao, “YOLOv9: Learning what you want to learn using programmable gradient information,” in *Proc. ECCV*, 2024. doi
24. A. Wang, H. Chen, L. Liu, K. Chen, Z. Lin et al., “YOLOv10: Real-time end-to-end object detection,” in *Proc. NeurIPS*, 2024. arXiv
25. Y. Tian, Q. Ye and D. Doermann, “YOLOv12: Attention-centric real-time object detectors,” arXiv:2502.12524, 2025. arXiv

26. W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu and A. C. Berg, "SSD: Single shot multibox detector," in *Proc. ECCV*, 2016. arXiv
27. H. Law and J. Deng, "CornerNet: Detecting objects as paired keypoints," in *Proc. ECCV*, 2018. doi
28. X. Zhou, D. Wang and P. Krähenbühl, "Objects as points," arXiv:1904.07850, 2019. arXiv
29. Z. Tian, C. Shen, H. Chen and T. He, "FCOS: Fully convolutional one-stage object detection," in *Proc. IEEE/CVF ICCV*, 2019. arXiv
30. N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov and S. Zagoruyko, "End-to-end object detection with transformers," in *Proc. ECCV*, 2020. doi
31. H. W. Kuhn, "The Hungarian method for the assignment problem," *Naval Research Logistics Quarterly*, vol. 2, 1955. doi
32. X. Zhu, W. Su, L. Lu, B. Li, X. Wang and J. Dai, "Deformable DETR: Deformable transformers for end-to-end object detection," in *Proc. ICLR*, 2021. arXiv
33. H. Zhang, F. Li, S. Liu, L. Zhang, H. Su, J. Zhu, L. M. Ni and H.-Y. Shum, "DINO: DETR with improved denoising anchor boxes for end-to-end object detection," arXiv:2203.03605, 2022. arXiv
34. Y. Zhao, W. Lv, S. Xu, J. Wei, G. Wang et al., "DETRs beat YOLOs on real-time object detection," in *Proc. IEEE/CVF CVPR*, 2024. doi
35. M. Minderer, A. Gritsenko, A. Stone, M. Neumann, D. Weissenborn et al., "Simple open-vocabulary object detection with vision transformers," in *Proc. ECCV*, 2022. arXiv
36. S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang et al., "Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection," in *Proc. ECCV*, 2024. doi
37. T. Cheng, L. Song, Y. Ge, W. Liu, X. Wang et al., "YOLO-World: Real-time open-vocabulary object detection," in *Proc. IEEE/CVF CVPR*, 2024. doi
38. N. Carion, L. Gustafson, Y.-T. Hu, S. Debnath, R. Hu et al., "SAM 3: Segment anything with concepts," arXiv:2511.16719, 2025. arXiv
39. G. Cheng, X. Yuan, X. Yao, K. Yan, Q. Zeng et al., "Towards large-scale small object detection: Survey and benchmarks," *IEEE TPAMI*, vol. 45, no. 11, 2023. arXiv
40. COCO Consortium, "COCO detection evaluation," official evaluation description (12 metrics, 101-point interpolation). link
41. M. Everingham, S. M. A. Eslami, L. Van Gool, C. K. I. Williams, J. Winn and A. Zisserman, "The PASCAL Visual Object Classes challenge: A retrospective," *International Journal of Computer Vision*, vol. 111, 2015. doi
42. S. Shao, Z. Li, T. Zhang, C. Peng et al., "Objects365: A large-scale, high-quality dataset for object detection," in *Proc. IEEE/CVF ICCV*, 2019. doi
43. A. Kuznetsova, H. Rom, N. Alldrin, J. Uijlings, I. Krasin et al., "The Open Images Dataset V4: Unified image classification, object detection, and visual relationship detection at scale," *International Journal of Computer Vision*, 2020. arXiv
44. A. Gupta, P. Dollár and R. Girshick, "LVIS: A dataset for large vocabulary instance segmentation," in *Proc. IEEE/CVF CVPR*, 2019. doi
45. T. Shehzadi, K. A. Hashmi, M. Liwicki, D. Stricker and M. Z. Afzal, "Object detection with transformers: A review," *Sensors*, vol. 25, no. 19, 2025. doi
46. C. Zhu and L. Chen, "A survey on open-vocabulary detection and segmentation: Past, present, and future," *IEEE TPAMI*, 2024. doi

DLCV 04 • Deep Image Restoration: Low-Level Vision

One degradation model, many tasks: how deep networks undo blur, noise, low resolution, haze, rain, snow and darkness, from SRCNN and DnCNN to Restormer, all-in-one models and diffusion priors.

Online edition: https://blog.shyandram.dev/en/learn/deep-learning/dl-04-image-restoration/

What you'll learn

- What "low-level vision" means, and why super-resolution, denoising, deblurring, dehazing, deraining, desnowing and low-light enhancement are really **one problem**: invert a degradation $y = \mathcal{D}(x) + n$.
- How deep restoration is set up in practice: paired data, **synthetic degradation pipelines**, and the gap to real-world (blind) degradations. You will build a pipeline and train a tiny residual denoiser in PyTorch.
- The milestones and current families for each task, from SRCNN, DnCNN, DehazeNet and the GoPro deblurring benchmark to SwinIR, Restormer, NAFNet, MambaIR and diffusion-based SR (SUPIR, OSEDiff).
- How **all-in-one** models (AirNet, TransWeather, PromptIR, AdaIR, DA-CLIP, InstructIR) handle unknown or mixed degradations with one network.
- Which losses and metrics to use, why PSNR and "looks real" disagree (the **perception–distortion trade-off**), and how NTIRE challenges track the field.

The big picture

Low-level vision is about the pixels themselves. Before a system can recognize a face, read a sign or detect a car, the image has to be clear enough to look at. Restoration models are the part of the pipeline that takes a damaged photo (too small, noisy, blurred, foggy, rainy, dark) and returns a better estimate of the clean scene.

> **Explain like I'm five**
>
> Imagine someone hands you a photo that went through the washing machine: it is smudged, speckled and faded. You have seen thousands of clean photos in your life, so you can guess what this one *used to* look like: faces have two eyes, edges of buildings are straight, the sky is smooth. A restoration network does the same thing. It has looked at millions of "before and after" pairs, so it learned what clean pictures look like and how washing-machine damage changes them, and it uses both to undo the damage.

Why it matters: every phone camera runs denoising, demosaicking and super-resolution before you ever see a picture. Satellite, microscope, medical and surveillance images are restored before they are measured. Video streaming services upscale low-bitrate frames. Self-driving cars need their detectors to work in rain and fog. And from a research point of view, restoration is where many architectural ideas were tested first: residual learning, channel attention, efficient transformers and diffusion priors all have well-known restoration variants.

About this tutorial

This tutorial is expanded from the author's earlier post Low-Level Vision Task — Image Restoration 簡介 (written in Traditional Chinese) [1]. It keeps the post's order and viewpoints, folds in its 2026 updates and corrections, and adds the math, code and survey literature.

We assume you know CNNs and backpropagation (DLCV 01) and the classical restoration chapter (IP 05), which covers noise models, Wiener filtering and the degradation model in detail. We link to it rather than repeat it.

What is low-level vision?

Plain version. High-level vision asks "what is in the picture?" (classification, detection). Low-level vision asks "what does the picture actually look like, pixel by pixel?" One is about *understanding*, the other about *perceiving*.

The analogy in the author's post [1] comes from how people interact with the world: we first *sense* (light hits the retina and we see edges, colors and contrast) and then *interpret* (that is a cup). Both are necessary. If the sensing is poor (the cup is blurry), the interpretation suffers. Low-level vision tasks exist to make the sensing good again: when we say an object looks "blurry", our perception of it is not sharp, and a deblurring task is created to make it "clear".

Precise version. A low-level vision task maps an image to an image of (usually) the same content:

$$f_\theta : \mathbb{R}^{H\times W\times C} \to \mathbb{R}^{H'\times W'\times C'},$$

where the output is a pixel-wise estimate rather than a label. H, W, C are the input height, width and channels, and H', W', C' those of the output ($H' = sH$ for super-resolution by a factor s, otherwise usually equal). This has three consequences that shape the whole field:

1. **Dense, high-resolution outputs.** Every pixel matters, so architectures cannot downsample aggressively and must keep fine detail, which makes them expensive at high resolution.
2. **Supervision is cheap to synthesize.** Take a clean image, damage it with a known process, and you have an input–target pair. This is a huge advantage over labeled data for recognition, and also the source of the field's main weakness (synthetic damage is not real damage).
3. **"Correct" is ambiguous.** Many clean images are consistent with one damaged image. Which one is "best" depends on whether you value fidelity to the truth or natural appearance, as we will see in the evaluation section.

The family of tasks is broad: deblurring, super-resolution, denoising, inpainting, color calibration, infrared–visible fusion and more. Papers with Code used to be the standard place to browse these task lists and leaderboards, but it has shut down and its address now redirects to Hugging Face's trending-papers page, so its per-task leaderboards are no longer available. A reliable way to follow progress today is the yearly **NTIRE** challenge reports (New Trends in Image Restoration and Enhancement, a CVPR workshop), which we discuss near the end [2], [3].

One problem, many faces: the degradation model

Plain version. Every task in this tutorial can be written as "the camera saw a clean scene x, something damaged it, and we got y." Only the damage changes. If you can write the damage down, you can name the task.

The unified view

$$y = \mathcal{D}(x) + n,$$

where x is the unknown clean image, $\mathcal{D}$ is a (possibly nonlinear, possibly unknown) degradation operator, n is noise, and y is what we observe. Restoration means estimating x from y. Here are the specific forms used in the post [1] and in the rest of this tutorial. Figure 1 shows each one applied to the same photo.

Blur (the textbook model). As in IP 05,

$$g(u,v) = (h * f)(u,v) + n(u,v),$$

where f is the sharp image, h the point spread function (blur kernel), $*$ 2-D convolution, n additive noise and g the observed image, all indexed by pixel coordinates (u,v). Denoising is the special case $h = \delta$ (a unit impulse, so no blur).

Super-resolution. The low-resolution image is a blurred, subsampled and noisy version of the high-resolution one:

$$y = (x * k)\downarrow_s + n,$$

where k is a blur (anti-aliasing) kernel, $\downarrow_s$ keeps every s-th pixel in each direction, and s is the scale factor ($2, 3, 4, \ldots$). The classic "bicubic" benchmark setting replaces all of this with MATLAB-style bicubic downsampling and no noise.

Compression. $y = \mathrm{JPEG}_q(x)$, where q is the quality factor. JPEG quantizes 8×8 DCT coefficients (see IP 08), producing blocking and ringing. It is not additive and not linear, so it is usually just added to the pipeline as a black box.

Haze (the atmospheric scattering model).

$$I(p) = J(p)\,t(p) + A\,(1 - t(p)), \qquad t(p) = e^{-\beta d(p)},$$

where p is a pixel, I the hazy image, J the haze-free scene radiance, A the global atmospheric light (the color of the haze), $t \in (0, 1]$ the transmission, β the scattering coefficient and d the scene depth. The first term is the attenuated scene; the second is "airlight" scattered toward the camera. Far objects (large d) get small t and fade into A.

Rain. For rain streaks the common model is additive,

$$I = B + R,$$

where B is the rain-free background and R the rain layer. Raindrops stuck on a lens or windshield behave differently: they block and refract the background, so a popular model is $I = (1 - M) \odot B + R$, where M is a binary raindrop mask, $\odot$ the element-wise product and R the light passing through the drops [39].

Low light (Retinex). Retinex theory [5] separates an image into reflectance and illumination:

$$S = R \cdot L,$$

where S is the observed image, R the reflectance (the intrinsic color and texture of surfaces, what we want), L the illumination and $\cdot$ the element-wise product. A dark photo has small L; enhancement estimates L and brightens it, while noise that was hidden in the dark gets amplified.

Figure 1 — One clean image x pushed through the degradation models of this section. Each restoration task tries to undo one (or several) of these.

Although none of these models is exact, they all have the same shape: a clean image goes in, an operator plus noise comes out. That is the post's central point [1]: seen through the degradation model, these are all **the same kind of problem**, and the same tools (and often the same networks) apply to all of them. Figure 2 organizes them by where the damage comes from.

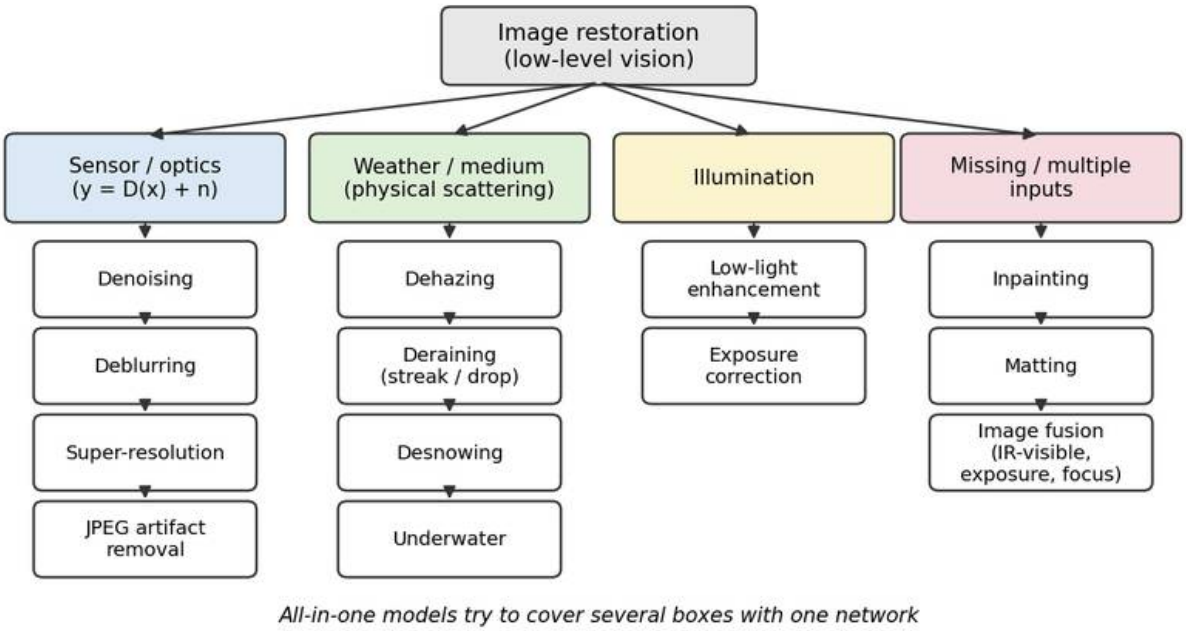


Figure 2 — A task taxonomy of low-level vision by the source of degradation. All-in-one models try to cover several boxes with a single network.

Why the problem is hard: ill-posedness

For almost every $\mathcal{D}$ above, many different x produce the same y. Downsampling by 4 throws away 15 of every 16 pixels; blur suppresses high frequencies to near zero; haze with $t \approx 0$ erases distant objects. The inverse problem is **ill-posed**, and we need a **prior** on what clean images look like. In the Bayesian (MAP) form,

$$\hat{x} = \arg\min_{x} \; \tfrac{1}{2}\|y - \mathcal{D}(x)\|_2^2 + \lambda\, \Phi(x),$$

where the first term (data fidelity) says "$\hat{x}$ must explain the observation", Φ is a regularizer encoding the prior (small Φ for natural-looking images), and $\lambda > 0$ balances the two. The whole history of the field is a history of better priors Φ.

Before deep learning: the era of hand-crafted priors

Before deep learning, each task had its own carefully reasoned prior, and success depended on having enough physical or statistical knowledge of the problem. Denoising used local smoothness, non-local self-similarity and sparsity; deblurring used Wiener and constrained least-squares filters plus heavy-tailed gradient priors; dehazing used the dark channel prior [25]; super-resolution used interpolation and example-based dictionaries; low-light enhancement used histogram equalization and Retinex decompositions. IP 05 covers the core of this toolbox

(noise models, mean/median/adaptive filters, inverse, Wiener and CLS filtering) and ends with a short history of the move to learned denoisers. The author's view in [1] is that once deep learning surpassed what these models could do, the decisive factor stopped being a strong theoretical prior for each task and became a strong **learning mechanism and feature extractor**. The degradation model did not disappear: it moved from the solver into the **data generator**.

The deep learning formulation

Plain version. Make lots of (damaged, clean) pairs, and train a network to map damaged to clean. Most of the hard work is in making the damage realistic.

Supervised training on pairs

Given a dataset of clean images $\{x_i\}$ and a degradation sampler, we build pairs (y_i, x_i) with $y_i = \mathcal{D}_i(x_i) + n_i$, and fit

$$\theta^\star = \arg\min_\theta \frac{1}{N} \sum_{i=1}^{N} \ell(f_\theta(y_i),\, x_i),$$

where f_θ is the restoration network with parameters θ, N the number of training pairs and ℓ a loss (L1, L2, perceptual, adversarial, ...; see the losses section). Training is done on random crops ("patches", e.g. 64×64 to 256×256) with flips and rotations, because convolutional and windowed-attention networks are translation-equivariant and full-resolution images do not fit in GPU memory.

A useful fact explains a lot of what follows: with the L2 loss, the optimal network outputs the **posterior mean** $f^\star(y) = \mathbb{E}[x \mid y]$. When many sharp images are consistent with y, their average is blurry. This is why MSE-trained super-resolution looks smooth, and why GANs and diffusion models were brought in.

Synthetic degradation pipelines

Pairs come from three sources. (1) **Synthetic**: degrade clean images with a known model (bicubic downsampling, Gaussian noise, Gaussian blur, JPEG). Cheap and unlimited, but limited by how realistic the model is. (2) **Captured**: photograph the same scene twice, e.g. a short and a long exposure, or a burst averaged into a clean reference (as in the SIDD real-noise dataset [22]), or averaging high-speed video frames to create real motion blur (GoPro [44]). Realistic but expensive and specific to one camera. (3) **Unpaired / self-supervised**: learn from degraded images alone, using GANs, cycle consistency or noise statistics. Most strong models today train on synthetic pairs from a rich degradation pipeline and evaluate on both synthetic and captured data.

A typical "classic" pipeline is blur → downsample → noise → JPEG, mirroring the order in which a camera and an image-sharing site would damage a photo:

```
import numpy as np, cv2, torch, torch.nn.functional as F
from skimage import data, img_as_float32

def gaussian_kernel(sigma, size=21):
    ax = torch.arange(size) - size // 2
    g = torch.exp(-ax**2 / (2 * sigma**2))
    k = torch.outer(g, g)
    return k / k.sum()

def degrade(x, scale=4, sigma_blur=1.6, sigma_noise=0.03, jpeg_q=40, rng=None):
    """x: float tensor (C,H,W) in [0,1]. Returns the low-quality image y = D(x) + n."""
    rng = rng or np.random.default_rng(0)
    C = x.shape[0]
    k = gaussian_kernel(sigma_blur).to(x)[None, None].repeat(C, 1, 1, 1)
    y = F.conv2d(F.pad(x[None], (10,) * 4, mode="reflect"), k, groups=C)   # 1) blur  h * x
    y = y[..., ::scale, ::scale]                                           # 2) downsample
     ↪  (.)↓s
    y = y + sigma_noise * torch.from_numpy(rng.standard_normal(y.shape).astype(np.float32))
     ↪  # 3) noise n
    y = y.clamp(0, 1)[0]
    img = (y.permute(1, 2, 0).numpy() * 255).round().astype(np.uint8)      # 4) JPEG round
     ↪  trip
    ok, buf = cv2.imencode(".jpg", img[..., ::-1], [cv2.IMWRITE_JPEG_QUALITY, jpeg_q])
    img = cv2.imdecode(buf, cv2.IMREAD_COLOR)[..., ::-1]
    return torch.from_numpy(img.copy()).permute(2, 0, 1).float() / 255

x = torch.from_numpy(img_as_float32(data.astronaut())).permute(2, 0, 1)    # (3, 512, 512)
y = degrade(x)
print(x.shape, "->", y.shape)        # torch.Size([3, 512, 512]) -> torch.Size([3, 128, 128])
```

In training, every parameter (kernel width and shape, noise level and type, JPEG quality, even the order) is **sampled at random per image**, so the network sees a distribution of degradations instead of a single one.

Real-world and blind degradations

A model trained on one fixed $\mathcal{D}$ (say, bicubic ×4) breaks on real photos, whose degradation is unknown, spatially varying and a mix of several effects. This is the **blind** (or real-world) restoration problem. Two broad strategies exist:

- **Make the synthetic distribution wide enough to cover reality.** Real-ESRGAN [4] applies the classic pipeline *twice* in sequence ("high-order" degradation, with randomized blur kernels, resizing, noise and JPEG in each round) and adds sinc filters to simulate ringing and overshoot. Trained on purely synthetic data, it generalizes much better to real low-quality images than bicubic-trained models.
- **Bring in a strong generative prior.** Pretrained text-to-image diffusion models already know what natural images look like. DiffBIR [16], SUPIR [17] and OSEDiff [18] condition such a

model on the degraded input, so that the prior "fills in" plausible detail. This is the main recent trend in real-world SR and is discussed below.

A third, complementary strategy is to **estimate the degradation** (noise level, kernel, degradation type) and feed it to the network, as FFDNet's noise-level map [21] and the all-in-one models do. For blind super-resolution, where the unknown is mostly the blur kernel k in $y = (x * k)\downarrow_s + n$, this line of work developed its own family of methods:

- **Condition the network on the kernel.** SRMD [87] gives a single SR network the degradation as extra input: the blur kernel, reduced to a low-dimensional vector, and the noise level are stretched into maps of the same size as the low-resolution image and concatenated with it. One network then handles many kernels and noise levels, provided someone supplies them. USRNet [91] goes further with **deep unfolding**: it unrolls a half-quadratic-splitting MAP solver into a network whose data module uses the given kernel, scale and noise level in closed form and whose prior module is a learned denoiser, so one model serves arbitrary known kernels.
- **Estimate the kernel, then correct it.** IKC [88] observes that a wrong kernel causes characteristic artifacts (too large gives over-sharpening, too small gives over-smoothing), and uses them to iteratively correct an initial kernel estimate while a kernel-conditioned SR network (SFTMD) restores the image. DAN [90] unfolds the alternation into one end-to-end network: an *estimator* predicts the kernel from both the low-resolution input and the current SR result, a *restorer* super-resolves with the current kernel, and the two alternate, the same idea as the classical deblurring loop above.
- **Estimate the kernel from the image alone.** KernelGAN [89] trains a small GAN on the single test image at test time: its generator learns to downscale the image so that the discriminator cannot tell the patches of the downscaled image from those of the original. Because patches recur across scales in natural images, the learned downscaler is an estimate of the image's own SR kernel, which is then given to a non-blind SR method.

Explicit-kernel methods are interpretable and accurate when the degradation really is "blur, downsample, noise" with a roughly uniform kernel. They are fragile when it is not (JPEG, camera processing, spatially varying blur), and errors in the estimated kernel propagate into the result. That motivated the **implicit** route: BSRGAN [92] trains on a practical degradation model with randomly shuffled blur (isotropic and anisotropic Gaussian kernels), downsampling, noise and JPEG steps, and Real-ESRGAN [4] uses its high-order pipeline; neither estimates a kernel at test time. Diffusion-prior methods [16]–[18] are implicit as well. The survey by Liu et al. [93] organizes blind SR methods by how they model the degradation and which data (external datasets or the single test image) they use, and compares them on synthetic and real images.

Super-resolution

Plain version. Make a small image bigger *and* sharper, i.e. predict the missing high-resolution pixels rather than just stretching the existing ones.

Problem setup

Single-image super-resolution (SISR) estimates $x \in \mathbb{R}^{sH \times sW \times C}$ from $y \in \mathbb{R}^{H \times W \times C}$ under $y = (x * k) \downarrow_s + n$. The traditional answer is **interpolation**: nearest-neighbor, bilinear and bicubic. Bicubic, which fits a cubic polynomial through a 4×4 neighborhood, is the best known and is still used everywhere, both as a baseline and as the standard way to create benchmark low-resolution inputs. Interpolation only redistributes existing information; it cannot recreate the high frequencies that $\downarrow_s$ removed.

How a network gets bigger: three upsampling strategies

Because the output is larger than the input, every SR network must decide *where* the resolution increases.

a. Pre-upsampling (interpolate first). SRCNN [6] bicubically enlarges the input to the target size, then applies three convolution layers (patch extraction, non-linear mapping, reconstruction). Simple and intuitive, but every layer runs at high resolution, which is slow, and the network must clean up interpolation artifacts.

b. Transposed convolution (deconvolution). FSRCNN [7] works at low resolution and enlarges only at the end with a learned transposed convolution, giving a large speed-up over SRCNN. Transposed convolutions with a stride that does not divide the kernel size overlap unevenly and tend to produce **checkerboard artifacts** [9], so they are now rare as the final SR upsampler, although they remain common elsewhere in neural networks (for example in decoders of segmentation and generative models).

c. Sub-pixel convolution (pixel shuffle). ESPCN [8] computes Cs^2 channels at low resolution and rearranges them into an image s times larger:

$$\mathrm{PS}(T)_{c,\,sh+i,\,sw+j} = T_{cs^2+si+j,\,h,\,w}, \qquad 0 \le i, j < s,$$

where T is the low-resolution feature tensor with Cs^2 channels, (h, w) a low-resolution position, (i, j) the sub-pixel offset and c the output channel. All learning happens at low resolution, and the rearrangement itself has no parameters. It is the dominant choice in modern SR networks (`torch.nn.PixelShuffle`).

Milestones

- **SRCNN** (2014/2016) [6]: the first end-to-end CNN for SR, three layers on a bicubic-upsampled input.
- **FSRCNN** (2016) [7] and **ESPCN** (2016) [8]: post-upsampling for real-time speed.
- **EDSR** (2017) [10]: a deep residual network with batch normalization removed (BN hurt the range flexibility needed for restoration), residual scaling for stable training, and a multi-scale variant. It won the NTIRE 2017 SR challenge.

- **RCAN** (2018) [11]: very deep "residual-in-residual" structure with **channel attention**, which re-weights feature channels using global statistics, so the network can focus on high-frequency information.
- **SRGAN** (2017) [12] and **ESRGAN** (2018) [13]: adversarial and perceptual (VGG feature) losses produce sharp, realistic textures instead of the blurry MSE average. ESRGAN improved the generator (Residual-in-Residual Dense Blocks without BN), used a relativistic discriminator and computed the perceptual loss on features before activation.

By around 2018 these models already produced results that looked much better than interpolation to human observers, and the field started to question how to *judge* SR at all (see the perception–distortion section).

Transformers, real-world SR and diffusion (current families)

- **Transformers.** SwinIR [14] (2021) uses residual Swin Transformer blocks (self-attention inside shifted local windows) for SR, denoising and JPEG artifact removal. HAT [15] (CVPR 2023) combines channel attention with window self-attention and an overlapping cross-attention module to "activate more pixels" for each output. Both are standard baselines.
- **State-space models.** MambaIR [49] and MambaIRv2 [50] replace attention with selective state-space (Mamba) scans that give a global receptive field at linear cost; see the backbones section.
- **Real-world (blind) SR.** Real-ESRGAN [4] and its high-order degradation pipeline, described above.
- **Diffusion priors.** DiffBIR [16] removes degradations first and then regenerates detail with a pretrained latent diffusion model; SUPIR [17] scales this up with a large diffusion backbone, a 20-million-image training set with text descriptions and negative-quality prompts; OSEDiff [18] distills the process into a **single** diffusion step that starts from the low-quality image itself, using variational score distillation to keep outputs realistic. These methods produce far more convincing textures on real photos, but they can also invent detail that was not in the scene.

Datasets and metrics

The standard training set is **DIV2K** [19] (2K-resolution images introduced for NTIRE 2017). Evaluation uses small classic test sets of natural, urban and comic images, reporting PSNR and SSIM on the luminance (Y) channel, usually after cropping s border pixels. Real-world SR is evaluated with perceptual and no-reference metrics (LPIPS, NIQE and others; see Evaluation).

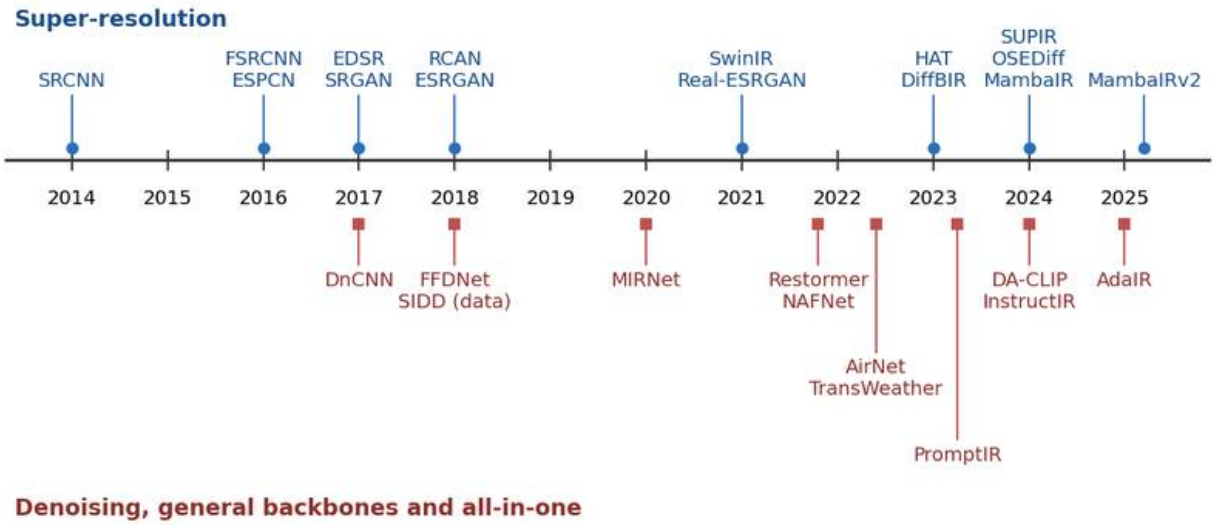


Figure 3 — Milestones in super-resolution (top) and in denoising, general backbones and all-in-one restoration (bottom), placed by publication year (positions within a year are only for legibility).

Denoising

Plain version. Remove the random grain from a photo without wiping out the real texture. Noise is the simplest degradation, so denoising became the testbed where most deep restoration ideas appeared first.

Problem setup

$y = x + n$. In the synthetic setting $n \sim \mathcal{N}(0, \sigma^2 I)$ (additive white Gaussian noise, AWGN, with standard deviation σ, usually quoted on a 0–255 scale such as $\sigma = 25$). Real camera noise is different: it is signal-dependent (photon shot noise grows with brightness), spatially correlated after demosaicking, and reshaped by the camera's processing pipeline.

Milestones

DnCNN [20] made two choices that became standard. It uses **residual learning**: instead of predicting x, the network $\mathcal{R}_\theta$ predicts the noise, and the output is

$$\hat{x} = y - \mathcal{R}_\theta(y),$$

where $\mathcal{R}_\theta(y) \approx n$. The residual (noise) is a "simpler" function to learn than the clean image, which made deep plain CNNs with batch normalization train well. A single DnCNN can also be trained for a range of noise levels (blind Gaussian denoising), and the same network handles SR and JPEG deblocking.

FFDNet [21] feeds a **noise-level map** M (same size as the image, value σ at each pixel) to the network along with a downsampled version of the input. Changing M at test time trades noise removal against detail preservation, and a non-uniform M handles spatially varying noise. It is fast enough to run on a CPU.

Real noise. The **SIDD** dataset [22] provides real smartphone noisy images with clean references obtained by careful capture and processing of image bursts. Models trained only on AWGN do poorly on it, which pushed the field toward realistic noise synthesis and captured training data.

General backbones. MIRNet [30], **Restormer** [23] and **NAFNet** [24] are general restoration architectures that reported state-of-the-art results on SIDD as well as on deblurring and other tasks. They are covered in the backbones section.

Code: a tiny residual denoiser

The snippet trains a five-layer DnCNN-style network on random 32×32 patches cut from four `skimage` images, with fresh Gaussian noise every step, then tests on an image it has never seen. On a laptop CPU it takes well under a minute (we measured about one minute on a heavily loaded machine).

```
import time, numpy as np, torch, torch.nn as nn
from skimage import data, img_as_float32
from skimage.color import rgb2gray

torch.manual_seed(0); torch.set_num_threads(2)
rng = np.random.default_rng(0)
sigma = 25 / 255                                    # noise level (on a [0,1] scale)
train_imgs = [img_as_float32(im) for im in (data.camera(), data.coins(), data.moon(),
 ↪  data.page())]
test_img = img_as_float32(rgb2gray(data.astronaut()))   # held out

def random_patches(n=32, p=32):
    batch = []
    for _ in range(n):
        im = train_imgs[rng.integers(len(train_imgs))]
        i, j = rng.integers(0, im.shape[0] - p), rng.integers(0, im.shape[1] - p)
        batch.append(im[i:i + p, j:j + p])
    return torch.from_numpy(np.stack(batch))[:, None]   # (n, 1, p, p)

class TinyDnCNN(nn.Module):
    """conv-ReLU, (depth-2) x conv-BN-ReLU, conv. Predicts the NOISE, not the image."""
    def __init__(self, depth=5, ch=32):
        super().__init__()
        layers = [nn.Conv2d(1, ch, 3, padding=1), nn.ReLU(inplace=True)]
        for _ in range(depth - 2):
            layers += [nn.Conv2d(ch, ch, 3, padding=1), nn.BatchNorm2d(ch),
             ↪  nn.ReLU(inplace=True)]
        layers += [nn.Conv2d(ch, 1, 3, padding=1)]
        self.body = nn.Sequential(*layers)
    def forward(self, y):
```

```
        return y - self.body(y)                          # x_hat = y - R(y): residual learning

net = TinyDnCNN()
opt = torch.optim.Adam(net.parameters(), lr=2e-3)
t0 = time.time()
for step in range(300):
    x = random_patches()
    y = x + sigma * torch.randn_like(x)                  # synthetic pairs, made on the fly
    loss = nn.functional.l1_loss(net(y), x)
    opt.zero_grad(); loss.backward(); opt.step()
print(f"trained in {time.time() - t0:.0f} s, last L1 = {loss.item():.4f}")

net.eval()
with torch.no_grad():
    x = torch.from_numpy(test_img)[None, None]
    y = x + sigma * torch.randn_like(x)
    x_hat = net(y).clamp(0, 1)
mse = lambda a, b: torch.mean((a - b) ** 2).item()
print(f"PSNR noisy {10*np.log10(1/mse(y.clamp(0,1), x)):.2f} dB -> denoised
 ↪  {10*np.log10(1/mse(x_hat, x)):.2f} dB")
# e.g. PSNR noisy 20.74 dB -> denoised 28.64 dB
```

Even this toy model, trained for 300 small steps on four images, gains about 8 dB on an unseen photo and beats a hand-tuned Gaussian filter (Figure 4). Drag the slider to compare the noisy input with the tiny network's output.

Noisy ($\sigma = 25$)

Tiny residual CNN

Slider comparing a grainy grayscale astronaut face with the same crop after denoising by a small CNN, which is clean but slightly smoothed.

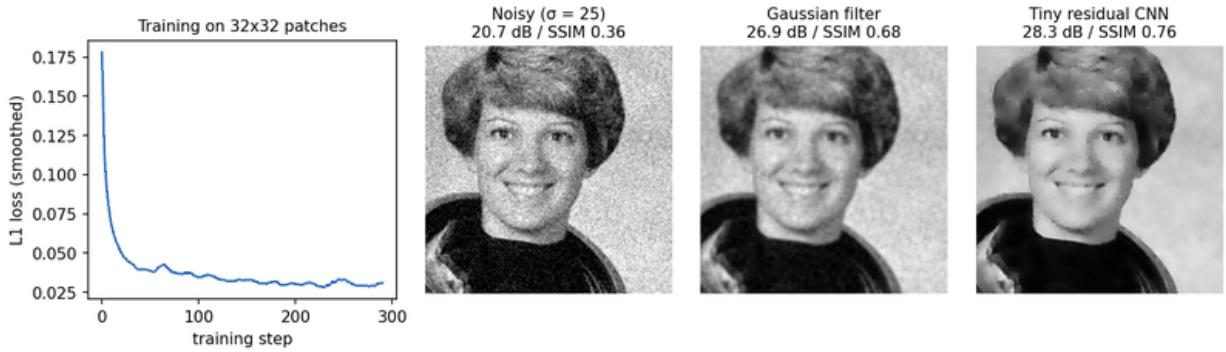


Figure 4 — The tiny residual denoiser. PSNR and SSIM are computed on the full 512 × 512 test image. The CNN removes more noise than a Gaussian filter while keeping edges sharper, but it also flattens fine texture, a typical MSE/L1 behavior. Values vary slightly between runs and machines.

Dehazing

Plain version. Remove the gray-white veil of fog or smog, restoring contrast and color, especially for distant objects. In the post's words it is an extension of denoising and deblurring, with a physical model attached.

The classical recipe and the dark channel prior

Classical methods estimate the unknowns of $I = Jt + A(1 - t)$ (the transmission map t and the airlight A), then invert the model. The hard part is finding a reliable relationship that pins those unknowns down. The most famous one is He et al.'s **dark channel prior** [25]. In haze-free outdoor images, most local patches contain some pixel that is very dark in at least one color channel (shadows, dark or colorful objects):

$$J^{\text{dark}}(p) = \min_{c\in\{r,g,b\}} \Big(\min_{q\in\Omega(p)} J^c(q) \Big) \approx 0,$$

where $\Omega(p)$ is a small patch around p and J^c is color channel c. Applying the same min–min operation to the haze model and using $J^{\text{dark}} \approx 0$ gives a transmission estimate

$$\tilde{t}(p) = 1 - \omega \min_c \Big(\min_{q\in\Omega(p)} \frac{I^c(q)}{A^c} \Big),$$

where $\omega < 1$ (0.95 in the paper) keeps a little haze for depth perception. Then $\hat{J} = (I - A)/\max(\tilde{t}, t_0) + A$, with a small t_0 to avoid dividing by zero. The prior fails on large bright regions similar to the airlight (white walls, sky), and the coarse patch-based t must be refined to follow edges.

Deep dehazing: two schools

The post [1] divides deep dehazing into two schools, and the split still describes the field well.

1. Physics-guided ("apply the formula"). Learn the unknown parameters, then invert the model. **DehazeNet** [26] uses a CNN (with Maxout units and a bounded ReLU) to regress the transmission t from hazy patches. **AOD-Net** [27] simplifies the scattering model further by folding t and A into a single function K:

$$J(p) = K(p)\, I(p) - K(p) + b, \qquad K(p) = \frac{\frac{1}{t(p)}(I(p) - A) + (A - b)}{I(p) - 1},$$

where b is a constant bias. A light-weight CNN predicts K, so the whole network is end-to-end, and it can be attached in front of a detector and trained jointly.

2. Strong general architectures ("big learner"). Skip the physics and learn $I \mapsto J$ directly with a powerful network. **GridDehazeNet** [28] uses an attention-based multi-scale grid; **FFA-Net** [29] stacks channel- and pixel-attention blocks with adaptive feature fusion. These designs transfer well to other degradations, which is exactly the route taken by general restoration networks such

as MIRNet [30]. Transformer dehazers followed: **DehazeFormer** [31] adapts the Swin Transformer to dehazing with a modified normalization layer, activation and spatial aggregation.

Datasets

- **RESIDE** [32]: a large benchmark with synthetic indoor and outdoor hazy images (generated from depth with the scattering model) plus real hazy images for task-driven evaluation. Its SOTS test set is the most quoted dehazing benchmark.
- **O-HAZE** [33] and **Dense-Haze** [34]: real hazy/haze-free pairs captured with professional haze machines, outdoor and with dense haze respectively. They are small but expose the synthetic-to-real gap.
- **KITTI** [35] is *not* a dehazing dataset. It is an autonomous-driving benchmark (stereo, optical flow, odometry, 3-D detection, tracking) recorded around Karlsruhe. Some dehazing papers synthesize fog on its images (it has depth from LiDAR and stereo) or use it to measure how dehazing affects downstream detection.

Desnowing

Plain version. Remove falling snow. It looks like a cousin of deraining, but it also behaves like dehazing.

Snow is harder than it looks. Snowflakes vary hugely in size and transparency, from opaque white blobs to semi-transparent streaks, and heavy snow also creates a **veiling effect** much like haze. So a desnowing model has to handle both the flakes and the fog-like veil. **DesnowNet** [36] introduced the **Snow100K** dataset (synthetic snow over real images, plus real snowy images) and a network that estimates both the translucency of snow and the residual it leaves. **JSTASR** [37] explicitly models snow size and transparency and adds a veiling-effect removal step. Desnowing is now usually studied inside "adverse weather" all-in-one models such as TransWeather [52].

Deraining

Plain version. Remove rain from photos. "Rain" means two very different things: thin streaks falling through the air, and drops sitting on the lens or windshield.

Rain streaks versus raindrops

- **Rain streaks** are thin, bright, oriented lines overlaid on the scene, modeled as $I = B + R$. Distant streaks accumulate into a haze-like veil. Methods learn to separate the two layers. An early influential deep method jointly detects and removes rain [38] and comes with widely used synthetic datasets of light and heavy rain.
- **Raindrops** on a glass surface act as tiny lenses: they hide and refract the background, are often out of focus, and have no consistent orientation. The attentive GAN of Qian et al. [39] uses an

attention map (where are the drops?) to guide both the generator and the discriminator, and introduced a paired raindrop dataset captured through glass with and without water drops.

Toward real scenes

Most deraining datasets are synthetic, and models trained on synthetic streaks often fail on real rain. The **NTIRE 2025 Day and Night Raindrop Removal for Dual-Focused Images** challenge [40] reflects the shift to real data: it uses the real-captured **Raindrop Clarity** dataset, covering daytime and nighttime scenes and both raindrop-focused and background-focused shots (14,139 training, 240 validation and 731 test images). The focus setting matters: when the camera focuses on the drops, the background is blurred too, so raindrop removal becomes partly deblurring.

Low-light enhancement (brief)

Plain version. Make a dark photo look properly exposed, with visible details and natural colors, without blowing up the noise.

Training compares dark images with normally exposed references of the same scene, for example the **LOL** dataset of paired low/normal-light images introduced with **RetinexNet** [41]. Classical methods rely on contrast adjustment (histogram equalization) and Retinex-based models (and, less often, an inverted atmospheric scattering model). Deep methods follow two lines: Retinex-inspired networks that decompose $S = R \cdot L$ and adjust L (RetinexNet [41]), and general well-designed networks. **Retinexformer** [42] (ICCV 2023), which its authors describe as the first Transformer-based low-light enhancement method, combines both: a one-stage Retinex formulation with illumination-guided attention. Low-light enhancement has enough of its own (zero-reference methods, curves, noise, color, video) to deserve a separate chapter: see DLCV 05.

Image fusion (brief)

Plain version. Combine several images of the same scene, each good at something different, into one image that has the best of each.

Three common settings appear in the post [1]: **infrared and visible** fusion (thermal targets from IR, textures and color from the visible camera), **multi-exposure** fusion (detail from under- and over-exposed shots, an HDR-like result) and **multi-focus** fusion (an all-in-focus image from shots focused at different depths). Unlike the other tasks, there is usually no ground-truth fused image, so deep fusion relies on unsupervised losses that keep gradients and intensities from the sources, or on GANs. The survey by Zhang et al. [43] organizes deep fusion methods by these scenarios and by learning strategy.

Deblurring

Plain version. Recover a sharp image from one smeared by camera shake, object motion or defocus.

Problem setup

Deblurring is the direct instance of $g = h * f + n$. If the kernel h is known it is **non-blind** deblurring (the setting of the classical Wiener and CLS filters in DIP 5, and of learned non-blind solvers); if h is unknown it is **blind** deblurring. Real motion blur in dynamic scenes is worse: different objects move differently and depth varies, so there is no single global h.

Blind deblurring: estimating the kernel

Plain version. If you knew exactly how the camera shook, undoing the blur would be a well-studied filtering problem. In blind deblurring you do not know the shake, so you must guess the shake *and* the sharp photo together. The trick is to guess them in turns: first sharpen with your current guess of the shake, then use the sharpest edges you got to improve the guess of the shake, and repeat.

The joint problem. For a spatially uniform blur, blind deconvolution solves

$$(\hat{f}, \hat{h}) = \arg\min_{f,h} \|h * f - g\|_2^2 + \lambda_f \, \phi(f) + \lambda_h \, \psi(h) \quad \text{subject to} \quad h \geq 0, \;\; \textstyle\sum_{u,v} h(u,v) = 1,$$

where g is the blurred image, f the unknown sharp image, h the unknown kernel (supported on a small window, e.g. 15×15 to 75×75 pixels for camera shake), ϕ an image prior (typically a sparse gradient penalty $\sum_p |\nabla f(p)|^\alpha$ with $0 < \alpha \leq 1$, because natural images have mostly flat regions and a few strong edges), ψ a kernel prior (e.g. $\|h\|_2^2$ or $\|h\|_1$, favoring compact, sparse motion paths), and $\lambda_f, \lambda_h > 0$ their weights. The constraints say a blur kernel only redistributes light: it is non-negative and preserves total brightness. They also remove one ambiguity, because $(c\,h, f/c)$ explains g as well as (h, f) for any $c > 0$. A second ambiguity cannot be removed: shifting f one way and h the other leaves $h * f$ unchanged, so the kernel is only recovered up to a translation.

Why it is badly ill-posed. Far more unknowns (f and h) than equations, and many pairs explain g exactly. Worse, the most obvious pair is wrong: $h = \delta$ (no blur) and $f = g$ fits the data perfectly. Levin et al. [74] showed that with the usual sparse gradient priors, the blurry image often has a *lower* prior cost than the sharp one, because blur shrinks gradients and ϕ rewards small gradients. So naive joint MAP estimation over (f, h) tends to return the no-blur solution. Their analysis points to a better-posed target: estimate only the kernel, integrating out the image,

$$\hat{h} = \arg\max_h \; p(h \mid g) = \arg\max_h \; p(h) \int p(g \mid f, h)\, p(f)\, df,$$

where $p(g \mid f, h)$ is the Gaussian likelihood given by the data term, $p(f)$ and $p(h)$ the priors behind ϕ and ψ, and the integral averages over all sharp images. A kernel has few unknowns compared

with the many pixel observations, so its posterior is much better determined than that of the full image. Once $\hat{h}$ is known, a non-blind deconvolution recovers f.

Classical remedies. The methods that worked in practice either approximate that marginalization or change the prior so the sharp solution wins:

- **Variational Bayes.** Fergus et al. [75] fit a heavy-tailed mixture-of-Gaussians model of image gradients and approximate the kernel posterior with variational Bayes, estimating the kernel coarse-to-fine; the image is then recovered by a standard non-blind method.
- **Edge prediction and selection.** Shan et al. [76] alternate kernel and image estimation with a natural-image gradient prior and a careful noise model to suppress ringing. Cho and Lee [77] made the alternation fast: they *predict* sharp edges from the current latent image with bilateral and shock filtering, and estimate the kernel in the gradient domain with FFTs. Xu and Jia [78] observed that thin edges narrower than the kernel can mislead estimation, so they select only informative edges in a first phase and refine the kernel's support in a second phase.
- **Priors that prefer sharp images.** Krishnan et al. [79] replace $\|\nabla f\|_1$ by the normalized sparsity $\|\nabla f\|_1 / \|\nabla f\|_2$, which is scale-invariant and *increases* with blur, so it no longer favors the blurry solution. Pan et al. [80] use the dark channel (the same quantity as in He et al.'s dehazing prior [25]): blur averages dark pixels with bright neighbors, so the dark channel becomes less sparse, and an L_0 penalty on it favors sharp images.

The standard recipe. Almost all classical blind deblurring methods follow the same loop. Starting from a delta kernel at the coarsest level of an image pyramid, alternate

$$f^{(t+1)} = \arg\min_{f} \|h^{(t)} * f - g\|_2^2 + \lambda_f \, \phi(f),$$

$$h^{(t+1)} = \arg\min_{h \geq 0,\ \sum h = 1} \|h * \nabla \tilde{f}^{(t+1)} - \nabla g\|_2^2 + \lambda_h \|h\|_2^2,$$

where t is the iteration, ∇ the image gradient, and $\tilde{f}$ the latent image after edge prediction or selection (sharpened, with weak gradients set to zero). The kernel step works in the gradient domain because convolution commutes with differentiation, $\nabla(h * f) = h * \nabla f$, and gradients are where the blur information sits; with a quadratic penalty it has a closed form in the Fourier domain, followed by projection onto the constraints. After a few iterations the kernel is upsampled to the next pyramid level (**coarse-to-fine**), which avoids poor local minima because a large blur is only a few pixels wide at coarse scales. Finally, with $\hat{h}$ fixed, a **non-blind deconvolution** produces the output image: Wiener or constrained least squares filtering as in IP 05, or a hyper-Laplacian or learned non-blind solver.

Deep learning and the kernel. Networks entered blind deblurring in three ways.

- **Predict the kernel (or a filter), then deconvolve.** Sun et al. [81] trained a CNN to classify the local motion kernel of each patch among a set of linear motions, smoothed the resulting non-uniform motion field with a Markov random field, then deconvolved. Schuler et al. [82] built a deep network with stages that mimic the classical loop (feature extraction, kernel estimation, image estimation) and trained it end-to-end on synthetic blur. Chakrabarti [83] trained a network to predict, for each patch, the Fourier coefficients of a deconvolution filter, used the result

to estimate one global kernel, and finished with standard non-blind deconvolution.

- **Self-supervised, per-image estimation.** Deep Image Prior [84] showed that the structure of an untrained generator network is itself a useful image prior. SelfDeblur [85] uses two generators, an encoder–decoder for the image and a fully connected network with a softmax output for the kernel (the softmax enforces $h \geq 0$ and $\sum h = 1$), and fits both to the single blurred image. No training data is needed, but each image takes an optimization run.
- **Kernel-free, end-to-end.** The multi-scale network of Nah et al. [44], DeblurGAN [86] (a conditional GAN with a content loss) and the general backbones Restormer and NAFNet map blurred to sharp images directly, never forming h.

Which is preferable? Kernel-based methods fit **spatially uniform** blur (camera shake of a distant, static scene), give an **interpretable** intermediate result that can be checked and reused (for example to deblur other frames from the same exposure), and generalize to new kernels as long as the physics holds; they are slower and break when the blur varies across the image or the image has few edges. Kernel-free networks handle **non-uniform** blur from object motion and depth, are fast at test time, and are the current state of the art on dynamic-scene benchmarks such as GoPro, but they generalize only as far as their training blur distribution and give no explicit kernel. The survey of Zhang et al. [69] summarizes this shift.

Code: a toy alternating blind deconvolution. The snippet blurs a 256×256 `camera` image with a curved 15×15 camera-shake kernel, then alternates a closed-form image step (a CLS-like filter with a quadratic gradient penalty) and a kernel step (least squares on the strongest 2% of shock-filtered gradients, projected onto $h \geq 0$, $\sum h = 1$). It runs in about a second on a CPU.

```
import numpy as np
from numpy.fft import fft2, ifft2
from scipy.ndimage import convolve, gaussian_filter, laplace
from skimage import data, img_as_float, transform

rng = np.random.default_rng(0)
f_true = transform.resize(img_as_float(data.camera()), (256, 256), anti_aliasing=True)
K = 15                                                  # kernel support K x K
h_true = np.zeros((K, K))
for t in np.linspace(0, 1, 40):                         # a curved "camera-shake" path
    h_true[round(3 + 8 * t), round(2 + 10 * t + 2 * np.sin(3 * t))] += 1
h_true = gaussian_filter(h_true, 0.5); h_true /= h_true.sum()
g = convolve(f_true, h_true, mode="wrap") + 0.005 * rng.standard_normal(f_true.shape)

def otf(h, shape):                                      # centered kernel -> transfer function
    p = np.zeros(shape); p[:h.shape[0], :h.shape[1]] = h
    return fft2(np.roll(p, (-(h.shape[0] // 2), -(h.shape[1] // 2)), (0, 1)))

G = fft2(g)
Dx, Dy = otf(np.array([[0, -1, 1]]), g.shape), otf(np.array([[0], [-1], [1]]), g.shape)

def image_step(h, lam):        # f = argmin ||h*f - g||^2 + lam ||grad f||^2  (closed form,
 ↪  CLS-like)
    H = otf(h, g.shape)
    return np.real(ifft2(np.conj(H) * G / (np.abs(H)**2 + lam * (np.abs(Dx)**2 +
     ↪  np.abs(Dy)**2))))
```

```
def shock(f, n=5, dt=0.5):     # edge prediction: a shock filter turns soft edges into steps
    f = gaussian_filter(f, 0.7)
    for _ in range(n):
        gy, gx = np.gradient(f)
        f = f - dt * np.sign(laplace(f)) * np.hypot(gx, gy)
    return f

def kernel_step(f, keep=0.02, beta=1e-3):   # least squares in the gradient domain, strong
 ↪  edges only
    num = den = 0
    for D in (Dx, Dy):
        fx = np.real(ifft2(D * fft2(f)))
        fx[np.abs(fx) < np.quantile(np.abs(fx), 1 - keep)] = 0
        Fx = fft2(fx)
        num, den = num + np.conj(Fx) * D * G, den + np.abs(Fx)**2
    h = np.roll(np.real(ifft2(num / (den + beta))), (K // 2, K // 2), (0, 1))[:K, :K]
    h = np.maximum(h, 0)                            # project onto h >= 0 ...
    return h / h.sum()                              # ... and sum(h) = 1

def best_shift(fn):            # blind deconvolution recovers h (and f) only up to a small
 ↪  translation
    return min(fn(dy, dx) for dy in range(-3, 4) for dx in range(-3, 4))
kernel_error = lambda h: best_shift(lambda dy, dx: np.linalg.norm(np.roll(h, (dy, dx), (0,
 ↪  1)) - h_true)) / np.linalg.norm(h_true)
psnr = lambda f: -best_shift(lambda dy, dx: 10 * np.log10(np.mean((np.clip(np.roll(f, (dy,
 ↪  dx), (0, 1)), 0, 1) - f_true)[8:-8, 8:-8] ** 2)))

h = np.zeros((K, K)); h[K // 2, K // 2] = 1
h = gaussian_filter(h, 1.0); h /= h.sum()           # start from an almost-delta kernel
 ↪  ("no blur")
print(f"kernel error at start: {kernel_error(h):.2f}")
for _ in range(30):                                 # alternate: image update, then kernel
 ↪  update
    h = kernel_step(shock(image_step(h, lam=0.02)))
print(f"kernel error after 30 alternations: {kernel_error(h):.2f}")
print(f"PSNR  blurry {psnr(g):.2f} dB | estimated kernel {psnr(image_step(h, 1e-2)):.2f} dB"
      f" | true kernel {psnr(image_step(h_true, 1e-2)):.2f} dB")
# kernel error at start: 1.19
# kernel error after 30 alternations: 0.57
# PSNR  blurry 22.59 dB | estimated kernel 27.11 dB | true kernel 29.67 dB
```

The relative kernel error roughly halves, and the final non-blind step with the estimated kernel recovers most of the gap between the blurry input and the true-kernel result. Be honest about what this toy is: the blur is synthetic, uniform and circular (so the FFT model is exact), the noise is low, and the parameters (`lam`, `keep`, the number of shock iterations) were tuned on this one image. There is no image pyramid, so larger kernels would fail, and with real photos you would also need boundary handling, saturated-pixel handling and robust losses. Both errors are measured after the best small shift, because of the translation ambiguity explained above.

Data and milestones

Nah et al. [44] (CVPR 2017) addressed this with the **GoPro** dataset: they recorded sharp video with a high-speed camera and averaged consecutive frames to produce realistically blurred images, keeping the middle frame as the sharp target. Their network is **multi-scale**: it restores a coarse version first, then refines at finer scales, like the coarse-to-fine schemes of classical blind deblurring, but without estimating a kernel at all. GoPro is still the most common deblurring benchmark.

Today the most common baselines are the general backbones **Restormer** [23] and **NAFNet** [24]. NAFNet's striking finding is that the usual nonlinear activations (ReLU, GELU, Sigmoid) are not needed: they can be replaced by a simple element-wise multiplication of two feature halves ("SimpleGate") or removed, giving a "nonlinear activation free" network that is both simpler and stronger on GoPro and SIDD. The deblurring survey by Zhang et al. [69] covers the wider landscape (blind and non-blind, video, defocus, losses).

Inpainting (brief)

Plain version. Fill in missing or damaged regions (scratches, removed objects, holes) with plausible content.

Inpainting is the restoration task closest to image **generation**: inside a large hole there is nothing to restore, only something to invent that matches the surroundings. The degradation is a mask, $y = m \odot x$, where m is 0 in the hole and 1 elsewhere. **Masked autoencoders** (MAE) [45] can be seen as inpainting used as a pre-training task: hide 75% of image patches and train a ViT to reconstruct them, which turns out to teach strong visual representations. For practical inpainting, **LaMa** [46] uses fast Fourier convolutions with an image-wide receptive field and training on large masks to fill big holes at high resolution; today, diffusion-based inpainting is the default for large, semantic edits.

Matting (brief)

Plain version. Cut out the foreground (a person, their hair) with soft, partially transparent edges so it can be pasted on a new background.

Matting assumes each pixel is a blend of foreground and background,

$$I = \alpha F + (1 - \alpha)B,$$

where $\alpha \in [0, 1]$ is the opacity (alpha matte), F the foreground color and B the background color. With seven unknowns per pixel (α and two RGB colors) and three observations, the problem is badly underdetermined, so classical methods ask the user for a **trimap** (definite foreground, definite background, unknown band). It can be seen as a soft variant of segmentation. **Background Matting** [47] replaces the trimap with an extra photo of the background without the subject, and

trains networks first on synthetic composites and then on real images with an adversarial loss that judges how natural the new composites look.

General backbones

Plain version. Across all these tasks, a handful of network designs keep winning. They are general "image-to-image engines" that you can train on any of the degradations above.

Backbone	Core idea	Why it suits restoration
U-Net [48]	Encoder–decoder with skip connections at every scale	Large receptive field from the downsampling path, fine detail from the skips
SwinIR [14]	Residual groups of shifted-window self-attention blocks at full resolution	Content-adaptive weights; strong for SR, denoising, JPEG
Restormer [23]	U-Net of transformer blocks; attention computed across **channels** (transposed attention) plus gated feed-forward	Cost linear in the number of pixels, so it scales to large images
NAFNet [24]	U-Net of simple conv blocks with LayerNorm, SimpleGate and simplified channel attention; no nonlinear activations	Very strong accuracy per FLOP; the default "simple baseline"
MambaIR / MambaIRv2 [49], [50]	Selective state-space (Mamba) scans with local enhancement and channel attention; v2 adds "attentive" non-causal scanning	Global receptive field at linear cost

Design reasons that recur. (1) **Global residual**: predict $x - y$ (or x minus an upsampled y) so the network learns only the missing part. (2) **No batch normalization** in most SR/restoration nets since EDSR [10]: BN statistics differ between training patches and full test images and hurt color/range fidelity. (3) **Large receptive field** at manageable cost: multi-scale U-Nets, windowed or channel attention, state-space scans. Restoration needs context (to tell a texture from noise) *and* full-resolution detail. (4) **Attention or gating** for content-adaptive processing: channel attention (RCAN), gating (NAFNet, Restormer). The post's "Extension" list predicted new architectures from ViT, flow, GAN and diffusion; by 2026 the state-space model joined CNNs and transformers as a third backbone option [49], [50].

All-in-one and universal restoration

Plain version. Instead of training one model for rain, another for haze and another for noise, train *one* model that figures out what is wrong with the image and fixes it.

Motivation

The post's "Extension" section lists all-in-one restoration as one of the hottest topics of 2023, starting around 2021. The motivation is practical: in a deployed system we do not know in advance which degradation (or mixture) a given image has, and weather can change or combine at any moment. A bank of task-specific models needs a reliable classifier in front and wastes memory; a single model that adapts is more convenient. The difficulty is **task interference**: different degradations need different, sometimes opposite operations (denoising smooths, deblurring sharpens), so naive multi-task training underperforms specialists.

Representative methods

- **AirNet** [51] (CVPR 2022), the "airnet" of the original post: a contrastive-learning degradation encoder extracts a representation of *how* the image is degraded, without labels, and that representation guides the restoration network. It handles unknown corruption types with one model; its noise + rain + haze setting became the standard "three-task" benchmark.
- **TransWeather** [52] (CVPR 2022): a single transformer encoder–decoder for rain, fog and snow, with learnable **weather-type queries** in the decoder.
- **PromptIR** [53] (NeurIPS 2023): a Restormer-style network with lightweight **prompt blocks**. Learnable prompt components are combined with weights predicted from the input features and injected into the decoder, encoding degradation-specific information. Tested on denoising, deraining and dehazing.
- **AdaIR** [54] (ICLR 2025): observes that different degradations affect **different frequency subbands**, then mines low- and high-frequency features from the input and modulates them adaptively. Covers five degradations: noise, haze, rain, motion blur and low light.
- **DA-CLIP** [55] (ICLR 2024): adapts the vision–language model CLIP with a trained controller so that its image encoder yields both a clean-content embedding and a degradation embedding from a corrupted image; these guide a restoration network through cross-attention.
- **InstructIR** [56] (ECCV 2024): the user writes a natural-language instruction ("remove the rain", "make it brighter"), a text encoder embeds it, and the embedding modulates the restoration network.

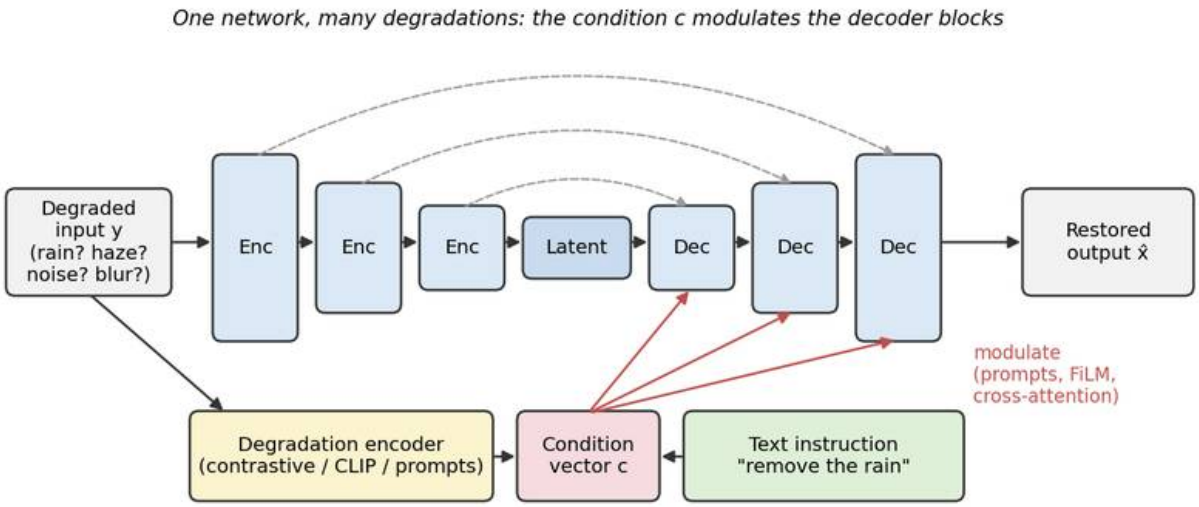


Figure 5 — The common template behind all-in-one restoration: an encoder–decoder backbone plus a degradation-aware condition c, inferred from the image (AirNet, PromptIR, AdaIR, DA-CLIP) or given as text (InstructIR), that modulates the decoder.

The post's original motivation is still the core; what changed is *who decides* the degradation type. It moved from fixed task labels to learned prompts, to vision–language embeddings and finally to human instructions, often backed by large pretrained models. The all-in-one survey [72] discusses this in detail.

Loss functions

Plain version. The loss decides what "good" means. Pixel losses reward being close to the truth on average; perceptual and adversarial losses reward looking real; frequency losses reward getting fine detail.

Pixel losses. With $\hat{x} = f_\theta(y)$,

$$\ell_2 = \frac{1}{P}\sum_p (\hat{x}_p - x_p)^2, \qquad \ell_1 = \frac{1}{P}\sum_p |\hat{x}_p - x_p|, \qquad \ell_\varepsilon = \frac{1}{P}\sum_p \sqrt{(\hat{x}_p - x_p)^2 + \varepsilon^2},$$

where P is the number of pixel values and ε a small constant that makes the last one a smooth, differentiable version of L1. L2 directly optimizes PSNR but penalizes large errors heavily and converges to the posterior mean. Zhao et al. [57] showed that the loss alone changes results noticeably with the architecture fixed, and that L1 and perceptually motivated losses (including combinations with SSIM) often beat L2 even on L2-related metrics. L1 is the default in most modern SR and restoration papers.

Perceptual (feature) loss. Compare images in the feature space of a pretrained classification network ϕ (usually VGG) [58]:

$$\ell_{\text{feat}} = \frac{1}{C_j H_j W_j} \|\phi_j(\hat{x}) - \phi_j(x)\|_2^2,$$

where ϕ_j is the feature map of layer j with C_j channels and size $H_j \times W_j$. It tolerates small misalignments and rewards matching edges and textures rather than exact values.

Adversarial loss. A discriminator D_ψ learns to tell restored from real images and the restorer learns to fool it, e.g. $\ell_{\text{adv}} = -\log D_\psi(\hat{x})$. This pushes outputs onto the manifold of natural images (SRGAN [12], ESRGAN [13], Real-ESRGAN [4]) at the cost of possible hallucinated texture. In practice GAN-based restorers use a weighted sum such as $\ell = \ell_1 + \lambda_p \ell_{\text{feat}} + \lambda_a \ell_{\text{adv}}$ with weights λ_p, λ_a.

Frequency losses. Networks tend to fit low frequencies first and under-fit fine detail. Adding a loss on the Fourier transform $\mathcal{F}$, such as $\|\mathcal{F}(\hat{x}) - \mathcal{F}(x)\|_1$ on complex values or on amplitude, directly penalizes missing high frequencies. The **focal frequency loss** [59] goes further and adaptively up-weights the frequency components that are currently hardest to reconstruct. Frequency terms are now common in deblurring and all-in-one models.

Evaluation metrics

Plain version. "Is the restored image close to the truth?" and "Does it look like a real photo?" are different questions, answered by different metrics, and you cannot maximize both at once.

Full-reference metrics

PSNR (peak signal-to-noise ratio):

$$\text{PSNR}(x, \hat{x}) = 10 \log_{10} \frac{\text{MAX}^2}{\text{MSE}(x, \hat{x})},$$

where MAX is the largest possible pixel value (1 or 255) and MSE is the mean squared error. It is measured in dB; +1 dB is a 21% reduction in MSE.

SSIM (structural similarity) [60] compares local means, variances and covariance:

$$\text{SSIM}(a, b) = \frac{(2\mu_a\mu_b + C_1)(2\sigma_{ab} + C_2)}{(\mu_a^2 + \mu_b^2 + C_1)(\sigma_a^2 + \sigma_b^2 + C_2)},$$

computed over local windows a, b of the two images and averaged, where μ are local means, σ^2 local variances, σ_{ab} the local covariance and C_1, C_2 small constants for stability. It correlates better with perceived structure than PSNR but still prefers smooth results.

LPIPS [61] measures distance between deep-network features (calibrated on human similarity judgments). Lower is better, and it rewards realistic textures that PSNR penalizes.

```
import numpy as np
from scipy import ndimage as ndi
```

```
from skimage import data, img_as_float32
from skimage.metrics import peak_signal_noise_ratio, structural_similarity

x = img_as_float32(data.camera())
y = np.clip(x + np.random.default_rng(0).normal(0, 0.1, x.shape), 0, 1).astype(np.float32)
x_hat = ndi.gaussian_filter(y, 1.0)

def psnr(ref, est, peak=1.0):
    mse = np.mean((ref.astype(np.float64) - est) ** 2)
    return 10 * np.log10(peak ** 2 / mse)

print(f"PSNR  ours {psnr(x, x_hat):.2f} dB | skimage {peak_signal_noise_ratio(x, x_hat,
 ↪  data_range=1):.2f} dB")
print(f"SSIM  {structural_similarity(x, x_hat, data_range=1):.3f}")
# PSNR  ours 27.20 dB | skimage 27.20 dB
# SSIM  0.639
```

Always report the conventions (RGB or Y channel, border crop, data range, 8-bit rounding): differences in these alone can shift published PSNR by tenths of a dB.

No-reference metrics

Plain version. A no-reference (or *blind*) metric looks at a single image and guesses how good a person would say it is, without a "correct answer" to compare against. It is like a teacher grading an essay without an answer key: the teacher relies on what good essays usually look like. Restoration needs such metrics because the photos we most want to fix (real blurry, noisy, rainy, dark photos) have no clean version anywhere.

Full-reference, reduced-reference and no-reference

Image quality assessment (IQA) methods are grouped by how much they know about the pristine image x:

- **Full-reference (FR)**: the whole x is available, and the metric compares $\hat{x}$ with it pixel by pixel or feature by feature (PSNR, SSIM [60], LPIPS [61]). This is what synthetic benchmarks use.
- **Reduced-reference (RR)**: only a few summary features of x are available (for example statistics sent alongside a compressed video), and the metric compares the same features computed on $\hat{x}$.
- **No-reference (NR, blind)**: only $\hat{x}$ is available. The metric must carry its own model of what "good" looks like.

Restoration needs NR metrics for three reasons. (1) **Real test images have no ground truth**: real-world SR, deraining and dehazing are evaluated on photos where nobody has the clean version. (2) **The reference itself can be imperfect or only one of many valid answers**: a GAN or diffusion output can look excellent yet have low PSNR because its texture differs from the reference's (the perception–distortion trade-off below). (3) **NR scores can steer systems at run time**, for example

to choose among outputs or parameters when no reference exists. The price: an NR metric is a *model of human opinion*, and every model has blind spots.

How NR metrics are validated: correlation with human opinion

Plain version. We trust an NR metric only if it ranks images the way people do. So researchers collect many human ratings on many images and check how well the metric's scores agree with them.

Mean opinion score. In a subjective study, each image i is rated by several observers, and the average rating is its **mean opinion score**, $\mathrm{MOS}_i = \frac{1}{R}\sum_{r=1}^{R} s_{ir}$, where s_{ir} is the score of rater r and R the number of raters. Some datasets publish a **differential** score (DMOS), based on the difference between the ratings of the distorted image and its reference; for DMOS, lower usually means better.

SRCC (Spearman rank-order correlation) measures whether the metric puts the images in the same *order* as people:

$$\mathrm{SRCC} = 1 - \frac{6\sum_{i=1}^{n} d_i^2}{n\,(n^2-1)},$$

where n is the number of test images and d_i the difference between the rank of image i by predicted score and its rank by MOS (this short form assumes no tied ranks; with ties one computes the Pearson correlation of the ranks). SRCC is unchanged by any monotonic transformation of the scores, so it measures **prediction monotonicity**.

PLCC (Pearson linear correlation) measures how *linearly* the scores agree:

$$\mathrm{PLCC} = \frac{\sum_{i=1}^{n}(q_i-\bar{q})(m_i-\bar{m})}{\sqrt{\sum_{i=1}^{n}(q_i-\bar{q})^2}\,\sqrt{\sum_{i=1}^{n}(m_i-\bar{m})^2}},$$

where q_i is the metric's score for image i, m_i its MOS, and $\bar{q}$, $\bar{m}$ their means. Because metric scales are arbitrary, it is common practice to first pass q_i through a fitted monotonic (usually logistic) function and compute PLCC on the mapped scores, so PLCC measures **prediction accuracy** after calibration. Both lie in $[-1, 1]$. For "lower is better" metrics such as NIQE the raw correlation with MOS is negative, and papers report its absolute value. A metric is typically trained on part of a dataset and tested on the rest, and its cross-dataset correlation is the more honest test of generalization.

Human-rated datasets. NR models are trained and tested on IQA datasets. They differ in *where the distortions come from*, which matters a lot for restoration:

Dataset	Distortions	What it contains
LIVE (Release 2) [104]	Synthetic	Images with five distortion types (JPEG, JPEG2000, Gaussian blur, white noise, JPEG2000 bit errors), with DMOS

Dataset	Distortions	What it contains
TID2013 [105]	Synthetic	25 reference images × 24 distortion types × 5 levels = 3,000 distorted images; scores from 971 experiments with observers in five countries
KADID-10k [106]	Synthetic	81 pristine images × 25 distortion types × 5 levels = 10,125 images, 30 crowdsourced ratings each
KonIQ-10k [107]	Authentic (in the wild)	10,073 photos with about 1.2 million ratings from 1,459 crowd workers
SPAQ [108]	Authentic (smartphones)	11,125 photos from 66 smartphones, with quality, attribute (brightness, colorfulness, contrast, noisiness, sharpness) and scene labels plus EXIF data
PIPAL [109]	Restoration outputs	Outputs of image restoration algorithms, including GAN-based ones, with over 1.13 million human judgments converted to scores by an Elo system

Synthetic datasets control the distortion exactly but contain only "textbook" damage. Authentic datasets contain real camera problems but no restoration outputs. Only datasets such as PIPAL contain the kind of images a restoration paper actually needs to score.

Family 1: natural-scene-statistics (NSS) models

Plain version. Clean photos share hidden statistical habits. If you remove local brightness and local contrast from a photo, the leftovers follow a bell-shaped histogram of a typical shape, and neighboring leftovers are only weakly related. Blur, noise and compression change that shape in characteristic ways. NSS metrics measure the shape and ask how unusual it is.

MSCN coefficients. For a grayscale image I, the **mean-subtracted contrast-normalized** (MSCN) coefficients are [94]

$$\hat{I}(i,j) = \frac{I(i,j) - \mu(i,j)}{\sigma(i,j) + C}, \qquad \mu(i,j) = \sum_{k,l} w_{k,l}\, I(i+k, j+l), \qquad \sigma(i,j) = \sqrt{\sum_{k,l} w_{k,l}\, \big(I(i+k,j+l) - \mu(i,j)\big)^2},$$

where (i,j) is a pixel position, w a small circularly symmetric Gaussian window whose weights sum to 1, μ and σ the local weighted mean and standard deviation, and C a small constant (1 for 8-bit images) that prevents division by zero in flat regions. This is a divisive normalization, loosely modeled on contrast gain control in early human vision.

The generalized Gaussian fit. The histogram of $\hat{I}$ is modeled by a zero-mean generalized Gaussian distribution (GGD),

$$f(t;\, \alpha, \sigma^2) = \frac{\alpha}{2\beta\,\Gamma(1/\alpha)} \exp\!\Big(-(|t|/\beta)^{\alpha}\Big), \qquad \beta = \sigma\sqrt{\frac{\Gamma(1/\alpha)}{\Gamma(3/\alpha)}},$$

where t is an MSCN value, $\alpha > 0$ the **shape** ($\alpha = 2$ is a Gaussian, $\alpha = 1$ a Laplacian; smaller α means a peakier histogram with heavier tails, larger α a flatter top), σ^2 the variance, β the corresponding scale and Γ the gamma function. A simple way to estimate α is **moment matching**: the ratio $\mathbb{E}[t^2]/(\mathbb{E}|t|)^2 = \Gamma(1/\alpha)\,\Gamma(3/\alpha)/\Gamma(2/\alpha)^2$ depends only on α, so one computes the left side from the data and searches for the α that matches it.

BRISQUE [94] (Blind/Referenceless Image Spatial Quality Evaluator) computes the GGD shape and variance of the MSCN coefficients, plus parameters of *asymmetric* GGDs fitted to products of neighboring MSCN coefficients in four orientations (horizontal, vertical and the two diagonals), at two image scales. A support-vector regressor trained on human scores of synthetically distorted images maps these features to a quality score. Lower is better. It is **opinion-aware**: it needs human scores for training.

NIQE [62] (Natural Image Quality Evaluator) is **opinion-unaware**: it never sees human scores or distorted images. It computes similar NSS features on sharp patches, fits a multivariate Gaussian (MVG) with mean ν_1 and covariance Σ_1 to features from a corpus of pristine images, fits another MVG (ν_2, Σ_2) to the test image's patches, and reports

$$D(\nu_1, \nu_2, \Sigma_1, \Sigma_2) = \sqrt{(\nu_1 - \nu_2)^\top \Big(\frac{\Sigma_1 + \Sigma_2}{2}\Big)^{-1} (\nu_1 - \nu_2)},$$

a Mahalanobis-like distance between the two Gaussians ($^\top$ is the transpose). Lower is better; zero means "statistically like the pristine corpus".

PIQE [96] (Perception-based Image Quality Evaluator) also uses MSCN coefficients but works **block by block**: it divides the image into 16×16 blocks, flags blocks that show blocking artifacts or noise, and pools the distortion of the flagged blocks into one score. It needs no training on opinion scores, and it gives a local map of where the image looks distorted. In the common MATLAB implementation the score lies between 0 and 100, lower is better.

Ma et al.'s score and the Perceptual Index. Ma et al. [95] ran human studies on super-resolved images and learned a no-reference metric for SR from those scores, using statistical features in spatial and frequency domains and a two-stage regression; their score ("Ma", higher is better) lies roughly between 0 and 10. The 2018 PIRM challenge on perceptual SR [63] combined it with NIQE into the **Perceptual Index**,

$$\mathrm{PI} = \tfrac{1}{2}((10 - \mathrm{Ma}) + \mathrm{NIQE}),$$

so that lower PI means better perceptual quality. PIRM ranked methods by PI within three regions of the perception–distortion plane, defined by RMSE at most 11.5, 12.5 and 16, so that realism was compared at a fixed distortion budget. PI is still reported in many perceptual SR papers.

Code: MSCN statistics in practice. The snippet computes MSCN coefficients, the GGD shape by moment matching, the MSCN variance and the correlation between horizontal neighbors (a simplified version of BRISQUE's pairwise features) on `camera` and on blurred, noisy and over-sharpened versions:

```
import numpy as np
from scipy.ndimage import gaussian_filter
```

```
from scipy.special import gamma
from skimage import data, img_as_float

def mscn(img, sigma=7/6, C=1.0):
    """Mean-subtracted contrast-normalized coefficients of a grayscale image in [0, 255]."""
    mu = gaussian_filter(img, sigma)
    sd = np.sqrt(np.abs(gaussian_filter(img * img, sigma) - mu * mu))
    return (img - mu) / (sd + C)

def ggd_shape(x):
    """Moment-matching estimate of the generalized Gaussian shape alpha (2 = Gaussian)."""
    alphas = np.arange(0.2, 10.0, 0.001)
    r_theory = gamma(1 / alphas) * gamma(3 / alphas) / gamma(2 / alphas) ** 2
    r_data = np.mean(x ** 2) / np.mean(np.abs(x)) ** 2
    return alphas[np.argmin((r_theory - r_data) ** 2)], np.mean(x ** 2)

rng = np.random.default_rng(0)
clean = img_as_float(data.camera()) * 255
versions = {
    "clean": clean,
    "blur (sigma=2)": gaussian_filter(clean, 2),
    "noise (sigma=20)": np.clip(clean + rng.normal(0, 20, clean.shape), 0, 255),
    "over-sharpened": np.clip(clean + 2.0 * (clean - gaussian_filter(clean, 2)), 0, 255),
}
for name, img in versions.items():
    m = mscn(img)
    alpha, var = ggd_shape(m)
    rho = np.mean(m[:, :-1] * m[:, 1:]) / var           # correlation of horizontal neighbors
    print(f"{name:17s} GGD shape {alpha:4.2f} | MSCN variance {var:4.2f} | neighbor corr.
     ↪  {rho:+.2f}")
# clean             GGD shape 1.56 | MSCN variance 0.28 | neighbor corr. +0.12
# blur (sigma=2)    GGD shape 0.87 | MSCN variance 0.04 | neighbor corr. +0.89
# noise (sigma=20)  GGD shape 3.12 | MSCN variance 0.68 | neighbor corr. -0.10
# over-sharpened    GGD shape 2.35 | MSCN variance 0.43 | neighbor corr. +0.09
```

Each distortion pushes the features in its own direction (Figure 6). **Blur** removes local contrast, so most MSCN values collapse toward zero: tiny variance, a very peaky histogram (small α) and strongly correlated neighbors. **Noise** adds independent energy: larger variance, a flat-topped histogram (α above 2) and neighbor correlation near zero (slightly negative, because subtracting the local mean couples neighbors). **Over-sharpening** moves the features *in the same direction as noise*. That is the core weakness of NSS metrics for restoration: they cannot tell restored texture from invented high-frequency energy. Note also that the absolute numbers depend on content and on C (the clean `camera` image is not a unit Gaussian), which is why BRISQUE needs a trained regressor and NIQE a pristine reference corpus rather than fixed thresholds.

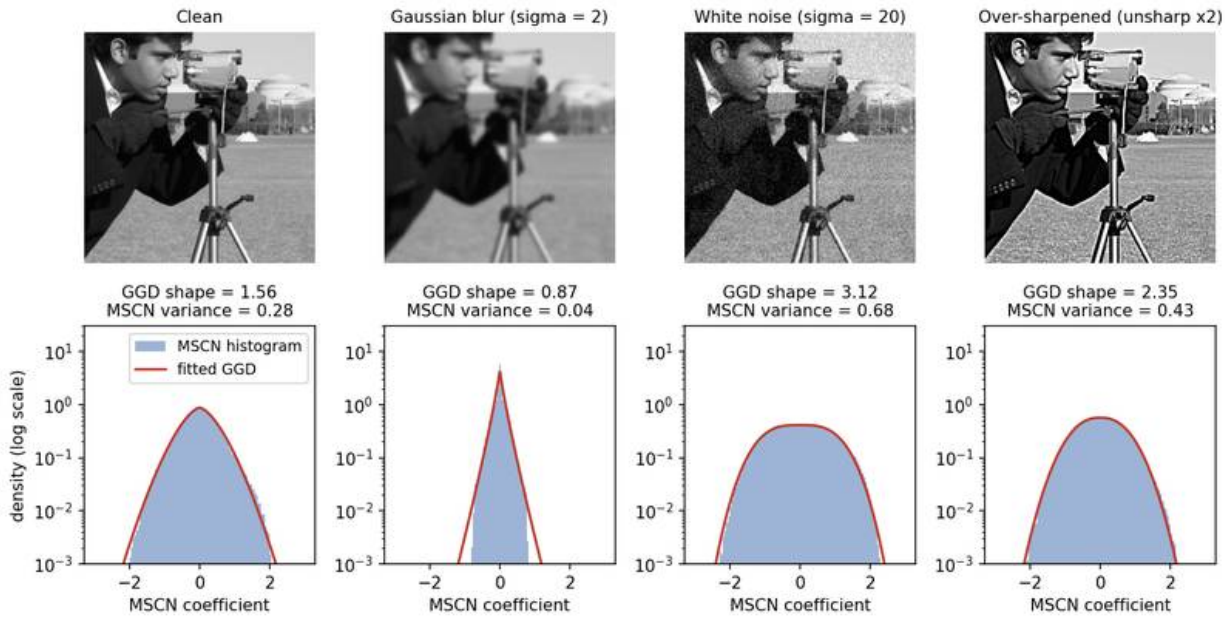


Figure 6 — MSCN statistics, the basis of BRISQUE, NIQE and PIQE. Blur makes the histogram narrow and peaky; noise and over-sharpening both make it wider and flatter, so an NSS metric sees them as similar changes.

Family 2: learned deep models

Plain version. Instead of hand-made statistics, train a neural network to predict human ratings directly. Deep features capture content, semantics and complex artifacts far better, but the model only knows the kinds of images and opinions it was trained on.

- **NIMA** [97] predicts the whole *distribution* of human ratings with a CNN, not a single number. With p_k the predicted probability of score k (on a 1–10 scale), the reported score is the mean $\hat{s} = \sum_k k\,p_k$, and the spread of the distribution says how much people would disagree. Checkpoints trained on aesthetic ratings judge "is this a nice photo", not "is this a faithful restoration".
- **HyperIQA** [98] targets authentic, in-the-wild distortions. It splits assessment into three stages: a backbone first *understands the content*, a **hyper network** then generates the weights of the quality-prediction network from that content (the "perception rule" adapts to what is in the picture, since the same blur is judged differently on a sky and on a face), and the generated network predicts the score.
- **MUSIQ** [99] is a multi-scale transformer that takes the image at its **native resolution and aspect ratio** plus downscaled versions, using hash-based 2-D spatial and scale embeddings, so no resizing or cropping changes the very artifacts being judged.
- **MANIQA** [100] combines vision-transformer features with transposed (channel) attention blocks and scale Swin-transformer blocks, and predicts the score with a dual-branch, patch-weighted head. It won the no-reference track of the NTIRE 2022 perceptual IQA challenge, whose data include outputs of restoration and GAN-based algorithms, so it is closer to the

restoration setting than metrics trained on camera distortions.
- **TOPIQ** [101] follows a **top-down** idea: high-level semantic features decide where to look, and a cross-scale attention mechanism uses them to weight lower-level, distortion-sensitive features (its network is called CFANet, with a ResNet-50 backbone). The same design works for FR and NR assessment and is light compared with transformer-based IQA models.

All of these are **opinion-aware** and output "higher is better" scores, and each depends on its training dataset: a model trained on KonIQ-10k and the same architecture trained on PIPAL can rank restoration methods differently.

Family 3: vision-language and large multimodal models

Plain version. Models trained on huge collections of images and text already "know" what a good or bad photo looks like. You can ask them, either by comparing the image with text prompts or by literally asking a chatbot-style model to rate it.

CLIP-IQA [102] compares the CLIP embedding of the image with the embeddings of an antonym prompt pair such as "Good photo." and "Bad photo.". With $\mathbf{x}$ the image embedding, $\mathbf{t}_1, \mathbf{t}_2$ the two text embeddings and $s_i = \mathbf{x}^\top \mathbf{t}_i / (\|\mathbf{x}\| \, \|\mathbf{t}_i\|)$ their cosine similarities, the score is

$$\text{CLIP-IQA} = \frac{e^{s_1}}{e^{s_1} + e^{s_2}} \in [0, 1],$$

higher is better. The basic version is **zero-shot** (no opinion scores at all), and other prompt pairs ("Noisy photo." / "Clean photo.", "Sharp photo." / "Blurry photo.") probe single attributes. The wording of the prompts changes the scores.

Q-Align [103] fine-tunes a large multimodal model to answer the way human raters do, with a **text rating level** (bad, poor, fair, good, excellent) instead of a number. At test time, with X_{l_i} the model's logit for level l_i, the score is the expected level under a softmax restricted to the five words,

$$S = \sum_{i=1}^{5} i \cdot \frac{e^{X_{l_i}}}{\sum_{j=1}^{5} e^{X_{l_j}}},$$

where $i = 1, \ldots, 5$ indexes the levels from bad to excellent. Such models are strong across datasets but heavy, and they inherit the biases of their base model.

Distribution-level metrics

Some metrics score a *set* of outputs rather than one image. **FID** [64] compares the distribution of Inception features of the restored set with that of a set of real images; it was designed for generative models and is used for generative restoration such as diffusion SR, but it needs many images and says nothing about any single one. **IS** (Inception Score), mentioned in the post, measures the confidence and diversity of class predictions; it suits class-conditional generation rather than restoration and is rarely used there today.

Pitfalls

- **Metrics can be gamed.** A restorer that adds grain or sharpens edges can improve NSS scores, as the code above shows, without becoming more faithful. Any differentiable metric (and any learned IQA network) used as a training loss or for model selection can be optimized directly, and the network may learn to please the metric rather than people. If a method uses CLIP or an IQA network in its loss, evaluating it with the same model is not independent.
- **Domain mismatch.** Most opinion-aware metrics were trained on synthetic distortions (LIVE, TID2013, KADID-10k) or on camera photos (KonIQ-10k, SPAQ). Restoration outputs contain different artifacts: hallucinated textures, over-smoothing, GAN patterns. This mismatch was the explicit motivation for PIPAL [109], whose authors found that existing IQA methods could not fairly evaluate GAN-based restoration. For low light, the problems are exposure and color, which most NSS features normalize away; see the low-light-specific discussion in DLCV 05.
- **Metrics disagree.** NSS, learned and vision-language metrics often rank the same methods differently, and all can disagree with people. Scores also depend on the implementation, checkpoint, color space, input size and whether the image is resized, so numbers copied from different papers are often not comparable.
- **"Higher NR quality" is not "more correct".** An NR metric cannot know whether a sharp face or line of text matches the real scene. A hallucinated license plate can score perfectly.

Recommended practice

1. When a reference exists, lead with FR metrics (PSNR, SSIM, LPIPS) and add NR metrics as a complement; place your method on the perception–distortion plane (next subsection).
2. On real images without references, report **several NR metrics from different families** (for example NIQE or PI, one or two learned models such as MUSIQ, MANIQA or TOPIQ, and a vision-language one such as CLIP-IQA or Q-Align), and say which way each one points.
3. Use one shared implementation and state it, with versions and checkpoints. The open-source **IQA-PyTorch** toolbox (`pyiqa`) [110] reimplements most metrics in this section (NIQE, BRISQUE, PIQE, the Ma score and PI, NIMA, HyperIQA, MUSIQ, MANIQA, TOPIQ, CLIP-IQA, Q-Align, and FR metrics) behind one interface.
4. Do not tune on the judge: keep at least one metric that played no role in training or model selection.
5. Back important claims with a **user study** (mean opinion scores or paired comparisons, blind and randomized); DLCV 05 describes MOS and Bradley–Terry paired-comparison protocols. When metrics disagree with each other or with people, say so.

Summary table

Metric	Family	Needs training?	Better	Mainly sensitive to
NIQE [62]	NSS	Pristine images only (opinion-unaware)	Lower	Deviations of local statistics: blur, noise, compression
BRISQUE [94]	NSS + regression	Yes, human scores on synthetic distortions	Lower	The same statistics, mapped to human scores
PIQE [96]	NSS, block-wise	No opinion scores	Lower	Blocking and noise in local blocks; gives a distortion map
Ma [95]	Handcrafted features + regression	Yes, human scores on SR outputs	Higher	SR artifacts, texture and sharpness
PI [63]	NSS + learned	Through its Ma part	Lower	Perceptual quality of SR; rewards texture, including amplified noise
NIMA [97]	Learned CNN	Yes, rating distributions	Higher	Technical or aesthetic quality, depending on checkpoint
HyperIQA [98]	Learned CNN + hyper network	Yes	Higher	Authentic distortions, judged in the context of content
MUSIQ [99]	Learned transformer	Yes	Higher	Multi-scale quality at native resolution
MANIQA [100]	Learned transformer	Yes	Higher	Restoration and GAN artifacts (trained on such data)
TOPIQ [101]	Learned CNN, top-down attention	Yes	Higher	Distortions in semantically important regions
CLIP-IQA [102]	Vision-language	No (zero-shot version)	Higher	Overall "good photo" impression; depends on prompts
Q-Align [103]	Large multimodal model	Yes, ratings as text levels	Higher	Human-like overall quality judgments
FID [64]	Distribution-level	Pretrained Inception features	Lower	Distribution gap between a set of outputs and real images

The perception–distortion trade-off

Blau and Michaeli [65] formalized the tension. Let $\Delta(x, \hat{x})$ be any distortion measure (MSE, $1 - $ SSIM, ...) and $d(p_X, p_{\hat{X}})$ a divergence between the distribution of real images and that of restored images (perceptual quality: small d means outputs are statistically indistinguishable from real images). The **perception–distortion function** is

$$P(D) = \min_{p_{\hat{X}|Y}} d(p_X, p_{\hat{X}}) \quad \text{subject to} \quad \mathbb{E}[\Delta(X, \hat{X})] \leq D,$$

where the minimum is over all (possibly random) restoration methods. They proved that $P(D)$

is non-increasing and (for divergences convex in their second argument) convex: there is an unattainable region near the origin. A method cannot be both minimal-distortion and perfectly realistic. In particular, the MMSE estimator $\mathbb{E}[x \mid y]$ has the lowest distortion but a blurry, unrealistic output distribution; to look real you must accept higher MSE. GANs (and now diffusion models) are principled ways to move along the bound.

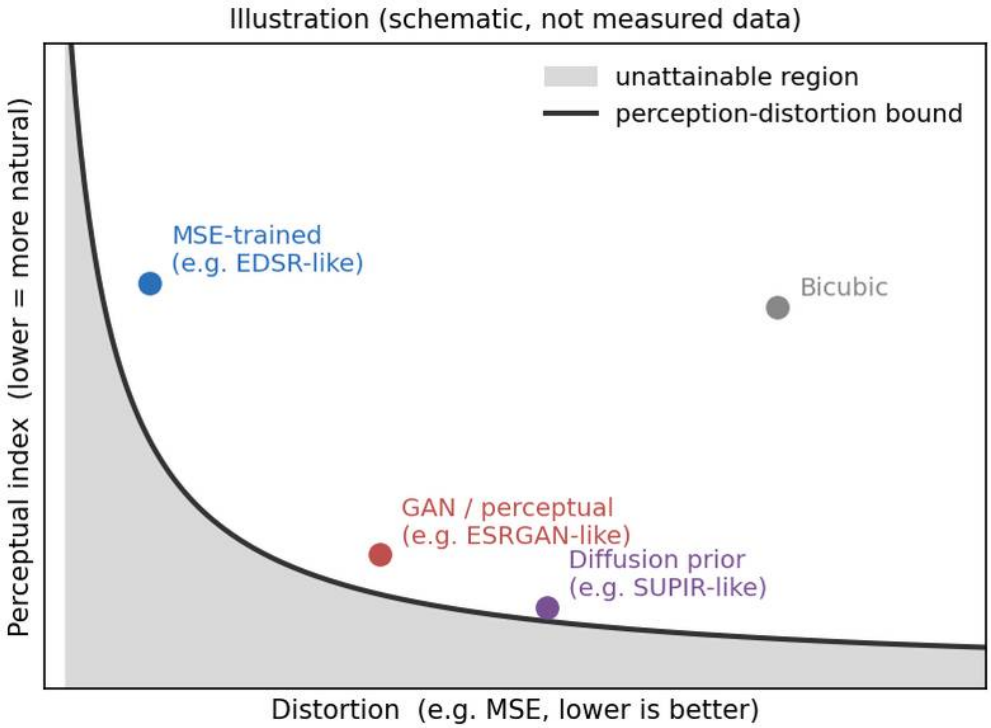


Figure 7 — Schematic of the perception–distortion trade-off [65] (*not measured data*). *The bound is convex and decreasing; methods trade fidelity* (*left*) *against realism* (*bottom*).

This is the update in the post's metric section: since diffusion models can generate plausible but unfaithful detail, the gap between fidelity (PSNR/SSIM) and perceptual quality has become central. The **NTIRE 2025** ×4 SR challenge therefore ran two tracks: a restoration track ranked by PSNR and a perceptual track ranked by a perceptual score [3], whereas the 2024 edition ranked by PSNR [2]. Practical advice: report both a distortion and a perceptual metric, and say where on the trade-off your method aims.

NTIRE challenges

Plain version. Each year a CVPR workshop runs public competitions on fixed datasets and publishes a report describing the winning ideas: the easiest way to see what currently works.

The NTIRE challenge reports are a good way to follow restoration now that per-task leaderboards are gone. Each report describes the task, data, evaluation protocol and the methods of top teams.

Examples cited in this tutorial:

- **NTIRE 2017 SR** [19]: introduced DIV2K; EDSR [10] won.
- **NTIRE 2024** ×4 **SR** [2]: bicubic ×4 on DIV2K, ranked by PSNR.
- **NTIRE 2025** ×4 **SR** [3]: separate restoration (PSNR) and perceptual tracks.
- **NTIRE 2025 raindrop removal** [40]: real day and night raindrop images with dual focus.

Two patterns appear across recent reports: winning entries are usually ensembles or large hybrids of the backbones above (HAT-, Restormer-, NAFNet- or Mamba-style, sometimes with diffusion priors) trained with heavy data and compute, and challenges increasingly use real captured data and perceptual tracks.

Modern view

The post closes with an extension list (medical images, multimodal, unsupervised, new architectures, all-in-one, low-level plus high-level, real-time, underwater). Most of these are now active areas. Here we summarize the survey literature task by task, then the 2023–2026 state of the art and the open problems.

Survey papers

- **Wang, Chen and Hoi, "Deep Learning for Image Super-resolution: A Survey" (TPAMI 2021)** [66]. Organizes SR into supervised, unsupervised and domain-specific SR; within supervised SR it dissects models by upsampling position (pre-, post-, progressive, iterative up-and-down), upsampling operator (interpolation, transposed convolution, sub-pixel), network design (residual, recursive, dense, attention, multi-path), losses and training strategies. Takeaway: most SR progress can be described as choices along these axes, and unsupervised/real-world SR was flagged as the main open direction, which the field then pursued.
- **Tian et al., "Deep Learning on Image Denoising: An Overview" (Neural Networks 2020)** [67]. Groups CNN denoisers into additive white noise, real noise, blind denoising, and hybrid noise with blur or low resolution, and compares them on public benchmarks. Takeaway: the realism of the noise model, not the network, is often the bottleneck.
- **Elad, Kawar and Vaksman, "Image Denoising: The Deep Learning Revolution and Beyond" (SIAM J. Imaging Sciences 2023)** [68]. Traces denoising from classical priors to deep networks, then argues that denoisers are now building blocks for other inverse problems (plug-and-play, regularization by denoising) and for diffusion-based image generation, and that restoration should embrace multiple valid solutions (posterior sampling) instead of one answer. Takeaway: the field's frontier moved from "better MMSE denoisers" to "sampling from the posterior".
- **Zhang et al., "Deep Image Deblurring: A Survey" (IJCV 2022)** [69]. Covers causes of blur, datasets and metrics, and a taxonomy of CNN deblurring by architecture, loss and application (faces, text, stereo), for both blind and non-blind settings. Takeaway: dynamic-scene deblurring moved from kernel estimation to end-to-end, multi-scale networks trained on captured

pairs such as GoPro.

- **Gui et al., "A Comprehensive Survey and Taxonomy on Single Image Dehazing Based on Deep Learning" (ACM Computing Surveys 2023)** [70]. Splits methods into supervised, semi-supervised and unsupervised, and within each by whether and how they use the scattering model; also reviews datasets, network modules, losses and metrics, with experiments on baselines. Takeaway: physics-guided and physics-free designs coexist, and the synthetic-to-real gap is the main open problem.
- **Yang et al., "Single Image Deraining: From Model-Based to Data-Driven and Beyond" (TPAMI 2021)** [71]. Reviews rain appearance models, model-based methods (priors and optimization) and data-driven methods (architectures, constraints, losses, datasets). Takeaway: realistic rain models and real-world evaluation matter more than further architecture tweaks on synthetic data.
- **Jiang et al., "A Survey on All-in-One Image Restoration: Taxonomy, Evaluation and Future Trends" (TPAMI, 2025)** [72]. Organizes all-in-one methods by architectural design, learning paradigm and core innovation (degradation representations, prompts, mixture-of-experts, language guidance), and consolidates datasets and evaluation protocols. Takeaway: comparisons are complicated by inconsistent task sets, and generalization to unseen and mixed degradations remains open.
- **Li et al., "Diffusion Models for Image Restoration and Enhancement: A Comprehensive Survey" (IJCV 2025)** [73]. Categorizes diffusion-based restoration by learning paradigm (supervised versus zero-shot use of a pretrained prior), conditioning strategy, framework design and modeling strategy, with focus on SR, deblurring, inpainting and blind/real-world restoration. Takeaway: diffusion gives the best perceptual quality but at high compute cost and with fidelity/hallucination concerns; distillation to few steps and better evaluation are key directions.

State of the art, 2023–2026

1. **All-in-one and blind restoration became the mainstream setting**, not a niche topic. Methods moved from learned degradation representations (AirNet) to prompts (PromptIR), frequency-aware adaptation (AdaIR) and language guidance (DA-CLIP, InstructIR).
2. **Pretrained diffusion models as generative priors** transformed real-world SR and blind restoration (DiffBIR, SUPIR), and one-step variants (OSEDiff) cut the inference cost from tens or hundreds of steps to one.
3. **State-space models** (MambaIR, MambaIRv2) joined CNNs and transformers as backbone options with global receptive fields at linear cost.
4. **Evaluation caught up with generation**: challenges separate fidelity from perceptual tracks [3], and real captured datasets (SIDD, O-HAZE, Dense-Haze, Raindrop Clarity) are standard alongside synthetic benchmarks.

Open problems

- **Real-world generalization.** Synthetic pipelines still miss real camera processing, spatially varying blur and mixed weather; models degrade on unseen combinations.
- **Fidelity versus hallucination.** Generative priors can invent text, faces or textures. For medical, forensic and scientific use this is unacceptable, and we lack good metrics for "faithful but sharp".
- **Evaluation.** PSNR/SSIM reward blur; no-reference metrics can be gamed; human studies are costly. Agreement between metrics and human preference for generative restoration is still weak.
- **Efficiency.** High-resolution, real-time (and video) restoration on phones and cameras remains a strong constraint, for transformers and diffusion models especially.
- **Task interference and scaling in all-in-one models**, and how to add a new degradation without retraining.
- **Low-level plus high-level.** Restoration that helps downstream detection or segmentation, rather than human viewers, is a different objective (task-driven evaluation, as in RESIDE [32]).

The author's perspective

The post ends with two observations that still hold. First, because each task is simple to state, the race for state of the art is largely won by special architectures, which means a lot of compute and complicated experiments; but these tasks are very practical and easy to apply, so there is much room in **applications** and in **combining tasks** (exactly where all-in-one restoration went). Second, compared with high-level tasks the learning curve is gentle: a beginner can easily picture what the model is doing (a damaged image goes in, a cleaner one comes out), which makes low-level vision a good door into deep learning research.

Key takeaways

- Low-level vision is about *perceiving* pixels well; almost every task is an instance of $y = \mathcal{D}(x) + n$ with a different $\mathcal{D}$ (blur, downsampling, JPEG, scattering, rain layer, Retinex illumination, mask).
- Deep learning moved the degradation model from the solver into the **data generator**: realistic, randomized synthetic pipelines (and captured pairs) are as important as the network.
- Upsampling in SR evolved from bicubic pre-upsampling (SRCNN) through transposed convolution (FSRCNN) to sub-pixel convolution (ESPCN), which is the standard today.
- A few general backbones dominate: U-Net-style CNNs (NAFNet), transformers (SwinIR, Restormer, HAT) and state-space models (MambaIR); key design habits are global residuals, no batch norm, large receptive fields and gating/attention.
- All-in-one models (AirNet, TransWeather, PromptIR, AdaIR, DA-CLIP, InstructIR) handle unknown degradations by conditioning one network on a learned, prompted or text-given degradation description.

- L2/L1 losses give faithful but smooth results; perceptual, adversarial and diffusion-based methods give realistic but less faithful ones. The **perception–distortion trade-off** says you cannot have both at the limit, so report both kinds of metric.
- Blind deblurring and blind SR must estimate the **kernel** as well as the image. Naive joint MAP prefers the no-blur solution [74]; working methods marginalize over the image, use edge prediction or sharpness-favoring priors, alternate image and kernel updates coarse-to-fine and finish with non-blind deconvolution. Deep methods either predict or condition on the kernel (SRMD, IKC, DAN, KernelGAN, SelfDeblur) or skip it entirely (GoPro-trained networks, BSRGAN, Real-ESRGAN).
- KITTI is a driving benchmark, not a dehazing dataset; for dehazing use RESIDE plus real pairs such as O-HAZE and Dense-Haze.
- No-reference metrics are models of human opinion, validated by SRCC/PLCC against mean opinion scores. NSS metrics (BRISQUE, NIQE, PIQE, PI) measure deviations of MSCN statistics; learned (NIMA, HyperIQA, MUSIQ, MANIQA, TOPIQ) and vision-language (CLIP-IQA, Q-Align) models learn from ratings. All can be gamed by sharpening or noise and suffer from the gap between their training distortions and restoration artifacts, so report several, together with FR metrics and a user study.

Exercises

1. **Degradation order.** Using the `degrade` function, compare (a) blur → downsample → noise → JPEG with (b) noise → blur → downsample → JPEG at the same parameters. Visually and with the noise standard deviation measured on a flat patch, explain why the order matters for what the network has to learn.

   Hint

   In (b), the blur and downsampling filter the noise: its standard deviation drops and it becomes spatially correlated (smooth blotches rather than per-pixel grain). In (a), the noise stays white at the low resolution. A network trained only on (a) will under-remove correlated noise at test time. This is one reason Real-ESRGAN randomizes and repeats the stages.

2. **Pixel shuffle by hand.** A feature tensor has shape $(Cs^2, H, W) = (4, 2, 2)$ with $s = 2, C = 1$, and channel k is filled with the constant k. Write the 4×4 output of pixel shuffle using the formula in the SR section, then check with `torch.nn.PixelShuffle(2)`.

   Hint

   Output pixel $(2h + i, 2w + j)$ takes channel $2i + j$. So every 2×2 block is $\begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix}$, and the 4×4 output repeats that block four times. Each output block gathers one value from each of the four channels at the same low-resolution position.

3. **Residual or direct?** Modify `TinyDnCNN.forward` to return `self.body(y)` (predict the clean image directly) and train both versions with the same seed and steps. Compare PSNR and the loss curves. Then repeat with $\sigma = 5/255$. When does residual learning help most?

Hint

With few training steps, the residual version usually converges faster and reaches a higher PSNR, because at initialization it already outputs roughly y (good PSNR) and only needs to learn a small correction. At low noise the direct version struggles most, since it must learn a near-identity mapping through several ReLU layers.

4. **Dark channel by numbers.** A hazy pixel has $I = (0.75, 0.78, 0.80)$, the darkest value of I^c/A^c over its patch and channels is 0.70, and $A = (0.9, 0.9, 0.9)$. With $\omega = 0.95$ and $t_0 = 0.1$, compute $\tilde{t}$ and the recovered $\hat{J}$ for this pixel.

Hint

$\tilde{t} = 1 - 0.95 \times 0.70 = 0.335$. Then $\hat{J}^c = (I^c - 0.9)/0.335 + 0.9$, giving about $(0.452, 0.542, 0.601)$. The output is darker and more saturated than the input, as expected when the veil is removed.

5. **Perception versus distortion.** Take the tiny denoiser's output and add a small amount of fresh Gaussian noise ($\sigma = 3/255$) to it. Compute PSNR and SSIM before and after. The image may look *more* natural (less plastic) although both metrics drop. Relate this to the perception–distortion function $P(D)$.

Hint

The MMSE-like output lies at low distortion but far from the natural-image distribution (too smooth). Adding grain moves the output distribution closer to real photos (lower d) at the price of higher MSE: a move along, or toward, the convex bound, not past it.

6. **Design an all-in-one experiment.** You must build a single model for noise ($\sigma \in [0, 50]$), rain streaks and haze. Describe the training data mix, how you would condition the network (no labels at test time), the baselines you would compare against, and how you would detect task interference.

Hint

Mix datasets per task with balanced sampling; add a degradation encoder (contrastive as in AirNet, or prompts as in PromptIR) trained jointly. Compare with three single-task models of the same backbone and size, and with a version without conditioning. Interference shows up as the all-in-one model falling behind the single-task model on one task while matching it on others; also test on mixed degradations (rain + haze) unseen in training.

7. **Kernel estimation by hand.** In the blind-deconvolution snippet, (a) remove the `shock(...)` call and set `keep=1.0` (use all gradients) and compare the kernel error and final PSNR; (b) start from a 3×3 box kernel instead of the near-delta; (c) replace `h_true` by a straight 25-pixel motion path in a 31×31 window. Explain each result with the discussion of Levin et al. [74] and the coarse-to-fine strategy.

Hint

(a) With all gradients, the many weak gradients of the smoothed latent image dominate the least-squares fit, and the estimate no longer improves (in our run the kernel error grew instead of shrinking and the final PSNR stayed at the blurry input's level). Selecting a few strong, well-predicted edges is what makes the kernel step informative, which is the practical answer to the "no-blur" bias of naive joint MAP. (b) The loop is a non-convex alternation, so the start matters; a different start can converge to a different local minimum. (c) A large kernel at full resolution is hard to find from a delta start: in our run the error dropped much less and most of the true-kernel PSNR was not recovered. Estimating the kernel on a downsampled image first (where it is only a few pixels long) and upsampling the result is what the image pyramid is for.

8. **Validating an NR metric.** Five restored images have mean opinion scores MOS $= (4.1, 3.2, 2.5, 1.8, 3.9)$ and NIQE scores $(3.4, 4.2, 5.1, 6.3, 3.0)$. (a) Compute SRCC by hand and interpret its sign. (b) Would SRCC change if you reported $\log(\text{NIQE})$ instead? Would PLCC? (c) Run the MSCN snippet on an over-sharpened version of a *denoised* image and explain why an NSS metric might rate it better than the plain denoised output.

Hint

(a) Ranking from smallest to largest, the MOS ranks of images 1–5 are $(5, 3, 2, 1, 4)$ and the NIQE ranks are $(2, 3, 4, 5, 1)$. The rank differences are $(3, 0, -2, -4, 3)$, so $\sum d_i^2 = 38$ and SRCC $= 1 - 6 \cdot 38/(5 \cdot 24) = -0.9$. The sign is negative because lower NIQE means better quality; one reports $|\text{SRCC}| = 0.9$. Only images 1 and 5 are swapped. (b) SRCC depends only on ranks, so any monotonic transform such as log leaves it unchanged; PLCC changes, which is why PLCC is usually computed after a fitted monotonic mapping. (c) Denoising leaves a smooth image whose MSCN statistics resemble mild blur (low variance, peaky histogram). Sharpening restores high-frequency energy and pushes the statistics back toward those of natural images, whether or not the added detail is real.

References

1. Shyandram, "Low-Level Vision Task-Image Restoration 簡介," Medium post (in Traditional Chinese), 2023; updated 2026. link
2. Z. Chen, Z. Wu, E. Zamfir, K. Zhang, Y. Zhang, R. Timofte et al., "NTIRE 2024 Challenge on Image Super-Resolution (×4): Methods and Results," in *Proc. IEEE/CVF CVPR Workshops*, 2024. arXiv
3. Z. Chen, K. Liu, J. Gong, J. Wang, L. Sun et al., "NTIRE 2025 Challenge on Image Super-Resolution (×4): Methods and Results," in *Proc. IEEE/CVF CVPR Workshops*, 2025. arXiv
4. X. Wang, L. Xie, C. Dong and Y. Shan, "Real-ESRGAN: Training Real-World Blind Super-Resolution with Pure Synthetic Data," in *Proc. IEEE/CVF ICCV Workshops*, 2021. arXiv
5. E. H. Land and J. J. McCann, "Lightness and Retinex Theory," *Journal of the Optical Society of America*, vol. 61, no. 1, pp. 1–11, 1971. doi
6. C. Dong, C. C. Loy, K. He and X. Tang, "Image Super-Resolution Using Deep Convolutional Networks," *IEEE TPAMI*, 2016 (conference version ECCV 2014). arXiv
7. C. Dong, C. C. Loy and X. Tang, "Accelerating the Super-Resolution Convolutional Neural Network," in *Proc. ECCV*, 2016. arXiv

8. W. Shi, J. Caballero, F. Huszár, J. Totz, A. P. Aitken, R. Bishop, D. Rueckert and Z. Wang, "Real-Time Single Image and Video Super-Resolution Using an Efficient Sub-Pixel Convolutional Neural Network," in *Proc. IEEE CVPR*, 2016. arXiv
9. A. Odena, V. Dumoulin and C. Olah, "Deconvolution and Checkerboard Artifacts," *Distill*, 2016. link
10. B. Lim, S. Son, H. Kim, S. Nah and K. M. Lee, "Enhanced Deep Residual Networks for Single Image Super-Resolution," in *Proc. IEEE CVPR Workshops*, 2017. arXiv
11. Y. Zhang, K. Li, K. Li, L. Wang, B. Zhong and Y. Fu, "Image Super-Resolution Using Very Deep Residual Channel Attention Networks," in *Proc. ECCV*, 2018. arXiv
12. C. Ledig, L. Theis, F. Huszár, J. Caballero, A. Cunningham, A. Acosta, A. Aitken, A. Tejani, J. Totz, Z. Wang and W. Shi, "Photo-Realistic Single Image Super-Resolution Using a Generative Adversarial Network," in *Proc. IEEE CVPR*, 2017. arXiv
13. X. Wang, K. Yu, S. Wu, J. Gu, Y. Liu, C. Dong, C. C. Loy, Y. Qiao and X. Tang, "ESRGAN: Enhanced Super-Resolution Generative Adversarial Networks," in *Proc. ECCV Workshops*, 2018. arXiv
14. J. Liang, J. Cao, G. Sun, K. Zhang, L. Van Gool and R. Timofte, "SwinIR: Image Restoration Using Swin Transformer," in *Proc. IEEE/CVF ICCV Workshops*, 2021. arXiv
15. X. Chen, X. Wang, J. Zhou, Y. Qiao and C. Dong, "Activating More Pixels in Image Super-Resolution Transformer," in *Proc. IEEE/CVF CVPR*, 2023. arXiv
16. X. Lin, J. He, Z. Chen, Z. Lyu, B. Dai, F. Yu, W. Ouyang, Y. Qiao and C. Dong, "DiffBIR: Towards Blind Image Restoration with Generative Diffusion Prior," arXiv:2308.15070, 2023. arXiv
17. F. Yu, J. Gu, Z. Li, J. Hu, X. Kong, X. Wang, J. He, Y. Qiao and C. Dong, "Scaling Up to Excellence: Practicing Model Scaling for Photo-Realistic Image Restoration In the Wild," in *Proc. IEEE/CVF CVPR*, 2024. arXiv
18. R. Wu, L. Sun, Z. Ma and L. Zhang, "One-Step Effective Diffusion Network for Real-World Image Super-Resolution," in *Proc. NeurIPS*, 2024. arXiv
19. E. Agustsson and R. Timofte, "NTIRE 2017 Challenge on Single Image Super-Resolution: Dataset and Study," in *Proc. IEEE CVPR Workshops*, 2017. doi
20. K. Zhang, W. Zuo, Y. Chen, D. Meng and L. Zhang, "Beyond a Gaussian Denoiser: Residual Learning of Deep CNN for Image Denoising," *IEEE Transactions on Image Processing*, vol. 26, no. 7, pp. 3142–3155, 2017. arXiv
21. K. Zhang, W. Zuo and L. Zhang, "FFDNet: Toward a Fast and Flexible Solution for CNN-Based Image Denoising," *IEEE Transactions on Image Processing*, 2018. arXiv
22. A. Abdelhamed, S. Lin and M. S. Brown, "A High-Quality Denoising Dataset for Smartphone Cameras," in *Proc. IEEE/CVF CVPR*, 2018. doi
23. S. W. Zamir, A. Arora, S. Khan, M. Hayat, F. S. Khan and M.-H. Yang, "Restormer: Efficient Transformer for High-Resolution Image Restoration," in *Proc. IEEE/CVF CVPR*, 2022. arXiv
24. L. Chen, X. Chu, X. Zhang and J. Sun, "Simple Baselines for Image Restoration," in *Proc. ECCV*, 2022. arXiv
25. K. He, J. Sun and X. Tang, "Single Image Haze Removal Using Dark Channel Prior," in *Proc. IEEE CVPR*, 2009. doi
26. B. Cai, X. Xu, K. Jia, C. Qing and D. Tao, "DehazeNet: An End-to-End System for Single Image Haze Removal," *IEEE Transactions on Image Processing*, 2016. arXiv
27. B. Li, X. Peng, Z. Wang, J. Xu and D. Feng, "AOD-Net: All-in-One Dehazing Network," in *Proc.*

IEEE ICCV, 2017. doi ·arXiv
28. X. Liu, Y. Ma, Z. Shi and J. Chen, "GridDehazeNet: Attention-Based Multi-Scale Network for Image Dehazing," in *Proc. IEEE/CVF ICCV*, 2019. arXiv
29. X. Qin, Z. Wang, Y. Bai, X. Xie and H. Jia, "FFA-Net: Feature Fusion Attention Network for Single Image Dehazing," in *Proc. AAAI*, 2020. arXiv
30. S. W. Zamir, A. Arora, S. Khan, M. Hayat, F. S. Khan, M.-H. Yang and L. Shao, "Learning Enriched Features for Real Image Restoration and Enhancement," in *Proc. ECCV*, 2020. arXiv
31. Y. Song, Z. He, H. Qian and X. Du, "Vision Transformers for Single Image Dehazing," *IEEE Transactions on Image Processing*, vol. 32, 2023. arXiv
32. B. Li, W. Ren, D. Fu, D. Tao, D. Feng, W. Zeng and Z. Wang, "Benchmarking Single-Image Dehazing and Beyond," *IEEE Transactions on Image Processing*, 2019. arXiv
33. C. O. Ancuti, C. Ancuti, R. Timofte and C. De Vleeschouwer, "O-HAZE: A Dehazing Benchmark with Real Hazy and Haze-Free Outdoor Images," in *Proc. IEEE/CVF CVPR Workshops*, 2018. doi
34. C. O. Ancuti, C. Ancuti, M. Sbert and R. Timofte, "Dense-Haze: A Benchmark for Image Dehazing with Dense-Haze and Haze-Free Images," in *Proc. IEEE ICIP*, 2019. doi
35. A. Geiger, P. Lenz and R. Urtasun, "Are We Ready for Autonomous Driving? The KITTI Vision Benchmark Suite," in *Proc. IEEE CVPR*, 2012. project page
36. Y.-F. Liu, D.-W. Jaw, S.-C. Huang and J.-N. Hwang, "DesnowNet: Context-Aware Deep Network for Snow Removal," *IEEE Transactions on Image Processing*, 2018. arXiv
37. W.-T. Chen, H.-Y. Fang, J.-J. Ding, C.-C. Tsai and S.-Y. Kuo, "JSTASR: Joint Size and Transparency-Aware Snow Removal Algorithm Based on Modified Partial Convolution and Veiling Effect Removal," in *Proc. ECCV*, 2020. doi
38. W. Yang, R. T. Tan, J. Feng, J. Liu, Z. Guo and S. Yan, "Deep Joint Rain Detection and Removal from a Single Image," in *Proc. IEEE CVPR*, 2017. arXiv
39. R. Qian, R. T. Tan, W. Yang, J. Su and J. Liu, "Attentive Generative Adversarial Network for Raindrop Removal from a Single Image," in *Proc. IEEE/CVF CVPR*, 2018. arXiv
40. X. Li, Y. Jin, X. Jin, Z. Wu, B. Li, Y. Wang et al., "NTIRE 2025 Challenge on Day and Night Raindrop Removal for Dual-Focused Images: Methods and Results," in *Proc. IEEE/CVF CVPR Workshops*, 2025. arXiv
41. C. Wei, W. Wang, W. Yang and J. Liu, "Deep Retinex Decomposition for Low-Light Enhancement," in *Proc. BMVC*, 2018. arXiv
42. Y. Cai, H. Bian, J. Lin, H. Wang, R. Timofte and Y. Zhang, "Retinexformer: One-stage Retinex-based Transformer for Low-light Image Enhancement," in *Proc. IEEE/CVF ICCV*, 2023. arXiv
43. H. Zhang, H. Xu, X. Tian, J. Jiang and J. Ma, "Image Fusion Meets Deep Learning: A Survey and Perspective," *Information Fusion*, vol. 76, pp. 323–336, 2021. doi
44. S. Nah, T. H. Kim and K. M. Lee, "Deep Multi-scale Convolutional Neural Network for Dynamic Scene Deblurring," in *Proc. IEEE CVPR*, 2017. doi ·arXiv
45. K. He, X. Chen, S. Xie, Y. Li, P. Dollár and R. Girshick, "Masked Autoencoders Are Scalable Vision Learners," in *Proc. IEEE/CVF CVPR*, 2022. arXiv
46. R. Suvorov, E. Logacheva, A. Mashikhin, A. Remizova, A. Ashukha, A. Silvestrov, N. Kong, H. Goka, K. Park and V. Lempitsky, "Resolution-robust Large Mask Inpainting with Fourier Convolutions," in *Proc. IEEE/CVF WACV*, 2022. arXiv
47. S. Sengupta, V. Jayaram, B. Curless, S. Seitz and I. Kemelmacher-Shlizerman, "Background

Matting: The World is Your Green Screen," in *Proc. IEEE/CVF CVPR*, 2020. arXiv
48. O. Ronneberger, P. Fischer and T. Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," in *Proc. MICCAI*, 2015. arXiv
49. H. Guo, J. Li, T. Dai, Z. Ouyang, X. Ren and S.-T. Xia, "MambaIR: A Simple Baseline for Image Restoration with State-Space Model," in *Proc. ECCV*, 2024. arXiv
50. H. Guo, Y. Guo, Y. Zha, Y. Zhang, W. Li, T. Dai, S.-T. Xia and Y. Li, "MambaIRv2: Attentive State Space Restoration," in *Proc. IEEE/CVF CVPR*, 2025. arXiv
51. B. Li, X. Liu, P. Hu, Z. Wu, J. Lv and X. Peng, "All-In-One Image Restoration for Unknown Corruption," in *Proc. IEEE/CVF CVPR*, 2022. code & paper
52. J. M. J. Valanarasu, R. Yasarla and V. M. Patel, "TransWeather: Transformer-based Restoration of Images Degraded by Adverse Weather Conditions," in *Proc. IEEE/CVF CVPR*, 2022. arXiv
53. V. Potlapalli, S. W. Zamir, S. Khan and F. S. Khan, "PromptIR: Prompting for All-in-One Image Restoration," in *Proc. NeurIPS*, 2023. arXiv
54. Y. Cui, S. W. Zamir, S. Khan, A. Knoll, M. Shah and F. S. Khan, "AdaIR: Adaptive All-in-One Image Restoration via Frequency Mining and Modulation," in *Proc. ICLR*, 2025. arXiv
55. Z. Luo, F. K. Gustafsson, Z. Zhao, J. Sjölund and T. B. Schön, "Controlling Vision-Language Models for Multi-Task Image Restoration," in *Proc. ICLR*, 2024. arXiv
56. M. V. Conde, G. Geigle and R. Timofte, "InstructIR: High-Quality Image Restoration Following Human Instructions," in *Proc. ECCV*, 2024. arXiv
57. H. Zhao, O. Gallo, I. Frosio and J. Kautz, "Loss Functions for Image Restoration with Neural Networks," *IEEE Transactions on Computational Imaging*, 2017 (arXiv title: "Loss Functions for Neural Networks for Image Processing"). arXiv
58. J. Johnson, A. Alahi and L. Fei-Fei, "Perceptual Losses for Real-Time Style Transfer and Super-Resolution," in *Proc. ECCV*, 2016. doi ·arXiv
59. L. Jiang, B. Dai, W. Wu and C. C. Loy, "Focal Frequency Loss for Image Reconstruction and Synthesis," in *Proc. IEEE/CVF ICCV*, 2021. arXiv
60. Z. Wang, A. C. Bovik, H. R. Sheikh and E. P. Simoncelli, "Image Quality Assessment: From Error Visibility to Structural Similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004. doi
61. R. Zhang, P. Isola, A. A. Efros, E. Shechtman and O. Wang, "The Unreasonable Effectiveness of Deep Features as a Perceptual Metric," in *Proc. IEEE/CVF CVPR*, 2018. arXiv
62. A. Mittal, R. Soundararajan and A. C. Bovik, "Making a 'Completely Blind' Image Quality Analyzer," *IEEE Signal Processing Letters*, 2013. doi
63. Y. Blau, R. Mechrez, R. Timofte, T. Michaeli and L. Zelnik-Manor, "The 2018 PIRM Challenge on Perceptual Image Super-resolution," in *Proc. ECCV Workshops*, 2018. arXiv
64. M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler and S. Hochreiter, "GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium," in *Proc. NeurIPS*, 2017. arXiv
65. Y. Blau and T. Michaeli, "The Perception-Distortion Tradeoff," in *Proc. IEEE/CVF CVPR*, 2018. arXiv
66. Z. Wang, J. Chen and S. C. H. Hoi, "Deep Learning for Image Super-Resolution: A Survey," *IEEE TPAMI*, 2021. arXiv
67. C. Tian, L. Fei, W. Zheng, Y. Xu, W. Zuo and C.-W. Lin, "Deep Learning on Image Denoising: An Overview," *Neural Networks*, vol. 131, pp. 251–275, 2020. arXiv
68. M. Elad, B. Kawar and G. Vaksman, "Image Denoising: The Deep Learning Revolution and

Beyond — A Survey Paper," *SIAM Journal on Imaging Sciences*, vol. 16, no. 3, pp. 1594–1654, 2023. doi ·arXiv
69. K. Zhang, W. Ren, W. Luo, W.-S. Lai, B. Stenger, M.-H. Yang and H. Li, "Deep Image Deblurring: A Survey," *International Journal of Computer Vision*, 2022. arXiv
70. J. Gui, X. Cong, Y. Cao, W. Ren, J. Zhang, J. Zhang, J. Cao and D. Tao, "A Comprehensive Survey and Taxonomy on Single Image Dehazing Based on Deep Learning," *ACM Computing Surveys*, 2023. doi ·arXiv
71. W. Yang, R. T. Tan, S. Wang, Y. Fang and J. Liu, "Single Image Deraining: From Model-Based to Data-Driven and Beyond," *IEEE TPAMI*, 2021. doi ·arXiv
72. J. Jiang, Z. Zuo, G. Wu, K. Jiang and X. Liu, "A Survey on All-in-One Image Restoration: Taxonomy, Evaluation and Future Trends," *IEEE TPAMI*, 2025. arXiv
73. X. Li, Y. Ren, X. Jin, C. Lan, X. Wang, W. Zeng, X. Wang and Z. Chen, "Diffusion Models for Image Restoration and Enhancement: A Comprehensive Survey," *International Journal of Computer Vision*, 2025. arXiv
74. A. Levin, Y. Weiss, F. Durand and W. T. Freeman, "Understanding and Evaluating Blind Deconvolution Algorithms," in *Proc. IEEE CVPR*, 2009, pp. 1964–1971. doi
75. R. Fergus, B. Singh, A. Hertzmann, S. T. Roweis and W. T. Freeman, "Removing Camera Shake from a Single Photograph," *ACM Transactions on Graphics* (Proc. SIGGRAPH), vol. 25, no. 3, pp. 787–794, 2006. doi
76. Q. Shan, J. Jia and A. Agarwala, "High-quality Motion Deblurring from a Single Image," *ACM Transactions on Graphics* (Proc. SIGGRAPH), vol. 27, no. 3, 2008. doi
77. S. Cho and S. Lee, "Fast Motion Deblurring," *ACM Transactions on Graphics* (Proc. SIGGRAPH Asia), vol. 28, no. 5, 2009. doi
78. L. Xu and J. Jia, "Two-Phase Kernel Estimation for Robust Motion Deblurring," in *Proc. ECCV*, 2010, pp. 157–170. doi
79. D. Krishnan, T. Tay and R. Fergus, "Blind Deconvolution Using a Normalized Sparsity Measure," in *Proc. IEEE CVPR*, 2011, pp. 233–240. doi
80. J. Pan, D. Sun, H. Pfister and M.-H. Yang, "Blind Image Deblurring Using Dark Channel Prior," in *Proc. IEEE CVPR*, 2016, pp. 1628–1636. doi
81. J. Sun, W. Cao, Z. Xu and J. Ponce, "Learning a Convolutional Neural Network for Non-uniform Motion Blur Removal," in *Proc. IEEE CVPR*, 2015. arXiv
82. C. J. Schuler, M. Hirsch, S. Harmeling and B. Schölkopf, "Learning to Deblur," *IEEE TPAMI*, vol. 38, no. 7, pp. 1439–1451, 2016. doi ·arXiv
83. A. Chakrabarti, "A Neural Approach to Blind Motion Deblurring," in *Proc. ECCV*, 2016. arXiv
84. D. Ulyanov, A. Vedaldi and V. Lempitsky, "Deep Image Prior," in *Proc. IEEE/CVF CVPR*, 2018, pp. 9446–9454. doi ·arXiv
85. D. Ren, K. Zhang, Q. Wang, Q. Hu and W. Zuo, "Neural Blind Deconvolution Using Deep Priors," in *Proc. IEEE/CVF CVPR*, 2020. arXiv
86. O. Kupyn, V. Budzan, M. Mykhailych, D. Mishkin and J. Matas, "DeblurGAN: Blind Motion Deblurring Using Conditional Adversarial Networks," in *Proc. IEEE/CVF CVPR*, 2018. arXiv
87. K. Zhang, W. Zuo and L. Zhang, "Learning a Single Convolutional Super-Resolution Network for Multiple Degradations," in *Proc. IEEE/CVF CVPR*, 2018. arXiv
88. J. Gu, H. Lu, W. Zuo and C. Dong, "Blind Super-Resolution With Iterative Kernel Correction," in *Proc. IEEE/CVF CVPR*, 2019. arXiv

89. S. Bell-Kligler, A. Shocher and M. Irani, “Blind Super-Resolution Kernel Estimation using an Internal-GAN,” in *Proc. NeurIPS*, 2019. link ·arXiv
90. Z. Luo, Y. Huang, S. Li, L. Wang and T. Tan, “Unfolding the Alternating Optimization for Blind Super Resolution,” in *Proc. NeurIPS*, 2020. arXiv
91. K. Zhang, L. Van Gool and R. Timofte, “Deep Unfolding Network for Image Super-Resolution,” in *Proc. IEEE/CVF CVPR*, 2020. arXiv
92. K. Zhang, J. Liang, L. Van Gool and R. Timofte, “Designing a Practical Degradation Model for Deep Blind Image Super-Resolution,” in *Proc. IEEE/CVF ICCV*, 2021. arXiv
93. A. Liu, Y. Liu, J. Gu, Y. Qiao and C. Dong, “Blind Image Super-Resolution: A Survey and Beyond,” *IEEE TPAMI*, 2022 (early access). doi ·arXiv
94. A. Mittal, A. K. Moorthy and A. C. Bovik, “No-Reference Image Quality Assessment in the Spatial Domain,” *IEEE Transactions on Image Processing*, vol. 21, no. 12, pp. 4695–4708, 2012. doi
95. C. Ma, C.-Y. Yang, X. Yang and M.-H. Yang, “Learning a No-Reference Quality Metric for Single-Image Super-Resolution,” *Computer Vision and Image Understanding*, vol. 158, pp. 1–16, 2017. doi ·arXiv
96. N. Venkatanath, D. Praneeth, M. C. Bh, S. S. Channappayya and S. S. Medasani, “Blind Image Quality Evaluation Using Perception Based Features,” in *Proc. Twenty First National Conference on Communications (NCC)*, 2015, pp. 1–6. doi ·MATLAB documentation
97. H. Talebi and P. Milanfar, “NIMA: Neural Image Assessment,” *IEEE Transactions on Image Processing*, vol. 27, no. 8, pp. 3998–4011, 2018. doi ·arXiv
98. S. Su, Q. Yan, Y. Zhu, C. Zhang, X. Ge, J. Sun and Y. Zhang, “Blindly Assess Image Quality in the Wild Guided by a Self-Adaptive Hyper Network,” in *Proc. IEEE/CVF CVPR*, 2020, pp. 3664–3673. doi
99. J. Ke, Q. Wang, Y. Wang, P. Milanfar and F. Yang, “MUSIQ: Multi-scale Image Quality Transformer,” in *Proc. IEEE/CVF ICCV*, 2021, pp. 5128–5137. doi ·arXiv
100. S. Yang, T. Wu, S. Shi, S. Lao, Y. Gong, M. Cao, J. Wang and Y. Yang, “MANIQA: Multi-dimension Attention Network for No-Reference Image Quality Assessment,” in *Proc. IEEE/CVF CVPR Workshops*, 2022, pp. 1190–1199. doi ·arXiv
101. C. Chen, J. Mo, J. Hou, H. Wu, L. Liao, W. Sun, Q. Yan and W. Lin, “TOPIQ: A Top-Down Approach From Semantics to Distortions for Image Quality Assessment,” *IEEE Transactions on Image Processing*, vol. 33, pp. 2404–2418, 2024. doi ·arXiv
102. J. Wang, K. C. K. Chan and C. C. Loy, “Exploring CLIP for Assessing the Look and Feel of Images,” in *Proc. AAAI Conference on Artificial Intelligence*, 2023. arXiv
103. H. Wu, Z. Zhang, W. Zhang, C. Chen, L. Liao, C. Li, Y. Gao, A. Wang, E. Zhang, W. Sun, Q. Yan, X. Min, G. Zhai and W. Lin, “Q-Align: Teaching LMMs for Visual Scoring via Discrete Text-Defined Levels,” in *Proc. ICML*, PMLR 235, pp. 54015–54029, 2024. link ·arXiv
104. H. R. Sheikh, M. F. Sabir and A. C. Bovik, “A Statistical Evaluation of Recent Full Reference Image Quality Assessment Algorithms,” *IEEE Transactions on Image Processing*, vol. 15, no. 11, pp. 3440–3451, 2006 (LIVE Image Quality Assessment Database). doi ·database
105. N. Ponomarenko, L. Jin, O. Ieremeiev, V. Lukin, K. Egiazarian, J. Astola, B. Vozel, K. Chehdi, M. Carli, F. Battisti and C.-C. J. Kuo, “Image Database TID2013: Peculiarities, Results and Perspectives,” *Signal Processing: Image Communication*, vol. 30, pp. 57–77, 2015. doi ·database
106. H. Lin, V. Hosu and D. Saupe, “KADID-10k: A Large-scale Artificially Distorted IQA

Database," in *Proc. Eleventh International Conference on Quality of Multimedia Experience (QoMEX)*, 2019. doi ·database
107. V. Hosu, H. Lin, T. Sziranyi and D. Saupe, "KonIQ-10k: An Ecologically Valid Database for Deep Learning of Blind Image Quality Assessment," *IEEE Transactions on Image Processing*, vol. 29, pp. 4041–4056, 2020. arXiv
108. Y. Fang, H. Zhu, Y. Zeng, K. Ma and Z. Wang, "Perceptual Quality Assessment of Smartphone Photography," in *Proc. IEEE/CVF CVPR*, 2020, pp. 3674–3683. doi
109. J. Gu, H. Cai, H. Chen, X. Ye, J. Ren and C. Dong, "PIPAL: A Large-Scale Image Quality Assessment Dataset for Perceptual Image Restoration," in *Proc. ECCV*, 2020. arXiv
110. C. Chen and J. Mo, "IQA-PyTorch: PyTorch Toolbox for Image Quality Assessment," software (pip package `pyiqa`), 2022. GitHub

DLCV 05 • Low-Light Image Enhancement

From histogram equalization and Retinex to curve estimation, Retinex transformers and diffusion models: how to brighten dark photos without amplifying noise, shifting colors or fooling the metrics.

Online edition: https://blog.shyandram.dev/en/learn/deep-learning/dl-05-low-light-enhancement/

What you'll learn

- Why low-light image enhancement (LLIE) is two problems at once, **tone enhancement** and **detail and noise restoration**, and why "just make it brighter" fails.
- How the classical toolbox works: brightness channels in HSV, HSI, YCbCr and Lab; histogram equalization and CLAHE; gamma curves; Retinex (SSR, MSR, LIME); and the dehazing-inversion trick.
- How deep LLIE methods are organized by their **training signal** (supervised, unrolled, unpaired, zero-reference, generative), with the math of the Retinex-Net decomposition losses and the Zero-DCE curve.
- How to evaluate enhancement honestly: full-reference metrics, no-reference metrics (NIQE, BRISQUE, PI, lightness order error, learned assessors) and user studies, the **GT-mean** pitfall, and task-driven evaluation.
- Which datasets and challenges (LOL, LOL-v2, LSRW, SDSD, SID, FiveK, ExDark, NTIRE 2024/2025) are used for what, and what remains unsolved.

The big picture

A photo taken in the dark is not just a normal photo with the brightness turned down. The sensor collected very few photons, so the picture is dim, noisy, and often has a color cast. Low-light image enhancement aims to produce the image you *would* have gotten with good light: well exposed, with natural colors, clean details, and no new artifacts.

> **Explain like I'm five**
>
> Imagine you are drawing in a room with the lights almost off. Your drawing comes out faint and smudgy, and you are not sure which crayon was which color. Turning on the lights later and photocopying the drawing "brighter" won't help: the smudges get brighter too, and the wrong colors stay wrong. A good low-light enhancer is like a friend who looks at your faint drawing and *redraws* it as it would look in daylight: same picture, right colors, without the smudges.

Why it matters: phones shoot in the dark every day, cars and robots must see at night, and surveillance, astronomy-like scientific imaging and medical endoscopy all fight the same physics. Low

light also hurts downstream models: a detector trained on daytime photos loses accuracy at night. That makes LLIE both a picture-quality task and a robustness task for computer vision.

About this tutorial

This tutorial is expanded from the author's earlier Medium post on LLIE [1], which is written in Traditional Chinese (read it here). It follows the post's outline and adds math, code, figures and a survey of the literature.

Goal and difficulty

Plain version. We want to map a dark image to a normally exposed one. The hard part is that brightening and repairing are different jobs: brightening changes *tone* (how light or dark each region is), while repairing recovers *details and clean colors* that the darkness and noise destroyed.

Precise version. Unlike image-to-image translation, which changes an image's *attributes* (summer to winter, photo to painting), LLIE should keep the scene and its content exactly as they are and only undo the damage caused by insufficient light. The post [1] splits that damage into two difficulties:

1. **Tone enhancement.** Lift the exposure, globally and locally, without crushing highlights or flattening contrast. Bright regions such as lamps must stay bright, dark regions must open up, and the ordering of brightness between regions should be preserved.
2. **Detail restoration.** Recover texture and color that are buried in noise and quantization, while suppressing that noise rather than amplifying it.

In deep learning, detail restoration is what a network does anyway once it is trained to output a clean image, so most method design goes into the *tone* part (which image properties or physical model to build in) and the *learning mechanism* (which supervision and losses to use), rather than into a dedicated detail module.

Why it is not simple brightening

A linear gain $y = c\,x$ with $c > 1$ fails in four ways, all visible in Figure 1:

- **Noise.** Photon arrival is random. With few photons, the signal-to-noise ratio is low, and a gain multiplies signal and noise equally. The image gets brighter, not cleaner.
- **Color cast.** Low light is often colored (tungsten, sodium street lamps), and the camera's white balance and the three color channels have different noise levels. Scaling the channels equally keeps the cast; scaling them separately may produce strange colors.
- **Saturation and over-exposure.** Pixels already near the top (lamps, windows, screens) clip at 1. A global gain big enough for the shadows destroys the highlights.
- **Quantization.** A dark 8-bit image may use only 20–40 distinct gray levels. Stretching them produces visible banding.

A model of low-light capture

It helps to have an explicit degradation model, both for understanding and for **synthesizing training data**. Work in *linear* light (values proportional to photon counts), not on gamma-encoded pixels (see IP 03 on gamma). A widely used model is the Poisson–Gaussian model of Foi et al. [7]:

$$z = \frac{1}{\eta}\,\mathcal{P}(\eta\, k\, x) + n, \qquad n \sim \mathcal{N}(0, \sigma_r^2),$$

where $x \in [0,1]$ is the clean linear image, $k \in (0,1]$ the fraction of light that reached the sensor, η a scale that converts intensity to photon counts (a larger η means more photons and less relative noise), $\mathcal{P}(\cdot)$ a Poisson random variable modeling **shot noise**, and n Gaussian **read noise** with standard deviation σ_r. The camera then applies a tone curve and quantizes. The variance of z is approximately $kx/\eta + \sigma_r^2$: it grows with the signal, so noise is *signal-dependent*, which is why "add Gaussian noise" is a poor stand-in for real low light.

```
import numpy as np
from skimage import data, img_as_float

rng = np.random.default_rng(0)
clean = img_as_float(data.coffee())            # H x W x 3, sRGB-encoded, in [0, 1]

def synth_low_light(x, k=0.05, peak=2000.0, read_std=8e-4, gamma=2.2):
    lin = k * x ** gamma                           # 1) less light, in linear units
    lin = rng.poisson(lin * peak) / peak           # 2) shot noise (Poisson)
    lin = lin + rng.normal(0, read_std, lin.shape) # 3) read noise (Gaussian)
    out = np.clip(lin, 0, 1) ** (1 / gamma)        # 4) re-encode for display
    return np.round(out * 255) / 255               # 5) 8-bit quantization

dark = synth_low_light(clean)
print(clean.mean(), dark.mean())                   # about 0.39 -> 0.10
```

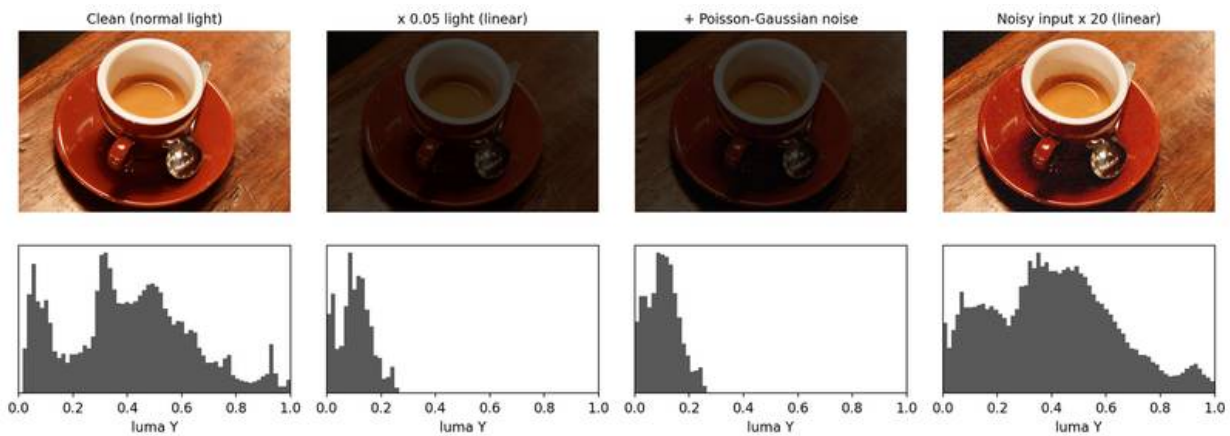


Figure 1 — Synthetic low light. Removing 95% of the light squeezes the histogram into the bottom quarter. Noise is hard to see in the dark image, but undoing the exposure with a perfect gain (right) reveals it: brightening and denoising are separate problems.

Interactive demonstration (online): open the online edition of this tutorial to use this demonstration.Open demonstration

The right panel is the most important picture in this tutorial: even the *ideal* brightness correction leaves the noise, which is why LLIE is a restoration problem (see DLCV 04 ·Image Restoration) and not only a tone-mapping problem.

Non-deep-learning LLIE and the characteristics of low-light images

Plain version. Before deep learning there were two main families: **histogram-based** methods, which redistribute gray levels, and **Retinex-based** methods, which split an image into "lighting" and "objects" and fix the lighting. A third idea, borrowed from dehazing, fits into the Retinex family, and statistical methods extend all of them.

Low-light images in color space

Many color spaces separate a "how bright" channel from "what color" channels. Different spaces call it value, intensity, luma or lightness, and each defines it differently. With $R, G, B \in [0, 1]$ (see IP 06 for the spaces themselves):

$$\begin{aligned} \text{HSV:} \quad & V = \max(R, G, B), \\ \text{HSI:} \quad & I = \tfrac{1}{3}(R + G + B), \\ \text{YCbCr (BT.601):} \quad & Y' = 0.299\,R' + 0.587\,G' + 0.114\,B', \\ \text{CIELAB:} \quad & L^* = 116 f(Y_{\text{lin}}/Y_n) - 16. \end{aligned}$$

Here R', G', B' are gamma-encoded values; Y_{lin} is linear-light luminance and Y_n that of the reference white; $f(t) = t^{1/3}$ above a small threshold and linear below it. Some remarks:

- V **is the "bright channel".** It is the per-pixel maximum, the counterpart of the *dark channel* (a per-patch minimum) used in dehazing [14]. In a dark image, V is the channel least damaged by the darkness, which is why LIME [17] uses it to initialize the illumination.
- **The luma weights.** ITU-R BT.601 [8] defines luma as $Y' = 0.299R' + 0.587G' + 0.114B'$. That full-range form is what JPEG/JFIF uses. For 8-bit *limited-range* ("studio") video, BT.601 places black at 16 and white at 235: $Y = 16 + 219\,Y'$. With $R', G', B' \in [0, 255]$ this becomes $Y = 16 + 0.257R' + 0.504G' + 0.098B'$, because $219/255 \times (0.299, 0.587, 0.114) \approx (0.257, 0.504, 0.098)$. Both formulas describe the same luma; they differ only in range and offset.
- L^* is designed to be perceptually uniform, so equal steps look roughly equal to a person. That makes it a good channel for contrast enhancement.

The practical lesson: since brightness can be separated from color, you can **enhance only the brightness channel** and keep the chroma. Figure 2 shows that the choice of channel matters a lot.

```
R, G, B = dark[..., 0], dark[..., 1], dark[..., 2]
V = dark.max(axis=2)                          # HSV value
```

```
I = dark.mean(axis=2)                          # HSI intensity
Y = 0.299 * R + 0.587 * G + 0.114 * B          # BT.601 luma, full range (JPEG/JFIF)
Y_limited = 16 + 219 * Y                       # 8-bit limited ("studio") range, 16..235
print(np.allclose(Y_limited, 16 + (65.481 * R + 128.553 * G + 24.966 * B)))  # True
```

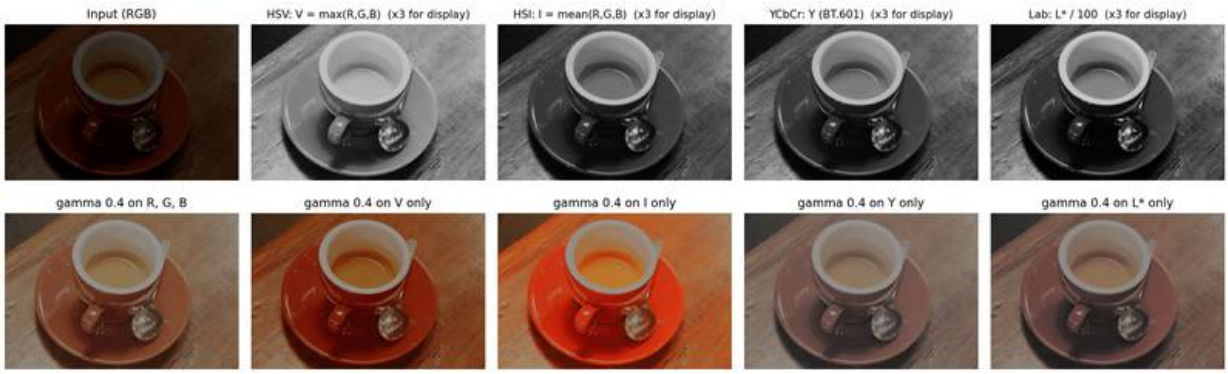


Figure 2 — The same gamma curve applied in different color spaces. Raising V or I scales all three channels by one factor, which keeps hue but strengthens saturation (very orange). Raising Y or L with fixed chroma desaturates the result. Neither is 'correct': tone and color are coupled.*

Two effects are visible. Scaling RGB by a common factor (HSV, HSI) preserves hue and keeps the ratio between channels, so saturated objects become *very* saturated. Changing only Y' or L^* while keeping Cb/Cr or a*/b* fixed adds brightness without adding color, so the image looks washed out. This coupling of tone and color is exactly what later methods such as HVI-CIDNet [27] try to handle with a learned color space.

Histogram equalization and CLAHE

Plain version. Histogram equalization (HE) spreads the gray levels of an image so that all brightness values are used about equally. A dark image, whose values are packed near zero, gets stretched across the full range.

Precise version. For an image with L levels, normalized histogram $p_r(r_k) = n_k/(MN)$, HE uses the scaled cumulative distribution function as the mapping:

$$s_k = T(r_k) = (L-1)\sum_{j=0}^{k} p_r(r_j), \qquad k = 0, \dots, L-1,$$

where n_k is the number of pixels at level r_k and MN the image size. The slope of T is proportional to the histogram, so heavily populated gray levels get pulled apart. The derivation is in IP 03. For color images, the usual recipe is to convert to a space with a brightness channel, equalize that channel only, and convert back, for example RGB → Lab → equalize L^* → RGB.

Global HE has two problems in low light: it over-stretches large flat regions (including their noise), and one mapping cannot serve both a dark corner and a bright lamp. **Adaptive HE** computes a mapping per tile, and **CLAHE** (contrast-limited AHE) clips each tile's histogram at a

limit before computing the CDF, which caps the slope of T and therefore the noise gain, then interpolates between neighboring tiles to avoid seams [9], [10].

Gamma correction

A power law $s = r^{\gamma}$ with $\gamma < 1$ lifts shadows more than highlights; its slope $\gamma r^{\gamma-1}$ is large near $r = 0$. It is cheap and monotonic but global: one γ for the whole picture, no denoising, and the same desaturation-or-oversaturation choice as above depending on where it is applied.

```
from skimage import color, exposure

gamma_out = dark ** 0.4                                    # power-law curve on every
 ↪  channel
lab = color.rgb2lab(dark)
L = lab[..., 0] / 100                                      # L* scaled to [0, 1]
he = lab.copy();    he[..., 0] = 100 * exposure.equalize_hist(L)
clahe = lab.copy(); clahe[..., 0] = 100 * exposure.equalize_adapthist(L, clip_limit=0.01)
he_out, clahe_out = color.lab2rgb(he), color.lab2rgb(clahe)
for name, im in [("gamma", gamma_out), ("HE", he_out), ("CLAHE", clahe_out)]:
    print(f"{name:6s} mean {im.mean():.3f}   std {im.std():.3f}")
```

Figure 3 — Classical enhancers on the noisy input of Figure 1. All of them brighten; none of them removes noise, and the Retinex outputs show strong color noise where the input was darkest.

Retinex theory

Plain version. Retinex says that what the eye (or camera) receives is "the object's own colors" multiplied by "the light falling on it". If we can estimate the light, we can divide it out and see

the object as it really is, then put back a better light.

Precise version. Land's Retinex theory [11] models an observed image S as

$$S(\mathbf{x}) = R(\mathbf{x}) \circ L(\mathbf{x}),$$

where $\mathbf{x}$ is the pixel position, $R \in [0,1]$ is the **reflectance** (a property of the surfaces, assumed constant under any lighting), L is the **illumination** (the light falling on the scene, which varies smoothly in space except at shadow boundaries), and $\circ$ is element-wise multiplication. Note the word: L is *illumination*, not luminance. "Luminance" usually refers to a brightness channel of a color space, such as Y above.

Recovering two unknowns from one product is ill-posed, so every Retinex method adds a prior. The classic one is "illumination is smooth".

Single-scale Retinex (SSR) [12] estimates L with a Gaussian blur G_σ and works in the log domain, where the product becomes a sum:

$$\log R_c(\mathbf{x}) = \log S_c(\mathbf{x}) - \log\left(G_\sigma * S_c\right)(\mathbf{x}),$$

for each color channel c, with $*$ convolution and σ the surround scale. Small σ gives strong local contrast and halos at edges; large σ gives better tone but weaker detail. **Multi-scale Retinex (MSR)** [13] averages SSR outputs at several scales (typically three) with weights w_n:

$$\log R_c^{\mathrm{MSR}} = \sum_{n=1}^{N} w_n \left[\log S_c - \log(G_{\sigma_n} * S_c)\right].$$

Because SSR and MSR work per channel, they tend to gray out the colors. **MSRCR** (MSR with color restoration) [13] multiplies the result by a color-restoration factor based on each channel's share of $R + G + B$; other variants apply MSR to an intensity channel only and keep the input's chromaticity.

```
from scipy import ndimage as ndi

def ssr(img, sigma=40.0, eps=1e-3):
    log_s = np.log(img + eps)
    log_l = np.log(ndi.gaussian_filter(img, sigma=(sigma, sigma, 0)) + eps)  # L ~ Gaussian
     ↪  blur
    r = log_s - log_l                                          # log R = log S - log L
    lo, hi = np.percentile(r, [1, 99])                         # robust stretch for display
    return np.clip((r - lo) / (hi - lo), 0, 1)

msr = np.mean([ssr(dark, s) for s in (15, 80, 250)], axis=0)   # MSR, equal weights
 ↪  (simplified)
```

For *enhancement* we usually do not want R alone, which looks flat and unnatural. We want to brighten L and recombine: $\hat{S} = R \circ \hat{L}$ with, for example, $\hat{L} = L^\gamma$ and $\gamma < 1$. This "decompose, adjust L, recombine" pattern is the template for most Retinex-based deep networks.

Figure 4 — A LIME-style Retinex decomposition. The illumination is initialized with max(R, G, B) and smoothed; the reflectance is S / L. Brightening L with a gamma of 0.35 and multiplying back gives a natural result, while R alone looks flat and over-saturated.

LIME: Retinex as optimization

LIME [17] estimates only the illumination. It initializes it with the bright channel, $\hat{T}(\mathbf{x}) = \max_c S_c(\mathbf{x})$, then refines it by solving a structure-aware smoothing problem: keep T close to $\hat{T}$, but make it smooth except where the image has strong structure. The enhanced image is S/T^γ, and because dividing amplifies noise, LIME optionally denoises afterward. LIME's images (often called the LIME set) became a standard test set, and the method is a good example of the classical recipe: a physical model, a hand-designed prior, and an optimization.

Another model: dehazing an inverted image

The Retinex model is not the only image-formation model. Dehazing uses the **atmospheric scattering model**:

$$I(\mathbf{x}) = J(\mathbf{x})\, t(\mathbf{x}) + A\,(1 - t(\mathbf{x})),$$

where I is the hazy observation, J the haze-free scene radiance, $t \in (0, 1]$ the transmission (how much scene light survives), and A the global atmospheric light. This model cannot be applied to a dark image directly, because a dark image does not look like a hazy one.

Dong et al. observed that an **inverted** low-light image, $1 - S$, *does* look hazy: dark regions become bright and low-contrast, like fog. They therefore invert the image, run a dehazing algorithm (in the spirit of the dark channel prior [14]), and invert the result back [15], [16]. In the notation above,

$$I_{\text{haze}} = 1 - S_{\text{Retinex}}, \qquad J_{\text{haze}} = 1 - \hat{S}_{\text{enhanced}}.$$

From the "what is reliable" angle, dehazing assumes the *dark* channel marks haze-free content; in a low-light image the dark channel is mostly damaged pixels, so attention shifts to the *bright* channel, the region least affected by the darkness. The LIME paper [17] analyzes this inversion approach and shows that it can be rewritten in Retinex form, so the two models make essentially the same assumption, with Retinex being the simpler statement.

Statistical approaches

Many classical methods add statistics or priors on top of these models: naturalness constraints that keep the lightness order of a scene (NPE [18]), layered or two-dimensional histogram models (the method behind the DICM set [59]), and fusion of several synthetic exposures. They are hand-crafted, need no training data, and generalize in a predictable way; their weak point is noise, which they usually ignore or hand to a separate denoiser.

Deep-learning LLIE

Plain version. Deep networks learn the mapping from dark to bright from data. The question is not whether to use a network but **what training signal** to use: paired dark/bright photos, unpaired photos, no reference at all, or a generative model of what bright images look like. Physics (usually Retinex) can be built into any of these.

Deep learning is itself a large optimization, and its black-box nature lets us drop many hand-made assumptions. A common first split is **theory-based** methods (usually Retinex) versus **theory-free** methods (strong generic networks). In practice the two mix, so the split is not binary. Figure 5 groups methods mainly by training signal instead, following the surveys [2], [3].

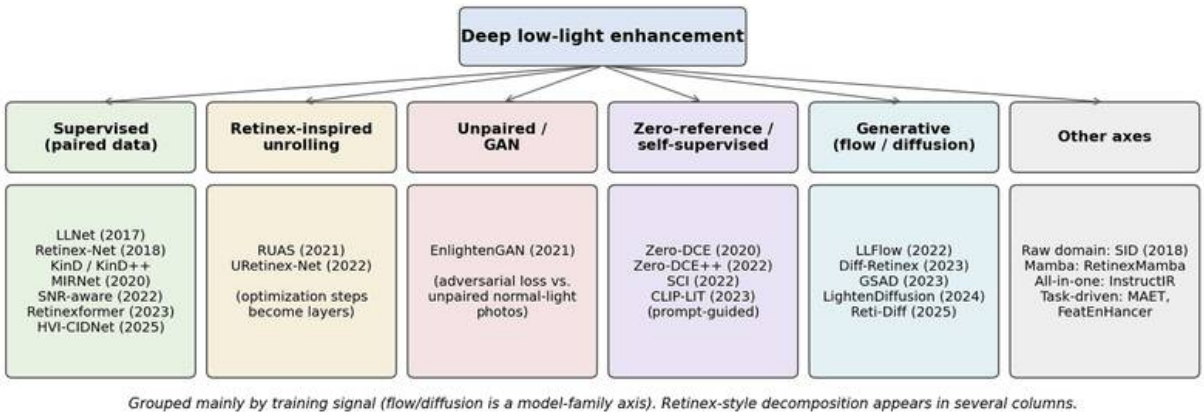


Figure 5 — A taxonomy of deep LLIE, mainly by training signal (the flow/diffusion column is a model-family axis). Retinex decomposition is an architectural choice that appears in several columns, not a column of its own.

Supervised methods (paired data)

Given pairs (x_i, y_i) of low-light and normal-light images, train f_θ to minimize a reconstruction loss, typically $\mathcal{L} = \|f_\theta(x) - y\|_1$ plus perceptual or structural terms. Paired data is expensive (two exposures of a static scene, or synthesis), and the network learns the *specific* brightness and color style of the reference images, which matters for evaluation later.

- **LLNet** [19] (2017) was one of the first deep LLIE models: a stacked sparse denoising autoencoder trained on synthetically darkened and noised patches, learning brightening and denoising jointly.
- **CNN + bright channel prior** (Tao et al. [20], 2017) handled the two difficulties in sequence: a CNN denoises first, then contrast is enhanced with an atmospheric-scattering-style model driven by the bright channel, an early example of combining learning with the inversion idea above.
- **MIRNet** [24] (2020) is a general restoration backbone that keeps a full-resolution stream alongside parallel lower-resolution streams, exchanges information between them, fuses them with selective-kernel attention, and was evaluated on enhancement as well as denoising and super-resolution.
- **SNR-aware** [25] (2022) estimates a per-pixel signal-to-noise map. Where SNR is high, local convolution is enough; where SNR is very low, the local neighborhood is unreliable, so the model uses long-range transformer attention to borrow information from elsewhere in the image.
- **HVI-CIDNet** [27] (2025) attacks the color-space problem of Figure 2: sRGB is very sensitive to color and gives color bias and brightness artifacts, and HSV fixes brightness but creates red and black noise. The authors propose the HVI space, built from polarized hue/saturation maps and a *learnable* intensity, plus a network that decouples color and intensity.

Retinex-based networks

Retinex-Net [21] (2018) changed the goal from "estimate R" to "**both the input and the output have their own R and L**". The pipeline is: decompose the low and normal images, enhance L to $\hat{L}$, recombine with R. It also introduced the LOL dataset (below). Its decomposition network is trained only through consistency, without ground-truth R or L, using

$$\mathcal{L}_{\text{decom}} = \mathcal{L}_{\text{recon}} + \lambda_{ir}\,\mathcal{L}_{ir} + \lambda_{is}\,\mathcal{L}_{is},$$

$$\mathcal{L}_{\text{recon}} = \sum_{i\in\{\text{low,normal}\}} \sum_{j\in\{\text{low,normal}\}} \lambda_{ij} \left\| R_i \circ I_j - S_j \right\|_1,$$

$$\mathcal{L}_{ir} = \|R_{\text{low}} - R_{\text{normal}}\|_1, \qquad \mathcal{L}_{is} = \sum_i \left\| \nabla I_i \circ \exp\left(-\lambda_g \nabla R_i\right) \right\|.$$

In the paper's notation $S = R \circ I$, so I here is the illumination (L in our notation). S_j is the input image j, R_i and I_i the decomposed reflectance and illumination of image i, and ∇ the horizontal plus vertical gradient. The terms say:

- $\mathcal{L}_{\text{recon}}$: each decomposition must reproduce its own image ($\lambda_{ii} = 1$), and crossing reflectance and illumination between the pair should still roughly work ($\lambda_{ij} = 0.001$ for $i \neq j$).
- $\mathcal{L}_{ir}$: the two images show the same scene, so their reflectances should be equal; this is how the network learns what "reflectance" means.
- $\mathcal{L}_{is}$: illumination should be smooth, *except* where the reflectance has a strong edge. The weight $\exp(-\lambda_g \nabla R)$ relaxes the smoothness penalty across object boundaries. This is the learned version of LIME's structure-aware prior.

The paper uses $\lambda_{ir} = 0.001$, $\lambda_{is} = 0.1$ and $\lambda_g = 10$. The enhancement network is trained with $\|R_{\text{low}} \circ \hat{I} - S_{\text{normal}}\|_1$ plus the same smoothness term.

The pattern spread quickly, and many names contain "Retinex":

- **KinD** [22] (2019) splits the work into three subnetworks: decomposition, *reflectance restoration* (because the dark image's reflectance is noisy and color-shifted, the "R is unchanged" assumption is relaxed), and illumination adjustment with a user-controllable brightness ratio. **KinD++** [23] (2021) refines the design to reduce visual defects.
- **Retinexformer** [26] (2023) notes that both capture and brightening corrupt the components, and rewrites the model with perturbations: $S = (R + \hat{R}) \circ (L + \hat{L})$, where $\hat{R}$ and $\hat{L}$ model noise, artifacts and color distortion. It is trained in **one stage** (instead of Retinex-Net's multi-stage training) with an L1 loss, and its **illumination-guided multi-head self-attention** uses the illumination estimate to steer attention, so regions with different exposure interact. It became a common baseline.
- **RetinexMamba** [42] (2024 preprint) keeps the Retinex-transformer structure and replaces attention with Mamba state-space blocks, which scale linearly with the number of pixels. "Retinex decomposition plus a new backbone" has been a stable design pattern for years.

A less common route builds on the atmospheric scattering model rather than plain Retinex. CPGA-Net [28] combines channel priors with gamma correction in a lightweight network that processes global and local information, and CPGA-Net+ [29] revisits the theoretical illumination model for efficiency. That line of work also reports that the features learned by different functional modules follow the physical roles of the priors, which suggests that a network guided by a model is not a completely opaque black box.

Retinex-inspired unrolling

Unrolling (or unfolding) takes an iterative optimization algorithm for the Retinex problem and turns each iteration into a network stage with learnable parameters. The architecture inherits the algorithm's structure and interpretability, and training replaces hand-tuned priors.

- **RUAS** [30] (2021) unrolls optimization models for illumination estimation and noise removal, searches the architecture of the prior modules, and is trained with a cooperative *reference-free* strategy, so it also belongs in the unsupervised column. It is very small and fast.
- **URetinex-Net** [31] (2022) unfolds a Retinex decomposition optimization into a network with learned modules for initialization, the unfolded updates of R and L, and illumination adjustment.

Unpaired training: EnlightenGAN

EnlightenGAN [32] (2021) uses a generative adversarial network (GAN) so that it needs only a set of dark images and an *unrelated* set of well-lit images. A U-Net generator, guided by a self-regularized attention map derived from the input's illumination, produces the enhanced image.

A **global** discriminator judges the whole image and a **local** discriminator judges random patches, so the output looks like a well-lit photo both overall and in detail. A **self-feature-preserving** loss compares VGG features of input and output, keeping content in place since there is no ground truth to do so.

Zero-reference methods and curve estimation

Plain version. Zero-DCE does not output a new image. It outputs a *brightening curve per pixel*, applies it, and is trained with losses that judge the result without any reference photo: "is it well exposed, is the color balanced, is the curve smooth, is local contrast kept?"

The LE curve. Zero-DCE [33] (2020) defines a quadratic **light-enhancement curve**

$$\mathrm{LE}(x;\alpha) = x + \alpha\, x\,(1-x), \qquad \alpha \in [-1,1],$$

where $x \in [0,1]$ is a pixel value. The curve has three useful properties: $\mathrm{LE}(0) = 0$ and $\mathrm{LE}(1) = 1$, so the range is preserved and no clipping is needed; it is monotonic for $|\alpha| \le 1$, because $\mathrm{d}\,\mathrm{LE}/\mathrm{d}x = 1 + \alpha(1-2x) \ge 0$, so brightness order is kept; and it is differentiable, so it can be trained by backpropagation. One quadratic is not flexible enough, so it is applied **iteratively**:

$$\mathrm{LE}_n(\mathbf{x}) = \mathrm{LE}_{n-1}(\mathbf{x}) + \mathcal{A}_n(\mathbf{x})\,\mathrm{LE}_{n-1}(\mathbf{x})(1 - \mathrm{LE}_{n-1}(\mathbf{x})),$$

with LE_0 the input, $n = 1, \ldots, 8$, and $\mathcal{A}_n$ a **per-pixel, per-channel** parameter map instead of a single α. A seven-layer CNN (DCE-Net, 32 channels per layer, about 79k parameters, no downsampling) predicts $8 \times 3 = 24$ maps with a tanh output. Each pixel thus gets its own high-order curve, which is how local adjustment arises from a global-looking formula.

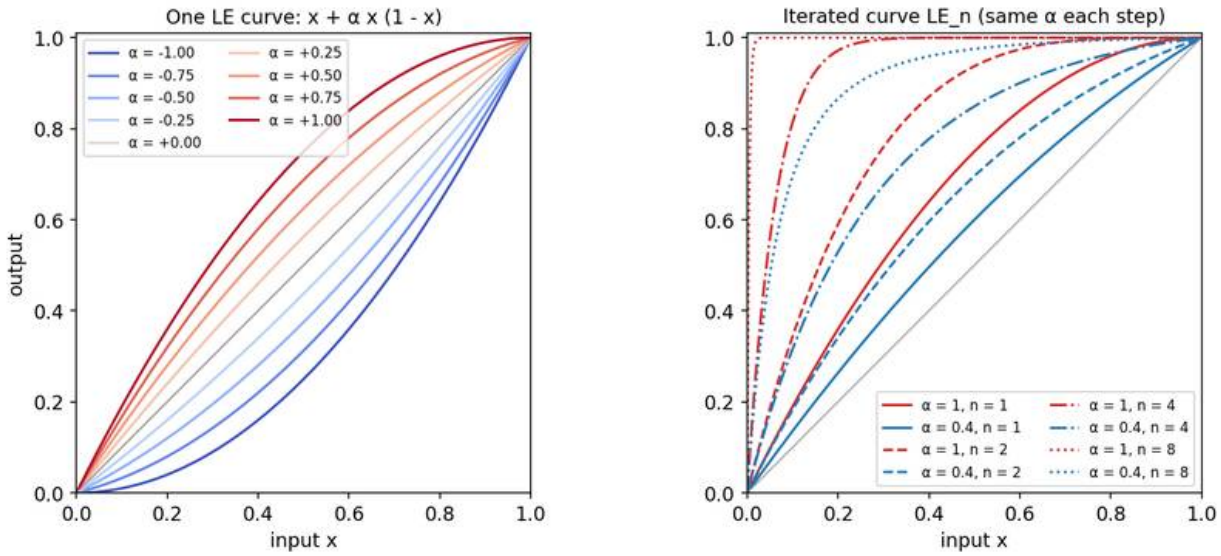


Figure 6 — The LE curve. Left: one iteration for α from −1 to 1. Right: iterating sharpens the curve; with $\alpha = 1$ and $n = 8$, a value of 0.02 is already mapped to almost 1, so tiny noisy values in the shadows are amplified enormously.

```
def le_iterate(x, alphas):
    """Zero-DCE curve: LE_n = LE_{n-1} + a_n * LE_{n-1} * (1 - LE_{n-1})."""
    y = x.copy()
    for a in alphas:              # a can be a scalar or a per-pixel map
        y = y + a * y * (1 - y)
    return y

x = np.array([0.0, 0.02, 0.1, 0.3, 0.6, 1.0])
print(le_iterate(x, [1.0] * 8).round(3))   # [0. 0.994 1. 1. 1. 1.]
print(le_iterate(x, [0.4] * 8).round(3))   # [0. 0.245 0.695 0.93 0.985 1.]
```

The first line shows the danger: for small x, each step multiplies by about $1 + \alpha$, so eight steps with $\alpha = 1$ give a gain near $2^8 = 256$. Whatever noise sits in the shadows is amplified by the same factor.

Non-reference losses. Zero-DCE is trained with four losses that need no ground truth:

$$\mathcal{L}_{spa} = \frac{1}{K}\sum_{i=1}^{K}\sum_{j\in\Omega(i)}\left(|Y_i - Y_j| - |I_i - I_j|\right)^2,$$

$$\mathcal{L}_{exp} = \frac{1}{M}\sum_{k=1}^{M}|Y_k - E|, \qquad \mathcal{L}_{col} = \sum_{(p,q)\in\varepsilon}(J^p - J^q)^2,$$

$$\mathcal{L}_{tv_A} = \frac{1}{N}\sum_{n=1}^{N}\sum_{c\in\{R,G,B\}}\left(|\nabla_x A_n^c| + |\nabla_y A_n^c|\right)^2.$$

In $\mathcal{L}_{spa}$ (**spatial consistency**), K is the number of 4×4 regions, $\Omega(i)$ the four neighbors of region i, and Y and I the mean intensities of a region in the enhanced and input images: contrast between neighboring regions should be preserved. In $\mathcal{L}_{exp}$ (**exposure control**), Y_k is the mean intensity of the k-th of M non-overlapping 16×16 regions and $E = 0.6$ a target well-exposedness level. In $\mathcal{L}_{col}$ (**color constancy**), J^p is the mean of channel p over the image and $\varepsilon = \{(R,G),(R,B),(G,B)\}$: the gray-world assumption that the average scene color is gray. In $\mathcal{L}_{tv_A}$ (**illumination smoothness**), N is the number of iterations and ∇_x, ∇_y horizontal and vertical gradients: neighboring pixels should get similar curves. The total is

$\mathcal{L}_{spa} + \mathcal{L}_{exp} + W_{col}\mathcal{L}_{col} + W_{tv_A}\mathcal{L}_{tv_A}$ with $W_{col} = 0.5$ and $W_{tv_A} = 20$ in the paper; training uses multi-exposure images from SICE [56], which provide both under- and over-exposed inputs.

Follow-ups refined the idea:

- **Zero-DCE++** [34] (2022) uses depthwise separable convolutions, predicts only **3** curve maps and reuses them across all iterations, and estimates them on a downsampled input before resizing them back. This cuts the model to about 10k parameters and makes it fast enough to run at interactive rates even on a CPU.
- **SCI** [35] (2022) learns illumination with a cascade of weight-sharing stages and a **self-calibrated** module that makes the stages' outputs converge during training, so that at test time

a **single** basic block suffices. It is trained with unsupervised losses.
- **CLIP-LIT** [36] (2023) targets *backlit* images. It uses the vision-language model CLIP and learns a pair of text prompts, one for backlit and one for well-lit images, then refines the prompts and the enhancement network iteratively, so the CLIP similarity to the "well-lit" prompt acts as the training signal.

A zero-reference toy in PyTorch

To see these losses at work, the code below trains a tiny Zero-DCE-style network for about half a minute on CPU. It trains on random crops of four `skimage` images, each darkened by a random factor in linear light, and is tested on the coffee image, which it never sees during training. No ground truth is used. (The loss weights differ from the paper's because the terms are normalized differently: squared rather than absolute exposure error and mean rather than summed gradients.)

```
import numpy as np
import torch
import torch.nn as nn
import torch.nn.functional as F
from skimage import data, img_as_float
from skimage.transform import resize

torch.manual_seed(0); rng = np.random.default_rng(0)
darken = lambda x, k: (k * x ** 2.2) ** (1 / 2.2)          # less light, in linear units
train = [resize(img_as_float(f()), (256, 256), anti_aliasing=True)
         for f in (data.astronaut, data.chelsea, data.rocket, data.immunohistochemistry)]

def batch(n=4, size=64):                                    # random crops, random darkness
    xs = []
    for _ in range(n):
        img = train[rng.integers(len(train))]
        i, j = rng.integers(0, 256 - size, 2)
        xs.append(darken(img[i:i + size, j:j + size], rng.uniform(0.02, 0.3)))
    return torch.tensor(np.stack(xs), dtype=torch.float32).permute(0, 3, 1, 2)

class TinyDCE(nn.Module):
    def __init__(self, n_iter=8, w=16):
        super().__init__()
        self.n_iter = n_iter
        self.body = nn.Sequential(
            nn.Conv2d(3, w, 3, padding=1), nn.ReLU(), nn.Conv2d(w, w, 3, padding=1),
            ↪  nn.ReLU(),
            nn.Conv2d(w, w, 3, padding=1), nn.ReLU(), nn.Conv2d(w, 3 * n_iter, 3,
            ↪  padding=1), nn.Tanh())
    def forward(self, x):
        alphas = self.body(x).chunk(self.n_iter, dim=1)    # 8 maps of shape B x 3 x H x W
        y = x
        for a in alphas:
            y = y + a * y * (1 - y)                         # the LE curve
        return y, torch.cat(alphas, 1)
```

```
def L_exp(y, E=0.6, p=16):   # mean brightness of 16x16 patches should be near E
    return (F.avg_pool2d(y.mean(1, keepdim=True), p) - E).pow(2).mean()
def L_col(y):                # gray-world: channel means should agree
    r, g, b = y.mean((2, 3)).unbind(1)
    return ((r - g) ** 2 + (r - b) ** 2 + (g - b) ** 2).mean()
def L_tv(a):                 # curve parameters should vary smoothly in space
    return (a[..., 1:, :] - a[..., :-1, :]).pow(2).mean() + (a[..., :, 1:] - a[..., :,
     ↪  :-1]).pow(2).mean()
def L_spa(y, x, p=4):        # keep the contrast between neighbouring 4x4 regions
    Y, X = (F.avg_pool2d(t.mean(1, keepdim=True), p) for t in (y, x))
    return sum(((torch.diff(Y, dim=d) - torch.diff(X, dim=d)) ** 2).mean() for d in (2, 3))

net = TinyDCE()
opt = torch.optim.Adam(net.parameters(), lr=1e-3)
for step in range(600):                                    # ~20-40 s on a laptop CPU
    x = batch()
    y, a = net(x)
    loss = L_spa(y, x) + 10 * L_exp(y) + 5 * L_col(y) + 200 * L_tv(a)
    opt.zero_grad(); loss.backward(); opt.step()

test = darken(img_as_float(data.coffee()), 0.05)           # an image never used in training
with torch.no_grad():
    out, _ = net(torch.tensor(test, dtype=torch.float32).permute(2, 0, 1)[None])
print(f"mean brightness: {test.mean():.3f} -> {out.mean():.3f}")   # about 0.10 -> 0.50
```

Drag the slider to compare the dark input and the toy's output:

Dark input

Zero-reference toy

The dark coffee-cup image and the toy network's output, which is much brighter, with visible wood grain and a lighter, slightly greenish and less saturated color than the original photo.

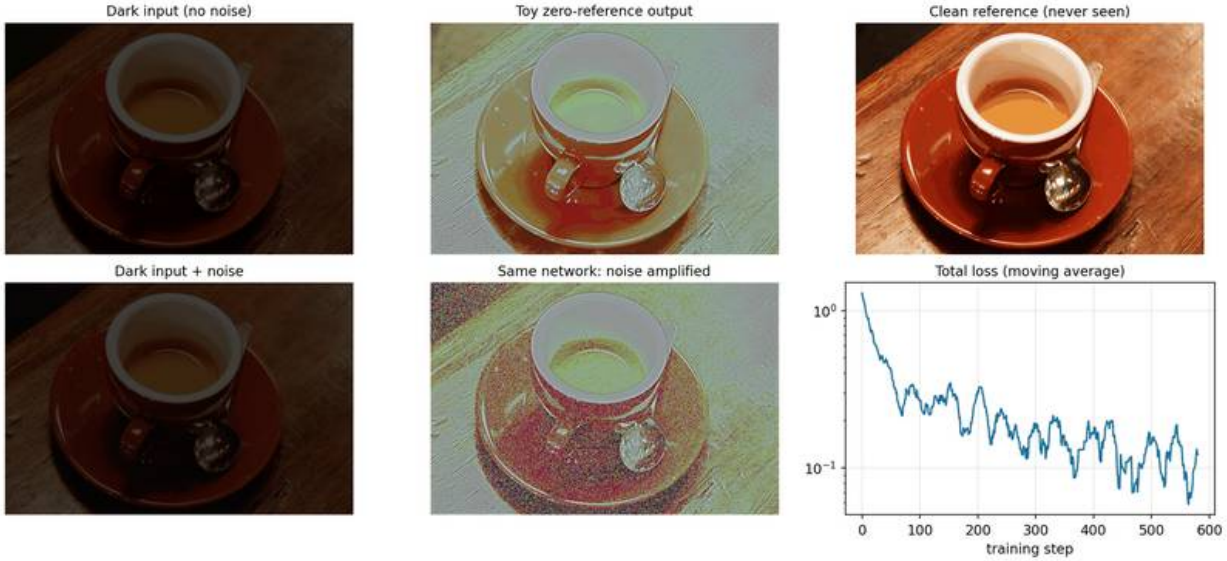


Figure 7 — What the zero-reference losses do and don't do. The exposure loss gives good brightness, but the gray-world color loss pulls a red-brown scene toward gray (top middle versus top right), and with real noise the per-pixel curves amplify it (bottom middle).

The toy shows the strengths and the limits of the zero-reference idea in one picture. Without a single reference image, the network learns a sensible brightening that transfers to an unseen photo. But each loss is a *prior*, and priors can be wrong: gray-world fails for scenes that really are red or brown, a fixed exposure target E flattens deliberately dark regions, and nothing in the losses asks for denoising. Real zero-reference methods are trained on many more images, and some add explicit noise handling, but these failure modes are visible in published results too.

Normalizing flows and diffusion models

LLFlow [37] (2022) observes that one dark image has *many* acceptable bright versions, and a pixel-wise loss averages them into a blurry compromise. It learns a conditional normalizing flow: an invertible network that maps normal-light images to a Gaussian distribution, conditioned on features of the dark input. Those features are guided by priors that echo the classical methods: a histogram-equalized version of the input, a color map (close to the Retinex reflectance) and a noise map.

From 2023, **diffusion models** entered LLIE. They generate the output by iteratively denoising random noise, conditioned on the dark image, and so can synthesize plausible detail that regression networks blur out:

- **Diff-Retinex** [38] (2023) first decomposes the image with a Retinex transformer, then uses conditional diffusion models to generate the restored reflectance and illumination.
- **GSAD** [39] (2023) adds a global structure-aware regularization that uses the image structure to guide a curvature-based constraint on the diffusion trajectory, plus an uncertainty-guided term for hard regions.
- **LightenDiffusion** [40] (2024) performs Retinex decomposition in a latent space and trains with *unpaired* data.
- **Reti-Diff** [41] (2025) runs a latent diffusion model to produce compact Retinex priors that guide a restoration transformer, which reduces the cost of running diffusion at full resolution.

The trade-off is cost and fidelity: many denoising steps make inference slow, and generated details may look right without being *true*. The survey by Adhikarla et al. [6] organizes this sub-field into six families: intrinsic decomposition, spectral and latent, accelerated, guided, multimodal, and autonomous.

Raw-domain low light

Everything above works on processed sRGB images, where the camera's pipeline (demosaicking, white balance, denoising, tone curve, compression) has already discarded information. **Learning to See in the Dark** [44] (2018) instead feeds **raw** sensor data to a U-Net that replaces the

whole pipeline. Its SID dataset has 5,094 short-exposure raw images paired with 424 distinct long-exposure references, from a Sony α7S II (Bayer sensor) and a Fujifilm X-T2 (X-Trans sensor), with exposure ratios of ×100, ×250 and ×300. Raw data keeps the linear relation between photons and values and has more bits, so extreme low light that is hopeless in sRGB becomes recoverable. The cost is that the model is tied to one sensor.

All-in-one restoration

Low-light enhancement is also one task inside general restoration models. **InstructIR** [43] (2024) takes a natural-language instruction ("make this photo brighter", "remove the noise") and restores the image accordingly, covering denoising, deblurring, dehazing and low-light enhancement in one network.

Low light for downstream tasks

If the goal is *detection* rather than pretty pictures, the usual baseline is "enhance, then detect" (see DLCV 03 ·Object Detection). But enhancement tuned for people is not necessarily what a detector needs. **MAET** [45] (2021) models the low-light degradation physically and trains the detector to also predict the degradation parameters, so that features become aware of it. **FeatEnHancer** [46] (2023) enhances hierarchical *features* inside the network, trained directly with the downstream loss, instead of enhancing pixels.

Figure 8 — A timeline of the methods in this tutorial (by publication year). Colors mark Retinex-based or unrolled designs, unpaired or zero-reference training, flow and diffusion models, and challenges.

Evaluation

Plain version. There are two kinds of scores. **Full-reference** metrics compare the output to a ground-truth photo; **no-reference** metrics judge the output alone. A third approach asks whether

the enhanced image helps a downstream task. Each answers a different question, and each can be gamed.

Full-reference metrics

With a reference y and output $\hat{y}$ (values in $[0,1]$), **PSNR** is

$$\mathrm{PSNR} = 10\log_{10}\frac{1}{\frac{1}{n}\|\hat{y}-y\|_2^2},$$

where n is the number of values. It is a pure pixel error. **SSIM** [47] compares local luminance, contrast and structure statistics, which correlates better with perception. **LPIPS** [48] measures the distance between deep features of the two images; lower is better, and it is sensitive to texture and blur that PSNR ignores.

No-reference metrics: why low light depends on them

Plain version. A no-reference (NR) metric looks at one image and guesses how good it is, with no "correct answer" to compare against. Low-light papers need these metrics because their most realistic test photos have no correct answer, and because even when one exists, it is only one of many acceptable brightness levels.

Precise version. Two facts push LLIE toward NR evaluation:

1. **Real test sets are unpaired.** LIME [17], MEF [58], NPE [18], DICM [59] and VV [60] are small sets of real, often non-uniformly lit photos with no ground truth. Paired sets such as LOL [21] exist, but they were captured under controlled conditions (a static scene, two exposures), and a model can do well on them without generalizing (the generalization gap reported in [2]). Results on the unpaired sets are therefore shown in almost every LLIE paper, and only NR metrics and human judgments can score them.
2. **The "correct" brightness is ambiguous.** Even with a reference, the normal-light image is one photographer's choice of exposure and color. An output that is a little brighter or warmer can look as good, yet lose several dB of PSNR (this is the reason for the GT-mean protocol below). An NR metric at least does not punish a different but valid exposure.

The price is that an NR metric is a *model of human preference*, and every model has blind spots. The general theory of NR image quality assessment (IQA), including how NIQE, BRISQUE, MUSIQ and CLIP-IQA are built, is covered in DLCV 04 ·Image Restoration. The subsections below summarize what each family measures and focus on how it behaves on *enhanced low-light* images.

Classical natural-scene-statistics metrics: NIQE, BRISQUE and PI

Plain version. Natural photos share statistical regularities: after you remove local brightness and contrast, what remains looks like a bell-shaped distribution of a particular shape. Blur, noise

and compression change that shape. Natural-scene-statistics (NSS) metrics measure how far an image's statistics are from those of clean photos.

MSCN coefficients. Both NIQE and BRISQUE start from **mean-subtracted contrast-normalized** (MSCN) coefficients of the grayscale image I [50]:

$$\hat{I}(i,j) = \frac{I(i,j) - \mu(i,j)}{\sigma(i,j) + C}, \qquad \mu(i,j) = \sum_{k,l} w_{k,l}\, I(i+k, j+l), \qquad \sigma(i,j) = \sqrt{\sum_{k,l} w_{k,l} (I(i+k,j+l) - \mu(i,j))^2},$$

where (i,j) is a pixel, w a small circularly symmetric Gaussian window normalized to sum to 1, μ and σ the local weighted mean and standard deviation, and C a small constant ($C = 1$ for 8-bit images) that avoids division by zero in flat regions. For pristine images the histogram of $\hat{I}$ is close to a unit Gaussian; distortions make it peakier (blur) or heavier-tailed (noise, blocking). The products of neighboring MSCN coefficients in four directions capture the remaining structure.

BRISQUE [50] fits a generalized Gaussian distribution to $\hat{I}$ and asymmetric generalized Gaussians to the neighbor products at two scales, and feeds the fitted shape and variance parameters to a support-vector regressor trained on human opinion scores for synthetic distortions. Lower is better.

NIQE [49] needs no opinion scores. It selects sharp patches, computes similar NSS features, fits a multivariate Gaussian (MVG) with mean ν_1 and covariance Σ_1 to a corpus of pristine images, fits another MVG (ν_2, Σ_2) to the test image, and reports

$$D(\nu_1, \nu_2, \Sigma_1, \Sigma_2) = \sqrt{(\nu_1 - \nu_2)^\top \left(\frac{\Sigma_1 + \Sigma_2}{2}\right)^{-1} (\nu_1 - \nu_2)},$$

a Mahalanobis-like distance between the two Gaussians. Lower is better: zero means "statistically like the pristine corpus".

PI. The Perceptual Index was introduced for the 2018 PIRM super-resolution challenge [63]:

$$\mathrm{PI} = \tfrac{1}{2}((10 - \mathrm{Ma}) + \mathrm{NIQE}),$$

where Ma is the learned no-reference score of Ma et al. [64] (higher is better, roughly 0 to 10, trained on human ratings of super-resolved images). Lower PI is better. Zero-DCE [33], for example, reports PI on the unpaired sets alongside a user study.

What they reward, and how they mislead on exposure. All three were designed for *structural* distortions (blur, noise, compression, ringing), not for exposure or color:

- MSCN normalization divides out local mean and contrast by construction, so a global brightness change barely moves the features. An under-enhanced image and a well-exposed one can get similar scores.
- Grayscale features ignore color casts and saturation, the very errors of Figure 2.
- Noise and over-sharpening add high-frequency energy, which can make a smooth, noise-amplified output look "more natural" to these statistics than a clean but slightly soft one. HE and CLAHE outputs, with strong local contrast, are a typical case.
- The pristine corpus and training distortions contain few dark, night or backlit scenes, so "natural" means "daylight-like".

Low-light-specific measures: lightness order, colorfulness and noise

Plain version. Some measures target what goes wrong in enhancement specifically: Did the method swap which regions are brighter than others? Are the colors vivid or washed out? How much noise did brightening reveal?

Lightness Order Error (LOE). The NPE paper [18] argues that a natural enhancement should keep the *relative order* of lightness in the scene: if a lamp was brighter than a wall before, it should still be after, because that order carries the direction of the light and the sense of depth. LOE counts how often the order of two pixels flips. With $L(x) = \max_{c\in\{R,G,B\}} I_c(x)$ the lightness of the input and $L_e(x)$ that of the enhanced image,

$$\mathrm{LOE} = \frac{1}{m}\sum_{x=1}^{m} \mathrm{RD}(x), \qquad \mathrm{RD}(x) = \sum_{y=1}^{m} U(L(x), L(y)) \oplus U(L_e(x), L_e(y)),$$

where m is the number of pixels, $U(p,q) = 1$ if $p \geq q$ and 0 otherwise, and $\oplus$ is exclusive-or, so $\mathrm{RD}(x)$ is the number of pixels whose order relative to x changed. Lower is better. Three practical points:

- The cost is $O(m^2)$, so implementations evaluate LOE on a subsampled image. The raw value grows with m, so numbers are comparable only at the same subsampling. Many papers do not state theirs.
- The reference is the **input**, not a clean image. Any monotonic tone curve applied in the max channel gets LOE = 0, including doing nothing at all. And because the noise in the dark input already scrambles the order of nearby pixels, a method that *removes* noise is penalized.
- LOE says nothing about whether the result is bright enough. It is a guard against unnatural local tone mapping, not a quality score.

Colorfulness. Hasler and Süsstrunk [65] fit a simple colorfulness measure to perceptual ratings, using opponent-color axes on 8-bit RGB:

$$rg = R - G, \qquad yb = \tfrac{1}{2}(R + G) - B, \qquad M = \sqrt{\sigma_{rg}^2 + \sigma_{yb}^2} + 0.3\sqrt{\mu_{rg}^2 + \mu_{yb}^2},$$

where μ and σ are the mean and standard deviation over the image, and the weight 0.3 comes from the authors' fit. Higher means more colorful. It is useful as a descriptive statistic for desaturation (luma-only enhancement, Figure 2) or oversaturation, but it has no notion of the *correct* color, and noise in the chroma channels inflates σ_{rg} and σ_{yb}.

Noise level of the output. Since brightening reveals noise (Figure 1), it helps to report the noise of the output. Immerkær's fast estimator [66] filters the image with a mask that cancels smooth structure and keeps noise,

$$N = \begin{bmatrix} 1 & -2 & 1 \\ -2 & 4 & -2 \\ 1 & -2 & 1 \end{bmatrix}, \qquad \hat{\sigma} = \sqrt{\frac{\pi}{2}}\,\frac{1}{6\,(W-2)(H-2)}\sum_{i,j} |(I * N)(i,j)|,$$

where $W \times H$ is the image size and the sum runs over interior pixels. It assumes white Gaussian noise, so it overestimates on textured images and underestimates spatially correlated noise, which

is what real camera pipelines and many networks produce. Use it to compare outputs of the *same* input, not across scenes.

The snippet below recreates the dark coffee image of Figure 1 and compares five "enhancements" plus the clean photo.

```
import numpy as np
from scipy import ndimage as ndi
from skimage import color, data, exposure, img_as_float

rng = np.random.default_rng(0)
clean = img_as_float(data.coffee())

def synth_low_light(x, k=0.05, peak=2000.0, read_std=8e-4, gamma=2.2):   # as in Figure 1
    lin = k * x ** gamma
    lin = rng.poisson(lin * peak) / peak
    lin = lin + rng.normal(0, read_std, lin.shape)
    return np.round(np.clip(lin, 0, 1) ** (1 / gamma) * 255) / 255

def loe(ref, out, step=8):
    """Lightness Order Error on a strided subsample (m = 50 x 75 = 3750 pixels here)."""
    L, Le = (im.max(axis=2)[::step, ::step].ravel() for im in (ref, out))
    U, Ue = L[:, None] >= L[None, :], Le[:, None] >= Le[None, :]
    return (U ^ Ue).sum(axis=1).mean()                  # mean over x of RD(x)

def colorfulness(im):
    """Hasler-Suesstrunk colorfulness, RGB scaled to 0..255."""
    R, G, B = (255 * im[..., c] for c in range(3))
    rg, yb = R - G, 0.5 * (R + G) - B
    return np.hypot(rg.std(), yb.std()) + 0.3 * np.hypot(rg.mean(), yb.mean())

def noise_sigma(im):
    """Immerkaer's noise estimate on BT.601 luma, in 8-bit units."""
    Y = 255 * (0.299 * im[..., 0] + 0.587 * im[..., 1] + 0.114 * im[..., 2])
    N = np.array([[1, -2, 1], [-2, 4, -2], [1, -2, 1]])
    H, W = Y.shape
    return np.sqrt(np.pi / 2) / (6 * (W - 2) * (H - 2)) * np.abs(ndi.convolve(Y, N)[1:-1,
     ↪  1:-1]).sum()

dark = synth_low_light(clean)
lab = color.rgb2lab(dark); L = lab[..., 0] / 100
he = lab.copy(); he[..., 0] = 100 * exposure.equalize_hist(L)
cl = lab.copy(); cl[..., 0] = 100 * exposure.equalize_adapthist(L, clip_limit=0.01)
outputs = {
    "dark input": dark,
    "gain x20 (linear)": np.clip(20 * dark ** 2.2, 0, 1) ** (1 / 2.2),
    "gamma 0.4": dark ** 0.4,
    "HE on L*": np.clip(color.lab2rgb(he), 0, 1),
    "CLAHE on L*": np.clip(color.lab2rgb(cl), 0, 1),
    "clean reference": clean,
}
for name, im in outputs.items():
    print(f"{name:18s} mean {im.mean():.2f}  LOE {loe(dark, im):6.1f}  "
          f"colorfulness {colorfulness(im):5.1f}  noise {noise_sigma(im):5.2f}")
```

```
# dark input          mean 0.10  LOE    0.0  colorfulness  23.1  noise  3.32
# gain x20 (linear)   mean 0.39  LOE    0.3  colorfulness  90.0  noise 12.93
# gamma 0.4           mean 0.34  LOE    0.0  colorfulness  61.6  noise 11.19
# HE on L*            mean 0.48  LOE  334.7  colorfulness  28.3  noise 16.17
# CLAHE on L*         mean 0.40  LOE  657.6  colorfulness  27.1  noise 23.06
# clean reference     mean 0.39  LOE  322.0  colorfulness  76.9  noise  4.69
```

Figure 9 — Brighter is not automatically better. HE is the brightest output but the most order-breaking after CLAHE and nearly colorless; the noisy linear gain is more 'colorful' than the clean photo; and the clean photo itself has a high LOE, because LOE is measured against the noisy input.

Read the numbers together, not one at a time:

- **Brightness alone ranks the wrong way.** HE has the highest mean (0.48) but also a large LOE, the second-highest noise and almost no color left. CLAHE's local remapping breaks lightness order the most.
- **Colorfulness is fooled by noise.** The linear gain scores 90, *higher* than the clean photo's 77, partly because amplified chroma noise raises σ_{rg} and σ_{yb}.
- **LOE penalizes restoration.** The clean photo, the ideal answer, gets LOE 322 against the noisy input, while doing nothing at all gets 0. Low LOE is necessary for a natural tone curve, not sufficient for quality.
- **Only the noise estimate separates the clean photo** (4.7) from the bright but noisy outputs (11 to 23). No single NR number captures "well exposed, natural color, and clean".

Learned no-reference assessors

Plain version. Instead of hand-made statistics, a neural network is trained to predict human ratings, or a large vision-language model is asked how good a photo looks. These models capture more of what people care about, including content and aesthetics, but they inherit the biases of their training data.

The main families, all of which can be applied to LLIE outputs (and which share common implementations, e.g. the IQA-PyTorch toolbox [75]):

- **NIMA** [67] (2018) predicts the *distribution* of human ratings over the score bins with a CNN, rather than a single number. The reported score is the mean of that distribution, $\hat{s} = \sum_k k\, p_k$, where p_k is the predicted probability of score k. Versions trained on aesthetic ratings measure "is this a nice photo", which is not the same as "is this a faithful enhancement".
- **MUSIQ** [68] (2021) is a multi-scale transformer that takes the image at its native resolution and aspect ratio, plus downscaled versions, using hash-based 2D position and scale embeddings. Avoiding resizing matters for LLIE, where noise and fine texture are exactly what resizing would hide. Higher is better.
- **CLIP-IQA** [69] (2023) uses the vision-language model CLIP with an antonym prompt pair, "Good photo." and "Bad photo.". With $\mathbf{x}$ the image embedding and $\mathbf{t}_1, \mathbf{t}_2$ the two text embeddings, $s_i = \mathbf{x}^\top \mathbf{t}_i / (\|\mathbf{x}\|\,\|\mathbf{t}_i\|)$ and the score is $e^{s_1}/(e^{s_1} + e^{s_2})$, in $[0, 1]$, higher is better. The basic version is zero-shot: it needs no opinion scores. Other prompt pairs ("Bright photo." / "Dark photo.", "Noisy photo." / "Clean photo.") probe single attributes, which is convenient for exposure, but the exact wording changes the score. The same CLIP-similarity idea is used as a *training* signal by CLIP-LIT [36], so a CLIP-based metric should not be the only judge of a CLIP-trained enhancer.
- **MANIQA** [70] (2022) combines a vision transformer with transposed (channel) attention and Swin-transformer blocks and a dual-branch patch-weighted score. It won the NTIRE 2022 perceptual IQA challenge, whose images include outputs of restoration and GAN-based algorithms, so it is closer to the "algorithm output" setting than metrics trained on camera distortions.
- **Q-Align** [71] (2024) fine-tunes a large multimodal model to answer with a *text rating level* (bad, poor, fair, good, excellent), mimicking how human raters work. With X_{l_i} the model's logit for level l_i, the score is $S = \sum_{i=1}^{5} i \cdot e^{X_{l_i}} / \sum_{j=1}^{5} e^{X_{l_j}}$, the expected level under a softmax restricted to the five words.
- **LLIE-specific assessors.** Kannan et al. [72] (2023) collected human opinion scores for images restored by different LLIE methods, benchmarked existing NR-IQA models on them, and proposed an *opinion-unaware* model that learns distortion-aware features of restored low-light images by self-supervised contrastive learning. Datasets of this kind are the right test bed for deciding which metric to trust on enhancement outputs.

Caveats for low light. Learned assessors are trained mostly on daylight photos. They tend to rate brighter, sharper and more saturated images higher, which favors over-enhancement, and a deliberately dark night scene can be penalized even when it is the faithful result. Their scores depend on the checkpoint (which training set), the input size and the implementation, so report

those. And when a method is tuned with a learned metric or the same backbone in its loss, that metric is no longer an independent judge.

User studies and paired comparisons

Plain version. In the end, people decide what looks good. The two standard ways to ask them are to rate images on a scale or to choose the better of two.

Mean opinion score (MOS). Each of N raters gives an image a score s_n on a fixed scale (often 1 to 5), and MOS $= \frac{1}{N}\sum_{n=1}^{N} s_n$. ITU-R BT.500 [73] standardizes such tests: viewing conditions, rating scales, single- and double-stimulus and comparison methods, and screening of unreliable raters. Zero-DCE [33], for example, had 15 subjects rate results on a 1–5 scale over the 202 images of NPE, LIME, MEF, DICM and VV, with instructions to look for over- or under-exposure, color deviation, and unnatural texture or noise.

Paired comparison. Absolute scales drift between raters; choosing between two images is easier and more consistent. The **Bradley–Terry** model [74] turns many such choices into a score per method: each method i has a strength $\pi_i > 0$ and

$$P(i \text{ preferred over } j) = \frac{\pi_i}{\pi_i + \pi_j},$$

with the π_i fitted by maximum likelihood to the observed wins. EnlightenGAN [32] used this protocol: 9 subjects compared pairs of outputs from five methods on 23 images, judging visible noise, exposure artifacts and unrealistic color or texture, and a Bradley–Terry fit gave the ranking.

Good practice: hide method names, randomize order and left/right position, show the input for context, use enough images *and* raters, report confidence intervals or significance, and state the display and viewing conditions. A study with a handful of raters and hand-picked images is a demonstration, not evidence.

Choosing and reporting no-reference metrics

Metric	Type	Needs training data?	Better	Sensitive to	Caveat for low light
NIQE [49]	NSS	Pristine images only (no opinion scores)	Lower	Blur, noise, compression in grayscale structure	Nearly blind to global exposure and color; can prefer noisy, high-contrast outputs
BRISQUE [50]	NSS + regression	Yes, opinion scores on synthetic distortions	Lower	Same features as NIQE, mapped to human scores	Trained on daylight distortions; ignores color cast

Metric	Type	Needs training data?	Better	Sensitive to	Caveat for low light
PI [63], [64]	NSS + learned	Ma score trained on super-resolution ratings	Lower	Texture and sharpness	Designed for super-resolution; rewards detail, including amplified noise
LOE [18]	Task-specific	No	Lower	Flips in lightness order relative to the input	Depends on subsampling; 0 for identity; penalizes denoising
Colorfulness [65]	Task-specific (descriptive)	No (fitted constants)	Context-dependent	Spread and offset of opponent colors	No notion of correct color; inflated by chroma noise
Noise estimate [66]	Task-specific (descriptive)	No	Lower	High-frequency residual	Assumes white noise; confused by texture and correlated noise
NIMA [67]	Learned	Yes, rating distributions	Higher	Aesthetics or technical quality, depending on checkpoint	Aesthetic models reward attractive, not faithful, results
MUSIQ [68]	Learned	Yes, opinion scores	Higher	Multi-scale quality at native resolution	Trained mostly on daylight photos; checkpoint-dependent
CLIP-IQA [69]	Learned (vision-language)	No for zero-shot	Higher	Semantic "good photo" impression; prompt-dependent	Prompt wording matters; not independent of CLIP-trained enhancers
MANIQA [70]	Learned	Yes, opinion scores	Higher	Restoration and GAN artifacts	Not trained on exposure errors specifically
Q-Align [71]	Learned (large multimodal model)	Yes, opinion scores as text levels	Higher	Overall human-like quality judgment	Heavy; biases of the base model; version-dependent
MOS / paired comparison [73], [74]	Human	Human raters	Higher MOS / higher strength	What people actually prefer	Costly; small studies are noisy; protocol must be reported

Practical recommendations:

1. **On paired sets**, lead with full-reference metrics (PSNR, SSIM, LPIPS) under the standard protocol, and add NR scores only as a complement.
2. **On unpaired sets**, report *several* NR metrics from different families (one NSS, one or two learned, LOE), state the implementation, checkpoint and image size, and show many visual

examples, including failures.

3. **Expect disagreement.** NSS and learned metrics often rank methods differently, and both can disagree with people; the survey of Li et al. [2] notes the frequent mismatch between metric scores and visual quality, and Liu and Fan [5] find a further gap between image quality and downstream task performance. When metrics disagree, say so instead of picking the favorable one.
4. **Do not optimize the judge.** If a method's loss uses a metric or the same backbone (for example CLIP), evaluate it with something else.
5. **Back claims with people.** For a new method, a properly designed user study or paired comparison is the strongest NR evidence.

The GT-mean pitfall

Plain version. On paired benchmarks, a small difference in *overall brightness* changes PSNR a lot. Some papers rescale each output so its average brightness matches the ground truth before scoring. That is information you never have in practice.

Precise version. "GT-mean" evaluation scales the output by $\lambda = \mathbb{E}[y]/\mathbb{E}[f(x)]$ and computes the metric on $\lambda f(x)$, where $\mathbb{E}[\cdot]$ is the image mean. Because LLIE is ambiguous about the "right" brightness, this can raise PSNR substantially. The Retinexformer authors provide this mode in their code for comparability with methods such as LLFlow and KinD and recent diffusion models, while explicitly saying they do *not* recommend it, because it uses the ground truth to correct the output [26]. Liao et al. [51] (2025) study the brightness mismatch between outputs and references and propose a GT-mean *loss* for training, reporting both standard and GT-mean metrics. When reading tables: check which setting is used, never mix the two, and prefer methods that are good under the standard protocol.

Task-driven evaluation

Since LLIE is often a preprocessing step, another test is to run a detector or segmenter on the enhanced images (on ExDark [57], for example) and report its accuracy. Liu and Fan's 2025 review [5] finds a disconnect: supervised methods often produce images with high perceptual quality but only modest gains for vision tasks, while zero-shot methods, despite lower image-quality scores, give more consistent gains across tasks. The benchmark of Liu et al. [4] also evaluates face detection in low light. A complete evaluation reports fidelity, no-reference quality, task performance, and runtime.

Benchmark datasets

Plain version. Datasets differ in whether they have pairs, whether the darkness is real or synthetic, whether they are raw or sRGB, and whether they carry labels for detection.

Dataset	Type	Content and what it is for
LOL (v1) [21]	Paired, real, sRGB	500 low/normal pairs captured by changing exposure (485 train / 15 test), 400×600. The most common LLIE benchmark; small test set.
LOL-v2 [52]	Paired, real + synthetic subsets	A larger *Real* subset of captured pairs and a *Synthetic* subset of synthesized pairs. Report the two subsets separately.
LSRW [53]	Paired, real	5,650 pairs from a Nikon D7500 (3,170) and a Huawei P40 Pro (2,480); 5,600 train, 50 test. Tests generalization across devices.
SDSD [54]	Paired video	Dynamic indoor and outdoor scenes captured twice with a mechatronic system for alignment; often split into frames for image tests.
SID [44]	Paired, raw	5,094 short-exposure raw images with 424 long-exposure references (Sony and Fuji). Extreme low light, raw-to-sRGB.
MIT-Adobe FiveK [55]	Paired, retouching	5,000 raw photos, each retouched by five experts. Used for tone and color enhancement (photo retouching).
SICE [56]	Multi-exposure sequences	Under- to over-exposed captures of each scene with a reference; used to train exposure correction (e.g. Zero-DCE).
VE-LOL [4]	Paired + detection	VE-LOL-L for enhancement and VE-LOL-H with face annotations for high-level evaluation.
ExDark [57]	Unpaired, labeled	7,363 low-light images, 12 object classes, 10 lighting conditions. Task-driven evaluation (detection).
LIME [17], MEF [58], NPE [18], DICM [59], VV [60]	Unpaired test sets	Small collections of real low-light or non-uniformly lit photos for visual comparison and no-reference metrics.

Challenges: NTIRE 2024 and 2025

The NTIRE workshop at CVPR ran LLIE challenges in consecutive years. The **NTIRE 2024** challenge [61] covered ultra-high-resolution (4K and beyond), non-uniform illumination, backlighting, extreme darkness and night scenes; it had 428 registered participants and 22 teams with valid submissions. The **NTIRE 2025** challenge [62] had 762 registered participants and 28 teams with valid entries. The reports summarize the winning designs and are a good snapshot of what works under a fixed, fair protocol.

Modern view

The field has moved from hand-made priors to learned models in under a decade, but the core questions in the post [1] are the same: how to model the light, how to handle noise, and how to know whether an output is good. The surveys below are the best entry points.

Li et al., TPAMI 2022 [2]. The most cited deep-LLIE survey. It organizes methods by learning strategy (supervised, reinforcement, unsupervised, zero-shot and semi-supervised learning), network structure, use of the Retinex model, data format (sRGB or raw), loss functions, and training and test data. It also introduces LLIV-Phone, an image and video dataset captured with many different phone cameras under diverse illumination, and an online platform for comparing methods. Its main lessons are a generalization gap (many methods struggle on the phone data) and a frequent mismatch between metric scores and visual quality.

Zheng et al., "Low-Light Image and Video Enhancement: A Comprehensive Survey and Beyond" [3]. It reviews traditional and deep methods with a similar taxonomy and goes further on data: it proposes SICE_Grad and SICE_Mix, which mix under- and over-exposure inside one image, and Night Wenzhou, an aerial and street night video set, to test methods on cases that existing benchmarks miss. It was the source of the post's own overview figures.

Liu et al., IJCV 2021 [4]. "Benchmarking low-light image enhancement and beyond" is a benchmark more than a survey: the VE-LOL dataset with paired data and face annotations, a comparison using full-reference, no-reference and semantic metrics, and a joint enhancement-plus-detection model. It made task-driven evaluation mainstream.

Liu and Fan, Neurocomputing 2025 [5]. This review asks which enhancement actually helps downstream vision and finds the disconnect described above between image-quality metrics and task performance, with zero-shot methods giving the most consistent task gains.

Adhikarla et al., 2025 [6]. A survey of diffusion models for LLIE with a six-family taxonomy and a performance analysis. Its message is that diffusion brings realism and flexibility but at a high inference cost, so acceleration, guidance and latent-space designs are the active topics.

NTIRE reports [61], [62]. These are not surveys but show the current practice of top teams under a common protocol, with high-resolution and difficult illumination.

State of the art (2023–2026). Three directions dominate. First, **Retinex-guided backbones**: transformers (Retinexformer [26]) and state-space models (RetinexMamba [42]) that keep a light-versus-reflectance structure. Second, **generative models**: diffusion (Diff-Retinex [38], GSAD [39], LightenDiffusion [40], Reti-Diff [41]) and flows (LLFlow [37]). Third, **better representations and protocols**: learned color spaces (HVI-CIDNet [27]), brightness-mismatch-aware training and evaluation (GT-Mean loss [51]), language-guided and all-in-one restoration (CLIP-LIT [36], InstructIR [43]), and task-aware enhancement (MAET [45], FeatEnHancer [46]).

Open problems.

- **Real noise.** Most sRGB methods train on data whose noise does not match real sensors after a camera pipeline. Better noise models [7], raw-domain training [44], or joint enhancement and denoising remain central.

- **Video consistency.** Frame-by-frame enhancement flickers. Paired video data (SDSD [54]) is scarce, and temporal consistency is still hard, especially with generative models.
- **Evaluation.** The ill-posed brightness target, GT-mean protocols, unreliable no-reference metrics for exposure, and the gap between perceptual and task metrics make comparisons fragile.
- **Efficiency on edge devices.** Phones and cars need real-time, low-power models. Curve methods (Zero-DCE++ [34], SCI [35]) and lightweight prior-based networks [28] show that tiny models can work, while diffusion models remain far too slow for this use.
- **Generalization.** Models trained on one dataset or camera often fail on another (the generalization gap reported in [2]); mixed over- and under-exposure, backlight and colored artificial light are still difficult.

Key takeaways

- LLIE is **tone enhancement plus restoration**: a perfect brightness correction still leaves noise, color cast and quantization (Figure 1).
- Brightness can be enhanced in a separate channel (V, I, Y', L^*), but the choice couples to saturation. BT.601 luma is $0.299R' + 0.587G' + 0.114B'$; $16 + 0.257R' + 0.504G' + 0.098B'$ is the same luma in 8-bit limited range.
- Retinex models an image as reflectance times **illumination**, $S = R \circ L$; SSR/MSR/LIME estimate L with smoothness priors, and the inverted-image dehazing approach rests on essentially the same assumption.
- Deep methods are best organized by **training signal**: supervised, unrolled, unpaired (EnlightenGAN), zero-reference (Zero-DCE, SCI, CLIP-LIT) and generative (LLFlow, diffusion). Retinex decomposition appears across them.
- Zero-DCE's LE curve $x + \alpha x(1-x)$ preserves range and order; its non-reference losses are priors (exposure, gray world, smoothness, contrast) that can fail on real noise and colorful scenes.
- Report full-reference, no-reference and task-driven results, **never mix GT-mean and standard numbers**, and remember that higher PSNR does not guarantee better detection.
- No-reference metrics each see one aspect: NIQE/BRISQUE/PI see structure but are nearly blind to exposure and color, LOE sees lightness order but not brightness or noise, colorfulness is inflated by chroma noise, and learned assessors inherit daylight biases. **Brighter is not automatically better** (Figure 9): report several NR metrics from different families, plus a well-designed user study.

Exercises

1. **Monotonic curves.** Show that $\mathrm{LE}(x;\alpha) = x + \alpha x(1-x)$ is monotonically non-decreasing on $[0, 1]$ if and only if $|\alpha| \leq 1$. What goes wrong for $\alpha = 1.5$?

   Hint

   The derivative is $1 + \alpha(1-2x)$, which is linear in x, so its minimum on $[0,1]$ is at an endpoint: $1+\alpha$ at $x = 0$ or $1-\alpha$ at $x = 1$. Both are ≥ 0 exactly when $|\alpha| \leq 1$. For $\alpha = 1.5$ the slope at $x = 1$ is -0.5,

so near-white pixels would get darker as the input gets brighter, reversing the brightness order and creating artifacts in highlights.

2. **Gain of the iterated curve.** For a small input x, show that eight iterations with a constant α multiply x by approximately $(1+\alpha)^8$. Use this to explain why zero-reference curve methods amplify shadow noise, and suggest one change to the losses or architecture that would reduce the problem.

   Hint

   For small x, $x(1-x) \approx x$, so each step gives $x \mapsto (1+\alpha)x$. With $\alpha = 1$ that is $2^8 = 256$. Noise with standard deviation σ becomes 256σ. Possible fixes: add a denoising branch or a noise-aware loss, limit α where the estimated SNR is low (in the spirit of SNR-aware), or penalize high-frequency energy in the output that is not present in a smoothed input.

3. **Two luma formulas.** A pixel has $(R', G', B') = (200, 100, 50)$ in 8 bits. Compute its full-range BT.601 luma and its limited-range luma. Show that they encode the same quantity.

   Hint

   Full range: $0.299 \cdot 200 + 0.587 \cdot 100 + 0.114 \cdot 50 = 124.2$. Limited range: $16 + 0.257 \cdot 200 + 0.504 \cdot 100 + 0.098 \cdot 50 \approx 122.7$. Converting back: $(122.7 - 16) \cdot 255/219 \approx 124.2$. The only differences are the scale $219/255$ and the offset 16.

4. **Retinex losses without ground truth.** In Retinex-Net, why is $\mathcal{L}_{ir} = \|R_{\text{low}} - R_{\text{normal}}\|_1$ needed? Describe a trivial decomposition that zeroes the self-reconstruction terms of $\mathcal{L}_{\text{recon}}$ and $\mathcal{L}_{is}$ but is useless, and explain how $\mathcal{L}_{ir}$ rules it out.

   Hint

   Set $I_i \equiv 1$ (perfectly smooth) and $R_i = S_i$. Then each image is reconstructed exactly and the smoothness loss is zero, but nothing has been decomposed. Since $S_{\text{low}} \neq S_{\text{normal}}$, this gives $R_{\text{low}} \neq R_{\text{normal}}$, so $\mathcal{L}_{ir}$ is large. Requiring equal reflectances forces the brightness difference between the pair into the illumination.

5. **GT-mean by hand.** Your model outputs an image with mean 0.40; the ground truth has mean 0.50, and otherwise the output is perfect up to that global scale. What is the GT-mean scale factor, and what is the PSNR before and after rescaling if the image is a constant? Why is the rescaled number not a fair comparison with a method evaluated normally?

   Hint

   $\lambda = 0.50/0.40 = 1.25$. Before: MSE $= 0.1^2 = 0.01$, PSNR $= 20$ dB. After rescaling the output equals the reference, so the error is 0 and PSNR is unbounded. A real method never has the ground-truth mean at test time, so GT-mean numbers reward the right *structure* while ignoring the hardest part of the task, choosing the brightness.

6. **Change the color prior.** In the PyTorch toy, replace the gray-world loss with a loss that keeps the *chromaticity* of the input, for example $\|y/\sum_c y_c - x/\sum_c x_c\|_1$. Train again and compare the coffee result with Figure 7. When is each prior preferable?

   Hint

   Add a small epsilon to the sums. Preserving input chromaticity keeps the red-brown cup red, but it also keeps any color cast of the light source (for example a sodium lamp). Gray world removes casts but desaturates scenes that are genuinely colorful. Real methods often combine a weak color prior with learned color correction from data.

7. **LOE by hand.** A three-pixel input has lightness $L = (0.1, 0.2, 0.3)$, and an enhancer outputs $L_e = (0.5, 0.4, 0.9)$. Compute $\mathrm{RD}(x)$ for each pixel and the LOE. Then explain why, in Figure 9, the clean reference gets a *higher* LOE (322) than the gamma curve (0), and what this implies for comparing a denoising enhancer with a plain tone curve using LOE alone.

   Hint

   Only the pair $(1, 2)$ changes order: $0.1 < 0.2$ in the input but $0.5 > 0.4$ in the output. That flips both $U(L_1, L_2)$ and $U(L_2, L_1)$, so $\mathrm{RD} = (1, 1, 0)$ (self-pairs never flip) and $\mathrm{LOE} = 2/3$. A gamma curve applied per channel is monotonic, so it keeps the order of $\max(R, G, B)$ exactly and gets 0. The clean image does not share the *noise* of the dark input, and that noise has scrambled the order of many nearby, nearly equal pixels, so the clean image disagrees with the input on those pairs. LOE therefore rewards keeping the input's noise and penalizes real restoration; it should be read together with a noise or quality measure, never alone.

References

1. Shyandram, "簡介 Deep Learning Low-Light Image Enhancement (LLIE)" (Introduction to deep-learning low-light image enhancement, in Chinese), Medium post, 2024; updated 2026. link
2. C. Li, C. Guo, L. Han, J. Jiang, M.-M. Cheng, J. Gu and C. C. Loy, "Low-light image and video enhancement using deep learning: A survey," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 12, pp. 9396–9416, 2022. arXiv
3. S. Zheng, Y. Ma, J. Pan, C. Lu and G. Gupta, "Low-light image and video enhancement: A comprehensive survey and beyond," arXiv:2212.10772, 2022. arXiv
4. J. Liu, D. Xu, W. Yang, M. Fan and H. Huang, "Benchmarking low-light image enhancement and beyond," *International Journal of Computer Vision*, vol. 129, no. 4, pp. 1153–1184, 2021. doi project
5. F. Liu and L. Fan, "A review of advancements in low-light image enhancement using deep learning," *Neurocomputing*, vol. 652, art. 131052, 2025. arXiv
6. E. Adhikarla, Y. Liu and B. D. Davison, "Diffusion models for low-light image enhancement: A multi-perspective taxonomy and performance analysis," arXiv:2510.05976, 2025. arXiv

7. A. Foi, M. Trimeche, V. Katkovnik and K. Egiazarian, “Practical Poissonian-Gaussian noise modeling and fitting for single-image raw-data,” *IEEE Transactions on Image Processing*, vol. 17, no. 10, pp. 1737–1754, 2008. doi
8. ITU-R, “Recommendation BT.601-7: Studio encoding parameters of digital television for standard 4:3 and wide-screen 16:9 aspect ratios,” 2011. ITU
9. S. M. Pizer, E. P. Amburn, J. D. Austin, et al., “Adaptive histogram equalization and its variations,” *Computer Vision, Graphics, and Image Processing*, vol. 39, no. 3, pp. 355–368, 1987. doi
10. K. Zuiderveld, “Contrast limited adaptive histogram equalization,” in *Graphics Gems IV*, Academic Press, 1994, pp. 474–485. doi
11. E. H. Land, “The retinex theory of color vision,” *Scientific American*, vol. 237, no. 6, pp. 108–128, 1977. doi
12. D. J. Jobson, Z. Rahman and G. A. Woodell, “Properties and performance of a center/surround retinex,” *IEEE Transactions on Image Processing*, vol. 6, pp. 451–462, 1997. doi
13. D. J. Jobson, Z. Rahman and G. A. Woodell, “A multiscale retinex for bridging the gap between color images and the human observation of scenes,” *IEEE Transactions on Image Processing*, vol. 6, no. 7, pp. 965–976, 1997. doi
14. K. He, J. Sun and X. Tang, “Single image haze removal using dark channel prior,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 33, no. 12, pp. 2341–2353, 2011. doi
15. X. Dong, Y. Pang and J. Wen, “Fast efficient algorithm for enhancement of low lighting video,” *ACM SIGGRAPH 2010 Posters*, 2010. SIGGRAPH history doi
16. X. Dong, G. Wang, Y. Pang, W. Li, J. Wen, W. Meng and Y. Lu, “Fast efficient algorithm for enhancement of low lighting video,” in *Proc. IEEE International Conference on Multimedia and Expo (ICME)*, 2011. doi
17. X. Guo, Y. Li and H. Ling, “LIME: Low-light image enhancement via illumination map estimation,” *IEEE Transactions on Image Processing*, vol. 26, no. 2, pp. 982–993, 2017. doi
18. S. Wang, J. Zheng, H.-M. Hu and B. Li, “Naturalness preserved enhancement algorithm for non-uniform illumination images,” *IEEE Transactions on Image Processing*, vol. 22, no. 9, pp. 3538–3548, 2013. doi
19. K. G. Lore, A. Akintayo and S. Sarkar, “LLNet: A deep autoencoder approach to natural low-light image enhancement,” *Pattern Recognition*, vol. 61, pp. 650–662, 2017. doi
20. L. Tao, C. Zhu, J. Song, T. Lu, H. Jia and X. Xie, “Low-light image enhancement using CNN and bright channel prior,” in *Proc. IEEE International Conference on Image Processing (ICIP)*, 2017, pp. 3215–3219. doi
21. C. Wei, W. Wang, W. Yang and J. Liu, “Deep retinex decomposition for low-light enhancement,” in *Proc. British Machine Vision Conference (BMVC)*, 2018. arXiv project
22. Y. Zhang, J. Zhang and X. Guo, “Kindling the darkness: A practical low-light image enhancer,” in *Proc. ACM International Conference on Multimedia (ACM MM)*, 2019, pp. 1632–1640. doi
23. Y. Zhang, X. Guo, J. Ma, W. Liu and J. Zhang, “Beyond brightening low-light images,” *International Journal of Computer Vision*, vol. 129, no. 4, pp. 1013–1037, 2021. doi
24. S. W. Zamir, A. Arora, S. Khan, et al., “Learning enriched features for real image restoration and enhancement,” in *Proc. European Conference on Computer Vision (ECCV)*, 2020, pp. 492–511. arXiv
25. X. Xu, R. Wang, C.-W. Fu and J. Jia, “SNR-aware low-light image enhancement,” in *Proc. IEEE/CVF CVPR*, 2022, pp. 17693–17703. doi

26. Y. Cai, H. Bian, J. Lin, H. Wang, R. Timofte and Y. Zhang, "Retinexformer: One-stage Retinex-based transformer for low-light image enhancement," in *Proc. IEEE/CVF ICCV*, 2023. arXiv code
27. Q. Yan, Y. Feng, C. Zhang, et al., "HVI: A new color space for low-light image enhancement," in *Proc. IEEE/CVF CVPR*, 2025. arXiv
28. S.-E. Weng, S.-G. Miaou and R. Christanto, "A lightweight low-light image enhancement network via channel prior and gamma correction," *International Journal of Pattern Recognition and Artificial Intelligence*, vol. 39, no. 12, art. 2554013, 2025. doi code
29. S.-E. Weng, C.-Y. Hsiao, L.-W. Lu, et al., "Rethinking theoretical illumination for efficient low-light image enhancement" (CPGA-Net+), arXiv:2409.05274, 2024. arXiv
30. R. Liu, L. Ma, J. Zhang, X. Fan and Z. Luo, "Retinex-inspired unrolling with cooperative prior architecture search for low-light image enhancement," in *Proc. IEEE/CVF CVPR*, 2021, pp. 10556–10565. arXiv
31. W. Wu, J. Weng, P. Zhang, X. Wang, W. Yang and J. Jiang, "URetinex-Net: Retinex-based deep unfolding network for low-light image enhancement," in *Proc. IEEE/CVF CVPR*, 2022, pp. 5891–5900. doi
32. Y. Jiang, X. Gong, D. Liu, et al., "EnlightenGAN: Deep light enhancement without paired supervision," *IEEE Transactions on Image Processing*, vol. 30, pp. 2340–2349, 2021. arXiv
33. C. Guo, C. Li, J. Guo, et al., "Zero-reference deep curve estimation for low-light image enhancement," in *Proc. IEEE/CVF CVPR*, 2020, pp. 1777–1786. arXiv
34. C. Li, C. Guo and C. C. Loy, "Learning to enhance low-light image via zero-reference deep curve estimation," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 8, 2022. arXiv
35. L. Ma, T. Ma, R. Liu, X. Fan and Z. Luo, "Toward fast, flexible, and robust low-light image enhancement," in *Proc. IEEE/CVF CVPR*, 2022, pp. 5627–5636. arXiv
36. Z. Liang, C. Li, S. Zhou, R. Feng and C. C. Loy, "Iterative prompt learning for unsupervised backlit image enhancement," in *Proc. IEEE/CVF ICCV*, 2023. arXiv
37. Y. Wang, R. Wan, W. Yang, H. Li, L.-P. Chau and A. C. Kot, "Low-light image enhancement with normalizing flow," in *Proc. AAAI Conference on Artificial Intelligence*, vol. 36, no. 3, pp. 2604–2612, 2022. arXiv
38. X. Yi, H. Xu, H. Zhang, L. Tang and J. Ma, "Diff-Retinex: Rethinking low-light image enhancement with a generative diffusion model," in *Proc. IEEE/CVF ICCV*, 2023. arXiv
39. J. Hou, Z. Zhu, J. Hou, H. Liu, H. Zeng and H. Yuan, "Global structure-aware diffusion process for low-light image enhancement," in *Advances in Neural Information Processing Systems* (*NeurIPS*), 2023. arXiv
40. H. Jiang, A. Luo, X. Liu, S. Han and S. Liu, "LightenDiffusion: Unsupervised low-light image enhancement with latent-Retinex diffusion models," in *Proc. European Conference on Computer Vision* (*ECCV*), 2024. arXiv
41. C. He, C. Fang, Y. Zhang, et al., "Reti-Diff: Illumination degradation image restoration with Retinex-based latent diffusion model," in *Proc. International Conference on Learning Representations* (*ICLR*), 2025. proceedings
42. J. Bai, Y. Yin, Q. He, Y. Li and X. Zhang, "RetinexMamba: Retinex-based Mamba for low-light image enhancement," arXiv:2405.03349, 2024. arXiv
43. M. V. Conde, G. Geigle and R. Timofte, "InstructIR: High-quality image restoration following

human instructions," in *Proc. European Conference on Computer Vision (ECCV)*, 2024. arXiv
44. C. Chen, Q. Chen, J. Xu and V. Koltun, "Learning to see in the dark," in *Proc. IEEE/CVF CVPR*, 2018, pp. 3291–3300. arXiv
45. Z. Cui, G.-J. Qi, L. Gu, S. You, Z. Zhang and T. Harada, "Multitask AET with orthogonal tangent regularity for dark object detection," in *Proc. IEEE/CVF ICCV*, 2021. arXiv
46. K. A. Hashmi, G. Kallempudi, D. Stricker and M. Z. Afzal, "FeatEnHancer: Enhancing hierarchical features for object detection and beyond under low-light vision," in *Proc. IEEE/CVF ICCV*, 2023. arXiv
47. Z. Wang, A. C. Bovik, H. R. Sheikh and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004. doi
48. R. Zhang, P. Isola, A. A. Efros, E. Shechtman and O. Wang, "The unreasonable effectiveness of deep features as a perceptual metric," in *Proc. IEEE/CVF CVPR*, 2018. arXiv
49. A. Mittal, R. Soundararajan and A. C. Bovik, "Making a 'completely blind' image quality analyzer," *IEEE Signal Processing Letters*, vol. 20, no. 3, pp. 209–212, 2013. doi
50. A. Mittal, A. K. Moorthy and A. C. Bovik, "No-reference image quality assessment in the spatial domain," *IEEE Transactions on Image Processing*, vol. 21, no. 12, pp. 4695–4708, 2012. doi
51. J. Liao, S. Hao, R. Hong and M. Wang, "GT-Mean loss: A simple yet effective solution for brightness mismatch in low-light image enhancement," in *Proc. IEEE/CVF ICCV*, 2025. arXiv
52. W. Yang, W. Wang, H. Huang, S. Wang and J. Liu, "Sparse gradient regularized deep Retinex network for robust low-light image enhancement," *IEEE Transactions on Image Processing*, vol. 30, pp. 2072–2086, 2021. doi
53. J. Hai, Z. Xuan, R. Yang, et al., "R2RNet: Low-light image enhancement via Real-low to Real-normal Network," *Journal of Visual Communication and Image Representation*, vol. 90, art. 103712, 2023. arXiv
54. R. Wang, X. Xu, C.-W. Fu, et al., "Seeing dynamic scene in the dark: A high-quality video dataset with mechatronic alignment," in *Proc. IEEE/CVF ICCV*, 2021, pp. 9680–9689. doi code
55. V. Bychkovsky, S. Paris, E. Chan and F. Durand, "Learning photographic global tonal adjustment with a database of input/output image pairs," in *Proc. IEEE CVPR*, 2011, pp. 97–104. dataset
56. J. Cai, S. Gu and L. Zhang, "Learning a deep single image contrast enhancer from multi-exposure images," *IEEE Transactions on Image Processing*, vol. 27, no. 4, pp. 2049–2062, 2018. doi
57. Y. P. Loh and C. S. Chan, "Getting to know low-light images with the Exclusively Dark dataset," *Computer Vision and Image Understanding*, vol. 178, pp. 30–42, 2019. arXiv
58. K. Ma, K. Zeng and Z. Wang, "Perceptual quality assessment for multi-exposure image fusion," *IEEE Transactions on Image Processing*, vol. 24, no. 11, pp. 3345–3356, 2015. doi
59. C. Lee, C. Lee and C.-S. Kim, "Contrast enhancement based on layered difference representation of 2D histograms," *IEEE Transactions on Image Processing*, vol. 22, no. 12, pp. 5372–5384, 2013. doi
60. V. Vonikakis, "Datasets" (source of the VV test images), personal web page. link
61. X. Liu, Z. Wu, A. Li, et al., "NTIRE 2024 challenge on low light image enhancement: Methods and results," in *Proc. IEEE/CVF CVPR Workshops*, 2024. arXiv
62. X. Liu, Z. Wu, F.-A. Vasluianu, et al., "NTIRE 2025 challenge on low light image enhancement:

Methods and results," in *Proc. IEEE/CVF CVPR Workshops*, 2025. arXiv

63. Y. Blau, R. Mechrez, R. Timofte, T. Michaeli and L. Zelnik-Manor, "The 2018 PIRM challenge on perceptual image super-resolution," in *Proc. European Conference on Computer Vision (ECCV) Workshops*, 2018. arXiv
64. C. Ma, C.-Y. Yang, X. Yang and M.-H. Yang, "Learning a no-reference quality metric for single-image super-resolution," *Computer Vision and Image Understanding*, vol. 158, pp. 1–16, 2017. doi arXiv
65. D. Hasler and S. E. Süsstrunk, "Measuring colorfulness in natural images," in *Proc. SPIE 5007, Human Vision and Electronic Imaging VIII*, 2003. doi
66. J. Immerkær, "Fast noise variance estimation," *Computer Vision and Image Understanding*, vol. 64, no. 2, pp. 300–302, 1996. doi
67. H. Talebi and P. Milanfar, "NIMA: Neural image assessment," *IEEE Transactions on Image Processing*, 2018. arXiv
68. J. Ke, Q. Wang, Y. Wang, P. Milanfar and F. Yang, "MUSIQ: Multi-scale image quality transformer," in *Proc. IEEE/CVF ICCV*, 2021. arXiv
69. J. Wang, K. C. K. Chan and C. C. Loy, "Exploring CLIP for assessing the look and feel of images," in *Proc. AAAI Conference on Artificial Intelligence*, 2023. arXiv
70. S. Yang, T. Wu, S. Shi, et al., "MANIQA: Multi-dimension attention network for no-reference image quality assessment," in *Proc. IEEE/CVF CVPR Workshops*, 2022, pp. 1190–1199. doi arXiv
71. H. Wu, Z. Zhang, W. Zhang, C. Chen, et al., "Q-Align: Teaching LMMs for visual scoring via discrete text-defined levels," in *Proc. International Conference on Machine Learning (ICML)*, PMLR 235, 2024. arXiv
72. V. Kannan, S. Malik, N. C. Babu and R. Soundararajan, "Quality assessment of low-light restored images: A subjective study and an unsupervised model," *IEEE Access*, vol. 11, pp. 68216–68230, 2023. doi arXiv
73. ITU-R, "Recommendation BT.500-15: Methodologies for the subjective assessment of the quality of television images," 2023. ITU
74. R. A. Bradley and M. E. Terry, "Rank analysis of incomplete block designs: The method of paired comparisons," *Biometrika*, vol. 39, no. 3–4, pp. 324–345, 1952. doi
75. C. Chen and J. Mo, "IQA-PyTorch: PyTorch toolbox for image quality assessment," software (pip package `pyiqa`). GitHub

DLCV 06 • Object Tracking: From Single-Object to Multi-Object

How computer vision follows objects through video: re-identification, single-object trackers from correlation filters to transformers and SAM 2, and multi-object tracking with Kalman filters, Hungarian matching, end-to-end models and the MOTA/IDF1/HOTA metrics.

Online edition: https://blog.shyandram.dev/en/learn/deep-learning/dl-06-object-tracking/

What you'll learn

- Where tracking sits after classification and detection, and the two jobs every tracker does: follow motion across frames and keep each object's identity.
- A clear taxonomy: re-identification versus temporal tracking, single-object (SOT/VOT) versus multi-object tracking (MOT), and multi-target multi-camera tracking (MTMCT).
- How single-object trackers evolved: correlation filters (MOSSE, KCF), Siamese networks (SiamFC, SiamRPN), one-stream transformers (OSTrack, SeqTrack) and video segmentation models (SAM 2, SAMURAI, SAM 3).
- Multi-object tracking in depth: the tracking-by-detection pipeline (detector, Kalman filter, appearance embedding, Hungarian assignment), the SORT family (SORT, DeepSORT, ByteTrack, OC-SORT, BoT-SORT), joint detection-and-embedding (JDE, FairMOT), end-to-end query-based trackers (MOTR, MOTRv2, MOTIP) and open-vocabulary tracking.
- How trackers are measured: CMC and mAP for Re-ID, success and precision for SOT, and MOTA, IDF1 and HOTA for MOT, including why HOTA was introduced.
- A working NumPy tracker you can run, with measured ID switches and metrics on a synthetic scene.

The big picture

A detector looks at one picture and says "there are three people here". A tracker watches a video and says "this is the *same* person as one second ago, and that one just walked behind a pillar and will come out again". Tracking adds time and memory to detection.

> **Explain like I'm five**
>
> Imagine you are watching a soccer game with your friend, and you each pick a favorite player. Finding every player on the field in one photo is **detection**. Keeping your eyes on *your* player as she runs, turns and gets hidden behind others for a moment is **single-object tracking**. Putting a numbered sticker on *every* player and making sure the stickers never get swapped, even when two players crash into each other, is **multi-object tracking**. And recognizing your player again when the TV switches to a different camera is **re-identification**.

Why it matters: tracking is how a self-driving car knows that a pedestrian is *walking toward* the road rather than just standing near it, how sports analytics measures how far each player ran, how a warehouse counts forklifts and robots, and how a video editor keeps a mask glued to a moving object. Each of these needs not just *where* things are, but *which one is which* over time.

About this tutorial

This tutorial is expanded from the author's earlier Medium post (written in Traditional Chinese), "物件追蹤 Object Tracking 簡介" [1], keeping its structure and adding the technical depth, a runnable tracker and an up-to-date survey. You can read the original post here.

From classification to detection to tracking

Plain version. Classification answers *what* is in an image. Detection answers *what and where*. Tracking answers *what, where, and which one, over time*.

The progression in [1] is a useful map of computer vision tasks:

1. **Image classification** maps an image I to a label y.
2. **Object detection** maps an image to a set of boxes with labels and confidence scores, $\mathcal{D} = \{(b_j, c_j, s_j)\}_{j=1}^{N}$, where b_j is a bounding box, c_j a class and $s_j \in [0, 1]$ a score. Detection imitates the human ability to locate targets in a scene; it is covered in depth in DLCV 03 ·Object Detection.
3. **Object tracking** takes a sequence of frames $I_1, \dots, I_T$ and outputs **trajectories**: for each object k, a sequence of boxes (or masks) $(b_t^k)_{t \in \mathcal{T}_k}$ over the set of frames $\mathcal{T}_k$ in which it is visible, all carrying the same identity k.

The key new ingredient is the identity label. Detection is stateless: frame t and frame $t+1$ are processed independently. Tracking is stateful: it carries information forward.

What tracking does

Plain version. Tracking imitates how people perceive moving things: we predict where something will go, look there, and keep treating it as the same object even when its appearance changes or it is briefly hidden.

More precisely, every tracker solves two coupled sub-problems:

- **Localization over time** (motion perception): estimate where each target is in each frame, ideally using the past to predict the future. This is what lets a tracker survive blur, missed detections and short occlusions.
- **Identity maintenance** (data association): decide which observation in frame t belongs to which target from frame $t-1$. This is what prevents two people who cross paths from swapping labels.

Single-object tracking puts most of its effort into the first problem, since there is only one identity. Multi-object tracking spends most of its effort on the second. Re-identification is the second problem with *no* motion information at all.

The difficulties that [1] lists for each family come from these two jobs: appearance changes (pose, lighting, scale, viewpoint), occlusion by other objects or the scene, similar-looking distractors, fast or non-linear motion, camera motion, and targets entering and leaving the scene.

A taxonomy of tracking

Plain version. You can sort trackers by *what* they follow (one object or many, one camera or many) and by *how* they keep identity (by appearance alone, or by following motion frame to frame).

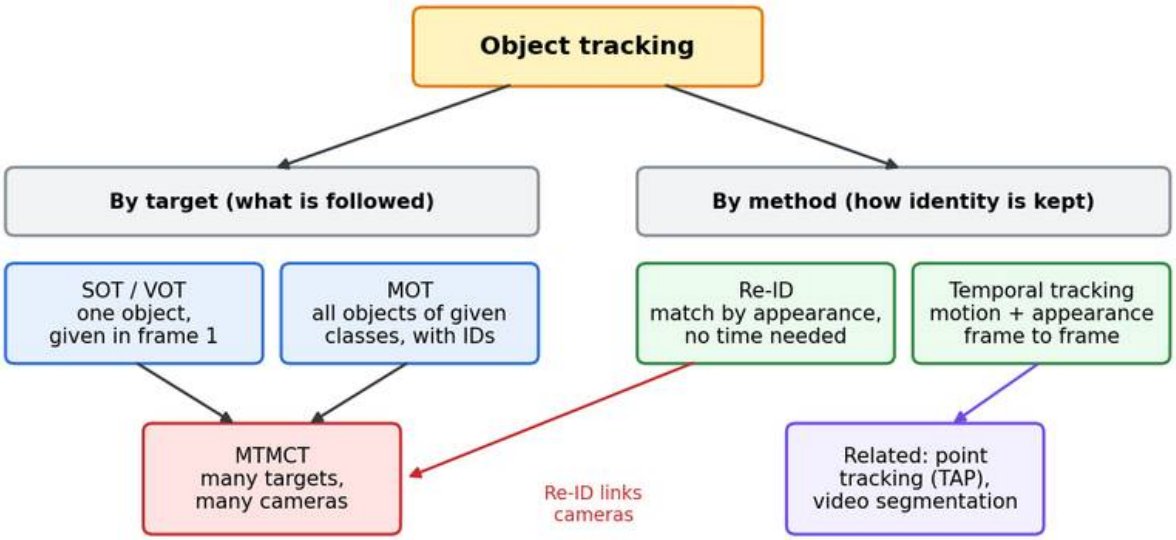


Figure 1 — A taxonomy of tracking. By target: single-object (SOT, also called VOT) and multi-object (MOT). By method: appearance-only re-identification and temporal tracking. The hardest combined setting is multi-target multi-camera tracking (MTMCT).

Following [1]:

- **By method.** *Re-identification* (*Re-ID*) matches a target across images that may be far apart in time or come from different cameras, using appearance (and sometimes pose) alone. *Temporal object tracking* follows targets frame by frame in one video stream, where motion continuity is available.
- **By target.** *Visual object tracking* (*VOT*), also called *single-object tracking* (*SOT*), follows one arbitrary object specified by a box in the first frame. *Multi-object tracking* (*MOT*) follows all objects

of given classes (for example, every pedestrian) through consecutive frames and maintains a consistent ID for each.
- **The combined goal.** *Multi-target multi-camera tracking* (*MTMCT*) tracks many targets across a network of cameras and keeps the same identity when a person leaves one camera's view and appears in another. It combines MOT within each camera with Re-ID between cameras.

Most of the classical literature, and most benchmarks, use **pedestrians** as targets [1], because surveillance and autonomous driving made people the most valuable class to track. Modern benchmarks add dancers, athletes, vehicles and open-vocabulary categories.

Person re-identification

Plain version. Re-ID is a search problem: given a photo of a person from one camera, find the same person in photos taken by other cameras, ranked from most to least likely.

Problem formulation

Re-ID is posed as **retrieval** [7]. A *query* image q and a *gallery* $\mathcal{G} = \{g_1, \dots, g_M\}$ of detected person crops from other cameras are mapped by a network f_θ to embeddings. The gallery is ranked by a distance, usually cosine distance

$$d(q, g) = 1 - \frac{f_\theta(q)^\top f_\theta(g)}{\|f_\theta(q)\| \, \|f_\theta(g)\|},$$

where $f_\theta(\cdot) \in \mathbb{R}^D$ is the D-dimensional embedding produced by the network with parameters θ. Training makes embeddings of the same identity close and of different identities far apart. A typical recipe combines an identity classification loss (cross-entropy over the C training identities) with a **triplet loss**

$$\mathcal{L}_{\text{tri}} = \left[\, d(a, p) - d(a, n) + m \,\right]_+,$$

where a is an anchor image, p a positive (same identity), n a negative (different identity), $m > 0$ a margin and $[x]_+ = \max(x, 0)$. Test identities never appear in training, so Re-ID is an open-set problem: the network must learn a general notion of "same person".

Challenges

[1] groups the difficulties into **data factors** and **environment factors**. Data factors: limited labeled identities, noisy boxes from the detector, partial bodies, low resolution. Environment factors: different viewpoints and camera color responses, lighting, occlusion, background clutter, and people who change clothes. Re-ID also raises clear **privacy** concerns, which is part of why surveillance-style datasets have been withdrawn (below). The original post names city-scale surveillance systems as the main application; today Re-ID is just as common as the appearance module inside MOT trackers and in retail and sports analytics.

Representative models

- **Strong baselines.** Luo et al. [14] showed that a plain ResNet-50 with a set of training "tricks" — warm-up learning rate, random erasing augmentation, label smoothing, last-stride 1 for a larger feature map, and a **BNNeck** (a batch-norm layer that separates the triplet-loss feature from the classification feature) — is a strong, reproducible baseline.
- **Centroid triplet loss (CTL).** The model the original post cites (ResNet-50 at 256 × 128 input) [15] replaces instance-to-instance triplets with distances to *class centroids*, and at retrieval time compares a query to identity centroids rather than to every gallery image.
- **CLIP-ReID.** Li et al. [16] adapt the vision-language model CLIP to Re-ID, where no text labels exist. In a first stage they learn identity-specific text tokens with the encoders frozen; in a second stage they fine-tune the image encoder using those text features as extra supervision.

Datasets

Dataset	Content	Status
Market-1501 [12]	1,501 identities, 6 cameras, detector-cropped boxes	available
MSMT17 [13]	4,101 identities, 15 indoor and outdoor cameras	available
DukeMTMC [10] (and DukeMTMC-reID)	8 synchronized campus cameras	**withdrawn in 2019**

DukeMTMC was introduced with the IDF1 metric for multi-camera tracking [10], and a Re-ID subset was widely used. Its creators withdrew it in 2019 after privacy and ethics concerns about recording people on a campus, and it should no longer be downloaded or used; Peng et al. [11] trace how derived copies kept circulating afterwards and argue for better dataset stewardship.

Metrics: CMC rank-k and mAP

The **cumulative matching characteristic** at rank k is the fraction of queries whose first correct match appears in the top k of the ranked gallery:

$$\mathrm{CMC}(k) = \frac{1}{|Q|} \sum_{q \in Q} \mathbb{1}\left[\, r_q \le k \,\right],$$

where Q is the set of queries and r_q is the rank of the first true match for query q. *Rank-1 accuracy* is $\mathrm{CMC}(1)$. Because a query usually has several true matches in the gallery, rank-k alone ignores how well the *others* are ranked. **Mean average precision** fixes this:

$$\mathrm{AP}(q) = \frac{1}{G_q} \sum_{i=1}^{G_q} \frac{i}{r_{q,i}}, \qquad \mathrm{mAP} = \frac{1}{|Q|} \sum_{q} \mathrm{AP}(q),$$

where G_q is the number of true matches of q in the gallery and $r_{q,i}$ is the rank of the i-th one. So $i/r_{q,i}$ is the precision at the moment the i-th true match is found.

```
import numpy as np

def cmc_map(dist, q_ids, g_ids, max_rank=5):
    """dist: (Q, G) distances between query and gallery embeddings."""
    order = np.argsort(dist, axis=1)                # closest gallery items first
    match = g_ids[order] == q_ids[:, None]          # (Q, G) True where same person
    cmc = (np.cumsum(match, axis=1) > 0).mean(axis=0)[:max_rank]
    aps = []
    for row in match:
        hits = np.flatnonzero(row)                  # ranks (0-based) of true matches
        precision_at_hits = np.arange(1, len(hits) + 1) / (hits + 1)
        aps.append(precision_at_hits.mean())
    return cmc, np.mean(aps)

rng = np.random.default_rng(0)
centres = rng.normal(size=(50, 64))                 # 50 identities, 64-D embeddings
g_ids = np.repeat(np.arange(50), 4)                 # 4 gallery images per identity
q_ids = np.arange(50)
gallery = centres[g_ids] + 1.6 * rng.normal(size=(200, 64))
query = centres[q_ids] + 1.6 * rng.normal(size=(50, 64))
norm = lambda x: x / np.linalg.norm(x, axis=1, keepdims=True)
dist = 1 - norm(query) @ norm(gallery).T            # cosine distance
cmc, mAP = cmc_map(dist, q_ids, g_ids)
print("rank-1 %.2f  rank-5 %.2f  mAP %.2f" % (cmc[0], cmc[4], mAP))
```

On this synthetic gallery (4 images per identity) the script prints `rank-1 0.76  rank-5 0.92  mAP 0.57`: most queries find *one* correct image near the top, but the other three are scattered, which only mAP reveals.

Single-object tracking

Plain version. You draw a box around something in the first frame; the tracker must keep a box on that same thing for the rest of the video, without knowing in advance what kind of object it is.

Problem formulation

Given frames $I_1, \dots, I_T$ and an initial box b_1, a single-object tracker outputs $\hat{b}_2, \dots, \hat{b}_T$. The target is **class-agnostic**: it could be a person, a bird or a coffee cup. This makes SOT a form of *one-shot* learning: the only example of the target is the first-frame patch, the **template** z. In each new frame the tracker searches a region x around the last position. The challenges [1] lists are complex appearance changes, unusual motion patterns, and environmental change such as lighting and occlusion.

Classical: correlation filters

Before deep learning, the fastest trackers learned a **correlation filter** online. MOSSE [17] learns a filter H in the Fourier domain so that correlating it with training patches F_i produces sharp Gaussian peaks G_i at the target center:

$$\min_{H^*} \sum_i |F_i \odot H^* - G_i|^2 \quad \Rightarrow \quad H^* = \frac{\sum_i G_i \odot F_i^*}{\sum_i F_i \odot F_i^*},$$

where capital letters are 2-D discrete Fourier transforms, $\odot$ is element-wise multiplication, * is complex conjugation and the division is element-wise. Because correlation becomes multiplication in the frequency domain (see IP 04), training and detection cost only a few FFTs, so these trackers run at hundreds of frames per second on a CPU. KCF [18] showed that training on *all cyclic shifts* of a patch gives a circulant data matrix that the DFT diagonalizes, which turns kernel ridge regression into element-wise operations and allows multi-channel features such as HOG. Their weaknesses are a fixed search window, boundary effects from the cyclic assumption, and hand-crafted features.

Siamese trackers

Plain version. Train one network to answer a single question — "where in this search image does the template appear?" — on huge amounts of video offline, then use it frozen at test time.

SiamFC [19] applies the *same* fully convolutional network φ to the template z and the search region x and cross-correlates the two feature maps:

$$f(z, x) = \varphi(z) \star \varphi(x) + b\,\mathbb{1},$$

where $\star$ is cross-correlation (the template features act as a convolution kernel), b is a learned bias and $\mathbb{1}$ is a map of ones. The output is a score map whose peak gives the new position. It is trained with a logistic loss on score-map positions labeled positive near the true center and negative elsewhere. Because nothing is updated online, it is fast and robust to drift, but it only predicts position, with scale handled by searching a small pyramid.

SiamRPN [20] adds a **region proposal network** after the Siamese features: one branch classifies anchors as target or background, another regresses box offsets. Tracking becomes "one-shot detection" with the template as the class definition, which yields accurate boxes with changing aspect ratios.

Transformer one-stream trackers

From 2022 the mainstream moved from Siamese CNNs to transformers. The key change is *where* the template and the search region interact (Figure 2). In a two-stream design they are encoded separately and compared once. In a **one-stream** design such as OSTrack [21], the image patches of z and x are concatenated into one token sequence and processed by a single Vision Transformer, so that every attention layer both extracts features and relates template to search. OSTrack also

drops search tokens that are clearly background in early layers to save computation. SeqTrack [22] goes further in simplicity: an encoder-decoder transformer generates the four box coordinates as a sequence of discrete tokens, autoregressively, trained with plain cross-entropy — the same recipe as language modeling.

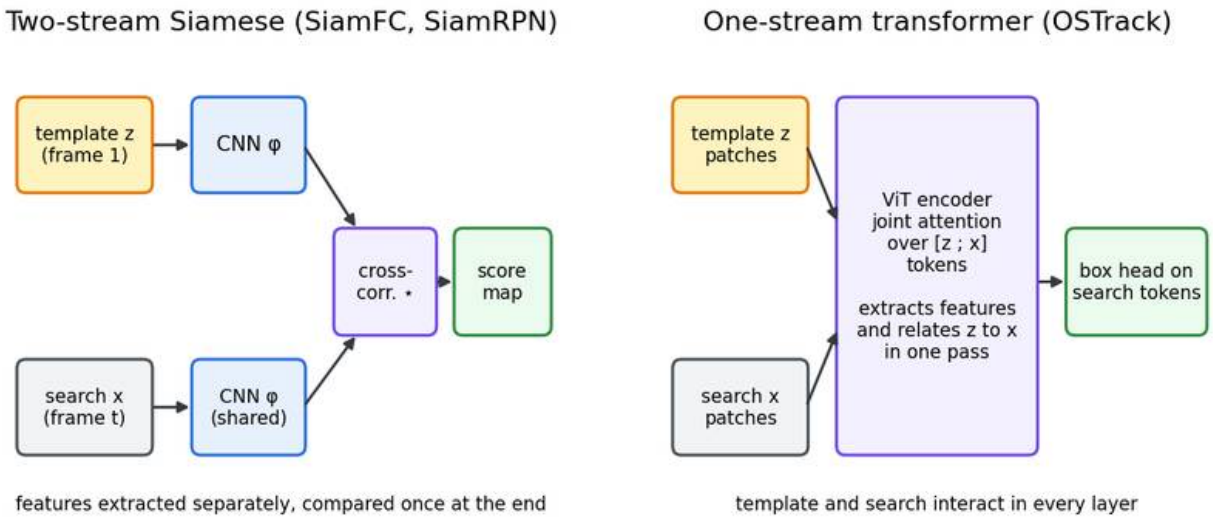


Figure 2 — Two-stream Siamese trackers compare template and search features once, at the end. One-stream transformer trackers let them interact in every layer.

Tracking with video segmentation models

Plain version. New "segment anything in video" models can be told "this object" with one click, and then follow it for the whole video as a pixel mask. That *is* single-object tracking, with a mask instead of a box.

SAM 2 [23] extends the Segment Anything model to video with a **streaming memory**: features and predicted masks from past frames, plus the prompted frames, are stored in a memory bank that the current frame attends to. Prompting one frame with a click, box or mask yields a "masklet" through the whole video, so it behaves like a mask-output SOT tracker. SAMURAI [24] adapts SAM 2 for tracking without any retraining: it adds a Kalman-filter motion model to choose among SAM 2's candidate masks and a motion-aware rule for which past frames enter memory, which helps in crowded scenes with similar distractors. SAM 3 [25] adds **promptable concept segmentation**: given a short noun phrase such as "yellow school bus" or an image exemplar, it detects, segments and tracks *all* matching instances in a video and gives each its own identity. That is a form of open-vocabulary MOT, and it blurs the old SOT/MOT boundary.

Benchmarks and metrics

Benchmark	What makes it distinct
OTB [26]	early standard benchmark with per-sequence attribute labels (occlusion, fast motion, ...); introduced the success and precision plots
VOT challenge [53]	yearly challenge series since 2013, with reset-based protocols; now VOTS, tracking *and* segmentation
LaSOT [27]	long-term: 1,400 sequences, more than 3.5 million frames, each frame manually annotated
GOT-10k [28]	one-shot protocol: training and test object classes do not overlap
TrackingNet [29]	large-scale, in-the-wild videos with a held-out evaluation server

The two classic measures, from OTB [26], are:

$$\text{Success}(\tau) = \frac{1}{T}\sum_{t=1}^{T} \mathbb{1}[\,\text{IoU}(\hat{b}_t, b_t) > \tau\,], \qquad \text{AUC} = \int_0^1 \text{Success}(\tau)\,d\tau,$$

$$\text{Precision}(\delta) = \frac{1}{T}\sum_{t=1}^{T} \mathbb{1}[\|\hat{c}_t - c_t\|_2 \le \delta],$$

where $\hat{b}_t, b_t$ are the predicted and ground-truth boxes, τ is an IoU threshold, $\hat{c}_t, c_t$ are their centers and δ is a pixel distance, conventionally 20. The AUC of the success plot is the headline number on OTB and LaSOT. GOT-10k reports **average overlap** (the mean IoU) and success rates at $\tau = 0.5$ and 0.75.

```
import numpy as np

def iou_xywh(a, b):
    """IoU between rows of a and b, boxes as [x, y, w, h] (top-left corner)."""
    x1 = np.maximum(a[:, 0], b[:, 0]); y1 = np.maximum(a[:, 1], b[:, 1])
    x2 = np.minimum(a[:, 0] + a[:, 2], b[:, 0] + b[:, 2])
    y2 = np.minimum(a[:, 1] + a[:, 3], b[:, 1] + b[:, 3])
    inter = np.clip(x2 - x1, 0, None) * np.clip(y2 - y1, 0, None)
    return inter / (a[:, 2] * a[:, 3] + b[:, 2] * b[:, 3] - inter)

def success_auc(pred, gt, thresholds=np.linspace(0, 1, 21)):
    ious = iou_xywh(pred, gt)
    curve = np.array([(ious > t).mean() for t in thresholds])
    return curve.mean()                              # area under the success plot

def precision_at(pred, gt, px=20):
    c_pred = pred[:, :2] + pred[:, 2:] / 2; c_gt = gt[:, :2] + gt[:, 2:] / 2
    return (np.linalg.norm(c_pred - c_gt, axis=1) <= px).mean()

rng = np.random.default_rng(0)
gt = np.c_[np.linspace(50, 400, 300), np.full(300, 80.0), np.full(300, 40.0), np.full(300,
 ↪  60.0)]
pred = gt + rng.normal(0, 4, gt.shape)
```

```
pred[200:230, :2] += 120                              # 30 frames lost on a distractor
print("success AUC %.3f   precision@20px %.3f" % (success_auc(pred, gt), precision_at(pred,
 ↪  gt)))
```

This prints `success AUC 0.647   precision@20px 0.900`: the 30 frames spent on a distractor cost precision 10 points, while the small jitter on the other frames costs AUC much more, because AUC also rewards tight boxes.

Multi-object tracking

Plain version. Find every object of interest in every frame, and give each one a number that stays the same for as long as it is in view — even when objects cross, hide behind each other, or leave and come back.

MOT tracks multiple targets **across consecutive frames** and maintains a consistent ID for each; a single frame on its own only supports detection. The challenges [1] lists are frequent occlusion, deciding when trajectories start and end, similar appearance between targets, and interaction between targets (people in a crowd move together and occlude each other).

Formally, given detections $\mathcal{D}_t$ in each frame, MOT seeks a set of trajectories $\mathcal{T} = \{\tau_k\}$ that partitions the true detections, discards false positives, and fills in missed ones. Offline methods solve this over the whole video, often as a graph or network-flow problem; **online** methods, which this section focuses on, must decide at frame t using only frames $1..t$, which is what real-time applications require.

The tracking-by-detection pipeline

The dominant design for a decade has been **tracking-by-detection** [2], [4]: run a detector on every frame, then link detections over time (Figure 3). Each frame goes through four steps:

1. **Detect.** A detector (Faster R-CNN, YOLO-family, DETR-family; see DLCV 03) gives boxes and scores.
2. **Predict.** A motion model moves every existing track forward to where it should be now.
3. **Describe.** Optionally, a Re-ID network embeds each detection crop.
4. **Associate.** Build a cost matrix between predicted tracks and detections and solve the assignment problem. Matched tracks are updated; unmatched detections may start new tracks (*birth*); tracks unmatched for too long are deleted (*death*).

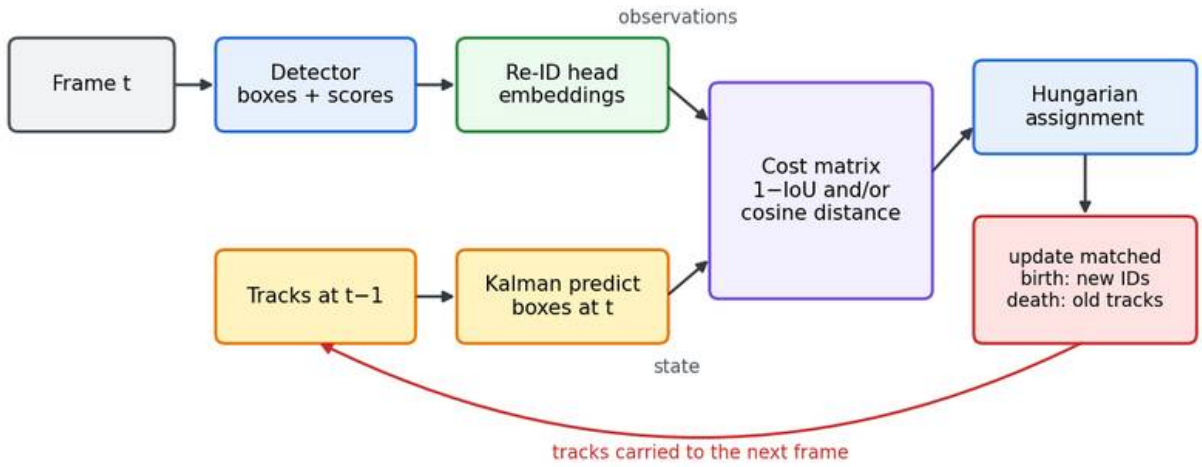


Figure 3 — Online tracking-by-detection. Observations (boxes, embeddings) and state (predicted tracks) meet in a cost matrix; the assignment result updates the state for the next frame.

Motion model: the Kalman filter

Plain version. Guess where the object is now from where it was and how fast it was moving; then nudge that guess toward the detection, trusting whichever of the two is less uncertain.

Nearly all tracking-by-detection systems use a **linear Kalman filter** [30] with a constant-velocity model. In our code the state of one track is

$$\mathbf{x} = [\, c_x,\ c_y,\ w,\ h,\ v_x,\ v_y \,]^\top,$$

where (c_x, c_y) is the box center, w, h its width and height, and (v_x, v_y) the center velocity in pixels per frame. (SORT uses the center, area and aspect ratio instead of w, h [32]; BoT-SORT returns to w and h [36].) The model has two steps.

Predict (time update):

$$\begin{aligned} \hat{\mathbf{x}}_{t|t-1} &= \mathbf{F}\,\hat{\mathbf{x}}_{t-1|t-1}, \\ \mathbf{P}_{t|t-1} &= \mathbf{F}\,\mathbf{P}_{t-1|t-1}\,\mathbf{F}^\top + \mathbf{Q}, \end{aligned}$$

where $\mathbf{F}$ is the 6×6 transition matrix (identity plus 1s that add v_x, v_y to c_x, c_y, since $\Delta t = 1$ frame), $\mathbf{P}$ is the state covariance (our uncertainty) and $\mathbf{Q}$ is the process noise covariance (how much the true motion can deviate from constant velocity).

Update (measurement update) with a matched detection $\mathbf{z}_t = [c_x, c_y, w, h]^\top$:

$$\begin{aligned} \mathbf{S}_t &= \mathbf{H}\,\mathbf{P}_{t|t-1}\,\mathbf{H}^\top + \mathbf{R}, \qquad \mathbf{K}_t = \mathbf{P}_{t|t-1}\,\mathbf{H}^\top\,\mathbf{S}_t^{-1}, \\ \hat{\mathbf{x}}_{t|t} &= \hat{\mathbf{x}}_{t|t-1} + \mathbf{K}_t(\mathbf{z}_t - \mathbf{H}\,\hat{\mathbf{x}}_{t|t-1}), \qquad \mathbf{P}_{t|t} = (\mathbf{I} - \mathbf{K}_t\mathbf{H})\,\mathbf{P}_{t|t-1}, \end{aligned}$$

where $\mathbf{H} = [\, \mathbf{I}_4 \ \ \mathbf{0} \,]$ picks the observed part of the state, $\mathbf{R}$ is the detector's measurement noise

covariance, $\mathbf{z}_t - \mathbf{H}\hat{\mathbf{x}}_{t|t-1}$ is the **innovation** (surprise), $\mathbf{S}_t$ its covariance, and $\mathbf{K}_t$ the **Kalman gain**. If the prediction is very uncertain ($\mathbf{P}$ large), the update moves the estimate almost all the way to the detection; if the detector is noisy ($\mathbf{R}$ large), it trusts the prediction. When a detection is missing, the tracker runs only the predict step ("coasting"), and $\mathbf{P}$ grows (Figure 4).

```
import numpy as np
from scipy.optimize import linear_sum_assignment

# ---------- 1. constant-velocity Kalman filter ----------
class KalmanBox:
    """State x = [cx, cy, w, h, vx, vy]; measurement z = [cx, cy, w, h]."""
    def __init__(self, z, q=1.0, r=4.0):
        self.x = np.r_[z, 0.0, 0.0]
        self.P = np.diag([r, r, r, r, 100.0, 100.0])    # unknown velocity: large variance
        self.F = np.eye(6); self.F[0, 4] = self.F[1, 5] = 1.0   # dt = 1 frame
        self.H = np.eye(4, 6)
        self.Q = q * np.diag([1, 1, 0.1, 0.1, 0.5, 0.5])
        self.R = r * np.eye(4)

    def predict(self):
        self.x = self.F @ self.x
        self.P = self.F @ self.P @ self.F.T + self.Q
        return self.x[:4]

    def update(self, z):
        S = self.H @ self.P @ self.H.T + self.R          # innovation covariance
        K = self.P @ self.H.T @ np.linalg.inv(S)          # Kalman gain
        self.x = self.x + K @ (z - self.H @ self.x)
        self.P = (np.eye(6) - K @ self.H) @ self.P
```

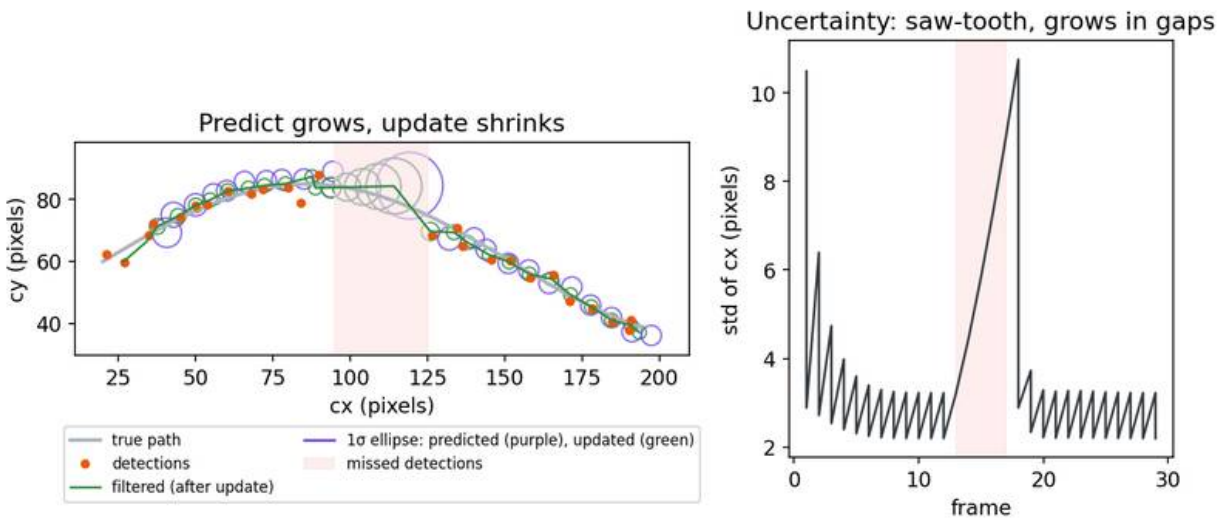


Figure 4 — Kalman predict and update on a synthetic track. Every predict step inflates the uncertainty, every update shrinks it. During five missed detections (pink) the filter coasts in a straight line and its

uncertainty grows; this is exactly when identity errors happen.

The squared **Mahalanobis distance** $d_M^2 = \boldsymbol{\nu}^\top \mathbf{S}^{-1}\boldsymbol{\nu}$ of an innovation $\boldsymbol{\nu}$ follows a χ^2 distribution with 4 degrees of freedom under the model, so DeepSORT [33] uses it to *gate* (forbid) implausible matches. The constant-velocity assumption is also the main weakness: dancers, athletes and anything filmed by a moving camera violate it.

Appearance embeddings

A Re-ID network (previous section) turns each detection crop into a unit vector $\mathbf{e}_j$. Each track keeps an appearance memory, for example an exponential moving average $\mathbf{m}_k \leftarrow \alpha\,\mathbf{m}_k + (1-\alpha)\,\mathbf{e}_j$ followed by renormalization, with $\alpha \in [0,1)$ (we use 0.9). The appearance cost is the cosine distance $1 - \mathbf{m}_k^\top \mathbf{e}_j$. Appearance is what lets a tracker recover an identity after a long occlusion, when motion prediction has become useless.

Association: cost matrices and the Hungarian algorithm

Plain version. Make a table of "how bad would it be to match track k with detection j" for every pair, then choose the one-to-one matching with the smallest total badness.

With K predicted tracks and N detections, build a cost matrix $\mathbf{C} \in \mathbb{R}^{K\times N}$, for example

$$C_{kj} = \lambda\,(1 - \mathrm{IoU}(\hat{b}_k, b_j)) + (1-\lambda)\,(1 - \mathbf{m}_k^\top \mathbf{e}_j),$$

where $\hat{b}_k$ is the predicted box of track k, b_j detection j's box and $\lambda \in [0,1]$ the weight of motion versus appearance ($\lambda = 1$ is pure IoU, as in SORT). Then solve the **linear assignment problem**

$$\min_{\mathbf{X}\in\{0,1\}^{K\times N}} \sum_{k,j} C_{kj} X_{kj} \quad \text{s.t.} \quad \sum_j X_{kj} \le 1,\ \sum_k X_{kj} \le 1,$$

so each track takes at most one detection and vice versa. The **Hungarian algorithm** [31] solves it exactly in polynomial time ($O(n^3)$ for $n = \max(K,N)$); SciPy's `linear_sum_assignment` implements a fast variant. Finally, any matched pair whose cost exceeds a threshold is rejected, which is how gating is applied in practice.

```
# ---------- 2. IoU and Hungarian association ----------
def iou_matrix(a, b):
    """a: (N,4), b: (M,4) boxes as [cx, cy, w, h] -> (N,M) IoU."""
    a1, a2 = a[:, None, :2] - a[:, None, 2:] / 2, a[:, None, :2] + a[:, None, 2:] / 2
    b1, b2 = b[None, :, :2] - b[None, :, 2:] / 2, b[None, :, :2] + b[None, :, 2:] / 2
    wh = np.clip(np.minimum(a2, b2) - np.maximum(a1, b1), 0, None)
    inter = wh[..., 0] * wh[..., 1]
    area = lambda x: x[..., 2] * x[..., 3]
    return inter / (area(a)[:, None] + area(b)[None, :] - inter + 1e-9)

def associate(cost, max_cost):
    """Hungarian assignment; pairs whose cost exceeds max_cost are rejected."""
    if cost.size == 0:
```

```
        return [], list(range(cost.shape[0])), list(range(cost.shape[1]))
    rows, cols = linear_sum_assignment(cost)
    pairs = [(r, c) for r, c in zip(rows, cols) if cost[r, c] <= max_cost]
    mr = {r for r, _ in pairs}; mc = {c for _, c in pairs}
    return (pairs, [r for r in range(cost.shape[0]) if r not in mr],
            [c for c in range(cost.shape[1]) if c not in mc])
```

A complete tracker on a synthetic scene

To see what each idea buys, we generate a scene with five pedestrians over 80 frames, designed to contain the classic failure cases:

- objects 0 and 1 **cross**, and while they overlap, object 1 (behind) gets only **low-confidence** detections;
- objects 2 and 3 meet and **bounce back** (non-linear motion, like dancers), with the rear one also low-confidence during the overlap;
- object 4 walks **behind a pole** for 8 frames, so its detections are low-confidence there;
- every detection is dropped with 5% probability, and there are occasional low-score false positives.

Each detection also carries a 16-D appearance vector: its object's fixed unit vector plus noise.

```
# ---------- 3. synthetic scene ----------
def make_scene(T=80, seed=0):
    rng = np.random.default_rng(seed)
    W, H = 30.0, 70.0
    t = np.arange(T, dtype=float)
    tr = {}
    # pair 1: plain crossing, B is behind A (occluded while they overlap)
    tr[0] = np.c_[80 + 5 * t, np.full(T, 120.0)]
    tr[1] = np.c_[480 - 5 * t, np.full(T, 128.0)]
    # pair 2: they meet and bounce back (non-linear motion, like dancers)
    m = 40
    x2 = np.where(t < m, 120 + 4 * t, 120 + 4 * m - 4 * (t - m))
    x3 = np.where(t < m, 440 - 4 * t, 440 - 4 * m + 4 * (t - m))
    tr[2] = np.c_[x2 + 10, np.full(T, 260.0)]
    tr[3] = np.c_[x3 - 10, np.full(T, 266.0)]
    # walker passing behind a pole (frames 30-37): detections become low-score
    tr[4] = np.c_[60 + 3 * t, 40 + 1.0 * t]
    gt = []                                   # per frame list of (id, box)
    dets = []                                 # per frame list of (box, score, gt_id)
    feats = {k: (lambda v: v / np.linalg.norm(v))(rng.normal(size=16)) for k in tr}
    for f in range(T):
        g, d = [], []
        boxes = {k: np.r_[tr[k][f], W, H] for k in tr}
        for k, b in boxes.items():
            g.append((k, b))
            score = rng.uniform(0.6, 0.95)
            if k == 1 and iou_matrix(b[None], boxes[0][None])[0, 0] > 0.2:
```

```
                score = rng.uniform(0.15, 0.45)       # occluded by object 0
            if k == 3 and iou_matrix(b[None], boxes[2][None])[0, 0] > 0.2:
                score = rng.uniform(0.15, 0.45)       # occluded by object 2
            if k == 4 and 30 <= f <= 37:
                score = rng.uniform(0.15, 0.45)       # behind the pole
            if rng.random() < 0.05:
                continue                               # random missed detection
            noisy = b + np.r_[rng.normal(0, 2.0, 2), rng.normal(0, 1.0, 2)]
            e = feats[k] + 0.15 * rng.normal(size=16)
            d.append((noisy, score, k, e / np.linalg.norm(e)))
        if rng.random() < 0.3:                         # low-score false positive
            fp = np.r_[rng.uniform(50, 550), rng.uniform(50, 320), W, H]
            e = rng.normal(size=16)
            d.append((fp, rng.uniform(0.1, 0.4), -1, e / np.linalg.norm(e)))
        gt.append(g); dets.append(d)
    return gt, dets
```

The tracker below has three switches. With both off it is **SORT-style**: Kalman prediction, IoU cost, Hungarian matching on high-score detections (score ≥ 0.5), tracks deleted after 2 missed frames. `use_low=True` adds a **BYTE**-style second pass: tracks left unmatched try the low-score detections, with IoU only. `use_app=True` adds the **appearance** cost to the first pass and a third pass that re-identifies tracks lost for up to 30 frames by appearance alone.

```
# ---------- 4. SORT-style tracker with optional BYTE and appearance ----------
class Track:
    def __init__(self, tid, z, emb):
        self.id, self.kf, self.emb = tid, KalmanBox(z), emb
        self.lost = 0

def run_tracker(dets, use_low=False, use_app=False, hi=0.5, max_age=2,
                app_age=30, lam=0.5):
    tracks, out, next_id = [], [], 0
    for d in dets:
        for tk in tracks:
            tk.kf.predict()
        boxes = np.array([x[0] for x in d]).reshape(-1, 4)
        scores = np.array([x[1] for x in d])
        embs = np.array([x[3] for x in d]).reshape(-1, 16)
        high = np.where(scores >= hi)[0]
        low = np.where((scores < hi) & (scores >= 0.1))[0]
        active = [tk for tk in tracks if tk.lost <= max_age]
        pred = np.array([tk.kf.x[:4] for tk in active]).reshape(-1, 4)
        # stage 1: high-score detections vs active tracks
        cost = 1 - iou_matrix(pred, boxes[high])
        gate = 0.7
        if use_app and len(active) and len(high):
            tE = np.array([tk.emb for tk in active])
            app = 1 - tE @ embs[high].T            # cosine distance
            cost = np.where(cost > 0.9, 9.0, lam * cost + (1 - lam) * app)
            gate = 0.6
        pairs, ut, ud = associate(cost, gate)
```

```
        matched = [(active[r], high[c]) for r, c in pairs]
        rest_tracks = [active[r] for r in ut]
        rest_high = [high[c] for c in ud]
        # stage 2 (BYTE): remaining tracks vs low-score detections, IoU only
        if use_low and rest_tracks and len(low):
            p2 = np.array([tk.kf.x[:4] for tk in rest_tracks])
            pairs2, ut2, _ = associate(1 - iou_matrix(p2, boxes[low]), 0.5)
            matched += [(rest_tracks[r], low[c]) for r, c in pairs2]
            rest_tracks = [rest_tracks[r] for r in ut2]
        # stage 3 (appearance only): long-lost tracks vs leftover high-score dets
        if use_app and rest_high:
            lost = [tk for tk in tracks if max_age < tk.lost <= app_age]
            if lost:
                app = 1 - np.array([tk.emb for tk in lost]) @ embs[rest_high].T
                pairs3, _, ud3 = associate(app, 0.3)
                matched += [(lost[r], rest_high[c]) for r, c in pairs3]
                rest_high = [rest_high[c] for c in ud3]
        for tk, j in matched:
            tk.kf.update(boxes[j]); tk.lost = 0
            tk.emb = 0.9 * tk.emb + 0.1 * embs[j]; tk.emb /= np.linalg.norm(tk.emb)
        hit = {id(tk) for tk, _ in matched}
        for tk in tracks:
            if id(tk) not in hit:
                tk.lost += 1
        for j in rest_high:                          # birth: unmatched high-score
            tracks.append(Track(next_id, boxes[j], embs[j])); next_id += 1
        tracks = [tk for tk in tracks if tk.lost <= (app_age if use_app else max_age)]
        out.append([(tk.id, tk.kf.x[:4].copy()) for tk in tracks if tk.lost == 0])
    return out
```

The metric code (MOTA, IDF1 and HOTA, explained below) and the experiment:

```
# ---------- 5. metrics: MOTA, IDF1, HOTA ----------
def frame_match(g, p, thr):
    if not g or not p:
        return []
    iou = iou_matrix(np.array([b for _, b in g]), np.array([b for _, b in p]))
    r, c = linear_sum_assignment(-iou)
    return [(g[i][0], p[j][0]) for i, j in zip(r, c) if iou[i, j] >= thr]

def mota(gt, hyp, thr=0.5):
    fn = fp = idsw = 0; ngt = 0; last = {}
    for g, p in zip(gt, hyp):
        m = frame_match(g, p, thr)
        ngt += len(g); fn += len(g) - len(m); fp += len(p) - len(m)
        for gi, pi in m:
            if gi in last and last[gi] != pi:
                idsw += 1
            last[gi] = pi
    return 1 - (fn + fp + idsw) / ngt, idsw

def idf1(gt, hyp, thr=0.5):
```

```
    gids = sorted({i for g in gt for i, _ in g}); pids = sorted({i for p in hyp for i, _ in
     ↪  p})
    gi = {k: n for n, k in enumerate(gids)}; pi = {k: n for n, k in enumerate(pids)}
    ov = np.zeros((len(gids), len(pids)))
    for g, p in zip(gt, hyp):
        if g and p:
            iou = iou_matrix(np.array([b for _, b in g]), np.array([b for _, b in p]))
            for a in range(len(g)):
                for b in range(len(p)):
                    ov[gi[g[a][0]], pi[p[b][0]]] += iou[a, b] >= thr
    r, c = linear_sum_assignment(-ov)
    idtp = ov[r, c].sum()
    n_gt = sum(len(g) for g in gt); n_p = sum(len(p) for p in hyp)
    return 2 * idtp / (n_gt + n_p)

def hota(gt, hyp, alphas=np.arange(0.05, 0.96, 0.05)):
    res = []
    for a in alphas:
        pairs = []; n_gt = n_p = 0
        for g, p in zip(gt, hyp):
            pairs += frame_match(g, p, a); n_gt += len(g); n_p += len(p)
        tp = len(pairs)
        if tp == 0:
            res.append((0, 0, 0)); continue
        cnt = {}
        for q in pairs:
            cnt[q] = cnt.get(q, 0) + 1
        gcount = {}; pcount = {}
        for g in gt:
            for i, _ in g: gcount[i] = gcount.get(i, 0) + 1
        for p in hyp:
            for i, _ in p: pcount[i] = pcount.get(i, 0) + 1
        ass = sum(cnt[q] / (gcount[q[0]] + pcount[q[1]] - cnt[q]) for q in pairs) / tp
        det = tp / (n_gt + n_p - tp)
        res.append((np.sqrt(det * ass), det, ass))
    return np.array(res).mean(axis=0)   # HOTA, DetA, AssA averaged over alpha

gt, dets = make_scene(seed=3)
for name, kw in [("IoU only", {}), ("+ low-score pass", dict(use_low=True)),
                 ("+ appearance", dict(use_low=True, use_app=True))]:
    hyp = run_tracker(dets, **kw)
    m, sw = mota(gt, hyp)
    print(f"{name:18s} MOTA {m:.2f}  IDF1 {idf1(gt, hyp):.2f}  "
          f"HOTA {hota(gt, hyp)[0]:.2f}  ID switches {sw}")
```

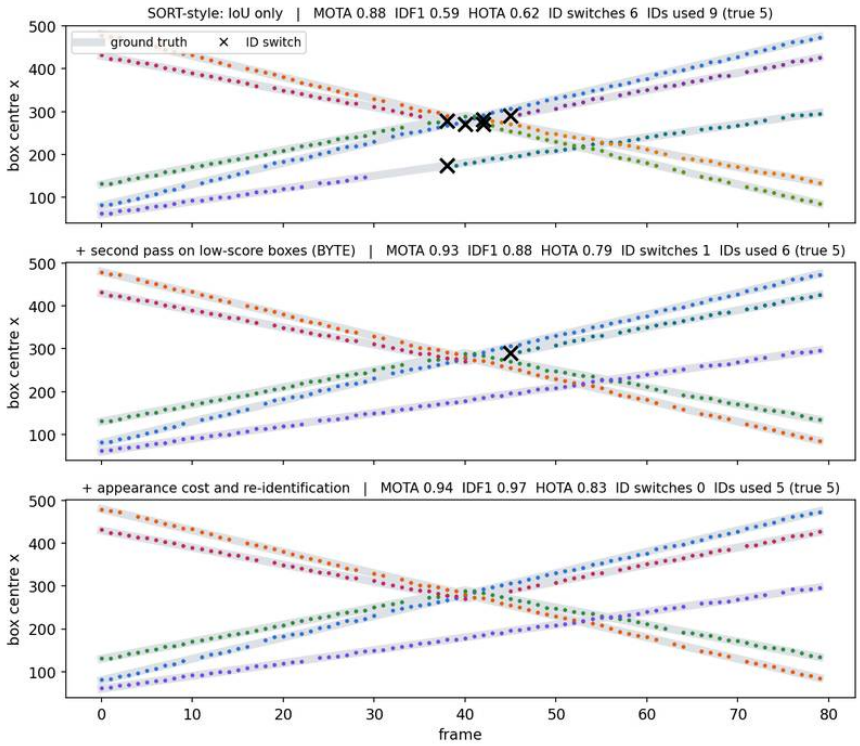


Figure 5 — The same synthetic detections tracked three ways. Colors are predicted IDs; black crosses mark ID switches. Low-score association removes fragmentation from occlusion; appearance fixes the bounce, where constant-velocity prediction points the wrong way.

The script prints:

Tracker	MOTA	IDF1	HOTA	ID switches	IDs used (true: 5)
SORT-style, IoU only	0.88	0.59	0.62	6	9
+ BYTE low-score pass	0.93	0.88	0.79	1	6
+ appearance and re-identification	0.94	0.97	0.83	0	5

Read it carefully. The IoU-only tracker loses the occluded objects because their detections fall below the score threshold, so their tracks die and come back with **new IDs** (fragmentation). The low-score pass keeps them alive, which fixes almost everything, but it cannot fix the bounce: the Kalman filter predicts the two dancers will keep going, so after the overlap one of them is not matched and later restarts with a new ID. Only appearance resolves that. Notice also how little MOTA moves (0.88 → 0.94) compared with IDF1 (0.59 → 0.97): MOTA is mostly a detection measure, a point we return to in the metrics section.

The SORT family

These ideas map directly onto the most cited online trackers:

Tracker	Key idea
SORT [32]	Kalman filter + IoU cost + Hungarian assignment; deliberately minimal and very fast. Showed that detector quality dominates tracking quality.
DeepSORT [33]	Adds a CNN appearance descriptor trained on a person Re-ID dataset, Mahalanobis gating, and a *matching cascade* that gives priority to recently seen tracks. Fewer ID switches through longer occlusions.
ByteTrack [34]	Associates **every** detection box: high-score boxes first, then the remaining tracks against **low-score** boxes using IoU only. Occluded objects often get low scores, so they are recovered instead of discarded, while low-score background boxes stay unmatched and are thrown away.
OC-SORT [35]	"Observation-centric": when a lost track is re-found, it re-runs the Kalman updates along a virtual trajectory between the last and new observations (fixing error accumulated while coasting), and adds a motion-direction consistency term to the cost. Strong on non-linear motion such as DanceTrack.
BoT-SORT [36]	ByteTrack-style association plus camera-motion compensation (global image registration), a Kalman state with w, h instead of aspect ratio, and a fused IoU/Re-ID cost.

The lesson of this family is that *carefully engineered association on top of a strong detector* is very hard to beat on pedestrian benchmarks. The 2025 survey of Adžemović [4] reaches the same conclusion: heuristic methods lead on dense scenes with mostly linear motion, while learned association does better when motion is complex.

Joint detection and embedding, and detector-based tracking

Running a separate Re-ID network on every crop is expensive. **JDE** [37] adds an embedding head to a one-stage detector so that boxes and appearance vectors come out of one forward pass. **FairMOT** [38] argued that this sharing had been "unfair" to Re-ID: anchor-based detectors produce features that are ambiguous for identity, and detection dominates training. It uses an anchor-free, CenterNet-style detector with two equal branches (detection and Re-ID) on a high-resolution feature map. **Tracktor** [39], which the original post also cites, removes explicit association for most cases: it feeds the previous frame's boxes to the detector's box-regression head, which pulls each box onto the object's new position and keeps its identity.

Tracking by query: end-to-end MOT

Plain version. Instead of hand-writing "predict, compare, match", give a transformer a memory slot per object and let it learn to update the slot each frame.

DETR-style detectors (see DLCV 03) represent objects as **queries**. MOTR [40] extends this to video: each detected object's output embedding becomes a **track query** that is passed to the next frame and keeps predicting the same object, while new *detect queries* find newborn objects. Identity is carried implicitly by the query; there is no Kalman filter and no Hungarian matching at inference. Training uses a tracklet-aware label assignment and a loss averaged over clips. The weakness was detection quality: one decoder must both detect and associate, and the two tasks conflict. MOTRv2 [41] bootstraps MOTR with proposals from a separate pretrained detector (YOLOX) used as anchors, which eases that conflict. MOTIP [42] (CVPR 2025) reformulates association as **ID prediction**: given the trajectories so far, each with an ID label drawn from a learnable ID dictionary, a decoder classifies each new detection into one of the existing IDs or "new". This is in-context, end-to-end trainable, and works with plain object-level features.

Open-vocabulary MOT

Classic MOT only tracks the classes in its training set. **OVTrack** [43] (CVPR 2023) tracks arbitrary categories by distilling knowledge from a vision-language model (CLIP) for both classification and association, and by learning robust appearance features from *hallucinated* image pairs produced with a diffusion model. SAM 3's concept prompts [25] approach the same goal from the segmentation side.

Datasets

Dataset	Domain	What it tests
MOT17 [44]	pedestrians, static and moving cameras	the standard pedestrian benchmark; the MOT16 sequences re-released as MOT17
MOT20 [45]	very crowded pedestrian scenes	heavy occlusion, small targets
DanceTrack [46]	group dancing	uniform appearance and diverse, non-linear motion; appearance cannot separate people
SportsMOT [47]	basketball, volleyball, football (240 sequences)	fast, variable-speed motion with similar but distinguishable appearance
BDD100K MOT [48]	driving video	multi-class tracking (cars, pedestrians, cyclists, ...) from a moving vehicle

The shift from MOT17 to DanceTrack and SportsMOT changed the field's priorities: on MOT17 a good detector and IoU matching go far, but on DanceTrack the motion model and association learning matter most.

Metrics: MOTA, IDF1, HOTA

Plain version. A tracker can fail in two ways: it can miss objects or invent them (detection errors), or it can mix up who is who (association errors). Different metrics weigh these two differently.

All three metrics first match predictions to ground truth in each frame at an IoU threshold. Let GT_t, FN_t, FP_t and IDSW_t be the number of ground-truth objects, misses, false positives and identity switches in frame t. An identity switch occurs when a ground-truth object is matched to a different predicted ID than at its previous match.

MOTA (CLEAR MOT [9]):

$$\mathrm{MOTA} = 1 - \frac{\sum_t (\mathrm{FN}_t + \mathrm{FP}_t + \mathrm{IDSW}_t)}{\sum_t \mathrm{GT}_t}.$$

It ranges from $-\infty$ to 1. Because there are usually far more detections than switches, FN and FP dominate it: MOTA is mostly a detection score. A switch is also counted only at the moment it happens, so a tracker that swaps two IDs once and keeps them swapped for 1,000 frames pays the same as one that swaps back after 1 frame.

IDF1 [10] computes a *global* one-to-one matching between ground-truth trajectories and predicted trajectories (again with the Hungarian algorithm) that maximizes the number of frames where they agree, then

$$\mathrm{IDF1} = \frac{2\,\mathrm{IDTP}}{2\,\mathrm{IDTP} + \mathrm{IDFP} + \mathrm{IDFN}},$$

where IDTP counts detections matched to the trajectory assigned to their ground-truth identity, and IDFP, IDFN the remaining predicted and ground-truth detections. IDF1 measures how long the tracker keeps the right identity, so it is mostly an association score — but because the matching is global, it can behave unintuitively, for example dropping when detection improves.

HOTA (Higher Order Tracking Accuracy [8]) was introduced because neither metric is balanced or decomposable. For a localization threshold α, with per-frame matches giving true positives (TP), FN and FP:

$$\mathrm{DetA}_\alpha = \frac{|\mathrm{TP}|}{|\mathrm{TP}| + |\mathrm{FN}| + |\mathrm{FP}|}, \qquad \mathrm{AssA}_\alpha = \frac{1}{|\mathrm{TP}|} \sum_{c \in \mathrm{TP}} \frac{|\mathrm{TPA}(c)|}{|\mathrm{TPA}(c)| + |\mathrm{FNA}(c)| + |\mathrm{FPA}(c)|},$$

$$\mathrm{HOTA}_\alpha = \sqrt{\mathrm{DetA}_\alpha \cdot \mathrm{AssA}_\alpha}, \qquad \mathrm{HOTA} = \frac{1}{19} \sum_{\alpha \in \{0.05,\, 0.10,\, \ldots,\, 0.95\}} \mathrm{HOTA}_\alpha.$$

For a matched pair c (ground-truth ID g, predicted ID p), $\mathrm{TPA}(c)$ is the set of all TPs in the video that also match g to p; $\mathrm{FNA}(c)$ the detections of g matched to another ID or missed; $\mathrm{FPA}(c)$ the detections of p matched to another object or to nothing. The geometric mean means a tracker must be good at *both* detection and association to score well, and DetA and AssA (and a localization accuracy, LocA) can be reported separately to diagnose failures. Averaging over α also rewards precise boxes. (Our code uses a simplified matching that maximizes IoU per frame; the official definition matches with a score that also favors pairs that have been associated before.)

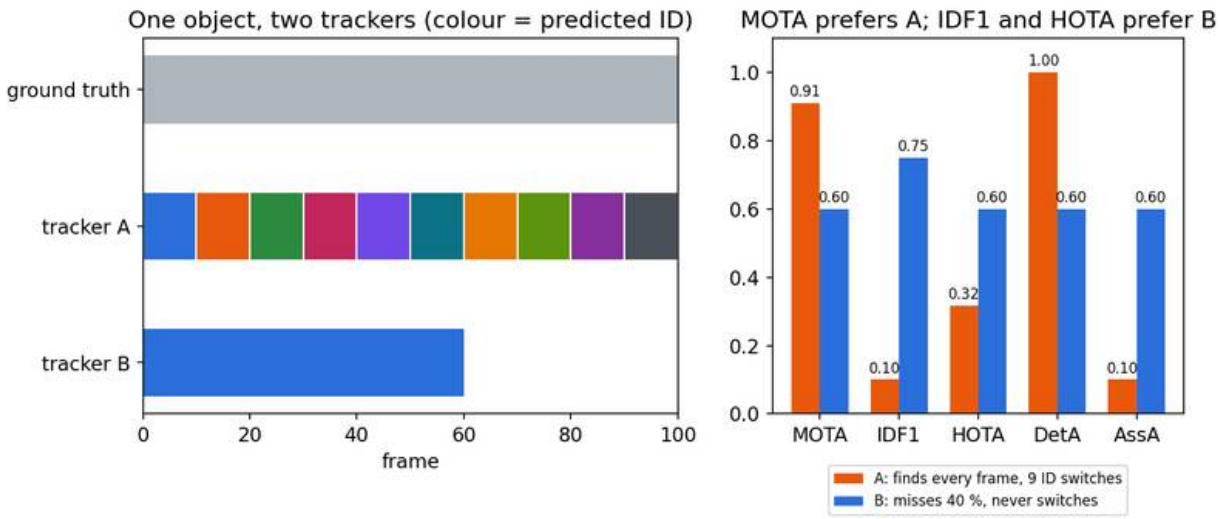


Figure 6 — Why HOTA was introduced. Tracker A detects every frame but switches identity nine times; tracker B misses 40% of frames but never switches. MOTA ranks A far ahead; IDF1 and HOTA rank B ahead, and DetA/AssA show exactly why.

Figure 6 is computed with the functions above: tracker A gets MOTA $= 1 - 9/100 = 0.91$ and AssA $= 0.10$, giving HOTA $= \sqrt{1.0 \times 0.10} \approx 0.32$; tracker B gets 0.60 on MOTA, DetA, AssA and HOTA, and IDF1 $= 2 \cdot 60/(100 + 60) = 0.75$. Today, papers on MOT17, MOT20, DanceTrack and SportsMOT report HOTA, IDF1 and MOTA together, and HOTA is usually the primary ranking metric.

Point tracking: a related trend

Plain version. Instead of following whole objects, follow individual *points* — a freckle on a cheek, a corner of a box — through the whole video, even when they are briefly hidden.

Tracking any point (TAP) asks, for a query point (x, y) in frame t_0, for its position and visibility in every other frame. It generalizes optical flow (two frames) to long, occlusion-aware trajectories, and is a natural successor to the classical keypoint tracking from IP 12. TAPIR [49] first matches the query feature independently in every frame to get a rough trajectory, then refines it with temporal convolutions. CoTracker [50] tracks many points *jointly* with a transformer, so points help one another: a point that becomes occluded can be inferred from its visible neighbors. CoTracker3 [51] simplifies the architecture and trains on real videos pseudo-labeled by existing trackers, instead of only synthetic data. Point trackers are now used as motion cues inside object trackers, video editing and robotics.

Multi-target multi-camera tracking

Plain version. Many cameras watch one place. Track everyone in each camera, then figure out that the woman leaving camera 3 is the same woman entering camera 5.

MTMCT is the ultimate goal named in [1]. A typical pipeline runs single-camera MOT to produce *tracklets*, embeds each tracklet with Re-ID features, and clusters tracklets across cameras using appearance, camera topology (which cameras connect, and travel times between them) and, with calibrated cameras, 3D positions on a ground plane. IDF1 was originally proposed for this setting [10]. The **AI City Challenge** is the main recurring benchmark. Its ninth edition (2025) [52] made Track 1 a multi-class **3D** multi-camera tracking task in warehouse-style environments, with people, humanoid robots, autonomous mobile robots and forklifts, calibrated cameras and 3D box annotations. That shift — from image boxes to calibrated 3D tracks across many views — is where MTMCT is heading.

Modern view

What the surveys say

- **Luo et al., "Multiple object tracking: A literature review" [3]** (*Artificial Intelligence*, 2021; first released 2014). The classical reference. It defines the MOT problem formally, categorizes methods by initialization (detection-based versus detection-free), processing mode (online versus offline) and output type (deterministic versus probabilistic), and analyzes the components — appearance model, motion model, interaction model, exclusion constraints, occlusion handling — and the inference methods. Read it to understand the problem independent of any network.
- **Ciaparrone et al., "Deep learning in video multi-object tracking: A survey" [2]** (*Neurocomputing*, 2020). Organizes deep MOT around four stages — detection, feature extraction/motion prediction, affinity computation and association — and reviews how deep learning entered each. Its main takeaway, from comparisons on MOTChallenge, is that detection quality and learned appearance features drive most gains.
- **Adžemović, "Deep learning-based multi-object tracking: A comprehensive survey from foundations to state-of-the-art" [4]** (arXiv, 2025). Splits tracking-by-detection into joint detection-and-embedding, heuristic-based, motion-based, affinity-learning and offline methods, and contrasts them with end-to-end trackers. It identifies 2022 (ByteTrack and MOTR) as the acceleration point and concludes that heuristic trackers lead on crowded, mostly linear scenes while learned association wins on complex motion.
- **Marvasti-Zadeh et al., "Deep learning for visual tracking: A comprehensive survey" [5]** (*IEEE T-ITS*, 2022). For SOT. Analyzes deep trackers along nine aspects (network architecture, exploitation, training, objective, output, use of correlation filters, aerial-view, long-term and online tracking) and compares them on OTB, VOT, LaSOT and aerial benchmarks.
- **Thangavel (Kugarajeevan) et al., "Transformers in single object tracking: An experimental survey" [6]** (*IEEE Access*, 2023). Classifies transformer trackers as CNN-Transformer hybrids, two-stream two-stage, and one-stream one-stage fully-transformer trackers, and evaluates their

robustness and efficiency; the one-stream design (OSTrack-style) emerges as the dominant pattern.

- **Ye et al., "Deep learning for person re-identification: A survey and outlook" [7]** (*IEEE TPAMI*, 2022). Separates closed-world Re-ID (feature learning, metric learning, ranking) from open-world Re-ID (heterogeneous data, raw images, noisy labels, unsupervised and open-set settings), proposes the AGW baseline and the mINP metric, which measures the cost of finding *all* correct matches.
- **Luiten et al., "HOTA" [8]** (*IJCV*, 2020). Not a survey, but it re-examines the evaluation of the entire field, shows the biases of MOTA and IDF1 with worked examples and user studies, and is the reason current leaderboards rank by HOTA.

State of the art, 2024–2026

- **SOT** is dominated by one-stream transformer trackers in the OSTrack/SeqTrack lineage and, increasingly, by segmentation foundation models: SAM 2 with motion-aware memory (SAMURAI) gives strong zero-shot tracking without tracking-specific training [23], [24].
- **MOT** has two coexisting camps. Heuristic tracking-by-detection (ByteTrack, OC-SORT, BoT-SORT and descendants) remains the default in practice: simple, fast and modular, with a strong detector doing most of the work. End-to-end trackers (MOTR, MOTRv2, MOTIP) lead where motion is complex and appearance is ambiguous, such as DanceTrack and SportsMOT [4], [42].
- **Open-vocabulary and promptable tracking** is moving into mainstream models: OVTrack established the task, and SAM 3 detects, segments and tracks instances of a text concept with identities [25], [43].
- **Point tracking** (TAPIR, CoTracker, CoTracker3) matured into a general motion primitive [49]–[51].
- **MTMCT** is moving to calibrated, multi-class 3D tracking (AI City Challenge 2025) [52].

Open problems

- **Long-term occlusion and re-entry.** Motion models are useless after a few seconds; appearance must carry identity, but appearance is unreliable for uniform clothing (DanceTrack, sports) and across cameras.
- **Non-linear motion and camera motion.** Constant-velocity Kalman filters remain the default and are the weak point on dance, sports and driving data; learned motion models are not yet clearly better everywhere.
- **End-to-end versus heuristic association.** End-to-end trackers are elegant and win on complex motion, but need lots of video training data and compute and still lose to simple heuristics on crowded pedestrian scenes. A unified design that is best on both is open.
- **Open-vocabulary and long-tail tracking.** Rare classes, and appearance features that generalize to classes never seen in training.
- **Evaluation.** HOTA fixed much, but benchmarks are pedestrian-heavy, and ID-level evaluation over hours of video and many cameras remains hard. **Privacy and dataset ethics** (the DukeMTMC lesson) constrain what data can be collected at all.

Key takeaways

- Tracking = detection + time: every tracker must both follow motion and keep identities, and the balance between the two defines SOT, MOT and Re-ID.
- Re-ID is retrieval with learned embeddings; report both CMC rank-k and mAP; DukeMTMC was withdrawn in 2019 and should not be used.
- SOT went from correlation filters (fast, hand-crafted) to Siamese matching (offline-trained) to one-stream transformers; SAM 2-style video segmentation now does SOT with masks, and SAM 3 extends it to many instances from a text prompt.
- Online MOT is a loop of detect → Kalman predict → cost matrix → Hungarian assignment → update/birth/death. Most practical gains come from what goes into the cost matrix and which detections are allowed to match.
- ByteTrack's low-score pass fixes occlusion fragmentation; appearance fixes non-linear motion and long gaps; end-to-end query trackers learn association directly.
- MOTA mostly measures detection, IDF1 mostly association; HOTA $= \sqrt{\text{DetA} \cdot \text{AssA}}$ (averaged over IoU thresholds) balances and decomposes both.
- The open frontier is long-term identity, non-linear motion, open-vocabulary categories and multi-camera 3D tracking.

Exercises

1. **Kalman gain intuition.** For a 1-D constant-position model with prior variance P and measurement noise R, show that the Kalman gain is $K = P/(P + R)$ and that the posterior variance is $PR/(P + R)$. What happens to K after five frames without detections if each predict step adds Q to P?

   Hint

   With $H = 1$, $S = P + R$ and $K = P/S$. The posterior variance is $(1 - K)P = PR/(P + R)$, smaller than both P and R. After five coasting steps the prior is $P + 5Q$, so K moves toward 1: the filter will trust the next detection much more, which is why a wrong match after an occlusion moves the track a lot.

2. **Hungarian by hand.** Tracks A and B have predicted IoUs with detections 1 and 2 of $\begin{bmatrix} 0.6 & 0.5 \\ 0.55 & 0.0 \end{bmatrix}$. What does greedy matching (highest IoU first) return, and what does the Hungarian algorithm on cost $1 - \text{IoU}$ return? Which total IoU is larger?

   Hint

   Greedy takes A–1 (0.6) first, leaving B–2 with IoU 0 (rejected), total 0.6. Hungarian picks A–2 and B–1, total $0.5 + 0.55 = 1.05$. Greedy leaves track B unmatched and starts a new ID for detection 2; this is one way ID switches appear.

3. **Break the tracker.** In the synthetic scene, change the bounce in `make_scene` so that objects 2 and 3 pass through each other instead (no velocity reversal). Which of the three trackers improves the most, and why?

Hint

With linear motion the constant-velocity prediction is correct through the overlap. With `seed=3`, the low-score tracker's IDF1 rises from 0.88 to about 0.96 and it uses the correct 5 IDs, almost matching the appearance tracker (about 0.96 as well); the IoU-only tracker still fragments occluded objects. Appearance helps most exactly when motion is non-linear, which is the design rationale of DanceTrack.

4. **Metric arithmetic.** A video has one object for 200 frames. A tracker detects it in all frames but uses ID 1 for frames 1–100 and ID 2 for frames 101–200. Compute MOTA, IDF1, DetA, AssA and HOTA at a single threshold, assuming perfect boxes.

Hint

MOTA $= 1 - 1/200 = 0.995$. The best global ID match covers 100 frames: IDF1 $= 2 \cdot 100/(200 + 200) = 0.5$. DetA $= 1$. For each TP, $|\text{TPA}| = 100$, $|\text{FNA}| = 100$, $|\text{FPA}| = 0$, so AssA $= 0.5$ and HOTA $= \sqrt{0.5} \approx 0.71$. One swap barely moves MOTA but halves AssA.

5. **Re-ID metrics.** In the Re-ID snippet, change the noise level from 1.6 to 2.2 and then to 1.0. How do rank-1 and mAP change, and which one is more sensitive? Explain using the definition of AP.

Hint

mAP falls faster than rank-1 as noise grows, because rank-1 only needs the single easiest true match at the top, while AP penalizes every true match that is pushed down the list. With 4 true matches per query, the hardest one usually decides how much AP is lost.

6. **Design question.** You must track forklifts and workers across 12 calibrated cameras in a warehouse. Sketch a pipeline using components from this tutorial, and say which metric you would report and why.

Hint

Per-camera detection plus a ByteTrack/BoT-SORT-style tracker; project boxes to the ground plane using calibration; cross-camera association by 3D position and time first, Re-ID appearance second (workers may wear identical vests); handle the two classes separately. Report HOTA (balanced, decomposable into DetA/AssA) and IDF1 (long-term identity across cameras), as in the AI City Challenge setting.

References

1. Shyandram, “物件追蹤 Object Tracking 簡介” (Introduction to object tracking), Medium post (in Traditional Chinese), 2023; updated 2026. link
2. G. Ciaparrone, F. Luque Sánchez, S. Tabik, L. Troiano, R. Tagliaferri and F. Herrera, “Deep learning in video multi-object tracking: A survey,” *Neurocomputing*, 2020. arXiv:1907.12740
3. W. Luo, J. Xing, A. Milan, X. Zhang, W. Liu and T.-K. Kim, “Multiple object tracking: A literature review,” *Artificial Intelligence*, vol. 293, 2021 (arXiv 2014). arXiv:1409.7618
4. M. Adžemović, “Deep learning-based multi-object tracking: A comprehensive survey from foundations to state-of-the-art,” arXiv:2506.13457, 2025. arXiv
5. S. M. Marvasti-Zadeh, L. Cheng, H. Ghanei-Yakhdan and S. Kasaei, “Deep learning for visual tracking: A comprehensive survey,” *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, 2022. arXiv:1912.00535
6. J. Thangavel (Kugarajeevan), T. Kokul, A. Ramanan and S. Fernando, “Transformers in single object tracking: An experimental survey,” *IEEE Access*, vol. 11, 2023. arXiv:2302.11867
7. M. Ye, J. Shen, G. Lin, T. Xiang, L. Shao and S. C. H. Hoi, “Deep learning for person re-identification: A survey and outlook,” *IEEE TPAMI*, vol. 44, 2022. arXiv:2001.04193
8. J. Luiten, A. Ošep, P. Dendorfer, P. Torr, A. Geiger, L. Leal-Taixé and B. Leibe, “HOTA: A higher order metric for evaluating multi-object tracking,” *International Journal of Computer Vision*, 2020. arXiv:2009.07736
9. K. Bernardin and R. Stiefelhagen, “Evaluating multiple object tracking performance: The CLEAR MOT metrics,” *EURASIP Journal on Image and Video Processing*, 2008. doi
10. E. Ristani, F. Solera, R. S. Zou, R. Cucchiara and C. Tomasi, “Performance measures and a data set for multi-target, multi-camera tracking,” *ECCV Workshop on Benchmarking Multi-Target Tracking*, 2016. arXiv:1609.01775
11. K. Peng, A. Mathur and A. Narayanan, “Mitigating dataset harms requires stewardship: Lessons from 1000 papers,” arXiv:2108.02922, 2021. arXiv
12. L. Zheng, L. Shen, L. Tian, S. Wang, J. Wang and Q. Tian, “Scalable person re-identification: A benchmark,” *ICCV*, 2015. doi
13. L. Wei, S. Zhang, W. Gao and Q. Tian, “Person transfer GAN to bridge domain gap for person re-identification,” *CVPR*, 2018. arXiv:1711.08565
14. H. Luo, Y. Gu, X. Liao, S. Lai and W. Jiang, “Bag of tricks and a strong baseline for deep person re-identification,” *CVPR Workshops*, 2019. arXiv:1903.07071
15. M. Wieczorek, B. Rychalska and J. Dąbrowski, “On the unreasonable effectiveness of centroids in image retrieval,” arXiv:2104.13643, 2021. arXiv
16. S. Li, L. Sun and Q. Li, “CLIP-ReID: Exploiting vision-language model for image re-identification without concrete text labels,” *AAAI*, 2023. arXiv:2211.13977
17. D. S. Bolme, J. R. Beveridge, B. A. Draper and Y. M. Lui, “Visual object tracking using adaptive correlation filters,” *CVPR*, 2010. doi
18. J. F. Henriques, R. Caseiro, P. Martins and J. Batista, “High-speed tracking with kernelized correlation filters,” *IEEE TPAMI*, vol. 37, 2015. arXiv:1404.7584
19. L. Bertinetto, J. Valmadre, J. F. Henriques, A. Vedaldi and P. H. S. Torr, “Fully-convolutional Siamese networks for object tracking,” arXiv:1606.09549, 2016 (ECCV 2016 workshops). arXiv
20. B. Li, J. Yan, W. Wu, Z. Zhu and X. Hu, “High performance visual tracking with Siamese region

proposal network," *CVPR*, 2018. doi
21. B. Ye, H. Chang, B. Ma, S. Shan and X. Chen, "Joint feature learning and relation modeling for tracking: A one-stream framework" (OSTrack), *ECCV*, 2022. arXiv:2203.11991
22. X. Chen, H. Peng, D. Wang, H. Lu and H. Hu, "SeqTrack: Sequence to sequence learning for visual object tracking," *CVPR*, 2023 (the arXiv entry was later extended as SeqTrackv2). arXiv:2304.14394
23. N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, C. Ryali, T. Ma et al., "SAM 2: Segment anything in images and videos," *ICLR*, 2025. arXiv:2408.00714
24. C.-Y. Yang, H.-W. Huang, W. Chai, Z. Jiang and J.-N. Hwang, "SAMURAI: Adapting segment anything model for zero-shot visual tracking with motion-aware memory," arXiv:2411.11922, 2024. arXiv
25. N. Carion, L. Gustafson, Y.-T. Hu et al., "SAM 3: Segment anything with concepts," *ICLR*, 2026. arXiv:2511.16719
26. Y. Wu, J. Lim and M.-H. Yang, "Object tracking benchmark," *IEEE TPAMI*, vol. 37, 2015. doi
27. H. Fan, L. Lin, F. Yang, P. Chu, G. Deng, S. Yu et al., "LaSOT: A high-quality benchmark for large-scale single object tracking," *CVPR*, 2019. arXiv:1809.07845
28. L. Huang, X. Zhao and K. Huang, "GOT-10k: A large high-diversity benchmark for generic object tracking in the wild," *IEEE TPAMI*, vol. 43, 2021. arXiv:1810.11981
29. M. Müller, A. Bibi, S. Giancola, S. Al-Subaihi and B. Ghanem, "TrackingNet: A large-scale dataset and benchmark for object tracking in the wild," arXiv:1803.10794, 2018 (ECCV 2018). arXiv
30. R. E. Kalman, "A new approach to linear filtering and prediction problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35–45, 1960. doi
31. H. W. Kuhn, "The Hungarian method for the assignment problem," *Naval Research Logistics Quarterly*, vol. 2, pp. 83–97, 1955. doi
32. A. Bewley, Z. Ge, L. Ott, F. Ramos and B. Upcroft, "Simple online and realtime tracking," *ICIP*, 2016. arXiv:1602.00763
33. N. Wojke, A. Bewley and D. Paulus, "Simple online and realtime tracking with a deep association metric," *ICIP*, 2017. doi
34. Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu and X. Wang, "ByteTrack: Multi-object tracking by associating every detection box," *ECCV*, 2022. arXiv:2110.06864
35. J. Cao, J. Pang, X. Weng, R. Khirodkar and K. Kitani, "Observation-centric SORT: Rethinking SORT for robust multi-object tracking," *CVPR*, 2023. arXiv:2203.14360
36. N. Aharon, R. Orfaig and B.-Z. Bobrovsky, "BoT-SORT: Robust associations multi-pedestrian tracking," arXiv:2206.14651, 2022. arXiv
37. Z. Wang, L. Zheng, Y. Liu, Y. Li and S. Wang, "Towards real-time multi-object tracking" (JDE), *ECCV*, 2020. arXiv:1909.12605
38. Y. Zhang, C. Wang, X. Wang, W. Zeng and W. Liu, "FairMOT: On the fairness of detection and re-identification in multiple object tracking," *IJCV*, 2021. arXiv:2004.01888
39. P. Bergmann, T. Meinhardt and L. Leal-Taixé, "Tracking without bells and whistles" (Tracktor), *ICCV*, 2019. arXiv:1903.05625
40. F. Zeng, B. Dong, Y. Zhang, T. Wang, X. Zhang and Y. Wei, "MOTR: End-to-end multiple-object tracking with transformer," *ECCV*, 2022. arXiv:2105.03247
41. Y. Zhang, T. Wang and X. Zhang, "MOTRv2: Bootstrapping end-to-end multi-object tracking

by pretrained object detectors," *CVPR*, 2023. arXiv:2211.09791
42. R. Gao, J. Qi and L. Wang, "Multiple object tracking as ID prediction" (MOTIP), *CVPR*, 2025. arXiv:2403.16848
43. S. Li, T. Fischer, L. Ke, H. Ding, M. Danelljan and F. Yu, "OVTrack: Open-vocabulary multiple object tracking," *CVPR*, 2023. arXiv:2304.08408
44. A. Milan, L. Leal-Taixé, I. Reid, S. Roth and K. Schindler, "MOT16: A benchmark for multi-object tracking," arXiv:1603.00831, 2016 (MOT16/MOT17). arXiv
45. P. Dendorfer, H. Rezatofighi, A. Milan, J. Shi, D. Cremers, I. Reid, S. Roth, K. Schindler and L. Leal-Taixé, "MOT20: A benchmark for multi object tracking in crowded scenes," arXiv:2003.09003, 2020. arXiv
46. P. Sun, J. Cao, Y. Jiang, Z. Yuan, S. Bai, K. Kitani and P. Luo, "DanceTrack: Multi-object tracking in uniform appearance and diverse motion," *CVPR*, 2022. arXiv:2111.14690
47. Y. Cui, C. Zeng, X. Zhao, Y. Yang, G. Wu and L. Wang, "SportsMOT: A large multi-object tracking dataset in multiple sports scenes," *ICCV*, 2023. arXiv:2304.05170
48. F. Yu, H. Chen, X. Wang, W. Xian, Y. Chen, F. Liu, V. Madhavan and T. Darrell, "BDD100K: A diverse driving dataset for heterogeneous multitask learning," *CVPR*, 2020. arXiv:1805.04687
49. C. Doersch, Y. Yang, M. Vecerik, D. Gokay, A. Gupta, Y. Aytar et al., "TAPIR: Tracking any point with per-frame initialization and temporal refinement," *ICCV*, 2023. arXiv:2306.08637
50. N. Karaev, I. Rocco, B. Graham, N. Neverova, A. Vedaldi and C. Rupprecht, "CoTracker: It is better to track together," *ECCV*, 2024. arXiv:2307.07635
51. N. Karaev, I. Makarov, J. Wang, N. Neverova, A. Vedaldi and C. Rupprecht, "CoTracker3: Simpler and better point tracking by pseudo-labelling real videos," arXiv:2410.11831, 2024. arXiv
52. Z. Tang, S. Wang, D. C. Anastasiu et al., "The 9th AI City Challenge," *ICCV Workshops*, 2025. arXiv:2508.13564
53. VOT Challenge organizers, "The Visual Object Tracking (VOT) challenge series," website, 2013–present. votchallenge.net

DLCV 07 • World Models: Video, Latent Dynamics and Planning

What a world model is, how model-based reinforcement learning uses one (World Models, the Dreamer line, MuZero), why video generators are proposed as world simulators, how latent dynamics and JEPA-style predictive representations work, and how to evaluate all of them, with complete CPU experiments.

Online edition: https://blog.shyandram.dev/en/learn/deep-learning/dl-07-world-models/

What you'll learn

- What a world model is, and the difference between an observation, a hidden state, a belief and an action.
- How model-based reinforcement learning uses a learned model in three different ways: planning online, learning a policy in imagination, and tree search with a value-equivalent model, with World Models, PlaNet, the Dreamer line, TD-MPC2 and MuZero as examples.
- Why video prediction and generation are proposed as "world simulators", why squared-error predictors produce blur, and what interactive generators such as Genie can and cannot show.
- How recurrent state-space models and JEPA-style predictive representations learn latent dynamics without (or with) reconstructing pixels.
- How to fit a small dynamics model, measure multi-step error, plan with it, and detect when a planner exploits model errors, all on a CPU.
- How to evaluate a world model, and which problems remain open.

The big picture

Tracking asks where an object went. A world model asks what could happen next, and especially what would happen **if an agent took a particular action**. A robot approaching a cup needs more than an object label: it needs to predict how its gripper and the cup will move, which outcomes are uncertain, and which action is likely to reach its goal.

> **Explain like I'm five**
>
> Before pushing a toy car, you imagine where it will go. You can imagine a gentle push and a strong push, pick the better one, push, and then look again to see what really happened. A world model is a learned version of that imagination. A planner is the part that tries out pushes in imagination and picks one. Looking again after each push is what keeps small imagination mistakes from piling up.

The term "world model" covers several research traditions. Ding et al. organize the literature around two functions: building internal representations that capture how the world works, and

predicting future states to simulate and guide decisions [1]. This chapter uses an operational definition: **a world model is a learned model that represents enough of an environment to predict relevant changes under specified conditions, usually including the agent's own actions.** Its usefulness depends on the task. A model for rendering a scene, a model for learning representations, and a model for controlling a robot may need very different information.

Observations, states, beliefs and actions

Plain version. A camera image is not the world; it is a partial view of it. A world model has to keep track of what it cannot currently see, like the speed of a ball or an object behind an occluder.

Let s_t denote the underlying environment state at time t, o_t the observation and a_t the executed action:

$$s_{t+1} \sim p(s_{t+1} \mid s_t, a_t), \qquad o_t \sim p(o_t \mid s_t).$$

The first distribution is the **transition** (dynamics) model; the second is the **observation** model. The state might contain object positions, velocities and contact conditions. An image shows only a projection, with occlusions and sensor noise; Szeliski covers the geometry and motion side of this observation process [3]. Two identical images can have different futures: a ball at the same position might be moving left or right.

A learned model therefore builds a representation from history:

$$z_t = E_\theta(o_{0:t}, a_{0:t-1}), \qquad p_\theta(z_{t+1} \mid z_t, a_t),$$

where E_θ is an encoder with parameters θ, $o_{0:t}$ is the sequence of observations up to t, and $p_\theta(z_{t+1} \mid z_t, a_t)$ is a learned latent transition. An encoder does not automatically make z_t a sufficient state. If it discards velocity, hidden objects or a relevant material property, the transition model cannot recover that information. Recurrent memory or a short video context helps, but must be tested under the actual occlusions and delays.

When the state is only partially observed, the principled object is a **belief**, a probability distribution over possible states given the history. Bishop's treatment of sequential latent-variable models (hidden Markov models and linear dynamical systems) gives the probabilistic background [2]. A compact latent vector can approximate useful aspects of this distribution; it need not map one-to-one onto human-readable physical variables.

A prediction factorization

For a proposed action sequence, the probability of future observations factorizes as

$$p_\theta(o_{t+1:t+H} \mid o_{\le t}, a_{0:t+H-1}) = \int p_\theta(z_t \mid o_{\le t}, a_{<t}) \prod_{k=t}^{t+H-1} p_\theta(z_{k+1} \mid z_k, a_k)\, p_\theta(o_{k+1} \mid z_{k+1})\, dz_{t:t+H},$$

where H is the prediction horizon. The three factors are **inference** (what is the current latent state given the past?), **transition** (how does it evolve under the actions?) and **rendering** (what would

the camera see?). A planner may work directly with z and skip the pixel decoder if its goal and cost are defined in latent space. Much of the design space in this chapter consists of choosing which of these factors to learn, and how.

Model-based reinforcement learning

Plain version. Reinforcement learning (RL) learns to act by trial and error. If the agent also learns how the world responds to its actions, it can practice in its head, which needs far fewer real trials.

The setting

An RL agent interacts with an environment over discrete steps. At each step it observes, chooses a_t from a **policy** $\pi(a_t \mid s_t)$, receives a reward r_t, and the environment moves on. The goal is to maximize the expected **return**

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k},$$

where $\gamma \in [0, 1)$ is a discount factor that weights near rewards more than distant ones. The **value** $V^\pi(s) = \mathbb{E}_\pi[G_t \mid s_t = s]$ is the expected return from state s when following π.

Model-free methods learn a policy or value directly from real transitions. **Model-based** methods also learn a model of transitions and rewards and use it to reduce real interaction. The idea is old: Sutton's Dyna architecture (1991) already combined learning from real experience with planning on experience simulated by a learned model, using the same value updates for both [4]. Moerland et al. survey the field and organize it along two axes: how the dynamics model is learned (handling stochasticity, uncertainty, partial observability, temporal abstraction) and how planning and learning are integrated (where to start planning, how much budget to give it, how to plan, and how to feed the results back into acting and learning) [5].

Three ways to use a model

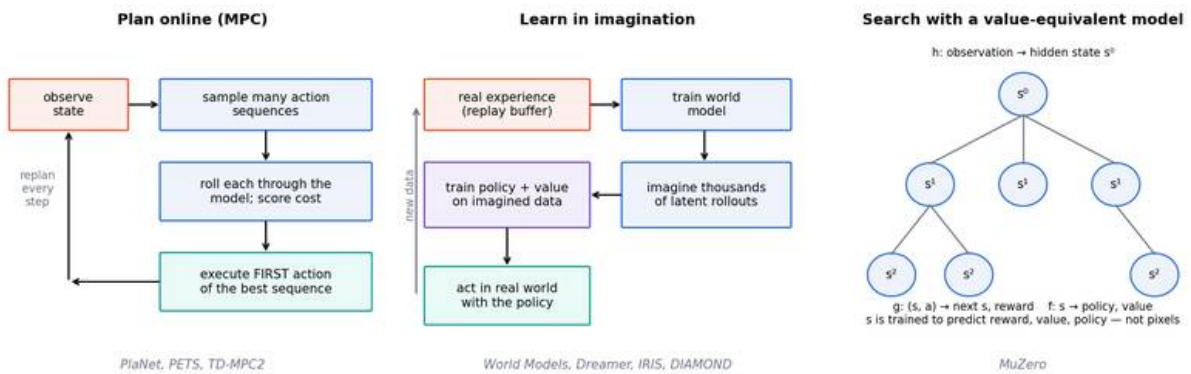


Figure 1 — Three ways to use a learned model. The same model can serve more than one role; the difference is where the decision is computed.

1. **Plan online (model-predictive control).** At every real step, search for a good action sequence by simulating candidates in the model, execute only the first action, observe, and repeat. PlaNet [7], PETS [29] and TD-MPC2 [11] work this way.
2. **Learn in imagination.** Train a policy (and usually a value function) on trajectories generated by the model, then act with that policy without searching at run time. World Models [6], the Dreamer line [8], [9], [10], IRIS [12] and DIAMOND [13] work this way.
3. **Search with a value-equivalent model.** Learn a model that predicts only what planning needs (rewards, values, policy), not observations, and run tree search in its latent space. MuZero [14] is the canonical example.

All three share one danger: the model is wrong somewhere, and an optimizer is very good at finding exactly those places. We return to this in the section on uncertainty and exploitation.

Why errors compound

Suppose the learned transition $\hat{f}$ has one-step error at most ε, and the true dynamics f are Lipschitz with constant L (a small change in state changes the next state by at most L times as much). Starting from the same state and applying the same actions, the gap e_k between predicted and true states after k steps obeys $e_{k+1} \le L e_k + \varepsilon$, so

$$e_H \le \varepsilon \sum_{k=0}^{H-1} L^k = \varepsilon \, \frac{L^H - 1}{L - 1} \quad (L \neq 1).$$

For $L > 1$ the bound grows exponentially with the horizon H; for $L < 1$ it saturates at $\varepsilon/(1 - L)$. This is a worst-case bound, but it explains two common design choices: replanning often (MPC) and using short model rollouts. Janner et al. made the second idea concrete: their MBPO algorithm branches short model rollouts from real states rather than imagining long trajectories from the start, and they argue both theoretically and empirically that this keeps the benefit of model data while limiting its bias [30].

World Models: compress, predict, control

Plain version. First learn to summarize each frame in a few numbers, then learn to predict the next summary, then train a tiny controller on top. The controller can even be trained entirely inside the model's "dream".

Ha and Schmidhuber's *World Models* (published at NeurIPS 2018 as "Recurrent World Models Facilitate Policy Evolution", with an interactive online version) made the idea popular in deep learning [6]. Their agent has three parts:

- **V (vision):** a variational autoencoder that compresses each 64×64 frame into a small latent vector z_t.

- **M (memory):** a recurrent network with a mixture-density output that predicts a distribution over z_{t+1} given z_t, the action and its hidden state h_t.
- **C (controller):** a deliberately tiny linear policy $a_t = W_c[z_t; h_t] + b_c$, trained with an evolution strategy rather than backpropagation.

In a car-racing task, the controller using V and M features solved the task; in a VizDoom task, the controller was trained entirely inside the environment simulated by M and then transferred to the real game [6]. The design separates *perception*, *prediction* and *decision*, and shows that a controller can be very small when the representation already carries the relevant dynamics. It also showed the central risk: a controller trained inside an imperfect dream can learn to exploit the dream's mistakes.

Latent dynamics: from PlaNet to the Dreamer line

Plain version. Instead of predicting future images, predict a compact hidden state and only decode images when training. A hidden state that mixes a deterministic memory with a random part can represent both what the agent remembers and what it is unsure about.

Recurrent state-space models

Hafner et al.'s PlaNet learned environment dynamics from images and chose actions by fast online planning in latent space; its key modeling idea is a latent dynamics model with both deterministic and stochastic components, the **recurrent state-space model** (RSSM) [7]. The RSSM keeps a deterministic memory h_t and a stochastic state z_t:

$$h_t = f_\theta(h_{t-1}, z_{t-1}, a_{t-1}), \qquad z_t \sim p_\theta(z_t \mid h_t) \ \text{(prior)}, \qquad z_t \sim q_\theta(z_t \mid h_t, o_t) \ \text{(posterior)}.$$

Here f_θ is a recurrent cell, the **prior** predicts z_t without seeing the current image (this is what is used when imagining), and the **posterior** also sees the image o_t (used during training and when filtering real observations). Heads read the model state (h_t, z_t) to predict the image $\hat{o}_t$, the reward $\hat{r}_t$ and a continuation flag (whether the episode goes on).

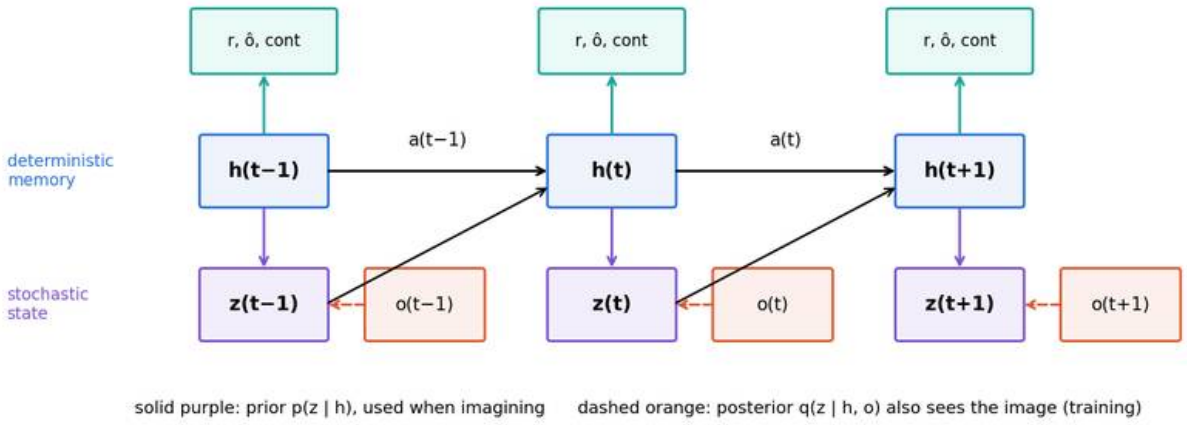


Figure 2 — An unrolled RSSM. Imagination uses only the prior path (solid); training additionally conditions z on the real image (dashed).

Training maximizes a variational lower bound on the likelihood of observed sequences. A simplified per-step loss is

$$\mathcal{L}_t = -\log p_\theta(o_t \mid h_t, z_t) - \log p_\theta(r_t \mid h_t, z_t) + \beta\, D_{\mathrm{KL}}(q_\theta(z_t \mid h_t, o_t) \,\|\, p_\theta(z_t \mid h_t)),$$

with $\beta > 0$ weighting the KL term. The reconstruction terms make the state informative; the KL term teaches the prior to predict what the posterior will see, which is exactly the skill needed for imagination. Real implementations add loss balancing and other stabilizers, so treat this as a teaching outline, not the exact loss of any one paper.

Dreamer: learning behavior in imagination

Dreamer (2020) kept the RSSM but replaced online planning with an actor–critic trained entirely on imagined latent trajectories. Its key move was to propagate analytic gradients of learned state values back through trajectories imagined in the compact latent space [8]. Starting from latent states inferred from real data, the agent imagines H steps with its actor, scores them with a learned critic, and trains the critic on bootstrapped targets such as the λ-return

$$V_t^\lambda = \hat{r}_t + \gamma\, \hat{c}_t \big[(1-\lambda)\, v_\psi(s_{t+1}) + \lambda\, V_{t+1}^\lambda \big],$$

where $\hat{r}_t$ and $\hat{c}_t$ are the predicted reward and continuation, v_ψ is the critic, and $\lambda \in [0,1]$ trades off one-step bootstrapping ($\lambda = 0$) against longer imagined returns ($\lambda = 1$). The actor is updated to increase these values.

The line continued:

- **DreamerV2** (2021) switched to *discrete* (categorical) latent variables and became the first agent to reach human-level performance on the 55-game Atari benchmark by learning behaviors inside a separately trained world model [9].

- **DreamerV3** (Nature, 2025) used robustness techniques based on normalization, balancing and transformations so that a single configuration outperformed specialized methods across more than 150 tasks, and was the first algorithm to collect diamonds in Minecraft from scratch without human data or curricula [10].

Two related lines show that the RSSM is one choice among several. **TD-MPC2** plans in the latent space of an *implicit*, decoder-free world model (no image reconstruction), reported strong results across 104 online RL tasks with a single set of hyperparameters, and trained a single 317M-parameter agent on 80 tasks [11]. **IRIS** replaced the RSSM with a discrete autoencoder plus an autoregressive Transformer over image tokens and learned from the equivalent of two hours of Atari gameplay [12]. **DIAMOND** went further toward pixels: it trains the agent inside a *diffusion* world model, arguing that discrete latents can drop visual details that matter for control, and also demonstrated the model as an interactive neural game engine for a 3D game [13].

MuZero: a model that predicts only what planning needs

Plain version. You do not have to predict what the screen will look like to play a good game. You only need to predict how good positions are, which moves look promising, and what rewards you will get. MuZero learns exactly that, and then searches.

AlphaZero-style tree search needs a simulator of the game rules. MuZero removes that requirement by learning a model that, applied repeatedly, predicts the quantities most directly relevant to planning: reward, action-selection policy and value [14]. It has three functions:

$$s^0 = h_\theta(o_1, \dots, o_t), \qquad (r^k, s^k) = g_\theta(s^{k-1}, a^k), \qquad (\mathbf{p}^k, v^k) = f_\theta(s^k),$$

where h_θ (representation) encodes the observation history into a hidden state s^0, g_θ (dynamics) maps a hidden state and a hypothetical action a^k to a predicted reward r^k and next hidden state, and f_θ (prediction) outputs a policy vector $\mathbf{p}^k$ and a value v^k. A Monte Carlo tree search unrolls g_θ to choose actions. Training unrolls the model for K steps along real trajectories and minimizes

$$\mathcal{L} = \sum_{k=0}^{K} \Big[\ell^r(u_{t+k}, r^k) + \ell^v(z_{t+k}, v^k) + \ell^p(\boldsymbol{\pi}_{t+k}, \mathbf{p}^k) \Big] + c\|\theta\|^2,$$

where u are observed rewards, z are value targets (bootstrapped returns), $\boldsymbol{\pi}$ are the visit-count distributions produced by the search, ℓ^r, ℓ^v, ℓ^p are per-quantity losses and c weights an L2 penalty.

The hidden state s^k is never asked to reconstruct an observation. It only has to be **value-equivalent**: planning with it must give the same rewards, values and policies as planning in the real environment would. With this design MuZero reached a new state of the art on 57 Atari games and, without being given the rules, matched AlphaZero's superhuman performance in Go, chess and shogi [14]. The lesson for vision is that a world model's target should be chosen by its use; predicting every pixel is one option, not a requirement.

Video prediction and generation as world simulators

Plain version. If a model can generate realistic future video, perhaps it has learned how the world works. Sometimes it has; often it has only learned what videos usually look like. Testing which is the case is the hard part.

Action-conditioned video prediction

Finn et al. trained an **action-conditioned video prediction** model on a dataset of 59,000 robot pushing interactions; by predicting pixel motion rather than raw pixels, the model generalized partly to unseen objects, and conditioning on different future actions produced a "visual imagination" of different futures [15]. This is a world model in the most literal sense: image in, action in, image out.

Why squared-error predictors blur

Real futures are uncertain. Suppose that from the same past, a ball goes left with probability $\frac{1}{2}$ and right with probability $\frac{1}{2}$. For squared error, the best single prediction is the **conditional mean**:

$$\hat{o}^\star = \arg\min_{\hat{o}} \ \mathbb{E}[\|o_{t+1} - \hat{o}\|^2 \mid o_{\le t}] = \mathbb{E}[o_{t+1} \mid o_{\le t}],$$

which here is two half-bright balls, an image that never happens. A generative model that **samples** one of the two futures produces a realistic image but, on average, a *higher* squared error.

```
import numpy as np

x = np.arange(64)
def frame(center):                      # a 1-D "image" with one bright ball
    return np.exp(-0.5 * ((x - center) / 2.0) ** 2)

# Same past, two equally likely futures: the ball bounces left or right.
futures = np.stack([frame(20), frame(44)])
mean_pred = futures.mean(axis=0)        # what an MSE-trained predictor converges to
sample_pred = futures[0]                # what a good generative sampler might output

def expected_mse(pred):                 # average over the two possible futures
    return np.mean([(pred - f) ** 2 for f in futures])

print(f"peak brightness: true future 1.00, mean prediction {mean_pred.max():.2f}")
print(f"expected MSE of mean prediction:   {expected_mse(mean_pred):.4f}")
print(f"expected MSE of a sharp sample:    {expected_mse(sample_pred):.4f}")
```

Output:

```
peak brightness: true future 1.00, mean prediction 0.50
expected MSE of mean prediction:   0.0277
expected MSE of a sharp sample:    0.0554
```

The sharp sample has exactly twice the expected error of the blurry mean. This is why pixel MSE rewards blur, why stochastic video models were introduced, and why evaluating a world model with a single squared-error number can prefer the wrong model.

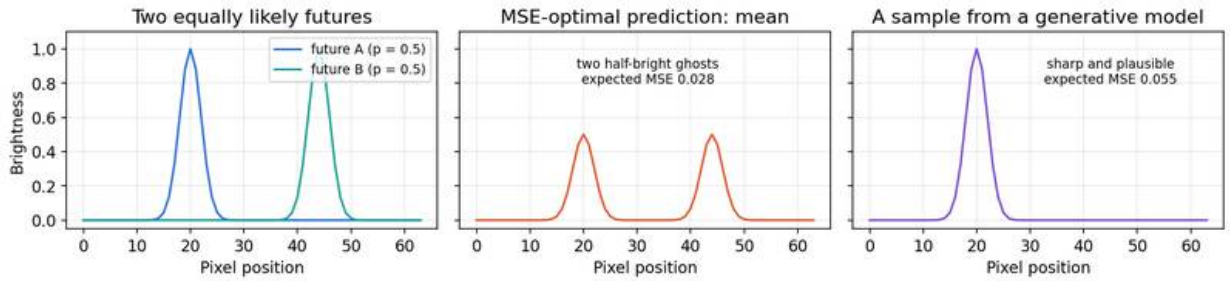


Figure 3 — Output of the code above. The MSE-optimal prediction averages incompatible futures; a sample is plausible but scores worse on MSE.

Babaeizadeh et al.'s SV2P addressed this by predicting a different possible future for each sample of a latent variable, and found substantially better predictions than the same model without stochasticity [16]. Modern video generators use the same principle at a much larger scale, with autoregressive token models or diffusion and flow models (see ML 07 and ML 08).

Large generative world models

- **GAIA-1** casts driving world modeling as next-token prediction over discrete tokens of video, text and action, producing realistic driving scenarios with control over ego-vehicle behavior and scene features [17].
- **UniSim** learns a "universal simulator" of real-world interaction by combining datasets that are rich along different axes (objects in images, dense actions in robot data, motion in navigation data); it simulates both high-level instructions and low-level controls, and policies trained purely in it were deployed zero-shot on real robots [18].
- **Genie** is a generative interactive environment trained without action labels on unlabelled Internet video. It consists of a video tokenizer, an autoregressive dynamics model and a *latent action model* that infers actions from changes between frames, so users can act frame by frame in generated worlds even though no action labels were used in training [19]. Such latent actions need not correspond to calibrated physical actuators.
- **Genie 3**, announced by Google DeepMind in August 2025, generates worlds from text that can be navigated in real time at 24 frames per second and 720p, keeping consistency for a few minutes. The announcement lists limitations: a constrained action space, difficulty modeling several independent agents, inaccurate real-world locations, and interaction limited to minutes rather than hours [20].

- **Cosmos** is NVIDIA's open platform of "world foundation models": pretrained video models, video tokenizers, a data-curation pipeline and post-training examples meant to be fine-tuned into task-specific world models for robots and vehicles [21].

Does realistic video mean physical understanding?

Two careful studies say: not necessarily. Motamed et al.'s **Physics-IQ** benchmark tests fluid dynamics, optics, solid mechanics, magnetism and thermodynamics; across current video models (including Sora, Runway, Pika, Lumiere, Stable Video Diffusion and VideoPoet), physical understanding was severely limited and unrelated to visual realism [22]. Kang et al. trained diffusion video models on a 2D simulator governed by classical mechanics and found perfect in-distribution generalization, measurable scaling for combinatorial generalization, but failure out of distribution; the models behaved as if retrieving the closest training example rather than abstracting a rule [23].

For robot control, also ask what an "action" means: force, joint target, gripper command, camera movement or an inferred token. A wrong unit, timing offset or control frequency changes the conditional problem even if the video looks convincing.

Predictive representations: the JEPA family

Plain version. Predicting every pixel wastes effort on details that do not matter, like leaf texture. Instead, predict an abstract *description* of the missing or future part. If the description is good, the prediction is useful for planning.

A **joint-embedding predictive architecture** (JEPA) encodes a visible *context* x and a hidden *target* y with encoders, and trains a predictor to map the context embedding to the target embedding:

$$\mathcal{L} = \| P_\phi(E_\theta(x),\, m) - \mathrm{sg}[\bar{E}(y)] \|^2,$$

where E_θ is the context encoder, $\bar{E}$ the target encoder, P_ϕ a predictor conditioned on information m about where the target is (for example its position in the image or time), and sg (stop-gradient) blocks gradients into the target branch. Nothing is reconstructed in pixel space. The danger is **collapse**: if both encoders output a constant, the loss is zero. Asymmetric designs (a predictor on one side only, stop-gradient, and a target encoder that is not trained by the same gradient) are used to prevent it; ML 05 discusses these self-supervised mechanisms in more detail.

- **I-JEPA** predicts the representations of several large target blocks of an image from one informative context block, and found the masking strategy (large, semantic targets; spatially distributed context) crucial for learning semantic features without hand-crafted augmentations [24].
- **V-JEPA** applies feature prediction as a stand-alone objective to video, trained on 2 million videos without pretrained image encoders, text, negatives or reconstruction, and produced frozen features that work for both motion- and appearance-based tasks [25].
- **V-JEPA 2** pretrains an action-free JEPA on over one million hours of Internet video, then post-trains a latent **action-conditioned** world model, V-JEPA 2-AC, on less than 62 hours of unla-

beled robot video; planning toward image goals with it enabled zero-shot pick-and-place on Franka arms in two labs without data from those environments, task-specific training or reward [26]. The split between passive video and action data makes the observation/action distinction of this chapter concrete.

- **V-JEPA 2.1** adds a dense predictive loss in which both visible and masked tokens contribute, deep self-supervision across intermediate layers, and multi-modal tokenizers for images and videos, aiming at spatially structured, temporally consistent dense features [27].
- **DINO-WM** shows the same idea with an off-the-shelf encoder: it predicts future DINOv2 patch features from offline trajectories and plans by optimizing action sequences toward the features of a goal image, without reconstructing pixels or learning a reward model [28].

Goals in feature space need care. A distance $\|z - z_{\text{goal}}\|^2$ rewards only what the representation preserves. If the representation ignores gripper closure, a planner may reach a visually similar state without actually grasping the object. Better feature quality alone does not establish that an encoder predicts the consequences of arbitrary actions.

How the families relate

Family	Prediction target	Main benefit	Question to test
Latent dynamics (RSSM, TD-MPC2)	compact state, reward, value	cheap, repeated rollouts	does the latent retain task-relevant dynamics?
Value-equivalent (MuZero)	reward, value, policy only	no effort on irrelevant detail	does it transfer when the reward changes?
Autoregressive video (GAIA-1, Genie, IRIS)	next frame or token	explicit sequential factorization	do errors drift over long rollouts?
Diffusion/flow video (DIAMOND, Cosmos)	jointly generated frames or chunks	expressive, uncertain futures	can actions steer outcomes at control latency?
Predictive embeddings (JEPA, DINO-WM)	target features	structure without pixel reconstruction	are actions represented and goals meaningful?

These are compatible choices. A Transformer can parameterize any of them; a latent world model can decode through diffusion; a video generator can use an autoregressive outer loop and a diffusion inner loop. Architecture, prediction target, temporal factorization and training objective are separate design decisions.

Learning dynamics from data

Training examples should retain ordering and action alignment:

$$(o_t,\ a_t,\ o_{t+1},\ r_t,\ \text{done}_{t+1}),$$

where done_{t+1} marks the end of an episode. Record whether each action happens before or after each camera timestamp. Split by whole episodes, environment instances and relevant subjects;

randomly splitting adjacent frames leaks nearly identical scenes and makes a transition predictor look better than it is (see ML 10 for leakage in general).

The data-collection policy matters. A model trained on slow, gentle interactions may fail for abrupt motion. Observational correlations are especially problematic when actions are absent: a model may learn that a cup moves when a hand approaches, yet have no evidence about what alternative pushes would do. This is the difference between passive video and interventional data.

A **one-step loss** measures prediction from real history. A **multi-step loss** unrolls the model's own predictions and penalizes their accumulated error. Unrolling changes training difficulty and gradient paths; it does not remove distribution shift outside the observed interactions.

Complete CPU experiment: fit, predict, plan

We use a two-dimensional state $x_t = (\text{position}, \text{velocity})$, a scalar action u_t and a damped linear system:

$$x_{t+1} = Ax_t + Bu_t + \epsilon_t,$$

where A is the 2×2 state-transition matrix, B the action-input vector and ϵ_t small Gaussian noise. The system is deliberately fully observed. It isolates dynamics and planning before adding image encoders and partial observability. Training and test episodes are generated independently. We fit only on training transitions and evaluate both one-step prediction and open-loop rollouts with the same held-out actions.

The planner uses the **cross-entropy method** (CEM): sample candidate action sequences from a Gaussian, keep the lowest-cost "elite" sequences under the model, refit the Gaussian's mean and spread to them, and repeat. It executes one action, receives a fresh real state and replans. This is model-predictive control.

```
import numpy as np

rng = np.random.default_rng(7)
A = np.array([[1.0, 0.1], [0.0, 0.94]])
B = np.array([0.005, 0.1])

def collect(n, steps=40):
    x = np.zeros((n, steps + 1, 2))
    x[:, 0] = rng.normal(size=(n, 2)) * [0.8, 0.3]
    u = rng.uniform(-1, 1, size=(n, steps))
    for t in range(steps):
        x[:, t + 1] = (x[:, t] @ A.T + u[:, t, None] * B
                       + rng.normal(scale=0.002, size=(n, 2)))
    return x, u

train, train_u = collect(120)
test, test_u = collect(80)
phi = np.column_stack([train[:, :-1].reshape(-1, 2), train_u.ravel()])
target = train[:, 1:].reshape(-1, 2)
```

```
W = np.linalg.lstsq(phi, target, rcond=None)[0]
W_no_action = np.linalg.lstsq(phi[:, :2], target, rcond=None)[0]

def predict(x, u):
    return np.concatenate([x, np.asarray(u)[..., None]], axis=-1) @ W

one_step = predict(test[:, :-1], test_u)
rollout = np.zeros_like(test)
rollout[:, 0] = test[:, 0]
for t in range(test_u.shape[1]):
    rollout[:, t + 1] = predict(rollout[:, t], test_u[:, t])
rmse = lambda a, b: np.sqrt(np.mean((a - b) ** 2))
print("One-step:", rmse(one_step, test[:, 1:]))
print("Without actions:", rmse(test[:, :-1] @ W_no_action, test[:, 1:]))
print("Final-horizon rollout:", rmse(rollout[:, -1], test[:, -1]))

def plan(x, mean, horizon=20, population=256):
    std = np.ones(horizon)
    for _ in range(4):
        candidates = np.clip(rng.normal(mean, std, (population, horizon)), -1, 1)
        imagined = np.broadcast_to(x, (population, 2)).copy()
        costs = np.zeros(population)
        for t in range(horizon):
            imagined = predict(imagined, candidates[:, t])
            costs += (imagined[:, 0] ** 2 + 0.15 * imagined[:, 1] ** 2
                      + 0.02 * candidates[:, t] ** 2)
        costs += 20 * imagined[:, 0] ** 2
        elite = candidates[np.argsort(costs)[:32]]
        mean = elite.mean(axis=0)
        std = elite.std(axis=0) + 0.05
    return mean

real = np.array([1.5, 0.0])
mean = np.zeros(20)
for _ in range(40):
    sequence = plan(real, mean)
    real = A @ real + B * sequence[0]  # real system; never used inside plan()
    mean = np.r_[sequence[1:], 0.0]
print("Controlled final state:", real)
assert rmse(one_step, test[:, 1:]) < 0.01
assert abs(real[0]) < 0.1
```

Output:

```
One-step: 0.0019866110218121832
Without actions: 0.04106321931941812
Final-horizon rollout: 0.013772759824274517
Controlled final state: [-0.00453985 -0.0210205 ]
```

The one-step RMSE (0.0020) is at the noise level; dropping the action input raises it twentyfold (0.0411), so the action genuinely matters. The open-loop rollout error at horizon 40 (0.0138) is

about seven times the one-step error even for this nearly perfect linear model, a mild case of the compounding derived above. MPC drives the position from 1.5 to about −0.0045, compared with 1.5 under zero action. These are synthetic results from this script, not published comparisons.

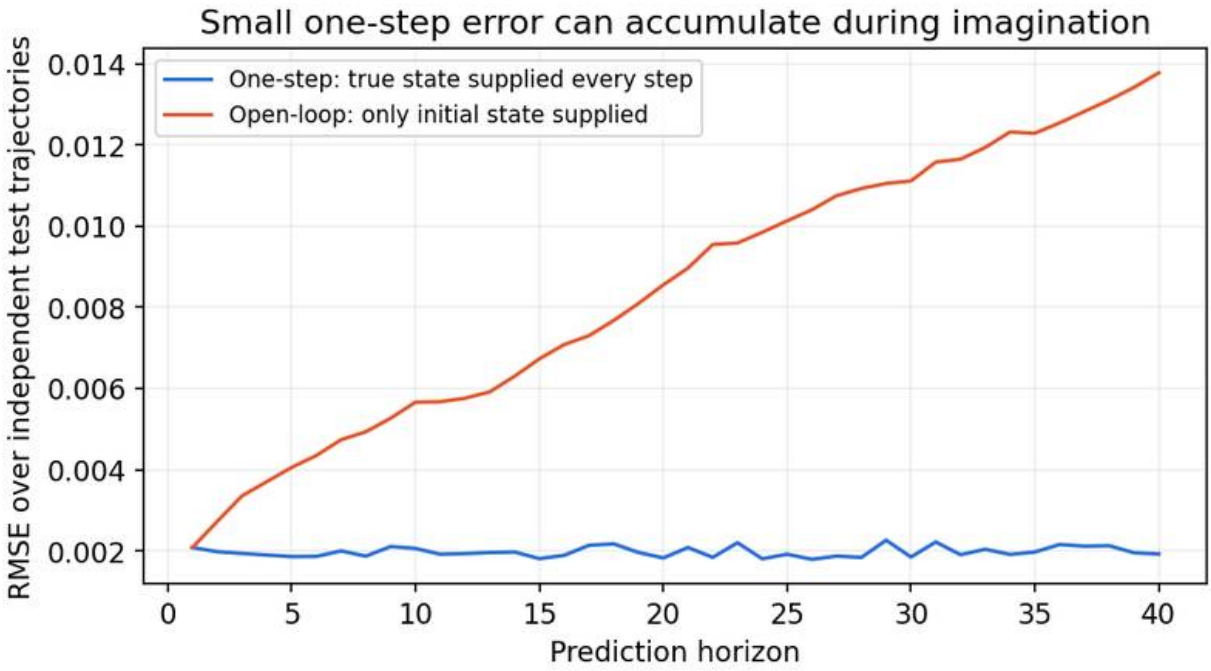


Figure 4 — Independent synthetic test episodes. One-step evaluation receives the true state each time; open-loop evaluation receives only the initial state and the held-out actions.

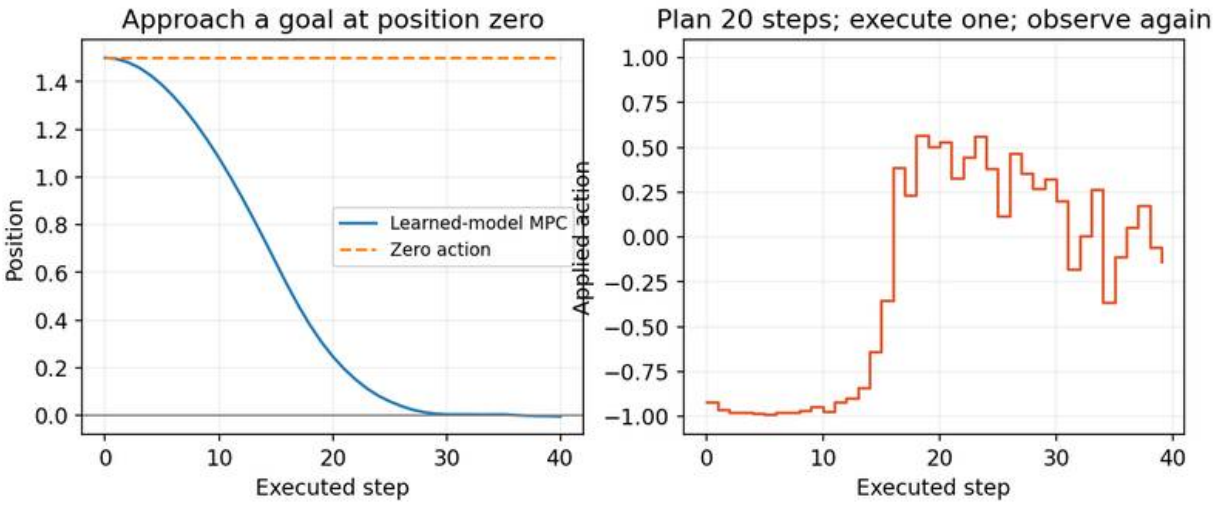


Figure 5 — CEM plans with the learned transition model. Only its first action is executed in the synthetic environment before replanning from a new observation.

Replacing x_t with an image embedding does not reproduce Dreamer or V-JEPA 2-AC: it adds an encoder-learning problem, latent uncertainty and the need for a valid goal or reward.

Planning, uncertainty and model exploitation

Plain version. A planner looks for the action the model likes best. If the model is wrong somewhere, the planner will happily go there. Knowing *where* the model is unsure is what keeps it honest.

A finite-horizon planning objective can be written as

$$a^*_{t:t+H-1} = \arg\min_{a_{t:t+H-1}} \mathbb{E}_\theta\Big[\sum_{k=1}^{H} c(z_{t+k}, a_{t+k-1}) + \lambda_{\mathrm{T}}\, c_{\mathrm{T}}(z_{t+H})\Big],$$

where c is a per-step cost, c_{T} a terminal cost weighted by λ_{T}, and the expectation is over the model's predicted latent states z. **Model-predictive control** executes the first action, observes again and replans. Feedback corrects drift, but cannot rescue an action whose immediate consequence was badly modeled.

Two kinds of uncertainty

Aleatoric uncertainty is inherent randomness (a coin flip, the ball that may go left or right); more data does not remove it. **Epistemic** uncertainty comes from lack of data (the model has never seen this situation); more data does remove it. Sampling a generative model can represent aleatoric uncertainty, but sample diversity is not proof of calibration. Epistemic uncertainty is commonly approximated with **ensembles**: several models trained on bootstrap resamples or with different initializations agree where data is dense and tend to disagree where it is not. Chua et al.'s PETS combined probabilistic networks (for aleatoric noise) in an ensemble (for epistemic uncertainty) with sampling-based uncertainty propagation and CEM planning, matching the final performance of strong model-free methods on several benchmarks with far fewer samples [29].

Code: a planner exploiting a learned model

The agent has logged data only for actions in $[-1, 1]$, but its planner may choose any action in $[-3, 3]$.

```
import numpy as np
from sklearn.neural_network import MLPRegressor

rng = np.random.default_rng(0)
true_reward = lambda a: np.sin(3 * a) * np.exp(-0.3 * a ** 2)   # unknown to the agent

# Logged interactions only cover actions in [-1, 1].
a_data = rng.uniform(-1, 1, 200)
r_data = true_reward(a_data) + 0.05 * rng.normal(size=200)
```

```
ensemble = []
for k in range(5):                        # bootstrap + different initialization
    idx = rng.integers(0, 200, 200)
    m = MLPRegressor(hidden_layer_sizes=(64, 64), max_iter=3000, random_state=k)
    ensemble.append(m.fit(a_data[idx, None], r_data[idx]))

grid = np.linspace(-3, 3, 601)
preds = np.stack([m.predict(grid[:, None]) for m in ensemble])
mean, std = preds.mean(0), preds.std(0)

for name, score in [("single model", preds[0]),
                    ("ensemble mean", mean),
                    ("pessimistic: mean - 2 std", mean - 2 * std)]:
    a_star = grid[np.argmax(score)]       # the "planner": pick the best-looking action
    print(f"{name:26s} picks a = {a_star:+.2f}, predicted {score.max():.2f}, "
          f"true {true_reward(a_star):.2f}")
print(f"mean std inside data {std[np.abs(grid) <= 1].mean():.3f}, "
      f"outside {std[np.abs(grid) > 1.5].mean():.3f}")
```

Output:

```
single model               picks a = -3.00, predicted 1.79, true -0.03
ensemble mean              picks a = -3.00, predicted 1.58, true -0.03
pessimistic: mean - 2 std  picks a = +0.43, predicted 0.87, true 0.91
mean std inside data 0.025, outside 0.321
```

Both the single model and the ensemble mean extrapolate a rising line outside the data, and the planner runs to the edge of the action space, where the true reward is about zero. Ensemble disagreement is thirteen times larger outside the data than inside, and a *pessimistic* score (mean minus two standard deviations) keeps the planner inside the data, where it finds an action with true reward 0.91. Restricting actions, shortening horizons, penalizing disagreement and testing with real feedback are the practical responses. Agreement among ensemble members is not proof of correctness, though: all five can share the same bias.

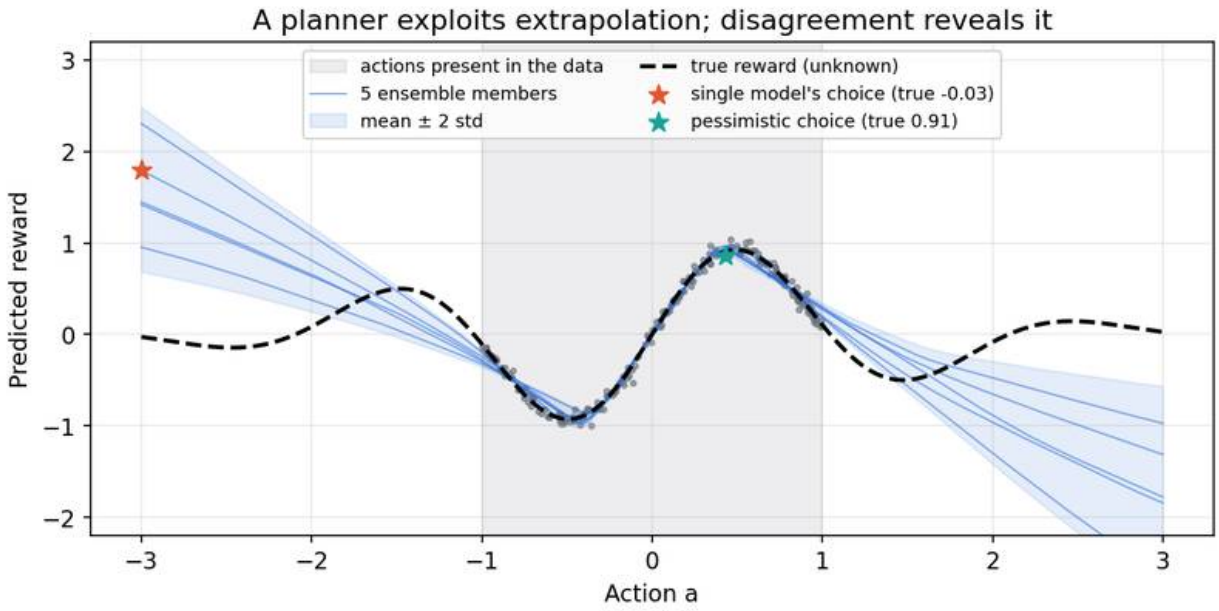


Figure 6 — Output of the code above. Optimizing a learned model drives the planner to where the model is least reliable; ensemble spread exposes those regions.

How to evaluate a world model

Question	Measurement	Important control
Does it predict known dynamics?	one-step and horizon-indexed state or feature error	independent episodes; fixed action sequence
Does appearance stay coherent?	reconstruction and video-quality metrics, identity consistency	compare camera motion and occlusions
Do actions matter correctly?	matched initial context with different specified actions	real action-conditioned reference trajectories
Are physical constraints respected?	collisions, object permanence, conservation; benchmarks such as Physics-IQ [22]	task-specific tolerances; out-of-distribution cases [23]
Does it help decisions?	success rate, return, interventions, control latency	test in the real or a trusted environment

Pixel fidelity and decision usefulness can disagree, as the blur example showed. An averaged prediction can have low squared error while showing an impossible scene; an accurate latent predictor can omit visually irrelevant texture. Report the target, horizon, conditions and output resolution with every metric.

For comparison, include a persistence baseline (predict "nothing changes"), an action-free predictor and a simple task-appropriate dynamics model. Evaluate held-out objects, viewpoints, action ranges and environment properties separately. Report closed-loop control results in addition to

imagined ones, and account for encoder and sampler latency: a planner that needs ten seconds per action may be useless for a robot running at 10 Hz.

Modern view

Surveys. Moerland et al. give the most systematic treatment of model-based RL: dynamics-model learning (stochasticity, uncertainty, partial observability, temporal abstraction), a taxonomy of how planning and learning are integrated, and implicit (value-equivalent, MuZero-style) approaches [5]. Ding et al. (ACM Computing Surveys) organize the broader "world model" literature by function, understanding the world versus predicting its future, and review applications in generative games, autonomous driving, robotics and social simulation [1]. Zhu et al. survey "general world models" from the video-generation side, covering video generators, autonomous-driving world models and world models inside autonomous agents, and discuss their limitations [31]. Guan et al. give an initial survey of world models specifically for autonomous driving [32].

State of the art, 2023–2026.

- *Model-based RL matured.* DreamerV3 showed a single configuration working across more than 150 tasks and was published in Nature in 2025 [10]; TD-MPC2 showed decoder-free latent planning scaling to a multi-task 317M-parameter agent [11]; IRIS and DIAMOND showed Transformer and diffusion world models competitive on the Atari 100k benchmark [12], [13].
- *Video generators were reframed as simulators.* GAIA-1, UniSim, Genie, Genie 3 and Cosmos trained large models on video with actions, text or inferred latent actions [17], [18], [19], [20], [21]. The strongest evidence that they help agents is UniSim's zero-shot transfer of policies trained inside the simulator [18]; the strongest caution is that realism does not imply physics [22], [23].
- *Predictive representations moved toward control.* The JEPA line went from images (I-JEPA) to video (V-JEPA) to action-conditioned planning on real robots (V-JEPA 2-AC) and denser features (V-JEPA 2.1) [24], [25], [26], [27]; DINO-WM showed that frozen self-supervised features can already support zero-shot planning [28].

Open problems.

- *Long-horizon consistency.* Errors compound; generated worlds drift; object identity and off-screen state are hard to keep for minutes, let alone hours [20].
- *Causal and physical correctness.* Models trained on passive video learn correlations; out-of-distribution physics fails [22], [23]. Interventional (action) data is scarce and expensive.
- *Action interfaces.* Latent actions inferred from video, robot joint commands and text instructions are different things; mapping between them reliably is unsolved.
- *Exploitation and uncertainty.* Planners and policies trained in imagination exploit model errors; calibrated epistemic uncertainty for large video models is largely missing.
- *Evaluation.* There is no agreed benchmark that ties prediction quality to downstream decision quality across domains; most papers still report either video metrics or task returns, rarely both.
- *Cost.* Sampling a large video model can be far slower than real time, which conflicts with closed-loop control.

Key takeaways

- A world model predicts how an environment changes, usually under the agent's actions; its prediction target and action interface determine what it can support.
- Observations are not states. Partial observability requires memory or belief estimation.
- A model can be used to plan online (MPC), to train a policy in imagination (World Models, Dreamer), or to search with a value-equivalent latent (MuZero).
- RSSMs combine deterministic memory with stochastic latents; the KL term trains the prior to imagine what the posterior will see.
- Squared-error video prediction averages possible futures into blur; stochastic and generative models trade lower likelihood of blur for realistic samples, but realism is not physical understanding.
- JEPA-style models predict features instead of pixels; they need anti-collapse design and meaningful goals in feature space.
- One-step errors compound over long horizons, and optimizers exploit model mistakes; ensembles, pessimism, short rollouts and replanning are the standard defenses.
- The strongest evidence connects controlled action interventions and held-out predictions to successful behavior in the real environment.

Exercises

1. **Action ablation.** In the CPU experiment, remove the action column. Compare one-step error, horizon-40 error and control behavior. Why can a short-term action-free predictor still look reasonable?

   Hint

   Imagined actions no longer affect the action-free predictor, so a planner cannot rank their consequences at all: every candidate gets the same predicted cost. Over one step the damped dynamics change the state only slightly, so persistence-like predictions still score moderately well, which is why one-step error alone is a weak test.

2. **Partial observations.** Hide velocity and fit position-to-position dynamics. Then use the last two positions and the previous action as features. When does this history help, and where does observation noise hurt?

   Hint

   The difference of two positions estimates velocity (a finite difference), so the two-position model recovers most of the state. Noise in each position is amplified when divided by the time step, so the benefit shrinks as observation noise grows.

3. **Planning horizon.** Compare horizons 5, 20 and 40, adjusting the initial mean to the chosen length. Measure final distance, action effort and runtime across several independent seeds.

Hint

Keep the cost definition fixed and count all model calls. Very short horizons cannot "see" that braking early avoids overshoot; very long horizons cost more and accumulate more model error, although for this linear system the error is small.

4. **Domain shift.** Collect training episodes with velocity damping 0.94, then execute control in an environment with damping 0.75. Compare open-loop execution of a plan with replanning at every step. Identify an action range where the training data provide weak coverage.

Hint

Separate the learned dynamics (fitted on 0.94) from the true execution dynamics (0.75); do not refit on evaluation transitions. Replanning uses fresh observations, so it partially corrects the biased model; open-loop execution cannot.

5. **Mean versus sample.** Extend the blur example to three equally likely futures at positions 10, 32 and 54. Compute the expected MSE of the mean prediction and of a single sharp sample. Then propose a metric that would prefer the sample.

Hint

The mean now has three one-third-height bumps; the expected MSEs are about 0.037 (mean) and 0.074 (sample), again a factor of two. That factor is general: if X and Y are independent draws from the same distribution, $\mathbb{E}\|X-Y\|^2 = 2\,\mathbb{E}\|X-\mathbb{E}X\|^2$, so a perfect sampler always has twice the MSE of the mean. A distribution-level metric (for example, comparing sets of generated samples with sets of real futures, as FID-style metrics do for images) or a best-of-N error over several samples would reward sharp, plausible predictions.

6. **Pessimism trade-off.** In the ensemble code, replace the factor 2 in "mean − 2 std" by 0, 0.5, 1 and 4. Plot the chosen action and its true reward for each. What goes wrong when the factor is too large?

Hint

At 0 the planner exploits extrapolation; as the factor grows it stays inside the data. Too much pessimism makes the planner avoid any action with even slight disagreement, so it may miss the best action inside the data and, in an RL setting, stop exploring.

References

1. J. Ding et al., "Understanding world or predicting future? A comprehensive survey of world models," *ACM Computing Surveys*, vol. 58, no. 3, 2025. doi arXiv
2. C. M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006, ch. 13 "Sequential Data". book

3. R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed., Springer, 2022. book
4. R. S. Sutton, "Dyna, an integrated architecture for learning, planning, and reacting," *ACM SIGART Bulletin*, vol. 2, no. 4, 1991. doi
5. T. M. Moerland, J. Broekens, A. Plaat and C. M. Jonker, "Model-based reinforcement learning: a survey," *Foundations and Trends in Machine Learning*, vol. 16, no. 1, 2023. doi arXiv
6. D. Ha and J. Schmidhuber, "Recurrent world models facilitate policy evolution," NeurIPS, 2018; interactive version "World Models," arXiv:1803.10122. paper interactive
7. D. Hafner et al., "Learning latent dynamics for planning from pixels," ICML, 2019. paper
8. D. Hafner, T. Lillicrap, J. Ba and M. Norouzi, "Dream to control: learning behaviors by latent imagination," ICLR, 2020. arXiv
9. D. Hafner, T. Lillicrap, M. Norouzi and J. Ba, "Mastering Atari with discrete world models," ICLR, 2021. arXiv
10. D. Hafner, J. Pasukonis, J. Ba and T. Lillicrap, "Mastering diverse control tasks through world models," *Nature*, vol. 640, 2025. doi arXiv
11. N. Hansen, H. Su and X. Wang, "TD-MPC2: scalable, robust world models for continuous control," ICLR, 2024. arXiv
12. V. Micheli, E. Alonso and F. Fleuret, "Transformers are sample-efficient world models," ICLR, 2023. arXiv
13. E. Alonso et al., "Diffusion for world modeling: visual details matter in Atari," NeurIPS, 2024. arXiv
14. J. Schrittwieser et al., "Mastering Atari, Go, chess and shogi by planning with a learned model," *Nature*, vol. 588, 2020. doi arXiv
15. C. Finn, I. Goodfellow and S. Levine, "Unsupervised learning for physical interaction through video prediction," NIPS, 2016. arXiv
16. M. Babaeizadeh, C. Finn, D. Erhan, R. H. Campbell and S. Levine, "Stochastic variational video prediction," arXiv:1710.11252, 2017. arXiv
17. A. Hu et al., "GAIA-1: a generative world model for autonomous driving," technical report, arXiv:2309.17080, 2023. arXiv
18. S. Yang et al., "Learning interactive real-world simulators," arXiv:2310.06114, 2023. arXiv
19. J. Bruce et al., "Genie: generative interactive environments," arXiv:2402.15391, 2024. arXiv
20. Google DeepMind, "Genie 3: a new frontier for world models," blog post, August 5, 2025. blog
21. NVIDIA (N. Agarwal et al.), "Cosmos world foundation model platform for physical AI," arXiv:2501.03575, 2025. arXiv
22. S. Motamed, L. Culp, K. Swersky, P. Jaini and R. Geirhos, "Do generative video models understand physical principles?," arXiv:2501.09038, 2025. arXiv
23. B. Kang et al., "How far is video generation from world model: a physical law perspective," ICML, 2025. arXiv
24. M. Assran et al., "Self-supervised learning from images with a joint-embedding predictive architecture," CVPR, 2023. doi arXiv
25. A. Bardes et al., "Revisiting feature prediction for learning visual representations from video," arXiv:2404.08471, 2024. arXiv
26. M. Assran et al., "V-JEPA 2: self-supervised video models enable understanding, prediction and planning," arXiv:2506.09985, 2025. arXiv
27. L. Mur-Labadia et al., "V-JEPA 2.1: unlocking dense features in video self-supervised learn-

ing," arXiv:2603.14482, 2026. arXiv
28. G. Zhou, H. Pan, Y. LeCun and L. Pinto, "DINO-WM: world models on pre-trained visual features enable zero-shot planning," arXiv:2411.04983, 2024. arXiv
29. K. Chua, R. Calandra, R. McAllister and S. Levine, "Deep reinforcement learning in a handful of trials using probabilistic dynamics models," NeurIPS, 2018. arXiv
30. M. Janner, J. Fu, M. Zhang and S. Levine, "When to trust your model: model-based policy optimization," NeurIPS, 2019. arXiv
31. Z. Zhu et al., "Is Sora a world simulator? A comprehensive survey on general world models and beyond," arXiv:2405.03520, 2024. arXiv
32. Y. Guan et al., "World models for autonomous driving: an initial survey," arXiv:2403.02622, 2024. arXiv

Part VII

Optimization

OPT 01 • Modeling Decisions and Convex Sets

Turn an engineering decision into variables, objectives and constraints; understand convex geometry, projections, separation and the limits of numerical solutions.

Online edition: https://blog.shyandram.dev/en/learn/optimization/opt-01-modeling-convex-sets/

What you'll learn

- Translate a decision into variables, an objective, constraints and fixed data.
- Distinguish feasible, optimal, infeasible, unbounded and unattained problems.
- Understand affine sets, convex combinations, convex hulls and cones.
- Recognize halfspaces, polyhedra, norm balls, ellipsoids and positive-semidefinite cones.
- Use convexity-preserving operations, projections and separating hyperplanes.
- Check what a numerical solution establishes about the model and the underlying application.

The big picture

An image restoration system must choose pixel values. A scheduler must choose resource allocations. A fitting procedure must choose coefficients. In each case there are many possible choices, a way to compare them, and restrictions on what choices are allowed. Optimization makes those three ingredients explicit.

This course follows Stephen Boyd and Lieven Vandenberghe's *Convex Optimization* [1]. Its emphasis is learning to recognize, formulate and solve problems whose geometry supports global guarantees. The first chapters develop sets, functions, standard problem forms and duality. Applications connect that theory to fitting, statistical estimation and geometric design. The final chapters examine numerical algorithms and modern extensions.

> **Explain like I'm five**
>
> Imagine choosing ingredients for a smoothie. You want a tasty drink, have a fixed budget, and cannot use ingredients that have run out. The ingredient amounts are your choices; taste is what you improve; budget and availability are rules. If mixing any two allowed recipes still gives an allowed recipe, the allowed recipes form a convex set.

The analogy is deliberately limited: taste need not be a convex function, and real ingredients can come in indivisible packages. The first skill is to identify which parts of a real decision fit a convex model and which parts do not. A convincing model is more valuable than a fast solver applied to the wrong equations.

Convex optimization is also useful around nonconvex learning. It can solve a linear classifier on fixed features, compute a constrained prediction, fit a calibration curve or construct a lower

bound for a harder problem. It does not turn an arbitrary neural network's training objective into a convex one. The guarantees belong to the particular mathematical problem being solved.

From a sentence to an optimization model

Plain version. Decide what you can change, what is already known, what makes a choice good, and which rules it must obey. Write these down before choosing an algorithm.

A general scalar optimization problem can be expressed as

$$\begin{aligned} \text{minimize}_{x \in \mathbb{R}^n} \quad & f_0(x) \\ \text{subject to} \quad & f_i(x) \le 0, \quad i = 1, \dots, m, \\ & h_j(x) = 0, \quad j = 1, \dots, p. \end{aligned}$$

Here x is the vector of n decision variables; f_0 is the scalar objective to minimize; f_i are m inequality constraint functions; and h_j are p equality constraint functions. Fixed measurements and design parameters enter these functions but are not decision variables unless the model explicitly makes them so.

For example, suppose two operations use amounts x_1 and x_2 of a shared resource. A capacity constraint might be $2x_1 + x_2 \le 5$, while $x_1, x_2 \ge 0$ rules out negative allocations. If the target allocation is b, an objective $\|x - b\|_2^2$ prefers choices close to that target. The numbers in this example specify a synthetic model; they are not measurements of a production system.

Units are part of the formulation. Adding a cost in dollars to a reconstruction error in intensity units requires a weighting convention. A coefficient that converts between their scales should be interpreted and tested. Changing image normalization from zero-to-255 to zero-to-one can change the effective regularization if other terms are left unchanged.

It is helpful to keep a small modeling ledger: each variable's meaning and unit, each fixed input's provenance, the justification for each constraint, and the interpretation of the objective. This avoids treating a convenient formula as if it were the only possible representation of the task.

Constraints and penalties express different commitments

A hard constraint says a choice is disallowed. A penalty says a violation or deviation is undesirable but can be exchanged against other benefits. The distinction matters even if both produce similar answers on one example.

Consider nonnegative image intensities. Enforcing $x \ge 0$ makes all feasible outputs nonnegative. Adding a quadratic penalty on negative entries may still permit some negativity if it improves the data term enough. Increasing a penalty is not automatically the same as enforcing an exact constraint, especially with finite numerical precision.

Likewise, a budget constraint sets a fixed resource limit. A cost penalty expresses a preference that depends on the tradeoff coefficient. Choosing that coefficient is part of the application, not

something convexity decides. Later chapters derive how dual multipliers quantify the sensitivity to changing hard constraints.

Worked model: a target allocation under a capacity limit

A target need not obey the available capacity. Optimization finds the nearest allowed allocation, rather than reducing every coordinate by the same amount.

Take the synthetic target $b = (3, 3)$ and minimize $\frac{1}{2}\|x - b\|_2^2$ with $x \geq 0$ and $2x_1 + x_2 \leq 5$. The target uses nine units of capacity, so it is infeasible. At the solution the capacity boundary is active. Its normal is $a = (2, 1)$, so the shortest correction points along a:

$$x^* = b - \alpha a, \qquad \alpha = \frac{a^\top b - 5}{a^\top a} = \frac{4}{5}, \qquad x^* = \left(\frac{7}{5}, \frac{11}{5}\right).$$

Here x^* is the candidate allocation, α is the correction scale, and a is the capacity normal. Substituting gives $2x_1^* + x_2^* = 5$ and both coordinates are positive, so the nonnegativity constraints do not alter this projection.

Why is this the closest point? For any feasible x, $a^\top(x - x^*) \leq 0$. Expand the objective difference:

$$\frac{1}{2}\|x - b\|_2^2 - \frac{1}{2}\|x^* - b\|_2^2 = \frac{1}{2}\|x - x^*\|_2^2 - \alpha a^\top(x - x^*) \geq 0.$$

Here x is any feasible competitor; all other symbols are defined above. Both terms are nonnegative. The candidate therefore minimizes the objective globally, whose value is $8/5$. This calculation connects the variables, the active rule, the correction direction and the optimum before introducing a solver [1].

Feasibility and optimal value

Plain version. An allowed answer and the best allowed answer are different objects. Sometimes there are no allowed answers; sometimes the model prefers choices that run away without bound.

The feasible set is

$$\mathcal{F} = \{x : f_i(x) \leq 0 \text{ for every } i,\ h_j(x) = 0 \text{ for every } j,\ x \text{ in all required domains}\}.$$

Here $\mathcal{F}$ is the set of points satisfying all constraints and function-domain restrictions. A point in $\mathcal{F}$ is feasible. The optimal value is

$$p^* = \inf_{x \in \mathcal{F}} f_0(x).$$

p^* is the greatest lower bound on achievable objective values. An optimizer x^* is a feasible point attaining that value: $f_0(x^*) = p^*$. The infimum can exist without an optimizer.

Four cases are worth distinguishing. A feasible problem can have an attained finite optimum. An infeasible problem has an empty feasible set. An unbounded-below problem has objective

values tending to negative infinity. An unattained problem can have a finite limiting value that no feasible point reaches.

For a concrete unattained example, minimize x subject to $x > 0$. The optimal value is zero, but zero is excluded. For an unbounded example, minimize $-x$ on $x \geq 0$. For an infeasible example, require both $x \geq 1$ and $x \leq 0$. These have very different implications even if a solver returns no finite optimizer.

Closedness and boundedness help establish existence. A continuous objective on a nonempty compact feasible set attains a minimum. A closed but unbounded set can also admit a minimizer if the objective grows sufficiently at infinity. The existence question concerns the model before any algorithm starts.

Numerical status is evidence about a finite-precision computation

A solver typically assesses constraint residuals, optimality conditions or a primal–dual gap against tolerances. A successful status does not imply exact real-number feasibility. It also does not establish that the measurements, objective or application constraints were correct.

If the application requires a genuinely safe limit, decide how numerical tolerances relate to that limit. Rescaling a constraint can change its raw residual without changing the feasible set. For example, $x \leq 1$ and $10^6 x \leq 10^6$ are equivalent mathematically but have very different unnormalized residual magnitudes.

Record the scale, tolerance and units of residuals. Do not compare objective values between runs whose objective definitions differ. If an algorithm stops because iterations are exhausted, report that condition rather than interpreting a nearly unchanged iterate as a proof of optimality.

Affine and convex sets

Plain version. An affine set contains complete lines through its points. A convex set only needs to contain the line segments between its points.

For points $u, v \in \mathbb{R}^n$ and scalar θ, a two-point affine combination is

$$x = \theta u + (1 - \theta)v.$$

Here u and v are vectors, x is their combination, and θ controls position along their line. Allowing every real θ traces the entire line. Restricting $0 \leq \theta \leq 1$ traces the segment between the endpoints.

A set C is affine if it contains all affine combinations of its points. It is convex if it contains all convex combinations, where coefficients are nonnegative and sum to one. Thus every affine set is convex, while a disk is convex without being affine.

For finitely many points the convex-combination rule is

$$x = \sum_{i=1}^{r} \theta_i v_i, \qquad \theta_i \geq 0, \qquad \sum_{i=1}^{r} \theta_i = 1.$$

v_i are the r endpoint vectors and θ_i are their weights. The combination is a weighted average. Nonnegative weights are essential: extrapolation can leave a convex set.

Convexity says nothing by itself about smooth boundaries, boundedness or closedness. An open ball is convex; a line is unbounded and convex; a single point is convex. The empty set is conventionally convex because there are no endpoint pairs violating the definition.

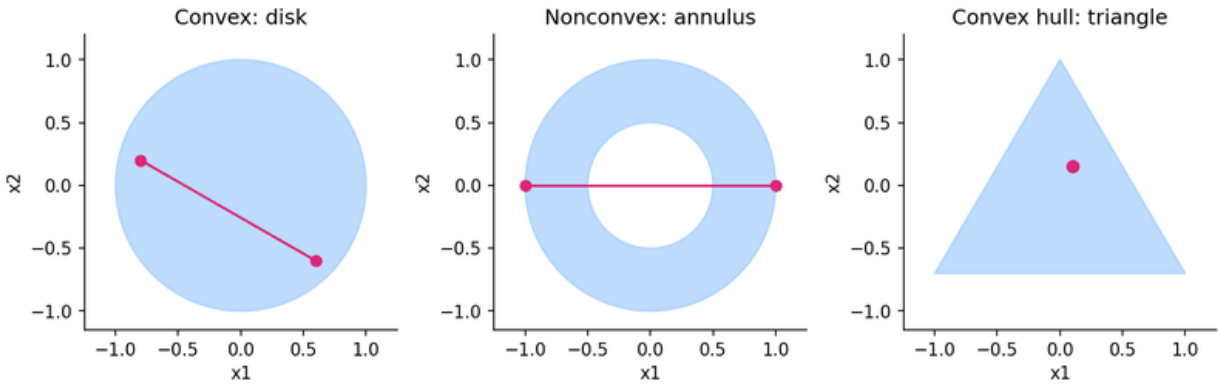


Figure 1 — Original synthetic geometry. A disk and a filled triangle are convex. The annulus is not: two allowed endpoints have a segment that passes through the excluded hole.

A counterexample disproves; sampled tests do not prove

To disprove convexity, find two points in the set whose midpoint or another segment point lies outside. This is often simpler than analyzing a complicated boundary. A ring fails because opposite points average to its excluded center.

To prove convexity, use the definition or a preservation rule. Testing many random pairs can reveal a bug but cannot establish the property for every pair. A tiny nonconvex notch may escape all sampled checks.

The distinction is similar to numerical gradient checks. They are valuable implementation diagnostics, while the mathematical argument addresses a universal statement. Use both for different purposes, and label their roles clearly.

Code: combinations and a nonconvex counterexample

This self-contained NumPy example verifies particular points. The mathematical proof for the disk follows from the triangle inequality, not from the assertions alone.

```
import numpy as np

u = np.array([0.8, 0.1])
v = np.array([-0.2, 0.7])
theta = np.linspace(0, 1, 101)
```

```
segment = theta[:, None] * u + (1 - theta[:, None]) * v
assert np.linalg.norm(segment, axis=1).max() <= 1 + 1e-12
ring_u, ring_v = np.array([1.0, 0.0]), np.array([-1.0, 0.0])
midpoint = (ring_u + ring_v) / 2
assert not (0.5 <= np.linalg.norm(midpoint) <= 1.2)
print("disk sampled-segment maximum norm:", np.linalg.norm(segment, axis=1).max())
print("ring midpoint:", midpoint, "norm:", np.linalg.norm(midpoint))
```

Convex hulls and relaxations

Plain version. A convex hull fills in every weighted average of a collection of choices. It is useful when discrete choices are hard, but its added points may be unavailable in the original application.

The convex hull of a set S is

$$\operatorname{conv}(S) = \left\{ \sum_{i=1}^{r} \theta_i v_i : r \text{ finite}, \ v_i \in S, \ \theta_i \geq 0, \ \sum_i \theta_i = 1 \right\}.$$

Here S is the original collection; r is the number of selected points in a finite combination; and $\operatorname{conv}(S)$ is the smallest convex set containing all of S.

Suppose an allocation must choose one of three configurations. The convex hull also permits mixtures. If mixtures correspond to time sharing, these points may be operationally meaningful. If the decision must select a single indivisible machine, a fractional hull point is a relaxation, not a feasible original answer.

For a minimization problem, enlarging the feasible set cannot increase the optimal value. Therefore a convex relaxation can provide a lower bound on a discrete problem. Turning its optimizer into an actual discrete choice requires another step, and the resulting rounded choice may violate constraints or lose objective quality.

Do not describe an exact optimum of the relaxed problem as an exact optimum of the original problem. Keep the original feasible set, relaxed set, relaxed lower bound and achieved original objective separate. The applications chapter returns to this distinction.

A working vocabulary of convex sets

Plain version. Most useful convex models are assembled from a few reusable geometric shapes. Learning those shapes makes an unfamiliar equation easier to classify.

Hyperplanes, halfspaces and polyhedra

A hyperplane and a closed halfspace have forms

$$H = \{x : a^\top x = b\}, \qquad P = \{x : a^\top x \leq b\}.$$

$x \in \mathbb{R}^n$ is a point, $a \in \mathbb{R}^n$ is a nonzero normal vector and $b \in \mathbb{R}$ is an offset. A hyperplane is affine. A halfspace contains points on one side and its boundary.

A polyhedron is an intersection of finitely many halfspaces and hyperplanes, often written $Ax \leq b$, $Ex = d$. Matrices A, E hold constraint coefficients; vectors b, d hold bounds. The inequality is componentwise. Polyhedra can be bounded, unbounded, lower-dimensional or empty.

A box is a polyhedron with coordinate bounds. A probability simplex adds nonnegativity and a sum-to-one equality. These forms show why probability vectors and bounded pixel values frequently fit convex constraints.

Norm balls and ellipsoids

A norm ball is

$$B = \{x : \|x - c\| \leq r\}.$$

c is its center, $r \geq 0$ is its radius and $\|\cdot\|$ is a norm. Norm triangle inequalities imply convexity. Different norms encode different geometry: a Euclidean ball is round, an infinity-norm ball is a box, and a one-norm ball has corners.

An ellipsoid can be written

$$\mathcal{E} = \{c + Au : \|u\|_2 \leq 1\}.$$

A is a matrix mapping the unit Euclidean ball, u is the auxiliary ball coordinate and c is the center. If A is invertible, an equivalent quadratic description uses a positive-definite shape matrix. Singular A produces a lower-dimensional image, which remains convex.

Cones and matrix inequalities

A convex cone is closed under nonnegative combinations:

$$\alpha u + \beta v \in K \quad \text{for } u, v \in K,\ \alpha, \beta \geq 0.$$

K is the cone, u, v are its points and α, β are nonnegative scalars. The weights do not have to sum to one. The nonnegative orthant is a simple example.

The second-order cone consists of pairs (t, z) satisfying $\|z\|_2 \leq t$, where t is scalar and z is a vector. It represents many norm constraints.

The positive-semidefinite cone contains symmetric matrices X satisfying

$$v^\top X v \geq 0 \quad \text{for every } v \in \mathbb{R}^n.$$

$X \in \mathbb{R}^{n \times n}$ is symmetric, and v is an arbitrary test vector. This condition is written $X \succeq 0$. It is not the same as requiring every matrix entry to be nonnegative. Covariance matrices naturally lie in this cone.

Generalized inequalities use a cone to define order: $x \preceq_K y$ means $y - x \in K$. For the orthant this becomes coordinatewise order; for symmetric matrices it becomes positive-semidefinite order. Stating the cone avoids confusing these different comparisons.

Operations that preserve convexity

Plain version. Once a set is known to be convex, certain operations create another convex set automatically. These rules are stronger and easier to reuse than repeated plotting.

Intersections of convex sets are convex. A point on the segment between two members remains in every set being intersected. This is why combining convex constraints is straightforward.

Affine images and inverse images preserve convexity. If C is convex, both the transformed set $\{Ax + b : x \in C\}$ and the set $\{x : Ax + b \in C\}$ are convex. Here A is a fixed matrix and b a fixed vector. Nonlinear transformations do not have this general guarantee.

Cartesian products and sums of convex sets are convex. Projection onto selected coordinates is an affine image, so eliminating auxiliary variables preserves convexity of the feasible set. This fact underlies many extended formulations: a model may be easier to express with extra variables.

Unions are usually not convex. Two disjoint balls illustrate the failure. Replacing "and" with "or" in a constraint specification can fundamentally change the problem, even when each branch is individually convex.

For matrix variables, the same reasoning works in a vector space of symmetric matrices. If X and Y are positive semidefinite, then $\theta X + (1 - \theta)Y$ is positive semidefinite for $0 \le \theta \le 1$: test the quadratic form against every vector.

Two short proofs you can reuse

Convexity proofs usually track a segment through the construction. Suppose $u, v \in C_1 \cap C_2$, where C_1, C_2 are convex. For $0 \le \theta \le 1$, the mixture $\theta u + (1 - \theta)v$ belongs to each set, hence to their intersection. The same argument works for any collection of convex sets.

For an affine image, let $y_1 = Au + b$ and $y_2 = Av + b$ with $u, v \in C$. Their mixture is

$$\theta y_1 + (1 - \theta)y_2 = A[\theta u + (1 - \theta)v] + b.$$

Here C is convex, A, b are fixed transformation data, and y_1, y_2 are image points. The bracketed point belongs to C, so the mixture belongs to the image. This is why eliminating auxiliary coordinates preserves a convex feasible set [1, Section 2.3].

The corresponding union proof fails: two endpoints can come from different sets. For instance, $-1 \in [-2, -1]$ and $1 \in [1, 2]$, but their midpoint zero lies in neither interval. Each interval is convex; the union is not. In modeling language, simultaneous convex rules preserve convexity, while an either-or choice can break it.

Projections and separating hyperplanes

Plain version. Projection finds the closest allowed point to an arbitrary point. The direction from that point toward its projection also reveals which side of a boundary contains the feasible set.

For a nonempty closed convex set C, Euclidean projection is

$$\Pi_C(y) = \arg\min_{x \in C} \frac{1}{2}\|x - y\|_2^2.$$

$y \in \mathbb{R}^n$ is the query point, x is the candidate feasible point and $\Pi_C(y)$ is the unique minimizing point. Strict convexity of squared Euclidean distance ensures uniqueness. Other distances or nonconvex sets need not have the same property.

Let $z = \Pi_C(y)$. Its optimality condition is

$$(y - z)^\top (x - z) \le 0 \quad \text{for every } x \in C.$$

Here z is the projection and x ranges over feasible points. The condition follows by considering a small step from z toward any $x \in C$ and requiring that squared distance cannot decrease.

If $y \notin C$, the vector $y - z$ is nonzero. It defines a hyperplane through z with the entire feasible set on the opposite side from y. This is a concrete separating-hyperplane construction.

More general separation theorems compare disjoint convex sets [1, Section 2.5]. Separation need not have a positive margin without additional assumptions. Closedness, compactness and distance between sets matter when asserting strict separation.

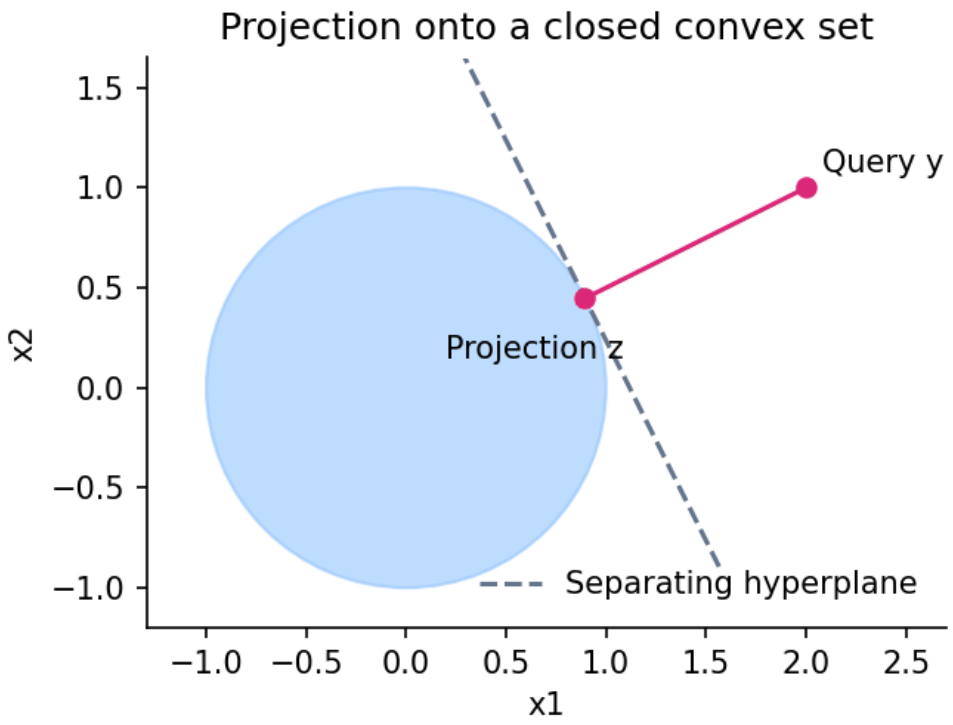


Figure 2 — Original analytical illustration of Euclidean projection onto the unit disk. The projection direction supplies a separating normal for this external point.

Code: box and ball projections

Projection onto a box clips each coordinate. Projection onto a Euclidean ball rescales a point only if it lies outside. These formulas follow from the geometry and are covered by the geometric applications in [1, Section 8.1].

```
import numpy as np

y = np.array([2.0, -1.5])
lower, upper = np.array([0.0, -1.0]), np.array([1.0, 1.0])
box_projection = np.clip(y, lower, upper)
radius = 1.0
ball_projection = y * min(1.0, radius / np.linalg.norm(y))
rng = np.random.default_rng(2)
points = rng.normal(size=(1000, 2))
points /= np.maximum(1.0, np.linalg.norm(points, axis=1))[:, None]
inequality = (points - ball_projection) @ (y - ball_projection)
assert inequality.max() <= 1e-12
assert np.all(box_projection >= lower) and np.all(box_projection <= upper)
print("box projection:", box_projection)
print("ball projection:", ball_projection)
```

The sampled check illustrates the projection inequality. It does not replace a proof for all feasible points. More complicated projections can themselves require solving an optimization problem; “projected gradient” does not imply that projection is inexpensive.

Deriving the projection test

The projection inequality comes from trying every feasible direction, not from drawing a tangent that looks right. Let $z = \Pi_C(y)$ and choose any $x \in C$. Convexity makes $z + \theta(x - z)$ feasible for $0 \le \theta \le 1$. Because z minimizes squared distance, its one-sided directional derivative must be nonnegative:

$$\frac{d}{d\theta}\frac{1}{2}\|z + \theta(x - z) - y\|_2^2\Big|_{\theta=0} = (z - y)^\top (x - z) \ge 0.$$

Here y is the query, z its projection, x a feasible comparison point and θ the segment parameter. This gives the stated separation inequality. It is also sufficient: expanding the distance at x yields the distance at z, plus a nonnegative inner product and $\frac{1}{2}\|x - z\|_2^2$.

Uniqueness follows just as directly. If distinct z_1, z_2 both minimized squared distance, their midpoint would be feasible and have a strictly smaller value. Closedness supplies existence: a minimizing sequence stays in a bounded distance sublevel set and has a convergent subsequence whose limit remains in C. Thus nonempty, closed and convex each plays a specific role [1].

Interior, relative interior and hidden equalities

Plain version. A feasible set can have room to move within its own plane even if it has no room to move in the full surrounding space.

Consider probability vectors whose coordinates sum to one. They lie in a hyperplane, so their ordinary interior in the full vector space is empty. They can nevertheless have strictly positive coordinates and an interior relative to that hyperplane.

The affine hull is the smallest affine set containing C. The relative interior is the interior measured within that affine hull. This distinction matters in feasibility conditions and duality. Requiring a full-dimensional interior would incorrectly exclude many useful equality-constrained models.

Equality constraints can also be hidden inside inequalities. If both $a^\top x \leq b$ and $a^\top x \geq b$ are present, they force equality. Recognizing such structure can improve interpretation and numerical treatment.

Strict feasibility does not mean the same thing as having a small computed violation. It is a mathematical property requiring relevant inequalities to hold strictly while equalities remain satisfied. The duality and interior-point chapters specify exactly when this condition is needed.

Convexity of a problem requires both sets and functions

Plain version. Convex allowed choices are only half of the story. The objective must also have the right shape, and equality constraints must preserve that geometry.

A convex minimization problem uses a convex objective, convex inequality functions and affine equalities on their appropriate domains [1]. The next chapter defines convex functions and proves the rules used to recognize them.

A convex feasible set alone does not guarantee a convex optimization problem. Minimizing a function with many valleys over a box remains nonconvex. Conversely, a convex objective with disconnected discrete constraints remains nonconvex.

When the whole problem is convex, every local minimizer is global. This does not guarantee a unique minimizer: a linear objective can be constant along an entire face. Stronger objective properties, such as strict convexity on the feasible set, can establish uniqueness when a minimum exists.

Neither globality nor uniqueness guarantees numerical ease at every scale. A huge dense problem may require more memory than available. An extremely ill-conditioned model may be difficult at tight tolerance. Convexity supplies structural advantages, while complexity and implementation still matter.

Why a local minimum must be global

Convexity excludes a better distant answer by connecting it to the current point. Suppose x is a local minimizer of convex f on convex feasible set C, but some feasible y has $f(y) < f(x)$. For any

$0 < \theta \leq 1$, define $z_\theta = (1-\theta)x + \theta y$. Then

$$f(z_\theta) \leq (1-\theta)f(x) + \theta f(y) < f(x).$$

Here z_θ is a feasible segment point and θ its mixing weight. For arbitrarily small positive θ, the point lies arbitrarily close to x and improves the objective, contradicting local minimality. No differentiability is needed [1].

This proof gives globality, not uniqueness. For example, minimizing x_1 on $0 \leq x_1, x_2 \leq 1$ gives every point $(0, x_2)$ as a global minimizer. Adding a strictly convex objective makes distinct tied minimizers impossible, because their feasible midpoint would have lower cost. Existence remains separate: minimizing x on $x > 0$ is convex but has no minimizer.

These three questions guide solver expectations. Convexity makes local minima global; strict curvature can select one answer; closedness and objective growth can ensure an answer exists. A numerical procedure needs an appropriate convergence argument in addition to all three.

Modern view

Plain version. Modern convex optimization still begins with modeling and geometry. Recent methods change how a feasible set is accessed and how much computation is spent obtaining an accurate certificate.

Grant, Boyd and Ye's disciplined convex programming framework [2] organizes models through verified composition rules. It is a modeling methodology, not an independent guarantee that any convenient expression is convex. An algebraically convex expression may need reformulation so its convexity is visible to the rule system.

Pokutta's 2024 survey of Frank–Wolfe methods [3] distinguishes access through projection, barrier functions or a linear minimization oracle. Its useful lesson here is geometric: a feasible set that is easy to describe may still be expensive to project onto. Linear minimization can sometimes be cheaper, and convex combinations preserve feasibility. Later algorithm chapters explain those updates and their assumptions.

The PDLP report by Applegate and colleagues [4], first posted in 2025 and revised in 2026, develops a practical first-order linear-programming solver. It connects a primal–dual formulation with scaling, restarts and feasibility polishing. This is an original solver study, not a survey. We use it as evidence that implementation details and the desired accuracy remain important even for a familiar convex problem class.

Current challenges include representing complex constraints without excessive auxiliary structure, exploiting sparse matrices, making tolerances meaningful under scaling, and maintaining certificates when models are embedded in larger pipelines. A learned feature extractor can be nonconvex while a subsequent allocation problem is convex; describe each stage's guarantees separately.

These sources are a curated reading path, not a claim that every optimization survey has been reviewed. The core theory in this course follows [1], while the modern sections identify precisely which extensions each source supports.

Key takeaways

- Variables, fixed data, objectives, constraints and units must be explicit.
- Feasibility, boundedness, attainment and numerical termination are different questions.
- Convex sets contain all weighted averages with nonnegative weights summing to one.
- Affine sets, polyhedra, norm balls and important cones provide reusable building blocks.
- Intersections and affine transformations preserve convexity; unions generally do not.
- Projection onto a closed convex set is unique and supplies a useful geometric certificate.
- Convex optimization guarantees concern the formulated problem and its stated assumptions.

Exercises

1. Formulate allocating three nonnegative resources under a total budget, with squared deviation from a chosen target. State all units and distinguish variables from data.

   Hint

   Use a vector x, nonnegativity, one sum constraint and a fixed target vector.

2. Prove directly that the intersection of two convex sets is convex. Give a counterexample for their union.

   Hint

   A segment stays in both sets; two separated intervals disprove the union rule.

3. A system can select one of three configurations. Explain when a convex mixture is an actual operational choice and when it is only a relaxation.

   Hint

   Consider time sharing versus an indivisible hardware choice.

4. Derive projection onto a centered Euclidean ball. Identify what happens when the query point is already inside.

   Hint

   Outside the ball, the closest point lies on the query's radial direction.

5. Give a finite-valued convex problem whose optimum is not attained, and explain which existence condition fails.

Hint

An open interval can exclude its limiting optimum.

6. A solver reports constraint residual 10^{-6}. Explain why its adequacy cannot be judged without constraint scale and application units.

Hint

Rescaling the same inequality changes the raw residual.

References

1. Stephen Boyd and Lieven Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004. **Reading: Chapters 1–2; Sections 4.1–4.2 and 8.1 for problem terminology and projections.** Official book page, free official PDF.
2. Michael Grant, Stephen Boyd and Yinyu Ye, "Disciplined Convex Programming," in *Global Optimization: From Theory to Implementation*, Springer, 2006. Author's publication page.
3. Sebastian Pokutta, "The Frank-Wolfe Algorithm: A Short Introduction," *Jahresbericht der Deutschen Mathematiker-Vereinigung*, 126, 2024. Open-access survey.
4. David Applegate, Mateo Díaz, Oliver Hinder, Haihao Lu, Miles Lubin, Brendan O'Donoghue and Warren Schudy, "PDLP: A Practical First-Order Method for Large-Scale Linear Programming," arXiv preprint, 2025, revised 2026. Verified paper record.

OPT 02 • Convex Functions, Composition and Conjugates

Recognize convex objectives through chords, supporting planes and curvature; build valid compositions, use perspectives and conjugates, and distinguish quasiconvexity.

Online edition: https://blog.shyandram.dev/en/learn/optimization/opt-02-convex-functions/

What you'll learn

- Define convexity with domains, chords and epigraphs.
- Use first-order and second-order tests without assuming differentiability everywhere.
- Recognize important scalar, norm and matrix functions.
- Build convex models using sums, affine substitutions, maxima, composition and perspectives.
- Derive conjugates and understand their connection to linear lower bounds.
- Distinguish convex, strongly convex, quasiconvex and log-concave structure.

The big picture

The previous chapter described allowed choices. This chapter describes how their scores change. A convex objective has a useful consistency: mixing two choices cannot produce a score worse than the corresponding mixture of their scores. Its local supporting planes provide global lower bounds.

This does not mean every familiar loss is convex in every variable. Squared error is convex in the predicted values and in linear model coefficients. It is generally not convex in all parameters of a multilayer neural network. Always specify the variable being optimized and the domain being considered.

> **Explain like I'm five**
>
> Imagine a bowl and a straight stick joining two points on its rim. The bowl stays below the stick. A little flat board touching the bowl from underneath stays underneath everywhere. Those two pictures explain convex chords and supporting planes.

Boyd and Vandenberghe's Chapter 3 [1] develops several ways to recognize this structure. The goal is not to memorize a huge catalog. It is to combine a small catalog with valid transformation rules, so a complicated objective comes with a reason for being convex.

Convexity is a statement about the complete function, including its domain. A formula may have positive curvature wherever defined, while its domain is disconnected. That does not define a convex function on the whole set. Domain restrictions deserve the same attention as derivatives.

Chords, domains and epigraphs

Plain version. Draw the straight line between two function values. For a convex function, the actual value at every intermediate input is no higher than that line.

For a function $f : C \to \mathbb{R}$ on convex domain C,

$$f(\theta x + (1-\theta)y) \le \theta f(x) + (1-\theta)f(y), \qquad 0 \le \theta \le 1.$$

$x, y \in C$ are arbitrary inputs, θ is a mixing weight, and f is convex if this inequality holds for every such choice. Reversing the inequality defines concavity. An affine function satisfies equality and is both convex and concave.

The condition compares inputs and outputs simultaneously. Convexity of the domain ensures the mixed input is available. A function does not qualify by passing the inequality only for a convenient subset of endpoint pairs.

The epigraph is

$$\text{epi} f = \{(x,t) : x \in C,\, f(x) \le t\}.$$

t is a scalar vertical coordinate and (x,t) is a point above or on the function graph. A function is convex exactly when its epigraph is a convex set. This provides a bridge from Chapter 1's geometry to objectives and inequality constraints.

An epigraph formulation introduces a variable t and minimizes it subject to $f(x) \le t$. It can make a nonsmooth objective easier to express using constraints. It does not change the mathematical optimum when the formulation is exact.

Jensen's inequality and expectations

The two-point inequality extends to finite weighted averages:

$$f\left(\sum_i \theta_i x_i\right) \le \sum_i \theta_i f(x_i), \qquad \theta_i \ge 0, \quad \sum_i \theta_i = 1.$$

x_i are inputs and θ_i are weights. For a random vector X with suitable integrability and domain conditions, the analogous statement is $f(\mathbb{E}[X]) \le \mathbb{E}[f(X)]$.

This explains why a convex cost applied after averaging can be smaller than the average individual cost. It does not say that averaging arbitrary model parameters improves prediction: the prediction function and its loss must be convex in those parameters for that conclusion.

Jensen's inequality also helps reason about risk and uncertainty. A nonlinear transformation of an expected measurement need not equal the expected transformed measurement. The direction depends on convexity or concavity, not on a vague claim that averages are always representative.

Supporting planes: the first-order test

Plain version. For a differentiable convex function, the tangent approximation is a global lower bound, not merely a local sketch.

On an open convex domain, differentiable convexity is equivalent to

$$f(y) \geq f(x) + \nabla f(x)^{\top}(y - x).$$

x is the point where the approximation is made, y is any domain point, and $\nabla f(x)$ is the gradient with respect to the optimization variable. The affine expression on the right supports the function from below.

One way to derive the inequality is to apply the convex chord rule at $x + \theta(y - x)$, rearrange it, divide by positive θ and let θ tend to zero. This turns a global mixing condition into a directional derivative bound.

If $\nabla f(x) = 0$, the supporting-plane inequality gives $f(y) \geq f(x)$ everywhere in the domain. Thus a stationary point is globally minimizing in an unconstrained differentiable convex problem. It need not be unique.

With constraints, a nonzero gradient can still be optimal because improving directions are disallowed. The duality and constrained-algorithm chapters express that fact precisely. Simply requiring a small gradient is not an appropriate stopping rule for every constrained problem.

Nonsmooth supporting vectors

A corner does not destroy convexity. For convex f, a subgradient g at x satisfies

$$f(y) \geq f(x) + g^{\top}(y - x) \quad \text{for every } y.$$

g is a vector supplying a supporting affine lower bound; the set of all such vectors is the subdifferential $\partial f(x)$. At a differentiable point it contains only the gradient. At the origin, the absolute-value function has all scalar subgradients in $[-1, 1]$.

The notation is useful for absolute residuals, maxima and sparsity penalties. A subgradient is not an ordinary derivative chosen arbitrarily at a corner: it must satisfy a global support inequality. Later chapters use this structure when discussing proximal methods.

For a convex unconstrained function, $0 \in \partial f(x)$ implies global optimality. A zero gradient and a zero included among many subgradients therefore express the same underlying support idea in different settings.

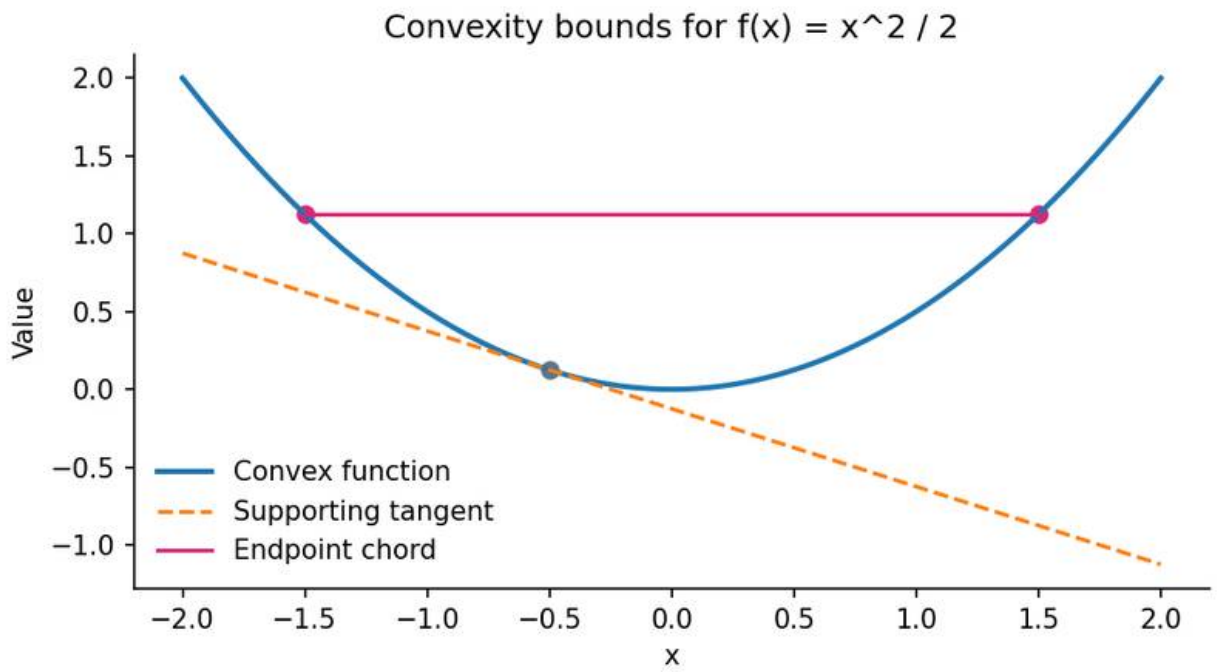


Figure 1 — Original analytical illustration for a scalar quadratic. Convex chords lie above the curve; its tangent at the selected point gives a global affine lower bound.

Curvature: the second-order test

Plain version. A smooth function is convex when it bends upward in every direction. Looking at only one coordinate at a time can miss a downward diagonal direction.

For twice continuously differentiable f on an open convex domain,

$$\nabla^2 f(x) \succeq 0 \quad \text{for every } x$$

is equivalent to convexity. $\nabla^2 f(x)$ is the symmetric Hessian matrix, and $\succeq 0$ means every directional quadratic form $v^\top \nabla^2 f(x) v$ is nonnegative.

Nonnegative diagonal Hessian entries are insufficient. The function $f(x_1, x_2) = x_1 x_2$ has zero coordinate second derivatives but an indefinite Hessian. Along one diagonal it curves upward, along the other downward.

The test is a global condition on the entire domain. Evaluating eigenvalues at ten points is a numerical diagnostic, not a proof. A symbolic Hessian with an analytical positive-semidefinite factorization can establish the property for all points.

Strict and strong convexity

Strict convexity requires a strict chord inequality for distinct endpoints and weights strictly between zero and one. When a minimum exists on a convex set, strict convexity ensures uniqueness.

Strong convexity in the Euclidean norm requires, for some $\mu > 0$,

$$f(y) \geq f(x) + \nabla f(x)^\top (y - x) + \frac{\mu}{2}\|y - x\|_2^2.$$

μ is a uniform curvature lower bound. For twice differentiable functions, it corresponds to $\nabla^2 f(x) \succeq \mu I$, where I is the identity matrix. Strong convexity is stronger than strict convexity; a function can be strictly convex without a positive uniform curvature bound.

Adding a positive quadratic regularizer often supplies strong convexity in the penalized coordinates. An unpenalized intercept or a restricted domain requires separate analysis. Do not infer strong convexity of every parameter merely from one regularization term.

What the Hessian test measures

A multivariable curvature test becomes a scalar test along every line. Fix a point x and direction v, and set $q(t) = f(x + tv)$ on the interval that stays in the domain. Differentiating twice gives

$$q''(t) = v^\top \nabla^2 f(x + tv) v.$$

Here t is the line coordinate, q the restricted scalar function, and v any direction. A positive-semidefinite Hessian makes every such restriction convex. Every segment is part of one of these lines, so the chord definition follows. Conversely, convexity on every line requires its second derivative to be nonnegative. This explains why the test examines quadratic forms in all directions rather than individual matrix entries [1].

Strict and strong convexity differ in a concrete example. The scalar function $f(x) = x^4$ is strictly convex on $\mathbb{R}$: its derivative $4x^3$ is strictly increasing. Yet $f''(0) = 0$, so no positive global μ satisfies the strong-convexity Hessian bound. Its minimum is unique, but the strong-convexity stopping bound in OPT 06 is unavailable. Adding $\frac{\mu}{2}x^2$, with chosen $\mu > 0$, supplies the missing curvature floor and changes the objective.

A catalog of useful functions

Plain version. A few familiar building blocks cover a large share of practical convex objectives. Their domains and signs determine how they can be used.

Affine functions $a^\top x + b$ are both convex and concave. Norms are convex because of positive homogeneity and the triangle inequality. Squared Euclidean norm is convex and smooth; the norm itself is nonsmooth at the origin.

A quadratic

$$f(x) = \frac{1}{2}x^\top Q x + c^\top x + d$$

is convex exactly when symmetric $Q \succeq 0$. Here Q sets curvature, c is a linear coefficient vector and d is a scalar constant. Positive-definite Q gives strong convexity with a bound set by its smallest eigenvalue.

Scalar exponential is convex everywhere. Negative logarithm is convex on positive inputs. Powers need their exponent and domain specified; statements such as “all power functions are convex” are false. For example, square root is concave on nonnegative inputs, while its negative is convex there.

The maximum coordinate and maximum of affine scores are convex and generally nonsmooth. A minimum of affine functions is concave, not generally convex. The sign of a modeling expression is therefore important.

Matrix-valued domains

The function $-\log\det X$ is convex for positive-definite symmetric matrices X [1, Section 3.1]. The variable is a matrix, and the domain excludes singular matrices.

For a symmetric perturbation H, its second directional derivative is

$$\operatorname{tr}(X^{-1}HX^{-1}H) \geq 0.$$

H is a direction in the space of symmetric matrices and tr is trace. After a congruence transformation this becomes a squared Frobenius norm, explaining the nonnegative curvature.

The largest eigenvalue of a symmetric matrix is convex because

$$\lambda_{\max}(X) = \sup_{\|v\|_2=1} v^\top X v.$$

$\lambda_{\max}$ denotes the largest eigenvalue and v ranges over unit vectors. Each quadratic form is affine in the matrix entries of X, so their pointwise supremum is convex. This is a useful example of analyzing which quantity is the variable.

Code: Hessian checks for a quadratic and a counterexample

This code checks constant Hessians, so the eigenvalue test applies at every input. It also verifies one supporting-plane inequality numerically.

```
import numpy as np

Q = np.array([[3.0, 1.0], [1.0, 2.0]])
bad = np.array([[0.0, 1.0], [1.0, 0.0]])
assert np.linalg.eigvalsh(Q).min() > 0
assert np.linalg.eigvalsh(bad).min() < 0
x, y = np.array([0.4, -0.6]), np.array([-1.0, 0.8])
f = lambda z: 0.5 * z @ Q @ z
```

```
support = f(x) + (Q @ x) @ (y - x)
gap = f(y) - support
assert np.isclose(gap, 0.5 * (y - x) @ Q @ (y - x))
print("quadratic Hessian eigenvalues:", np.linalg.eigvalsh(Q))
print("product Hessian eigenvalues:", np.linalg.eigvalsh(bad))
print("supporting-plane gap:", gap)
```

Sums, substitutions and pointwise maxima

Plain version. Build a complicated objective from known functions through operations that preserve convexity. The preservation rule is the proof.

A nonnegative weighted sum of convex functions is convex. Negative coefficients generally reverse or destroy the property. An integral or expectation of convex functions is convex when the result is well-defined and the distribution is fixed independently of the decision variable.

Affine substitution preserves convexity:

$$g(x) = f(Ax + b).$$

f is convex, A is a fixed matrix and b a fixed vector. The domain of g contains points whose transformed values lie in the domain of f. This rule establishes convexity of squared residuals, norm residuals and many regularized linear estimators.

If f_i are convex, then $g(x) = \max_i f_i(x)$ is convex on their common domain. An epigraph variable t expresses this as $f_i(x) \le t$ for every i. This models the worst residual, worst scenario cost or maximum coordinate without needing a smooth objective.

An infimum over a jointly convex auxiliary variable also preserves convexity under the appropriate formulation:

$$g(x) = \inf_y F(x, y).$$

F is jointly convex in (x, y) and y is the eliminated variable. Joint convexity matters; convexity in each block separately is insufficient. This rule connects partial minimization with the projection of an epigraph.

What does not automatically preserve convexity

Products of convex functions are not generally convex. Dividing one convex function by another is not generally convex. Taking a minimum of convex functions can create multiple valleys.

For example, x^2y^2 is not jointly convex merely because it is a product of two nonnegative convex expressions. A reciprocal or exponential outside an expression needs a composition argument with the right monotonicity. Mathematical plausibility is not a substitute for checking a rule.

Multiplying a fixed parameter by a variable is affine. Multiplying two decision variables is bilinear and generally nonconvex. Many modeling errors arise from accidentally treating a learned or

unknown quantity as if it were fixed.

Composition with monotonicity

Plain version. An outer convex function can preserve convexity when it reacts to each inner argument in the right direction.

Suppose

$$F(x) = h(g_1(x), \dots, g_k(x)).$$

h is an outer function and g_i are scalar inner functions. A standard sufficient rule is: h is convex, and for each argument it is nondecreasing when g_i is convex, nonincreasing when g_i is concave, or the inner function is affine. Domain conditions and the appropriate extended-value interpretation are included in the rule [1, Section 3.2].

For instance, exponential is convex and increasing, so exponentiating a convex inner expression preserves convexity. Negative logarithm is convex and decreasing, so applying it to a positive concave inner expression gives a convex function.

The monotonicity qualifier cannot be omitted. Squaring a convex expression is not always convex because square is decreasing on negative inputs and increasing on positive ones. Squaring a nonnegative convex expression is a valid composition, while arbitrary signs require another argument.

These are sufficient recognition rules, not a complete decision procedure for every algebraic expression. An expression may be convex even though its current syntax fails the rules. Rewriting $\sqrt{1 + x^2}$ as a Euclidean norm exposes convexity directly.

A composition rule seen through derivatives

For scalar smooth functions, the chain rule shows where the monotonicity requirement enters. Set $F(x) = h(g(x))$. Then

$$F''(x) = h''(g(x))[g'(x)]^2 + h'(g(x))g''(x).$$

Here primes denote scalar derivatives, g is the inner function and h the outer function. If h is convex and nondecreasing and g is convex, both terms are nonnegative. If h is convex and nonincreasing while g is concave, the second term is again nonnegative. Composition rules also handle nonsmooth functions, but this calculation exposes the signs [1].

Take the convex inner expression $g(x) = x^2 - 1$ and the convex outer function $h(t) = t^2$. Their composition is $(x^2 - 1)^2$, whose second derivative is $12x^2 - 4$. It is negative at zero, so the composition is nonconvex. The inner output is negative near zero, where the square function is decreasing; the required sign agreement fails.

When the domain is convex and a convex inner expression is nonnegative throughout it, squaring is valid because the outer function is nondecreasing there. This is why squared Euclidean residual

norms are convex but “square any convex loss expression” is not a valid general rule.

Log-sum-exp and stable evaluation

Plain version. Log-sum-exp is a smooth upper approximation to the largest score. It also provides the normalization behind probability-like weights.

For $x \in \mathbb{R}^n$,

$$f(x) = \log\left(\sum_{i=1}^{n} e^{x_i}\right), \qquad p_i = \frac{e^{x_i}}{\sum_j e^{x_j}}.$$

x_i are scalar coordinates; p_i are normalized positive weights summing to one. Its gradient is p and its Hessian is

$$\nabla^2 f(x) = \operatorname{diag}(p) - pp^\top.$$

$\operatorname{diag}(p)$ places the weights on a diagonal. For any direction v, the quadratic form is the weighted variance of v_i, which is nonnegative. This proves convexity.

Adding the same constant to every coordinate shifts the function by that constant. The all-ones direction therefore has zero Hessian curvature; the function is not strongly convex on the entire space.

For temperature $\tau > 0$, define $f_\tau(x) = \tau \log \sum_i e^{x_i/\tau}$. Then

$$\max_i x_i \le f_\tau(x) \le \max_i x_i + \tau \log n.$$

τ controls smoothness and the approximation error. A small temperature approaches the maximum but can make numerical optimization less well-conditioned. Stable evaluation subtracts the largest scaled coordinate before exponentiating.

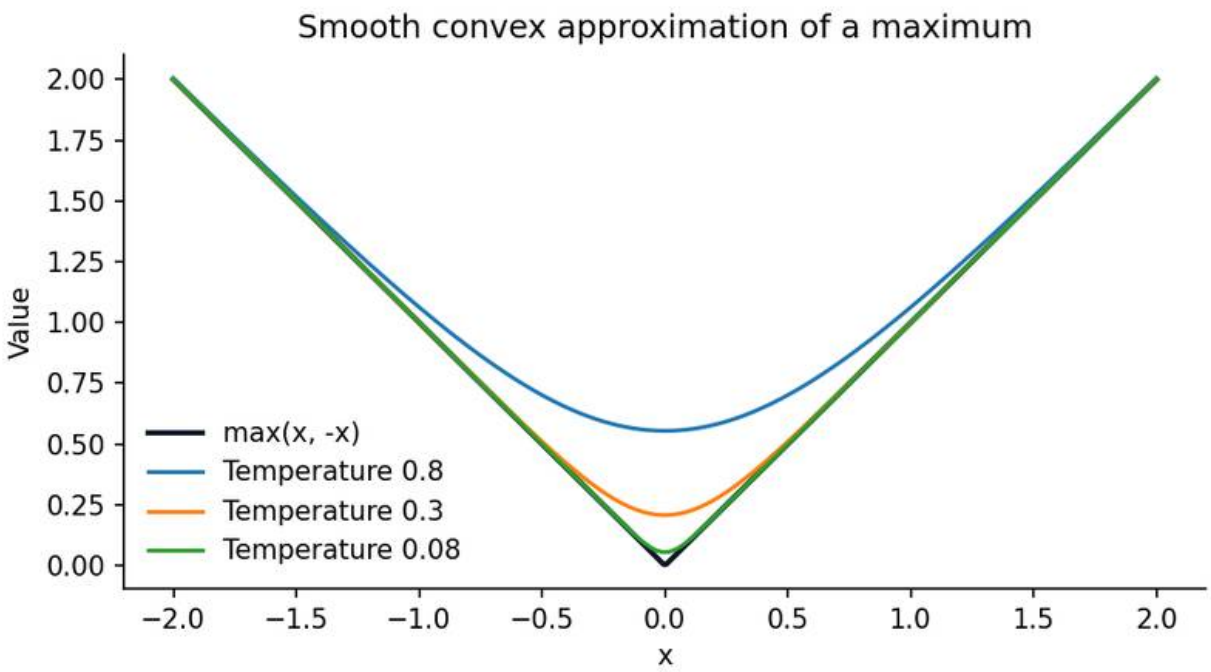


Figure 2 — Original analytical illustration. Temperature-scaled log-sum-exp lies above the maximum of the scores; smaller temperature gives a closer but sharper approximation.

Code: stable log-sum-exp and a Jensen check

This example avoids overflow at large inputs, verifies the Hessian and illustrates one finite Jensen comparison.

```
import numpy as np
from scipy.special import logsumexp

x = np.array([1000.0, 999.0, 997.0])
value = logsumexp(x)
p = np.exp(x - value)
H = np.diag(p) - np.outer(p, p)
assert np.linalg.eigvalsh(H).min() >= -1e-12
assert np.isclose(p.sum(), 1)
u, v = np.array([1., -1., 0.]), np.array([-2., 2., 1.])
theta = 0.3
left = logsumexp(theta * u + (1 - theta) * v)
right = theta * logsumexp(u) + (1 - theta) * logsumexp(v)
assert left <= right + 1e-12
print("stable value:", value, "weights:", p)
print("Jensen gap:", right - left)
```

Perspectives: normalize a variable without losing convexity

Plain version. A perspective combines a cost at a normalized input with the amount of scale used. This creates useful convex expressions that look like ratios.

For convex f, its perspective is

$$g(x,t) = tf(x/t), \qquad t > 0, \quad x/t \in \operatorname{dom} f.$$

x is a vector, t a positive scalar scale and g is jointly convex in both variables. The positive-scale domain is essential.

Taking $f(u) = \|u\|_2^2$ gives $g(x,t) = \|x\|_2^2/t$. This quadratic-over-linear function is jointly convex, even though arbitrary ratios are not.

The rule can be proved by mixing two pairs (x_1,t_1) and (x_2,t_2), then recognizing that their normalized input is a convex combination with weights proportional to θt_1 and $(1-\theta)t_2$. Apply convexity of f and multiply by the combined positive scale.

Perspective formulations arise when cost depends on a normalized rate, allocated duration or positive mass. They can also reveal exact conic representations. Keep physical meaning and domain constraints visible: permitting zero or negative scale without a careful closed extension changes the function.

Perspective proof: the weights change after normalization

Ordinary averaging does not commute with division by a variable scale. The perspective remains convex because normalization produces a different, still valid pair of weights.

Take two domain points $(x_1,t_1),(x_2,t_2)$ with $t_1,t_2 > 0$ and mixing weight $\theta \in [0,1]$. Define

$$\bar{x} = \theta x_1 + (1-\theta)x_2, \qquad \bar{t} = \theta t_1 + (1-\theta)t_2,$$
$$\beta = \frac{\theta t_1}{\bar{t}}, \qquad \frac{\bar{x}}{\bar{t}} = \beta\frac{x_1}{t_1} + (1-\beta)\frac{x_2}{t_2}.$$

Here $\bar{x},\bar{t}$ are the mixed numerator and scale; $\beta \in [0,1]$ is the normalized weight. Apply convexity of f to the last line and multiply by $\bar{t} > 0$. The coefficients become $\bar{t}\beta = \theta t_1$ and $\bar{t}(1-\beta) = (1-\theta)t_2$, giving exactly the convex chord bound for $g(x,t) = tf(x/t)$ [1].

For the quadratic-over-linear example, take $(x_1,t_1) = (2,1)$ and $(x_2,t_2) = (2,3)$ with $\theta = 1/2$. Their costs are 4 and $4/3$. The mixed point $(2,2)$ costs 2, below their average $8/3$. Its normalized input is 1, obtained with weight $\beta = 1/4$ on the first normalized input 2 and $3/4$ on the second $2/3$. The scale-dependent weights are the key step; simply averaging the normalized inputs would give a different number.

Conjugates and linear lower bounds

Plain version. A conjugate asks how much an arbitrary linear score can exceed a function. The answer describes the tightest intercept for a supporting line with that slope.

The convex conjugate is

$$f^*(y) = \sup_{x \in \operatorname{dom} f} \{y^\top x - f(x)\}.$$

y is a candidate slope vector, x is the original variable and $f^*(y)$ is the supremum gap. A finite conjugate value yields the global lower bound

$$f(x) \geq y^\top x - f^*(y).$$

This follows immediately from the supremum definition and is the Fenchel inequality [1, Section 3.3]. Conjugates are useful for deriving dual problems and splitting algorithms.

For $f(x) = \frac{1}{2} x^\top Q x$ with positive-definite Q, maximizing the gap gives $Qx = y$. Therefore

$$f^*(y) = \frac{1}{2} y^\top Q^{-1} y.$$

Q is the quadratic curvature matrix. In numerical code solve $Qx = y$ rather than explicitly inverting Q.

The conjugate of an indicator of a set is its support function. An extended-value indicator is zero on the allowed set and positive infinity elsewhere. This makes a constraint part of an objective without using a finite penalty.

For proper closed convex functions, conjugating twice recovers the original function. These regularity conditions matter: without them, the double conjugate describes the appropriate closed convex envelope rather than blindly reproducing every original formula.

Code: a conjugate certificate

Equality holds in the quadratic Fenchel bound at the matching slope $y = Qx$. The code checks both equality and a nonnegative gap elsewhere.

```
import numpy as np

Q = np.array([[4., 1.], [1., 2.]])
x = np.array([0.5, -0.8])
y = np.array([1.2, 0.3])
f = lambda z: 0.5 * z @ Q @ z
conjugate = lambda slope: 0.5 * slope @ np.linalg.solve(Q, slope)
gap = f(x) + conjugate(y) - x @ y
matching = Q @ x
assert gap >= -1e-12
assert np.isclose(f(x) + conjugate(matching), x @ matching)
```

```
print("Fenchel gap:", gap)
print("matching slope:", matching)
```

A nonsmooth conjugate with a visible domain

The conjugate can turn a nonsmooth penalty into a simple restriction on slopes. Let $f(x) = \lambda\|x\|_1$ with fixed $\lambda > 0$. Separating the coordinates in its definition gives

$$f^*(y) = \sum_i \sup_{x_i \in \mathbb{R}} (y_i x_i - \lambda|x_i|) = \begin{cases} 0, & \|y\|_\infty \le \lambda, \\ +\infty, & \|y\|_\infty > \lambda. \end{cases}$$

Here x_i, y_i are coordinates of the original variable and slope; $\|y\|_\infty = \max_i |y_i|$. When $|y_i| \le \lambda$, each coordinate expression is at most zero, attained at $x_i = 0$. When one slope exceeds that threshold in magnitude, choosing x_i with its sign and increasing magnitude makes the expression unbounded.

Thus the domain of the conjugate is an infinity-norm ball. The corresponding lower bound $y^\top x \le \lambda\|x\|_1$ is tight when nonzero coordinates have matching slopes $y_i = \lambda \operatorname{sign}(x_i)$. This links norm geometry, subgradients and dual feasibility in one calculation [1, Section 3.3]. OPT 08 uses the same threshold structure to derive a proximal update.

Quasiconvexity and logarithmic structure

Plain version. Some functions are not convex, but every set of inputs below a chosen score is convex. Other functions become convex or concave after taking a logarithm. These are useful structures with different guarantees.

A function is quasiconvex if all sublevel sets are convex:

$$C_\alpha = \{x : f(x) \le \alpha\}.$$

α is any real threshold and C_α is its sublevel set. Equivalently, values at mixed inputs do not exceed the larger endpoint value. This is weaker than the convex chord inequality.

Every convex function is quasiconvex. A monotone scalar function on an interval is quasiconvex even when its curvature is not consistently upward. Under appropriate formulations, quasiconvex minimization can use bisection on a threshold and solve a convex feasibility problem at each step [1, Sections 3.4 and 4.2].

For positive f, log-convexity means $\log f$ is convex; log-concavity means $\log f$ is concave. Positivity and the domain are part of these definitions. A density with concave log density has log-concave structure, but that does not make every parameter-estimation problem involving it convex in every parameterization.

Some geometric programs become convex after substituting logarithms of positive variables. This is an exact variable transformation in a specified problem class, not permission to take logarithms of arbitrary objectives and constraints. The next chapter derives the relevant form.

Convexity of a formula versus convexity of an implementation

A mathematically convex expression can be evaluated poorly. Overflow in an exponential, rounding near a matrix boundary or an accidentally broadcast array can invalidate the computed quantity while leaving the intended function convex.

Keep a symbolic curvature argument and a numerical evaluation check as separate evidence. A Hessian sampled at a few points may reveal a counterexample, but cannot prove nonnegative curvature everywhere. A stable implementation likewise does not prove the intended expression is convex.

When parameters are fixed during a solve, the convexity claim concerns decision variables at those fixed parameter values. A coefficient changing sign can invalidate a composition rule. Record which parameter ranges preserve the claim, rather than calling an entire parameterized family convex without qualification.

The same distinction applies to a learned function. Convexity in an input used later as a decision variable does not establish convexity in the weights being trained. Curvature rules describe one optimization problem at a time.

Modern view

Plain version. Convexity-preserving design is still useful when functions are learned from data. A learned function can be constrained to remain convex in its decision input without being convex in its training parameters.

Disciplined convex programming [2] formalizes composition rules so a model carries a recognizable convexity argument. The taxonomy separates affine, convex and concave expressions, tracks monotonicity and respects domains. Failing a syntactic rule can mean a reformulation is needed; it does not automatically disprove mathematical convexity.

Schaller, Bemporad and Boyd's 2025 manuscript on learning parametric convex functions [3] studies fitting functions that remain convex in the variable for valid parameter values and are compatible with disciplined programming. It supports a specific modern connection: learning a component of an optimization model while preserving decision-variable convexity. It is an original research manuscript, not a review or a guarantee about unrestricted neural losses.

Pokutta's 2024 survey [4] organizes constrained smooth methods around geometric access and uses convexity, smoothness and strong convexity to analyze updates. Its practical lesson is to keep curvature assumptions separate: global convexity, uniform lower curvature and an upper smoothness bound play different roles.

Open problems include choosing useful structured function classes without losing expressiveness, certifying domains under noisy learned parameters, handling nondifferentiable solution maps, and evaluating generalization of learned models. An optimization certificate proves a property of the fitted objective; it does not prove that the objective matches future observations.

Key takeaways

- Convexity includes a convex domain and a chord inequality.
- Tangents and subgradients give global lower bounds.
- Positive-semidefinite Hessians establish smooth convexity in every direction.
- Strict convexity, strong convexity and smoothness are different assumptions.
- Composition rules need monotonicity, signs and domains.
- Perspectives and conjugates expose useful structure hidden in ratios and constraints.
- Quasiconvex and log-convex formulations require their own reasoning.

Exercises

1. Prove the first-order supporting-plane inequality from the chord definition for differentiable functions.

   Hint

   Rearrange the chord inequality, divide by the mixing weight and take its limit to zero.

2. Show why nonnegative Hessian diagonal entries do not prove convexity, using a two-variable example.

   Hint

   Use $x_1 x_2$ and test diagonal directions.

3. Reformulate $\sqrt{1 + x^2}$ to make its convexity visible through a norm.

   Hint

   Write the norm of $(1, x)$.

4. Derive the perspective of squared Euclidean norm and identify its domain.

   Hint

   Substitute x/t and multiply by positive t.

5. Derive the conjugate of $\frac{1}{2}\|x\|_2^2$ and find when Fenchel's inequality is equality.

   Hint

   Maximize the quadratic gap by differentiation.

6. Give a quasiconvex function on an interval that is not convex, and verify its sublevel sets.

Hint

A monotone concave function such as square root on positive inputs has interval sublevel sets.

References

1. Stephen Boyd and Lieven Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004. **Reading: Chapter 3; Section 4.2 for quasiconvex feasibility.** Official book page, free official PDF.
2. Michael Grant, Stephen Boyd and Yinyu Ye, "Disciplined Convex Programming," in *Global Optimization: From Theory to Implementation*, Springer, 2006. Author's publication page.
3. Michael Schaller, Alberto Bemporad and Stephen Boyd, "Learning Parametric Convex Functions," manuscript, June 2025. Verified manuscript page.
4. Sebastian Pokutta, "The Frank-Wolfe Algorithm: A Short Introduction," *Jahresbericht der Deutschen Mathematiker-Vereinigung*, 126, 2024. Open-access survey.

OPT 03 • Linear, Quadratic, Conic and Geometric Programs

Recognize standard convex problem families, derive exact epigraph and cone reformulations, transform geometric programs, and inspect scaling and multiobjective tradeoffs.

Online edition: https://blog.shyandram.dev/en/learn/optimization/opt-03-convex-programs/

What you'll learn

- Identify linear, quadratic and convex quadratically constrained programs.
- Turn absolute values, maxima and norm constraints into exact formulations.
- Express second-order and semidefinite cone constraints with the right ordering.
- Transform a geometric program into a convex problem using positive variables and logarithms.
- Separate exact reformulation, convex relaxation and approximation.
- Explain scaling, feasibility checks and multiobjective tradeoffs before selecting a solver.

The big picture

A model can be convex while its written form hides that fact. A sum of absolute residuals looks nonsmooth but becomes a linear program with auxiliary variables. A norm bound becomes a cone constraint. A positive-variable design can become convex after a logarithmic change of variables.

Boyd and Vandenberghe's Chapter 4 [1] organizes these problem families. Learning them serves two purposes: it supplies reusable modeling patterns and helps choose numerical methods that exploit structure. The class is determined by equations and domains, not by the label attached to an application.

> **Explain like I'm five**
>
> Different puzzles use different boxes of pieces. Sometimes a puzzle looks unfamiliar because its pieces are turned sideways. Rewriting the rules can reveal that it belongs to a familiar box. But adding new pieces changes the puzzle, so distinguish a rewrite from a relaxation.

The examples in this chapter use synthetic small problems. They show exact mathematical relationships and numerical checks, not benchmark results. The code uses the installed numerical libraries without introducing a separate modeling-system dependency.

The convex standard form

Plain version. Convexity requires the objective, inequalities and equalities to fit together. It is not enough for the objective alone to be convex.

The standard form is

$$\begin{aligned} \text{minimize}_x \quad & f_0(x) \\ \text{subject to} \quad & f_i(x) \le 0, \quad i = 1, \dots, m, \\ & Ax = b. \end{aligned}$$

$x \in \mathbb{R}^n$ is the decision vector; $f_0, \dots, f_m$ are convex functions on their relevant common domain; A is a fixed matrix and b a fixed equality target. The feasible set is convex because convex sublevel sets and affine equalities are intersected.

A concave function can appear on the other side: $g(x) \ge a$ is convex-feasible when g is concave, because $a - g(x) \le 0$ is a convex inequality. Equality to a nonlinear convex function is generally not convex. For example, $\|x\|_2^2 = 1$ describes a sphere surface, while $\|x\|_2^2 \le 1$ describes a convex ball.

Writing an equality as two inequalities does not fix this issue. One of the two directions reverses the curvature. If the physical problem truly requires the surface, replacing it by a ball is a relaxation and may change the answer.

Exact reformulation versus changing the problem

An exact reformulation preserves the achievable original decisions and objective after auxiliary variables are eliminated or a one-to-one transformation is reversed. A relaxation enlarges the allowed set. An approximation changes functions or constraints to nearby ones.

For an epigraph variable t, minimizing t with $f(x) \le t$ is exact because the optimizer can choose the smallest permitted t. Merely imposing the inequality inside another model may not force equality if the remaining objective does not prefer smaller t.

When a reformulation adds variables, prove both directions: every original feasible point can be extended to a reformulated point, and every reformulated point maps back to an original feasible decision with the correct objective relation. This small proof prevents many modeling errors.

Linear programming

Plain version. A linear program uses straight-line costs and rules. Its geometry is a polyhedron, which can have corners, faces and directions extending to infinity.

An inequality-and-equality linear program is

$$\begin{aligned} \text{minimize}_x \quad & c^\top x \\ \text{subject to} \quad & Gx \le h, \qquad Ax = b. \end{aligned}$$

c is the fixed objective coefficient vector; G, h define inequality bounds; A, b define equalities. Bounds on individual variables can be included separately or as rows of G.

A linear objective is both convex and concave. The restrictions need not force a unique solution. When an objective is parallel to a feasible face, every point on that face can be optimal.

A standard nonnegative form writes $Ax = b$, $x \geq 0$. Introduce nonnegative slack variables to turn inequalities into equalities, and represent a free scalar as the difference of two nonnegative scalars if this form is needed.

These transformations are exact but can increase size and change numerical conditioning. A solver interface may accept bounds and inequalities directly, so there is no need to expand everything manually for its own sake.

Absolute residuals become linear constraints

To fit measurements b using design matrix A, consider

$$\min_x \|Ax - b\|_1.$$

A is a fixed design matrix, x its coefficient vector and b the measurement vector. Introduce one variable t_i for each residual:

$$-t \leq Ax - b \leq t, \qquad \min_{x,t} \mathbf{1}^\top t.$$

t bounds residual magnitudes and $\mathbf{1}$ is the all-ones vector. Each feasible t_i is at least the absolute residual, and minimizing their sum tightens the bounds. Nonnegativity of t follows from the paired inequalities but can be specified explicitly.

Minimax fitting instead introduces one scalar t and minimizes it subject to $-t\mathbf{1} \leq Ax - b \leq t\mathbf{1}$. This controls the largest absolute residual rather than their sum. The modeling change expresses a different application preference.

Code: solve a small allocation LP

The SciPy interface is documented in [2]. This example checks success and feasibility independently, rather than printing the optimizer without checking its status.

```
import numpy as np
from scipy.optimize import linprog

c = np.array([-3.0, -2.0])
G = np.array([[1.0, 1.0], [2.0, 1.0]])
h = np.array([4.0, 5.0])
result = linprog(c, A_ub=G, b_ub=h, bounds=[(0, None), (0, None)])
assert result.success, result.message
x = result.x
assert np.all(G @ x <= h + 1e-8)
```

```
assert np.all(x >= -1e-8)
assert np.allclose(x, [1, 3], atol=1e-7)
print("allocation:", x, "objective:", c @ x)
print("slacks:", h - G @ x)
```

The negative objective coefficients encode maximizing a positive benefit through minimization. The problem is bounded because the capacity constraints restrict nonnegative choices. Removing a relevant bound can make the model unbounded; that is a modeling outcome, not a solver defect.

Quadratic programming and convex QCQPs

Plain version. A quadratic program replaces the linear cost with a curved bowl while keeping linear rules. Quadratic inequality constraints are also possible if their bowls face upward.

A convex quadratic program, QP, has objective

$$f_0(x) = \frac{1}{2}x^\top Qx + c^\top x + d, \qquad Q \succeq 0,$$

with affine constraints. Q is a symmetric positive-semidefinite curvature matrix, c is a linear coefficient vector and d a constant that does not change the optimizer.

Least squares is a QP: expanding $\frac{1}{2}\|Ax - b\|_2^2$ gives $Q = A^\top A$ and $c = -A^\top b$. A quadratic regularizer adds positive diagonal curvature. If the design matrix is rank deficient, the unregularized minimizer may not be unique.

A convex quadratically constrained quadratic program, QCQP, also has inequalities

$$\frac{1}{2}x^\top Q_i x + a_i^\top x + r_i \leq 0, \qquad Q_i \succeq 0.$$

Q_i is each constraint's curvature matrix, a_i a linear coefficient vector and r_i its offset. Arbitrary quadratic constraints do not meet this condition. An indefinite quadratic constraint can be nonconvex even if the objective is strongly convex.

Numerical structure matters

An unconstrained positive-definite QP solves $Qx = -c$. A constrained QP may have an optimizer at a boundary, so solving the unconstrained system and clipping afterward is generally wrong.

Clipping is correct for projection onto a box in ordinary Euclidean distance. A general quadratic couples coordinates; adjusting one coordinate changes the best value of others. The correct constrained solution must respect that coupling.

The size and sparsity of Q matter. Forming $A^\top A$ explicitly can destroy useful sparsity or worsen conditioning. A model can be written as a quadratic while its implementation retains residual operators. The algorithms chapters explain factorization and matrix-vector-product choices.

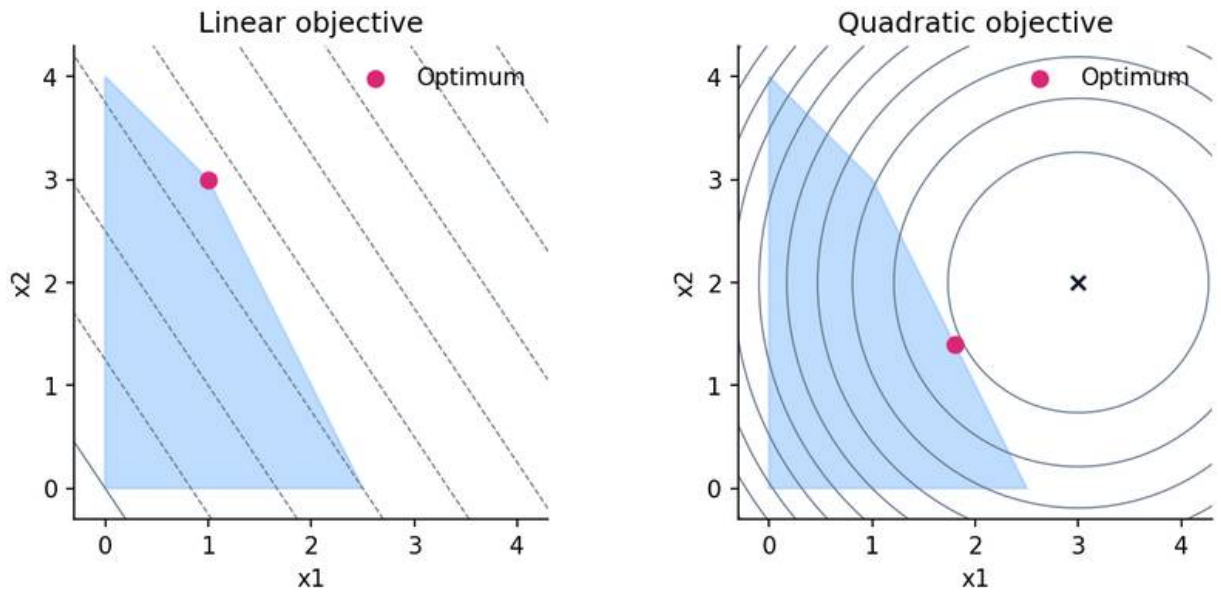


Figure 1 — Original analytical comparison. A linear cost favors a polygon corner in this example; a quadratic cost selects the closest point to its target. Both use the same convex feasible set.

Worked counterexample: why clipping a QP is wrong

Coupled curvature can make changing one coordinate shift the best value of another. Consider the convex quadratic

$$f(x) = \frac{1}{2}(x-a)^\top Q(x-a), \qquad Q = \begin{bmatrix} 2 & 1 \\ 1 & 2 \end{bmatrix}, \quad a = \begin{bmatrix} 2 \\ 0 \end{bmatrix}, \quad x_1 \le 1.$$

Here $x \in \mathbb{R}^2$ is the variable, a the unconstrained target and Q the positive-definite curvature matrix. Simply clipping a produces $(1,0)$ with objective one. With $x_1 = 1$, however, stationarity in the unrestricted second coordinate gives $(x_1 - 2) + 2x_2 = 0$, hence $x_2 = 1/2$.

The candidate $(1, 1/2)$ has objective $3/4$. Its gradient is $(-3/2, 0)$, balanced by multiplier $3/2$ on $x_1 - 1 \le 0$. Feasibility, multiplier sign, stationarity and complementary slackness therefore verify the optimum using OPT 04's conditions. Coordinate clipping ignores the off-diagonal entries of Q; Euclidean box projection works because its quadratic separates by coordinate.

Second-order cone programming

Plain version. A second-order cone expresses a norm bound with a scalar on the right. It turns many curved constraints into a standard geometric form.

A common SOCP inequality is

$$\|A_i x + b_i\|_2 \le c_i^\top x + d_i.$$

A_i, b_i define the vector inside the norm; c_i, d_i define the affine scalar bound; and x is the decision vector. The inequality automatically requires its right-hand side to be nonnegative.

Do not square both sides without retaining that sign condition. The squared inequality can allow a negative right-hand side and introduce extra points. The cone formulation carries the sign correctly.

A rotated second-order cone can encode quadratic-over-linear expressions. In a common convention, it requires $u, v \geq 0$ and $2uv \geq \|z\|_2^2$. The factor of two is a convention and must match the implementation.

Using $u = t$, $v = 1/2$, $z = Ax - b$ expresses $\|Ax - b\|_2^2 \leq t$. Different scaling choices give the same mathematical set but can change numerical behavior.

Robust linear constraints

Suppose a constraint coefficient is uncertain: $a = \bar{a} + Eu$ with $\|u\|_2 \leq 1$. Requiring $a^\top x \leq b$ for every allowed u gives

$$\bar{a}^\top x + \|E^\top x\|_2 \leq b.$$

$\bar{a}$ is the nominal coefficient vector, E maps unit-ball uncertainty and b is a fixed bound. The norm is the maximum of $u^\top E^\top x$ over the unit ball.

This is an exact robust counterpart for the specified uncertainty set. It does not guarantee protection against all real-world uncertainty; the set itself must be justified. A larger set can be more conservative, while an incorrect set can miss important disturbances.

The geometry of uncertainty determines the reformulation. Box uncertainty gives a one-norm term; ellipsoidal uncertainty gives a Euclidean norm after a transformation. These connections follow from support functions developed in the previous chapter [1].

Derive the robust counterpart rather than guessing a penalty

The robust norm term is the exact worst allowed disturbance. Expand the uncertain linear constraint:

$$\sup_{\|u\|_2 \leq 1} (\bar{a} + Eu)^\top x = \bar{a}^\top x + \sup_{\|u\|_2 \leq 1} u^\top E^\top x = \bar{a}^\top x + \|E^\top x\|_2.$$

Here x is the decision, $\bar{a}$ the nominal coefficient, E the uncertainty map and u an allowed disturbance. The last equality follows from the inner-product norm bound, with equality at $u = E^\top x / \|E^\top x\|_2$ when the denominator is nonzero. When it is zero, every disturbance contributes zero.

For the synthetic constraint $(1 + 0.2u_1)x_1 + (2 + 0.1u_2)x_2 \leq 4$ with $\|u\|_2 \leq 1$, the exact robust rule is $x_1 + 2x_2 + \sqrt{0.04x_1^2 + 0.01x_2^2} \leq 4$. At $x = (1, 1)$ its worst left side is $3 + \sqrt{0.05}$, so it is robust feasible. The maximizing disturbance is proportional to $(0.2, 0.1)$. No tuning weight was invented: the added term follows from the stated uncertainty geometry [1].

Semidefinite programming

Plain version. A semidefinite program constrains a whole matrix to have nonnegative curvature in every vector direction. Checking entries individually is not enough.

A linear matrix inequality is

$$F(x) = F_0 + \sum_{i=1}^{n} x_i F_i \succeq 0.$$

$F_0, \ldots, F_n$ are fixed symmetric matrices; x_i are scalar decision variables; and $F(x) \succeq 0$ requires positive semidefiniteness. A linear objective with affine equalities and such constraints defines an SDP.

If all matrices are diagonal, positive semidefiniteness reduces to nonnegative diagonal entries, giving linear inequalities. General matrices include relationships between coordinates that cannot be captured by entrywise nonnegativity.

For example, the symmetric matrix

$$X = \begin{bmatrix} 1 & r \\ r & 1 \end{bmatrix}$$

is positive semidefinite exactly when $-1 \le r \le 1$. Here r is a scalar off-diagonal correlation. Negative r is allowed despite entrywise negativity, while an excessively large positive r makes the matrix indefinite.

Schur complements expose hidden forms

For positive-definite P, a block matrix condition satisfies

$$\begin{bmatrix} P & q \\ q^\top & t \end{bmatrix} \succeq 0 \quad \iff \quad t - q^\top P^{-1} q \ge 0.$$

P is a positive-definite symmetric block, q a vector and t a scalar. The right side is the Schur complement. The positive-definiteness assumption is important; singular blocks need an additional range condition or a generalized argument.

This identity converts some matrix inverse expressions into affine matrix constraints when the block entries are affine in decision variables. Whether the result is convex depends on that affine structure, not just on displaying a block matrix.

Prove the Schur condition by completing a square

A block matrix is positive semidefinite when its quadratic form is nonnegative for every vector. For $P \succ 0$, a vector v and scalar α give

$$\begin{bmatrix} v \\ \alpha \end{bmatrix}^{\top} \begin{bmatrix} P & q \\ q^{\top} & t \end{bmatrix} \begin{bmatrix} v \\ \alpha \end{bmatrix} = v^{\top} P v + 2\alpha q^{\top} v + \alpha^2 t$$
$$= (v + \alpha P^{-1} q)^{\top} P (v + \alpha P^{-1} q) + \alpha^2 (t - q^{\top} P^{-1} q).$$

Here P is the positive-definite block, q a compatible vector, t a scalar block, and v, α arbitrary test coordinates. If $t \geq q^{\top} P^{-1} q$, both terms are nonnegative. Conversely, choose $\alpha = 1$ and $v = -P^{-1} q$; the first term vanishes, forcing that scalar inequality. This proves both directions of the equivalence [1].

It is also a modeling recipe. With fixed P and affine vector $q(x)$, the upper bound $q(x)^{\top} P^{-1} q(x) \leq t$ becomes a matrix inequality affine in (x, t). If P is a matrix decision variable, the same block representation can still be affine in its entries on the positive-definite domain. A nonlinear term can therefore have a convex epigraph even when its original inverse expression hides the structure.

The assumption $P \succ 0$ enables the square completion and inverse. A singular P cannot be handled by silently using the same formula; the vector's range condition becomes part of the equivalence.

Code: distinguish matrix and entrywise order

This code checks a family with known eigenvalues and demonstrates a valid Schur-complement boundary.

```
import numpy as np

for r in [-0.8, 0.8, 1.2]:
    X = np.array([[1., r], [r, 1.]])
    eigenvalues = np.linalg.eigvalsh(X)
    print("r:", r, "eigenvalues:", eigenvalues)
    assert (eigenvalues.min() >= -1e-12) == (abs(r) <= 1)
P = np.array([[2., .3], [.3, 1.]])
q = np.array([.4, -.2])
t = q @ np.linalg.solve(P, q)
block = np.block([[P, q[:, None]], [q[None, :], np.array([[t]])]])
assert np.linalg.eigvalsh(block).min() >= -1e-12
print("Schur boundary t:", t)
```

This is a spectral check, not a full SDP solver. The course uses small direct calculations to explain the model without implying that a general nonlinear optimizer supplies all the certificates of a dedicated conic method.

Geometric programming

Plain version. Some positive-variable models look nonconvex in their original coordinates but become convex when multiplication turns into addition under logarithms.

In geometric programming, a monomial has the specialized form

$$m(x) = c \prod_{i=1}^{n} x_i^{a_i}, \qquad c > 0, \quad x_i > 0.$$

c is a positive coefficient and a_i are arbitrary real exponents. This differs from the elementary algebra convention requiring nonnegative integer powers.

A posynomial is a sum of such monomials:

$$f(x) = \sum_{k=1}^{K} c_k \prod_{i} x_i^{a_{ik}}, \qquad c_k > 0.$$

K is the number of terms, c_k their positive coefficients and a_{ik} their exponents. Negative coefficients are excluded.

A geometric program minimizes a posynomial, constrains posynomials to be at most one, and constrains monomials to equal one, with positive decision variables [1, Section 4.5]. Arbitrary sums, signs or equalities do not fit this class.

Derive the logarithmic transformation

Set $y_i = \log x_i$, so $x_i = e^{y_i}$. A monomial becomes $\exp(a^\top y + \log c)$; its logarithm is affine. A posynomial's logarithm becomes

$$\log f(e^y) = \log \sum_{k=1}^{K} \exp(a_k^\top y + \log c_k).$$

a_k holds the exponent vector of term k. This is log-sum-exp of affine expressions, which is convex.

The logarithm is strictly increasing, so minimizing the original positive objective and minimizing its log produce the same optimizer after reversing the variable transformation. Posynomial inequalities become convex log-sum-exp inequalities, and monomial equalities become affine equalities.

The objective values are transformed, however: the log objective is not numerically the same as the original cost. Convert back when reporting the application quantity.

This is an exact change of variables on the positive domain. It is different from dropping negative terms or changing a non-monomial equality to an inequality. Those operations can create a different problem.

Code: a positive allocation in log coordinates

Minimize $1/x + 1/y$ subject to $x + y \le 4$ and $x, y > 0$. Symmetry and the convex original objective establish the solution $x = y = 2$; the transformed problem verifies the same answer numerically.

```
import numpy as np
from scipy.optimize import minimize
from scipy.special import logsumexp
```

```
objective = lambda z: logsumexp(-z)
constraint = {"type": "ineq",
              "fun": lambda z: np.log(4.) - logsumexp(z)}
result = minimize(objective, np.log([1., 1.]),
                  constraints=[constraint], method="SLSQP",
                  options={"ftol": 1e-12, "maxiter": 100})
assert result.success, result.message
x = np.exp(result.x)
assert x.sum() <= 4 + 1e-7
assert np.allclose(x, [2., 2.], atol=1e-5)
print("positive variables:", x)
print("original objective:", np.sum(1 / x))
print("transformed objective:", result.fun)
```

The numerical interface is documented in [3]. This small demonstration checks against an analytical solution. It does not make this general-purpose method a replacement for specialized geometric-programming solvers on all scales.

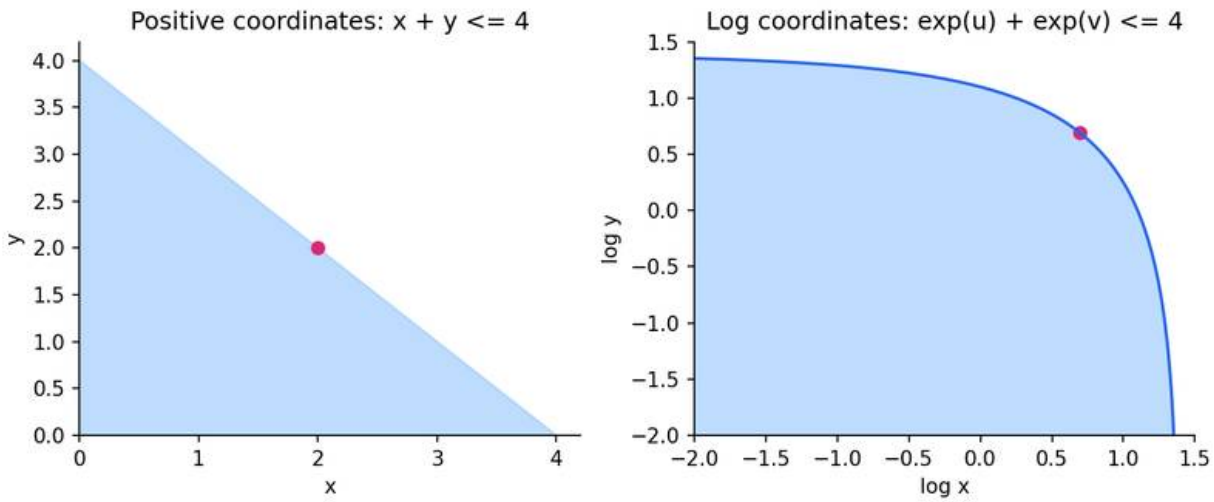


Figure 2 — Original analytical geometry for $x + y \le 4$ with positive variables. In log coordinates its constraint is log-sum-exp $\le \log 4$; axes in the two panels represent different variables.

Worked design: an asymmetric positive allocation

A logarithmic transformation preserves the optimum, even when symmetry cannot supply the answer. Minimize $4/x + 1/y$ under $x + y \le 6$ with $x, y > 0$. Since both terms decrease as their variables grow, the optimum uses the full budget.

Set $y = 6 - x$. Differentiating the remaining scalar objective gives

$$-\frac{4}{x^2} + \frac{1}{(6-x)^2} = 0 \quad \Longrightarrow \quad x = 2(6-x) \quad \Longrightarrow \quad (x^*, y^*) = (4, 2).$$

Here x, y are positive allocations and the star denotes an optimum. The second derivative is positive on $0 < x < 6$, so the stationary point is the unique minimum. Its original cost is $3/2$.

In logarithmic variables $s = \log x$, $t = \log y$, the objective becomes $\log(4e^{-s} + e^{-t})$ and the restriction becomes $\log(e^s + e^t) \le \log 6$. Both are log-sum-exp expressions. The recovered solution is $(s^*, t^*) = (\log 4, \log 2)$, with transformed cost $\log(3/2)$. Reporting $\log(3/2)$ as the original cost would mix two coordinate systems. This example gives an analytical check for the entire transformation [1, Section 4.5].

Quasiconvex problems through feasibility

Plain version. Some useful objectives are not convex, but asking whether a proposed score is achievable gives a convex feasibility problem. A sequence of threshold questions can then find the best score.

Suppose the sets $\{x : f_0(x) \le \alpha\}$ are convex and can be represented by convex constraints for each fixed threshold α. Maintain a lower and upper bound on the optimal score. Test the midpoint threshold: if its feasible set is nonempty, lower the upper bound; if not, raise the lower bound.

Here the threshold is fixed while solving each feasibility problem. Treating it as another free variable can introduce nonconvex products or ratios. The convexity of every threshold problem does not imply joint convexity in the threshold and original decision.

For example, a positive-denominator affine ratio $(a^\top x + b)/(c^\top x + d)$ has convex sublevel sets on a convex domain where $c^\top x + d > 0$. At a fixed threshold α, the score inequality becomes $a^\top x + b \le \alpha(c^\top x + d)$, an affine constraint in x. Positivity of the denominator is essential to multiplying without reversing the inequality.

Numerical feasibility must be handled carefully. An approximate solver may be unable to distinguish an empty set from an extremely narrow feasible region near the optimal threshold. A failed iteration or uncertain status is not automatically an infeasibility certificate. Use an appropriate feasibility procedure and report the threshold interval and tolerances.

This approach is covered by the quasiconvex discussion in [1]. It broadens the course beyond convex objectives while keeping the role of convexity precise: the subproblems are convex, and bisection supplies the outer scalar search.

A modeling audit before solving

Take one original application requirement at a time and translate it to the chosen variables. Check whether its domain is enforced, whether a sign was changed correctly, and whether the formulation preserves its units. Then trace how auxiliary variables are eliminated.

Consider a worst-case norm constraint expressed through a scalar upper bound. If the auxiliary scalar is absent from the objective, it may remain unnecessarily loose. That can still be correct if only an upper limit is required, but it cannot be interpreted as a measured norm without evaluating the norm itself.

Consider a lifted matrix representing outer products. Requiring positive semidefiniteness is convex, while requiring it to equal a vector outer product introduces a rank restriction. Dropping that restriction is a relaxation. The matrix solution may supply a useful bound while lacking a corresponding original vector.

Finally, inspect what happens at domain boundaries. A geometric-program variable cannot simply be set to zero because its logarithm would be undefined. A log-determinant matrix cannot become singular without leaving the finite objective domain. A reformulation that accepts those boundary points needs an explicitly justified extension.

These checks are part of mathematical validation. A numerical solver can correctly optimize an accidentally relaxed or incorrectly signed problem. Comparing against a small analytically solved case is useful because it tests the entire chain from problem statement to arrays and back to interpretation.

Multiobjective decisions and Pareto tradeoffs

Plain version. When two goals conflict, optimization cannot decide your preferences for you. It can describe the tradeoff and compute answers for specified preferences.

For objective vector $F(x) = (f_1(x), \dots, f_k(x))$, a feasible point is Pareto optimal if no other feasible point improves one component without worsening another. Here k is the number of goals.

A positive weighted sum

$$\min_x \sum_{i=1}^{k} w_i f_i(x), \qquad w_i > 0,$$

uses weights w_i encoding preferences and units. Any attained optimizer is Pareto optimal. A zero weight can ignore a component and yield only a weaker efficiency statement.

Weighted sums are not the only choice. One goal can be minimized while others are constrained below prescribed limits. This often matches applications more naturally: minimize reconstruction error subject to a memory budget, rather than inventing a universal conversion from bytes to error.

The scalarization gives a convex problem when the scalar objective and constraints follow convex rules. A tradeoff curve obtained by varying weights describes this model; it does not establish every achievable system behavior if some upstream components or discrete choices were excluded.

Modeling and numerical checks

Plain version. The written problem and the executed arrays must describe the same decision. Small implementation mistakes can silently change constraints.

Check variable bounds explicitly. Some numerical interfaces default to nonnegative variables, while others default to free variables. Leaving defaults unexamined can change an intended regression coefficient into a nonnegative one.

Check inequality signs. A mathematical convention $g(x) \leq 0$ and an interface convention $g(x) \geq 0$ can coexist in the same library ecosystem. Translate the condition deliberately.

Check dimensions and symmetry. A matrix constraint on an asymmetric array does not mean what the symmetric semidefinite notation means. Accidental broadcasting can replace a scalar constraint with a vector of different constraints.

Scale coordinates and objective terms to meaningful ranges, preserving the relationship to original variables. Rescaling can improve numerical conditioning but must be reversed for interpretation and residual reporting. A computed answer should be checked in the original formulation as well as the transformed one.

Finally, separate status from certificates. Success of a generic constrained optimizer can support a small example, especially when an analytical optimum is available. For a production convex model, inspect primal feasibility, dual feasibility and gap using methods appropriate to the problem class.

The written model should remain reviewable after solver conversion. Keep variable units, domain restrictions, transformations and any relaxation in the same report. If a transformed solution is returned, map it back to original coordinates before checking physical constraints and reporting the application objective.

This is especially useful for logarithmic coordinates, auxiliary epigraph variables and lifted matrix models. A feasible transformed point can be misunderstood when reported without its map, and an auxiliary objective can differ from the original expression if its defining restrictions were not enforced accurately.

Modern view

Plain version. Standard problem classes remain useful in current research. New implementations often gain speed by exploiting structure rather than changing the mathematical class.

Disciplined convex programming [4] organizes exact transformations through a graph of verified expressions. Its taxonomy tracks curvature and monotonicity and can turn high-level descriptions into standard forms. The key lesson is to make the convexity argument explicit before selecting a solver.

Applegate and colleagues' PDLP report [5], posted in 2025 and revised in 2026, focuses on large linear programs using a practical first-order primal–dual method. It adds preconditioning, restarting and feasibility polishing around the underlying optimization formulation. Its results concern the tested problems and requested accuracy; they do not imply one universal winner for all linear programs.

Schaller, Bemporad and Boyd [6] study fitting parametric convex functions compatible with disciplined programming. This brings learned objective or constraint components into a convex-modeling workflow, provided the decision-variable convexity and valid parameter domains are preserved.

Open issues include choosing representations that avoid unnecessary expansion, exploiting spar-

sity in cone constraints, certifying infeasibility reliably under scaling, and differentiating through models near active-set changes. A convex class gives a foundation for analysis; a complete system still needs suitable data, tolerances and engineering validation.

Key takeaways

- Convex standard form uses convex objectives and inequalities with affine equalities.
- Auxiliary variables can expose an exact model, but must be justified in both directions.
- Linear, quadratic, second-order-cone and semidefinite programs encode different geometry.
- Matrix order and coordinatewise order are distinct.
- Geometric programming requires positive variables and restricted monomial/posynomial structure.
- Multiobjective optimization needs explicit preferences or limits.
- Status, original-variable feasibility and optimality certificates must be checked separately.

Exercises

1. Convert minimax absolute-residual fitting into an LP and prove equivalence.

   Hint

   Use one scalar bound for every signed residual.

2. Explain why squaring a norm bound can add points if its affine right side may be negative.

   Hint

   The original norm is nonnegative.

3. Derive the allowed range of r in a two-by-two unit-diagonal positive-semidefinite matrix.

   Hint

   Eigenvalues are $1 + r$ and $1 - r$.

4. Transform minimizing $x^{-1} + y^{-1}$ under $x + y \leq 4$ into logarithmic coordinates and recover the original objective.

   Hint

   Substitute exponentials and take logarithms of positive expressions.

5. Construct a convex objective whose equality-level set is not convex.

Hint

Use a squared norm at a positive level.

6. Compare minimizing a weighted error–cost sum with minimizing error under a fixed cost limit. State how their application meanings differ.

Hint

One encodes exchange rates, the other a hard commitment.

References

1. Stephen Boyd and Lieven Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004. **Reading: Chapter 4, especially 4.3–4.7; Section 6.1 for norm fitting.** Official book, free PDF.
2. SciPy developers, "linprog," *SciPy API documentation*, accessed 2026. Verified interface.
3. SciPy developers, "minimize," *SciPy API documentation*, accessed 2026. Optimization interface.
4. Michael Grant, Stephen Boyd and Yinyu Ye, "Disciplined Convex Programming," in *Global Optimization: From Theory to Implementation*, Springer, 2006. Author's publication page.
5. David Applegate, Mateo Díaz, Oliver Hinder, Haihao Lu, Miles Lubin, Brendan O'Donoghue and Warren Schudy, "PDLP: A Practical First-Order Method for Large-Scale Linear Programming," arXiv preprint, 2025, revised 2026. Verified paper record.
6. Michael Schaller, Alberto Bemporad and Stephen Boyd, "Learning Parametric Convex Functions," manuscript, June 2025. Verified manuscript.

OPT 04 • Duality, KKT Conditions and Certificates

Derive Lagrange lower bounds, distinguish weak and strong duality, verify optimality through KKT conditions, and interpret multipliers, sensitivity and infeasibility certificates.

Online edition: https://blog.shyandram.dev/en/learn/optimization/opt-04-duality-kkt/

What you'll learn

- Derive a lower bound on a constrained minimization problem from a Lagrangian.
- Distinguish dual feasibility, weak duality, strong duality and attainment.
- State the KKT conditions and identify when they certify global optimality.
- Interpret complementary slackness and sensitivity with the correct multiplier signs.
- Construct a simple infeasibility certificate and extend dual order to cones.
- Report residuals and gaps without claiming more than numerical evidence supports.

The big picture

Finding a good feasible answer gives an upper bound on the best possible cost. How do we know that a much better answer is not hidden elsewhere? A dual problem produces lower bounds. When an upper and lower bound meet, they certify the optimum.

Boyd and Vandenberghe's Chapter 5 [1] connects this idea to geometry, supporting planes, multipliers and numerical algorithms. Duality is useful even when we already know how to solve the primal problem: it verifies an answer, describes the value of limited resources and reveals alternative computational structure.

> **Explain like I'm five**
>
> You found a recipe costing ten coins. A clever argument says every allowed recipe must cost at least ten coins. Together they prove your recipe is cheapest. Finding the recipe and proving the lower bound are different jobs.

The guarantee must respect signs and assumptions. A value computed from arbitrary multipliers may not be a lower bound. A small stationarity residual alone is not a complete certificate. Strong duality is not automatic for every convex formulation.

This chapter develops the derivations with explicit domains and uses small examples whose exact answers can be checked. The resulting reasoning extends to large problems, but the implementation details and tolerances remain part of the numerical assessment.

From constraints to the Lagrangian

Plain version. Attach a price to every inequality and equality. Then minimize the priced objective without explicitly enforcing those constraints.

Consider

$$\min_{x \in D} f_0(x) \quad \text{subject to} \quad f_i(x) \le 0, \quad h_j(x) = 0.$$

x is the decision vector, D is the common domain retained during minimization, f_0 is the objective, f_i are inequalities and h_j are equalities.

The Lagrangian is

$$L(x, \lambda, \nu) = f_0(x) + \sum_{i=1}^{m} \lambda_i f_i(x) + \sum_{j=1}^{p} \nu_j h_j(x).$$

λ_i are inequality multipliers and ν_j equality multipliers. For the convention $f_i \le 0$, require $\lambda_i \ge 0$. Equality multipliers are unrestricted because equality residuals vanish at feasible points regardless of sign.

At a feasible point, the inequality contribution is nonpositive and the equality contribution zero. Thus $L(x, \lambda, \nu) \le f_0(x)$ for nonnegative λ.

Calling these multipliers "penalties" can be misleading. This Lagrangian is not simply a large positive cost added to violations during an unconstrained minimization. Its role is to create a lower-bound family, and the sign condition is selected for that purpose.

Define the dual function

$$g(\lambda, \nu) = \inf_{x \in D} L(x, \lambda, \nu).$$

g is the Lagrange dual function; its input is the multiplier pair. It takes the infimum over every point in D, not just feasible points.

For any feasible x and nonnegative λ,

$$g(\lambda, \nu) \le L(x, \lambda, \nu) \le f_0(x).$$

The first inequality follows from taking an infimum. The second follows from primal feasibility and multiplier signs. Therefore g is a lower bound on the primal optimal value.

If the infimum is negative infinity, the bound is valid but uninformative. Dual feasibility includes multipliers with an appropriate finite dual value when we want a useful certificate.

Why the dual problem is concave

Plain version. The lower bound changes with prices. We choose the prices giving the strongest bound, and that maximization has a concave objective.

For each fixed x, L is affine in the multipliers. The pointwise infimum of affine functions is concave. Therefore g is concave even if the original problem is not convex.

The dual problem is

$$\begin{aligned} \text{maximize}_{\lambda,\nu} \quad & g(\lambda, \nu) \\ \text{subject to} \quad & \lambda \geq 0. \end{aligned}$$

Let its optimal value be d^*. Weak duality states $d^* \leq p^*$, where p^* is the primal optimal value. This result does not require convexity, only the lower-bound construction with the correct signs.

A concave maximization is a convex optimization problem in the usual sense. However, evaluating the dual function can itself be difficult if it requires a hard nonconvex minimization. "The dual is concave" does not automatically provide a practical oracle.

The dual also need not have the same dimension as the primal. There is one variable for each explicitly dualized constraint. Retaining some restrictions in D rather than attaching multipliers changes the written dual representation, while the lower-bound logic remains consistent.

The concavity proof in three lines

The dual is concave because the bound at mixed prices is at least the same mixture of the individual bounds. Let $z = (\lambda, \nu)$ collect the multipliers, and take two allowed choices z_1, z_2 and $\theta \in [0, 1]$. Affinity of the Lagrangian in prices gives

$$\begin{aligned} g(\theta z_1 + (1-\theta)z_2) &= \inf_{x \in D} [\theta L(x, z_1) + (1-\theta) L(x, z_2)] \\ &\geq \theta \inf_{x \in D} L(x, z_1) + (1-\theta) \inf_{x \in D} L(x, z_2) \\ &= \theta g(z_1) + (1-\theta) g(z_2). \end{aligned}$$

Here D is the retained primal domain and g the dual function; consider finite dual values for this comparison. Each summand at a shared x is at least its own infimum, which explains the middle inequality. Nothing in that step assumes the primal objective is convex [1].

This distinction matters computationally. For a nonconvex primal, merely finding a local minimum of L does not evaluate its infimum. That local value can exceed $g(z)$ and cease to be a valid lower bound. Dual concavity concerns the exact dual function, not any approximate routine called a dual evaluator.

A complete one-dimensional derivation

Plain version. A small example shows how prices, the unconstrained minimizer and the bound fit together.

Minimize $\frac{1}{2}(x-2)^2$ subject to $x \le 1$. Here x is scalar. The constrained optimizer is $x^* = 1$, with cost $p^* = 1/2$.

Using multiplier $\lambda \ge 0$,

$$L(x,\lambda) = \frac{1}{2}(x-2)^2 + \lambda(x-1).$$

The unconstrained Lagrangian minimizer satisfies $x-2+\lambda = 0$, so $x_\lambda = 2-\lambda$. Substitution yields

$$g(\lambda) = \lambda - \frac{1}{2}\lambda^2.$$

x_λ depends on the selected multiplier and need not be primal feasible. The dual is a concave parabola, maximizing at $\lambda^* = 1$ with $g(\lambda^*) = 1/2$. Upper and lower bounds meet.

The optimum multiplier balances the objective's desire to increase x against the active upper-bound constraint. If λ is too small, the Lagrangian minimizer lies outside the allowed region. If it is too large, the bound becomes weaker despite the larger multiplier.

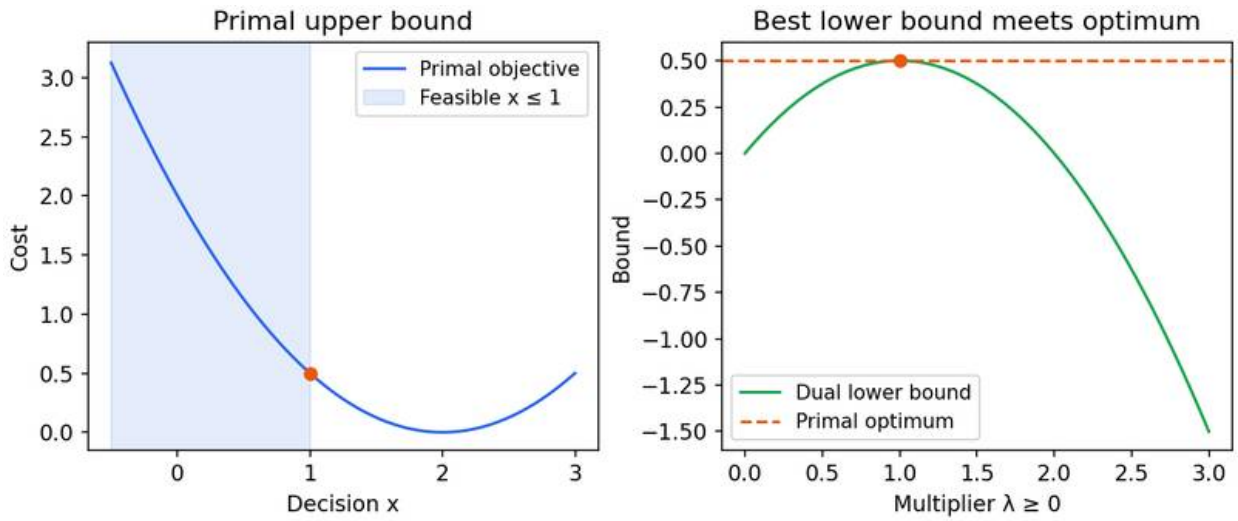


Figure 1 — Original analytical derivation for minimizing $(x-2)^2 / 2$ under $x \le 1$. The feasible primal optimizer and the best dual bound both have value $1/2$.

Code: verify the gap and all KKT components

This standalone calculation checks primal feasibility, multiplier sign, stationarity, complementary slackness and equality of the two values.

```
import numpy as np

x, multiplier = 1.0, 1.0
objective = 0.5 * (x - 2)**2
dual_value = multiplier - 0.5 * multiplier**2
constraint = x - 1
stationarity = x - 2 + multiplier
complementarity = multiplier * constraint
assert constraint <= 0 and multiplier >= 0
assert abs(stationarity) < 1e-12
assert abs(complementarity) < 1e-12
assert np.isclose(objective, dual_value)
print("primal value:", objective, "dual value:", dual_value)
print("stationarity:", stationarity, "complementarity:", complementarity)
```

Strong duality and constraint qualifications

Plain version. Weak duality always gives the ordering. Strong duality says the best lower bound is exact, which needs additional conditions.

Strong duality means $d^* = p^*$. It is a statement about optimal values, not automatically about whether optimizers exist. A problem can have a value approached by a sequence without attaining it.

For a convex primal with affine equalities, a standard sufficient condition is Slater's condition [1, Section 5.2]:

$$\exists x \in \operatorname{relint} D : f_i(x) < 0 \text{ for all } i, \qquad Ax = b.$$

D is the common function domain, $\operatorname{relint} D$ its relative interior, and A, b define the affine equality constraints. This says there is a point strictly satisfying inequalities while remaining in the appropriate interior and satisfying equalities.

Under these assumptions, strong duality holds. With a finite optimal value, the dual optimum is also attained. The primal attainment question can still require separate existence reasoning.

There are refined conditions for affine inequalities, so requiring all of them strictly can be stronger than necessary. A failure to find a Slater point does not by itself disprove strong duality. It means this sufficient argument has not been established.

Convexity alone is not enough in every formulation. Domains can be degenerate and constraint qualifications can fail. Rather than saying "convex problems always have zero gap," state the condition being used or verify a valid matching certificate directly.

Domain restrictions are part of duality

If an objective contains a logarithm or inverse, its domain remains in the Lagrangian minimization. Treating it as a function on all real inputs silently changes the problem.

Alternatively, an extended-value function can encode a domain by assigning positive infinity outside it. This is mathematically convenient, but numerical code should not rely on accidental invalid floating-point values as a modeling convention.

In a conic formulation, domain and cone restrictions can be dualized or retained according to a consistent representation. The choice affects the dual expression and sign conventions. Derive the bound from the stated primal rather than copying a formula with incompatible signs.

Zero gap can coexist with no optimal multiplier

A failed constraint qualification does not force a positive duality gap, but it can remove multiplier attainment. Consider minimizing x subject to $x^2 \le 0$, with scalar $x \in \mathbb{R}$. The only feasible point is zero, so $p^* = 0$. There is no strictly feasible point.

The Lagrangian is $L(x, \lambda) = x + \lambda x^2$. Minimizing over x gives

$$g(\lambda) = \begin{cases} -1/(4\lambda), & \lambda > 0, \\ -\infty, & \lambda = 0. \end{cases}$$

Here $\lambda \ge 0$ prices the quadratic restriction and g is its dual bound. For positive λ, the minimizing point is $-1/(2\lambda)$. As λ increases without bound, $g(\lambda)$ approaches zero. Thus $d^* = p^* = 0$, yet no finite multiplier attains the dual optimum.

At the primal solution, stationarity would require $1 + 2\lambda x = 0$, which becomes $1 = 0$. No KKT multiplier exists for this representation. Replacing the equivalent restriction by the equality $x = 0$ gives a multiplier $\nu = -1$. The feasible decision is unchanged, but the regularity and multiplier description improve. This is why OPT 01's hidden equalities and OPT 07's initialization deserve attention [1].

KKT optimality conditions

Plain version. KKT combines allowed primal choices, allowed prices, balanced forces and the rule that slack constraints carry no positive price.

For differentiable convex f_0, f_i and affine equality $Ax = b$, the conditions are

$$\begin{aligned} & f_i(x) \le 0, \qquad Ax = b, \\ & \lambda_i \ge 0, \\ & \lambda_i f_i(x) = 0, \\ & \nabla f_0(x) + \sum_i \lambda_i \nabla f_i(x) + A^\top \nu = 0. \end{aligned}$$

x is a primal candidate, λ are inequality multipliers, ν are equality multipliers, and the final row is stationarity of the Lagrangian. These are the Karush–Kuhn–Tucker conditions.

For this convex differentiable problem, satisfying all conditions is sufficient for primal and dual optimality. Nonnegative multipliers make the Lagrangian convex in x; stationarity makes x its

global minimizer; feasibility and complementary slackness make the Lagrangian value equal the objective.

Under a suitable constraint qualification such as Slater's condition, optimal points also admit multipliers satisfying KKT. This supplies necessity as well as sufficiency. Keep the two logical directions distinct.

For nonconvex problems, a stationary Lagrangian can be a local maximum or saddle in x. KKT may be necessary under regularity but does not generally certify a global optimum. A concave quadratic at its interior stationary point is a simple counterexample.

Complementary slackness is one-way information

If $f_i(x) < 0$, complementary slackness forces $\lambda_i = 0$. If $\lambda_i > 0$, the constraint is active: $f_i(x) = 0$.

An active constraint can nevertheless have zero multiplier. Degeneracy or redundant constraints can make prices nonunique. Therefore "active" and "positive price" are not interchangeable labels.

At a smooth constrained optimum, the negative objective gradient lies in the combination of active constraint normals and equality normals with appropriate signs. This gives a geometric explanation of stationarity: feasible descent directions cannot reduce the objective.

Numerical near-activity is tolerance-dependent. A constraint with a tiny slack and tiny multiplier may be difficult to classify discretely. Report actual values where the distinction matters instead of forcing a confident active/inactive label.

Equality-constrained quadratics

Plain version. For a quadratic objective and linear equalities, the KKT equations form one linear system. This reveals the algebra used by later constrained algorithms.

For objective $\frac{1}{2}x^\top Qx + c^\top x$ under $Ax = b$,

$$\begin{bmatrix} Q & A^\top \\ A & 0 \end{bmatrix} \begin{bmatrix} x \\ \nu \end{bmatrix} = \begin{bmatrix} -c \\ b \end{bmatrix}.$$

Q is symmetric positive semidefinite, c is the objective coefficient vector, A the equality matrix, b its target, and ν the equality multiplier.

The block matrix is generally indefinite even though the optimization problem is convex. A solver requiring a positive-definite matrix cannot be used blindly on this full KKT system.

If Q is positive definite and A has independent rows, the system is nonsingular. More general cases require considering curvature on the equality nullspace and redundant constraints. A pseudoinverse may produce one solution but should not hide the underlying rank issue.

The equality multiplier balances the gradient in directions normal to the feasible affine set. Its sign is unconstrained. Removing a constraint changes both the feasible set and the meaning of its

multiplier.

Deriving an LP dual carefully

Plain version. A dual formula is not universal notation. Its signs follow from the exact primal convention.

Take the LP $\min c^\top x$ subject to $Ax = b$, $x \geq 0$. Retain nonnegativity in the domain. The Lagrangian is $c^\top x + \nu^\top (Ax - b)$.

Taking the infimum over nonnegative x is finite exactly when $c + A^\top \nu \geq 0$. If any coefficient is negative, that coordinate can grow without bound and send the infimum to negative infinity.

Set $y = -\nu$. The dual becomes

$$\max_y b^\top y \quad \text{subject to} \quad A^\top y \leq c.$$

y is an unrestricted equality-price vector. For any feasible pair,

$$c^\top x - b^\top y = x^\top (c - A^\top y) \geq 0.$$

The gap is the inner product of nonnegative primal coordinates and nonnegative dual slack. This is weak duality in a particularly transparent form.

Using a primal maximization or reversed inequalities changes the dual signs. Rather than memorizing separate tables, deriving the Lagrangian and checking the bound can prevent mistakes.

A fitting dual built from the norm conjugate

Duality can turn a parameter penalty into a residual restriction. Consider sparse linear fitting with objective $\frac{1}{2}\|Ax - b\|_2^2 + \lambda\|x\|_1$, where A, b are fixed and $\lambda > 0$. Introduce residual variable $r = Ax - b$ and an unrestricted equality multiplier y:

$$L(x, r, y) = \frac{1}{2}\|r\|_2^2 + \lambda\|x\|_1 + y^\top (Ax - b - r).$$

Here x is the parameter vector and y has one coordinate per observation. Minimizing in r gives $r = y$ and value $-\|y\|_2^2/2$ for the residual terms. Minimizing in x is finite exactly when $\|A^\top y\|_\infty \leq \lambda$, by the norm-conjugate calculation in OPT 02. The dual is therefore

$$\text{maximize}_y \quad -\frac{1}{2}\|y\|_2^2 - b^\top y \qquad \text{subject to} \quad \|A^\top y\|_\infty \leq \lambda.$$

Here y is the dual variable; the norm restriction is dual feasibility. At an optimum it coincides with the fitted residual, while its correlations with the design columns must stay within the penalty threshold. This links the lower-bound construction to the geometry of sparse fitting [1, Chapters 5–6].

For synthetic $A = I$, $b = (2, -0.3, 0.1)$ and $\lambda = 0.4$, the primal solution is $x = (1.6, 0, 0)$ and dual candidate $y = x - b = (-0.4, 0.3, -0.1)$. Both are feasible for their respective problems and

both objective values equal 0.77. Matching values certify the fit; observing zero coordinates alone would not.

Sensitivity and the value of a constraint

Plain version. A multiplier measures how the best cost responds to changing a constraint, under specified regularity. Its units depend on the constraint's units.

Perturb inequalities to $f_i(x) \le u_i$ and equalities to $Ax - b = v$. Let the best cost be $p^*(u, v)$. If strong duality holds at zero perturbation and optimal multipliers exist,

$$p^*(u, v) \ge p^*(0, 0) - \lambda^{*\top} u - \nu^{*\top} v.$$

u and v are right-side perturbations; λ^*, ν^* are optimal unperturbed multipliers. Positive u_i relaxes an inequality under the chosen convention.

The inequality gives a global affine lower bound on the perturbed value. If the value function is differentiable at the original problem, its derivatives are $-\lambda^*$ and $-\nu^*$.

Without differentiability, multipliers can describe supporting slopes rather than one unique derivative. A large multiplier signals sensitivity in that local modeling direction, not a universal physical importance score.

A scalar example with a changing bound

Minimize $\frac{1}{2}(x - 2)^2$ under $x \le b$. The best point is $\min(2, b)$ and the value is

$$p^*(b) = \frac{1}{2} \max(2 - b, 0)^2, \qquad \lambda^*(b) = \max(2 - b, 0).$$

b is the allowed upper bound. Where differentiable, $dp^*/db = -\lambda^*(b)$. Once the bound exceeds the unconstrained target, relaxing it further does not help and the multiplier is zero.

Scaling the constraint changes the multiplier reciprocally. Multiplying $x - b \le 0$ by a positive factor preserves feasible points but rescales its associated price. Compare sensitivity only after accounting for units and normalization.

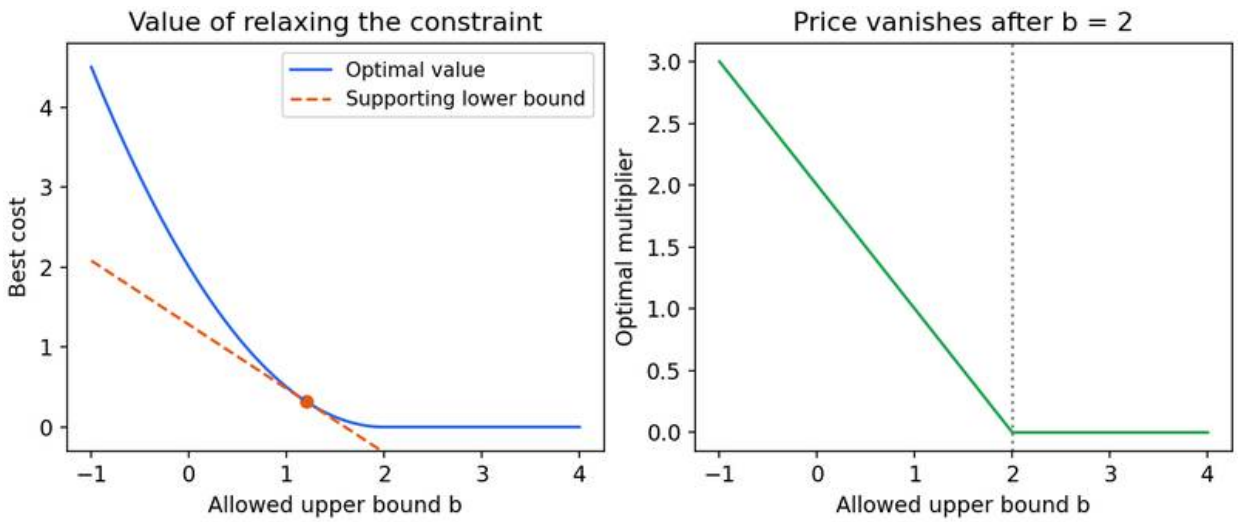


Figure 2 — Original analytical sensitivity example. Relaxing the bound reduces cost while it is binding; after the unconstrained target is allowed, the value and multiplier are zero.

Code: a finite-difference sensitivity check

```
import numpy as np

value = lambda b: 0.5 * max(2 - b, 0)**2
b, eps = 1.2, 1e-5
multiplier = max(2 - b, 0)
derivative = (value(b + eps) - value(b - eps)) / (2 * eps)
assert np.isclose(derivative, -multiplier, atol=1e-8)
test_bounds = np.linspace(-1, 4, 50)
lower_bound = value(b) - multiplier * (test_bounds - b)
assert np.all([value(t) for t in test_bounds] >= lower_bound - 1e-12)
print("finite-difference derivative:", derivative)
print("negative multiplier:", -multiplier)
```

Deriving sensitivity from the same lower bound

The sensitivity inequality needs no new principle. Use the original optimal multipliers on the perturbed Lagrangian. For any point feasible under $f_i(x) \le u_i$ and $h_j(x) = v_j$,

$$f_0(x) \ge g(\lambda^*, \nu^*) - {\lambda^*}^\top u - {\nu^*}^\top v.$$

Here u, v are the perturbation vectors and the starred multipliers are optimal for the unperturbed problem. Taking the infimum over perturbed feasible points and using strong duality at the original problem produces the previously stated value-function bound. The minus sign follows from subtracting the perturbed right-hand sides inside the Lagrangian.

For the scalar bound example at $b = 1$, the value is $1/2$ and $\lambda^* = 1$. Increasing the bound by δ with $0 < \delta < 1$ gives actual value $(1 - \delta)^2/2$. Its decrease is $\delta - \delta^2/2$, while the multiplier predicts first-order decrease δ. The lower-bound prediction $1/2 - \delta$ stays below the actual value, with difference $\delta^2/2$.

If the restriction is instead written $10(x - b) \leq 0$, its multiplier is $1/10$. The derivative of this restriction with respect to b is -10, so its product with the multiplier still predicts slope -1. Prices depend on representation; the predicted change in the original cost is consistent [1, Section 5.6].

Infeasibility certificates

Plain version. A valid weighted combination of constraints can prove that no choice satisfies them. That is stronger evidence than an algorithm failing to find a point.

For inequalities $Gx \leq h$, suppose a vector y satisfies

$$y \geq 0, \qquad G^\top y = 0, \qquad h^\top y < 0.$$

G, h define the inequality system and y is a nonnegative certificate vector. If a feasible x existed, multiplying and summing would give $0 = y^\top Gx \leq y^\top h < 0$, a contradiction.

This is the geometry behind a theorem of alternatives such as the forms discussed in [1, Section 5.8]. It distinguishes a mathematical certificate from an unsuccessful search.

In floating point, the equalities and strict sign must be assessed relative to scale. A nearly zero transpose residual and a tiny negative certificate value may be inconclusive under accumulated errors. Numerical certification requires a suitable tolerance analysis.

Code: incompatible scalar inequalities

The constraints are $x \leq 0$ and $x \geq 1$. Their nonnegative sum proves impossibility.

```
import numpy as np

G = np.array([[1.], [-1.]])
h = np.array([0., -1.])
certificate = np.array([1., 1.])
assert np.all(certificate >= 0)
assert np.allclose(G.T @ certificate, 0)
assert h @ certificate < 0
print("transpose residual:", G.T @ certificate)
print("certificate contradiction:", h @ certificate)
```

Cones, saddle points and numerical gaps

Plain version. The lower-bound argument extends to cone inequalities by choosing prices from the matching dual cone. A primal–dual solution can also be viewed as a saddle point.

For a cone K, its dual cone is

$$K^* = \{y : y^\top z \geq 0 \text{ for every } z \in K\}.$$

z ranges over the original cone and y is a price vector. When a constraint is expressed as belonging to K, choose signs so the dual-cone inner product supplies the intended lower bound. Orthants, second-order cones and positive-semidefinite cones have familiar self-dual structures under their usual inner products.

For matrix variables, the inner product is trace: $\langle X, Y \rangle = \operatorname{tr}(XY)$ for symmetric matrices. Complementary slackness is then a trace condition, not entrywise multiplication.

Under the appropriate primal–dual assumptions, a saddle point satisfies

$$L(x^*, \lambda, \nu) \leq L(x^*, \lambda^*, \nu^*) \leq L(x, \lambda^*, \nu^*).$$

x^* and (λ^*, ν^*) are optimal primal and dual points; x ranges over the domain and $\lambda \geq 0$. The first side maximizes prices at the feasible point; the second minimizes decisions at the optimal prices.

In practice, an objective difference is a certified gap only when both points are appropriately feasible and the dual value is correctly evaluated. Approximate feasibility can require corrected bounds. A relative gap should use a documented denominator that remains meaningful near zero or with negative objective values.

A complete LP certificate and an invalid near-solution

Consider minimizing $x_1 + 3x_2$ subject to $x_1 + x_2 = 1$ and $x \geq 0$. The primal point $(1, 0)$ has cost one. In the LP dual convention derived above, the scalar dual variable $y = 1$ is feasible because $(y, y) \leq (1, 3)$, and its value is one. These matching bounds prove optimality without enumerating every feasible point.

The dual slack is $(0, 2)$. Its inner product with the primal point is zero, making complementary slackness explicit. The second coordinate is expensive relative to the equality price and receives zero allocation. This interpretation follows from the coefficients and the stated equality; it is not a general feature-importance conclusion.

Now take the approximate point $(1.01, -0.01)$. It satisfies the equality exactly and has an apparently better objective, 0.98. It violates nonnegativity. Comparing that objective with the dual value produces a negative difference, but does not refute weak duality: the candidate is outside the feasible set.

A routine that displays only equality residual and objective could therefore report a misleading result. Check every bound and cone restriction, including those encoded through a solver's variable-bound argument rather than an explicit matrix row.

Clipping the negative coordinate gives $(1.01, 0)$, which then violates the equality. A subsequent normalization gives $(1, 0)$ in this particular example. Neither clipping nor normalization is a universally valid repair procedure for coupled constraints.

Approximate residuals need corrected reasoning

For an approximately feasible LP pair, the algebraic objective difference is

$$c^\top x - b^\top y = x^\top s + y^\top r, \qquad s = c - A^\top y, \quad r = Ax - b.$$

x, y are candidate primal and dual vectors, s the dual slack and r the primal equality residual. The identity is exact algebra; a nonnegative certified gap additionally needs the required feasibility properties.

If $x \geq 0$ and $s \geq 0$ but $r \neq 0$, the residual term can have either sign. A small residual multiplied by a large price can matter. This is why an absolute equality tolerance by itself is not a universal objective-accuracy guarantee.

If dual slack is only bounded below by $-\varepsilon$ componentwise and $x \geq 0$, then $x^\top s \geq -\varepsilon \|x\|_1$. Here $\varepsilon \geq 0$ measures the worst negative slack. Large primal magnitude can amplify a tiny dual violation.

This inequality does not turn the approximate dual objective into a lower bound for every feasible primal point unless a valid bound on those points is also available. An unbounded feasible set makes that correction especially delicate. Avoid describing an arbitrary approximate multiplier as an exact global certificate.

Report primal residuals, negative-slack magnitudes, multiplier norms and objective differences together. If a rigorous certificate is required, use a valid repair, verified error bound or higher-precision check appropriate to the problem structure.

Modern view

Plain version. Modern large-scale methods use primal–dual structure directly, while applications increasingly differentiate through constrained solutions. Both rely on the same certificate and regularity ideas.

Applegate and colleagues' PDLP report [2], first posted in 2025 and revised in 2026, develops large-scale LP computation with practical primal–dual steps and feasibility polishing. Its relevance here is that a useful system monitors feasibility as well as cost improvement; a low objective error with unusable constraint violations is a different outcome.

Pokutta's 2024 survey [3] explains primal and dual convergence for Frank–Wolfe methods. A linear minimization over the feasible set gives a first-order gap from the supporting-plane inequality. This is related to a lower-bound certificate, while its assumptions and computation differ from a generic Lagrange multiplier dual.

Agrawal and colleagues' differentiable convex optimization layers [4] connect structured formulations with differentiation through solutions. The original paper is from 2019; it is included as a

foundation for current optimization-layer work, not mislabeled as a new 2026 result. Differentiability and conditioning can fail near degeneracy or active-set changes, so a valid convex solution does not automatically imply a well-behaved gradient.

Current questions include obtaining reliable certificates at very large scale, interpreting uncertainty when constraints are learned, differentiating through nonunique solutions and balancing precision against application latency. Keep optimization accuracy, model validity and learning generalization as separate evaluations.

Key takeaways

- Feasible primal points provide upper bounds; feasible dual points provide lower bounds.
- Multiplier signs follow the exact inequality convention.
- The dual function is concave even when the primal is nonconvex.
- Strong duality needs suitable conditions or a directly verified matching certificate.
- KKT is sufficient for convex differentiable problems and needs regularity for necessity.
- Complementary slackness and sensitivity have precise one-way and sign rules.
- Numerical residuals and gaps must be interpreted with scale and feasibility.

Exercises

1. Derive the dual of minimizing $\frac{1}{2}(x-a)^2$ under $x \le b$, treating a, b as fixed data.

Hint

Minimize the Lagrangian over the scalar variable before maximizing its multiplier.

2. Give an active inequality with zero multiplier at an optimum.

Hint

A redundant bound at an unconstrained minimizer works.

3. Derive the LP dual using nonnegative variables retained in the domain.

Hint

The linear coefficient must be nonnegative for a finite infimum.

4. Explain why KKT stationarity at the origin does not certify minimizing $-x^2$ on $[-1, 1]$.

Hint

The origin is a maximum of the objective.

5. Construct a nonnegative weighted contradiction proving a pair of scalar inequalities infeasible.

Hint

Add an upper bound below a required lower bound.

6. Rescale an inequality by ten. Derive the corresponding multiplier rescaling and explain why primal sensitivity stays consistent.

Hint

Preserve the Lagrangian product.

References

1. Stephen Boyd and Lieven Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004. **Reading: Chapter 5, including duality, KKT, perturbation, alternatives and generalized inequalities.** Official book, free PDF.
2. David Applegate, Mateo Díaz, Oliver Hinder, Haihao Lu, Miles Lubin, Brendan O'Donoghue and Warren Schudy, "PDLP: A Practical First-Order Method for Large-Scale Linear Programming," arXiv preprint, 2025, revised 2026. Verified paper.
3. Sebastian Pokutta, "The Frank-Wolfe Algorithm: A Short Introduction," *Jahresbericht der Deutschen Mathematiker-Vereinigung*, 126, 2024. Open-access survey.
4. Akshay Agrawal, Brandon Amos, Shane Barratt, Stephen Boyd, Steven Diamond and J. Zico Kolter, "Differentiable Convex Optimization Layers," *NeurIPS*, 2019. Official publication page.

OPT 05 • Approximation, Statistical Estimation and Experiment Design

Choose convex residual losses, regularizers and shape constraints, formulate likelihood and detector problems, and allocate measurements through optimal experiment design.

Online edition: https://blog.shyandram.dev/en/learn/optimization/opt-05-fitting-estimation/

What you'll learn

- Select a residual norm by the errors that matter, rather than habit.
- Distinguish parameter regularization, robust residual losses and uncertain measurement models.
- Impose monotonicity and convexity on fitted functions using linear inequalities.
- Recognize convex likelihood and probability-estimation formulations.
- Design randomized detectors and understand moment-based risk bounds.
- Allocate measurement effort using information matrices and explicit design criteria.

The big picture

An optimization model can fit a curve to observations, estimate a distribution or decide which observations to collect next. These tasks share a common pattern: choose a representation, state how disagreement is measured, and specify which choices are allowed.

Boyd and Vandenberghe's Chapters 6–7 [1] connect approximation and statistical estimation. Read them together: the same squared residual can mean a geometric approximation error, a likelihood under a noise model, or one component of a constrained design objective. Those interpretations impose different assumptions even when the numerical formula is identical.

> **Explain like I'm five**
>
> You want a ruler that passes near several dots. One rule worries about the average mismatch; another worries about the worst dot; a third refuses to bend too much. Before finding the best ruler, decide which meaning of "best" matches the job.

A successful solver answers the stated optimization problem. It does not prove that the measurement assumptions are true, that a fitted curve predicts new data, or that a sensor allocation is physically achievable. This chapter keeps those questions visible alongside the mathematics.

The examples use synthetic observations and tiny measurement models. They show how the pieces work without hiding downloads, pretrained systems or application-specific assumptions inside the code.

Residual norms express priorities

Plain version. A residual is the difference between a prediction and an observation. Choosing a norm determines which patterns of error are expensive.

For a linear model,

$$r = Ax - b, \qquad \min_x \|Ax - b\|.$$

$A \in \mathbb{R}^{m \times n}$ is a fixed design matrix, $b \in \mathbb{R}^m$ the observations, $x \in \mathbb{R}^n$ the parameters and $r \in \mathbb{R}^m$ the residual vector. The chosen norm acts on residuals, not directly on parameters.

The Euclidean norm or its square emphasizes large residuals through their squared contribution. The absolute-value norm adds residual magnitudes. The maximum norm minimizes the largest absolute residual and can be appropriate when every measured error must remain below a uniform tolerance [1, Section 6.1].

$$\|r\|_2^2 = \sum_i r_i^2, \qquad \|r\|_1 = \sum_i |r_i|, \qquad \|r\|_\infty = \max_i |r_i|.$$

r_i is residual coordinate i. All three objectives are convex when r is affine in x. Squaring a nonnegative Euclidean norm preserves its minimizers, but changes objective values, derivatives and the tradeoff with any additional penalty.

For absolute residuals, introduce $t_i \geq r_i$ and $t_i \geq -r_i$, then minimize $\sum_i t_i$. For a uniform tolerance introduce one scalar t and impose $-t \leq r_i \leq t$. These are LP representations when there are only affine additional restrictions.

Three meanings of the best constant

A single fitted number exposes the difference between residual objectives. Take the synthetic observations $b = (0, 1, 8)$ and predict the same scalar c for all three. The resulting design matrix is a column of ones.

For squared loss, differentiating $c^2 + (c-1)^2 + (c-8)^2$ gives $6c - 18 = 0$, hence $c = 3$. For absolute loss, the slope is the number of observations below c minus the number above it, away from the data points. The slope changes from negative to positive at the median, so $c = 1$ minimizes the sum. For maximum absolute loss, balance the two extreme errors: $c = 8 - c$, hence $c = 4$.

Here c is the fitted constant and b the fixed observations. At their respective optima, squared residual sum is 38, absolute residual sum is 8, and maximum residual is 4. These are different quantities with different units; comparing their raw numerical sizes would be meaningless. The important comparison is the returned prediction: average fit chooses three, median fit chooses one, and uniform-error fit chooses four. The observation eight pulls the mean strongly but does not move the median beyond the middle observation [1].

For a real task, ask which error pattern is costly. Uniform instrument tolerances suggest maximum error. A specified Gaussian measurement model suggests squared error. Contaminated responses

can motivate bounded-influence residual losses. None of these choices follows solely from the convenience of a solver.

Units and weighted errors

Measurements with different units cannot be added sensibly without a stated scaling convention. A residual of one pixel and a residual of one second are not automatically comparable.

A weighted least-squares objective can be written

$$(Ax - b)^\top W(Ax - b), \qquad W \succeq 0.$$

W is a fixed positive-semidefinite weighting matrix. If W is the inverse of a known positive-definite noise covariance, the objective has a statistical interpretation. If weights encode priorities, explain that interpretation instead.

Off-diagonal entries can describe correlated measurement errors. Replacing a full covariance by diagonal weights is a modeling approximation, not just an implementation detail. Whitening with a matrix factor transforms the residual before fitting and should preserve the intended quadratic form.

A large residual is not automatically an outlier. It could reveal a wrong model, a poorly scaled feature, genuine rare behavior or a corrupted observation. Inspect the generating process before deciding which errors should lose influence.

Least-norm solutions and regularization

Plain version. Matching the observations may leave many parameter choices. A norm on parameters chooses among them or trades fit against complexity.

A least-norm problem has the form

$$\min_x \|x\| \quad \text{subject to} \quad Ax = b.$$

x is the parameter vector, and A, b are fixed. This differs from a residual-norm problem: here exact consistency is required, and the objective selects a small parameter vector [1, Section 6.2].

If the equality is inconsistent, the problem is infeasible. Replacing it with a soft residual term changes the question. In an underdetermined consistent system, the Euclidean least-norm solution can be expressed through a suitable pseudoinverse, but numerical rank and singular values still matter.

Regularized approximation instead balances two goals:

$$\min_x \frac{1}{2}\|Ax - b\|_2^2 + \lambda R(x), \qquad \lambda \geq 0.$$

R is a convex penalty and λ its nonnegative weight. The factor one-half simplifies derivatives. Changing that factor while keeping λ fixed changes the tradeoff.

For $R(x) = \frac{1}{2}\|x\|_2^2$, the objective is a ridge or quadratic regularization model [1, Section 6.3]. Positive λ makes the Hessian $A^\top A + \lambda I$ positive definite. For $R(x) = \|x\|_1$, the penalty encourages exact zero coordinates through its nonsmooth geometry.

An intercept is often treated separately because penalizing it has a different meaning from penalizing feature effects. Feature units also change a parameter norm. Standardizing columns or using an explicitly scaled penalty makes this choice visible.

Derive quadratic shrinkage direction by direction

Quadratic regularization reduces the instability caused by weakly observed parameter directions. For the ridge objective already defined, setting its gradient to zero gives

$$(A^\top A + \lambda I)x^* = A^\top b.$$

Here x^* is the optimizer, A, b are the fixed design and observations, I is the identity, and $\lambda > 0$ is the penalty weight. Let $A = U\Sigma V^\top$ be a singular-value decomposition, with orthonormal singular vectors in U, V and singular values σ_i in Σ. In direction v_i, the coefficient is

$$v_i^\top x^* = \frac{\sigma_i}{\sigma_i^2 + \lambda} u_i^\top b.$$

Here u_i, v_i are paired singular vectors. Without regularization, a nonzero singular value divides the observation coefficient by σ_i; a tiny value can amplify measurement error. With regularization, the denominator never falls below λ. A nullspace direction receives zero in this fully penalized model.

For $A = \mathrm{diag}(1, 0.1)$ and $b = (1, 0.1)$, the unregularized solution is $(1, 1)$. With $\lambda = 0.1$, the solution becomes $(10/11, 1/11)$. The weakly observed second direction is shrunk much more strongly. This explains both the numerical stability and the introduced bias: regularization does not learn that the second true coefficient is small. It imposes a preference when the observations constrain that direction weakly [1, Section 6.3].

Constraints and penalties are related but different

The constrained model $\min \|Ax - b\|_2^2$ subject to $R(x) \le \rho$ directly specifies an allowable complexity budget. Its Lagrange multiplier can connect it to a penalty formulation under appropriate conditions.

That connection does not mean a particular numerical λ corresponds to an obvious ρ in advance. Nonunique solutions and inactive restrictions can complicate the correspondence. Choose the form that expresses the actual requirement.

Regularization can improve conditioning, but that numerical advantage is distinct from predictive improvement. A well-conditioned biased model may predict poorly. Select tuning parameters with a valid evaluation split when prediction is the goal, and keep the fitting objective separate from the final evaluation metric.

Robust losses and robust measurement models

Plain version. "Robust" can mean reducing the influence of unusually large residuals, or protecting against uncertainty in the measurement operator. These are different models.

One convex residual loss is the Huber function:

$$\phi_\delta(r) = \begin{cases} \frac{1}{2}r^2, & |r| \le \delta, \\ \delta|r| - \frac{1}{2}\delta^2, & |r| > \delta. \end{cases}$$

r is a scalar residual and $\delta > 0$ the transition scale. The derivative equals r near zero and saturates at magnitude δ outside. Summing this loss over affine residuals gives a convex problem [1, Section 6.1].

The transition has the units of the residual. Multiplying all observations by a scale while leaving δ unchanged alters which points receive reduced influence. The loss is not immune to every corruption pattern; high-leverage errors in the design matrix require separate attention.

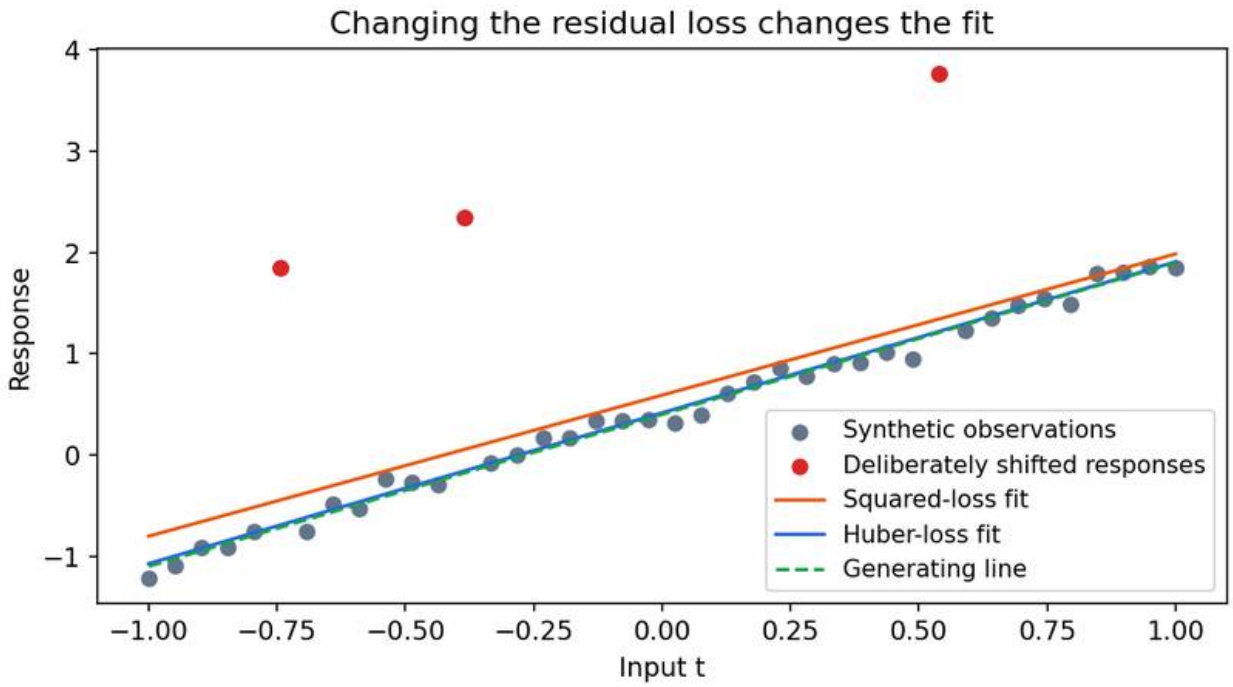


Figure 1 — Original synthetic observations and fitted solutions. Some responses were deliberately shifted upward. This illustrative example compares residual losses; it is not a benchmark or a general robustness guarantee.

Code: fit a line with two losses

The data and shift are generated locally. The smooth optimization call follows the official SciPy interface [2]. We compare solutions and verify the robust objective's gradient instead of asserting

that one estimator always wins.

```
import numpy as np
from scipy.optimize import minimize

rng = np.random.default_rng(15)
t = np.linspace(-1, 1, 40)
A = np.column_stack([np.ones_like(t), t])
b = 0.4 + 1.5*t + rng.normal(0, 0.08, len(t))
b[[5, 12, 30]] += 2.5
least_squares = np.linalg.lstsq(A, b, rcond=None)[0]
delta = 0.2

def huber_objective(x):
    r = A @ x - b
    loss = np.where(abs(r) <= delta, 0.5*r*r,
                    delta*abs(r) - 0.5*delta**2)
    return loss.sum(), A.T @ np.clip(r, -delta, delta)

fit = minimize(huber_objective, least_squares, jac=True,
               method="BFGS", options={"gtol": 1e-9})
assert np.linalg.norm(huber_objective(fit.x)[1]) < 1e-6
print("squared-loss coefficients:", np.round(least_squares, 4))
print("Huber-loss coefficients:", np.round(fit.x, 4))
print("robust objective:", round(fit.fun, 6))
```

Uncertain operators

Suppose the matrix itself may vary in a specified set $\mathcal{A}$. A worst-case approximation model is

$$\min_{x} \sup_{A \in \mathcal{A}} \|Ax - b\|.$$

x is the decision, b the fixed target, and $\mathcal{A}$ the allowed measurement matrices. Each fixed-matrix norm objective is convex in x, so their pointwise supremum is convex [1, Section 6.4].

Convexity does not guarantee that evaluating this supremum is easy. A finite uncertainty set can yield finitely many epigraph restrictions. Structured ellipsoidal uncertainty can permit conic reformulations. A vaguely described "small perturbation" is not enough to select the correct counterpart.

An expected squared-error model is different. If $A = \bar{A} + U$, with $\mathbb{E}[U] = 0$, then

$$\mathbb{E}\|Ax - b\|_2^2 = \|\bar{A}x - b\|_2^2 + x^\top P x, \qquad P = \mathbb{E}[U^\top U].$$

$\bar{A}$ is the mean operator, U its random variation and P a positive-semidefinite second-moment matrix. The cross term vanishes under the zero-mean assumption. Here regularization follows from a specified uncertainty model, rather than an arbitrary complexity preference.

Worst-case and expected protection answer different questions. An uncertainty set is not a probability distribution. A covariance model does not promise control of every possible realization.

Function fitting with shape constraints

Plain version. We can ask a fitted function to be increasing or convex while keeping the optimization convex in its coefficients.

For fixed basis functions $\psi_j(t)$,

$$f(t) = \sum_{j=1}^{n} x_j \psi_j(t).$$

t is the input, ψ_j are known basis functions, and x_j are fitted coefficients. Evaluating at fixed sample inputs makes the predictions affine in x, even when the basis functions are nonlinear in t.

This distinction matters: convexity in the optimization variables is not the same as convexity in the function's input. A polynomial regression may be a convex coefficient-fitting problem while the fitted polynomial is neither increasing nor convex.

At ordered knots $t_1 < \cdots < t_m$, let $z_i = f(t_i)$ be fitted values. Monotonicity can be imposed by $z_{i+1} \geq z_i$. Convexity of the piecewise-linear interpolant requires

$$\frac{z_{i+1} - z_i}{t_{i+1} - t_i} \leq \frac{z_{i+2} - z_{i+1}}{t_{i+2} - t_{i+1}}.$$

z_i are variables and the knot spacings are fixed positive numbers. These slope-order inequalities are linear in fitted values. Unlike sampling a derivative of an arbitrary flexible function, they guarantee the shape of this particular interpolant throughout its knot interval.

Code: monotone absolute-error fitting as an LP

We fit five ordered measurements. The auxiliary variables represent absolute residual bounds. SciPy's LP bounds are explicitly specified [3], including unrestricted fitted values.

```
import numpy as np
from scipy.optimize import linprog

observed = np.array([0., 2., 1., 3., 4.])
n = len(observed)
I = np.eye(n)
order = np.zeros((n-1, n))
for i in range(n-1):
    order[i, i], order[i, i+1] = 1, -1
G = np.vstack([np.hstack([I, -I]),
               np.hstack([-I, -I]),
               np.hstack([order, np.zeros((n-1, n))])])
h = np.r_[observed, -observed, np.zeros(n-1)]
fit = linprog(np.r_[np.zeros(n), np.ones(n)], A_ub=G, b_ub=h,
              bounds=[(None, None)]*n + [(0, None)]*n,
              method="highs")
assert fit.success
z = fit.x[:n]
```

```
assert np.all(np.diff(z) >= -1e-9)
assert np.isclose(abs(z-observed).sum(), fit.fun)
print("monotone fitted values:", z)
print("total absolute residual:", fit.fun)
```

Absolute-error solutions may be nonunique. The code verifies the property and objective rather than requiring one particular plateau value. Adding a strictly convex secondary objective would select one solution, but would be a different stated problem.

Constraints outside the observed interval require care. Piecewise-linear interpolation supplies a clear in-range function; extrapolation is an additional rule. Shape restrictions can improve plausibility while introducing bias if the underlying process violates them.

Likelihood and convex parameterizations

Plain version. A likelihood model can generate a convex objective, but the chosen parameterization and domain determine whether that is true.

For independent observations y_i under a distribution with natural parameter θ,

$$p(y \mid \theta) = \exp(\theta^\top T(y) - A(\theta) + c(y)).$$

$T(y)$ is a sufficient-statistic vector, $A(\theta)$ the log-partition function and $c(y)$ a parameter-independent base term. For an exponential-family model, A is convex on its natural domain.

Ignoring terms independent of θ, the negative log-likelihood is

$$\ell(\theta) = NA(\theta) - \theta^\top \sum_{i=1}^{N} T(y_i).$$

N is the number of observations. This is convex in the natural parameter. An arbitrary nonlinear change of parameter does not necessarily preserve convexity [1, Section 7.1].

For Bernoulli observations with success probability $p \in (0, 1)$ and k successes out of N, the negative log-likelihood is $-k \log p - (N - k) \log(1 - p)$. Its interior optimum is $p = k/N$ when both outcomes are observed.

If every observation is a success or every observation a failure, the optimum over the open domain may only be approached at a boundary. Allowing boundary distributions or adding a prior changes the existence question. A program that clips probabilities should record that clipping as a domain choice.

Maximum a posteriori estimation adds the negative log prior. A convex negative log prior preserves convexity in the selected coordinates. “Bayesian” alone does not guarantee a convex optimization problem, and a point estimate does not represent the entire posterior uncertainty.

Nonparametric probabilities

For discrete outcomes, estimate a probability vector q on the simplex:

$$q_j \geq 0, \qquad \sum_j q_j = 1.$$

q_j is the probability of outcome j. Given counts n_j, the negative log-likelihood is $-\sum_j n_j \log q_j$, with the appropriate extended-value convention for positive counts at zero probability.

Known moments become affine restrictions, such as $\sum_j q_j a_j = \mu$ for fixed outcome features a_j and known mean μ. Maximizing entropy, equivalently minimizing $\sum_j q_j \log q_j$, selects a distribution subject to such constraints [1, Section 7.2].

Moment restrictions should come from the same population and units as the data. Inconsistent empirical constraints can make the simplex model infeasible. Softening them adds a modeling decision about acceptable uncertainty, rather than repairing an algebraic nuisance.

Detector design as a linear program

Plain version. Choosing decisions after observing discrete evidence can be optimized directly. Allowing randomized decisions makes the feasible set convex.

Suppose p_{kj} is the known probability of observation k under hypothesis j. Let T_{ik} be the probability of deciding hypothesis i after seeing k.

$$T_{ik} \geq 0, \qquad \sum_i T_{ik} = 1, \qquad D_{ij} = \sum_k T_{ik} p_{kj}.$$

D_{ij} is the probability of deciding i when j is true. T is the decision variable; the conditional probabilities p_{kj} are fixed. Each entry of D is linear in T.

Weighted error costs, lower bounds on correct detection and upper bounds on false decisions can therefore be expressed as linear objectives and restrictions [1, Section 7.3]. A prior distribution on hypotheses can weight overall expected error, but should not be silently substituted for a conditional-error guarantee.

For binary hypotheses, likelihood-ratio threshold rules emerge from suitable optimal detector formulations. A boundary observation may need randomization to meet an exact error budget. Deterministic rules are a restricted subset; forcing them introduces discrete choices.

An optimized detector is only as valid as its conditional distributions. Distribution shift, estimated likelihoods and calibration uncertainty belong in the assessment. Numerical optimality under a specified probability table does not validate that table.

Worked detector: why randomization can be useful

A false-positive budget can fall between two deterministic rules. Suppose the observation is either a or b. Under hypothesis H_0, its probabilities are $(0.8, 0.2)$; under H_1, they are $(0.2, 0.8)$. These are

synthetic conditional probability tables, not measured frequencies.

Let $t_a, t_b \in [0,1]$ be the probabilities of deciding H_1 after each observation. Maximizing true-positive probability with false-positive budget 0.1 is the LP

$$\begin{aligned} \text{maximize}_{t_a,t_b} \quad & 0.2t_a + 0.8t_b \\ \text{subject to} \quad & 0.8t_a + 0.2t_b \le 0.1, \\ & 0 \le t_a, t_b \le 1. \end{aligned}$$

Here the objective is detection probability under H_1, and the first restriction is error probability under H_0. Spending one unit of error budget on b yields four units of detection probability, whereas spending it on a yields only one quarter. Allocate the budget to b: $t_a = 0$, $t_b = 1/2$, giving true-positive probability 0.4.

A deterministic decision on b would cost 0.2, beyond the budget. With this discrete observation alphabet, every nontrivial deterministic rule violates the stated budget, so the only allowed deterministic rule always decides H_0 and detects nothing. Randomization fills the convex space between deterministic rules and achieves the desired operating point [1, Section 7.3]. This is randomizing a decision after a known observation, not treating unknown probabilities as decision variables.

Moment and exponential risk bounds

Plain version. Sometimes we know moments but not the whole distribution. Bounds can control a tail event conservatively, with assumptions stated explicitly.

For a scalar random variable Y with mean μ and finite variance σ^2, Chebyshev's inequality gives

$$\Pr(|Y - \mu| \ge a) \le \frac{\sigma^2}{a^2}, \qquad a > 0.$$

a is a deviation threshold. The bound uses only the first two moments and can be loose. An upper bound above one can be replaced by one; it is not an estimated event probability [1, Section 7.4].

For a threshold b, an exponential or Chernoff bound uses

$$\Pr(Y \ge b) \le \inf_{s>0} \exp(-sb)\, \mathbb{E}[\exp(sY)].$$

s is a positive auxiliary parameter, and the expectation must be finite where used. Optimizing its logarithm gives $\inf_{s>0}\{\log \mathbb{E}[\exp(sY)] - sb\}$.

The log moment-generating function is convex in s on its domain. This does not automatically make every joint design-and-risk problem convex in all its variables. Check how the random variable depends on decisions and which distribution assumptions are retained.

Tail bounds can support conservative design constraints. They do not replace checking whether moments were reliably estimated. Heavy tails, dependence and finite-sample uncertainty can make an assumed moment-generating function unsuitable.

Choosing the next measurements

Plain version. Estimation quality depends on what we measure, not only on how we fit. Convex experiment design allocates continuous measurement effort before any discrete rounding.

For candidate measurement vectors $a_j \in \mathbb{R}^n$, independent unit-variance noise and normalized weights w_j, define

$$M(w) = \sum_j w_j a_j a_j^\top, \qquad w_j \geq 0, \quad \sum_j w_j = 1.$$

M is the information matrix per normalized measurement effort. The covariance of a corresponding linear estimator scales with its inverse when the matrix is positive definite, the measurement model is correct and total effort is accounted for.

D-optimal design minimizes $-\log\det M$, which targets an uncertainty-volume criterion. A-optimal design minimizes $\operatorname{tr}(M^{-1})$, targeting total coordinate variance. E-optimal design maximizes the smallest eigenvalue of M, targeting the weakest measured direction [1, Section 7.5].

These are different scalar summaries. Units and coordinate changes can affect their interpretation differently. A small total variance may hide one poorly measured direction; a small uncertainty volume does not promise a bound on every coordinate.

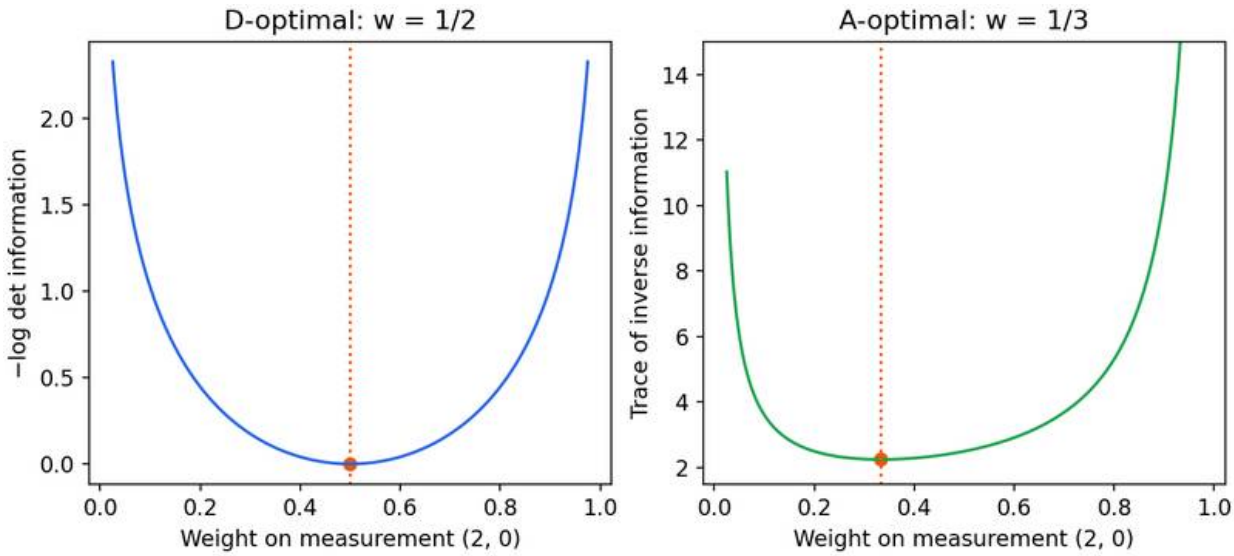


Figure 2 — Original analytical two-direction design. The D-optimal and A-optimal allocations differ because volume and total coordinate variance are different goals. The curves assume independent unit-variance noise and positive effort in both directions.

Code: two design criteria with exact checks

Use candidate vectors $(2,0)$ and $(0,1)$, with first weight w and second weight $1-w$. The information matrix is diagonal: $M = \mathrm{diag}(4w, 1-w)$. Both criteria diverge at an unobserved direction.

```
import numpy as np
from scipy.optimize import minimize_scalar

def d_loss(w):
    return -np.log(4*w) - np.log(1-w)

def a_loss(w):
    return 1/(4*w) + 1/(1-w)

d = minimize_scalar(d_loss, bounds=(1e-6, 1-1e-6), method="bounded")
a = minimize_scalar(a_loss, bounds=(1e-6, 1-1e-6), method="bounded")
assert np.isclose(d.x, 0.5, atol=1e-5)
assert np.isclose(a.x, 1/3, atol=1e-5)
print("D-optimal first weight:", round(d.x, 6))
print("A-optimal first weight:", round(a.x, 6))
```

With a fixed number of experiments, actual counts are integers. Continuous weights are a relaxation or a randomized sampling policy. Rounding can destroy a budget or leave a direction unmeasured, so reassess the resulting matrix after converting weights into a schedule.

If candidate vectors do not span the parameter space, no weighting makes the information positive definite. Adding a prior matrix may regularize the design, but changes its interpretation to prior-plus-measurement information.

Derive why the two design criteria allocate differently

The two-direction experiment can be solved before running numerical code. With candidate vectors $(2,0)$ and $(0,1)$ and weights $w, 1-w$, its information matrix is $M = \mathrm{diag}(4w, 1-w)$. Let $0 < w < 1$ so both directions are measured.

The D-optimal objective and its derivative are

$$f_D(w) = -\log(4w) - \log(1-w), \qquad f_D'(w) = -\frac{1}{w} + \frac{1}{1-w}.$$

Here f_D is the negative log-determinant criterion. Stationarity gives $w = 1/2$. The strength factor four changes the objective by a constant and does not change this D-optimal allocation.

For A-optimal design,

$$f_A(w) = \frac{1}{4w} + \frac{1}{1-w}, \qquad f_A'(w) = -\frac{1}{4w^2} + \frac{1}{(1-w)^2}.$$

Here f_A is the inverse-information trace criterion. Positive denominators turn its stationary equation into $1 - w = 2w$, giving $w = 1/3$. Each scalar objective has positive second derivative and diverges at the endpoints, so these stationary points are the unique global minima [1, Section 7.5].

The interpretation is useful: the first measurement already observes its direction more strongly, so minimizing total variance sends more effort to the weaker second direction. Minimizing uncertainty volume balances multiplicative directional information instead and gives equal effort. If measurement cost or noise variance differs, incorporate those changes in the information matrix and budget before repeating the derivation. Neither allocation is universally best; each answers its chosen uncertainty question.

A fitting and design audit

Start with the application quantity that must be accurate. Decide whether it is a prediction, parameter component, worst residual or a distributional probability. Then state the representation and units before selecting a numerical loss.

For each constraint, ask whether it is physically required, statistically estimated or introduced for numerical convenience. Those roles imply different responses to infeasibility. A physical limit may need a revised design; an empirical moment may need uncertainty; redundant algebra may need removal.

After fitting, verify domain membership, shape restrictions and solver residuals. Then assess generalization or uncertainty using an appropriate independent procedure. An optimum of a training objective and a reliable future prediction are separate claims.

For design, inspect the information spectrum, available budget, noise assumptions and schedule conversion. Report the final implementable schedule alongside the relaxed objective. This keeps the most consequential approximation visible.

What an uncertainty ellipsoid actually says

An inverse information matrix describes a covariance only under the corresponding measurement and estimator assumptions. Turning it into a region with a stated coverage probability needs a distributional argument and a correctly chosen threshold. Drawing an ellipse from that matrix alone does not establish a confidence level.

For a positive-definite covariance C, an ellipsoid can be written as $(x - \hat{x})^\top C^{-1}(x - \hat{x}) \le c$. Here $\hat{x}$ is a center and $c > 0$ is a threshold. The matrix specifies shape; the threshold specifies size. Coverage additionally depends on the actual distribution of estimation error.

A prior covariance, a sampling covariance and a numerical Hessian inverse need not have the same interpretation. They can coincide under particular models, but should not be exchanged without checking those models. Likewise, an estimator's optimization tolerance and its sampling uncertainty have different units and sources.

When measurements are repeated, independent equal-variance assumptions justify a simple additive information model. Correlated repeats can provide less new information than their count

suggests. A design that ignores that correlation may confidently allocate effort to redundant measurements.

This distinction affects stopping precision. Solving far below the uncertainty in the input coefficients may offer little practical value, while violating a hard physical bound by a seemingly small amount may still be unacceptable. Set optimization tolerances against the relevant application scale, rather than choosing precision solely from the number of available floating-point digits.

Finally, uncertainty visualization should state whether it displays an analytical model, an estimated covariance or repeated synthetic experiments. The figures in this chapter show illustrative fits and objective curves. They do not display empirically calibrated coverage regions or claim a particular confidence probability.

Modern view

Plain version. Current work combines structured convex estimation with large-scale first-order computation and learned convex components. The convexity guarantee belongs to the stated variables and domains.

Pokutta's 2024 survey [4] organizes Frank–Wolfe methods around linear minimization over a convex feasible set, convergence guarantees and useful variants. Simplex-constrained experiment design is a natural setting for this viewpoint: an extreme-point search can avoid a full projection. The objective still needs the required smoothness and positive-definite domain control.

Schaller, Bemporad and Boyd's 2025 manuscript [5] studies learning parametric convex functions for subsequent optimization. It connects fitting with disciplined curvature rules: a learned component can preserve convexity in decision variables for valid fixed parameters. Training the component is a separate problem and need not itself be convex.

These sources cover complementary questions: how to move through a structured feasible set, and how to construct functions usable inside such a set. Neither replaces uncertainty modeling, held-out evaluation or the physical feasibility of a design.

Current challenges include reliable uncertainty under model mismatch, informative measurements under resource limits, shape restrictions that remain appropriate outside the sample interval, and scalable updates when observations arrive sequentially. Distinguish a provably convex estimator from a complete validated statistical pipeline.

Key takeaways

- Residual norms, parameter norms and evaluation metrics answer different questions.
- Regularization weights depend on objective normalization and feature units.
- Robust residual losses and uncertain-operator models protect against different errors.
- Fixed nonlinear bases can still yield convex coefficient fitting.
- Likelihood convexity depends on parameterization and domain.
- Randomized discrete-evidence detectors can be formulated as LPs.

- Experiment-design criteria differ, and continuous weights require an implementation plan.

Exercises

1. Formulate maximum absolute residual fitting as an LP, including unrestricted parameters.

   Hint

   Use one nonnegative residual bound.

2. Scale observations and predictions by ten. Determine how a Huber transition and a quadratic regularization weight should change to preserve a chosen equivalent objective.

   Hint

   Write the scaled objective explicitly; the answer depends on whether parameters and penalty units are also scaled.

3. Add convexity constraints to the five-knot monotone-fitting example, using unequal positive knot spacings.

   Hint

   Order adjacent slopes.

4. Derive the Bernoulli maximum-likelihood estimate, and explain the open-domain boundary cases.

   Hint

   Set the likelihood derivative to zero, then inspect endpoints.

5. Construct a two-hypothesis randomized detector LP with a stated false-positive budget.

   Hint

   Use decision probabilities per observation, and conditional error as a linear expression.

6. Derive the two-direction D- and A-optimal weights in the code, then explain why rounding a small measurement budget needs another check.

   Hint

   Differentiate the scalar criteria; inspect whether each rounded direction receives a measurement.

References

1. Stephen Boyd and Lieven Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004. **Reading: Chapter 6, Sections 6.1–6.5; Chapter 7, Sections 7.1–7.5.** Official book, free PDF.
2. SciPy developers, "scipy.optimize.minimize" and "scipy.optimize.minimize_scalar," *SciPy API documentation*, accessed 2026. Multivariate interface, scalar interface.
3. SciPy developers, "scipy.optimize.linprog," *SciPy API documentation*, accessed 2026. Official documentation.
4. Sebastian Pokutta, "The Frank-Wolfe Algorithm: A Short Introduction," *Jahresbericht der Deutschen Mathematiker-Vereinigung*, 126, 2024. Open-access survey.
5. Michael Schaller, Alberto Bemporad and Stephen Boyd, "Learning Parametric Convex Functions," manuscript, 2025. Official manuscript page.

OPT 06 • Gradient Descent, Newton Steps and Numerical Reliability

Derive descent directions and line searches, compare gradient and Newton methods, interpret convergence assumptions, and implement a stable convex logistic solver.

Online edition: https://blog.shyandram.dev/en/learn/optimization/opt-06-gradient-newton/

What you'll learn

- Separate a search direction from the step length chosen along it.
- Derive a safe gradient step under a Lipschitz-gradient assumption.
- Implement Armijo backtracking with domain and iteration safeguards.
- Explain conditioning, norm-dependent steepest descent and Newton curvature.
- Interpret a Newton decrement and the assumptions behind rapid local convergence.
- Verify derivatives, linear solves and stopping evidence in a CPU example.

The big picture

Convexity tells us which answers are globally optimal. An algorithm tells us how to move toward one using information that can be computed. Those are complementary questions: a convex problem can still be badly scaled, expensive or numerically delicate.

Boyd and Vandenberghe's Chapter 9 [1] develops unconstrained descent methods, gradient descent, steepest descent, Newton's method, self-concordance and implementation. This chapter follows that progression, keeping the cost of each step and the meaning of a stopping test visible.

> **Explain like I'm five**
>
> You are walking down a hill. A slope arrow tells you which way goes down. A map of how the hill bends can suggest a smarter route. Either way, you still check that the next step lands safely and actually lowers your height.

The examples solve deterministic convex objectives. Their guarantees should not be silently transferred to a neural network's nonconvex training objective, a noisy derivative or a nonsmooth penalty. Useful ideas can transfer, but the proof assumptions must be checked again.

We first use a quadratic with an exact solution, then implement a smooth regularized logistic objective from scratch. Both run on a CPU with locally generated data and no solver package beyond NumPy.

A direction and a distance

Plain version. A descent iteration chooses where to point, then how far to travel. Combining those choices too early hides why a step is safe.

Write an iteration as

$$x_{k+1} = x_k + t_k d_k.$$

x_k is the current decision vector, d_k a search direction and $t_k > 0$ the step length. For a differentiable objective f, a strict descent direction satisfies $\nabla f(x_k)^\top d_k < 0$.

The inner product is the directional derivative at zero step. By differentiability, a sufficiently small positive step lowers the objective and remains in an open domain around the current point. A large step can still increase it or leave the domain.

The direction may depend on a gradient, a Hessian, a metric or other structure. The line search evaluates candidate distances. Counting only accepted iterations can hide many rejected objective evaluations.

If the gradient is zero for a differentiable convex objective on an open convex domain, the supporting-plane inequality certifies a global minimizer. With constraints, zero unconstrained gradient is generally not the correct optimality condition; the next chapter uses constrained systems.

Exact and inexact line searches

An exact line search minimizes $f(x + td)$ over allowed positive distances. That auxiliary one-dimensional problem can be costly or unnecessary.

An inexact search accepts sufficient decrease rather than solving the line problem precisely. It should also preserve the objective's domain. For a logarithmic term, testing a candidate with an invalid argument is a failed domain test, not evidence about its objective.

Floating-point evaluation adds another distinction: a mathematically decreasing step can become indistinguishable from the current value at machine precision. Repeatedly shrinking until a comparison happens to pass is not a robust termination rule.

Gradient descent and its assumptions

Plain version. Gradient descent points opposite the gradient. A curvature bound tells us how small a fixed step must be.

The update is

$$x_{k+1} = x_k - \eta \nabla f(x_k).$$

$\eta > 0$ is the step size. Assume the gradient is Lipschitz with constant L on the relevant region:

$$\|\nabla f(y) - \nabla f(x)\|_2 \le L\|y - x\|_2.$$

x, y are points whose connecting segment stays in that region. L measures how rapidly the gradient changes; it is not a regularization coefficient or a loss value.

The smoothness inequality is

$$f(x + d) \le f(x) + \nabla f(x)^\top d + \frac{L}{2}\|d\|_2^2.$$

d is a trial displacement. Substituting $d = -\nabla f(x)/L$ gives

$$f\left(x - \frac{1}{L}\nabla f(x)\right) \le f(x) - \frac{1}{2L}\|\nabla f(x)\|_2^2.$$

The step $1/L$ therefore supplies descent whenever the assumptions and domain requirement hold. For a twice-differentiable objective, an upper Hessian eigenvalue bound can establish such an L [1, Section 9.3].

When only convexity and smoothness are assumed, objective convergence is typically sublinear under the standard fixed-step analysis. Strong convexity adds a positive lower curvature bound and permits linear geometric convergence. Those words describe mathematical rates, not a promise that every implementation will be fast.

From one-step decrease to a convergence rate

Descent proves that the objective improves; a convergence rate quantifies how much work reaches a target error. For a convex differentiable function on $\mathbb{R}^n$ with L-Lipschitz gradient and an attained minimizer x^*, use $x^+ = x - \nabla f(x)/L$. The gradient supporting bound gives

$$f(x) - f(x^*) \le \nabla f(x)^\top (x - x^*).$$

Here x is the current point, x^+ its next iterate and x^* any minimizer. Expanding the squared distance after the update gives

$$\|x^+ - x^*\|_2^2 = \|x - x^*\|_2^2 - \frac{2}{L}\nabla f(x)^\top (x - x^*) + \frac{1}{L^2}\|\nabla f(x)\|_2^2.$$

All symbols retain their definitions above. Combine the first inequality with the earlier smoothness decrease bound to obtain

$$f(x^+) - f(x^*) \le \frac{L}{2}\left(\|x - x^*\|_2^2 - \|x^+ - x^*\|_2^2\right).$$

Summing this inequality over the first k updates cancels the intermediate distances. Since objective values decrease, the last error is no larger than the average of those k errors:

$$f(x_k) - f(x^*) \le \frac{L\|x_0 - x^*\|_2^2}{2k}, \qquad k \ge 1.$$

Here x_0 is the start and x_k the point after k updates. This is the concrete meaning of an $O(1/k)$ objective rate: halving the guaranteed error generally requires twice as many iterations under this

bound. It is a worst-case guarantee with these assumptions, not a claim about every observed trace [1].

Strong convexity and a stopping bound

If f is μ-strongly convex, its supporting model includes $\frac{\mu}{2}\|y - x\|_2^2$. Minimizing that lower model yields

$$f(x) - f^* \leq \frac{\|\nabla f(x)\|_2^2}{2\mu}.$$

f^* is the global minimum and $\mu > 0$ a valid strong-convexity constant. A small gradient then gives an objective-gap bound with known units.

Without a verified positive μ, the same formula is not a certificate. A small gradient can coexist with a large distance to the solution on a very flat objective. A small parameter update can instead reflect an excessively small step.

Avoid stopping solely because several printed objective values look identical. Check a suitable residual, the model's scale, the progress trend and an iteration or time limit.

Conditioning changes the path

Plain version. An elongated bowl has steep and flat directions. A step safe for the steep direction can make progress painfully slow along the flat one.

For a positive-definite quadratic,

$$f(x) = \frac{1}{2}x^\top Q x + c^\top x, \qquad \nabla f(x) = Qx + c.$$

Q is symmetric positive definite and c fixed. The Hessian is Q. Its smallest and largest eigenvalues supply μ and L.

The ratio $\kappa = L/\mu$ describes this Euclidean curvature conditioning. A large ratio means a single scalar step size must accommodate very different directional scales. Coordinate units can therefore change the behavior of gradient descent substantially.

Standardizing variables or choosing a suitable metric can improve a path, but must transform constraints, coefficients and stopping norms consistently. A rescaled algorithm is not judged fairly by an untransformed parameter-distance threshold.

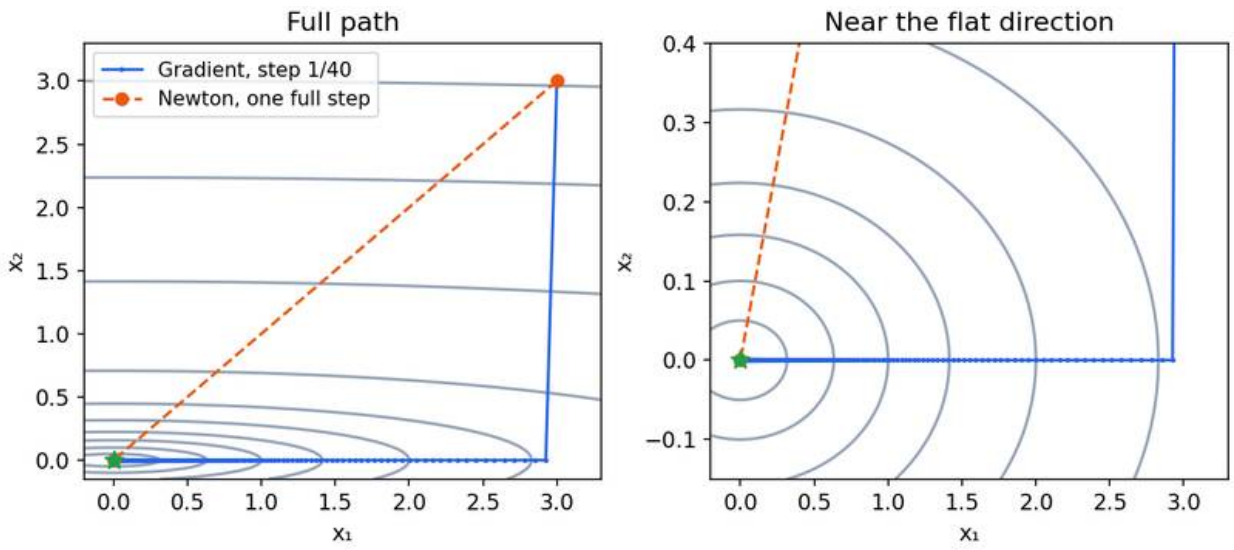


Figure 1 — Original analytical quadratic $f(x) = (x_1^2 + 40x_2^2)/2$. The gradient path uses step 1/40; Newton reaches the exact quadratic minimizer in one full step. This example isolates conditioning and does not compare total runtime on general problems.

Code: compare a fixed gradient step and a Newton step

The objective is minimized at zero. We compare objective decrease and verify the Newton linear system rather than using an explicit matrix inverse.

```
import numpy as np

Q = np.diag([1., 40.])
x = np.array([3., 3.])
start = x.copy()
values = [0.5*x @ Q @ x]
for _ in range(200):
    x = x - (Q @ x)/40
    values.append(0.5*x @ Q @ x)
direction = np.linalg.solve(Q, -(Q @ start))
newton = start + direction
assert np.all(np.diff(values) <= 1e-12)
assert np.allclose(Q @ direction + Q @ start, 0)
assert np.linalg.norm(newton) < 1e-12
print("gradient objective after 200 steps:", round(values[-1], 8))
print("Newton objective after one step:", 0.5*newton @ Q @ newton)
```

A one-step Newton solution is special to an exact positive-definite quadratic with an accurate linear solve. On a general smooth function, curvature changes as we move and a full step may fail.

Read the quadratic path one eigenvector at a time

A quadratic makes the effect of the step size exact. Let $e_k = x_k - x^*$ be the error for a positive-definite quadratic with curvature Q. Gradient descent gives $e_{k+1} = (I - \eta Q)e_k$. In an eigenvector of Q with eigenvalue q_i, the error coefficient is multiplied by $1 - \eta q_i$.

Here η is the fixed step, I the identity, and $q_i > 0$ the directional curvature. All coefficients contract precisely when $0 < \eta < 2/L$, where L is the largest eigenvalue. If $\eta > 2/L$, the steepest-direction error grows in magnitude even though the objective is convex.

For the chapter's $Q = \mathrm{diag}(1, 40)$ and $\eta = 1/40$, the steep coordinate disappears after one update, while the flat coordinate is multiplied by $39/40$ each time. Starting from $(3, 3)$ therefore leaves first-coordinate error $3(39/40)^k$ after k updates. The gradual path is a consequence of directional curvature, not random numerical failure.

For a quadratic with smallest eigenvalue μ, choosing $\eta = 2/(L+\mu)$ balances the magnitudes of the extreme contraction factors. Their worst value is $(L-\mu)/(L+\mu)$. This choice improves the bound relative to a conservative step, but relies on known spectral endpoints and does not replace a line search for arbitrary nonquadratic functions [1].

Armijo backtracking

Plain version. Start with a candidate distance and shrink it until the decrease is large enough relative to the local slope.

The sufficient-decrease condition is

$$f(x + td) \le f(x) + \alpha t \nabla f(x)^\top d.$$

d is a descent direction, $t > 0$ the tested distance, and $\alpha \in (0, 1)$ a fixed decrease fraction. Backtracking repeatedly multiplies t by $\beta \in (0, 1)$ until the domain and decrease tests pass [1, Section 9.2].

For Newton methods, a usual choice keeps $\alpha < 1/2$ so full steps are accepted near a regular optimum under the standard local analysis. This is an assumption-aware choice, not a universal optimal parameter.

An implementation needs a maximum number of trials, a minimum meaningful step, finite-value checks and a clear failure report. Otherwise an invalid direction or an incorrect gradient can produce an endless shrinking loop.

What a rejected step tells us

A rejection can mean the initial distance was too ambitious, the curvature model was inaccurate far from the solution, the domain was crossed or the derivatives were wrong. It does not by itself identify which reason applies.

Check the directional derivative first. If it is nonnegative, ordinary descent backtracking lacks its intended justification. A modified direction may be needed, or the candidate may already meet an optimality test.

Repeated rejection near an optimum can come from floating-point cancellation. Compare objective and derivative scales before demanding an improvement below representable precision. Termination should explain which numerical condition was met.

Steepest descent depends on a norm

Plain version. "Steepest" requires a definition of one unit of movement. Different norms choose different directions.

The normalized steepest direction solves

$$\min_{\|d\| \le 1} \nabla f(x)^\top d.$$

d is the direction, $\|\cdot\|$ the chosen movement norm, and x the current point. Under the Euclidean norm, a nonzero gradient gives direction $-\nabla f(x)/\|\nabla f(x)\|_2$.

For a positive-definite metric matrix P, define $\|d\|_P = \sqrt{d^\top P d}$. A corresponding unnormalized descent direction is

$$d = -P^{-1}\nabla f(x).$$

P determines directional cost. The directional derivative is $-\nabla f(x)^\top P^{-1}\nabla f(x)$, strictly negative for a nonzero gradient.

Choosing P resembles preconditioning: we account for known scale or curvature without necessarily forming the true Hessian. The usefulness depends on how cheaply the metric system can be solved and how well it represents the problem.

With a coordinate-based movement norm, steepest directions may select a coordinate or distribute movement differently. The book's treatment [1, Section 9.4] explains why steepest descent is a family rather than one coordinate-independent procedure.

Newton's local quadratic model

Plain version. Newton uses a quadratic approximation of the objective, then solves for the minimizer of that approximation.

At the current point, the model is

$$m(d) = f(x) + g^\top d + \frac{1}{2} d^\top H d, \qquad g = \nabla f(x),\ H = \nabla^2 f(x).$$

d is the displacement, g the gradient and H the Hessian. When H is positive definite, the model's unique minimizing displacement solves

$$Hd = -g.$$

This is the Newton direction. Compute it through a linear solve or factorization, not by forming an inverse. Positive definiteness ensures a descent direction whenever $g \neq 0$.

Convexity gives a positive-semidefinite Hessian, which may be singular. A full Newton step is not automatically defined. A regularized Hessian, restricted subspace or another direction can be appropriate, but each changes the stated numerical procedure.

The Newton decrement

Define

$$\lambda_N(x)^2 = g^\top H^{-1} g = -g^\top d.$$

λ_N is the Newton decrement and H is assumed positive definite. The quadratic model predicts a decrease of $\lambda_N^2/2$ at its minimizing step.

For an exact quadratic, that prediction equals the objective gap. For a general function, it is a local model quantity; a rigorous gap interpretation needs additional assumptions. Do not label the predicted decrease an unconditional global error certificate.

The direction and decrement transform coherently under an invertible affine change of coordinates when derivatives and solves are exact. This is one reason Newton behaves differently from Euclidean gradient descent. Floating-point conditioning and implementation still matter.

Local convergence is conditional

With suitable smoothness, a nonsingular positive-definite Hessian near the minimizer and an appropriate line search, Newton eventually accepts full steps and can converge quadratically [1, Section 9.5].

Quadratic convergence describes error shrinking approximately as the square of the previous error in a sufficiently small neighborhood. It does not imply that the first few steps from any initial point exhibit that behavior.

Errors in gradients, Hessians and linear solves can limit the rate. Far from the solution, safeguards may be essential. Near the solution, requesting a residual below derivative accuracy can waste work or trigger misleading failures.

Why Newton respects a change of coordinates

Newton adjusts direction using curvature, so changing variable units does not change its ideal physical step. Let $x = Tz+b$ with invertible matrix T and fixed offset b, and define $\tilde{f}(z) = f(Tz+b)$. At corresponding points, the chain rule gives

$$\tilde{g} = T^\top g, \qquad \widetilde{H} = T^\top H T.$$

Here g, H are the gradient and positive-definite Hessian in x coordinates, while tildes denote the same derivatives in z coordinates. The transformed Newton equation is

$$T^\top H T\,\Delta z = -T^\top g \quad \Longrightarrow \quad H(T\Delta z) = -g.$$

Here Δz is the displacement in new coordinates. Uniqueness of the solve gives $T\Delta z = \Delta x$, the original Newton displacement. Its predicted decrease is invariant too: $\tilde{g}^\top \tilde{H}^{-1}\tilde{g} = g^\top H^{-1} g$.

Ordinary gradient descent instead maps its transformed step to $-\eta T T^\top g$, generally a different direction from $-\eta g$. This connects coordinate scaling to the metric in steepest descent. Newton's invariance assumes exact derivatives and solves; an ill-conditioned coordinate transformation can still harm a floating-point implementation [1, Section 9.5].

Self-concordance and controlled curvature

Plain version. Some convex functions control how quickly curvature changes through their own local curvature. This gives a special route to analyzing Newton steps.

For a standard self-concordant one-dimensional function,

$$|f'''(t)| \le 2\big(f''(t)\big)^{3/2}.$$

t is the scalar input and primes are derivatives, with nonnegative second derivative. In multiple dimensions, this condition is applied along lines in the domain [1, Section 9.6].

The logarithmic barrier $-\log t$ on $t > 0$ satisfies the scalar relation. This links unconstrained Newton analysis to the interior-point methods in the next chapter.

Self-concordance is more specific than ordinary smooth convexity. Its usefulness is that local norms can control changes without a single global Euclidean Hessian-Lipschitz constant. The theory includes precise decrement-based regions and complexity statements; those require the proper normalization and barrier assumptions.

A function containing logarithms is not automatically a valid barrier for every constraint representation. Its domain, boundary behavior and scaling must match the optimization problem. Distinguish a smooth likelihood term from a constraint barrier even if both contain a logarithm.

A stable convex logistic solver

Plain version. A complete implementation should evaluate the objective stably, verify its derivatives and stop using an optimality quantity.

For fixed rows $a_i^\top$ and binary outcomes $y_i \in \{0, 1\}$, use

$$f(w) = \frac{1}{N}\sum_{i=1}^{N}\left[\log(1 + \exp(a_i^\top w)) - y_i a_i^\top w\right] + \frac{\rho}{2}\|w\|_2^2.$$

w is the parameter vector, N the observation count and $\rho > 0$ a regularization coefficient. Every coordinate is regularized in this example. The fixed-design logistic likelihood follows the convex-estimation reasoning in [1, Chapter 7].

Let $p_i = 1/(1 + \exp(-a_i^\top w))$. The derivatives are

$$g = \frac{1}{N} A^\top (p - y) + \rho w, \qquad H = \frac{1}{N} A^\top \operatorname{diag}(p_i(1 - p_i)) A + \rho I.$$

A stacks the rows, p, y are observation vectors, and I is the identity. The positive ρI gives a valid global strong-convexity constant $\mu = \rho$.

Use a stable softplus evaluation instead of taking the logarithm of a potentially overflowed exponential. The following bounded synthetic scores permit a direct sigmoid; for unrestricted extreme scores, use a numerically stable sigmoid implementation as well.

Derive the logistic curvature bounds

A stable objective is also easier to optimize when its curvature constants are known. For one score $s = a_i^\top w$, the derivative of $\log(1 + e^s)$ is p_i, and its second derivative is $p_i(1 - p_i)$. Thus subtracting the linear term $y_i s$ gives gradient $(p_i - y_i)a_i$ and Hessian $p_i(1 - p_i)a_i a_i^\top$.

Here a_i is a fixed feature vector, w the parameter vector, $y_i \in \{0, 1\}$ the observed label and p_i the sigmoid probability. Average over observations and add the quadratic penalty to obtain the derivatives displayed above. Since $0 \le p_i(1 - p_i) \le 1/4$,

$$\rho I \preceq \nabla^2 f(w) \preceq \left(\rho + \frac{\|A\|_2^2}{4N} \right) I.$$

Here f is the regularized average logistic objective, A the feature matrix, N the observation count, $\rho > 0$ the penalty, and $\|A\|_2$ its largest singular value. The lower bound follows from full-coordinate regularization; the upper bound follows by testing $v^\top A^\top \operatorname{diag}(p_i(1 - p_i))Av$ for arbitrary direction v.

Consequently $\mu = \rho$ and $L = \rho + \|A\|_2^2/(4N)$ are valid global constants. They provide a safe gradient step and an objective-gap stopping bound for this specific model. If the intercept is left unpenalized, the same positive lower bound for every coordinate no longer follows. The mathematical formula, the penalty implementation and the stopping test must agree [1].

Code: Newton with derivative and stopping checks

This is an optimization demonstration, not a classification benchmark. It checks a directional derivative before solving and verifies the final gradient, decrease sequence and strong-convexity gap bound.

```
import numpy as np

rng = np.random.default_rng(16)
```

```
A = rng.normal(size=(160, 5))
truth = np.array([0.8, -0.5, 0.3, 0., 0.4])
prob = 1/(1+np.exp(-(A @ truth)))
y = (rng.random(len(A)) < prob).astype(float)
rho = 0.1

def evaluate(w):
    z = A @ w
    p = 1/(1+np.exp(-z))
    value = np.mean(np.logaddexp(0, z)-y*z) + rho*(w @ w)/2
    gradient = A.T @ (p-y)/len(y) + rho*w
    Hessian = (A.T * (p*(1-p))) @ A/len(y) + rho*np.eye(A.shape[1])
    return value, gradient, Hessian

w = np.zeros(A.shape[1])
direction = rng.normal(size=len(w))
eps = 1e-5
finite_difference = (evaluate(w+eps*direction)[0] -
                     evaluate(w-eps*direction)[0])/(2*eps)
assert np.isclose(finite_difference, evaluate(w)[1] @ direction,
                  atol=1e-8)
history = []
for iteration in range(30):
    value, g, H = evaluate(w)
    history.append(value)
    if np.linalg.norm(g) < 1e-9:
        break
    step_direction = np.linalg.solve(H, -g)
    slope = g @ step_direction
    assert slope < 0
    step = 1.0
    for trial in range(50):
        candidate = w + step*step_direction
        candidate_value = evaluate(candidate)[0]
        if np.isfinite(candidate_value) and candidate_value <= value+0.1*step*slope:
            w = candidate
            break
        step *= 0.5
    else:
        raise RuntimeError("backtracking did not accept a step")
else:
    raise RuntimeError("iteration limit reached")

value, g, H = evaluate(w)
assert np.linalg.norm(g) < 1e-9
assert np.all(np.diff(history) <= 1e-12)
assert np.linalg.eigvalsh(H).min() >= rho-1e-12
print("accepted Newton steps:", len(history)-1)
print("objective:", round(value, 8))
print("gradient norm:", np.linalg.norm(g))
print("objective-gap upper bound:", (g @ g)/(2*rho))
```

The derivative check tests one direction and one point; it cannot prove every derivative correct.

Add checks at representative nonzero points when changing the model. Hessian symmetry, eigenvalue expectations and solve residuals provide additional independent evidence.

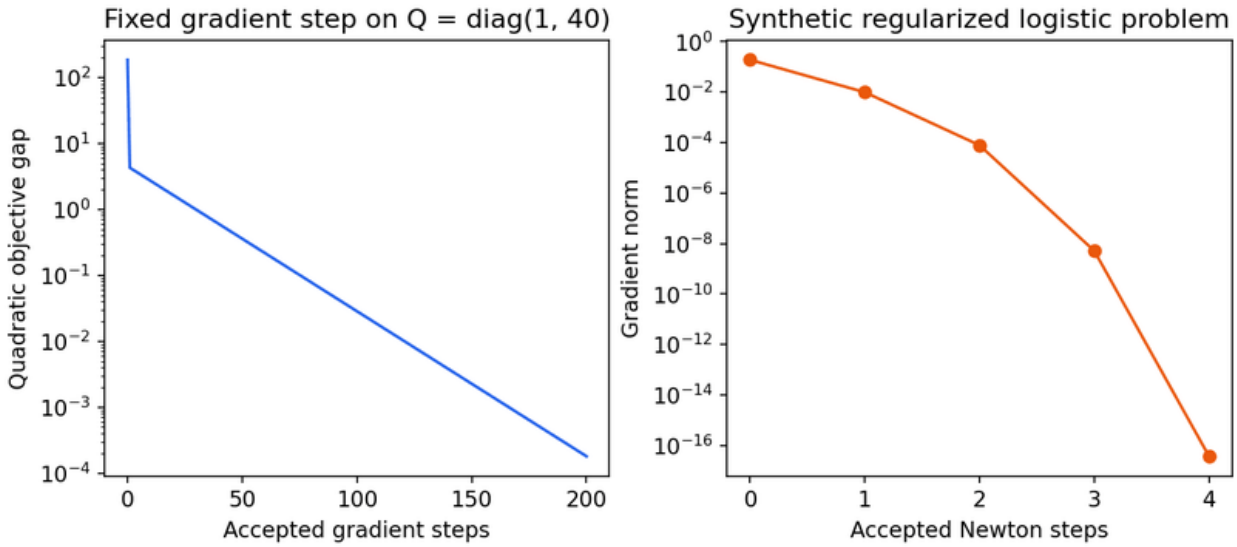


Figure 2 — Original computed convergence traces. The left panel has a known zero quadratic optimum; the right panel shows a gradient norm for a synthetic logistic objective. Accepted steps are not a runtime or cross-problem benchmark.

Implementation and stopping reports

Plain version. A solver result should describe why it stopped, what it verified and what computational work it performed.

Dense Newton stores a matrix whose size grows quadratically with the number of variables. Factorization can dominate the cost. Gradient methods often need only matrix-vector products, making many cheap iterations attractive for large problems.

Sparse matrices, low-rank terms and block structure can alter that comparison substantially. Exploit structure only after checking that the transformed system remains equivalent and that factorization fill-in is manageable [1, Section 9.7 and Appendix C].

A Hessian-vector product can support an iterative linear solve without explicitly storing the Hessian. Then the inner solve has its own residual and tolerance. A loosely solved Newton system need not reproduce the exact Newton direction or convergence rate.

Report objective values, a suitable gradient or stationarity residual, domain checks, iteration count and the stated stopping condition. Where timing matters, include derivative evaluations and line-search trials rather than only accepted steps.

Scaling the objective changes gradient magnitudes. A fixed absolute gradient tolerance is therefore not comparable across arbitrary loss normalization. Relative criteria need a documented scale and should remain meaningful near zero.

An iteration limit is a resource stop, not an optimality certificate. A line-search failure is an algorithmic diagnosis, not proof of nonexistence. Retain the final candidate and residual evidence so that the result can be assessed.

Exact quadratic line searches reveal a tradeoff

For the positive-definite quadratic above and a nonzero descent direction d, the objective along the line is a scalar quadratic. Differentiating with respect to distance gives

$$t^* = -\frac{g^\top d}{d^\top Q d}.$$

$g = Qx + c$ is the current gradient, Q the positive-definite curvature and $t^* > 0$ the minimizing line distance. This formula is an exact line search for this particular quadratic, not a general smooth objective.

For the negative gradient direction, the numerator becomes $g^\top g$ and the denominator is $g^\top Q g$. The chosen distance accounts for curvature along that direction. It can outperform a conservative fixed step without correcting the underlying elongated geometry.

Successive directions can still zigzag. Exact minimization along one line does not imply that the line points directly toward the global minimizer. This makes the roles of direction and distance particularly clear: better distance selection cannot fully replace better directional geometry.

A Newton direction uses the whole quadratic curvature system, reaching the minimizer with distance one in exact arithmetic. A metric approximation may reduce the system cost while giving a less accurate route. The fair comparison includes how the metric is built and reused, not just the length of the visible path.

If a matrix factorization is reusable across many right-hand sides, its initial cost can be amortized. If the matrix changes at each iteration, that advantage may shrink. Likewise, an objective evaluation can be cheap in a tiny tutorial but expensive in an application with a simulation inside it. State the actual computational primitives before making a speed claim.

Derivative checks need a scale

A central finite difference divides the difference of two nearby objective values by twice the perturbation. Making the perturbation smaller reduces the ideal truncation error at first, but eventually amplifies floating-point cancellation.

Use several perturbation sizes and compare relative as well as absolute discrepancies. Scale the direction sensibly, choose test points away from a domain boundary, and inspect whether the result stabilizes over a useful interval. One failed tiny-step comparison may indicate numerical resolution rather than an incorrect derivative.

For a Hessian, compare a gradient difference with a Hessian-vector product. This tests a different layer of the implementation from checking objective differences alone. Also verify symmetry

when the objective is twice continuously differentiable and the intended Hessian should be symmetric.

An automatic derivative system can remove much manual algebra but still differentiate the implemented expression rather than the intended model. Incorrect broadcasting, a misplaced reduction or an accidentally regularized intercept can yield a perfectly consistent derivative of the wrong objective.

Record these checks beside the formulation. The goal is to connect symbols to arrays, not merely to produce a green test. Once the implementation matches the model, numerical tolerances and algorithm assumptions can be assessed meaningfully.

A reproducible comparison also needs a starting point, objective normalization, stopping rule and numerical precision. Changing any of these can alter apparent convergence. Store the parameters and random seed with a synthetic experiment so that a later figure can be regenerated from the same stated problem.

For deterministic algorithms, small platform differences may still affect the final few digits or a marginal line-search comparison. Verify properties with justified tolerances instead of asserting one exact decimal trajectory. The examples here check descent, linear-system consistency and gradient evidence, which are the quantities tied to the formulation.

Modern view

Plain version. Modern convex algorithms trade curvature information, projections, proximal operations and communication costs. The right choice depends on structure and required accuracy.

Dvurechensky, Shtern and Staudigl's first-order survey [2] provides a broad foundation, including non-Euclidean proximal methods, acceleration and projection-free approaches. It was published in 2021; it is a background map rather than a new 2026 result.

Condat and colleagues' 2023 SIAM Review survey [3] unifies proximal splitting for high-dimensional nonsmooth convex problems through primal–dual product spaces and chosen metrics. It explains why adding an absolute-value penalty changes the appropriate primitive: an ordinary smooth Newton formula cannot simply differentiate through the kink.

Pokutta's 2024 Frank–Wolfe survey [4] focuses on linear minimization over constrained sets and its convergence and certificate structure. It complements the gradient/Newton comparison by asking whether a projection or expensive system solve can be avoided.

Together these reviews give a practical taxonomy: smooth versus nonsmooth objectives, inexpensive versus expensive feasible-set operations, Euclidean versus structured geometry, and local curvature versus cheap first-order steps. Recent variants do not remove those modeling distinctions.

Current issues include adaptive methods with reliable certificates, precision limits in large linear systems, communication in distributed problems and differentiating through solver outputs. Match the algorithm to the model, the hardware and the requested error measure; compare total work at a comparable accuracy.

Key takeaways

- A descent direction and a step length are separate choices.
- Fixed gradient steps require a valid curvature and domain argument.
- Poor conditioning slows a scalar-step gradient method.
- Backtracking needs safeguards and correct derivative signs.
- Newton uses a linear solve and has conditional local convergence.
- A decrement is a model quantity unless its gap assumptions are established.
- Reliable reports include stopping evidence and the cost of rejected work.

Exercises

1. Derive the smoothness decrease inequality for a step of size $1/L$.

   Hint

   Substitute the negative scaled gradient into the upper quadratic model.

2. Replace the quadratic eigenvalue 40 by 400. Predict and test how fixed-step gradient progress changes.

   Hint

   The flat-direction contraction depends on the eigenvalue ratio.

3. Implement backtracking for $-\log x + x^2$ on $x > 0$, checking the domain before every objective evaluation.

   Hint

   Reject nonpositive candidates immediately.

4. Verify the logistic gradient in three random directions at a nonzero parameter vector.

   Hint

   Compare central differences and inner products using several finite-difference scales.

5. Show why a singular positive-semidefinite Hessian does not always define a unique Newton direction.

   Hint

   Inspect the linear system's nullspace and consistency.

6. Rescale the logistic objective by one hundred. Determine which stopping quantity and tolerance changes preserve the original requested objective accuracy.

Hint

Transform the gradient, strong-convexity constant and desired gap together.

References

1. Stephen Boyd and Lieven Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004. **Reading: Chapter 9, Sections 9.1–9.7; Appendix C for linear systems; Chapter 7 for the logistic likelihood connection.** Official book, free PDF.
2. Pavel Dvurechensky, Shimrit Shtern and Mathias Staudigl, "First-Order Methods for Convex Optimization," *EURO Journal on Computational Optimization*, 9, 100015, 2021. Verified open manuscript, published article.
3. Laurent Condat, Daichi Kitahara, Andrés Contreras and Akira Hirabayashi, "Proximal Splitting Algorithms for Convex Optimization: A Tour of Recent Advances, with New Twists," *SIAM Review*, 65(2), 375–435, 2023. Published survey, open manuscript.
4. Sebastian Pokutta, "The Frank-Wolfe Algorithm: A Short Introduction," *Jahresbericht der Deutschen Mathematiker-Vereinigung*, 126, 2024. Open-access survey.

OPT 07 • Equality Constraints and Interior-Point Methods

Solve constrained Newton systems, follow the logarithmic central path, distinguish feasibility from strict feasibility, and check primal–dual certificates.

Online edition: https://blog.shyandram.dev/en/learn/optimization/opt-07-constrained-interior-point/

What you'll learn

- Extend Newton's method to a convex objective with affine equalities.
- Derive a logarithmic barrier, its central path and the associated duality gap.
- Implement a small feasible barrier solver with an explicit line search.
- Distinguish feasible, strictly feasible and infeasible constraint systems.
- Interpret primal–dual residuals, complementarity and infeasibility certificates.
- Compare modern interior-point and first-order solver designs using their numerical demands.

The big picture

Constraints describe what a solution is allowed to do. A solver cannot ignore them while improving the objective. Equality constraints restrict every permitted step to a lower-dimensional space; inequality constraints create boundaries the solver must handle. Interior-point methods approach useful boundary solutions through a sequence of problems whose iterates remain inside those boundaries.

> **Explain like I'm five**
>
> Imagine finding the lowest place in a fenced garden. You must stay in the garden, and a painted line might require you to walk along one particular path. Start away from the fence. An imaginary cushion pushes you back whenever you get too close. Gradually make the cushion weaker, so you can approach the low point beside the fence without stepping outside.

The reading is Boyd and Vandenberghe, Chapters 10–11 [1]. Chapter 10 treats equality-constrained minimization; Chapter 11 develops barriers, the central path, initialization and primal–dual methods. Read them after gradient and Newton methods and return to duality and optimality when a certificate seems mysterious. Bubeck's algorithmic monograph [2] supplies a complementary view of complexity and solver families.

This chapter uses synthetic low-dimensional problems so you can check every claim independently. The code is a teaching implementation, with assertions and transparent assumptions. A production conic solver adds scaling, sparse factorizations, numerical regularization, termination policies and infeasibility handling. Understanding the small problem helps explain those engineering choices.

1. Equality constraints change the permitted directions

An affine equality must remain satisfied while the objective decreases. If the current point satisfies $Ax = b$, a step Δx preserves feasibility exactly when $A\Delta x = 0$. This gives a geometric explanation before any matrix derivation: permitted steps lie in the nullspace of the equality matrix.

Consider the convex problem

$$\begin{aligned} \text{minimize}_x \quad & f(x) \\ \text{subject to} \quad & Ax = b. \end{aligned}$$

Here $x \in \mathbb{R}^n$ is the variable; f is a twice-differentiable convex objective; $A \in \mathbb{R}^{p\times n}$ is fixed; and $b \in \mathbb{R}^p$ is fixed. We initially assume A has independent rows and the equality system has a solution. The Lagrange multiplier $\nu \in \mathbb{R}^p$ accounts for the equality restriction.

At a feasible iterate, minimize the quadratic approximation along feasible directions. The resulting Newton system is

$$\begin{bmatrix} H & A^\top \\ A & 0 \end{bmatrix} \begin{bmatrix} \Delta x \\ w \end{bmatrix} = - \begin{bmatrix} \nabla f(x) \\ 0 \end{bmatrix}.$$

Here $H = \nabla^2 f(x)$ is the Hessian; Δx is the proposed primal step; w is the equality multiplier of the local quadratic subproblem; and the zero block has size $p \times p$. The second block equation enforces $A\Delta x = 0$. The multiplier w solves the local model; it is not automatically the increment of a separately maintained multiplier estimate.

The Hessian need only have positive curvature along nonzero feasible directions to make the constrained quadratic model well behaved. A direction excluded by the equalities need not have positive curvature. This distinction explains why a full Hessian can be singular while the constrained minimizer is unique. Check the actual feasible subspace before diagnosing singularity as nonuniqueness.

The block matrix is a Karush–Kuhn–Tucker, or KKT, matrix [1]. It is generally indefinite even when H is positive definite. Do not apply a positive-definite factorization to the entire matrix merely because the original objective is convex. Its structure reflects simultaneous primal and dual equations. For a small dense example, a general linear solve is adequate.

An alternative eliminates equalities. Choose one feasible point x_0 and a matrix F whose columns span the nullspace of A. Write $x = x_0 + Fz$, where $z \in \mathbb{R}^{n-p}$ is the reduced variable. Then solve the unconstrained problem in z. Elimination can simplify a tiny problem but destroy sparsity in a large one. Both formulations express the same geometry; the computational representation determines the cost.

Reduced curvature and block elimination

A singular Hessian need not make the constrained step ambiguous. Minimize $f(x) = x_1^2/2$ under $x_2 = 3$. The Hessian is $\operatorname{diag}(1, 0)$, but the only feasible directions change x_1. The unique minimizer is $(0, 3)$: the equality removes precisely the flat direction.

More generally, let F contain independent columns spanning the nullspace of A. The reduced Hessian is $F^\top HF$. Positive definiteness of this smaller matrix gives a unique local quadratic minimizer along feasible directions, even when H is singular. Here H is the current Hessian and A the full-row-rank equality matrix [1, Chapter 10].

If H itself is positive definite, the KKT step can also be derived by eliminating the primal direction. Write $g = \nabla f(x)$ and use the first block equation:

$$\Delta x = -H^{-1}(g + A^\top w),$$
$$(AH^{-1}A^\top)w = -AH^{-1}g.$$

Here Δx is the step and w the local equality multiplier. The second line follows by substituting the first into $A\Delta x = 0$. The smaller matrix is positive definite when A has independent rows: for nonzero v, its quadratic form is $(A^\top v)^\top H^{-1}(A^\top v) > 0$.

These equations explain the block solve, but the inverse notation is algebraic. Compute products through solves with H and compare the cost of this elimination with a direct sparse KKT factorization. In the singular example above, the nullspace formulation remains valid while this full-Hessian elimination does not.

2. A constrained quadratic with an inspectable certificate

A quadratic objective makes the KKT equations exact rather than approximate. Solving the block system produces a candidate; checking equality feasibility and stationarity determines whether the candidate is an optimality certificate for this convex problem.

For

$$f(x) = \tfrac{1}{2}x^\top Qx + c^\top x,$$

$Q \in \mathbb{R}^{n\times n}$ is a symmetric positive-semidefinite matrix, $c \in \mathbb{R}^n$ is fixed, and x is the variable. An optimal pair (x^*, ν^*) satisfies $Qx^* + c + A^\top \nu^* = 0$ and $Ax^* = b$. The superscript star denotes an optimal value. These equations are sufficient for a feasible convex quadratic; uniqueness requires additional curvature on feasible directions.

```
import numpy as np

Q = np.diag([2.0, 4.0])
c = np.array([-2.0, -1.0])
A = np.array([[1.0, 1.0]])
b = np.array([1.0])
K = np.block([[Q, A.T], [A, np.zeros((1, 1))]])
solution = np.linalg.solve(K, np.concatenate([-c, b]))
x, nu = solution[:2], solution[2:]
primal = A @ x - b
dual = Q @ x + c + A.T @ nu
print("Primal solution:", x)
print("Equality multiplier:", nu)
print("Residual norms:", np.linalg.norm(primal), np.linalg.norm(dual))
```

```
assert np.allclose(x, [5/6, 1/6])
assert np.linalg.norm(primal) < 1e-12
assert np.linalg.norm(dual) < 1e-12
```

The coordinates sum to one. The unconstrained optimum would have coordinates one and one quarter, so it would violate the total constraint. The multiplier adjusts the stationarity equation to account for that restriction. A small objective value alone would not establish this adjustment; the residual checks make it explicit.

Residuals depend on units. Multiplying the equality by a large constant leaves the feasible set unchanged while changing the raw primal residual and multiplier scale. Report the units and scaling of the original constraints, and compare residuals against meaningful tolerances. A solver's internal normalized stopping test should be translated back into the application's measurements.

Do not form a matrix inverse to compute this solution. A linear solve is more direct and usually more stable. In a repeated problem, factorization reuse can reduce work, but only when the matrix structure permits it. Updating a linear term is different from updating the Hessian or the constraint matrix; distinguish these changes when planning warm starts and caching.

For nonlinear objectives, recompute the local curvature and use a feasible line search. A full Newton step can leave the objective's domain even if it preserves the equalities. The domain is an additional restriction: a logarithm or a positive-definite matrix variable is not defined everywhere. The equality equations alone do not protect it.

3. Inequalities become a smooth interior problem

A boundary can be represented by a barrier that grows without bound as the iterate approaches it. The barrier is finite only at strictly feasible points. This lets a smooth constrained Newton method handle the inequality through the objective, while retaining explicit affine equalities.

Consider convex inequalities $f_i(x) \le 0$ for $i = 1, \dots, m$, with affine equalities $Ax = b$. Define

$$\phi(x) = -\sum_{i=1}^{m} \log(-f_i(x)), \qquad F_t(x) = t f_0(x) + \phi(x).$$

Here f_0 is the original convex objective; each f_i is a convex inequality function; m is the number of scalar inequalities; ϕ is the logarithmic barrier; and $t > 0$ controls the relative importance of the objective. The domain requires every $f_i(x) < 0$. The barrier problem minimizes F_t subject to $Ax = b$.

Constraint functions should be expressed consistently in chosen units, or normalized by a positive reference scale before taking a logarithm. Positive rescaling changes the barrier by an additive constant and does not change its minimizer, although finite-precision calculations can still be affected. The mathematical invariance is not a reason to neglect numerical scaling.

For a twice-differentiable inequality, its barrier contribution has gradient $\nabla f_i(x)/(-f_i(x))$ and Hessian

$$\frac{\nabla f_i(x)\nabla f_i(x)^\top}{f_i(x)^2} + \frac{\nabla^2 f_i(x)}{-f_i(x)}.$$

Here ∇f_i is the gradient column vector and $\nabla^2 f_i$ is its Hessian. Both terms are positive semidefinite in the strictly feasible domain because f_i is convex. The expression also shows rapidly increasing curvature near an active boundary, where $-f_i(x)$ becomes small.

An inequality representation affects the path. Repeating an identical constraint repeats its barrier contribution, changing the approximate problem for finite t. The exact feasible set remains unchanged. This illustrates an important separation: equivalent constraint sets need not produce identical intermediate iterates. Canonicalization and scaling are algorithmic decisions with observable effects.

The barrier cannot begin on the boundary. If a nonnegative variable starts at zero, $-\log x$ is undefined. Use a strictly positive initialization or reformulate a variable that is forced to zero by equalities. Adding an arbitrary positive offset to a constraint changes the optimization problem and must not be mistaken for an innocuous implementation fix.

4. The central path links interior points to dual bounds

For each barrier weight, solve the interior problem accurately. The resulting minimizers form the central path. Increasing the weight weakens the barrier relative to the original objective, allowing the solution to approach a constrained optimum on the boundary.

Let $x(t)$ minimize F_t subject to $Ax = b$. Define

$$\lambda_i(t) = \frac{1}{t[-f_i(x(t))]}, \qquad \nu(t) = \frac{w(t)}{t}.$$

Here $\lambda_i(t) > 0$ is the inequality multiplier; $w(t)$ is the equality multiplier for the scaled barrier problem; and $\nu(t)$ is the multiplier in the original objective's scale. The variables t, f_i and $x(t)$ retain their definitions above.

Barrier stationarity becomes original Lagrangian stationarity, while complementarity becomes $\lambda_i(t)[-f_i(x(t))] = 1/t$. Each scalar inequality contributes the same positive product. At a true optimum, complementary slackness requires those products to vanish; the barrier path approaches that condition as t increases.

For an exactly centered point satisfying the stated assumptions, the associated primal–dual gap is

$$f_0(x(t)) - g(\lambda(t), \nu(t)) = \frac{m}{t}.$$

Here g is the Lagrange dual function, a lower bound on the optimal objective for dual-feasible multipliers. The gap bounds objective suboptimality because the primal value is an upper bound and the dual value is a lower bound. This is a statement about the formulated convex problem, not a guarantee that the formulation matches the application.

The equality requires exact centering and a valid dual construction. A numerical iterate with a large stationarity residual cannot claim an exact gap merely by printing m/t. Use the theoretical quantity to plan the barrier schedule, then check the actual residuals and, where available, evaluate a genuine dual lower bound. This prevents an outer-loop counter from masquerading as a certificate.

An outer barrier method therefore alternates two activities: solve a fixed-t interior problem, then increase t. Reuse the previous solution as the next starting point. The next problem usually has a nearby minimizer, but very large changes in t can make centering expensive. Choose the increase based on measured centering behavior rather than assuming that fewer outer iterations always mean less computation.

Where the gap m/t actually comes from

The central-path gap is a sum of explicit prices times explicit slacks. At an exactly centered point, divide barrier stationarity by t:

$$\nabla f_0(x(t)) + \sum_i \frac{\nabla f_i(x(t))}{t[-f_i(x(t))]} + A^\top \frac{w(t)}{t} = 0.$$

Here $x(t)$ minimizes the barrier problem, $w(t)$ is its equality multiplier, f_0,f_i are the original objective and inequalities, and $t > 0$ is the barrier weight. Substituting the defined $\lambda_i(t) = 1/[t(-f_i(x(t)))]$ and $\nu(t) = w(t)/t$ makes this exactly the original Lagrangian's stationarity equation.

The Lagrangian is convex in x because the original functions are convex and the inequality multipliers are nonnegative. Stationarity therefore makes $x(t)$ its global minimizer, so evaluating L at this point really computes the dual function. Equality feasibility removes the equality term, giving

$$g(\lambda(t), \nu(t)) = f_0(x(t)) + \sum_i \lambda_i(t) f_i(x(t)) = f_0(x(t)) - \frac{m}{t}.$$

Here g is the dual function and m the scalar inequality count. The gap follows by subtracting this lower bound from the feasible primal objective. The central point is usually not the original optimizer: it trades objective improvement against positive slack. Its multipliers already supply a certificate whose error decreases as the barrier weakens [1, Section 11.2].

For a desired theoretical gap at most $\varepsilon > 0$, choose $t \geq m/\varepsilon$ and then solve the centering problem accurately enough. The choice of t supplies a target; the stationarity solve is what justifies using the formula. Increasing t without centering omits the key equality in the proof.

Interactive demonstration (online): open the online edition of this tutorial to use this demonstration.Open demonstration

5. A complete one-dimensional barrier experiment

Use a constrained quadratic whose true optimum is easy to check. We minimize $\frac{1}{2}(x-2)^2$ subject to $0 \le x \le 1$. The unconstrained minimizer is two; the closest permitted point is one, with objective one half. A barrier solution stays strictly between zero and one and approaches the upper endpoint.

For this example,

$$F_t(x) = \frac{t}{2}(x-2)^2 - \log x - \log(1-x).$$

Here $x \in (0,1)$ is the scalar variable and $t > 0$ is the objective weight. The first and second derivatives are $F_t'(x) = t(x-2) - 1/x + 1/(1-x)$ and $F_t''(x) = t + 1/x^2 + 1/(1-x)^2$. Every denominator is positive in the barrier domain.

```
import numpy as np

def center(t, x):
    def value(z):
        return 0.5*t*(z-2)**2 - np.log(z) - np.log1p(-z)
    for _ in range(100):
        grad = t*(x-2) - 1/x + 1/(1-x)
        hess = t + 1/x**2 + 1/(1-x)**2
        step = -grad/hess
        if grad*grad/hess < 1e-16:
            break
        alpha = 1.0
        for _ in range(100):
            trial = x + alpha*step
            if 0 < trial < 1 and value(trial) <= value(x) + 0.01*alpha*grad*step:
                break
            alpha *= 0.5
        else:
            raise RuntimeError("Line search failed")
        x = trial
    else:
        raise RuntimeError("Centering iteration limit reached")
    return x
x = 0.5
for t in [1., 4., 16., 64., 256., 1024.]:
    x = center(t, x)
    lower, upper = 1/(t*x), 1/(t*(1-x))
    stationarity = x-2-lower+upper
    primal = 0.5*(x-2)**2
    dual = 2-upper-0.5*(-2-lower+upper)**2
    gap = primal-dual
    print(int(t), round(x, 6), round(gap, 7), round(2/t, 7))
    assert 0 < x < 1
    assert abs(stationarity) < 1e-5
    assert abs(gap-2/t) < 1e-8
    assert primal-0.5 <= gap + 1e-8
```

The feasible backtracking check runs before the objective comparison. This prevents logarithms from being evaluated outside their domain. The sufficient-decrease test uses the directional derivative, and the Newton decrement supplies the centering test. Maximum iteration counts turn an unexpected failure into an explicit error instead of an infinite loop.

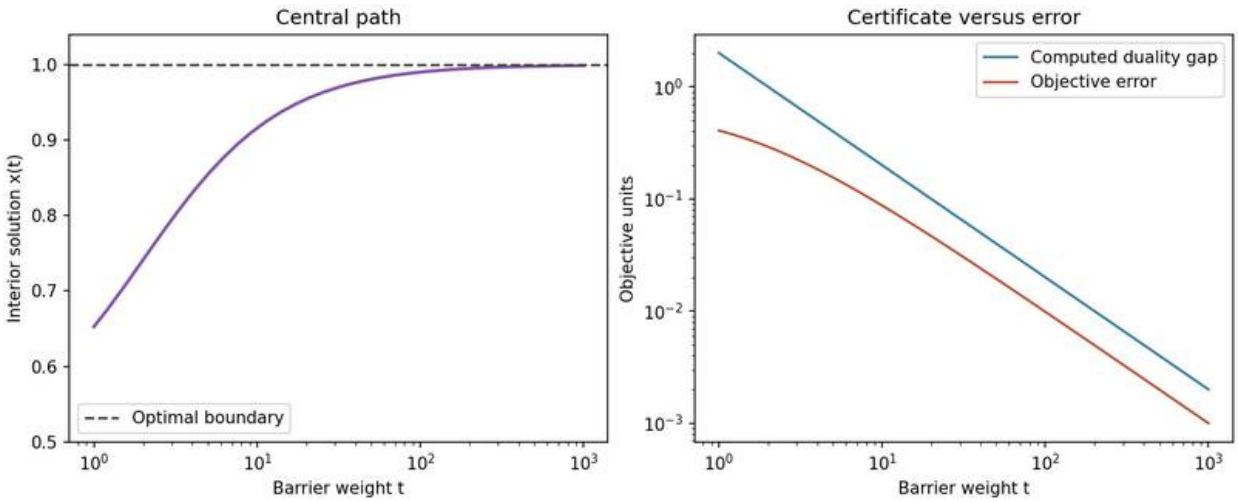


Figure 1 — A synthetic constrained quadratic makes the central-path certificate visible: the solution approaches one from the interior and the computed dual gap bounds its objective error.

The experiment computes a genuine dual value using the unconstrained minimum of the scalar Lagrangian. It therefore checks the gap separately from the prescribed $2/t$. In a larger nonlinear problem, evaluating the dual function may itself require another optimization. A primal–dual residual test is then useful, but describe what it certifies and under which regularity assumptions.

Near the endpoint, `1 - x` becomes small and the barrier Hessian becomes large. A smaller error target can make the numerical system harder even though the mathematical problem is unchanged. Stop when the solution is accurate enough in meaningful units. Demanding every last representable digit can waste work or expose conditioning that the application never needed to resolve.

6. Feasibility, strict feasibility and Phase I

A feasible solution may exist without any strictly feasible solution. A barrier for the original inequality representation then has no domain. This is different from infeasibility, and a solver must distinguish the two cases before trying to follow a central path.

A simple Phase I construction introduces one scalar s and solves

$$\begin{aligned} \text{minimize}_{x,s} \quad & s \\ \text{subject to} \quad & f_i(x) \le s \quad (i = 1, \dots, m), \\ & Ax = b. \end{aligned}$$

Here s bounds the largest inequality violation and is an additional variable. A point with $s < 0$ is strictly feasible for the original inequalities. A certified positive lower bound on the optimal s

proves infeasibility. An optimal value zero can mean boundary-only feasibility; nonattainment and numerical uncertainty require separate attention. Finite iteration failure does not itself prove any of these outcomes.

For the contradictory system $x \le 0$ and $x \ge 1$, the violation measure is $\max(x, 1-x)$. Its minimum is one half at $x = 1/2$. Equal nonnegative weights on the two inequalities also produce an explicit infeasibility certificate.

```
import numpy as np

A = np.array([[1.0], [-1.0]])
b = np.array([0.0, -1.0])
y = np.array([0.5, 0.5])
print("A transpose y:", A.T @ y)
print("b transpose y:", b @ y)
assert np.all(y >= 0)
assert np.allclose(A.T @ y, 0)
assert b @ y < 0
# If A x <= b existed, multiplying by y would imply 0 <= b@y < 0.
grid = np.linspace(-1, 2, 301)
violation = np.maximum(grid, 1-grid)
assert np.isclose(violation.min(), 0.5)
# A different system is feasible only on its boundary.
A_boundary = np.array([[1.0], [-1.0]])
b_boundary = np.zeros(2)
assert np.all(A_boundary @ np.array([0.0]) <= b_boundary)
```

The grid is an illustration of the already derived minimum, rather than a proof that an arbitrary optimization is globally solved. The certificate is the decisive argument: if $Ax \le b$, multiplying by $y \ge 0$ would imply $0 = y^\top Ax \le y^\top b < 0$, a contradiction. The certificate checks data and multipliers directly.

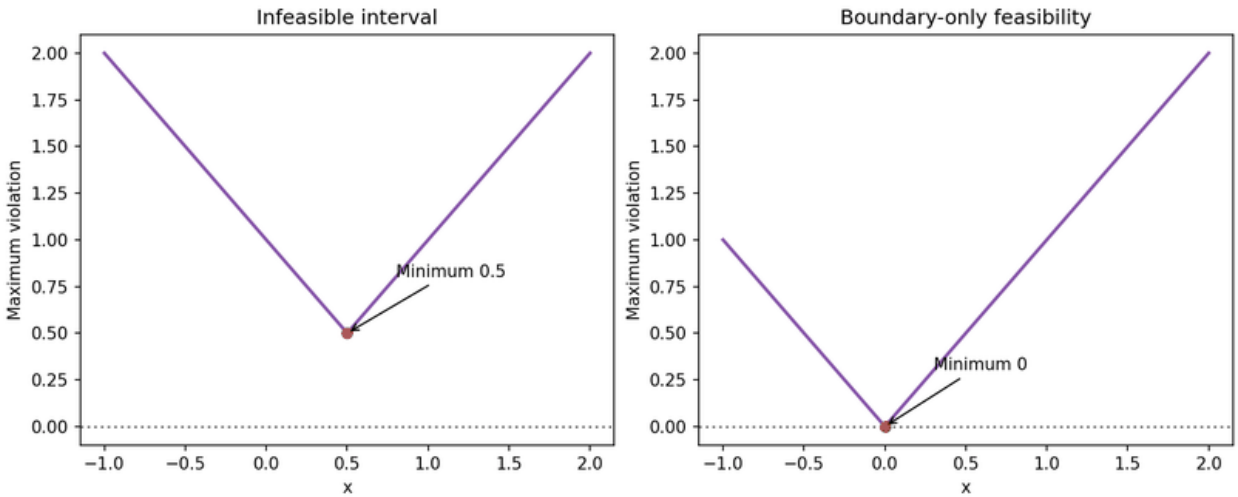


Figure 2 — Positive minimum violation indicates the contradictory interval is infeasible; zero minimum in

the second example identifies a feasible point with no strict interior.

For the boundary-only system $x \leq 0$ and $-x \leq 0$, the only feasible point is zero. Eliminate the forced variable or represent the condition as an equality. Merely nudging the starting point fails because one inequality becomes violated whichever direction you move. The relevant interior is the relative interior after the forced equality is accounted for.

Phase I also depends on the domains of the f_i. Relaxing their inequality values does not automatically make logarithms, matrix inverses or other expressions defined. Initialize inside the function domains or choose an appropriate reformulation. Inspect domain feasibility separately from the affine and inequality residuals.

Phase I has normalized dual prices

The maximum-violation formulation connects initialization to dual certificates. Attach multipliers $\lambda_i \geq 0$ to $f_i(x) - s \leq 0$ and multiplier ν to $Ax - b = 0$. Its Lagrangian is

$$L_I(x, s, \lambda, \nu) = \left(1 - \sum_i \lambda_i\right) s + \sum_i \lambda_i f_i(x) + \nu^\top (Ax - b).$$

Here x, s are Phase I variables, λ, ν its multipliers, and f_i, A, b the original restriction data. Since s is unrestricted, its infimum is finite only when $\sum_i \lambda_i = 1$. The inequality prices therefore form normalized nonnegative weights. A finite lower bound on the remaining weighted function bounds the largest original violation [1, Section 11.4].

For $f_1(x) = x$ and $f_2(x) = 1 - x$, choose $\lambda_1 = \lambda_2 = 1/2$. Their weighted sum is exactly $1/2$ for every x. Thus the Phase I optimum is at least $1/2$; choosing $x = 1/2, s = 1/2$ attains it. This supplies matching primal and dual values, proving that the original inequalities are infeasible.

For the boundary-only pair $f_1(x) = x, f_2(x) = -x$, the same normalized sum is zero. The feasible Phase I point $x = 0, s = 0$ matches it, but no negative s is possible. Zero violation can therefore prove ordinary feasibility without providing the strict interior needed by a logarithmic barrier. Eliminating the forced equality is the appropriate next modeling step.

7. Primal–dual steps and reliable stopping

A primal–dual method works directly with approximate optimality equations rather than completely solving each separate barrier problem. It maintains positive inequality multipliers and positive slacks, then solves a linearized system to reduce stationarity, feasibility and complementarity errors together.

Write the slack of inequality i as $s_i = -f_i(x) > 0$ and its multiplier as $\lambda_i > 0$. For the original problem, the residuals include

$$r_{\text{dual}} = \nabla f_0(x) + \sum_i \lambda_i \nabla f_i(x) + A^\top \nu,$$
$$r_{\text{pri}} = Ax - b,$$
$$\eta = \sum_i \lambda_i s_i.$$

Here r_{dual} is the stationarity residual, r_{pri} is the equality residual, and η is the sum of complementarity products. The functions, matrix, multipliers and slacks have been defined above. For affine inequalities, separate slack equations can be included explicitly; nonlinear formulations may represent them through $f_i(x)$.

A small η with a large feasibility residual is not a valid approximate solution. Nor does a small gradient in unconstrained coordinates establish constrained optimality. Require all relevant residuals, multiplier signs and domain conditions to pass. When the multipliers and primal point are feasible and stationarity holds, the complementarity sum corresponds to the duality gap; otherwise that interpretation needs qualification.

A fraction-to-boundary line search limits the step so the slacks and multipliers remain positive. It is a numerical safeguard for the algorithm's domain, not a separate solution constraint. An infeasible-start method can initially violate affine equalities while still respecting the interior conditions needed by its formulas. Its merit function must reduce residuals, rather than simply evaluate the original objective at an infeasible point.

Sparse linear algebra often dominates the solve. The pattern of nonzero entries determines factorization fill, memory use and whether reuse is possible. A small iteration count can coexist with expensive iterations. Record factorization time and peak memory alongside iteration count when comparing methods. A dense educational implementation hides these costs and should not be used to predict large-problem performance.

Check the solution in the original formulation after canonicalization. A solver may introduce auxiliary variables and cones, normalize data, or remove redundant rows. These operations help computation, but the final application needs original-unit constraints and an interpretable objective. Record solver status, residuals, tolerances, formulation version and any postprocessing that modifies the returned point.

Linearize the primal–dual equations

A primal–dual Newton step targets the optimality equations jointly. For a quadratic objective with $Q \succeq 0$, affine inequalities $Gx + s = h$, and equalities $Ax = b$, use positive slack vector s and positive multiplier vector λ. Let c be the objective's linear coefficient and ν the equality multiplier.

Define $r_d = Qx + c + G^\top \lambda + A^\top \nu$, $r_g = Gx + s - h$, and $r_a = Ax - b$. With a chosen complementarity target $\tau > 0$, linearization gives

$$
\begin{aligned}
Q\Delta x + G^{\top}\Delta\lambda + A^{\top}\Delta\nu &= -r_d,\\
G\Delta x + \Delta s &= -r_g,\\
A\Delta x &= -r_a,\\
\Lambda\Delta s + S\Delta\lambda &= \tau\mathbf{1} - S\lambda.
\end{aligned}
$$

Here G, h specify the affine inequality coefficients and bounds; A, b specify the equalities; r_d, r_g, r_a are stationarity, inequality-feasibility and equality-feasibility residuals; and τ is the complementarity target. The quantities $\Delta x, \Delta s, \Delta\lambda, \Delta\nu$ are increments; $S = \operatorname{diag}(s)$, $\Lambda = \operatorname{diag}(\lambda)$, and $\mathbf{1}$ is an all-ones vector. The last row linearizes the products $s_i\lambda_i = \tau$, dropping only the second-order increment products. Unlike a full barrier centering solve, one step simultaneously reduces feasibility, stationarity and complementarity errors [1, Section 11.7].

If an increment has a negative slack coordinate, positivity requires step length $\alpha < -s_i/\Delta s_i$ in that coordinate. Apply the same rule to negative multiplier increments, take the smallest permitted bound, and stay slightly inside it. This derives the fraction-to-boundary safeguard rather than treating it as an arbitrary clipping rule. A further residual-decrease test is still needed: preserving positivity alone does not guarantee convergence.

8. Interpret a certificate at the application boundary

An optimality certificate answers a mathematical question about a specified model. An application decision also depends on whether that model includes the right constraints, quantities and uncertainty. Keep these two checks separate in the final report: numerical correctness cannot repair an omitted requirement.

Suppose a resource-allocation model limits total power but overlooks a per-device limit. The solver can return a globally optimal, accurately certified allocation that one device cannot implement. Add the missing constraint and solve again; do not describe the earlier result as a solver failure. Conversely, if every required constraint is present but the returned point violates one beyond tolerance, the issue belongs to numerical acceptance or the solve itself.

Choose tolerances before seeing the answer. A temperature bound, a budget equality and a dimensionless probability constraint need different interpretations of error. Write the acceptable deviations in original units, then connect them to solver tolerances and scaling. If a downstream rounding operation changes the variables, recheck feasibility after rounding. A certificate for the unrounded vector does not automatically certify the displayed or implemented decision.

Dual multipliers can explain sensitivity when the assumptions from the duality chapter hold. A large multiplier on a tight constraint suggests that a small relaxation may strongly affect the objective locally. It is not a prediction for an arbitrary large change, especially when the active constraints change. Test a small perturbation, resolve the problem and compare the observed objective change with the local prediction.

Near-degenerate problems deserve careful interpretation. Several almost redundant constraints can produce poorly determined multipliers while the primal decision is quite stable. A solver's multiplier variation may then overstate uncertainty in the decision itself. Examine both primal

sensitivity and dual sensitivity, and avoid reporting a shadow price with more precision than the data and conditioning justify.

Warm starts also belong to this boundary. Yesterday's solution may be near today's optimum but violate today's updated constraints. A feasible-start barrier method must first find a new interior point. A primal–dual or homogeneous-embedding solver may handle a different initialization strategy. Pass a warm start only through a method that supports its feasibility state; do not clip variables independently when clipping can break coupled equalities.

A useful result bundle includes the formulation and data versions, original objective, primal variables, relevant multipliers, maximum original-unit violations, a gap or lower bound when valid, and the termination status. Add the measured time and memory if performance matters. This bundle lets another reader distinguish a trustworthy approximate optimum, an infeasibility certificate, an unresolved numerical outcome and a model that needs revision.

Modern view

Interior-point theory remains useful while current solvers make different choices about precision, memory and hardware. The following readings provide an algorithmic taxonomy and verified modern evidence through October 4, 2026.

Bubeck's monograph [2] organizes convex optimization around oracle information, geometry and structure, covering first-order, structural and interior-point approaches. Its useful comparison is the cost of an iteration together with the guarantees attached to the oracle and assumptions. Our synthesis is to choose a method by the operations the problem makes cheap: matrix–vector products, projections, proximal maps or factorizations.

Parikh and Boyd's proximal survey [3] maps nonsmooth, constrained and distributed algorithms through small convex subproblems. It complements the barrier viewpoint: a projection or proximal operation can enforce a constraint without representing its interior through logarithms. These operations can be inexpensive or costly depending on the set. A problem's mathematical convexity does not determine which operation is affordable.

Goulart and Chen's 2026 Clarabel paper [4] describes an interior-point solver for conic problems with quadratic objectives, using a homogeneous embedding and supporting several cone types. It connects textbook ideas to software that also addresses infeasibility and computational structure. Its comparisons apply to the tested problem sets and solver configurations; they do not establish one universal winner for every convex model.

Applegate et al.'s PDLP manuscript [5], first posted in 2025 and revised in 2026, develops a practical first-order approach to large-scale linear programming. It adds techniques such as preprocessing, scaling and adaptive behavior around matrix–vector operations. It is a preprint, and its performance claims are empirical for the reported workloads. The comparison with interior-point methods should include requested accuracy, memory budget and original-unit residuals.

Solver family	Expensive operation	Typical structural question	What to verify
Equality-constrained Newton	KKT linear solve	can sparsity and factorization be reused?	feasibility and stationarity
Barrier/interior-point	changing curvature and constrained solve	is there an appropriate interior and cone representation?	domains, residuals, gap or certificates
Projection/proximal	projection or proximal subproblem	is the constraint or nonsmooth term simple to process?	iterate residual and feasible output
Large-scale first-order LP	matrix–vector products	does memory or factorization dominate the alternative?	scaled and original residuals at target accuracy

The current methodological lesson is broader than a preference for first- or second-order methods. Formulation, preprocessing, scaling, linear algebra and stopping policy are part of the algorithm's practical behavior. Comparing only the named update equation leaves out much of the system that determines reliability.

Open questions include accurate certificates for poorly scaled models, robustness when the feasible region is nearly lower-dimensional, memory limits in large factorizations and effective warm starts across changing constraints. A learned prediction can offer an initial point, but it still requires a feasibility and optimality check. Approximate solutions are useful when their errors are measured in the quantities the application can tolerate.

For a solver experiment, construct a family with one varying property: condition number, number of active constraints, sparsity or requested tolerance. Keep the mathematical optimum or an independently checked bound where possible. Report failures and inaccurate statuses alongside timings. Such a controlled family teaches more than a single successful solve with default settings.

Key takeaways

- Equality-constrained Newton steps remain in the nullspace of the equality matrix.
- The KKT matrix is generally indefinite even for a convex objective.
- A logarithmic barrier requires strict feasibility in the relevant domain.
- Exactly centered points yield a dual construction with gap m/t under the stated assumptions.
- Numerical iterates need actual feasibility, stationarity and complementarity checks.
- Boundary-only feasibility differs from infeasibility; Phase I needs a certificate to distinguish them reliably.
- Compare solver cost, precision and memory in the original problem's units.

Exercises

1. **Constrained curvature.** Construct a positive-semidefinite quadratic whose full Hessian is singular but whose objective has a unique minimizer on an affine equality set.

Hint

Let a zero-curvature direction be excluded by the equality. Check positive curvature only along nonzero vectors satisfying the homogeneous equality.

2. **Equality scaling.** Multiply the quadratic example's equality by 1000. Compare the solution, multiplier and residual scales. Explain which quantities should remain invariant.

Hint

The feasible set is unchanged. The equality multiplier scales inversely, and raw residuals scale with the row. Verify stationarity in both representations.

3. **Duplicate barriers.** Duplicate the upper-bound inequality in the scalar example. Compare the finite-t central path and the count used in the theoretical gap.

Hint

The feasible set is identical, but the upper-bound logarithm appears twice. There are now three scalar barrier contributions.

4. **Initialization.** Start the scalar barrier solver at zero and explain the failure. Give a principled repair for this problem and for a variable forced to zero by equalities.

Hint

A free nonnegative variable can begin strictly positive. A forced zero variable needs elimination or a suitable equality representation, rather than a numerical nudge.

5. **Infeasibility certificate.** Generalize the contradictory interval to $x \leq a$ and $x \geq b$ with $a < b$. Derive its minimum maximum violation and nonnegative certificate.

Hint

Balance the two violations at the midpoint. Equal weights cancel the variable and leave a negative right-hand-side sum.

6. **Stopping policy.** A solver reports tiny complementarity but a nonzero equality residual. Write the additional checks needed before accepting its output, and explain why an objective-only report is insufficient.

Hint

Include original-unit equality and inequality residuals, domain checks, multiplier signs, stationarity and a valid lower bound when available.

References

1. S. Boyd and L. Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004, Chapters 10–11. Book and official PDF
2. S. Bubeck, "Convex Optimization: Algorithms and Complexity," Foundations and Trends in Machine Learning, 2015. Monograph
3. N. Parikh and S. Boyd, "Proximal Algorithms," Foundations and Trends in Optimization, 2014. Author page and article
4. P. J. Goulart and Y. Chen, "Clarabel: An Interior-Point Solver for Conic Programs with Quadratic Objectives," Mathematical Programming Computation, 2026. Published paper · Author manuscript
5. D. Applegate, M. Díaz, O. Hinder, H. Lu, M. Lubin, B. O'Donoghue and W. Schudy, "PDLP: A Practical First-Order Method for Large-Scale Linear Programming," arXiv preprint, 2025, revised 2026. Paper

OPT 08 • Geometric Applications, Splitting and Modern Methods

Turn geometric and signal models into convex programs, choose proximal, ADMM or conditional-gradient operations, and evaluate an optimization pipeline with explicit certificates.

Online edition: https://blog.shyandram.dev/en/learn/optimization/opt-08-applications-modern-methods/

What you'll learn

- Model distances, centers, separation and placement using convex geometry.
- Distinguish a convex application model from a relaxation of a discrete or nonconvex problem.
- Derive proximal updates for nonsmooth terms and constraints.
- Implement splitting for synthetic signal denoising and conditional gradients on a simplex.
- Choose methods by their expensive operations and check appropriate convergence evidence.
- Build a reproducible optimization capstone and connect it to modern surveys and methods.

The big picture

Convex optimization becomes useful when a problem's structure tells you both how to model it and how to solve it. A geometric model may make projections simple. A signal model may separate a smooth data term from a nonsmooth prior. A distributed model may make local solves cheap while communication is expensive. The right algorithm works with that structure rather than hiding it.

> **Explain like I'm five**
>
> Imagine arranging a workshop. Every machine needs some space, the doors must stay clear, and you want short walking routes. First decide which arrangements are allowed. Then choose what "better" means. Some rules let you improve the layout smoothly; other rules say a machine must be entirely left or entirely right of another. Those either-or rules need special care. A smooth-looking drawing does not make every layout problem convex.

The geometric reading follows Boyd and Vandenberghe's Chapter 8, with modeling and approximation connections to Chapters 4 and 6 [1]. The algorithmic extensions use Parikh and Boyd's proximal survey [2], Boyd et al.'s ADMM review [3], Condat et al.'s splitting survey [4] and Pokutta's conditional-gradient introduction [5]. The chapter closes the track by connecting these operations to a complete modeling, solving and validation workflow.

All experiments use synthetic data and run locally on a CPU. Their purpose is to expose the objective, assumptions, residuals and failure modes. A smaller numerical objective is meaningful only after the feasible set, data scale and evaluation protocol have been checked.

1. Distances, projections and separation

A distance problem asks how close a point or set can get to another while obeying its restrictions. A projection finds the nearest allowed point. These geometric operations become building blocks for both applications and optimization algorithms.

For a nonempty closed convex set $C \subseteq \mathbb{R}^n$, define

$$\Pi_C(v) = \arg\min_{x \in C} \tfrac{1}{2}\|x - v\|_2^2.$$

Here $v \in \mathbb{R}^n$ is a fixed query point; x is the optimization variable; $\|\cdot\|_2$ is Euclidean norm; and $\Pi_C(v)$ is the projection. The objective is strictly convex in x, so the minimizer is unique. Closedness ensures attainment; nonemptiness ensures the problem has a feasible point.

For a box, projection clips each coordinate to its interval. For an affine set, it solves a constrained quadratic. For a positive-semidefinite matrix cone, a projection involves the spectrum of a symmetric matrix. These are all convex projections, but their costs differ substantially. "Projected gradient" is a useful name only after the projection cost is understood.

A projection satisfies $(v - p)^\top (x - p) \le 0$ for every $x \in C$, where $p = \Pi_C(v)$. The inequality says that the displacement from the projection to the query points away from every feasible direction. It supplies an optimality check and, when the query lies outside the set, a separating hyperplane.

Distances between two convex sets can be modeled by minimizing $\|x - y\|_2$ with $x \in C$ and $y \in D$, where D is a second set. The norm is convex in the pair. A finite infimum need not be attained for arbitrary unbounded sets; verify existence before interpreting an algorithm's iterates as converging to a closest pair. For compact sets, the geometry is simpler.

Classification by separation also has a geometric interpretation. A linear score $a^\top x + b$ places a hyperplane between classes, where a is its normal and b its offset. Margin constraints can be convex when the data and labels are fixed. Choosing a representation jointly with all classifier parameters can create a different, nonconvex learning problem. The Pattern Recognition track develops the statistical context of these models.

2. Centers, ellipsoids and placement

A center can mean several different things. The center minimizing maximum distance to observations differs from the center of the largest ball fitting inside a feasible region. The analytic center of an inequality representation is another object. State the geometry before calling a point "the center."

For a polyhedron described by $a_i^\top x \le b_i$, the largest contained Euclidean ball has center c and radius r obtained from

$$\begin{aligned} \text{maximize}_{c,r} \quad & r \\ \text{subject to} \quad & a_i^\top c + r\|a_i\|_2 \le b_i \quad \text{for every } i, \\ & r \ge 0. \end{aligned}$$

Here $a_i \in \mathbb{R}^n$ and $b_i \in \mathbb{R}$ are fixed inequality data; $c \in \mathbb{R}^n$ and $r \in \mathbb{R}$ are variables. The worst displacement of length at most r in direction a_i contributes $r\|a_i\|_2$. Every constraint is affine in (c, r) because the normal vectors are fixed.

This center balances distance to boundaries in Euclidean units. Multiplying an inequality by a positive constant scales both its normal norm and right-hand side, preserving the geometric condition. A zero normal needs separate inspection: its inequality is either always satisfied or impossible. Do not divide by its norm blindly.

An ellipsoid allows different radii in different directions. Write it as $E = \{c + Bu : \|u\|_2 \le 1\}$, where B is a positive-definite symmetric shape matrix, c is the center and u is an auxiliary unit-ball coordinate. Containment in a halfspace becomes $a_i^\top c + \|B^\top a_i\|_2 \le b_i$. Maximizing $\log\det B$ models maximum volume up to an additive constant [1]. The domain $B \succ 0$ is essential.

Centering and enclosing are different constraints. An inscribed ellipsoid must stay inside a region, while an enclosing ellipsoid must contain given points. Their convenient convex parameterizations differ. A visually plausible shape matrix can lead to a nonconvex formulation if inverses and variable products are introduced carelessly. Trace each transformation to its domain and convexity rule.

Placement problems often minimize sums of distances between movable points and fixed anchors. If connectivity and constraints are fixed, Euclidean norms yield a convex objective. Choosing which connections exist introduces discrete variables. Floorplanning can be convex after selecting a relative ordering of rectangles, but unknown nonoverlap choices contain disjunctions. The original physical problem and a convex subproblem need separate labels.

Worked center: a rectangle has more than one answer

The largest inscribed ball need not have a unique center. Take the rectangle $0 \le x_1 \le 6, 0 \le x_2 \le 2$. The ball constraints reduce to $r \le c_1 \le 6 - r$ and $r \le c_2 \le 2 - r$, with $r \ge 0$.

Here $c = (c_1, c_2)$ is the center and r its radius. The shorter side forces $r \le 1$. At maximum radius one, every center $(c_1, 1)$ with $1 \le c_1 \le 5$ is optimal. The problem is convex, yet its optimizer is not unique. The symmetric center $(3, 1)$ is one valid answer, rather than a uniquely implied one.

For an inscribed ellipsoid, a symmetric positive-definite B maps the unit ball to the shape. Its volume is the unit-ball volume multiplied by $\det B$, because a linear map scales volume by its determinant. Maximizing $\log\det B$ therefore preserves volume maximizers. The containment rule follows by maximizing $a_i^\top (c + Bu)$ over $\|u\|_2 \le 1$, producing $a_i^\top c + \|B^\top a_i\|_2$. This combines the support-function calculation of OPT 03 with the matrix curvature of OPT 02 [1, Chapter 8].

The convex formulation minimizes $-\log\det B$ over the containment constraints. The halfspace normals are fixed, so the norm term is convex in B. Allowing B to become singular would leave the finite log-determinant domain and collapse the ellipsoid's volume.

3. Relaxations are models with an explicit interpretation

A relaxation enlarges the feasible set to make a hard problem easier. It can produce a useful bound or a candidate that later needs rounding. A fractional solution is implementable only when the application permits mixtures, sharing or time allocation.

Suppose a binary configuration variable z_i must lie in $\{0, 1\}$. Replacing that constraint by $0 \le z_i \le 1$ is a convex relaxation of the discrete requirement when the remaining formulation is convex. For minimization, enlarging the feasible set can only decrease the optimum, so the relaxed value is a lower bound on the original optimum. The bound does not mean the relaxed point is a valid discrete decision.

After rounding, evaluate the original objective and constraints again. Independent coordinate rounding can violate a total budget, a connectivity requirement or an equality. A repaired feasible rounded solution gives an upper bound; the relaxed optimum gives a lower bound. Their difference is meaningful when both values refer to the same original objective and the lower value is the exact relaxed optimum or a certified dual or error-corrected bound. An approximate relaxed primal objective alone is generally an upper bound on the relaxed optimum; it need not be a lower bound on the original discrete optimum.

The same care applies to convex surrogates in vision and learning. A norm penalty can encourage sparsity without solving a general count-minimization problem exactly. A convex data term may be a deliberate approximation to a physical measurement process. State whether the model is a statistical estimator, a computational relaxation or an exact reformulation. These purposes support different claims.

Parameter selection also belongs outside the numerical certificate. Solving a regularized problem to high accuracy does not prove that its regularization strength predicts well on new data. Select the strength using a valid validation protocol, then test once on held-out data. Keep numerical optimization error separate from model-selection and generalization error.

4. Proximal operations handle nonsmooth structure

A nonsmooth term need not be approximated by a steep smooth function. A proximal operation balances that term against staying near a proposed point. Many penalties and constraints admit a simple closed-form operation, making them suitable for first-order methods [2].

For a proper closed convex function g and step scale $\eta > 0$, define

$$\operatorname{prox}_{\eta g}(v) = \arg\min_z \left[g(z) + \frac{1}{2\eta}\|z - v\|_2^2 \right].$$

Here $v \in \mathbb{R}^n$ is the proposed point; $z \in \mathbb{R}^n$ is the subproblem variable; g is allowed to take positive infinity to encode restrictions; and η scales the quadratic proximity term. Under the stated conditions the subproblem has a unique minimizer.

For the indicator of a closed convex set, the proximal operation is projection. For $g(z) = \lambda\|z\|_1$, with $\lambda \ge 0$, it is coordinatewise soft thresholding: $\operatorname{sign}(v_i)\max(|v_i| - \eta\lambda, 0)$. Here i indexes coordinates.

The operation can create exact zeros; ordinary gradient descent on the absolute-value penalty is undefined at zero.

To minimize $f(x) + g(x)$ with differentiable convex f, proximal gradient takes

$$x^{k+1} = \operatorname{prox}_{\eta g}\left(x^k - \eta \nabla f(x^k)\right).$$

Here k is the iteration index; ∇f is the smooth gradient; and η is a step size selected to meet the method's assumptions. If the gradient is L-Lipschitz, a conservative fixed choice is $0 < \eta \leq 1/L$. The constant L bounds gradient variation, rather than the objective value itself.

For constrained least squares, this update becomes gradient followed by projection. For sparse least squares, it becomes gradient followed by soft thresholding. More complicated compositions, such as a nonsmooth function applied to a linear difference operator, may make the direct proximal map expensive. Splitting introduces auxiliary variables to keep the cheap operations separate.

A useful residual is the scaled update $G_\eta(x) = [x - \operatorname{prox}_{\eta g}(x - \eta \nabla f(x))]/\eta$. Here G_η is a proximal-gradient mapping. A zero mapping satisfies the first-order optimality condition for this convex composite problem. Its magnitude is a convergence diagnostic; interpret it together with scale, feasibility and an objective bound where available.

Derive soft thresholding at the corner

The proximal map solves the nonsmooth corner exactly. For one coordinate, minimize $\lambda|z| + (z - v)^2/(2\eta)$ with $\eta > 0$ and $\lambda \geq 0$. The optimality condition is

$$0 \in \lambda \partial |z| + \frac{z - v}{\eta}.$$

Here v is the proposed coordinate, z the output and $\partial|z|$ the absolute-value subdifferential. For $z > 0$, it equals one, so $z = v - \eta\lambda$, valid only if $v > \eta\lambda$. For $z < 0$, it equals minus one, giving $z = v + \eta\lambda$, valid only if $v < -\eta\lambda$. At zero, the subdifferential is $[-1, 1]$, so zero is optimal precisely when $|v| \leq \eta\lambda$ [2].

These three cases give the threshold formula. For the synthetic vector $v = (2, -0.3, 0.1)$ and threshold $\eta\lambda = 0.4$, the output is $(1.6, 0, 0)$. A nonzero input can become exactly zero because zero has a whole interval of supporting slopes.

The proximal-gradient update follows by minimizing a local upper model of the smooth term: replace $f(z)$ by $f(x) + \nabla f(x)^\top (z - x) + \|z - x\|_2^2/(2\eta)$ and retain $g(z)$ exactly. Completing the square centers this model at $v = x - \eta \nabla f(x)$. Thus the method uses a gradient step for the smooth cost and an exact small solve for the nonsmooth cost. When $1/\eta \geq L$, the smoothness inequality makes this model an upper bound, explaining the conservative step condition.

5. Split a signal model into simple subproblems

Piecewise-constant signals can be estimated by balancing agreement with observations against neighboring differences. A quadratic data term discourages large corrections; an absolute-difference penalty encourages many neighboring values to match. The resulting convex estimator is useful for a controlled denoising experiment, with behavior different from uniform smoothing.

Let $y \in \mathbb{R}^n$ contain noisy observations and $D \in \mathbb{R}^{(n-1)\times n}$ compute adjacent differences. Solve

$$\operatorname{minimize}_x \quad \tfrac{1}{2}\|x - y\|_2^2 + \lambda \|Dx\|_1.$$

Here $x \in \mathbb{R}^n$ is the estimated signal and $\lambda \geq 0$ is the regularization strength. This is one-dimensional total-variation regularization as treated in proximal and splitting references [2–4]. The term describes local variation; it does not assert that every real signal is piecewise constant.

Introduce $z = Dx$ and apply the alternating direction method of multipliers, or ADMM [3]. In scaled form,

$$\begin{aligned} x^{k+1} &= (I + \rho D^\top D)^{-1}\left[y + \rho D^\top (z^k - u^k)\right], \\ z^{k+1} &= \operatorname{shrink}(Dx^{k+1} + u^k, \lambda/\rho), \\ u^{k+1} &= u^k + Dx^{k+1} - z^{k+1}. \end{aligned}$$

Here $\rho > 0$ is the penalty parameter; I is the identity matrix; u is the scaled dual variable; and shrink denotes coordinatewise soft thresholding defined above. The inverse notation describes the solution mathematically; the implementation uses a factorization and solve. The split equality is satisfied at convergence, rather than necessarily at each intermediate iterate.

```
import numpy as np
from scipy.linalg import cho_factor, cho_solve

rng = np.random.default_rng(8)
n = 48
clean = np.r_[np.zeros(16), np.ones(16), np.full(16, 0.3)]
y = clean + rng.normal(0, 0.16, n)
D = np.diff(np.eye(n), axis=0)
lam, rho = 0.4, 4.0
factor = cho_factor(np.eye(n) + rho*D.T @ D)
x, z, u = y.copy(), np.zeros(n-1), np.zeros(n-1)
for k in range(2000):
    x = cho_solve(factor, y + rho*D.T @ (z-u))
    z_old = z.copy()
    v = D @ x + u
    z = np.sign(v) * np.maximum(np.abs(v)-lam/rho, 0)
    u += D @ x-z
    primal = np.linalg.norm(D @ x-z)
    dual = rho*np.linalg.norm(D.T @ (z-z_old))
    if max(primal, dual) < 1e-7:
        break
objective = lambda a: 0.5*np.sum((a-y)**2) + lam*np.abs(D @ a).sum()
```

```
print("Iterations and residuals:", k+1, primal, dual)
print("Objectives, raw and estimated:", objective(y), objective(x))
print("Synthetic MSE, raw and estimated:", np.mean((y-clean)**2), np.mean((x-clean)**2))
assert max(primal, dual) < 1e-7
assert objective(x) <= objective(y) + 1e-8
assert np.mean((x-clean)**2) < np.mean((y-clean)**2)
```

The last assertion checks this specific synthetic example; it is not a theorem that regularization always improves reconstruction. Strong regularization can flatten a real edge or remove a narrow peak. The objective chooses a compromise under the model, while clean-signal error requires ground truth that a real deployment may not have.

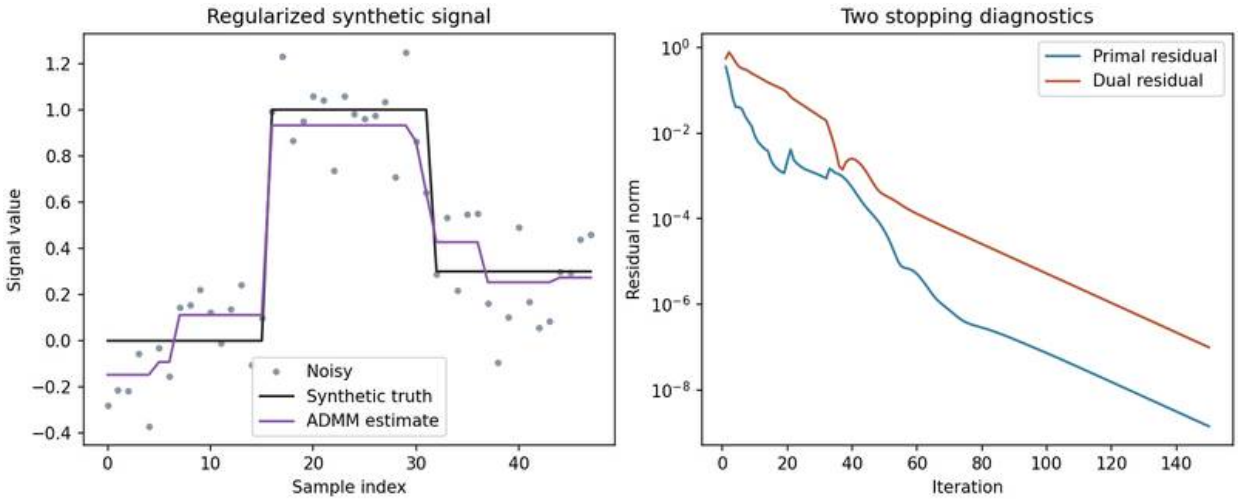


Figure 1 — Splitting separates a quadratic solve from soft thresholding. Convergence requires both residuals to tend to zero, without requiring monotone decrease; the denoising improvement belongs to this constructed signal and regularization choice.

The factorization is reused because D and ρ are fixed. Changing the regularization strength only changes thresholding; changing ρ changes the linear system and requires refactorization. If ρ changes while retaining the same unscaled multiplier, rescale u consistently. Otherwise the update silently changes the dual state.

For images, adjacent differences become horizontal and vertical operators, and the penalty can combine them in different ways. Boundary conditions and pixel scale affect the estimator. This chapter's one-dimensional result cannot be reported as an image-restoration benchmark. Use Image Processing for measurement models and Deep Learning for Computer Vision for learned restoration comparisons.

Derive the split updates and their residuals

The ADMM equations come from retaining an explicit difference variable rather than taking a difficult composite proximal map. With $z = Dx$, form the scaled augmented objective

$$\frac{1}{2}\|x-y\|_2^2+\lambda\|z\|_1+\frac{\rho}{2}\|Dx-z+u\|_2^2-\frac{\rho}{2}\|u\|_2^2.$$

Here x is the signal, y the observations, D the difference matrix, z the split differences, u the scaled multiplier, and $\rho>0$ the penalty. The last term is constant during the primal subproblems. Differentiating in x yields $(I+\rho D^\top D)x=y+\rho D^\top(z-u)$. Minimizing in z is the soft-threshold problem centered at $Dx+u$ with threshold λ/ρ. The multiplier update then accumulates the split-equality error [3].

Why monitor two residuals? The primal residual $r_p=Dx-z$ checks the coupling. The dual residual $r_d=\rho D^\top(z^{k+1}-z^k)$ checks the mismatch left in stationarity by the sequential update. Here k is the iteration index; changing the sign convention does not change the residual norm.

The data term makes this particular objective strongly convex in x, so the denoised signal is unique even though neighboring differences can be zero at nonsmooth corners. The residuals explain whether the split algorithm has reached that solution; they do not measure agreement with an unknown clean signal. The synthetic experiment can inspect both because its generating signal is available.

6. Conditional gradients use a linear oracle

Projection can be expensive even when minimizing a linear function over the set is easy. Conditional-gradient methods replace projection with a linear optimization oracle and a convex-combination update. Their iterates remain feasible when initialized feasibly [5].

For differentiable convex f on a compact convex set C, solve

$$v^k\in\arg\min_{v\in C}\nabla f(x^k)^\top v,\qquad x^{k+1}=(1-\gamma_k)x^k+\gamma_k v^k.$$

Here x^k is the current feasible point; v^k is a linear-oracle solution; and $\gamma_k\in[0,1]$ is the step weight. The new point is feasible by convexity. The Frank–Wolfe algorithm [5] is this basic conditional-gradient construction with suitable step choices and convergence assumptions.

A useful certificate is

$$g_{\mathrm{FW}}(x)=\nabla f(x)^\top(x-v(x)).$$

Here $v(x)$ solves the linear oracle at x. For an exact oracle and convex differentiable objective, the supporting-hyperplane inequality gives $f(x)-f(x^*)\le g_{\mathrm{FW}}(x)$, where x^* is a minimizer in C. The gap is therefore an upper bound on objective error at the current feasible point. An approximate oracle needs an additional error allowance.

On the probability simplex $C=\{x\ge 0:\sum_i x_i=1\}$, the linear oracle selects the coordinate with the smallest gradient. It returns a simplex vertex. This makes the method useful for mixtures where a new component can be selected cheaply. Sparse iterates arise because each update adds a vertex; representation and accuracy trade-offs still need examination.

```
import numpy as np
```

```
target = np.array([-0.1, 0.6, 0.5])
def project_simplex(v):
    sorted_v = np.sort(v)[::-1]
    threshold = (np.cumsum(sorted_v)-1)/np.arange(1, len(v)+1)
    active = np.flatnonzero(sorted_v > threshold)[-1]
    return np.maximum(v-threshold[active], 0)
optimum = project_simplex(target)
x = np.full(3, 1/3)
for k in range(1000):
    grad = x-target
    vertex = np.eye(3)[int(np.argmin(grad))]
    gap = float(grad @ (x-vertex))
    error = 0.5*np.sum((x-target)**2)-0.5*np.sum((optimum-target)**2)
    assert error <= gap + 1e-12
    if gap < 1e-4:
        break
    gamma = 2/(k+2)
    x = (1-gamma)*x + gamma*vertex
# Recompute certificate at the returned iterate.
grad = x-target
vertex = np.eye(3)[int(np.argmin(grad))]
gap = float(grad @ (x-vertex))
print("Simplex iterate:", x)
print("Frank–Wolfe gap:", gap)
assert np.all(x >= 0) and np.isclose(x.sum(), 1)
assert gap < 0.01
```

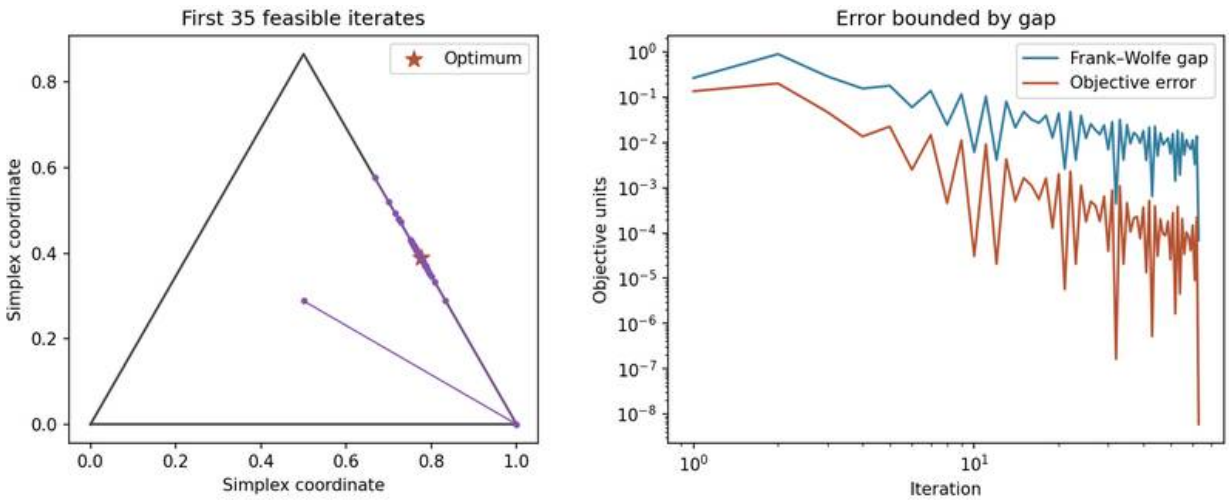


Figure 2 — A linear oracle supplies feasible vertices and a gap certificate. The path can zigzag even while objective error decreases.

The reference projection in the example computes the exact quadratic optimum for comparison. The conditional-gradient method itself never calls that projection. The distinction lets you test its gap bound rather than inferring convergence only from a decreasing trace. Recompute the gap after the last update so it describes the point actually returned.

A compact feasible set and a smooth objective support the usual convergence analysis. An unbounded oracle or a nonsmooth objective changes the problem. Variant methods can address additional structure, but do not transfer the basic guarantee without checking their assumptions. The survey [5] explains alternatives and their geometric motivation.

Prove the oracle gap, including an inexact oracle

The conditional-gradient gap is the improvement allowed by the best linearized feasible direction. Convexity gives $f(x^*) \geq f(x) + \nabla f(x)^\top (x^* - x)$. Since the exact oracle minimizes that linear score over C,

$$f(x) - f(x^*) \leq \nabla f(x)^\top (x - x^*) \leq \nabla f(x)^\top (x - v) = g_{\mathrm{FW}}(x).$$

Here x is the current feasible point, x^* an optimizer, and v the exact linear-oracle solution. The first inequality uses objective convexity; the second uses the oracle's comparison with x^*. These are the two ingredients that turn a computable directional quantity into a global error bound [5].

If a feasible approximate oracle returns $\tilde{v}$ satisfying $\nabla f(x)^\top \tilde{v} \leq \min_{z \in C} \nabla f(x)^\top z + \delta$, with known $\delta \geq 0$, then the valid bound is the computed gap plus δ. Here δ is the oracle's linear-objective error allowance. Feasibility of the update remains intact because $\tilde{v} \in C$, even when the gap needs correction.

For a simplex gradient $(2, -1, 3)$, the exact oracle chooses its second vertex, whose score is -1. Choosing the first vertex gives score 2, requiring allowance $\delta = 3$. At $x = (1/3, 1/3, 1/3)$, the exact gap is $7/3$, but this approximate-oracle gap is $-2/3$. Adding three recovers $7/3$. A negative uncorrected gap here indicates a poor oracle, not an objective error below zero.

7. Choose a method by structure, accuracy and information

There is no universally best update equation. The modeling decisions determine what information is available and which operations are inexpensive. Compare those operations before comparing a method's name or its iteration count.

For a modest smooth problem with exploitable curvature, Newton or an interior-point method may reach a precise solution through relatively few expensive steps. For a huge sparse model, factorization memory can dominate; first-order methods may use cheaper matrix–vector products. For a simple nonsmooth penalty, a proximal operation can be exact and cheap. For a complicated set with an easy linear oracle, conditional gradients may avoid a costly projection.

Distributed optimization introduces communication as another expensive operation. Local objectives may be separable while consensus constraints connect variables. ADMM can expose local subproblems, but convergence time must include communication rounds and local solve cost. A mathematically distributed formulation does not imply the implementation runs in parallel; the denoising example is entirely serial.

Primal–dual hybrid-gradient methods handle saddle-point formulations using primal and dual updates [4]. Their step sizes depend on operator norms and the structure of the split terms. Prac-

tical linear-programming systems such as PDLP [6] add preprocessing and adaptive techniques around that structure. A short theoretical recurrence and a production solver are different levels of description.

Requested precision can reverse a comparison. A method that rapidly produces a useful approximate solution may take much longer to achieve a very small certificate gap. Another method may spend substantial setup time but converge efficiently to high precision. Plot time to several stated tolerances, and report original-unit feasibility alongside the gap. This makes the trade-off reviewable.

Memory and repeated solves matter too. A parameter sweep can reuse data structures, factorization or a previous solution when supported. A different sparsity pattern may prevent that reuse. Include preprocessing and setup in the first-solve measurement, then separately report warm repeated solves. Do not compare one method's warm run with another's cold run silently.

Consensus: local solves and a shared average

Distributed ADMM makes the communication mechanism explicit. Suppose J participants share a vector decision but each has a local convex cost f_j. Introduce local copies x_j and shared variable z, and solve $\min \sum_{j=1}^{J} f_j(x_j)$ with $x_j = z$ for every j.

In scaled form, one consensus iteration is

$$
\begin{aligned}
x_j^{k+1} &= \arg\min_x \left[f_j(x) + \frac{\rho}{2} \|x - z^k + u_j^k\|_2^2 \right], \\
z^{k+1} &= \frac{1}{J} \sum_{j=1}^{J} (x_j^{k+1} + u_j^k), \\
u_j^{k+1} &= u_j^k + x_j^{k+1} - z^{k+1}.
\end{aligned}
$$

Here u_j is participant j's scaled multiplier, $\rho > 0$ the penalty, and k the iteration index. The first row consists of independent local solves. The second row minimizes the sum of quadratic consensus terms in z, whose derivative sets z to their average. The third row accumulates disagreement [3].

For scalar costs $f_j(x) = \frac{1}{2}(x - a_j)^2$, the local solve is $x_j^{k+1} = [a_j + \rho(z^k - u_j^k)]/(1 + \rho)$, where a_j is a local target. Two participants with targets zero and four must agree at the globally optimal value two. Their individually optimal values zero and four violate consensus; averaging and multiplier updates coordinate the compromise.

Communication is therefore part of the algorithm: the shared update needs each local quantity, while each local solve needs the latest shared value. A parallel implementation must measure message cost and synchronization as well as local arithmetic. Additional shared constraints change the z subproblem, often from a simple average to a constrained proximal operation.

8. A complete convex-optimization capstone

A capstone should make the formulation and evidence reproducible. Choose a synthetic signal estimator, geometric placement task or resource model. Write variables and units, a convexity argument, domains and a feasible-set interpretation before running the solver. State any relaxation and how its output becomes an implementable decision.

Build one small instance with an analytic answer or an independently computed certificate. Use it to check signs, shapes, scaling and stopping rules. Then vary one property at a time: noise strength, regularization, condition number, sparsity, problem size or tolerance. Keep the data seed fixed when comparing algorithms on the same instance.

Record objective values and residuals at the point actually returned. For splitting, track the coupling residual and the dual residual. For a conditional-gradient solve, report the current gap and oracle accuracy. For a barrier solve, distinguish the outer target from the computed certificate. For a relaxation, keep the lower bound and feasible rounded upper bound separate.

Assess the application outcome alongside numerical convergence. A denoiser may accurately minimize a penalty that removes a real event. A placement solver may obey every modeled rule while omitting a doorway requirement. A resource optimizer may be sensitive to an uncertain demand estimate. These are model-validation findings, not reasons to blur the meaning of a mathematical certificate.

For learned pipelines, split data before selecting regularization or fitting any preprocessing. Repeat solves on a validation set, choose the rule, then report the held-out outcome. The ML/DL evaluation chapter provides the broader learning protocol. Numerical convergence and predictive evaluation answer different questions and should appear in separate columns of the result table.

Modern view

The modern reading map combines surveys that organize algorithms with recent work that changes practical modeling or solving. The sources below were checked through October 4, 2026; unpublished manuscripts are identified explicitly.

Parikh and Boyd [2] organize proximal methods around the operation of solving a small convex subproblem. **Boyd et al. [3]** organize ADMM around separability, coupling and distributed statistical applications. Our teaching synthesis is to examine which subproblems become easy after a split and which new residuals must then be controlled. Splitting is valuable when it reduces total work, not simply because an objective has several terms.

Condat et al. [4] unify proximal splitting through primal–dual product spaces and monotone inclusions, revisiting parameter ranges and algorithmic variants. The reading connects apparently different recurrences through their operators and metrics. Use it to identify a method's actual assumptions, especially when a smooth quadratic term permits choices different from a generic smooth function. The unified perspective prevents arbitrary parameter combinations from being treated as guaranteed methods.

Pokutta's 2024 introduction [5] maps conditional gradients through linear oracles and feasible combinations. It helps compare projection-based methods with oracle-based alternatives and explains why feasible-set geometry affects behavior. The practical takeaway is to price the oracle and the representation, then use a valid gap. A cheap oracle can still require many iterations to obtain the precision an application needs.

The 2024 multi-robot survey by Shorinwa et al. [7] distinguishes distributed first-order, sequential convex-programming and ADMM approaches. It places communication, coordination and physical constraints alongside mathematical updates. Some robotics formulations are nonconvex or solved by successive convex subproblems, so convergence of a convex subproblem does not certify a global solution of the original trajectory or coordination task.

Applegate et al.'s PDLP manuscript [6], posted in 2025 and revised in 2026, provides recent evidence for practical large-scale first-order linear programming. **Schaller, Bemporad and Boyd's 2025 manuscript [8]** instead fits parameterized convex functions compatible with disciplined programming. The former concerns a solver pipeline; the latter concerns learned functions that retain convexity in decision variables. Both are manuscripts, and neither makes arbitrary neural-network training convex.

Question	Useful family or reading	Evidence to collect
Is a constraint projection cheap?	projected/proximal methods [2]	projection cost and optimality residual
Can terms be split into easy pieces?	ADMM and primal–dual splitting [3,4]	local solve cost and coupling residuals
Is linear minimization easier than projection?	conditional gradients [5]	oracle cost, feasible iterate and gap
Does communication dominate?	distributed-method taxonomy [7]	rounds, messages, local work and physical feasibility
Is the linear model too large for factorization?	practical first-order LP [6]	memory, accuracy and original-unit residuals
Can learned functions preserve decision-variable convexity?	parametric convex modeling [8]	structural restrictions and held-out model adequacy

Our synthesis is that modeling and computation increasingly meet in reusable structured components: convex subproblems, inexpensive proximal maps, learned convex functions and hardware-aware solver operations. Their value still depends on an explicit formulation, valid assumptions and measured end-to-end behavior. A solver benchmark does not establish generalization; a prediction benchmark does not establish feasibility.

Open problems include automatic structure detection, robust certificates under poor scaling, communication-efficient solves, differentiability when active constraints change and reliable integration of learned predictions with exact constraints. A useful research question isolates one issue and preserves an independently checkable outcome. For example: "Does this warm start reduce total solve time while preserving the same certificate tolerance?"

The track now supplies a complete path: convex sets and functions, problem formulation, duality, estimation, numerical methods, constrained solvers and structured applications. Continue by choosing one textbook exercise family and building a small original experiment around its

assumptions. The central habit is to state what is optimized, why the model is convex, what the algorithm computes and what evidence justifies accepting the result.

Key takeaways

- Geometric modeling includes projection, separation, centers, ellipsoids and structured placement.
- Unknown discrete choices and disjunctions can make the original application nonconvex.
- Relaxed lower bounds and feasible implementable solutions have different roles.
- Proximal operations handle simple nonsmooth penalties and constraints directly.
- ADMM splits coupling into tractable operations and requires primal and dual residual checks.
- Conditional gradients maintain feasibility through a linear oracle and supply a gap under stated assumptions.
- Compare original-unit accuracy, setup, memory, repeated solves and application outcomes.

Exercises

1. **Geometric center.** Derive the largest inscribed ball for an axis-aligned rectangle. Explain why its center may be nonunique when one side is longer than the other.

   Hint

   The shorter side limits the radius. The center can move along the longer dimension while maintaining enough boundary clearance.

2. **Floorplanning model.** Write convex nonoverlap constraints for two rectangles after choosing which one is left of the other. Explain what changes if the ordering is unknown.

   Hint

   A fixed order gives an affine separation inequality. Unknown order requires an either-or choice, which is a disjunction rather than one convex halfspace.

3. **Regularization sweep.** Run the denoising example with five regularization strengths. Compare objective value, synthetic reconstruction error and the number of nearly zero adjacent differences.

   Hint

   Objectives with different regularization strengths are different functions, so their raw values are not a fair convergence comparison. Keep model quality and solve accuracy separate.

4. **Penalty update.** Change the ADMM penalty halfway through the experiment. Update the factorization and scaled dual variable consistently, then compare residual traces with the fixed-penalty version.

Hint

The unscaled multiplier is the penalty times the scaled dual variable. Preserve that product when changing the penalty.

5. **Inexact oracle.** Replace the simplex linear oracle with a deliberately suboptimal vertex. Identify how its objective error changes the certificate bound and whether the iterate remains feasible.

Hint

Feasibility follows from convex combinations of vertices. The gap based on an inexact oracle needs an additive bound on the oracle's linear-objective error.

6. **Capstone report.** Compare two appropriate algorithms on one synthetic convex application. Include the formulation, convexity proof, stopping policy, original-unit residuals, timing, memory and one model failure.

Hint

Use identical data and acceptance tolerances. Separate setup from repeated-solve costs, and report an unresolved numerical run rather than counting it as a successful solve.

References

1. S. Boyd and L. Vandenberghe, *Convex Optimization*, Cambridge University Press, 2004, Chapter 8, with Chapters 4 and 6 for modeling and approximation. Book
2. N. Parikh and S. Boyd, "Proximal Algorithms," Foundations and Trends in Optimization, 2014. Survey
3. S. Boyd, N. Parikh, E. Chu, B. Peleato and J. Eckstein, "Distributed Optimization and Statistical Learning via the Alternating Direction Method of Multipliers," Foundations and Trends in Machine Learning, 2011. Review
4. L. Condat, D. Kitahara, A. Contreras and A. Hirabayashi, "Proximal Splitting Algorithms for Convex Optimization: A Tour of Recent Advances, with New Twists," SIAM Review, 2023. Author manuscript
5. S. Pokutta, "The Frank-Wolfe Algorithm: A Short Introduction," Jahresbericht der Deutschen Mathematiker-Vereinigung, 2024. Survey
6. D. Applegate, M. Díaz, O. Hinder, H. Lu, M. Lubin, B. O'Donoghue and W. Schudy, "PDLP: A Practical First-Order Method for Large-Scale Linear Programming," arXiv preprint, 2025, revised 2026. Paper

7. O. Shorinwa, T. Halsted, J. Yu and M. Schwager, “Distributed Optimization Methods for Multi-Robot Systems: Part 2—A Survey,” IEEE Robotics & Automation Magazine, 2024. Survey
8. M. Schaller, A. Bemporad and S. Boyd, “Learning Parametric Convex Functions,” manuscript, June 2025. Author page

Part VIII

Information Theory

IT 01 • Entropy, Relative Entropy and Mutual Information

Build information measures from probabilities, derive chain rules and data processing, and distinguish exact identities from the uncertainty of estimating them from data.

Online edition: https://blog.shyandram.dev/en/learn/information-theory/it-01-entropy-mutual-information/

What you'll learn

- Explain why a logarithm converts probability into an additive measure of surprise.
- Calculate entropy, conditional entropy, relative entropy and mutual information.
- Derive chain rules and identify the equality conditions in fundamental inequalities.
- Explain what processing can discard, preserve or add without confusing randomness with information.
- Check information identities using finite probability tables and complete CPU examples.
- Distinguish a population quantity, an empirical estimate and a learning objective.

The big picture

A sensor sends a sequence of measurements. Some values are predictable; others are surprising. A compressor benefits from predictability, while a receiver benefits from measurements that reduce uncertainty about something it wants to know. These two benefits are related but different. A stream can be highly random and contain almost no useful evidence about a scene.

> **Explain like I'm five**
>
> Imagine guessing what is inside a box. "It is a toy" may not help much if every box contains a toy. "It is the only red train" can settle the question. Information theory measures how much uncertainty a message removes, using a probability model of the possibilities.

This track follows Thomas M. Cover and Joy A. Thomas's *Elements of Information Theory*, second edition [1]. Its quantities have operational roles: entropy will guide compression, mutual information will describe communication limits, and relative entropy will connect probability mismatch to extra description length and statistical discrimination. We begin with finite alphabets so every quantity can be checked by explicit summation.

The Stanford entropy notes [2] give an accessible companion treatment. We use explicit synthetic probability models so the definitions and numerical checks can be inspected directly.

How the track fits together

Tutorial	Main question	Core second-edition chapters
IT 01	what does information measure?	1–2, basic tools from 17
IT 02	why do long sequences become predictable in aggregate?	3–4
IT 03	how can descriptions become short without losing symbols?	5, 13–14
IT 04	how much can a noisy channel communicate reliably?	7
IT 05	what changes for continuous and Gaussian variables?	8–9
IT 06	what rate is required when some distortion is allowed?	10
IT 07	how do information measures support inference and decisions?	6, 11–12, 16
IT 08	what changes with several users, constraints and learning?	15, advanced tools from 17

This grouping covers the book's main themes without pretending that every theorem fits into a short derivation. Work through definitions, then numerical checks, then proofs. If a calculation feels mysterious, make the probability table explicit before adding a neural estimator or a communication system.

Random variables, outcomes and the probability model

A random variable describes what could be observed before the observation is known. Its probabilities belong to a model of the experiment. The same observed value can be surprising under one model and ordinary under another.

Let X take values in a finite alphabet $\mathcal{X}$. Write

$$p_X(x) = \Pr(X = x), \qquad p_X(x) \geq 0, \qquad \sum_{x \in \mathcal{X}} p_X(x) = 1.$$

Here X is the random variable, x is a possible outcome, $\mathcal{X}$ is the set of outcomes, p_X is its probability mass function, and Pr denotes probability. A joint distribution $p_{XY}(x, y)$ assigns probabilities to pairs. Marginals come from summation:

$$p_X(x) = \sum_y p_{XY}(x, y), \qquad p_Y(y) = \sum_x p_{XY}(x, y).$$

Here Y is another finite random variable, y is one of its outcomes, and p_Y is its marginal mass function. Conditional probabilities use $p_{X|Y}(x \mid y) = p_{XY}(x, y)/p_Y(y)$ when $p_Y(y) > 0$. Conditioning on an event with zero probability needs separate conventions and will not contribute to the finite sums below.

We use base-two logarithms unless explicitly stated otherwise. Results are then in **bits**. Natural logarithms give **nats**; dividing a value in nats by $\ln 2$ converts it to bits. A bit is a unit of information, not necessarily a literal stored binary digit in a particular file format.

Keep the experiment fixed while comparing distributions. If one model describes grayscale intensities and another describes whole images, their numerical entropies are not directly comparable. Changing the variable, alphabet, quantization or conditioning information changes the question.

Surprise becomes additive through a logarithm

An unlikely outcome brings more surprise. Two independent observations should bring the sum of their individual surprises. A logarithm makes this addition agree with probability multiplication.

Define the self-information of an outcome:

$$\iota_X(x) = -\log_2 p_X(x).$$

Here $\iota_X(x)$ is the surprise associated with outcome x under model p_X. If independent outcomes have probabilities a and b, their joint probability is ab, and $-\log_2(ab) = -\log_2 a - \log_2 b$. This is why doubling the number of equally likely possibilities adds one bit rather than multiplying the information value.

An outcome with probability one has zero surprise. A probability-1/8 outcome has three bits. These statements describe the probability model before seeing the outcome; observing an unusual symbol does not by itself make that symbol intrinsically valuable.

If a model gives an observed outcome probability zero, its surprise is infinite. In a prediction application, that may indicate a model-support error: the model declared an actually possible event impossible. Adding a small numerical constant changes the model. It can be useful, but it must not be confused with evaluating the original distribution.

The expected surprise is entropy:

$$H(X) = -\sum_x p_X(x) \log_2 p_X(x) = \mathbb{E}[\iota_X(X)].$$

Here $H(X)$ is the entropy of X, and $\mathbb{E}$ averages over its probability distribution. The convention $0 \log 0 = 0$ follows by taking a limit; outcomes that never occur do not contribute to the average.

Entropy is therefore a property of a distribution, not of one realized sample. A rare outcome can have high surprise even when the average entropy is small. An image may contain a visually striking pattern and still be easy for a suitable probability model to predict.

Entropy bounds and the binary example

Uncertainty is zero when the outcome is certain. For a fixed number of possibilities, uncertainty is greatest when all are equally likely.

If an alphabet has K possible values,

$$0 \le H(X) \le \log_2 K.$$

Here $K = |\mathcal{X}|$ is the alphabet size. The lower bound is attained by a point mass; the upper bound is attained by the uniform distribution. If only K_+ outcomes have positive probability, the sharper bound uses $\log_2 K_+$.

For a binary variable with probability p of outcome one,

$$h_2(p) = -p \log_2 p - (1-p) \log_2 (1-p).$$

Here h_2 is the binary entropy function and $p \in [0, 1]$. It is symmetric around $p = 1/2$. Its maximum is one bit, and it approaches zero near either endpoint. A highly biased source is easier to predict than a fair source even though both use the same two-symbol alphabet.

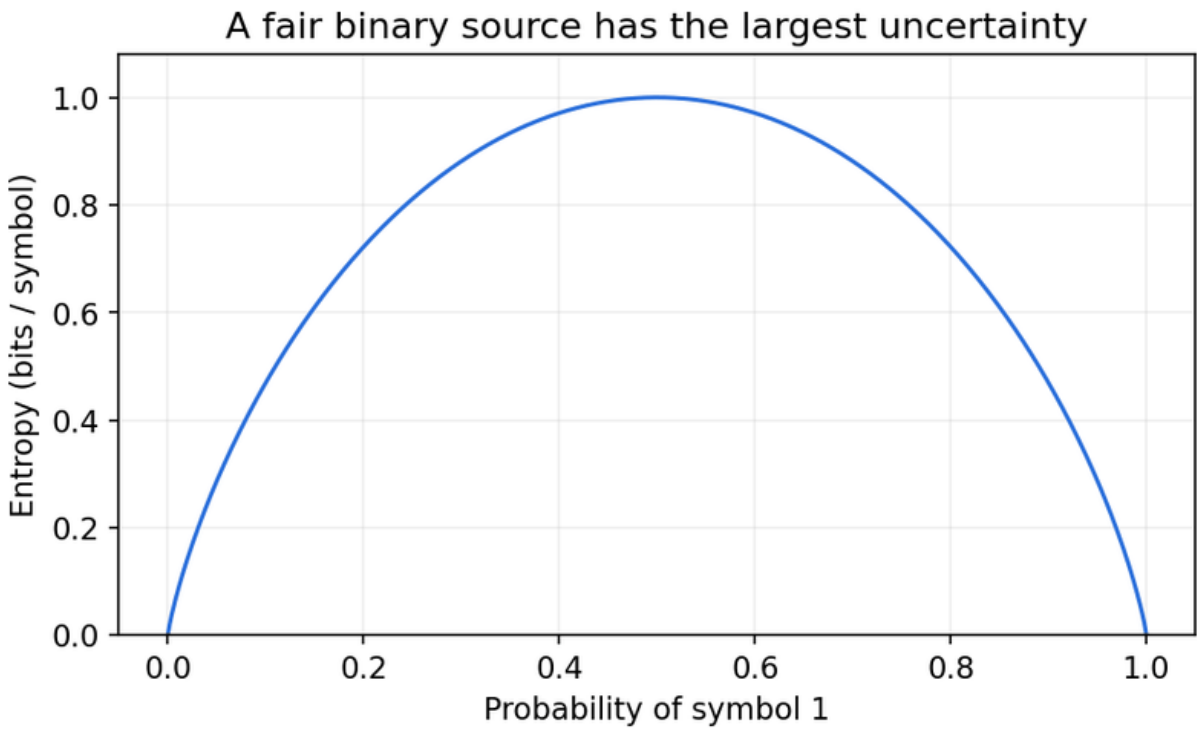


Figure 1 — Analytical binary entropy. The number of possible symbols stays fixed while their probabilities change.

Entropy is concave in the distribution. Mixing two source distributions while hiding which one was selected cannot reduce entropy below their weighted average. Revealing that selection may remove part of the added uncertainty. This explains why mixtures often appear less predictable than their individual components.

The maximum-entropy bound can be proved using relative entropy to a uniform model; we will obtain it after defining divergence. The proof is useful because the same nonnegativity argument will also establish why conditioning reduces average entropy and why mutual information cannot be negative.

Joint entropy and the chain rule

Describing two variables can be cheaper than describing each independently. Once the first is known, shared structure may make the second easier to predict.

Joint entropy is

$$H(X,Y) = -\sum_{x,y} p_{XY}(x,y)\log_2 p_{XY}(x,y).$$

Here $H(X,Y)$ measures uncertainty about the pair (X,Y). Conditional entropy is

$$H(Y \mid X) = -\sum_{x,y} p_{XY}(x,y)\log_2 p_{Y|X}(y \mid x).$$

Here $H(Y \mid X)$ is average remaining uncertainty about Y when X is available. It weights the entropy of each conditional distribution by the probability of the corresponding x.

Using $p_{XY}(x,y) = p_X(x)p_{Y|X}(y \mid x)$ gives

$$H(X,Y) = H(X) + H(Y \mid X).$$

Every term refers to the same joint model. Taking the logarithm of the product creates two terms, and summing the first over y leaves the marginal $p_X(x)$. This derivation explains the identity rather than merely providing a formula to memorize.

For a sequence $X_1, \ldots, X_n$:

$$H(X_1, \ldots, X_n) = \sum_{i=1}^{n} H(X_i \mid X_1, \ldots, X_{i-1}).$$

Here n is the number of variables and i is the sequence index; the first term has no conditioning variables. This rule requires no independence assumption. Under independence, every conditional term equals its marginal entropy, so the joint entropy is the sum.

For an image, this distinction matters. A histogram describes marginal pixel frequencies but discards spatial dependencies. Two images with the same histogram can have very different predictability when nearby pixels are known. The sequence model determines whether those dependencies can shorten a description.

Conditioning helps on average, not necessarily for every outcome

Knowing another variable cannot make the optimal average prediction worse. But a particular observation can reveal that you are in an unusually uncertain situation.

$$H(X \mid Y) \le H(X), \qquad H(X,Y) \le H(X) + H(Y).$$

Here the first inequality compares average conditional uncertainty with marginal uncertainty; the second follows from the chain rule. Equality holds when X and Y are independent.

The phrase "on average" is essential. Suppose most days a sensor always outputs zero, but on rare diagnostic days it produces a fair binary outcome. Learning that today is a diagnostic day increases uncertainty relative to the marginal mixture. Learning the day type nevertheless lowers average uncertainty when averaged over both normal and diagnostic days.

Another distinction concerns more conditioning:

$$H(X \mid Y, Z) \leq H(X \mid Y).$$

Here Z is additional information. This inequality concerns the entropies of one fixed joint distribution. It does not say that a finite neural network trained with extra inputs must generalize better. Optimization, limited data and spurious correlations can prevent a learned predictor from reaching the optimal conditional prediction.

Conditional entropy is nonnegative for finite variables. Later, differential entropy of continuous variables will behave differently. Do not transfer every finite-alphabet inequality to raw continuous entropy without checking its assumptions.

A numerical conditioning check

Make the diagnostic-day example explicit: let $Y = 1$ denote a diagnostic day with probability 0.1. On ordinary days $X = 0$; on diagnostic days X is a fair bit. Then the marginal probability of $X = 1$ is $0.1 \times 0.5 = 0.05$, so $H(X) = h_2(0.05) = 0.286397$ bits.

Knowing that this is a diagnostic day raises uncertainty to $H(X \mid Y = 1) = 1$ bit. Averaging over all days instead gives $H(X \mid Y) = 0.9 \times 0 + 0.1 \times 1 = 0.1$ bits. The conditioning inequality holds because it compares 0.1 with 0.286397, rather than comparing every individual conditional entropy with the marginal entropy. The day label removes $I(X; Y) = 0.186397$ bits on average.

Relative entropy measures probability mismatch

A probability model can waste description length by expecting the wrong outcomes. Relative entropy measures that mismatch in the direction of the distribution that actually generates observations.

$$D(p\|q) = \sum_x p(x) \log_2 \frac{p(x)}{q(x)}.$$

Here p and q are probability mass functions on the same alphabet, and $D(p\|q)$ is their relative entropy, also called Kullback–Leibler divergence [1]. Terms with $p(x) = 0$ contribute zero. If $p(x) > 0$ but $q(x) = 0$, the divergence is infinite.

Nonnegativity follows from concavity of the logarithm:

$$\begin{aligned} -D(p\|q) &= \mathbb{E}_p\left[\log_2 \frac{q(X)}{p(X)}\right] \\ &\leq \log_2 \sum_{x:p(x)>0} q(x) \leq 0. \end{aligned}$$

Here $\mathbb{E}_p$ means expectation under p. The first inequality is Jensen's inequality; the final sum is at most one. Equality requires matching distributions. This proof also handles a model that places some probability outside the source support.

The direction matters. $D(p\|q)$ generally differs from $D(q\|p)$. Divergence is not a metric: it does not obey all the requirements of a geometric distance, including symmetry and a general triangle inequality. Calling it a "distance" informally should not justify operations that require a genuine metric.

Compare a fair source with a model that gives probability 0.9 to outcome one. The model is overconfident in one direction and pays for the frequent unexpected zeros. Reversing the distributions describes a different experiment and a different average penalty.

For a uniform distribution $u(x) = 1/K$, expanding the divergence gives $D(p\|u) = \log_2 K - H(p)$. Since it is nonnegative, the entropy upper bound follows. The equality condition also establishes why the uniform source uniquely maximizes entropy on a fixed finite alphabet.

Cross-entropy connects prediction and coding

Cross-entropy scores observations using a proposed probability model. It contains both the source's unavoidable uncertainty and the cost of choosing an inaccurate model.

$$H(p, q) = -\sum_x p(x) \log_2 q(x) = H(p) + D(p\|q).$$

Here $H(p, q)$ is cross-entropy, $H(p)$ is the entropy of the true distribution, and q supplies the predicted probabilities. Under idealized probability coding, $-\log_2 q(x)$ is a description-length target. Actual codes also involve integer lengths, framing, model description and implementation overhead.

If p stays fixed while parameters change q, minimizing population cross-entropy also minimizes divergence. With data, the empirical mean negative log-probability estimates this objective:

$$\widehat{H}_N = -\frac{1}{N} \sum_{j=1}^{N} \log_2 q_\theta(x_j).$$

Here N is the number of observations, x_j is observation j, and θ denotes model parameters. The hat marks an empirical quantity. Its connection to the population average depends on how observations were sampled and whether they were reused to select the model.

A classifier applies the same idea conditionally to labels given features. A generative model applies it to observations or sequences. A low training value is evidence about fit to training observations, not automatically about future data. This distinction connects information theory to the evaluation rules in the ML/DL track.

When training code uses natural logarithms, the loss is in nats. Multiplying by a positive constant changes gradient scale but not the mathematical optimum. Comparing numerical losses across papers requires consistent units, normalization and prediction targets.

Worked example: one table, three information questions

Read the following table as a population model: each cell is the probability of observing that pair. It describes two fair bits that agree more often than they disagree.

Joint probability	$Y = 0$	$Y = 1$	Row total
$X = 0$	0.4	0.1	0.5
$X = 1$	0.1	0.4	0.5
Column total	0.5	0.5	1.0

Rows and columns sum to the marginal probabilities. Both marginals are fair, so $H(X) = H(Y) = 1$ bit. After observing $X = 0$, divide that row by its total 0.5: the conditional probabilities of Y are 0.8 and 0.2. The other row reverses them and has the same entropy.

$$H(Y \mid X) = 0.5h_2(0.2) + 0.5h_2(0.2) = 0.721928,$$
$$H(X, Y) = 1 + 0.721928 = 1.721928,$$
$$I(X; Y) = 1 - 0.721928 = 0.278072.$$

Here h_2 is binary entropy, $H(Y \mid X)$ averages the two conditional rows, $H(X, Y)$ describes the pair, and $I(X; Y)$ is the reduction in uncertainty about either bit after seeing the other. All displayed values are in bits and rounded to six decimals; the code below computes them without rounding.

The same mutual information comes from comparing this joint table with the independent table, which has probability 0.25 in every cell. In agreeing cells the ratio is $0.4/0.25 = 1.6$; in disagreeing cells it is $0.1/0.25 = 0.4$. Their weighted log ratios sum to 0.278072. A single cell's log ratio can be negative even though the average divergence is nonnegative.

Now keep a fair one-bit source but predict probabilities 0.1 and 0.9. Its entropy stays one bit. Its cross-entropy becomes $-0.5 \log_2(0.1) - 0.5 \log_2(0.9) = 1.736966$ bits, leaving 0.736966 bits of model mismatch. The joint-table question measures dependence; this prediction question measures a wrong model. Both use divergence, but they compare different distributions.

This table also gives a simple interpretation for classification. Predicting $Y = X$ is correct with probability 0.8, while the conditional distribution still contains 0.721928 bits of uncertainty. Accuracy counts correct decisions; entropy evaluates an entire predictive distribution. Neither number can be substituted for the other.

Code: exact finite-table checks

The following complete CPU example masks zero-probability terms instead of replacing them with a small constant. The helper functions also check that inputs are normalized probability arrays.

```
import numpy as np

def probability(a):
```

```
    a = np.asarray(a, dtype=float)
    assert np.all(a >= 0) and np.isclose(a.sum(), 1)
    return a

def entropy(a):
    a = probability(a)
    positive = a > 0
    return -np.sum(a[positive] * np.log2(a[positive]))

def divergence(p, q):
    p, q = probability(p), probability(q)
    assert p.shape == q.shape
    positive = p > 0
    if np.any(q[positive] == 0):
        return np.inf
    return np.sum(p[positive] * np.log2(p[positive] / q[positive]))

joint = probability([[0.4, 0.1], [0.1, 0.4]])
px, py = joint.sum(axis=1), joint.sum(axis=0)
independent = np.outer(px, py)
hx, hy, hxy = entropy(px), entropy(py), entropy(joint)
information = divergence(joint, independent)
print("H(X), H(Y), H(X,Y):", hx, hy, hxy)
print("H(Y|X), I(X;Y):", hxy - hx, information)
assert np.isclose(information, hx + hy - hxy)
assert np.isclose(divergence(independent, independent), 0)

p, q = probability([0.5, 0.5]), probability([0.1, 0.9])
cross = -np.sum(p * np.log2(q))
print("Source entropy / cross-entropy / mismatch:",
      entropy(p), cross, divergence(p, q))
assert np.isclose(cross, entropy(p) + divergence(p, q))
assert np.isinf(divergence(probability([0.5, 0.5]), probability([0, 1])))
```

The identities are checked on a specified population table. An empirical table built from finite observations adds estimation uncertainty; the algebra can be exact while the estimated value is statistically inaccurate.

Mutual information measures dependence

Two variables share information when knowing one changes what we expect about the other. Shared information is not limited to linear relationships.

$$\begin{aligned} I(X;Y) &= D(p_{XY} \| p_X p_Y) \\ &= H(X) + H(Y) - H(X,Y) \\ &= H(X) - H(X \mid Y). \end{aligned}$$

Here $I(X;Y)$ is mutual information and $p_X p_Y$ is the product of the marginals. It is nonnegative, symmetric and zero exactly under independence. Its upper bound is the smaller of the two

marginal entropies.

The divergence form asks how different the actual pair distribution is from an independent pair model. The entropy form asks how much average uncertainty one variable removes about the other. These viewpoints agree but emphasize different applications.

Correlation is narrower: zero correlation does not generally establish independence. Nonlinear dependencies can be strong even when a particular linear summary is zero. Conversely, estimating mutual information well in a high-dimensional space is often harder than estimating a correlation.

Conditional mutual information is

$$I(X;Y \mid Z) = H(X \mid Z) - H(X \mid Y,Z).$$

Here Z specifies the conditioning information. This quantity averages dependence within each conditional distribution. It is nonnegative, but it can be larger or smaller than unconditional $I(X;Y)$.

For independent fair binary X and Y, let $Z = X \oplus Y$ be their parity. Unconditionally, X tells us nothing about Y. Given Z, knowing X determines Y. Thus the unconditional mutual information is zero and the conditional value is one bit. Conditioning can reveal dependencies rather than merely subtracting a fixed amount of information.

The information chain rule is

$$I(X;Y,Z) = I(X;Y) + I(X;Z \mid Y).$$

Here (Y,Z) is a joint observation. The second term measures the extra information supplied by Z after Y is already available. Adding both unconditional values would double-count shared evidence or miss joint effects.

Mutual information as average predictive improvement

Expand the joint-distribution ratio using $p_{XY}(x,y) = p_X(x)p_{Y|X}(y \mid x)$. The marginal factor cancels, giving

$$I(X;Y) = \sum_x p_X(x) D(p_{Y|X=x} \| p_Y).$$

Here $p_{Y|X=x}$ is the conditional distribution of Y after observing $X = x$, p_Y the marginal prediction made without that observation, and divergence uses bits. This form expresses information as the average change from an unconditional prediction to a conditional one.

It also proves independence is the equality case. Every conditional divergence is nonnegative. Their weighted sum is zero only when each positive-probability x leaves the prediction unchanged, which is exactly the finite-table independence condition. In the worked table, each conditional row differs from the fair marginal by the same divergence, $1 - h_2(0.2)$.

Under logarithmic prediction loss, the best unconditional predictor pays $H(Y)$, and the best conditional predictor pays $H(Y \mid X)$. Their difference is mutual information. This is a population

statement about optimal distributions: a fitted predictor can fail to realize the gain because it lacks data, capacity or an adequate optimization procedure. The derivation identifies the achievable predictive benefit and the reason its empirical estimate needs separate validation.

Data processing, invertibility and added randomness

Processing cannot create new evidence about an original source if it has access only to the current observation. It can add random variation, but variation is not the same as information about that source.

A Markov relation $X \to Y \to Z$ means

$$p(x, y, z) = p_X(x)p_{Y|X}(y \mid x)p_{Z|Y}(z \mid y).$$

Here Z is generated from Y without extra access to X. This condition implies the data-processing inequality:

$$I(X; Z) \leq I(X; Y).$$

The same X appears on both sides. The inequality does not assert $H(Z) \leq H(Y)$ for arbitrary randomized processing. Adding independent noise can increase output entropy while decreasing information about the input.

A useful proof expands $I(X; Y, Z)$ in two orders. The Markov relation makes $I(X; Z \mid Y) = 0$. The other expansion includes nonnegative $I(X; Y \mid Z)$, leaving $I(X; Y) = I(X; Z) + I(X; Y \mid Z)$. Equality means that Z loses no relevant information about X beyond what Y already supplies.

A deterministic transformation obeys $H(f(X)) \leq H(X)$ because it merges possible outcomes. If it is one-to-one on the support, no uncertainty is lost. An invertible transformation of either variable also preserves mutual information. Compression through a many-to-one quantizer is different: it can discard dependencies.

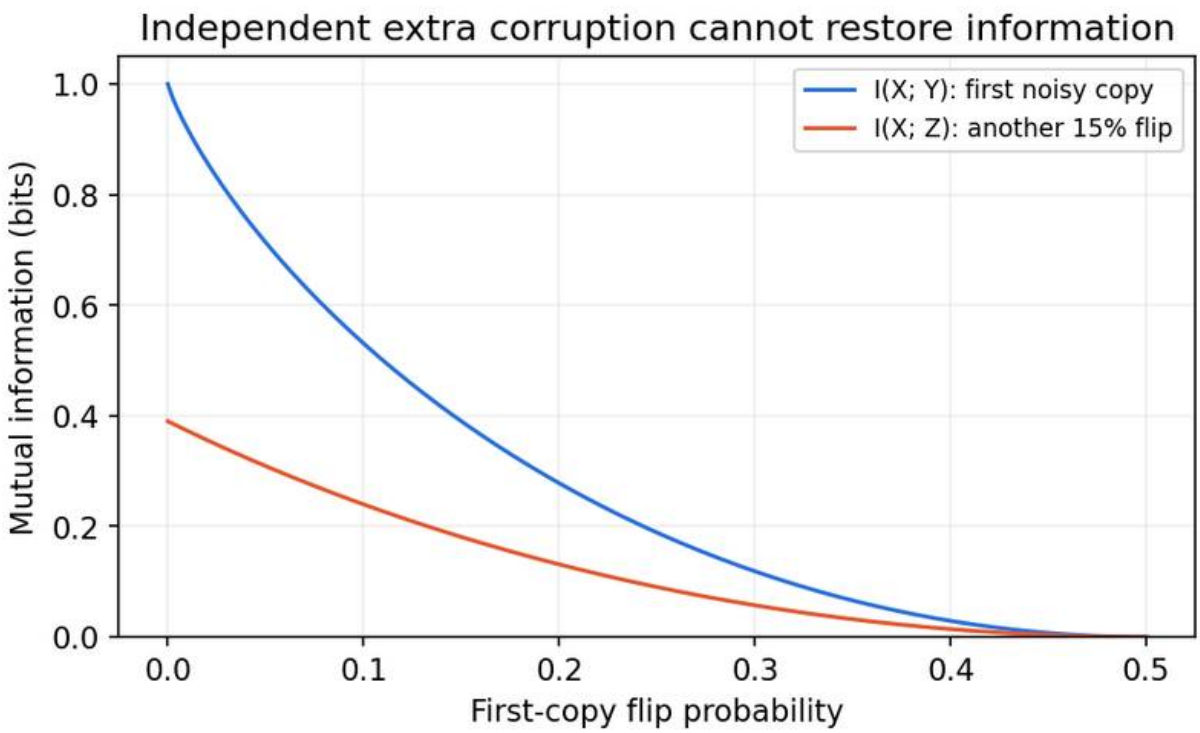


Figure 2 — Analytical data processing. The second channel independently flips its input with probability 0.15; the curves compare information about the same original source.

Code: processing and parity

This second standalone example constructs the full three-variable probability table. It checks the processing inequality and the parity example without a fitted estimator.

```
import numpy as np

def entropy(p):
    p = np.asarray(p, dtype=float)
    assert np.all(p >= 0) and np.isclose(p.sum(), 1)
    q = p[p > 0]
    return -np.sum(q * np.log2(q))

def mutual(joint):
    return entropy(joint.sum(axis=1)) + entropy(joint.sum(axis=0)) - entropy(joint)

pxy = np.array([[0.4, 0.1], [0.1, 0.4]])
channel = np.array([[0.85, 0.15], [0.15, 0.85]])
pxyz = pxy[:, :, None] * channel[None, :, :]
pxz = pxyz.sum(axis=1)
print("Before / after processing:", mutual(pxy), mutual(pxz))
assert mutual(pxz) <= mutual(pxy) + 1e-12
```

```
parity = np.zeros((2, 2, 2))
for x in range(2):
    for y in range(2):
        parity[x, y, x ^ y] = 0.25
pxz, pyz = parity.sum(axis=1), parity.sum(axis=0)
pz = parity.sum(axis=(0, 1))
conditional = entropy(pxz) + entropy(pyz) - entropy(pz) - entropy(parity)
print("Parity I(X;Y) / I(X;Y|Z):",
      mutual(parity.sum(axis=2)), conditional)
assert np.isclose(conditional, 1)
```

When a reported estimated mutual information violates data processing, possible causes include sampling error, different estimator biases or a violated Markov assumption. The exact theorem has not ceased to hold.

Estimation requires more than an identity

Counting observed outcomes is easy. Deciding how much unobserved probability remains is not. Large alphabets and strong dependence can make information estimates fragile.

For counts N_x among N observations, the empirical distribution and plug-in entropy are

$$\hat{p}_N(x) = \frac{N_x}{N}, \qquad \widehat{H}_N = -\sum_x \hat{p}_N(x) \log_2 \hat{p}_N(x).$$

Here N_x is the count of outcome x, N is the total sample count and hats denote estimates. Unobserved outcomes have empirical probability zero even if their true probability is positive.

For a fixed finite alphabet and independent sampling, the estimate converges under suitable conditions as samples grow. At small sample sizes, it typically underestimates entropy because the empirical distribution misses possible outcomes and appears too concentrated. Estimating a joint distribution requires covering a larger alphabet than either marginal alone.

Quantization also defines the variable. Binning a continuous feature into 8 levels and into 256 levels asks different finite-alphabet questions. Finer bins can add genuine resolution but also create more sparse cells. A smooth-looking information curve does not remove that trade-off.

The following standalone simulation checks average plug-in estimates for a known uniform source. It uses repeated synthetic samples, not a confidence claim from one draw.

```
import numpy as np

rng = np.random.default_rng(31)
alphabet = 20
truth = np.log2(alphabet)
for n in [40, 400, 4000]:
    estimates = []
    for _ in range(200):
```

```
        sample = rng.integers(alphabet, size=n)
        p = np.bincount(sample, minlength=alphabet) / n
        positive = p > 0
        estimates.append(-np.sum(p[positive] * np.log2(p[positive])))
    print("N:", n, "true entropy:", round(truth, 4),
          "mean estimate:", round(np.mean(estimates), 4))
assert abs(np.mean(estimates) - truth) < 0.02
```

For practical work, specify the sampling unit, feature transformations, estimator family, hyperparameter selection and uncertainty assessment. Repeated frames from one recording are not equivalent to independently sampled scenes. Evaluating an estimator on distributions with known information values is a useful first check.

Modern view

The basic quantities remain the same, but current applications often involve high-dimensional learned representations. The difficult step is frequently estimating or optimizing the quantity under a model, rather than writing its definition.

Butakov and colleagues [3] study mutual-information estimation through normalizing flows in NeurIPS 2024. Their method uses invertible transformations to simplify estimation while preserving the target information under stated assumptions. This illustrates a distinction: changing coordinates can help an estimator without creating information. The paper is an original method, not a survey or evidence that every neural estimator is unbiased.

Aubret, Matignon and Hassas's 2023 survey [4] organizes intrinsic motivation through **surprise, novelty and skill learning**. It shows why an information quantity must be tied to its target: unpredictable observations, newly visited situations and controllable skills are different objectives. Our interpretation is that "maximize entropy" needs a specified variable and conditioning structure before it becomes a useful design rule.

El Gamal's 2026 *Information Theory and Statistical Learning* manuscript [5] connects divergence-based objectives with predictive and generative modeling. It is a **draft chapter proposed for a forthcoming third edition**, not the published third edition or a replacement for the second-edition chapter numbers used here.

Open problems include reliable estimation in sparse high-dimensional spaces, comparison of estimators with different biases, dependence-aware evaluation and the relationship between a convenient information objective and the actual task. A learning method can improve an estimated lower bound without proving that its downstream decisions improved.

Use this chapter as a checklist when reading such claims: Which random variables? Which joint distribution? Which units? Is the number exact, estimated, bounded or optimized? What supports the conditional-independence assumption? These questions connect mathematical precision with useful empirical evidence.

Key takeaways

- Entropy is expected surprise under a probability distribution, not a property of one observed value.
- Chain rules account for shared structure and require no independence assumption.
- Conditioning reduces average uncertainty; individual conditional outcomes can behave differently.
- Relative entropy is directional, support-sensitive and nonnegative.
- Cross-entropy separates source uncertainty from model mismatch.
- Mutual information measures dependence; conditional dependence can differ sharply from unconditional dependence.
- Data processing limits evidence about a fixed source, while output randomness may increase.
- Empirical information estimates inherit sampling, quantization and model assumptions.

Exercises

1. A three-symbol source has probabilities $(1/2, 1/4, 1/4)$. Compute each outcome's surprise and the entropy. Compare it with a uniform three-symbol source.

   Hint

   Average the logarithmic surprises with their probabilities.

2. Construct a joint binary table with uniform marginals and mutual information zero, then another with mutual information one bit. Explain their conditional entropies.

   Hint

   Compare an outer product with a diagonal table.

3. Expand $D(p\|u)$ for a uniform model on K symbols and use nonnegativity to prove the entropy upper bound. State its equality condition.

   Hint

   The divergence becomes $\log_2 K - H(p)$.

4. Give a concrete model where $H(X \mid Y = y) > H(X)$ for one value y, while the average conditioning inequality still holds.

   Hint

   Use rare fair diagnostic days and common deterministic days.

5. Extend the processing example with a third noisy channel. Check mutual information after every stage and identify the independence assumptions.

Hint

Multiply transition probabilities before summing out intermediate states.

6. Repeat the entropy-estimation simulation with a larger alphabet and the same sample sizes. Describe which conclusions concern the estimator rather than the true entropy.

Hint

Record unseen-symbol counts as well as estimated entropy.

References

1. T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006, Chapters 1–2 and 17. Publisher book and contents.
2. T. Weissman, "Entropy, Relative Entropy, and Mutual Information," Stanford EE276 Lecture 2 notes, 2024. Primary lecture notes.
3. I. D. Butakov, A. D. Tolmachev, S. V. Malanchuk, A. M. Neopryatnaya and A. A. Frolov, "Mutual Information Estimation via Normalizing Flows," NeurIPS, 2024. Proceedings.
4. A. Aubret, L. Matignon and S. Hassas, "An Information-Theoretic Perspective on Intrinsic Motivation in Reinforcement Learning: A Survey," *Entropy*, 25(2), 327, 2023. Published survey.
5. A. El Gamal, "Information Theory and Statistical Learning," proposed textbook chapter, arXiv preprint, 2026. Draft and publication status.

IT 02 • Typical Sequences, AEP and Entropy Rates

Derive why long source blocks concentrate near entropy, connect typical sets to coding, and distinguish stationary dependent sources from independent and nonergodic models.

Online edition: https://blog.shyandram.dev/en/learn/information-theory/it-02-typicality-entropy-rates/

What you'll learn

- Derive the asymptotic equipartition property for an independent finite source.
- Bound the probability and size of a typical set without confusing the two.
- Explain why the most likely sequence need not be representative.
- Derive an entropy rate for a stationary finite-state Markov source.
- Distinguish stationarity, independence and ergodicity through a concrete counterexample.
- Connect conditional and joint typicality to compression and communication proofs.

The big picture

A single draw is uncertain, yet a long sequence often has stable aggregate behavior. A biased binary source might produce an unpredictable next bit while consistently spending about the same description length per symbol over a large block. Typicality makes that observation precise.

> **Explain like I'm five**
>
> You cannot predict every bead pulled from a bag. But after many pulls, you can predict roughly how many colors will appear and how surprising the whole string will be. Most likely strings then fit into a smaller catalog than all imaginable strings.

This chapter follows Cover and Thomas's Chapters 3–4 [1]. Stanford's primary AEP notes [2] provide a compact independent-source proof. We will derive the result, explain its finite-block limitations, and extend the viewpoint to dependence. Later tutorials turn these existence arguments into actual encoders and decoders.

Typicality describes sequences relative to a specified source distribution. It does not mean visually ordinary, semantically correct or typical of every population. A sample can be probable under a misspecified model and still be unsuitable for the application.

Block probabilities and average surprise

Instead of asking how surprising one symbol is, ask how many bits of surprise the whole block has, then divide by its length. For independent symbols, this becomes an average of individual

surprises.

Let $X_1, \ldots, X_n$ be independent and identically distributed according to a mass function p on finite alphabet $\mathcal{X}$. Define the block $X^n = (X_1, \ldots, X_n)$:

$$p(x^n) = \prod_{i=1}^{n} p(x_i), \qquad J_n = -\frac{1}{n} \log_2 p(X^n) = \frac{1}{n} \sum_{i=1}^{n} -\log_2 p(X_i).$$

Here n is block length, x^n is a particular sequence, i is its position, and J_n is the random average surprise in bits per symbol. The equality uses independence. Its expectation is $H(X)$, the entropy of one source symbol.

The finite-alphabet setting ensures the surprise is integrable on positive-probability outcomes. An alphabet may formally include impossible symbols, but a sampled block does not contain them. The law of large numbers therefore gives

$$J_n \xrightarrow{\Pr} H(X).$$

The arrow denotes convergence in probability: for any fixed positive tolerance, the chance that J_n differs from entropy by more than that tolerance goes to zero. This is the **asymptotic equipartition property**, or AEP [1,2].

"Equipartition" is approximate and asymptotic. It does not say that every block has the same probability, nor that all symbols become equally frequent. It says most probability mass concentrates on blocks with approximately the same exponential probability scale.

For a Bernoulli source with probability p of symbol one, a block containing k ones has

$$-\frac{1}{n} \log_2 p(x^n) = -\frac{k}{n} \log_2 p - \left(1 - \frac{k}{n}\right) \log_2 (1 - p).$$

Here k is the observed count and k/n its empirical fraction. When that fraction approaches p, average surprise approaches binary entropy $h_2(p)$. The number of ones is not generally close to half the block length.

The typical set separates probability from counting

Most likely observations fit into a catalog of plausible blocks. Each block in that catalog may be individually unlikely, but together they carry almost all source probability.

For tolerance $\epsilon > 0$, define

$$A_\epsilon^{(n)} = \left\{ x^n : \left| -\frac{1}{n} \log_2 p(x^n) - H(X) \right| \leq \epsilon \right\}.$$

Here $A_\epsilon^{(n)}$ is the weakly typical set, $|\cdot|$ inside the braces denotes absolute value, and ϵ is a tolerance in bits per symbol. Its elements satisfy

$$2^{-n(H(X)+\epsilon)} \leq p(x^n) \leq 2^{-n(H(X)-\epsilon)}.$$

These are bounds on the probability of one typical block. The AEP separately states that $\Pr(X^n \in A_\epsilon^{(n)})$ approaches one.

For sufficiently large n, suppose that typical-set probability is at least $1-\delta$, where $\delta > 0$ is a chosen probability tolerance. Then

$$(1-\delta)2^{n(H(X)-\epsilon)} \le |A_\epsilon^{(n)}| \le 2^{n(H(X)+\epsilon)}.$$

Here $|A_\epsilon^{(n)}|$ denotes the number of typical sequences, not their probability. The upper bound follows because each typical block has probability at least $2^{-n(H+\epsilon)}$ and the total probability cannot exceed one. The lower bound follows because the set carries probability at least $1-\delta$ while each of its blocks has probability at most $2^{-n(H-\epsilon)}$.

Separating ϵ and δ makes the roles clear: one controls description-length tolerance, the other controls excluded probability mass. A textbook may use the same small parameter for both after choosing a sufficiently large block. That shorthand should not hide the distinction in finite calculations.

There are $|\mathcal{X}|^n$ possible sequences. If $H(X) < \log_2 |\mathcal{X}|$, the typical catalog has an exponentially smaller scale. This is the counting fact behind compression: not every conceivable block needs a short description if a small error probability is allowed.

The most probable block is not necessarily typical

A single very likely-looking string can be unrepresentative. Many slightly less likely strings can collectively dominate the probability.

For an independent binary source with $p = 0.2$, the all-zero block is the most likely individual block. Its average surprise is

$$-\frac{1}{n}\log_2(0.8^n) = -\log_2(0.8).$$

Here 0.8 is the zero probability. This value differs from $h_2(0.2)$. For a narrow tolerance, the all-zero string remains outside the typical set even though no individual string is more probable.

Its total probability 0.8^n tends to zero. Meanwhile there are many blocks with approximately one fifth ones; their collective probability approaches one. Probability mass and individual ranking are different objects.

The same distinction appears in generative modeling. A maximum-probability sequence need not resemble a representative sample from the model. Typicality provides a mathematical example of the difference, not a guarantee that a particular sampling algorithm produces useful language or images.

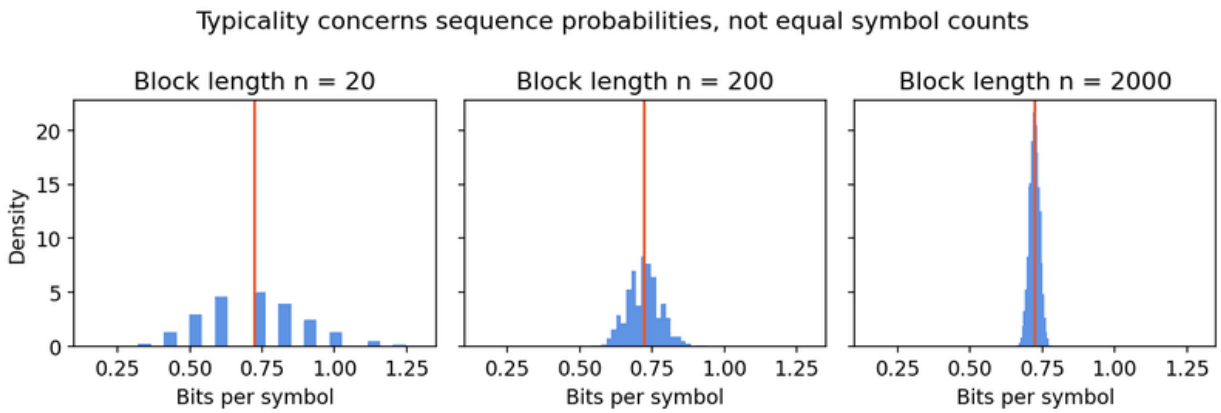


Figure 1 — Synthetic binomial draws, 5,000 blocks per length. The orange line is the exact source entropy; the horizontal axis describes surprise, not the fraction of ones.

Code: exact typical probability and finite counts

The first complete example uses a binomial count distribution to sum typical-set probability exactly up to floating-point arithmetic. A separate small enumeration checks the cardinality bound.

```
import itertools
import numpy as np
from scipy.stats import binom

p, epsilon = 0.2, 0.1
entropy = -p * np.log2(p) - (1 - p) * np.log2(1 - p)
for n in [20, 200, 2000]:
    k = np.arange(n + 1)
    per_symbol = -(k / n) * np.log2(p) - (1 - k / n) * np.log2(1 - p)
    typical = np.abs(per_symbol - entropy) <= epsilon
    probability = binom.pmf(k, n, p)[typical].sum()
    print("Length:", n, "typical probability:", probability)

n, epsilon = 10, 0.15
count, mass = 0, 0.0
for block in itertools.product([0, 1], repeat=n):
    k = sum(block)
    probability = p**k * (1 - p)**(n - k)
    surprise = -np.log2(probability) / n
    if abs(surprise - entropy) <= epsilon:
        count += 1
        mass += probability
assert count <= 2**(n * (entropy + epsilon))
assert count >= mass * 2**(n * (entropy - epsilon))
print("Small-block typical count / mass:", count, mass)
```

Small-block typical mass need not be close to one. “Asymptotic” is a statement about growing

block length with a fixed tolerance, not permission to apply the large-block conclusion at any length.

Worked example: count strings, then add their probabilities

Use the small enumeration above: $n = 10$, $p = 0.2$ and $\epsilon = 0.15$. A block with k ones has average surprise $0.321928 + 0.2k$ bits per symbol, while source entropy is 0.721928. Only $k = 2$ satisfies the tolerance. This determines membership before we count anything.

There are $\binom{10}{2} = 45$ such strings. Each has probability $0.2^2 \times 0.8^8 = 0.0067108864$, so the typical set has probability $45 \times 0.0067108864 = 0.301989888$. In contrast, the single all-zero string has probability $0.8^{10} = 0.1073741824$. It outranks each typical string individually, but carries less probability than the 45 strings together.

These calculations answer three separate questions: **membership** is decided by surprise tolerance, **cardinality** counts accepted strings, and **mass** sums their probabilities. A catalog of 45 strings needs six fixed-length index bits because five bits identify only 32 alternatives. Yet that catalog covers only about 30.2% of this short-block source. The length-10 catalog alone is therefore a poor almost-lossless code, despite its rate of six tenths of a bit per symbol.

The entropy bound does not fail. Its high-probability conclusion requires longer blocks at this fixed tolerance. Cardinality rounding and excluded probability are finite-block costs that the limiting argument does not erase from a short experiment.

For comparison, scoring the same blocks with a fair-bit model would give exactly one bit per symbol to every block. That model cannot distinguish the all-zero string from a two-one string by surprise. The underlying source has not changed; the scoring model has. A typical-set computation must always specify which of those distributions supplies the probabilities.

Empirical frequencies and combinatorial intuition

A block's symbol frequencies define a group of many strings with the same source probability. Counting that group explains how individually unlikely strings can dominate collectively.

For a binary block with exactly k ones, the number of arrangements and the total probability of that count are

$$N_{n,k} = \binom{n}{k}, \qquad \Pr(K_n = k) = \binom{n}{k} p^k (1-p)^{n-k}.$$

Here $N_{n,k}$ counts strings, K_n is the random number of ones, $\binom{n}{k}$ is the binomial coefficient, and p is the source probability of one. Every arrangement with that count has the same probability, but the number of arrangements varies dramatically.

At the extremes, there is just one all-zero string and one all-one string. Near the center, there are many arrangements. For a biased source, the probability assigned to each arrangement and the number of arrangements pull in different directions; their product peaks near the source's expected count.

This viewpoint also clarifies why empirical-frequency typicality and surprise typicality are different. Under a fair binary source, every length-n string has probability 2^{-n} and average surprise exactly one bit. Therefore every possible string is weakly typical for any positive tolerance, including all zeros.

The probability of observing an all-zero string still tends to zero. A frequency-based typical set can exclude it by requiring the fraction of ones to stay near one half. Its probability also tends to one, but it has a stricter membership rule.

Neither definition is "wrong." Each is useful for a different proof. A probability-scale statement may need only weak typicality; a construction that matches empirical conditional distributions may require stronger frequency control. Always check the definition actually used, especially when a proof moves from marginal source coding to joint input-output relationships.

A source-coding argument and its limitation

Number the typical blocks and send their catalog index. With enough symbols in each block, this uses close to entropy bits per symbol while failing only on a small excluded set.

Since the typical set has at most $2^{n(H+\epsilon)}$ members, an index takes at most approximately $n(H+\epsilon)$ bits, plus rounding and framing. A decoder uses the same catalog to reconstruct the selected block.

This is an **almost-lossless fixed-length** construction. It does not encode every possible block correctly. It can reserve one extra symbol for failure, but that flag does not reconstruct an atypical block. To make a truly lossless scheme, provide an escape representation with enough information to recover it.

A simple converse explains why rates well below entropy cannot cover most probability mass. Suppose a code can correctly represent at most 2^{nR} blocks. Intersect those blocks with the typical set. Each typical block has probability at most $2^{-n(H-\epsilon)}$, so their accepted mass is at most

$$2^{nR}2^{-n(H(X)-\epsilon)} = 2^{-n(H(X)-\epsilon-R)}.$$

Here R is the fixed code rate in bits per source symbol. If $R < H(X)-\epsilon$, this typical accepted mass approaches zero. The omitted atypical mass also approaches zero, so the code cannot reconstruct the source with high probability.

This argument assumes the encoder-decoder pair has a limited number of distinct messages. It is a counting proof, not an efficient implementation. Enumerating a huge catalog is generally impractical. The next chapter studies prefix coding, probability coding and adaptive models that turn theoretical rates into computable procedures.

Add an escape branch to make the catalog lossless

A catalog that sometimes fails can be turned into a code that reconstructs every allowed finite block. Send a one-bit branch flag. If the block is typical, send its fixed-length catalog index.

Otherwise, send an uncompressed representation of the block using a fixed number of bits per symbol.

Let $m = \lceil \log_2 |\mathcal{X}| \rceil$ and $\ell_n = \lceil \log_2 |A_\epsilon^{(n)}| \rceil$ for a nonempty typical set. The expected length satisfies

$$\frac{\mathbb{E}[L_n]}{n} = \frac{1}{n} + \frac{P_T \ell_n}{n} + (1 - P_T)m \le \frac{2}{n} + P_T(H(X) + \epsilon) + (1 - P_T)m.$$

Here L_n is total block length in bits, m is the fixed raw symbol width, ℓ_n the typical index width, $P_T = \Pr(X^n \in A_\epsilon^{(n)})$ the typical probability, and $n, \epsilon, H(X)$ retain their earlier meanings. The equality counts the flag and the two branches; the inequality uses typical-set cardinality and upward integer rounding.

The decoder reads the flag first, so it knows which fixed branch length to read. The typical branch returns a shared catalog entry; the escape branch reconstructs the raw symbols. No block is silently dropped. If a small finite block has an empty typical set, use only the raw branch until a usable catalog exists.

For fixed ϵ, AEP gives $P_T \to 1$, making the escape contribution vanish per symbol. The flag and rounding costs also vanish as n grows. Then the expected rate approaches no more than $H(X) + \epsilon$. Choosing smaller tolerance after establishing sufficiently long blocks brings the rate arbitrarily close to entropy.

The construction reveals the difference between fixed-rate almost-lossless and variable-length fully lossless coding. The escape branch can be long on rare blocks, so the latter conclusion concerns **expected length**, not a short bound for every block. It also assumes encoder and decoder share the catalog and source model. This is a proof of attainable rates; explicit enumeration is generally too expensive to be a practical compression algorithm.

Finite-block concentration depends on the source

Long blocks are useful, but how long is enough depends on the source and the tolerance. Rare symbols can produce large surprise fluctuations.

For independent symbols with finite surprise variance,

$$\mathrm{Var}(J_n) = \frac{V}{n}, \qquad V = \mathrm{Var}[-\log_2 p(X)].$$

Here V is the source surprise variance and Var denotes variance. This follows from adding independent variances and dividing by n^2. A general variance bound gives

$$\Pr(|J_n - H(X)| > \epsilon) \le \frac{V}{n\epsilon^2}.$$

Here ϵ is the same surprise tolerance as before. The bound can be loose, but it makes the dependence explicit. Very small probabilities can increase surprise variance, and demanding a narrow tolerance increases the required block length.

An empirical standard deviation across blocks is useful, but it is not a universal finite-block coding guarantee. Dependence changes the variance calculation; rare events may be missed by a small simulation; model mismatch changes the center of concentration.

If a source p is scored by model q, independent block average surprise converges to $H(p, q)$ under integrability conditions, not to $H(p)$. The difference is relative entropy. Typical under the model and typical under the actual source are separate questions.

Entropy rates account for dependence

Neighboring symbols often help predict one another. The information required per symbol over a long sequence can therefore be smaller than the entropy of a single marginal symbol.

For a stochastic process $X_1, X_2, \ldots$, define its entropy rate when the limit exists:

$$\bar{H}(X) = \lim_{n \to \infty} \frac{1}{n} H(X_1, \ldots, X_n).$$

Here $\bar{H}(X)$ is the average new uncertainty per symbol and n is block length. This definition concerns block entropy, not a fitted model's loss on one realized sequence.

The chain rule gives

$$\frac{1}{n} H(X^n) = \frac{1}{n} \sum_{i=1}^{n} H(X_i \mid X^{i-1}).$$

Here $X^{i-1} = (X_1, \ldots, X_{i-1})$ is the preceding history, with empty history at $i = 1$. Dependence makes these conditional terms different from marginal entropy.

A process is **stationary** when shifting all time indices leaves every finite joint distribution unchanged. This is stronger than a constant mean or covariance. Under finite-alphabet stationarity, the conditional entropy given a longer past is nonincreasing and bounded below by zero, so its limit exists. The block-average entropy has the same limit.

The equality of these limits comes from averaging a convergent conditional-entropy sequence. Stationarity permits moving a fixed-length history to a common time index, where adding more past information reduces conditional entropy.

Independent identically distributed sources are a special case: each conditional entropy equals $H(X_1)$. A dependent stationary source may have the same marginal distribution at every time while requiring much less new information per step.

Markov sources provide an explicit calculation

A first-order Markov model needs only the current state to predict the next. Its long-run uncertainty averages the uncertainty in each transition row.

Let a finite Markov source have transition matrix P:

$$P_{ij} = \Pr(X_{t+1} = j \mid X_t = i), \qquad \pi_j = \sum_i \pi_i P_{ij}.$$

Here i and j index source states, t is time, P_{ij} is a transition probability, and π is a stationary state distribution. Each row of P sums to one; π is nonnegative and sums to one.

If the process starts in π, stationarity and the Markov property give

$$\bar{H}(X) = H(X_{t+1} \mid X_t) = -\sum_i \pi_i \sum_j P_{ij} \log_2 P_{ij}.$$

The inner sum is the entropy of transition row i; the outer sum weights rows by stationary occupancy. The finite block entropy is

$$H(X^n) = H(\pi) + (n-1)\bar{H}(X).$$

Here $H(\pi)$ is the entropy of the initial state. The chain rule contributes one initial term and $n-1$ identical transition terms.

For a symmetric binary source that flips with probability a, the stationary distribution is uniform. Its marginal entropy is one bit, while its entropy rate is $h_2(a)$. With $a = 0.1$, most next symbols repeat the current symbol, reducing fresh uncertainty.

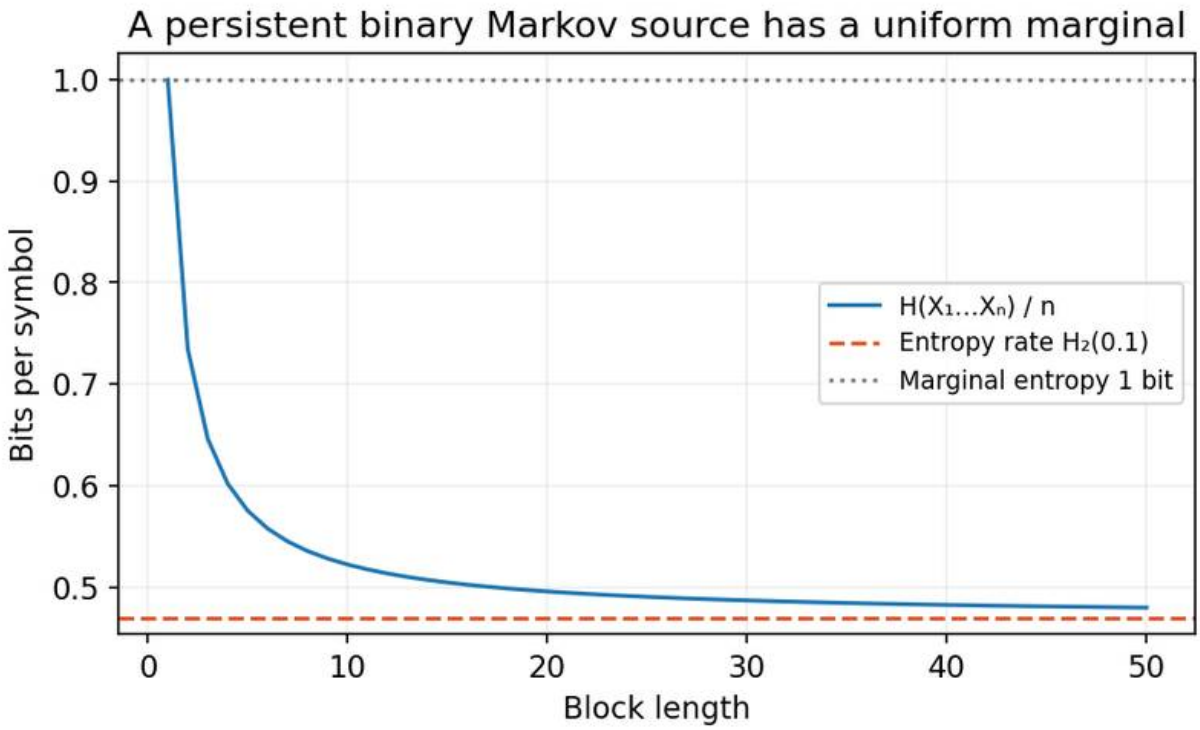


Figure 2 — Analytical stationary binary Markov example. Uniform single-symbol frequencies do not imply an independent source.

Code: predictability without changing the marginal

This standalone experiment generates stationary binary Markov blocks using independent flip events. It compares observed average surprise with the exact block-entropy formula.

```
import numpy as np

rng = np.random.default_rng(32)
a = 0.1
rate = -a * np.log2(a) - (1 - a) * np.log2(1 - a)
for n in [64, 256, 1024]:
    initial = rng.integers(2, size=1000)
    flips = rng.binomial(1, a, size=(1000, n - 1))
    states = np.column_stack([
        initial, initial[:, None] ^ np.bitwise_xor.accumulate(flips, axis=1)])
    surprise = 1 + (-flips * np.log2(a) - (1 - flips) * np.log2(1 - a)).sum(axis=1)
    expectation = (1 + (n - 1) * rate) / n
    print("Length:", n, "one-frequency:", states.mean(),
          "mean surprise rate:", surprise.mean() / n,
          "exact entropy per symbol:", expectation)
    assert abs(states.mean() - 0.5) < 0.08
    assert abs(surprise.mean() / n - expectation) < 0.02
print("Entropy rate:", rate, "marginal entropy:", 1.0)
```

The initial state is sampled uniformly, which makes this source stationary. Starting every block at zero changes finite-block distributions; the asymptotic rate can still agree under suitable transition assumptions, but the exact initial-entropy term changes.

Why a fair marginal can still be cheap to predict

For the symmetric Markov example with flip probability $a = 0.1$, the first bit costs one bit of uncertainty. Each later bit contributes $h_2(0.1) = 0.468996$ bits. At length ten, the block entropy is $1 + 9h_2(0.1) \approx 5.220960$ bits, or 0.522096 bits per symbol. Ten independent fair bits would instead have ten bits of block entropy.

A useful representation is the initial bit followed by nine flip indicators. This mapping is invertible: starting from the first bit, exclusive-or each flip indicator to recover the next state. It makes the dependence visible without throwing away any source information. The indicators are biased independent bits under this particular model, so their distribution is easier to code than ten marginal fair states.

The long-run rate approaches 0.468996 because the one-bit initialization cost is divided by an increasing block length. Predictability comes from the transition relationship, even though every time position is marginally fair. An image histogram similarly cannot reveal all gains available from predicting pixels using their neighbors.

Hidden states and the choice of observation

A transition model may describe an internal state while the receiver sees only a noisy measurement. The observed sequence then has its own prediction problem.

Suppose S_t is a finite hidden Markov state and Y_t an emitted observation. A model may factor a block as

$$p(s^n, y^n) = p(s_1) \prod_{t=1}^{n-1} p(s_{t+1} \mid s_t) \prod_{t=1}^{n} p(y_t \mid s_t).$$

Here s^n is a state sequence, y^n an observation sequence, and the factors describe transitions and emissions. Summing over hidden sequences gives the observed likelihood $p(y^n)$.

The entropy rate of Y is generally not just the stationary average of emission-row entropies. Past observations help infer the hidden state, and residual state uncertainty affects prediction. The relevant terms are $H(Y_t \mid Y^{t-1})$, not only $H(Y_t \mid S_t)$.

This is a useful warning for vision: a model of object dynamics and a model of camera measurements refer to different random processes. More sensor noise may increase observation entropy while decreasing useful information about the scene. Rate calculations must state which process is being described.

Ergodicity matters for a common typical value

Stationarity says distributions do not drift with time. It does not guarantee that one long realization explores all parts of a mixed population.

Under appropriate stationary ergodic finite-source assumptions, the long-block normalized surprise converges to a single entropy-rate value. Ergodicity rules out a persistent hidden choice that divides the process into separately behaving stationary components.

Consider choosing a component C once: with probability one half, produce zeros forever; with probability one half, produce independent fair bits. Each component is stationary, and the mixture is stationary.

Conditional block entropy is $H(X^n \mid C) = n/2$. The identity

$$H(X^n) = H(X^n \mid C) + I(C; X^n)$$

uses component label C, block X^n and mutual information I. Since $I(C; X^n) \leq H(C) = 1$, the block-average entropy tends to one half bit.

Nevertheless, a realized sequence from the deterministic component has normalized surprise approaching zero; one from the fair component usually has normalized surprise approaching one. There is no single common value near one half for most realizations. An entropy-rate limit alone does not justify an ergodic AEP conclusion.

Code: a stationary nonergodic counterexample

This complete example computes the mixture block probability analytically from the number of ones. It avoids exponentiating a very small fair-block probability before taking its logarithm.

```
import numpy as np

rng = np.random.default_rng(33)
for n in [128, 1024]:
    component = rng.integers(2, size=2000)
    counts = rng.binomial(n, 0.5, size=2000) * component
    # A nonzero sequence can only come from the fair component:
    # its probability is 0.5 * 2**(-n), so surprise is n + 1.
    all_zero_surprise = -np.log2(0.5 + 0.5 * np.exp2(-float(n)))
    per_symbol = np.where(counts == 0, all_zero_surprise, n + 1) / n
    print("Length:", n, "deterministic mean:", per_symbol[component == 0].mean(),
          "fair mean:", per_symbol[component == 1].mean(),
          "fraction within 0.1 of 0.5:", np.mean(abs(per_symbol - 0.5) <= 0.1))
    assert np.mean(abs(per_symbol - 0.5) <= 0.1) == 0
```

This example is a model of persistent heterogeneity, not time drift. A dataset containing different source types can require conditional models or explicit mixtures rather than assuming one long recording represents the whole population.

Joint and conditional typicality

Two individually ordinary sequences can form an extraordinary pair. Communication proofs need the relationship between input and output, not only separate histograms.

For independent draws of pairs (X_i, Y_i) from joint mass function p_{XY}, consider the joint and conditional average surprises:

$$-\frac{1}{n}\log_2 p(X^n, Y^n) \to H(X, Y), \qquad -\frac{1}{n}\log_2 p(Y^n \mid X^n) \to H(Y \mid X).$$

Here arrows mean convergence in probability under the specified independent-pair model. X^n and Y^n are paired blocks; dependence within each pair is allowed.

A jointly typical pair is checked against the joint source relationship. Marginal typicality alone does not ensure this. If Y is a noisy copy of X, two unrelated fair-bit sequences can each have typical marginal counts but the wrong mismatch frequency.

The per-block information density is

$$\imath(x^n; y^n) = \log_2 \frac{p(x^n, y^n)}{p(x^n)p(y^n)}.$$

Here $\imath$ measures evidence for the joint pair relative to an independent pair model. Under independent paired sampling, its normalized average approaches $I(X; Y)$ when integrable.

This connects to channel decoding: a receiver looks for codewords whose relationship to the received block is consistent with the channel. It also connects to matching and representation learning, where positive pairs must be defined carefully. A negative pair drawn from the wrong population changes the reference distribution.

Strong typicality for finite alphabets additionally constrains empirical frequencies of each symbol or pair, respecting impossible events. Weak typicality constrains average log-probability. Neither notion should be silently substituted for the other in a proof whose assumptions use one specific definition.

Modern view

The distinction between average surprise and meaningful output is particularly relevant to modern probability models. Long contexts and learned predictors can model dependence beyond a fixed Markov order, but their loss is still scored under a particular observation and tokenization model.

El Gamal's 2026 proposed learning chapter [3] gives a current divergence-based bridge to predictive and generative models. It is a draft supplement, not a published third edition. Our inference from the source-coding viewpoint is that sequence likelihood should always be reported with its prediction target, units and conditioning context.

Huang and Wu's 2024 review [4] surveys learned image compression through the distinction between coding for human reconstruction and coding for machine analysis. Its taxonomy highlights that a useful compressed representation depends on the consumer. Entropy rate supplies a source-description limit, but it does not alone specify a task's acceptable distortion.

Three practical questions remain open in many applications: how quickly asymptotic behavior becomes useful at actual block lengths; whether a fitted source model remains valid across populations; and how dependencies created by sampling, tokenization and preprocessing affect estimated rates.

Finite evaluation should compare independent blocks or acquisition groups and retain atypical cases. Reporting only average surprise can hide a rare source component with a much higher rate. Measuring likelihood under one model cannot establish that all source components share the same typical set.

The next chapter makes the bridge concrete: a coder must define its probability model, update rules, termination convention and treatment of unexpected symbols. A rate theorem guides those choices; a round-trip test checks whether the actual implementation reconstructs the intended data.

Key takeaways

- AEP concentrates normalized block surprise under stated source assumptions.
- Typical-set probability and cardinality are different quantities with different bounds.

- Most-probable individual blocks need not carry most collective probability mass.
- Typical catalogs explain almost-lossless coding limits, not efficient implementations.
- Entropy rate measures new uncertainty after dependence is exploited.
- Stationarity does not guarantee ergodicity or a common realized surprise rate.
- Joint typicality checks relationships that marginal typicality can miss.
- Finite-block behavior, source mismatch and heterogeneity must accompany asymptotic claims.

Exercises

1. For a Bernoulli source with $p = 0.1$, calculate the all-zero block surprise rate and source entropy. Choose a tolerance that excludes all-zero blocks for every length.

   Hint

   Compare $-\log_2(0.9)$ with $h_2(0.1)$.

2. Repeat exact typical-mass computation at tolerances 0.05, 0.1 and 0.2. Explain why smaller tolerance can require much longer blocks.

   Hint

   The average-surprise window narrows.

3. Derive the typical-set lower and upper cardinality bounds using separate ϵ and δ. Identify which individual-probability bound is used in each step.

   Hint

   Bound a sum by count times the largest or smallest allowed block probability.

4. A binary Markov chain has transition matrix $P = \begin{pmatrix} 0.9 & 0.1 \\ 0.3 & 0.7 \end{pmatrix}$. Find its stationary distribution, marginal entropy and entropy rate.

   Hint

   Solve $\pi = \pi P$ and weight transition-row entropies.

5. In the nonergodic mixture example, change the component probabilities to one quarter deterministic and three quarters fair. Find the block entropy-rate limit and realized surprise-rate limits.

   Hint

   The persistent component still prevents one common realized value.

6. Construct two marginally typical fair-bit sequences that are not jointly typical under a 10% flip channel. Explain why separate symbol counts fail to detect the mismatch.

Hint

Use a pair with too many or too few disagreements.

References

1. T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006, Chapters 3–4. Publisher book and contents.
2. T. Weissman, "Asymptotic Equipartition Property," Stanford EE276 Lecture 3 notes, 2024. Primary lecture notes.
3. A. El Gamal, "Information Theory and Statistical Learning," proposed textbook chapter, arXiv preprint, 2026. Draft and publication status.
4. C.-H. Huang and J.-L. Wu, "Unveiling the Future of Human and Machine Coding: A Survey of End-to-End Learned Image Compression," *Entropy*, 26(5), 357, 2024. Published review.

IT 03 • Lossless Coding and Universal Compression

Build decodable Huffman and arithmetic examples, derive the entropy limit, and connect adaptive universal coding with prediction and algorithmic complexity.

Online edition: https://blog.shyandram.dev/en/learn/information-theory/it-03-lossless-universal-coding/

What you'll learn

- Distinguish injective, uniquely decodable and prefix-free descriptions.
- Derive the entropy lower bound using the Kraft–McMillan inequality.
- Construct a Huffman code and test exact reconstruction.
- Explain arithmetic intervals and implement a short exact-arithmetic round trip.
- Relate adaptive probability prediction, universal coding and model overhead.
- Separate computable compression from ideal algorithmic complexity.

The big picture

Compression replaces a message with a shorter representation that a receiver can interpret. The crucial word is **interpret**. Assigning every symbol a short string is insufficient if concatenated strings become ambiguous. A useful coder specifies how a message begins and ends, how probabilities are obtained, and exactly how the receiver reverses the representation.

> **Explain like I'm five**
>
> You and a friend invent nicknames for toys. Common toys get short nicknames. But if two nicknames run together and look like another nickname, your friend cannot unpack the box correctly. A compression rule needs both short names and an unambiguous way to read them.

The reading map is Cover and Thomas's Chapter 5 for lossless codes, Chapter 13 for universal source coding, and Chapter 14 for Kolmogorov complexity [1]. Stanford's variable-length and Huffman lecture notes [2,3] give accessible primary derivations. This tutorial develops original examples rather than reproducing the book's tables or exercises.

The preceding chapter described an almost-lossless typical-set catalog. Here the requirement is exact reconstruction of every allowed message. The price includes escape representations for unexpected symbols and any information needed to identify the model. Later, rate–distortion theory relaxes reconstruction explicitly; it must not be mixed into a claim of losslessness.

What a lossless code must preserve

A compressed representation must identify the original message, including its boundaries. Two different messages cannot become the same transmitted object.

An encoder f and decoder g satisfy

$$g(f(x)) = x$$

for every allowed source message x. Here $f(x)$ is its representation and g is the reconstruction rule. This identity requires f to be injective on the allowed message set.

For symbol-by-symbol codes, a source string is represented by concatenating its symbols' codewords. A **uniquely decodable** code gives a different concatenation for every different source sequence. A **prefix-free** code has no codeword that begins with another complete codeword. Prefix freedom permits decoding immediately upon reaching a leaf of a code tree.

Prefix-free implies uniquely decodable, but the reverse need not hold. An injective mapping of isolated symbols does not even imply unique decodability of concatenations. For example, giving three symbols representations 0, 1 and 01 makes the third symbol indistinguishable from the first followed by the second.

Supplying a separator can repair ambiguity, but the separator occupies transmitted space. Supplying the message length helps determine termination, but length alone does not repair an ambiguous symbol parse. Any side information used by the decoder belongs to the communication protocol and needs explicit accounting.

An empty codeword is possible for a source with only one known symbol and externally supplied length. It is unsuitable for the ordinary multi-symbol tree used below. Our example assigns at least one bit to each symbol and assumes the alphabet, tree and message boundary are shared.

Kraft–McMillan turns a tree into a probability budget

Very short codewords consume a large share of the available binary tree. Too many short words leave no room for the remaining symbols.

For a binary uniquely decodable code with integer lengths ℓ_i,

$$K = \sum_i 2^{-\ell_i} \leq 1.$$

Here i indexes source symbols, ℓ_i is the number of bits in symbol i's codeword, and K is the Kraft sum. Conversely, any finite nonnegative integer length assignment satisfying the inequality can be realized by a prefix code, subject to the usual singleton convention [1,3].

For a prefix tree, each length-ℓ leaf occupies a fraction $2^{-\ell}$ of the full tree at any sufficiently deep level. Prefix freedom makes the occupied descendants disjoint, so their fractions cannot sum above one. The uniquely decodable extension needs a counting argument across concatenated codewords; it is not simply the tree picture applied to a nonprefix code.

This inequality is an existence test for lengths. It does not give a unique tree, specify a bitstream format, or guarantee that a code is efficient under the actual source distribution. Many valid trees have unnecessarily large expected length.

The entropy lower bound

A reversible binary description cannot, on average, be shorter than the source's uncertainty. An apparently better rate means some cost or assumption has changed.

Let p_i be source probabilities and define normalized code-induced probabilities $q_i = 2^{-\ell_i}/K$. Then

$$\mathbb{E}[\ell(X)] = H(p) + D_{\mathrm{KL}}(p\|q) - \log_2 K \geq H(p).$$

Here X is a random source symbol, $\mathbb{E}$ is expectation, $H(p)$ its entropy, D_{KL} relative entropy in bits, and K the Kraft sum. Both extra terms are nonnegative because divergence is nonnegative and $K \leq 1$.

The equality decomposes inefficiency into distribution mismatch and unused tree capacity. For a complete tree, $K = 1$; a mismatch between p and the dyadic probabilities induced by integer lengths still causes overhead. Equality is possible when the probabilities match a suitable complete binary tree.

Choosing $\ell_i = \lceil -\log_2 p_i \rceil$ satisfies the Kraft inequality and gives

$$H(X) \leq \mathbb{E}[\ell(X)] < H(X) + 1.$$

The ceiling rounds upward to an integer. The strict upper bound applies to positive-probability finite symbols with the usual nondegenerate convention. The extra bit is per coded unit; coding length-n source blocks as units reduces the corresponding per-symbol rounding cost to less than $1/n$.

Block coding is not free: the block alphabet can grow exponentially, storing the table can be expensive, and delay increases. Arithmetic coding avoids assigning a separate fixed tree leaf to every possible long message.

Huffman coding: merge the least likely alternatives

Repeatedly combine the two least likely symbols into a temporary symbol. Undoing those merges yields a binary tree whose common symbols have shorter paths.

Huffman coding minimizes

$$L = \sum_i p_i \ell_i$$

over binary prefix codes for the specified finite source distribution. Here L is expected length, p_i is symbol probability, and ℓ_i is the resulting leaf depth. Optimality concerns this model and this class of symbol codes, not every possible block or context-dependent coder [1,3].

An optimal tree can place two least likely symbols at sibling leaves of maximum depth. Swapping a smaller probability into a longer path cannot increase expected length. Merging those siblings reduces the optimization to a smaller alphabet; recursively solving the smaller problem yields the greedy algorithm.

Equal probabilities may produce multiple equally optimal trees. A deterministic tie rule matters when encoder and decoder rebuild a tree independently. Our implementation uses an increasing counter to make heap comparisons unambiguous, without comparing heterogeneous tree objects.

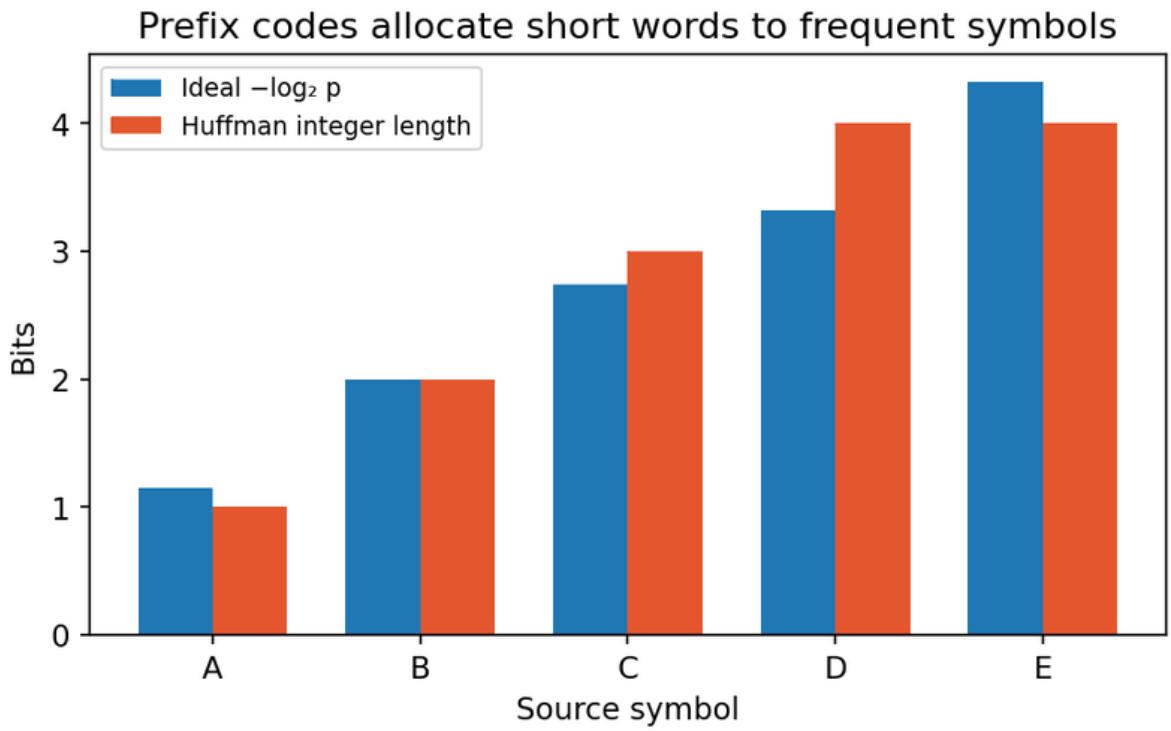


Figure 1 — Probabilities 0.45, 0.25, 0.15, 0.10 and 0.05. Integer symbol-code lengths approximate ideal surprise; they need not equal it for each symbol.

Why the greedy merge preserves optimality

The exchange argument has a concrete cost calculation. If $p_i \geq p_j$ but symbol i is deeper than j, swapping their leaf labels changes expected length by

$$\Delta L = (p_i - p_j)(\ell_j - \ell_i) \leq 0.$$

Here p_i, p_j are the two probabilities, $\ell_i > \ell_j$ their original depths and ΔL the swapped cost minus the original cost. A higher-probability symbol never benefits from being placed deeper than a lower-probability one.

In an optimal binary tree, deepest leaves can be chosen as siblings; otherwise an unnecessary lone branch can be shortened. The exchange argument permits putting the two smallest proba-

bilities in such sibling leaves without increasing cost. Replace them by one temporary symbol of probability $p_i + p_j$. Its depth is one less than either original leaf, so

$$L_{\text{original}} = L_{\text{merged}} + (p_i + p_j).$$

Here the two L values are expected lengths before and after merging. The added term is fixed, independent of how the rest of the reduced tree is arranged. Therefore an optimal reduced tree gives an optimal original tree, and a supposedly better original tree would produce a better reduced tree. Repeating this argument justifies the greedy algorithm rather than merely observing that its output looks reasonable [1,3].

Work through the five-symbol tree

For probabilities 0.45, 0.25, 0.15, 0.10 and 0.05, the merge sequence is easy to follow by hand. Combine 0.05 with 0.10 to form 0.15. Combine that temporary weight with the original 0.15 to form 0.30. Then combine 0.25 with 0.30 to form 0.55, and finally 0.45 with 0.55 to form one.

A symbol's code length is the number of merges between its leaf and the root. The resulting lengths are 1, 2, 3, 4 and 4. One valid assignment is:

Symbol	Probability	Codeword	Length
A	0.45	`0`	1
B	0.25	`10`	2
C	0.15	`110`	3
D	0.10	`1110`	4
E	0.05	`1111`	4

The code is prefix-free: after reading `0`, the receiver immediately knows A; after reading `111`, it still needs one more bit to decide between D and E. Left–right swaps can change codewords while preserving their lengths and expected cost.

The Kraft sum is $1/2 + 1/4 + 1/8 + 1/16 + 1/16 = 1$. Expected length is $0.45 \times 1 + 0.25 \times 2 + 0.15 \times 3 + 0.10 \times 4 + 0.05 \times 4 = 2$ bits per symbol. Source entropy is approximately 1.977235 bits, so this symbol code pays about 0.022765 bits of rounding mismatch per symbol. A fixed three-bit code would be sufficient for five symbols, but would spend an extra bit on every symbol relative to this expected length.

Notice that A's one-bit description is shorter than its ideal surprise $-\log_2(0.45)$. Other symbols compensate. The theorem constrains the probability-weighted average, rather than requiring every individual codeword to be at least its own surprise. The program below may choose different bit labels for tied nodes, but reproduces these lengths and the two-bit expectation.

Code: build a tree, encode, decode and account for length

This complete example uses the same synthetic probabilities as the figure. It tests every pair of codewords for prefix freedom and reconstructs an independently sampled message. The printed

payload excludes a transmitted tree and length header; those exclusions are explicit.

```
import heapq
import itertools
import numpy as np

symbols = list("ABCDE")
p = np.array([0.45, 0.25, 0.15, 0.10, 0.05])
counter = itertools.count()
heap = [(float(w), next(counter), s) for s, w in zip(symbols, p)]
heapq.heapify(heap)
while len(heap) > 1:
    wa, _, a = heapq.heappop(heap)
    wb, _, b = heapq.heappop(heap)
    heapq.heappush(heap, (wa + wb, next(counter), (a, b)))
tree = heap[0][2]
code = {}
def visit(node, prefix=""):
    if isinstance(node, str):
        code[node] = prefix or "0"
    else:
        visit(node[0], prefix + "0")
        visit(node[1], prefix + "1")
visit(tree)
assert all(not b.startswith(a) for i, a in enumerate(code.values())
           for j, b in enumerate(code.values()) if i != j)

rng = np.random.default_rng(34)
message = "".join(rng.choice(symbols, size=2000, p=p))
payload = "".join(code[s] for s in message)
inverse = {v: k for k, v in code.items()}
decoded, prefix = [], ""
for bit in payload:
    prefix += bit
    if prefix in inverse:
        decoded.append(inverse[prefix])
        prefix = ""
assert not prefix and "".join(decoded) == message
entropy = -np.sum(p * np.log2(p))
expected = sum(w * len(code[s]) for s, w in zip(symbols, p))
assert entropy <= expected < entropy + 1
print("Code:", code)
print("Entropy / expected length:", entropy, expected)
print("Payload bits per symbol:", len(payload) / len(message))
```

The entropy bound concerns expected length under the source, not every individual finite message. A particular sample can be shorter or longer than the expected rate. Tree transmission may dominate a tiny message; sharing a stable alphabet model across many messages amortizes it.

If probabilities are estimated from the message itself, the receiver cannot use that estimate unless it is transmitted or reproducible from already decoded symbols. Fitting a histogram and reporting only the resulting payload hides a real model cost.

Arithmetic coding: identify an interval for the whole message

Start with a unit interval. Each observed symbol selects a probability-sized subinterval. Repeated selection leaves a small interval that uniquely describes the whole message.

With cumulative model probability $F_t(s)$ below symbol s and conditional probability $q_t(s)$, update bounds by

$$L_t = L_{t-1} + (U_{t-1} - L_{t-1})F_t(x_t),$$
$$U_t = L_{t-1} + (U_{t-1} - L_{t-1})[F_t(x_t) + q_t(x_t)].$$

Here $[L_t, U_t)$ is the interval after t symbols, x_t is the current symbol, F_t and q_t use the same already decoded history at both endpoints, and the initial interval is $[0, 1)$. The second update uses the old lower bound, not the newly updated one.

The final width is

$$U_n - L_n = \prod_{t=1}^{n} q_t(x_t) = q(x^n).$$

Here n is message length and $q(x^n)$ the model's joint probability. A sufficiently short binary description selecting a point or contained dyadic interval identifies the message. Ideal interval schemes incur only a bounded termination overhead for the whole block, rather than integer rounding at every symbol.

Practical implementations use finite integer registers, renormalization and explicit termination. A real number is not a transmitted object, and infinitely precise arithmetic is not a production codec. The short example below deliberately uses exact rational values to demonstrate the interval invariant without pretending to implement a full file format.

Read the first three interval updates

Use the example's ordered model: A has probability 1/2, B has 1/3, and C has 1/6. Their cumulative starts are 0, 1/2 and 5/6. For the prefix `ABA`, the updates are:

Symbols read	Lower bound	Upper bound	Width
none	0	1	1
`A`	0	1/2	1/2
`AB`	1/4	5/12	1/6
`ABA`	1/4	1/3	1/12

After A, the available interval has width 1/2. B selects the portion from one half to five sixths **within that interval**, giving lower bound $0 + (1/2)(1/2) = 1/4$ and upper bound $0 + (1/2)(5/6) = 5/12$. The next A takes the lower half of this width-1/6 interval. The final width is $1/2 \times 1/3 \times 1/2 = 1/12$, exactly the model probability of `ABA`.

To decode, normalize the tag's position inside the current interval. That normalized position selects A, B or C using the same cumulative partition. The receiver then updates the interval and

repeats. The model and message length make the process reversible; the tiny interval explains the ideal description cost. Exact rational tags teach the mechanism, while finite integer implementations decide which actual bits to emit.

Code: a short exact interval round trip

The encoder outputs a rational tag, while the decoder is supplied the shared model and message length. The numerator and denominator are conceptual side information here; their serialized size is not claimed to be the optimal arithmetic-code length.

```
from fractions import Fraction

model = {"A": Fraction(1, 2), "B": Fraction(1, 3), "C": Fraction(1, 6)}
message = "ABACAB"
cumulative = {}
running = Fraction(0)
for symbol, probability in model.items():
    cumulative[symbol] = running
    running += probability
assert running == 1
low, high = Fraction(0), Fraction(1)
joint = Fraction(1)
for symbol in message:
    width = high - low
    old_low = low
    low = old_low + width * cumulative[symbol]
    high = old_low + width * (cumulative[symbol] + model[symbol])
    joint *= model[symbol]
assert high - low == joint
tag = (low + high) / 2

low_d, high_d = Fraction(0), Fraction(1)
decoded = []
for _ in range(len(message)):
    position = (tag - low_d) / (high_d - low_d)
    symbol = next(s for s in model
                  if cumulative[s] <= position < cumulative[s] + model[s])
    width, old_low = high_d - low_d, low_d
    low_d = old_low + width * cumulative[symbol]
    high_d = old_low + width * (cumulative[symbol] + model[symbol])
    decoded.append(symbol)
assert "".join(decoded) == message
print("Final interval:", low, high, "width:", high - low)
print("Exact decoded message:", "".join(decoded))
```

A zero model probability for a real symbol makes the interval vanish. The codec needs a positive escape route or an alphabet model covering all valid inputs. Merely clipping probabilities in the encoder is insufficient unless the decoder applies the same normalization and integer quantization.

Modeling cost is cross-entropy, not just entropy

An excellent bit-packing algorithm cannot repair a poor probability model. Unexpected symbols still need long descriptions.

For actual block distribution p and coding model q,

$$\mathbb{E}_p[-\log_2 q(X^n)] = H_p(X^n) + D_{\mathrm{KL}}(p_{X^n} \| q_{X^n}).$$

Here $\mathbb{E}_p$ averages under the actual source, $H_p(X^n)$ is actual block entropy, and the divergence compares full block distributions. Finite coder and framing overhead adds further bits.

A model that uses preceding symbols can approach an entropy rate rather than just marginal entropy. An independence model on a persistent Markov sequence throws away predictable repetition. Conversely, a richer context model can overfit, require excessive memory, or assign fragile probabilities to unseen histories.

For evaluation, reserve independent recordings or acquisition groups. Report actual serialized size, including headers, model selection flags and required side streams. If weights are pretrained and permanently shared, state that convention; if weights are adapted per message, count their updates.

The relevant comparison is a complete encoder–decoder protocol under common assumptions. Comparing an ideal log-probability from one method with actual bytes from another can be useful diagnostically, but it is not a direct codec comparison.

Framing and malformed streams

An honest experiment also specifies what happens at the boundary of a message. A prefix tree tells the decoder where a symbol ends; it does not necessarily tell the decoder when the entire stream ends. A length header, end-of-message symbol or container boundary provides that separate fact.

Padding to a byte boundary can add bits that look like valid codewords. The receiver therefore needs the number of meaningful bits or the original symbol count. In the tree example, the payload is a Python string of bits used for teaching, not a byte-efficient container.

The decoder should reject an incomplete final codeword and impossible transitions instead of silently returning a shorter result. A checksum or other integrity mechanism may detect corruption, but detecting a damaged file is a separate requirement from compressing an undamaged one. If such a mechanism is part of the chosen protocol, its transmitted size also counts.

These details matter most for small messages and fair comparisons. Asymptotic statements can make a fixed header negligible per symbol, while an actual short file still pays every header bit.

Universal coding learns while decoding

When the source probability is unknown, both endpoints can update the same predictor from symbols already transmitted. No future symbol needs to be revealed for that update.

A sequential model factors as

$$q(x^n) = \prod_{t=1}^{n} q_t(x_t \mid x^{t-1}).$$

Here x^{t-1} is the preceding decoded history, and q_t is a normalized conditional distribution. The encoder observes the current symbol; the decoder learns it from the interval and then performs the same state update.

For an unknown independent Bernoulli source, the Krichevsky–Trofimov predictor uses

$$q_t(1 \mid x^{t-1}) = \frac{N_1(t-1) + 1/2}{t}, \qquad q_t(0 \mid x^{t-1}) = 1 - q_t(1 \mid x^{t-1}).$$

Here $N_1(t-1)$ counts previous ones and $t-1$ is the number of previous observations. The denominator is previous count plus one. Half-count smoothing gives positive probability to either next symbol, including the first observation [1].

This predictor is the posterior predictive distribution of a Bernoulli mixture with a beta half-count prior. It is universal for the stated Bernoulli family in the appropriate asymptotic coding sense; it is not automatically universal for arbitrary dependent sources.

Define sequence regret against an oracle model:

$$\mathcal{R}_n(x^n) = -\log_2 q(x^n) + \log_2 p(x^n).$$

Here p is the chosen oracle source and q the adaptive model. Expected regret under that fixed source is a block divergence and nonnegative. Realized regret on one sample may be negative, because the mixture can assign that particular sample higher probability than the oracle.

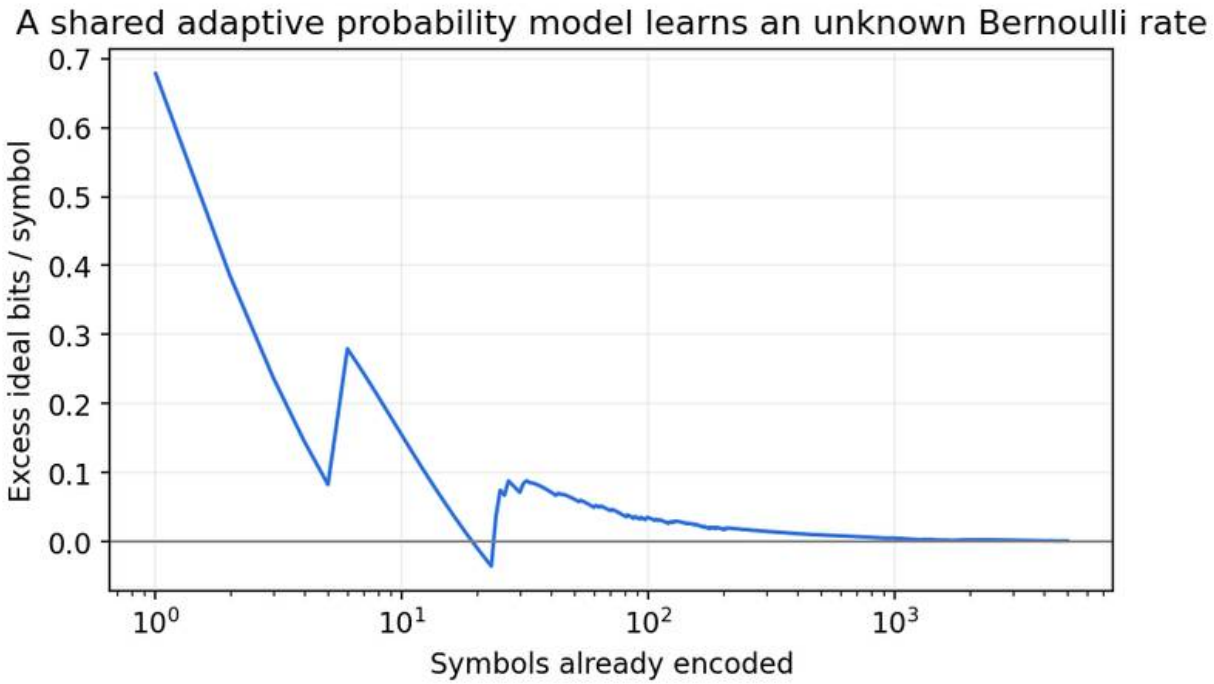


Figure 2 — Seeded synthetic Bernoulli sequence. The curve is realized log-loss difference per symbol, not a serialized byte count or a monotonic guarantee.

Code: adaptation and an abrupt source change

This complete experiment measures ideal sequential lengths. It compares a fixed fair-bit model, a matched oracle and half-count adaptation. A second sequence changes its generating probability halfway through, illustrating the limitation of a stationary count model.

```
import numpy as np

rng = np.random.default_rng(23)
def adaptive_length(bits):
    ones, loss = 0, 0.0
    for t, bit in enumerate(bits):
        q = (ones + 0.5) / (t + 1)
        loss -= np.log2(q if bit else 1 - q)
        ones += int(bit)
    return loss

for n in [100, 1000, 5000]:
    bits = rng.binomial(1, 0.2, size=n)
    oracle = -np.sum(bits * np.log2(0.2) + (1 - bits) * np.log2(0.8))
    print("Length:", n, "oracle:", oracle / n,
          "adaptive:", adaptive_length(bits) / n, "fair model:", 1.0)
bits = np.r_[rng.binomial(1, 0.1, size=2500),
             rng.binomial(1, 0.9, size=2500)]
one_model = adaptive_length(bits)
two_models = adaptive_length(bits[:2500]) + adaptive_length(bits[2500:])
print("Changed source, whole / reset ideal rate:", one_model / 5000, two_models / 5000)
assert two_models < one_model
```

The reset comparison supplies the change point for free. A real codec must identify it or apply a predetermined reset schedule shared with the receiver. Counting a reset's benefit while omitting its signaling cost overstates the achievable saving.

There are two different benchmarks: regret against a fixed oracle probability, and regret against the best parameter fitted retrospectively to the observed sequence. They are not interchangeable. A finite sample may favor a parameter different from its generating probability, even when the source is correctly specified.

See a causal predictor learn one symbol at a time

Before any observation, the half-count model assigns either bit probability $1/2$. If the decoded prefix is `001`, the next-one probability is $(1+1/2)/(3+1) = 3/8$, while next-zero probability is $5/8$. Both endpoints obtain this state from the same prefix, so neither needs a transmitted histogram.

The model pays uncertainty early because it does not know the source bias. As counts accumulate under a fixed Bernoulli source, that initial learning cost is spread across more symbols. An alternating string has fair overall counts but perfect predictability from its previous symbol. The count-only model misses that relationship and keeps predicting near one half. Universal within a Bernoulli family therefore means learning an unknown bias, not learning every possible sequence structure.

Dictionaries, dependence and universal source classes

A dictionary coder learns recurring phrases rather than only symbol frequencies. Long repeated patterns can then be represented by references to earlier phrases.

Lempel–Ziv families construct descriptions from a growing set of previously observed strings [1]. Their asymptotic universality results specify a source class, such as stationary ergodic finite-alphabet processes, and a precise algorithm. They do not assert that any dictionary implementation beats every probabilistic model on every finite file.

Dictionary entries, phrase boundaries and reference indices must be reproducible by the decoder. As the dictionary grows, indices need more bits. A dictionary parser's phrase count is not itself the compressed length; an integer coding rule and termination format are also needed.

Different source structures favor different mechanisms. A single-symbol predictor captures a bias; a context predictor captures short dependencies; phrase dictionaries exploit repeated substrings; a learned model may exploit structure shared across a training population. All still require reversible symbol transmission for a lossless claim.

Universality is relative to a benchmark and limiting regime. Unknown stationarity parameters are one problem; a changing alphabet or arbitrary adversarial source is another. Memory constraints, latency, short messages and reset policy can dominate the asymptotic benefit.

It is useful to ask four questions before quoting a universal-rate result: what class of sources is covered; what reference length is used; whether convergence is in expectation or along realizations; and whether model, dictionary and framing costs are included.

Kolmogorov complexity is an ideal limit of description

Imagine the shortest computer program that prints a string and stops. Structured strings can have short programs; most long strings cannot.

Relative to a fixed universal description machine U, define

$$K_U(x) = \min_{p:U(p)=x} |p|.$$

Here x is a finite string, p a terminating program for machine U, and $|p|$ its program length in bits. Some formulations use prefix-free programs; the machine and convention must be stated [1].

Changing between suitable universal machines alters complexity by an additive machine-dependent constant. That does not make the value practically machine-independent for short strings. More importantly, the exact shortest-program function is not computable in general. One cannot turn the definition into an algorithm that always finds the best compressor.

A lossless compressor gives an upper bound on description complexity once its decoder and framing are included. Its output length is not the exact Kolmogorov complexity. Nor does a compressor failing to shorten a string prove the string is algorithmically random.

A counting argument shows why no injective compressor can shorten every fixed-length input. There are more length-n strings than binary descriptions shorter than n. Any scheme giving some inputs shorter descriptions must leave other inputs unshortened or longer, under an honest boundary convention.

This is compatible with excellent compression on a structured source distribution. Compression exploits nonuniform probability or repeated structure; it does not remove the counting constraint across all possible inputs.

Modern view

Yang, Mandt and Theis's published 2023 monograph [4] separates lossless probability coding from lossy representation design. Its lossless taxonomy includes autoregressive, latent-variable and invertible models; its discussion emphasizes compatibility between model factorization and the entropy coder. We use that taxonomy to ask how probabilities reach an executable decoder.

A modern autoregressive network can replace the simple count predictor while retaining the chain-rule protocol. Its next-symbol probabilities must be available from the already decoded context. Latent-variable models need a specific protocol for any transmitted latent information; a low variational training objective is not automatically the final bitstream length.

The 2026 proposed learning chapter by El Gamal [5] provides a current divergence-centered connection between compression and statistical learning. It is a draft chapter. The classical mismatch identity remains useful: improved predictive probability can reduce ideal coding cost, while computation and synchronization determine whether that reduction is practical.

Open problems include adapting efficiently to distribution shifts, reducing sequential decoding cost, comparing complete formats fairly, and making model updates affordable for small messages. A larger predictor can save payload bits and simultaneously consume more shared memory or computation.

For a concrete evaluation, record three quantities separately: held-out negative log-probability, actual file size including required side information, and encoder/decoder runtime under the same hardware assumptions. Only the second establishes an achieved compression ratio; only a round-trip assertion establishes exact reconstruction.

The next chapter switches roles: a source coder removes predictable redundancy, while a channel coder introduces carefully structured redundancy to survive noise. Both use probability, but they solve different reconstruction problems.

Key takeaways

- Losslessness is an exact encoder–decoder identity with explicit boundaries.
- Kraft–McMillan constrains lengths; entropy lower-bounds their expected value.
- Huffman is optimal for the specified binary symbol-prefix problem.
- Arithmetic intervals spread rounding overhead across a whole message.
- Model mismatch adds divergence before finite implementation overhead.
- Adaptive coding requires synchronized causal updates and positive support.
- Universal performance depends on a stated source class and benchmark.
- Computable compression lengths are upper bounds on ideal algorithmic descriptions.

Exercises

1. Construct two different symbol sequences that collide under codewords 0, 1 and 01. Explain why a supplied number of symbols may still leave ambiguity for other choices.

   Hint

   Start with the third symbol versus the first two.

2. Verify the Kraft sum for lengths 1, 2, 3, 4, 4. Find a prefix tree realizing them and compare expected length under the example distribution.

   Hint

   Sum powers of two and assign leaves without ancestors.

3. Change the Huffman probabilities to a uniform five-symbol source. Explain why ties change the tree but not the optimum expected length.

   Hint

   Keep a deterministic tie rule.

4. Modify the exact arithmetic example to use a context-dependent binary predictor. Make the decoder update from decoded symbols and verify the final interval width.

   Hint

   Use exact half-count fractions and identical histories.

5. Compare half-count adaptation on constant, alternating and independent biased binary strings. Identify which source structures a single count model misses.

Hint

Frequencies do not encode ordering.

6. A learned predictor saves 0.1 ideal bits per symbol but requires 8,000 transmitted adaptation bits. Ignoring other changes, calculate the message length needed to offset that overhead.

Hint

Divide total overhead by the per-symbol saving.

References

1. T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006, Chapters 5, 13–14. Publisher book and contents.
2. T. Weissman, "Variable Length Lossless Compression," Stanford EE276 Lecture 4 notes, 2024. Primary lecture notes.
3. T. Weissman, "Kraft-McMillan Inequality and Huffman Coding," Stanford EE276 Lecture 5 notes, 2024. Primary lecture notes.
4. Y. Yang, S. Mandt and L. Theis, "An Introduction to Neural Data Compression," *Foundations and Trends in Computer Graphics and Vision*, 15(2), 113–200, 2023. Published monograph, author version.
5. A. El Gamal, "Information Theory and Statistical Learning," proposed textbook chapter, arXiv preprint, 2026. Draft and publication status.

IT 04 • Channel Capacity and Reliable Communication

Derive finite-channel capacity, explain coding achievability and Fano's converse, and compute optimal signaling distributions with Blahut–Arimoto.

Online edition: https://blog.shyandram.dev/en/learn/information-theory/it-04-channel-capacity/

What you'll learn

- Define a discrete memoryless channel and distinguish messages from channel symbols.
- Derive capacities of binary symmetric and binary erasure channels.
- Explain random-code achievability through joint typicality and packing.
- Use Fano's inequality to derive a converse bound on reliable rate.
- Compute capacity and a diagnostic optimality gap with Blahut–Arimoto.
- Separate asymptotic limits from finite-block reliability and task-based communication.

The big picture

A noisy channel does not have to reproduce every transmitted symbol correctly for a receiver to recover the message. A code represents each message by a longer structured sequence. The receiver compares the noisy observation with the possible sequences and decides which message was sent.

> **Explain like I'm five**
>
> If a friend hears some letters incorrectly, sending a carefully chosen longer nickname can help. The extra letters make different nicknames harder to confuse. Sending too many different nicknames through the same noisy space eventually makes them overlap.

Source coding removes predictable redundancy; channel coding adds useful redundancy. These are different objectives. Repeating a bit is an elementary channel code, but not necessarily a good use of the channel. The central question is how many distinct messages can be communicated per channel use while their reconstruction error tends to zero.

The main reading is Cover and Thomas's Chapter 7; the book's computational discussion connects capacity optimization with the Blahut–Arimoto method [1]. Stanford's communication and converse notes [2,3] give primary derivations. We develop the theorem's assumptions and a numerical experiment, then explain why practical short messages need additional evaluation.

Define the channel before its capacity

A channel specifies how an input symbol is randomly transformed into an output symbol. The source distribution and the channel transformation are separate objects.

For finite input alphabet $\mathcal{X}$ and output alphabet $\mathcal{Y}$, a discrete memoryless channel is a row-stochastic matrix

$$W_{xy} = \Pr(Y = y \mid X = x), \qquad p(y^n \mid x^n) = \prod_{t=1}^{n} W_{x_t y_t}.$$

Here x, y index symbols, $W_{xy} \geq 0$ and each row sums to one. The blocks x^n, y^n have length n, and t indexes a use. The factorization defines memorylessness and a time-invariant transition law.

Memorylessness concerns outputs **conditioned on the entire input block**. A code's input symbols may be dependent, because all are determined by one message. Their unconditional outputs can consequently be dependent too. Assuming independent inputs when proving a converse would restrict the permitted codes unnecessarily.

The sender chooses an input distribution $q_x = \Pr(X = x)$ for a single-letter calculation. It induces output probability

$$r_y = \sum_x q_x W_{xy}.$$

Here r_y is the output marginal. The channel W stays fixed as q changes. Altering the propagation law, receiver observations, resource constraints or available feedback defines a different communication problem.

Rows that are identical cannot distinguish their corresponding inputs. Distinct rows may be easier or harder to distinguish depending on overlap. Output alphabet size gives a crude upper bound on information, but the transition probabilities determine the actual usable distinction.

Rate and reliability are operational quantities

A communication rate counts message information divided by channel uses. Reliability refers to recovering the whole message, not simply making most channel symbols correct.

Let a message M be uniform over N alternatives. A length-n code maps it to X^n and estimates $\widehat{M}$ from received Y^n. Define

$$R_n = \frac{\log_2 N}{n}, \qquad P_e^{(n)} = \Pr(\widehat{M} \neq M).$$

Here N is codebook size, R_n is bits per channel use, and $P_e^{(n)}$ is average message-error probability under the specified uniform message and channel. These are different from symbol-error rate.

A rate R is achievable if there is a sequence of codes with asymptotic rates at least R and message error tending to zero. Capacity is the supremum of achievable rates:

$$C = \sup\{R : R \text{ is achievable}\} = \max_q I_q(X;Y).$$

Here q ranges over all input probability distributions on the finite alphabet and I_q is mutual information under $q_x W_{xy}$. The second equality is the channel-coding theorem for the stated finite memoryless channel [1,2].

The theorem describes rates strictly below capacity in its standard achievability statement. It does not promise zero error at finite length or provide a fast decoder. Capacity also differs from **zero-error capacity**, which demands that allowed output sets never confuse messages. A channel can have positive ordinary capacity while every finite block still has a nonzero confusion probability.

The average-error formulation has a uniform message assumption. Maximum error across messages is another operational criterion; standard finite memoryless results connect the capacity formulations by expurgating poor codewords with negligible asymptotic rate loss. At finite length their reliability statistics can differ.

A binary symmetric channel

Each bit is independently flipped with the same probability. A fair input makes full use of the two output alternatives.

For crossover probability e, the binary symmetric channel has

$$W = \begin{pmatrix} 1-e & e \\ e & 1-e \end{pmatrix}, \qquad C_{\mathrm{BSC}} = 1 - h_2(e).$$

Here $e \in [0,1]$ is the flip probability, h_2 is binary entropy, and capacity is bits per use. Conditional output entropy is $h_2(e)$ for either input. Since $H(Y) \le 1$, mutual information is at most $1 - h_2(e)$; a fair input attains the bound.

At $e = 0$, one use conveys one bit without noise. At $e = 1/2$, output is independent of input and capacity is zero. At $e = 1$, every bit flips deterministically, so the receiver can invert it and capacity is again one. Noise severity is therefore not simply the raw flip fraction across the entire interval.

The figure restricts crossover probability to zero through one half, where the ordinary interpretation of increasing uncertainty is natural. The complementary branch follows by relabeling outputs.

A binary erasure channel

The receiver either sees the correct bit or a visible “missing” marker. Knowing which observations are missing is useful information.

A binary erasure channel with erasure probability e has outputs zero, one and an erasure symbol:

$$W = \begin{pmatrix} 1-e & 0 & e \\ 0 & 1-e & e \end{pmatrix}, \qquad C_{\mathrm{BEC}} = 1 - e.$$

Here rows correspond to input zero and one, while columns correspond to the two correct symbols and the erasure. The erasure event is independent of the input.

Conditioned on a nonerased output, the input is known. Conditioned on erasure, its uncertainty remains $H(X)$. Therefore $I(X;Y) = (1-e)H(X)$ and a fair input attains $1-e$ bits per use.

The same numerical event probability produces different capacities for flips and visible erasures. An erasure tells the receiver where information is missing; an unmarked flip can actively suggest the wrong alternative. The distinction follows from their different output laws, not a universal ranking for every physical communication system.

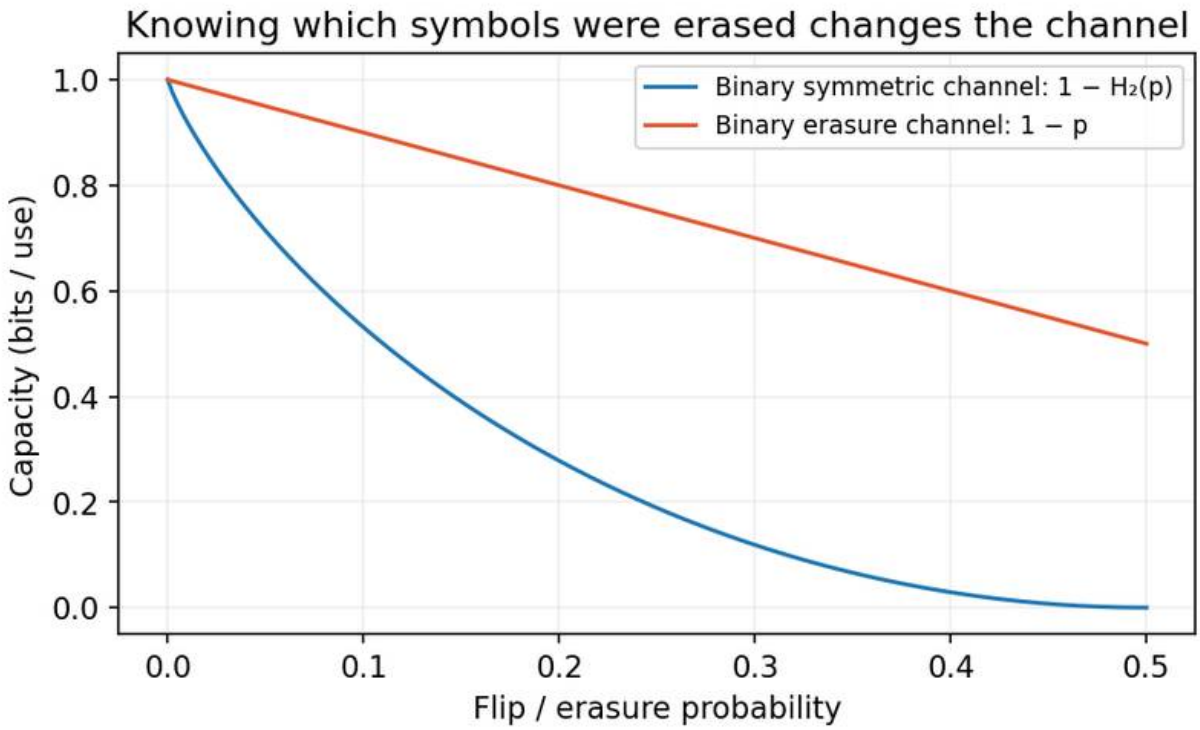


Figure 1 — Analytical capacities under one binary input per channel use. Marked erasures and unmarked flips are different channel models.

Asymmetric channels need asymmetric signaling

When one input is more reliable than another, a uniform input distribution need not be best. Capacity optimizes how often inputs are used.

Consider a Z channel where zero arrives correctly and one becomes zero with probability a:

$$W = \begin{pmatrix} 1 & 0 \\ a & 1-a \end{pmatrix}, \qquad I(q) = h_2(q(1-a)) - q\,h_2(a).$$

Here a is the one-to-zero transition probability, and $q = \Pr(X = 1)$ is the scalar input parameter. Output one has probability $q(1 - a)$; conditional output entropy is $qh_2(a)$.

The first term rewards varied outputs, while the second accounts for noise paid when using input one. Their tradeoff can place the maximum away from $q = 1/2$. Transmitting only the reliable zero is not useful either: a constant input conveys no message information.

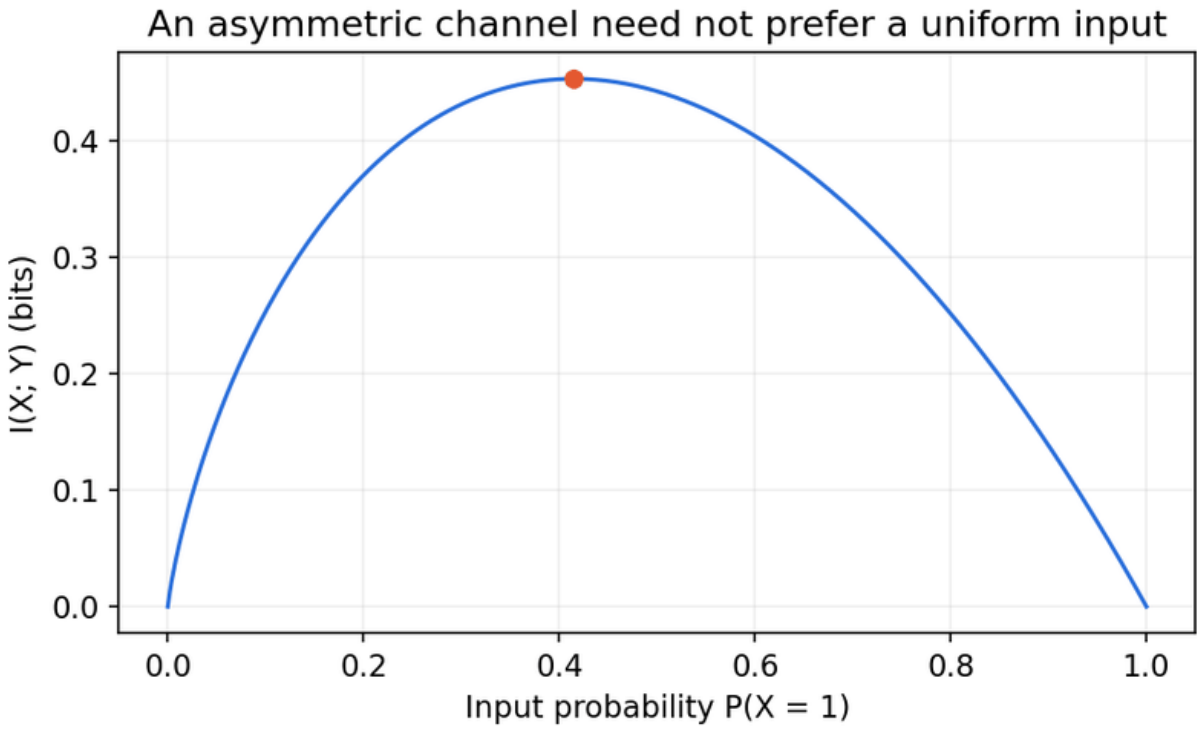


Figure 2 — Analytical input-distribution optimization. Reliability of one symbol does not justify transmitting it exclusively.

Mutual information is concave as a function of the input distribution because output entropy is concave and conditional output entropy is linear. Concavity makes numerical optimization tractable. Flat maxima can still exist when different inputs are redundant, so an optimal input distribution need not be unique.

Why rates below capacity are achievable

Choose many long codewords that look like valid source blocks. A received block usually matches its true codeword’s channel relationship, while it rarely matches an unrelated codeword.

Fix an input distribution q and generate approximately 2^{nR} codewords independently from q^n. A jointly typical decoder looks for one codeword consistent with the received output under the joint law $q_x W_{xy}$ [1].

There are two main errors: the true input-output pair fails its typicality test, or another codeword passes. The first probability vanishes by the joint AEP. An independent competing codeword has a joint-typicality probability at an exponential scale near $2^{-nI_q(X;Y)}$.

A union bound over the competing codewords gives the schematic scale

$$P_{\text{competing}} \lesssim 2^{nR} 2^{-n(I_q(X;Y)-\eta)}.$$

Here $P_{\text{competing}}$ is the probability of a competing typical codeword, $\eta > 0$ absorbs the typicality tolerances, and $\lesssim$ suppresses finite prefactors and detailed bounds. This is proof intuition, not an exact finite-length performance formula.

When $R < I_q(X;Y) - \eta$, the competing-error bound vanishes. Choosing a maximizing input and sufficiently small tolerance gives rates below capacity. A small average error over random codebooks implies at least one deterministic codebook has small error.

The argument does not require transmitting the random seed for each message. The codebook is fixed and known to both endpoints before communication. If a practical system chooses or sends new codebooks, that setup protocol must be counted separately.

Random coding proves existence. Searching every codeword in an exponentially large codebook is generally infeasible, and typicality is not a direct recipe for a deployed decoder. A capacity theorem identifies a target; practical structured codes solve the additional complexity problem.

Fano's inequality limits what a receiver can infer

If the receiver frequently guesses wrong, uncertainty remains about the message. Conversely, a small error probability means little conditional message uncertainty remains.

With $N \geq 2$ possible messages and any estimate based on Y^n,

$$H(M \mid Y^n) \leq h_2(P_e) + P_e \log_2(N-1) \leq 1 + P_e \log_2 N.$$

Here M is the true message, Y^n the received block, $P_e = \Pr(\widehat{M} \neq M)$, and h_2 binary entropy. This is Fano's inequality [1,3].

To see the logic, introduce an error indicator. Describing whether an error happened costs at most its binary entropy. If no error happened, the estimate identifies the message. If an error happened, at most $N-1$ alternatives remain. Averaging yields the first bound.

For a uniform message, $H(M) = \log_2 N$. Data processing and channel memorylessness bound

$$I(M;Y^n) \leq I(X^n;Y^n) \leq \sum_{t=1}^{n} I(X_t;Y_t) \leq nC.$$

Here X^n is the transmitted codeword. The middle inequality allows dependent code symbols: output entropy is at most the sum of marginal output entropies, while conditional output entropy separates under the memoryless channel.

Combining the two bounds gives

$$(1 - P_e)\log_2 N \le nC + 1, \qquad P_e \ge 1 - \frac{C}{R_n} - \frac{1}{nR_n}.$$

Here $R_n = \log_2 N/n$ and the lower bound is meaningful only when its right-hand side is positive. A fixed rate strictly above capacity cannot have vanishing error.

This elementary converse establishes a positive error floor above capacity, not by itself convergence of error to one. Strong-converse statements require an additional theorem and assumptions. It is good practice to state exactly what the displayed inequality proves.

Worked rate, error and source-budget calculations

For crossover probability $e = 0.1$, the BSC capacity is $1 - h_2(0.1) = 0.531004$ bits per use. The BEC at the same erasure probability has capacity 0.9. This is a difference between observation models: the BEC reveals which symbols were lost.

A length-100 code at rate 0.8 contains 2^{80} possible messages. Substitute the BSC capacity into the elementary converse:

$$P_e \ge 1 - \frac{0.5310044064}{0.8} - \frac{1}{100 \times 0.8} \approx 0.323744.$$

Here P_e is average whole-message error, 100 is the number of channel uses and 0.8 the message bits per use. The rounded capacity in the substitution retains more digits than the reported answer. This code cannot achieve a 1% message-error target under the stated channel, whatever its decoder. The bound does not determine the exact error of any specific code.

At rate 0.5 the same displayed lower bound is negative, so it provides only the trivial bound zero. That is consistent with asymptotic achievability below capacity, but it does not demonstrate that length 100 already provides the desired reliability. The two questions are answered by different evidence: a theorem for growing block length and a finite-code calculation or experiment.

Now take an independent binary source with probability 0.2 of one. Its entropy is 0.721928 bits per source symbol. Transporting it almost losslessly over this BSC requires more than $0.721928/0.531004 \approx 1.359552$ channel uses per source symbol in the asymptotic separation setting. One use is insufficient; two uses leave a rate margin. A raw-source representation would require one bit per source symbol before channel coding, illustrating why exploiting source bias and protecting against noise are separate design steps.

The bandwidth ratio need not be an integer. Over a long block, a rational or approximate ratio is implemented by allocating the corresponding total number of channel uses. Latency and finite-block overhead remain additional constraints.

Computing capacity with Blahut–Arimoto

Start with an input mixture, examine how distinguishable each channel row is from its output mixture, then move probability toward informative rows. Repeat until an upper and lower bound nearly meet.

For current positive input probabilities q_x and induced r_y, define row divergences in natural units:

$$d_x = \sum_{y:W_{xy}>0} W_{xy} \ln \frac{W_{xy}}{r_y}, \qquad q_x^{\text{new}} = \frac{q_x e^{d_x}}{\sum_j q_j e^{d_j}}.$$

Here d_x is the divergence between channel row x and the current output distribution, ln is the natural logarithm, and j indexes inputs. The denominator uses the same current distribution; the update does not replace it with a uniform prior. Values in nats convert to bits by division by $\ln 2$.

For the current distribution, the lower and upper bounds are

$$L(q) = \frac{\sum_x q_x d_x}{\ln 2}, \qquad U(q) = \frac{\max_x d_x}{\ln 2}, \qquad L(q) \le C \le U(q).$$

Here $L(q)$ is current mutual information and $U(q)$ is a dual-style bound obtained by using the current output mixture as a candidate reference distribution. The gap gives a useful numerical stopping criterion.

This unconstrained algorithm applies to a fixed finite channel. An input-cost constraint requires a modified optimization; one cannot compute unconstrained capacity and claim compliance with a power or symbol-use budget afterward.

Why the numerical upper bound is valid

A stopping gap is useful only if its endpoints actually bracket the optimum. Fix the current output mixture r, then consider **any** candidate input distribution u, whose output mixture is $r_u = uW$. Expanding the logarithmic ratios gives

$$\sum_x u_x D_{\text{nats}}(W_x \| r) = I_{u,\text{nats}}(X;Y) + D_{\text{nats}}(r_u \| r).$$

Here W_x is channel row x, u_x its candidate usage probability, r the fixed reference output distribution, r_u the candidate output marginal and $I_{u,\text{nats}}$ mutual information under u. All quantities in this identity use natural logarithms.

The final divergence is nonnegative, so the candidate information is no larger than the weighted sum on the left. That sum is no larger than its largest row divergence. Because this holds for every candidate u, the current maximum row divergence upper-bounds capacity. The current input's own information is achievable in the single-letter optimization and supplies the lower bound.

At an optimum, inputs assigned positive probability have equal informative row divergences, while unused inputs cannot offer a larger marginal gain. The updates redistribute probability toward rows with greater current divergence until these conditions nearly balance. The gap therefore measures how much improvement could remain for this matrix, rather than merely how small the latest parameter update was.

Code: validate analytic capacities and optimize the Z channel

The complete implementation removes output columns that are impossible for all inputs, masks zero row entries and uses a stable logarithmic normalization. It returns the current input distribution associated with its lower-bound rate.

```
import numpy as np
from scipy.special import logsumexp
from scipy.optimize import minimize_scalar

def binary_entropy(p):
    if p == 0 or p == 1:
        return 0.0
    return -p * np.log2(p) - (1 - p) * np.log2(1 - p)

def capacity(W, tolerance=1e-10, maximum=20000):
    W = np.asarray(W, dtype=float)
    assert W.ndim == 2 and np.all(W >= 0)
    assert np.allclose(W.sum(axis=1), 1)
    W = W[:, W.sum(axis=0) > 0]
    q = np.full(W.shape[0], 1 / W.shape[0])
    positive = W > 0
    for iteration in range(maximum):
        r = q @ W
        ratio = np.zeros_like(W)
        np.log(np.divide(W, r, out=np.ones_like(W), where=positive),
               out=ratio, where=positive)
        d = (W * ratio).sum(axis=1)
        lower, upper = q @ d / np.log(2), d.max() / np.log(2)
        if upper - lower < tolerance:
            return lower, q, upper - lower, iteration + 1
        log_q = np.log(q) + d
        q = np.exp(log_q - logsumexp(log_q))
    raise RuntimeError("Capacity iteration did not meet the requested gap")

e = 0.1
bsc = np.array([[1-e, e], [e, 1-e]])
bec = np.array([[1-e, 0, e], [0, 1-e, e]])
for W, analytic in [(bsc, 1-binary_entropy(e)), (bec, 1-e)]:
    value, q, gap, steps = capacity(W)
    print("Capacity / analytic / input / gap:", value, analytic, q, gap)
    assert abs(value - analytic) < 1e-8
a = 0.35
z = np.array([[1, 0], [a, 1-a]])
value, q, gap, steps = capacity(z)
objective = lambda p: binary_entropy(p*(1-a)) - p*binary_entropy(a)
reference = minimize_scalar(lambda p: -objective(p), bounds=(0, 1), method="bounded")
assert abs(value + reference.fun) < 1e-8
print("Z capacity / P(input one) / gap / iterations:", value, q[1], gap, steps)
```

Agreement with analytic binary cases checks units and orientation. Agreement with independent scalar optimization checks the asymmetric case. These are meaningful tests because they would

catch using column-stochastic probabilities, omitting the current q factor or mixing natural and base-two logarithms.

Nearly redundant inputs and extreme probabilities can converge slowly or cause floating-point underflow. A small reported gap is a numerical certificate for this finite matrix within computational precision, not evidence that the matrix accurately describes a real channel.

A repetition experiment shows the finite-block tradeoff

Repeating one bit helps majority decoding, but consumes more uses per message bit. Higher reliability does not automatically mean a capacity-efficient code.

Repeating a bit k times through an independent BSC and majority decoding, with odd k, gives

$$P_e(k) = \sum_{j=(k+1)/2}^{k} \binom{k}{j} e^j (1-e)^{k-j}, \qquad R = \frac{1}{k}.$$

Here j is the number of flipped copies, e the crossover probability, $P_e(k)$ the one-bit message error, and R the code rate. This is an exact finite-code calculation, not a capacity formula.

```
import numpy as np
from scipy.stats import binom

rng = np.random.default_rng(35)
e = 0.1
for copies in [1, 3, 5, 9]:
    noise = rng.binomial(1, e, size=(50000, copies))
    observed_error = np.mean(noise.sum(axis=1) > copies // 2)
    exact_error = binom.sf(copies // 2, copies, e)
    print("Copies / rate / simulated error / exact error:",
          copies, 1 / copies, observed_error, exact_error)
    assert abs(observed_error - exact_error) < 0.005
```

We may assume a zero message bit when simulating this symmetric code, because either transmitted bit has the same error probability. More copies reduce error for this one-bit problem while driving rate toward zero. Capacity instead permits codebooks carrying a growing number of bits at a positive asymptotic rate.

A finite evaluation should specify block length, rate, message-error criterion, channel parameters and decoding rule. Reporting only a capacity number does not characterize a short code's latency, complexity or reliability.

Resource constraints and different channel assumptions

A capacity calculation answers one precisely specified question. Changing a transmitter's budget, adding memory or providing a different observation changes that question.

If input x has nonnegative cost $c(x)$ and the average budget per use is B, the constrained single-letter problem is

$$C(B) = \max_{q:\sum_x q_x c(x) \le B} I_q(X;Y).$$

Here $c(x)$ is the input-use cost, B the allowed mean, and q a probability distribution satisfying it. The channel remains the same finite memoryless matrix. This formula concerns an average cost convention; a strict peak constraint on each symbol defines a different feasible set.

For an asymmetric binary channel, a constraint on the frequency of input one can exclude the unconstrained maximizing distribution. The result should be a capacity–cost function rather than one unconstrained number with a power label attached.

Likewise, throughput in bits per second requires a specified number of channel uses per second. Bits per use are not automatically bits per sample, pixel or token. When comparing modulation or representation choices, state how each choice changes the use count and available resources.

Feedback lets an encoder observe earlier channel outputs before choosing the next input. For the ordinary memoryless point-to-point model, noiseless causal feedback does not increase the asymptotic capacity [1,3]. It can still change finite-code design, stopping rules and achievable error behavior. This result should not be transferred without checking to channels with memory or multiple users.

A channel with a persistent hidden noise state may produce bursts rather than independent flips. A single transition matrix estimated by pooling all uses hides that dependence. The capacity proof above uses the conditional product law and cannot simply treat those bursts as independent draws with an average flip probability.

These distinctions also arise when a receiver supplies side information. If the side information reveals an erasure position, a noise state or part of the original message, it changes the receiver's observation. Compare the entire observation model, rather than crediting a decoder for information that a baseline was denied.

Estimating a channel is a separate experiment

Numerical optimization can be exact for a wrong matrix. Before trusting the rate, determine whether the transition probabilities describe the operating conditions.

With labeled input–output observations, a simple row estimate is

$$\widehat{W}_{xy} = \frac{N_{xy}}{\sum_z N_{xz}}.$$

Here N_{xy} counts observed input x paired with output y, z ranges over outputs, and $\widehat{W}$ is an empirical channel matrix. Every input row needs observations. A zero empirical count does not prove an event impossible.

Finite samples create uncertainty in both the estimated capacity and the maximizing distribution. Repeating experiments across independent acquisitions can expose variability; conditioning on

known operating regimes can reveal different channel laws. The optimizer's gap measures computational convergence for one matrix, while experimental uncertainty concerns which matrix should be optimized.

Training and evaluation should also use separate observations when a decoder or input strategy is fitted from data. Selecting a favorable input distribution on the same tiny sample used to estimate its performance can produce an optimistic rate estimate.

A useful report therefore gives the sampling protocol, counts per input, operating conditions, empirical row probabilities, and held-out reconstruction results. Rare output events deserve explicit treatment because assigning them zero probability can distort both a decoder and an information estimate.

How source information meets channel information

A source coder packs useful uncertainty into bits; a channel coder carries those bits reliably. Their rates must be compared using the same source-to-channel use budget.

If a finite memoryless source has entropy $H(S)$ bits per source symbol and b channel uses are available per source symbol, the familiar asymptotic separation condition is

$$H(S) < bC.$$

Here S is a source symbol, b is the use ratio and C the point-to-point capacity. Under the standard long-block assumptions, source and channel coding can separately approach their limits with a margin [1].

The inequality does not specify a practical block size or require independent design to be optimal at finite latency. It also does not assert general source–channel separation for every network. Keeping its assumptions visible prevents the attractive formula from being applied to unrelated constraints.

For lossy reconstruction, the needed source rate becomes a rate–distortion function rather than full entropy. For continuous amplitudes, physical constraints determine channel capacity. These two extensions motivate the next chapters.

Modern view

Nguyen and colleagues' 2025 published survey [4] organizes semantic communication into receiver-intention modeling, generative models, and deep joint source–channel coding. Its taxonomy distinguishes systems by what they aim to preserve and how transmitter and receiver coordinate, rather than treating every learned system as the same codec.

Those objectives can change the required reconstruction. A task receiver might need a class label rather than every original pixel. That can reduce required source information, but it does not increase the Shannon capacity of an unchanged channel under unchanged constraints.

Generative reconstruction similarly needs an explicit criterion. A plausible output can have a different relationship to the original than an exact decoded message. Evaluating task accuracy or perceptual distortion is legitimate when stated, but those quantities cannot replace message-error probability in a bit-reliability claim.

Our inference from the classical model is to keep three layers separate: what the source receiver needs, what the channel physically conveys, and how the code uses finite resources. Joint optimization can improve their practical match without invalidating any of the corresponding limits.

Open questions include robust performance when the channel law shifts, reliable operation at short block lengths, shared-model synchronization, and evaluation across different tasks and receiver knowledge. Survey results for one task or channel do not establish a universal semantic rate.

The next chapter changes alphabets from finite symbols to continuous amplitudes. It introduces differential entropy and power constraints before deriving Gaussian capacity; substituting a continuous density into a discrete entropy formula would miss essential unit dependence.

Key takeaways

- A memoryless channel factors outputs given inputs; code symbols need not be independent.
- Capacity is a reliable-message rate, equal to optimized mutual information under stated assumptions.
- Flips and marked erasures have different channel capacities.
- Asymmetric channels may favor nonuniform input probabilities.
- Random coding establishes existence through typicality and packing.
- Fano and data processing bound rates with vanishing message error.
- Blahut–Arimoto optimizes a finite channel and supplies a numerical gap.
- Finite reliability and semantic objectives require their own explicit evaluation.

Exercises

1. Explain why a BSC with crossover probability 0.9 has the same capacity as one with probability 0.1. Describe the receiver's output relabeling.

Hint

Invert every output.

2. Derive the BEC mutual information for an arbitrary binary input probability rather than assuming a fair input.

Hint

Erasures leave the original input entropy.

3. Change the Z-channel parameter to 0.1 and 0.7. Compare optimized input probabilities with one half and explain the competing entropy terms.

Hint

Optimize output entropy minus weighted row entropy.

4. For a length-100 code of rate 0.8 through a BSC with crossover probability 0.1, calculate the elementary Fano lower bound on message error. State what it does and does not prove.

Hint

Use $1 - C/R - 1/(nR)$ and keep a zero lower bound if negative.

5. Duplicate a row in a channel matrix and rerun the capacity routine. Explain why the value is unchanged while an optimal input mixture may cease to be unique.

Hint

Identical rows produce identical conditional outputs.

6. Compare repetition lengths 3 and 9 at equal total channel-use budgets. Explain why the smaller one-bit error cannot alone decide which system carries more reliably decoded information.

Hint

Count message bits as well as their errors.

References

1. T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006, Chapter 7 and the computational capacity/rate–distortion discussion. Publisher book and contents.
2. T. Weissman, "Communication and Channel Capacity," Stanford EE276 Lecture 7 notes, 2024. Primary lecture notes.
3. T. Weissman, "Channel Coding Theorem: Converse Part," Stanford EE276 Lecture 10 notes, 2024. Primary lecture notes.
4. L. X. Nguyen, A. D. Raha, P. S. Aung, D. Niyato, Z. Han and C. S. Hong, "A Contemporary Survey on Semantic Communications: Theory of Mind, Generative AI, and Deep Joint Source-Channel Coding," *IEEE Communications Surveys & Tutorials*, 2025, DOI: 10.1109/COMST.2025.3616973. Published-paper metadata and author version.

IT 05 • Differential Entropy and Gaussian Channels

Handle continuous information with explicit units, derive Gaussian capacity under a power budget, and optimize parallel channels by water-filling.

Online edition: https://blog.shyandram.dev/en/learn/information-theory/it-05-continuous-gaussian-channels/

What you'll learn

- Define differential entropy without treating a density as a probability.
- Track units and coordinate changes while preserving mutual information.
- Derive the Gaussian maximum-entropy property from relative entropy.
- Obtain real Gaussian-channel capacity under an average-power budget.
- Allocate power across parallel channels through water-filling.
- Distinguish analytical information limits from estimation and implementation costs.

The big picture

An amplitude can take a continuum of values, but a receiver with noise cannot distinguish infinitely many messages for free. More precise measurements require more resources, and a transmitter cannot generally use arbitrarily large amplitudes. Continuous information theory keeps those constraints visible.

> **Explain like I'm five**
>
> You can point anywhere along a ruler, but your friend's blurry eyes cannot tell apart every tiny difference. Pointing farther apart helps, yet the ruler has a size limit. The useful question is how many reliably different signals fit within the available space.

This chapter follows Cover and Thomas's Chapters 8–9 [1], supplemented by Stanford's continuous-variable and Gaussian-channel notes [2,3]. The progression matters: we first define density-based information, then prove a distributional bound, then use it to optimize communication under a power constraint.

Continuous formulas also appear in vision features, latent representations and sensor noise models. Their numerical values can change under a change of units even when the scene has not changed. Mutual information is the more stable object for comparing relationships.

A density is not a point probability

A continuous density describes probability per unit of measurement. A density value above one is perfectly possible and does not violate probability rules.

For a real random variable X with density f_X,

$$\Pr(X \in A) = \int_A f_X(x)\,dx, \qquad \int_{\mathbb{R}} f_X(x)\,dx = 1.$$

Here A is a measurable region, x is a coordinate in the chosen units, and $f_X(x)$ is density rather than the probability of the exact point. Under an ordinary continuous density, each exact point has probability zero.

A uniform variable on an interval of width one tenth has density ten on that interval. Its probability over the whole interval is still one. Taking the negative logarithm of a density can therefore yield a negative value.

The differential entropy is

$$h(X) = -\int f_X(x) \log_2 f_X(x)\,dx.$$

Here h is differential entropy in base-two units and the integral is taken where the density is defined. The expression assumes the integral exists in the relevant sense. It can be negative or infinite, and some distributions have no well-defined finite entropy.

Unlike discrete entropy, differential entropy alone is not a minimum count of bits needed for exact representation of a continuously distributed value. Exact infinite-precision reconstruction is a different, generally unbounded requirement.

The notation deliberately uses lowercase h for differential entropy and uppercase H for discrete entropy. They share algebraic relationships under appropriate conditions, but their interpretation and sign constraints differ.

Quantization explains the relationship to actual symbols

A real instrument records bins, not infinitely precise points. As bins shrink, describing the bin index costs more bits.

Quantize a scalar density into bins of width $\Delta > 0$. For a sufficiently regular density and fine quantization,

$$H(X_\Delta) \approx h(X) + \log_2 \frac{1}{\Delta}.$$

Here X_Δ is the discrete bin index, Δ is measured in the same coordinate units as x, and the approximation is a fine-bin asymptotic relationship subject to regularity and tail conditions.

Each bin probability is approximately density times bin width. Its negative log-probability is therefore approximately negative log-density minus log-width. Averaging gives the formula.

The extra term is not a small correction. It diverges as resolution increases. A finite differential entropy does not make arbitrarily precise values cheap to encode.

At coarse resolution, sharp discontinuities or heavy tails, the approximation may be poor. Real quantizers may have saturation, nonuniform bins or a finite range. These details change the discrete distribution and must be included in an actual coding experiment.

For a sensor, both resolution and physical noise matter. Increasing stored numerical precision beyond measurement precision need not create new information about the scene. It can merely create more digits representing uncertain amplitudes.

Code: coordinate units and quantized entropy

This standalone experiment uses a unit uniform variable, computes empirical bin-index entropy at different resolutions, and compares an analytic differential-entropy change under scaling.

```
import numpy as np

rng = np.random.default_rng(36)
x = rng.uniform(0, 1, size=100000)
def discrete_entropy(values):
    _, counts = np.unique(values, return_counts=True)
    p = counts / counts.sum()
    return -np.sum(p * np.log2(p))
for bins in [10, 100, 1000]:
    indices = np.floor(x * bins).astype(int)
    estimated = discrete_entropy(indices)
    theoretical = np.log2(bins)
    print("Bins / estimated index entropy / exact:", bins, estimated, theoretical)
    assert abs(estimated - theoretical) < 0.03
scale = 0.1
h_x, h_scaled = 0.0, np.log2(scale)
print("Differential entropy before / after scaling:", h_x, h_scaled)
assert h_scaled < 0
```

The negative scaled entropy is not negative uncertainty in a discrete message. It reflects density per coordinate unit. The finite bin estimate remains nonnegative and has sampling bias that grows when there are many rare bins.

Coordinate changes alter differential entropy

Measuring the same length in millimeters instead of meters stretches the numerical distribution. Differential entropy changes with that stretching.

For a nonzero scalar a and translation b,

$$h(aX + b) = h(X) + \log_2 |a|.$$

Here a scales coordinates and b translates them. A translation does not alter density volume; scaling multiplies it by $|a|$. For an invertible matrix A applied to a d-dimensional vector,

$$h(AX + b) = h(X) + \log_2 |\det A|.$$

Here A is a nonsingular d by d matrix, b a translation vector, and $\det A$ its determinant. Both sides require well-defined entropies.

A smooth one-to-one nonlinear coordinate map T has the corresponding Jacobian correction:

$$h(T(X)) = h(X) + \mathbb{E} \log_2 |\det J_T(X)|.$$

Here J_T is the derivative matrix of T, evaluated at random X, and the expectation averages its log-volume change. This identity requires appropriate differentiability, invertibility and integrability.

It explains why latent density values depend on representation coordinates. An invertible transformation can change differential entropy without discarding information. Comparing two feature spaces by raw differential entropy alone can confuse coordinate expansion with information quality.

Derive the scale correction from the density

The correction can be understood directly rather than memorized. Let $V = aX + b$ with $a \neq 0$. Probability is preserved under the coordinate change, so

$$f_V(v) = \frac{1}{|a|} f_X\left(\frac{v - b}{a}\right).$$

Here V is the transformed variable, v its coordinate, f_V its density, and a, b are scale and translation. Density decreases by $|a|$ when the coordinate interval expands by that factor. Consequently its negative log-density gains the constant $\log_2 |a|$. Averaging over the unchanged event probabilities gives the entropy correction.

Take a uniform source on an interval of width 0.1. Its differential entropy is $\log_2(0.1) = -3.321928$. Quantize it into ten equal bins of width 0.01: each bin has probability one tenth, and the discrete entropy is $\log_2(10) = 3.321928$ bits. Here the fine-bin relationship is exact because the density is constant and the bins align with the support.

Change units by multiplying all coordinates by 100. The interval width becomes ten, differential entropy becomes $\log_2(10) = 3.321928$, and bin width becomes one. The same ten outcomes still cost 3.321928 bits. Scaling changes the density entropy and bin-width correction in opposite directions; it does not create additional distinguishable events. Changing only the number of bins would change the observation, whereas changing coordinate units does not.

Relative entropy and mutual information remain meaningful

Comparing densities by a ratio cancels shared coordinate units. A relationship between two variables should not disappear when either is measured in a different reversible coordinate system.

For densities f and g,

$$D_{\mathrm{KL}}(f\|g) = \int f(x)\log_2\frac{f(x)}{g(x)}\,dx, \qquad I(X;Y) = D_{\mathrm{KL}}(f_{XY}\|f_X f_Y).$$

Here f_{XY} is a joint density and $f_X f_Y$ is the independent product reference. Divergence is nonnegative, with an infinite value when support or integrability demands it.

Under separately invertible coordinate changes of X and Y, the Jacobian factors cancel in the density ratio and integral, so mutual information is unchanged. This statement does not apply to a noninvertible projection or quantization; those can discard information and are governed by data processing.

When all entropies are finite and compatible,

$$I(X;Y) = h(X) + h(Y) - h(X,Y) = h(Y) - h(Y \mid X).$$

Here $h(X,Y)$ is joint differential entropy and $h(Y \mid X)$ is expected conditional differential entropy. Conditional differential entropy may be negative. The resulting mutual information remains nonnegative.

The density-ratio definition is safer when entropy differences would subtract infinities. A noiseless deterministic identity between genuinely continuous variables can have infinite mutual information and a singular joint law; ordinary full-dimensional joint-density formulas then need care.

For a jointly Gaussian scalar pair with correlation ρ,

$$I(X;Y) = -\frac{1}{2}\log_2(1-\rho^2).$$

Here ρ is the correlation coefficient and $|\rho| < 1$ makes covariance nonsingular. Correlation is sufficient for this formula only under joint Gaussianity; a general dependence cannot be summarized by one covariance number.

Gaussian distributions maximize entropy at fixed covariance

Among distributions with a specified covariance, a Gaussian spreads uncertainty most broadly in the differential-entropy sense. This is a constrained statement, not a claim that every source is Gaussian.

A nonsingular d-dimensional Gaussian with covariance Σ has

$$h(G) = \frac{1}{2}\log_2\left((2\pi e)^d \det\Sigma\right).$$

Here G is the Gaussian vector, d its dimension, Σ its positive-definite covariance, and e the base of natural logarithms. Its mean does not affect entropy.

For another density f with the same mean and covariance, compare it with Gaussian density g. The Gaussian log-density is a constant plus a quadratic form. Its expectation under f equals its expectation under g because the first two moments match. Thus

$$D_{\mathrm{KL}}(f\|g) = h(G) - h(X) \geq 0.$$

Here X has density f, and the equality assumes the needed finite quantities. It proves the maximum-entropy property and identifies equality with the Gaussian distribution [1,2].

In one dimension, fixed variance σ^2 gives maximum entropy $\frac{1}{2}\log_2(2\pi e\sigma^2)$. Specifying only support gives a different extremizer: a uniform density on that bounded support. Constraints determine the maximum-entropy distribution.

This proof is valuable beyond communication. It makes a modeling choice transparent: adopting a Gaussian from a mean and covariance adds no further shape constraint in the maximum-entropy formulation. It does not certify the fit, rule out multimodality or make a Gaussian likelihood robust to outliers.

Code: Gaussian information and reversible scaling

The complete example estimates covariance from a synthetic Gaussian pair, computes its Gaussian-model information, and verifies that an invertible scaling changes entropy while preserving the information formula.

```
import numpy as np

rng = np.random.default_rng(37)
rho = 0.7
covariance = np.array([[1, rho], [rho, 1]])
samples = rng.multivariate_normal([0, 0], covariance, size=100000)
estimated_cov = np.cov(samples, rowvar=False)
def gaussian_entropy(cov):
    sign, logdet = np.linalg.slogdet(np.atleast_2d(cov))
    assert sign > 0
    d = np.atleast_2d(cov).shape[0]
    return 0.5 * (d * np.log(2*np.pi*np.e) + logdet) / np.log(2)
def gaussian_information(cov):
    return (gaussian_entropy([[cov[0, 0]]]) +
            gaussian_entropy([[cov[1, 1]]]) - gaussian_entropy(cov))
estimated = gaussian_information(estimated_cov)
exact = -0.5 * np.log2(1-rho**2)
transform = np.diag([100, 0.2])
scaled_cov = transform @ estimated_cov @ transform.T
assert abs(gaussian_information(scaled_cov) - estimated) < 1e-12
assert abs(gaussian_entropy(scaled_cov) - gaussian_entropy(estimated_cov)
           - np.log2(abs(np.linalg.det(transform)))) < 1e-12
assert abs(estimated - exact) < 0.02
print("Estimated Gaussian-model MI / exact MI:", estimated, exact)
print("Entropy shift:", gaussian_entropy(scaled_cov) - gaussian_entropy(estimated_cov))
```

This is a Gaussian parametric calculation, not a general-purpose mutual-information estimator. On a non-Gaussian dataset, plugging in covariance answers a Gaussian-model question and can miss substantial nonlinear dependence.

Derive the real Gaussian-channel capacity

Signal power increases the spread of distinguishable outputs; noise blurs them. The capacity depends on their ratio, under the precise Gaussian model.

Consider one real amplitude per use:

$$Y = X + Z, \qquad Z \sim \mathcal{N}(0, N), \qquad Z \perp X, \qquad \mathbb{E}[X^2] \le P.$$

Here X is the input, Y the output, Z independent zero-mean Gaussian noise, $N > 0$ noise variance, $P \ge 0$ average input power, and $\perp$ denotes independence. Across uses the noise is independent with the same law.

Conditioning on X leaves only translated noise, so $h(Y \mid X) = h(Z)$. Output variance is at most $P + N$. Gaussian maximum entropy therefore yields

$$I(X;Y) \le \frac{1}{2}\log_2(2\pi e(P+N)) - \frac{1}{2}\log_2(2\pi eN) = \frac{1}{2}\log_2\left(1 + \frac{P}{N}\right).$$

Here P/N is the signal-to-noise ratio. Independent Gaussian input $X \sim \mathcal{N}(0, P)$ makes the output Gaussian and attains this single-letter bound. The corresponding coding theorem establishes

$$C(P) = \frac{1}{2}\log_2\left(1 + \frac{P}{N}\right)$$

bits per real channel use under the standard average-power model [1,3]. The maximizing input has zero mean because a deterministic mean consumes power without increasing distinguishable variation.

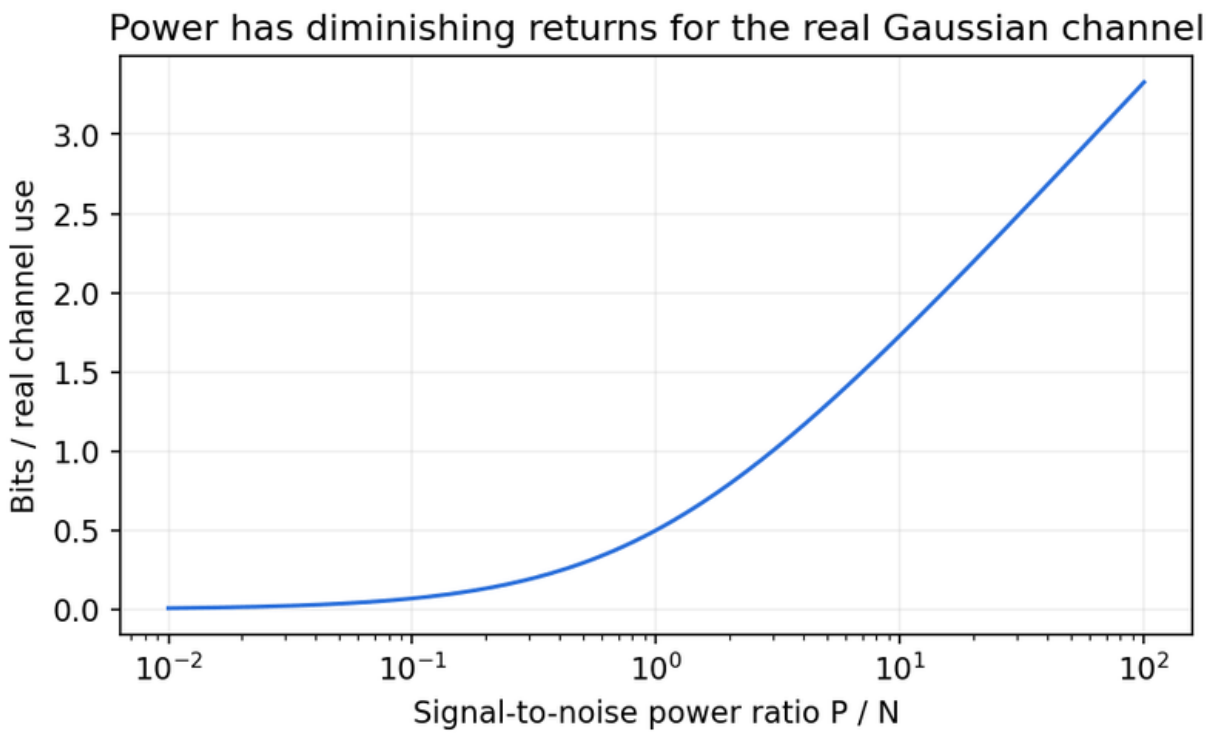


Figure 1 — One real amplitude per use, independent Gaussian noise and average-power constraint. The factor one half belongs to this real-valued convention.

An average-power constraint does not impose a strict maximum on every amplitude. Adding a peak-amplitude constraint changes the optimizing input distribution and generally the value. A finite constellation also defines a more restricted signaling problem.

The analogous complex-valued formula often lacks the one-half factor because a complex use includes two real dimensions, under a corresponding noise and power convention. Comparing the expressions without defining a use can create a false factor-of-two improvement.

A geometric view and its limits

Over many uses, noise places the observation in a cloud around the transmitted codeword. Good codewords are far enough apart that the clouds are usually distinguishable.

For n independent Gaussian noise samples, squared norm concentrates near nN:

$$\frac{1}{n}\sum_{t=1}^{n} Z_t^2 \to N.$$

Here Z_t is noise at use t, and the arrow denotes the appropriate law-of-large-numbers convergence. Power-constrained codewords lie near an input radius on the scale $\sqrt{nP}$, while outputs occupy a larger scale involving $P + N$.

Comparing high-dimensional output volume with noise-cloud volume suggests a count on the scale $(1 + P/N)^{n/2}$, whose normalized binary logarithm is the capacity expression. This is a geometric explanation, not an assertion that Gaussian noise has bounded support.

Noise can leave any finite sphere with positive probability. Reliable communication requires that atypical observations become sufficiently rare, not that noise clouds are literally disjoint for every possible outcome. That distinction separates vanishing-error capacity from zero-error communication.

The geometry also explains why a one-use diagram can be misleading. Long blocks allow many coordinates to cooperate, and their aggregate norms concentrate. A short-block constellation has its own distance and decoding tradeoffs.

Parallel channels and water-filling

When several independent channels have different noise levels, distribute power where it creates the greatest marginal increase in information. Some very noisy channels may receive no power.

For m parallel real Gaussian channels with variances $N_i > 0$, allocate powers $P_i \geq 0$ under total P_{total}:

$$\max_{\sum_i P_i \leq P_{\text{total}}} \sum_{i=1}^{m} \frac{1}{2} \log_2 \left(1 + \frac{P_i}{N_i}\right).$$

Here i indexes one parallel dimension, m is their count, and the objective is bits per joint use of all dimensions. Independent Gaussian inputs attain the component rates.

The derivative with respect to an active power is $1/[2 \ln 2 (N_i + P_i)]$. Equalizing derivatives under a common budget yields

$$P_i = (\mu - N_i)_+, \qquad \sum_i (\mu - N_i)_+ = P_{\text{total}}.$$

Here μ is the water level and $(u)_+ = \max(u, 0)$. The notation reflects filling above noise floors: active channels reach a common total signal-plus-noise level.

This follows from the optimality conditions of a concave optimization problem. A channel whose noise floor exceeds the level receives zero; assigning it positive power would give a smaller marginal return than an active lower-noise channel.

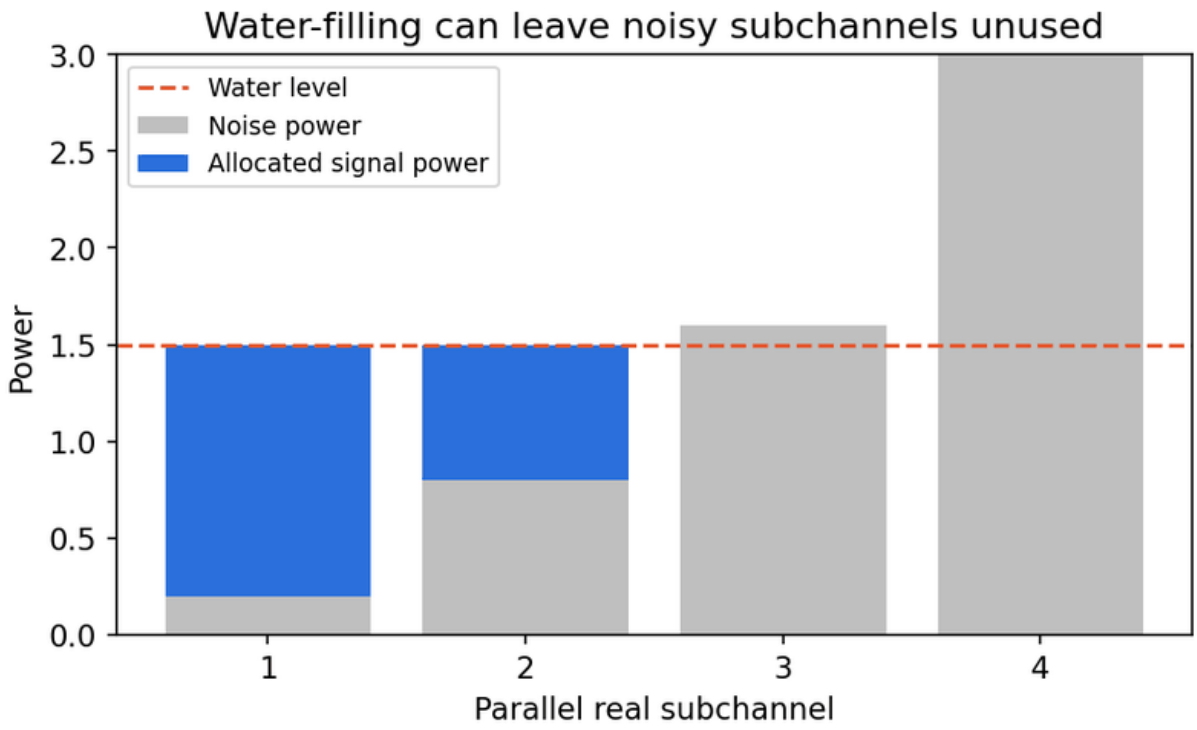


Figure 2 — Total signal power 2.0 across four independent real channels. Water-filling is an analytical resource allocation under the stated model.

Solve the plotted allocation by hand

The noise floors are 0.2, 0.8, 1.6 and 3.0, and the total signal-power budget is two. First suppose only the first channel is active. Its total level would be $0.2 + 2 = 2.2$, which rises above the second floor. Leaving the second channel unused cannot be optimal.

Try the first two active channels. Their powers sum to $(\mu - 0.2) + (\mu - 0.8) = 2$, giving $\mu = 1.5$. This is above both active floors and below the next floor 1.6, so the assumed active set is consistent. Allocate powers 1.3, 0.7, zero and zero.

The resulting joint-use rate is $\frac{1}{2}\log_2(7.5) + \frac{1}{2}\log_2(1.875) = 1.906891$ bits. Equal allocation of 0.5 to each channel gives about 1.561252 bits. Both designs spend total power two; the improvement comes from its placement, not an enlarged budget.

The third channel becomes active when the level exceeds 1.6, which first requires budget $(1.6 - 0.2) + (1.6 - 0.8) = 2.2$. The fourth becomes active above level three, at budget $(3 - 0.2) + (3 - 0.8) + (3 - 1.6) = 6.4$. These thresholds explain the changing active set in a capacity–power curve.

Equal signal-plus-noise levels apply only to the active channels. A channel with a floor above the level keeps zero signal power. This complementary condition is why blindly forcing equal total levels across all four channels would produce impossible negative powers.

Code: solve the level and compare equal allocation

This complete example solves the level by bisection, verifies the budget and active optimality condition, and compares its rate with equal power.

```
import numpy as np

noise = np.array([0.2, 0.8, 1.6, 3.0])
budget = 2.0
low, high = noise.min(), noise.max() + budget
for _ in range(100):
    level = (low + high) / 2
    if np.maximum(level-noise, 0).sum() > budget:
        high = level
    else:
        low = level
level = (low + high) / 2
power = np.maximum(level-noise, 0)
equal = np.full(len(noise), budget / len(noise))
rate = lambda allocation: 0.5*np.log2(1+allocation/noise).sum()
assert abs(power.sum()-budget) < 1e-12
assert np.allclose(noise[power > 0]+power[power > 0], level)
assert rate(power) >= rate(equal)
print("Water level / power allocation:", level, power)
print("Water-filled / equal-allocation joint rate:", rate(power), rate(equal))
```

The objective counts one simultaneous use of every dimension. Dividing by the number of dimensions gives a different per-dimension average. Neither number becomes bits per second until a use rate is supplied.

With colored Gaussian noise, a covariance eigenbasis can create parallel noise modes under compatible linear input and power assumptions. With channel gains, the effective noise floor depends on gain as well as physical noise. A zero gain cannot be inverted into a useful channel.

Vector channels connect geometry with covariance

Several measurements can carry overlapping information about several input coordinates. A matrix describes mixing, while covariance describes the noise cloud and the chosen signal spread.

Consider the real linear channel

$$Y = AX + Z, \qquad Z \sim \mathcal{N}(0, K_Z), \qquad X \sim \mathcal{N}(0, K_X), \qquad Z \perp X.$$

Here X is a d-dimensional input, Y an m-dimensional output, A an m by d channel matrix, K_X an input covariance and K_Z a positive-definite noise covariance. The input covariance can be positive semidefinite, allowing unused directions.

The output covariance is $AK_XA^\top + K_Z$. Subtracting Gaussian output and conditional noise entropies gives

$$I(X;Y) = \frac{1}{2}\log_2 \frac{\det(AK_XA^\top + K_Z)}{\det K_Z}.$$

Here $A^\top$ is the matrix transpose and determinants refer to output-dimensional covariance matrices. The positive noise covariance keeps the denominator and output covariance nonsingular, even if the signal has lower rank.

Under an average total-power convention, $\operatorname{tr} K_X \le P$ constrains the sum of input coordinate variances. Maximizing the displayed information over positive-semidefinite K_X gives the corresponding Gaussian vector-channel problem [1]. The trace notation means the sum of diagonal entries, and P is total power per vector use.

The useful directions depend jointly on A and K_Z. A large raw coordinate variance may lie in a direction strongly attenuated by the channel. Correlated noise may make a direction less informative than its marginal variance suggests. Diagonalizing a compatible transformed problem reveals parallel modes where the same marginal-return principle applies.

This example makes clear why preprocessing and physical channel changes must be distinguished. An invertible receiver coordinate transform changes both the signal matrix and the noise covariance; the determinant ratio stays invariant. A projection discards output directions and can reduce information. A noninvertible transmitter map changes which input signals can be produced.

For numerical work, compute log-determinants with a stable factorization instead of taking a large determinant and then its logarithm. A huge determinant can overflow while its logarithm is moderate. Check positive definiteness and matrix dimensions before interpreting the answer.

Conditional entropy and deterministic representations

A representation can be numerically continuous but carry only a finite set of possible states. Its mathematical distribution depends on how it was created, not the array's floating-point storage type.

The differential-entropy definitions assume densities with respect to continuous coordinate volume. A discrete label stored as a floating-point number remains discrete. A vector produced by a deterministic map of a finite dataset has finite empirical support, even if its coordinates are real-valued.

A lower-dimensional manifold inside a larger feature space has a singular distribution relative to full-dimensional volume. Fitting a full-dimensional density implicitly introduces a noise or smoothing model. The resulting entropy belongs to that model and can depend strongly on its scale.

Likewise, adding tiny independent noise to a deterministic continuous relationship makes a density-based calculation possible, but the noise level becomes part of the information question. Letting that level approach zero can send mutual information to infinity. Reporting a finite value without the smoothing convention hides this dependency.

These observations are useful when comparing neural representations. The information relevant

to a discrete task label can remain finite, while information between an exactly deterministic continuous input and representation can be singular. A compression or predictive objective must specify which variables, noise mechanisms and quantizers define its rate.

The practical remedy is to state the observation model before estimating information: discrete bins, an explicit noisy channel, a parametric density or a fixed-resolution coding protocol. Each is legitimate for its stated purpose, and each answers a different question.

Why vanishing added noise can make information diverge

Consider $X \sim \mathcal{N}(0,1)$ and representation $V = X + \varepsilon$, where independent noise $\varepsilon \sim \mathcal{N}(0,\tau^2)$ has positive variance τ^2. This is a Gaussian channel with signal power one, so $I(X;V) = \frac{1}{2}\log_2(1 + 1/\tau^2)$. With $\tau^2 = 1$, information is one half bit. With $\tau^2 = 0.01$, it is approximately 3.329106 bits. As noise variance tends to zero, the number grows without bound.

At exactly zero noise, $V = X$ is an exact continuous copy. The pair lies on a line in the two-dimensional joint space, so it has no ordinary two-dimensional joint density. This is why directly subtracting marginal and joint differential entropies at that endpoint is unsafe. Finite-resolution bins would instead give a finite discrete information question. The noise or resolution convention determines which question a neural-representation estimate answers.

Modeling and estimation: what the formulas certify

A Gaussian formula certifies a Gaussian problem. It does not convert every covariance estimate into a reliable physical communication limit.

The scalar capacity derivation uses independent additive Gaussian noise, an average second-moment budget and an unbounded real input alphabet. A sensor with clipping, quantization, signal-dependent noise or a changing noise distribution may violate one or more assumptions.

For a bounded output quantizer, information cannot exceed the logarithm of its number of possible labels. A real-valued capacity formula calculated before quantization is an upper reference for a different observation model; it is not automatically attainable by the quantized receiver.

Likewise, using a Gaussian approximation for a non-Gaussian source's entropy produces a moment-based upper bound under the needed conditions. It is not a measured entropy value. Good agreement of means and covariances does not establish agreement of tails or dependence.

Finite sample estimates of covariance can be singular when dimension is too high relative to observations. Then a log-determinant formula may fail. Regularizing the covariance creates a modified statistical model; the regularization value affects the reported information and should be stated.

For evaluation, keep the physical noise model, coordinate convention, estimated parameters and observed decoder performance separate. A synthetic test can verify algebra and implementation, while independently collected data must verify the assumptions.

Modern view

Butakov and colleagues' NeurIPS 2024 paper [4] studies mutual-information estimation through normalizing-flow transformations. It provides a modern example of using invertible density models to help estimation, while retaining the need to assess the estimator rather than interpreting coordinate entropy as intrinsic information.

The review perspective in contemporary semantic-communication literature [5] distinguishes receiver modeling, generative reconstruction and learned joint coding. For continuous channels, our inference is that those designs should report power normalization, real or complex use conventions, receiver quantization and channel mismatch explicitly.

An attractive reconstructed image can coexist with low faithful information about a particular original detail. A task-based objective may intentionally discard that detail; this is a rate–distortion choice rather than a different differential-entropy convention.

Open problems include robust information estimation in high dimension, evaluating likelihood models under shift, handling finite precision consistently at both endpoints, and designing learned systems whose resource accounting matches the physical channel. Invertibility and Gaussian assumptions simplify mathematical analysis, but neither substitutes for those checks.

The next chapter asks how much source information is needed when some reconstruction error is acceptable. Gaussian capacity concerns transporting information; Gaussian rate–distortion concerns how much information the source demands.

Key takeaways

- Density is probability per coordinate unit, not a point probability.
- Differential entropy can be negative and changes under coordinate scaling.
- Fine quantization adds a resolution-dependent discrete description cost.
- Relative entropy and mutual information survive reversible coordinate changes.
- Gaussian maximum entropy is a consequence of fixed moment constraints.
- Real Gaussian capacity includes an explicit average-power budget and one-half factor.
- Water-filling equalizes marginal returns across active parallel channels.
- Parametric information formulas and empirical decoder reliability answer different questions.

Exercises

1. Find the differential entropy of a uniform variable on an interval of width 0.01. Explain why its negative value does not permit a negative-length code.

Hint

Use log interval width and separate bin coding.

2. Quantize a unit uniform source into 16, 64 and 256 equal bins. Compute exact discrete entropy and relate it to differential entropy.

Hint

Each bin is equiprobable.

3. Verify mutual-information invariance under separate nonzero scalings of a jointly Gaussian pair. What changes if one variable is rounded to a single bit?

Hint

Determinants cancel for reversible scales; rounding is not invertible.

4. Compare capacity gains from doubling power at signal-to-noise ratios 0.1 and 10. Explain why absolute rate increments behave differently from relative power increments.

Hint

Subtract capacities before and after doubling.

5. Increase the water-filling budget until the third and fourth channels become active. Calculate the activation thresholds from the noise floors.

Hint

Set the water level to the next noise floor.

6. Explain which assumptions fail for a clipped receiver with two output labels. Give an information upper bound independent of signal power.

Hint

Output entropy is at most one bit.

References

1. T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006, Chapters 8–9. Publisher book and contents.
2. T. Weissman, "Continuous Random Variables," Stanford EE276 Lecture 8 notes, 2024. Primary lecture notes.
3. T. Weissman, "AWGN Channel," Stanford EE276 Lecture 9 notes, 2024. Primary lecture notes.
4. I. D. Butakov, A. D. Tolmachev, S. V. Malanchuk, A. M. Neopryatnaya and A. A. Frolov, "Mutual Information Estimation via Normalizing Flows," *NeurIPS*, 2024. Published paper.

5. L. X. Nguyen, A. D. Raha, P. S. Aung, D. Niyato, Z. Han and C. S. Hong, “A Contemporary Survey on Semantic Communications: Theory of Mind, Generative AI, and Deep Joint Source-Channel Coding,” *IEEE Communications Surveys & Tutorials*, 2025. Published-paper metadata and author version.

IT 06 • Rate–Distortion and Lossy Representations

Derive binary and Gaussian rate–distortion limits, compute finite-alphabet test channels, and evaluate quantization and learned representations with explicit distortion.

Online edition: https://blog.shyandram.dev/en/learn/information-theory/it-06-rate-distortion/

What you'll learn

- Specify a distortion function before calling a reconstruction acceptable.
- Distinguish operational rate from an informational test channel.
- Derive Bernoulli/Hamming and Gaussian/squared-error rate–distortion functions.
- Explain covering achievability and the converse's information accounting.
- Compute finite-alphabet rate–distortion points with alternating updates.
- Relate practical quantization and learned objectives to honest rate and quality measurements.

The big picture

Lossless coding asks for the exact original. Lossy coding asks for a useful substitute. The second question cannot be answered until "useful" has a mathematical meaning: which differences matter, how much they cost, and whether the constraint concerns average error or rare large failures.

> **Explain like I'm five**
>
> You describe a drawing to a friend with only a few words. You cannot mention every tiny dot. If the goal is to recognize the animal, its shape matters more than every background speck. If the goal is to reproduce the drawing exactly, those same omissions are errors.

This chapter follows Cover and Thomas's Chapter 10 [1] and Stanford's primary rate–distortion notes [2,3]. The central result minimizes mutual information over hypothetical reconstruction laws. That optimization defines a limit; it does not by itself build a finite file format or guarantee the behavior of a neural decoder.

The distinction connects directly to vision. A pixel reconstruction, a machine-readable feature and a plausible generated image preserve different properties. Their rates and distortions should be compared only after the objectives and resource conventions are made explicit.

Choose the distortion before the codec

An error measure specifies which source differences the receiver tolerates. There is no universal distortion score that captures every task.

Let X be a source symbol and $\widehat{X}$ a reconstruction in a possibly different alphabet. A nonnegative distortion function is $d(x, \hat{x})$. For a length-n block,

$$d_n(x^n, \hat{x}^n) = \frac{1}{n} \sum_{t=1}^{n} d(x_t, \hat{x}_t).$$

Here t indexes symbols, n is block length, and d_n is average per-symbol distortion. A common constraint is $\mathbb{E}[d_n(X^n, \widehat{X}^n)] \leq D$, with D the allowed average and expectation under the specified source and code.

Hamming distortion is zero for a correct symbol and one otherwise. Squared-error distortion is $(x - \hat{x})^2$ for real amplitudes. They answer different questions: the first counts mismatches; the second weights large numerical deviations more heavily.

Squared error also depends on units. Scaling source and reconstruction by a multiplies distortion by a^2. A rate–distortion curve should therefore state source normalization, dimensional averaging and the distortion's units.

The usual additive model assigns a cost to each symbol and averages. A structured task may need a block-level criterion: preserving a relation, a rare feature or a decision. It can still be studied, but it is not automatically the same single-letter problem.

Average distortion does not bound the worst individual reconstruction. A small mean can coexist with rare large errors. If the application requires a tail probability, a maximum error or per-group guarantees, include that separate condition rather than reading it into a mean.

The reproduction alphabet is another part of the definition. Restricting reconstructions to a small codebook, integer pixels or a particular model family reduces the feasible set. An unconstrained theoretical optimum may then be unattainable for that design. Conversely, allowing a receiver to output a task label rather than an image changes both the alphabet and the distortion question. A fair comparison should state these choices before showing any rates, so readers can tell whether two systems preserve the same object.

Operational and informational rate–distortion

A real code sends one of finitely many descriptions. A test channel describes a possible statistical relationship between source and reconstruction. The theorem connects their asymptotic limits.

A block encoder maps X^n to one of M indices, and a decoder maps the index to $\widehat{X}^n$. Its fixed rate is

$$R_n = \frac{1}{n} \log_2 M.$$

Here M is the number of possible transmitted descriptions and R_n is bits per source symbol. The operational function $R(D)$ is the infimum of asymptotically achievable rates at the specified distortion.

For an independent memoryless source and an appropriate additive distortion setting, the rate–distortion theorem gives

$$R(D) = \min_{p(\hat{x}|x):\, \mathbb{E}[d(X,\widehat{X})]\le D} I(X;\widehat{X}).$$

Here the source law $p(x)$ is fixed, while $p(\hat{x} \mid x)$ varies over normalized reconstruction laws. The minimum notation applies to settings where a minimizer exists; broader continuous cases can require an infimum and regularity conditions [1,2].

This reconstruction law is often called a **test channel**. Unlike communication capacity, its input source is not optimized. Capacity maximizes transmitted information under a given physical channel; rate–distortion minimizes required information under a reconstruction constraint.

A stochastic test channel is a mathematical object describing the desired joint distribution. The operational theorem uses block codes to approach its information rate. Independently drawing a noisy reconstruction for every source symbol does not itself explain how few bits a receiver needs to reproduce those draws.

Basic shape and zero-rate reconstruction

Allowing more error cannot require more bits. At some distortion, the receiver can always output one fixed reconstruction and need no message.

Define

$$D_0 = \min_{\hat{x}} \mathbb{E}[d(X,\hat{x})].$$

Here D_0 is the best constant-reconstruction average distortion and $\hat{x}$ is chosen independently of the observed source. At $D \ge D_0$, zero rate is feasible.

The function $R(D)$ is nonnegative, nonincreasing and convex in the standard setting. Nonincrease follows because a larger distortion allowance includes all previously feasible codes.

Convexity has a time-sharing interpretation. For fraction α of a long block, use one code; for the rest, use another. Then

$$R(\alpha D_1 + (1-\alpha)D_2) \le \alpha R(D_1) + (1-\alpha)R(D_2).$$

Here $\alpha \in [0,1]$ is the share, D_1, D_2 the component distortions, and the right side their average rates. Predetermined block boundaries avoid per-symbol signaling of the choice.

At zero distortion, a finite source with an identity-only zero-cost condition returns the ordinary entropy limit. If the distortion considers several different source values equally correct, zero distortion may require less than source entropy. For a genuinely continuous Gaussian source under squared error, the zero-distortion rate diverges.

Those qualifications matter. Saying "zero distortion means entropy" without specifying the distortion's zero set or alphabet can erase the very reason that a task representation compresses.

Binary sources with Hamming error

A fair binary source costs one bit per symbol to preserve exactly. Permitting a controlled mismatch fraction reduces the necessary rate.

For $X \sim \text{Bernoulli}(p)$ with $0 < p \leq 1/2$ and Hamming distortion,

$$R(D) = \begin{cases} h_2(p) - h_2(D), & 0 \leq D \leq p, \\ 0, & D \geq p. \end{cases}$$

Here p is the probability of one, D is mismatch probability, and h_2 is binary entropy. The zero-rate reconstruction is always zero, which makes errors on fraction p of source symbols.

To obtain a lower bound, let $E = X \oplus \widehat{X}$ be the mismatch indicator, where $\oplus$ denotes binary exclusive-or. Then

$$I(X;\widehat{X}) = H(X) - H(E \mid \widehat{X}) \geq h_2(p) - h_2(D).$$

Here E has probability of one at most $D \leq p \leq 1/2$, and binary entropy increases over that range. Conditioning cannot increase discrete entropy.

Equality is reached by a **backward** construction: choose $\widehat{X}$ and independent $E \sim \text{Bernoulli}(D)$, then set $X = \widehat{X} \oplus E$. The reconstruction probability satisfies

$$\Pr(\widehat{X} = 1) = \frac{p - D}{1 - 2D}.$$

Here $D < p \leq 1/2$ avoids the singular fair-source endpoint; endpoints follow by continuity or a direct constant reconstruction. The construction gives the required source law and makes mismatch independent of reconstruction.

For a fair source, the curve reduces to $1 - h_2(D)$ through $D = 1/2$. For a biased source, naively flipping X forward with probability D generally does not attain the same optimum. The direction of the independence condition is part of the derivation.

Build the optimal binary joint table

The backward construction becomes clearer when its probabilities are written out. Take source probability $p = 0.2$ and allowed mismatch $D = 0.1$. If $q = \Pr(\widehat{X} = 1)$, independence of reconstruction and mismatch gives

$$p = q(1 - D) + (1 - q)D = D + q(1 - 2D).$$

Here p is the required source-one probability, q the reconstruction-one probability and D the independent backward flip probability. Solving gives $q = (0.2 - 0.1)/(1 - 0.2) = 0.125$. Multiplying reconstruction probabilities by independent error probabilities yields:

Joint probability	$\widehat{X} = 0$	$\widehat{X} = 1$	Source marginal
$X = 0$	0.7875	0.0125	0.8

Joint probability	$\widehat{X} = 0$	$\widehat{X} = 1$	Source marginal
$X = 1$	0.0875	0.1125	0.2
Reconstruction marginal	0.875	0.125	1.0

The off-diagonal entries sum to 0.1, exactly the allowed distortion. Dividing either reconstruction column by its total shows a 0.1 probability that the source differs from that reconstruction. Thus $H(X \mid \widehat{X}) = h_2(0.1)$ and required information is $h_2(0.2) - h_2(0.1) = 0.252933$ bits per source symbol.

The equivalent **forward** conditional channel is asymmetric. Given $X = 0$, it outputs one with probability $0.0125/0.8 = 0.015625$. Given $X = 1$, it outputs zero with probability $0.0875/0.2 = 0.4375$. The large error on rare ones and small error on common zeros achieve the specified overall error while meeting the optimal posterior uncertainty.

By contrast, independently flipping the source forward with probability 0.1 preserves the same average distortion, but produces reconstruction-one probability 0.26. Its information is $h_2(0.26) - h_2(0.1) = 0.357751$ bits, greater than the optimum. "Ten percent noise" is therefore insufficient to identify an optimal test channel: the direction and independence relationship matter.

```
import numpy as np

def entropy(p):
    p = np.asarray(p, dtype=float)
    positive = p > 0
    return -np.sum(p[positive] * np.log2(p[positive]))

source_one, D = 0.2, 0.1
q = (source_one-D)/(1-2*D)
joint = np.array([[(1-q)*(1-D), q*D],
                  [(1-q)*D, q*(1-D)]])
px, pr = joint.sum(axis=1), joint.sum(axis=0)
information = entropy(px)+entropy(pr)-entropy(joint)
assert np.allclose(px, [0.8, 0.2])
assert np.isclose(joint[0, 1]+joint[1, 0], D)
assert np.allclose([joint[1, 0]/pr[0], joint[0, 1]/pr[1]], D)
exact = entropy(px)-entropy([D, 1-D])
assert np.isclose(information, exact)
print("Backward joint table:", joint)
print("Optimal information:", information)
```

This calculation verifies a statistical optimum. It still needs a long-block code to translate that information rate into a transmitted reconstruction index.

A Gaussian source under squared error

A broad Gaussian source needs more bits for the same absolute precision. Requiring smaller error grows rate logarithmically.

For $X \sim \mathcal{N}(0, \sigma^2)$ and $d(x, \hat{x}) = (x - \hat{x})^2$,

$$R(D) = \max\left\{0, \frac{1}{2}\log_2 \frac{\sigma^2}{D}\right\}, \qquad D > 0.$$

Here σ^2 is source variance and D the mean squared error in the same squared units. At $D \geq \sigma^2$, a zero reconstruction requires no bits. As $D \to 0$, rate grows without bound [1,3].

Let $E = X - \widehat{X}$ be reconstruction error. For compatible finite differential entropies,

$$\begin{aligned} I(X; \widehat{X}) &= h(X) - h(E \mid \widehat{X}) \\ &\geq h(X) - h(E) \geq \frac{1}{2}\log_2 \frac{\sigma^2}{D}. \end{aligned}$$

Here conditioning reduces differential entropy when mutual information is well defined, and Gaussian maximum entropy bounds error entropy by its second moment. The expression is a lower bound in the nonzero-rate range.

Achievability of the informational minimum uses independent $\widehat{X} \sim \mathcal{N}(0, \sigma^2 - D)$ and $E \sim \mathcal{N}(0, D)$, with $X = \widehat{X} + E$. This is again a backward test channel; the error is independent of reconstruction, not of the source.

The equivalent distortion–rate relation is $D(R) = \sigma^2 2^{-2R}$. It refers to asymptotically optimized block coding. A simple scalar quantizer generally lies above the limit at its resulting rate.

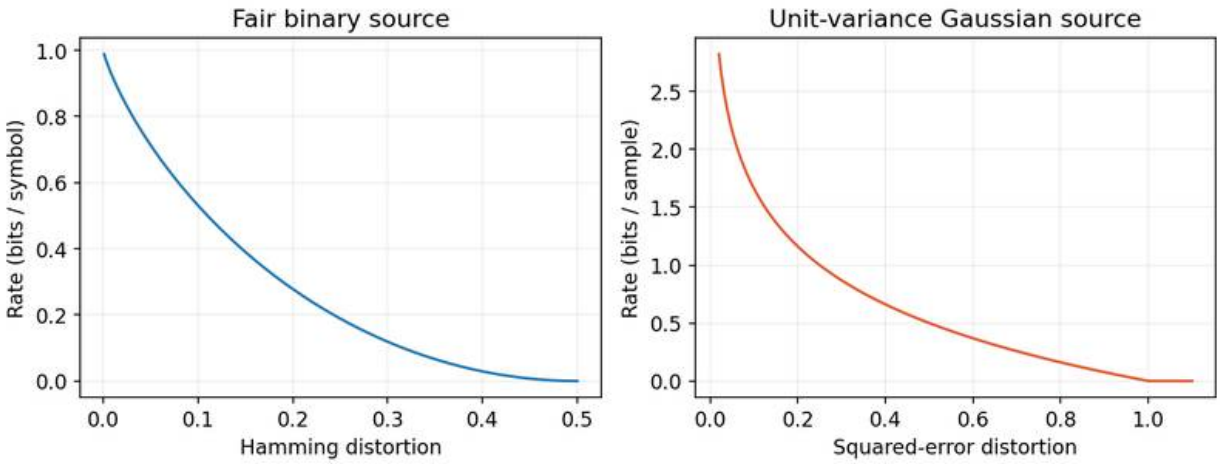


Figure 1 — Analytical examples with different distortion meanings. The horizontal values cannot be compared as a common quality score.

Code: test the Gaussian backward construction

This complete synthetic experiment verifies the source variance, error variance and information formula. It does not serialize a block code; it tests the optimizing joint distribution.

```
import numpy as np

rng = np.random.default_rng(38)
variance, distortion = 1.0, 0.2
reconstruction = rng.normal(0, np.sqrt(variance-distortion), size=100000)
error = rng.normal(0, np.sqrt(distortion), size=100000)
source = reconstruction + error
rho = np.corrcoef(source, reconstruction)[0, 1]
information = -0.5 * np.log2(1-rho**2)
exact = 0.5 * np.log2(variance/distortion)
print("Source variance / error MSE:", source.var(), np.mean(error**2))
print("Error-reconstruction correlation:", np.corrcoef(error, reconstruction)[0, 1])
print("Gaussian-model information / exact rate:", information, exact)
assert abs(source.var()-variance) < 0.02
assert abs(np.mean(error**2)-distortion) < 0.01
assert abs(information-exact) < 0.02
```

Independence is built into the sampling, while the finite correlation is only a check. Small empirical correlation alone would not establish independence for arbitrary non-Gaussian variables.

Why covering differs from channel packing

A lossy source code needs enough reconstruction representatives to cover likely source blocks within acceptable error. A channel code instead separates transmitted representatives to resist noise.

Fix a feasible source–reconstruction joint distribution. Draw many reconstruction codewords from its marginal. For a typical source block, a random reconstruction has a matching joint relationship with a probability on the exponential scale $2^{-nI(X;\widehat{X})}$.

With more than approximately $2^{nI(X;\widehat{X})}$ representatives and a small rate margin, a suitable match becomes likely. The encoder selects its index; the decoder returns that shared reconstruction. Joint typicality connects the chosen pair to the target distortion.

This is a covering argument. The exact proof needs typicality definitions and source/distortion assumptions; the exponential scale explains the rate without serving as a finite-code performance formula.

The converse uses the transmitted index J:

$$nR_n \geq H(J) \geq I(X^n;\widehat{X}^n) \geq \sum_{t=1}^{n} R(D_t) \geq nR\left(\frac{1}{n}\sum_t D_t\right).$$

Here J ranges over M descriptions, $R_n = \log_2 M/n$, $D_t = \mathbb{E}[d(X_t,\widehat{X}_t)]$, and the source is memoryless. The first information bound uses that reconstruction is determined by the index; the per-symbol step uses source independence and the informational definition; the last step is convexity.

The code cannot convey more information about the source than its index contains. This accounting explains why a decoder's sophisticated prior cannot recover arbitrary missing details at no source rate. It can choose useful or probable reconstructions according to the specified distortion.

Compute a finite test channel

For discrete alphabets, alternate between a reconstruction marginal and a source-conditioned reconstruction law. A penalty parameter moves along the rate–distortion frontier.

Minimize the natural-unit objective $I_{\text{nats}}(X;\widehat{X}) + \beta\mathbb{E}[d]$ with $\beta \geq 0$. Given current reconstruction marginal $q_{\hat{x}}$,

$$V(\hat{x} \mid x) = \frac{q_{\hat{x}} \exp[-\beta d(x,\hat{x})]}{\sum_z q_z \exp[-\beta d(x,z)]}, \qquad q_{\hat{x}}^{\text{new}} = \sum_x p_x V(\hat{x} \mid x).$$

Here p_x is the fixed source law, V is the test channel, z indexes reconstructions, and β measures distortion penalty in natural units. Rates reported in bits divide the natural-unit information by $\ln 2$.

The Blahut–Arimoto alternating computation is discussed in the textbook [1]. A positive initial marginal covers all candidate reconstruction symbols. At a solution, compute actual distortion and mutual information; do not label β itself as the distortion.

Where the exponential test-channel update comes from

The alternating rule follows from a constrained minimization, not a heuristic resemblance to a probability distribution. Temporarily fix a positive reconstruction marginal q_j. For source state i, minimize the row contribution

$$\sum_j V_{ij}[\ln(V_{ij}/q_j) + \beta d_{ij}], \qquad \sum_j V_{ij} = 1.$$

Here V_{ij} is the conditional probability of reconstruction j given source i, q_j the fixed reference marginal, d_{ij} distortion and β the natural-unit distortion penalty. The positive source weight p_i multiplies this row and does not change its minimizing conditional distribution.

Add a row-normalization multiplier η_i and differentiate with respect to V_{ij}. The stationary equation is $\ln(V_{ij}/q_j) + 1 + \beta d_{ij} + \eta_i = 0$. Exponentiation makes V_{ij} proportional to $q_j \exp(-\beta d_{ij})$; row normalization supplies the denominator in the displayed update.

Next fix V. Its induced marginal $q_j^{\text{new}} = \sum_i p_i V_{ij}$ minimizes the cross-entropy part over normalized q. Alternating these two steps lowers the penalized objective under the stated finite-alphabet setting. It also explains the geometry: larger β concentrates each row toward low-distortion reconstructions, while the marginal factor favors reconstructions used by the population.

For a fair binary source with Hamming distortion, symmetry keeps q uniform. Off-diagonal probability is therefore $D = 1/(1+e^{\beta})$, and the rate is $1 - h_2(D)$. This analytic relation is what the code

checks. It gives a direct interpretation of the penalty without mistaking the penalty value for an achieved distortion.

Code: compare with exact binary curves

The implementation checks convergence of the reconstruction marginal and validates the resulting points against independent analytic formulas. Its tolerance is a numerical update tolerance, not the upper–lower capacity gap from the previous chapter.

```
import numpy as np
from scipy.special import logsumexp

def entropy(p):
    p = np.asarray(p)
    return -np.sum(p[p > 0] * np.log2(p[p > 0]))
def rd_point(p, distortion_matrix, beta):
    p = np.asarray(p, dtype=float)
    d = np.asarray(distortion_matrix, dtype=float)
    q = np.full(d.shape[1], 1/d.shape[1])
    for _ in range(50000):
        scores = np.log(q)[None, :] - beta*d
        V = np.exp(scores-logsumexp(scores, axis=1, keepdims=True))
        updated = p @ V
        if np.max(abs(updated-q)) < 1e-13:
            q = updated
            break
        q = updated
    else:
        raise RuntimeError("Rate-distortion updates did not converge")
    joint = p[:, None]*V
    positive = joint > 0
    ratio = np.divide(V, q[None, :], out=np.ones_like(V), where=positive)
    rate = np.sum(joint[positive]*np.log2(ratio[positive]))
    distortion = np.sum(joint*d)
    return rate, distortion, V

d = np.array([[0, 1], [1, 0]])
for beta in [0.0, 1.0, 2.0, 4.0]:
    rate, D, V = rd_point([0.5, 0.5], d, beta)
    exact = 1-entropy([D, 1-D])
    assert abs(rate-exact) < 1e-10
    print("Penalty / D / rate / exact:", beta, D, rate, exact)
rate, D, V = rd_point([0.8, 0.2], d, 4.0)
exact = entropy([0.8, 0.2])-entropy([D, 1-D])
assert D < 0.2 and abs(rate-exact) < 1e-9
print("Biased-source D / rate / exact:", D, rate, exact)
```

For arbitrary distortion matrices, a small marginal-update difference is a useful convergence check but not a rigorous global numerical certificate. Inspect objective changes, supports and indepen-

dent special cases. Very large penalties can underflow probability entries or require a more careful implementation.

Scalar quantization is a concrete achievable design

Round source values to a finite-resolution grid, then losslessly describe their grid indices. The rounding creates distortion; probability coding reduces index rate.

With step Δ, use

$$K = \text{round}(X/\Delta), \qquad \widehat{X} = \Delta K, \qquad D_\Delta = \mathbb{E}[(X - \widehat{X})^2].$$

Here K is the discrete index, $\widehat{X}$ the grid reconstruction, and D_Δ its squared-error distortion. An entropy-coded index has a rate governed by the index distribution, plus practical coding costs.

For sufficiently smooth densities at small step, a common approximation is $D_\Delta \approx \Delta^2/12$. The approximation treats quantization error as approximately uniform inside a cell; it need not hold at coarse steps, near clipping or for structured concentrated values.

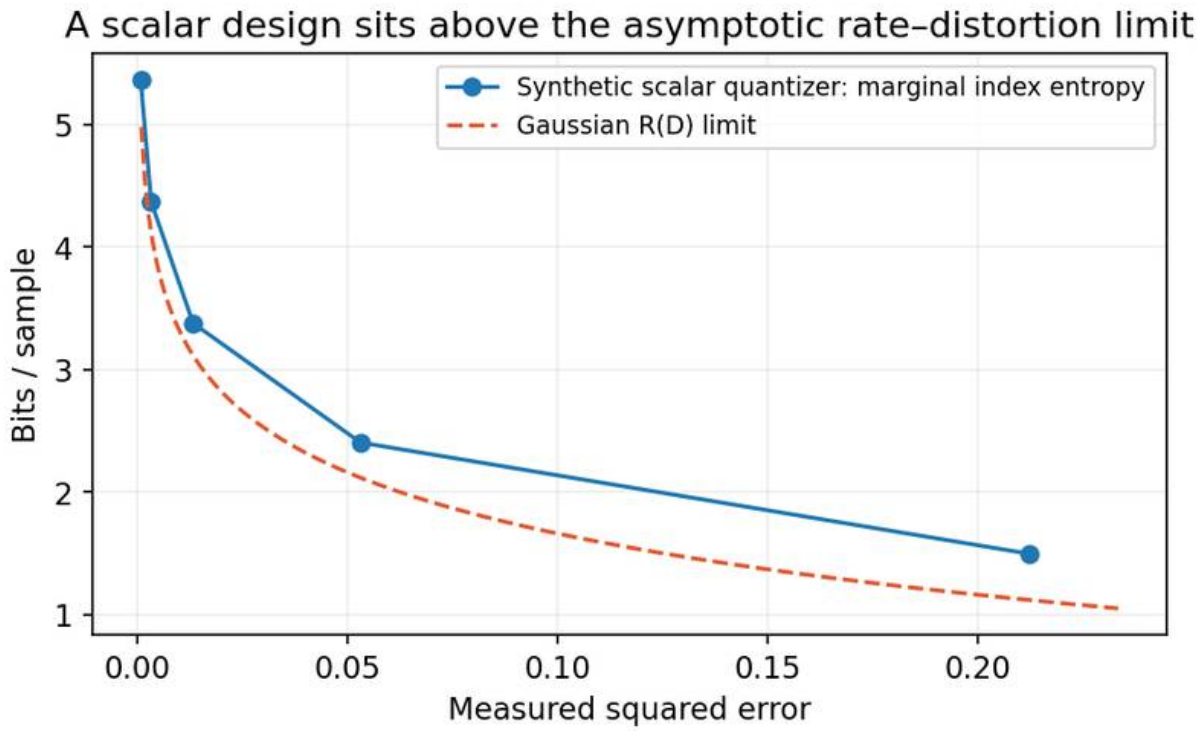


Figure 2 — 20,000 synthetic unit-Gaussian samples. Index entropy is an ideal marginal coding statistic; no actual bitstream length is claimed.

```
import numpy as np
```

```
rng = np.random.default_rng(26)
source = rng.normal(size=20000)
for step in [0.1, 0.2, 0.4, 0.8, 1.6]:
    index = np.rint(source/step).astype(int)
    reconstruction = step*index
    _, counts = np.unique(index, return_counts=True)
    p = counts/counts.sum()
    rate_estimate = -np.sum(p*np.log2(p))
    mse = np.mean((source-reconstruction)**2)
    gaussian_limit = max(0, 0.5*np.log2(1/mse))
    print("Step / index entropy / MSE / reference rate:",
          step, rate_estimate, mse, gaussian_limit)
```

The empirical distribution is measured on the same sample being quantized, so it is a descriptive statistic. To compare a learned entropy model, fit it on separate training observations and score a fixed held-out set. Transmit or share its parameters under an explicit convention.

Index entropy is not an actual compressed file size. A codec also specifies index support, out-of-range handling, reconstruction step, message length and entropy-coder termination. Finite empirical estimates can fluctuate around theoretical relationships without invalidating the population theorem.

Parallel Gaussian sources use reverse water-filling

A limited description budget should preserve the most variable source directions more carefully. Low-variance directions can be discarded when their entire variation is within the error allowance.

For independent Gaussian source coordinates with variances σ_i^2, use total squared-error budget D_{total}:

$$R(D_{\text{total}}) = \min_{\sum_i D_i \le D_{\text{total}}} \sum_i \max\left\{0, \frac{1}{2}\log_2 \frac{\sigma_i^2}{D_i}\right\}.$$

Here i indexes coordinates, D_i is their allocated mean squared error, and R is bits per complete source vector. Within the positive-rate range, differentiating the objective makes the optimal active distortions equal:

$$D_i = \min(\theta, \sigma_i^2), \qquad \sum_i \min(\theta, \sigma_i^2) = D_{\text{total}}.$$

Here θ is the reverse water level, chosen for a budget below total source variance. A coordinate with variance below the level is reconstructed as its mean, paying its full variance and no rate.

The similarity to channel water-filling is instructive, but the roles differ. Channel allocation spends signal power above noise floors to maximize transported information. Source allocation permits error up to source variances to minimize required description information.

For a correlated Gaussian vector, an orthogonal covariance eigenbasis produces independent coordinates and preserves total squared error. The allocation then acts on eigenvalues. A nonlinear task distortion or a non-Gaussian distribution need not reduce to this same eigenvalue calculation.

This also explains the benefit and limitation of a low-dimensional reconstruction. Discarding a low-variance coordinate is inexpensive for total squared error, but that coordinate could still be decisive for a particular label. Task relevance and variance are different criteria.

A reverse-water-filling allocation

Take three independent Gaussian coordinates with variances 4, 1 and 0.25, and allow total squared error one per vector. Giving each coordinate error one third would waste the third coordinate's budget: reconstructing it as zero already incurs only 0.25.

The optimal common error level for the remaining two coordinates solves $\theta + \theta + 0.25 = 1$, so $\theta = 0.375$. The distortions are 0.375, 0.375 and 0.25; their sum is one. Required rate is

$$R = \frac{1}{2}\log_2 \frac{4}{0.375} + \frac{1}{2}\log_2 \frac{1}{0.375} + 0 \approx 2.415037.$$

Here R is bits per three-coordinate source vector. The first two coordinates receive positive description rate, while the third is reconstructed as its mean and costs zero rate. The distortion budget is a sum per vector, not a mean per coordinate; dividing the budget by three would express the same operating point using a different convention.

The shared error level follows by differentiating each active coordinate's rate: the magnitude of the rate decrease per extra unit of distortion is $1/(2 \ln 2\, D_i)$. Equalizing that marginal saving makes active D_i equal. Once D_i reaches its source variance, there is no further rate to save, so that coordinate leaves the active set.

A small-variance coordinate may still encode an important label. Reverse water-filling is optimal for this Gaussian squared-error problem, while a label-preservation requirement would need a different distortion. This example makes the link between optimization conditions and the actual receiver objective explicit.

Comparing operating points without changing the problem

A fair rate–distortion comparison keeps source data, quality criteria and resource assumptions fixed. Otherwise a better-looking curve may be solving an easier problem.

An empirical operating point consists of measured rate and measured distortion under one complete protocol. Its description should specify whether the rate is fixed length, average variable length, ideal negative log-probability or actual serialized bits.

The theoretical curve in this chapter uses a memoryless population source and asymptotically long codes. A finite experiment has sampling uncertainty, finite blocks, a restricted model family and implementation costs. Reporting all of those as one gap obscures which component is responsible.

Keep the same held-out observations for different codec settings. Fit quantizers, probability models and any learned transforms using training data. Use validation data to choose penalties or operating points. Reserve the final test set to estimate the selected protocol's performance.

Report distortion across meaningful groups when the population is heterogeneous. An average over common cases can hide a subgroup whose reconstructions are much worse. This does not invalidate a mean-distortion theorem; it shows that the selected mean may not fully express the desired application.

Also state decoding randomness. A deterministic decoder and a randomized generative decoder can have different error distributions at the same stored index. Repeated reconstructions of the same test input reveal that variability; selecting the best random reconstruction after seeing the original introduces a selection step that a real receiver may not have.

Finally, keep shared side information comparable. A decoder with a task-specific database, pretrained prior or transmitted adaptation receives a different protocol from one without it. Shared resources can be useful, but their availability and cost convention need to accompany the claimed frontier.

Representations, tasks and learned rate penalties

A representation can discard irrelevant detail while preserving a task. That helps only when the retained task is specified and evaluated on the intended population.

A learned encoder may produce quantized latent Z with entropy model q, while a decoder returns $\widehat{X}$. A common training objective has the form

$$\mathcal{L} = \mathbb{E}[-\log_2 q(Z)] + \lambda \mathbb{E}[d(X, \widehat{X})].$$

Here the first term is an ideal coding-cost surrogate in bits, $\lambda \geq 0$ balances the specified distortion, and expectations use the training distribution. This objective does not automatically equal the population rate–distortion function.

The latent alphabet, entropy-model mismatch, finite coder, side streams and model costs affect actual rate. The architecture restricts achievable reconstructions. Optimization can find a local or model-constrained tradeoff rather than the information-theoretic minimum.

If the receiver needs a label Y, an appropriate criterion can evaluate the decision inferred from Z instead of reconstructing all of X. That is a different source-distortion problem. A feature preserving one label can discard information needed by a later, unrelated task.

A perceptual criterion may reward plausible texture even when texture differs from the source. A faithful reconstruction criterion may penalize the same change. Neither objective is universally correct; their distinction needs to remain visible in text, figures and evaluation.

Multiple distortions can be reported together without collapsing them into one unexplained score. For example, separate numerical fidelity, a stated task error and a distributional realism criterion.

Modern view

The 2023 monograph by Yang, Mandt and Theis [4] separates lossless entropy coding from lossy transform design, quantization, perceptual objectives and task-oriented coding. Its taxonomy helps locate which component of a learned system changes rate, which changes distortion, and which changes the evaluation goal.

Huang and Wu's 2024 review [5] explicitly contrasts human and machine consumers of learned image compression. The shared lesson is that a compressed representation's usefulness is receiver-dependent; one reconstruction score cannot establish preservation for every downstream analysis.

Hofer and Böhme's 2024 preprint [6] develops a taxonomy of neural-image miscompression. It motivates examining the kinds of changed content, rather than assuming that average numerical error completely characterizes reconstruction behavior. We cite its verified preprint status.

Our inference from rate–distortion theory is to evaluate a **frontier**, not one favorable operating point: measured bitstream size versus explicitly defined distortion on a fixed test population. Keep source normalization, model-sharing assumptions and decoder resource budgets consistent.

Open questions include task transfer after irreversible compression, distortion measures for rare details, performance under distribution shift, and fair resource accounting for generative priors. Longer blocks or more elaborate decoders can improve a particular tradeoff while adding latency or relying on stronger shared assumptions.

The next chapter extends information beyond coding: divergence describes statistical evidence, maximum entropy selects distributions under constraints, and logarithmic growth connects prediction with decision-making.

Key takeaways

- Distortion defines what a lossy reconstruction is allowed to change.
- Rate–distortion fixes the source and optimizes a reconstruction law.
- Its standard frontier is nonincreasing and convex.
- Binary and Gaussian optima use carefully defined backward test channels.
- Covering and information accounting connect test channels to block codes.
- Alternating numerical updates require explicit logarithm and penalty units.
- Quantizer index entropy differs from actual serialized rate.
- Learned or perceptual objectives must be judged against their stated receiver and distortion.

Exercises

1. For a Bernoulli source with probability 0.2, compute the zero-rate distortion and rates at distortions 0, 0.05 and 0.1.

Hint

Use $h_2(p) - h_2(D)$ within the nonzero-rate range.

2. Construct the backward binary joint law at $p = 0.2$, $D = 0.1$. Compare it with independently flipping the source forward by 0.1.

Hint

Make mismatch independent of reconstruction.

3. For a Gaussian source of variance 4, compute the rates needed for squared errors 1 and 0.25. Explain the result after changing coordinate units.

Hint

Rate depends on a variance-to-distortion ratio.

4. Extend the finite test-channel code to a three-symbol source and an asymmetric distortion matrix. Report the penalty, actual rate and actual distortion separately.

Hint

Check row normalization and reconstruct the joint law.

5. Add clipping to scalar quantization. Compare errors near the center and tails instead of reporting only the mean.

Hint

Track saturation cases separately.

6. Design two different distortion functions for a tiny binary image: exact pixel preservation and preservation of whether any foreground pixel exists. Explain why their zero-distortion rates differ.

Hint

The task identifies equivalence classes of images.

References

1. T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006, Chapter 10. Publisher book and contents.

2. T. Weissman, “Lossy Compression & Rate Distortion Theory,” Stanford EE276 Lecture 13 notes, 2024. Primary lecture notes.
3. T. Weissman, “Lossy Compression & Rate Distortion Theory Continued,” Stanford EE276 Lecture 14 notes, 2024. Primary lecture notes.
4. Y. Yang, S. Mandt and L. Theis, “An Introduction to Neural Data Compression,” *Foundations and Trends in Computer Graphics and Vision*, 15(2), 113–200, 2023. Published monograph, author version.
5. C.-H. Huang and J.-L. Wu, “Unveiling the Future of Human and Machine Coding: A Survey of End-to-End Learned Image Compression,” *Entropy*, 26(5), 357, 2024. Published review.
6. N. Hofer and R. Böhme, “A Taxonomy of Miscompressions: Preparing Image Forensics for Neural Compression,” arXiv preprint, 2024. Primary preprint.

IT 07 • Statistical Evidence, Maximum Entropy and Growth

Connect likelihood ratios, rare empirical distributions, Fisher information, maximum entropy and logarithmic growth through explicit statistical models.

Online edition: https://blog.shyandram.dev/en/learn/information-theory/it-07-statistics-decision-growth/

What you'll learn

- Interpret a likelihood ratio as evidence rather than a posterior probability.
- Use empirical types to explain divergence-controlled rare-event rates.
- Distinguish hypothesis-testing error tradeoffs from parameter estimation.
- Derive Fisher information and a regular unbiased-estimation bound.
- Construct maximum-entropy distributions from explicit constraints.
- Relate log-optimal allocation and universal mixtures to coding regret.

The big picture

Information measures are not restricted to storage and communication. They also quantify how observations distinguish competing explanations, how strongly a distribution changes with a parameter, and how costly a poor predictive decision can become over repeated trials.

> **Explain like I'm five**
>
> Two bags have different color mixtures. After seeing many beads, you can ask which bag better explains them. The same probabilities can help you describe the bead string, estimate the mixture, or choose a strategy in a repeated game. Each question uses the observations differently.

The book map is Cover and Thomas's Chapter 11 for statistics, Chapter 12 for maximum entropy, Chapter 6 for gambling and coding, and Chapter 16 for portfolio theory [1]. Stanford's method-of-types notes [2] provide an accessible primary finite-alphabet derivation.

We use mathematical decision games with fully specified probabilities and payoffs. Their purpose is to expose information identities, not to infer practical returns from historical data. In every section, distinguish the source law, the chosen model, the observed sample and the decision criterion.

Likelihood ratios compare explanations

A sample supports one model over another when that model assigns the sample a larger probability. Repeated independent observations add their log evidence.

For two fully specified distributions P and Q with compatible support, define

$$\Lambda(x^n) = \log \frac{P(x^n)}{Q(x^n)} = \sum_{t=1}^{n} \log \frac{P(x_t)}{Q(x_t)}.$$

Here x^n is an observed length-n sample, the final equality assumes independent observations under each model, and log may be natural or base two if used consistently. Positive values favor P in the likelihood comparison.

Under P, expected base-two log ratio is $nD_{\mathrm{KL}}(P\|Q)$. Under Q, it is $-nD_{\mathrm{KL}}(Q\|P)$. The different directions arise because expectation is taken under different laws.

A likelihood ratio is not a posterior probability. To calculate posterior odds, multiply by prior odds. To choose an action, incorporate the losses of wrong decisions. A threshold of zero is appropriate for certain symmetric prior-and-loss choices, not every test.

For simple hypotheses, the Neyman–Pearson likelihood-ratio principle describes the most powerful test at a specified false-alarm constraint [1]. “Simple” means the distributions are fixed, not families with unknown fitted parameters.

If parameters are estimated from the same sample, the test is a different procedure. Model selection, nuisance parameters and threshold calibration can alter its finite error behavior. It is useful to start with fully specified toy laws before interpreting a fitted classifier’s score as evidence.

Testing errors form a tradeoff

Making it harder to declare an alternative reduces false alarms but can miss more real alternatives. Neither error should disappear from the report.

Set null distribution P_0 and alternative P_1. A rejection region A gives

$$\alpha = P_0(X^n \in A), \qquad \beta = P_1(X^n \notin A).$$

Here α is the type-I error or false-alarm probability, β the type-II error or missed-alternative probability, and A the set where the test declares the alternative.

For Bernoulli probabilities $p_0 < p_1$, the log likelihood ratio of alternative to null is an increasing function of the count of ones. A threshold on that count therefore implements a likelihood-ratio test.

Discrete counts may prevent reaching an arbitrary false-alarm target exactly with a deterministic threshold. Randomization at a boundary can achieve an intermediate operating point mathematically; a deployed rule should state whether it uses such randomization.

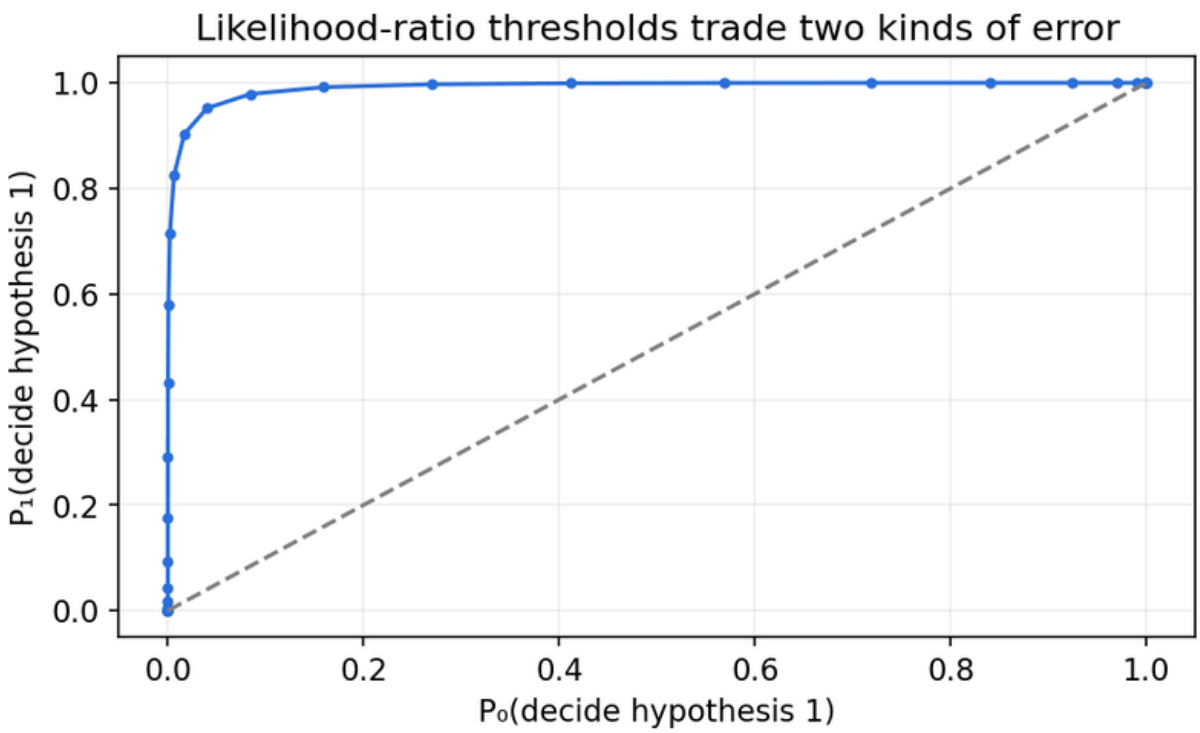


Figure 1 — Exact binomial probabilities. Detection probability is one minus type-II error; each point corresponds to a stated count threshold.

Code: exact errors and a Monte Carlo check

This complete example chooses the first count threshold satisfying a 5% false-alarm limit and calculates its missed-alternative probability. It then verifies the two laws independently.

```
import numpy as np
from scipy.stats import binom

n, p0, p1 = 30, 0.3, 0.6
thresholds = np.arange(n+2)
false_alarm = binom.sf(thresholds-1, n, p0)
detection = binom.sf(thresholds-1, n, p1)
threshold = thresholds[np.flatnonzero(false_alarm <= 0.05)[0]]
alpha = binom.sf(threshold-1, n, p0)
beta = binom.cdf(threshold-1, n, p1)
rng = np.random.default_rng(40)
observed_alpha = np.mean(rng.binomial(n, p0, 100000) >= threshold)
observed_beta = np.mean(rng.binomial(n, p1, 100000) < threshold)
print("Threshold / alpha / beta:", threshold, alpha, beta)
print("Simulation alpha / beta:", observed_alpha, observed_beta)
assert abs(observed_alpha-alpha) < 0.005
```

```
assert abs(observed_beta-beta) < 0.005
```

The selected threshold uses known distributions, not a test-set search. In a learned application, tune the threshold on validation data and estimate its errors on separate test observations. Data groups or time dependence can change the effective independence assumption.

The proportion of alternatives in an evaluation set affects the frequency of correct declarations, but it does not change the two conditional errors for a fixed test and fixed laws. A test can have a small false-alarm probability and still produce many false declarations when alternatives are extremely rare. To interpret those declarations, combine the conditional errors with the intended prior prevalence. This is a direct consequence of conditional probability, not a failure of the likelihood-ratio principle. State whether a reported quantity is conditioned on the null, conditioned on the alternative, or conditioned on the test's output.

Worked evidence is not the same as confidence

For the example's null probability 0.3 and alternative probability 0.6, a length-30 block with k ones has alternative-to-null log likelihood ratio

$$\Lambda_k = k \ln \frac{0.6}{0.3} + (30 - k) \ln \frac{0.4}{0.7}.$$

Here k is the observed count, Λ_k uses natural logarithms, and the two fractions compare the probabilities of one and zero under the competing laws. With 18 ones, $\Lambda_{18} \approx 5.761260$, giving likelihood ratio about 317.748375. This evidence favors the alternative strongly.

If the prior alternative probability is 0.01, prior odds are $0.01/0.99$. Posterior odds become approximately $317.748375 \times 0.01/0.99 = 3.209580$, corresponding to posterior probability about 0.762447. The likelihood ratio is not a 99.7% confidence value: the prior changes the posterior interpretation.

An even more practical distinction concerns all positive declarations from the threshold test. The code chooses threshold 14, with false-alarm probability about 0.040053 and missed-alternative probability about 0.048112. If only 1% of cases come from the alternative, the probability that a positive declaration is truly alternative is

$$\Pr(H_1 \mid \text{positive}) = \frac{\pi(1-\beta)}{\pi(1-\beta) + (1-\pi)\alpha} \approx 0.193588.$$

Here $\pi = 0.01$ is prior alternative prevalence, α the test's false-alarm probability and β its missed-alternative probability. The notation H_1 denotes the event that the alternative generated the case. Most positives in this population are false alarms, even though each conditional error is below 5%.

This conclusion follows from counting expected positive cases from the two populations. It does not change the likelihood-ratio optimality result or the measured conditional errors. A decision report should name its conditioning event so readers can distinguish a particular sample's posterior from the average composition of all positive declarations.

```
import numpy as np
from scipy.stats import binom

n, p0, p1, prior = 30, 0.3, 0.6, 0.01
count = 18
log_ratio = count*np.log(p1/p0)+(n-count)*np.log((1-p1)/(1-p0))
posterior = 1/(1+((1-prior)/prior)*np.exp(-log_ratio))
threshold = 14
alpha = binom.sf(threshold-1, n, p0)
beta = binom.cdf(threshold-1, n, p1)
positive_precision = prior*(1-beta)/(prior*(1-beta)+(1-prior)*alpha)
assert alpha <= 0.05 and binom.sf(threshold-2, n, p0) > 0.05
print("Likelihood ratio / sample posterior:", np.exp(log_ratio), posterior)
print("Probability alternative given positive:", positive_precision)
```

Empirical types reveal why divergence controls rare events

Many strings share the same histogram. Their number grows like an entropy exponent, while their individual source probabilities shrink like a cross-entropy exponent. The difference is divergence.

For a finite alphabet of size k, the type $\hat{p}$ of a length-n sequence is its empirical symbol distribution. Its type class $T(\hat{p})$ contains every sequence with those counts. Under independent source law q,

$$q(x^n) = 2^{-n[H(\hat{p})+D_{\mathrm{KL}}(\hat{p}\|q)]}.$$

Here x^n has type $\hat{p}$, $H(\hat{p})$ is empirical entropy, and divergence is measured in bits. The formula follows by grouping identical log probabilities by count.

The class size is bounded, with a conservative polynomial factor, by

$$(n+1)^{-k}2^{nH(\hat{p})} \le |T(\hat{p})| \le 2^{nH(\hat{p})}.$$

Here $|T|$ is the number of sequences; n and k are block length and alphabet size. Multiplying by one sequence probability gives

$$(n+1)^{-k}2^{-nD_{\mathrm{KL}}(\hat{p}\|q)} \le q(T(\hat{p})) \le 2^{-nD_{\mathrm{KL}}(\hat{p}\|q)}.$$

Here $q(T)$ is total type-class probability. The polynomial factor matters at finite length but disappears from the normalized logarithmic exponent [1,2].

The orientation is worth emphasizing: the first argument is the unusual empirical type; the second is the source actually generating observations. This differs from a coding mismatch calculation averaged under a fixed true source.

For sets of empirical distributions, the dominant exponential behavior is governed by the smallest divergence among allowed types. Sanov's theorem formalizes that large-deviation statement under its finite-alphabet and set-regularity conditions [1].

Code: exact binary type probability versus its exponent

This standalone example compares a binomial probability with the exponential bounds. It uses logarithms to avoid underflow and checks the finite polynomial gap.

```
import numpy as np
from scipy.stats import binom

q, target = 0.3, 0.6
for n in [30, 100, 300, 1000]:
    count = int(round(n*target))
    empirical = count/n
    divergence = (empirical*np.log2(empirical/q) +
                  (1-empirical)*np.log2((1-empirical)/(1-q)))
    negative_log_probability = -binom.logpmf(count, n, q)/np.log(2)
    exponent = n*divergence
    assert exponent <= negative_log_probability + 1e-10
    assert negative_log_probability <= exponent + 2*np.log2(n+1) + 1e-10
    print("Length / observed normalized exponent / divergence:",
          n, negative_log_probability/n, divergence)
```

A rare exact type is not the same event as a one-sided tail or a typical neighborhood. The exponent governing a tail comes from optimizing over all counts in that event, not blindly substituting its most extreme count.

From one type to a set of types

For an event defined by a collection $\mathcal{A}$ of empirical distributions, there are at most $(n+1)^k$ possible types. Summing the individual upper bounds gives

$$q(\hat{p} \in \mathcal{A}) \le (n+1)^k 2^{-n \inf_{v \in \mathcal{A}_n} D_{\mathrm{KL}}(v\|q)}.$$

Here $\mathcal{A}_n$ is the subset of $\mathcal{A}$ realizable at length n, v is a candidate type, and the infimum selects the least surprising type under source q. An empty event has probability zero and is treated separately.

The most probable portion of a rare event often lies at its boundary closest to the source, rather than at its most dramatic extreme. For a binary source with probability 0.3, a tail requiring frequency at least 0.6 is governed near frequency 0.6, not frequency one.

Finite type grids matter: the required boundary may fall between attainable fractions. Exact binomial sums handle that discreteness. The asymptotic divergence picture explains the trend while leaving rounding and polynomial factors visible.

This viewpoint is useful for interpreting apparent distribution shifts. A statistically unusual histogram can be evidence against a source law, but it does not identify why the law failed. Dependence, selection bias, measurement changes and a different source can all alter observed frequencies.

Asymmetric and symmetric hypothesis exponents

Different ways of budgeting the two testing errors lead to different information limits. Divergence is one such limit, not the answer to every classification problem.

Suppose observations are independent under null P or alternative Q, and type-I error is kept below a fixed nontrivial tolerance. The optimal type-II exponent approaches

$$-\frac{1}{n}\ln \beta_n \to D_{\text{nats}}(P\|Q).$$

Here β_n is alternative probability of incorrectly accepting the null, and D_{nats} is divergence with natural logarithms. This is the classical Stein asymptotic regime under suitable source/support conditions [1].

The direction follows the acceptance region: typical null observations must remain accepted, and the alternative probability of looking null has a P-to-Q divergence exponent. Relabeling hypotheses reverses the divergence and can change the exponent.

With fixed nonzero priors and a symmetric Bayesian error objective, the exponent instead involves Chernoff information:

$$C_{\text{Ch}}(P,Q) = -\min_{0\le s\le 1} \ln \sum_x P(x)^s Q(x)^{1-s}.$$

Here s interpolates between the two laws and x ranges over their finite alphabet. This quantity is symmetric despite the asymmetric appearance of the summand.

These exponents describe large independent samples, not exact errors for a short sample. The binomial experiment above is a direct finite calculation. Using an asymptotic exponent as an exact probability can be inaccurate when prefactors and decision boundaries matter.

Composite hypotheses, dependence, changing populations and unequal losses require their own analysis. An information-theoretic limit can explain which source differences help discrimination while leaving practical calibration and sampling design unresolved.

Sufficiency and data processing in inference

A summary is sufficient for a parameter when it retains all the sample's evidence about that parameter within the specified model family.

For a family $p_\theta(x^n)$, a statistic $T(X^n)$ is sufficient when the conditional sample law given T does not depend on θ. A factorization criterion expresses this as

$$p_\theta(x^n) = g_\theta(T(x^n))h(x^n).$$

Here θ is the parameter, T the statistic, g_θ the parameter-dependent factor and h a parameter-independent factor, under the relevant family and domination assumptions [1].

For independent Bernoulli trials, the number of ones is sufficient for the unknown probability. The ordering carries no extra parameter evidence in that family. For a Markov family, the ordering can

matter; a single count of ones is no longer generally sufficient.

If a random parameter Θ has a specified prior, sufficiency gives the conditional relation needed for $I(\Theta; X^n) = I(\Theta; T(X^n))$. Arbitrary summaries satisfy only the data-processing upper bound.

The qualification "within the family" matters. A statistic sufficient for one assumed model may discard evidence that the model is wrong. A mean can estimate a location while hiding multimodality, dependence or an outlier pattern.

This connects to learned representations: retaining information about one target is different from retaining every diagnostic property of the raw data. The task and source family must accompany a sufficiency claim.

Fisher information measures local parameter sensitivity

Fisher information measures how strongly the log probability changes when a parameter moves slightly. It is a local property of a model, distinct from mutual information between random variables.

For a scalar parameter θ and regular density or mass function p_θ, define score and information:

$$s_\theta(X) = \frac{\partial}{\partial \theta} \ln p_\theta(X), \qquad \mathcal{J}(\theta) = \mathbb{E}_\theta[s_\theta(X)^2].$$

Here s_θ is the score, $\mathcal{J}$ Fisher information per observation, and expectation is under the parameterized model. Under regularity, score expectation is zero.

For n independent observations, information adds to $n\mathcal{J}(\theta)$. The Cramér–Rao inequality for a regular unbiased estimator $\hat{\theta}$ gives

$$\mathrm{Var}_\theta(\hat{\theta}) \geq \frac{1}{n\mathcal{J}(\theta)}.$$

Here variance describes repeated samples at the fixed parameter. The bound requires differentiability, exchange of differentiation and integration, and appropriate support and unbiasedness assumptions [1].

For Bernoulli probability $\theta \in (0,1)$, $\mathcal{J}(\theta) = 1/[\theta(1-\theta)]$. The sample mean is unbiased with variance $\theta(1-\theta)/n$, achieving that bound.

The information changes with parameter coordinates. Reparameterizing a probability as a log odds multiplies the information by the squared parameter derivative. This is compatible with an unchanged underlying statistical family; the units of parameter error have changed.

Irregular models with parameter-dependent support or biased estimators need different bounds. A small empirical estimator variance alone does not prove accuracy if its mean is displaced from the parameter.

Derive the Bernoulli information and variance bound

For one Bernoulli observation $x \in \{0, 1\}$, log likelihood is $x \ln \theta + (1 - x) \ln(1 - \theta)$. Differentiating gives

$$s_\theta(x) = \frac{x}{\theta} - \frac{1 - x}{1 - \theta} = \frac{x - \theta}{\theta(1 - \theta)}.$$

Here $\theta \in (0, 1)$ is the probability of one and s_θ the score. Its two possible values are $1/\theta$ and $-1/(1 - \theta)$. Averaging their squares with probabilities θ and $1 - \theta$ yields $\mathcal{J}(\theta) = 1/[\theta(1 - \theta)]$.

Why does its reciprocal bound estimation variance? Let T be a regular unbiased estimator based on n independent observations, and let S_n be the sum of their scores. Differentiate unbiasedness, $\mathbb{E}_\theta T = \theta$, under the regularity assumptions:

$$1 = \frac{d}{d\theta} \mathbb{E}_\theta T = \mathbb{E}_\theta[(T - \theta) S_n].$$

Here T is a fixed function of the observed sample, $S_n = \sum_{i=1}^{n} s_\theta(X_i)$ is its total score, and $\mathbb{E}_\theta S_n = 0$. The right side is the covariance between estimator and score. Squared covariance cannot exceed the product of variances. Since $\mathrm{Var}_\theta(S_n) = n\mathcal{J}(\theta)$, the variance bound follows.

For $\theta = 0.3$ and $n = 100$, information per observation is approximately 4.761905 and the variance lower bound is 0.0021. The sample mean reaches it because its centered error is proportional to the total score. Increasing independent sample size by four divides this variance by four and its standard deviation by two. Independence is what makes score variances add; repeatedly measuring the same correlated trial would not justify that scaling.

Fisher information is local divergence curvature

Under suitable twice-differentiable regularity, neighboring parameter distributions satisfy

$$D_{\mathrm{nats}}(p_\theta \| p_{\theta+\delta}) = \frac{1}{2} \mathcal{J}(\theta) \delta^2 + o(\delta^2).$$

Here δ is a small parameter displacement, $\mathcal{J}(\theta)$ is the natural-log score information, and $o(\delta^2)$ denotes a remainder whose ratio to δ^2 tends to zero. With base-two divergence, divide the leading term by $\ln 2$.

The first derivative vanishes because the score has zero expectation. The second derivative gives the Fisher information under the regular model. This links local estimation sensitivity with the global divergence used in testing and coding.

It is a local approximation. A large parameter move can involve different curvature, changing supports or multiple modes. Treating one Fisher value as an exact distance across a whole model family would discard those differences.

Maximum entropy makes constraints explicit

If only a few expectations are known, choose the distribution with largest entropy among those matching them. The answer depends on the stated constraints and allowed support.

On a finite alphabet with features $\phi_j(x)$, impose $\mathbb{E}[\phi_j(X)] = m_j$. Maximizing entropy with normalization gives an exponential-family form

$$p_\lambda(x) = \frac{\exp(\sum_j \lambda_j \phi_j(x))}{Z(\lambda)}, \qquad Z(\lambda) = \sum_x \exp(\sum_j \lambda_j \phi_j(x)).$$

Here λ_j are multipliers chosen to satisfy the moment constraints, Z normalizes the mass function, and the positive-sign convention is a choice of multiplier sign. Existence and interior-support conditions matter.

Differentiating $\ln Z$ gives the constrained feature expectations. The entropy optimum is not determined by a mean alone unless support is also known. On a bounded interval, fixed support without other constraints gives a uniform density; fixed real mean and variance give a Gaussian; nonnegative support and a fixed mean give an exponential density [1].

Maximum entropy is a transparent modeling principle, not evidence that the real source belongs to the chosen family. Unmeasured dependencies and higher moments remain possible. A useful analysis records constraints, fits multipliers and then tests the resulting predictions on separate data.

The exponential form also explains its relationship to divergence. If a candidate distribution matches the specified feature expectations of the maximum-entropy model, averaging the model's log probability depends only on those shared moments. Relative entropy from the candidate to the model then equals the model entropy minus candidate entropy, using the same logarithm base.

That is another instance of a constraint turning a cross-entropy term into a fixed quantity. The Gaussian maximum-entropy proof from the previous chapter is a continuous moment-constrained example. This common algebra clarifies why maximum entropy, exponential families and information projections repeatedly appear together.

Derive and interpret the maximum-entropy distribution

Use natural logarithms during the optimization; this only rescales entropy and its multipliers. Add the normalization constraint and feature constraints to the entropy objective:

$$\mathcal{F}(p) = -\sum_x p_x \ln p_x + \alpha\left(\sum_x p_x - 1\right) + \sum_j \lambda_j \left(\sum_x p_x \phi_j(x) - m_j\right).$$

Here p_x are candidate probabilities, α is the normalization multiplier, λ_j the feature multipliers, $\phi_j(x)$ feature j and m_j its required expectation. At an interior optimum, differentiating each probability gives $-\ln p_x - 1 + \alpha + \sum_j \lambda_j \phi_j(x) = 0$. Exponentiating and normalizing yields the earlier exponential form.

For states 0, 1, 2 with mean 0.7, there is one feature $\phi(x) = x$. Relative weights are $1, e^{\lambda}, e^{2\lambda}$. A uniform distribution would have mean one, so the desired smaller mean requires a negative multiplier. The code finds $\lambda \approx -0.466121$, with probabilities approximately 0.494780, 0.310440 and 0.194780.

This model spreads probability over all allowed states while matching the mean. A point mass or a two-state mixture could satisfy some moment constraints too, but would have lower entropy here. At a feasible boundary, some probabilities can vanish and finite interior multipliers may no longer suffice. The support, moments and existence conditions are therefore part of the mathematical problem, rather than choices made after selecting a favored density.

Code: solve a finite maximum-entropy model and verify Fisher variance

This complete example finds a three-state distribution with a specified mean and compares Bernoulli estimator variance with its exact unbiased bound.

```
import numpy as np
from scipy.optimize import brentq
from scipy.special import softmax

states = np.array([0.0, 1.0, 2.0])
target_mean = 0.7
probability = lambda lam: softmax(lam*states)
lam = brentq(lambda value: probability(value) @ states-target_mean, -20, 20)
p = probability(lam)
assert abs(p @ states-target_mean) < 1e-12
print("Maximum-entropy multiplier / probabilities:", lam, p)

rng = np.random.default_rng(41)
theta, n = 0.3, 100
estimates = rng.binomial(n, theta, size=100000)/n
fisher = 1/(theta*(1-theta))
bound = 1/(n*fisher)
print("Empirical estimator variance / exact bound:", estimates.var(), bound)
assert abs(estimates.var()-bound) < 0.0001
```

The maximum-entropy calculation uses exactly the enumerated support. Allowing other states would change the optimization. The variance calculation checks a synthetic regular case, not a universal lower bound for every estimator design.

Logarithmic growth turns mismatch into a decision cost

In a repeated multiplicative game, a strategy's errors compound. Maximizing expected logarithmic growth makes the best allocation match the outcome probabilities under the stated fair-payoff model.

Let outcome i occur with probability p_i. Allocate fraction $q_i > 0$ to its contingent payoff, with $\sum_i q_i = 1$. A fair reference law $r_i > 0$ pays multiplier $1/r_i$ when outcome i occurs. Then expected base-two log growth is

$$g(p, q; r) = \sum_i p_i \log_2 \frac{q_i}{r_i} = D_{\mathrm{KL}}(p\|r) - D_{\mathrm{KL}}(p\|q).$$

Here p is the outcome law, q the allocation, r the reciprocal-payoff probability vector, and g is log multiplier per trial. The identity uses normalized laws and the stipulated contingent-payoff game [1].

The optimum is $q = p$ on the source support. A mismatched strategy loses exactly $D(p\|q)$ expected log growth. For two outcomes with equal multiplier two, r is fair and the optimum value is $1 - h_2(p)$.

This is the Kelly connection between prediction and growth. It does not mean every portfolio has contingent fair payoffs or that probabilities are known. Adding cash, transaction costs, leverage restrictions or different payoffs changes the optimization.

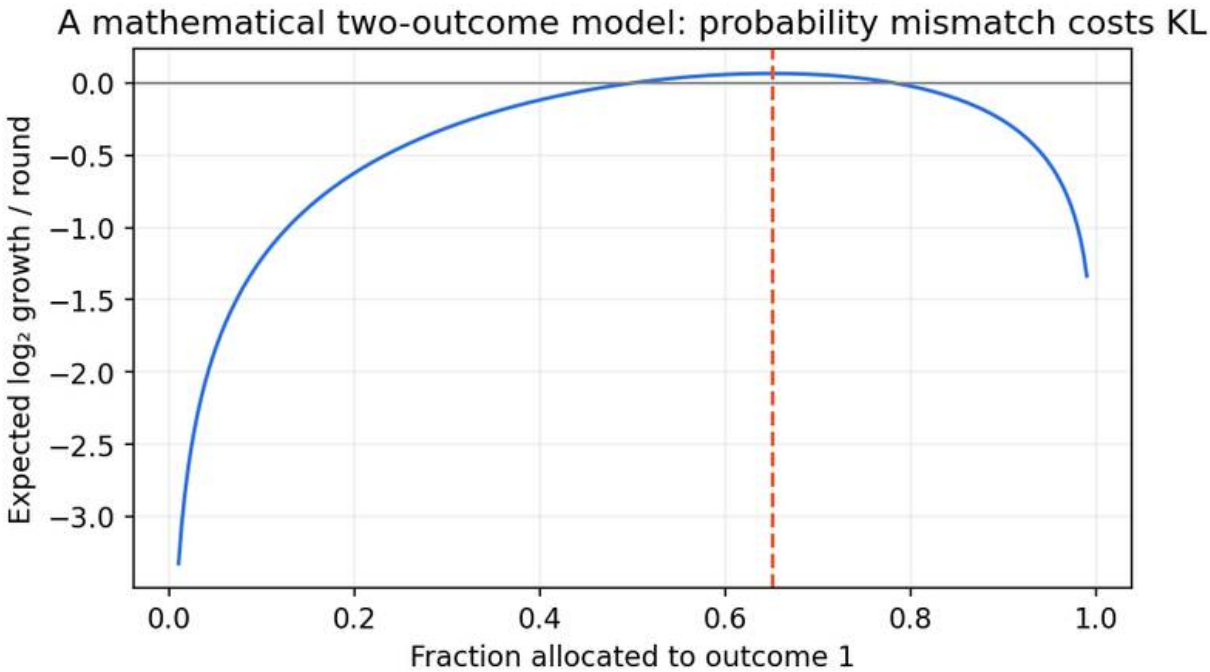


Figure 2 — Analytical two-outcome payoff model. The decrease from the optimum is probability-model divergence.

If a side observation Y is available before each outcome, optimize allocation conditionally. The improvement over a fixed optimal allocation is

$$\mathbb{E}_Y[g(p_{X|Y}, p_{X|Y}; r)] - g(p_X, p_X; r) = I(X; Y).$$

Here X is the outcome, Y the side information, and the reference payoff law r is unchanged. Mu-

tual information has an operational value in this particular log-growth game.

Universal mixtures connect portfolios with coding

Instead of committing to one fixed allocation, mix the wealth of many fixed strategies. Past outcomes then determine how much each strategy contributes.

For a positive return vector R_t and a fixed simplex allocation b, cumulative wealth is

$$W_n(b) = \prod_{t=1}^{n} b^\top R_t, \qquad W_n^{\mathrm{mix}} = \int W_n(b)\, \pi(db).$$

Here t indexes trials, b has nonnegative entries summing to one, R_t contains each component's relative payoff, and π is a prior measure over fixed allocations. The mixture has an equivalent sequential allocation obtained by weighting strategies by their past wealth.

Cover and Thomas's Chapter 16 develops universal-portfolio ideas and their long-run comparisons with a best constant allocation under stated conditions [1]. The benchmark is a fixed allocation in hindsight, not an unrestricted future-knowing trader.

For the two-outcome fair contingent game, a uniform prior on the probability parameter yields the same predictive mechanism as a beta mixture. Its sequential probability of one is $(N_1 + 1)/(t + 2)$ after t observations. This is a whole-count prior, distinct from the half-count predictor in lossless coding.

Code: mixture growth and the best constant allocation

This complete example computes wealth in logarithms and uses the beta integral for the uniform mixture. It compares a known source strategy with the retrospectively best fixed strategy and the causal mixture.

```
import numpy as np
from scipy.special import betaln

rng = np.random.default_rng(42)
p, n = 0.65, 2000
outcomes = rng.binomial(1, p, size=n)
ones, zeros = int(outcomes.sum()), n-int(outcomes.sum())
fitted = ones/n
oracle_log_wealth = n*np.log(2)+ones*np.log(p)+zeros*np.log(1-p)
best_log_wealth = n*np.log(2)+ones*np.log(fitted)+zeros*np.log(1-fitted)
mixture_log_wealth = n*np.log(2)+betaln(ones+1, zeros+1)
sequential_log_wealth, previous_ones = 0.0, 0
for t, outcome in enumerate(outcomes):
    q = (previous_ones+1)/(t+2)
    sequential_log_wealth += np.log(2*(q if outcome else 1-q))
    previous_ones += int(outcome)
assert abs(sequential_log_wealth-mixture_log_wealth) < 1e-9
```

```
assert mixture_log_wealth <= best_log_wealth
print("Source / best fixed / mixture growth bits per trial:",
      oracle_log_wealth/n/np.log(2), best_log_wealth/n/np.log(2),
      mixture_log_wealth/n/np.log(2))
print("Mixture regret to best fixed, bits:",
      (best_log_wealth-mixture_log_wealth)/np.log(2))
```

This is a controlled synthetic payoff experiment. The best fixed parameter uses the entire sequence and is a benchmark, while the mixture updates causally. Log wealth minus a best benchmark has the same algebra as negative-log-probability regret, with opposite orientation.

Modern view

Hellström, Durisi, Guedj and Raginsky's published 2025 monograph [3] unifies information-theoretic and PAC-Bayesian generalization bounds. Its taxonomy separates expectation, probability and conditional-mutual-information formulations, making clear that a statistical guarantee needs a specified random object and evaluation criterion.

El Gamal's 2026 proposed learning chapter [4] connects divergence identities with statistical learning through a textbook-oriented presentation. It is a draft supplement. We use its current perspective alongside the published second edition, without treating a proposed chapter as a completed new edition.

Our inference is that the same quantity can have several operational meanings only after the model is specified. Divergence can be a coding penalty, an unusual-type exponent, an evidence rate or a log-growth loss. The direction, logarithm base and limiting regime can differ among those uses.

Open problems in learned inference include reliable estimation of information in large parameter spaces, guarantees under dependent or shifted data, and bounds that remain informative for actual model selection procedures. A formal inequality can become vacuous when its information term is infinite or difficult to estimate.

For practical work, pair theoretical identities with independent finite experiments: exact toy probabilities to verify implementation, held-out likelihood to assess model fit, and decision-specific errors to evaluate use. No single entropy number replaces all three.

The final chapter combines multiple sources and receivers, develops stronger inequalities, and connects these foundations to modern learning and a small distributed-information capstone.

Key takeaways

- Likelihood evidence, posterior odds and decision thresholds are distinct.
- Testing reports both false-alarm and missed-alternative probabilities.
- Type probabilities have divergence exponents with explicit argument orientation.
- Hypothesis-testing exponents depend on the chosen error regime.
- Sufficiency is relative to a model family and target parameter.

- Fisher information gives local regular-estimation limits.
- Maximum entropy depends on support and stated constraints.
- Log-growth mismatch and universal mixture regret share coding identities.

Exercises

1. Reverse the null and alternative in the binomial example. Find a threshold meeting a 5% false-alarm limit and compare the resulting missed-alternative error.

   Hint

   The rejection region becomes a low-count event.

2. Derive the type-probability identity by grouping symbols with equal empirical counts. Explain why the divergence arguments have their displayed order.

   Hint

   Use counts times log source probability.

3. Compare the Stein divergences in both directions for Bernoulli probabilities 0.3 and 0.6. Explain why the asymmetric results do not contradict a symmetric Chernoff exponent.

   Hint

   Fix the acceptance region before choosing a divergence.

4. Reparameterize a Bernoulli probability by log odds. Derive its Fisher information and explain the changed error units.

   Hint

   Multiply by the squared probability derivative.

5. Solve a maximum-entropy distribution on states 0, 1, 2, 3 with mean 1.2. Then add a second-moment constraint and describe the extra multiplier.

   Hint

   The second feature is squared state.

6. In the fair contingent game, calculate expected growth at source probability 0.65 for allocations 0.5, 0.65 and 0.8. Verify the divergence loss and explain which strategy knows future outcomes.

   Hint

   None knows the future; the hindsight benchmark only fits a constant after observation.

References

1. T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006, Chapters 6, 11–12 and 16. Publisher book and contents.
2. T. Weissman, "Method of Types," Stanford EE276 Lecture 15 notes, 2024. Primary lecture notes.
3. F. Hellström, G. Durisi, B. Guedj and M. Raginsky, "Generalization Bounds: Perspectives from Information Theory and PAC-Bayes," *Foundations and Trends in Machine Learning*, 18(1), 1–223, 2025. Publisher metadata, author version.
4. A. El Gamal, "Information Theory and Statistical Learning," proposed textbook chapter, arXiv preprint, 2026. Draft and publication status.

IT 08 • Networks, Information Inequalities and Learning

Analyze distributed sources and multiple users, connect advanced inequalities with learning, and run a finite distributed-decoding capstone.

Online edition: https://blog.shyandram.dev/en/learn/information-theory/it-08-networks-information-learning/

What you'll learn

- Describe rate regions for distributed sources and multiple-access channels.
- Explain why broadcast, relay and general networks need different arguments.
- Apply log-sum, Pinsker, entropy-power and conditional-information inequalities.
- Distinguish representation information from information about a training sample.
- Run a small distributed parity-code decoder with explicit finite errors.
- Connect the book's complete chapter map to current learning research without overstating limits.

The big picture

One sender and one receiver lead to a single capacity. Several sources or receivers lead to a region of compatible rates. Improving one rate may reduce another, and information can be useful jointly even when each component looks useless alone.

> **Explain like I'm five**
>
> Two friends each know one clue. Either clue alone may tell you nothing, but together they solve the puzzle. If both friends speak at once, they also have to share the same noisy room. We must track which clues are shared, who hears what, and how much each friend can say.

This final tutorial follows Cover and Thomas's Chapter 15 on networks and Chapter 17 on inequalities [1]. Earlier chapters covered the other fifteen book chapters through the reading map in IT 01. We end with a finite synthetic experiment that connects source dependence, coding, decoding and task information.

The course uses the published second edition, supplemented by current surveys and a clearly labeled 2026 proposed chapter.

Distributed source coding: correlation can save rate

Two encoders can separately describe correlated observations while a joint decoder uses their relationship. The encoders need not each send every bit independently.

Let independent pairs (X_t, Y_t) follow a fixed finite joint law. Encoder one observes only X^n, encoder two only Y^n, and a common decoder reconstructs both blocks. The Slepian–Wolf asymptotically almost-lossless region is

$$R_X \geq H(X \mid Y), \qquad R_Y \geq H(Y \mid X), \qquad R_X + R_Y \geq H(X, Y).$$

Here R_X, R_Y are bits per paired source sample sent by the separate encoders, and the entropies use the specified joint source. Boundary inequalities describe the closure; codes operate with suitable rate margins and vanishing block error [1].

The individual bounds assume the other source can help at decoding. The sum bound counts total joint uncertainty. Neither encoder is assumed to see the other's current block; allowing that observation would define a different problem.

For fair X and $Y = X \oplus E$ with independent Bernoulli noise probability e, both marginal entropies are one and joint entropy is $1 + h_2(e)$. Separately ignoring dependence would require two bits per pair asymptotically; joint decoding can exploit the smaller joint uncertainty.

One corner sends Y at its full marginal rate and compresses X toward $H(X \mid Y)$. The decoder already has the reconstructed Y, which acts as side information. This corner motivates the parity-syndrome experiment below.

Random binning supplies an existence argument: assign source blocks to bins and use the other decoded source to find the likely matching block in the signaled bin. The bin index tells less than a full source index, while the correlation resolves ambiguity.

This result is not exact reconstruction of every possible finite pair. Rare or ambiguous blocks can fail. Truly lossless finite protocols require an additional escape or other treatment if every allowed realization must be recovered.

Why random bins produce three rate constraints

Each encoder randomly assigns every possible source block to a bin and sends its bin index. The decoder searches for a jointly typical pair in the two indicated bins. A competing pair can be wrong in three ways: only X is wrong, only Y is wrong, or both are wrong.

Competing pair	Typical candidate count scale	Collision probability per candidate
Wrong X, correct Y	$2^{nH(X\|Y)}$	2^{-nR_X}
Correct X, wrong Y	$2^{nH(Y\|X)}$	2^{-nR_Y}
Both wrong	$2^{nH(X,Y)}$	$2^{-n(R_X+R_Y)}$

Here n is paired block length, R_X, R_Y the sent bin-index rates, and the entropy exponents use the fixed joint source. These scales omit the small typicality tolerances and finite prefactors. Independent bin assignments give the collision factors, while joint typicality limits how many candidates need consideration.

Multiply each count by its collision probability and use a union bound. All three terms vanish with suitable margins exactly when the two individual rates exceed their conditional entropies

and their sum exceeds joint entropy. The true pair fails typicality with a probability that also vanishes. Averaging over assignments then proves some fixed shared binning rule has small block error [1].

This proof explains both the correlation gain and its limit. Decoder side information reduces the number of candidate X blocks from the full source catalog to a conditional catalog. It does not eliminate the need to distinguish the remaining candidates. The sum constraint handles simultaneous ambiguity that neither individual bound can remove alone.

For fair bits with flip probability 0.1, conditional entropy is about 0.468996 and joint entropy 1.468996 bits per pair. The corner rates are approximately $(0.468996, 1)$ and $(1, 0.468996)$. A symmetric point has rate 0.734498 for each encoder; assigning 0.5 to each passes the individual bounds but fails the sum bound. These are limiting boundary values, so actual asymptotic constructions use margins rather than promising a short-block code at the boundary.

Multiple access: independent messages share one output

Two senders can both communicate through a common receiver. The receiver's observation limits their combined information as well as each individual rate.

For a finite memoryless channel $W(y \mid x_1, x_2)$ with independent messages, the region for a fixed product input law includes

$$R_1 \le I(X_1; Y \mid X_2), \qquad R_2 \le I(X_2; Y \mid X_1), \qquad R_1 + R_2 \le I(X_1, X_2; Y).$$

Here X_1, X_2 are sender inputs, Y the shared output, and R_1, R_2 bits per common channel use. Union over product input laws and time-sharing gives the ordinary two-user multiple-access capacity region [1].

Conditioning in an individual bound reflects the receiver being able to distinguish that user's message when the other input is known. The sum bound addresses joint decoding. Maximizing each individual expression separately does not remove the shared sum constraint.

Time-sharing introduces a common predetermined variable T and product inputs given T. The corresponding bounds condition each expression on T, and convexify operating points. The senders do not gain arbitrary message-dependent cooperation simply because they share a schedule.

For independent real Gaussian signals with powers P_1, P_2 and additive noise variance N,

$$\begin{aligned} R_1 &\le \tfrac{1}{2} \log_2(1 + P_1/N), \\ R_2 &\le \tfrac{1}{2} \log_2(1 + P_2/N), \\ R_1 + R_2 &\le \tfrac{1}{2} \log_2(1 + (P_1 + P_2)/N). \end{aligned}$$

Here one use contains both simultaneous real inputs; the signals have independent messages and the standard average-power constraints. The one-half factors retain the real-valued convention.

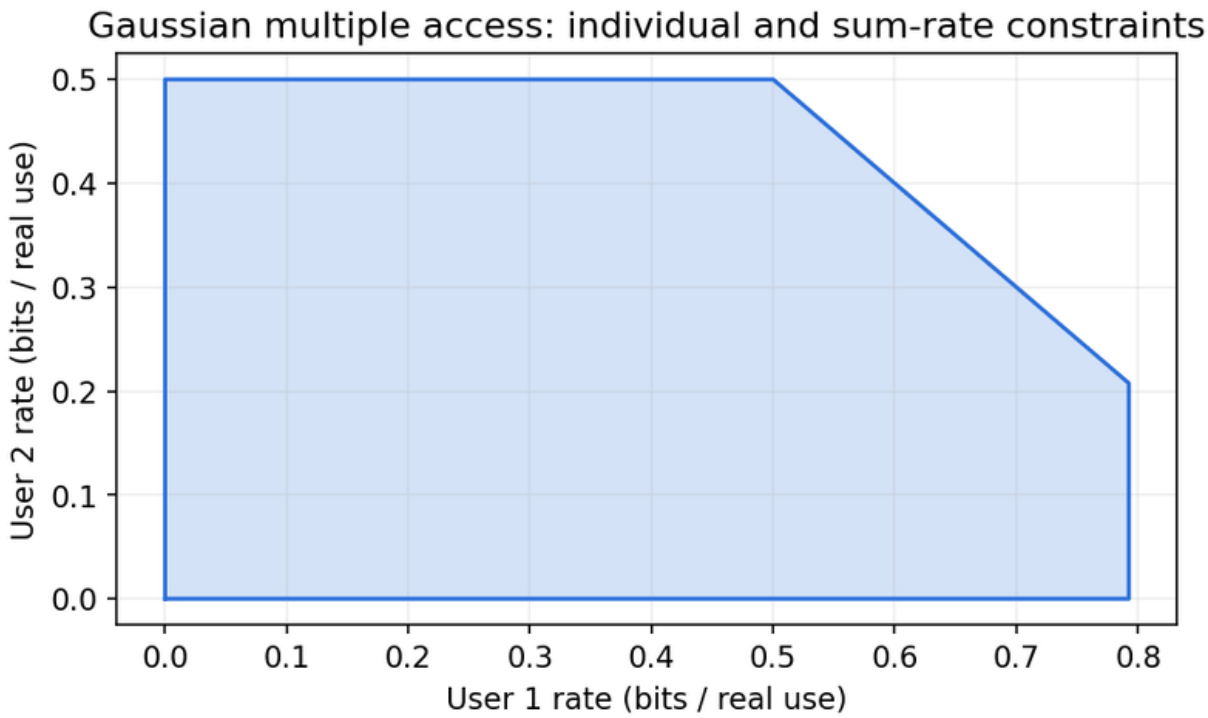


Figure 1 — P_1=2, P_2=1 and N=1. Individual maxima cannot both be reached simultaneously because of the sum-rate constraint.

Successive decoding can achieve corner points by first decoding one message while treating the other as noise, then removing the decoded contribution. The order changes which user receives the more favorable conditional rate. Joint decoding and time-sharing describe the broader region.

Work through the two Gaussian decoding orders

For the figure's powers $P_1 = 2$, $P_2 = 1$ and noise variance $N = 1$, user one's individual bound is about 0.792481 bits per use, user two's is 0.5, and the shared sum bound is one. The pair $(0.75, 0.4)$ satisfies both individual bounds but violates the sum. The pair $(0.6, 0.35)$ satisfies all three.

Decode user one first. Its decoder initially treats user two's independent Gaussian signal as additional noise of variance one, so its rate is limited to $\frac{1}{2}\log_2(1 + 2/(1 + 1)) = 0.5$. After successful decoding and subtraction, user two sees only the original noise and reaches 0.5. This order gives corner $(0.5, 0.5)$.

Decode user two first instead. Its initial rate is $\frac{1}{2}\log_2(1 + 1/(1 + 2)) \approx 0.207519$. After removing that signal, user one reaches $\frac{1}{2}\log_2(1 + 2/1) \approx 0.792481$. This gives corner $(0.792481, 0.207519)$. Both corners have total rate one, but decoding order allocates that total differently.

The subtraction argument assumes the first message was decoded correctly. A wrong first decision can propagate to the second, so a finite receiver must measure joint message error as well

as individual errors. The asymptotic corner calculation guides design without hiding that implementation issue.

Broadcast and relay channels change the information pattern

One sender with two receivers is not the reverse of two senders with one receiver. A relay that hears and retransmits creates another distinct pattern.

In a physically degraded broadcast channel, $X \to Y_1 \to Y_2$ makes receiver one stronger in the specified statistical sense. A superposition construction uses an auxiliary variable U with $U \to X \to Y_1 \to Y_2$:

$$R_2 \le I(U;Y_2), \qquad R_1 \le I(X;Y_1 \mid U).$$

Here R_2 is the weaker receiver's private-message rate, R_1 the stronger receiver's private-message rate, and U the layer both can resolve. Union over appropriate $p(u,x)$ gives the degraded-channel capacity region [1].

The Markov order is an assumption about receiver observations, not simply a ranking by sample accuracy. A general broadcast channel need not be degraded and does not inherit this exact region.

For a memoryless relay, let source input be X_s, relay input X_r, relay observation Y_r and destination observation Y_d. A cut-set upper bound is

$$C_{\text{relay}} \le \max_{p(x_s,x_r)} \min\{I(X_s;Y_r,Y_d \mid X_r), I(X_s,X_r;Y_d)\}.$$

Here the first cut separates the source from relay plus destination; the second separates source plus relay from the destination. The input joint law in the upper bound permits statistical cooperation as an outer relaxation.

A cut-set bound is not automatically achievable. Relays may decode a message, describe an observation, or use other causal strategies, each with its own conditions. General network capacity remains unresolved in many settings.

The lesson is to draw the actual information pattern before applying a formula: independent messages, correlated sources, side observations, causal relays and receiver demands are different assumptions. A scalar point-to-point capacity cannot summarize all of them.

Joint usefulness can be invisible to marginal information

Two individually uninformative features can jointly determine a target. Ranking only one feature at a time may discard both.

Let X_1, X_2 be independent fair bits and $Y = X_1 \oplus X_2$. Then

$$I(X_1;Y) = I(X_2;Y) = 0, \qquad I(X_1,X_2;Y) = 1, \qquad I(X_1;Y \mid X_2) = 1.$$

Here Y is the binary exclusive-or target. Each marginal input leaves the target fair; knowing both gives an exact answer. Conditioning on the other input reveals the first one's full relevance.

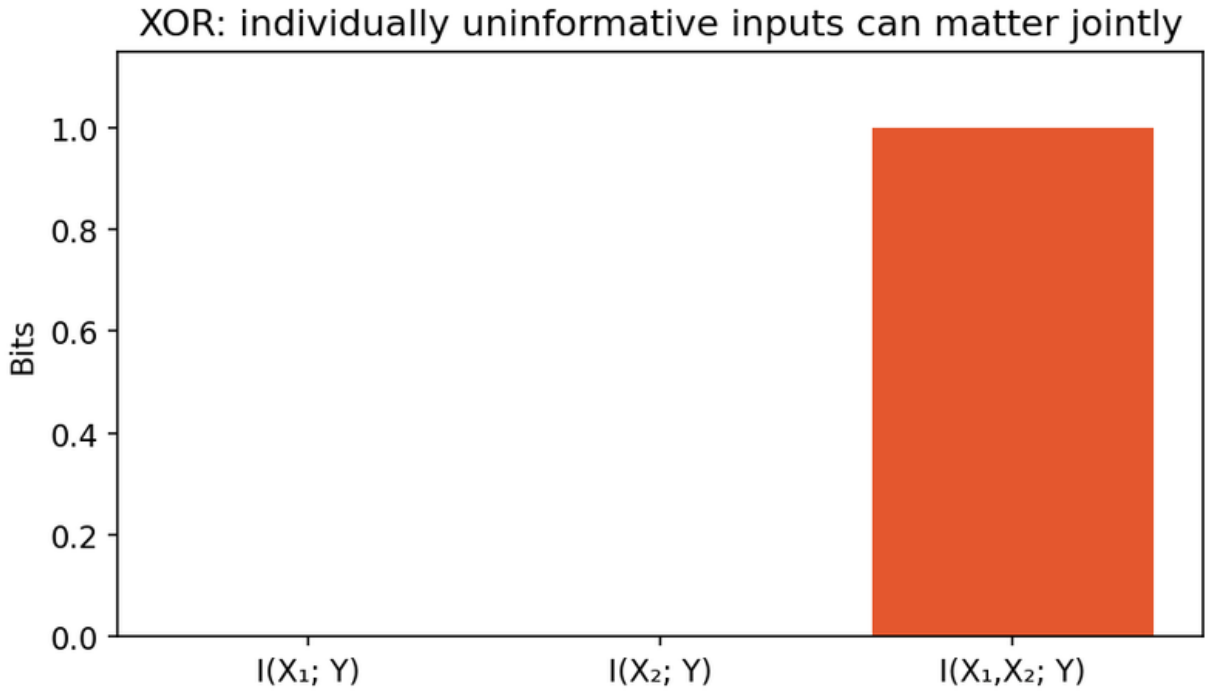


Figure 2 — Exact synthetic joint law. This illustrates joint usefulness without invoking a particular decomposition of redundancy and synergy.

The word synergy is useful informally for this joint-only information, but a general quantitative redundancy–synergy decomposition requires an additional definition. The three mutual-information values alone do not define a universal decomposition for arbitrary variables.

This matters for multimodal learning. A modality that seems weak by itself may be useful in context. Conversely, two strong marginal features may carry largely overlapping information. Analyze the actual joint law and receiver task rather than equating marginal relevance with unique contribution.

Computing a function is another distributed problem

Suppose separate encoders observe independent fair blocks X_1^n and X_2^n, and the decoder needs only their coordinatewise exclusive-or. The output has one bit of entropy per pair. It is tempting to conclude that one total bit per pair must suffice.

That conclusion ignores who sees the inputs. Give the decoder the second raw block as extra information. If it can recover the exclusive-or from the two transmitted indices, it can then recover the first block. The second encoder's index supplies no additional first-block information because the inputs are independent.

Consequently the first encoder must convey approximately one bit per coordinate for vanishing error in this example. By symmetry, so must the second. The distributed sum requirement can therefore be two bits even though the desired function has entropy one.

If one central encoder observed both inputs, it could instead transmit the exclusive-or itself at one bit per pair. The target is unchanged, but the observation pattern differs. This is a concrete reason to keep the network diagram beside a task-oriented rate claim.

The example also separates two meanings of “joint information.” The receiver may need a joint computation, while the senders lack joint observation. A learned system cannot assume a centrally available feature merely because that feature would be sufficient after it had been computed.

Real applications can have correlated inputs, interaction, common randomness or additional side information, each changing the problem. Do not transfer the independent noninteractive example’s rate conclusion to those settings without deriving the corresponding constraints.

Stronger inequalities extend the toolkit

Information inequalities often compare what remains after merging outcomes, changing a distribution, or adding independent noise. Their units and assumptions decide what they imply.

For nonnegative numbers a_i and positive b_i, the log-sum inequality states

$$\sum_i a_i \ln \frac{a_i}{b_i} \geq A \ln \frac{A}{B}, \qquad A = \sum_i a_i, \quad B = \sum_i b_i.$$

Here i indexes entries, A, B are their sums, and zero terms use the standard continuous convention. The all-zero A case has both sides zero. This inequality underlies divergence convexity and aggregation forms of data processing [1].

Applying it inside each cell of a partition shows that merging outcomes cannot increase relative entropy. Fine measurements can distinguish distributions in ways a coarse summary no longer reveals.

For finite probability vectors p, q, Pinsker’s inequality gives

$$\mathrm{TV}(p, q) = \frac{1}{2} \sum_i |p_i - q_i| \leq \sqrt{\frac{1}{2} D_{\mathrm{nats}}(p \| q)}.$$

Here total variation bounds differences in event probabilities, and divergence uses natural logarithms. If divergence is reported in bits, multiply it by $\ln 2$ inside the square root. A small divergence therefore controls event-probability differences, while large divergence can make the bound uninformative.

Discrete entropy is also submodular:

$$H(X_A) + H(X_B) \geq H(X_{A \cup B}) + H(X_{A \cap B}).$$

Here A, B are sets of variable indices, X_A is the corresponding collection, and the empty collection has entropy zero. The difference is a conditional mutual information between the nonoverlapping

parts given the intersection, so it is nonnegative.

This is a relation among entropies of subsets, not a statement that every task-relevance function is submodular. The XOR target is a useful reminder that feature information can increase when another feature is supplied.

Entropy power and added Gaussian noise

Differential entropy can be converted into a variance-like quantity. Adding independent continuous signals cannot make their combined entropy power smaller than the sum.

For a d-dimensional vector with finite natural-unit differential entropy, define entropy power

$$\mathcal{N}(X) = \frac{1}{2\pi e} \exp\left(\frac{2h_{\text{nats}}(X)}{d}\right).$$

Here d is dimension and h_{nats} uses natural logarithms. A scalar Gaussian has entropy power equal to its variance; an isotropic Gaussian vector has the common coordinate variance.

The entropy-power inequality for independent vectors of the same dimension states

$$\mathcal{N}(X + Y) \geq \mathcal{N}(X) + \mathcal{N}(Y).$$

Here X, Y must satisfy the relevant density and finiteness assumptions. Equality holds for independent Gaussian vectors with proportional covariance matrices in the standard nondegenerate setting [1].

The de Bruijn identity connects noise smoothing with Fisher information:

$$\frac{d}{dt} h_{\text{nats}}(X + \sqrt{t}\, G) = \frac{1}{2} \mathcal{J}_{\text{loc}}(X + \sqrt{t}\, G).$$

Here G is an independent standard Gaussian vector, $t > 0$ is noise variance per coordinate, and $\mathcal{J}_{\text{loc}}(V) = \mathbb{E}\|\nabla \ln f_V(V)\|^2$ is location Fisher information. Smoothness and integrability conditions are required.

This Fisher information is a density-gradient quantity related to location estimation, not an arbitrary parameter's Fisher information from the previous chapter. Keeping that definition explicit avoids confusing two similarly named objects.

Adding noise can increase observation entropy while reducing information about the original. There is no contradiction: entropy power concerns output spread; data processing concerns retained relationships. Modern noisy representation models must distinguish those two claims.

Representation information and sample information

A feature can preserve a target while limiting raw input detail. A trained model can also reveal how much it depends on a particular training sample. These are different information questions.

An information-bottleneck objective for $Y \to X \to Z$ is

$$\min_{p(z|x)} I(X;Z) - \beta I(Z;Y).$$

Here X is the observation, Y the chosen relevance target, Z a possibly stochastic representation, and $\beta \geq 0$ balances compression against target information. Goldfeld and Polyanskiy's published tutorial [2] develops this theory and its learning connections.

The objective selects a task-relative statistical representation. It does not establish that every neural network spontaneously follows the same optimization, nor that one representation preserves every future target.

A bottleneck that removes nuisance without losing the label

A finite example makes the representation objective operational. Let label Y and nuisance bit N be independent fair bits, and let observation $X = (Y, N)$ contain both. Compare a full representation $Z = X$, a label-only representation $Z = Y$, and a constant representation.

Representation	$I(X;Z)$	$I(Z;Y)$	Objective $I(X;Z) - \beta I(Z;Y)$
Both bits	2	1	$2 - \beta$
Label only	1	1	$1 - \beta$
Constant	0	0	0

All information values are bits. The relevance weight β is dimensionless. The label-only representation retains everything needed for this label and spends one less bit than the full representation. For $\beta < 1$, a constant wins; for $\beta > 1$, retaining the label wins among these choices. At $\beta = 1$, label-only and constant tie.

This conclusion is not limited to those three examples. The information chain rule gives

$$I(X;Z) - \beta I(Z;Y) = (1-\beta) I(Y;Z) + I(N;Z \mid Y).$$

Here $X = (Y, N)$, N is the independent nuisance bit and Z any representation generated from X. The conditional term is nonnegative. For $\beta \leq 1$, zero is therefore a global lower bound, attained by a constant. For $\beta > 1$, using $I(Y;Z) \leq H(Y) = 1$ gives lower bound $1 - \beta$, attained by $Z = Y$. The task-preserving optimum follows from the chain rule and an explicit source, rather than an informal claim that compression always improves learning.

Now change the future target to $Y' = Y \oplus N$. The old label-only representation has zero information about this new fair target, while the full observation determines it. Irreversible task-specific compression limits future task transfer even when it is optimal for the original relevance criterion.

The notation also separates the two information questions in this section. These values concern one observation and its representation. The learning bound below instead concerns a model W and the whole training sample S. Reducing nuisance information in Z does not directly establish a smaller $I(W;S)$ or a numerical generalization guarantee.

For learning, let $S = (Z_1, \ldots, Z_n)$ be an independent training sample and W the learned model. Define population and empirical losses

$$L(W) = \mathbb{E}_{Z\sim P}[\ell(W,Z)], \qquad \hat{L}_S(W) = \frac{1}{n}\sum_{i=1}^{n} \ell(W,Z_i).$$

Here P is the population law, a fresh Z is independent of training, and ℓ is the chosen loss. Now the relevant information term can be $I(W;S)$, rather than feature information $I(X;Z)$.

If every fixed model's loss is σ-sub-Gaussian under P, a standard expectation bound has the form

$$\left|\mathbb{E}[L(W) - \hat{L}_S(W)]\right| \le \sqrt{\frac{2\sigma^2 I_{\text{nats}}(W;S)}{n}}.$$

Here expectation averages the sample and any learner randomness, σ is the common concentration parameter, and information uses natural logarithms. Bounded loss in $[0,1]$ permits $\sigma = 1/2$ [3].

This is an expectation bound, not automatically a high-probability certificate for one trained network. A deterministic continuous learner can have infinite sample information, making it vacuous. Conditional-information frameworks and other refinements address different randomization and sampling conventions [3].

Code: an exact finite learning bound

This complete example learns only a constant binary classifier by majority label. Because its output is a deterministic bit, $I(W;S) = H(W)$; all expectations can be summed over binomial counts.

```
import numpy as np
from scipy.stats import binom

p = 0.65
for n in [5, 20, 100]:
    counts = np.arange(n+1)
    mass = binom.pmf(counts, n, p)
    prediction = counts >= n/2
    training_loss = np.where(prediction, (n-counts)/n, counts/n)
    population_loss = np.where(prediction, 1-p, p)
    gap = mass @ (population_loss-training_loss)
    probability_one = mass[prediction].sum()
    probabilities = np.array([probability_one, 1-probability_one])
    positive = probabilities > 0
    information_nats = -np.sum(probabilities[positive]*np.log(probabilities[positive]))
    bound = np.sqrt(information_nats/(2*n))
    assert abs(gap) <= bound + 1e-12
    print("Length / expected gap / information bound:", n, gap, bound)
```

The calculation includes the selection dependence between model and sample. Treating the chosen classifier as independent of its training labels would erase the very generalization gap being analyzed.

Capstone: side-information decoding with parity bins

The encoder sends a short parity label for its binary block. The decoder combines that label with a correlated block to select the most likely original.

For binary block $x \in \{0,1\}^n$ and public binary matrix H with r independent rows, transmit syndrome

$$s = Hx \pmod 2.$$

Here H has shape r by n, s contains r parity bits, and multiplication uses binary-field addition and multiplication. Each syndrome identifies a bin of candidate blocks.

The decoder has $y = x \oplus e$, with independent flip probability below one half, and chooses a syndrome-compatible candidate minimizing Hamming distance to y. For this specified law, that is maximum conditional likelihood.

The encoder never sees y. The matrix and block length are shared; the decoder obtains y as side information. The payload rate is r/n, excluding any protocol for delivering or reconstructing that side information.

Thus the experiment corresponds to a source-coding corner, rather than compressing both observations for free. If the second block must also be transmitted, its description length belongs in the total rate. Shared side information is an assumption with a source, not an unexplained decoder advantage.

Code: exhaustive finite decoding and full-block error

The complete CPU example uses ten-bit blocks so exhaustive search is practical. It checks matrix rank over the binary field rather than using real-valued numerical rank.

```
import numpy as np

rng = np.random.default_rng(43)
n, trials, flip = 10, 3000, 0.1
patterns = ((np.arange(2**n)[:, None] >> np.arange(n)) & 1).astype(np.uint8)
source_indices = rng.integers(2**n, size=trials)
source = patterns[source_indices]
side = source ^ rng.binomial(1, flip, size=source.shape).astype(np.uint8)

def binary_rank(matrix):
    work = matrix.copy()
    rank = 0
    for column in range(work.shape[1]):
        pivots = np.flatnonzero(work[rank:, column])
        if len(pivots) == 0:
            continue
        pivot = rank + pivots[0]
        work[[rank, pivot]] = work[[pivot, rank]]
        for row in range(work.shape[0]):
```

```
            if row != rank and work[row, column]:
                work[row] ^= work[rank]
        rank += 1
        if rank == work.shape[0]:
            break
    return rank

for parity_bits in [4, 6, 8]:
    while True:
        H = rng.integers(2, size=(parity_bits, n), dtype=np.uint8)
        if binary_rank(H) == parity_bits:
            break
    weights = 2**np.arange(parity_bits)
    syndromes = ((patterns @ H.T) % 2) @ weights
    bins = [np.flatnonzero(syndromes == s) for s in range(2**parity_bits)]
    assert all(len(group) == 2**(n-parity_bits) for group in bins)
    decoded = np.empty(trials, dtype=int)
    for t, (index, observation) in enumerate(zip(source_indices, side)):
        candidates = bins[syndromes[index]]
        distances = np.count_nonzero(patterns[candidates] != observation, axis=1)
        decoded[t] = candidates[np.argmin(distances)]
    block_error = np.mean(decoded != source_indices)
    print("Parity bits / payload rate / full-block error:",
          parity_bits, parity_bits/n, block_error)
conditional_entropy = -flip*np.log2(flip)-(1-flip)*np.log2(1-flip)
print("Asymptotic side-information reference:", conditional_entropy)
```

This is a finite demonstration, not a capacity-achieving construction. Different random matrices produce different errors; increasing parity count need not improve every unrelated sampled matrix monotonically. The reference entropy is a long-block limit, not an exact ten-bit prediction.

Tie handling is deterministic through candidate order. Noise can move the side block closer to an incorrect syndrome-compatible source. A full round-trip assertion for every noisy trial would therefore be the wrong test: the protocol intentionally reports finite message errors.

For a truly lossless file protocol, provide an escape mechanism or interactive correction with explicit transmitted cost. Here the learning goal is to see how side information narrows bins and how finite ambiguity remains.

Modern view

The published 2025 monograph by Hellström and colleagues [3] maps information-theoretic and PAC-Bayesian generalization through expectation, probability and conditional-information formulations. That taxonomy helps distinguish a bound on average selection dependence from a guarantee for one deployed predictor.

Nguyen and colleagues' 2025 semantic-communication survey [4] distinguishes receiver-intention modeling, generative decoding and learned joint coding. Its network perspective makes receiver knowledge and task goals central; those must be included in the mathematical

information pattern.

El Gamal's proposed 2026 chapter [5] extends the textbook conversation toward statistical learning. Its draft status remains explicit. Modern inference and generative models can use the same divergence machinery while posing additional computational and evaluation problems.

Open questions include general multiuser regions, short-block distributed codes, realistic dependence and channel shift, task criteria for rare content, and useful information bounds for high-dimensional learned systems. A new model architecture does not solve these questions merely by producing plausible samples.

Our course-level inference is to ask five questions before interpreting an information claim: which random variables are involved; who observes each one; what source and channel laws are assumed; which reconstruction or decision is required; and how rate, error and resource cost are measured.

An information audit can turn those questions into a compact experiment record:

Choice	What to record
Source	Population, grouping and dependence assumptions
Observation	Variables available separately at each encoder and receiver
Description	Alphabet, actual bitstream, shared parameters and side streams
Channel	Conditional law, memory, power and use convention
Goal	Exact message, lossy reconstruction or specified task
Evaluation	Fixed test set, error definition, rate and runtime

The record is useful even when no tight theorem is known. It prevents a number from silently changing meaning between training, mathematical analysis and deployment. It also makes negative results informative: a poor rate may come from a probability model, a reconstruction constraint or a finite decoder, rather than from the same underlying limitation.

A reading and project path after the course

Revisit the full book in three passes. First consolidate Chapters 1–5 and 7–10 with the entropy, coding, channel and distortion examples. Then connect Chapters 6 and 11–14 to prediction, evidence, maximum entropy, universal codes and description complexity. Finally study Chapters 15–17 with the network assumptions and inequalities kept beside every theorem.

For a small project, choose one explicit joint law or independently collected dataset and one receiver objective. Implement a baseline encoder–decoder, report actual rate and decision error, then change only one component at a time. A probability model, a quantizer and a channel code answer different experimental questions.

The autoregressive vision chapter connects sequential probability factorization with modern models. The representation chapter develops task features, and the language-model chapter connects conditional prediction with token sequences.

These connections are strongest when the information definitions stay concrete. A tokenization choice changes the alphabet and unit; a learned latent changes the observation; a generated reconstruction changes the distortion question. State those choices before comparing numerical information values.

Key takeaways

- Networks produce compatible rate regions rather than one universal scalar rate.
- Distributed decoding exploits correlation without granting encoders unobserved inputs.
- Multiple-access individual rates remain subject to a shared sum constraint.
- Broadcast and relay formulas depend on their specific information pattern.
- Joint task information can be invisible to individual feature information.
- Advanced inequalities require explicit units, supports and independence assumptions.
- Representation information and sample-dependent generalization are distinct objects.
- Finite decoding, actual rate and honest error reporting complete the theoretical picture.

Exercises

1. For fair X and $Y = X \oplus E$ with flip probability 0.2, calculate the Slepian–Wolf corner rates and joint entropy. Explain what each encoder observes.

   Hint

   Use $h_2(0.2)$ for conditional entropy.

2. Compute the two Gaussian multiple-access corner points for the figure parameters. Verify the sum constraint and describe the successive-decoding order.

   Hint

   One individual rate reaches its maximum and the other fills the sum bound.

3. Explain why an arbitrary broadcast channel cannot inherit the physically degraded region. Identify the missing conditional relation.

   Hint

   Check $X \to Y_1 \to Y_2$.

4. Verify Pinsker's bound for probability vectors $(0.6, 0.4)$ and $(0.5, 0.5)$, using both natural and base-two divergence consistently.

   Hint

   Convert bits to nats with $\ln 2$.

5. Change parity-code block length to 12 and compare at several parity counts. Keep exhaustive-search cost visible and report block errors rather than only bit errors.

Hint

The candidate table has 2^n rows.

6. For the XOR target, compare keeping only X_1 with keeping the pair. Then explain why a deterministic continuous representation's raw-input information can be infinite while its discrete task information stays bounded.

Hint

Separate target entropy from exact continuous relationships.

References

1. T. M. Cover and J. A. Thomas, *Elements of Information Theory*, 2nd ed., Wiley, 2006, Chapters 15 and 17. Publisher book and contents.
2. Z. Goldfeld and Y. Polyanskiy, "The Information Bottleneck Problem and Its Applications in Machine Learning," *IEEE Journal on Selected Areas in Information Theory*, 1(1), 2020, DOI: 10.1109/JSAIT.2020.2991561. Institutional final-manuscript record.
3. F. Hellström, G. Durisi, B. Guedj and M. Raginsky, "Generalization Bounds: Perspectives from Information Theory and PAC-Bayes," *Foundations and Trends in Machine Learning*, 18(1), 1–223, 2025. Publisher metadata, author version.
4. L. X. Nguyen, A. D. Raha, P. S. Aung, D. Niyato, Z. Han and C. S. Hong, "A Contemporary Survey on Semantic Communications: Theory of Mind, Generative AI, and Deep Joint Source-Channel Coding," *IEEE Communications Surveys & Tutorials*, 2025. Published-paper metadata and author version.
5. A. El Gamal, "Information Theory and Statistical Learning," proposed textbook chapter, arXiv preprint, 2026. Draft and publication status.

Part IX

Blog Essays

Introducing Pixels → Perception

A bilingual place to study image processing, machine learning, and the ideas behind them through explanations, code, and interactive experiments.

Online edition: https://blog.shyandram.dev/en/blog/introducing-pixels-perception/

An image begins as an array of numbers. Understanding what those numbers represent leads to questions about signals, patterns, learning, and uncertainty. How can a small operation change the appearance of an image? How does a model learn to recognize something? What should we check before trusting its output?

I'm introducing Pixels → Perception, a tutorial website in English and Traditional Chinese that brings these questions into a connected learning path. It covers image processing, pattern recognition, machine learning, deep learning, computer vision, natural language processing, optimization, and information theory. The aim is to help readers move between an intuitive explanation, the mathematics behind it, and an experiment they can inspect for themselves.

There is a particular gap this site tries to address: the distance between recognizing an equation and understanding what happens when you use it. A definition may be clear on the page, yet its assumptions can become much easier to see when a parameter changes, a plot moves, or an example fails. The tutorials are organized to make those connections explicit.

The learning section has eight tracks:

- **Foundations** introduces the site, suggested learning paths, and basic ways to represent and work with images.
- **Image Processing** develops the tools for treating images as signals, from intensity and spatial operations to restoration, compression, and segmentation.
- **Natural Language Processing** explores how to represent language and build systems that work with text, connecting linguistic ideas with learning methods.
- **Pattern Recognition** focuses on features, statistical decisions, classification, and the uncertainty involved in recognizing patterns.
- **Machine Learning** covers mathematical foundations, neural networks, learned representations, generative models, and evaluation.
- **Deep Learning for Computer Vision** follows those ideas into tasks such as image classification, object detection, restoration, enhancement, and tracking.
- **Optimization** examines how to formulate a problem, express its constraints, and choose a way to solve it.
- **Information Theory** studies uncertainty, compression, and communication, providing another perspective on ideas that appear throughout the other tracks.

These subjects share a great deal of common ground. The mathematical tools used to reason about a visual model also help explain a language model. A question about image quality can lead back to the assumptions of an imaging process. A training problem can become clearer when

you examine the objective being optimized. Keeping the tracks together makes it easier to follow those connections without losing the structure of an individual subject.

The tutorials follow a consistent rhythm. They begin with learning goals and a plain-language picture of the topic, then develop definitions, equations, and examples. Code and figures give readers something concrete to examine. Later sections connect the fundamentals with research literature, discuss limitations, and end with takeaways and exercises. Prerequisite links help identify where to go when an explanation assumes something unfamiliar.

For example, the interactive playground includes an image-filtering experiment. You can edit a small grid of weights and see the output change. That creates a useful learning loop: make a prediction, adjust the weights, inspect the result, and return to the explanation. Other experiments explore ideas from the broader curriculum. The controls run in the browser, so you can start experimenting without setting up a programming environment.

The code examples serve a different purpose: they let you inspect the operations themselves. Readers who want to go further can change an example, compare outputs, or work through an exercise. Some tutorials also link to a runnable notebook. The small examples are intended to explain mechanisms; they do not reproduce the scale or performance of a full research system.

Both language versions are intended to support the same learning experience. The Traditional Chinese pages use terminology familiar to readers in Taiwan, and the language switcher connects corresponding pages. This can be useful when you understand an idea more comfortably in one language but need to recognize its terminology in the other.

The site also has a blog alongside the structured tutorials. The curriculum provides an ordered route through a subject; the blog provides room for introductions, notes, and discussions that sit outside that sequence. Search is available when you want to return to a specific idea without following a track from the beginning.

I also want to be clear about how the site was made. Much of its draft content and code was created with AI assistance. Human direction shapes the topics and course structure, and human review remains essential. **The content is still under review.** An explanation can be readable and still contain a mistake, and a working example can leave an important assumption unstated. Readers should check the cited sources when a detail matters and treat small demonstrations as a starting point for investigation.

If you are new to the subjects, start with the getting-started guide and Images as Arrays. If you already have a particular question, browse the tracks or open the playground and follow an experiment back to its related lesson. You can read in sequence, revisit a mathematical prerequisite, or spend a few minutes exploring one idea.

The invitation is simple: choose a question, make a prediction, and try an example. I hope Pixels → Perception helps turn something that looks abstract on a page into something you can explain, test, and use.

Visit Pixels → Perception in English or 繁體中文.

Welcome to Pixels → Perception

What this site covers, how the tutorials are organized, and how to run the code.

Online edition: https://blog.shyandram.dev/en/blog/welcome/

This site is a computer vision course you can read in order or dip into. It starts with images as arrays of numbers and builds up to modern deep networks.

How it is organized

The **Learn** section has eight tracks: Foundations, Image Processing, Natural Language Processing, Pattern Recognition, Machine Learning, Deep Learning for Computer Vision, Optimization and Information Theory. Each tutorial lists its prerequisites, so you can always see what to read first.

Running the code

Some tutorials provide a matching Jupyter notebook; others include standalone code blocks. When a notebook is available, click **Open in Colab** at the top of the page to run it in your browser.

The blog, where this post lives, is for everything outside the curriculum: paper notes, tool reviews and longer essays.